

# RX32G410 参考手册

文档编号：RM00006

基于 Arm®Cortex®-M4 内核的 32 位专业电机微控制器

版本：V0.7

## 目录

|                                                    |    |
|----------------------------------------------------|----|
| 1 简介 .....                                         | 40 |
| 2 文档约定 .....                                       | 41 |
| 2.1 寄存器缩写列表 .....                                  | 41 |
| 2.2 词汇表 .....                                      | 41 |
| 3 系统和存储器概述 .....                                   | 42 |
| 3.1 系统架构 .....                                     | 42 |
| 3.2 存储器结构 .....                                    | 42 |
| 3.2.1 介绍 .....                                     | 42 |
| 3.2.2 存储器映像和寄存器边界地址 .....                          | 43 |
| 3.3 内置 SRAM .....                                  | 45 |
| 3.3.1 奇偶校验 .....                                   | 46 |
| 3.3.2 CCM SRAM 读保护 .....                           | 46 |
| 3.3.3 CCM SRAM 擦除 .....                            | 46 |
| 3.4 Flash 概述 .....                                 | 46 |
| 3.5 Boot 配置 .....                                  | 46 |
| 4 内置 Flash 存储器 (FLASH) .....                       | 47 |
| 4.1 介绍 .....                                       | 47 |
| 4.2 Flash 主要特征 .....                               | 47 |
| 4.3 Flash 功能描述 .....                               | 47 |
| 4.3.1 Flash 结构 .....                               | 47 |
| 4.3.2 读取等待周期 .....                                 | 48 |
| 4.3.3 指令预取 .....                                   | 49 |
| 4.3.4 Flash 擦除和编程 .....                            | 49 |
| 4.3.5 解锁 Flash .....                               | 49 |
| 4.3.6 Flash 擦除流程 .....                             | 49 |
| 4.4 Flash 寄存器描述 .....                              | 50 |
| 4.4.1 Flash 访问控制寄存器 (FLASH_ACR) .....              | 50 |
| 4.4.2 Flash 键寄存器 (FLASH_KEYR) .....                | 51 |
| 4.4.3 Flash OPTKEY 寄存器 (FLASH_OPTKEYR) .....       | 51 |
| 4.4.4 Flash 状态寄存器 (FLASH_SR) .....                 | 51 |
| 4.4.5 Flash 控制寄存器 (FLASH_CR) .....                 | 53 |
| 4.4.6 Flash ECC 寄存器 (FLASH_ECCR) .....             | 54 |
| 4.4.7 Flash 选项字节寄存器 (FLASH_OPTR) .....             | 56 |
| 4.4.8 Flash BANK1 写保护 A 地址寄存器 (FLASH_WRP1AR) ..... | 57 |
| 4.4.9 Flash BANK1 写保护 B 地址寄存器 (FLASH_WRP1BR) ..... | 57 |
| 5 CRC 计算单元 .....                                   | 59 |
| 5.1 CRC 简介 .....                                   | 59 |
| 5.2 CRC 主要特性 .....                                 | 59 |
| 5.3 CRC 功能说明 .....                                 | 59 |
| 5.4 CRC 寄存器 .....                                  | 60 |
| 5.4.1 数据寄存器 (CRC_DR) .....                         | 60 |

|                               |    |
|-------------------------------|----|
| 5.4.2 独立数据寄存器 (CRC_IDR)       | 60 |
| 5.4.3 控制寄存器 (CRC_CR)          | 60 |
| 6 电源控制 (PWR)                  | 61 |
| 6.1 概述                        | 61 |
| 6.2 电源管理器                     | 61 |
| 6.2.1 上电复位 (POR) 和掉电复位 (PDR)  | 61 |
| 6.2.1 可编程电压检测器 (PVD)          | 62 |
| 6.3 低功耗模式                     | 63 |
| 6.3.1 运行模式                    | 64 |
| 6.3.2 低功耗模式                   | 64 |
| 6.3.1 睡眠模式                    | 65 |
| 6.3.2 停止 0 模式                 | 66 |
| 6.3.1 停止 1 模式                 | 67 |
| 6.3.2 待机模式                    | 68 |
| 6.4 低功耗模式下的自动唤醒 (AWU)         | 69 |
| 6.5 PWR 寄存器描述                 | 70 |
| 6.5.1 电源控制寄存器 (PWR_CR1)       | 70 |
| 6.5.2 电源控制寄存器 (PWR_CR2)       | 70 |
| 6.5.3 电源控制寄存器 (PWR_CR3)       | 71 |
| 6.5.4 电源控制寄存器 (PWR_CR4)       | 72 |
| 6.5.5 电源状态寄存器 1 (PWR_SR1)     | 73 |
| 6.5.6 电源状态寄存器 2 (PWR_SR2)     | 74 |
| 6.5.7 电源状态清除寄存器 (PWR_SCR)     | 74 |
| 7 备份寄存器 (BKP)                 | 75 |
| 7.1 BKP 简介                    | 75 |
| 7.2 BKP 特性                    | 75 |
| 7.3 BKP 功能描述                  | 75 |
| 7.3.1 侵入检测                    | 75 |
| 7.3.2 RTC 校准                  | 75 |
| 7.4 BKP 寄存器描述                 | 75 |
| 7.4.1 RTC 时钟校准寄存器 (BKP_RTCCR) | 75 |
| 7.4.2 备份控制寄存器 (BKP_CR)        | 76 |
| 7.4.3 备份控制/状态寄存器 (BKP_CSR)    | 77 |
| 8 复位和时钟控制 (RCC)               | 78 |
| 8.1 复位                        | 78 |
| 8.1.1 系统复位                    | 78 |
| 8.1.2 电源复位                    | 78 |
| 8.1.3 备份域复位                   | 79 |
| 8.2 时钟                        | 79 |
| 8.2.1 HSE 时钟                  | 80 |
| 8.2.2 HSI 时钟                  | 81 |
| 8.2.3 PLL                     | 82 |
| 8.2.4 LSI 时钟                  | 82 |

|                                               |     |
|-----------------------------------------------|-----|
| 8.2.5 系统时钟 (SYSCLK) 选择                        | 82  |
| 8.2.6 时钟安全系统 (CSS)                            | 82  |
| 8.2.7 RTC 时钟                                  | 82  |
| 8.2.8 看门狗时钟                                   | 83  |
| 8.2.9 时钟输出                                    | 83  |
| 8.3 RCC 寄存器描述                                 | 84  |
| 8.3.1 时钟控制寄存器 (RCC_CR)                        | 84  |
| 8.3.2 内部时钟源寄存器 (RCC_ICSR)                     | 85  |
| 8.3.3 时钟配置寄存器 (RCC_CFGR)                      | 86  |
| 8.3.4 PLL 配置寄存器 (RCC_PLLCFGR)                 | 88  |
| 8.3.5 定时器时钟配置寄存器 (RCC_CFGR1)                  | 90  |
| 8.3.6 时钟中断使能寄存器 (RCC_CIER)                    | 90  |
| 8.3.7 时钟中断标志寄存器 (RCC_CIFR)                    | 91  |
| 8.3.8 时钟中断清除寄存器 (RCC_CICR)                    | 92  |
| 8.3.9 AHB1 外设复位寄存器 (RCC_AHB1RSTR)             | 93  |
| 8.3.10 AHB2 外设复位寄存器 (RCC_AHB2RSTR)            | 94  |
| 8.3.11 APB1 外设复位寄存器 1 (RCC_APB1RSTR1)         | 96  |
| 8.3.12 APB2 外设复位寄存器 (RCC_APB2RSTR)            | 98  |
| 8.3.13 AHB1 外设时钟使能寄存器 (RCC_AHB1ENR)           | 99  |
| 8.3.14 AHB2 外设时钟使能寄存器 (RCC_AHB2ENR)           | 100 |
| 8.3.15 APB1 外设时钟使能寄存器 1 (RCC_APB1ENR1)        | 102 |
| 8.3.16 APB2 外设时钟使能寄存器 (RCC_APB2ENR)           | 104 |
| 8.3.17 独立外设时钟配置寄存器 (RCC_CCIPR)                | 105 |
| 8.3.18 RTC 域控制寄存器 (RCC_BDCR)                  | 105 |
| 8.3.19 控制/状态寄存器 (RCC_CSR)                     | 107 |
| 9 通用和复用功能 I/O (GPIO)                          | 109 |
| 9.1 简介                                        | 109 |
| 9.2 GPIO 主要特征                                 | 109 |
| 9.3 GPIO 功能描述                                 | 109 |
| 9.3.1 通用 IO (GPIO)                            | 111 |
| 9.3.2 IO 引脚复用功能                               | 111 |
| 9.3.3 I/O 端口控制寄存器                             | 112 |
| 9.3.4 I/O 端口数据寄存器                             | 112 |
| 9.3.5 I/O 数据位处理                               | 112 |
| 9.3.6 I/O 锁定机制                                | 112 |
| 9.3.7 I/O 复用功能输入/输出                           | 112 |
| 9.3.8 外部中断/唤醒线                                | 113 |
| 9.3.9 输入模式配置                                  | 113 |
| 9.3.10 输出模式配置                                 | 113 |
| 9.3.11 复用功能配置                                 | 114 |
| 9.3.12 模拟功能配置                                 | 115 |
| 9.4 GPIO 寄存器描述                                | 116 |
| 9.4.1 GPIO 端口模式寄存器 (GPIOx_MODER) (x = A to F) | 116 |



|                                                         |     |
|---------------------------------------------------------|-----|
| 9.4.2 GPIO 端口输出类型寄存器 (GPIOx_OTYPE) (x = A to F) .....   | 116 |
| 9.4.3 GPIO 端口输出速度寄存器 (GPIOx_OSPEEDR) (x = A to F) ..... | 117 |
| 9.4.4 GPIO 端口上/下拉寄存器 (GPIOx_PUPDR) (x = A to F) .....   | 117 |
| 9.4.5 GPIO 端口输入数据寄存器 (GPIOx_IDR) (x = A to F) .....     | 118 |
| 9.4.6 GPIO 端口输出数据寄存器 (GPIOx_ODR) (x = A to F) .....     | 118 |
| 9.4.7 GPIO 端口位置位/清零寄存器 (GPIOx_BSRR) (x = A to F) .....  | 118 |
| 9.4.8 GPIO 端口配置锁寄存器 (GPIOx_LCKR) (x = A to F) .....     | 119 |
| 9.4.9 GPIO 复用功能低位寄存器 (GPIOx_AFR_L) (x = A to F) .....   | 120 |
| 9.4.10 GPIO 复用功能高位寄存器 (GPIOx_AFR_H) (x = A to F) .....  | 120 |
| 10 系统配置控制器 (SYSCFG) .....                               | 122 |
| 10.1 SYSCFG 主要特征 .....                                  | 122 |
| 10.2 SYSCFG 寄存器 .....                                   | 122 |
| 10.2.1 外部中断配置寄存器 1 (SYSCFG_EXTICR1) .....               | 122 |
| 10.2.2 外部中断配置寄存器 2 (SYSCFG_EXTICR2) .....               | 122 |
| 10.2.3 外部中断配置寄存器 3 (SYSCFG_EXTICR3) .....               | 123 |
| 10.2.4 外部中断配置寄存器 4 (SYSCFG_EXTICR4) .....               | 123 |
| 10.2.5 SYSCFG CCM SRAM 控制和状态寄存器 (SYSCFG_SCSR) .....     | 124 |
| 10.2.6 SYSCFG 配置寄存器 2 (SYSCFG_CFGR2) .....              | 124 |
| 10.2.7 SYSCFG CCM SRAM 键寄存器 (SYSCFG_SKR) .....          | 125 |
| 11 嵌套向量中断控制器 (NVIC) .....                               | 126 |
| 11.1 NVIC 主要特征 .....                                    | 126 |
| 11.2 中断向量表 .....                                        | 126 |
| 12 外部中断/事件控制器 (EXTI) .....                              | 129 |
| 12.1 介绍 .....                                           | 129 |
| 12.2 EXTI 的主要特性 .....                                   | 129 |
| 12.3 EXTI 功能介绍 .....                                    | 129 |
| 12.3.1 EXTI 框图 .....                                    | 130 |
| 12.3.2 唤醒事件管理 .....                                     | 130 |
| 12.3.3 外设异步中断 .....                                     | 130 |
| 12.3.4 硬件中断选择 .....                                     | 131 |
| 12.3.5 硬件事件选择 .....                                     | 131 |
| 12.3.6 软件中断/事件选择 .....                                  | 131 |
| 12.4 EXTI 中断/事件线映射 .....                                | 131 |
| 12.5 EXTI 寄存器描述 .....                                   | 133 |
| 12.5.1 EXTI 中断屏蔽寄存器 (EXTI_IMR) .....                    | 133 |
| 12.5.2 EXTI 事件屏蔽寄存器 (EXTI_EMR) .....                    | 133 |
| 12.5.3 EXTI 上升沿触发选择寄存器 (EXTI_RTSR) .....                | 134 |
| 12.5.4 EXTI 下降沿触发选择寄存器 (EXTI_FTSR) .....                | 134 |
| 12.5.5 EXTI 软件中断事件寄存器 (EXTI_SWIER) .....                | 135 |
| 12.5.6 EXTI 挂起寄存器 (EXTI_PR) .....                       | 136 |
| 13 可配置逻辑单元 (CLC) .....                                  | 137 |
| 13.1 简介 .....                                           | 137 |
| 13.2 CLC 设置 .....                                       | 139 |

|                                                 |     |
|-------------------------------------------------|-----|
| 13.3 输入源.....                                   | 140 |
| 13.3.1 多路复用器.....                               | 140 |
| 13.3.2 逻辑输入门.....                               | 140 |
| 13.3.3 逻辑功能.....                                | 141 |
| 13.3.4 软件输入 .....                               | 141 |
| 13.4 输出 .....                                   | 141 |
| 13.5 应用逻辑 .....                                 | 141 |
| 13.5.1 组合逻辑 .....                               | 141 |
| 13.5.2 状态逻辑.....                                | 141 |
| 13.6 CLC 中断 .....                               | 142 |
| 13.7 睡眠模式操作 .....                               | 142 |
| 13.8 空闲模式下的操作 .....                             | 142 |
| 13.9 复位 .....                                   | 142 |
| 13.10 CLC 寄存器描述 .....                           | 142 |
| 13.10.1 可配置的逻辑单元 x 控制寄存器 (CLC_CON) .....        | 142 |
| 13.10.2 可配置的逻辑单元 x 输入 MUX 选择寄存器 (CLC_SEL) ..... | 144 |
| 13.10.3 可配置的逻辑单元 x 源使能寄存器 (CLC_GLS) .....       | 144 |
| 14 CORDIC 协同处理器 (CORDIC) .....                  | 146 |
| 14.1 简介 .....                                   | 146 |
| 14.2 CORDIC 主要特征 .....                          | 146 |
| 14.3 CORDIC 功能描述 .....                          | 146 |
| 14.3.1 一般的描述 .....                              | 146 |
| 14.3.2 CORDIC 功能 .....                          | 146 |
| 14.3.3 定点表示法 .....                              | 151 |
| 14.3.4 比例系数 .....                               | 152 |
| 14.3.5 精度 .....                                 | 152 |
| 14.3.6 Zero-overhead 模式 .....                   | 155 |
| 14.3.7 轮询方式 .....                               | 156 |
| 14.3.8 中断模式 .....                               | 156 |
| 14.3.9 直接存储器存取模式 .....                          | 156 |
| 14.4 CORDIC 寄存器描述 .....                         | 158 |
| 14.4.1 CORDIC 控制/状态寄存器 (CORDIC_CSR) .....       | 158 |
| 14.4.2 CORDIC 参数寄存器 (CORDIC_WRDAT) .....        | 159 |
| 14.4.3 CORDIC 结果寄存器 (CORDIC_RDATA) .....        | 160 |
| 15 FMAC 滤波加速器 (FMAC) .....                      | 161 |
| 15.1 概述 .....                                   | 161 |
| 15.2 特性 .....                                   | 161 |
| 15.3 FMAC 功能描述 .....                            | 162 |
| 15.3.1 通用描述 .....                               | 162 |
| 15.3.2 本地存储与缓存 .....                            | 163 |
| 15.3.3 输入缓存 .....                               | 163 |
| 15.3.4 输出缓存 .....                               | 166 |
| 15.4 FMAC 寄存器描述 .....                           | 168 |

|                                             |     |
|---------------------------------------------|-----|
| 15.4.1 FMAC X1 buffer 配置寄存器 (FMAC_X1BUFCFG) | 168 |
| 15.4.2 FMAC X2 buffer 配置寄存器 (FMAC_X2BUFCFG) | 169 |
| 15.4.3 FMAC_Y buffer 配置寄存器 (FMAC_YBUFCFG)   | 169 |
| 15.4.4 FMAC 参数寄存器 (FMAC_PARAM)              | 170 |
| 15.4.5 FMAC 控制寄存器 (FMAC_CR)                 | 171 |
| 15.4.6 FMAC 状态寄存器 (FMAC_SR)                 | 172 |
| 15.4.7 FMAC 写数据寄存器 (FMAC_WDATA)             | 173 |
| 15.4.8 FMAC 读数据寄存器 (FMAC_RDATA)             | 173 |
| 16 模拟数字转换器 (ADC)                            | 174 |
| 16.1 简介                                     | 174 |
| 16.2 ADC 主要特征                               | 174 |
| 16.3 ADC 功能描述                               | 174 |
| 16.3.1 ADC 引脚/通道                            | 176 |
| 16.3.2 ADC 开关控制                             | 178 |
| 16.3.3 ADC 时钟                               | 178 |
| 16.3.4 通道选择                                 | 178 |
| 16.3.5 单次转换模式                               | 178 |
| 16.3.6 连续转换模式                               | 179 |
| 16.3.7 时序图                                  | 179 |
| 16.3.8 模拟看门狗                                | 179 |
| 16.3.9 扫描模式                                 | 180 |
| 16.3.10 外部触发转换                              | 180 |
| 16.3.11 间断模式                                | 181 |
| 16.4 校准                                     | 182 |
| 16.5 数据对齐                                   | 183 |
| 16.6 可编程的通道采样时间                             | 183 |
| 16.7 外部触发转换                                 | 183 |
| 16.8 DMA 请求                                 | 186 |
| 16.9 双 ADC 模式                               | 186 |
| 16.9.1 同步注入模式                               | 187 |
| 16.9.2 同步规则模式                               | 188 |
| 16.9.3 快速交叉模式                               | 188 |
| 16.9.4 慢速交叉模式                               | 189 |
| 16.9.5 交替触发模式                               | 190 |
| 16.9.6 混合的规则/注入同步模式                         | 191 |
| 16.9.7 混合的同步规则+交替触发模式                       | 191 |
| 16.9.8 混合同步注入 + 交叉模式                        | 192 |
| 16.9.9 独立模式                                 | 192 |
| 16.10 温度传感器 (TPS)                           | 192 |
| 16.11 ADC 中断                                | 193 |
| 16.12 ADC 寄存器描述                             | 194 |
| 16.12.1 ADC 状态寄存器 (ADC_SR)                  | 194 |
| 16.12.2 ADC 控制寄存器 1 (ADC_CR1)               | 195 |

|                                                |     |
|------------------------------------------------|-----|
| 16.12.3 ADC 控制寄存器 2 (ADC_CR2)                  | 197 |
| 16.12.4 ADC 采样时间寄存器 1 (ADC1_SMPR1)             | 199 |
| 16.12.5 ADC 采样时间寄存器 2 (ADC1_SMPR2)             | 200 |
| 16.12.6 ADC 注入通道数据偏移寄存器 x (ADC_JOFRx) (x=1..4) | 200 |
| 16.12.7 ADC 看门狗高阈值寄存器 (ADC_HTR)                | 201 |
| 16.12.8 ADC 看门狗低阈值寄存器 (ADC_LTR)                | 201 |
| 16.12.9 ADC 规则序列寄存器 1 (ADC_SQR1)               | 201 |
| 16.12.10 ADC 规则序列寄存器 2 (ADC_SQR2)              | 202 |
| 16.12.11 ADC 规则序列寄存器 3 (ADC_SQR3)              | 202 |
| 16.12.12 ADC 注入序列寄存器 (ADC_JSQR)                | 202 |
| 16.12.13 ADC 注入数据寄存器 (ADC_JDRx) (x=1..4)       | 203 |
| 16.12.14 ADC 通用数据寄存器 (ADC_DR)                  | 203 |
| 16.12.15 ADC 校准寄存器 (ADC_CAL)                   | 204 |
| 16.12.16 ADC TPS 校准寄存器 (ADC_TPS)               | 204 |
| 16.12.17 ADC 规则序列数据寄存器 (ADC_DRx) (x=1..8)      | 204 |
| 17 数字/模拟转换 (DAC)                               | 205 |
| 17.1 DAC 简介                                    | 205 |
| 17.2 主要特征                                      | 205 |
| 17.3 功能描述                                      | 205 |
| 17.3.1 DAC 框图                                  | 205 |
| 17.3.2 DAC 引脚和内部信号                             | 206 |
| 17.3.3 DAC 通道使能                                | 209 |
| 17.3.1 DAC 数据格式                                | 209 |
| 17.3.2 DAC 转换                                  | 210 |
| 17.3.3 DAC 输出电压                                | 211 |
| 17.3.4 DAC 触发器选择                               | 211 |
| 17.3.5 DMA 请求                                  | 212 |
| 17.3.6 噪音的产生                                   | 212 |
| 17.3.7 三角波生成                                   | 213 |
| 17.3.8 DAC 锯齿波的产生                              | 214 |
| 17.3.9 DAC 通道缓冲校准                              | 216 |
| 17.3.10 双 DAC 通道转换模式 (如果有双通道)                  | 217 |
| 17.3.1 低功耗模式的 DAC                              | 221 |
| 17.4 DAC 寄存器描述                                 | 222 |
| 17.4.1 DAC 控制寄存器 (DAC_CR)                      | 222 |
| 17.4.2 DAC 软件触发寄存器 (DAC_SWTRIGR)               | 225 |
| 17.4.3 DAC 通道 1 12 位右对齐数据保持寄存器 (DAC_DHR12R1)   | 226 |
| 17.4.4 DAC 通道 1 12 位左对齐数据保持寄存器 (DAC_DHR12L1)   | 226 |
| 17.4.5 DAC 通道 1 8 位右对齐数据保持寄存器 (DAC_DHR8R1)     | 226 |
| 17.4.6 DAC 通道 2 12 位右对齐数据保持寄存器 (DAC_DHR12R2)   | 227 |
| 17.4.7 DAC 通道 2 12 位左对齐数据保持寄存器 (DAC_DHR12L2)   | 228 |
| 17.4.8 DAC 通道 2 8 位右对齐数据保持寄存器 (DAC_DHR8R2)     | 228 |
| 17.4.9 双 DAC 12 位右对齐数据保持寄存器 (DAC_DHR12RD)      | 228 |

|                                                |     |
|------------------------------------------------|-----|
| 17.4.10 双 DAC12 位左对齐数据保持寄存器 (DAC_DHR12LD)      | 229 |
| 17.4.11 双 DAC 8 位右对齐数据保持寄存器 (DAC_DHR8RD)       | 229 |
| 17.4.12 DAC 通道 1 数据输出寄存器 (DAC_DOR1)            | 229 |
| 17.4.13 DAC 通道 2 数据输出寄存器 (DAC_DOR2)            | 230 |
| 17.4.14 DAC 状态寄存器 (DAC_SR)                     | 230 |
| 17.4.15 DAC 校准寄存器 (DAC_CCR)                    | 231 |
| 17.4.16 DAC 模式控制寄存器 (DAC_MCR)                  | 231 |
| 17.4.17 DAC 通道 1 锯齿寄存器 (DAC_STR1)              | 231 |
| 17.4.18 DAC 通道 2 锯齿寄存器 (DAC_STR2)              | 232 |
| 17.4.19 DAC 锯齿模式寄存器 (DAC_STMODR)               | 232 |
| 18 直接存储器访问控制器 (DMA)                            | 234 |
| 18.1 简介                                        | 234 |
| 18.2 DMA 主要特征                                  | 234 |
| 18.3 DMA 请求映射                                  | 234 |
| 18.4 DMA 功能说明                                  | 235 |
| 18.4.1 DMA 框图                                  | 235 |
| 18.4.2 DMA 引脚和内部信号                             | 236 |
| 18.4.3 DMA 传输                                  | 236 |
| 18.4.4 仲裁器                                     | 236 |
| 18.4.5 DMA 通道                                  | 237 |
| 18.4.6 编程传输方向, 分配源/目标                          | 238 |
| 18.4.7 可编程的数据传输宽度、对齐方式和数据大小端                   | 238 |
| 18.4.8 错误管理                                    | 240 |
| 18.4.9 中断                                      | 240 |
| 18.5 DMA 寄存器描述                                 | 240 |
| 18.5.1 DMA 中断状态寄存器 (DMA_ISR)                   | 240 |
| 18.5.2 DMA 中断寄存器标志清除寄存器 (DMA_IFCR)             | 241 |
| 18.5.3 DMA 通道 x 配置寄存器 (DMA_CCRx) (x=1..8)      | 242 |
| 18.5.4 DMA 通道 x 传输数量寄存器 (DMA_CNDTRx) (x=1..8)  | 243 |
| 18.5.5 DMA 通道 x 外设地址寄存器 (DMA_CPARx) (x=1..8)   | 244 |
| 18.5.6 DMA 通道 x 外设存储地址寄存器 (DMA_CMARx) (x=1..8) | 244 |
| 19 DMA 请求多路复用器 (DMAMUX)                        | 245 |
| 19.1 简介                                        | 245 |
| 19.2 主要特性                                      | 245 |
| 19.3 功能描述                                      | 246 |
| 19.3.1 框图                                      | 246 |
| 19.3.2 DMAMUX 信号                               | 246 |
| 19.3.3 DMAMUX 通道                               | 247 |
| 19.3.4 DMAMUX 请求线路多路复用器                        | 247 |
| 19.3.5 DMAMUX 请求生成器                            | 250 |
| 19.4 DMAMUX 中断                                 | 250 |
| 19.5 DMAMUX 映射                                 | 251 |
| 19.6 DMAMUX 寄存器描述                              | 253 |

|                                                           |     |
|-----------------------------------------------------------|-----|
| 19.6.1 DMAMUX 请求线路多路复用器通道 x 配置寄存器 (DMAMUX_CxCR) (x=0..15) | 253 |
| 19.6.2 DMAMUX 请求线路多路复用器中断通道状态寄存器 (DMAMUX_CSR)             | 254 |
| 19.6.3 DMAMUX 请求线路多路复用器中断清除标志寄存器 (DMAMUX_CFR)             | 254 |
| 19.6.4 DMAMUX 请求生成器通道 x 配置寄存器 (DMAMUX_RGxCR) (x=0..3)     | 254 |
| 19.6.5 DMAMUX 请求生成器中断状态寄存器 (DMAMUX_RGSR)                  | 255 |
| 19.6.6 DMAMUX 请求生成器中断清除标志寄存器 (DMAMUX_RGCFR)               | 255 |
| 20 比较器 (COMP)                                             | 256 |
| 20.1 简介                                                   | 256 |
| 20.2 主要特性                                                 | 256 |
| 20.3 功能描述                                                 | 256 |
| 20.3.1 COMP 框图                                            | 256 |
| 20.3.2 COMP 引脚和内部信号                                       | 257 |
| 20.3.3 COMP 复位和时钟                                         | 257 |
| 20.3.4 COMP 锁定机制                                          | 257 |
| 20.3.5 COMP 防误触发功能                                        | 258 |
| 20.3.6 COMP 迟滞                                            | 259 |
| 20.3.7 COMP 输出遮蔽功能                                        | 259 |
| 20.3.8 COMP 低功耗模式                                         | 260 |
| 20.3.9 COMP 中断                                            | 260 |
| 20.4 COMP 寄存器描述                                           | 261 |
| 20.4.1 比较器 x 控制和状态寄存器 (COMP_CxCR) (x=1/2/3/4)             | 261 |
| 20.4.2 比较器 x 校正寄存器 (COMP_RGxCAL) (x=1/2/3/4)              | 264 |
| 21 运算放大器 (OPAMP)                                          | 265 |
| 21.1 简介                                                   | 265 |
| 21.2 主要特性                                                 | 265 |
| 21.3 功能描述                                                 | 265 |
| 21.3.1 OPAMP 输出重定向到内部 ADC 通道                              | 265 |
| 21.3.2 OPAMP 复位和时钟                                        | 266 |
| 21.3.3 初始化配置                                              | 266 |
| 21.3.4 OPAMP 模式                                           | 266 |
| 21.3.5 OPAMP PGA 放大                                       | 270 |
| 21.3.6 校准                                                 | 270 |
| 21.3.7 OPAMP 低功耗模式                                        | 272 |
| 21.4 OPAMP 寄存器描述                                          | 272 |
| 21.4.1 OPAMPx 控制状态寄存器 (OPAMPx_CSR) (x=1..4)               | 272 |
| 21.4.2 OPAMP1 控制状态寄存器 (OPAMP1_CR2)                        | 274 |
| 21.4.3 OPAMP2 控制状态寄存器 (OPAMP2_CR2)                        | 274 |
| 21.4.4 OPAMP3 控制状态寄存器 (OPAMP3_CR2)                        | 275 |
| 21.4.5 OPAMP4 控制状态寄存器 (OPAMP4_CR2)                        | 275 |
| 22 高分辨率定时器 (HRTIM)                                        | 277 |
| 22.1 HRTIM 简介                                             | 277 |
| 22.2 HRTIM 主要特征                                           | 277 |
| 22.3 HRTIM 功能描述                                           | 278 |



|                                                         |     |
|---------------------------------------------------------|-----|
| 22.3.1 综合描述.....                                        | 278 |
| 22.3.2 HRTIM 引脚和内部信号.....                               | 280 |
| 22.3.3 时钟.....                                          | 283 |
| 22.3.4 定时器 A.F 定时单元.....                                | 287 |
| 22.3.5 主定时器.....                                        | 308 |
| 22.3.6 上下计数模式.....                                      | 310 |
| 22.3.7 置位/复位事件优先级和窄脉冲管理.....                            | 317 |
| 22.3.8 外部事件全局条件.....                                    | 320 |
| 22.3.9 定时单元中的外部事件过滤.....                                | 325 |
| 22.3.10 延迟保护.....                                       | 331 |
| 22.3.11 寄存器预加载和更新管理.....                                | 337 |
| 22.3.12 带有“大于”比较的 PWM 模式.....                           | 341 |
| 22.3.13 事件在多个定时器之间的传播.....                              | 342 |
| 22.3.14 输出管理.....                                       | 347 |
| 22.3.15 突发模式控制器.....                                    | 349 |
| 22.3.16 斩波.....                                         | 357 |
| 22.3.17 故障保护.....                                       | 358 |
| 22.3.18 辅助输出.....                                       | 363 |
| 22.3.19 将 HRTIM 与其他定时器或 HRTIM 实例同步.....                 | 366 |
| 22.3.20 ADC 触发.....                                     | 369 |
| 22.3.21 DAC 触发.....                                     | 373 |
| 22.3.22 中断.....                                         | 377 |
| 22.3.23 DMA.....                                        | 379 |
| 22.3.24 HRTIM 初始化.....                                  | 383 |
| 22.3.25 Debug 调试.....                                   | 384 |
| 22.4 应用案例.....                                          | 385 |
| 22.4.1 降压转换器.....                                       | 385 |
| 22.4.2 具有同步整流功能的降压转换器.....                              | 386 |
| 22.4.3 多相转换器.....                                       | 387 |
| 22.4.4 过渡模式功率因数校正.....                                  | 388 |
| 22.5 HRTIM 寄存器.....                                     | 391 |
| 22.5.1 HRTIM 主定时器控制寄存器 (HRTIM_MCR).....                 | 391 |
| 22.5.2 HRTIM 主定时器中断状态寄存器 (HRTIM_MISR).....              | 394 |
| 22.5.3 HRTIM 主定时器中断清除寄存器 (HRTIM_MICR).....              | 395 |
| 22.5.4 HRTIM 主定时器 DMA/中断使能寄存器 (HRTIM_MDIER).....        | 396 |
| 22.5.5 HRTIM 主定时器计数器寄存器 (HRTIM_MCNT).....               | 398 |
| 22.5.6 HRTIM 主定时器周期寄存器 (HRTIM_MPER).....                | 398 |
| 22.5.7 HRTIM 主定时器重复寄存器 (HRTIM_MREP).....                | 399 |
| 22.5.8 HRTIM 主定时器比较 1 寄存器 (HRTIM_MCMP1R).....           | 399 |
| 22.5.9 HRTIM 主定时器比较 2 寄存器 (HRTIM_MCMP2R).....           | 400 |
| 22.5.10 HRTIM 主定时器比较 3 寄存器 (HRTIM_MCMP3R).....          | 400 |
| 22.5.11 HRTIM 主定时器比较 4 寄存器 (HRTIM_MCMP4R).....          | 401 |
| 22.5.12 HRTIM 定时器 x 控制寄存器 (HRTIM_TIMxCR) (x = A~F)..... | 402 |

|                                                                   |     |
|-------------------------------------------------------------------|-----|
| 22.5.13 HRTIM 定时器 x 中断状态寄存器 (HRTIM_TIMxISR) (x = A~F).....        | 407 |
| 22.5.14 HRTIM 定时器 x 中断清除寄存器 (HRTIM_TIMxICR) (x = A~F).....        | 409 |
| 22.5.15 HRTIM 定时器 x DMA 中断使能寄存器 (HRTIM_TIMxDIER) (x = A~F) .....  | 410 |
| 22.5.16 HRTIM 定时器 x 计数器寄存器 (HRTIM_CNTxR) (x = A~F).....           | 414 |
| 22.5.17 HRTIM 定时器 x 周期寄存器 (HRTIM_PERxR) (x = A~F) .....           | 414 |
| 22.5.18 HRTIM 定时器 x 重复寄存器 (HRTIM_REPxR) (x = A~F) .....           | 415 |
| 22.5.19 HRTIM 定时器 x 比较 1 寄存器 (HRTIM_CMP1xR) (x = A~F) .....       | 415 |
| 22.5.20 HRTIM 定时器 x 比较 1 复合寄存器 (HRTIM_CMP1CxR) (x = A~F).....     | 416 |
| 22.5.21 HRTIM 定时器 x 比较 2 寄存器 (HRTIM_CMP2xR) (x = A~F).....        | 416 |
| 22.5.22 HRTIM 定时器 x 比较 3 寄存器 (HRTIM_CMP3xR) (x = A~F).....        | 417 |
| 22.5.23 HRTIM 定时器 x 比较 4 寄存器 (HRTIM_CMP4xR) (x = A~F).....        | 417 |
| 22.5.24 HRTIM 定时器 x 捕获 1 寄存器 (HRTIM_CPT1xR) (x = A~F) .....       | 418 |
| 22.5.25 HRTIM 定时器 x 捕获 2 寄存器 (HRTIM_CPT2xR) (x = A~F) .....       | 419 |
| 22.5.26 HRTIM 定时器 x 死区时间寄存器 (HRTIM_DTxR) (x = A~F) .....          | 420 |
| 22.5.27 HRTIM 定时器 x 输出 1 置位寄存器 (HRTIM_SETx1R) (x = A~F) .....     | 422 |
| 22.5.28 HRTIM 定时器 x 输出 1 复位寄存器 (HRTIM_RSTx1R) (x = A~F) .....     | 425 |
| 22.5.29 HRTIM 定时器 x 输出 2 置位寄存器 (HRTIM_SETx2R) (x = A~F) .....     | 425 |
| 22.5.30 HRTIM 定时器 x 输出 2 复位寄存器 (HRTIM_RSTx2R) (x = A~F) .....     | 426 |
| 22.5.31 HRTIM 定时器 x 外部事件过滤寄存器 1 (HRTIM_EEFxR1) (x = A~F).....     | 427 |
| 22.5.32 HRTIM 定时器 x 外部事件过滤寄存器 2 (HRTIM_EEFxR2) (x = A~F) .....    | 429 |
| 22.5.33 HRTIM 定时器 A 复位寄存器 (HRTIM_RSTAR) .....                     | 430 |
| 22.5.34 HRTIM 定时器 B 复位寄存器 (HRTIM_RSTBR).....                      | 432 |
| 22.5.35 HRTIM 定时器 C 复位寄存器 (HRTIM_RSTCR).....                      | 434 |
| 22.5.36 HRTIM 定时器 D 复位寄存器 (HRTIM_RSTDR).....                      | 436 |
| 22.5.37 HRTIM 定时器 E 复位寄存器 (HRTIM_RSTER).....                      | 438 |
| 22.5.38 HRTIM 定时器 F 复位寄存器 (HRTIM_RSTFR).....                      | 440 |
| 22.5.39 HRTIM 定时器 x 斩波寄存器 (HRTIM_CHPxR) (x = A~F) .....           | 442 |
| 22.5.40 HRTIM 定时器 A 捕获 1 控制寄存器 (HRTIM_CPT1ACR) .....              | 443 |
| 22.5.41 HRTIM 定时器 B 捕获 1 控制寄存器 (HRTIM_CPT1BCR) .....              | 443 |
| 22.5.42 HRTIM 定时器 C 捕获 1 控制寄存器 (HRTIM_CPT1CCR) .....              | 443 |
| 22.5.43 HRTIM 定时器 D 捕获 1 控制寄存器 (HRTIM_CPT1DCR) .....              | 444 |
| 22.5.44 HRTIM 定时器 E 捕获 1 控制寄存器 (HRTIM_CPT1ECR) .....              | 444 |
| 22.5.45 HRTIM 定时器 F 捕获 1 控制寄存器 (HRTIM_CPT1FCR) .....              | 444 |
| 22.5.46 HRTIM 定时器 x 捕获 2 控制寄存器 (HRTIM_CPT2xCR) (x = A~F) .....    | 445 |
| 22.5.47 HRTIM 定时器 x 输出寄存器 (HRTIM_OUTxR) (x = A~F).....            | 451 |
| 22.5.48 HRTIM 定时器 x 故障寄存器 (HRTIM_FLTxR) (x = A~F) .....           | 454 |
| 22.5.49 HRTIM 定时器 x 控制寄存器 2 (HRTIM_TIMxCR2) (x = A~F) .....       | 455 |
| 22.5.50 HRTIM 定时器 x 外部事件滤波寄存器 3 (HRTIM_TIMxEEFR3) (x = A~F) ..... | 458 |
| 22.5.51 HRTIM 控制寄存器 1 (HRTIM_CR1).....                            | 459 |
| 22.5.52 HRTIM 控制寄存器 2 (HRTIM_CR2).....                            | 461 |
| 22.5.53 HRTIM 中断状态寄存器 (HRTIM_ISR).....                            | 463 |
| 22.5.54 HRTIM 中断清除寄存器 (HRTIM_ICR) .....                           | 464 |
| 22.5.55 HRTIM 中断使能寄存器 (HRTIM_IER) .....                           | 465 |



|                                                            |     |
|------------------------------------------------------------|-----|
| 22.5.56 HRTIM 输出使能寄存器 (HRTIM_OENR)                         | 466 |
| 22.5.57 HRTIM 输出禁用寄存器 (HRTIM_ODISR)                        | 467 |
| 22.5.58 HRTIM 输出禁用状态寄存器 (HRTIM_ODSR)                       | 468 |
| 22.5.59 HRTIM 突发模式控制寄存器 (HRTIM_BMCR)                       | 469 |
| 22.5.60 HRTIM 突发模式触发寄存器 (HRTIM_BMTRGR)                     | 471 |
| 22.5.61 HRTIM 突发模式比较寄存器 (HRTIM_BCMMPR)                     | 473 |
| 22.5.62 HRTIM 突发模式周期寄存器 (HRTIM_BMPER)                      | 473 |
| 22.5.63 HRTIM 定时器外部事件控制寄存器 1 (HRTIM_EECR1)                 | 474 |
| 22.5.64 HRTIM 定时器外部事件控制寄存器 2 (HRTIM_EECR2)                 | 476 |
| 22.5.65 HRTIM 定时器外部事件控制寄存器 3 (HRTIM_EECR3)                 | 477 |
| 22.5.66 HRTIM ADC 触发 1 寄存器 (HRTIM_ADC1R)                   | 478 |
| 22.5.67 HRTIM ADC 触发 2 寄存器 (HRTIM_ADC2R)                   | 479 |
| 22.5.68 HRTIM ADC 触发 3 寄存器 (HRTIM_ADC3R)                   | 480 |
| 22.5.69 HRTIM ADC 触发 4 寄存器 (HRTIM_ADC4R)                   | 482 |
| 22.5.70 HRTIM DLL 控制寄存器 (HRTIM_DLLCR)                      | 484 |
| 22.5.71 HRTIM 故障输入寄存器 1 (HRTIM_FLTINR1)                    | 485 |
| 22.5.72 HRTIM 故障输入寄存器 2 (HRTIM_FLTINR2)                    | 487 |
| 22.5.73 HRTIM 突发 DMA 主定时器更新寄存器 (HRTIM_BDMUPR)              | 489 |
| 22.5.74 HRTIM 突发 DMA 定时器 x 更新寄存器 (HRTIM_BDTxUPR) (x = A~F) | 490 |
| 22.5.75 HRTIM 突发 DMA 数据寄存器 (HRTIM_BDMADR)                  | 491 |
| 22.5.76 HRTIM ADC 扩展触发寄存器 (HRTIM_ADCER)                    | 492 |
| 22.5.77 HRTIM ADC 触发更新寄存器 (HRTIM_ADCUR)                    | 495 |
| 22.5.78 HRTIM ADC 后缩放器寄存器 1 (HRTIM_ADCPS1)                 | 496 |
| 22.5.79 HRTIM ADC 后缩放器寄存器 2 (HRTIM_ADCPS2)                 | 497 |
| 22.5.80 HRTIM 故障输入寄存器 3 (HRTIM_FLTINR3)                    | 498 |
| 22.5.81 HRTIM 故障输入寄存器 4 (HRTIM_FLTINR4)                    | 500 |
| 22.5.82 HRTIM 选项寄存器 (HRTIM_OPT)                            | 501 |
| 23 高级控制定时器 (TIM1/TIM8)                                     | 502 |
| 23.1 TIM1/TIM8 简介                                          | 502 |
| 23.2 TIM1/TIM8 主要特征                                        | 502 |
| 23.3 TIM1/TIM8 功能描述                                        | 503 |
| 23.3.1 框图                                                  | 503 |
| 23.3.2 TIM1/TIM8 引脚和内部信号                                   | 504 |
| 23.3.3 时基单元                                                | 509 |
| 23.3.4 计数模式                                                | 511 |
| 23.3.5 重复计数器                                               | 521 |
| 23.3.6 外部触发输入                                              | 522 |
| 23.3.7 时钟选择                                                | 523 |
| 23.3.8 捕获比较通道                                              | 526 |
| 23.3.9 输入捕获模式                                              | 529 |
| 23.3.10 PWM 输入模式                                           | 529 |
| 23.3.11 强制输出模式                                             | 530 |
| 23.3.12 输出比较模式                                             | 530 |

|                                                            |     |
|------------------------------------------------------------|-----|
| 23.3.13 PWM 模式 .....                                       | 532 |
| 23.3.14 非对称 PWM 模式 .....                                   | 540 |
| 23.3.15 组合 PWM 模式 .....                                    | 540 |
| 23.3.16 三相组合 PWM 模式 .....                                  | 541 |
| 23.3.17 互补输出和死区插入 .....                                    | 542 |
| 23.3.18 使用刹车功能 .....                                       | 545 |
| 23.3.19 双向刹车输入 .....                                       | 550 |
| 23.3.20 在外部事件时清除 tim_ocxref 信号 .....                       | 551 |
| 23.3.21 生成 6 步 PWM .....                                   | 552 |
| 23.3.22 单脉冲模式 .....                                        | 554 |
| 23.3.23 可重触发单脉冲模式 .....                                    | 555 |
| 23.3.24 在比较模式下的脉冲 .....                                    | 556 |
| 23.3.25 编码器接口模式 .....                                      | 557 |
| 23.3.26 方向位输出 .....                                        | 575 |
| 23.3.27 UIF 位重映射 .....                                     | 576 |
| 23.3.28 定时器输入异或功能 .....                                    | 576 |
| 23.3.29 与霍尔传感器接口 .....                                     | 576 |
| 23.3.30 定时器同步 .....                                        | 578 |
| 23.3.31 ADC 触发器 .....                                      | 582 |
| 23.3.32 DMA 突发模式 .....                                     | 582 |
| 23.3.33 TIM1/TIM8/ DMA 请求 .....                            | 583 |
| 23.3.34 调试模式 .....                                         | 584 |
| 23.4 TIM1/TIM8 低功耗模式 .....                                 | 584 |
| 23.5 TIM1/TIM8 中断 .....                                    | 584 |
| 23.6 TIM1/TIM8 寄存器描述 .....                                 | 585 |
| 23.6.1 TIMx 控制寄存器 1 (TIMx_CR1) (x = 1, 8) .....            | 585 |
| 23.6.2 TIMx 控制寄存器 2 (TIMx_CR2) (x = 1, 8) .....            | 586 |
| 23.6.3 TIMx 从模式控制寄存器 (TIMx_SMCR) (x = 1, 8) .....          | 590 |
| 23.6.4 TIMx DMA/中断使能寄存器 (TIMx_DIER) (x = 1, 8) .....       | 594 |
| 23.6.5 TIMx 状态寄存器 (TIMx_SR) (x = 1, 8) .....               | 595 |
| 23.6.6 TIMx 事件产生寄存器 (TIMx_EGR) (x = 1, 8) .....            | 598 |
| 23.6.7 TIMx 捕获/比较模式寄存器 1 [复用] (TIMx_CCMR1) (x=1, 8) .....  | 599 |
| 23.6.8 TIMx 捕获/比较模式寄存器 1[复用] (TIMx_CCMR1) (x = 1, 8) ..... | 601 |
| 23.6.9 TIMx 捕获/比较模式寄存器 2 [复用] (TIMx_CCMR2) (x=1, 8) .....  | 604 |
| 23.6.10 TIMx 捕获/比较模式寄存器 2 [复用] (TIMx_CCMR2) (x=1, 8) ..... | 605 |
| 23.6.11 TIMx 捕获/比较使能寄存器 (TIMx_CCER) (x = 1, 8) .....       | 608 |
| 23.6.12 TIMx 计数器 (TIMx_CNT) (x = 1, 8) .....               | 612 |
| 23.6.13 TIMx 预分频器 (TIMx_PSC) (x = 1, 8) .....              | 612 |
| 23.6.14 TIMx 自动重装载寄存器 (TIMx_ARR) (x = 1, 8) .....          | 613 |
| 23.6.15 TIMx 重复计数寄存器 (TIMx_RCR) (x = 1, 8) .....           | 613 |
| 23.6.16 TIMx 捕获/比较寄存器 1 (TIMx_CCR1) (x = 1, 8) .....       | 614 |
| 23.6.17 TIMx 捕获/比较寄存器 2 (TIMx_CCR2) (x = 1, 8) .....       | 614 |
| 23.6.18 TIMx 捕获/比较寄存器 3 (TIMx_CCR3) (x = 1, 8) .....       | 615 |

|                                                    |     |
|----------------------------------------------------|-----|
| 23.6.19 TIMx 捕获/比较寄存器 4 (TIMx_CCR4) (x = 1, 8)     | 616 |
| 23.6.20 TIMx 刹车和死区寄存器 (TIMx_BDTR) (x = 1, 8)       | 617 |
| 23.6.21 TIMx 捕获/比较寄存器 5 (TIMx_CCR5) (x = 1, 8)     | 622 |
| 23.6.22 TIMx 捕获/比较寄存器 6 (TIMx_CCR6) (x = 1, 8)     | 623 |
| 23.6.23 TIMx 捕获/比较模式寄存器 3 (TIMx_CCMR3) (x = 1, 8)  | 623 |
| 23.6.24 TIMx 定时器死区寄存器 2 (TIMx_DTR2) (x = 1, 8)     | 624 |
| 23.6.25 TIMx 定时器编码器控制寄存器 (TIMx_ECR) (x = 1, 8)     | 625 |
| 23.6.26 TIMx 定时器输入选择寄存器 (TIMx_TISEL) (x=1, 8)      | 626 |
| 23.6.27 TIMx 复用功能选择寄存器 1 (TIMx_AF1) (x = 1, 8)     | 627 |
| 23.6.28 TIMx 复用功能选择寄存器 2 (TIMx_AF2) (x = 1, 8)     | 629 |
| 23.6.29 TIMx 捕获/比较寄存器 1N (TIMx_CCR1N) (x = 1, 8)   | 631 |
| 23.6.30 TIMx 捕获/比较寄存器 2N (TIMx_CCR2N) (x = 1, 8)   | 631 |
| 23.6.31 TIMx 捕获/比较寄存器 3N (TIMx_CCR3N) (x = 1, 8)   | 632 |
| 23.6.32 TIMx 捕获/比较寄存器 4N (TIMx_CCR4N) (x = 1, 8)   | 632 |
| 23.6.33 TIMx 刹车输入控制选择 (TIMx_BICS) (x = 1, 8)       | 633 |
| 23.6.34 TIMx DMA 控制寄存器 (TIMx_DCR) (x = 1, 8)       | 634 |
| 23.6.35 TIMx 用于完整传输的 DMA 地址 (TIMx_DMAR) (x = 1, 8) | 635 |
| 24 通用定时器 (TIM2/TIM3/TIM4/TIM5)                     | 636 |
| 24.1 TIM2/TIM3/TIM4/TIM5 简介                        | 636 |
| 24.2 TIM2/TIM3/TIM4/TIM5 主要功能                      | 636 |
| 24.3 TIM2/TIM3/TIM4/TIM5 实现                        | 636 |
| 24.4 TIM2/TIM3/TIM4/TIM5 功能描述                      | 637 |
| 24.4.1 框图                                          | 637 |
| 24.4.2 TIM2/TIM3/TIM4/TIM5 引脚和内部信号                 | 638 |
| 24.4.3 时基单元                                        | 641 |
| 24.4.4 计数器模式                                       | 643 |
| 24.4.5 时钟选择                                        | 653 |
| 24.4.6 捕获/比较通道                                     | 656 |
| 24.4.7 输入捕获模式                                      | 658 |
| 24.4.8 PWM 输入模式                                    | 659 |
| 24.4.9 强制输出模式                                      | 660 |
| 24.4.10 输出比较模式                                     | 660 |
| 24.4.11 PWM 模式                                     | 661 |
| 24.4.12 非对称 PWM 模式                                 | 670 |
| 24.4.13 组合 PWM 模式                                  | 670 |
| 24.4.14 在外部事件时清除 tim_ocxref 信号                     | 671 |
| 24.4.15 单脉冲模式                                      | 673 |
| 24.4.16 可重触发单脉冲模式                                  | 674 |
| 24.4.17 在比较模式下的脉冲                                  | 675 |
| 24.4.18 编码器接口模式                                    | 677 |
| 24.4.19 方向位输出                                      | 694 |
| 24.4.20 UIF 位重映射                                   | 695 |
| 24.4.21 定时器输入异或功能                                  | 695 |

|                                                           |     |
|-----------------------------------------------------------|-----|
| 24.4.22 定时器和外部触发的同步 .....                                 | 695 |
| 24.4.23 定时器同步 .....                                       | 698 |
| 24.4.24 ADC 触发 .....                                      | 703 |
| 24.4.25 DMA 突发模式 .....                                    | 703 |
| 24.4.26 TIM2/TIM3/TIM4/TIM5 DMA 请求 .....                  | 704 |
| 24.4.27 调试模式 .....                                        | 705 |
| 24.5 TIM2/TIM3/TIM4/TIM5 低功耗模式 .....                      | 705 |
| 24.6 TIM2/TIM3/TIM4/TIM5 中断 .....                         | 705 |
| 24.7 TIM2/TIM3/TIM4/TIM5 寄存器描述 .....                      | 706 |
| 24.7.1 TIMx 控制寄存器 1 (TIMx_CR1) (x=2~5) .....              | 706 |
| 24.7.2 TIMx 控制寄存器 2 (TIMx_CR2) (x=2~5) .....              | 707 |
| 24.7.3 TIMx 从模式控制寄存器 (TIMx_SMCR) (x=2~5) .....            | 709 |
| 24.7.4 TIMx DMA/中断使能寄存器 (TIMx_DIER) (x=2~5) .....         | 713 |
| 24.7.5 TIMx 状态寄存器 (TIMx_SR) (x=2~5) .....                 | 714 |
| 24.7.6 TIMx 事件产生寄存器 (TIMx_EGR) (x=2~5) .....              | 716 |
| 24.7.7 TIMx 捕获/比较模式寄存器 1 [复用] (TIMx_CCMR1) (x=2~5) .....  | 717 |
| 24.7.8 TIMx 捕获/比较模式寄存器 1 [复用] (TIMx_CCMR1) (x=2~5) .....  | 719 |
| 24.7.9 TIMx 捕获/比较模式寄存器 2 [复用] (TIMx_CCMR2) (x=2~5) .....  | 721 |
| 24.7.10 TIMx 捕获/比较模式寄存器 2 [复用] (TIMx_CCMR2) (x=2~5) ..... | 722 |
| 24.7.11 TIMx 捕获/比较使能寄存器 (TIMx_CCER) (x=2~5) .....         | 723 |
| 24.7.12 TIMx 计数器 (TIMx_CNT) (x = 3, 4) .....              | 725 |
| 24.7.13 TIMx 计数器 (TIMx_CNT) (x = 2, 5) .....              | 725 |
| 24.7.14 TIMx 预分频器 (TIMx_PSC) (x=2~5) .....                | 726 |
| 24.7.15 TIMx 自动重装载寄存器 (TIMx_ARR) (x = 3, 4) .....         | 726 |
| 24.7.16 TIMx 自动重装载寄存器 (TIMx_ARR) (x = 2, 5) .....         | 726 |
| 24.7.17 TIMx 捕获/比较寄存器 1 (TIMx_CCR1) (x = 3, 4) .....      | 727 |
| 24.7.18 TIMx 捕获/比较寄存器 1 (TIMx_CCR1) (x = 2, 5) .....      | 727 |
| 24.7.19 TIMx 捕获/比较寄存器 2 (TIMx_CCR2) (x = 3, 4) .....      | 728 |
| 24.7.20 TIMx 捕获/比较寄存器 2 (TIMx_CCR2) (x = 2, 5) .....      | 729 |
| 24.7.21 TIMx 捕获/比较寄存器 3 (TIMx_CCR3) (x = 3, 4) .....      | 730 |
| 24.7.22 TIMx 捕获/比较寄存器 3 (TIMx_CCR3) (x = 2, 5) .....      | 731 |
| 24.7.23 TIMx 捕获/比较寄存器 4 (TIMx_CCR4) (x = 3, 4) .....      | 732 |
| 24.7.24 TIMx 捕获/比较寄存器 4 (TIMx_CCR4) (x = 2, 5) .....      | 733 |
| 24.7.25 TIMx 编码器控制寄存器 (TIMx_ECR) (x=2~5) .....            | 734 |
| 24.7.26 TIMx 定时器输入选择寄存器 (TIMx_TISEL) (x=2~5) .....        | 735 |
| 24.7.27 TIMx 复用功能选择寄存器 1 (TIMx_AF1) (x=2~5) .....         | 736 |
| 24.7.28 TIMx 复用功能选择寄存器 2 (TIMx_AF2) (x=2~5) .....         | 737 |
| 24.7.29 TIMx DMA 控制寄存器 (TIMx_DCR) (x=2~5) .....           | 738 |
| 24.7.30 TIMx 用于完整传输的 DMA 地址 (TIMx_DMAR) (x=2~5) .....     | 739 |
| 25 通用定时器 (TIM15/TIM16/TIM17) .....                        | 740 |
| 25.1 TIM15/TIM16/TIM17 简介 .....                           | 740 |
| 25.2 TIM15 主要功能 .....                                     | 740 |
| 25.3 TIM16/TIM17 主要功能 .....                               | 741 |

|                                                   |     |
|---------------------------------------------------|-----|
| 25.4 TIM15/TIM16/TIM17 功能描述.....                  | 742 |
| 25.4.1 框图 .....                                   | 742 |
| 25.4.2 TIM15/TIM16/TIM17 引脚和内部信号.....             | 744 |
| 25.4.3 时基单元 .....                                 | 747 |
| 25.4.4 计数器模式 .....                                | 749 |
| 25.4.5 重复计数器 .....                                | 752 |
| 25.4.6 时钟选择 .....                                 | 753 |
| 25.4.7 捕获/比较通道 .....                              | 755 |
| 25.4.8 输入捕获模式 .....                               | 757 |
| 25.4.9 PWM 输入模式（仅 TIM15） .....                    | 758 |
| 25.4.10 强制输出模式 .....                              | 759 |
| 25.4.11 输出比较模式 .....                              | 759 |
| 25.4.12 PWM 模式 .....                              | 760 |
| 25.4.13 组合 PWM 模式（仅 TIM15） .....                  | 765 |
| 25.4.14 互补输出和死区插入 .....                           | 766 |
| 25.4.15 使用刹车功能 .....                              | 769 |
| 25.4.16 双向刹车输入 .....                              | 773 |
| 25.4.17 在外部事件时清除 tim_ocxref 信号.....               | 774 |
| 25.4.18 单脉冲模式 .....                               | 774 |
| 25.4.19 可重触发单脉冲模式（仅 TIM15） .....                  | 776 |
| 25.4.20 UIF 位重映射 .....                            | 777 |
| 25.4.21 定时器输入异或功能（仅 TIM15） .....                  | 777 |
| 25.4.22 定时器和外部触发的同步（仅 TIM15） .....                | 777 |
| 25.4.23 定时器同步（仅 TIM15） .....                      | 780 |
| 25.4.24 使用定时器输出作为其他定时器的触发器（仅 TIM16/TIM17） .....   | 780 |
| 25.4.25 ADC 触发（仅 TIM15） .....                     | 780 |
| 25.4.26 DMA 突发模式 .....                            | 780 |
| 25.4.27 TIM15/TIM16/TIM17 DMA 请求 .....            | 781 |
| 25.4.28 调试模式 .....                                | 781 |
| 25.5 TIM15/TIM16/TIM17 低功耗模式.....                 | 782 |
| 25.6 TIM15/TIM16/TIM17 中断.....                    | 782 |
| 25.7 TIM15 寄存器描述.....                             | 783 |
| 25.7.1 TIM15 控制寄存器 1（TIM15_CR1） .....             | 783 |
| 25.7.2 TIM15 控制寄存器 2（TIM15_CR2） .....             | 784 |
| 25.7.3 TIM15 从模式控制寄存器（TIM15_SMCR） .....           | 786 |
| 25.7.4 TIM15 DMA/中断使能寄存器（TIM15_DIER） .....        | 787 |
| 25.7.5 TIM15 状态寄存器（TIM15_SR） .....                | 788 |
| 25.7.6 TIM15 事件产生寄存器（TIM15_EGR） .....             | 790 |
| 25.7.7 TIM15 捕获/比较模式寄存器 1 [复用]（TIM15_CCMR1） ..... | 791 |
| 25.7.8 TIM15 捕获/比较模式寄存器 1 [复用]（TIM15_CCMR1） ..... | 792 |
| 25.7.9 TIM15 捕获/比较使能寄存器（TIM15_CCER） .....         | 795 |
| 25.7.10 TIM15 计数器（TIM15_CNT） .....                | 798 |
| 25.7.11 TIM15 预分频器（TIM15_PSC） .....               | 798 |



|                                                      |     |
|------------------------------------------------------|-----|
| 25.7.12 TIM15 自动重载寄存器 (TIM15_ARR)                    | 799 |
| 25.7.13 TIM15 重复计数寄存器 (TIM15_RCR)                    | 799 |
| 25.7.14 TIM15 捕获/比较寄存器 1 (TIM15_CCR1)                | 800 |
| 25.7.15 TIM15 捕获/比较寄存器 2 (TIM15_CCR2)                | 801 |
| 25.7.16 TIM15 刹车和死区寄存器 (TIM15_BDTR)                  | 801 |
| 25.7.17 TIM15 定时器死区寄存器 2 (TIM15_DTR2)                | 804 |
| 25.7.18 TIM15 定时器输入选择寄存器 (TIM15_TISEL)               | 805 |
| 25.7.19 TIM15 复用功能选择寄存器 1 (TIM15_AF1)                | 806 |
| 25.7.20 TIM15 复用功能选择寄存器 2 (TIM15_AF2)                | 808 |
| 25.7.21 TIM15 DMA 控制寄存器 (TIM15_DCR)                  | 808 |
| 25.7.22 TIM15 用于完整传输的 DMA 地址 (TIM15_DMAR)            | 809 |
| 25.8 TIM16/TIM17 寄存器描述                               | 810 |
| 25.8.1 TIMx 控制寄存器 1 (TIMx_CR1) (x=16~17)             | 810 |
| 25.8.2 TIMx 控制寄存器 2 (TIMx_CR2) (x=16~17)             | 811 |
| 25.8.3 TIMx DMA/中断使能寄存器 (TIMx_DIER) (x=16~17)        | 812 |
| 25.8.4 TIMx 状态寄存器 (TIMx_SR) (x=16~17)                | 813 |
| 25.8.5 TIMx 事件产生寄存器 (TIMx_EGR) (x=16~17)             | 814 |
| 25.8.6 TIMx 捕获/比较模式寄存器 1 [复用] (TIMx_CCMR1) (x=16~17) | 815 |
| 25.8.7 TIMx 捕获/比较模式寄存器 1 [复用] (TIMx_CCMR1) (x=16~17) | 816 |
| 25.8.8 TIMx 捕获/比较使能寄存器 (TIMx_CCER) (x=16~17)         | 818 |
| 25.8.9 TIMx 计数器 (TIMx_CNT) (x=16~17)                 | 821 |
| 25.8.10 TIMx 预分频器 (TIMx_PSC) (x=16~17)               | 821 |
| 25.8.11 TIMx 自动重载寄存器 (TIMx_ARR) (x=16~17)            | 822 |
| 25.8.12 TIMx 重复计数寄存器 (TIMx_RCR) (x=16~17)            | 822 |
| 25.8.13 TIMx 捕获/比较寄存器 1 (TIMx_CCR1) (x=16~17)        | 823 |
| 25.8.14 TIMx 刹车和死区寄存器 (TIMx_BDTR) (x=16~17)          | 824 |
| 25.8.15 TIMx 定时器死区寄存器 2 (TIMx_DTR2) (x=16~17)        | 827 |
| 25.8.16 TIMx 定时器输入选择寄存器 (TIMx_TISEL) (x=16~17)       | 828 |
| 25.8.17 TIMx 复用功能选择寄存器 1 (TIMx_AF1) (x=16~17)        | 828 |
| 25.8.18 TIMx 复用功能选择寄存器 2 (TIMx_AF2) (x=16~17)        | 830 |
| 25.8.19 TIMx 选项寄存器 1 (TIMx_OR1) (x=16~17)            | 830 |
| 25.8.20 TIMx DMA 控制寄存器 (TIMx_DCR) (x=16~17)          | 831 |
| 25.8.21 TIMx 用于完整传输的 DMA 地址 (TIMx_DMAR) (x=16~17)    | 831 |
| 26 基本定时器 (TIM6/TIM7)                                 | 832 |
| 26.1 TIM6/TIM7 简介                                    | 832 |
| 26.2 TIM6/TIM7 主要功能                                  | 832 |
| 26.3 TIM6/TIM7 功能描述                                  | 832 |
| 26.3.1 框图                                            | 832 |
| 26.3.2 TIM6/TIM7 内部信号                                | 833 |
| 26.3.3 TIM6/TIM7 时钟                                  | 833 |
| 26.3.4 时基单元                                          | 833 |
| 26.3.5 计数器模式                                         | 835 |
| 26.3.6 UIF 位重映射                                      | 843 |



|                                                   |     |
|---------------------------------------------------|-----|
| 26.3.7 ADC 触发.....                                | 843 |
| 26.3.8 TIM6/TIM7 DMA 请求.....                      | 843 |
| 26.3.9 调试模式.....                                  | 844 |
| 26.4 TIM6/TIM7 低功耗模式.....                         | 844 |
| 26.5 TIM6/TIM7 中断.....                            | 844 |
| 26.6 TIMx 寄存器描述.....                              | 845 |
| 26.6.1 TIMx 控制寄存器 1 (TIMx_CR1) (x=6/7) .....      | 845 |
| 26.6.2 TIMx 控制寄存器 2 (TIMx_CR2) (x=6/7) .....      | 847 |
| 26.6.3 TIMx/DMA/中断使能寄存器 (TIMx_DIER) (x=6/7) ..... | 847 |
| 26.6.4 TIMx 状态寄存器 (TIMx_SR) (x=6/7) .....         | 848 |
| 26.6.5 TIMx 事件产生寄存器 (TIMx_EGR) (x=6/7) .....      | 848 |
| 26.6.6 TIMx 计数器 (TIMx_CNT) (x=6/7) .....          | 848 |
| 26.6.7 TIMx 预分频器 (TIMx_PSC) (x=6/7) .....         | 849 |
| 26.6.8 TIMx 自动重载寄存器 (TIMx_ARR) (x=6/7) .....      | 849 |
| 27 电压基准缓冲器 (VREFBUF) .....                        | 850 |
| 27.1 简介.....                                      | 850 |
| 27.2 VREFBUF 功能描述.....                            | 850 |
| 27.3 VREFBUF 校准.....                              | 850 |
| 27.4 VREFBUF 寄存器描述.....                           | 851 |
| 27.4.1 VREFBUF 控制状态寄存器 (VREFBUF_CSR) .....        | 851 |
| 27.4.2 VREFBUF 控制寄存器 (VREFBUF_CCR) .....          | 851 |
| 28 实时时钟 (RTC) .....                               | 853 |
| 28.1 RTC 简介.....                                  | 853 |
| 28.2 主要特性.....                                    | 853 |
| 28.3 功能描述.....                                    | 853 |
| 28.3.1 概述.....                                    | 853 |
| 28.3.2 复位过程.....                                  | 854 |
| 28.3.3 配置 RTC 寄存器.....                            | 855 |
| 28.3.4 RTC 标志的设置.....                             | 855 |
| 28.4 RTC 寄存器描述.....                               | 856 |
| 28.4.1 RTC 控制寄存器高位 (RTC_CRH) .....                | 856 |
| 28.4.2 RTC 控制寄存器低位 (RTC_CRL) .....                | 857 |
| 28.4.3 RTC 预分频装载寄存器 (RTC_PRLH/RTC_PRLL) .....     | 858 |
| 28.4.4 RTC 预分频器余数寄存器 (RTC_DIVH / RTC_DIVL) .....  | 859 |
| 28.4.5 RTC 计数器寄存器 (RTC_CNTH/RTC_CNTL) .....       | 859 |
| 28.4.6 RTC 闹钟寄存器 (RTC_ALRH/RTC_ALRL) .....        | 860 |
| 29 独立看门狗 (IWDG) .....                             | 861 |
| 29.1 简介.....                                      | 861 |
| 29.2 IWDG 主要性能.....                               | 861 |
| 29.3 IWDG 功能描述.....                               | 861 |
| 29.3.1 硬件看门狗.....                                 | 861 |
| 29.3.2 寄存器访问保护.....                               | 861 |
| 29.3.3 调试模式.....                                  | 861 |

|                                |     |
|--------------------------------|-----|
| 29.4 IWDG 寄存器描述.....           | 862 |
| 29.4.1 键寄存器 (IWDG_KR) .....    | 862 |
| 29.4.2 预分频寄存器 (IWDG_PR) .....  | 863 |
| 29.4.3 重装载寄存器 (IWDG_RLR) ..... | 863 |
| 29.4.4 状态寄存器 (IWDG_SR) .....   | 864 |
| 30 窗口看门狗 (WWDG) .....          | 865 |
| 30.1 WWDG 简介.....              | 865 |
| 30.2 WWDG 主要特性.....            | 865 |
| 30.3 WWDG 功能描述.....            | 865 |
| 30.4 如何编写看门狗超时程序 .....         | 867 |
| 30.5 调试模式 .....                | 867 |
| 30.6 WWDG 寄存器描述.....           | 868 |
| 30.6.1 控制寄存器 (WWDG_CR) .....   | 868 |
| 30.6.2 配置寄存器 (WWDG_CFR) .....  | 868 |
| 30.6.3 状态寄存器 (WWDG_SR) .....   | 869 |
| 31 USB 全速设备接口 (USB) .....      | 870 |
| 31.1 简介 .....                  | 870 |
| 31.2 主要特征.....                 | 870 |
| 31.3 USB 框图.....               | 871 |
| 31.4 USB 功能描述.....             | 871 |
| 31.4.1 USB 功能模块描述.....         | 872 |
| 31.5 编程中需要考虑的问题 .....          | 873 |
| 31.5.1 系统复位和上电复位 .....         | 873 |
| 31.5.2 双缓冲端点.....              | 877 |
| 31.5.3 同步传输.....               | 878 |
| 31.5.4 挂起/恢复事件.....            | 879 |
| 31.6 USB 寄存器描述 .....           | 880 |
| 31.6.1 通用寄存器.....              | 880 |
| 31.6.2 端点寄存器.....              | 885 |
| 31.6.3 缓冲区描述表.....             | 888 |
| 32 控制器局域网 (bxCAN) .....        | 891 |
| 32.1 简介.....                   | 891 |
| 32.2 bxCAN 主要特点.....           | 891 |
| 32.3 bxCAN 总体描述.....           | 891 |
| 32.3.1 CAN 2.0B 主动内核.....      | 892 |
| 32.3.2 控制、状态和配置寄存器.....        | 892 |
| 32.3.3 发送邮箱.....               | 892 |
| 32.3.4 接收过滤器.....              | 892 |
| 32.4 bxCAN 工作模式.....           | 893 |
| 32.4.1 初始化模式 .....             | 893 |
| 32.4.2 正常模式.....               | 894 |
| 32.4.3 睡眠模式(低功耗).....          | 894 |
| 32.5 测试模式.....                 | 895 |

|                                                                |     |
|----------------------------------------------------------------|-----|
| 32.5.1 静默模式.....                                               | 895 |
| 32.5.2 环回模式.....                                               | 896 |
| 32.5.3 环回静默模式.....                                             | 897 |
| 32.6 处于调试模式时.....                                              | 897 |
| 32.7 bxCAN 功能描述.....                                           | 897 |
| 32.7.1 发送处理.....                                               | 897 |
| 32.7.2 时间触发通信模式.....                                           | 899 |
| 32.7.3 接收管理.....                                               | 899 |
| 32.7.4 标识符过滤.....                                              | 901 |
| 32.7.5 报文存储.....                                               | 905 |
| 32.7.6 出错管理.....                                               | 906 |
| 32.7.7 位时间特性.....                                              | 907 |
| 32.8 bxCAN 中断.....                                             | 909 |
| 32.9 寄存器访问保护.....                                              | 910 |
| 32.10 CAN 控制和状态寄存器描述.....                                      | 911 |
| 32.10.1 CAN 主控制寄存器 (CAN_MCR) .....                             | 911 |
| 32.10.2 CAN 主状态寄存器 (CAN_MSR) .....                             | 912 |
| 32.10.3 CAN 发送状态寄存器 (CAN_TSR) .....                            | 913 |
| 32.10.4 CAN 接收 FIFO 0 寄存器 (CAN_RF0R) .....                     | 915 |
| 32.10.5 CAN 接收 FIFO 1 寄存器 (CAN_RF1R) .....                     | 916 |
| 32.10.6 CAN 中断使能寄存器 (CAN_IER) .....                            | 916 |
| 32.10.7 CAN 错误状态寄存器 (CAN_ESR) .....                            | 917 |
| 32.10.8 CAN 位时序寄存器 (CAN_BTR).....                              | 918 |
| 32.11 CAN 邮箱寄存器描述.....                                         | 919 |
| 32.11.1 发送邮箱标识符寄存器 (CAN_TiRx) (x=0..2) .....                   | 920 |
| 32.11.2 发送邮箱数据长度和时间戳寄存器 (CAN_TDTxR) (x=0..2) .....             | 920 |
| 32.11.3 发送邮箱低字节数据寄存器 (CAN_TDLxR) (x=0..2) .....                | 921 |
| 32.11.4 发送邮箱高字节数据寄存器 (CAN_TDHxR) (x=0..2) .....                | 921 |
| 32.11.5 接收 FIFO 邮箱标识符寄存器 (CAN_RiRx) (x=0..1) .....             | 922 |
| 32.11.6 接收 FIFO 邮箱数据长度和时间戳寄存器 (CAN_RDTxR) (x=0..1) .....       | 922 |
| 32.11.7 接收 FIFO 邮箱低字节数据寄存器 (CAN_RDLxR) (x=0..1) .....          | 923 |
| 32.11.8 接收 FIFO 邮箱高字节数据寄存器 (CAN_RDHxR) (x=0..1) .....          | 923 |
| 32.12 CAN 过滤器寄存器描述.....                                        | 924 |
| 32.12.1 CAN 过滤器主控寄存器 (CAN_FMR) .....                           | 924 |
| 32.12.2 CAN 过滤器模式寄存器 (CAN_FM1R) .....                          | 924 |
| 32.12.3 CAN 过滤器位宽寄存器 (CAN_FS1R) .....                          | 924 |
| 32.12.4 CAN 过滤器 FIFO 关联寄存器 (CAN_FFA1R) .....                   | 925 |
| 32.12.5 CAN 过滤器激活寄存器 (CAN_FA1R) .....                          | 925 |
| 32.12.6 CAN 过滤器组 i 的寄存器 x (CAN_FiRx) ) (i=0..13; x=1..2) ..... | 926 |
| 33 串行外设接口 (SPI) .....                                          | 927 |
| 33.1 SPI 简介.....                                               | 927 |
| 33.2 SPI 主要特征.....                                             | 927 |
| 33.2.1 SPI 特征.....                                             | 927 |

|                                           |     |
|-------------------------------------------|-----|
| 33.3 SPI 功能描述                             | 927 |
| 33.3.1 概述                                 | 927 |
| 33.3.2 配置 SPI 为从模式                        | 932 |
| 33.3.3 配置 SPI 为主模式                        | 932 |
| 33.3.4 配置 SPI 为单工通信                       | 933 |
| 33.3.5 数据发送与接收过程                          | 934 |
| 33.3.6 CRC 计算                             | 939 |
| 33.3.7 状态标志                               | 940 |
| 33.3.8 关闭 SPI                             | 941 |
| 33.3.9 利用 DMA 的 SPI 通信                    | 941 |
| 33.3.10 错误标志                              | 943 |
| 33.3.11 SPI 中断                            | 944 |
| 33.4 SPI 寄存器描述                            | 944 |
| 33.4.1 SPI 控制寄存器 1 (SPI_CR1)              | 944 |
| 33.4.2 SPI 控制寄存器 2 (SPI_CR2)              | 946 |
| 33.4.3 SPI 状态寄存器 (SPI_SR)                 | 946 |
| 33.4.4 SPI 数据寄存器 (SPI_DR)                 | 947 |
| 33.4.5 SPI CRC 多项式寄存器 (SPI_CRCPR)         | 948 |
| 33.4.6 SPI Rx CRC 寄存器 (SPI_RXCRCR)        | 948 |
| 33.4.7 SPI Tx CRC 寄存器 (SPI_TXCRCR)        | 948 |
| 34 I <sup>2</sup> C 接口 (I <sup>2</sup> C) | 950 |
| 34.1 I <sup>2</sup> C 简介                  | 950 |
| 34.2 I <sup>2</sup> C 主要特点                | 950 |
| 34.3 I <sup>2</sup> C 功能描述                | 951 |
| 34.3.1 模式选择                               | 951 |
| 34.3.2 I <sup>2</sup> C 从模式               | 952 |
| 34.3.3 I <sup>2</sup> C 主模式               | 955 |
| 34.3.4 错误条件                               | 957 |
| 34.3.5 SDA/SCL 线控制                        | 958 |
| 34.3.6 SMBus                              | 959 |
| 34.3.7 DMA 请求                             | 961 |
| 34.3.8 包错误校验 (PEC)                        | 962 |
| 34.4 I <sup>2</sup> C 中断请求                | 962 |
| 34.5 I <sup>2</sup> C 调试模式                | 964 |
| 34.6 I <sup>2</sup> C 寄存器描述               | 965 |
| 34.6.1 控制寄存器 1 (I2C_CR1)                  | 965 |
| 34.6.2 控制寄存器 2 (I2C_CR2)                  | 967 |
| 34.6.3 自身地址寄存器 1 (I2C_OAR1)               | 968 |
| 34.6.4 自身地址寄存器 2 (I2C_OAR2)               | 968 |
| 34.6.5 数据寄存器 (I2C_DR)                     | 969 |
| 34.6.6 状态寄存器 1 (I2C_SR1)                  | 969 |
| 34.6.7 状态寄存器 2 (I2C_SR2)                  | 972 |
| 34.6.8 时钟控制寄存器 (I2C_CCR)                  | 973 |

|                              |     |
|------------------------------|-----|
| 34.6.9 TRISE 寄存器 (I2C_TRISE) | 974 |
| 35 通用异步收发器 (UART)            | 975 |
| 35.1 UART 介绍                 | 975 |
| 35.2 UART 主要特性               | 975 |
| 35.3 UART 功能概述               | 976 |
| 35.3.1 UART 特性描述             | 977 |
| 35.3.2 发送器                   | 978 |
| 35.3.3 接收器                   | 980 |
| 35.3.4 分数波特率的产生              | 984 |
| 35.3.5 UART 接收器容忍时钟的变化       | 985 |
| 35.3.6 校验控制                  | 985 |
| 35.3.7 LIN (局域互联网) 模式        | 986 |
| 35.3.8 单线半双工通信               | 988 |
| 35.4 UART 中断请求               | 989 |
| 35.5 UART 模式配置               | 990 |
| 35.6 UART 寄存器描述              | 991 |
| 35.6.1 状态寄存器 (UART_SR)       | 991 |
| 35.6.2 数据寄存器 (UART_DR)       | 993 |
| 35.6.3 波特比率寄存器 (UART_BRR)    | 993 |
| 35.6.4 控制寄存器 1 (UART_CR1)    | 993 |
| 35.6.5 控制寄存器 2 (UART_CR2)    | 995 |
| 35.6.6 控制寄存器 3 (UART_CR3)    | 996 |
| 36 版本历史                      | 998 |

## 表目录

|                                    |     |
|------------------------------------|-----|
| 表 3.1 RX32G410 系列存储器映像和外设边界地址..... | 44  |
| 表 4.1 Flash 结构.....                | 47  |
| 表 4.2 等待周期和 CPU 时钟频率对应关系 .....     | 48  |
| 表 6.1 睡眠模式.....                    | 65  |
| 表 6.2 停止 0 模式.....                 | 67  |
| 表 6.3 停止 1 模式.....                 | 67  |
| 表 6.4 待机模式.....                    | 69  |
| 表 9.1 端口位配置表 <sup>(1)</sup> .....  | 110 |
| 表 11.1 RX32G410 系列向量表 .....        | 126 |
| 表 12.1 EXTI 线路连接 .....             | 132 |
| 表 13.1 示例逻辑函数.....                 | 140 |
| 表 14.1 列出了 CORDIC 协同处理器支持的函数.....  | 146 |
| 表 14.2 余弦函数参数.....                 | 147 |
| 表 14.3 正弦参数.....                   | 147 |
| 表 14.4 相位参数.....                   | 148 |
| 表 14.5 模量参数.....                   | 148 |
| 表 14.6 反正切参数.....                  | 149 |
| 表 14.7 双曲余弦参数.....                 | 149 |
| 表 14.8 双曲正弦参数.....                 | 149 |
| 表 14.9 双曲反正切参数.....                | 150 |
| 表 14.10 自然对数参数.....                | 150 |
| 表 14.11 自然对数标度系数及其对应范围.....        | 151 |
| 表 14.12 平方根参数.....                 | 151 |
| 表 14.13 平方根比例系数和相应的范围.....         | 151 |
| 表 14.14 精度 vs.迭代次数 .....           | 154 |
| 表 16.1 ADC 引脚.....                 | 176 |
| 表 16.2 ADC1 通道 .....               | 176 |
| 表 16.3 ADC2 通道 .....               | 177 |
| 表 16.4 ADC3 通道.....                | 177 |
| 表 16.5 模拟看门狗通道选择.....              | 180 |
| 表 16.6 ADC1 规则通道的外部触发信号 .....      | 183 |
| 表 16.7 ADC2 规则通道的外部触发信号 .....      | 184 |
| 表 16.8 ADC3 规则通道的外部触发信号 .....      | 184 |
| 表 16.9 ADC1 注入通道的外部触发信号 .....      | 185 |
| 表 16.10 ADC2 注入通道的外部触发信号 .....     | 185 |
| 表 16.11 ADC3 注入通道的外部触发信号 .....     | 186 |
| 表 16.12 ADC 中断 .....               | 193 |
| 表 17.1 DAC 引脚.....                 | 206 |
| 表 17.2 DAC 输入/输出引脚.....            | 206 |
| 表 17.3 DAC1 通道 1 互联.....           | 206 |
| 表 17.4 DAC 输入/输出信号.....            | 206 |



|                                                               |     |
|---------------------------------------------------------------|-----|
| 表 17.5 DAC1 通道 2 互联.....                                      | 207 |
| 表 17.6 DAC2 通道 1 互联.....                                      | 207 |
| 表 17.7 DAC2 通道 2 互联.....                                      | 208 |
| 表 17.8 DAC3 通道 1 互联.....                                      | 208 |
| 表 17.9 DAC3 通道 2 互联.....                                      | 209 |
| 表 17.10 HFSEL 描述.....                                         | 211 |
| 表 18.1 DMA 内部输入输出信号 .....                                     | 236 |
| 表 18.2 可编程的数据传输宽度和大小端操作（当 PINC = MINC = 1） .....              | 239 |
| 表 18.3 DMA 中断请求 .....                                         | 240 |
| 表 19.1 DMAMUX：多路复用器输入源 .....                                  | 246 |
| 表 19.2 DMAMUX 中断.....                                         | 250 |
| 表 19.3 分配多路复用器输入到 DMAMUX 资源表 .....                            | 251 |
| 表 19.4 DMAMUX：触发输入源.....                                      | 252 |
| 表 19.5 DMAMUX：同步输入源.....                                      | 252 |
| 表 20.1 COMPx 正向输入分配.....                                      | 257 |
| 表 20.2 COMPx 反向输入分配.....                                      | 257 |
| 表 20.3 COMPx_P 端和 ADC 的 IO 对应关系.....                          | 258 |
| 表 20.4 COMPx_N 端和 ADC 的 IO 对应关系.....                          | 258 |
| 表 20.5 遮蔽源.....                                               | 259 |
| 表 20.6 比较器在低功耗模式下的行为 .....                                    | 260 |
| 表 21.1 运放 OPAMPx 内部输出和 ADC 通道之间可能的分配连接 .....                  | 265 |
| 表 21.2 工作模式及校正 .....                                          | 271 |
| 表 21.3. 低功耗模式对 OPAMP 的影响.....                                 | 272 |
| 表 22.1 HRTIM 输入/输出汇总 .....                                    | 280 |
| 表 22.2 外部事件映射及相关功能 .....                                      | 282 |
| 表 22.3 更新使能输入和源 .....                                         | 282 |
| 表 22.4 来自通用定时器的突发模式时钟源.....                                   | 282 |
| 表 22.5 默认输入 .....                                             | 283 |
| 表 22.6 HRTIM DAC 触发器连接.....                                   | 283 |
| 表 22.7 定时器分辨率和最小 PWM 频率（ $f_{HRTIM} = 192\text{MHz}$ 时） ..... | 284 |
| 表 22.8 周期寄存器与比较寄存器的最小和最大值.....                                | 288 |
| 表 22.9 定时器工作模式.....                                           | 288 |
| 表 22.10 定时器 A 到 F 之间的事件映射.....                                | 294 |
| 表 22.11 交错模式选择 .....                                          | 295 |
| 表 22.12 交错模式下比较 1 至 3 的值 .....                                | 296 |
| 表 22.13 死区分辨率和最大绝对值 .....                                     | 307 |
| 表 22.14 翻转事件目标和模式编程.....                                      | 314 |
| 表 22.15 EEFLTR[3:0]代码取决于 UDM 位设置 .....                        | 316 |
| 表 22.16 外部事件特征 .....                                          | 322 |
| 表 22.17 输出置位/复位延迟和抖动与外部事件工作模式的关系 .....                        | 323 |
| 表 22.18 每个定时器的过滤信号映射 .....                                    | 326 |
| 表 22.19 每个定时器的窗口信号映射（EEFLTR[3:0] = 1111） .....                | 328 |
| 表 22.20 HRTIM 可预装载控制寄存器和关联的更新源.....                           | 338 |

|                                                    |     |
|----------------------------------------------------|-----|
| 表 22.21 主定时器更新事件传播 .....                           | 343 |
| 表 22.22 TIMx 更新事件传播 .....                          | 343 |
| 表 22.23 能够生成更新事件的复位事件 .....                        | 345 |
| 表 22.24 用于定时器复位的更新事件传播 .....                       | 345 |
| 表 22.25 输出状态编程, x= A..F, y= 1 或 2.....             | 347 |
| 表 22.26 突发模式的定时器输出编程 .....                         | 350 |
| 表 22.27 故障输入 .....                                 | 359 |
| 表 22.28 采样速率和滤波器长度与 FLTxF[3:0]和时钟设置的关系 .....       | 361 |
| 表 22.29 故障输入消隐事件 .....                             | 361 |
| 表 22.30 故障 1 至 6 计数器复位源 .....                      | 362 |
| 表 22.31 同步事件的作用与定时器工作模式之间的关系 .....                 | 367 |
| 表 22.32 DAC 双通道触发示例 .....                          | 375 |
| 表 22.33 HRTIM 中断汇总 .....                           | 378 |
| 表 22.34 HRTIM DMA 请求总结 .....                       | 380 |
| 表 23.1 TIM 输入/输出引脚 .....                           | 504 |
| 表 23.2 TIM 内部输入/输出信号 .....                         | 504 |
| 表 23.3 连接到 tim_ti1 的输入多路复用器 .....                  | 505 |
| 表 23.4 连接到 tim_ti2 的输入多路复用器 .....                  | 505 |
| 表 23.5 连接到 tim_ti3 的输入多路复用器 .....                  | 505 |
| 表 23.6 连接到 tim_ti4 的输入多路复用器 .....                  | 506 |
| 表 23.7 TIMx 内部触发连接 .....                           | 506 |
| 表 23.8 连接到 tim_etr 的输入多路复用器 .....                  | 507 |
| 表 23.9 定时器刹车互连 .....                               | 507 |
| 表 23.10 定时器刹车 2 互连 .....                           | 507 |
| 表 23.11 系统中断互连 .....                               | 508 |
| 表 23.12 连接到 ocref_clr 的输入多路复用器 .....               | 508 |
| 表 23.13 CCR 和 ARR 寄存器的变化抖动模式 .....                 | 538 |
| 表 23.14 CCR 寄存器在中心对齐 PWM 模式下的变化抖动模式 .....          | 539 |
| 表 23.15 与 tim_brk/tim_brk2 输入相对应的定时器输出行为 .....     | 549 |
| 表 23.16 刹车保护解除条件 .....                             | 550 |
| 表 23.17 计数方向与编码器信号的关系 (CC1P = CC2P = 0) .....      | 558 |
| 表 23.18 计数方向与编码器信号及极性设置的关系 .....                   | 562 |
| 表 23.19 DMA 请求 .....                               | 583 |
| 表 23.20 低功耗模式对 TIM1/TIM8 的影响 .....                 | 584 |
| 表 23.21 中断请求 .....                                 | 584 |
| 表 23.22 带刹车功能的互补输出通道 tim_ocx 和 tim_ocxn 的控制位 ..... | 611 |
| 表 24.1 RX32G410 系列通用定时器 .....                      | 636 |
| 表 24.2 TIM 输入/输出引脚 .....                           | 638 |
| 表 24.3 TIM 内部输入/输出信号 .....                         | 638 |
| 表 24.4 连接到 tim_ti1 的输入多路复用器 .....                  | 639 |
| 表 24.5 连接到 tim_ti2 的输入多路复用器 .....                  | 639 |
| 表 24.6 连接到 tim_ti3 的输入多路复用器 .....                  | 639 |
| 表 24.7 连接到 tim_ti4 的输入多路复用器 .....                  | 639 |

|                                                                  |     |
|------------------------------------------------------------------|-----|
| 表 24.8 TIMx 内部触发连接.....                                          | 640 |
| 表 24.9 连接到 tim_etr 的输入多路复用器.....                                 | 640 |
| 表 24.10 连接到 ocref_clr 的输入多路复用器.....                              | 641 |
| 表 24.11 CCR 和 ARR 寄存器的变化抖动模式.....                                | 668 |
| 表 24.12 CCR 寄存器在中心对齐 PWM 模式下的变化抖动模式.....                         | 669 |
| 表 24.13 计数方向与编码器信号的关系 (CC1P = CC2P = 0) .....                    | 678 |
| 表 24.14 计数方向与编码器信号及极性设置的关系.....                                  | 681 |
| 表 24.15 DMA 请求.....                                              | 704 |
| 表 24.16 低功耗模式对 TIM2/TIM3/TIM4/TIM5 的影响.....                      | 705 |
| 表 24.17 中断请求.....                                                | 705 |
| 表 24.18 标准 tim_ocx 通道的输出控制位.....                                 | 724 |
| 表 25.1 TIM 输入/输出引脚.....                                          | 744 |
| 表 25.2 TIM 内部输入/输出信号.....                                        | 744 |
| 表 25.3 连接到 tim_ti1 的输入多路复用器.....                                 | 745 |
| 表 25.4 连接到 tim_ti2 的输入多路复用器.....                                 | 745 |
| 表 25.5 TIMx 内部触发连接.....                                          | 745 |
| 表 25.6 定时器刹车互连.....                                              | 746 |
| 表 25.7 系统中断互连.....                                               | 746 |
| 表 25.8 连接到 ocref_clr 的输入多路复用器.....                               | 746 |
| 表 25.9 CCR 和 ARR 寄存器的变化抖动模式.....                                 | 765 |
| 表 25.10 刹车保护解除条件.....                                            | 773 |
| 表 25.11 DMA 请求.....                                              | 781 |
| 表 25.12 低功耗模式对 TIM15/TIM16/TIM17 的影响.....                        | 782 |
| 表 25.13 中断请求.....                                                | 782 |
| 表 25.14 带刹车功能的互补输出通道 tim_oc1 和 tim_oc1n 的控制位 (TIM15) .....       | 797 |
| 表 25.15 带刹车功能的互补输出通道 tim_oc1 和 tim_oc1n 的控制位 (TIM16/TIM17) ..... | 820 |
| 表 26.1 TIM 内部输入/输出信号.....                                        | 833 |
| 表 26.2 TIMx_ARR 寄存器的变化抖动模式.....                                  | 843 |
| 表 26.3 DMA 请求.....                                               | 843 |
| 表 26.4 低功耗模式对 TIM6/TIM7 的影响.....                                 | 844 |
| 表 26.5 中断请求.....                                                 | 844 |
| 表 29.1 看门狗超时时间 (32kHz 的输入时钟 (LSI)) <sup>(1)</sup> .....          | 862 |
| 表 31.1 双缓冲批量端点缓冲区标识定义.....                                       | 877 |
| 表 31.2 双缓冲批量端点的缓冲区使用标识.....                                      | 877 |
| 表 31.3 同步端点的缓冲区使用标识.....                                         | 878 |
| 表 31.4 唤醒事件检测.....                                               | 880 |
| 表 31.5 接收状态编码.....                                               | 887 |
| 表 31.6 端点类型编码.....                                               | 887 |
| 表 31.7 端点特殊类型定义.....                                             | 887 |
| 表 31.8 发送状态编码.....                                               | 888 |
| 表 31.9 分组缓冲区大小的定义.....                                           | 890 |
| 表 32.1 发送邮箱寄存器列表.....                                            | 905 |
| 表 32.2 接收邮箱寄存器列表.....                                            | 905 |

|                                                |     |
|------------------------------------------------|-----|
| 表 33.1 SPI 中断请求 .....                          | 944 |
| 表 34.1 SMBus 与 I <sup>2</sup> C 的比较 .....      | 959 |
| 表 34.2 I <sup>2</sup> C 中断请求表: .....           | 962 |
| 表 35.1 检测噪声的数据采样 .....                         | 983 |
| 表 35.2 设置波特率时的误差计算 .....                       | 984 |
| 表 35.3 当 DIV_Fraction=0 时, UART 接收器的容忍度 .....  | 985 |
| 表 35.4 当 DIV_Fraction!=0 时, UART 接收器的容忍度 ..... | 985 |
| 表 35.5 帧格式 .....                               | 986 |
| 表 35.6 UART 中断请求 .....                         | 989 |
| 表 35.7 UART 模式设置 <sup>(1)</sup> .....          | 990 |
| 表 36.1 版本历史 .....                              | 998 |

## 图目录

|                                      |     |
|--------------------------------------|-----|
| 图 3.1 存储器映像 .....                    | 43  |
| 图 5.1 CRC 计算单元框图 .....               | 59  |
| 图 6.1 电源框图 .....                     | 61  |
| 图 6.2 上电复位和掉电复位波形图 .....             | 62  |
| 图 6.3 PVD 门限 .....                   | 62  |
| 图 8.1 复位电路框图 .....                   | 79  |
| 图 8.2 时钟树 .....                      | 80  |
| 图 9.1 I/O 端口位基本结构 .....              | 110 |
| 图 9.2 输入浮空/上拉/下拉结构 .....             | 113 |
| 图 9.3 输出框图 .....                     | 114 |
| 图 9.4 复用功能框图 .....                   | 114 |
| 图 9.5 模拟引脚框图 .....                   | 115 |
| 图 12.1 可配置的中断/事件框图 .....             | 130 |
| 图 12.2 外部中断/事件 GPIO 映射 .....         | 132 |
| 图 13.1 可配置逻辑单元 .....                 | 137 |
| 图 13.2 CLC 输入源选择 .....               | 138 |
| 图 13.3 逻辑功能组合选项 .....                | 139 |
| 图 14.1 三角函数的 CODRIC 收敛性 .....        | 152 |
| 图 14.2 双曲函数的 CODRIC 收敛性 .....        | 153 |
| 图 14.3 平方根的 CODRIC 收敛性 .....         | 154 |
| 图 15.1 滤波加速模块框图 .....                | 162 |
| 图 15.2 输入缓存区域 .....                  | 164 |
| 图 15.3 循环输入 buffer .....             | 165 |
| 图 15.4 循环输入 buffer 的读写操作 .....       | 166 |
| 图 15.5 循环输出 buffer .....             | 167 |
| 图 15.6 循环输出 buffer 读写操作 .....        | 168 |
| 图 16.1 ADC1 框图 .....                 | 175 |
| 图 16.2 ADC2 框图 .....                 | 175 |
| 图 16.3 ADC3 框图 .....                 | 176 |
| 图 16.4 ADC 转换时序图 .....               | 179 |
| 图 16.5 模拟看门狗警戒区 .....                | 180 |
| 图 16.6 注入转换延迟 .....                  | 181 |
| 图 16.7 校准时序图 .....                   | 182 |
| 图 16.8 数据右对齐 .....                   | 183 |
| 图 16.9 数据左对齐 .....                   | 183 |
| 图 16.10 双 ADC 框图 .....               | 187 |
| 图 16.11 4 个通道的同步注入模式 .....           | 188 |
| 图 16.12 16 个通道的同步规则模式 .....          | 188 |
| 图 16.13 在 1 个通道上连续转换模式下的快速交叉模式 ..... | 189 |
| 图 16.14 在 1 个通道上的慢速交叉模式 .....        | 190 |
| 图 16.15 交替触发：每个 ADC1 的注入通道组 .....    | 190 |

|                                                            |     |
|------------------------------------------------------------|-----|
| 图 16.16 交替触发：在间断模式下每个 ADC 上的 4 个注入通道 .....                 | 191 |
| 图 16.17 交替+规则同步 .....                                      | 191 |
| 图 16.18 触发事件发生在注入转换期间 .....                                | 192 |
| 图 16.19 交叉的单通道转换被注入序列 CH11 和 CH12 中断 .....                 | 192 |
| 图 16.20 温度传感器和 VBG 通道框图 .....                              | 193 |
| 图 17.1 DAC 框图 .....                                        | 205 |
| 图 17.2 单 DAC 通道模式下的数据寄存器 .....                             | 210 |
| 图 17.3 双 DAC 通道模式下的数据寄存器 .....                             | 210 |
| 图 17.4 触发器禁用时转换时序 .....                                    | 211 |
| 图 17.5 DAC LFSR 寄存器计算算法 .....                              | 213 |
| 图 17.6 DAC 转换（SW 触发使能）与 LFSR 波产生 .....                     | 213 |
| 图 17.7 DAC 三角波产生 .....                                     | 214 |
| 图 17.8 DAC 转换（SW 触发使能）与三角波产生 .....                         | 214 |
| 图 17.9 DAC 锯齿波产生（STDIRx = 0） .....                         | 215 |
| 图 17.10 DAC 锯齿波产生（STDIRx = 1） .....                        | 215 |
| 图 17.11 DAC 锯齿形 STINCTRIG 和 STRSTTRIG 优先级 .....            | 216 |
| 图 18.1 DMA 框图 .....                                        | 235 |
| 图 19.1 DMAMUX 框图 .....                                     | 246 |
| 图 19.2 DMAMUX 请求多路复用通道的同步模式 .....                          | 248 |
| 图 19.3 DMA 请求线路多路复用器通道的时间生成 .....                          | 249 |
| 图 20.1 COMP 框图 .....                                       | 256 |
| 图 20.2 比较器迟滞 .....                                         | 259 |
| 图 20.3 比较器输出遮蔽图 .....                                      | 260 |
| 图 21.1 单机模式:外部增益设置模式 .....                                 | 267 |
| 图 21.2 从动件的配置 .....                                        | 268 |
| 图 21.3 PGA 模式，内部增益设置(x4/x8/x12/x16/x24/x32) .....          | 269 |
| 图 21.4 PGA 模式，内部增益设置(x4/x8/x12/x16/x24/x32)，反相输入用于滤波 ..... | 270 |
| 图 22.1 高分辨率定时器框图 .....                                     | 279 |
| 图 22.2 计数器和捕获寄存器格式与时钟预分频系数 .....                           | 285 |
| 图 22.3 定时器 A.F 框图 .....                                    | 287 |
| 图 22.4 连续定时器工作模式 .....                                     | 289 |
| 图 22.5 单发定时器工作模式 .....                                     | 289 |
| 图 22.6 定时器复位重新同步（预分频比大于 32） .....                          | 291 |
| 图 22.7 连续模式下的重复率与 HRTIM_REPxR 内容 .....                     | 292 |
| 图 22.8 单发模式下的重复计数器行为 .....                                 | 292 |
| 图 22.9 比较事件对输出的操作：发生比较 1 时置位，发生比较 2 时复位 .....              | 294 |
| 图 22.10 定时器 A 定时单元捕获电路 .....                               | 298 |
| 图 22.11 自动延迟概述（仅限比较 2 寄存器） .....                           | 299 |
| 图 22.12 自动延迟比较 .....                                       | 300 |
| 图 22.13 半触发模式样例 .....                                      | 302 |
| 图 22.14 推挽模式框图 .....                                       | 303 |
| 图 22.15 推挽模式样例 .....                                       | 304 |
| 图 22.16 带死区的推挽 .....                                       | 305 |



|                                                           |     |
|-----------------------------------------------------------|-----|
| 图 22.17 已插入死区的互补输出 .....                                  | 306 |
| 图 22.18 死区插入与死区符号 (1 表示负死区) .....                         | 306 |
| 图 22.19 低脉冲宽度的互补输出 ( $SDTRx = SDTFx = 0$ ) .....          | 307 |
| 图 22.20 低脉冲宽度的互补输出 ( $SDTRx = SDTFx = 1$ ) .....          | 307 |
| 图 22.21 低脉冲宽度的互补输出 ( $SDTRx = 0, SDTFx = 1$ ) .....       | 308 |
| 图 22.22 低脉冲宽度的互补输出 ( $SDTRx = 1, SDTFx = 0$ ) .....       | 308 |
| 图 22.23 主定时器概览 .....                                      | 309 |
| 图 22.24 在上下计数模式时的基本对称波形 .....                             | 310 |
| 图 22.25 在上下计数模式时的复杂对称波形 .....                             | 311 |
| 图 22.26 在上下计数模式时的基本对称波形 .....                             | 311 |
| 图 22.27 在上下计数模式时的外部事件管理 .....                             | 312 |
| 图 22.28 纵横的上下计数操作 .....                                   | 312 |
| 图 22.29 交叉的上下计数操作 .....                                   | 313 |
| 图 22.30 推挽上下模式样例 .....                                    | 313 |
| 图 22.31 带“大于”比较功能的上下计数模式 .....                            | 314 |
| 图 22.32 输出在周期事件上设置的上下模式, $OUTROM[1:0]=10$ .....           | 315 |
| 图 22.33 在上下计数模式时重复计数器的表现 .....                            | 316 |
| 图 22.34 窄脉冲生成的短距离置位/复位管理 .....                            | 318 |
| 图 22.35 外部事件调节概述(代表 1 个通道) .....                          | 321 |
| 图 22.36 外部事件下降沿的延迟 (计数器复位和输出置位) .....                     | 324 |
| 图 22.37 外部事件的延迟 (发生外部事件时, 输出复位) .....                     | 324 |
| 图 22.38 事件消隐模式 .....                                      | 325 |
| 图 22.39 事件延迟模式 .....                                      | 325 |
| 图 22.40 采用边沿有效触发的外部触发消隐 .....                             | 327 |
| 图 22.41 外部触发消隐, 电平有效触发 .....                              | 327 |
| 图 22.42 事件窗口模式 .....                                      | 328 |
| 图 22.43 采用边沿有效触发的外部触发窗口 .....                             | 329 |
| 图 22.44 外部触发窗口, 电平有效触发 .....                              | 329 |
| 图 22.45 外部触事件计数器 - 通道 A .....                             | 330 |
| 图 22.46 外部事件计数器累积模式 ( $EEVxRSTM = 1, EEVxCNT = 2$ ) ..... | 331 |
| 图 22.47 延迟空闲模式进入 .....                                    | 332 |
| 图 22.48 突发模式和延迟保护优先级 ( $DIDL = 0$ ) .....                 | 333 |
| 图 22.49 突发模式和延迟保护优先级 ( $DIDL = 1$ ) .....                 | 334 |
| 图 22.50 均衡空闲保护示例 .....                                    | 335 |
| 图 22.51 重同步定时器更新 (在 $HRTIM\_TIMBCR$ 中 $TAU=1$ ) .....     | 340 |
| 图 22.52 在“大于”PWM 模式下提前开启和提前关闭的行为 .....                    | 342 |
| 图 22.53 输出管理概览 .....                                      | 347 |
| 图 22.54 HRTIM 输出状态和转换 .....                               | 348 |
| 图 22.55 突发模式工作示例 .....                                    | 349 |
| 图 22.56 发生外部事件时触发突发模式 .....                               | 352 |
| 图 22.57 使能死区且 $IDLESx = 1$ 时的延迟突发模式进入 .....               | 353 |
| 图 22.58 死区内的延迟突发模式进入 .....                                | 354 |
| 图 22.59 死区发生器使能时的突发模式退出 .....                             | 355 |

|                                                                                              |     |
|----------------------------------------------------------------------------------------------|-----|
| 图 22.60 突发模式仿真示例 .....                                                                       | 357 |
| 图 22.61 载波频率信号插入 .....                                                                       | 357 |
| 图 22.62 已使能斩波模式的 HRTIM 输出 .....                                                              | 358 |
| 图 22.63 故障保护电路 (完全表示了 FAULT1, 部分显示了 FAULT2.6) .....                                          | 359 |
| 图 22.64 故障信号过滤 (FLTxF[3:0]= 0010: $f_{\text{SAMPLING}} = f_{\text{HRTIM}}$ , $N = 4$ ) ..... | 360 |
| 图 22.65 故障计数器累加模式 (FLTxRSTM = 1, FLTxCNT[3:0] = 2) .....                                     | 362 |
| 图 22.66 辅助输出 .....                                                                           | 364 |
| 图 22.67 突发模式期间的辅助输出和主输出 (DIDLx = 0) .....                                                    | 365 |
| 图 22.68 退出突发模式时辅助输出上的死区失真 .....                                                              | 365 |
| 图 22.69 同步启动模式下计数器的行为 .....                                                                  | 369 |
| 图 22.70 ADC 触发选择概述 .....                                                                     | 370 |
| 图 22.71 ADC 触发 .....                                                                         | 371 |
| 图 22.72 向上计数模式下的 ADC 触发后缩放 .....                                                             | 372 |
| 图 22.73 向上/向下计数模式下的 ADC 触发后缩放 .....                                                          | 372 |
| 图 22.74 将多个更新合并到一路 hrtim_dac_trgx 输出 .....                                                   | 373 |
| 图 22.75 DAC 斜率补偿触发 .....                                                                     | 376 |
| 图 22.76 DAC 触发器概述 .....                                                                      | 377 |
| 图 22.77 DMA 突发概览 .....                                                                       | 381 |
| 图 22.78 突发 DMA 工作流程图 .....                                                                   | 382 |
| 图 22.79 DMA 突发传输后进行寄存器更新 .....                                                               | 383 |
| 图 22.80 降压转换器拓扑结构 .....                                                                      | 385 |
| 图 22.81 双降压转换器管理 .....                                                                       | 386 |
| 图 22.82 同步整流取决于输出电流 .....                                                                    | 386 |
| 图 22.83 采用同步整流功能实现降压 .....                                                                   | 387 |
| 图 22.84 三相交错降压转换器 .....                                                                      | 387 |
| 图 22.85 三相交错降压转换器控制 .....                                                                    | 388 |
| 图 22.86 过渡模式功率因数校正 (PFC) .....                                                               | 389 |
| 图 22.87 过渡模式功率因数校正 (PFC) 波形 .....                                                            | 390 |
| 图 23.1 高级控制定时器框图 .....                                                                       | 503 |
| 图 23.2 当预分频器的参数从 1 变成 2 时, 计数器的时序图 .....                                                     | 510 |
| 图 23.3 当预分频器的参数从 1 变成 4 时, 计数器的时序图 .....                                                     | 510 |
| 图 23.4 计数器时序图, 内部时钟分频系数为 1 .....                                                             | 511 |
| 图 23.5 计数器时序图, 内部时钟分频系数为 2 .....                                                             | 512 |
| 图 23.6 计数器时序图, 内部时钟分频系数为 4 .....                                                             | 512 |
| 图 23.7 计数器时序图, 内部时钟分频系数为 N .....                                                             | 513 |
| 图 23.8 计数器时序图, 当 ARPE=0 时的更新事件 (TIMx_ARR 没有预装入) .....                                        | 513 |
| 图 23.9 计数器时序图, 当 ARPE=1 时的更新事件 (预装入了 TIMx_ARR) .....                                         | 514 |
| 图 23.10 计数器时序图, 内部时钟分频系数为 1 .....                                                            | 515 |
| 图 23.11 计数器时序图, 内部时钟分频系数为 2 .....                                                            | 516 |
| 图 23.12 计数器时序图, 内部时钟分频系数为 4 .....                                                            | 516 |
| 图 23.13 计数器时序图, 内部时钟分频系数为 N .....                                                            | 516 |
| 图 23.14 计数器时序图, 当没有使用重复计数器时的更新事件 .....                                                       | 517 |
| 图 23.15 计数器时序图, 内部时钟分频系数为 1, TIMx_ARR=0x6 .....                                              | 518 |

|                                                     |     |
|-----------------------------------------------------|-----|
| 图 23.16 计数器时序图, 内部时钟分频系数为 2                         | 519 |
| 图 23.17 计数器时序图, 内部时钟分频系数为 4, TIMx_ARR=0x36          | 519 |
| 图 23.18 计数器时序图, 内部时钟分频系数为 N                         | 520 |
| 图 23.19 计数器时序图, ARPE=1 时的更新事件 (计数器下溢)               | 520 |
| 图 23.20 计数器时序图, ARPE=1 时的更新事件 (计数器上溢)               | 521 |
| 图 23.21 不同模式下更新速率的例子, 及 TIMx_RCR 的寄存器设置             | 522 |
| 图 23.22 外部触发输入框图                                    | 523 |
| 图 23.23 一般模式下的控制电路, 内部时钟分频系数为 1                     | 523 |
| 图 23.24 外部触发输入框图                                    | 524 |
| 图 23.25 外部时钟模式 1 下的控制电路                             | 525 |
| 图 23.26 外部触发输入框图                                    | 525 |
| 图 23.27 外部时钟模式 2 下的控制电路                             | 526 |
| 图 23.28 捕获/比较通道 (如: 通道 1 输入部分)                      | 527 |
| 图 23.29 捕获/比较通道 1 的主电路                              | 527 |
| 图 23.30 捕获/比较通道的输出阶段 (通道 1, 通道 2、3 和 4 同上)          | 528 |
| 图 23.31 捕获/比较通道的输出阶段 (通道 5, 通道 6 同上)                | 528 |
| 图 23.32 PWM 输入模式时序                                  | 530 |
| 图 23.33 输出比较模式, 翻转 OC1                              | 531 |
| 图 23.34 边沿对齐的 PWM 波形 (ARR=8)                        | 533 |
| 图 23.35 中央对齐的 PWM 波形 (ARR=8)                        | 534 |
| 图 23.36 抖动原理                                        | 535 |
| 图 23.37 抖动模式下的数据格式和寄存器编码                            | 536 |
| 图 23.38 PWM 分辨率 vs 频率                               | 537 |
| 图 23.39 PWM 抖动模式                                    | 538 |
| 图 23.40 中心对齐 PWM 模式下抖动对占空比的影响                       | 539 |
| 图 23.41 生成两个具有 50% 占空比的相位偏移 PWM 信号                  | 540 |
| 图 23.42 通道 1 和通道 3 的组合 PWM 模式                       | 541 |
| 图 23.43 每周具有多个触发脉冲的三相组合 PWM 信号                      | 542 |
| 图 23.44 带死区插入的互补输出                                  | 543 |
| 图 23.45 非对称死区时间                                     | 544 |
| 图 23.46 死区波形延迟大于负脉冲                                 | 544 |
| 图 23.47 死区波形延迟大于正脉冲                                 | 545 |
| 图 23.48 刹车和刹车 2 电路概述                                | 546 |
| 图 23.49 tim_brk 在发生刹车事件时的各种输出行为 (OSSR=1)            | 548 |
| 图 23.50 当 tim_brk 和 tim_brk2 有效 (OSSR=1) 时 PWM 输出状态 | 549 |
| 图 23.51 当 tim_brk 有效 (OSSR=0) 时 PWM 输出状态            | 549 |
| 图 23.52 输出重定向 (未表示 tim_brk2 请求)                     | 551 |
| 图 23.53 tim_ocref_clr 输入选择复用器                       | 551 |
| 图 23.54 清除 TIMx 的 tim_ocxref                        | 552 |
| 图 23.55 COM 事件生成 6 步 PWM 的示例 (OSSR=1)               | 553 |
| 图 23.56 单脉冲模式的例子                                    | 554 |
| 图 23.57 可重触发单脉冲模式                                   | 555 |
| 图 23.58 脉冲发生器电路                                     | 556 |

|                                                      |     |
|------------------------------------------------------|-----|
| 图 23.59 在比较事件上的脉冲生成，适用于边沿对齐模式 .....                  | 556 |
| 图 23.60 在同时触发的情况下延长脉冲宽度 .....                        | 557 |
| 图 23.61 编码器接口模式下的计数器工作示例 .....                       | 559 |
| 图 23.62 tim_tifp1 极性反相时的编码器接口模式示例 .....              | 559 |
| 图 23.63 正交编码器的计数模式 .....                             | 560 |
| 图 23.64 方向加时钟编码器模式 .....                             | 561 |
| 图 23.65 方向时钟编码器模式 (CC1P=CC2P=0) .....                | 561 |
| 图 23.66 方向时钟编码器模式 (CC1P=CC2P=1) .....                | 562 |
| 图 23.67 索引门控选项 .....                                 | 563 |
| 图 23.68 抖动的索引信号 .....                                | 563 |
| 图 23.69 IPOS[1:0] = 11 时的索引生成 .....                  | 564 |
| 图 23.70 通道 A 上带索引门控的计数器读取 (IPOS[1:0] = 11) .....     | 565 |
| 图 23.71 非门控模式下索引的计数器读取 (IPOS[1:0] = 00) .....        | 565 |
| 图 23.72 通道 A 和 B 上带索引门控的计数器读取 .....                  | 566 |
| 图 23.73 窄索引脉冲情况下的编码器模式行为 (IPOS[1:0] = 11) .....      | 567 |
| 图 23.74 计数器重置窄索引脉冲 (更近距离, ARR = 0x07) .....          | 568 |
| 图 23.75 x1 和 x2 模式下的索引行为 (IPOS[1:0] = 01) .....      | 569 |
| 图 23.76 定向索引灵敏度 .....                                | 570 |
| 图 23.77 作为 FIDX 位设置的功能的计数器重置 .....                   | 570 |
| 图 23.78 时钟+方向模式下的索引行为, IPOS[0] = 1.....              | 571 |
| 图 23.79 方向时钟模式下的索引行为, IPOS[0] = 1.....               | 571 |
| 图 23.80 正交编码信号的状态图 .....                             | 572 |
| 图 23.81 递增计数编码器错误检测 .....                            | 573 |
| 图 23.82 递减计数编码器错误检测 .....                            | 574 |
| 图 23.83 更新时带有预加载转移的编码器模式更改 (SMSPS = 0) .....         | 575 |
| 图 23.84 测量 3 个信号边缘之间的时间间隔 .....                      | 576 |
| 图 23.85 霍尔传感器接口的实例 .....                             | 578 |
| 图 23.86 复位模式下的控制电路 .....                             | 579 |
| 图 23.87 门控模式下的控制电路 .....                             | 580 |
| 图 23.88 触发器模式下的控制电路 .....                            | 580 |
| 图 23.89 外部时钟模式 2+触发模式下的控制电路 .....                    | 582 |
| 图 24.1 通用定时器框图 .....                                 | 637 |
| 图 24.2 当预分频器的参数从 1 变到 2 时，计数器的时序图 .....              | 642 |
| 图 24.3 当预分频器的参数从 1 变到 4 时，计数器的时序图 .....              | 642 |
| 图 24.4 计数器时序图，内部时钟分频系数为 1.....                       | 643 |
| 图 24.5 计数器时序图，内部时钟分频系数为 2.....                       | 644 |
| 图 24.6 计数器时序图，内部时钟分频系数为 4 .....                      | 644 |
| 图 24.7 计数器时序图，内部时钟分频系数为 N.....                       | 645 |
| 图 24.8 计数器时序图，当 ARPE=0 时的更新事件 (TIMx_ARR 没有预装入) ..... | 645 |
| 图 24.9 计数器时序图，当 ARPE=1 时的更新事件 (预装入了 TIMx_ARR) .....  | 646 |
| 图 24.10 计数器时序图，内部时钟分频系数为 1.....                      | 647 |
| 图 24.11 计数器时序图，内部时钟分频系数为 2.....                      | 647 |
| 图 24.12 计数器时序图，内部时钟分频系数为 4 .....                     | 648 |



|                                            |     |
|--------------------------------------------|-----|
| 图 24.13 计数器时序图, 内部时钟分频系数为 N                | 648 |
| 图 24.14 计数器时序图, 更新事件                       | 649 |
| 图 24.15 计数器时序图, 内部时钟分频系数为 1, TIMx_ARR=0x6  | 650 |
| 图 24.16 计数器时序图, 内部时钟分频系数为 2                | 651 |
| 图 24.17 计数器时序图, 内部时钟分频系数为 4, TIMx_ARR=0x36 | 651 |
| 图 24.18 计数器时序图, 内部时钟分频系数为 N                | 652 |
| 图 24.19 计数器时序图, ARPE=1 时的更新事件 (计数器下溢)      | 653 |
| 图 24.20 计数器时序图, ARPE=1 时的更新事件 (计数器溢出)      | 653 |
| 图 24.21 一般模式下的控制电路, 内部时钟分频系数为 1            | 654 |
| 图 24.22 tim_ti2 外部时钟连接例子                   | 654 |
| 图 24.23 外部时钟模式 1 下的控制电路                    | 655 |
| 图 24.24 外部触发输入框图                           | 655 |
| 图 24.25 外部时钟模式 2 下的控制电路                    | 656 |
| 图 24.26 捕获/比较通道 (如: 通道 1 输入部分)             | 657 |
| 图 24.27 捕获/比较通道 1 的主电路                     | 657 |
| 图 24.28 捕获/比较通道的输出阶段 (通道 1, 通道 2、3 和 4 同上) | 658 |
| 图 24.29 PWM 输入模式时序                         | 660 |
| 图 24.30 输出比较模式, 翻转 tim_oc1                 | 661 |
| 图 24.31 边沿对齐的 PWM 波形 (ARR=8)               | 663 |
| 图 24.32 中央对齐的 PWM 波形 (APR=8)               | 664 |
| 图 24.33 抖动原理                               | 665 |
| 图 24.34 抖动模式下的数据格式和寄存器编码                   | 666 |
| 图 24.35 PWM 分辨率 vs 频率 (16 位模式)             | 667 |
| 图 24.36 PWM 分辨率 vs 频率 (32 位模式)             | 667 |
| 图 24.37 PWM 抖动模式                           | 668 |
| 图 24.38 中心对齐 PWM 模式下抖动对占空比的影响              | 669 |
| 图 24.39 生成两个具有 50% 占空比的相位偏移 PWM 信号         | 670 |
| 图 24.40 通道 1 和通道 3 的组合 PWM 模式              | 671 |
| 图 24.41 tim_ocref_clr 输入选择复用器              | 672 |
| 图 24.42 清除 TIMx 的 tim_ocxref               | 672 |
| 图 24.43 单脉冲模式的例子                           | 673 |
| 图 24.44 可重触发单脉冲模式                          | 675 |
| 图 24.45 脉冲发生器电路                            | 675 |
| 图 24.46 在比较事件上的脉冲生成, 适用于边沿对齐模式和编码器模式       | 676 |
| 图 24.47 在同时触发的情况下延长脉冲宽度                    | 677 |
| 图 24.48 编码器接口模式下的计数器工作示例                   | 678 |
| 图 24.49 tim_ti1fp1 极性反相时的编码器接口模式示例         | 679 |
| 图 24.50 正交编码器的计数模式                         | 679 |
| 图 24.51 方向加时钟编码器模式                         | 680 |
| 图 24.52 方向时钟编码器模式 (CC1P=CC2P=0)            | 681 |
| 图 24.53 方向时钟编码器模式 (CC1P=CC2P=1)            | 681 |
| 图 24.54 索引门控选项                             | 682 |
| 图 24.55 抖动的索引信号                            | 683 |

|                                                             |     |
|-------------------------------------------------------------|-----|
| 图 24.56 IPOS[1:0] = 11 时的索引生成 .....                         | 684 |
| 图 24.57 通道 A 上带索引门控的计数器读取 (IPOS[1:0] = 11) .....            | 684 |
| 图 24.58 非门控模式下索引的计数器读取 (IPOS[1:0] = 00) .....               | 685 |
| 图 24.59 通道 A 和 B 上带索引门控的计数器读取 .....                         | 685 |
| 图 24.60 窄索引脉冲情况下的编码器模式行为 (IPOS[1:0] = 11) .....             | 686 |
| 图 24.61 计数器重置窄索引脉冲 (更近距离, ARR = 0x07) .....                 | 687 |
| 图 24.62 x1 和 x2 模式下的索引行为 (IPOS[1:0] = 01) .....             | 688 |
| 图 24.63 定向索引灵敏度 .....                                       | 689 |
| 图 24.64 作为 FIDX 位设置的功能的计数器重置 .....                          | 689 |
| 图 24.65 时钟+方向模式下的索引行为, IPOS[0] = 1 .....                    | 690 |
| 图 24.66 方向时钟模式下的索引行为, IPOS[0] = 1 .....                     | 690 |
| 图 24.67 正交编码信号的状态图 .....                                    | 691 |
| 图 24.68 递增计数编码器错误检测 .....                                   | 692 |
| 图 24.69 递减计数编码器错误检测 .....                                   | 693 |
| 图 24.70 更新时带有预加载转移的编码器模式更改 (SMSPS = 0) .....                | 694 |
| 图 24.71 复位模式下的控制电路 .....                                    | 696 |
| 图 24.72 门控模式下的控制电路 .....                                    | 696 |
| 图 24.73 触发器模式下的控制电路 .....                                   | 697 |
| 图 24.74 外部时钟模式 2+触发模式下的控制电路 .....                           | 698 |
| 图 24.75 主/从定时器的例子 .....                                     | 699 |
| 图 24.76 仅定时器通道 1 主/从连接示例 .....                              | 699 |
| 图 24.77 TIM_mstr 的 tim_oc1ref 控制 TIM_slv .....              | 700 |
| 图 24.78 通过使能 TIM_mstr 可以控制 TIM_slv .....                    | 701 |
| 图 24.79 使用 TIM_mstr 的更新触发 TIM_slv .....                     | 702 |
| 图 24.80 利用 TIM_mstr 的使能触发 TIM_slv .....                     | 702 |
| 图 24.81 使用 TIM_mstr 的 tim_ti1 输入触发 TIM_mstr 和 TIM_slv ..... | 703 |
| 图 25.1 TIM15 框图 .....                                       | 742 |
| 图 25.2 TIM16/TIM17 框图 .....                                 | 743 |
| 图 25.3 当预分频器的参数从 1 变到 2 时, 计数器的时序图 .....                    | 748 |
| 图 25.4 当预分频器的参数从 1 变到 4 时, 计数器的时序图 .....                    | 748 |
| 图 25.5 计数器时序图, 内部时钟分频系数为 1 .....                            | 749 |
| 图 25.6 计数器时序图, 内部时钟分频系数为 2 .....                            | 750 |
| 图 25.7 计数器时序图, 内部时钟分频系数为 4 .....                            | 750 |
| 图 25.8 计数器时序图, 内部时钟分频系数为 N .....                            | 751 |
| 图 25.9 计数器时序图, 当 ARPE=0 时的更新事件 (TIMx_ARR 没有预装入) .....       | 751 |
| 图 25.10 计数器时序图, 当 ARPE=1 时的更新事件 (预装入了 TIMx_ARR) .....       | 752 |
| 图 25.11 取决于模式和 TIMx_RCR 寄存器设置的更新频率示例 .....                  | 753 |
| 图 25.12 一般模式下的控制电路, 内部时钟分频系数为 1 .....                       | 754 |
| 图 25.13 tim_ti2 外部时钟连接例子 .....                              | 754 |
| 图 25.14 外部时钟模式 1 下的控制电路 .....                               | 755 |
| 图 25.15 捕获/比较通道 (如: 通道 1 输入部分) .....                        | 755 |
| 图 25.16 捕获/比较通道 1 的主电路 .....                                | 756 |
| 图 25.17 捕获/比较通道的输出阶段 (通道 1) .....                           | 756 |



|                                                 |     |
|-------------------------------------------------|-----|
| 图 25.18 捕获比较通道的输出阶段 (TIM15 的通道 2)               | 757 |
| 图 25.19 PWM 输入模式时序                              | 759 |
| 图 25.20 输出比较模式, 翻转 tim_oc1                      | 760 |
| 图 25.21 边沿对齐的 PWM 波形 (ARR=8)                    | 761 |
| 图 25.22 抖动原理                                    | 762 |
| 图 25.23 抖动模式下的数据格式和寄存器编码                        | 763 |
| 图 25.24 PWM 分辨率 vs 频率                           | 764 |
| 图 25.25 PWM 抖动模式                                | 764 |
| 图 25.26 通道 1 和通道 2 的组合 PWM 模式                   | 766 |
| 图 25.27 带死区插入的互补输出                              | 767 |
| 图 25.28 非对称死区时间                                 | 768 |
| 图 25.29 死区波形延迟大于负脉冲                             | 768 |
| 图 25.30 死区波形延迟大于正脉冲                             | 768 |
| 图 25.31 刹车电路概述                                  | 770 |
| 图 25.32 tim_brk 在发生刹车事件时的各种输出行为 (OSSR=1)        | 772 |
| 图 25.33 输出重定向                                   | 774 |
| 图 25.34 tim_ocref_clr 输入选择复用器                   | 774 |
| 图 25.35 单脉冲模式的例子                                | 775 |
| 图 25.36 可重触发单脉冲模式                               | 776 |
| 图 25.37 测量两个信号边沿之间的间隔时间                         | 777 |
| 图 25.38 复位模式下的控制电路                              | 778 |
| 图 25.39 门控模式下的控制电路                              | 779 |
| 图 25.40 触发器模式下的控制电路                             | 779 |
| 图 26.1 基本定时器框图                                  | 832 |
| 图 26.2 一般模式下的控制电路, 内部时钟分频系数为 1                  | 833 |
| 图 26.3 当预分频器的参数从 1 变到 2 时, 计数器的时序图              | 834 |
| 图 26.4 当预分频器的参数从 1 变到 4 时, 计数器的时序图              | 835 |
| 图 26.5 计数器时序图, 内部时钟分频系数为 1                      | 836 |
| 图 26.6 计数器时序图, 内部时钟分频系数为 2                      | 836 |
| 图 26.7 计数器时序图, 内部时钟分频系数为 4                      | 837 |
| 图 26.8 计数器时序图, 内部时钟分频系数为 N                      | 837 |
| 图 26.9 计数器时序图, 当 ARPE=0 时的更新事件 (TIMx_ARR 没有预装入) | 838 |
| 图 26.10 计数器时序图, 当 ARPE=1 时的更新事件 (预装入了 TIMx_ARR) | 839 |
| 图 26.11 抖动原理                                    | 840 |
| 图 26.12 抖动模式下的数据格式和寄存器编码                        | 841 |
| 图 26.13 PWM 分辨率 vs 频率                           | 842 |
| 图 26.14 PWM 抖动模式                                | 842 |
| 图 28.1 简化的 RTC 框图                               | 854 |
| 图 28.2 RTC 秒和闹钟波形图示例, PR=0003, ALARM=00004      | 855 |
| 图 28.3 RTC 溢出波形图示例, PR=0003                     | 856 |
| 图 29.1 独立看门狗框图                                  | 862 |
| 图 30.1 窗口看门狗框图                                  | 865 |
| 图 30.2 窗口看门狗时序图                                 | 867 |

|                                                                             |     |
|-----------------------------------------------------------------------------|-----|
| 图 31.1 USB 框图 .....                                                         | 871 |
| 图 31.2 分组缓冲区对应的缓冲区描述表项定位 .....                                              | 874 |
| 图 32.1 CAN 网拓扑结构 .....                                                      | 892 |
| 图 32.2 CAN 框图 .....                                                         | 893 |
| 图 32.3 bxCAN 工作模式 .....                                                     | 895 |
| 图 32.4 bxCAN 工作在静默模式 .....                                                  | 896 |
| 图 32.5 bxCAN 工作在环回模式 .....                                                  | 896 |
| 图 32.6 bxCAN 工作在环回静默模式 .....                                                | 897 |
| 图 32.7 发送邮箱状态 .....                                                         | 899 |
| 图 32.8 接收 FIFO 状态 .....                                                     | 900 |
| 图 32.9 过滤器组位宽设置—寄存器组织 .....                                                 | 902 |
| 图 32.10 过滤器编号的例子 .....                                                      | 903 |
| 图 32.11 过滤器机制的例子 .....                                                      | 904 |
| 图 32.12 CAN 错误状态图 .....                                                     | 906 |
| 图 32.13 位时序 .....                                                           | 907 |
| 图 32.14 各种 CAN .....                                                        | 908 |
| 图 32.15 事件标志和中断产生 .....                                                     | 909 |
| 图 32.16 RX 和 TX 邮箱 .....                                                    | 920 |
| 图 33.1 SPI 框图 .....                                                         | 928 |
| 图 33.2 单主和单从应用 .....                                                        | 929 |
| 图 33.3 硬件/软件的从选择管理 .....                                                    | 930 |
| 图 33.4 数据时钟时序图 .....                                                        | 931 |
| 图 33.5 主模式、全双工模式下 (BIDIMODE=0 并且 RXONLY=0) 连续传输时, TXE/RXNE/BSY 的变化示意图 ..... | 936 |
| 图 33.6 从模式、全双工模式下 (BIDIMODE=0 并且 RXONLY=0) 连续传输时, TXE/RXNE/BSY 的变化示意图 ..... | 936 |
| 图 33.7 主设备只发送模式 (BIDIMODE=0 并且 RXONLY=0) 下连续传输时, TXE/BSY 变化示意图 .....        | 937 |
| 图 33.8 从设备只发送模式 (BIDIMODE=0 并且 RXONLY=0) 下连续传输时, TXE/BSY 变化示意图 .....        | 937 |
| 图 33.9 只接收模式 (BIDIMODE=0 并且 RXONLY=1) 下连续传输时, RXNE 变化示意图 .....              | 938 |
| 图 33.10 非连续传输发送 (BIDIMODE=0 并且 RXONLY=0) 时, TXE/BSY 变化示意图 .....             | 939 |
| 图 33.11 使用 DMA 发送 .....                                                     | 942 |
| 图 33.12 使用 DMA 接收 .....                                                     | 943 |
| 图 34.1 I <sup>2</sup> C 总线协议 .....                                          | 951 |
| 图 34.2 I <sup>2</sup> C 的功能框图 .....                                         | 952 |
| 图 34.3 从发送器的传送序列图 .....                                                     | 954 |
| 图 34.4 从接收器的传送序列图 .....                                                     | 955 |
| 图 34.5 主发送器传送序列图 .....                                                      | 956 |
| 图 34.6 主接收器传送序列图 .....                                                      | 957 |
| 图 34.7 I <sup>2</sup> C 中断映射图 .....                                         | 964 |
| 图 35.1 UART 框图 .....                                                        | 977 |
| 图 35.2 字长设置 .....                                                           | 978 |

|                                          |     |
|------------------------------------------|-----|
| 图 35.3 配置停止位 .....                       | 979 |
| 图 35.4 发送时 TC/TXE 的变化情况 .....            | 980 |
| 图 35.5 起始位侦测 .....                       | 981 |
| 图 35.6 检测噪声的数据采样 .....                   | 983 |
| 图 35.7 说明了断开符号检测器状态机的行为和断开符号标志的关系。 ..... | 987 |
| 图 35.8 给出了一个断开帧的例子。 .....                | 988 |
| 图 35.9 UART 中断映像图 .....                  | 990 |

## 1 简介

本参考手册用于帮助开发者理解并使用睿兴科技（南京）有限公司（后文简称“睿兴”或“RX”）的芯片。本文提供了 RX32G410 系列微控制器的存储和外设的完整信息。

相关封装和电气特性请参考相应的数据手册。

本参考手册和数据手册均可从睿兴官网 [www.rxtek-icore.com](http://www.rxtek-icore.com) 获得。

## 2 文档约定

### 2.1 寄存器缩写列表

|                           |                                              |
|---------------------------|----------------------------------------------|
| read/write (rw)           | 软件可以读取和写入这个位。                                |
| read-only (r)             | 软件只能读取这个位。                                   |
| write-only (w)            | 软件只能写入这个位。读取这个位将返回复位值。                       |
| read/clear write0 (rc_w0) | 软件可以读取这个位，并且可以通过写入 0 来清零这个位。写入 1 对这个位没有影响。   |
| read/clear write1 (rc_w1) | 软件可以读取这个位，并且可以通过写入 1 来清零这个位。写入 0 对这个位没有影响。   |
| read/set (rs)             | 软件可以读取和设置这个位。写入 0 对这个位没有影响。                  |
| read/write once (rwo)     | 软件只能写入这个位一次，但可以在任何时候读取它。只有复位操作才能将这个位返回到其复位值。 |
| Reserved (Res.)           | 保留位，必须保持在复位值。                                |

### 2.2 词汇表

Quad word: 四字, 128 位长度数据。  
Double word: 双字, 64 位长度数据。  
Word: 字, 32 位长度数据。  
Half-word: 半字, 16 位长度数据。  
Byte: 字节, 8 位长度数据。

## 3 系统和存储器概述

### 3.1 系统架构

RX32G410 系列集成了带 FPU 的 Arm®Cortex®-M4 的内核，是电机专用的高性能微控制器。

- 五个驱动单元
  - I-bus
  - D-bus
  - S-bus
  - DMA1
  - DMA2
- 四个被动单元
  - 内置 Flash
  - 内置 SRAM
  - 内置 CCM SRAM
  - AHB 外设包括 AHB 到 APB 的桥和 APB 外设（连接在 APB1 和 APB2）

### 3.2 存储器结构

#### 3.2.1 介绍

程序存储器，数据存储器，寄存器和 I/O 口统一编址在 4G 的线性地址空间里。

字节在存储器中以小端格式编码。一个字中编号最低的字节被认为是该字的最低有效字节，最高地址字节是最高有效字节。



### 3.2.2 存储器映像和寄存器边界地址

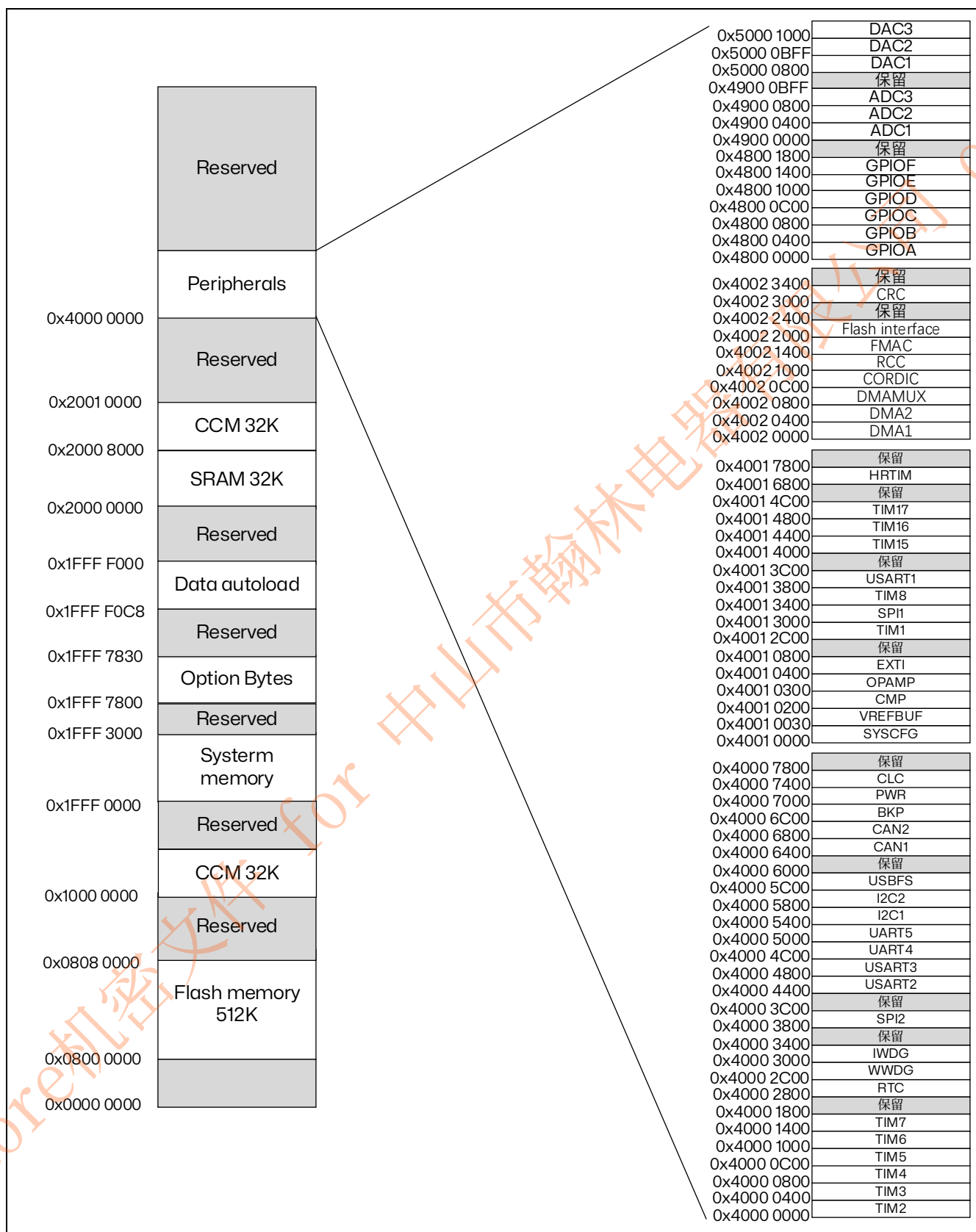


图 3.1 存储器映像

所有未分配给片上存储器和外围设备的内存映射区域都被配置为“Reserved”或者“保留”。有关可用内存和寄存器区域的详细映射，请参阅下表。

下表列出了设备中可用外设的边界地址。

表 3.1 RX32G410 系列存储器映像和外设边界地址

| 总线   | 边界地址                      | 外设              |
|------|---------------------------|-----------------|
| AHB2 | 0x5000 1000 - 0x5000 13FF | DAC3            |
|      | 0x5000 0C00 - 0x5000 0FFF | DAC2            |
|      | 0x5000 0800 - 0x5000 0BFF | DAC1            |
|      | 0x5000 0C00 - 0x4900 07FF | 保留              |
|      | 0x4900 0800 - 0x4900 0BFF | ADC3            |
|      | 0x4900 0400 - 0x4900 07FF | ADC2            |
|      | 0x4900 0000 - 0x4900 03FF | ADC1            |
|      | 0x4800 1800 - 0x48FF FFFF | 保留              |
|      | 0x4800 1400 - 0x4800 17FF | GPIOF           |
|      | 0x4800 1000 - 0x4800 13FF | GPIOE           |
|      | 0x4800 0C00 - 0x4800 0FFF | GPIOD           |
|      | 0x4800 0800 - 0x4800 0BFF | GPIOC           |
|      | 0x4800 0400 - 0x4800 07FF | GPIOB           |
|      | 0x4800 0000 - 0x4800 03FF | GPIOA           |
| AHB1 | 0x4002 3400 - 0x47FF FFFF | 保留              |
|      | 0x4002 3000 - 0x4002 33FF | CRC             |
|      | 0x4002 2400 - 0x4002 2FFF | 保留              |
|      | 0x4002 2000 - 0x4002 23FF | Flash interface |
|      | 0x4002 1400 - 0x4002 1FFF | FMAC            |
|      | 0x4002 1000 - 0x4002 13FF | RCC             |
|      | 0x4002 0C00 - 0x4002 0FFF | CORDIC          |
|      | 0x4002 0800 - 0x4002 0BFF | DMAMUX          |
|      | 0x4002 0400 - 0x4002 07FF | DMA2            |
|      | 0x4002 0000 - 0x4002 03FF | DMA1            |
| APB2 | 0x4001 7800 - 0x4001 FFFF | 保留              |
|      | 0x4001 6800 - 0x4001 77FF | HRTIM           |
|      | 0x4001 4C00 - 0x4001 67FF | 保留              |
|      | 0x4001 4800 - 0x4001 4BFF | TIM17           |
|      | 0x4001 4400 - 0x4001 47FF | TIM16           |
|      | 0x4001 4000 - 0x4001 43FF | TIM15           |
|      | 0x4001 3C00 - 0x4001 3FFF | 保留              |
|      | 0x4001 3800 - 0x4001 3BFF | USART1          |
|      | 0x4001 3400 - 0x4001 37FF | TIM8            |
|      | 0x4001 3000 - 0x4001 33FF | SPI1            |
|      | 0x4001 2C00 - 0x4001 2FFF | TIM1            |
|      | 0x4001 0800 - 0x4001 0BFF | 保留              |

| 总线   | 边界地址                      | 外设      |
|------|---------------------------|---------|
|      | 0x4001 0400 - 0x4001 07FF | EXTI    |
|      | 0x4001 0300 - 0x4001 03FF | OPAMP   |
|      | 0x4001 0200 - 0x4001 02FF | COMP    |
|      | 0x4001 0030 - 0x4001 01FF | VREFBUF |
|      | 0x4001 0000 - 0x4001 0029 | SYSCFG  |
| APB1 | 0x4000 7800 - 0x4000 FFFF | 保留      |
|      | 0x4000 7400 - 0x4000 77FF | CLC     |
|      | 0x4000 7000 - 0x4000 73FF | PWR     |
|      | 0x4000 6C00 - 0x4000 6FFF | BKP     |
|      | 0x4000 6800 - 0x4000 6BFF | CAN2    |
|      | 0x4000 6400 - 0x4000 7BFF | CAN1    |
|      | 0x4000 6000 - 0x4000 63FF | 保留      |
|      | 0x4000 5C00 - 0x4000 5FFF | USBFS   |
|      | 0x4000 5800 - 0x4000 5BFF | I2C2    |
|      | 0x4000 5400 - 0x4000 57FF | I2C1    |
|      | 0x4000 5000 - 0x4000 53FF | UART5   |
|      | 0x4000 4C00 - 0x4000 4FFF | UART4   |
|      | 0x4000 4800 - 0x4000 4BFF | USART3  |
|      | 0x4000 4400 - 0x4000 47FF | USART2  |
|      | 0x4000 3C00 - 0x4000 43FF | 保留      |
|      | 0x4000 3800 - 0x4000 3BFF | SPI2    |
|      | 0x4000 3400 - 0x4000 37FF | 保留      |
|      | 0x4000 3000 - 0x4000 33FF | IWDG    |
|      | 0x4000 2C00 - 0x4000 2FFF | WWDG    |
|      | 0x4000 2800 - 0x4000 2BFF | RTC     |
|      | 0x4000 1800 - 0x4000 27FF | 保留      |
|      | 0x4000 1400 - 0x4000 17FF | TIM7    |
|      | 0x4000 1000 - 0x4000 13FF | TIM6    |
|      | 0x4000 0C00 - 0x4000 0FFF | TIM5    |
|      | 0x4000 0800 - 0x4000 0BFF | TIM4    |
|      | 0x4000 0400 - 0x4000 07FF | TIM3    |
|      | 0x4000 0000 - 0x4000 03FF | TIM2    |

### 3.3 内置 SRAM

RX32G410 系列内置总计 64KB 的 SRAM。

- 32KB 的 SRAM（映射在 0x2000 0000）
- 32KB 的 CCM SRAM（映射在 0x1000 0000 和 SRAM 的结尾处）

这些 SRAM 可以以字节（8 位）、半字节（16 字节）或字（32 位）进行访问。这些存储器可以在没有等待周期的情况下由 CPU 或者 DMA 寻址，需要特别注意的是，映射在 0x1000 0000 的 CCM SRAM 不可以通过 DMA 寻址。

*注意：不建议将代码和数据一起放在 CCM SRAM 中，这样会造成效率降低。*

### 3.3.1 奇偶校验

整个 CCM SRAM 区域具有奇偶校验功能，SRAM 区域不具备。

可以通过 FLASH\_CR 的 CCM\_PE 位来使用 CCM SRAM 的奇偶校验功能。

在写入 CCM SRAM 时，将计算并存储奇偶校验位，在读取时会自动检查；当某一位发生错误时，则产生一个 NMI 中断。CCM SRAM 的奇偶校验错误相关标志位参考 SYSCFG\_CFGR2 寄存器的 CFGR2\_SPF 和 CFGR2\_SPF2。

*注意：在使能 CCM SRAM 的奇偶校验功能之前，建议先将整个 CCM SRAM 区域进行软件初始化，避免在读取未初始化的区域时出现奇偶校验错误，或者保证在读取 CCM SRAM 的某块区域之前已经对该区域进行过擦除或者写入操作。*

### 3.3.2 CCM SRAM 读保护

CCM SRAM 的读保护通过设置 RDP 选项字节来激活的。

### 3.3.3 CCM SRAM 擦除

使用 FLASH\_CR 的 CCM\_RST 位，可以在系统复位的时候擦除 CCM SRAM。

也可以使用 SYSCFG\_SKR 寄存器的 KEY[7:0]解除软件擦写保护，然后使用 SCSR\_CCMER 启动擦除。

## 3.4 Flash 概述

Flash 有两个不同的物理区域组成：

- 主 Flash 块。用于存储应用代码和数据。
- 信息块，主要包含三个部分：
  - 选项字节：用于硬件和存储器保护。
  - 系统存储器：包含 RX 专用代码。

Flash 接口基于 AHB 协议进行指令和数据访问。Flash 寄存器具有 Flash 的操作（擦除/写入）功能。

## 3.5 Boot 配置

RX32G410 的 BOOT 配置需要通过软件跳转的方式实现。可以跳转到系统存储器（映射在 0x1FFF 0000）执行 RX 专用代码（Bootloader）。

## 4 内置 Flash 存储器 (FLASH)

### 4.1 介绍

Flash 接口管理 CPU AHB ICode 和 Dcode 对 Flash 进行访问，具有 Flash 擦除和编程功能以及读写保护机制。

Flash 接口通过一个包含指令预取和缓存系统来加速代码的执行。

### 4.2 Flash 主要特征

无 ECC 模式：

- 总计 512KB 的 Flash 与双 bank 架构，支持读写操作 (RWW)，读写需不同 bank。
- Flash 读取支持字节 (8 位)、半字 (16 位) 和字 (32 位) 操作。
- Flash 写入支持 128 bits 操作。
- 支持页 (1KB) 擦除、bank 擦除和全擦除
- 指令预取：128bit
- 指令缓存：64X128bit 的 ICode 缓存线 (1KB RAM)
- 数据缓存：16X128bit 的 DCode 缓存线 (256B RAM)

ECC 模式开启：

- 总计 256KB 的 Flash 与双 bank 架构，支持读写操作 (RWW)，读写需不同 bank。
- Flash 读取支持字节 (8 位)、半字 (16 位) 和字 (32 位) 操作。
- Flash 写入支持 64 bits 操作。
- 支持页 (512B) 擦除、bank 擦除和全擦除
- 指令预取：64bit
- 指令缓存：64X64bit 的 ICode 缓存线 (512B RAM)
- 数据缓存：16X64bit 的 DCode 缓存线 (128B RAM)
- 错误码纠正 ECC：需启动 ECC Mode 选项
  - ECC Mode 位宽减半 (128bits->64bits) 有效容量为 256KB
  - 每 byte 有对应 8bit 纠错码 (可检测两个以内错误，只有一个错误时可校正)
- 读保护由选项字节激活 (RDP level 0 & 1 w/o 2)
- 写保护
- 选项字节加载器

### 4.3 Flash 功能描述

#### 4.3.1 Flash 结构

Flash 包含 512 页 (每页 1KB) 的主存储块和信息块，如下表所示。

表 4.1 Flash 结构

| 模块   | 名称  | 地址                        | 大小(字节) |
|------|-----|---------------------------|--------|
| 主存储块 | 页 0 | 0x0800 0000 - 0x0800 03FF | 1K     |
|      | 页 1 | 0x0800 0400 - 0x0800 07FF | 1K     |
|      | 页 2 | 0x0800 0800 - 0x0800 0BFF | 1K     |
|      | 页 3 | 0x0800 0C00 - 0x0800 0FFF | 1K     |

| 模块             | 名称            | 地址                        | 大小(字节) |
|----------------|---------------|---------------------------|--------|
|                | 页 4           | 0x0800 1000 - 0x0800 13FF | 1K     |
|                | ...           | ...                       | ...    |
|                | 页 127         | 0x0801 FC00 - 0x0801 FFFF | 1K     |
|                | ...           | ...                       | ...    |
|                | 页 255         | 0x0803 FC00 - 0x0803 FFFF | 1K     |
|                | ...           | ...                       | ...    |
|                | 页 511         | 0x0807 FC00 - 0x0807 FFFF | 1K     |
| 信息块            | 系统存储器         | 0x1FFF 0000 - 0x1FFF 07FF | 2K     |
|                | 选择字节          | 0x1FFF 7800 - 0x1FFF 782F | 48     |
| 闪存存储器<br>接口寄存器 | FLASH_ACR     | 0x4002 2000 - 0x4002 2003 | 4      |
|                | 保留            | 0x4002 2004 - 0x4002 2007 | 4      |
|                | FLASH_KEYR    | 0x4002 2008 - 0x4002 200B | 4      |
|                | FLASH_OPTKEYR | 0x4002 200C - 0x4002 200F | 4      |
|                | FLASH_SR      | 0x4002 2010 - 0x4002 2013 | 4      |
|                | FLASH_CR      | 0x4002 2014 - 0x4002 2017 | 4      |
|                | FLASH_ECCR    | 0x4002 2018 - 0x4002 201B | 4      |
|                | 保留            | 0x4002 201C - 0x4002 201F | 4      |
|                | FLASH_OPTR    | 0x4002 2020 - 0x4002 2023 | 4      |
|                | FLASH_WRP1AR  | 0x4002 202C - 0x4002 202F | 4      |
|                | FLASH_WRP1BR  | 0x4002 2030 - 0x4002 2033 | 4      |

### 4.3.2 读取等待周期

为了正确读取 Flash 的指令和数据，必须根据 CPU 的时钟（HCLK）在 Flash 访问控制寄存器（FLASH\_ACR）中配置等待周期（Latency）。等待周期和 CPU 时钟频率的关系如下表所示。

表 4.2 等待周期和 CPU 时钟频率对应关系

| 等待周期 (WS)<br>(Latency) | HCLK(MHz) |
|------------------------|-----------|
| 0 WS(1 CPU cycles)     | ≤38       |
| 1 WS(2 CPU cycles)     | ≤76       |
| 2 WS(3 CPU cycles)     | ≤115      |
| 3 WS(4 CPU cycles)     | ≤153      |
| 4 WS(5 CPU cycles)     | ≤192      |

复位后，CPU 时钟频率是 16MHz，FLASH\_ACR 中配置为 1 等待周期。

修改 CPU 频率时，必须执行以下步骤以保证足够的 Flash 访问等待周期：

**增加 CPU 频率：**

1. 修改 Flash 访问控制寄存器（FLASH\_ACR）中 Latency 的等待周期。
2. 读取并确认 FLASH\_ACR 寄存器中的等待周期。
3. 修改 CPU 频率。
4. 确认 CPU 频率。



#### 降低 CPU 频率：

1. 修改 CPU 频率。
2. 确认 CPU 频率。
3. 修改 Flash 访问控制寄存器 (FLASH\_ACR) 中 Latency 的等待周期。
4. 读取并确认 FLASH\_ACR 寄存器中的等待周期。

### 4.3.3 指令预取

Cortex®-M4 通过 ICode 总线取指令，通过 DCode 总线取数据。预取缓冲区可以提高 ICode 总线的访问效率。

预取缓冲区 (2×64bit)：CPU 每次取指最多为 32 位的字，取一条指令时，下一条指令已经在缓冲区内等待，这样 CPU 可以工作在更高的主频。

复位后预取缓冲区默认关闭。只有在 SYSCLK 小于 40MHz 且 AHB 没有预分频 (SYSCLK 必须等于 HCLK) 时才可以开启或关闭预取缓冲区。通常情况下，建议在芯片初始化阶段开启或关闭缓冲区，此时芯片运行在内部 16MHz 晶振上。

### 4.3.4 Flash 擦除和编程

Flash 支持在线编程和应用编程。

在线编程 (in-circuit programming, ICP) 是指使用 JTAG、SWD 协议或者 bootloader 更新 Flash 的内容。

应用编程 (in-application programming, IAP) 是指使用芯片支持的通讯协议更新 Flash 的内容。IAP 允许用户在芯片运行过程中更新 Flash 的内容。然而要实现 IAP，需要预先使用 ICP 将部分应用代码编写进 Flash 中。

如果在 Flash 操作期间复位芯片将无法保证 Flash 的内容正确。

### 4.3.5 解锁 Flash

复位后，Flash 控制寄存器 (Flash control register, FLASH\_CR) 不可写入，这是为了防止 Flash 由于干扰产生误操作。通过以下流程可以解锁 FLASH\_CR：

1. 在 Flash 密钥寄存器 (Flash key register, FLASH\_FKEYR) 写入 KEY1 = 0x45670123。
2. 在 Flash 密钥寄存器 (Flash key register, FLASH\_FKEYR) 写入 KEY2 = 0xCDEF89AB。

任何错误的流程将会导致 FLASH\_CR 上锁直到系统复位。一旦执行了错误的写入密钥流程，将会产生一个总线错误和一个 Hard Fault 中断。

FLASH\_CR 可以通过写入其中的 LOCK 位来上锁。

**注意：**FLASH\_CR 寄存器在 Flash 状态寄存器 (Flash status register, FLASH\_SR) 的 BSY 位置起时不可被写入。在 BSY 位置起时，任何写入动作将会导致 AHB 总线挂起直到 BSY 位被清除。

### 4.3.6 Flash 擦除流程

Flash 擦除可以分为页擦和全擦。全擦不会擦除信息区 (系统存储器和选项字节)。

#### 页擦

页擦流程如下：

1. 确认 FLASH\_SR 寄存器的 BSY 位以保证没有正在执行的 Flash 操作。
2. 检查并清除之前的 Flash 操作造成的错误。

3. 将 FLASH\_CR 的 PER 位置 1，写 PNB 选择要擦除的页。
4. 将 FLASH\_CR 的 STRT 位置 1。
5. 等 FLASH\_SR 中 BSY 被清除。

## 4.4 Flash 寄存器描述

### 4.4.1 Flash 访问控制寄存器 (FLASH\_ACR)

地址偏移: 0x00

复位值: 0x0000 0600

访问: 无等待, 支持字, 半字和字节访问

|          |    |    |       |       |      |      |            |          |    |    |    |              |    |    |    |
|----------|----|----|-------|-------|------|------|------------|----------|----|----|----|--------------|----|----|----|
| 31       | 30 | 29 | 28    | 27    | 26   | 25   | 24         | 23       | 22 | 21 | 20 | 19           | 18 | 17 | 16 |
| Reserved |    |    |       |       |      |      |            |          |    |    |    |              |    |    |    |
| 15       | 14 | 13 | 12    | 11    | 10   | 9    | 8          | 7        | 6  | 5  | 4  | 3            | 2  | 1  | 0  |
| Reserved |    |    | DCRST | ICRST | DCEN | ICEN | PRFT<br>EN | Reserved |    |    |    | LATENCY[3:0] |    |    |    |
|          |    |    | rw    | rw    | rw   | rw   | rw         |          |    |    |    | rw           | rw | rw | rw |

Bits 31:13 保留, 必须保持为复位值。

Bit 12 **DCRST**: 数据缓存使能

0: 数据缓存不复位

1: 复位数据缓存

仅在 DCEN=0 时可写。

Bit 11 **ICRST**: 指令缓存复位

0: 指令缓存不复位

1: 复位指令缓存

仅在 ICEN=0 时可写。

Bit 10 **DCEN**: 数据缓存使能

0: 关闭数据缓存

1: 使能数据缓存

Bit 9 **ICEN**: 指令缓存使能

0: 关闭指令缓存

1: 使能指令缓存

Bit 8 **PRFTEN**: 预取缓冲区使能

0: 关闭预取缓冲区

1: 使能预取缓冲区

Bits 7:4 保留, 必须保持为复位值。

Bits 3:0 **LATENCY[3:0]**: 等待周期

该位表示 SYSCLK (系统时钟) 周期与 Flash 访问时间的比例。

000: 零等待周期

001: 一个等待周期

010: 两个等待周期

011: 三个等待周期

100: 四个等待周期

...

#### 4.4.2 Flash 键寄存器 (FLASH\_KEYR)

地址偏移: 0x08

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| KEYR[31:16] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| KEYR[15:0]  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:0 KEYR: FPEC 键

KEY1: 0x4567 0123

KEY2: 0XCDEF 89AB

#### 4.4.3 Flash OPTKEY 寄存器 (FLASH\_OPTKEYR)

地址偏移: 0x0C

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

|                |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31             | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| OPTKEYR[31:16] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw             | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |
| 15             | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| OPTKEYR[15:0]  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw             | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:0 OPTKEYR: FPEC 键

KEY1: 0x0819 2A3B

KEY2: 0X4C5D 6E7F

#### 4.4.4 Flash 状态寄存器 (FLASH\_SR)

地址偏移: 0x10

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

|          |       |          |    |    |    |    |    |       |       |       |       |       |     |       |       |
|----------|-------|----------|----|----|----|----|----|-------|-------|-------|-------|-------|-----|-------|-------|
| 31       | 30    | 29       | 28 | 27 | 26 | 25 | 24 | 23    | 22    | 21    | 20    | 19    | 18  | 17    | 16    |
| Reserved |       |          |    |    |    |    |    |       |       |       |       |       |     |       | BSY   |
|          |       |          |    |    |    |    |    |       |       |       |       |       |     |       | r     |
| 15       | 14    | 13       | 12 | 11 | 10 | 9  | 8  | 7     | 6     | 5     | 4     | 3     | 2   | 1     | 0     |
| OPTV     | RD    | Reserved |    |    |    |    |    | PGS   | SIZE  | PGA   | WRP   | PROG  | Res | OPE   | EOP   |
| ERR      | ERR   |          |    |    |    |    |    | ERR   | ERR   | ERR   | ERR   | ERR   |     | RR    |       |
| rc_w1    | rc_w1 |          |    |    |    |    |    | rc_w1 | rc_w1 | rc_w1 | rc_w1 | rc_w1 |     | rc_w1 | rc_w1 |

Bits 31:17 保留, 必须保持为复位值。

Bit 16 BSY: 操作进行位

该位指示闪存操作正在进行。在闪存操作开始时, 该位被设置为 ‘1’; 在操作结束或发生错误时该位被清除为 ‘0’。

Bit 15 OPTVERR: 选项字节有效错误

当读取的选项字节不是用户配置的选项字节时, 由硬件设置为 ‘1’。如果选项字节没有被正确加载, OPTVERR 将在每次系统复位后重新配置为 ‘1’。

该位写 1 清零。

**Bit 14 FLASH\_SR\_RDERR: PCROP 读错误**

当通过 D-bus 读取的地址属于闪存中的读取保护区域（PCROP 保护）时，由硬件设置。如果在 Flash\_CR 中设置了 RDERRIE，则会产生中断。通过写入 ‘1’ 来清除此错误标志。

Bits 13:8 保留，必须保持为复位值。

**Bit 7 PGSERR: 编程顺序错误**

当 PG 未设置时，对 Flash 进行写访问，该位由硬件设置为 ‘1’。当由于之前的错误而出现 PROGERR, SIZERR, PGAERR, WRPERR 时，也由硬件设置。

该位写 1 清零。

**Bit 6 SIZERR: 编程 size 错误**

当通过字节访问或半字访问时，由硬件设置为 ‘1’。仅允许双字编程

该位写 1 清零。

**Bit 5 PGAERR: 编程对齐错误**

当要编程的数据在标准编程时不能被包含在同一个 64 位闪存行中时，由硬件设置 ‘1’ 该位写 1 清零。

**Bit 4 WRPERR: 写保护错误**

试图对写保护的闪存地址编程或擦除时，硬件设置这位为 ‘1’。

该位写 1 清零。

**Bit 3 PROGERR: 编程错误**

试图对内容不是 0xFFFF FFFF FFFF FFFF 的双字地址编程时，硬件设置该位为 ‘1’，除非编程数据为 0x0000 0000 0000 0000

该位写 1 清零。

Bits 2 保留，必须保持为复位值。

**Bit 1 OPERR: 操作错误**

当对 Flash 闪存操作（编程/擦除）不成功时，由硬件设置为 ‘1’。

仅当错误中断（ERRIE=1）被使能时被设置。

该位写 1 清零。

**Bit 0 EOP: 操作结束**

当闪存操作（编程/擦除）完成时，硬件设置这位为 ‘1’。

注：每次成功的编程或擦除都会设置 EOP 状态。

仅当操作中断（EOPIE=1）被使能时被设置。

该位写 1 清零。

#### 4.4.5 Flash 控制寄存器 (FLASH\_CR)

地址偏移: 0x14

复位值: 0xC000 0000

访问: 无等待, 支持字, 半字和字节访问

| 31   | 30       | 29       | 28 | 27 | 26 | 25         | 24       | 23    | 22    | 21       | 20 | 19 | 18 | 17      | 16    |
|------|----------|----------|----|----|----|------------|----------|-------|-------|----------|----|----|----|---------|-------|
| LOCK | OPT LOCK | Reserved |    |    |    | OBL_LAUNCH | RDERRIE  | ERRIE | EOPIE | Reserved |    |    |    | OPTSTRT | START |
| rs   | rs       |          |    |    |    | rc_w1      | rw       | rw    | rw    |          |    |    |    | rs      | rs    |
| 15   | 14       | 13       | 12 | 11 | 10 | 9          | 8        | 7     | 6     | 5        | 4  | 3  | 2  | 1       | 0     |
| MER2 | SYSWRE   | Reserved |    |    |    | BKER       | PNB[7:0] |       |       |          |    |    |    | MER1    | PER   |
| rw   |          |          |    |    |    | rw         | rw       | rw    | rw    | rw       | rw | rw | rw | rw      | rw    |

Bit 31 **LOCK**: 锁定位

只能写 ‘1’。当该位为‘1’时表示 FPEC 和 FLASH\_CR 被锁住。在检测到正确的解锁序列后, 硬件清除此位为‘0’。

在一次不成功的解锁操作后, 下次系统复位前, 该位不能再被改变。

Bit 30 **OPTLOCK**: 选项字节锁定位

该位仅可以被设置 ‘1’。设置后, FLASH\_CR 寄存器中与选项字节有关的所有位和选项页都被锁定。硬件检测到解锁顺序后, 清除该位。在对 OPTLOCK 位进行解锁顺序之前, 必须先清除 LOCK 位。

在解锁操作不成功的情况下, 该位始终保持设置, 直到下一次重置。

Bits 29:28 保留, 必须保持为复位值。

Bit 27 **OBL\_LAUNCH**: 强制加载选项字节

当设置 1 时, 强制重新加载选项字节。只有选项字节加载完成, 该位才被清除。

如果 OPTLOCK 被设置, 则该位不能被写入。

0: 选项字节加载完成

1: 请求加载选项字节

Bit 26 **RDERRIE**: **PCROP** 读取错误中断使能

该位用于在 FLASH\_SR 寄存器中的 RDERP 位被设置为 1 时使能中断生成。

0: 禁止 PCROP 读错误中断;

1: 使能 PCROP 读错误中断

Bit 25 **ERRIE**: 错误中断使能

该位用于在 FPEC (闪存编程/擦除控制器) 发生错误时使能中断生成 (当 FLASH\_SR 寄存器中的 PGERR/WRPRTERR 位被设置时)。

0: 禁止中断生成;

1: 使能中断生成

Bit 24 **EOPIE**: 操作结束中断使能

该位用于在 FLASH\_SR 寄存器中的 EOP 位变为 1 时使能中断生成。

0: 禁止中断生成;

1: 使能中断生成

Bit 17 **OPTSTRT**: 开始修改选项字节

当该位写 ‘1’ 时将触发一次选项字节操作。该位只可由软件置为 ‘1’ 并在 BSY 变为 ‘0’ 时清为 0。

Bit 16 **START**: 开始

当该位写‘1’时将触发一次擦除操作。该位只可由软件置为‘1’并在 BSY 变为‘0’时清为 0。

Bit 15 **MER2**: BANK2 全擦除操作

0: 禁止 BANK2 全擦除

1: BANK2 全擦除使能

Bit 14 **SYSWRE**: 系统存储器写操作

0: 禁止系统存储器写操作

1: 使能系统存储器写操作

Bit 11 **BKER**: BANK 擦除

DBANK=1:

0: 选择 BANK1 进行页擦除

1: 选择 BANK2 进行页擦除

Bits 10:3 **PNB[7:0]**: 擦除页码选择

00000000: page 0

00000001: page 1

00000010: page 2

00000011: page 3

....

11111111: page 255

Bit 2 **MER1**: BANK1 全擦除操作

0: 禁止 BANK1 全擦除

1: BANK1 全擦除使能

Bit 1 **PER**: 页擦除操作

0: 禁止页擦除

1: 页擦除使能

Bit 0 **PG**: 编程操作

0: 禁止编程操作

1: 编程操作使能

#### 4.4.6 Flash ECC 寄存器 (FLASH\_ECCR)

地址偏移: 0x18

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

| 31             | 30    | 29        | 28        | 27       | 26 | 25 | 24 | 23         | 22  | 21           | 20     | 19              | 18 | 17 | 16 |
|----------------|-------|-----------|-----------|----------|----|----|----|------------|-----|--------------|--------|-----------------|----|----|----|
| ECCD           | ECCC  | EC<br>CD2 | EC<br>CC2 | Reserved |    |    |    | ECC<br>CIE | Res | SYSF_<br>ECC | BK_ECC | ADDR_ECC[20:16] |    |    |    |
| rc_w1          | rc_w1 | rc_w1     | rc_w1     |          |    |    |    | rw         |     | r            | r      | r               | r  | r  | r  |
| 15             | 14    | 13        | 12        | 11       | 10 | 9  | 8  | 7          | 6   | 5            | 4      | 3               | 2  | 1  | 0  |
| ADDR_ECC[15:0] |       |           |           |          |    |    |    |            |     |              |        |                 |    |    |    |
| r              | r     | r         | r         | r        | r  | r  | r  | r          | r   | r            | r      | r               | r  | r  | r  |

Bit 31 **ECCD**: ECC 检测

DBANK=1

当检测到两个 ECC 错误时, 由硬件置位。当该位置起时, 将产生一个 NMI。

写 1 清除。



Bit 30 **ECCC**: ECC 校正

DBANK=1

当检测到一个 ECC 错误并纠正后, 有硬件置位。如果设置了 ECCCIE,则会产生中断。  
写 1 清除。

Bit 29 **ECCD2**: ECC 检测

DBANK=0

当检测到两个 ECC 错误时, 由硬件置位。当该位置起时, 将产生一个 NMI。  
写 1 清除

Bit 28 **ECCC2**: ECC2 校正

DBANK=0

当检测到一个 ECC 错误并纠正后, 有硬件置位。如果设置了 ECCCIE,则会产生中断。  
写 1 清除。

Bits 27:25 保留, 必须保持为复位值。

Bit 24 **ECCCIE**: ECC 纠错中断使能

0: 关闭 ECCC 中断

1: 开启 ECCC 中断

Bit 23 保留, 必须保持为复位值。

Bit 22 **SYSF\_ECC**: system Flash ECC 错误

该位表明 ECC 纠错或双 ECC 纠错位于 system Flash 中。

Bit 21 **BK\_ECC**: ECC 错误发生区域

DBANK=1

该位表明 ECC 纠错或双 ECC 纠错所发生的 BANK.

0: BANK1

1: BANK2

DBANK=0

如果 SYSF\_ECC=1, 表明 ECC 错误与哪个 BANK 有关。

如果 SYSF\_ECC=0, 则 BK\_ECC 必须为 0

Bits 20:0 **ADDR\_ECC[20:0]**: ECC 错误地址

DBANK=0

该位表明 Flash 中出现 ECC 错误或双 ECC 错误的地址。

DBANK=1

该位表明 Bank 中出现 ECC 错误或双 ECC 错误的地址。

#### 4.4.7 Flash 选项字节寄存器 (FLASH\_OPTR)

地址偏移: 0x20

复位值: 0xFxxx xxxx 该寄存器从 Flash OBL 中加载。

访问: 无等待, 支持字, 半字和字节访问

| 31                | 30               | 29         | 28        | 27       | 26       | 25           | 24      | 23       | 22   | 21  | 20   | 19      | 18         | 17        | 16      |
|-------------------|------------------|------------|-----------|----------|----------|--------------|---------|----------|------|-----|------|---------|------------|-----------|---------|
| RDP1_DIS_FSH_MAIN | RDP1_PC_CHK_MODE | Res        | nRST_MODE | nBOOT0   | nSWBOOT0 | CCM_SRAM_RST | SRAM_PE | Nboot1   | DBNK | ECC | BFB2 | WWDG_SW | IWDG_STDBY | IWDG_STOP | IWDG_SW |
| rw                | rw               |            | rw        | rw       | rw       | rw           | rw      | rw       | r    | rw  | rw   | rw      | rw         | rw        | rw      |
| 15                | 14               | 13         | 12        | 11       | 10       | 9            | 8       | 7        | 6    | 5   | 4    | 3       | 2          | 1         | 0       |
| Reserved          |                  | nRST_STDBY | nRST_STOP | Reserved |          |              |         | RDP[7:0] |      |     |      |         |            |           |         |
|                   |                  | rw         | rw        |          |          |              |         | rw       | rw   | rw  | rw   | rw      | rw         | rw        | rw      |

Bit 31 RDP1\_DIS\_FSH\_MAIN:

0: RDP1 通过 PC 位置禁用闪存访问 (在 SRAM 或系统引导时不允许)

1: RDP1 在 DebugActive、SRAM\_BOOT、SYSTEM\_BOOT 时禁用所有主闪存访问 (仅当 rdp 级别为 0 时可改变)

Bit 30 RDP1\_PC\_CHK\_MODE:

RDP1 通过 PC 位置禁用闪存访问 (当 RDP1\_DIS\_FSH\_MAIN=0)

0: 包含闪存启动

1: 仅检查非闪存启动 (仅当 rdp 级别为 0 时可改变)

Bits 29 保留, 必须保持为复位值。

Bit 28 nRST\_MODE:

0: GPIO 模式, 仅内部使用

1: 复位输入输出

Bit 27 nBOOT0: BOOT0 选择位

0: Boot0=0

1: Boot0=1

Bit 26 nSWBOOT0: 选择 BOOT0 模式

0: Boot0 取决于 nBoot0 位

1: Boot0 取决于 PB8/Boot0 引脚

Bit 25 CCM\_SRAM\_RST: 系统复位时擦除 CCM\_SRAM

0: 系统复位时擦除 CCM\_SRAM

1: 系统复位时不擦除 CCM\_SRAM

Bit 24 SRAM\_PE: 开启 SRAM 和 CCM\_SRAM 奇偶校验

0: 开启 CCM\_SRAM 奇偶校验

1: 关闭 CCM\_SRAM 奇偶校验

Bit 22 Nboot1: Boot 配置

1: 和 BOOT0 位一起配置, 该位选择从 Flash 主闪存, SRAM1 或系统内存中选择引导模式。

Bit 22 DBANK:

1: 双 BANK 模式

Bit 21 ECC:

1: 对于 ECC 模式 (使用一半的闪存来存储 ECC 数据)

Bit 20 **BFB2**:

0: 禁用双 bank 启动

Bit 19 **WWDG\_SW**: 窗口看门狗选择

0: 硬件窗口看门狗

1: 软件窗口看门狗

Bit 18 **IWDG\_STDBY**: 待机模式下独立看门狗冻结

0: 待机模式下冻结独立看门狗

1: 待机模式下独立看门狗运行

Bit 17 **IWDG\_STOP**: 停机模式下独立看门狗冻结

0: 停机模式下冻结独立看门狗

1: 停机模式下独立看门狗运行

Bit 16 **IWDG\_SW**: 独立看门狗选择

0: 硬件独立看门狗

1: 软件独立看门狗

Bits 15:14 保留, 必须保持为复位值。

Bit 13 **nRST\_STDBY**:

0: 当进入待机模式时产生复位

1: 当进入待机模式时不复位

Bit 12 **nRST\_STOP**:

0: 当进入停机模式时产生复位

1: 当进入停机模式时不复位

Bits 7:0 **RDP[7:0]**: 读保护级别

0xAA: 0 级, 读保护未激活

0xCC: 2 级, 芯片读保护激活

其他: 1 级, memories 读保护激活

#### 4.4.8 Flash BANK1 写保护 A 地址寄存器 (FLASH\_WRP1AR)

地址偏移: 0x2C

复位值: 0x00xx 00xx 该寄存器从 Flash OBL 中加载。

访问: 无等待, 支持字, 半字和字节访问

|          |    |    |    |    |    |    |                  |    |    |    |    |    |    |    |    |
|----------|----|----|----|----|----|----|------------------|----|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24               | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |    | WRP1A_END[24:16] |    |    |    |    |    |    |    |    |
|          |    |    |    |    |    |    | rw               | rw | rw | rw | rw | rw | rw | rw | rw |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8                | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    |    |    |    | WRP1A_STRT[8:0]  |    |    |    |    |    |    |    |    |
|          |    |    |    |    |    |    | rw               | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:23 保留, 必须保持为复位值。

Bits 24:16 **WRP1A\_END[24:16]**: 写保护第一个区域“A”结束偏移量

Bits 15:7 保留, 必须保持为复位值。

Bits 8:0 **WRP1A\_STRT[8:0]**: 写保护第一个区域“A”开始偏移量

#### 4.4.9 Flash BANK1 写保护 B 地址寄存器 (FLASH\_WRP1BR)

地址偏移: 0x30

复位值: 0x00xx 00xx 该寄存器从 Flash OBL 中加载。

访问：无等待，支持字，半字和字节访问

| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24               | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|----------|----|----|----|----|----|----|------------------|----|----|----|----|----|----|----|----|
| Reserved |    |    |    |    |    |    | WRP1B_END[24:16] |    |    |    |    |    |    |    |    |
|          |    |    |    |    |    |    | rw               | rw | rw | rw | rw | rw | rw | rw | rw |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8                | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    |    |    |    | WRP1B_STRT[8:0]  |    |    |    |    |    |    |    |    |
|          |    |    |    |    |    |    | rw               | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:23 保留，必须保持为复位值。

Bits 24:16 **WRP1B\_END[24:16]**：写保护第一个区域“B”结束偏移量

Bits 15:7 保留，必须保持为复位值。

Bits 8:0 **WRP1B\_STRT[8:0]**：写保护第一个区域“B”开始偏移量

## 5 CRC 计算单元

### 5.1 CRC 简介

CRC（循环冗余校验）计算单元使用一个固定的多项式发生器从一个 32 位的数据字中产生 CRC 码。在众多的应用中，基于 CRC 的技术还常用来验证数据传输或存储的完整性。根据 EN/IEC 60335-1 标准的规定，这些技术提供了验证 Flash 完整性的方法。CRC 计算单元有助于在运行期间计算软件的签名，并将该签名与链接时生成并存储在指定存储单元的参考签名加以比较。

### 5.2 CRC 主要特性

- 使用 CRC-32（以太网）多项式：0x4C11DB7
  - $X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11} + X^{10} + X^8 + X^7 + X^5 + X^4 + X^2 + X + 1$
- 单输入/输出 32 位数据寄存器
- CRC 计算在 4 个 AHB 时钟周期（HCLK）内完成
- 8 位通用寄存器（可用于临时存储）

下图为 CRC 计算单元框图

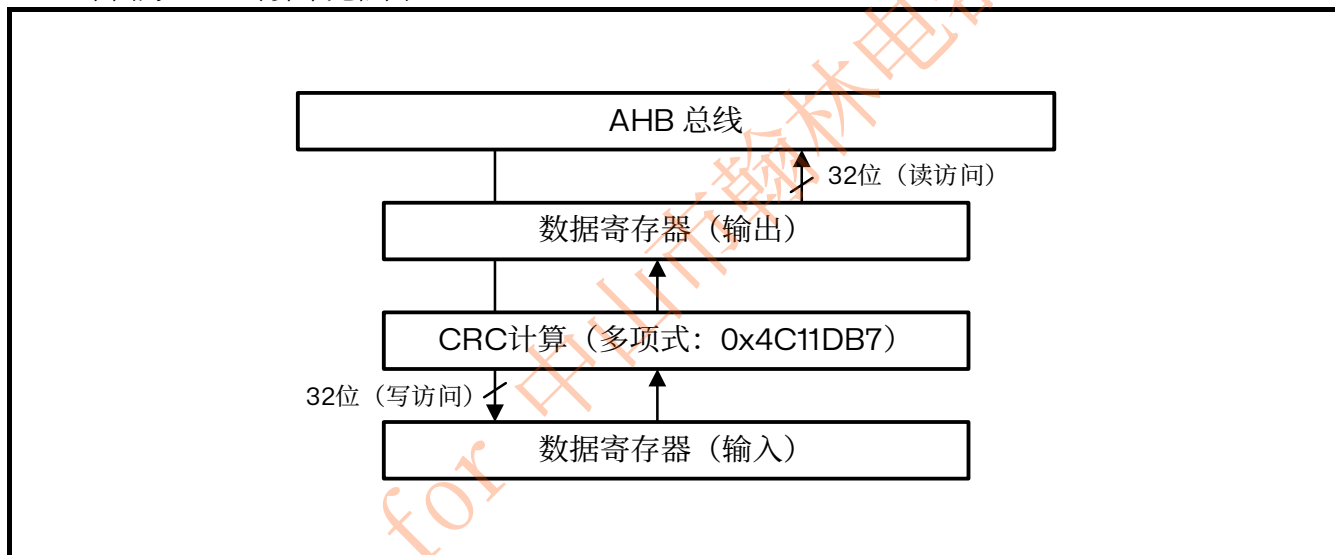


图 5.1 CRC 计算单元框图

### 5.3 CRC 功能说明

CRC 计算单元主要由单个 32 位数据寄存器组成，该寄存器：

- 用作输入寄存器，向 CRC 计算器中输入新数据（向寄存器写入数据时）
- 可保存之前的 CRC 计算结果（读取寄存器时）

对数据寄存器的每个写操作都会把当前新输入的数值和之前生成在数据寄存器中的 CRC 值再做一次 CRC 计算（CRC 计算针对整个 32 位数据字完成，而非逐字节进行）。

CRC 计算的时候，写操作被阻塞，因此允许执行背靠背写访问或连续的读写访问。

使用 CRC\_CR 寄存器中的 RESET 控制位即可将 CRC 计算器复位为 0xFFFF FFFF。此操作不影响 CRC\_IDR 寄存器的内容。

## 5.4 CRC 寄存器

CRC 计算单元包含两个数据寄存器和一个控制寄存器。CRC 寄存器必须按字（32 位）访问。

### 5.4.1 数据寄存器（CRC\_DR）

偏移地址：0x00

复位值：0xFFFF FFFF

|            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31         | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DR [31:16] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw         | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |
| 15         | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| DR [15:0]  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw         | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:0 DR[31:16]: 数据寄存器位

向 CRC 计算器写入新数据时用作输入寄存器。  
读取寄存器时可读出之前的 CRC 计算结果。

### 5.4.2 独立数据寄存器（CRC\_IDR）

地址偏移：0x04

复位值：0x0000 0000

|            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31         | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15         | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| IDR [15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw         | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:8 保留，必须保持复位值。

Bit 15:0 IDR[15:0]: 通用 16 位数据寄存器位

可用作一个字节的临时存储单元。  
此寄存器不受 CRC\_CR 寄存器中 RESET 位产生的 CRC 复位影响。

### 5.4.3 控制寄存器（CRC\_CR）

地址偏移：0x08

复位值：0x0000 0000

|          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |       |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|-------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16    |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    |    |       |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0     |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    |    | RESET |
|          |    |    |    |    |    |    |    |    |    |    |    |    |    |    | w     |

Bits 31:1 保留，必须保持复位值。

Bit 0 RESET:

复位 CRC 计算单元并将数据寄存器设为 0xFFFF FFFF。  
此位只能置 1，将由硬件自动进行清零。



## 6.1 概述

n x 100 nF  
+1 x 4.7  $\mu$ F



## 6.2 电源管理器

### 6.2.1 上电复位 (POR) 和掉电复位 (PDR)

RX32G410 内部有一个完整的上电复位（POR）和掉电复位（PDR）电路，当供电电压达到 2.7V 时，系统既能正常工作。

当 VDD/VDDA 低于指定的限位电压  $V_{POR}/V_{PDR}$  时，系统保持为复位状态，无需外部复位电路。关于上电复位和掉电复位的细节请参考数据手册的电气特性部分。

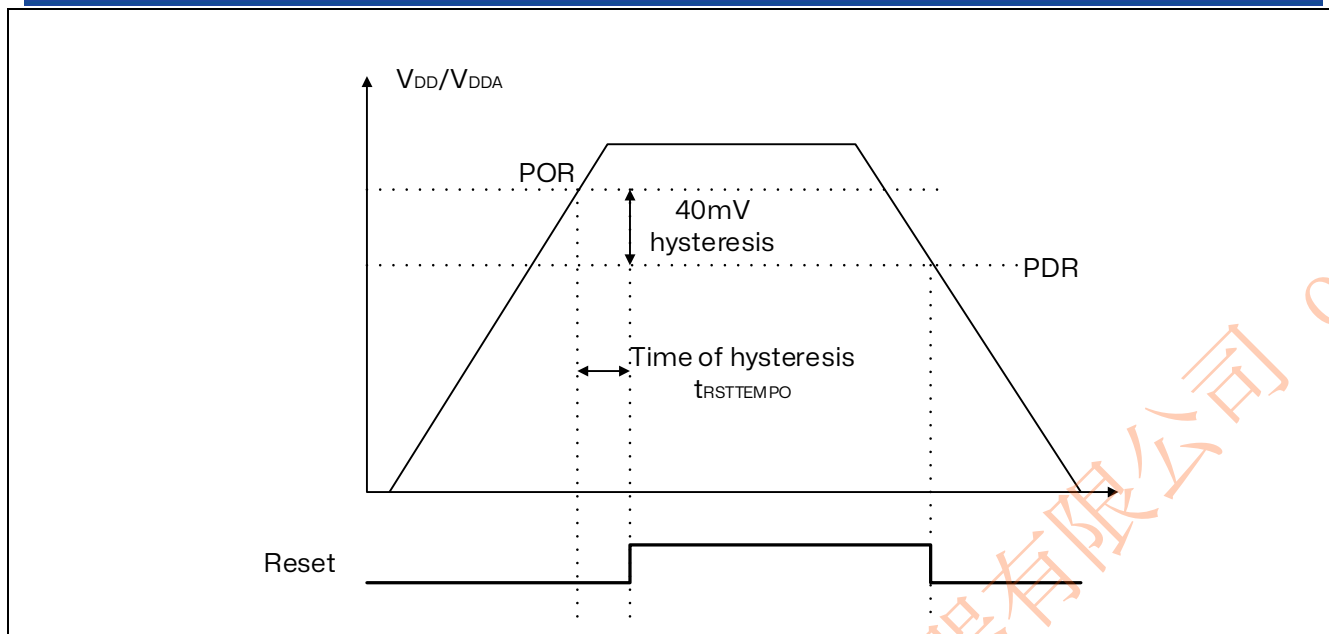


图 6.2 上电复位和掉电复位波形图

### 6.2.1 可编程电压检测器 (PVD)

用户可以利用 PVD 对 VDD 电压与电源控制寄存器 2 (PWR\_CR2) 中的 PLS[2:0] 位进行比较来监控电源，这几位选择监控电压的阈值。通过设置 PVDE 位来使能 PVD。

电源状态寄存器 2 (PWR\_SR2) 中的 PVDO 标志用来表明 VDD 是高于还是低于 PVD 的电压阈值。该事件在内部连接到外部中断的线 16，如果该中断在外部中断寄存器中是使能的，该事件就会产生中断。当 VDD 下降到 PVD 阈值以下和（或）当 VDD 上升到 PVD 阈值之上时，根据外部中断第 16 线的上升/下降边沿触发设置，就会产生 PVD 中断。

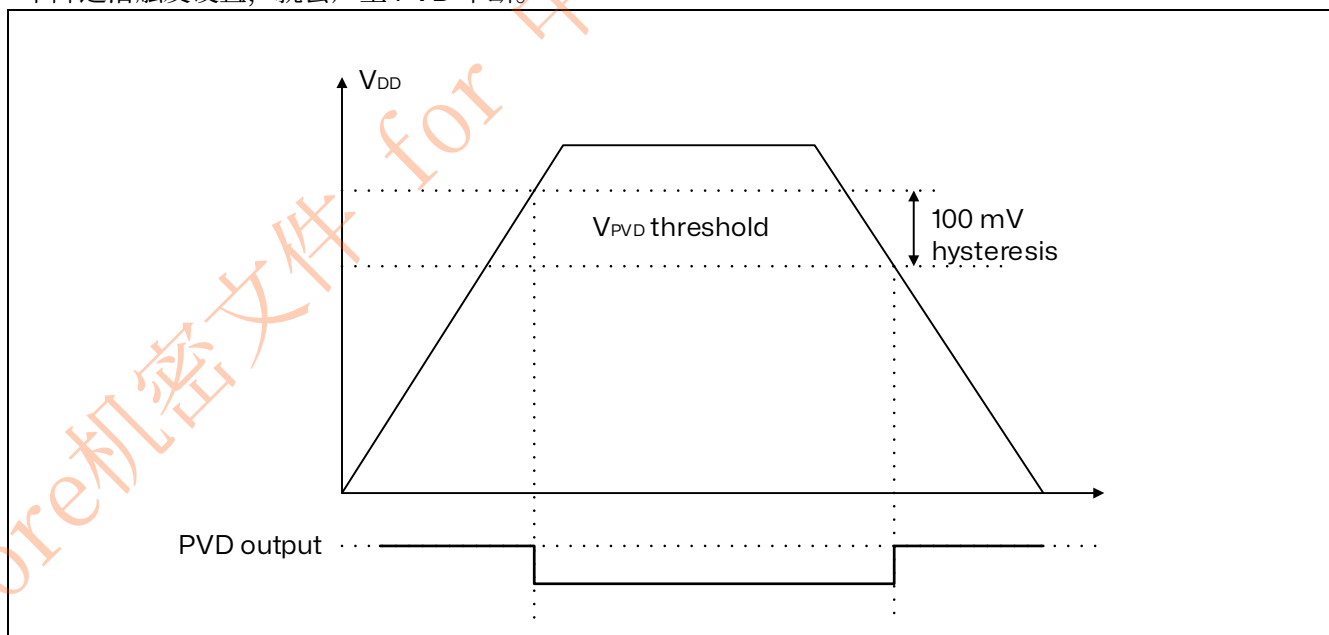


图 6.3 PVD 门限

### 6.3 低功耗模式

默认情况下，在系统或电源复位以后，微控制器处于运行状态。当 CPU 不需继续运行时（例如等待外部事件时），可以利用多种低功耗模式来节省功耗。用户需要根据最低电源消耗、最快速启动时间和可用的唤醒源等条件，选定一个最佳的低功耗模式。

RX32G410 支持三种低功耗模式，用户可以在以下模式中进行选择：

- 睡眠模式：CPU 时钟被关闭。所有外设（包括带 FPU 核心的 Cortex®-M4 外设，如 NVIC、SysTick 等）继续运行，并且 CPU 可以被中断或者事件唤醒。
- 停止 0 和停止 1 模式：保留 SRAM 和寄存器内容。关闭除 LSI 以外所有时钟。该系列可以被任一外部中断线从停止模式中唤醒。

在停止 0 模式下，内部 LDO 配置为正常模式，这允许最快的唤醒时间但功耗更高。活动的外设和唤醒源与停止 1 模式相同。

- 待机模式：待机模式用于实现最低的功耗，关闭 LDO1P7 和 LDO1P2，关闭除 LSI 以外所有时钟。该系列可以被外部复位（NRST 引脚）、IWDG 复位、RTC 唤醒以及 WKUP 唤醒引脚的上升沿和下降沿从待机模式中唤醒。

此外，可以通过以下方式降低运行模式的功耗：

- 降低系统时钟速度
- 当 APB 和 AHB 外设未使用时对它们进行时钟门控。

表 6.1 低功耗模式一览

| 模式                 | 进入                                   | 唤醒                                   | 唤醒后时钟      | 对时钟影响          | LDO |     |
|--------------------|--------------------------------------|--------------------------------------|------------|----------------|-----|-----|
|                    |                                      |                                      |            |                | 1P7 | 1P2 |
| 睡眠<br>(立即睡眠或退出时睡眠) | WFI                                  | 任一中断                                 | 与进入睡眠模式前一致 | CPU 时钟关闭       | 正常  | 正常  |
|                    | WFE                                  | 唤醒事件                                 |            |                |     |     |
| 停止 0               | LPMS= “00” + SLEEPDEEP 位 + WFI 或 WFE | 任意外部中断<br>(在 EXTI 寄存器中配置)<br>指定外设事件  | HSI        | 关闭除 LSI 以外所有时钟 | 正常  | 正常  |
| 停止 1               | LPMS= “01” + SLEEPDEEP 位 + WFI 或 WFE |                                      |            |                | 低功耗 | 低功耗 |
| 待机                 | LPMS= “11” + SLEEPDEEP 位 + WFI 或 WFE | WKUP 引脚电平变化、RTC 唤醒事件、NRST 复位、IWDG 复位 | HSI        | 关闭除 LSI 以外所有时钟 | 关闭  | 关闭  |

#### 调试模式

默认情况下，如果在进行调试微处理器时，使微处理器进入停止 0、停止 1 或待机模式，将失去调试连接。这是因为内核失去了时钟。

通过设置 DBGMCU\_CR 寄存器中的对应配置位，可以在低功耗模式下调试软件。有关更多详细信息，请参阅章节：调试支持（DBG）。

### 6.3.1 运行模式

#### 降低系统时钟速度

在运行模式下，可通过对预分频寄存器编程来降低系统时钟（SYSCLK、HCLK、PCLK）速度。进入睡眠模式之前，也可以使用这些预分频器降低外设速度。

更多详细信息，请参阅第 8.3.3 节：时钟配置寄存器（RCC\_CFGR）。

#### 外设时钟门控

在运行模式下，可随时停止各外设和存储器的 HCLK 和 PCLK 以降低功耗。

要进一步降低睡眠模式的功耗，可在执行 WFI 或 WFE 指令之前禁止外设时钟。

外设时钟门控由 RCC\_AHBENR 和 RCC\_APBxENR 寄存器控制。

### 6.3.2 低功耗模式

#### 进入低功耗模式

MCU 通过执行 WFI（等待中断）或 WFE（等待事件）指令，或在从 ISR 返回时设置带 FPU 核心的 Cortex®-M4 系统控制寄存器中的 SLEEPONEXIT 位来进入低功耗模式。

通过 WFI 或 WFE 进入低功耗模式仅在无挂起中断或无挂起事件时执行。

#### 退出低功耗模式

从睡眠模式和停止模式退出低功耗模式取决于进入低功耗模式的方式：

- 如果使用 WFI 指令或从 ISR 返回进入低功耗模式，任何由 NVIC 确认的外部中断都可以唤醒设备。
- 如果使用 WFE 指令进入低功耗模式，MCU 会在事件发生时立即退出低功耗模式。唤醒事件可以通过以下方式生成：
  - NVIC IRQ 中断。
    - 当带 FPU 核心的 Cortex®-M4 系统控制寄存器中的 SEVONPEND = 0 时。  
通过在外设控制寄存器和 NVIC 中使能一个中断。当 MCU 从 WFE 恢复时，必须清除外设中断挂起位和 NVIC 外设 IRQ 通道挂起位（在 NVIC 中断清除挂起寄存器中）。  
只有具有足够优先级的 NVIC 中断才会唤醒并中断 MCU。
    - 当带 FPU 核心的 Cortex®-M4 系统控制寄存器中的 SEVONPEND = 1 时。  
通过在外设控制寄存器中使能一个中断，并可选地在 NVIC 中使能。当 MCU 从 WFE 恢复时，必须清除外设中断挂起位，如果使能，还需清除 NVIC 外设 IRQ 通道挂起位（在 NVIC 中断清除挂起寄存器中）。  
所有 NVIC 中断都会唤醒 MCU，即使是被禁用的中断。但只有具有足够优先级的已使能 NVIC 中断才会唤醒并中断 MCU。
  - 事件  
将 EXTI 线路配置为事件模式。当 CPU 从 WFE 恢复时，无需清除 EXTI 外设中断挂起位或 NVIC IRQ 通道挂起位，因为与事件线路相对应的挂起位并未设置。可能需要在外设中清除中断标志。

从待机模式退出低功耗模式的方式包括外部复位（NRST 引脚）、IWDG 复位、使能的 WKUPx 引脚

之一的电平变化或 RTC 事件。

从待机模式唤醒后，程序执行将以与复位后相同的方式重新启动（启动引脚采样、选项字节加载、获取复位向量等）。

### 6.3.1 睡眠模式

#### 睡眠模式下的 I/O 状态

睡眠模式下，所有 I/O 保持与运行模式相同的状态。

#### 进入睡眠模式

睡眠模式是根据“进入低功耗模式”一节中的描述进入的，当带 FPU 核心的 Cortex®-M4 系统控制寄存器中的 SLEEPDEEP 位被清除时。

请参考下表：睡眠模式，以获取关于如何进入睡眠模式的详细信息。

有关如何进入睡眠模式的详细信息，请参见下表。

#### 退出睡眠模式

睡眠模式的退出是根据“退出低功耗模式”一节中描述的方法进行的。

请参考下表：睡眠模式，以获取关于如何退出睡眠模式的更多详细信息。

表 6.1 睡眠模式

| 睡眠模式 | 描述                                                                                                                                                                                                                             |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 进入模式 | 当以下情况发生时，使用 WFI（等待中断）或 WFE（等待事件）：<br>– SLEEPDEEP = 0<br>– 没有待处理的中断（对于 WFI）或事件（对于 WFE）<br>请参考带 FPU 核心的 Cortex®-M4 系统控制寄存器                                                                                                       |
|      | 当从 ISR 返回时：<br>– SLEEPDEEP = 0 并且<br>– SLEEPONEXIT = 1<br>– 没有待处理的中断<br>请参考带 FPU 核心的 Cortex®-M4 系统控制寄存器                                                                                                                        |
| 退出模式 | 如果 WFI 或从 ISR 返回被用于进入<br>中断：请参考表 11.1 RX32G410 系列向量表<br>如果 WFE 被用于进入且 SEVONPEND = 0：<br>唤醒事件：请参考第 12.3.2 节：唤醒事件管理<br>如果 WFE 被用于进入且 SEVONPEND = 1：<br>即使 NVIC 中已禁用中断，也可能产生中断：请参考表 10.1 RX32G410 系列向量表或唤醒事件：请参考第 12.3.2 节：唤醒事件管理 |
| 唤醒延迟 | 无                                                                                                                                                                                                                              |

### 6.3.2 停止 0 模式

停止 0 模式基于带 FPU 核心的 Cortex®-M4 深度睡眠模式基础上结合了外设的时钟控制机制。该模式下，内部 LDO 配置为正常模式。关闭除 LSI 以外所有时钟。内部 SRAM 和寄存器内容都将保留。

#### 停止 0 模式下的 I/O 状态

在停止 0 模式下，所有 I/O 引脚保持与运行模式相同的状态。

#### 进入停止 0 模式

停止 0 模式是根据“进入低功耗模式”一节中描述的方法进入的，此时带 FPU 核心的 Cortex®-M4 系统控制寄存器中的 SLEEPDEEP 位被设置。

请参考下表：停止 0 模式，以获取关于如何进入停止 0 模式的详细信息。

如果正在进行 Flash 写入操作，Flash 的操作完成后，MCU 才进入停止 0 模式。

如果正在进行对 APB 的访问，APB 访问完成后，MCU 才进入停止 0 模式。

在停止 0 模式下，可以通过编程单个控制位来选择以下功能：

- 独立看门狗 (IWDG)：通过向其 Key 寄存器写入或通过硬件选项来启动 IWDG。一旦启动，除非进行复位，否则无法停止。请参阅第 29 章节：独立看门狗 IWDG。
- 实时时钟 (RTC)：通过 RTC 域控制寄存器 (RCC\_BDCR) 中的 RTCEN 位进行配置。
- 内部 RC 振荡器 (LSI)：通过控制/状态寄存器 (RCC\_CSR) 中的 LSION 位进行配置。

在停止 0 模式下，多个外设可以使用，并可能增加功耗，如果它们被使能并通过 LSI 进行时钟驱动，或者当它们请求 HSI 时钟时：I2Cx (x=1,2)、UARTx (x=1,2...5)。

DACx (x=1,2,3)、OPAMPs 和 COMP 可以在停止 0 模式下使用，PVD 也可以。如果它们不需要，则必须通过软件禁用以节省功耗。

ADCx (x=1,2,3)、TPS 温度传感器和 VREFBUF 缓冲区在停止 0 模式下可能会消耗功率，除非在进入此模式之前将它们禁用。

#### 退出停止 0 模式

停止 0 模式的退出遵循“进入低功耗模式”一节中的说明。

请参考下表：停止 0 模式，以获取关于如何退出停止 0 模式的详细信息。

当内部 LDO 在低功耗模式下运行时，从停止 0 模式唤醒并使用 HSI 时会产生额外的启动延迟。通过在停止 0 模式期间保持内部调节器开启，虽然启动时间减少，但功耗会增加。



表 6.2 停止 0 模式

| 停止 0 模式 | 描述                                                                                                                                                                                                                                                                                                                                                      |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 进入模式    | 当以下情况发生时，使用 WFI（等待中断）或 WFE（等待事件）： <ul style="list-style-type: none"> <li>– SLEEPDEEP = 1（Cortex®-M4 系统控制寄存器）</li> <li>– 没有待处理的中断（对于 WFI）或事件（对于 WFE）</li> <li>– LPMS = “00”（电源控制寄存器 1 (PWR_CR1)）</li> </ul>                                                                                                                                              |
|         | 当从 ISR 返回时： <ul style="list-style-type: none"> <li>– SLEEPDEEP = 1（Cortex®-M4 系统控制寄存器）</li> <li>– SLEEPONEXIT = 1</li> <li>– 没有待处理的中断</li> <li>– LPMS = “00”（电源控制寄存器 1 (PWR_CR1)）</li> </ul>                                                                                                                                                            |
|         | 注：要进入停止 0 模式，所有 EXTI 线挂起位（在挂起寄存器 1 (EXTI_PR1)）和生成唤醒中断的外设标志必须清除。否则，停止 0 模式进入步骤被忽略，程序继续执行。                                                                                                                                                                                                                                                                |
| 退出模式    | <p>如果 WFI 或从 ISR 返回被用于进入任一外部中断线配置为中断模式（在 NVIC 中必须使能相应的 EXTI 中断向量）。中断源可以是外部中断或具备唤醒能力的外设。请参考表 11.1 RX32G410 系列向量表</p> <p>如果 WFE 被用于进入且 SEVONPEND = 0：任一外部中断线配置为事件模式。请参考第 12.3.2 节：唤醒事件管理</p> <p>如果 WFE 被用于进入且 SEVONPEND = 1：任一外部中断线配置为中断模式（即使在 NVIC 中相应的 EXTI 中断向量被禁止）。中断源可以是外部中断或具备唤醒能力的外设。请参考表 11.1 RX32G410 系列向量表</p> <p>唤醒事件：请参考第 12.3.2 节：唤醒事件管理</p> |
| 唤醒延迟    | 最长唤醒时间包括：HSI 唤醒时间+从停止 0 模式的闪存唤醒时间                                                                                                                                                                                                                                                                                                                       |

### 6.3.1 停止 1 模式

停止 1 模式与停止 0 模式相同，但内部 LDO 进入低功耗模式。

参考下表以详细了解如何进入和退出停止 1 模式。

表 6.3 停止 1 模式

| 停止 1 模式 | 描述                                                                                                                                                                                                         |
|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 进入模式    | 当以下情况发生时，使用 WFI（等待中断）或 WFE（等待事件）： <ul style="list-style-type: none"> <li>– SLEEPDEEP = 1（Cortex®-M4 系统控制寄存器）</li> <li>– 没有待处理的中断（对于 WFI）或事件（对于 WFE）</li> <li>– LPMS = “01”（电源控制寄存器 1 (PWR_CR1)）</li> </ul> |
|         | 当从 ISR 返回时： <ul style="list-style-type: none"> <li>– SLEEPDEEP = 1（Cortex®-M4 系统控制寄存器）</li> <li>– SLEEPONEXIT = 1</li> <li>– 没有待处理的中断</li> </ul>                                                           |

|      |                                                                                                                                                                                                                                                                                                                                                          |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|      | - LPMS = “01” (电源控制寄存器 1 (PWR_CR1))<br><br><i>注：要进入停止 1 模式，所有 EXTI 线挂起位（在挂起寄存器 1(EXTI_PR1)）和生成唤醒中断的外设标志必须清除。否则，停止 1 模式进入步骤被忽略，程序继续执行。</i>                                                                                                                                                                                                                |
| 退出模式 | 如果 WFI 或从 ISR 返回被用于进入任一外部中断线配置为中断模式（在 NVIC 中必须使能相应的 EXTI 中断向量）。中断源可以是外部中断或具备唤醒能力的外设。<br>请参考表 11.1 RX32G410 系列向量表<br>如果 WFE 被用于进入且 SEVONPEND = 0：<br>任一外部中断线配置为事件模式。<br>请参考第 12.3.2 节：唤醒事件管理<br>如果 WFE 被用于进入且 SEVONPEND = 1：<br>任一外部中断线配置为中断模式（即使在 NVIC 中相应的 EXTI 中断向量被禁止）。中断源可以是外部中断或具备唤醒能力的外设。<br>请参考表 11.1 RX32G410 系列向量表<br>唤醒事件：请参考第 12.3.2 节：唤醒事件管理 |
| 唤醒延迟 | 最长唤醒时间包括：HSI 唤醒时间和调压器从低功耗模式唤醒的时间+从停止 1 模式的闪存唤醒时间                                                                                                                                                                                                                                                                                                         |

### 6.3.2 待机模式

待机模式可以实现 MCU 的最低功耗。该模式是在带 FPU 核心的 Cortex®-M4 深度睡眠模式时关闭内部线性稳压器 LDO1P7 和 LDO1P2。PLL、HSI 和 HSE 振荡器也被关闭。

SRAM 和寄存器内容会丢失，除了备份域和待机电路中的寄存器。

#### 待机模式下的 I/O 状态

在待机模式下，I/O 可以保持在模拟状态。

PC13 上的 RTC 侵入在待机模式下是可用的。4 个唤醒引脚（WKUPx, x=1,2,3,4）也是可用的。

#### 进入待机模式

待机模式是根据“进入低功耗模式”一节中的说明进入的，当带 FPU 核心的 Cortex®-M4 系统控制寄存器中的 SLEEPDEEP 位被设置时。

请参考下表：待机模式，以获取关于如何进入待机模式的详细信息。

在待机模式下，可以通过对各控制位进行编程来选择以下功能：

- 独立看门狗 (IWDG)：通过向其 Key 寄存器写入或通过硬件选项来启动 IWDG。一旦启动，除非进行复位，否则无法停止。请参阅第 29 章节：独立看门狗 IWDG。
- 实时时钟 (RTC)：通过 RTC 域控制寄存器 (RCC\_BDCR) 中的 RTCEN 位进行配置。
- 内部 RC 振荡器 (LSI)：通过控制/状态寄存器 (RCC\_CSR) 中的 LSION 位进行配置。

#### 退出待机模式

待机模式的退出遵循“进入低功耗模式”一节中的说明。电源状态寄存器 1 (PWR\_SR1) 中的 SBF 状态标志指示 MCU 处于待机模式。从待机模式唤醒后，除了电源状态寄存器 1 (PWR\_SR1) 外，所有寄存器都将被重置。

请参考下表：待机模式，以获取关于如何退出待机模式的更多详细信息。

表 6.4 待机模式

| 待机模式 | 描述                                                                                                                                                                                                                                                                                  |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 进入模式 | 当以下情况发生时，使用 WFI（等待中断）或 WFE（等待事件）： <ul style="list-style-type: none"> <li>– SLEEPDEEP = 1（Cortex®-M4 系统控制寄存器）</li> <li>– 没有待处理的中断（对于 WFI）或事件（对于 WFE）</li> <li>– LPMS = “11”（电源控制寄存器 1（PWR_CR1））</li> <li>– WUFx 位被清除（电源状态寄存器 1（PWR_SR1））</li> </ul>                                  |
|      | 当从 ISR 返回时： <ul style="list-style-type: none"> <li>– SLEEPDEEP = 1（Cortex®-M4 系统控制寄存器）</li> <li>– SLEEPONEXIT = 1</li> <li>– 没有待处理的中断</li> <li>– LPMS = “11”（电源控制寄存器 1（PWR_CR1））</li> <li>– WUFx 位被清除（电源状态寄存器 1（PWR_SR1））</li> <li>– RTC 标志对应的选择唤醒源（RTC 闹钟、RTC 唤醒、侵入）被清除</li> </ul> |
| 退出模式 | WKUP 唤醒引脚的电平变化、RTC 事件、外部复位（NRST 引脚）、IWDG 复位                                                                                                                                                                                                                                         |
| 唤醒延迟 | 复位阶段                                                                                                                                                                                                                                                                                |

## 6.4 低功耗模式下的自动唤醒（AWU）

RTC 可以在不需要依赖外部中断的情况下唤醒低功耗模式下的微控制器（自动唤醒模式）。RTC 提供一个可编程的时间基数，用于周期性从停止（0 或 1）或待机模式下唤醒。RTC 时钟源选择 LSI 可以实现此功能。为此，可以通过在 RTC 域控制寄存器（RCC\_BDCR）中编程 RTCSEL[1:0]位来选择 LSI 作为 RTC 时钟源：

- 低功耗内部 RC 振荡器（LSI）

使用该时钟源，节省了一个 32.768kHz 晶振的成本。但是 RC 振荡器将少许增加电源消耗。

为了用 RTC 闹钟事件将系统从停止模式下唤醒，必须进行如下操作：

- 配置外部中断线 17 为上升沿触发。
- 配置 RTC 使其可产生 RTC 闹钟事件。

如果要从待机模式中唤醒，不必配置外部中断线 17。

## 6.5 PWR 寄存器描述

可以用半字（16 位）或字（32 位）的方式操作这些外设寄存器。

### 6.5.1 电源控制寄存器（PWR\_CR1）

地址偏移：0x00

复位值：0x0000 0000

从待机模式唤醒后，此寄存器将被重置。

|          |    |    |    |    |    |    |    |           |    |    |    |    |    |    |    |
|----------|----|----|----|----|----|----|----|-----------|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23        | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |    |    |           |    |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7         | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    |    |    |    |    | Reserved  |    |    |    |    |    |    |    |
|          |    |    |    |    |    |    |    | DBP       |    |    |    |    |    |    |    |
|          |    |    |    |    |    |    |    | rw        |    |    |    |    |    |    |    |
|          |    |    |    |    |    |    |    | LPMS[1:0] |    |    |    |    |    |    |    |
|          |    |    |    |    |    |    |    |           |    |    |    | rw |    |    |    |

Bit 31:9 保留，必须保持复位值。

Bit 8 DBP：取消后备区域的写保护

在复位后，RTC 和后备寄存器处于被保护状态以防意外写入。

设置这位允许写入这些寄存器。

0：禁止写入 RTC 和后备寄存器

1：允许写入 RTC 和后备寄存器

注：如果 RTC 的时钟是 HSE/128，该位必须保持为 ‘1’。

Bit 7:2 保留，必须保持复位值。

Bit 1:0 LPMS[1:0]：低功耗模式选择

00：停止 0 模式

01：停止 1 模式

10：保留

11：待机模式

### 6.5.2 电源控制寄存器（PWR\_CR2）

地址偏移：0x04

复位值：0x0000 0000

从待机模式唤醒后，此寄存器将被重置。

|          |    |    |    |    |    |    |    |    |    |    |    |          |    |      |    |
|----------|----|----|----|----|----|----|----|----|----|----|----|----------|----|------|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19       | 18 | 17   | 16 |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |          |    |      |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3        | 2  | 1    | 0  |
| Reserved |    |    |    |    |    |    |    |    |    |    |    | PLS[2:0] |    | PVDE |    |
|          |    |    |    |    |    |    |    |    |    |    |    | rw       | rw | rw   | rw |

Bit 31:4 保留，必须保持复位值。

Bit 3:1 PLS[2:0]：PVD 电平选择

这些位用于选择电源电压监测器的电压阈值

000：2.2V

100：2.6V

001：2.3V

101：2.7V

010：2.4V

110：2.8V

011：2.5V

111：2.9V

注：当SYSCFG\_CFGR2 寄存器的PVDL 位被设置时此位被写保护。

此保护仅能通过系统复位来重置。

Bit 0 **PVDE**：电源电压监测器(PVD)使能

0：禁止 PVD

1：开启 PVD

注：当SYSCFG\_CFGR2 寄存器的PVDL 位被设置时此位被写保护。

此保护仅能通过系统复位来重置。

### 6.5.3 电源控制寄存器 (PWR\_CR3)

地址偏移：0x08

复位值：0x0000 0000

退出待机模式以及 RCC\_APB1RSTR1 寄存器中的 PWRRST 位被设置时，该寄存器不会被重置。

访问：访问此寄存器需要比标准 APB 访问更多的 APB 周期（写操作需要 3 个周期，读操作需要 2 个周期）。

|          |          |    |    |    |    |    |    |    |    |    |    |       |       |       |       |
|----------|----------|----|----|----|----|----|----|----|----|----|----|-------|-------|-------|-------|
| 31       | 30       | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19    | 18    | 17    | 16    |
| Reserved |          |    |    |    |    |    |    |    |    |    |    |       |       |       |       |
| 15       | 14       | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3     | 2     | 1     | 0     |
| EWUL     | Reserved |    |    |    |    |    |    |    |    |    |    | EWUP4 | EWUP3 | EWUP2 | EWUP1 |
| rw       |          |    |    |    |    |    |    |    |    |    |    | rw    | rw    | rw    | rw    |

Bit 31:16 保留，必须保持复位值。

Bit 15 **EWUL**：使能内部唤醒线（RTC/IWDG）

0：禁止内部唤醒线唤醒

1：使能内部唤醒线

Bit 14:4 保留，必须保持为复位值。

Bit 3 **EWUP4**：使能 WKUP 引脚 4

0：WKUP 引脚 4 为通用 I/O。WKUP 引脚 4 上的事件不能将 CPU 从待机模式唤醒

1：WKUP 引脚 4 用于将 CPU 从待机模式唤醒，WKUP 引脚 4 上的上升沿/下降沿将系统从待机模式唤醒，有效边沿通过 PWR\_CR4 的 WP4 设置。

Bit 2 **EWUP3**：使能 WKUP 引脚 3

0：WKUP 引脚 3 为通用 I/O。WKUP 引脚 3 上的事件不能将 CPU 从待机模式唤醒

1：WKUP 引脚 3 用于将 CPU 从待机模式唤醒，WKUP 引脚 3 上的上升沿/下降沿将系统从待机模式唤醒，有效边沿通过 PWR\_CR4 的 WP3 设置。

Bit 1 **EWUP2**：使能 WKUP 引脚 2

0：WKUP 引脚 2 为通用 I/O。WKUP 引脚 2 上的事件不能将 CPU 从待机模式唤醒

1：WKUP 引脚 2 用于将 CPU 从待机模式唤醒，WKUP 引脚 2 上的上升沿/下降沿将系统从待机模式唤醒，有效边沿通过 PWR\_CR4 的 WP2 设置。

Bit 0 **EWUP1**：使能 WKUP 引脚 1

0：WKUP 引脚 1 为通用 I/O。WKUP 引脚 1 上的事件不能将 CPU 从待机模式唤醒

1：WKUP 引脚 1 用于将 CPU 从待机模式唤醒，WKUP 引脚 1 上的上升沿/下降沿将系统从待机模式唤醒，有效边沿通过 PWR\_CR4 的 WP1 设置。

## 6.5.4 电源控制寄存器 (PWR\_CR4)

地址偏移: 0x0C

复位值: 0x0000 0000

退出待机模式以及 RCC\_APB1RSTR1 寄存器中的 PWRRST 位被设置时, 该寄存器不会被重置。

访问: 访问此寄存器需要比标准 APB 访问更多的 APB 周期 (写操作需要 3 个周期, 读操作需要 2 个周期)。

|          |    |    |    |    |    |    |    |    |    |    |    |     |     |     |     |
|----------|----|----|----|----|----|----|----|----|----|----|----|-----|-----|-----|-----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19  | 18  | 17  | 16  |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |     |     |     |     |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3   | 2   | 1   | 0   |
| Reserved |    |    |    |    |    |    |    |    |    |    |    | WP4 | WP3 | WP2 | WP1 |
|          |    |    |    |    |    |    |    |    |    |    |    | rw  | rw  | rw  | rw  |

Bits 31:4 保留, 必须保持为复位值。

Bit 3 **WP4**: WKUP4 引脚唤醒极性

此位定义了用于外部唤醒引脚上的事件检测极性, WKUP4

0: 检测到高电平 (上升沿)

1: 检测到低电平 (下降沿)

Bit 2 **WP3**: WKUP3 引脚唤醒极性

此位定义了用于外部唤醒引脚上的事件检测极性, WKUP3

0: 检测到高电平 (上升沿)

1: 检测到低电平 (下降沿)

Bit 1 **WP2**: WKUP2 引脚唤醒极性

此位定义了用于外部唤醒引脚上的事件检测极性, WKUP2

0: 检测到高电平 (上升沿)

1: 检测到低电平 (下降沿)

Bit 0 **WP1**: WKUP1 引脚唤醒极性

此位定义了用于外部唤醒引脚上的事件检测极性, WKUP1

0: 检测到高电平 (上升沿)

1: 检测到低电平 (下降沿)



### 6.5.5 电源状态寄存器 1 (PWR\_SR1)

地址偏移: 0x10

复位值: 0x0000 0000

退出待机模式以及 RCC\_APB1RSTR1 寄存器中的 PWRRST 位被设置时, 该寄存器不会被重置。

访问: 与此标准 APB 读取相比, 读取此寄存器需要额外的 2 个 APB 周期。

|          |          |    |    |    |    |    |     |          |    |    |    |      |      |      |      |
|----------|----------|----|----|----|----|----|-----|----------|----|----|----|------|------|------|------|
| 31       | 30       | 29 | 28 | 27 | 26 | 25 | 24  | 23       | 22 | 21 | 20 | 19   | 18   | 17   | 16   |
| Reserved |          |    |    |    |    |    |     |          |    |    |    |      |      |      |      |
| 15       | 14       | 13 | 12 | 11 | 10 | 9  | 8   | 7        | 6  | 5  | 4  | 3    | 2    | 1    | 0    |
| WUFI     | Reserved |    |    |    |    |    | SBF | Reserved |    |    |    | WUF4 | WUF3 | WUF2 | WUF1 |
| r        |          |    |    |    |    |    | r   |          |    |    |    | r    | r    | r    | r    |

Bit 31:16 保留, 必须保持复位值。

Bit 15 **WUFI**: 内部唤醒标志 (IWDG/RTC 唤醒)

当在内部唤醒线上检测到唤醒时, 设置此位。当所有内部唤醒源被清除时该位被清除。

Bit 14:9 保留, 必须保持复位值。

Bit 8 **SBF**: 待机标志

该位由硬件设置, 并只能由 POR/PDR (上电/掉电复位) 或设置电源状态清除寄存器 (PWR\_SCR) 的 CSBF 位清除。

0: 系统不在待机模式

1: 系统进入待机模式

Bit 7:4 保留, 必须保持复位值。

Bit 3 **WUF4**: WKUP4 引脚唤醒标志

当在唤醒引脚 WKUP4 检测到一个唤醒事件时此位被设置, 被清除通过向电源状态清除寄存器 (PWR\_SCR) 的 CWUF4 位写 “1”。

Bit 2 **WUF3**: WKUP3 引脚唤醒标志

当在唤醒引脚 WKUP3 检测到一个唤醒事件时此位被设置, 被清除通过向电源状态清除寄存器 (PWR\_SCR) 的 CWUF3 位写 “1”。

Bit 1 **WUF2**: WKUP2 引脚唤醒标志

当在唤醒引脚 WKUP2 检测到一个唤醒事件时此位被设置, 被清除通过向电源状态清除寄存器 (PWR\_SCR) 的 CWUF2 位写 “1”。

Bit 0 **WUF1**: WKUP1 引脚唤醒标志

当在唤醒引脚 WKUP1 检测到一个唤醒事件时此位被设置, 被清除通过向电源状态清除寄存器 (PWR\_SCR) 的 CWUF1 位写 “1”。

### 6.5.6 电源状态寄存器 2 (PWR\_SR2)

地址偏移: 0x14

复位值: 0x0000 0000

|          |    |    |    |      |          |    |    |    |    |    |    |    |    |    |    |
|----------|----|----|----|------|----------|----|----|----|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27   | 26       | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |      |          |    |    |    |    |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11   | 10       | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    | PVDO | Reserved |    |    |    |    |    |    |    |    |    |    |
|          |    |    |    | r    |          |    |    |    |    |    |    |    |    |    |    |

Bit 31:12 保留, 必须保持复位值。

Bit 11 PVDO: PVD 输出

当 PVD 被 PVDE 位使能后该位才有效

0:  $V_{DD}/V_{DDA}$  高于由 PLS[2:0]选定的 PVD 阈值

1:  $V_{DD}/V_{DDA}$  低于由 PLS[2:0]选定的 PVD 阈值

注: 在待机模式下 PVD 被停止。因此, 待机模式后或复位后, 直到设置 PVDE 位之前, 该位为 0。

Bit 10:0 保留, 必须保持复位值。

### 6.5.7 电源状态清除寄存器 (PWR\_SCR)

地址偏移: 0x18

复位值: 0x0000 0000

访问: 与此标准 APB 写入相比, 写入此寄存器需要额外的 3 个 APB 周期。

|          |          |    |    |    |    |    |      |          |    |    |    |       |       |       |       |
|----------|----------|----|----|----|----|----|------|----------|----|----|----|-------|-------|-------|-------|
| 31       | 30       | 29 | 28 | 27 | 26 | 25 | 24   | 23       | 22 | 21 | 20 | 19    | 18    | 17    | 16    |
| Reserved |          |    |    |    |    |    |      |          |    |    |    |       |       |       |       |
| 15       | 14       | 13 | 12 | 11 | 10 | 9  | 8    | 7        | 6  | 5  | 4  | 3     | 2     | 1     | 0     |
| CWUFI    | Reserved |    |    |    |    |    | CSBF | Reserved |    |    |    | CWUF4 | CWUF3 | CWUF2 | CWUF1 |
| w        |          |    |    |    |    |    | w    |          |    |    |    | w     | w     | w     | w     |

Bit 31:16 保留, 必须保持复位值。

Bit 15 CWUFI: 清除内部唤醒标志

写 1 清除 PWR\_SR1 寄存器的内部唤醒标志 WUFI

Bit 14:9 保留, 必须保持复位值。

Bit 8 CSBF: 清除待机标志

写 1 清除 PWR\_SR1 寄存器的待机标志 SBF

Bit 7:4 保留, 必须保持复位值。

Bit 3 CWUF4: 清除唤醒标志 4

写 1 清除 PWR\_SR1 寄存器的 WKUP4 引脚唤醒标志 WUF4

Bit 2 CWUF3: 清除唤醒标志 3

写 1 清除 PWR\_SR1 寄存器的 WKUP3 引脚唤醒标志 WUF3

Bit 1 CWUF2: 清除唤醒标志 2

写 1 清除 PWR\_SR1 寄存器的 WKUP2 引脚唤醒标志 WUF2

Bit 0 CWUF1: 清除唤醒标志 1

写 1 清除 PWR\_SR1 寄存器的 WKUP1 引脚唤醒标志 WUF1

## 7 备份寄存器 (BKP)

### 7.1 BKP 简介

此外, BKP 控制寄存器用来管理侵入检测和 RTC 校准功能。

复位后, 对备份寄存器和 RTC 的访问被禁止, 并且备份域被保护以防止可能存在的意外的写操作。执行以下操作可以使能对备份寄存器和 RTC 的访问。

- 通过设置寄存器 RCC\_APB1ENR 的 PWREN 和 BKPEN 位来打开电源和后备接口的时钟。
- 电源控制寄存器 (PWR\_CR) 的 DBP 位来使能对后备寄存器和 RTC 的访问。

### 7.2 BKP 特性

- 84 字节数据后备寄存器
- 用来管理防侵入检测并具有中断功能的状态/控制寄存器
- 用来存储 RTC 校验值的校验寄存器
- 在 PC13 引脚 (当该引脚不用于侵入检测时) 上输出 RTC 校准时钟, RTC 闹钟脉冲或者秒脉冲

### 7.3 BKP 功能描述

#### 7.3.1 侵入检测

当 TAMPER 引脚上的信号从 ‘0’ 变成 ‘1’ 或者从 ‘1’ 变成 ‘0’ (取决于备份控制寄存器 BKP\_CR 的 TPAL 位), 会产生一个侵入检测事件。侵入检测事件将所有数据备份寄存器内容清除。然而为了避免丢失侵入事件, 侵入检测信号是边沿检测的信号与侵入检测允许位的逻辑与, 从而在侵入检测引脚被允许前发生的侵入事件也可以被检测到。

- 当 TPAL=0 时: 如果在启动侵入检测 TAMPER 引脚前 (通过设置 TPE 位) 该引脚已经为高电平, 一旦启动侵入检测功能, 则会产生一个额外的侵入事件 (尽管在 TPE 位置 ‘1’ 后并没有出现上升沿)。
- 当 TPAL=1 时: 如果在启动侵入检测引脚 TAMPER 前 (通过设置 TPE 位) 该引脚已经为低电平, 一旦启动侵入检测功能, 则会产生一个额外的侵入事件 (尽管在 TPE 位置 ‘1’ 后并没有出现下降沿)。

设置 BKP\_CSR 寄存器的 TPIE 位为 ‘1’, 当检测到侵入事件时就会产生一个中断。

在一个侵入事件被检测到并被清除后, 侵入检测引脚 TAMPER 应该被禁止。然后, 在再次写入备份数据寄存器前重新用 TPE 位启动侵入检测功能。这样, 可以阻止软件在侵入检测引脚上仍然有侵入事件时对备份数据寄存器进行写操作。这相当于对侵入引脚 TAMPER 进行电平检测。

#### 7.3.2 RTC 校准

为方便测量, RTC 时钟可以经 64 分频输出到侵入检测引脚 TAMPER 上。通过设置 RTC 校验寄存器 (BKP\_RTCCR) 的 CCO 位来开启这一功能。

通过配置 CAL[6:0]位, 此时钟可以最多减慢 121ppm。

### 7.4 BKP 寄存器描述

#### 7.4.1 RTC 时钟校准寄存器 (BKP\_RTCCR)

地址偏移: 0x2C

复位值：0x0000

访问：无等待，支持字，半字访问

| 15       | 14 | 13 | 12 | 11 | 10 | 9    | 8    | 7   | 6        | 5  | 4  | 3  | 2  | 1  | 0  |
|----------|----|----|----|----|----|------|------|-----|----------|----|----|----|----|----|----|
| Reserved |    |    |    |    |    | ASOS | ASOE | CCO | CAL[6:0] |    |    |    |    |    |    |
|          |    |    |    |    |    | rw   | rw   | rw  | rw       | rw | rw | rw | rw | rw | rw |

Bits 15:10 保留，必须保持为复位值。

**Bit 9 ASOS**：闹钟或秒输出选择

当设置了 ASOE 位，ASOS 位可用于选择在 TAMPER 引脚上输出的是 RTC 秒脉冲还是闹钟脉冲信号。

0：输出 RTC 闹钟脉冲

1：输出秒脉冲

注：该位只能被后备区的复位所清除

**Bit 8 ASOE**：闹钟或秒输出使能

根据 ASOS 位的设置，该位允许 RTC 闹钟或秒脉冲输出到 TAMPER 引脚上。

输出脉冲的宽度为一个 RTC 时钟的周期。设置了 ASOE 位时不能开启 TAMPER 的功能。

注：该位只能被后备区的复位所清除

**Bit 7 CCO**：校准时钟输出

0：无影响

1：此位置 1 可以在侵入检测引脚输出经 64 分频后的 RTC 时钟。当 CCO 位置 1 时，必须关闭侵入检测功能以避免检测到无用的侵入信号。

注：当 VDD 供电断开时，该位被清除。

**Bits 6:0 CAL[6:0]**：校准值

校准值表示在每  $2^{20}$  个时钟脉冲内将有多少个时钟脉冲被跳过。这可以用来对 RTC 进行校准，以  $1000000/2^{20}$ ppm 的比例减慢时钟。

RTC 时钟可以被减慢 0~121ppm。

## 7.4.2 备份控制寄存器 (BKP\_CR)

地址偏移：0x30

复位值：0x0000

访问：无等待，支持字，半字访问

| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1    | 0   |
|----------|----|----|----|----|----|---|---|---|---|---|---|---|---|------|-----|
| Reserved |    |    |    |    |    |   |   |   |   |   |   |   |   | TPAL | TPE |
|          |    |    |    |    |    |   |   |   |   |   |   |   |   | rw   | rw  |

Bits 15:2 保留，必须保持为复位值。

**Bit 1 TPAL**：侵入检测 TAMPER 引脚有效电平

0：侵入检测 TAMPER 引脚上的高电平会清除所有数据备份寄存器（如果 TPE 位为 1）

1：侵入检测 TAMPER 引脚上的低电平会清除所有数据备份寄存器（如果 TPE 位为 1）

**Bit 0 TPE**：侵入检测 TAMPER 引脚使能

0：侵入检测 TAMPER 引脚作为通用 I/O 口使用

1：开启侵入检测引脚作为侵入检测使用

注：同时设置 TPAL 和 TPE 位总是安全的。然而，同时清除两者会产生一个假的侵入事件。因此，推荐只在 TPE 为 0 时才改变 TPAL 位的状态。

### 7.4.3 备份控制/状态寄存器 (BKP\_CSR)

地址偏移: 0x34

复位值: 0x0000

访问: 无等待, 支持字, 半字访问

| 15       | 14 | 13 | 12 | 11 | 10 | 9   | 8   | 7        | 6 | 5 | 4 | 3 | 2    | 1   | 0   |
|----------|----|----|----|----|----|-----|-----|----------|---|---|---|---|------|-----|-----|
| Reserved |    |    |    |    |    | TIF | TEF | Reserved |   |   |   |   | TPIE | CTI | CTE |
|          |    |    |    |    |    | r   | r   |          |   |   |   |   | rw   | w   | w   |

Bits 15:10 保留, 必须保持为复位值。

Bit 9 **TIF**: 侵入中断标志

当检测到有侵入事件且 TPIE 位为 1 时, 此位由硬件置 1。通过向 CTI 位写 1 来清除此标志位 (同时也清除了中断)。如果 TPIE 位被清除, 则此位也会被清除。

0: 无侵入中断

1: 已产生侵入中断

注意: 仅当系统复位或由待机模式唤醒后才复位该位

Bit 8 **TEF**: 侵入事件标志

当检测到侵入事件时此位由硬件置 1。通过向 CTE 位写 1 可清除此标志位。

0: 无侵入事件

1: 已检测到侵入事件

Bits 7:3 保留, 必须保持为复位值。

Bit 2 **TPIE**: 侵入 TAMPER 引脚中断使能

0: 禁止侵入检测中断

1: 允许侵入检测中断 (BKP\_CR 寄存器的 TPE 位也必须被置 1)

注 1: 侵入中断无法将系统内核从低功耗模式唤醒。

注 2: 仅当系统复位或由待机模式唤醒后才复位该位。

Bit 1 **CTI**: 清除侵入检测中断

此位只能写入, 读出值为 0。

0: 无效

1: 清除侵入检测中断和 TIF 侵入检测中断标志

Bit 0 **CTE**: 清除侵入检测事件

此位只能写入, 读出值为 0。

0: 无效

1: 清除 TEF 侵入检测事件标志 (并复位侵入检测器)

## 8 复位和时钟控制 (RCC)

### 8.1 复位

RX32G410 系列支持三种复位，分别是系统复位，上电复位和备份区域复位。

#### 8.1.1 系统复位

除了时钟控制器的 RCC\_CSR 寄存器中的复位标志位和备份区域中的寄存器以外，系统复位将复位所有寄存器至它们的复位状态。

当发生以下任一事件时，产生一个系统复位：

1. NRST 引脚上的低电平（外部复位）
2. 窗口看门狗计数终止（WWDG 复位）
3. 独立看门狗计数终止（IWDG 复位）
4. 软件复位（SW 复位）
5. 低功耗管理复位

可通过查看 RCC\_CSR 控制状态寄存器中的复位状态标志位识别复位事件来源。

#### 软件复位

通过将 Cortex®-M4 中断应用和复位控制寄存器中的 SYSRESETREQ 位置 1，可实现软件复位。请参考 Cortex®-M4 技术参考手册获得进一步信息。

#### 低功耗管理复位

在以下两种情况下可产生低功耗管理复位：

1. 在进入待机模式时产生低功耗管理复位：  
通过将用户选择字节中的 nRST\_STDBY 位清零将使能该复位。这时，即使执行了进入待机模式的过程，系统将被复位而不是进入待机模式。
2. 在进入停止模式时产生低功耗管理复位：  
通过将用户选择字节中的 nRST\_STOP 位清零将使能该复位。这时，即使执行了进入停止模式的过程，系统将被复位而不是进入停机模式。

#### 8.1.2 电源复位

当以下事件中之一发生时，产生电源复位：

1. 上电/掉电复位（POR/PDR 复位）
2. 从待机模式中返回

电源复位将复位除了备份区域外的所有寄存器。

下图中复位源将最终作用于 RESET 引脚，并在复位过程中保持低电平。复位入口矢量被固定在地址 0x0000\_0004。

芯片内部的复位信号会在 NRST 引脚上输出，脉冲发生器保证每一个（外部或内部）复位源都能有至少 20μs 的脉冲延时；当 NRST 引脚被拉低产生外部复位时，它将产生复位脉冲。



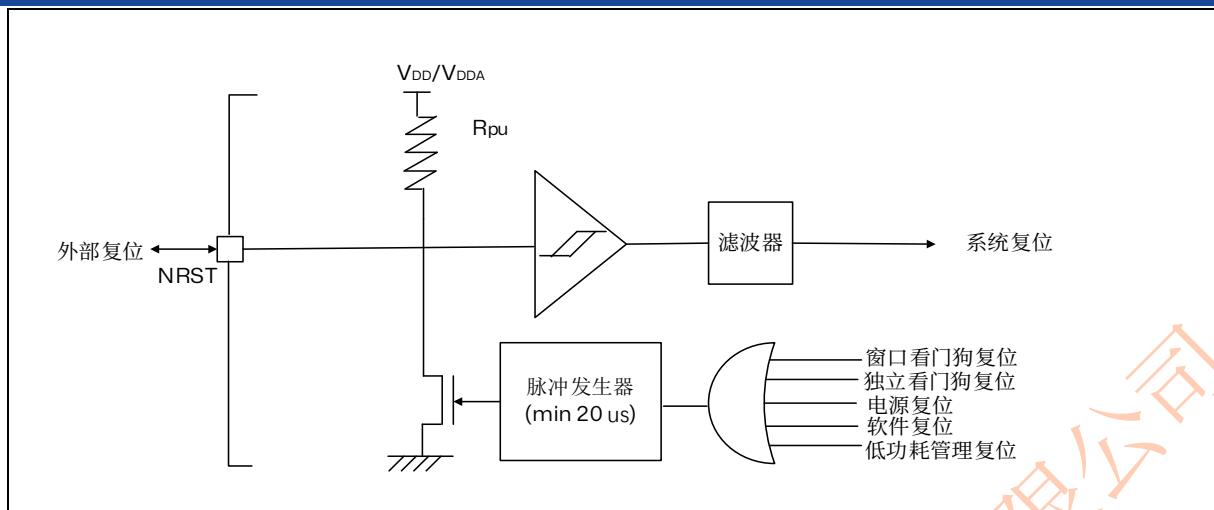


图 8.1 复位电路框图

### 8.1.3 备份域复位

备份区域拥有两个专门的复位，它们只影响备份区域。

当以下事件之一发生时，产生备份区域复位。

1. 软件复位，备份区域复位可由设置备份域控制寄存器 (RCC\_BDCR) 中的 BDRST 位产生。
2. 在  $V_{DD}$  掉电的前提下， $V_{DD}$  上电将引发备份区域复位。

### 8.2 时钟

4 种不同的时钟源可被用来驱动系统时钟 (SYSCLK)：

- HSI
- LSI
- PLL
- HSE

该芯片还有 1 种二级时钟源：

- 32kHz 低速内部 RC，可以用于驱动独立看门狗和通过程序选择驱动 RTC。RTC 用于从停机/待机模式下自动唤醒系统。

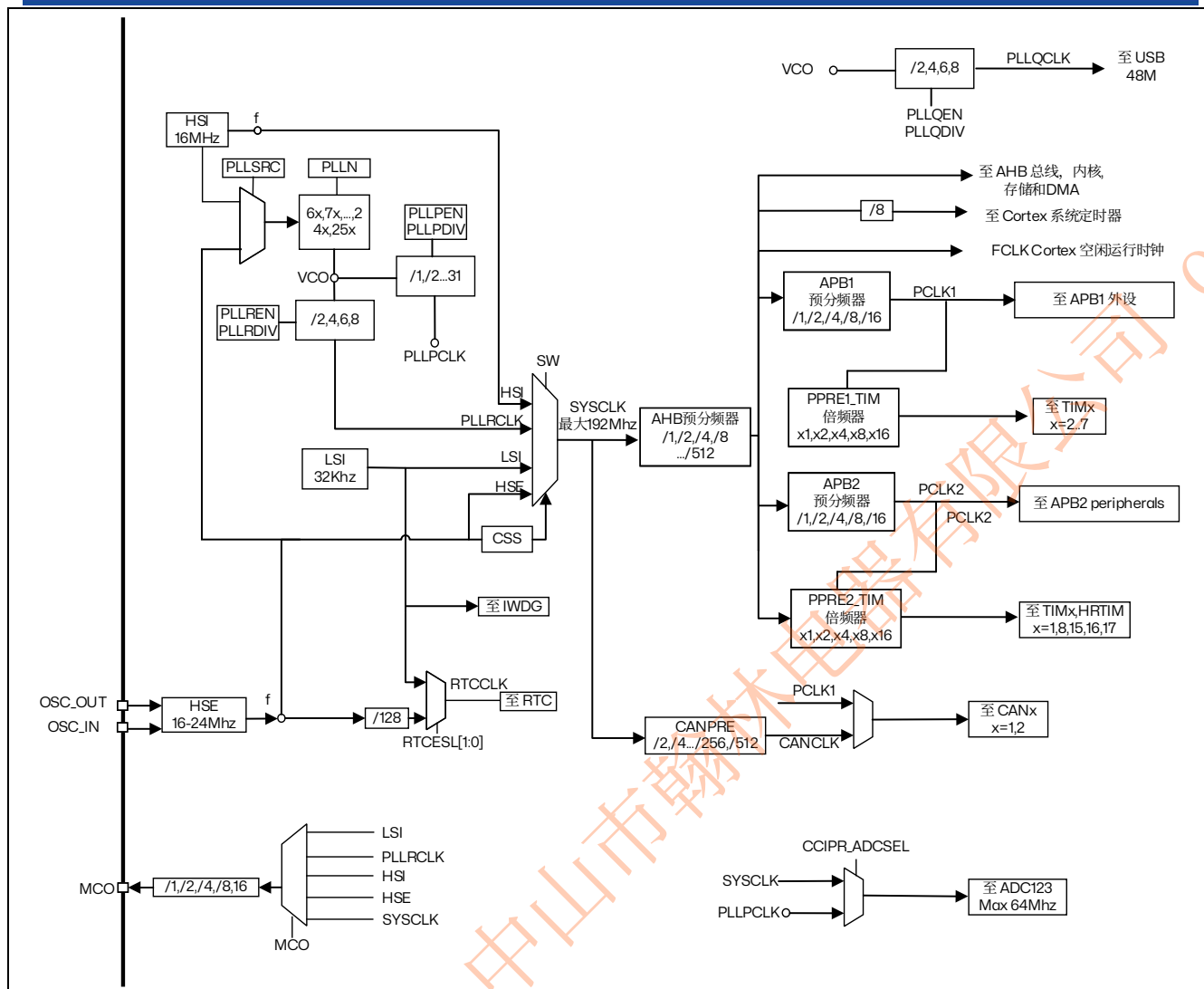


图 8.2 时钟树

用户可通过多个预分频器配置 AHB、APB1 和 APB2 域的频率，最大允许频率是 192MHz。

通过对 SysTick 控制与状态寄存器的设置，可选择 AHB 时钟 (HCLK) 8 分频后或 Cortex (HCLK) 时钟作为 SysTick 时钟。ADC 时钟由 SYSCLK 或 PLLPCLK 产生。

定时器时钟频率分配由硬件按以下 2 种情况自动设置：

1. 如果相应的 APB 预分频系数是 1，定时器的时钟频率与所在 APB 总线频率一致。
2. 否则，定时器的时钟频率被设为与其相连的 APB 总线频率的 2 倍。

PLLCLK 允许的最小频率为 12MHz。

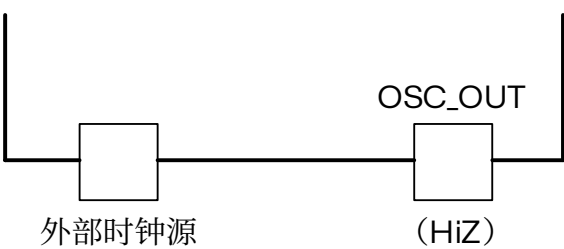
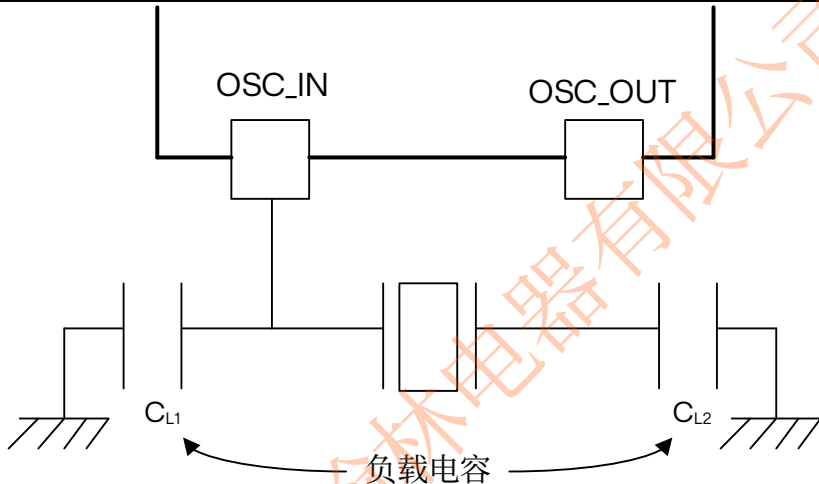
FCLK 是 Cortex®-M4 的自由运行时钟。

### 8.2.1 HSE 时钟

高速外部时钟信号 (HSE) 由以下两种时钟源产生：

- HSE 外部晶体/陶瓷谐振器
- HSE 用户外部时钟

为了减少时钟输出的失真和缩短启动稳定时间，晶体/陶瓷谐振器和负载电容器必须尽可能地靠近振荡器引脚。负载电容值必须根据所选择的振荡器来调整。

|          | 硬件配置                                                                                |
|----------|-------------------------------------------------------------------------------------|
| 外部时钟     |   |
| 晶体/陶瓷谐振器 |  |

#### 外部时钟源（HSE 旁路）

在这个模式里，必须提供外部时钟。它的频率最高可达 50MHz。用户可通过设置在时钟控制寄存器中的 HSEBYP 和 HSEON 位来选择这一模式。外部时钟信号（50%占空比的方波、正弦波或三角波）必须连到 SOC\_IN 引脚，同时保证 OSC\_OUT 引脚悬空。

#### 外部晶体/陶瓷谐振器（HSE 晶体）

16~24Mz 外部振荡器可为系统提供非常精确的主时钟。进一步信息可参考数据手册的电气特性部分。

在时钟控制寄存器（RCC\_CR）中的 HSERDY 位用来指示高速外部振荡器是否稳定。在启动时，直到这一位被硬件置‘1’，时钟才被释放出来。如果在时钟中断寄存器（RCC\_CIR）中允许产生中断，将会产生相应中断。HSE 晶体可以通过设置时钟控制寄存器（RCC\_CR）中的 HSEON 位被启动和关闭。

### 8.2.2 HSI 时钟

HSI 时钟信号由内部 16MHz 的 RC 振荡器产生，可直接作为系统时钟或在 2 分频后作为 PLL 输入。HSI RC 振荡器能够在不需要任何外部器件的条件下提供系统时钟。

#### 校准

制造工艺决定了不同芯片的 RC 振荡器频率会不同，这就是为什么每个芯片的 HSI 时钟频率在出厂前已经被校准到 1.5%（25℃）的原因。系统复位时，工厂校准值被装载到时钟控制寄存器的 HSICAL[7:0] 位。

如果用户的应用基于不同的电压或环境温度，这将会影响 RC 振荡器的精度。可以通过时钟控制寄存器里的 HSITRIM[4:0]位来调整 HSI 频率。

时钟控制寄存器中的 HSI RDY 位用来指示 HSI RC 振荡器是否稳定。在时钟启动过程中，直到这一位被硬件置‘1’，HSI RC 输出时钟才被释放。HSI RC 可由时钟控制寄存器中的 HSION 位来启动和关闭。

### 8.2.3 PLL

内部 PLL 可以用来倍频 HSI 和 HSE RC 的输出时钟。

PLL 的设置（选择 HSI 振荡器或 HSI 振荡器除 2，和选择倍频系数）必须在其被激活前完成。一旦 PLL 被激活，这些参数就不能被改动。

如果 PLL 中断在时钟中断寄存器里被允许，当 PLL 准备就绪时，可产生中断申请。

### 8.2.4 LSI 时钟

LSI RC 担当一个低功耗时钟源的角色，它可以在停机和待机模式下保持运行，为独立看门狗和自动唤醒单元提供时钟。LSI 时钟频率大约 32KHz（在 30KHz 和 60KHz 之间）。进一步信息请参考数据手册中有关电气特性部分。

LSI RC 可以通过控制/状态寄存器（RCC\_CSR）里的 LSION 位来启动或关闭。

在控制/状态寄存器（RCC\_CSR）里的 LSIRDY 位指示低速内部振荡器是否稳定。在启动阶段，直到这个位被硬件设置为‘1’后，此时钟才被释放。如果在时钟中断寄存器（RCC\_CIR）里被允许，将产生 LSI 中断申请。

### 8.2.5 系统时钟（SYSCLK）选择

系统复位后，HSI 振荡器被选为系统时钟。当时钟源被直接或通过 PLL 间接作为系统时钟时，不能被停止。

只有当目标时钟源准备就绪了（经过启动稳定阶段的延迟或 PLL 稳定），从一个时钟源到另一个时钟源的切换才会发生。在被选择时钟源没有就绪时，系统时钟的切换不会发生。直至目标时钟源就绪，才发生切换。

在时钟控制寄存器（RCC\_CR）里的状态位指示哪个时钟已经准备好了，哪个时钟目前被用作系统时钟。

### 8.2.6 时钟安全系统（CSS）

时钟安全系统可通过软件激活。在这种情况下，HSE 振荡器启动延迟后使能时钟检测器，当该振荡器停止时禁用。

如果 HSE 时钟出现故障，HSE 振荡器将自动禁用，并向高级控制定时器（TIM1/TIM8 以及 TIM15/16/17）的 break 输入和 hrtim\_sys\_flt 发送时钟故障事件，并生成中断以通知软件关于该故障（时钟安全系统中断 CSSI），从而使 MCU 执行救援操作。CSSI 与带 FPU 的 Cortex®-M4 的 NMI（不可屏蔽中断）异常向量相关联。

注意：一旦 CSS 被使能，如果 HSE 时钟失败，将发生 CSS 中断并自动生成 NMI。NMI 将无限期地执行，除非清除 CSS 中断挂起位。因此，在 NMI ISR 中，用户必须通过设置时钟中断清除寄存器（RCC\_CICR）中的 CSSC 位来清除 CSS 中断。

如果 HSE 振荡器直接或间接用作系统时钟（间接意味着：它用作 PLL 输入时钟，而 PLL 时钟用作系统时钟），则检测到的故障将导致系统时钟切换到 HSI 振荡器，并禁用 HSE 振荡器。如果 HSE 时钟（是否分频）是 PLL 的时钟输入，而 PLL 用作系统时钟，当故障发生时，PLL 也会被禁用。

### 8.2.7 RTC 时钟

RTC 时钟固定为 LSI，还可以选择 HSE 的 1287 分频。

### 8.2.8 看门狗时钟

如果独立看门狗已经由硬件选项或软件启动，LSI 振荡器将被强制在打开状态，并且不能被关闭。在 LSI 振荡器稳定后，时钟供应给 IWDG。

### 8.2.9 时钟输出

微控制器允许输出时钟信号到外部 MCO 引脚。

相应的 GPIO 端口寄存器必须被配置为相应功能。以下四个时钟信号可被选作 MCO 时钟：

- SYSCLK
- HSI
- PLLCLK
- LSI
- HSE

时钟的选择由时钟配置寄存器（RCC\_CFGR）中的 MCO[2:0]位控制。

## 8.3 RCC 寄存器描述

### 8.3.1 时钟控制寄存器 (RCC\_CR)

地址偏移: 0x00

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

| 31           | 30 | 29           | 28 | 27       | 26      | 25      | 24    | 23       | 22 | 21 | 20       | 19    | 18      | 17      | 16    |
|--------------|----|--------------|----|----------|---------|---------|-------|----------|----|----|----------|-------|---------|---------|-------|
| HSE_CTR[1:0] |    | HSI_CTR[1:0] |    | Reserved |         | PLL RDY | PLLON | Reserved |    |    | HSE BIAS | CSSON | HSE BYP | HSE RDY | HSEON |
| rw           | rw | rw           | rw |          |         | r       | rw    |          |    |    | rw       | rs    | rw      | r       | rw    |
| 15           | 14 | 13           | 12 | 11       | 10      | 9       | 8     | 7        | 6  | 5  | 4        | 3     | 2       | 1       | 0     |
| Reserved     |    |              |    |          | HSI RDY | Res     | HSION | Reserved |    |    |          |       |         |         |       |
|              |    |              |    |          | r       |         | rw    |          |    |    |          |       |         |         |       |

Bit 31:30 **HSE\_CTR[1:0]**: HSE 时钟滤波控制位

00: 0.0ns (默认)

01: 0.9ns

10: 4.1ns

11: 6.1ns

Bit 29:28 **HSI\_CTR[1:0]**: HSI 时钟滤波控制位

00: 0.0ns (默认)

01: 0.9ns

10: 4.1ns

11: 6.1ns

Bit 27:26 保留, 必须保持复位值。

Bit 25 **PLL RDY**: PLL 时钟就绪标志

PLL (VCO) 锁定后由硬件置“1”。

0: PLL 未锁定;

1: PLL 锁定。

Bit 24 **PLLON**: PLL 使能

由软件置“1”或清零。

当进入待机和停止模式时, 该位由硬件清零。当 PLL 时钟被用作或被选择将要作为系统时钟时, 该位不能被清零。

0: PLL 关闭;

1: PLL 使能。

Bit 23:21 保留, 必须保持复位值。

Bit 20 **HSEBIAS**: HSE BIAS 控制位

当 CMUOPT=1 时该寄存器写入有效

1: HSE BIAS 电阻连接 (默认)

0: HSE BIAS 电阻断开

Bit 19 **CSSON**: 时钟安全系统使能

由软件置“1”或清零以使能时钟监测器。

0: 时钟监测器关闭;

1: 如果外部 12-24MHz 振荡器就绪, 时钟监测器开启。



**Bit 18 HSEBYP: 外部高速时钟旁路**

在调试模式下由软件置“1”或清零来旁路外部晶体振荡器。只有在外部振荡器关闭的情况下，才能写入该位。

- 0: 外部振荡器没有旁路;
- 1: 外部振荡器被旁路。

**Bit 17 HSERDY: 外部高速时钟就绪标志**

由硬件置“1”来指示外部振荡器已经稳定。在 HSEON 位清零后，该位需要 6 个外部振荡器周期清零。

- 0: 外部振荡器没有就绪;
- 1: 外部振荡器就绪。

**Bit 16 HSEON: 外部高速时钟使能**

由软件置“1”或清零。

当进入待机和停止模式时，该位由硬件清零，关闭外部振荡器。当外部振荡器被用作或被选择将要作为系统时钟时，该位不能被清零。

- 0: HSE 振荡器关闭;
- 1: HSE 振荡器开启。

Bit 15:11 保留，必须保持复位值。

**Bit 10 HSIIRDY: 内部高速时钟就绪标志**

由硬件置“1”来指示内部 16MHz 振荡器已经稳定。在 HSION 位清零后，该位需要 6 个内部 16MHz 振荡器周期清零。

- 0: 内部 16MHz 振荡器没有就绪;
- 1: 内部 16MHz 振荡器就绪。

**Bit 8 HSION: 内部高速时钟使能**

由软件置“1”或清零。

当从待机和停止模式返回或用作系统时钟的外部振荡器发生故障时，该位由硬件置‘1’来启动内部 16MHz 的 RC 振荡器。当内部 16MHz 振荡器被直接或间接地用作或被选择将要作为系统时钟时，该位不能被清零。

- 0: 内部 16MHz 振荡器关闭;
- 1: 内部 16MHz 振荡器开启。

Bit 7:0 保留，必须保持复位值。

**8.3.2 内部时钟源寄存器 (RCC\_ICSR)**

地址偏移: 0x04

复位值: 0x0000 0000

访问: 无等待，支持字，半字和字节访问

|          |    |    |    |    |    |    |    |             |    |    |    |    |    |    |    |
|----------|----|----|----|----|----|----|----|-------------|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23          | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |    |    | HSICAL[7:0] |    |    |    |    |    |    |    |
|          |    |    |    |    |    |    |    | r           | r  | r  | r  | r  | r  | r  | r  |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7           | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    |    |    |    |    |             |    |    |    |    |    |    |    |

Bit 31:24 保留，必须保持复位值。

**Bit 23:16 HSICAL[7:0]: 内部高速时钟校准**

这些位 (HSITRIM) 在启动时初始化为工厂编程的 HSI 校准微调值。当写入 HSITRIM

时，HSICAL 将更新为 HSITRIM 和工厂微调值的总和。

Bit 15:0 保留，必须保持复位值。

### 8.3.3 时钟配置寄存器 (RCC\_CFGR)

地址偏移: 0x08

复位值: 0x4000 0000

访问: 0 到 2 个等待周期, 支持字, 半字和字节访问

| 31       | 30          | 29         | 28 | 27          | 26         | 25 | 24 | 23        | 22 | 21 | 20 | 19         | 18     | 17      | 16 |
|----------|-------------|------------|----|-------------|------------|----|----|-----------|----|----|----|------------|--------|---------|----|
| Res      | MCOPRE[2:0] |            |    | MCOSEL[3:0] |            |    |    | Reserved  |    |    |    | CAN<br>SEL | CANPRE |         |    |
|          | rw          | rw         | rw | rw          | rw         | rw | rw |           |    |    |    |            | rw     | rw      | rw |
| 15       | 14          | 13         | 12 | 11          | 10         | 9  | 8  | 7         | 6  | 5  | 4  | 3          | 2      | 1       | 0  |
| Reserved |             | PPRE2[2:0] |    |             | PPRE1[2:0] |    |    | HPRE[3:0] |    |    |    | SWS[1:0]   |        | SW[1:0] |    |
|          |             | rw         | rw | rw          | rw         | rw | rw | rw        | rw | rw | rw | rw         | rw     | rw      |    |

Bit 31 保留，必须保持复位值。

Bits 30:28 **MCOFRE[2:0]**: 微控制器时钟输出预分频器

这些位由软件设置和清除。

强烈建议在使能 MCO 输出之前更改此预分频器。

000: MCO 不分频

001: MCO 分频为 2

010: MCO 分频为 4

011: MCO 分频为 8

100: MCO 分频为 16

其他值: 不允许

Bits 27:24 **MCOSEL[3:0]**: 微控制器时钟输出

这些位由软件设置和清除。

0000: MCO 输出禁用, MCO 上无时钟

0001: 选择 SYSCLK 系统时钟

0010: 保留, 必须保持复位值

0011: 选择 HSI 时钟

0100: 选择 HSE 时钟

0101: 选择 PLL 时钟

0110: 选择 LSI 时钟

其他值: 保留

注意: 在启动期间或 MCO 时钟源切换期间, 此时钟输出可能会有些被截断的周期。

Bit 23:20 保留, 必须保持复位值。

Bits 19 **CANSEL**: CANbus 时钟来源选择

0: PCLK1

1: CANCLK

Bits 18:16 **CANPRE**: CANCLK 时钟分频选择

000: 2 分频

001: 4 分频

010: 8 分频

011: 16 分频

100: 64 分频

101: 128 分频

110: 256 分频

111: 512 分频

Bit 15:14 保留, 必须保持复位值。

Bits 13:11 **PPRE2[2:0]**: 高速 APB 预分频 (APB2)

由软件置 1 或清 0 来控制高速 APB2 时钟 (PCLK2) 的预分频系数。

0xx: HCLK 不分频

100: HCLK 2 分频

101: HCLK 4 分频

110: HCLK 8 分频

111: HCLK 16 分频

Bits 10:8 **PPRE1[2:0]**: 低速 APB 预分频 (APB1)

由软件置 1 或清 0 来控制低速 APB1 时钟 (PCLK1) 的预分频系数。

0xx: HCLK 不分频

100: HCLK 2 分频

101: HCLK 4 分频

110: HCLK 8 分频

111: HCLK 16 分频

Bits 7:4 **HPRE[3:0]**: AHB 预分频 (APB1)

由软件置 1 或清 0 来控制 AHB 时钟的预分频系数。

0xxx: SYSCLK 不分频

1000: SYSCLK 2 分频 1100: SYSCLK 64 分频

1001: SYSCLK 4 分频 1101: SYSCLK 128 分频

1010: SYSCLK 8 分频 1110: SYSCLK 256 分频

1011: SYSCLK 16 分频 1111: SYSCLK 512 分频

Bits 3:2 **SWS[1:0]**: 系统时钟切换状态

由硬件置 1 或清 0 来指示哪一个时钟源被作为系统时钟

00: LSI 用作系统时钟

01: HSI 振荡器用作系统时钟

10: HSE 用作系统时钟

11: PLL 用作系统时钟

Bits 1:0 **SW[1:0]**: 系统时钟切换

由软件设置和清除, 以选择系统时钟源 (SYSCLK)。

在退出停止和待机模式或 HSE 振荡器故障时, 由硬件配置以强制选择 HSI 振荡器。

00: LSI 用作系统时钟

01: HSI 振荡器用作系统时钟

10: HSE 用作系统时钟

11: PLL 用作系统时钟

### 8.3.4 PLL 配置寄存器 (RCC\_PLLCFGR)

地址偏移: 0x0C

复位值: 0x0000 1000

访问: 无等待, 支持字, 半字和字节访问

这个寄存器用于根据以下公式配置 PLL 的时钟输出:

- $f(\text{VCO clock}) = f(\text{PLL 时钟输入}) \times (\text{PLL N} / \text{PLL M})$
- $f(\text{PLL P}) = f(\text{VCO 时钟}) / \text{PLL P}$
- $f(\text{PLL Q}) = f(\text{VCO 时钟}) / \text{PLL Q}$
- $f(\text{PLL R}) = f(\text{VCO 时钟}) / \text{PLL R}$

|              |            |    |    |    |           |    |         |          |           |    |         |          |    |             |         |
|--------------|------------|----|----|----|-----------|----|---------|----------|-----------|----|---------|----------|----|-------------|---------|
| 31           | 30         | 29 | 28 | 27 | 26        | 25 | 24      | 23       | 22        | 21 | 20      | 19       | 18 | 17          | 16      |
| PLLPDIV[4:0] |            |    |    |    | PLLR[1:0] |    | PLL REN | Res      | PLLQ[1:0] |    | PLL QEN | Reserved |    | PLL P       | PLL PEN |
| rw           | rw         | rw | rw | rw | rw        | rw | rw      |          | rw        | rw | rw      |          |    | rw          | rw      |
| 15           | 14         | 13 | 12 | 11 | 10        | 9  | 8       | 7        | 6         | 5  | 4       | 3        | 2  | 1           | 0       |
| Res          | PLL N[6:0] |    |    |    |           |    |         | Reserved |           |    |         |          |    | PLLSRC[1:0] |         |
|              | rw         | rw | rw | rw | rw        | rw | rw      |          |           |    |         |          |    | rw          | rw      |

Bits 31:27 PLLPDIV[4:0]: 主 PLL “P” 分频系数

由软件设置和清除以控制 PLL “P” 频率。PLL “P” 输出时钟频率 = VCO 频率 / PLLPDIV。

00000: PLL “P” 时钟由位 PLLP 控制

00001: 保留

00010: PLL “P” 时钟 = VCO / 2

...

11111: PLL “P” 时钟 = VCO / 31

Bits 26:25 PLLR[1:0]: 主 PLL 的 “R” 时钟 (系统时钟) 分频系数

由软件设置和清除, 以控制主 PLL 输出时钟 PLLCLK 的频率。这个输出可以选择作为系统时钟。这些位只能在 PLL 被禁用时写入。PLL “R” 输出时钟频率 = VCO 频率 / PLLR, 其中 PLLR 可以是 2、4、6 或 8。

00: PLLR = 2

01: PLLR = 4

10: PLLR = 6

11: PLLR = 8

注意: 软件必须正确设置这些位, 以确保该域上的频率不超过 192MHz。

Bit 24 PLLREN: PLL “R” 时钟输出使能

由软件设置和重置, 以使能 PLL 的 “R” 时钟输出 (用作系统时钟)。

当 PLL 的 “R” 时钟输出被用作系统时钟时, 此位不能写入。

为了节省功耗, 当不使用 PLL 的 “R” 时钟输出时, 应将 PLLREN 的值设置为 0。

0: PLL “R” 时钟输出禁用

1: PLL “R” 时钟输出使能

Bit 23 保留, 必须保持复位值。

Bits 22:21 PLLQ[1:0]: 主 PLL 的 “Q” 时钟分频系数

由软件设置和清除, 以控制主 PLL 输出时钟 PLL “Q” 的频率。这个输出可以选择用于系统时钟。这些位只能在 PLL 被禁用时写入。

PLL “Q” 输出时钟频率 = VCO 频率 / PLLQ, 其中 PLLQ 可以是 2、4、6 或 8。

00: PLLQ = 2

01: PLLQ = 4

10: PLLQ = 6

11: PLLQ = 8

注意：软件必须正确设置这些位，以确保该域上的频率不超过 192 MHz。

Bit 20 **PLLQEN**: 主 PLL “Q” 时钟输出使能

由软件设置和重置，以使能 PLL 的 “Q” 时钟输出。

为了节省功耗，当不使用 PLL 的 “Q” 时钟输出时，应将 PLLQEN 的值设置为 0。

0: PLL “Q” 时钟输出禁用

1: PLL “Q” 时钟输出使能

Bits 19:18 保留，必须保持复位值。

Bit 17 **PLL P**: 主 PLL 的 “P” 时钟分频系数

由软件设置和清除，以控制主 PLL 输出时钟 PLL “P” 的频率。这些位只能在 PLL 被禁用时写入。

当 PLLPDIV[4:0] 被设置为 “00000” 时，PLL “P” 输出时钟频率 = VCO 频率 / PLLP，其中 PLLP 可以是 7 或 17。

0: PLLP = 7

1: PLLP = 17

注意：软件必须正确设置这些位，以确保该域上的频率不超过 192MHz。

Bit 16 **PLL PEN**: 主 PLL “P” 时钟输出使能

通过软件设置和重置以使能 PLL 的 “P” 时钟输出。

为了节省功耗，当不使用 PLL 的 “P” 时钟输出时，应将 PLLPEN 设置为 0。

0: 禁用 PLL “P” 时钟输出

1: 使能 PLL “P” 时钟输出

Bit 15 保留，必须保持复位值。

Bit 14:8 **PLL N[6:0]**: 主 PLL VCO 乘法系数

通过软件设置和清除以控制 VCO 的乘法系数。这些位仅在 PLL 被禁用时可写。

VCO 输出频率 = VCO 输入频率 × PLLN，其中  $6 \leq \text{PLLN} \leq 50$

注意：VCO 的输出频率不能超过 400MHz

0000110: 6x

0000111: 7x

0001000: 8x

...

0010111: 23x

0011000: 24x

0011001: 25x

其他: 6x

Bit 7:2 保留，必须保持复位值。

Bit 1:0 **PLL SRC[1:0]**: 主 PLL 时钟源

通过软件设置和清除以选择 PLL 时钟源。这些位仅在 PLL 被禁用时可写。

为了节省功耗，当不使用 PLL 时，应将 PLLSRC 设置为 00。

00: 不向 PLL 发送时钟

01: 不向 PLL 发送时钟

10: 选择 HSI 作为 PLL 时钟源

11: 选择 HSE 作为 PLL 时钟源

### 8.3.5 定时器时钟配置寄存器 (RCC\_CFGR1)

地址偏移: 0x10

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

|          |    |    |    |    |    |    |    |    |    |           |    |          |    |           |    |
|----------|----|----|----|----|----|----|----|----|----|-----------|----|----------|----|-----------|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21        | 20 | 19       | 18 | 17        | 16 |
| Reserved |    |    |    |    |    |    |    |    |    |           |    |          |    |           |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5         | 4  | 3        | 2  | 1         | 0  |
| Reserved |    |    |    |    |    |    |    |    |    | PPRE2_TIM |    | Reserved |    | PPRE1_TIM |    |
|          |    |    |    |    |    |    |    |    |    | rw        | rw |          |    | rw        | rw |

Bit 5:4 PPRE2\_TIM[1:0]: 高速 APB2\_TIM 倍频 (APB2\_TIM)

由软件来控制低速 APB2\_TIM 时钟的倍频系数。

00: PCLK2 1x (如果 PPRE2=1) 或 PCLK2 2x (如果 PPRE2 >= 2)

01: PCLK2 4x (需要 PPRE2 >= 4 否则无效为 2x)

10: PCLK2 8x (需要 PPRE2 >= 8 否则无效为 2x)

11: PCLK2 16x (需要 PPRE2 >= 16 否则无效为 2x)

Bit 1:0 PPRE1\_TIM[1:0]: 低速 APB1\_TIM 倍频 (APB1\_TIM)

由软件来控制低速 APB1\_TIM 时钟的倍频系数。

00: PCLK1 1x (如果 PPRE1=1) 或 PCLK1 2x (如果 PPRE1 >= 2)

01: PCLK1 4x (需要 PPRE1 >= 4 否则无效为 2x)

10: PCLK1 8x (需要 PPRE1 >= 8 否则无效为 2x)

11: PCLK1 16x (需要 PPRE1 >= 16 否则无效为 2x)

### 8.3.6 时钟中断使能寄存器 (RCC\_CIER)

偏移地址: 0x18

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

|          |    |    |    |    |    |    |    |    |    |              |              |              |          |    |              |
|----------|----|----|----|----|----|----|----|----|----|--------------|--------------|--------------|----------|----|--------------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21           | 20           | 19           | 18       | 17 | 16           |
| Reserved |    |    |    |    |    |    |    |    |    |              |              |              |          |    |              |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5            | 4            | 3            | 2        | 1  | 0            |
| Reserved |    |    |    |    |    |    |    |    |    | PLL<br>RDYIE | HSE<br>RDYIE | HSI<br>RDYIE | Reserved |    | LSI<br>RDYIE |
|          |    |    |    |    |    |    |    |    |    | rw           | rw           | rw           |          |    | rw           |

Bit 31:6 保留, 必须保持为复位值。

Bit 5 PLLRDYIE: PLL 就绪中断使能

由软件设置和清除, 以使能/禁用由 PLL 锁定引起的中断。

0: PLL 锁定中断禁用

1: PLL 锁定中断使能

Bit 4 HSERDYIE: HSE 就绪中断使能

由软件设置和清除, 以使能/禁用由 HSE 振荡器稳定引起的中断。

0: HSE 就绪中断禁用

1: HSE 就绪中断使能



Bit 3 **HSIRDYIE**: HSI 就绪中断使能

由软件设置和清除，以使能/禁用由 HSI 振荡器稳定引起的中断。

0: HSI 就绪中断禁用

1: HSI 就绪中断使能

Bit 2:1 保留，必须保持为复位值。

Bit 0 **LSIRDYIE**: LSI 就绪中断使能

由软件设置和清除，以使能/禁用由 LSI 振荡器稳定引起的中断。

0: LSI 就绪中断禁用

1: LSI 就绪中断使能

### 8.3.7 时钟中断标志寄存器 (RCC\_CIFR)

偏移地址: 0x1C

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

|          |    |    |    |    |    |    |      |          |    |             |             |             |          |    |             |
|----------|----|----|----|----|----|----|------|----------|----|-------------|-------------|-------------|----------|----|-------------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24   | 23       | 22 | 21          | 20          | 19          | 18       | 17 | 16          |
| Reserved |    |    |    |    |    |    |      |          |    |             |             |             |          |    |             |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8    | 7        | 6  | 5           | 4           | 3           | 2        | 1  | 0           |
| Reserved |    |    |    |    |    |    | CSSF | Reserved |    | PLL<br>RDYF | HSE<br>RDYF | HSI<br>RDYF | Reserved |    | LSI<br>RDYF |
|          |    |    |    |    |    |    | r    |          |    | r           | r           | r           |          |    | r           |

Bit 31:9 保留，必须保持为复位值。

Bit 8 **CSSF**: 时钟安全系统中断标志

当 HSE 振荡器检测到故障时，由硬件设置。

通过软件设置 CSSC 位来清除。

0: HSE 时钟故障未触发时钟安全中断

1: HSE 时钟故障触发了时钟安全中断

Bit 7:6 保留，必须保持为复位值。

Bit 5 **PLL RDYF**: PLL 准备就绪中断标志

当 PLL 锁定且 PLLRDYDIE 位被设置时，由硬件设置。

通过软件设置 PLLRDYC 位来清除。

0: PLL 锁定未触发时钟就绪中断

1: PLL 锁定触发了时钟就绪中断

Bit 4 **HSE RDYF**: HSE 准备就绪中断标志

当 HSE 时钟稳定且 HSERDYDIE 位被设置时，由硬件设置。

通过软件设置 HSERDYC 位来清除。

0: HSE 振荡器未触发时钟就绪中断

1: HSE 振荡器触发了时钟就绪中断

Bit 3 **HSI RDYF**: HSI 准备就绪中断标志

当 HSI 时钟稳定且 HSIRDYDIE 位被设置（响应于设置 HSION，参考时钟控制寄存器(RCC\_CR)) 时，由硬件设置。当 HSION 未设置但 HSI 振荡器通过外设的时钟请求被使能时，此位不会被设置，且不会产生中断。

通过软件设置 HSIRDYC 位来清除。

0: HSI 振荡器未触发时钟就绪中断

1: HSI 振荡器触发了时钟就绪中断

Bit 2:1 保留，必须保持为复位值。

Bit 0 **LSIRDYF**: LSI 准备就绪中断标志

当 LSI 时钟稳定且 LSIRDYDIE 位被设置时，由硬件设置。

通过软件设置 LSIRDYC 位来清除。

0: LSI 振荡器未触发时钟就绪中断

1: LSI 振荡器触发了时钟就绪中断

### 8.3.8 时钟中断清除寄存器 (RCC\_CICR)

偏移地址: 0x20

复位值: 0x0000 0000

访问: 无等待，支持字，半字和字节访问

|          |    |    |    |    |    |    |      |          |    |          |          |          |          |    |          |
|----------|----|----|----|----|----|----|------|----------|----|----------|----------|----------|----------|----|----------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24   | 23       | 22 | 21       | 20       | 19       | 18       | 17 | 16       |
| Reserved |    |    |    |    |    |    |      |          |    |          |          |          |          |    |          |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8    | 7        | 6  | 5        | 4        | 3        | 2        | 1  | 0        |
| Reserved |    |    |    |    |    |    | CSSC | Reserved |    | PLL RDYC | HSE RDYC | HSI RDYC | Reserved |    | LSI RDYC |
|          |    |    |    |    |    |    | r    |          |    | r        | r        | r        |          |    | r        |

Bit 31:9 保留，必须保持为复位值。

Bit 8 **CSSC**: 时钟安全系统中断清除

此位由软件设置以清除 CSSF 标志。

0: 无效果

1: 清除 CSSF 标志

Bit 7:6 保留，必须保持为复位值。

Bit 5 **PLL RDYC**: PLL 就绪中断清除

此位由软件设置以清除 PLLRDYF 标志。

0: 无效果

1: 清除 PLLRDYF 标志

Bit 4 **HSE RDYC**: HSE 就绪中断清除

此位由软件设置以清除 HSERDYF 标志。

0: 无效果

1: 清除 HSERDYF 标志

Bit 3 **HSIRDYC**: HSI 就绪中断清除

此位由软件设置以清除 HSIRDYF 标志。

0: 无效果

1: 清除 HSIRDYF 标志

Bit 2:1 保留，必须保持为复位值。

Bit 0 **LSIRDYC**: LSI 就绪中断清除

此位由软件设置以清除 LSIRDYF 标志。

0: 无效果

1: 清除 LSIRDYF 标志

### 8.3.9 AHB1 外设复位寄存器 (RCC\_AHB1RSTR)

偏移地址：0x28

复位值：0x0000 0000

访问：无等待，支持字，半字和字节访问

|          |    |    |            |          |    |    |    |    |    |    |             |               |                |             |             |
|----------|----|----|------------|----------|----|----|----|----|----|----|-------------|---------------|----------------|-------------|-------------|
| 31       | 30 | 29 | 28         | 27       | 26 | 25 | 24 | 23 | 22 | 21 | 20          | 19            | 18             | 17          | 16          |
| Reserved |    |    |            |          |    |    |    |    |    |    |             |               |                |             |             |
| 15       | 14 | 13 | 12         | 11       | 10 | 9  | 8  | 7  | 6  | 5  | 4           | 3             | 2              | 1           | 0           |
| Reserved |    |    | CRC<br>RST | Reserved |    |    |    |    |    |    | FMAC<br>RST | CORDIC<br>RST | DMAMUX1<br>RST | DMA2<br>RST | DMA1<br>RST |
|          |    |    | rw         |          |    |    |    |    |    |    | rw          | rw            | rw             | rw          | rw          |

Bit 31:13 保留，必须保持为复位值。

Bit 12 **CRCRST**: CRC 复位

由软件设置和清除。

0: 无效果

1: 复位 CRC

Bit 11:5 保留，必须保持为复位值。

Bit 4 **FMACRST**: FMAC 复位

由软件设置和清除。

0: 无效果

1: 复位 FMAC

Bit 3 **CORDICRST**: CORDIC 复位

由软件设置和清除。

0: 无效果

1: 复位 CORDIC

Bit 2 **DMAMUX1RST**: DMAMUX1 复位

由软件设置和清除。

0: 无效果

1: 复位 DMAMUX1

Bit 1 **DMA2RST**: DMA2 复位

由软件设置和清除。

0: 无效果

1: 复位 DMA2

Bit 0 **DMA1RST**: DMA1 复位

由软件设置和清除。

0: 无效果

1: 复位 DMA1

### 8.3.10 AHB2 外设复位寄存器 (RCC\_AHB2RSTR)

偏移地址: 0x2C

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

|             |             |             |          |    |    |    |    |    |    |              |              |              |              |              |              |
|-------------|-------------|-------------|----------|----|----|----|----|----|----|--------------|--------------|--------------|--------------|--------------|--------------|
| 31          | 30          | 29          | 28       | 27 | 26 | 25 | 24 | 23 | 22 | 21           | 20           | 19           | 18           | 17           | 16           |
| Reserved    |             |             |          |    |    |    |    |    |    |              |              |              | DAC3<br>RST  | DAC2<br>RST  | DAC1<br>RST  |
|             |             |             |          |    |    |    |    |    |    |              |              |              | rw           | rw           | rw           |
| 15          | 14          | 13          | 12       | 11 | 10 | 9  | 8  | 7  | 6  | 5            | 4            | 3            | 2            | 1            | 0            |
| ADC3<br>RST | ADC2<br>RST | ADC1<br>RST | Reserved |    |    |    |    |    |    | GPIOF<br>RST | GPIOE<br>RST | GPIOD<br>RST | GPIOC<br>RST | GPIOB<br>RST | GPIOA<br>RST |
| rw          | rw          | rw          |          |    |    |    |    |    |    | rw           | rw           | rw           | rw           | rw           | rw           |

Bit 31:13 保留, 必须保持为复位值。

Bit 18 **DAC3RST**: DAC3 复位

由软件设置和清除。

0: 无效果

1: 复位 DAC3

Bit 17 **DAC2RST**: DAC2 复位

由软件设置和清除。

0: 无效果

1: 复位 DAC2

Bit 16 **DAC1RST**: DAC1 复位

由软件设置和清除。

0: 无效果

1: 复位 DAC1

Bit 15 **ADC3RST**: ADC3 复位

由软件设置和清除。

0: 无效果

1: 复位 ADC3

Bit 14 **ADC2RST**: ADC2 复位

由软件设置和清除。

0: 无效果

1: 复位 ADC2

Bit 13 **ADC1RST**: ADC1 复位

由软件设置和清除。

0: 无效果

1: 复位 ADC1 接口

Bit 12:6 保留, 必须保持为复位值。

Bit 5 **GPIOFRST**: GPIO 端口 F 复位

由软件设置和清除。

0: 无效果

1: 复位 GPIO 端口 F

Bit 4 **GPIOERST**: GPIO 端口 E 复位

由软件设置和清除。

0: 无效果

1: 复位 GPIO 端口 E

Bit 3 **GPIODRST**: GPIO 端口 D 复位

由软件设置和清除。

0: 无效果

1: 复位 GPIO 端口 D

Bit 2 **GPIOCRST**: GPIO 端口 C 复位

由软件设置和清除。

0: 无效果

1: 复位 GPIO 端口 C

Bit 1 **GPIOBRST**: GPIO 端口 B 复位

由软件设置和清除。

0: 无效果

1: 复位 GPIO 端口 B

Bit 0 **GPIOARST**: GPIO 端口 A 复位

由软件设置和清除。

0: 无效果

1: 复位 GPIO 端口 A

### 8.3.11 APB1 外设复位寄存器 1 (RCC\_APB1RSTR1)

偏移地址：0x38

复位值：0x0000 0000

访问：无等待，支持字，半字和字节访问

| 31       | 30          | 29          | 28         | 27       | 26 | 25         | 24  | 23         | 22          | 21          | 20           | 19           | 18           | 17           | 16          |
|----------|-------------|-------------|------------|----------|----|------------|-----|------------|-------------|-------------|--------------|--------------|--------------|--------------|-------------|
| Reserved |             | CLCR<br>RST | PWR<br>RST | Reserved |    | CAN<br>RST | Res | USB<br>RST | I2C2<br>RST | I2C1<br>RST | UART5<br>RST | UART4<br>RST | UART3<br>RST | UART2<br>RST | Res         |
|          |             | rw          | rw         |          |    | rw         |     | rw         | rw          | rw          | rw           | rw           | rw           | rw           |             |
| 15       | 14          | 13          | 12         | 11       | 10 | 9          | 8   | 7          | 6           | 5           | 4            | 3            | 2            | 1            | 0           |
| Res      | SPI2<br>RST | Reserved    |            |          |    |            |     |            |             | TIM7<br>RST | TIM6<br>RST  | TIM5<br>RST  | TIM4<br>RST  | TIM3<br>RST  | TIM2<br>RST |
|          | rw          |             |            |          |    |            |     |            |             | rw          | rw           | rw           | rw           | rw           | rw          |

Bit 31:30 保留，必须保持为复位值。

Bit 29 **CLCR**RST：CLC 复位

由软件设置和清除。

0：无效果

1：复位 CLC

Bit 28 **PWR**RST：电源接口复位

由软件设置和清除。

0：无效果

1：复位 PWR

Bit 27:26 保留，必须保持为复位值。

Bit 25 **CAN**RST：CAN 复位

由软件设置和清除。

0：无效果

1：复位 CAN

Bit 24 保留，必须保持为复位值。

Bit 23 **USB**RST：USB 复位

由软件设置和清除。

0：无效果

1：复位 USB

Bit 22 **I2C2**RST：I2C2 复位

由软件设置和清除。

0：无效果

1：复位 I2C2

Bit 21 **I2C1**RST：I2C1 复位

由软件设置和清除。

0：无效果

1：复位 I2C1

Bit 20 **UART5**RST：UART5 复位

由软件设置和清除。

0：无效果



1: 复位 UART5

Bit 19 **UART4RST**: UART4 复位

由软件设置和清除。

0: 无效果

1: 复位 UART4

Bit 18 **UART3RST**: UART3 复位

由软件设置和清除。

0: 无效果

1: 复位 UART3

Bit 17 **UART2RST**: UART2 复位

由软件设置和清除。

0: 无效果

1: 复位 UART2

Bit 16:15 保留, 必须保持为复位值。

Bit 14 **SPI2RST**: SPI2 复位

由软件设置和清除。

0: 无效果

1: 复位 SPI2

Bit 13:6 保留, 必须保持为复位值。

Bit 5 **TIM7RST**: TIM7 定时器复位

由软件设置和清除。

0: 无效果

1: 复位 TIM7

Bit 4 **TIM6RST**: TIM6 定时器复位

由软件设置和清除。

0: 无效果

1: 复位 TIM6

Bit 3 **TIM5RST**: TIM5 定时器复位

由软件设置和清除。

0: 无效果

1: 复位 TIM5

Bit 2 **TIM4RST**: TIM4 定时器复位

由软件设置和清除。

0: 无效果

1: 复位 TIM4

Bit 1 **TIM3RST**: TIM3 定时器复位

由软件设置和清除。

0: 无效果

1: 复位 TIM3

Bit 0 **TIM2RST**: TIM2 定时器复位

由软件设置和清除。

0: 无效果

## 1: 复位 TIM2

## 8.3.12 APB2 外设复位寄存器 (RCC\_APB2RSTR)

偏移地址: 0x40

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

|          |              |             |             |             |               |          |    |    |    |    |    |    |              |               |              |
|----------|--------------|-------------|-------------|-------------|---------------|----------|----|----|----|----|----|----|--------------|---------------|--------------|
| 31       | 30           | 29          | 28          | 27          | 26            | 25       | 24 | 23 | 22 | 21 | 20 | 19 | 18           | 17            | 16           |
| Reserved |              |             |             |             | HRTIM1<br>RST | Reserved |    |    |    |    |    |    | TIM17<br>RST | TIM16<br>RST  | TIM15<br>RST |
|          |              |             |             |             | rw            |          |    |    |    |    |    |    | rw           | rw            | rw           |
| 15       | 14           | 13          | 12          | 11          | 10            | 9        | 8  | 7  | 6  | 5  | 4  | 3  | 2            | 1             | 0            |
| Res      | UART1<br>RST | TIM8<br>RST | SPI1<br>RST | TIM1<br>RST | Reserved      |          |    |    |    |    |    |    |              | SYSCF<br>GRST |              |
|          | rw           | rw          | rw          | rw          |               |          |    |    |    |    |    |    |              | rw            |              |

Bit 31:27 保留, 必须保持为复位值。

Bit 26 **HRTIM1RST**: HRTIM1 复位

由软件设置和清除。

0: 无效果

1: 复位 HRTIM1

Bit 25:19 保留, 必须保持为复位值。

Bit 18 **TIM17RST**: TIM17 定时器复位

由软件设置和清除。

0: 无效果

1: 复位 TIM17 定时器

Bit 17 **TIM16RST**: TIM16 定时器复位

由软件设置和清除。

0: 无效果

1: 复位 TIM16 定时器

Bit 16 **TIM15RST**: TIM15 定时器复位

由软件设置和清除。

0: 无效果

1: 复位 TIM15 定时器

Bit 15 保留, 必须保持为复位值。

Bit 14 **USART1RST**: USART1 复位

由软件设置和清除。

0: 无效果

1: 复位 USART1

Bit 13 **TIM8RST**: TIM8 定时器复位

由软件设置和清除。

0: 无效果

1: 复位 TIM8 定时器

Bit 12 **SPI1RST**: SPI1 复位

由软件设置和清除。

0: 无效果

1: 复位 SPI1

Bit 11 **TIM1RST**: TIM1 定时器复位

由软件设置和清除。

0: 无效果

1: 复位 TIM1 定时器

Bit 10:1 保留, 必须保持为复位值。

Bit 0 **SYSCFGRST**: SYSCFG + COMP + OPAMP + VREFBUF 复位

0: 无效果

1: 复位 SYSCFG、COMP、OPAMP 和 VREFBUF

### 8.3.13 AHB1 外设时钟使能寄存器 (RCC\_AHB1ENR)

偏移地址: 0x48

复位值: 0x0000 0100

访问: 无等待, 支持字, 半字和字节访问

注: 当外设时钟不活跃时, 不支持对外设寄存器的读写访问。

|          |    |    |           |          |    |    |             |          |    |    |            |              |               |            |            |
|----------|----|----|-----------|----------|----|----|-------------|----------|----|----|------------|--------------|---------------|------------|------------|
| 31       | 30 | 29 | 28        | 27       | 26 | 25 | 24          | 23       | 22 | 21 | 20         | 19           | 18            | 17         | 16         |
| Reserved |    |    |           |          |    |    |             |          |    |    |            |              |               |            |            |
| 15       | 14 | 13 | 12        | 11       | 10 | 9  | 8           | 7        | 6  | 5  | 4          | 3            | 2             | 1          | 0          |
| Reserved |    |    | CRC<br>EN | Reserved |    |    | FLASH<br>EN | Reserved |    |    | FMAC<br>EN | CORDIC<br>EN | DMAM<br>UX1EN | DMA2<br>EN | DMA1<br>EN |
|          |    |    | rw        |          |    |    | rw          |          |    |    | rw         | rw           | rw            | rw         | rw         |

Bit 31:13 保留, 必须保持为复位值。

Bit 12 **CRCEN**: CRC 时钟使能

由软件设置和清除。

0: CRC 时钟禁用

1: CRC 时钟使能

Bit 11:9 保留, 必须保持为复位值。

Bit 8 **FLASHEN**: 闪存接口时钟使能

由软件设置和清除。这个位只能在 Flash 处于掉电模式时被禁用。

0: 闪存接口时钟禁用

1: 闪存接口时钟使能

Bit 7:5 保留, 必须保持为复位值。

Bit 4 **FMACEN**: FMAC 使能

由软件设置和重置。

0: FMAC 时钟禁用

1: FMAC 时钟使能

Bit 3 **CORDICEN**: CORDIC 时钟使能

由软件设置和重置。

0: CORDIC 时钟禁用

1: CORDIC 时钟使能

Bit 2 **DMAMUX1EN**: DMAMUX1 时钟使能

由软件设置和重置。

0: DMAMUX1 时钟禁用

1: DMAMUX1 时钟使能

Bit 1 **DMA2EN**: DMA2 时钟使能

由软件设置和清除。

0: DMA2 时钟禁用

1: DMA2 时钟使能

Bit 0 **DMA1EN**: DMA1 时钟使能

由软件设置和清除。

0: DMA1 时钟禁用

1: DMA1 时钟使能

### 8.3.14 AHB2 外设时钟使能寄存器 (RCC\_AHB2ENR)

偏移地址: 0x4C

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

注: 当外设时钟不活跃时, 不支持对外设寄存器的读写访问。

|            |            |            |          |    |    |    |    |    |    |             |             |            |             |             |             |
|------------|------------|------------|----------|----|----|----|----|----|----|-------------|-------------|------------|-------------|-------------|-------------|
| 31         | 30         | 29         | 28       | 27 | 26 | 25 | 24 | 23 | 22 | 21          | 20          | 19         | 18          | 17          | 16          |
| Reserved   |            |            |          |    |    |    |    |    |    |             |             |            | DAC3<br>EN  | DAC2<br>EN  | DAC1<br>EN  |
|            |            |            |          |    |    |    |    |    |    |             |             |            | rw          | rw          | rw          |
| 15         | 14         | 13         | 12       | 11 | 10 | 9  | 8  | 7  | 6  | 5           | 4           | 3          | 2           | 1           | 0           |
| ADC3<br>EN | ADC2<br>EN | ADC1<br>EN | Reserved |    |    |    |    |    |    | GPIOF<br>EN | GPIOE<br>EN | GIOD<br>EN | GPIOC<br>EN | GPIOB<br>EN | GPIOA<br>EN |
| rw         | rw         | rw         |          |    |    |    |    |    |    | rw          | rw          | rw         | rw          | rw          | rw          |

Bit 31:13 保留, 必须保持为复位值。

Bit 18 **DAC3EN**: DAC3 时钟使能

由软件设置和清除。

0: DAC3 时钟禁用

1: DAC3 时钟使能

Bit 17 **DAC2EN**: DAC2 时钟使能

由软件设置和清除。

0: DAC2 时钟禁用

1: DAC2 时钟使能

Bit 16 **DAC1EN**: DAC1 时钟使能

由软件设置和清除。

0: DAC1 时钟禁用

1: DAC1 时钟使能

Bit 15 **ADC3EN**: ADC3 时钟使能

由软件设置和清除。

0: ADC3 时钟禁用

1: ADC3 时钟使能

Bit 14 **ADC2EN**: ADC2 时钟使能

由软件设置和清除。

0: ADC2 时钟禁用

1: ADC2 时钟使能

Bit 13 **ADC1EN**: ADC1 时钟使能

由软件设置和清除。

0: ADC1 时钟禁用

1: ADC1 时钟使能

Bit 12:6 保留，必须保持为复位值。

Bit 5 **GPIOFEN**: GPIO 端口 F 时钟使能

由软件设置和清除。

0: IO 端口 F 时钟禁用

1: IO 端口 F 时钟使能

Bit 4 **GPIOEEN**: GPIO 端口 E 时钟使能

由软件设置和清除。

0: IO 端口 E 时钟禁用

1: IO 端口 E 时钟使能

Bit 3 **GPIODEN**: GPIO 端口 D 时钟使能

由软件设置和清除。

0: IO 端口 D 时钟禁用

1: IO 端口 D 时钟使能

Bit 2 **GPIOCEN**: GPIO 端口 C 时钟使能

由软件设置和清除。

0: IO 端口 C 时钟禁用

1: IO 端口 C 时钟使能

Bit 1 **GPIOBEN**: GPIO 端口 B 时钟使能

由软件设置和清除。

0: IO 端口 B 时钟禁用

1: IO 端口 B 时钟使能

Bit 0 **GPIOAEN**: GPIO 端口 A 时钟使能

由软件设置和清除。

0: IO 端口 A 时钟禁用

1: IO 端口 A 时钟使能

### 8.3.15 APB1 外设时钟使能寄存器 1 (RCC\_APB1ENR1)

偏移地址：0x58

复位值：0x0000 0400

访问：无等待，支持字，半字和字节访问

注：当外设时钟不活跃时，不支持对外设寄存器的读写访问。

|          |            |           |           |            |           |          |           |            |            |             |             |             |             |            |            |
|----------|------------|-----------|-----------|------------|-----------|----------|-----------|------------|------------|-------------|-------------|-------------|-------------|------------|------------|
| 31       | 30         | 29        | 28        | 27         | 26        | 25       | 24        | 23         | 22         | 21          | 20          | 19          | 18          | 17         | 16         |
| Reserved | CLC<br>EN  | PWR<br>EN | BKP<br>EN | Res        | CAN<br>EN | Res      | USB<br>EN | I2C2<br>EN | I2C1<br>EN | UART5<br>EN | UART4<br>EN | UART3<br>EN | UART2<br>EN | Res        | Res        |
|          |            |           |           |            |           |          |           |            |            |             |             |             |             |            |            |
| 15       | 14         | 13        | 12        | 11         | 10        | 9        | 8         | 7          | 6          | 5           | 4           | 3           | 2           | 1          | 0          |
| Res      | SPI2<br>EN | Reserved  | Reserved  | WWDG<br>EN | Reserved  | Reserved | Reserved  | Reserved   | TIM7<br>EN | TIM6<br>EN  | TIM5<br>EN  | TIM4<br>EN  | TIM3<br>EN  | TIM2<br>EN | TIM2<br>EN |
|          |            |           |           |            |           |          |           |            |            |             |             |             |             |            |            |
|          | rw         |           |           | rs         |           |          |           |            | rw         | rw          | rw          | rw          | rw          | rw         | rw         |

Bit 31:30 保留，必须保持为复位值。

Bit 29 **CLCEN**：CLC 时钟使能

由软件设置和清除。

0：CLC 时钟禁用

1：CLC 时钟使能

Bit 28 **PWREN**：电源接口时钟使能

由软件设置和清除。

0：电源接口时钟禁用

1：电源接口时钟使能

Bit 27 **BKPEN**：备份接口时钟使能

由软件设置和清除。

0：备份接口时钟禁用

1：备份接口时钟使能

Bit 26 保留，必须保持为复位值。

Bit 25 **CANEN**：CAN 时钟使能

由软件设置和清除。

0：CAN 时钟禁用

1：CAN 时钟使能

Bit 24 保留，必须保持为复位值。

Bit 23 **USBEN**：USB 时钟使能

由软件设置和清除。

0：USB 时钟禁用

1：USB 时钟使能

Bit 22 **I2C2EN**：I2C2 时钟使能

由软件设置和清除。

0：I2C2 时钟禁用

1：I2C2 时钟使能

Bit 21 **I2C1EN**：I2C1 时钟使能

由软件设置和清除。



0: I2C1 时钟禁用

1: I2C1 时钟使能

Bit 20 **UART5EN**: UART5 时钟使能

由软件设置和清除。

0: UART5 时钟禁用

1: UART5 时钟使能

Bit 19 **UART4EN**: UART4 时钟使能

由软件设置和清除。

0: UART4 时钟禁用

1: UART4 时钟使能

Bit 18 **UART3EN**: UART3 时钟使能

由软件设置和清除。

0: UART3 时钟禁用

1: UART3 时钟使能

Bit 17 **UART2EN**: UART2 时钟使能

由软件设置和清除。

0: UART2 时钟禁用

1: UART2 时钟使能

Bit 16:15 保留，必须保持为复位值。

Bit 14 **SPI2EN**: SPI2 时钟使能

由软件设置和清除。

0: SPI2 时钟禁用

1: SPI2 时钟使能

Bit 13:12 保留，必须保持为复位值。

Bit 11 **WWDGEN**: 窗口看门狗时钟使能

由软件设置以使能窗口看门狗时钟。由硬件系统复位清除。

如果 WWDG\_SW 选项位被复位，该位也可以由硬件设置。

0: 窗口看门狗时钟禁用

1: 窗口看门狗时钟使能

Bit 10:6 保留，必须保持为复位值。

Bit 5 **TIM7EN**: TIM7 定时器时钟使能

由软件设置和清除。

0: TIM7 时钟禁用

1: TIM7 时钟使能

Bit 4 **TIM6EN**: TIM6 定时器时钟使能

由软件设置和清除。

0: TIM6 时钟禁用

1: TIM6 时钟使能

Bit 3 **TIM5EN**: TIM5 定时器时钟使能

由软件设置和清除。

0: TIM5 时钟禁用

1: TIM5 时钟使能

Bit 2 **TIM4EN**: TIM4 定时器时钟使能

由软件设置和清除。

0: TIM4 时钟禁用

1: TIM4 时钟使能

Bit 1 **TIM3EN**: TIM3 定时器时钟使能

由软件设置和清除。

0: TIM3 时钟禁用

1: TIM3 时钟使能

Bit 0 **TIM2EN**: TIM2 定时器时钟使能

由软件设置和清除。

0: TIM2 时钟禁用

1: TIM2 时钟使能

### 8.3.16 APB2 外设时钟使能寄存器 (RCC\_APB2ENR)

偏移地址: 0x60

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

| 31       | 30          | 29         | 28        | 27         | 26           | 25       | 24 | 23 | 22 | 21 | 20 | 19 | 18          | 17          | 16           |
|----------|-------------|------------|-----------|------------|--------------|----------|----|----|----|----|----|----|-------------|-------------|--------------|
| Reserved |             |            |           |            | HRTIM1<br>EN | Reserved |    |    |    |    |    |    | TIM17<br>EN | TIM16<br>EN | TIM15<br>EN  |
|          |             |            |           |            | rw           |          |    |    |    |    |    |    | rw          | rw          | rw           |
| 15       | 14          | 13         | 12        | 11         | 10           | 9        | 8  | 7  | 6  | 5  | 4  | 3  | 2           | 1           | 0            |
| Res      | UART1<br>EN | TIM8<br>EN | SPH<br>EN | TIM1<br>EN | Reserved     |          |    |    |    |    |    |    |             |             | SYS<br>CFGEN |
|          | rw          | rw         | rw        | rw         |              |          |    |    |    |    |    |    |             |             | rw           |

Bit 31:27 保留, 必须保持为复位值。

Bit 26 **HRTIM1EN**: HRTIM1 时钟使能

由软件设置和清除。

0: HRTIM1 时钟禁用

1: HRTIM1 时钟使能

Bit 25:19 保留, 必须保持为复位值。

Bit 18 **TIM17EN**: TIM17 定时器时钟使能

由软件设置和清除。

0: TIM17 定时器时钟禁用

1: TIM17 定时器时钟使能

Bit 17 **TIM16EN**: TIM16 定时器时钟使能

由软件设置和清除。

0: TIM16 定时器时钟禁用

1: TIM16 定时器时钟使能

Bit 16 **TIM15EN**: TIM15 定时器时钟使能

由软件设置和清除。

0: TIM15 定时器时钟禁用

1: TIM15 定时器时钟使能

Bit 15 保留，必须保持为复位值。

Bit 14 **UART1EN**: UART1 时钟使能

由软件设置和清除。

0: USART1 时钟禁用

1: USART1 时钟使能

Bit 13 **TIM8EN**: TIM8 定时器时钟使能

由软件设置和清除。

0: TIM8 定时器时钟禁用

1: TIM8 定时器时钟使能

Bit 12 **SPI1EN**: SPI1 时钟使能

由软件设置和清除。

0: SPI1 时钟禁用

1: SPI1 时钟使能

Bit 11 **TIM1EN**: TIM1 定时器时钟使能

由软件设置和清除。

0: TIM1 定时器时钟禁用

1: TIM1 定时器时钟使能

Bit 10:1 保留，必须保持为复位值。

Bit 0 **SYSCFGEN**: SYSCFG + COMP + OPAMP + VREFBUF 时钟使能

0: SYSCFG + COMP + VREFBUF + OPAMP 时钟禁用

1: SYSCFG + COMP + VREFBUF + OPAMP 时钟使能

### 8.3.17 独立外设时钟配置寄存器 (RCC\_CCIPR)

偏移地址: 0x88

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

|          |    |             |    |          |    |    |    |    |    |    |    |    |    |    |    |
|----------|----|-------------|----|----------|----|----|----|----|----|----|----|----|----|----|----|
| 31       | 30 | 29          | 28 | 27       | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    | ADCSEL[1:0] |    | Reserved |    |    |    |    |    |    |    |    |    |    |    |
|          |    | rw          | rw |          |    |    |    |    |    |    |    |    |    |    |    |
| 15       | 14 | 13          | 12 | 11       | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |             |    |          |    |    |    |    |    |    |    |    |    |    |    |

Bit 31:30 保留，必须保持为复位值。

Bit 29:28 **ADCSEL[1:0]**: ADC1/2 时钟源选择

这些位通过软件设置和清除，以选择 ADC 接口使用的时钟源。

00: 未选择时钟

01: 选择 PLL “P” 时钟作为 ADC1/2 的时钟

10: 选择系统时钟作为 ADC1/2 的时钟

11: 保留

Bit 27:0 保留，必须保持为复位值。

### 8.3.18 RTC 域控制寄存器 (RCC\_BDCR)

偏移地址: 0x90

复位值：0x0000 0000

访问：0 ≤ 等待状态 ≤ 3，支持字、半字和字节访问。

等待状态：在连续访问此寄存器时插入等待状态。

|          |          |    |    |    |    |             |    |          |    |    |    |    |    |    |       |
|----------|----------|----|----|----|----|-------------|----|----------|----|----|----|----|----|----|-------|
| 31       | 30       | 29 | 28 | 27 | 26 | 25          | 24 | 23       | 22 | 21 | 20 | 19 | 18 | 17 | 16    |
| Reserved |          |    |    |    |    |             |    |          |    |    |    |    |    |    | BDRST |
|          |          |    |    |    |    |             |    |          |    |    |    |    |    |    | rw    |
| 15       | 14       | 13 | 12 | 11 | 10 | 9           | 8  | 7        | 6  | 5  | 4  | 3  | 2  | 1  | 0     |
| RTCEN    | Reserved |    |    |    |    | RTCSEL[1:0] |    | Reserved |    |    |    |    |    |    |       |
| rw       |          |    |    |    |    | rw          | rw |          |    |    |    |    |    |    |       |

Bit 31:30 保留，必须保持为复位值。

Bit 16 **BDRST**: RTC 域软件复位

通过软件设置和清除。

0: 复位未激活

1: 复位整个 RTC 域

Bit 15 **RTCEN**: RTC 时钟使能

通过软件设置和清除。

0: RTC 时钟禁用

1: RTC 时钟使能

Bit 14:10 保留，必须保持为复位值。

Bit 9:8 **RTCSEL[1:0]**: RTC 时钟源选择

通过软件设置以选择 RTC 的时钟源。一旦 RTC 时钟源被选定，除非 RTC 域被复位，或者检测到 LSE 故障（LSECSSD 被设置），否则它不能再被更改。可以使用 BDRST 位来重置它们。

00: 无时钟

01: 无时钟

10: 使用 LSI 振荡器时钟作为 RTC 时钟

11: HSE 振荡器时钟除以 32 后作为 RTC 时钟

Bit 7:0 保留，必须保持为复位值。

### 8.3.19 控制/状态寄存器 (RCC\_CSR)

偏移地址: 0x94

复位值: 0x0C00 0000

访问:  $0 \leq \text{等待状态} \leq 3$ , 支持字、半字和字节访问。

等待状态: 在连续访问此寄存器时插入等待状态。

| 31           | 30           | 29           | 28          | 27          | 26          | 25          | 24   | 23       | 22 | 21 | 20 | 19 | 18     | 17 | 16    |
|--------------|--------------|--------------|-------------|-------------|-------------|-------------|------|----------|----|----|----|----|--------|----|-------|
| LPWR<br>RSTF | WWDG<br>RSTF | IWDG<br>RSTF | SFT<br>RSTF | POR<br>RSTF | PIN<br>RSTF | OBL<br>RSTF | RMVF | Reserved |    |    |    |    |        |    |       |
| r            | r            | r            | r           | r           | r           | r           | rw   |          |    |    |    |    |        |    |       |
| 15           | 14           | 13           | 12          | 11          | 10          | 9           | 8    | 7        | 6  | 5  | 4  | 3  | 2      | 1  | 0     |
| Reserved     |              |              |             |             |             |             |      |          |    |    |    |    | LSIRDY |    | LSION |
|              |              |              |             |             |             |             |      |          |    |    |    |    | rw     |    | rw    |

Bit 31:30 保留, 必须保持为复位值。

Bit 31 **LPWRRSTF**: 低功耗复位标志

在低功耗管理复位发生时由硬件置 1;

由软件通过写 RMVF 位清除。

0: 没有发生非法模式复位

1: 发生了非法模式复位

Bit 30 **WWDGRSTF**: 窗口看门狗复位标志

在窗口看门狗复位发生时由硬件置 1;

由软件通过写 RMVF 位清除

通过向 RMVF 位写入来清除。

0: 没有发生窗口看门狗复位

1: 发生了窗口看门狗复位

Bit 29 **IWDGRSTF**: 独立窗口看门狗复位标志

在独立看门狗复位发生在 VDD 区域时由硬件置 1;

由软件通过写 RMVF 位清除。

0: 没有发生独立看门狗复位

1: 发生了独立看门狗复位

Bit 28 **SFTRSTF**: 软件复位标志

在软件复位发生时由硬件置 1;

由软件通过写 RMVF 位清除。

0: 没有发生软件复位

1: 发生了软件复位

Bit 27 **PORRSTF**: 上电/掉电复位标志

在上电/掉电复位发生时由硬件置 1;

由软件通过写 RMVF 位清除。

0: 没有上电/掉电复位发生;

1: 发生上电/掉电复位。

Bit 26 **PINRSTF**: 引脚复位标志

在 NRST 引脚复位发生时由硬件置 1;

由软件通过写 RMVF 位清除。

0: 没有从 NRST 引脚发生复位

1: 从 NRST 引脚发生了复位

Bit 25 **OBLRSTF**: 选项字节加载器复位标志

由硬件设置，当从选项字节加载发生复位时。

通过向 RMVF 位写入来清除。

0: 没有从选项字节加载发生复位

1: 从选项字节加载发生了复位

Bit 24 **RMVF**: 移除复位标志

由软件设置以清除复位标志。

0: 无效果

1: 清除复位标志

Bit 23:2 保留，必须保持为复位值。

Bit 1 **LSIRDY**: 内部低速振荡器就绪

由硬件置 1 或清 0 来指示内部 32kHz RC 振荡器是否就绪。在 LSION 清零后，3 个内部 32kHz RC 振荡器的周期后 LSIRDY 被清零。

0: LSI 振荡器未就绪

1: LSI 振荡器就绪

Bit 0 **LSION**: 内部低速振荡器使能

由软件设置和清除。

0: 内部 32kHz RC 振荡器关闭

1: 内部 32kHz RC 振荡器开启



## 9 通用和复用功能 I/O (GPIO)

### 9.1 简介

每个通用 I/O 有 4 个 32 位配置寄存器 (GPIOx\_MODER, GPIOx\_OTYPER, GPIOx\_OSPEEDR 和 GPIOx\_PUPDR), 2 个 32 位数据寄存器 (GPIOx\_IDR 和 GPIOx\_ODR), 1 个 32 位置位/清零寄存器 (GPIOx\_BSRR)。另外, 所有 GPIO 有 1 个 32 位锁位寄存器 (GPIOx\_LCKR) 和 2 个复用功能配置寄存器 (GPIOx\_AFR1 和 GPIOx\_AFR2)。

### 9.2 GPIO 主要特征

- 多个受控 I/O
- 输出状态: 推挽或开漏 + 上拉/下拉
- 从输出数据寄存器 (GPIOx\_ODR) 或外设 (复用功能输出) 输出数据
- I/O 速度可选
- 输入状态: 浮空, 上拉/下拉, 模拟
- 从输入数据寄存器 (GPIOx\_IDR) 或外设 (复用功能输入) 输入数据
- 置位和清零寄存器 (GPIOx\_BSRR) 对 GPIOx\_ODR 进行访问
- 提供用来冻结 I/O 端口配置的锁定机制 (GPIOx\_LCKR)
- 模拟功能
- 复用功能选择寄存器
- 高度灵活的引脚复用允许使用 I/O 引脚作为 GPIO 或者几个外设复用功能之一

### 9.3 GPIO 功能描述

根据数据手册中列出的每个 I/O 端口的特定硬件特征, GPIO 端口的每个位可以由软件分别配置成多种模式:

- 输入浮空
- 输入上拉
- 输入下拉
- 模拟
- 上拉/下拉开漏输出
- 上拉/下拉推挽输出
- 上拉/下拉复用开漏输出
- 上拉/下拉复用推挽输出

每个 I/O 端口位可以自由编程, I/O 端口寄存器支持字、半字或字节访问。GPIOx\_BSRR 寄存器允许对任何 GPIO 寄存器进行读/更改的独立访问, 这样在读和更改访问之间产生 IRQ 时不会发生危险。

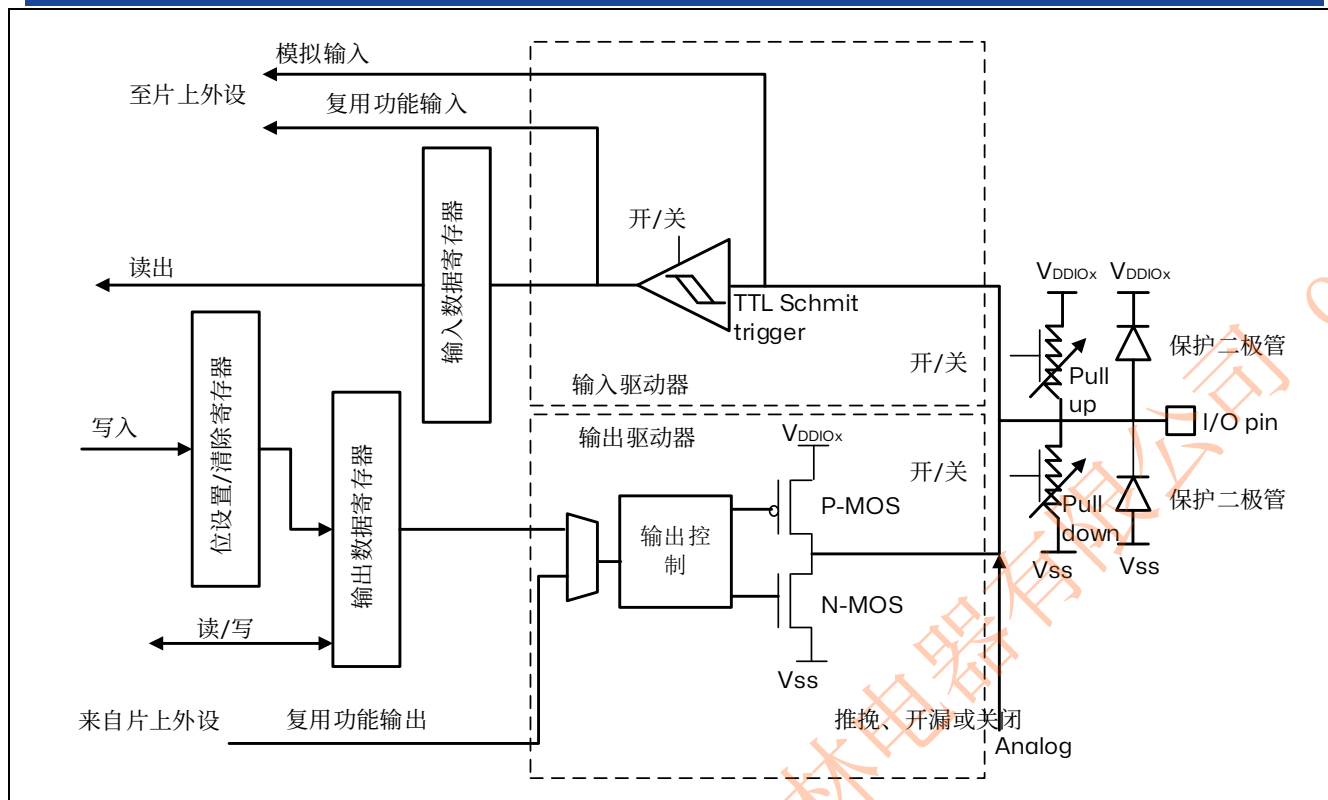


图 9.1 I/O 端口位基本结构

表 9.1 端口位配置表 <sup>(1)</sup>

| MODE[1:0] | OTYPE | OSPEED[1:0] | PUPD[1:0] | I/O 配置        |         |
|-----------|-------|-------------|-----------|---------------|---------|
| 01        | 0     | SPEED[1:0]  | 00        | GP 输出         | PP      |
|           | 0     |             | 01        | GP 输出         | PP + PU |
|           | 0     |             | 10        | GP 输出         | PP + PD |
|           | 0     |             | 11        | 保留            |         |
|           | 1     |             | 00        | GP 输出         | OD      |
|           | 1     |             | 01        | GP 输出         | OD + PU |
|           | 1     |             | 10        | GP 输出         | OD + PD |
|           | 1     |             | 11        | 保留 (GP 输出 OD) |         |
| 10        | 0     | SPEED[1:0]  | 00        | AF            | PP      |
|           | 0     |             | 01        | AF            | PP + PU |
|           | 0     |             | 10        | AF            | PP + PD |
|           | 0     |             | 11        | 保留            |         |
|           | 1     |             | 00        | AF            | OD      |
|           | 1     |             | 01        | AF            | OD + PU |
|           | 1     |             | 10        | AF            | OD + PD |
|           | 1     |             | 11        | 保留            |         |
| 00        | x     | x           | 00        | 输入            | 浮空      |
|           | x     | x           | 01        | 输入            | PU      |
|           | x     | x           | 10        | 输入            | PD      |

| MODE[1:0] | OTYPE | OSPEED[1:0] | PUPD[1:0] | I/O 配置    |    |
|-----------|-------|-------------|-----------|-----------|----|
|           | x     | x           | 11        | 保留 (输入浮空) |    |
| 11        | x     | x           | 00        | 输入/输出     | 模拟 |
|           | x     | x           | 01        | 保留        |    |
|           | x     | x           | 10        |           |    |
|           | x     | x           | 11        |           |    |

3. GP = 通用功能, PP = 推挽, PU = 上拉, PD = 下拉, OD = 开漏, AF = 复用。

### 9.3.1 通用 IO (GPIO)

复位期间和刚复位后, 复用功能未开启, I/O 端口默认是浮空模拟。

复位后, 调试引脚被置于复用上拉或下拉模式:

- JTDI 处于上拉状态
- JTCK/SWCLK 处于下拉状态
- JTMS/SWDAT 处于下拉状态
- NJTRST 处于上拉状态
- JTDO 处于浮空状态

当作为输出配置时, 写到输出数据寄存器上的值 (GPIOx\_ODR) 输出到相应的 I/O 引脚。可以以推挽模式或开漏模式使用输出驱动器。

输入数据寄存器 (GPIOx\_IDR) 在每个 AHB1 时钟周期捕捉 IO 引脚上的数据。

所有 GPIO 引脚有一个内部弱上拉和弱下拉, 可以通过 GPIOx\_PUPDR 寄存器配置。

### 9.3.2 IO 引脚复用功能

芯片的 I/O 引脚通过一个多路复用器连接到外设模块, 同时只能将一个外设复用功能连接到一个 I/O。因此, 同一个引脚上的复用功能不会出现冲突。

每一个 I/O 引脚有一个 16 路多路复用器 (AF0 到 AF15), 可以通过 GPIOx\_AFRL (复用 0 到 7) 和 GPIOx\_AFRH (复用 8 到 15):

- 复位后, 多路复用器选在复用功能 0 (AF0)。可以通过 GPIOx\_MODER 寄存器将 I/O 配置成复用功能模式。
- 引脚的复用功能请参考芯片数据手册。

除了这种灵活的 I/O 复用架构之外, 各外设还可以将复用功能映射到不同 I/O 引脚, 这可以优化小型封装中可用外设的数量。

要将 I/O 配制成所需功能, 请按照以下步骤操作:

- **调试功能:** 在芯片复位后, 会将这些引脚指定为专用引脚, 可供调试器立即使用。
- **GPIO:** 使用 GPIOx\_MODER 寄存器将所需的 I/O 配置为输出或输入模式。
- **外设复用功能:**
  - 使用 GPIOx\_AFRL 和 GPIOx\_AFRH 将 I/O 连接到指定的复用功能上。
  - 使用 GPIOx\_OTYPER, GPIOx\_PUPDR 和 GPIOx\_OSPEEDER 寄存器分别配置类型、上拉/下拉和输出速度。
  - 使用 GPIOx\_MODER 寄存器将 I/O 配置为复用模式。
- **模拟功能:** 对于 ADC、COMP、PGA 和 OPA 等模拟外设, 在 GPIOx\_MODER 寄存器中将所需 I/O 配置为模拟通道。

### 9.3.3 I/O 端口控制寄存器

每个 GPIO 端口有 4 个 32 位内存映射控制寄存器 (GPIOx\_MODER, GPIOx\_OTYPER, GPIOx\_OSPEEDR 和 GPIOx\_PUPDR), 可以配置最多 16 个 I/O。GPIOx\_MODER 寄存器用于选择 I/O 模式 (输入, 输出, 复用, 模拟)。GPIOx\_OTYPER 和 GPIOx\_OSPEEDR 寄存器用于选择输出类型 (推挽或开漏) 和速度。GPIOx\_PUPDR 寄存器用于选择任意 I/O 方向的上拉/下拉。

### 9.3.4 I/O 端口数据寄存器

每个 GPIO 有两个 32 位内存映射数据寄存器: 输入和输出数据寄存器 (GPIOx\_IDR 和 GPIOx\_ODR)。GPIOx\_ODR 存储要输出的数据, 可读可写。通过 I/O 输入的数据存储在输入数据寄存器 (GPIOx\_IDR) 中, 这是一个只读寄存器。

### 9.3.5 I/O 数据位处理

位设置和清零寄存器 (GPIOx\_BSRR) 是一个 32 位寄存器, 允许应用程序置位和清零输出数据寄存器 (GPIOx\_ODR) 中的每个位。

对于 GPIOx\_ODR 中的每一位, 对应 GPIOx\_BSRR 中的两个控制位: BS (i) 和 BR (i)。当 BS (i) 写入 1 时, 置位相应的 ODR (i) 位。当 BR (i) 写入 1 时, 清零 ODR (i) 对应的位。

将 GPIOx\_BSRR 中的任何位写入 0 不会对 GPIOx\_ODR 中的相应位产生任何影响。如果同时对 BS (i) 和 BR (i) 写 1, 则置位操作具有优先级。

使用 GPIOx\_BSRR 寄存器来更改 GPIOx\_ODR 中单个位的值是“一次”效果, 它不会锁定 GPIOx\_ODR 位。GPIOx\_ODR 位始终可以直接访问。GPIOx\_BSRR 寄存器提供了一种执行位处理的方法。

在位级编程 GPIOx\_ODR 时, 软件不需要禁用中断: 可以在单个 AHB1 写访问中修改一个或多个位。

### 9.3.6 I/O 锁定机制

通过对 GPIOx\_LCKR 寄存器应用特定的写序列, 可以冻结 GPIO 控制寄存器。冻结寄存器是 GPIOx\_MODER、GPIOx\_OTYPER、GPIOx\_OSPEEDR、GPIOx\_PUPDR、GPIOx\_AFRL 和 GPIOx\_AFRH。

要写入 GPIOx\_LCKR 寄存器, 必须应用特定的写/读序列。当正确的 LOCK 序列应用于该寄存器的第 16 位时, LCKR[15:0] 的值用于锁定 I/O 的配置 (在写入序列中 LCKR[15:0] 的值必须相同)。当 LOCK 序列应用于端口位时, 该端口位的值不能再被修改, 直到下一次 MCU 复位或外设复位。每个 GPIOx\_LCKR 位冻结控制寄存器中相应的位 (GPIOx\_MODER、GPIOx\_OTYPER、GPIOx\_OSPEEDR、GPIOx\_PUPDR、GPIOx\_AFRL)。

LOCK 序列只能通过对 GPIOx\_LCKR 寄存器的一个字 (32 位长) 访问来执行, 因为 GPIOx\_LCKR 位 16 必须与 [15:0] 位同时设置。

### 9.3.7 I/O 复用功能输入/输出

提供了两个寄存器 GPIOx\_AFRL 和 GPIOx\_AFRH 来为每个 I/O 选择一个可用的复用输入/输出。使用这些寄存器, 用户可以根据应用程序的需要将复用功能连接到其他引脚。

使用 GPIOx\_AFRL 和 GPIOx\_AFRH 复用寄存器在每个 GPIO 上多路复用外设功能。因此, 应用程序可以为每个 I/O 选择任意一个功能。AF 选择信号对于复用输入和复用输出是共性的, 因此对于给定 I/O 的复用功能输入/输出选择单个通道。

具体 GPIO 的复用功能情况请参考数据手册。

### 9.3.8 外部中断/唤醒线

所有端口都具有外部中断功能。要使用外部中断功能，需要将端口配置为输入模式。

### 9.3.9 输入模式配置

当 I/O 端口被配置为输入时：

- 输出缓冲区被禁用
- Schmitt 触发器输入激活
- 根据 GPIOx\_PUPDR 寄存器中的值，上拉和下拉电阻被激活
- 每个 AHB1 时钟周期，I/O 引脚上的数据被采样到的输入数据寄存器
- 输入数据寄存器提供 I/O 状态

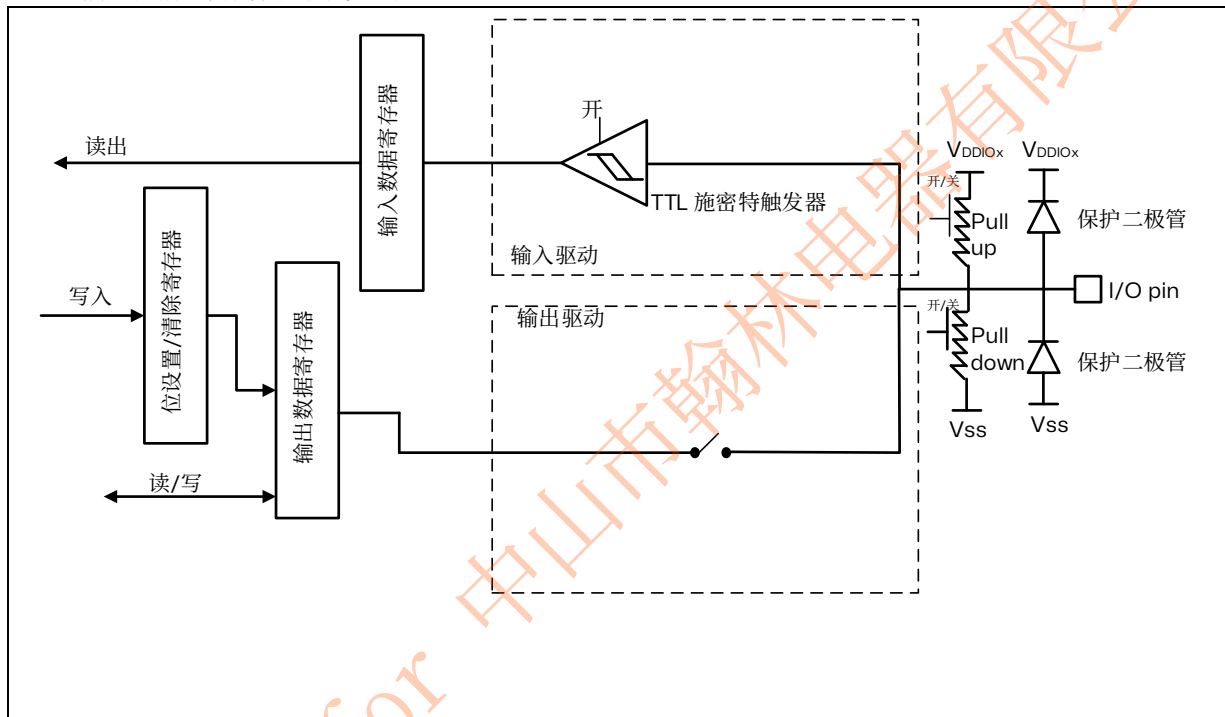


图 9.2 输入浮空/上拉/下拉结构

### 9.3.10 输出模式配置

当 I/O 端口被配置为输出时：

- 输出缓冲区已使能：
- 开漏模式：输出寄存器中的“0”开启 N-MOS，而输出寄存器中的“1”使端口处于 Hi-Z 状态（P-MOS 未被开启）
- 推挽模式：输出寄存器中的“0”开启 N-MOS，而输出寄存器中的“1”开启 P-MOS
- Schmitt 触发器输入激活
- 根据 GPIOx\_PUPDR 寄存器中的值，上拉和下拉电阻被激活
- 每个 AHB1 时钟周期，I/O 引脚上的数据被采样到的输入数据寄存器
- 输入数据寄存器提供 I/O 状态
- 输出数据寄存器可读取最后写入的值

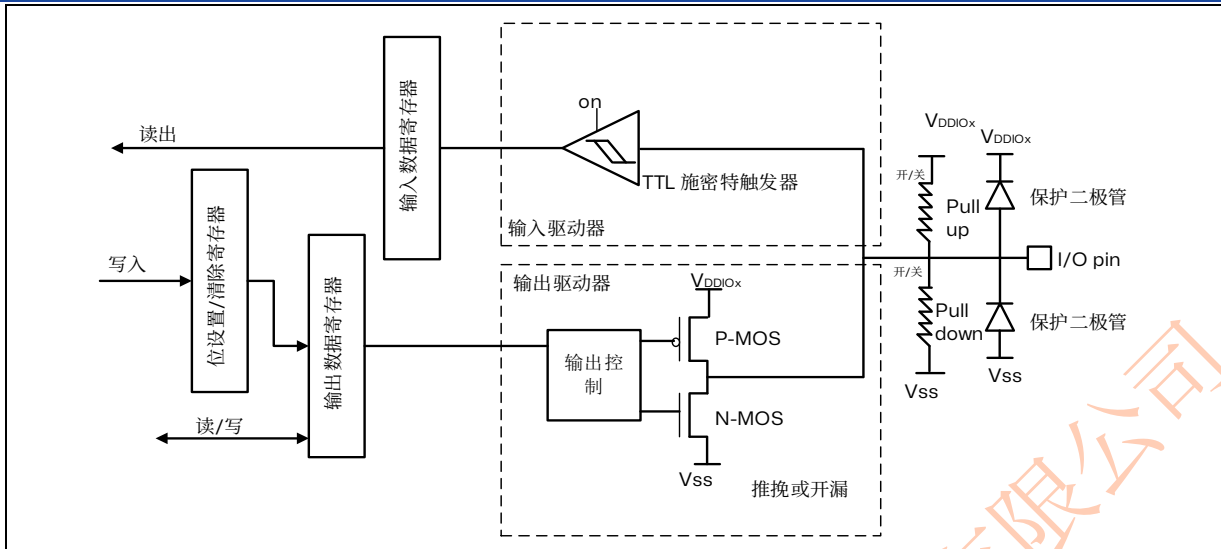


图 9.3 输出框图

### 9.3.11 复用功能配置

当 I/O 端口被配置为复用功能时：

- 输出缓冲区可以配置为开漏或推挽模式
- 输出缓冲区由来自外围设备的信号驱动（发射器使能和数据）
- Schmitt 触发器输入激活
- 根据 GPIOx\_PUPDR 寄存器中的值，上拉和下拉电阻被激活
- 每个 AHB1 时钟周期，I/O 引脚上的数据被采样到输入数据寄存器
- 输入数据寄存器提供 I/O 状态

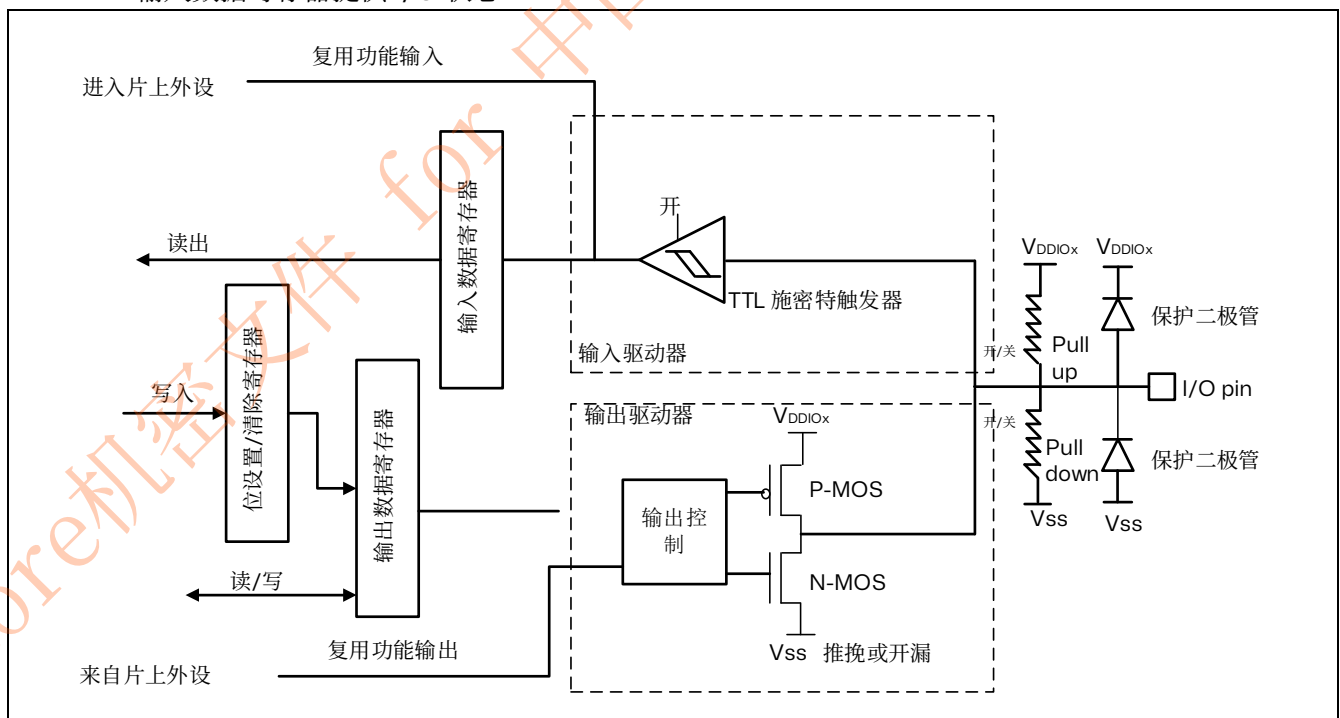


图 9.4 复用功能框图

### 9.3.12 模拟功能配置

当 I/O 端口配置为模拟配置时：

- 输出缓冲区被禁用
- Schmitt 触发输入是失活的，为 I/O 引脚的每个模拟值提供零消耗。Schmitt 触发器的输出被强制为恒定值（0）。
- 输入数据寄存器读得到值固定为“0”

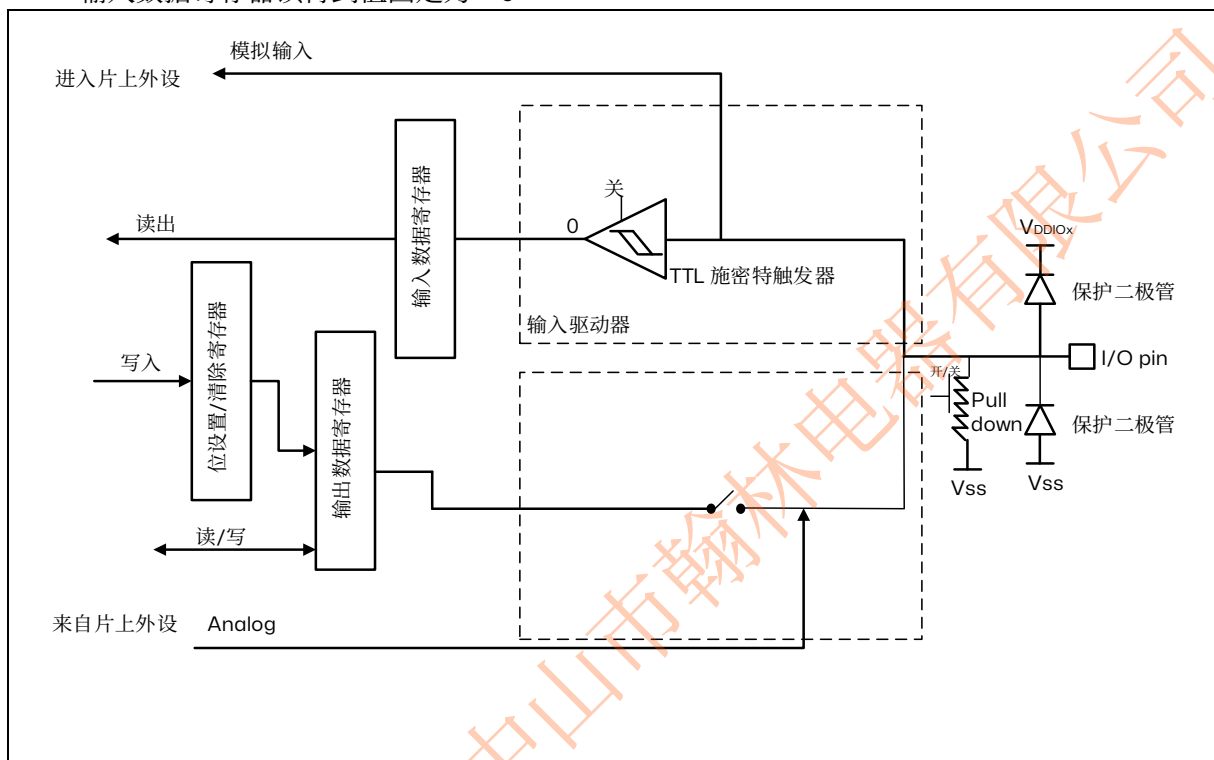


图 9.5 模拟引脚框图



## 9.4 GPIO 寄存器描述

基地址：

0x4800 0000 (GPIOA)

0x4800 0400 (GPIOB)

0x4800 0800 (GPIOC)

0x4800 0C00 (GPIOD)

0x4800 1000 (GPIOE)

### 9.4.1 GPIO 端口模式寄存器 (GPIOx\_MODER) (x = A to F)

地址偏移：0x00

端口 A 复位值：0xABFF FFFF

端口 B 复位值：0xFFFF FEBF

其他端口复位值：0xFFFF FFFF

访问：无等待，支持字，半字和字节访问

| 31           | 30 | 29           | 28 | 27           | 26 | 25           | 24 | 23           | 22 | 21           | 20 | 19          | 18 | 17          | 16 |
|--------------|----|--------------|----|--------------|----|--------------|----|--------------|----|--------------|----|-------------|----|-------------|----|
| MODER15[1:0] |    | MODER14[1:0] |    | MODER13[1:0] |    | MODER12[1:0] |    | MODER11[1:0] |    | MODER10[1:0] |    | MODER9[1:0] |    | MODER8[1:0] |    |
| rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw          | rw | rw          | rw |
| 15           | 14 | 13           | 12 | 11           | 10 | 9            | 8  | 7            | 6  | 5            | 4  | 3           | 2  | 1           | 0  |
| MODER7[1:0]  |    | MODER6[1:0]  |    | MODER5[1:0]  |    | MODER4[1:0]  |    | MODER3[1:0]  |    | MODER2[1:0]  |    | MODER1[1:0] |    | MODER0[1:0] |    |
| rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw          | rw | rw          | rw |

Bits 31:0 MODERy[1:0]：端口 x 配置 I/O 引脚 y (y = 0...15)

这些位通过软件写入，用于配置 I/O 方向模式。

00：输入

01：通用输出模式

10：复用功能模式

11：模拟模式（复位状态）

### 9.4.2 GPIO 端口输出类型寄存器 (GPIOx\_OTYPE) (x = A to F)

地址偏移：0x04

复位值：0x0000 0000

访问：无等待，支持字，半字和字节访问

| 31       | 30   | 29   | 28   | 27   | 26   | 25  | 24  | 23  | 22  | 21  | 20  | 19  | 18  | 17  | 16  |
|----------|------|------|------|------|------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| Reserved |      |      |      |      |      |     |     |     |     |     |     |     |     |     |     |
| 15       | 14   | 13   | 12   | 11   | 10   | 9   | 8   | 7   | 6   | 5   | 4   | 3   | 2   | 1   | 0   |
| OT15     | OT14 | OT13 | OT12 | OT11 | OT10 | OT9 | OT8 | OT7 | OT6 | OT5 | OT4 | OT3 | OT2 | OT1 | OT0 |
| rw       | rw   | rw   | rw   | rw   | rw   | rw  | rw  | rw  | rw  | rw  | rw  | rw  | rw  | rw  | rw  |

Bits 31:16 保留，必须保持为复位值。

Bits 15:0 OTy：端口 x 配置 I/O 引脚 y (y = 15 to 0)

这些位由软件写入，用于配置 I/O 输出类型。

0：输出推挽（复位状态）

1：输出开漏

### 9.4.3 GPIO 端口输出速度寄存器 (GPIOx\_OSPEEDR) (x = A to F)

地址偏移: 0x08

端口 A 复位值: 0x0C00 0000

端口 B 复位值: 0x0000 00C0

其他端口复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

|               |    |               |    |               |    |               |    |               |    |               |    |              |    |              |    |
|---------------|----|---------------|----|---------------|----|---------------|----|---------------|----|---------------|----|--------------|----|--------------|----|
| 31            | 30 | 29            | 28 | 27            | 26 | 25            | 24 | 23            | 22 | 21            | 20 | 19           | 18 | 17           | 16 |
| OSPEED15[1:0] |    | OSPEED14[1:0] |    | OSPEED13[1:0] |    | OSPEED12[1:0] |    | OSPEED11[1:0] |    | OSPEED10[1:0] |    | OSPEED9[1:0] |    | OSPEED8[1:0] |    |
| rw            | rw | rw            | rw | rw            | rw | rw            | rw | rw            | rw | rw            | rw | rw           | rw | rw           | rw |
| 15            | 14 | 13            | 12 | 11            | 10 | 9             | 8  | 7             | 6  | 5             | 4  | 3            | 2  | 1            | 0  |
| OSPEED7[1:0]  |    | OSPEED6[1:0]  |    | OSPEED5[1:0]  |    | OSPEED4[1:0]  |    | OSPEED3[1:0]  |    | OSPEED2[1:0]  |    | OSPEED1[1:0] |    | OSPEED0[1:0] |    |
| rw            | rw | rw            | rw | rw            | rw | rw            | rw | rw            | rw | rw            | rw | rw           | rw | rw           | rw |

Bits 31:0 OSPEED[15:0][1:0]: 端口 x 配置 I/O 引脚 y (y = 0..15)

这些位通过软件写入, 用于配置 I/O 输出速度。

00: 低速

01: 中速

10: 高速

11: 超高速

### 9.4.4 GPIO 端口上/下拉寄存器 (GPIOx\_PUPDR) (x = A to F)

地址偏移: 0x0C

端口 A 复位值: 0x6400 0000

端口 B 复位值: 0x0000 0100

其他端口复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

|             |    |             |    |             |    |             |    |             |    |            |    |            |    |            |    |
|-------------|----|-------------|----|-------------|----|-------------|----|-------------|----|------------|----|------------|----|------------|----|
| 31          | 30 | 29          | 28 | 27          | 26 | 25          | 24 | 23          | 22 | 21         | 20 | 19         | 18 | 17         | 16 |
| PUPD15[1:0] |    | PUPD14[1:0] |    | PUPD13[1:0] |    | PUPD12[1:0] |    | PUPD10[1:0] |    | PUPD9[1:0] |    | PUPD8[1:0] |    | PUPD7[1:0] |    |
| rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw         | rw | rw         | rw | rw         | rw |
| 15          | 14 | 13          | 12 | 11          | 10 | 9           | 8  | 7           | 6  | 5          | 4  | 3          | 2  | 1          | 0  |
| PUPD7[1:0]  |    | PUPD6[1:0]  |    | PUPD5[1:0]  |    | PUPD4[1:0]  |    | PUPD3[1:0]  |    | PUPD2[1:0] |    | PUPD1[1:0] |    | PUPD0[1:0] |    |
| rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw         | rw | rw         | rw | rw         | rw |

Bits 31:0 PUPD[15:0][1:0]: 端口 x 配置 I/O 引脚 y (y = 0...15)

这些位通过软件写入, 用于配置 I/O 上拉或下拉。

00: 浮空

01: 上拉

10: 下拉

11: 保留

### 9.4.5 GPIO 端口输入数据寄存器 (GPIOx\_IDR) (x = A to F)

地址偏移: 0x10

复位值: 0x0000 XXXX

访问: 无等待, 支持字访问

| 31       | 30    | 29    | 28    | 27    | 26    | 25   | 24   | 23   | 22   | 21   | 20   | 19   | 18   | 17   | 16   |
|----------|-------|-------|-------|-------|-------|------|------|------|------|------|------|------|------|------|------|
| Reserved |       |       |       |       |       |      |      |      |      |      |      |      |      |      |      |
| 15       | 14    | 13    | 12    | 11    | 10    | 9    | 8    | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| IDR15    | IDR14 | IDR13 | IDR12 | IDR11 | IDR10 | IDR9 | IDR8 | IDR7 | IDR6 | IDR5 | IDR4 | IDR3 | IDR2 | IDR1 | IDR0 |
| r        | r     | r     | r     | r     | r     | r    | r    | r    | r    | r    | r    | r    | r    | r    | r    |

Bits 31:16 保留, 必须保持为复位值。

Bits 15:0 ID[15:0]: 端口 x 输入数据 I/O 引脚 y (y = 0...15)

这些位为只读形式, 只能在字模式下访问。它们包含相应 I/O 端口的输入值。

### 9.4.6 GPIO 端口输出数据寄存器 (GPIOx\_ODR) (x = A to F)

地址偏移: 0x14

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

| 31       | 30   | 29   | 28   | 27   | 26   | 25  | 24  | 23  | 22  | 21  | 20  | 19  | 18  | 17  | 16  |
|----------|------|------|------|------|------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| Reserved |      |      |      |      |      |     |     |     |     |     |     |     |     |     |     |
| 15       | 14   | 13   | 12   | 11   | 10   | 9   | 8   | 7   | 6   | 5   | 4   | 3   | 2   | 1   | 0   |
| OD15     | OD14 | OD13 | OD12 | OD11 | OD10 | OD9 | OD8 | OD7 | OD6 | OD5 | OD4 | OD3 | OD2 | OD1 | OD0 |
| rw       | rw   | rw   | rw   | rw   | rw   | rw  | rw  | rw  | rw  | rw  | rw  | rw  | rw  | rw  | rw  |

Bits 31:16 保留, 必须保持为复位值。

Bits 15:0 OD[15:0]: 端口 x 输出数据 I/O 引脚 y (y = 0...15)

这些位可以由软件读取和写入。

注意: 对于原子置位/复位, 通过写入 GPIOx\_BSRR 寄存器, 可分别对 ODR 位进行置位和复位 (x = A..F)。

### 9.4.7 GPIO 端口位置/清零寄存器 (GPIOx\_BSRR) (x = A to F)

地址偏移: 0x18

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

| 31   | 30   | 29   | 28   | 27   | 26   | 25  | 24  | 23  | 22  | 21  | 20  | 19  | 18  | 17  | 16  |
|------|------|------|------|------|------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| BR15 | BR14 | BR13 | BR12 | BR11 | BR10 | BR9 | BR8 | BR7 | BR6 | BR5 | BR4 | BR3 | BR2 | BR1 | BR0 |
| w    | w    | w    | w    | w    | w    | w   | w   | w   | w   | w   | w   | w   | w   | w   | w   |
| 15   | 14   | 13   | 12   | 11   | 10   | 9   | 8   | 7   | 6   | 5   | 4   | 3   | 2   | 1   | 0   |
| BS15 | BS14 | BS13 | BS12 | BS11 | BS10 | BS9 | BS8 | BS7 | BS6 | BS5 | BS4 | BS3 | BS2 | BS1 | BS0 |
| w    | w    | w    | w    | w    | w    | w   | w   | w   | w   | w   | w   | w   | w   | w   | w   |

Bits 31:16 BRy: 端口 x 清零 I/O 引脚 y (y = 0...15)

这些位只能写。读取这些位将返回值 0x0000。

0: 对应 ODx 位不动

1: 清零对应的 ODx 位

注: 如果同时配置 BSx 和 BRx, 则 BSx 优先。

Bits 15:0 **BSy**: 端口 x 置位 I/O 引脚 y (y = 0...15)  
 这些位只能写。读取这些位将返回值 0x0000。  
 0: 对应 ODx 位不动  
 1: 置位对应的 ODx 位

#### 9.4.8 GPIO 端口配置锁寄存器 (GPIOx\_LCKR) (x = A to F)

地址偏移: 0x1C

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

|          |       |       |       |       |       |      |      |      |      |      |      |      |      |      |      |
|----------|-------|-------|-------|-------|-------|------|------|------|------|------|------|------|------|------|------|
| 31       | 30    | 29    | 28    | 27    | 26    | 25   | 24   | 23   | 22   | 21   | 20   | 19   | 18   | 17   | 16   |
| Reserved |       |       |       |       |       |      |      |      |      |      |      |      |      |      | LCKK |
|          |       |       |       |       |       |      |      |      |      |      |      |      |      |      | rw   |
| 15       | 14    | 13    | 12    | 11    | 10    | 9    | 8    | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| LCK15    | LCK14 | LCK13 | LCK12 | LCK11 | LCK10 | LCK9 | LCK8 | LCK7 | LCK6 | LCK5 | LCK4 | LCK3 | LCK2 | LCK1 | LCK0 |
| rw       | rw    | rw    | rw    | rw    | rw    | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   |

Bits 31:17 保留, 必须保持为复位值。

Bit 16 **LCKK**: 锁键

这个位可以随时读取。只能使用锁键写入序列修改。

0: 端口配置锁位键不激活

1: 端口配置锁位键被激活。下次系统复位前 GPIOx\_LCKR 寄存器被锁定。

锁键写顺序:

WR LCKR[16] = '1' + LCKR[15:0]

WR LCKR[16] = '0' + LCKR[15:0]

WR LCKR[16] = '1' + LCKR[15:0]

RD LCKR

RD LCKR[16] = '1' (此读取操作是可选的, 但可以用来确认锁键已被激活)

注意: 在 LOCK 键写顺序中, LCK[15:0] 的值不能改变。

锁序列中的任何错误都将中止锁。

在端口的任何位上的第一个锁定序列之后, LCKK 位上的任何读访问都返回1, 直到下一次 MCU 复位或外围设备复位。

Bits 15:0 **LCK[15:0]**: 端口 x 锁 I/O 引脚 y (y = 0...15)

这些位是可读/可写的, 但只有当 LCKK 位为 '0' 时才能写入。

0: 端口配置不锁定

1: 端口配置锁定

### 9.4.9 GPIO 复用功能低位寄存器 (GPIOx\_AFRL) (x = A to F)

地址偏移: 0x20

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

| 31         | 30 | 29 | 28 | 27         | 26 | 25 | 24 | 23         | 22 | 21 | 20 | 19         | 18 | 17 | 16 |
|------------|----|----|----|------------|----|----|----|------------|----|----|----|------------|----|----|----|
| AFRL7[3:0] |    |    |    | AFRL6[3:0] |    |    |    | AFRL5[3:0] |    |    |    | AFRL4[3:0] |    |    |    |
| rw         | rw | rw | rw | rw         | rw | rw | rw | rw         | rw | rw | rw | rw         | rw | rw | rw |
| 15         | 14 | 13 | 12 | 11         | 10 | 9  | 8  | 7          | 6  | 5  | 4  | 3          | 2  | 1  | 0  |
| AFRL3[3:0] |    |    |    | AFRL2[3:0] |    |    |    | AFRL1[3:0] |    |    |    | AFRL0[3:0] |    |    |    |
| rw         | rw | rw | rw | rw         | rw | rw | rw | rw         | rw | rw | rw | rw         | rw | rw | rw |

Bits 31:0 **AFSEL[7:0][3:0]**: 端口 x I/O 引脚 y 的替代功能选择 (y = 0...7)

这些位由软件写入, 以配置备用功能 I/O。

0000: AF0

0001: AF1

0010: AF2

0011: AF3

0100: AF4

0101: AF5

0110: AF6

0111: AF7

1000: AF8

1001: AF9

1010: AF10

1011: AF11

1100: AF12

1101: AF13

1110: AF14

1111: AF15

### 9.4.10 GPIO 复用功能高位寄存器 (GPIOx\_AFRH) (x = A to F)

地址偏移: 0x24

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

| 31          | 30 | 29 | 28 | 27          | 26 | 25 | 24 | 23          | 22 | 21 | 20 | 19          | 18 | 17 | 16 |
|-------------|----|----|----|-------------|----|----|----|-------------|----|----|----|-------------|----|----|----|
| AFRL15[3:0] |    |    |    | AFRL14[3:0] |    |    |    | AFRL13[3:0] |    |    |    | AFRL12[3:0] |    |    |    |
| rw          | rw | rw | rw | rw          | rw | rw | rw | rw          | rw | rw | rw | rw          | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11          | 10 | 9  | 8  | 7           | 6  | 5  | 4  | 3           | 2  | 1  | 0  |
| AFRL11[3:0] |    |    |    | AFRL10[3:0] |    |    |    | AFRL9[3:0]  |    |    |    | AFRL8[3:0]  |    |    |    |
| rw          | rw | rw | rw | rw          | rw | rw | rw | rw          | rw | rw | rw | rw          | rw | rw | rw |

Bits 31:0 **AFSEL[15:8][3:0]**: 端口 x I/O 引脚 y 的替代功能选择 (y = 8...15)

这些位由软件写入, 以配置备用功能 I/O。

0000: AF0

0001: AF1

0010: AF2

0011: AF3

0100: AF4  
0101: AF5  
0110: AF6  
0111: AF7  
1000: AF8  
1001: AF9  
1010: AF10  
1011: AF11  
1100: AF12  
1101: AF13  
1110: AF14  
1111: AF15

## 10 系统配置控制器 (SYSCFG)

### 10.1 SYSCFG 主要特征

RX32G410 系列具备一组系统配置寄存器。系统配置控制器的主要用途如下：

- 配置外部中断线连接到 GPIO

### 10.2 SYSCFG 寄存器

基地址：0x4001 0000

#### 10.2.1 外部中断配置寄存器 1 (SYSCFG\_EXTICR1)

地址偏移：0x08

复位值：0x0000 0000

访问：无等待，支持字，半字和字节访问

|            |    |    |    |            |    |    |    |            |    |    |    |            |    |    |    |
|------------|----|----|----|------------|----|----|----|------------|----|----|----|------------|----|----|----|
| 31         | 30 | 29 | 28 | 27         | 26 | 25 | 24 | 23         | 22 | 21 | 20 | 19         | 18 | 17 | 16 |
| Reserved   |    |    |    |            |    |    |    |            |    |    |    |            |    |    |    |
| 15         | 14 | 13 | 12 | 11         | 10 | 9  | 8  | 7          | 6  | 5  | 4  | 3          | 2  | 1  | 0  |
| EXTI3[3:0] |    |    |    | EXTI2[3:0] |    |    |    | EXTI1[3:0] |    |    |    | EXTI0[3:0] |    |    |    |
| rw         | rw | rw | rw | rw         | rw | rw | rw | rw         | rw | rw | rw | rw         | rw | rw | rw |

Bits 31:16 保留，必须保持为复位值。

Bits 15:0 **EXTIx[3:0]**：EXTIx 配置 (x = 3 到 0)

这些位可由软件读写，用于选择 EXTIx 外部中断的输入源。

0000: PAx

0001: PBx

0010: PCx

0011: PDx

0100: PEx

0101: PFx

#### 10.2.2 外部中断配置寄存器 2 (SYSCFG\_EXTICR2)

地址偏移：0x0C

复位值：0x0000 0000

访问：无等待，支持字，半字和字节访问

|            |    |    |    |            |    |    |    |            |    |    |    |            |    |    |    |
|------------|----|----|----|------------|----|----|----|------------|----|----|----|------------|----|----|----|
| 31         | 30 | 29 | 28 | 27         | 26 | 25 | 24 | 23         | 22 | 21 | 20 | 19         | 18 | 17 | 16 |
| Reserved   |    |    |    |            |    |    |    |            |    |    |    |            |    |    |    |
| 15         | 14 | 13 | 12 | 11         | 10 | 9  | 8  | 7          | 6  | 5  | 4  | 3          | 2  | 1  | 0  |
| EXTI7[3:0] |    |    |    | EXTI6[3:0] |    |    |    | EXTI5[3:0] |    |    |    | EXTI4[3:0] |    |    |    |
| rw         | rw | rw | rw | rw         | rw | rw | rw | rw         | rw | rw | rw | rw         | rw | rw | rw |

Bits 31:16 保留，必须保持为复位值。

Bits 15:0 **EXTIx[3:0]**：EXTIx 配置 (x = 7 到 4)

这些位可由软件读写，用于选择 EXTIx 外部中断的输入源。

0000: PAx

0001: PBx



0010: PCx

0011: PDx

0100: PEx

0101: PFx

### 10.2.3 外部中断配置寄存器 3 (SYSCFG\_EXTICR3)

地址偏移: 0x10

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

|             |    |    |    |             |    |    |    |            |    |    |    |            |    |    |    |
|-------------|----|----|----|-------------|----|----|----|------------|----|----|----|------------|----|----|----|
| 31          | 30 | 29 | 28 | 27          | 26 | 25 | 24 | 23         | 22 | 21 | 20 | 19         | 18 | 17 | 16 |
| Reserved    |    |    |    |             |    |    |    |            |    |    |    |            |    |    |    |
| 15          | 14 | 13 | 12 | 11          | 10 | 9  | 8  | 7          | 6  | 5  | 4  | 3          | 2  | 1  | 0  |
| EXTI11[3:0] |    |    |    | EXTI10[3:0] |    |    |    | EXTI9[3:0] |    |    |    | EXTI8[3:0] |    |    |    |
| rw          | rw | rw | rw | rw          | rw | rw | rw | rw         | rw | rw | rw | rw         | rw | rw | rw |

Bits 31:16 保留, 必须保持为复位值。

Bits 15:0 **EXTIx[3:0]**: EXTIx 配置 (x = 11 到 8)

这些位可由软件读写, 用于选择 EXTIx 外部中断的输入源。

0000: PAx

0001: PBx

0010: PCx

0011: PDx

0100: PEx

0101: PFx

### 10.2.4 外部中断配置寄存器 4 (SYSCFG\_EXTICR4)

地址偏移: 0x14

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

|             |    |    |    |             |    |    |    |             |    |    |    |             |    |    |    |
|-------------|----|----|----|-------------|----|----|----|-------------|----|----|----|-------------|----|----|----|
| 31          | 30 | 29 | 28 | 27          | 26 | 25 | 24 | 23          | 22 | 21 | 20 | 19          | 18 | 17 | 16 |
| Reserved    |    |    |    |             |    |    |    |             |    |    |    |             |    |    |    |
| 15          | 14 | 13 | 12 | 11          | 10 | 9  | 8  | 7           | 6  | 5  | 4  | 3           | 2  | 1  | 0  |
| EXTI15[3:0] |    |    |    | EXTI14[3:0] |    |    |    | EXTI13[3:0] |    |    |    | EXTI12[3:0] |    |    |    |
| rw          | rw | rw | rw | rw          | rw | rw | rw | rw          | rw | rw | rw | rw          | rw | rw | rw |

Bits 31:16 保留, 必须保持为复位值。

Bits 15:0 **EXTIx[3:0]**: EXTIx 配置 (x = 15 到 12)

这些位可由软件读写, 用于选择 EXTIx 外部中断的输入源。

0000: PAx

0001: PBx

0010: PCx

0011: PDx

0100: PEx

0101: PFx

### 10.2.5 SYSCFG CCM SRAM 控制和状态寄存器 (SYSCFG\_SCSR)

地址偏移: 0x18

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

|          |    |    |    |    |    |    |    |    |    |    |    |    |    |            |           |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|------------|-----------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17         | 16        |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    |            |           |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1          | 0         |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    | CCM<br>BSY | CCM<br>ER |
|          |    |    |    |    |    |    |    |    |    |    |    |    |    | r          | rw        |

Bits 31:2 保留, 必须保持为复位值。

Bit 1 **CCMBSY**: CCM SRAM 因擦除操作而繁忙

0: 没有 CCM SRAM 擦除操作正在运行

1: CCM SRAM 擦除操作正在运行中

Bit 0 **CCMER**: CCM SRAM 擦除

设置此位启动硬件 CCM SRAM 擦除操作。

该位在 CCM SRAM 擦除操作结束时自动清除

### 10.2.6 SYSCFG 配置寄存器 2 (SYSCFG\_CFGR2)

地址偏移: 0x1C

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

|          |    |    |    |    |    |       |       |          |    |    |    |      |      |     |     |
|----------|----|----|----|----|----|-------|-------|----------|----|----|----|------|------|-----|-----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25    | 24    | 23       | 22 | 21 | 20 | 19   | 18   | 17  | 16  |
| Reserved |    |    |    |    |    |       |       |          |    |    |    |      |      |     |     |
| 15       | 14 | 13 | 12 | 11 | 10 | 9     | 8     | 7        | 6  | 5  | 4  | 3    | 2    | 1   | 0   |
| Reserved |    |    |    |    |    | SPF2  | SPF   | Reserved |    |    |    | ECCL | PVDL | SPL | CLL |
|          |    |    |    |    |    | rc_w1 | rc_w1 |          |    |    |    | rs   | rs   | rs  | rs  |

Bits 31:10 保留, 必须保持为复位值。

Bit 9 **SPF2**: SRAM1 和 CCM SRAM 资料奇偶校验错误标志

当检测到 SRAM1 或 CCM SRAM 奇偶校验错误时, 该位由硬件设置。

软件通过输入 1 来清除

0: 表示未检测到奇偶错误

1: 检测到奇偶校验错误

Bit 8 **SPF**: SRAM1 和 CCM SRAM 指令奇偶校验错误标志

当检测到 SRAM1 或 CCM SRAM 奇偶校验错误时, 该位由硬件设置。

软件通过输入 1 来清除

0: 表示未检测到奇偶错误

1: 检测到奇偶校验错误

Bits 7:4 保留, 必须保持为复位值。

Bit 3 **ECCL**: ECC 锁

该位由软件设置, 只有通过系统重置才能清除。

它可以用来使能和锁定 Flash ECC 错误连接到 TIM1/8/15/16/17 break 输入和

HRTIM1 的 hrtim\_sys\_flt 输入

0: ECC 错误从 TIM1/8/15/16/17 break 输入和 HRTIM1 的 hrtim\_sys\_flt 输入断开。

1: ECC 错误连接到 TIM1/8/15/16/17 break 输入和 HRTIM1 的 hrtim\_sys\_flt 输入。

Bit 2 **PVDL**: PVD 锁使能位

该位由软件设置，只有通过系统重置才能清除。

它可以用来使能和锁定 PVD 连接到 TIM1/8/15/16/17 break 输入和 HRTIM1 的 hrtim\_sys\_flt 输入，以及 PWR\_CR2 寄存器中的 PVDE 和 PLS[2:0]

0: PVD 中断从 TIM1/8/15/16/17 break 输入和 HRTIM1 的 hrtim\_sys\_flt 输入断开，PVDE 和 PLS[2:0]位可以由应用程序编程。

1: PVD 中断连接到 TIM1/8/15/16/17 break 输入和 HRTIM1 的 hrtim\_sys\_flt 输入，PVDE 和 PLS[2:0]位只读。

Bit 1 **SPL**: SRAM1 和 CCM SRAM 校验锁位

该位由软件设置，只有通过系统重置才能清除。它可以用来使能和锁定 SRAM1 或 CCM SRAM 奇偶校验错误信号连接到 TIM1/8/15/16/17 break 输入和 HRTIM1 的 hrtim\_sys\_flt 输入

0: CCM SRAM 校验错误信号从 TIM1/8/15/16/17 break 输入和 HRTIM1 的 hrtim\_sys\_flt 输入断开

1: CCM SRAM 奇偶校验错误信号连接到 TIM1/8/15/16/17 break 输入和 HRTIM1 的 hrtim\_sys\_flt 输入

Bit 0 **CLL**: Cortex®-M4 LOCKUP (Hardfault) 输出使能位

该位由软件设置，只有通过系统重置才能清除。它可以用来使能和锁定 Cortex®-M4 与 FPU LOCKUP (Hardfault) 输出到 TIM1/8/15/16/17 break 输入和 hrtim\_sys\_flt 输入的连接。

0: Cortex-M4 LOCKUP 输出从 TIM1/8/15/16/17 break 输入和 HRTIM1 的 hrtim\_sys\_flt 输入断开。

1: Cortex-M4 LOCKUP 输出连接到 TIM1/8/15/16/17 break 输入和 HRTIM1 的 hrtim\_sys\_flt 输入。

## 10.2.7 SYSCFG CCM SRAM 键寄存器 (SYSCFG\_SKR)

地址偏移: 0x24

复位值: 0x0000 0000

访问: 无等待, 支持字, 半字和字节访问

|          |    |    |    |    |    |    |    |          |    |    |    |    |    |    |    |
|----------|----|----|----|----|----|----|----|----------|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23       | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |    |    |          |    |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7        | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    |    |    |    |    | KEY[7:0] |    |    |    |    |    |    |    |
|          |    |    |    |    |    |    |    | rw       | rw | rw | rw | rw | rw | rw | rw |

Bits 31:8 保留，必须保持为复位值。

Bit 7:0 **KEY[7:0]**: CCM SRAM 写保护用于软件擦除

解锁 SYSCFG\_SCSR 寄存器中 CCMER 位的写保护，需要执行以下步骤

1、将“0xCA”写入 KEY[7:0]

2、将“0x53”写入 KEY[7:0]

注: 写除 0xCA, 0x53 的任意值来开启擦除保护

## 11 嵌套向量中断控制器 (NVIC)

### 11.1 NVIC 主要特征

- 102 个可屏蔽中断通道 (不包含 16 个带 FPU 的 Cortex®-M4 中断线)
- 16 个可编程的优先等级 (使用 4 位中断优先级)
- 低延迟的异常和中断处理
- 电源管理控制
- 系统控制寄存器的实现

嵌套向量中断控制器 (NVIC) 和处理器核的接口紧密相连, 可以实现低延迟的中断处理和高效地处理晚到的中断。

### 11.2 中断向量表

表 11.1 RX32G410 系列向量表

| 位置 | 优先级 | 优先级类型 | 名称                 | 说明                                  | 地址                           |
|----|-----|-------|--------------------|-------------------------------------|------------------------------|
|    | -   | -     | -                  | 保留                                  | 0x0000_0000                  |
|    | -3  | 固定    | Reset              | 复位                                  | 0x0000_0004                  |
|    | -2  | 固定    | NMI                | 不可屏蔽中断<br>RCC 时钟安全系统(CSS)联接到 NMI 向量 | 0x0000_0008                  |
|    | -1  | 固定    | 硬件失效(HardFault)    | 所有类型的失效                             | 0x0000_000C                  |
|    | 0   | 可设置   | 存储管理(MemManage)    | 存储器管理                               | 0x0000_0010                  |
|    | 1   | 可设置   | 总线错误(BusFault)     | 预取指失败, 存储器访问失败                      | 0x0000_0014                  |
|    | 2   | 可设置   | 错误应用(UsageFault)   | 未定义的指令或非法状态                         | 0x0000_0018                  |
|    | -   | -     | -                  | 保留                                  | 0x0000_001C ~<br>0x0000_002B |
|    | 3   | 可设置   | SVCall             | 通过 SWI 指令的系统服务调用                    | 0x0000_002C                  |
|    | 4   | 可设置   | 调试监控(DebugMonitor) | 调试监控器                               | 0x0000_0030                  |
|    | -   | -     | -                  | 保留                                  | 0x0000_0034                  |
|    | 5   | 可设置   | PendSV             | 可挂起的系统服务                            | 0x0000_0038                  |
|    | 6   | 可设置   | SysTick            | 系统嘀嗒定时器                             | 0x0000_003C                  |
| 0  | 7   | 可设置   | WWDG               | 窗口定时器中断                             | 0x0000_0040                  |
| 1  | 8   | 可设置   | PVD                | 连到 EXTI 的电源电压检测(PVD)中断              | 0x0000_0044                  |
| 2  | 9   | 可设置   | TAMPER             | 侵入检测中断                              | 0x0000_0048                  |
| 3  | 10  | 可设置   | RTC                | 实时时钟(RTC)全局中断                       | 0x0000_004C                  |
| 4  | 11  | 可设置   | FLASH              | 闪存全局中断                              | 0x0000_0050                  |
| 5  | 12  | 可设置   | RCC                | 复位和时钟控制(RCC)中断                      | 0x0000_0054                  |
| 6  | 13  | 可设置   | EXTI0              | EXTI 线 0 中断                         | 0x0000_0058                  |
| 7  | 14  | 可设置   | EXTI1              | EXTI 线 1 中断                         | 0x0000_005C                  |
| 8  | 15  | 可设置   | EXTI2              | EXTI 线 2 中断                         | 0x0000_0060                  |
| 9  | 16  | 可设置   | EXTI3              | EXTI 线 3 中断                         | 0x0000_0064                  |
| 10 | 17  | 可设置   | EXTI4              | EXTI 线 4 中断                         | 0x0000_0068                  |
| 11 | 18  | 可设置   | DMA1 通道 1          | DMA1 通道 1 全局中断                      | 0x0000_006C                  |

| 位置 | 优先级 | 优先级类型 | 名称                                           | 说明                                                                                             | 地址                           |
|----|-----|-------|----------------------------------------------|------------------------------------------------------------------------------------------------|------------------------------|
| 12 | 19  | 可设置   | DMA1 通道 2                                    | DMA1 通道 2 全局中断                                                                                 | 0x0000_0070                  |
| 13 | 20  | 可设置   | DMA1 通道 3                                    | DMA1 通道 3 全局中断                                                                                 | 0x0000_0074                  |
| 14 | 21  | 可设置   | DMA1 通道 4                                    | DMA1 通道 4 全局中断                                                                                 | 0x0000_0078                  |
| 15 | 22  | 可设置   | DMA1 通道 5                                    | DMA1 通道 5 全局中断                                                                                 | 0x0000_007C                  |
| 16 | 23  | 可设置   | DMA1 通道 6                                    | DMA1 通道 6 全局中断                                                                                 | 0x0000_0080                  |
| 17 | 24  | 可设置   | DMA1 通道 7                                    | DMA1 通道 7 全局中断                                                                                 | 0x0000_0084                  |
| 18 | 25  | 可设置   | ADC1_2                                       | ADC1 和 ADC2 的全局中断                                                                              | 0x0000_0088                  |
| 19 | 26  | 可设置   | USB_HP_CAN1_TX                               | USB 设备高优先级和 CAN1 发送中断                                                                          | 0x0000_008C                  |
| 20 | 27  | 可设置   | USB_LP_CAN1_RX0                              | USB 设备低优先级和 CAN1 接收 0 中断                                                                       | 0x0000_0090                  |
| 21 | 28  | 可设置   | CAN1_RX1                                     | CAN1 接收 1 中断                                                                                   | 0x0000_0094                  |
| 22 | 29  | 可设置   | CAN1_SCE                                     | CAN1 SCE 中断                                                                                    | 0x0000_0098                  |
| 23 | 30  | 可设置   | EXTI9_5                                      | EXTI 线[9:5]中断                                                                                  | 0x0000_009C                  |
| 24 | 31  | 可设置   | TIM1_BRK/TIM15                               | TIM1 Break/TIM15 global interrupts                                                             | 0x0000_00A0                  |
| 25 | 32  | 可设置   | TIM1_UP/TIM16                                | TIM1 Update/TIM16 global interrupts                                                            | 0x0000_00A4                  |
| 26 | 33  | 可设置   | TIM1_TRG_COM/<br>TIM17/<br>TIM1_DIR/TIM1_IDX | TIM1 trigger and commutation/TIM17<br>interrupts/TIM1 Direction Change<br>interrupt/TIM1 Index | 0x0000_00A8                  |
| 27 | 34  | 可设置   | TIM1_CC                                      | TIM1 捕获比较中断                                                                                    | 0x0000_00AC                  |
| 28 | 35  | 可设置   | TIM2                                         | TIM2 全局中断                                                                                      | 0x0000_00B0                  |
| 29 | 36  | 可设置   | TIM3                                         | TIM3 全局中断                                                                                      | 0x0000_00B4                  |
| 30 | 37  | 可设置   | TIM4                                         | TIM4 全局中断                                                                                      | 0x0000_00B8                  |
| 31 | 38  | 可设置   | I2C1_EV                                      | I2C1 事件中断                                                                                      | 0x0000_00BC                  |
| 32 | 39  | 可设置   | I2C1_ER                                      | I2C1 错误中断                                                                                      | 0x0000_00C0                  |
| 33 | 40  | 可设置   | I2C2_EV                                      | I2C2 事件中断                                                                                      | 0x0000_00C4                  |
| 34 | 41  | 可设置   | I2C2_ER                                      | I2C2 错误中断                                                                                      | 0x0000_00C8                  |
| 35 | 42  | 可设置   | SPI1                                         | SPI1 全局中断                                                                                      | 0x0000_00CC                  |
| 36 | 43  | 可设置   | SPI2                                         | SPI2 全局中断                                                                                      | 0x0000_00D0                  |
| 37 | 44  | 可设置   | USART1                                       | USART1 全局中断                                                                                    | 0x0000_00D4                  |
| 38 | 45  | 可设置   | USART2                                       | USART2 全局中断                                                                                    | 0x0000_00D8                  |
| 39 | 46  | 可设置   | USART3                                       | USART3 全局中断                                                                                    | 0x0000_00DC                  |
| 40 | 47  | 可设置   | EXTI15_10                                    | EXTI 线[15:10]中断                                                                                | 0x0000_00E0                  |
| 41 | 48  | 可设置   | RTCAlarm                                     | 连到 EXTI 的 RTC 闹钟中断                                                                             | 0x0000_00E4                  |
| 42 | 49  | 可设置   | USB WakeUp                                   | USB 通过 EXTI 线唤醒中断                                                                              | 0x0000_00E8                  |
| 43 | 50  | 可设置   | TIM8_BRK                                     | TIM8 刹车中断                                                                                      | 0x0000_00EC                  |
| 44 | 51  | 可设置   | TIM8_UP                                      | TIM8 更新中断                                                                                      | 0x0000_00F0                  |
| 45 | 52  | 可设置   | TIM8_TRG_COM                                 | TIM8 触发和通信中断                                                                                   | 0x0000_00F4                  |
| 46 | 53  | 可设置   | TIM8_CC                                      | TIM8 捕获比较中断                                                                                    | 0x0000_00F8                  |
| 47 | 54  | 可设置   | ADC3                                         | ADC3 全局中断                                                                                      | 0x0000_00FC                  |
| -  | -   | -     | 保留                                           | 保留                                                                                             | 0x0000_0100 -<br>0x0000_0104 |
| 50 | 57  | 可设置   | TIM5                                         | TIM5 global interrupt                                                                          | 0x0000_0108                  |
| -  | -   | -     | 保留                                           | 保留                                                                                             | 0x0000_010C                  |

| 位置  | 优先级 | 优先级类型 | 名称                 | 说明                                                          | 地址                           |
|-----|-----|-------|--------------------|-------------------------------------------------------------|------------------------------|
| 52  | 59  | 可设置   | UART4              | UART4 global interrupt                                      | 0x0000_0110                  |
| 53  | 60  | 可设置   | UART5              | UART5 global inter                                          | 0x0000_0114                  |
| 54  | 61  | 可设置   | TIM6_DACUNDER      | TIM6 and DAC1/3 underrun global interrupt                   | 0x0000_0118                  |
| 55  | 62  | 可设置   | TIM6_DACUNDER      | TIM7 and DAC2 undeglobal interrupt                          | 0x0000_011C                  |
| 56  | 63  | 可设置   | DMA2 通道 1          | DMA2 通道 1 全局中断                                              | 0x0000_0120                  |
| 57  | 64  | 可设置   | DMA2 通道 2          | DMA2 通道 2 全局中断                                              | 0x0000_0124                  |
| 58  | 65  | 可设置   | DMA2 通道 3          | DMA2 通道 3 全局中断                                              | 0x0000_0128                  |
| 59  | 66  | 可设置   | DMA2 通道 4          | DMA2 通道 4 全局中断                                              | 0x0000_012C                  |
| 60  | 67  | 可设置   | DMA2 通道 5          | DMA2 通道 5 全局中断                                              | 0x0000_0130                  |
| -   | -   | -     | 保留                 | 保留                                                          | 0x0000_0134 -<br>0x0000_013C |
| 64  | 71  | 可设置   | CMP1_2_3           | COMP1/COMP2/COMP3 through EXTI lines<br>21/22/29 interrupts | 0x0000_0140                  |
| 65  | 72  | 可设置   | CMP4               | CMP4 全局中断 COMP4 through EXTI lines<br>30 interrupts         | 0x0000_0144                  |
| -   | -   | -     | 保留                 | 保留                                                          | 0x0000_0148                  |
| 67  | 74  | 可设置   | HRTIM_Master_IRQn  | HRTIM master timer interrupt (hrtim_it1)                    | 0x0000_014C                  |
| 68  | 75  | 可设置   | HRTIM_TIMA_IRQn    | HRTIM timer A interrupt (hrtim_it2)                         | 0x0000_0150                  |
| 69  | 76  | 可设置   | HRTIM_TIMB_IRQn    | HRTIM timer B interrupt (hrtim_it3)                         | 0x0000_0154                  |
| 70  | 77  | 可设置   | HRTIM_TIMC_IRQn    | HRTIM timer C interrupt (hrtim_it4)                         | 0x0000_0158                  |
| 71  | 78  | 可设置   | HRTIM_TIMD_IRQn    | HRTIM timer D interrupt (hrtim_it5)                         | 0x0000_015C                  |
| 72  | 79  | 可设置   | HRTIM_TIME_IRQn    | HRTIM timer E interrupt (hrtim_it6)                         | 0x0000_0160                  |
| 73  | 80  | 可设置   | HRTIM_TIM_FLT_IRQn | HRTIM fault interrupt (hrtim_it8)                           | 0x0000_0164                  |
| 74  | 81  | 可设置   | HRTIM_TIMF_IRQn    | hrtim_it7 / HRTIM timer F interrupt                         | 0x0000_0168                  |
| -   | -   | -     | 保留                 | 保留                                                          | 0x0000_016C -<br>0x0000_0194 |
| 86  | 93  | 可设置   | CAN2_TX            | CAN2 发送中断                                                   | 0x0000_0198                  |
| 87  | 94  | 可设置   | CAN2_RX0           | CAN2 接收 0 中断                                                | 0x0000_019C                  |
| 88  | 95  | 可设置   | CAN2_RX1           | CAN2 接收 1 中断                                                | 0x0000_01A0                  |
| 89  | 96  | 可设置   | CAN2_SCE           | CAN2 SCE 中断                                                 | 0x0000_01A4                  |
| 90  | 97  |       | CLC1               | CLC1 Positive or NegativeEdge interrupt                     | 0x0000_01A8                  |
| 91  | 98  |       | CLC2               | CLC2 Positive or NegativeEdge interrupt                     | 0x0000_01AC                  |
| 92  | 99  |       | CLC3               | CLC3 Positive or NegativeEdge interrupt                     | 0x0000_01B0                  |
| 93  | 100 |       | CLC4               | CLC4 Positive or NegativeEdge interrupt                     | 0x0000_01B4                  |
| 94  | 101 |       | DMAMUX_OVR         | DMAMUX Overrun interrupt                                    | 0x0000_01B8                  |
| -   | -   | -     | 保留                 | 保留                                                          | 0x0000_01BC                  |
| 96  | 103 |       | DMA1_CH8           | DMA1 channel 8 interrupt                                    | 0x0000_01C0                  |
| 97  | 104 |       | DMA2_CH6           | DMA2 channel 6 interrupt                                    | 0x0000_01C4                  |
| 98  | 105 |       | DMA2_CH7           | DMA2 channel 7 interrupt                                    | 0x0000_01C8                  |
| 99  | 106 |       | DMA2_CH8           | DMA2 channel 8 interrupt                                    | 0x0000_01CC                  |
| 100 | 107 |       | CORDIC             | CORDIC interrupt                                            | 0x0000_01D0                  |
| 101 | 108 |       | FMAC               | FMAC interrupt                                              | 0x0000_01D4                  |



## 12 外部中断/事件控制器(EXTI)

### 12.1 介绍

EXTI 的主要特性如下:

- 支持多达 23 个软件事件/中断请求
- 每个事件/中断线上都有独立的屏蔽
- 可配置上升沿或下降沿 (仅限可配置线)
- 专用状态位 (仅限可配置线)
- 模拟事件/中断请求 (仅限可配置线)

### 12.2 EXTI 的主要特性

外部中断/事件控制器(EXTI)管理外部和内部异步事件/中断,并向 CPU/中断控制器生成事件请求和向电源控制器生成唤醒请求。

EXTI 允许管理多个事件线,这些事件线可以从停止模式唤醒。

这些线路要么是可配置的,要么是直接的:

- 线路是可配置的:活动边可以独立选择,状态标志表示中断的来源。可配置的线路由 I/O 外部中断和少数外设使用。
- 线路是直接的:它们被一些外设用来从停止事件或中断中产生唤醒。状态标志位由外设提供。

每个输入线都可以独立地被中断或事件屏蔽。

该控制器还允许通过软件模拟事件或中断,通过写入专用寄存器与相应的事件线进行多路复用。

### 12.3 EXTI 功能介绍

对于可配置的中断线,应该配置并使能中断线以生成中断。这是通过用所需的边缘检测编程两个触发寄存器,并通过向中断屏蔽寄存器中的相应位写入 '1' 来使能中断请求来完成的。当选择的边缘出现在中断线上时,就会产生中断请求。与中断线对应的挂起位也被设置。通过在挂起寄存器中写入 '1' 来清除此请求。

对于直接中断线,中断在默认情况下在中断屏蔽寄存器中使能,并且在挂起寄存器中没有相应的挂起位。

为了生成事件,应该配置并使能事件线。这是通过用所需的边缘检测编程两个触发寄存器并通过向事件屏蔽寄存器中的相应位写入 '1' 来使能事件请求来完成的。当选定的边缘出现在事件线上时,就会产生一个事件脉冲。未设置事件线对应的挂起位。

对于可配置行,中断/事件请求也可以由软件通过在软件中断/事件寄存器中写入 '1' 来生成。

注意:只有当系统处于 Stop 模式时,才会触发与直线相关的中断或事件。如果系统仍在运行,则 EXTI 不会产生中断/事件。



### 12.3.1 EXTI 框图

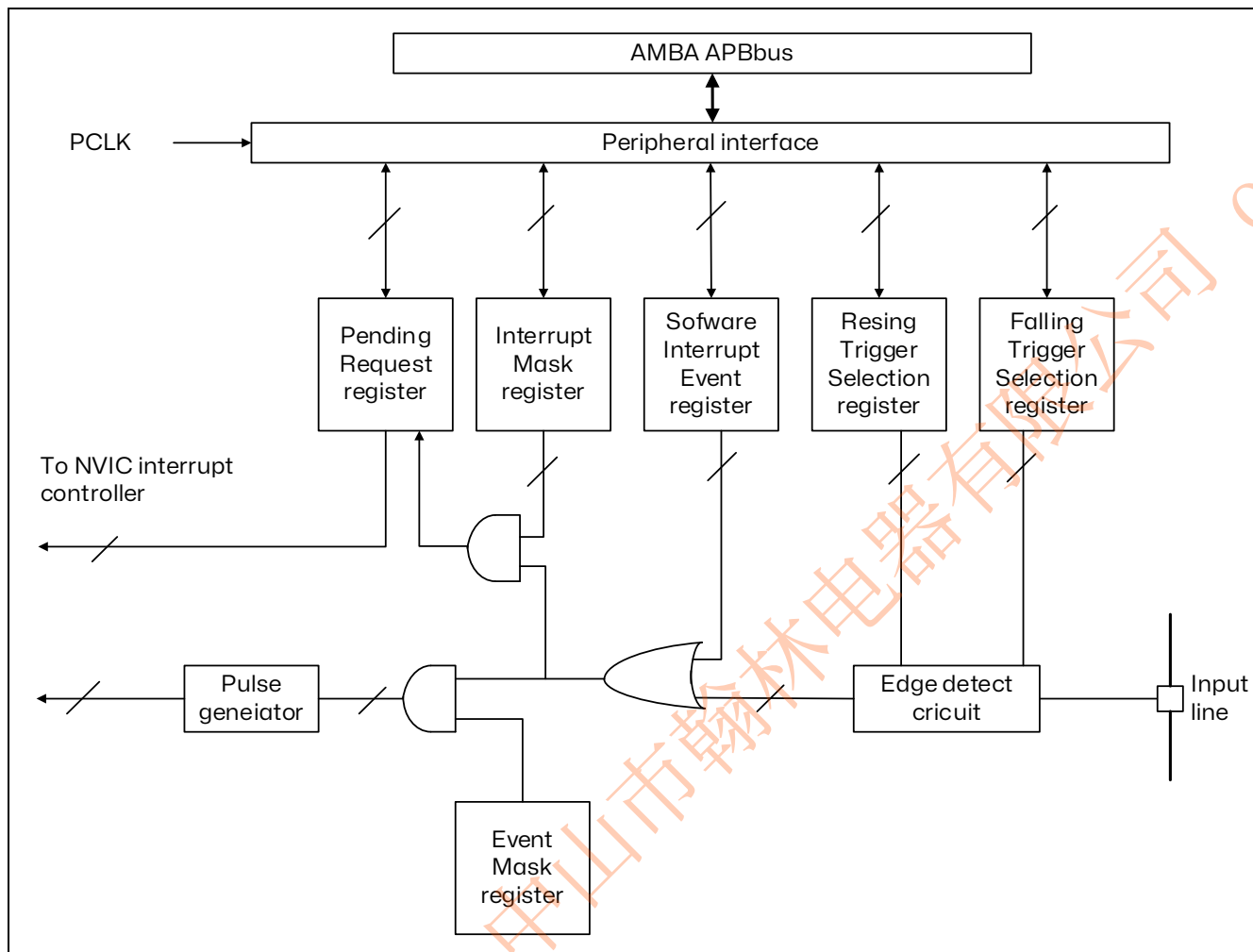


图 12.1 可配置的中断/事件框图

### 12.3.2 唤醒事件管理

RX32G410 能够处理外部或内部事件来唤醒核心(WFE)。唤醒事件可以通过以下方式生成:

- 在外设控制寄存器中使能中断，但不在 NVIC 中使能中断，并在 Cortex-M4 系统控制寄存器中使能 SEVONPEND 位。当 MCU 从 WFE 恢复时，必须清除 EXTI 外设中断挂起位和外设 NVIC 中断通道挂起位(在 NVIC 中断清除挂起寄存器中)
- 通过在事件模式下配置 EXTI 线。当 CPU 从 WFE 恢复时，不需要清除外设中断挂起位或 NVIC 中断通道挂起位，因为没有设置与事件线对应的挂起位。

### 12.3.3 外设异步中断

有些外设能够在系统处于运行模式和停止模式时生成事件，从而允许从停止模式唤醒系统。

为了完成这个任务，外设生成一个同步到系统时钟(例如 APB 时钟)和一个异步版本的事件。这个异步事件连接到一条 EXTI 直线上。

注意:很少有具有停止唤醒功能的外设连接到 EXTI 可配置线路。EXTI 允许从停止模式到唤醒。

#### 12.3.4 硬件中断选择

要配置线路为中断源，请使用以下步骤：

1. 配置相应的屏蔽位(EXTI\_IMR)。
2. 配置中断线的触发选择位(EXTI\_RTSR 和 EXTI\_FTSR)。
3. 配置控制映射到 EXTI 的 NVIC 中断通道的使能和屏蔽位，以便能够正确识别来自其中一条 EXTI 线路的中断。

注意：直线不需要任何 EXTI 配置。

#### 12.3.5 硬件事件选择

要配置线路为事件源，请使用以下步骤：

1. 配置相应的屏蔽位(EXTI\_EMR)。
2. 配置事件线的触发选择位(EXTI\_RTSR 和 EXTI\_FTSR)。

#### 12.3.6 软件中断/事件选择

任何可配置的线都可以配置为软件中断/事件线。生成软件中断的过程如下：

1. 配置对应的屏蔽位(EXTI\_IMR,EXTI\_EMR)。
2. 设置软件中断寄存器的请求位(EXTI\_SWIER)。

### 12.4 EXTI 中断/事件线映射

GPIO 连接到 16 条可配置的中断/事件线（参见下图）

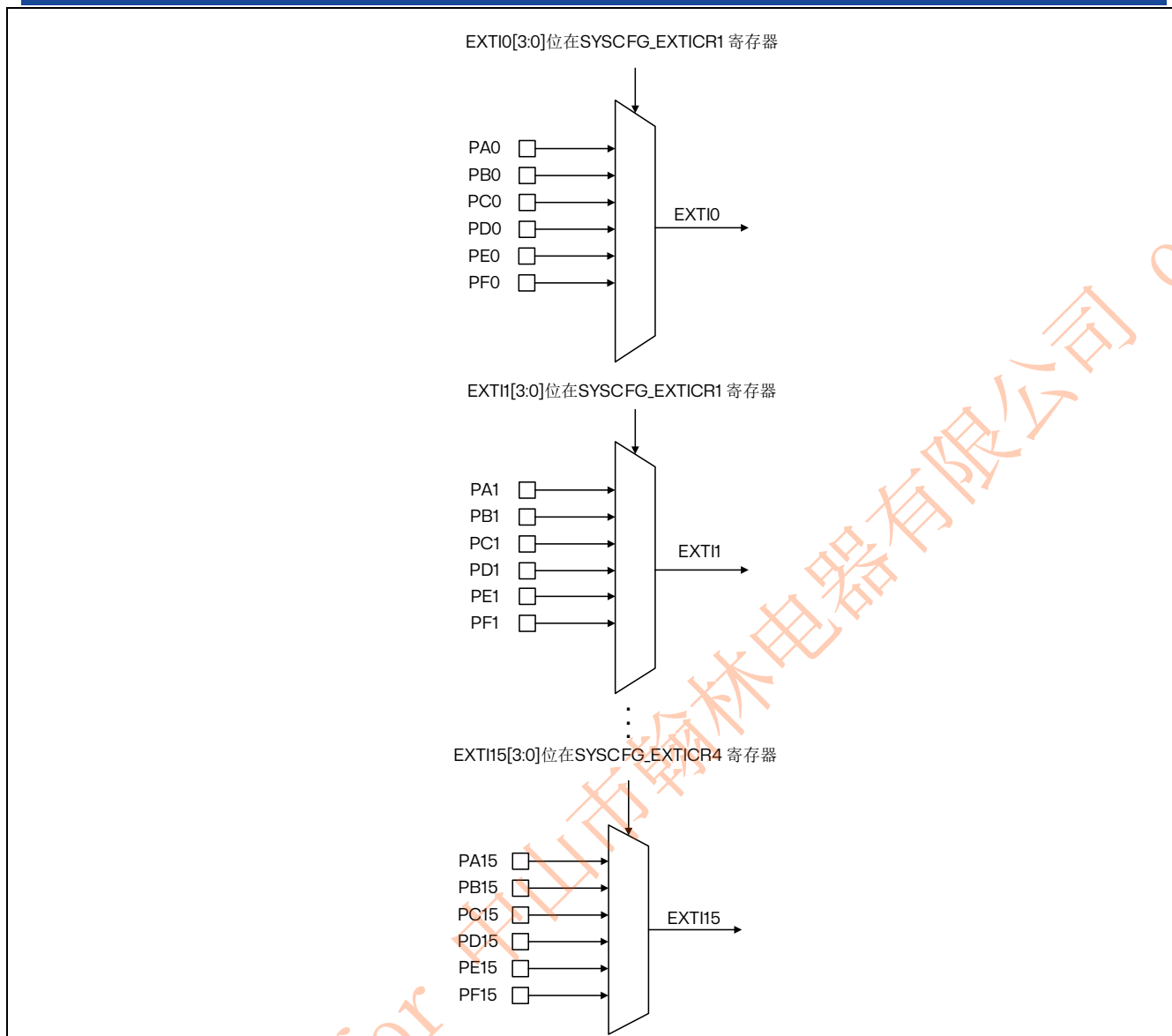


图 12.2 外部中断/事件 GPIO 映射

表 12.1 EXTI 线路连接

| EXTI 线 | 信号源            | 类型  |
|--------|----------------|-----|
| 0-15   | GPIO           | 可配置 |
| 16     | PVD            | 可配置 |
| 17     | RTC 闹钟事件       | 可配置 |
| 18     | USB 设备 FS 唤醒事件 | 直接  |
| 19     | 保留             |     |
| 20     | 保留             |     |
| 21     | COMP1 输出       | 可配置 |
| 22     | COMP2 输出       | 可配置 |
| 23-28  | 保留             |     |
| 29     | COMP3 输出       | 可配置 |
| 30     | COMP4 输出       | 可配置 |

## 12.5 EXTI 寄存器描述

基地址：0x4001 0400 - 0x4001 07FF

外设寄存器必须通过字(32 位)来访问。

### 12.5.1 EXTI 中断屏蔽寄存器 (EXTI\_IMR)

地址偏移量：0x00

复位值：连线路设为“1”，其他线路设为“0”。

|      |      |      |          |      |      |     |     |     |      |      |          |     |     |      |      |      |
|------|------|------|----------|------|------|-----|-----|-----|------|------|----------|-----|-----|------|------|------|
| 31   | 30   | 29   | 28       | 27   | 26   | 25  | 24  | 23  | 22   | 21   | 20       | 19  | 18  | 17   | 16   |      |
| Res  | IM30 | IM29 | Reserved |      |      |     |     |     | IM22 | IM21 | Reserved |     |     | IM18 | IM17 | IM16 |
|      | rw   | rw   |          |      |      |     |     |     | rw   | rw   |          |     |     | rw   | rw   |      |
| 15   | 14   | 13   | 12       | 11   | 10   | 9   | 8   | 7   | 6    | 5    | 4        | 3   | 2   | 1    | 0    |      |
| IM15 | IM14 | IM13 | IM12     | IM11 | IM10 | IM9 | IM8 | IM7 | IM6  | IM5  | IM4      | IM3 | IM2 | IM1  | IM0  |      |
| rw   | rw   | rw   | rw       | rw   | rw   | rw  | rw  | rw  | rw   | rw   | rw       | rw  | rw  | rw   | rw   |      |

Bit 31 保留，必须保持为复位值。

Bit 30:29 **IMx**：线 x 上的中断屏蔽 (x=30~29)

0：屏蔽来自线 x 上的中断请求

1：开放来自线 x 上的中断请求

注意：为了在默认情况下使能中断，直接线路的复位值被设置为“1”。

Bit 28:23 保留，必须保持为复位值。

Bit 22:21 **IMx**：线 x 上的中断屏蔽 (x=22~21)

0：屏蔽来自线 x 上的中断请求

1：开放来自线 x 上的中断请求

注意：为了在默认情况下使能中断，直接线路的复位值被设置为“1”。

Bit 20:19 保留，必须保持为复位值。

Bit 18:0 **IMx**：线 x 上的中断屏蔽 (x=18~0)

0：屏蔽来自线 x 上的中断请求

1：开放来自线 x 上的中断请求

注意：为了在默认情况下使能中断，直接线路的复位值被设置为“1”。

### 12.5.2 EXTI 事件屏蔽寄存器 (EXTI\_EMR)

地址偏移量：0x04

复位值：0x0000 0000

|      |      |      |          |      |      |     |     |     |      |      |          |     |     |      |      |      |
|------|------|------|----------|------|------|-----|-----|-----|------|------|----------|-----|-----|------|------|------|
| 31   | 30   | 29   | 28       | 27   | 26   | 25  | 24  | 23  | 22   | 21   | 20       | 19  | 18  | 17   | 16   |      |
| Res  | EM30 | EM29 | Reserved |      |      |     |     |     | EM22 | EM21 | Reserved |     |     | EM18 | EM17 | EM16 |
|      | rw   | rw   |          |      |      |     |     |     | rw   | rw   |          |     |     | rw   | rw   |      |
| 15   | 14   | 13   | 12       | 11   | 10   | 9   | 8   | 7   | 6    | 5    | 4        | 3   | 2   | 1    | 0    |      |
| EM15 | EM14 | EM13 | EM12     | EM11 | EM10 | EM9 | EM8 | EM7 | EM6  | EM5  | EM4      | EM3 | EM2 | EM1  | EM0  |      |
| rw   | rw   | rw   | rw       | rw   | rw   | rw  | rw  | rw  | rw   | rw   | rw       | rw  | rw  | rw   | rw   |      |

Bit 31 保留，必须保持为复位值。

Bit 30:29 **EMx**：线 x 上的事件屏蔽 (x=30~29)

0：屏蔽来自线 x 上的事件请求

1：开放来自线 x 上的事件请求

Bit 28:23 保留，必须保持为复位值。

Bits 22:21 **EMx**: 线 x 上的事件屏蔽 (x=22~21)

0: 屏蔽来自线 x 上的事件请求

1: 开放来自线 x 上的事件请求

Bit 20:19 保留，必须保持为复位值。

Bit 18:0 **EMx**: 线 x 上的事件屏蔽 (x=18~0)

0: 屏蔽来自线 x 上的事件请求

1: 开放来自线 x 上的事件请求

### 12.5.3 EXTI 上升沿触发选择寄存器 (EXTI\_RTSR)

地址偏移量: 0x08

复位值: 0x0000 0000

| 31   | 30   | 29   | 28       | 27   | 26   | 25  | 24  | 23  | 22   | 21   | 20       | 19  | 18  | 17   | 16   |      |
|------|------|------|----------|------|------|-----|-----|-----|------|------|----------|-----|-----|------|------|------|
| Res  | RT30 | RT29 | Reserved |      |      |     |     |     | RT22 | RT21 | Reserved |     |     | RT18 | RT17 | RT16 |
|      | rw   | rw   |          |      |      |     |     |     | rw   | rw   |          |     |     | rw   | rw   |      |
| 15   | 14   | 13   | 12       | 11   | 10   | 9   | 8   | 7   | 6    | 5    | 4        | 3   | 2   | 1    | 0    |      |
| RT15 | RT14 | RT13 | RT12     | RT11 | RT10 | RT9 | RT8 | RT7 | RT6  | RT5  | RT4      | RT3 | RT2 | RT1  | RT0  |      |
| rw   | rw   | rw   | rw       | rw   | rw   | rw  | rw  | rw  | rw   | rw   | rw       | rw  | rw  | rw   | rw   |      |

Bit 31 保留，必须保持为复位值。

Bit 30:29 **RTx**: 线 x 上的上升沿触发事件配置位 (x=30~29)

0: 禁止输入线 x 上的上升沿触发 (中断和事件)

1: 允许输入线 x 上的上升沿触发 (中断和事件)

Bit 28:23 保留，必须保持为复位值。

Bit 22:21 **RTx**: 线 x 的上升触发事件配置位 (x=22~21)

0: 禁止输入线 x 上的上升沿触发 (中断和事件)

1: 允许输入线 x 上的上升沿触发 (中断和事件)

Bit 20:19 保留，必须保持为复位值。

Bit 18:0 **RTx**: 线 x 的上升触发事件配置位 (x=18~0)

0: 禁止输入线 x 上的上升沿触发 (中断和事件)

1: 允许输入线 x 上的上升沿触发 (中断和事件)

注意: 外部唤醒线是边沿触发的, 这些线上不能出现毛刺信号。如果在 EXTI\_RTISR 寄存器的写操作期间发生可配置中断线上的上升沿, 则不设置挂起位。在同一中断线上, 可以同时设置上升沿和下降沿触发。

### 12.5.4 EXTI 下降沿触发选择寄存器 (EXTI\_FTSR)

地址偏移量: 0x0C

复位值: 0x0000 0000

| 31   | 30   | 29   | 28       | 27   | 26   | 25  | 24  | 23  | 22   | 21   | 20       | 19  | 18  | 17   | 16   |      |
|------|------|------|----------|------|------|-----|-----|-----|------|------|----------|-----|-----|------|------|------|
| Res  | FT30 | FT29 | Reserved |      |      |     |     |     | FT22 | FT21 | Reserved |     |     | FT18 | FT17 | FT16 |
|      | rw   | rw   |          |      |      |     |     |     | rw   | rw   |          |     |     | rw   | rw   |      |
| 15   | 14   | 13   | 12       | 11   | 10   | 9   | 8   | 7   | 6    | 5    | 4        | 3   | 2   | 1    | 0    |      |
| FT15 | FT14 | FT13 | FT12     | FT11 | FT10 | FT9 | FT8 | FT7 | FT6  | FT5  | FT4      | FT3 | FT2 | FT1  | FT0  |      |
| rw   | rw   | rw   | rw       | rw   | rw   | rw  | rw  | rw  | rw   | rw   | rw       | rw  | rw  | rw   | rw   |      |

Bit 31 保留，必须保持为复位值。

Bit 30:29 **FTx**: 线 x 上的下降沿触发事件配置位 (x=30~29)

0: 禁止输入线 x 上的下降沿触发 (中断和事件)

1: 允许输入线 x 上的下降沿触发 (中断和事件)

Bit 28:23 保留，必须保持为复位值。

Bit 22:21 **FTx**: 线 x 上的下降沿触发事件配置位 (x=22~21)

0: 禁止输入线 x 上的下降沿触发 (中断和事件)

1: 允许输入线 x 上的下降沿触发 (中断和事件)

Bit 20:19 保留，必须保持为复位值。

Bit 18:0 **FTx**: 线 x 上的下降沿触发事件配置位 (x=18~0)

0: 禁止输入线 x 上的下降沿触发 (中断和事件)

1: 允许输入线 x 上的下降沿触发 (中断和事件)

注意：外部唤醒线是边沿触发的，这些线上不能出现毛刺信号。如果在 EXTIFTSR 寄存器进行写操作期间发生可配置中断线上的下降沿，则不设置挂起位。在同一中断线上，可以同时设置上升沿和下降沿触发。

### 12.5.5 EXTI 软件中断事件寄存器 (EXTISWIER)

地址偏移量: 0x10

复位值: 0x0000 0000

| 31    | 30    | 29    | 28       | 27    | 26    | 25   | 24   | 23   | 22    | 21    | 20       | 19   | 18    | 17    | 16    |
|-------|-------|-------|----------|-------|-------|------|------|------|-------|-------|----------|------|-------|-------|-------|
| Res   | SWI30 | SWI29 | Reserved |       |       |      |      |      | SWI22 | SWI21 | Reserved |      | SWI18 | SWI17 | SWI16 |
|       | rw    | rw    |          |       |       |      |      |      | rw    | rw    |          |      | rw    | rw    |       |
| 15    | 14    | 13    | 12       | 11    | 10    | 9    | 8    | 7    | 6     | 5     | 4        | 3    | 2     | 1     | 0     |
| SWI15 | SWI14 | SWI13 | SWI12    | SWI11 | SWI10 | SWI9 | SWI8 | SWI7 | SWI6  | SWI5  | SWI4     | SWI3 | SWI2  | SWI1  | SWI0  |
| rw    | rw    | rw    | rw       | rw    | rw    | rw   | rw   | rw   | rw    | rw    | rw       | rw   | rw    | rw    | rw    |

Bit 31 保留，必须保持为复位值。

Bit 30:29 **SWIx**: 线 x 上的软件中断 (x=30~29)

如果在 EXTIMR 和 EXTIEMR 中允许产生中断，当它位于 '0' 时，写入 '1' 将设置 EXTI\_PR 中相应的挂起位，从而产生中断请求。

通过清除 EXTI\_PR 寄存器中相应的位 (通过在该位中写入 '1') 来清除该位。

Bit 28:23 保留，必须保持复位值。

Bit 22:21 **SWIx**: 线 x 上的软件中断 (x=22~21)

如果在 EXTIMR 和 EXTIEMR 中允许产生中断，当它位于 '0' 时，写入 '1' 将设置 EXTI\_PR 中相应的挂起位，从而产生中断请求。

通过清除 EXTI\_PR 寄存器中相应的位 (通过在该位中写入 '1') 来清除该位。

Bit 20:19 保留，必须保持为复位值。

Bit 18:0 **SWIx**: 线 x 上的软件中断 (x=18~0)

如果在 EXTIMR 和 EXTIEMR 中允许产生中断，当它位于 '0' 时，写入 '1' 将设置 EXTI\_PR 中相应的挂起位，从而产生中断请求。

通过清除 EXTI\_PR 寄存器中相应的位 (通过在该位中写入 '1') 来清除该位。

## 12.5.6 EXTI 挂起寄存器 (EXTI\_PR)

地址偏移量: 0x14

复位值: undefined

| 31    | 30    | 29    | 28       | 27    | 26    | 25   | 24   | 23   | 22    | 21    | 20       | 19   | 18    | 17    | 16    |
|-------|-------|-------|----------|-------|-------|------|------|------|-------|-------|----------|------|-------|-------|-------|
| Res   | PIF30 | PIF29 | Reserved |       |       |      |      |      | PIF22 | PIF21 | Reserved |      | PIF18 | PIF17 | PIF16 |
|       | rw    | rw    |          |       |       |      |      |      | rw    | rw    |          |      | rw    | rw    | rw    |
| 15    | 14    | 13    | 12       | 11    | 10    | 9    | 8    | 7    | 6     | 5     | 4        | 3    | 2     | 1     | 0     |
| PIF15 | PIF14 | PIF13 | PIF12    | PIF11 | PIF10 | PIF9 | PIF8 | PIF7 | PIF6  | PIF5  | PIF4     | PIF3 | PIF2  | PIF1  | PIF0  |
| rw    | rw    | rw    | rw       | rw    | rw    | rw   | rw   | rw   | rw    | rw    | rw       | rw   | rw    | rw    | rw    |

Bit 31 保留, 必须保持为复位值。

Bit 30:29 **PIF<sub>x</sub>**: 线 x 上挂起的中断标志 (x=30~29)

0: 没有发生触发请求

1: 发生了触发请求

当在外部中断线上发生了选择的边沿事件, 通过向该位写入 '1' 来清除该位。

Bit 28:23 保留, 必须保持复位值。

Bit 22:21 **PIF<sub>x</sub>**: 线 x 上挂起的中断标志 (x=22~21)

0: 没有发生触发请求

1: 发生了触发请求

当在外部中断线上发生了选择的边沿事件, 通过向该位写入 '1' 来清除该位。

Bit 20:19 保留, 必须保持为复位值。

Bit 18:0 **PIF<sub>x</sub>**: 线 x 上挂起的中断标志 (x=18~0)

0: 没有发生触发请求

1: 发生了触发请求

当在外部中断线上发生了选择的边沿事件, 通过向该位写入 '1' 来清除该位。



## 13 可配置逻辑单元 (CLC)

### 13.1 简介

可配置的逻辑单元 (CLC) 模块允许用户将信号组合指定为逻辑函数的输入，并使用逻辑输出来控制其他外围设备或 I/O 引脚。这为嵌入式设计提供了更大的灵活性和潜力，因为 CLC 模块可以在软件执行的限制之外运行，并支持大量的输出设计。

CLC 由四个主要部分组成，如图 11.1 所示。首先，输入数据选择将路由输入信号发送到四个数据门，如图 11.2 所示。然后，每四个数据门都可以选择 32 个输入信号中的任何一个，以传递到图 11.3 中所示的逻辑函数。然后将逻辑函数的输出提供给内部逻辑和外部 pin，并可以生成中断。CLC 模块的输出可以被路由到另一个 CLC 模块的输入，产生更复杂的逻辑函数。

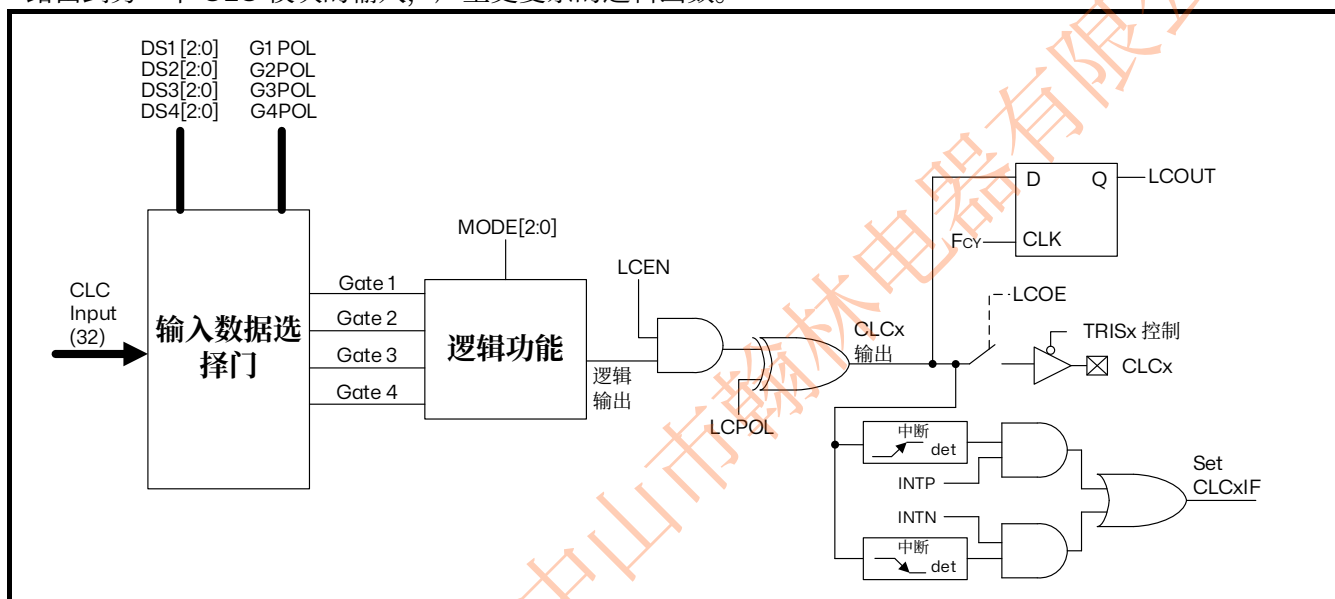


图 13.1 可配置逻辑单元

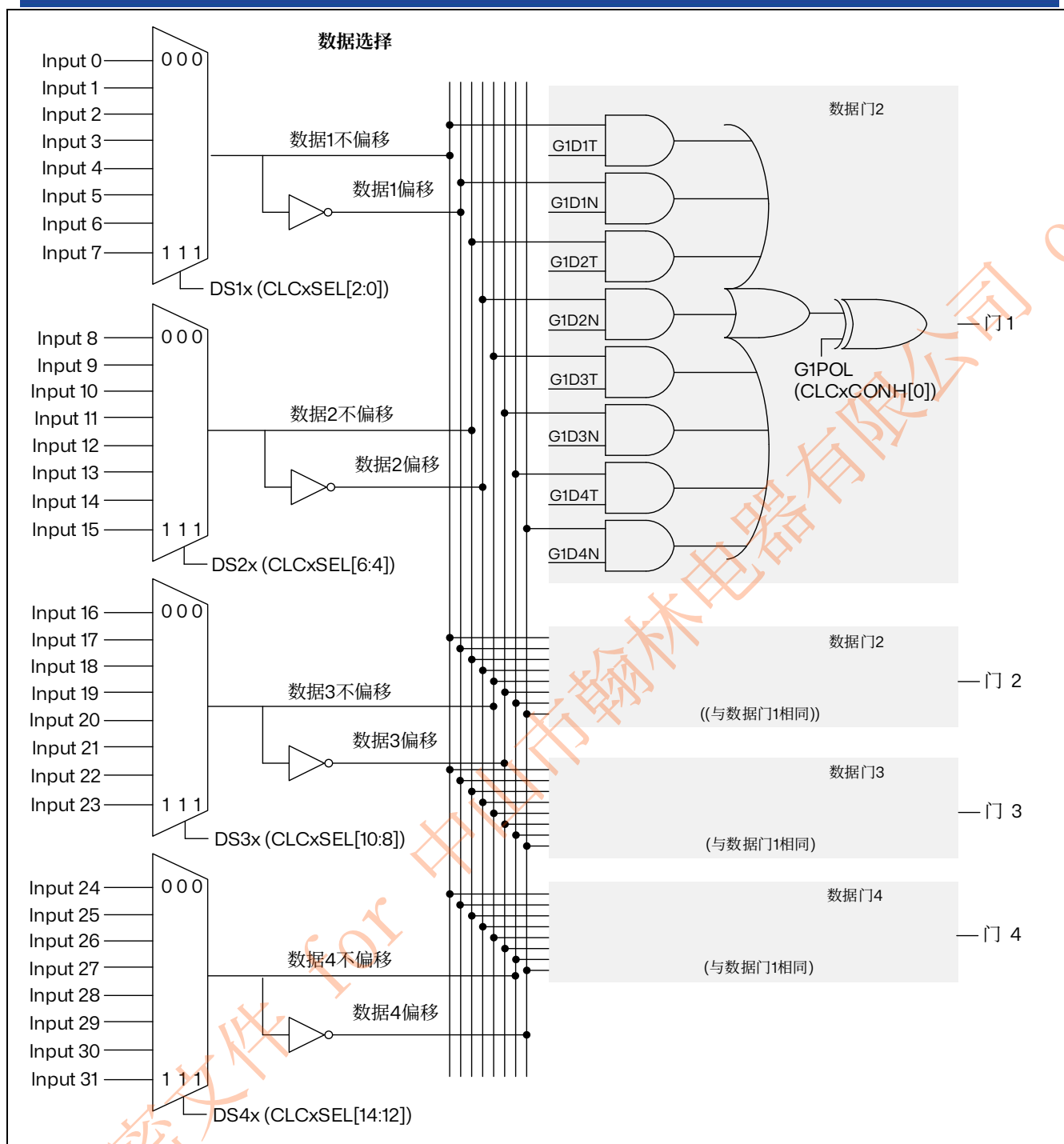


图 13.2 CLC 输入源选择

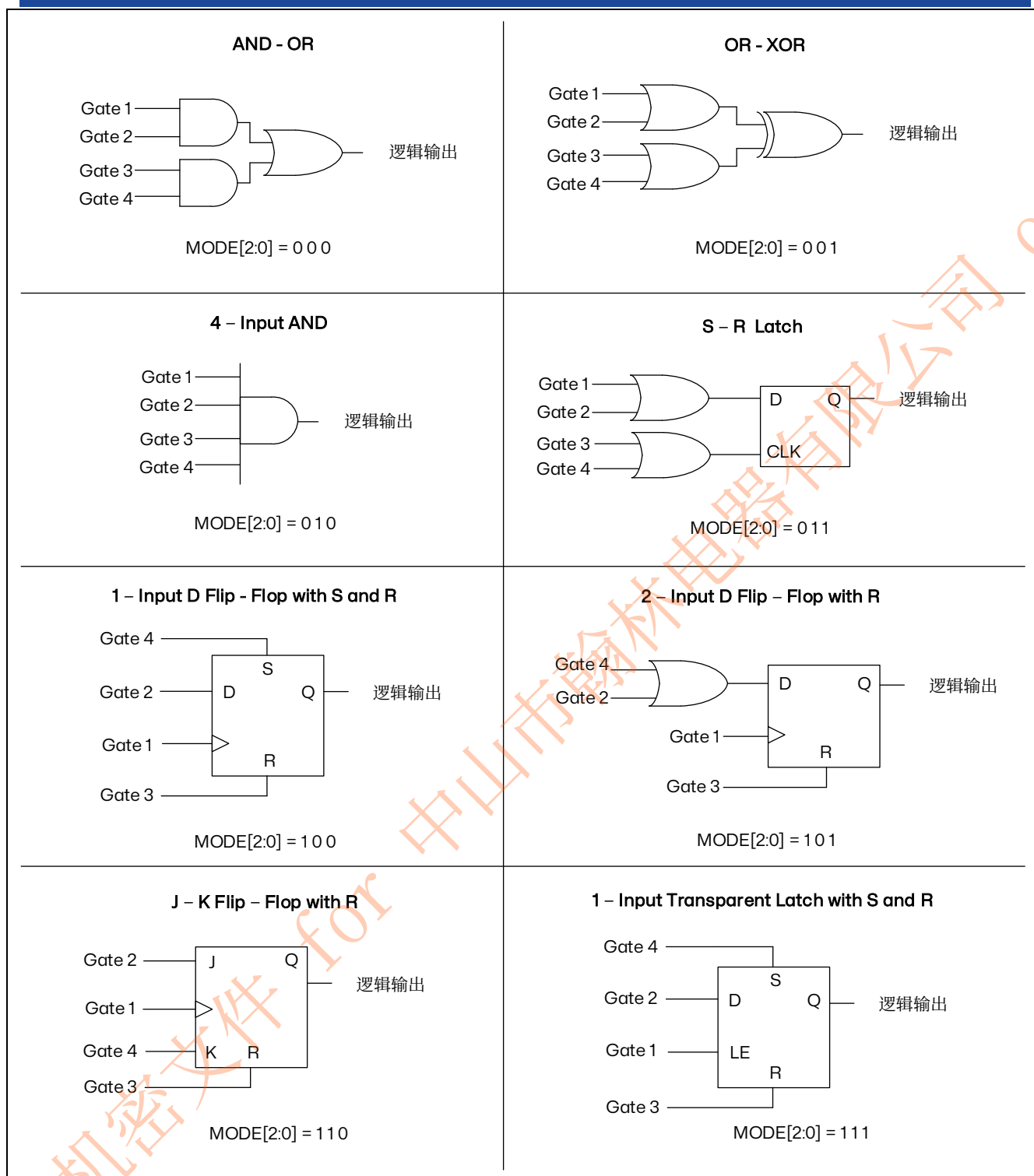


图 13.3 逻辑功能组合选项

## 13.2 CLC 设置

CLCxCONL 选择逻辑函数，并决定和控制 I / O 引脚。CLCxCONH 控制输出信号极性。要使 CLC 工作，LCEN (CLCxCONL<15>) 必须设置为 1。在 LCEN 清零时，可以设定所有寄存器

CLCxSEL (寄存器 2-3) 寄存器控制，输入信号被路由到图 11.2 的输入总线。在数据总线中可以使用 True (T) 和否定 (N) 值。

CLCxGLSL(寄存器 2-4)和 CLCxGLSH (寄存器 2-5) 注册从数据总线中选择哪一个信号应用于输入或门。真值的和去返的输入是单独使能的；不要对相同的信号同时使能两者。

CLC 模块输出的最终极性由 LCPOL (CLCxCONL<5>) 控制。当 LCPOL = 1, 输出是反相的, 当 LCPOL = 0 时, 输出是不反相的。GxPOL 位 (CLCxCONH<3:0>) 控制逻辑函数输入的极性。

INTP 和 INTN 引脚(CLCxCONL<11:10>)使 CLC 输出的上升和下降沿更新中断。

LCOUT 位是只读的, 反映了逻辑单元输出的状态。将 CLCxOUT 信号输出到 I / O 引脚, 设置 LCOE 位, 并将 I / O 配置为数字输出。在一些设备上, CLCxOUT 信号是通过外围 Pin 选择 (PPS) 提供的, 需要进行相应的配置。

### 13.3 输入源

CLC 中的每个逻辑单元可接受 4 个输入, 每个输入均来自 4 个数据门之一。每个数据门都连接到 8 个输入源。数据门允许在每个输入源的反相或不反相极性之间进行选择。可供 CLC 使用的输入源因器件而异。

#### 13.3.1 多路复用器

该模块具有 4 个输入源多路开关。多路开关输入通过设置 CLCxSEL 寄存器中的控制位进行选择, 以定义要通过 4 个数据选择多路开关中的每个多路开关选择的数据源。4 个数据选择多路开关的每个多路开关都送至 4 个逻辑功能输入门之一, 如图 11.3 所示。该模块具有基于每个输入源多路开关的输出构造的内部数据总线 (见图 11.2)。数据总线同时具有每个选定输入源的真值 (T) 和取反 (N) 版本。因此, 内部数据总线上最多具有 8 个信号可连接到逻辑功能的输入门。

#### 13.3.2 逻辑输入门

4 个逻辑输入门用于将来自数据选择多路开关的输入源送至 4 个逻辑功能输入。每个输入源信号的真值和取反形式都可供每个逻辑门使用。输入信号源使用 CLCxGLS 寄存器来进行使能, 供每个逻辑功能输入使用。最多可以使能 8 个信号, 供每个逻辑功能输入使用。对于 4 个逻辑功能输入中的每个输入, 可以使能 8 个信号源中任意数量的信号源。每个逻辑门均提供输入信号的逻辑或 (OR)。选定 (真值或取反) 信号通过逻辑或来构成门输出数据。逻辑 NAND 通过使用 GxPOL 位更改输出极性而获得。如果需要逻辑与 (AND), 可以根据 DeMorgan 法则, 选择取反输入并反转输出极性。如果所有输入均取反, 并施加到 NOR 逻辑, 结果将与逻辑与完全相同。写为代数

形式:

$$C = A \text{ AND } B$$

等同于:

$$C = \text{NOT}(\text{NOT}(A) \text{ OR } \text{NOT}(B)).$$

表 11.1 总结了可以通过使用门控制位获得的基本功能。该表显示的是使用全部 4 个输入多路开关源的情形, 但输入门可以配置为使用较少的源。如果未选择任何输入 (CLCxGLS = 0x00), 则输出将为 0 或 1, 具体取决于 GxPOL 位。

表 13.1 示例逻辑函数

| CLCxGLS | GxPOL 位 | 功能                    |
|---------|---------|-----------------------|
| 0xAAAA  | 0       | OR (D1, D2, D3, D4)   |
| 0xAAAA  | 1       | NOR (D1, D2, D3, D4)  |
| 0x5555  | 0       | NAND (D1, D2, D3, D4) |
| 0x5555  | 1       | AND (D1, D2, D3, D4)  |

|        |   |           |
|--------|---|-----------|
| 0x0000 | 0 | Logic '0' |
| 0x0000 | 1 | Logic '1' |

### 13.3.3 逻辑功能

有 8 种可用的逻辑功能，包括：

- AND-OR
- OR-XOR
- AND
- S-R 锁存器
- 带置 1 和复位功能的 D 触发器
- 带复位功能的 D 触发器
- 带复位功能的 J-K 触发器
- 带置 1 和复位功能的透明锁存器

这些逻辑功能如图 11.3 所示。每种逻辑功能具有 4 个输入和 1 个输出。MODE<2:0> 位

(CLCxCONL<2:0>) 用于设置逻辑单元的功能行为。共有 4 个组合选项和 4 个状态选项。3 个状态选项将输入门 1 定义为上升沿时钟，具有 D 和 J-K 触发器的传统含义。第 4 个状态选项 (MODE<2:0>位 = 111) 为透明锁存器；当锁存器使能 (LE) 为真时，Q 跟随 D 变化；当 LE 为假时，Q 保持状态。对于同时具有 S (置 1) 和 R (复位) 输入的选项，当 S 或 R 为逻辑 1 时，输出将随时钟异步变化；R 将优先。

### 13.3.4 软件输入

通过将 CLCxGLSL / H 位与逻辑门与“0”相关联的 CLCxGLSL / H 位，并写入适当的 GxPOL 位 (见表 11.1)，可以直接控制逻辑函数。门输出将等于 GxPOL 比特的值。

### 13.4 输出

LCOUT (CLCxCONL<6>) 是逻辑单元输出，并被路由到 I / O 左左 pin 或设备内的其他模块。在所有情况下，在 LCPOL 逆变器后接收信号值。为了观察 I / O pin 的输出，用户需要设置 LCOE (CLCxCONL<7>)。

### 13.5 应用逻辑

CLC 同时提供了组合和状态 (见图 11.3) 逻辑功能选项。输入门的输出将施加到逻辑功能。如果 CLCxGLS = 0x00，当 GxPOL 位 (CLCxCONH<3:0>) 清零时，功能将接收到逻辑 0，当 GxPOL 位置 1 时，将接收到逻辑 1。

#### 13.5.1 组合逻辑

组合功能 (MODE<2:0> = 010、001 和 000) 基于输入门的 AND/OR 逻辑进行构造。4 输入 AND 逻辑可以提供 OR 逻辑功能，方法是根据 DeMorgan 法则对输入和输出进行反相。对 XOR 逻辑的输出进行反相等效于对其中一个输入 (但不是同时对两个输入) 进行反相。

当 LCEN (CLCxCONL<15>) 清零时，SR 功能 (MODE<2:0> = 011) 不会受影响，对于状态逻辑寄存器也是如此。锁存器为复位优先型，这意味着复位信号优先于可能存在的任何置 1 信号。

#### 13.5.2 状态逻辑

状态功能同时包含了带异步置 1 (S) 和复位 (R) 功能的 D 触发器和 J-K 触发器。输入门 1 提供上升沿时钟。如果需要下降沿时钟，可以通过门逻辑 (G1POL) 对门 1 进行反相。输入门 2 (有时还有门 4)

向寄存器或锁存器输入提供数据。在透明锁存器模式 (MODE<2:0> = 111) 下工作时, 如果 LE 为高电平, 则输出 Q 跟随 D 变化, 如果 LE 为低电平, 则输出 Q 保持状态。

各种模式可能共用也可能不共用状态存储器, 切换模式可能会也可能不会改变状态变量的状态。对于所有模式, 寄存器均为复位优先。

### 13.6 CLC 中断

CLC 模块具有两种类型的可允许中断: 上升沿中断事件和下降沿中断事件。这两种中断事件分别通过 INTP (CLCxCONL<11>) 和 INTN (CLCxCONL<10>) 控制位使能。

出现其中任何有效中断时都会将 CLC 中断标志 CLCIF 置 1。使能该模块 (LCEN = 1) 时, 如果在 INTP = 1 时出现上升沿输出或在 INTN = 1 时出现下降沿事件, 将发生这种情况。

如果 CLC 逻辑的初始输出状态为 1, 并且 INTP = 1, 则在 LCEN 设置为 1 时会产生中断。类似地, 如果 CLC 的初始输出状态为 0, 并且 INTN = 1, 则会产生中断。这些条件必须在软件中进行检测并清除。类似地, 如果在使能 CLC 模块时 INTP 或 INTN 置 1, 则可能会产生假中断。

用户应确保清除在 CLC 模块初始化过程中可能出现的任何假中断事件。

如果 CLC 中断允许位 CLCIE 清零, 则不会产生中断。但是, 如果出现中断条件, CLCIF 位仍会置 1。用户可通过在中断服务程序 (Interrupt Service Routine, ISR) 中清零 CLCIF 清除该中断。

### 13.7 睡眠模式操作

CLC 模块不会受睡眠模式影响, 因为它不依赖于系统时钟源工作。但是, 一些输入源在睡眠期间可能会被禁止, 所以功能可能被中断。如果时钟源继续工作, 模块也将继续工作。

### 13.8 空闲模式下的操作

CLC 模块不会受空闲模式影响, 因为它不依赖于系统时钟源工作。但是, 一些输入源在空闲期间可能会被禁止, 所以功能可能被中断。如果时钟源继续工作, 模块也将继续工作。

### 13.9 复位

向 LCEN 位写入 0 时, 所有状态逻辑功能的输出将复位为 0。系统复位会将 CLCxCONL、CLCxCONH、CLCxSEL、CLCxGLSL 和 CLCxGLSH 寄存器恢复为默认状态并禁止该模块。执行器件复位会将模块寄存器中的所有位恢复为默认状态。在复位之后, 所有逻辑功能的输出为 0; 这同时包括锁存器和触发器功能。当器件复位置为有效 (LCEN (CLCxCONL<15>) = 0) 时, 状态逻辑将复位, 逻辑功能的输出将强制为低电平。

### 13.10 CLC 寄存器描述

基地址: 0x4000 7400, 0x4000 7440, 0x4000 7480, 0x4000 74C0

#### 13.10.1 可配置的逻辑单元 x 控制寄存器 (CLC\_CON)

地址偏移量: 0x00

复位值: 0x0000 0000

|          |          |    |    |      |      |        |        |      |       |       |          |       |       |           |       |
|----------|----------|----|----|------|------|--------|--------|------|-------|-------|----------|-------|-------|-----------|-------|
| 31       | 30       | 29 | 28 | 27   | 26   | 25     | 24     | 23   | 22    | 21    | 20       | 19    | 18    | 17        | 16    |
| Reserved |          |    |    |      |      |        |        |      |       |       |          | G4POL | G3POL | G2POL     | G1POL |
|          |          |    |    |      |      |        |        |      |       |       |          | rw    | rw    | rw        | rw    |
| 15       | 14       | 13 | 12 | 11   | 10   | 9      | 8      | 7    | 6     | 5     | 4        | 3     | 2     | 1         | 0     |
| LCEN     | Reserved |    |    | INTP | INTN | INTPIF | INTNIF | LCOE | LCOUT | LCPOL | Reserved |       |       | MODE[2:0] |       |
|          |          |    |    | rw   | rw   | rw     | rw     | rw   | rw    | rw    |          |       |       | rw        | rw    |



Bits 31:20 保留，必须保持为复位值。

Bit 19 **G4POL**: 门 4 极性控制位

- 1: 在应用于逻辑单元时，通道 4 逻辑输出是倒置的
- 0: 通道 4 逻辑输出不倒转

Bit 18 **G3POL**: 门 3 极性控制位

- 1: 在应用于逻辑单元时，通道 3 逻辑输出是倒置的
- 0: 通道 3 逻辑输出不倒转

Bit 17 **G2POL**: 门 2 极性控制位

- 1: 在应用于逻辑单元时，通道 2 逻辑输出是倒置的
- 0: 通道 2 逻辑输出不倒转

Bit 16 **G1POL**: 门 1 极性控制位

- 1: 在应用于逻辑单元时，通道 1 逻辑输出是倒置的
- 0: 通道 1 逻辑输出不倒转

Bit 15 **LCEN**: CLCx 使能位

- 0: CLCx 是禁用，具有逻辑零输出
- 1: CLCx 使能并混合输入信号

Bits 14:12 保留，必须保持为复位值。

Bit 11 **INTP**: CLCx 正边缘中断使能位

- 0: 中断不会生成
- 1: 当在 LCOUT 出现一个上升的边缘时，就会产生中断

Bit 10 **INTN**: CLCx 负边缘中断使能位

- 0: 中断不会生成
- 1: 当在 LCOUT 出现下降边缘时，会产生中断

Bit 9 **INTPIF**: CLCx 正边缘中断标志位

- 0: 正边缘事件没有发生，写 0 到清除
- 1: 正边缘事件发生

Bit 8 **INTNIF**: CLCx 负边缘中断标志位

- 0: 负边缘事件没有发生，写 0 到清除
- 1: 负边缘事件发生

Bit 7 **LCOE**: CLCx 端口使能位

- 0: CLCx 端口 pin 输出禁用
- 1: 使能 CLCx 端口 pin 输出

Bit 6 **LCOUT**: CLCx 数据输出状态位

- 0: CLCx 输出低
- 1: CLCx 输出高

Bit 5 **LCPOL**: CLCx 输出极性控制位

- 0: 模块的输出不倒置
- 1: 模块的输出是倒置的

Bits 4:3 保留，必须保持为复位值。

Bits 2:0 **MODE[2:0]**: CLCx 模位

- 000: AND-OR 逻辑
- 001: OR-XOR 逻辑
- 010: 4 输入 AND 逻辑



- 101: SR 锁存器
- 100: 带置 1 和复位功能的单输入 D 触发器
- 101: 带复位功能的 2 输入 D 触发器
- 110: 带复位功能的 JK 触发器
- 111: 带置 1 和复位功能的单输入透明锁存器

### 13.10.2 可配置的逻辑单元 x 输入 MUX 选择寄存器 (CLC\_SEL)

地址偏移量: 0x04

复位值: 0x0000 0000

| 31       | 30       | 29 | 28 | 27  | 26       | 25 | 24 | 23  | 22       | 21 | 20 | 19  | 18       | 17 | 16 |
|----------|----------|----|----|-----|----------|----|----|-----|----------|----|----|-----|----------|----|----|
| Reserved |          |    |    |     |          |    |    |     |          |    |    |     |          |    |    |
| 15       | 14       | 13 | 12 | 11  | 10       | 9  | 8  | 7   | 6        | 5  | 4  | 3   | 2        | 1  | 0  |
| Res      | DS4[2:0] |    |    | Res | DS3[2:0] |    |    | Res | DS2[2:0] |    |    | Res | DS1[2:0] |    |    |
|          | rw       | rw | rw |     | rw       | rw | rw |     | rw       | rw | rw |     | rw       | rw | rw |

Bit 31:15 保留, 必须保持为复位值。

Bit 14:12 **DS4[2:0]**: 数据选择 MUX 4 信号选择位

Bit 11 保留, 必须保持为复位值。

Bit 10:8 **DS3[2:0]**: 数据选择 MUX 3 信号选择位

Bit 7 保留, 必须保持为复位值。

Bit 6:4 **DS2[2:0]**: 数据选择 MUX 2 信号选择位

Bit 3 保留, 必须保持为复位值。

Bits 2:0 **DS1[2:0]**: 数据选择 MUX 1 信号选择位

### 13.10.3 可配置的逻辑单元 x 源使能寄存器 (CLC\_GLS)

地址偏移量: 0x08

复位值: 0x0000 0000

| 31    | 30    | 29    | 28    | 27    | 26    | 25    | 24    | 23    | 22    | 21    | 20    | 19    | 18    | 17    | 16    |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| G4D4T | G4D4N | G4D3T | G4D3N | G4D2T | G4D2N | G4D1T | G4D1N | G3D4T | G3D4N | G3D3T | G3D3N | G3D2T | G3D2N | G3D1T | G3D1N |
| rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    |
| 15    | 14    | 13    | 12    | 11    | 10    | 9     | 8     | 7     | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| G2D4T | G2D4N | G2D3T | G2D3N | G2D2T | G2D2N | G2D1T | G2D1N | G1D4T | G1D4N | G1D3T | G1D3N | G1D2T | G1D2N | G1D1T | G1D1N |
| rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    |

Bits 31 **G4D4T**: 与 G1 相同

Bits 30 **G4D4N**: 与 G1 相同

Bits 29 **G4D3T**: 与 G1 相同

Bits 28 **G4D3N**: 与 G1 相同

Bits 27 **G4D2T**: 与 G1 相同

Bits 26 **G4D2N**: 与 G1 相同

Bits 25 **G4D1T**: 与 G1 相同

Bits 24 **G4D1N**: 与 G1 相同

Bits 23 **G3D4T**: 与 G1 相同

Bits 22 **G3D4N**: 与 G1 相同

Bits 21 **G3D3T**: 与 G1 相同

Bits 20 **G3D3N**: 与 G1 相同

Bits 19 **G3D2T**: 与 G1 相同

- Bits 18 **G3D2N**: 与 G1 相同
- Bits 17 **G3D1T**: 与 G1 相同
- Bits 16 **G3D1N**: 与 G1 相同
- Bits 15 **G2D4T**: 与 G1 相同
- Bits 14 **G2D4N**: 与 G1 相同
- Bits 13 **G2D3T**: 与 G1 相同
- Bits 12 **G2D3N**: 与 G1 相同
- Bits 11 **G2D2T**: 与 G1 相同
- Bits 10 **G2D2N**: 与 G1 相同
- Bits 9 **G2D1T**: 与 G1 相同
- Bits 8 **G2D1N**: 与 G1 相同
- Bits 7 **G1D4T**: 门 4 数据源 4 真使能位
- 0: 门 4 数据源 4 信号被禁用
  - 1: 门 4 数据源 4 信号被使能
- Bits 6 **G1D4N**: 门 4 数据源 4 否定使能位
- 0: 门 4 的数据源 4 反向信号被禁用
  - 1: 数据源 4 反向信号被使能用于门 4
- Bits 5 **G1D3T**: 门 3 数据源 3 真使能位
- 0: 门 3 数据源 3 信号被禁用
  - 1: 门 3 数据源 3 信号被使能
- Bits 4 **G1D3N**: 门 3 数据源 3 否定使能位
- 0: 门 3 的数据源 3 反向信号被禁用
  - 1: 数据源 3 反向信号被使能用于门 3
- Bits 3 **G1D2T**: 门 2 数据源 2 真使能位
- 0: 门 2 数据源 2 信号被禁用
  - 1: 门 2 数据源 2 信号被使能
- Bits 2 **G1D2N**: 门 2 数据源 2 否定使能位
- 0: 门 2 的数据源 2 反向信号被禁用
  - 1: 数据源 2 反转信号被使能用于门 2
- Bits 1 **G1D1T**: 门 1 数据源 1 真使能位
- 0: 门 1 数据源 1 信号被禁用
  - 1: 门 1 数据源 1 信号被使能
- Bits 0 **G1D1N**: 门 1 数据源 1 否定使能位
- 0: 门 1 的数据源 1 反向信号被禁用
  - 1: 数据源 1 反向信号被使能用于门 1

## 14 CORDIC 协同处理器 (CORDIC)

### 14.1 简介

CORDIC 协处理器为通常用于电机控制、计量、信号处理和许多其他应用的数学函数（主要是三角函数）提供硬件加速。

与软件实现相比，它加快了这些函数的计算速度，从而可以使用较低的工作频率，或者释放处理器周期以执行其他任务。

### 14.2 CORDIC 主要特征

- 24 位 CORDIC 旋转引擎
- 圆形和双曲模式
- 旋转和矢量模式
- 函数：正弦，余弦，sinh，cosh，atan，atan2，atanh，模数，平方根，自然对数
- 可编程精度
- 低延迟 AHB 从接口
- 结果可以立即读取，无需轮询或中断
- DMA 读写通道
- DMA 多寄存器读/写

### 14.3 CORDIC 功能描述

#### 14.3.1 一般的描述

CORDIC 是求解三角函数和双曲函数的一种经济高效的连续逼近算法。

在三角(圆)模式中，角  $\theta$  的正弦和余弦是通过旋转单位向量[1,0]通过减少角度来确定的，直到旋转角度的累积和等于输入角  $\theta$ 。旋转后向量的 x 和 y 直角分量分别对应于  $\theta$  的余弦和正弦。反之，与  $\arctan(y/x)$  对应的向量[x,y]的角度，是通过旋转[x,y]，依次减小角度，得到单位向量[1,0]。旋转角度的累积和给出了原始矢量的角度。

CORDIC 算法也可用于计算双曲函数 (sinh, cosh, atanh)，方法是将连续的圆周旋转替换为沿着双曲的步骤。

其他函数可以从上面描述的基本函数派生出来。

#### 14.3.2 CORDIC 功能

使用协处理器的第一步是通过相应地编程 CORDIC\_CR 寄存器的 FUNC 位域来选择所需的功能。

表 14.1 列出了 CORDIC 协同处理器支持的函数

| 功能         | Primary argument(ARG1) | Secondary argument (ARG2) | Primary result(RES1)  | Secondary result(RES2) |
|------------|------------------------|---------------------------|-----------------------|------------------------|
| Cosine     | angle $\theta$         | modulus m                 | $m \cdot \cos \theta$ | $m \cdot \sin \theta$  |
| Sine       | angle $\theta$         | modulus m                 | $m \cdot \sin \theta$ | $m \cdot \cos \theta$  |
| Phase      | x                      | y                         | $\text{atan2}(y,x)$   | $\sqrt{x^2 + y^2}$     |
| Modulus    | x                      | y                         | $\sqrt{x^2 + y^2}$    | $\text{atan2}(y,x)$    |
| Arctangent | x                      | none                      | $\tan^{-1} x$         | none                   |

| 功能                    | Primary argument(ARG1) | Secondary argument (ARG2) | Primary result(RES1) | Secondary result(RES2) |
|-----------------------|------------------------|---------------------------|----------------------|------------------------|
| Hyperbolic cosine     | x                      | none                      | cosh x               | sinh x                 |
| Hyperbolic sine       | x                      | none                      | sinh x               | cosh x                 |
| Hyperbolic arctangent | x                      | none                      | tanh-1 x             | none                   |
| Natural logarithm     | x                      | none                      | ln x                 | none                   |
| Square root           | x                      | none                      | $\sqrt{x}$           | none                   |

一些函数接受两个输入参数(ARG1 和 ARG2)，有些函数同时生成两个结果(RES1 和 RES2)。这是算法的副作用，意味着只需要一个操作就可以获得两个值。例如，当进行极坐标到直角坐标的转换时， $\sin\theta$  也会产生  $\cos\theta$ ， $\cos\theta$  也会产生  $\sin\theta$ 。对于矩形到极的转换(相位(x,y)，模(x,y))和双曲函数(cosh $\theta$ ，sinh $\theta$ )也是如此。

注：指数函数  $\exp x$  可以表示为  $\sinh x$  与  $\cosh x$  的和。

更进一步，以 N 为底的对数， $\log_n x$ ，可以通过  $\ln x$  乘以常数 K 得到，其中  $K = 1/\ln N$ 。

对于某些函数(atan, log, sqrt)，可以使用缩放系数(参见 10.3.4 节)将函数的范围扩展到 q1.31 定点格式所支持的最大值[-1,1]之外。对于所有其他圆形函数，比例系数必须设置为 0，对于双曲函数，比例系数必须设置为 1。

## Cosine 余弦

表 14.2 余弦函数参数

| 参数    | 描述                                          | 范围      |
|-------|---------------------------------------------|---------|
| ARG1  | Angle $\theta$ in radians, divided by $\pi$ | [-1, 1] |
| ARG2  | Modulus m                                   | [0, 1]  |
| RES1  | $m \cdot \cos \theta$                       | [-1, 1] |
| RES2  | $m \cdot \sin \theta$                       | [-1, 1] |
| SCALE | Not applicable                              | 0       |

这个函数计算一个角在  $-\pi$  到  $\pi$  范围内的余弦值。它还可以用于执行极坐标到矩形的转换。

主要的参数是以弧度为单位的角  $\theta$ 。在对 ARG1 进行编程之前，它必须除以  $\pi$ 。

第二个参数是模数 m。如果 m 大于 1，则必须在软件中应用缩放以使其适应 ARG2 的 q1.31 范围。

主要结果 RES1 是角度的余弦值乘以模量。

第二个结果 RES2 是角度的正弦值乘以模量。

## Sine 正弦

表 14.3 正弦参数

| 参数    | 描述                                          | 范围      |
|-------|---------------------------------------------|---------|
| ARG1  | Angle $\theta$ in radians, divided by $\pi$ | [-1, 1] |
| ARG2  | Modulus m                                   | [0, 1]  |
| RES1  | $m \cdot \sin \theta$                       | [-1, 1] |
| RES2  | $m \cdot \cos \theta$                       | [-1, 1] |
| SCALE | Not applicable                              | 0       |

这个函数计算一个角在 $-\pi$  到  $\pi$  范围内的正弦值。它还可以用于执行极坐标到矩形的转换。

主要的参数是以弧度为单位的角 $\theta$ 。在对 ARG1 进行编程之前，它必须除以  $\pi$ 。

第二个参数是模数  $m$ 。如果  $m$  大于 1，则必须在软件中应用缩放以使其适应 ARG2 的 q1.31 范围。

主要结果 RES1 是角度的正弦值乘以模量。

第二个结果 RES2 是角的余弦值乘以模量。

## Phase 相位

表 14.4 相位参数

| 参数    | 描述                                                | 范围        |
|-------|---------------------------------------------------|-----------|
| ARG1  | x coordinate                                      | $[-1, 1]$ |
| ARG2  | y coordinate                                      | $[-1, 1]$ |
| RES1  | Phase angle $\theta$ in radians, divided by $\pi$ | $[-1, 1]$ |
| RES2  | Modulus $m$                                       | $[0, 1]$  |
| SCALE | Not applicable                                    | 0         |

这个函数计算向量  $v = [x \ y]$ （也称为  $\text{atan2}(y, x)$ ）在 $-\pi$  到  $\pi$  范围内的相位角。它还可以用于执行矩形到极坐标的转换。

主要参数是  $x$  坐标，也就是向量在  $x$  轴方向上的大小。如果  $|x| > 1$ ，必须在软件中应用缩放以使其适应 ARG1 的 q1.31 范围。

第二个参数是  $y$  坐标，也就是向量在  $y$  轴方向上的大小。如果  $|y| > 1$ ，必须在软件中应用缩放以使其适应 ARG2 的 q1.31 范围。

主要结果 RES1 是矢量  $v$  的相位角  $\theta$ ，RES1 必须乘以  $\pi$  才能得到以弧度为单位的角度。请注意，由于相位角的圆形性质，接近  $\pi$  的值有时可能包裹到 $-\pi$ 。

二次结果 RES2 是模量，由下式给出： $|v| = \sqrt{x^2 + y^2}$  如果  $|v| > 1$ ，RES2 的结果饱和到 1。

## Modulus 系数

表 14.5 模量参数

| 参数    | 描述                   | 范围        |
|-------|----------------------|-----------|
| ARG1  | x coordinate         | $[-1, 1]$ |
| ARG2  | y coordinate         | $[-1, 1]$ |
| RES1  | Modulus $m$          | $[0, 1]$  |
| RES2  | Phase angle $\theta$ | $[-1, 1]$ |
| SCALE | Not applicable       | 0         |

这个函数计算向量  $v = [x \ y]$  的大小或模量。它还可以用于执行矩形到极坐标的转换。

主要参数是  $x$  坐标，也就是向量在  $x$  轴方向上的大小。如果  $|x| > 1$ ，必须在软件中应用缩放以使其适应 ARG1 的 q1.31 范围。

第二个参数是  $y$  坐标，也就是向量在  $y$  轴方向上的大小。如果  $|y| > 1$ ，必须在软件中应用缩放以使其适应 ARG2 的 q1.31 范围。

主要结果 RES1 是模量，由下式给出： $|v| = \sqrt{x^2 + y^2}$ 。如果  $|v| > 1$ ，RES1 的结果饱和到 1。

次要结果 RES2 是矢量  $v$  的相位角  $\theta$ ，RES2 必须乘以  $\pi$  才能得到以弧度为单位的角度。请注意，由于相位角的圆形性质，接近  $\pi$  的值有时可能包裹到 $-\pi$ 。

## Arctangent 反正切

表 14.6 反正切参数

| 参数    | 描述                                                    | 范围      |
|-------|-------------------------------------------------------|---------|
| ARG1  | $x \cdot 2^{-n}$                                      | [-1, 1] |
| ARG2  | Not applicable                                        | -       |
| RES1  | $2^{-n} \cdot \tan^{-1} x$ , in radians, divided by p | [-1, 1] |
| RES2  | Not applicable                                        | -       |
| SCALE | n                                                     | [0 7]   |

这个函数计算输入参数  $x$  的  $\arctan$  或  $\text{inverse tan}$ 。

主要参数 ARG1 是输入值  $x = \tan\theta$ 。如果  $|x| > 1$ ，则必须在软件中应用  $2^{-n}$  的比例系数使  $-1 < x \cdot 2^{-n} < 1$ 。缩放值  $x \cdot 2^{-n}$  在 ARG1 中编程，比例系数  $n$  必须在 scale 参数中编程。

注意，允许的最大输入值是  $\tan\theta = 128$ ，这对应于一个角  $\theta = 89.55$  度。对于  $|x| > 128$ ，必须使用软件方法来查找  $\tan^{-1} x$ 。

次要参数 ARG2 未被使用。

主要结果 RES1 是角  $\theta = \tan^{-1} x$ 。RES1 必须乘以  $2^n \cdot \pi$  才能得到以弧度为单位的角。

次要结果 RES2 是未使用的。

## Hyperbolic cosine 双曲余弦

表 14.7 双曲余弦参数

| 参数    | 描述                     | 范围              |
|-------|------------------------|-----------------|
| ARG1  | $x \cdot 2^{-n}$       | [-0.559, 0.559] |
| ARG2  | Not applicable         | -               |
| RES1  | $2^{-n} \cdot \cosh x$ | [0.5, 0.846]    |
| RES2  | $2^{-n} \cdot \sinh x$ | [-0.683 0.683]  |
| SCALE | n                      | 1               |

该函数用于计算双曲角  $x$  的双曲余弦值，也可用于计算指数函数  $e^x = \cosh x + \sinh x$  和  $e^{-x} = \cosh x - \sinh x$ 。

主要参数是双曲角  $x$ 。只支持 -1.118 到 +1.118 范围内的  $x$  值。由于  $\cosh x$  的最小值为 1，超出了 q1.31 格式的范围，因此必须在软件中使用  $2^{-n}$  的缩放系数。系数  $n = 1$  必须在 SCALE 参数中编程。

没有使用次要参数。

主要结果 RES1 是双曲余弦， $\cosh x$ 。RES1 必须乘以 2 才能得到正确的结果。

次要结果 RES2 是双曲正弦  $\sinh x$ ，RES2 必须乘以 2 才能得到正确的结果。

## Hyperbolic sine 双曲正弦

表 14.8 双曲正弦参数

| 参数   | 描述               | 范围              |
|------|------------------|-----------------|
| ARG1 | $x \cdot 2^{-n}$ | [-0.559, 0.559] |
| ARG2 | Not applicable   | -               |



| 参数    | 描述                     | 范围                |
|-------|------------------------|-------------------|
| RES1  | $2^{-n} \cdot \sinh x$ | $[-0.683, 0.683]$ |
| RES2  | $2^{-n} \cdot \cosh x$ | $[0.5, 0.846]$    |
| SCALE | $n$                    | 1                 |

该函数用于计算双曲角  $x$  的双曲正弦值，也可用于计算指数函数  $e^x = \cosh x + \sinh x$  和  $e^{-x} = \cosh x - \sinh x$ 。

主要参数是双曲角  $x$ 。只支持-1.118 到+1.118 范围内的  $x$  值。对于所有输入值，必须在软件中应用  $2^{-n}$  的比例系数，其中  $n = 1$ 。缩放值  $x \cdot 0.5$  在 ARG1 中编程，系数  $n = 1$  必须在 SCALE 参数中编程。

没有使用次要参数。

主要结果 RES1 是双曲正弦  $\sinh x$ 。RES1 必须乘以 2 才能得到正确的结果。

次要结果 RES2 是双曲余弦， $\cosh x$ ，RES2 必须乘以 2 才能得到正确的结果。

### 双曲反正切 Hyperbolic arctangent

表 14.9 双曲反正切参数

| 参数    | 描述                                    | 范围                |
|-------|---------------------------------------|-------------------|
| ARG1  | $x \cdot 2^{-n}$                      | $[-0.403, 0.403]$ |
| ARG2  | Not applicable                        | -                 |
| RES1  | $2^{-n} \cdot \operatorname{atanh} x$ | $[-0.559, 0.559]$ |
| RES2  | 不适用                                   | -                 |
| SCALE | $n$                                   | 1                 |

这个函数计算输入参数  $x$  的双曲反正切。

主要参数是输入值  $x$ 。只支持-0.806 到+0.806 范围内的  $x$  值。值  $x$  必须按  $2^{-n}$  倍缩放，其中  $n = 1$ 。缩放值  $x \cdot 0.5$  在 ARG1 中编程，系数  $n = 1$  必须在 SCALE 参数中编程。

没有使用次要参数。

主要结果是双曲反正切， $\operatorname{atanh} x$ 。RES1 必须乘以 2 才能得到正确的值。

次要结果不被使用。

### 自然对数 Natural logarithm

表 14.10 自然对数参数

| 参数    | 描述                       | 范围                |
|-------|--------------------------|-------------------|
| ARG1  | $x \cdot 2^{-n}$         | $[0.054, 0.875]$  |
| ARG2  | Not applicable           | -                 |
| RES1  | $2^{-(n+1)} \cdot \ln x$ | $[-0.279, 0.137]$ |
| RES2  | Not applicable           | -                 |
| SCALE | $n$                      | $[1, 4]$          |

这个函数计算输入参数  $x$  的自然对数。

主要参数是输入值  $x$ 。只支持范围在 0.107 到 9.35 之间的  $x$  值。值  $x$  必须按  $2^{-n}$  倍缩放，使  $x \cdot 2^{-n} < 1 - 2^{-n}$ 。缩放值  $x \cdot 2^{-n}$  在 ARG1 中编程，系数  $n$  必须在 SCALE 参数中编程。

下表列出了有效的缩放系数  $n$ ，以及  $x$  和 ARG1 对应的范围。



表 14.11 自然对数标度系数及其对应范围

| n | X range              | ARG1 range                        |
|---|----------------------|-----------------------------------|
| 1 | $0.107 \leq x < 1$   | $0.0535 \leq \text{ARG1} < 0.5$   |
| 2 | $1 \leq x < 3$       | $0.25 \leq \text{ARG1} < 0.75$    |
| 3 | $3 \leq x < 7$       | $0.375 \leq \text{ARG1} < 0.875$  |
| 4 | $7 \leq x \leq 9.35$ | $0.4375 \leq \text{ARG1} < 0.584$ |

没有使用次要参数。

主要结果是自然对数  $\ln x$ 。RES1 必须乘以  $2^{(n+1)}$  才能得到正确的值。

次要结果不被使用。

## 平方根 Square root

表 14.12 平方根参数

| 参数    | 描述               | 范围            |
|-------|------------------|---------------|
| ARG1  | $x \cdot 2^{-n}$ | [0.027 0.875] |
| ARG2  | Not applicable   | -             |
| RES1  | $2^{-n}\sqrt{x}$ | [0.04 1]      |
| RES2  | Not applicable   | -             |
| SCALE | n                | [0 2]         |

这个函数计算输入参数  $x$  的平方根。

主要参数是输入值  $x$ 。只支持范围在 0.027 到 2.34 之间的  $x$  值。值  $x$  必须按  $2^{-n}$  倍缩放，使  $x \cdot 2^{-n} < (1 - 2^{-(n-2)})$ 。

缩放值  $x \cdot 2^{-n}$  在 ARG1 中编程，系数  $n$  必须在 SCALE 参数中编程。

下表列出了有效的缩放系数  $n$ ，以及  $x$  和 ARG1 对应的范围。

表 14.13 平方根比例系数和相应的范围

| n | X range                  | ARG1 range                           |
|---|--------------------------|--------------------------------------|
| 0 | $0.027 \leq x < 0.75$    | $0.027 \leq \text{ARG1} < 0.75$      |
| 1 | $0.75 \leq x < 1.75$     | $0.375 \leq \text{ARG1} < 0.875$     |
| 2 | $1.75 \leq x \leq 2.341$ | $0.4375 \leq \text{ARG1} \leq 0.585$ |

没有使用次要参数。

主要结果是  $x$  的平方根，RES1 必须乘以  $2^n$  才能得到正确的值。

次要结果不被使用。

### 14.3.3 定点表示法

CORDIC 以定点带符号整数格式操作。输入和输出值可以是 q1.31 或 q1.15。

在 q1.31 格式中，数字由 1 个符号位和 31 个小数位（二进制小数位）表示。因此，数字范围是 -1 (0x80000000) 到  $1 - 2^{-31}$  (0x7FFFFFFF)。

在 q1.15 格式中，数字范围是 1 (0x8000) 到  $1 - 2^{-15}$  (0x7FFF)。这种格式的优点是可以将两个输入参数打包到一个 32 位写入操作中，并且可以在一个 32 位读取操作中获取两个结果。

### 14.3.4 比例系数

10.3.2 节中列出的几个函数指定了缩放系数 SCALE。这允许将函数输入范围扩展到覆盖 CORDIC 支持的全部值范围，而不会使输入、输出或内部寄存器饱和。如果比例系数是需要，它必须在软件中计算并编程到 CORDIC\_CSR 寄存器的 SCALE 位域中。在对 CORDIC\_WDATA 寄存器中的缩放值进行编程之前，必须对输入参数进行相应的缩放。还必须取消从 CORDIC\_RDATA 寄存器中读取的结果的缩放。

注：由于缩放值的截断，缩放系数会导致精度损失。

### 14.3.5 精度

结果的精度取决于 CORDIC 迭代的次数。对于三角函数（正弦、余弦、相位、模），算法以每次迭代一个二进制数的恒定速率收敛，见图 12.1。

对于双曲函数（双曲正弦、双曲余弦、自然对数），由于 CORDIC 算法的特性，收敛速度不太稳定（参见图 12.2）。平方根函数的收敛速度大约是双曲函数的两倍（参见图 12.3）。

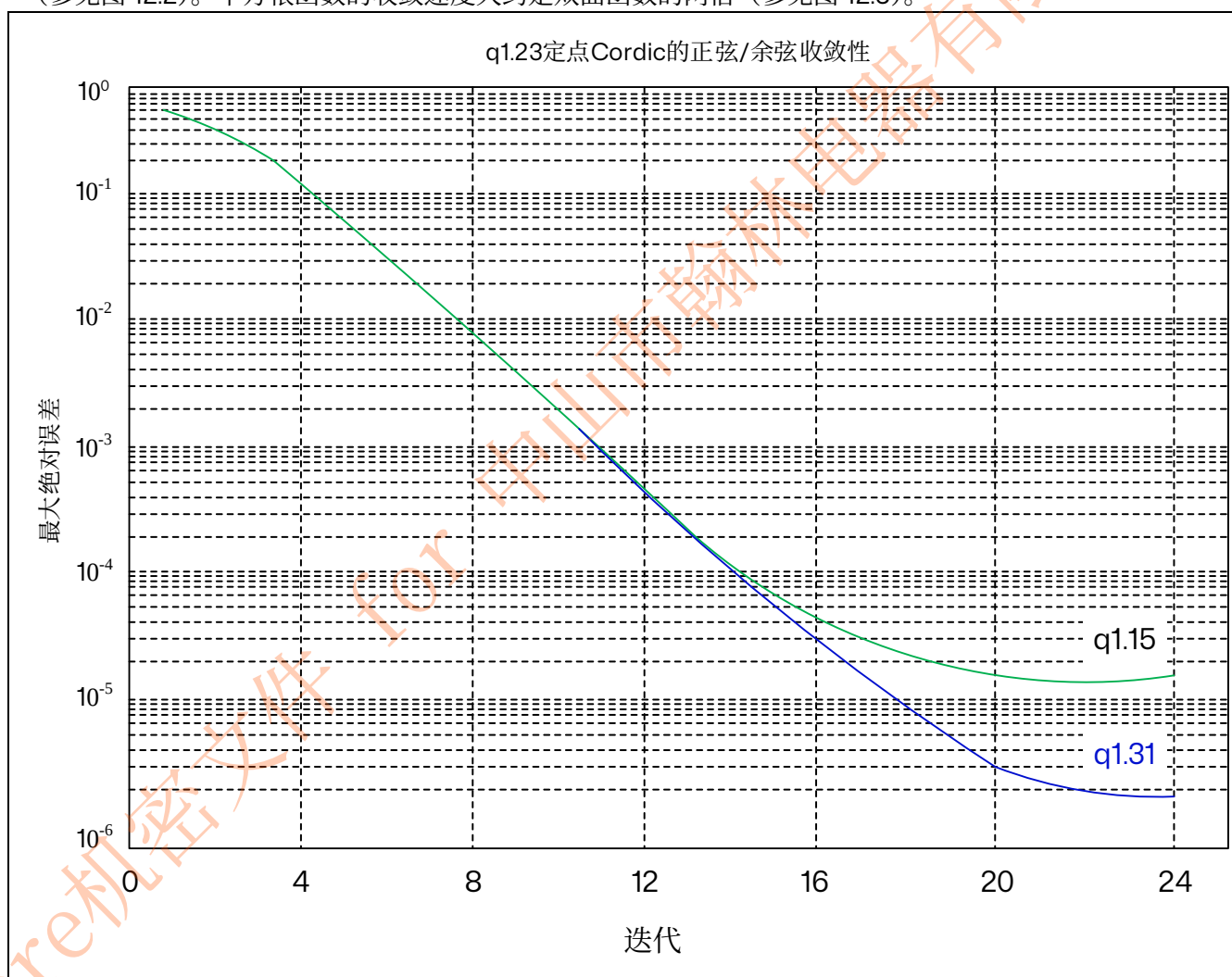


图 14.1 三角函数的 CODRIC 收敛性

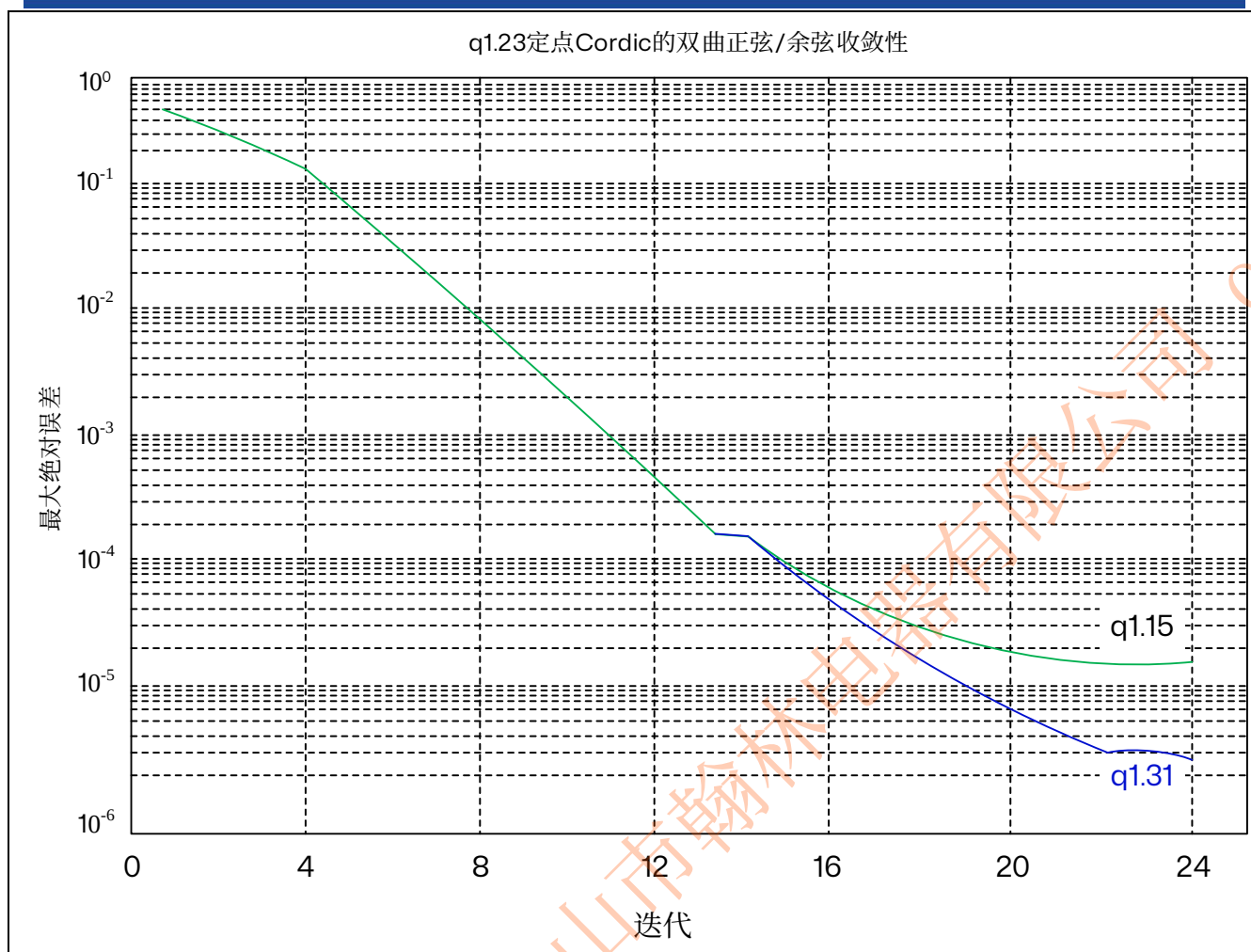


图 14.2 双曲函数的 CODRIC 收敛性

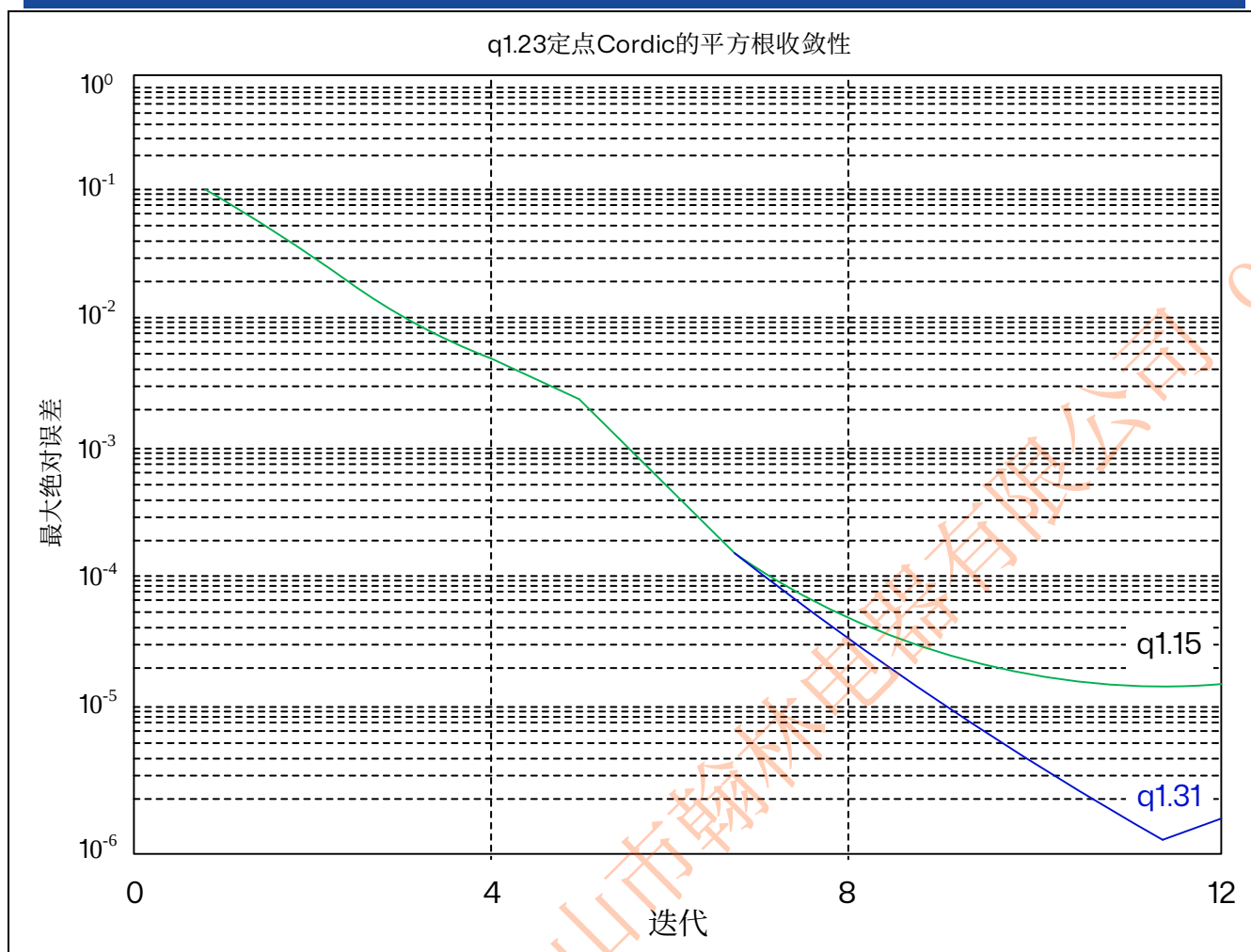


图 14.3 平方根的 CODRIC 收敛性

随着量化误差的增大，收敛速度逐渐降低。

CORDIC 每个时钟周期可以执行四次迭代。对于每个函数，下表显示了每四次迭代后剩余的最大误差，以及达到该精度所需的时钟周期数。从这个表中，可以在 CORDIC\_CR 寄存器的 PRECISION 位域中确定和编程所需的周期数。

一旦编程的迭代次数完成，协处理器就会停止，并且可以立即读取结果。

表 14.14 精度 vs.迭代次数

| Function                                                  | Number of iterations | Number of cycles | Max residual error <sup>(1)</sup> |              |
|-----------------------------------------------------------|----------------------|------------------|-----------------------------------|--------------|
|                                                           |                      |                  | q1.31 format                      | q1.15 format |
| Sin, Cos, Phase <sup>(2)</sup> , Mod, Atan <sup>(4)</sup> | 4                    | 1                | $2^{-3}$                          | $2^{-3}$     |
|                                                           | 8                    | 2                | $2^{-7}$                          | $2^{-7}$     |
|                                                           | 12                   | 3                | $2^{-11}$                         | $2^{-11}$    |
|                                                           | 16                   | 4                | $2^{-15}$                         | $2^{-15}$    |
|                                                           | 20                   | 5                | $2^{-18}$                         | $2^{-16}$    |
|                                                           | 24                   | 6                | $2^{-19}$                         | $2^{-16}$    |
| Sinh, Cosh, Atanh, Ln <sup>(3)</sup>                      | 4                    | 1                | $2^{-2}$                          | $2^{-2}$     |
|                                                           | 8                    | 2                | $2^{-6}$                          | $2^{-6}$     |

| Function            | Number of iterations | Number of cycles | Max residual error <sup>(1)</sup> |              |
|---------------------|----------------------|------------------|-----------------------------------|--------------|
|                     |                      |                  | q1.31 format                      | q1.15 format |
|                     | 12                   | 3                | $2^{-10}$                         | $2^{-10}$    |
|                     | 16                   | 4                | $2^{-13}$                         | $2^{-13}$    |
|                     | 20                   | 5                | $2^{-17}$                         | $2^{-15}$    |
|                     | 24                   | 6                | $2^{-18}$                         | $2^{-15}$    |
| Sqrt <sup>(4)</sup> | 4                    | 1                | $2^{-7}$                          | $2^{-7}$     |
|                     | 8                    | 2                | $2^{-14}$                         | $2^{-14}$    |
|                     | 12                   | 3                | $2^{-19}$                         | $2^{-15}$    |

1. 最大剩余误差是在给定次数的迭代后，与在双精度浮点中执行的相同计算相比，剩余的最大误差。可能会产生额外的舍入误差，对于 q15 格式最多为  $2^{-16}$ ，对于 q31 格式最多为  $2^{-20}$ 。
2. 对于模量 > 0.5。当量化误差变得显著时，可实现的精度与模量的大小成比例地降低。
3. SCALE = 1。如果使用更高的比例系数，则可实现的精度会成比例地降低。
4. SCALE = 0。如果使用更高的比例系数，则可实现的精度会成比例地降低。

### 14.3.6 Zero-overhead 模式

使用协处理器的最快方法是预先编程 CORDIC\_CSR 寄存器，其中包含要执行的函数 (FUNC)，所需的时钟周期数 (PRECISION)，输入和输出值的大小 (ARGSIZE, RESSIZE)，输入参数 (NARGS) 和/或结果 (NRES) 的数量，以及缩放系数 (SCALE)，如果适用的话。

随后，通过将输入参数写入 CORDIC\_WDATA 寄存器触发计算。一旦写入了正确数量的输入参数（并且任何正在进行的计算已经完成），就会使用这些输入参数和当前的 CORDIC\_CSR 设置启动一个新的计算。如果没有更改，则不需要重新编程 CORDIC\_CSR 寄存器。

如果需要双 32 位输入参数 (ARGSIZE = 0, NARGS = 1)，则必须先写入主输入参数 ARG1，然后再写入辅助参数 ARG2。如果次要参数在一系列计算中保持不变，则在第一次计算开始后，通过将参数的数量重新编程为 1 (NARGS = 0)，可以避免第二次写入。次要参数保持其编程值，只要函数没有改变。

*注意：复位后 ARG2 被设置为+1 (0x7FFFFFFF)。*

如果使用两个 16 位参数 (ARGSIZE = 1)，它们必须打包成一个 32 位字，ARG1 在最不重要的半字中，ARG2 在最重要的半字中。然后将打包的 32 位字写入 CORDIC\_WDATA 寄存器。在这种情况下，只需要写一次 (NARGS = 0)。

对于只接受一个输入参数 ARG1 的函数，建议设置 NARGS = 0。

如果 NARGS = 1，则必须执行对 CORDIC\_WDATA 的第二次写操作来触发计算。本例中不使用 ARG2 数据。

一旦计算开始，任何读取 CORDIC\_RDATA 寄存器的尝试都会在返回结果之前插入总线等待状态，直到计算完成。因此，软件可以写入输入并立即读取结果，而无需轮询查看其是否有效。或者，处理器可以在读取结果之前等待适当数量的时钟周期。这段时间可用于为下一个计算编程 CORDIC\_CSR 寄存器，并在需要时准备下一个输入数据。当计算正在进行时，可以重新编程 CORDIC\_CSR 寄存器，而不会影响正在进行的计算的结果。以同样的方式，一旦考虑了前面的参数，就可以用下一个参数更新 CORDIC\_WDATA 寄存器。下一个参数和设置保持挂起，直到前一个计算完成。

当计算完成后，可以从 CORDIC\_RDATA 寄存器中读取结果。

如果预期两个 32 位结果 (NRES = 1, RESSIZE = 0)，则首先读取主结果 (RES1)，然后读取辅助结果 (RES2)。如果只期望一个 32 位的结果 (NRES = 0, RESSIZE = 0)，则在第一次读取时输出 RES1。

如果期望得到 16 位的结果 (RESSIZE = 1)，那么对 CORDIC\_RDATA 的单次读取将获取打包成 32



位字的两个结果。RES1 在下半字，RES2 在上半字。在这种情况下，建议将 NRES = 0 编程。如果 NRES = 1，必须执行第二次读取 CORDIC\_RDATA，以便为下一个操作释放 CORDIC\_RDATA。第二次读取的数据必须被丢弃。

当读取了预期数量的结果时，如果已经写入了预期数量的参数，则开始下一次计算。这意味着在任何时候，都可能有一个计算正在进行，或者等待读取结果，还有一个操作挂起。当操作挂起时，对 CORDIC\_WDATA 的任何进一步访问都将取消挂起的操作并覆盖数据。

下面的序列总结了零开销模式下 CORDIC\_IP 的使用：

1. 用适当的设置对 CORDIC\_CSR 寄存器进行编程。
2. 在 CORDIC\_WDATA 寄存器中为第一个计算编写参数。这将启动第一个计算。
3. 如果需要，为下一次计算更新 CORDIC\_CSR 寄存器设置。
4. 在 CORDIC\_WDATA 寄存器中为下一次计算编写参数。
5. 从 CORDIC\_RDATA 寄存器读取结果。这将触发下一个计算。
6. 请转步骤 3。

### 14.3.7 轮询方式

当 CORDIC\_RDATA 寄存器中的新结果可用时，将在 CORDIC\_CSR 寄存器中设置 RRDY 标志。可以通过读取寄存器来轮询标志。它通过读取 CORDIC\_RDATA 寄存器（一次或两次取决于 CORDIC\_CSR 寄存器的 NRES 位域）来重置。

轮询 RRDY 标志所花费的时间比直接读取 CORDIC\_RDATA 寄存器稍长，因为没有在结果可用时立即读取结果。然而，处理器和总线接口在读取 CORDIC\_CSR 寄存器时不会停止，因此，如果不能接受停止处理器（例如，必须为低延迟中断提供服务），则该模式可能会引起兴趣。

### 14.3.8 中断模式

通过在 CORDIC\_CSR 寄存器中设置中断使能（IE）位，只要设置了 RRDY 标志，就会生成一个中断。当该标志被重置时，中断被清除。

这种模式允许在中断服务程序下读取计算结果，因此相对于其他任务具有优先级。然而，由于中断处理延迟，它比直接读取结果或轮询标志要慢。

### 14.3.9 直接存储器存取模式

如果在 CORDIC\_CSR 寄存器中设置了 DMA 写使能（DMAWEN）位，并且没有操作挂起，则生成 DMA 写通道请求。DMA 控制器可以将主输入参数（ARG1）从内存传输到 CORDIC\_WDATA 寄存器。

写入寄存器将取消 DMA 请求。如果 CORDIC\_CSR 寄存器中的 NARGS = 1，则生成第二个 DMA 写通道请求以将辅助输入参数（ARG2）传输到 CORDIC\_WDATA 寄存器中。当所有输入参数都被写入，并且任何正在进行的计算都已完成（通过读取结果）时，将启动一个新的计算并生成另一个 DMA 写通道请求。

如果在 CORDIC\_CSR 寄存器中设置了 DMA 读使能（DMAREN）位，则 RRDY 标志变为活动状态将生成 DMA 读通道请求。然后 DMA 控制器可以将主结果（RES1）从 CORDIC\_RDATA 寄存器传输到内存。读取寄存器使 DMA 请求失效。如果 CORDIC\_CSR 寄存器中的 NRES = 1，则生成第二个 DMA 请求以读出辅助结果（RES2）。当所有的结果都被读取后，RRDY 标志被解除。

DMA 读写通道可以单独使能。如果使能了这两个通道，CORDIC 可以在没有处理器干预的情况下自主地在数据缓冲区上执行重复计算。这允许处理器执行其他任务。对于写通道，DMA 控制器以存储器到外设模式操作，对于读通道，DMA 控制器以外设到存储器模式操作。注意，该序列是由设置 DMAWEN 标志的处理器启动的。此后，DMA 读写请求的生成速度与 CORDIC 处理数据的速度一样快。

在某些情况下，输入数据可以存储在存储器中，而输出数据每隔一定时间被传送到另一个外设，例如数模转换器。在这种情况下，目标外设每次需要新数据时都会生成一个 DMA 请求。DMA 控制器可以直接从 CORDIC\_RDATA 寄存器中获取下一个样本（在这种情况下，DMA 控制器以内存到外设模式操作，即使源是外设寄存器）。读取结果的行为允许 CORDIC 开始一个新的计算，这个计算反过来生成一个 DMA 写通道请求，DMA 控制器将下一个输入值传输到 CORDIC\_WDATA 寄存器。开启 DMA 写通道（DMAWEN = 1），但不能开启读通道。

以类似的方式，来自其他外设（如 ADC）的数据可以直接传输到 CORDIC\_WDATA 寄存器（在外设到内存模式下）。不能使能 DMA 写通道。如果 DMAREN = 1，则 CORDIC 处理输入数据并在完成时生成 DMA 读请求。DMA 控制器然后将结果从 CORDIC\_RDATA 寄存器传输到内存（外设到内存模式）。

*注意：没有生成 DMA 请求来编程 CORDIC\_CSR 寄存器。因此，DMA 模式只有在使用相同的设置重复执行相同的功能时才有用。*

在一系列 DMA 传输期间，比例系数不能改变。

*注意：由于 DMA 执行对 CORDIC\_WDATA 或 CORDIC\_RDATA 寄存器的访问，每个 DMA 请求必须被确认。如果在此之前发生了对相关寄存器的无关访问，则确认被过早地断言，并且可能阻塞 DMA 通道。因此，当使能 DMA 读通道时，必须避免 CPU 访问 CORDIC\_RDATA 寄存器。类似地，当使能 DMA 写通道时，处理器必须避免访问 CORDIC\_WDATA 寄存器。*



## 14.4 CORDIC 寄存器描述

### 14.4.1 CORDIC 控制/状态寄存器 (CORDIC\_CSR)

地址偏移: 0x04

复位值: 0x0000 0050

| 31       | 30       | 29 | 28 | 27 | 26         | 25 | 24 | 23             | 22          | 21          | 20    | 19        | 18         | 17         | 16  |
|----------|----------|----|----|----|------------|----|----|----------------|-------------|-------------|-------|-----------|------------|------------|-----|
| RRDY     | Reserved |    |    |    |            |    |    |                | ARG<br>SIZE | RES<br>SIZE | NARGS | NRES      | DMA<br>WEN | DMA<br>REN | IEN |
| r        |          |    |    |    |            |    |    |                | rw          | rw          | rw    | rw        | rw         | rw         | rw  |
| 15       | 14       | 13 | 12 | 11 | 10         | 9  | 8  | 7              | 6           | 5           | 4     | 3         | 2          | 1          | 0   |
| Reserved |          |    |    |    | SCALE[2:0] |    |    | PRECISION[3:0] |             |             |       | FUNC[3:0] |            |            |     |
|          |          |    |    |    | rw         | rw | rw | rw             | rw          | rw          | rw    | rw        | rw         | rw         | rw  |

Bit 31 **RRDY**: 结果就绪标志

0: 输出寄存器中没有新结果

1: CORDIC\_RDATA 寄存器包含新数据。

当 CORDIC 操作完成时, 这个位由硬件设置。当 CORDIC\_RDATA 寄存器被读取 (NRES+1)次时, 它被硬件重置。

当这个位被设置时, 如果 IEN 位也被设置, 则断言 CORDIC 中断。如果设置了 DMAREN 位, 则生成 DMA 读通道请求。设置此位时, 不会启动新的计算。

Bits 30:23 保留, 必须保持为复位值。

Bit 22 **ARGSIZE**: 输入数据宽度

0: 32 位

1: 16 位 ARGSIZE 选择用于表示输入数据的位数。

如果选择 32 位数据, CORDIC\_WDATA 寄存器需要 q1.31 格式的参数。

如果选择 16 位数据, CORDIC\_WDATA 寄存器需要 q1.15 格式的参数。

主要参数(ARG1)被写入最不重要的半字, 次要参数(ARG2)被写入最重要的半字。

Bit 21 **RESSIZE**: 输出数据宽度

0: 32 位

1: 16 位 RESSIZE 选择用于表示输出数据的位数。

如果选择 32 位数据, CORDIC\_RDATA 寄存器包含 q1.31 格式的结果。

如果选择 16 位数据, CORDIC\_RDATA 的最低有效半字包含 q1.15 格式的主结果 (RES1), 而最高有效半字包含 q1.15 格式的辅助结果(RES2)。

Bit 20 **NARGS**: CORDIC\_WDATA 寄存器期望的参数数

0: 下一次计算只需要一个 32 位的写(如果 ARGSIZE = 1, 则需要两个 16 位的值)。

1: 必须将两个 32 位的值写入 CORDIC\_WDATA 寄存器以触发下一次计算。

read 返回该位的当前状态。

Bit 19 **NRES**: CORDIC\_RDATA 寄存器中的结果数

0: 在下一次计算完成时, 只有一个 32 位值(如果 RESSIZE = 1, 则只有两个 16 位值)被传输到 CORDIC\_RDATA 寄存器。从 CORDIC\_RDATA 读取的一个值重置 RRDY 标志。

1: 在下一次计算完成时, 两个 32 位的值被转移到 CORDIC\_RDATA 寄存器。重置 RRDY 标志需要从 CORDIC\_RDATA 读取两次。

read 返回该位的当前状态。

Bit 18 **DMAWEN**: 使能 DMA 写通道

0: 禁用。没有生成 DMA 写请求。

1: 使能。当没有操作挂起时，在 DMA 写通道上生成请求。这个位由软件设置和清除。  
read 返回该位的当前状态。

Bit 17 **DMAREN**: 使能 DMA 读通道

0: 禁用。没有产生 DMA 读请求。

1: 使能。只要设置了 RRDY 标志，就会在 DMA 读通道上生成请求。  
该钻头由软件设置和清除。read 返回该位的当前状态。

Bit 16 **IEN**: 使能中断。

0: 禁用。不生成中断请求。

1: 使能。只要设置了 RRDY 标志，就会生成一个中断请求。  
该钻头由软件设置和清除。read 返回该位的当前状态。

Bits 15:11 保留，必须保持为复位值。

Bits 10:9 **SCALE[2:0]**: 比例系数

该位域的值表示应用于参数和/或结果的比例系数。一个值  $n$  意味着参数已经乘以了一个系数  $2^{-n}$ ，并且/或者结果需要乘以  $2^n$ 。每个函数的比例系数的适用性和合适的范围参见 10.3.2 节。

Bits 7:4 **PRECISION[3:0]**: 精度要求(迭代次数)

0: 保留

1 到 15: (迭代次数)/4 要确定给定精度所需的迭代次数，请参见表 10.14。  
请注意，对于大多数函数，该位域的推荐取值范围是 3 到 6。

Bits 3:0 **FUNC[3:0]**: 功能

0: 余弦

1: 正弦

2: 阶段

3: 模

4: arctan

5: 双曲余弦

6: 双曲正弦

7: arctan

8: 自然对数

9: 平方根

10 至 15: 保留

#### 14.4.2 CORDIC 参数寄存器 (CORDIC\_WRDAT)

地址偏移: 0x00

复位值: 0xFFFF XXXX

| 31         | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| ARG[31:16] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| w          | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  |
| 15         | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| ARG[15:0]  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| w          | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  |

Bits 31:0 **ARG[31:0]**: 函数输入参数

这个寄存器是用 CORDIC\_CSR 寄存器 FUNC 位域中选择的函数的输入参数编程的。

如果选择 32 位格式(CORDIC\_CSR。ARGSIZE = 0)和两个输入参数是必需的(CORDIC\_CSR。NARGS = 1)，则需要对该寄存器进行两次连续写操作。

第一个写入主参数(ARG1)，第二个写入辅助参数(ARG2)。

如果选择 32 位格式，并且只需要一个输入参数(NARGS = 0)，则只需要对该寄存器进行一次写操作，其中包含主参数(ARG1)。

如果选择 16 位格式(CORDIC\_CSR。ARGSIZE = 1)，对该寄存器的一次写入包含两个参数。主要参数(ARG1)在下半部分，ARG[15:0]，次要参数(ARG2)在上半部分，ARG[31:16]。在这种情况下，NARGS 必须设置为 0。

关于每个函数所需的实参及其允许的范围，请参阅第 10.3.2 节。

当写入所需数量的参数后，CORDIC 计算由 CORDIC\_CSR 指定的函数。如果之前的任何计算都已完成，则使用提供的输入参数进行 FUNC。如果正在进行计算，ARG1 和 arg2 值将保持挂起，直到计算完成并读取结果。在此期间，对寄存器的写入将取消挂起的操作并覆盖参数数据。

### 14.4.3 CORDIC 结果寄存器 (CORDIC\_RDATA)

地址偏移：0x08

复位值：0x0000 0000

| 31         | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| RES[31:16] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| w          | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  |
| 15         | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| RES[15:0]  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| w          | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  |

Bits 31:0 RES[31:0]: 函数的结果

如果选择 32 位格式(CORDIC\_CSR。RESSIZE = 0)和两个输出值(CORDIC\_CSR。NRES = 1)，当设置 RRDY 标志时，必须读取该寄存器两次。第一次读取获取主结果(RES1)。第二次读取

获取辅助结果(RES2)并重置 RRDY。

如果选择 32 位格式，并且只期望一个输出值(NRES = 0)，则只需要对该寄存器进行一次读取以获取主结果(RES1)并重置 RRDY 标志。

如果选择 16 位格式 (CORDIC\_CSR。RESSIZE = 1)，这个寄存器在下半部分(RES[15:0])包含主要结果(RES1)，在上半部分(RES[31:16])包含次要结果(RES2)。在这种情况下，NRES 必须设置为 0，并且只执行一次读操作。

从这个寄存器中读取将重置 CORDIC\_CSR 寄存器中的 RRDY 标志。

## 15 FMAC 滤波加速器 (FMAC)

### 15.1 概述

滤波加速器可进行向量算术操作，包含乘法器、累加器，以及本地存储器地址产生逻辑，用于在滤波加速器本地存储中索引向量数据。

滤波加速模块支持输入、输出循环翻转缓存，以便于实现有限/无限冲击响应数字滤波器。

滤波加速模块将 CPU 从冗长而频繁的滤波操作中解放出来，节省 CPU 算力用于其他任务。同时与 CPU 的软件滤波相比，硬件滤波加速模块可以较大程度提升滤波计算速度。

### 15.2 特性

- 16 位×16 位乘法器
- 24+2 位累加器，支持饱和处理
- 16 位数据输入、输出
- 256×16 位本地数据存储
- 本地存储最多可以定义 3 个数据缓存区域（两个输入缓存，一个输出缓存）缓存基地址和缓存大小可以通过寄存器配置
- 输入、输出缓存可以作为循环 buffer 使用
- 滤波函数：FIR，直接 1 型 IIR
- 向量操作：点积，卷积，相关
- AHB 总线接口
- 支持 DMA 读写数据

## 15.3 FMAC 功能描述

### 15.3.1 通用描述

FMAC 滤波加速模块如图 13.1 所示。

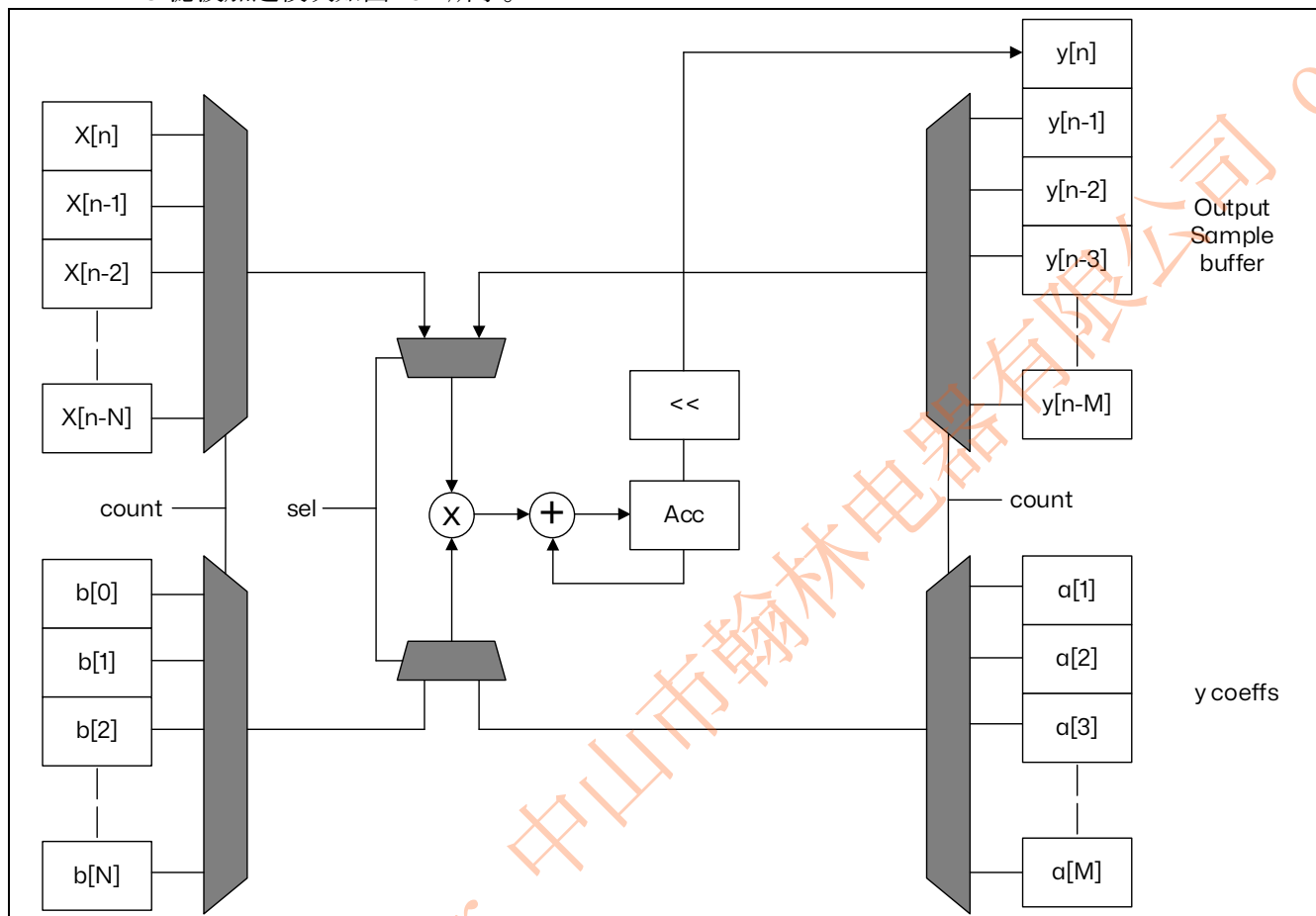


图 15.1 滤波加速模块框图

该模块的核心是定点乘累加器 (MAC)。乘累加器从本地存储中读取两个 16 位有符号数作为输入，相乘以后与之前的和进行累加。本地存储中的数据地址由一系列的地址指针来决定。地址指针可以加载初始值，递增，递减，也可以被内部硬件复位回初值。所有的指针和 MAC 模块都是由内部的硬件逻辑控制来完成指定的操作序列。

如果要计算点积，两个输入向量被 CPU 或 DMA 存入 FMAC 本地存储中。当相应的操作被配置并开始执行时，每次硬件逻辑从本地存储读取来自两个向量的一对数据作为 MAC 输入，乘累加后暂存至内部中间结果寄存器，当所有向量元素均计算完成后，累加器的输出作为最终结果被写回本地存储，等待 CPU 或 DMA 读出。

如果要作为 FIR 滤波器（卷积）使用，FMAC 需要反复计算滤波器系数和输入向量的点积。每个采样周期，输入向量位移一个采样数据，最早的一个采样数据被丢弃，加入一个新的采样数据，然后使用相同的滤波器系数向量重新计算一次点积。如此往复。

如果要作为 IIR 滤波器使用，需要在 FIR 的基础上再进行计算输出结果与反馈系数向量的点积并与 FIR 的滤波结果相加。

更多的滤波器操作实现细节见后续章节介绍。

### 15.3.2 本地存储与缓存

FMAC 包含有一个 256×16 位的可读写存储器作为 FMAC 的本地存储

- 输入数据（输入向量元素）存储在两个输入 buffer X1 和 X2 中。
- 输出数据（操作结果）存储在 buffer Y 中。
- 缓存基地址和缓存大小由以下变量指定：
  - x1\_base 为 X1 buffer 在本地存储中的基地址，
  - x2\_base 为 X2 buffer 在本地存储中的基地址，
  - y\_base 为 Y buffer 在本地存储中的基地址，
  - x1\_buf\_size 为 X1 buffer 的缓存大小，即其内包含多少个 16bit 有符号数
  - x2\_buf\_size 为 X2 buffer 的缓存大小，即其内包含多少个 16bit 有符号数
  - y\_buf\_size 为 Y buffer 的缓存大小，即其内包含多少个 16bit 有符号数

以上参数均可通过模块的寄存器进行配置。

CPU 或 DMA 可以通过 FMAC 模块的初始化功能，也可以通过直接写入，来初始化各个缓存的数据。写入的数据根据硬件写指针存入目标 buffer，每次数据写入，硬件写指针自动加 1，当写指针到达 buffer 尾时，再次回到 buffer 基地址。初始化功能用于滤波操作前的数据预装载，或者初始化滤波器系数。

#### 缓存配置

缓存基地址和缓存大小通过 X1, X2, Y buffer 的配置寄存器进行配置。基地址可以是本地存储的任意位置，只要基地址+缓存大小不超过本地存储地址范围（0x00~0xFF）即可。即基地址+缓存大小 ≤256。

缓存的大小和位置没有限制，缓存甚至可以重叠，但是要解决好数据读写先后的问题。对于滤波操作来说，一般不建议缓存重叠，容易出现不易调试的错误。

当缓存作为循环 buffer 使用时，可以留有一定的动态余量（headroom, d）。同时可以设置 FIFO 的水线深度（watermark level）来调整 CPU 和 DMA 的访问频次。动态余量和水线深度应该根据应用需求来进行适当选择。为了最大化 FMAC 的吞吐率，输入 buffer 应该永不为空，因此 d 应该大于水线深度，以防止因为中断和 DMA 操作的延迟导致 FMAC 等待数据输入而暂时停滞。另一方面，如果输入数据的填充速度小于被 FMAC 处理的速度，输入缓存可能会被 FMAC 读空，并等待 CPU 或 DMA 写入下一个数据。d 可以等于水线深度，确保输入读取不会溢出。

### 15.3.3 输入缓存

X1 和 X2 buffer 为 FMAC 的输入缓存，用于存储输入数据。FMAC 的每次乘法操作从 X1 读取一个数据，从 X2 读取一个数据，然后相乘。FMAC 内部硬件指针，控制读取数据相对于缓存基地址的偏移。指针由硬件根据操作配置自动管理。

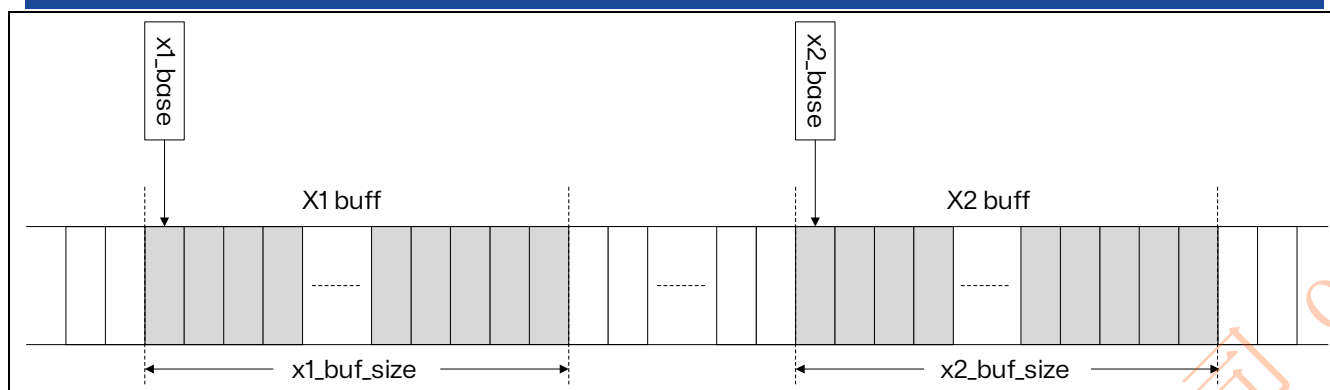


图 15.2 输入缓存区域

X1 buffer 可以作为循环 buffer 使用，只要 buffer 有空闲空间即可写入输入数据。buffer 在操作前可以进行初始化，也可以不进行初始化。如果操作开始时 buffer 内部没有数据，则 buffer 空的标志会触发 CPU 或 DMA 进行新数据装填直到有足够的数据进行滤波计算。



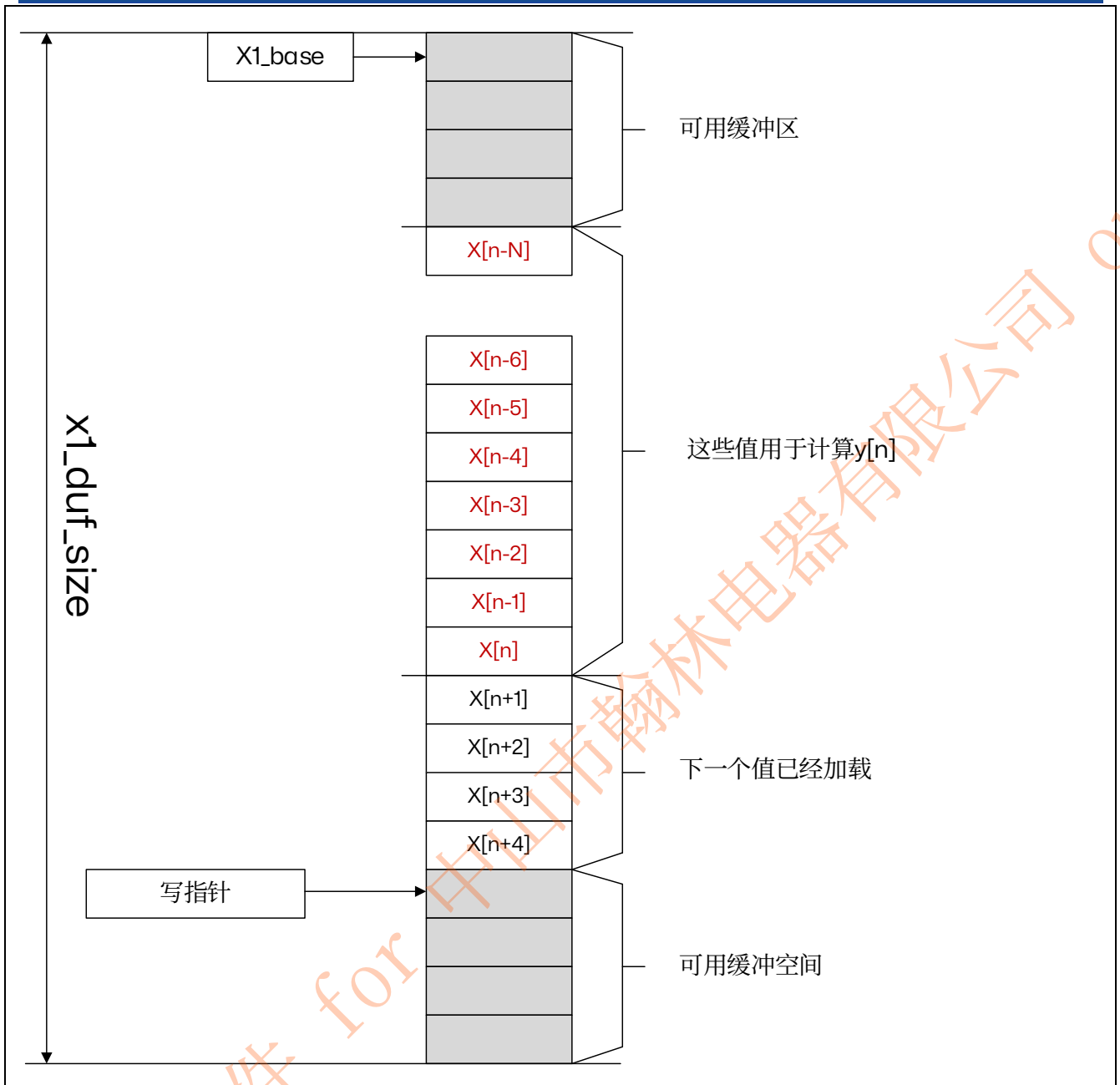


图 15.3 循环输入 buffer

X2 buffer 无法作为循环 buffer 使用，而且通常需要预装载。在滤波操作中，X2 buffer 用于存储滤波器系数。

当 X1 buffer 作为循环 buffer 使用时，x1\_buf\_size 应略大于当前操作所需数据个数（滤波器阶数），这样 buffer 可以一直存在未经 FMAC 使用的新数据。图 14.3 展示了 FMAC 在进行滤波操作时的缓存分布情况。当计算输出  $y[n]$  时，FMAC 需要使用  $N+1$  个输入数据， $x[n-N]$  至  $x[n]$ 。当计算完毕  $y[n]$ ，FMAC 开始使用  $x[n+1-N]$  至  $x[n+1]$  计算  $y[n+1]$ ，而最早输入的数据  $x[n-N]$  不再被 FMAC 所需要，可以被新数据替换。

CPU 或 DMA 需要在 FMAC 需要数据时进行及时填装，否则 FMAC 会将 buffer 空标志置位，并暂停 FMAC 内部的滤波计算操作，直到新的数据填入 buffer。X1 buffer 硬件上保证了不会出现因为读取而造成的下溢出。

举例来说，如果采样滤波流程是受控于定时器或是 ADC 外设，则 buffer 会定期地进入空状态，因为 FMAC 处理数据的速度快于采样数据产生的速度。这是正常特性。

如果 buffer 内的空闲空间小于水线阈值(FMAC\_X1BUFCFG.FULL\_WM)，则 buffer 的满标志会置位。如果满标志未置位，则 FMAC 会产生中断请求，请求更多的数据填入 buffer。水线的存在使得一次中断填入多个数据成为可能，且不会发生 buffer 上溢出。

如果发生了上溢出，则 FMAC\_SR.OVFL 会置位，且写入数据会被丢弃，写指针维持不变。X1 buffer 没有下溢出标志。

X1 buffer 在滤波操作器件的读写动作如图 14.4 所示。本例为一个 8 阶 FIR 滤波器，水线阈值设置为 4。

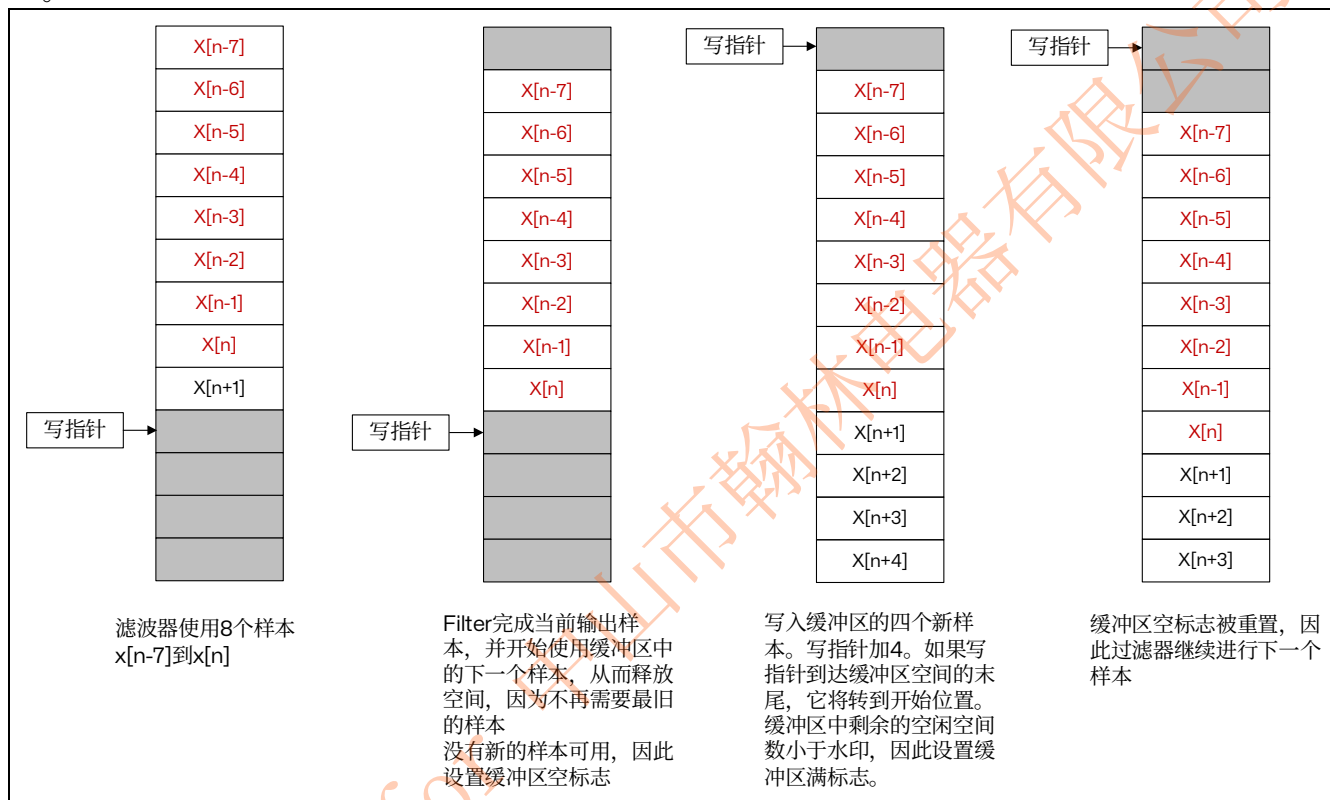


图 15.4 循环输入 buffer 的读写操作

### 15.3.4 输出缓存

Y buffer 用于存储累加器的输出，并等待 CPU 或 DMA 读取。每读取一次，Y buffer 的读指针加 1，当指针遇到 buffer 尾时，自动回到 buffer 基地址。

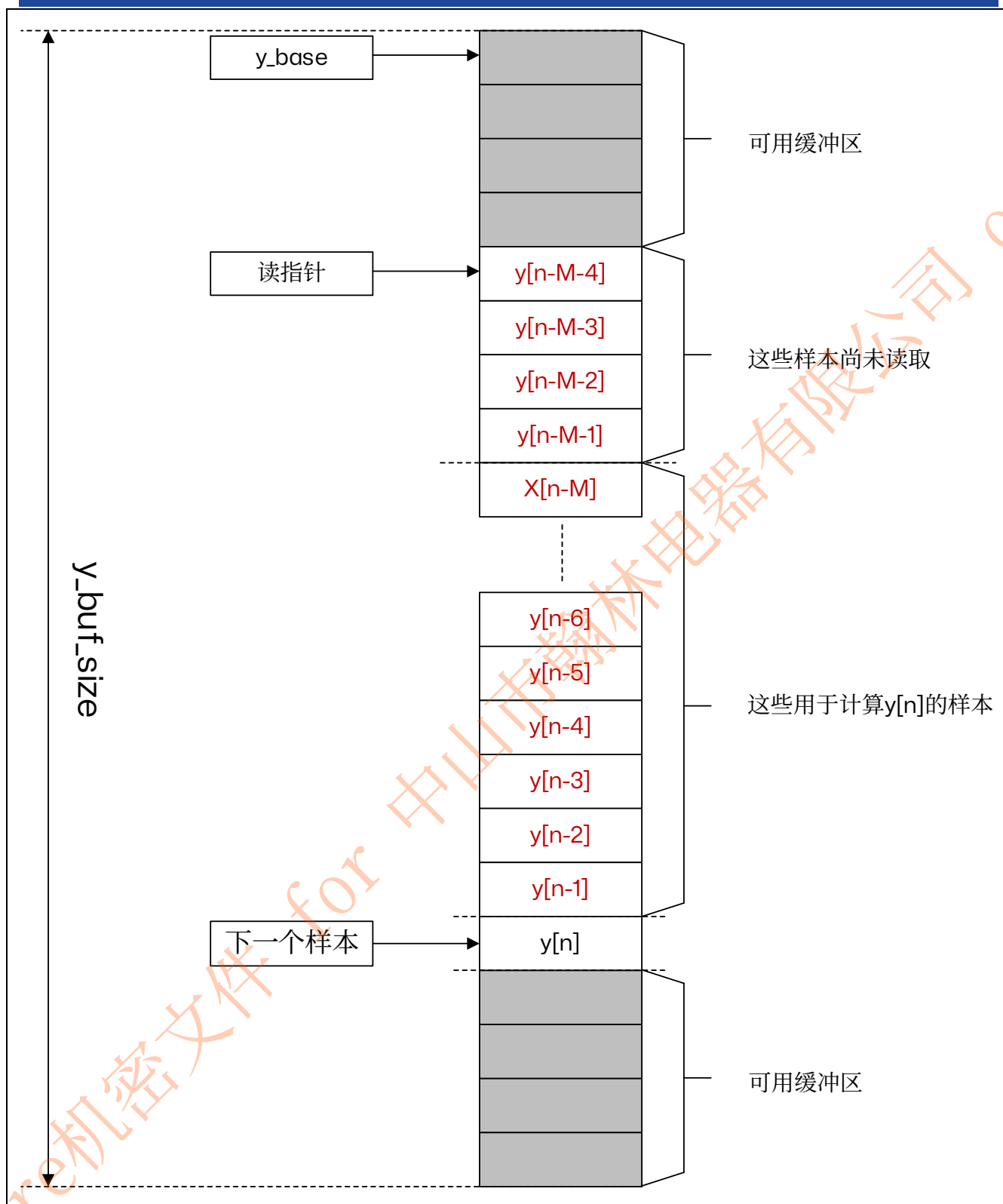


图 15.5 循环输出 buffer

Y buffer 也可以作为循环 buffer 使用。如果待写入的输出数据地址与读指针重合，即与未读取的结果数据重叠，则 buffer 满标志置位，FMAC 会中断计算操作直到 Y buffer 中的结果数据被读走。

当进行 IIR 滤波操作时，Y buffer 需要预装载 M 个历史输出数据  $y[n-M]$  至  $y[n-1]$ ，用于计算下一个输出数据  $y[n]$ 。每次 Y buffer 填入一个新的数据，最旧的数据  $y[n-M]$  会被替换掉。

如果 Y buffer 内的未读取数据个数少于水印阈值 (FMAC\_YBUFCFG.EMPTY\_WM), 则 Y buffer 空标志置位。如果 Y buffer 不为空, 则会通过中断请求 CPU 或 DMA 读取其中的输出数据。水印阈值的存在使得 CPU 可以运行一次中断服务函数读取多个输出数据, 而无下溢出风险。但如果发生了下溢出, FMAC\_CR.UNFL 的标志会置位。这种情况下 Y buffer 读指针维持不变, 读取的数据为读指针指向的数据。

Y buffer 作为循环 buffer 使用时的读写操作如下图所示。本例为一个 7 阶 IIR 滤波器, 水印阈值设置为 4。

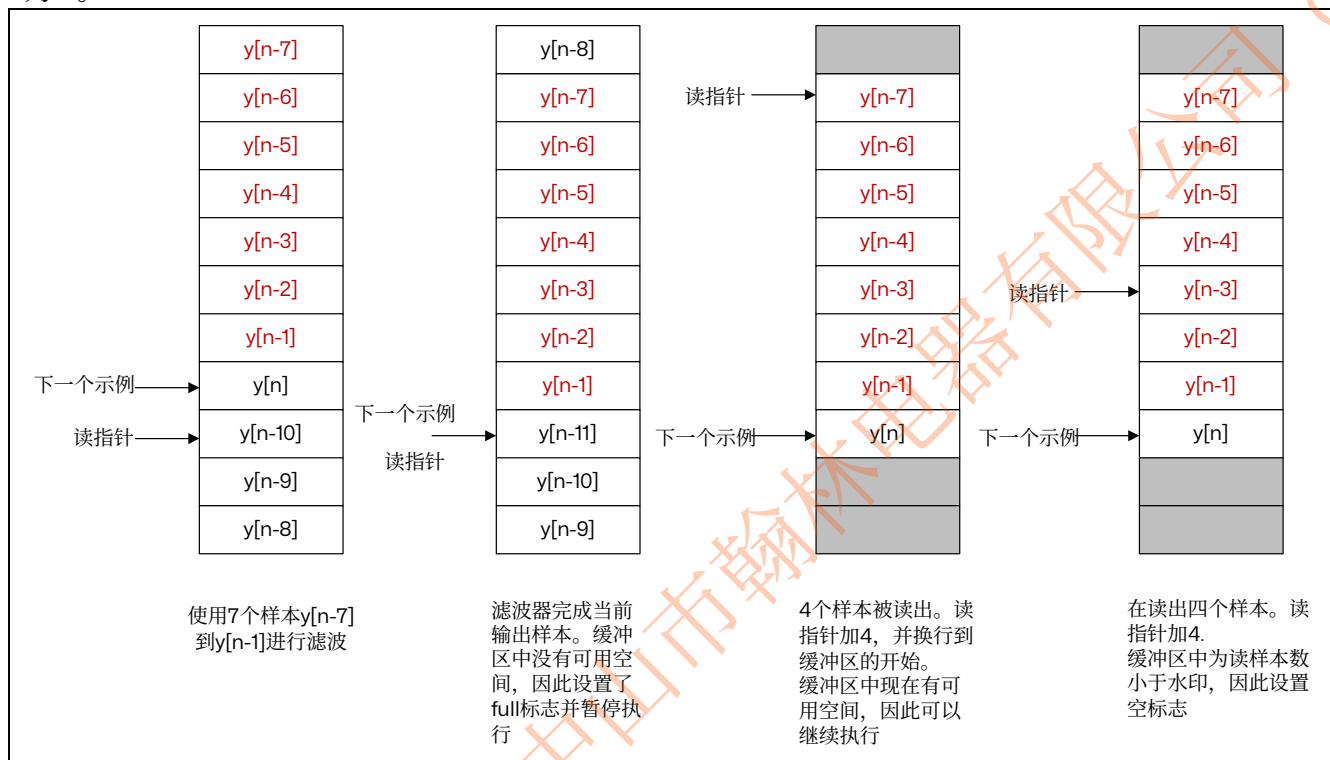


图 15.6 循环输出 buffer 读写操作

## 15.4 FMAC 寄存器描述

### 15.4.1 FMAC X1 buffer 配置寄存器 (FMAC\_X1BUFCFG)

地址偏移: 0x00

复位值: 0x0000 0000

访问: 字访问

该寄存器仅当 FMAC\_PARAM 寄存器中的 START=0 时被修改。

|                  |    |    |    |    |    |              |    |                  |    |    |    |    |    |    |    |
|------------------|----|----|----|----|----|--------------|----|------------------|----|----|----|----|----|----|----|
| 31               | 30 | 29 | 28 | 27 | 26 | 25           | 24 | 23               | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved         |    |    |    |    |    | FULL_WM[1:0] |    | Reserved         |    |    |    |    |    |    |    |
|                  |    |    |    |    |    | rw           | rw |                  |    |    |    |    |    |    |    |
| 15               | 14 | 13 | 12 | 11 | 10 | 9            | 8  | 7                | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| X1_BUF_SIZE[7:0] |    |    |    |    |    |              |    | X1_BUF_SIZE[7:0] |    |    |    |    |    |    |    |
| rw               | rw | rw | rw | rw | rw | rw           | rw | rw               | rw | rw | rw | rw | rw | rw | rw |

Bits 31:26 保留, 必须保持为复位值。

Bit 25:24 **FULL\_WM[1:0]**: X1 buffer 满水印阈值。

当 X1 buffer 中的空闲位置少于  $2^{\text{FULL\_WM}}$  时, X1 buffer 满标志置位。

0: 水印阈值为 1

1: 水线阈值为 2

2: 水线阈值为 4

3: 水线阈值为 8

设置水线阈值大于 1 可以在一次中断处理中传输多个数据到 X1 buffer 中。

当使用 DMA 写 X1 buffer 时水线阈值应该设置为 1。

Bits 23:16 保留，必须保持为复位值。

Bit 15:8 **X1\_BUF\_SIZE[7:0]**: X1 buffer 大小

X1 buffer 中包含的 16bit 有符号数个数，buffer 尺寸最小值为滤波器前馈阶数 (+水线阈值-1)

Bit 7:0 **X1\_BASE[7:0]**: X1 buffer 基地址

#### 15.4.2 FMAC X2 buffer 配置寄存器 (FMAC\_X2BUFCFG)

地址偏移: 0x04

复位值: 0x0000 0000

访问: 字访问

|                  |    |    |    |    |    |    |    |              |    |    |    |    |    |    |    |
|------------------|----|----|----|----|----|----|----|--------------|----|----|----|----|----|----|----|
| 31               | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23           | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved         |    |    |    |    |    |    |    |              |    |    |    |    |    |    |    |
| 15               | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7            | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| X2_BUF_SIZE[7:0] |    |    |    |    |    |    |    | X2_BASE[7:0] |    |    |    |    |    |    |    |
| rw               | rw | rw | rw | rw | rw | rw | rw | rw           | rw | rw | rw | rw | rw | rw | rw |

Bits 31:16 保留，必须保持为复位值。

Bit 15:8 **X2\_BUF\_SIZE**: X2 buffer 大小

X2 buffer 中包含的中包含的 16bit 有符号数个数，这个位域在操作进行中 ((START=1) 时无法修改

Bit 7:0 **X2\_BASE**: X2 buffer 基地址

#### 15.4.3 FMAC\_Y buffer 配置寄存器 (FMAC\_YBUFCFG)

地址偏移: 0x08

复位值: 0x0000 0000

访问: 字访问

该寄存器仅当 FMAC\_PARAM 寄存器中的 START= 0 时被修改。

|                |    |    |    |    |    |               |    |             |    |    |    |    |    |    |    |
|----------------|----|----|----|----|----|---------------|----|-------------|----|----|----|----|----|----|----|
| 31             | 30 | 29 | 28 | 27 | 26 | 25            | 24 | 23          | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved       |    |    |    |    |    | EMPTY_WM[1:0] |    | Reserved    |    |    |    |    |    |    |    |
|                |    |    |    |    |    | rw            | rw |             |    |    |    |    |    |    |    |
| 15             | 14 | 13 | 12 | 11 | 10 | 9             | 8  | 7           | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| YBUF_SIZE[7:0] |    |    |    |    |    |               |    | Y_BASE[7:0] |    |    |    |    |    |    |    |
| rw             | rw | rw | rw | rw | rw | rw            | rw | rw          | rw | rw | rw | rw | rw | rw | rw |

Bits 31:26 保留，必须保持为复位值。

Bit 25:24 **EMPTY\_WM**: Y buffer 空水线阈值。

当 Ybuffer 中的有效数据数量少于  $2^{\text{FULL\_WM}}$  时，Y buffer 空标志置位。

0: 水线阈值为 1

1: 水线阈值为 2

2: 水线阈值为 4

3: 水线阈值为 8

设置水准阈值大于 1 可以在一次中断处理中从 Y buffer 读取多个数据中。

当使用 DMA 读取 Y buffer 时水准阈值应该设置为 1

Bits 23:16 保留，必须保持为复位值。

Bit 15:8 **Y\_BUF\_SIZE**: Y buffer 大小

Y buffer 中包含的中包含的 16bit 有符号数个数。对于 FIR 滤波器，buffer 尺寸最小值为 1 (+水准阈值)，对于 IIR 滤波器，buffer 尺寸最小值为滤波器反馈阶数 (+水准阈值)

Bit 7:0 **Y\_BASE**: Y buffer 基地址

#### 15.4.4 FMAC 参数寄存器 (FMAC\_PARAM)

地址偏移: 0x0C

复位值: 0x0000 0000

访问: 字访问

| 31     | 30        | 29 | 28 | 27 | 26 | 25 | 24 | 23     | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|--------|-----------|----|----|----|----|----|----|--------|----|----|----|----|----|----|----|
| START  | FUNC[6:0] |    |    |    |    |    |    | R[7:0] |    |    |    |    |    |    |    |
| rw     | rw        | rw | rw | rw | rw | rw | rw | rw     | rw | rw | rw | rw | rw | rw | rw |
| 15     | 14        | 13 | 12 | 11 | 10 | 9  | 8  | 7      | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Q[7:0] |           |    |    |    |    |    |    | P[7:0] |    |    |    |    |    |    |    |
| rw     | rw        | rw | rw | rw | rw | rw | rw | rw     | rw | rw | rw | rw | rw | rw | rw |

Bit 31 **START**: 使能操作

0: 停止 FMAC 执行

1: 开始 FMAC 执行 (FMAC 执行过程中)

写入 1 触发 FUNC 指定操作开始执行，写入 0 可以停止执行中的操作，初始化操作完成后此位由硬件自动清零。

Bits 30:28 保留，必须保持为复位值。

Bits 27:24 **FUNC**: 函数

0: Reserved

1: Load X1 buffer

2: Load X2 buffer

3: Load Y buffer

4: to 7: Reserved

8: Convolution (FIR filter)

9: IIR filter (direct form 1)

此位域在 START=1 时无法修改

Bits 23:16 **R**: 输入参数 R

此位域在 START=1 时无法修改

Bits 15:8 **Q**: 输入参数 Q

此位域在 START=1 时无法修改

Bits 7:0 **P**: 输入参数 P

此位域在 START=1 时无法修改

### 15.4.5 FMAC 控制寄存器 (FMAC\_CR)

地址偏移: 0x10

复位值: 0x0000 0000

| 31         | 30       | 29 | 28 | 27 | 26 | 25         | 24         | 23       | 22 | 21 | 20         | 19          | 18          | 17   | 16    |
|------------|----------|----|----|----|----|------------|------------|----------|----|----|------------|-------------|-------------|------|-------|
| Reserved   |          |    |    |    |    |            |            |          |    |    |            |             |             |      | RESET |
|            |          |    |    |    |    |            |            |          |    |    |            |             |             |      | rw    |
| 15         | 14       | 13 | 12 | 11 | 10 | 9          | 8          | 7        | 6  | 5  | 4          | 3           | 2           | 1    | 0     |
| CLIP<br>EN | Reserved |    |    |    |    | DMA<br>WEN | DMA<br>REN | Reserved |    |    | SATI<br>EN | UNFL<br>IEN | OVFL<br>IEN | WIEN | RIEN  |
| rw         |          |    |    |    |    | rw         | rw         |          |    |    | rw         | rw          | rw          | rw   | rw    |

Bits 31:17 保留, 必须保持为复位值。

Bit 16 **RESET**: 复位 FMAC 模块

Bit 15 **CLIPEN**: 累加器输出饱和使能

0: 禁用, 累加器增益控制后的输出被钳位在 q1.15 范围内, 丢弃高位

1: 使能, 累加器增益控制后的输出超出 q1.15 时, 根据符号的不同饱和到最大正值或负值

Bit 9 **DMAWEN**: DMA 写 X1 buffer 请求使能

0: 禁用, 无 DMA 写请求产生

1: 使能, 当 X1 不满时有 DMA 写请求产生

此位只能在 START=0 时进行改写

Bit 8 **DMAREN**: DMA 读请求使能

0: 禁用, 无 DMA 读请求产生

1: 使能, 当 Y 不空时有 DMA 读请求产生

此位只能在 START=0 时进行改写

Bit 4 **SATIEN**: 累加器饱和中断使能

0: 禁用

1: 使能, 当 FMAC\_SR.SAT 置位时产生中断请求

Bit 3 **UNFLIEN**: Ybuffer 下溢中断使能

0: 禁用, 无 DMA 读请求产生

1: 使能, 当 FMAC\_SR.UNFL 置位时产生中断请求

Bit 2 **OVFLIEN**: X1buffer 上溢中断使能

0: 禁用,

1: 使能, 当 FMAC\_SR.OVFL 置位时产生中断请求

Bit 1 **WIEN**: X1buffer 写请求中断使能

0: 禁用,

1: 使能, 当 X1 buffer 不满时, 向 CPU 发出写请求中断

Bit 0 **RIEN**: Ybuffer 读请求使能

0: 禁用,

1: 使能, 当 Y buffer 不空时, 向 CPU 发出读请求中断



### 15.4.6 FMAC 状态寄存器 (FMAC\_SR)

地址偏移: 0x14

复位值: 0x0000 0001

|          |    |    |    |    |     |      |      |          |    |    |    |    |        |    |    |
|----------|----|----|----|----|-----|------|------|----------|----|----|----|----|--------|----|----|
| 31       | 30 | 29 | 28 | 27 | 26  | 25   | 24   | 23       | 22 | 21 | 20 | 19 | 18     | 17 | 16 |
| Reserved |    |    |    |    |     |      |      |          |    |    |    |    |        |    |    |
| 15       | 14 | 13 | 12 | 11 | 10  | 9    | 8    | 7        | 6  | 5  | 4  | 3  | 2      | 1  | 0  |
| Reserved |    |    |    |    | SAT | UNFL | OVFL | Reserved |    |    |    |    | X1FULL | Y  |    |
|          |    |    |    |    | r   | r    | r    |          |    |    |    |    | r      | r  |    |

Bits 31:11 保留, 必须保持为复位值。

Bit 10 **SAT**: 饱和错误标志

当累加器输出结果超出 Q1.15Q 范围时, 此位置位

0: 未发生饱和事件

1: 监测到饱和事件发生。如果 FMAC\_CR.SATIEN 置位, 则产生中断请求

Bit 9 **UNFLIEN**: Ybuffer 下溢错误标志

当 Y buffer 中无有效数据且软件或 DMA 读取 FMAC\_RDATA 时产生下溢出事件

0: 未发生下溢出事件

1: 监测到下溢出事件发生。如果 FMAC\_CR.UNFLIEN 置位, 则产生中断请求

Bit 8 **OVFLIEN**: X1buffer 上溢错误标志

当 X1 buffer 中无空闲且软件或 DMA 通过 FMAC\_WDATA 向 X1 buffer 写入数据时产生上溢出事件

0: 未发生上溢出事件

1: 监测到上溢出事件发生。如果 FMAC\_CR.OVFLIEN 置位, 则产生中断请求

Bit 1 **X1FULL**: X1buffer 满标志

0: X1buffer 不满时

1: X1buffer 满

Bit 0 **YEMPTY**: 空标志

0: Ybuffer 不空

1: Ybuffer 空

### 15.4.7 FMAC 写数据寄存器 (FMAC\_WDATA)

地址偏移: 0x18

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| WDATA[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:16 保留, 必须保持为复位值。

Bits 15:0 **WDATA**: 写入数据

向此寄存器写入数据, 数据被写入写指针指向的本地存储地址, 写后写指针自动加 1

### 15.4.8 FMAC 读数据寄存器 (FMAC\_RDATA)

地址偏移: 0x1C

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| RDATA[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:16 保留, 必须保持为复位值。

Bits 15:0 **RDATA**: 读出数据

从寄存器读出数据, 数据被从读指针指向的本地存储地址读出, 读后读指针自动加 1

## 16 模拟数字转换器 (ADC)

### 16.1 简介

12 位 ADC 是一种逐次逼近型模拟数字转换器。各通道的 A/D 转换可以单次、连续、扫描或间断模式执行。ADC 的结果可以左对齐或右对齐方式存储在 16 位数据寄存器中。

模拟看门狗特性允许应用程序检测输入电压是否超出用户定义的高/低阈值。

ADC 的输入时钟不得超过 64MHz，它是由 SYSCLK 或 PLLPCLK 产生。

### 16.2 ADC 主要特征

- 3 个 ADC
- 12 位 ADC
- 快慢通道：快速通道：4M/慢速通道：2M
- 可选左右对齐
- ADC 时钟可选 SYSCLK 或者 PLLPCLK，最大 64M
- ADC1&ADC2 支持双 ADC
- 支持 DMA 功能
- 支持自校准
- 通道采样时间可配置
- 支持注入/规则通道
- 注入结果有 OFFEST 功能
- 支持单次/连续模式
- 支持单通道/扫描多通道模式，注入序列最长 4，规则序列最长 8
- 触发源可选
- 支持模拟看门狗 AWD 功能
- ADC 输入范围：VREF- ≤ VIN ≤ VREF+

### 16.3 ADC 功能描述

下图为 ADC1/2/3 模块框图。

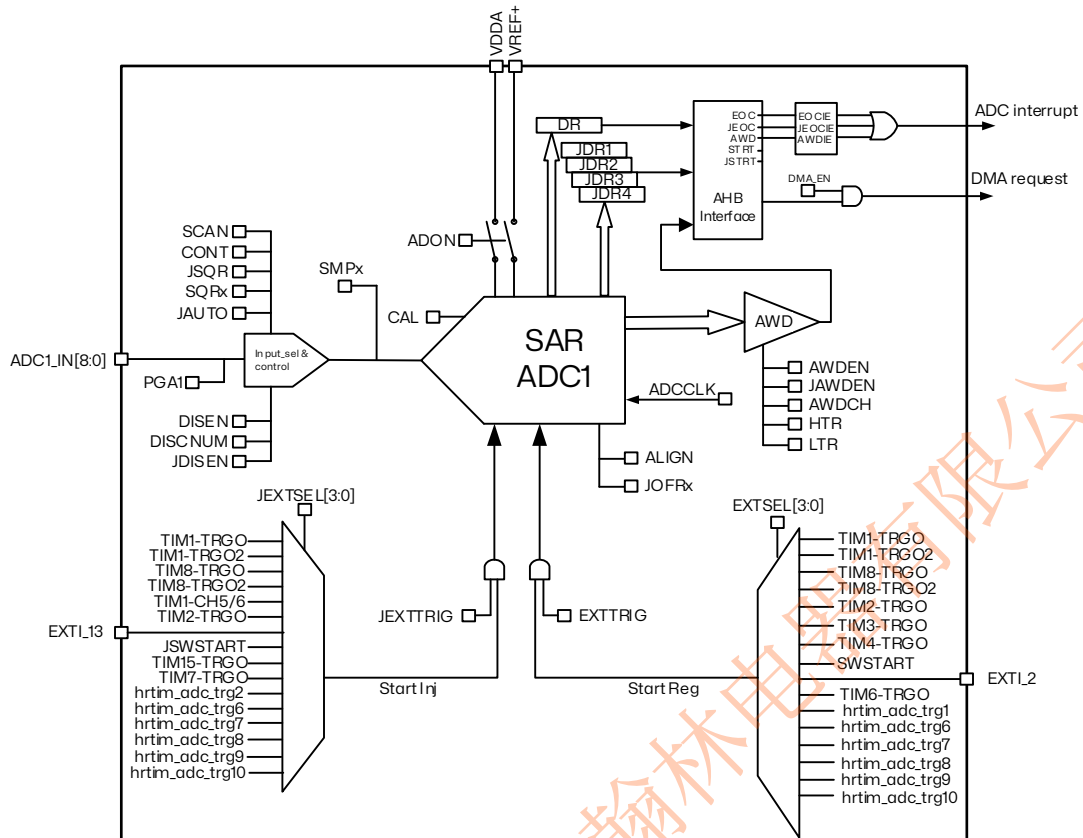


图 16.1 ADC1 框图

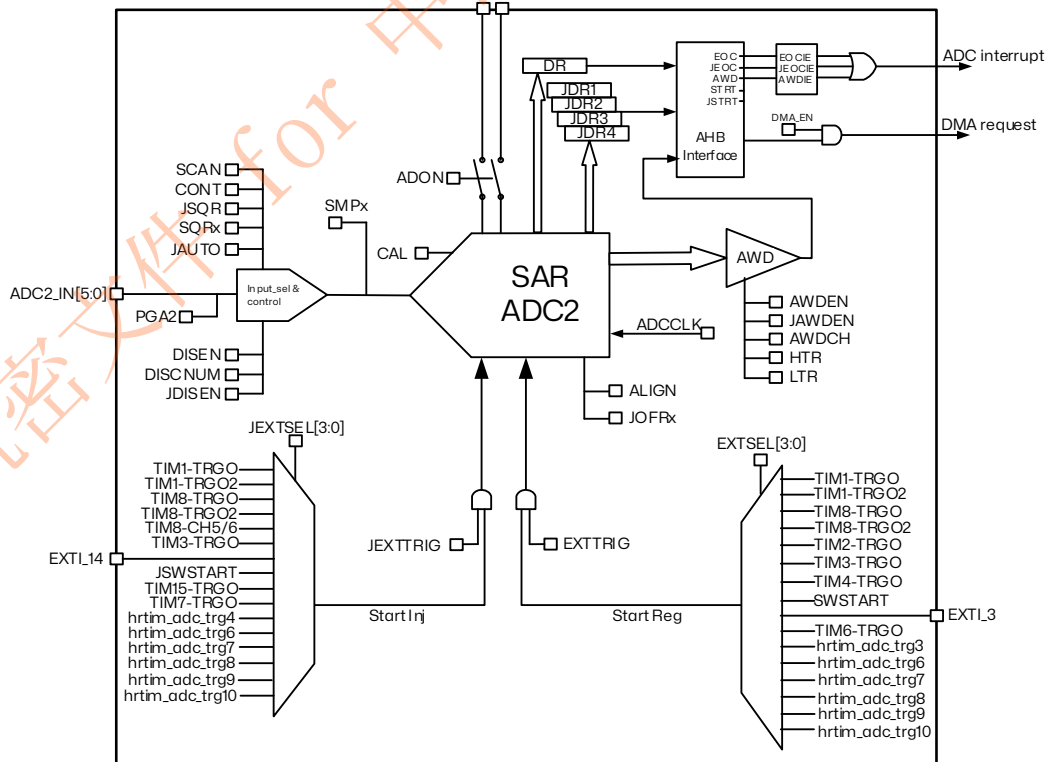


图 16.2 ADC2 框图

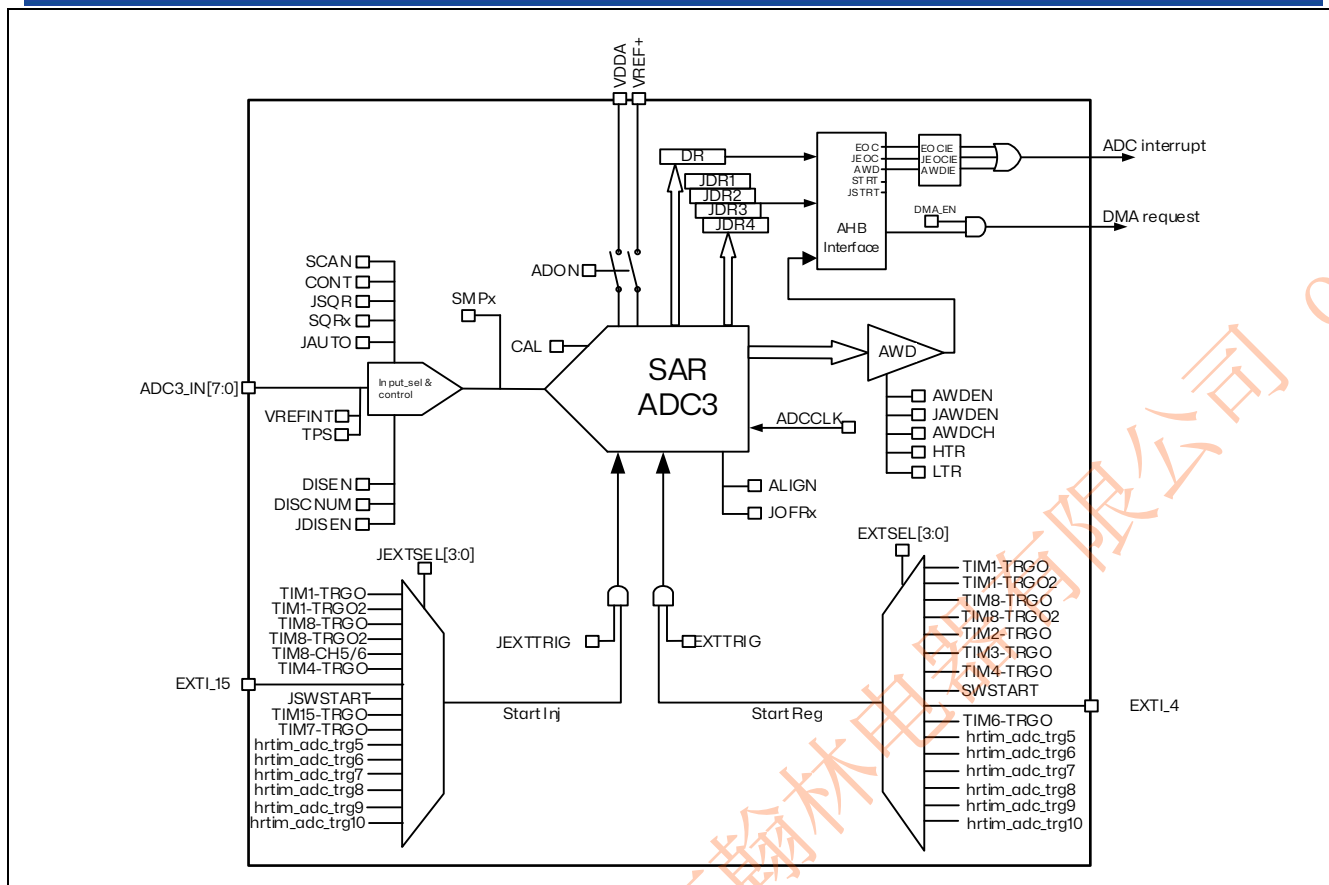


图 16.3 ADC3 框图

### 16.3.1 ADC 引脚/通道

表 16.1 ADC 引脚

| 引脚           | 信号类型       | 注解                                     |
|--------------|------------|----------------------------------------|
| VREF+        | 输入, 模拟参考正极 | ADC 的正极参考电压, $2V \leq VREF+ \leq VDDA$ |
| VDDA         | 输入, 模拟电源   | 等效于 VDD 的模拟电源, $VDDA = VDD$            |
| VREF-        | 输入, 模拟参考负极 | ADC 使用的低端/负极参考电压, $VREF- = VSSA$       |
| VSSA         | 输入, 模拟电源地  | 等效于 VSS 的模拟电源地                         |
| ADCx_IN[m:0] | 模拟输入信号     | 模拟输入通道                                 |

表 16.2 ADC1 通道

| 通道编号 | 模拟信号来源 | IO Type |
|------|--------|---------|
| 通道 0 | VSSA   | FAST_4M |
| 通道 1 | PC14   | SLOW_2M |
| 通道 2 | PC15   | SLOW_2M |
| 通道 3 | PC0    | SLOW_2M |
| 通道 4 | PA0    | SLOW_2M |
| 通道 5 | PA1    | SLOW_2M |
| 通道 6 | PA3    | SLOW_2M |

| 通道编号  | 模拟信号来源     | IO Type |
|-------|------------|---------|
| 通道 7  | PC1        | FAST_4M |
| 通道 8  | PC2        | FAST_4M |
| 通道 9  | PC3        | FAST_4M |
| 通道 10 | PA4        | FAST_4M |
| 通道 11 | OPAMP1_OUT | FAST_4M |
| 通道 12 | OPAMP2_OUT | FAST_4M |
| 通道 13 | PC13       | SLOW_2M |
| 通道 14 | 保留         | SLOW_2M |
| 通道 15 | VREF       | SLOW_2M |

表 16.3 ADC2 通道

| 通道编号  | 模拟信号来源     | IO Type |
|-------|------------|---------|
| 通道 0  | VSSA       | FAST_4M |
| 通道 1  | PC4        | SLOW_2M |
| 通道 2  | PC5        | SLOW_2M |
| 通道 3  | PE9        | SLOW_2M |
| 通道 4  | PE10       | SLOW_2M |
| 通道 5  | PE13       | SLOW_2M |
| 通道 6  | 保留         | SLOW_2M |
| 通道 7  | PA7        | FAST_4M |
| 通道 8  | PB0        | FAST_4M |
| 通道 9  | OPAMP1_OUT | FAST_4M |
| 通道 10 | OPAMP2_OUT | FAST_4M |
| 通道 11 | OPAMP3_OUT | FAST_4M |
| 通道 12 | OPAMP4_OUT | FAST_4M |
| 通道 13 | 保留         | SLOW_2M |
| 通道 14 | 保留         | SLOW_2M |
| 通道 15 | TPS        | SLOW_2M |

表 16.4 ADC3 通道

| 通道编号 | 模拟信号来源 | IO Type |
|------|--------|---------|
| 通道 0 | VSSA   | FAST_4M |
| 通道 1 | PE14   | SLOW_2M |
| 通道 2 | PE15   | SLOW_2M |
| 通道 3 | PB10   | SLOW_2M |
| 通道 4 | PB11   | SLOW_2M |
| 通道 5 | PB13   | SLOW_2M |
| 通道 6 | PB14   | SLOW_2M |
| 通道 7 | PD8    | FAST_4M |
| 通道 8 | PD9    | FAST_4M |

| 通道编号  | 模拟信号来源     | IO Type |
|-------|------------|---------|
| 通道 9  | PD10       | FAST_4M |
| 通道 10 | PB12       | FAST_4M |
| 通道 11 | OPAMP3_OUT | FAST_4M |
| 通道 12 | OPAMP4_OUT | FAST_4M |
| 通道 13 | VREFINT    | SLOW_2M |

注: PGA 输出公式:  $gain * (V_P - V_N) + V_{CC}/2$ 。

### 16.3.2 ADC 开关控制

通过设置 ADC\_CR2 寄存器的 ADON 位可给 ADC 上电。当第一次设置 ADON 位时, 它将 ADC 从断电状态下唤醒。

ADC 上电延迟一段时间后 ( $t_{STAB}$ ), 再次设置 ADON 位时开始进行转换。

通过清除 ADON 位可以停止转换, 并将 ADC 置于断电模式。

### 16.3.3 ADC 时钟

由时钟控制器提供的 ADCCLK 时钟可同步 PLLPCLK 或 SYSCLK。RCC 控制器为 ADC 时钟提供一个可选时钟分频。

### 16.3.4 通道选择

RX32G410 每个 ADC 有多路通道。可以把转换组织成两组: 规则组和注入组。在任意多个通道上以任意顺序进行的一系列转换构成成组转换。

- **规则组:** 由多达 8 个转换组成。规则通道和它们的转换顺序在 ADC\_SQRx 寄存器中选择。规则组中转换的总数应写入 ADC\_SQR1 寄存器的 L[2:0]位中。
- **注入组:** 由多达 4 个转换组成。注入通道和它们的转换顺序在 ADC\_JSQR 寄存器中选择。注入组里的转换总数目应写入 ADC\_JSQR 寄存器的 JL[1:0]位中。

如果 ADC\_SQRx 或 ADC\_JSQR 寄存器在转换期间被更改, 当前的转换被清除, 一个新的启动脉冲将发送到 ADC 以转换新选择的组。

### 16.3.5 单次转换模式

单次转换模式下, ADC 只执行一次转换。该模式既可通过设置 ADC\_CR2 寄存器的 ADON 位 (只适用于规则通道) 启动也可通过外部触发启动 (适用于规则通道或注入通道), 这时 CONT 位为 0。一旦选择通道的转换完成:

一旦选择通道的转换完成:

- 如果一个规则通道被转换:
    - 转换数据被储存在对应的 16 位 ADC\_DRx ( $x=1..8$ ) 寄存器中
    - EOC (转换结束) 标志被置位
    - 如果设置了 EOCIE 则产生中断
  - 如果一个注入通道被转换:
    - 转换数据被储存在 16 位的 ADC\_JDR1 寄存器中
    - EOC / JEOC (注入转换结束) 标志被设置
- EOC\_OPT 可选择注入转换结束
1. EOC/JEOC
  2. JEOC



– 如果设置了 JEOPIC 位则产生中断  
然后 ADC 停止。

### 16.3.6 连续转换模式

在连续转换模式中，当前面 ADC 转换一结束马上就启动另一次转换。此模式可通过外部触发启动或通过设置 ADC\_CR2 寄存器上的 ADON 位启动，此时 CONT 位是 1。

每个转换后：

- 如果一个规则通道被转换：
  - 转换数据被储存在 16 位 ADC\_DR 寄存器中
  - EOC（转换结束）标志被置位
  - 如果设置了 EOPIC 则产生中断

### 16.3.7 时序图

如下图所示，ADC 在开始精确转换前需要一个稳定时间  $t_{STAB}$ 。在开始 ADC 转换和 14 个时钟周期后，EOC 标志被设置，16 位 ADC 数据寄存器包含转换的结果。

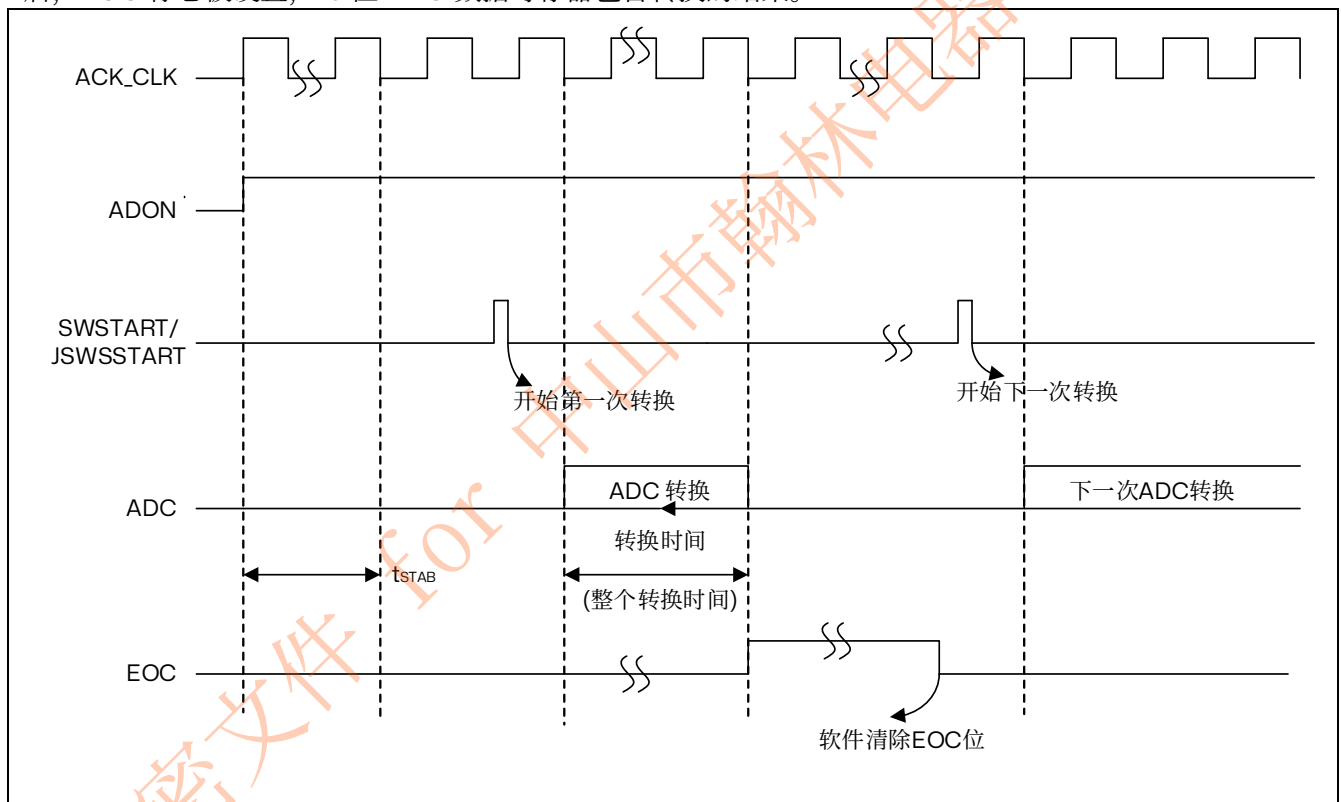


图 16.4 ADC 转换时序图

### 16.3.8 模拟看门狗

如果被 ADC 转换的模拟电压低于低阈值或高于高阈值，AWD 模拟看门狗状态位被设置。阈值位于 ADC\_HTR 和 ADC\_LTR 寄存器的最低 12 个有效位中。通过设置 ADC\_CR1 寄存器的 AWDIE 位以允许产生相应中断。

阈值独立于由 ADC\_CR2 寄存器上的 ALIGN 位选择的数据对齐模式。比较是在对齐之前完成的。

通过配置 ADC\_CR1 寄存器，模拟看门狗可以作用于 1 个或多个通道。

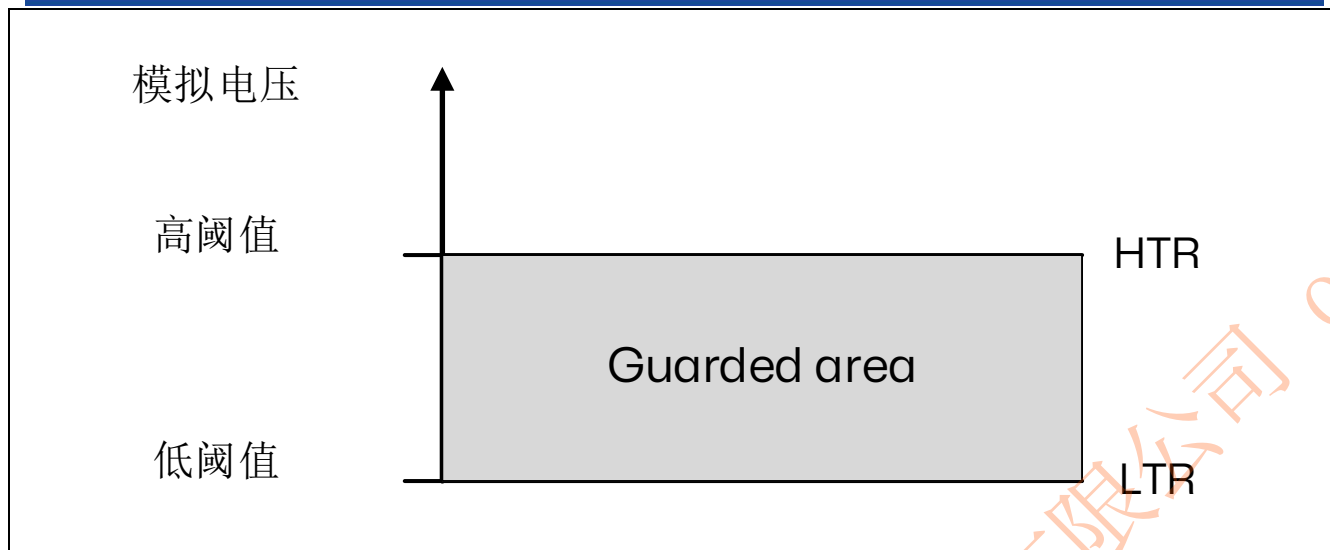


图 16.5 模拟看门狗警戒区

表 16.5 模拟看门狗通道选择

| 模拟看门狗警戒通道                  | ADC_CR1 寄存器控制位 |         |          |
|----------------------------|----------------|---------|----------|
|                            | AWDSGL 位       | AWDEN 位 | JAWDEN 位 |
| 无                          | 任意值            | 0       | 0        |
| 所有注入通道                     | 0              | 0       | 1        |
| 所有规则通道                     | 0              | 1       | 0        |
| 所有注入和规则通道                  | 0              | 1       | 1        |
| 单一的 <sup>(1)</sup> 注入通道    | 1              | 0       | 1        |
| 单一的 <sup>(1)</sup> 规则通道    | 1              | 1       | 0        |
| 单一的 <sup>(1)</sup> 注入或规则通道 | 1              | 1       | 1        |

1. 由 AWDCH[4:0]位选择

### 16.3.9 扫描模式

此模式用来扫描一组模拟通道。

扫描模式可通过设置 ADC\_CR1 寄存器的 SCAN 位来选择。一旦这个位被设置，ADC 扫描所有被 ADC\_SQRX 寄存器（对规则通道）或 ADC\_JSQR（对注入通道）选中的所有通道。在每个组的每个通道上执行单次转换。在每个转换结束时，同一组的下一个通道被自动转换。如果设置了 CONT 位，转换不会在选择组的最后一个通道上停止，而是再次从选择组的第一个通道继续转换。如果设置了 DMA 位，在每个转换后，DMA 控制器把规则组通道的转换数据传输到 SRAM 中。而注入通道转换的数据总是存储在 ADC\_JDRx 寄存器中。

### 16.3.10 外部触发转换

#### 触发注入

清除 ADC\_CR1 寄存器的 JAUTO 位，并且设置 SCAN 位，即可使用触发注入功能。

1. 利用外部触发或通过设置 ADC\_CR2 寄存器的 ADON 位，启动一组规则通道的转换。
2. 如果在规则通道转换期间产生一外部注入触发，当前转换被复位，注入通道序列被以单次扫描方式进行转换。

- 然后，恢复上次被中断的规则组通道转换。如果在注入转换期间产生一规则事件（前一次的规则序列已经完成），注入转换不会被中断，但是规则序列将在注入序列结束后被执行。

注：当使用触发的注入转换时，必须保证触发事件的间隔长于注入序列。例如：序列长度为28个ADC时钟周期（即2个具有1.5个时钟间隔采样时间的转换），触发之间最小的间隔必须是29个ADC时钟周期。

### 自动注入

如果设置了JAUTO位，在规则组通道之后，注入组通道被自动转换。这可以用来转换在ADC\_SQRx和ADC\_JSQR寄存器中设置的多至12个转换序列。

在此模式里，必须禁止注入通道的外部触发。

如果除JAUTO位外还设置了CONT位，规则通道至注入通道的转换序列被连续执行。

当从规则转换切换到注入序列或从注入转换切换到规则序列时，会自动插入2个ADC时钟间隔的延迟。

注：不可能同时使用自动注入和间断模式。

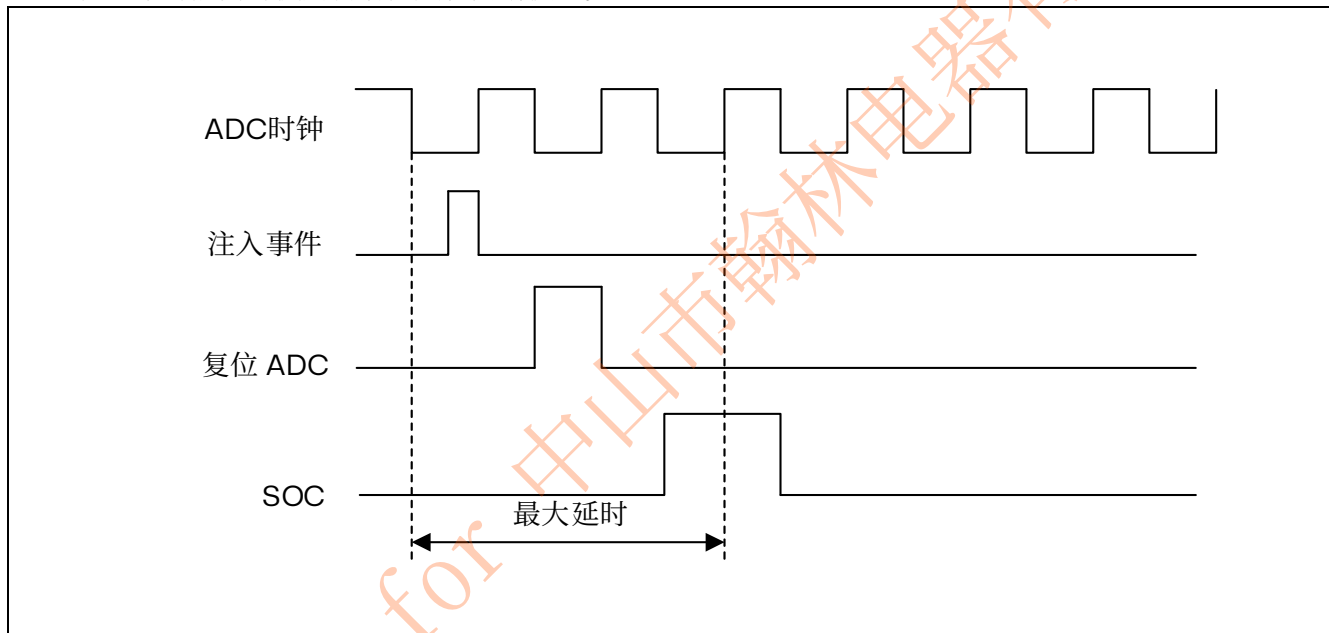


图 16.6 注入转换延迟

### 16.3.11 间断模式

#### 规则组

此模式通过设置ADC\_CR1寄存器上的DISCEN位激活。它可以用来执行一个短序列的n次转换（ $n \leq 8$ ），此转换是ADC\_SQRx寄存器所选择的转换序列的一部分。数值n由ADC\_CR1寄存器的DISCNUM[2:0]位给出。

一个外部触发信号可以启动ADC\_SQRx寄存器中描述的下一轮n次转换，直到此序列所有的转换完成为止。总的序列长度由ADC\_SQR1寄存器的L[3:0]定义。

举例：

$n = 3$ ，被转换通道 = 0、1、2、3、6、7、9、10

第一次触发：转换的序列为 0、1、2

第二次触发：转换的序列为 3、6、7

第三次触发：转换的序列为 9、10，并产生 EOC 事件

第四次触发：转换的序列为 0、1、2

注意：当以间断模式转换一个规则组时，转换序列结束后不自动从头开始。

当所有子组被转换完成，下一次触发启动第一个子组的转换。在上面的例子中，第四次触发重新转换第一子组的通道 0、1 和 2。

## 注入组

此模式通过设置 ADC\_CR1 寄存器的 JDISCEN 位激活。在一个外部触发事件后，该模式按通道顺序逐个转换 ADC\_JSQR 寄存器中选择的序列。

一个外部触发信号可以启动 ADC\_JSQR 寄存器选择的下一个通道序列的转换，直到序列中所有的转换完成为止。总的序列长度由 ADC\_JSQR 寄存器的 JL[1:0] 位定义。

例子：

n=1，被转换的通道 = 1、2、3

第一次触发：通道 1 被转换

第二次触发：通道 2 被转换

第三次触发：通道 3 被转换，并且产生 EOC 和 JEOP 事件

第四次触发：通道 1 被转换

注意：当完成所有注入通道转换，下个触发启动第 1 个注入通道的转换。在上述例子中，第四个触发重新转换第 1 个注入通道 1。

不能同时使用自动注入和间断模式。

## 16.4 校准

ADC 有一个内置自校准模式。校准可大幅减小因内部电容器组的变化而造成的精度误差。在校准期间，在每个电容器上都会计算出一个误差修正码（数字值），这个码用于消除在随后的转换中每个电容器上产生的误差。

通过设置 ADC\_CR2 寄存器的 CAL 位启动校准。一旦校准结束，CAL 位被硬件复位，可以开始正常转换。建议在上电时执行一次 ADC 校准。校准阶段结束后，校准码储存在 CAL\_DR 中。

注意：建议在每次上电后执行一次校准。

启动校准前，ADC 必须处于上电状态 (ADON = '1') 超过至少两个 ADC 时钟周期。

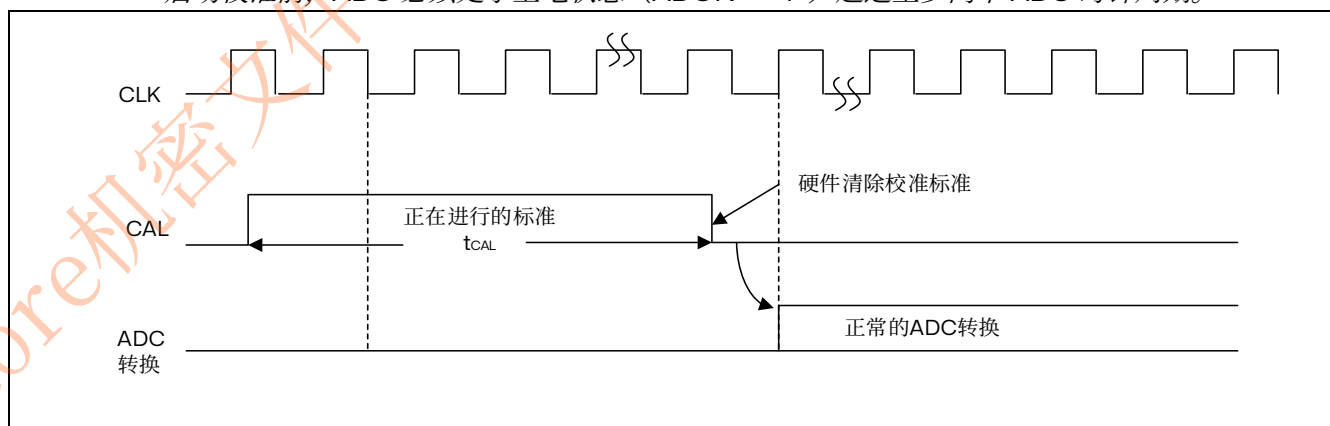


图 16.7 校准时序图

## 16.5 数据对齐

ADC\_CR2 寄存器中的 ALIGN 位选择转换后数据储存的对齐方式。数据可以左对齐或右对齐。

注入组通道转换的数据值已经减去了在 ADC\_JOFRx 寄存器中定义的偏移量，因此结果可以是一个负值。SEXT 位是扩展的符号值。

对于规则组通道，不需减去偏移值，因此只有 12 个位有效。

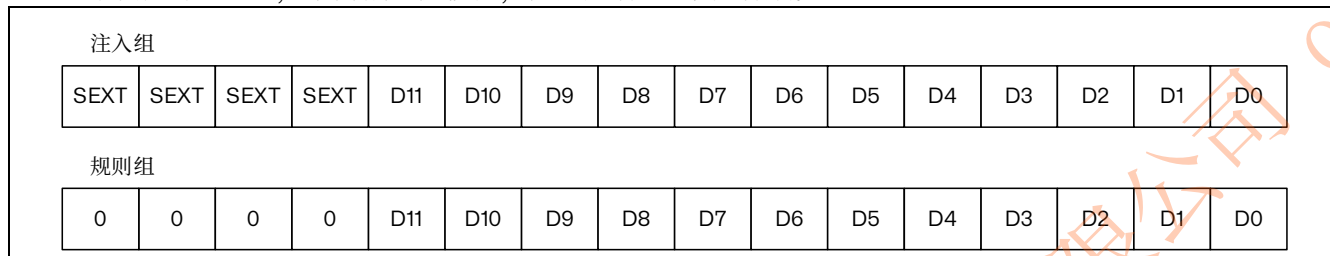


图 16.8 数据右对齐

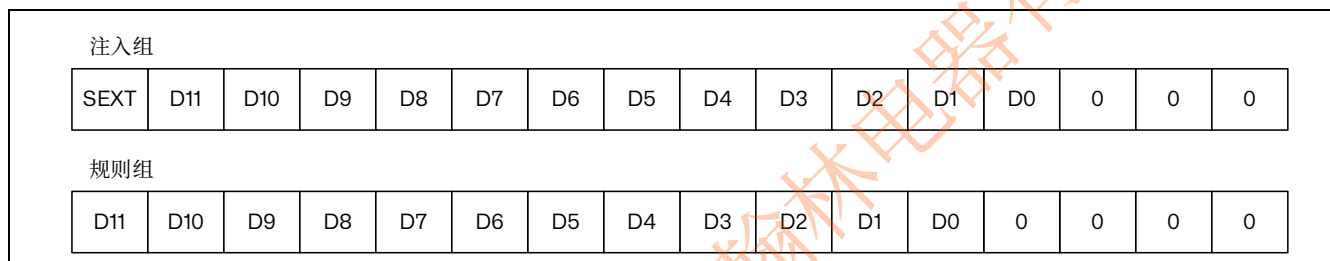


图 16.9 数据左对齐

## 16.6 可编程的通道采样时间

ADC 使用若干个 ADC\_CLK 周期对输入电压采样，采样周期数目可以通过 ADC\_SMPR 的 SMP[2:0] 位更改。每个通道可以分别用不同的时间采样。

总转换时间如下计算：

$$T_{CONV} = \text{采样时间} + 12.5 \text{ 个周期}$$

例如：

当 ADCCLK = 64MHz，采样时间位 3.5 周期

$$T_{CONV} = 3.5 + 12.5 = 16 \text{ 个周期} = 250\text{ns}$$

## 16.7 外部触发转换

转换可以由外部事件触发（例如定时器捕获，EXTI 线）。如果设置了 EXTTRIG/JEXTTRIG 控制位，则外部事件就能够触发转换。EXTSEL[3:0]和 JEXTSEL[3:0]控制位允许应用程序选择所有可能的事件中的某一个，可以触发规则和注入组的采样。

注意：当外部触发信号被选为 ADC 规则或注入转换时，只有它的上升沿可以启动转换。

表 16.6 ADC1 规则通道的外部触发信号

| 触发源        | 类型        | EXTSEL[3:0] |
|------------|-----------|-------------|
| TIM1_TRGO  | 片上定时器内部信号 | 0000        |
| TIM1_TRGO2 |           | 0001        |
| TIM8_TRGO  |           | 0010        |

|                 |           |      |
|-----------------|-----------|------|
| TIM8_TRGO2      |           | 0011 |
| TIM2-TRGO       |           | 0100 |
| TIM3-TRGO       |           | 0101 |
| TIM4-TRGO       |           | 0110 |
| SWSTART         | 软件控制位     | 0111 |
| EXTI_2          | 外部引脚信号    | 1000 |
| TIM6-TRGO       | 片上定时器内部信号 | 1001 |
| hrtim_adc_trg1  |           | 1010 |
| hrtim_adc_trg6  |           | 1011 |
| hrtim_adc_trg7  |           | 1100 |
| hrtim_adc_trg8  |           | 1101 |
| hrtim_adc_trg9  |           | 1110 |
| hrtim_adc_trg10 |           | 1111 |

表 16.7 ADC2 规则通道的外部触发信号

| 触发源             | 类型        | EXTSEL[3:0] |
|-----------------|-----------|-------------|
| TIM1_TRGO       | 片上定时器内部信号 | 0000        |
| TIM1_TRGO2      |           | 0001        |
| TIM8_TRGO       |           | 0010        |
| TIM8_TRGO2      |           | 0011        |
| TIM2-TRGO       |           | 0100        |
| TIM3-TRGO       |           | 0101        |
| TIM4-TRGO       |           | 0110        |
| SWSTART         | 软件控制位     | 0111        |
| EXTI_3          | 外部引脚信号    | 1000        |
| TIM6-TRGO       | 片上定时器内部信号 | 1001        |
| hrtim_adc_trg3  |           | 1010        |
| hrtim_adc_trg6  |           | 1011        |
| hrtim_adc_trg7  |           | 1100        |
| hrtim_adc_trg8  |           | 1101        |
| hrtim_adc_trg9  |           | 1110        |
| hrtim_adc_trg10 |           | 1111        |

表 16.8 ADC3 规则通道的外部触发信号

| 触发源        | 类型        | EXTSEL[3:0] |
|------------|-----------|-------------|
| TIM1_TRGO  | 片上定时器内部信号 | 0000        |
| TIM1_TRGO2 |           | 0001        |
| TIM8_TRGO  |           | 0010        |
| TIM8_TRGO2 |           | 0011        |
| TIM2-TRGO  |           | 0100        |
| TIM3-TRGO  |           | 0101        |

|                 |           |      |
|-----------------|-----------|------|
| TIM4-TRGO       |           | 0110 |
| SWSTART         | 软件控制位     | 0111 |
| EXTI_4          | 外部引脚信号    | 1000 |
| TIM6-TRGO       | 片上定时器内部信号 | 1001 |
| hrtim_adc_trg5  |           | 1010 |
| hrtim_adc_trg6  |           | 1011 |
| hrtim_adc_trg7  |           | 1100 |
| hrtim_adc_trg8  |           | 1101 |
| hrtim_adc_trg9  |           | 1110 |
| hrtim_adc_trg10 |           | 1111 |

表 16.9 ADC1 注入通道的外部触发信号

| 触发源             | 类型        | EXTSEL[3:0] |
|-----------------|-----------|-------------|
| TIM1-TRGO       | 片上定时器内部信号 | 0000        |
| TIM1-TRGO2      |           | 0001        |
| TIM8-TRGO       |           | 0010        |
| TIM8-TRGO2      |           | 0011        |
| TIM1-CH5/6      |           | 0100        |
| TIM2-TRGO       |           | 0101        |
| EXTI 线 13       | 外部引脚信号    | 0110        |
| JSWSTART        | 软件控制位     | 0111        |
| TIM15-TRGO      | 片上定时器内部信号 | 1000        |
| TIM7-TRGO       |           | 1001        |
| hrtim_adc_trg2  |           | 1010        |
| hrtim_adc_trg6  |           | 1011        |
| hrtim_adc_trg7  |           | 1100        |
| hrtim_adc_trg8  |           | 1101        |
| hrtim_adc_trg9  |           | 1110        |
| hrtim_adc_trg10 |           | 1111        |

表 16.10 ADC2 注入通道的外部触发信号

| 触发源        | 类型        | EXTSEL[3:0] |
|------------|-----------|-------------|
| TIM1-TRGO  | 片上定时器内部信号 | 0000        |
| TIM1-TRGO2 |           | 0001        |
| TIM8-TRGO  |           | 0010        |
| TIM8-TRGO2 |           | 0011        |
| TIM8-CH5/6 |           | 0100        |
| TIM3-TRGO  |           | 0101        |
| EXTI 线 14  | 外部引脚信号    | 0110        |
| JSWSTART   | 软件控制位     | 0111        |
| TIM15-TRGO | 片上定时器内部信号 | 1000        |



|                 |  |      |
|-----------------|--|------|
| TIM7-TRGO       |  | 1001 |
| hrtim_adc_trg4  |  | 1010 |
| hrtim_adc_trg6  |  | 1011 |
| hrtim_adc_trg7  |  | 1100 |
| hrtim_adc_trg8  |  | 1101 |
| hrtim_adc_trg9  |  | 1110 |
| hrtim_adc_trg10 |  | 1111 |

表 16.11 ADC3 注入通道的外部触发信号

| 触发源             | 类型        | EXTSEL[3:0] |
|-----------------|-----------|-------------|
| TIM1-TRGO       | 片上定时器内部信号 | 0000        |
| TIM1-TRGO2      |           | 0001        |
| TIM8-TRGO       |           | 0010        |
| TIM8-TRGO2      |           | 0011        |
| TIM8-CH5/6      |           | 0100        |
| TIM4-TRGO       |           | 0101        |
| EXTI 线 15       | 外部引脚信号    | 0110        |
| JSWSTART        | 软件控制位     | 0111        |
| TIM15-TRGO      | 片上定时器内部信号 | 1000        |
| TIM7-TRGO       |           | 1001        |
| hrtim_adc_trg5  |           | 1010        |
| hrtim_adc_trg6  |           | 1011        |
| hrtim_adc_trg7  |           | 1100        |
| hrtim_adc_trg8  |           | 1101        |
| hrtim_adc_trg9  |           | 1110        |
| hrtim_adc_trg10 |           | 1111        |

软件触发事件可以通过对寄存器 ADC\_CR2 的 SWSTART 或 JSWSTART 位置 1 产生。

规则组的转换可以被注入触发打断。

## 16.8 DMA 请求

当多个规则通道转换完成时，转换数据被储存在对应的 16 位 ADC\_DRx (x=1..8) 寄存器中。

当需要使用 DMA 功能时，只有在规则通道的转换结束时才产生 DMA 请求，并将转换的数据从 ADC\_DRx (x=1..8) 寄存器传输到用户指定的目的地址。

## 16.9 双 ADC 模式

在双 ADC 模式里，根据 ADC1\_CR1 寄存器中 CR1\_DUALMOD[3:0]位所选的模式，转换的启动可以是 ADC1 主和 ADC2 从的交替触发或同步触发。

**注意：**在双 ADC 模式里，当转换配置成由外部事件触发时，用户必须将其设置成仅触发主 ADC，从 ADC 设置成软件触发，这样可以防止意外的触发从转换。但是，主和从 ADC 的外部触发必须同时被激活。

共有 6 种模式：

- 同步注入模式

- 同步规则模式
- 快速交叉模式
- 慢速交叉模式
- 交替触发模式
- 独立模式

双重组合模式：

- 同步注入模式 + 同步规则模式
- 同步规则模式 + 交替触发模式
- 同步注入模式 + 交叉模式

注意：在双ADC模式里，为了在主数据寄存器上读取从转换数据，必须使能DMA位，即使不使用DMA传输规则通道数据。

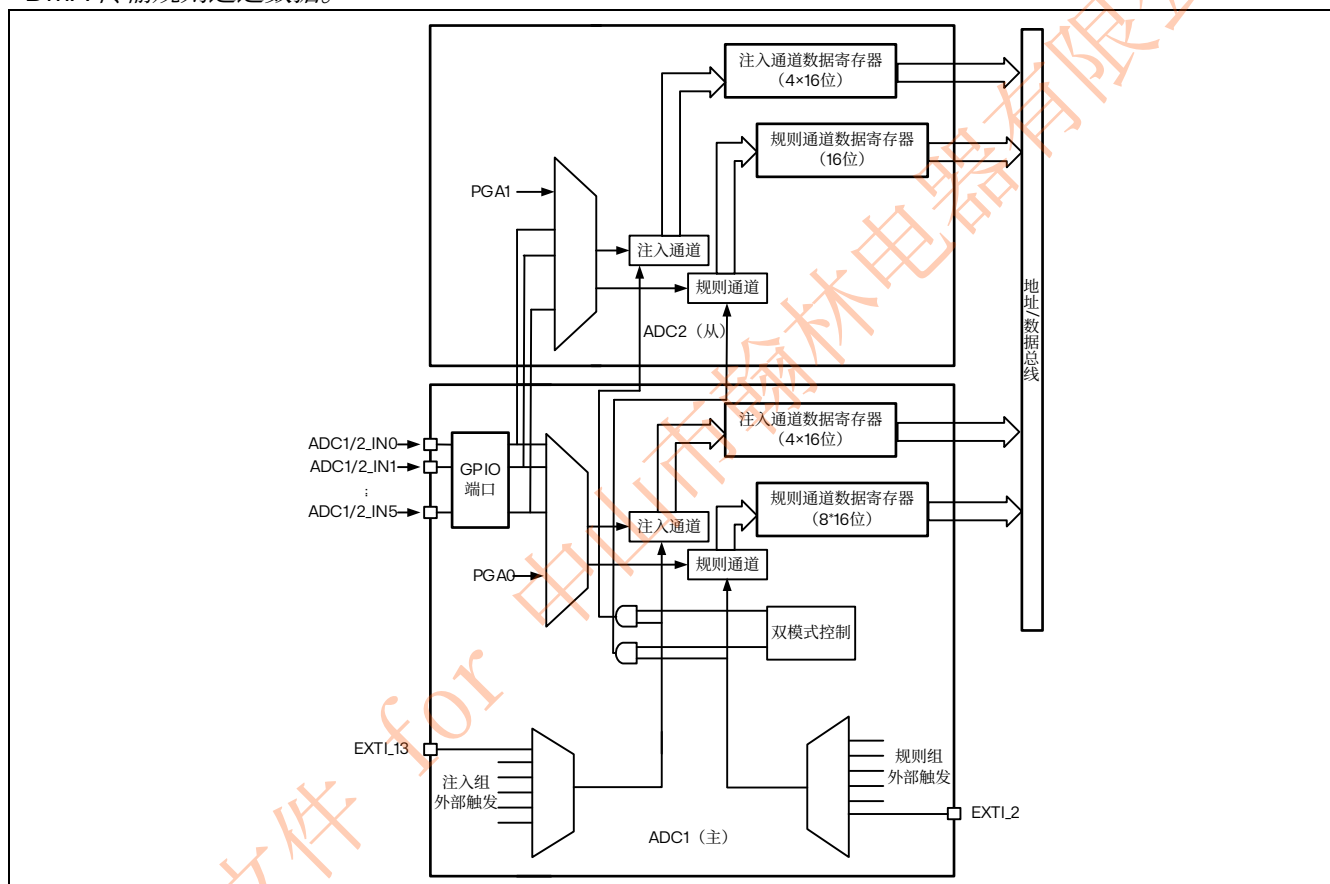


图 16.10 双 ADC 框图

### 16.9.1 同步注入模式

此模式转换一个注入通道组。外部触发来自ADC1的注入组多路开关（由ADC1\_CR2寄存器的JEXTSEL[3:0]选择），它同时给ADC2提供同步触发。

注意：不要在2个ADC上转换相同的通道（两个ADC在同一个通道上的采样时间不能重叠）。

在ADC1或ADC2的转换结束时：

- 转换的数据存储在每个ADC接口的ADC\_JDRx寄存器中。
- 当所有ADC1/ADC2注入通道都被转换时，产生JEOC中断（若任一ADC接口开放了中断）。

注意：在同步模式中，必须转换具有相同时间长度的序列，或保证触发的间隔比2个序列中较长的序

列长，否则当较长序列的转换还未完成时，具有较短序列的 ADC 转换可能会被重启。

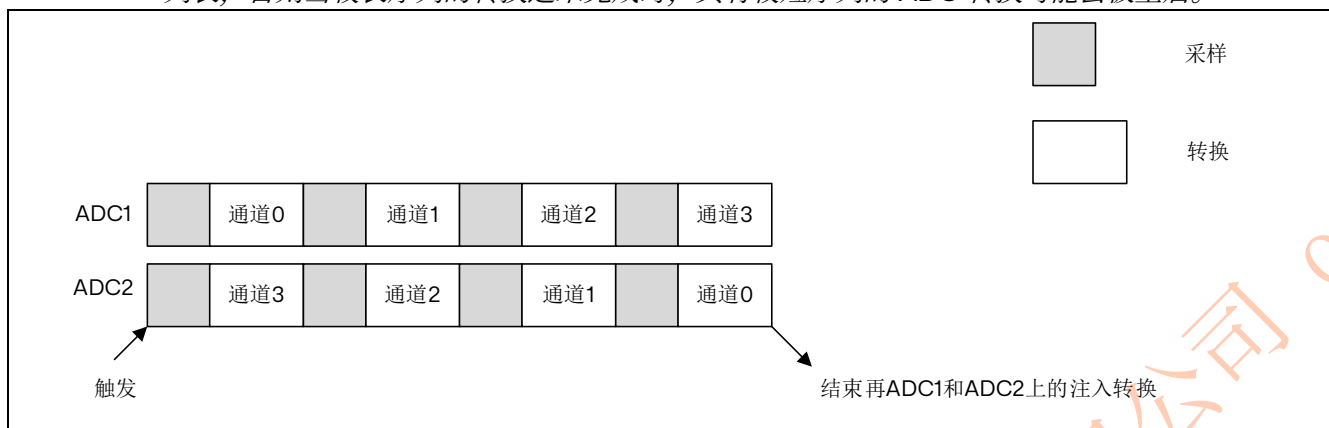


图 16.11 4 个通道的同步注入模式

### 16.9.2 同步规则模式

此模式在规则通道组上执行。外部触发来自 ADC1 的规则组多路开关（由 ADC1\_CR2 寄存器的 EXTSEL[2:0] 选择），它同时给 ADC2 提供同步触发。

**注意：**不要在 2 个 ADC 上转换相同的通道（两个 ADC 在同一个通道上的采样时间不能重叠）。

在 ADC1 或 ADC2 的转换结束时：

- 产生一个 32 位 DMA 传输请求（如果设置了 DMA 位），32 位的 ADC1\_DR 寄存器内容传输到 SRAM 中，它上半个字包含 ADC2 的转换数据，低半个字包含 ADC1 的转换数据。
- 当所有 ADC1/ADC2 规则通道都被转换完时，产生 EOC 中断（若任一 ADC 接口开放了中断）。

**注：**在同步规则模式中，必须转换具有相同时间长度的序列，或保证触发的间隔比 2 个序列中较长的序列长，否则当较长序列的转换还未完成时，具有较短序列的 ADC 转换可能会被重启。

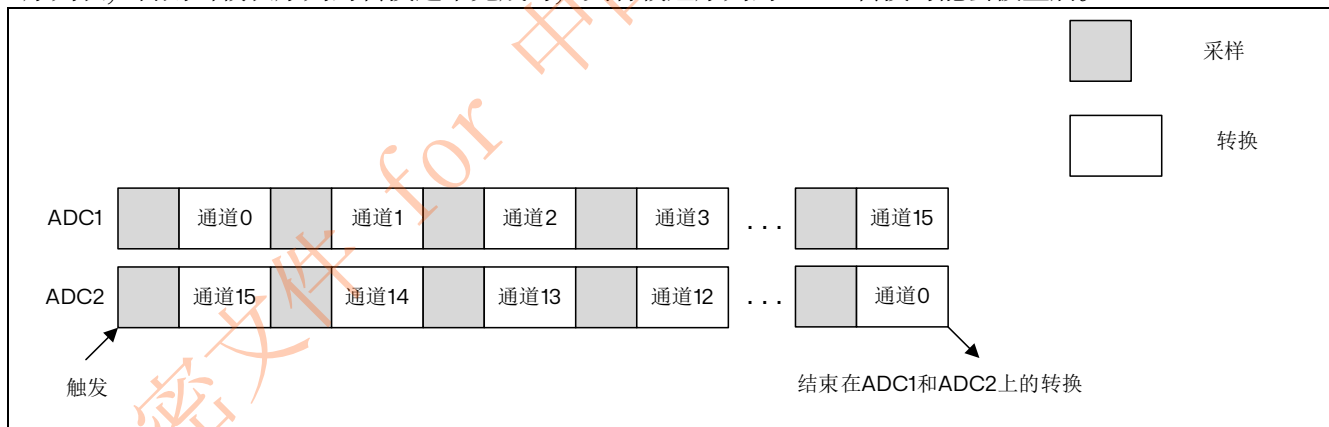


图 16.12 16 个通道的同步规则模式

### 16.9.3 快速交叉模式

此模式只适用于规则通道组。外部触发来自 ADC1 的规则通道多路开关。外部触发产生后：

- ADC2 立即启动
- ADC1 在延迟 7 个 ADC 时钟周期后启动

如果同时设置了 ADC1 和 ADC2 的 CONT 位，所选的两个 ADC 规则通道将被连续地转换。

ADC1 产生一个 EOC 中断后（由 EOCIE 使能），产生一个 32 位的 DMA 传输请求（如果设置了

DMA 位), ADC1\_DR 寄存器的 32 位数据被传输到 SRAM, ADC1\_DR 的上半个字包含 ADC2 的转换数据, 低半个字包含 ADC1 的转换数据。

注意: 最大允许采样时间<7 个 ADCCLK 周期, 避免 ADC1 和 ADC2 转换相同通道时发生两个采样周期的重叠。

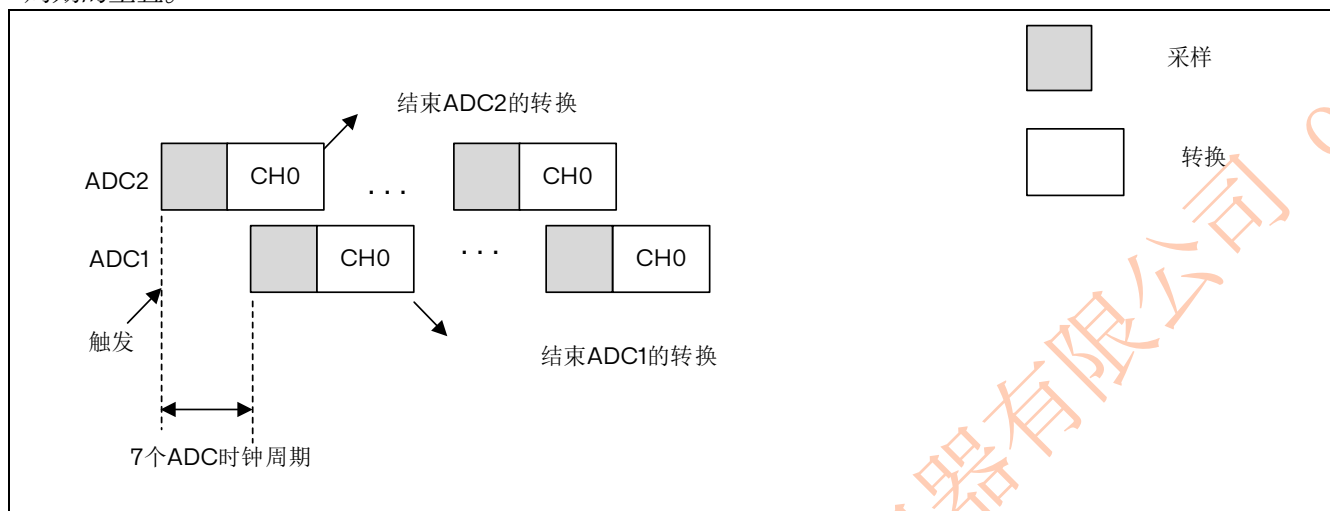


图 16.13 在 1 个通道上连续转换模式下的快速交叉模式

#### 16.9.4 慢速交叉模式

此模式只适用于规则通道组。外部触发来自 ADC1 的规则通道多路开关。外部触发产生后:

- ADC2 立即启动
- ADC1 在延迟 14 个 ADC 时钟周期后启动
- 在延迟第二次 14 个 ADC 周期后 ADC2 再次启动, 如此循环。

注意: 最大允许采样时间<14 个 ADCCLK 周期, 以避免和下个转换重叠。

ADC1 产生一个 EOC 中断后 (由 EOCIE 使能), 产生一个 32 位的 DMA 传输请求 (如果设置了 DMA 位), ADC1\_DR 寄存器的 32 位数据被传输到 SRAM, ADC1\_DR 的上半个字包含 ADC2 的转换数据, 低半个字包含 ADC1 的转换数据。

在 28 个 ADC 时钟周期后自动启动新的 ADC2 转换。

在这个模式下不能设置 CONT 位, 因为它将连续转换所选择的规则通道。

注意: 应用程序必须确保当使用交叉模式时, 不能有注入通道的外部触发产生。

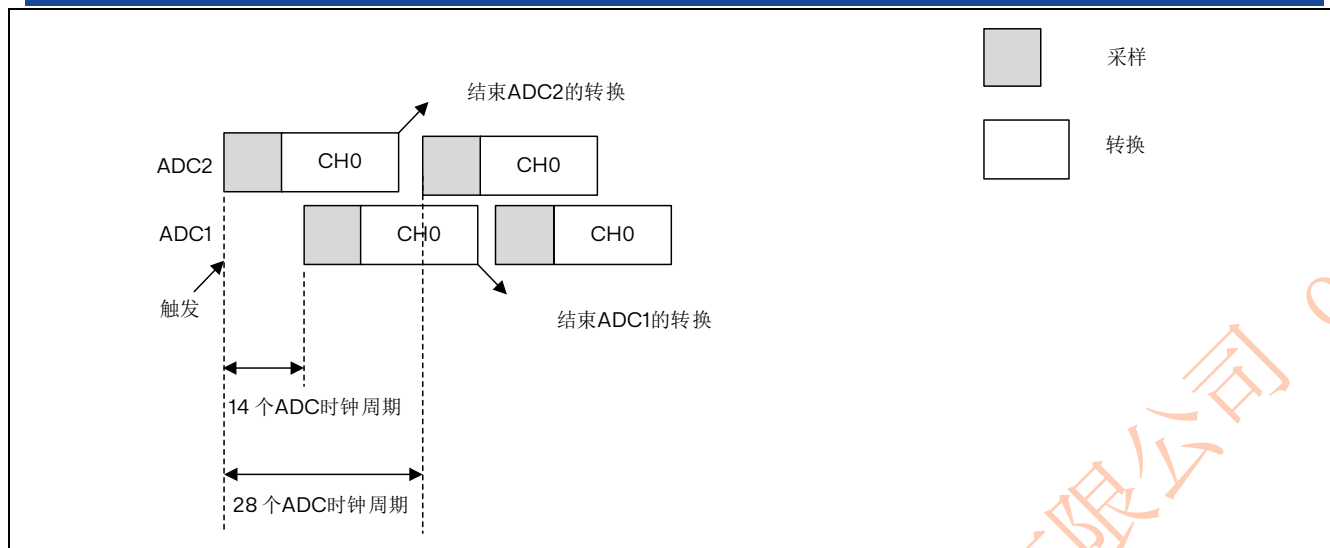


图 16.14 在 1 个通道上的慢速交叉模式

### 16.9.5 交替触发模式

此模式只适用于注入通道组。外部触发源来自 ADC1 的注入通道多路开关。

- 当第一个触发产生时，ADC1 上的所有注入组通道被转换。
- 当第二个触发到达时，ADC2 上的所有注入组通道被转换。
- 如此循环.....

如果允许产生 JEOC 中断，在所有 ADC1 注入组通道转换后产生一个 JEOC 中断。

如果允许产生 JEOC 中断，在所有 ADC2 注入组通道转换后产生一个 JEOC 中断。

当所有注入组通道都转换完后，如果又有另一个外部触发，交替触发处理从转换 ADC1 注入组通道重新开始。

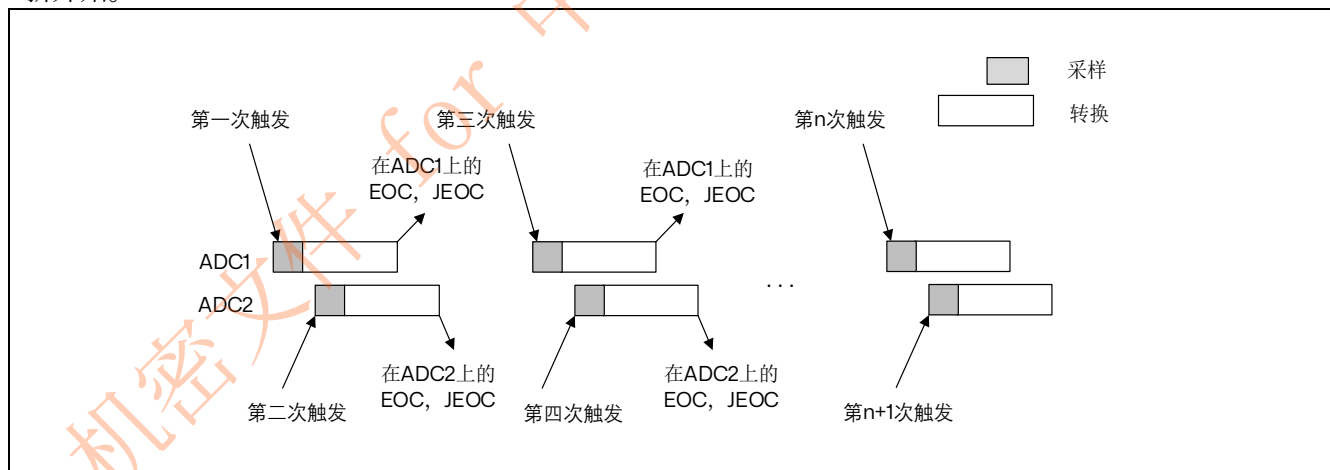


图 16.15 交替触发：每个 ADC1 的注入通道组

如果 ADC1 和 ADC2 上同时使用了注入间断模式：

- 当第一个触发产生时，ADC1 上的第一个注入通道被转换。
- 当第二个触发到达时，ADC2 上的第一个注入通道被转换。
- 如此循环.....

如果允许产生 JEOC 中断，在所有 ADC1 注入组通道转换后产生一个 JEOC 中断。

如果允许产生 JEOC 中断，在所有 ADC2 注入组通道转换后产生一个 JEOC 中断。  
当所有注入组通道都转换完后，如果又有另一个外部触发，则重新开始交替触发过程。

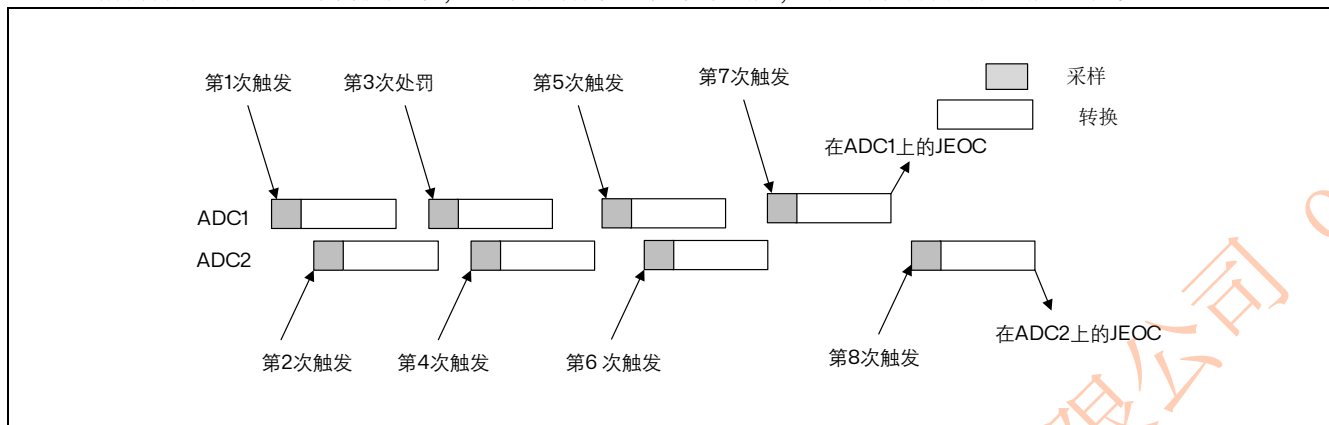


图 16.16 交替触发：在间断模式下每个 ADC 上的 4 个注入通道

### 16.9.6 混合的规则/注入同步模式

规则组同步转换可以被中断，以启动注入组的同步转换。

注：在混合的规则/注入同步模式中，必须转换具有相同时间长度的序列，或保证触发的间隔比 2 个序列中较长的序列长，否则当较长序列的转换还未完成时，具有较短序列的 ADC 转换可能会被重启。

### 16.9.7 混合的同步规则+交替触发模式

规则组同步转换可以被中断，以启动注入组交替触发转换。下图显示了一个规则同步转换被交替触发所中断。

注入交替转换在注入事件到达后立即启动。如果规则转换已经在运行，为了在注入转换后确保同步，所有的 ADC(主和从)的规则转换被停止，并在注入转换结束时同步恢复。

注：在混合的同步规则+交替触发模式中，必须转换具有相同时间长度的序列，或保证触发的间隔比 2 个序列中较长的序列长，否则当较长序列的转换还未完成时，具有较短序列的 ADC 转换可能会被重启。

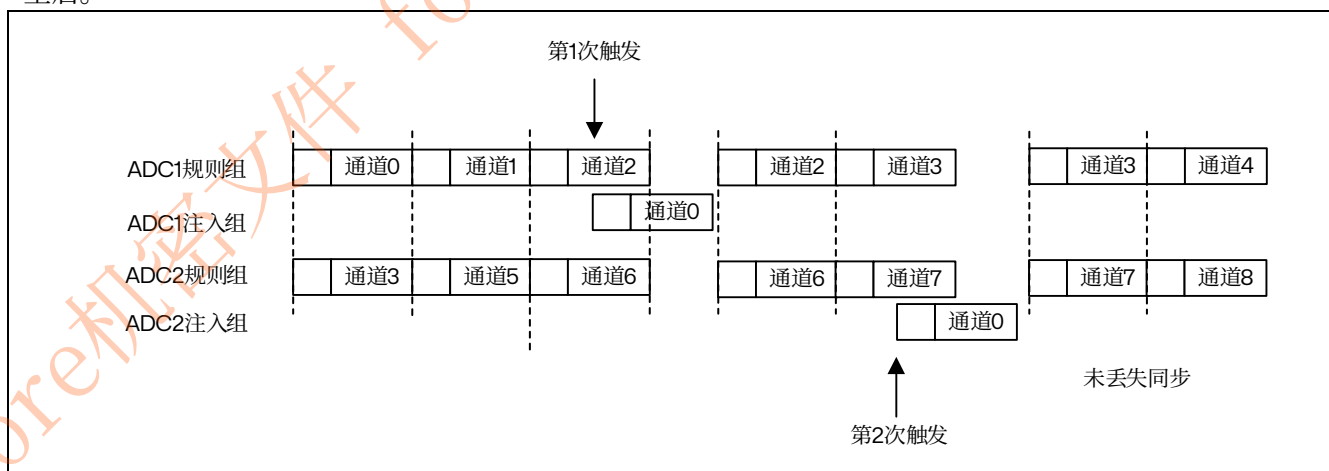


图 16.17 交替+规则同步

如果触发事件发生在一个中断了规则转换的注入转换期间，这个触发事件将被忽略。下图示出了这种情况的操作（第 2 个触发被忽略）。

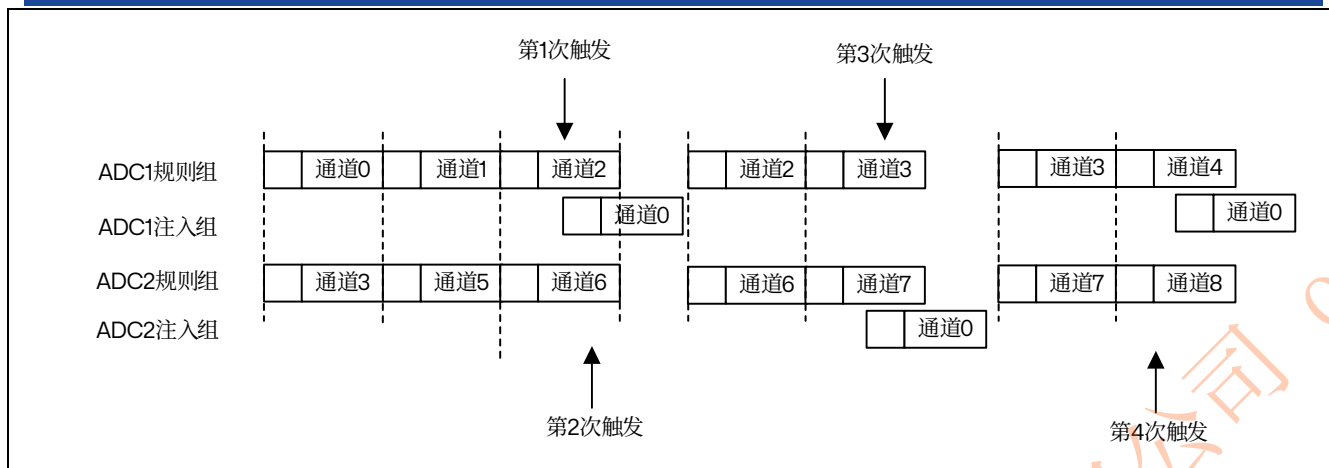


图 16.18 触发事件发生在注入转换期间

### 16.9.8 混合同步注入 + 交叉模式

一个注入事件可以中断一个交叉转换。这种情况下，交叉转换被中断，注入转换被启动，在注入序列转换结束时，交叉转换被恢复。下图是这种情况的一个例子。

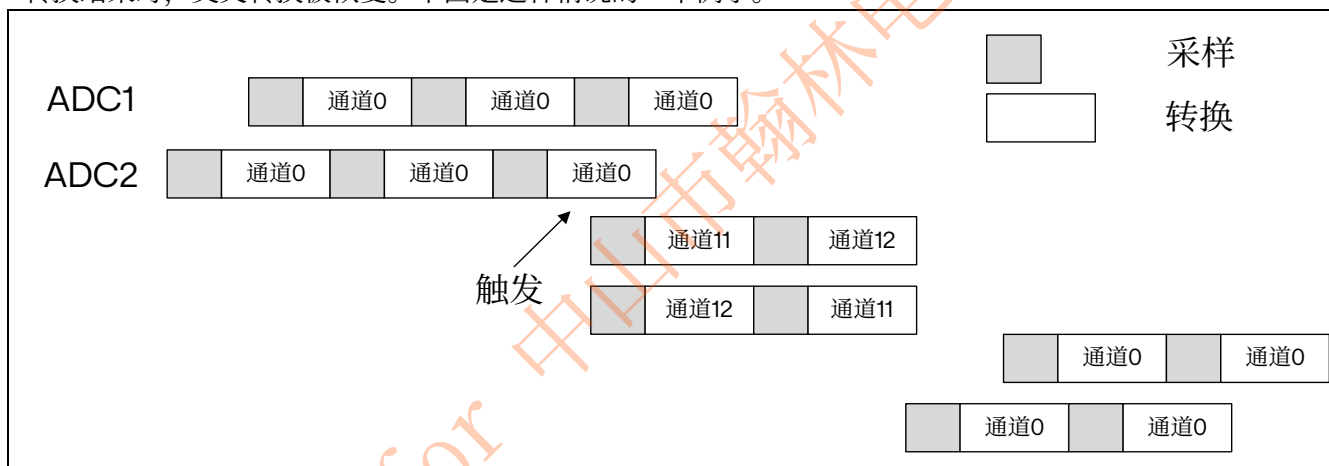


图 16.19 交叉的单通道转换被注入序列 CH11 和 CH12 中断

### 16.9.9 独立模式

此模式里，双 ADC 同步不工作，每个 ADC 接口独立工作。

### 16.10 温度传感器（TPS）

温度传感器可以用来测量器件周围的温度。

温度传感器在内部和 ADC2 的通道 15 相连，此通道把传感器输出的电压转换成数字值。

温度传感器输出电压随温度线性变化，由于生产过程中的变化，温度变化曲线的偏移在不同芯片上会有不同（最多相差 45°C）。

注意：必须设置 TSVREFE 位激活内部通道：ADC2\_IN15（温度传感器）和 ADC1\_IN15（V<sub>BG</sub>）的转换。

内部温度传感器更适合于检测温度的变化，而不是测量绝对的温度。如果需要测量精确的温度，应该使用一个外置的温度传感器。



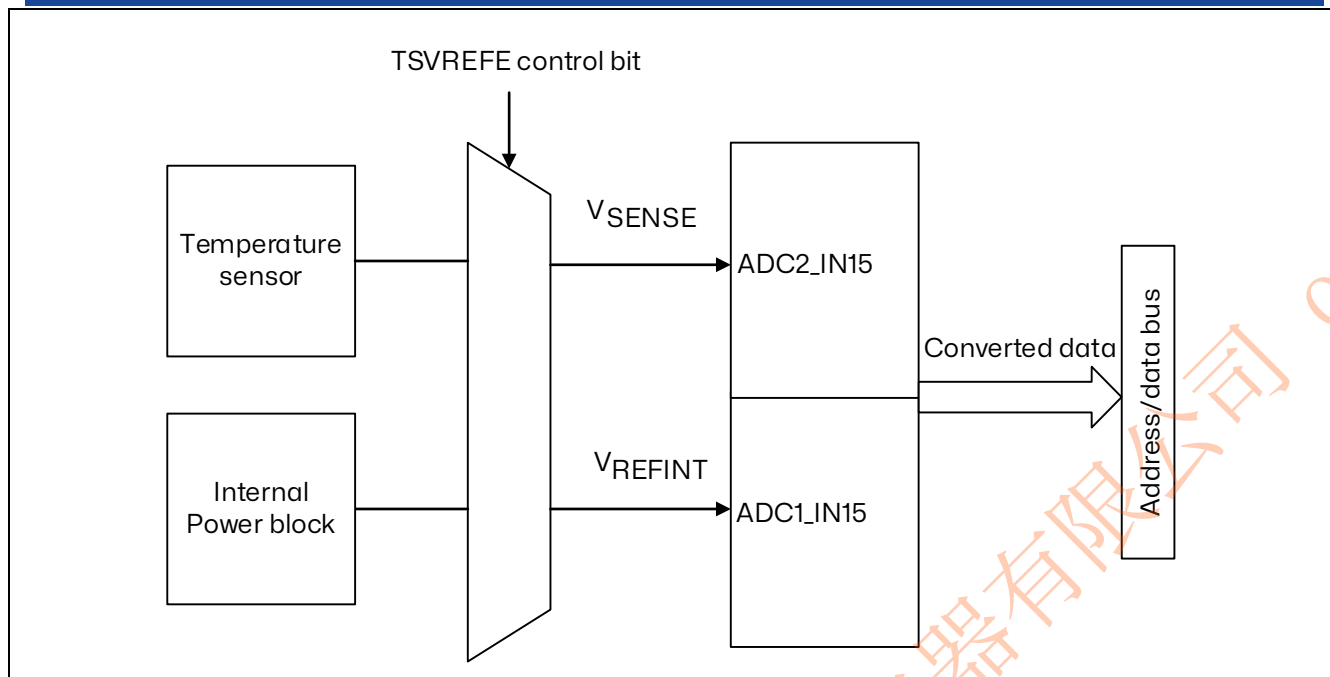


图 16.20 温度传感器和 VBG 通道框图

## 16.11 ADC 中断

规则和注入组转换结束时能产生中断，当模拟看门狗状态位被设置时也能产生中断。它们都有独立的中断使能位。

注意：ADC1 和 ADC2 的中断映射在同一个中断向量上，而 ADC3 的中断有自己的中断向量。

ADC\_SR 寄存器中有 2 个其他标志，但是它们没有相关联的中断：

- JSTRT（注入组通道转换的启动）
- STRT（规则组通道转换的启动）

表 16.12 ADC 中断

| 中断事件        | 事件标志 | 使能控制位  |
|-------------|------|--------|
| 规则组转换结束     | EOC  | EOCIE  |
| 注入组转换结束     | JEOC | JEOCIE |
| 设置了模拟看门狗状态位 | AWD  | AWDIE  |

## 16.12 ADC 寄存器描述

基地址：0x4900 0000 (ADC1), 0x4900 0400 (ADC2), 0x4900 0800 (ADC3)

### 16.12.1 ADC 状态寄存器 (ADC\_SR)

地址偏移：0x00

复位值：0x0000 0000

访问：无等待，仅支持字访问

|          |    |    |    |    |    |    |    |    |    |    |       |       |       |       |       |
|----------|----|----|----|----|----|----|----|----|----|----|-------|-------|-------|-------|-------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20    | 19    | 18    | 17    | 16    |
| Reserved |    |    |    |    |    |    |    |    |    |    |       |       |       |       |       |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4     | 3     | 2     | 1     | 0     |
| Reserved |    |    |    |    |    |    |    |    |    |    | STRT  | JSTRT | JEOC  | EOC   | AWD   |
|          |    |    |    |    |    |    |    |    |    |    | rc_w0 | rc_w0 | rc_w0 | rc_w0 | rc_w0 |

Bits 31:5 保留，必须保持为复位值。

Bit 4 **STRT**：规则通道开始位

该位由硬件在规则通道转换开始时设置，由软件清除。

0：规则通道转换未开始

1：规则通道转换已开始

Bit 3 **JSTRT**：注入通道开始位

该位由硬件在注入通道转换开始时设置，由软件清除。

0：注入通道转换未开始

1：注入通道转换已开始

Bit 2 **JEOC**：注入通道转换结束位

该位由硬件在所有注入通道组转换结束时设置，由软件清除。

0：转换未完成

1：转换完成

Bit 1 **EOC**：转换结束位

该位由硬件在（规则或注入）通道组转换结束时设置，由软件清除或由读取 ADC\_DR 时清除。

0：转换未完成

1：转换完成

Bit 0 **AWD**：模拟看门狗标志位

该位由硬件在转换的电压值超出了 ADC\_LTR 和 ADC\_HTR 寄存器定义的范围时设置，由软件清除。

0：没有发生模拟看门狗事件

1：发生模拟看门狗事件

## 16.12.2 ADC 控制寄存器 1 (ADC\_CR1)

地址偏移: 0x04

复位值: 0x0000 0000

访问: 无等待, 仅支持字访问

| 31           | 30 | 29 | 28          | 27         | 26           | 25         | 24   | 23         | 22         | 21       | 20         | 19           | 18 | 17 | 16 |
|--------------|----|----|-------------|------------|--------------|------------|------|------------|------------|----------|------------|--------------|----|----|----|
| Reserved     |    |    |             |            | AWDFILT[2:0] |            |      | AWD<br>EN  | JAWD<br>EN | Reserved |            | DUALMOD[3:0] |    |    |    |
|              |    |    |             |            | rw0          | rw0        | rw0  | rw         | rw         |          |            | rw           | rw |    |    |
| 15           | 14 | 13 | 12          | 11         | 10           | 9          | 8    | 7          | 6          | 5        | 4          | 3            | 2  | 1  | 0  |
| DISCNUM[2:0] |    |    | JDISC<br>EN | DISC<br>EN | JAUTO        | AWD<br>SGL | SCAN | JEOC<br>IE | AWDIE      | EOCIE    | AWDCH[4:0] |              |    |    |    |
| rw           | rw | rw | rw          | rw         | rw           | rw         | rw   | rw         | rw         | rw       | rw         | rw           | rw | rw | rw |

Bits 31:27 保留, 必须保持为复位值。

Bits 26:24 **AWDFILT[2:0]**: 模拟监视滤波参数

当 AWDEN & JWADEN 为 0 时, 或监测通道变化, AWDFILT counter clear

000: 无作用

001: 两个连续的检测产生一个 AWDx 信号或一个中断

...

111: 连续 8 个检测产生一个 AWDx 信号或一个中断

注: AWDFILT 使用上只会转一个 channel AWDFILT 使用上不考虑 REG&INJ 交错

Bit 23 **AWDEN**: 在规则通道上开启模拟看门狗

该位由软件设置和清除。

0: 在规则通道上禁用模拟看门狗

1: 在规则通道上使用模拟看门狗

Bit 22 **JAWDEN**: 在注入通道上开启模拟看门狗

该位由软件设置和清除。

0: 在注入通道上禁用模拟看门狗

1: 在注入通道上使用模拟看门狗

Bits 21:20 保留, 必须保持为复位值。

Bits 19:16 **DUALMOD[3:0]**: 双模式选择

软件使用这些位选择操作模式。

0000: 独立模式

0001: 混合的同步规则+注入同步模式

0010: 混合的同步规则+交替触发模式

0011: 混合同步注入+快速交叉模式

0100: 混合同步注入+慢速交叉模式

0101: 注入同步模式

0110: 规则同步模式

0111: 快速交叉模式

1000: 慢速交叉模式

1001: 交替触发模式

注: 在 ADC2 和 ADC3 中这些位为保留位

在双模式中, 改变通道的配置会产生一个重新开始的条件, 这将导致同步丢失。建议在  
进行任何配置改变前关闭双模式。

**Bits 15:13 DISCNUM[2:0]: 中断模式通道计数**

软件通过这些位定义在间断模式下，收到外部触发后转换规则通道的数目。

000: 1 个通道

001: 2 个通道

.....

111: 8 个通道

**Bit 12 JDISCEN: 注入通道上的间断模式**

该位由软件设置和清除，用于开启或关闭注入通道组上的间断模式。

0: 注入通道组上禁用间断模式

1: 注入通道组上使用间断模式

**Bit 11 DISCEN: 规则通道上的间断模式**

该位由软件设置和清除，用于开启或关闭规则通道组上的间断模式。

0: 规则通道组上禁用间断模式

1: 规则通道组上使用间断模式

**Bit 10 JAUTO: 自动的注入通道组转换**

该位由软件设置和清除，用于开启或关闭规则通道组转换结束后自动的注入通道组转换。

0: 关闭自动的注入通道组转换

1: 开启自动的注入通道组转换

**Bit 9 AWDSGL: 扫描模式中在一个单一的通道上使用看门狗**

该位由软件设置和清除，用于开启或关闭由 AWDCH[4:0]位指定的通道上的模拟看门狗功能

0: 在所有的通道上使用模拟看门狗

1: 在单一通道上使用模拟看门狗

**Bit 8 SCAN: 扫描模式**

该位由软件设置和清除，用于开启或关闭扫描模式。在扫描模式中，转换由 ADC\_SQRx 或 ADC\_JSQRx 寄存器选中的通道。

0: 关闭扫描模式

1: 使用扫描模式

注：如果分别设置了 EOCIE 或 JEOCIE 位，只在最后一个通道转换完毕后才产生 EOC 或 JEOC 中断。

**Bit 7 JEOCIE: 允许产生注入通道转换结束中断**

该位由软件设置和清除，用于禁止或允许所有注入通道转换结束后产生中断。

0: 禁止 JEOC 中断

1: 允许 JEOC 中断。当硬件设置 JEOC 位时产生中断。

**Bit 6 AWDIE: 允许产生模拟看门狗中断**

该位由软件设置和清除，用于禁止或允许模拟看门狗产生中断。在扫描模式下，如果看门狗检测到超范围的数值时，只有在设置了该位时扫描才会中止。

0: 禁止模拟看门狗中断

1: 允许模拟看门狗中断

**Bit 5 EOCIE: 允许产生 EOC 中断**

该位由软件设置和清除，用于禁止或允许转换结束后产生中断。

0: 禁止 EOC 中断

1: 允许 EOC 中断。当硬件设置 EOC 位时产生中断。

Bits 4:0 **AWDCH[4:0]**: 模拟看门狗通道选择位

这些位由软件设置和清除，用于选择模拟看门狗保护的输入通道。

0000: ADC 模拟输入通道 0

0001: ADC 模拟输入通道 1

.....

1011: ADC 模拟输入通道 11

1100: ADC 模拟输入通道 12

1101: ADC 模拟输入通道 13

1110: ADC 模拟输入通道 14

1111: ADC 模拟输入通道 15

### 16.12.3 ADC 控制寄存器 2 (ADC\_CR2)

地址偏移: 0x08

复位值: 0x0100 0000

访问: 无等待, 仅支持字访问

| 31       | 30           | 29 | 28 | 27 | 26    | 25   | 24  | 23       | 22    | 21    | 20   | 19          | 18  | 17   | 16   |
|----------|--------------|----|----|----|-------|------|-----|----------|-------|-------|------|-------------|-----|------|------|
| Reserved |              |    |    |    |       |      |     | TS       | SW    | JSW   | EXT  | EXTSEL[3:0] |     |      |      |
|          |              |    |    |    |       |      |     | VREFE    | START | START | TRIG |             |     |      |      |
|          |              |    |    |    |       |      |     | r        | rw    | rw    | rw   | rw          | rw  | rw   | rw   |
| 15       | 14           | 13 | 12 | 11 | 10    | 9    | 8   | 7        | 6     | 5     | 4    | 3           | 2   | 1    | 0    |
| JEXT     | JEXTSEL[3:0] |    |    |    | ALIGN | Res. | DMA | Reserved |       |       |      | RST         | CAL | CONT | ADON |
| TRIG     |              |    |    |    |       |      |     |          |       |       |      | CAL         |     |      |      |
| rw       | rw           | rw | rw | rw | rw    |      | rw  |          |       |       |      | rw          | rw  | rw   | rw   |

Bits 31:24 保留, 必须保持为复位值。

Bit 23 **TSVREFE**: 温度传感器和 VBG 使能

该位由软件设置和清除, 用于开启或禁止温度传感器和 VBG 通道。该位仅出现在 ADC1 中。

0: 禁止温度传感器和 VBG

1: 使能温度传感器和 VBG

Bit 22 **SWSTART**: 开始转换规则通道

由软件设置该位以启动转换, 转换开始后硬件马上清除此位。如果在 EXTSEL[3:0]位中选择了 SWSTART 为触发事件, 该位用于启动一组规则通道的转换,

0: 复位状态;

1: 开始转换规则通道。

Bit 21 **JSWSTART**: 开始转换注入通道

由软件设置该位以启动转换, 转换开始后硬件马上清除此位。如果在 JEXTSEL[3:0]位中选择了 JSWSTART 为触发事件, 该位用于启动一组注入通道的转换。

0: 复位状态

1: 开始转换注入通道

Bit 20 **EXTTRIG**: 规则通道的外部触发转换模式

该位由软件设置和清除, 用于开启或禁止可以启动规则通道组转换的外部触发事件。

0: 不用外部事件启动转换

1: 使用外部事件启动转换

Bits 19:16 **EXTSEL[3:0]**: 选择启动规则通道组转换的外部事件

这些位选择用于启动规则通道组转换的外部事件。

ADC1/2/3 的触发配置如下：

0000: TIM1-TRGO  
0001: TIM1-TRGO2  
0010: TIM8-TRGO  
0011: TIM8-TRGO2  
0100: TIM2-TRGO  
0101: TIM3-TRGO  
0110: TIM4-TRGO  
0111: SWSTART  
1000: EXTI\_2 (ADC2: EXTI\_3, ADC3: EXTI\_4)  
1001: TIM6-TRGO  
1010: hrtim\_adc\_trg1 (ADC2: hrtim\_adc\_trg3, ADC3: hrtim\_adc\_trg5)  
1011: hrtim\_adc\_trg6  
1100: hrtim\_adc\_trg7  
1101: hrtim\_adc\_trg8  
1110: hrtim\_adc\_trg9  
1111: hrtim\_adc\_trg10

Bit 15 **JEXTTRIG**: 注入通道的外部触发转换模式

该位由软件设置和清除，用于开启或禁止可以启动注入通道组转换的外部触发事件。

0: 不用外部事件启动转换

1: 使用外部事件启动转换

Bits 14:11 **JEXTSEL[3:0]**: 选择启动注入通道组转换的外部事件

这些位选择用于启动注入通道组转换的外部事件。

ADC1/2/3 的触发配置如下：

0000: TIM1\_TRGO  
0001: TIM1\_TRGO2  
0010: TIM8\_TRGO  
0011: TIM8\_TRGO2  
0100: TIM1\_CC4/CC5 (ADC2/3: TIM8\_CC4/CC5)  
0101: TIM2\_TRGO (ADC2: TIM3\_TRGO, ADC3: TIM4\_TRGO)  
0110: EXTI13 (ADC2: EXTI14, ADC3: EXTI15)  
0111: JSWSTART  
1000: TIM15\_TRGO  
1001: TIM7\_TRGO  
1010: hrtim\_adc\_trg2 (ADC2: hrtim\_adc\_trg4, ADC3: hrtim\_adc\_trg5)  
1011: hrtim\_adc\_trg6  
1100: hrtim\_adc\_trg7  
1101: hrtim\_adc\_trg8  
1110: hrtim\_adc\_trg9  
1111: hrtim\_adc\_trg10

Bit 10 **ALIGN**: 数据对齐

该位由软件设置和清除。

0: 右对齐

1: 左对齐

Bit 9 保留, 必须保持为复位值。

Bit 8 **DMA**: 直接存储器访问模式

该位由软件设置和清除。

0: 不使用 DMA 模式

1: 使用 DMA 模式

Bits 7:4 保留, 必须保持为复位值。

Bit 3 **RSTCAL**: 复位校准

该位由软件设置并由硬件清除。在校准寄存器被初始化后该位将被清除。

0: 校准寄存器已初始化

1: 初始化校准寄存器

注: 如果正在进行转换时设置 RSTCAL, 清除校准寄存器需要额外的周期。

Bit 2 **CAL**: A/D 校准

该位由软件设置以开始校准, 并在校准结束时由硬件清除。

0: 校准完成

1: 开始校准

Bit 1 **CONT**: 连续转换

该位由软件设置和清除。如果设置了此位, 则转换将连续进行直到该位被清除。

0: 单次转换模式

1: 连续转换模式

Bit 0 **ADON**: 开/关 A/D 转换器

该位由软件设置和清除。当该位为 0 时, 写入 1 将把 ADC 从断电模式下唤醒。当该位为 1 时, 写入 1 将启动转换。

0: 关闭 ADC 转换/校准, 并进入断电模式

1: 开启 ADC 并启动转换

注: 如果在这个寄存器中与 ADON 一起还有其他位被改变, 则转换不被触发。这是为了防止触发错误的转换。

## 16.12.4 ADC 采样时间寄存器 1 (ADC1\_SMPR1)

地址偏移: 0x0C

复位值: 0x0000 0000

访问: 无等待, 仅支持字访问

| 31       | 30         | 29 | 28 | 27         | 26 | 25 | 24         | 23 | 22 | 21         | 20 | 19 | 18         | 17         | 16 |
|----------|------------|----|----|------------|----|----|------------|----|----|------------|----|----|------------|------------|----|
| Reserved |            |    |    |            |    |    |            |    |    |            |    |    |            | SMP15[2:1] |    |
|          |            |    |    |            |    |    |            |    |    |            |    |    |            | rw         | rw |
| 15       | 14         | 13 | 12 | 11         | 10 | 9  | 8          | 7  | 6  | 5          | 4  | 3  | 2          | 1          | 0  |
| SMP15[0] | SMP14[2:0] |    |    | SMP13[2:0] |    |    | SMP12[2:0] |    |    | SMP11[2:0] |    |    | SMP10[2:0] |            |    |
| rw       | rw         | rw | rw | rw         | rw | rw | rw         | rw | rw | rw         | rw | rw | rw         | rw         | rw |

Bits 31:30 保留, 必须保持为复位值。

Bits 29:0 **SMPx[2:0]**: 通道 x 的采样时间 ( $x = 15..10$ )

这些位用于独立地选择每个通道的采样时间。在采样周期中通道选择位必须保持不变。

000: 3.5 周期

001: 7.5 周期



010: 11.5 周期  
011: 13.5 周期  
100: 28.5 周期  
101: 55.5 周期  
110: 71.5 周期  
111: 640.5 周期

### 16.12.5 ADC 采样时间寄存器 2 (ADC1\_SMPR2)

地址偏移: 0x10

复位值: 0x0000 0000

访问: 无等待, 仅支持字访问

| 31      | 30 | 29        | 28 | 27 | 26        | 25 | 24 | 23        | 22 | 21 | 20        | 19 | 18 | 17        | 16 |
|---------|----|-----------|----|----|-----------|----|----|-----------|----|----|-----------|----|----|-----------|----|
| Res.    |    | SMP9[2:0] |    |    | SMP8[2:0] |    |    | SMP7[2:0] |    |    | SMP6[2:0] |    |    | SMP5[2:1] |    |
|         |    | rw        | rw | rw | rw        | rw | rw | rw        | rw | rw | rw        | rw | rw | rw        | rw |
| 15      | 14 | 13        | 12 | 11 | 10        | 9  | 8  | 7         | 6  | 5  | 4         | 3  | 2  | 1         | 0  |
| SMP5[0] |    | SMP4[2:0] |    |    | SMP3[2:0] |    |    | SMP2[2:0] |    |    | SMP1[2:0] |    |    | SMP0[2:0] |    |
| rw      | rw | rw        | rw | rw | rw        | rw | rw | rw        | rw | rw | rw        | rw | rw | rw        | rw |

Bits 31:30 保留, 必须保持为复位值。

Bits 29:0 **SMPx[2:0]**: 通道 x 的采样时间 ( $x = 9..0$ )

这些位用于独立地选择每个通道的采样时间。在采样周期中通道选择位必须保持不变。

000: 3.5 周期  
001: 7.5 周期  
010: 11.5 周期  
011: 13.5 周期  
100: 28.5 周期  
101: 55.5 周期  
110: 71.5 周期  
111: 640.5 周期

### 16.12.6 ADC 注入通道数据偏移寄存器 x (ADC\_JOFRx) ( $x=1..4$ )

地址偏移: 0x14 - 0x20

复位值: 0x0000 0000

访问: 无等待, 仅支持字访问

| 31       | 30 | 29 | 28 | 27             | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|----------|----|----|----|----------------|----|----|----|----|----|----|----|----|----|----|----|
| Reserved |    |    |    |                |    |    |    |    |    |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11             | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    | JOFFSETx[11:0] |    |    |    |    |    |    |    |    |    |    |    |
|          |    |    |    | rw             | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:12 保留, 必须保持为复位值。

Bits 11:0 **JOFFSETx[11:0]**: 注入通道 x 的数据偏移 ( $x = 1..4$ )

当转换注入通道时, 这些位定义了用于从原始转换数据中减去的数值。转换的结果可以在 ADC\_JDRx 寄存器中读出。

### 16.12.7 ADC 看门狗高阈值寄存器 (ADC\_HTR)

地址偏移: 0x24

复位值: 0x0000 0FFF

访问: 无等待, 仅支持字访问

|          |    |    |    |           |    |    |    |    |    |    |    |    |    |    |    |
|----------|----|----|----|-----------|----|----|----|----|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27        | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |           |    |    |    |    |    |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11        | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    | HTR[11:0] |    |    |    |    |    |    |    |    |    |    |    |
|          |    |    |    | rw        | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:12 保留, 必须保持为复位值。

Bits 11:0 HTR[11:0]: 模拟看门狗高阈值

这些位定义了模拟看门狗的阈值高限。

### 16.12.8 ADC 看门狗低阈值寄存器 (ADC\_LTR)

地址偏移: 0x28

复位值: 0x0000 0000

访问: 无等待, 仅支持字访问

|          |    |    |    |           |    |    |    |    |    |    |    |    |    |    |    |
|----------|----|----|----|-----------|----|----|----|----|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27        | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |           |    |    |    |    |    |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11        | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    | LTR[11:0] |    |    |    |    |    |    |    |    |    |    |    |
|          |    |    |    | rw        | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:12 保留, 必须保持为复位值。

Bits 11:0 LTR[11:0]: 模拟看门狗低阈值

这些位定义了模拟看门狗的阈值低限。

### 16.12.9 ADC 规则序列寄存器 1 (ADC\_SQR1)

地址偏移: 0x2C

复位值: 0x0000 0000

访问: 无等待, 仅支持字访问

|          |    |    |    |    |    |    |    |    |    |        |    |    |          |    |    |
|----------|----|----|----|----|----|----|----|----|----|--------|----|----|----------|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21     | 20 | 19 | 18       | 17 | 16 |
| Reserved |    |    |    |    |    |    |    |    |    | L[2:0] |    |    | Reserved |    |    |
|          |    |    |    |    |    |    |    |    |    | rw     | rw | rw |          |    |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5      | 4  | 3  | 2        | 1  | 0  |
| Reserved |    |    |    |    |    |    |    |    |    |        |    |    |          |    |    |

Bits 31:23 保留, 必须保持为复位值。

Bits 22:20 L[2:0]: 规则通道序列长度

这些位由软件定义在规则通道转换序列中的通道数目。

000: 1 个转换

001: 2 个转换

.....

111: 8 个转换

Bits 19:0 保留, 必须保持为复位值。

### 16.12.10 ADC 规则序列寄存器 2 (ADC\_SQR2)

地址偏移: 0x30

复位值: 0x0000 0000

访问: 无等待, 仅支持字访问

|          |    |    |    |    |    |          |    |    |    |    |          |    |    |    |    |
|----------|----|----|----|----|----|----------|----|----|----|----|----------|----|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25       | 24 | 23 | 22 | 21 | 20       | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |          |    |    |    |    |          |    |    |    |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9        | 8  | 7  | 6  | 5  | 4        | 3  | 2  | 1  | 0  |
| Reserved |    |    |    |    |    | SQ8[4:0] |    |    |    |    | SQ7[4:0] |    |    |    |    |
|          |    |    |    |    |    | rw       | rw | rw | rw | rw | rw       | rw | rw | rw | rw |

Bits 31:10 保留, 必须保持为复位值。

Bits 9:0 **SQx[3:0]**: 规则序列中的第 x 个转换 (x = 8..7)

这些位由软件定义转换序列中的第 x 个转换通道的编号

### 16.12.11 ADC 规则序列寄存器 3 (ADC\_SQR3)

地址偏移: 0x34

复位值: 0x0000 0000

访问: 无等待, 仅支持字访问

|          |    |          |    |    |    |    |          |    |    |    |    |          |    |    |    |
|----------|----|----------|----|----|----|----|----------|----|----|----|----|----------|----|----|----|
| 31       | 30 | 29       | 28 | 27 | 26 | 25 | 24       | 23 | 22 | 21 | 20 | 19       | 18 | 17 | 16 |
| Reserved |    | SQ6[4:0] |    |    |    |    | SQ5[4:0] |    |    |    |    | SQ4[4:1] |    |    |    |
|          |    | rw       | rw | rw | rw | rw | rw       | rw | rw | rw | rw | rw       | rw | rw | rw |
| 15       | 14 | 13       | 12 | 11 | 10 | 9  | 8        | 7  | 6  | 5  | 4  | 3        | 2  | 1  | 0  |
| SQ4[0]   |    | SQ3[4:0] |    |    |    |    | SQ2[4:0] |    |    |    |    | SQ1[4:0] |    |    |    |
| rw       | rw | rw       | rw | rw | rw | rw | rw       | rw | rw | rw | rw | rw       | rw | rw | rw |

Bits 31:30 保留, 必须保持为复位值。

Bits 29:0 **SQx[3:0]**: 规则序列中的第 x 个转换 (x = 6..1)

这些位由软件定义转换序列中的第 x 个转换通道的编号

### 16.12.12 ADC 注入序列寄存器 (ADC\_JSQR)

地址偏移: 0x38

复位值: 0x0000 0000

访问: 无等待, 仅支持字访问

|          |    |           |    |    |    |    |           |    |    |         |    |           |    |    |    |
|----------|----|-----------|----|----|----|----|-----------|----|----|---------|----|-----------|----|----|----|
| 31       | 30 | 29        | 28 | 27 | 26 | 25 | 24        | 23 | 22 | 21      | 20 | 19        | 18 | 17 | 16 |
| Reserved |    |           |    |    |    |    |           |    |    | JL[1:0] |    | JSQ4[4:1] |    |    |    |
|          |    |           |    |    |    |    |           |    |    | rw      | rw | rw        | rw | rw | rw |
| 15       | 14 | 13        | 12 | 11 | 10 | 9  | 8         | 7  | 6  | 5       | 4  | 3         | 2  | 1  | 0  |
| JSQ4[0]  |    | JSQ3[4:0] |    |    |    |    | JSQ2[4:0] |    |    |         |    | JSQ1[4:0] |    |    |    |
| rw       | rw | rw        | rw | rw | rw | rw | rw        | rw | rw | rw      | rw | rw        | rw | rw | rw |

Bits 31:22 保留, 必须保持为复位值。

Bits 21:20 **JL[1:0]**: 注入通道序列长度 (Injected sequence length)

这些位由软件定义在规则通道转换序列中的通道数目。

00: 1 个转换

01: 2 个转换

10: 3 个转换

11: 4 个转换

Bits 19:0 JSQx[4:0]: 注入序列中的第 x 个转换

这些位由软件定义转换序列中的第 x 个转换通道的编号

### 16.12.13 ADC 注入数据寄存器 (ADC\_JDRx) (x=1..4)

地址偏移: 0x3C-0x48

复位值: 0x0000 0000

访问: 无等待, 仅支持字访问

|            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31         | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15         | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| JDRx[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| r          | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  |

Bits 31:16 保留, 必须保持为复位值。

Bits 15:0 JDATA[15:0]: 注入转换的数据

这些位为只读, 包含了注入通道的转换结果。数据是左对齐或右对齐。

### 16.12.14 ADC 通用数据寄存器 (ADC\_DR)

地址偏移: 0x4C

复位值: 0x0000 0000

访问: 无等待, 仅支持字访问

|                |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31             | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| ADC2DATA[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| r              | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  |
| 15             | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| DATA[15:0]     |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| r              | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  |

Bits 31:16 ADC2DATA[15:0]: ADC2 转换的数据

在 ADC1 中: 双模式下, 这些位包含了 ADC2 转换的规则通道数据。

在 ADC2 和 ADC3 中: 不使用这些位。

Bits 15:0 DATA[15:0]: 规则转换的数据

这些位为只读, 包含了规则通道的转换结果。数据是左对齐或右对齐。

### 16.12.15 ADC 校准寄存器 (ADC\_CAL)

地址偏移: 0x54

复位值: 0x0000 0000

访问: 无等待, 仅支持字访问

|          |    |    |    |    |    |    |    |    |               |    |    |    |    |    |    |
|----------|----|----|----|----|----|----|----|----|---------------|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22            | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |    |    |    |               |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6             | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    |    |    |    |    |    | CAL_DATA[6:0] |    |    |    |    |    |    |
|          |    |    |    |    |    |    |    |    | r             | r  | r  | r  | r  | r  | r  |

Bits 31:16 **CAL\_DATA[6:0]**: ADC 自校准的值写入该寄存器

可以手动写入值, 在 ADC 转换中生效

### 16.12.16 ADC TPS 校准寄存器 (ADC\_TPS)

地址偏移: 0x64

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |             |    |    |    |    |    |    |    |
|----------|----|----|----|----|----|----|----|-------------|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23          | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |    |    |             |    |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7           | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    |    |    |    |    | OFFSET[7:0] |    |    |    |    |    |    |    |
|          |    |    |    |    |    |    |    | rw          | rw | rw | rw | rw | rw | rw | rw |

Bits 31:8 保留, 必须保持为复位值。

Bits 11:0 **OFFSET[7:0]**: TPS 的校准值写入该寄存器

可以手动写入值, 也可以自加载

转换 TPS 时生效, 转换值+OFFSET

Bit 7 为符号位

### 16.12.17 ADC 规则序列数据寄存器 (ADC\_DRx) (x=1..8)

地址偏移: 0x68-0x84

复位值: 0x0000 0000

访问: 无等待, 仅支持字访问

|                     |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---------------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31                  | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DRx_ADC2DATA [15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| r                   | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  |
| 15                  | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| DRx[15:0]           |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| r                   | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  |

Bits 31:16 **ADC2DATA[15:0]**: ADC2 转换的数据

在 ADC1 中: 双模式下, 这些位包含了 ADC2 转换的规则通道数据。

在 ADC2 和 ADC3 中: 不使用这些位。

Bits 15:0 **DATA[15:0]**: 规则转换的数据

这些位为只读, 包含了规则通道的转换结果。数据是左对齐或右对齐。

## 17 数字/模拟转换 (DAC)

### 17.1 DAC 简介

数字/模拟转换模块 (DAC) 是 12 位数字输入，电压输出的数字/模拟转换器。DAC 可以配置为 8 位或 12 位模式，也可以与 DMA 控制器配合使用数据可以设置成左对齐或右对齐

### 17.2 主要特征

- 3 个 DAC
- 支持 8/12bit
- 支持左/右对齐
- 支持三角波/锯齿波/和噪声模式
- 支持 DAC 双通道模式
- 支持 DMA 溢出功能
- 触发源可选
- 输出支持 buff 开/关模式
- 支持自校准
- Input voltage reference, VREF+

### 17.3 功能描述

#### 17.3.1 DAC 框图

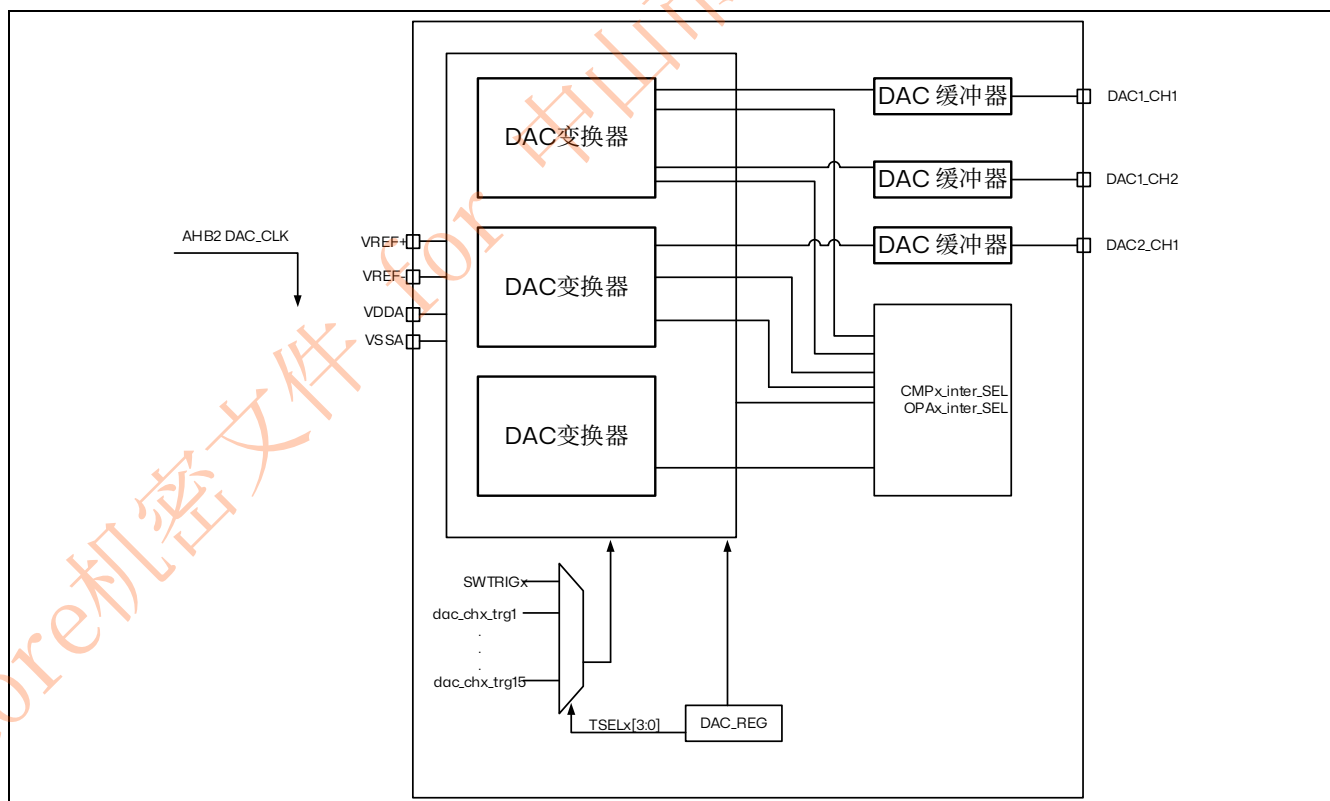


图 17.1 DAC 框图

### 17.3.2 DAC 引脚和内部信号

表 17.1 DAC 引脚

| DAC1      | IO   | IO Type | DAC2      | IO   | IO Type | DAC3      | IO  | IO Type |
|-----------|------|---------|-----------|------|---------|-----------|-----|---------|
| DAC1_OUT1 | PE9  | TT_A    | DAC2_OUT1 | PE13 | TT_A    | DAC3_OUT1 | 走内部 | -       |
| DAC1_OUT2 | PE10 | TT_A    | DAC2_OUT2 | 走内部  | -       | DAC3_OUT2 | 走内部 | -       |

DAC 包括:

- 最多两个输出通道
- DACx\_OUTy 可以从输出引脚断开并用作普通 GPIO
- dac\_outx 可以使用内部引脚连接到片上外设, 如比较器, 运算放大器和 ADC (如果可用)。

DAC 输出接到 GPIO 还是走内部连接是固定的, 若输出接到 GPIO 则没有内部路径到其他外设。

DAC 输出可以缓冲或不缓冲。(连接到外设无缓冲)

表 17.2 DAC 输入/输出引脚

| 引脚名称      | 信号类型      | 备注                                        |
|-----------|-----------|-------------------------------------------|
| VREF+     | 输入, 模拟参考正 | 较高/正参考电压为 DAC, $V_{REF+} \leq V_{DDAmax}$ |
| VDDA      | 输入, 模拟电源  | 模拟电源                                      |
| VSSA      | 输入, 模拟电源地 | 模拟电源接地                                    |
| DACx_OUTy | 模拟输出信号    | DACx 通道模拟输出                               |

表 17.3 DAC1 通道 1 互联

| 内部信号名                           | TSEL1[3:0] | 信号源                  |
|---------------------------------|------------|----------------------|
| dac_ch1_dma                     | 双向         | DAC 通道 1 DMA 请求/确认   |
| dac_ch2_dma                     | 双向         | DAC 通道 2 DMA 请求/确认   |
| dac_ch1_trgx (x = 1 to 15)      | 输入         | DAC 通道 1 触发器输入       |
| dac_ch2_trgx (x = 1 to 15)      | 输入         | DAC 通道 2 触发器输入       |
| dac_ch1_inc_trgx (x = 1 to 15)  | 输入         | DAC 通道 e1 锯齿-增量触发器输入 |
| dac_chn2_inc_trgx (x = 1 to 15) | 输入         | DAC 通道 e2 锯齿-增量触发器输入 |
| dac_unr_it                      | 输出         | 直流中断                 |
| dac_hclk                        | 输入         | DAC 圆周时钟             |
| dac_out1                        | 模拟输出       | 芯片外围设备的 DAC 通道 1 输出  |
| dac_out2                        | 模拟输出       | 芯片外围设备的 DAC 通道 2 输出  |

表 17.4 DAC 输入/输出信号

| 信号名                     | TSEL1[3:0] | 信号源        |
|-------------------------|------------|------------|
|                         | 0000       | SWTRIGx    |
| dac_chx_trg1 (x = 1, 2) | 0001       | TIM8_TRGO  |
| dac_chx_trg2 (x = 1, 2) | 0010       | TIM7_TRGO  |
| dac_chx_trg3 (x = 1, 2) | 0011       | TIM15_TRGO |
| dac_chx_trg4 (x = 1, 2) | 0100       | TIM2_TRGO  |
| dac_chx_trg5 (x = 1, 2) | 0101       | TIM4_TRGO  |



| 信号名                      | TSEL1[3:0] | 信号源                  |
|--------------------------|------------|----------------------|
| dac_chx_trg6 (x = 1, 2)  | 0110       | EXTI9                |
| dac_chx_trg7 (x = 1, 2)  | 0111       | TIM6_TRGO            |
| dac_chx_trg8 (x = 1, 2)  | 1000       | TIM3_TRGO            |
| dac_chx_trg9 (x = 1, 2)  | 1001       | hrtim_dac_reset_trg1 |
| dac_chx_trg10 (x = 1, 2) | 1010       | hrtim_dac_reset_trg2 |
| dac_chx_trg11 (x = 1, 2) | 1011       | hrtim_dac_reset_trg3 |
| dac_chx_trg12 (x = 1, 2) | 1100       | hrtim_dac_reset_trg4 |
| dac_chx_trg13 (x = 1, 2) | 1101       | hrtim_dac_reset_trg5 |
| dac_chx_trg14 (x = 1, 2) | 1110       | hrtim_dac_reset_trg6 |
| dac_chx_trg15 (x = 1, 2) | 1111       | hrtim_dac_trg1       |

表 17.5 DAC1 通道 2 互联

| 信号名                          | TSEL1[3:0] | 信号源                 |
|------------------------------|------------|---------------------|
|                              | 0000       | SWTRIGBx            |
| dac_inc_chx_trg1 (x = 1, 2)  | 0001       | TIM8_TRGO           |
| dac_inc_chx_trg2 (x = 1, 2)  | 0010       | TIM7_TRGO           |
| dac_inc_chx_trg3 (x = 1, 2)  | 0011       | TIM15_TRGO          |
| dac_inc_chx_trg4 (x = 1, 2)  | 0100       | TIM2_TRGO           |
| dac_inc_chx_trg5 (x = 1, 2)  | 0101       | TIM4_TRGO           |
| dac_inc_chx_trg6 (x = 1, 2)  | 0110       | EXTI10              |
| dac_inc_chx_trg7 (x = 1, 2)  | 0111       | TIM6_TRGO           |
| dac_inc_chx_trg8 (x = 1, 2)  | 1000       | TIM3_TRGO           |
| dac_inc_chx_trg9 (x = 1, 2)  | 1001       | hrtim_dac_step_trg1 |
| dac_inc_chx_trg10 (x = 1, 2) | 1010       | hrtim_dac_step_trg2 |
| dac_inc_chx_trg11 (x = 1, 2) | 1011       | hrtim_dac_step_trg3 |
| dac_inc_chx_trg12 (x = 1, 2) | 1100       | hrtim_dac_step_trg4 |
| dac_inc_chx_trg13 (x = 1, 2) | 1101       | hrtim_dac_step_trg5 |
| dac_inc_chx_trg14 (x = 1, 2) | 1110       | hrtim_dac_step_trg6 |
|                              | 1111       |                     |

表 17.6 DAC2 通道 1 互联

| 信号名                     | TSEL1[3:0] | 信号源        |
|-------------------------|------------|------------|
|                         | 0000       | SWTRIGx    |
| dac_chx_trg1 (x = 1, 2) | 0001       | TIM8_TRGO  |
| dac_chx_trg2 (x = 1, 2) | 0010       | TIM7_TRGO  |
| dac_chx_trg3 (x = 1, 2) | 0011       | TIM15_TRGO |
| dac_chx_trg4 (x = 1, 2) | 0100       | TIM2_TRGO  |
| dac_chx_trg5 (x = 1, 2) | 0101       | TIM4_TRGO  |
| dac_chx_trg6 (x = 1, 2) | 0110       | EXTI9      |
| dac_chx_trg7 (x = 1, 2) | 0111       | TIM6_TRGO  |

| 信号名                      | TSEL1[3:0] | 信号源                  |
|--------------------------|------------|----------------------|
| dac_chx_trg8 (x = 1, 2)  | 1000       | TIM3_TRGO            |
| dac_chx_trg9 (x = 1, 2)  | 1001       | hrtim_dac_reset_trg1 |
| dac_chx_trg10 (x = 1, 2) | 1010       | hrtim_dac_reset_trg2 |
| dac_chx_trg11 (x = 1, 2) | 1011       | hrtim_dac_reset_trg3 |
| dac_chx_trg12 (x = 1, 2) | 1100       | hrtim_dac_reset_trg4 |
| dac_chx_trg13 (x = 1, 2) | 1101       | hrtim_dac_reset_trg5 |
| dac_chx_trg14 (x = 1, 2) | 1110       | hrtim_dac_reset_trg6 |
| dac_chx_trg15 (x = 1, 2) | 1111       | hrtim_dac_trg2       |

表 17.7 DAC2 通道 2 互联

| 信号名                          | TSEL1[3:0] | 信号源                 |
|------------------------------|------------|---------------------|
|                              | 0000       | SWTRIGBx            |
| dac_inc_chx_trg1 (x = 1, 2)  | 0001       | TIM8_TRGO           |
| dac_inc_chx_trg2 (x = 1, 2)  | 0010       | TIM7_TRGO           |
| dac_inc_chx_trg3 (x = 1, 2)  | 0011       | TIM15_TRGO          |
| dac_inc_chx_trg4 (x = 1, 2)  | 0100       | TIM2_TRGO           |
| dac_inc_chx_trg5 (x = 1, 2)  | 0101       | TIM4_TRGO           |
| dac_inc_chx_trg6 (x = 1, 2)  | 0110       | EXTI10              |
| dac_inc_chx_trg7 (x = 1, 2)  | 0111       | TIM6_TRGO           |
| dac_inc_chx_trg8 (x = 1, 2)  | 1000       | TIM3_TRGO           |
| dac_inc_chx_trg9 (x = 1, 2)  | 1001       | hrtim_dac_step_trg1 |
| dac_inc_chx_trg10 (x = 1, 2) | 1010       | hrtim_dac_step_trg2 |
| dac_inc_chx_trg11 (x = 1, 2) | 1011       | hrtim_dac_step_trg3 |
| dac_inc_chx_trg12 (x = 1, 2) | 1100       | hrtim_dac_step_trg4 |
| dac_inc_chx_trg13 (x = 1, 2) | 1101       | hrtim_dac_step_trg5 |
| dac_inc_chx_trg14 (x = 1, 2) | 1110       | hrtim_dac_step_trg6 |
|                              | 1111       |                     |

表 17.8 DAC3 通道 1 互联

| 信号名                     | TSEL1[3:0] | 信号源                  |
|-------------------------|------------|----------------------|
|                         | 0000       | SWTRIGx              |
| dac_chx_trg1 (x = 1, 2) | 0001       | TIM1_TRGO            |
| dac_chx_trg2 (x = 1, 2) | 0010       | TIM7_TRGO            |
| dac_chx_trg3 (x = 1, 2) | 0011       | TIM15_TRGO           |
| dac_chx_trg4 (x = 1, 2) | 0100       | TIM2_TRGO            |
| dac_chx_trg5 (x = 1, 2) | 0101       | TIM4_TRGO            |
| dac_chx_trg6 (x = 1, 2) | 0110       | EXTI9                |
| dac_chx_trg7 (x = 1, 2) | 0111       | TIM6_TRGO            |
| dac_chx_trg8 (x = 1, 2) | 1000       | TIM3_TRGO            |
| dac_chx_trg9 (x = 1, 2) | 1001       | hrtim_dac_reset_trg1 |

| 信号名                      | TSEL1[3:0] | 信号源                  |
|--------------------------|------------|----------------------|
| dac_chx_trg10 (x = 1, 2) | 1010       | hrtim_dac_reset_trg2 |
| dac_chx_trg11 (x = 1, 2) | 1011       | hrtim_dac_reset_trg3 |
| dac_chx_trg12 (x = 1, 2) | 1100       | hrtim_dac_reset_trg4 |
| dac_chx_trg13 (x = 1, 2) | 1101       | hrtim_dac_reset_trg5 |
| dac_chx_trg14 (x = 1, 2) | 1110       | hrtim_dac_reset_trg6 |
| dac_chx_trg15 (x = 1, 2) | 1111       | hrtim_dac_trg3       |

表 17.9 DAC3 通道 2 互联

| 信号名                          | TSEL1[3:0] | 信号源                 |
|------------------------------|------------|---------------------|
|                              | 0000       | SWTRIGBx            |
| dac_inc_chx_trg1 (x = 1, 2)  | 0001       | TIM1_TRGO           |
| dac_inc_chx_trg2 (x = 1, 2)  | 0010       | TIM7_TRGO           |
| dac_inc_chx_trg3 (x = 1, 2)  | 0011       | TIM15_TRGO          |
| dac_inc_chx_trg4 (x = 1, 2)  | 0100       | TIM2_TRGO           |
| dac_inc_chx_trg5 (x = 1, 2)  | 0101       | TIM4_TRGO           |
| dac_inc_chx_trg6 (x = 1, 2)  | 0110       | EXTI10              |
| dac_inc_chx_trg7 (x = 1, 2)  | 0111       | TIM6_TRGO           |
| dac_inc_chx_trg8 (x = 1, 2)  | 1000       | TIM3_TRGO           |
| dac_inc_chx_trg9 (x = 1, 2)  | 1001       | hrtim_dac_step_trg1 |
| dac_inc_chx_trg10 (x = 1, 2) | 1010       | hrtim_dac_step_trg2 |
| dac_inc_chx_trg11 (x = 1, 2) | 1011       | hrtim_dac_step_trg3 |
| dac_inc_chx_trg12 (x = 1, 2) | 1100       | hrtim_dac_step_trg4 |
| dac_inc_chx_trg13 (x = 1, 2) | 1101       | hrtim_dac_step_trg5 |
| dac_inc_chx_trg14 (x = 1, 2) | 1110       | hrtim_dac_step_trg6 |
|                              | 1111       |                     |

### 17.3.3 DAC 通道使能

每个 DAC 通道可以通过在 DAC\_CR 寄存器中设置相应的 ENx 位来上电。然后在 t<sub>WAKEUP</sub> 启动时间后使能 DAC 通道。

注意：ENx 位只使能模拟 DAC 通道。即使 ENx 位复位，也使能 DAC 通道数字接口。

### 17.3.1 DAC 数据格式

- 单 DAC 通道有三种可能性：

- 8 位右对齐：软件必须将数据加载到 DAC\_DHR8Rx[7:0] 位（存储到 DHRx[11:4] 位）
- 12 位左对齐：软件必须将数据加载到 DAC\_DHR12Lx[15:4] 位（存储到 DHRx[11:0] 位）
- 12 位右对齐：软件必须将数据加载到 DAC\_DHR12Rx[11:0] 位（存储在 DHRx[11:0] 位中）。

根据加载的 DAC\_DHR<sub>yyyx</sub> 寄存器，用户写入的数据被移动并存储到相应的 DHRx（数据保存寄存器，这是内部非内存映射寄存器）。然后 DHRx 寄存器通过软件触发器或外部事件触发器自动加载到 DORx 寄存器中。

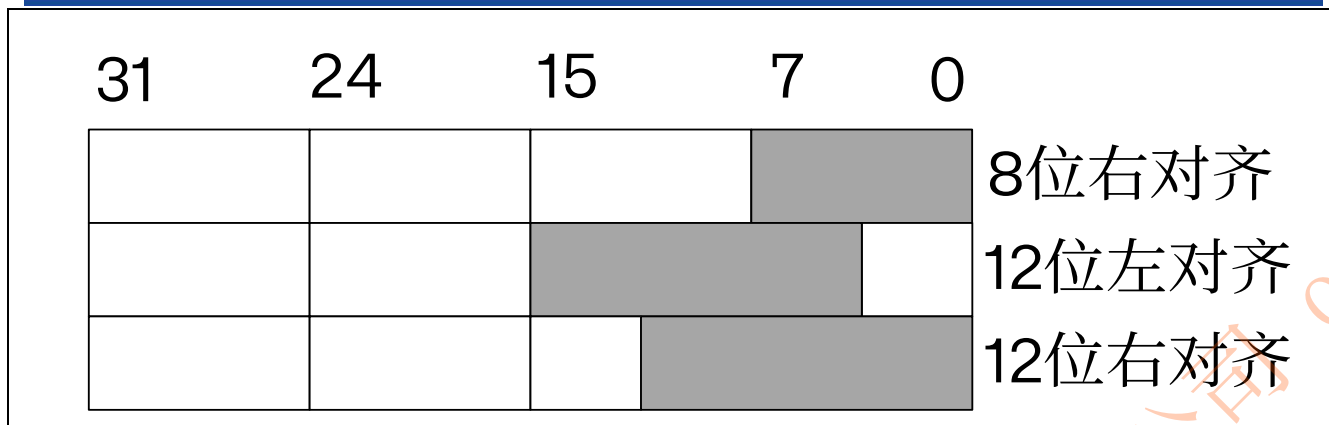


图 17.2 单 DAC 通道模式下的数据寄存器

● 双 DAC 通道（可用时）有三种可能性：

- 8 位右对齐：DAC 通道 1 的数据被加载到 DAC\_DHR8RD[7:0]位（存储在 DHR1[11:4]位中）和 DAC 通道 2 的数据被加载到 DAC\_DHR8RD[15:8]位（存储在 DHR2[11:4]位中）
- 12 位左对齐：DAC 通道 1 的数据被加载到 DAC\_DHR12LD[15:4]位（存储在 DHR1[11:0]位中），DAC 通道 2 的数据被加载到 DAC\_DHR12LD[31:20]位（存储在 DHR2[11:0]位中）
- 12 位右对齐：DAC 通道 1 的数据被加载到 DAC\_DHR12RD[11:0]位中（存储在 DHR1[11:0]位中），DAC 通道 2 的数据被加载到 DAC\_DHR12RD[27:16]位中（存储在 DHR2[11:0]位中）

根据加载的 DAC\_DHR<sub>yyy</sub>D 寄存器，用户写入的数据被移动并存储在 DHR1 和 DHR2 中（数据保持寄存器，它们是内部非内存映射寄存器）。然后，DHR1 和 DHR2 寄存器分别通过软件触发器或外部事件触发器自动加载到 DAC\_DOR1 和 DOR2 寄存器中。

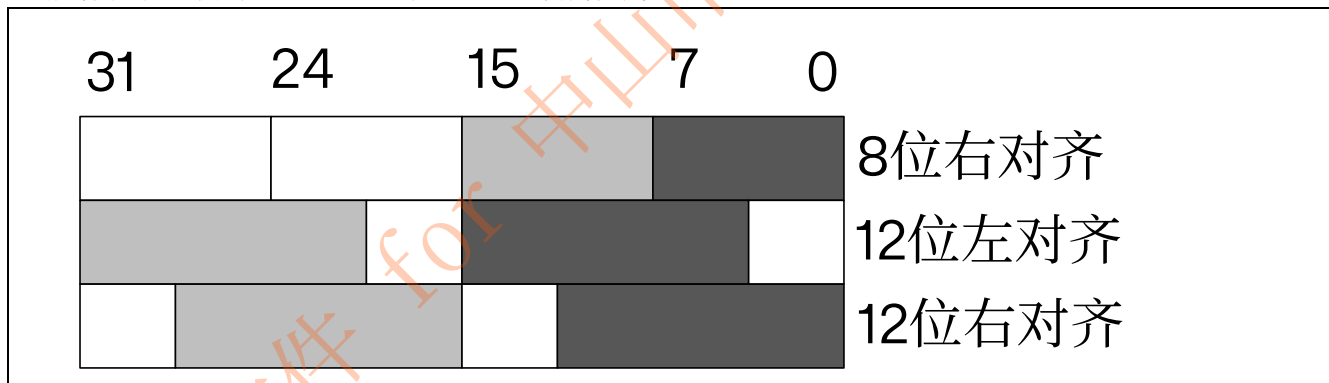


图 17.3 双 DAC 通道模式下的数据寄存器

### 17.3.2 DAC 转换

DAC\_DOR<sub>x</sub> 不能直接写入，任何数据传输到 DAC 通道必须通过加载 DAC\_DHR<sub>x</sub> 寄存器来执行（写入操作到 DAC\_DHR8R<sub>x</sub>、DAC\_DHR12L<sub>x</sub>、DAC\_DHR12R<sub>x</sub>、DAC\_DHR8RD、DAC\_DHR12RD 或 DAC\_DHR12LD）。

如果没有选择硬件触发，存储在 DAC\_DHR<sub>x</sub> 寄存器中的数据在一个 dac\_hclk 时钟周期后自动转移到 DAC\_DOR<sub>x</sub> 寄存器中（DAC\_CR 寄存器中的 TEN<sub>x</sub> 位被重置）。但是，当选择硬件触发器（设置 DAC\_CR 寄存器中的 TEN<sub>x</sub> 位）并发生触发器时，在触发信号后执行三个 dac\_hclk 时钟周期的传输。

当 DAC\_DOR<sub>x</sub> 加载 DAC\_DHR<sub>x</sub> 内容时，根据电源电压和模拟输出负载的不同，经过一段时间的沉淀后，模拟输出电压才可用。

当 dac\_hclk 时钟速率大于 80mhz 时，需要设置 DAC\_MCR 的 HFSEL 位。它为从 DAC\_DHRx 寄存器到 DAC\_DORx 寄存器的传输增加了额外的延迟。

有关 DAC\_DORx 更新速率取决于 HFSEL 位和 dac\_hclk 时钟频率的限制，请参阅下面的表 HFSEL 描述。

如果在不允许的时间段内发生数据更新或软件/硬件触发事件，则外围设备的行为是不可预测的。

上述时序仅与 DAC 接口的限制有关。另请参考产品数据表中的 tsettlement 参数值。

表 17.10 HFSEL 描述

| HFSEL[1:0] | AHB 频率               | 功能                          |
|------------|----------------------|-----------------------------|
| 00         | <=80Hz               | DAC_DOR 更新速率高达 3 个 AHB 时钟周期 |
| 01         | >80Hz <sup>(1)</sup> | DAC_DOR 更新速率高达 5 个 AHB 时钟周期 |
| 10         | >160Hz               | DAC_DOR 更新速率高达 7 个 AHB 时钟周期 |
| 11         | Reserved             | -                           |

1、最大 AHB 频率的值请参考数据手册

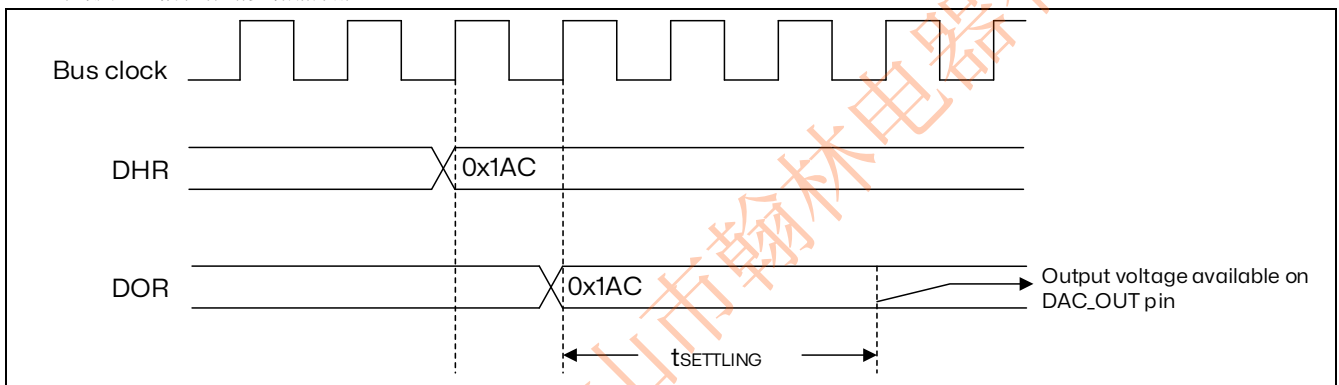


图 17.4 触发器禁用时转换时序

### 17.3.3 DAC 输出电压

数字输入通过 0 和 VREF+ 之间的线性转换转换为输出电压。

每个 DAC 通道引脚上的模拟输出电压由以下公式确定：

$$\text{DACoutput} = V_{\text{REF}} \times \frac{\text{DOR}}{4096}$$

### 17.3.4 DAC 触发器选择

如果 TENx 控制位被设置，那么转换可以由外部事件（定时器计数器，外部中断线）触发。

TSELx[3:0] 控制位决定在 16 个可能的事件中哪一个触发转换，如 DAC\_CR 寄存器的 TSELx[3:0] 位所示。

这些事件可以是软件触发器，也可以是硬件触发器。参考章节 22.4.2：DAC 引脚和内部信号的连接表。

每次 DAC 接口检测到所选触发源上的上升沿（参见下表）时，存储在 DAC\_DHRx 寄存器中的最后一个数据被传输到 DAC\_DORx 寄存器中。在触发发生后，DAC\_DORx 寄存器更新三个 dac\_hclk 周期。

如果选择了软件触发，一旦设置了 SWTRIG 位，转换就会开始。

一旦用 DAC\_DHRx 寄存器内容加载了 DAC\_DORx 寄存器，硬件就会重置 SWTRIG。

锯齿波生成的复位触发选择和增量触发选择分别通过 STRSTTRIGSELx 和 STINCTRIGSELx 控制位执行。

STRSTTRIGSELx 映射类似于 TSELx。参考章节：DAC 引脚和内部信号。

注意：设置 ENx 位时，不能修改 TSELx[3:0]位。

当选择软件触发器时，从 DAC\_DHRx 寄存器到 DAC\_DORx 寄存器的传输只需要一个 dac\_hclk 时钟周期。

### 17.3.5 DMA 请求

每个 DAC 通道都有 DMA 功能。两个 DMA 通道用于服务 DAC 通道 DMA 请求。

当设置 DMAENx 位时发生外部触发器（但不是软件触发器）时，当传输完成时，DAC\_DHRx 寄存器的值被传输到 DAC\_DORx 寄存器中，并生成 DMA 请求。

在双模式下，如果设置了两个 DMAENx 位，则生成两个 DMA 请求。如果只需要一个 DMA 请求，则只需要设置相应的 DMAENx 位。通过这种方式，应用程序可以通过使用一个 DMA 请求和一个唯一的 DMA 通道在双模式下管理两个 DAC 通道。

由于 DAC\_DHRx 到 DAC\_DORx 的数据传输发生在 DMA 请求之前，因此必须在第一个触发事件发生之前将第一批数据写入 DAC\_DHRx。

### DMA 欠运行

DAC DMA 请求没有排队，因此如果第二个外部触发器在接收到第一个外部触发器的确认之前到达（第一个请求），则不会发出新的请求，并且设置 DAC\_SR 寄存器中的 DMA 通道下运行标志 DMAUDRx，报告错误情况。DAC 通道继续转换旧数据。

软件必须通过写入 1 来清除 DMAUDRx 标志，清除使用的 DMA 流的 DMAEN 位，并重新初始化 DMA 和 DAC 通道，以正确地重新启动传输。

软件必须修改 DAC 触发转换频率或减轻 DMA 工作负载，以避免新的 DMA 欠运行。最后，通过使能 DMA 数据传输和转换触发器，可以恢复 DAC 转换。

对于每个 DAC 通道，如果 DAC\_CR 寄存器中相应的 DMAUDRIEx 位被使能，也会产生中断。

### 双数据模式

当 DMAENx 位设置位时，每两个外部触发器（软件触发器除外）产生一个 DAC DMA 请求：

1. 当检测到第一个触发器时，将 DAC\_DHRx 和 DAC\_DHRBx 寄存器的值传输到 DAC\_DORx 和 DAC\_DORBx 寄存器中。实际的 DAC 数据被加载到 DAC\_DORx 寄存器中。然后生成一个 DMA 请求。DMA 将新数据写入 DAC\_DHRx 和 DAC\_DHRBx 数据寄存器。
2. 当检测到下一个触发器时，实际的 DAC 数据被加载到 DAC\_DHRBx 寄存器中。第二个触发器不生成任何 DMA 请求。DORSTATx 位表示哪个 DOR 数据实际加载到模拟 DAC 输入中。

在 DMA 双数据模式下也支持 DMA 欠运行功能。

在 DMA 双模式下，DMA 请求只能处理一个 DAC 通道。为了在 DMA 双通道模式下使用两个通道输出，每个 DMA 通道必须单独配置。

从双数据模式切换到单数据模式，或者从单数据模式切换到双数据模式，需要满足以下条件：

- 必须禁用 DAC。
- DMAEN 位必须清除(ENx = 0, DMAEN = 0)。

### 17.3.6 噪音的产生

为了产生可变幅度伪噪声，LFSR（线性反馈移位寄存器）是可用的。通过将 WAVEx[1:0]设置为 01 来选择 DAC 噪声产生。LFSR 中的预加载值为 0xAAA。该寄存器在每个触发事件之后按照特定的计算方法更新三个 dac\_hclk 时钟周期。



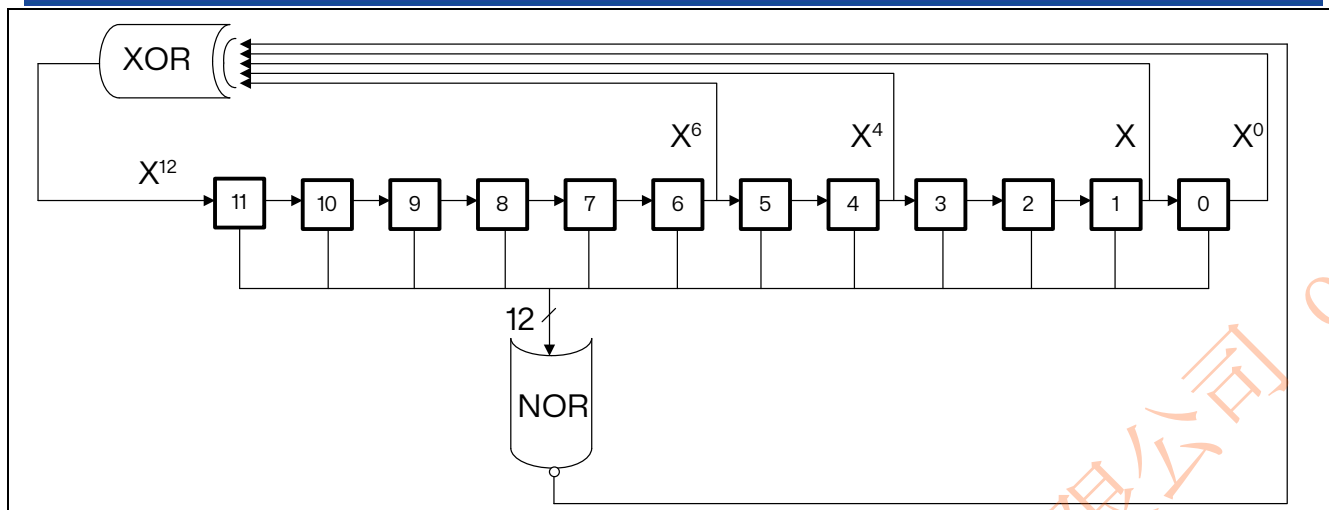


图 17.5 DAC LFSR 寄存器计算算法

LFSR 值可以部分或全部被 DAC\_CR 寄存器中的 MAMPx[3:0]位掩盖，它被加到 DAC\_DHRx 内容中而不会溢出，然后这个值被转移到 DAC\_DORx 寄存器中。

如果 LFSR 为 0x0000，则向其注入 ‘1’（防锁定机制）。

可以通过复位 WAVEx[1:0]位来复位 LFSR 波的产生。

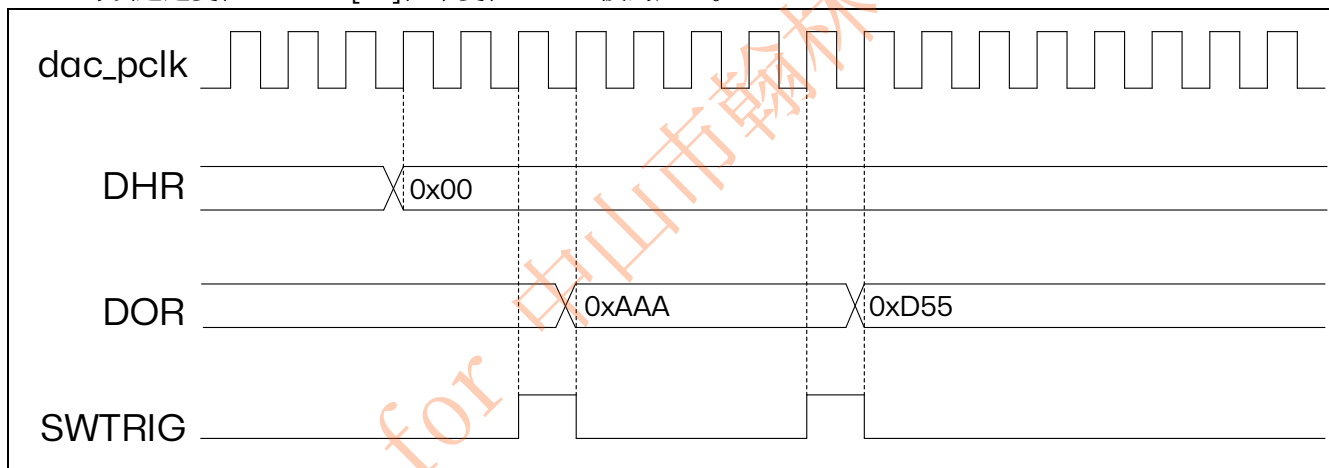


图 17.6 DAC 转换（SW 触发使能）与 LFSR 波产生

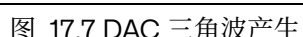
注意：DAC 触发器必须通过在 DAC\_CR 寄存器中设置 TENx 位来使能噪声产生。

### 17.3.7 三角波生成

可以在 DC 或者缓慢变化的信号上加一个小幅度的三角波。设置 WAVEx[1:0]位为 ‘10’ 选择 DAC 的三角波生成功能。设置 DAC\_CR 寄存器的 MAMPx[3:0]位来选择三角波的幅度。内部的三角波计数器每次触发事件之后 3 个 HCLK 时钟周期后累加 1。计数器的值与 DAC\_DHRx 寄存器的数值相加并丢弃溢出位后写入 DAC\_DORx 寄存器。在传入 DAC\_DORx 寄存器的数值小于 MAMP[3:0]位定义的最大幅度时，三角波计数器逐步累加。一旦达到设置的最大幅度，则计数器开始递减，达到 0 后再开始累加，周而复始。

将 WAVEx[1:0]位置 ‘0’ 可以复位三角波的生成。





2. MAMP[3:0]位必须在使能 DAC 之前设置，否则其值不能修改。

DAC 可以产生锯齿形波形。需要对初始值、增量值和方向控制进行特定的寄存器设置:

- 通过在 DAC\_CR 寄存器中将 WAVEx[1:0] 设置为 11 来选择 DAC 锯齿波的产生。
- 锯齿计数器的初始值（重置值）通过 DAC\_STRx 寄存器中的 STRSTDATAx[11:0] 位配置。
- 增量值由 DAC\_STRx 寄存器中的 STINCDATAx[15:0] 位定义。
- 锯齿方向由 DAC\_STRx 寄存器中的 STDIRx 位定义。

DAC 输出使用那些计数器值的 12MSB。当计数器达到 0x0000 或 0xFFFF 时，表示该值饱和。锯齿复位触发信号将计数器的值初始化为 STRSTDATAx[11:0](第 12 位到第 15 位设置为 0000)值。

增量触发器和复位触发器必须通过 STINCTRIGSELx[3:0]和 STRSTTRIGSELx[3:0]位来选择。

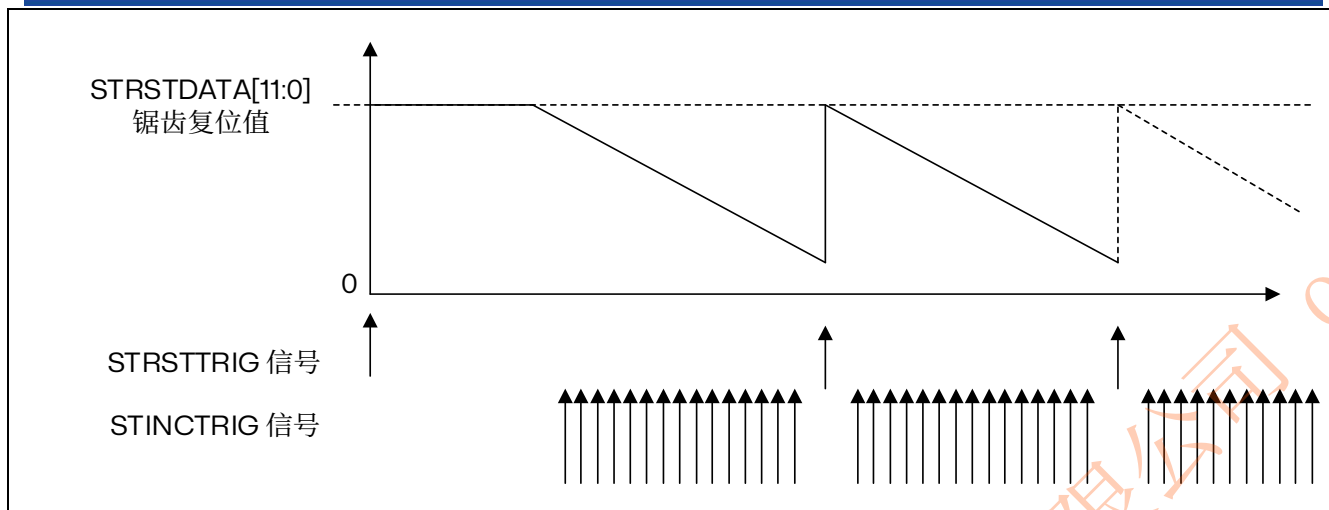


图 17.9 DAC 锯齿波产生 (STDIRx = 0)

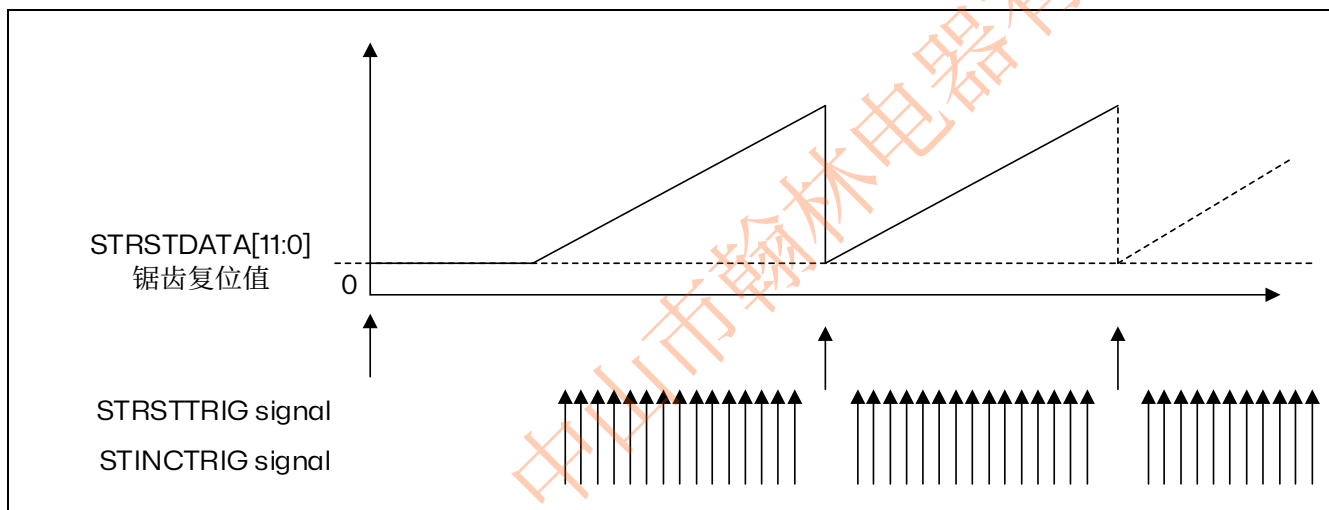


图 17.10 DAC 锯齿波产生 (STDIRx = 1)

strstrtrig 信号的优先级高于 STINCTRIG 信号。触发信号不能快于表 187:HFSEL 描述中定义的 DAC\_DORx 更新速率。如果断言的 STINCTRIG 快于允许的数据更新速率，则忽略 STINCTRIG 触发器。如果 strstrtrig 信号在 STINCTRIG 之后和 DAC\_DORx 更新速率约束之前应用，则 strstrtrig 被搁置。然后，在数据增加之后，立即应用重置触发器。

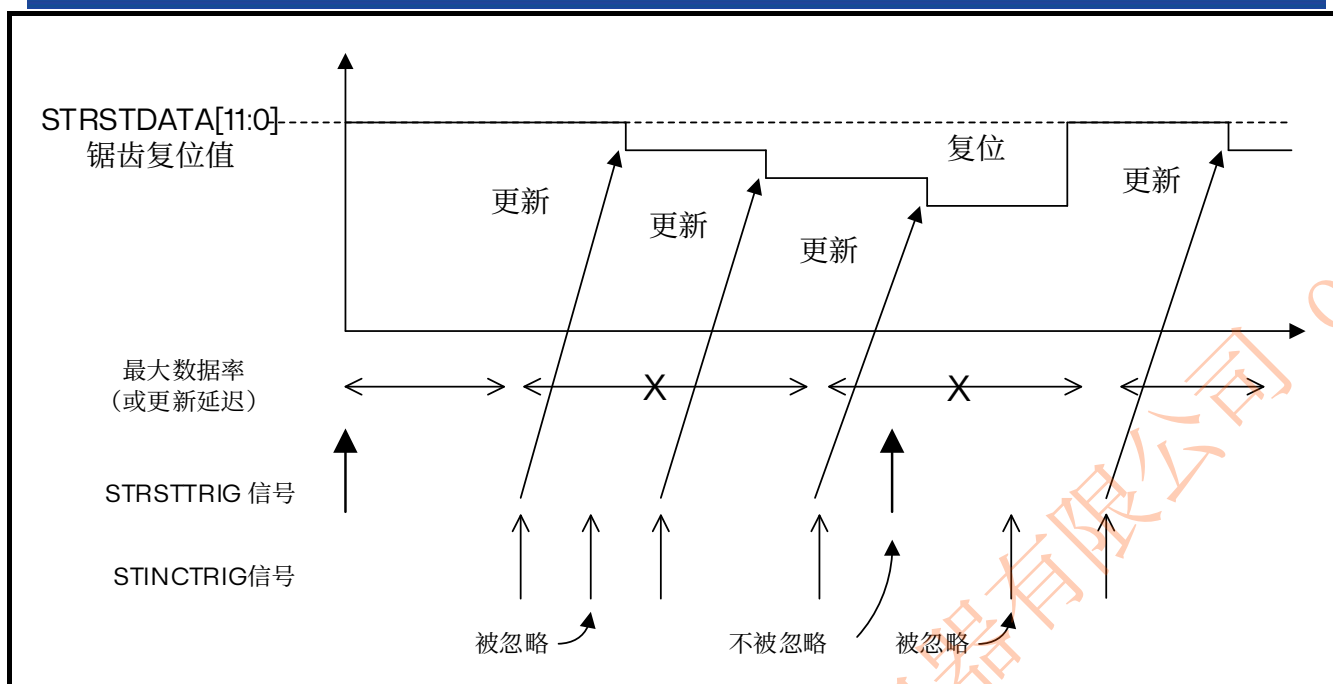


图 17.11 DAC 锯齿形 STINCTRIG 和 STRSTTRIG 优先级

### 17.3.9 DAC 通道缓冲校准

n 位数模转换器(DAC)的传递函数为:

$$V_{OUT} = ((D/2^N) \times G \times V_{ref}) + V_{OS}$$

其中  $V_{OUT}$  为模拟输出， $D$  为数字输入， $G$  为增益， $V_{REF}$  为标称满量程电压， $V_{OS}$  为失调电压。对于理想的 DAC 通道， $G = 1$ ， $V_{OS} = 0$ 。

由于输出缓冲器的特性，电压偏置可能因部分而异，并在模拟输出上引入绝对偏置误差。为了补偿  $V_{os}$ ，需要进行校准。

校准仅在 DAC 通道 x 在使能缓冲区的情况下运行时有效。如果在缓冲区关闭时应用于其他模式，则没有效果。

- 缓冲输出与引脚内部/外部连接断开，并置于三状态模式 (HiZ)。
- 缓冲器作为比较器来感知中间码值 0x800，并通过内部桥将其与 VREF+/2 信号进行比较，然后根据比较结果(CAL FLAGx 位)将其输出信号切换为 0 或 1。

提供两种校准技术:

- 工厂微调（默认设置）

DAC 缓冲偏移量是工厂微调。在 DAC\_CCR 寄存器中的 OTRIMx[4:0]位的默认值是工厂微调值，一旦 DAC 数字接口复位就加载它。

- ## ● 用户微调

当操作条件与标称工厂微调条件不同时，特别是当 VDDA 电压，温度，VREF+ 值发生变化时，用户微调可以完成，并且可以在应用过程中通过软件在任何时候完成。

注：请参阅数据表，了解标称工厂微调条件的更多详细信息。此外，当移除VDD（例如设备进入STANDBY或VBAT模式）时，需要进行校准。

执行用户微调校准的步骤如下:

1. 如果 DAC 通道是活动的, 将 0 写入 DAC\_CR 中的 ENx 位以禁用该通道。

2. BOFFx 位写入 0, 使能 DAC 通道输出缓存
3. 通过将 DAC\_CR 寄存器中的 CENx 位设置为 1, 启动 DAC 通道校准。
4. 应用微调算法:
  - a) 将代码写入 OTRIMx[4:0]位, 从 00000b 开始。
  - b) 等待  $t_{OFFTRIMmax}$  延时。
  - c) 检查 DAC\_SR 中的 CAL\_FLAGx 位是否设置为 1。
  - d) 如果 CAL\_FLAGx 设置为 1, 则找到 OTRIMx[4:0]微调代码, 可以在设备运行期间使用该代码来补偿输出值, 否则增加 OTRIMx[4:0]并再次重复从(a)到(d)的子步骤。

软件算法可以使用连续逼近或二分法技术来更快地计算和设置 OTRIMx[4:0]位的内容。

CAL\_FLAGx 位的交换/开关表明偏移量被正确补偿, 相应的修剪代码必须保存在 DAC\_CCR 寄存器的 OTRIMx[4:0]位中。

注意: 在写入 OTRIMx[4:0]位和读取 DAC\_SR 寄存器中的 CAL\_FLAGx 位之间必须遵守  $t_{TRIM}$  延迟, 以便获得正确的值。该参数在数据表电气特性部分中指定。

如果 VDDA, VREF+和温度条件在设备运行过程中没有变化, 而它更经常进入待机和 VBAT 模式, 软件可能会将在第一次用户校准中找到的 OTRIMx[4:0]位存储在闪存或备用寄存器中。然后在设备再次供电时直接加载/写入它们, 从而避免等待新的校准时间。

当设置了 CENx 位时, 不允许设置 ENx 位。

### 17.3.10 双 DAC 通道转换模式 (如果有双通道)

为了在需要同时使用两个 DAC 通道的应用中有效地利用总线带宽, 实现了三个双寄存器:DHR8RD、DHR12RD 和 DHR12LD。然后需要一个唯一的寄存器访问来同时驱动两个 DAC 通道。对于波的生成, 不需要访问 DHRxxxD 寄存器。因此, 两个输出通道可以独立或同时使用。

使用两个 DAC 通道和这些双寄存器可以实现 15 种转换模式。但是, 如果需要, 可以使用单独的 DHRx 寄存器获得所有转换模式。

所有的模式都在下面的段落中描述。

#### 独立触发, 不产生波浪

采用该转换方式配置 DAC 时, 需要按照以下顺序配置:

1. 配置 DAC。设置两个 DAC 通道触发使能位 TEN1 和 TEN2。
2. 通过在 TSEL1 和 TSEL2 位域中设置不同的值来配置不同的触发源。
3. 将双 DAC 通道数据加载到所需的 DHR 寄存器 (DAC\_DHR12RD、DAC\_DHR12LD 或 DAC\_DHR8RD)。

当 DAC 通道 1 触发器到达时, DHR1 寄存器被转移到 DAC\_DOR1 (三个 dac\_hclk 时钟周期之后)。

当 DAC 通道 2 触发器到达时, DHR2 寄存器被转移到 DAC\_DOR2 (三个 dac\_hclk 时钟周期之后)。

#### 具有单 LFSR 生成的独立触发器

采用该转换方式配置 DAC 时, 需要按照以下顺序配置:

- 1.配置 DAC。设置两个 DAC 通道触发使能位 TEN1 和 TEN2。
2. 通过在 TSEL1 和 TSEL2 位域中设置不同的值来配置不同的触发源。
3. 配置两个 DAC 通道 WAVEx[1:0]位为 01, 并且在 MAMPx[3:0]位中配置相同的 LFSR 掩码值。
4. 将双 DAC 通道数据加载到所需的 DHR 寄存器(DAC\_DHR12RD、DAC\_DHR12LD 或 DAC\_DHR8RD)。

当 DAC channel1 触发器到达时, 具有相同掩码的 LFSR1 计数器被添加到 DHR1 寄存器中, 并将总和传输到 DAC\_DOR1(三个 dac\_hclk 时钟周期之后)。然后更新 LFSR1 计数器。

当 DAC 通道 2 触发器到达时, 具有相同掩码的 LFSR2 计数器被添加到 DHR2 寄存器中, 并将总和传输到 DAC\_DOR2(三个 dac\_hclk 时钟周期之后)。然后更新 LFSR2 计数器。

#### 具有不同 LFSR 生成的独立触发器

采用该转换方式配置 DAC 时, 需要按照以下顺序配置:

1. 配置 DAC。设置两个 DAC 通道触发使能位 TEN1 和 TEN2。
2. 通过在 TSEL1 和 TSEL2 位域中设置不同的值来配置不同的触发源。
3. 配置两个 DAC 通道 WAVEx[1:0] 位为 01, 并在 MAMP1[3:0] 和 MAMP2[3:0] 位中设置不同的 LFSR 掩码值。
4. 将双 DAC 通道数据加载到所需的 DHR 寄存器(DAC\_DHR12RD、DAC\_DHR12LD 或 DAC\_DHR8RD)。

当 DAC channel1 触发器到达时, LFSR1 计数器与 MAMP1[3:0] 配置的掩码一起被添加到 DHR1 寄存器中, 并且总和被转移到 DAC\_DOR1(三个 dac\_hclk 时钟周期后)。然后更新 LFSR1 计数器。

当一个 DAC 通道 2 触发器到达时, LFSR2 计数器, 带着 MAMP2[3:0] 配置的掩码, 被添加到 DHR2 寄存器中, 总和被转移到 DAC\_DOR2(三个 dac\_hclk 时钟周期后)。然后更新 LFSR2 计数器。

#### 独立触发器, 单三角生成

采用该转换方式配置 DAC 时, 需要按照以下顺序配置:

1. 配置 DAC。设置两个 DAC 通道触发使能位 TEN1 和 TEN2。
2. 通过在 TSEL1 和 TSEL2 位域中设置不同的值来配置不同的触发源。
3. 将两个 DAC 通道 WAVEx[1:0] 位配置为 1x, 并且在 MAMPx[3:0] 位中设置相同的最大幅度值。
4. 将双 DAC 通道数据加载到所需的 DHR 寄存器 (DAC\_DHR12RD、DAC\_DHR12LD 或 DAC\_DHR8RD)。

当 DAC channel1 触发器到达时, 具有相同三角形幅度的 DAC channel1 三角形计数器被添加到 DHR1 寄存器中, 并将总和转移到 DAC\_DOR1 (三个 dac\_hclk 时钟周期之后)。然后更新 DAC channel1 三角形计数器。

当 DAC 通道 2 触发器到达时, 具有相同三角形幅度的 DAC 通道 2 三角形计数器被添加到 DHR2 寄存器中, 并将总和转移到 DAC\_DOR2 (三个 dac\_hclk 时钟周期之后)。然后更新 DAC 通道 2 三角形计数器。

#### 不同三角形生成的独立触发器

采用该转换方式配置 DAC 时, 需要按照以下顺序配置:

1. 配置 DAC。设置两个 DAC 通道触发使能位 TEN1 和 TEN2。
2. 通过在 TSEL1 和 TSEL2 位中设置不同的值来配置不同的触发源。
3. 将两个 DAC 通道 WAVEx[1:0] 位配置为 1x, 并在 MAMP1[3:0] 和 MAMP2[3:0] 位中设置不同的最大幅度值。
4. 将双 DAC 通道数据加载到所需的 DHR 寄存器 (DAC\_DHR12RD、DAC\_DHR12LD 或 DAC\_DHR8RD)。

当 DAC channel1 触发器到达时, DAC channel1 三角形计数器, 其三角形幅度由 MAMP1[3:0] 配置, 被添加到 DHR1 寄存器中, 其总和为传输到 DAC\_DOR1 (三个 dac\_hclk 时钟周期后)。然后更新 DAC channel1 三角形计数器。



当 DAC channel2 触发器到达时, DAC channel2 三角形计数器, 其三角形幅度由 MAMP2[3:0]配置, 被添加到 DHR2 寄存器中, 并将总和转移到 DAC\_DOR2 (三个 dac\_hclk 时钟周期后)。然后更新 DAC 通道 2 三角形计数器。

### 具有单锯齿生成的独立触发器

采用该转换方式配置 DAC 时, 需要按照以下顺序配置:

1. 配置 DAC。通过在 STRSTTRIGSEL1[3:0], STRSTTRIGSEL2[3:0], STINCTRIGSEL2[3:0]和 STINCTRIGSEL1[3:0]位中设置不同的值来配置不同的触发源。
2. 配置两个 DAC 通道 WAVEx[1:0]位为 11, 并为每个寄存器设置相同的 STRSTDATAx[11:0], STINCDAx[15:0]和 STDIRx 值。

当 DAC channel1 触发器到达时, DAC channel1 锯齿计数器更新 dh1 寄存器并将其传输到 DAC\_DOR1 (三个 AHB 时钟周期之后)。

当 DAC channel2 触发器到达时, DAC channel2 锯齿计数器更新 dh1 寄存器并将其传输到 DAC\_DOR1 (三个 AHB 时钟周期之后)。

### 具有不同锯齿波产生的独立触发器

采用该转换方式配置 DAC 时, 需要按照以下顺序配置:

1. 配置 DAC。通过在 STRSTTRIGSEL1[23:0], STRSTTRIGSEL2[23:0], STINCTRIGSEL2[3:0]和 STINCTRIGSEL1[3:0]位中设置不同的值来配置不同的触发源。
2. 配置两个 DAC 通道 WAVEx[1:0]位为 11, 并为每个寄存器设置不同的 STRSTDATAx[11:0], STINCDAx[15:0]和 STDIRx 值。

当 DAC channel1 触发器到达时, DAC channel1 锯齿计数器更新 dh1 寄存器并将其加载到 DAC\_DOR1 (三个 AHB 时钟周期之后)。

当 DAC channel2 触发器到达时, DAC channel2 锯齿计数器更新 DHR2 寄存器并将其加载到 DAC\_DOR1 (三个 AHB 时钟周期之后)。

### 软件同步启动

采用该转换方式配置 DAC 时, 需要按以下顺序配置:

- 将双 DAC 通道数据加载到所需的 DHR 寄存器(DAC\_DHR12RD、DAC\_DHR12LD 或 DAC\_DHR8RD)。

在这个配置中, 一个 dac\_hclk 时钟周期之后, DHR1 和 DHR2 寄存器分别被转移到 DAC\_DOR1 和 DAC\_DOR2 中。

### 同时触发, 不产生波浪

采用该转换方式配置 DAC 时, 需要按照以下顺序配置:

1. 配置 DAC。设置两个 DAC 通道触发使能位 TEN1 和 TEN2。
2. 通过在 TSEL1 和 TSEL2 位域中设置相同的值, 为两个 DAC 通道配置相同的触发源。
3. 将双 DAC 通道数据加载到所需的 DHR 寄存器 (DAC\_DHR12RD、DAC\_DHR12LD 或 DAC\_DHR8RD)。

当触发器到达时, DHR1 和 DHR2 寄存器分别被转移到 DAC\_DOR1 和 DAC\_DOR2 中 (在三个 dac\_hclk 时钟周期之后)。

### 同时触发单 LFSR 产生

1. 采用该转换方式配置 DAC 时，需要遵循以下顺序：
2. 设置两个 DAC 通道触发使能位 TEN1 和 TEN2。
3. 通过在 TSEL1 和 TSEL2 位域中设置相同的值，为两个 DAC 通道配置相同的触发源。
4. 配置两个 DAC 通道 WAVEx[1:0] 位为 01，并且在 MAMPx[3:0] 位中配置相同的 LFSR 掩码值。
5. 将双 DAC 通道数据加载到所需的 DHR 寄存器（DHR12RD、DHR12LD 或 DHR8RD）。

当触发器到达时，将具有相同掩码的 LFSR1 计数器添加到 DHR1 寄存器中，并将总和传输到 DAC\_DOR1（三个 dac\_hclk 时钟周期之后）。然后更新 LFSR1 计数器。与此同时，具有相同掩码的 LFSR2 计数器被添加到 DHR2 寄存器中，并将总和传输到 DAC\_DOR2（三个 dac\_hclk 时钟周期后）。然后更新 LFSR2 计数器。

#### 同时触发不同 LFSR 的产生

采用该转换方式配置 DAC 时，需要按照以下顺序配置：

1. 配置 DAC。设置两个 DAC 通道触发使能位 TEN1 和 TEN2
2. 通过在 TSEL1 和 TSEL2 位域中设置相同的值，为两个 DAC 通道配置相同的触发源。
3. 配置两个 DAC 通道 WAVEx[1:0] 位为 01，并使用 MAMP1[3:0] 和 MAMP2[3:0] 位设置不同的 LFSR 掩码值。
4. 将双 DAC 通道数据加载到所需的 DHR 寄存器（DAC\_DHR12RD、DAC\_DHR12LD 或 DAC\_DHR8RD）。

当触发器到达时，带有 MAMP1[3:0] 配置的掩码的 LFSR1 计数器被添加到 DHR1 寄存器中，并且总和被转移到 DAC\_DOR1（三个 dac\_hclk 时钟周期后）。然后更新 LFSR1 计数器。

同时，使用 MAMP2[3:0] 配置掩码的 LFSR2 计数器被添加到 DHR2 寄存器中，并在三个 dac\_hclk 时钟周期后将总和转移到 DAC\_DOR2 中。然后更新 LFSR2 计数器。

#### 同时触发单三角生成

采用该转换方式配置 DAC 时，需要按照以下顺序配置：

1. 配置 DAC。设置两个 DAC 通道触发使能位 TEN1 和 TEN2
2. 通过在 TSEL1 和 TSEL2 位域中设置相同的值，为两个 DAC 通道配置相同的触发源。
3. 将两个 DAC 通道 WAVEx[1:0] 位配置为 1x，使用 MAMPx[3:0] 位配置相同的最大幅度值。
4. 将双 DAC 通道数据加载到所需的 DHR 寄存器（DAC\_DHR12RD、DAC\_DHR12LD 或 DAC\_DHR8RD）。

当触发器到达时，具有相同三角形幅度的 DAC 通道 1 三角形计数器被添加到 DHR1 寄存器中，并将总和转移到 DAC\_DOR1（三个 dac\_hclk 时钟周期之后）。然后更新 DAC channel1 三角形计数器。

同时，将具有相同三角形幅度的 DAC 通道 2 三角形计数器添加到 DHR2 寄存器中，并将其和传输到 DAC\_DOR2（三个 dac\_hclk 时钟周期后）。然后更新 DAC 通道 2 三角形计数器。

#### 同时触发不同三角形生成

采用该转换方式配置 DAC 时，需要按照以下顺序配置：

1. 配置 DAC。设置两个 DAC 通道触发使能位 TEN1 和 TEN2
2. 通过在 TSEL1 和 TSEL2 位域中设置相同的值，为两个 DAC 通道配置相同的触发源。
3. 将两个 DAC 通道 WAVEx[1:0] 位配置为 1x，并在 MAMP1[3:0] 和 MAMP2[3:0] 位中设置不同的最大幅度值。
4. 将双 DAC 通道数据加载到所需的 DHR 寄存器（DAC\_DHR12RD、DAC\_DHR12LD 或 DAC\_DHR8RD）。



当触发器到达时，带有 MAMP1[3:0]配置的三角形幅度的 DAC 通道 1 三角形计数器被添加到 DHR1 寄存器中，并且总和被转移到 DAC\_DOR1（三个 AHB 时钟周期后）。然后更新 DAC channel1 三角形计数器。

与此同时，DAC 通道 2 三角形计数器，其三角形幅度由 MAMP2[3:0]配置，被添加到 DHR2 寄存器中，并将总和转移到 DAC\_DOR2（三个 dac\_hclk 时钟周期后）。然后更新 DAC channel2 三角形计数器。

#### 同时触发单锯齿波产生

采用该转换方式配置 DAC 时，需要按照以下顺序配置：

1. 配置 DAC。通过在 STRSTTRIGSEL1[3:0]，STRSTTRIGSEL2[3:0]，STINCTRIGSEL2[3:0]和 STINCTRIGSEL1[3:0]位中设置相同的值，为两个 DAC 通道配置相同的触发源。
2. 配置两个 DAC 通道 WAVEx[1:0]位为 11，并为每个寄存器设置相同的 STRSTDATAx[11:0]，STINCDAx[15:0]和 STDIRx 值。

当触发器到达时，DAC 通道 1/2 锯齿计数器更新 DHR1 和 DHR2 寄存器并将它们加载到 DAC\_DOR1/2（三个 AHB 时钟周期之后）。

#### 同时触发不同锯齿波产生

采用该转换方式配置 DAC 时，需要按照以下顺序配置：

1. 配置 DAC。通过在 STRSTTRIGSEL1[3:0]，STRSTTRIGSEL2[3:0]，STINCTRIGSEL2[3:0] bits 和 STINCTRIGSEL1[3:0] bits 中设置相同的值，为两个 DAC 通道配置相同的触发源。
2. 配置两个 DAC 通道 WAVEx[1:0]位为 11，并为每个寄存器设置不同的 STRSTDATAx[11:0]，STINCDAx[15:0]，STDIRx 值。

当触发器到达时，DAC 通道 1/2 锯齿计数器更新 DHR1 和 DHR2 寄存器，并将它们独立地加载到 DAC\_DOR1/2（三个 AHB 时钟周期后）。

### 17.3.1 低功耗模式的 DAC

| 低功耗模式   | DAC 运行情况 |
|---------|----------|
| Sleep   | 可运行      |
|         | 可运行      |
| Stop    | 可运行      |
|         | 可运行      |
| Standby | 关闭       |
|         | 关闭       |

## 17.4 DAC 寄存器描述

有关寄存器说明中使用的缩写，请参见第 2 章节：文档约定中的第 1 节。

外设寄存器必须按字（32 位）进行访问。

### 17.4.1 DAC 控制寄存器（DAC\_CR）

偏移地址：0x0000

复位值：0x0000 0000

| 31    | 30   | 29            | 28         | 27         | 26 | 25 | 24 | 23         | 22 | 21         | 20 | 19 | 18 | 17   | 16  |
|-------|------|---------------|------------|------------|----|----|----|------------|----|------------|----|----|----|------|-----|
| BOFF2 | CEN2 | DMA<br>UDRIE2 | DMA<br>EN2 | MAMP2[3:0] |    |    |    | WAVE2[1:0] |    | TSEL2[3:0] |    |    |    | TEN2 | EN2 |
| rw    | rw   | rw            | rw         | rw         | rw | rw | rw | rw         | rw | rw         | rw | rw | rw | rw   | rw  |
| 15    | 14   | 13            | 12         | 11         | 10 | 9  | 8  | 7          | 6  | 5          | 4  | 3  | 2  | 1    | 0   |
| BOFF1 | CEN1 | DMA<br>UDRIE1 | DMA<br>EN1 | MAMP1[3:0] |    |    |    | WAVE1[1:0] |    | TSEL1[3:0] |    |    |    | TEN1 | EN1 |
| rw    | rw   | rw            | rw         | rw         | rw | rw | rw | rw         | rw | rw         | rw | rw | rw | rw   | rw  |

Bit 31 **BOFF2**: 关闭 DAC 通道 2 输出缓存

该位由软件设置和清除，用来使能/关闭 DAC 通道 2 的输出缓存。

0: 使能 DAC 通道 2 输出缓存

1: 关闭 DAC 通道 2 输出缓存。

注：DAC3 通道 2 走内部，无 buffer。

注：BOFF on 时只能对外输出至 IO；BOFF off 时只能走内部通道给 CMP，内外部通道无法同时使用

Bit 30 **CEN2**: DAC 通道 2 校准使能

此位由软件置 1 和清零，用于使能/禁止 DAC 通道 2 校准，仅当 DAC\_CR 中的位 EN2=0 时才写入（仅当 DAC 通道已禁止时才可以进入/退出校准模式），否则会忽略写操作。

0: DAC 通道 2 处于正常运行模式

1: DAC 通道 2 处于校准模式

注：DAC2 通道 2 和 DAC3 通道 2 走内部，无校准。

Bit 29 **DMAUDRIE2**: DAC 通道 2 DMA 下溢中断使能

此位由软件置 1 和清零。

0: 禁止 DAC2 通道 DMA 下溢中断

1: 使能 DAC2 通道 DMA 下溢中断

Bit 28 **DMAEN2**: DAC 通道 2 DMA 使能

此位由软件置 1 和清零。

0: 关闭 DAC 通道 2 DMA 模式；

1: 使能 DAC 通道 2 DMA 模式。

**Bit 27:24 MAMP2[3:0]: DAC 通道 2 掩码/振幅选择器**

这些位由软件写入，用于在生成噪声波模式下选择掩码，或者在生成三角波模式下选择振幅。

- 0000: 不屏蔽 LSFR 位 0 / 三角波振幅等于 1;
- 0001: 不屏蔽 LSFR 位[1:0] / 三角波振幅等于 3;
- 0010: 不屏蔽 LSFR 位[2:0] / 三角波振幅等于 7;
- 0011: 不屏蔽 LSFR 位[3:0] / 三角波振幅等于 15;
- 0100: 不屏蔽 LSFR 位[4:0] / 三角波振幅等于 31;
- 0101: 不屏蔽 LSFR 位[5:0] / 三角波振幅等于 63;
- 0110: 不屏蔽 LSFR 位[6:0] / 三角波振幅等于 127;
- 0111: 不屏蔽 LSFR 位[7:0] / 三角波振幅等于 255;
- 1000: 不屏蔽 LSFR 位[8:0] / 三角波振幅等于 511;
- 1001: 不屏蔽 LSFR 位[9:0] / 三角波振幅等于 1023;
- 1010: 不屏蔽 LSFR 位[10:0] / 三角波振幅等于 2047;
- ≥1011: 不屏蔽 LSFR 位[11:0] / 三角波振幅等于 4095。

**Bit 23:22 WAVE2[1:0]: DAC 通道 2 噪声/三角波生成使能**

这些位由软件置 1 或清零。

- 00: 关闭波形生成;
- 01: 使能噪声波形发生器;
- 10: 使能三角波发生器;
- 11: 使能锯齿波发生器

注: 只在位 TEN2 = 1 (使能 DAC 通道 2 触发) 时使用

**Bit 21:18 TSEL2[3:0]: DAC 通道 2 触发选择**

该位用于选择 DAC 通道 2 的外部触发事件。

- 0000: SWTRIG1
- 0001: dac\_ch1\_trg1
- ....
- 1111: dac\_ch1\_trg15

注意: 该位只能在 TEN1=1 (DAC 通道 2 触发使能) 时生效。

**Bit 17 TEN2: DAC 通道 2 触发使能**

该位由软件设置和清除，用来使能/关闭 DAC 通道 1 的触发。

- 0: 关闭 DAC 通道 2 触发，写入寄存器 DAC\_DHRx 的数据在 1 个 APB1 时钟周期后自动传入寄存器 DAC\_DOR1;
- 1: 使能 DAC 通道 2 触发，(TESL1 决定触发源) 写入寄存器 DAC\_DHRx 的数据在 3 个 APB1 时钟周期后传入寄存器 DAC\_DOR1。

注意: 如果选择软件触发，写入寄存器 DAC\_DHRx 的数据只需要 1 个 APB1 时钟周期就可以传入寄存器 DAC\_DOR1。

**Bit 16 EN2: DAC 通道 2 使能**

该位由软件设置和清除，用来使能/失能 DAC 通道 2。

- 0: 关闭 DAC 通道 2;
- 1: 使能 DAC 通道 2。

**Bit 15 BOFF1: 关闭 DAC 通道 1 输出缓存**

该位由软件设置和清除，用来使能/关闭 DAC 通道 1 的输出缓存。

0: 使能 DAC 通道 1 输出缓存

1: 关闭 DAC 通道 1 输出缓存。

注: DAC3 通道 1 走内部, 无 buffer。

注: BOFF on 时只能对外输出至 IO; BOFF off 时只能走内部通道给 CMP, 内外部通道无法同时使用

**Bit 14 CEN1:** DAC 通道 1 校准使能

该位由软件置 1 和清零，用于使能/禁止 DAC 通道 1 校准，仅当 DAC\_CR 中的位 EN1=0 时才可写入（仅当 DAC 通道已禁止时才可以进入/退出校准模式），否则会忽略写操作。

0: DAC 通道 1 处于正常运行模式

1: DAC 通道 1 处于校准模式

注: (DAC3 通道 1 走内部, 无校准。)

**Bit 13 DMAUDRIE1:** DAC1 通道 DMA 下溢中断使能

此位由软件置 1 和清零。

0: 禁止 DAC1 通道 DMA 下溢中断

1: 使能 DAC1 通道 DMA 下溢中断

**Bit 12 DMAEN1:** DAC 通道 1 DMA 使能

该位由软件设置和清除。

0: 关闭 DAC 通道 1 DMA 模式;

1: 使能 DAC 通道 1 DMA 模式。

**Bit 11:8 MAMP1[3:0]:** DAC 通道 1 屏蔽/幅值选择器

由软件设置，用来在噪声生成模式下选择屏蔽位，在三角波生成模式下选择波形的幅值。

0000: 不屏蔽 LSFR 位 0 / 三角波幅值等于 1;

0001: 不屏蔽 LSFR 位[1:0] / 三角波幅值等于 3;

0010: 不屏蔽 LSFR 位[2:0] / 三角波幅值等于 7;

0011: 不屏蔽 LSFR 位[3:0] / 三角波幅值等于 15;

0100: 不屏蔽 LSFR 位[4:0] / 三角波幅值等于 31;

0101: 不屏蔽 LSFR 位[5:0] / 三角波幅值等于 63;

0110: 不屏蔽 LSFR 位[6:0] / 三角波幅值等于 127;

0111: 不屏蔽 LSFR 位[7:0] / 三角波幅值等于 255;

1000: 不屏蔽 LSFR 位[8:0] / 三角波幅值等于 511;

1001: 不屏蔽 LSFR 位[9:0] / 三角波幅值等于 1023;

1010: 不屏蔽 LSFR 位[10:0] / 三角波幅值等于 2047;

≥1011: 不屏蔽 LSFR 位[11:0] / 三角波幅值等于 4095。

**Bit 7:6 WAVE1[1:0]:** DAC 通道 1 噪声/三角波生成使能

由软件设置和清除。

00: 关闭波形生成;

01: 使能噪声波形发生器;

10: 使能三角波发生器;

11: 使能锯齿波发生器

**Bits 5:2 TSEL1[3:0]:** DAC 通道 1 触发选择

该位用于选择 DAC 通道 1 的外部触发事件。

0000: SWTRIG1

0001: dac\_ch1\_trg1

....

1111: dac\_ch1\_trg15

注意：该位只能在 TEN1=1（DAC 通道 1 触发使能）时生效。

#### Bit 1 TEN1: DAC 通道 1 触发使能

该位由软件设置和清除，用来使能/关闭 DAC 通道 1 的触发。

0: 关闭 DAC 通道 1 触发，写入寄存器 DAC\_DHRx 的数据在 1 个 APB1 时钟周期后自动传入寄存器 DAC\_DOR1;

1: 使能 DAC 通道 1 触发，（TESL1 决定触发源）写入寄存器 DAC\_DHRx 的数据在 3 个 APB1 时钟周期后传入寄存器 DAC\_DOR1。

注意：如果选择软件触发，写入寄存器 DAC\_DHRx 的数据只需要 1 个 APB1 时钟周期就可以传入寄存器 DAC\_DOR1。

#### Bit 0 EN1: DAC 通道 1 使能

该位由软件设置和清除，用来使能/失能 DAC 通道 1。

0: 关闭 DAC 通道 1;

1: 使能 DAC 通道 1。

### 17.4.2 DAC 软件触发寄存器 (DAC\_SWTRIGR)

偏移地址: 0x0004

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |    |    |    |    |    |    |          |          |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----------|----------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17       | 16       |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    | SWTRIGB2 | SWTRIGB1 |
|          |    |    |    |    |    |    |    |    |    |    |    |    |    | rw       | rw       |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1        | 0        |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    | SWTRIG2  | SWTRIG1  |
|          |    |    |    |    |    |    |    |    |    |    |    |    |    | rw       | rw       |

Bits 31:18 保留，必须保持为复位值。

#### Bit 17 SWTRIGB2: DAC channel 1 软件触发 B

该位由软件在软件触发模式（锯齿形生成）中触发 DAC 的软件设置的它被硬件清除了。

0: 没有扳机

1: 锯齿形增量触发

#### Bit 16 SWTRIGB1: DAC channel 2 软件触发 B

该位由软件在软件触发模式（锯齿形生成）中触发 DAC 的软件设置的它被硬件清除了。

0: 没有扳机

1: 锯齿形增量触发

Bits 15:2 保留，必须保持为复位值。

#### Bit 1 SWTRIG2: DAC 通道 2 软件触发

该位由软件设置和清除，用来使能/关闭软件触发。

0: 关闭 DAC 通道 2 软件触发;

1: 使能 DAC 通道 2 软件触发。

注意：一旦寄存器 DAC\_DHR2 的数据传入寄存器 DAC\_DOR2, (1 个 APB1 时钟周期后)该位由硬件置 '0'。

Bit 0 **SWTRIG1**: DAC 通道 1 软件触发

该位由软件设置和清除, 用来使能/关闭软件触发。

0: 关闭 DAC 通道 1 软件触发;

1: 使能 DAC 通道 1 软件触发。

注意：一旦寄存器 DAC\_DHR1 的数据传入寄存器 DAC\_DOR1, (1 个 APB1 时钟周期后)该位由硬件置 '0'。

### 17.4.3 DAC 通道 1 12 位右对齐数据保持寄存器 (DAC\_DHR12R1)

偏移地址: 0x0008

复位值: 0x0000 0000

|          |    |    |    |                |    |    |    |    |    |    |    |    |    |    |    |
|----------|----|----|----|----------------|----|----|----|----|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27             | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |                |    |    |    |    |    |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11             | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    | DACC1DHR[11:0] |    |    |    |    |    |    |    |    |    |    |    |
|          |    |    |    | rw             | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:12 保留, 必须保持为复位值。

Bits 11:0 **DACC1DHR[11:0]**: DAC 通道 1 的 12 位右对齐数据

该位由软件写入, 表示 DAC 通道 1 的 12 位数据。

### 17.4.4 DAC 通道 1 12 位左对齐数据保持寄存器 (DAC\_DHR12L1)

偏移地址: 0x000C

复位值: 0x0000 0000

|                |    |    |    |    |    |    |    |    |    |    |    |          |    |    |    |
|----------------|----|----|----|----|----|----|----|----|----|----|----|----------|----|----|----|
| 31             | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19       | 18 | 17 | 16 |
| Reserved       |    |    |    |    |    |    |    |    |    |    |    |          |    |    |    |
| 15             | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3        | 2  | 1  | 0  |
| DACC1DHR[11:0] |    |    |    |    |    |    |    |    |    |    |    | Reserved |    |    |    |
| rw             | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |          |    |    |    |

Bits 31:16 保留, 必须保持为复位值。

Bits 15:4 **DACC1DHR[11:0]**: DAC 通道 1 的 12 位左对齐数据

该位由软件写入, 表示 DAC 通道 1 的 12 位数据。

Bits 3:0 保留, 必须保持为复位值。

### 17.4.5 DAC 通道 1 8 位右对齐数据保持寄存器 (DAC\_DHR8R1)

偏移地址: 0x10

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |               |    |    |    |    |    |    |    |
|----------|----|----|----|----|----|----|----|---------------|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23            | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |    |    |               |    |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7             | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    |    |    |    |    | DACC1DHR[7:0] |    |    |    |    |    |    |    |
|          |    |    |    |    |    |    |    | rw            | rw | rw | rw | rw | rw | rw | rw |

Bits 31:8 保留, 必须保持为复位值。

Bits 7:0 **DACC1DHR[7:0]**: DAC 通道 1 的 8 位右对齐数据

该位由软件写入，表示 DAC 通道 1 的 8 位数据。

#### 17.4.6 DAC 通道 2 12 位右对齐数据保持寄存器 (DAC\_DHR12R2)

偏移地址: 0x14

复位值: 0x0000 0000

|          |    |    |    |                |    |    |    |    |    |    |    |    |    |    |    |
|----------|----|----|----|----------------|----|----|----|----|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27             | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |                |    |    |    |    |    |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11             | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    | DACC2DHR[11:0] |    |    |    |    |    |    |    |    |    |    |    |
|          |    |    |    | rw             | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:12 保留，必须保持为复位值。

Bits 11:0 **DACC2DHR[11:0]**: DAC 通道 2 的 12 位右对齐数据

该位由软件写入，表示 DAC 通道 2 的 12 位数据。



### 17.4.7 DAC 通道 2 12 位左对齐数据保持寄存器 (DAC\_DHR12L2)

偏移地址: 0x18

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |    |    |    |    |          |    |    |    |
|----------|----|----|----|----|----|----|----|----|----|----|----|----------|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19       | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |          |    |    |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3        | 2  | 1  | 0  |
| DACC2DHR |    |    |    |    |    |    |    |    |    |    |    | Reserved |    |    |    |
| rw       | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |          |    |    |    |

Bits 31:16 保留, 必须保持为复位值。

Bits 15:4 **DACC2DHR[11:0]**: DAC 通道 2 的 12 位左对齐数据

该位由软件写入, 表示 DAC 通道 2 的 12 位数据。

Bits 3:0 保留, 必须保持为复位值。

### 17.4.8 DAC 通道 2 8 位右对齐数据保持寄存器 (DAC\_DHR8R2)

偏移地址: 0x1C

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |               |    |    |    |    |    |    |    |
|----------|----|----|----|----|----|----|----|---------------|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23            | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |    |    |               |    |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7             | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    |    |    |    |    | DACC2DHR[7:0] |    |    |    |    |    |    |    |
|          |    |    |    |    |    |    |    | rw            | rw | rw | rw | rw | rw | rw | rw |

Bits 31:8 保留, 必须保持为复位值。

Bits 7:0 **DACC2DHR[7:0]**: DAC 通道 2 的 8 位右对齐数据

该位由软件写入, 表示 DAC 通道 2 的 8 位数据。

### 17.4.9 双 DAC 12 位右对齐数据保持寄存器 (DAC\_DHR12RD)

偏移地址: 0x20

复位值: 0x0000 0000

|          |    |    |    |                |    |    |    |    |    |    |    |    |    |    |    |
|----------|----|----|----|----------------|----|----|----|----|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27             | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    | DACC2DHR[11:0] |    |    |    |    |    |    |    |    |    |    |    |
|          |    |    |    | rw             | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |
| 15       | 14 | 13 | 12 | 11             | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    | DACC1DHR[11:0] |    |    |    |    |    |    |    |    |    |    |    |
|          |    |    |    | rw             | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:28 保留, 必须保持为复位值。

Bits 27:16 **DACC2DHR[11:0]**: DAC 通道 2 的 12 位右对齐数据

该位由软件写入, 用于为 DAC 通道 2 指定 12 位数据。

Bits 15:12 保留, 必须保持为复位值。

Bits 15:4 **DACC1DHR[11:0]**: DAC 通道 1 的 12 位右对齐数据

该位由软件写入, 用于为 DAC 通道 1 指定 12 位数据。

#### 17.4.10 双 DAC12 位左对齐数据保持寄存器 (DAC\_DHR12LD)

偏移地址: 0x24

复位值: 0x0000 0000

|                |    |    |    |    |    |    |    |    |    |    |    |          |    |    |    |
|----------------|----|----|----|----|----|----|----|----|----|----|----|----------|----|----|----|
| 31             | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19       | 18 | 17 | 16 |
| DACC2DHR[11:0] |    |    |    |    |    |    |    |    |    |    |    | Reserved |    |    |    |
| rw             | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |          |    |    |    |
| 15             | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3        | 2  | 1  | 0  |
| DACC1DHR[11:0] |    |    |    |    |    |    |    |    |    |    |    | Reserved |    |    |    |
| rw             | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |          |    |    |    |

Bits 31:20 **DACC2DHR[11:0]**: DAC 通道 2 的 12 位左对齐数据

该位由软件写入, 用于为 DAC 通道 2 指定 12 位数据.

Bits 19:16 保留, 必须保持为复位值。

Bits 15:4 **DACC1DHR[11:0]**: DAC 通道 1 的 12 位左对齐数据

该位由软件写入, 用于为 DAC 通道 1 指定 12 位数据.

Bits 3:0 保留, 必须保持为复位值。

#### 17.4.11 双 DAC 8 位右对齐数据保持寄存器 (DAC\_DHR8RD)

偏移地址: 0x28

复位值: 0x0000 0000

|               |    |    |    |    |    |    |    |               |    |    |    |    |    |    |    |
|---------------|----|----|----|----|----|----|----|---------------|----|----|----|----|----|----|----|
| 31            | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23            | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved      |    |    |    |    |    |    |    |               |    |    |    |    |    |    |    |
| 15            | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7             | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| DACC2DHR[7:0] |    |    |    |    |    |    |    | DACC1DHR[7:0] |    |    |    |    |    |    |    |
| rw            | rw | rw | rw | rw | rw | rw | rw | rw            | rw | rw | rw | rw | rw | rw | rw |

Bits 31:16 保留, 必须保持为复位值。

Bits 15:8 **DACC2DHR[7:0]**: DAC 通道 2 的 8 位右对齐数据

该位由软件写入, 用于为 DAC 通道 2 指定 8 位数据.

Bits 7:0 **DACC1DHR[7:0]**: DAC 通道 1 的 8 位右对齐数据

该位由软件写入, 用于为 DAC 通道 1 指定 8 位数据.

#### 17.4.12 DAC 通道 1 数据输出寄存器 (DAC\_DOR1)

偏移地址: 0x2C

复位值: 0x0000 0000

|          |    |    |    |               |    |    |    |    |    |    |    |    |    |    |    |
|----------|----|----|----|---------------|----|----|----|----|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27            | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |               |    |    |    |    |    |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11            | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    | DACC1DOR[7:0] |    |    |    |    |    |    |    |    |    |    |    |
|          |    |    |    | r             | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  |

Bits 31:12 保留, 必须保持为复位值。

Bits 11:0 **DACC1DOR[7:0]**: DAC 通道 1 输出数据

只读, 表示 DAC 通道 1 的输出数据。

### 17.4.13 DAC 通道 2 数据输出寄存器 (DAC\_DOR2)

偏移地址: 0x30

复位值: 0x0000 0000

|          |    |    |    |               |    |    |    |    |    |    |    |    |    |    |    |
|----------|----|----|----|---------------|----|----|----|----|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27            | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |               |    |    |    |    |    |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11            | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    | DACC2DOR[7:0] |    |    |    |    |    |    |    |    |    |    |    |
|          |    |    |    | r             | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  |

Bits 31:12 保留, 必须保持为复位值。

Bits 11:0 DACC2DOR[11:0]: DAC 通道 2 输出数据

只读, 表示 DAC 通道 2 的输出数据。

### 17.4.14 DAC 状态寄存器 (DAC\_SR)

偏移地址: 0x34

复位值: 0x0000 0000

|     |           |          |          |    |    |    |    |    |    |    |    |    |    |    |    |
|-----|-----------|----------|----------|----|----|----|----|----|----|----|----|----|----|----|----|
| 31  | 30        | 29       | 28       | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Res | CAL_FLAG2 | DMA_UDR2 | Reserved |    |    |    |    |    |    |    |    |    |    |    |    |
|     | r         | rc_w1    |          |    |    |    |    |    |    |    |    |    |    |    |    |
| 15  | 14        | 13       | 12       | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Res | CAL_FLAG1 | DMA_UDR1 | Reserved |    |    |    |    |    |    |    |    |    |    |    |    |
|     | r         | rc_w1    |          |    |    |    |    |    |    |    |    |    |    |    |    |

Bit 31 保留, 必须保持为复位值。

Bit 30 CAL\_FLAG2: DAC 通道 2 校准偏移状态

此位通过硬件置 1 和清零。

0: 校准调整值小于偏移校正

1: 校准调整值等于或大于偏移校正

注: (DAC2 通道 2, DAC3 通道 2 走内部, 无该标志)

Bit 29 DMAUDR2: DAC 2 通道 DMA 下溢标志

此位由硬件置 1, 由软件清零 (写入 1)。

0: DAC 2 通道未发生 DMA 下溢错误状况

1: DAC 2 通道发生 DMA 下溢错误状况 (当前所选触发源以高于 DMA 服务能力的频率驱动 DAC 2 通道转换)。

Bits 28:15 保留, 必须保持为复位值。

Bit 14 CAL\_FLAG1: DAC 通道 1 校准偏移状态

此位通过硬件置 1 和清零。

0: 校准调整值小于偏移校正

1: 校准调整值等于或大于偏移校正

注: (DAC3 通道一走内部, 无该标志)

Bit 13 DMAUDR1: DAC 1 通道 DMA 下溢标志

此位由硬件置 1, 由软件清零 (写入 1)。

0: DAC 1 通道未发生 DMA 下溢错误状况

1: DAC 1 通道发生 DMA 下溢错误状况 (当前所选触发源以高于 DMA 服务能力的频率

驱动 DAC 1 通道转换)

Bits 12:0 保留, 必须保持为复位值。

#### 17.4.15 DAC 校准寄存器 (DAC\_CCR)

偏移地址: 0x38

复位值: 0x0011 0011

|          |    |    |    |    |    |    |    |    |    |    |             |    |    |    |    |
|----------|----|----|----|----|----|----|----|----|----|----|-------------|----|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20          | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |    |    |    |    |    | OTRIM2[4:0] |    |    |    |    |
|          |    |    |    |    |    |    |    |    |    |    | rw          | rw | rw | rw | rw |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4           | 3  | 2  | 1  | 0  |
| Reserved |    |    |    |    |    |    |    |    |    |    | OTRIM1[4:0] |    |    |    |    |
|          |    |    |    |    |    |    |    |    |    |    | rw          | rw | rw | rw | rw |

Bits 31:21 保留, 必须保持为复位值。

Bits 20:16 **OTRIM2[4:0]**: DAC 通道 2 偏移调整值

注: (DAC2 通道 2, DAC3 通道 2 走内部, 无该值)

Bits 15:5 保留, 必须保持为复位值。

Bits 4:0 **OTRIM1[4:0]**: DAC 通道 1 偏移调整值

注: (DAC2 通道 2, DAC3 通道 2 走内部, 无该值)

#### 17.4.16 DAC 模式控制寄存器 (DAC\_MCR)

偏移地址: 0x3C

复位值: 0x0000 0000

|            |    |          |    |    |    |    |    |    |    |    |    |    |    |    |    |
|------------|----|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31         | 30 | 29       | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved   |    |          |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15         | 14 | 13       | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| HFSEL[1:0] |    | Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw         | rw |          |    |    |    |    |    |    |    |    |    |    |    |    |    |

Bit 31:16 保留, 必须保持为复位值。

Bit 15:14 **HFSEL[1:0]**: 高频接口模式选择

00: 关闭高频接口模式, DAC\_DOR 更新速率达到 3 个 AHB 时钟周期

01: 高频接口模式兼容 AHB>80 MHz, 使 DAC\_DOR 更新速率达到 5 个 AHB 时钟周期

10: 高频接口模式兼容 AHB>160 MHz, 使 DAC\_DOR 更新速率达到 7 个 AHB 时钟周期

11: 保留

Bits 13:0 保留, 必须保持为复位值。

#### 17.4.17 DAC 通道 1 锯齿寄存器 (DAC\_STR1)

偏移地址: 0x58

复位值: 0x0000 0000

|                  |    |        |                  |    |    |    |    |    |    |    |    |    |    |    |    |
|------------------|----|--------|------------------|----|----|----|----|----|----|----|----|----|----|----|----|
| 31               | 30 | 29     | 28               | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| STINCDATA1[15:0] |    |        |                  |    |    |    |    |    |    |    |    |    |    |    |    |
| rw               | rw | rw     | rw               | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |
| 15               | 14 | 13     | 12               | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved         |    | STDIR1 | STRSTDATA1[11:0] |    |    |    |    |    |    |    |    |    |    |    |    |
|                  |    | rw     | rw               | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:15 **STINCDATA1[15:0]**: DAC 通道 1 锯齿增量值

Bits 15:13 保留, 必须保持为复位值。

Bit 12 **STDIR1**: DAC 通道 1 锯齿方向设置

该位是由软件编写的,以选择锯齿步进的方向

0: 递减

1: 递增

Bits 11:0 **STRSTDATA1[11:0]**: DAC 通道 1 锯齿复位值

#### 17.4.18 DAC 通道 2 锯齿寄存器 (DAC\_STR2)

偏移地址: 0x5C

复位值: 0x0000 0000

| 31               | 30 | 29 | 28     | 27               | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|------------------|----|----|--------|------------------|----|----|----|----|----|----|----|----|----|----|----|
| STINCDATA2[15:0] |    |    |        |                  |    |    |    |    |    |    |    |    |    |    |    |
| rw               | rw | rw | rw     | rw               | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |
| 15               | 14 | 13 | 12     | 11               | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved         |    |    | STDIR1 | STRSTDATA2[11:0] |    |    |    |    |    |    |    |    |    |    |    |
|                  |    |    | rw     | rw               | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:15 **STINCDATA2[15:0]**: DAC 通道 2 锯齿增量值

Bits 15:13 保留, 必须保持为复位值。

Bit 12 **STDIR2**: DAC 通道 2 锯齿方向设置

该位是由软件编写的,以选择锯齿步进的方向

0: 递减

1: 递增

Bits 11:0 **STRSTDATA2[11:0]**: DAC 通道 2 锯齿复位值

#### 17.4.19 DAC 锯齿模式寄存器 (DAC\_STMODR)

偏移地址: 0x60

复位值: 0x0000 0000

| 31       | 30 | 29 | 28 | 27                 | 26 | 25 | 24 | 23       | 22 | 21 | 20 | 19                 | 18 | 17 | 16 |
|----------|----|----|----|--------------------|----|----|----|----------|----|----|----|--------------------|----|----|----|
| Reserved |    |    |    | STINCTRIGSEL2[3:0] |    |    |    | Reserved |    |    |    | STRSTTRIGSEL2[3:0] |    |    |    |
|          |    |    |    | rw                 | rw | rw | rw |          |    |    |    | rw                 | rw | rw | rw |
| 15       | 14 | 13 | 12 | 11                 | 10 | 9  | 8  | 7        | 6  | 5  | 4  | 3                  | 2  | 1  | 0  |
| Reserved |    |    |    | STINCTRIGSEL1[3:0] |    |    |    | Reserved |    |    |    | STRSTTRIGSEL1[3:0] |    |    |    |
|          |    |    |    | rw                 | rw | rw | rw |          |    |    |    | rw                 | rw | rw | rw |

Bits 31:28 保留, 必须保持为复位值。

Bits 27:24 **STINCTRIGSEL2[3:0]**: DAC 通道 2 锯齿触发选择

0000: SWTRIGB2

0001: dac\_inc\_inc\_ch2\_trg1

.....

1111: dac\_inc\_inc\_ch2\_trg15

Bits 23:20 保留, 必须保持为复位值。

Bits 19:16 **STRSTTRIGSEL2[3:0]**: DAC 通道 2 锯齿复位触发选择

0000: SWTRIGB2

0001: dac\_ch2\_trg1

.....

1111: dac\_ch2\_trg15

映射与 TSEL2[3:0]相同。

Bits 15:12 保留，必须保持为复位值。

Bits 11:8 **STINCTRIGSEL1[3:0]**: DAC 通道 1 锯齿触发选择

0000: SWTRIGB1

0001: dac\_inc\_inc\_trg1

.....

1111: dac\_inc\_inc\_ch1\_trg15

Bits 7:4 保留，必须保持为复位值。

Bits 3:0 **STRSTTRIGSEL1[3:0]**: DAC 通道 1 锯齿复位触发选择

0000: SWTRIGB1

0001: dac\_ch1\_trg1

.....

1111: dac\_ch1\_trg15

映射与 TSEL1[3:0]相同。

## 18 直接存储器访问控制器 (DMA)

### 18.1 简介

直接存储器访问 (DMA) 用于在外设与存储器之间以及存储器与存储器之间提供高速数据传输。可以在无需任何 CPU 操作的情况下通过 DMA 快速移动数据。这样节省的 CPU 资源可供其它操作使用。

两个 DMA 控制器有 16 个通道(DMA1 有 8 个通道, DMA2 有 8 个通道), 每一个 DMA 控制器都用于管理一个或多个外设的存储器访问请求。每个通道都有一个仲裁器, 用于处理 DMA 请求间的优先级。

### 18.2 DMA 主要特征

- 单个 AHB 主机
- 外设到内存, 内存到外设, 内存到内存和外设到外设数据传输
- 访问, 作为源和目的地, 到片上内存映射设备, 如闪存, SRAM, AHB 和 APB 外设
- 所有 DMA 通道独立配置:
  - 每个通道与来自外设的 DMA 请求信号相关联, 或与内存到内存传输中的软件触发器相关联。该配置由软件完成。
  - 请求之间的优先级可以通过软件 (每个通道 4 个级别: 非常高, 高, 中, 低) 和硬件 (例如通道 1 的请求优先级高于通道 2 的请求) 进行编程。
  - 源和目标的传输大小是独立的 (字节、半字、字), 模拟打包和拆包。源地址和目的地址必须在数据大小上保持一致。
  - 可编程的数据传输数量: 0 到  $2^{16} - 1$
- 为 DMA1 控制器生成单个中断请求, 或在通道级别处理任何所有中断请求。中断请求是由三种 DMA 事件中的任何一种引起的: 传输完成、传输一半或传输错误。
- 为 DMA2 控制器生成每个通道的中断请求。每个中断请求都是由三种 DMA 事件中的任何一种引起的: 传输完成、传输一半或传输错误。

### 18.3 DMA 请求映射

DMA 控制器通过 DMAMUX 外设连接到来自 AHB/APB 外设的 DMA 请求。

有关不同请求的映射, 请参阅 DMAMUX 实现。



## 18.4 DMA 功能说明

### 18.4.1 DMA 框图

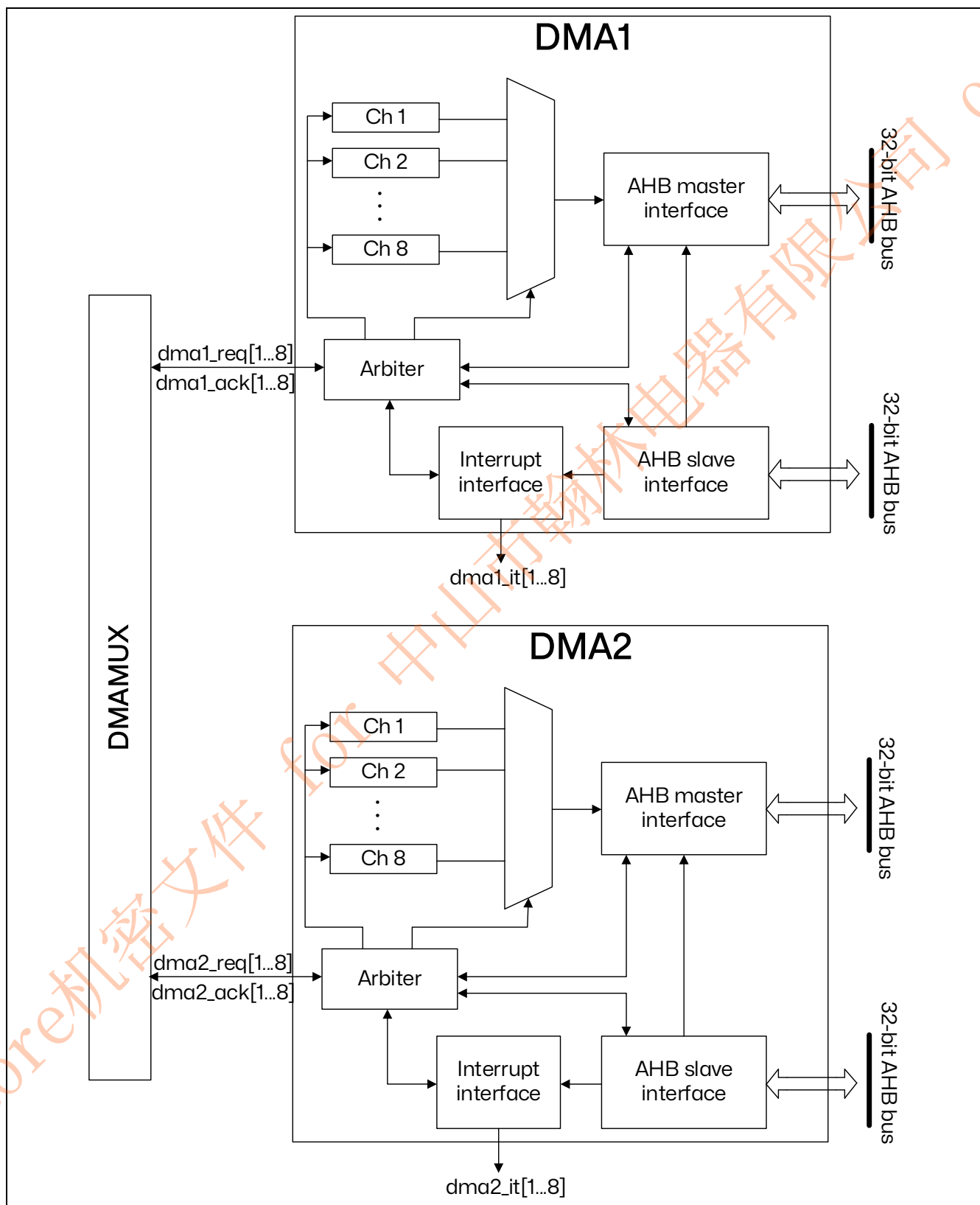


图 18.1 DMA 框图

DMA 控制器通过与其他系统主机共享 AHB 系统总线来执行直接内存传输。总线矩阵实现了循环调度。当 CPU 和 DMA 目标相同的目的地(内存或外设)时，DMA 请求可能会停止对系统总线的访问。

根据其通过 AHB 从接口的配置，DMA 控制器在 DMA 通道及其相关的接收请求之间进行仲裁。DMA 控制器还在单个 AHB 端口主机上调度 DMA 数据传输。

DMA 控制器为每个到中断控制器的通道生成一个中断。

## 18.4.2 DMA 引脚和内部信号

表 18.1 DMA 内部输入输出信号

| 信号         | 信号类型 | 描述      |
|------------|------|---------|
| dma_req[x] | 输入   | 通道 x 请求 |
| dma_ack[x] | 输出   | 通道 x 回应 |
| dma_it[x]  | 输出   | 通道 x 中断 |

## 18.4.3 DMA 传输

该软件在通道级配置 DMA 控制器，以便执行由 AHB 总线传输序列组成的块传输。

DMA 块传输可以从外设请求，或者在存储器到存储器传输的情况下由软件触发。

事件发生后，发生单个 DMA 传输的以下步骤：

1. 外设向 DMA 控制器发送单个 DMA 请求信号。
2. DMA 控制器根据与此外设请求相关联的通道的优先级为请求提供服务。
3. 一旦 DMA 控制器授予外设，DMA 控制器就会向外设发送一个确认。
4. 外设一旦得到 DMA 控制器的确认，就释放它的请求。
5. 一旦请求被外设解除断言，DMA 控制器就释放确认。

外设可以再发出一个请求并启动另一个 DMA 传输。

当外设是传输的源或目的地时，使用请求/确认协议。例如，在内存到外设传输的情况下，外设通过将其单个请求信号驱动到 DMA 控制器来启动传输。DMA 控制器从内存中读取单个数据，并将该数据写入外设。

对于给定的通道 x，DMA 块传输由以下重复序列组成：

- 单个 DMA 传输，封装单个数据的两个 AHB 传输，通过 DMA AHB 总线主机：
  - 从外设数据寄存器或者从当前外设/存储器地址寄存器指示的存储器地址取数据，第一次传输时的开始地址是 DMA\_CPARx 或 DMA\_CMARx 寄存器指定的外设基地址或存储器单元。
  - 存数据到外设数据寄存器或者当前外设/存储器地址寄存器指示的存储器地址，第一次传输时的开始地址是 DMA\_CPARx 或 DMA\_CMARx 寄存器指定的外设基地址或存储器单元。
- 执行一次 DMA\_CNDTRx 寄存器的递减操作，该寄存器包含未完成的操作数目。
- 重复这个序列，直到 DMA\_CNDTRx 为空。

注意：AHB 主总线源/目的地地址必须与传输到源/目的地的单个数据的编程大小一致。

## 18.4.4 仲裁器

仲裁器根据通道请求的优先级来启动外设/存储器的访问。

优先权管理分 2 个阶段：

- 软件：每个通道的优先级可以在 DMA\_CCRx 寄存器中设置，有 4 个等级：
  - 最高优先级
  - 高优先级
  - 中等优先级

– 低优先级

- 硬件：如果 2 个请求有相同的软件优先级，则较低编号的通道比较高编号的通道有较高的优先权。举个例子，通道 2 优先于通道 4。

当通道 x 被编程为内存到内存模式的块传输时，在该通道 x 的每个单个 DMA 传输之间考虑重新仲裁。每当有另一个并发的活动请求通道时，DMA 仲裁器自动交替并授予其他最高优先级的请求通道，该通道可能比内存到内存通道的优先级低。

#### 18.4.5 DMA 通道

每个通道都可以在有固定地址的外设寄存器和存储器地址之间执行 DMA 传输。DMA 传输的数据量是可编程的，最大达到 65535。包含要传输的数据项数量的寄存器，在每次传输后递减。

##### 可编程的数据量

外设和存储器的传输数据量可以通过 DMA\_CCRx 寄存器中的 PSIZE 和 MSIZE 位编程。

##### 指针增量

通过设置 DMA\_CCRx 寄存器中的 PINC 和 MINC 标志位，外设和存储器的指针在每次传输后可以有选择地完成自动增量。当设置为增量模式时，下一个要传输的地址将是前一个地址加上增量值，增量值取决于所选的数据宽度为 1、2 或 4。第一个传输的地址是存放在 DMA\_CPARx/DMA\_CMARx 寄存器中地址。在传输过程中，这些寄存器保持它们初始的数值，软件不能改变和读出当前正在传输的地址（它在内部的当前外设/存储器地址寄存器中）。

当通道配置为非循环模式时，传输结束后（即传输计数变为 0）将不再产生 DMA 操作。要开始新的 DMA 传输，需要在关闭 DMA 通道的情况下，在 DMA\_CNDTRx 寄存器中重新写入传输数目。

*注意：如果通道 x 被禁用，DMA 寄存器不会重置。DMA 通道寄存器(DMA\_CCRx、DMA\_CPARx 和 DMA\_CMARx)保留在通道配置阶段编程的初始值。*

在循环模式下，最后一次传输结束时，DMA\_CNDTRx 寄存器的内容会自动地被重新加载为其初始数值，内部的当前外设/存储器地址寄存器也被重新加载为 DMA\_CPARx/DMA\_CMARx 寄存器设定的初始基地址。

##### 通道配置过程

下面是配置 DMA 通道 x 的过程（x 代表通道号）：

1. 在 DMA\_CPARx 寄存器中设置外设寄存器的地址。发生外设数据传输请求时，这个地址将是数据传输的源或目标。
2. 在 DMA\_CMARx 寄存器中设置数据存储器的地址。发生外设数据传输请求时，传输的数据将从这个地址读出或写入这个地址。
3. 在 DMA\_CNDTRx 寄存器中设置要传输的数据量。在每个数据传输后，这个数值递减。
4. 在 DMA\_CCRx 寄存器的 PL[1:0]位中设置通道的优先级。
5. 在 DMA\_CCRx 寄存器中设置数据传输的方向、循环模式、外设和存储器的增量模式、外设和存储器的数据宽度、传输一半产生中断或传输完成产生中断。
6. 设置 DMA\_CCRx 寄存器的 ENABLE 位，启动该通道。

一旦启动了 DMA 通道，它既可响应连到该通道上的外设的 DMA 请求。

当传输一半的数据后，半传输标志（HTIF）被置 1，当设置了允许半传输中断位（HTIE）时，将产生一个中断请求。在数据传输结束后，传输完成标志（TCIF）被置 1，当设置了允许传输完成中断位（TCIE）时，将产生一个中断请求。

### 循环模式

循环模式用于处理循环缓冲区和连续的数据传输（如 ADC 的扫描模式）。在 DMA\_CCRx 寄存器中的 CIRC 位用于开启这一功能。

当启动了循环模式，数据传输的数目变为 0 时，将会自动地被恢复成配置通道时设置的初值，DMA 操作将会继续进行。

为了停止循环传输，软件需要在禁用 DMA 通道之前停止外设生成 DMA 请求（例如退出 ADC 扫描模式）。

软件必须在开始/使能传输之前，以及在停止循环传输之后显式地编程 DMA\_CNDTRx 值。

### 存储器到存储器模式

DMA 通道的操作可以在没有外设请求的情况下进行，这种操作就是存储器到存储器模式。

当设置了 DMA\_CCRx 寄存器中的 MEM2MEM 位之后，在软件设置了 DMA\_CCRx 寄存器中的 EN 位启动 DMA 通道时，DMA 传输将马上开始。当 DMA\_CNDTRx 寄存器变为 0 时，DMA 传输结束。

存储器到存储器模式不能与循环模式同时使用。

### 外设到外设模式

任何 DMA 通道都可以在外设到外设模式下运行：

- 当选择来自外设的硬件请求来触发 DMA 通道时，该外设是 DMA 启动器，并将数据从该外设传输到/从属于另一个内存映射外设的寄存器（该寄存器未配置为 DMA 模式）。
- 当没有外设请求被选择并连接到 DMA 通道时，软件通过设置 DMA\_CCRx 寄存器的 M2M 位来配置寄存器到寄存器的传输。

### 18.4.6 编程传输方向，分配源/目标

DMA\_CCRx 寄存器的 DIR 位的值设置传输的方向，因此，它标识源和目标，而不管源/目标类型（外设或内存）：

- DIR = 1 通常定义内存到外设的传输。更一般地说，如果 DIR = 1：
  - 源属性由 DMA\_MARx 寄存器、MSIZE[1:0] 位域和 DMA\_CCRx 寄存器的 MINC 位定义。不管它们通常的命名是什么，这些“内存”寄存器、位域和位被用来在外设到外设模式下定义源外设。
  - 目标属性由 DMA\_PARx 寄存器、PSIZE[1:0] 位域和 DMA\_CCRx 寄存器的 pin 位定义。不管它们通常的命名是什么，这些“外设”寄存器、位域和位被用来在内存对内存模式下定义目标内存。
- DIR = 0 通常定义一个外设到内存的传输。更一般地说，如果 DIR = 0：
  - 源属性由 DMA\_PARx 寄存器、PSIZE[1:0] 位域和 DMA\_CCRx 寄存器的 pin 位定义。不管它们通常的命名是什么，这些“外设”寄存器、位域和位被用来在内存到内存模式下定义源内存。
  - 而目标属性是由 DMA\_MARx 寄存器、MSIZE[1:0] 位域和 DMA\_CCRx 寄存器的 MINC 位定义的不管它们通常的命名是什么，这些“内存”寄存器、位域和位被用来在外设到外设模式下定义目标外设。

### 18.4.7 可编程的数据传输宽度、对齐方式和数据大小端

当 PSIZE 和 MSIZE 不相同，DMA 模块按照下表进行数据对齐。

表 18.2 可编程的数据传输宽度和大小端操作 (当 PINC = MINC = 1)

| 源端宽度 | 目标宽度 | 传输数目 | 源: 地址/数据                                                      | 传输操作                                                                                                                                                                                                             | 目标: 地址/数据                                                     |
|------|------|------|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| 8    | 8    | 4    | 0x0/B0<br>0x1/B1<br>0x2/B2<br>0x3/B3                          | 1: 在 0x0 读 B0[7:0], 在 0x0 写 B0[7:0]<br>2: 在 0x1 读 B1[7:0], 在 0x1 写 B1[7:0]<br>3: 在 0x2 读 B2[7:0], 在 0x2 写 B2[7:0]<br>4: 在 0x3 读 B3[7:0], 在 0x3 写 B3[7:0]                                                         | 0x0/B0<br>0x1/B1<br>0x2/B2<br>0x3/B3                          |
| 8    | 16   | 4    | 0x0/B0<br>0x1/B1<br>0x2/B2<br>0x3/B3                          | 1: 在 0x0 读 B0[7:0], 在 0x0 写 00B0[15:0]<br>2: 在 0x1 读 B1[7:0], 在 0x2 写 00B1[15:0]<br>3: 在 0x2 读 B2[7:0], 在 0x4 写 00B2[15:0]<br>4: 在 0x3 读 B3[7:0], 在 0x6 写 00B3[15:0]                                             | 0x0/00B0<br>0x2/00B1<br>0x4/00B2<br>0x6/00B3                  |
| 8    | 32   | 4    | 0x0/B0<br>0x1/B1<br>0x2/B2<br>0x3/B3                          | 1: 在 0x0 读 B0[7:0], 在 0x0 写 000000B0[31:0]<br>2: 在 0x1 读 B1[7:0], 在 0x4 写 000000B1[31:0]<br>3: 在 0x2 读 B2[7:0], 在 0x8 写 000000B2[31:0]<br>4: 在 0x3 读 B3[7:0], 在 0xC 写 000000B3[31:0]                             | 0x0/000000B0<br>0x4/000000B1<br>0x8/000000B2<br>0xC/000000B3  |
| 16   | 8    | 4    | 0x0/B1B0<br>0x2/B3B2<br>0x4/B5B4<br>0x6/B7B6                  | 1: 在 0x0 读 B1B0[15:0], 在 0x0 写 B0[7:0]<br>2: 在 0x2 读 B3B2[15:0], 在 0x1 写 B2[7:0]<br>3: 在 0x4 读 B5B4[15:0], 在 0x2 写 B4[7:0]<br>4: 在 0x6 读 B7B6[15:0], 在 0x3 写 B6[7:0]                                             | 0x0/B0<br>0x1/B2<br>0x2/B4<br>0x3/B6                          |
| 16   | 16   | 4    | 0x0/B1B0<br>0x2/B3B2<br>0x4/B5B4<br>0x6/B7B6                  | 1: 在 0x0 读 B1B0[15:0], 在 0x0 写 B1B0[15:0]<br>2: 在 0x2 读 B3B2[15:0], 在 0x2 写 B3B2[15:0]<br>3: 在 0x4 读 B5B4[15:0], 在 0x4 写 B5B4[15:0]<br>4: 在 0x6 读 B7B6[15:0], 在 0x6 写 B7B6[15:0]                                 | 0x0/B1B0<br>0x2/B3B2<br>0x4/B5B4<br>0x6/B7B6                  |
| 16   | 32   | 4    | 0x0/B1B0<br>0x2/B3B2<br>0x4/B5B4<br>0x6/B7B6                  | 1: 在 0x0 读 B1B0[15:0], 在 0x0 写 0000B1B0[31:0]<br>2: 在 0x2 读 B3B2[15:0], 在 0x4 写 0000B3B2[31:0]<br>3: 在 0x4 读 B5B4[15:0], 在 0x8 写 0000B5B4[31:0]<br>4: 在 0x6 读 B7B6[15:0], 在 0xC 写 0000B7B6[31:0]                 | 0x0/0000B1B0<br>0x4/0000B3B2<br>0x8/0000B5B4<br>0xC/0000B7B6  |
| 32   | 8    | 4    | 0x0/B3B2B1B0<br>0x4/B7B6B5B4<br>0x8/BBBBAB9B8<br>0xC/BFBEBDBC | 1: 在 0x0 读 B3B2B1B0[31:0], 在 0x0 写 B0[7:0]<br>2: 在 0x4 读 B7B6B5B4[31:0], 在 0x1 写 B4[7:0]<br>3: 在 0x8 读 BBBAB9B8[31:0], 在 0x2 写 B8[7:0]<br>4: 在 0xC 读 BFBEBDBC[31:0], 在 0x3 写 BC[7:0]                             | 0x0/B0<br>0x1/B4<br>0x2/B8<br>0x3/BC                          |
| 32   | 16   | 4    | 0x0/B3B2B1B0<br>0x4/B7B6B5B4<br>0x8/BBBBAB9B8<br>0xC/BFBEBDBC | 1: 在 0x0 读 B3B2B1B0[31:0], 在 0x0 写 B1B0[15:0]<br>2: 在 0x4 读 B7B6B5B4[31:0], 在 0x2 写 B5B4[15:0]<br>3: 在 0x8 读 BBBAB9B8[31:0], 在 0x4 写 B9B8[15:0]<br>4: 在 0xC 读 BFBEBDBC[31:0], 在 0x6 写 BDBC[15:0]                 | 0x0/B1B0<br>0x2/B5B4<br>0x4/B9B8<br>0x6/BDBC                  |
| 32   | 32   | 4    | 0x0/B3B2B1B0<br>0x4/B7B6B5B4<br>0x8/BBBBAB9B8<br>0xC/BFBEBDBC | 1: 在 0x0 读 B3B2B1B0[31:0], 在 0x0 写 B3B2B1B0[31:0]<br>2: 在 0x4 读 B7B6B5B4[15:0], 在 0x4 写 B7B6B5B4[31:0]<br>3: 在 0x8 读 BBBAB9B8[31:0], 在 0x8 写 BBBAB9B8[31:0]<br>4: 在 0xC 读 BFBEBDBC[31:0], 在 0xC 写 BFBEBDBC[31:0] | 0x0/B3B2B1B0<br>0x4/B7B6B5B4<br>0x8/BBBBAB9B8<br>0xC/BFBEBDBC |

### 操作一个不支持字节或半字写的 AHB 设备

当 DMA 模块开始一个 AHB 的字节或半字写操作时, 数据将在 HWDATA[31:0]总线中未使用的部分重复。因此, 如果 DMA 以字节或半字写入不支持字节或半字写操作的 AHB 设备时 (即 HSIZE 不适用于该模块), 不会发生错误, DMA 将按照下面两个例子写入 32 位 HWDATA 数据:

- 当 HSIZE=半字时, 写入半字 '0xABCD', DMA 将设置 HWDATA 总线为 '0xABCDABCD'。
- 当 HSIZE=字节时, 写入字节 '0xAB', DMA 将设置 HWDATA 总线为 '0xABABABAB'。假定 AHB/APB 桥是一个 AHB 的 32 位从设备, 它不处理 HSIZE 参数, 它将按照下述方式把任何 AHB 上的字节或半字按 32 位传送到 APB 上:
- 一个 AHB 上对地址 0x0 (或 0x1、0x2 或 0x3) 的写字节数据 '0xB0' 操作, 将转换到 APB 上对地址 0x0 的写字数据 '0xB0B0B0B0' 操作。
- 一个 AHB 上对地址 0x0 (或 0x2) 的写半字数据 '0xB1B0' 操作, 将转换到 APB 上对地址 0x0 的写字数据 '0xB1B0B1B0' 操作。例如, 如果要写入 APB 后备寄存器 (与 32 位地址对齐的 16 位寄存器), 需要配置存储器数据源宽度 (MSIZE) 为 '16 位', 外设目标数据宽度 (PSIZE) 为 '32'。



位’。

#### 18.4.8 错误管理

读写一个保留的地址区域，将会产生 DMA 传输错误。当在 DMA 读写操作时发生 DMA 传输错误时，硬件会自动地清除发生错误的通道所对应的通道配置寄存器（DMA\_CCRx）的 EN 位，该通道操作被停止。此时，在 DMA\_IFR 寄存器中对应该通道的传输错误中断标志位（TEIF）将被置位，如果在 DMA\_CCRx 寄存器中设置了传输错误中断允许位，则将产生中断。

当软件在涉及外设的通道上收到传输错误通知时，软件必须首先在 DMA 模式下停止该外设，以便禁用任何未决或未来的 DMA 请求。然后软件通常可以在 DMA 模式下为新的传输重新配置 DMA 和外设。

#### 18.4.9 中断

每个 DMA 通道都可以在 DMA 传输过半、传输完成和传输错误时产生中断。为应用的灵活性考虑，通过设置寄存器的不同位来打开这些中断。

表 18.3 DMA 中断请求

| 中断事件 | 事件标志位 | 使能控制位 |
|------|-------|-------|
| 传输过半 | HTIF  | HTIE  |
| 传输完成 | TCIF  | TCIE  |
| 传输错误 | TEIF  | TEIE  |

### 18.5 DMA 寄存器描述

基地址：

0x4002 0000（DMA1）

0x4002 0400（DMA2）

#### 18.5.1 DMA 中断状态寄存器（DMA\_ISR）

偏移地址：0x00

复位值：0x0000 0000

| 31    | 30    | 29    | 28   | 27    | 26    | 25    | 24   | 23    | 22    | 21    | 20   | 19    | 18    | 17    | 16   |
|-------|-------|-------|------|-------|-------|-------|------|-------|-------|-------|------|-------|-------|-------|------|
| TEIF8 | HTIF8 | TCIF8 | GIF8 | TEIF7 | HTIF7 | TCIF7 | GIF7 | TEIF6 | HTIF6 | TCIF6 | GIF6 | TEIF5 | HTIF5 | TCIF5 | GIF5 |
| r     | r     | r     | r    | r     | r     | r     | r    | r     | r     | r     | r    | r     | r     | r     | r    |
| 15    | 14    | 13    | 12   | 11    | 10    | 9     | 8    | 7     | 6     | 5     | 4    | 3     | 2     | 1     | 0    |
| TEIF4 | HTIF4 | TCIF4 | GIF4 | TEIF3 | HTIF3 | TCIF3 | GIF3 | TEIF2 | HTIF2 | TCIF2 | GIF2 | TEIF1 | HTIF1 | TCIF1 | GIF1 |
| r     | r     | r     | r    | r     | r     | r     | r    | r     | r     | r     | r    | r     | r     | r     | r    |

Bit  $4*(x-1)+3$  TEIFx：通道 x 的传输错误标志（ $x = 1 \dots 8$ ）（Channel x transfer error flag）

硬件设置这些位。在 DMA\_IFCR 寄存器的相应位写入 ‘1’ 可以清除这里对应的标志位。

0：在通道 x 没有传输错误（TE）；

1：在通道 x 发生了传输错误（TE）。

Bit  $4*(x-1)+2$  HTIFx：通道 x 的半传输标志（ $x = 1 \dots 8$ ）（Channel x half transfer flag）

硬件设置这些位。在 DMA\_IFCR 寄存器的相应位写入 ‘1’ 可以清除这里对应的标志位。

0：在通道 x 没有半传输事件（HT）；

1：在通道 x 产生了半传输事件（HT）。

Bit  $4*(x-1)+1$  TCIFx：通道 x 的传输完成标志（ $x = 1 \dots 8$ ）（Channel x transfer complete flag）

硬件设置这些位。在 DMA\_IFCR 寄存器的相应位写入 ‘1’ 可以清除这里对应的标志位。

0: 在通道 x 没有传输完成事件 (TC);

1: 在通道 x 产生了传输完成事件 (TC)。

Bit  $4*(x-1)+0$  **GIFx**: 通道 x 的全局中断标志 ( $x = 1 \dots 8$ ) (Channel x global interrupt flag)

硬件设置这些位。在 DMA\_IFCR 寄存器的相应位写入 ‘1’ 可以清除这里对应的标志位。

0: 在通道 x 没有 TE、HT 或 TC 事件;

1: 在通道 x 产生了 TE、HT 或 TC 事件。

## 18.5.2 DMA 中断寄存器标志清除寄存器 (DMA\_IFCR)

偏移地址: 0x04

复位值: 0x0000 0000

| 31     | 30     | 29     | 28    | 27     | 26     | 25     | 24    | 23     | 22     | 21     | 20    | 19     | 18     | 17     | 16    |
|--------|--------|--------|-------|--------|--------|--------|-------|--------|--------|--------|-------|--------|--------|--------|-------|
| CTEIF8 | CHTIF8 | CTCIF8 | CGIF8 | CTEIF7 | CHTIF7 | CTCIF7 | CGIF7 | CTEIF6 | CHTIF6 | CTCIF6 | CGIF6 | CTEIF5 | CHTIF5 | CTCIF5 | CGIF5 |
| rw     | rw     | rw     | rw    | rw     | rw     | rw     | rw    | rw     | rw     | rw     | rw    | rw     | rw     | rw     | rw    |
| 15     | 14     | 13     | 12    | 11     | 10     | 9      | 8     | 7      | 6      | 5      | 4     | 3      | 2      | 1      | 0     |
| CTEIF4 | CHTIF4 | CTCIF4 | CGIF4 | CTEIF3 | CHTIF3 | CTCIF3 | CGIF3 | CTEIF2 | CHTIF2 | CTCIF2 | CGIF2 | CTEIF1 | CHTIF1 | CTCIF1 | CGIF1 |
| rw     | rw     | rw     | rw    | rw     | rw     | rw     | rw    | rw     | rw     | rw     | rw    | rw     | rw     | rw     | rw    |

Bit  $4*(x-1)+3$  **CTEIFx**: 清除通道 x 的传输错误标志 ( $x = 1 \dots 8$ ) (Channel x transfer error clear)

这些位由软件设置和清除。

0: 不起作用

1: 清除 DMA\_ISR 寄存器中的对应 TEIF 标志。

Bit  $4*(x-1)+2$  **CHTIFx**: 清除通道 x 的半传输标志 ( $x = 1 \dots 8$ ) (Channel x half transfer clear)

这些位由软件设置和清除。

0: 不起作用

0: 清除 DMA\_ISR 寄存器中的对应 HTIF 标志。

Bit  $4*(x-1)+1$  **CTCIFx**: 清除通道 x 的传输完成标志 ( $x = 1 \dots 8$ ) (Channel x transfer complete clear)

这些位由软件设置和清除。

0: 不起作用

0: 清除 DMA\_ISR 寄存器中的对应 TCIF 标志。

Bit  $4*(x-1)+0$  **CGIFx**: 清除通道 x 的传全局中断标志 ( $x = 1 \dots 8$ ) (Channel x transfer complete clear)

这些位由软件设置和清除。

0: 不起作用

0: 清除 DMA\_ISR 寄存器中的对应 GIF、TEIF、HTIF、TCIF 标志。



### 18.5.3 DMA 通道 x 配置寄存器 (DMA\_CCRx) (x=1..8)

偏移地址:  $0x08 + 20 \times (\text{通道编号} - 1)$

复位值: 0x0000

| 31       | 30  | 29      | 28 | 27         | 26 | 25         | 24 | 23   | 22   | 21   | 20  | 19   | 18   | 17   | 16 |
|----------|-----|---------|----|------------|----|------------|----|------|------|------|-----|------|------|------|----|
| Reserved |     |         |    |            |    |            |    |      |      |      |     |      |      |      |    |
| 15       | 14  | 13      | 12 | 11         | 10 | 9          | 8  | 7    | 6    | 5    | 4   | 3    | 2    | 1    | 0  |
| Res      | M2M | PL[1:0] |    | MSIZE[1:0] |    | PSIZE[1:0] |    | MINC | PINC | CIRC | DIR | TEIE | HTIE | TCIE | EN |
|          | rw  | rw      | rw | rw         | rw | rw         | rw | rw   | rw   | rw   | rw  | rw   | rw   | rw   | rw |

Bits 31:15 保留, 必须保持为复位值。

Bit 14 **M2M**: 存储器到存储器模式 (Memory to memory mode)

该位由软件设置和清除。

0: 非存储器到存储器模式;

1: 启动存储器到存储器模式。

Bits 13:12 **PL[1:0]**: 通道优先级 (Channel priority level)

位由软件设置和清除。

00: 低

01: 中

10: 高

11: 最高

Bits 11:10 **MSIZE[1:0]**: 存储器数据宽度 (Memory size)

这些位由软件设置和清除。

00: 8 位

01: 16 位

10: 32 位

11: 保留

Bits 9:8 **PSIZE[1:0]**: 外设数据宽度 (Peripheral size)

这些位由软件设置和清除。

00: 8 位

01: 16 位

10: 32 位

11: 保留

Bit 7 **MINC**: 存储器地址增量模式 (Memory increment mode)

该位由软件设置和清除。

0: 不执行存储器地址增量操作

1: 执行存储器地址增量操作

Bit 6 **PINC**: 外设地址增量模式 (Peripheral increment mode)

该位由软件设置和清除。

0: 不执行外设地址增量操作

1: 执行外设地址增量操作

Bit 5 **CIRC**: 循环模式 (Circular mode)

该位由软件设置和清除。

0: 不执行循环操作

1: 执行循环操作

**Bit 4 DIR:** 数据传输方向 (Data transfer direction)

该位由软件设置和清除。

0: 从外设读

1: 从存储器读

**Bit 3 TEIE:** 允许传输错误中断 (Transfer error interrupt enable)

该位由软件设置和清除。

0: 禁止 TE 中断

0: 允许 TE 中断

**Bit 2 HTIE:** 允许半传输中断 (Half transfer interrupt enable)

该位由软件设置和清除。

0: 禁止 HT 中断

0: 允许 HT 中断

**Bit 1 TCIE:** 允许传输完成中断 (Transfer complete interrupt enable)

该位由软件设置和清除。

0: 禁止 TC 中断

0: 允许 TC 中断

**Bit 0 EN:** 通道开启 (Channel enable)

该位由软件设置和清除。

0: 通道不工作

1: 通道开启

#### 18.5.4 DMA 通道 x 传输数量寄存器 (DMA\_CNDTRx) (x=1...8)

偏移地址:  $0x0C + 20 \times (\text{通道编号} - 1)$

复位值: 0x0000

|           |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31        | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15        | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| NDT[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rW        | rW | rW | rW | rW | rW | rW | rW | rW | rW | rW | rW | rW | rW | rW | rW |

Bits 31:16 保留, 必须保持为复位值。

**Bits 15:0 NDT[15:0]:** 数据传输数量 (Number of data to transfer)

传输数量为 0 至 65535。这个寄存器只能在通道不工作 (DMA\_CCRx 的 EN=0) 时写入。通道开启后该寄存器变为只读, 指示剩余的待传输字节数目。寄存器内容在每次 DMA 传输后递减。

数据传输结束后, 寄存器的内容或者变为 0; 或者当该通道配置为自动重加载模式时, 寄存器的内容将被自动重新加载为之前配置时的数值。

当寄存器的内容为 0 时, 无论通道是否开启, 都不会发生任何数据传输。

### 18.5.5 DMA 通道 x 外设地址寄存器 (DMA\_CPARx) (x=1..8)

偏移地址:  $0x10 + 20 \times (\text{通道编号} - 1)$

复位值:  $0x0000\ 0000$

当开启通道 (DMA\_CCRx 的 EN=1) 时不能写该寄存器。

|           |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31        | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| PA[31:16] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw        | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |
| 15        | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| PA[15:0]  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw        | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:16 PA[31:0]: 外设地址 (Peripheral address)

设数据寄存器的基地址, 作为数据传输的源或目标。

当 PSIZE= '01' (16 位), 不使用 PA[0]位。操作自动地与半字地址对齐。

当 PSIZE= '10' (32 位), 不使用 PA[1:0]位。操作自动地与字地址对齐。

### 18.5.6 DMA 通道 x 外设存储地址寄存器 (DMA\_CMARx) (x=1..8)

偏移地址:  $0x14 + 20 \times (\text{通道编号} - 1)$

复位值:  $0x0000\ 0000$

当开启通道 (DMA\_CCRx 的 EN=1) 时不能写该寄存器。

|           |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31        | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| MA[31:16] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw        | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |
| 15        | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| MA[15:0]  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw        | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:0 MA[31:0]: 存储器地址

存储器地址作为数据传输的源或目标。

当 MSIZE= '01' (16 位), 不使用 MA[0]位。操作自动地与半字地址对齐。

当 MSIZE= '10' (32 位), 不使 MA[1:0]位。操作自动地与字地址对齐。

## 19 DMA 请求多路复用器 (DMAMUX)

### 19.1 简介

外设通过设置 DMA 请求信号来指示 DMA 传输请求。DMA 请求一直处于挂起状态，直到产生 DMA 确认信号的 DMA 控制器为它服务，并且相应的 DMA 请求信号被解除声明。

在本文档中，没有明确显示或描述 DMA 请求/确认协议所需的一组控制信号，它被称为 DMA 请求行。

DMAMUX 请求多路复用器支持在外设和产品的 DMA 控制器之间路由 DMA 请求线。路由功能由可编程多通道 DMA 请求线多路复用器保证。每个通道选择一个唯一的 DMA 请求行，无条件地或同步地从它的 DMAMUX 同步输入的事件。DMAMUX 也可用作 DMA 请求发生器，从可编程事件输入触发信号。

### 19.2 主要特性

- 多达 16 通道可编程 DMA 请求线多路复用器输出
- 4 通道 DMA 请求生成器
- 20 触发输入到 DMA 请求生成器
- 20 同步输入
- 每个 DMA 请求生成器通道:
  - DMA 请求触发输入选择器
  - DMA 请求计数器
  - 所选 DMA 请求触发输入的事件溢出标志
- 每个 DMA 请求线多路复用器通道输出:
  - 从外设输入 88 个 DMA 请求
  - 一个 DMA 请求行输出
  - 同步输入选择器
  - DMA 请求计数器
  - 选定同步输入的事件溢出标志
  - 一个事件输出，用于 DMA 请求链

## 19.3 功能描述

### 19.3.1 框图

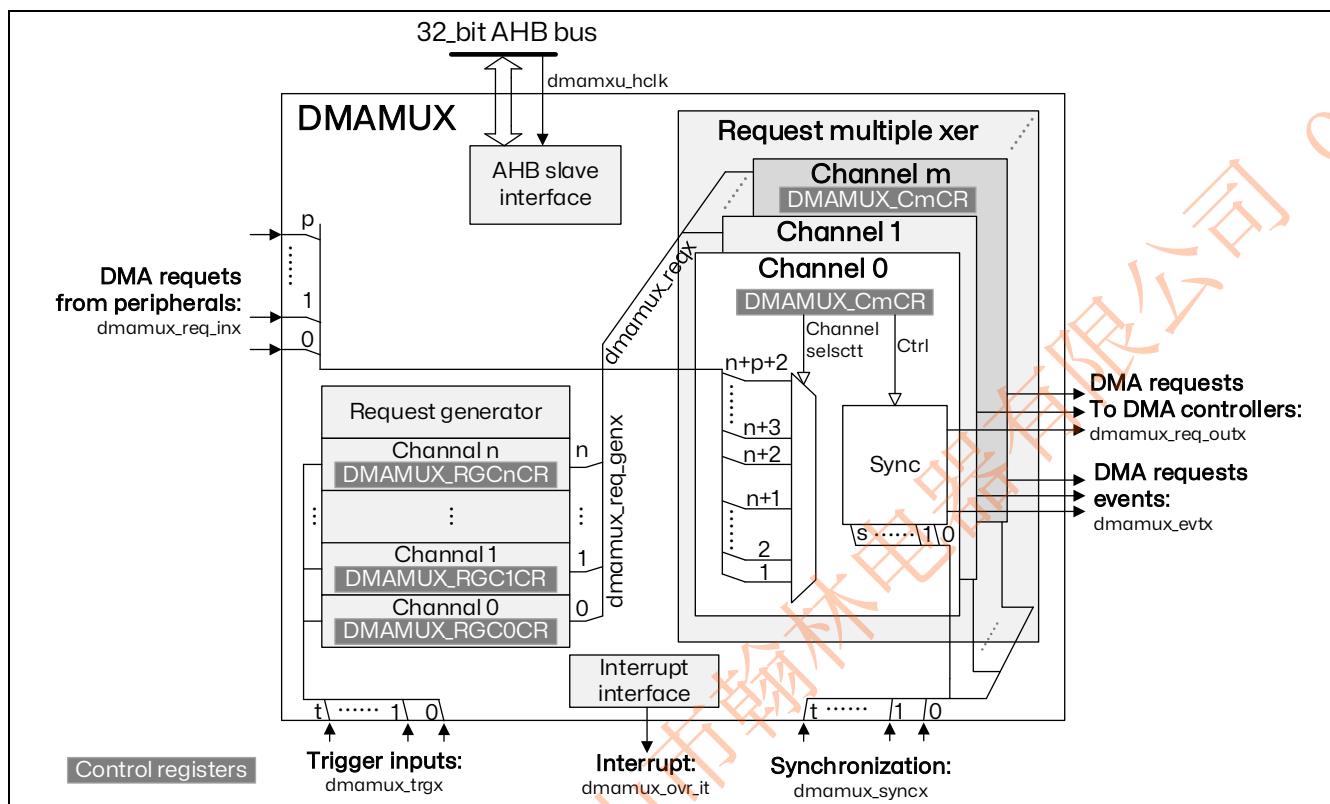


图 19.1 DMAMUX 框图

DMAMUX 有两个主要的子块：请求行多路复用器和请求行生成器。

实现分配：

- DMAMUX 请求多路复用子块输入 (dmamux\_req\_inx) 从外设 (dmamux\_req\_inx) 和从 DMAMUX 请求生成器子块通道 (dmamux\_req\_genx)
- DMAMUX 请求输出到 DMA 控制器通道 (dmamux\_req\_outx)
- 内部或外部信号到 DMA 请求触发输入 (dmamux\_trgx)
- 内部或外部信号到同步输入 (dmamux\_syncx)

### 19.3.2 DMAMUX 信号

表 19.1 DMAMUX：多路复用器输入源

| 信号              | 描述                          |
|-----------------|-----------------------------|
| dmamux_hclk     | AHB 时钟                      |
| dmamux_req_inx  | DMA 从外设请求线路输入               |
| dmamux_trgx     | DMA 请求触发输入 (请求生成器子块)        |
| dmamux_req_genx | 请求生成器子块通道输出                 |
| dmamux_reqx     | 请求多路复用子块输入 (来自外设请求和请求生成器通道) |
| dmamux_syncx    | 同步输入 (请求多路复用子块)             |

|                 |                 |
|-----------------|-----------------|
| dmamux_req_outx | 请求输出（到 DMA 控制器） |
| dmamux_evtx     | 事件中断            |
| dmamux_over_it  | 溢出中断            |

### 19.3.3 DMAMUX 通道

DMAMUX 通道是 DMAMUX 请求多路复用器通道，根据所选的请求多路复用器输入，它可能包括一个额外的 DMAMUX 请求生成器通道。

DMAMUX 请求多路复用通道连接并专用于 DMA 控制器的单个通道。

#### 通道配置步骤

按照以下顺序配置 DMAMUX x 通道和相应的 DMA 通道 y:

1. 除了使能通道 y 外，完成 DMA 通道 y 的设置和配置。
2. 完整地设置和配置相关的 DMAMUX x 通道。
3. 最后，通过在 DMA y 通道寄存器中设置 EN 位激活 DMA 通道 y。

### 19.3.4 DMAMUX 请求线路多路复用器

具有多个通道的 DMAMUX 请求多路复用器确保 DMA 请求/确认控制信号的实际路由，称为 DMA 请求线。

每个 DMA 请求线与 DMAMUX 请求线多路复用器的所有通道并行连接。

DMA 请求要么来自外设，要么来自 DMAMUX 请求生成器。

DMAMUX 请求线路多路复用器通道 x 选择由 DMAMUX\_CxCR 寄存器中的 DMAREQ\_ID 位域配置的 DMA 请求线路号。

注意：dareq\_id 位域的空值对应于没有选择 DMA 请求行。

注意：一个相同的非空 DMAREQ\_ID 不能被编程到不同的 x 和 y DMAMUX 请求多路复用器通道(通过 DMAMUX\_CxCR 和 DMAMUX\_CyCR)，除非应用程序保证两个连接的 DMA 通道不是同时活动的。

在 DMA 请求选择之上，如果需要，可以配置和使能同步模式和/或事件生成。

#### 同步模式和通道事件生成

每个 DMAMUX 请求线路多路复用器通道 x 可以通过在 DMAMUX\_CxCR 寄存器中设置同步使能 (SE)位来单独同步。

DMAMUX 有多个同步输入。同步输入以并行方式连接到请求多路复用器的所有通道。

同步输入是通过给定通道 x 的 DMAMUX\_CxCR 寄存器中的 SYNC\_ID 位域选择的。

当通道处于这种同步模式时，一旦检测到所选输入同步信号上的可编程上升/下降沿，通过 DMAMUX\_CxCR 寄存器的 SPOL[1:0]位域，所选输入 DMA 请求线被传播到多路复用器通道输出。

此外，在 DMAMUX 请求多路复用器的内部有一个可编程 DMA 请求计数器，可用于通道请求输出的生成，也可能用于事件的生成。通道 x 输出上的事件生成通过 DMAMUX\_CxCR 寄存器的 EGE 位(事件生成使能)使能。

如下图所示，在同步输入的检测边缘上，挂起的所选输入 DMA 请求线连接到 DMAMUX 多路复用器通道 x 输出。

**注意：**如果同步事件发生，而没有挂起的选择的输入 DMA 请求行，它将被丢弃。在再次发生同步事件之前，以下断言的输入请求行不会连接到 DMAMUX 多路复用器通道输出。

从这一点开始，每次 DMA 控制器为连接的 DMAMUX 请求提供服务时(被服务的请求被撤销)，DMAMUX 请求计数器将递减。在其欠运行时，DMA 请求计数器自动加载 DMAMUX\_CxCR 寄存器的

NBREQ 位域中的值，并且输入 DMA 请求线与多路复用器通道 x 输出断开。

因此，在检测到同步事件之后，转移到多路复用器通道 x 输出的 DMA 请求的数量等于 NBREQ 位域中的值加上 1。

注：NBREQ 位域值只有在相应多路复用通道 x 的同步使能位 SE 和事件生成使能位 EGE 同时不使能的情况下，才能由软件写入

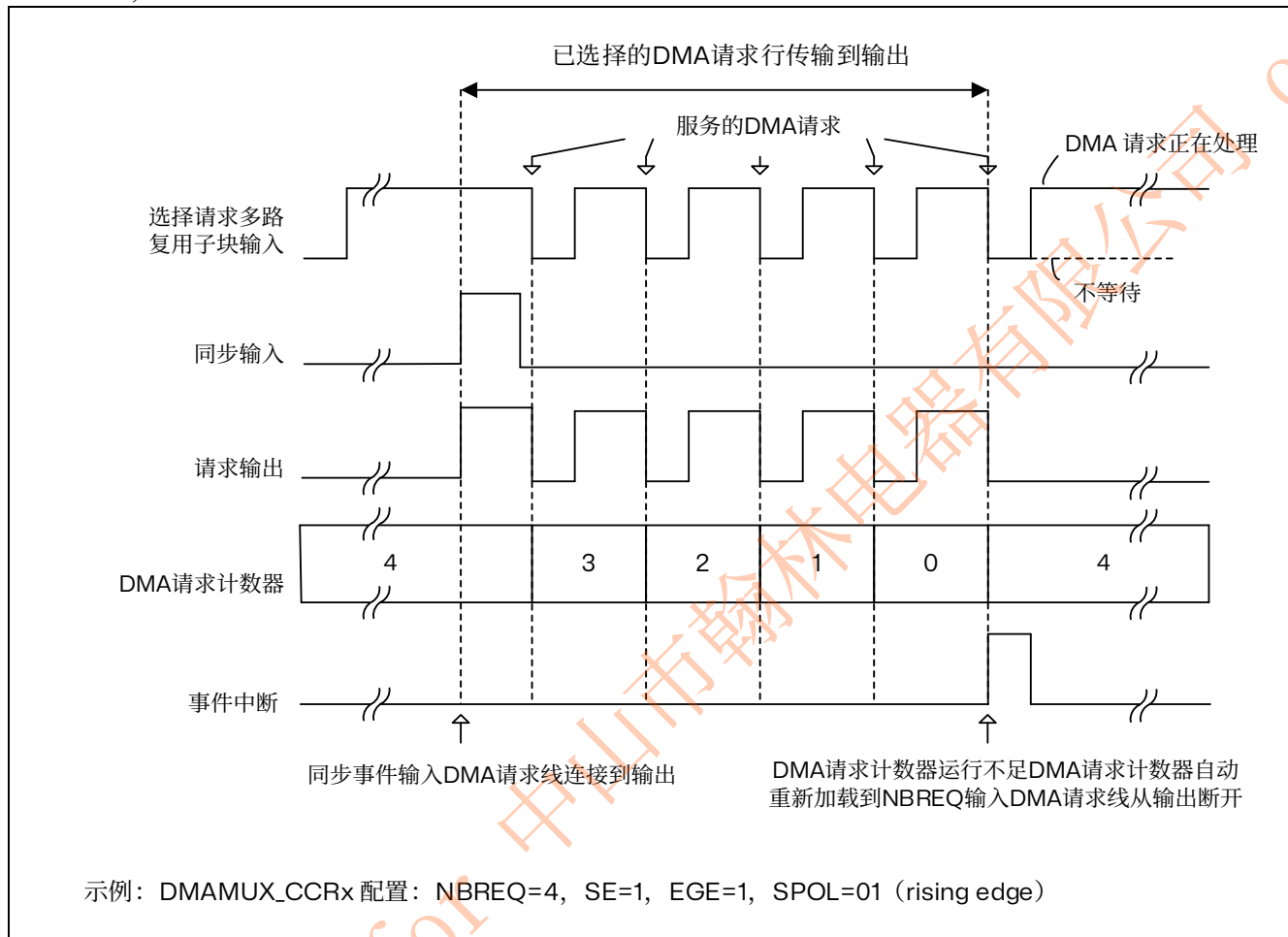


图 19.2 DMAMUX 请求多路复用通道的同步模式



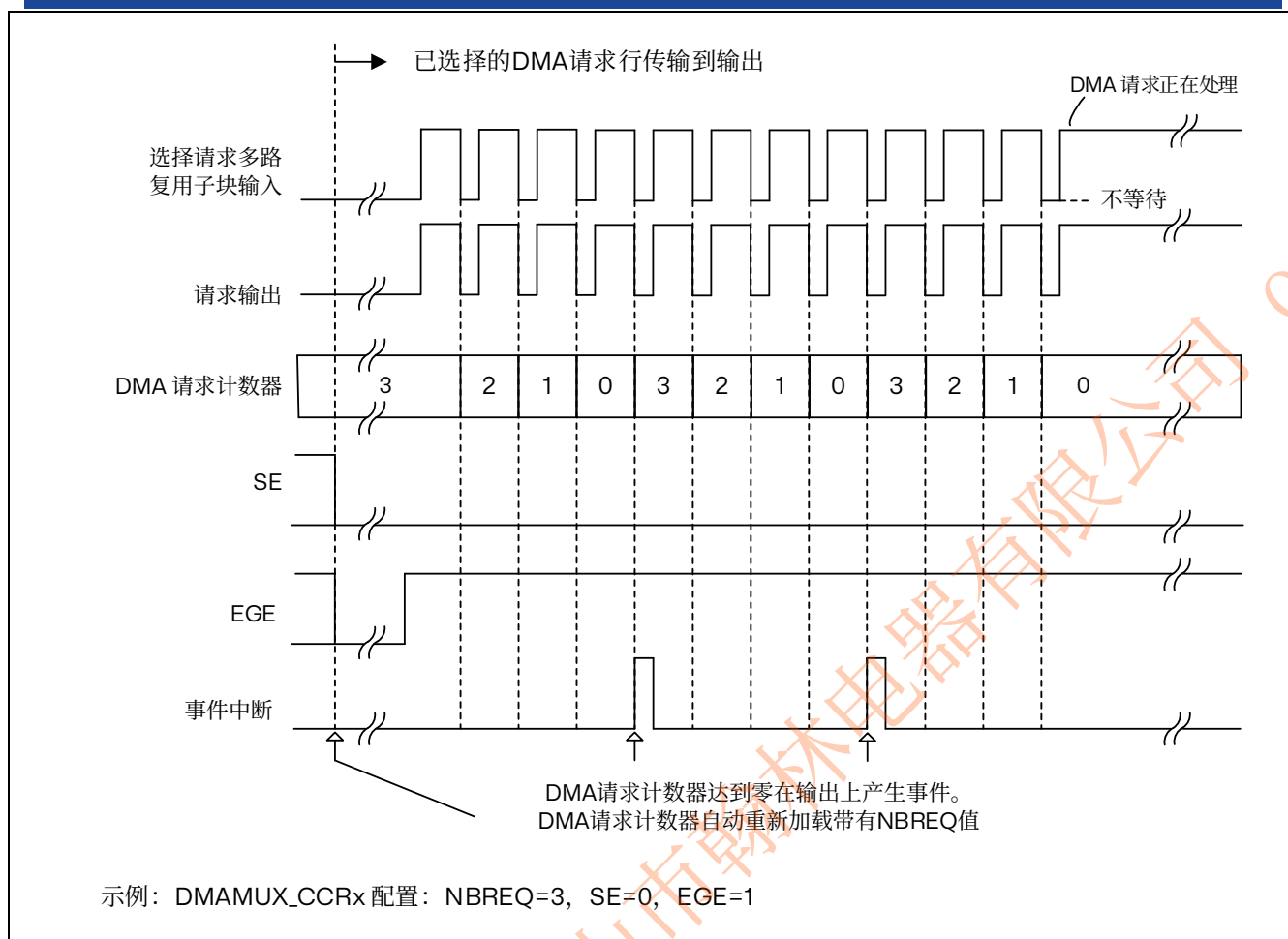


图 19.3 DMA 请求线路多路复用器通道的时间生成

如果使能了 EGE，当其 DMA 请求计数器被自动重新加载编程的 NBREQ 位域值时，多路复用器通道产生一个通道事件，作为一个 AHB 时钟周期的脉冲，如上图所示。

注：如果使能了 EGE，且 NBREQ = 0，则每服务一次 DMA 请求都会产生一个事件。

注：如果沿边缘的状态在两个以上的 AHB 时钟周期内保持稳定，则检测到同步事件(边缘)。

在写入 DMAMUX\_CxCR 寄存器时，同步事件在三个 AHB 时钟周期内被屏蔽。

### 同步超限和中断

如果在请求计数器运行不足(通过 DMAMUX\_CxCR 寄存器的 NBREQ 位域编程的内部请求计数器)之前发生新的同步事件，则在 DMAMUX\_CSR 状态寄存器中设置同步超时标志位 SOF<sub>x</sub>。

注意：请求多路复用器通道 x 同步必须禁用 (DMAMUX\_CxCR.SE = 0) 在完成对 DMA 控制器的相关通道的使用时。否则，在检测到新的同步事件时，由于没有从 DMA 控制器接收到 DMA 确认(即没有服务请求)，会出现同步溢出。

通过在 DMAMUX\_CFR 寄存器中设置相关的清除同步溢出标志位 CSOF<sub>x</sub>，可以重置溢出标志 SOF<sub>x</sub>。

如果在 DMAMUX\_CxCR 寄存器中设置了同步溢出中断使能位 SOIE，则设置同步溢出标志会产生中断。

### 19.3.5 DMAMUX 请求生成器

DMAMUX 请求生成器根据其 DMA 请求触发器输入上的触发事件生成 DMA 请求。

DMAMUX 请求生成器有多个通道。DMA 请求触发输入与所有通道并行连接。

DMAMUX 请求生成器通道的输出是 DMAMUX 请求线路多路复用器的输入。

每个 DMAMUX 请求生成器通道 x 在相应的 DMAMUX\_RGxCR 寄存器中都有一个使能位 GE(生成器使能)。

DMAMUX 请求生成器通道 x 的 DMA 请求触发输入是通过相应 DMAMUX\_RGxCR 寄存器中的 SIG\_ID(触发信号 ID)位域选择的。

DMA 请求触发输入上的触发事件可以是上升沿、下降沿或任意一条。通过相应 DMAMUX\_RGxCR 寄存器中的 GPOL(发生器极性)位域选择活动边。

在触发事件发生时，相应的生成器通道开始在其输出上生成 DMA 请求。每次连接的 DMA 控制器为 DMAMUX 生成的请求提供服务时(被服务的请求被撤销)，一个内置的(DMAMUX 请求生成器内部的)DMA 请求计数器就会递减。在运行不足时，请求生成器通道停止生成 DMA 请求，DMA 请求计数器在下一个触发事件时自动重新加载到其编程值。

因此，触发事件后产生的 DMA 请求数为 GNBREQ + 1。

注:只有当对应发生器通道 x 的使能 GE 位关闭时，才需要软件写入 GNBREQ 位域值。

没有硬件写保护。

如果沿边缘的状态在两个以上的 AHB 时钟周期内保持稳定，则检测到触发事件(边缘)。

在写入 DMAMUX\_RGxCR 寄存器时，触发事件在三个 AHB 时钟周期内被屏蔽。

#### 触发超限和中断

如果新的 DMA 请求触发事件发生在 DMAMUX 请求生成器计数器运行不足之前(通过 DMAMUX\_RGxCR 寄存器的 GNBREQ 位域编程的内部计数器)，并且如果请求生成器通道 x 通过 GE 使能，那么请求触发事件溢出标志位 OFx 由 DMAMUX\_RGSR 寄存器状态中的硬件断言。

注意: 请求生成器通道 x 必须禁用(DMAMUX\_RGxCR)。在使用完 DMA 控制器的相关通道时，GE = 0)。否则，在检测到一个新的触发事件时，由于没有从 DMA 接收到确认(即没有服务请求)而导致触发器溢出。

通过在 DMAMUX\_RGCFR 寄存器中设置相关的清除溢出标志位 COFx 来重置溢出标志 OFx。

如果在 DMAMUX\_RGxCR 寄存器中设置了 DMA 请求触发事件溢出中断使能位 OIE，则设置 DMAMUX 请求触发溢出标志会产生中断。

### 19.4 DMAMUX 中断

DMAMUX 中断

中断可以在以下情况下产生：

- 每个 DMA 请求线多路复用器通道中的同步事件溢出
- 每个 DMA 请求生成器通道中的触发事件溢出

对于每种情况，每个通道单独的中断使能，状态和清除标志寄存器位可用。

表 19.2 DMAMUX 中断

| 中断信号         | 中断事件                           | 标志位  | 清除位   | 使能位  |
|--------------|--------------------------------|------|-------|------|
| dmamuxovr_it | DMAMUX 请求线路多路复用器的通道 x 上的同步事件溢出 | SOFx | CSOFx | SOIE |

|  |                           |     |      |     |
|--|---------------------------|-----|------|-----|
|  | DMAMUX 请求生成器的通道 x 上触发事件溢出 | OFx | COFx | OIE |
|--|---------------------------|-----|------|-----|

## 19.5 DMAMUX 映射

表 19.3 分配多路复用器输入到 DMAMUX 资源表

| DMA 请求<br>MUX 输入 | 输入源           | DMA 请求<br>MUX 输入 | 输入源       | DMA 请求<br>MUX 输入 | 输入源                          |
|------------------|---------------|------------------|-----------|------------------|------------------------------|
| 1                | DMAMUX_Req G0 | 44               | TIM1_CH3  | 87               | 保留                           |
| 2                | DMAMUX_Req G1 | 45               | TIM1_CH4  | 88               | 保留                           |
| 3                | DMAMUX_Req G2 | 46               | TIM1_UP   | 89               | 保留                           |
| 4                | DMAMUX_Req G3 | 47               | TIM1_TRIG | 90               | 保留                           |
| 5                | ADC1          | 48               | TIM1_COM  | 91               | 保留                           |
| 6                | DAC1_CH1      | 49               | TIM8_CH1  | 92               | 保留                           |
| 7                | DAC1_CH2      | 50               | TIM8_CH2  | 93               | 保留                           |
| 8                | TIM6_UP       | 51               | TIM8_CH3  | 94               | 保留                           |
| 9                | TIM7_UP       | 52               | TIM8_CH4  | 95               | HRTIM_MASTER<br>(hrtim_dma1) |
| 10               | SPI1_RX       | 53               | TIM8_UP   | 96               | HRTIM_TIMA<br>(hrtim_dma2)   |
| 11               | SPI1_TX       | 54               | TIM8_TRIG | 97               | HRTIM_TIMB<br>(hrtim_dma3)   |
| 12               | SPI2_RX       | 55               | TIM8_COM  | 98               | HRTIM_TIMC<br>(hrtim_dma4)   |
| 13               | SPI2_TX       | 56               | TIM2_CH1  | 99               | HRTIM_TIMD<br>(hrtim_dma5)   |
| 14               | 保留            | 57               | TIM2_CH2  | 100              | HRTIM_TIME<br>(hrtim_dma6)   |
| 15               | 保留            | 58               | TIM2_CH3  | 101              | HRTIM_TIMF<br>(hrtim_dma7)   |
| 16               | I2C1_RX       | 59               | TIM2_CH4  | 102              | DAC3_CH1                     |
| 17               | I2C1_TX       | 60               | TIM2_UP   | 103              | DAC3_CH2                     |
| 18               | I2C2_RX       | 61               | TIM3_CH1  | 104              | 保留                           |
| 19               | I2C2_TX       | 62               | TIM3_CH2  | 105              | 保留                           |
| 20               | 保留            | 63               | TIM3_CH3  | 106              | 保留                           |
| 21               | 保留            | 64               | TIM3_CH4  | 107              | 保留                           |
| 22               | 保留            | 65               | TIM3_UP   | 108              | 保留                           |
| 23               | 保留            | 66               | TIM3_TRIG | 109              | 保留                           |
| 24               | USART1_RX     | 67               | TIM4_CH1  | 110              | FMAC_Read                    |
| 25               | USART1_TX     | 68               | TIM4_CH2  | 111              | FMAC_Write                   |
| 26               | USART2_RX     | 69               | TIM4_CH3  | 112              | Cordic_Read                  |
| 27               | USART2_TX     | 70               | TIM4_CH4  | 113              | Cordic_Write                 |

| DMA 请求<br>MUX 输入 | 输入源       | DMA 请求<br>MUX 输入 | 输入源        | DMA 请求<br>MUX 输入 | 输入源      |
|------------------|-----------|------------------|------------|------------------|----------|
| 28               | USART3_RX | 71               | TIM4_UP    | 114              | 保留       |
| 29               | USART3_TX | 72               | TIM5_CH1   | 115              | 保留       |
| 30               | UART4_RX  | 73               | TIM5_CH2   | 116              | DAC2_CH2 |
| 31               | UART4_TX  | 74               | TIM5_CH3   | 117              | 保留       |
| 32               | UART5_RX  | 75               | TIM5_CH4   | 118              | 保留       |
| 33               | UART5_TX  | 76               | TIM5_UP    | 119              | 保留       |
| 34               | 保留        | 77               | TIM5_TRIG  | 120              | 保留       |
| 35               | 保留        | 78               | TIM15_CH1  | 121              | 保留       |
| 36               | ADC2      | 79               | TIM15_UP   | 122              | 保留       |
| 37               | ADC3      | 80               | TIM15_TRIG | 123              | 保留       |
| 28               | 保留        | 81               | TIM15_COM  | 124              | 保留       |
| 29               | 保留        | 82               | TIM16_CH1  | 125              | 保留       |
| 40               | 保留        | 83               | TIM16_UP   | 126              | 保留       |
| 41               | DAC2_CH1  | 84               | TIM17_CH1  | 127              | 保留       |
| 42               | TIM1_CH1  | 85               | TIM17_UP   | 保留               | 保留       |
| 43               | TIM1_CH2  | 86               | 保留         | 保留               | 保留       |

表 19.4 DMAMUX：触发输入源

| 触发输入 | 输入源        | 触发输入 | 输入源               |
|------|------------|------|-------------------|
| 0    | EXTILINE0  | 16   | DMAMUX1_ch0_event |
| 1    | EXTILINE1  | 17   | DMAMUX1_ch1_event |
| 2    | EXTILINE2  | 18   | DMAMUX1_ch2_event |
| 3    | EXTILINE3  | 19   | DMAMUX1_ch3_event |
| 4    | EXTILINE4  | 20   |                   |
| 5    | EXTILINE5  | 21   | 保留                |
| 6    | EXTILINE6  | 22   | 保留                |
| 7    | EXTILINE7  | 23   | 保留                |
| 8    | EXTILINE8  | 24   | 保留                |
| 9    | EXTILINE9  | 25   | 保留                |
| 10   | EXTILINE10 | 26   | 保留                |
| 11   | EXTILINE11 | 27   | 保留                |
| 12   | EXTILINE12 | 28   | 保留                |
| 13   | EXTILINE13 | 29   | 保留                |
| 14   | EXTILINE14 | 30   | 保留                |
| 15   | EXTILINE15 | 31   | 保留                |

表 19.5 DMAMUX：同步输入源

| 同步输入 | 输入源       | 同步输入 | 输入源               |
|------|-----------|------|-------------------|
| 0    | EXTILINE0 | 16   | DMAMUX1_ch0_event |

| 同步输入 | 输入源        | 同步输入 | 输入源               |
|------|------------|------|-------------------|
| 1    | EXTILINE1  | 17   | DMAMUX1_ch1_event |
| 2    | EXTILINE2  | 18   | DMAMUX1_ch2_event |
| 3    | EXTILINE3  | 19   | DMAMUX1_ch3_event |
| 4    | EXTILINE4  | 20   |                   |
| 5    | EXTILINE5  | 21   | 保留                |
| 6    | EXTILINE6  | 22   | 保留                |
| 7    | EXTILINE7  | 23   | 保留                |
| 8    | EXTILINE8  | 24   | 保留                |
| 9    | EXTILINE9  | 25   | 保留                |
| 10   | EXTILINE10 | 26   | 保留                |
| 11   | EXTILINE11 | 27   | 保留                |
| 12   | EXTILINE12 | 28   | 保留                |
| 13   | EXTILINE13 | 29   | 保留                |
| 14   | EXTILINE14 | 30   | 保留                |
| 15   | EXTILINE15 | 31   | 保留                |

## 19.6 DMAMUX 寄存器描述

基地址：0x4002 0800

### 19.6.1 DMAMUX 请求线路多路复用器通道 x 配置寄存器 (DMAMUX\_CxCR) (x=0...15)

偏移地址：0x00 + 0x04\*x

复位值：0x0000 0000

|          |    |    |              |    |    |     |      |            |                |    |    |    |           |    |    |
|----------|----|----|--------------|----|----|-----|------|------------|----------------|----|----|----|-----------|----|----|
| 31       | 30 | 29 | 28           | 27 | 26 | 25  | 24   | 23         | 22             | 21 | 20 | 19 | 18        | 17 | 16 |
| Reserved |    |    | SYNC_ID[4:0] |    |    |     |      | NBREQ[4:0] |                |    |    |    | SPOL[1:0] |    | SE |
|          |    |    | rw           | rw | rw | rw  | rw   | rw         | rw             | rw | rw | rw | rw        | rw | rw |
| 15       | 14 | 13 | 12           | 11 | 10 | 9   | 8    | 7          | 6              | 5  | 4  | 3  | 2         | 1  | 0  |
| Reserved |    |    |              |    |    | EGE | SOIE | Res        | DMAREQ_ID[6:0] |    |    |    |           |    |    |
|          |    |    |              |    |    | rw  | rw   |            | rw             | rw | rw | rw | rw        | rw | rw |

Bits 31:29 保留，必须保持为复位值。

Bits 28:24 **SYNC\_ID[4:0]**：同步识别，选择同步输入

Bits 23:19 **NBREQ[4:0]**：转发 DMA 请求数(减 1)

定义同步事件之后转发给 DMA 请求的数量，生成输出事件之前的 DMA 请求的数量。

只能在 SE 和 EGE 位为低时才能被写入

Bits 18:17 **SPOL[1:0]**：同步极性，定义所选同步输入的边缘极性：

00：无事件，即不同步不检测

01：上升沿

10：下降沿

11：上升和下降沿

Bits 15:10 保留，必须保持为复位值。

Bit 9 **EGE**：事件生成使能

0：禁止事件生成

1：使能事件生成

Bit 8 **SOIE**: 同步超限中断使能

0: 中断禁止

1: 中断使能

Bit 7 保留, 必须保持为复位值。

Bits 6:0 **DMAREQ\_ID[6:0]**: DMA 请求识别

选择输入 DMA 请求, 参考分配多路复用器输入到 DMAMUX 资源表。

### 19.6.2 DMAMUX 请求线路多路复用器中断通道状态寄存器 (DMAMUX\_CSR)

偏移地址: 0x80

复位值: 0x0000 0000

|          |       |       |       |       |       |      |      |      |      |      |      |      |      |      |      |
|----------|-------|-------|-------|-------|-------|------|------|------|------|------|------|------|------|------|------|
| 31       | 30    | 29    | 28    | 27    | 26    | 25   | 24   | 23   | 22   | 21   | 20   | 19   | 18   | 17   | 16   |
| Reserved |       |       |       |       |       |      |      |      |      |      |      |      |      |      |      |
| 15       | 14    | 13    | 12    | 11    | 10    | 9    | 8    | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| SOF15    | SOF14 | SOF13 | SOF12 | SOF11 | SOF10 | SOF9 | SOF8 | SOF7 | SOF6 | SOF5 | SOF4 | SOF3 | SOF2 | SOF1 | SOF0 |
| r        | r     | r     | r     | r     | r     | r    | r    | r    | r    | r    | r    | r    | r    | r    | r    |

Bits 31:16 保留, 必须保持为复位值。

Bits 15:0 **SOF[15:0]**: 同步溢出事件标志

当 DMA 请求线路多路复用器通道 x 上发生同步事件, 而 DMA 请求计数器值低于 NBREQ 时此标志置起。

通过向 DMAMUX\_CFR 寄存器中相应的 CSOFx 位写 1 来清除该标志

### 19.6.3 DMAMUX 请求线路多路复用器中断清除标志寄存器 (DMAMUX\_CFR)

偏移地址: 0x84

复位值: 0x0000 0000

|          |        |        |        |        |        |       |       |       |       |       |       |       |       |       |       |
|----------|--------|--------|--------|--------|--------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 31       | 30     | 29     | 28     | 27     | 26     | 25    | 24    | 23    | 22    | 21    | 20    | 19    | 18    | 17    | 16    |
| Reserved |        |        |        |        |        |       |       |       |       |       |       |       |       |       |       |
| 15       | 14     | 13     | 12     | 11     | 10     | 9     | 8     | 7     | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| CSOF15   | CSOF14 | CSOF13 | CSOF12 | CSOF11 | CSOF10 | CSOF9 | CSOF8 | CSOF7 | CSOF6 | CSOF5 | CSOF4 | CSOF3 | CSOF2 | CSOF1 | CSOF0 |
| w        | w      | w      | w      | w      | w      | w     | w     | w     | w     | w     | w     | w     | w     | w     | w     |

Bits 31:16 保留, 必须保持为复位值。

Bits 15:0 **CSOF[15:0]**: 清除同步溢出事件标志

在每位写入 1 清除 DMAMUX\_CSR 寄存器中相应的溢出标志 SOFx

### 19.6.4 DMAMUX 请求生成器通道 x 配置寄存器 (DMAMUX\_RGxCR) (x=0...3)

偏移地址: 0x100 + 0x04\*x

复位值: 0x0000 0000

|          |    |    |    |    |    |    |     |             |    |    |             |           |    |    |    |
|----------|----|----|----|----|----|----|-----|-------------|----|----|-------------|-----------|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24  | 23          | 22 | 21 | 20          | 19        | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |    |     | GNBREQ[4:0] |    |    |             | GPOL[1:0] |    | GE |    |
|          |    |    |    |    |    |    |     | rw          | rw | rw | rw          | rw        | rw | rw | rw |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8   | 7           | 6  | 5  | 4           | 3         | 2  | 1  | 0  |
| Reserved |    |    |    |    |    |    | OIE | Reserved    |    |    | SIG_ID[4:0] |           |    |    |    |
|          |    |    |    |    |    |    | rw  |             |    |    | rw          | rw        | rw | rw | rw |

Bits 31:16 保留, 必须保持为复位值。

Bits 23:19 **GNBREQ[4:0]**: 要生成的 DMA 请求数(减 1)

定义在触发器事件之后生成的 DMA 请求的数量, 生成的 DMA 请求的时间数量是



GNBREQ+1

只能在禁止 GE 位时写入

Bits 18:17 **GPOL[1:0]**: DMA 请求发生器极性, 定义所选触发器输入的边缘极性:

00: 无事件, 即没有触发检测和生成

01: 上升沿

10: 下降沿

11: 上升和下降沿

Bit 16 **GE**: DMA 请求发生器通道 x 使能

0: 禁止 DMA 请求发生器通道 x

1: 使能 DMA 请求发生器通道 x

Bits 15:9 保留, 必须保持为复位值。

Bit 8 **OIE**: 触发溢出中断使能

0: 禁止触发溢出事件

1: 使能触发溢出事件

Bits 7:5 保留, 必须保持为复位值。

Bits 4:0 **SIG\_ID[4:0]**: 信号识别

选择 DMA 请求生成器的通道 x 使用的 DMA 请求触发器输入

### 19.6.5 DMAMUX 请求生成器中断状态寄存器 (DMAMUX\_RGSR)

偏移地址: 0x140

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |    |    |    |    |     |     |     |     |
|----------|----|----|----|----|----|----|----|----|----|----|----|-----|-----|-----|-----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19  | 18  | 17  | 16  |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |     |     |     |     |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3   | 2   | 1   | 0   |
|          |    |    |    |    |    |    |    |    |    |    |    | OF3 | OF2 | OF1 | OF0 |
|          |    |    |    |    |    |    |    |    |    |    |    | r   | r   | r   | r   |

Bits 31:16 保留, 必须保持为复位值。

Bits 15:0 **CSOF[3:0]**: 触发溢出事件标志

当 DMA 请求生成器通道 x 上发生新的触发事件, 在请求计数器运行之前 (通过 DMAMUX\_RGxCR 寄存器的 GNBREQ 位域编写的内部请求计数器), 此标志置起。

通过向 DMAMUX\_RGCFR 寄存器中相应的 COFx 位写 1 来清除该标志

### 19.6.6 DMAMUX 请求生成器中断清除标志寄存器 (DMAMUX\_RGCFR)

偏移地址: 0x144

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |    |    |    |    |      |      |      |      |
|----------|----|----|----|----|----|----|----|----|----|----|----|------|------|------|------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19   | 18   | 17   | 16   |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |      |      |      |      |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3    | 2    | 1    | 0    |
|          |    |    |    |    |    |    |    |    |    |    |    | COF3 | COF2 | COF1 | COF0 |
|          |    |    |    |    |    |    |    |    |    |    |    | w    | w    | w    | w    |

Bits 31:16 保留, 必须保持为复位值。

Bits 15:0 **CSOF[3:0]**: 清除同步溢出事件标志

在每位写入 1 清除 DMAMUX\_RGSR 寄存器中相应的溢出标志 OFx



## 20 比较器 (COMP)

### 20.1 简介

该设备嵌入了 4 个超高速模拟比较器。

这些比较器可用于各种功能，包括：

- 在模拟信号的触发下从低功耗模式唤醒
- 模拟信号调理
- 与定时器的 PWM 输出结合使用时，构成逐周期电流控制环路

### 20.2 主要特性

- 每个比较器具有可配置的正负输入，用于灵活的电压选择：
  - 多路 I/O 引脚
  - DAC 通道
  - 通过调节器（缓冲分压器）提供的内部参考电压和三个因数分压值（1/4、1/2、3/4）
- 可编程迟滞
- 输出重定向到 I/O 或定时器输入，用于触发快速 PWM 关断的刹车事件
- 输出遮蔽，以抵抗开关噪声
- 每个通道生成的中断，可用于从睡眠和停止模式唤醒

### 20.3 功能描述

#### 20.3.1 COMP 框图

一个比较器 COMP 框图如下图所示：

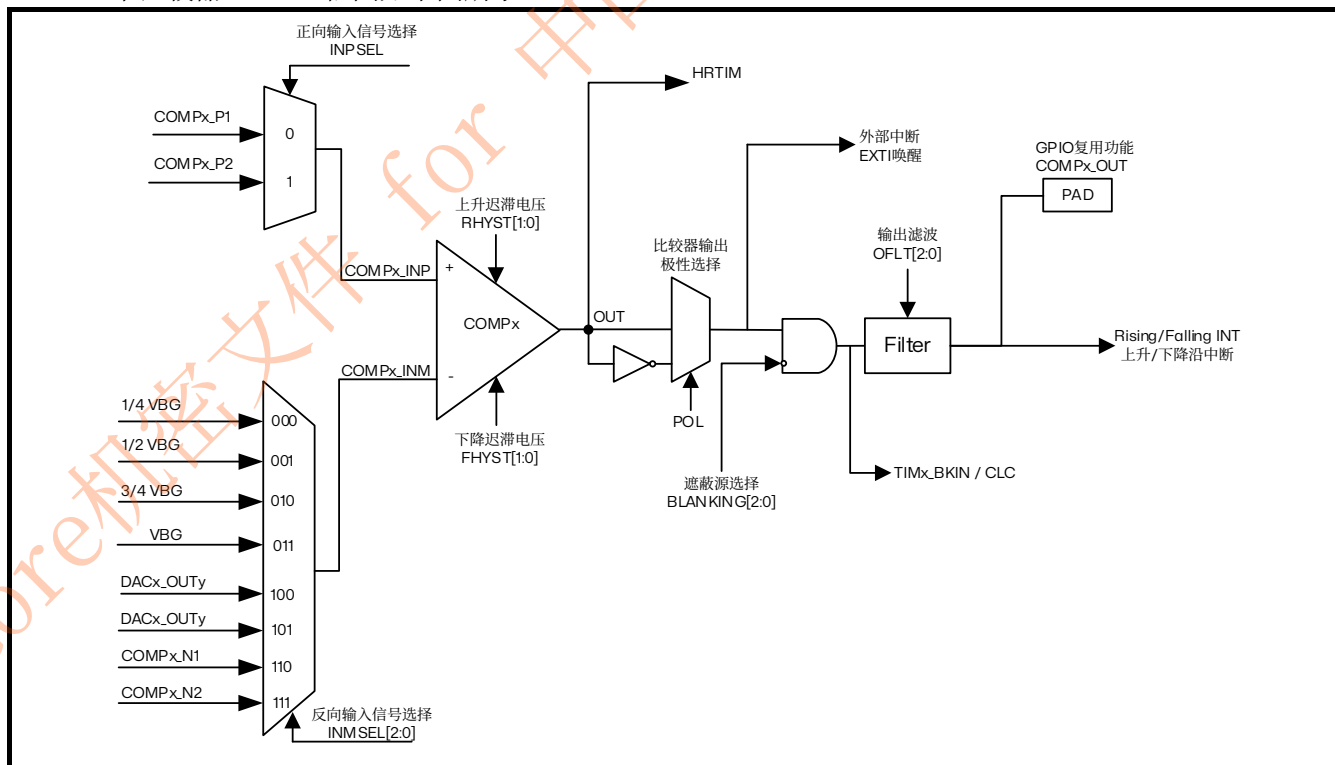


图 20.1 COMP 框图

### 20.3.2 COMP 引脚和内部信号

用作比较器输入的 I/O 必须在 GPIO 寄存器中配置为模拟模式。

比较器输出可以使用数据手册“复用功能映射”表中给出的复用功能通道连接到 I/O。

输出也可以在内部重定向到用于以下用途的各种定时器输入：

- 使用 BKIN 和 BKIN2 输入紧急关断 PWM 信号
- 使用 OCREF\_CLR 输入进行逐周期电流控制
- 用于时序测量的输入捕捉

可以同时将比较器输出内部和外部重定向。

表 20.1 COMPx 正向输入分配

| INPSEL | COMP1_INP | COMP2_INP | COMP3_INP | COMP4_INP |
|--------|-----------|-----------|-----------|-----------|
| 0      | PA0       | PA1       | PB13      | PB14      |
| 1      | PC15      | PC5       | PE8       | PE12      |

表 20.2 COMPx 反向输入分配

| INMSEL[2:0] | COMP1_INM | COMP2_INM | COMP3_INM | COMP4_INM |
|-------------|-----------|-----------|-----------|-----------|
| 000         | 1/4 VBG   |           |           |           |
| 001         | 1/2 VBG   |           |           |           |
| 010         | 3/4 VBG   |           |           |           |
| 011         | VBG       |           |           |           |
| 100         | DAC3_CH1  | DAC3_CH2  | DAC3_CH1  | DAC3_CH2  |
| 101         | DAC1_CH1  | DAC1_CH2  | DAC2_CH1  | DAC2_CH2  |
| 110         | PC0       | PC1       | PD9       | PD10      |
| 111         | PC14      | PC4       | PE7       | PE11      |

### 20.3.3 COMP 复位和时钟

由时钟控制器提供的 COMP 时钟与 APB2 时钟同步。

RCC 控制器中没有专用于 COMP 时钟使能控制位。复位和时钟使能位是 COMP 和 SYSCFG 共用的。

注：重要：极性选择逻辑和输出重定向到端口独立于 APB 时钟工作。这使得比较器即使在停止模式下也能工作。

### 20.3.4 COMP 锁定机制

比较器能用于安全的用途，比如过流或者过热保护。在某些特定的安全需求的应用中，有必要保证比较器设置不能被无效寄存器访问或者程序计数器破坏所改变。

为此，比较器控制和状态寄存器可以是写保护的（只读）。

一旦编程完成，就可以设置 COMP\_CxCR 寄存器的 LOCK 位。这会导致大部分寄存器变成只读的，具体位参阅寄存器章节。

只能通过 MCU 复位来移除写保护。

### 20.3.5 COMP 防误触发功能

#### (1) 当 COMP 和 ADC 同时使用时

当 COMP 的 P 或 N 端与 ADC 采样通道是同一个 IO 时，当外部接 RC 电路或 ADC 在通道转换时造成压差，此时 COMP 可能被 ADC 在通道转换时产生的压差影响而导致误触发。

当 COMP\_CxCR 寄存器的 BAS\_EN=1 时，根据 COMP\_CxCR 寄存器的 INPSEL 和 COMP\_CxCR 寄存器的 INMSEL[2:0] (x=1/2/3/4) 的配置，在对应的 ADC 采样通道采样时对 COMP 输出进行遮蔽，这样可以防止由 ADC 的转换压差和 RC 电路造成的 COMP 误触发。

下两张表分别为 COMPx 的 P 端/N 端和 ADC 的 IO 对应关系。

表 20.3 COMPx\_P 端和 ADC 的 IO 对应关系

| CxCR_INPSEL | 值 | P 端输入    | ADC 采样通道 |
|-------------|---|----------|----------|
| C1CR_INPSEL | 0 | COMP1_P1 | ADC1_IN4 |
|             | 1 | COMP1_P2 | ADC1_IN2 |
| C2CR_INPSEL | 0 | COMP2_P1 | ADC1_IN5 |
|             | 1 | COMP2_P2 | ADC2_IN2 |
| C3CR_INPSEL | 0 | COMP3_P1 | ADC3_IN5 |
|             | 1 | COMP3_P2 | -        |
| C4CR_INPSEL | 0 | COMP4_P1 | ADC3_IN6 |
|             | 1 | COMP4_P2 | -        |

表 20.4 COMPx\_N 端和 ADC 的 IO 对应关系

| CxCR_INMSEL | 值   | N 端输入    | ADC 采样通道 |
|-------------|-----|----------|----------|
| C1CR_INMSEL | 110 | COMP1_N1 | ADC1_IN3 |
|             | 111 | COMP1_N2 | ADC1_IN1 |
| C2CR_INMSEL | 110 | COMP2_N1 | ADC1_IN7 |
|             | 111 | COMP2_N2 | ADC2_IN1 |
| C3CR_INMSEL | 110 | COMP3_N1 | ADC3_IN8 |
|             | 111 | COMP3_N2 | -        |
| C4CR_INMSEL | 110 | COMP4_N1 | ADC3_IN9 |
|             | 111 | COMP4_N2 | -        |

#### (2) 当 COMP 切换引脚时

当 COMP 的 P 端或 N 端切换引脚会产生电压变化，此时 COMP 会被影响而导致误触发。

当需要切换引脚时，COMP 分别提供了硬件和软件两种遮蔽方式。

硬件遮蔽默认开启，软件遮蔽需要额外配置，具体操作如下：

##### 硬件遮蔽：

- COMP 的 P 端和 N 端切换默认硬件遮蔽。即 CxCR\_CH\_BLANK\_EN 默认为 1 开启遮蔽功能。

##### 软件遮蔽：

- 使能 CxCR\_CH\_DIS\_BLANKING 打开遮蔽
- 使能 CxCR\_CH\_DIS 关闭 COMP 的 P 和 N 通道
- 切换 P 或 N 通道
- 失能 CxCR\_CH\_DIS 开启 COMP 的 P 和 N 通道
- 失能 CxCR\_CH\_DIS\_BLANKING 关闭遮蔽

### 20.3.6 COMP 迟滞

该比较器具有可编程迟滞，可在有噪声信号时避免发生意外输出转换。它是对称的，既作用于比较器输出的下降沿，又作用于比较器输出的上升沿。迟滞可在不需要时（例如，退出低功耗模式时）禁止，以便使用外部组件强制迟滞值。

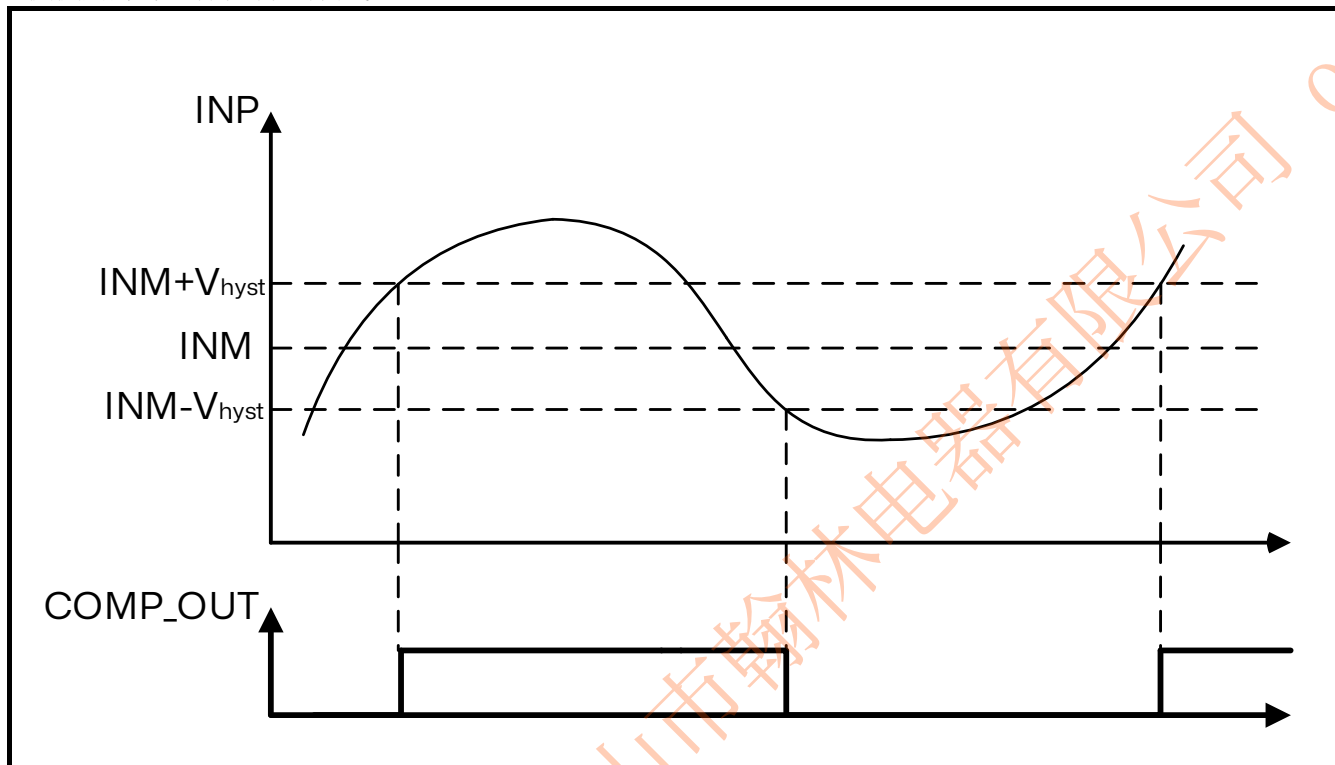


图 20.2 比较器迟滞

### 20.3.7 COMP 输出遮蔽功能

消除功能的目的是防止电流调节在 PWM 周期开始时（通常是功率开关反并联二极管中的恢复电流）出现短时尖峰时跳闸。这里通过设置一个带有定时器输出比较信号的遮蔽窗来实现的。每个比较器通道有软件通过对应的 COMP\_CxCR 寄存器的 BLANKING[2:0]位域单独选择遮蔽源，如下表：遮蔽源所示。

表 20.5 遮蔽源

| BLANKSEL[2:0] | COMP1     | COMP2     | COMP3     | COMP4     |
|---------------|-----------|-----------|-----------|-----------|
| 001           | TIM1_OC5  | TIM1_OC5  | TIM1_OC5  | TIM1_OC5  |
| 010           | TIM2_OC3  | TIM2_OC3  | TIM2_OC3  | TIM2_OC3  |
| 011           | TIM3_OC3  | TIM3_OC3  | TIM3_OC3  | TIM3_OC3  |
| 100           | TIM8_OC5  | TIM8_OC5  | TIM8_OC5  | TIM8_OC5  |
| 101           | TIM5_OC3  | TIM5_OC3  | TIM5_OC3  | TIM5_OC3  |
| 110           | TIM15_OC1 | TIM15_OC1 | TIM15_OC1 | TIM15_OC1 |
| 111           | TIM4_OC3  | TIM4_OC3  | TIM4_OC3  | TIM4_OC3  |

对遮蔽信号进行取反后与比较器级联输出进行逻辑与操作，以产生比较器通道 x 的输出。请参考下图中提供的示例。

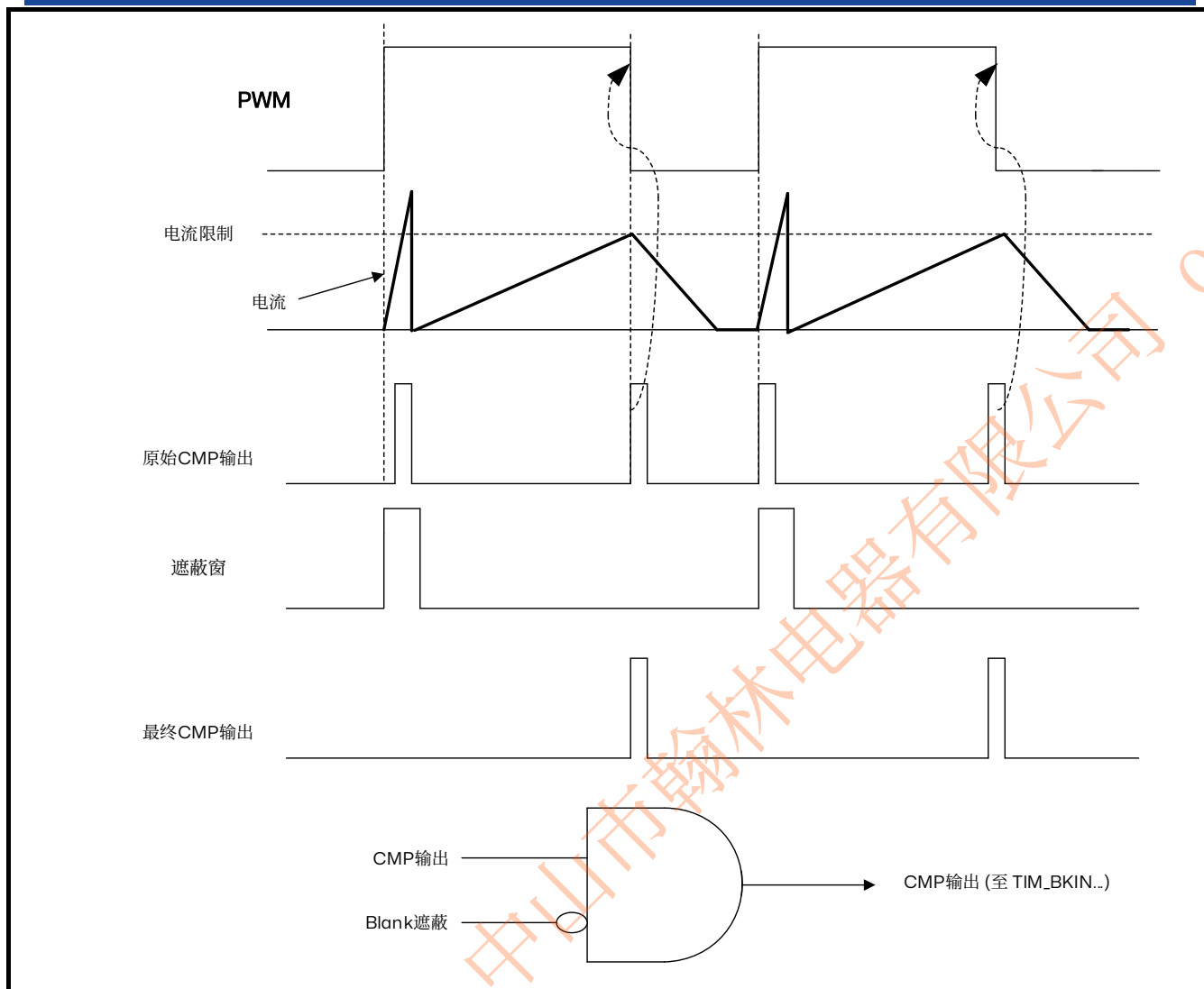


图 20.3 比较器输出遮蔽图

### 20.3.8 COMP 低功耗模式

表 20.6 比较器在低功耗模式下的行为

| 模式      | 描述                             |
|---------|--------------------------------|
| Sleep   | 对比较器没有影响。<br>比较器中断可使器件退出睡眠模式。  |
| Stop    | 对比较器没有影响。<br>比较器中断可使器件退出停止模式。  |
| Standby | 导致比较器寄存器断电。<br>退出待机模式后必须重新初始化。 |

### 20.3.9 COMP 中断

比较器输出从内部连接到扩展中断和事件控制器。每个比较器都具有其各自的 EXTI 线，能够产生中断或事件，相同的机制用于退出睡眠和停止的低功耗模式。

要使能 COMPx 中断，需要遵循以下顺序：

1. 将 EXTI 线配置为中断模式，选择上升沿、下降沿或任一边沿有效，然后使能与 COMPx 输出事件对应的 EXTI 线。
2. 配置并使能映射到相应 EXTI 线的 NVIC IRQ 通道。
3. 使能 COMPx。

中断事件通过 COMP\_CxCR 标志进行标记。

## 20.4 COMP 寄存器描述

### 20.4.1 比较器 x 控制和状态寄存器 (COMP\_CxCR) (x=1/2/3/4)

地址偏移:  $0x000+0x04*(x-1)$

复位值: 0x0000 0002

| 31   | 30        | 29    | 28    | 27                      | 26         | 25         | 24     | 23  | 22          | 21            | 20 | 19  | 18  | 17                  | 16 |
|------|-----------|-------|-------|-------------------------|------------|------------|--------|-----|-------------|---------------|----|-----|-----|---------------------|----|
| LOCK | OUT       | LIF   | FIF   | RIF                     | HIF        | FHYST[1:0] |        | LIE | BRGEN       | BLANKING[2:0] |    |     | HIE | RHYST[1:0]          |    |
| rw   | r         | rc_w0 | rc_w0 | rc_w0                   | rc_w0      | rw         | rw     | rw  | rw          | rw            | rw | rw  | rw  | rw                  | rw |
| 15   | 14        | 13    | 12    | 11                      | 10         | 9          | 8      | 7   | 6           | 5             | 4  | 3   | 2   | 1                   | 0  |
| POL  | OFLT[2:0] |       |       | CH_DIS_<br>BLANK<br>ING | CH_<br>DIS | BAS_<br>EN | INPSEL | Res | INMSEL[2:0] |               |    | FIE | RIE | CH_<br>BLANK_<br>EN | EN |
|      |           |       |       |                         |            |            |        |     |             |               |    |     |     |                     |    |
| rw   | rw        | rw    | rw    |                         | rw         | rw         | rw     |     | rw          | rw            | rw | rw  | rw  | rw                  | rw |

#### Bit 31 LOCK: COMP\_CxCR 寄存器锁定

该位由软件设置和清除。此位设置后，除了 CH\_BALNK\_EN、INPSEL、POL、CH\_DIS、CH\_DIS\_BLANKING、RIF、FIF、HIF、LIF、RIE、FIE、HIE、LIE 以外，CxCR[31:0]的其他位只能读（对应的 CAL 寄存器也会锁上，只有 APB2RSTR\_SYSCFG\_RST 置 1 可解除 lock）

0: 解锁，除了 OUT 只读以外，CxCR[31:0]的其他位能读/写

1: 锁定

#### Bit 30 OUT: 比较器 x 输出状态

只读标志反映了在极性选择器和遮蔽之前的比较器 x 的输出电平

0: 低输出

1: 高输出

#### Bit 29 LIF: 低电平输出中断事件标志位

0: 无低电平输出中断事件

1: 低电平输出中断事件发生

#### Bit 28 FIF: 下降沿中断事件标志位

0: 无下降沿中断事件

1: 下降沿中断事件发生

#### Bit 27 RIF: 上升沿中断事件标志位

0: 无上升沿中断事件

1: 上升沿中断事件发生

#### Bit 26 HIF: 高电平输出中断事件标志位

0: 无高电平输出中断事件

1: 高电平输出中断事件发生

#### Bit 25:24 FHYST[1:0]: 下降迟滞电压

00: 0mv

01: 20mv

10: 40mv

11: 60mv

Bit 23 **LIE**: 低电平输出中断使能

0: 使能低电平输出中断

1: 禁止低电平输出中断

Bit 22 **BRGEN**: VBG 标量电阻桥使能

此软件控制的位使 VBG 标量电阻桥能够工作。若要禁用电阻桥，必须将所有 COMP\_CxCSR 寄存器的 BRGEN 位设置为禁用状态。当电阻桥被禁用时，输入选择器的 1/4 VBG、1/2 VBG 和 3/4 VBG 输入将不再获得 VBG 电阻桥分压后的电压，而是直接接收到 VBG 电压。

0: 禁止

1: 使能

Bit 21:19 **BLANKING[2:0]**: 比较器 x 遮蔽源选择

此位域由软件控制，为比较器通道 x 选择遮蔽源，详情如表 20.3 遮蔽源所示。

000: 无遮蔽

001: TIM1 OC5 被选为遮蔽源

010: TIM2 OC3 被选为遮蔽源

011: TIM3 OC3 被选为遮蔽源

100: TIM8 OC5 被选为遮蔽源

101: TIM5 OC3 被选为遮蔽源

110: TIM15 OC1 被选为遮蔽源

111: TIM4 OC3 被选为遮蔽源

Bit 18 **HIE**: 高电平输出中断使能

0: 使能高电平输出中断

1: 禁止高电平输出中断

Bit 17:16 **RHYST[1:0]**: 上升迟滞电压

00: 0mv

01: 20mv

10: 40mv

11: 60mv

Bit 15 **POL**: 比较器输出极性

此位由软件选择控制比较器 x 的输出极性

0: 同向输出

1: 反向输出

Bit 14:12 **OFLT[2:0]**: 输出滤波，连续的 PCLK 时钟比较输出不变则认为有效，否则保持不变

000: 无滤波

001: 16 个时钟周期

010: 32 个时钟周期

011: 64 个时钟周期

100: 128 个时钟周期

101: 256 个时钟周期

110: 512 个时钟周期

111: 1024 个时钟周期



Bit 11 **CH\_DIS\_BLANKING**: 切换通道前进行 BLANKING 遮蔽

0: 通道切换前不开启遮蔽

1: 通道切换前开启遮蔽

Bit 10 **CH\_DIS**: P 端或 N 端通道切换时软件主动关闭通道

0: 通道切换时不主动关闭通道

1: 通道切换时软件主动关闭通道, 避免切换引脚造成 COMP 误触发

Bit 9 **BAS\_EN**: ADC 采样的遮蔽使能

0: 遮蔽 ADC 采样禁止

1: 遮蔽 ADC 采样使能 (参考表 20.3 和表 20.4 获得 COMP 和 ADC 的 IO 对应关系)

Bit 8 **INPSEL**: 比较器 x 正向输入信号选择

此位由软件控制来选择比较器 x 的正向输入 COMPx\_INP 的信号, 如表 20.1 所示:

COMPx 正向输入分配

0: COMPx\_P1

1: COMPx\_P2

Bit 7 保留, 必须保持为复位值。

Bit 6:4 **INMSEL[2:0]**: 比较器 x 反向输入信号选择

此位域由软件控制来选择比较器 x 的反向输入 COMPx\_INM 的信号, 如表 20.2 所示:

COMPx 反向输入分配

000: 1/4 VBG

001: 1/2 VBG

010: 3/4 VBG

011: VBG

100: DAC3\_CH1(COMP1) DAC3\_CH2(COMP2)

DAC3\_CH1(COMP3) DAC3\_CH2(COMP4)

101: DAC1\_CH1(COMP1) DAC1\_CH2(COMP2)

DAC2\_CH1(COMP3) DAC2\_CH2(COMP4)

110: COMPx\_N1

111: COMPx\_N2

Bit 3 **FIE**: 下降沿中断使能

0: 下降沿中断禁止

1: 下降沿中断使能

Bit 2 **RIE**: 上升沿中断使能

0: 上升沿中断禁止

1: 上升沿中断使能

Bit 1 **CH\_BLANK\_EN**: 硬件切换 P 端和 N 端通道遮蔽

由硬件默认为 1 开启 P 端和 N 端通道切换遮蔽, 可软件关闭。

具体参考章节 20.3.5 比较器防误触发功能。

0: 硬件遮蔽关闭

1: 硬件遮蔽使能

Bit 0 **EN**: 比较器 x 使能

0: 禁止

1: 使能

注: 比较器的时钟由 RCC\_APB2ENR 的 SYSCFGEN 位控制。

## 20.4.2 比较器 x 校正寄存器 (COMP\_RGxCAL) (x=1/2/3/4)

地址偏移:  $0x040 + 0x04 \times (x-1)$

复位值: 0x0000 0808

|          |    |    |    |           |    |    |    |          |    |     |     |           |    |    |    |
|----------|----|----|----|-----------|----|----|----|----------|----|-----|-----|-----------|----|----|----|
| 31       | 30 | 29 | 28 | 27        | 26 | 25 | 24 | 23       | 22 | 21  | 20  | 19        | 18 | 17 | 16 |
| Reserved |    |    |    |           |    |    |    |          |    |     |     |           |    |    |    |
| 15       | 14 | 13 | 12 | 11        | 10 | 9  | 8  | 7        | 6  | 5   | 4   | 3         | 2  | 1  | 0  |
| Reserved |    |    |    | NADJ[3:0] |    |    |    | Reserved |    | NEN | PEN | PADJ[3:0] |    |    |    |
|          |    |    |    | rw        | rw | rw | rw |          |    | rw  | rw  | rw        | rw | rw | rw |

Bit 31:12 保留, 必须保持为复位值。

Bit 11:8 **NADJ[3:0]**: 比较器 NMOS offset 偏置调节

Bit 7:6 保留, 必须保持为复位值。

Bit 5 **NEN**: 比较器 NMOS 校正使能位

0: COMP NMOS 校正禁止

1: COMP NMOS 校正使能

Bit 4 **PEN**: 比较器 PMOS 校正使能位

0: COMP PMOS 校正禁止

1: COMP PMOS 校正使能

Bit 3:0 **PADJ[3:0]**: 比较器 PMOS offset 偏置调节

## 21 运算放大器 (OPAMP)

### 21.1 简介

该器件嵌入四个运算放大器，每个运算放大器有两个输入和一个输出。三个 I/O 可以连接到外部引脚，从而实现任何类型的外部互连。运算放大器可以在内部配置为从动器，作为一个非逆增益范围为 4 到 32 放大器。

### 21.2 主要特性

- 轨对轨输入和输出电压范围
- 低输入偏置电流
- 输入偏置电压低
- 高频增益带宽
- 高速模式，实现更好的回转速率

### 21.3 功能描述

OPAMP 有几种模式。每个 OPAMP 可以单独使能，当禁用输出是高阻抗。当使能时，它可以处于校准模式，然后断开 OPAMP 的所有输入，或处于功能模式。

#### 21.3.1 OPAMP 输出重定向到内部 ADC 通道

运算放大器输出可以通过在 OPAMPx\_CSR 寄存器中设置 OPAINTOEN 位在内部重定向到 ADC 通道。在本例中，给定 OPAMPx\_OUT 输出所映射的 GPIO 是空闲的，可以用于其他目的。ADC 可以内部测量 OPAMPx\_OUT 电压。如下表所示。

表 21.1 运放 OPAMPx 内部输出和 ADC 通道之间可能的分配连接

| 信号          | 引脚            | 内部的                    | 描述                                           |
|-------------|---------------|------------------------|----------------------------------------------|
| OPAMP1_VINM | PA3 (VINM_N1) | OPAMP1_OUT<br>or PGA   | 由位 PGA_GAIN 和 VM_SEL 控制                      |
|             | PC3 (VINM_N2) |                        |                                              |
|             | PC1 (VINM_N3) |                        |                                              |
| OPAMP1_VINP | PA2 (VINP_P1) |                        | 由位 VP_SEL 控制。                                |
|             | PC0 (VINP_P2) |                        |                                              |
| OPAMP1_OUT  | PA4           | ADC1_IN11<br>ADC2_IN9  | 当 OPAMP 使能，OPAMP 内部输出关闭时，引脚连接。ADC 输入由 ADC 控制 |
| OPAMP2_VINM | PA6 (VINM_N1) | OPAMP2_OUT or<br>PGA   | 由位 PGA_GAIN 和 VM_SEL 控制                      |
|             | PC4 (VINM_N2) |                        |                                              |
|             | PE7 (VINM_N3) |                        |                                              |
| OPAMP2_VINP | PA5 (VINP_P1) |                        | 由位 VP_SEL 控制                                 |
|             | PE8 (VINP_P2) |                        |                                              |
| OPAMP2_OUT  | PA7           | ADC1_IN12<br>ADC2_IN10 | 当 OPAMP 使能，OPAMP 内部输出关闭时，引脚连接。ADC 输入由 ADC 控制 |
| OPAMP3_VINM | PB1 (VINM_N1) | OPAMP3_OUT or          | 由位 PGA_GAIN 和 VM_SEL 控制                      |

| 信号          | 引脚             | 内部的                    | 描述                                             |
|-------------|----------------|------------------------|------------------------------------------------|
|             | PC5 (VINM_N2)  | PGA                    |                                                |
|             | PE9 (VINM_N3)  |                        |                                                |
| OPAMP3_VINP | PB2 (VINP_P1)  |                        | 由位 VP_SEL 控制                                   |
|             | PE10 (VINP_P2) |                        |                                                |
| OPAMP3_OUT  | PB0            | ADC2_IN11<br>ADC3_IN11 | 当 OPAMP 使能, OPAMP 内部输出关闭时, 引脚连接。ADC 输入由 ADC 控制 |
| OPAMP4_VINM | PB11 (VINM_N1) | OPAMP4_OUT or<br>PGA   | 由位 PGA_GAIN 和 VM_SEL 控制                        |
|             | PD8 (VINM_N2)  |                        |                                                |
|             | PE11 (VINM_N3) |                        |                                                |
| OPAMP4_VINP | PB10 (VINP_P1) |                        | 由位 VP_SEL 控制                                   |
|             | PE12 (VINP_P2) |                        |                                                |
| OPAMP4_OUT  | PB12           | ADC2_IN12<br>ADC3_IN12 | 当 OPAMP 使能, OPAMP 内部输出关闭时, 引脚连接。ADC 输入由 ADC 控制 |

### 21.3.2 OPAMP 复位和时钟

时钟控制器提供的 OPAMP 时钟与 PCLK2 (APB2 时钟) 同步。在 RCC 控制器中没有提供时钟使能控制位。要为 OPAMP 使用时钟源, 必须在 RCC 控制器中设置 SYSCFG 时钟使能控制位。

OPAEN 位表示 OPAMP 操作的使能和禁用。在使能 OPAEN 位之前, 必须更改 OPAMP 寄存器配置, 以避免对输出产生虚假影响。

当不再需要运算放大器的输出时, 可以关闭运算放大器以节省功率。当 OPAMP 被禁用时, 之前设置的所有配置 (包括校准) 都将保持。

### 21.3.3 初始化配置

运算放大器的默认配置是一个功能模式, 其中三个输入/输出连接到外部引脚。在默认模式下, 运算放大器使用工厂微调值进行偏移校准。可以调整微调值。默认配置为正常模式, 提供标准的性能。可以设置位 OPAHSM, 以便将运算放大器切换到高速模式以获得更好的转换速率。正常和高速模式特性在数据表中定义。

一旦 OPAMPx\_CSR 寄存器中的 OPAEN 位被设置, 运算放大器就开始工作。

注意: 输入和输出引脚必须在相应的 GPIOx\_MODER 寄存器中配置为模拟模式 (默认状态)。

### 21.3.4 OPAMP 模式

运算放大器的输入和输出都可以在终端上访问。该放大器可用于多种配置环境:

- 独立模式 (外部增益设置模式)
- 从动件配置模式
- PGA 模式

注意: 信号的阻抗必须保持在一个水平以下, 以避免输入泄漏产生明显的伪影 (由于源的电阻下降)。有关详细信息, 请参阅数据表中的电气特性部分。

#### 独立模式 (外部增益设置模式)

给出在独立模式下使用 OPAMP 的过程。

从 OPAMPx\_CSR 的默认值和 GPIOx\_MODER 的默认状态开始，只要设置 OPAEN 位，两个输入引脚和输出引脚就连接到运放。

这个默认配置使用工厂微调值，并在正常模式下运行（最高性能）。OPAMP 的行为可以改变如下：

- OPAHSM 可设置为“运算放大器高速”模式，以获得高转换率。
- USERTRIM 可以设置为修改输入偏移量的修剪值。

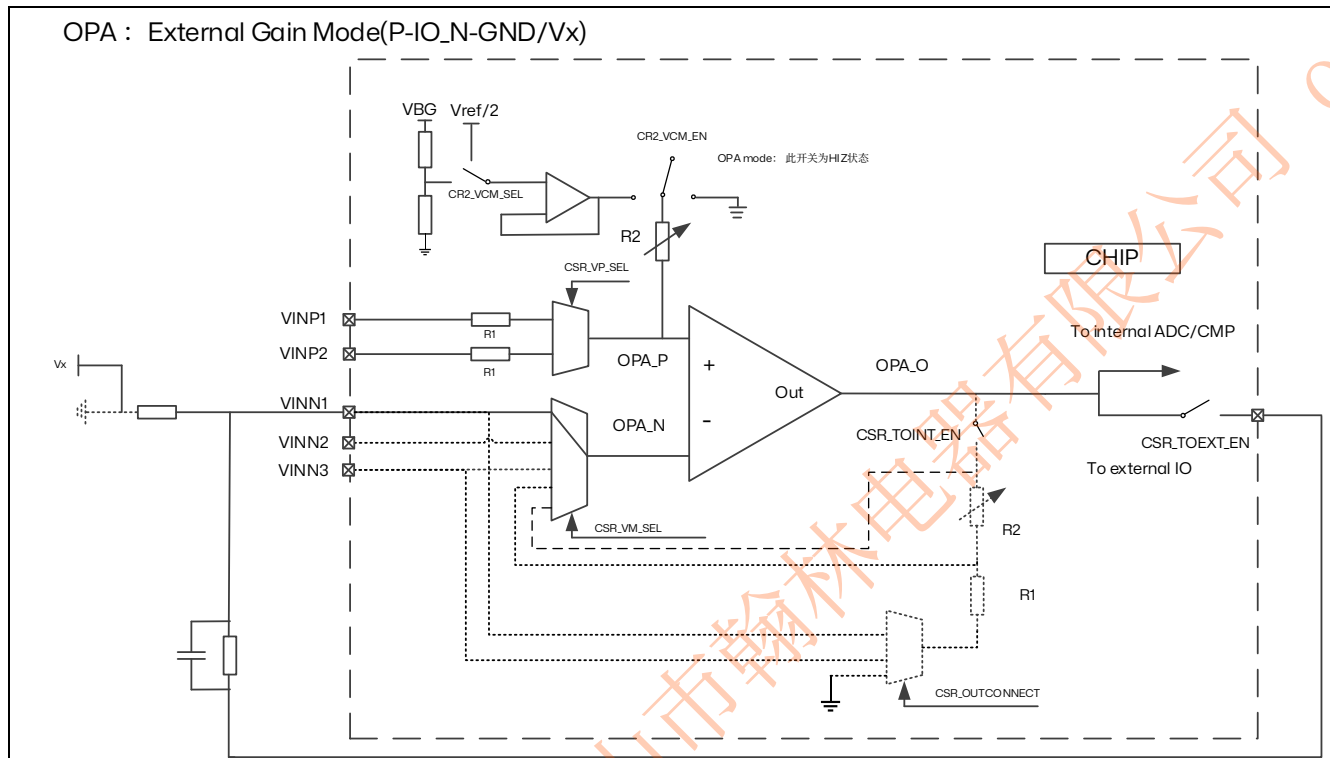


图 21.1 单机模式:外部增益设置模式

### 电压跟随器模式配置方式

给出 OPAMP 在从动模式下的使用过程。

- 配置 VM\_SEL 位为 OPAMPx\_OUT 连接到 OPAMPx\_VINM 输入 “10000”
- 配置 VP\_SEL 位为 GPIO 连接到 OPAMPx\_VINP 输入 “00”
- 一旦 OPAEN 位被设置，引脚 OPAMPx\_VINP 上的电压被缓冲到引脚 OPAMPx\_OUT。

注意：OPAMPx\_VINM 对应的引脚可用于其他用途。

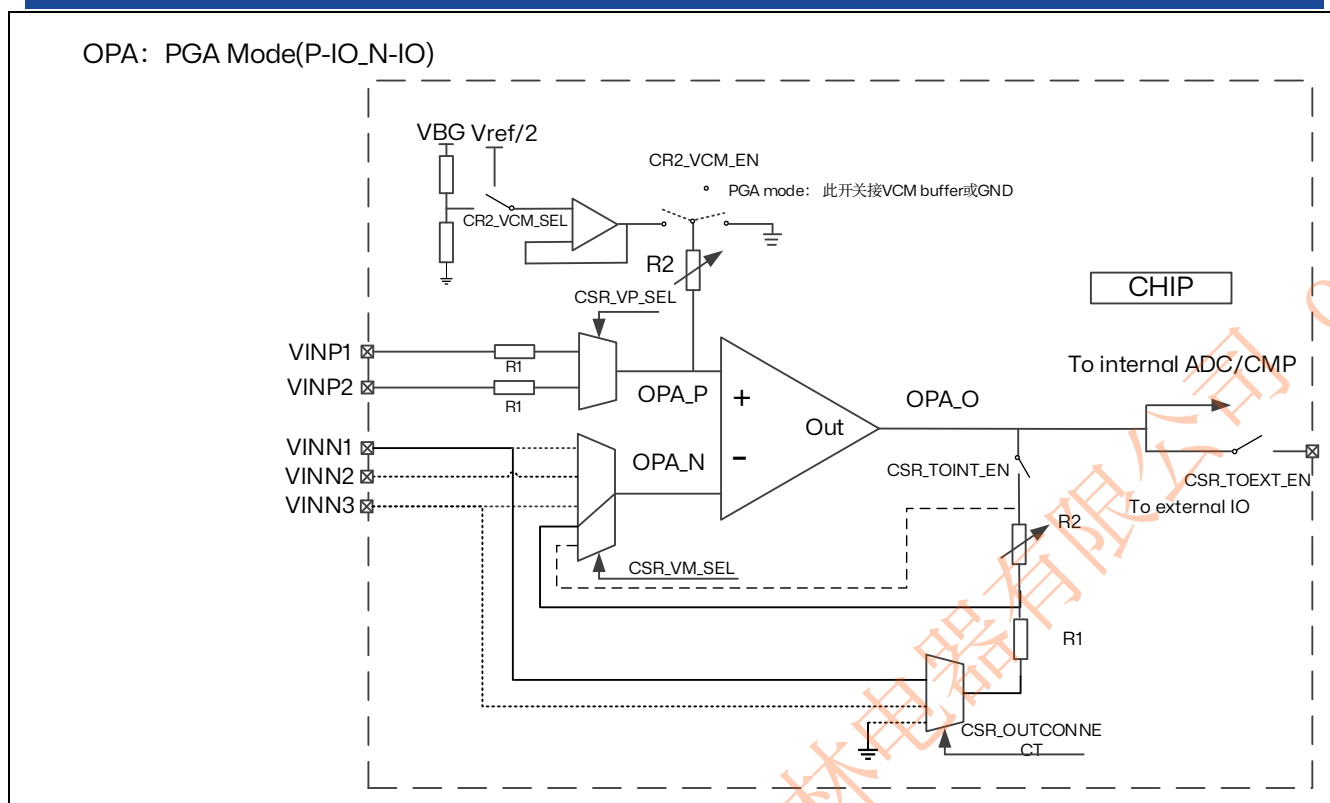
OPAMP 输出端的信号也被视为 ADC 输入。因此，假设输入信号频率为，配置为跟随模式的 OPAMP 可用于在将输入信号馈送到 ADC 输入之前对其进行阻抗自适应与运算放大器增益带宽规格兼容。



介绍 OPAMP 作为可编程增益放大器的实现过程:

- 一旦设置 OPAEN 位，引脚 OPAMPx\_VINP 上的电压被所选增益放大，并且在引脚 OPAMPx\_OUT 上可见。

注意：为避免饱和，输入电压必须低于  $V_{DDA}$  除以所选增益。



### 可编程增益放大器模式与外部滤波

介绍使用 OPAMP 通过外部滤波放大输入信号幅度的过程。

- 配置 VM\_SEL 位为反馈电阻连接到 OPAMP<sub>x</sub>\_VINM 输入 “1010”
- 配置 PGA\_GAIN 位为“带滤波的内部增益 4、8、12、16、24 或 32” VINM0”， 000 ~ 101
- 配置 VP\_SEL 位为“GPIO 连接到 OPAMP<sub>x</sub>\_VINP”。

VINM 上的任何外部连接都可以与内部 PGA 并联使用，例如电容可以在 OPAMP\_OUT 和 VINM 之间连接以进行滤波(参见数据表中 PGA 电阻网络中使用的电阻器的值)。



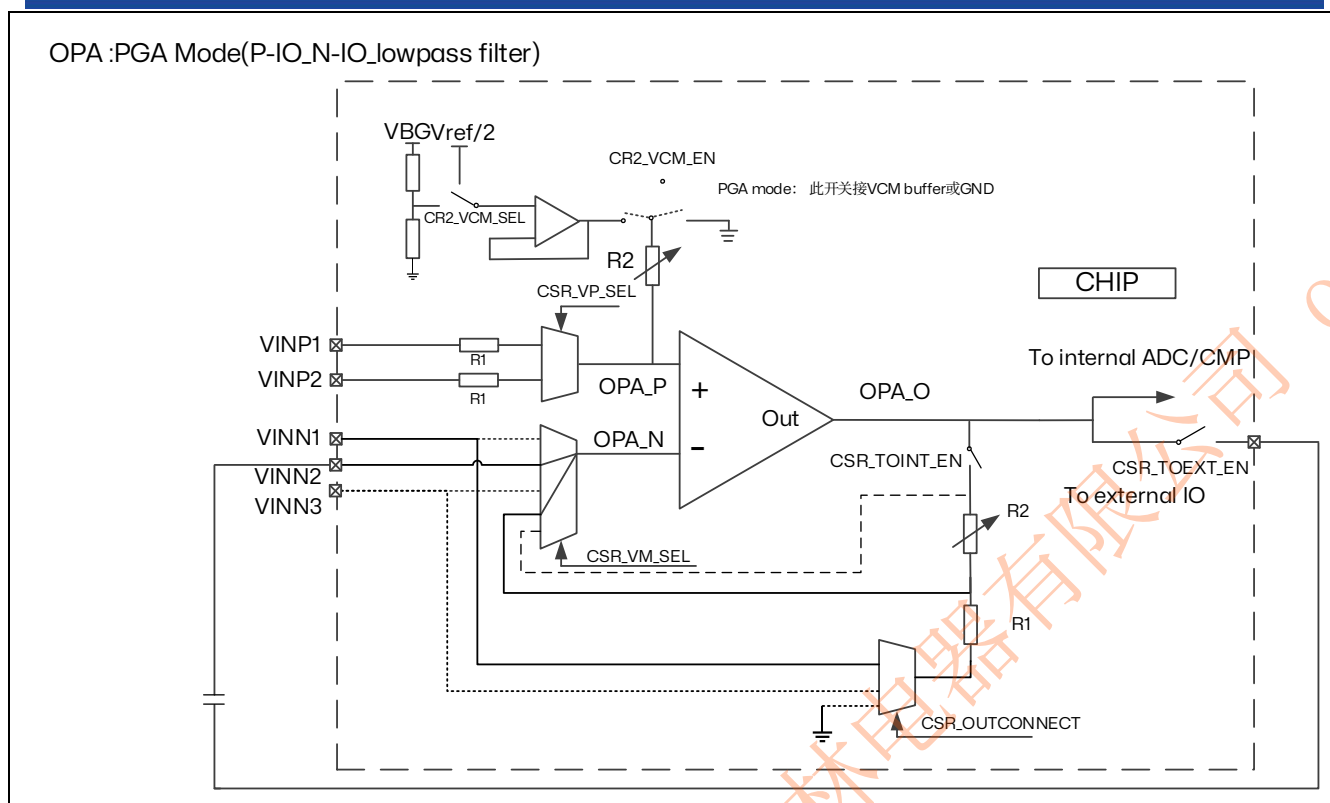


图 21.4 PGA 模式，内部增益设置(x4/x8/x12/x16/x24/x32)，反相输入用于滤波

### 21.3.5 OPAMP PGA 放大

当 OPAMP 配置为 PGA 模式时，增益可编程为 x4、x8、x12、x16、x24、x32（非反相配置）。

当 OPAMP 配置为非反相模式时，增益仅取决于内部电阻分压器。当配置为反相模式时，增益系数不仅定义了片上反馈电阻，还定义了信号源输出阻抗。如果信号源输出阻抗与 PGA 的输入反馈电阻相比不可忽略，则会产生增益误差。

### 21.3.6 校准

OPAMP 偏置值使用微调电路最小化。在启动时，微调值初始化为预设的“工厂”微调值。如果 OPAMP 在不同出厂修整条件的条件下使用，每个运放也可以由用户进行修整。

每个运算放大器可以由用户调整。特定寄存器允许在正常模式和高速模式下具有不同的微调值。

校准的目的是尽可能地消除 OPAMP 输入偏置电压。校准电路允许在稳定的电压和温度条件下将输入失调电压降低。

对于每个运算放大器和两个调整值(OPAMPx\_CSR 寄存器中的 TRIMOFFSETP 和 TRIMOFFSETN)需要调整，一个用于 N 差分对，一个用于 P 差分对。

用户可以使用 OPAMPx\_CSR 寄存器中的 USERTRIM 位从“工厂”值切换到“用户”修剪值。该位在启动时重置，因此默认情况下将“工厂”值应用于 OPAMP 选项寄存器。

偏移调整 TRIMOFFSETP 和 TRIMOFFSETN 位通常在校准操作初始化后配置，将位 CALON 设置为 1。当 CALON = 1 时，运算放大器的输入与 I/O 断开。

- 将 CALSEL 设置为 01 初始化 P 差分对的偏移校准（使用低电压参考）。
- 将 CALSEL 重置为 11 初始化 N 差分对的偏移校准（使用高压参考）。

当 CALON = 1 时，位 CALOUT 反映了 CALSEL 和 OPAHSM 选择的修剪值的影响。软件必须将

OPAMP 控制寄存器中的 TRIMOFFSETN 位从 0x00 增加到导致 OPAMP 寄存器中的 CALOUT 位从 0 变为 1 的第一个值。如果 CALOUT 位复位，则偏移量被正确校准，并且必须存储相应的微调值。在微调值改变后，CALOUT 标志需要长达 1 毫秒才能变得稳定。

注意：微调值越接近最佳微调值，稳定所需的时间越长（在任何情况下，最大稳定时间保持在 1ms 以下）。

表 21.2 工作模式及校正

| 模式                | 控制位   |        |       |        | 输出     |            |
|-------------------|-------|--------|-------|--------|--------|------------|
|                   | OPAEN | OPAHSM | CALON | CALSEL | Vout   | CALOUTflag |
| 正常模式              | 1     | 0      | 0     | X      | analog | 0          |
| 高速模式              | 1     | 1      | 0     | X      | analog | 0          |
| Offset cal N      | 1     | X      | 1     | 11     | analog | X          |
| Offset cal P diff | 1     | X      | 1     | 10     | analog | X          |

### 校准程序

1. 将 OPAMPx\_CSR 中的 OPAEN 位设置为 1，使能运算放大器。
2. 将 OPAMPx\_CSR 寄存器中的 USERTRIM 位设置为 1。
3. 选择校准模式(参见表 10-2：工作模式与校准)。步骤 3 到步骤 4 要重复四次。对于第一次迭代，选择 Normal 模式和 N 对差分。该校准模式对应于 OPAMPx\_CSR 寄存器中的 OPAHSM = 0 和 CALSEL = 11。
4. OPAMPx\_OTR 中的 TRIMOFFSETN[4:0]从 0b00000 开始递增，直到 OPAMPx\_CSR 中的 CALOUT 变为 1。

注意：在写入 TRIMOFFSETP 和 TRIMOFFSETN 位和读取 CALOUT 值之间，以获得正确的 CALOUT 值。

换流意味着正确补偿，相应的修剪代码必须保存在 TRIMOFFSETP 和 TRIMOFFSETN 位中。

重复步骤 3 到步骤 4：

- 正常模式和 P 差分对，CALSEL = 01
- 高速模式和 N 差分对，CALSEL = 11
- 高速模式和 P 差分对，CALSEL = 01

如果不使用某种模式，则不需要进行相应的校准。

所有运算放大器可同时校准。

注意：在整个校准阶段，运放输出的外部连接不能上拉或下拉大于 500  $\mu$ A 的电流。

如果 OPAMP 输出内部连接到 ADC 通道并与输出引脚断开(OPAMPx\_CSR 寄存器中的 OPAINTOEN = 1)，则偏移量微调过程与 OPAMP 输出连接到输出引脚(OPAINTOEN = 0)的情况不同。校准过程与上述类似，但 CALOUT 位变化检测不能如步骤 4 所示使用。相反，ADC 输出数据必须用作检测 OPAMP 输出变化的指标：CALOUT 从 0 到 1 的变化对应于 ADC 输出数据从接近最小 ADC 输出的值变化到接近最大 ADC 输出的值(ADC 作为连接到 OPAMP 输出的比较器)。另一种解决方案是将 OPAINTOEN = 0 进行校准，然后将 OPAINTOEN 更改为 1。在这种情况下，OPAMP 输出 GPIO 在校准期间切换，必须注意该 GPIO 上没有冲突。

## 21.3.7 OPAMP 低功耗模式

表 21.3. 低功耗模式对 OPAMP 的影响

| 模式              | 描述                              |
|-----------------|---------------------------------|
| Sleep           | 没影响                             |
| Stop 0 / Stop 1 | 没有效果，OPAMP 寄存器内容保留。             |
| Standby         | OPAMP 寄存器断电，必须在退出待机或关机模式后重新初始化。 |
| Shutdown        |                                 |

## 21.4 OPAMP 寄存器描述

基地址：0x4001 0300

### 21.4.1 OPAMPx 控制状态寄存器 (OPAMPx\_CSR) (x=1...4)

地址偏移：0x00 + 4\*(x-1)

复位值：0x0000 0000

| 31        | 30      | 29          | 28               | 27    | 26        | 25     | 24               | 23      | 22          | 21 | 20               | 19 | 18       | 17      | 16 |
|-----------|---------|-------------|------------------|-------|-----------|--------|------------------|---------|-------------|----|------------------|----|----------|---------|----|
| LOCK      | CAL OUT | TOADC_EN    | TRIMOFFSETP[4:0] |       |           |        | TRIMOFFSETN[4:0] |         |             |    | OUTCONNECT [1:0] |    |          | GAIN[2] |    |
| rw        | rw      | rw          | rw               | rw    | rw        | rw     | rw               | rw      | rw          | rw | rw               | rw | rw       | rw      | rw |
| 15        | 14      | 13          | 12               | 11    | 10        | 9      | 8                | 7       | 6           | 5  | 4                | 3  | 2        | 1       | 0  |
| GAIN[1:0] |         | CALSEL[1:0] |                  | CALON | USER TRIM | VP_SEL | TOEXT_EN         | OPA_HSM | VM_SEL[4:0] |    |                  |    | FORCE_VP | OPAEN   |    |
| rw        | rw      | rw          | rw               | rw    | rw        | rw     | rw               | rw      | rw          | rw | rw               | rw | rw       | rw      | rw |

Bit 31 **LOCK**: OPAMPx\_CSR 锁

0: OPAMPx\_CSR 可读写

1: OPAMPx\_CSR 只读

Bit 30 **CALOUT**: 运算放大器 1 校准输出

校准模式期间，此信号翻转时，微调偏移。

Bit 29 **TOADC\_EN**:

0: OPAx 输出至内部 ADC/CMP 禁止

1: OPAx 输出至内部 ADC/CMP 使能

Bits 28:24 **TRIMOFFSETP[4:0]**: 修整 PMOS 差动对 (只有在 USER\_TRIM =1 时可被修改)

Bits 23:19 **TRIMOFFSETN[4:0]**: 修整 NMOS 差动对 (只有在 USER\_TRIM =1 时可被修改)

Bits 18:17 **OUTCONNECT[1:0]**: OPAMPx 的输出连接

00: 不连接

01: OPAMPx 的输出经过内部电阻接地

10: OPAMPx 的输出经过内部电阻接到 OPAMPx\_N1

11: OPAMPx 的输出经过内部电阻接到 OPAMPx\_N3

Bits 16:14 **GAIN[2:0]**: PGAx 放大倍率选择

000: 4x

001: 8x

010: 12x

011: 16x

100: 24x

101: 32x

Bits 13:12 **CALSEL[1:0]**: OPAMPx 校准选择

00:  $0.033 \times V_{DDA}$  作为 OPAMPx 输入

01:  $0.1 \times V_{DDA}$  作为 OPAMPx 输入(作为 PMOS 校准)

10:  $0.5 \times V_{DDA}$  作为 OPAMPx 输入

11:  $0.9 \times V_{DDA}$  作为 OPAMPx 输入(作为 NMOS 校准)

Bit 11 **CALON**: OPAMPx 校准模式使能

0: 正常模式

1: 校准模式 (通过硬件打开所有开关)

Bit 10 **USERTRIM**: 用于从“出厂” OPAMPx 偏移微调值切换到“用户”OPAMPx 偏移微调值  
对于正常模式和低功耗模式, 该位均有效。

0: 使用“出厂”微调代码

1: 使用“用户”微调代码

Bit 9 **VP\_SEL**: P 端选择

0: OPAMPx\_P1

1: OPAMPx\_P2

Bit 8 **TOEXT\_EN**: OPAMPx 输出 PIN

0: OPAMPx 输出至外部 IO 引脚使能

1: OPAMPx 输出至外部 IO 引脚禁止

Bit 7 **OPAHSM**: OPAMPx 高速模式使能

0: 高速模式禁止

1: 高速模式使能

Bits 6:2 **VM\_SEL[4:0]**: M 端选择

00001: OPAMPx\_N1 引脚连接到 OPAMP VINM 输入

00010: OPAMPx\_N2 引脚连接到 OPAMP VINM 输入

00100: OPAMPx\_N3 引脚连接到 OPAMP VINM 输入

01000: 反馈电阻连接到 OPAMP 的 VINM 输入 (PGA 模式), 反相输入的选择取决于 PGA\_GAIN 设置

10000: opamp\_out 连接到 OPAMP VINM 输入(跟随模式)

注: 5bit 可任意同时选择搭配

Bit 1 **FORCE\_VP**: 模式选择

0: 正常模式: 非反相输入连接到输入。

1: 校准验证模式: 非反相输入连接至校准参考电压。

Bit 0 **OPAEN**: 使能运算放大器

0: 关闭运算放大器

1: 使能运算放大器

### 21.4.2 OPAMP1 控制状态寄存器 (OPAMP1\_CR2)

地址偏移: 0x10

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |    |    |    |    |    |               |             |            |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|---------------|-------------|------------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18            | 17          | 16         |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |               |             |            |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2             | 1           | 0          |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    | VBGINT<br>_EN | VCM<br>_SEL | VCM<br>_EN |
|          |    |    |    |    |    |    |    |    |    |    |    |    | rw            | rw          | rw         |

Bits 31:3 保留, 必须保持为复位值。

Bit 2 **VBGINT\_EN**: Bandgap Buffer 使能信号

0: 禁止 buffer

1: 使能 buffer

Banggap 电压经 buffer 输入至 OPA, 且任一 OPA 的此信号使能后, ADC\_VBGINT\_EN 也会打开

Bit 1 **VCM\_SEL**: OPA1/2 P 端输入外接偏置电压选择

0: 偏置电压选择为 VBG/2

1: 偏置电压选择为 Vref/2

注: OPA1 和 OPA2 共用一个偏置电压

Bit 0 **VCM\_EN**: OPA1 内部 VCM 选择线

当 OPAMP1\_CSR\_VM\_SEL=01000 该引脚才可以使用。

0: VCM 接地 (AGND)

1: VCM 接 VCMBUF

### 21.4.3 OPAMP2 控制状态寄存器 (OPAMP2\_CR2)

地址偏移: 0x14

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |    |    |    |    |    |               |     |            |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|---------------|-----|------------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18            | 17  | 16         |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |               |     |            |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2             | 1   | 0          |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    | VBGINT<br>_EN | Res | VCM<br>_EN |
|          |    |    |    |    |    |    |    |    |    |    |    |    | rw            |     | rw         |

Bits 31:3 保留, 必须保持为复位值。

Bit 2 **VBGINT\_EN**: Bandgap Buffer 使能信号

0: 禁止 buffer

1: 使能 buffer

Banggap 电压经 buffer 输入至 OPA, 且任一 OPA 的此信号使能后, ADC\_VBGINT\_EN 也会打开

Bit 1 保留, 必须保持为复位值。

Bit 0 **VCM\_EN**: OPA2 内部 VCM 选择线

当 OPAMP2\_CSR\_VM\_SEL=01000 该引脚才可以使用。

0: VCM 接地 (AGND)

1: VCM 接 VCMBUF

#### 21.4.4 OPAMP3 控制状态寄存器 (OPAMP3\_CR2)

地址偏移: 0x18

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |    |    |    |    |    |               |             |            |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|---------------|-------------|------------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18            | 17          | 16         |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |               |             |            |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2             | 1           | 0          |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    | VBGINT<br>_EN | VCM<br>_SEL | VCM<br>_EN |
|          |    |    |    |    |    |    |    |    |    |    |    |    | rw            | rw          | rw         |

Bits 31:3 保留, 必须保持为复位值。

Bit 2 **VBGINT\_EN**: Bandgap Buffer 使能信号

0: 禁止 buffer

1: 使能 buffer

Banggap 电压经 buffer 输入至 OPA, 且任一 OPA 的此信号使能后, ADC\_VBGINT\_EN 也会打开

Bit 1 **VCM\_SEL**: OPA3/4 P 端输入外接偏置电压选择

0: 偏置电压选择为 VBG/2

1: 偏置电压选择为 Vref/2

注: OPA3 和 OPA4 共用一个偏置电压

Bit 0 **VCM\_EN**: OPA3 内部 VCM 选择线

当 OPAMP3\_CSR\_VM\_SEL=01000 该引脚才可以使用。

0: VCM 接地 (AGND)

1: VCM 接 VCMBUF

#### 21.4.5 OPAMP4 控制状态寄存器 (OPAMP4\_CR2)

地址偏移: 0x1C

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |    |    |    |    |    |               |     |            |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|---------------|-----|------------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18            | 17  | 16         |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |               |     |            |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2             | 1   | 0          |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    | VBGINT<br>_EN | Res | VCM<br>_EN |
|          |    |    |    |    |    |    |    |    |    |    |    |    | rw            |     | rw         |

Bits 31:3 保留, 必须保持为复位值。

Bit 2 **VBGINT\_EN**: Bandgap Buffer 使能信号

0: 禁止 buffer

1: 使能 buffer

Banggap 电压经 buffer 输入至 OPA, 且任一 OPA 的此信号使能后, ADC\_VBGINT\_EN 也会打开

Bit 1 保留, 必须保持为复位值。

Bit 0 **VCM\_EN**: OPA4 内部 VCM 选择线

当 OPAMP4\_CSR\_VM\_SEL=01000 该引脚才可以使用。

0: VCM 接地 (AGND)

1: VCM 接 VCMBUF



## 22 高分辨率定时器 (HRTIM)

### 22.1 HRTIM 简介

高分辨率定时器可生成多达 12 路高度精确定时数字信号，主要用于驱动开关模式电源或照明系统等电源转换系统，但也可用于，一般对时间分辨率有极高要求的应用。

该定时器采用模块化架构，可生成独立波形或耦合波形。波形由独立式定时信号（使用计数器和比较单元）以及多种外部事件（如模拟或数字反馈以及同步信号）确定，因此可生成大量不同的控制信号（PWM、相移、恒定 Ton...），从而满足大部分转换拓扑的需求。

为实现控制和监测用途，该定时器还具有定时测量功能，并连接到内置的 ADC 和 DAC 转换器。此外，该定时器还具有轻载管理模式，能够处理各种故障机制，从而实现安全关断。

### 22.2 HRTIM 主要特征

- 高分辨率时序单元：
  - 163 ps 分辨率，可对电压和温度变化进行补偿
  - 所有输出均支持全分辨率，可在触发单脉冲模式下调整占空比、频率和脉宽
  - 6 个 16 位定时单元（每个定时单元包含 1 个独立计数器和 4 个比较单元）
  - 12 路输出可通过任何定时单元控制，每条通道多达 32 个置位/复位源
  - 模块化结构可满足多种配有 1 或 2 个开关的独立转换器的需求，也可满足少数大型多开关拓扑的需求
- 多达 10 个外部事件，可用于任何定时单元
  - 可编程极性和边沿有效性
  - 5 个事件用于快速异步模式
  - 5 个事件用于可编程数字滤波器
  - 利用消隐和窗口模式实现伪事件过滤
- 多条通道可连接到内置模拟外设
  - 4 个用于 ADC 转换器的触发信号
  - 3 个用于 DAC 转换器的触发信号
  - 3 个用于模拟信号调理的比较器
- 丰富的保护机制
  - 6 路故障输入可组合使用并关联到任何定时单元
  - 可编程极性和边沿有效性，数字滤波器
  - 对谐振变换器配有专门的延时保护
- 多个 HRTIM 实例可与外部同步输入/输出同步
- 多功能输出级
  - 高分辨率死区时间插入（低至 651 ps）
  - 可编程输出极性
  - 斩波模式
- 突发模式控制器，可同时处理多个转换器上的轻载操作
- 8 个中断向量，每个向量最多具有 14 个源
- 7 个 DMA 请求，最多具有 14 个源，可通过突发模式实现多寄存器更新

## 22.3 HRTIM 功能描述

### 22.3.1 综合描述

HRTIM 可分为以下几个子实体：

- 主定时器
- 定时单元（定时器 A 到定时器 F）
- 输出级
- 突发模式控制器
- 外部事件和故障信号调节逻辑，由所有定时器共享
- 系统接口

主定时器基于 16 位递增计数器。它可通过 4 个比较单元置位/复位 12 路输出中的任何一路，并向 6 个定时单元提供同步信号。其主要用途是使定时单元受唯一的时钟源控制。交错降压转换器是一个典型的应用示例，主定时器在其中管理着多个单元之间的相移。

定时器单元既可以独立工作，也可以与其他定时器（包括主定时器）配合工作。每个定时器都可控制两路输出。输出置位/复位事件可以由定时单元比较寄存器触发，或者由来自主定时器、其他定时器的外部事件触发。

输出级有多种用途：

- 在互补 PWM 模式下配置 2 路输出时添加死区
- 将载波频率添加到调制信号上
- 通过将输出异步置为预定义的安全电平来管理故障事件

在轻载运行的情况下，突发模式控制器可控制一个或多个定时器。突发长度和周期、以及输出的空闲状态可通过编程设定。

外部事件和故障信号调理逻辑包括：

- 输入选择 MUX（例如，为给定外部事件通道选择数字输入或片上时钟源）
- 极性和边沿有效性编程
- 数字滤波（对 12 条通道中的 5 条通道进行滤波）

系统接口允许 HRTIM 与 MCU 的其余部分进行交互：

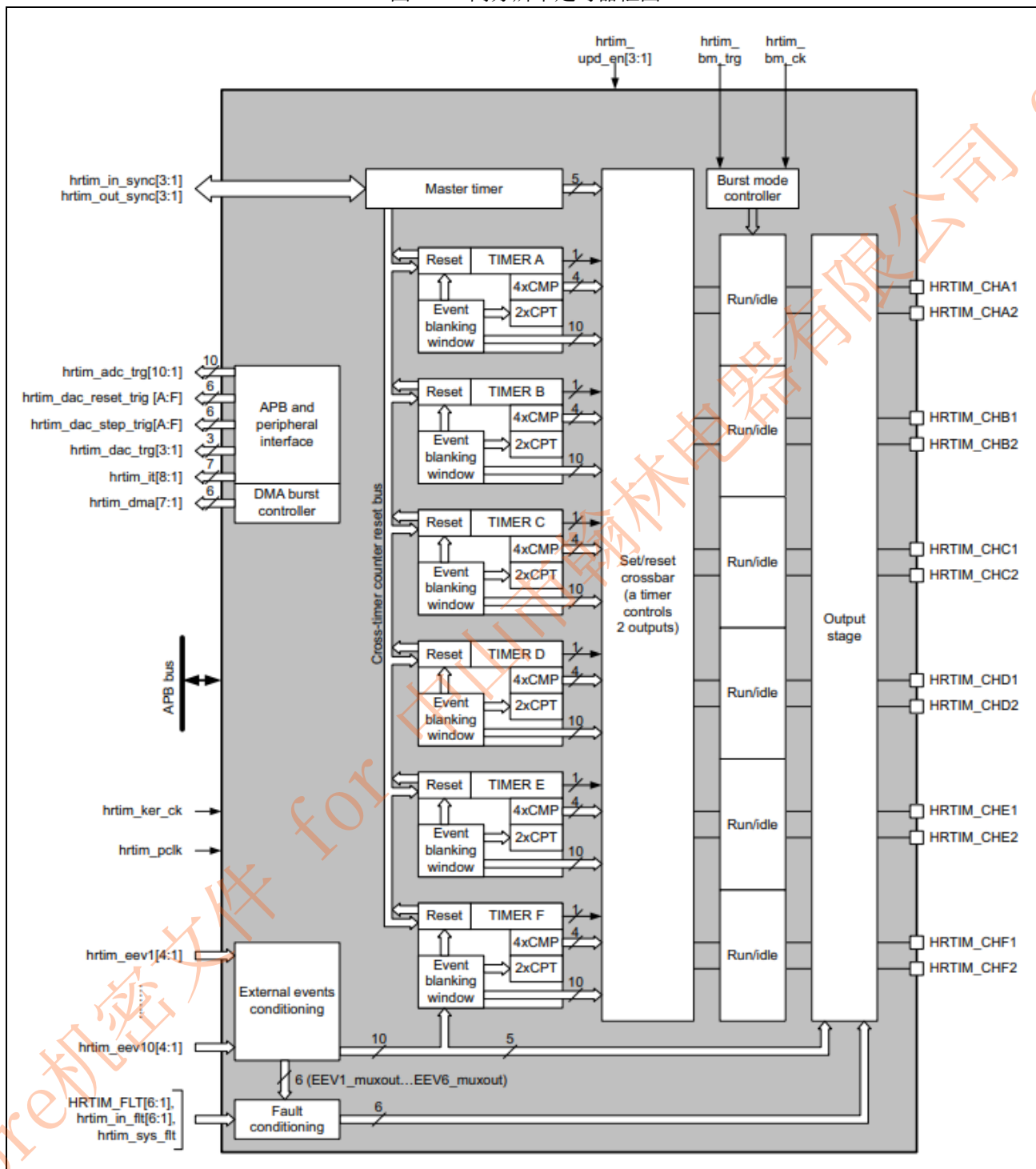
- 向 CPU 发出中断请求
- 通过 DMA 控制器自动访问存储器，包括 HRTIM 特有的突发模式
- 触发 ADC 和 DAC 转换器

HRTIM 寄存器被分为 8 个组：

- 主定时器寄存器
- 定时器 A 到定时器 F 寄存器
- 通用寄存器，用于所有定时器单元共用的功能

注：按照书写惯例，文本和寄存器中提到的6个定时单元用“x”字母表示，其中x可以是A到F中的任何值。定时器的框图如下图所示。

图 22.1 高分辨率定时器框图



### 22.3.2 HRTIM 引脚和内部信号

本节中的表格汇总了 HRTIM 的输入和输出，包括片上和片外的信号。

表 22.1 HRTIM 输入/输出汇总

| 信号名称                                                                                                                                                                            | 信号类型 | 描述                                                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-----------------------------------------------------------------------------------------------------------------------------------|
| HRTIM_CHA1,<br>HRTIM_CHA2,<br>HRTIM_CHB1,<br>HRTIM_CHB2,<br>HRTIM_CHC1,<br>HRTIM_CHC2,<br>HRTIM_CHD1,<br>HRTIM_CHD2,<br>HRTIM_CHE1,<br>HRTIM_CHE2,<br>HRTIM_CHF1,<br>HRTIM_CHF2 | 输出   | 主要的 HRTIM 定时器输出。<br>这些输出成对工作时（HRTIM_CHx1 & HRTIM_CHx2），可以插入死区或独立工作。                                                               |
| hrtim_in_flt1[4:1]<br>hrtim_in_flt2[4:1]<br>hrtim_in_flt3[4:1]<br>hrtim_in_flt4[4:1]<br>hrtim_in_flt5[4:1]<br>hrtim_in_flt6[4:1]                                                | 数字输入 | 故障输入：置为有效时立即禁止 HRTIM 输出（12 路片上输入和 6 路片外 HRTIM_FLTx 输入）。                                                                           |
| hrtim_sys_flt                                                                                                                                                                   | 数字输入 | 涵盖 MCU 内部故障事件（时钟安全系统、SRAM 奇偶校验错误、带 FPU 的 Cortex®-M4 LOCKUP（HardFault）、PVD 输出）的系统故障。                                               |
| hrtim_in_sync[3:1]                                                                                                                                                              | 数字输入 | 将整个 HRTIM 与其他内部或外部定时器资源进行同步的同步输入：<br>hrtim_in_sync1：保留<br>hrtim_in_sync2：时钟源为 TIM1_TRGO 输出<br>hrtim_in_sync3：时钟源为 HRTIM_SCIN 输入引脚 |
| hrtim_out_sync[2:1]                                                                                                                                                             | 数字输出 | 此输出用于级联或同步多个片上或片外 HRTIM 实例：<br>hrtim_out_sync1：保留<br>hrtim_out_sync2：目标是片外 HRTIM 或外设（通过 HRTIM_SCOUT 输出引脚）                         |
| hrtim_eev1[4:1]<br>hrtim_eev2[4:1]<br>hrtim_eev3[4:1]<br>hrtim_eev4[4:1]<br>hrtim_eev5[4:1]<br>hrtim_eev6[4:1]                                                                  | 数字输入 | 外部事件：10 个事件均可在 4 个时钟源中选择，可选择片上时钟源（来自其他内置外设：比较器、ADC 模拟看门狗、TIMx 定时器、触发输出）或片外时钟源（HRTIM_EEVx 输入引脚）。                                   |

| 信号名称                                                | 信号类型 | 描述                                                                         |
|-----------------------------------------------------|------|----------------------------------------------------------------------------|
| hrtim_eev7[4:1]                                     |      |                                                                            |
| hrtim_eev8[4:1]                                     |      |                                                                            |
| hrtim_eev9[4:1]                                     |      |                                                                            |
| hrtim_eev10[4:1]                                    |      |                                                                            |
| hrtim_upd_en[3:1]                                   | 数字输入 | HRTIM 寄存器更新使能输入 hrtim_upd_en[3:1] (片上互连) 会触发从影子寄存器到活动寄存器的传输操作              |
| hrtim_bm_trg                                        | 数字输入 | 在这个输入上的脉冲触发一个突发模式输入。这个输入连接到 TIM7_TRGO 输出                                   |
| hrtim_bm_ck[4:1]                                    | 数字输入 | 突发模式时钟 (片上互连)                                                              |
| hrtim_adc_trg[10:1]                                 | 数字输出 | ADC 转换开始触发信号。hrtim_adc_trg1..hrtim_adc_trg10 输出在这里称为 ADC 触发 1 到 ADC 触发 10。 |
| hrtim_dac_trg[3:1]                                  | 数字输出 | DAC 转换更新触发信号。                                                              |
| hrtim_dac_reset_trg[6:1]<br>hrtim_dac_step_trg[6:1] | 数字输出 | 双通道 DAC 触发。                                                                |
| hrtim_it[8:1]                                       | 数字输出 | 中断请求。                                                                      |
| hrtim_dma[7:1]                                      | 数字输出 | DMA 请求。                                                                    |
| hrtim_pclk                                          | -    | APB 时钟。                                                                    |
| hrtim_ker_ck                                        | -    | HRTIM 内核时钟 (APB2 时钟)。                                                      |

表 22.2 外部事件映射及相关功能

| 外部事件通道            | 快速模式 | 数字滤波 | 平衡故障定时器 A,B,C | 平衡故障定时器 D,E,F | 源 (EExSRC[1:0]) |          |            |           | 比较器和每个封装可用的输入源 |                 |
|-------------------|------|------|---------------|---------------|-----------------|----------|------------|-----------|----------------|-----------------|
|                   |      |      |               |               | 源 1 (00)        | 源 2 (01) | 源 3 (10)   | 源 4 (11)  | 48-pin         | 64-pin 和 80-pin |
| hrtim_eev 1[4:1]  | 是    | -    | -             | -             | PC12            | COMP2    | TIM1_TRGO  | ADC1_AWD1 | 比较器            | 比较器 & 输入        |
| hrtim_eev 2[4:1]  | 是    | -    | -             | -             | PC11            | COMP4    | TIM2_TRGO  | -         | 比较器            | 比较器 & 输入        |
| hrtim_eev 3[4:1]  | 是    | -    | -             | -             | PB7             | -        | TIM3_TRGO  | -         | 比较器 & 输入       | 比较器 & 输入        |
| hrtim_eev 4[4:1]  | 是    | -    | -             | -             | PB6             | COMP1    | -          | ADC2_AWD1 | 比较器 & 输入       | 比较器 & 输入        |
| hrtim_eev 5[4:1]  | 是    | -    | -             | -             | PB9             | COMP3    | -          | -         | 比较器 & 输入       | 比较器 & 输入        |
| hrtim_eev 6[4:1]  | -    | 是    | 是             | -             | PB5             | COMP2    | COMP1      | -         | 比较器 & 输入       | 比较器 & 输入        |
| hrtim_eev 7[4:1]  | -    | 是    | 是             | -             | PB4             | COMP4    | TIM7_TRGO  | ADC3_AWD1 | 比较器 & 输入       | 比较器 & 输入        |
| hrtim_eev 8[4:1]  | -    | 是    | -             | 是             | PB8             | -        | COMP3      | -         | 比较器 & 输入       | 比较器 & 输入        |
| hrtim_eev 9[4:1]  | -    | 是    | -             | 是             | PB3             | -        | TIM15_TRGO | COMP4     | 比较器 & 输入       | 比较器 & 输入        |
| hrtim_eev 10[4:1] | -    | 是    | -             | -             | PC5/<br>PC6     | -        | TIM6_TRGO  | -         | -              | 比较器 & 输入        |

表 22.3 更新使能输入和源

| hrtim_upd_en[3:1] | 更新源       |
|-------------------|-----------|
| hrtim_upd_en1     | TIM16_OC  |
| hrtim_upd_en2     | TIM17_OC  |
| hrtim_upd_en3     | TIM6_TRGO |

表 22.4 来自通用定时器的突发模式时钟源

| hrtim_bm_ck[4:1] | BMCLK[3:0] | 时钟源       |
|------------------|------------|-----------|
| hrtim_bm_ck1     | 0110       | TIM16_OC  |
| hrtim_bm_ck2     | 0111       | TIM17_OC  |
| hrtim_bm_ck3     | 1000       | TIM7_TRGO |
| hrtim_bm_ck4     | 1001       | 保留        |

表 22.5 默认输入

| 默认通道               | 外部输入<br>FLTxsSRC[1:0]<br>=00 | 片上源<br>FLTxsSRC[1:0]<br>=01 | 外部输入<br>FLTxsSRC[1:0]<br>=10 | 片上源<br>FLTxsSRC[1:0]<br>=11 |
|--------------------|------------------------------|-----------------------------|------------------------------|-----------------------------|
| hrtim_in_flt1[4:1] | PA12                         | COMP2                       | EEV1                         | N/A                         |
| hrtim_in_flt2[4:1] | PA15                         | COMP4                       | EEV2                         | N/A                         |
| hrtim_in_flt3[4:1] | PE6                          | -                           | EEV3                         | N/A                         |
| hrtim_in_flt4[4:1] | PB11                         | COMP1                       | EEV4                         | N/A                         |
| hrtim_in_flt5[4:1] | PA11/PC7                     | COMP3                       | EEV5                         | N/A                         |
| hrtim_in_flt6[4:1] | PC10                         | -                           | EEV6                         | N/A                         |

表 22.6 HRTIM DAC 触发器连接

| HRTIM DAC<br>触发器                            | DAC1_CH1<br>DAC1_CH2 | DAC2_CH1 | DAC3_CH1<br>DAC3_CH2 |
|---------------------------------------------|----------------------|----------|----------------------|
| hrtim_dac_trg1                              | 是                    | -        | -                    |
| hrtim_dac_trig2                             | -                    | 是        | -                    |
| hrtim_dac_trig3                             | -                    | -        | 是                    |
| hrtim_dac_reset_trg1<br>hrtim_dac_step_trg1 | 是                    | 是        | 是                    |
| hrtim_dac_reset_trg2<br>hrtim_dac_step_trg2 | 是                    | 是        | 是                    |
| hrtim_dac_reset_trg3<br>hrtim_dac_step_trg3 | 是                    | 是        | 是                    |
| hrtim_dac_reset_trg4<br>hrtim_dac_step_trg4 | 是                    | 是        | 是                    |
| hrtim_dac_reset_trg5<br>hrtim_dac_step_trg5 | 是                    | 是        | 是                    |
| hrtim_dac_reset_trg6<br>hrtim_dac_step_trg6 | 是                    | 是        | 是                    |

### 22.3.3 时钟

HRTIM 必须由  $t_{HRTIM}$  APB2 时钟提供，以实现全分辨率。 $t_{HRTIM}$  时钟周期通过使用边缘定位逻辑均匀分为最多 32 个中间步骤。HRTIM 中存在的所有时钟都源自这个参考时钟。

#### 术语定义

- $f_{HRTIM}$ : 主 HRTIM 时钟 (hrtim\_ker\_ck)。所有后续时钟均由该时钟源生成，并与该时钟源同步。
- $f_{HRCK}$ : 高分辨率等效时钟。考虑到  $f_{HRTIM}$  时钟周期被 32 分割，它相当于  $192 \times 32 = 6.14$  GHz 的频率。



- $f_{DTG}$ : 死区发生器时钟。为了方便起见, 本文档中仅使用  $t_{DTG}$  周期 ( $t_{DTG} = 1/f_{DTG}$ )。
- $f_{CHPFRQ}$ : 斩波级时钟源。
- $f_{1STPW}$ : 定义斩波模式下初始脉冲长度的时钟源。为方便起见, 本文档中仅使用  $t_{1STPW}$  周期 ( $t_{1STPW} = 1/f_{1STPW}$ )。
- $f_{BRST}$ : 突发模式控制器计数器时钟。
- $f_{SAMPLING}$ : 对故障或外部事件输入进行采样时所需的时钟。
- $f_{FLTS}$ : 由用作  $f_{SAMPLING}$  时钟源的  $f_{HRTIM}$  生成的时钟, 用于对故障事件进行过滤。
- $f_{EEVS}$ : 由用作  $f_{SAMPLING}$  时钟源的  $f_{HRTIM}$  生成的时钟, 用于对外部事件进行过滤。

## 定时器时钟和预分频器

HRTIM 中的每个定时器都有独立的时钟预分频器, 以供用户调整定时器分辨率。(参见下表)。

表 22.7 定时器分辨率和最小 PWM 频率 ( $f_{HRTIM} = 192\text{MHz}$  时)

| CKPSC[2:0] | 预分频比 | $f_{HRCK}$ 等效频率                                | 分辨率      | 最小 PWM 频率 |
|------------|------|------------------------------------------------|----------|-----------|
| 000        | 1    | $192 \times 32 \text{ MHz} = 6.14 \text{ GHz}$ | 163 ps   | 93.74 kHz |
| 001        | 2    | $192 \times 16 \text{ MHz} = 3.07 \text{ GHz}$ | 326 ps   | 46.87 kHz |
| 010        | 4    | $192 \times 8 \text{ MHz} = 1.54 \text{ GHz}$  | 651 ps   | 23.49 kHz |
| 011        | 8    | $192 \times 4 \text{ MHz} = 768 \text{ MHz}$   | 1.30 ns  | 11.75 kHz |
| 100        | 16   | $192 \times 2 \text{ MHz} = 384 \text{ MHz}$   | 2.60 ns  | 5.86 kHz  |
| 101        | 32   | 192 MHz                                        | 5.21 ns  | 2.93 kHz  |
| 110        | 64   | $192 / 2 \text{ MHz} = 96 \text{ MHz}$         | 10.41 ns | 1.47 kHz  |
| 111        | 128  | $192 / 4 \text{ MHz} = 48 \text{ MHz}$         | 20.83 ns | 0.73 kHz  |

高分辨率可用于边沿定位、PWM 周期调整以及外部触发的脉冲持续时间。

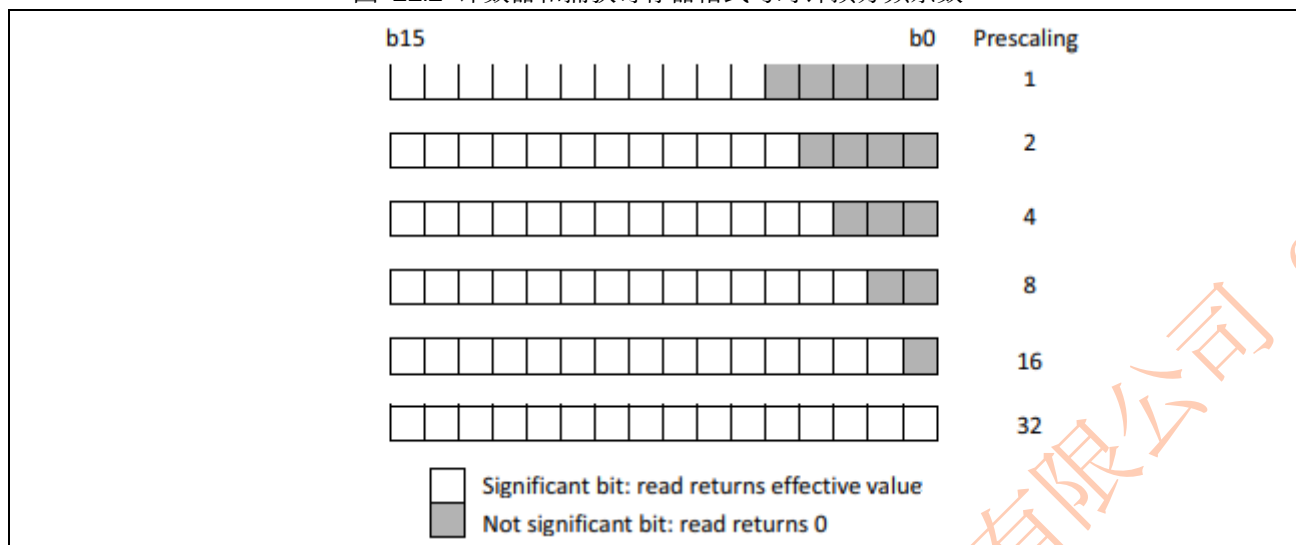
然而, 以下功能并不支持高分辨率:

- 定时器计数器的读写访问
- 捕获单元

对于时钟预分频比小于 32 ( $CKPSC[2:0] < 5$ ) 的情况, 计数器和捕获寄存器的最低有效位 (LSB) 是不重要的。这些最低有效位无法写入 (仅针对计数器寄存器), 并且在读取时返回 0。

例如, 如果  $CKPSC[2:0] = 2$  (预分频比为 4), 向计数器寄存器写入 0xFFFF 会得到一个有效的值 0xFFF8。相反, 任何介于 0xFFFF 和 0xFFF8 之间的计数器值在读取时都会显示为 0xFFF8。

图 22.2 计数器和捕获寄存器格式与时钟预分频系数



### 初始化

启动时，务必先初始化预分频器位域，然后再写入比较和周期寄存器。定时器使能后（HRTIM\_MCR 寄存器中的 MCEN 或 TxCEN 位已置 1），不能修改预分频器。

如果有多个定时器已使能，预分频器会与先启动的定时器的预分频器同步。

**警告：**仅当计数器与输出行为与其他定时器的信息和信号无关时，主定时器和 TIMA.E 定时器才能使用不同的预分频比。如果以下某个事件从一个定时单元（或主定时器）传输到另一个定时单元，则务必要在这些定时器中配置相同的预分频比：输出置位/复位事件、计数器复位事件、更新事件、外部事件过滤或捕获触发。预分频系数不相等会导致结果不可预测。

### 死区发生器时钟

死区时间的预分频器由  $(f_{HRTIM} \times 8) / 2^{(DTPRSC[2:0])}$  提供，通过 HRTIM\_DTxR 寄存器中 DTPRSC[2:0] 位编程。

当  $f_{HRTIM} = 192\text{MHz}$  时， $t_{DTG}$  的范围从 651ps 到 83.3ns。

### 斩波级时钟

斩波级时钟源  $f_{CHPFRQ}$  由  $f_{HRTIM}$  生成，其分频系数范围为 16 到 256，因此当  $f_{HRTIM} = 192\text{MHz}$  时， $749.997\text{kHz} \leq f_{CHPFRQ} \leq 12\text{MHz}$ 。

$t_{1STPW}$  是在斩波模式下初始脉冲的长度，它使用 HRTIM\_CHPxR 寄存器中的 STRPW[3:0] 位进行编程，具体如下：

$$t_{1STPW} = (STRPW[3:0] + 1) \times 16 \times t_{HRTIM}$$

它使用  $f_{HRTIM} / 16$  作为时钟源（对于  $f_{HRTIM} = 192\text{MHz}$ ，为 12MHz）。

### 突发模式预分频器

突发模式控制器计数器时钟  $f_{BRST}$  可由多个时钟源提供，其中一个时钟源由  $f_{HRTIM}$  生成。  
在这种情况下， $f_{BRST}$  的范围从  $f_{HRTIM}$  到  $f_{HRTIM} / 32768$ （对于  $f_{HRTIM} = 192\text{MHz}$ ，为  $5.859\text{kHz}$ ）。

### 故障输入采样时钟

故障输入噪声抑制滤波器具有一个由  $f_{SAMPLING}$  定义的时间常数，它可以是  $f_{HRTIM}$  或  $f_{FLTS}$ 。  
 $f_{FLTS}$  是从  $f_{HRTIM}$  派生而来，对于  $f_{HRTIM} = 192\text{MHz}$ ，其范围从  $192\text{MHz}$  到  $24\text{MHz}$ 。

### 外部事件输入采样时钟

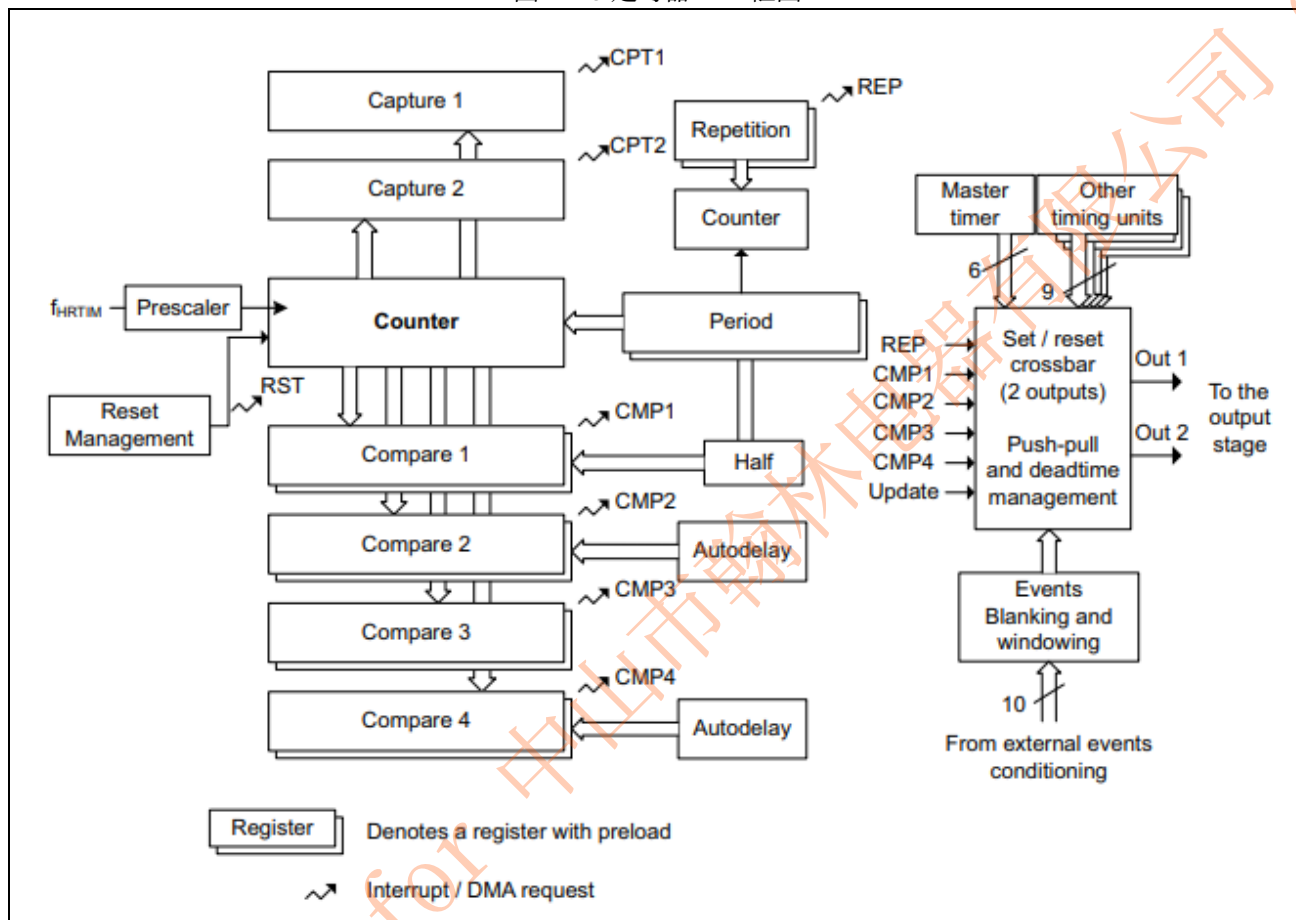
外部事件输入噪声抑制滤波器具有一个由  $f_{SAMPLING}$  定义的时间常数，它可以是  $f_{HRTIM}$  或  $f_{EEVS}$ 。  
 $f_{EEVS}$  是从  $f_{HRTIM}$  派生而来，对于  $f_{HRTIM} = 192\text{MHz}$ ，其范围同样从  $192\text{MHz}$  到  $24\text{MHz}$ 。

### 22.3.4 定时器 A.F 定时单元

HRTIM 嵌入了 6 个完全相同的定时单元（这些定时单元由采用自动重载机制的 16 位递增计数器组成，用于定义计数周期）、4 个比较单元和 2 个捕获单元，如下图所示。

每个单元都包含对 2 路输出的所有控制功能，因此可作为独立定时器工作。

图 22.3 定时器 A.F 框图



周期和比较值必须与高分辨率实现相关的下限和上限之间，这些限制在下表中列出：

然而，以下功能并不支持高分辨率：

- 最小值必须大于或等于 3 个 f<sub>HRTIM</sub> 时钟周期。值 0x0000 仅可以写入 CMP1 和 CMP3 寄存器中，以跳过 PWM 脉冲。有关详细信息，请参阅：“空占空比异常情况”一节。
- 最大值必须小于或等于 0xFFFF - 1 个 f<sub>HRTIM</sub> 时钟周期

表 22.8 周期寄存器与比较寄存器的最小和最大值

| CKPSC[2:0] 值 | 最小值 <sup>(1)</sup> | 最大值     |
|--------------|--------------------|---------|
| 0            | 0x0060             | 0xFFDF  |
| 1            | 0x0030             | 0xFFEF  |
| 2            | 0x0018             | 0xFFFF7 |
| 3            | 0x000C             | 0xFFFFB |
| 4            | 0x0006             | 0xFFFFD |
| ≥ 5          | 0x0003             | 0xFFFFD |

1. 值 0x0000 仅可以写入 CMP1 和 CMP3 寄存器中，以跳过 PWM 脉冲。有关详细信息，请参阅“空占空比异常情况”一节。

注：比较值大于周期寄存器值不会产生比较匹配事件。

### 计数器工作模式

定时器 A..F 可在连续（自由运行）模式下工作，也可以单发方式工作，此时会由复位事件触发开始计数，工作模式通过 HRTIM\_TIMxCR 控制寄存器中的 CONT 位设置。附加的 RETRIG 位可用于选择单发操作是可再触发的或不可再触发的。工作模式的详细情况在下表以及下图中进行了总结。

表 22.9 定时器工作模式

| CONT | RETRIG | 工作模式        | 启动/停止条件<br>时钟和事件生成                                                                                                                     |
|------|--------|-------------|----------------------------------------------------------------------------------------------------------------------------------------|
| 0    | 0      | 单发<br>不可再触发 | 将 TxEN 位置 1 会使能定时器，但不会启动计数器。<br>第一个复位事件会触发计数器开始计数，但计数器在达到 PER 值之前会忽略任何后续复位事件。<br>随后会生成 PER 事件，计数器停止计数。<br>发生复位事件后，计数器会重新从 0x0000 开始计数。 |
| 0    | 1      | 单发<br>可再触发  | 将 TxEN 位置 1 会使能定时器，但不会启动计数器。<br>如果计数器停止计数，则复位事件会使计数器开始计数，否则会将计数器清零。计数器达到 PER 值后，会生成 PER 事件，计数器会停止计数。发生复位事件后，计数器会重新从 0x0000 开始计数。       |
| 1    | x      | 连续模式        | 将 TxEN 位置 1 会使能定时器，同时会启动计数器。<br>计数器达到 PER 值后，会翻转到 0x0000 并重新开始计数。可以随时复位计数器。                                                            |

可以随时清零 TxEN 位，以禁止定时器并停止计数。

图 22.4 连续定时器工作模式

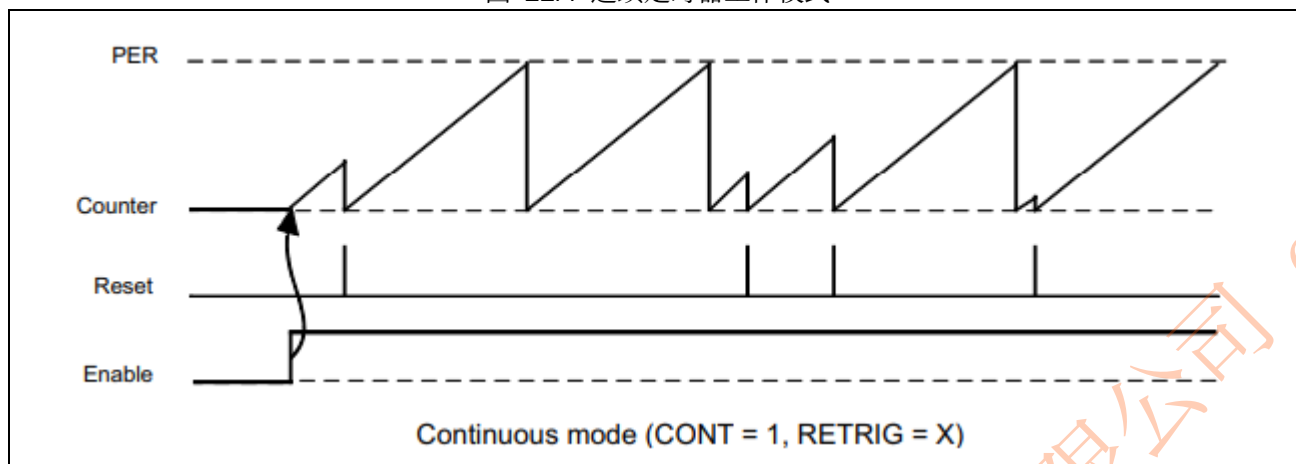
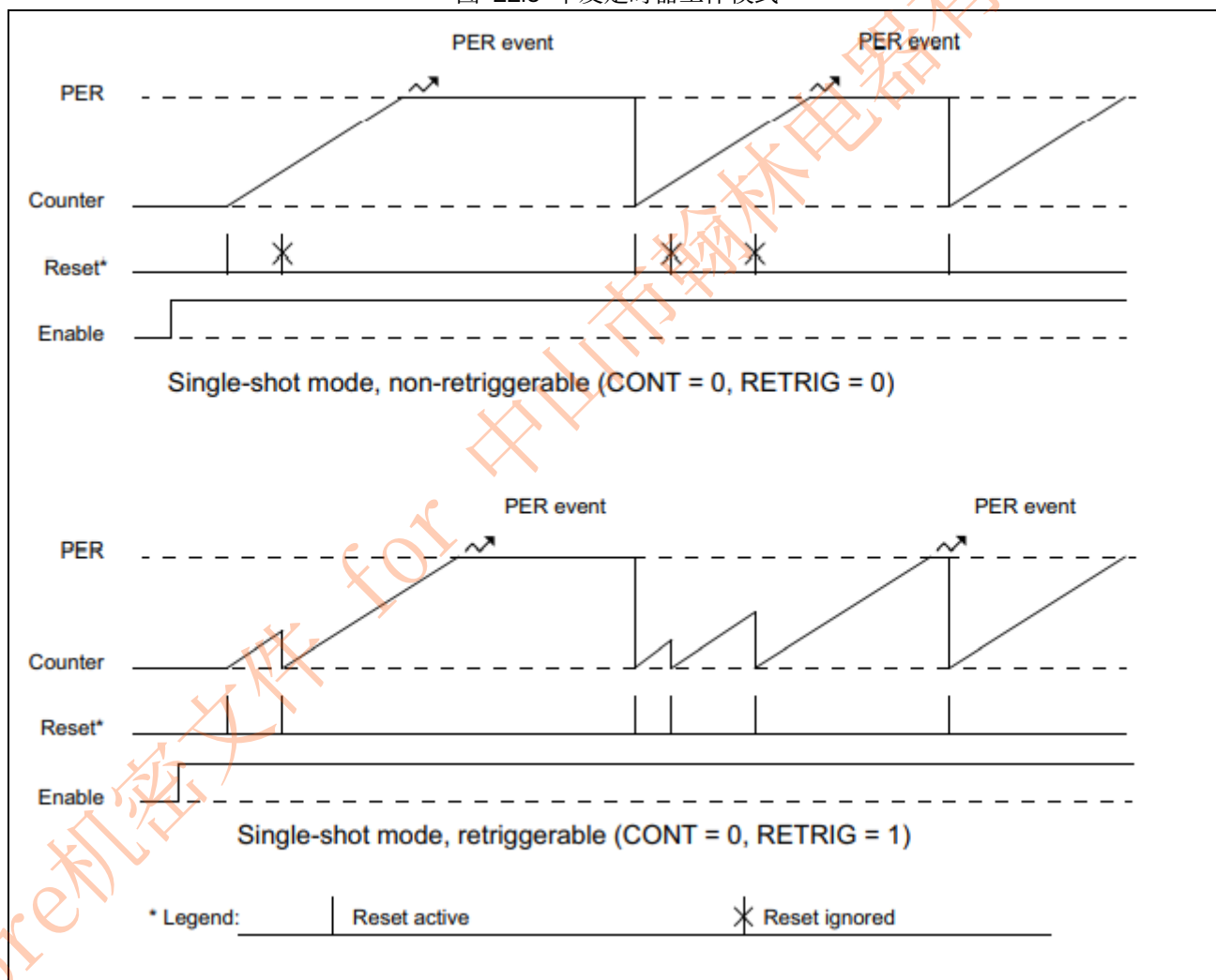


图 22.5 单发定时器工作模式



## 翻转事件

在连续模式下，如果计数器在达到 HRTIM\_PERxR 寄存器中设置的周期值后恢复为 0，则会生成计数器翻转事件。

该事件在 HRTIM 中用于多种用途：

- 置位/复位输出
- 触发寄存器内容更新（从预装载寄存器传输到活动寄存器）
- 触发中断请求（IRQ）或直接内存访问（DMA）请求
- 作为突发模式时钟源或突发启动触发信号
- 作为 ADC 触发信号
- 使重复计数器递减

如果初始计数器值大于定时器启动时的周期值，或者在计数器已超过该值时设置了新周期，计数器不会复位：计数器将在达到最大周期值时溢出，并且重复计数器不会递减。

## 定时器复位

定时单元计数器的复位可通过多达 30 种事件触发，这些事件可同时在 HRTIM\_RSTxR 寄存器中选择，具体包括以下复位源：

- 定时单元：比较 2、比较 4 和更新（3 个事件）
- 主定时器：复位和比较 1..4（5 个事件）
- 外部事件 EXTEVNT1..10（10 个事件）
- 所有其他定时单元（例如，对于定时器 A，有定时器 B..F）：比较 1、2 和 4（12 个事件）

可同时选择多个事件处理多个复位源。在这种情况下，会对多个复位请求进行或运算。如果在同一  $f_{HRTIM}$  时钟周期内生成 2 个计数器复位事件，则会考虑后一个定时器复位事件。

此外，还可以使用 HRTIM\_CR2 寄存器中的 TxRST 位对计数器执行软件复位。这些控制位分组到一个寄存器中，从而可同时复位多个计数器。

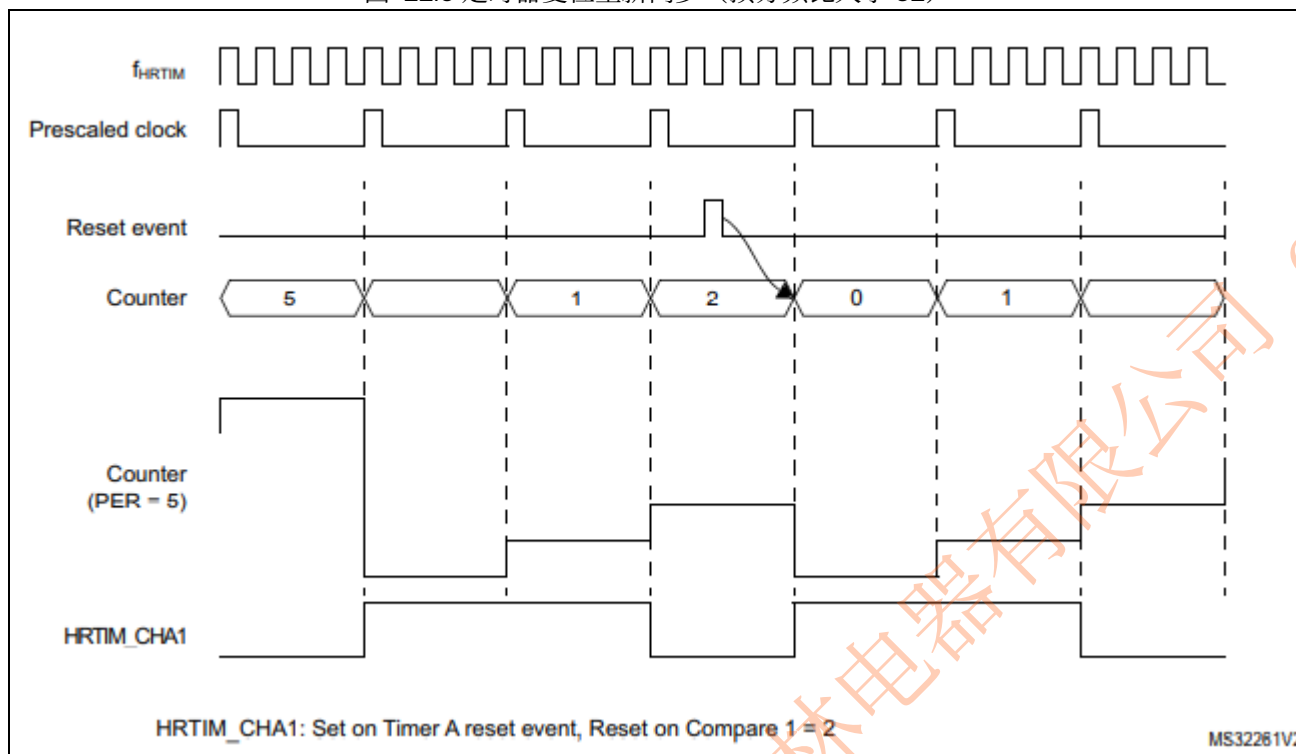
仅当相关计数器已使能后（TxCEN 位置 1），才会考虑复位请求。

当  $f_{HRTIM}$  时钟的分频比大于 32（计数周期大于  $f_{HRTIM}$ ）时，计数器复位事件会延迟到预分频时钟的下一有效边沿。这样可确保在输出跳变同步到复位事件（通常是恒定 Ton 时间转换器）时生成的波形无抖动。

下图展示了当时钟分频比为 128（ $f_{HRTIM}$  除以 4）时的复位处理方式。



图 22.6 定时器复位重新同步（预分频比大于 32）



## 重复计数器

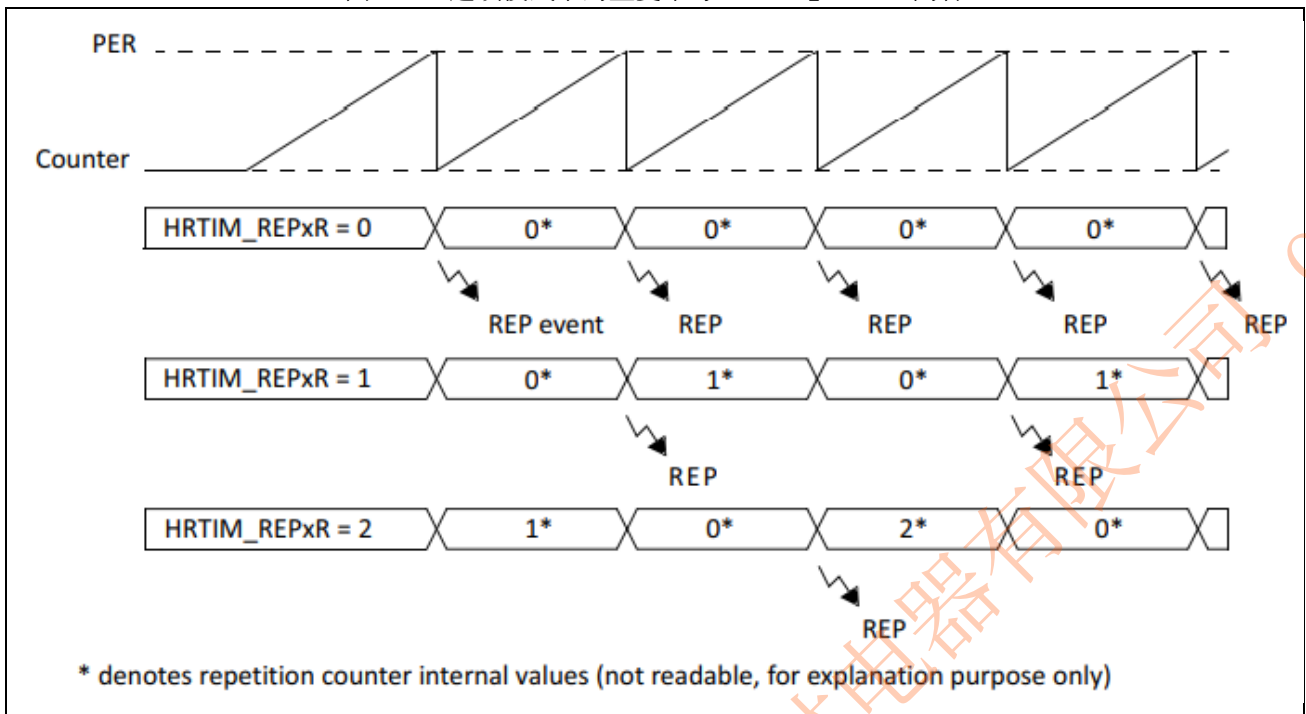
软件通常会在达到周期值时生成中断，从而在下一周期开始之前留出最长的时间用于处理。重复计数器的主要用途是通过分离开关频率和中断频率来调整周期中断率并分担 CPU 的负荷。

定时单元包含重复计数器。该计数器无法读取，但只可使用 HRTIM\_REPxR 寄存器中的自动重载值进行编程。

定时器使能后（TXCEN 位置 1），重复计数器会初始化为 HRTIM\_REPxR 寄存器的内容。定时器使能后，每次计数器由于复位事件或计数器翻转而清空时，重复计数器都会减 1。当重复计数器达到 0 后，会发出 REP 中断或 DMA 请求（若使能，使用 HRTIM\_DIER 寄存器中的 REPIE 和 REPDE 位）。

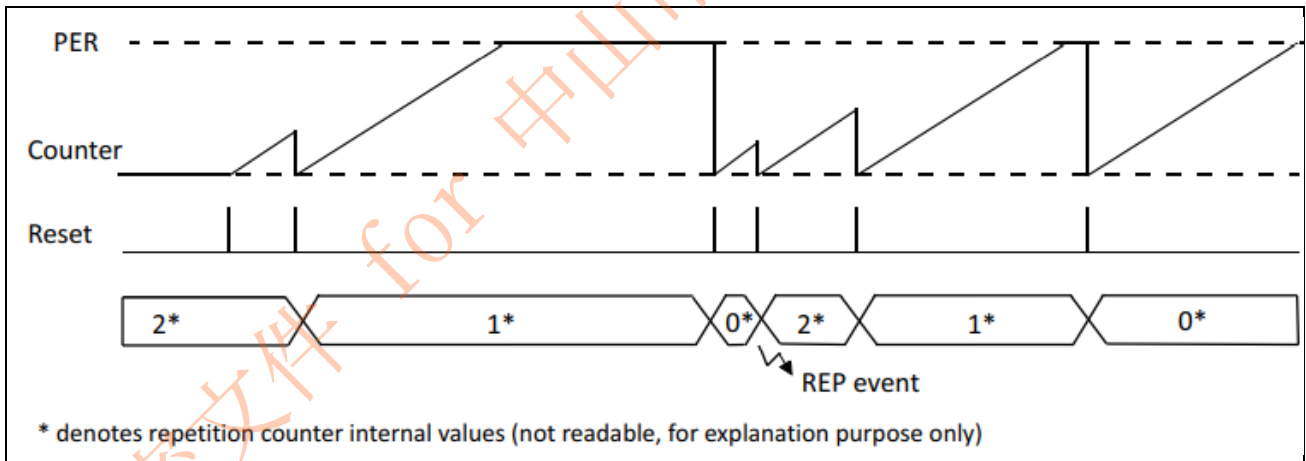
如果 HRTIM\_REPxR 寄存器设为 0，会在每个周期产生中断。如果值大于 0，则会  $(HRTIM\_REPxR + 1)$  个周期后产生 REP 中断。下图显示了连续模式下重复计数器取不同值时的操作。

图 22.7 连续模式下的重复率与 HRTIM\_REPxR 内容



无论在连续模式还是单发模式下，如果计数器在达到周期值（可变频率操作）之前复位，则也可使用重复计数器（如下图所示）。复位会使重复计数器在计数器使能后（TxCEN 位置 1）执行第一次启动时递减。

图 22.8 单发模式下的重复计数器行为



来自 HRTIM\_SCIN[3:1]源的复位或启动事件会像其他任何复位事件一样使重复计数器递减。但在通过 SYNCIN 启动的单发模式下（HRTIM\_TIMxCR 寄存器中的 SYNCSTRTx 位置 1），重复计数器仅会在周期后出现第一个复位事件时递减。任何后续的复位事件都不会更改重复计数器的值，直至计数器通过新的 HRTIM\_SCIN[3:1]输入请求重启。

## 置位/复位纵横开关

“置位”事件相当于跳变为输出有效状态，而“复位”事件则相当于跳变为输出无效状态。

波形的极性在输出级中定义，以适应正逻辑或负逻辑外部组件：对于正极性 ( $POLx = 0$ )，有效电平对应于逻辑电平 1；对于负极性 ( $POLx = 1$ )，有效电平对应于逻辑电平 0。

每个定时单元都会控制两路输出的置位/复位纵横开关。这两路输出可通过多达 32 种事件置位、复位或切换，这些事件可从以下源中选择：

- 定时单元：周期、比较 1..4、寄存器更新（6 个事件）
- 主定时器：周期、比较 1..4、HRTIM 同步（6 个事件）
- 所有其他定时单元（例如，对于定时器 A，则为定时器 B..F）：TIMEVNT1..9（下表中描述的 9 个事件）
- 外部事件 EXTEVNT1..10（10 个事件）
- 软件强制（1 个事件）

注：在递增/递减计数模式（UDM 位置 1）下，计数器的周期事件根据 OUTROM[1:0] 位设置来定义。

事件源会进行或运算，可同时选择多个事件。

每路输出均由两个 32 位寄存器控制，一个寄存器包含置位编码（HRTIM\_SETxyR），另一个寄存器包含复位编码（HRTIM\_RSTxyR），其中 x 代表定时单元 A..F，y 代表输出 1 或 2（例如 HRTIM\_SETA1R、HRTIM\_RSTC2R...）。

如果为置位和复位选择了相同事件，则会切换输出状态。每个 HRTIM 周期输出状态的切换次数不能超过 1 次：如果同一周期中有两个连续的切换事件，则仅会考虑第一个切换事件。

仅当计数器使能后（TxCEN 位置 1），才会考虑置位和复位请求，但软件在定时器启动时强制请求允许预置输出的情况除外。

下表汇总了来自其他定时单元的可用于置位和复位输出的事件。编号对应于寄存器中列出的定时器事件（如 TIMEVNTx），空白位置表示不可用事件。

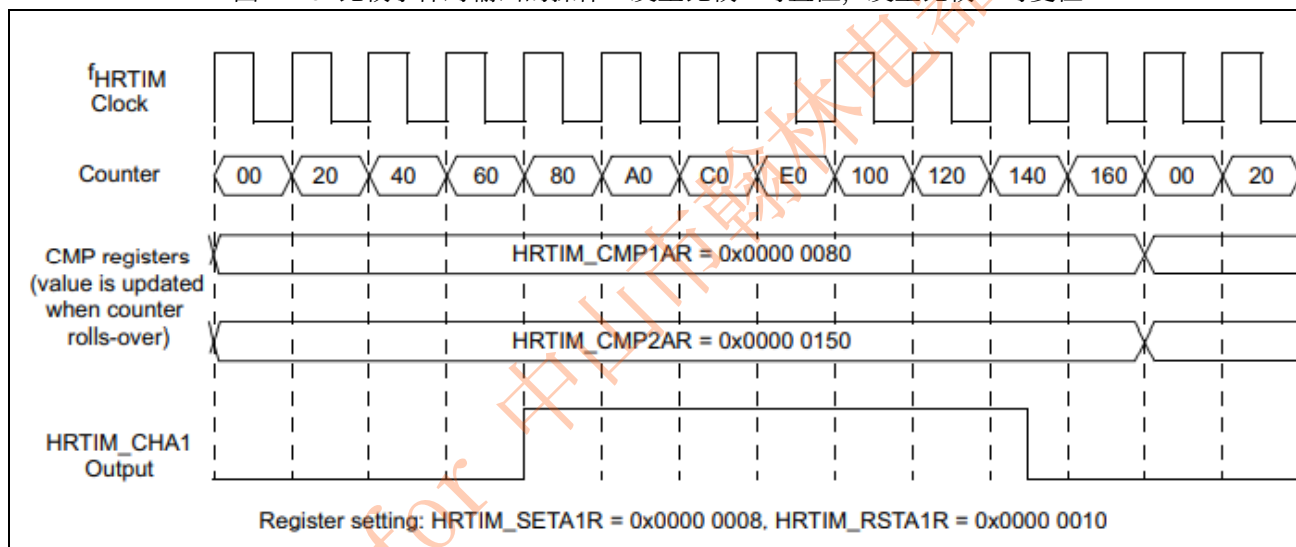
例如，定时器 A 输出可由以下事件置位或复位：定时器 B 比较 1、2，定时器 C 比较 2 和 3... 定时器 E 比较 3 将列为 HRTIM\_SETA1R 中的 TIMEVNT7。

表 22.10 定时器 A 到 F 之间的事件映射

| 源  | 定时器 A |      |      |      | 定时器 B |      |      |      | 定时器 C |      |      |      | 定时器 D |      |      |      | 定时器 E |      |      |      | 定时器 F |      |      |      |
|----|-------|------|------|------|-------|------|------|------|-------|------|------|------|-------|------|------|------|-------|------|------|------|-------|------|------|------|
|    | CMP1  | CMP2 | CMP3 | CMP4 | CMP1  | CMP2 | CMP3 | CMP4 | CMP1  | CMP2 | CMP3 | CMP4 | CMP1  | CMP2 | CMP3 | CMP4 | CMP1  | CMP2 | CMP3 | CMP4 | CMP1  | CMP2 | CMP3 | CMP4 |
| 目标 | TA    | -    | -    | -    | 1     | 2    | -    | -    | -     | 3    | 4    | -    | 5     | 6    | -    | -    | -     | -    | 7    | 8    | -     | -    | -    | 9    |
|    | TB    | 1    | 2    | -    | -     | -    | -    | -    | -     | -    | 3    | 4    | -     | -    | 5    | 6    | 7     | 8    | -    | -    | -     | -    | 9    | -    |
|    | TC    | -    | 1    | 2    | -     | -    | 3    | 4    | -     | -    | -    | -    | -     | 5    | -    | 6    | -     | -    | 7    | 8    | -     | 9    | -    | -    |
|    | TD    | 1    | -    | -    | 2     | -    | 3    | -    | 4     | -    | -    | 5    | -     | -    | -    | -    | 6     | -    | -    | 7    | 8     | -    | 9    | -    |
|    | TE    | -    | -    | -    | 1     | -    | -    | 2    | 3     | 4    | 5    | -    | -     | 6    | 7    | -    | -     | -    | -    | -    | -     | -    | 8    | 9    |
|    | TF    | -    | -    | 1    | -     | 2    | -    | -    | 3     | 4    | -    | -    | 5     | -    | -    | 6    | 7     | -    | 8    | 9    | -     | -    | -    | -    |

下图展示了如何使用两个比较事件来生成 PWM（脉冲宽度调制）信号。

图 22.9 比较事件对输出的操作：发生比较 1 时置位，发生比较 2 时复位



### 发生更新事件时置位/复位

在更新时进行的置位或复位事件是以低分辨率进行的。当 CKPSC[2:0] < 5 时，高分辨率延迟被设置为最大值，因此与其他比较置位/复位事件相比，更新时的置位/复位事件总是滞后的，其抖动在 0 到 31/32 个  $f_{HRTIM}$  时钟周期之间变化。

### 半占空比模式

此模式用于生成占空比固定为 50%、频率可变的方波信号（通常用于使用谐振拓扑的转换器），允许在设定新周期时自动将占空比强制设为周期值的一半。

要使能此模式，应向 HRTIM\_TIMxCR 寄存器中的 HALF 位写入 1。HRTIM\_PERxR 寄存器写入数值后，会自动将比较 1 值更新为 HRTIM\_PERxR/2 值。

生成方波的输出必须编程为在发生 CMP1 事件时进行一次跳变，在发生周期事件时进行一次跳变，具体如下：

- HRTIM\_SETxyR = 0x0000 0008, HRTIM\_RSTxyR = 0x0000 0004, 或者
- HRTIM\_SETxyR = 0x0000 0004, HRTIM\_RSTxyR = 0x0000 0008

HALF 模式会覆盖 HRTIM\_CMP1xR 寄存器的内容。访问 HRTIM\_PERxR 寄存器仅会更新比较 1 内部寄存器。用户可访问的 HRTIM\_CMP1xR 寄存器不会更新为 HRTIM\_PERxR / 2 的值。

当使能预装载 (PREEN = 1、MUDIS、TxUDIS) 时，则会在发生更新事件时刷新比较 1 活动寄存器。如果禁止预装载 (PREEN = 0)，则在 HRTIM\_PERxR 写入数值后，比较 1 活动寄存器会立即更新。

如果使能了 HALF 模式，周期必须大于或等于 6 个  $f_{HRTIM}$  时钟周期（如果 CKPSC[2:0] = 0，则为 0xC0；如果 CKPSC[2:0] = 1，则为 0x60；如果 CKPSC[2:0] = 2，则为 0x30；依此类推）。

### 交错模式

交错模式是对半周期模式的补充，有助于实现交错拓扑。

当更新 HRTIM\_PERxR 的值时，该模式允许自动重新计算比较寄存器的内容。

选择交错模式是通过 HRTIM\_MCR 和 HRTIM\_TIMxCR 寄存器中的 HALF 位和 IL[1:0] 位来完成的，如下表所示。

表 22.11 交错模式选择

| HALF 位 | INTLVD[1:0] 位 | 模式          |
|--------|---------------|-------------|
| 0      | 00            | 失能          |
| 0      | 01            | 三重交错 (120°) |
| 0      | 10            | 四重交错 (90°)  |
| 0      | 11            | 保留          |
| 1      | xx            | 双重交错 (180°) |

下表列出了三种可用模式的比较值。比较寄存器的内容将被覆盖。相应的比较事件可用于触发输出置位/复位或重置从属定时器。

表 22.12 交错模式下比较 1 至 3 的值

| 模式     | 双重交错<br>180° | 三重交错<br>120° | 四重交错<br>90°  |
|--------|--------------|--------------|--------------|
| CMP1 值 | PERxR/2      | PERxR/3      | PERxR/4      |
| CMP2 值 | 无影响          | 2x (PERxR/3) | PERxR/2      |
| CMP3 值 | 无影响          | 无影响          | 3x (PERxR/4) |

注：在半占空比模式和交错模式下，比较寄存器由硬件控制，直接写入这些寄存器不会产生影响。但是，写入的值会被存储在预加载寄存器中，并在退出这些模式后的更新事件上生效。

注：三重和四重交错模式不应与其他使用 CMP2 的模式（如双通道 DAC 触发模式和半触发占空比模式）同时使用。

### 零占空比异常情况

对于宽度小于 3 个  $t_{HRTIM}$  时钟周期的脉冲，不支持高分辨率行为（参见章节：置位/复位事件优先级和窄脉冲管理），并且禁止在 HRTIM\_TIMxCMPy 寄存器中写入任何严格小于  $f_{HRTIM}$  时钟 3 个周期的值（如果 CKPSC[2:0] = 0，则为 0x60；如果 CKPSC[2:0] = 1，则为 0x30；如果 CKPSC[2:0] = 2，则为 0x18；依此类推）（参见章节：HRTIM 定时器 x 比较 1 寄存器（HRTIM\_CMP1xR）（x = A 到 F））。

然而，要跳过输出脉冲并实现零占空比，只需在满足以下条件时，在以下两个寄存器中写入零值即可：HRTIM\_TIMxCMP1 和 HRTIM\_TIMxCMP3：

- 输出设置 (SET) 事件由周期 (PERIOD) 事件生成
- 输出复位 (RESET) 由比较 1（或比较 3）事件生成
- 比较 1（或比较 3）事件在定时器单元本身内是有效的，并且不用于其他定时单元

对于其他任何用例，可以通过使用完全相同的比较值（大于  $f_{HRTIM}$  时钟的 3 个周期）来编程 SET 和 RESET 事件来实现。在这种情况下，输出将被强制复位（参见章节：置位/复位事件优先级和窄脉冲管理）。

### 交换模式

此模式允许通过单个位访问来交换两个输出：输出 1 信号连接到输出 2 引脚，而输出 2 信号连接到输出 1 引脚。输出的交换通过 HRTIM\_CR2 寄存器中的 SWPx 位触发，并在下一个更新事件上生效。

交换发生在置位/复位纵横开关单元之前，如下所示：

- 如果 SWPx = 0，则 HRTIM\_SETx1R 和 HRTIM\_RSTx1R 编码为输出 1，HRTIM\_SETx2R 和 HRTIM\_RSTx2R 编码为输出 2
- 如果 SWPx = 1，则 HRTIM\_SETx1R 和 HRTIM\_RSTx1R 编码为输出 2，HRTIM\_SETx2R 和 HRTIM\_RSTx2R 编码为输出 1

交换模式仅影响预加载寄存器，而不影响有效寄存器。

注：在使用交换模式时，必须使能预加载模式。



因此，它不会修改与输出阶段相连的正规输出并行的辅助输出（有关详细信息，请参阅后续章节）。它们提供以下内部状态、事件和信号：

- O1CPY、O2CPY、SETxy 和 RSTxy 状态标志，以及相应的中断和 DMA 请求
- 输出设置/复位时的捕获触发器（TA2、TB2、TC2、TD2、TE2、TF2）
- 使用 Tx2 输出副本生成的外部事件滤波器

例如，当 SWP = 0 时，SETx1 标志与输出 1 相关；当 SWPx = 1 时，它与输出 2 相关。

同样，交换模式不会改变 HRTIM\_OUTxR 寄存器中控制位的归属（DIDLx、CHPx、FAULTx[1:0]、IDLESx、POLx 位）。例如，POL1 位控制输出 1 的极性，无论 SWP 位的值如何。

注：在推挽模式（在 HRTIM\_TIMxCR 寄存器中 PSHPLL = 1）下，SWPx 位将被忽略。

## 捕获

定时单元能够在内部和外部事件的触发下捕获计数器值。捕获的目的是：

- 测量事件到达时间或发生间隔。
- 在自动延迟模式下更新比较 2 和比较 4 的值（参见：自动延迟模式）。

捕获操作以  $f_{\text{HRTIM}}$  的分辨率进行：对于时钟预分频比小于 32 ( $\text{CKPSC}[2:0] < 5$ ) 的情况，寄存器的最低有效位是不重要的（读取为 0）。

定时器包含 2 个捕获寄存器：HRTIM\_CPT1xR 和 HRTIM\_CPT2xR。捕获触发事件在 HRTIM\_CPT1xCR 和 HRTIM\_CPT2xCR 寄存器中编程。

定时单元计数器的捕获可以由多达 28 个事件触发，这些事件可以同时 HRTIM\_CPT1xCR 和 HRTIM\_CPT2xCR 寄存器中选择，来源包括：

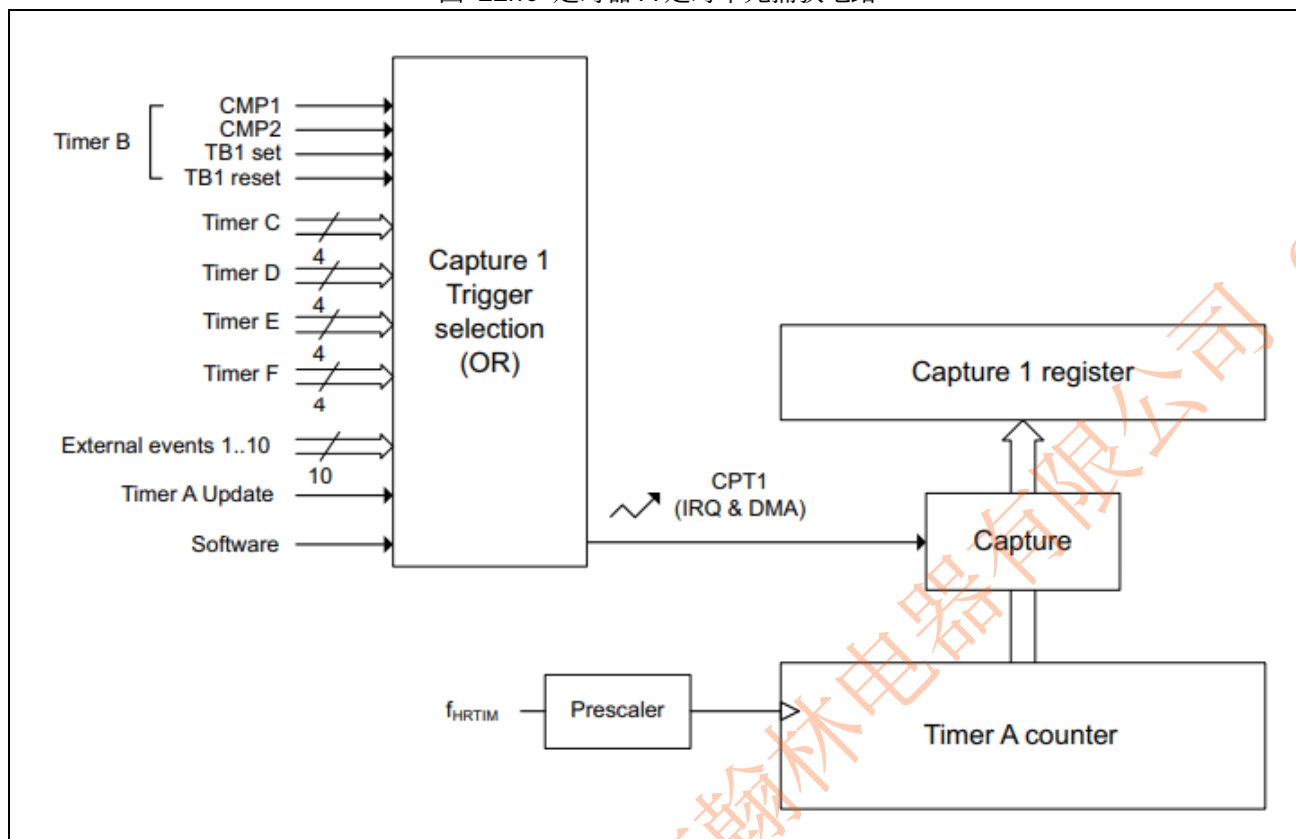
- 外部事件，EXTEVNT1..10（10 个事件）
- 所有其他定时单元（例如，对于定时器 A，则为定时器 B..F）：比较 1、2 和输出 1 置位/复位事件（16 个事件）
- 定时单元：更新（1 个事件）
- 软件捕获（1 个事件）

可同时选择多个事件处理多个捕获触发信号。在这种情况下，会对多个并发触发请求进行或运算。如果 HRTIM\_TIMxDIER 寄存器中的 CPTxIE 和 CPTxDE 位已置 1，捕获可生成中断或 DMA 请求。

电路未采用避免重复捕获的机制：即使前一个值未读取、或者捕获标志未清零，也会触发新捕获。



图 22.10 定时器 A 定时单元捕获电路



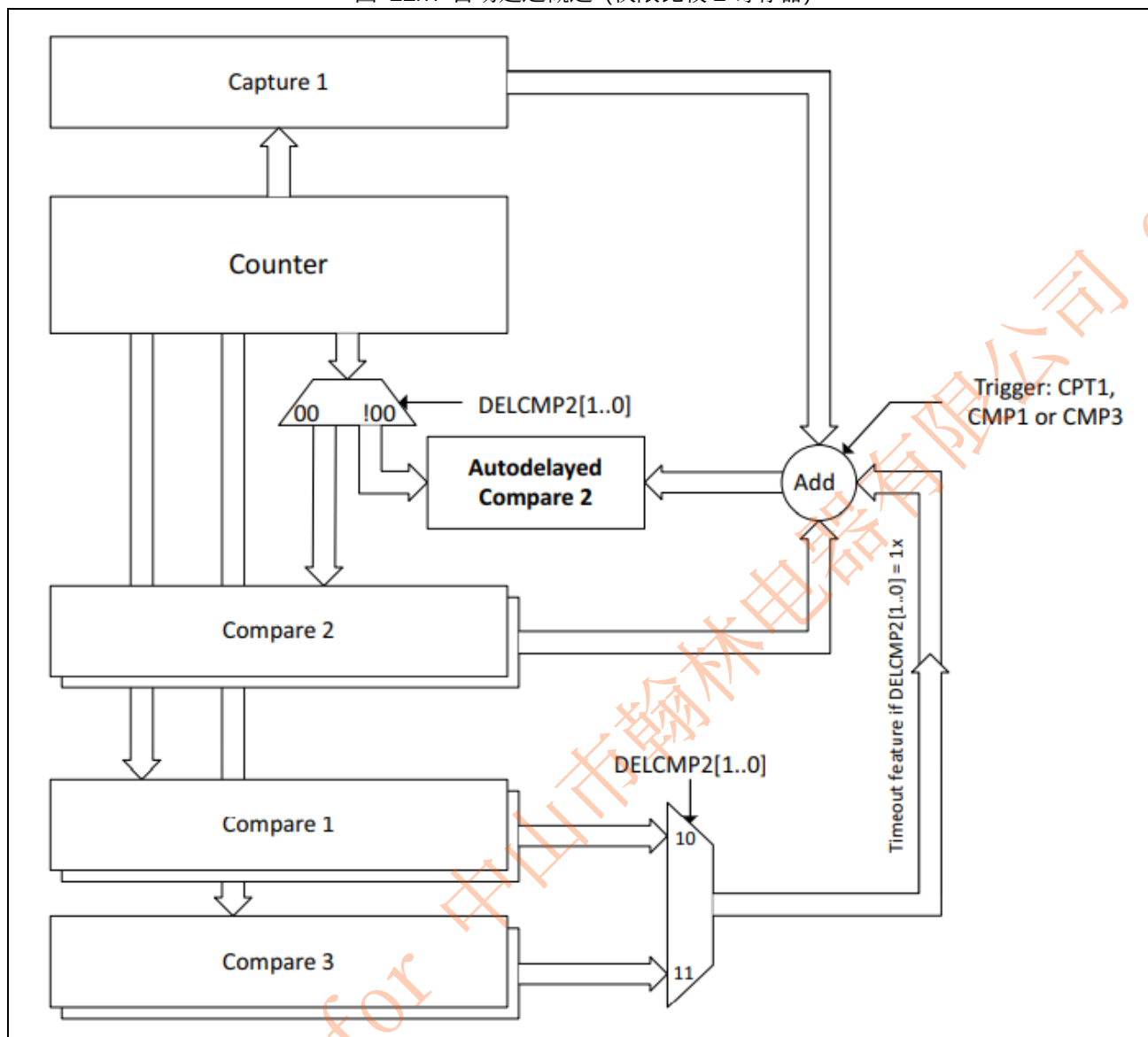
## 自动延迟模式

此模式可相对于捕获事件生成比较事件，这样一来，便可在捕获后的设定时间进行输出更改等操作。在这种情况下，会独立于定时器计数器值进行比较匹配。这样可生成时序与外部事件同步的波形，而无需进行软件计算和中断维护。

只要未触发捕获事件，便会忽略 HRTIM\_CMPxR 寄存器的内容（计数器值与比较值相匹配的情况下，不会生成比较事件）。一旦触发了捕获事件，则会对在 HRTIM\_CMPxR 中编程的比较值与 HRTIM\_CPTxyR 中捕获的计数器值求和，并会用得出的结果更新内部自动延迟比较计数器，如下图所示。自动延迟比较寄存器属于定时单元内部的寄存器，不能读取。HRTIM\_CMPxR 预装载寄存器的内容在计算后不会修改。

此特性仅适用于比较 2 和比较 4 寄存器。比较 2 寄存器与捕获 1 相关联，而比较 4 寄存器与捕获 2 相关联。与常规模式一样，HRTIM\_CMP2xR 和 HRTIM\_CMP4xR 比较寄存器不能编程为小于 3 个  $f_{HRTIM}$  时钟周期的值。

图 22.11 自动延迟概述 (仅限比较 2 寄存器)



自动延迟比较寄存器的有效期从捕获开始，到周期事件为止：计数器达到周期值后，系统会重新设置，比较寄存器在发生捕获之前会处于禁用状态。

HRTIM\_TIMxCR 寄存器中的 DELCMP2[1:0]和 DELCMP4[1:0]位可配置自动延迟模式，具体如下：

- 00  
常规比较模式：会直接将 HRTIM\_CMP2xR 和 HRTIM\_CMP4xR 寄存器的内容与计数器值进行比较。
- 01  
自动延迟模式：会重新计算比较 2 和比较 4 寄存器值，并在捕获 1/2 事件后用计算值与计数器值进行比较。
- 10 或 11  
具有超时的自动延迟模式：会重新计算比较 2 和比较 4 寄存器值，并在捕获 1/2 事件后用计算值与计数器值进行比较；如果缺少捕获 1/2 事件，则在比较 1 匹配（DELCMPx[1:0]= 10）或比较 3 匹配（DELCMPx[1:0]= 11）后用计算值与计数器值进行比较，以便实现超时功能。

进行捕获时，会与  $(\text{HRTIM\_CMP2}/4 \times R + \text{HRTIM\_CPT1}/2 \times R)$  值进行比较。如果周期内未触发捕获，行为将取决于  $\text{DELCMPx}[1:0]$  值：

- $\text{DELCMPx}[1:0] = 01$ ：未生成比较事件
- $\text{DELCMPx}[1:0] = 10$  或  $11$ ：与 2 个比较寄存器值之和进行比较（例如  $\text{HRTIM\_CMP2} \times R + \text{HRTIM\_CMP1} \times R$ ）。如果捕获是在  $\text{CMPx} + \text{CMP1}$ （或  $\text{CMPx} + \text{CMP3}$ ）后触发的，则不会考虑捕获。

下一 PWM 周期开始时再次使能捕获。

如果自动延迟求和的结果大于  $0xFFFF$ （溢出），则会忽略该值且新周期开始之前不会生成比较事件。

注：如果从一个值重新编程为另一个值，以便正确地重新初始化自动延迟机制，则必须复位  $\text{DELCMPx}[1:0]$  位域，例如：

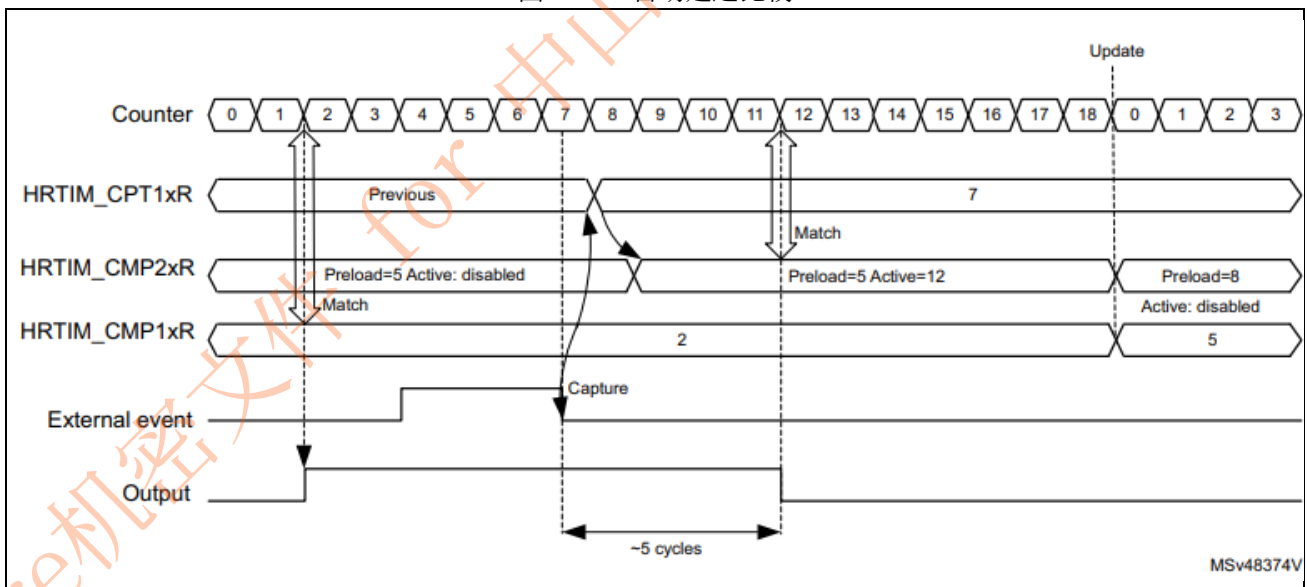
- $\text{DELCMPx}[1:0] = 10$
- $\text{DELCMPx}[1:0] = 00$
- $\text{DELCMPx}[1:0] = 11$

下图举例说明了以下信号是怎样生成的：

- 当计数器等于比较 1 值时，输出置位
- 给定外部事件下降沿 5 个周期后，输出复位

注：为了简化图示，本例中未使用高分辨率 ( $\text{CKPSC}[2:0] = 101$ )，因此计数器以  $f_{\text{HRTIM}}$  的速率递增。同样，图中所示的外部事件信号不含任何重新同步延迟：实际情况下，由于需要通过内部重新同步阶段来处理外部输入信号，因此下降沿与捕获事件之间会有 1 到 2 个  $f_{\text{HRTIM}}$  时钟周期的延迟。

图 22.12 自动延迟比较



使用常规比较通道（例如比较 1）进行输出置位：计数器与比较寄存器的内容匹配后，输出会立即变为有效状态。

使用延迟比较进行输出复位：仅当发生了捕获事件时，才会生成比较事件。如果计数器与延迟比较值（计数值=4）匹配，则不会生成事件。捕获事件由外部事件触发后，捕获寄存器的内容会立即与延迟比较值相加，得出新的比较值。示例中，自动延迟值 4 与的捕获值 7 相加，得出自动延迟比较寄存器中的值 11。从此时起，可生成比较事件，并会在计数器等于 11 时生成，比较事件会使输出复位。

#### 自动延迟模式下的重复捕获管理

使能自动延迟模式时（DEL CMPx[1:0]=01, 10, 11），将阻止重复捕获。

如果同一计数周期内出现多个捕获请求，只会考虑使用第一个捕获请求来计算自动延迟比较值。仅可在以下条件下进行新捕获：

- 自动延迟比较具有匹配的计数器值（比较事件）
- 计数器已翻转（周期）
- 定时器已复位

#### 更改自动延迟比较值

如果已预装载自动延迟比较值（PREEN 位置 1），则无论比较寄存器是何时写入数值的、是否发生了捕获事件（请参见下图，其中，延迟是在计数器翻转时更改的），都会在发生下一更新事件（例如周期事件）时考虑新的比较值。

如果已禁止预装载（PREEN 位复位），则即使比较值在捕获事件发生后进行了修改，也会立即考虑新的比较值，如下例所示：

1. 在 t1 时刻，DEL CMP2 = 1。
2. 在 t2 时刻，CMP2\_act = 0x40 => 比较禁止
3. 在 t3 时刻，发生捕获事件，捕获到值 CPTR1 = 0x20。=> 比较使能，比较值= 0x60
4. 在 t4 时刻，CMP2\_act = 0x100（在计数器达到值 CPTR1 + 0x40 之前）=> 比较仍处于使能状态，新比较值= 0x120
5. 在 t5 时刻，计数器达到周期值=> 比较禁止，cmp2\_act = 0x100

同样，如果 CMP1（CMP3）的值在 DEL CMPx = 10 或 11 时发生了变化，并且已禁止预装载：

1. 在 t1 时刻，DEL CMP2 = 2。
2. 在 t2 时刻，CMP2\_act = 0x40 => 比较禁止
3. 在 t3 时刻，发生 CMP3 事件 - 发生捕获 1 事件之前 CMP3\_act = 0x50 => 比较使能，比较值= 0x90
4. 在 t4 时刻，CMP3\_act = 0x100（计数器达到 0x90 之前）=> 比较仍处于使能状态，将在 CMP3\_act= 0x140 时发生比较 2 事件。

#### 半触发模式

此模式的目的是允许同步两个具有可变频率操作且需要 180°相位偏移的交错转换器。基本原理是建立一个主从系统。

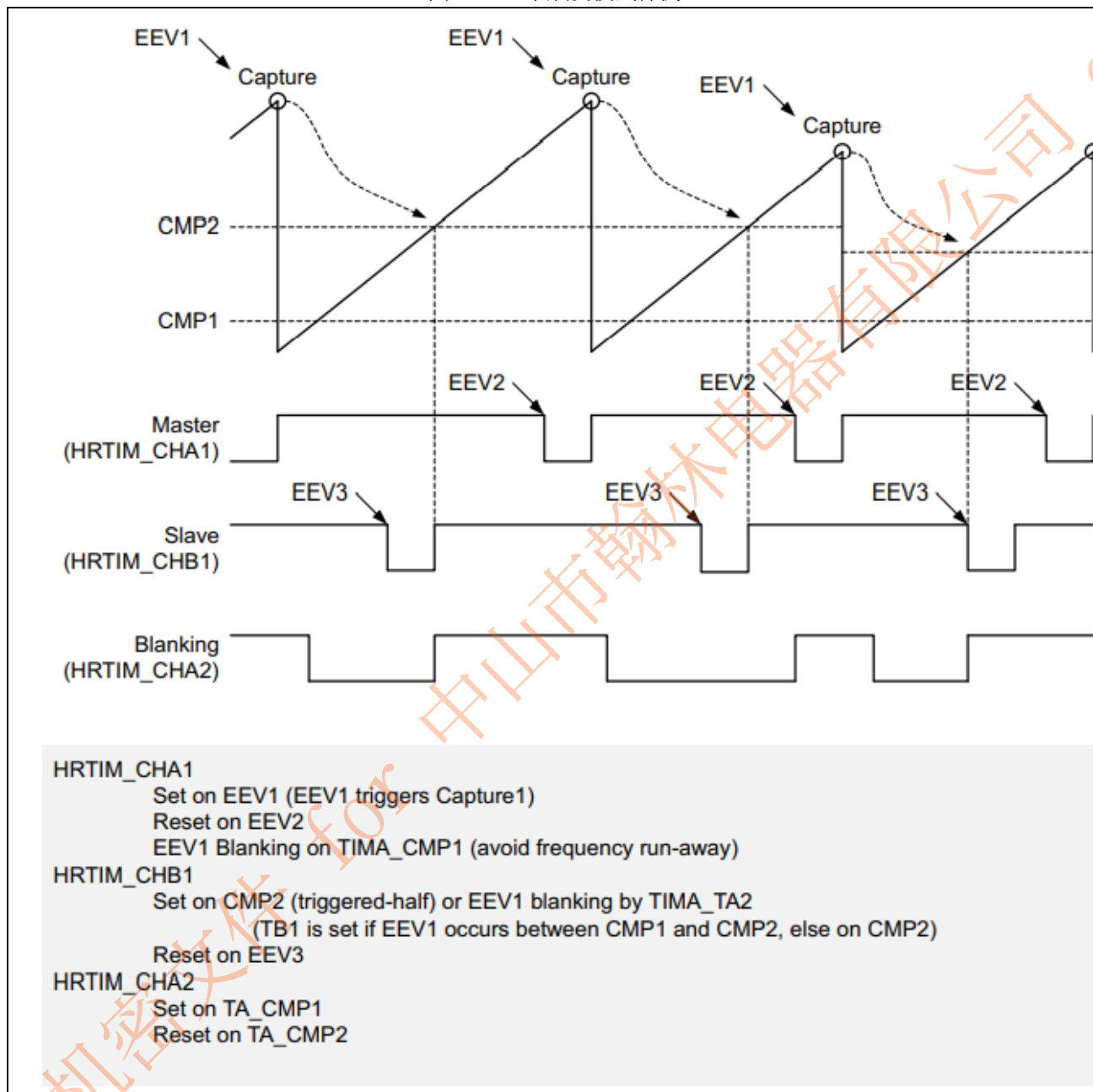
从转换器的同步是基于主转换器前一个开关周期进行连续调整的。这通过使用捕获单元来实现。主转换器的开关周期被捕获，除以 2，然后由硬件存储在比较 2 寄存器中。比较 2 寄存器包含一个值，该值等于捕获周期的一半，即主转换器的开关周期。然后，可以使用比较 2 事件来触发管理从转换器的第二个计时单元。

通过设置 HRTIM\_TIMxCR2 寄存器中的 TRGHF 位来使能此模式。一旦计时器开始运行（TxEN 位设置），该位就不能更改。

半触发模式不能与使用 CMP2 的其他模式（双通道 DAC 触发、交错和平衡空闲模式）同时使用。

用户可以写入 CMP2 的初始值，但一旦触发第一次捕获，该值就会被忽略。当 TRGHLF 位被复位时，CMP2 的预加载机制将被禁用。

图 22.13 半触发模式样例



注：在半触发模式下，比较2 寄存器由硬件控制，写入该寄存器不会产生任何效果。但是，写入的值会存储在预加载寄存器中，并在退出该模式后的更新事件上生效。

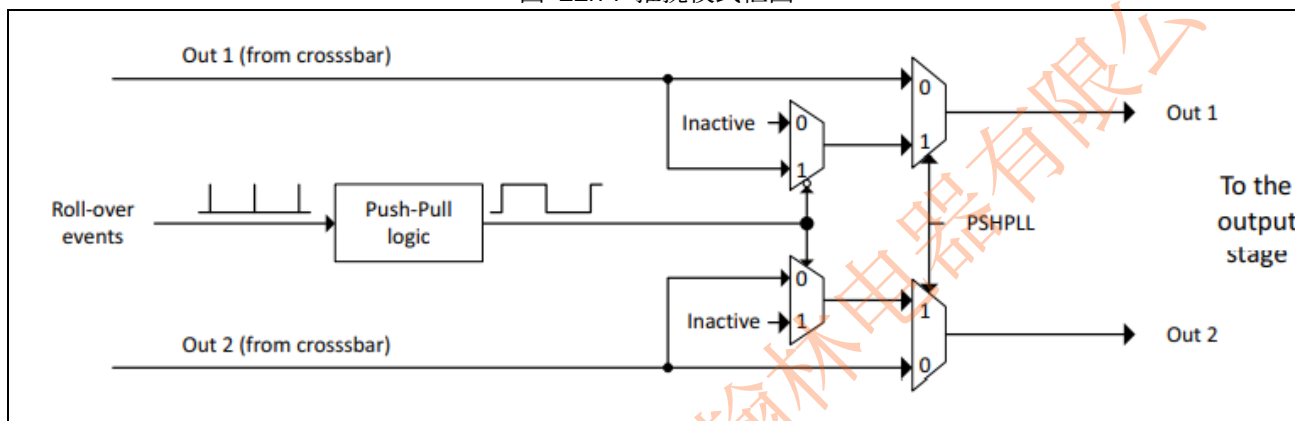
## 推挽模式

该模式的主要目的是使用推挽拓扑驱动转换器。如果需要使用延迟空闲保护（通常用于谐振转换器），也需要使能该模式（请参考章节：延迟保护）。

通过将 HRTIM\_TIMxCR 寄存器中的 PSHPLL 位置 1，即可使能推挽模式。

在该模式下，会按照周期将纵横开关生成的信号交替地施加到输出 1 和输出 2 上（如果信号施加到输出 1 上，则输出 2 保持其无效状态，反之亦然）。重定向速率（推挽频率）由定时器的周期事件定义，如下图所示。推挽周期是定时器计数周期的两倍。

图 22.14 推挽模式框图



推挽模式在定时器以连续模式和单次触发模式运行时都可用。要停止推挽操作并在重新使能之前重置计数器，必须禁用定时器。

为了获得正确的行为，必须选择作为计数器复位源的事件来设置（或复位）输出。如果输出在周期时设置，则必须设置输出；否则，必须复位它。如果不这样做，输出从其无效周期切换到有效周期时可能会出现错误（可能会意外上升或保持低位）。

两路输出的信号波形由 HRTIM\_SETxyR 和 HRTIM\_RSTxyR 定义。需要  $\text{HRTIM\_SETx1R} = \text{HRTIM\_SETx2R}$  且  $\text{HRTIM\_RSTx1R} = \text{HRTIM\_RSTx2R}$ ，才能使两路输出的波形完全相同，并实现平衡的操作。不过，仍可对两路输出进行不同的编程，以实现其他用途。

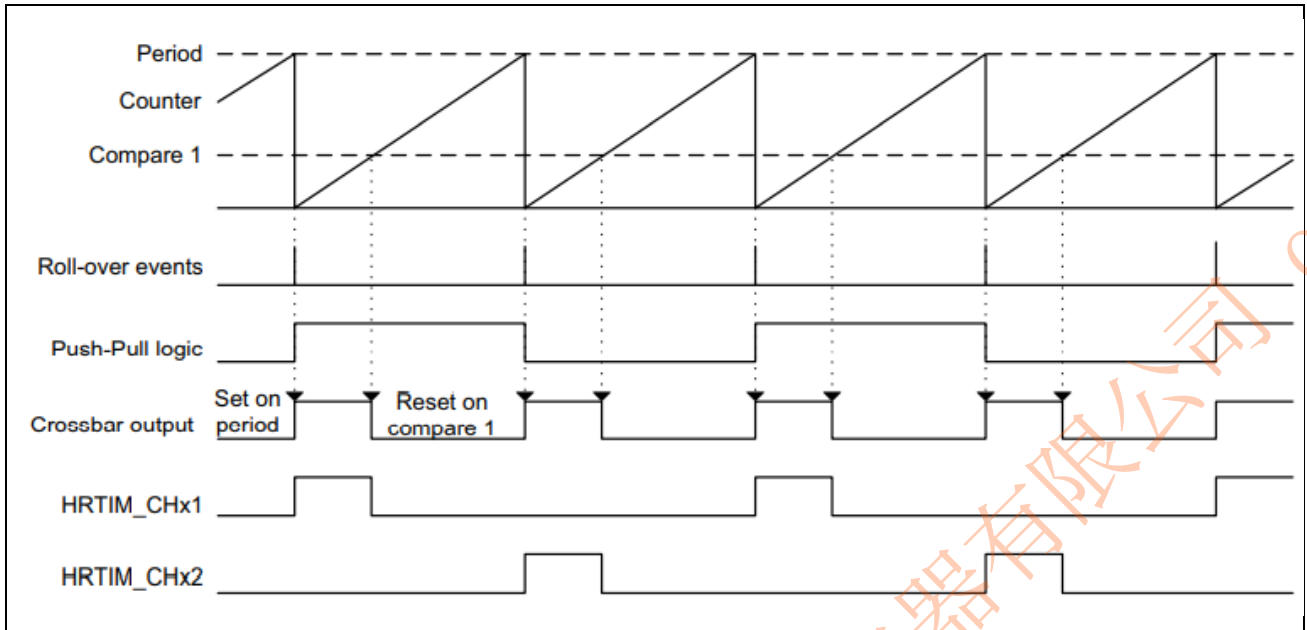
HRTIM\_TIMxISR 中的 CPPSAT 状态位指示目前哪一路输出上的信号有效。CPPSTAT 在推挽模式禁用时复位。

在下图中提供的示例中，定时器内部波形的定义如下：

- 发生周期事件时，输出置位
- 发生比较 1 匹配事件时，输出复位



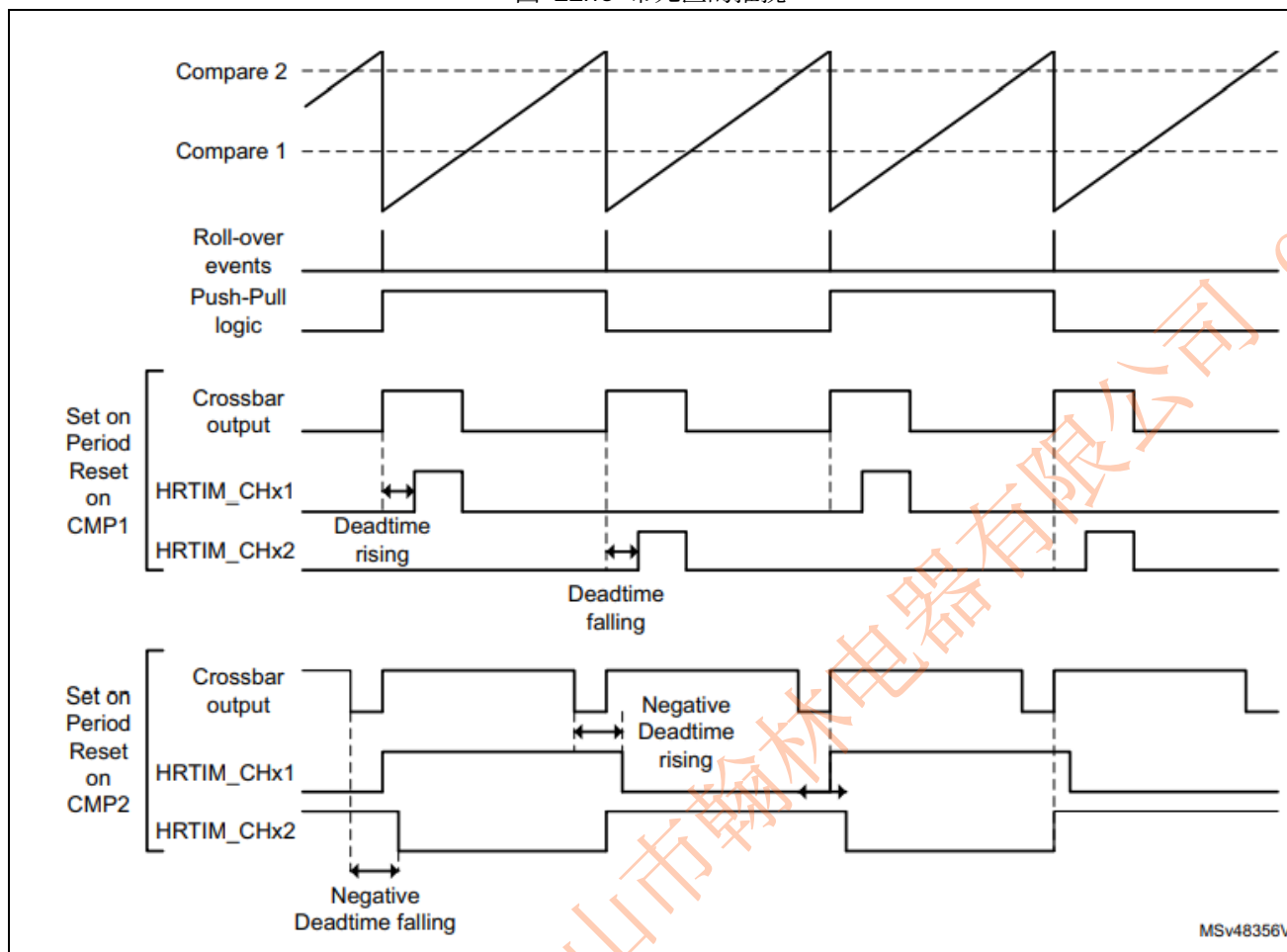
图 22.15 推挽模式样例



下图展示了在推挽模式下如何插入死区时间，包括正死区时间和负死区时间。在这种情况下，输出不再进行互补，死区时间分别应用于每个输出（纵横开关的输出 1 和输出 2 都被使用）。



图 22.16 带死区的推挽



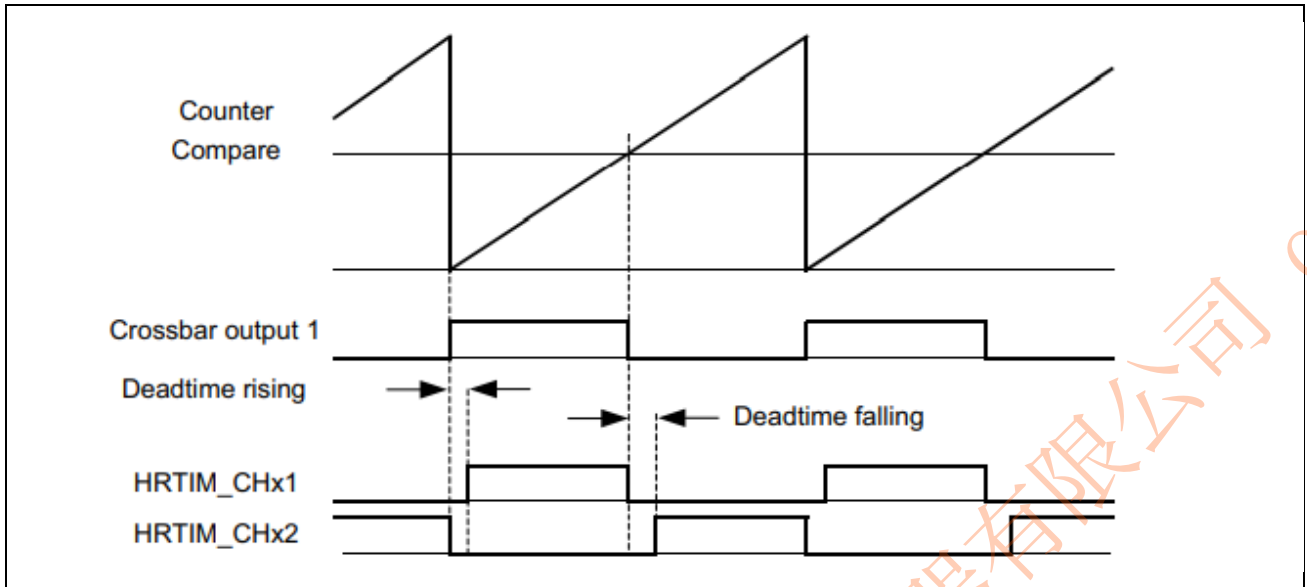
## 死区时间

死区插入单元可通过单个参考波形生成一对互补信号，并且有效状态跳变之间的延迟可编程。这通常用于使用半桥或全桥的拓扑，可简化软件操作流程：只需要对 1 个波形进行编程和控制即可驱动两路输出。

死区插入通过将 HRTIM\_OUTxR 寄存器的 DTEN 位置 1 来使能。互补信号基于为输出 1 定义的参考波形构建而成，使用 HRTIM\_SETx1R 和 HRTIM\_RSTx1R 寄存器：如果 DTEN 位置 1，则 HRTIM\_SETx2R 和 HRTIM\_RSTx2R 寄存器无效。

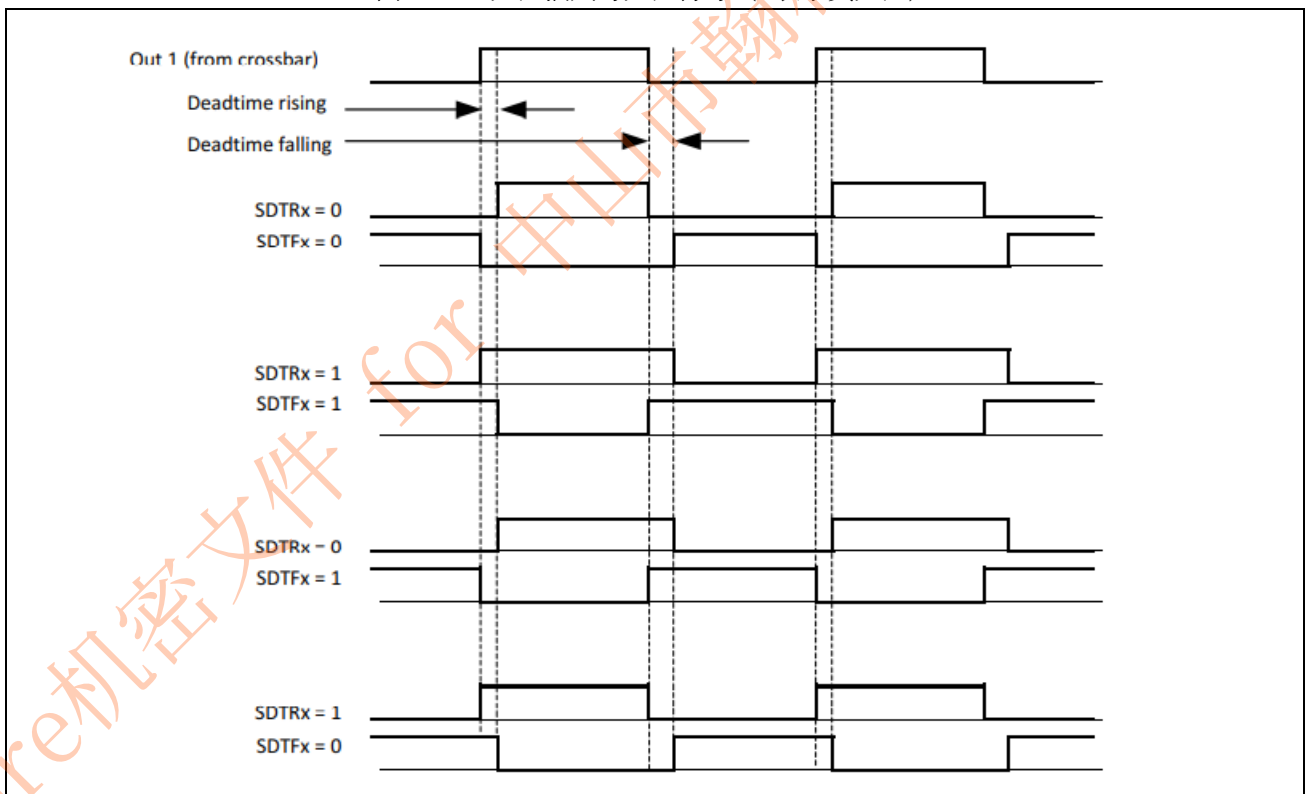
可按照与参考波形上升沿与下降沿的关系定义两个死区，如下图所示。

图 22.17 已插入死区的互补输出



如果需要使用一些控制叠加，可定义负死区值，此时要使用死区符号位（HRTIM\_DT<sub>x</sub>R 寄存器中的 SDTF<sub>x</sub> 和 SDTR<sub>x</sub> 位）。下图显示了各符号对应的互补信号波形。

图 22.18 死区插入与死区符号（1 表示负死区）



死区值使用 DT<sub>F</sub>[8:0] 和 DT<sub>R</sub>[8:0] 位域定义，基于根据 DTPRSC[2:0] 位预分频的特定时钟，具体如下：

$$t_{DTx} = \pm DTx[8:0] \times t_{DTG}$$

其中 x 可以是 R 或 F，而  $t_{DTG} = (2^{(DTPRSC[2:0])}) \times (t_{HRTIM} / 8)$ 。

下表给出了根据预分频值得出的分辨率和最大绝对值。

表 22.13 死区分辨率和最大绝对值

| DTPRSC[2:0] | $t_{DTG}$        | $t_{DTx}$ 最大值   | $f_{HRTIM} = 192 \text{ MHz}$ |                                       |
|-------------|------------------|-----------------|-------------------------------|---------------------------------------|
|             |                  |                 | $t_{DTG} \text{ (ns)}$        | $ t_{DTx}  \text{ 最大值} (\mu\text{s})$ |
| 000         | $t_{HRTIM} / 8$  | $511 * t_{DTG}$ | 0.65                          | 0.34                                  |
| 001         | $t_{HRTIM} / 4$  |                 | 1.30                          | 0.66                                  |
| 010         | $t_{HRTIM} / 2$  |                 | 2.60                          | 1.33                                  |
| 011         | $t_{HRTIM}$      |                 | 5.22                          | 2.67                                  |
| 100         | $2 * t_{HRTIM}$  |                 | 10.41                         | 5.32                                  |
| 101         | $4 * t_{HRTIM}$  |                 | 22.60                         | 10.65                                 |
| 110         | $8 * t_{HRTIM}$  |                 | 41.67                         | 21.29                                 |
| 111         | $16 * t_{HRTIM}$ |                 | 83.34                         | 42.59                                 |

下两张图展示了在所有死区配置下，死区发生器对脉宽小于死区值的参考波形的处理方式。

图 22.19 低脉冲宽度的互补输出 ( $SDTRx = SDTFx = 0$ )

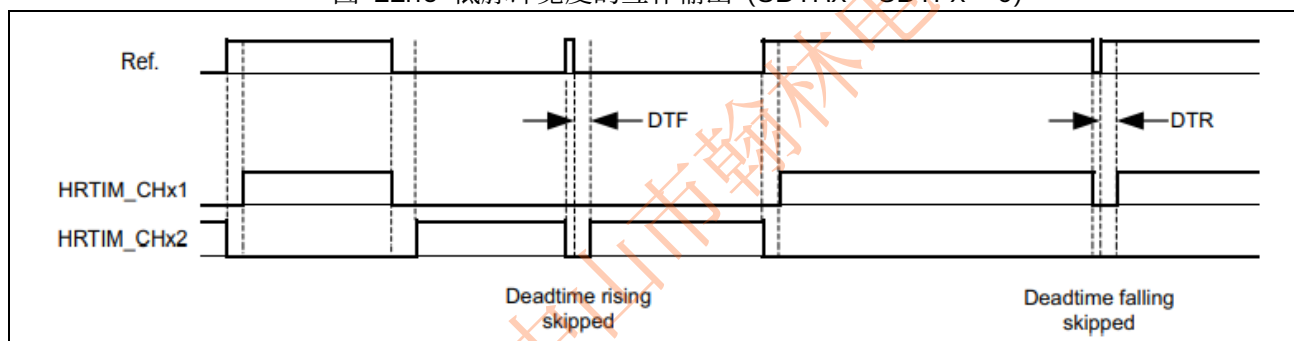


图 22.20 低脉冲宽度的互补输出 ( $SDTRx = SDTFx = 1$ )

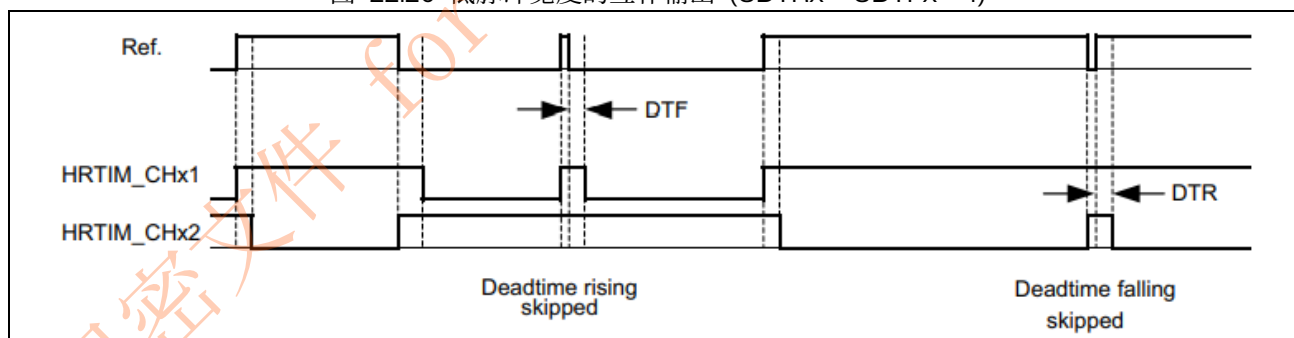
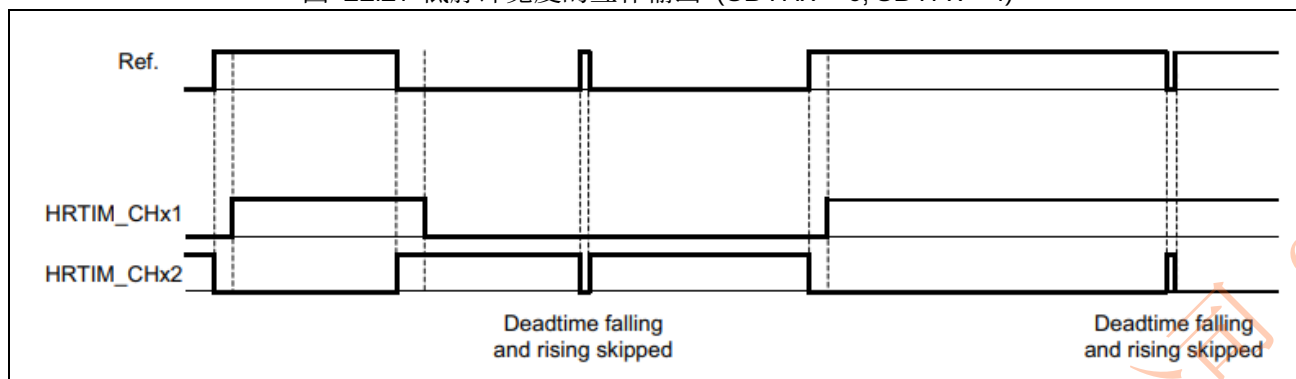
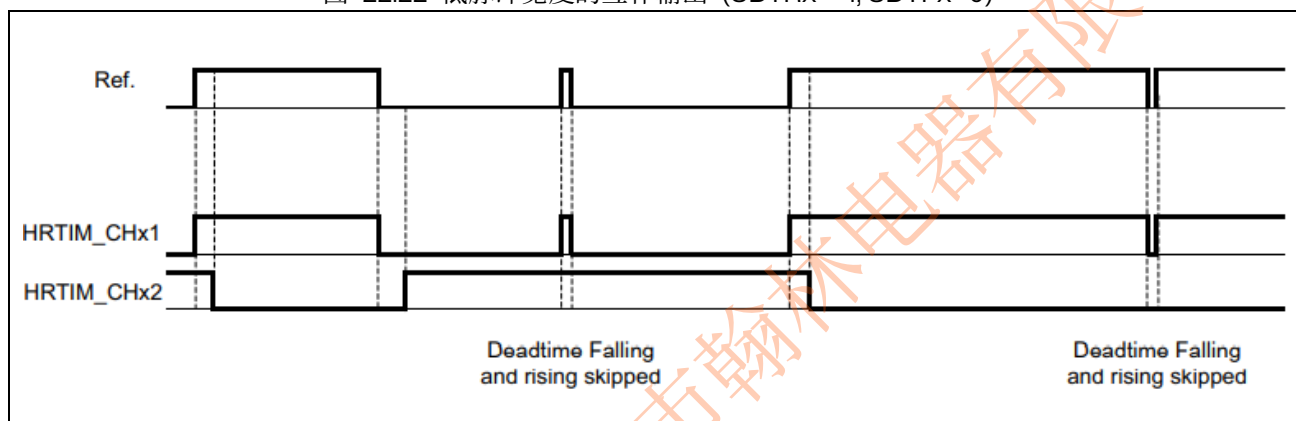


图 22.21 低脉冲宽度的互补输出 ( $SDTRx = 0, SDTFx = 1$ )

图 22.22 低脉冲宽度的互补输出 ( $SDTRx = 1, SDTFx = 0$ )


出于安全方面的考虑，可使用  $DTFLKx$ 、 $DTRLKx$ 、 $DTFSLKx$  和  $DTRSLKx$  锁定死区的符号和/或数值，避免对死区寄存器进行误写操作。一旦这些位置 1，在下次系统复位之前，关联位和位域将变为只读状态。

**警告：**在以下情况下，不得更改 DTEN 位：

- 当定时器被使能时 ( $TxEN$  位被置 1)
- 当定时器的输出被另一个定时器置位/复位时 ( $TxEN$  复位时)

否则，会导致不可预测的行为。

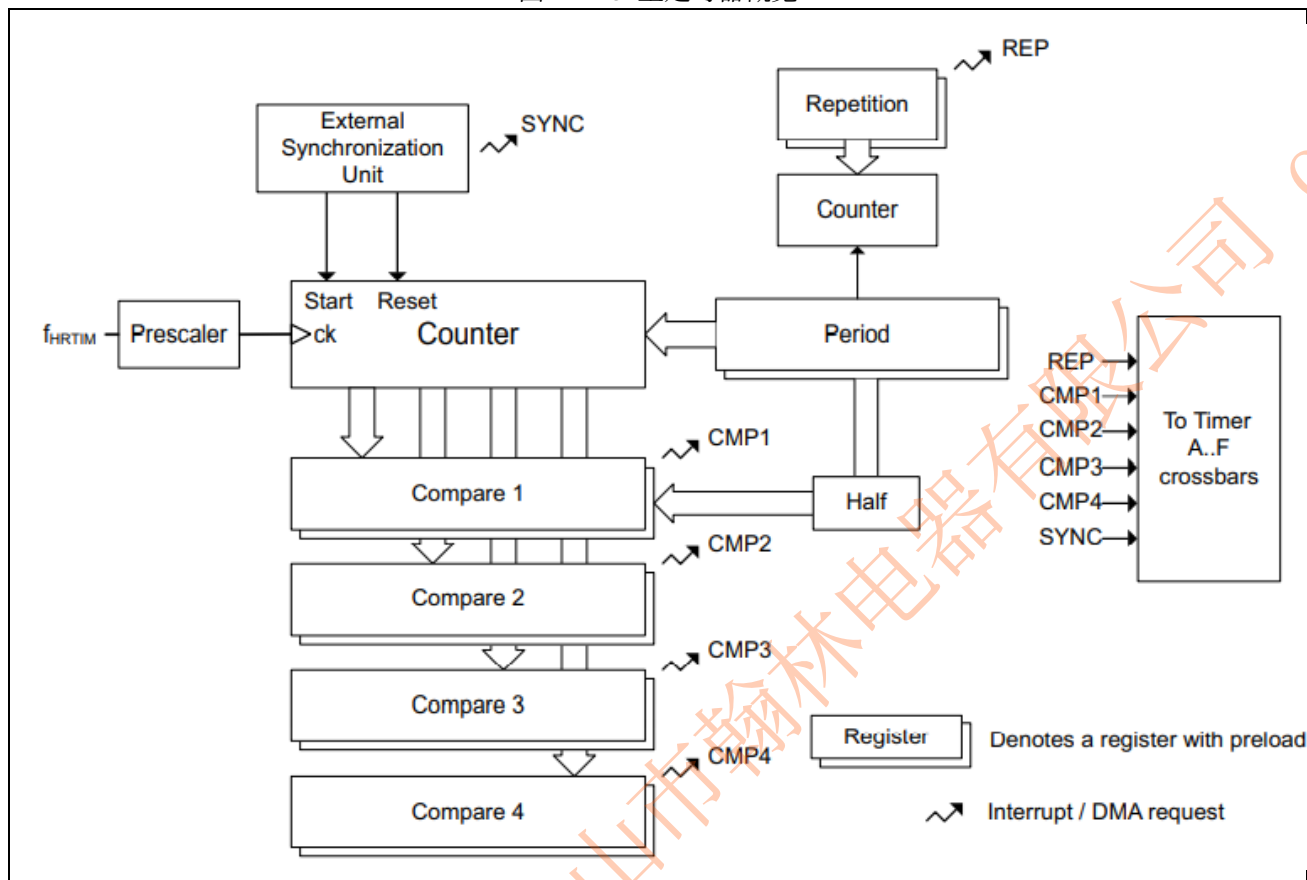
因此，必须禁用定时器 ( $TxCEN$  位复位)，并确保相应的输出也被禁用。

对于 DTEN 必须在突发模式使能的同时置 1 以确保进入突发模式后就有死区的这一特殊情况 ( $BME = 1$ 、 $DIDL = 1$ 、 $IDLEM = 1$ )，需要在将 DTEN 位置 1 之前通过软件命令将两路输出强制设为其 IDLES 状态 ( $SST$ 、 $RST$  位)。这样做是为了避免在死区尚未使能之前就进入突发模式时会造成的不良后果。

### 22.3.5 主定时器

主定时器的主要用途是向 6 个定时单元提供公用信号，以便进行同步或置位/复位输出。主定时器不会直接控制任何输出，但仍可间接地供置位/复位纵横开关使用。

图 22.23 主定时器概览



- 主定时器未关联输出，也没有输出相关的控制
- 主定时器没有自己的纵横开关单元，也没有推挽或死区模式
- 主定时器只能通过外部同步电路复位
- 主定时器没有捕获单元，也没有自动延迟模式
- 主定时器不含外部事件消隐和窗口电路
- 主定时器的中断/DMA 请求数量有限：比较 1.4、重复、寄存器更新和外部同步事件

主定时器还会利用 MCU 内部和外部（输入/输出）资源处理整个 HRTIM 定时器的外部同步（请参见章节：将 HRTIM 与其他定时器或 HRTIM 实例同步）

主定时器控制寄存器的映射偏移与定时单元寄存器的偏移相同。

### 22.3.6 上下计数模式

HRTIM 原生设计为向上计数器。然而，它也提供了一种带有上下计数器的操作模式，也称为中心对齐模式。

这种模式通过 HRTIM\_TIMxCR2 寄存器中的 UDM 位来使能。一旦定时器开始运行(TxEN 位被设置)，就不应更改此位。此模式仅适用于 TIMA..F。主定时器仅工作在向上计数模式。

并非所有 HRTIM 功能都支持上下计数。本节将详细介绍与向上计数模式相比的功能差异。

在上下计数模式中，HRTIM\_PERxR 中的周期必须预加载（或静态）。它只能在周期事件或计数器复位时更新。

设置/复位纵横开关的编程方式如下：

来自定时单元的事件将根据计数器的向上/向下方向来设置/复位输出：

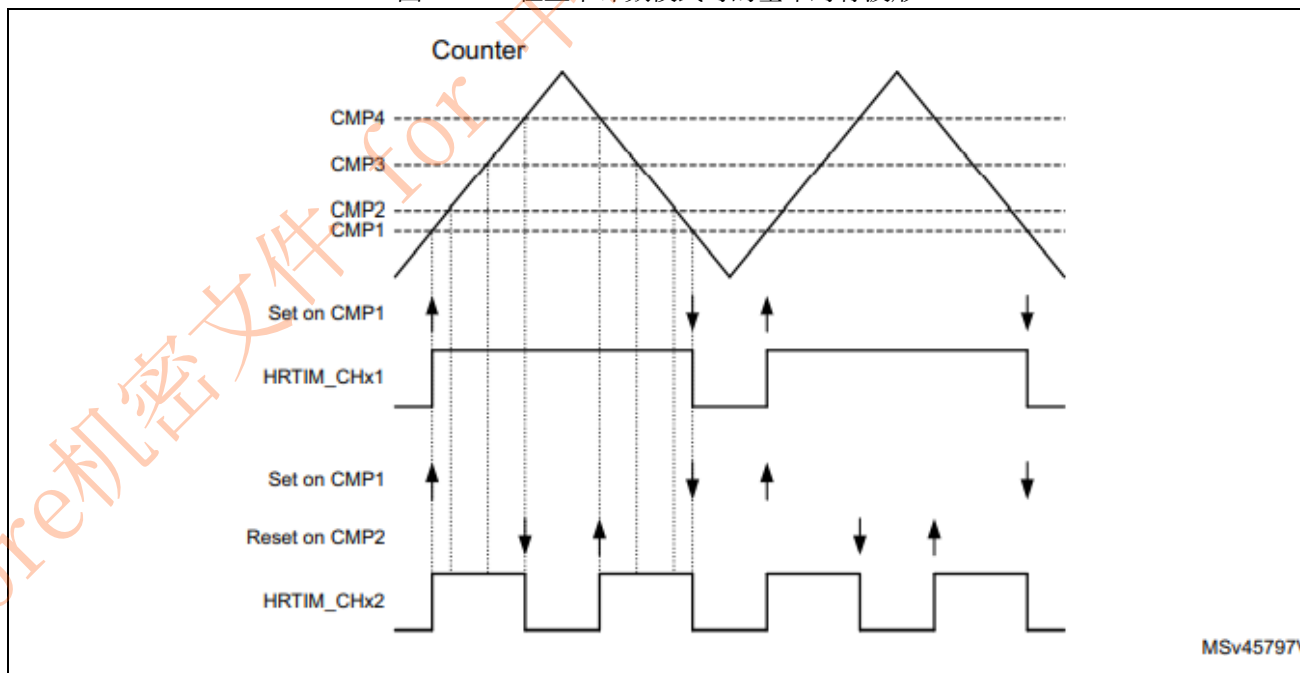
- 如果事件在 HRTIM\_SETxyR 寄存器中使能，则它在向上计数期间设置输出，在向下计数期间复位输出。
- 如果事件在 HRTIM\_RSTxyR 寄存器中使能，则它在向上计数期间复位输出，在向下计数期间设置输出。
- 如果事件同时在 HRTIM\_SETxyR 和 HRTIM\_RSTxyR 寄存器中使能，则输出会切换。

这适用于：

- 定时单元：周期、比较 1..4、寄存器更新（6 个事件）
- 主定时器：周期、比较 1..4、HRTIM 同步（6 个事件）
- 所有其他定时单元（例如，对于定时器 A 的定时器 B..F）：TIMEVNT1..9（表 218 中描述的 9 个事件）

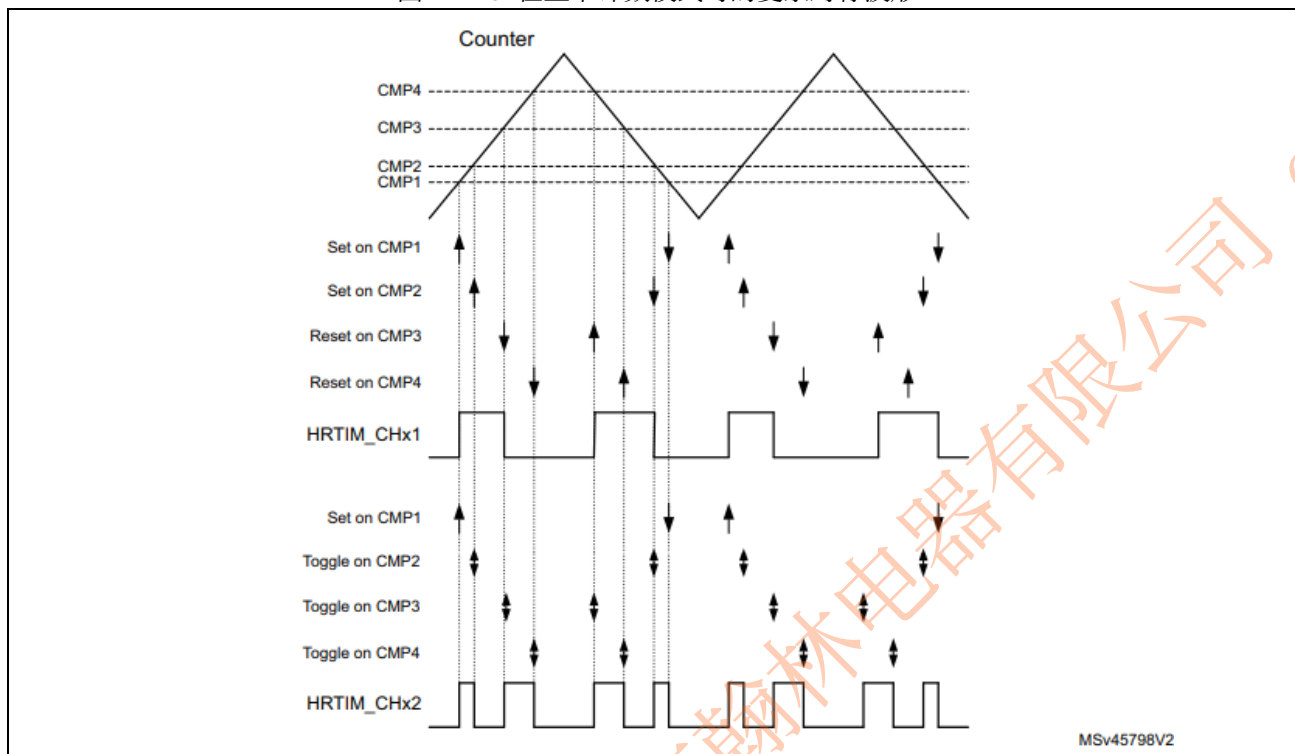
下图展示了如何生成基本波形。

图 22.24 在上下计数模式时的基本对称波形



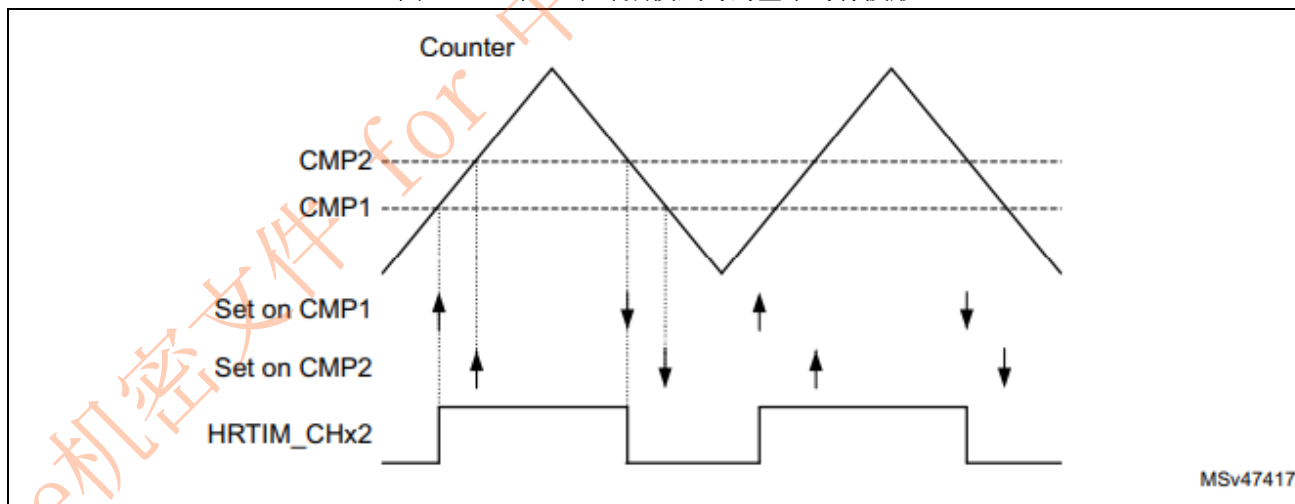
下图展示了如何使用四个可用的比较单元和切换模式来生成一些更复杂的波形。

图 22.25 在上下计数模式时的复杂对称波形



下图示了如何生成一个不对称波形。在这种情况下，需要注意的是，为了使波形不对称，比较单元 2 的值必须大于比较单元 1 的值。

图 22.26 在上下计数模式时的基本对称波形

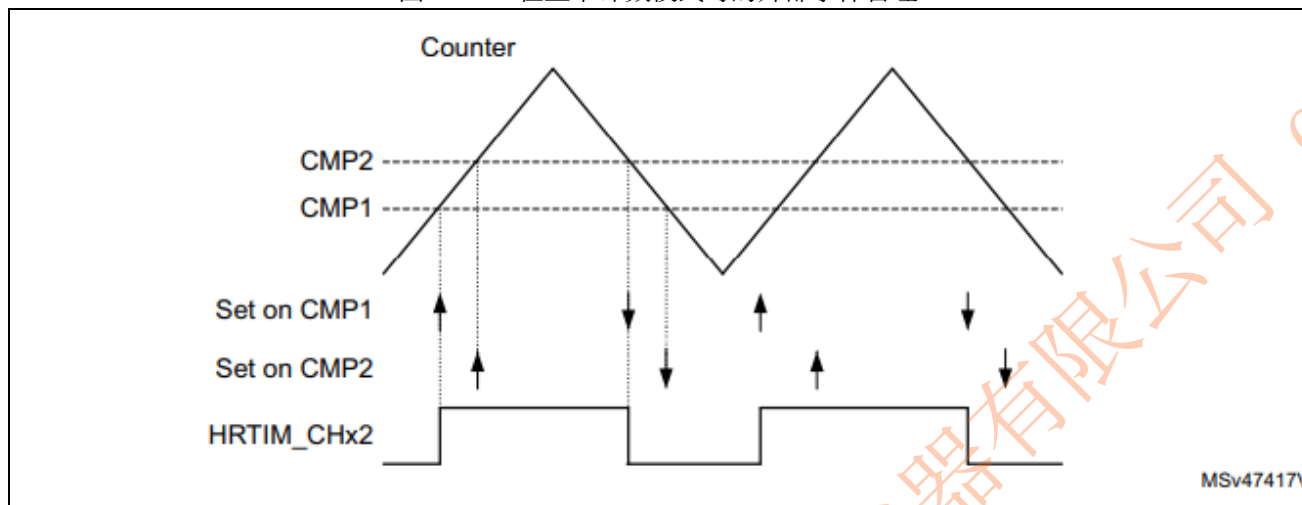


注：对于不对称操作，需要确保  $CMP2 > CMP1$ 。



软件强制位和外部事件 EXTEVNT1..10 的行为在仅上升计数模式和上下计数模式中是一致的。下图展示了如何响应外部事件来缩短脉冲宽度。

图 22.27 在上下计数模式时的外部事件管理



上下计数模式既可用于连续操作模式，也可用于单次触发（可重触发和非重触发）操作模式。在重置时，计数器会重新从 0 开始。下图展示了在单次可重触发模式下，定时器 B 的计数器行为。

图 22.28 纵横的上下计数操作

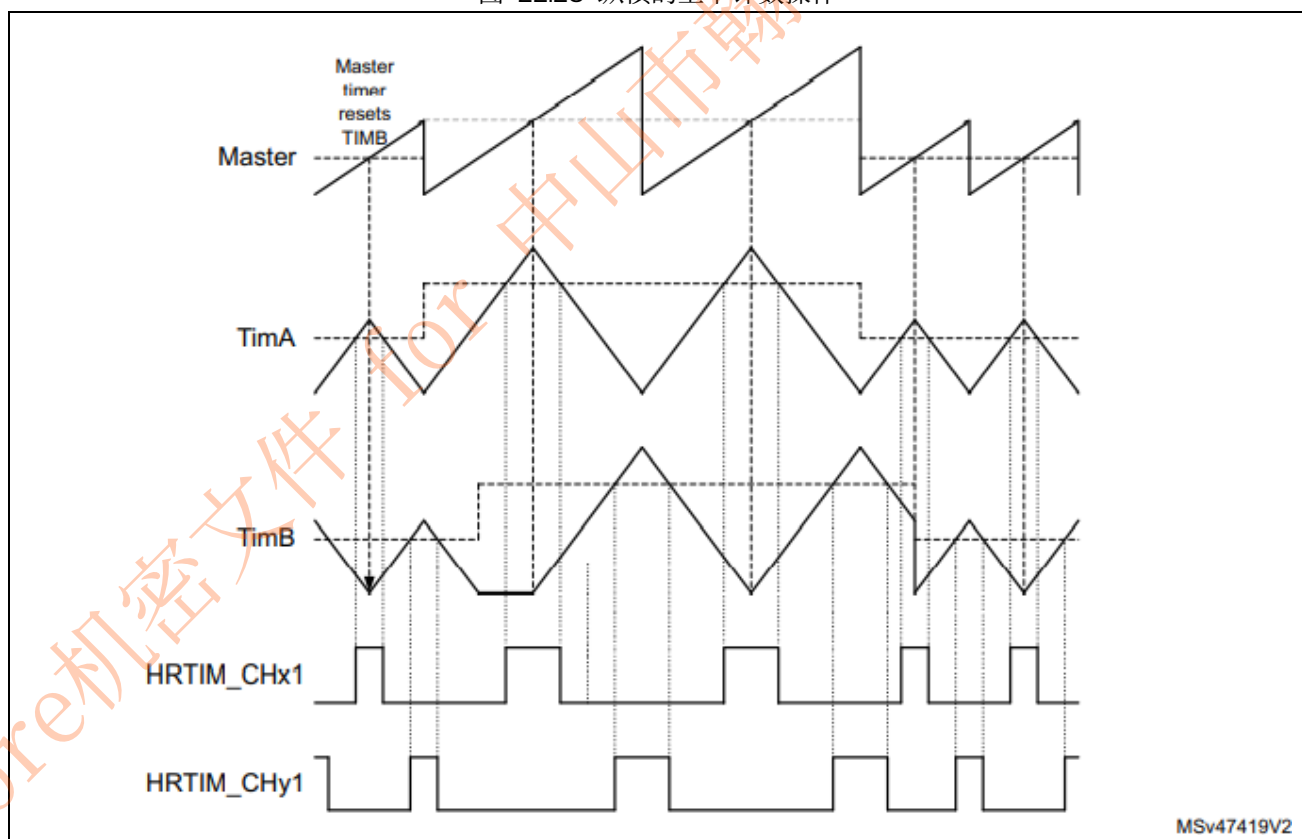
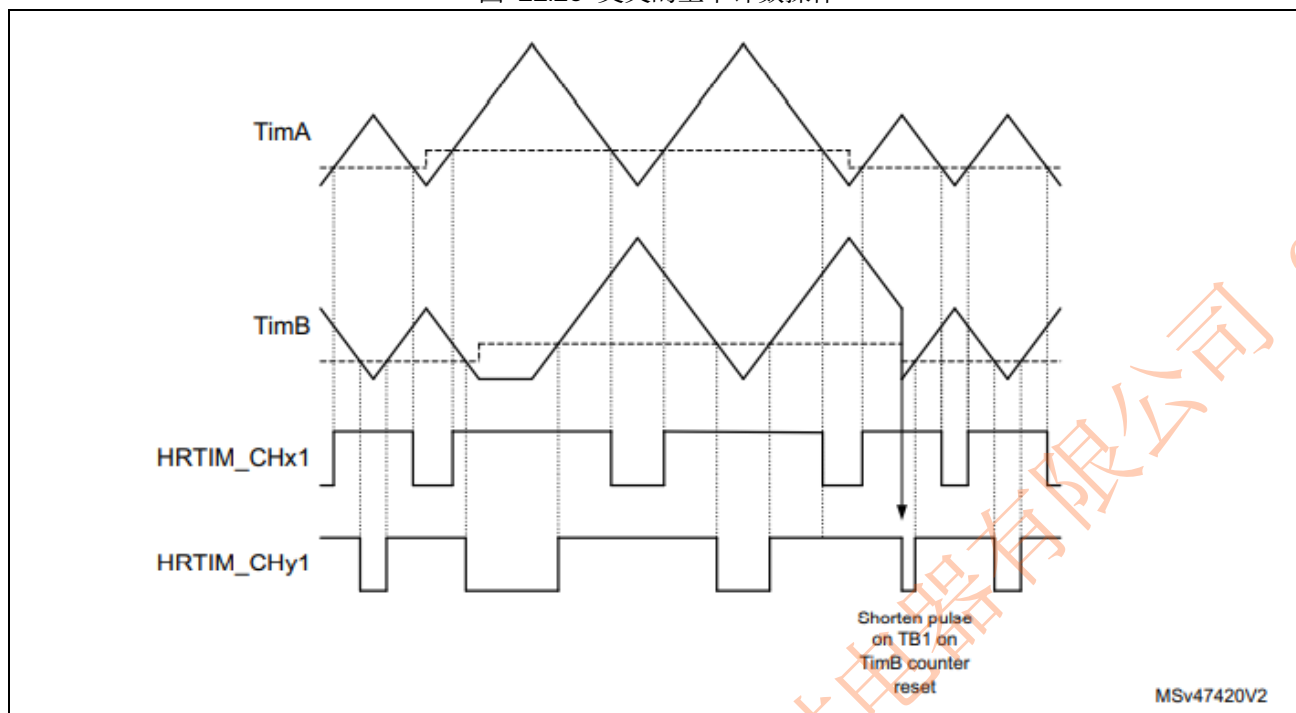


图 22.29 交叉的上下计数操作



MSv47420V2

注：在上下计数模式下，比较值必须低于周期值 ( $TIMx\_PER - 0xC0$  如果  $CKPSC[2:0] = 0$ ,  $TIMx\_PER - 0x60$  如果  $CKPSC[2:0] = 1$ ,  $TIMx\_PER - 0x30$  如果  $CKPSC[2:0] = 2$ , ...), 且至少为  $f_{HRTIM}$  时钟的3个周期。这适用于在定时单元内部生成的比较事件。对于在其他定时单元中生成的比较事件，必须避免在计数器方向改变（计数器为0、周期事件或计数器重置）的  $f_{HRTIM}$  时钟的1个周期之内发生任何事件。

上下计数模式支持以下功能：

- 半模式
- 死区时间插入
- 推挽模式，当计数器=0时交替进行推挽操作（见下图）。
- 延迟空闲
- 突发模式
- 使用“大于”比较的 PWM 模式（见下图）。

图 22.30 推挽上下模式样例

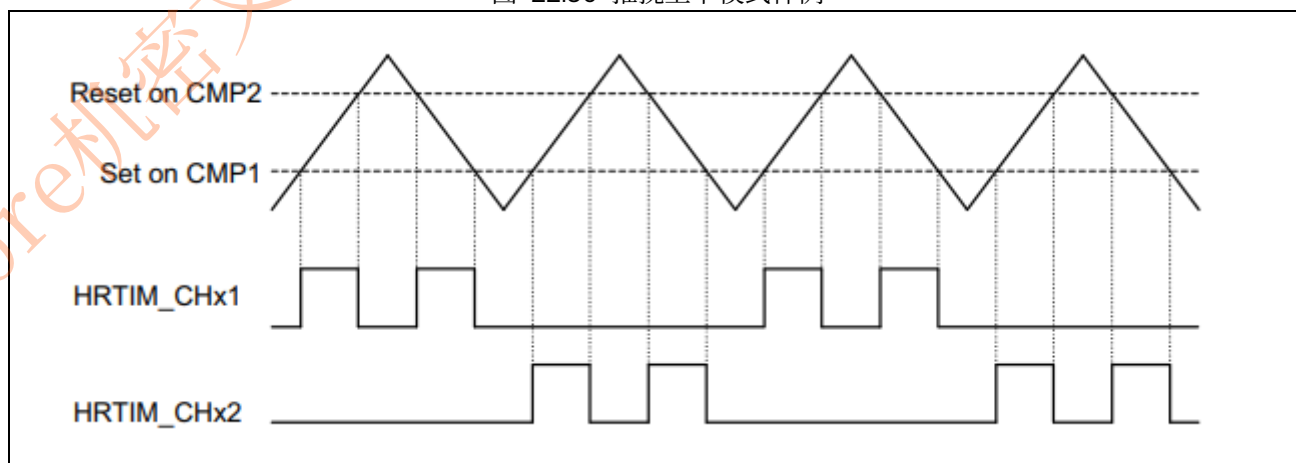
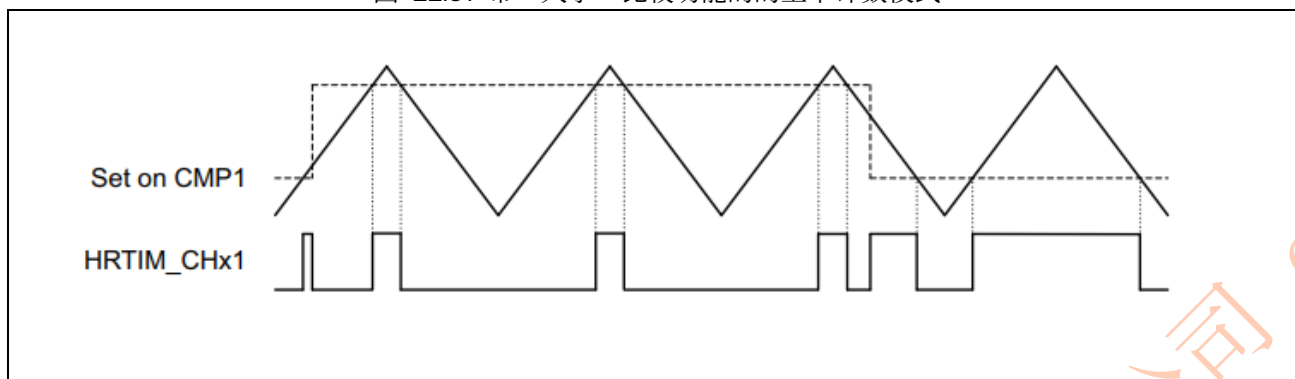


图 22.31 带“大于”比较功能的上下计数模式



警告：上下计数模式不支持以下功能：

- 自动延迟模式
- 平衡空闲
- 半触发模式

捕获功能在以下方面有所差异：

- 在上升计数时，捕获值是从计数开始计算的
- 在下降计数时，捕获值是以 PER 事件为参考的
- 捕获寄存器的第 16 位保存计数方向状态

在上下计数模式下，计数器翻转事件的定义不同，以支持各种操作条件。它可以在以下情况下生成：

- 当计数器等于 0 时（“谷值”模式）
- 当计数器达到在 HRTIM\_PERxR 中设置的周期值时（“峰值”模式）
- 当满足上述两个条件中的任何一个时（即 0 或 HRTIM\_PERxR 的值）

在 HRTIM 中，此事件用于多种目的。生成模式（谷值、峰值或两者）可以根据目标单独编程。下表总结了使用案例以及 HRTIM\_TIMxCR2 寄存器中与之相关的翻转模式（xxROM[1:0]）编程位。

表 22.14 翻转事件目标和模式编程

| 使用翻转事件                    | 编程位         |
|---------------------------|-------------|
| 输出设置/重置                   | OUTROM[1:0] |
| 注册内容更新触发器(从预加载转移到活动)      | ROM[1:0]    |
| IRQ 和/或 DMA 请求触发器         | ROM[1:0]    |
| 突发模式时钟源和/或突发启动触发器         | BMROM[1:0]  |
| ADC 触发器(详情请参阅 ADC 后缩放器部分) | ADROM[1:0]  |
| 外部事件过滤                    | ROM[1:0]    |
| 重复计数减量                    | ROM[1:0]    |
| 故障和事件计数器                  | FEROM[1:0]  |

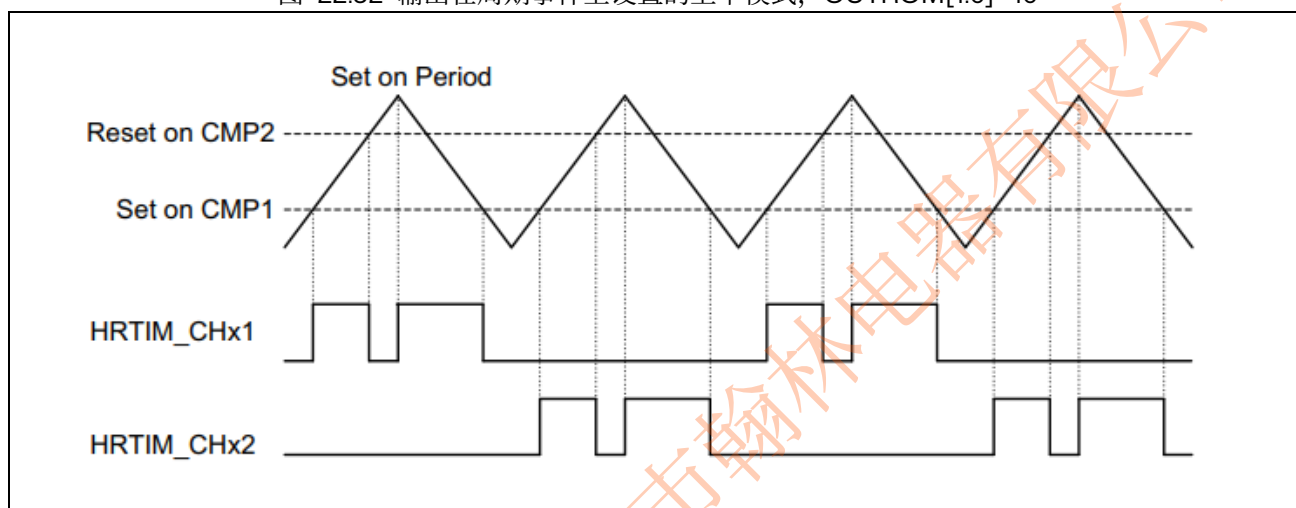
注：对于同时考虑重置和翻转的事件（IRQ/DMA 和 TxRSTU），ROM[1:0] 仅影响翻转的生成。无论 ROM[1:0] 的值如何，重置事件始终会被考虑。

翻转事件的生成定义如下，根据 xxROM[1:0] 位域的设置：

- xxROM[1:0] = 00：当满足两个条件中的任何一个时（即 0 或 HRTIM\_PERxR 的值）生成事件
- xxROM[1:0] = 01：当计数器等于 0 时（“谷值”模式）或计数器被重置时生成事件
- xxROM[1:0] = 10：当计数器达到在 HRTIM\_PERxR 中设置的周期值时（“峰值”模式）生成事件

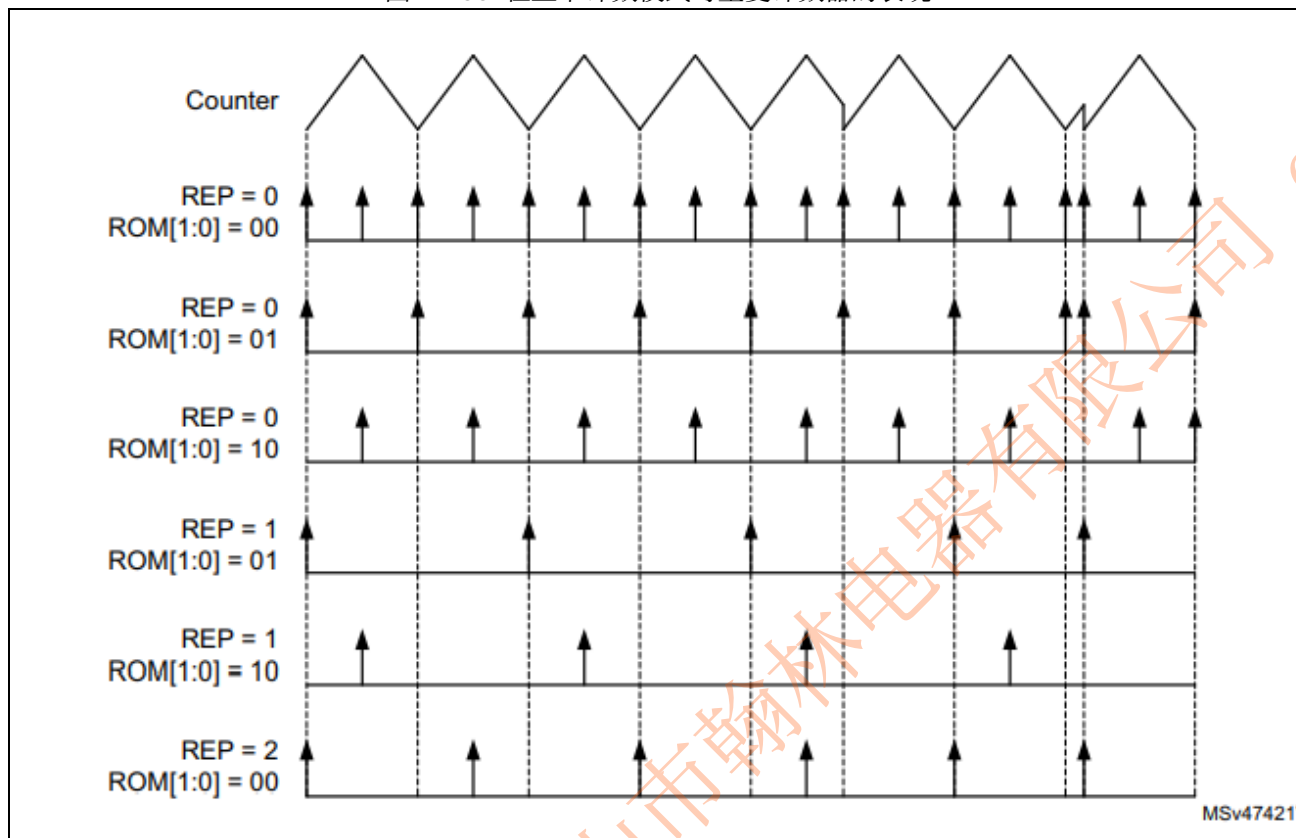
下图展示了具有周期事件设置和 OUTROM[1:0] = 10 的推挽上下模式。

图 22.32 输出在周期事件上设置的上下模式，OUTROM[1:0]=10



下图展示了在上下计数模式下重复计数器是如何递减的。

图 22.33 在上下计数模式时重复计数器的表现



双通道 DAC 触发器在上升计数模式下的工作方式相同。

事件消隐和窗口化功能有所不同，以便在输出脉冲内的可编程时间内执行消隐或窗口化操作。EExFLTR[3:0]编码取决于 UDM 位的设置，如下表所示。每当使用翻转事件进行消隐或窗口化时，ROM[1:0]编程适用于定义其生成时间。

表 22.15 EExFLTR[3:0]代码取决于 UDM 位设置

| EE1FLTR[3:0] | 向上计数模式<br>(UDM = 0)                | 向上向下计数模式<br>(UDM = 1)            |
|--------------|------------------------------------|----------------------------------|
| 0010         | 消隐从计数器复位/翻转<br>到比较 2               | 消隐从比较 1 到比较 2,<br>仅在向上计数阶段       |
| 0100         | 消隐从计数器复位/翻转<br>到比较 4               | 消隐从比较 3 到比较 4,<br>仅在向上计数阶段       |
| 1101         | 窗口化从计数器复位/翻转<br>到比较 2              | 窗口化从比较 2 到比较 3,<br>仅在向上计数阶段      |
| 1110         | 窗口化从计数器复位/翻转<br>到比较 3              | 窗口化从比较 2 到比较 3,<br>仅在向下计数阶段      |
| 1111         | 来自另一个定时单位的窗口化:<br>TIMWIN 源(详见后续表格) | 窗口化从向上计数阶段的<br>比较 2 到向下计数阶段的比较 3 |

### 22.3.7 置位/复位事件优先级和窄脉冲管理

本节介绍了在 3 个连续  $t_{HRTIM}$  周期内出现多个置位和/或复位请求时，输出波形是如何产生的。

#### 情况 1: 时钟预分频器 CKPSC[2:0] < 5

在每个  $t_{HRTIM}$  周期内，会执行一个仲裁过程，分三个步骤：

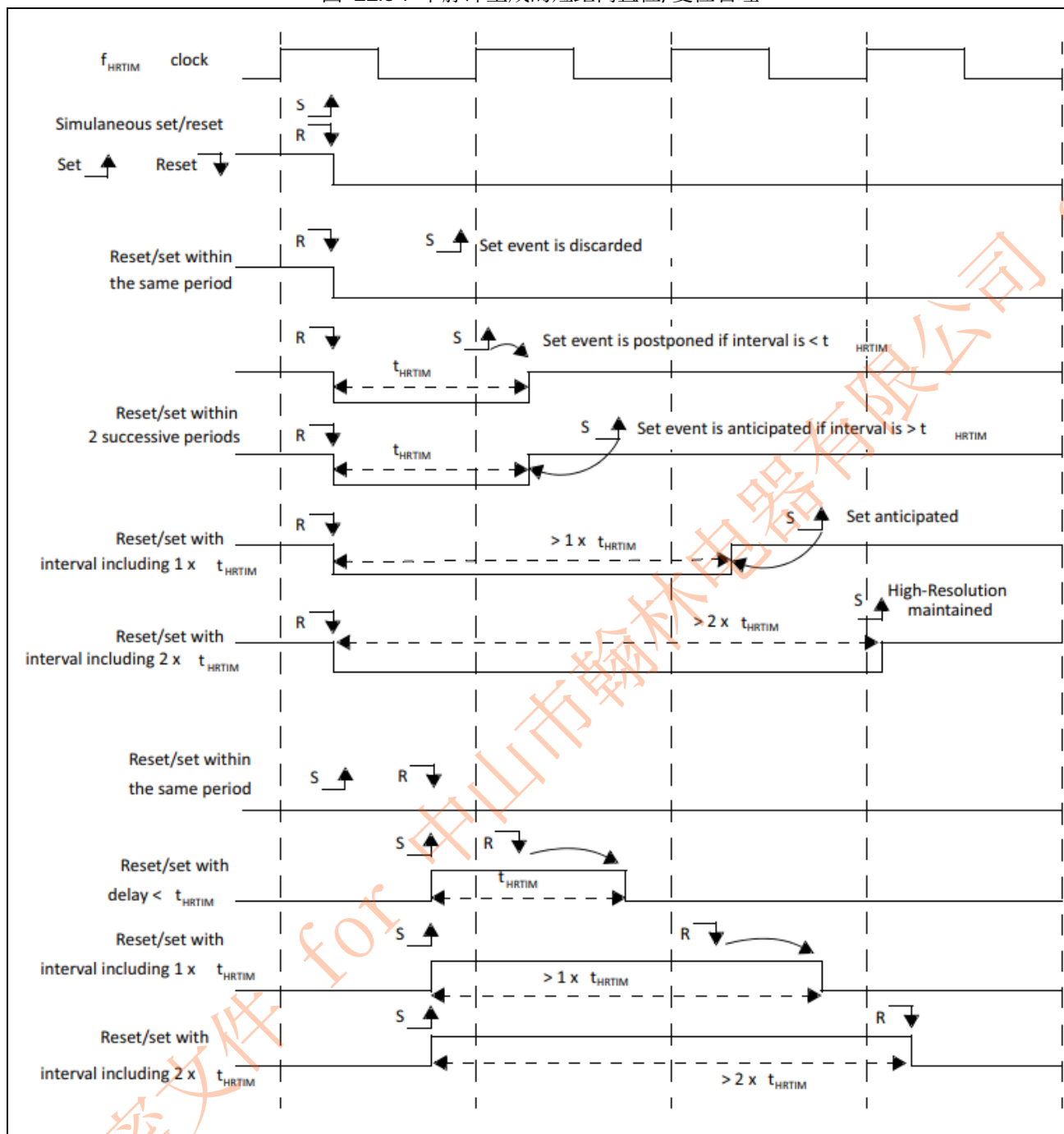
1. 对于每个有效事件，会确定所需输出跳变（置位、复位或切换）。
2. 在有效事件之间执行预定义的仲裁（从最高到最低优先级 CMP4 → CMP3 → CMP2 → CMP1 → PER，参见章节：并发置位请求/并发复位请求）。
3. 在低分辨率事件和已经赢得预定义仲裁的事件之间，执行基于高分辨率延迟的仲裁，其中复位具有最高优先级。

如果两个不同源发出的置位和复位请求是同时的，则复位操作的优先级最高。如果设置和重置请求之间的间隔小于 2 个  $f_{HRTIM}$  周期，则行为取决于时间间隔以及与  $f_{HRTIM}$  时钟的对齐情况，如下图所示。

注：如果置位和复位请求是同时发生的，并且来自同一个定时单元，则应用 CMPx 优先级，如上述步骤 2 所示。例如，假设 CMP2 = CMP4：

- 如果 CMP2 执行置位操作而 CMP4 执行复位操作，则输出被复位。
- 如果 CMP2 执行复位操作而 CMP4 执行置位操作，则输出被置位。

图 22.34 窄脉冲生成的短距离置位/复位管理



如果置位和复位事件是在同一个  $t_{HRTIM}$  周期内生成的，那么复位事件具有最高优先级，置位事件将被忽略。

如果置位和复位事件是在两个周期之间生成的，且间隔小于  $t_{HRTIM}$  周期，那么将生成一个 1 个  $t_{HRTIM}$  周期的脉冲。

如果置位和复位事件是在小于 2 个  $t_{HRTIM}$  周期的间隔内生成的，那么将生成一个 2 个  $t_{HRTIM}$  周期的脉冲。

如果置位和复位事件是在 2 到 3 个  $t_{HRTIM}$  周期之间的间隔内生成的，且间隔超过 2 个完整的  $t_{HRTIM}$  周期，那么可以使用高分辨率功能。



如果置位和复位事件是在大于 3 个  $t_{HRTIM}$  周期的间隔内生成的，那么总是可以使用高分辨率。

#### 并发置位请求/并发复位请求

如果为置位事件选择了多个源，则在同一  $f_{HRTIM}$  时钟周期内出现多个置位请求时，会执行仲裁。

如果相邻定时器 (TIMEVNT1..9) 发出多个请求，则会考虑最先发出的请求。仲裁分 2 步执行，具体取决于：

1. 源 (CMP4 → CMP3 → CMP2 → CMP1)，
2. 延迟。

如果主定时器在同一  $f_{HRTIM}$  时钟周期内发出多个请求，则会应用预定义的仲裁，并会考虑单个请求 (优先级从高到低)：

MSTCMP4 → MSTCMP3 → MSTCMP2 → MSTCMP1 → MSTCMPEP

注：建议避免从主定时器向给定的定时器生成多个置位 (复位) 请求，其间隔应小于  $3 \times t_{HRTIM}$ ，以保持高分辨率。

如果在同一  $f_{HRTIM}$  时钟周期内出现多个定时器内部请求，则会应用预定义的仲裁，并会按照以下优先级顺序考虑请求，而不考虑有效定时 (优先级从高到低)

CMP4 → CMP3 → CMP2 → CMP1 → PER

注：实际上，仅当同时使用自动延迟比较 2 和比较 4 时 (也就是由于有效置位/复位与外部事件相关联，无法通过优先级确定的情况下)，才要首先考虑这一点。在这种情况下，优先级最高的信号必须与 CMP4 事件相关联

最后，最高优先级会分配给非定时相关的事件：EXTEVNT1..10、RESYNC (来自 SYNC 事件 (SYNCRSTx 或 SYNCSTRTx 置 1 时) 或软件复位)、更新和软件置位 (SST)。更新事件被视为具有最大延迟 (如果 PSC = 0，则为  $0 \times 1F$ )。

总之，在事件紧密相邻 (在同一  $f_{HRTIM}$  时钟周期内发生) 的情况下，有效的置位 (复位) 事件将在以下事件之间进行仲裁：

- 任何 TIMEVNT1..9 事件
- 来自主定时器的单个源 (根据上面给出的固定仲裁)
- 来自定时器的单个源
- “低分辨率事件”

仲裁原则同样适用于并发复位请求。在这种情况下，复位请求的优先级最高。

#### **情况 2：时钟预分频器 CKPSC[2:0] ≥ 5**

当高分辨率无效时，窄脉冲管理得到简化。

在预分频器时钟周期内发生的置位或复位事件会延迟到预分频时钟的下一有效边沿 (针对计数器复位)，即使仍在每个  $t_{HRTIM}$  周期进行仲裁的情况也不例外。

如果同一预分频器时钟周期内发生复位事件后又发生置位事件，将考虑最新事件。

### 22.3.8 外部事件全局条件

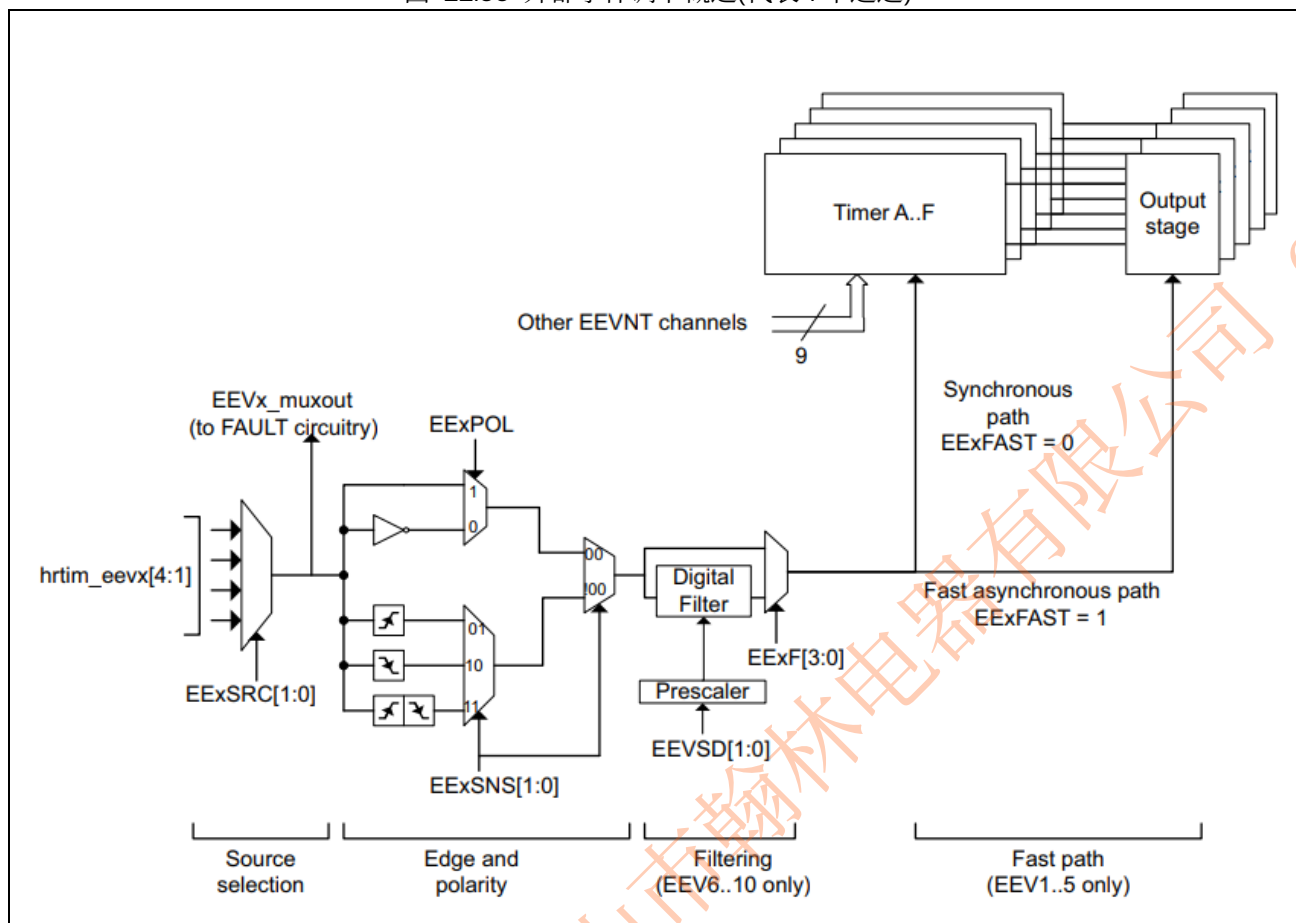
HRTIM 定时器可处理并非定时器中生成的事件，此类事件被称为“外部事件”。外部事件来自多个片上或片下源：

- 内置比较器
- 数字输入引脚（通常连接到片下比较器和过零检测器）
- 其他外设的片上事件（ADC 的模拟看门狗和通用定时器触发输出）。

外部事件调节电路可为给定通道选择信号源（使用 4:1 多路复用器），并可将信号源转换为可由纵横开关单元处理的信息（例如，通过在外部事件通道上检测到的下降沿来触发输出复位）。

最多可调节 10 个外部事件通道，这些外部事件通道可同时用于 6 个定时器中的任何一个。由于这种调节通常取决于外部组件（比如过零检测器）和环境条件（滤波器设置通常与应用噪声级和信号有关），因此对于所有定时器是通用的。下图显示了单条通道的调节逻辑概览。

图 22.35 外部事件调节概述(代表 1 个通道)



10 个外部事件是通过 HRTIM\_EECR1 和 HRTIM\_EECR2 寄存器进行初始化的：

- 使用 EExSRC[1:0] 位来选择最多 4 个源
- 使用 EExSNS[1:0] 位选择采用电平有效还是边沿有效（上升沿、下降沿或两种边沿）
- 如果选择采用电平有效，则使用 EExPOL 位选择极性
- 使用 EExFAST 位为外部事件 1 到 5 使用低延迟模式（见“外部事件延迟”一节）

注：即使 EESNS 位已复位（选择电平有效），用作复位、捕获、突发模式、ADC 触发和延迟保护的触发信号的外部事件也是边沿有效：如果 POL = 0，触发在外部事件上升沿有效；如果 POL = 1，触发在外部事件下降沿有效。

计数器禁止后（Tx Cen 位复位），会立即丢弃外部事件，以防止任何输出状态更改和计数器复位，但用作 ADC 触发信号的外部事件除外。

此外，还可以使用 HRTIM\_EECR3 寄存器中的 EExF[3:0] 位为外部事件 6 到 10 使能数字噪声滤波器。

数字滤波器由计数器组成，需要使用 N 个有效采样来验证输出跳变。如果输入值在计数器达到 N 值之前发生变化，计数器会复位，跳变会被丢弃（视为伪事件）。如果计数器达到 N，跳变被视为有效，并会作为正确的外部事件进行传输。

因此，数字滤波器会向正在进行滤波的外部事件添加延迟，延迟时长视采样时钟和滤波器长度（预期的有效采样数）而定。

采样时钟为  $f_{\text{HRTIM}}$  时钟或由  $f_{\text{HRTIM}}$  预分频而生成的特定时钟  $f_{\text{EEVS}}$ ，通过 HRTIM\_EECR3 寄存器中的 EEVSD[1:0] 位定义。

下表总结了与 10 个外部事件通道相关联的可用源和特性。

表 22.16 外部事件特征

| 外部事件通道           | 快速模式 | 数字滤波 | 平衡故障<br>定时器 A,B,C | 平衡故障<br>定时器 D,E,F |
|------------------|------|------|-------------------|-------------------|
| hrtim_eev1[4:1]  | 是    | -    | -                 | -                 |
| hrtim_eev2[4:1]  | 是    | -    | -                 | -                 |
| hrtim_eev3[4:1]  | 是    | -    | -                 | -                 |
| hrtim_eev4[4:1]  | 是    | -    | -                 | -                 |
| hrtim_eev5[4:1]  | 是    | -    | -                 | -                 |
| hrtim_eev6[4:1]  | -    | 是    | 是                 | -                 |
| hrtim_eev7[4:1]  | -    | 是    | 是                 | -                 |
| hrtim_eev8[4:1]  | -    | 是    | -                 | 是                 |
| hrtim_eev9[4:1]  | -    | 是    | -                 | 是                 |
| hrtim_eev10[4:1] | -    | 是    | -                 | -                 |

### 外部事件延迟

外部事件调节能够根据性能预期调整外部事件处理时间（以及关联的延迟）：

- 常规工作模式，在该模式下，会在对输出纵横开关进行操作之前，使用时钟对外部事件进行重新采样。此过程会增加一些延迟，但可以访问所有纵横开关功能，从而生成外部触发的高分辨率脉冲。
- 快速工作模式，在该模式下，外部事件与输出操作之间的延迟得到最大限度地缩短。该模式便于实现超快速过流保护等功能。

HRTIM\_EECR1 寄存器中的 EExFAST 位可用于定义通道 1 到 5 的工作模式。这会影响输出脉冲上存在的延迟和抖动，具体见下表。

表 22.17 输出置位/复位延迟和抖动与外部事件工作模式的关系

| EExFAST | 响应时间延迟                      | 响应时间抖动                  | 输出脉冲上的抖动<br>(计数器通过外部事件复位)                          |
|---------|-----------------------------|-------------------------|----------------------------------------------------|
| 0       | 5 到 6 个<br>$f_{HRTIM}$ 时钟周期 | 1 周期的<br>$f_{HRTIM}$ 时钟 | 无抖动，脉冲宽度保持在高分辨率                                    |
| 1       | 最小延迟<br>(取决于比较器还是数字输入)      | 最短延迟                    | 1 个 $f_{HRTIM}$ 时钟周期的抖动，<br>脉冲宽度分辨率可低至 $t_{HRTIM}$ |

当电平检测敏感时，设置 (EExSNS[1:0] = 00)，才能使用 EExFAST 模式；不能用于边沿检测敏感的设置。

可以对外部事件应用事件过滤（消隐和窗口，EExFLTR[3:0] != 0000）。在这种情况下，EExLTCHx 位必须复位：不支持延迟模式，也不支持窗口超时功能。

注：相关 EExFAST 位置 1 后，不得修改外部事件配置（源和极性）。

快速外部事件不能用于切换输出：必须在 HRTIM\_SETxyR 或 HRTIM\_RSTxyR 寄存器中使能，而不能在两个寄存器中同时使能。

如果置位事件和复位事件（来自 2 个独立的快速外部事件）同时发生，则复位事件在纵横开关中的优先级最高，输出变为无效状态。

如果 EExFAST 位置 1，则在外部事件发生后的 11 个  $f_{HRTIM}$  时钟周期内，输出都不能更改。

下两图给出了进行输出置位/复位和计数器复位时对外部事件的实际响应时间示例。

图 22.36 外部事件下降沿的延迟（计数器复位和输出置位）

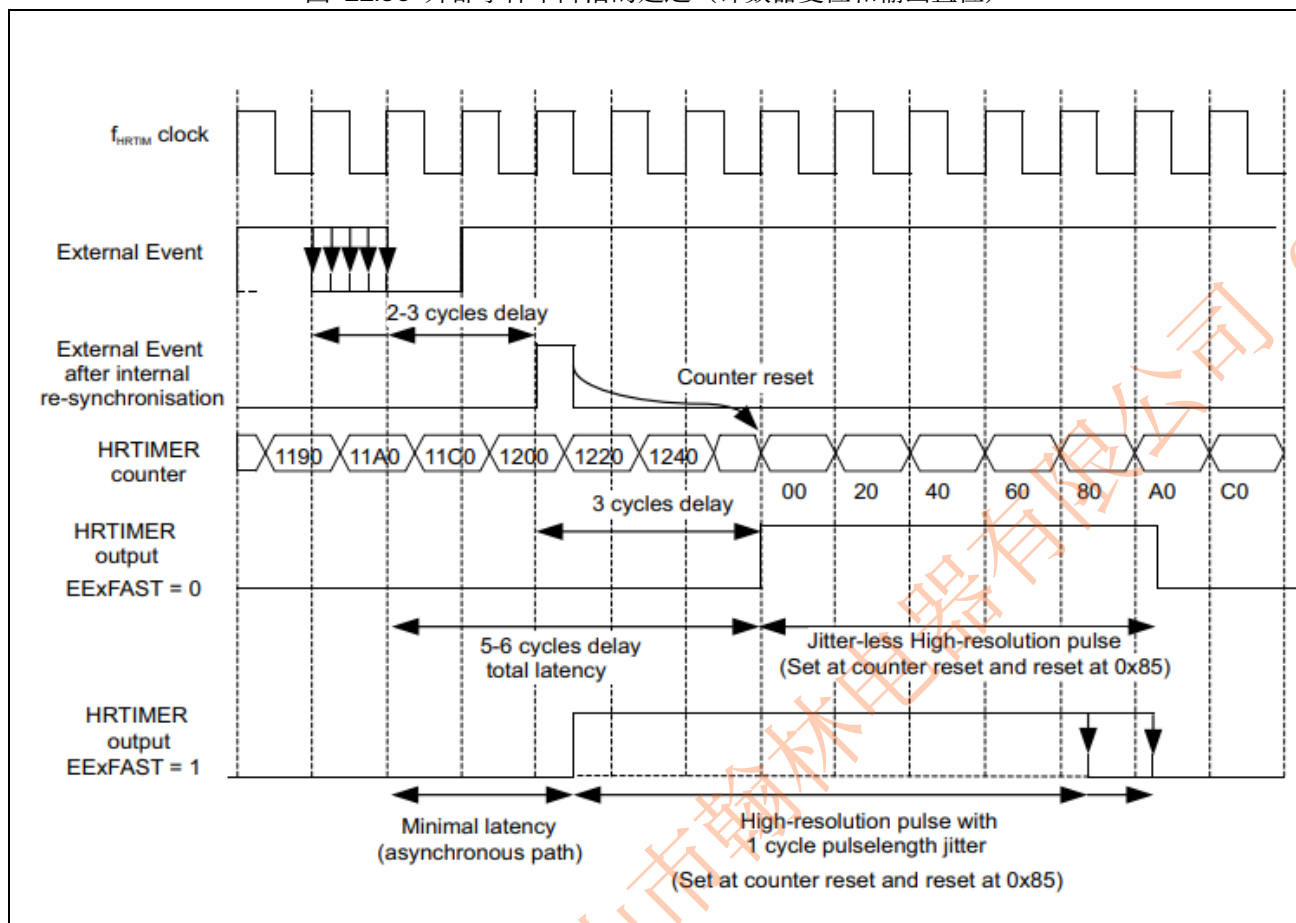
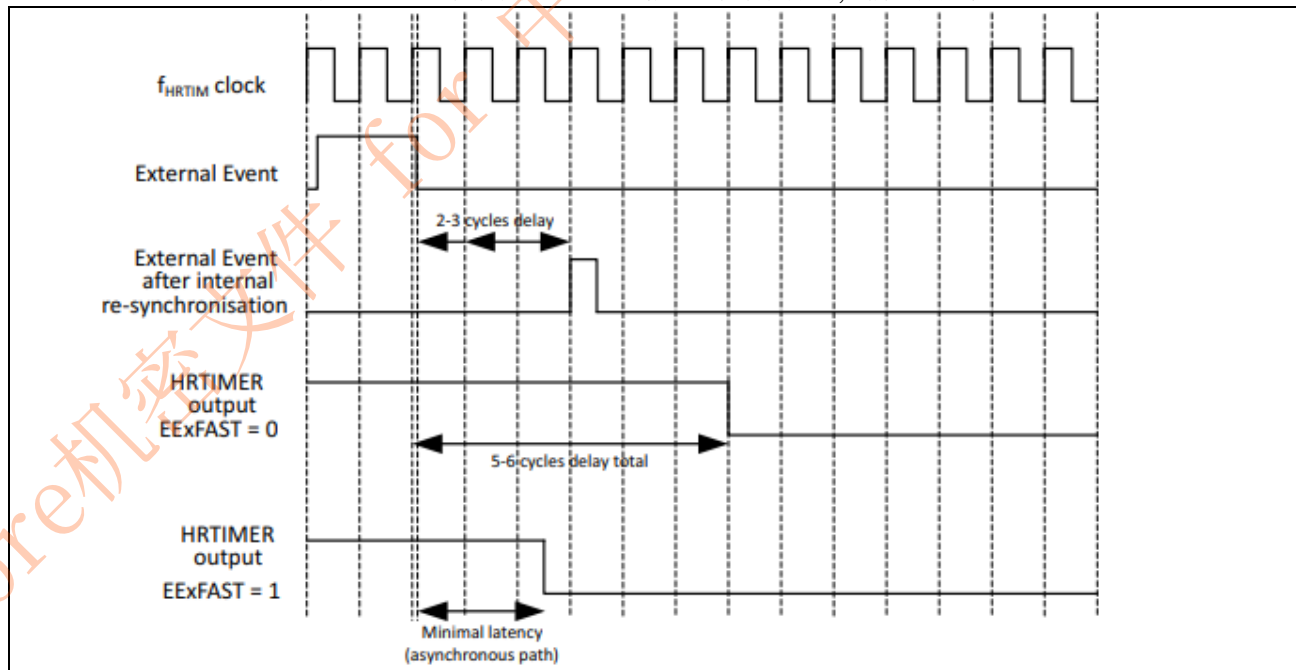


图 22.37 外部事件的延迟（发生外部事件时，输出复位）



### 22.3.9 定时单元中的外部事件过滤

经过调节后，有 10 个外部事件可供所有定时单元使用。

这些事件可直接使用，且在定时单元计数器使能后（Tx<sub>CEN</sub> 位置 1）立即生效。

此外，还可对这些事件进行过滤，以便在限制时间段内执行操作，该时间段通常与计数周期有关。可执行两种操作：

- 消隐模式，在定义的时间段内屏蔽外部事件
- 窗口模式，仅在定义的时间段内使能外部事件

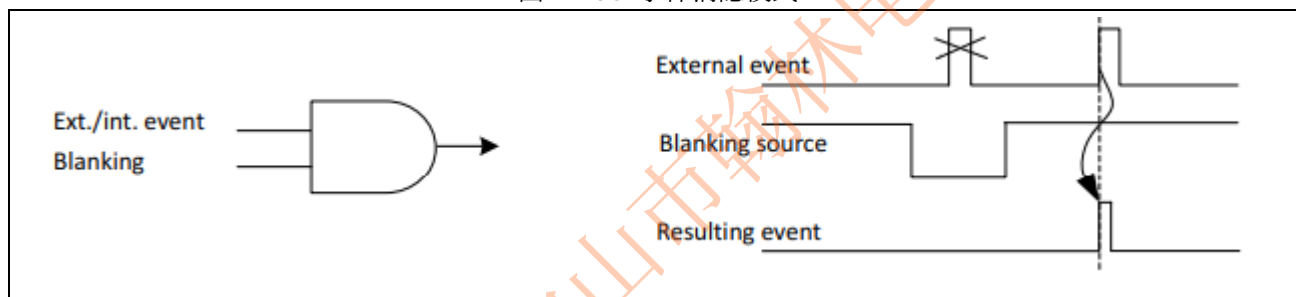
这些模式通过 HRTIM\_EEFxR1 和 HRTIM\_EEFxR2 寄存器中的 HRTIM\_EE<sub>x</sub>FLTR[3:0] 位来使能。

TimerA..F 这 6 个定时单元都具有针对这 10 个外部事件的可编程过滤器设置。

#### 消隐模式

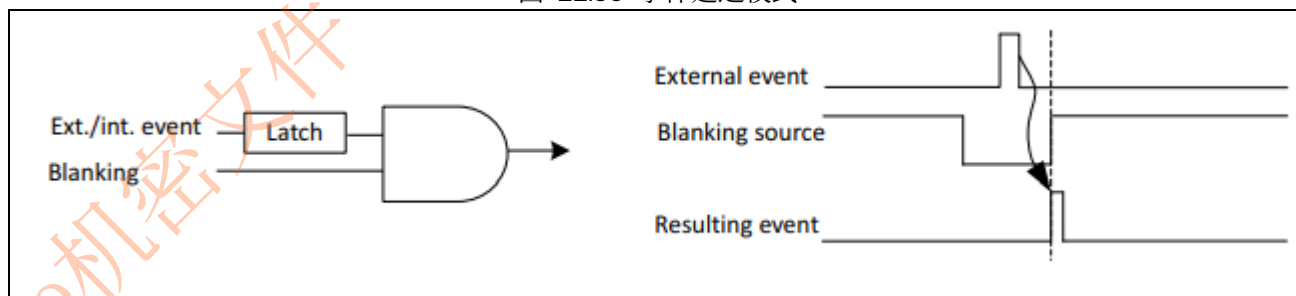
在事件消隐模式下（如下图所示），如果外部事件发生在给定的消隐周期内，则会被忽略。举例来说，为了避免 PWM 周期开始时电流限制因开关噪声而失效，可使用此模式。当 EE<sub>x</sub>FLTR[3:0] 位域的值在 0001 到 1100 范围内时，该模式有效。

图 22.38 事件消隐模式



在事件延迟模式下，不会立即考虑外部事件，而是会将其保存（锁存）下来，并在消隐周期结束后立即生成外部事件，如下图所示。可将 HRTIM\_EEFxR1 和 HRTIM\_EEFxR2 寄存器中的 EE<sub>x</sub>LATCH 位置 1 来使能此模式。

图 22.39 事件延迟模式





消隐信号来自多个源：

- 定时器本身：消隐持续时间从计数器复位开始，到比较匹配为止（对于比较1到比较4，EExFLTR[3:0] = 0001 到 0100）。在上下计数模式（UDM 位设置为 1）下，计数器复位事件根据 ROM[1:0]位的设置来确定。
- 来自其他定时单元（EExFLTR[3:0] = 0101 到 1100）：消隐持续时间从选定的定时单元计数器复位开始，到其中一个比较匹配为止，或者可完全编程为 Tx2 输出上的波形。在这种情况下，只要 Tx2 信号无效，事件就会被屏蔽（不需要使能输出，而且会在输出级之前获取信号）。

EExFLTR[3:0] 配置 0101 到 1100 在位说明中被称为 TIMFLTR1 到 TIMFLTR8，各定时单元的配置在位说明中的含义有所不同。下表给出了每个定时器的 8 个可用选项：CMPx 是指计数器复位到比较匹配期间进行消隐，Tx2 是指通过 HRTIM\_SETx2 和 HRTIM\_RSTx2 寄存器定义的定时单元 TIMx 输出 2 波形。举例来说，定时器 B（TIMFLTR6）是定时器 C 输出 2 波形。

表 22.18 每个定时器的过滤信号映射

| 源  | 定时器 A |      |      |     | 定时器 B |      |      |     | 定时器 C |      |      |     | 定时器 D |      |      |     | 定时器 E |      |      |     | 定时器 F |      |      |     |
|----|-------|------|------|-----|-------|------|------|-----|-------|------|------|-----|-------|------|------|-----|-------|------|------|-----|-------|------|------|-----|
|    | CMP1  | CMP2 | CMP4 | TA2 | CMP1  | CMP2 | CMP4 | TB2 | CMP1  | CMP2 | CMP4 | TC2 | CMP1  | CMP2 | CMP4 | TD2 | CMP1  | CMP2 | CMP4 | TE2 | CMP1  | CMP2 | CMP4 | TF2 |
| 目标 | TA    | -    | -    | -   | 1     | -    | 2    | 3   | 4     | -    | 5    | -   | 7     | -    | -    | -   | -     | 8    | -    | -   | 6     | -    | -    | -   |
|    | TB    | 1    | -    | 2   | 3     | -    | -    | -   | 4     | 5    | -    | -   | -     | 7    | -    | -   | 8     | -    | -    | -   | -     | 6    | -    | -   |
|    | TC    | -    | 1    | -   | -     | 2    | -    | 3   | -     | -    | -    | -   | 5     | -    | 6    | 7   | -     | -    | 8    | -   | 4     | -    | -    | -   |
|    | TD    | 1    | -    | -   | -     | -    | 2    | -   | -     | 3    | 4    | -   | 5     | -    | -    | -   | 6     | -    | 7    | -   | -     | -    | 8    | -   |
|    | TE    | -    | 1    | -   | -     | 2    | -    | -   | -     | 3    | -    | -   | -     | 6    | -    | 7   | 8     | -    | -    | -   | -     | -    | 4    | 5   |
|    | TF    | -    | -    | 1   | -     | -    | 2    | -   | -     | -    | -    | 3   | -     | -    | 4    | 5   | -     | 6    | -    | 7   | 8     | -    | -    | -   |

下两图举例说明了常规模式和延迟模式下所有边沿有效和电平有效的外部事件消隐。

图 22.40 采用边沿有效触发的外部触发消隐

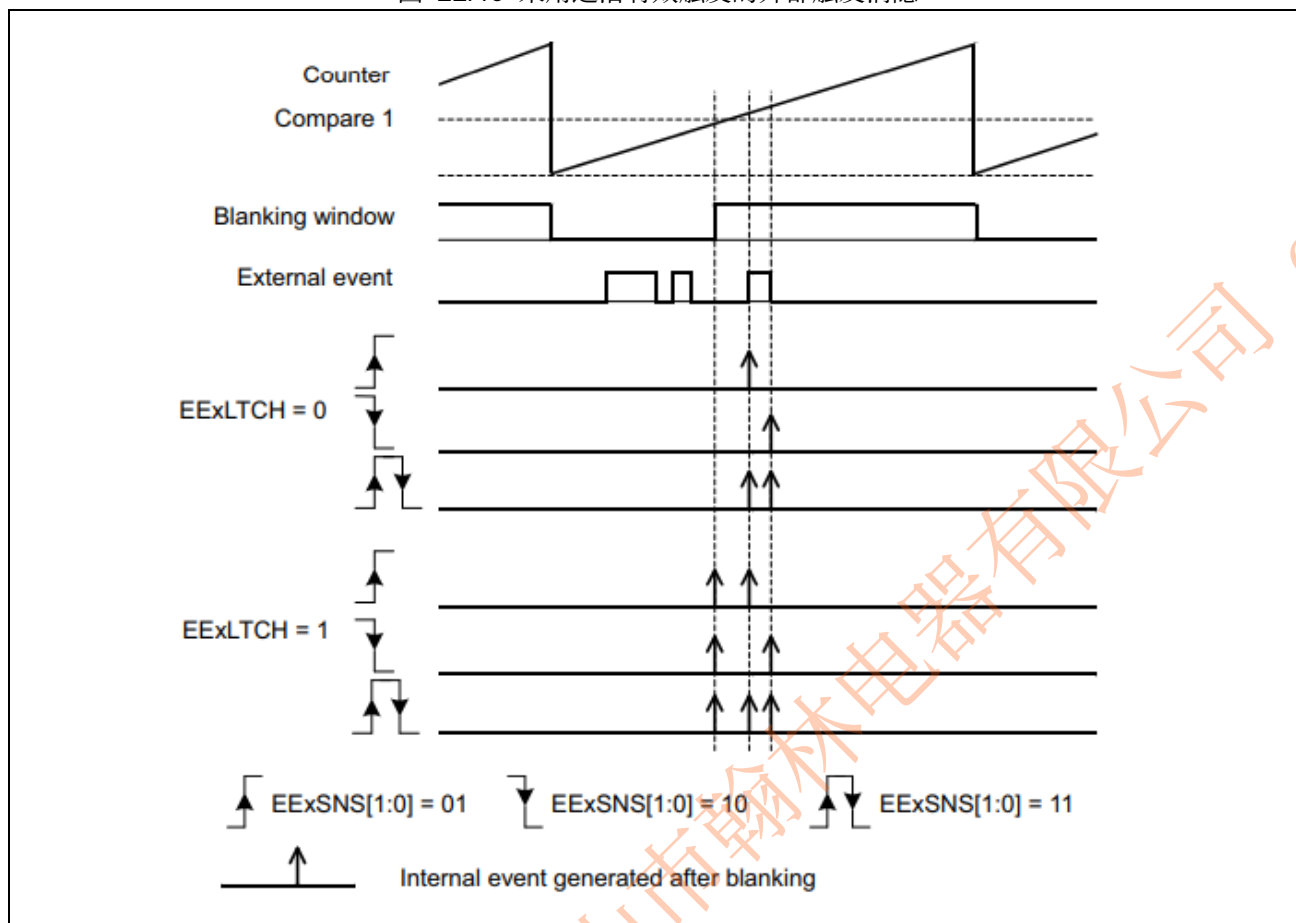
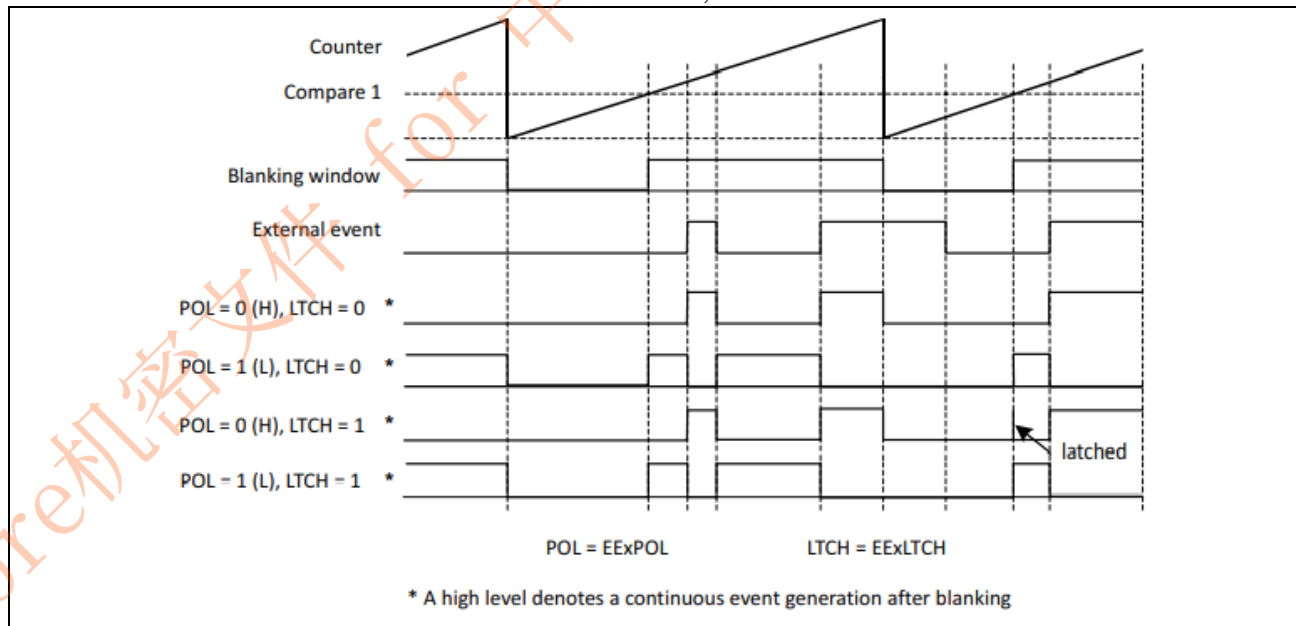


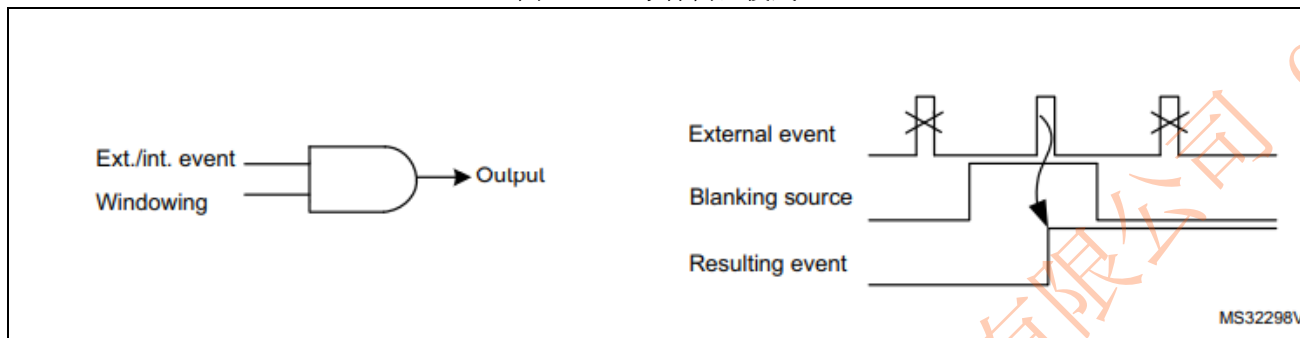
图 22.41 外部触发消隐，电平有效触发



## 窗口模式

在事件窗口模式下，仅当事件在给定事件窗口内发生时才会被考虑，否则会被忽略。当 EExFLTR[3:0] 的值在 1101 到 1111 范围内时，该模式有效。

图 22.42 事件窗口模式



EEFxR1 和 EEFxR2 寄存器中的 EExLTCH 可锁存信号，如果该位置 1：在这种情况下，如果事件在窗口期间发生，但在窗口结束时延迟，也会被接受。

- 如果 EExLTCH 位复位，且信号在窗口期间发生，则信号会直接通过。
- 如果 EExLTCH 位复位，并且无信号发生，则会在窗口结束时生成超时事件。

窗口模式可用于过滤同步信号。当缺少预期的同步事件时（例如在转换器启动期间），可通过超时生成功能强制生成默认同步事件。

每个外部事件窗口有 3 个源，其编码如下：

- 1101 和 1110：窗口持续时间从计数器复位开始，到比较匹配为止（分别是比较 2 和比较 3）。在上下计数模式（UDM 位设置为 1）下，计数器复位事件根据 ROM[1:0]位的设置来确定。
- 1111：窗口与另一定时单元相关，持续时间从计数器复位开始，到其比较 2 匹配为止。源被称为 TIMWIN 的位中说明，请参见下表。举例来说，定时器 B 中的外部事件可通过从定时器 A 计数器复位开始、到定时器 A 比较 2 为止的窗口进行过滤。

表 22.19 每个定时器的窗口信号映射 (EEFLTR[3:0] = 1111)

| 目标            | 定时器 A         | 定时器 B         | 定时器 C         | 定时器 D         | 定时器 E         | 定时器 F         |
|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
| TIMWIN<br>(源) | 定时器 B<br>CMP2 | 定时器 A<br>CMP2 | 定时器 D<br>CMP2 | 定时器 C<br>CMP2 | 定时器 F<br>CMP2 | 定时器 E<br>CMP2 |

注：如果外部事件是在快速模式下编程的，则不支持生成超时事件

下两图介绍了如何根据 EExLTCH 位的设置生成事件，以实现各种边沿有效和电平有效触发。为了便于理解，专门针对超时事件进行了说明。

图 22.43 采用边沿有效触发的外部触发窗口

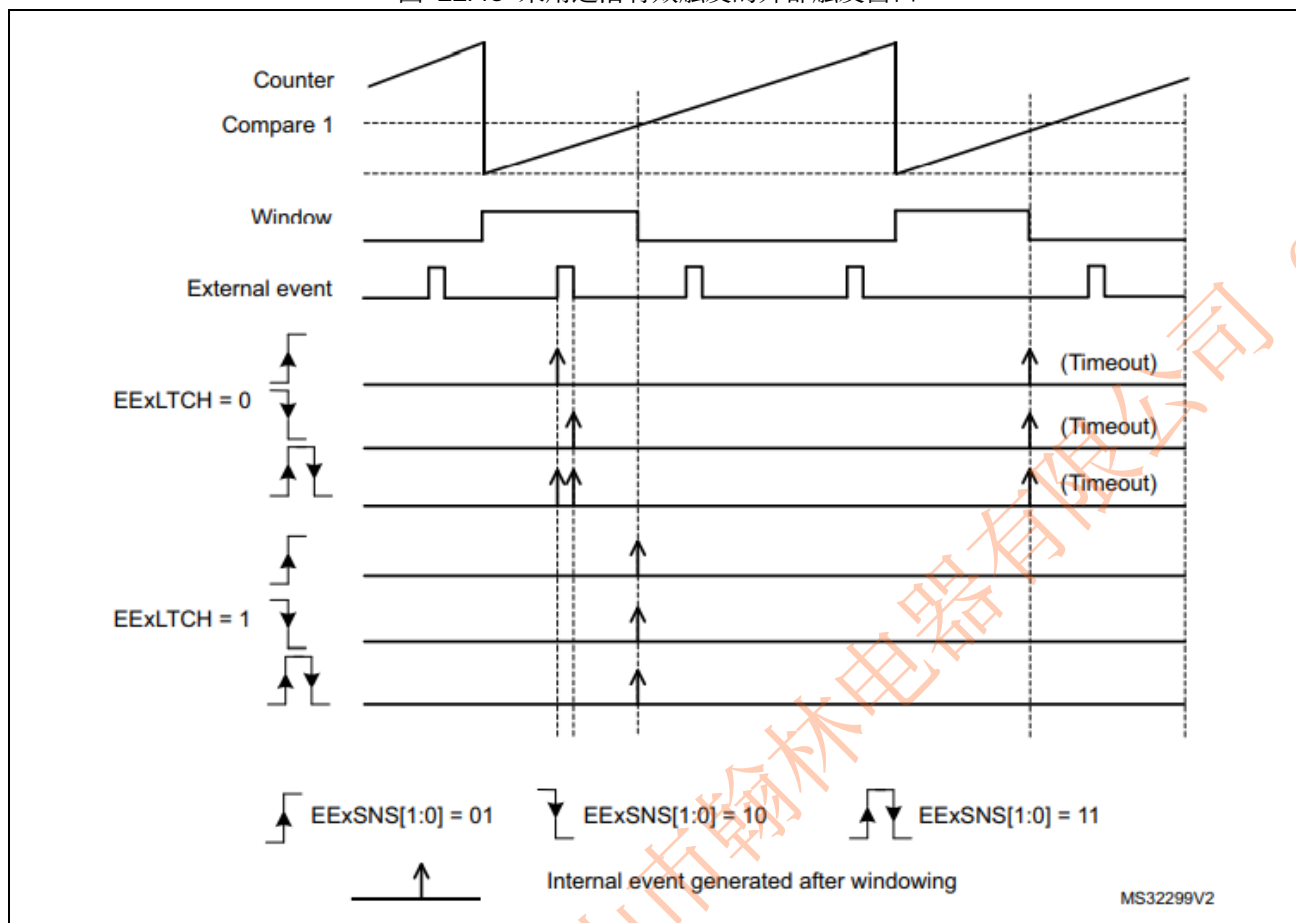
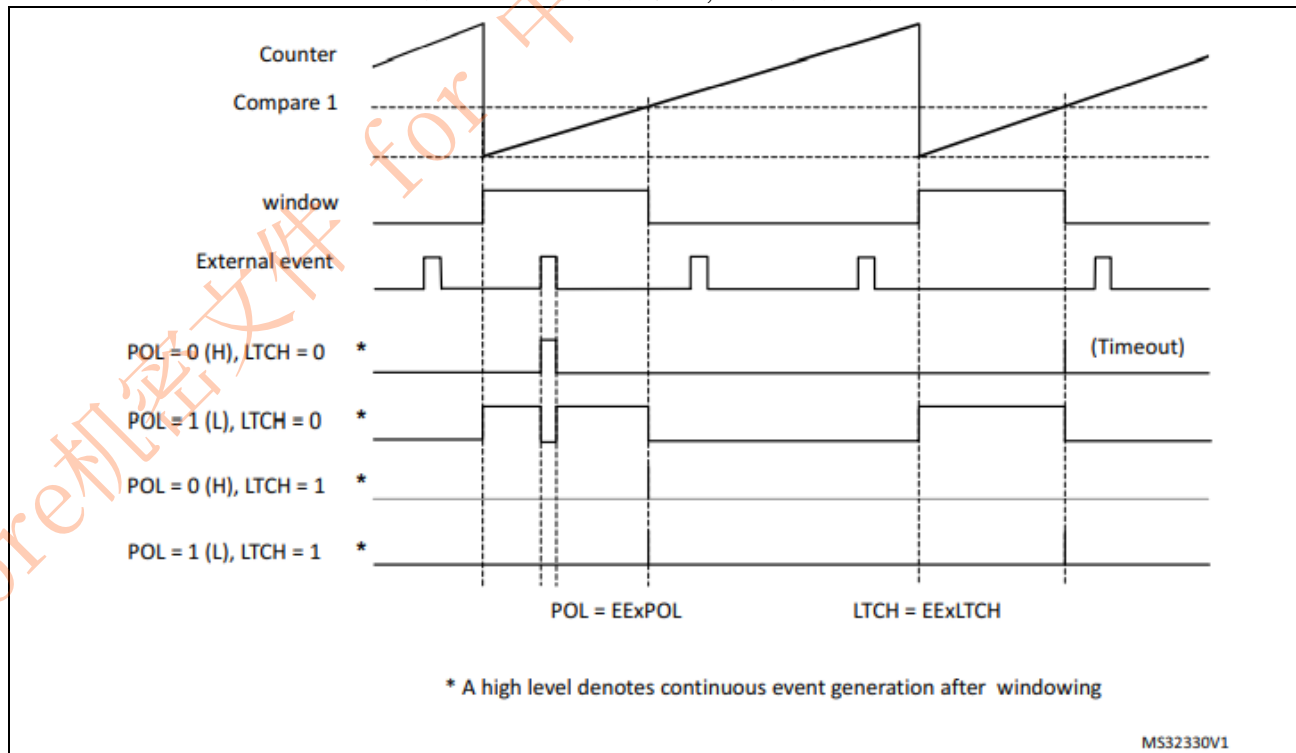


图 22.44 外部触发窗口，电平有效触发

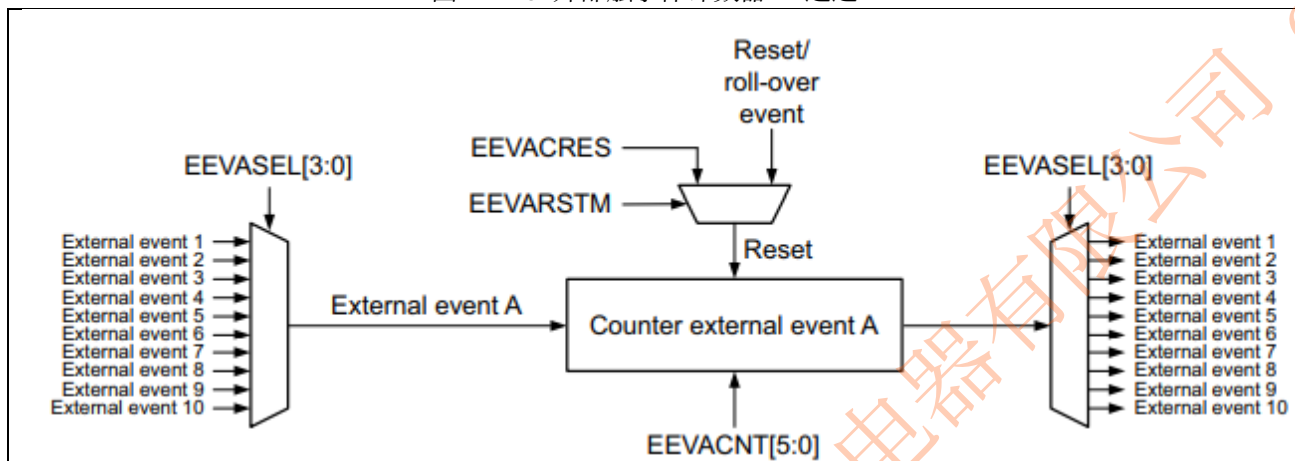


## 外部事件计数器

每个定时单元还在滤波单元后面配备了一个外部事件计数器，通常用于实现谷底跳过（Valley Skipping）功能。

该电路允许对经过滤波的 10 个外部事件中的任何一个进行进一步滤波，如下图所示。

图 22.45 外部触事件计数器 - 通道 A



计数器通过 HRTIM\_EEFxR3 寄存器中的 EEVACE 位使能。此模式仅对边沿敏感的外部事件有效（EEASNS[1:0]位=01、10 或 11）。

只有在有效边沿的数量大于或等于（EEVACNT[5:0]+1）中编程的值时，外部事件才会传播到定时器。

有两种操作模式可供选择：

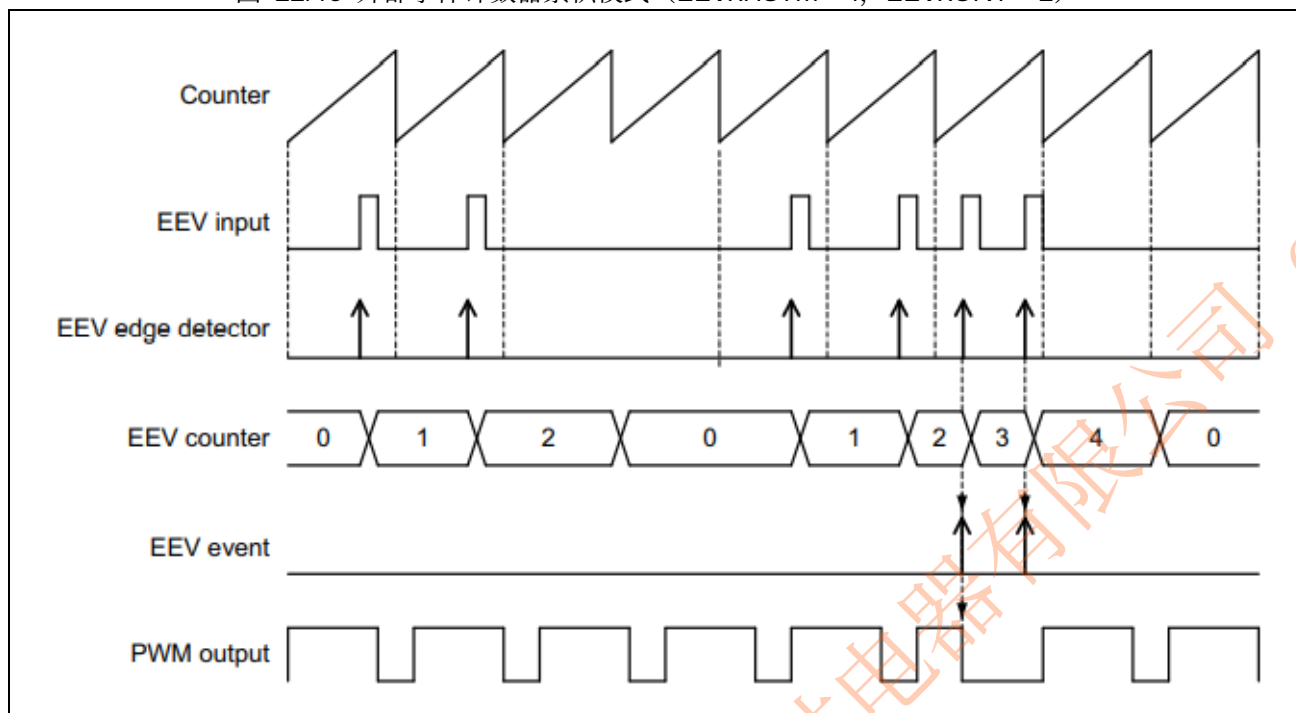
- 当 EEVARSTM 位被重置时，外部事件计数器在每个复位/溢出事件上重置：外部事件仅在给定 PWM 周期内出现多次时才有效。
- 当 EEVARSTM 位被设置时，外部事件计数器仅在事件在最后一个 PWM 周期内没有出现时重置。这是一种累积模式，事件必须在多个 PWM 周期内至少出现一次，如下图所示。

在编程计数器值之后，必须使能外部事件计数器（在设置 EEVACNT[5:0]位之后，必须设置 EEVACE 位）。

一旦计数器被使能，EEVACNT[5:0]位就可以在任何时候实时更改。新值将根据 EEVARSTM 位的编程在下一个复位/溢出事件时生效，或者在软件复位（EEVACRES 位设置）后生效。

一旦设置了 EEVACE 位，就不应再修改 EEVASEL[3:0]位。

图 22.46 外部事件计数器累积模式 (EEVxRSTM = 1, EEVxCNT = 2)



### 22.3.10 延迟保护

如果需要在有效脉冲结束后或推挽周期结束后以延迟的方式关断 PWM 输出，HRTIM 通常会为谐振转换器提供特定的保护机制。这些功能通过 HRTIM\_OUTxR 寄存器中的 DLYPRTEN 位使能，并将使用特定的外部事件通道。

#### 延迟空闲模式

在该模式下，有效电平会在保护激活之前结束。所选外部事件会使输出在有效脉冲结束时进入空闲模式（由 HRTIM\_RSTx1R 或 HRTIM\_RSTx2R 中的输出复位事件定义）

一旦保护被触发，会永久保持空闲模式，但计数器会继续工作，直至输出重新使能。Tx1OEN 和 Tx2OEN 位不受延迟空闲模式进入的影响。要退出延迟空闲模式并恢复操作，需要将 Tx1OEN 和 Tx2OEN 位重写为 1。输出状态将在发出输出使能命令后第一次跳变为有效状态时更改。

注：在有效脉冲结束之前，进入了延迟空闲模式便不能立即退出：务必先确保输出处于空闲状态，然后再重新开始运行模式。具体做法可以是等到下一周期，或者轮询 TIMxISR 寄存器中的 O1CPY 和/或 O2CPY 状态位。

延迟空闲模式可应用于单路输出 (DLYPRT[2:0] = x00 或 x01)，也可应用于两路输出 (DLYPRT[2:0] = x10)。

进入响应延迟空闲模式后，可通过生成中断或 DMA 请求以作出响应。外部事件到达后，会立即将 HRTIM\_TIMxISR 中的 DLYPRT 标志置 1，和输出上的有效脉冲是否结束无关。

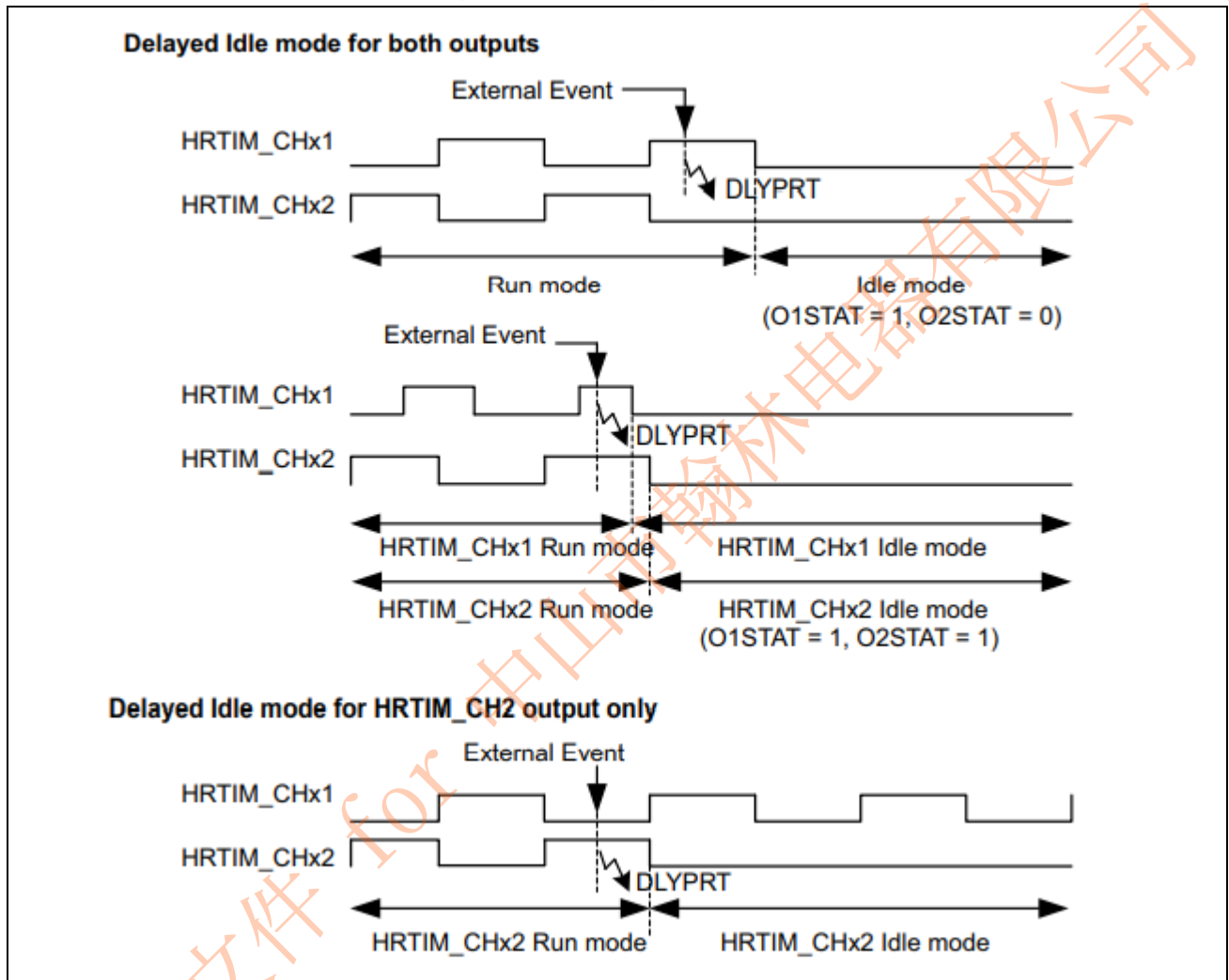
延迟空闲模式触发后，可通过 HRTIM\_TIMxISR 中的 O1STAT 和 O2STAT 确定输出状态。即使延迟空闲模式应用于单路输出，也会更新这两个状态位。如果使能了推挽模式，HRTIM\_TIMxISR 中的 IPPSTAT 标志会指示延迟保护请求在哪一周发生。

无论定时器采用何种工作模式（常规、推挽、死区），均可使用此模式。此模式仅支持 2 个外部事件：

- hrtim\_evt6 和 hrtim\_evt7（对于定时器 A、B 和 C）
- hrtim\_evt8 和 hrtim\_evt9（对于定时器 D、E 和 F）

仅当计数器使能后（TxCEN 位置 1），才能触发延迟保护模式。即使 TxEN 位已复位，在 TxyOEN 位置 1 之前，延迟保护模式仍保持有效状态。

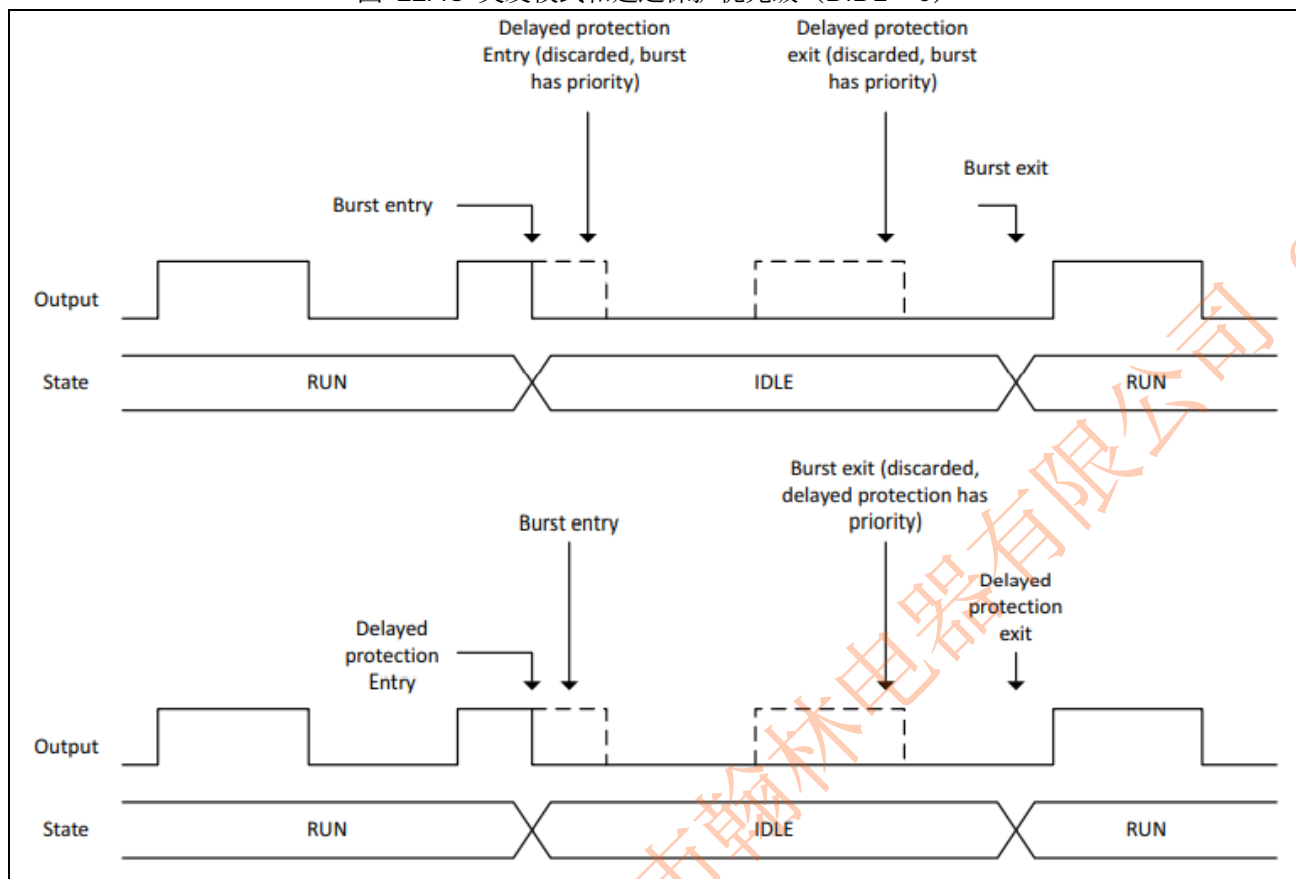
图 22.47 延迟空闲模式进入



延迟空闲模式的优先级要高于突发模式：一旦触发了延迟空闲保护，就会丢弃任何突发模式退出请求。相反，如果在突发模式有效时退出延迟保护，突发模式将正常恢复，输出将保持为空闲状态，直至退出突发模式。下图给出了这些不同情况的概览。

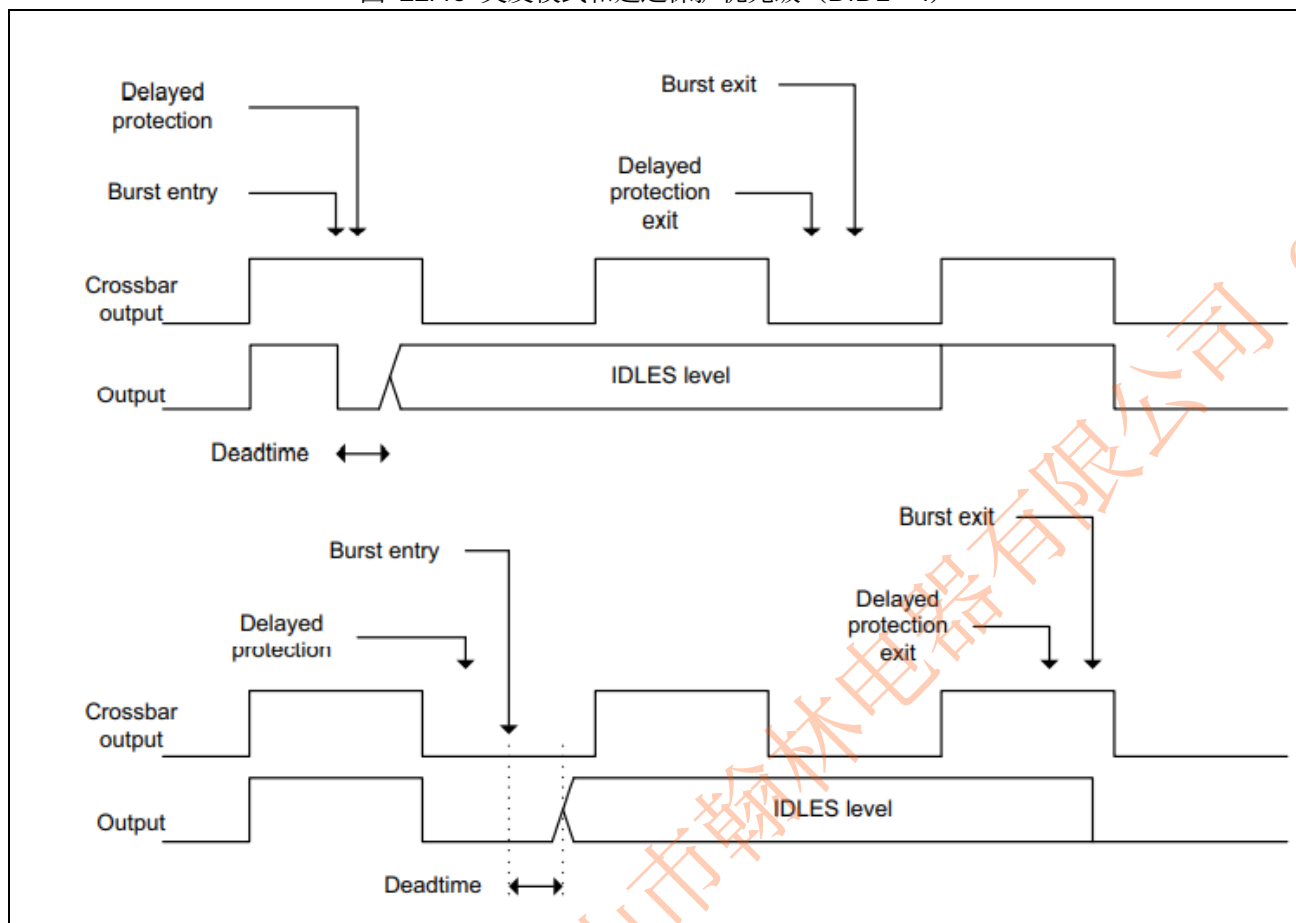


图 22.48 突发模式和延迟保护优先级 (DIDL = 0)



如果使能了延迟突发模式进入 (DIDL 位置 1)，则会应用相同的优先级，如下图所示。

图 22.49 突发模式和延迟保护优先级 (DIDL = 1)



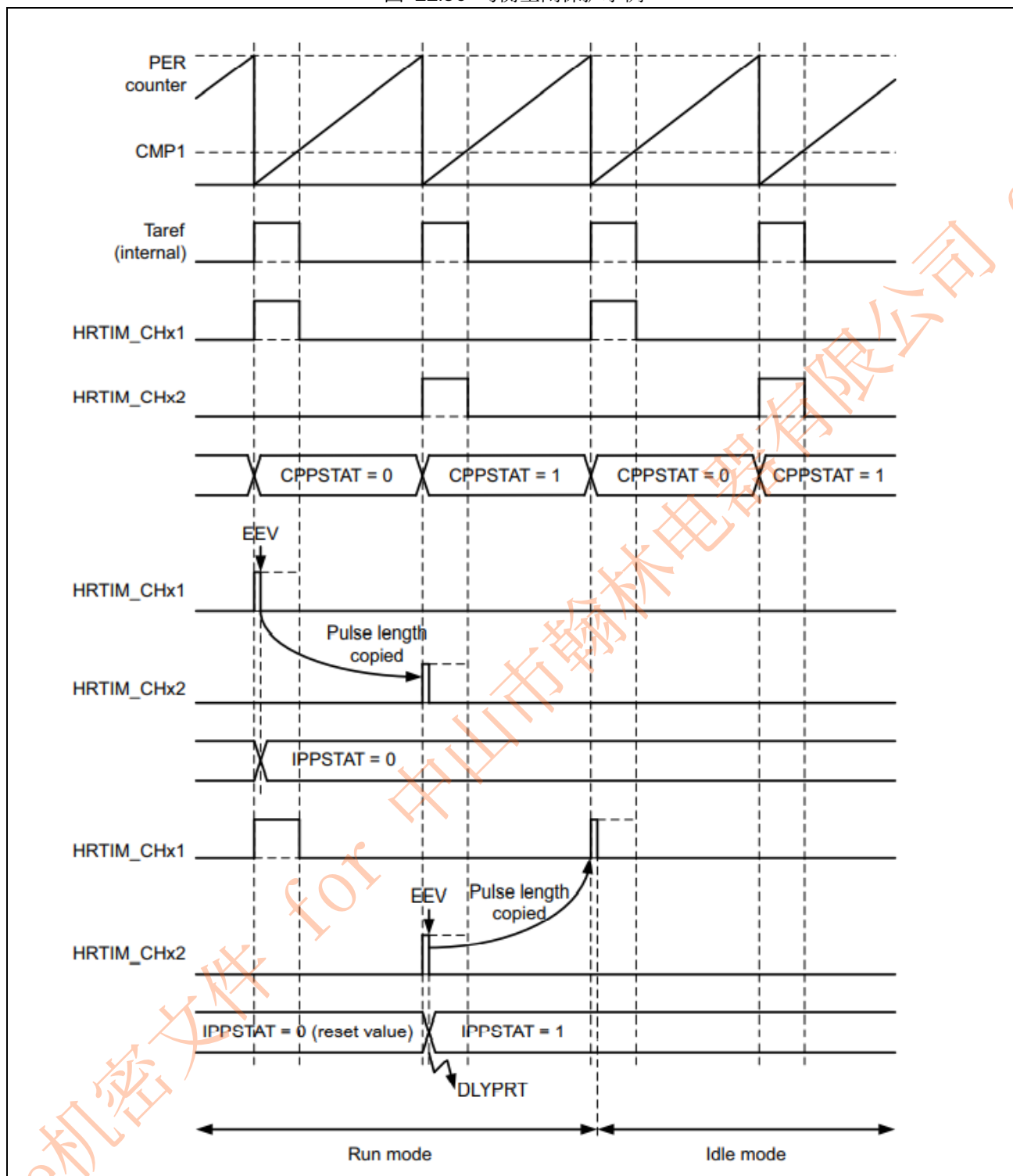
### 均衡空闲

仅在推挽模式下可用，在有效脉冲因保护方面的原因而缩短的情况下，可在两路输出上实现均衡脉宽。终止时间早于设定时间的脉宽会复制到另一路输出上，两路输出随后会进入空闲状态，直至通过软件恢复正常工作。此模式通过向 HRTIM\_OUTxR 中的 DLYPRT[2:0]位域写入 x11 来使能。

此模式仅支持 2 个外部事件：

- hrtim\_evt6 和 hrtim\_evt7（对于定时器 A、B 和 C）
- hrtim\_evt8 和 hrtim\_evt9（对于定时器 D、E 和 F）

图 22.50 均衡空闲保护示例



如果均衡空闲模式已使能，所选外部事件会触发将计数器值捕获到比较 4 活动寄存器（用户不可访问该值）。推挽模式会额外保持一个周期，以重复缩短的脉冲：保持常规输出置位事件的同时，会生成新的输出复位事件。

随后会进入空闲模式，输出会采用由 HRTIM\_OUTxR 寄存器中的 IDLESx 位定义的电平。均衡空闲模式进入由 DLYPRT 标志指示，IPPSTAT 标志则指示外部事件在哪一周期的发生，可用于确定缩短脉冲的顺序（先是 A1，然后是 A2，反之亦然）。

定时器操作不会中断（计数器继续运行）。

要使用均衡空闲模式，需要执行下列初始化操作：

- 定时器在连续模式下工作（CONT = 1）
- 使能推挽模式
- HRTIM\_CMP4xR 必须设为 0，并且内容要传输到活动寄存器（例如，强制进行软件更新）
- DELCMP4[1:0] 位域必须设为 00（禁止自动延迟模式）
- DLYPRT[2:0] = x11（使能延迟保护）

注：均衡空闲工作期间，不得对 HRTIM\_CMP4xR 寄存器执行写操作。保留 CMP4 事件，不得用于其他用途。

在均衡空闲模式下，建议避免使用多个外部事件或基于软件的复位事件，后者会导致输出复位。如果同一周期内此类事件先于均衡空闲请求到达，则会导致输出脉冲不均衡（第 1 个脉冲长度由外部事件或软件复位定义，而第 2 个脉冲长度则由均衡空闲模式进入定义）。

均衡空闲模式下可用的最小脉宽为 4 个  $f_{HRTIM}$  时钟周期（当 CKPSC[2:0] = 0 时为 0x80，CKPSC[2:0] = 1 时为 0x40，CKPSC[2:0] = 2 时为 0x20，依此类推）。如果在计数器达到其最小值之前进行捕获，会先将当前脉冲最多延长为 4 个  $f_{HRTIM}$  时钟周期，然后再将其复制到次级输出。在任何情况下，脉宽始终都是均衡的。

Tx1OEN 和 Tx2OEN 位不受均衡空闲模式进入的影响。要退出均衡空闲模式并恢复操作，需要同时将 Tx1OEN 和 Tx2OEN 位重写为 1。输出状态将在输出使能后的第一次有效跳变时更改。

可采用与延迟空闲模式进入相似的方法恢复操作。例如，如果外部事件到达时输出 1 激活（延迟空闲模式在输出 2 脉冲后生效），可先为输出 1 启动重启序列。为此，需要轮询 HRTIM\_TIMxISR 寄存器中的 CPPSTAT 位。使用上例（IPPSTAT 标志等于 0），操作将在 CPPSTAT 位为 0 零时恢复。

为了获得特定的重启序列，可轮询 CPPSTAT 了解哪一输出将先激活。这样一来，便可使用与空闲模式进入序列相同的序列进行重启：如果 EEV 在输出 1 激活期间到达，则将在输出 1 激活时（CPPSTAT = 0）发起重启序列。

注：正在执行脉冲均衡序列时，不得禁止均衡空闲模式。需要等到 CMP4 标志置 1（指示序列已结束）才能复位 DLYPRTEN 位。

仅当计数器使能后（TxCEN 位置 1），才能触发均衡空闲保护模式。即使 TxCEN 位已复位，在 TxyOEN 位置 1 之前，延迟保护模式仍保持有效状态。

下列情况下，均衡空闲模式可与突发模式一起使用：

- TxBM 位必须被重置（突发期间保持计数器时钟）
- 输出处于突发空闲状态时，不得触发均衡空闲保护。

均衡空闲模式的优先级要高于突发模式：一旦触发了均衡空闲保护，就会丢弃任何突发模式退出请求。相反，如果在突发模式有效时退出延迟保护，突发模式将正常恢复。

注：虽然输出状态在空闲模式下会冻结，但在延迟保护后的空闲时间段内，仍会在辅助输出上生成一些事件

- 输出置位/复位中断或DMA 请求
- 基于输出信号的外部事件过滤
- 由置位/复位触发的捕获事件

### 均衡空闲自动恢复

均衡空闲模式可以配置为在触发后自动恢复操作。

一旦缩短的脉冲被复制到备用输出，脉冲宽度将重置为其原始值，定时器恢复操作：两个输出将继续处于运行模式。

通过在 HRTIM\_OUTxR 寄存器中设置 BIAR 位来使能此功能。

此模式仅当 HRTIM\_PERxR 中的周期大于  $f_{\text{HRTIM}}$  时钟的 6 个周期时才应使用，即如果 CKPSC[2:0] = 0，则为 0xC0；如果 CKPSC[2:0] = 1，则为 0x60；如果 CKPSC[2:0] = 2，则为 0x30，以此类推。

注：此位仅在 DLYPRT[2:0] = 011 或 111 时才有意义，否则将被忽略。

注：在均衡空闲自动恢复模式下，必须将 IDLES 状态设置为无效状态。

### 22.3.11 寄存器预加载和更新管理

大部分 HRTIM 寄存器均已缓冲，可根据需要进行预装载。这样做通常可避免波形被与有效事件（置位/复位）不同步的寄存器更新更改。

使能预装载模式后，受访问的寄存器变为影子寄存器。收到软件发出的更新请求或与事件同步的更新请求后，影子寄存器的内容会传输到活动寄存器。

默认情况下，HRTIM\_MCR 和 HRTIM\_TIMxCR 寄存器中的 PREEN 位会复位，寄存器不会预装载：任何写操作均会直接更新活动寄存器。如果 PREEN 位在定时器运行且预装载已使能时复位，预装载寄存器的内容会直接传输到活动寄存器中。

每个定时单元和主定时器都有自己的 PREEN 位。如果 PREEN 已置 1，仅当发生更新事件时，预装载寄存器才会使能，其内容才会传输到活动寄存器。

如果需要使用预装载功能，可选择两种方法初始化定时器：

- 刚好在定时器初始化结束时使能 PREEN 位，以在定时器使能之前将预装载寄存器内容传输到活动寄存器（通过将 MCEN 和 TxCEN 位置 1）。
- 在初始化过程中的任何时间使能 PREEN 位，并恰好在启动之前强制进行软件更新。

下表列出了可预装载的寄存器，并总结了可用更新事件。

表 22.20 HRTIM 可预装载控制寄存器和关联的更新源

| 定时器               | 可预装载寄存器                                                                                                                                                                                                                                       | 预装载使能                                                                 | 更新源                                                                                                                                                          |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 主定时器              | HRTIM_DIER<br>HRTIM_MPER<br>HRTIM_MREP<br>HRTIM_MCMP1R<br>HRTIM_MCMP2R<br>HRTIM_MCMP3R<br>HRTIM_MCMP4R                                                                                                                                        | HRTIM_MCR<br>中的 PREEN 位                                               | 软件<br>重复事件<br>突发 DMA 事件<br>突发 DMA 事件后的重复事件                                                                                                                   |
| 定时器 x<br>x = A..F | HRTIM_TIMxDIER<br>HRTIM_TIMxPER<br>HRTIM_TIMxREP<br>HRTIM_TIMxCMP1R<br>HRTIM_TIMxCMP1CR<br>HRTIM_TIMxCMP2R<br>HRTIM_TIMxCMP3R<br>HRTIM_TIMxCMP4R<br>HRTIM_DTxD<br>HRTIM_SETx1R<br>HRTIM_RSTx1R<br>HRTIM_SETx2R<br>HRTIM_RSTx2R<br>HRTIM_RSTxD | HRTIM_TIMxCR<br>中的 PREEN 位                                            | 软件<br>TIMx 重复事件<br>TIMx 复位事件<br>突发 DMA 事件<br>来自其他定时器 (TIMy, 主定时器) 的更新事件<br>突发 DMA 事件后的更新事件<br>更新使能输入 hrtim_upd_en[3:1]<br>更新使能输入 hrtim_upd_en[3:1]<br>后的更新事件 |
| HRTIM<br>通用寄存器    | HRTIM_ADC1R<br>HRTIM_ADC2R<br>HRTIM_ADC3R<br>HRTIM_ADC4R                                                                                                                                                                                      | TIMx 或主定时器更新，取决于 HRTIM_CR1 中的<br>ADxUSRC[2:0]位 (如果所选定定时器中的 PREEN = 1) |                                                                                                                                                              |

主定时器有 4 个更新选项：

1. 软件：将 1 写入 HRTIM\_CR2 中的 MSWU 位会立即强制更新寄存器。在这种情况下，会取消任何待定的硬件更新请求。
2. 当主计数器翻转且主重复计数器等于 0 时，会进行更新。当 HRTIM\_MCR 中的 MREPU 位置 1 时，会使能此操作。
3. 突发 DMA 结束后进行更新。当 HRTIM\_MCR 中的 BRSTDMA[1:0] = 01 时，会使能此操作。可以同时设置 MREPU = 1 和 BRSTDMA = 01。  
注：如果 SWU 位置 1，可在突发序列结束后立即进行更新（即强制更新模式）。如果 SWU 位复位，将在突发序列结束后发生下一个更新事件时进行更新。
4. 突发 DMA 结束后主计数器翻转时，会进行更新。当 HRTIM\_MCR 中的 BRSTDMA[1:0] = 10 时，会使能此操作。

中断或 DMA 请求可通过主定时器更新事件生成。



各个定时器 (TIMA..F) 还可以通过 ([3:0]) 方式进行更新:

- 软件: 将 1 写入 HRTIM\_CR2 中的 TxSWU 位会立即强制更新寄存器。在这种情况下, 会取消任何挂起的硬件更新请求。
- 当计数器翻转且重复计数器等于 0 时, 会进行更新。当 HRTIM\_TIMxCR 中的 TxREPU 位置 1 时, 会使能此操作。
- 当计数器复位或在连续模式下翻转时, 会进行更新。当 HRTIM\_TIMxCR 中的 TxRSTU 位置 1 时, 会使能此操作。此操作用于在单发模式下工作的定时器等。
- 突发 DMA 结束后立即进行更新。当 HRTIM\_TIMxCR 中的 UPDGAT[3:0] = 0001 时, 会使能此操作。
- 在突发 DMA 结束后发生更新事件时会进行更新 (更新事件可通过 TxREPU、MSTU 或 TxU 使能)。当 HRTIM\_TIMxCR 中的 UPDGAT[3:0] = 0010 时, 会使能此操作。
- 更新使能输入 hrtim\_upd\_en[3:1] 上接收到请求时, 会进行更新。当 HRTIM\_TIMxCR 中的 UPDGAT[3:0] = 0011、0100、0101 时, 会使能此操作。
- 在更新使能输入 hrtim\_upd\_en[3:1] 上接收到请求后发生更新事件时会进行更新 (更新事件可通过 TxREPU、MSTU 或 TxU 使能)。当 HRTIM\_TIMxCR 中的 UPDGAT[3:0] = 0110、0111、1000 时, 会使能此操作。
- 与其他任何定时器或主定时器更新同步进行更新 (例如, TIMA 可与 TIMB 同步更新)。此操作通过将 HRTIM\_TIMxCR 寄存器中的 MSTU 和 TxU 位置 1 来使能, 用于需要使用多个定时器的转换器。

更新使能输入 hrtim\_upd\_en[3:1] 可使更新事件与来自通用定时器的片上事件同步。这些输入为上升沿有效。

表 22.3 列出了更新使能输入与片上源之间的连接关系。

这样可将低频更新请求与高频信号同步 (例如, 在不得不用 100Hz 的速率更新 100KHz 的翻转计数器时)。

注: 当 CKPSC[2:0] > 5 时, 更新事件会同步到预分频器时钟。

来自相邻定时器 (当 MSTU、TAU、TBU、TCU、TDU、TEU、TFU 位被设置时) 或来自软件更新 (TxSWU 位) 的更新可以立即被考虑, 也可以与定时器的复位/翻转事件重新同步。这通过 HRTIM\_TIMxCR 寄存器中的 RSYNCU 位来实现, 如下图所示:

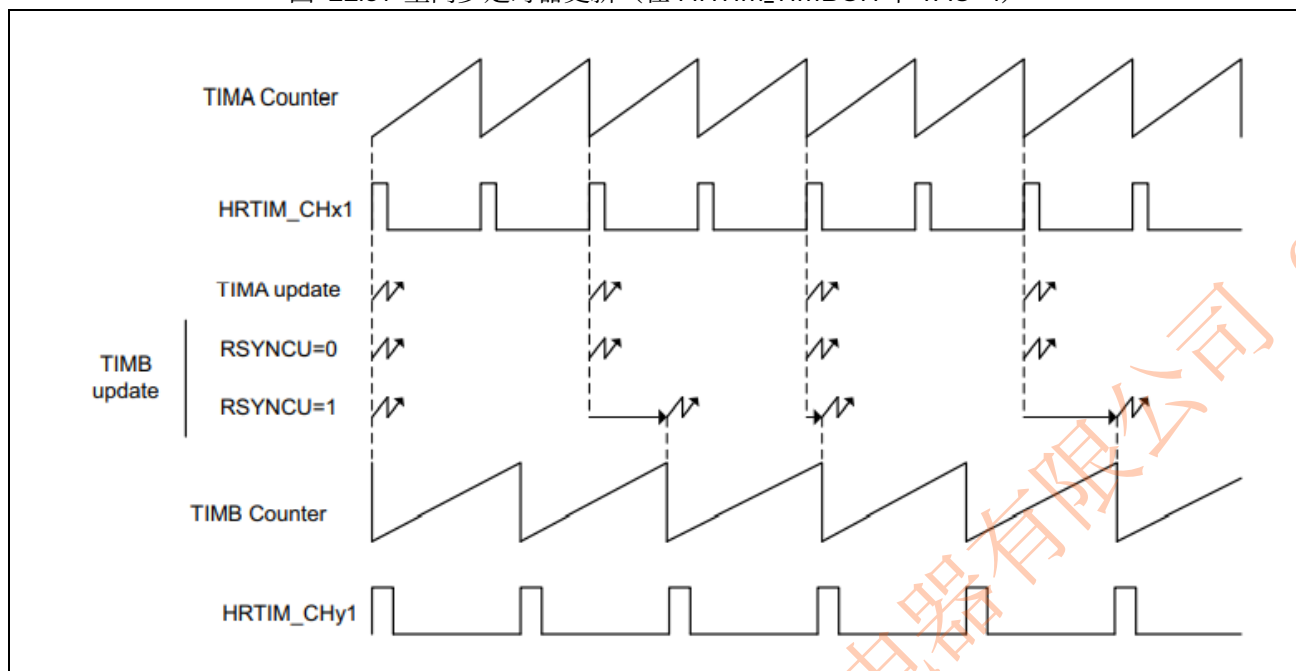
- RSYNCU = 0: 来自相邻定时器的更新立即被考虑。
- RSYNCU = 1: 来自相邻定时器的更新将在下一个复位/翻转事件上被考虑。

只有当 UPDGAT[3:0] = 0000 时, RSYNCU 位才有意义, 否则它将被忽略。

Timx 更新事件可以生成中断或 DMA 请求。



图 22.51 重同步定时器更新（在 HRTIM\_TIMBCR 中 TAU=1）



无论选择哪一种更新事件，HRTIM\_CR1 寄存器中的 MUDIS 和 TxUDIS 位都可以暂时禁止将预装载寄存器的内容传输至活动寄存器。这样便可修改并联的定时器中的多个寄存器。这些位复位后，会立即执行常规更新事件。

MUDIS 和 TxUDIS 位全部分组到同一寄存器中。这样便可同时禁止和恢复更新并联工作的定时器（不需要同步）。下例为实际用例。第一个电源转换器通过主定时器、TIMB 和 TIMC 控制。TIMB 和 TIMC 必须同时通过主定时器重复事件更新。第二个转换器与 TIMA、TIMD 和 TIME 并行运行，TIMD、TIME 必须通过 TIMA 重复事件更新。

第一个转换器：

在 HRTIM\_MCR 中，MREPU 位已置 1：将在主定时器计数器重复周期结束时进行更新。在 HRTIM\_TIMBCR 和 HRTIM\_TIMCCR 中，MSTU 位必须置 1 才能通过主定时器同时更新 TIMB 和 TIMC 定时器。

如果必须通过软件调整电源转换器的设定值，则必须在对寄存器执行写访问更新数值（例如比较值）之前将 HRTIM\_CR 寄存器的 MUDIS、TBUDIS 和 TCUDIS 位置 1。从此刻起，会忽略任何硬件更新请求，并可无风险地访问预装载寄存器，将其中的内容传输到活动寄存器中。软件处理结束后，必须将 MUDIS、TBUDIS 和 TCUDIS 位复位。发生主定时器重复事件后，会立即将预装载寄存器的内容传输到活动寄存器。

第二个转换器：

在 HRTIM\_TIMACR 中，TAREPU 位已置 1：将在定时器 A 计数器重复周期结束时进行更新。在 HRTIM\_TIMDCR 和 HRTIM\_TIMECR 中，TAU 位必须置 1 才能通过定时器 A 同时更新 TIMD 和 TIME 定时器。

如果必须通过软件调整电源转换器的设定值,则必须在对寄存器执行写访问更新数值(例如比较值)之前将 HRTIM\_CR 寄存器的 TAUDIS、TDUDIS 和 TEUDIS 位置 1。从此刻起,会忽略任何硬件更新请求,并可无风险地访问预装载寄存器,将其中的内容传输到活动寄存器中。软件处理结束后,可将 TAUDIS、TDUDIS 和 TEUDIS 位复位:发生定时器 A 重复事件后,会立即将预装载寄存器的内容传输到活动寄存器。

### 22.3.12 带有“大于”比较的 PWM 模式

CMP1 和 CMP3 寄存器生成的 PWM 信号具有一种特定的无延迟更新模式。它允许在 PWM 周期内尽快应用新的占空比值,而无需等待当前 PWM 周期完成。这减少了软件控制循环中的整体延迟时间。如下图所示,这最终可以实现:

- 如果新的比较值低于当前计数器值,且当前比较值高于计数器值(在写入新值时),输出将提前关闭。
- 如果新的比较值高于计数器值,且当前比较值也高于计数器值(在写入新值时),输出将提前打开,重新使能输出。

当新的比较值和当前比较值都低于计数器时,输出信号保持不变。

此功能仅适用于 CMP1 或 CMP3 的复位事件,并使用 HRTIM\_TIMxCR2 寄存器中的 GTCMP1 和 GTCMP3 使能位来使能。

当相应的 GTCMPx 位被设置时,预加载机制对于比较寄存器是不起作用的,无论 PREEN 位的值如何。此模式旨在写入新值后尽快考虑新的比较值,而无需等待预加载到有效寄存器的传输。

这些位定义了比较 1 和比较 3 的操作模式,如下:

- GTCMPx = 0: 当计数器等于比较值时,生成比较 x 事件(比较匹配模式)。如果比较值在运行时被更改,比较事件可能不会被生成。
- GTCMPx = 1: 当计数器大于比较值时,生成比较 x 事件。如果比较值在运行时被更改,新的比较值将与当前计数器值进行比较,并可以生成输出置位或复位。

“大于”比较模式会根据比较结果的不同,导致纵横开关的行为有所区别。让我们考虑 CMP1 事件执行输出复位的情况。当写入新的比较值时,会考虑以下两种情况:

- 如果新的比较值低于计数器值,将发出复位事件,这可能会导致输出提前关闭。
- 如果新的比较值高于计数器值,将生成一个置位事件,以便在计数器实际超过比较值并触发复位之前,重新激活输出值(提前打开)。

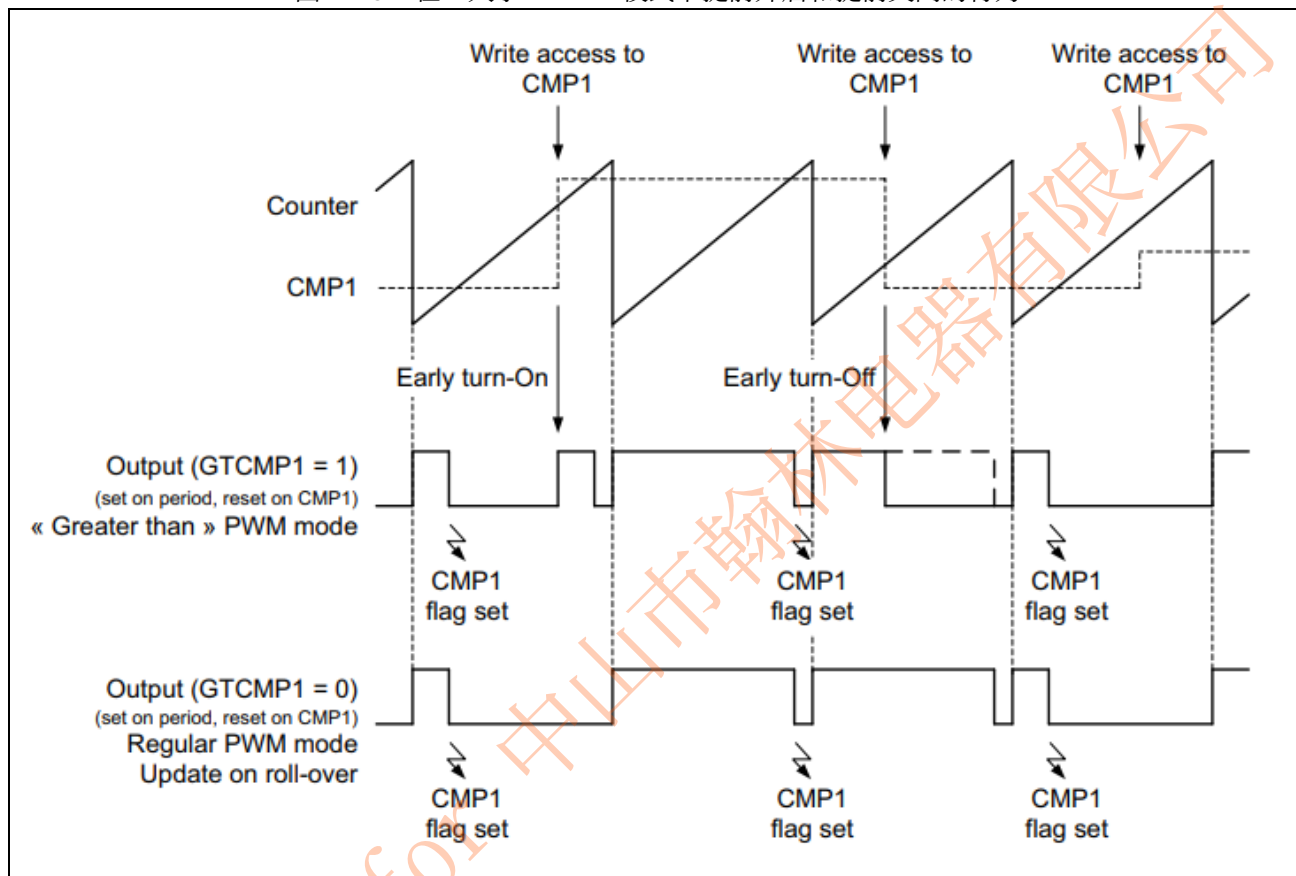
“大于”比较模式同时支持置位和复位操作。

“大于”比较模式仅应用于以下配置:

1. 在固定频率配置中，周期事件必须触发输出置位，而“大于”比较则触发输出复位（或者，如果“大于”比较触发置位，那么周期事件必须触发复位）。
2. 对于可变频率配置，选作计数器复位源的事件也必须选作定时器输出的置位或复位源（与“大于”比较事件的方向相反）。

注：当 CMP1 和/或 CMP3 模式在半模式和交错模式下由硬件控制时，不应使用“大于”模式。

图 22.52 在“大于”PWM 模式下提前开启和提前关闭的行为



立即更新模式意味着在写入寄存器的同时，预加载寄存器的内容会被传输到有效寄存器中。当 GTCMP1 和/或 GTCMP3 位被设置时，它们各自的预加载机制将被禁用（对于 HRTIM\_TIMxCMP1 和/或 HRTIM\_TIMxCMP3 寄存器），无论 PREEN 位的值如何。

注：在上图所示的情况下，对于延迟开启和提前关闭，不会生成比较中断标志（HRTIM\_TIMxISR 中的 CMP1 和 CMP3）。

注：“大于”比较不能在同一个输出的 CMP1 和 CMP3 上都进行（GTCMP1 和 GTCMP3 位不能同时被设置）。

### 22.3.13 事件在多个定时器之间的传播

HRTIM 提供多种方式在多个定时单元（包括主定时器）之间逐级传输事件或共享事件，以充分利用其模块化架构的优势。这些是需要使用多路同步输出的转换器的主要特性。

本节总结了各种选项，并详细说明了事件是否会在 HRTIM 中传播以及怎样传播。

### 主计时器更新触发的 TIMx 更新

下表中列出的源将生成主计时器更新。该表指出了是否可使用源事件在任何 TIMx 定时单元中触发同步更新。

工作条件：HRTIM\_TIMxCR 寄存器中的 MSTU 位置 1。

表 22.21 主计时器更新事件传播

| 源                                  | 条件                | 传播 | 注释                                           |
|------------------------------------|-------------------|----|----------------------------------------------|
| 突发 DMA 结束                          | BRSTDMA[1:0] = 01 | 无  | 必须在 TIMxCR 中完成 (UPDGAT[3:0] = 0001)          |
| 突发 DMA 结束后的翻转事件                    | BRSTDMA[1:0] = 10 | 有  | -                                            |
| 由计数器翻转引起的重复事件                      | MREPU = 1         | 有  | -                                            |
| 由计数器复位 (通过 HRTIM_SCIN 或软件) 引起的重复事件 |                   | 无  | -                                            |
| 软件更新                               | MSWU = 1          | 无  | 所有软件更新位 (TxSWU) 被分组在 HRTIM_CR2 寄存器中, 可用于同时更新 |

### 由 TIMy 更新触发的 TIMx 更新

下表中列出的源将生成 TIMy 更新。该表指出了是否可使用给定事件在另一定时器或多个 TIMx 定时单元中触发同步更新。

工作条件：HRTIM\_TIMxCR 寄存器中的 TyU 位置 1 (源 = TIMy, 目标 = TIMx)。

表 22.22 TIMx 更新事件传播

| 源                                | 条件                                           | 传播 | 注释                                                      |
|----------------------------------|----------------------------------------------|----|---------------------------------------------------------|
| 突发 DMA 结束                        | UPDGAT[3:0] = 0001                           | 无  | 必须直接在 HRTIM_TIMxCR 中进行 (UPDGAT[3:0] = 0001)             |
| 由更新使能输入 hrtim_upd_en[3:1] 引发的更新  | UPDGAT[3:0] = 0011, 0100, 0101               | 无  | 必须直接在 HRTIM_TIMxCR 中进行 (UPDGAT[3:0] = 0011、0100、0101)   |
| 主计时器更新                           | HRTIM_TIMyCR 中的 MSTU = 1                     | 无  | 必须在 HRTIM_TIMxCR 中的 MSTU = 1 时进行                        |
| 另一个 TIMx 更新 (TIMz > TIMy > TIMx) | HRTIM_TIMyCR 中的 TzU = 1<br>TIMxCR 中的 TyU = 1 | 无  | 必须在 HRTIM_TIMxCR 中的 TzU = 1 时进行 HRTIM_TIMyCR 中的 TzU = 1 |
| 由计数器翻转引发的重复事件                    | TyREPU = 1                                   | 有  | -                                                       |
| 由计数器复位引发的重复事件                    | TyREPU = 1                                   | -  | 请参见下文中的计数器复位情况                                          |
| 计数器翻转                            | TyRSTU = 1                                   | 有  | -                                                       |
| 计数器软件复位                          | HRTIM_CR2                                    | 无  | 可与 HRTIM_CR2 寄存器中的更新同时                                  |

| 源                     | 条件                              | 传播 | 注释                                        |
|-----------------------|---------------------------------|----|-------------------------------------------|
|                       | 中的 $TyRST = 1$                  |    | 进行与更新同时完成                                 |
| TIMz 比较引发的计数器复位       | HRTIM_RSTyR 中的 TIMzCMPn         | 有  | -                                         |
| 外部事件引发的计数器复位          | HRTIM_RSTyR 中的 EXTEVNTn         | 有  | -                                         |
| 主定时器比较或主定时器周期引发的计数器复位 | HRTIM_RSTyR 中的 MSTCMPn 或 MSTPER | 有  | -                                         |
| TIMy 比较引发的计数器复位       | HRTIM_RSTyR 中的 CMPn             | 有  | -                                         |
| 更新引起的计数器复位            | HRTIM_RSTyR 中的 UPDT             | 无  | 传播会导致进入锁定状态（更新会引发复位，进而引发更新）               |
| HRTIM_SCIN 引发的计数器复位   | HRTIM_TIMyCR 中的 SYNCRSTy        | 无  | -                                         |
| 软件更新                  | $TySWU = 1$                     | 无  | 所有软件更新位（TxSWU）均分组到 HRTIM_CR2 寄存器中，可用于同步更新 |

## 引发 TIMx 更新的 TIMx 计数器复位

下表列出了计数器复位源，并指出了这些源是否可用于生成更新事件。

工作条件：HRTIM\_TIMxCR 寄存器中的 TxRSTU 位。

表 22.23 能够生成更新事件的复位事件

| 源          | 条件                       | 传播 | 注释                          |
|------------|--------------------------|----|-----------------------------|
| 计数器翻转      | -                        | 有  | -                           |
| 更新事件       | HRTIM_RSTxR 中的 UPDT      | 无  | 传播会导致进入锁定状态（更新会引发复位，进而引发更新） |
| 外部事件       | HRTIM_RSTxR 中的 EXTEVTn   | 有  | -                           |
| TIMy 比较    | HRTIM_RSTxR 中的 TIMyCMPn  | 有  | -                           |
| 主定时器比较     | HRTIM_RSTxR 中的 MSTCMPn   | 有  | -                           |
| 主定时器周期     | HRTIM_RSTxR 中的 MSTPER    | 有  | -                           |
| 比较 2 和比较 4 | HRTIM_RSTxR 中的 CMPn      | 有  | -                           |
| 软件         | in HRTIM_CR2 中 TxRST=1   | 有  | -                           |
| HRTIM_SCIN | HRTIM_TIMxCR 中的 SYNCRSTx | 有  | -                           |

## 引发 TIMx 计数器复位的 TIMx 更新

下表列出了更新事件源，并指出了这些源是否可用于生成计数器复位事件。

工作条件：HRTIM\_RSTxR 寄存器中的 UPDT 位置 1。

表 22.24 用于定时器复位的更新事件传播

| 源                               | 条件                                                       | 传播 | 注释 |
|---------------------------------|----------------------------------------------------------|----|----|
| 突发 DMA 结束                       | UPDGAT[3:0] = 0001                                       | 有  | -  |
| 由更新使能输入 hrtim_upd_en[3:1] 引发的更新 | UPDGAT[3:0] = 0011, 0100, 0101                           | 有  | -  |
| 突发 DMA 后的翻转引发的<br>主定时器更新        | HRTIM_TIMxCR 中 MSTU = 1<br>HRTIM_MCR 中 BRSTDMA[1:0] = 10 | 有  | -  |
| 翻转后的重复事件引发的<br>主定时器更新           | HRTIM_TIMxCR 中 MSTU = 1                                  | 有  | -  |

|                                             |                                                                                                     |   |                                             |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------|---|---------------------------------------------|
| 计数器复位（通过软件或 HRTIM_SCIN 复位）后的重复事件引发的主定时器更新   | HRTIM_MCR<br>中 MREPU = 1                                                                            | 无 | -                                           |
| 软件触发的<br>主定时器更新                             | HRTIM_TIMxCR<br>中 MSTU = 1<br>HRTIM_CR2<br>中 MSWU = 1 in                                            | 无 | 所有软件更新位 (TxSWU) 均分组到 HRTIM_CR2 寄存器中，可用于同步更新 |
| TIMy 计数器翻转引发的<br>TIMy 更新                    | HRTIM_TIMxCR<br>中 TyU = 1<br>HRTIM_TIMyCR<br>中 TyRSTU = 1                                           | 有 | -                                           |
| TIMy 重复事件引发的<br>TIMy 更新                     | HRTIM_TIMxCR<br>中 TyU = 1<br>HRTIM_TIMyCR<br>中 TyREPU = 1                                           | 有 | -                                           |
| 外部事件或 TIMy 比较<br>（通过 TIMy 复位）引发的<br>TIMy 更新 | HRTIM_TIMxCR<br>中 TyU = 1<br>HRTIM_TIMyCR<br>中 TyRSTU = 1<br>HRTIM_RSTyCR<br>中 EXTEVNTn<br>或 CMP4/2 | 有 | -                                           |
| 上面列出的源以外的源<br>引发的 TIMy 更新                   | HRTIM_TIMxCR<br>中 TyU = 1                                                                           | 无 | -                                           |
| 翻转后的重复事件                                    | HRTIM_TIMxCR                                                                                        | 有 | -                                           |
| 计数器复位后的重复事件                                 | 中 TxREPU = 1                                                                                        | 无 | -                                           |
| 定时器复位                                       | HRTIM_TIMxCR<br>中 TxRSTU = 1                                                                        | 无 | 传播会导致进入锁定状态（复位会引发更新，进而引发复位）                 |
| 软件                                          | HRTIM_CR2<br>中 TxSWU                                                                                | 无 | -                                           |



### 22.3.14 输出管理

每个定时单元都控制着一对输出。输出有三种工作状态：

- RUN：此状态为主要工作状态，在该状态下，输出会获取在纵横开关单元中设定的有效电平或无效电平。
- IDLE：当输出通过软件禁用或处于突发模式工作期间（此时，正常工作模式下会暂时禁止输出，更多详细信息请参考第 15 小节），该状态是 HRTIM 复位后的默认工作状态。该状态永久有效或无效。
- FAULT：此状态为安全状态，FAULTx 输入上存在关断请求时会进入此状态。该状态可以是永久有效、无效或高阻态（Hi-Z）。

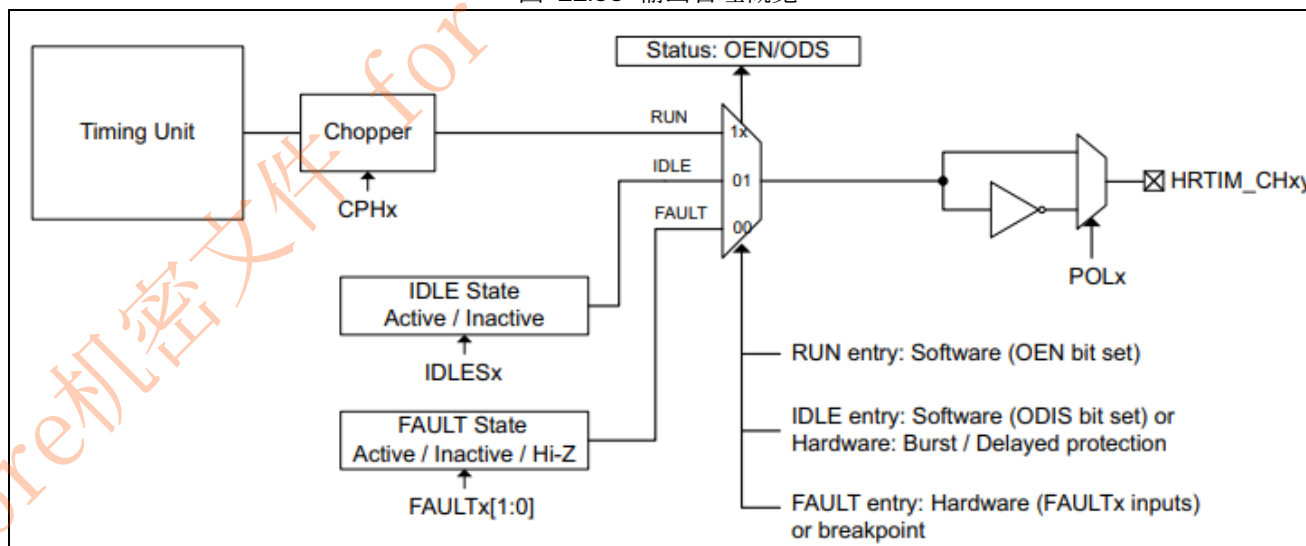
输出状态由 HRTIM\_OENR 寄存器中的 TxyOEN 位以及 HRTIM\_ODSR 寄存器中的 TxyODS 位指示，如下表所示。

表 22.25 输出状态编程，x= A..F, y= 1 或 2

| TxyOEN (控制/状态)<br>(由软件置 1, 硬件清 0) | TxyODS (状态) | 输出工作状态 |
|-----------------------------------|-------------|--------|
| 1                                 | x           | RUN    |
| 0                                 | 0           | IDLE   |
| 0                                 | 1           | FAULT  |

TxyOEN 位既是控制位，也是状态位：该位必须通过软件置 1，才能使输出处于 RUN 模式。输出恢复 IDLE 或 FAULT 模式时，该位会通过硬件清零。TxyOEN 位清零后，TxyODS 位会指示输出处于 IDLE 状态还是 FAULT 状态。HRTIM\_ODISR 寄存器中的第三位可通过软件禁止输出。

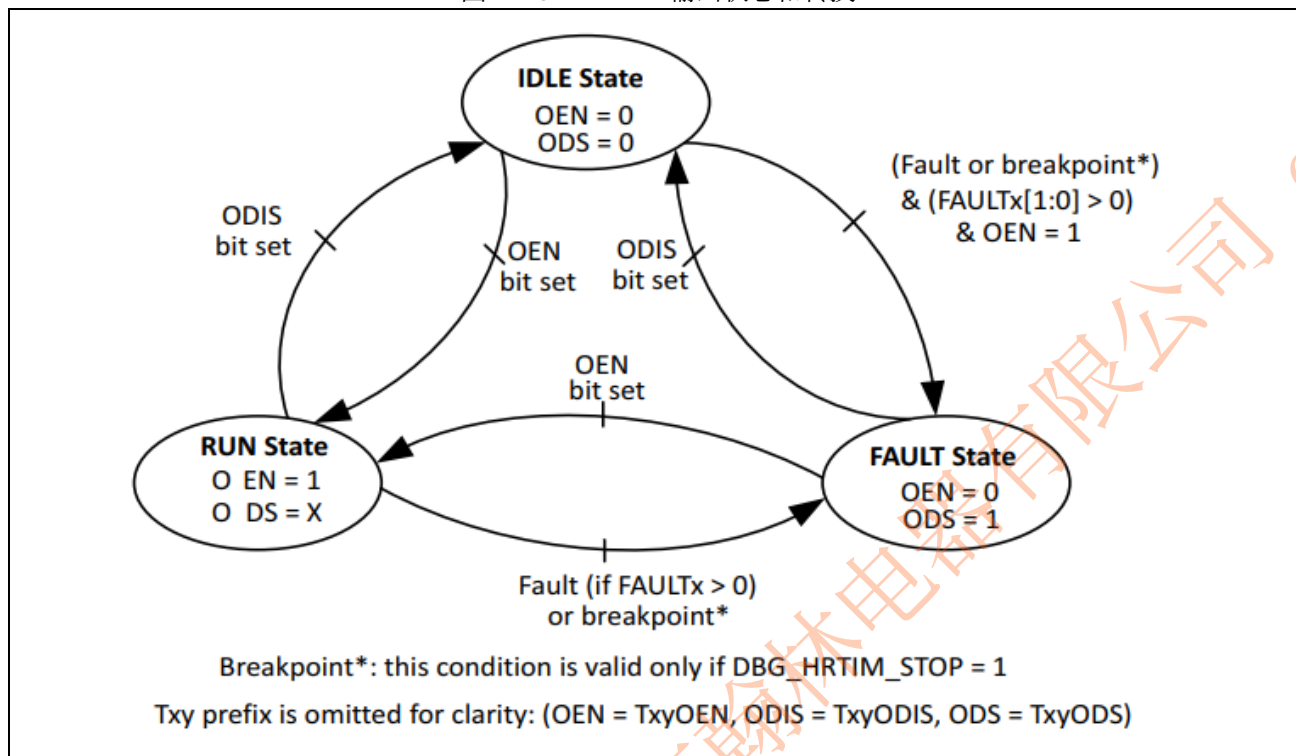
图 22.53 输出管理概览



下图总结了三种状态对应的位值以及转换的触发方式。任何外部或内部故障源均可触发故障。

当突发模式或延迟保护有效时，可进入空闲状态。

图 22.54 HRTIM 输出状态和转换



FAULT 和 IDLE 电平定义为有效或无效。有效（或无效）是指定时器输出上可引发电源开关闭合（对于无效状态，则为电源开关断开）的电平。

IDLE 状态的优先级最高：即使 FAULT 条件仍有效，也可以实现由 ODIS 位置 1 触发的 FAULT → IDLE 转换。

FAULT 状态的优先级要高于 RUN 状态：如果 TxyOEN 位置 1 的同时发生了故障事件，则将进入 FAULT 状态。IDLE → FAULT 转换中给出了相应的条件（如上图所示）：需要使能故障保护（FAULTx[1:0] 位 = 01、10、11），且 Txy OEN 位在故障有效时置 1（如果 DBG\_HRTIM\_STOP = 1，则在断点期间置 1）。

输出极性通过 HRTIM\_OUTxR 中的 POLx 位编程。如果 POLx = 0，极性为正（输出高电平有效）；如果 POLx = 1，极性为负（输出低电平有效）。实际上，极性的定义取决于要驱动电源开关（PMOS 与 NMOS）或栅极驱动器极性。

每路输出 FAULT 状态下的输出电平通过 HRTIM\_OUTxR 中的 FAULTx[1:0] 位配置，具体如下：

- 00：输出永不进入故障状态，停留在 RUN 或 IDLE 状态
- 01：处于 FAULT 状态时，输出为有效电平
- 10：处于 FAULT 状态时，输出为无效电平
- 11：处于 FAULT 状态时，输出为三态。必须通过诸如上拉或下拉电阻在外部强制进入安全状态。

注：只要输出处于 FAULT 状态，就不应更改 FAULTx[1:0] 位。

IDLE 状态下的输出电平通过 HRTIM\_OUTxR 中的 IDLESx 位配置，具体如下：

- 0：处于 IDLE 状态时，输出为无效电平
- 1：处于 IDLE 状态时，输出为有效电平

当 TxyOEN 位置 1 进入 RUN 状态时，输出会立即连接到纵横开关输出。如果定时器时钟停止，电平将为无效（HRTIM 复位后）或相当于 RUN 电平（定时器停止、且输出禁止时）。

在 HRTIM 初始化过程中，在使输出进入 RUN 模式之前，可在 HRTIM\_SETx1R 和 HRTIM\_RSTx1R 寄存器中使用软件强制输出置位和复位预先设定输出电平。

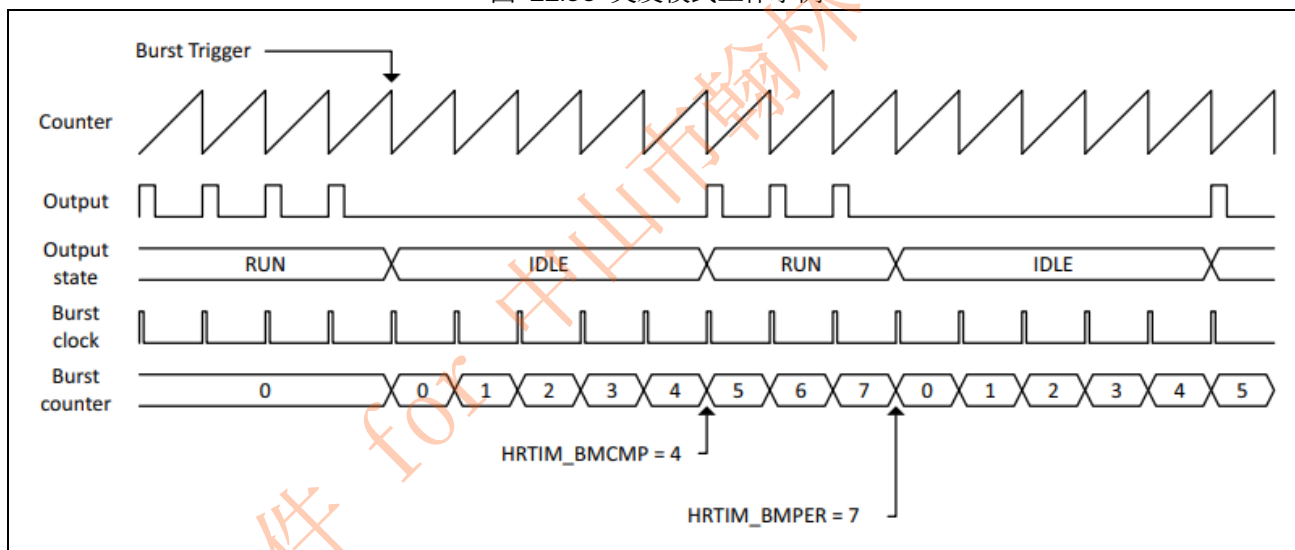
### 22.3.15 突发模式控制器

突发模式控制器可通过硬件使输出交替地进入 IDLE 状态和 RUN 状态，以便通过可编程周期性和占空比跳过一些开关周期。

轻载运行时，突发模式工作常用于电源转换器中。该模式减少了输出跳变次数以及关联的开关损耗，从而显著提高转换器效率。

在突发模式下运行时，会在空闲周期（等于几个计数周期）后输出一个或几个脉冲，空闲周期内通常不会生成任何输出脉冲，如下图中的示例所示。

图 22.55 突发模式工作示例



突发模式控制器包括：

- 可由各种源提供时钟的计数器（HRTIM 之内或之外，通常是 PWM 周期结束时）。
- 定义空闲周期数的比较寄存器：HRTIM\_BMCMP。
- 定义突发重复率的周期寄存器（相当于空闲周期与运行周期之和）：HRTIM\_BMPER。

突发模式控制器能够接管对 10 路 PWM 输出中的任何一路的控制。突发模式工作期间，每路输出的状态通过 HRTIM\_OUTxR 寄存器中的 IDLESx 和 IDLEMx 位编程，如下表所述。

表 22.26 突发模式的定时器输出编程

| IDLEMx | IDLESx | 突发模式时的输出状态        |
|--------|--------|-------------------|
| 0      | X      | 无操作：输出不受突发模式操作的影响 |
| 1      | 0      | 在突发期间输出无效         |
| 1      | 1      | 在突发期间输出有效         |

注：突发模式有效时，不得更改 IDLEMx 位。

突发模式控制器仅会作用于输出级。空闲周期内仍会生成一些事件：

- 输出置位/复位中断或 DMA 请求
- 基于 Tx2 输出信号的外部事件过滤
- 由输出置位/复位触发的捕获事件

突发模式期间，即使 TxBM 位已置 1，HRTIM\_SCOUT 输出上也不会生成启动事件或复位事件。

### 工作模式

在给定的定时单元上使用突发模式之前，必须使能计数器（TxCEN 位置 1）。突发模式通过 HRTIM\_BMCR 寄存器中的 BME 位使能。

通过 HRTIM\_BMCR 寄存器中的 BMOM 位，定时单元可在连续模式或单发模式下工作。BMOM = 1 时，使能连续模式。在 HRTIM\_BMCR 中的 BMSTAT 位复位终止突发工作之前，都会保持突发工作状态。

在单发模式下 (BMOM = 0)，会执行一次空闲序列，随后会触发突发模式，之后会立即恢复正常定时器操作。

空闲周期和运行周期的持续时间通过突发模式计数器和 2 个寄存器定义。HRTIM\_BMCMRPR 寄存器以计数定义所选定时器处于空闲状态的持续时间（空闲周期）。HRTIM\_BMPER 定义总突发模式周期（空闲周期与运行周期之和）。触发初始突发模式后，空闲周期长度为 HRTIM\_BMCMRPR+1，总突发周期为 HRTIM\_BMPER+1。

注：突发模式周期不得小于或等于通过 DTRx[8:0] 和 DTFx[8:0] 位域定义的死区持续时间。

突发模式工作期间，定时单元和主定时器的计数器可停止和复位。HRTIM\_BMCR 保留了 6 个控制位来实现此目的：MTBM（主定时器）和 TABM..TEBM（用于定时器 A..E）。

当 MTBM 或 TxBM 位复位时，会保留计数器时钟。举例来说，这样可在多相位系统中保持与其他定时器的相位关系。

如果 MTBM 或 TxBM 位已置 1，则在突发空闲周期期间，相应的计数器会停止，并会保持在复位状态下。这样便可使定时器在退出空闲状态时重新开始一个完整的周期。如果 SYNC SRC[1:0] = 00 或 10（主定时器启动或定时器 A 启动时的同步输出），退出突发模式时，会在 HRTIM\_SCOUT 输出上发送一个脉冲。

注：当均衡空闲模式激活时 (DLYPRT[1:0] = 0x11)，不得将 TxBM 位置 1。

## 突发模式时钟

突发模式控制器计数器可由多个时钟源提供时钟，具体通过 HRTIM\_BMCR 寄存器中的 BMCLK[3:0] 位来选择：

- BMCLK[3:0] = 0000 至 0101：主定时器和 TIMA.E 的复位/翻转事件。这样可使突发模式空闲周期和运行周期与定时单元计数周期对齐（均处于自由运行和计数器复位模式）。
- BMCLK[3:0] = 0110 至 1001：时钟由 hrtim\_bm\_ck[4:1] 输入提供，这些输入连接到通用定时器，如下表所示。在这种情况下，突发模式空闲周期和运行周期不必与定时单元计数周期对齐（输出上的脉冲可能中断，从而导致波形的占空比被修改）。
- BMCLK[3:0] = 1010：f<sub>HRTIM</sub> 时钟由通过 HRTIM\_BMCR 寄存器中的 BMPRSC[3:0] 位定义的系数进行预分频。在这种情况下，突发模式空闲周期和运行周期不必与定时单元计数周期对齐（输出上的脉冲可能中断，从而导致波形的占空比被修改）。

TIMx OC 输出上的脉冲宽度必须至少为 N 个 f<sub>HRTIM</sub> 时钟周期，才能被 HRTIM 突发模式控制器检测到。

## 突发模式触发

要触发突发模式工作，可使用 32 个源，这些源通过 HRTIM\_BMTRGR 寄存器选择：

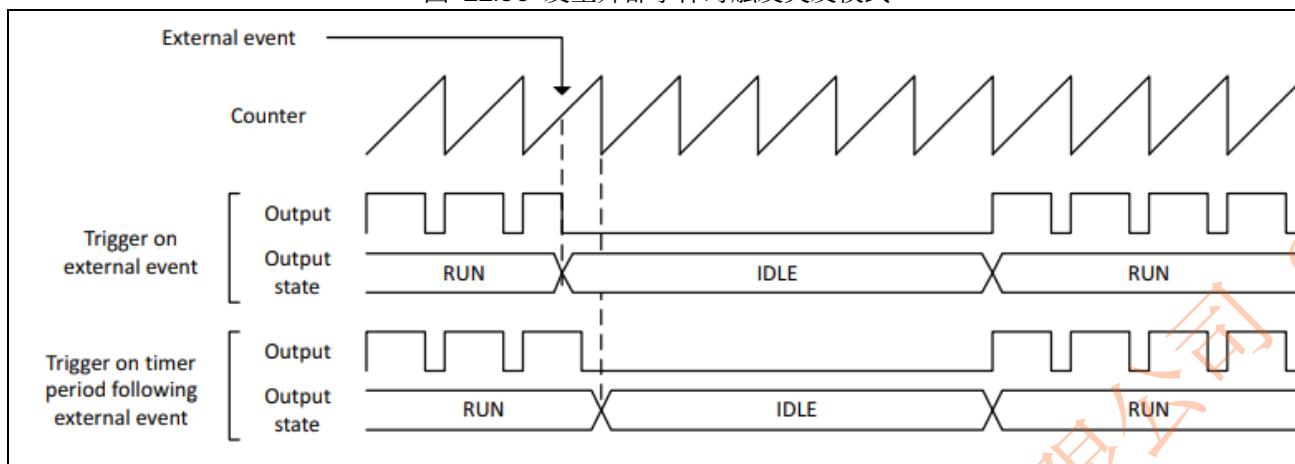
- 软件触发（通过软件置 1，通过硬件复位）
- 6 个主定时器事件：重复、复位/翻转、比较 1 到 4
- 来自定时器 A.E 的 5 × 4 个事件：重复、复位/翻转、比较 1 到 2
- 外部事件 7（包括 TIMA 事件过滤）和 8（包括 TIMD 事件过滤）
- 外部事件 7 之后的定时器 A 周期（包括 TIMA 事件过滤）
- 外部事件 8 之后的定时器 D 周期（包括 TIMD 事件过滤）
- hrtim\_bm\_trg 输入上的片上事件（连接到通用定时器的 TRGO 输出），具体细节见下表。

这些源可结合起来使用，实现多源并发触发。

突发模式不可再次触发。在连续模式下，突发模式终止之前，会忽略新的触发；而在单发模式下，包含运行周期的当前突发结束之前（HRTIM\_BMPER+1 个周期），会忽略触发。这一点同样适用于软件触发（即使软件位已丢弃，也会通过硬件复位）。

下图显示了突发模式是如何响应外部事件而启动的（立即启动，或在事件后的定时器周期启动）。

图 22.56 发生外部事件时触发突发模式



对于 TAEV7 和 TDEEV8 组合触发（在外部事件后的定时器周期触发），无论采用何种突发模式编程以及正在进行何种突发操作，外部事件检测始终有效：

- 如果在外部事件与定时器周期事件之间突发模式已使能（BME=1）或触发已使能（BMTRG 寄存器中的 TAEV7 或 TDEEV8 位置 1），则会触发突发模式。
- 即使外部事件在突发结束之前发生，也会再次触发单发突发模式（只要突发之后发生相应的周期事件）。

注：TAEV7 和 TDEEV8 触发仅在周期事件后有效。如果计数器在周期事件之前复位，则会丢弃挂起的 hrtim\_evt7 和 hrtim\_evt8 事件。

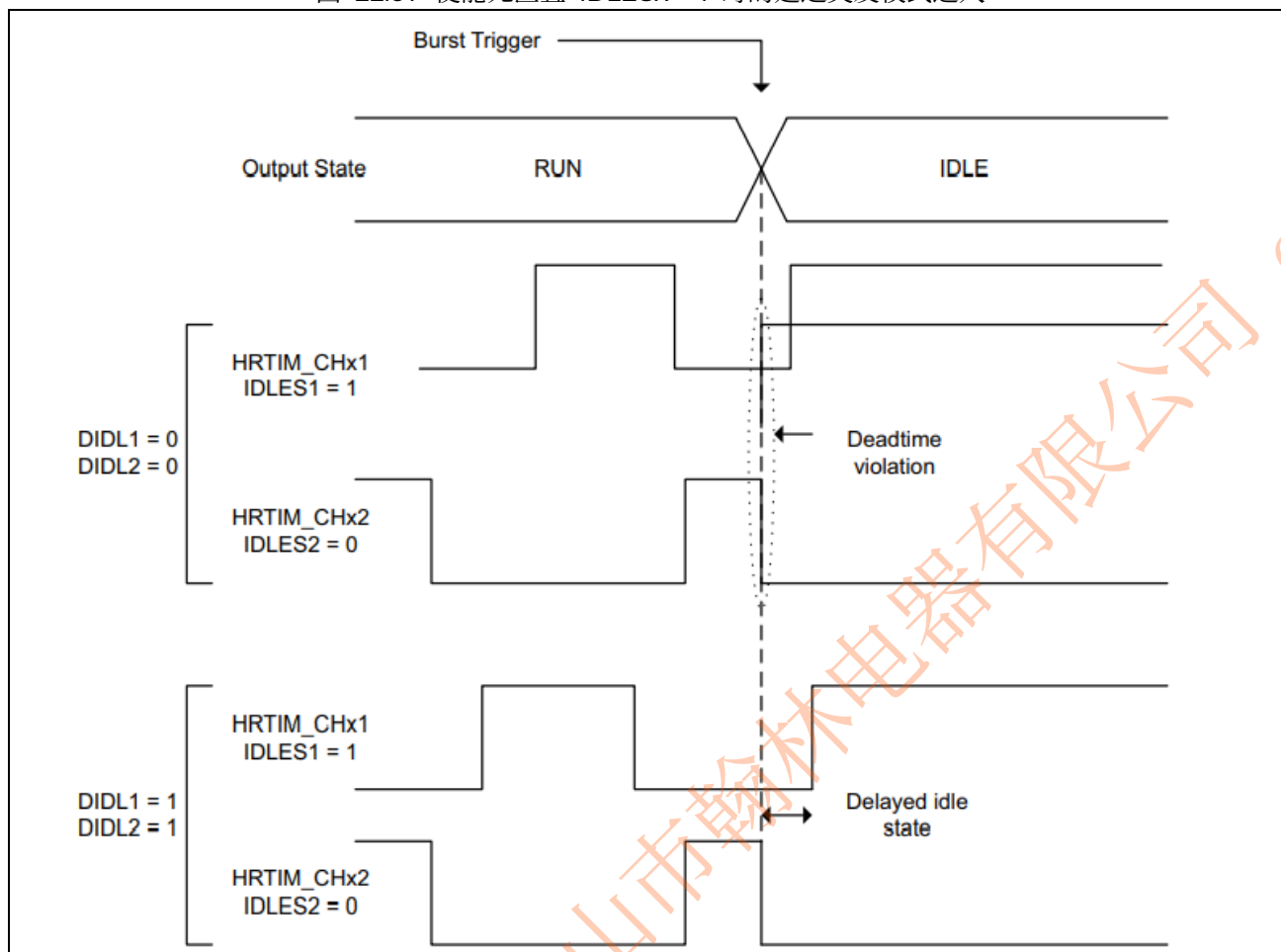
### 突发模式延迟进入

默认情况下，输出将在突发模式触发后立即获取其空闲电平（空闲电平取决于 IDLES1 和 IDLES2 设置）。

还可以延迟进入突发模式，并在输出获取其空闲状态之前的可编程周期内将输出强制为无效状态。驱动两路互补输出（其中一路输出具有有效空闲状态）时，可利用此特性避免超出死区，如下图所示。这样可避免半桥中出现击穿电流的风险，但会导致对突发模式进入的延迟响应。



图 22.57 使能死区且 IDLESx = 1 时的延迟突发模式进入



延迟突发进入模式通过 HRTIM\_OUTxR 寄存器中的 DIDLx 位使能（每路输出一个使能位）。该模式会在输出获取其空闲状态之前强制插入死区。每路 TIMx 输出都有自己的死区值：

- 当 DIDL1 = 1 时，为输出 1 上的 DTRx[8:0]
- 当 DIDL2 = 1 时，为输出 2 上的 DTFx[8:0]

仅当其中一路输出在突发模式期间具有有效空闲电平（IDLES=1）、且使用的死区为正值（SDTR/SDTF 设为 0）时，才可将 DIDLx 位置 1。

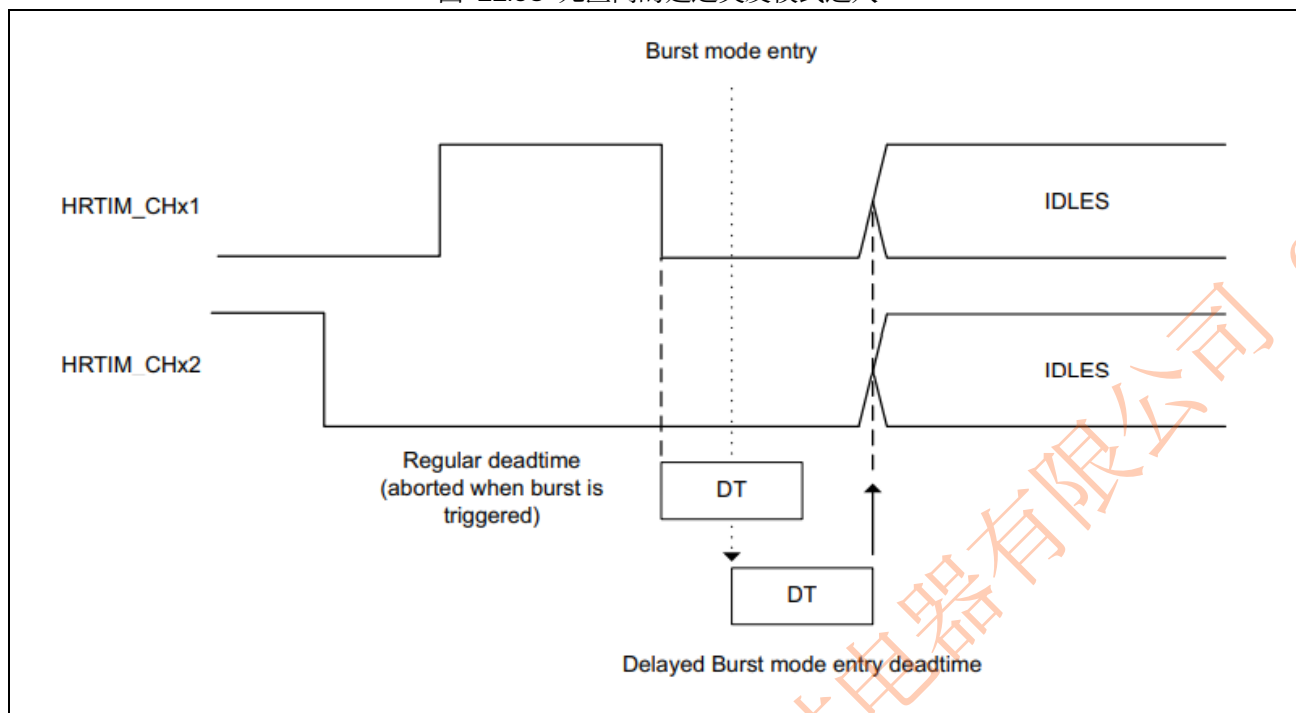
注：延迟突发进入模式使用死区发生器资源。因此，如果 2 个 DIDLx 位中有任何一位置 1，且相应的定时单元使用死区插入（HRTIM\_OUTxR 中的 DTEN 位置 1），则无法将 timerx 输出 2 用作外部事件过滤器（Tx2 过滤信号不可用）。

如果由 DTRx[8:0] 和 DTFx[8:0] 定义的持续时间小于 3 个  $f_{HRTIM}$  时钟循环周期，则必须应用第 7 小节中列出的与窄脉冲管理相关的限制。

如果突发模式进入在常规死区内到达，死区会中止，并会重新开始新的对应于无效周期的死区，如下图所示。



图 22.58 死区内的延迟突发模式进入



### 突发模式退出

突发模式退出通过软件强制进行（连续模式下），或在经过空闲周期后进行（单发模式下）。两种情况下，计数器都会立即重启（如果计数器通过 MTBM 或 TxBM 位=1 保持在复位状态），但输出状态仅会在编程的置位/复位事件后才会从空闲模式有效跳变为工作模式

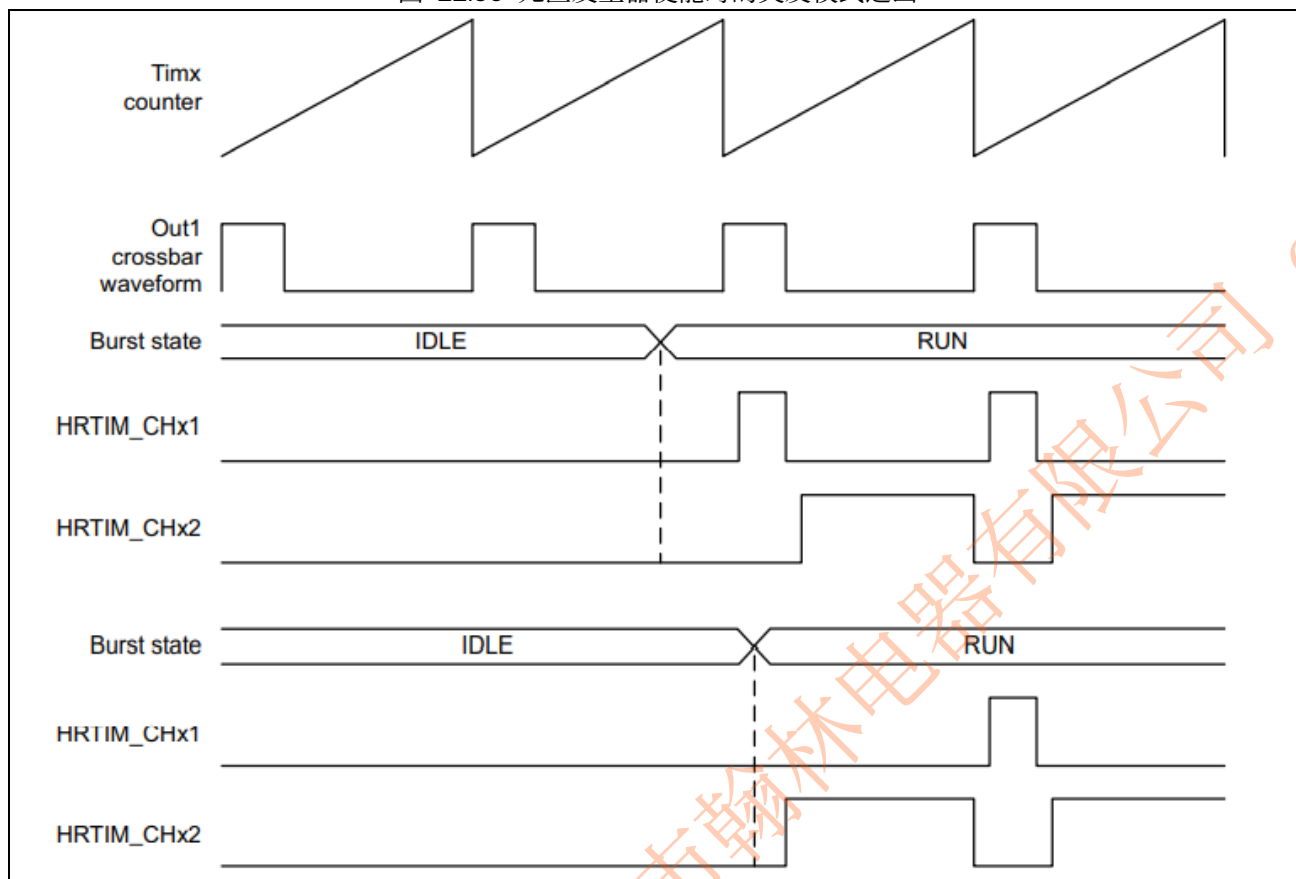
如果 HRTIM\_IER 寄存器中的 BMPERIE 使能位置 1，则会在单发模式和连续模式下生成突发周期中断。该中断可用于在连续突发模式下将突发模式退出与突发周期进行同步。

下图显示了死区使能后是如何恢复正常工作的。虽然突发模式退出是立即进行的，但仅对任何互补输出上的第一个置位事件有效。

图中显示了两种不同情况：

1. 纵横开关输出波形上的信号无效时，突发模式结束。Tx1 输出上发生置位事件时会恢复 Tx1 和 Tx2 的有效状态，Tx2 输出不会在退出突发模式时获取互补电平。
2. 突发模式在纵横开关输出波形有效时结束：Tx2 输出上发生置位事件时会恢复工作，Tx1 不会在退出突发模式时立即获取有效电平。

图 22.59 死区发生器使能时的突发模式退出



如果使能了推挽模式，上述行为会略有不同。如果输出无效，推挽模式会在周期开始使强制进行输出复位，如果前一周期内输出为高电平，则会对称地将输出强设置为有效电平。

因此，即使未明确编程跳变，也可在退出突发模式时复位空闲状态有效的输出。出于对称性的原因，即使未明确编程跳变，只要输出进入空闲状态时为有效电平，即可在退出突发模式时将输出置 1。

### 突发模式寄存器预装载和更新

当 **BMPREN** 位（突发模式预装载使能）可进行突发模式比较并预装载周期寄存器（**HRTIM\_BMCMP** 和 **HRTIM\_BMPER**）。

当 **BMPREN** 置 1 时，预装载寄存器的内容会传输到活动寄存器：

- 当突发模式被使能时（**BME** = 1），
- 在突发模式周期结束时。

如果对 **HRTIM\_BMPER** 周期寄存器进行写操作，则会暂时禁止更新，直到向 **HRTIM\_BMCMP** 比较寄存器进行写操作为止，这样可确保两个寄存器在进行修改时是一致的。

如果只需要更改比较寄存器，则只需要进行一次写操作。如果只需要更改周期，还需要重新写入比较寄存器，以便将新值纳入考虑范围。

当 BMPREN 位复位时，对 BMCMPR 和 BMPER 进行写访问会直接更新活动寄存器。此时，对于以下 2 种情况，需要考虑何时在总突发周期内进行更新：

a) 比较寄存器更新

如果新的比较值大于当前突发模式计数器值，则会在当前周期内考虑新的比较值。

如果新的比较值小于当前突发模式计数器值，在连续模式下，会在下一个周期内考虑新的比较值；在单发模式下，则会忽略新的比较值（不会出现比较匹配的情况，并将持续处于空闲状态，直至空闲周期结束）。

b) 周期寄存器更新

如果新的周期值大于当前突发模式计数器值，则会在当前周期内考虑此更改。

注：如果新的周期值小于当前突发模式计数器值，则不会考虑新的周期值，突发模式计数器将溢出（达到 0xFFFF 时溢出），更改将在下一周期生效。在单发模式下，计数器将在达到 0xFFFF 时翻转，突发模式将重新启动并再持续一个周期，直到达到新的编程值。

### 使用复合寄存器模拟突发模式

突发模式控制器仅会控制单个转换器的一个定时器或一组定时器。需要为多个独立定时器使用突发模式时，可以使用 DMA 和 HRTIM\_CMP1CxR 复合寄存器仿真简单的突发模式控制器，复合寄存器会保存重复寄存器和比较 1 寄存器的别名。

此方法适用于仅需要使用简单 PWM 来更新占空比的转换器（通常是降压转换器）。在这种情况下，使用 CMP1 寄存器复位输出（并定义占空比），而输出会在发生周期事件时置 1。

此时，对 CMP1CxR 进行一次 32 位写访问便足以定义占空比（使用 CMP1 值）以及保持该占空比的周期数（使用重复值）。要实现突发模式，只需要在连续模式下通过 DMA（在发生重复事件时）传输两个 32 位数据，数据结构如下：

CMPC1xR = {REP\_Run; CMP1 = Duty\_Cycle}, {REP\_Idle; CMP1 = 0}

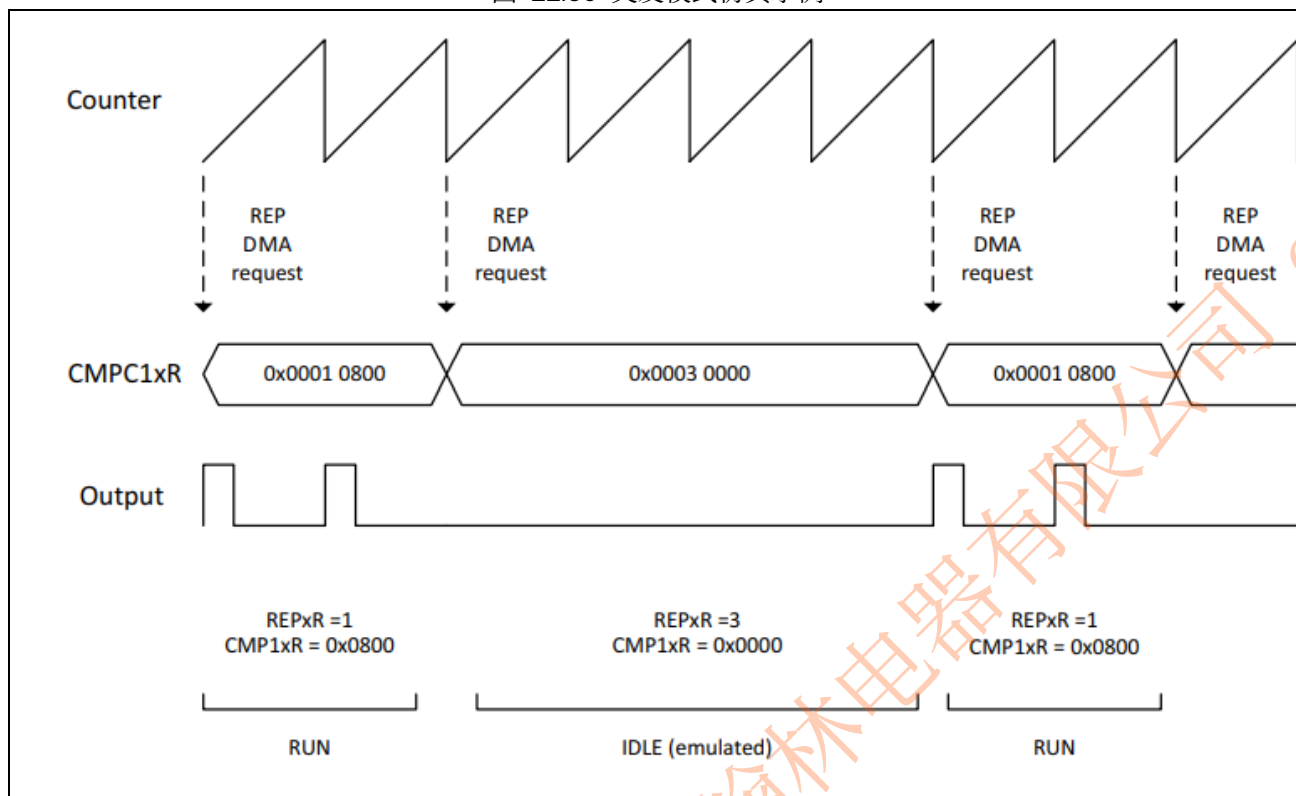
举例来说，数值：

{0x0003 0000}: CMP1 = 0, 持续 3 个周期

{0x0001 0800}: CMP1 = 0x0800, 持续 1 个周期

提供的突发模式每 6 个 PWM 周期会有 2 个周期有效，如下图所示。

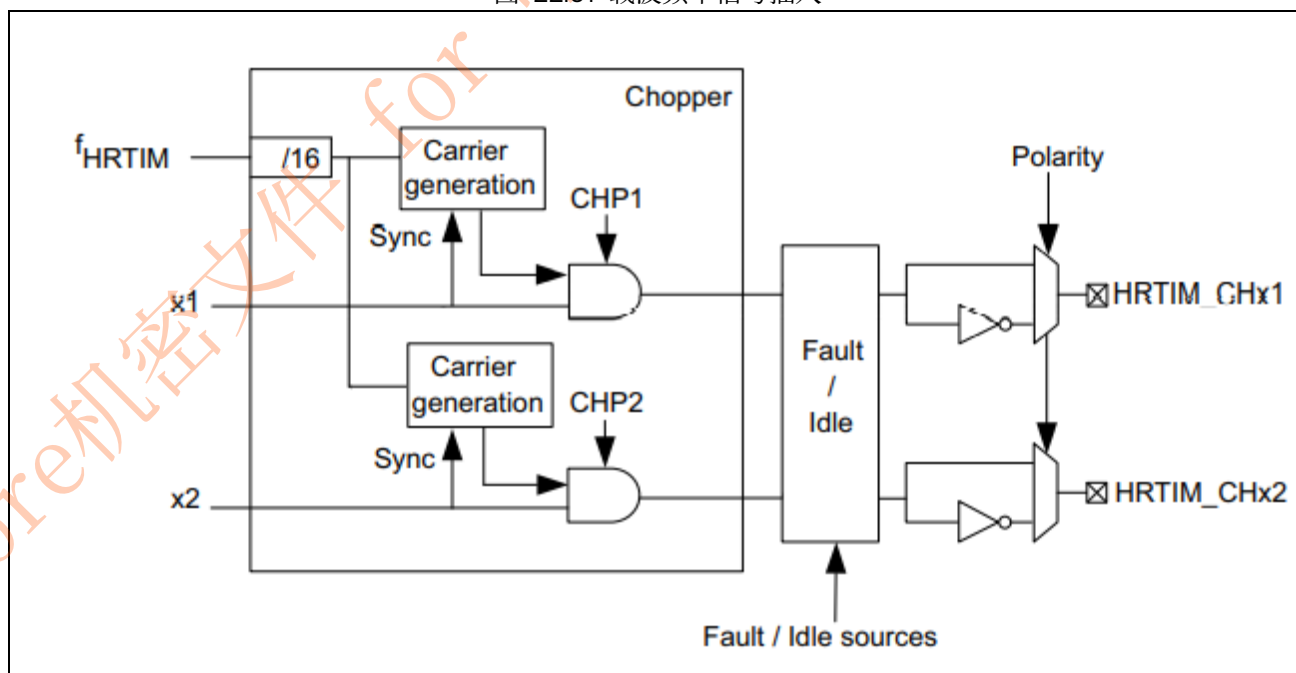
图 22.60 突发模式仿真示例



### 22.3.16 斩波

可在定时单元输出信号上添加高频载波以驱动隔离变压器。此操作是在插入极性之前对输出级执行的(如下图所示), 会使用 HRTIM\_OUTxR 寄存器中的 CHP1 和 CHP2 位分别在输出 1 和输出 2 上使能斩波。

图 22.61 载波频率信号插入



可通过 HRIM\_CHPxR 寄存器调整斩波参数，以在脉冲开始处定义一个特定脉宽，后跟频率和占空比可编程的载波频率，如下图所示。

CARFRQ[3:0] 位按照公式  $F_{CHPFRQ} = f_{HRTIM} / (16 \times (CARFRQ[3:0] + 1))$  定义频率，范围从 12 MHz 到 750 KHz（对于  $f_{HRTIM} = 192 \text{ MHz}$ ）。

可通过 CARDTY[2:0] 调整占空比（步长为 1/8），占空比范围为 0/8 到 7/8。CARDTY[2:0] = 000 时（占空比 = 0/8），输出波形仅包含参考波形上升沿之后的启动脉冲，不会添加任何载波。

初始脉冲的脉宽是通过 STRPW[3:0] 位域定义的，具体如下：

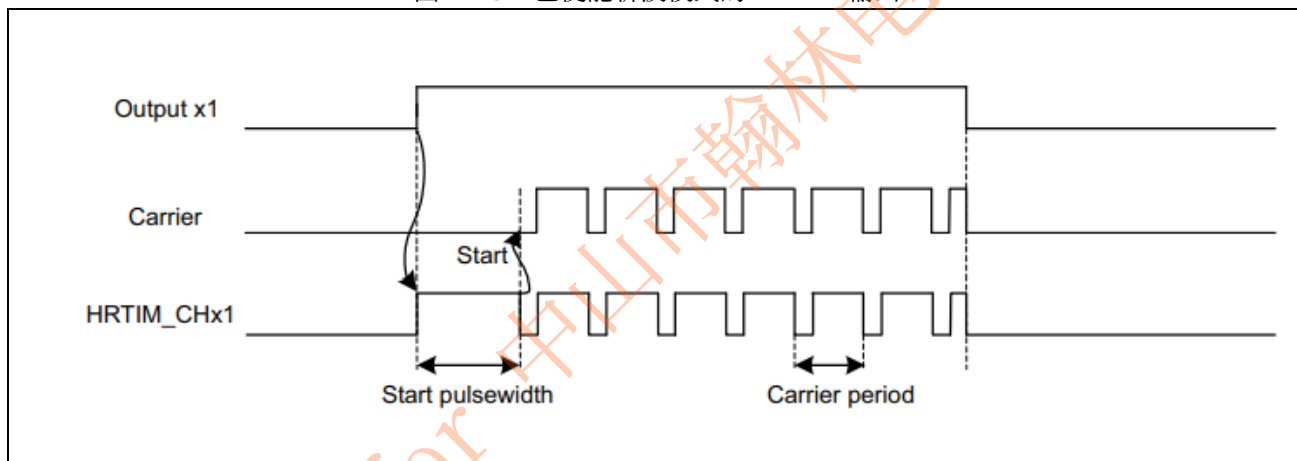
$t_{1STRPW} = (STRPW[3:0] + 1) \times 16 \times t_{HRTIM}$ 。脉宽范围为 83.3 ns 到 1.33  $\mu\text{s}$ （对于  $f_{HRTIM} = 192 \text{ MHz}$ ）。

载波频率参数是根据  $f_{HRTIM}$  频率定义的，与 CKPSC[2:0] 设置无关。

在斩波模式下，载波频率和初始脉宽会通过逻辑与函数与参考波形相结合。初始脉冲结束时执行同步，以获得重复的信号波形。

斩波信号会在输出波形有效状态结束时停止，不会等到当前载波周期结束。因此，斩波信号包含的脉冲可能比编程的脉冲短。

图 22.62 已使能斩波模式的 HRTIM 输出



注：必须在通过 HRTIM\_OENR 寄存器中的 TxyOEN 位使能输出之前将 CHP1 和 CHP2 位置 1。斩波模式激活时（两个 CHPx 位中至少有一个置 1），不能修改 CARFRQ[2:0]、CARDTY[2:0] 和 STRPW[3:0] 位域。

### 22.3.17 故障保护

HRTIMER 具有通用的故障保护电路，可在发生异常操作时禁止输出。一旦触发故障，输出便会进入预定义的安全状态。输出通过软件重新使能之前，都会保持此状态。如果是永久故障请求，即使软件尝试重新使能输出，输出也将保持其故障状态，直至故障源消失。

HRTIM 具有 6 个 FAULT 输入通道；所有通道均可用，并可结合起来用于 6 个定时单元中的每一个，如下图所示。

[illegible]

下表总结了 6 个故障通道的每个通道可用的信号来源:

| 故障通道            | 外部输入<br>FLT <sub>x</sub> SRC[1:0] = 00 | 片上源<br>FLT <sub>x</sub> SRC[1:0] = 01 | 外部输入<br>FLT <sub>x</sub> SRC[1:0] = 10 | 片上源<br>FLT <sub>x</sub> SRC[1:0] = 11 |
|-----------------|----------------------------------------|---------------------------------------|----------------------------------------|---------------------------------------|
| hrtim_flt1[4:1] | HRTIM_FLT1                             | 比较器输出                                 | EEV1_muxout                            | N/A                                   |
| hrtim_flt2[4:1] | HRTIM_FLT2                             | 比较器输出                                 | EEV2_muxout                            | N/A                                   |
| hrtim_flt3[4:1] | HRTIM_FLT3                             | 比较器输出                                 | EEV3_muxout                            | N/A                                   |
| hrtim_flt4[4:1] | HRTIM_FLT4                             | 比较器输出                                 | EEV4_muxout                            | N/A                                   |
| hrtim_flt5[4:1] | HRTIM_FLT5                             | 比较器输出                                 | EEV5_muxout                            | N/A                                   |
| hrtim_flt6[4:1] | HRTIM_FLT6                             | 比较器输出                                 | EEV6_muxout                            | N/A                                   |

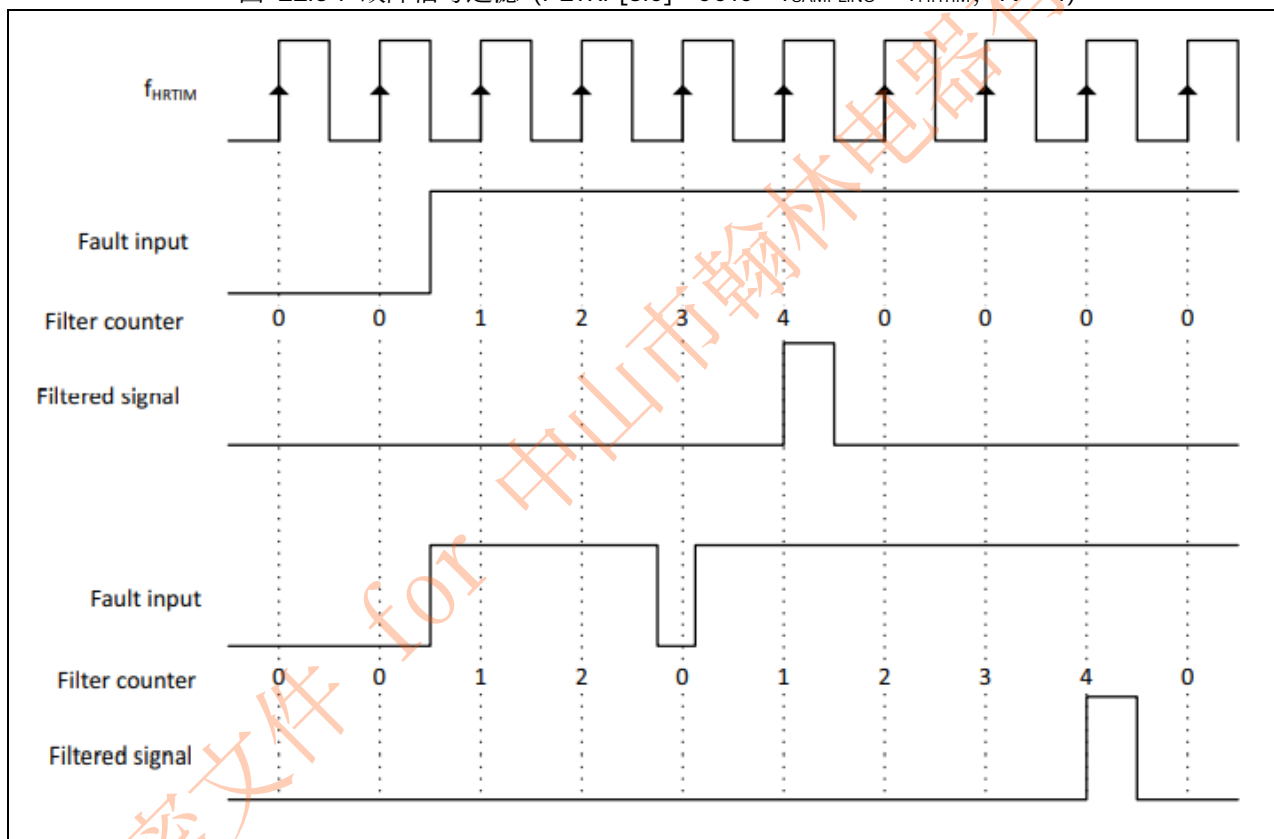
上述表中提到的  $EEVx\_muxout$  事件是在  $hrtim\_eevx[4:1]$  输入复用器之后产生的，该复用器由  $EExSRC[1:0]$  位控制。

可通过  $HRTIM\_FLTINRx$  寄存器中的  $FLTxFP$  极性位选择信号的极性，以定义有效电平。如果  $FLTxFP = 0$ ，信号低电平有效；如果  $FLTxFP = 1$ ，信号高电平有效。

可在极性设置后过滤故障信息。如果  $FLTxF[3:0]$  位域设为 0000，则不会对信号进行过滤，信号将独立于  $f_{HRTIM}$  时钟异步操作。对于所有其他  $FLTxF[3:0]$  位域值，会对信号进行数字过滤。数字滤波器由计数器组成，需要使用  $N$  个有效样本来验证输出跳变。如果输入值在计数器达到  $N$  值之前发生变化，计数器会复位，跳变会被丢弃（视为伪事件）。如果计数器达到  $N$ ，跳变被视为有效，并会作为正确的外部事件进行传输。

因此，数字滤波器会向正在进行滤波的外部事件添加延迟，延迟时长视采样时钟和滤波器长度（预期的有效样本数）而定。下图显示了伪故障信号的过滤方式。

图 22.64 故障信号过滤 ( $FLTxF[3:0] = 0010$ :  $f_{SAMPLING} = f_{HRTIM}$ ,  $N = 4$ )



过滤周期从 2 个  $f_{HRTIM}$  时钟周期到 8 个 32 分频的  $f_{FLTS}$  时钟周期。 $f_{FLTS}$  通过  $HRTIM\_FLTINR2$  寄存器中的  $FLTSD[1:0]$  位定义。

下表总结了采样速率和滤波器长度。必须将滤波器长度减去 1 个采样时钟周期的抖动，以考虑因采样造成的不确定性，从而实现有效过滤。



表 22.28 采样速率和滤波器长度与 FLTFxF[3:0]和时钟设置的关系

| -<br>FLTFxF[3:0] | f <sub>FLTS</sub> 与 FLTSD[1:0] |                         |                          |                          | 当 f <sub>HRTIM</sub> =192 MHz 时的滤波器长度    |                                             |
|------------------|--------------------------------|-------------------------|--------------------------|--------------------------|------------------------------------------|---------------------------------------------|
|                  | 00                             | 01                      | 10                       | 11                       | 最小                                       | 最大                                          |
| 0001,0010,0011   | f <sub>HRTIM</sub>             | f <sub>HRTIM</sub>      | f <sub>HRTIM</sub>       | f <sub>HRTIM</sub>       | f <sub>HRTIM</sub> , N = 2<br>10.4 ns    | f <sub>HRTIM</sub> , N = 8<br>41.7 ns       |
| 0100, 0101       | f <sub>HRTIM</sub> / 2         | f <sub>HRTIM</sub> / 4  | f <sub>HRTIM</sub> / 8   | f <sub>HRTIM</sub> / 16  | f <sub>HRTIM</sub> / 2, N = 6<br>62.5 ns | f <sub>HRTIM</sub> / 16, N = 8<br>667 ns    |
| 0110, 0111       | f <sub>HRTIM</sub> / 4         | f <sub>HRTIM</sub> / 8  | f <sub>HRTIM</sub> / 16  | f <sub>HRTIM</sub> / 32  | f <sub>HRTIM</sub> / 4, N = 6<br>125 ns  | f <sub>HRTIM</sub> / 32, N = 8<br>1.33 μs   |
| 1000, 1001       | f <sub>HRTIM</sub> / 8         | f <sub>HRTIM</sub> / 16 | f <sub>HRTIM</sub> / 32  | f <sub>HRTIM</sub> / 64  | f <sub>HRTIM</sub> / 8, N = 6<br>250 ns  | f <sub>HRTIM</sub> / 64, N = 8<br>2.67 μs   |
| 1010, 1011, 1100 | f <sub>HRTIM</sub> / 16        | f <sub>HRTIM</sub> / 32 | f <sub>HRTIM</sub> / 64  | f <sub>HRTIM</sub> / 128 | f <sub>HRTIM</sub> / 16, N = 5<br>417 ns | f <sub>HRTIM</sub> / 128, N = 8<br>5.33 μs  |
| 1101, 1110, 1111 | f <sub>HRTIM</sub> / 32        | f <sub>HRTIM</sub> / 64 | f <sub>HRTIM</sub> / 128 | f <sub>HRTIM</sub> / 256 | f <sub>HRTIM</sub> / 32, N = 5<br>833 ns | f <sub>HRTIM</sub> / 256, N = 8<br>10.67 μs |

### 故障消隐和事件计数

故障输入可以暂时禁用，以消隐伪故障事件。消隐源列在以下表中。

表 22.29 故障输入消隐事件

| -<br>故障输入       | FLTxBLS = 0, 重置对齐窗口 |                 | FLTxBLS = 1 移动窗口 |                 |
|-----------------|---------------------|-----------------|------------------|-----------------|
|                 | 消隐窗口开始              | 消隐窗口结束          | 消隐窗口开始           | 消隐窗口结束          |
| hrtim_flt1[4:1] | Timer A 重置/翻转       | Timer A CMP3 事件 | Timer A CMP4 事件  | Timer A CMP3 事件 |
| hrtim_flt2[4:1] | Timer B 重置/翻转       | Timer B CMP3 事件 | Timer B CMP4 事件  | Timer B CMP3 事件 |
| hrtim_flt3[4:1] | Timer C 重置/翻转       | Timer C CMP3 事件 | Timer C CMP4 事件  | Timer C CMP3 事件 |
| hrtim_flt4[4:1] | Timer D 重置/翻转       | Timer D CMP3 事件 | Timer D CMP4 事件  | Timer D CMP3 事件 |
| hrtim_flt5[4:1] | Timer E 重置/翻转       | Timer E CMP3 事件 | Timer E CMP4 事件  | Timer E CMP3 事件 |
| hrtim_flt6[4:1] | Timer F 重置/翻转       | Timer F CMP3 事件 | Timer F CMP4 事件  | Timer F CMP3 事件 |

故障计数器还允许忽略多个伪故障事件并定义接受标准。

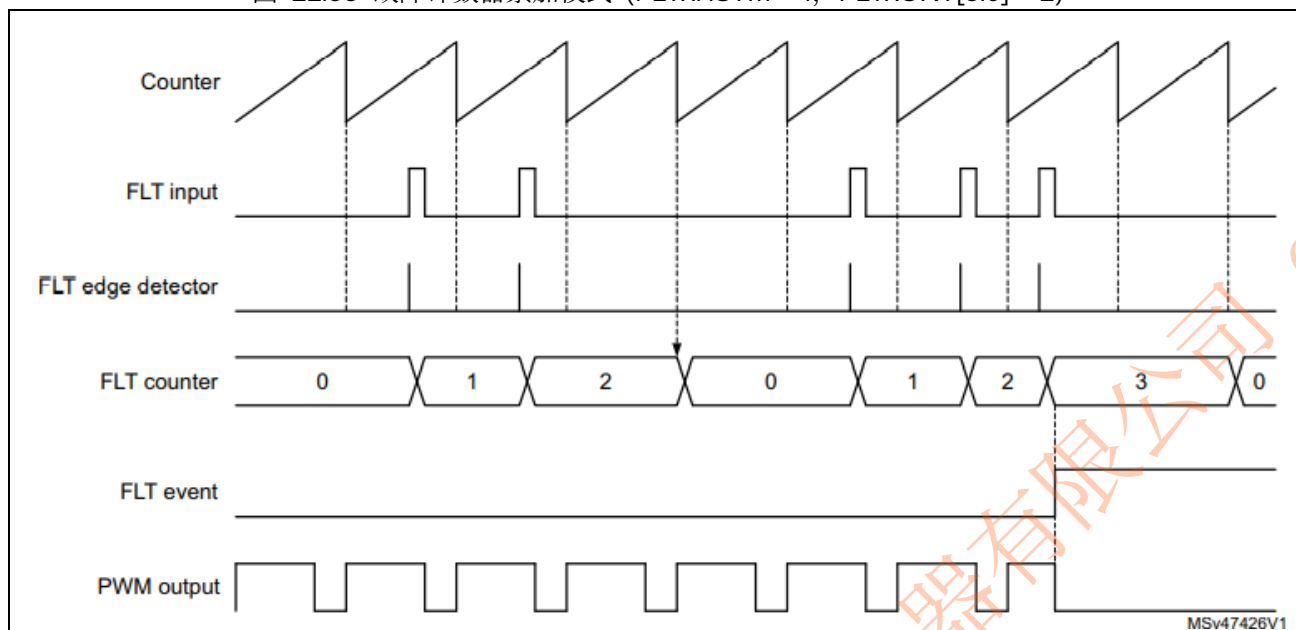
FLTxCNT[3:0]位域选择 FAULTx 计数器的阈值。当事件数量等于 FLTxCNT[3:0]的值时，故障被视为有效。

FLTxRSTM 选择 FAULTx 计数器的复位模式：

- 0：在复位/溢出事件时，故障计数器硬件复位，如下表所示。
- 1：仅在每个复位/溢出事件发生时，如果在上一个计数周期内没有事件发生，故障计数器才会复位，如下图所示。

故障计数器可以在任何时候通过软件使用 FLTxCRES 位进行复位。

图 22.65 故障计数器累加模式 (FLT<sub>x</sub>RSTM = 1, FLT<sub>x</sub>CNT[3:0] = 2)



给定的 FLT<sub>x</sub> 输入计数器可以由单个源进行复位。下表指示了与特定故障关联的定时器单元。这并不意味着故障线不能由多个定时器共享（例如，FLT1 带有使能的事件计数器，可以同时作用于定时器 A、定时器 B 和定时器 C）。

表 22.30 故障 1 至 6 计数器复位源

| 故障输入            | 故障计数复位源       |
|-----------------|---------------|
| hrtim_flt1[4:1] | Timer A 复位/翻转 |
| hrtim_flt2[4:1] | Timer B 复位/翻转 |
| hrtim_flt3[4:1] | Timer C 复位/翻转 |
| hrtim_flt4[4:1] | Timer D 复位/翻转 |
| hrtim_flt5[4:1] | Timer E 复位/翻转 |
| hrtim_flt6[4:1] | Timer F 复位/翻转 |

#### 系统故障输入 (hrtim\_sys\_flt)

此故障由 MCU 的 B 类电路提供（有关详细信息，请参见系统配置控制器 (SYSCFG) 部分），对应于来自以下源的系统故障：

- 时钟安全系统
- SRAM 奇偶校验器
- 带 FPU 的 Cortex®-M4-lockup 信号
- PVD 检测器
- Flash ECC 双错误检测

此输入会覆盖 FAULT 输入，并禁用所有 FAULT<sub>y</sub>[1:0] = 01、10、11 的输出。

对于每条 FAULT 通道，可通过 HRTIM\_FLTxR 寄存器中仅可写入一次的 FLT<sub>x</sub>LCK 位锁定 FLT<sub>x</sub>E、FLT<sub>x</sub>P、FLT<sub>x</sub>SRC、FLT<sub>x</sub>F[3:0]位（使其成为只读位），以实现功能安全。使能后，故障调节设置会冻结，直到下一次 HRTIM 复位或系统复位。

按上述方法对故障信号进行调节后，信号会发送到定时单元。对于任一定时单元，都会使用 HRTIM\_FLTxR 寄存器中的 FLT1EN 到 FLT6EN 位使能 6 条故障通道，并且可同时选择 6 条通道（只要输出受故障机制保护，便会自动使能 sysfault）。这样便可实现：

- 一条故障通道同时禁止多个定时单元
- 多条故障通道进行或运算来禁止单个定时单元

HRTIM\_FLTxR 寄存器中仅可写入一次的 FLT<sub>x</sub>LCK 位可在下一次复位之前锁定 FLT<sub>x</sub>EN 位（使其成为只读位），以实现功能安全。使能后，定时单元故障相关设置会冻结，直到下一次 HRTIM 复位或系统复位。

对于每个定时器，故障期间的输出状态是通过 HRTIM\_OUTxR 寄存器中的 FAULT1[1:0]和 FAULT2[1:0]位定义的。

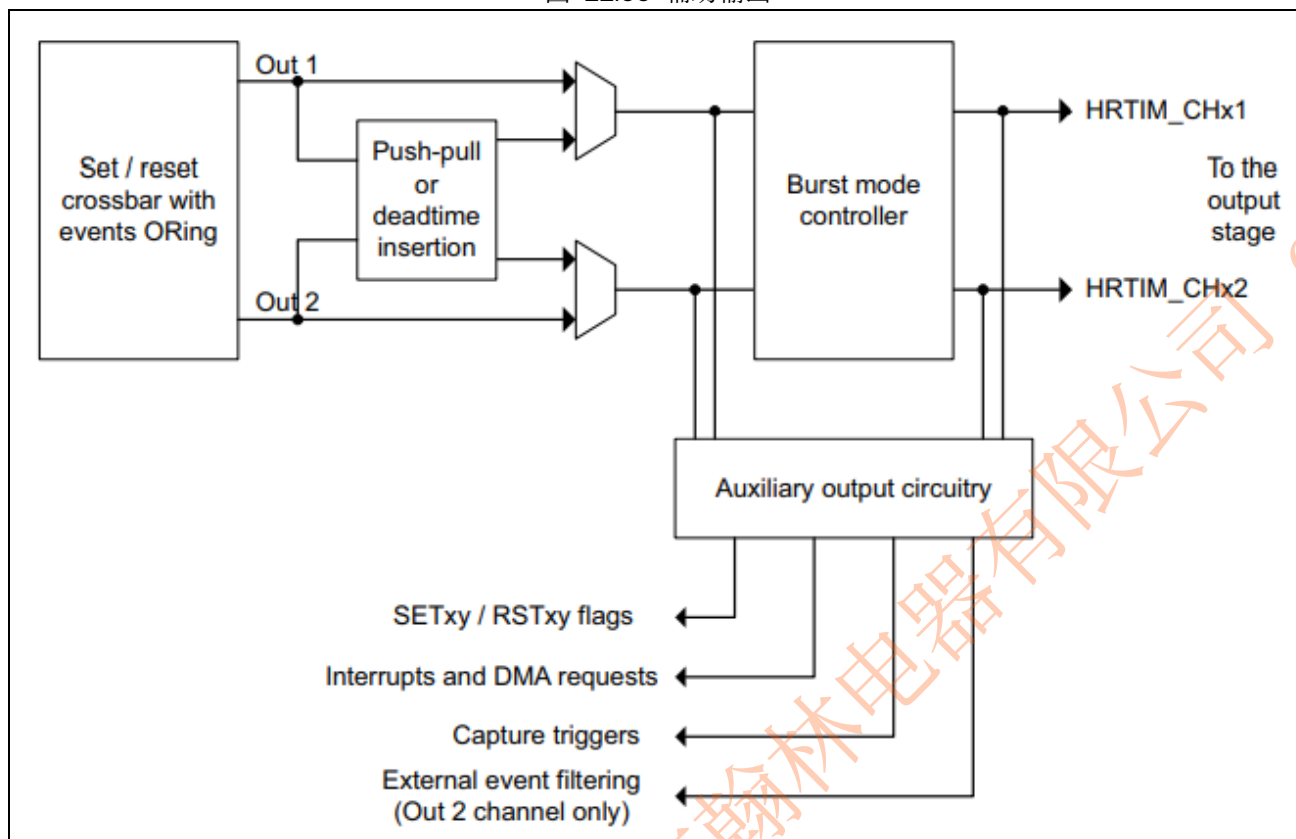
### 22.3.18 辅助输出

定时器 A 和 E 所具有的辅助输出与进入输出级的常规输出并行工作。辅助输出可提供以下内部状态、事件和信号：

- SET<sub>xy</sub> 和 RST<sub>xy</sub> 状态标志，以及相应的中断和 DMA 请求
- 输出置位/复位后的捕获触发事件
- Tx2 输出复制后的外部事件过滤信号

辅助输出会在突发模式控制器之前或之后获取，具体视 HRTIM 工作模式而定。下图给出了概览。

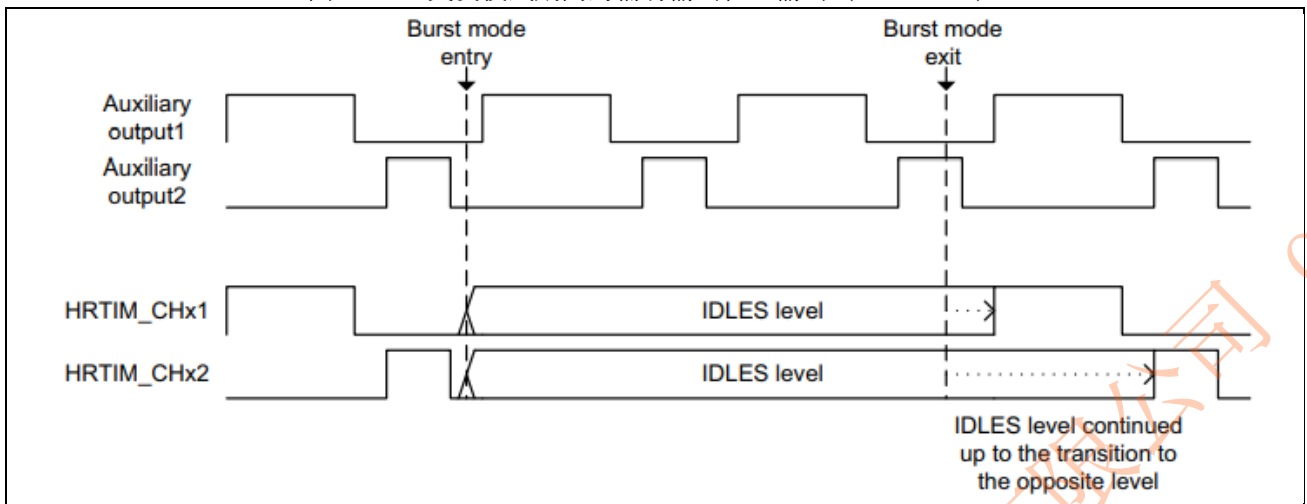
图 22.66 辅助输出



默认情况下，辅助输出是输出 Tx1 和 Tx2 的副本。例外情况包括：

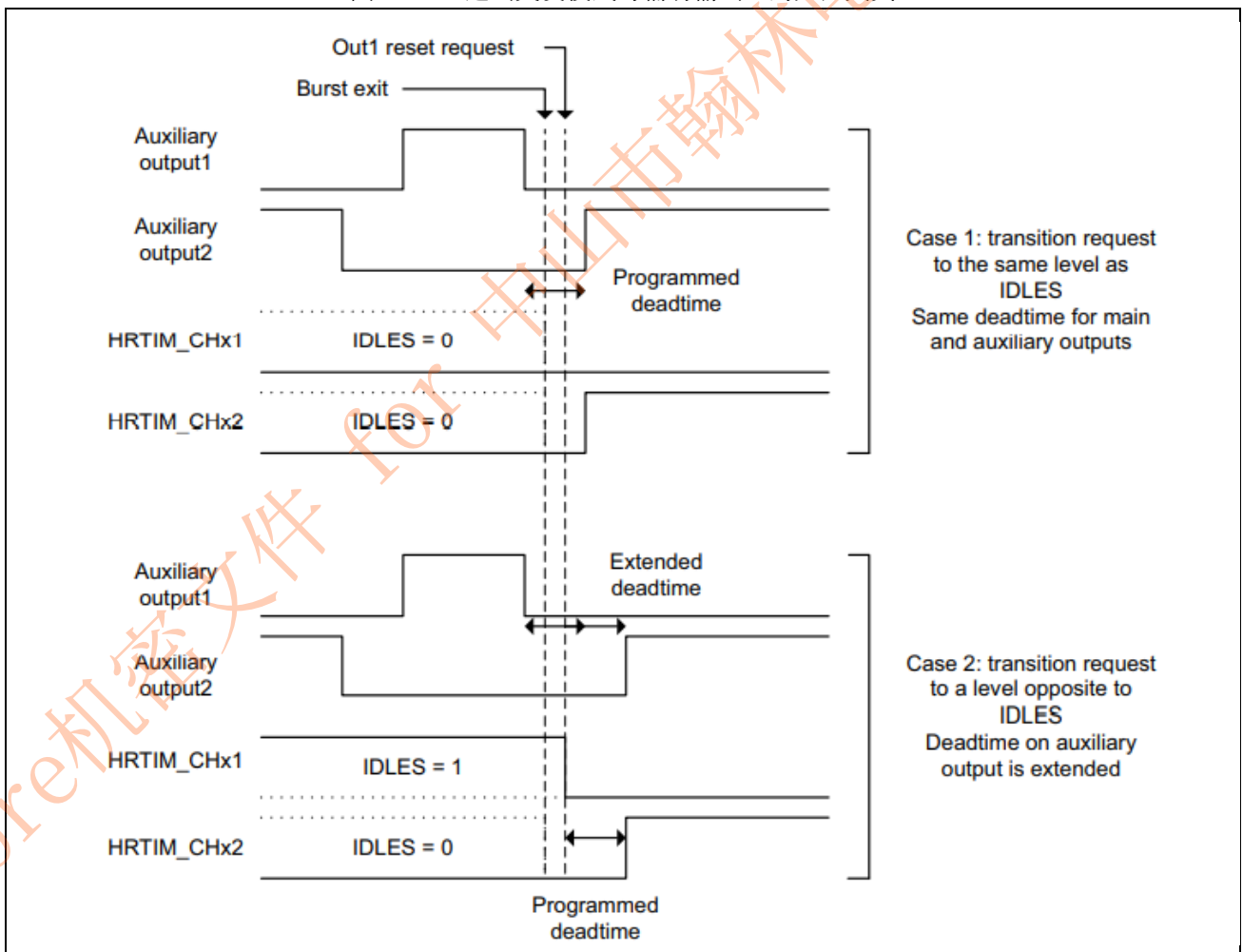
- 禁止死区时 ( $DTEN = 0$ ) 的延迟空闲和均衡空闲保护。触发保护后，辅助输出会保持不变，并会遵循纵横开关发出的信号。相反，如果使能了死区 ( $DTEN = 1$ )，主输出和辅助输出都会强制设为无效电平。
- 突发模式 ( $TCEN=1$ ,  $IDLEMx=1$ )；有两种情况：
  - a) 如果  $DTEN=0$  或  $DIDLx=0$ ，辅助输出将不受突发模式进入的影响，而会继续遵循纵横开关发出的参考信号（请参见下图）。
  - b) 如果同时使能了死区 ( $DTEN=1$ ) 和延迟突发模式进入 ( $DIDLx=1$ )，则辅助输出与主输出的行为相同。它们会在死区持续时间结束后强制设为 IDLES 电平，随后会在所有突发周期内保持这一电平。突发模式终止后，会保持 IDLES 电平，直至跳变到相反电平（与主输出相似）。

图 22.67 突发模式期间的辅助输出和主输出 (DIDLx = 0)



如果在死区内发生从突发模式退出或者在延迟保护后重新使能输出，则辅助输出上的信号会略有失真。这种情况下，应用到辅助输出的死区会延长，以便能够符合主输出上的死区要求。下图给出了一些例子。

图 22.68 退出突发模式时辅助输出上的死区失真



### 22.3.19 将 HRTIM 与其他定时器或 HRTIM 实例同步

HRTIM 可作为主单元（生成同步信号）或从单元（等待触发被同步）来同步多个 HRTIM 实例。此功能也可用于将 HRTIM 与其他外部或片上定时器进行同步。同步电路在主定时器内进行控制。

#### 同步输出

本节介绍了必须对 HRTIM 进行何种配置才能同步外部资源并充当主定时器单元。

可选择四种事件作为要发送到同步输出的源, 方法是使用 HRTIM\_MCR 寄存器中的 SYNCSRC[1:0]位, 具体如下:

- 00: 主定时器启动  
当 MCEN 位置 1, 或者定时器在单发模式下达到周期值后重新启动时, 会生成该事件。  
如果计数期间发生复位 (CONT 或 RETRIG 位置 1), 也会生成该事件。
- 01: 主定时器比较 1 事件
- 10: 定时器 A 启动  
当 TACEN 位置 1, 或者计数器复位并重新开始计数 (响应此次复位) 时, 会生成该事件。以下计数器复位事件不会传播到同步输出: 连续模式下的计数器翻转、单发不可再触发模式下被丢弃的复位请求。仅当计数期间发生复位时 (CONT 或 RETRIG 位置 1), 才会考虑复位。
- 11: 定时器 A 比较 1 事件

HRTIM\_MCR 寄存器中的 SYNCOUT[1:0]位指定了同步事件的生成方式。

如果 SYNCOUT[1:0] = 1x, 则会在 HRTIM\_SCOUT 输出引脚上生成同步脉冲。SYNCOUT[0]位指定了同步信号的极性。如果 SYNCOUT[0] = 0, HRTIM\_SCOUT 引脚具有低空闲电平, 并会发出长度为 16 个  $f_{HRTIM}$  时钟周期的正脉冲进行同步。如果 SYNCOUT[0] = 1, 空闲电平为高电平, 并会生成负脉冲。

注: 同步脉冲后会有 16 个  $f_{HRTIM}$  时钟周期的空闲电平, 在此期间, 会丢弃任何新的同步请求。因此, 最大同步频率为  $f_{HRTIM}/32$ 。

SYNCOUT[1:0]位使能后 (也就是位域值不是 00 后), 会立即在 HRTIM\_SCOUT 引脚施加空闲电平。

必须在配置 MCU 输出和计数器使能之前执行同步输出初始化步骤, 具体执行步骤如下:

1. 配置 HRTIM\_MCR 中的 SYNCOUT[1:0]和 SYNCSRC[1:0]位域
2. 配置 HRTIM\_SCOUT 引脚 (请参见通用 I/O 部分)
3. 使能主定时器或定时器 A 计数器 (MCEN 或 TACEN 位置 1)

使能同步输入模式并同时启动计数器 (使用 SYNCSTRTM/SYNCSTRTx 位) 和同步输出模式 (SYNCSRC[1:0] = 00 或 10) 后, 仅当计数器将在运行时启动或复位时, 才会生成输出脉冲。如果复位请求将计数器清零而不启动计数器, 则不会影响同步输出。



## 同步输入

HRTIM 可通过外部源进行同步，具体通过对 HRTIM\_MCR 寄存器中的 SYNCIN[1:0] 位进行编程来实现：

- 00：禁止同步输入
- 01：保留配置
- 10：片上定时器 TRGO 输出连接到 hrtim\_in\_sync[2]输入。
- 11：HRTIM\_SCIN 输入引脚（hrtim\_in\_sync[3]）上的正脉冲

目标定时器（主定时器或定时单元）使能后（MCEN 和/或 TxCEN 位置 1），不能更改此位域。

HRTIM\_SCIN 输入为上升沿有效。定时器行为是通过 HRTIM\_MCR 和 HRTIM\_TIMxCr 寄存器中的下列位定义的（有关详细信息，请参见下表）：

- 同步启动：传入信号会启动定时器的计数器（SYNCSTRM 和/或 SYNCSTRTx 位置 1）。TxCEN（MCEN）位必须置 1 才能使能定时器并使计数器准备好启动。在连续模式下，计数器在收到同步信号之前不会启动。
- 同步复位：传入信号会复位计数器（SYNCRSTM 和/或 SYNCRSTx 位置 1）。该事件会像其他任何复位事件一样使重复计数器递减。

仅当相关计数器使能后（MCEN 或 TxCEN 位置 1），才考虑同步事件。同步请求会触发 SYNC 中断。

注：如果当前计数器值大于有效周期值，同步启动事件会复位计数器。

同步事件的作用取决于定时器工作模式，请参见下表中总结的内容。

表 22.31 同步事件的作用与定时器工作模式之间的关系

| 工作模式    | SYNC RSTx | SYNC STRTx | SYNC 复位或启动事件后的行为                                                                                                                                                                                                                                |
|---------|-----------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 单次不可再触发 | 0         | 1          | <p>当计数器停止并且满足以下条件时，会考虑启动事件：</p> <ul style="list-style-type: none"> <li>- 当 MCEN 或 TxCEN 位被置 1 时</li> <li>- 已达到周期时间。</li> </ul> <p>如果在计数器因达到周期值而停止时发生启动事件，则会将计数器复位。复位请求会将计数器清零，但不会启动计数器（计数器仅可通过同步事件重启）。任何在计数期间发生的复位事件都会被忽略（与在常规不可再触发模式下时一样）。</p> |
|         | 1         | X          | <p>复位事件将启动定时器计数操作。仅当计数器停止并且满足以下条件时，才会考虑复位事件：</p> <ul style="list-style-type: none"> <li>- 当 MCEN 或 TxCEN 位被置 1 时</li> <li>- 已达到周期时间。</li> </ul> <p>如果选择了多个复位请求（来自 HRTIM_SCIN 和内部事件），则仅会考虑第一个到达的请求。</p>                                          |

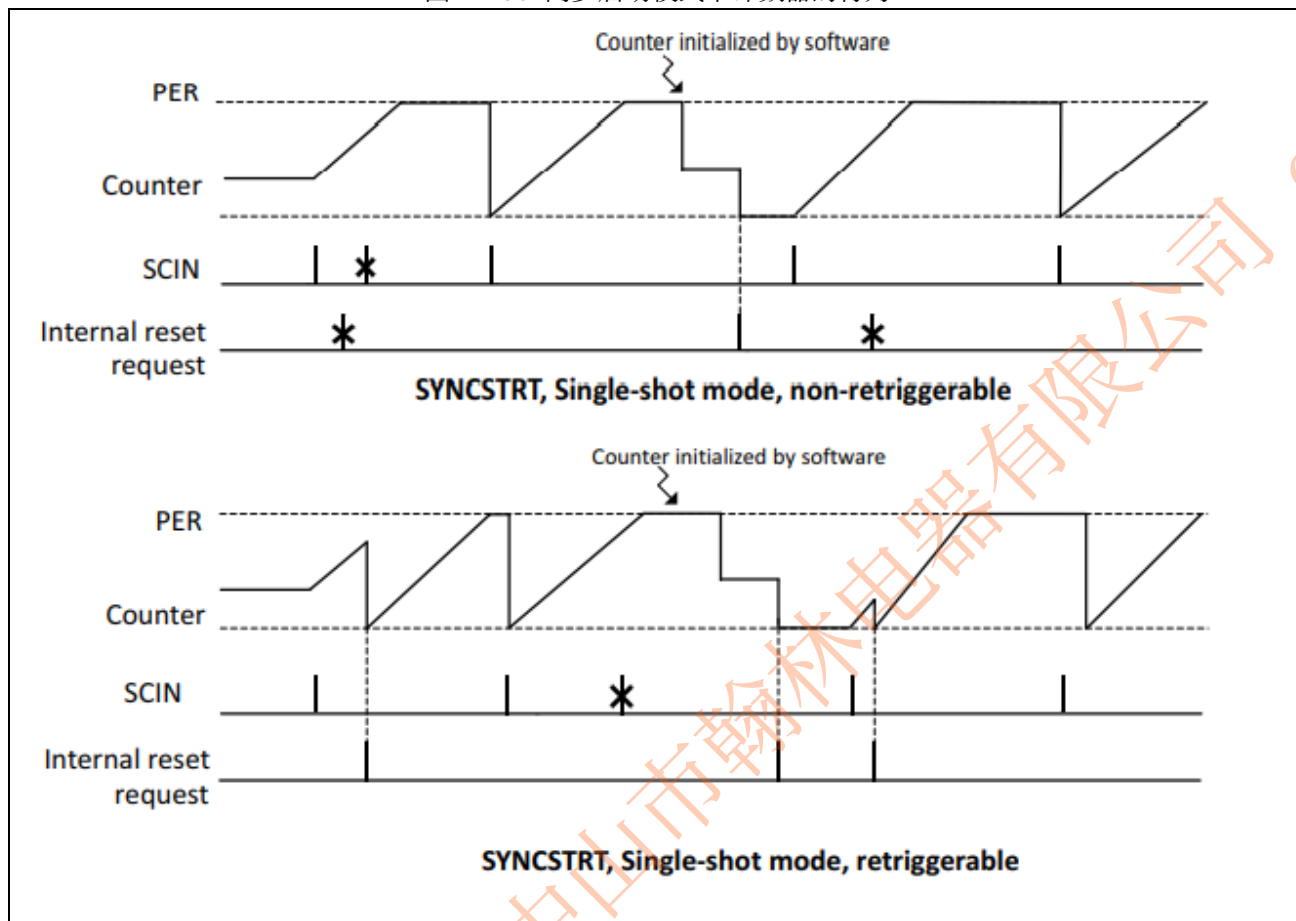


| 工作模式   | SYNC RSTx | SYNC STRTx | SYNC 复位或启动事件后的行为                                                                                                                                      |
|--------|-----------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| 单发可再触发 | 0         | 1          | 仅当计数器未启动或周期结束时，计数器启动才有效。任何在计数器启动后发生的同步事件都无效。<br>如果在计数器因达到周期值而停止时发生启动事件，则会将计数器复位。复位请求会将计数器清零，但不会启动计数器（计数器仅可通过同步事件启动）。计数过程中发生的复位事件会被考虑（与在常规可再触发模式下时一样）。 |
|        | 1         | X          | HRTIM_SCIN 发出的复位请求会被当作来自内部事件的 HRTIM 计数器复位请求进行考虑，并将启动或重新启动定时器计数操作。如果选择了多个复位请求，则会考虑第一个到达的请求。                                                            |
| 连续模式   | 0         | 1          | 定时器已使能（MCEN 或 TxCEN 位置 1），并正在等待同步事件启动计数器。计数器启动后发生的任何同步事件均不起作用（计数器仅可通过同步事件启动）。复位请求会将计数器清零，但不会启动计数器。                                                    |
|        | 1         | X          | HRTIM_SCIN 发出的复位请求会被当作来自内部事件的 HRTIM 计数器复位请求进行考虑，并将启动或重新启动定时器计数操作。如果选择了多个复位请求，则会考虑第一个到达的请求。                                                            |

当同步复位事件与周期事件发生在同一个  $f_{HRTIM}$  时钟周期内时，该复位操作会被推迟到下一个编程设定的周期事件（因为这两个事件都会导致计数器翻转）。这仅在使能高分辨率模式时适用（即  $CKPSC[2:0] < 5$ ）。

下图显示了单发模式下如何进行同步启动。

图 22.69 同步启动模式下计数器的行为



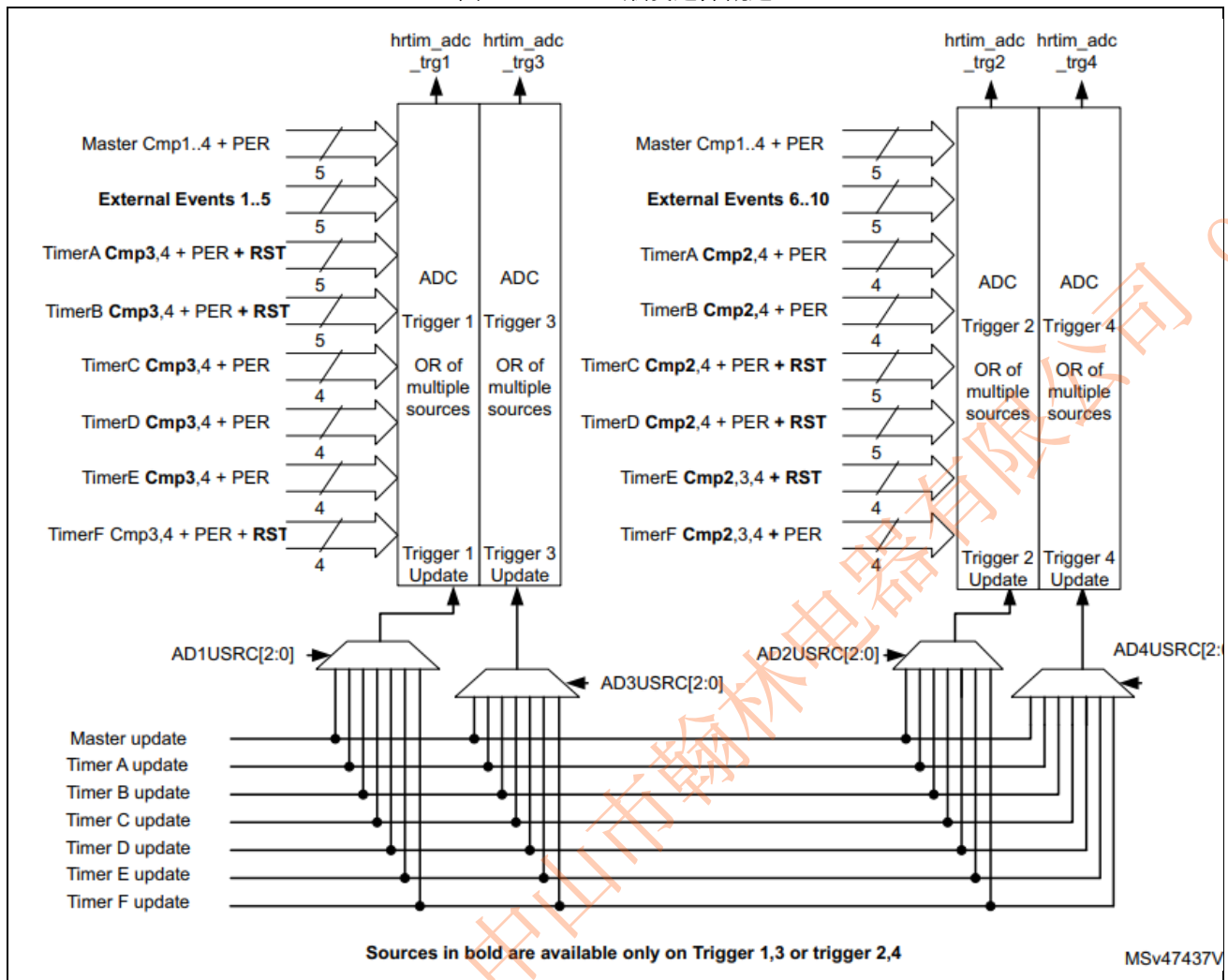
### 22.3.20 ADC 触发

ADC 可以由主定时器和 6 个定时单元触发。

ADC 的常规定序器和注入定序器各有 10 个独立的触发源。外部事件可用作触发信号。这类触发信号会在 HRTIM\_EECRx 寄存器中定义的调节结束后立即获取，并且与 EEFR1 和 EEFR2 寄存器设置无关。

在 HRTIM\_ADC1R 到 HRTIM\_ADC4R 寄存器中，最多可以将 32 个事件组合（逻辑或运算）用于 ADC 触发 1 到 4，如下图所示。ADC 触发 1/3 和 2/4 使用一组相同的源。可通过同时选择多个源的方式在单个开关周期内进行多次触发。典型用例是用于非重叠多相转换器，可通过单个 ADC 触发输出对所有相位进行连续采样。

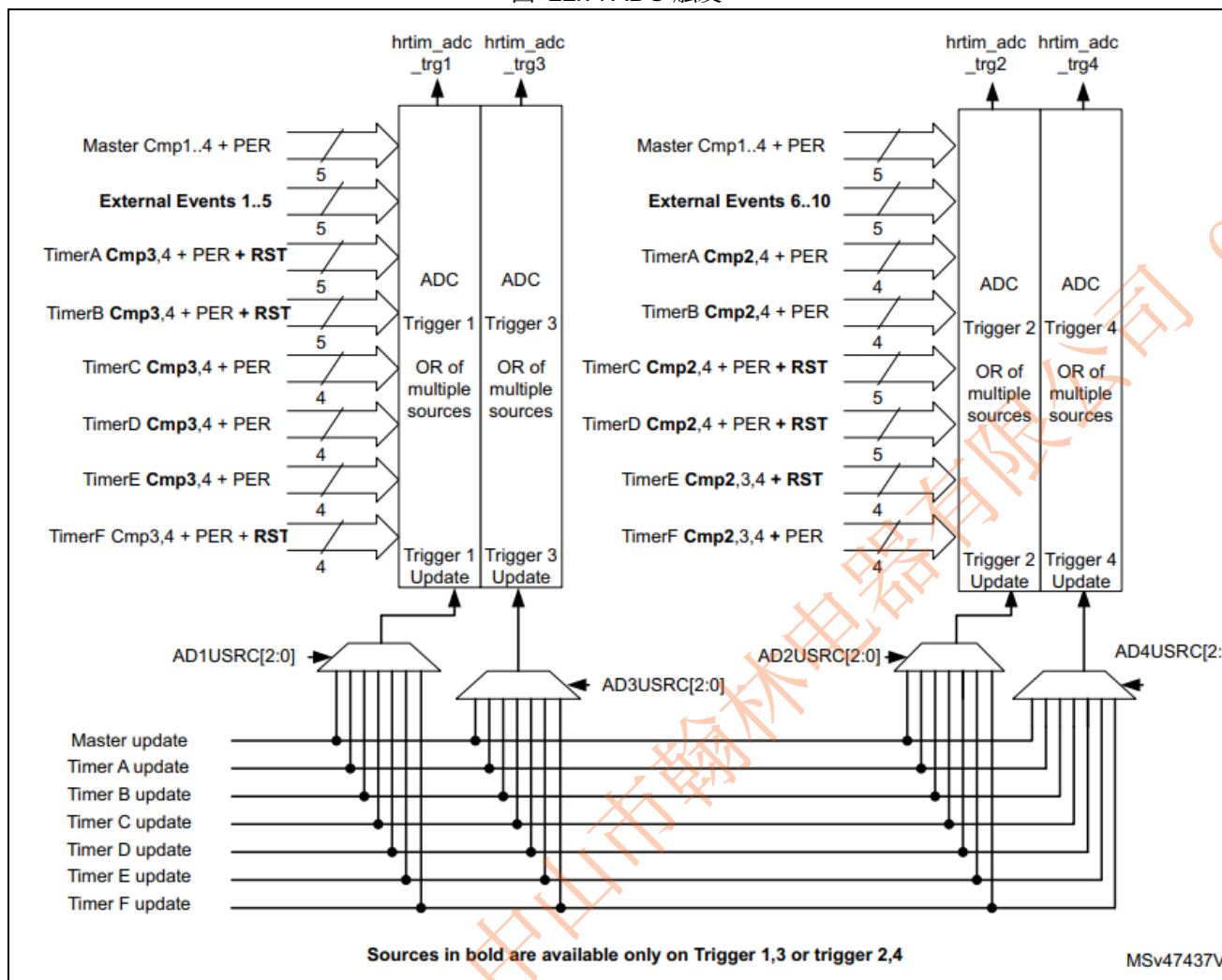
图 22.70 ADC 触发选择概述



ADC 触发 5 到 10 在 HRTIM\_AD\_CER 寄存器中进行配置，如下图所示。ADC 触发 5/7/9 和 6/8/10 使用一组相同的源。

对于这些触发，一次只能选择一个源（从 32 个可能的事件中选择一个）。

图 22.71 ADC 触发



HRTIM\_ADC1R 至 HRTIM\_ADC4R 和 HRTIM\_ADCER 寄存器会预装载，并可与相关定时器同步更新。更新源是通过 HRTIM\_CR1 和 HRTIM\_ADCUR 寄存器中的 ADxUSRC[2:0] 位进行定义。

例如，如果 ADC 触发 1 输出定时器 A 的 CMP2 事件（HRTIM\_ADC1R = 0x0000 0400），则 HRTIM\_ADC1R 通常与定时器 A（AD1USRC[2:0] = 001）同时更新。

如果源定时器中禁止预装载（PREEN 位复位），HRTIM\_ADCxR 寄存器也不会预装载：写访问会立即更新触发源。

## ADC 后缩放器

后缩放单元允许降低 ADC 触发率，如下图所示。

每个 ADC 触发率都可以使用 HRTIM\_ADCxPS1 和 HRTIM\_ADCxPS2 寄存器中的 ADCxPSC[4:0] 位进行单独调整。

在中心对齐模式下，ADC 触发率还依赖于源定时器中编程的 ADROM[1:0] 位域，如图 21.73 所示。

ADROM[1:0] 位域用于编码可以触发 ADC 的任何事件：复位、翻转（周期）和比较事件：

- ADROM[1:0] = 00：在向上和向下计数阶段都生成事件
- ADROM[1:0] = 01：在向下计数阶段生成事件
- ADROM[1:0] = 10：在向上计数阶段生成事件

ADC 后缩放器的编程寄存器是预加载的，并且可以在不停止定时器的情况下实时更新。

图 22.72 向上计数模式下的 ADC 触发后缩放

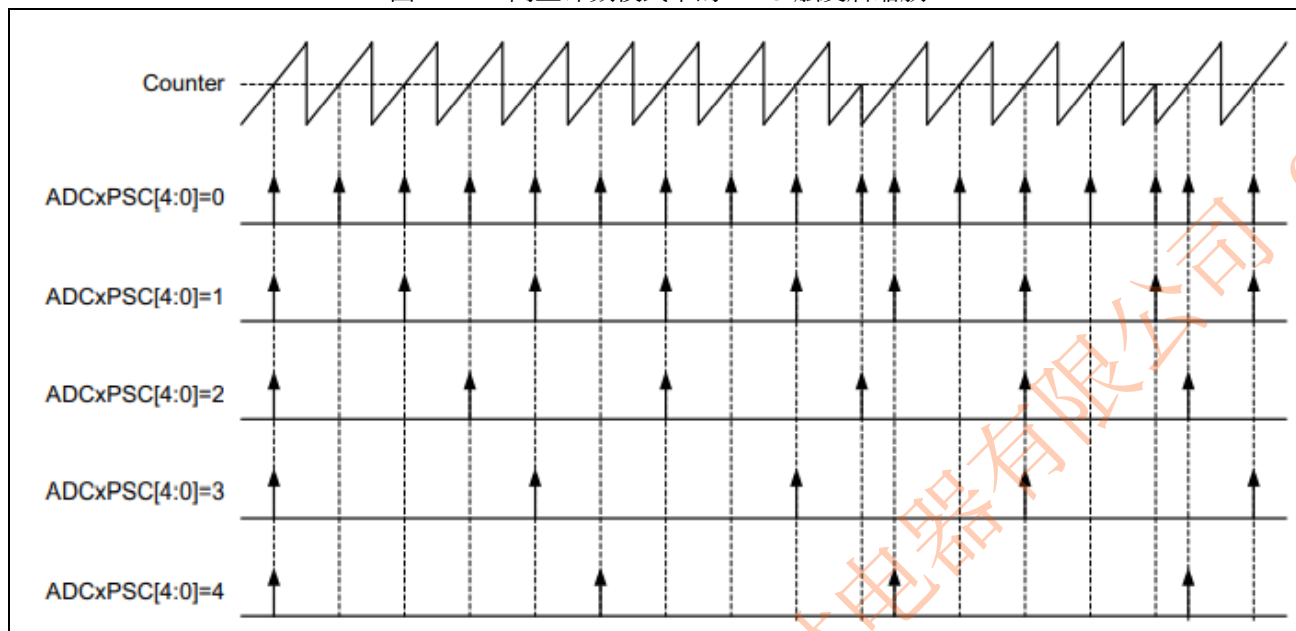
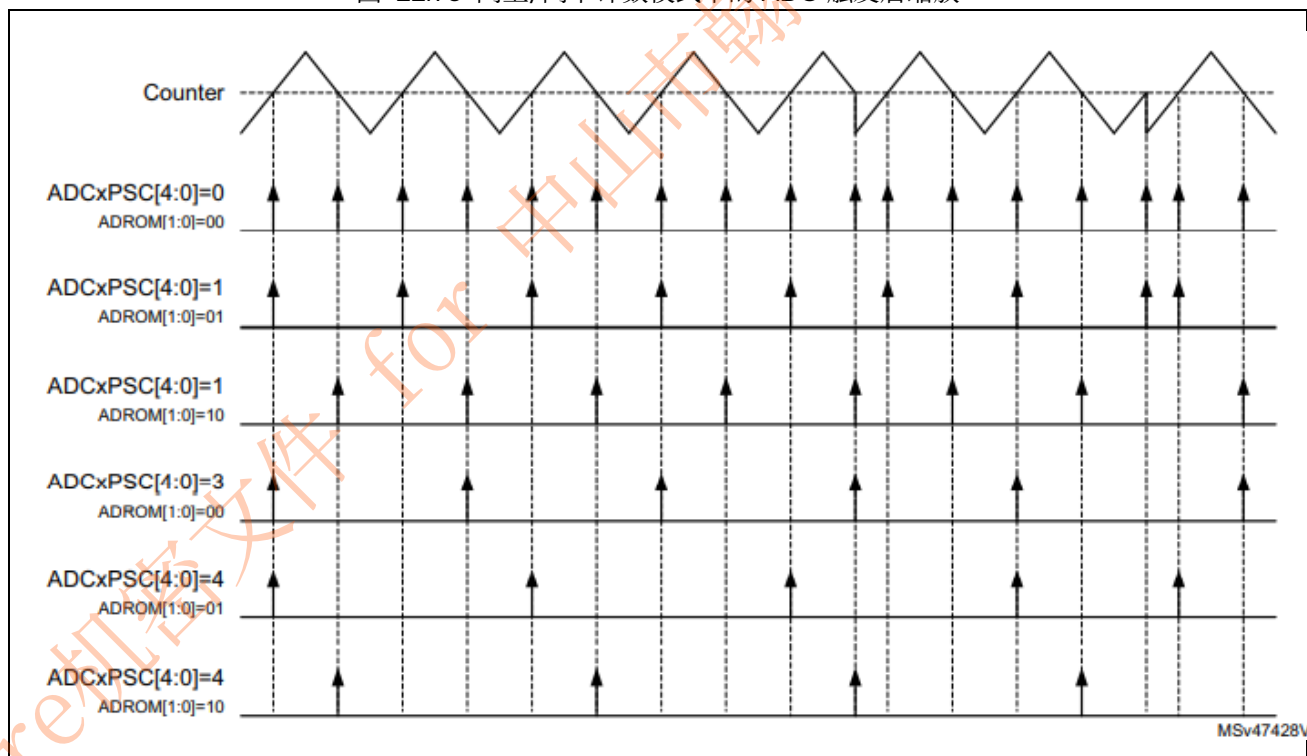


图 22.73 向上/向下计数模式下的 ADC 触发后缩放



### 22.3.21 DAC 触发

HRTIM 可使嵌入式 DAC 与定时器同步更新。

主定时器和定时器单元发出的更新事件可在 3 路 hrtim\_dac\_trgx 输出的任意一路上生成 DAC 更新触发信号。

注：每个定时器都有自己的 DAC 相关控制寄存器

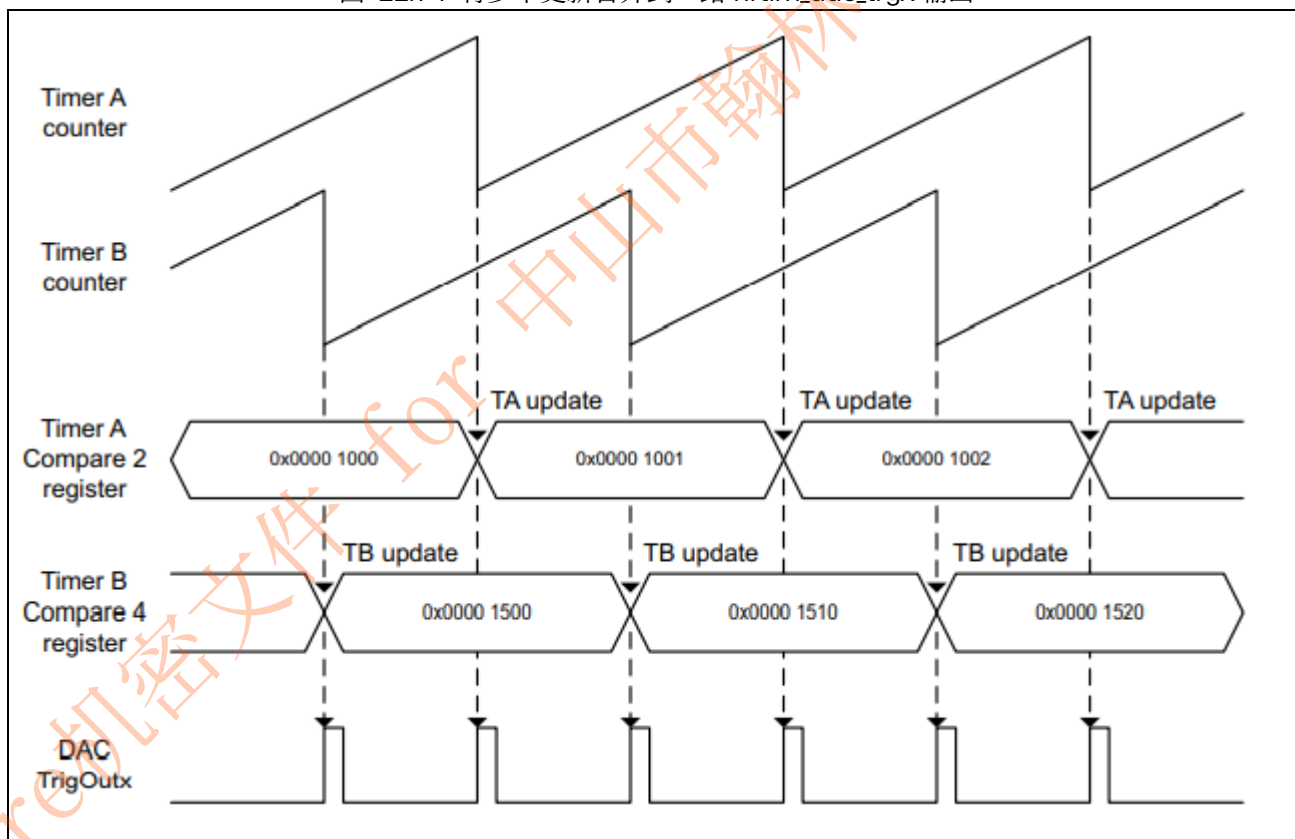
HRTIM\_MCR 和 HRTIM\_TIMxCR 寄存器 DACSYNC[1:0]位的编程值含义如下：

- 00：未生成更新
- 01：在 hrtim\_dac\_trg1 上生成更新
- 10：在 hrtim\_dac\_trg2 上生成更新
- 11：在 hrtim\_dac\_trg3 上生成更新

会在 hrtim\_dac\_trgx 输出上生成 1 个  $f_{HRTIM}$  时钟周期的输出脉冲。

如果多个定时器中使能了 DACSYNC[1:0]位，hrtim\_dac\_trgx 输出将由所有定时器更新事件的或运算结果决定。例如，如果定时器 A 和定时器 B 中 DACSYNC = 1，那么定时器 A 中的更新事件将与定时器 B 中的更新事件进行或运算，在相应的 hrtim\_dac\_trgx 输出上生成 DAC 更新触发信号，如下图所示。

图 22.74 将多个更新合并到一路 hrtim\_dac\_trgx 输出



关于 DAC 的连接，请参考表 17.6：HRTIM DAC 触发器连接。



## 双通道 DAC 触发

斜率补偿技术和滞后控制可以使用 HRTIM 内置功能和 DAC 锯齿波发生器轻松实现。其原理是，让 DAC 生成一个与 PWM 周期同步的递减锯齿波，或者一个与 PWM 信号同步的方波。

这种模式通过 TIMxCR2 寄存器中的 DCDE 位来使能。一旦定时器开始运行 (TxEN 位设置)，该位就不能更改。

它使用两个触发输出，如下图所示：

- hrtim\_dac\_reset\_trgx 生成 DAC 复位/更新事件
- hrtim\_dac\_step\_trgx 生成 DAC 值增量变化的请求

TIMxCR2 寄存器中的 DCDR 位定义了 hrtim\_dac\_reset\_trgx 触发器的生成时机：

- DCDR = 0：触发器在计数器复位或翻转事件上生成
- DCDR = 1：触发器在输出 1 设置事件上生成

注：当 DCDE 位复位（双通道 DAC 触发禁用）时，DCDR 位没有意义。

TIMxCR2 寄存器中的 DCDS 位定义了 hrtim\_dac\_step\_trgx 触发器的生成时机：

- DCDS = 0：触发器在比较事件 2 上生成
- DCDS = 1：触发器在输出 1 复位事件上生成

DCDR 和 DCDS 位允许覆盖以下使用场景：

- 边沿对齐斜率补偿 (DCDR = DCDS = 0)：DAC 的锯齿波在 PWM 周期开始时启动，并在周期内生成多个触发器
- 中心对齐斜率补偿 (DCDR = 1, DCDS = 0)：DAC 的锯齿波在输出设置事件上启动，并在周期内生成多个触发器
- 滞后控制器：当输出状态改变时，每个周期内必须改变 DAC 值两次。每个 PWM 周期生成两个触发器。在边沿对齐模式 (DCDR=0, DCDS=1) 下，触发器在计数器复位或翻转时生成。在中心对齐模式 (DCDR=1, DCDS=1) 下，触发器在输出设置时生成。

当 DCDE 设置且 DCDS 位复位时，比较器 2 具有特定的工作模式。一旦发生比较匹配，有效比较值将自动更新，以便触发器可以周期性重复，周期等于 CMP2 值，如下图所示。

使用 DCDS 位复位的双通道 DAC 触发（使用比较事件 2）不应与使用 CMP2 的模式（三重/四重交错和半触发周期模式）同时使用。

注：CMP2 值可以实时更改。新值将在下一个比较匹配时生效。

注：当 DCDS 位复位时，CMP2 值不应被其他机制修改：交错、半触发周期和平衡空闲模式必须禁用。



下表给出了一个示例，用于在一个 PWM 周期内生成 6 个触发器。它表明有必要将除法结果向上取整。

考虑一个计数器周期  $TIMxPER = 8192$ 。将 8192 除以 6 得到 1365.33。

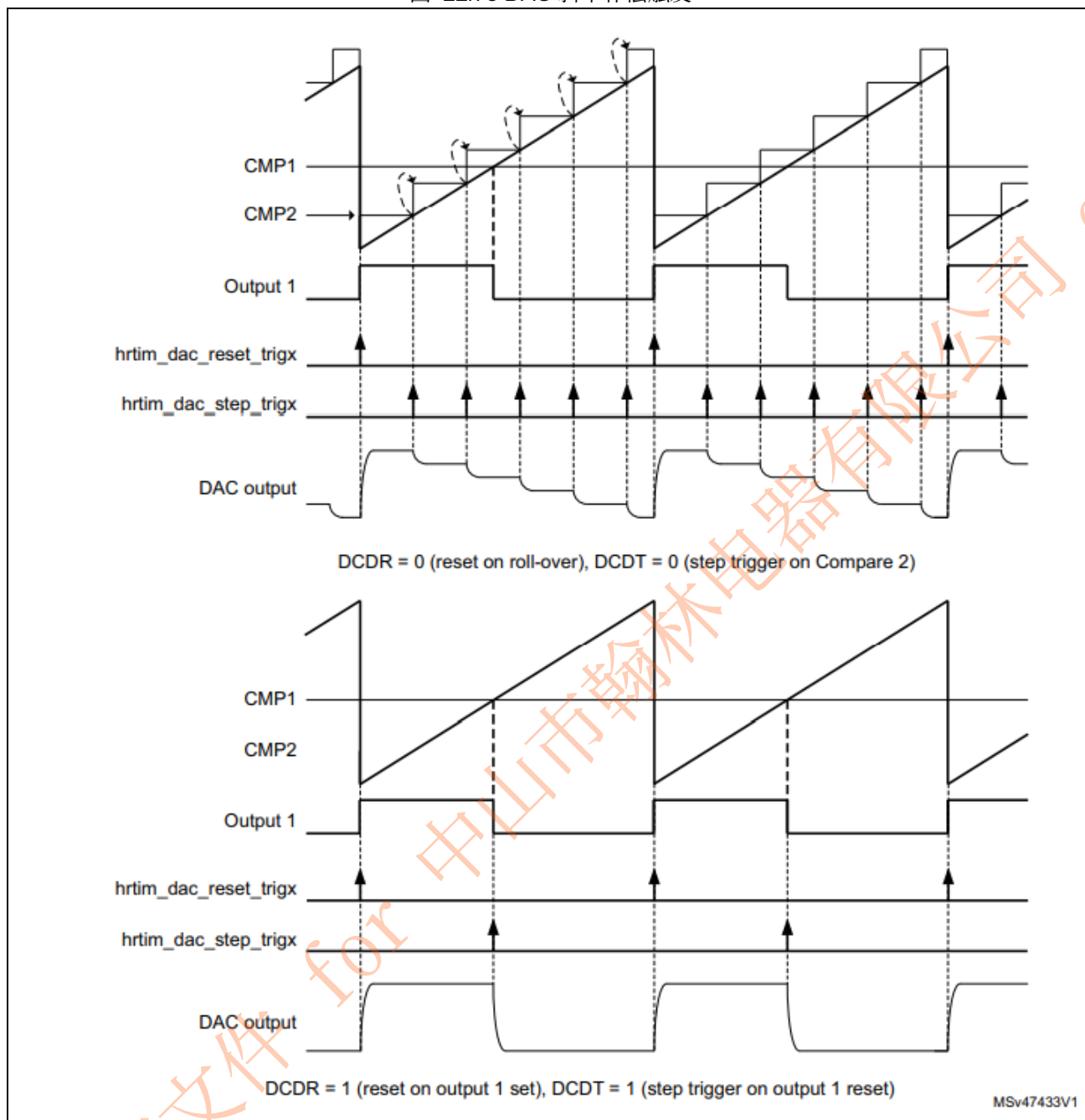
- x 向下取整的值：1365：会生成 7 个触发器，第 6 个和第 7 个非常接近（分别在计数器 = 8190 和 8192 时）
- x 向上取整的值：1366：会生成 6 个触发器。第 6 个触发器 `dac_step_trg`（在计数器 = 8192 时）会因计数器从 8192 翻转到 0 而被中止。

表 22.32 DAC 双通道触发示例

| -           | CMP2 = 1365 | dac_trg | dac_step_trg | CMP2 = 1366 | dac_trg | dac_step_trg |
|-------------|-------------|---------|--------------|-------------|---------|--------------|
| 计<br>数<br>值 | 1365        | -       | 1            | 1366        | -       | 1            |
|             | 2730        | -       | 2            | 2732        | -       | 2            |
|             | 4095        | -       | 3            | 4098        | -       | 3            |
|             | 5460        | -       | 4            | 5464        | -       | 4            |
|             | 6825        | -       | 5            | 6830        | -       | 5            |
|             | 8190        | -       | 6            | 8192        | 6       | -            |
|             | 8192        | 7       | -            | 1366        | -       | 1            |
|             | 1365        | -       | 1            | 2732        | -       | 2            |
|             | ...         | -       | -            | ...         | -       | -            |

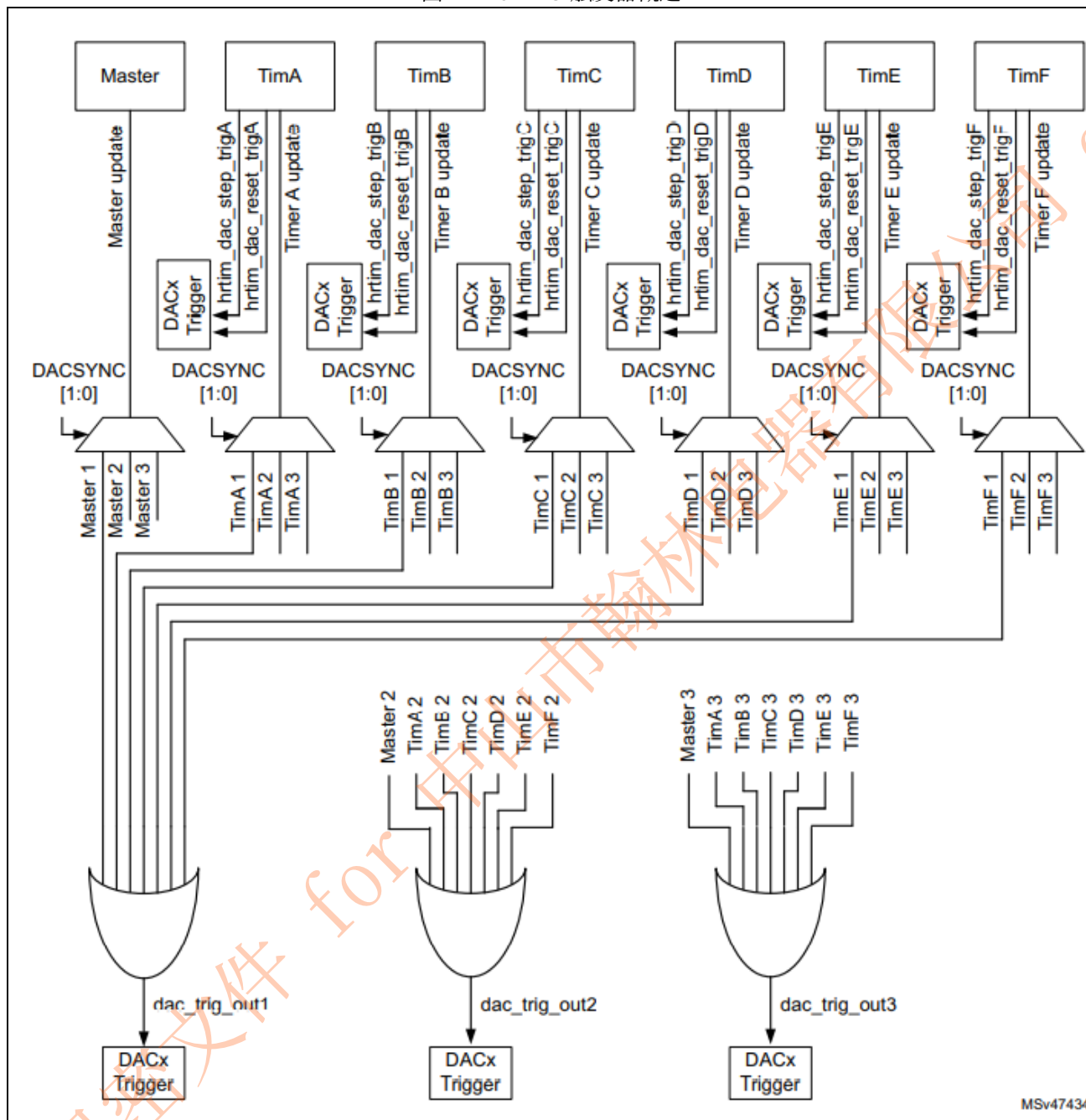
注：在中心对齐模式下，每个切换周期必须有偶数个触发器，以避免在计数器峰值附近出现不均匀间隔的触发器。

图 22.75 DAC 斜率补偿触发



下图提供了所有可用 DAC 触发器的概述。

图 22.76 DAC 触发器概述



### 22.3.22 中断

主定时器可以生成 7 个中断：

- 主定时器寄存器更新
- 接收到同步事件
- 主定时器重复事件
- 主定时器比较 1 到 4 事件

每个定时单元可以生成 14 个中断：

- 延迟保护触发
- 计数器复位或翻转事件
- 输出 1 和输出 2 复位（从有效电平跳变为无效电平）
- 输出 1 和输出 2 置位（从无效电平跳变为有效电平）
- 捕获 1 和 2 事件
- 定时单元寄存器更新
- 重复事件
- 比较 1 到 4 事件

会为整个 HRTIM 生成 8 个全局中断：

- 系统故障和故障 1 到 6（不考虑定时单元的因素）
- DLL 校准完成
- 突发模式周期结束

中断请求会分到 8 个向量组中，具体如下：

- hrtim\_it1: 主定时器中断（主定时器更新、同步输入、重复、MCMP1..4）和全局中断（故障除外，如突发模式周期和 DLL 就绪中断）
- hrtim\_it2: TIMA 中断
- hrtim\_it3: TIMB 中断
- hrtim\_it4: TIMC 中断
- hrtim\_it5: TIMD 中断
- hrtim\_it6: TIME 中断
- hrtim\_it7: TIMF 中断
- hrtim\_it8: 专用的故障中断向量，用于允许高优先级中断处理

下表总结了中断请求及其映射以及关联的控制和状态位。

表 22.33 HRTIM 中断汇总

| 中断向量      | 中断事件            | 事件标志位  | 使能控制位    | 标志清零位   |
|-----------|-----------------|--------|----------|---------|
| hrtim_it1 | 突发模式周期结束        | BMPER  | BMPERIE  | BMPERC  |
|           | DLL 校准完成        | DLLRDY | DLLRDYIE | DLLRDYC |
|           | 主定时器寄存器更新       | MUPD   | MUPDIE   | MUPDC   |
|           | 接收到同步事件         | SYNC   | SYNCIE   | SYNCC   |
|           | 主定时器重复事件        | MREP   | MREPIE   | MREPC   |
|           | 主定时器比较 1 到 4 事件 | MCMP1  | MCMP1IE  | MCP1C   |
|           |                 | MCMP2  | MCMP2IE  | MCP2C   |
|           |                 | MCMP3  | MCMP3IE  | MCP3C   |
|           |                 | MCMP4  | MCMP4IE  | MCP4C   |
| hrtim_it2 | 延迟保护已触发         | DLYPRT | DLYPRTIE | DLYPRTC |
| hrtim_it3 | 计数器复位或翻转事件      | RST    | RSTIE    | RSTC    |

|                                                  |                             |        |          |         |
|--------------------------------------------------|-----------------------------|--------|----------|---------|
| hrtim_it4<br>hrtim_it5<br>hrtim_it6<br>hrtim_it7 | 输出 1 和输出 2 复位（从有效电平跳变为无效电平） | RSTx1  | RSTx1IE  | RSTx1C  |
|                                                  |                             | RSTx2  | RSTx2IE  | RSTx2C  |
|                                                  | 输出 1 和输出 2 置位（从无效电平跳变为有效电平） | SETx1  | SETx1IE  | SETx1C  |
|                                                  |                             | SETx2  | SETx2IE  | SETx2C  |
|                                                  | 捕获 1 和 2 事件                 | CPT1   | CPT1IE   | CPT1C   |
|                                                  |                             | CPT2   | CPT2IE   | CPT2C   |
|                                                  | 定时单元寄存器更新                   | UPD    | UPDIE    | UPDC    |
|                                                  | 重复事件                        | REP    | REPIE    | REPC    |
|                                                  | 比较 1 到 4 事件                 | CMP1   | CMP1IE   | CMP1C   |
|                                                  |                             | CMP2   | CMP2IE   | CMP2C   |
|                                                  |                             | CMP3   | CMP3IE   | CMP3C   |
|                                                  |                             | CMP4   | CMP4IE   | CMP4C   |
| hrtim_it8                                        | 系统故障                        | SYSFLT | SYSFLTIE | SYSFLTC |
|                                                  | 故障 1 到 6                    | FLT1   | FLT1IE   | FLT1C   |
|                                                  |                             | FLT2   | FLT2IE   | FLT2C   |
|                                                  |                             | FLT3   | FLT3IE   | FLT3C   |
|                                                  |                             | FLT4   | FLT4IE   | FLT4C   |
|                                                  |                             | FLT5   | FLT5IE   | FLT5C   |
|                                                  |                             | FLT6   | FLT6IE   | FLT6C   |

### 22.3.23 DMA

大部分能够生成中断的事件也可以生成 DMA 请求，甚至可以同时生成中断和 DMA 请求。每个定时器（主定时器、TIMA...F）都有自己的 DMA 使能寄存器。

各 DMA 请求会在进行或运算后发送到 7 条通道中，具体如下：

- 主定时器 1 条通道
- 每个定时单元（TIMA...F）各 1 条通道

注：禁止 DMA 通道之前（TIMxDIER 中的 DMA 使能位复位），需要先禁止 DMA 控制器。

下表总结了事件及其关联的 DMA 使能位。

表 22.34 HRTIM DMA 请求总结

| DMA 通道                                                                                                                                             | 事件                              | DMA 功能 | DMA 使能位  |
|----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|--------|----------|
| hrtim_dma1<br>(主定时器)                                                                                                                               | 突发模式周期结束                        | 无      | N/A      |
|                                                                                                                                                    | DLL 校准完成                        | 无      | N/A      |
|                                                                                                                                                    | 主定时器寄存器更新                       | 有      | MUPDDE   |
|                                                                                                                                                    | 接收到同步事件                         | 有      | SYNCDE   |
|                                                                                                                                                    | 主定时器重复事件                        | 有      | MREPDE   |
|                                                                                                                                                    | 主定时器比较 1 到 4 事件                 | 有      | MCMP1DE  |
|                                                                                                                                                    |                                 | 有      | MCMP2DE  |
|                                                                                                                                                    |                                 | 有      | MCMP3DE  |
|                                                                                                                                                    |                                 | 有      | MCMP4DE  |
| hrtim_dma2<br>(定时器 A)<br>hrtim_dma3<br>(定时器 B)<br>hrtim_dma4<br>(定时器 C)<br>hrtim_dma5<br>(定时器 D)<br>hrtim_dma6<br>(定时器 E)<br>hrtim_dma7<br>(定时器 F) | 延迟保护已触发                         | 有      | DLYPRTDE |
|                                                                                                                                                    | 计数器复位或翻转事件                      | 有      | RSTDE    |
|                                                                                                                                                    | 输出 1 和输出 2 复位<br>(从有效电平跳变为无效电平) | 有      | RSTx1DE  |
|                                                                                                                                                    | 输出 1 和输出 2 置位<br>(从无效电平跳变为有效电平) | 有      | RSTx2DE  |
|                                                                                                                                                    | 捕获 1 和 2 事件                     | 有      | SETx1DE  |
|                                                                                                                                                    |                                 |        | SETx2DE  |
|                                                                                                                                                    | 捕获 1 和 2 事件                     | 有      | CPT1DE   |
|                                                                                                                                                    |                                 |        | CPT2DE   |
|                                                                                                                                                    | 定时单元寄存器更新                       | 有      | UPDDE    |
|                                                                                                                                                    | 重复事件                            | 有      | REPDE    |
|                                                                                                                                                    | 比较器 1 到 4 事件                    | 有      | CMP1DE   |
|                                                                                                                                                    |                                 | 有      | CMP2DE   |
|                                                                                                                                                    |                                 | 有      | CMP3DE   |
|                                                                                                                                                    |                                 | 有      | CMP4DE   |
| N/A                                                                                                                                                | 系统故障                            | 无      | N/A      |
|                                                                                                                                                    | 故障 1 至 6                        | 无      | N/A      |
|                                                                                                                                                    | 突发模式周期结束                        | 无      | N/A      |
|                                                                                                                                                    | DLL 校准完成                        | 无      | N/A      |

### 突发 DMA 传输

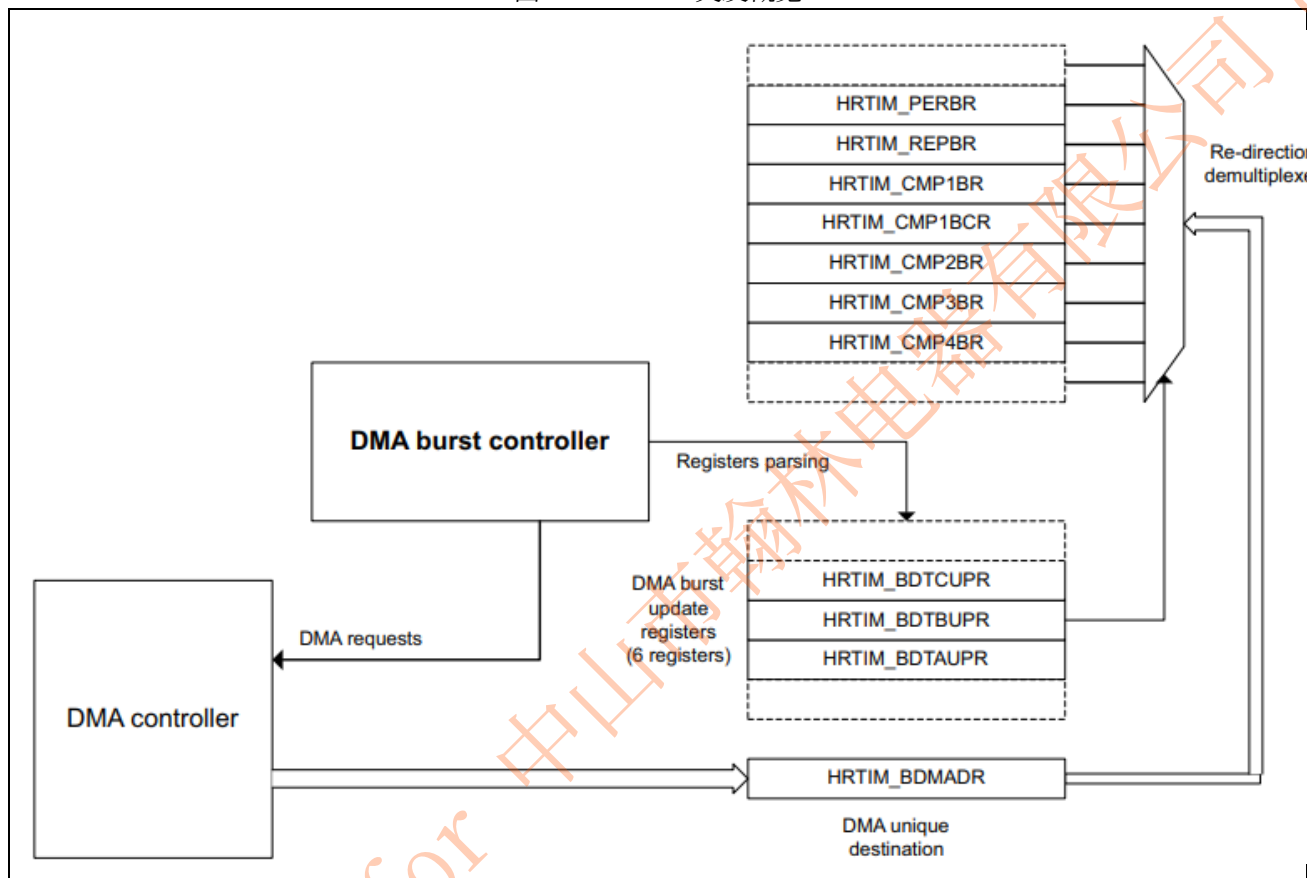
除了标准 DMA 请求之外，HRTIM 还配有 DMA 突发控制器，可通过单个 DMA 请求更新多个寄存器。具体包括：

- 仅通过一条 DMA 通道更新多个数据寄存器，
- 如果转换器使用多路定时器输出，可动态地对一个或多个定时单元重新进行编程。

突发 DMA 功能仅可用于一条 DMA 通道，但可选择 6 条通道中的任意一条进行突发 DMA 传输。

通过 DMA 写入，编程那些寄存器是核心内容。主定时器和 TIMA.E 包含突发 DMA 更新寄存器，其中的大部分控制和数据寄存器都关联到选择位：HRTIM\_BDMUPR、HRTIM\_BDTAUPR 到 HRTIM\_BDTEUPR（这一点仅适用于寄存器的写访问）。重定向机制允许自动将 DMA 写访问转发到 HRTIM 寄存器，如下图所示。

图 22.77 DMA 突发概览



发生 DMA 触发时，HRTIM 会生成多个 32 位 DMA 请求，并会解析更新寄存器。如果控制位置 1，写访问会重定向到关联的寄存器。如果此位复位，则会跳过寄存器更新并恢复寄存器解析，直至检测到此位再次置 1 触发新请求。对 6 个更新寄存器（HRTIM\_BDMUPR、5x HRTIM\_BDTxUPR）进行解析后，突发结束，系统准备好处理另一 DMA 触发（请参见下图中的流程图）。

注：任何在突发正在进行时发生的触发都会被丢弃，但最后一次数据传输期间发生的触发除外。

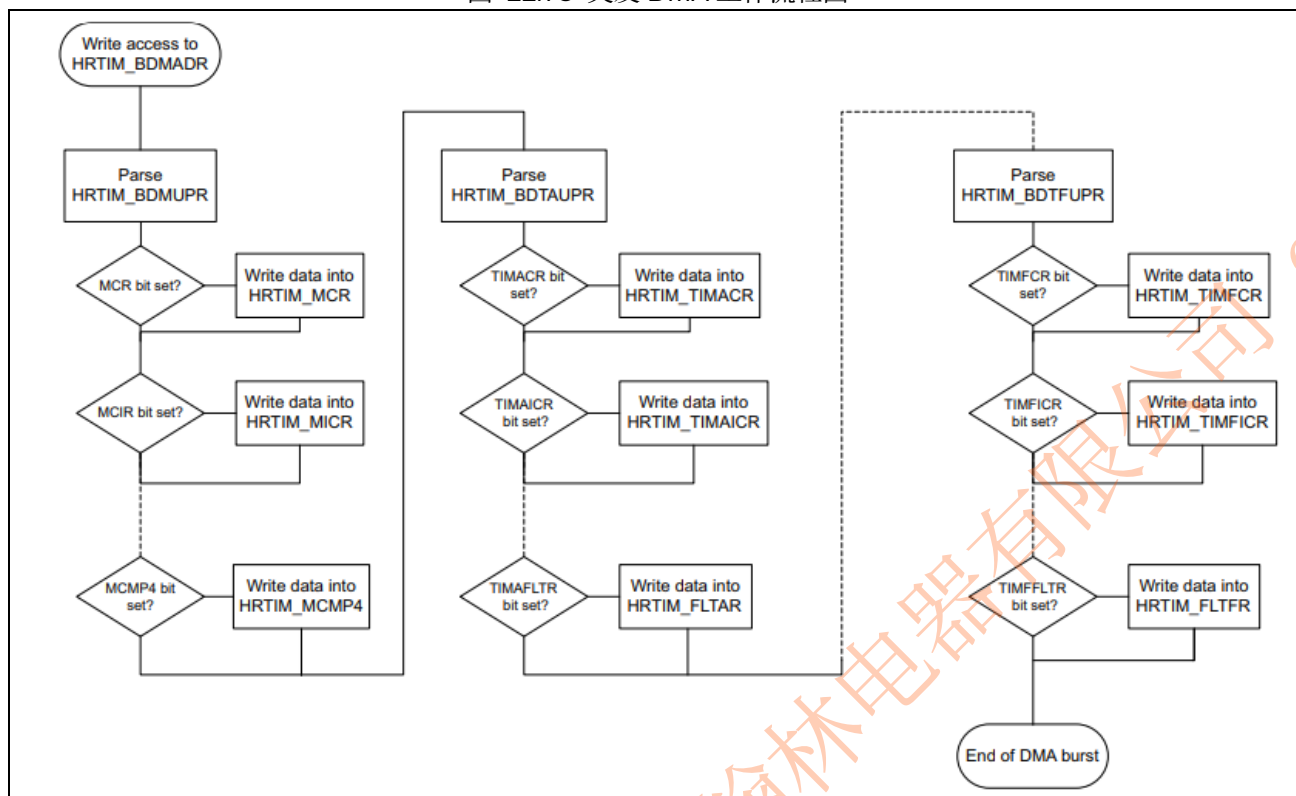
突发 DMA 模式永久使能（没有使能位）。突发 DMA 工作是通过 HRTIM\_BDMADR 寄存器进行的第一次写访问启动的。

仅需要使 DMA 控制器指向作为目标的 HRTIM\_BDMADR 寄存器，在存储器中指向禁止了外设递增模式的外设配置（HRTIM 会在内部处理到最终目标寄存器的数据重发）。

如果突发 DMA 模式在事务进行过程中中断，要重新初始化突发 DMA 模式，至少需要写入 6 个更新寄存器中的其中一个。



图 22.78 突发 DMA 工作流程图



DMA 突发结束后，有多种选项可供使用，具体视寄存器更新策略而定。

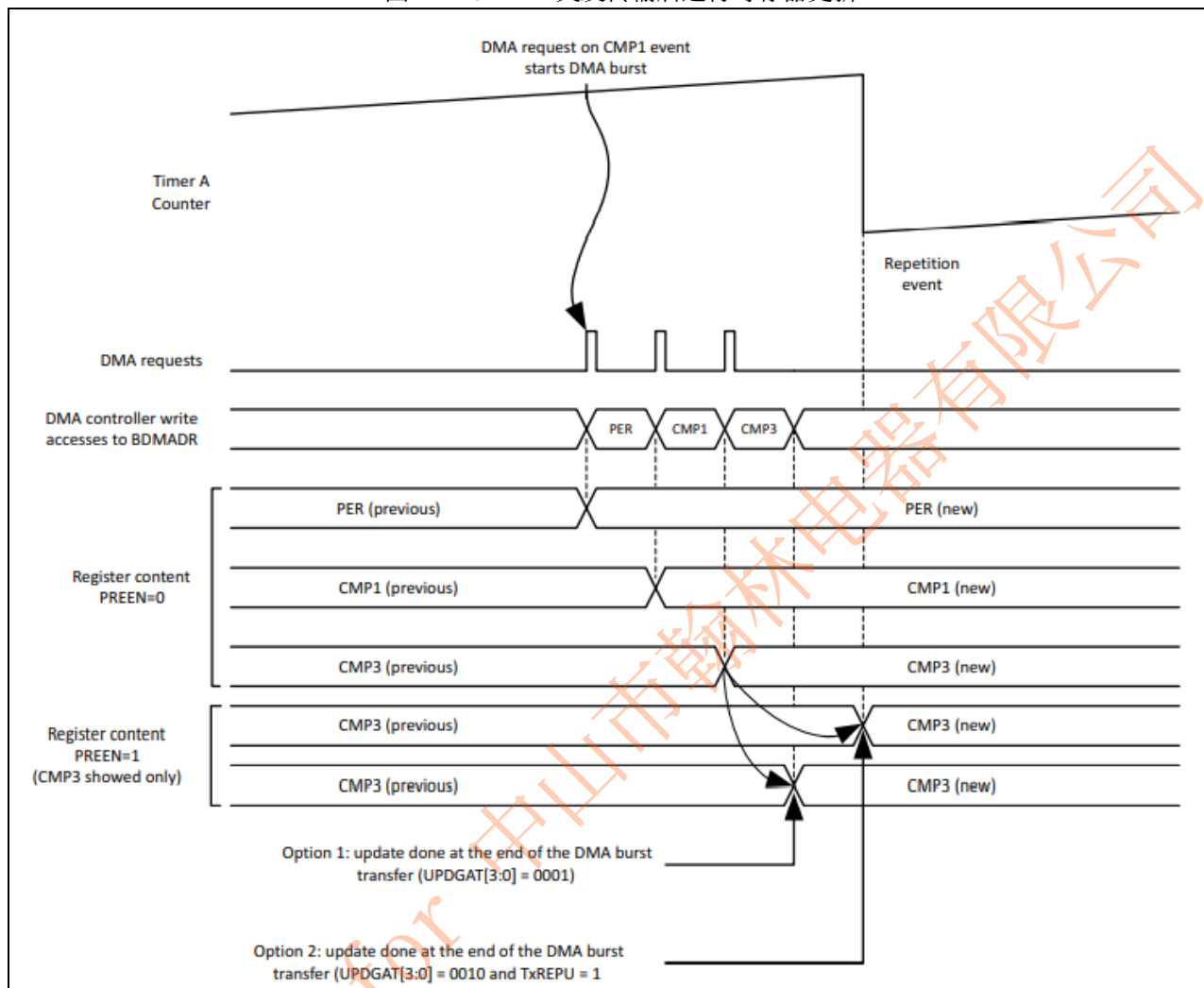
如果 PREEN 位复位（禁止预装载），通过 DMA 写入的值会立即传输到活动寄存器中，并会按照 DMA 事务的速度连续更新寄存器。

当预装载使能时（PREEN 位置 1），有 3 个用例：

1. 更新独立于 DMA 突发传输进行（HRTIM\_TIMxCR 中的 UPDGAT[3:0] = 0000，HRTIM\_MCR 中的 BRSTDMA[1:0] = 00）。在这种情况下，如果需要同时考虑所有传输的数据，用户必须检查 DMA 突发是否在发生更新事件之前结束。相反，如果更新事件在 DMA 传输进行时发生，则仅会装载部分寄存器，完整的寄存器更新将需要 2 个连续的更新事件。
2. 更新在 DMA 突发传输完成后进行（HRTIM\_TIMxCR 中的 UPDGAT[3:0] = 0000，HRTIM\_MCR 中的 BRSTDMA[1:0] = 01）。此模式可确保同时传输所有新寄存器值。此操作独立于计数器值进行，如有必要，可与常规更新事件相结合（例如，TxRSTU 置 1 的情况下，在发生计数器复位事件时进行更新）。
3. 在更新在 DMA 突发传输完成后发生更新事件时进行（HRTIM\_TIMxCR 中的 UPDGAT[3:0] = 0010，HRTIM\_MCR 中的 BRSTDMA[1:0] = 10）。此模式可确保对所有已传输数据、以及与常规更新事件的同步、与定时器计数器的同步进行一致的更新。在这种情况下，如果正在进行传输时发生常规更新请求，该请求会被丢弃，并会在下一次发出更新请求时进行有效更新。

下图上的时序图显示了 3 种情况下的活动寄存器内容：PREEN=0、UPDGAT[3:0] = 0001 以及 UPDGAT[3:0] = 0001 (PREEN = 1 时)。

图 22.79 DMA 突发传输后进行寄存器更新



### 22.3.24 HRTIM 初始化

本节介绍了建议的 HRTIM 初始化程序，包括其他相关 MCU 外设。

必须在复位和时钟控制单元 (RCC) 中使能 HRTIM 时钟源，同时遵守 DLL 锁定所需的  $f_{HRTIM}$  范围。

必须通过设置 HRTIM\_DLLCR 寄存器中的 CAL 位来启动 DLL 校准。

只有高分辨率单元准备就绪后，才能启动 HRTIM 主单元和定时单元。这由 DLLRDY 标志位指示。可以在恢复初始化之前轮询 DLLRDY 标志位，或者在初始化 HRTIM 或其他 MCU 外设的其他寄存器时，在后台运行校准。在这种情况下，必须在启动计数器之前检查 DLLRDY 标志位（如果需要，可以通过在 HRTIM\_IER 中使能 DLLRDYIE 标志来发出校准结束中断）。一旦 DLL 校准完成，必须设置 CALEN 位以使其定期执行，并补偿潜在的电压和温度漂移。校准周期性通过 HRTIM\_DLLCR 寄存器中的 CALRTE[1:0] 位域定义。

HRTIM 控制寄存器可按照电源转换器拓扑和定时单元用例进行初始化。必须配置所有输入（源、极性、边沿有效性）。

最后必须设置 HRTIM 输出，设置顺序如下：

- 必须使用 HRTIM\_OUTxR 中的 POLx 位定义极性
- 必须使用 HRTIM\_OUTxR 中的 FAULTx[1:0] 和 IDLESx 位配置 FAULT 和 IDLE 状态

HRTIM 输出已准备好连接到 MCU 的 I/O。在 GPIO 控制器中，必须按照产品数据手册中的复用功能映射表对所选 HRTIM I/O 进行配置。

从此刻起，HRTIM 会控制处于 IDLE 状态的输出。

要将输出配置为 RUN 模式，应将 HRTIM\_OENR 寄存器中的 TxyOEN 位置 1。RUN 模式下发生第一个有效置位/复位事件之前，2 路输出都处于无效状态。只要 TxCEN 位复位，任何输出置位/复位事件（使用 SST、SRT 的软件请求除外）都会被忽略，突发模式请求也是如此（会忽略 IDLEM 位值）。

同样，任何来自突发模式控制器的计数器复位请求也会被忽略（如果 TxBM 位置 1）。

注：如果使能了死区插入（DTEN 位置 1），则需要通过软件强制设置输出状态（使用 SST 和 RS 位），使输出在进入 RUN 模式后立即处于互补状态。

最终可通过将 HRTIM\_MCR 中的 TxCEN 或 MCEN 位置 1 的方式启动 HRTIM 工作。

如果 HRTIM 外设通过复位和时钟控制器复位，HRTIM 输出会进入 IDLE 模式且为低电平。建议先断开 HRTIMER 与输出的连接（使用 GPIO 控制器），然后再执行外设复位。

### 22.3.25 Debug 调试

当微控制器进入调试模式（带 FPU 的 Cortex®-M4 内核停止）时，TIMx 计数器会根据 DBG 模块中的 DBG\_HRTIM\_STOP 配置位选择继续正常工作或者停止工作。

- DBG\_HRTIM\_STOP = 0：无行为变化，HRTIM 继续工作。
- DBG\_HRTIM\_STOP = 1：所有 HRTIM 定时器（包括主定时器）均停止工作。

如果 FAULTx[1:0] = 01、10、11，RUN 模式下的输出会进入 FAULT 状态，如果 FAULTx[1:0] = 00，输出会保持当前状态。处于空闲状态的输出会保持此状态。即使 MCU 退出停止模式，也会永久保持此状态。这样可在执行步进期间保持安全状态。可通过将 TxyOEN 位置 1 再次使能输出（需要使用调试器）。

#### MCU 停止工作期间的定时器行为（DBG\_HRTIM\_STOP = 1 时）

置位/复位纵横开关、死区和推挽单元、空闲/均衡故障检测以及在 RUN 模式下驱动正常输出的所有逻辑均不受调试的影响。输出将在内部维持栓所状态，以便在 TxyOEN 再次置 1 时（MCU 停止工作期间或停止工作之后）重新获取输出的常规信号。如果输出已禁止，关联的触发信号和滤波器也会跟随内部波形。

MCU 停止工作期间，FAULT 输入和事件（任何源）会使能。

如果此时发生故障，则在 MCU 停止工作期间，故障状态位会置 1，TxyOEN 位会复位（TxyOEN 和 TxyODS 不受 DBG\_HRTIM\_STOP 位状态的影响）。

在调试模式下，会丢弃同步、计数器复位、启动、复位-启动事件以及捕获事件。这样做是为了确保所有相关寄存器在 MCU 停止期间的稳定性。

计数器到达断点时会停止计数。但计数器使能信号不会置位；因此退出调试模式时不会发出启动事件。所有计数器复位和捕获触发都会被禁用，外部事件也会被禁用（只要 MCU 停止工作，就会忽略这类事件）。输出 SET 和 RST 标志会冻结，但强制软件置位/复位的情况除外。调试过程中会屏蔽电平有效事件，但退出调试模式时，电平有效事件会立即再次激活。对于边沿有效事件，如果 MCU 停止工作期间信号保持有效，退出调试模式时不会生成新边沿。

更新事件会被丢弃。这样可避免触发对 hrtim\_upd\_en[3:1]输入的更新。DMA 触发会被禁止。突发模式电路冻结：触发会被忽略，突发模式计数器停止工作。

虽然 MCU 处于停止状态，但 DLL 校准不会被阻塞（DLLRDY 标志可以被设置）。

## 22.4 应用案例

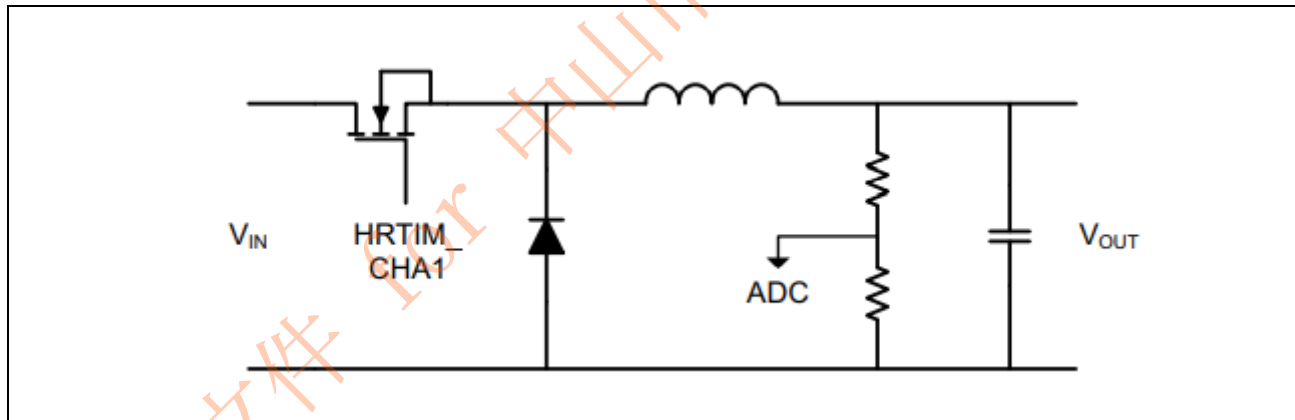
### 22.4.1 降压转换器

降压转换器通常用于降压。HRTIM 最多能够以 7 个独立的开关频率控制 12 个降压转换器。转换器通常以固定频率工作， $V_{in}/V_{out}$  比取决于应用到电源开关的占空比  $D$ ：

$$V_{out} = D \times V_{in}$$

下图给出了降压转换器的拓扑结构，以及连接到 ADC 进行电压读取的接口。

图 22.80 降压转换器拓扑结构

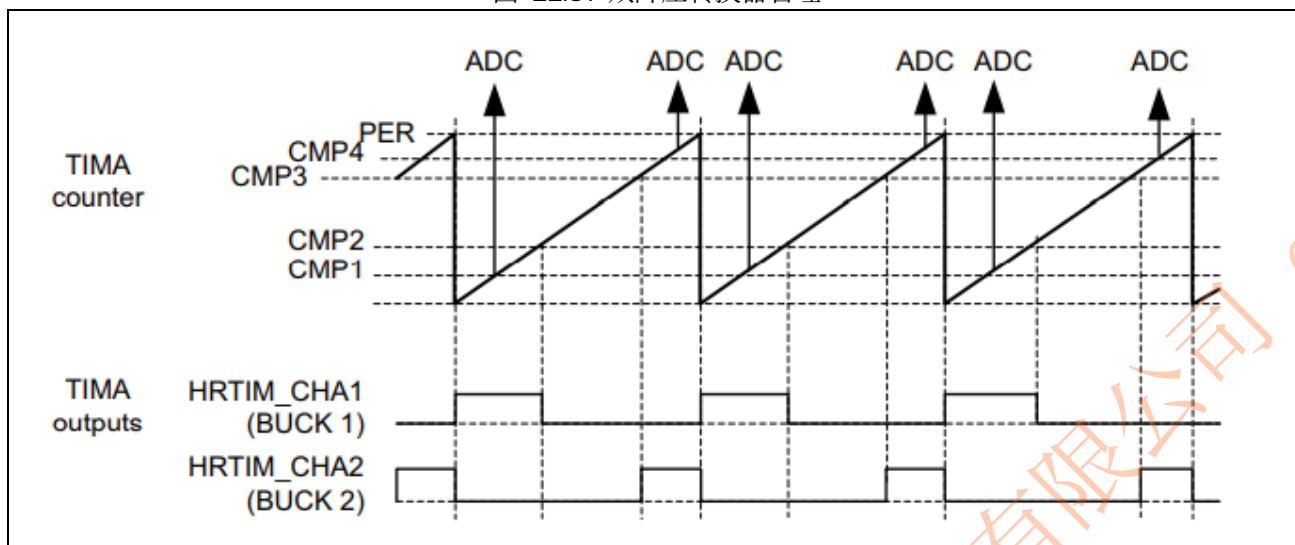


下图显示了如何利用相同频率的 PWM 信号来管理两个转换器。输出的定义如下：

- HRTIM\_CHA1 在周期开始时置位，在发生 CMP1 时复位
- HRTIM\_CHA2 在发生 CMP3 时置位，在发生 PER 时复位

ADC 每个周期会触发两次，分别通过 CMP2 和 CMP4 事件在 ON 时间的中点精确触发。

图 22.81 双降压转换器管理

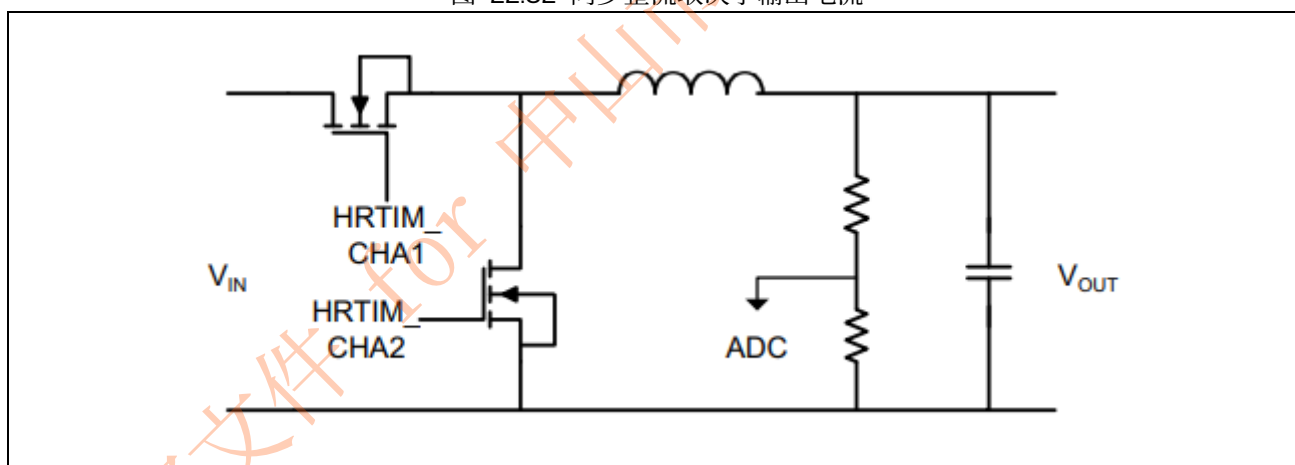


定时器 A..E 提供 12 个成对耦合的降压转换器（每对转换器的开关频率均相同）或 7 个完全独立的转换器（每个转换器都有不同的开关频率），第二种情况使用主定时器作为第 7 个时基。

### 22.4.2 具有同步整流功能的降压转换器

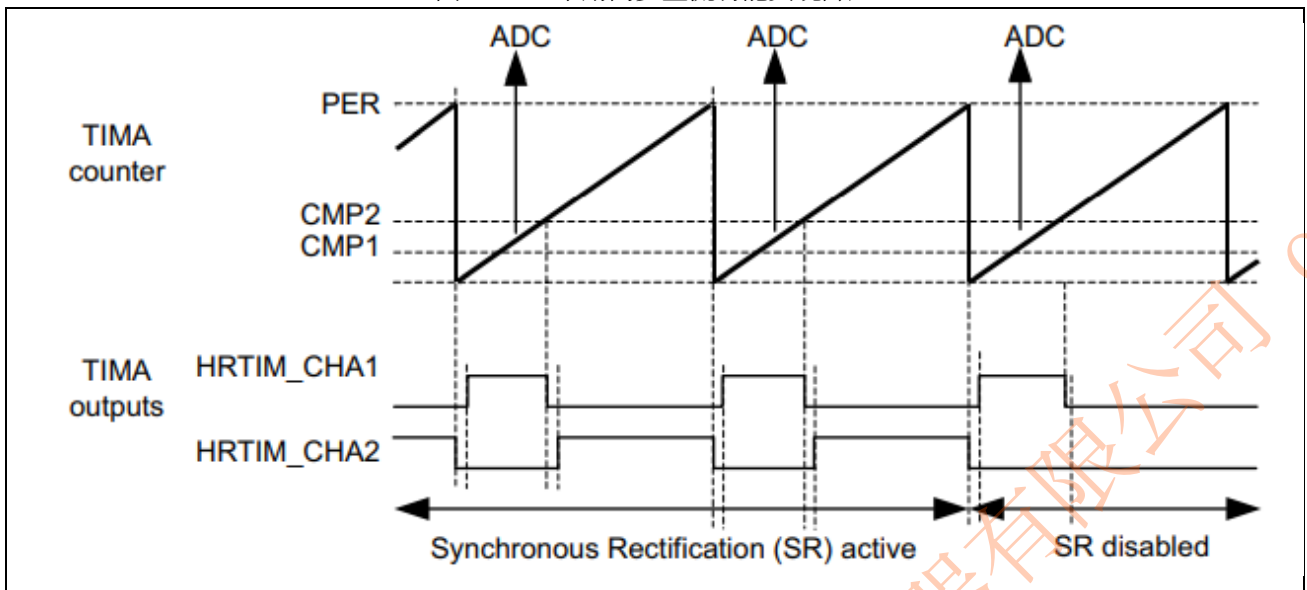
同步整流使用 FET 代替续流二极管，从而最大限度地降低降压转换器中的损耗。同步整流可实时开启或关闭，具体视输出电流水平而定（如下图所示）。

图 22.82 同步整流取决于输出电流



与单开关降压转换器的主要区别是增加了死区，从而可根据 HRTIM\_CHA1 上的参考波形在 HRTIM\_CHA2 上生成几乎互补的波形（如下图所示）。

图 22.83 采用同步整流功能实现降压



### 22.4.3 多相转换器

多相技术可应用于多种电源转换拓扑（降压、反激）。此项技术的主要优势为：

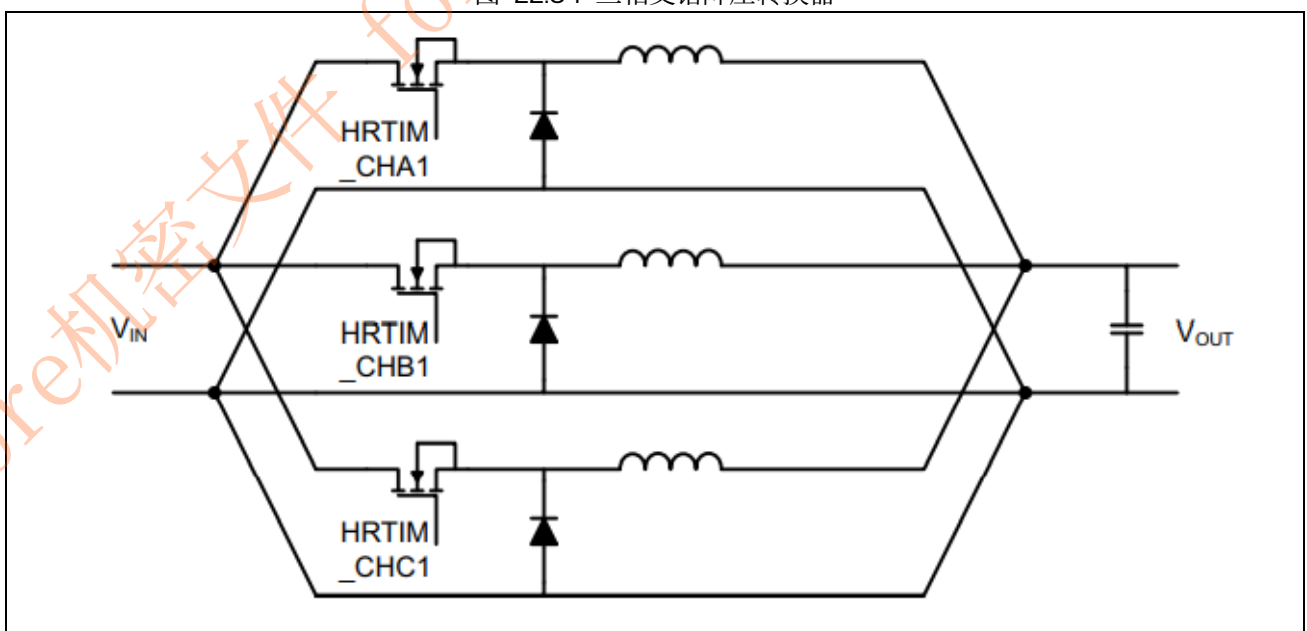
- 减少输入和输出电容的电流波纹
- 减少电磁干扰（EMI）
- 通过动态更改相位数（相位分离）提高轻载条件下的效率

HRTIM 能够管理多个转换器。可控制的转换器数量取决于所使用的拓扑和资源(包括ADC 触发信号)：

- 5 个具有同步整流 (SR) 功能的降压转换器，使用主定时器和 5 个定时器
- 4 个降压转换器（无 SR 功能），使用主定时器和 2 个定时器

下图显示了三相交错降压转换器的拓扑。

图 22.84 三相交错降压转换器





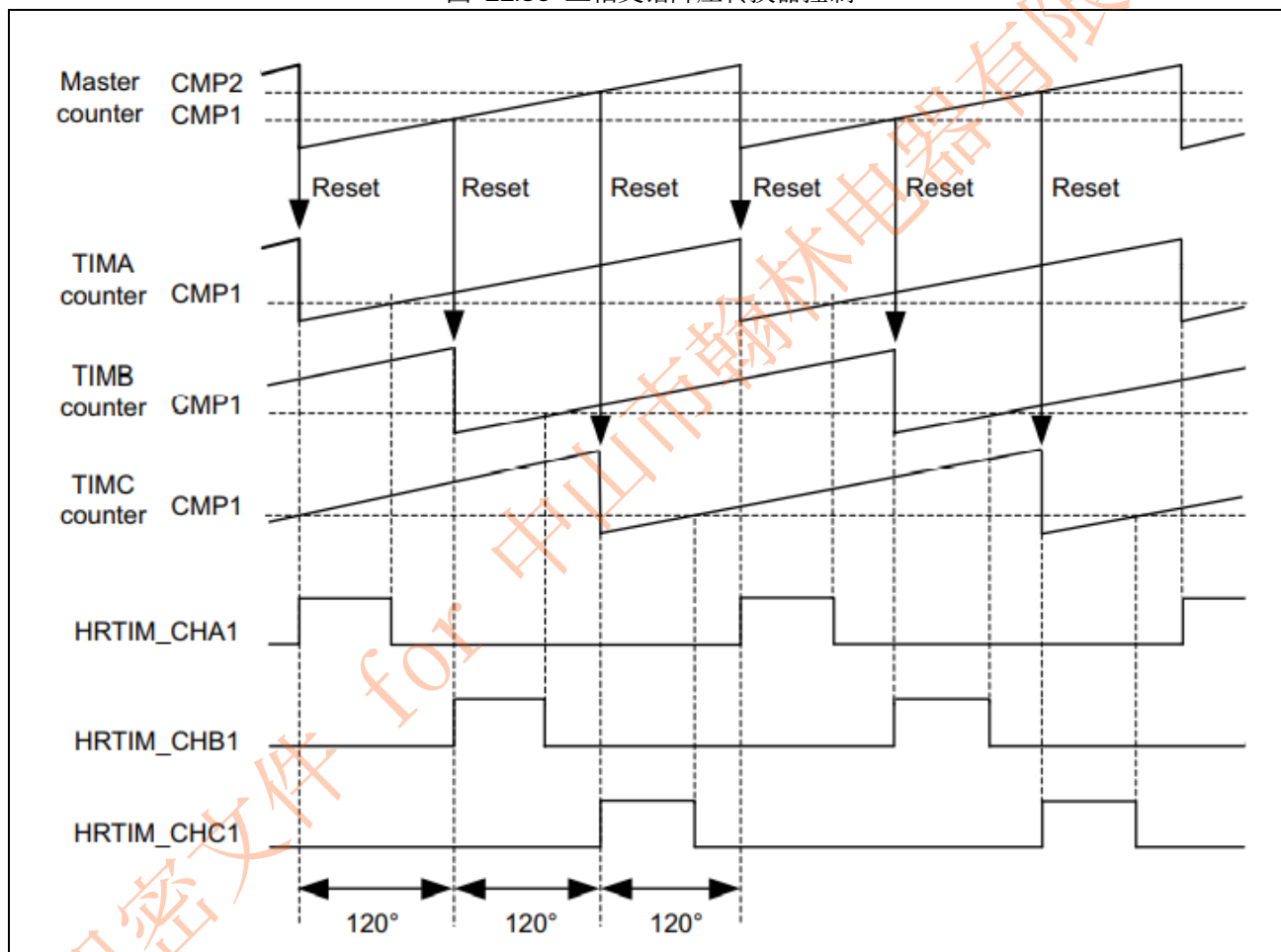
主定时器负责进行相位管理：它通过定期复位定时器来定义转换器之间的相位关系。相移为  $360^\circ$  除以相位数，所给示例中的相移为  $120^\circ$ 。

占空比随后会编程到每个定时器中。输出的定义如下：

- HRTIM\_CHA1 在主定时器周期开始时置位，在发生 TACMP1 时复位
- HRTIM\_CHB1 在发生主定时器 MCMP1 时置位，在发生 TBCMP1 时复位
- HRTIM\_CHC1 在发生主定时器 MCMP2 时置位，在发生 TCCMP1 时复位

发生 TxCMP2 比较事件时会生成 ADC 触发信号。由于所有 ADC 触发源的相位均由于转换器拓扑而发生了偏移，因此可以将所有触发源合并为单个 ADC 触发源，从而节省 ADC 资源（例如将 1 个 ADC 常规通道用于整个多相转换器）

图 22.85 三相交错降压转换器控制

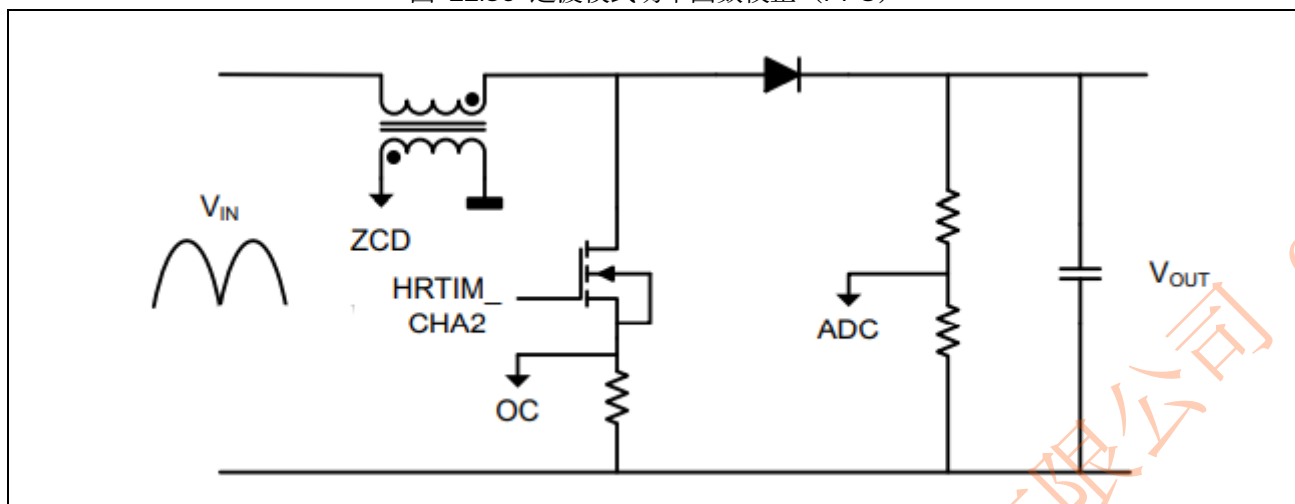


#### 22.4.4 过渡模式功率因数校正

基本工作原理是，在固定的  $T_{on}$  时间内建立进入电感的电流。该电流将随后在  $T_{off}$  时间内消退，当电流为零时重新开始此循环周期。可通过过零检查电路 (ZCD) 检测此现象，如下图所示。由于有恒定  $T_{on}$  时间，电感中的电流峰值与整流的交流输入电压成比例，这提供了功率因数校正。



图 22.86 过渡模式功率因数校正 (PFC)



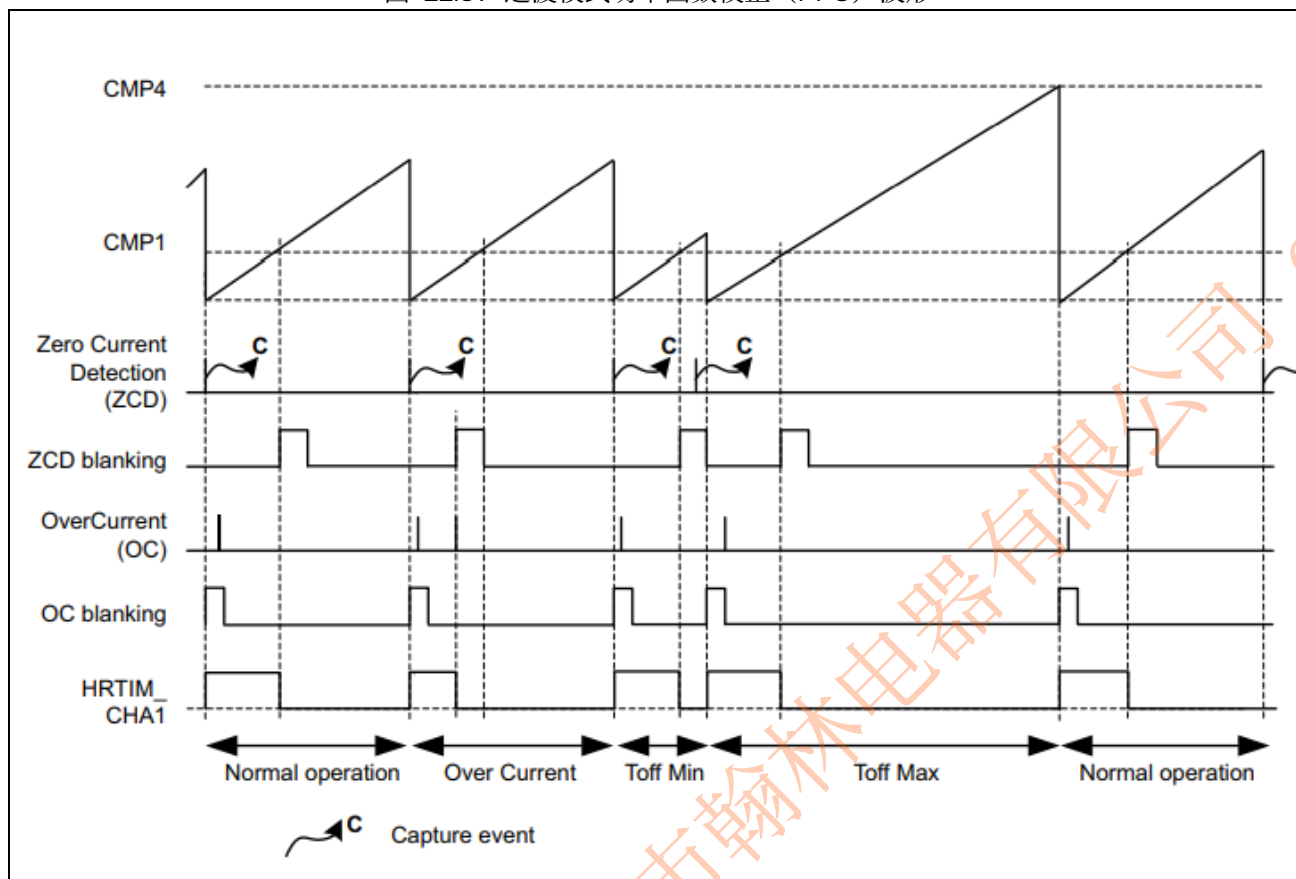
此转换器具有恒定的  $T_{on}$  时间，由于  $T_{off}$  时间变化其频率也是变化的（取决于输入电压）。它还必须包括一些功能，以便在未检测到过零电压时工作，或在过流 (OC) 时限制  $T_{on}$  时间。OC 反馈通常由内置比较器调节，发送到外部事件输入。

下图显示了各工作模式中的波形，其参数定义如下：

- $T_{on} Min$ ：屏蔽伪过流（续流二极管恢复电流），用 OC 消隐表示。
- $T_{on} Max$ ：实际上是转换器置位点。它由 CMP1 定义。
- $T_{off} Min$ ：在电流限值接近零时（退磁非常快）限制频率。它由 CMP2 定义。
- $T_{off} Max$ ：避免在无 ZCD 发生时系统卡死。自动延迟模式下它由 CMP4 定义。

由于值必须与输出下降沿相关，因此两个  $T_{off}$  值都会自动延迟。

图 22.87 过渡模式功率因数校正 (PFC) 波形



## 22.5 HRTIM 寄存器

### 22.5.1 HRTIM 主定时器控制寄存器 (HRTIM\_MCR)

偏移地址：0x000

复位值：0x0000 0000

| 31           | 30           | 29           | 28           | 27           | 26           | 25           | 24           | 23           | 22           | 21           | 20           | 19           | 18           | 17           | 16           |
|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| BRSTDMA[1:0] | MREPU        | Res          | PREEN        | DACSYNC[1:0] | Reserved     |              |              |              | TFCEN        | TECEN        | TDCEN        | TCCEN        | TBCEN        | TACEN        | MCEN         |
| rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           |
| 15           | 14           | 13           | 12           | 11           | 10           | 9            | 8            | 7            | 6            | 5            | 4            | 3            | 2            | 1            | 0            |
| SYNCSRC[1:0] | SYNCSRC[1:0] | SYNCSRC[1:0] | SYNCSRC[1:0] | SYNCSRC[1:0] | SYNCSRC[1:0] | SYNCSRC[1:0] | SYNCSRC[1:0] | SYNCSRC[1:0] | SYNCSRC[1:0] | SYNCSRC[1:0] | SYNCSRC[1:0] | SYNCSRC[1:0] | SYNCSRC[1:0] | SYNCSRC[1:0] | SYNCSRC[1:0] |
| rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           |

Bit 31:30 **BRSTDMA[1:0]**: 突发 DMA 更新 (Burst DMA update)

这些位定义更新与突发 DMA 事务的相对关系。

00: 独立于 DMA 突发传输进行更新。

01: DMA 突发传输结束时进行更新。

10: 在 DMA 突发传输结束后的主定时器翻转时进行更新。此模式仅适用于连续模式。

11: 保留

Bit 29 **MREPU**: 主定时器重复更新 (Master timer repetition update)

此位定义主定时器重复周期结束后是否进行更新 (因翻转或复位事件而更新)。

仅当 BRSTDMA[1:0] = 00 或 01 时, 才可将 MREPU 置 1。

0: 禁止与重复有关的更新

1: 使能与重复有关的更新

Bit 28 保留, 必须保持为复位值。

Bit 27 **PREEN**: 预加载使能 (Preload enable)

此位可使能寄存器预装载机制, 并定义对存储器映射寄存器的写访问是在 HRTIM 的活动寄存器中进行还是在预装载寄存器中进行。

0: 禁止预装载: 直接对活动寄存器进行写访问

1: 使能预装载: 对预装载寄存器进行写访问

Bit 26:25 **DACSYNC[1:0]**: DAC 同步 (DAC synchronization)

发生主定时器更新事件时, 可使能并生成 DAC 同步事件。这些位会定义在哪路输出上发送 DAC 同步事件 (详情请参考章节: DAC 触发连接)。

00: 未生成 DAC 触发事件

01: 在 hrtim\_dac\_trg1 上生成触发事件

10: 在 hrtim\_dac\_trg2 上生成触发事件

11: 在 hrtim\_dac\_trg3 上生成触发事件

Bit 24:23 保留, 必须保持为复位值。

Bit 22 **TFCEN**: 定时器 F 计数器使能 (Timer F counter enable)

此位用于启动定时器 F 计数器。

0: 禁用定时器 F 计数器

1: 使能定时器 F 计数器

注: 此位至少在 8 个  $f_{HRTIM}$  时钟周期内不得更改。

Bit 21 **TECEN**: 定时器 E 计数器使能 (Timer E counter enable)

此位用于启动定时器 E 计数器。

0: 禁用定时器 E 计数器

1: 使能定时器 E 计数器

注: 此位至少在 8 个  $f_{HRTIM}$  时钟周期内不得更改。

Bit 20 **TDCEN**: 定时器 D 计数器使能 (Timer D counter enable)

此位用于启动定时器 D 计数器。

0: 禁用定时器 D 计数器

1: 使能定时器 D 计数器

注: 此位至少在 8 个  $f_{HRTIM}$  时钟周期内不得更改。

Bit 19 **TCCEN**: 定时器 C 计数器使能 (Timer C counter enable)

此位用于启动定时器 C 计数器。

0: 禁用定时器 C 计数器

1: 使能定时器 C 计数器

注: 此位至少在 8 个  $f_{HRTIM}$  时钟周期内不得更改。

Bit 18 **TBCEN**: 定时器 B 计数器使能 (Timer B counter enable)

此位用于启动定时器 B 计数器。

0: 禁用定时器 B 计数器

1: 使能定时器 B 计数器

注: 此位至少在 8 个  $f_{HRTIM}$  时钟周期内不得更改。

Bit 17 **TACEN**: 定时器 A 计数器使能 (Timer A counter enable)

此位用于启动定时器 A 计数器。

0: 禁用定时器 A 计数器

1: 使能定时器 A 计数器

注: 此位至少在 8 个  $f_{HRTIM}$  时钟周期内不得更改。

Bit 16 **MCEN**: 主定时器计数器使能 (Master timer counter enable)

此位用于启动主定时器的计数器。

0: 禁止主定时器计数器

1: 使能主定时器计数器

注: 此位至少在 8 个  $f_{HRTIM}$  时钟周期内不得更改。

Bit 15:14 **SYNCSRC[1:0]**: 同步源 (Synchronization source)

这些位用于定义将在同步输出 SYNCOUT[2:1]上发送的源和事件。

00: 主定时器启动

01: 主定时器比较 1 事件

10: 定时器 A 启动/复位

11: 定时器 A 比较 1 事件

Bit 13:12 **SYNCOUT[1:0]**: 同步输出 (Synchronization output)

这些位用于定义同步输出事件的路由和调节。

00: 禁用

01: 保留。

10: 在 HRTIM\_SCOUT 输出上的正脉冲 (16 个  $f_{HRTIM}$  时钟周期)

11: 在 HRTIM\_SCOUT 输出上的负脉冲 (16 个  $f_{HRTIM}$  时钟周期)

注: 一旦计数器被使能 (TxCEN 位置1), 就不应修改此位域。

Bit 11 **SYNCSRTM**: 主定时器同步启动 (Synchronization starts master)

该位能够使主定时器在接收到同步输入事件时启动:

0: 对主定时器没有影响

1: 同步输入事件会启动主定时器

Bit 10 **SYNCRSTM**: 主定时器同步复位 (Synchronization resets master)

该位能够使主定时器在接收到同步输入事件时复位:

0: 对主定时器没有影响

1: 同步输入事件会复位主定时器

Bit 9:8 **SYNCIN[1:0]**: 同步输入 (Synchronization input)

这些位定义了同步输入源:

00: 禁用。HRTIM 不进行同步, 以独立模式运行。

01: 保留。

10: 内部事件通过 hrtim\_in\_sync[2]: HRTIM 与片上定时器同步 (见同步输入部分)。

11: 外部事件通过 hrtim\_in\_sync[3]: HRTIM\_SCIN 输入上的正脉冲触发 HRTIM。

注: 一旦受影响的定时器被使能, 就不能更改这个参数。

Bit 7:6 **INTLVD[1:0]**: 交错模式 (Interleaved mode)

这个位域仅在 HALF 位被清零时有效。它使能了交错模式:

00: 交错模式禁用

01: 三重交错模式: 当写入 HRTIM\_MPER 寄存器时, HRTIM\_MCMP1R 有效寄存器会自动更新为  $HRTIM\_MPER/3$  的值, HRTIM\_MCMP2R 有效寄存器会自动更新为  $2 \times (HRTIM\_MPER/3)$  的值。

10: 四重交错模式: 当写入 HRTIM\_MPER 寄存器时, HRTIM\_MCMP1R 有效寄存器会自动更新为  $HRTIM\_MPER/4$  的值, HRTIM\_MCMP2R 有效寄存器会自动更新为  $HRTIM\_MPER/2$  的值, HRTIM\_MCMP3R 有效寄存器会自动更新为  $3 \times (HRTIM\_MPER/4)$  的值。

11: 交错模式禁用

Bit 5 **HALF**: 半占空比模式 (Half mode)

此位用于使能半占空比模式: HRTIM\_MPER 寄存器写入内容后, HRTIM\_MCMP1R 活动寄存器会自动更新为  $HRTIM\_MPER/2$  值。

0: 禁止半占空比模式

1: 使能半占空比模式

Bit 4 **RETRIG**: 可再触发模式 (Re-triggerable mode)

这个位定义了主定时器计数器在单次触发模式下的行为:

0: 定时器不可再触发: 仅当计数器停止 (周期结束) 时才能进行计数器复位。

1: 定时器可再触发: 无论计数器的状态 (运行中或已停止), 都可以进行计数器复位。

Bit 3 **CONT**: 连续模式 (Continuous mode)

这个位定义了主定时器计数器在单次触发模式下的行为:

0: 定时器以单次触发模式工作, 并在达到 MPER 值时停止。

1: 定时器以连续 (自由运行) 模式工作, 并在达到 MPER 值时翻转到零。

Bit 2:0 **CKPSC [2:0]**: 时钟预分频器 (Clock prescaler)

这些位定义了主定时器高分辨率时钟的预分频器比率。  
 计数器时钟等效频率 ( $f_{\text{COUNTER}}$ ) 等于  $f_{\text{HRCK}} / 2^{\text{CKPSC}[2:0]}$ 。  
 一旦定时器被使能，预分频比率就不能再修改。

## 22.5.2 HRTIM 主定时器中断状态寄存器 (HRTIM\_MISR)

偏移地址: 0x004

复位值: 0x0000 0000

|         |    |    |    |    |    |    |    |      |      |      |       |       |       |       |    |
|---------|----|----|----|----|----|----|----|------|------|------|-------|-------|-------|-------|----|
| 31      | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23   | 22   | 21   | 20    | 19    | 18    | 17    | 16 |
| Resverd |    |    |    |    |    |    |    |      |      |      |       |       |       |       |    |
| 15      | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7    | 6    | 5    | 4     | 3     | 2     | 1     | 0  |
| Resverd |    |    |    |    |    |    |    | MUPD | SYNC | MREP | MCMP4 | MCMP3 | MCMP2 | MCMP1 |    |
|         |    |    |    |    |    |    |    | r    | r    | r    | r     | r     | r     | r     |    |

Bit 31:7 保留，必须保持复位值。

Bit 6 **MUPD**: 主定时器更新中断标志 (Master update interrupt flag)

当主定时器寄存器被更新时，该位由硬件置 1。

0: 未发生主定时器更新中断

1: 已发生主定时器更新中断

Bit 5 **SYNC**: 同步输入中断标志 (Sync input interrupt flag)

当接收到同步输入事件时，该位由硬件置 1。

0: 未发生同步输入中断

1: 已发生同步输入中断

Bit 4 **MREP**: 主定时器重复中断标志 (Master repetition interrupt flag)

当主定时器重复周期结束时，该位由硬件置 1。

0: 未发生主定时器重复中断

1: 已发生主定时器重复中断

Bit 3 **MCMP4**: 主定时器比较 4 中断标志 (Master compare 4 interrupt flag)

参考 MCMP1 的描述

Bit 2 **MCMP3**: 主定时器比较 3 中断标志 (Master compare 3 interrupt flag)

参考 MCMP1 的描述

Bit 1 **MCMP2**: 主定时器比较 2 中断标志 (Master compare 2 interrupt flag)

参考 MCMP1 的描述

Bit 0 **MCMP1**: 主定时器比较 1 中断标志 (Master compare 1 interrupt flag)

当主定时器计数器与主定时器比较 1 寄存器中编程的值匹配时，该位由硬件置 1。

0: 未发生主定时器比较 1 中断

1: 已发生主比定时器较 1 中断

### 22.5.3 HRTIM 主定时器中断清除寄存器 (HRTIM\_MICR)

偏移地址: 0x008

复位值: 0x0000 0000

|         |    |    |    |    |    |    |    |    |      |      |      |      |      |      |      |
|---------|----|----|----|----|----|----|----|----|------|------|------|------|------|------|------|
| 31      | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22   | 21   | 20   | 19   | 18   | 17   | 16   |
| Resverd |    |    |    |    |    |    |    |    |      |      |      |      |      |      |      |
| 15      | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| Resverd |    |    |    |    |    |    |    |    | MUPD | SYNC | MREP | MCMP | MCMP | MCMP | MCMP |
|         |    |    |    |    |    |    |    |    | C    | C    | C    | 4C   | 3C   | 2C   | 1C   |
|         |    |    |    |    |    |    |    |    | w    | w    | w    | w    | w    | w    | w    |

Bit 31:7 保留, 必须保持复位值。

Bit 6 **MUPDC**: 主更新中断标志清除 (Master update interrupt flag clear)

向此位写入 1 将清除 HRTIM\_MISR 寄存器中的 MUPD 标志。

Bit 5 **SYNCC**: 同步输入中断标志清除 (Sync input interrupt flag clear)

向此位写入 1 将清除 HRTIM\_MISR 寄存器中的 SYNC 标志。

Bit 4 **MREPC**: 重复中断标志清除 (Repetition interrupt flag clear)

向此位写入 1 将清除 HRTIM\_MISR 寄存器中的 MREP 标志。

Bit 3 **MCMP4C**: 主比较 4 中断标志清除 (Master compare 4 interrupt flag clear)

向此位写入 1 将清除 HRTIM\_MISR 寄存器中的 MCMP4 标志。

Bit 2 **MCMP3C**: 主比较 3 中断标志清除 (Master compare 3 interrupt flag clear)

向此位写入 1 将清除 HRTIM\_MISR 寄存器中的 MCMP3 标志。

Bit 1 **MCMP2C**: 主比较 2 中断标志清除 (Master compare 2 interrupt flag clear)

向此位写入 1 将清除 HRTIM\_MISR 寄存器中的 MCMP2 标志。

Bit 0 **MCMP1C**: 主比较 1 中断标志清除 (Master compare 1 interrupt flag clear)

向此位写入 1 将清除 HRTIM\_MISR 寄存器中的 MCMP1 标志。



## 22.5.4 HRTIM 主定时器 DMA/中断使能寄存器 (HRTIM\_MDIER)

偏移地址: 0x00C

复位值: 0x0000 0000

| 31      | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22   | 21   | 20   | 19   | 18   | 17   | 16   |
|---------|----|----|----|----|----|----|----|----|------|------|------|------|------|------|------|
| Resverd |    |    |    |    |    |    |    |    | MUPD | SYNC | MREP | MCMP | MCMP | MCMP | MCMP |
|         |    |    |    |    |    |    |    |    | DE   | DE   | DE   | 4DE  | 3DE  | 2DE  | 1DE  |
|         |    |    |    |    |    |    |    |    | rw   | rw   | rw   | rw   | rw   | rw   | rw   |
| 15      | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| Resverd |    |    |    |    |    |    |    |    | MUPD | SYNC | MREP | MCMP | MCMP | MCMP | MCMP |
|         |    |    |    |    |    |    |    |    | IE   | IE   | IE   | 4IE  | 3IE  | 2IE  | 1IE  |
|         |    |    |    |    |    |    |    |    | rw   | rw   | rw   | rw   | rw   | rw   | rw   |

Bit 31:23 保留, 必须保持复位值。

Bit 22 **MUPDDE**: 主定时器更新 DMA 请求使能 (Master update DMA request enable)

此位由软件设置和清除, 以使能/禁用主定时器更新 DMA 请求。

0: 主定时器更新 DMA 请求禁用

1: 主定时器更新 DMA 请求使能

Bit 21 **SYNCDE**: 同步输入 DMA 请求使能 (Sync input DMA request enable)

此位由软件设置和清除, 以使能/禁用同步输入 DMA 请求。

0: 同步输入 DMA 请求禁用

1: 同步输入 DMA 请求使能

Bit 20 **MREPDE**: 主定时器重复 DMA 请求使能 (Master repetition DMA request enable)

此位由软件设置和清除, 以使能/禁用主定时器重复 DMA 请求。

0: 重复 DMA 请求禁用

1: 重复 DMA 请求使能

Bit 19 **MCMP4DE**: 主定时器比较 4 DMA 请求使能 (Master compare 4 DMA request enable)

参考 MCMP1DE 的描述

Bit 18 **MCMP3DE**: 主定时器比较 3 DMA 请求使能 (Master compare 3 DMA request enable)

参考 MCMP1DE 的描述

Bit 17 **MCMP2DE**: 主定时器比较 2 DMA 请求使能 (Master compare 2 DMA request enable)

参考 MCMP1DE 的描述

Bit 16 **MCMP1DE**: 主定时器比较 1 DMA 请求使能 (Master compare 1 DMA request enable)

此位由软件设置和清除, 以使能/禁用主定时器比较 1 的 DMA 请求。

0: 比较 1 DMA 请求禁用

1: 比较 1 DMA 请求使能

Bit 15:7 保留, 必须保持复位值。

Bit 6 **MUPDIE**: 主定时器更新中断使能 (Master update interrupt enable)

此位由软件设置和清除, 以使能/禁用主定时器寄存器更新的中断。

0: 主定时器更新中断禁用

1: 主定时器更新中断使能

Bit 5 **SYNCIE**: 同步输入中断使能 (Sync input interrupt enable)

此位由软件设置和清除, 以使能/禁用同步输入中断。

0: 同步输入中断禁用

1: 同步输入中断使能

Bit 4 **MREPIE**: 主定时器重复中断使能 (Master repetition interrupt enable)

此位由软件设置和清除, 以使能/禁用主定时器重复中断。

0: 主定时器重复中断禁用

1: 主定时器重复中断使能

Bit 3 **MCMP4IE**: 主定时器比较 4 中断使能 (Master compare 4 interrupt enable)

参考 MCMP1IE 的描述

Bit 2 **MCMP3IE**: 主定时器比较 3 中断使能 (Master compare 3 interrupt enable)

参考 MCMP1IE 的描述

Bit 1 **MCMP2IE**: 主定时器比较 2 中断使能 (Master compare 2 interrupt enable)

参考 MCMP1IE 的描述

Bit 0 **MCMP1IE**: 主定时器比较 1 中断使能 (Master compare 1 interrupt enable)

此位由软件设置和清除, 以使能/禁用主定时器比较 1 中断。

0: 比较 1 中断禁用

1: 比较 1 中断使能

### 22.5.5 HRTIM 主定时器计数器寄存器 (HRTIM\_MCNTR)

偏移地址: 0x010

复位值: 0x0000 0000

|            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31         | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Resverd    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15         | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| MCNT[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw         | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:16 保留, 必须保持复位值。

Bit 15:0 **MCNT[15:0]**: 计数器值 (Counter value)

该位域存储主定时器的当前计数器值。这个寄存器仅当主定时器停止时(即 HRTIM\_MCR 寄存器中的 MCEN 位为 0) 才允许写入。

注: 当 HR 时钟的预分频比率低于 32 (即 CKPSC[2:0] 小于 5) 时, 计数器的最低有效位没有意义。这些位无法写入, 并且在读取时返回 0。

注: 如果设置的计数器值大于 HRTIM\_MPER 寄存器的值, 则定时器的行为无法得到保证。

### 22.5.6 HRTIM 主定时器周期寄存器 (HRTIM\_MPER)

偏移地址: 0x014

复位值: 0x0000 FFDF

|            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31         | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Resverd    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15         | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| MPER[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw         | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:16 保留, 必须保持复位值。

Bit 15:0 **MPER[15:0]**: 主定时器周期值 (Master timer period value)

此寄存器定义了计数器溢出值。

周期值必须大于或等于  $f_{HRTIM}$  时钟的 3 个周期, 即如果 CKPSC[2:0] = 0, 则为 0x60;

如果 CKPSC[2:0] = 1, 则为 0x30; 如果 CKPSC[2:0] = 2, 则为 0x18, 以此类推。

最大值为 0x0000FFDF。

### 22.5.7 HRTIM 主定时器重复寄存器 (HRTIM\_MREP)

偏移地址：0x018

复位值：0x0000 0000

|         |    |    |    |    |    |    |    |           |    |    |    |    |    |    |    |
|---------|----|----|----|----|----|----|----|-----------|----|----|----|----|----|----|----|
| 31      | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23        | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Resverd |    |    |    |    |    |    |    |           |    |    |    |    |    |    |    |
| 15      | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7         | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
|         |    |    |    |    |    |    |    | MREP[7:0] |    |    |    |    |    |    |    |
|         |    |    |    |    |    |    |    | rw        | rw | rw | rw | rw | rw | rw | rw |

Bit 31:8 保留，必须保持复位值。

Bit 7:0 **MREP[7:0]**：主定时器重复周期值 (Master timer repetition period value)

此寄存器保存主定时器计数器的重复周期值。该寄存器为预装载寄存器，禁止预装载时则为活动寄存器。

### 22.5.8 HRTIM 主定时器比较 1 寄存器 (HRTIM\_MCMP1R)

偏移地址：0x01C

复位值：0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Resverd     |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| MCMP1[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:16 保留，必须保持复位值。

Bit 15:0 **MCMP1[15:0]**：主定时器比较 1 值 (Master timer compare 1 value)

该寄存器保存主定时器比较 1 值。该寄存器为预装载寄存器，禁止预装载时则为活动寄存器。

比较值必须大于或等于 3 个  $f_{HRTIM}$  时钟周期，即如果 CKPSC[2:0] = 0，则为 0x60；如果 CKPSC[2:0] = 1，则为 0x30；如果 CKPSC[2:0] = 2，则为 0x18，以此类推。

### 22.5.9 HRTIM 主定时器比较 2 寄存器 (HRTIM\_MCMP2R)

偏移地址: 0x024

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Resverd     |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| MCMP2[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:16 保留, 必须保持复位值。

Bit 15:0 **MCMP2[15:0]**: 主定时器比较 2 值 (Master timer compare 2 value)

该寄存器保存主定时器比较 2 值。该寄存器为预装载寄存器, 禁止预装载时则为活动寄存器。

比较值必须大于或等于 3 个  $f_{HRTIM}$  时钟周期, 即如果 CKPSC[2:0] = 0, 则为 0x60; 如果 CKPSC[2:0] = 1, 则为 0x30; 如果 CKPSC[2:0] = 2, 则为 0x18, 以此类推。

### 22.5.10 HRTIM 主定时器比较 3 寄存器 (HRTIM\_MCMP3R)

偏移地址: 0x028

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Resverd     |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| MCMP3[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:16 保留, 必须保持复位值。

Bit 15:0 **MCMP3[15:0]**: 主定时器比较 3 值 (Master timer compare 3 value)

该寄存器保存主定时器比较 3 值。该寄存器为预装载寄存器, 禁止预装载时则为活动寄存器。

比较值必须大于或等于 3 个  $f_{HRTIM}$  时钟周期, 即如果 CKPSC[2:0] = 0, 则为 0x60; 如果 CKPSC[2:0] = 1, 则为 0x30; 如果 CKPSC[2:0] = 2, 则为 0x18, 以此类推。

## 22.5.11 HRTIM 主定时器比较 4 寄存器 (HRTIM\_MCMP4R)

偏移地址: 0x02C

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Resverd     |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| MCMP4[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:16 保留, 必须保持复位值。

Bit 15:0 **MCMP4[15:0]**: 主定时器比较 4 值 (Master timer compare 4 value)

该寄存器保存主定时器比较 4 值。该寄存器为预装载寄存器, 禁止预装载时则为活动寄存器。

比较值必须大于或等于 3 个  $f_{HRTIM}$  时钟周期, 即如果 CKPSC[2:0] = 0, 则为 0x60; 如果 CKPSC[2:0] = 1, 则为 0x30; 如果 CKPSC[2:0] = 2, 则为 0x18, 以此类推。

## 22.5.12 HRTIM 定时器 x 控制寄存器 (HRTIM\_TIMxCR) (x = A~F)

偏移地址：块 A：0x080

偏移地址：块 B：0x100

偏移地址：块 C：0x180

偏移地址：块 D：0x200

偏移地址：块 E：0x280

偏移地址：块 F：0x300

复位值：0x0000 0000

| 31           | 30 | 29           | 28 | 27            | 26           | 25          | 24          | 23  | 22     | 21   | 20     | 19   | 18          | 17     | 16  |
|--------------|----|--------------|----|---------------|--------------|-------------|-------------|-----|--------|------|--------|------|-------------|--------|-----|
| UPDGAT[3:0]  |    |              |    | PREEN         | DACSINC[1:0] |             | MSTU        | TEU | TDU    | TCU  | TBU    | TAU  | TxRSTU      | TxREPU | TFU |
| rw           | rw | rw           | rw | rw            | rw           | rw          | rw          | rw  | rw     | rw   | rw     | rw   | rw          | rw     | rw  |
| 15           | 14 | 13           | 12 | 11            | 10           | 9           | 8           | 7   | 6      | 5    | 4      | 3    | 2           | 1      | 0   |
| DELCMP4[1:0] |    | DELCMP2[1:0] |    | SYNCS<br>TRTx | SYNCR<br>STx | RSYNCR<br>U | INTLVD[1:0] |     | PSHPLL | HALF | RETRIG | CONT | CKPSCx[2:0] |        |     |
| rw           | rw | rw           | rw | rw            | rw           | rw          | rw          | rw  | rw     | rw   | rw     | rw   | rw          | rw     | rw  |



**Bit 31:28 UPDGAT[3:0]: 更新门控 (Update gating)**

这些位定义了更新操作相对于突发 DMA 事务和外部更新请求如何发生, 这些外部更新请求通过更新使能输入 hrtim\_upd\_en[3:1]进行。

更新事件可以包括: MSTU、TFU、TEU、TDU、TCU、TBU、TAU、TxRSTU、TxREPU。

0000: 更新独立于 DMA 突发传输发生

0001: 当 DMA 突发传输完成时发生更新

0010: 在 DMA 突发传输完成后的更新事件发生时进行更新

0011: 在 hrtim\_upd\_en1 的上升沿时发生更新

0100: 在 hrtim\_upd\_en2 的上升沿时发生更新

0101: 在 hrtim\_upd\_en3 的上升沿时发生更新

0110: 在 hrtim\_upd\_en1 上升沿后的更新事件发生时进行更新

0111: 在 hrtim\_upd\_en2 上升沿后的更新事件发生时进行更新

1000: 在 hrtim\_upd\_en3 上升沿后的更新事件发生时进行更新

其他: 保留

注: 在编程新值之前, 必须重置此位域。

对于UPDGAT[3:0]的值等于0001、0011、0100、0101时, 可能有多个并发的更新源(例如RSTU 和DMA 突发)。

**Bit 27 PREEN: 预加载使能 (Preload enable)**

此位使能寄存器的预加载机制, 并定义写入预加载寄存器时是写入有效寄存器还是预加载寄存器。

0: 预加载禁用: 直接对活动寄存器进行写访问

1: 预加载使能: 对预装载寄存器进行写访问

**Bit 26:25 DACSYNC[1:0]: DAC 同步 (DAC synchronization)**

当定时器更新发生时, 会生成一个 DAC 同步事件。这些位定义了 DAC 同步是在哪个输出上发送的 (详情参考章节: DAC 触发连接)。

00: 不生成 DAC 触发事件

01: 在 hrtim\_dac\_trg1 上生成触发事件

10: 在 hrtim\_dac\_trg2 上生成触发事件

11: 在 hrtim\_dac\_trg3 上生成触发事件

**Bit 24 MSTU: 主定时器更新 (Master timer update)**

当主定时器更新时, 会触发寄存器更新。

0: 由主定时器触发的更新禁用

1: 由主定时器触发的更新使能

**Bit 23 TEU: 定时器 E 更新 (Timer E update)**

当定时器 E 更新时, 会触发寄存器更新。

0: 由定时器 E 触发的更新禁用

1: 由定时器 E 触发的更新使能

注: 这个位是为HRTIM\_TIMECR 保留的。它仅对HRTIM\_TIMACR、HRTIM\_TIMBCR、HRTIM\_TIMCCR、HRTIM\_TIMDCR、HRTIM\_TIMFCR 可用。

**Bit 22 TDU: 定时器 D 更新 (Timer D update)**

当定时器 D 更新时, 会触发寄存器更新。

0: 由定时器 D 触发的更新禁用

1: 由定时器 D 触发的更新使能

注: 这个位是为 HRTIM\_TIMECR 保留的。它仅对 HRTIM\_TIMACR、HRTIM\_TIMBCR、HRTIM\_TIMCCR、HRTIM\_TIMDCR、HRTIM\_TIMFCR 可用。

Bit 21 TCU: 定时器 C 更新 (Timer C update)

当定时器 C 更新时, 会触发寄存器更新。

0: 由定时器 C 触发的更新禁用

1: 由定时器 C 触发的更新使能

注: 这个位是为 HRTIM\_TIMECR 保留的。它仅对 HRTIM\_TIMACR、HRTIM\_TIMBCR、HRTIM\_TIMCCR、HRTIM\_TIMDCR、HRTIM\_TIMFCR 可用。

Bit 20 TBU: 定时器 B 更新 (Timer B update)

当定时器 B 更新时, 会触发寄存器更新。

0: 由定时器 B 触发的更新禁用

1: 由定时器 B 触发的更新使能

注: 这个位是为 HRTIM\_TIMECR 保留的。它仅对 HRTIM\_TIMACR、HRTIM\_TIMBCR、HRTIM\_TIMCCR、HRTIM\_TIMDCR、HRTIM\_TIMFCR 可用。

Bit 19 TAU: 定时器 A 更新 (Timer A update)

当定时器 A 更新时, 会触发寄存器更新。

0: 由定时器 A 触发的更新禁用

1: 由定时器 A 触发的更新使能

注: 这个位是为 HRTIM\_TIMECR 保留的。它仅对 HRTIM\_TIMACR、HRTIM\_TIMBCR、HRTIM\_TIMCCR、HRTIM\_TIMDCR、HRTIM\_TIMFCR 可用。

Bit 18 TxRSTU: 定时器 x 复位更新 (Timer x reset update)

当定时器 x 的计数器复位或在连续模式下达到周期值后翻转到 0 时会触发寄存器更新。

0: 由定时器 x 复位或翻转触发的更新禁用

1: 由定时器 x 复位或翻转触发的更新使能

Bit 17 TxREPU: 定时器 x 重复更新 (Timer x repetition update)

当计数器翻转且 HRTIM\_REPx = 0 时, 会触发寄存器更新。

0: 重复时更新禁用

1: 重复时更新使能

Bit 16 TFU: 定时器 F 更新 (Timer F update)

当定时器 F 更新时, 会触发寄存器更新。

0: 由定时器 F 触发的更新禁用

1: 由定时器 F 触发的更新使能

注: 这个位是为 HRTIM\_TIMFCR 保留的。它仅对 HRTIM\_TIMACR、HRTIM\_TIMBCR、HRTIM\_TIMCCR、HRTIM\_TIMDCR、HRTIM\_TIMECR 可用。

Bit 15:14 DELCMP4[1:0]: CMP4 自动延迟模式 (CMP4 auto-delayed mode)

这个位域定义了比较寄存器是工作在标准模式 (当计数器值等于比较值时立即发出比较匹配), 还是工作在自动延迟模式 (参见“自动延迟模式”章节)。

这个位域定义了比较寄存器是工作在标准模式 (当计数器值等于比较值时立即发出比较匹配), 还是工作在自动延迟模式 (参见“自动延迟模式”章节)。

00: CMP4 寄存器始终有效 (标准比较模式)

01: CMP4 值在捕获 2 事件发生后重新计算并生效

10: CMP4 值在捕获 2 事件发生后重新计算并生效, 或者在比较 1 匹配后重新计算并生效 (如果缺少捕获 2 事件, 则作为超时功能)

11: CMP4 值在捕获 2 事件发生后重新计算并生效, 或者在比较 3 匹配后重新计算并生效 (如果缺少捕获事件, 则作为超时功能)

注: 一旦计数器使能 (TxCEN 位设置), 就不应修改这个位域。

Bit 13:12 **DELCMP2 [1:0]**: CMP2 自动延迟模式 (CMP2 auto-delayed mode)

这个位域定义了比较寄存器是工作在标准模式 (当计数器值等于比较值时立即发出比较匹配), 还是工作在自动延迟模式 (参见“自动延迟模式”章节)。

00: CMP2 寄存器始终活跃 (标准比较模式)

01: CMP2 值在捕获 1 事件发生后重新计算并生效

10: CMP2 值在捕获 1 事件发生后重新计算并生效, 或者在比较 1 匹配后重新计算并生效 (如果缺少捕获事件, 则作为超时功能)

11: CMP2 值在捕获 1 事件发生后重新计算并生效, 或者在比较 3 匹配后重新计算并生效 (如果缺少捕获事件, 则作为超时功能)

注: 一旦计数器使能 (TxCEN 位设置), 就不应修改这个位域。

Bit 11 **SYNCSTRTx**: 同步启动定时器 x (Synchronization starts timer x)

这个位定义了定时器 x 在同步事件发生后的行为:

0: 对定时器 x 无影响

1: 同步输入事件启动定时器 x

Bit 10 **SYNCRSTx**: 同步复位定时器 x (Synchronization resets timer x)

这个位定义了定时器 x 在同步事件发生后的行为:

0: 对定时器 x 无影响

1: 同步输入事件复位定时器 x

Bit 9 **RSYNCU**: 重新同步更新 (Re-synchronized update)

这个位指定了来自定时单元外部的更新源是否需要进行同步:

0: 来自相邻定时器 (当 MSTU、TAU、TBU、TCU、TDU、TEU、TFU 位设置时) 或来自软件更新 (TxSWU 位) 的更新会立即被考虑

1: 来自相邻定时器 (当 MSTU、TAU、TBU、TCU、TDU、TEU、TFU 位设置时) 或来自软件更新 (TxSWU 位) 的更新会在下一个重置/翻转事件时被考虑

注: 此位仅在 UPDGAT[3:0] = 0000 时有效, 否则会被忽略。

Bit 8:7 **INTLVD[1:0]**: 交错模式 (Interleaved mode)

此位域在 HALF 位被重置时有效, 用于使能交错模式。

00: 交错模式禁用

01: 三重交错模式: 当写入 HRTIM\_PERxR 寄存器时, HRTIM\_CMP1xR 有效寄存器会自动更新为  $HRTIM\_PERxR/3$  的值, HRTIM\_CMP2xR 有效寄存器会自动更新为  $2 \times (HRTIM\_PERxR/3)$  的值。

10: 四重交错模式: 当写入 HRTIM\_PERxR 寄存器时, HRTIM\_CMP1xR 有效寄存器会自动更新为  $HRTIM\_PERxR/4$  的值, HRTIM\_CMP2xR 有效寄存器会自动更新为  $HRTIM\_PERxR/2$  的值, HRTIM\_CMP3xR 有效寄存器会自动更新为  $3 \times (HRTIM\_PERxR/4)$  的值。

11: 交错模式禁用

Bit 6 **PSHPLL**: 推挽模式使能 (Push-pull mode enable)

此位用于使能推挽模式。

0: 推挽模式禁用

1: 推挽模式使能

注: 一旦计数器使能 (TxCEN 位设置), 就不应修改这个位域。

Bit 5 **HALF**: 半占空比模式使能 (Half mode enable)

此位用于使能半占空比模式: 当写入 HRTIM\_PERxR 寄存器时, HRTIM\_CMP1xR 有效寄存器会自动更新为 HRTIM\_PERxR/2 的值。

0: 半占空比模式禁用

1: 半占空比模式使能

Bit 4 **RETRIG**: 可再触发模式 (Re-triggerable mode)

此位定义了计数器在单次触发模式下的行为。

0: 定时器不可再触发: 如果计数器停止 (在单次触发模式下周期已过或在连续模式下计数器停止), 则执行计数器复位。

1: 定时器可再触发: 无论计数器的状态如何, 都会执行计数器复位。

Bit 3 **CONT**: 连续模式 (Continuous mode)

此位定义了定时器的模式。

0: 定时器以单次触发模式运行, 并在达到 TIMxPER 值时停止。

1: 定时器以连续模式运行, 并在达到 TIMxPER 值时翻转到零。

Bit 2:0 **CKPSCx[2:0]**: HRTIM 定时器 x 时钟预分频器 (HRTIM timer x clock prescaler)

这些位定义了主定时器分辨率时钟的预分频比例。

计数器的等效频率 ( $f_{\text{COUNTER}}$ ) 等于  $f_{\text{HRCK}} / 2^{\text{CKPSC}[2:0]}$ 。

一旦定时器使能, 就不能修改预分频比例。

### 22.5.13 HRTIM 定时器 x 中断状态寄存器 (HRTIM\_TIMxISR) (x = A~F)

偏移地址：块 A：0x084

偏移地址：块 B：0x104

偏移地址：块 C：0x184

偏移地址：块 D：0x204

偏移地址：块 E：0x284

偏移地址：块 F：0x304

复位值：0x0000 0000

| 31       | 30     | 29  | 28    | 27    | 26    | 25    | 24   | 23   | 22  | 21    | 20    | 19         | 18         | 17          | 16          |
|----------|--------|-----|-------|-------|-------|-------|------|------|-----|-------|-------|------------|------------|-------------|-------------|
| Reserved |        |     |       |       |       |       |      |      |     | O2CPY | O1CPY | O2<br>STAT | O1<br>STAT | IPP<br>STAT | CPP<br>STAT |
|          |        |     |       |       |       |       |      |      |     | r     | r     | r          | r          | r           | r           |
| 15       | 14     | 13  | 12    | 11    | 10    | 9     | 8    | 7    | 6   | 5     | 4     | 3          | 2          | 1           | 0           |
| Res      | DLYPRT | RST | RSTx2 | SETx2 | RSTx1 | SETx1 | CPT2 | CPT1 | UPD | Res   | REP   | CMP4       | CMP3       | CMP2        | CMP1        |
|          | r      | r   | r     | r     | r     | r     | r    | r    | r   |       | r     | r          | r          | r           | r           |

Bit 31:22 保留，必须保持复位值。

Bit 21 **O2CPY**：输出 2 副本 (Output 2 copy)

此状态位是输出级之前输出 2 状态的原始副本 (斩波、极性)，允许在延迟保护后重新使能输出之前检查当前输出状态。

0：输出 2 无效

1：输出 2 有效

Bit 20 **O1CPY**：输出 1 副本 (Output 1 copy)

此状态位是输出级之前输出 1 状态的原始副本 (斩波、极性)，允许在延迟保护后重新使能输出之前检查当前输出状态。

0：输出 1 无效

1：输出 1 有效

Bit 19 **O2STAT**：输出 2 状态 (Output 2 status)

此状态位指示延迟空闲保护触发时的输出 2 状态。进入任何新的延迟保护时，会更新此位。均衡空闲模式下，不会更新此位。

0：输出 2 无效

1：输出 2 有效

Bit 18 **O1STAT**：输出 1 状态 (Output 1 status)

此状态位指示延迟空闲保护触发时的输出 1 状态。进入任何新的延迟保护时，会更新此位。均衡空闲模式下，不会更新此位。

0：输出 1 无效

1：输出 1 有效

Bit 17 **IPPSTAT**：空闲推挽状态 (Idle push-pull Status)

此状态位指示触发保护时，在推挽模式、均衡故障模式或延迟空闲模式下将信号应用到哪路输出 (不考虑输出状态有效还是无效)。

0：输出 1 有效时进行保护，输出 2 强制无效

1：输出 2 有效时进行保护，输出 1 强制无效



Bit 16 **CPPSTAT**: 当前推挽状态 (Current push-pull status)

此状态位指示推挽模式下当前将信号应用到哪路输出上。此位仅在该配置中有意义。

0: 信号应用到输出 1 上, 输出 2 强制无效

1: 信号应用到输出 2 上, 输出 1 强制无效

Bit 15 保留, 必须保持复位值。

Bit 14 **DLYPRT**: 延迟保护标志 (Delayed protection flag)

此位指示延迟空闲或均衡空闲模式进入。

0: 没有发生延迟保护中断

1: 发生了延迟空闲或均衡空闲模式进入

Bit 13 **RST**: 复位和/或翻转中断标志 (Reset and/or roll-over interrupt flag)

定时器 x 在连续模式下复位或翻转时, 此位由硬件置 1。

0: 没有发生 TIMx 计数器复位/翻转中断

1: 发生了 TIMx 计数器复位/翻转中断

Bit 12 **RSTx2**: 输出 2 复位中断标志 (Output 2 reset interrupt flag)

请参考 RSTx1 的描述

Bit 11 **SETx2**: 输出 2 置位中断标志 (Output 2 set interrupt flag)

请参考 SETx1 的描述

Bit 10 **RSTx1**: 输出 1 复位中断标志 (Output 1 reset interrupt flag)

当 Tx1 输出复位时 (从有效模式变为无效模式), 此位由硬件置 1。

0: 没有发生 Tx1 输出复位中断

1: 发生了 Tx1 输出复位中断

Bit 9 **SETx1**: 输出 1 置位中断标志 (Output 1 set interrupt flag)

当 Tx1 输出复位时 (从有效模式变为无效模式), 此位由硬件置 1。

0: 没有发生 Tx1 输出置位中断

1: 发生了 Tx1 输出置位中断

Bit 8 **CPT2**: 捕获 2 中断标志 (Capture 2 interrupt flag)

请参考 CPT1 的描述。

Bit 7 **CPT1**: 捕获 1 中断标志 (Capture 1 interrupt flag)

发生定时器 x 捕获 1 事件时, 该位由硬件置 1。

0: 没有发生定时器 x 捕获 1 中断

1: 发生了定时器 x 捕获 1 中断

Bit 6 **UPD**: 更新中断标志 (Update interrupt flag)

发生定时器 x 捕获 1 事件时, 该位由硬件置 1。

0: 没有发生定时器 x 更新中断

1: 发生了定时器 x 更新中断

Bit 5 保留, 必须保持复位值。

Bit 4 **REP**: 重复中断标志 (Repetition interrupt flag)

定时器 x 重复周期已过时, 该位由硬件置 1。

0: 没有发生定时器 x 重复中断

1: 发生定时器 x 重复中断

Bit 3 **CMP4**: 比较 4 中断标志 (Compare 4 interrupt flag)

请参考 CMP1 的描述。

Bit 2 **CMP3**: 比较 3 中断标志 (Compare 3 interrupt flag)

请参考 CMP1 的描述。

Bit 1 **CMP2**: 比较 2 中断标志 (Compare 2 interrupt flag)

请参考 CMP1 的描述。

Bit 0 **CMP1**: 比较 1 中断标志 (Compare 1 interrupt flag)

当定时器 x 计数器值与在比较 1 寄存器中编程的值相匹配时, 此位由硬件置 1。

0: 没有发生比较 1 中断

1: 发生了比较 1 中断

## 22.5.14 HRTIM 定时器 x 中断清除寄存器 (HRTIM\_TIMxICR) (x = A~F)

偏移地址: 块 A: 0x088

偏移地址: 块 B: 0x108

偏移地址: 块 C: 0x188

偏移地址: 块 D: 0x208

偏移地址: 块 E: 0x288

偏移地址: 块 F: 0x308

复位值: 0x0000 0000

|          |             |          |            |            |            |            |           |           |          |     |          |           |           |           |           |
|----------|-------------|----------|------------|------------|------------|------------|-----------|-----------|----------|-----|----------|-----------|-----------|-----------|-----------|
| 31       | 30          | 29       | 28         | 27         | 26         | 25         | 24        | 23        | 22       | 21  | 20       | 19        | 18        | 17        | 16        |
| Reserved |             |          |            |            |            |            |           |           |          |     |          |           |           |           |           |
| 15       | 14          | 13       | 12         | 11         | 10         | 9          | 8         | 7         | 6        | 5   | 4        | 3         | 2         | 1         | 0         |
| Res      | DLYPR<br>TC | RST<br>C | RSTx2<br>C | SETx2<br>C | RSTx1<br>C | SETx1<br>C | CPT2<br>C | CPT1<br>C | UPD<br>C | Res | REP<br>C | CMP4<br>C | CMP3<br>C | CMP2<br>C | CMP1<br>C |
|          | w           | w        | w          | w          | w          | w          | w         | w         | w        |     | w        | w         | w         | w         | w         |

Bit 31:15 保留, 必须保持复位值。

Bit 14 **DLYPRTC**: 延迟保护标志清除 (Delayed protection flag clear)

向此位写入 1 可清除 HRTIM\_TIMxISR 寄存器中的 DLYPRT 标志。

Bit 13 **RSTC**: 复位中断标志清除 (Reset interrupt flag clear)

向此位写入 1 可清除 HRTIM\_TIMxISR 寄存器中的 RST 标志。

Bit 12 **RSTx2C**: 输出 2 复位标志清除 (Output 2 reset flag clear)

向此位写入 1 可清除 HRTIM\_TIMxISR 寄存器中的 RSTx2 标志。

Bit 11 **SETx2C**: 输出 2 置位标志清除 (Output 2 set flag clear)

向此位写入 1 可清除 HRTIM\_TIMxISR 寄存器中的 SETx2 标志。

Bit 10 **RSTx1C**: 输出 1 复位标志清除 (Output 1 reset flag clear)

向此位写入 1 可清除 HRTIM\_TIMxISR 寄存器中的 RSTx1 标志。

Bit 9 **SETx1C**: 输出 1 置位标志清除 (Output 1 set flag clear)

向此位写入 1 可清除 HRTIM\_TIMxISR 寄存器中的 SETx1 标志。



Bit 8 **CPT2C**: 捕获 2 中断标志清除 (Capture 2 interrupt flag clear)  
 向此位写入 1 可清除 HRTIM\_TIMxISR 寄存器中的 CPT2 标志。

Bit 7 **CPT1C**: 捕获 1 中断标志清除 (Capture 1 interrupt flag clear)  
 向此位写入 1 可清除 HRTIM\_TIMxISR 寄存器中的 CPT1 标志。

Bit 6 **UPDC**: 更新中断标志清除 (Update interrupt flag clear)  
 向此位写入 1 可清除 HRTIM\_TIMxISR 寄存器中的 UPD 标志。

Bit 5 保留, 必须保持复位值。

Bit 4 **REPC**: 重复中断标志清除 (Repetition interrupt flag clear)  
 向此位写入 1 可清除 HRTIM\_TIMxISR 寄存器中的 REP 标志。

Bit 3 **CMP4C**: 比较 4 中断标志清除 (Compare 4 interrupt flag clear)  
 向此位写入 1 可清除 HRTIM\_TIMxISR 寄存器中的 CMP4 标志。

Bit 2 **CMP3C**: 比较 3 中断标志清除 (Compare 3 interrupt flag clear)  
 向此位写入 1 可清除 HRTIM\_TIMxISR 寄存器中的 CMP3 标志。

Bit 1 **CMP2C**: 比较 2 中断标志清除 (Compare 2 interrupt flag clear)  
 向此位写入 1 可清除 HRTIM\_TIMxISR 寄存器中的 CMP2 标志。

Bit 0 **CMP1C**: 比较 1 中断标志清除 (: Compare 1 interrupt flag clear)  
 向此位写入 1 可清除 HRTIM\_TIMxISR 寄存器中的 CMP1 标志。

## 22.5.15 HRTIM 定时器 x DMA 中断使能寄存器 (HRTIM\_TIMxDIER) (x = A~F)

偏移地址: 块 A: 0x08C

偏移地址: 块 B: 0x10C

偏移地址: 块 C: 0x18C

偏移地址: 块 D: 0x20C

偏移地址: 块 E: 0x28C

偏移地址: 块 F: 0x30C

复位值: 0x0000 0000

| 31  | 30           | 29    | 28          | 27          | 26          | 25          | 24         | 23         | 22    | 21  | 20        | 19         | 18         | 17         | 16         |
|-----|--------------|-------|-------------|-------------|-------------|-------------|------------|------------|-------|-----|-----------|------------|------------|------------|------------|
| Res | DLYPR<br>TDE | RSTDE | RSTx2<br>DE | SETx2<br>DE | RSTx1<br>DE | SETx1<br>DE | CPT2<br>DE | CPT1<br>DE | UPDDE | Res | REP<br>DE | CMP4<br>DE | CMP3<br>DE | CMP2<br>DE | CMP1<br>DE |
|     | r            | r     | r           | r           | r           | r           | r          | r          | r     |     | r         | r          | r          | r          | r          |
| 15  | 14           | 13    | 12          | 11          | 10          | 9           | 8          | 7          | 6     | 5   | 4         | 3          | 2          | 1          | 0          |
| Res | DLYPR<br>TIE | RSTIE | RSTx2<br>IE | SETx2<br>IE | RSTx1<br>IE | SETx1<br>IE | CPT2<br>IE | CPT1<br>IE | UPDIE | Res | REP<br>IE | CMP4<br>IE | CMP3<br>IE | CMP2<br>IE | CMP1<br>IE |
|     | r            | r     | r           | r           | r           | r           | r          | r          | r     |     | r         | r          | r          | r          | r          |

Bit 31 保留，必须保持复位值。

Bit 30 **DLYPRTDE**: 延迟保护 DMA 请求使能 (Delayed protection DMA request enable)

此位由软件设置和清除，以使能/禁用延迟保护的 DMA 请求。

0: 延迟保护 DMA 请求禁用

1: 延迟保护 DMA 请求使能

Bit 29 **RSTDE**: 复位/翻转 DMA 请求使能 (Reset/roll-over DMA request enable)

此位由软件设置和清除以在定时器 x 计数器复位或连续模式下翻转时使能/禁用 DMA 请求。

0: 定时器 x 计数器复位/翻转 DMA 请求禁用

1: 定时器 x 计数器复位/翻转 DMA 请求使能

Bit 28 **RSTx2DE**: 输出 2 复位 DMA 请求使能 (Output 2 reset DMA request enable)

参考 RSTx1DE 的描述。

Bit 27 **SETx2DE**: 输出 2 置位 DMA 请求使能 (Output 2 set DMA request enable)

参考 SETx1DE 的描述。

Bit 26 **RSTx1DE**: 输出 1 复位 DMA 请求使能 (Output 1 reset DMA request enable)

此位由软件设置和清除，以使能/禁用 Tx1 输出复位的 DMA 请求。

0: Tx1 输出复位 DMA 请求禁用

1: Tx1 输出复位 DMA 请求使能

Bit 25 **SETx1DE**: 输出 1 设置 DMA 请求使能 (Output 1 set DMA request enable)

此位由软件设置和清除，以使能/禁用 Tx1 输出设置的 DMA 请求。

0: Tx1 输出置位 DMA 请求禁用

1: Tx1 输出置位 DMA 请求使能

Bit 24 **CPT2DE**: 捕获 2 DMA 请求使能 (Capture 2 DMA request enable)

参考 CPT1DE 的描述。

Bit 23 **CPT1DE**: 捕获 1 DMA 请求使能 (Capture 1 DMA request enable)

此位由软件设置和清除，以使能/禁用捕获 1 的 DMA 请求。

0: 捕获 1 DMA 请求禁用

1: 捕获 1 DMA 请求使能

Bit 22 **UPDDE**: 更新 DMA 请求使能 (Update DMA request enable)

此位由软件设置和清除，以在更新事件时使能/禁用 DMA 请求。

0: 更新 DMA 请求禁用

1: 更新 DMA 请求使能

Bit 21 保留，必须保持复位值。

Bit 20 **REPDE**: 重复 DMA 请求使能 (Repetition DMA request enable)

此位由软件设置和清除，以在重复事件时使能/禁用 DMA 请求。

0: 重复 DMA 请求禁用

1: 重复 DMA 请求使能

Bit 19 **CMP4DE**: 比较 4 DMA 请求使能 (Compare 4 DMA request enable)

参考 CMP1DE 的描述。

Bit 18 **CMP3DE**: 比较 3 DMA 请求使能 (Compare 3 DMA request enable)

参考 CMP1DE 的描述。

Bit 17 **CMP2DE**: 比较 2 DMA 请求使能 (Compare 2 DMA request enable)

参考 CMP1DE 的描述。

Bit 16 **CMP1DE**: 比较 1 DMA 请求使能 (Compare 1 DMA request enable)

此位由软件设置和清除, 以使能/禁用比较 1 的 DMA 请求。

0: 比较 1 DMA 请求禁用

1: 比较 1 DMA 请求使能

Bit 15 保留, 必须保持复位值。

Bit 14 **DLYPRTIE**: 延迟保护中断使能 (Delayed protection interrupt enable)

此位由软件设置和清除, 以使能/禁用延迟保护的中断。

0: 延迟保护中断禁用

1: 延迟保护中断使能

Bit 13 **RSTIE**: 复位/翻转中断使能 (Reset/roll-over interrupt enable)

此位由软件设置和清除, 以在定时器 x 计数器复位或在连续模式下翻转时使能/禁用中断。

0: 定时器 x 计数器复位/翻转中断禁用

1: 定时器 x 计数器复位/翻转中断使能

Bit 12 **RSTx2IE**: 输出 2 复位中断使能 (Output 2 reset interrupt enable)

请参考 RSTx1IE 的描述。

Bit 11 **SETx2IE**: 输出 2 设置中断使能 (Output 2 set interrupt enable)

请参考 SETx1IE 的描述。

Bit 10 **RSTx1IE**: 输出 1 复位中断使能 (Output 1 reset interrupt enable)

此位由软件设置和清除, 以使能/禁用 Tx1 输出复位的中断。

0: Tx1 输出复位中断禁用

1: Tx1 输出复位中断使能

Bit 9 **SETx1IE**: 输出 1 设置中断使能 (Output 1 set interrupt enable)

此位由软件设置和清除, 以使能/禁用 Tx1 输出设置的中断。

0: Tx1 输出设置中断禁用

1: Tx1 输出设置中断使能

Bit 8 **CPT2IE**: 捕获 2 中断使能 (Capture 2 interrupt enable)

请参考 CPT1IE 的描述。

Bit 7 **CPT1IE**: 捕获 1 中断使能 (Capture 1 interrupt enable)

此位由软件设置和清除, 以使能/禁用捕获 1 的中断。

0: 捕获 1 中断禁用

1: 捕获 1 中断使能

Bit 6 **UPDIE**: 更新中断使能 (Update interrupt enable)

此位由软件设置和清除, 以使能/禁用更新事件的中断。

0: 更新中断禁用

1: 更新中断使能

Bit 5 保留, 必须保持复位值。

Bit 4 **REPIE**: 重复中断使能 (Repetition interrupt enable)

此位由软件设置和清除, 以使能/禁用重复事件的中断。

0: 重复中断禁用

1: 重复中断使能

Bit 3 **CMP4IE**: 比较 4 中断使能 (Compare 4 interrupt enable)

请参考 CMP1IE 的描述。

Bit 2 **CMP3IE**: 比较 3 中断使能 (Compare 3 interrupt enable)

请参考 CMP1IE 的描述。

Bit 1 **CMP2IE**: 比较 2 中断使能 (Compare 2 interrupt enable)

请参考 CMP1IE 的描述。

Bit 0 **CMP1IE**: 比较 1 中断使能 (Compare 1 interrupt enable)

此位由软件设置和清除, 以使能/禁用比较 1 的中断。

0: 比较 1 中断禁用

1: 比较 1 中断使能

## 22.5.16 HRTIM 定时器 x 计数器寄存器 (HRTIM\_CNTxR) (x = A~F)

偏移地址：块 A：0x090

偏移地址：块 B：0x110

偏移地址：块 C：0x190

偏移地址：块 D：0x210

偏移地址：块 E：0x290

偏移地址：块 F：0x310

复位值：0x0000 0000

|            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31         | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Resverd    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15         | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CNTx[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw         | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:16 保留，必须保持复位值。

Bit 15:0 **CNTx[15:0]**：定时器 x 计数器值 (Timer x counter value)

此寄存器保存了定时器 x 的计数值。该值仅当定时器停止 (HRTIM\_TIMxCR 寄存器中的 TxCEN=0) 时才能写入。

注：当 HR 时钟预分频比低于 32 (CKPSC[2:0] 小于 5) 时，计数器的最低有效位并不重要。它们无法被写入，并且在读取时返回 0。

注：如果计数值大于 HRTIM\_PERxR 寄存器的值，则无法保证定时器的行为。

## 22.5.17 HRTIM 定时器 x 周期寄存器 (HRTIM\_PERxR) (x = A~F)

偏移地址：块 A：0x094

偏移地址：块 B：0x114

偏移地址：块 C：0x194

偏移地址：块 D：0x214

偏移地址：块 E：0x294

偏移地址：块 F：0x314

复位值：0x0000 FFDF

|            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31         | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Resverd    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15         | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| PERx[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw         | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:16 保留，必须保持复位值。

Bit 15:0 **PERx[15:0]**：定时器 x 的周期值 (Timer x period value)

此寄存器保存定时器 x 的周期值。

如果禁止预装载，该寄存器保存预装载寄存器的内容和活动寄存器的内容。

周期值必须大于或等于 3 个  $f_{HRTIM}$  时钟周期，即如果 CKPSC[2:0]=0，则为 0x60；如果 CKPSC[2:0]=1，则为 0x30；如果 CKPSC[2:0]=2，则为 0x18；以此类推。

最大值为 0x0000 FFDF。

## 22.5.18 HRTIM 定时器 x 重复寄存器 (HRTIM\_REPxR) (x = A~F)

偏移地址：块 A：0x098

偏移地址：块 B：0x118

偏移地址：块 C：0x198

偏移地址：块 D：0x218

偏移地址：块 E：0x298

偏移地址：块 F：0x318

复位值：0x0000 0000

|         |    |    |    |    |    |    |    |           |    |    |    |    |    |    |    |
|---------|----|----|----|----|----|----|----|-----------|----|----|----|----|----|----|----|
| 31      | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23        | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Resverd |    |    |    |    |    |    |    |           |    |    |    |    |    |    |    |
| 15      | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7         | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Resverd |    |    |    |    |    |    |    | REPx[7:0] |    |    |    |    |    |    |    |
|         |    |    |    |    |    |    |    | rw        | rw | rw | rw | rw | rw | rw | rw |

Bit 31:16 保留，必须保持复位值。

Bit 15:0 **REPx[7:0]**：定时器 x 的重复周期值 (Timer x repetition period value)

此寄存器保存重复周期值。

该寄存器保存预装载寄存器的内容，如果禁止预装载，则会保存活动寄存器的内容。

## 22.5.19 HRTIM 定时器 x 比较 1 寄存器 (HRTIM\_CMP1xR) (x = A~F)

偏移地址：块 A：0x09C

偏移地址：块 B：0x11C

偏移地址：块 C：0x19C

偏移地址：块 D：0x21C

偏移地址：块 E：0x29C

偏移地址：块 F：0x31C

复位值：0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Resverd     |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CMP1x[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:16 保留，必须保持复位值。

Bit 15:0 **CMP1x[15:0]**：定时器 x 的比较 1 值 (Timer x compare 1 value)

此寄存器保存比较 1 值。

如果预装载功能被禁用，此寄存器将保存预装载寄存器的内容或有效寄存器的内容。

比较值必须为 0 或大于或等于 3 个  $f_{HRTIM}$  时钟周期，即如果 CKPSC[2:0]=0，则为 0x60；如果 CKPSC[2:0]=1，则为 0x30；如果 CKPSC[2:0]=2，则为 0x18 以此类推。0 值的编程遵循“空占空比异常情况”一节中描述的用例。

## 22.5.20 HRTIM 定时器 x 比较 1 复合寄存器 (HRTIM\_CMP1CxR) (x = A~F)

偏移地址：块 A：0x0A0

偏移地址：块 B：0x120

偏移地址：块 C：0x1A0

偏移地址：块 D：0x220

偏移地址：块 E：0x2A0

偏移地址：块 F：0x320

复位值：0x0000 0000

|             |    |    |    |    |    |    |    |           |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|-----------|----|----|----|----|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23        | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved    |    |    |    |    |    |    |    | REPx[7:0] |    |    |    |    |    |    |    |
|             |    |    |    |    |    |    |    | rw        | rw | rw | rw | rw | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7         | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CMP1x[15:0] |    |    |    |    |    |    |    |           |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw        | rw | rw | rw | rw | rw | rw | rw |

Bit 31:16 保留，必须保持复位值。

Bit 23:16 **REPx[7:0]**：定时器 x 的重复值（HRTIM\_REPx 寄存器中的别名）

此位域是 HRTIMx\_REPxR 寄存器中 REPx[7:0]位域的别名。

Bit 15:0 **CMP1x[15:0]**：定时器 x 的比较值 1（Timer x compare 1 value）

此位域是 HRTIMx\_CMP1xR 寄存器中 CMP1x[15:0]位域的别名。

## 22.5.21 HRTIM 定时器 x 比较 2 寄存器 (HRTIM\_CMP2xR) (x = A~F)

偏移地址：块 A：0x0A4

偏移地址：块 B：0x124

偏移地址：块 C：0x1A4

偏移地址：块 D：0x224

偏移地址：块 E：0x2A4

偏移地址：块 F：0x324

复位值：0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Resverd     |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CMP2x[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:16 保留，必须保持复位值。

Bit 15:0 **CMP2x[15:0]**：定时器 x 的比较 2 值（Timer x compare 2 value）

此寄存器保存比较 2 值。

该寄存器保存预装载寄存器的内容，如果禁止预装载，则会保存活动寄存器的内容。

比较值必须为 0 或大于或等于 3 个  $f_{HRTIM}$  时钟周期，即如果 CKPSC[2:0]=0，则为 0x60；如果 CKPSC[2:0]=1，则为 0x30；如果 CKPSC[2:0]=2，则为 0x18 以此类推。

如果通过 HRTIM\_TIMxCR 中的 DELCMP2[1:0]位使能，此寄存器将作为自动延迟比较寄存器工作。



## 22.5.22 HRTIM 定时器 x 比较 3 寄存器 (HRTIM\_CMP3xR) (x = A~F)

偏移地址：块 A：0x0A8

偏移地址：块 B：0x128

偏移地址：块 C：0x1A8

偏移地址：块 D：0x228

偏移地址：块 E：0x2A8

偏移地址：块 F：0x328

复位值：0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Resverd     |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CMP3x[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:16 保留，必须保持复位值。

Bit 15:0 **CMP3x[15:0]**：定时器 x 的比较 3 值 (Timer x compare 3 value)

此寄存器保存比较 3 值。

该寄存器保存预装载寄存器的内容，如果禁止预装载，则会保存活动寄存器的内容。

比较值必须为 0 或大于或等于 3 个  $f_{HRTIM}$  时钟周期，即如果 CKPSC[2:0]=0，则为 0x60；如果 CKPSC[2:0]=1，则为 0x30；如果 CKPSC[2:0]=2，则为 0x18 以此类推。

0 值的编程遵循“空占空比异常情况”一节中描述的用例。

## 22.5.23 HRTIM 定时器 x 比较 4 寄存器 (HRTIM\_CMP4xR) (x = A~F)

偏移地址：块 A：0x0AC

偏移地址：块 B：0x12C

偏移地址：块 C：0x1AC

偏移地址：块 D：0x22C

偏移地址：块 E：0x2AC

偏移地址：块 F：0x32C

复位值：0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Resverd     |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CMP4x[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:16 保留，必须保持复位值。

Bit 15:0 **CMP4x[15:0]**：定时器 x 的比较 4 值 (Timer x compare 4 value)

此寄存器保存比较 4 值。

该寄存器保存预装载寄存器的内容，如果禁止预装载，则会保存活动寄存器的内容。

比较值必须为 0 或大于或等于 3 个  $f_{HRTIM}$  时钟周期，即如果 CKPSC[2:0]=0，则为 0x60；如果 CKPSC[2:0]=1，则为 0x30；如果 CKPSC[2:0]=2，则为 0x18 以此类推。

如果通过 HRTIM\_TIMxCR 中的 DELCMP4[1:0]位使能，此寄存器将作为自动延迟比较寄存器工作。

## 22.5.24 HRTIM 定时器 x 捕获 1 寄存器 (HRTIM\_CPT1xR) (x = A~F)

偏移地址：块 A：0x0B0

偏移地址：块 B：0x130

偏移地址：块 C：0x1B0

偏移地址：块 D：0x230

偏移地址：块 E：0x2B0

偏移地址：块 F：0x330

复位值：0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |     |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|-----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16  |
| Reserved    |    |    |    |    |    |    |    |    |    |    |    |    |    |    | DIR |
|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    | r   |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0   |
| CPT1x[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |     |
| r           | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r   |

Bit 31:17 保留，必须保持复位值。

Bit 16 **DIR**：定时器 x 捕获 1 方向状态 (Timer x capture 1 direction status)

此寄存器保存捕获 1 事件发生时的计数方向值：

0：定时器正在向上计数

1：定时器正在向下计数

在向上计数模式 (UDM 位复位) 下，DIR 位始终读取为 0。

Bit 15:0 **CPT1x[15:0]**：定时器 x 捕获 1 值 (Timer x capture 1 value)

此寄存器保存捕获 1 事件发生时的计数器值。

**注意：** 在上下计数模式 (UDM 位设置为 1) 下，捕获值是指：

- 向上计数时的计数复位
- 向下计数时的 PER 事件

DIR 位允许在读取捕获值时区分上下计数阶段。

**注意：** 这是一个常规分辨率寄存器：对于 HR 时钟预分频比低于 32 (CKPSC[2:0] < 5) 的情况，计数器的最低有效位不重要。它们无法写入，并且在读取时返回 0。

## 22.5.25 HRTIM 定时器 x 捕获 2 寄存器 (HRTIM\_CPT2xR) (x = A~F)

偏移地址：块 A：0x0B4

偏移地址：块 B：0x134

偏移地址：块 C：0x1B4

偏移地址：块 D：0x234

偏移地址：块 E：0x2B4

偏移地址：块 F：0x334

复位值：0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |     |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|-----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16  |
| Reserved    |    |    |    |    |    |    |    |    |    |    |    |    |    |    | DIR |
|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    | r   |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0   |
| CPT2x[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |     |
| r           | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r   |

Bit 31:17 保留，必须保持复位值。

Bit 16 **DIR**：定时器 x 捕获 1 方向状态 (Timer x capture 1 direction status)

此寄存器保存捕获 1 事件发生时的计数方向值：

0：定时器正在向上计数

1：定时器正在向下计数

在向上计数模式 (UDM 位复位) 下，DIR 位始终读取为 0。

Bit 15:0 **CPT2x[15:0]**：定时器 x 捕获 2 值 (Timer x capture 2 value)

此寄存器保存捕获 2 事件发生时的计数器值。

**注意：** 在上下计数模式 (UDM 位设置为1) 下，捕获值是指：

- 向上计数时的计数复位
- 向下计数时的 PER 事件

DIR 位允许在读取捕获值时区分上下计数阶段。

**注意：** 这是一个常规分辨率寄存器：对于 HR 时钟预分频比低于 32 (CKPSC[2:0] < 5) 的情况，计数器的最低有效位不重要。它们无法写入，并且在读取时返回 0。

## 22.5.26 HRTIM 定时器 x 死区时间寄存器 (HRTIM\_DT<sub>x</sub>R) (x = A~F)

偏移地址：块 A：0x0B8

偏移地址：块 B：0x138

偏移地址：块 C：0x1B8

偏移地址：块 D：0x238

偏移地址：块 E：0x2B8

偏移地址：块 F：0x338

复位值：0x0000 0000

| 31         | 30          | 29       | 28          | 27 | 26 | 25    | 24        | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|------------|-------------|----------|-------------|----|----|-------|-----------|----|----|----|----|----|----|----|----|
| DTFLK<br>x | DTFSL<br>Kx | Reserved |             |    |    | SDTFx | DTFx[8:0] |    |    |    |    |    |    |    |    |
| rwo        | rwo         |          |             |    |    | rw    | rw        | rw | rw | rw | rw | rw | rw | rw | rw |
| 15         | 14          | 13       | 12          | 11 | 10 | 9     | 8         | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| DTRLK<br>x | DTRSL<br>Kx | Res      | DTPRSC[1:0] |    |    | SDTRx | DTRx[8:0] |    |    |    |    |    |    |    |    |
| rwo        | rwo         |          | rw          | rw | rw | rw    | rw        | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31 **DTFLKx**：死区时间下降沿锁定 (Deadtime falling lock)

若使能这一仅可写入一次的位，则会禁止修改死区 (符号和值)。

0：可写入死区下降沿值和符号

1：只能读取死区下降沿值和符号

注：此位不能预加载。

Bit 30 **DTFSLKx**：死区时间下降沿符号锁定 (Deadtime falling sign lock)

若使能这一仅可写入一次的位，则会禁止修改死区下降沿的符号。

0：可写入死区下降沿符号

1：只能读取死区下降沿符号

注：此位不能预加载。

Bit 29:26 保留，必须保持复位值。

Bit 25 **SDTFx**：死区时间下降沿符号值 (Sign deadtime falling value)

该寄存器决定了死区为正值 (信号不重叠) 还是负值 (信号重叠)。

0：死区在下降沿为正值

1：死区在下降沿为负值

Bit 24:16 **DTFx[8:0]**：死区时间下降沿值 (Deadtime falling value)

该寄存器保存参考 PWM 信号下降沿之后的死区值。

$$t_{DTF} = DTFx[8:0] \times t_{DTG}$$

Bit 15 **DTRLKx**：死区时间上升沿锁定 (Deadtime rising lock)

若使能这一仅可写入一次的位，则会禁止修改死区 (符号和值)。

0：可写入死区上升沿值和符号

1：只能读取死区上升沿值和符号

注：此位不能预加载。

Bit 14 **DTRSLKx**: 死区时间上升沿符号锁定 (Deadtime rising sign lock)

若使能这一仅可写入一次的位, 则会禁止修改死区上升沿的符号。

0: 可写入死区上升沿符号

1: 只能读取死区上升沿符号

注: 此位不能预加载。

Bit 13 保留, 必须保持复位值。

Bit 12:10 **DTPRSC[2:0]**: 死区时间预分频器 (Deadtime prescaler)

该寄存器保存死区时钟预分频器的值。

$$t_{DTG} = (2^{(DTPRSC[2:0])}) \times (t_{HRTIM} / 8)$$

(例如: 000: 651 ps, 001: 1.302 ns,...)

如果使能了任何锁定位 (DTFLKs, DTFSLKx, DTRLKx, DTRSLKx), 此位域将变为只读。

Bit 9 **SDTRx**: 死区时间上升沿符号值 (Sign deadtime rising value)

该寄存器决定了死区为正值还是负值 (信号重叠)。

0: 死区在上升沿为正值

1: 死区在上升沿为负值

Bit 8:0 **DTRx[2:0]**: 死区时间上升沿值 (Deadtime rising value)

该寄存器保存参考 PWM 信号上升沿之后的死区值。

$$t_{DTR} = DTRx[8:0] \times t_{DTG}$$

## 22.5.27 HRTIM 定时器 x 输出 1 置位寄存器 (HRTIM\_SETx1R) (x = A~F)

偏移地址: 块 A: 0x0BC

偏移地址: 块 B: 0x13C

偏移地址: 块 C: 0x1BC

偏移地址: 块 D: 0x23C

偏移地址: 块 E: 0x2BC

偏移地址: 块 F: 0x33C

复位值: 0x0000 0000

| 31           | 30            | 29           | 28           | 27           | 26           | 25           | 24           | 23           | 22           | 21           | 20           | 19           | 18           | 17           | 16           |
|--------------|---------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| UP<br>DATE   | EXT<br>EVNT10 | EXT<br>EVNT9 | EXT<br>EVNT8 | EXT<br>EVNT7 | EXT<br>EVNT6 | EXT<br>EVNT5 | EXT<br>EVNT4 | EXT<br>EVNT3 | EXT<br>EVNT2 | EXT<br>EVNT1 | TIM<br>EVNT9 | TIM<br>EVNT8 | TIM<br>EVNT7 | TIM<br>EVNT6 | TIM<br>EVNT5 |
| rw           | rw            | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           |
| 15           | 14            | 13           | 12           | 11           | 10           | 9            | 8            | 7            | 6            | 5            | 4            | 3            | 2            | 1            | 0            |
| TIM<br>EVNT4 | TIM<br>EVNT3  | TIM<br>EVNT2 | TIM<br>EVNT1 | MST<br>CMP4  | MST<br>CMP3  | MST<br>CMP2  | MST<br>CMP1  | MST<br>PER   | CMP4         | CMP3         | CMP2         | CMP1         | PER          | RESYN<br>C   | SST          |
| rw           | rw            | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           |

Bit 31 **UPDATE**: 寄存器更新 (Registers update) (从预装载寄存器传输到活动寄存器)

寄存器更新事件会强制将输出设为其有效状态。

Bit 30 **EXTEVNT10**: 外部事件 10 (External event 10)

参考 EXTEVNT1 的描述。

Bit 29 **EXTEVNT9**: 外部事件 9 (External event 9)

参考 EXTEVNT1 的描述。

Bit 28 **EXTEVNT8**: 外部事件 8 (External event 8)

参考 EXTEVNT1 的描述。

Bit 27 **EXTEVNT7**: 外部事件 7 (External event 7)

参考 EXTEVNT1 的描述。

Bit 26 **EXTEVNT6**: 外部事件 6 (External event 6)

参考 EXTEVNT1 的描述。

Bit 25 **EXTEVNT5**: 外部事件 5 (External event 5)

参考 EXTEVNT1 的描述。

Bit 24 **EXTEVNT4**: 外部事件 4 (External event 4)

参考 EXTEVNT1 的描述。

Bit 23 **EXTEVNT3**: 外部事件 3 (External event 3)

参考 EXTEVNT1 的描述。

Bit 22 **EXTEVNT2**: 外部事件 2 (External event 2)

参考 EXTEVNT1 的描述。

Bit 21 **EXTEVNT1**: 外部事件 1 (External event 1)

外部事件 1 会强制将输出设为其有效状态。

Bit 20 **TIMEVNT9**: 定时器事件 9 (Timer event 9)

参考 TIMEVNT1 的描述。

Bit 19 **TIMEVNT8**: 定时器事件 8 (Timer event 8)

参考 TIMEVNT1 的描述。

Bit 18 **TIMEVNT7**: 定时器事件 7 (Timer event 7)

参考 TIMEVNT1 的描述。

Bit 17 **TIMEVNT6**: 定时器事件 6 (Timer event 6)

参考 TIMEVNT1 的描述。

Bit 16 **TIMEVNT5**: 定时器事件 5 (Timer event 5)

参考 TIMEVNT1 的描述。

Bit 15 **TIMEVNT4**: 定时器事件 4 (Timer event 4)

参考 TIMEVNT1 的描述。

Bit 14 **TIMEVNT3**: 定时器事件 3 (Timer event 3)

参考 TIMEVNT1 的描述。

Bit 13 **TIMEVNT2**: 定时器事件 2 (Timer event 2)

参考 TIMEVNT1 的描述。

Bit 12 **TIMEVNT1**: 定时器事件 1 (Timer event 1)

定时器事件 1 会强制将输出设为其有效状态

Bit 11 **MSTCMP4**: 主比较 4 (Master compare 4)

主定时器比较 4 事件会强制将输出设为其有效状态。

Bit 10 **MSTCMP3**: 主比较 3 (Master compare 3)

主定时器比较 3 事件会强制将输出设为其有效状态。

Bit 9 **MSTCMP2**: 主比较 2 (Master compare 2)

主定时器比较 2 事件会强制将输出设为其有效状态。

Bit 8 **MSTCMP1**: 主比较 1 (Master compare 1)

主定时器比较 1 事件会强制将输出设为其有效状态。

Bit 7 **MSTPER**: 主周期 (Master period)

主定时器计数器在连续模式下翻转或主定时器在单发模式下复位都会强制将输出设为其有效状态。

Bit 6 **CMP4**: 定时器 x 比较 4 (Timer x compare 4)

定时器 A 比较 4 事件会强制将输出设为其有效状态。

Bit 5 **CMP3**: 定时器 x 比较 3 (Timer x compare 3)

定时器 A 比较 3 事件会强制将输出设为其有效状态。

Bit 4 **CMP2**: 定时器 x 比较 2 (Timer x compare 2)

定时器 A 比较 2 事件会强制将输出设为其有效状态。

Bit 3 **CMP1**: 定时器 x 比较 1 (Timer x compare 1)

定时器 A 比较 1 事件会强制将输出设为其有效状态。



Bit 2 **PER**: 定时器 x 周期 (Timer x period)

定时器 A 周期事件会强制将输出设为其有效状态。

注: 在上下模式 (UDM 位设置为1) 中, 计数器周期事件根据 OUTROM[1:0] 位的设置来定义。

Bit 1 **RESYNC**: 定时器 A 重新同步 (Timer A resynchronization)

只有来自软件或 SYNC 输入的定时器 A 复位事件才会强制将输出设为其有效状态。

注: 当 RESYNC=1 时, 其他定时器复位不会影响输出。

Bit 0 **SST**: 软件设置触发器 (Software set trigger)

该位会强制将输出设为其有效状态。该位仅可通过软件置 1, 并由硬件复位。

注: 此位不是预加载的。

## 22.5.28 HRTIM 定时器 x 输出 1 复位寄存器 (HRTIM\_RSTx1R) (x = A~F)

偏移地址: 块 A: 0x0C0

偏移地址: 块 B: 0x140

偏移地址: 块 C: 0x1C0

偏移地址: 块 D: 0x240

偏移地址: 块 E: 0x2C0

偏移地址: 块 F: 0x340

复位值: 0x0000 0000

| 31           | 30            | 29           | 28           | 27           | 26           | 25           | 24           | 23           | 22           | 21           | 20           | 19           | 18           | 17           | 16           |
|--------------|---------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| UP<br>DATE   | EXT<br>EVNT10 | EXT<br>EVNT9 | EXT<br>EVNT8 | EXT<br>EVNT7 | EXT<br>EVNT6 | EXT<br>EVNT5 | EXT<br>EVNT4 | EXT<br>EVNT3 | EXT<br>EVNT2 | EXT<br>EVNT1 | TIM<br>EVNT9 | TIM<br>EVNT8 | TIM<br>EVNT7 | TIM<br>EVNT6 | TIM<br>EVNT5 |
| rw           | rw            | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           |
| 15           | 14            | 13           | 12           | 11           | 10           | 9            | 8            | 7            | 6            | 5            | 4            | 3            | 2            | 1            | 0            |
| TIM<br>EVNT4 | TIM<br>EVNT3  | TIM<br>EVNT2 | TIM<br>EVNT1 | MST<br>CMP4  | MST<br>CMP3  | MST<br>CMP2  | MST<br>CMP1  | MST<br>PER   | CMP4         | CMP3         | CMP2         | CMP1         | PER          | RESYN<br>C   | SRT          |
| rw           | rw            | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           |

Bit 31:0 请参考 HRTIM\_SETx1R 位的描述。

这些位定义了可强制将 Tx1 输出设为其无效状态的源。

## 22.5.29 HRTIM 定时器 x 输出 2 置位寄存器 (HRTIM\_SETx2R) (x = A~F)

偏移地址: 块 A: 0x0C4

偏移地址: 块 B: 0x144

偏移地址: 块 C: 0x1C4

偏移地址: 块 D: 0x244

偏移地址: 块 E: 0x2C4

偏移地址: 块 F: 0x344

复位值: 0x0000 0000

| 31           | 30            | 29           | 28           | 27           | 26           | 25           | 24           | 23           | 22           | 21           | 20           | 19           | 18           | 17           | 16           |
|--------------|---------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| UP<br>DATE   | EXT<br>EVNT10 | EXT<br>EVNT9 | EXT<br>EVNT8 | EXT<br>EVNT7 | EXT<br>EVNT6 | EXT<br>EVNT5 | EXT<br>EVNT4 | EXT<br>EVNT3 | EXT<br>EVNT2 | EXT<br>EVNT1 | TIM<br>EVNT9 | TIM<br>EVNT8 | TIM<br>EVNT7 | TIM<br>EVNT6 | TIM<br>EVNT5 |
| rw           | rw            | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           |
| 15           | 14            | 13           | 12           | 11           | 10           | 9            | 8            | 7            | 6            | 5            | 4            | 3            | 2            | 1            | 0            |
| TIM<br>EVNT4 | TIM<br>EVNT3  | TIM<br>EVNT2 | TIM<br>EVNT1 | MST<br>CMP4  | MST<br>CMP3  | MST<br>CMP2  | MST<br>CMP1  | MST<br>PER   | CMP4         | CMP3         | CMP2         | CMP1         | PER          | RESYN<br>C   | SST          |
| rw           | rw            | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           |

Bit 31:0 请参考 HRTIM\_SETx1R 位的描述。

这些位定义了可强制将 Tx2 输出设为其无效状态的源。

### 22.5.30 HRTIM 定时器 x 输出 2 复位寄存器 (HRTIM\_RSTx2R) (x = A~F)

偏移地址：块 A：0x0C8

偏移地址：块 B：0x148

偏移地址：块 C：0x1C8

偏移地址：块 D：0x248

偏移地址：块 E：0x2C8

偏移地址：块 F：0x348

复位值：0x0000 0000

| 31           | 30            | 29           | 28           | 27           | 26           | 25           | 24           | 23           | 22           | 21           | 20           | 19           | 18           | 17           | 16           |
|--------------|---------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| UP<br>DATE   | EXT<br>EVNT10 | EXT<br>EVNT9 | EXT<br>EVNT8 | EXT<br>EVNT7 | EXT<br>EVNT6 | EXT<br>EVNT5 | EXT<br>EVNT4 | EXT<br>EVNT3 | EXT<br>EVNT2 | EXT<br>EVNT1 | TIM<br>EVNT9 | TIM<br>EVNT8 | TIM<br>EVNT7 | TIM<br>EVNT6 | TIM<br>EVNT5 |
| rw           | rw            | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           |
| 15           | 14            | 13           | 12           | 11           | 10           | 9            | 8            | 7            | 6            | 5            | 4            | 3            | 2            | 1            | 0            |
| TIM<br>EVNT4 | TIM<br>EVNT3  | TIM<br>EVNT2 | TIM<br>EVNT1 | MST<br>CMP4  | MST<br>CMP3  | MST<br>CMP2  | MST<br>CMP1  | MST<br>PER   | CMP4         | CMP3         | CMP2         | CMP1         | PER          | RESYN<br>C   | SRT          |
| rw           | rw            | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           |

Bit 31:0 请参考 HRTIM\_SETx1R 位的描述。

这些位定义了可强制将 Tx2 输出设为其无效状态的源。

### 22.5.31 HRTIM 定时器 x 外部事件过滤寄存器 1 (HRTIM\_EEFxR1) (x = A~F)

偏移地址：块 A：0x0CC

偏移地址：块 B：0x14C

偏移地址：块 C：0x1CC

偏移地址：块 D：0x24C

偏移地址：块 E：0x2CC

偏移地址：块 F：0x34C

复位值：0x0000 0000

|              |    |    |              |     |              |    |             |     |              |     |              |    |             |     |             |
|--------------|----|----|--------------|-----|--------------|----|-------------|-----|--------------|-----|--------------|----|-------------|-----|-------------|
| 31           | 30 | 29 | 28           | 27  | 26           | 25 | 24          | 23  | 22           | 21  | 20           | 19 | 18          | 17  | 16          |
| Reserved     |    |    | EE5FLTR[3:0] |     |              |    | EE5LT<br>CH | Res | EE4FLTR[3:0] |     |              |    | EE4LT<br>CH | Res | EE3FLTR[3]  |
|              |    |    | rw           | rw  | rw           | rw | rw          |     | rw           | rw  | rw           | rw | rw          |     |             |
| 15           | 14 | 13 | 12           | 11  | 10           | 9  | 8           | 7   | 6            | 5   | 4            | 3  | 2           | 1   | 0           |
| EE3FLTR[2:0] |    |    | EE3LT<br>CH  | Res | EE2FLTR[3:0] |    |             |     | EE2LT<br>CH  | Res | EE1FLTR[3:0] |    |             |     | EE1LT<br>CH |
| rw           | rw | rw | rw           |     | rw           | rw | rw          | rw  | rw           |     | rw           | rw | rw          | rw  | rw          |

Bit 31:29 保留，必须保持复位值。

Bit 28:25 **EE5FLTR[3:0]**：外部事件 5 过滤器（External event 5 filter）

参考 EE1FLTR[3:0]的描述。

Bit 24 **EE5LTCH**：外部事件 5 锁存（External event 5 latch）

参考 EE1LTCH 的描述

Bit 23 保留，必须保持复位值。

Bit 22:19 **EE4FLTR[3:0]**：外部事件 4 过滤器（External event 4 filter）

参考 EE1FLTR[3:0]的描述。

Bit 18 **EE4LTCH**：外部事件 4 锁存（External event 4 latch）

参考 EE1LTCH 的描述

Bit 17 保留，必须保持复位值。

Bit 16:13 **EE3FLTR[3:0]**：外部事件 3 过滤器（External event 3 filter）

参考 EE1FLTR[3:0]的描述。

Bit 12 **EE3LTCH**：外部事件 3 锁存（External event 3 latch）

参考 EE1LTCH 的描述

Bit 11 保留，必须保持复位值。

Bit 10:7 **EE2FLTR[3:0]**：外部事件 2 过滤器（External event 2 filter）

参考 EE1FLTR[3:0]的描述。

Bit 6 **EE2LTCH**：外部事件 2 锁存（External event 2 latch）

参考 EE1LTCH 的描述。

Bit 5 保留，必须保持复位值。

Bit 4:1 EE1FLTR[3:0]: 外部事件 1 过滤器 (External event 1 filter)

0000: 无滤波

0001: 消隐持续时间从计数器复位/翻转到比较 1

0010: 向上计数模式 (UDM 位复位), 消隐持续时间从计数器复位/翻转到比较 2

上下计数模式 (UDM 位置位): 仅在向上计数阶段从比较 1 到比较 2 的消隐。

0011: 消隐持续时间从计数器复位/翻转到比较 3

0100: 向上计数模式 (UDM 位复位), 消隐持续时间从计数器复位/翻转到比较 4

上下计数模式 (UDM 位置位): 仅在向上计数阶段从比较 3 到比较 4 的消隐。

0101: 消隐与另一定时单元相关: TIMFLTR1 源

0110: 消隐与另一定时单元相关: TIMFLTR2 源

0111: 消隐与另一定时单元相关: TIMFLTR3 源

1000: 消隐与另一定时单元相关: TIMFLTR4 源

1001: 消隐与另一定时单元相关: TIMFLTR5 源

1010: 消隐与另一定时单元相关: TIMFLTR6 源

1011: 消隐与另一定时单元相关: TIMFLTR7 源

1100: 消隐与另一定时单元相关: TIMFLTR8 源

1101: 向上计数模式 (UDM 位复位), 窗口持续时间从计数器复位/翻转到比较 2

上下计数模式 (UDM 位置位): 窗口持续时间仅在向上计数阶段从比较 2 到比较 3

1110: 向上计数模式 (UDM 位复位), 窗口持续时间从计数器复位/翻转到比较 3

上下计数模式 (UDM 位置位): 窗口持续时间仅在向上计数阶段从比较 2 到比较 3

1111: 向上计数模式 (UDM 位复位), 窗口与另一定时单元相关: TIMWIN 源

上下计数模式 (UDM 位置位): 窗口持续时间从向上计数阶段的比较 2 到向下计数阶段的比较 3。

注: 每当使用比较寄存器进行滤波时, 其值必须大于 0。

一旦计数器被使能 (TxCEN 位设置), 则不应修改此位域。

Bit 0 EE1LTCH: 外部事件 1 锁存 (External event 1 latch)

0: 如果事件 1 在消隐期间发生, 会被忽略, 如果在窗口持续时间内发生, 则会通过。

1: 事件 1 会锁存并延迟, 直至消隐或窗口周期结束。

注: 如果 EE1LTCH = 0, 则会在窗口模式下生成超时事件 (EE1FLTR[3:0]=1101、1110、1111), 但外部事件在快速模式下编程的情况除外 (EEFAST = 1)。

一旦计数器被使能 (TxCEN 位置 1), 则不应修改此位域。

## 22.5.32 HRTIM 定时器 x 外部事件过滤寄存器 2 (HRTIM\_EEFxR2) (x = A~F)

偏移地址：块 A：0x0D0

偏移地址：块 B：0x150

偏移地址：块 C：0x1D0

偏移地址：块 D：0x250

偏移地址：块 E：0x2D0

偏移地址：块 F：0x350

复位值：0x0000 0000

|              |    |    |               |     |              |    |              |     |              |     |              |    |             |     |                |
|--------------|----|----|---------------|-----|--------------|----|--------------|-----|--------------|-----|--------------|----|-------------|-----|----------------|
| 31           | 30 | 29 | 28            | 27  | 26           | 25 | 24           | 23  | 22           | 21  | 20           | 19 | 18          | 17  | 16             |
| Reserved     |    |    | EE10FLTR[3:0] |     |              |    | EE10LT<br>CH | Res | EE9FLTR[3:0] |     |              |    | EE9LT<br>CH | Res | EE8FLT<br>R[3] |
|              |    |    | rw            | rw  | rw           | rw | rw           |     | rw           | rw  | rw           | rw | rw          |     | rw             |
| 15           | 14 | 13 | 12            | 11  | 10           | 9  | 8            | 7   | 6            | 5   | 4            | 3  | 2           | 1   | 0              |
| EE8FLTR[2:0] |    |    | EE8LT<br>CH   | Res | EE7FLTR[3:0] |    |              |     | EE7LT<br>CH  | Res | EE6FLTR[3:0] |    |             |     | EE6LT<br>CH    |
| rw           | rw | rw | rw            |     | rw           | rw | rw           | rw  | rw           |     | rw           | rw | rw          | rw  | rw             |

Bit 31:29 保留，必须保持复位值。

Bit 28:25 **EE10FLTR[3:0]**：外部事件 10 过滤器（External event 10 filter）

参考 EE1FLTR[3:0]的描述。

Bit 24 **EE10LTCH**：外部事件 10 锁存（External event 10 latch）

参考 EE1LTCH 的描述

Bit 23 保留，必须保持复位值。

Bit 22:19 **EE9FLTR[3:0]**：外部事件 9 过滤器（External event 9 filter）

参考 EE1FLTR[3:0]的描述。

Bit 18 **EE9LTCH**：外部事件 9 锁存（External event 9 latch）

参考 EE1LTCH 的描述

Bit 17 保留，必须保持复位值。

Bit 16:13 **EE8FLTR[3:0]**：外部事件 8 过滤器（External event 8 filter）

参考 EE1FLTR[3:0]的描述。

Bit 12 **EE8LTCH**：外部事件 8 锁存（External event 8 latch）

参考 EE1LTCH 的描述

Bit 11 保留，必须保持复位值。

Bit 10:7 **EE7FLTR[3:0]**：外部事件 7 过滤器（External event 7 filter）

参考 EE1FLTR[3:0]的描述。

Bit 6 **EE7LTCH**：外部事件 7 锁存（External event 7 latch）

参考 EE1LTCH 的描述。

Bit 5 保留，必须保持复位值。

Bit 4:1 **EE6FLTR[3:0]**: 外部事件 6 过滤器 (External event 6 filter)

参考 EE1FLTR[3:0]的描述。

Bit 0 **EE6LTCH**: 外部事件 6 锁存 (External event 6 latch)

参考 EE1LTCH 的描述。

### 22.5.33 HRTIM 定时器 A 复位寄存器 (HRTIM\_RSTAR)

偏移地址: 0x0D4

复位值: 0x0000 0000

| 31           | 30           | 29           | 28           | 27           | 26           | 25           | 24           | 23           | 22           | 21           | 20           | 19           | 18            | 17           | 16           |
|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|---------------|--------------|--------------|
| TIMF<br>CMP2 | TIME<br>CMP4 | TIME<br>CMP2 | TIME<br>CMP1 | TIMD<br>CMP4 | TIMD<br>CMP2 | TIMD<br>CMP1 | TIMC<br>CMP4 | TIMC<br>CMP2 | TIMC<br>CMP1 | TIMB<br>CMP4 | TIMB<br>CMP2 | TIMB<br>CMP1 | EXT<br>EVNT10 | EXT<br>EVNT9 | EXT<br>EVNT8 |
| rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw            | rw           | rw           |
| 15           | 14           | 13           | 12           | 11           | 10           | 9            | 8            | 7            | 6            | 5            | 4            | 3            | 2             | 1            | 0            |
| EXT<br>EVNT7 | EXT<br>EVNT6 | EXT<br>EVNT5 | EXT<br>EVNT4 | EXT<br>EVNT3 | EXT<br>EVNT2 | EXT<br>EVNT1 | MST<br>CMP4  | MST<br>CMP3  | MST<br>CMP2  | MST<br>CMP1  | MST<br>PER   | CMP4         | CMP2          | UPDT         | TIMF<br>CMP1 |
| rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw            | rw           | rw           |

Bit 31 **TIMFCMP2**: 定时器 F 比较 2 (Timer F compare 2)

当定时器 F 的比较 2 事件发生时，定时器 A 的计数器将被复位。

Bit 30 **TIMECPM4**: 定时器 E 比较 4 (Timer E compare 4)

当定时器 E 的比较 4 事件发生时，定时器 A 的计数器将被复位。

Bit 29 **TIMECMP2**: 定时器 E 比较 2 (Timer E compare 2)

当定时器 E 的比较 2 事件发生时，定时器 A 的计数器将被复位。

Bit 28 **TIMECMP1**: 定时器 E 比较 1 (Timer E compare 1)

当定时器 E 的比较 1 事件发生时，定时器 A 的计数器将被复位。

Bit 27 **TIMDCMP4**: 定时器 D 比较 4 (Timer D compare 4)

当定时器 D 的比较 4 事件发生时，定时器 A 的计数器将被复位。

Bit 26 **TIMDCMP2**: 定时器 D 比较 2 (Timer D compare 2)

当定时器 D 的比较 2 事件发生时，定时器 A 的计数器将被复位。

Bit 25 **TIMDCMP1**: 定时器 D 比较 1 (Timer D compare 1)

当定时器 D 的比较 1 事件发生时，定时器 A 的计数器将被复位。

Bit 24 **TIMCCMP4**: 定时器 C 比较 4 (Timer C compare 4)

当定时器 C 的比较 4 事件发生时，定时器 A 的计数器将被复位。

Bit 23 **TIMCCMP2**: 定时器 C 比较 2 (Timer C compare 2)

当定时器 C 的比较 2 事件发生时，定时器 A 的计数器将被复位。

Bit 22 **TIMCCMP1**: 定时器 C 比较 1 (Timer C compare 1)

当定时器 C 的比较 1 事件发生时，定时器 A 的计数器将被复位。

Bit 21 **TIMBCMP4**: 定时器 B 比较 4 (Timer B compare 4)

当定时器 B 的比较 4 事件发生时，定时器 A 的计数器将被复位。

Bit 20 **TIMBCMP2**: 定时器 B 比较 2 (Timer B compare 2)

当定时器 B 的比较 2 事件发生时，定时器 A 的计数器将被复位。



- Bit 19 **TIMBCMP1**: 定时器 B 比较 1 (Timer B compare 1)  
当定时器 B 的比较 1 事件发生时, 定时器 A 的计数器将被复位。
- Bit 18 **EXTEVNT10**: 外部事件 10 (External event)  
当外部事件 10 发生时, 定时器 A 的计数器将被复位。
- Bit 17 **EXTEVNT9**: 外部事件 9 (External event 9)  
当外部事件 9 发生时, 定时器 A 的计数器将被复位。
- Bit 16 **EXTEVNT8**: 外部事件 8 (External event 8)  
当外部事件 8 发生时, 定时器 A 的计数器将被复位。
- Bit 15 **EXTEVNT7**: 外部事件 7 (External event 7)  
当外部事件 7 发生时, 定时器 A 的计数器将被复位。
- Bit 14 **EXTEVNT6**: 外部事件 6 (External event 6)  
当外部事件 6 发生时, 定时器 A 的计数器将被复位。
- Bit 13 **EXTEVNT5**: 外部事件 5 (External event 5)  
当外部事件 5 发生时, 定时器 A 的计数器将被复位。
- Bit 12 **EXTEVNT4**: 外部事件 4 (External event 4)  
当外部事件 4 发生时, 定时器 A 的计数器将被复位。
- Bit 11 **EXTEVNT3**: 外部事件 3 (External event 3)  
当外部事件 3 发生时, 定时器 A 的计数器将被复位。
- Bit 10 **EXTEVNT2**: 外部事件 2 (External event 2)  
当外部事件 2 发生时, 定时器 A 的计数器将被复位。
- Bit 9 **EXTEVNT1**: 外部事件 1 (External event 1)  
当外部事件 1 发生时, 定时器 A 的计数器将被复位。
- Bit 8 **MSTCMP4**: 主定时器比较 4 (Master compare 4)  
当主定时器的比较 4 事件发生时, 定时器 A 的计数器将被复位。
- Bit 7 **MSTCMP3**: 主定时器比较 3 (Master compare 3)  
当主定时器的比较 3 事件发生时, 定时器 A 的计数器将被复位。
- Bit 6 **MSTCMP2**: 主定时器比较 2 (Master compare 2)  
当主定时器的比较 2 事件发生时, 定时器 A 的计数器将被复位。
- Bit 5 **MSTCMP1**: 主定时器比较 1 (Master compare 1)  
当主定时器的比较 1 事件发生时, 定时器 A 的计数器将被复位。
- Bit 4 **MSTPER**: 主定时器周期 (Master timer period)  
当主定时器的周期事件发生时, 定时器 A 的计数器将被复位。
- Bit 3 **CMP4**: 定时器 A 比较 4 复位 (Timer A compare 4 reset)  
当定时器 A 的比较 4 事件发生时, 定时器 A 的计数器将被复位。
- Bit 2 **CMP2**: 定时器 A 比较 2 复位 (Timer A compare 2 reset)  
当定时器 A 的比较 2 事件发生时, 定时器 A 的计数器将被复位。
- Bit 1 **UPDT**: 定时器 A 更新复位 (Timer A update reset)  
当更新事件发生时, 定时器 A 的计数器将被复位。
- Bit 0 **TIMFCMP1**: 定时器 F 比较 1 (Timer F compare 1)  
当定时器 F 的比较 1 事件发生时, 定时器 A 的计数器将被复位。

### 22.5.34 HRTIM 定时器 B 复位寄存器 (HRTIM\_RSTBR)

偏移地址: 0x154

复位值: 0x0000 0000

| 31           | 30           | 29           | 28           | 27           | 26           | 25           | 24           | 23           | 22           | 21           | 20           | 19           | 18            | 17           | 16           |
|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|---------------|--------------|--------------|
| TIMF<br>CMP2 | TIME<br>CMP4 | TIME<br>CMP2 | TIME<br>CMP1 | TIMD<br>CMP4 | TIMD<br>CMP2 | TIMD<br>CMP1 | TIMC<br>CMP4 | TIMC<br>CMP2 | TIMC<br>CMP1 | TIMA<br>CMP4 | TIMA<br>CMP2 | TIMA<br>CMP1 | EXT<br>EVNT10 | EXT<br>EVNT9 | EXT<br>EVNT8 |
| rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw            | rw           | rw           |
| 15           | 14           | 13           | 12           | 11           | 10           | 9            | 8            | 7            | 6            | 5            | 4            | 3            | 2             | 1            | 0            |
| EXT<br>EVNT7 | EXT<br>EVNT6 | EXT<br>EVNT5 | EXT<br>EVNT4 | EXT<br>EVNT3 | EXT<br>EVNT2 | EXT<br>EVNT1 | MST<br>CMP4  | MST<br>CMP3  | MST<br>CMP2  | MST<br>CMP1  | MST<br>PER   | CMP4         | CMP2          | UPDT         | TIMF<br>CMP1 |
| rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw            | rw           | rw           |

Bit 31 **TIMFCMP2**: 定时器 F 比较 2 (Timer F compare 2)

当定时器 F 的比较 2 事件发生时, 定时器 B 的计数器将被复位。

Bit 30 **TIMECPM4**: 定时器 E 比较 4 (Timer E compare 4)

当定时器 E 的比较 4 事件发生时, 定时器 B 的计数器将被复位。

Bit 29 **TIMECMP2**: 定时器 E 比较 2 (Timer E compare 2)

当定时器 E 的比较 2 事件发生时, 定时器 B 的计数器将被复位。

Bit 28 **TIMECMP1**: 定时器 E 比较 1 (Timer E compare 1)

当定时器 E 的比较 1 事件发生时, 定时器 B 的计数器将被复位。

Bit 27 **TIMDCMP4**: 定时器 D 比较 4 (Timer D compare 4)

当定时器 D 的比较 4 事件发生时, 定时器 B 的计数器将被复位。

Bit 26 **TIMDCMP2**: 定时器 D 比较 2 (Timer D compare 2)

当定时器 D 的比较 2 事件发生时, 定时器 B 的计数器将被复位。

Bit 25 **TIMDCMP1**: 定时器 D 比较 1 (Timer D compare 1)

当定时器 D 的比较 1 事件发生时, 定时器 B 的计数器将被复位。

Bit 24 **TIMCCMP4**: 定时器 C 比较 4 (Timer C compare 4)

当定时器 C 的比较 4 事件发生时, 定时器 B 的计数器将被复位。

Bit 23 **TIMCCMP2**: 定时器 C 比较 2 (Timer C compare 2)

当定时器 C 的比较 2 事件发生时, 定时器 B 的计数器将被复位。

Bit 22 **TIMCCMP1**: 定时器 C 比较 1 (Timer C compare 1)

当定时器 C 的比较 1 事件发生时, 定时器 B 的计数器将被复位。

Bit 21 **TIMACMP4**: 定时器 A 比较 4 (Timer A compare 4)

当定时器 A 的比较 4 事件发生时, 定时器 B 的计数器将被复位。

Bit 20 **TIMACMP2**: 定时器 A 比较 2 (Timer A compare 2)

当定时器 A 的比较 2 事件发生时, 定时器 B 的计数器将被复位。

Bit 19 **TIMACMP1**: 定时器 A 比较 1 (Timer A compare 1)

当定时器 A 的比较 1 事件发生时, 定时器 B 的计数器将被复位。

Bit 18 **EXTEVNT10**: 外部事件 10 (External event 10)

当外部事件 10 发生时, 定时器 B 的计数器将被复位。

Bit 17 **EXTEVNT9**: 外部事件 9 (External event 9)

当外部事件 9 发生时, 定时器 B 的计数器将被复位。

- Bit 16 **EXTEVNT8**: 外部事件 8 (External event 8)  
当外部事件 8 发生时, 定时器 B 的计数器将被复位。
- Bit 15 **EXTEVNT7**: 外部事件 7 (External event 7)  
当外部事件 7 发生时, 定时器 B 的计数器将被复位。
- Bit 14 **EXTEVNT6**: 外部事件 6 (External event 6)  
当外部事件 6 发生时, 定时器 B 的计数器将被复位。
- Bit 13 **EXTEVNT5**: 外部事件 5 (External event 5)  
当外部事件 5 发生时, 定时器 B 的计数器将被复位。
- Bit 12 **EXTEVNT4**: 外部事件 4 (External event 4)  
当外部事件 4 发生时, 定时器 B 的计数器将被复位。
- Bit 11 **EXTEVNT3**: 外部事件 3 (External event 3)  
当外部事件 3 发生时, 定时器 B 的计数器将被复位。
- Bit 10 **EXTEVNT2**: 外部事件 2 (External event 2)  
当外部事件 2 发生时, 定时器 B 的计数器将被复位。
- Bit 9 **EXTEVNT1**: 外部事件 1 (External event 1)  
当外部事件 1 发生时, 定时器 B 的计数器将被复位。
- Bit 8 **MSTCMP4**: 主定时器比较 4 (Master compare 4)  
当主定时器的比较 4 事件发生时, 定时器 B 的计数器将被复位。
- Bit 7 **MSTCMP3**: 主定时器比较 3 (Master compare 3)  
当主定时器的比较 3 事件发生时, 定时器 B 的计数器将被复位。
- Bit 6 **MSTCMP2**: 主定时器比较 2 (Master compare 2)  
当主定时器的比较 2 事件发生时, 定时器 B 的计数器将被复位。
- Bit 5 **MSTCMP1**: 主定时器比较 1 (Master compare 1)  
当主定时器的比较 1 事件发生时, 定时器 B 的计数器将被复位。
- Bit 4 **MSTPER**: 主定时器周期 (Master timer period)  
当主定时器的周期事件发生时, 定时器 B 的计数器将被复位。
- Bit 3 **CMP4**: 定时器 B 比较 4 复位 (Timer B compare 4 reset)  
当定时器 B 的比较 4 事件发生时, 定时器 B 的计数器将被复位。
- Bit 2 **CMP2**: 定时器 B 比较 2 复位 (Timer B compare 2 reset)  
当定时器 B 的比较 2 事件发生时, 定时器 B 的计数器将被复位。
- Bit 1 **UPDT**: 定时器 B 更新复位 (Timer B update reset)  
当更新事件发生时, 定时器 B 的计数器将被复位。
- Bit 0 **TIMFCMP1**: 定时器 F 比较 1 (Timer F compare 1)  
当定时器 F 的比较 1 事件发生时, 定时器 B 的计数器将被复位。

### 22.5.35 HRTIM 定时器 C 复位寄存器 (HRTIM\_RSTCR)

偏移地址: 0x2D4

复位值: 0x0000 0000

| 31           | 30           | 29           | 28           | 27           | 26           | 25           | 24           | 23           | 22           | 21           | 20           | 19           | 18            | 17           | 16           |
|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|---------------|--------------|--------------|
| TIMF<br>CMP2 | TIME<br>CMP4 | TIME<br>CMP2 | TIME<br>CMP1 | TIMD<br>CMP4 | TIMD<br>CMP2 | TIMD<br>CMP1 | TIMB<br>CMP4 | TIMB<br>CMP2 | TIMB<br>CMP1 | TIMA<br>CMP4 | TIMA<br>CMP2 | TIMA<br>CMP1 | EXT<br>EVNT10 | EXT<br>EVNT9 | EXT<br>EVNT8 |
| rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw            | rw           | rw           |
| 15           | 14           | 13           | 12           | 11           | 10           | 9            | 8            | 7            | 6            | 5            | 4            | 3            | 2             | 1            | 0            |
| EXT<br>EVNT7 | EXT<br>EVNT6 | EXT<br>EVNT5 | EXT<br>EVNT4 | EXT<br>EVNT3 | EXT<br>EVNT2 | EXT<br>EVNT1 | MST<br>CMP4  | MST<br>CMP3  | MST<br>CMP2  | MST<br>CMP1  | MST<br>PER   | CMP4         | CMP2          | UPDT         | TIMF<br>CMP1 |
| rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw            | rw           | rw           |

Bit 31 **TIMFCMP2**: 定时器 F 比较 2 (Timer F compare 2)

当定时器 F 的比较 2 事件发生时, 定时器 C 的计数器将被复位。

Bit 30 **TIMECPM4**: 定时器 E 比较 4 (Timer E compare 4)

当定时器 E 的比较 4 事件发生时, 定时器 C 的计数器将被复位。

Bit 29 **TIMECMP2**: 定时器 E 比较 2 (Timer E compare 2)

当定时器 E 的比较 2 事件发生时, 定时器 C 的计数器将被复位。

Bit 28 **TIMECMP1**: 定时器 E 比较 1 (Timer E compare 1)

当定时器 E 的比较 1 事件发生时, 定时器 C 的计数器将被复位。

Bit 27 **TIMDCMP4**: 定时器 D 比较 4 (Timer D compare 4)

当定时器 D 的比较 4 事件发生时, 定时器 C 的计数器将被复位。

Bit 26 **TIMDCMP2**: 定时器 D 比较 2 (Timer D compare 2)

当定时器 D 的比较 2 事件发生时, 定时器 C 的计数器将被复位。

Bit 25 **TIMDCMP1**: 定时器 D 比较 1 (Timer D compare 1)

当定时器 D 的比较 1 事件发生时, 定时器 C 的计数器将被复位。

Bit 24 **TIMBCMP4**: 定时器 B 比较 4 (Timer B compare 4)

当定时器 B 的比较 4 事件发生时, 定时器 C 的计数器将被复位。

Bit 23 **TIMBCMP2**: 定时器 B 比较 2 (Timer B compare 2)

当定时器 B 的比较 2 事件发生时, 定时器 C 的计数器将被复位。

Bit 22 **TIMBCMP1**: 定时器 B 比较 1 (Timer B compare 1)

当定时器 B 的比较 1 事件发生时, 定时器 C 的计数器将被复位。

Bit 21 **TIMACMP4**: 定时器 A 比较 4 (Timer A compare 4)

当定时器 A 的比较 4 事件发生时, 定时器 C 的计数器将被复位。

Bit 20 **TIMACMP2**: 定时器 A 比较 2 (Timer A compare 2)

当定时器 A 的比较 2 事件发生时, 定时器 C 的计数器将被复位。

Bit 19 **TIMACMP1**: 定时器 A 比较 1 (Timer A compare 1)

当定时器 A 的比较 1 事件发生时, 定时器 C 的计数器将被复位。

Bit 18 **EXTEVNT10**: 外部事件 10 (External event 10)

当外部事件 10 发生时, 定时器 C 的计数器将被复位。

Bit 17 **EXTEVNT9**: 外部事件 9 (External event 9)

当外部事件 9 发生时, 定时器 C 的计数器将被复位。

- Bit 16 **EXTEVNT8**: 外部事件 8 (External event 8)  
当外部事件 8 发生时, 定时器 C 的计数器将被复位。
- Bit 15 **EXTEVNT7**: 外部事件 7 (External event 7)  
当外部事件 7 发生时, 定时器 C 的计数器将被复位。
- Bit 14 **EXTEVNT6**: 外部事件 6 (External event 6)  
当外部事件 6 发生时, 定时器 C 的计数器将被复位。
- Bit 13 **EXTEVNT5**: 外部事件 5 (External event 5)  
当外部事件 5 发生时, 定时器 C 的计数器将被复位。
- Bit 12 **EXTEVNT4**: 外部事件 4 (External event 4)  
当外部事件 4 发生时, 定时器 C 的计数器将被复位。
- Bit 11 **EXTEVNT3**: 外部事件 3 (External event 3)  
当外部事件 3 发生时, 定时器 C 的计数器将被复位。
- Bit 10 **EXTEVNT2**: 外部事件 2 (External event 2)  
当外部事件 2 发生时, 定时器 C 的计数器将被复位。
- Bit 9 **EXTEVNT1**: 外部事件 1 (External event 1)  
当外部事件 1 发生时, 定时器 C 的计数器将被复位。
- Bit 8 **MSTCMP4**: 主定时器比较 4 (Master compare 4)  
当主定时器的比较 4 事件发生时, 定时器 C 的计数器将被复位。
- Bit 7 **MSTCMP3**: 主定时器比较 3 (Master compare 3)  
当主定时器的比较 3 事件发生时, 定时器 C 的计数器将被复位。
- Bit 6 **MSTCMP2**: 主定时器比较 2 (Master compare 2)  
当主定时器的比较 2 事件发生时, 定时器 C 的计数器将被复位。
- Bit 5 **MSTCMP1**: 主定时器比较 1 (Master compare 1)  
当主定时器的比较 1 事件发生时, 定时器 C 的计数器将被复位。
- Bit 4 **MSTPER**: 主定时器周期 (Master timer period)  
当主定时器的周期事件发生时, 定时器 C 的计数器将被复位。
- Bit 3 **CMP4**: 定时器 C 比较 4 复位 (Timer C compare 4 reset)  
当定时器 C 的比较 4 事件发生时, 定时器 C 的计数器将被复位。
- Bit 2 **CMP2**: 定时器 C 比较 2 复位 (Timer C compare 2 reset)  
当定时器 C 的比较 2 事件发生时, 定时器 C 的计数器将被复位。
- Bit 1 **UPDT**: 定时器 C 更新复位 (Timer C update reset)  
当更新事件发生时, 定时器 C 的计数器将被复位。
- Bit 0 **TIMFCMP1**: 定时器 F 比较 1 (Timer F compare 1)  
当定时器 F 的比较 1 事件发生时, 定时器 C 的计数器将被复位。



### 22.5.36 HRTIM 定时器 D 复位寄存器 (HRTIM\_RSTDR)

偏移地址: 0x254

复位值: 0x0000 0000

| 31           | 30           | 29           | 28           | 27           | 26           | 25           | 24           | 23           | 22           | 21           | 20           | 19           | 18            | 17           | 16           |
|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|---------------|--------------|--------------|
| TIMF<br>CMP2 | TIME<br>CMP4 | TIME<br>CMP2 | TIME<br>CMP1 | TIMC<br>CMP4 | TIMC<br>CMP2 | TIMC<br>CMP1 | TIMB<br>CMP4 | TIMB<br>CMP2 | TIMB<br>CMP1 | TIMA<br>CMP4 | TIMA<br>CMP2 | TIMA<br>CMP1 | EXT<br>EVNT10 | EXT<br>EVNT9 | EXT<br>EVNT8 |
| rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw            | rw           | rw           |
| 15           | 14           | 13           | 12           | 11           | 10           | 9            | 8            | 7            | 6            | 5            | 4            | 3            | 2             | 1            | 0            |
| EXT<br>EVNT7 | EXT<br>EVNT6 | EXT<br>EVNT5 | EXT<br>EVNT4 | EXT<br>EVNT3 | EXT<br>EVNT2 | EXT<br>EVNT1 | MST<br>CMP4  | MST<br>CMP3  | MST<br>CMP2  | MST<br>CMP1  | MST<br>PER   | CMP4         | CMP2          | UPDT         | TIMF<br>CMP1 |
| rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw            | rw           | rw           |

Bit 31 **TIMFCMP2**: 定时器 F 比较 2 (Timer F compare 2)

当定时器 F 的比较 2 事件发生时, 定时器 D 的计数器将被复位。

Bit 30 **TIMECPM4**: 定时器 E 比较 4 (Timer E compare 4)

当定时器 E 的比较 4 事件发生时, 定时器 D 的计数器将被复位。

Bit 29 **TIMECMP2**: 定时器 E 比较 2 (Timer E compare 2)

当定时器 E 的比较 2 事件发生时, 定时器 D 的计数器将被复位。

Bit 28 **TIMECMP1**: 定时器 E 比较 1 (Timer E compare 1)

当定时器 E 的比较 1 事件发生时, 定时器 D 的计数器将被复位。

Bit 27 **TIMCCMP4**: 定时器 C 比较 4 (Timer C compare 4)

当定时器 C 的比较 4 事件发生时, 定时器 D 的计数器将被复位。

Bit 26 **TIMCCMP2**: 定时器 C 比较 2 (Timer C compare 2)

当定时器 C 的比较 2 事件发生时, 定时器 D 的计数器将被复位。

Bit 25 **TIMCCMP1**: 定时器 C 比较 1 (Timer C compare 1)

当定时器 C 的比较 1 事件发生时, 定时器 D 的计数器将被复位。

Bit 24 **TIMBCMP4**: 定时器 B 比较 4 (Timer B compare 4)

当定时器 B 的比较 4 事件发生时, 定时器 D 的计数器将被复位。

Bit 23 **TIMBCMP2**: 定时器 B 比较 2 (Timer B compare 2)

当定时器 B 的比较 2 事件发生时, 定时器 D 的计数器将被复位。

Bit 22 **TIMBCMP1**: 定时器 B 比较 1 (Timer B compare 1)

当定时器 B 的比较 1 事件发生时, 定时器 D 的计数器将被复位。

Bit 21 **TIMACMP4**: 定时器 A 比较 4 (Timer A compare 4)

当定时器 A 的比较 4 事件发生时, 定时器 D 的计数器将被复位。

Bit 20 **TIMACMP2**: 定时器 A 比较 2 (Timer A compare 2)

当定时器 A 的比较 2 事件发生时, 定时器 D 的计数器将被复位。

Bit 19 **TIMACMP1**: 定时器 A 比较 1 (Timer A compare 1)

当定时器 A 的比较 1 事件发生时, 定时器 D 的计数器将被复位。

Bit 18 **EXTEVNT10**: 外部事件 10 (External event 10)

当外部事件 10 发生时, 定时器 D 的计数器将被复位。

Bit 17 **EXTEVNT9**: 外部事件 9 (External event 9)

当外部事件 9 发生时, 定时器 D 的计数器将被复位。

- Bit 16 **EXTEVNT8**: 外部事件 8 (External event 8)  
当外部事件 8 发生时, 定时器 D 的计数器将被复位。
- Bit 15 **EXTEVNT7**: 外部事件 7 (External event 7)  
当外部事件 7 发生时, 定时器 D 的计数器将被复位。
- Bit 14 **EXTEVNT6**: 外部事件 6 (External event 6)  
当外部事件 6 发生时, 定时器 D 的计数器将被复位。
- Bit 13 **EXTEVNT5**: 外部事件 5 (External event 5)  
当外部事件 5 发生时, 定时器 D 的计数器将被复位。
- Bit 12 **EXTEVNT4**: 外部事件 4 (External event 4)  
当外部事件 4 发生时, 定时器 D 的计数器将被复位。
- Bit 11 **EXTEVNT3**: 外部事件 3 (External event 3)  
当外部事件 3 发生时, 定时器 D 的计数器将被复位。
- Bit 10 **EXTEVNT2**: 外部事件 2 (External event 2)  
当外部事件 2 发生时, 定时器 D 的计数器将被复位。
- Bit 9 **EXTEVNT1**: 外部事件 1 (External event 1)  
当外部事件 1 发生时, 定时器 D 的计数器将被复位。
- Bit 8 **MSTCMP4**: 主定时器比较 4 (Master compare 4)  
当主定时器的比较 4 事件发生时, 定时器 D 的计数器将被复位。
- Bit 7 **MSTCMP3**: 主定时器比较 3 (Master compare 3)  
当主定时器的比较 3 事件发生时, 定时器 D 的计数器将被复位。
- Bit 6 **MSTCMP2**: 主定时器比较 2 (Master compare 2)  
当主定时器的比较 2 事件发生时, 定时器 D 的计数器将被复位。
- Bit 5 **MSTCMP1**: 主定时器比较 1 (Master compare 1)  
当主定时器的比较 1 事件发生时, 定时器 D 的计数器将被复位。
- Bit 4 **MSTPER**: 主定时器周期 (Master timer period)  
当主定时器的周期事件发生时, 定时器 D 的计数器将被复位。
- Bit 3 **CMP4**: 定时器 D 比较 4 复位 (Timer D compare 4 reset)  
当定时器 D 的比较 4 事件发生时, 定时器 D 的计数器将被复位。
- Bit 2 **CMP2**: 定时器 D 比较 2 复位 (Timer D compare 2 reset)  
当定时器 D 的比较 2 事件发生时, 定时器 D 的计数器将被复位。
- Bit 1 **UPDT**: 定时器 D 更新复位 (Timer D update reset)  
当更新事件发生时, 定时器 D 的计数器将被复位。
- Bit 0 **TIMFCMP1**: 定时器 F 比较 1 (Timer F compare 1)  
当定时器 F 的比较 1 事件发生时, 定时器 D 的计数器将被复位。



## 22.5.37 HRTIM 定时器 E 复位寄存器 (HRTIM\_RSTER)

偏移地址: 0x2D4

复位值: 0x0000 0000

| 31           | 30           | 29           | 28           | 27           | 26           | 25           | 24           | 23           | 22           | 21           | 20           | 19           | 18            | 17           | 16           |
|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|---------------|--------------|--------------|
| TIMF<br>CMP2 | TIMD<br>CMP4 | TIMD<br>CMP2 | TIMD<br>CMP1 | TIMC<br>CMP4 | TIMC<br>CMP2 | TIMC<br>CMP1 | TIMB<br>CMP4 | TIMB<br>CMP2 | TIMB<br>CMP1 | TIMA<br>CMP4 | TIMA<br>CMP2 | TIMA<br>CMP1 | EXT<br>EVNT10 | EXT<br>EVNT9 | EXT<br>EVNT8 |
| rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw            | rw           | rw           |
| 15           | 14           | 13           | 12           | 11           | 10           | 9            | 8            | 7            | 6            | 5            | 4            | 3            | 2             | 1            | 0            |
| EXT<br>EVNT7 | EXT<br>EVNT6 | EXT<br>EVNT5 | EXT<br>EVNT4 | EXT<br>EVNT3 | EXT<br>EVNT2 | EXT<br>EVNT1 | MST<br>CMP4  | MST<br>CMP3  | MST<br>CMP2  | MST<br>CMP1  | MST<br>PER   | CMP4         | CMP2          | UPDT         | TIMF<br>CMP1 |
| rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw            | rw           | rw           |

Bit 31 **TIMFCMP2**: 定时器 F 比较 2 (Timer F compare 2)

当定时器 F 的比较 2 事件发生时, 定时器 E 的计数器将被复位。

Bit 30 **TIMDCMP4**: 定时器 D 比较 4 (Timer D compare 4)

当定时器 D 的比较 4 事件发生时, 定时器 E 的计数器将被复位。

Bit 29 **TIMDCMP2**: 定时器 D 比较 2 (Timer D compare 2)

当定时器 D 的比较 2 事件发生时, 定时器 E 的计数器将被复位。

Bit 28 **TIMDCMP1**: 定时器 D 比较 1 (Timer D compare 1)

当定时器 D 的比较 1 事件发生时, 定时器 E 的计数器将被复位。

Bit 27 **TIMCCMP4**: 定时器 C 比较 4 (Timer C compare 4)

当定时器 C 的比较 4 事件发生时, 定时器 E 的计数器将被复位。

Bit 26 **TIMCCMP2**: 定时器 C 比较 2 (Timer C compare 2)

当定时器 C 的比较 2 事件发生时, 定时器 E 的计数器将被复位。

Bit 25 **TIMCCMP1**: 定时器 C 比较 1 (Timer C compare 1)

当定时器 C 的比较 1 事件发生时, 定时器 E 的计数器将被复位。

Bit 24 **TIMBCMP4**: 定时器 B 比较 4 (Timer B compare 4)

当定时器 B 的比较 4 事件发生时, 定时器 E 的计数器将被复位。

Bit 23 **TIMBCMP2**: 定时器 B 比较 2 (Timer B compare 2)

当定时器 B 的比较 2 事件发生时, 定时器 E 的计数器将被复位。

Bit 22 **TIMBCMP1**: 定时器 B 比较 1 (Timer B compare 1)

当定时器 B 的比较 1 事件发生时, 定时器 E 的计数器将被复位。

Bit 21 **TIMACMP4**: 定时器 A 比较 4 (Timer A compare 4)

当定时器 A 的比较 4 事件发生时, 定时器 E 的计数器将被复位。

Bit 20 **TIMACMP2**: 定时器 A 比较 2 (Timer A compare 2)

当定时器 A 的比较 2 事件发生时, 定时器 E 的计数器将被复位。

Bit 19 **TIMACMP1**: 定时器 A 比较 1 (Timer A compare 1)

当定时器 A 的比较 1 事件发生时, 定时器 E 的计数器将被复位。

Bit 18 **EXTEVNT10**: 外部事件 10 (External event 10)

当外部事件 10 发生时, 定时器 E 的计数器将被复位。

Bit 17 **EXTEVNT9**: 外部事件 9 (External event 9)

当外部事件 9 发生时, 定时器 E 的计数器将被复位。

- Bit 16 **EXTEVNT8**: 外部事件 8 (External event 8)  
当外部事件 8 发生时, 定时器 E 的计数器将被复位。
- Bit 15 **EXTEVNT7**: 外部事件 7 (External event 7)  
当外部事件 7 发生时, 定时器 E 的计数器将被复位。
- Bit 14 **EXTEVNT6**: 外部事件 6 (External event 6)  
当外部事件 6 发生时, 定时器 E 的计数器将被复位。
- Bit 13 **EXTEVNT5**: 外部事件 5 (External event 5)  
当外部事件 5 发生时, 定时器 E 的计数器将被复位。
- Bit 12 **EXTEVNT4**: 外部事件 4 (External event 4)  
当外部事件 4 发生时, 定时器 E 的计数器将被复位。
- Bit 11 **EXTEVNT3**: 外部事件 3 (External event 3)  
当外部事件 3 发生时, 定时器 E 的计数器将被复位。
- Bit 10 **EXTEVNT2**: 外部事件 2 (External event 2)  
当外部事件 2 发生时, 定时器 E 的计数器将被复位。
- Bit 9 **EXTEVNT1**: 外部事件 1 (External event 1)  
当外部事件 1 发生时, 定时器 E 的计数器将被复位。
- Bit 8 **MSTCMP4**: 主定时器比较 4 (Master compare 4)  
当主定时器的比较 4 事件发生时, 定时器 E 的计数器将被复位。
- Bit 7 **MSTCMP3**: 主定时器比较 3 (Master compare 3)  
当主定时器的比较 3 事件发生时, 定时器 E 的计数器将被复位。
- Bit 6 **MSTCMP2**: 主定时器比较 2 (Master compare 2)  
当主定时器的比较 2 事件发生时, 定时器 E 的计数器将被复位。
- Bit 5 **MSTCMP1**: 主定时器比较 1 (Master compare 1)  
当主定时器的比较 1 事件发生时, 定时器 E 的计数器将被复位。
- Bit 4 **MSTPER**: 主定时器周期 (Master timer period)  
当主定时器的周期事件发生时, 定时器 E 的计数器将被复位。
- Bit 3 **CMP4**: 定时器 E 比较 4 复位 (Timer E compare 4 reset)  
当定时器 E 的比较 4 事件发生时, 定时器 E 的计数器将被复位。
- Bit 2 **CMP2**: 定时器 E 比较 2 复位 (Timer E compare 2 reset)  
当定时器 E 的比较 2 事件发生时, 定时器 E 的计数器将被复位。
- Bit 1 **UPDT**: 定时器 E 更新复位 (Timer E update reset)  
当更新事件发生时, 定时器 E 的计数器将被复位。
- Bit 0 **TIMFCMP1**: 定时器 F 比较 1 (Timer F compare 1)  
当定时器 F 的比较 1 事件发生时, 定时器 E 的计数器将被复位。

## 22.5.38 HRTIM 定时器 F 复位寄存器 (HRTIM\_RSTFR)

偏移地址: 0x354

复位值: 0x0000 0000

| 31           | 30           | 29           | 28           | 27           | 26           | 25           | 24           | 23           | 22           | 21           | 20           | 19           | 18            | 17           | 16           |
|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|---------------|--------------|--------------|
| TIME<br>CMP2 | TIMD<br>CMP4 | TIMD<br>CMP2 | TIMD<br>CMP1 | TIMC<br>CMP4 | TIMC<br>CMP2 | TIMC<br>CMP1 | TIMB<br>CMP4 | TIMB<br>CMP2 | TIMB<br>CMP1 | TIMA<br>CMP4 | TIMA<br>CMP2 | TIMA<br>CMP1 | EXT<br>EVNT10 | EXT<br>EVNT9 | EXT<br>EVNT8 |
| rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw            | rw           | rw           |
| 15           | 14           | 13           | 12           | 11           | 10           | 9            | 8            | 7            | 6            | 5            | 4            | 3            | 2             | 1            | 0            |
| EXT<br>EVNT7 | EXT<br>EVNT6 | EXT<br>EVNT5 | EXT<br>EVNT4 | EXT<br>EVNT3 | EXT<br>EVNT2 | EXT<br>EVNT1 | MST<br>CMP4  | MST<br>CMP3  | MST<br>CMP2  | MST<br>CMP1  | MST<br>PER   | CMP4         | CMP2          | UPDT         | TIME<br>CMP1 |
| rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw            | rw           | rw           |

Bit 31 **TIMECPM2**: 定时器 E 比较 2 (Timer E compare 2)

当定时器 F 的比较 2 事件发生时, 定时器 E 的计数器将被复位。

Bit 30 **TIMDCPM4**: 定时器 D 比较 4 (Timer D compare 4)

当定时器 D 的比较 4 事件发生时, 定时器 F 的计数器将被复位。

Bit 29 **TIMDCMP2**: 定时器 D 比较 2 (Timer D compare 2)

当定时器 D 的比较 2 事件发生时, 定时器 F 的计数器将被复位。

Bit 28 **TIMDCMP1**: 定时器 D 比较 1 (Timer D compare 1)

当定时器 D 的比较 1 事件发生时, 定时器 F 的计数器将被复位。

Bit 27 **TIMCCMP4**: 定时器 C 比较 4 (Timer C compare 4)

当定时器 C 的比较 4 事件发生时, 定时器 F 的计数器将被复位。

Bit 26 **TIMCCMP2**: 定时器 C 比较 2 (Timer C compare 2)

当定时器 C 的比较 2 事件发生时, 定时器 F 的计数器将被复位。

Bit 25 **TIMCCMP1**: 定时器 C 比较 1 (Timer C compare 1)

当定时器 C 的比较 1 事件发生时, 定时器 F 的计数器将被复位。

Bit 24 **TIMBCMP4**: 定时器 B 比较 4 (Timer B compare 4)

当定时器 B 的比较 4 事件发生时, 定时器 F 的计数器将被复位。

Bit 23 **TIMBCMP2**: 定时器 B 比较 2 (Timer B compare 2)

当定时器 B 的比较 2 事件发生时, 定时器 F 的计数器将被复位。

Bit 22 **TIMBCMP1**: 定时器 B 比较 1 (Timer B compare 1)

当定时器 B 的比较 1 事件发生时, 定时器 F 的计数器将被复位。

Bit 21 **TIMACMP4**: 定时器 A 比较 4 (Timer A compare 4)

当定时器 A 的比较 4 事件发生时, 定时器 F 的计数器将被复位。

Bit 20 **TIMACMP2**: 定时器 A 比较 2 (Timer A compare 2)

当定时器 A 的比较 2 事件发生时, 定时器 F 的计数器将被复位。

Bit 19 **TIMACMP1**: 定时器 A 比较 1 (Timer A compare 1)

当定时器 A 的比较 1 事件发生时, 定时器 F 的计数器将被复位。

Bit 18 **EXTEVNT10**: 外部事件 10 (External event 10)

当外部事件 10 发生时, 定时器 F 的计数器将被复位。

Bit 17 **EXTEVNT9**: 外部事件 9 (External event 9)

当外部事件 9 发生时, 定时器 F 的计数器将被复位。

- Bit 16 **EXTEVNT8**: 外部事件 8 (External event 8)  
当外部事件 8 发生时, 定时器 F 的计数器将被复位。
- Bit 15 **EXTEVNT7**: 外部事件 7 (External event 7)  
当外部事件 7 发生时, 定时器 F 的计数器将被复位。
- Bit 14 **EXTEVNT6**: 外部事件 6 (External event 6)  
当外部事件 6 发生时, 定时器 F 的计数器将被复位。
- Bit 13 **EXTEVNT5**: 外部事件 5 (External event 5)  
当外部事件 5 发生时, 定时器 F 的计数器将被复位。
- Bit 12 **EXTEVNT4**: 外部事件 4 (External event 4)  
当外部事件 4 发生时, 定时器 F 的计数器将被复位。
- Bit 11 **EXTEVNT3**: 外部事件 3 (External event 3)  
当外部事件 3 发生时, 定时器 F 的计数器将被复位。
- Bit 10 **EXTEVNT2**: 外部事件 2 (External event 2)  
当外部事件 2 发生时, 定时器 F 的计数器将被复位。
- Bit 9 **EXTEVNT1**: 外部事件 1 (External event 1)  
当外部事件 1 发生时, 定时器 F 的计数器将被复位。
- Bit 8 **MSTCMP4**: 主定时器比较 4 (Master compare 4)  
当主定时器的比较 4 事件发生时, 定时器 F 的计数器将被复位。
- Bit 7 **MSTCMP3**: 主定时器比较 3 (Master compare 3)  
当主定时器的比较 3 事件发生时, 定时器 F 的计数器将被复位。
- Bit 6 **MSTCMP2**: 主定时器比较 2 (Master compare 2)  
当主定时器的比较 2 事件发生时, 定时器 F 的计数器将被复位。
- Bit 5 **MSTCMP1**: 主定时器比较 1 (Master compare 1)  
当主定时器的比较 1 事件发生时, 定时器 F 的计数器将被复位。
- Bit 4 **MSTPER**: 主定时器周期 (Master timer period)  
当主定时器的周期事件发生时, 定时器 F 的计数器将被复位。
- Bit 3 **CMP4**: 定时器 F 比较 4 复位 (Timer F compare 4 reset)  
当定时器 F 的比较 4 事件发生时, 定时器 F 的计数器将被复位。
- Bit 2 **CMP2**: 定时器 F 比较 2 复位 (Timer F compare 2 reset)  
当定时器 F 的比较 2 事件发生时, 定时器 F 的计数器将被复位。
- Bit 1 **UPDT**: 定时器 F 更新复位 (Timer F update reset)  
当更新事件发生时, 定时器 F 的计数器将被复位。
- Bit 0 **TIMECMP1**: 定时器 E 比较 1 (Timer E compare 1)  
当定时器 E 的比较 1 事件发生时, 定时器 F 的计数器将被复位。

### 22.5.39 HRTIM 定时器 x 斩波寄存器 (HRTIM\_CHPxR) (x = A~F)

偏移地址：块 A：0x0D8

偏移地址：块 B：0x158

偏移地址：块 C：0x1D8

偏移地址：块 D：0x258

偏移地址：块 E：0x2D8

偏移地址：块 F：0x358

复位值：0x0000 0000

|          |    |    |    |    |            |    |    |    |             |    |    |             |    |    |    |
|----------|----|----|----|----|------------|----|----|----|-------------|----|----|-------------|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26         | 25 | 24 | 23 | 22          | 21 | 20 | 19          | 18 | 17 | 16 |
| Reserved |    |    |    |    |            |    |    |    |             |    |    |             |    |    |    |
| 15       | 14 | 13 | 12 | 11 | 10         | 9  | 8  | 7  | 6           | 5  | 4  | 3           | 2  | 1  | 0  |
| Reserved |    |    |    |    | STRPW[3:0] |    |    |    | CARDTY[2:0] |    |    | CARFRQ[3:0] |    |    |    |
|          |    |    |    |    | rw         | rw | rw | rw | rw          | rw | rw | rw          | rw | rw | rw |

Bit 31:11 保留，必须保持复位值。

Bit 10:7 **STRPW[3:0]**：定时器 x 启动脉冲宽度 (Timer x start pulsewidth)

该寄存器定义输出信号上升沿后的初始脉宽。

如果其中一个 CHPx 位置 1，此位域不能修改。

$$t_{\text{STRPW}} = (\text{STRPW}[3:0] + 1) \times 16 \times t_{\text{HRTIM}}$$

0000: 83.3 ns (1/12 MHz)

...

1111: 1.33 μs (16/12 MHz)

Bit 6:4 **CARDTY[2:0]**：定时器 x 斩波占空比值 (Timer x chopper duty cycle value)

该寄存器定义载波信号的占空比。如果其中一个 CHPx 位置 1，此位域不能修改。

000: 0/8 (即，只会出现第一个脉冲)

...

111: 7/8

Bit 3:0 **CARFRQ[3:0]**：定时器 x 载波频率值 (Timer x carrier frequency value)

此寄存器定义了载波频率  $F_{\text{CHPFRQ}} = f_{\text{HRTIM}} / (16 \times (\text{CARFRQ}[3:0] + 1))$ 。

如果其中一个 CHPx 位置 1，此位域不能修改。

0000: 12 MHz ( $f_{\text{HRTIM}} / 16$ )

...

1111: 750 KHz ( $f_{\text{HRTIM}} / 256$ )

## 22.5.40 HRTIM 定时器 A 捕获 1 控制寄存器 (HRTIM\_CPT1ACR)

偏移地址: 0x0DC

复位值: 0x0000 0000

| 31   | 30   | 29  | 28  | 27    | 26    | 25    | 24    | 23    | 22    | 21    | 20    | 19    | 18    | 17  | 16  |
|------|------|-----|-----|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-----|-----|
| TE   | TE   | TE1 | TE1 | TD    | TD    | TD1   | TD1   | TC    | TC    | TC1   | TC1   | TB    | TB    | TB1 | TB1 |
| CMP2 | CMP1 | RST | SET | CMP2  | CMP1  | RST   | SET   | CMP2  | CMP1  | RST   | SET   | CMP2  | CMP1  | RST | SET |
| rw   | rw   | rw  | rw  | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw  | rw  |
| 15   | 14   | 13  | 12  | 11    | 10    | 9     | 8     | 7     | 6     | 5     | 4     | 3     | 2     | 1   | 0   |
| TF   | TF   | TF1 | TF1 | EXEV1 | EXEV9 | EXEV8 | EXEV7 | EXEV6 | EXEV5 | EXEV4 | EXEV3 | EXEV2 | EXEV1 | UPD | SW  |
| CMP2 | CMP1 | RST | SET | OCPT  | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT | CPT |
| rw   | rw   | rw  | rw  | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw  | rw  |

Bit 31:0 参见 HRTIM\_CPT2xCR 位描述

## 22.5.41 HRTIM 定时器 B 捕获 1 控制寄存器 (HRTIM\_CPT1BCR)

偏移地址: 0x15C

复位值: 0x0000 0000

| 31   | 30   | 29  | 28  | 27    | 26    | 25    | 24    | 23    | 22    | 21    | 20    | 19    | 18    | 17  | 16  |
|------|------|-----|-----|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-----|-----|
| TE   | TE   | TE1 | TE1 | TD    | TD    | TD1   | TD1   | TC    | TC    | TC1   | TC1   | TF    | TF    | TF1 | TF1 |
| CMP2 | CMP1 | RST | SET | CMP2  | CMP1  | RST   | SET   | CMP2  | CMP1  | RST   | SET   | CMP2  | CMP1  | RST | SET |
| rw   | rw   | rw  | rw  | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw  | rw  |
| 15   | 14   | 13  | 12  | 11    | 10    | 9     | 8     | 7     | 6     | 5     | 4     | 3     | 2     | 1   | 0   |
| TA   | TA   | TA1 | TA1 | EXEV1 | EXEV9 | EXEV8 | EXEV7 | EXEV6 | EXEV5 | EXEV4 | EXEV3 | EXEV2 | EXEV1 | UPD | SW  |
| CMP2 | CMP1 | RST | SET | OCPT  | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT | CPT |
| rw   | rw   | rw  | rw  | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw  | rw  |

Bit 31:0 参见 HRTIM\_CPT2xCR 位描述

## 22.5.42 HRTIM 定时器 C 捕获 1 控制寄存器 (HRTIM\_CPT1CCR)

偏移地址: 0x1DC

复位值: 0x0000 0000

| 31   | 30   | 29  | 28  | 27    | 26    | 25    | 24    | 23    | 22    | 21    | 20    | 19    | 18    | 17  | 16  |
|------|------|-----|-----|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-----|-----|
| TE   | TE   | TE1 | TE1 | TD    | TD    | TD1   | TD1   | TF    | TF    | TF1   | TF1   | TB    | TB    | TB1 | TB1 |
| CMP2 | CMP1 | RST | SET | CMP2  | CMP1  | RST   | SET   | CMP2  | CMP1  | RST   | SET   | CMP2  | CMP1  | RST | SET |
| rw   | rw   | rw  | rw  | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw  | rw  |
| 15   | 14   | 13  | 12  | 11    | 10    | 9     | 8     | 7     | 6     | 5     | 4     | 3     | 2     | 1   | 0   |
| TA   | TA   | TA1 | TA1 | EXEV1 | EXEV9 | EXEV8 | EXEV7 | EXEV6 | EXEV5 | EXEV4 | EXEV3 | EXEV2 | EXEV1 | UPD | SW  |
| CMP2 | CMP1 | RST | SET | OCPT  | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT | CPT |
| rw   | rw   | rw  | rw  | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw  | rw  |

Bit 31:0 参见 HRTIM\_CPT2xCR 位描述



### 22.5.43 HRTIM 定时器 D 捕获 1 控制寄存器 (HRTIM\_CPT1DCR)

偏移地址: 0x25C

复位值: 0x0000 0000

| 31   | 30   | 29  | 28  | 27    | 26    | 25    | 24    | 23    | 22    | 21    | 20    | 19    | 18    | 17  | 16  |
|------|------|-----|-----|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-----|-----|
| TE   | TE   | TE1 | TE1 | TF    | TF    | TF1   | TF1   | TC    | TC    | TC1   | TC1   | TB    | TB    | TB1 | TB1 |
| CMP2 | CMP1 | RST | SET | CMP2  | CMP1  | RST   | SET   | CMP2  | CMP1  | RST   | SET   | CMP2  | CMP1  | RST | SET |
| rw   | rw   | rw  | rw  | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw  | rw  |
| 15   | 14   | 13  | 12  | 11    | 10    | 9     | 8     | 7     | 6     | 5     | 4     | 3     | 2     | 1   | 0   |
| TA   | TA   | TA1 | TA1 | EXEV1 | EXEV9 | EXEV8 | EXEV7 | EXEV6 | EXEV5 | EXEV4 | EXEV3 | EXEV2 | EXEV1 | UPD | SW  |
| CMP2 | CMP1 | RST | SET | OCPT  | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT | CPT |
| rw   | rw   | rw  | rw  | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw  | rw  |

Bit 31:0 参见 HRTIM\_CPT2xCR 位描述

### 22.5.44 HRTIM 定时器 E 捕获 1 控制寄存器 (HRTIM\_CPT1ECR)

偏移地址: 0x2DC

复位值: 0x0000 0000

| 31   | 30   | 29  | 28  | 27    | 26    | 25    | 24    | 23    | 22    | 21    | 20    | 19    | 18    | 17  | 16  |
|------|------|-----|-----|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-----|-----|
| TF   | TF   | TF1 | TF1 | TD    | TD    | TD1   | TD1   | TC    | TC    | TC1   | TC1   | TB    | TB    | TB1 | TB1 |
| CMP2 | CMP1 | RST | SET | CMP2  | CMP1  | RST   | SET   | CMP2  | CMP1  | RST   | SET   | CMP2  | CMP1  | RST | SET |
| rw   | rw   | rw  | rw  | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw  | rw  |
| 15   | 14   | 13  | 12  | 11    | 10    | 9     | 8     | 7     | 6     | 5     | 4     | 3     | 2     | 1   | 0   |
| TA   | TA   | TA1 | TA1 | EXEV1 | EXEV9 | EXEV8 | EXEV7 | EXEV6 | EXEV5 | EXEV4 | EXEV3 | EXEV2 | EXEV1 | UPD | SW  |
| CMP2 | CMP1 | RST | SET | OCPT  | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT | CPT |
| rw   | rw   | rw  | rw  | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw  | rw  |

Bit 31:0 参见 HRTIM\_CPT2xCR 位描述

### 22.5.45 HRTIM 定时器 F 捕获 1 控制寄存器 (HRTIM\_CPT1FCR)

偏移地址: 0x35C

复位值: 0x0000 0000

| 31   | 30   | 29  | 28  | 27    | 26    | 25    | 24    | 23    | 22    | 21    | 20    | 19    | 18    | 17  | 16  |
|------|------|-----|-----|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-----|-----|
| TE   | TE   | TE1 | TE1 | TD    | TD    | TD1   | TD1   | TC    | TC    | TC1   | TC1   | TB    | TB    | TB1 | TB1 |
| CMP2 | CMP1 | RST | SET | CMP2  | CMP1  | RST   | SET   | CMP2  | CMP1  | RST   | SET   | CMP2  | CMP1  | RST | SET |
| rw   | rw   | rw  | rw  | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw  | rw  |
| 15   | 14   | 13  | 12  | 11    | 10    | 9     | 8     | 7     | 6     | 5     | 4     | 3     | 2     | 1   | 0   |
| TA   | TA   | TA1 | TA1 | EXEV1 | EXEV9 | EXEV8 | EXEV7 | EXEV6 | EXEV5 | EXEV4 | EXEV3 | EXEV2 | EXEV1 | UPD | SW  |
| CMP2 | CMP1 | RST | SET | OCPT  | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT   | CPT | CPT |
| rw   | rw   | rw  | rw  | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw  | rw  |

Bit 31:0 参见 HRTIM\_CPT2xCR 位描述



## 22.5.46 HRTIM 定时器 x 捕获 2 控制寄存器 (HRTIM\_CPT2xCR) (x = A~F)

偏移地址：块 A：0x0E0

偏移地址：块 B：0x160

偏移地址：块 C：0x1E0

偏移地址：块 D：0x260

偏移地址：块 E：0x2E0

偏移地址：块 F：0x360

复位值：0x0000 0000

| 31         | 30         | 29         | 28         | 27            | 26           | 25           | 24           | 23           | 22           | 21           | 20           | 19           | 18           | 17         | 16         |
|------------|------------|------------|------------|---------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|------------|------------|
| TF<br>CMP2 | TF<br>CMP1 | TF1<br>RST | TF1<br>SET | TF<br>CMP2    | TF<br>CMP1   | TF1<br>RST   | TF1<br>SET   | TF<br>CMP2   | TF<br>CMP1   | TF1<br>RST   | TF1<br>SET   | TF<br>CMP2   | TF<br>CMP1   | TF1<br>RST | TF1<br>SET |
| /          | /          | /          | /          | /             | /            | /            | /            | /            | /            | /            | /            | /            | /            | /          | /          |
| TE<br>CMP2 | TE<br>CMP1 | TE1<br>RST | TE1<br>SET | TD<br>CMP2    | TD<br>CMP1   | TD1<br>RST   | TD1<br>SET   | TC<br>CMP2   | TC<br>CMP1   | TC1<br>RST   | TC1<br>SET   | TB<br>CMP2   | TB<br>CMP1   | TB1<br>RST | TB1<br>SET |
| rw         | rw         | rw         | rw         | rw            | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw         | rw         |
| 15         | 14         | 13         | 12         | 11            | 10           | 9            | 8            | 7            | 6            | 5            | 4            | 3            | 2            | 1          | 0          |
| TF<br>CMP2 | TF<br>CMP1 | TF1<br>RST | TF1<br>SET | EXEV1<br>0CPT | EXEV9<br>CPT | EXEV8<br>CPT | EXEV7<br>CPT | EXEV6<br>CPT | EXEV5<br>CPT | EXEV4<br>CPT | EXEV3<br>CPT | EXEV2<br>CPT | EXEV1<br>CPT | UPD<br>CPT | SW<br>CPT  |
| /          | /          | /          | /          |               |              |              |              |              |              |              |              |              |              |            |            |
| TA<br>CMP2 | TA<br>CMP1 | TA1<br>RST | TA1<br>SET |               |              |              |              |              |              |              |              |              |              |            |            |
| rw         | rw         | rw         | rw         | rw            | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw           | rw         | rw         |

Bit 31: 在 HRTIM\_CPT2ACR、HRTIM\_CPT2BCR、HRTIM\_CPT2CCR、HRTIM\_CPT2DCR、HRTIM\_CPT2FCR 中:

**TECMP2:** 定时器 E 比较 2 (Timer E compare 2)

参考 TACMP2 描述。

在 HRTIM\_CPT2ECR 中:

**TFCMP2:** 定时器 F 比较 2 (Timer F compare 2)

参考 TACMP2 描述。

Bit 30: 在 HRTIM\_CPT2ACR、HRTIM\_CPT2BCR、HRTIM\_CPT2CCR、HRTIM\_CPT2DCR、HRTIM\_CPT2FCR 中:

**TECMP1:** 定时器 E 比较 1 (Timer E compare 1)

参考 TACMP1 描述。

在 HRTIM\_CPT2ECR 中:

**TFCMP1:** 定时器 F 比较 1 (Timer F compare 1)

参考 TACMP1 描述。

Bit 29: 在 HRTIM\_CPT2ACR、HRTIM\_CPT2BCR、HRTIM\_CPT2CCR、HRTIM\_CPT2DCR、HRTIM\_CPT2FCR 中:

**TE1RST:** 定时器 E 输出 1 复位 (Timer E output 1 reset)

参考 TA1RST 描述。

在 HRTIM\_CPT2ECR 中:

**TF1RST:** 定时器 F 输出 1 复位 (Timer F output 1 reset)

参考 TA1RST 描述。

Bit 28: 在 HRTIM\_CPT2ACR、HRTIM\_CPT2BCR、HRTIM\_CPT2CCR、HRTIM\_CPT2DCR、HRTIM\_CPT2FCR 中:

**TE1SET:** 定时器 E 输出 1 置位 (Timer E output 1 set)

参考 TA1SET 描述。

在 HRTIM\_CPT2ECR 中:

**TF1SET:** 定时器 F 输出 1 置位 (Timer F output 1 set)

参考 TA1SET 描述。

Bit 27: 在 HRTIM\_CPT2ACR、HRTIM\_CPT2BCR、HRTIM\_CPT2CCR、HRTIM\_CPT2ECR、HRTIM\_CPT2FCR 中:

**TDCMP2:** 定时器 D 比较 2 (Timer D compare 2)

参考 TACMP2 描述。

在 HRTIM\_CPT2DCR 中:

**TFCMP2:** 定时器 F 比较 2 (Timer F compare 2)

参考 TACMP2 描述。

Bit 26: 在 HRTIM\_CPT2ACR、HRTIM\_CPT2BCR、HRTIM\_CPT2CCR、HRTIM\_CPT2ECR、HRTIM\_CPT2FCR 中:

**TDCMP1:** 定时器 D 比较 1 (Timer D compare 1)

参考 TACMP1 描述。

在 HRTIM\_CPT2DCR 中:

**TFCMP1:** 定时器 F 比较 1 (Timer F compare 1)

参考 TACMP1 描述。

Bit 25: 在 HRTIM\_CPT2ACR、HRTIM\_CPT2BCR、HRTIM\_CPT2CCR、HRTIM\_CPT2ECR、HRTIM\_CPT2FCR 中:

**TD1RST:** 定时器 D 输出 1 复位 (Timer D output 1 reset)

参考 TA1RST 描述。

在 HRTIM\_CPT2DCR 中:

**TF1RST:** 定时器 F 输出 1 复位 (Timer F output 1 reset)

参考 TA1RST 描述。

Bit 24: 在 HRTIM\_CPT2ACR、HRTIM\_CPT2BCR、HRTIM\_CPT2CCR、HRTIM\_CPT2ECR、HRTIM\_CPT2FCR 中:

**TD1SET:** 定时器 D 输出 1 置位 (Timer D output 1 set)

参考 TA1SET 描述。

在 HRTIM\_CPT2DCR 中:

**TF1SET:** 定时器 F 输出 1 置位 (Timer F output 1 set)

参考 TA1SET 描述。

Bit 23: 在 HRTIM\_CPT2ACR、HRTIM\_CPT2BCR、HRTIM\_CPT2DCR、HRTIM\_CPT2ECR、HRTIM\_CPT2FCR 中:

**TCCMP2:** 定时器 C 比较 2 (Timer C compare 2)

参考 TACMP2 描述。

在 HRTIM\_CPT2CCR 中:

**TFCMP2:** 定时器 F 比较 2 (Timer F compare 2)

参考 TACMP2 描述。

Bit 22: 在 HRTIM\_CPT2ACR、HRTIM\_CPT2BCR、HRTIM\_CPT2DCR、HRTIM\_CPT2ECR、HRTIM\_CPT2FCR 中:

**TCCMP1:** 定时器 C 比较 1 (Timer C compare 1)

参考 TACMP1 描述。

在 HRTIM\_CPT2CCR 中:

**TFCMP1:** 定时器 F 比较 1 (Timer F compare 1)

参考 TACMP1 描述。

Bit 21: 在 HRTIM\_CPT2ACR、HRTIM\_CPT2BCR、HRTIM\_CPT2DCR、HRTIM\_CPT2ECR、HRTIM\_CPT2FCR 中:

**TC1RST**: 定时器 C 输出 1 复位 (Timer C output 1 reset)  
参考 TA1RST 描述。

在 HRTIM\_CPT2CCR 中:

**TF1RST**: 定时器 F 输出 1 复位 (Timer F output 1 reset)  
参考 TA1RST 描述。

Bit 20: 在 HRTIM\_CPT2ACR、HRTIM\_CPT2BCR、HRTIM\_CPT2DCR、HRTIM\_CPT2ECR、HRTIM\_CPT2FCR 中:

**TC1SET**: 定时器 C 输出 1 置位 (Timer C output 1 set)  
参考 TA1SET 描述。

在 HRTIM\_CPT2CCR 中:

**TF1SET**: 定时器 F 输出 1 置位 (Timer F output 1 set)  
参考 TA1SET 描述。

Bit 19: 在 HRTIM\_CPT2ACR、HRTIM\_CPT2CCR、HRTIM\_CPT2DCR、HRTIM\_CPT2ECR、HRTIM\_CPT2FCR 中:

**TBCMP2**: 定时器 B 比较 2 (Timer B compare 2)  
参考 TACMP2 描述。

在 HRTIM\_CPT2BCR 中:

**TFCMP2**: 定时器 F 比较 2 (Timer F compare 2)  
参考 TACMP2 描述。

Bit 18: 在 HRTIM\_CPT2ACR、HRTIM\_CPT2CCR、HRTIM\_CPT2DCR、HRTIM\_CPT2ECR、HRTIM\_CPT2FCR 中:

**TBCMP1**: 定时器 B 比较 1 (Timer B compare 1)  
参考 TACMP1 描述。

在 HRTIM\_CPT2BCR 中:

**TFCMP1**: 定时器 F 比较 1 (Timer F compare 1)  
参考 TACMP1 描述。

Bit 17: 在 HRTIM\_CPT2ACR、HRTIM\_CPT2CCR、HRTIM\_CPT2DCR、HRTIM\_CPT2ECR、HRTIM\_CPT2FCR 中:

**TB1RST**: 定时器 B 输出 1 复位 (Timer B output 1 reset)  
参考 TA1RST 描述。

在 HRTIM\_CPT2BCR 中:

**TF1RST**: 定时器 F 输出 1 复位 (Timer F output 1 reset)  
参考 TA1RST 描述。

Bit 16: 在 HRTIM\_CPT2ACR、HRTIM\_CPT2CCR、HRTIM\_CPT2DCR、HRTIM\_CPT2ECR、HRTIM\_CPT2FCR 中:

**TB1SET:** 定时器 B 输出 1 置位 (Timer B output 1 set)  
参考 TA1SET 描述。

在 HRTIM\_CPT2BCR 中:

**TF1SET:** 定时器 F 输出 1 置位 (Timer F output 1 set)  
参考 TA1SET 描述。

Bit 15: 在 HRTIM\_CPT2BCR、HRTIM\_CPT2CCR、HRTIM\_CPT2DCR、HRTIM\_CPT2ECR、HRTIM\_CPT2FCR 中:

**TACMP2:** 定时器 A 比较 2 (Timer A compare 2)  
0: 无操作  
1: 定时器 A 比较 2 触发捕获 2

在 HRTIM\_CPT2ACR 中:

**TFCMP2:** 定时器 F 比较 2 (Timer F compare 2)  
参考 TACMP2 描述。

Bit 14: 在 HRTIM\_CPT2BCR、HRTIM\_CPT2CCR、HRTIM\_CPT2DCR、HRTIM\_CPT2ECR、HRTIM\_CPT2FCR 中:

**TACMP1:** 定时器 A 比较 1 (Timer A compare 1)  
0: 无操作  
1: 定时器 A 比较 1 触发捕获 2

在 HRTIM\_CPT2ACR 中:

**TFCMP1:** 定时器 F 比较 1 (Timer F compare 1)  
参考 TACMP1 描述。

Bit 13: 在 HRTIM\_CPT2BCR、HRTIM\_CPT2CCR、HRTIM\_CPT2DCR、HRTIM\_CPT2ECR、HRTIM\_CPT2FCR 中:

**TA1RST:** 定时器 A 输出 1 复位 (Timer A output 1 reset)  
0: 无操作  
1: TA1 输出从有效电平跳变为无效电平会触发捕获 2

在 HRTIM\_CPT2ACR 中:

**TF1RST:** 定时器 F 输出 1 复位 (Timer F output 1 reset)  
参考 TA1RST 描述。

Bit 12: 在 HRTIM\_CPT2BCR、HRTIM\_CPT2CCR、HRTIM\_CPT2DCR、HRTIM\_CPT2ECR、HRTIM\_CPT2FCR 中:

**TA1SET:** 定时器 A 输出 1 置位 (Timer A output 1 set)  
0: 无操作  
1: TA1 输出从有效电平跳变为无效电平会触发捕获 2

在 HRTIM\_CPT2ACR 中:

**TF1SET**: 定时器 F 输出 1 置位 (Timer F output 1 set)

参考 TA1SET 描述。

Bit 11 **EXEV10CPT**: 外部事件 10 捕获 (External event 10 capture)

参考 EXEV1CPT 描述。

Bit 10 **EXEV9CPT**: 外部事件 9 捕获 (External event 9 capture)

参考 EXEV1CPT 描述。

Bit 9 **EXEV8CPT**: 外部事件 8 捕获 (External event 8 capture)

参考 EXEV1CPT 描述。

Bit 8 **EXEV7CPT**: 外部事件 7 捕获 (External event 7 capture)

参考 EXEV1CPT 描述。

Bit 7 **EXEV6CPT**: 外部事件 6 捕获 (External event 6 capture)

参考 EXEV1CPT 描述。

Bit 6 **EXEV5CPT**: 外部事件 5 捕获 (External event 5 capture)

参考 EXEV1CPT 描述。

Bit 5 **EXEV4CPT**: 外部事件 4 捕获 (External event 4 capture)

参考 EXEV1CPT 描述。

Bit 4 **EXEV3CPT**: 外部事件 3 捕获 (External event 3 capture)

参考 EXEV1CPT 描述。

Bit 3 **EXEV2CPT**: 外部事件 2 捕获 (External event 2 capture)

参考 EXEV1CPT 描述。

Bit 2 **EXEV1CPT**: 外部事件 1 捕获 (External event 1 capture)

0: 无操作

1: 外部事件 1 触发捕获 2

Bit 1 **UPDCPT**: 更新捕获 (Update capture)

0: 无操作

1: 更新事件触发捕获 2

Bit 0 **SWCPT**: 软件捕获 (Software capture)

0: 无操作

1: 此位通过软件强制触发捕获 2。此位仅可置位, 由硬件复位。

## 22.5.47 HRTIM 定时器 x 输出寄存器 (HRTIM\_OUTxR) (x = A~F)

偏移地址：块 A：0x0E4

偏移地址：块 B：0x164

偏移地址：块 C：0x1E4

偏移地址：块 D：0x264

偏移地址：块 E：0x2E4

偏移地址：块 F：0x364

复位值：0x0000 0000

| 31       | 30   | 29  | 28          | 27 | 26 | 25           | 24   | 23    | 22   | 21          | 20 | 19     | 18     | 17   | 16  |
|----------|------|-----|-------------|----|----|--------------|------|-------|------|-------------|----|--------|--------|------|-----|
| Reserved |      |     |             |    |    |              |      | DIDL2 | CHP2 | FAULT2[1:0] |    | IDLES2 | IDLEM2 | POL2 | Res |
|          |      |     |             |    |    |              |      | rw    | rw   | rw          | rw | rw     | rw     | rw   |     |
| 15       | 14   | 13  | 12          | 11 | 10 | 9            | 8    | 7     | 6    | 5           | 4  | 3      | 2      | 1    | 0   |
| Res      | BIAR | Res | DLYPRT[2:0] |    |    | DLYPR<br>TEN | DTEN | DIDL1 | CHP1 | FAULT1[1:0] |    | IDLES1 | IDLEM1 | POL1 | Res |
|          | rw   |     | rw          | rw | rw | rw           | rw   | rw    | rw   | rw          | rw | rw     | rw     | rw   |     |

Bit 31:24 保留，必须保持复位值。

Bit 23 **DIDL2**：进入突发模式空闲状态的输出 2 死区

此位会在输出切换为其空闲状态之前强制插入死区，从而延迟进入空闲模式。此设置仅适用于在突发模式工作期间进入空闲状态的情况。

0：编程的空闲状态立即应用到输出 2

1：死区（无效电平）会在进入空闲模式之前插入到输出 2 上。死区值由 DTFx[8:0]设置

注：一旦定时器 x 被使能，这个参数就不能更改。

仅当其中一路输出在突发模式期间有效（IDLES=1）、且死区为正值（SDTR/SDTF 设为 0）时，才可设置 DIDL=1。

Bit 22 **CHP2**：输出 2 斩波使能（Output 2 chopper enable）

该位在输出 2 上使能斩波。

0：输出信号不变

1：输出信号被载波信号斩波

注：一旦定时器 x 被使能，这个参数就不能更改。

Bit 21:20 **FAULT2[1:0]**：输出 2 故障状态（Output 2 fault state）

这些位选择故障事件后的输出 2 状态。

00：无动作：输出不受故障输入影响，保持在运行状态。

01：有效

10：无效

11：高阻态

注：如果 FLTENx 位已置 1，或者输出处于 FAULT 状态，则定时器 x 使能后（TxCEN 位置 1），不能更改此参数。

Bit 19 **IDLES2**：输出 2 空闲状态（Output 2 idle state）

此位选择输出 2 的空闲状态。

0：无效

1：有效

注：在 HRTIM 控制输出之前，必须设置此参数。

Bit 18 **IDLEM2**：输出 2 空闲模式（Output 2 idle mode）



此位选择输出 2 的空闲模式。

0: 无操作: 输出不受突发模式工作的影响。

1: 当突发模式控制器请求时, 输出处于空闲状态。

注: 此位进行了预装载, 可在运行期间更改, 但不能在突发模式激活时更改。

Bit 17 **POL2**: 输出 2 极性 (Output 2 polarity)

此位选择输出 2 的极性。

0: 正极性 (输出高电平有效)

1: 负极性 (输出低电平有效)

注: 一旦定时器 x 被使能, 这个参数就不能更改。

Bit 16:15 保留, 必须保持复位值。

Bit 14 **BIAR**: 均衡空闲自动恢复 (Balanced Idle Automatic Resume)

此位选择是否在均衡空闲事件后自动重新使能输出。

此位仅在  $DLYPRT[2:0] = 011$  或  $111$  时有意义, 否则会被忽略。

0: 禁用

1: 使能

注: 一旦定时器 x 被使能, 这个参数就不能更改。

Bit 13 保留, 必须保持复位值。

Bit 12:10 **DLYPRT[2:0]**: 延迟保护 (Delayed protection)

这些位定义应用延迟保护机制的源和输出。

在 **HRTIM\_OUTAR**、**HRTIM\_OUTBR**、**HRTIM\_OUTCR** 中:

000: 发生外部事件 6 时输出 1 进入延迟空闲状态

001: 发生外部事件 6 时输出 2 进入延迟空闲状态

010: 发生外部事件 6 时输出 1 和输出 2 均进入延迟空闲状态

011: 发生外部事件 6 时进入均衡空闲状态

100: 发生外部事件 7 时输出 1 进入延迟空闲状态

101: 发生外部事件 7 时输出 2 进入延迟空闲状态

110: 发生外部事件 7 时输出 1 和输出 2 均进入延迟空闲状态

111: 发生外部事件 7 时进入均衡空闲状态

在 **HRTIM\_OUTDR**、**HRTIM\_OUTER**、**HRTIM\_OUTFR** 中:

000: 发生外部事件 8 时输出 1 进入延迟空闲状态

001: 发生外部事件 8 时输出 2 进入延迟空闲状态

010: 发生外部事件 8 时输出 1 和输出 2 均进入延迟空闲状态

011: 发生外部事件 8 时进入均衡空闲状态

100: 发生外部事件 9 时输出 1 进入延迟空闲状态

101: 发生外部事件 9 时输出 2 进入延迟空闲状态

110: 发生外部事件 9 时输出 1 和输出 2 均进入延迟空闲状态

111: 发生外部事件 9 时进入均衡空闲状态

注: 一旦使能延迟保护 (**DLYPRTEN** 位设置), 则不得修改此位域。

Bit 9 **DLYPRTEN**: 延迟保护使能 (Delayed protection enable)

此位使能延迟保护方案。

0: 无动作

1: 根据 **DLYPRT[2:0]** 位使能延迟保护

注：一旦定时器x 被使能 (TxEN 位设置)，则不能更改此参数。

**Bit 8 DTEN:** 死区时间使能 (Deadtime enable)

此位使能输出 1 和输出 2 上的死区时间插入。

0: 输出 1 和输出 2 信号是独立的。

1: 在输出 1 和输出 2 之间插入死区时间 (参考信号是输出 1 信号发生器)

注：一旦定时器开始运行 (TxEN 位设置) 或其输出被另一个定时器使能和置位/重置，则不能更改此参数。

**Bit 7 DIDL1:** 进入突发模式空闲状态的输出 1 死区时间

此位会在输出切换为其空闲状态之前强制插入死区，从而延迟进入空闲模式。此设置仅适用于在突发模式工作期间进入空闲状态的情况。

0: 编程的空闲状态立即应用于输出 1

1: 死区 (无效电平) 会在进入空闲模式之前插入到输出 1 上。死区值由 DTRx[8:0] 置位

注：一旦定时器x 被使能，则不能更改此参数。

仅当其中一路输出在突发模式期间有效 (IDLES=1)、且死区为正值 (SDTR/SDTF 设为 0) 时，才可设置 DIDL=1。

**Bit 6 CHP1:** 输出 1 斩波使能 (Output 1 chopper enable)

此位使能输出 1 上的斩波。

0: 输出信号不变

1: 输出信号被载波信号斩波

注：一旦定时器x 被使能，则不能更改此参数。

**Bit 5:4 FAULT1[1:0]:** 输出 1 故障状态 (Output 1 fault state)

这些位选择故障事件后输出 1 的状态

00: 无操作: 输出不受故障输入影响，保持在运行模式。

01: 有效状态

10: 无效状态

11: 高阻态

注：如果 FLTENx 位已置 1，或者输出处于 FAULT 状态，则定时器x 使能后 (TxCEN 位置 1)，不能更改此参数

**Bit 3 IDLES1:** 输出 1 空闲状态 (Output 1 Idle State)

此位选择输出 1 的空闲状态。

0: 无效状态

1: 有效状态

注：在 HRTIM 控制输出之前，必须设置此参数。

**Bit 2 IDLEM1:** 输出 1 空闲模式 (Output 1 Idle mode)

此位选择输出 1 的空闲模式。

0: 无操作: 输出不受突发模式操作影响。

1: 当突发模式控制器请求时，输出处于空闲状态。

注：此位进行了预装载，可在运行期间更改，但不能在突发模式激活时更改。

**Bit 1 POL1:** 输出 1 极性 (Output 1 polarity)

此位选择输出 1 的极性。

0: 正极性 (输出有效高电平)

1: 负极性 (输出有效低电平)

注: 一旦定时器x 被使能, 则无法更改此参数。

Bit 0 保留, 必须保持复位值。

## 22.5.48 HRTIM 定时器 x 故障寄存器 (HRTIM\_FLTxR) (x = A~F)

偏移地址: 块 A: 0x0E8

偏移地址: 块 B: 0x168

偏移地址: 块 C: 0x1E8

偏移地址: 块 D: 0x268

偏移地址: 块 E: 0x2E8

偏移地址: 块 F: 0x368

复位值: 0x0000 0000

|          |          |    |    |    |    |    |    |    |    |        |        |        |        |        |        |
|----------|----------|----|----|----|----|----|----|----|----|--------|--------|--------|--------|--------|--------|
| 31       | 30       | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21     | 20     | 19     | 18     | 17     | 16     |
| FTLCK    | Reserved |    |    |    |    |    |    |    |    |        |        |        |        |        |        |
| rw       |          |    |    |    |    |    |    |    |    |        |        |        |        |        |        |
| 15       | 14       | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5      | 4      | 3      | 2      | 1      | 0      |
| Reserved |          |    |    |    |    |    |    |    |    | FLT6EN | FLT5EN | FLT4EN | FLT3EN | FLT2EN | FLT1EN |
|          |          |    |    |    |    |    |    |    |    | rw     | rw     | rw     | rw     | rw     | rw     |

Bit 31 **FTLCK**: 故障源锁定 (Fault sources lock)

0: FLT1EN..FLT6EN 位可读/写

1: FLT1EN..FLT6EN 位只读

FTLCK 位为一次性写入。一旦设置, 直到下一次系统复位前无法修改。

Bit 30:6 保留, 必须保持复位值。

Bit 5 **FLT6EN**: 故障 6 使能 (Fault 6 enable)

0: 忽略故障 6 输入

1: 故障 6 输入有效并禁用 HRTIM 输出

Bit 4 **FLT5EN**: 故障 5 使能 (Fault 5 enable)

0: 忽略故障 5 输入

1: 故障 5 输入有效并禁用 HRTIM 输出

Bit 3 **FLT4EN**: 故障 4 使能 (Fault 4 enable)

0: 忽略故障 4 输入

1: 故障 4 输入有效并禁用 HRTIM 输出

Bit 2 **FLT3EN**: 故障 3 使能 (Fault 3 enable)

0: 忽略故障 3 输入

1: 故障 3 输入有效并禁用 HRTIM 输出

Bit 1 **FLT2EN**: 故障 2 使能 (Fault 2 enable)

0: 忽略故障 2 输入

1: 故障 2 输入有效并禁用 HRTIM 输出

Bit 0 **FLT1EN**: 故障 1 使能 (Fault 1 enable)

0: 忽略故障 1 输入

1: 故障 1 输入有效并禁用 HRTIM 输出

## 22.5.49 HRTIM 定时器 x 控制寄存器 2 (HRTIM\_TIMxCR2) (x = A~F)

偏移地址：块 A：0x0EC

偏移地址：块 B：0x16C

偏移地址：块 C：0x1EC

偏移地址：块 D：0x26C

偏移地址：块 E：0x2EC

偏移地址：块 F：0x36C

复位值：0x0000 0000

|            |    |            |    |            |    |             |    |          |    |     |            |          |      |            |            |
|------------|----|------------|----|------------|----|-------------|----|----------|----|-----|------------|----------|------|------------|------------|
| 31         | 30 | 29         | 28 | 27         | 26 | 25          | 24 | 23       | 22 | 21  | 20         | 19       | 18   | 17         | 16         |
| Reserved   |    |            |    |            |    |             |    |          |    |     | TRG<br>HLF | Reserved |      | GT<br>CMP3 | GT<br>CMP1 |
|            |    |            |    |            |    |             |    |          |    |     | rw         |          |      | rw         | rw         |
| 15         | 14 | 13         | 12 | 11         | 10 | 9           | 8  | 7        | 6  | 5   | 4          | 3        | 2    | 1          | 0          |
| FEROM[1:0] |    | BMROM[1:0] |    | ADROM[1:0] |    | OUTROM[1:0] |    | ROM[1:0] |    | Res | UDM        | Res      | DCDR | DCDS       | DCDE       |
| rw         | rw | rw         | rw | rw         | rw | rw          | rw | rw       | rw |     | rw         |          | rw   | rw         | rw         |

Bit 30:18 保留，必须保持复位值。

Bit 20 **TRGHLF**：半触发模式 (Triggered-half mode)

此位域定义比较 2 寄存器是在标准模式下工作 (计数器等于比较值时立即发出比较匹配)，还是在半触发模式下工作 (请参阅章节 22.3.4 中的半触发模式)。

0：CMP2 寄存器仅由用户写入 (标准比较模式)

1：当捕获 1 事件发生时，CMP2 的值由硬件设置。它被加载为 (捕获 1 值除以 2) 的结果。初始值可以由用户写入 (只要 TRGHLF 被复位)，但在触发第一个捕获后被忽略 (当设置 TRGHLF 位时，CMP2 的预加载机制被禁用)。

注：计数器使能后 (Tx Cen 位设置)，不得修改此位域。

Bit 19:18 保留，必须保持复位值。

Bit 17 **GTCMP3**：大于比较 3 的 PWM 模式 (Greater than compare 3 PWM mode)

此位定义比较 3 的操作模式。

0：当计数器等于比较值时生成比较 3 事件 (比较匹配模式)

1：当计数器大于比较值时生成比较 3 事件。如果比较值在运行时更改，则将新的比较值与当前计数器值进行比较，并可以生成输出置位或复位。

Bit 16 **GTCMP1**：大于比较 1 的 PWM 模式 (Greater than compare 1 PWM mode)

此位定义比较 1 的操作模式。

0：当计数器等于比较值时生成比较 1 事件 (比较匹配模式)

1：当计数器大于比较值时生成比较 1 事件。如果比较值在运行时更改，则将新的比较值与当前计数器值进行比较，并可以生成输出置位或复位。

**Bit 15:14 FEROM[1:0]: 故障和事件翻转模式 (Fault and event roll-over mode)**

此位域定义在上下计数模式下何时生成溢出。它仅涉及故障和事件计数器所使用的翻转事件。

00: 当计数器等于 0 或等于 HRTIM\_PERxR 值时生成事件

01: 当计数器等于 0 时生成事件

10: 当计数器等于 HRTIM\_PERxR 时生成事件

11: 保留

注: 此设置仅当 UDM 位设置时适用。否则它不重要。

注: 一旦定时器开始运行 (TxEN 位设置), 此位域无法更改。

**Bit 13:12 BMROM[1:0]: 突发模式翻转模式 (Burst mode roll-over mode)**

此位域定义在上下计数模式下何时生成翻转。它仅涉及在突发模式控制器中使用的翻转事件, 作为时钟以及突发模式触发。

00: 当计数器等于 0 或等于 HRTIM\_PERxR 值时生成事件

01: 当计数器等于 0 时生成事件

10: 当计数器等于 HRTIM\_PERxR 时生成事件

11: 保留

注: 此设置仅当 UDM 位设置时适用。否则它不重要。

注: 一旦定时器开始运行 (TxEN 位设置), 此参数无法更改。

**Bit 11:10 ADROM[1:0]: ADC 翻转模式 (ADC roll-over mode)**

此位域定义在上下计数模式下何时生成翻转。它仅涉及触发 ADC 的翻转事件。

00: 当计数器等于 0 或等于 HRTIM\_PERxR 值时生成事件

01: 当计数器等于 0 时生成事件

10: 当计数器等于 HRTIM\_PERxR 时生成事件

11: 保留

注: 此设置仅当 UDM 位设置时适用。否则它不重要。

注: 一旦定时器开始运行 (TxEN 位设置), 此位域无法更改。

**Bit 9:8 OUTROM[1:0]: 输出翻转模式 (Output roll-over mode)**

此位域定义在上下计数模式下何时生成翻转。它仅涉及根据 HRTIM\_SETxyR 和 HRTIM\_RSTxyR 设置来设置和/或重置输出的翻转事件。

00: 当计数器等于 0 或等于 HRTIM\_PERxR 值时生成事件

01: 当计数器等于 0 时生成事件

10: 当计数器等于 HRTIM\_PERxR 时生成事件

11: 保留

注: 此设置仅当 UDM 位设置时适用。否则它不重要。

注: 一旦定时器开始运行 (TxEN 位设置), 此位域无法更改。

**Bit 7:6 ROM[1:0]: 翻转模式 (Roll-over mode)**

此位域定义在上下计数模式下何时生成翻转。它仅涉及具有以下目标的翻转事件: 更新触发 (将内容从预加载寄存器传输到有效寄存器)、IRQ 和 DMA 请求、重复计数器递减和外部事件过滤。

00: 当计数器等于 0 或等于 HRTIM\_PERxR 值时生成事件

01: 当计数器等于 0 时生成事件

10: 当计数器等于 HRTIM\_PERxR 时生成事件

11: 保留

注: 此设置仅当 UDM 位设置时适用。否则它不重要。

注: 一旦定时器开始运行 (TxEN 位设置), 此位域无法更改。

Bit 5 保留, 必须保持复位值。

Bit 4 UDM: 上下计数模式 (Up-Down mode)

此位定义计数器是否以向上或上下计数模式运行。

0: 计数器以向上计数模式运行

1: 计数器以上下计数模式运行

注: 一旦定时器开始运行 (TxEN 位设置), 此位无法更改。

Bit 3 保留, 必须保持复位值。

Bit 2 DCDR: 双通道 DAC 复位触发 (Dual channel DAC reset trigger)

此位定义何时生成 hrtim\_dac\_reset\_trgx 触发。

0: 在计数器复位或溢出事件时生成触发

1: 在输出 1 设置事件时生成触发

注: 当 DCDE 位复位 (双通道 DAC 触发禁用) 时, DCDR 位无效。

Bit 1 DCDS: 双通道 DAC 步进触发 (Dual channel DAC Step trigger)

此位定义何时生成 hrtim\_dac\_step\_trgx 触发。

0: 在比较 2 事件时生成触发

1: 在输出 1 复位事件时生成触发

注: 当 DCDE 位复位 (双通道 DAC 触发禁用) 时, DCDS 位无效。

位 0 DCDE: 双通道 DAC 触发使能 (Dual channel DAC trigger enable)

此位使能双通道 DAC 触发机制。

0: 双通道 DAC 触发禁用

1: 双通道 DAC 触发使能

注: 一旦定时器开始运行 (TxEN 位设置), 此位无法更改。



## 22.5.50 HRTIM 定时器 x 外部事件滤波寄存器 3 (HRTIM\_TIMxEEFR3) (x = A~F)

偏移地址：块 A：0x0F0

偏移地址：块 B：0x170

偏移地址：块 C：0x1F0

偏移地址：块 D：0x270

偏移地址：块 E：0x2F0

偏移地址：块 F：0x370

复位值：0x0000 0000

|          |    |              |    |    |    |    |    |              |    |    |    |     |              |              |            |
|----------|----|--------------|----|----|----|----|----|--------------|----|----|----|-----|--------------|--------------|------------|
| 31       | 30 | 29           | 28 | 27 | 26 | 25 | 24 | 23           | 22 | 21 | 20 | 19  | 18           | 17           | 16         |
| Reserved |    |              |    |    |    |    |    |              |    |    |    |     |              |              |            |
| 15       | 14 | 13           | 12 | 11 | 10 | 9  | 8  | 7            | 6  | 5  | 4  | 3   | 2            | 1            | 0          |
| Reserved |    | EEVACNT[5:0] |    |    |    |    |    | EEVASEL[3:0] |    |    |    | Res | EEVA<br>RSTM | EEVA<br>CRES | EEVA<br>CE |
|          |    | rw           | rw | rw | rw | rw | rw | rw           | rw | rw | rw |     | rw           | rw           | rw         |

Bit 30:14 保留，必须保持复位值。

Bit 13:8 **EEVACNT[5:0]**：外部事件 A 计数器 (External event A counter)

此位域选择外部事件 A 计数器的阈值。当事件数量等于 (EEVACNT[5:0]+1) 值时，该事件被视为有效。

Bit 7:4 **EEVASEL[3:0]**：外部事件 A 选择 (External event A Selection)

此位选择外部事件 A 的源。

0：外部事件 1 用作外部事件 A 源

1：外部事件 2 用作外部事件 A 源

...

9：外部事件 10 用作外部事件 A 源

其他：保留

Bit 3 保留，必须保持复位值。

Bit 2 **EEVARSTM**：外部事件 A 复位模式 (External event A reset mode)

此位选择外部事件 x 计数器的复位模式。

0：外部事件计数器 A 在每个复位/翻转事件时复位

1：外部事件计数器 A 仅在最后一个计数周期内没有事件发生时，在每个复位/翻转事件时复位

Bit 1 **EEVACRES**：外部事件 A 计数器复位 (External event A counter reset)

此位复位外部事件 A 计数器。它由软件设置并由硬件复位。

0：无操作

1：外部事件计数器 A 复位

Bit 0 **EEVACE**：外部事件 A 计数器使能 (External event A counter enable)

此位使能外部事件 x 计数器。

0：外部事件 A 计数器禁用

1：外部事件 A 计数器使能



## 22.5.51 HRTIM 控制寄存器 1 (HRTIM\_CR1)

偏移地址：0x380

复位值：0x0000 0000

| 31       | 30 | 29 | 28 | 27           | 26 | 25 | 24           | 23 | 22   | 21           | 20   | 19   | 18           | 17   | 16    |
|----------|----|----|----|--------------|----|----|--------------|----|------|--------------|------|------|--------------|------|-------|
| Reserved |    |    |    | AD4USRC[2:0] |    |    | AD3USRC[2:0] |    |      | AD2USRC[2:0] |      |      | AD1USRC[2:0] |      |       |
|          |    |    |    | rw           | rw | rw | rw           | rw | rw   | rw           | rw   | rw   | rw           | rw   | rw    |
| 15       | 14 | 13 | 12 | 11           | 10 | 9  | 8            | 7  | 6    | 5            | 4    | 3    | 2            | 1    | 0     |
| Reserved |    |    |    |              |    |    |              |    | TF   | TE           | TD   | TC   | TB           | TA   | MUDIS |
|          |    |    |    |              |    |    |              |    | UDIS | UDIS         | UDIS | UDIS | UDIS         | UDIS | UDIS  |
|          |    |    |    |              |    |    |              |    | rw   | rw           | rw   | rw   | rw           | rw   | rw    |

Bit 30:28 保留，必须保持复位值。

Bit 27:25 **AD4USRC[2:0]**: ADC 触发 4 更新源 (ADC trigger 4 update source)

参考 AD1USRC[2:0]的描述。

Bit 24:22 **AD3USRC[2:0]**: ADC 触发 3 更新源 (ADC trigger 3 update source)

参考 AD1USRC[2:0]的描述。

Bit 21:19 **AD2USRC[2:0]**: ADC 触发 2 更新源 (ADC trigger 2 update source)

参考 AD1USRC[2:0]的描述。

Bit 18:16 **AD1USRC[2:0]**: ADC 触发 1 更新源 (ADC trigger 1 update source)

这些位定义将触发 HRTIM\_ADC1R 寄存器更新的源 (从预装载寄存器传输到活动寄存器)。它只定义源定时器。准确条件在定时器自身的 HRTIM\_MCR 或 HRTIM\_TIMxCR 中定义。

000: 主定时器

001: 定时器 A

010: 定时器 B

011: 定时器 C

100: 定时器 D

101: 定时器 E

110: 定时器 F

111: 保留

Bit 15:7 保留，必须保持复位值。

Bit 6 **TFUDIS**: 定时器 F 更新禁止 (Timer F update disable)

参考 TAUDIS 的描述。

Bit 5 **TEUDIS**: 定时器 E 更新禁止 (Timer E update disable)

参考 TAUDIS 的描述。

Bit 4 **TDUDIS**: 定时器 D 更新禁止 (Timer D update disable)

参考 TAUDIS 的描述。

Bit 3 **TCUDIS**: 定时器 C 更新禁止 (Timer C update disable)

参考 TAUDIS 的描述。

Bit 2 **TBUDIS**: 定时器 B 更新禁止 (Timer B update disable)

参考 TAUDIS 的描述。

Bit 1 **TAUDIS**: 定时器 A 更新禁止 (Timer A update disable)

此位由软件置 1 和清零，用于暂时使能/禁止定时器 A 上的更新事件生成。

0: 使能更新。生成所选源后立即进行更新。

1: 禁止更新。暂时禁止更新，使软件能够写入多个需要同时考虑的寄存器。

Bit 0 **MUDIS**: 主定时器更新使能 (Master update disable)

此位由软件置 1 和清零，用于暂时使能/禁止更新事件生成。

0: 使能更新。

1: 禁止更新。暂时禁止更新，使软件能够写入多个需要同时考虑的寄存器。

## 22.5.52 HRTIM 控制寄存器 2 (HRTIM\_CR2)

偏移地址: 0x384

复位值: 0x0000 0000

| 31       | 30    | 29    | 28    | 27    | 26    | 25    | 24   | 23  | 22  | 21   | 20   | 19   | 18   | 17   | 16   |
|----------|-------|-------|-------|-------|-------|-------|------|-----|-----|------|------|------|------|------|------|
| Reserved |       |       |       |       |       |       |      |     |     | SWPF | SWPE | SWPD | SWPC | SWPB | SWPA |
|          |       |       |       |       |       |       |      |     |     | rw   | rw   | rw   | rw   | rw   | rw   |
| 15       | 14    | 13    | 12    | 11    | 10    | 9     | 8    | 7   | 6   | 5    | 4    | 3    | 2    | 1    | 0    |
| Res      | TFRST | TERST | TDRST | TCRST | TBRST | TARST | MRST | Res | TF  | TE   | TD   | TC   | TB   | TA   | MSWU |
|          | rw    | rw    | rw    | rw    | rw    | rw    | rw   |     | SWU | SWU  | SWU  | SWU  | SWU  | SWU  | rw   |

Bit 31:22 保留, 必须保持复位值。

Bit 21 **SWPF**: 交换定时器 F 输出 (Swap timer F outputs)

参考 SWPA 的描述。

注: 当使能推挽模式 (PSHPLL = 1) 时, 此位无效。

Bit 20 **SWPE**: 交换定时器 E 输出 (Swap timer E outputs)

参考 SWPA 的描述。

注: 当使能推挽模式 (PSHPLL = 1) 时, 此位无效。

Bit 19 **SWPD**: 交换定时器 D 输出 (Swap timer D outputs)

参考 SWPA 的描述。

注: 当使能推挽模式 (PSHPLL = 1) 时, 此位无效。

Bit 18 **SWPC**: 交换定时器 C 输出 (Swap timer C outputs)

参考 SWPA 的描述。

注: 当使能推挽模式 (PSHPLL = 1) 时, 此位无效。

Bit 17 **SWPB**: 交换定时器 B 输出 (Swap timer B outputs)

参考 SWPA 的描述。

注: 当使能推挽模式 (PSHPLL = 1) 时, 此位无效。

Bit 16 **SWPA**: 交换定时器 A 输出 (Swap timer A outputs)

此位允许交换定时器 A 的输出。

0: HRTIM\_SETA1R 和 HRTIM\_RSTA1R 编码为输出 A1, HRTIM\_SETA2R 和 HRTIM\_RSTA2R 编码为输出 A2

1: HRTIM\_SETA1R 和 HRTIM\_RSTA1R 编码为输出 A2, HRTIM\_SETA2R 和 HRTIM\_RSTA2R 编码为输出 A1

注: 当使能推挽模式 (PSHPLL = 1) 时, 此位无效。

Bit 15 保留, 必须保持复位值。

Bit 14 **TFRST**: 定时器 F 计数器软件复位 (Timer F counter software reset)

参考 TARST 的描述。

Bit 13 **TERST**: 定时器 E 计数器软件复位 (Timer E counter software reset)

参考 TARST 的描述。

Bit 12 **TDRST**: 定时器 D 计数器软件复位 (Timer D counter software reset)

参考 TARST 的描述。

Bit 11 **TCRST**: 定时器 C 计数器软件复位 (Timer C counter software reset)

参考 TARST 的描述。

Bit 10 **TBRST**: 定时器 B 计数器软件复位 (Timer B counter software reset)

参考 TARST 的描述。

Bit 9 **TARST**: 定时器 A 计数器软件复位 (Timer A counter software reset)

将此位置 1 会复位定时器 A 计数器。

此位自动通过硬件复位。

Bit 8 **MRST**: 主定时器计数器软件复位 (Master counter software reset)

将此位置 1 会复位主定时器计数器。

此位自动通过硬件复位。

Bit 7 保留, 必须保持复位值。

Bit 6 **TFSWU**: 定时器 F 软件更新 (Timer F software update)

参考 TASWU 的描述。

Bit 5 **TESWU**: 定时器 E 软件更新 (Timer E software update)

参考 TASWU 的描述。

Bit 4 **TDSWU**: 定时器 D 软件更新 (Timer D software update)

参考 TASWU 的描述。

Bit 3 **TCSWU**: 定时器 C 软件更新 (Timer C software update)

参考 TASWU 的描述。

Bit 2 **TBSWU**: 定时器 B 软件更新 (Timer B software update)

参考 TASWU 的描述。

Bit 1 **TASWU**: 定时器 A 软件更新 (Timer A software update)

此位由软件置 1, 并由硬件自动复位。它会强制立即执行从预装载寄存器到活动寄存器的传输操作, 并会取消任何挂起的更新请求。

Bit 0 **MSWU**: 主定时器软件更新 (Master timer software update)

此位由软件置 1, 并由硬件自动复位。它会强制立即执行从主定时器中的预装载寄存器到活动寄存器的传输操作, 并会取消任何挂起的更新请求。

## 22.5.53 HRTIM 中断状态寄存器 (HRTIM\_ISR)

偏移地址: 0x388

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |    |      |            |      |      |      |      |      |
|----------|----|----|----|----|----|----|----|----|------|------------|------|------|------|------|------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22   | 21         | 20   | 19   | 18   | 17   | 16   |
| Reserved |    |    |    |    |    |    |    |    |      |            |      |      |      | BMPE | DLL  |
|          |    |    |    |    |    |    |    |    |      |            |      |      |      | R    | RDY  |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6    | 5          | 4    | 3    | 2    | 1    | 0    |
| Reserved |    |    |    |    |    |    |    |    | FLT6 | SYS<br>FLY | FLT5 | FLT4 | FLT3 | FLT2 | FLT1 |
|          |    |    |    |    |    |    |    |    | r    |            | r    | r    | r    | r    | r    |

Bit 31:18 保留, 必须保持复位值。

Bit 17 **BMPE**: 突发模式周期中断标志 (Burst mode period interrupt flag)

单发突发模式工作结束、或连续模式下突发模式周期结束时, 此位会由硬件置 1。通过软件向该位写入 1 可将其清零。

0: 未发生突发模式周期中断

1: 发生突发模式周期中断

Bit 16 **DLLRDY**: DLL 就绪中断标志 (DLL Ready Interrupt Flag)

当 DLL 校准完成时, 硬件会设置此位。通过软件将其写为 1 来清除。

0: 未发生 DLL 校准就绪中断

1: DLL 校准就绪中断发生

Bit 15:7 保留, 必须保持复位值。

Bit 6 **FLT6**: 故障 6 中断标志 (Fault 6 interrupt flag)

参考 FLT1 的描述。

Bit 5 **SYSFLT**: 系统故障中断标志 (System fault interrupt flag)

参考 FLT1 的描述。

Bit 4 **FLT5**: 故障 5 中断标志 (Fault 5 interrupt flag)

参考 FLT1 的描述。

Bit 3 **FLT4**: 故障 4 中断标志 (Fault 4 interrupt flag)

参考 FLT1 的描述。

Bit 2 **FLT3**: 故障 3 中断标志 (Fault 3 interrupt flag)

参考 FLT1 的描述。

Bit 1 **FLT2**: 故障 2 中断标志 (Fault 2 interrupt flag)

参考 FLT1 的描述。

Bit 0 **FLT1**: 故障 1 中断标志 (Fault 1 interrupt flag)

当发生故障 1 事件时, 硬件会设置此位。通过软件将其写为 1 来清除。

0: 未发生故障 1 中断

1: 发生故障 1 中断

## 22.5.54 HRTIM 中断清除寄存器 (HRTIM\_ICR)

偏移地址: 0x38C

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |    |       |             |       |       |       |       |       |
|----------|----|----|----|----|----|----|----|----|-------|-------------|-------|-------|-------|-------|-------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22    | 21          | 20    | 19    | 18    | 17    | 16    |
| Reserved |    |    |    |    |    |    |    |    |       |             |       |       |       | BMPE  | DLL   |
|          |    |    |    |    |    |    |    |    |       |             |       |       |       | RC    | RDYC  |
|          |    |    |    |    |    |    |    |    |       |             |       |       |       | w     | w     |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6     | 5           | 4     | 3     | 2     | 1     | 0     |
| Reserved |    |    |    |    |    |    |    |    | FLT6C | SYS<br>FLYC | FLT5C | FLT4C | FLT3C | FLT2C | FLT1C |
|          |    |    |    |    |    |    |    |    | w     | w           | w     | w     | w     | w     | w     |

Bit 31:18 保留, 必须保持复位值。

Bit 17 **BMPERC**: 突发模式周期标志清除 (Burst mode period flag clear)

向此位写入 1 将清除 HRTIM\_ISR 寄存器中的 BMPER 标志。

Bit 16 **DLLRDYC**: DLL 就绪中断标志清除 (DLL Ready Interrupt flag Clear)

向此位写入 1 将清除 HRTIM\_ISR 寄存器中的 DLLRDY 标志。

Bit 15:7 保留, 必须保持复位值。

Bit 6 **FLT6C**: 故障 6 中断标志清除 (Fault 6 interrupt flag clear)

向此位写入 1 将清除 HRTIM\_ISR 寄存器中的 FLT6 标志。

Bit 5 **SYSFLT**: 系统故障中断标志清除 (System fault interrupt flag clear)

向此位写入 1 将清除 HRTIM\_ISR 寄存器中的 SYSFLT 标志。

Bit 4 **FLT5C**: 故障 5 中断标志清除 (Fault 5 interrupt flag clear)

向此位写入 1 将清除 HRTIM\_ISR 寄存器中的 FLT5 标志。

Bit 3 **FLT4C**: 故障 4 中断标志清除 (Fault 4 interrupt flag clear)

向此位写入 1 将清除 HRTIM\_ISR 寄存器中的 FLT4 标志。

Bit 2 **FLT3C**: 故障 3 中断标志清除 (Fault 3 interrupt flag clear)

向此位写入 1 将清除 HRTIM\_ISR 寄存器中的 FLT3 标志。

Bit 1 **FLT2C**: 故障 2 中断标志清除 (Fault 2 interrupt flag clear)

向此位写入 1 将清除 HRTIM\_ISR 寄存器中的 FLT2 标志。

Bit 0 **FLT1C**: 故障 1 中断标志清除 (Fault 1 interrupt flag clear)

向此位写入 1 将清除 HRTIM\_ISR 寄存器中的 FLT1 标志。

## 22.5.55 HRTIM 中断使能寄存器 (HRTIM\_IER)

偏移地址: 0x390

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |    |      |       |        |        |        |        |        |
|----------|----|----|----|----|----|----|----|----|------|-------|--------|--------|--------|--------|--------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22   | 21    | 20     | 19     | 18     | 17     | 16     |
| Reserved |    |    |    |    |    |    |    |    |      |       |        |        |        | BMPE   | DLL    |
|          |    |    |    |    |    |    |    |    |      |       |        |        |        | RIE    | RDYIE  |
|          |    |    |    |    |    |    |    |    |      |       |        |        |        | rw     | rw     |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6    | 5     | 4      | 3      | 2      | 1      | 0      |
| Reserved |    |    |    |    |    |    |    |    | FLT6 | SYS   | FLT5IE | FLT4IE | FLT3IE | FLT2IE | FLT1IE |
|          |    |    |    |    |    |    |    |    | IE   | FLYIE |        |        |        |        |        |
|          |    |    |    |    |    |    |    |    | rw   | rw    | rw     | rw     | rw     | rw     | rw     |

Bit 31:18 保留, 必须保持复位值。

Bit 17 **BMPE**: 突发模式周期中断使能 (Burst mode period interrupt enable)

此位由软件设置和清除, 以使能/禁用突发模式周期中断。

0: 突发模式周期中断禁用

1: 突发模式周期中断使能

Bit 16 **DLLRDYIE**: DLL 就绪中断使能 (DLL Ready Interrupt Enable)

此位由软件设置和清除, 以使能/禁用 DLL 就绪中断。

0: DLL 就绪中断禁用

1: DLL 就绪中断使能

Bit 15:7 保留, 必须保持复位值。

Bit 6 **FLT6IE**: 故障 6 中断使能 (Fault 6 interrupt enable)

参考 FLT1IE 的描述。

Bit 5 **SYSFLT IE**: 系统故障中断使能 (System fault interrupt enable)

参考 FLT1IE 的描述。

Bit 4 **FLT5 IE**: 故障 5 中断使能 (Fault 5 interrupt enable)

参考 FLT1IE 的描述。

Bit 3 **FLT4 IE**: 故障 4 中断使能 (Fault 4 interrupt enable)

参考 FLT1IE 的描述。

Bit 2 **FLT3 IE**: 故障 3 中断使能 (Fault 3 interrupt enable)

参考 FLT1IE 的描述。

Bit 1 **FLT2 IE**: 故障 2 中断使能 (Fault 2 interrupt enable)

参考 FLT1IE 的描述。

Bit 0 **FLT1 IE**: 故障 1 中断使能 (Fault 1 interrupt enable)

此位由软件设置和清除, 以使能/禁用故障 1 中断。

0: 故障 1 中断禁用

1: 故障 1 中断使能



## 22.5.56 HRTIM 输出使能寄存器 (HRTIM\_OENR)

偏移地址: 0x394

复位值: 0x0000 0000

| 31       | 30 | 29 | 28 | 27   | 26   | 25   | 24   | 23   | 22   | 21   | 20   | 19   | 18   | 17   | 16   |
|----------|----|----|----|------|------|------|------|------|------|------|------|------|------|------|------|
| Reserved |    |    |    |      |      |      |      |      |      |      |      |      |      |      |      |
| 15       | 14 | 13 | 12 | 11   | 10   | 9    | 8    | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| Reserved |    |    |    | TF2O | TF1O | TE2O | TE1O | TD2O | TD1O | TC2O | TC1O | TB2O | TB1O | TA2O | TA1O |
|          |    |    |    | EN   | EN   | EN   | EN   | EN   | EN   | EN   | EN   | EN   | EN   | EN   | EN   |
|          |    |    |    | rs   | rs   | rs   | rs   | rs   | rs   | rs   | rs   | rs   | rs   | rs   | rs   |

Bit 31:12 保留, 必须保持复位值。

Bit 11 **TF2OEN**: 定时器 F 输出 2 使能 (Timer F output 2 enable)

参考 TA1OEN 的描述。

Bit 10 **TF1OEN**: 定时器 F 输出 1 使能 (Timer F output 1 enable)

参考 TA1OEN 的描述。

Bit 9 **TE2OEN**: 定时器 E 输出 2 使能 (Timer E output 2 enable)

参考 TA1OEN 的描述。

Bit 8 **TE1OEN**: 定时器 E 输出 1 使能 (Timer E output 1 enable)

参考 TA1OEN 的描述。

Bit 7 **TD2OEN**: 定时器 D 输出 2 使能 (Timer D output 2 enable)

参考 TA1OEN 的描述。

Bit 6 **TD1OEN**: 定时器 D 输出 1 使能 (Timer D output 1 enable)

参考 TA1OEN 的描述。

Bit 5 **TC2OEN**: 定时器 C 输出 2 使能 (Timer C output 2 enable)

参考 TA1OEN 的描述。

Bit 4 **TC1OEN**: 定时器 C 输出 1 使能 (Timer C output 1 enable)

参考 TA1OEN 的描述。

Bit 3 **TB2OEN**: 定时器 B 输出 2 使能 (Timer B output 2 enable)

参考 TA1OEN 的描述。

Bit 2 **TB1OEN**: 定时器 B 输出 1 使能 (Timer B output 1 enable)

参考 TA1OEN 的描述。

Bit 1 **TA2OEN**: 定时器 A 输出 2 使能 (Timer A output 2 enable)

参考 TA1OEN 的描述。

Bit 0 **TA1OEN**: 定时器 A 输出 1 使能 (Timer A output 1 enable)

设置此位将使能定时器 A 的输出 1。写入“0”无影响。

读取此位将返回输出使能/禁用状态。

当与定时器相关的故障输入处于有效状态时, 此位会由硬件异步清除。

0: 输出 A1 禁用。输出处于故障或空闲状态。

1: 输出 A1 使能

注: 禁止状态对应空闲和故障两种状态。输出禁止状态由 HRTIM\_ODSR 寄存器中的 TA1ODS 位提供。

## 22.5.57 HRTIM 输出禁用寄存器 (HRTIM\_ODISR)

偏移地址: 0x398

复位值: 0x0000 0000

| 31       | 30 | 29 | 28 | 27   | 26   | 25   | 24   | 23   | 22   | 21   | 20   | 19   | 18   | 17   | 16   |
|----------|----|----|----|------|------|------|------|------|------|------|------|------|------|------|------|
| Reserved |    |    |    |      |      |      |      |      |      |      |      |      |      |      |      |
| 15       | 14 | 13 | 12 | 11   | 10   | 9    | 8    | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| Reserved |    |    |    | TF2  | TF1  | TE2  | TE1  | TD2  | TD1  | TC2  | TC1  | TB2  | TB1  | TA2  | TA1  |
|          |    |    |    | ODIS | ODIS | ODIS | ODIS | ODIS | ODIS | ODIS | ODIS | ODIS | ODIS | ODIS | ODIS |
|          |    |    |    | w    | w    | w    | w    | w    | w    | w    | w    | w    | w    | w    | w    |

Bit 31:12 保留, 必须保持复位值。

Bit 11 **TF2ODIS**: 定时器 F 输出 2 禁用 (Timer F output 2 disable)

参考 TA1ODIS 的描述。

Bit 10 **TF1ODIS**: 定时器 F 输出 1 禁用 (Timer F output 1 disable)

参考 TA1ODIS 的描述。

Bit 9 **TE2ODIS**: 定时器 E 输出 2 禁用 (Timer E output 2 disable)

参考 TA1ODIS 的描述。

Bit 8 **TE1ODIS**: 定时器 E 输出 1 禁用 (Timer E output 1 disable)

参考 TA1ODIS 的描述。

Bit 7 **TD2ODIS**: 定时器 D 输出 2 禁用 (Timer D output 2 disable)

参考 TA1ODIS 的描述。

Bit 6 **TD1ODIS**: 定时器 D 输出 1 禁用 (Timer D output 1 disable)

参考 TA1ODIS 的描述。

Bit 5 **TC2ODIS**: 定时器 C 输出 2 禁用 (Timer C output 2 disable)

参考 TA1ODIS 的描述。

Bit 4 **TC1ODIS**: 定时器 C 输出 1 禁用 (Timer C output 1 disable)

参考 TA1ODIS 的描述。

Bit 3 **TB2ODIS**: 定时器 B 输出 2 禁用 (Timer B output 2 disable)

参考 TA1ODIS 的描述。

Bit 2 **TB1ODIS**: 定时器 B 输出 1 禁用 (Timer B output 1 disable)

参考 TA1ODIS 的描述。

Bit 1 **TA2ODIS**: 定时器 A 输出 2 禁用 (Timer A output 2 disable)

参考 TA1ODIS 的描述。

Bit 0 **TA1ODIS**: 定时器 A 输出 1 禁用 (Timer A output 1 enable)

将此位置 1 会禁止定时器 A 输出 1。输出从运行状态或故障状态进入空闲状态。

写入“0”无影响。

## 22.5.58 HRTIM 输出禁用状态寄存器 (HRTIM\_ODSR)

偏移地址: 0x39C

复位值: 0x0000 0000

| 31       | 30 | 29 | 28 | 27         | 26         | 25         | 24         | 23         | 22         | 21         | 20         | 19         | 18         | 17         | 16         |
|----------|----|----|----|------------|------------|------------|------------|------------|------------|------------|------------|------------|------------|------------|------------|
| Reserved |    |    |    |            |            |            |            |            |            |            |            |            |            |            |            |
| 15       | 14 | 13 | 12 | 11         | 10         | 9          | 8          | 7          | 6          | 5          | 4          | 3          | 2          | 1          | 0          |
| Reserved |    |    |    | TF2<br>ODS | TF1<br>ODS | TE2<br>ODS | TE1<br>ODS | TD2<br>ODS | TD1<br>ODS | TC2<br>ODS | TC1<br>ODS | TB2<br>ODS | TB1<br>ODS | TA2<br>ODS | TA1<br>ODS |
|          |    |    |    | r          | r          | r          | r          | r          | r          | r          | r          | r          | r          | r          | r          |

Bit 31:12 保留, 必须保持复位值。

Bit 11 **TF2ODS**: 定时器 F 输出 2 禁用状态 (Timer F output 2 disable status)

参考 TA1ODS 的描述。

Bit 10 **TF1ODS**: 定时器 F 输出 1 禁用状态 (Timer F output 1 disable status)

参考 TA1ODS 的描述。

Bit 9 **TE2ODS**: 定时器 E 输出 2 禁用状态 (Timer E output 2 disable status)

参考 TA1ODS 的描述。

Bit 8 **TE1ODS**: 定时器 E 输出 1 禁用状态 (Timer E output 1 disable status)

参考 TA1ODS 的描述。

Bit 7 **TD2ODS**: 定时器 D 输出 2 禁用状态 (Timer D output 2 disable status)

参考 TA1ODS 的描述。

Bit 6 **TD1ODS**: 定时器 D 输出 1 禁用状态 (Timer D output 1 disable status)

参考 TA1ODS 的描述。

Bit 5 **TC2ODS**: 定时器 C 输出 2 禁用状态 (Timer C output 2 disable status)

参考 TA1ODS 的描述。

Bit 4 **TC1ODS**: 定时器 C 输出 1 禁用状态 (Timer C output 1 disable status)

参考 TA1ODS 的描述。

Bit 3 **TB2ODS**: 定时器 B 输出 2 禁用状态 (Timer B output 2 disable status)

参考 TA1ODS 的描述。

Bit 2 **TB1ODS**: 定时器 B 输出 1 禁用状态 (Timer B output 1 disable status)

参考 TA1ODS 的描述。

Bit 1 **TA2ODS**: 定时器 A 输出 2 禁用状态 (Timer A output 2 disable status)

参考 TA1ODS 的描述。

Bit 0 **TA1ODS**: 定时器 A 输出 1 禁用状态 (Timer A output 1 enable status)

读取此位将返回输出禁用状态。当输出处于有效状态时 (Tx1OEN 或 Tx2OEN = 1), 此位没有意义。

0: 输出 A1 禁用, 处于空闲状态。

1: 输出 A1 禁用, 处于故障状态。

## 22.5.59 HRTIM 突发模式控制寄存器 (HRTIM\_BMCR)

偏移地址: 0x3A0

复位值: 0x0000 0000

| 31         | 30       | 29 | 28 | 27 | 26         | 25          | 24 | 23 | 22   | 21         | 20   | 19   | 18   | 17   | 16   |
|------------|----------|----|----|----|------------|-------------|----|----|------|------------|------|------|------|------|------|
| BMSTA<br>T | Reserved |    |    |    |            |             |    |    | TFBM | TEBM       | TDBM | TCBM | TBBM | TABM | MTBM |
| rc_w0      |          |    |    |    |            |             |    |    | rw   | rw         | rw   | rw   | rw   | rw   | rw   |
| 15         | 14       | 13 | 12 | 11 | 10         | 9           | 8  | 7  | 6    | 5          | 4    | 3    | 2    | 1    | 0    |
| Reserved   |          |    |    |    | BMPR<br>EN | BMPRSC[3:0] |    |    |      | BMCLK[3:0] |      |      |      | BMOM | BME  |
|            |          |    |    |    | rw         | rw          | rw | rw | rw   | rw         | rw   | rw   | rw   | rw   | rw   |

Bit 31 **BMSTAT**: 突发模式状态 (Burst mode status)

此位提供当前工作状态。

0: 正常工作

1: 正在进行突发操作。将此位写入 0 会使突发模式提前终止。

Bit 30:23 保留, 必须保持复位值。

Bit 22 **TFBM**: 定时器 F 突发模式 (Timer F burst mode)

参考 TABM 描述。

Bit 21 **TEBM**: 定时器 E 突发模式 (Timer E burst mode)

参考 TABM 描述。

Bit 20 **TDBM**: 定时器 D 突发模式 (Timer D burst mode)

参考 TABM 描述。

Bit 19 **TCBM**: 定时器 C 突发模式 (Timer C burst mode)

参考 TABM 描述。

Bit 18 **TBBM**: 定时器 B 突发模式 (Timer B burst mode)

参考 TABM 描述。

Bit 17 **TABM**: 定时器 A 突发模式 (Timer A burst mode)

此位定义定时器在突发模式工作期间的行为。突发模式使能后, 不能更改此位域。

0: 定时器 A 计数器时钟保持, 定时器正常工作

1: 定时器 A 计数器时钟停止, 计数器复位

注: 当均衡空闲模式激活时 (DLYPRT[2:0] = 0x11), 不得将此位置 1

Bit 16 **MTBM**: 主定时器突发模式 (Master timer burst mode)

此位定义定时器在突发模式工作期间的行为。突发模式使能后, 不能更改此位域。

0: 主定时器计数器时钟保持, 定时器正常工作

1: 主定时器计数器时钟停止, 计数器复位

Bit 15:11 保留, 必须保持复位值。

Bit 10 **BMPREN**: 突发模式预装载使能 (Burst mode preload enable)

此位可使能寄存器预装载机制, 并定义对可预装载寄存器 (HRTIM\_BMCMR、HRTIM\_BMPER) 的写访问是在活动寄存器中进行还是在预装载寄存器中进行。

0: 禁止预装载: 直接对活动寄存器进行写访问

1: 使能预装载: 对预装载寄存器进行写访问

**Bit 9:6 BMPRSC[3:0]: 突发模式预分频器 (Burst mode prescaler)**

定义突发模式控制器的  $f_{HRTIM}$  时钟预分频比。突发模式使能后, 不能更改此位域。

0000: 时钟不分频

0001: 2 分频

0010: 4 分频

0011: 8 分频

0100: 16 分频

0101: 32 分频

0110: 64 分频

0111: 128 分频

1000: 256 分频

1001: 512 分频

1010: 1024 分频

1011: 2048 分频

1100: 4096 分频

1101: 8192 分频

1110: 16384 分频

1111: 32768 分频

**Bit 5:2 BMCLK[3:0]: 突发模式时钟源 (Burst mode clock source)**

此位域定义突发模式计数器的时钟源。当突发模式使能时, 它无法更改。

0000: 主定时器计数器复位/翻转

0001: 定时器 A 计数器复位/翻转

0010: 定时器 B 计数器复位/翻转

0011: 定时器 C 计数器复位/翻转

0100: 定时器 D 计数器复位/翻转

0101: 定时器 E 计数器复位/翻转

0110: 片上事件 1 (hrtim\_bm\_ck1) 作为突发模式计数器时钟

0111: 片上事件 2 (hrtim\_bm\_ck2) 作为突发模式计数器时钟

1000: 片上事件 3 (hrtim\_bm\_ck3) 作为突发模式计数器时钟

1001: 片上事件 4 (hrtim\_bm\_ck4) 作为突发模式计数器时钟

1010: 预分频  $f_{HRTIM}$  时钟 (根据 BMPRSC[3:0] 设置)

1011: 定时器 F 计数器复位/翻转

其他: 保留

**Bit 1 BMOM: 突发模式工作模式 (Burst mode operating mode)**

此位定义进入一次突发模式还是连续工作。

0: 单发模式

1: 连续工作

**Bit 0 BME: 突发模式使能 (Burst mode enable)**

此位用于启动已准备好接收启动触发信号的突发模式控制器。

向此位写入 0 会使突发模式提前终止。

0: 突发模式禁用

1: 突发模式使能

## 22.5.60 HRTIM 突发模式触发寄存器 (HRTIM\_BMTRGR)

偏移地址: 0x3A4

复位值: 0x0000 0000

| 31     | 30   | 29   | 28     | 27    | 26     | 25     | 24    | 23     | 22   | 21   | 20   | 19   | 18  | 17  | 16 |
|--------|------|------|--------|-------|--------|--------|-------|--------|------|------|------|------|-----|-----|----|
| OCHPEV | EEV8 | EEV7 | TDEEV8 | TAEV7 | TECMP2 | TECMP1 | TEREP | TFCMP1 | TD   | TF   | TD   | TD   | TF  | TC  | TC |
| rw     | rw   | rw   | rw     | rw    | rw     | rw     | rw    | rw     | rw   | rw   | rw   | rw   | rw  | rw  | rw |
| 15     | 14   | 13   | 12     | 11    | 10     | 9      | 8     | 7      | 6    | 5    | 4    | 3    | 2   | 1   | 0  |
| TC     | TB   | TB   | TB     | TB    | TA     | TA     | TA    | TA     | MST  | MST  | MST  | MST  | MST | MST | SW |
| RST    | CMP2 | CMP1 | REP    | RST   | CMP2   | CMP1   | REP   | RST    | CMP4 | CMP3 | CMP2 | CMP1 | REP | RST |    |
| rw     | rw   | rw   | rw     | rw    | rw     | rw     | rw    | rw     | rw   | rw   | rw   | rw   | rw  | rw  | rw |

Bit 31 **OCHPEV**: 片上事件 (On-chip event)

当 hrtim\_bm\_trg 输入 (连接到通用定时器 TRGO 输出) 上发生上升沿时, 将触发突发模式进入。

Bit 30 **EEV8**: 外部事件 8 (External Event 8) (应用 TIMD 过滤器)

通过 TIMD 过滤器调节的外部事件 8 会启动突发模式工作。

Bit 29 **EEV7**: 外部事件 7 (External Event 7) (应用 TIMA 过滤器)

通过 TIMA 过滤器调节的外部事件 7 会启动突发模式工作。

Bit 28 **TDEEV8**: 外部事件 8 后的定时器 D 周期 (Timer D period following External Event 8)

外部事件 8 (通过 TIMD 过滤器调节) 后的定时器 D 周期会启动突发模式工作。

Bit 27 **TAEV7**: 外部事件 7 后的定时器 A 周期 (Timer A period following External Event 7)

外部事件 7 (通过 TIMA 过滤器调节) 后的定时器 A 周期会启动突发模式工作。

Bit 26 **TECMP2**: 定时器 E 比较 2 事件 (Timer E compare 2 event)

参考 TACMP1 的描述。

Bit 25 **TECMP1**: 定时器 E 比较 1 事件 (Timer E compare 1 event)

参考 TACMP1 的描述。

Bit 24 **TEREP**: 定时器 E 重复 (Timer E repetition)

参考 TAREP 的描述。

Bit 23 **TFCMP1**: 定时器 F 比较 1 事件 (Timer F compare 1 event)

参考 TACMP1 的描述。

Bit 22 **TD**: 定时器 D 比较 2 事件 (Timer D compare 2 event)

参考 TACMP1 的描述。

Bit 21 **TFREP**: 定时器 F 重复 (Timer F repetition)

参考 TAREP 的描述。

Bit 20 **TDREP**: 定时器 D 重复 (Timer D repetition)

参考 TAREP 的描述。

Bit 19 **TDRST**: 定时器 D 复位或翻转 (Timer D reset or roll-over)

参考 TARST 的描述。

Bit 18 **TFRST**: 定时器 F 复位 (Timer F reset)

参考 TARST 的描述。



Bit 17 **TCCMP1**: 定时器 C 比较 1 事件 (Timer C compare 1 event)

参考 TACMP1 的描述。

Bit 16 **TCREP**: 定时器 C 重复 (Timer C repetition)

参考 TAREP 的描述。

Bit 15 **TCRST**: 定时器 C 复位或翻转 (Timer C reset or roll-over)

参考 TARST 的描述。

Bit 14 **TBCMP2**: 定时器 B 比较 2 事件 (Timer B compare 2 event)

参考 TACMP1 的描述。

Bit 13 **TBCMP1**: 定时器 B 比较 1 事件 (Timer B compare 1 event)

参考 TACMP1 的描述。

Bit 12 **TBREP**: 定时器 B 重复 (Timer B repetition)

参考 TAREP 的描述。

Bit 11 **TBRST**: 定时器 B 复位或翻转 (Timer B reset or roll-over)

参考 TARST 的描述。

Bit 10 **TACMP2**: 定时器 A 比较 2 事件 (Timer A compare 2 event)

参考 TACMP1 的描述。

Bit 9 **TACMP1**: 定时器 A 比较 1 事件 (Timer A compare 1 event)

定时器 A 的比较 1 事件将启动突发模式操作。

Bit 8 **TAREP**: 定时器 A 重复 (Timer A repetition)

定时器 A 的重复事件将启动突发模式操作。

Bit 7 **TARST**: 定时器 A 复位或翻转 (Timer A reset or roll-over)

定时器 A 的复位或翻转事件将启动突发模式操作。

Bit 6 **MSTCMP4**: 主定时器比较 4 (Master compare 4)

参考 MSTCMP1 的描述。

Bit 5 **MSTCMP3**: 主定时器比较 3 (Master compare 3)

参考 MSTCMP1 的描述。

Bit 4 **MSTCMP2**: 主定时器比较 2 (Master compare 2)

参考 MSTCMP1 的描述。

Bit 3 **MSTCMP1**: 主定时器比较 1 (Master compare 1)

主定时器的比较 1 事件将启动突发模式工作。

Bit 2 **MSTREP**: 主定时器重复 (Master repetition)

主定时器重复事件将启动突发模式工作。

Bit 1 **MSTRST**: 主定时器复位或翻转 (Master reset or roll-over)

主定时器复位和翻转事件将启动突发模式工作。

Bit 0 **SW**: 软件启动 (Software start)

此位由软件置 1, 并由硬件自动复位。

如果此位置 1, 则会立即启动突发模式工作。

如果突发模式未使能 (BME 位复位), 此位无效。



## 22.5.61 HRTIM 突发模式比较寄存器 (HRTIM\_BMCMR)

偏移地址: 0x3A8

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| BMCMR[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:16 保留, 必须保持复位值。

Bit 15:0 **BMCMR[15:0]**: 突发模式比较值 (Burst mode compare value)

这个位域定义了所选定时器处于空闲状态的周期数。

该寄存器保留预装载寄存器的内容, 如果禁止预装载, 则会保留活动寄存器的内容。

注: 如果使用未进行预分频的  $f_{HRTIM}$  时钟作为突发模式时钟源 (BMCLK[3:0] = 1010 且 BMPRESC[3:0] = 0000) 时, BMCMR[15:0] 不能设置为 0x0000。

## 22.5.62 HRTIM 突发模式周期寄存器 (HRTIM\_BMPER)

偏移地址: 0x3AC

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| BMPER[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:16 保留, 必须保持复位值。

Bit 15:0 **BMPER[15:0]**: 突发模式周期 (Burst mode period)

定义突发模式的重复周期。

如果禁止预装载, 则该寄存器保存预装载寄存器的内容或活动寄存器的内容。

注: 当突发模式被使能时, BMPER[15:0] 的值不能为零。

### 22.5.63 HRTIM 定时器外部事件控制寄存器 1 (HRTIM\_EECR1)

偏移地址: 0x3B0

复位值: 0x0000 0000

| 31             | 30 | 29          | 28 | 27          | 26 | 25          | 24 | 23          | 22 | 21          | 20 | 19          | 18 | 17          | 16 |             |    |             |    |                |  |
|----------------|----|-------------|----|-------------|----|-------------|----|-------------|----|-------------|----|-------------|----|-------------|----|-------------|----|-------------|----|----------------|--|
| Reserved       |    | EE5FA<br>ST |    | EE5SNS[1:0] |    | EE5POL      |    | EE5SRC[1:0] |    | EE4FA<br>ST |    | EE4SNS[1:0] |    | EE4POL      |    | EE4SRC[1:0] |    | EE3FA<br>ST |    | EE3SNS<br>S[1] |  |
|                |    | rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw             |  |
| 15             | 14 | 13          | 12 | 11          | 10 | 9           | 8  | 7           | 6  | 5           | 4  | 3           | 2  | 1           | 0  |             |    |             |    |                |  |
| EE3SNS<br>S[0] |    | EE3POL      |    | EE3SRC[1:0] |    | EE2FA<br>ST |    | EE2SNS[1:0] |    | EE2POL      |    | EE2SRC[1:0] |    | EE1FA<br>ST |    | EE1SNS[1:0] |    | EE1POL      |    | EE1SRC[1:0]    |  |
| rw             | rw | rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw             |  |

Bit 31:30 保留, 必须保持复位值。

Bit 29 **EE5FAST**: 外部事件 5 快速模式 (External event 5 fast mode)

参考 EE1FAST 的描述。

Bit 28:27 **EE5SNS[1:0]**: 外部事件 5 有效性 (External event 5 sensitivity)

参考 EE1SNS[1:0] 的描述。

Bit 26 **EE5POL**: 外部事件 5 极性 (External event 5 polarity)

参考 EE1POL 的描述。

Bit 25:24 **EE5SRC[1:0]**: 外部事件 5 源 (External event 5 source)

参考 EE1SRC[1:0] 的描述。

Bit 23 **EE4FAST**: 外部事件 4 快速模式 (External event 4 fast mode)

参考 EE1FAST 的描述。

Bit 22:21 **EE4SNS[1:0]**: 外部事件 4 有效性 (External event 4 sensitivity)

参考 EE1SNS[1:0] 的描述。

Bit 20 **EE4POL**: 外部事件 4 极性 (External event 4 polarity)

参考 EE1POL 的描述。

Bit 19:18 **EE4SRC[1:0]**: 外部事件 4 源 (External event 4 source)

参考 EE1SRC[1:0] 的描述。

Bit 17 **EE3FAST**: 外部事件 3 快速模式 (External event 3 fast mode)

参考 EE1FAST 的描述。

Bit 16:15 **EE3SNS[1:0]**: 外部事件 3 有效性 (External event 3 sensitivity)

参考 EE1SNS[1:0] 的描述。

Bit 14 **EE3POL**: 外部事件 3 极性 (External event 3 polarity)

参考 EE1POL 的描述。

Bit 13:12 **EE3SRC[1:0]**: 外部事件 3 源 (External event 3 source)

参考 EE1SRC[1:0] 的描述。

Bit 11 **EE2FAST**: 外部事件 2 快速模式 (External event 2 fast mode)

参考 EE1FAST 的描述。

Bit 10:9 **EE2SNS[1:0]**: 外部事件 2 有效性 (External event 2 sensitivity)

参考 EE1SNS[1:0] 的描述。

Bit 8 **EE2POL**: 外部事件 2 极性 (External event 2 polarity)

参考 EE1POL 的描述。

Bits 7:6 **EE2SRC[1:0]**: 外部事件 2 源 (External event 2 source)

参考 EE1SRC[1:0] 的描述。

Bit 5 **EE1FAST**: 外部事件 1 快速模式 (External event 1 fast mode)

0: 外部事件 1 会在作用于输出之前通过 HRTIM 逻辑进行重新同步, 以增加  $f_{HRTIM}$  时钟相关延迟

1: 外部事件异步作用于输出上 (低延迟模式)

注: 使用了事件的计数器使能后 (TxCEN 位置 1), 不得修改此位。

Bits 4:3 **EE1SNS[1:0]**: 外部事件 1 有效性 (External event 1 sensitivity)

00: 由 EE1POL 位定义有效电平

01: 上升沿, 与 EE1POL 位值无关

10: 下降沿, 与 EE1POL 位值无关

11: 上升沿和下降沿, 与 EE1POL 位值无关

Bit 2 **EE1POL**: 外部事件 1 极性 (External event 1 polarity)

仅当 EE1SNS[1:0] = 00 时, 此位才有效。

0: 外部事件在高电平时有效

1: 外部事件在低电平时有效

注: 一旦定时器 x 被使能, 此参数就不能更改。它必须在设置 EE1FAST 位之前配置。

Bits 1:0 **EE1SRC[1:0]**: 外部事件 1 源 (External event 1 source)

此位域用于选择外部事件 1 的源。。

00: hrtim\_eev1\_1

01: hrtim\_eev1\_2

10: hrtim\_eev1\_3

11: hrtim\_eev1\_4

注: 一旦定时器 x 被使能, 此参数就不能更改。它必须在设置 EE1FAST 位之前配置。

## 22.5.64 HRTIM 定时器外部事件控制寄存器 2 (HRTIM\_EECR2)

偏移地址：0x3B4

复位值：0x0000 0000

| 31        | 30     | 29          | 28           | 27  | 26          | 25           | 24     | 23          | 22          | 21  | 20          | 19          | 18     | 17          | 16        |
|-----------|--------|-------------|--------------|-----|-------------|--------------|--------|-------------|-------------|-----|-------------|-------------|--------|-------------|-----------|
| Reserved  |        |             | EE10SNS[1:0] |     | EE10POL     | EE10SRC[1:0] |        | Res         | EE9SNS[1:0] |     | EE9POL      | EE9SRC[1:0] |        | Res         | EE8SNS[1] |
|           |        |             | rw           | rw  | rw          | rw           | rw     |             | rw          | rw  | rw          | rw          | rw     |             | rw        |
| 15        | 14     | 13          | 12           | 11  | 10          | 9            | 8      | 7           | 6           | 5   | 4           | 3           | 2      | 1           | 0         |
| EE8SNS[0] | EE8POL | EE8SRC[1:0] |              | Res | EE7SNS[1:0] |              | EE7POL | EE7SRC[1:0] |             | Res | EE6SNS[1:0] |             | EE6POL | EE6SRC[1:0] |           |
| rw        | rw     | rw          | rw           |     | rw          | rw           | rw     | rw          | rw          |     | rw          | rw          | rw     | rw          | rw        |

Bit 31:29 保留，必须保持复位值。

Bit 28:27 **EE10SNS[1:0]**：外部事件 10 有效性 (External event 10 sensitivity)

参考 EE1SNS[1:0]的描述。

Bit 26 **EE10POL**：外部事件 10 极性 (External event 10 polarity)

参考 EE1POL 的描述。

Bit 25:24 **EE10SRC[1:0]**：外部事件 10 源 (External event 10 source)

参考 EE1SRC[1:0]的描述。

Bit 23 保留，必须保持复位值。

Bit 22:21 **EE9SNS[1:0]**：外部事件 9 有效性 (External event 9 sensitivity)

参考 EE1SNS[1:0]的描述。

Bit 20 **EE9POL**：外部事件 9 极性 (External event 9 polarity)

参考 EE1POL 的描述。

Bit 19:18 **EE9SRC[1:0]**：外部事件 9 源 (External event 9 source)

参考 EE1SRC[1:0]的描述。

Bit 17 保留，必须保持复位值。

Bit 16:15 **EE8SNS[1:0]**：外部事件 8 有效性 (External event 8 sensitivity)

参考 EE1SNS[1:0]的描述。

Bit 14 **EE8POL**：外部事件 8 极性 (External event 8 polarity)

参考 EE1POL 的描述。

Bit 13:12 **EE8SRC[1:0]**：外部事件 8 源 (External event 8 source)

参考 EE1SRC[1:0]的描述。

Bit 11 保留，必须保持复位值。

Bit 10:9 **EE7SNS[1:0]**：外部事件 7 有效性 (External event 7 sensitivity)

参考 EE1SNS[1:0]的描述。

Bit 8 **EE7POL**：外部事件 7 极性 (External event 7 polarity)

参考 EE1POL 的描述。

Bit 7:6 **EE7SRC[1:0]**：外部事件 7 源 (External event 7 source)

参考 EE1SRC[1:0]的描述。

Bit 5 保留，必须保持复位值。

Bit 4:3 **EE6SNS[1:0]**: 外部事件 6 有效性 (External event 6 sensitivity)

请参考 EE1SNS[1:0] 的描述。

Bit 2 **EE6POL**: 外部事件 6 极性 (External event 6 polarity)

请参考 EE1POL 的描述。

Bit 1:0 **EE6SRC[1:0]**: 外部事件 6 源 (External event 6 source)

请参考 EE1SRC[1:0] 的描述。

## 22.5.65 HRTIM 定时器外部事件控制寄存器 3 (HRTIM\_EECR3)

偏移地址: 0x3B8

复位值: 0x0000 0000

|            |    |          |    |            |    |           |    |          |    |           |    |           |    |          |    |
|------------|----|----------|----|------------|----|-----------|----|----------|----|-----------|----|-----------|----|----------|----|
| 31         | 30 | 29       | 28 | 27         | 26 | 25        | 24 | 23       | 22 | 21        | 20 | 19        | 18 | 17       | 16 |
| EEVSD[1:0] |    | Reserved |    | EE10F[3:0] |    |           |    | Reserved |    | EE9F[3:0] |    |           |    | Reserved |    |
| rw         | rw |          |    | rw         | rw | rw        | rw |          |    | rw        | rw | rw        | rw |          |    |
| 15         | 14 | 13       | 12 | 11         | 10 | 9         | 8  | 7        | 6  | 5         | 4  | 3         | 2  | 1        | 0  |
| EE8F[3:0]  |    |          |    | Reserved   |    | EE7F[3:0] |    |          |    | Reserved  |    | EE6F[3:0] |    |          |    |
| rw         | rw | rw       | rw |            |    | rw        | rw | rw       | rw |           |    | rw        | rw | rw       | rw |

Bit 31:30 **EEVSD[1:0]**: 外部事件采样时钟分频比 (External event sampling clock division)

此位域指示定时器时钟频率 ( $f_{HRTIM}$ ) 与数字滤波器使用的外部事件信号采样时钟 ( $f_{EEVS}$ ) 之间的分频比。

00:  $f_{EEVS} = f_{HRTIM}$

01:  $f_{EEVS} = f_{HRTIM} / 2$

10:  $f_{EEVS} = f_{HRTIM} / 4$

11:  $f_{EEVS} = f_{HRTIM} / 8$

Bit 29:28 保留, 必须保持复位值。

Bit 27:24 **EE10F[3:0]**: 外部事件 10 过滤器 (External event 10 filter)

请参考 EE6F[3:0] 的描述。

Bit 23:22 保留, 必须保持复位值。

Bit 21:18 **EE9F[3:0]**: 外部事件 9 过滤器 (External event 9 filter)

请参考 EE6F[3:0] 的描述。

Bit 17:16 保留, 必须保持复位值。

Bit 15:12 **EE8F[3:0]**: 外部事件 8 过滤器 (External event 8 filter)

请参考 EE6F[3:0] 的描述。

Bit 11:10 保留, 必须保持复位值。

Bit 9:6 **EE7F[3:0]**: 外部事件 7 过滤器 (External event 7 filter)

参考 EE6F[3:0]的描述。

Bit 4:5 保留, 必须保持复位值。

Bit 3:0 **EE6F[3:0]**: 外部事件 6 过滤器 (External event 6 filter)

这个位域定义了用于采样外部事件 6 输入的频率以及应用于 EEV6 的数字滤波器的长度。数字滤波器由计数器组成, 需要使用 N 个有效样本来验证输出跳变。

0000: 滤波器禁用

0001:  $f_{\text{SAMPLING}} = f_{\text{HRTIM}}$ ,  $N = 2$

0010:  $f_{\text{SAMPLING}} = f_{\text{HRTIM}}$ ,  $N = 4$

0011:  $f_{\text{SAMPLING}} = f_{\text{HRTIM}}$ ,  $N = 8$

0100:  $f_{\text{SAMPLING}} = f_{\text{EEVS}} / 2$ ,  $N = 6$

0101:  $f_{\text{SAMPLING}} = f_{\text{EEVS}} / 2$ ,  $N = 8$

0110:  $f_{\text{SAMPLING}} = f_{\text{EEVS}} / 4$ ,  $N = 6$

0111:  $f_{\text{SAMPLING}} = f_{\text{EEVS}} / 4$ ,  $N = 8$

1000:  $f_{\text{SAMPLING}} = f_{\text{EEVS}} / 8$ ,  $N = 6$

1001:  $f_{\text{SAMPLING}} = f_{\text{EEVS}} / 8$ ,  $N = 8$

1010:  $f_{\text{SAMPLING}} = f_{\text{EEVS}} / 16$ ,  $N = 5$

1011:  $f_{\text{SAMPLING}} = f_{\text{EEVS}} / 16$ ,  $N = 6$

1100:  $f_{\text{SAMPLING}} = f_{\text{EEVS}} / 16$ ,  $N = 8$

1101:  $f_{\text{SAMPLING}} = f_{\text{EEVS}} / 32$ ,  $N = 5$

1110:  $f_{\text{SAMPLING}} = f_{\text{EEVS}} / 32$ ,  $N = 6$

1111:  $f_{\text{SAMPLING}} = f_{\text{EEVS}} / 32$ ,  $N = 8$

## 22.5.66 HRTIM ADC 触发 1 寄存器 (HRTIM\_ADC1R)

偏移地址: 0x3BC

复位值: 0x0000 0000

| 31            | 30            | 29            | 28            | 27            | 26           | 25           | 24            | 23            | 22           | 21           | 20           | 19            | 18            | 17           | 16           |
|---------------|---------------|---------------|---------------|---------------|--------------|--------------|---------------|---------------|--------------|--------------|--------------|---------------|---------------|--------------|--------------|
| ADC1<br>TEPER | ADC1<br>TEC4  | ADC1<br>TEC3  | ADC1<br>TFRST | ADC1<br>TDPER | ADC1<br>TDC4 | ADC1<br>TDC3 | ADC1<br>TFPER | ADC1<br>TCPER | ADC1<br>TCC4 | ADC1<br>TCC3 | ADC1<br>TFC4 | ADC1<br>TBRST | ADC1<br>TBPER | ADC1<br>TBC4 | ADC1<br>TBC3 |
| rw            | rw            | rw            | rw            | rw            | rw           | rw           | rw            | rw            | rw           | rw           | rw           | rw            | rw            | rw           | rw           |
| 15            | 14            | 13            | 12            | 11            | 10           | 9            | 8             | 7             | 6            | 5            | 4            | 3             | 2             | 1            | 0            |
| ADC1<br>TFC3  | ADC1<br>TARST | ADC1<br>TAPER | ADC1<br>TAC4  | ADC1<br>TAC3  | ADC1<br>TFC2 | ADC1<br>EEV5 | ADC1<br>EEV4  | ADC1<br>EEV3  | ADC1<br>EEV2 | ADC1<br>EEV1 | ADC1<br>MPER | ADC1<br>MC4   | ADC1<br>MC3   | ADC1<br>MC2  | ADC1<br>MC1  |
| rw            | rw            | rw            | rw            | rw            | rw           | rw           | rw            | rw            | rw           | rw           | rw           | rw            | rw            | rw           | rw           |

Bit 31:0 参考 HRTIM\_ADC3R 位的描述。

## 22.5.67 HRTIM ADC 触发 2 寄存器 (HRTIM\_ADC2R)

偏移地址: 0x3C0

复位值: 0x0000 0000

| 31            | 30           | 29            | 28           | 27            | 26            | 25            | 24            | 23           | 22            | 21            | 20           | 19           | 18           | 17            | 16           |
|---------------|--------------|---------------|--------------|---------------|---------------|---------------|---------------|--------------|---------------|---------------|--------------|--------------|--------------|---------------|--------------|
| ADC2<br>TERST | ADC2<br>TEC4 | ADC2<br>TEC3  | ADC2<br>TEC2 | ADC2<br>TDRST | ADC2<br>TDPER | ADC2<br>TDC4  | ADC2<br>TFPER | ADC2<br>TDC2 | ADC2<br>TCPER | ADC2<br>TCPER | ADC2<br>TCC4 | ADC2<br>TFC4 | ADC2<br>TCC2 | ADC2<br>TBPER | ADC2<br>TBC4 |
| rw            | rw           | rw            | rw           | rw            | rw            | rw            | rw            | rw           | rw            | rw            | rw           | rw           | rw           | rw            | rw           |
| 15            | 14           | 13            | 12           | 11            | 10            | 9             | 8             | 7            | 6             | 5             | 4            | 3            | 2            | 1             | 0            |
| ADC2<br>TFC3  | ADC2<br>TBC2 | ADC2<br>TAPER | ADC2<br>TAC4 | ADC2<br>TFC2  | ADC2<br>TAC2  | ADC2<br>EEV10 | ADC2<br>EEV9  | ADC2<br>EEV8 | ADC2<br>EEV7  | ADC2<br>EEV6  | ADC2<br>MPER | ADC2<br>MC4  | ADC2<br>MC3  | ADC2<br>MC2   | ADC2<br>MC1  |
| rw            | rw           | rw            | rw           | rw            | rw            | rw            | rw            | rw           | rw            | rw            | rw           | rw           | rw           | rw            | rw           |

Bit 31:0 参考 HRTIM\_ADC4R 位的描述。



## 22.5.68 HRTIM ADC 触发 3 寄存器 (HRTIM\_ADC3R)

偏移地址: 0x3C4

复位值: 0x0000 0000

| 31            | 30            | 29            | 28            | 27            | 26           | 25           | 24            | 23            | 22           | 21           | 20           | 19            | 18            | 17           | 16           |
|---------------|---------------|---------------|---------------|---------------|--------------|--------------|---------------|---------------|--------------|--------------|--------------|---------------|---------------|--------------|--------------|
| ADC3<br>TEPER | ADC3<br>TEC4  | ADC3<br>TEC3  | ADC3<br>TFRST | ADC3<br>TDPER | ADC3<br>TDC4 | ADC3<br>TDC3 | ADC3<br>TFPER | ADC3<br>TCPER | ADC3<br>TCC4 | ADC3<br>TCC3 | ADC3<br>TFC4 | ADC3<br>TBRST | ADC3<br>TBPER | ADC3<br>TBC4 | ADC3<br>TBC3 |
| rw            | rw            | rw            | rw            | rw            | rw           | rw           | rw            | rw            | rw           | rw           | rw           | rw            | rw            | rw           | rw           |
| 15            | 14            | 13            | 12            | 11            | 10           | 9            | 8             | 7             | 6            | 5            | 4            | 3             | 2             | 1            | 0            |
| ADC3<br>TFC3  | ADC3<br>TARST | ADC3<br>TAPER | ADC3<br>TAC4  | ADC3<br>TAC3  | ADC3<br>TFC2 | ADC3<br>EEV5 | ADC3<br>EEV4  | ADC3<br>EEV3  | ADC3<br>EEV2 | ADC3<br>EEV1 | ADC3<br>MPER | ADC3<br>MC4   | ADC3<br>MC3   | ADC3<br>MC2  | ADC3<br>MC1  |
| rw            | rw            | rw            | rw            | rw            | rw           | rw           | rw            | rw            | rw           | rw           | rw           | rw            | rw            | rw           | rw           |

Bit 31 **ADC3TEPER**: 定时器 E 周期时的 ADC 触发 3

参考 ADC3TAPER 的描述。

Bit 30 **ADC3TEC4**: 定时器 E 比较 4 时的 ADC 触发 3

参考 ADC3TFC2 的描述。

Bit 29 **ADC3TEC3**: 定时器 E 比较 3 时的 ADC 触发 3

参考 ADC3TFC2 的描述。

Bit 28 **ADC3TFRST**: 定时器 F 重置和计数器翻转时的 ADC 触发 3 <sup>(1)</sup>

参考 ADC3TARST 的描述。

Bit 27 **ADC3TDPER**: 定时器 D 周期时的 ADC 触发 3

参考 ADC3TAPER 的描述。

Bit 26 **ADC3TDC4**: 定时器 D 比较 4 时的 ADC 触发 3

参考 ADC3TFC2 的描述。

Bit 25 **ADC3TDC3**: 定时器 D 比较 3 时的 ADC 触发 3

参考 ADC3TFC2 的描述。

Bit 24 **ADC3TFPER**: 定时器 F 周期时的 ADC 触发 3

参考 ADC3TAPER 的描述。

Bit 23 **ADC3TCPER**: 定时器 C 周期时的 ADC 触发 3

参考 ADC3TAPER 的描述。

Bit 22 **ADC3TCC4**: 定时器 C 比较 4 时的 ADC 触发 3

参考 ADC3TFC2 的描述。

Bit 21 **ADC3TCC3**: 定时器 C 比较 3 时的 ADC 触发 3

参考 ADC3TFC2 的描述。

Bit 20 **ADC3TFC4**: 定时器 F 比较 4 时的 ADC 触发 3

参考 ADC3TAC2 的描述。

Bit 19 **ADC3TBRST**: 定时器 B 复位和计数器翻转时的 ADC 触发 3 <sup>(1)</sup>

此位可在发生定时器 B 复位和翻转事件时在 ADC 触发 1 输出上生成 ADC 触发信号。

Bit 18 **ADC3TBPER**: 定时器 B 周期时的 ADC 触发 3

参考 ADC3TAPER 的描述。

Bit 17 **ADC3TBC4**: 定时器 B 比较 4 时的 ADC 触发 3

参考 ADC3TFC2 的描述。

Bit 16 **ADC3TBC3**: 定时器 B 比较 3 时的 ADC 触发 3

参考 ADC3TFC2 的描述。

Bit 15 **ADC3TFC3**: 定时器 B 比较 2 时的 ADC 触发 3

参考 ADC3TFC2 的描述。

Bit 14 **ADC3TARST**: 定时器 A 复位和计数器翻转时的 ADC 触发 3

此位可在发生定时器 A 复位和翻转事件时在 ADC 触发 1 输出上生成 ADC 触发信号。

Bit 13 **ADC3TAPER**: 定时器 A 周期时的 ADC 触发 3

此位可在发生定时器 A 周期事件时在 ADC 触发 3 输出上生成 ADC 触发信号。

Bit 12 **ADC3TAC4**: 定时器 A 比较 4 时的 ADC 触发 3

参考 ADC3TFC2 的描述。

Bit 11 **ADC3TAC3**: 定时器 A 比较 3 时的 ADC 触发 3

参考 ADC3TFC2 的描述。

Bit 10 **ADC3TFC2**: 定时器 F 比较 2 时的 ADC 触发 3

此位可在发生定时器 F 比较 2 事件时在 ADC 触发 3 输出上生成 ADC 触发信号。

Bit 9 **ADC3EEV5**: 外部事件 5 时的 ADC 触发 3 <sup>(1)</sup>

参考 ADC3EEV1 的描述。

Bit 8 **ADC3EEV4**: 外部事件 4 时的 ADC 触发 3 <sup>(1)</sup>

参考 ADC3EEV1 的描述。

Bit 7 **ADC3EEV3**: 外部事件 3 时的 ADC 触发 3 <sup>(1)</sup>

参考 ADC3EEV1 的描述。

Bit 6 **ADC3EEV2**: 外部事件 2 时的 ADC 触发 3 <sup>(1)</sup>

参考 ADC3EEV1 的描述。

Bit 5 **ADC3EEV1**: 外部事件 1 时的 ADC 触发 3 <sup>(1)</sup>

此位可在发生外部事件 1 时在 ADC 触发 3 输出上生成 ADC 触发信号。

Bit 4 **ADC3MPER**: 主定时器周期时的 ADC 触发 3

此位可在发生主定时器周期事件时在 ADC 触发 3 输出上生成 ADC 触发信号。

Bit 3 **ADC3MC4**: 主定时器比较 4 时的 ADC 触发 3

参考 ADC3MC1 的描述。

Bit 2 **ADC3MC3**: 主定时器比较 3 时的 ADC 触发 3

参考 ADC3MC1 的描述。

Bit 1 **ADC3MC2**: 主定时器比较 2 时的 ADC 触发 3

参考 ADC3MC1 的描述。

Bit 0 **ADC3MC1**: 主定时器比较 1 时的 ADC 触发 3

此位可在发生主定时器比较 1 事件时在 ADC 触发 3 输出上生成 ADC 触发信号。

1. 这些触发与 HRTIM\_ADC2R/HRTIM\_ADC4R 和 HRTIM\_ADC1R/HRTIM\_ADC3R 是不同的。

## 22.5.69 HRTIM ADC 触发 4 寄存器 (HRTIM\_ADC4R)

偏移地址: 0x3C8

复位值: 0x0000 0000

| 31            | 30           | 29            | 28           | 27            | 26            | 25            | 24            | 23           | 22            | 21            | 20           | 19           | 18           | 17            | 16           |
|---------------|--------------|---------------|--------------|---------------|---------------|---------------|---------------|--------------|---------------|---------------|--------------|--------------|--------------|---------------|--------------|
| ADC4<br>TERST | ADC4<br>TEC4 | ADC4<br>TEC3  | ADC4<br>TEC2 | ADC4<br>TDRST | ADC4<br>TDPER | ADC4<br>TDC4  | ADC4<br>TFPER | ADC4<br>TDC2 | ADC4<br>TCPER | ADC4<br>TCPER | ADC4<br>TCC4 | ADC4<br>TFC4 | ADC4<br>TCC2 | ADC4<br>TBPER | ADC4<br>TBC4 |
| rw            | rw           | rw            | rw           | rw            | rw            | rw            | rw            | rw           | rw            | rw            | rw           | rw           | rw           | rw            | rw           |
| 15            | 14           | 13            | 12           | 11            | 10            | 9             | 8             | 7            | 6             | 5             | 4            | 3            | 2            | 1             | 0            |
| ADC4<br>TFC3  | ADC4<br>TBC2 | ADC4<br>TAPER | ADC4<br>TAC4 | ADC4<br>TFC2  | ADC4<br>TAC2  | ADC4<br>EEV10 | ADC4<br>EEV9  | ADC4<br>EEV8 | ADC4<br>EEV7  | ADC4<br>EEV6  | ADC4<br>MPER | ADC4<br>MC4  | ADC4<br>MC3  | ADC4<br>MC2   | ADC4<br>MC1  |
| rw            | rw           | rw            | rw           | rw            | rw            | rw            | rw            | rw           | rw            | rw            | rw           | rw           | rw           | rw            | rw           |

Bit 31 **ADC4TERST**: 定时器 E 复位和计数器翻转时的 ADC 触发 4 <sup>(1)</sup>

参考 ADC4TCRST 的描述。

Bit 30 **ADC4TEC4**: 定时器 E 比较 4 时的 ADC 触发 4

参考 ADC4TAC2 的描述。

Bit 29 **ADC4TEC3**: 定时器 E 比较 3 时的 ADC 触发 4

参考 ADC4TAC2 的描述。

Bit 28 **ADC4TEC2**: 定时器 E 比较 2 时的 ADC 触发 4

参考 ADC4TAC2 的描述。

Bit 27 **ADC4TDRST**: 定时器 D 复位和计数器翻转时的 ADC 触发 4 <sup>(1)</sup>

参考 ADC4TCRST 的描述。

Bit 26 **ADC4TDPER**: 定时器 D 周期时的 ADC 触发 4

参考 ADC4TAPER 的描述。

Bit 25 **ADC4TDC4**: 定时器 D 比较 4 时的 ADC 触发 4

参考 ADC4TAC2 的描述。

Bit 24 **ADC4TFPER**: 定时器 F 周期时的 ADC 触发 4

参考 ADC4TAPER 的描述。

Bit 23 **ADC4TDC2**: 定时器 D 比较 2 时的 ADC 触发 4

参考 ADC4TAC2 的描述。

Bit 22 **ADC4TCRST**: 定时器 C 复位和计数器翻转时的 ADC 触发 4 <sup>(1)</sup>

此位可在发生定时器 C 复位和翻转事件时在 ADC 触发 4 输出上生成 ADC 触发信号。

Bit 21 **ADC4TCPER**: 定时器 C 周期时的 ADC 触发 4

请参见 ADC4TAPER 说明

Bit 20 **ADC4TCC4**: 定时器 C 比较 4 时的 ADC 触发 4

参考 ADC4TAC2 的描述。

Bit 19 **ADC4TFC4**: 定时器 F 比较 3 时的 ADC 触发 4

参考 ADC4TAC2 的描述。

Bit 18 **ADC4TCC2**: 定时器 C 比较 2 时的 ADC 触发 4

参考 ADC4TAC2 的描述。

Bit 17 **ADC4TBPER**: 定时器 B 周期时的 ADC 触发 4

参考 ADC4TAPER 的描述。

Bit 16 **ADC4TBC4**: 定时器 B 比较 4 时的 ADC 触发 4

参考 ADC4TAC2 的描述。

Bit 15 **ADC4TFC3**: 定时器 F 比较 3 时的 ADC 触发 4

参考 ADC4TAC2 的描述。

Bit 14 **ADC4TBC2**: 定时器 B 比较 2 时的 ADC 触发 4

参考 ADC4TAC2 的描述。

Bit 13 **ADC4TAPER**: 定时器 A 周期时的 ADC 触发 4

此位可在发生定时器 A 事件时在 ADC 触发 4 输出上生成 ADC 触发信号。

Bit 12 **ADC4TAC4**: 定时器 A 比较 4 时的 ADC 触发 4

参考 ADC4TAC2 的描述。

Bit 11 **ADC4TFC2**: 定时器 F 比较 2 时的 ADC 触发 4

参考 ADC4TAC2 的描述。

Bit 10 **ADC4TAC2**: 定时器 A 比较 2 时的 ADC 触发 4

此位可在发生定时器 A 比较 2 时在 ADC 触发 4 输出上生成 ADC 触发信号。

Bit 9 **ADC4EEV10**: 外部事件 10 时的 ADC 触发 4 <sup>(1)</sup>

参考 ADC4EEV6 的描述。

Bit 8 **ADC4EEV9**: 外部事件 9 时的 ADC 触发 4 <sup>(1)</sup>

参考 ADC4EEV6 的描述。

Bit 7 **ADC4EEV8**: 外部事件 8 时的 ADC 触发 4 <sup>(1)</sup>

参考 ADC4EEV6 的描述。

Bit 6 **ADC4EEV7**: 外部事件 7 时的 ADC 触发 4 <sup>(1)</sup>

参考 ADC4EEV6 的描述。

Bit 5 **ADC4EEV6**: 外部事件 6 时的 ADC 触发 4 <sup>(1)</sup>

此位可在发生外部事件 6 时在 ADC 触发 4 输出上生成 ADC 触发信号。

Bit 4 **ADC4MPER**: 主定时器周期时的 ADC 触发 4

此位可在发生主定时器周期事件时在 ADC 触发 4 输出上生成 ADC 触发信号。

Bit 3 **ADC4MC4**: 主定时器比较 4 时的 ADC 触发 4

参考 ADC4MC1 的描述。

Bit 2 **ADC4MC3**: 主定时器比较 3 时的 ADC 触发 4

参考 ADC4MC1 的描述。

Bit 1 **ADC4MC2**: 主定时器比较 2 时的 ADC 触发 4

参考 ADC4MC1 的描述。

Bit 0 **ADC4MC1**: 主比定时器较 1 时的 ADC 触发 4

此位可在发生主定时器比较 1 事件时在 ADC 触发 4 输出上生成 ADC 触发信号。

1. 这些触发与 HRTIM\_ADC1R/HRTIM\_ADC3R 和 HRTIM\_ADC2R/HRTIM\_ADC4R 不同。

## 22.5.70 HRTIM\_DLL 控制寄存器 (HRTIM\_DLLCR)

偏移地址: 0x3CC

复位值: 0x0000 0000

|          |    |    |    |    |    |    |     |          |    |    |    |             |    |       |     |
|----------|----|----|----|----|----|----|-----|----------|----|----|----|-------------|----|-------|-----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24  | 23       | 22 | 21 | 20 | 19          | 18 | 17    | 16  |
| Reserved |    |    |    |    |    |    |     |          |    |    |    |             |    |       |     |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8   | 7        | 6  | 5  | 4  | 3           | 2  | 1     | 0   |
| Reserved |    |    |    |    |    |    | CKS | Reserved |    |    |    | CALRTE[1:0] |    | CALEN | CAL |
|          |    |    |    |    |    |    | rw  |          |    |    |    | rw          | rw | rw    | rw  |

Bit 31:9 保留, 必须保持复位值。

Bit 8 **CKS**: HRTIM 时钟状态 (HRTIM clock status)

此位定义了 DLL 当前所处 HRTIM 时钟的状态。

0:  $f_{HRTIM} = 80\text{MHz} \sim 140\text{MHz}$

1:  $f_{HRTIM} = 140\text{MHz} \sim 192\text{MHz}$

Bit 7:4 保留, 必须保持复位值。

Bit 3:2 **CALRTE[1:0]**: DLL 校准速率 (DLL Calibration rate)

这定义了 DLL 校准的周期性。

00:  $1048576 * t_{HRTIM}$  (对于  $f_{HRTIM} = 192\text{MHz}$ , 为 5.461ms)

01:  $131072 * t_{HRTIM}$  (对于  $f_{HRTIM} = 192\text{MHz}$ , 为 683 $\mu\text{s}$ )

10:  $16384 * t_{HRTIM}$  (对于  $f_{HRTIM} = 192\text{MHz}$ , 为 85 $\mu\text{s}$ )

11:  $2048 * t_{HRTIM}$  (对于  $f_{HRTIM} = 192\text{MHz}$ , 为 11 $\mu\text{s}$ )

Bit 1 **CALEN**: DLL 校准使能 (DLL Calibration Enable)

此位根据 CALRTE[1:0]位设置使能周期性 DLL 校准。当 CALEN 位复位时, 可以使用 CAL 位以单次模式启动校准。

0: 禁用周期性校准

1: 根据 CALRTE[1:0] 设置进行周期性校准

注: CALEN 和 CAL 位不能同时设置

Bit 0 **CAL**: DLL 校准开始 (DLL Calibration Start)

此位启动 DLL 校准过程。它只写不读。

0: 无校准请求

1: 开始校准

注: CAL 和 CALEN 位不能同时设置

## 22.5.71 HRTIM 故障输入寄存器 1 (HRTIM\_FLTINR1)

偏移地址: 0x3D0

复位值: 0x0000 0000

| 31          | 30         | 29 | 28 | 27 | 26             | 25    | 24    | 23          | 22         | 21 | 20 | 19 | 18             | 17    | 16    |
|-------------|------------|----|----|----|----------------|-------|-------|-------------|------------|----|----|----|----------------|-------|-------|
| FLT4LC<br>K | FLT4F[3:0] |    |    |    | FLT4<br>SRC[0] | FLT4P | FLT4E | FLT3LC<br>K | FLT3F[3:0] |    |    |    | FLT3<br>SRC[0] | FLT3P | FLT3E |
| rw          | rw         | rw | rw | rw | rw             | rw    | rw    | rw          | rw         | rw | rw | rw | rw             | rw    | rw    |
| 15          | 14         | 13 | 12 | 11 | 10             | 9     | 8     | 7           | 6          | 5  | 4  | 3  | 2              | 1     | 0     |
| FLT2LC<br>K | FLT2F[3:0] |    |    |    | FLT2<br>SRC[0] | FLT2P | FLT2E | FLT1LC<br>K | FLT1F[3:0] |    |    |    | FLT1<br>SRC[0] | FLT1P | FLT1E |
| rw          | rw         | rw | rw | rw | rw             | rw    | rw    | rw          | rw         | rw | rw | rw | rw             | rw    | rw    |

Bit 31 **FLT4LCK**: 故障 4 锁定 (Fault 4 lock)

参考 HRTIM\_FLTINR2 寄存器中的 FLT5LCK 描述。

Bit 30:27 **FLT4F[3:0]**: 故障 4 过滤器 (Fault 4 filter)

参考 HRTIM\_FLTINR2 寄存器中的 FLT5F[3:0]描述。

Bit 26 **FLT4SRC**: 故障 4 源位 0 (Fault 4 source bit 0)

参考 HRTIM\_FLTINR2 寄存器中的 FLT5SRC[0]描述。

Bit 25 **FLT4P**: 故障 4 极性 (Fault 4 polarity)

参考 HRTIM\_FLTINR2 寄存器中的 FLT5P 描述。

Bit 24 **FLT4E**: 故障 4 使能 (Fault 4 enable)

参考 HRTIM\_FLTINR2 寄存器中的 FLT5E 描述。

Bit 23 **FLT3LCK**: 故障 3 锁定 (Fault 3 lock)

参考 HRTIM\_FLTINR2 寄存器中的 FLT5LCK 描述。

Bit 22:19 **FLT3F[3:0]**: 故障 3 过滤器 (Fault 3 filter)

参考 HRTIM\_FLTINR2 寄存器中的 FLT5F[3:0]描述。

Bit 18 **FLT3SRC[0]**: 故障 3 源位 0 (Fault 3 source bit 0)

参考 HRTIM\_FLTINR2 寄存器中的 FLT5SRC[0]描述。

Bit 17 **FLT3P**: 故障 3 极性 (Fault 3 polarity)

参考 HRTIM\_FLTINR2 寄存器中的 FLT5P 描述。

Bit 16 **FLT3E**: 故障 3 使能 (Fault 3 enable)

参考 HRTIM\_FLTINR2 寄存器中的 FLT5E 描述。

Bit 15 **FLT2LCK**: 故障 2 锁定 (Fault 2 lock)

参考 HRTIM\_FLTINR2 寄存器中的 FLT5LCK 描述。

Bit 14:11 **FLT2F[3:0]**: 故障 2 过滤器 (Fault 2 filter)

参考 HRTIM\_FLTINR2 寄存器中的 FLT5F[3:0]描述。

Bit 10 **FLT2SRC[0]**: 故障 2 源位 0 (Fault 2 source bit 0)

参考 HRTIM\_FLTINR2 寄存器中的 FLT5SRC[0]描述。

Bit 9 **FLT2P**: 故障 2 极性 (Fault 2 polarity)

参考 HRTIM\_FLTINR2 寄存器中的 FLT2P 描述。

Bit 8 **FLT2E**: 故障 2 使能 (Fault 2 enable)

参考 HRTIM\_FLTINR2 寄存器中的 FLT5E 描述。

Bit 7 **FLT1LCK**: 故障 1 锁定 (Fault 1 lock)

参考 HRTIM\_FLTINR2 寄存器中的 FLT5LCK 描述。

Bit 6:3 **FLT1F[3:0]**: 故障 1 过滤器 (Fault 1 filter)

参考 HRTIM\_FLTINR2 寄存器中的 FLT5F[3:0]描述。

Bit 2 **FLT1SRC[0]**: 故障 1 源位 0 (Fault 1 source bit 0)

参考 HRTIM\_FLTINR2 寄存器中的 FLT5SRC[0]描述。

Bit 1 **FLT1P**: 故障 1 极性 (Fault 1 polarity)

参考 HRTIM\_FLTINR2 寄存器中的 FLT5P 描述。

Bit 0 **FLT1E**: 故障 1 使能 (Fault 1 enable)

参考 HRTIM\_FLTINR2 寄存器中的 FLT5E 描述。



## 22.5.72 HRTIM 故障输入寄存器 2 (HRTIM\_FLTINR2)

偏移地址: 0x3D4

复位值: 0x0000 0000

| 31          | 30         | 29 | 28 | 27 | 26             | 25         | 24    | 23          | 22         | 21             | 20             | 19             | 18             | 17             | 16             |
|-------------|------------|----|----|----|----------------|------------|-------|-------------|------------|----------------|----------------|----------------|----------------|----------------|----------------|
| Reserved    |            |    |    |    |                | FLTSD[1:0] |       | Reserved    |            | FLT6<br>SRC[1] | FLT5<br>SRC[1] | FLT4<br>SRC[1] | FLT3<br>SRC[1] | FLT2<br>SRC[1] | FLT1<br>SRC[1] |
|             |            |    |    |    |                | rw         | rw    |             |            | rw             | rw             | rw             | rw             | rw             | rw             |
| 15          | 14         | 13 | 12 | 11 | 10             | 9          | 8     | 7           | 6          | 5              | 4              | 3              | 2              | 1              | 0              |
| FLT6<br>LCK | FLT6F[3:0] |    |    |    | FLT6<br>SRC[0] | FLT6P      | FLT6E | FLT5<br>LCK | FLT5F[3:0] |                |                |                | FLT5<br>SRC[0] | FLT5P          | FLT5E          |
| rw          | rw         | rw | rw | rw | rw             | rw         | rw    | rw          | rw         | rw             | rw             | rw             | rw             | rw             | rw             |

Bit 31:26 保留, 必须保持为复位值。

Bit 25:24 **FLTSD[1:0]**: 故障采样时钟分频 (Fault sampling clock division)

此位域指示定时器时钟频率 ( $f_{HRTIM}$ ) 与数字滤波器使用的故障信号采样时钟 ( $f_{FLTS}$ ) 之间的分频比。

00:  $f_{FLTS} = f_{HRTIM}$

01:  $f_{FLTS} = f_{HRTIM} / 2$

10:  $f_{FLTS} = f_{HRTIM} / 4$

11:  $f_{FLTS} = f_{HRTIM} / 8$

注: 此位域必须先于任何 FLTxE 使能位写入。

Bit 23:22 保留, 必须保持为复位值。

Bit 21 **FLT6SRC[1]**: 故障 6 源位 1 (Fault 6 source bit 1)

参考 FLT5SRC[0] 的描述。

Bit 20 **FLT5SRC[1]**: 故障 5 源位 1 (Fault 5 source bit 1)

参考 FLT5SRC[0] 的描述。

Bit 19 **FLT4SRC[1]**: 故障 4 源位 1 (Fault 4 source bit 1)

参考 FLT5SRC[0] 的描述。

Bit 18 **FLT3SRC[1]**: 故障 3 源位 1 (Fault 3 source bit 1)

参考 FLT5SRC[0] 的描述。

Bit 17 **FLT2SRC[1]**: 故障 2 源位 1 (Fault 2 source bit 1)

参考 FLT5SRC[0] 的描述。

Bit 16 **FLT1SRC[1]**: 故障 1 源位 1 (Fault 1 source bit 1)

参考 FLT5SRC[0] 的描述。

Bit 15 **FLT6LCK**: 故障 6 锁定 (Fault 6 lock)

参考 FLT5LCK 的描述。

Bit 14:11 **FLT6F[3:0]**: 故障 6 过滤器 (Fault 6 filter)

参考 FLT5F[3:0] 的描述。

Bit 10 **FLT6SRC[0]**: 故障 6 源位 0 (Fault 6 source bit 0)

参考 FLT5SRC[0] 的描述。

Bit 9 **FLT6P**: 故障 6 极性 (Fault 6 polarity)

参考 FLT5P 的描述。

Bit 8 **FLT6E**: 故障 6 使能 (Fault 6 enable)

参考 FLT5E 的描述。

**Bit 7 FLT5LCK: 故障 5 锁定 (Fault 5 lock)**

FLT5LCK 位会修改故障编程位的写属性, 以防止误写访问修改写属性。

该位仅可写入一次。该位置 1 后, 在下次系统复位之前不能进行修改。

0: FLT5E、FLT5P、FLT5SRC[1:0]、FLT5F[3:0]和 FLT5BLK 位是可读写的。

1: FLT5E、FLT5P、FLT5SRC[1:0]、FLT5F[3:0]和 FLT5BLK 位不能被写入 (只读模式)。

**Bit 6:3 FLT5F[3:0]: 故障 5 滤波器 (Fault 5 filter)**

此位域可定义 FLT5 输入的采样频率和应用于 FLT5 的数字滤波器长度。数字滤波器由事件计数器组成, 每 N 个事件才视为一个有效边沿:

0000: 无滤波器, FLT5 异步工作

0001:  $f_{\text{SAMPLING}} = f_{\text{HRTIM}}$ ,  $N = 2$

0010:  $f_{\text{SAMPLING}} = f_{\text{HRTIM}}$ ,  $N = 4$

0011:  $f_{\text{SAMPLING}} = f_{\text{HRTIM}}$ ,  $N = 8$

0100:  $f_{\text{SAMPLING}} = f_{\text{FLTS}} / 2$ ,  $N = 6$

0101:  $f_{\text{SAMPLING}} = f_{\text{FLTS}} / 2$ ,  $N = 8$

0110:  $f_{\text{SAMPLING}} = f_{\text{FLTS}} / 4$ ,  $N = 6$

0111:  $f_{\text{SAMPLING}} = f_{\text{FLTS}} / 4$ ,  $N = 8$

1000:  $f_{\text{SAMPLING}} = f_{\text{FLTS}} / 8$ ,  $N = 6$

1001:  $f_{\text{SAMPLING}} = f_{\text{FLTS}} / 8$ ,  $N = 8$

1010:  $f_{\text{SAMPLING}} = f_{\text{FLTS}} / 16$ ,  $N = 5$

1011:  $f_{\text{SAMPLING}} = f_{\text{FLTS}} / 16$ ,  $N = 6$

1100:  $f_{\text{SAMPLING}} = f_{\text{FLTS}} / 16$ ,  $N = 8$

1101:  $f_{\text{SAMPLING}} = f_{\text{FLTS}} / 32$ ,  $N = 5$

1110:  $f_{\text{SAMPLING}} = f_{\text{FLTS}} / 32$ ,  $N = 6$

1111:  $f_{\text{SAMPLING}} = f_{\text{FLTS}} / 32$ ,  $N = 8$

注: 此位域仅在 FLT5E 使能位复位时写入。当 FLT5LOCK 被编程时, 此位域将被修改。

**Bit 2 FLT5SRC[0]: 故障 5 源位 0 (Fault 5 source bit 0)**

FLT5SRC[1:0]位域用于选择 FAULT5 输入源

00: 故障 5 输入是 HRTIM\_FLT5 输入引脚

01: 故障 5 输入连接到 COMPx 输出

10: 故障 5 输入是 EEV5\_muxout 输入引脚

11: 保留

注: 此位域仅在 FLT5E 使能位复位时写入。

**Bit 1 FLT5P: 故障 5 极性 (Fault 5 polarity)**

此位选择 FAULT5 输入的极性。

0: 故障 5 输入为低电平有效

1: 故障 5 输入为高电平有效

注: 此位域仅在 FLT5E 使能位复位时写入。

**Bit 0 FLT5E: 故障 5 使能 (Fault 5 enable)**

此位使能全局 FAULT5 输入电路。

0: 故障 5 输入禁用

1: 故障 5 输入使能

### 22.5.73 HRTIM 突发 DMA 主定时器更新寄存器 (HRTIM\_BDMUPR)

偏移地址: 0x3D8

复位值: 0x0000 0000

|          |    |    |    |    |    |       |       |       |       |      |      |      |       |      |     |
|----------|----|----|----|----|----|-------|-------|-------|-------|------|------|------|-------|------|-----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25    | 24    | 23    | 22    | 21   | 20   | 19   | 18    | 17   | 16  |
| Reserved |    |    |    |    |    |       |       |       |       |      |      |      |       |      |     |
| 15       | 14 | 13 | 12 | 11 | 10 | 9     | 8     | 7     | 6     | 5    | 4    | 3    | 2     | 1    | 0   |
| Reserved |    |    |    |    |    | MCMP4 | MCMP3 | MCMP2 | MCMP1 | MREP | MPER | MCNT | MDIER | MICR | MCR |
|          |    |    |    |    |    | rw    | rw    | rw    | rw    | rw   | rw   | rw   | rw    | rw   | rw  |

Bit 31:10 保留, 必须保持为复位值。

Bit 9 **MCMP4**: MCMP4R 寄存器更新使能 (MCMP4R register update enable)

请参考 MCR 描述。

Bit 8 **MCMP3**: MCMP3R 寄存器更新使能 (MCMP3R register update enable)

请参考 MCR 描述。

Bit 7 **MCMP2**: MCMP2R 寄存器更新使能 (MCMP2R register update enable)

请参考 MCR 描述。

Bit 6 **MCMP1**: MCMP1R 寄存器更新使能 (MCMP1R register update enable)

请参考 MCR 描述。

Bit 5 **MREP**: MREP 寄存器更新使能 (MREP register update enable)

请参考 MCR 描述。

Bit 4 **MPER**: MPER 寄存器更新使能 (MPER register update enable)

请参考 MCR 描述。

Bit 3 **MCNT**: MCNTR 寄存器更新使能 (MCNTR register update enable)

请参考 MCR 描述。

Bit 2 **MDIER**: MDIER 寄存器更新使能 (MDIER register update enable)

请参考 MCR 描述。

Bit 1 **MICR**: MICR 寄存器更新使能 (MICR register update enable)

请参考 MCR 描述。

Bit 0 **MCR**: MCR 寄存器更新使能 (MCR register update enable)

此位用于定义主定时器 MCR 寄存器是否包含在要通过突发 DMA 更新的寄存器列表中。

0: MCR 寄存器不会通过突发 DMA 访问更新

1: MCR 寄存器会通过突发 DMA 访问更新

## 22.5.74 HRTIM 突发 DMA 定时器 x 更新寄存器 (HRTIM\_BDTxUPR) (x = A~F)

偏移地址: 块 A: 0x3DC

偏移地址: 块 B: 0x3E0

偏移地址: 块 C: 0x3E4

偏移地址: 块 D: 0x3E8

偏移地址: 块 E: 0x3EC

偏移地址: 块 F: 0x3F4

复位值: 0x0000 0000

| 31            | 30            | 29            | 28            | 27            | 26           | 25           | 24           | 23           | 22            | 21          | 20           | 19           | 18           | 17           | 16            |
|---------------|---------------|---------------|---------------|---------------|--------------|--------------|--------------|--------------|---------------|-------------|--------------|--------------|--------------|--------------|---------------|
| Reserved      |               |               |               |               |              |              |              |              | TIMx<br>EEFR3 | TIMx<br>CR2 | TIMx<br>FLTR | TIMx<br>OUTR | TIMx<br>CHPR | TIMx<br>RSTR | TIMx<br>EEFR2 |
|               |               |               |               |               |              |              |              |              | rw            | rw          | rw           | rw           | rw           | rw           | rw            |
| 15            | 14            | 13            | 12            | 11            | 10           | 9            | 8            | 7            | 6             | 5           | 4            | 3            | 2            | 1            | 0             |
| TIMx<br>EEFR1 | TIMx<br>RST2R | TIMx<br>SET2R | TIMx<br>RST1R | TIMx<br>SET1R | TIMx<br>DTxR | TIMx<br>CMP4 | TIMx<br>CMP3 | TIMx<br>CMP2 | TIMx<br>CMP1  | TIMx<br>REP | TIMx<br>PER  | TIMx<br>CNT  | TIMx<br>DIER | TIMx<br>ICR  | TIMx<br>CR    |
| rw            | rw            | rw            | rw            | rw            | rw           | rw           | rw           | rw           | rw            | rw          | rw           | rw           | rw           | rw           | rw            |

Bit 30:14 保留, 必须保持复位值。

Bit 22 **TIMxEEFR3**: HRTIM\_EEFxR3 寄存器更新使能  
请参考 TIMxCR 描述。

Bit 21 **TIMxCR2**: HRTIM\_TIMxCR2 寄存器更新使能  
请参考 TIMxCR 描述。

Bit 20 **TIMxFLTR**: HRTIM\_FLTxR 寄存器更新使能  
请参考 TIMxCR 描述。

Bit 19 **TIMxOUTR**: HRTIM\_OUTxR 寄存器更新使能  
请参考 TIMxCR 描述。

Bit 18 **TIMxCHPR**: HRTIM\_CHPxR 寄存器更新使能  
请参考 TIMxCR 描述。

Bit 17 **TIMxRSTR**: HRTIM\_RSTxR 寄存器更新使能  
请参考 TIMxCR 描述。

Bit 16 **TIMxEEFR2**: HRTIM\_EEFxR2 寄存器更新使能  
请参考 TIMxCR 描述。

Bit 15 **TIMxEEFR1**: HRTIM\_EEFxR1 寄存器更新使能  
请参考 TIMxCR 描述。

Bit 14 **TIMxRST2R**: HRTIM\_RST2xR 寄存器更新使能  
请参考 TIMxCR 描述。

Bit 13 **TIMxSET2R**: HRTIM\_SET2xR 寄存器更新使能  
请参考 TIMxCR 描述。

Bit 12 **TIMxRST1R**: HRTIM\_RST1xR 寄存器更新使能  
请参考 TIMxCR 描述。

Bit 11 **TIMxSET1R**: HRTIM\_SET1xR 寄存器更新使能

请参考 TIMxCR 描述。

Bit 10 **TIMxDTR**: HRTIM\_DTxR 寄存器更新使能

请参考 TIMxCR 描述。

Bit 9 **TIMxCMP4**: HRTIM\_CMP4xR 寄存器更新使能

请参考 TIMxCR 描述。

Bit 8 **TIMxCMP3**: HRTIM\_CMP3xR 寄存器更新使能

请参考 TIMxCR 描述。

Bit 7 **TIMxCMP2**: HRTIM\_CMP2xR 寄存器更新使能

请参考 TIMxCR 描述。

Bit 6 **TIMxCMP1**: HRTIM\_CMP1xR 寄存器更新使能

请参考 TIMxCR 描述。

Bit 5 **TIMxREP**: HRTIM\_REPxR 寄存器更新使能

请参考 TIMxCR 描述。

Bit 4 **TIMxPER**: HRTIM\_PERxR 寄存器更新使能

请参考 TIMxCR 描述。

Bit 3 **TIMxCNT**: HRTIM\_CNTxR 寄存器更新使能

请参考 TIMxCR 描述。

Bit 2 **TIMxDIER**: HRTIM\_TIMxDIER 寄存器更新使能

请参考 TIMxCR 描述。

Bit 1 **TIMxICR**: HRTIM\_TIMxICR 寄存器更新使能

请参考 TIMxCR 描述。

Bit 0 **TIMxCR**: HRTIM\_TIMxCR 寄存器更新使能

此位用于定义主定时器 MCR 寄存器是否包含在要通过突发 DMA 更新的寄存器列表中。

0: HRTIM\_TIMxCR 寄存器不会通过突发 DMA 访问更新

1: HRTIM\_TIMxCR 寄存器会通过突发 DMA 访问更新

## 22.5.75 HRTIM 突发 DMA 数据寄存器 (HRTIM\_BDMADR)

偏移地址: 0x3F0

复位值: 0x0000 0000

|               |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31            | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| BDMADR[31:16] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| wo            | wo | wo | wo | wo | wo | wo | wo | wo | wo | wo | wo | wo | wo | wo | wo |
| 15            | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| BDMADR[15:0]  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| wo            | wo | wo | wo | wo | wo | wo | wo | wo | wo | wo | wo | wo | wo | wo | wo |

Bit 31:0 **BDMADR[31:0]**: 突发 DMA 数据寄存器 (Burst DMA data register)

对该寄存器的写访问会触发以下操作:

- 将数据值复制到在 BDTxUPR 和 BDMUPR 寄存器位中使能的寄存器中
- 将寄存器指针递增到下一个要填充的位置

## 22.5.76 HRTIM ADC 扩展触发寄存器 (HRTIM\_ADCER)

偏移地址: 0x3F8

复位值: 0x0000 0000

| 31  | 30            | 29 | 28 | 27 | 26 | 25           | 24 | 23 | 22 | 21 | 20           | 19 | 18 | 17 | 16 |
|-----|---------------|----|----|----|----|--------------|----|----|----|----|--------------|----|----|----|----|
| Res | ADC10TRG[4:0] |    |    |    |    | ADC9TRG[4:0] |    |    |    |    | ADC8TRG[4:0] |    |    |    |    |
|     | rw            | rw | rw | rw | rw | rw           | rw | rw | rw | rw | rw           | rw | rw | rw | rw |
| 15  | 14            | 13 | 12 | 11 | 10 | 9            | 8  | 7  | 6  | 5  | 4            | 3  | 2  | 1  | 0  |
| Res | ADC7TRG[4:0]  |    |    |    |    | ADC6TRG[4:0] |    |    |    |    | ADC5TRG[4:0] |    |    |    |    |
|     | rw            | rw | rw | rw | rw | rw           | rw | rw | rw | rw | rw           | rw | rw | rw | rw |

Bit 31 保留, 必须保持为复位值。

Bit 30:26 **ADC10TRG[4:0]**: ADC 触发 10 选择 (ADC trigger 10 selection)

此位组选择 ADC 触发 10 的来源。

请参考 ADC6TRG[4:0]的描述。

Bit 25:21 **ADC9TRG[4:0]**: ADC 触发 9 选择 (ADC trigger 9 selection)

此位组选择 ADC 触发 9 的来源。

请参考 ADC5TRG[4:0]的描述。

Bit 20:16 **ADC8TRG[4:0]**: ADC 触发 8 选择 (ADC trigger 8 selection)

此位组选择 ADC 触发 8 的来源。

请参考 ADC6TRG[4:0]的描述。

Bit 15 保留, 必须保持为复位值。

Bits 14:10 **ADC7TRG[4:0]**: ADC 触发 7 选择 (ADC trigger 7 selection)

这些位用于选择 ADC 触发 7 的来源。

请参考 ADC5TRG[4:0]的描述。

Bits 9:5 **ADC6TRG[4:0]**: ADC 触发 6 选择 (ADC trigger 6 selection)

这些位用于选择 ADC 触发 6 的来源。具体的选项如下：

- 0: 触发于主定时器比较 1
- 1: 触发于主定时器比较 2
- 2: 触发于主定时器比较 3
- 3: 触发于主定时器比较 4
- 4: 触发于主定时器周期
- 5: 触发于外部事件 6
- 6: 触发于外部事件 7
- 7: 触发于外部事件 8
- 8: 触发于外部事件 9
- 9: 触发于外部事件 10
- 10: 触发于定时器 A 比较 2
- 11: 触发于定时器 A 比较 4
- 12: 触发于定时器 A 周期
- 13: 触发于定时器 B 比较 2
- 14: 触发于定时器 B 比较 4
- 15: 触发于定时器 B 周期
- 16: 触发于定时器 C 比较 2
- 17: 触发于定时器 C 比较 4
- 18: 触发于定时器 C 周期
- 19: 触发于定时器 C 重置和计数器溢出
- 20: 触发于定时器 D 比较 2
- 21: 触发于定时器 D 比较 4
- 22: 触发于定时器 D 周期
- 23: 触发于定时器 D 重置和计数器溢出
- 24: 触发于定时器 E 比较 2
- 25: 触发于定时器 E 比较 3
- 26: 触发于定时器 E 比较 4
- 27: 触发于定时器 E 重置和计数器溢出
- 28: 触发于定时器 F 比较 2
- 29: 触发于定时器 F 比较 3
- 30: 触发于定时器 F 比较 4
- 31: 触发于定时器 F 周期



Bits 4:0 **ADC5TRG[4:0]**: ADC 触发 5 选择 (ADC trigger 5 selection)

这些位用于选择 ADC 触发 5 的来源。具体的选项如下:

- 0: 触发于主定时器比较 1
- 1: 触发于主定时器比较 2
- 2: 触发于主定时器比较 3
- 3: 触发于主定时器比较 4
- 4: 触发于主定时器周期
- 5: 触发于外部事件 1
- 6: 触发于外部事件 2
- 7: 触发于外部事件 3
- 8: 触发于外部事件 4
- 9: 触发于外部事件 5
- 10: 触发于定时器 A 比较 3
- 11: 触发于定时器 A 比较 4
- 12: 触发于定时器 A 周期
- 13: 触发于定时器 A 重置和计数器溢出
- 14: 触发于定时器 B 比较 3
- 15: 触发于定时器 B 比较 4
- 16: 触发于定时器 B 周期
- 17: 触发于定时器 B 重置和计数器溢出
- 18: 触发于定时器 C 比较 3
- 19: 触发于定时器 C 比较 4
- 20: 触发于定时器 C 周期
- 21: 触发于定时器 D 比较 3
- 22: 触发于定时器 D 比较 4
- 23: 触发于定时器 D 周期
- 24: 触发于定时器 E 比较 3
- 25: 触发于定时器 E 比较 4
- 26: 触发于定时器 E 周期
- 27: 触发于定时器 F 比较 2
- 28: 触发于定时器 F 比较 3
- 29: 触发于定时器 F 比较 4
- 30: 触发于定时器 F 周期
- 31: 触发于定时器 F 重置和计数器溢出

## 22.5.77 HRTIM ADC 触发更新寄存器 (HRTIM\_ADCUR)

偏移地址: 0x3FC

复位值: 0x0000 0000

|          |              |    |    |     |              |    |    |     |               |    |    |     |              |    |    |
|----------|--------------|----|----|-----|--------------|----|----|-----|---------------|----|----|-----|--------------|----|----|
| 31       | 30           | 29 | 28 | 27  | 26           | 25 | 24 | 23  | 22            | 21 | 20 | 19  | 18           | 17 | 16 |
| Reserved |              |    |    |     |              |    |    |     | AD10USRC[2:0] |    |    | Res | AD9USRC[2:0] |    |    |
|          |              |    |    |     |              |    |    |     | rw            | rw | rw |     | rw           | rw | rw |
| 15       | 14           | 13 | 12 | 11  | 10           | 9  | 8  | 7   | 6             | 5  | 4  | 3   | 2            | 1  | 0  |
| Res      | AD8USRC[2:0] |    |    | Res | AD7USRC[2:0] |    |    | Res | AD6USRC[2:0]  |    |    | Res | AD5USRC[2:0] |    |    |
|          | rw           | rw | rw |     | rw           | rw | rw |     | rw            | rw | rw |     | rw           | rw | rw |

Bit 31:23 保留, 必须保持为复位值。

Bits 22:20 **AD10USRC[2:0]**: ADC 触发 10 更新源 (ADC trigger 10 update source)

参考 AD5USRC[2:0]的描述。

Bit 19 保留, 必须保持复位值。

Bits 18:16 **AD9USRC[2:0]**: ADC 触发 9 更新源 (ADC trigger 9 update source)

参考 AD5USRC[2:0]的描述。

Bit 15 保留, 必须保持复位值。

Bits 14:12 **AD8USRC[2:0]**: ADC 触发 8 更新源 (ADC trigger 8 update source)

参考 AD5USRC[2:0]的描述。

Bit 11 保留, 必须保持复位值。

Bits 10:8 **AD7USRC[2:0]**: ADC 触发 7 更新源 (ADC trigger 7 update source)

参考 AD5USRC[2:0]的描述。

Bit 7 保留, 必须保持复位值。

Bits 6:4 **AD6USRC[2:0]**: ADC 触发 6 更新源 (ADC trigger 6 update source)

参考 AD5USRC[2:0]的描述。

Bit 3 保留, 必须保持复位值。

Bits 2:0 **AD5USRC[2:0]**: ADC 触发 5 更新源 (ADC trigger 5 update source)

这些位定义了触发 HRTIM\_ADCER 寄存器中 ADC5TRG[4:0]位域更新的源 (从预加载寄存器传输到有效寄存器)。它仅定义了源定时器。

确切的条件在定时器本身中定义, 通过 HRTIM\_MCR 寄存器中的 BRSTDMA[1:0]位域或 HRTIM\_TIMxCR 寄存器中的 UPDGAT[3:0]位域。

000: 主定时器

001: 定时器 A

010: 定时器 B

011: 定时器 C

100: 定时器 D

101: 定时器 E

110: 定时器 F

111: 保留

## 22.5.78 HRTIM ADC 后缩放器寄存器 1 (HRTIM\_ADCPS1)

偏移地址: 0x400

复位值: 0x0000 0000

| 31           | 30 | 29 | 28           | 27  | 26           | 25 | 24 | 23  | 22           | 21  | 20           | 19 | 18 | 17  | 16         |
|--------------|----|----|--------------|-----|--------------|----|----|-----|--------------|-----|--------------|----|----|-----|------------|
| Reserved     |    |    | ADC5PSC[4:0] |     |              |    |    | Res | ADC4PSC[4:0] |     |              |    |    | Res | ADC3PSC[4] |
|              |    |    | rw           | rw  | rw           | rw | rw |     | rw           | rw  | rw           | rw | rw |     |            |
| 15           | 14 | 13 | 12           | 11  | 10           | 9  | 8  | 7   | 6            | 5   | 4            | 3  | 2  | 1   | 0          |
| ADC3PSC[3:0] |    |    |              | Res | ADC2PSC[4:0] |    |    |     |              | Res | ADC1PSC[4:0] |    |    |     |            |
| rw           | rw | rw | rw           |     | rw           | rw | rw | rw  | rw           |     | rw           | rw | rw | rw  | rw         |

Bit 31:29 保留, 必须保持为复位值。

Bits 28:24 **ADC5PSC[4:0]**: ADC 5 后缩放器 (ADC 5 post scaler)

参考 ADC1PSC[4:0] 的描述。

Bit 23 保留, 必须保持复位值。

Bits 22:18 **ADC4PSC[4:0]**: ADC 4 后缩放器 (ADC 4 post scaler)

参考 ADC1PSC[4:0] 的描述。

Bit 17 保留, 必须保持复位值。

Bits 16:12 **ADC3PSC[4:0]**: ADC 3 后缩放器 (ADC 3 post scaler)

参考 ADC1PSC[4:0] 的描述。

Bit 11 保留, 必须保持复位值。

Bits 10:6 **ADC2PSC[4:0]**: ADC 2 后缩放器 (ADC 2 post scaler)

参考 ADC1PSC[4:0] 的描述。

Bit 5 保留, 必须保持复位值。

Bits 4:0 **ADC1PSC[4:0]**: ADC 1 后缩放器 (ADC 1 post scaler)

这些位用于选择 ADC 1 的后缩放器比率。

## 22.5.79 HRTIM ADC 后缩放器寄存器 2 (HRTIM\_ADCPS2)

偏移地址：0x404

复位值：0x0000 0000

|              |    |    |               |     |              |    |    |     |              |     |              |    |    |     |            |
|--------------|----|----|---------------|-----|--------------|----|----|-----|--------------|-----|--------------|----|----|-----|------------|
| 31           | 30 | 29 | 28            | 27  | 26           | 25 | 24 | 23  | 22           | 21  | 20           | 19 | 18 | 17  | 16         |
| Reserved     |    |    | ADC10PSC[4:0] |     |              |    |    | Res | ADC9PSC[4:0] |     |              |    |    | Res | ADC8PSC[4] |
|              |    |    | rw            | rw  | rw           | rw | rw |     | rw           | rw  | rw           | rw | rw |     |            |
| 15           | 14 | 13 | 12            | 11  | 10           | 9  | 8  | 7   | 6            | 5   | 4            | 3  | 2  | 1   | 0          |
| ADC8PSC[3:0] |    |    |               | Res | ADC7PSC[4:0] |    |    |     |              | Res | ADC6PSC[4:0] |    |    |     |            |
| rw           | rw | rw | rw            |     | rw           | rw | rw | rw  | rw           |     | rw           | rw | rw | rw  | rw         |

Bit 31:29 保留，必须保持为复位值。

Bits 28:24 **ADC10PSC[4:0]**: ADC 10 后缩放器 (ADC 10 post scaler)

参考 ADC1PSC[4:0] 的描述。

Bit 23 保留，必须保持复位值。

Bits 22:18 **ADC9PSC[4:0]**: ADC 9 后缩放器 (ADC 9 post scaler)

参考 ADC1PSC[4:0] 的描述。

Bit 17 保留，必须保持复位值。

Bits 16:12 **ADC8PSC[4:0]**: ADC 8 后缩放器 (ADC 8 post scaler)

参考 ADC1PSC[4:0] 的描述。

Bit 11 保留，必须保持复位值。

Bits 10:6 **ADC7PSC[4:0]**: ADC 7 后缩放器 (ADC 7 post scaler)

参考 ADC1PSC[4:0] 的描述。

Bit 5 保留，必须保持复位值。

Bits 4:0 **ADC6PSC[4:0]**: ADC 6 后缩放器 (ADC 6 post scaler)

参考 ADC1PSC[4:0] 的描述。

## 22.5.80 HRTIM 故障输入寄存器 3 (HRTIM\_FLTINR3)

偏移地址: 0x408

复位值: 0x0000 0000

| 31           | 30           | 29           | 28 | 27 | 26 | 25           | 24           | 23           | 22           | 21           | 20 | 19 | 18 | 17           | 16           |
|--------------|--------------|--------------|----|----|----|--------------|--------------|--------------|--------------|--------------|----|----|----|--------------|--------------|
| FLT4<br>RSTM | FLT4<br>CRES | FLT4CNT[3:0] |    |    |    | FLT4<br>BLKS | FLT4<br>BLKE | FLT3<br>RSTM | FLT3<br>CRES | FLT3CNT[3:0] |    |    |    | FLT3<br>BLKS | FLT3<br>BLKE |
| rw           | rw           | rw           | rw | rw | rw | rw           | rw           | rw           | rw           | rw           | rw | rw | rw | rw           | rw           |
| 15           | 14           | 13           | 12 | 11 | 10 | 9            | 8            | 7            | 6            | 5            | 4  | 3  | 2  | 1            | 0            |
| FLT2<br>RSTM | FLT2<br>CRES | FLT2CNT[3:0] |    |    |    | FLT2<br>BLKS | FLT2<br>BLKE | FLT1<br>RSTM | FLT1<br>CRES | FLT1CNT[3:0] |    |    |    | FLT1<br>BLKS | FLT1<br>BLKE |
| rw           | rw           | rw           | rw | rw | rw | rw           | rw           | rw           | rw           | rw           | rw | rw | rw | rw           | rw           |

Bit 31 **FLT4RSTM**: 故障 4 复位模式 (Fault 4 reset mode)

参考 FLT5RSTM 的描述。

Bit 30 **FLT4CRES**: 故障 4 计数器复位 (Fault 4 counter reset)

参考 FLT5CRES 的描述。

Bits 29:26 **FLT4CNT[3:0]**: 故障 4 计数器 (Fault 4 counter)

参考 FLT5CNT 的描述。

Bit 25 **FLT4BLKS**: 故障 4 消隐源 (Fault 4 blanking source)

参考 FLT5BLKS 的描述。

Bit 24 **FLT4BLKE**: 故障 4 消隐使能 (Fault 4 blanking enable)

参考 FLT5BLKE 的描述。

Bit 23 **FLT3RSTM**: 故障 3 复位模式 (Fault 3 reset mode)

参考 FLT5RSTM 的描述。

Bit 22 **FLT3CRES**: 故障 3 计数器复位 (Fault 3 counter reset)

参考 FLT5CRES 的描述。

Bits 21:18 **FLT3CNT[3:0]**: 故障 3 计数器 (Fault 3 counter)

参考 FLT5CNT 的描述。

Bit 17 **FLT3BLKS**: 故障 3 消隐源 (Fault 3 blanking source)

参考 FLT5BLKS 的描述。

Bit 16 **FLT3BLKE**: 故障 3 消隐使能 (Fault 3 blanking enable)

参考 FLT5BLKE 的描述。

Bit 15 **FLT2RSTM**: 故障 2 复位模式 (Fault 2 reset mode)

参考 FLT5RSTM 的描述。

Bit 14 **FLT2CRES**: 故障 2 计数器复位 (Fault 2 counter reset)

参考 FLT5CRES 的描述。

Bits 13:10 **FLT2CNT[3:0]**: 故障 2 计数器 (Fault 2 counter)

参考 FLT5CNT 的描述。

Bit 9 **FLT2BLKS**: 故障 2 消隐源 (Fault 2 blanking source)

参考 FLT5BLKS 的描述。

Bit 8 **FLT2BLKE**: 故障 2 消隐使能 (Fault 2 blanking enable)

参考 FLT5BLKE 的描述。

Bit 7 **FLT1RSTM**: 故障 1 复位模式 (Fault 1 reset mode)

参考 FLT5RSTM 的描述。

Bit 6 **FLT1CRES**: 故障 1 计数器复位 (Fault 1 counter reset)

参考 FLT5CRES 的描述。

Bits 5:2 **FLT1CNT[3:0]**: 故障 1 计数器 (Fault 1 counter)

参考 FLT5CNT 的描述。

Bit 1 **FLT1BLKS**: 故障 1 消隐源 (Fault 1 blanking source)

参考 FLT5BLKS 的描述。

Bit 0 **FLT1BLKE**: 故障 1 消隐使能 (Fault 1 blanking enable)

参考 FLT5BLKE 的描述。

## 22.5.81 HRTIM 故障输入寄存器 4 (HRTIM\_FLTINR4)

偏移地址: 0x40C

复位值: 0x0000 0000

| 31           | 30           | 29           | 28 | 27 | 26 | 25           | 24           | 23           | 22           | 21           | 20 | 19 | 18 | 17           | 16           |
|--------------|--------------|--------------|----|----|----|--------------|--------------|--------------|--------------|--------------|----|----|----|--------------|--------------|
| Reserved     |              |              |    |    |    |              |              |              |              |              |    |    |    |              |              |
| 15           | 14           | 13           | 12 | 11 | 10 | 9            | 8            | 7            | 6            | 5            | 4  | 3  | 2  | 1            | 0            |
| FLT6<br>RSTM | FLT6<br>CRES | FLT6CNT[3:0] |    |    |    | FLT6<br>BLKS | FLT6<br>BLKE | FLT5<br>RSTM | FLT5<br>CRES | FLT5CNT[3:0] |    |    |    | FLT5<br>BLKS | FLT5<br>BLKE |
| rw           | rw           | rw           | rw | rw | rw | rw           | rw           | rw           | rw           | rw           | rw | rw | rw | rw           | rw           |

Bit 31:16 保留, 必须保持为复位值。

Bit 15 **FLT6RSTM**: 故障 6 复位模式 (Fault 6 reset mode)

参考 FLT5RSTM 的描述。

Bit 14 **FLT6CRES**: 故障 6 计数器复位 (Fault 6 counter reset)

参考 FLT5CRES 的描述。

Bit 13:10 **FLT6CNT[3:0]**: 故障 6 计数器 (Fault 6 counter)

参考 FLT5CNT 的描述。

Bit 9 **FLT6BLKS**: 故障 6 消隐源 (Fault 6 blanking source)

参考 FLT5BLKS 的描述。

Bit 8 **FLT6BLKE**: 故障 6 消隐使能 (Fault 6 blanking enable)

参考 FLT5BLKE 的描述。

Bit 7 **FLT5RSTM**: 故障 5 复位模式 (Fault 5 reset mode)

此位选择 FAULT5 计数器的复位模式

0: FAULT5 计数器在每个复位/溢出事件时复位

1: 如果上一个计数周期内没有发生故障, 则 FAULT5 计数器在每个复位/溢出事件时复位。

注: 此位域仅在 FLT5E 使能位复位时写入。

Bit 6 **FLT5CRES**: 故障 5 计数器复位 (Fault 5 counter reset)

此位复位 FAULT5 计数器。它由软件设置并由硬件复位。

0: 无操作

1: FAULT5 计数器复位

Bit 5:2 **FLT5CNT[3:0]**: 故障 5 计数器 (Fault 5 counter)

此位域选择 FAULT5 计数器的阈值。当事件数量等于 (FLT5CNT[3:0]+1) 的值时, 故障被视为有效。

Bit 1 **FLT5BLKS**: 故障 5 消隐源 (Fault 5 blanking source)

FLT5BLKS 位选择 FAULT5 的消隐源。

0: 故障 5 复位对齐的消隐窗口

1: 故障 5 移动消隐窗口

注: 此位域仅在 FLT5E 使能位复位时写入。

Bit 0 **FLT5BLKE**: 故障 5 消隐使能 (Fault 5 blanking enable)

FLT5BLKE 位选择 FAULT5 的消隐模式。消隐源由 FLT5BLKS 位定义。



0: 故障 5 无消隐

1: 故障 5 消隐模式

注: 此位域仅在 FLT5E 使能位复位时写入。

## 22.5.82 HRTIM 选项寄存器 (HRTIM\_OPT)

偏移地址: 0x420

复位值: 0x0000 0000

| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18            | 17       | 16 |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|---------------|----------|----|
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |               |          |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2             | 1        | 0  |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    | DLL<br>BYPASS | Reserved |    |
| rw       | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw            | rw       | rw |

Bit 31:3 保留, 必须保持为复位值。

Bit 2 DLLBYPASS: DLL 旁路 (DLL BYPASS)

此位使能 DLL 旁路。

0: 禁止 DLL 旁路

1: 使能 DLL 旁路

Bit 1:0 保留, 必须保持为复位值。

## 23 高级控制定时器 (TIM1/TIM8)

### 23.1 TIM1/TIM8 简介

高级控制定时器 (TIM1/TIM8) 由一个 16 位的自动装载计数器组成, 它由一个可编程的预分频器驱动。

它适合多种用途, 包括包括测量输入信号的脉冲宽度 (输入捕获), 或者产生输出波形 (输出比较、PWM、嵌入死区时间的互补 PWM)。

使用定时器预分频器和 RCC 时钟控制预分频器, 可以实现脉冲宽度和波形周期从几个微秒到几个毫秒的调节。

高级控制定时器 (TIM1/TIM8) 和通用定时器 (TIMy) 是完全独立的, 它们不共享任何资源。它们可以同步操作。

### 23.2 TIM1/TIM8 主要特征

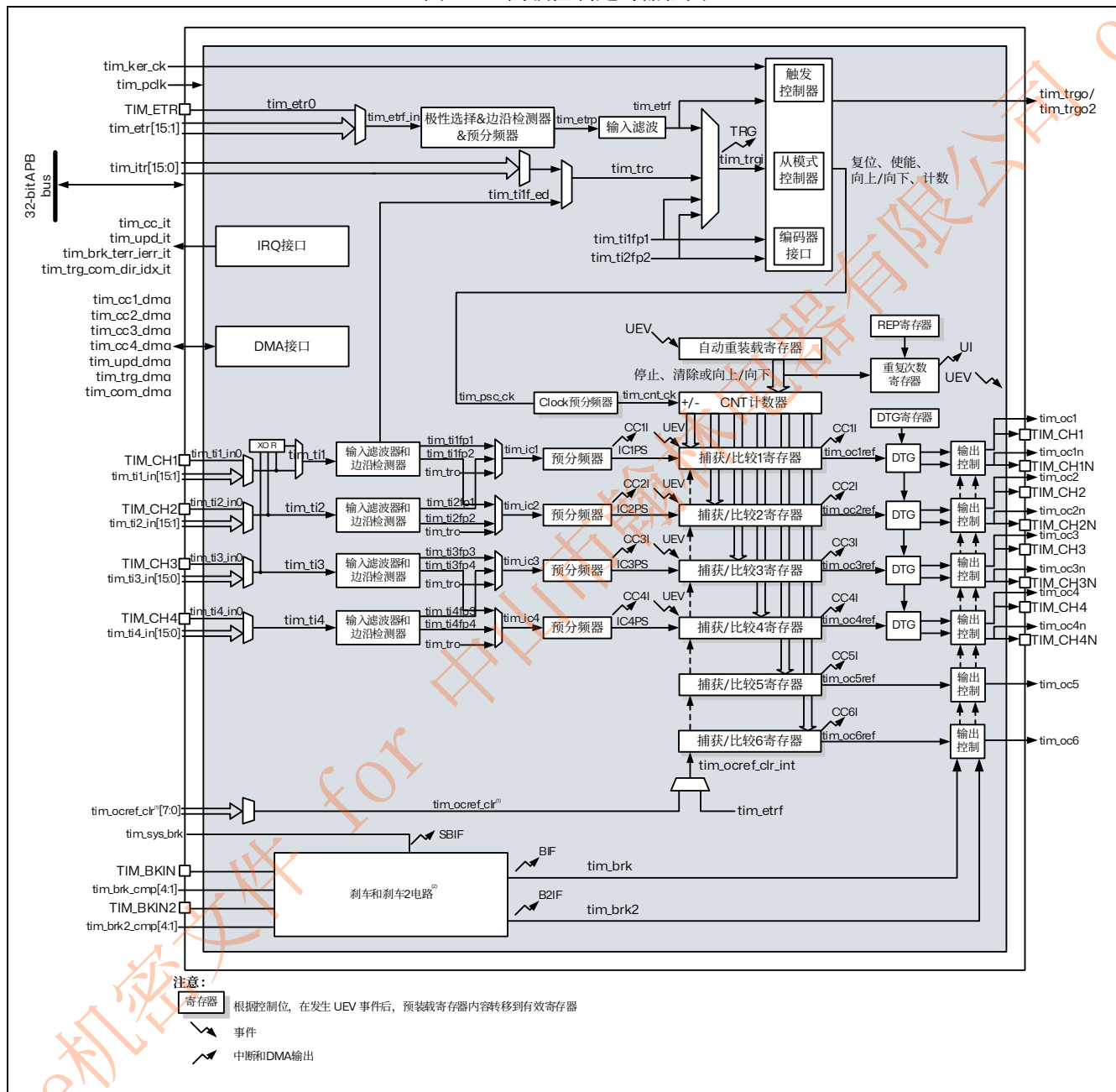
TIM1/TIM8 定时器的功能包括:

- 16 位向上、向下、向上/下自动装载计数器
- 16 位可编程 (可以实时修改) 预分频器, 计数器时钟频率的分频系数为 1~65536 之间的任意数值
- 多达 6 个独立通道:
  - 输入捕获 (除了通道 5 和 6)
  - 输出比较
  - PWM 生成 (边缘或中间对齐模式)
  - 单脉冲模式输出
- 死区时间可编程的互补输出
- 使用外部信号控制定时器和定时器互联的同步电路
- 允许在指定数目的计数器周期之后更新定时器寄存器的重复计数器
- 用于将定时器输出信号置于安全的用户选择配置的两路刹车输入信号
- 如下事件发生时产生中断/DMA 请求:
  - 更新: 计数器向上溢出/向下溢出, 计数器初始化 (通过软件或者内部/外部触发)
  - 触发事件 (计数器启动、停止、初始化或者由内部/外部触发计数)
  - 输入捕获
  - 输出比较
- 支持针对定位的增量 (正交) 编码器和霍尔传感器电路
- 触发输入作为外部时钟或者按周期的电流管理
- 非对称模式

## 23.3 TIM1/TIM8 功能描述

### 23.3.1 框图

图 23.1 高级控制定时器框图



1. 这一特性详见下一节：TIM1/TIM8 引脚和内部信号
2. 参见刹车和刹车 2 电路以获得更多细节

### 23.3.2 TIM1/TIM8 引脚和内部信号

本章节中的表格总结了 TIM 的输入和输出。

表 23.1 TIM 输入/输出引脚

| 引脚名称                                         | 信号类型  | 描述                                                                                                                                                                 |
|----------------------------------------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TIM_CH1<br>TIM_CH2<br>TIM_CH3<br>TIM_CH4     | 输入/输出 | 定时器多用途通道。<br>每个通道都可以用于捕获、比较或 PWM（脉冲宽度调制）。<br>TIM_CH1 和 TIM_CH2 还可以作为外部时钟（时钟频率低于 tim_ker_ck 的 1/4）、外部触发器和四象限编码器输入使用。<br>TIM_CH1、TIM_CH2 和 TIM_CH3 可以用于与数字霍尔效应传感器接口。 |
| TIM_CH1N<br>TIM_CH2N<br>TIM_CH3N<br>TIM_CH4N | 输出    | 由 TIM_CHx 输出派生的定时器的互补输出，具有插入死区时间的能力。                                                                                                                               |
| TIM_ETR                                      | 输入    | 外部触发器输入。这个输入可被用作外部触发器或外部时钟源。如果使用 tim_etr_in 预分频器，则该输入可以接收频率高于 tim_ker_ck 的时钟。                                                                                      |
| TIM_BKIN<br>TIM_BKIN2                        | 输入/输出 | 刹车和刹车 2 输入。这些输入也可以配置为双向模式。                                                                                                                                         |

表 23.2 TIM 内部输入/输出信号

| 内部信号名称                                                                       | 信号类型 | 描述                                                                                                    |
|------------------------------------------------------------------------------|------|-------------------------------------------------------------------------------------------------------|
| tim_ti1_in[15:0]<br>tim_ti2_in[15:0]<br>tim_ti3_in[15:0]<br>tim_ti4_in[15:0] | 输入   | 内部定时器输入总线。<br>tim_ti1_in[15:0]和 tim_ti2_in[15:0]输入可用于捕获或作为外部时钟（时钟频率低于 tim_ker_ck 的 1/4），以及用于四象限编码器信号。 |
| tim_etr[15:0]                                                                | 输入   | 外部触发器内部输入总线。这些输入可被用作触发器、外部时钟或用于硬件的逐周期脉冲宽度控制。如果使用 tim_etr_in 预分频器，这些输入可以接收频率高于 tim_ker_ck 的时钟。         |
| tim_itr[15:0]                                                                | 输入   | 内部触发输入总线。这些输入可用于从模式控制器或作为输入时钟(tim_ker_ck 时钟的 1/4 以下)。                                                 |
| tim_trgo/tim_trgo2                                                           | 输出   | 内部触发器输出。这些触发器被其他定时器或其他外设使用。                                                                           |
| tim_ocref_clr[7:0]                                                           | 输入   | 定时器 tim_ocref_clr 输入总线。这些输入可以用来清除 tim_ocxref 信号，通常用于硬件的逐周期脉冲宽度控制。                                     |
| tim_brk_cmp[4:1]                                                             | 输入   | 内部信号的刹车输入。                                                                                            |
| tim_brk2_cmp[4:1]                                                            | 输入   | 内部信号的刹车 2 输入。                                                                                         |
| tim_sys_brk[n:0]                                                             | 输入   | 系统中断输入。该输入收集 MCU 的系统级错误。                                                                              |
| tim_pclk                                                                     | 输入   | 定时器 APB 时钟。                                                                                           |
| tim_ker_ck                                                                   | 输入   | 定时器内核时钟。                                                                                              |

| 内部信号名称                                                   | 信号类型 | 描述                      |
|----------------------------------------------------------|------|-------------------------|
| tim_cc_it                                                | 输出   | 定时器捕获/比较中断。             |
| tim_upd_it                                               | 输出   | 定时器更新事件中断。              |
| tim_brk_terr_ierr_it                                     | 输出   | 定时器刹车、刹车 2、转换错误和索引错误中断。 |
| tim_trg_com_dir_idx_it                                   | 输出   | 定时器触发、换向、方向和索引中断。       |
| tim_cc1_dma<br>tim_cc2_dma<br>tim_cc3_dma<br>tim_cc4_dma | 输出   | 定时器捕获/比较 1 至 4 dma 请求。  |
| tim_upd_dma                                              | 输出   | 定时器更新 dma 请求。           |
| tim_trg_dma                                              | 输出   | 定时器触发 dma 请求。           |
| tim_com_dma                                              | 输出   | 定时器换向 dma 请求。           |

下表列出连接到 tim\_ti[4:1]输入多路复用器的源

表 23.3 连接到 tim\_ti1 的输入多路复用器

| tim_ti1 输入       | 源         |           |
|------------------|-----------|-----------|
|                  | TIM1      | TIM8      |
| tim_ti1_in0      | TIM1_CH1  | TIM8_CH1  |
| tim_ti1_in1      | comp1_out | comp1_out |
| tim_ti1_in2      | comp2_out | comp2_out |
| tim_ti1_in3      | comp3_out | comp3_out |
| tim_ti1_in4      | comp4_out | comp4_out |
| tim_ti1_in[15:5] | 保留        |           |

表 23.4 连接到 tim\_ti2 的输入多路复用器

| tim_ti2 输入       | 源        |          |
|------------------|----------|----------|
|                  | TIM1     | TIM8     |
| tim_ti2_in0      | TIM1_CH2 | TIM8_CH2 |
| tim_ti2_in[15:1] | 保留       |          |

表 23.5 连接到 tim\_ti3 的输入多路复用器

| tim_ti3 输入       | 源        |          |
|------------------|----------|----------|
|                  | TIM1     | TIM8     |
| tim_ti3_in0      | TIM1_CH3 | TIM8_CH3 |
| tim_ti3_in[15:1] | 保留       |          |

表 23.6 连接到 tim\_ti4 的输入多路复用器

| tim_ti4 输入       | 源        |          |
|------------------|----------|----------|
|                  | TIM1     | TIM8     |
| tim_ti4_in0      | TIM1_CH4 | TIM8_CH4 |
| tim_ti4_in[15:1] | 保留       |          |

下表列出连接到 tim\_itr 输入多路复用器的内部源

表 23.7 TIMx 内部触发连接

| TIMx           | TIM1            | TIM8            |
|----------------|-----------------|-----------------|
| tim_itr0       | 保留              | tim1_trgo       |
| tim_itr1       | tim2_trgo       | tim2_trgo       |
| tim_itr2       | tim3_trgo       | tim3_trgo       |
| tim_itr3       | tim4_trgo       | tim4_trgo       |
| tim_itr4       | tim5_trgo       | tim5_trgo       |
| tim_itr5       | tim8_trgo       | 保留              |
| tim_itr6       | tim15_trgo      | tim15_trgo      |
| tim_itr7       | tim16_oc1       | tim16_oc1       |
| tim_itr8       | tim17_oc1       | tim17_oc1       |
| tim_itr9       | 保留              |                 |
| tim_itr10      | hrtim_out_sync2 | hrtim_out_sync2 |
| tim_itr[15:11] | 保留              |                 |

下表列出连接到 tim\_etr 输入多路复用器的内部源

表 23.8 连接到 tim\_etr 的输入多路复用器

| 定时器外部<br>触发输入信号 | 定时器外部触发信号分配 |           |
|-----------------|-------------|-----------|
|                 | TIM1        | TIM8      |
| tim_etr0        | TIM1_ETR    | TIM8_ETR  |
| tim_etr1        | comp1_out   | comp1_out |
| tim_etr2        | comp2_out   | comp2_out |
| tim_etr3        | comp3_out   | comp3_out |
| tim_etr4        | comp4_out   | comp4_out |
| tim_etr5        | 保留          |           |
| tim_etr6        |             |           |
| tim_etr7        |             |           |
| tim_etr8        | adc1_awd1   | adc2_awd1 |
| tim_etr9        | 保留          | 保留        |
| tim_etr10       |             | 保留        |
| tim_etr11       |             | adc3_awd1 |
| tim_etr12       |             | 保留        |
| tim_etr13       |             | 保留        |
| tim_etr[15:14]  | 保留          |           |

下三张表列出了连接到 tim\_brk 和 tim\_brk2 输入的源

表 23.9 定时器刹车互连

| tim_brk 输入   | TIM1          | TIM8          |
|--------------|---------------|---------------|
| TIM_BKIN     | TIM1_BKIN pin | TIM8_BKIN pin |
| tim_brk_cmp1 | comp1_out     | comp1_out     |
| tim_brk_cmp2 | comp2_out     | comp2_out     |
| tim_brk_cmp3 | comp3_out     | comp3_out     |
| tim_brk_cmp4 | comp4_out     | comp4_out     |
| tim_brk_cmp5 | 保留            |               |
| tim_brk_cmp6 |               |               |
| tim_brk_cmp7 |               |               |
| tim_brk_cmp8 |               |               |

表 23.10 定时器刹车 2 互连

| tim_brk2 输入   | TIM1           | TIM8           |
|---------------|----------------|----------------|
| TIM_BKIN2     | TIM1_BKIN2 pin | TIM8_BKIN2 pin |
| tim_brk2_cmp1 | comp1_out      | comp1_out      |
| tim_brk2_cmp2 | comp2_out      | comp2_out      |
| tim_brk2_cmp3 | comp3_out      | comp3_out      |
| tim_brk2_cmp4 | comp4_out      | comp4_out      |



表 23.11 系统中断互连

| tim_sys_brk<br>输入 | TIM1 / TIM8               | SYSCFG_CFGR2<br>寄存器的使能位 |
|-------------------|---------------------------|-------------------------|
| tim_sys_brk0      | 带 FPU 的 Cortex®-M4 LOCKUP | CLL                     |
| tim_sys_brk1      | 可编程电压检测器 (PVD)            | PVDL                    |
| tim_sys_brk2      | SRAM 奇偶校验错误               | SPL                     |
| tim_sys_brk3      | Flash ECC 错误              | ECCL                    |
| tim_sys_brk4      | 时钟安全系统(CSS)               | 无 (始终使能)                |

下表列出连接到 tim\_ocref\_clr 输入多路复用器的内部源

表 23.12 连接到 ocref\_clr 的输入多路复用器

| 定时器 OCREF<br>清除信号 | 定时器 tim_ocref_clr 信号分配 |           |
|-------------------|------------------------|-----------|
|                   | TIM1                   | TIM8      |
| tim_ocref_clr0    | comp1_out              | comp1_out |
| tim_ocref_clr1    | comp2_out              | comp2_out |
| tim_ocref_clr2    | comp3_out              | comp3_out |
| tim_ocref_clr3    | comp4_out              | comp4_out |
| tim_ocref_clr4    | 保留                     |           |
| tim_ocref_clr5    |                        |           |
| tim_ocref_clr6    |                        |           |
| tim_ocref_clr7    |                        |           |

### 23.3.3 时基单元

可编程高级控制定时器的主要模块是一个 16 位计数器及其相关的自动重载寄存器。计数器可递增计数、递减计数或同时进行递增和递减计数。计数器的时钟可通过预分频器进行分频。

计数器、自动重载寄存器和预分频器寄存器可通过软件进行读写。即使在计数器运行时也可执行读写操作。

时基单元包含：

- 计数器寄存器 (TIMx\_CNT)
- 预分频器寄存器 (TIMx\_PSC)
- 自动装载寄存器 (TIMx\_ARR)
- 重复次数寄存器 (TIMx\_RCR)

自动重载寄存器是预装载的。对自动重载寄存器执行写入或读取操作时会访问预装载寄存器。预装载寄存器的内容既可以直接传送到影子寄存器，也可以在每次发生更新事件 (UEV) 时传送到影子寄存器，这取决于 TIMx\_CR1 寄存器中的自动重载预装载使能位 (ARPE)。当计数器达到上溢值（或者在递减计数时达到下溢值）并且 TIMx\_CR1 寄存器中的 UDIS 位为 0 时，将发送更新事件。该更新事件也可由软件产生。下文将针对各配置的更新事件的产生 进行详细介绍。

计数器由预分频器输出 CK\_CNT 提供时钟，仅当 TIMx\_CR1 寄存器中的计数器启动位 (CEN) 置 1 时，才会启动计数器（有关计数器使能的更多详细信息，另请参见从模式控制器的相关说明）。

注：计数器将在 TIMx\_CR1 寄存器的 CEN 位置 1 时刻的一个时钟周期后开始计数。

#### 预分频器描述

预分频器可对计数器时钟频率进行分频，分频系数介于 1 和 65536 之间。该预分频器基于 TIMx\_PSC 寄存器中的 16 位寄存器所控制的 16 位计数器。由于该控制寄存器具有缓冲功能，因此可对预分频器进行实时更改。而新的预分频比将在下一更新事件发生时被采用。

下两张图以一些示例说明在预分频比实时变化时计数器的行为：

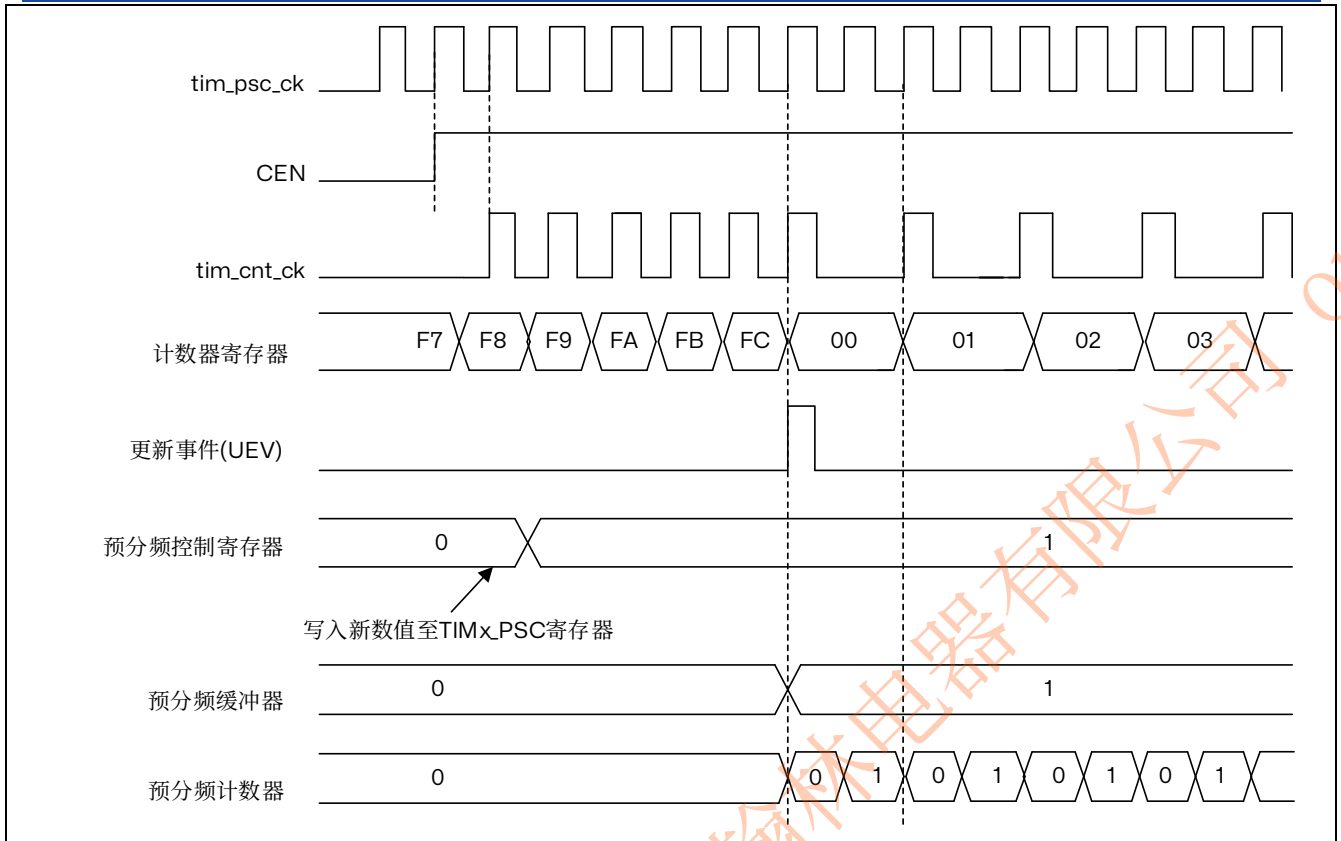


图 23.2 当预分频器的参数从1变成2时，计数器的时序图

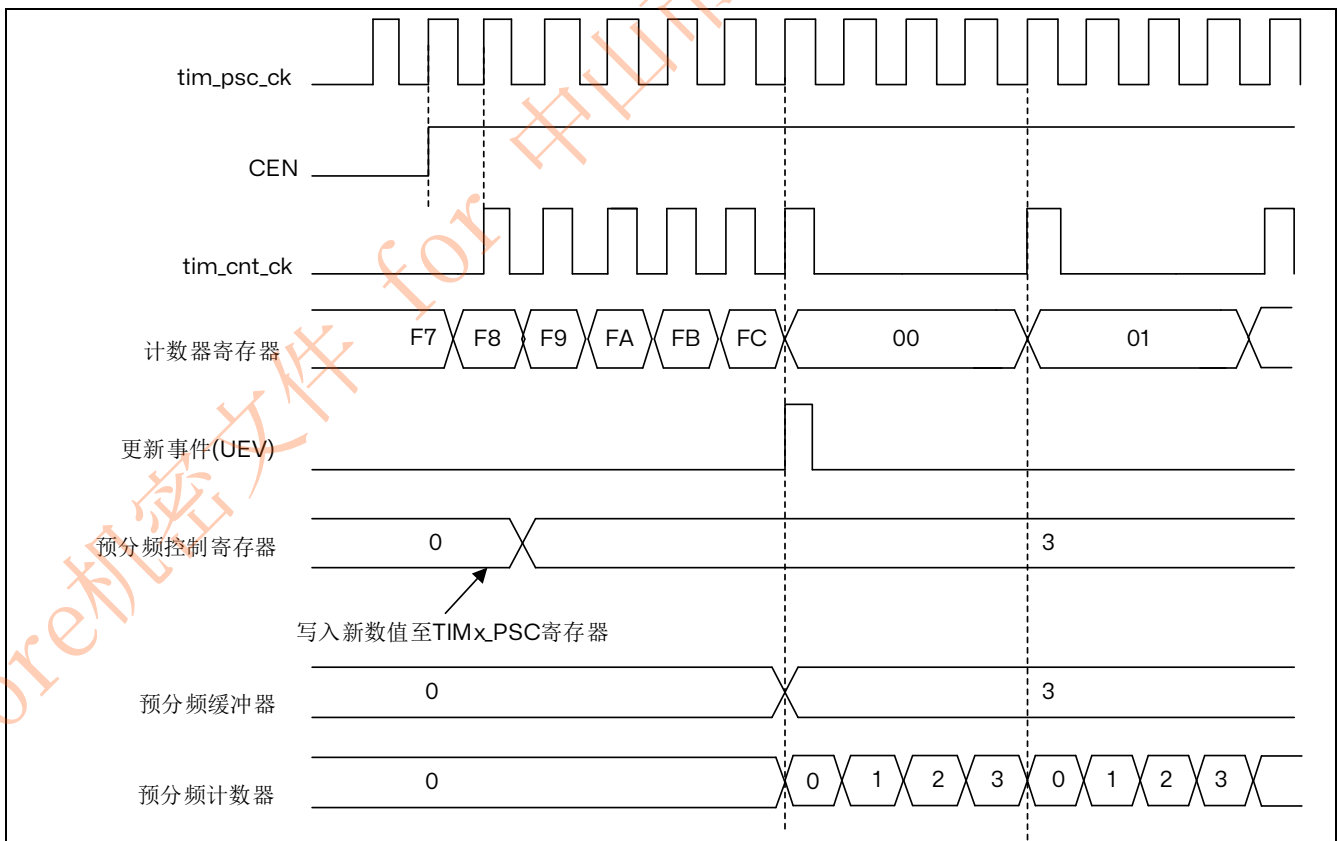


图 23.3 当预分频器的参数从1变成4时，计数器的时序图

### 23.3.4 计数模式

#### 递增计数模式

在递增计数模式下，计数器从 0 计数到自动重载值（TIMx\_ARR 寄存器的内容），然后重新从 0 开始计数并生成计数器上溢事件。

如果使用重复计数器，则当递增计数的重复次数达到重复计数器寄存器中编程的次数加一次（TIMx\_RCR+1）后，将生成更新事件(UEV)。否则，将在每次计数器上溢时产生更新事件。

将 TIMx\_EGR 寄存器的 UG 位置 1（通过软件或使用从模式控制器）时，也将产生更新事件。

通过软件将 TIMx\_CR1 寄存器中的 UDIS 位置 1 可禁止 UEV 事件。这可避免向预装载寄存器写入新值时更新影子寄存器。在 UDIS 位写入 0 之前不会产生任何更新事件。不过，计数器和预分频器计数器都会重新从 0 开始计数（而预分频比保持不变）。此外，如果 TIMx\_CR1 寄存器中的 URS 位（更新请求选择）已置 1，则将 UG 位置 1 会生成更新事件 UEV，但不会将 UIF 标志置 1（因此，不会发送任何中断或 DMA 请求）。

发生更新事件时，将更新所有寄存器且将更新标志（TIMx\_SR 寄存器中的 UIF 位）置 1（取决于 URS 位）：

- 重复计数器中将重新装载 TIMx\_RCR 寄存器的内容
- 自动重载影子寄存器将以预装载值(TIMx\_ARR)进行更新
- 预分频器的缓冲区中将重新装载预装载值（TIMx\_PSC 寄存器的内容）

以下各图以一些示例说明当 TIMx\_ARR=0x36 时不同时钟频率下计数器的行为。

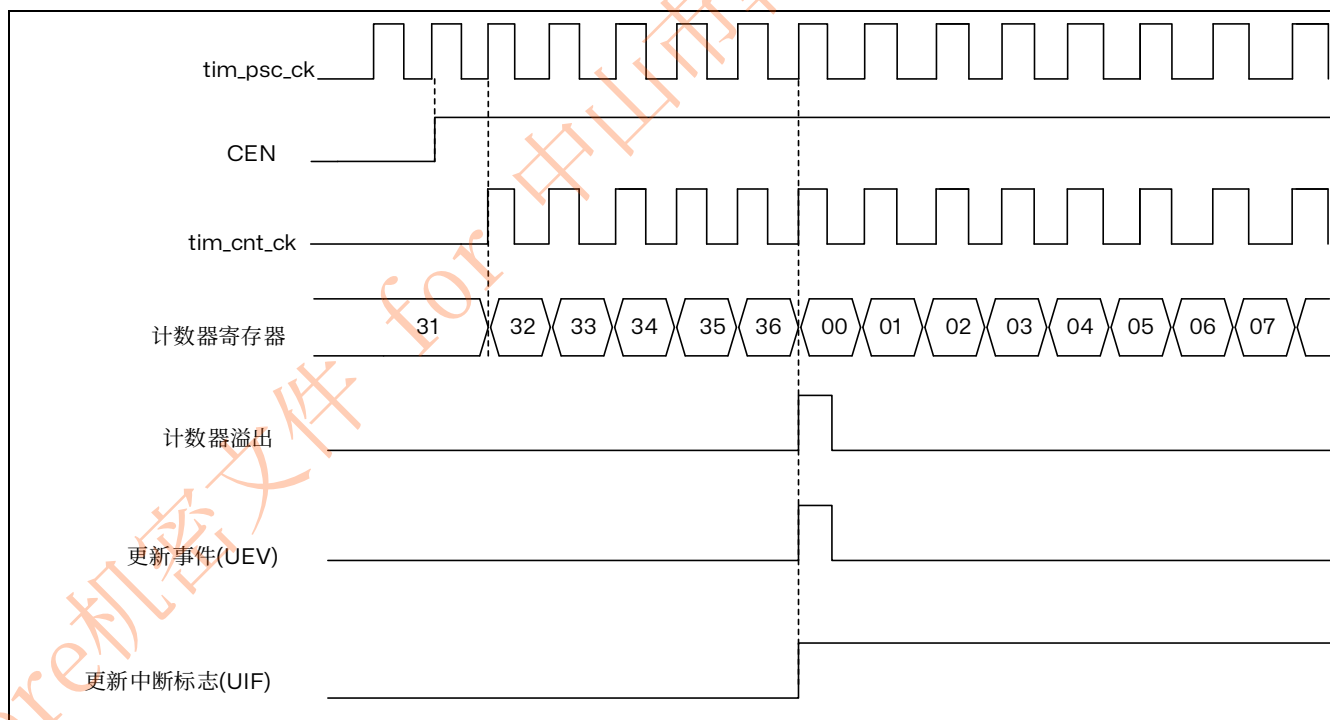


图 23.4 计数器时序图，内部时钟分频系数为 1

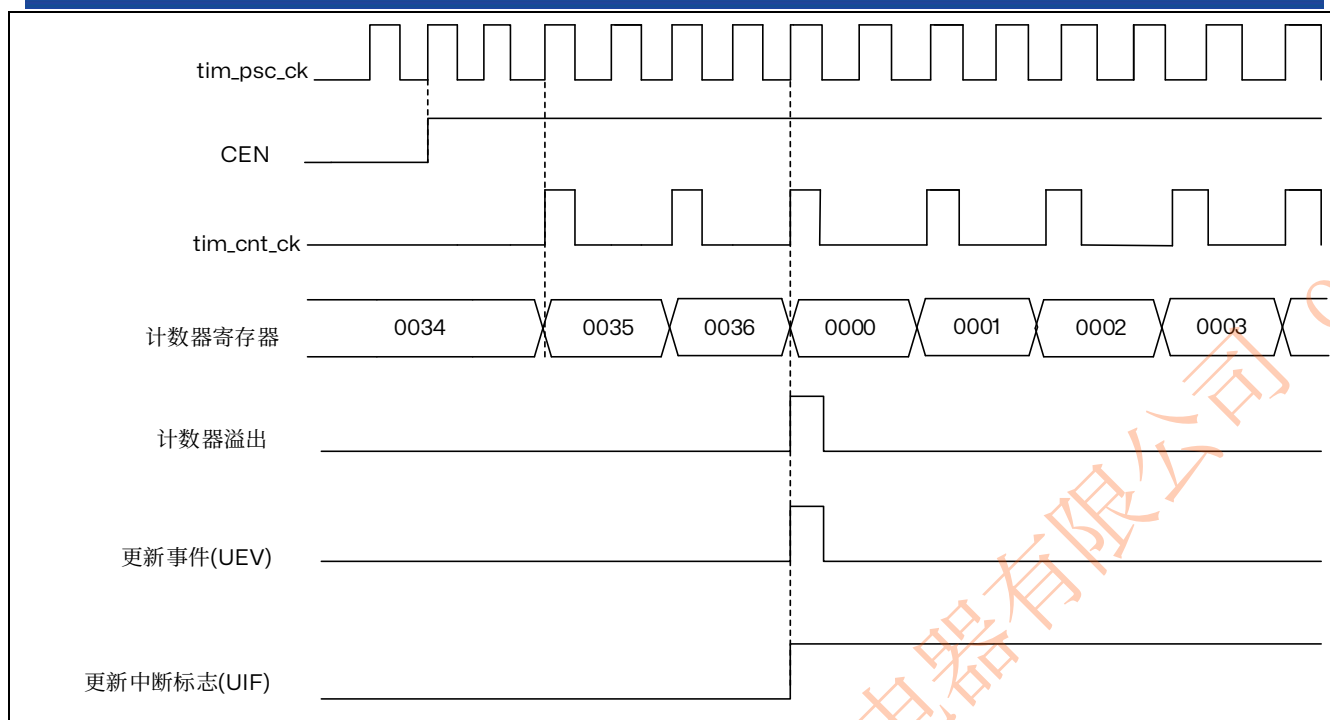


图 23.5 计数器时序图，内部时钟分频系数为 2

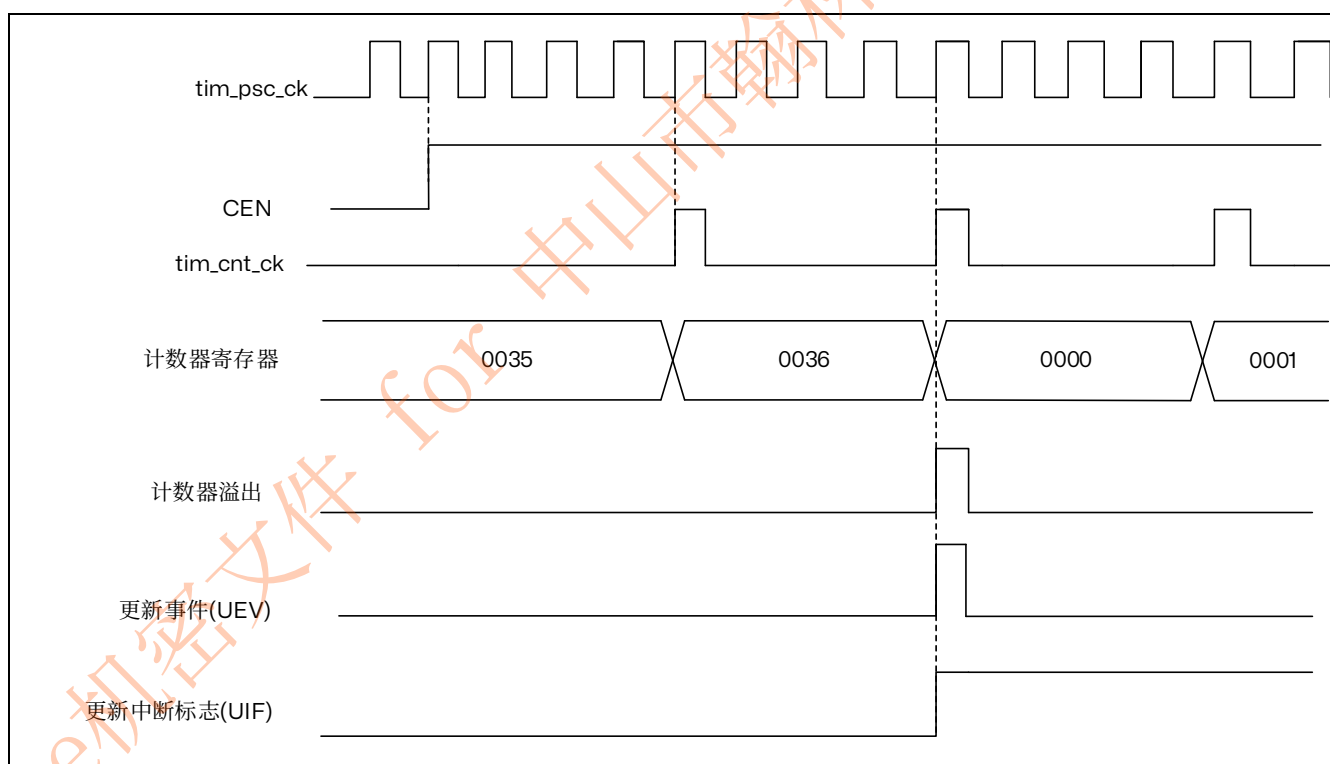


图 23.6 计数器时序图，内部时钟分频系数为 4

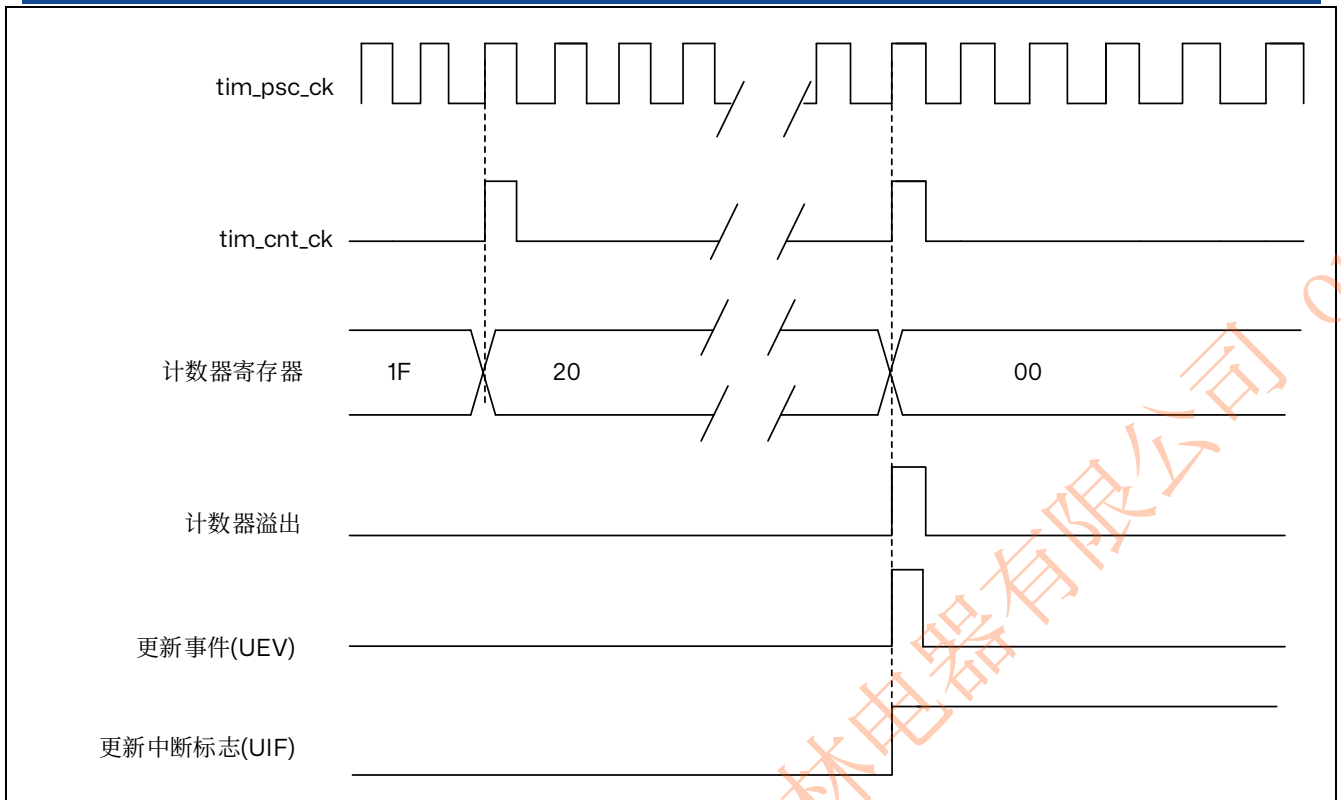


图 23.7 计数器时序图，内部时钟分频系数为 N

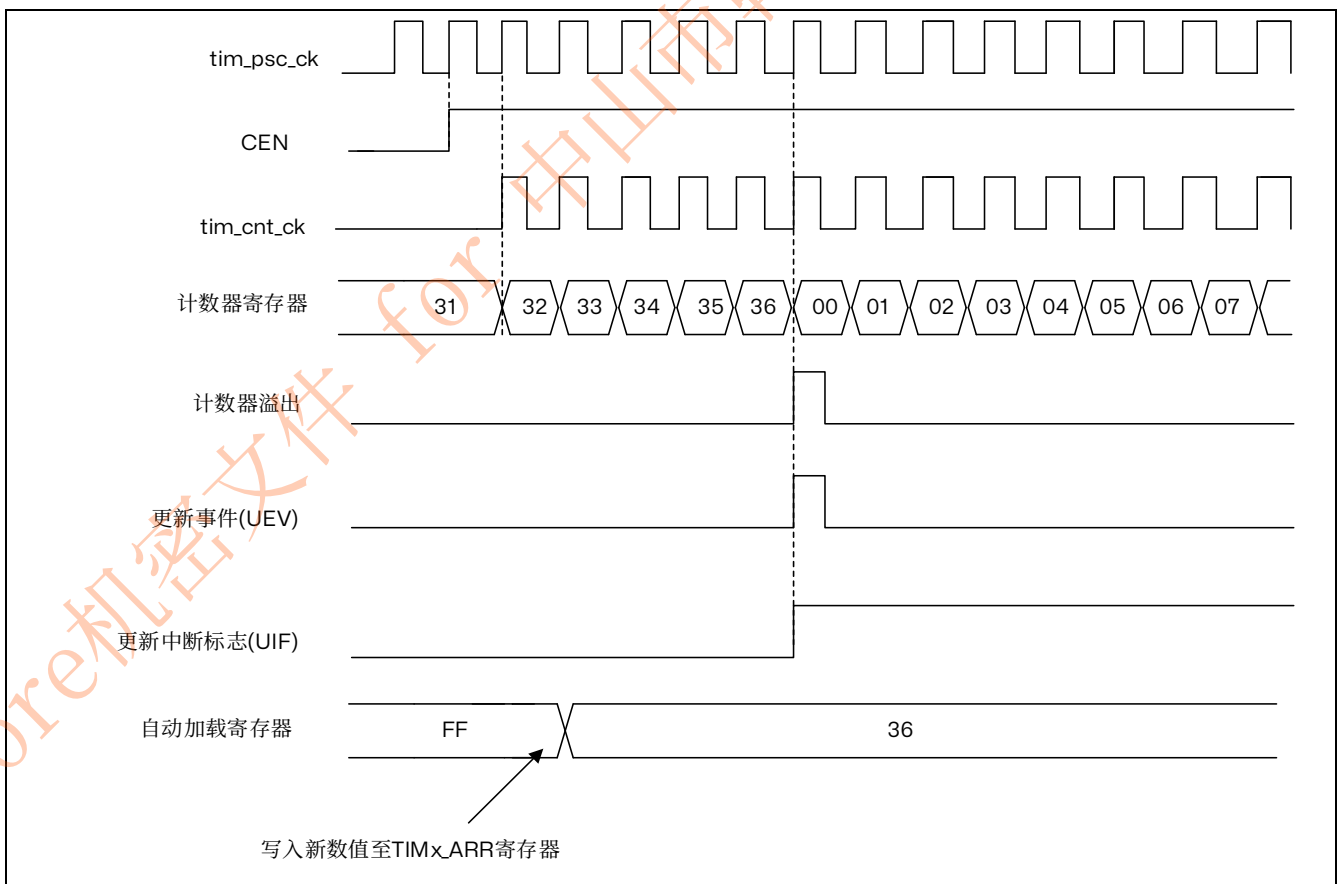


图 23.8 计数器时序图，当 ARPE=0 时的更新事件 (TIMx\_ARR 没有预装入)

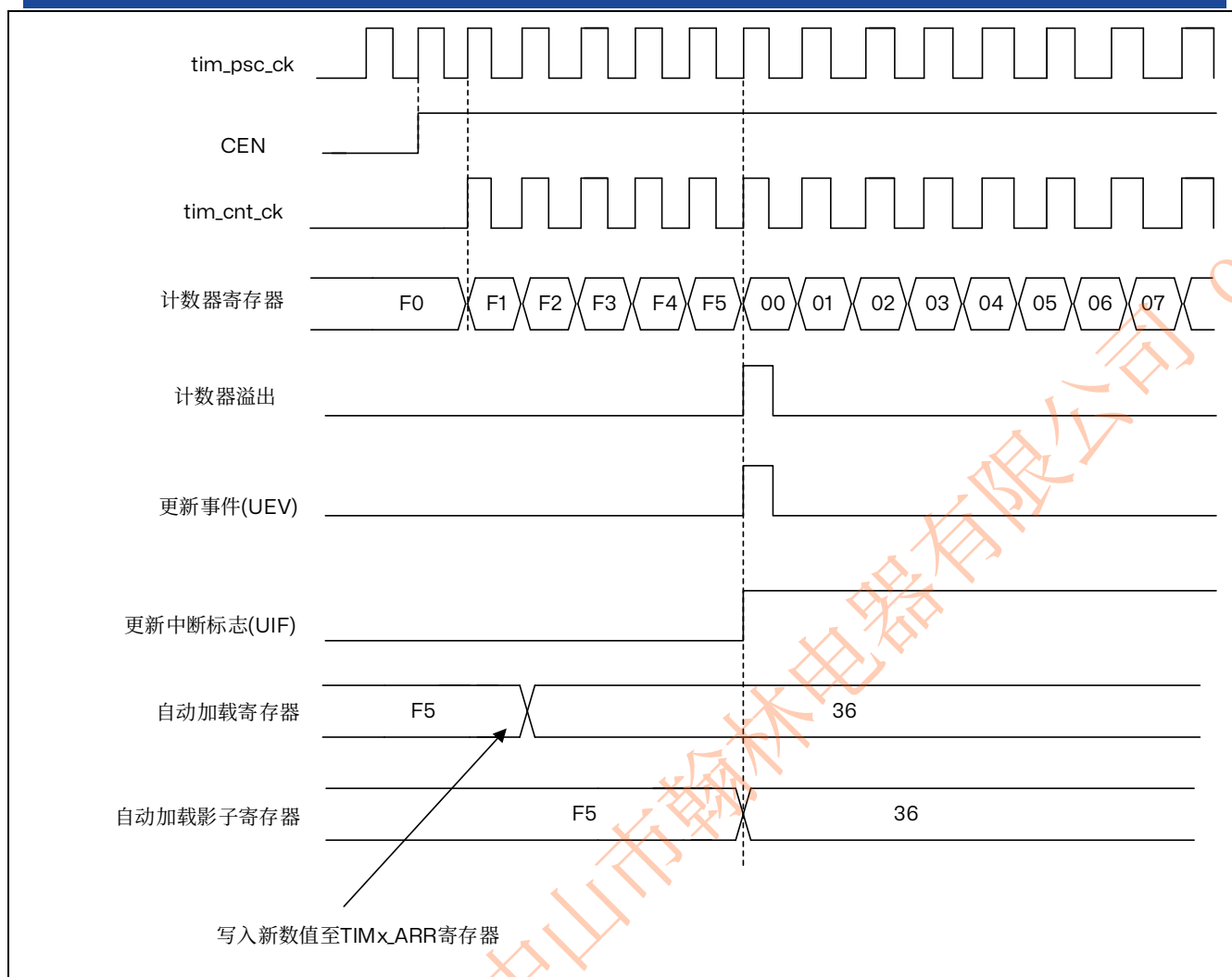


图 23.9 计数器时序图，当 ARPE=1 时的更新事件（预装入了 TIMx\_ARR）

## 递减计数模式

在递减计数模式下，计数器从自动重载值（TIMx\_ARR 寄存器的内容）开始递减计数到 0，然后重新从自动重载值开始计数并生成计数器下溢事件。

如果使用重复计数器，则当递减计数的重复次数达到重复计数器寄存器中编程的次数加一次 (TIMx\_RCR+1) 后，将生成更新事件(UEV)。否则，将在每次计数器下溢时产生更新事件。

将 TIMx\_EGR 寄存器的 UG 位置 1（通过软件或使用从模式控制器）时，也将产生更新事件。

通过软件将 TIMx\_CR1 寄存器中的 UDIS 位置 1 可禁止 UEV 更新事件。这可避免向预装载寄存器写入新值时更新影子寄存器。在 UDIS 位写入 0 之前不会产生任何更新事件。不过，计数器会重新从当前自动重载值开始计数，而预分频器计数器则重新从 0 开始计数（但预分频比保持不变）。

此外，如果 TIMx\_CR1 寄存器中的 URS 位（更新请求选择）已置 1，则将 UG 位置 1 会生成更新事件 UEV，但不会将 UIF 标志置 1（因此，不会发送任何中断请求或 DMA 请求）。这样一来，如果在发生捕获事件时将计数器清零，将不会同时产生更新中断和捕获中断。

发生更新事件时，将更新所有寄存器且将更新标志（TIMx\_SR 寄存器中的 UIF 位）置 1（取决于 URS 位）：

- 重复计数器中将重新装载 TIMx\_RCR 寄存器的内容



- 预分频器的缓冲区中将重新装载预装载值（TIMx\_PSC 寄存器的内容）
- 自动重载活动寄存器将以预装载值（TIMx\_ARR 寄存器的内容）进行更新。注意，自动重载寄存器会在计数器重载之前得到更新，因此，下一个计数周期就是我们所希望的新的周期长度

以下各图以一些示例说明当 TIMx\_ARR=0x36 时不同时钟频率下计数器的行为。

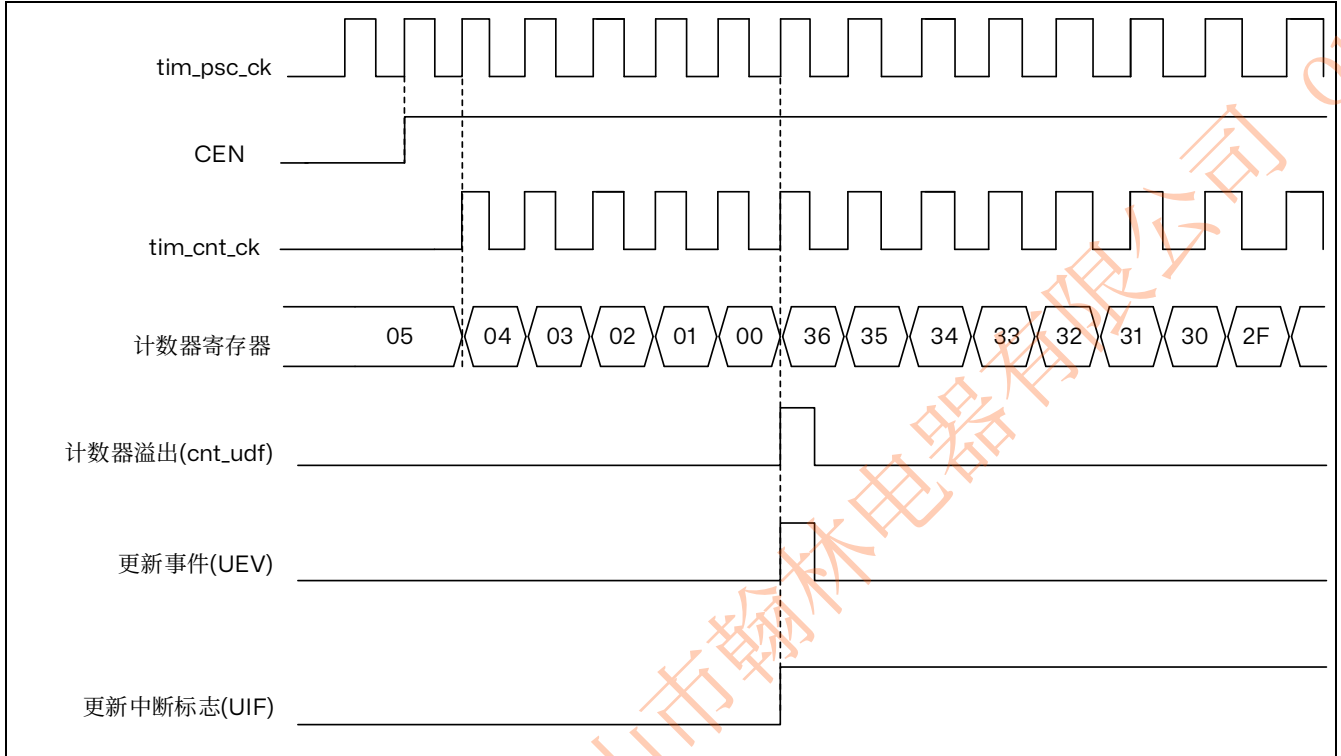


图 23.10 计数器时序图，内部时钟分频系数为 1

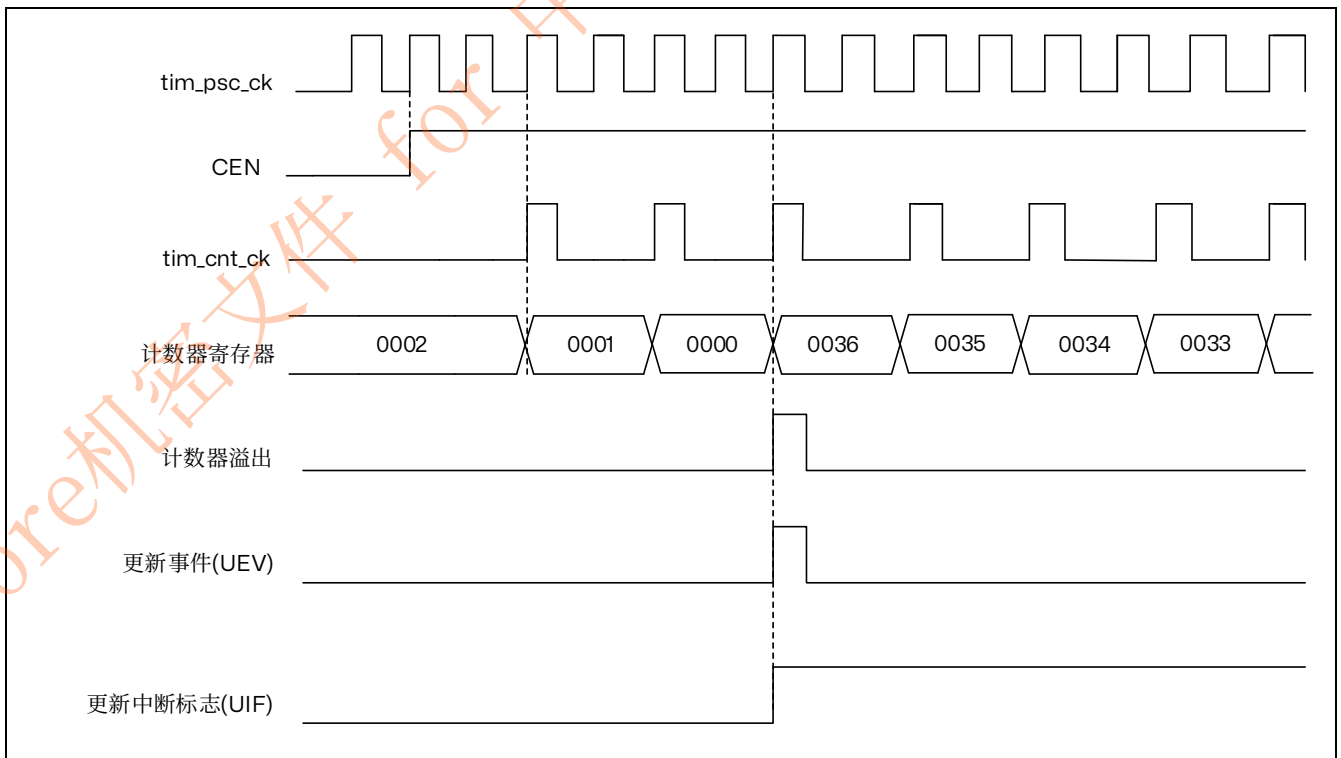


图 23.11 计数器时序图，内部时钟分频系数为 2

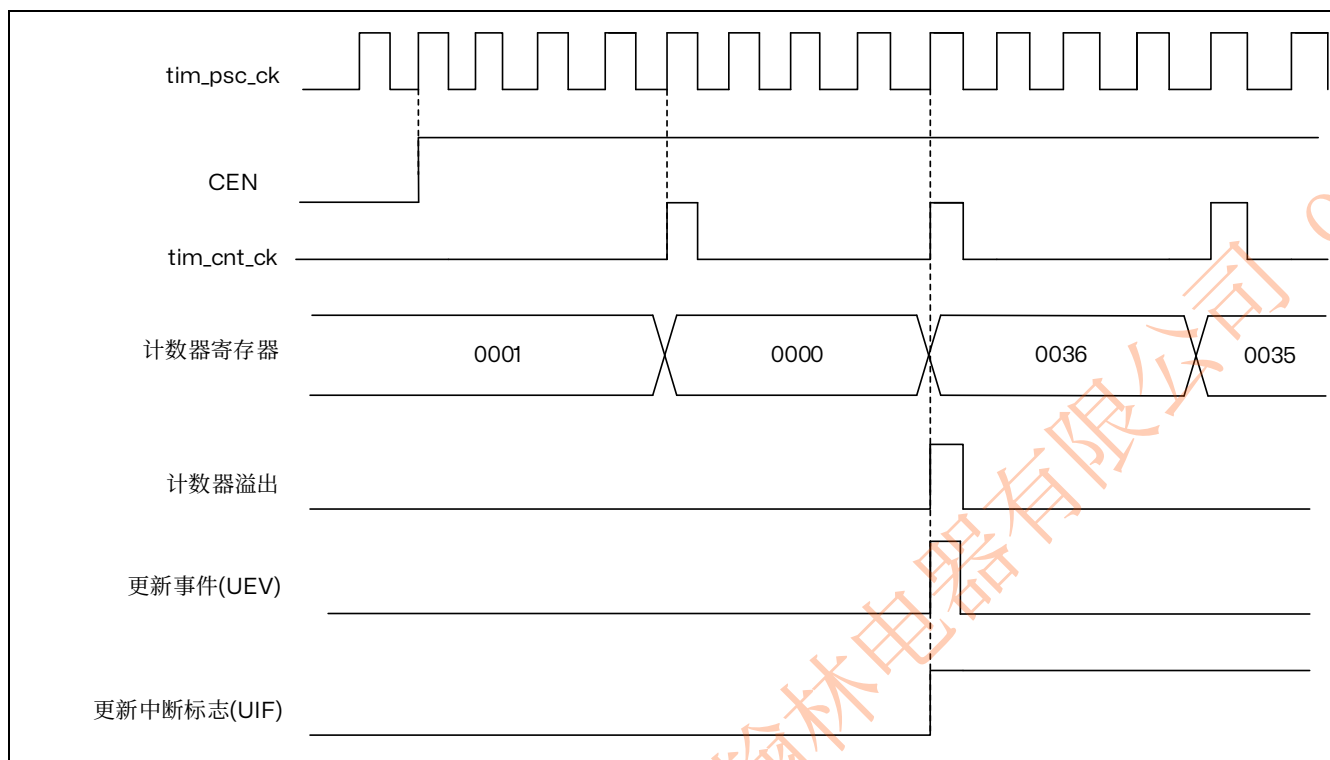


图 23.12 计数器时序图，内部时钟分频系数为 4

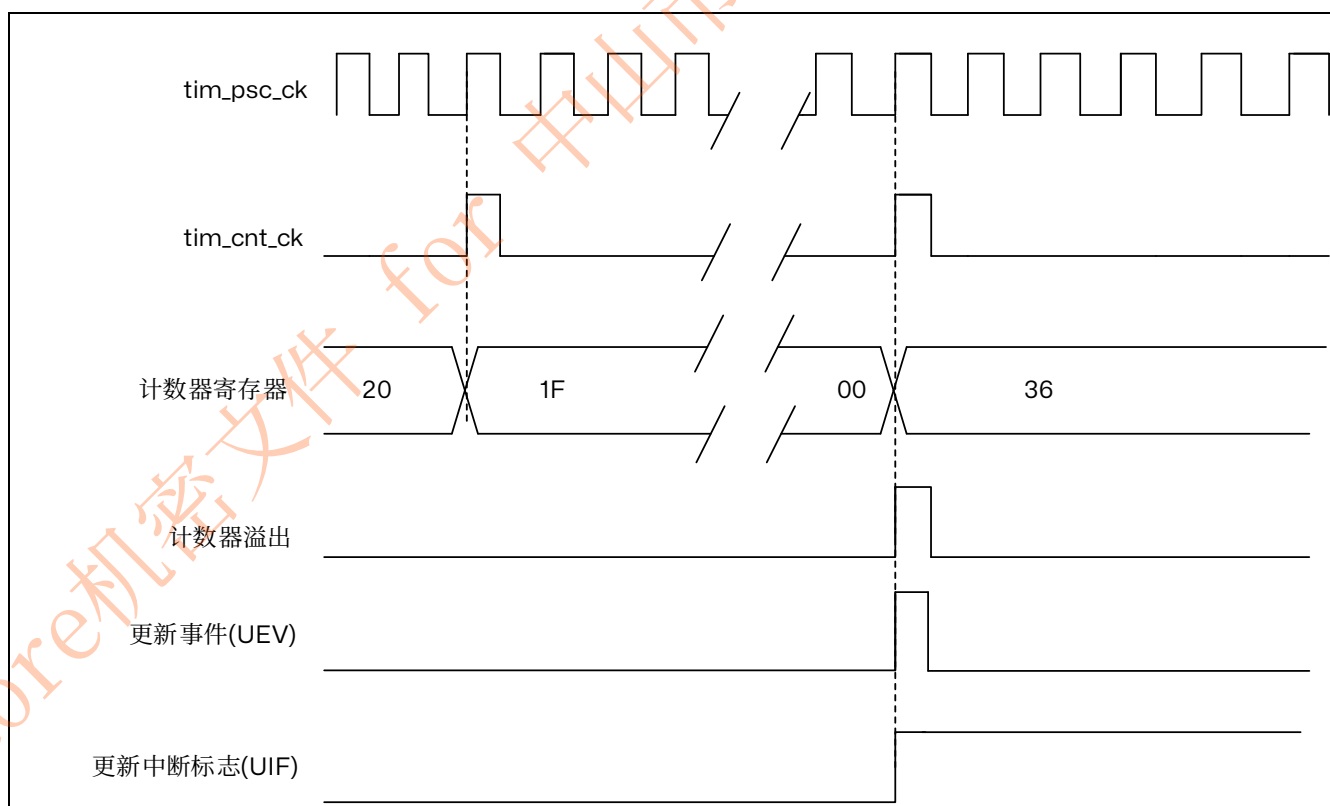


图 23.13 计数器时序图，内部时钟分频系数为 N

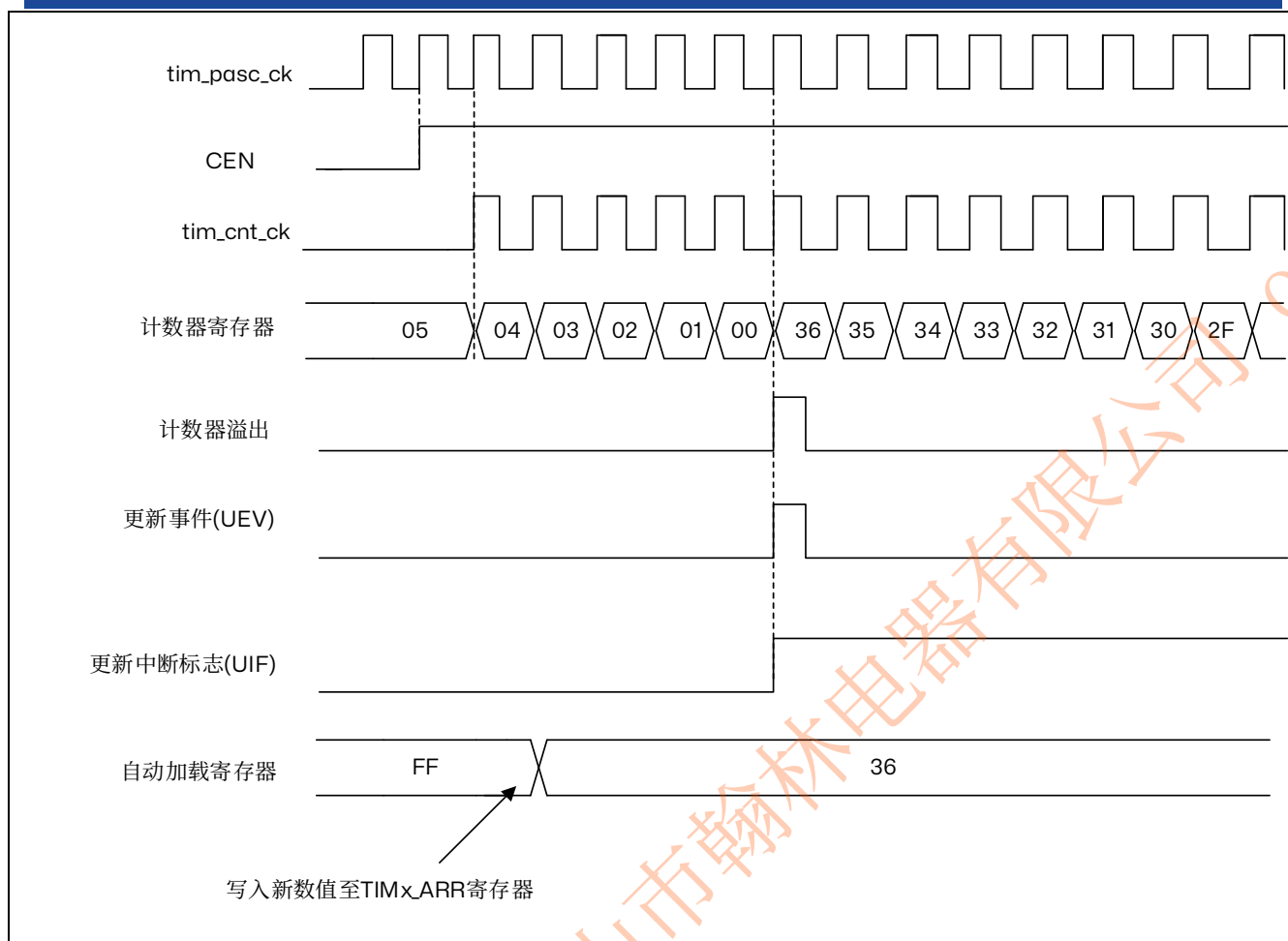


图 23.14 计数器时序图，当没有使用重复计数器时的更新事件

### 中心对齐模式（递增/递减计数）

在中心对齐模式下，计数器从 0 开始计数到自动重载值（TIMx\_ARR 寄存器的内容）- 1，生成计数器上溢事件；然后从自动重载值开始向下计数到 1 并生成计数器下溢事件。之后从 0 开始重新计数。

当 TIMx\_CR1 寄存器中的 CMS 位不为“00”时，中心对齐模式有效。将通道配置为输出模式时，其输出比较中断标志将在以下模式下置 1，即：计数器递减计数（中心对齐模式 1，CMS = “01”）、计数器递增计数（中心对齐模式 2，CMS = “10”）以及计数器递增/递减计数（中心对齐模式 3，CMS = “11”）。

在此模式下，TIMx\_CR1 寄存器的 DIR 方向位不可写入值，而是由硬件更新并指示当前计数器方向。

每次发生计数器上溢和下溢时都会生成更新事件，或将 TIMx\_EGR 寄存器中的 UG 位置 1（通过软件或使用从模式控制器）也可以生成更新事件。这种情况下，计数器以及预分频器计数器将重新从 0 开始计数。

通过软件将 TIMx\_CR1 寄存器中的 UDIS 位置 1 可禁止 UEV 更新事件。这可避免向预装载寄存器写入新值时更新影子寄存器。在 UDIS 位写入 0 之前不会产生任何更新事件。不过，计数器仍会根据当前自动重载值进行递增和递减计数。

此外，如果 TIMx\_CR1 寄存器中的 URS 位（更新请求选择）已置 1，则将 UG 位置 1 会生成 UEV 更新事件，但不会将 UIF 标志置 1（因此，不会发送任何中断或 DMA 请求）。这样一来，如果在发生捕获事件时将计数器清零，将不会同时产生更新中断和捕获中断。

发生更新事件时，将更新所有寄存器且将更新标志（TIMx\_SR 寄存器中的 UIF 位）置 1（取决于

- 重复计数器中将重新装载 TIMx\_RCR 寄存器的内容
- 预分频器的缓冲区中将重新装载预装载值 (TIMx\_PSC 寄存器的内容)
- 自动重载活动寄存器将以预装载值 (TIMx\_ARR 寄存器的内容) 进行更新。注意, 如果更新操作是由计数器上溢触发的, 则自动重载寄存器在重载计数器之前更新, 因此, 下一个计数周期就是我们所希望的新的周期长度 (计数器被重载新的值)。

The diagram illustrates the timing sequence for the TIM2 counter. It shows the following signals and their behavior:

- tim\_psc\_ck**: Prescaler clock, a periodic square wave.
- CEN**: Counter Enable, which starts low and transitions high at a specific point.
- tim\_cnt\_ck**: Counter clock, which starts when CEN goes high and follows the prescaler clock frequency.
- 计数器寄存器** (Counter Register): Shows the sequence of values from 04 down to 00, then 01 through 06, and back down to 03. The values are shown in a sequence of trapezoidal shapes.
- 计数器下溢** (Counter Underflow): A pulse that occurs when the counter reaches 00.
- 计数器溢出** (Counter Overflow): A pulse that occurs when the counter reaches 06.
- 更新事件(UEV)** (Update Event): A pulse that occurs when the counter reaches 06.
- 更新中断标志(UIF)** (Update Interrupt Flag): A pulse that occurs when the counter reaches 06.

图 23.15 计数器时序图，内部时钟分频系数为1，TIMx\_ARR=0x6

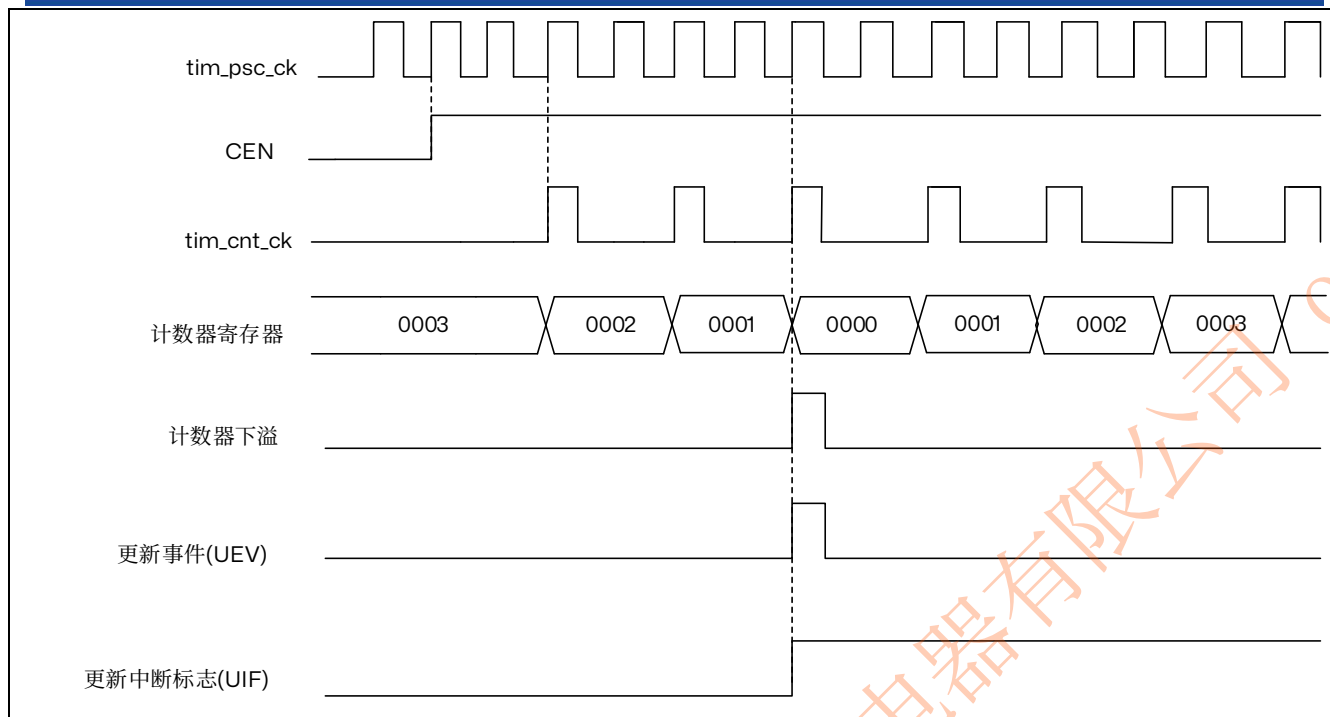


图 23.16 计数器时序图，内部时钟分频系数为 2

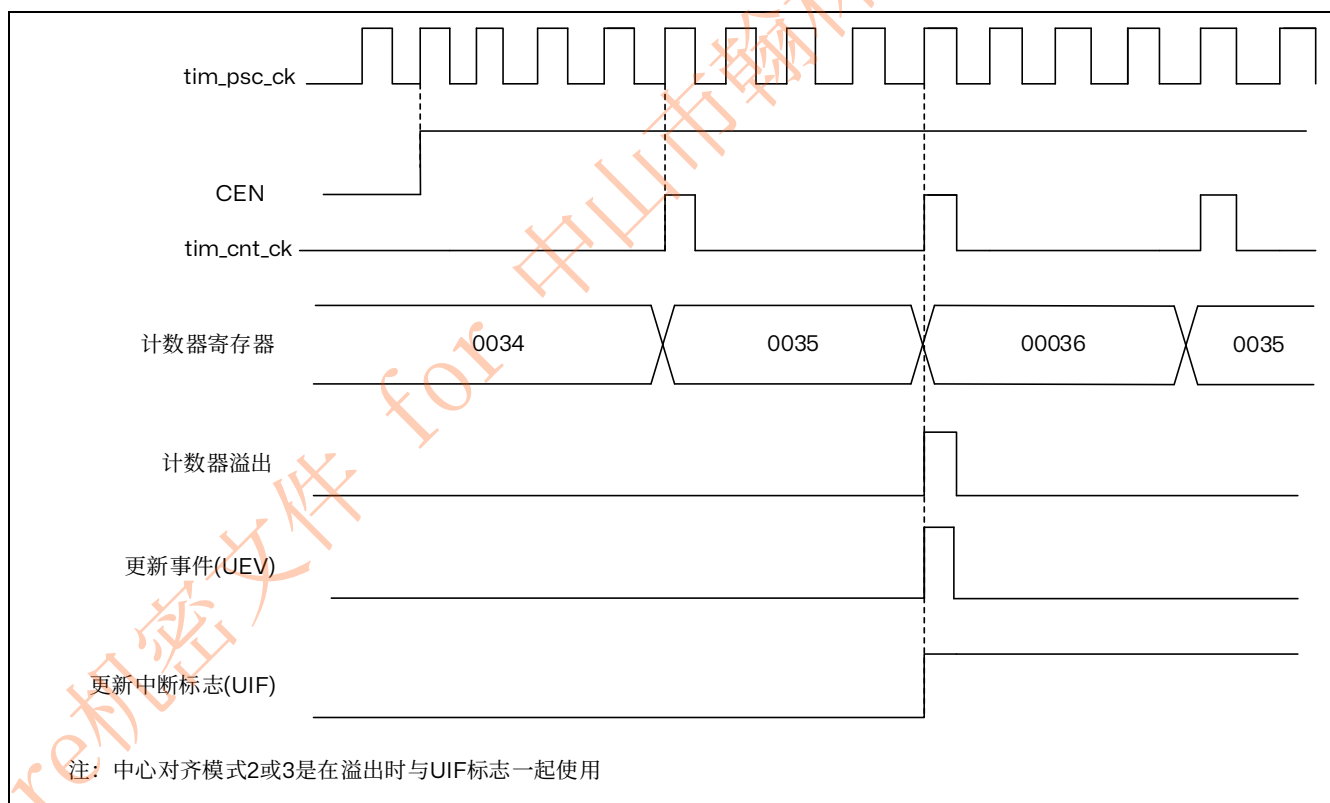


图 23.17 计数器时序图，内部时钟分频系数为 4，TIMx\_ARR=0x36

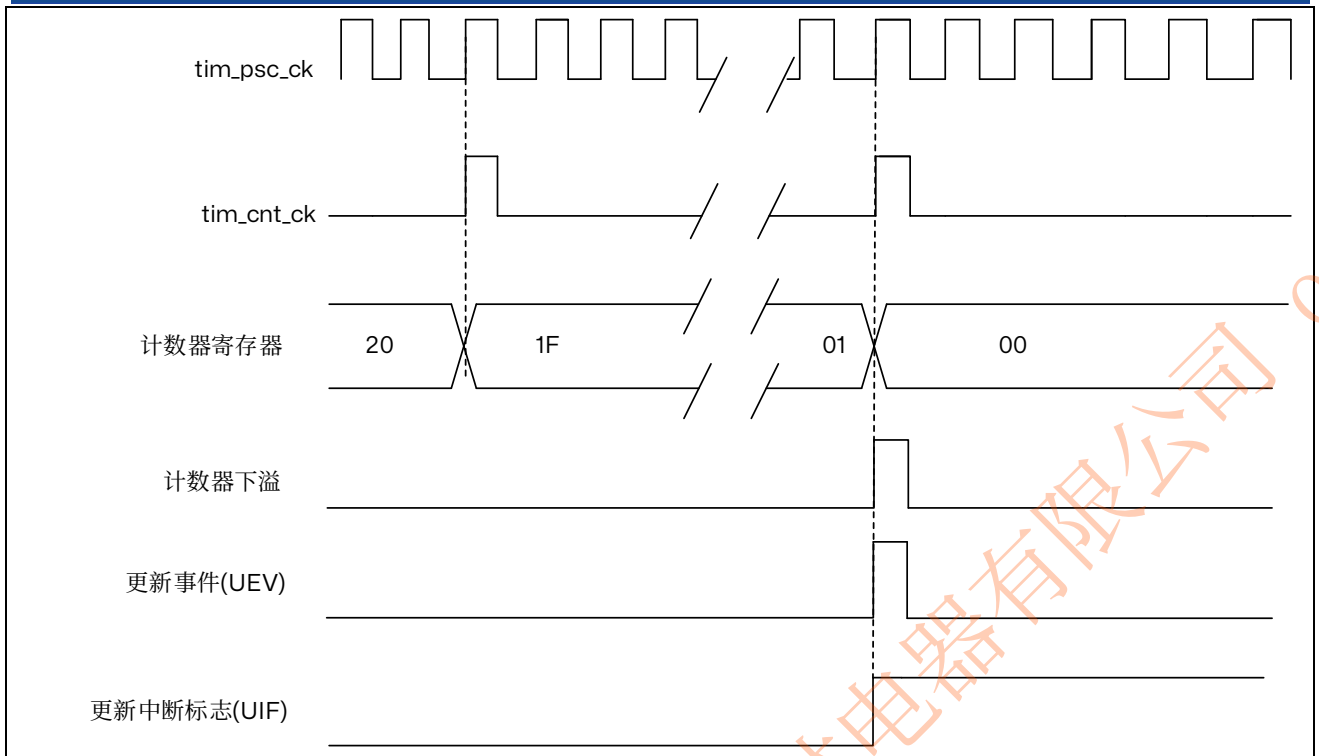


图 23.18 计数器时序图，内部时钟分频系数为 N

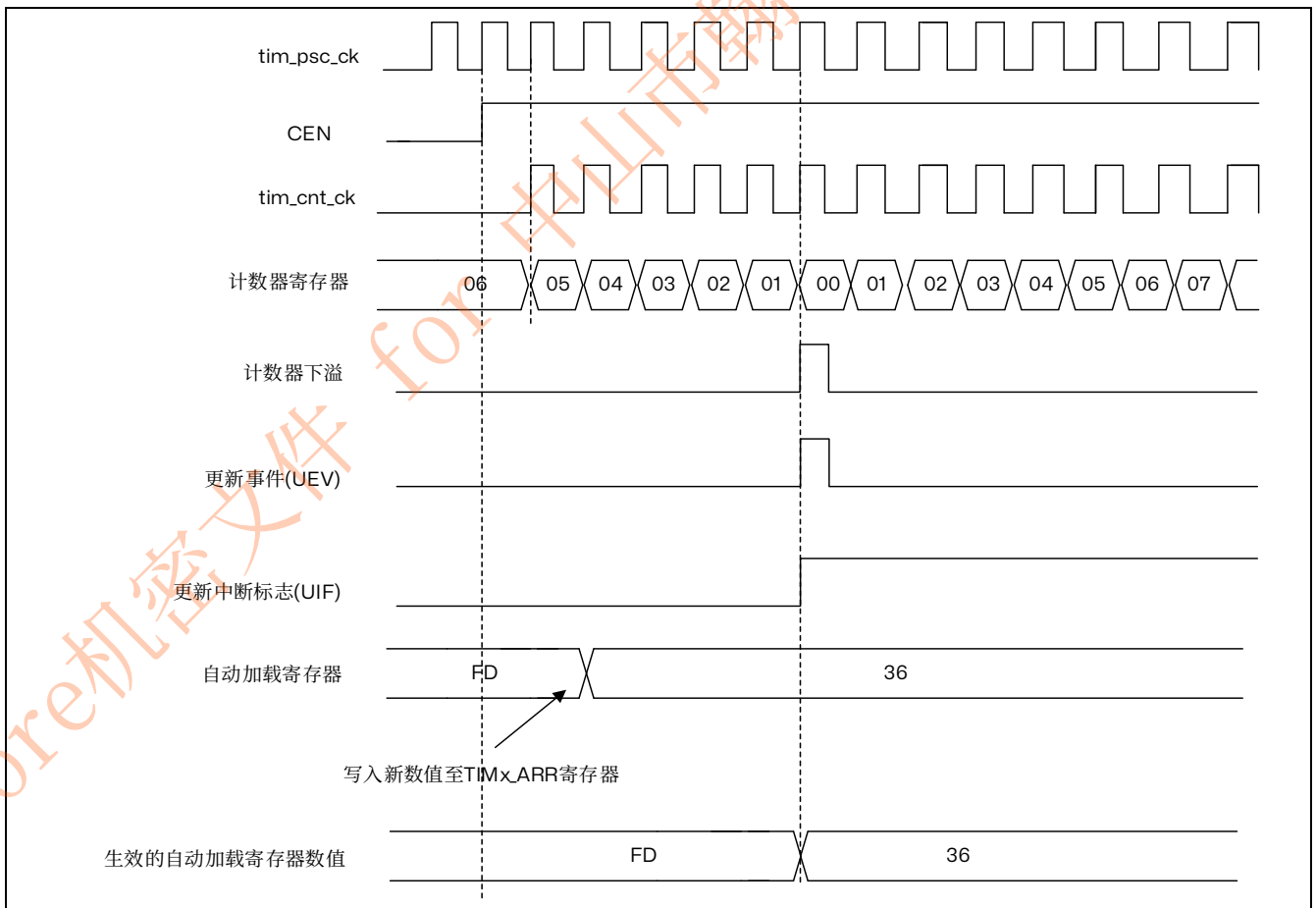


图 23.19 计数器时序图，ARPE=1 时的更新事件（计数器下溢）

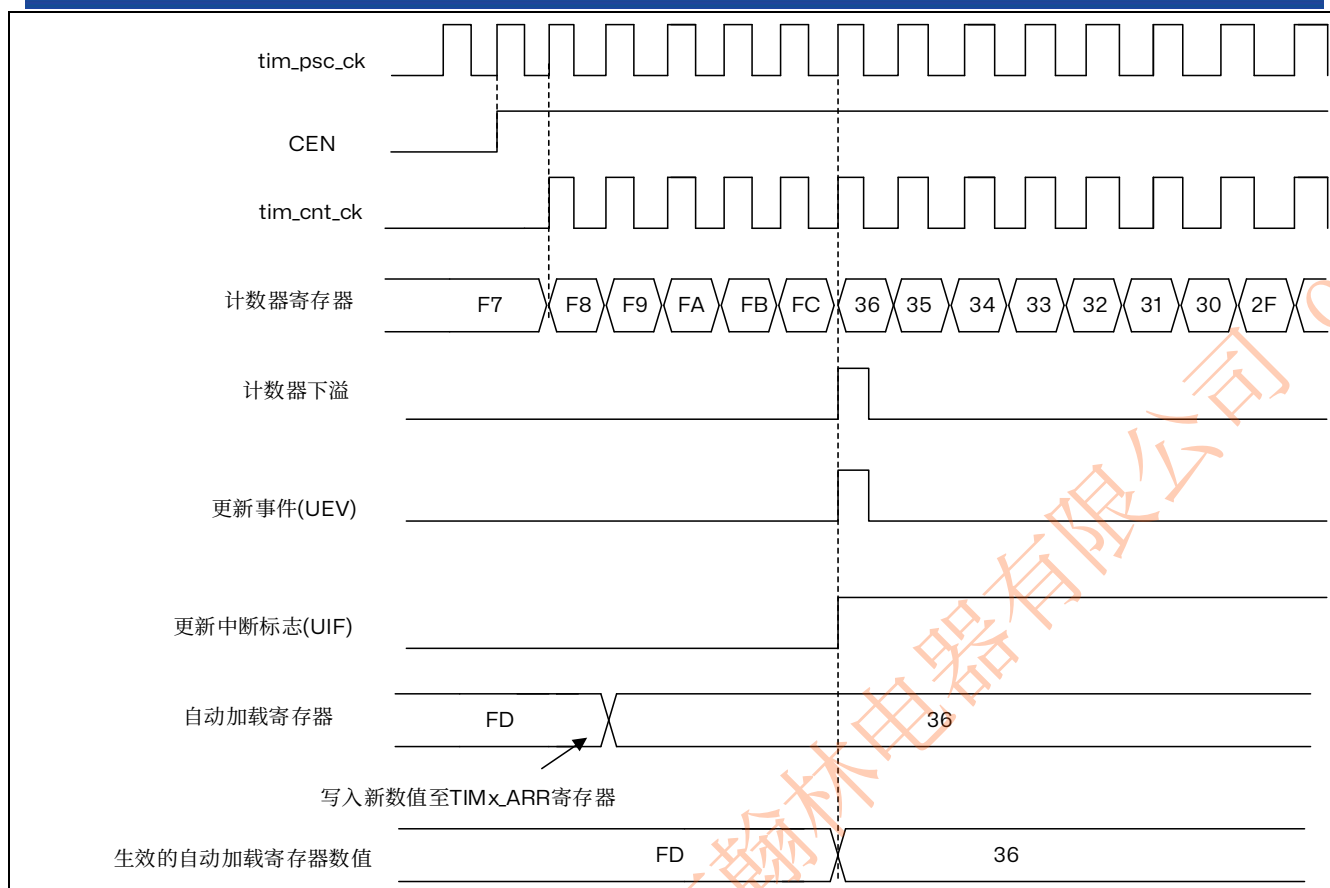


图 23.20 计数器时序图，ARPE=1 时的更新事件（计数器上溢）

### 23.3.5 重复计数器

时基单元介绍如何因计数器上溢/下溢而生成更新事件(UEV)。实际上，只有当重复计数器达到零时，才会生成更新事件。这在生成 PWM 信号时很有用。

这意味着，每当发生 N+1 个计数器上溢或下溢（其中，N 是 TIMx\_RCR 重复计数器寄存器中的值），数据就将从预装载寄存器转移到影子寄存器（TIMx\_ARR 自动重载寄存器、TIMx\_PSC 预分频器寄存器以及比较模式下的 TIMx\_CCRx 捕获/比较寄存器）。

重复计数器在下列情况下递减：

- 递增计数模式下的每个计数器上溢。
- 递减计数模式下的每个计数器下溢。
- 中心对齐模式下每个计数器上溢和计数器下溢。

尽管这限制了最大重复次数为 32768 个 PWM 周期，但它使得每个 PWM 周期可以更新两次占空比。在中心对齐模式下，每个 PWM 周期仅刷新一次比较寄存器时，由于模式的对称性，最大分辨率为  $2 \times T_{ck}$ 。

重复计数器是自动重载类型；其重复率为 TIMx\_RCR 寄存器所定义的值。当更新事件由软件（通过将 TIMx\_EGR 寄存器的 UG 位置 1）或硬件（通过从模式控制器）生成时，无论重复计数器的值为多少，更新事件都将立即发生，并且在重复计数器中重新装载 TIMx\_RCR 寄存器的内容。

在中心对齐模式下，如果 RCR 值为奇数，更新事件将在上溢或下溢时发生，这取决于何时写入 RCR 寄存器以及何时启动计数器。如果在启动计数器前写入 RCR，则 UEV 在上溢时发生。如果在启动计数器后写入 RCR，则 UEV 在下溢时发生。



例如，如果  $RCR = 3$ ，UEV 将在每个周期的第四个上溢或下溢事件时生成（取决于何时写入 RCR）。

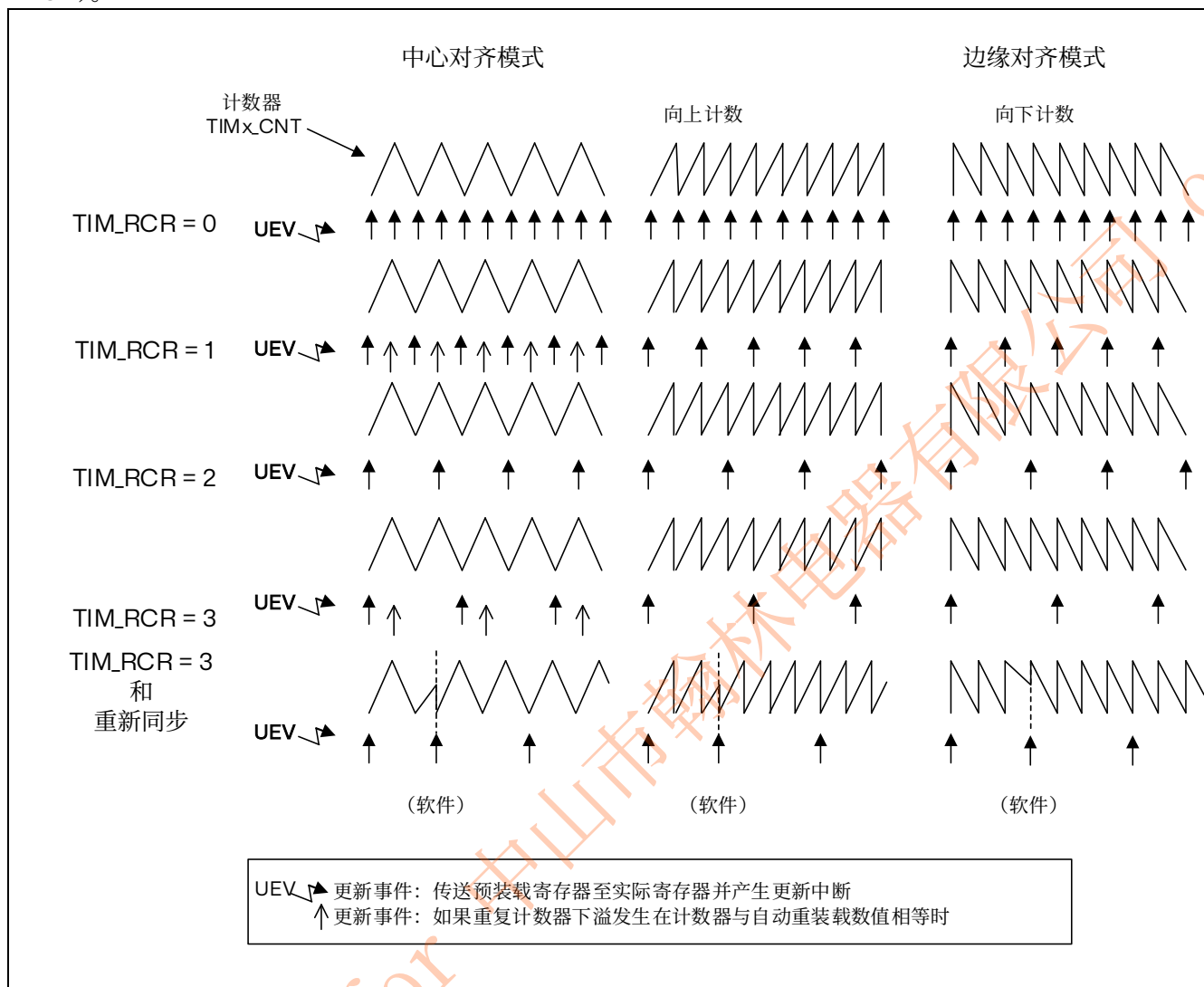


图 23.21 不同模式下更新速率的例子，及 TIMx\_RCR 的寄存器设置

### 23.3.6 外部触发输入

定时器具有一个外部触发输入  $tim\_etr\_in$ 。它可以用于：

- 外部时钟（外部时钟模式 2）
- 从模式触发
- PWM 重置输入，用于逐周期电流调节

下图描述了  $tim\_etr\_in$  输入调节。输入极性由 TIMxSMCR 寄存器的 ETP 位定义。触发器可以使用 ETPS[1:0] 位域编程的分频器进行预分频，并使用 ETF[3:0] 位域进行数字滤波。所得信号 ( $tim\_etrf$ ) 可用于三种目的：作为外部时钟、用于输出调节（通常用于重置 PWM 输出以进行电流限制）以及作为从模式控制器的触发器。

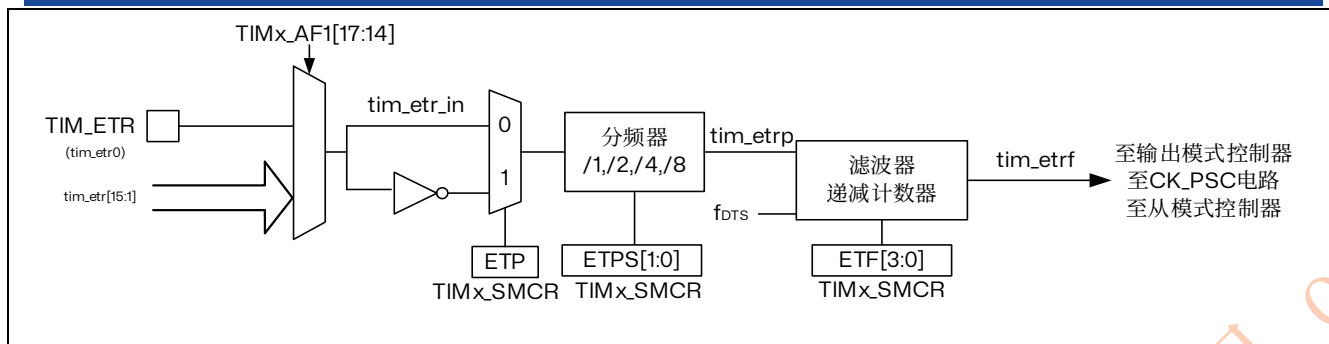


图 23.22 外部触发输入框图

tim\_etr\_in 输入来自多个来源：输入引脚（默认配置）或内部源。选择是通过 TIMx\_AF1 寄存器中的 ETRSEL[3:0] 位域完成的。

有关连接到 etr\_in 输入的源列表，请参考之前的章节：TIM1/TIM8 引脚和内部信号。

### 23.3.7 时钟选择

计数器时钟可由下列时钟源提供：

- 内部时钟 (tim\_ker\_ck)
- 外部时钟模式 1：外部输入引脚 (tim\_ti1 or tim\_ti2)
- 外部时钟模式 2：外部触发输入 (tim\_etr\_in)
- 外编码器模式

#### 内部时钟源 (tim\_ker\_ck)

如果禁止从模式控制器 (SMS=000)，则 CEN 位、DIR 位 (TIMx\_CR1 寄存器中) 和 UG 位 (TIMx\_EGR 寄存器中) 为实际控制位，并且只能通过软件进行更改 (UG 除外，仍保持自动清零)。当对 CEN 位写入 1 时，预分频器的时钟就由内部时钟 tim\_ker\_ck 提供。

下图显示了正常模式下控制电路与递增计数器的行为（没有预分频的情况下）。

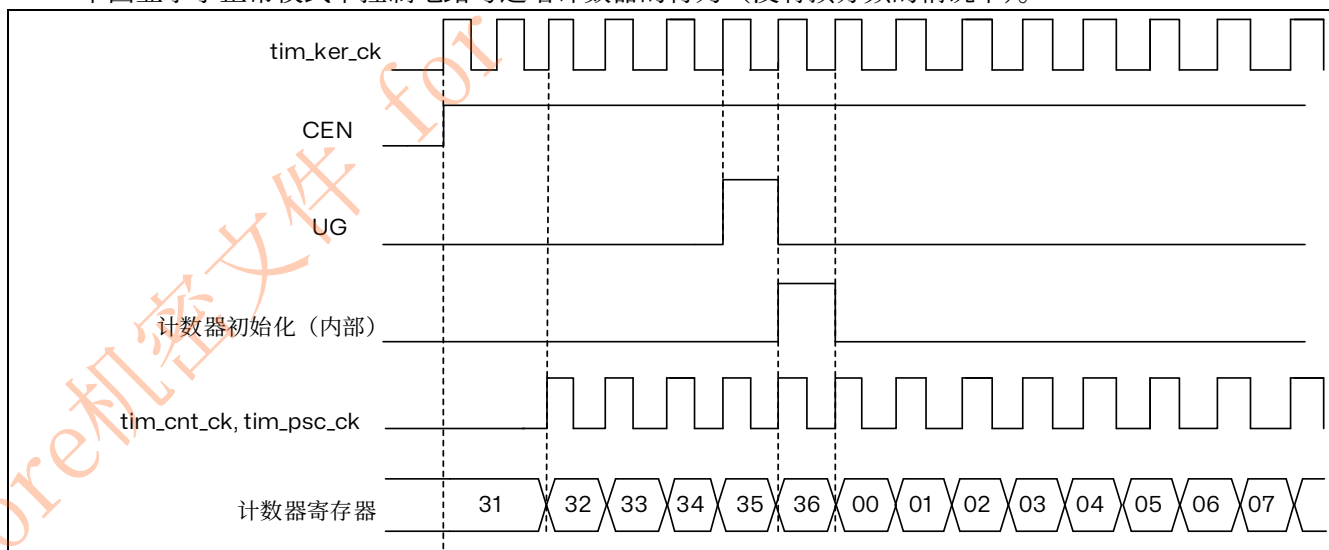


图 23.23 一般模式下的控制电路，内部时钟分频系数为 1

#### 外部时钟源模式 1

The diagram illustrates the TIMx\_TI2 input channel (CH2) architecture. It shows the signal path from the input `tim_ti2_in0` through a multiplexer (controlled by `TIMx_TISEL` and `TI2SEL[3:0]`) to a filter (controlled by `ICF[3:0]` in `TIMx_CCMR1`). The filtered signal `tim_ti2` is then processed by an edge detector (controlled by `CC2P` in `TIMx_CCER`) to generate `tim_ti2f_rising` and `tim_ti2f_falling` signals. These signals are fed into a multiplexer (controlled by `TIMx_SMCR` and `TS[4:0]`) which also receives `tim_itr`, `tim_ti1_ed`, `tim_ti1fp1`, `tim_ti2fp2`, and `etr` signals. The output of this multiplexer is `tim_trgi` and `tim_etr`, which are then used to generate the `tim_psc_ck` (prescaler clock) and `tim_ker_ck` (internal clock) signals. The `tim_psc_ck` is also controlled by `ECE` and `SMS[2:0]` in `TIMx_SMCR`.

1. 编码范围从01000 至11111被保留。

图 23.24 外部触发输入框图

1. 通过在 TIMx\_CCMR1 寄存器中写入 CC2S = “01” 来配置通道 2，使其能够检测 tim\_ti2 输入的上升沿。
2. 通过在 TIMx\_CCMR1 寄存器中写入 IC2F[3:0] 位来配置输入滤波时间（如果不需要任何滤波，请保持 IC2F=0000）。
3. 通过在 TIMx\_CCER 寄存器中写入 CC2P=0 和 CC2NP=0 来选择上升沿极性。
4. 通过在 TIMx\_SMCR 寄存器中写入 SMS=111，使定时器在外部时钟模式 1 下工作。
5. 通过在 TIMx\_SMCR 寄存器中写入 TS=00110 来选择 tim\_ti2 作为触发输入源。
6. 通过在 TIMx\_CR1 寄存器中写入 CEN=1 来使能计数器。

当tim\_ti2 出现上升沿时, 计数器便会计数一次并且TIF 标志置1。

tim\_ti2 的上升沿与实际计数器时钟之间的延迟是由于tim\_ti2 输入的重新同步电路引起的。

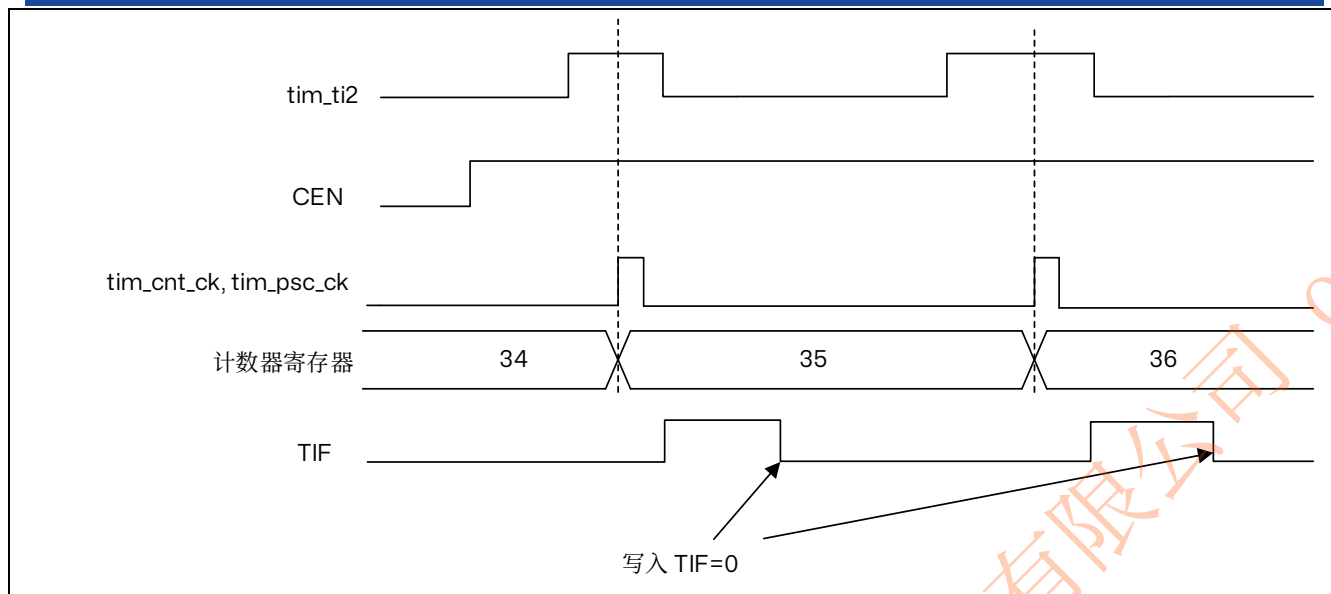


图 23.25 外部时钟模式 1 下的控制电路

## 外部时钟模式 2

通过在 TIMx\_SMCR 寄存器中写入 ECE=1 可选择此模式。

计数器可在外部触发输入 tim\_etr\_in 出现上升沿或下降沿时计数。

下图介绍了外部触发输入模块。

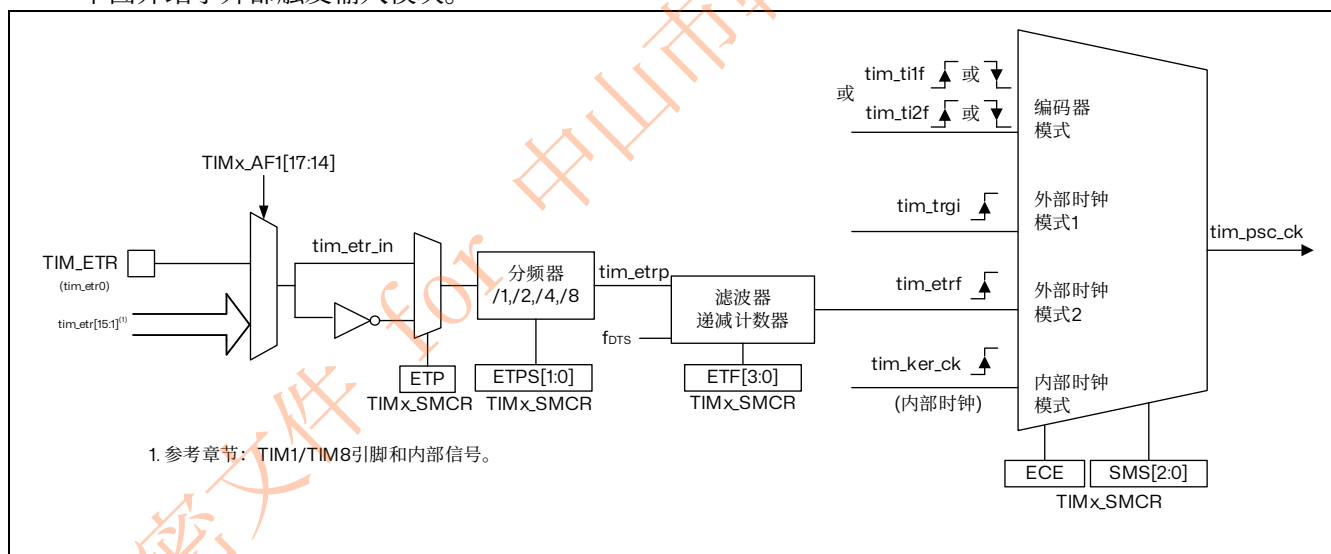


图 23.26 外部触发输入框图

例如，要使递增计数器在 tim\_etr\_in 每出现 2 个上升沿时计数，请执行以下步骤：

1. 由于此例中不需滤波器，因此在 TIMx\_SMCR 寄存器中写入 ETF[3:0]=0000。
  2. 通过在 TIMx\_SMCR 寄存器中写入 ETPS[1:0]=01 来设置预分频器。
  3. 通过在 TIMx\_SMCR 寄存器中写入 ETP=0 来选择 ETR 引脚的上升沿检测。
  4. 通过在 TIMx\_SMCR 寄存器中写入 ECE=1 来使能外部时钟模式 2。
  5. 通过在 TIMx\_CR1 寄存器中写入 CEN=1 来使能计数器。
- tim\_etr\_in 每出现 2 个上升沿，计数器计数一次。

在 `tim_etr_in` 的上升沿和计数器实际时钟之间的延时取决于在 `tim_etrp` 信号端的重新同步电路。因此，计数器可以正确捕获的最大频率最高为 `tim_ker_ck` 频率的 $\frac{1}{4}$ 。当 `ETRP` 信号更快时，用户应通过适当的 `ETPS` 预分频器设置对外部信号进行分频。

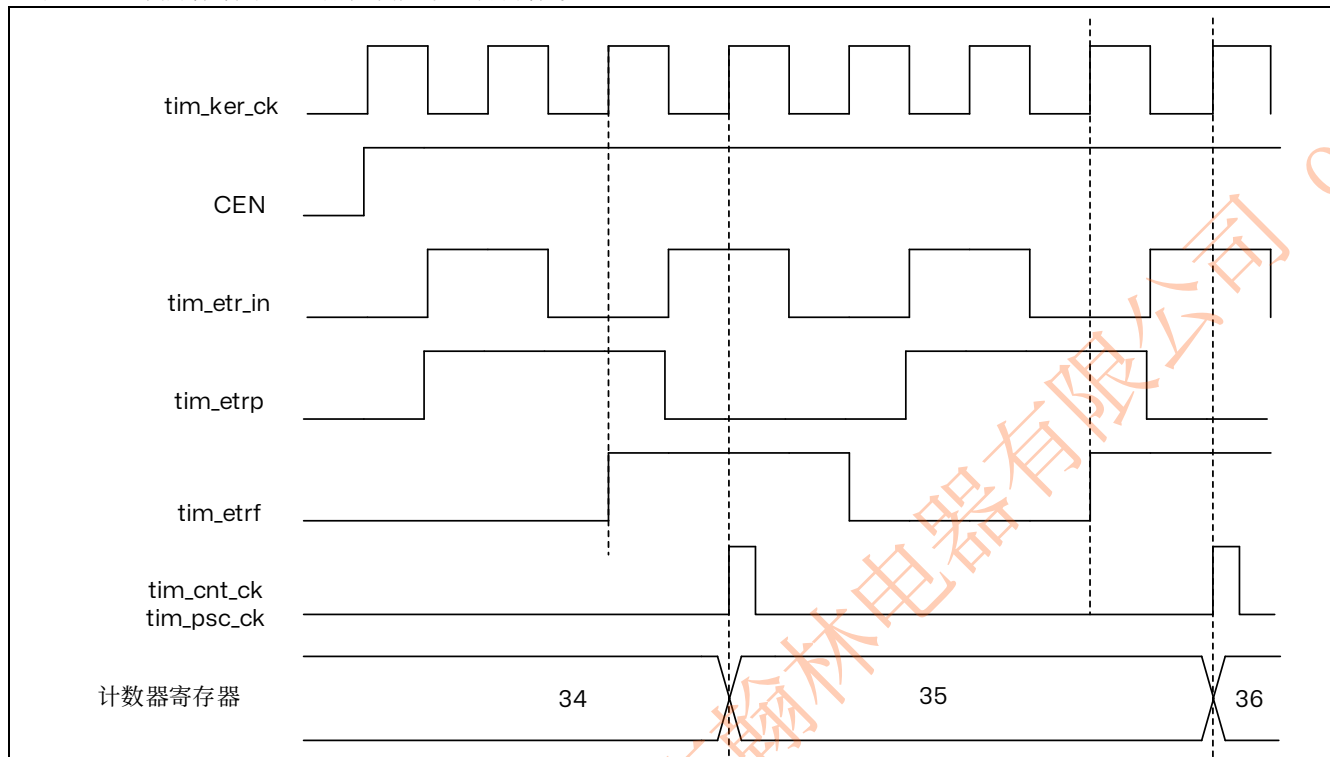


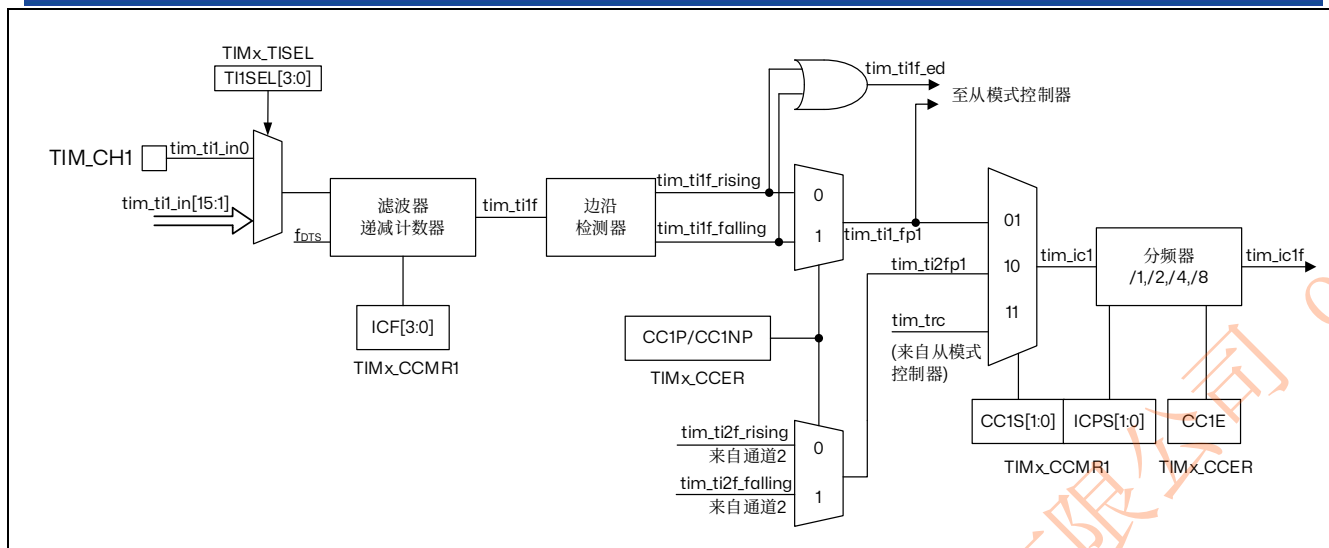
图 23.27 外部时钟模式 2 下的控制电路

### 23.3.8 捕获比较通道

每个捕获/比较通道均围绕一个捕获/比较寄存器（包括一个影子寄存器）、一个捕获输入阶段（包括数字滤波、多路复用和预分频器，但通道 5 和通道 6 除外）和一个输出阶段（比较器和输出控制）构建而成。

下列图提供了一个捕获/比较通道的概述。

输入阶段对相应的 `tim_tix` 输入进行采样，生成一个滤波后的信号 `tim_tixf`。然后，带有极性选择功能的边沿检测器生成一个信号（`tim_tixfpy`），该信号可用作从模式控制器的触发输入，也可用作捕获命令。该信号先进行预分频（`ICxPS`），而后再进入捕获寄存器。



输出阶段生成一个中间波形作为基准：tim\_ocxref（高电平有效）。链的末端决定最终输出信号的极性。

图 23.28 捕获/比较通道（如：通道 1 输入部分）

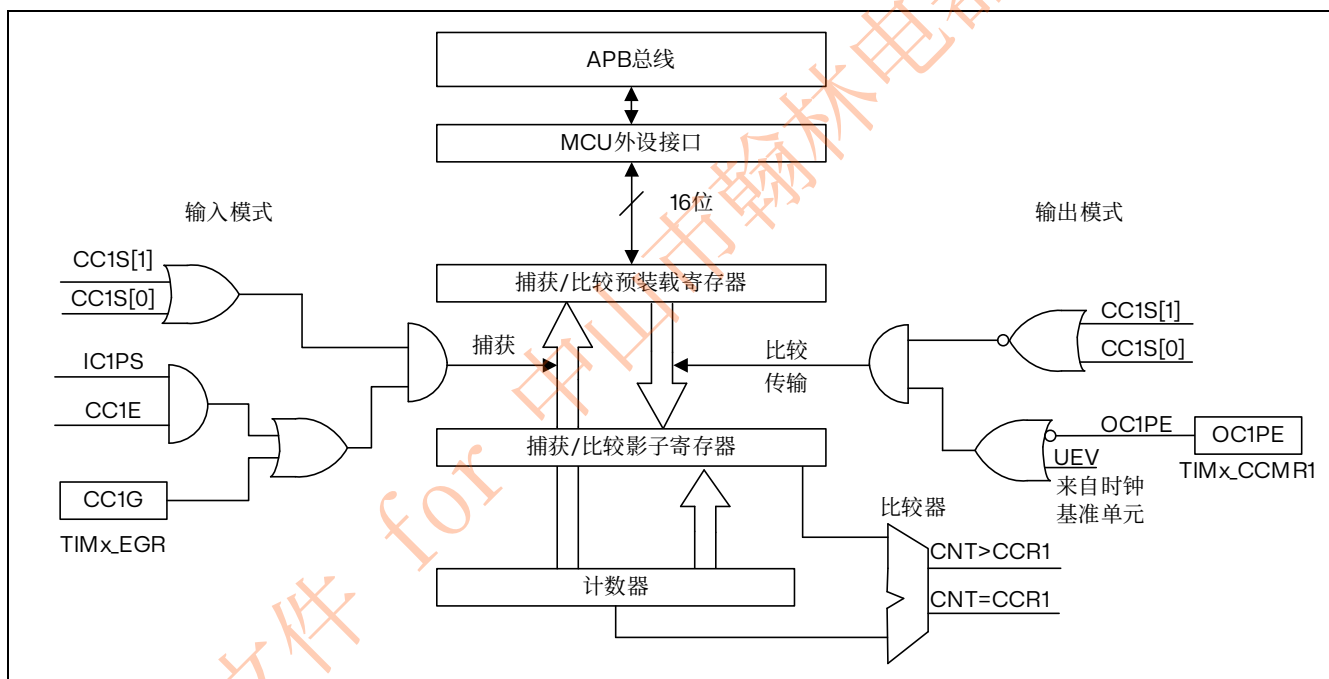
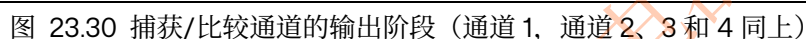


图 23.29 捕获/比较通道 1 的主电路



在捕获模式下，捕获实际发生在影子寄存器中，然后将影子寄存器的内容复制到预装载寄存器中。

在比较模式下，预装载寄存器的内容将被复制到影子寄存器中，然后将影子寄存器的内容与计数器进行比较。



### 23.3.9 输入捕获模式

在输入捕获模式下，当相应的 ICx 信号检测到跳变沿后，将使用捕获/比较寄存器(TIMx\_CCRx) 来锁存计数器的值。发生捕获事件时，会将相应的 CCxIF 标志 (TIMx\_SR 寄存器) 置 1，并可发送中断或 DMA 请求 (如果已使能)。如果发生捕获事件时 CCxIF 标志已处于高位，则会将重复捕获标志 CCxOF (TIMx\_SR 寄存器) 置 1。可通过软件向 CCxIF 写入 0 来给 CCxIF 清零，或读取存储在 TIMx\_CCRx 寄存器中的已捕获数据。向 CCxOF 写入 0 后会将其清零。

以下示例说明了如何在 tim\_ti1 输入出现上升沿时将计数器的值捕获到 TIMx\_CCR1 中。具体操作步骤如下：

1. 选择有效输入：TIMx\_CCR1 必须连接到 tim\_ti1 输入，因此向 TIMx\_CCMR1 寄存器中的 CC1S 位写入 01。只要 CC1S 不等于 00，就会将通道配置为输入模式，并且 TIMx\_CCR1 寄存器将处于只读状态。
2. 根据连接到定时器的信号，对所需的输入滤波时间进行编程 (如果输入为 tim\_tix 输入之一，则对 TIMx\_CCMRx 寄存器中的 ICxF 位进行编程)。假设信号变化时，输入信号最多在 5 个内部时钟周期内发生抖动。因此，我们必须将滤波时间设置为大于 5 个内部时钟周期。在检测到 8 个具有新电平的连续采样 (以  $f_{DTS}$  频率采样) 后，可以确认 tim\_ti1 上的跳变沿。然后向 TIMx\_CCMR1 寄存器中的 IC1F 位写入 0011。
3. 通过向 TIMx\_CCER 寄存器中的 CC1P 位和 CC1NP 位写入 0，选择 tim\_ti1 通道的有效转换边沿 (本例中为上升沿)。
4. 对输入预分频器进行编程。在本例中，我们希望每次有效转换时都执行捕获操作，因此需要禁止预分频器 (向 TIMx\_CCMR1 寄存器中的 IC1PS 位写入 00)。
5. 通过将 TIMx\_CCER 寄存器中的 CC1E 位置 1，允许将计数器的值捕获到捕获寄存器中。
6. 如果需要，可通过将 TIMx\_DIER 寄存器中的 CC1IE 位置 1 来使能相关中断请求，和/或通过 TIMx\_DIER 寄存器中的 CC1DE 位置 1 来使能相关 DMA 请求。

发生输入捕获时：

- 发生有效跳变沿时，TIMx\_CCR1 寄存器会获取计数器的值。
- 将 CC1IF 标志置 1 (中断标志)。如果至少发生了两次连续捕获，但 CC1IF 标志未被清零，这样 CC1OF 捕获溢出标志会被置 1。
- 根据 CC1IE 位生成一个中断。
- 根据 CC1DE 位生成一个 DMA 请求

要处理重复捕获，建议在读出捕获溢出标志之前读取数据。这样可避免丢失在读取捕获溢出标志之后与读取数据之前可能出现的重复捕获信息。

注：通过软件将 TIMx\_EGR 寄存器中的相应 CCxG 位置 1 可生成 IC 中断和/或 DMA 请求。

### 23.3.10 PWM 输入模式

此模式允许测量连接到单个 tim\_tix 输入的 PWM 信号的周期和占空比：

- TIMx\_CCR1 寄存器保持周期值 (两个连续上升沿之间的间隔)。
- TIMx\_CCR2 寄存器保持脉冲宽度 (两个连续上升沿和下降沿之间的间隔)。

此模式是输入捕获模式的一个特例。其实现步骤与输入捕获模式基本相同，仅存在以下不同之处：

- 两个 ICx 信号被映射至同一个 tim\_tixfp1 输入。
- 这两个 ICx 信号在边沿处有效，但极性相反。
- 选择两个 tim\_tixfp 信号之一作为触发输入，并将从模式控制器配置为复位模式。

可通过以下步骤测量施加在 tim\_ti1 上的 PWM 信号的周期和脉冲宽度：

1. 选择 TIMx\_CCR1 的有效输入：向 TIMx\_CCMR1 寄存器中的 CC1S 位写入 01（选择 tim\_ti1）。
2. 选择 tim\_ti1fp1 的有效极性（用于 TIMx\_CCR1 中的捕获和计数器清零）：向 CC1P 位和 CC1NP 位写入 “0”（上升沿有效）。
3. 选择 TIMx\_CCR2 的有效输入：向 TIMx\_CCMR1 寄存器中的 CC2S 写入 10（选择 tim\_ti1）。
4. 选择 tim\_ti1fp2 的有效极性（用于 TIMx\_CCR2 中的捕获）：向 CC2P 位写入 1 和 CC2NP 位写入 “10”（下降沿有效）。
5. 选择有效触发输入：向 TIMx\_SMCR 寄存器中的 TS 位写入 00101（选择 tim\_ti1fp1）。
6. 将从模式控制器配置为复位模式：向 TIMx\_SMCR 寄存器中的 SMS 位写入 0100。
7. 使能捕获：向 TIMx\_CCER 寄存器中的 CC1E 位和 CC2E 位写入 “1”。

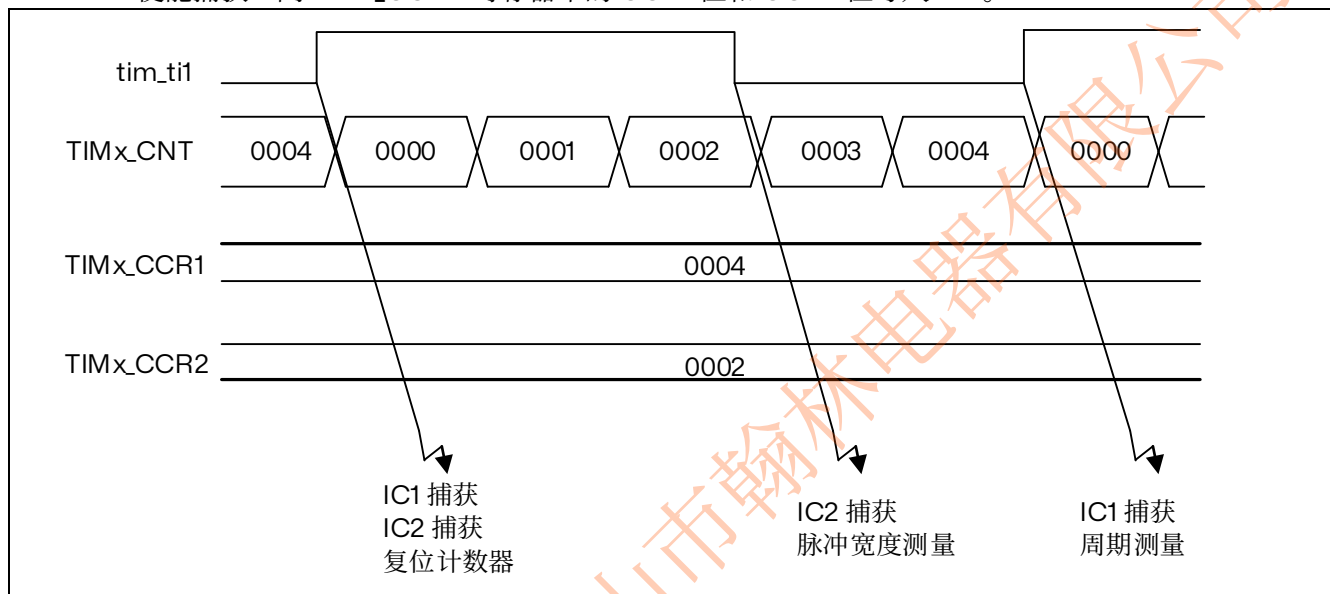


图 23.32 PWM 输入模式时序

### 23.3.11 强制输出模式

在输出模式（TIMx\_CCMRx 寄存器中的 CCxS 位 = 00）下，可直接由软件将每个输出比较信号（tim\_ocxref 和 tim\_ocx/tim\_ocxn）强制设置为有效电平或无效电平，而无需考虑输出比较寄存器和计数器之间的任何比较结果。

要将输出比较信号（tim\_ocxref/tim\_ocx）强制设置为有效电平，只需向相应 TIMx\_CCMRx 寄存器中的 OCxM 位写入 0101。tim\_ocxref 进而强制设置为高电平（tim\_ocxref 始终为高电平有效），同时 tim\_ocx 获取 CCxP 极性位的相反值。

例如：CCxP=0（tim\_ocx 高电平有效）=> tim\_ocx 强制设置为高电平。

通过向 TIMx\_CCMRx 寄存器中的 OCxM 位写入 100，可将 tim\_ocxref 信号强制设置为低电平。无论如何，TIMx\_CCRx 影子寄存器与计数器之间的比较仍会执行，而且允许将标志置 1。因此可发送相应的中断和 DMA 请求。下面的输出比较模式一节对此进行了介绍。

### 23.3.12 输出比较模式

此功能用于控制输出波形，或指示已经过某一段时间。通道 1 至 4 可以输出，而通道 5 和 6 仅可在微控制器内部使用（例如，用于复合波形生成或 ADC 触发）。

当捕获/比较寄存器与计数器之间相匹配时，输出比较功能：

- 将为相应的输出引脚分配一个可编程值，该值由输出比较模式（TIMx\_CCMRx 寄存器中的

OCxM 位) 和输出极性 (TIMx\_CCER 寄存器中的 CCxP 位) 定义。匹配时, 输出引脚既可保持其电平 (OCXM=0000), 也可设置为有效电平(OCXM=0001)、无效电平(OCXM=0010)或进行翻转(OCXM=0011)。

- 将中断状态寄存器中的标志置 1 (TIMx\_SR 寄存器中的 CCxIF 位)。
- 如果相应中断使能位 (TIMx\_DIER 寄存器中的 CCXIE 位) 置 1, 将生成中断。
- 如果相应 DMA 使能位 (TIMx\_DIER 寄存器的 CCxDE 位, TIMx\_CR2 寄存器的 CCDS 位, 用来选择 DMA 请求) 置 1, 将发送 DMA 请求。

使用 TIMx\_CCMRx 寄存器中的 OCxPE 位, 可将 TIMx\_CCRx 寄存器配置为带或不带预装载寄存器。

在输出比较模式下, 更新事件 UEV 对 tim\_ocxref 和 tim\_ocx 出毫无影响。同步的精度可以达到计数器的一个计数周期。输出比较模式也可用于输出单脉冲 (在单脉冲模式下)。

步骤:

1. 选择计数器时钟 (内部、外部、预分频器)。
2. 在 TIMx\_ARR 和 TIMx\_CCRx 寄存器中写入所需数据。
3. 如果要生成中断请求, 则需将 CCxIE 位置 1。
4. 选择输出模式。例如:
  - 当 CNT 与 CCRx 匹配时, 写入 OCxM = 0011 以翻转 tim\_ocx 输出引脚
  - 写入 OCxPE = 0 以禁止预装载寄存器
  - 写入 CCxP = 0 以选择高电平有效极性
  - 写入 CCxE = 1 以使能输出
5. 通过将 TIMx\_CR1 寄存器中的 CEN 位置 1 来使能计数器。

可通过软件随时更新 TIMx\_CCRx 寄存器以控制输出波形, 前提是未使能预加载寄存器 (OCxPE=“0”, 否则仅当发生下一个更新事件 UEV 时, 才会更新 TIMx\_CCRx 影子寄存器)。下图给出了一个示例。

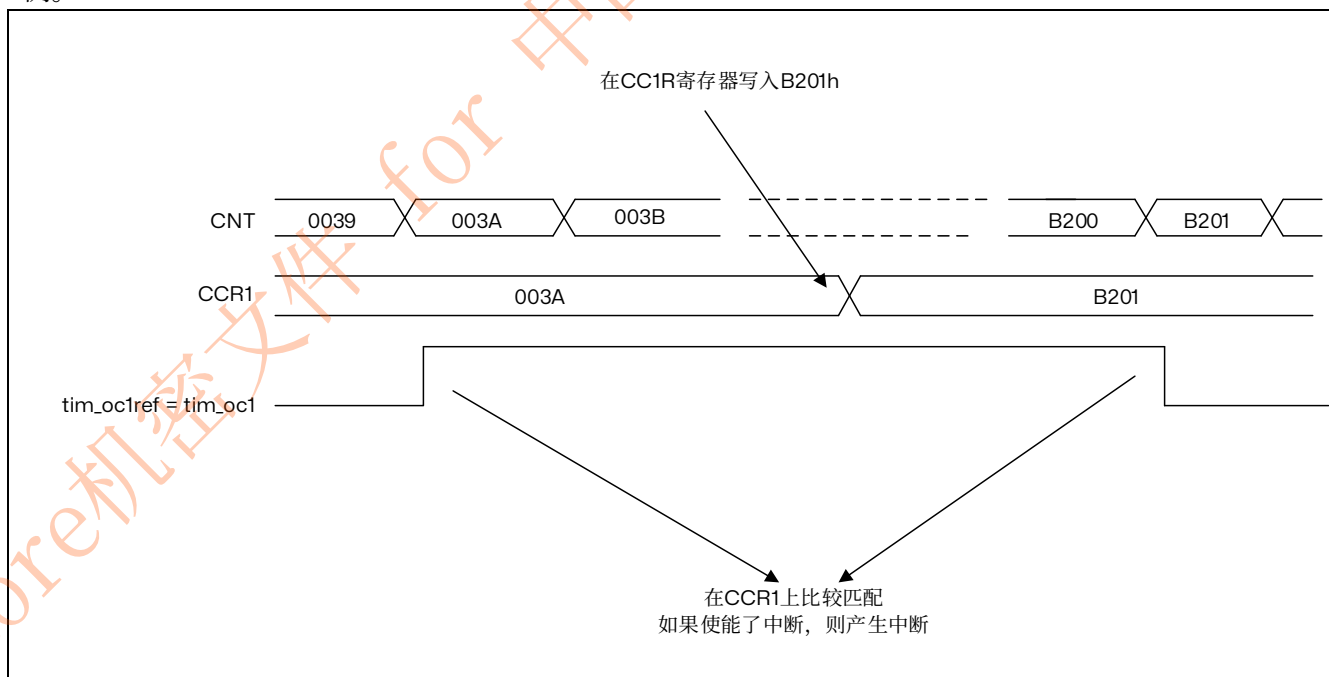


图 23.33 输出比较模式, 翻转 OC1

### 23.3.13 PWM 模式

脉冲宽度调制模式可以生成一个信号，该信号频率由 TIMx\_ARR 寄存器值决定，其占空比则由 TIMx\_CCRx 寄存器值决定。

通过向 TIMx\_CCMRx 寄存器中的 OCxM 位写入“0110”（PWM 模式 1）或“0111”（PWM 模式 2），可以独立选择各通道（每个 OCx 输出对应一个 PWM）的 PWM 模式。必须通过将 TIMx\_CCMRx 寄存器中的 OCxPE 位置 1 使能相应预装载寄存器，最后通过将 TIMx\_CR1 寄存器中的 ARPE 位置 1 使能自动重载预装载寄存器（在递增计数或中心对齐模式下）。

由于只有在发生更新事件时预装载寄存器才会传送到影子寄存器，因此启动计数器之前，必须通过将 TIMx\_EGR 寄存器中的 UG 位置 1 来初始化所有寄存器。

tim\_ocx 极性可使用 TIMx\_CCER 寄存器的 CCxP 位来编程。既可以设为高电平有效，也可以设为低电平有效。通过 CCxE、CCxNE、MOE、OSSI 和 OSSR 位（TIMx\_CCER 和 TIMx\_BDTR 寄存器）的组合使能 OCx 输出。有关详细信息，请参见 TIMx\_CCER 寄存器说明。

在 PWM 模式（1 或 2）下，TIMx\_CNT 总是与 TIMx\_CCRx 进行比较，以确定是  $TIMx\_CNT \leq TIMx\_CCRx$  还是  $TIMx\_CNT > TIMx\_CCRx$ （取决于计数器计数方向）。

根据 TIMx\_CR1 寄存器中的 CMS 位状态，定时器能够产生边沿对齐模式或中心对齐模式的 PWM 信号。

#### PWM 边沿对齐模式

- 递增计数配置

当 TIMx\_CR1 寄存器中的 DIR 位为低时执行递增计数。请参见递增计数模式一节。

以下以 PWM 模式 1 为例。只要  $TIMx\_CNT < TIMx\_CCRx$ ，PWM 参考信号 tim\_ocxref 便为高电平，否则为低电平。如果 TIMx\_CCRx 中的比较值大于自动重载值（TIMx\_ARR 中），则 tim\_ocxref 保持为“1”。如果比较值为 0，则 tim\_ocxref 保持为“0”。下图举例介绍边沿对齐模式的一些 PWM 波形（TIMx\_ARR=8）。

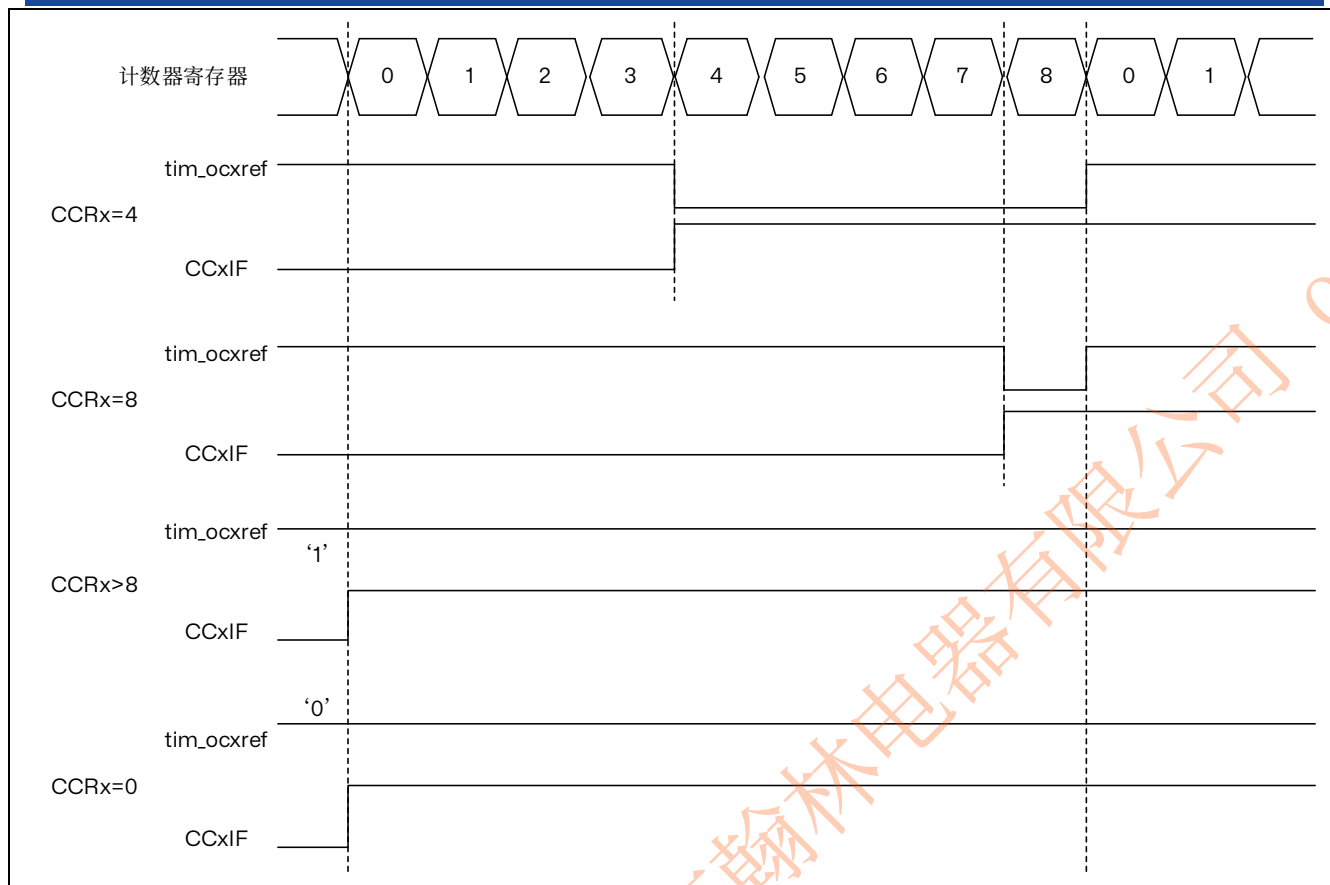


图 23.34 边沿对齐的 PWM 波形 (ARR=8)

### ● 递减计数配置

当 TIMx\_CR1 寄存器中的 DIR 位为高时执行递减计数。请参见递减计数模式一节。

在 PWM 模式 1 下，只要  $TIMx\_CNT > TIMx\_CCRx$ ，参考信号 tim\_ocxref 即为低电平，否则其为高电平。如果 TIMx\_CCRx 中的比较值大于 TIMx\_ARR 中的自动重载值，tim\_ocxref 保持为“1”。此模式下不可能产生 0% 的 PWM 波形。

### PWM 中央对齐模式

当 TIMx\_CR1 寄存器中的 CMS 位不为“00”（其余所有配置对 tim\_ocxref/tim\_ocx 信号具有相同的作用），中心对齐模式生效。根据 CMS 位的配置，可以在计数器递增计数、递减计数或同时递增和递减计数时将比较标志置 1。TIMx\_CR1 寄存器中的方向位(DIR)由硬件更新，不得通过软件更改。请参见中心对齐模式（递增/递减计数）。

下图显示了中心对齐模式的 PWM 波形，在此例中：

- TIMx\_ARR=8
- PWM 模式为 PWM 模式 1，
- 在根据 TIMx\_CR1 寄存器中 CMS=01 而选择的中心对齐模式 1 下，当计数器递减计数时，比较标志置 1。

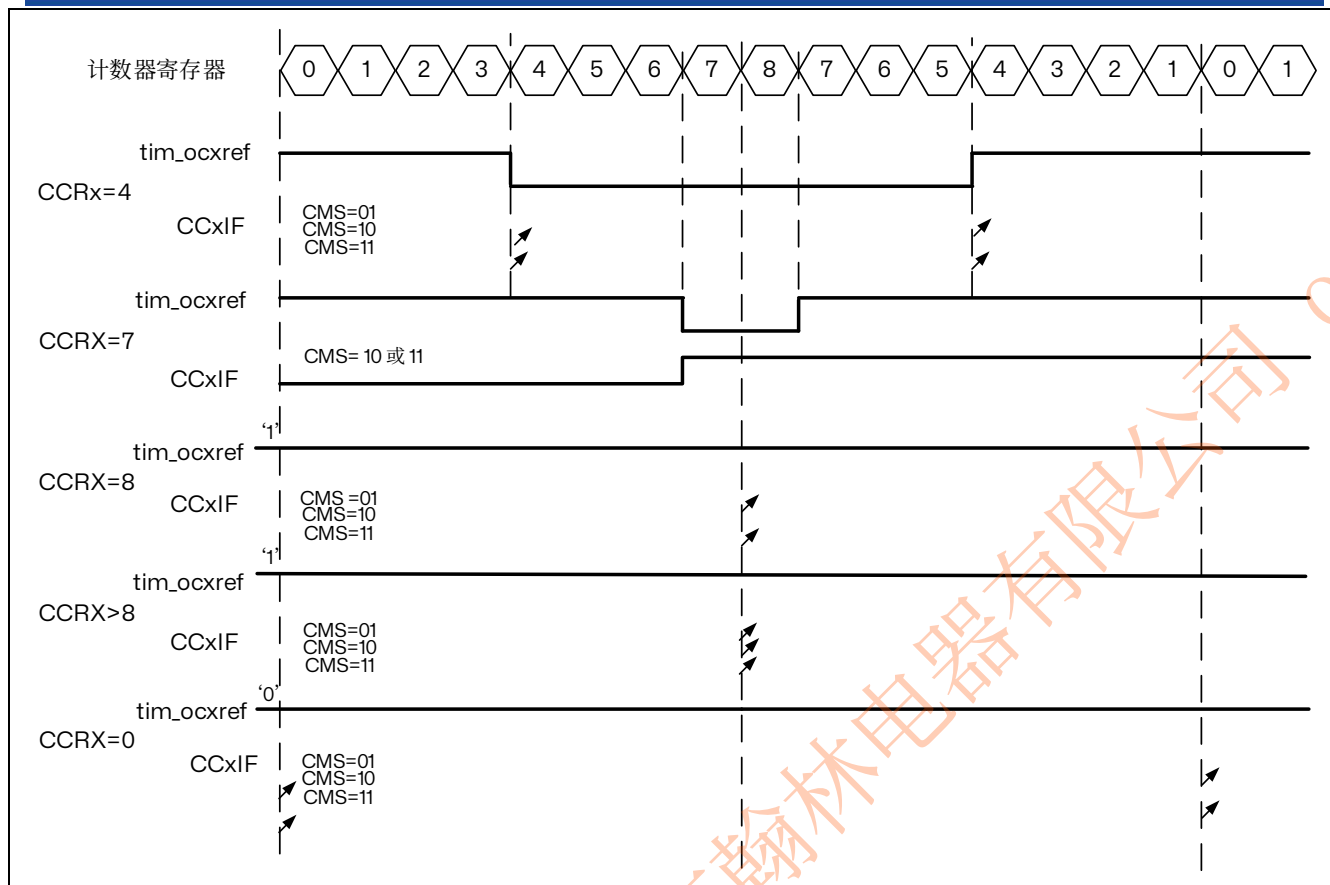


图 23.35 中央对齐的 PWM 波形 (ARR=8)

关于使用中央对齐模式的提示：

- 启动中心对齐模式时将使用当前的递增/递减计数配置。这意味着计数器将根据写入 TIMx\_CR1 寄存器中 DIR 位的值进行递增或递减计数。此外，不得同时通过软件修改 DIR 和 CMS 位。
- 不建议在运行中心对齐模式时对计数器执行写操作，否则将发生意想不到的结果。尤其是：
  - 如果写入计数器中的值大于自动重载值(TIMx\_CNT>TIMx\_ARR)，计数方向不会更新。例如，如果计数器之前递增计数，则继续递增计数。
  - 如果向计数器写入 0 或 TIMx\_ARR 的值，计数方向会更新，但不生成更新事件 UEV。
- 使用中心对齐模式最为保险的方法是：在启动计数器前通过软件生成更新（将 TIMx\_EGR 寄存器中的 UG 位置 1），并且不要在计数器运行过程中对其执行写操作。

### 抖动模式

通过使能抖动模式，使用 TIMx\_CR1 寄存器中的 DITHEN 位，可以提高 PWM 模式的有效分辨率。这既适用于 CCR（用于增加占空比分辨率），也适用于 ARR（用于增加 PWM 频率分辨率）。

工作原理是在 16 个连续的 PWM 周期中，以预定义的模式使实际的 CCR（或 ARR）值略有变化（增加或减少一个定时器时钟周期）。这使得占空比或 PWM 期间的平均值提高了 16 倍分辨率。下图展示了应用于 4 个连续 PWM 周期的抖动原理。



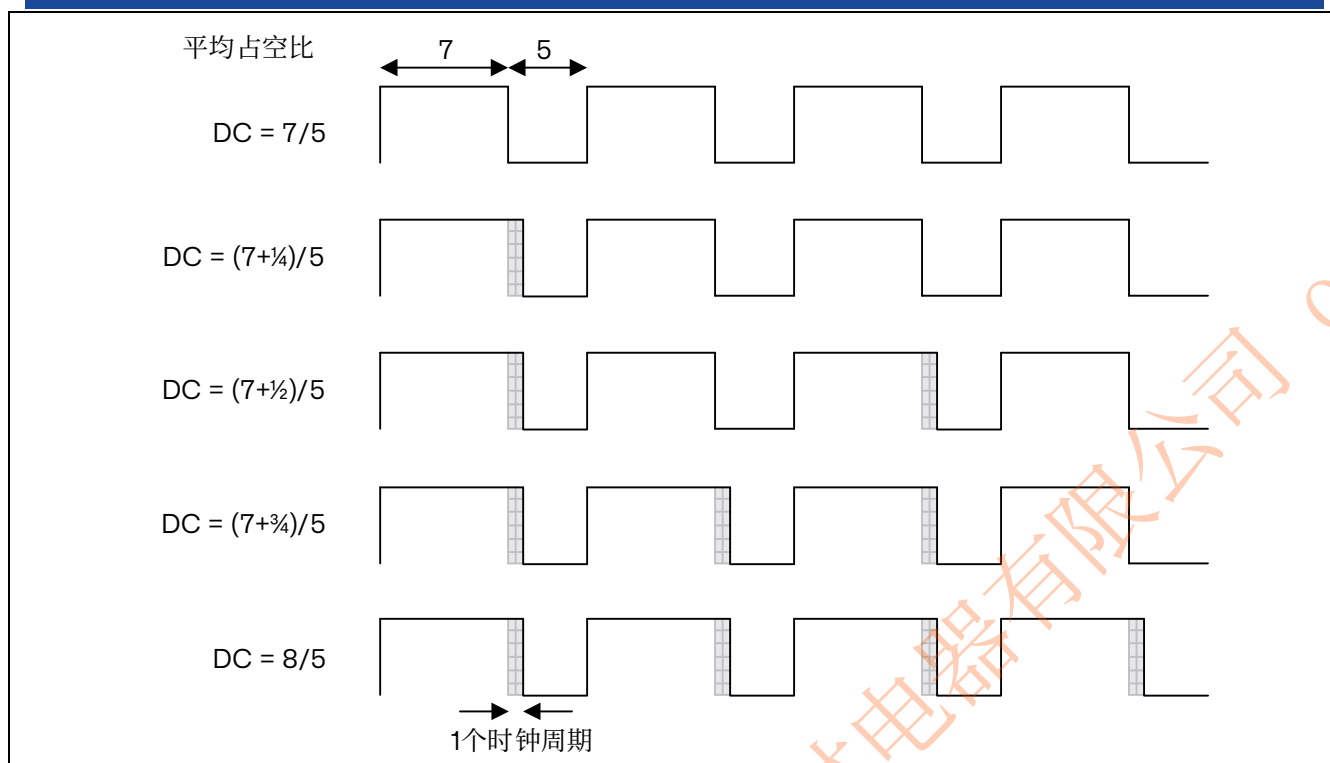


图 23.36 抖动原理

当抖动模式被使能时，寄存器编码将如下所示进行变化（参见下图示例）

- 4 个最低有效位（LSB）用于编码增强的分辨率部分（小数部分）
- 最高有效位（MSB）左移至 19:4 位，用于编码基本值

注：如果设置清除 DITHEN 位，ARR 和 CCR 值将自动更新。（例如，如果 ARR=0x05 且 DITHEN=0，它将更新为 ARR=0x50 且 DITHEN=1）。

1. 必须重置 CEN 和 ARPE 位
2. ARR[3:0] 位必须重置
3. DITHEN 位必须重置
4. CCIF 标志必须清除
5. 可以设置 CEN 位（最终设置 ARPE=1）。



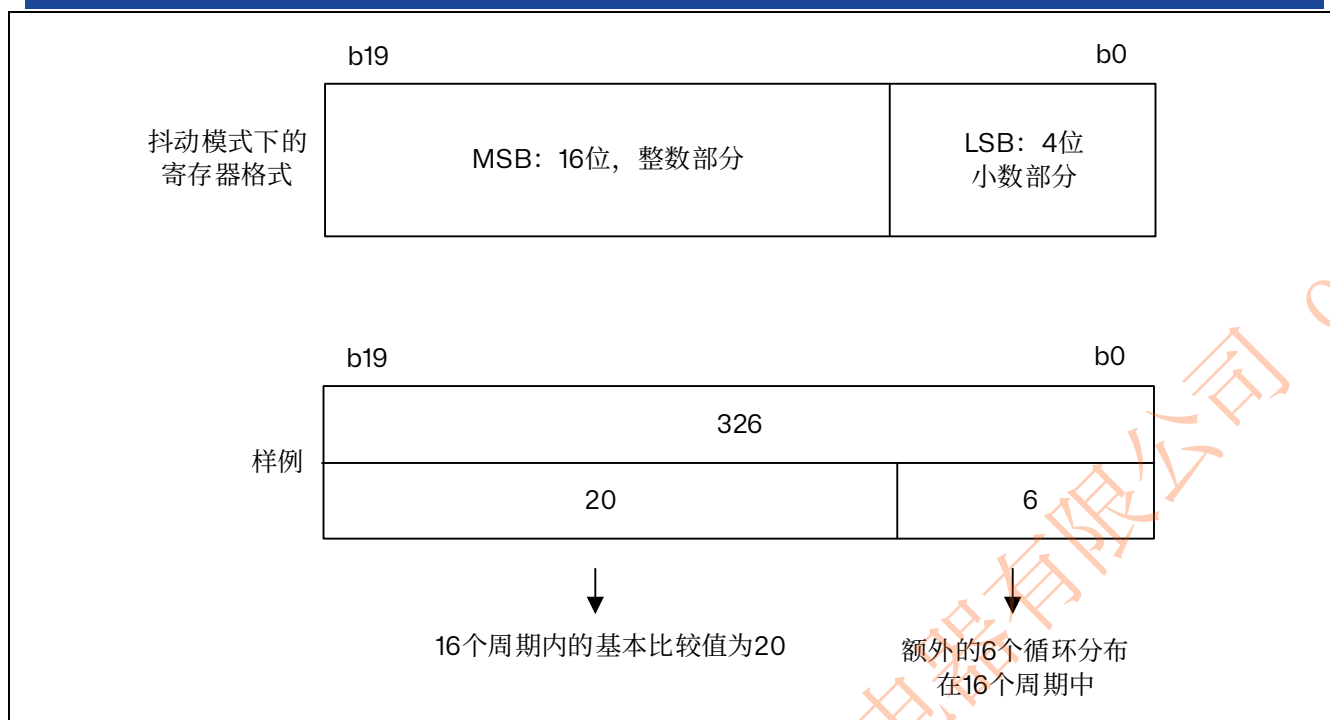


图 23.37 抖动模式下的数据格式和寄存器编码

最小频率由以下公式给出:

$$\text{分辨率} = \frac{F_{Tim}}{F_{pwm}} \Rightarrow F_{pwmMin} = \frac{F_{Tim}}{\text{最大分辨率}}$$

$$\text{禁用抖动模式: } F_{pwmMin} = \frac{F_{Tim}}{65536}$$

$$\text{使能抖动模式: } F_{pwmMin} = \frac{F_{Tim}}{65535 + \frac{15}{16}}$$

注意: 在抖动模式下, TIMx\_ARR 和 TIMx\_CCRy 的最大值限制为 0xFFFFEF (对应于整数部分的 65534 和抖动部分的 15)。

如下图所示, 无论 PWM 频率如何, 抖动模式都可以增加 PWM 分辨率。

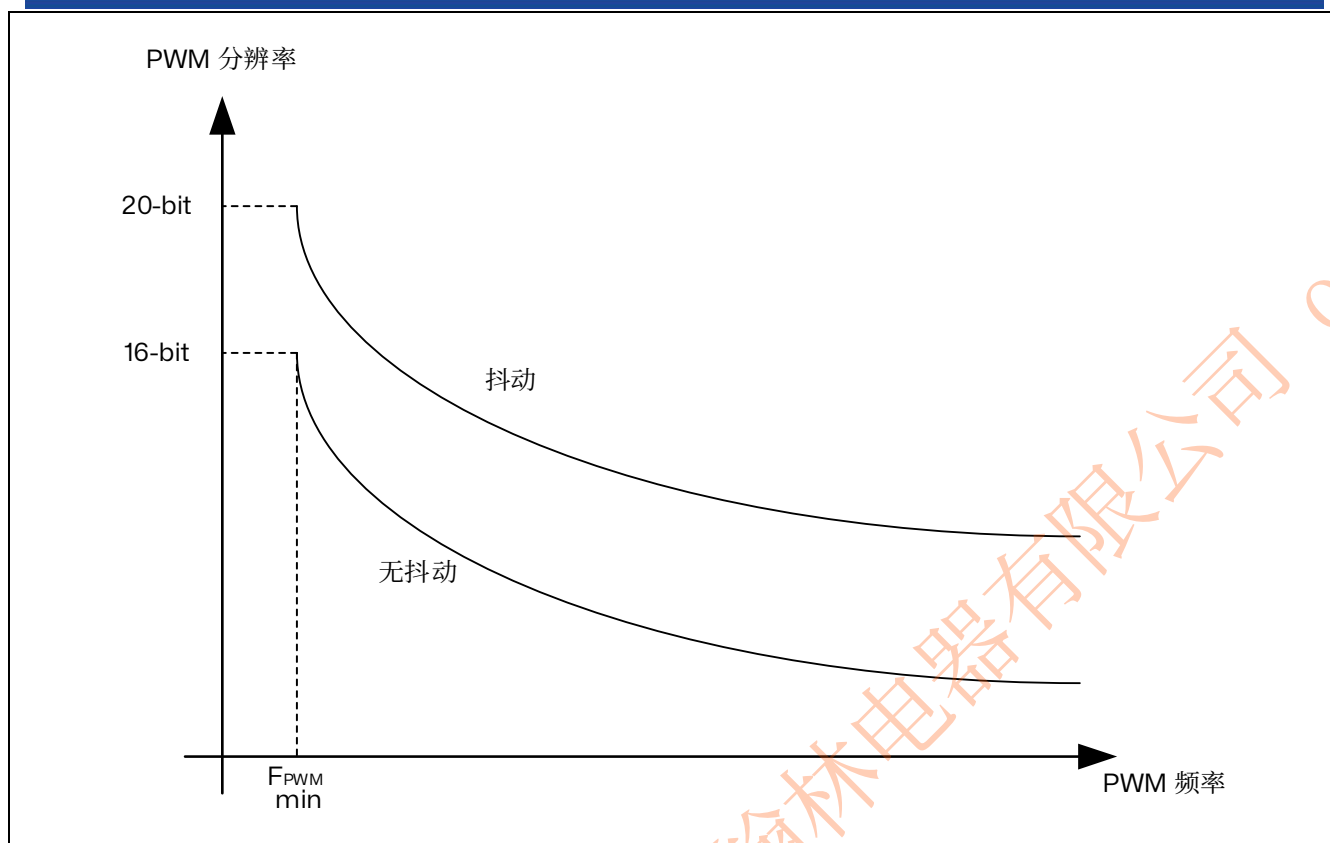


图 23.38 PWM 分辨率 vs 频率

如下图所示，占空比和/或周期的变化分散在 16 个连续周期。

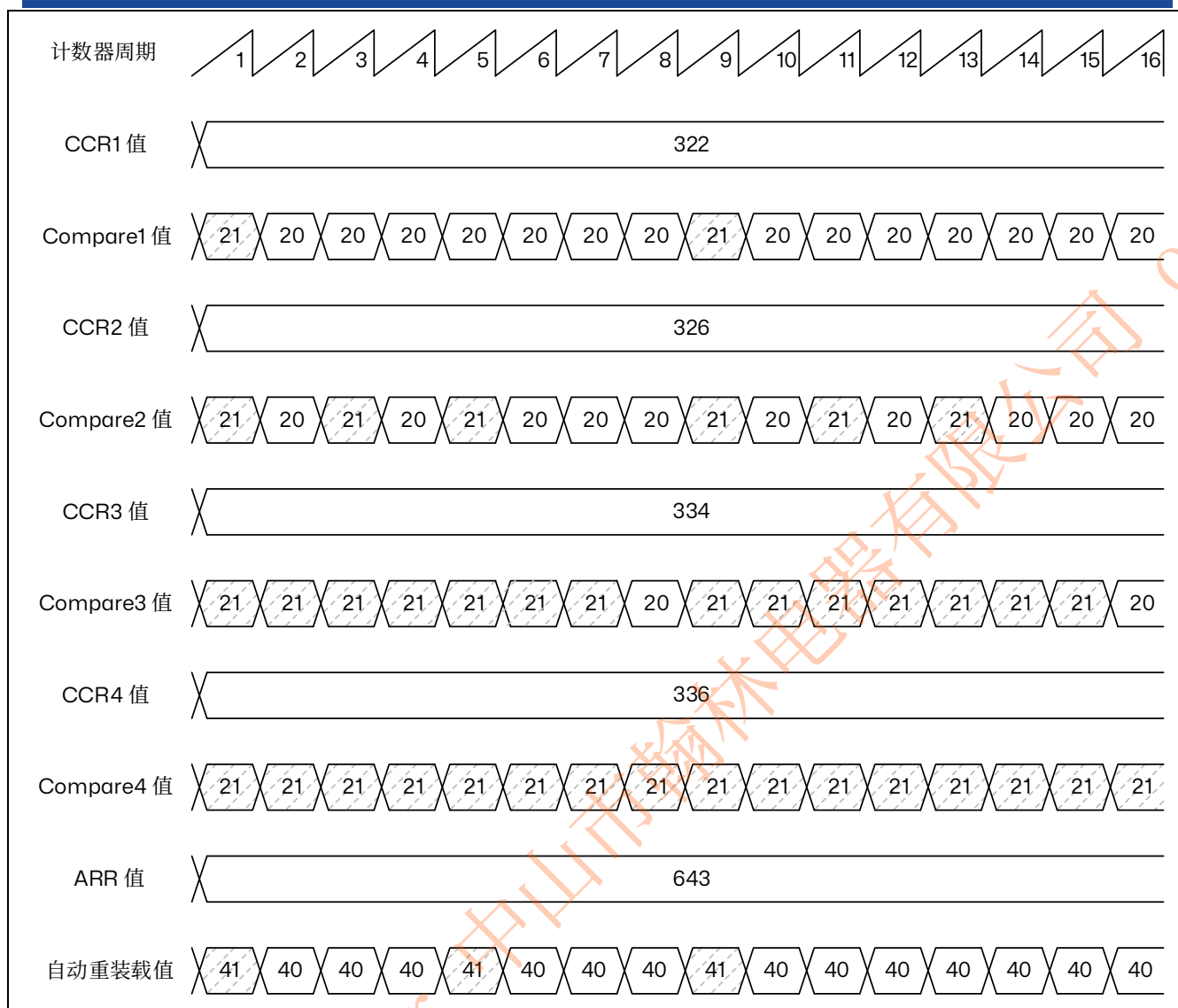


图 23.39 PWM 抖动模式

自动重新加载和比较值的增量按照下表中描述的特定模式分布。抖动序列是为了使增量分布尽可能均匀并最大限度地减少整体波动。

表 23.13 CCR 和 ARR 寄存器的变化抖动模式

| LSB 值 | PWM 周期 |   |    |   |    |   |    |   |    |    |    |    |    |    |    |    |
|-------|--------|---|----|---|----|---|----|---|----|----|----|----|----|----|----|----|
|       | 1      | 2 | 3  | 4 | 5  | 6 | 7  | 8 | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| 0000  | -      | - | -  | - | -  | - | -  | - | -  | -  | -  | -  | -  | -  | -  | -  |
| 0001  | +1     | - | -  | - | -  | - | -  | - | -  | -  | -  | -  | -  | -  | -  | -  |
| 0010  | +1     | - | -  | - | -  | - | -  | - | +1 | -  | -  | -  | -  | -  | -  | -  |
| 0011  | +1     | - | -  | - | +1 | - | -  | - | +1 | -  | -  | -  | -  | -  | -  | -  |
| 0100  | +1     | - | -  | - | +1 | - | -  | - | +1 | -  | -  | -  | +1 | -  | -  | -  |
| 0101  | +1     | - | +1 | - | +1 | - | -  | - | +1 | -  | -  | -  | +1 | -  | -  | -  |
| 0110  | +1     | - | +1 | - | +1 | - | -  | - | +1 | -  | +1 | -  | +1 | -  | -  | -  |
| 0111  | +1     | - | +1 | - | +1 | - | +1 | - | +1 | -  | +1 | -  | +1 | -  | -  | -  |

|      |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |
|------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|
| 1000 | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | - |
| 1001 | +1 | +1 | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | - |
| 1010 | +1 | +1 | +1 | -  | +1 | -  | +1 | -  | +1 | +1 | +1 | -  | +1 | -  | +1 | - |
| 1011 | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | -  | +1 | - |
| 1100 | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | - |
| 1101 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | - |
| 1110 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | -  | +1 | +1 | +1 | +1 | +1 | +1 | +1 | - |
| 1111 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | - |

在中心对齐 PWM 模式下（TIMx\_CR1 寄存器中的 CMS 位不为“00”），也提供了抖动模式。在这种模式下，抖动模式考虑到递增递减计数阶段，应用于 8 个连续的 PWM 周期，如下图所示。

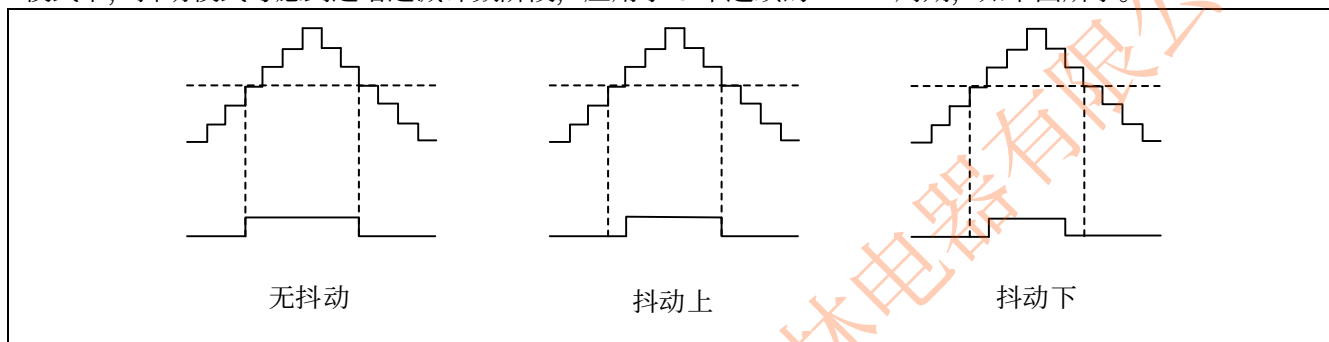


图 23.40 中心对齐 PWM 模式下抖动对占空比的影响

下表显示了如何在中心对齐 PWM 模式下添加抖动模式。

表 23.14 CCR 寄存器在中心对齐 PWM 模式下的变化抖动模式

| LSB 值 | PWM 周期 |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |
|-------|--------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|
|       | 1      |    | 2  |    | 3  |    | 4  |    | 5  |    | 6  |    | 7  |    | 8  |   |
|       | 上      | 下  | 上  | 下  | 上  | 下  | 上  | 下  | 上  | 下  | 上  | 下  | 上  | 下  | 上  | 下 |
| 0000  | -      | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | - |
| 0001  | +1     | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | - |
| 0010  | +1     | -  | -  | -  | -  | -  | -  | -  | +1 | -  | -  | -  | -  | -  | -  | - |
| 0011  | +1     | -  | -  | -  | +1 | -  | -  | -  | +1 | -  | -  | -  | -  | -  | -  | - |
| 0100  | +1     | -  | -  | -  | +1 | -  | -  | -  | +1 | -  | -  | -  | +1 | -  | -  | - |
| 0101  | +1     | -  | +1 | -  | +1 | -  | -  | -  | +1 | -  | -  | -  | +1 | -  | -  | - |
| 0110  | +1     | -  | +1 | -  | +1 | -  | -  | -  | +1 | -  | +1 | -  | +1 | -  | -  | - |
| 0111  | +1     | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | -  | - |
| 1000  | +1     | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | - |
| 1001  | +1     | +1 | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | - |
| 1010  | +1     | +1 | +1 | -  | +1 | -  | +1 | -  | +1 | +1 | +1 | -  | +1 | -  | +1 | - |
| 1011  | +1     | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | -  | +1 | - |
| 1100  | +1     | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | - |
| 1101  | +1     | +1 | +1 | +1 | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | - |
| 1110  | +1     | +1 | +1 | +1 | +1 | +1 | +1 | -  | +1 | +1 | +1 | +1 | +1 | +1 | +1 | - |
| 1111  | +1     | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | - |

### 23.3.14 非对称 PWM 模式

非对称模式允许生成两个中心对齐的 PWM 信号，并具有可编程的相位偏移。虽然频率由 TIMx\_ARR 寄存器的值决定，但占空比和相位偏移由一对 TIMx\_CCRx 寄存器确定。一个寄存器控制 PWM 在计数上升期间，第二个寄存器在计数下降期间，这样 PWM 每半个 PWM 周期调整一次：

- tim\_oc1refc (或 tim\_oc2refc) 由 TIMx\_CCR1 和 TIMx\_CCR2 控制
- tim\_oc3refc (或 tim\_oc4refc) 由 TIMx\_CCR3 和 TIMx\_CCR4 控制

通过将“1110”（非对称 PWM 模式 1）或“1111”（非对称 PWM 模式 2）写入 TIMx\_CCMRx 寄存器的 OCxM 位，可以在两个通道上独立选择非对称 PWM 模式（每个 CCR 寄存器对一个 tim\_ocx 输出）。

*注意：为了兼容性原因，OCxM[3:0] 位域被分为两个部分，最高有效位与最低 3 位不连续。*

当某个通道用作非对称 PWM 通道时，其互补通道也可以使用。例如，如果 tim\_oc1refc 信号在通道 1 上生产（非对称 PWM 模式 1），可以在通道 2 上输出 tim\_oc1refc 信号，或者产生非对称 PWM 模式 1 的结果 tim\_oc2refc 信号。

下图标识使用非对称 PWM 模式生成信号的示例（通道 1 到 4 配置为非对称 PWM 模式 2）。与死区时间生成器一起使用，可以控制全桥移相 DC 到 DC 转换器。

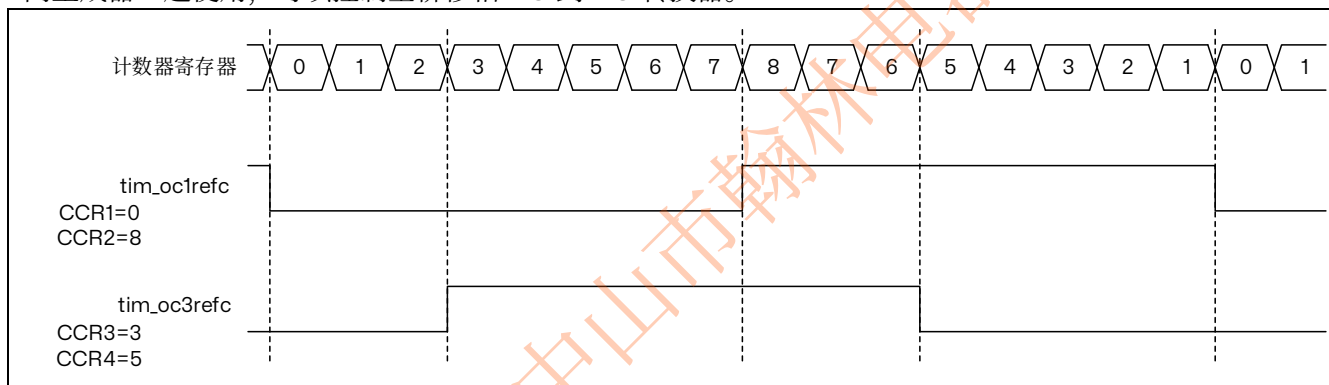


图 23.41 生成两个具有 50%占空比的相位偏移 PWM 信号

### 23.3.15 组合 PWM 模式

组合 PWM 模式允许生成两个边缘对齐或中心对齐的 PWM 信号，各自脉冲之间的可编程延迟和相位偏移。虽然频率由 TIMx\_ARR 寄存器的值决定，但占空比和延迟由两个 TIMx\_CCRx 寄存器确定。结果信号 tim\_ocxrefc 由两个参考 PWM 的 OR 或 AND 逻辑组合而成：

- tim\_oc1refc (或 tim\_oc2refc) 由 TIMx\_CCR1 和 TIMx\_CCR2 控制
- tim\_oc3refc (或 tim\_oc4refc) 由 TIMx\_CCR3 和 TIMx\_CCR4 控制

通过在 TIMx\_CCMRx 寄存器的 OCxM 位中写入“1100”（组合 PWM 模式 1）或“1101”（组合 PWM 模式 2），可以在两个通道上独立选组合 PWM 模式（每个 CCR 寄存器对一个 tim\_ocx 输出）。

当某个通道用作组合 PWM 通道时，其互补通道必须配置为相反的 PWM 模式（例如，一个通道为组合 PWM 模式 1，另一个通道为组合 PWM 模式 2）。

*注意：为了兼容性原因，OCxM[3:0] 位域被分为两个部分，最高有效位与最低 3 位不连续。*

下图表示使用组合 PWM 模式生成信号的示例，该示例的配置如下：

- 通道 1 配置为组合 PWM 模式 2。
- 通道 2 配置为 PWM 模式 1。
- 通道 3 配置为组合 PWM 模式 2。

— 通道 4 配置为 PWM 模式 1。

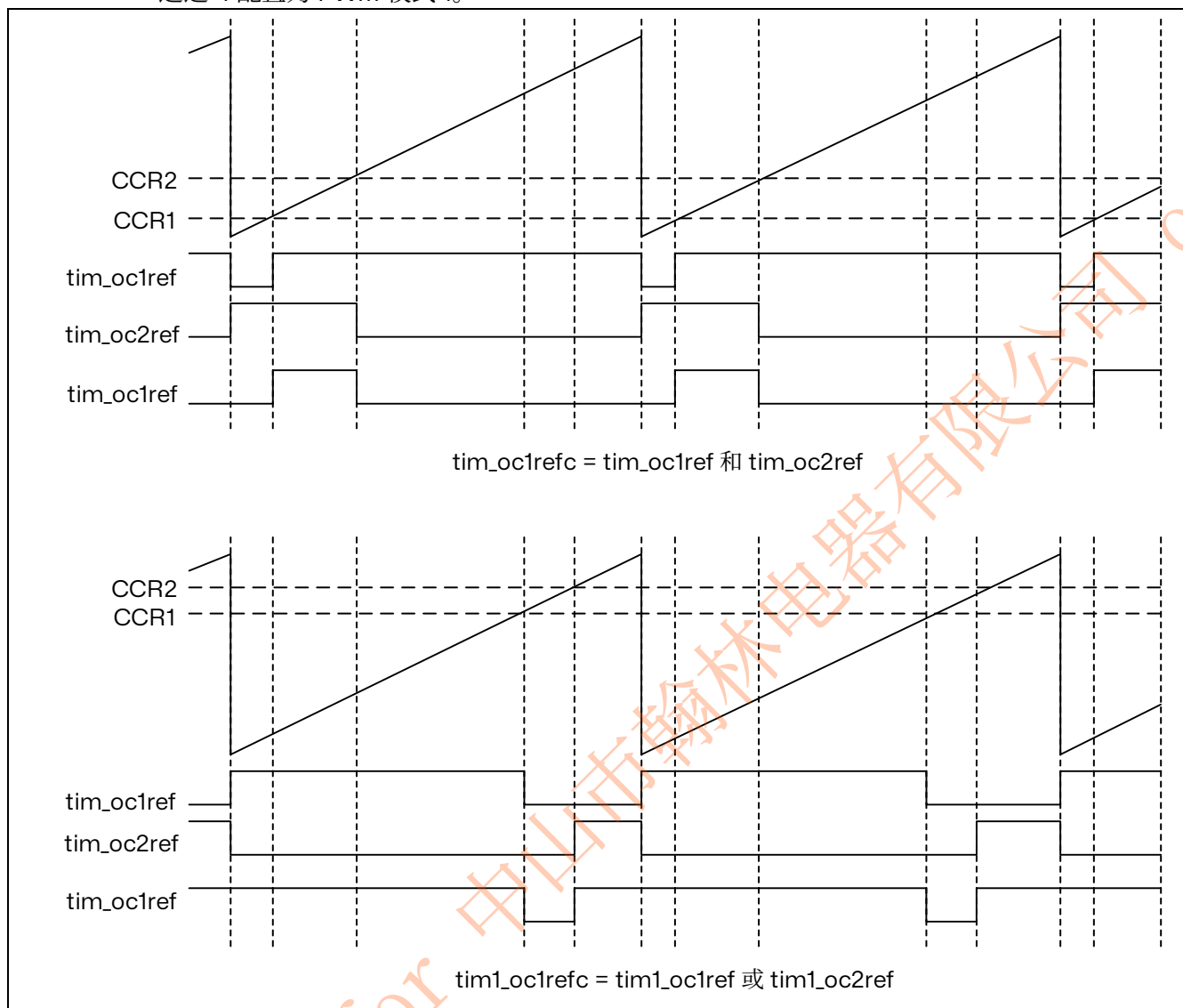


图 23.42 通道 1 和通道 3 的组合 PWM 模式

### 23.3.16 三相组合 PWM 模式

三相组合 PWM 模式允许生成一个到三个以脉冲中间的单个可编程信号 AND 为中心对齐的 PWM 信号。使用 tim\_oc5ref 信号定义结果组合信号。TIMx\_CCR5 中的 3 位 GC5C[3:1]允许选择 tim\_oc5ref 与哪个参考信号组合。结果信号 tim\_ocxrefc 由两个参考 PWM 的 AND 逻辑组合而成。

- 如果设置了 GC5C1, 则 tim\_oc1refc 由 TIMx\_CCR1 和 TIMx\_CCR5 控制
- 如果设置了 GC5C2, 则 tim\_oc2refc 由 TIMx\_CCR2 和 TIMx\_CCR5 控制
- 如果设置了 GC5C3, 则 tim\_oc3refc 由 TIMx\_CCR3 和 TIMx\_CCR5 控制

通过设置 3 位 GC5C[3:1]中的至少一位, 可以在通道 1 到 3 上独立选择三相组合 PWM 模式。

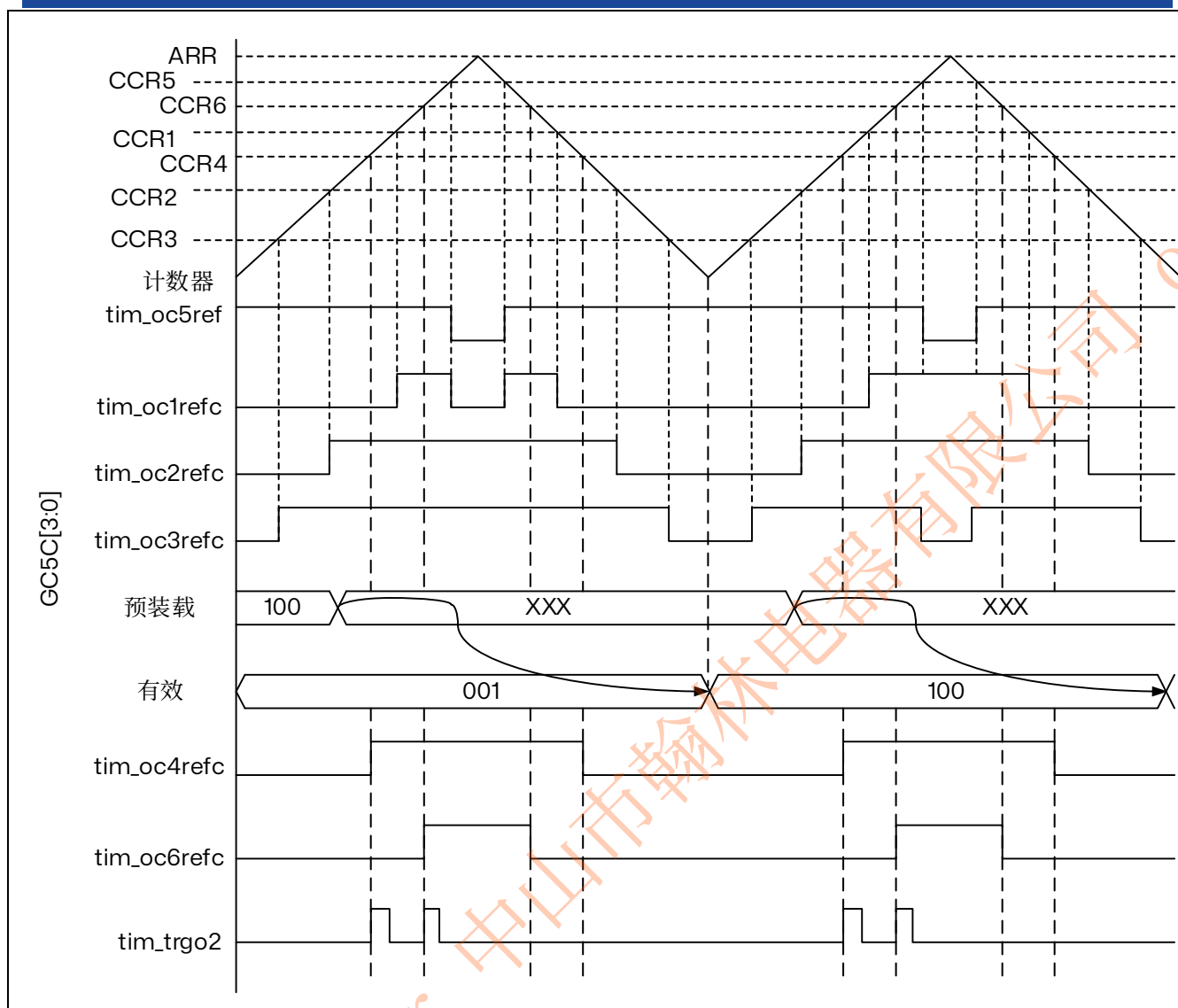


图 23.43 每周期具有多个触发脉冲的三相组合 PWM 信号

tim\_trgo2 波形显示了 ADC 如何对给定的三相 PWM 信号进行同步。更多详细信息请参考本章后续小节：ADC 触发器。

### 23.3.17 互补输出和死区插入

高级控制定时器（TIM1/TIM8）可以输出两路互补信号，并管理输出的关断与接通瞬间。

这段时间通常称为死区，用户必须根据与输出相连接的器件及其特性（电平转换器的固有延迟、开关器件产生的延迟...）来调整死区时间。

每路输出可以独立选择输出极性（主输出 tim\_ocx 或互补输出 tim\_ocxn）。可通过对 TIMx\_CCER 寄存器中的 CCxP 和 CCxNP 位执行写操作来完成极性选择。

互补信号 tim\_ocx 和 tim\_ocxn 通过以下多个控制位的组合进行激活：TIMx\_CCER 寄存器中的 CCxE 和 CCxNE 位以及 TIMx\_BDTR 和 TIMx\_CR2 寄存器中的 MOE、OISx、OISxN、OSSI 和 OSSR 位。更多详细信息，请参考表格：带刹车功能的互补输出通道 tim\_ocx 和 tim\_ocxn 的控制位。应当注意，切换至 idle（MOE 下降到 0）的时刻，死区仍然有效。

死区时间插入功能是同时设置 CCxE 和 CCxNE 位以及（如果存在刹车电路）MOE 位来使能的。每



个通道都有一个 10 位的死区时间生成器。将基于参考波形 `tim_ocxref` 生成 2 个输出 `tim_ocx` 和 `tim_ocxn`。如果 `tim_ocx` 和 `tim_ocxn` 为高电平有效：

- 输出信号 `tim_ocx` 与参考信号相同，只是其上升沿相对参考上升沿存在延迟。
- 输出信号 `tim_ocxn` 与参考信号相反，并且其上升沿相对参考下降沿存在延迟。

如果延迟时间大于有效输出（`tim_ocx` 或 `tim_ocxn`）的宽度，则不会产生相应的脉冲。

下图所示为死区发生器的输出信号与参考信号 `tim_ocxref` 之间的关系。（在这些示例中，假定 `CCxP=0`、`CCxNP=0`、`MOE=1`、`CCxE=1` 并且 `CCxNE=1`）

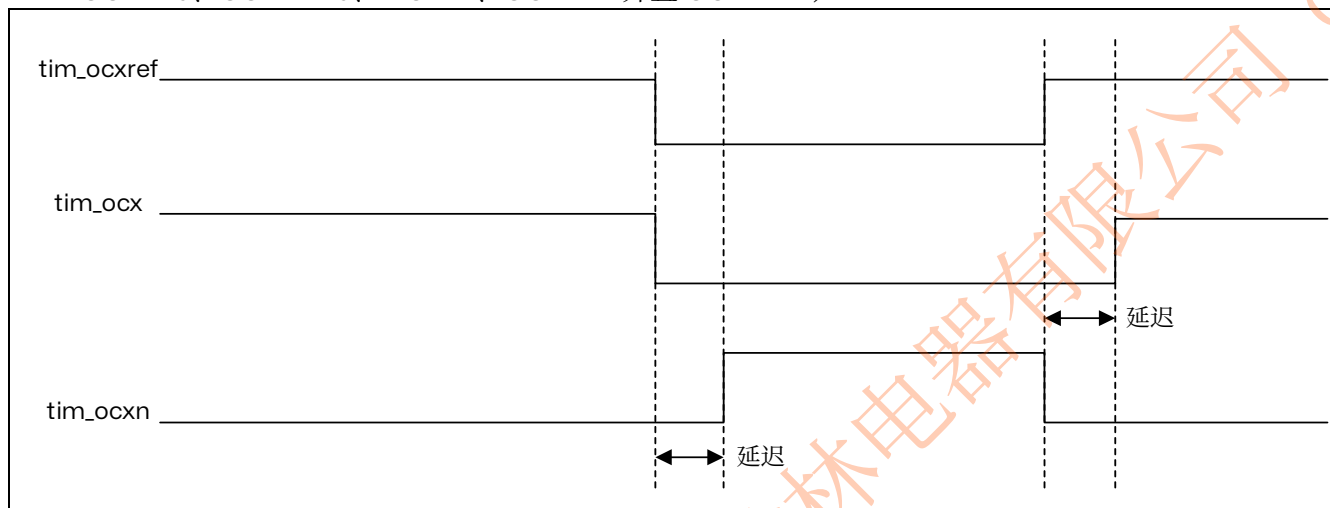


图 23.44 带死区插入的互补输出

`TIMx_DTR2` 中的 `DTAE` 位允许对参考信号上升沿和下降沿的死区值进行区分，如下图所示。

在非对称模式（`DTAE=1`）下，上升沿参考死区由 `TIMx_BDTR` 寄存器中的 `DTG[7:0]` 位域定义，而下降沿参考死区由 `TIMx_DTR2` 寄存器中的 `DTGF[7:0]` 位域定义。在使能计数器之前，必须写入 `DTAE` 位，并且在 `CEN=1` 时不得修改。

使用预加载机制可以在 PWM 操作期间动态更新死区值。当 `TIMx_DTR2` 寄存器中的 `DTPE` 位被设置时，死区位域 `DTG[7:0]` 和 `DTGF[7:0]` 将被预加载。预加载值将在下一个更新事件上加载到活动寄存器中。

注意：如果 `DTPE` 位在计数器使能时被使能。自上次更新以来写入的新值将被丢弃，并使用先前的值。

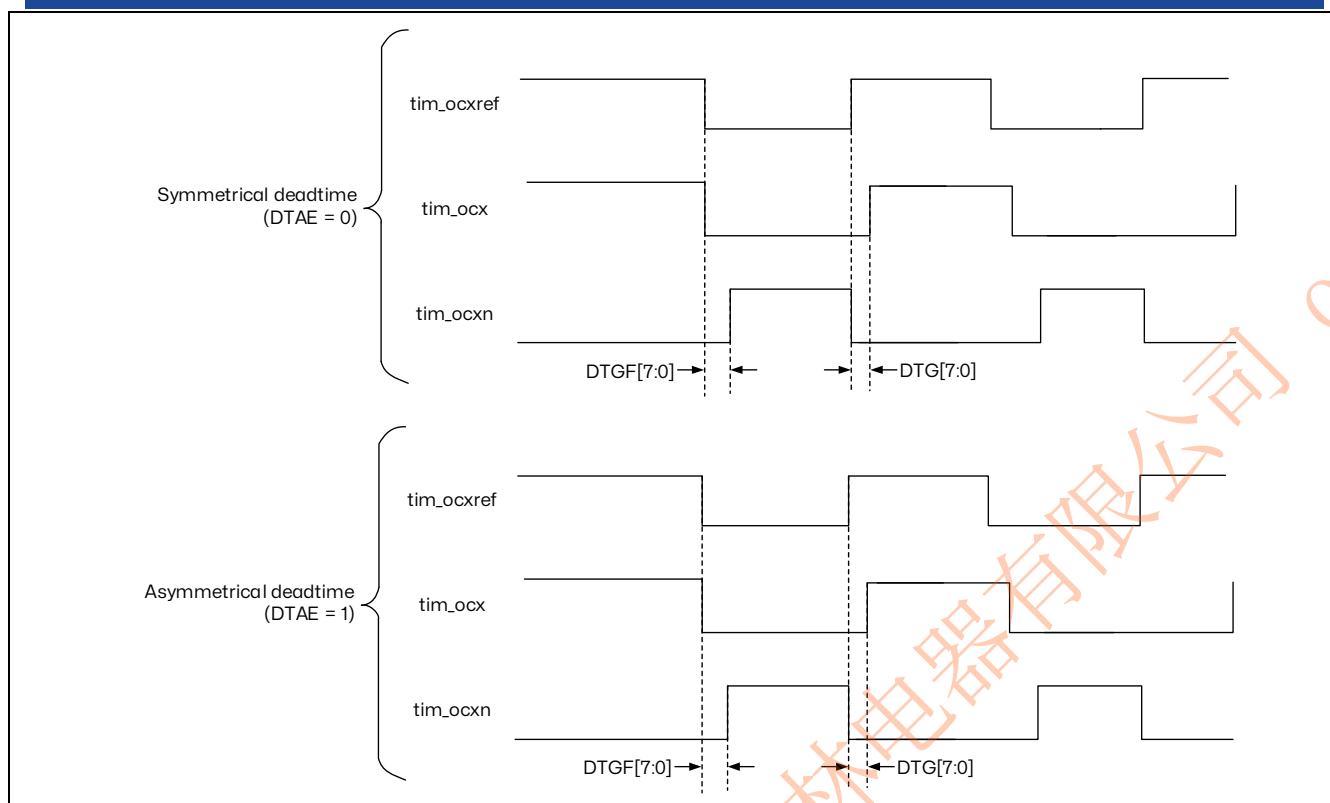


图 23.45 非对称死区时间

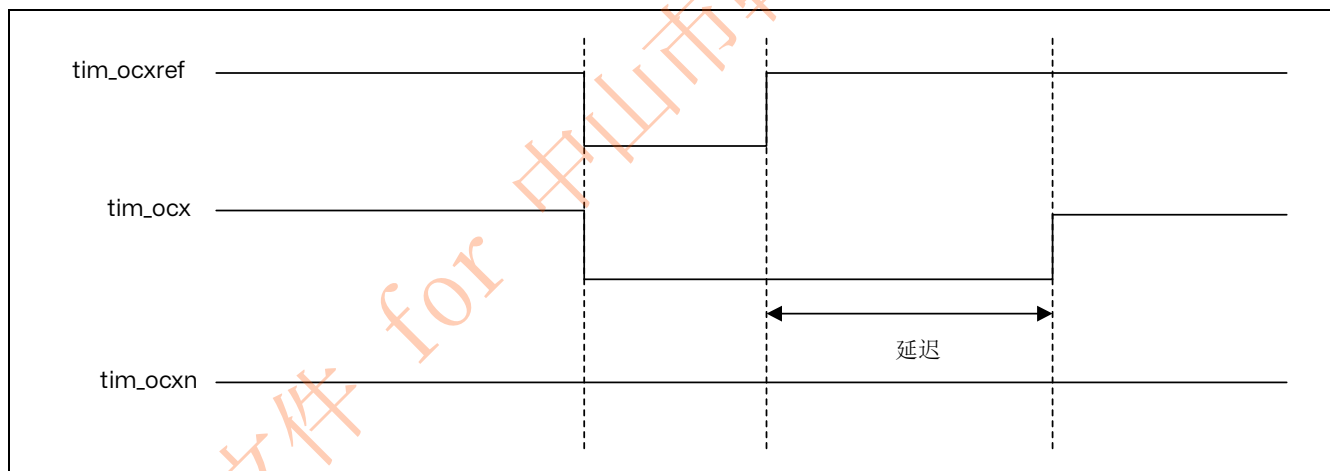


图 23.46 死区波形延迟大于负脉冲

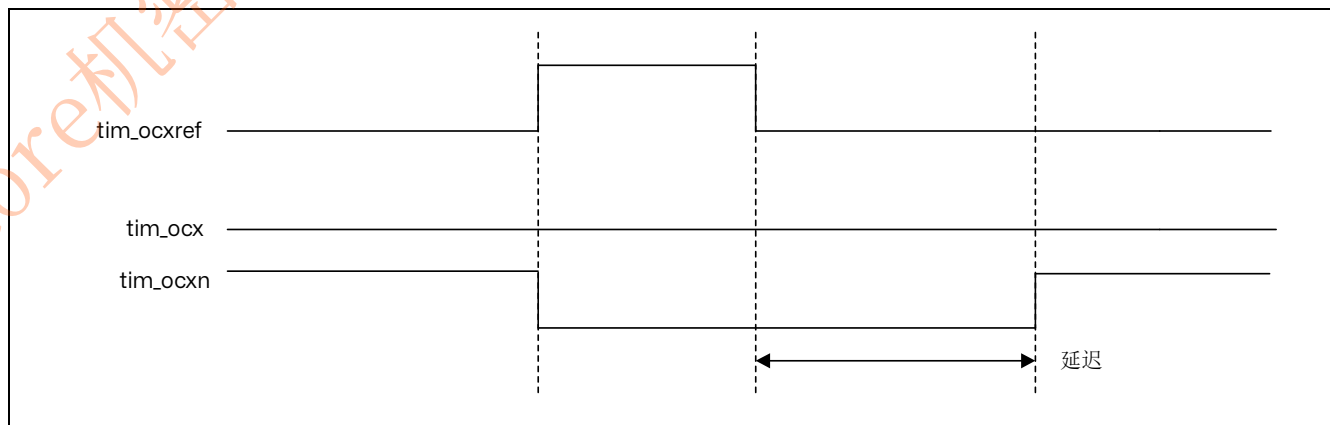


图 23.47 死区波形延迟大于正脉冲

死区延迟对于所有通道均相同，可通过 TIMx\_BDTR 寄存器中的 DTG 位进行编程。有关延迟时间计算的信息，请参见后续小节：TIMx 刹车和死区寄存器 (TIMx\_BDTR) (x=1, 8)。

### 将 OCxREF 重定向到 OCx 或 OCxN

在输出模式（强制输出模式、输出比较模式或 PWM 模式）下，通过配置 TIMx\_CCER 寄存器中的 CCxE 和 CCxNE 位，可将 tim\_ocxref 重定向到 tim\_ocx 输出或 tim\_ocxn 输出。

通过此功能，可以在一个输出上发送特定波形（如 PWM 或静态有效电平），而同时使互补输出保持其无效电平。或者，使两个输出同时保持无效电平，或者两个输出同时处于有效电平，两者互补并且带死区。

注：如果仅使能 tim\_ocxn (CCxE=0, CCxNE=1)，两者不互补，一旦 tim\_ocxref 为高电平，tim\_ocxn 即变为有效。例如，如果 CCxNP=0，则 tim\_ocxn = tim\_ocxref。另一方面，如果同时使能 tim\_ocx 和 tim\_ocxref (CCxE=CCxNE=1)，tim\_ocx 在 tim\_ocxref 为高电平时变为有效，而 tim\_ocxn 则与之互补，在 tim\_ocxref 为低电平时变为有效。

### 23.3.18 使用刹车功能

刹车功能的目的是保护由定时器产生的 PWM 信号驱动的功率开关。两个刹车输入通常连接到功率级和三相逆变器的故障输出。当激活时，刹车电路会关闭 PWM 输出并强制它们进入预定义的安全状态。还可以选择许多内部 MCU 事件来触发输出关闭。

刹车具有两个通道。一个刹车通道可以收集系统级故障（时钟故障、ECC/奇偶校验错误，.....）和应用故障（来自输入引脚和内置比较器），并可以在死区时间后将输出强制到一个预定义的水平（无论是有效还是无效）。一个刹车 2 通道只包括应用故障，并能够将输出强制到非活动状态。

刹车期间输出使能信号和输出电平取决于几个控制位：

- TIMx\_BDTR 寄存器中的 MOE 位允许通过软件使能/禁用输出，并在发生刹车或刹车 2 事件时重置。
- TIMx\_BDTR 寄存器中的 OSS1 位定义了定时器是否在非活动状态下控制输出或将其控制权释放给 GPIO 控制器（通常是为了使其处于高阻模式）。
- TIMx\_CR2 寄存器中的 OISx 和 OISxN 位设置输出关闭电平，可以是有效电平或无效电平。无论 OISx 和 OISxN 的值如何，tim\_ocx 和 tim\_ocxn 输出不能同时在给定时间内设置为有效电平。更多详细信息请参考后续表格：具有刹车功能的互补 tim\_ocx 和 tim\_ocxn 通道的输出控制位。

当退出重置状态时，刹车电路被禁用，并且 MOE 位为低电平。通过设置 TIMx\_BDTR 寄存器中的 BKE 和 BK2E 位，可以使能刹车功能。通过配置相同的寄存器中的 BKP 和 BK2P 位，可以选择刹车输入极性。BKEx 和 BKPx 可以同时修改。在写入 BKEx 和 BKPx 位后，需要等待 1 个 APB 时钟周期才能使写入生效。因此，在写入操作后，需要等待 1 个 APB 时钟周期才能正确地读取该位。

由于 MOE 下降沿可能是异步的，因此在实际信号（作用于输出）和同步控制位（在 TIMx\_BDTR 寄存器中访问）之间插入了一个重新同步电路。这导致异步信号和同步信号之间存在一些延迟。特别是当 MOE 被设置为 1 而之前它是低电平时，在正确读取之前必须插入一个延迟（伪指令）。这是因为写入作用于异步信号，而读取反映的是同步信号。

刹车 (tim\_brk) 通道的源为：

- 连接到 TIMx\_BKIN 引脚的外部源（根据 GPIO 备用功能选择寄存器中的选择而定），具有极性选择和可选的数字滤波

● 内部源:

- 来自 tim\_brk\_cmpx 输入 (请参考章节: TIM1/TIM8 引脚和内部信号)
- 来自 tim\_sys\_brk 输入的系统中断请求 (请参考章节: TIM1/TIM8 引脚和内部信号)

刹车 2 (tim\_brk2) 的源为:

- 连接到 TIMx\_BKIN2 引脚的外部源 (根据 GPIO 备用功能选择寄存器中的选择而定), 具有极性选择和可选的数字滤波。
- 来自 tim\_brk2\_cmpx 输入的内部源 (请参考章节: TIM1/TIM8 引脚和内部信号)

也可以通过软件使用 TIMx\_EGR 寄存器中的 BG 和 B2G 位生成刹车事件。

所有源在进入定时器的 tim\_brk 或 tim\_brk2 输入之前进行 OR 操作, 如下图所示。

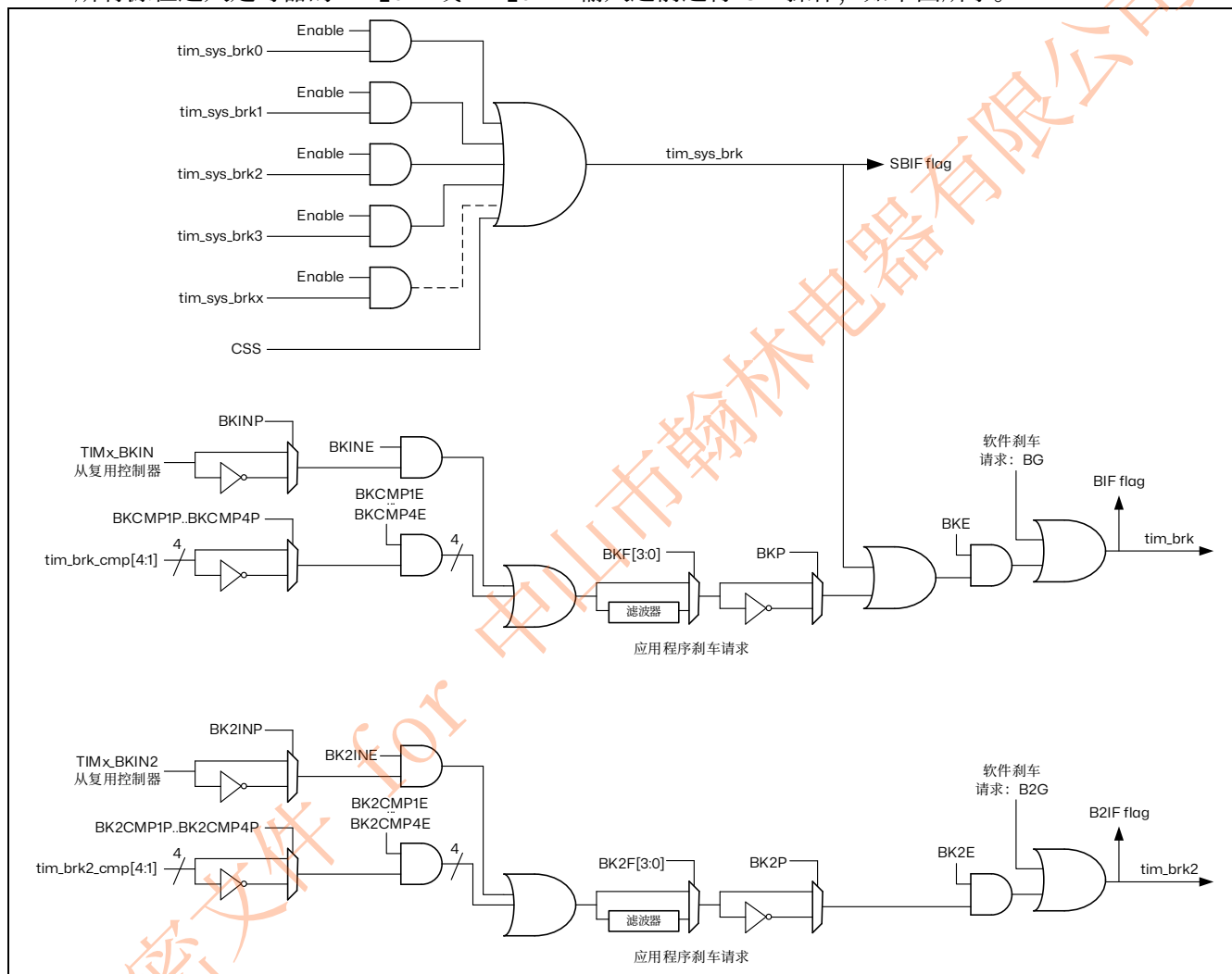


图 23.48 刹车和刹车 2 电路概述

注: 只有当可编程滤波器禁用时, 才保证异步 (无时钟) 操作。如果使能, 必须使用故障安全时钟模式 (例如使用内部 PLL 和/或 CSS) 来保证刹车事件得到处理。

当其中一个刹车发生时 (在所选的刹车输入线上检测到选定电平):

- MOE 位会被异步清除, 使输出处于非活动状态、空闲状态或甚至将控制权释放给 GPIO 控制器 (由 OSSI 位选择)。即使 MCU 振荡器关闭, 此功能也会使能。
- 一旦 MOE=0, 每个输出通道将驱动 TIMx\_CR2 寄存器中 OISx 位编程的电平。如果 OSSI=0, 定时器将释放输出控制 (由 GPIO 控制器接管), 否则使能输出保持高电平。

- 当使用互补输出时：
  - 输出首先处于无效状态（取决于极性）。这是异步完成的，因此即使没有向定时器提供时钟也可以正常工作。
  - 如果定时器的时钟仍然存在，则重新激活死区生成器，以便在死区后驱动 OISx 和 OISxN 位中编程的电平的输出。即使在这种情况下，tim\_ocx 和 tim\_ocxn 也不能同时驱动到其活动电平。请注意，由于在 MOE 上的重新同步，死区持续时间比平常稍长（大约 2 个 tim\_ker\_ck 时钟周期）。
  - 如果 OSS1=0，定时器将释放输出控制（由 GPIO 控制器接管，该控制器强制高阻态），否则使能输出保持或变为高电平，一旦 CCxE 或 CCxNE 位为高电平。
- 设置刹车标志位（TIMx\_SR 寄存器中的 SBIF、BIF 和 B2IF 位）。如果 TIMx\_DIER 寄存器中的 BIE 位已设置，则生成中断。如果 TIMx\_DIER 寄存器中的 BDE 位已设置，则生成 DMA 请求。
- 如果 TIMx\_BDTR 寄存器中的 AOE 位已设置，则在下一个更新事件（UEV）时自动重新设置 MOE 位。例如，这可以用于执行调节。否则，MOE 保持低电平，直到应用程序再次将其设置为“1”。在这种情况下，它可以用于安全，并将中断输入连接到来自功率驱动器、温度传感器或任何安全组件的警报。

注：如果 MOE 被 CPU 复位，而 AOE 位已设置，则输出处于空闲状态，并强制处于无效电平或高阻态，具体取决于 OSS1 值。如果 MOE 和 AOE 位都被 CPU 复位，则输出处于禁用状态，并驱动在 TIMx\_CR2 寄存器中 OISx 位编程的电平。

注：刹车输入在电平上有效。因此，当刹车输入处于有效状态时（无论是自动还是通过软件设置），MOE 都无法设置。同时，状态标志 BIF 和 B2IF 不能清除。

除刹车输入和输出管理外，刹车电路内部还实施了写保护，用以保护应用的安全。通过该功能，用户可冻结多个参数配置（死区持续时间、tim\_ocx/tim\_ocxn 极性和禁止时的状态、OCxM 配置、刹车使能和极性）。可以通过 TIMx\_BDTR 寄存器中的 LOCK 位，从 3 种保护级别中进行选择。请参考章节：TIMx 刹车和死区寄存器（TIMx\_BDTR）（x=1, 8）。MCU 复位后只能对 LOCK 位执行一次写操作。

下图所示为输出对刹车响应行为的示例。

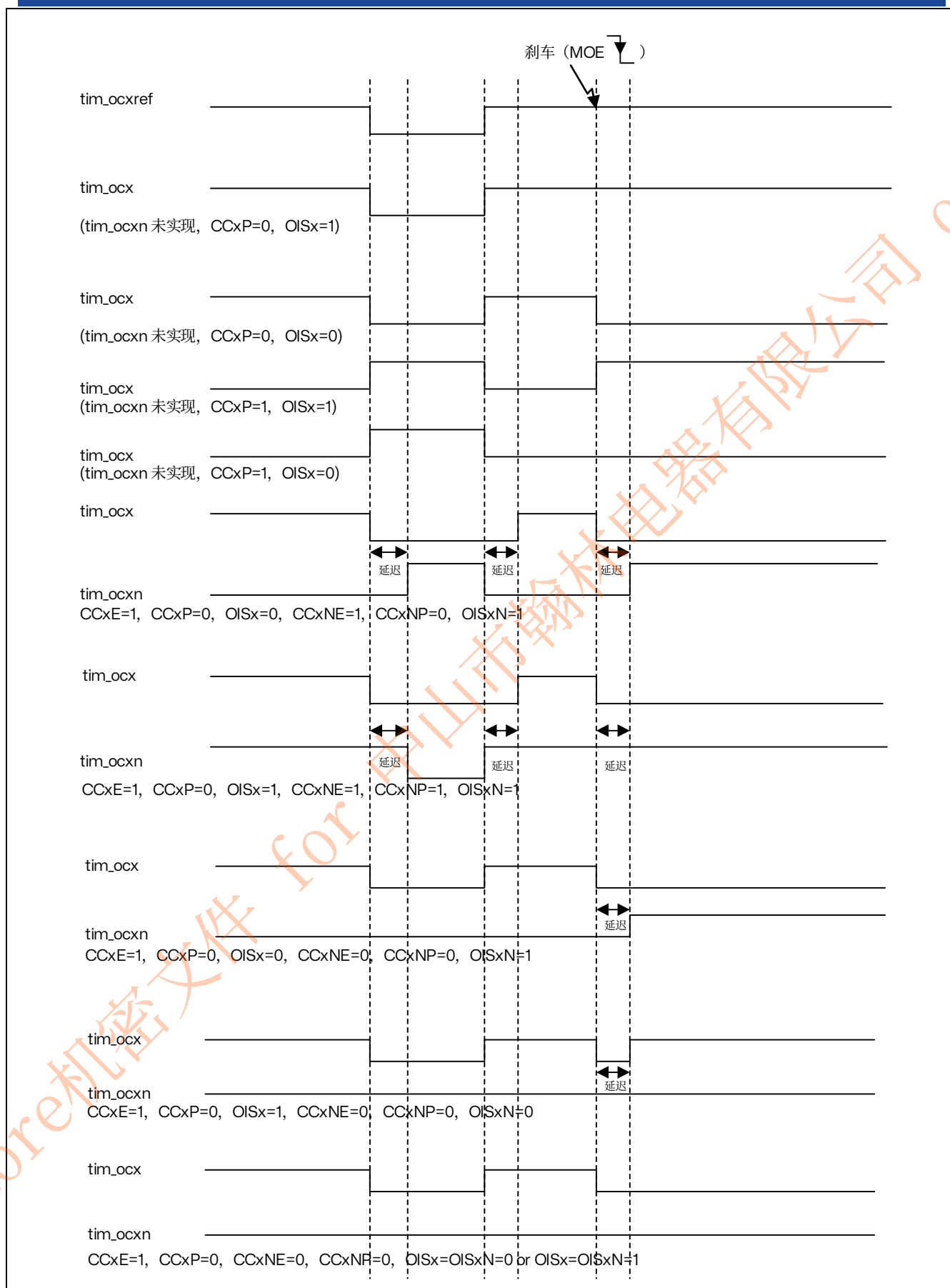


图 23.49 tim\_brk 在发生刹车事件时的各种输出行为 (OSSI=1)

两个中断输入在计时器输出上具有不同的行为：

- tim\_brk 输入可以禁用（非活动状态）或强制 PWM 输出为预定义的安全状态。
- tim\_brk2 只能禁用（非活动状态）PWM 输出。

如下表所述，tim\_brk 的优先级高于 tim\_brk2 输入。

注：tim\_brk2 只能与 OSSI = OSS1 = 1 一起使用。

表 23.15 与 tim\_brk/tim\_brk2 输入相对应的定时器输出行为

| tim_brk | tim_brk2 | 定时器<br>输出状态                                                                                                  | 典型用例                  |                       |
|---------|----------|--------------------------------------------------------------------------------------------------------------|-----------------------|-----------------------|
|         |          |                                                                                                              | Tim_ocxn 输出<br>(低侧开关) | Tim_ocxn 输出<br>(高侧开关) |
| 有效      | X        | <ul style="list-style-type: none"> <li>在无效后，强制输出状态（在死区之后）</li> <li>如果 OSS1=0，则输出禁用（由 GPIO 逻辑接管控制）</li> </ul> | 在死区时间插入后打开            | 关                     |
| 无效      | 有效       | 无效                                                                                                           | 关                     | 关                     |

下图给出了 tim\_ocx 和 tim\_ocxn 输出在 tim\_brk 和 tim\_brk2 输入上存在活动信号的情况下的行为示例。在这种情况下，两个输出都具有高电平有效极性（TIMx\_CCER 寄存器中的 CCxP = CCxNP = 0）。

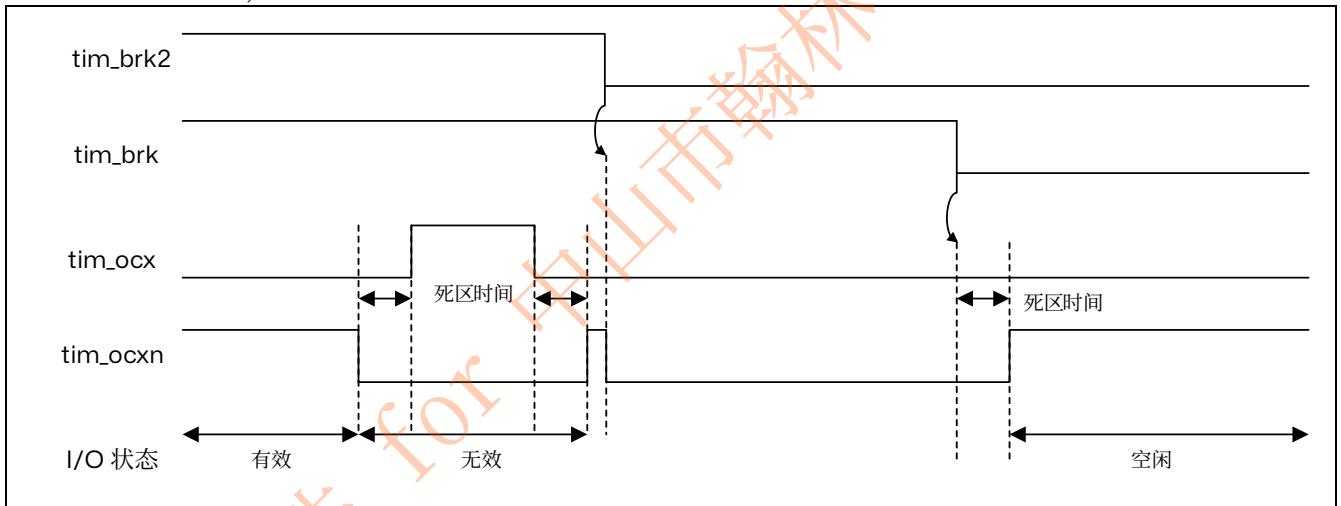


图 23.50 当 tim\_brk 和 tim\_brk2 有效 (OSSI=1) 时 PWM 输出状态

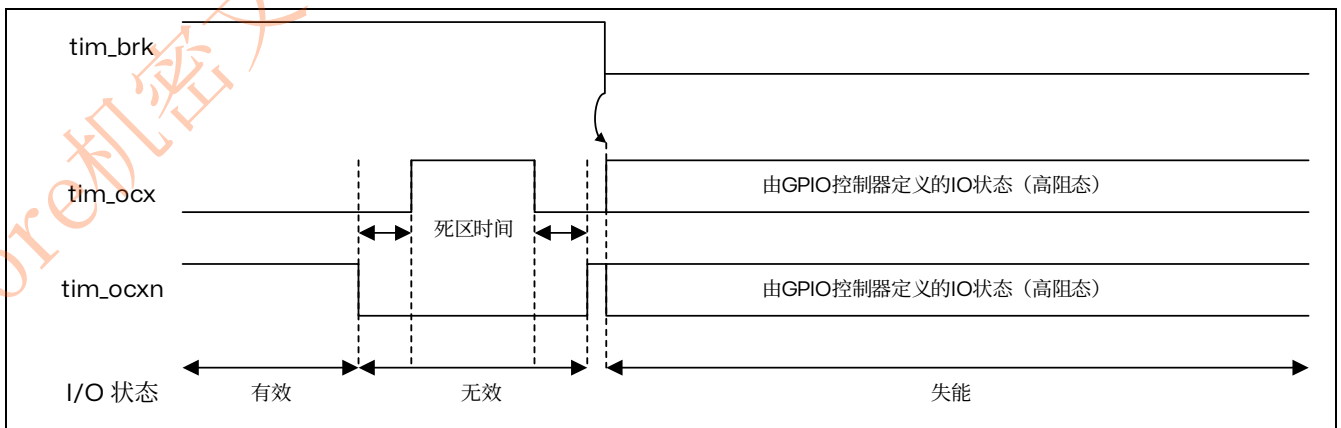


图 23.51 当 tim\_brk 有效 (OSSI=0) 时 PWM 输出状态



### 23.3.19 双向刹车输入

TIM1/TIM8 具有双向刹车 I/O，如下图所示。

这为以下方面提供支持：

- 板级全局刹车信号，用于向外部 MCU 或门驱动器发送故障信号，使用唯一的引脚作为输入和输出状态引脚
- 内部刹车源和多个外部开漏源 OR 连接在一起，以触发唯一的刹车事件，当多个内部和外部刹车源必须合并时

使用 TIMxBDTR 寄存器中的 BKBID 和 BK2BID 位配置 tim\_brk 和 tim\_brk2 输入为双向模式。

BKBID 编程位可以使用 TIMxBDTR 寄存器中的 LOCK 位锁定为只读模式（在 LOCK 级别 1 或更高）。

tim\_brk 和 tim\_brk2 输入都支持双向模式，并且要求 I/O 配置为开漏模式，使用低电平有效极性（使用 BKINP、BKP、BK2INP 和 BK2P 位）。来自系统（例如 CSS）、片上外设或刹车输入的任何刹车请求都会强制将刹车输入置于低电平，以指示故障事件。出于安全原因，如果极性位没有正确设置（高电平有效），则双向模式将被抑制。

刹车软件事件（BG 和 B2G）也会强制将刹车 I/O 置为“0”，以指示外部组件定时器已进入刹车状态。然而，只有使能了刹车（BKE 或 B2KE = 1）时，这才有效。当使用 BKE 或 B2KE = 0 生成软件刹车事件时，输出将处于安全状态，并且会设置刹车标志，但 TIMx\_BKIN 和 TIMx\_BKIN2 I/O 没有影响。

安全解发机制可防止系统被彻底锁定（刹车输入低电平触发刹车，强制该输入为低电平）。

当 BKDSRM（BK2DSRM）位设置为 1 时，这将释放刹车输出以清除故障信号并允许重新给系统装载。

在任何时候都不能禁用刹车保护电路：

- 刹车输入路径始终处于有效状态：即使设置了 BKDSRM（BK2DSRM）位并释放了开漏控制，刹车事件也是有效的。这可以防止在存在刹车条件时重新启动 PWM 输出。
- 只要输出使能（MOE 位设置），BKDSRM（BK2DSRM）位就不能解除刹车保护（参见下表）。

表 23.16 刹车保护解除条件

| MOE | BKBID<br>(BK2BID) | BKDSRM<br>(BK2DSRM) | 刹车保护状态 |
|-----|-------------------|---------------------|--------|
| 0   | 0                 | X                   | 装载     |
| 0   | 1                 | 0                   | 装载     |
| 0   | 1                 | 1                   | 解除     |
| 1   | X                 | X                   | 装载     |

#### 装载和重装载刹车电路

默认情况下（外设复位配置），刹车电路（输入或双向模式）处于装载状态。

在发生刹车（刹车 2）事件后，必须按照以下步骤重新装载保护：

- BKDSRM（BK2DSRM）位必须设置为释放输出控制
- 软件必须等待系统刹车条件消失（如果有的话），并清除 SBIF 状态标志（或在重新装载之前系统地清除它）
- 软件必须轮询 BKDSRM（BK2DSRM）位，直到硬件清除该位（当应用程序刹车条件消失时）从这一点开始，刹车电路处于装载和有效状态，并且可以设置 MOE 位以重新使能 PWM 输出。

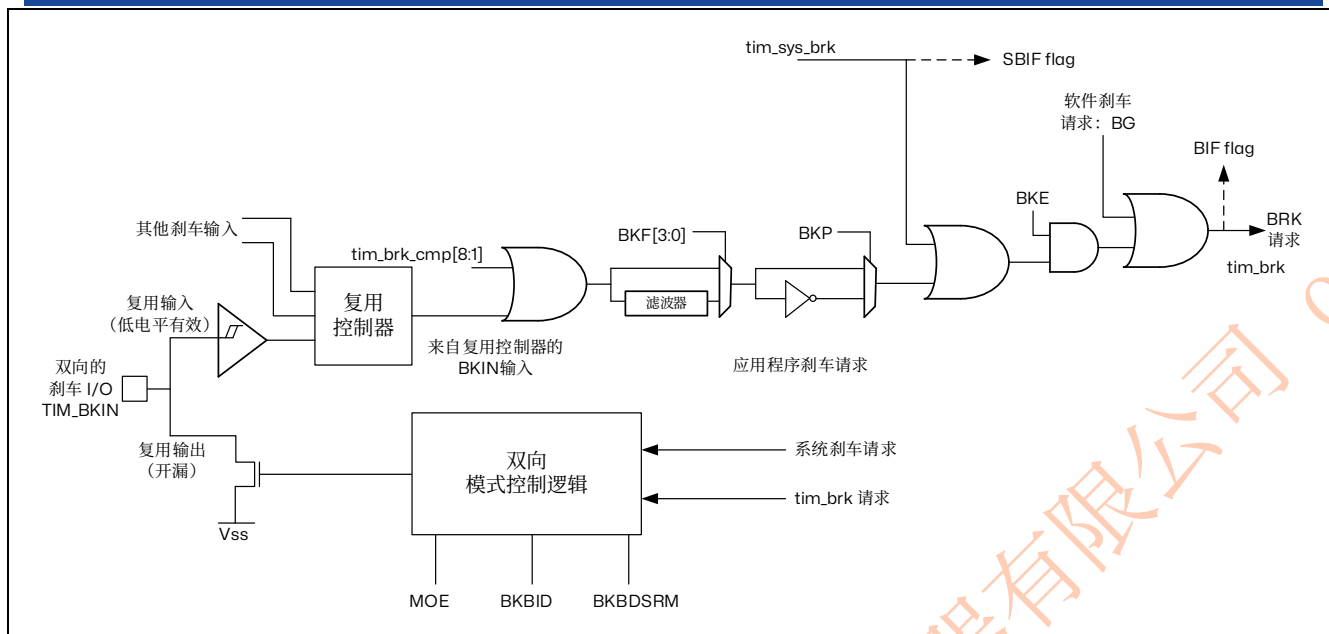


图 23.52 输出重定向（未表示 tim\_brk2 请求）

### 23.3.20 在外部事件时清除 tim\_ocxref 信号

对于给定通道，在 ETRF 输入施加高电平（相应 TIMx\_CCMRx 寄存器中的 OCxCE 使能位置“1”），可使 OCxREF 信号变为低电平。OCxREF 信号将保持低电平，直到发生下一更新事件(UEV)。

此功能仅能用于输出比较模式和 PWM 模式，而不适用于强制输出模式。可以通过配置 TIMx\_SMCR 寄存器中的 OCCS 位，将 tim\_ocref\_clr\_int 输入选择为 tim\_ocref\_clr 输入和 tim\_etrfr（滤波后的 tim\_etr\_in）之间。

tim\_ocref\_clr 输入可以在多个输入中进行选择，使用 TIMx\_AF2 寄存器中的 OCRSEL[2:0] 位域，如下图所示。请参考章节：TIM1/TIM8 引脚和内部信号，以获取产品中可用源的列表。

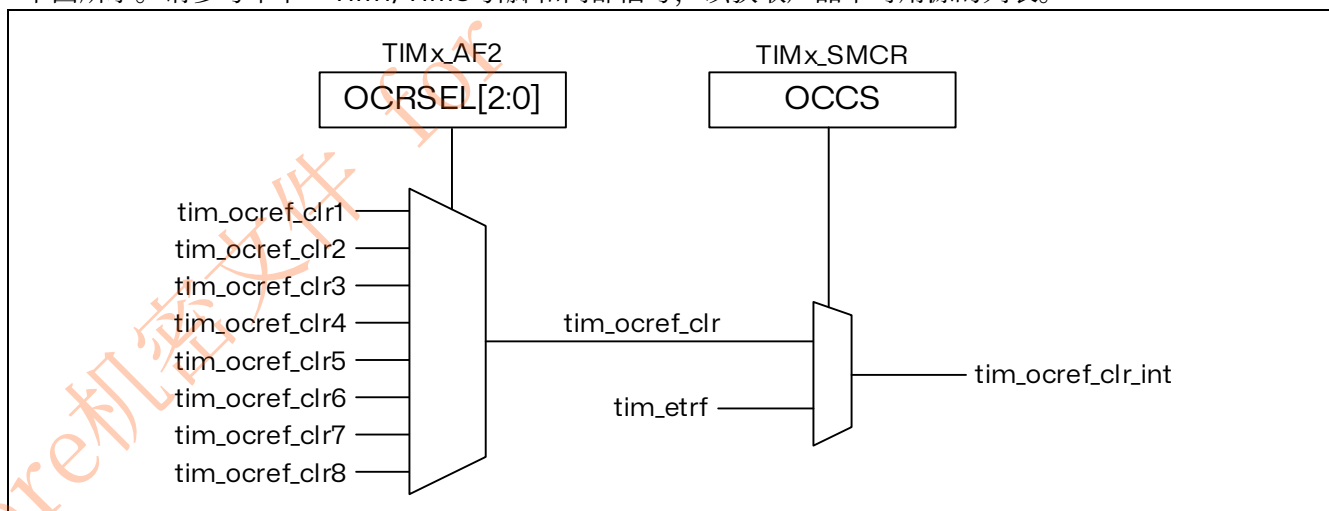


图 23.53 tim\_ocref\_clr 输入选择复用器

当 tim\_etrfr 被选择。tim\_etr\_in 必须如下配置：

1. 必须关闭外部触发预分频器：TIMx\_SMCR 寄存器中的 ETPS[1:0] 位清除为 00。
2. 必须禁止外部时钟模式 2：TIMx\_SMCR 寄存器中的 ECE 位清除为 0。

- 下图对比了使能位 OCxCE 在不同值下的情况, 显示了当 tim\_etrif 输入变为高电平时 tim\_etrif 信号的行为。在本例中, 定时器 TIMx 编程为 PWM 模式。

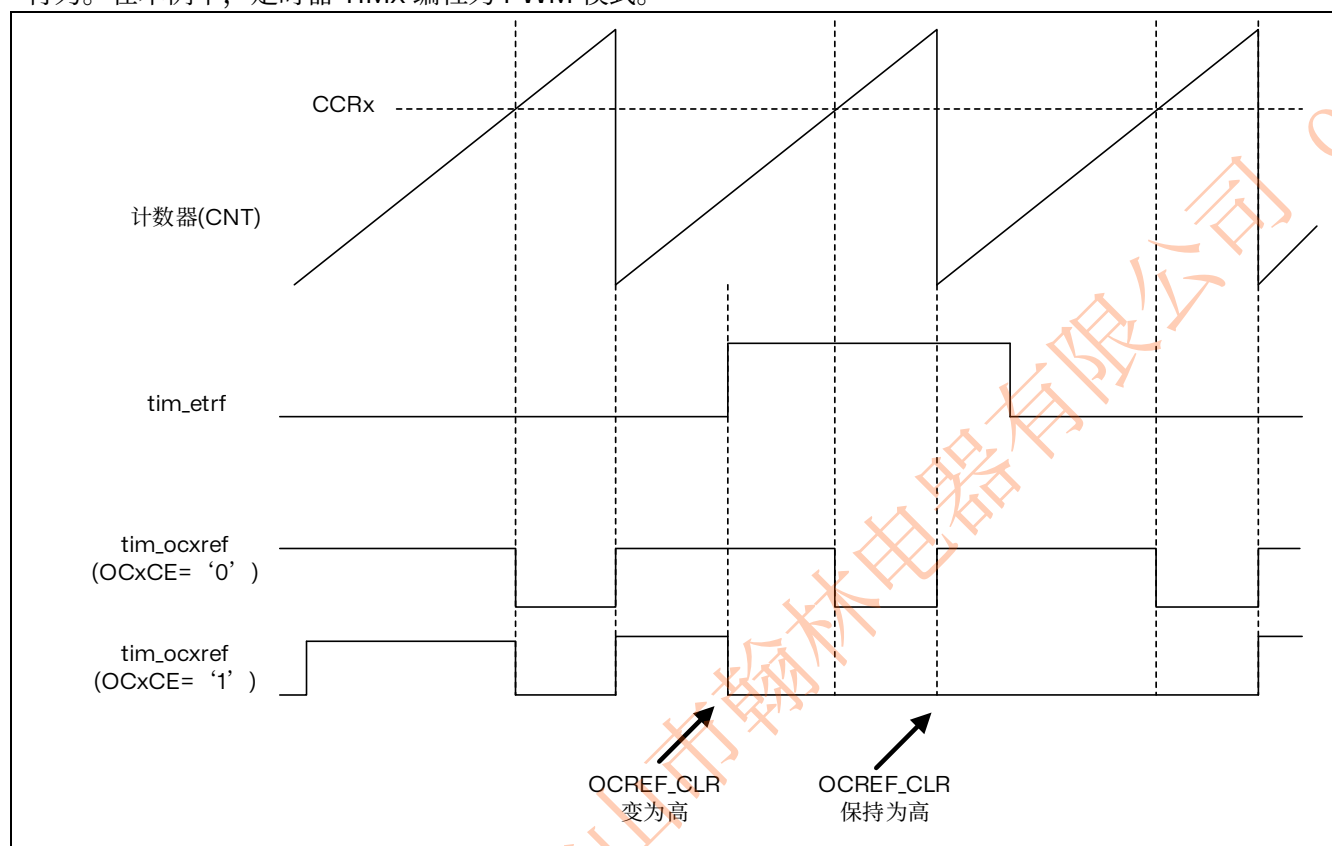


图 23.54 清除 TIMx 的 tim ocxref

注意：如果 PWM 的占空比为 100% ( $CCR_x > ARR$ )，则下次计数器溢出时会再次使能 `tim_ocxref`。

### 23.3.21 生成 6 步 PWM

当通道使用互补输出时，OCxM、CCxE 和 CCxNE 位上提供预装载位。发生 COM 换向事件时，这些预装载位将传输到影子位。因此，用户可以预先编程下一步骤的配置，并同时更改所有通道的配置。COM 可由软件通过将 TIMx\_EGR 寄存器中的 COM 位置 1 而生成，也可以由硬件在 tim\_trgi 上升沿生成。

发生 COM 事件时，某个标志位（TIMx\_SR 寄存器中的 COMIF 位）将会置 1。这时将产生中断（如果 TIMx\_DIER 寄存器中的 COMIE 位置 1）或 DMA 请求（如果 TIMx\_DIER 寄存器中的 COMDE 位置 1）。

下图以 3 种不同的编程配置为例，显示了发生 COM 事件时 tim\_ocx 和 tim\_ocxn 输出的行为。

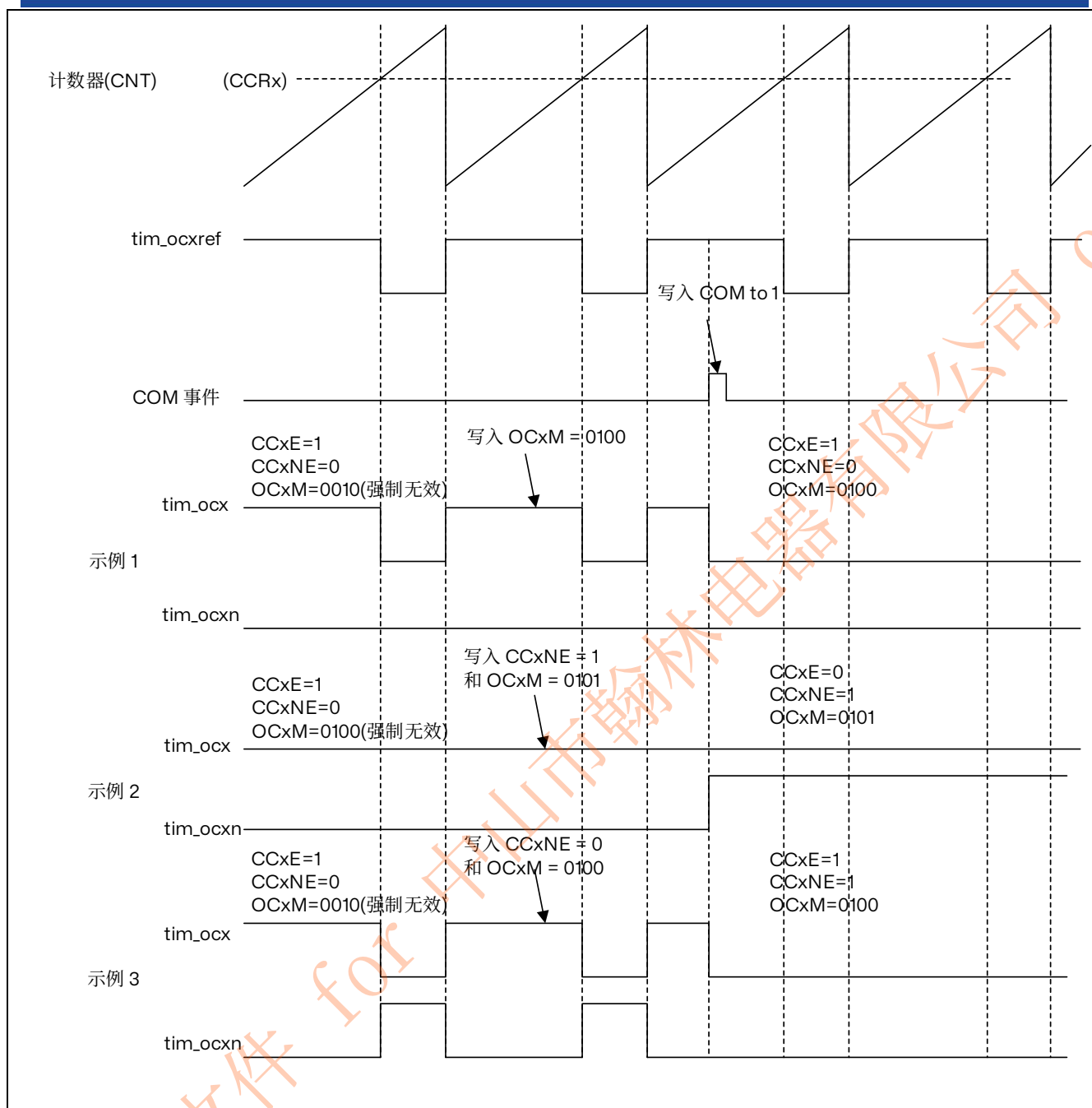


图 23.55 COM 事件生成 6 步 PWM 的示例 (OSSR=1)

### 23.3.22 单脉冲模式

单脉冲模式（OPM）是上述模式的一个特例。在这种模式下，计数器可以在一个激励信号的触发下启动，并可在一段可编程的延时后产生一个脉宽可编程的脉冲。

可以通过从模式控制器启动计数器。可以在输出比较模式或 PWM 模式下生成波形。将 TIMx\_CR1 寄存器中的 OPM 位置 1，即可选择单脉冲模式。这样，发生下一更新事件 UEV 时，计数器将自动停止。

只有当比较值与计数器初始值不同时，才能正确产生一个脉冲。启动前（定时器等待触发时），必须进行如下配置：

- 递增计数模式下： $CNT < CCRx \leq ARR$ （特别注意， $0 < CCRx$ ）
- 递减计数模式下： $CNT > CCRx$

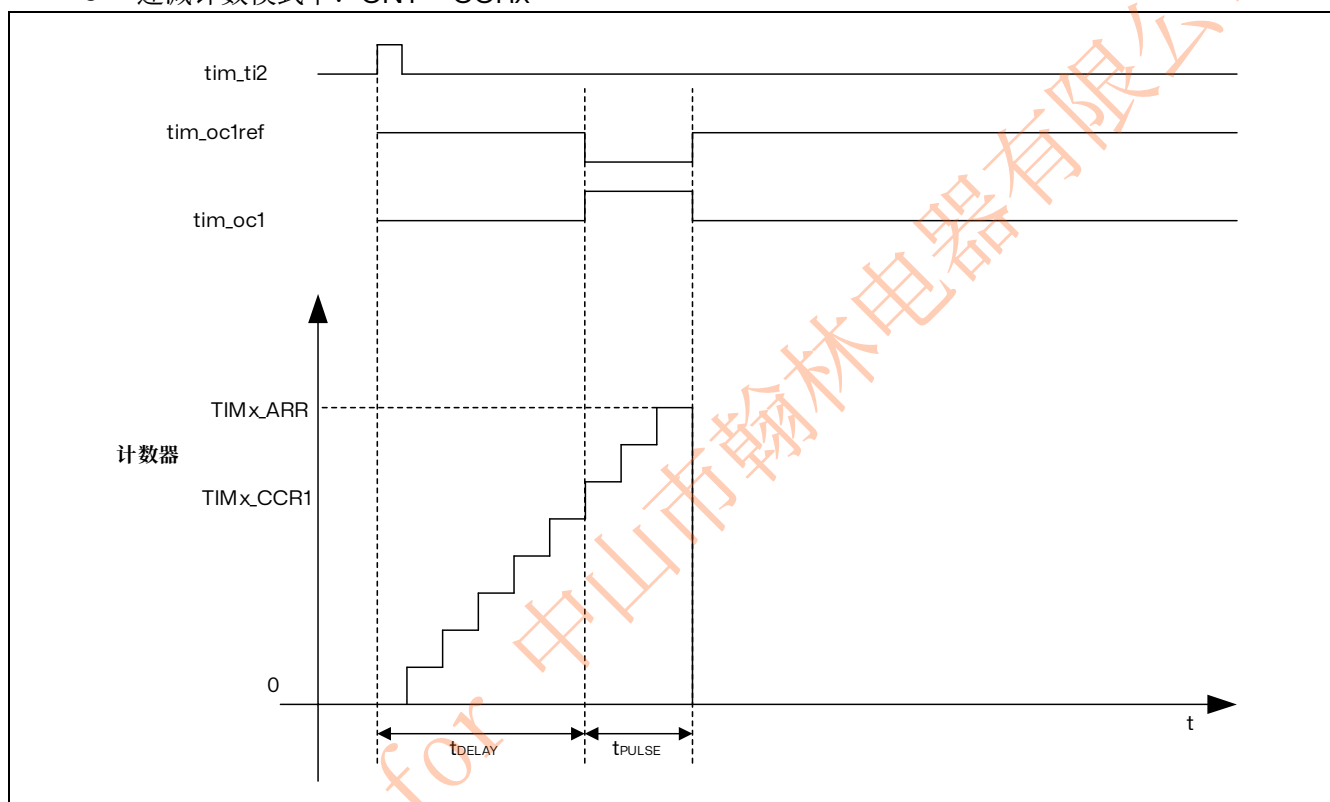


图 23.56 单脉冲模式的例子

例如，用户希望达到这样的效果：在 tim\_ti2 输入引脚检测到正沿时，经过  $t_{DELAY}$  的延迟，在 tim\_oc1 上产生一个长度为  $t_{PULSE}$  的正脉冲。

使用 tim\_ti2fp2 作为触发 1：

1. 在 TIMx\_CCMR1 寄存器中写入 CC2S=“01”，将 tim\_ti2fp2 映射到 tim\_ti2。
2. 在 TIMx\_CCER 寄存器中写入 CC2P=“0”和 CC2NP=“0”，使 tim\_ti2fp2 能够检测上升沿。
3. 在 TIMx\_SMCR 寄存器中写入 TS=00110，将 tim\_ti2fp2 配置为从模式控制器的触发 (tim\_trgi)。
4. 在 TIMx\_SMCR 寄存器中写入 SMS=“110”（触发模式），使用 tim\_ti2fp2 启动计数器。

OPM 波形通过对比较寄存器执行写操作来定义（考虑时钟频率和计数器预分频器）。

- $t_{DELAY}$  由写入 TIMx\_CCR1 寄存器的值定义。
- $t_{PULSE}$  由自动重载值与比较值 (TIMx\_ARR - TIMx\_CCR1) 之差来定义。
- 假设希望产生这样的波形：信号在发生比较匹配时从“0”变为“1”，在计数器达到自动重载值

时由“1”变为“0”。为此，应在 TIMx\_CCMR1 寄存器中写入 OC1M=111，以使能 PWM 模式 2。如果需要，可选择在 TIMx\_CCMR1 寄存器的 OC1PE 和 TIMx\_CR1 寄存器的 ARPE 中写入“1”，以使能预装载寄存器。这种情况下，必须在 TIMx\_CCR1 寄存器中写入比较值并在 TIMx\_ARR 寄存器中写入自动重载值，通过将 UG 位置 1 来产生更新，然后等待 tim\_ti2 上的外部触发事件。本例中，CC1P 的值为“0”。

在本例中，TIMx\_CR1 寄存器中的 DIR 和 CMS 位应为低。

由于仅需要 1 个脉冲（单脉冲模式），因此应向 TIMx\_CR1 寄存器的 OPM 位写入“1”，以便在发生下一更新事件（计数器从自动重载值返回到 0）时使计数器停止计数。TIMx\_CR1 寄存器中的 OPM 位置“0”时，即选择重复模式。

#### 特例：tim\_ocx 快速使能：

在单脉冲模式下，tim\_tix 输入的边沿检测会将 CEN 位置 1，表示使能计数器。然后，在计数器值与比较值之间发生比较时，将切换输出。但是，完成这些操作需要多个时钟周期，这会限制可能的最小延迟（ $t_{\text{DELAY}}$  最小值）。

如果要输出延迟时间最短的波形，可以将 TIMx\_CCMRx 寄存器中的 OCxFE 位置 1。这样会强制 tim\_ocxref（和 tim\_ocx）对激励信号做出响应，而不再考虑比较的结果。其新电平与发生比较匹配时相同。仅当通道配置为 PWM1 或 PWM2 模式时，OCxFE 才会起作用。

### 23.3.23 可重触发单脉冲模式

此模式允许计数器在接收到刺激后开始计数，并生成一个具有可编程长度的脉冲，但与上一节中描述的非重触发单脉冲模式存在以下差异：

- 只要触发器发生，脉冲就会立即开始（没有可编程的延迟）
- 如果在先前触发的脉冲完成之前发生新的触发，则脉冲会被延长

定时器必须处于从模式，TIMx\_SMCR 寄存器的 SMS[3:0] 位设置为“1000”（组合重置+触发模式），OCxM[3:0] 位设置为“1000”或“1001”，用于可重触发 OPM 模式 1 或 2。

如果定时器配置为递增计数模式，则相应的 CCRx 必须设置为 0（ARR 寄存器设置脉冲长度）。如果定时器配置为递减计数模式，CCRx 必须大于或等于 ARR。

注：OCxM[[3:0] 和 SMS[3:0] 位域由于兼容性的原因被分成两个部分，最高有效位与最低 3 位不是连续的。OCxM 这种模式不能与中央对齐 PWM 模式一起使用。TIMx\_CR1 寄存器中的 CMS[1:0] 必须设置为 00。

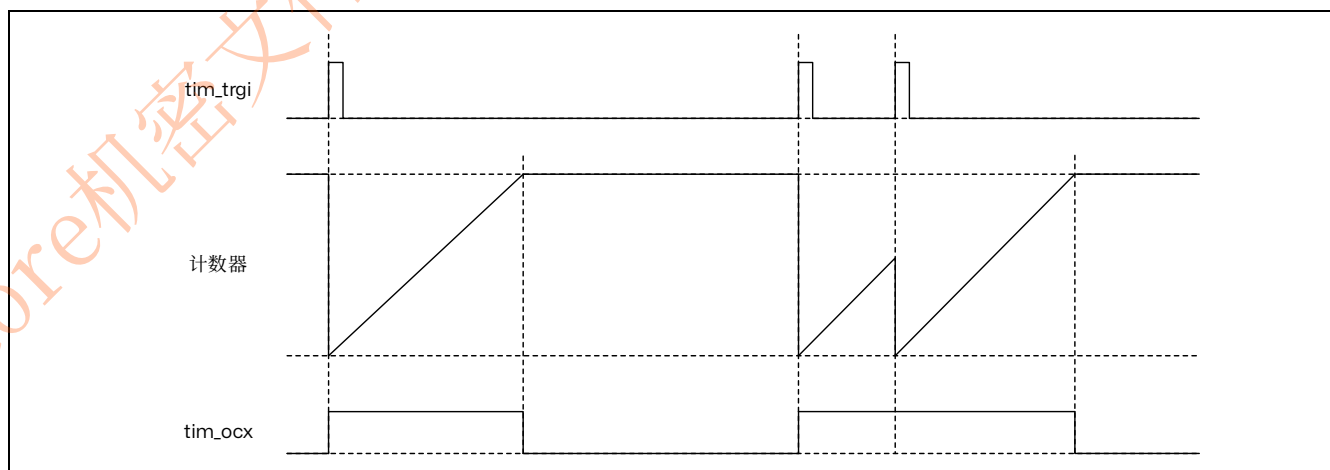


图 23.57 可重触发单脉冲模式

### 23.3.24 在比较模式下的脉冲

在比较匹配事件发生时，可以生成一个脉冲。当计数器的值等于给定的比较值时，可以生成一个具有可编程脉冲宽度的信号，用于调试或同步。

此模式适用于任何从机模式选择，包括边沿对齐和中心对齐计数模式。它仅适用于通道 3 和通道 4。脉冲发生器是唯一的，由两个通道共享，如下图所示。

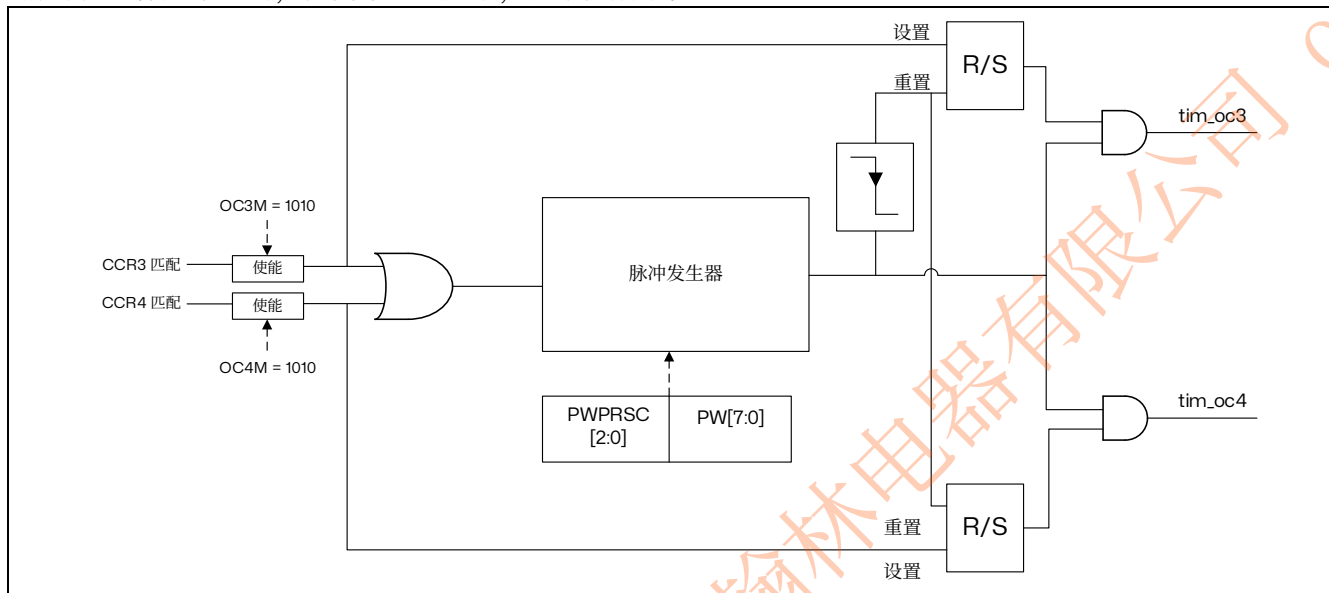


图 23.58 脉冲发生器电路

下图展示了如何在边沿对齐模式下生成脉冲。

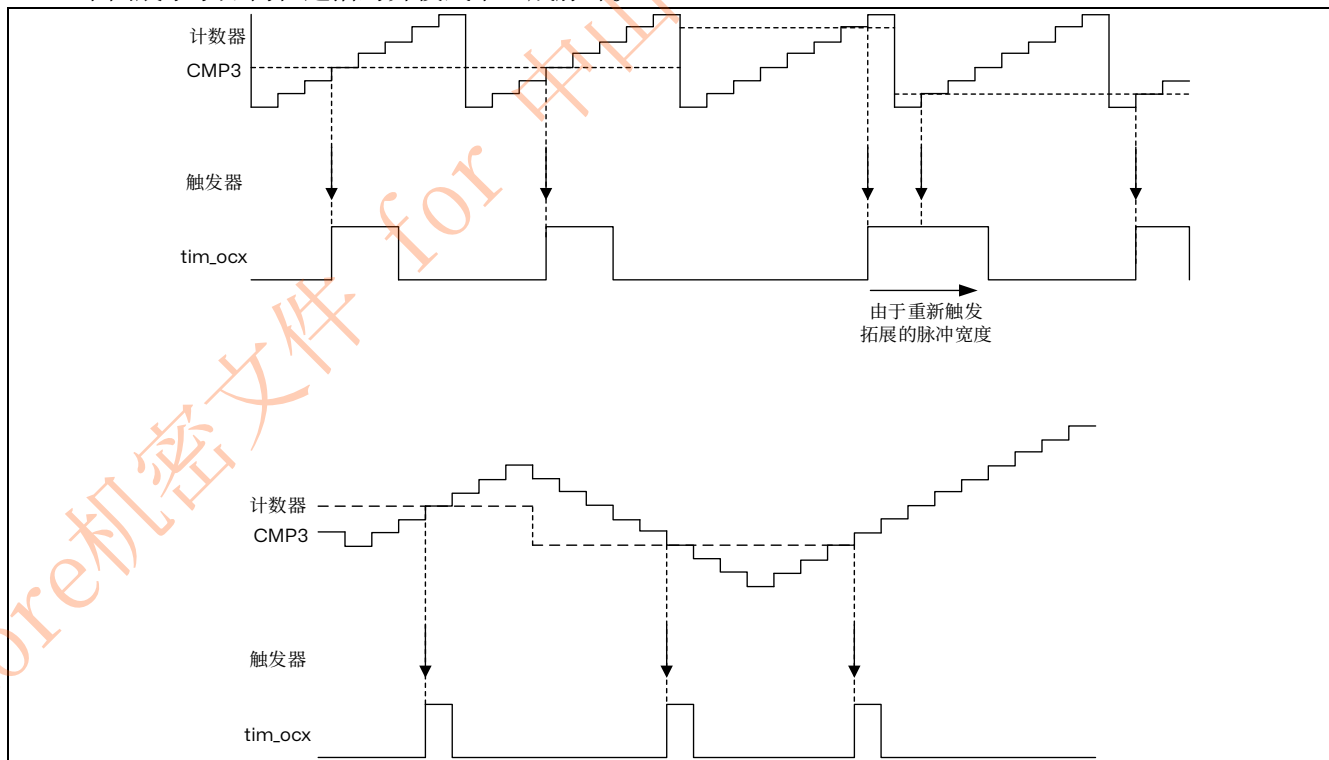


图 23.59 在比较事件上的脉冲生成，适用于边沿对齐模式



此输出比较模式是通过 TIMx\_CCMR2 寄存器中的 OC3M[3:0]和 OC4M[3:0]位域进行选择的。

脉冲宽度通过寄存器中的 PW[7:0]位域进行编程，使用特定的时钟，并根据 PWPRSC[2:0]位的预设进行预分频，具体如下：

$$t_{PW} = PW[7:0] \times t_{PWG}$$

$$\text{其中 } t_{PWG} = (2^{(PWPRSC[2:0])}) \times t_{tim\_ker\_ck}$$

给出了分辨率和最大值，具体取决于预分频器的值。

脉冲可以重新触发：在脉冲正在进行时触发新的脉冲，会导致脉冲被延长。

注：如果同时使能两个通道，只要一个通道上的触发器不与并发输出上生成的脉冲重叠，脉冲就会独立发出。相反，如果两个触发器重叠，与第一个到达的触发器相关的脉冲宽度会被延长（由于重新触发），而最后一个到达的触发器的脉冲宽度是正确的（如下图所示）。

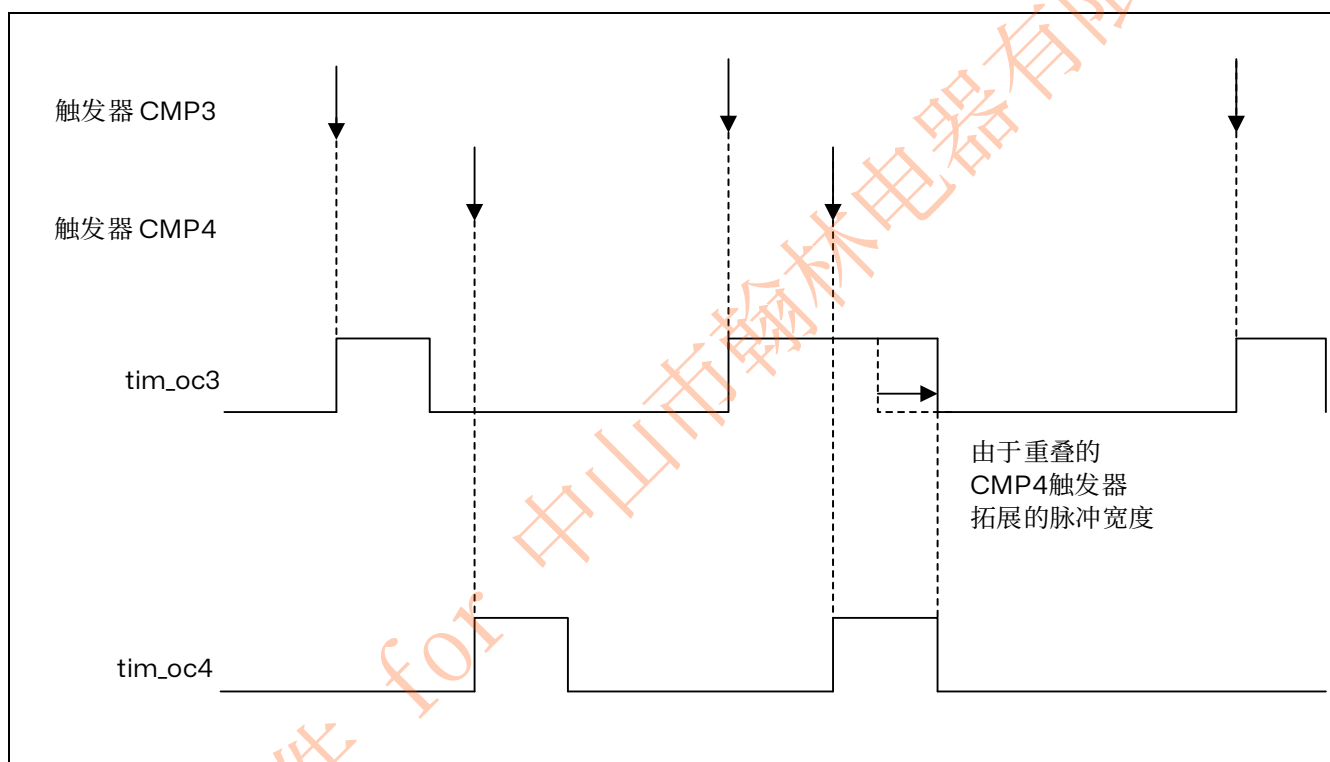


图 23.60 在同时触发的情况下延长脉冲宽度

### 23.3.25 编码器接口模式

#### 正交编码器

选择编码器接口模式时，如果计数器仅在 tim\_ti2 边沿处计数，在 TIMx\_SMCR 寄存器中写入 SMS=0001；如果计数器仅在 tim\_ti1 边沿处计数，写入 SMS=0010；如果计数器在 tim\_ti1 和 tim\_ti2 边沿处均计数，则写入 SMS=0011。

通过编程 TIMx\_CCER 寄存器的 CC1P 和 CC2P 位，选择 tim\_ti1 和 tim\_ti2 极性。如果需要，还可对输入滤波器进行编程。CC1NP 和 CC2NP 必须保持为低电平。

tim\_ti1 和 tim\_ti2 两个输入用于连接增量编码器。请参见下表。如果使能计数器（在 TIMx\_CR1 寄存器的 CEN 位中写入“1”），则计数器的时钟由 tim\_ti1fp1 或 tim\_ti2fp2 上的每次有效信号转换提供。

tim\_ti1fp1 和 tim\_ti2fp2 是进行输入滤波器和极性选择后 tim\_ti1 和 tim\_ti2 的信号，如果不进行滤波和反相，则 tim\_ti1fp1=tim\_ti1，tim\_ti2fp2=tim\_ti2。将根据两个输入的信号转换序列，产生计数脉冲和方向信号。根据该信号转换序列，计数器相应递增或递减计数，同时硬件对 TIMx\_CR1 寄存器的 DIR 位进行相应修改。任何输入（tim\_ti1 或 tim\_ti2）发生信号转换时，都会计算 DIR 位，无论计数器是仅在 tim\_ti1 或 tim\_ti2 边沿处计数，还是同时在 tim\_ti1 和 tim\_ti2 处计数。

编码器接口模式就相当于带有方向选择的外部时钟。这意味着，计数器仅在 0 到 TIMx\_ARR 寄存器中的自动重载值之间进行连续计数（根据具体方向，从 0 递增计数到 ARR，或从 ARR 递减计数到 0）。因此，在启动前必须先配置 TIMx\_ARR。同样，捕获、比较、预分频器、触发输出功能继续正常工作。编码器模式和外部时钟模式 2 不兼容，因此不能同时选择。

在此模式下，计数器会根据增量编码器的速度和方向自动进行修改，因此，其内容始终表示编码器的位置。计数方向对应于所连传感器的旋转方向。下表汇总了可能的组合（假设 tim\_ti1 和 tim\_ti2 不同时切换）。

表 23.17 计数方向与编码器信号的关系 (CC1P = CC2P = 0)

| 有效边沿                           | SMS[3:0] | 相对信号的电平<br>(tim_ti1fp1 对应 tim_ti2,<br>tim_ti2fp2 对应 tim_ti1) | tim_ti1fp1 信号 |     | tim_ti2fp2 信号 |     |
|--------------------------------|----------|--------------------------------------------------------------|---------------|-----|---------------|-----|
|                                |          |                                                              | 上升            | 下降  | 上升            | 下降  |
| 仅 x1 模式在<br>tim_ti1 计数         | 1110     | 高                                                            | 递减            | 递增  | 不计数           | 不计数 |
|                                |          | 低                                                            | 不计数           | 不计数 | 不计数           | 不计数 |
| 仅 x1 模式在<br>tim_ti2 计数         | 1111     | 高                                                            | 不计数           | 不计数 | 递增            | 递减  |
|                                |          | 低                                                            | 不计数           | 不计数 | 不计数           | 不计数 |
| 仅 x2 模式在<br>tim_ti1 计数         | 0001     | 高                                                            | 递减            | 递增  | 不计数           | 不计数 |
|                                |          | 低                                                            | 递增            | 递减  | 不计数           | 不计数 |
| 仅 x2 模式在<br>tim_ti2 计数         | 0010     | 高                                                            | 不计数           | 不计数 | 递增            | 递减  |
|                                |          | 低                                                            | 不计数           | 不计数 | 递减            | 递增  |
| x4 模式在 tim_ti1<br>和 tim_ti2 计数 | 0011     | 高                                                            | 递减            | 递增  | 递增            | 递减  |
|                                |          | 低                                                            | 递增            | 递减  | 递减            | 递增  |

外部增量编码器可直接与 MCU 相连，无需外部接口逻辑。不过，通常使用比较器将编码器的差分输出转换为数字信号。这样大幅提高了抗噪声性能。用于指示机械零位的第三个编码器输出可与外部中断输入相连，用以触发计数器复位。

下图以计数器工作为例，说明了计数信号的生成和方向控制。同时也说明了选择双边沿时如何对输入抖动进行补偿。将传感器靠近其中一个切换点放置时可能出现这种情况。本例中假设配置如下：

- CC1S= “01” (TIMx\_CCMR1 寄存器，tim\_ti1fp1 映射到 tim\_ti1 上)。
- CC2S= “01” (TIMx\_CCMR2 寄存器，tim\_ti1fp2 映射到 tim\_ti2 上)。
- CC1P= “0”，CC1NP= “0” (TIMx\_CCER 寄存器，tim\_ti1fp1 未反相，tim\_ti1fp1= tim\_ti1)。
- CC2P= “0”，CC2NP= “0” (TIMx\_CCER 寄存器，tim\_ti1fp2 未反相，tim\_ti1fp2= tim\_ti2)。
- SMS= “011” (TIMx\_SMCR 寄存器，两个输入在上升沿和下降沿都处于有效状态)。
- CEN= “1” (TIMx\_CR1 寄存器，使能计数器)。

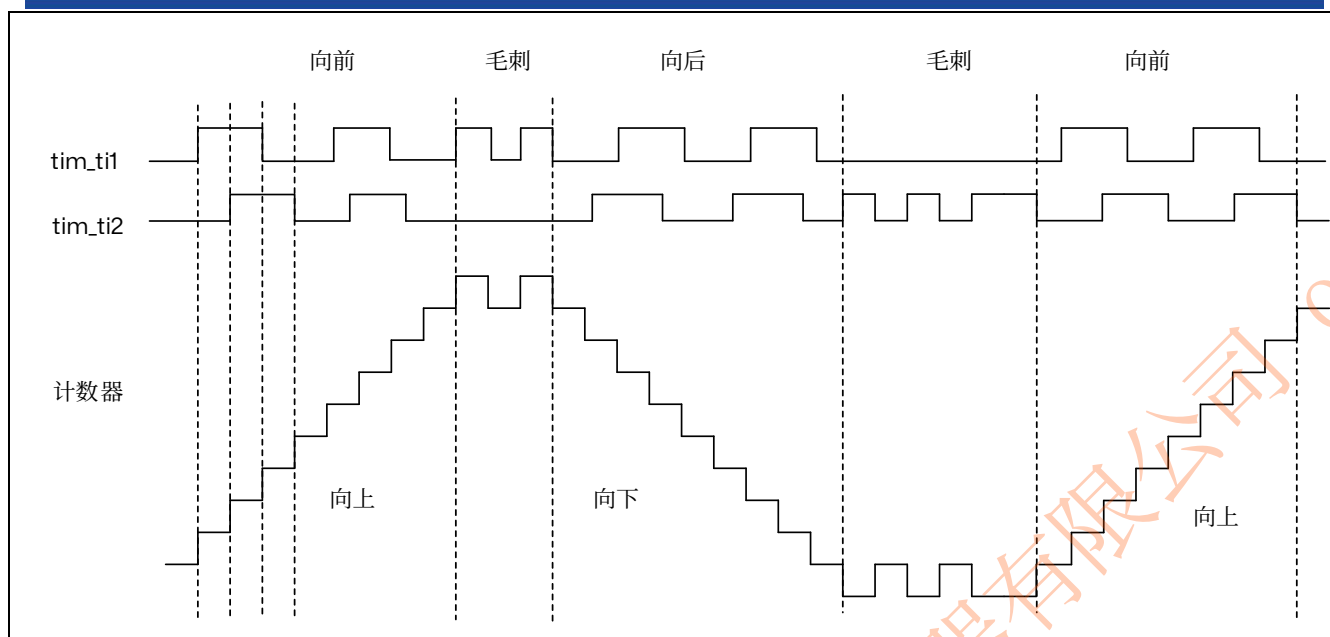


图 23.61 编码器接口模式下的计数器工作示例

下图举例说明 `tim_ti1fp1` 极性反相时计数器的行为（除 `CC1P=“1”` 外，其它配置与上例相同）。

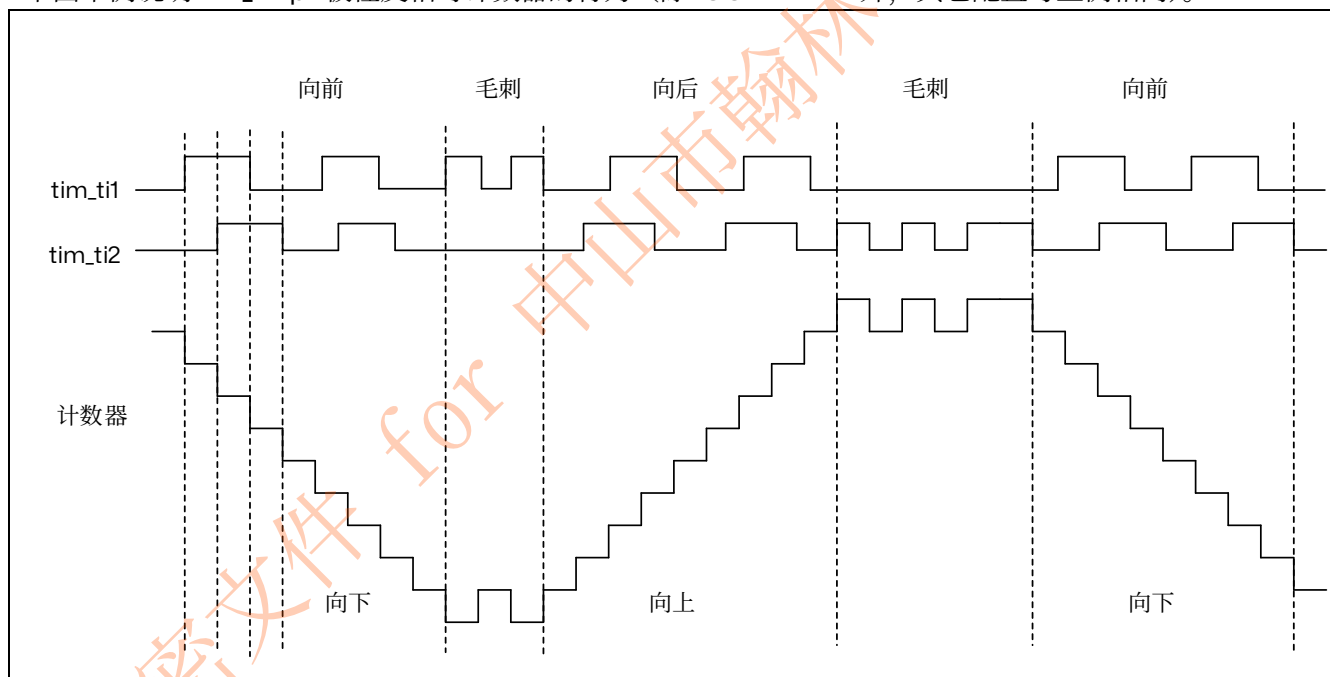


图 23.62 `tim_ti1fp1` 极性反相时的编码器接口模式示例

下图显示了各种计数模式下速度反转期间计时器计数器的值。

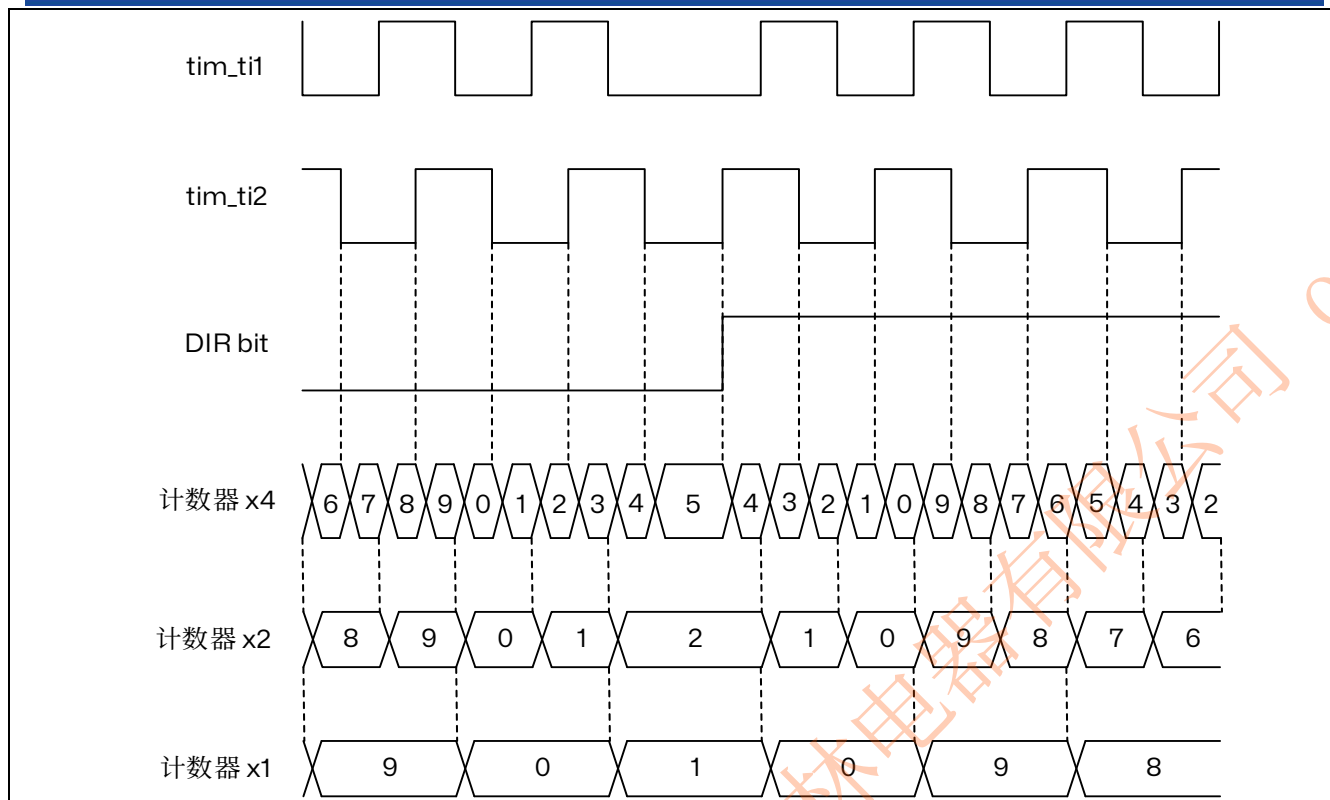


图 23.63 正交编码器的计数模式

当定时器配置为编码器接口模式时，可以提供传感器当前位置的信息。通过测量两个编码器事件之间的周期，使用配置为捕获模式的第二个定时器，可以获得动态信息（速度、加速度、减速度）。指示机械零的编码器输出可用于此目的。根据两个事件之间的时间，还可以定期读取计数器的值。如果可用，可以通过将计数器的值锁存到第三个输入捕获寄存器来实现（此时捕获信号必须是周期性的，并且可以由另一个定时器生成）。当可用时，也可以通过 DMA 请求读取其值。

TIMx\_CR1 寄存器中的 IUFREMAP 位将更新中断标志（UIF）的连续副本强制复制到定时器计数器寄存器的第 31 位（TIMxCNT[31]）。这允许原子地读取计数器的值和由 UIFCPY 标志触发的潜在的翻转条件。这可以通过避免在后台任务（计数器读取）和中断（更新中断）之间共享处理来简化角速度的计算。

UIF 和 UIFCPY 标志之间没有延迟。

在 32 位定时器实现中，当 IUFREMAP 位被设置时，在读取访问时，计数器的第 31 位将被 UIFCPY 标志覆盖（计数器最高有效位仅可在写入模式下访问）。

### 时钟加方向编码器模式

除了正交编码器模式外，定时器还支持其他类型的编码器。

在下图中显示的“时钟加方向”模式下，时钟在 tim\_ti2 上提供单线，而方向使用 tim\_ti1 输入强制。此模式通过 TIMx\_SMCR 寄存器中的 SMS[3:0] 位域使能，如下所示：

- 1010: x2 模式，计数器在时钟的上升沿和下降沿都更新
- 1011: x1 模式，计数器在单个时钟边沿更新，根据 CC2P 位的值而定：CC2P=0 对应上升沿敏感，CC2P=1 对应下降沿敏感。

tim\_ti1 上的方向信号极性通过 CC1P 位设置：0 对应正极性（当 tim\_ti1 为高电平时向上计数，当 tim\_ti1 为低电平时向下计数），CC1P=1 对应负极性（当 tim\_ti1 为低电平时向上计数）。

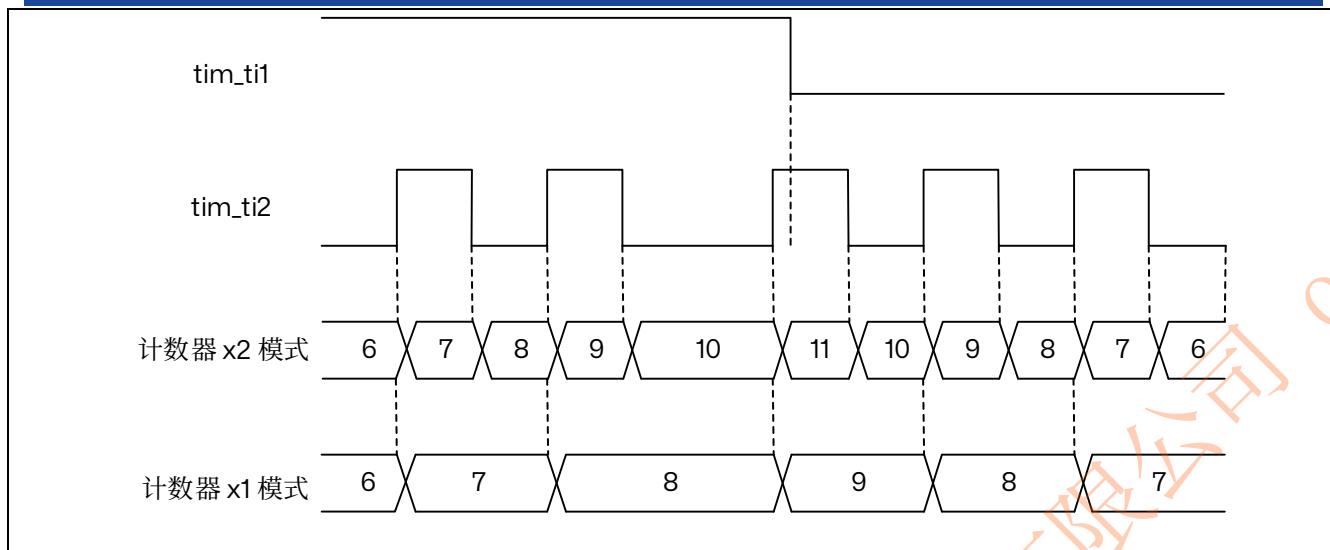


图 23.64 方向加时钟编码器模式

### 方向时钟编码器模式

在下图中的“方向时钟”模式下，时钟在两条线上提供，根据方向一次使用一条，以便有一条向上计数时钟线和一条向下计数时钟线。

此模式通过 TIMx\_SMCR 寄存器中的 SMS[3:0] 位域使能，如下所示：

- 1100: x2 模式，计数器在两条时钟线的上升沿和下降沿都更新。CC1P 和 CC2P 位编码时钟空闲状态。CCxP=0 对应高电平空闲状态（参考下图），CCxP=1 对应低电平空闲状态（参考下图）。
- 1101: x1 模式，计数器在单个时钟边沿更新，根据 CC1P 和 CC2P 位的值而定。CCxP=0 对应下降沿敏感性和高电平空闲状态（参考下图），CCxP=1 对应上升沿敏感性和低电平空闲状态（参考下图）。

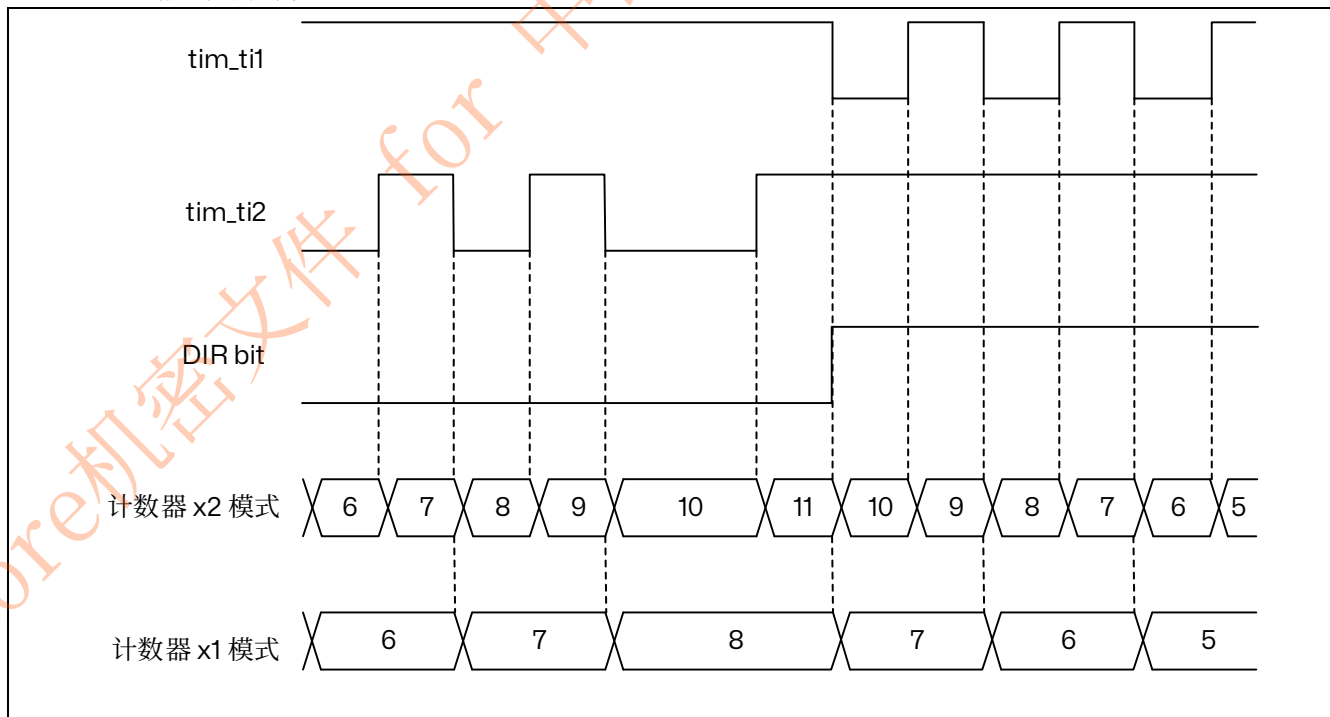


图 23.65 方向时钟编码器模式 (CC1P=CC2P=0)

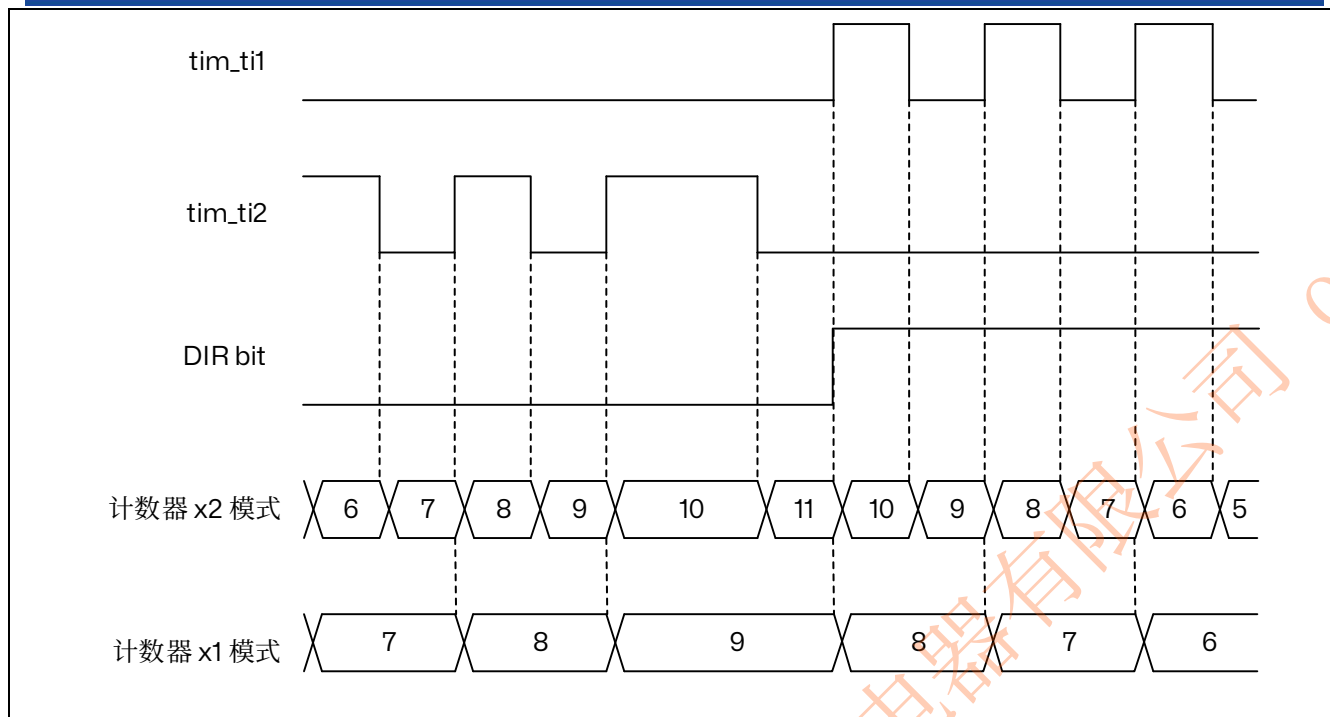


图 23.66 方向时钟编码器模式 (CC1P=CC2P=1)

下表详细说明了方向时钟模式对任何输入转变的操作方式。

表 23.18 计数方向与编码器信号及极性设置的关系

| 方向时钟模式          | SMS[3:0] | 相对信号的电平<br>(tim_ti1fp1 对应 tim_ti2,<br>tim_ti2fp2 对应 tim_ti1) | tim_ti1fp1 信号 |     | tim_ti2fp2 信号 |     |
|-----------------|----------|--------------------------------------------------------------|---------------|-----|---------------|-----|
|                 |          |                                                              | 上升            | 下降  | 上升            | 下降  |
| x2 模式<br>CCxP=0 | 1100     | 高                                                            | 递减            | 递减  | 递增            | 递增  |
|                 |          | 低                                                            | 不计数           | 不计数 | 不计数           | 不计数 |
| x2 模式<br>CCxP=1 | 1100     | 高                                                            | 不计数           | 不计数 | 不计数           | 不计数 |
|                 |          | 低                                                            | 递减            | 递减  | 递增            | 递增  |
| x1 模式<br>CCxP=0 | 1101     | 高                                                            | 不计数           | 递减  | 不计数           | 递增  |
|                 |          | 低                                                            | 不计数           | 不计数 | 不计数           | 不计数 |
| x1 模式<br>CCxP=1 | 1101     | 高                                                            | 不计数           | 不计数 | 不计数           | 不计数 |
|                 |          | 低                                                            | 递减            | 不计数 | 递增            | 不计数 |

## 索引输入

计数器可以通过来自编码器的索引信号进行复位，以指示绝对参考位置。索引信号必须连接到 tim\_etr\_in 输入。它可以使用数字输入滤波器进行滤波。

通过设置 TIMX\_ECR 寄存器中的 IE 位，可以使能索引功能。只有在编码器模式下，当 SMS[3:0] 位域具有以下值时，才可以设置 IE 位：0001、0010、011、1010、1011、1100、1101、1110、1111。

根据下图所示，有几种可设置的编码器选项，用于索引脉冲的调节：

- 与 A 和 B 门控：脉冲宽度为 1/4 的通道周期，与 A 和 B 边缘对齐
- 与 A（或与 B）门控：脉冲宽度为 1/2 的通道周期，与通道 A（或通道 B）的两个边缘对齐
- 非门控：脉冲宽度可达一个通道周期，不与任何边缘对齐

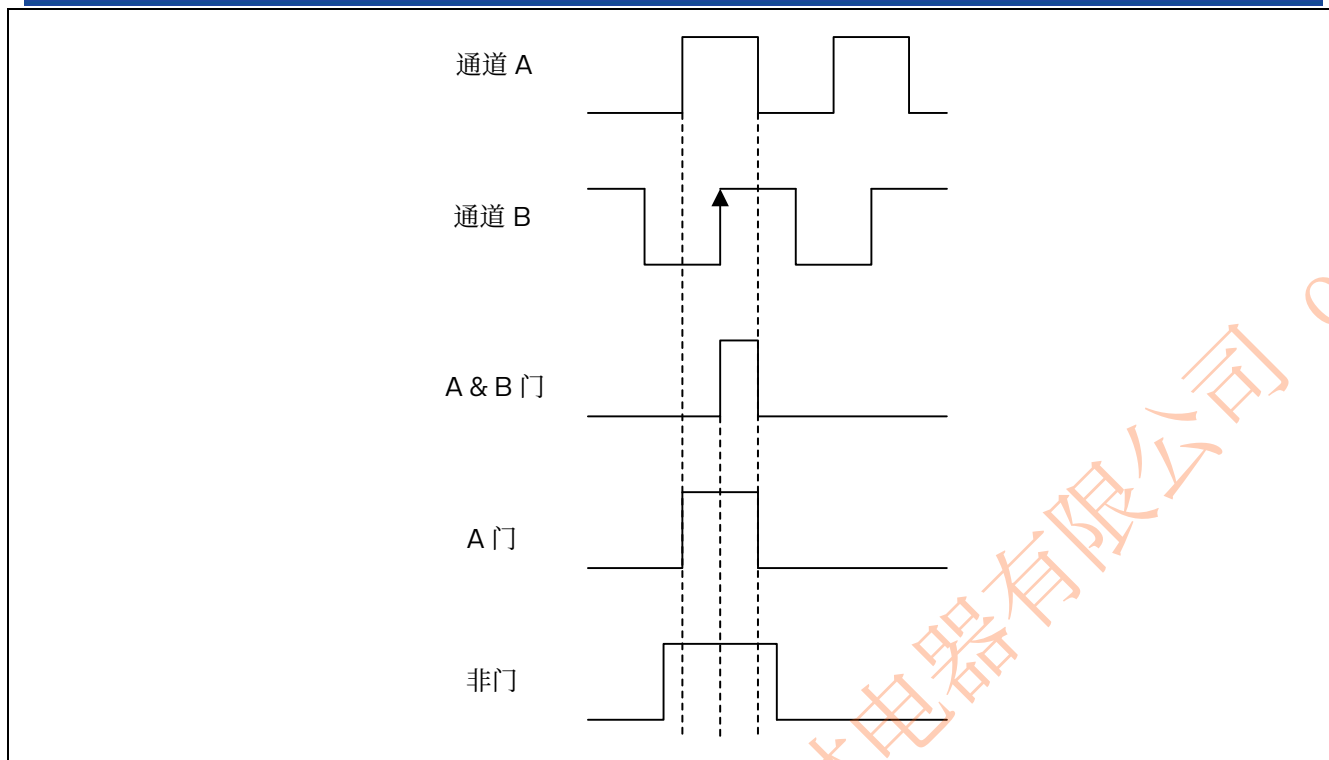


图 23.67 索引门控选项

电路可以容忍索引信号的抖动，无论门控模式如何，如下图所示。

在非门控模式下，信号必须严格低于 2 个编码器周期。如果脉冲宽度大于或等于 2 个编码器周期，则计数器会多次复位。

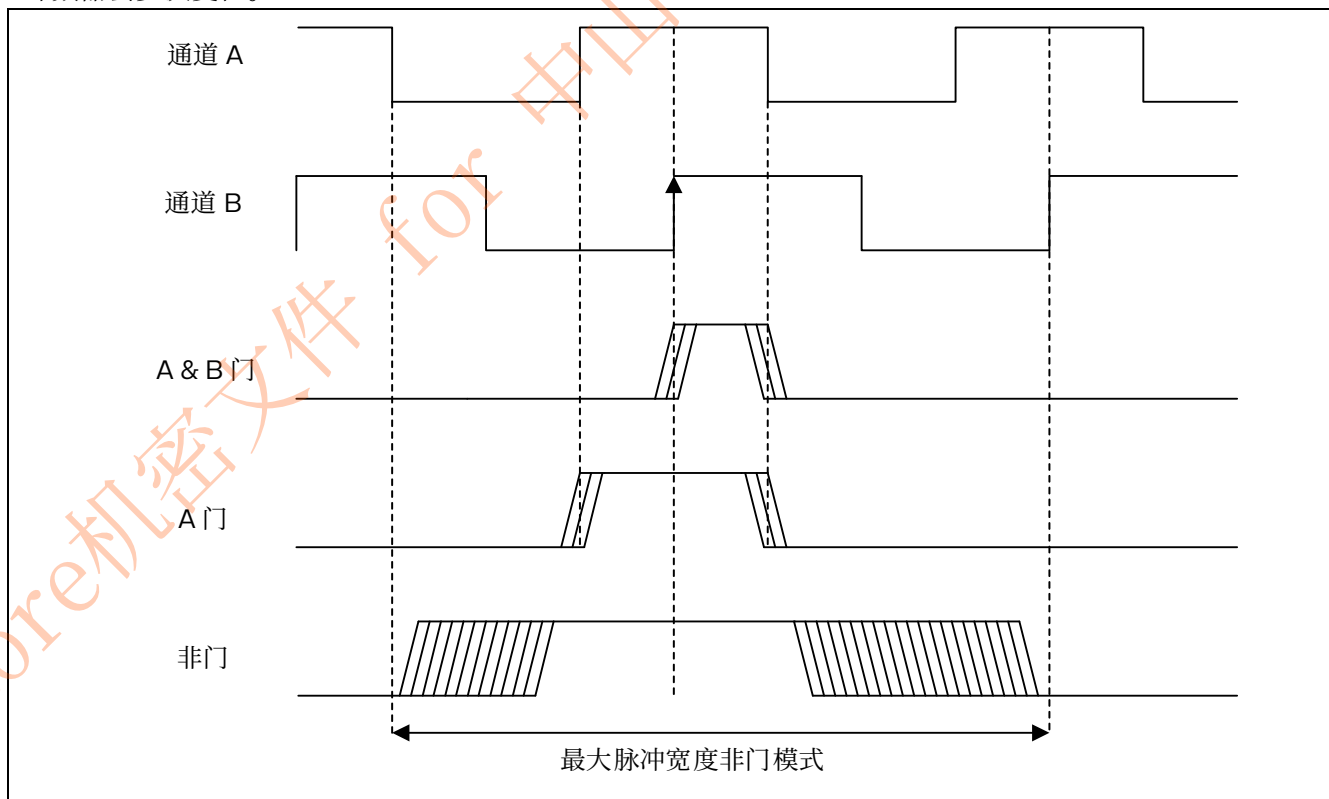


图 23.68 抖动的索引信号



定时器支持 3 种门控选项，不需要任何特定的编程。只需要定义在哪个编码器状态（即通道 A 和通道 B 状态组合）上，索引必须同步，使用 TIMx\_ECR 寄存器中的 IPOS[1:0] 位域。

索引检测事件根据计数方向的不同而有所不同，以确保速度反转时的对称操作：

- 递增计数时（DIR 位=0），计数器被复位。
- 递减计数时，计数器被设置为 TIMx\_ARR。

这使得索引可以在相同的机械角度位置上生成，无论计数方向如何。下图显示了一个简单的例子中（一个编码器每次机械旋转提供 4 个边缘），索引在哪个位置生成。

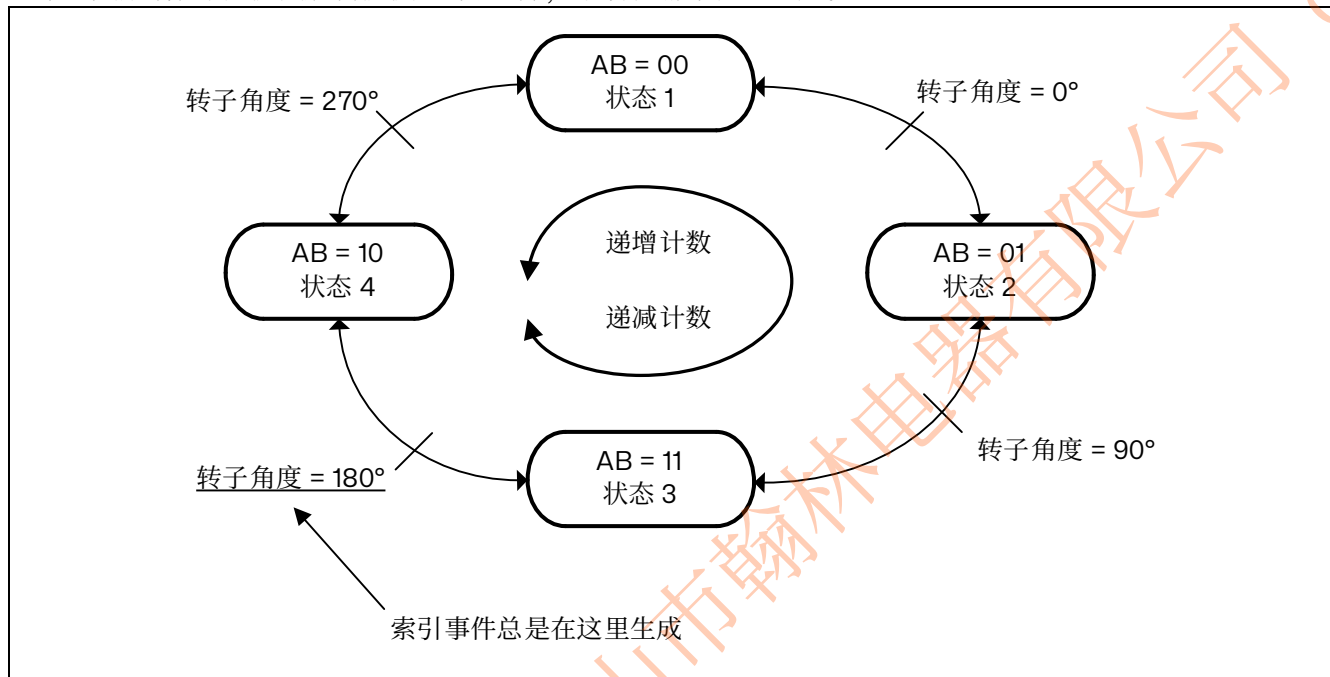


图 23.69 IPOS[1:0] = 11 时的索引生成

下图展示了 IPOS[1:0] = 11 时的波形和相应值。它表明，计数器值被强制的时刻会自动根据计数方向进行调整：

- 递增计数时（DIR 位=0），当编码器状态为“11”（ChA=1, ChB=1）时，计数器设为 0。
- 递减计数时（DIR 位=1），当退出“11”状态时，计数器设为 TIMx\_ARR。

可以在索引检测事件发生时发出中断。

箭头指示在哪个转换上生成索引事件中断。

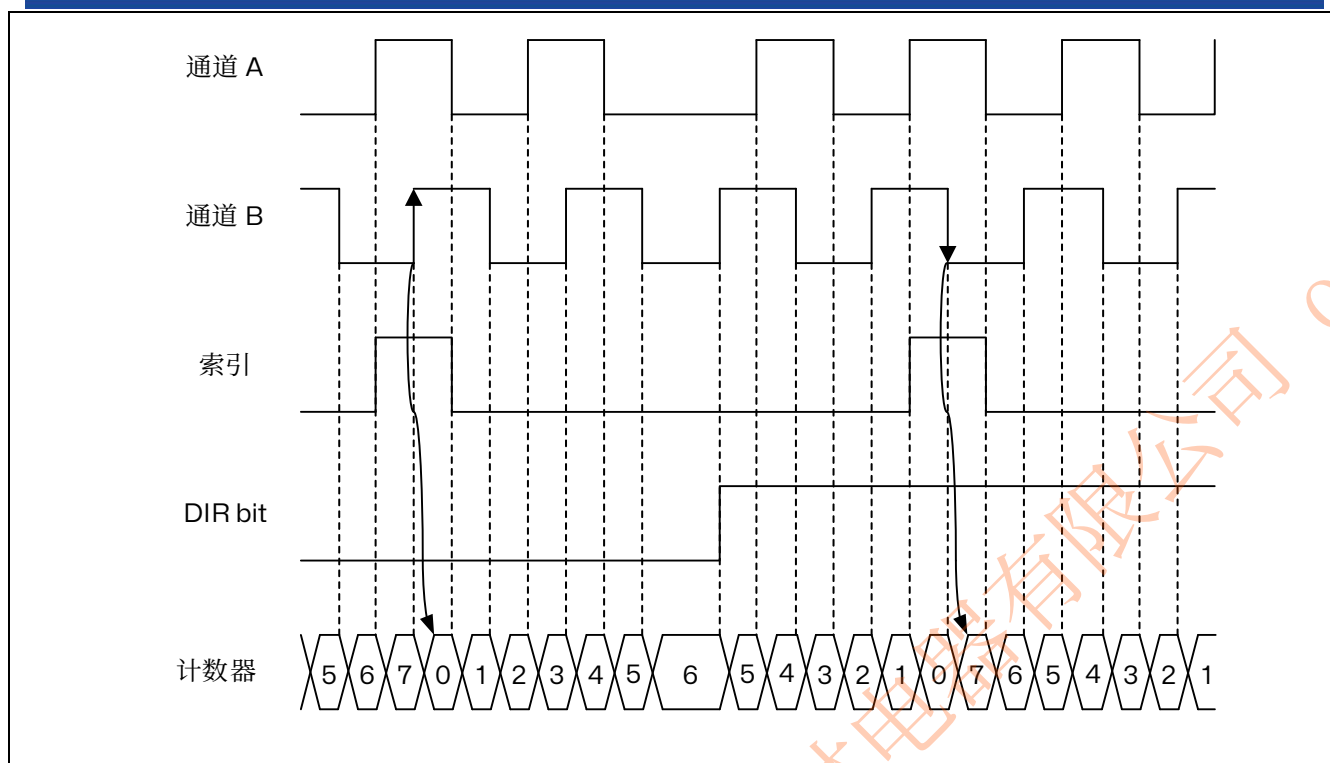


图 23.70 通道 A 上带索引门控的计数器读取 (IPOS[1:0] = 11)

下图展示了非门控模式的波形和相应值。箭头指示在哪个转换上生成索引事件。

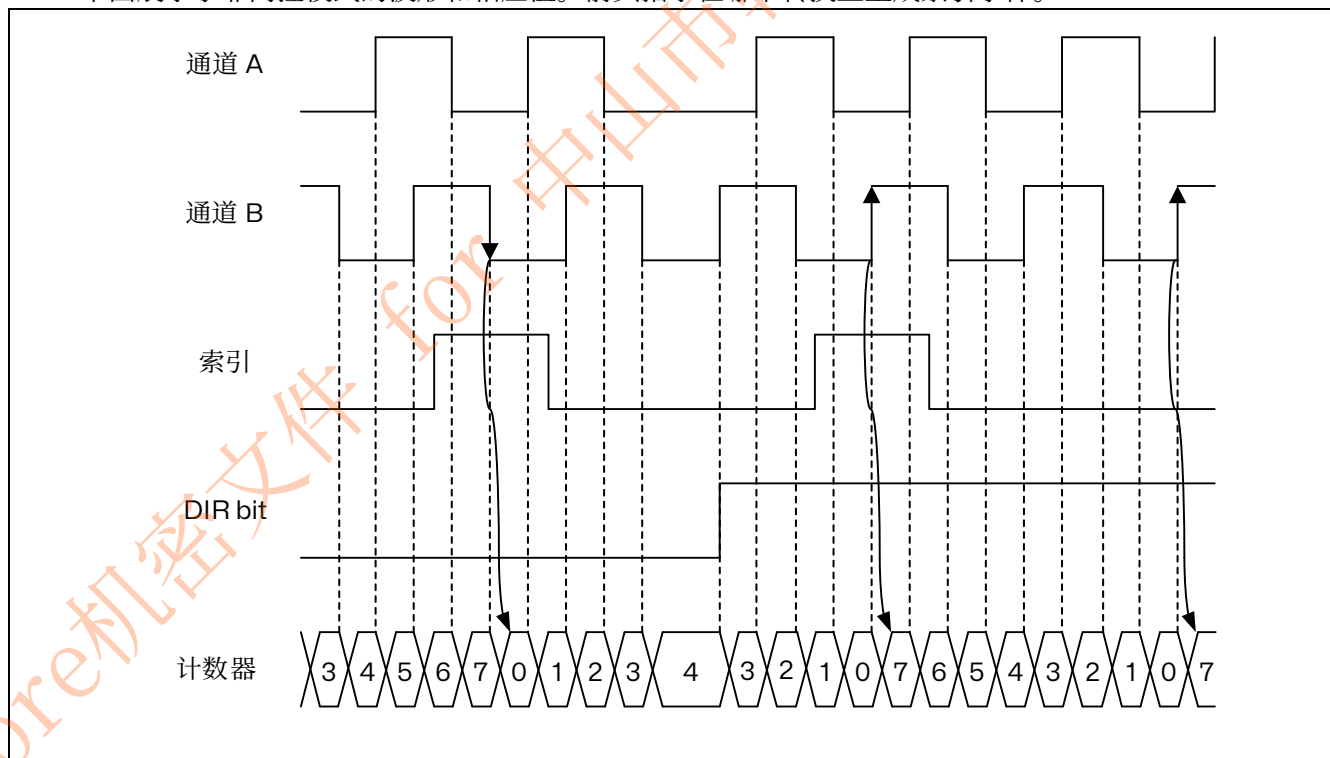


图 23.71 非门控模式下索引的计数器读取 (IPOS[1:0] = 00)

下图展示了针对各种脉冲对齐情况下的“在 A 和 B 门控”模式处理方式。箭头指示在哪个转换上生成索引事件。

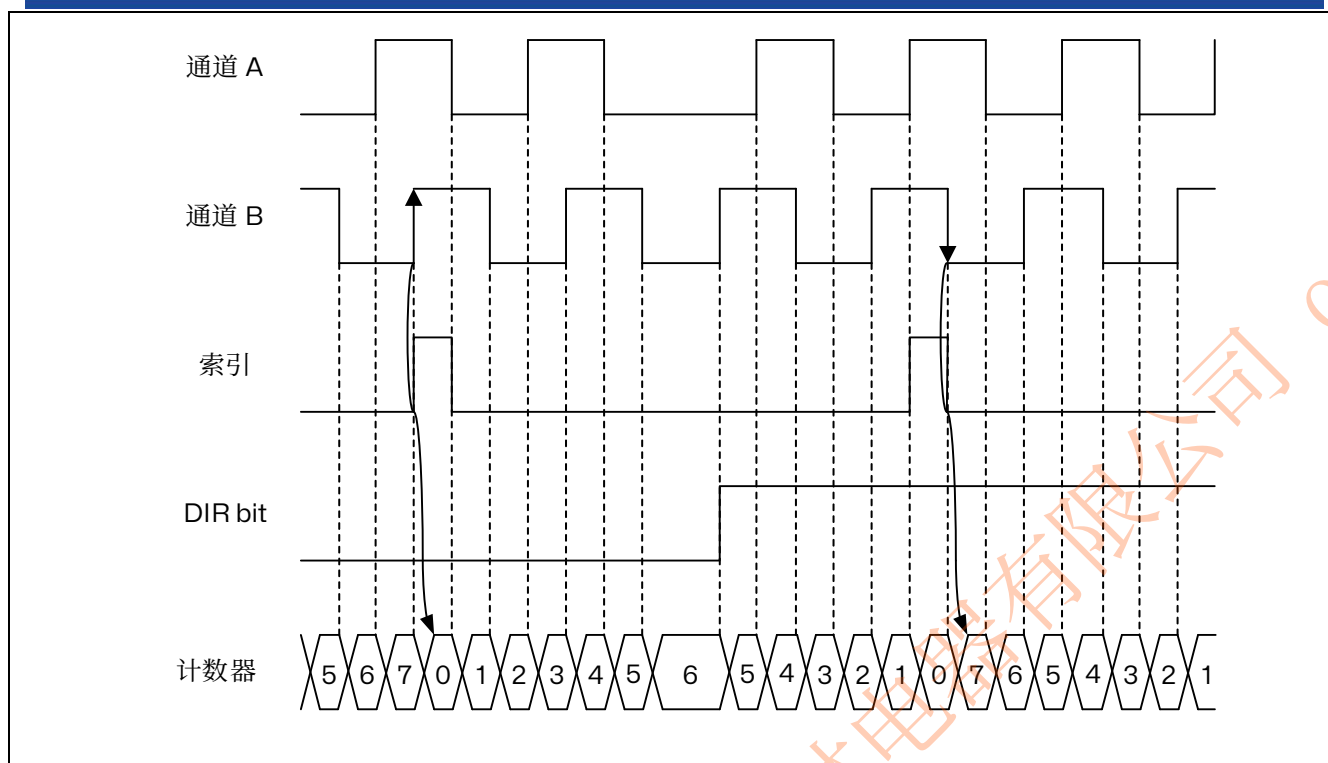


图 23.72 通道 A 和 B 上带索引门控的计数器读取

下两张图详细说明了后续索引脉冲可能比编码器时钟周期的四分之一窄的情况。

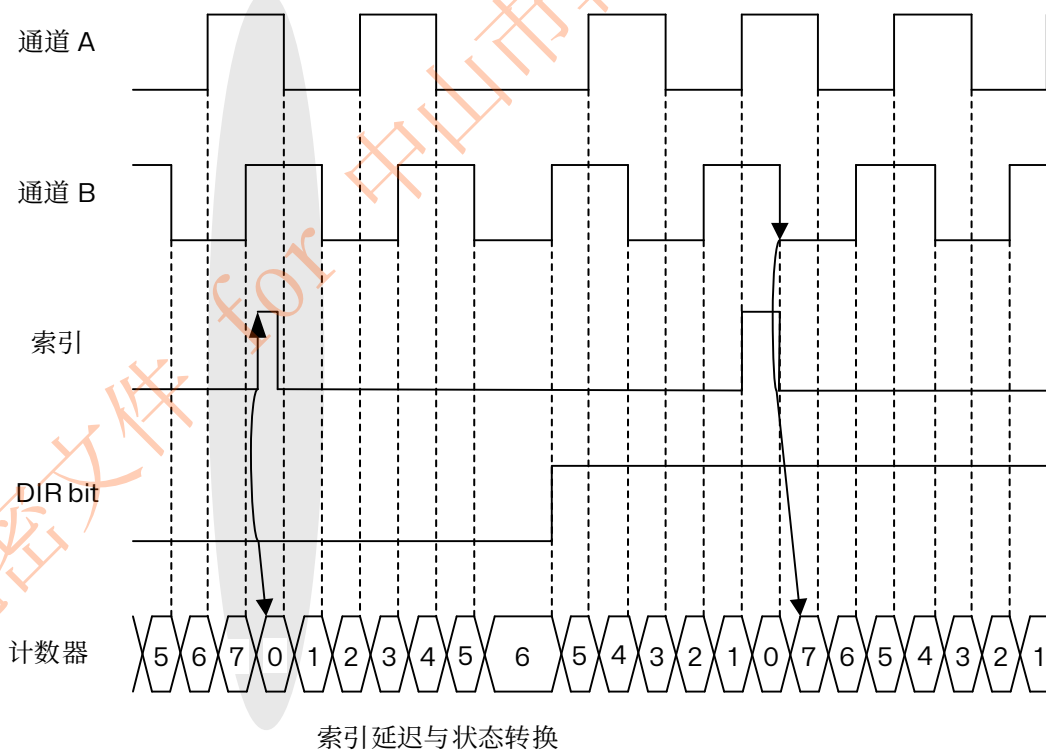
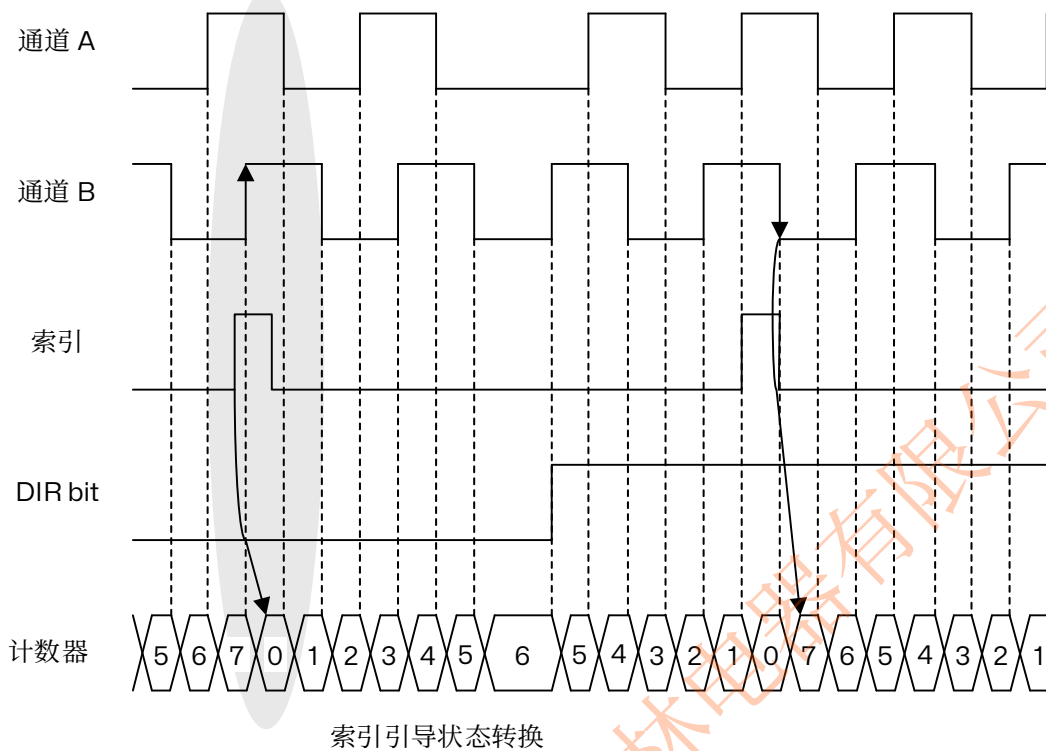


图 23.73 窄索引脉冲情况下的编码器模式行为 (IPOS[1:0] = 11)

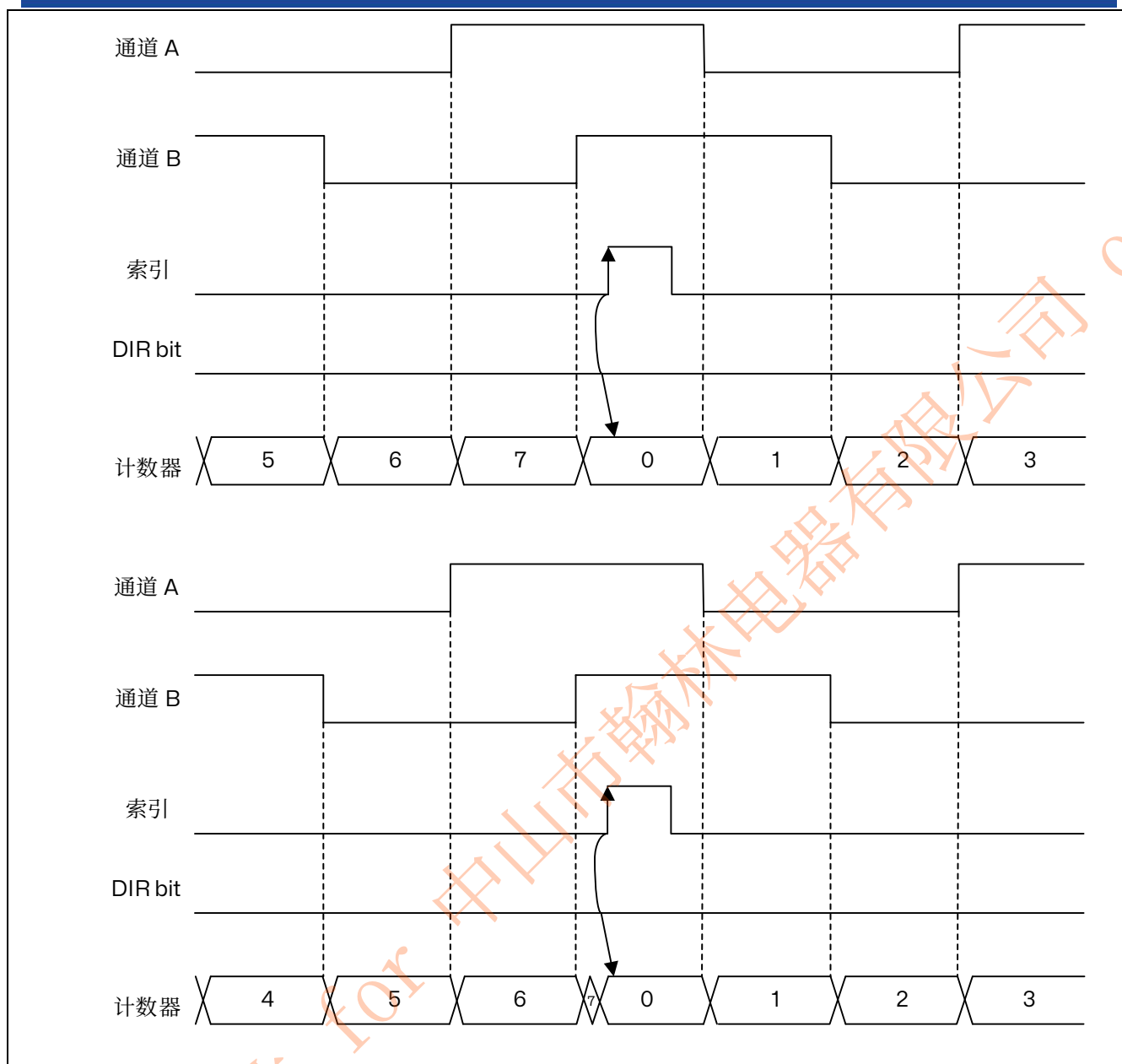


图 23.74 计数器重置窄索引脉冲（更近距离，ARR = 0x07）

下图说明了 x1 和 x2 模式下的索引管理方式。

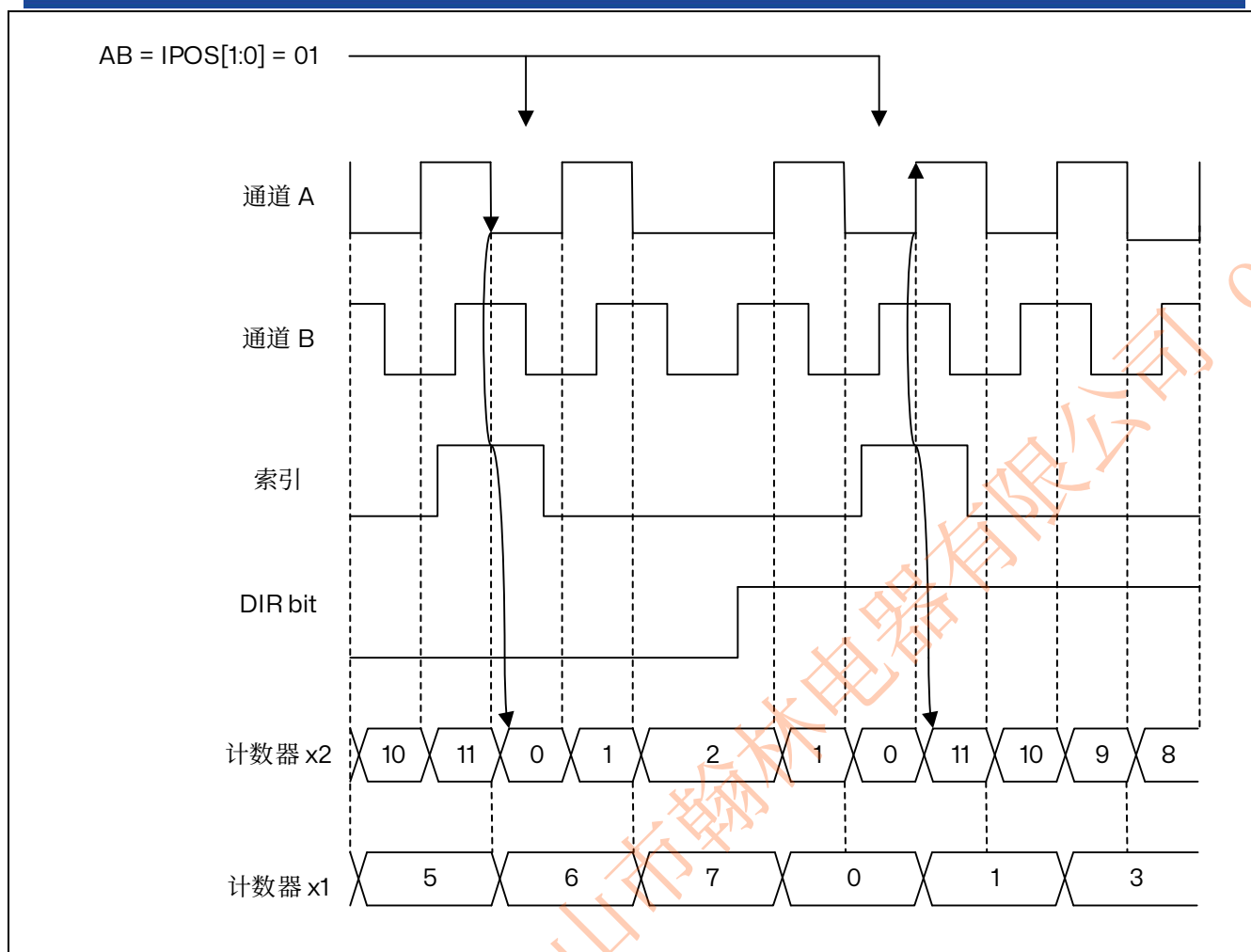


图 23.75 x1 和 x2 模式下的索引行为 (IPOS[1:0] = 01)

#### 方向索引灵敏度

TIMx\_ECR 寄存器中的 IDIR[1:0] 位域允许索引仅在选定计数方向上有效。

下图显示了根据 IDIR[1:0] 值索引和计数器重置事件之间的关系。

注：当 IE 位被复位时（禁用索引模式），必须写入 IDR[1:0] 位域。

注：在时钟+ 方向模式下不支持定向索引灵敏度。当 SMS[3:0] = 1010 或 1011 时，IDIR[1:0] 必须设置为 00。

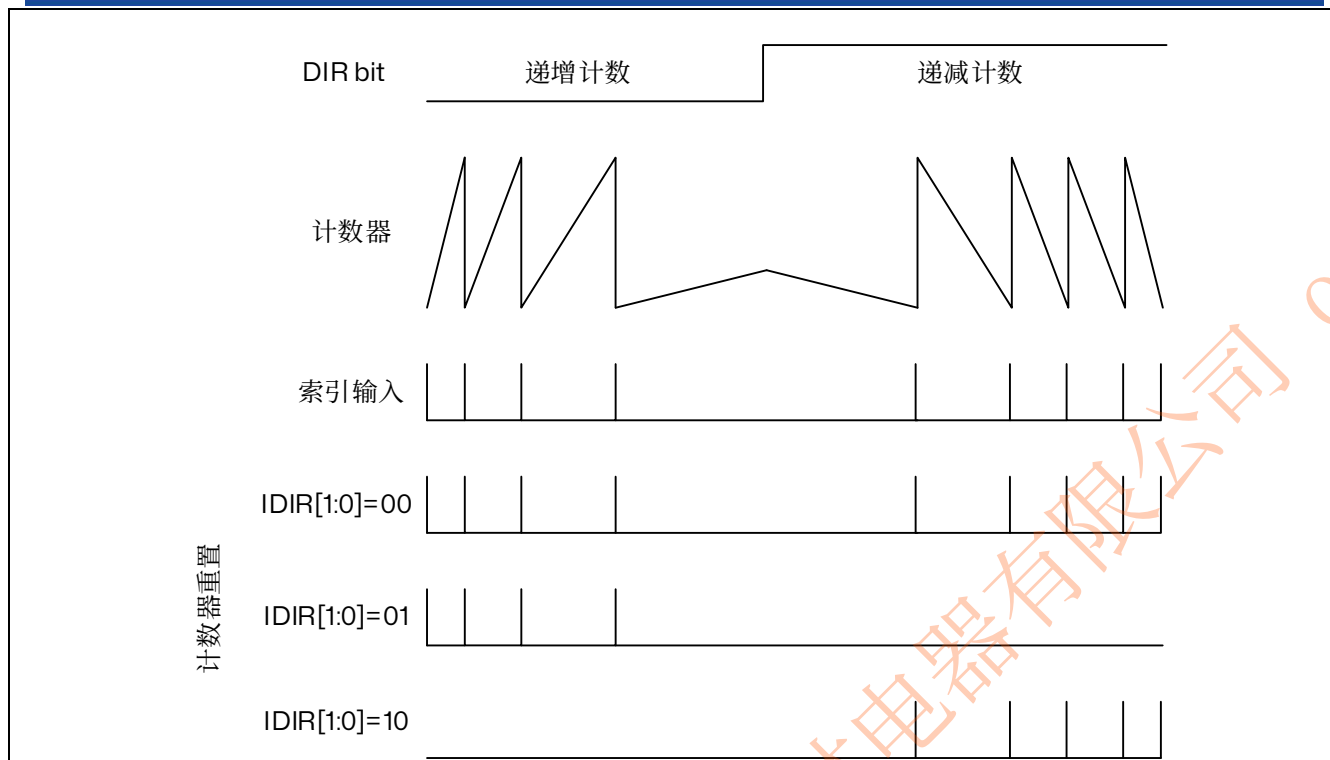


图 23.76 定向索引灵敏度

### 特殊的第一索引事件管理

TIMx\_ECR 寄存器中的 FIDX 位允许仅捕获一次索引，如下图所示。一旦第一个索引到达，任何后续索引都会被忽略。如果需要，可以通过将 FIDX 位写入 0 并再次将其设置为 1 来重新启动电路。

注：当 FIDX = 1 时，如果方向在位置 0 处改变（索引激活），索引可以发布两次（IDXFI 标志设置）。

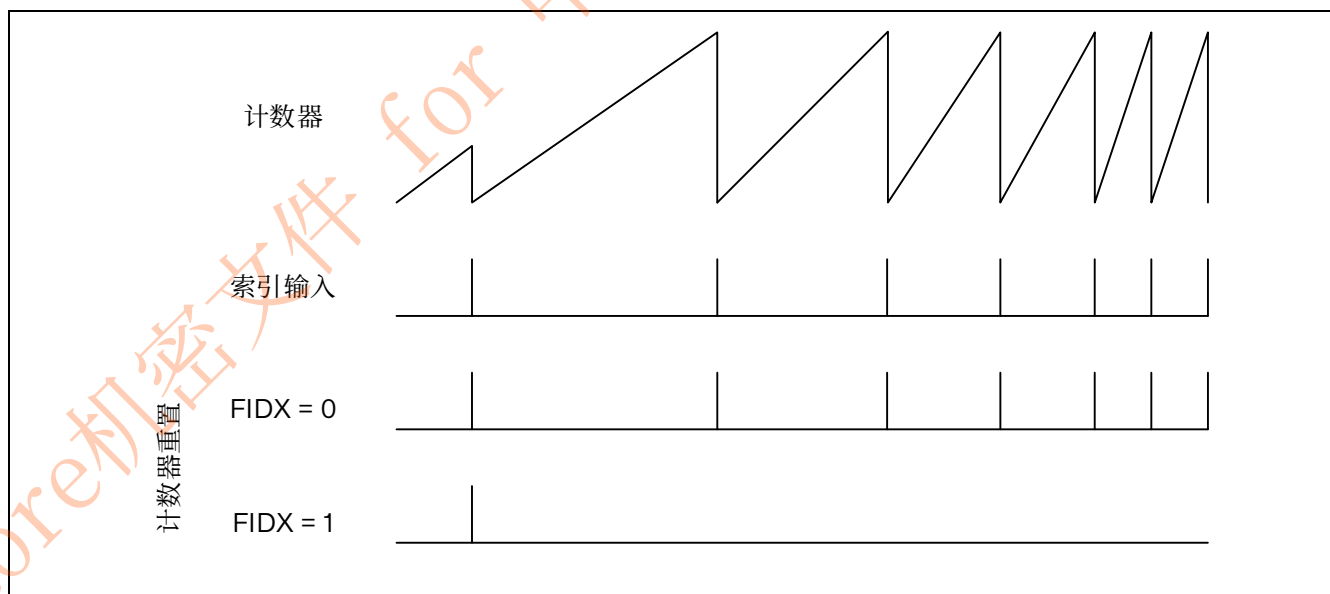


图 23.77 作为 FIDX 位设置的功能的计数器重置

### 非正交模式下的索引管理



下面两张图详细说明了当 SMS[3:0]位域等于 1010、1011、1100、1101 时，如何在方向时钟模式和时钟加方向模式下管理索引。

对于这两种模式，索引灵敏度通过 IPOS[0]位进行设置，具体如下：

- IPOS[0] = 0: 在时钟低电平上检测索引
- IPOS[0] = 1: 在时钟高电平上检测索引

IPOS[1]位不重要。

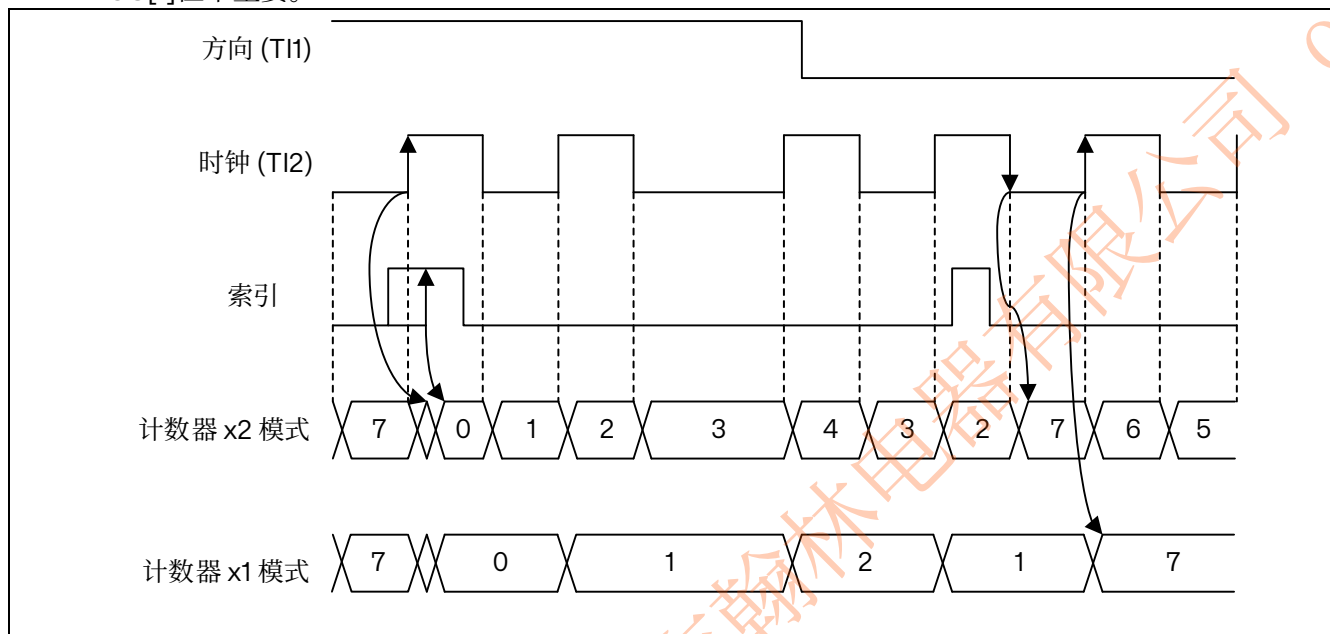


图 23.78 时钟+方向模式下的索引行为, IPOS[0] = 1

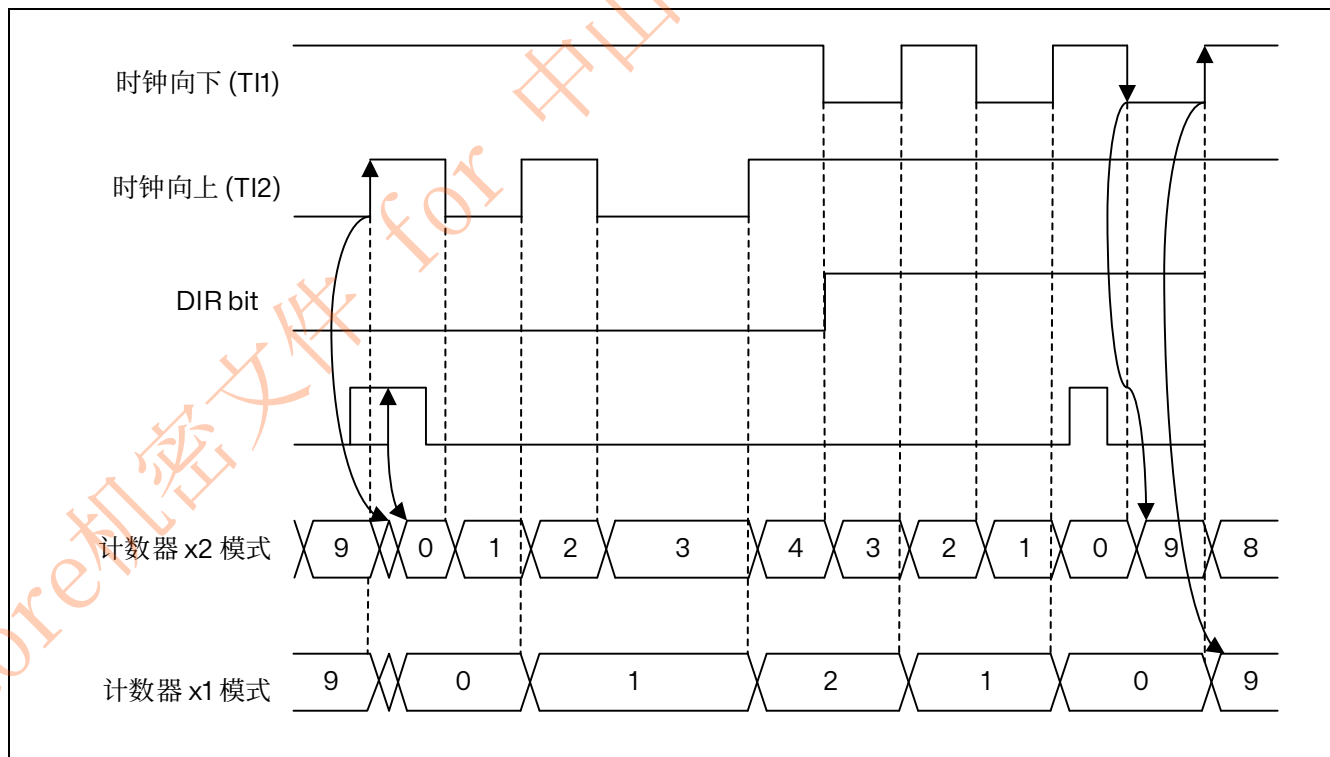


图 23.79 方向时钟模式下的索引行为, IPOS[0] = 1

## 编码器错误管理

对于具有 2 个正交信号的编码器配置，可以检测到转换错误。2 个输入的读数对应于一个 2 位灰度码，该灰度码可以表示为状态图，如以下图所示。期望单个比特立即更改。错误的转换会在 TIMx\_SR 状态寄存器中设置 TERRF 中断标志。如果 TIMx\_DIER 寄存器中的 TERRIE 位被设置，将产生转换错误中断。

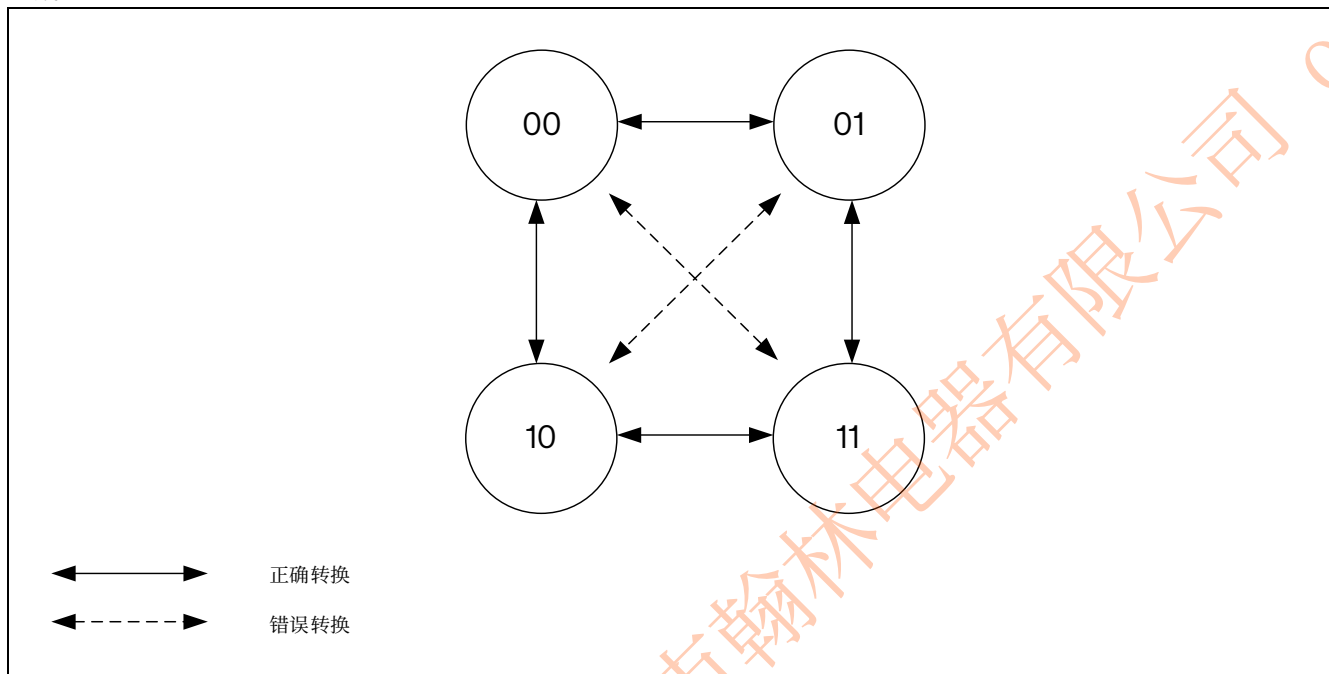


图 23.80 正交编码信号的状态图

对于具有索引信号的编码器，可以检测到导致每转脉冲过多的异常操作。一个编码器每转提供 N 个脉冲，则每转有 4xN 个计数。索引信号每 4xN 个时钟周期重置一次计数器。

如果计数器的值从 TIMx\_ARR 增加到 0 或从 0 减少到 TIMx\_ARR 的值而没有索引事件，则报告为索引位置错误。

可使用 TIMx\_ARR 寄存器来编程溢出阈值。一个 1000 线的编码器导致计数器的值在 0 和 3999 之间（在 4 倍读取模式下）。必须通过设置  $TIMx\_ARR = 3999 + 1 = 4000$  来编程溢出检测阈值。

错误断言被延迟到在递增计数时从 0 到 1 的转换。这是为了处理门控 A 和 B 模式中的窄索引脉冲，如下图所示。

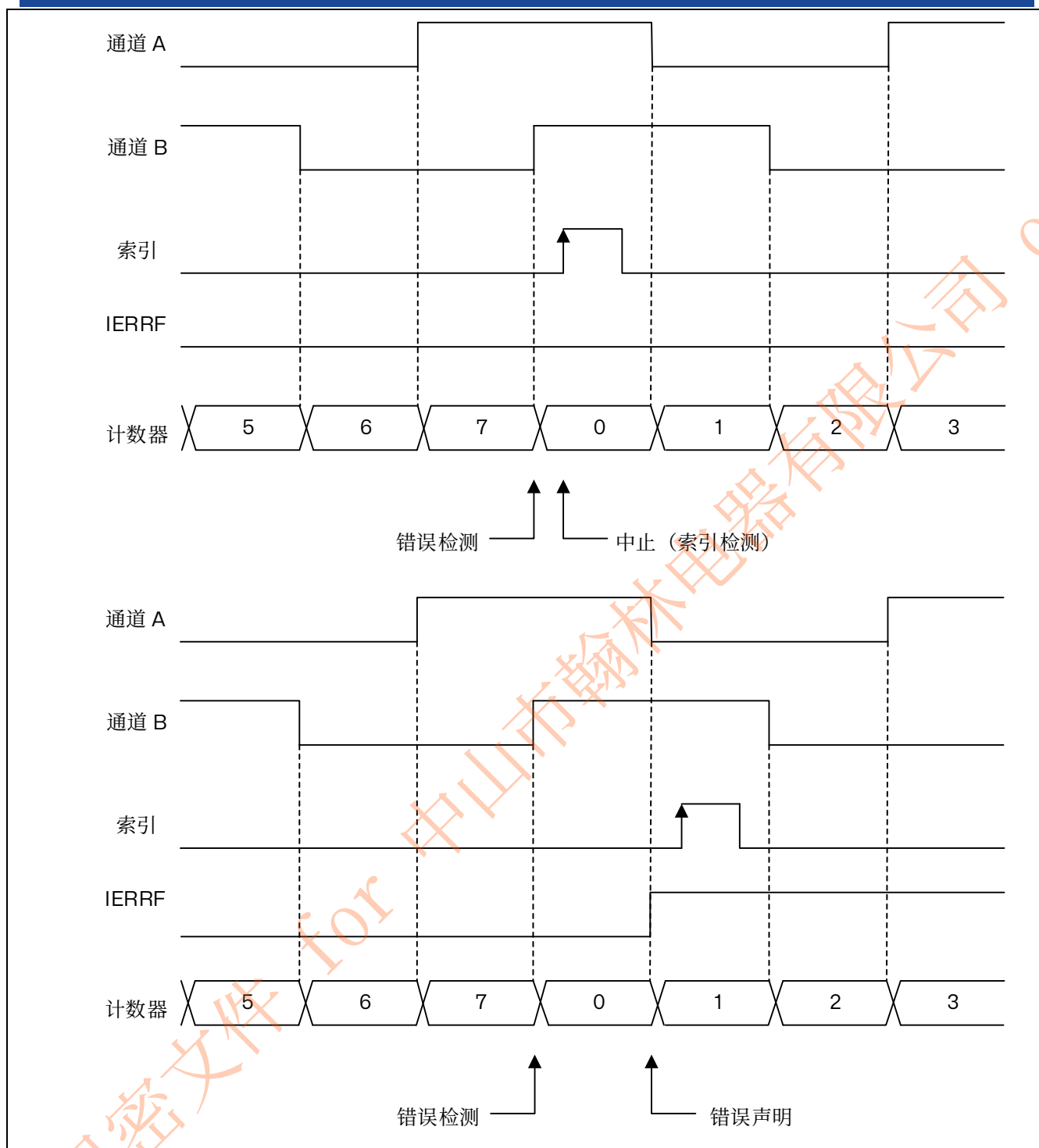


图 23.81 递增计数编码器错误检测

在递减计数模式下，检测是由从 1 到 0 的初步转换引起的。这是为了处理门控 A 和 B 模式中的窄索引脉冲，如下图所示，以避免编码器在索引检测后立即在 TIMx\_ARR 和 0 之间抖动时出现任何错误检测。

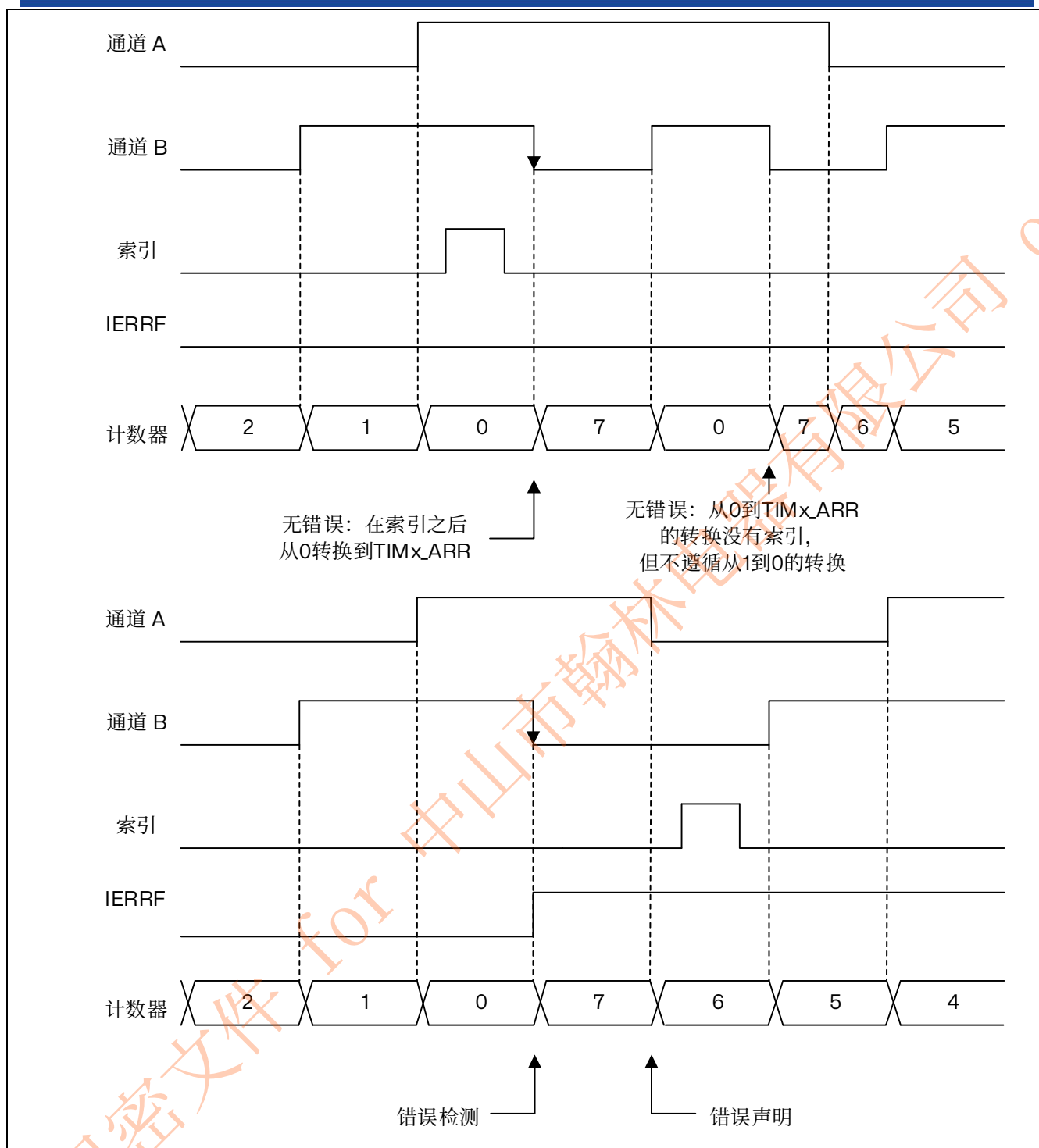


图 23.82 递减计数编码器错误检测

索引错误会在 TIMx\_SR 状态寄存器中设置 IERRF 中断标志。如果 TIMx\_DIER 寄存器中的 IERRIE 位被设置, 将产生索引错误中断。

### 功能编码器中断

编码器模式下也可使用以下中断:

- 方向改变: 在编码器模式下, 任何计数方向的改变都会导致 TIMx\_CR1 寄存器中的 DIR 位发生翻转。方向改变会设置 TIMx\_SR 状态寄存器中的 DIRF 中断标志。如果 DIRIE 位在 TIMx\_DIER 寄

寄存器中被设置，则产生方向改变中断。

- 索引事件：索引事件会设置 TIMx\_SR 状态寄存器中的 IDXF 中断标志。如果 IDXIE 位在 TIMx\_DIER 寄存器中被设置，则产生索引中断。

### 运行时更新编码器模式时从机模式选择预加载

在运行过程中，可能需要从一个编码器模式切换到另一个编码器模式。这通常在高速度下完成，以降低更新中断频率，从 x4 模式切换到 x2 模式再到 x1 模式，如下图所示。

为此，可以预加载 SMS[3:0]位。通过设置 TIMx\_SMCR 寄存器中的 SMSPE 使能位可以使能此功能。从 SMS[3:0]预加载值转换到有效值可以由 TIMx\_SMCR 寄存器中的 SMSPS 位选择触发。

- SMSPS = 0：当计数器递增时溢出并在计数器递减时下溢时，由更新事件(UEV)触发的转换。只有在索引被禁用时（位 IE = 0）才可以使用此模式。
- SMSPS = 1：由索引事件触发的转换。

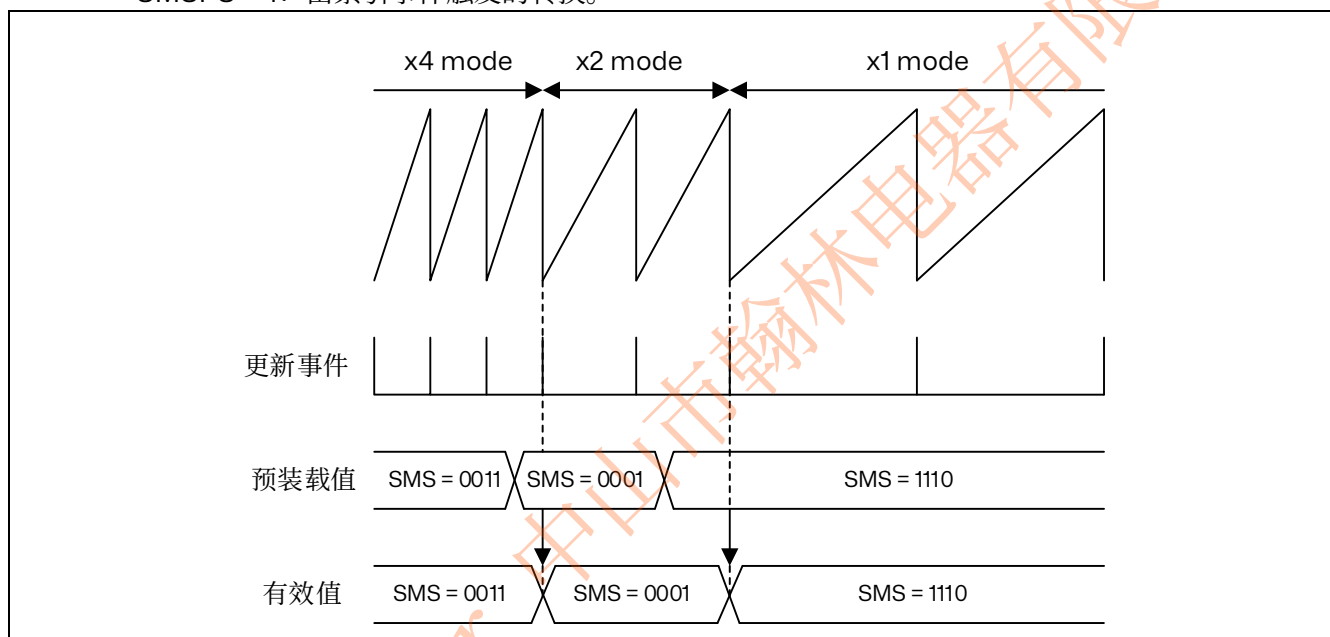


图 23.83 更新时带有预加载转移的编码器模式更改 (SMSPS = 0)

### 编码器时钟输出

编码器模式的操作原理并不完全适合于高分辨率速度测量，在低速时，因为它需要相对较长的积分时间才能获得足够数量的时钟边缘和精确的测量。

在低速时，更好的解决方案是进行边到边的时钟周期测量。这可以通过使用从属定时器来实现。定时器可以在 tim\_trgo 输出上输出编码器时钟信息。然后，从属定时器可以进行周期测量并为每个编码器时钟边缘提供速度信息。

通过将 TIMx\_CR2 寄存器中的 MMS[3:0]位域设置为 1000 可以使能此模式。对于以下 SMS[3:0]值有效：0001、0010、0011、1010、1011、1100、1101、1110、1111。不允许任何其他 SMS[3:0]代码，并可能导致意外行为。

#### 23.3.26 方向位输出

可以将方向信号从定时器输出到 tim\_oc3n 和 tim\_oc4 输出信号 (TIMx\_CR1 寄存器中的 DIR 位的副本)。这是通过将 TIMx\_CCMR2 寄存器中的 OC3M[3:0]或 OC4M[3:0]位域设置为 1011 来实现的。

此功能可用于在编码器模式下监视计数方向（或旋转方向），或在中心对齐的 PWM 模式下具有指示

上/下相位的信号。

### 23.3.27 UIF 位重映射

TIMx\_CR1 寄存器中的 UIFREMAP 位强制将更新中断标志 UIF 连续复制到定时器计数器寄存器的第 31 位 (TIMxCNT[31])。这允许以原子方式读取计数器的值和一个潜在的由 UIFCPY 标志指示的翻转条件。在特定情况下，它可以避免由背景任务（读取计数器）和中断（更新中断）之间的处理引起的竞争条件，从而简化计算。

UIF 和 UIFCPY 标志之间没有延迟。

### 23.3.28 定时器输入异或功能

TIMx\_CR2 寄存器中的 TI1S 位，可将通道 1 的输入滤波器连接到异或门的输出，从而将 tim\_ti1, tim\_ti2, tim\_ti3 这三个输入引脚组合在一起。

异或输出能够被用于所有定时器的输入功能，如触发或输入捕获。这对于测量两个输入信号之间的边缘间隔非常方便，如下图所示。

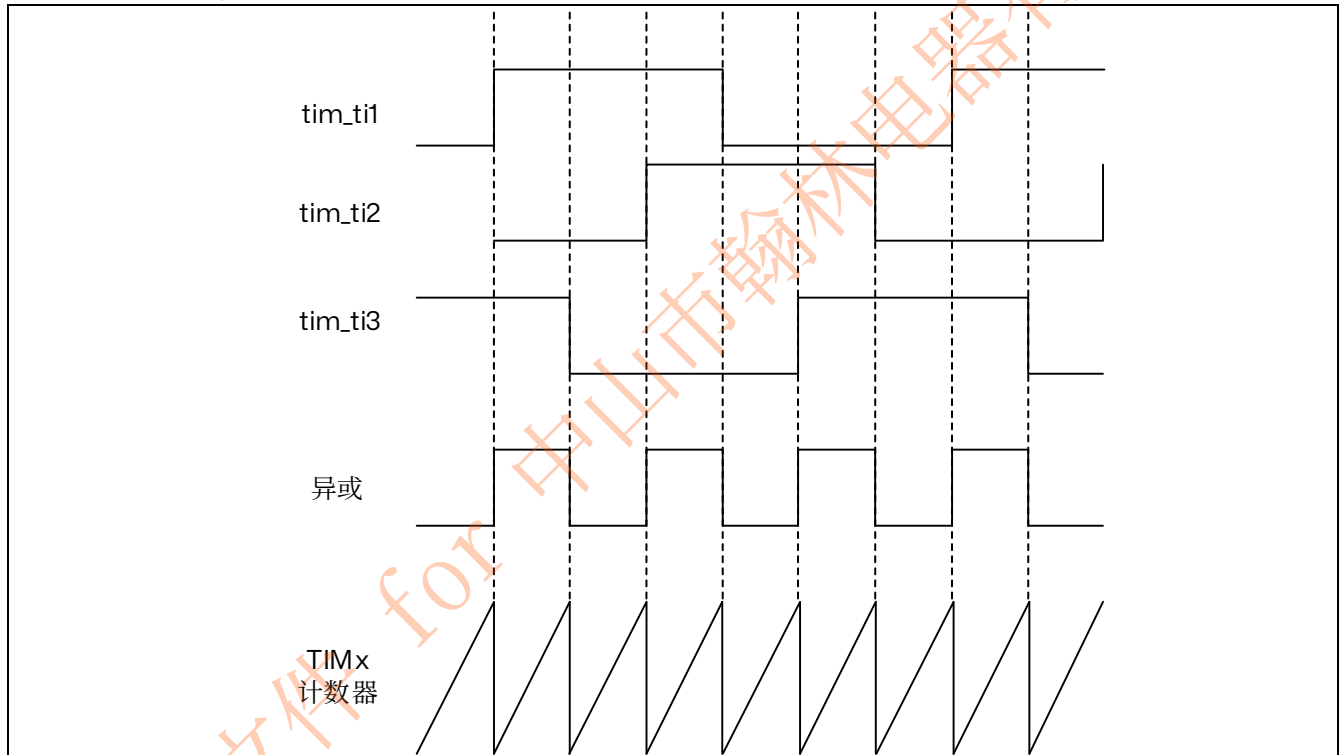


图 23.84 测量 3 个信号边缘之间的时间间隔

### 23.3.29 与霍尔传感器接口

这是通过使用高级控制定时器生成 PWM 信号来驱动电机，并使用另一个被称为“接口定时器”的 TIMx 定时器（如下图）来实现的。“接口定时器”捕获通过 XOR 连接到 tim\_ti1 输入通道（通过设置 TIMx\_CR2 寄存器中的 TI1S 位来选择）的 3 个定时器输入引脚（tim\_ti1、tim\_ti2 和 tim\_ti3）。

从属模式控制器配置为复位模式；从属输入是 tim\_tif\_ed。因此，每当 3 个输入中的任何一个切换时，计数器都会从 0 开始重新计数。这创建了一个由霍尔输入上的任何变化触发的时间基准。

在“接口定时器”上，捕获/比较通道 1 配置为捕获模式，捕获信号是 tim\_trc。捕获的值对应于输入上两次变化之间的时间间隔，提供了有关电机速度的信息。

“接口定时器”可以用于输出模式，以生成一个脉冲，该脉冲会改变高级控制定时器的通道配置（通过触发 COM 事件）。高级控制定时器用于生成驱动电机的 PWM 信号。为此，必须编程接口定时器的通道，以便在编程延迟后生成正脉冲（在输出比较或 PWM 模式下）。该脉冲通过 `tim_trgo` 输出发送到高级控制定时器。

例如：每当连接到其中一个 TIMx 定时器的霍尔输入发生变化时，都希望在编程延迟后更改高级控制定时器的 PWM 配置。

- 通过将 TIMx\_CR2 寄存器中的 TI1S 位设置为“1”，使 3 个定时器输入经过异或运算后进入 `tim_ti1` 输入通道，
- 时基编程：将 TIMx\_ARR 写入最大值（计数器必须通过 `tim_ti1` 的变化清零。设置预分频器，以得到最大计数器周期，该周期长于传感器上两次变化的间隔时间
- 将通道 1 配置为捕获模式（选择 `tim_trc`）：将 TIMx\_CCMR1 寄存器中的 CC1S 位写入“01”。如果需要，还可以编程数字滤波器，
- 设置通道 2 为 PWM2 模式，并具有所需延时：将 TIMx\_CCMR1 寄存器中的 OC2M 位写入“111”，将 CC2S 位写入“00”，
- 选择 `tim_oc2ref` 作为 `tim_trgo` 上的触发输出：将 TIMx\_CR2 寄存器中的 MMS 位写入“101”，在高级控制定时器中，必须选择正确的 `tim_itrx` 输入作为触发输入，定时器被编程为生成 PWM 信号，捕获/比较控制信号被预加载（在 TIMx\_CR2 寄存器中设置 `CCPC=1`），并且 COM 事件由触发输入控制（在 TIMx\_CR2 寄存器中设置 `CCUS=1`）。PWM 控制位（`CCxE`、`OCxM`）在下一步的 COM 事件后写入（这可以在由 `tim_oc2ref` 的上升边沿生成的中断子程序中完成）。

下图描述了这个例子。



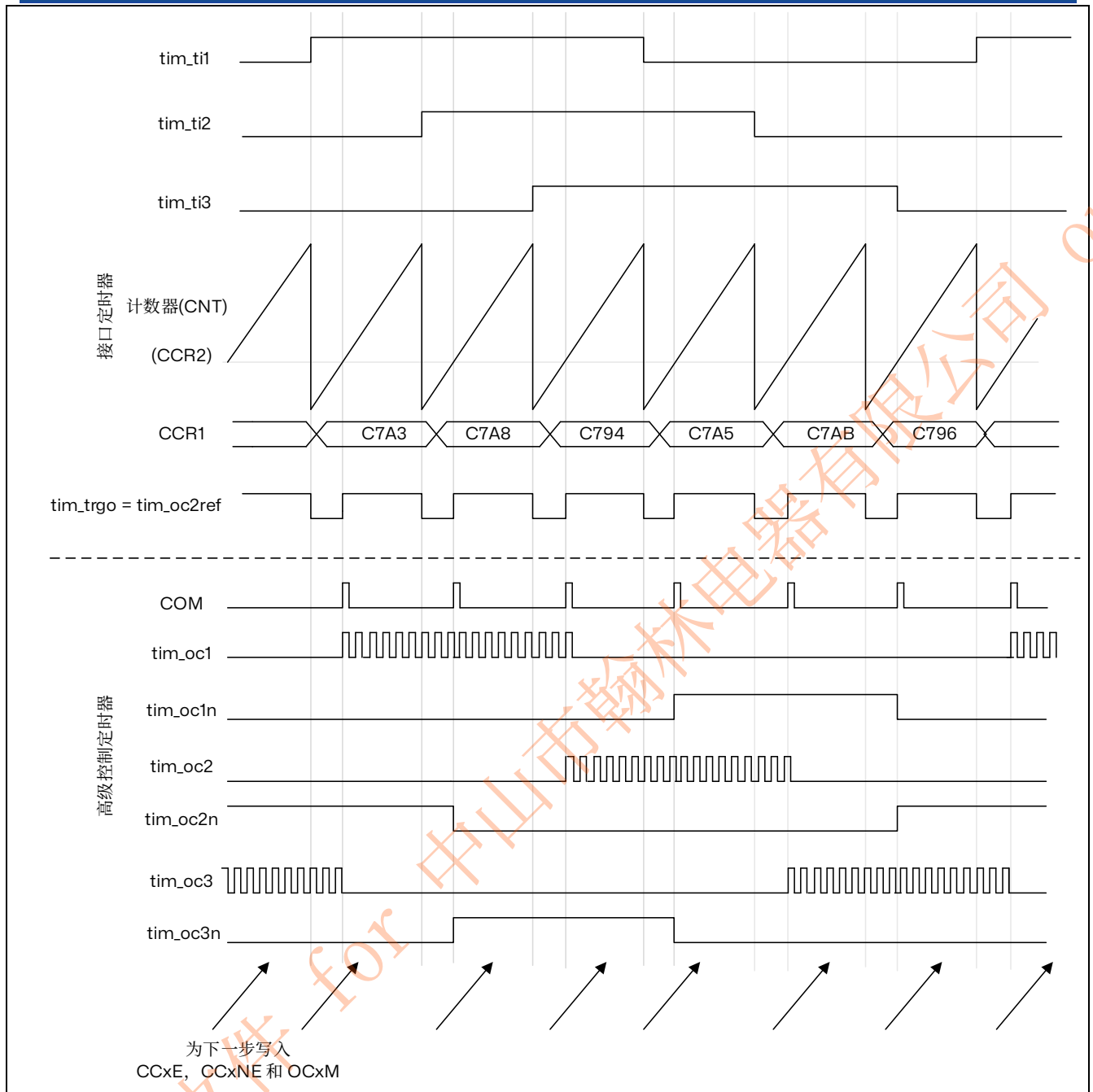


图 23.85 霍尔传感器接口的实例

### 23.3.30 定时器同步

TIMx 计时器在内部连接在一起，用于计时器同步或链接。它们可以在几种模式下同步：复位模式，门控模式，触发模式，复位+触发和门控+复位模式。

#### 从模式：复位模式

计数器及其预分量器可以在响应触发器输入上的事件时重新初始化。此外，如果来自 TIMx\_CR1 寄存器的 URS 位较低，则生成一个更新事件 UEV。然后更新所有预加载的寄存器 (TIMx\_ARR, TIMx\_CCRx)。

以 tim\_ti1 输入上升沿清除上行计数器为例：

- 配置通道 1 检测 tim\_til 上升沿。配置输入过滤器持续时间（在本例中，我们不需要任何过滤器，因此保持 IC1F=0000）。捕获预分频器不用于触发，因此不需要配置。CC1S 位只选择输入捕获源，在 TIMx\_CCMR1 寄存器中 CC1S = 01。在 TIMx\_CCER 寄存器中写入 CC1P=0 和 CC1NP=“0”以验证极性（并仅检测上升沿）。
- 在 TIMx\_SMCR 寄存器中写入 SMS=100，将定时器设置为复位模式。通过在 TIMx\_SMCR 寄存器中写入 TS=00101，选择 tim\_til 作为输入源。
- 通过在 TIMx\_CR1 寄存器中写入 CEN=1 来启动计数器。

计数器开始在内部时钟上计数，然后正常工作，直到 tim\_til 上升沿。当 tim\_til 上升时，计数器被清除并从 0 重新开始。同时，触发标志被设置（TIMx\_SR 寄存器中的 TIF 位），中断请求或 DMA 请求可以被发送（取决于 TIMx\_DIER 寄存器中的 TIE 和 TDE 位）。

下图显示了当自动重新加载寄存器 TIMx\_ARR=0x36 时的行为。

tim\_til 上的上升沿和计数器的实际复位之间的延迟是由于 tim\_til 输入上的重新同步电路造成的。

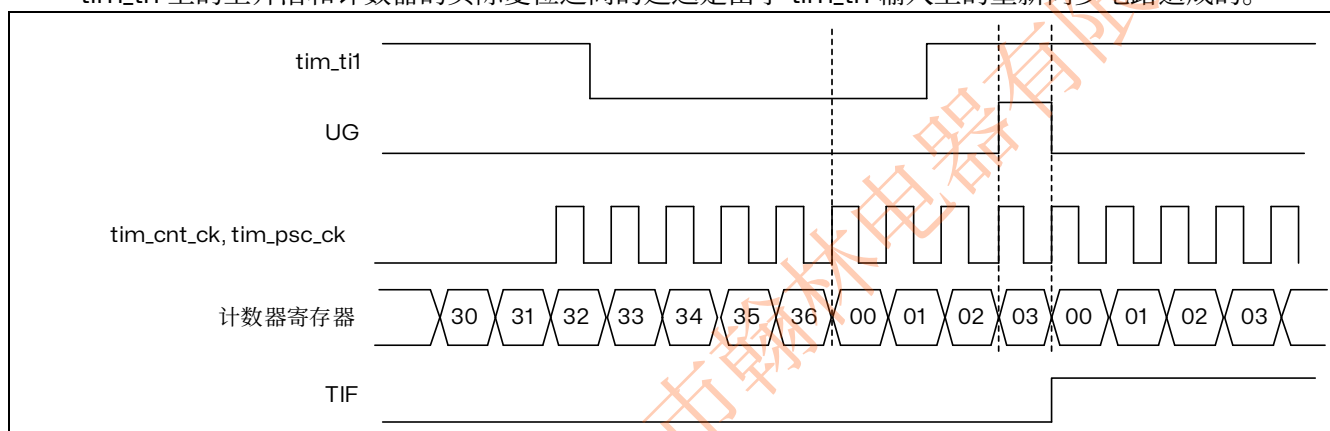


图 23.86 复位模式下的控制电路

### 从模式：门控模式

计数器的开启取决于所选输入的电平。

下面以 tim\_til 输入为低电平为例进行说明：

- 配置通道 1 检测 tim\_til 的低电平。配置输入过滤器持续时间（在本例中，我们不需要任何过滤器，因此保持 IC1F=0000）。捕获预分频器不用于触发，因此不需要配置。CC1S 位只选择输入捕获源，在 TIMx\_CCMR1 寄存器中 CC1S=01。在 TIMx\_CCER 寄存器中写入 CC1P=1 和 CC1NP=“0”以验证极性（并仅检测低电平）。
- 将定时器配置为门控模式，在 TIMx\_SMCR 寄存器中写入 SMS=101。通过在 TIMx\_SMCR 寄存器中写入 TS=00101，选择 tim\_til 作为输入源。
- 通过在 TIMx\_CR1 寄存器中写入 CEN=1 来使能计数器（在门控模式下，如果 CEN=0，无论触发输入电平是多少，计数器都不会启动）。

只要 tim\_til 为低，计数器就开始在内部时钟上计数，一旦 tim\_til 变为高，计数器就停止计数。

TIMx\_SR 寄存器中的 TIF 标志是在计数器启动或停止时设置的。

tim\_til 上的上升沿和计数器的实际停止之间的延迟是由于 tim\_til 输入上的再同步电路。

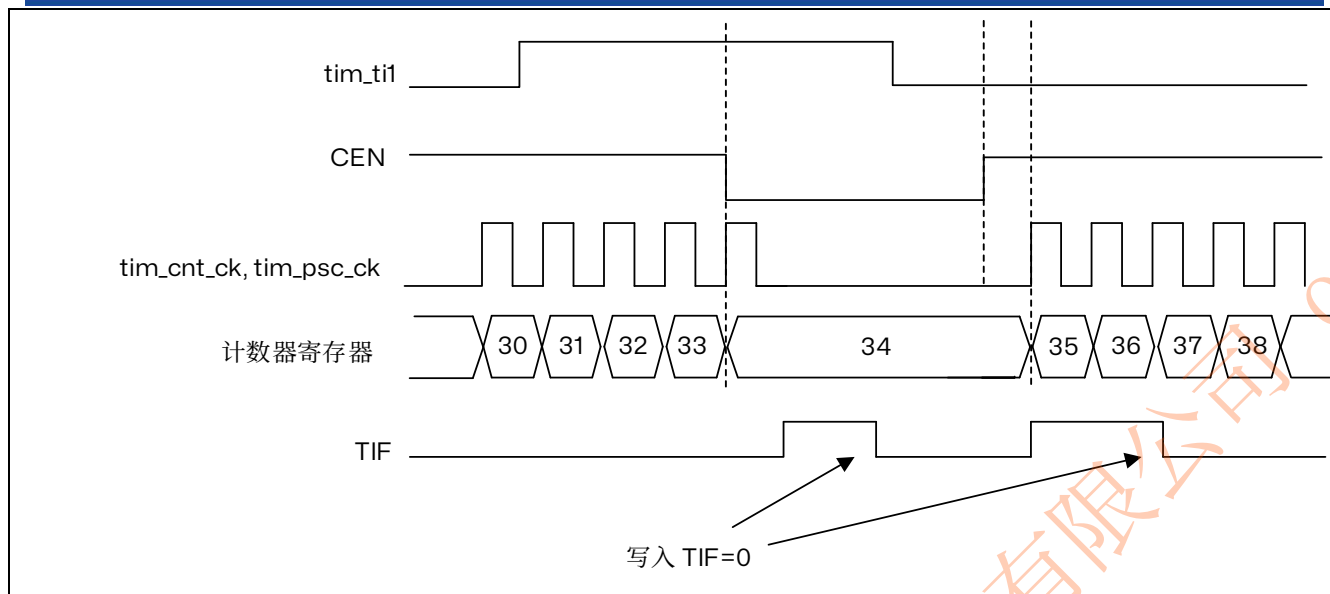


图 23.87 门控模式下的控制电路

### 从模式：触发模式

计数器可以响应选定输入的事件而启动。

在下面的示例中，上行计数器响应 tim\_ti2 输入的上升沿启动：

- 配置通道 2 检测 tim\_ti2 的上升沿。配置输入过滤器持续时间（在本例中，我们不需要任何过滤器，因此保持 IC2F=0000）。捕获预分频器不用于触发，因此不需要配置。CC2S 位配置为只选择输入捕获源，在 TIMx\_CCMR1 寄存器中 CC2S=01。在 TIMx\_CCER 寄存器中写入 CC2P=1 和 CC2NP=0 以验证极性（并且仅检测低电平）。
- 在 TIMx\_SMCR 寄存器中写入 SMS=110，将定时器配置为触发模式。通过在 TIMx\_SMCR 寄存器中写入 TS=00110，选择 tim\_ti2 作为输入源。

当 tim\_ti2 出现上升沿时，计数器开始在内部时钟上计数，并设置 TIF 标志。

tim\_ti2 上的上升沿和计数器实际开始之间的延迟是由于 tim\_ti2 输入上的再同步电路造成的。

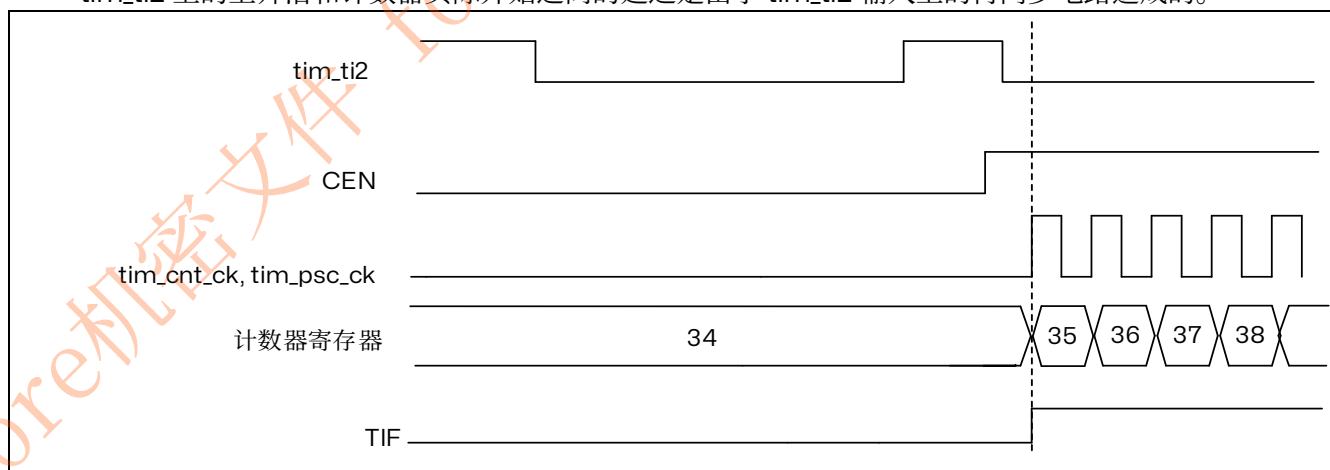


图 23.88 触发器模式下的控制电路

### 从模式：复位+触发模式

在这种情况下，选择的触发输入(tim\_trgi)的上升沿重新初始化计数器，生成寄存器的更新，并启动计数器。

该模式用于单脉冲模式。

#### 从模式：门控+复位模式

触发输入(tim\_trgi)高时使能计数器。一旦触发变低，计数器停止并复位。计数器的开始和停止都是受控的。

该模式允许检测超出范围的 PWM 信号(占空比超过最大期望值)。

#### 从模式：外部时钟模式 2 + 触发模式

外部时钟模式 2 可以在另一个从模式之外使用（除了外部时钟模式 1 和编码器模式）。在这种情况下，tim\_etr\_in 信号用作外部时钟输入，可以选择另一个输入作为触发输入(复位模式、门控模式或触发模式)。建议不要通过 TIMx\_SMCR 寄存器的 TS 位选择 tim\_etr\_in 作为 tim\_trgi。

在下面的例子中，一旦出现 tim\_ti1 的上升沿，上计数器在 tim\_etr\_in 信号的每个上升沿上递增：

1. 通过编程 TIMx\_SMCR 寄存器配置外部触发输入电路，如下所示：

- ETF = 0000: 无滤波
- ETPS=00: 去使能预分频器
- ETP=0: 检测 tim\_etr\_in 上升沿和 ECE=1 使能外部时钟模式 2。

2. 通道 1 配置如下，检测 TI 上升沿：

- IC1F=0000: 无滤波器。
- capture precaler 不用于触发，不需要配置。
- CC1S=01 在 TIMx\_CCMR1 寄存器中只选择输入捕获源
- CC1P=0 和 CC1NP= '0' 在 TIMx\_CCER 寄存器中验证极性(只检测上升沿)。

3. 在 TIMx\_SMCR 寄存器中写入 SMS=110，将定时器配置为触发模式。通过在 TIMx\_SMCR 寄存器中写入 TS=00101，选择 tim\_ti1 作为输入源。

tim\_ti1 上的上升沿使能计数器并设置 TIF 标志。然后计数器计数 tim\_etr\_in 上升边。

tim\_etr\_in 信号的上升沿和计数器的实际复位之间的延迟是由于 tim\_etrp 输入上的再同步电路造成的。

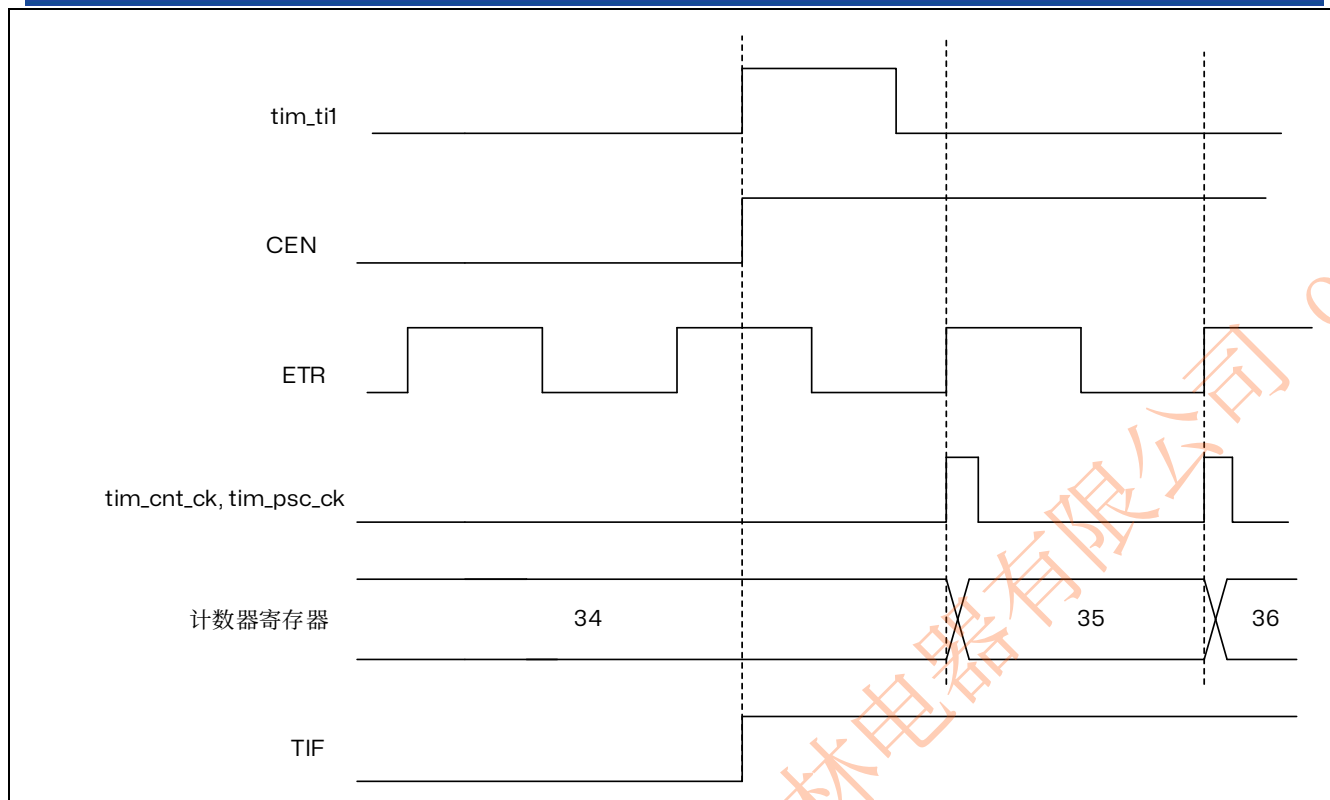


图 23.89 外部时钟模式 2+触发模式下的控制电路

注：接收tim\_trgo 或tim\_trgo2 信号的从外设（定时器，ADC，...）的时钟必须在从主定时器接收事件之前使能，并且时钟频率（预分频器）在从主定时器接收触发器时不能实时更改。

### 23.3.31 ADC 触发器

定时器可以用各种内部信号（例如重置、使能或比较事件）生成 ADC 触发事件。也可以生成由内部边缘检测器发出的脉冲。例如：

- OC4ref 的上升沿和下降沿
- OC5ref 的上升沿或 OC6ref 的下降沿

触发器在 tim\_trgo2 内部线路发出，该线路被重定向到 ADC。共有 16 个可能的事件，可以使用 TIMx\_CR2 寄存器中的 MMS2[3:0]位选择。前章节：三相组合 PWM 模式中给出了一个 3 相马达驱动的应用示例。

注：接收tim\_trgo 或tim\_trgo2 信号的从属外设（定时器，ADC 等）的时钟必须在从主定时器接收事件之前使能，并且时钟频率（预分频器）在从主定时器接收触发器时不能实时更改。

注：ADC 的时钟必须在从主定时器接收事件之前使能，并且在接收到定时器的触发器时不能更改

### 23.3.32 DMA 突发模式

TIMx 定时器具有在单个事件上生成多个 DMA 请求的能力。其主要目的是能够在不增加软件开销的情况下多次重新编程定时器的一部分，但它也可以用于在固定间隔内连续读取多个寄存器。

DMA 控制器的目标地址是唯一的，必须指向虚拟寄存器 TIMx\_DMAR。在给定的定时器事件上，定时器启动一系列 DMA 请求（突发）。对 TIMx\_DMAR 寄存器的每次写入实际上都被重定向到定时器寄存器之一。

TIMx\_DCR 寄存器中的 DBL[4:0]位设置了 DMA 突发长度。当对 TIMx\_DMAR 地址进行读取或写入访问时，定时器会识别出突发传输，即传输的数量（以半字或字节为单位）。

TIMx\_DCR 寄存器中的 DBA[4:0]位定义了 DMA 传输的 DMA 基地址（当通过 TIMx\_DMAR 地址进行读取/写入访问时）。DBA 是从 TIMx\_CR1 寄存器的地址开始定义的偏移量：

示例：

00000: TIMx\_CR1

00001: TIMx\_CR2

00010: TIMx\_SMCR

作为一个例子，定时器 DMA 突发模式用于在更新事件发生时更新 CCRx 寄存器（x = 2, 3, 4）的内容，其中 DMA 将半字传输到 CCRx 寄存器中。

这是通过以下步骤完成的：

1. 配置相应的 DMA 通道如下：
  - 无滤波器 DMA 通道外设地址是 DMAR 寄存器的地址。
  - DMA 通道内存地址是 RAM 中缓冲区的地址，其中包含要通过 DMA 传输到 CCRx 寄存器的数据。
  - 要传输的数据数量 = 3（见下文注释）。
  - 循环模式禁用。
2. 通过配置 DBA 和 DBL 位域来配置 DCR 寄存器，如下所示：
 

DBL = 3 次传输，DBA = 0xE。
3. 使能 TIMx 更新 DMA 请求（在 DIER 寄存器中设置 UDE 位）。
4. 使能 TIMx
5. 使能 DMA 通道

此示例适用于每个 CCRx 寄存器都需要更新一次的情况。例如，如果要更新每个 CCRx 寄存器两次，则要传输的数据数量应为 6。让我们以一个在 RAM 中包含 data1, data2, data3, data4, data5 和 data6 的缓冲区为例。数据按照以下方式传输到 CCRx 寄存器：在第一个更新 DMA 请求时，data1 传输到 CCR2，data2 传输到 CCR3，data3 传输到 CCR4；在第二个更新 DMA 请求时，data4 传输到 CCR2，data5 传输到 CCR3，data6 传输到 CCR4。

注：可以将空值写入保留的寄存器。

### 23.3.33 TIM1/TIM8/ DMA 请求

TIM1/TIM8 以生成 DMA 请求，如下表所示。

表 23.19 DMA 请求

| DMA 请求信号    | DMA 缩写   | DMA 请求   | 使能控制位 |
|-------------|----------|----------|-------|
| tim_upd_dma | TIM_UP   | 更新       | UDE   |
| tim_cc1_dma | TIM_CH1  | 捕获/比较 1  | CC1DE |
| tim_cc2_dma | TIM_CH2  | 捕获/比较 2  | CC2DE |
| tim_cc3_dma | TIM_CH3  | 捕获/比较 3  | CC3DE |
| tim_cc4_dma | TIM_CH4  | 捕获/比较 4  | CC4DE |
| tim_com_dma | TIM_COM  | 换向 (COM) | COMDE |
| tim_trg_dma | TIM_TRIG | 触发       | TDE   |

### 23.3.34 调试模式

当微控制器进入调试模式（带有 FPU 的 Cortex®-M4 内核停止）时，TIMx 计数器的行为取决于 DBG 模块中的 DBG\_TIMx\_STOP 配置位，它可以继续正常工作或停止。

在调试模式下，每个定时器的行为可以通过 Debug 支持（DBG）模块的专用配置位进行编程。

出于安全考虑，当计数器停止时，输出被禁用（就像将 MOE 位复位一样）。输出可以强制处于非活动状态（OSSI 位=1），或者由 GPIO 控制器接管控制（OSSI 位=0）以强制其处于高阻态。

有关更多详细信息，请参阅调试部分。

### 23.4 TIM1/TIM8 低功耗模式

表 23.20 低功耗模式对 TIM1/TIM8 的影响

| 模式 | 描述                        |
|----|---------------------------|
| 睡眠 | 无影响，外设是运行的。中断会导致设备退出睡眠模式。 |
| 停机 | 定时器操作被停止并保留其寄存器内容。无法生成中断。 |
| 待机 | 定时器被断电并且退出待机模式后必须重新初始化。   |

### 23.5 TIM1/TIM8 中断

TIM1/TIM8 可以生成多个中断，如下表所示。

表 23.21 中断请求

| 中断缩写        | 中断事件     | 事件标志  | 使能控制位  | 中断清除方法    | 从睡眠模式退出 | 从停机和待机退出 |
|-------------|----------|-------|--------|-----------|---------|----------|
| TIM_UP      | 更新       | UIF   | UIE    | UIF 写 0   | 是       | 否        |
| TIM_CC      | 捕获/比较 1  | CC1IF | CC1IE  | CC1IF 写 0 | 是       | 否        |
|             | 捕获/比较 2  | CC2IF | CC2IE  | CC2IF 写 0 | 是       | 否        |
|             | 捕获/比较 3  | CC3IF | CC3IE  | CC3IF 写 0 | 是       | 否        |
|             | 捕获/比较 4  | CC4IF | CC4IE  | CC4IF 写 0 | 是       | 否        |
| TIM_TRG_COM | 换向 (COM) | COMIF | COMIE  | COMIF 写 0 | 是       | 否        |
| TIM_TRG     | 触发       | TIF   | TIE    | TIF 写 0   | 是       | 否        |
| TIM_DIR_IDX | 索引       | IDXF  | IDXIE  | IDXF 写 0  | 是       | 否        |
|             | 方向       | DIRF  | DIRIE  | DIRF 写 0  | 是       | 否        |
| TIM_BRK     | 刹车       | BIF   | BIE    | BIF 写 0   | 是       | 否        |
|             | 刹车 2     | B2IF  |        | B2IF 写 0  | 是       | 否        |
|             | 系统中断     | SBIF  |        | SBIF 写 0  | 是       | 否        |
| TIM_IERR    | 索引错误     | IERRF | IERRIE | IERRF 写 0 | 是       | 否        |
| TIM_TER     | 转换错误     | TERRF | TERRIE | TERRF 写 0 | 是       | 否        |



## 23.6 TIM1/TIM8 寄存器描述

在寄存器描述中使用的缩写列表请参考章节 2.1。

### 23.6.1 TIMx 控制寄存器 1 (TIMx\_CR1) (x = 1, 8)

偏移地址: 0x000

复位值: 0x0000

| 15       | 14 | 13 | 12         | 11           | 10  | 9        | 8  | 7    | 6        | 5  | 4   | 3   | 2   | 1    | 0   |
|----------|----|----|------------|--------------|-----|----------|----|------|----------|----|-----|-----|-----|------|-----|
| Reserved |    |    | DITH<br>EN | UIFRE<br>MAP | Res | CKD[1:0] |    | ARPE | CMS[1:0] |    | DIR | OPM | URS | UDIS | CEN |
|          |    |    | rw         | rw           |     | rw       | rw | rw   | rw       | rw | rw  | rw  | rw  | rw   | rw  |

Bit 15:13 保留, 必须保持复位值。

Bit 12 **DITHEN**: 使能抖动 (Dithering enable)

0: 抖动禁止

1: 抖动使能

注: DITHEN 位只能在 CEN 位复位时修改

Bit 11 **UIFREMAP**: UIF 状态位重映射 (UIF status bit remapping)

0: 无重映射。UIF 状态位未复制到 TIMx\_CNT 寄存器位 31

1: 重映射使能。UIF 状态位复制到 TIMx\_CNT 寄存器位 31

Bit 10 保留, 必须保持复位值。

Bit 9:8 **CKD[1:0]**: 时钟分频 (Clock division)

此位域指示定时器时钟 (tim\_ker\_ck) 频率与死区发生器以及数字滤波器 (tim\_etr\_in, tim\_tix) 所使用的死区及采样时钟 (t\_DTS) 之间的分频比

00:  $t_{DTS} = t_{tim\_ker\_ck}$

01:  $t_{DTS} = 2 \times t_{tim\_ker\_ck}$

10:  $t_{DTS} = 4 \times t_{tim\_ker\_ck}$

11: 保留, 不要设置成此值

Bit 7 **ARPE**: 自动重载预装载使能 (Auto-reload preload enable)

0: TIMx\_ARR 寄存器不进行缓冲;

1: TIMx\_ARR 寄存器进行缓冲。

Bit 6:5 **CMS[1:0]**: 中心对齐模式选择 (Center-aligned mode selection)

00: 边沿对齐模式。计数器根据方向位 (DIR) 递增计数或递减计数。

01: 中心对齐模式 1。计数器交替进行递增计数和递减计数。仅当计数器递减计数时, 配置为输出的通道 (TIMx\_CCMRx 寄存器中的 CxS=00) 的输出比较中断标志才置 1。

10: 中心对齐模式 2。计数器交替进行递增计数和递减计数。仅当计数器递增计数时, 配置为输出的通道 (TIMx\_CCMRx 寄存器中的 CxS=00) 的输出比较中断标志才置 1。

11: 中心对齐模式 3。计数器交替进行递增计数和递减计数。当计数器递增计数或递减计数时, 配置为输出的通道 (TIMx\_CCMRx 寄存器中的 CxS=00) 的输出比较中断标志都会置 1。

注: 只要计数器处于使能状态 (CEN=1), 就不得从边沿对齐模式切换为中心对齐模式。

Bit 4 **DIR**: 方向 (Direction)

- 0: 计数器递增计数;
- 1: 计数器递减计数。

注: 当定时器配置为中心对齐模式或编码器模式时, 该位为只读状态。

Bit 3 **OPM**: 单脉冲模式 (One pulse mode)

- 0: 计数器在发生更新事件时不会停止计数;
- 1: 计数器在发生下一更新事件时停止计数 (将 CEN 位清零)

Bit 2 **URS**: 更新请求源 (Update request source)

此位由软件置 1 和清零, 用以选择 UEV 事件源。

- 0: 使能时, 所有以下事件都会生成更新中断或 DMA 请求。此类事件包括:
  - 计数器上溢/下溢
  - 将 UG 位置 1
  - 通过从模式控制器生成的更新事件
- 1: 使能时, 只有计数器上溢/下溢会生成更新中断或 DMA 请求。

Bit 1 **UDIS**: 更新禁止 (Update disable)

此位由软件置 1 和清零, 用以使能/禁止 UEV 事件生成。

- 0: 使能 UEV。更新 (UEV) 事件可通过以下事件之一生成:
  - 计数器上出/下溢
  - 将 UG 位置 1
  - 通过从模式控制器生成的更新事件

然后更新影子寄存器的值。

- 1: 禁止 UEV。不会生成更新事件, 各影子寄存器的值 (ARR、PSC 和 CCRx) 保持不变。但如果将 UG 位置 1, 或者从从模式控制器接收到硬件复位, 则会重新初始化计数器和预分频器。

Bit 0 **CEN**: 计数器使能 (Counter enable)

- 0: 禁止计数器;
- 1: 使能计数器。

注: 只有事先通过软件将 CEN 位置 1, 才可以使用外部时钟和门控模式。而触发模式可通过硬件自动将 CEN 位置 1。

## 23.6.2 TIMx 控制寄存器 2 (TIMx\_CR2) (x = 1, 8)

偏移地址: 0x004

复位值: 0x0000 0000

|          |      |       |      |       |      |        |      |           |          |    |    |      |      |     |      |
|----------|------|-------|------|-------|------|--------|------|-----------|----------|----|----|------|------|-----|------|
| 31       | 30   | 29    | 28   | 27    | 26   | 25     | 24   | 23        | 22       | 21 | 20 | 19   | 18   | 17  | 16   |
| Reserved |      |       |      |       |      | MMS[3] | Res  | MMS2[3:0] |          |    |    | Res  | OIS6 | Res | OIS5 |
|          |      |       |      |       |      | rw     |      | rw        | rw       | rw | rw |      | rw   |     | rw   |
| 15       | 14   | 13    | 12   | 11    | 10   | 9      | 8    | 7         | 6        | 5  | 4  | 3    | 2    | 1   | 0    |
| OIS4N    | OIS4 | OIS3N | OIS3 | OIS2N | OIS2 | OIS1N  | OIS1 | TI1S      | MMS[2:0] |    |    | CCDS | CCUS | Res | CCPC |
| rw       | rw   | rw    | rw   | rw    | rw   | rw     | rw   | rw        | rw       | rw | rw | rw   | rw   |     | rw   |

Bit 31:26 保留，必须保持复位值。

Bit 24 保留，必须保持复位值。

Bit 23:20 **MMS2[3:0]**: 主模式选择 2 (Master mode selection 2)

这些位允许将信息发送到 ADC 进行同步 (tim\_trgo2) 以进行选择。组合如下:

0000: **重置** - 来自 TIMx\_EGR 寄存器的 UG 位用作触发器输出 (tim\_trgo2)。如果重置是由触发器输入 (配置为重置模式的从机模式控制器) 生成的, 则 tim\_trgo2 上的信号与实际重置相比会有所延迟。

0001: **使能** - 计数器使能信号 CNT\_EN 用作触发器输出 (tim\_trgo2)。它对于同时启动多个定时器或在一段时间内控制从机定时器的使能窗口是有用的。当在门控模式下配置时, 计数器使能信号是由 CEN 控制位和触发器输入之间的逻辑 AND 生成的。当计数器使能信号由触发器输入控制时, 除选择了主/从模式的情况外 (请参见 TIMx\_SMCR 寄存器中的 MSM 位描述), tim\_trgo2 上会有延迟。

0010: **更新** - 选择更新事件作为触发器输出 (tim\_trgo2)。例如, 主定时器可以用作从机定时器的预分频器。

0011: **比较脉冲** - 当 CC1IF 标志被设置时 (即使它已经是高电平), 当发生捕获或比较匹配时, 触发输出会发送一个正脉冲 (tim\_trgo2)。

0100: **比较** - tim\_oc1refc 信号用作触发器输出 (tim\_trgo2)

0101: **比较** - tim\_oc2refc 信号用作触发器输出 (tim\_trgo2)

0110: **比较** - tim\_oc3refc 信号用作触发器输出 (tim\_trgo2)

0111: **比较** - tim\_oc4refc 信号用作触发器输出 (tim\_trgo2)

1000: **比较** - tim\_oc5refc 信号用作触发器输出 (tim\_trgo2)

1001: **比较** - tim\_oc6refc 信号用作触发器输出 (tim\_trgo2)

1010: **比较脉冲** - tim\_oc4refc 上升或下降沿在 tim\_trgo2 上生成脉冲

1011: **比较脉冲** - tim\_oc6refc 上升或下降沿在 tim\_trgo2 上生成脉冲

1100: **比较脉冲** - tim\_oc4refc 或 tim\_oc6refc 上升沿在 tim\_trgo2 上生成脉冲

1101: **比较脉冲** - tim\_oc4refc 上升或 tim\_oc6refc 下降沿在 tim\_trgo2 上生成脉冲

1110: **比较脉冲** - tim\_oc5refc 或 tim\_oc6refc 上升沿在 tim\_trgo2 上生成脉冲

1111: **比较脉冲** - tim\_oc5refc 上升或 tim\_oc6refc 下降沿在 tim\_trgo2 上生成脉冲

注: 在从主定时器接收事件之前, 必须使能从机定时器或 ADC 的时钟, 并且在从主定时器接收触发器时不能随意更改。

Bit 19 保留, 必须保持复位值。

Bit 18 **OIS6**: 输出空闲状态 6 (OC6 输出)

参见 OIS1 位

Bit 17 保留, 必须保持复位值。

Bit 16 **OIS5**: 输出空闲状态 5 (OC5 输出)

参见 OIS1 位

Bit 15 **OIS4N**: 输出空闲状态 4 (OC4N 输出)

参见 OIS1 位

Bit 14 **OIS4**: 输出空闲状态 4 (OC4 输出)

参见 OIS1 位

Bit 13 **OIS3N**: 输出空闲状态 3 (OC3N 输出)

参见 OIS1 位

Bit 12 **OIS3**: 输出空闲状态 3 (OC3 输出)

参见 OIS1 位

Bit 11 **OIS2N**: 输出空闲状态 2 (OC2N 输出)

参见 OIS1 位

Bit 10 **OIS2**: 输出空闲状态 2 (OC2 输出)

参见 OIS1 位

Bit 9 **OIS1N**: 输出空闲状态 1 (OC1N 输出)

0: 当 MOE=0 时, 经过死区时间后 tim\_oc1n=0

1: 当 MOE=0 时, 经过死区时间后 tim\_oc1n=1

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 1、2 或 3, 此位即无法修改。

Bit 8 **OIS1**: 输出空闲状态 1 (OC1 输出)

0: 当 MOE=0 时, (如果 OC1N 有效, 则经过死区时间之后) tim\_oc1=0

1: 当 MOE=0 时, (如果 OC1N 有效, 则经过死区时间之后) tim\_oc1=1

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 1、2 或 3, 此位即无法修改。

Bit 7 **TI1S**: tim\_ti1 选择

0: tim\_ti1\_in[15:0]多路复用器输出被连接至 tim\_ti1 输入

1: tim\_ti1\_in[15:0], tim\_ti2\_in[15:0]和 tim\_ti3\_in[15:0]多路复用器输出被异或并连接至 tim\_ti1 输入

Bit 25, 6:4 **MMS[3:0]**: 主模式选择

这些位可选择主模式下将要发送到从定时器以实现同步的信息(tim\_trgo)。这些位的组合如下:

- 0000: **复位** - TIMx\_EGR 寄存器中的 UG 位用作触发输出 (tim\_trgo)。如果复位由触发输入生成 (从模式控制器配置为复位模式), 则 TRGO 上的信号相比实际复位会有延迟。
- 0001: **使能** - 计数器使能信号 CNT\_EN 用作触发输出 (tim\_trgo)。该触发输出可用于同时启动多个定时器, 或者控制在一段时间内使能从定时器。计数器使能信号可由 CEN 控制位产生。当配置为门控模式时, 也可由触发输入产生。当计数器使能信号由触发输入控制时, tim\_trgo 上会存在延迟, 选择主/从模式时除外 (请参见 TIMx\_SMCR 寄存器中 MSM 位的说明)。
- 0010: **更新** - 选择更新事件作为触发输出 (tim\_trgo)。例如, 主定时器可用作从定时器的预分频器。
- 0011: **比较脉冲** - 一旦发生比较匹配事件, 当 CC1IF 被置 1 时 (即使已为高电平), 触发输出都会发送一个正脉冲。(tim\_trgo)。
- 0100: **比较** - OC1REF 信号用作触发输出 (tim\_trgo)
- 0101: **比较** - OC2REF 信号用作触发输出 (tim\_trgo)
- 0110: **比较** - OC3REF 信号用作触发输出 (tim\_trgo)
- 0111: **比较** - OC4REF 信号用作触发输出 (tim\_trgo)
- 1000: **编码器时钟输出** - 编码器时钟信号被用于触发器输出 (tim\_trgo)。此编码对以下 SMS[3:0]值是有效的: 0001, 0010, 0011, 1010, 1011, 1100, 1101, 1110, 1111。任何其他的 SMS[3:0]编码是不被允许的, 这可能导致不可预知的行为。

其他编码保留

注: 在从主定时器接收事件之前, 必须使能从机定时器或 ADC 的时钟, 并且在从主定时器接收触发器时不能随意更改。

Bit 3 **CCDS**: 捕获/比较 DMA 选择 (Capture/compare DMA selection)

- 0: 发生 CCx 事件时发送 CCx DMA 请求
- 1: 发生更新事件时发送 CCx DMA 请求

Bit 2 **CCUS**: 捕获/比较控制更新选择

- 0: 如果捕获/比较控制位 (CCPC=1) 进行预装载, 仅通过将 COMG 位置 1 来对这些位进行更新;
- 1: 如果捕获/比较控制位 (CCPC=1) 进行预装载, 可通过将 COMG 位置 1 或 TRGI 的上升沿对这些位进行更新。

注: 此位仅对具有互补输出的通道有效。。

Bit 1 保留, 必须保持复位值。

Bit 0 **CCPC**: 捕获/比较预装载控制

- 0: CCxE, CCxNE 和 OCxM 位未进行预装载;
- 1: CCxE, CCxNE 和 OCxM 位进行了预装载, 写入这些位后, 仅当发生换向事件 (COM) (COMG 位置 1 或在 TRGI 上检测到上升沿, 取决于 CCUS 位) 时才会对这些位进行更新。

注: 此位仅对具有互补输出的通道有效。

### 23.6.3 TIMx 从模式控制寄存器 (TIMx\_SMCR) (x = 1, 8)

偏移地址: 0x008

复位值: 0x0000 0000

|          |     |           |    |          |    |       |       |          |         |         |    |          |          |    |        |
|----------|-----|-----------|----|----------|----|-------|-------|----------|---------|---------|----|----------|----------|----|--------|
| 31       | 30  | 29        | 28 | 27       | 26 | 25    | 24    | 23       | 22      | 21      | 20 | 19       | 18       | 17 | 16     |
| Reserved |     |           |    |          |    | SMSPS | SMSPE | Reserved |         | TS[4:3] |    | Reserved |          |    | SMS[3] |
|          |     |           |    |          |    | rw    | rw    |          |         | rw      | rw |          |          |    | rw     |
| 15       | 14  | 13        | 12 | 11       | 10 | 9     | 8     | 7        | 6       | 5       | 4  | 3        | 2        | 1  | 0      |
| ETP      | ECE | ETPS[1:0] |    | ETF[3:0] |    |       |       | MSM      | TS[2:0] |         |    | OCCS     | SMS[2:0] |    |        |
| rw       | rw  | rw        | rw | rw       | rw | rw    | rw    | rw       | rw      | rw      | rw | rw       | rw       | rw | rw     |

Bit 31:26 保留, 必须保持复位值。

Bit 25 **SMSPS**: SMS 预加载源 (SMS preload source)

此位选择触发 SMS[3:0]位域从预加载到活动的事件来源:

0: 由定时器的更新事件触发

1: 由索引事件触发

Bit 24 **SMSPE**: SMS 预加载使能 (SMS preload enable)

此位选择是否使能 SMS[3:0]位域的预加载:

0: SMS[3:0]位域不进行预加载

1: 使能 SMS[3:0]预加载

Bit 23:22 保留, 必须保持复位值。

Bit 21:20 **TS[4:3]**: 触发器选项- 位 4:3 (Trigger selection - bit 4:3)

参考 TS[2:0]的描述 - 位 6:4

Bit 19:17 保留, 必须保持复位值。

Bit 16 **SMS[3]**: 参考 SMS[2:0]的描述

Bit 15 **ETP**: 外部触发极性 (External trigger polarity)

此位可选择将 tim\_etr\_in 还是 tim\_etr\_in 的反相用于触发操作

0: tim\_etr\_in 未反相, 高电平或上升沿有效。

1: tim\_etr\_in 反相, 低电平或下降沿有效。

Bit 14 **ECE**: 外部时钟使能位 (External clock enable)

此位可使能外部时钟模式 2。

0: 禁止外部时钟模式 2

1: 使能外部时钟模式 2。计数器时钟由 tim\_etrif 信号的任意有效边沿提供。

注: 将 ECE 位置 1 与选择外部时钟模式 1 并将 tim\_trgi 连接到 tim\_etrif (SMS=111 且 TS=00111) 具有相同效果。

外部时钟模式 2 可以和以下从模式同时使用: 复位模式、门控模式和触发模式。不过此类情况下 tim\_trgi 不得连接 tim\_etrif (TS 位不得为 00111)。

如果同时使能外部时钟模式 1 和外部时钟模式 2, 则外部时钟输入为 tim\_etrif。



**Bit 13:12 ETPS[1:0]: 外部触发预分频器 (External trigger prescaler)**

外部触发信号 `tim_etrp` 频率不得超过 `TIMxCLK` 频率的 1/4。可通过使能预分频器来降低 `tim_etrp` 频率。这种方法在 `tim_etr_in` 输入快速外部时钟时非常有用。

00: 预分频器关闭

01: 2 分频 `tim_etr_in` 频率

10: 4 分频 `tim_etr_in` 频率

11: 8 分频 `tim_etr_in` 频率

**Bit 11:8 ETF[3:0]: 外部触发滤波器 (External trigger filter)**

此位域可定义 `tim_etrp` 信号的采样频率和适用于 `tim_etrp` 的数字滤波时间。数字滤波器由事件计数器组成，每 `N` 个事件才视为一个有效边沿：

0000: 无滤波器，按 `fDTS` 频率进行采样

0001:  $f_{\text{SAMPLING}} = f_{\text{CK\_INT}}$ ,  $N=2$ 。

0010:  $f_{\text{SAMPLING}} = f_{\text{CK\_INT}}$ ,  $N=4$ 。

0011:  $f_{\text{SAMPLING}} = f_{\text{CK\_INT}}$ ,  $N=8$ 。

0100:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/2$ ,  $N=6$ 。

0101:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/2$ ,  $N=8$ 。

0110:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/4$ ,  $N=6$ 。

0111:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/4$ ,  $N=8$ 。

1000:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/8$ ,  $N=6$ 。

1001:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/8$ ,  $N=8$ 。

1010:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$ ,  $N=5$ 。

1011:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$ ,  $N=6$ 。

1100:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$ ,  $N=8$ 。

1101:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$ ,  $N=5$ 。

1110:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$ ,  $N=6$ 。

1111:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$ ,  $N=8$ 。

**Bit 7 MSM: 主/从模式 (Master/slave mode)**

0: 不执行任何操作；

1: 当前定时器的触发输入事件(`tim_trgi`)的动作被推迟，以使当前定时器与其从定时器实现完美同步（通过 `tim_trgo`）。此设置适用于由单个外部事件对多个定时器进行同步的情况。



**Bit 6:4 TS[2:0]: 触发选择 (Trigger selection)**

此位域和位 TS[4:3]组合

此位域可选择将要用于同步计数器的触发输入。

00000: 内部触发 0 (tim\_itr0)

00001: 内部触发 1 (tim\_itr1)

00010: 内部触发 2 (tim\_itr2)

00011: 内部触发 3 (tim\_itr3)

00100: tim\_ti1 边沿检测器 (tim\_ti1f\_ed)

00101: 滤波后的定时器输入 1 (tim\_ti1fp1)

00110: 滤波后的定时器输入 2 (tim\_ti1fp2)

00111: 外部触发输入 (tim\_etr)

01000: 内部触发器 4 (tim\_itr4)

01001: 内部触发器 5 (tim\_itr5)

01010: 内部触发器 6 (tim\_itr6)

01011: 内部触发器 7 (tim\_itr7)

01100: 内部触发器 8 (tim\_itr8)

01101: 内部触发器 9 (tim\_itr9)

01110: 内部触发器 10 (tim\_itr10)

01111: 内部触发器 11 (tim\_itr11)

10000: 内部触发器 12 (tim\_itr12)

10001: 内部触发器 13 (tim\_itr13)

10010: 内部触发器 14 (tim\_itr14)

10011: 内部触发器 15 (tim\_itr15)

其他: 保留

有关定时器 ITRx 含义的详细信息, 请参见章节 TIMx 引脚和内部信号中的表格: TIMx 内部触发连接。

*注: 这些位只能在未使用的情况下 (例如, SMS=000 时) 进行更改, 以避免转换时出现错误的边沿检测。*

**Bit 3 OCCS: OCREF 清除源选择 (OCREF clear selection)**

此位用于选择 OCREF 清除源。

0: tim\_ocref\_clr\_int 信号连接到 tim\_ocref\_clr 输入

1: tim\_ocref\_clr\_int 信号连接到 tim\_etr

Bit 16, 2:0 **SMS[2:0]**: 从模式选择 (Slave mode selection)

当选择了外部信号, 触发信号 (tim\_trgi) 的有效边沿与选中的外部输入极性相关 (见输入控制寄存器和控制寄存器的说明)

0000: 关闭从模式 - 如果 CEN=1, 则预分频器直接由内部时钟驱动。

0001: 编码器模式 1, X2 模式 - 计数器根据 tim\_ti1fp1 电平在 tim\_ti2fp2 边沿递增/递减计数。

0010: 编码器模式 2, X2 模式- 计数器根据 tim\_ti2fp2 电平在 tim\_ti1fp1 边沿递增/递减计数。

0011: 编码器模式 3, X4 模式 - 计数器在 tim\_ti1fp1 和 tim\_ti2fp2 的边沿计数, 计数的方向取决于另外一个信号的电平。

0100: 复位模式 - 选中的触发输入 (tim\_trgi) 的上升沿重新初始化计数器, 并且产生一个更新寄存器的信号。

0101: 门控模式 - 当触发输入 (tim\_trgi) 为高时, 计数器的时钟开启。一旦触发输入变为低, 则计数器停止 (但不复位)。计数器的启动和停止都是受控的。

0110: 触发模式 - 计数器在触发输入 tim\_trgi 的上升沿启动 (但不复位), 只有计数器的启动是受控的。

0111: 外部时钟模式 1- 选中的触发输入 (tim\_trgi) 的上升沿驱动计数器。

1000: 组合重置+触发器模式-选定触发器输入 (tim\_trgi) 的上升沿会重新初始化计数器, 生成寄存器的更新并启动计数器。

1001: 组合门控+重置模式-当触发器输入 (tim\_trgi) 为高电平时, 计数器时钟使能。一旦触发器变为低电平, 计数器将停止并重置。同时控制计数器的启动和停止。

1010: 编码器模式: 时钟加方向, x2 模式。

1011: 编码器模式: 时钟加方向, x1 模式, CC2P 设置 tim\_ti2fp2 边沿敏感度。

1100: 编码器模式: 方向时钟, x2 模式。

1101: 编码器模式: 方向时钟, x1 模式, CC1P 和 CC2P 设置 tim\_ti1fp1 和 tim\_ti2fp2 边沿敏感度。

1110: 四象限编码器模式: x1 模式, 仅计算 tim\_ti1fp1 边沿, CC1P 设置边沿敏感度。

1111: 四象限编码器模式: x1 模式, 仅计算 tim\_ti2fp2 边沿, CC2P 设置边沿敏感度。

注: 如果选择 tim\_ti1f\_ed 作为触发输入 (TS=00100), 则不得使用门控模式。实际上 tim\_ti1f\_ed 为 TI1F 上的每个过渡输出 1 个脉冲, 而门控模式检查触发信号的电平。

注: 接收 tim\_trgo 或 tim\_trgo2 信号的从机外设 (定时器、ADC 等) 的时钟必须在从主定时器接收事件之前使能, 并且时钟频率 (预分频器) 在从主定时器接收触发器时不能随意更改。

### 23.6.4 TIMx DMA/中断使能寄存器 (TIMx\_DIER) (x = 1, 8)

偏移地址: 0x00C

复位值: 0x0000 0000

|          |     |       |       |       |       |       |     |        |        |       |       |          |       |       |     |
|----------|-----|-------|-------|-------|-------|-------|-----|--------|--------|-------|-------|----------|-------|-------|-----|
| 31       | 30  | 29    | 28    | 27    | 26    | 25    | 24  | 23     | 22     | 21    | 20    | 19       | 18    | 17    | 16  |
| Reserved |     |       |       |       |       |       |     | TERRIE | IERRIE | DIRIE | IDXIE | Reserved |       |       |     |
|          |     |       |       |       |       |       |     | rw     | rw     | rw    | rw    |          |       |       |     |
| 15       | 14  | 13    | 12    | 11    | 10    | 9     | 8   | 7      | 6      | 5     | 4     | 3        | 2     | 1     | 0   |
| Res      | TDE | COMDE | CC4DE | CC3DE | CC2DE | CC1DE | UDE | BIE    | TIE    | COMIE | CC4IE | CC3IE    | CC2IE | CC1IE | UIE |
|          | rw  | rw    | rw    | rw    | rw    | rw    | rw  | rw     | rw     | rw    | rw    | rw       | rw    | rw    | rw  |

Bit 31:24 保留, 必须保持复位值。

Bit 23 **TERRIE**: 转换错误中断使能 (Transition error interrupt enable)

0: 禁止转换错误中断;

1: 使能转换错误中断。

Bit 22 **IERRIE**: 索引错误中断使能 (Index error interrupt enable)

0: 禁止索引错误中断;

1: 使能索引错误中断。

Bit 21 **TERRIE**: 方向改变使能 (Direction change interrupt enable)

0: 禁止方向改变中断;

1: 使能方向改变中断。

Bit 20 **IDXIE**: 索引中断使能 (Index interrupt enable)

0: 禁止索引中断;

1: 使能索引中断。

Bit 19:15 保留, 必须保持复位值。

Bit 14 **TDE**: 触发 DMA 请求使能 (Trigger DMA request enable)

0: 禁止触发 DMA 请求;

1: 使能触发 DMA 请求。

Bit 13 **COMDE**: 换向 DMA 请求使能 (COM DMA request enable)

0: 禁止换向 DMA 请求;

1: 使能换向 DMA 请求。

Bit 12 **CC4DE**: 捕获/比较 4 DMA 请求使能 (Capture/Compare 4 DMA request enable)

0: 禁止 CC4 DMA 请求;

1: 使能 CC4 DMA 请求。

Bit 11 **CC3DE**: 捕获/比较 3 DMA 请求使能 (Capture/Compare 3 DMA request enable)

0: 禁止 CC3 DMA 请求;

1: 使能 CC3 DMA 请求。

Bit 10 **CC2DE**: 捕获/比较 2 DMA 请求使能 (Capture/Compare 2 DMA request enable)

0: 禁止 CC2 DMA 请求;

1: 使能 CC2 DMA 请求。

Bit 9 **CC1DE**: 捕获/比较 1 DMA 请求使能 (Capture/Compare 1 DMA request enable)

0: 禁止 CC1 DMA 请求;

1: 使能 CC1 DMA 请求。

Bit 8 **UDE**: 更新 DMA 请求使能 (Capture/Compare 1 DMA request enable)

0: 禁止更新 DMA 请求;

1: 使能更新 DMA 请求。

Bit 7 **BIE**: 刹车中断使能 (Break interrupt enable)

0: 禁止刹车中断;

1: 使能刹车中断。

Bit 6 **TIE**: 触发信号 (TGRI) 中断使能 (Trigger interrupt enable)

0: 禁止触发信号(TGRI) 中断;

1: 使能触发信号(TGRI) 中断。

Bit 5 **COMIE**: COM 中断使能 (COM interrupt enable)

0: 禁止 COM 中断;

1: 使能 COM 中断。

Bit 4 **CC4IE**: 捕获/比较 4 中断使能 (Capture/Compare 4 interrupt enable)

0: 禁止 CC4 中断;

1: 使能 CC4 中断。

Bit 3 **CC3IE**: 捕获/比较 3 中断使能 (Capture/Compare 3 interrupt enable)

0: 禁止 CC3 中断;

1: 使能 CC3 中断。

Bit 2 **CC2IE**: 捕获/比较 2 中断使能 (Capture/Compare 2 interrupt enable)

0: 禁止 CC2 中断;

1: 使能 CC2 中断。

Bit 1 **CC1IE**: 捕获/比较 1 中断使能 (Capture/Compare 1 interrupt enable)

0: 禁止 CC1 中断;

1: 使能 CC1 中断。

Bit 0 **UIE**: 更新中断使能 (Update interrupt enable)

0: 禁止更新中断;

### 23.6.5 TIMx 状态寄存器 (TIMx\_SR) (x = 1, 8)

偏移地址: 0x010

复位值: 0x0000 0000

|          |    |       |       |       |       |       |       |       |       |       |       |          |       |       |       |
|----------|----|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|----------|-------|-------|-------|
| 31       | 30 | 29    | 28    | 27    | 26    | 25    | 24    | 23    | 22    | 21    | 20    | 19       | 18    | 17    | 16    |
| Reserved |    |       |       |       |       |       |       | TERRF | IERRF | DIRF  | IDXF  | Reserved |       | CC6IF | CC5IF |
|          |    |       |       |       |       |       |       | rc_w0 | rc_w0 | rc_w0 | rc_w0 |          |       | rc_w0 | rc_w0 |
| 15       | 14 | 13    | 12    | 11    | 10    | 9     | 8     | 7     | 6     | 5     | 4     | 3        | 2     | 1     | 0     |
| Reserved |    | SBIF  | CC4OF | CC3OF | CC2OF | CC1OF | B2IF  | BIF   | TIF   | COMIF | CC4IF | CC3IF    | CC2IF | CC1IF | UIF   |
|          |    | rc_w0 | rc_w0 | rc_w0 | rc_w0 | rc_w0 | rc_w0 | rc_w0 | rc_w0 | rc_w0 | rc_w0 | rc_w0    | rc_w0 | rc_w0 | rc_w0 |

Bit 31:24 保留，必须保持复位值。

Bit 23 **TERRF**: 转换错误中断标志 (Transition error interrupt flag)

当在编码器模式下检测到转换错误时，硬件会设置此标志当软件将其写入“0”时，该标志将被清除

0: 未检测到编码器转换错误

1: 已检测到编码器转换错误

Bit 22 **IERRF**: 索引错误中断标志 (Index error interrupt flag)

当检测到索引错误时，硬件会设置此标志当软件将其写入“0”时，该标志将被清除

0: 未检测到索引错误

1: 已检测到索引错误

Bit 21 **DIRF**: 方向改变中断标志 (Direction change interrupt flag)

当编码器模式中的方向发生变化时 (TIMx\_CR 中的 DIR 位值正在改变)，硬件会设置此标志当软件将其写入“0”时，该标志将被清除

0: 方向未改变

1: 方向已改变

Bit 20 **IDXF**: 索引中断标志 (Index interrupt flag)

当检测到索引事件时，硬件会设置此标志当软件将其写入“0”时，该标志将被清除

0: 未发生索引事件

1: 已发生索引事件

Bit 19:18 保留，必须保持复位值。

Bit 17 **CC6IF**: 比较 6 中断标志

参考 CC1IF 描述

注: Channel 6 只能被配置为输出。

Bit 16 **CC5IF**: 比较 5 中断标志

参考 CC1IF 描述

注: Channel 5 只能被配置为输出。

Bit 15:14 保留，必须保持复位值。

Bit 13 **SBIF**: 系统刹车中断标志 (System break interrupt flag)

该标志由硬件在系统刹车输入激活时立即设置。如果系统刹车输入不处于有效状态，则可以通过软件清除该标志。

要重新启动 PWM 操作，必须将此标志重置为 0。

0: 未发生刹车事件。

1: 在系统刹车输入上检测到有效电平。如果 TIMx\_DIER 寄存器中的 BIE=1，则生成中断。

Bit 12 **CC4OF**: 捕获/比较 4 重复捕获标志

参考 CC1OF 的描述

Bit 11 **CC3OF**: 捕获/比较 3 重复捕获标志

参考 CC1OF 的描述

Bit 10 **CC2OF**: 捕获/比较 2 重复捕获标志

参考 CC1OF 的描述

**Bit 9 CC1OF: 捕获/比较 1 重复捕获标志**

仅当将相应通道配置为输入捕获模式时，此标志位才会由硬件置 1。通过软件写入“0”可将该位清零。

0: 未检测到重复捕获

1: TIMx\_CCR1 寄存器中已捕获到计数器值且 CC1IF 标志已置 1。

**Bit 8 BI2F: 刹车 2 中断标志 (Break 2 interrupt flag)**

只要刹车 2 输入变为有效状态，此标志便由硬件置 1。刹车 2 输入无效后可通过软件对其清零。

0: 无刹车事件产生；

1: 刹车 2 输入上检测到有效电平。如果 TIMx\_DIER 寄存器上 BIE=1 则产生一个中断。

**Bit 7 BIF: 刹车中断标志 (Break interrupt flag)**

只要刹车输入变为有效状态，此标志便由硬件置 1。刹车输入无效后可通过软件对其清零。

0: 无刹车事件产生；

1: 刹车输入上检测到有效电平。如果 TIMx\_DIER 寄存器上 BIE=1 则产生一个中断。

**Bit 6 TIF: 触发中断标志 (Trigger interrupt flag)**

在除门控模式以外的所有模式下，当使能从模式控制器后在 tim\_trgi 输入上检测到有效边沿时，该标志将由硬件置 1。选择门控模式时，该标志将在计数器启动或停止时置 1。但需要通过软件清零。

0: 未发生触发事件。

1: 触发中断挂起。

**Bit 5 COMIF: COM 中断标志 (COM interrupt flag)**

此标志在发生 COM 事件时（捕获/比较控制位 CCxE、CCxNE 和 OCxM 已更新时）由硬件置 1。但需要通过软件清零。

0: 未发生 COM 事件。

1: COM 中断挂起。

**Bit 4 CC4IF: 捕获/比较 4 中断标志 (Capture/Compare 4 interrupt flag)**

参考 CC1IF 描述。

**Bit 3 CC3IF: 捕获/比较 3 中断标志 (Capture/Compare 3 interrupt flag)**

参考 CC1IF 描述。

**Bit 2 CC2IF: 捕获/比较 2 中断标志 (Capture/Compare 2 interrupt flag)**

参考 CC1IF 描述。

**Bit 1 CC1IF: 捕获/比较 1 中断标志 (Capture/Compare 1 interrupt flag)**

此标志由硬件设置，它可以通过软件（输入捕获或输出比较模式）或读取 TIMx\_CCR1 寄存器（仅输入捕获模式）来清除。

0: 无比较匹配/无输入捕获发生

1: 发生比较匹配或输入捕获

**如果通道 CC1 配置为输出：**当计数器 TIMx\_CNT 的内容与 TIMx\_CCR1 寄存器内容匹配时，会设置此标志。当 TIMx\_CCR1 的值大于 TIMx\_ARR 的值时，CC1IF 位将在计数器发生上溢（递增计数模式和增减计数模式下）或下溢（递减计数模式下）时变为高电平。在中心对齐模式下有 3 种可能的标志设置选项，请参考 TIMx\_CR1 寄存器中的 CMS 位以获取完整描述。



如果通道 **CC1** 配置为输入：当在 **TIMx\_CCR1** 寄存器中捕获到计数器值时（在 **TIMx\_CCER** 中定义的 **CC1P** 和 **CC1NP** 位设置所确定的 **IC1** 上检测到边沿时），会设置此位。

Bit 0 **UIF**：更新中断标志（Update interrupt flag）

该位在发生更新事件时通过硬件置 1。但需要通过软件清零。

0：未发生更新。

1：更新中断挂起。该位在以下情况下更新寄存器时由硬件置 1：

- **TIMx\_CR1** 寄存器中的 **UDIS=0**，并且重复计数器值上溢或下溢时（重复计数器=0 时更新）。
- **TIMx\_CR1** 寄存器中的 **URS=0** 且 **UDIS=0**，并且由软件使用 **TIMx\_EGR** 寄存器中的 **UG** 位重新初始化 **CNT** 时。
- **TIMx\_CR1** 寄存器中的 **URS=0** 且 **UDIS=0**，并且 **CNT** 由触发事件重新初始化时（请参见章节：TIMx 从模式控制寄存器（**TIMx\_SMCR**）（ $x=1, 8$ ））。

### 23.6.6 TIMx 事件产生寄存器（TIMx\_EGR）（ $x=1, 8$ ）

偏移地址：0x014

复位：0x0000

| 15       | 14 | 13 | 12 | 11 | 10   | 9    | 8   | 7  | 6  | 5    | 4    | 3    | 2    | 1    | 0  |
|----------|----|----|----|----|------|------|-----|----|----|------|------|------|------|------|----|
| Reserved |    |    |    |    | CC6G | CC5G | B2G | BG | TG | COMG | CC4G | CC3G | CC2G | CC1G | UG |
|          |    |    |    |    | w    | w    | w   | w  | w  | w    | w    | w    | w    | w    | w  |

Bit 15:11 保留，必须保持复位值。

Bit 10 **CC6G**：捕获/比较 6 事件生成（Capture/Compare 6 generation）

参考 **CC1G** 描述。

Bit 9 **CC5G**：捕获/比较 5 事件生成（Capture/Compare 5 generation）

参考 **CC1G** 描述。

Bit 8 **B2G**：刹车 2 生成（Break generation）

此位由软件置 1 以生成事件，并由硬件自动清零。

0：不执行任何操作；

1：生成刹车 2 事件。MOE 位清零且 BIF 标志置 1。使能后可发生相关中断事件或 DMA 传输事件。

Bit 7 **BG**：刹车生成（Break generation）

此位由软件置 1 以生成事件，并由硬件自动清零。

0：不执行任何操作；

1：生成刹车事件。MOE 位清零且 BIF 标志置 1。使能后可发生相关中断或 DMA 传输事件。



Bit 6 **TG**: 生成触发信号 (Trigger generation)

此位由软件置 1 以生成事件, 并由硬件自动清零。

0: 不执行任何操作;

1: TIMx\_SR 寄存器中的 TIF 标志置 1。使能后可发生相关中断或 DMA 传输事件。

Bit 5 **COMG**: 捕获/比较控制更新生成 (Capture/Compare control update generation)

该位可通过软件置 1, 并由硬件自动清零

0: 不执行任何操作;

1: CCPC 位置 1 时, 可更新 CCxE、CCxNE 和 OCxM 位。

注: 此位仅对具有互补输出的通道有效。

Bit 4 **CC4G**: 捕获/比较 4 生成 (Capture/Compare 4 generation)

参考 CC1G 描述。

Bit 3 **CC3G**: 捕获/比较 3 生成 (Capture/Compare 3 generation)

参考 CC1G 描述。

Bit 2 **CC2G**: 捕获/比较 2 生成 (Capture/Compare 2 generation)

参考 CC1G 描述。

Bit 1 **CC1G**: 捕获/比较 1 生成 (Capture/Compare 1 generation)

此位由软件置 1 以生成事件, 并由硬件自动清零。

0: 不执行任何操作;

1: 通道 1 上生成捕获/比较事件:

若通道 **CC1** 配置为输出:

使能时, CC1IF 标志置 1 并发送相应的中断或 DMA 请求。

如果通道 **CC1** 配置为输入:

TIMx\_CCR1 寄存器中将捕获到计数器当前值。使能时, CC1IF 标志置 1 并发送相应的中断或 DMA 请求。如果 CC1IF 标志已为高电平, CC1OF 标志将置 1。

Bit 0 **UG**: 更新生成 (Update generation)

该位可通过软件置 1, 并由硬件自动清零。

0: 不执行任何操作;

1: 重新初始化计数器并生成一个寄存器更新事件。请注意, 预分频器计数器也将清零 (但预分频比不受影响)。如果选择中心对齐模式或 DIR=0 (递增计数), 计数器将清零; 如果 DIR=1 (递减计数), 计数器将使用自动重载值 (TIMx\_ARR)。

### 23.6.7 TIMx 捕获/比较模式寄存器 1 [复用] (TIMx\_CCMR1) (x=1, 8)

偏移地址: 0x018

复位值: 0x0000 0000

相同的寄存器可以用于输入捕获模式 (本节) 或输出比较模式 (下节)。通道方向通过配置相应的 CCxS 位进行定义。此寄存器的所有其它位在输入模式和输出模式下的功能均不同。可以独立组合这两种模式 (例如, 通道 1 用作输入捕获模式而通道 2 用作输出比较模式)

|           |    |    |    |             |    |           |    |           |    |    |    |             |    |           |    |
|-----------|----|----|----|-------------|----|-----------|----|-----------|----|----|----|-------------|----|-----------|----|
| 31        | 30 | 29 | 28 | 27          | 26 | 25        | 24 | 23        | 22 | 21 | 20 | 19          | 18 | 17        | 16 |
| Reserved  |    |    |    |             |    |           |    |           |    |    |    |             |    |           |    |
| 15        | 14 | 13 | 12 | 11          | 10 | 9         | 8  | 7         | 6  | 5  | 4  | 3           | 2  | 1         | 0  |
| IC2F[3:0] |    |    |    | IC2PSC[1:0] |    | CC2S[1:0] |    | IC1F[3:0] |    |    |    | IC1PSC[1:0] |    | CC1S[1:0] |    |
| rw        | rw | rw | rw | rw          | rw | rw        | rw | rw        | rw | rw | rw | rw          | rw | rw        | rw |

## 输入捕获模式

Bit 31:16 保留，始终读为 0。

Bit 15:12 **IC2F[3:0]**: 输入捕获 2 滤波器 (Input capture 2 filter)

Bit 11:10 **IC2PSC[1:0]**: 输入/捕获 2 预分频器 (input capture 2 prescaler)

Bit 9:8 **CC2S[1:0]**: 捕获/比较 2 选择 (Capture/compare 2 selection)

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00: CC2 通道被配置为输出;

01: CC2 通道被配置为输入, tim\_ic2 映射在 tim\_ti2 上;

10: CC2 通道被配置为输入, tim\_ic2 映射在 tim\_ti1 上;

11: CC2 通道被配置为输入, tim\_ic2 映射在 tim\_trc 上。此模式仅在通过 TS 位

(TIMx\_SMCR 寄存器) 选择内部触发输入时有效

注: 仅当通道关闭时 (TIMx\_CCER 中的 CC2E = 0), 才可向 CC2S 位写入数据。

#### Bit 7:4 IC1F[3:0]: 输入捕获 1 滤波器 (Input capture 1 filter)

此位域可定义 tim\_ti1 输入的采样频率和适用于 tim\_ti1 的数字滤波器带宽。数字滤波器由事件计数器组成，每 N 个事件才视为一个有效边沿：

0000: 无滤波器，按  $f_{DTS}$  频率进行采样

0001:  $f_{SAMPLING} = f_{tim\_ker\_ck}$ ,  $N=2$ 。

0010:  $f_{SAMPLING} = f_{tim\_ker\_ck}$ ,  $N=4$ 。

0011:  $f_{SAMPLING} = f_{tim\_ker\_ck}$ ,  $N=8$ 。

0100:  $f_{SAMPLING} = f_{DTS}/2$ ,  $N=6$ 。

0101:  $f_{SAMPLING} = f_{DTS}/2$ ,  $N=8$ 。

0110:  $f_{SAMPLING} = f_{DTS}/4$ ,  $N=6$ 。

0111:  $f_{SAMPLING} = f_{DTS}/4$ ,  $N=8$ 。

1000:  $f_{SAMPLING} = f_{DTS}/8$ ,  $N=6$ 。

1001:  $f_{SAMPLING} = f_{DTS}/8$ ,  $N=8$ 。

1010:  $f_{SAMPLING} = f_{DTS}/16$ ,  $N=5$ 。

1011:  $f_{SAMPLING} = f_{DTS}/16$ ,  $N=6$ 。

1100:  $f_{SAMPLING} = f_{DTS}/16$ ,  $N=8$ 。

1101:  $f_{SAMPLING} = f_{DTS}/32$ ,  $N=5$ 。

1110:  $f_{SAMPLING} = f_{DTS}/32$ ,  $N=6$ 。

1111:  $f_{SAMPLING} = f_{DTS}/32$ ,  $N=8$ 。

#### Bit 3:2 IC1PSC[1:0]: 输入/捕获 1 预分频器 (Input capture 1 prescaler)

此位域定义 CC1 输入 (tim\_ic1) 的预分频比。

只要 CC1E=0 (TIMx\_CCER 寄存器)，预分频器便立即复位。

00: 无预分频器，捕获输入上每检测到一个边沿便执行捕获

01: 每发生 2 个事件便执行一次捕获；

10: 每发生 4 个事件便执行一次捕获；

11: 每发生 8 个事件便执行一次捕获。

#### Bit 1:0 CC1S[1:0]: 捕获/比较 1 选择 (Capture/Compare 1 selection)

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00: CC1 通道被配置为输出；

01: CC1 通道被配置为输入，tim\_ic1 映射在 tim\_ti1 上；

10: CC1 通道被配置为输入，tim\_ic1 映射在 tim\_ti2 上；

11: CC1 通道被配置为输入，tim\_ic1 映射在 tim\_trc 上。此模式仅在通过 TS 位 (TIMx\_SMCR 寄存器) 选择内部触发输入时有效

注：仅当通道关闭时 (TIMx\_CCER 中的 CC1E = 0)，才可向 CC1S 位写入数据。

### 23.6.8 TIMx 捕获/比较模式寄存器 1[复用] (TIMx\_CCMR1) (x = 1, 8)

偏移地址: 0x018

复位值: 0x0000 0000

相同的寄存器可以用于输入捕获模式 (本节) 或输出比较模式 (下节)。通道方向通过配置相应的 CCxS 位进行定义。此寄存器的所有其它位在输入模式和输出模式下的功能均不同。可以独立组合这两种模式 (例如，通道 1 用作输入捕获模式而通道 2 用作输出比较模式)

| 31       | 30        | 29 | 28 | 27    | 26    | 25        | 24      | 23       | 22        | 21 | 20 | 19    | 18    | 17        | 16      |
|----------|-----------|----|----|-------|-------|-----------|---------|----------|-----------|----|----|-------|-------|-----------|---------|
| Reserved |           |    |    |       |       |           | OC2M[3] | Reserved |           |    |    |       |       |           | OC1M[3] |
|          |           |    |    |       |       |           | rw      |          |           |    |    |       |       |           | rw      |
| 15       | 14        | 13 | 12 | 11    | 10    | 9         | 8       | 7        | 6         | 5  | 4  | 3     | 2     | 1         | 0       |
| OC2CE    | OC2M[2:0] |    |    | OC2PE | OC2FE | CC2S[1:0] |         | OC1CE    | OC1M[2:0] |    |    | OC1PE | OC1FE | CC1S[1:0] |         |
| rw       | rw        | rw | rw | rw    | rw    | rw        | rw      | rw       | rw        | rw | rw | rw    | rw    | rw        | rw      |

## 输出比较模式

Bit 31:25 保留，必须保持复位值。

Bit 23:17 保留，必须保持复位值。

Bit 15 **OC2CE**: 输出比较 2 清零使能 (Output Compare 2 clear enable)

Bit 24, 14:12 **OC2M[3:0]**: 输出比较 2 模式 (Output Compare 2 mode)

Bit 11 **OC2PE**: 输出比较 2 预装载使能 (Output Compare 2 preload enable)

Bit 10 **OC2FE**: 输出比较 2 快速使能 (Output Compare 2 fast enable)

Bit 9:8 **CC2S[1:0]**: 捕获/比较 2 选择 (Capture/Compare 2 selection)

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00: CC2 通道配置为输出

01: CC2 通道配置为输入, tim\_ic2 映射到 tim\_ti2 上

10: CC2 通道配置为输入, tim\_ic2 映射到 tim\_ti1 上

11: CC2 通道配置为输入, tim\_ic2 映射到 tim\_trc 上。此模式仅在通过 TS 位 (TIMx\_SMCR 寄存器) 选择内部触发输入时有效

注: 仅当通道关闭时 (TIMx\_CCER 中的 CC2E = 0), 才可向 CC2S 位写入数据

Bit 7 **OC1CE**: 输出比较 1 清零使能 (Output Compare 1 clear enable)

0: tim\_oc1ref 不受 tim\_ocref\_clr\_int 信号影响;

1: tim\_oc1ref 在 tim\_ocref\_clr\_int 信号中被检测到高电平, tim\_oc1ref 立即清零。  
(tim\_ocref\_clr 输入或 tim\_etrif 输入)

Bit 16, 6:4 OC1M[3:0]: 输出比较 1 模式 (Output Compare 1 mode)

这些位定义提供 tim\_oc1 和 tim\_oc1n 的输出参考信号 tim\_oc1ref 的行为。tim\_oc1ref 为高电平有效, 而 tim\_oc1 和 tim\_oc1n 的有效电平则取决于 CC1P 位和 CC1NP 位。

0000: **冻结** - 输出比较寄存器 TIMx\_CCR1 与计数器 TIMx\_CNT 进行比较不会对输出造成任何影响。(该模式用于生成时基)。

0001: 将通道 1 设置为匹配时输出有效电平。当计数器 TIMx\_CNT 与捕获/比较寄存器 1 (TIMx\_CCR1) 匹配时, tim\_oc1ref 信号强制变为高电平。

0010: 将通道 1 设置为匹配时输出无效电平。当计数器 TIMx\_CNT 与捕获/比较寄存器 1 (TIMx\_CCR1) 匹配时, tim\_oc1ref 信号强制变为低电平。

0011: **翻转** - TIMx\_CNT=TIMx\_CCR1 时, tim\_oc1ref 发生翻转。

0100: **强制变为无效电平** - tim\_oc1ref 强制变为低电平。

0101: **强制变为有效电平** - tim\_oc1ref 强制变为高电平。

0110: **PWM 模式 1** - 在递增计数模式下, 只要 TIMx\_CNT < TIMx\_CCR1, 通道 1 便为有效状态, 否则为无效状态。在递减计数模式下, 只要 TIMx\_CNT > TIMx\_CCR1, 通道 1 便为无效状态 (tim\_oc1ref=0), 否则为有效状态 (tim\_oc1ref=1)。

0111: **PWM 模式 2** - 在递增计数模式下, 只要 TIMx\_CNT < TIMx\_CCR1, 通道 1 便为无效状态, 否则为有效状态。在递减计数模式下, 只要 TIMx\_CNT > TIMx\_CCR1, 通道 1 便为有效状态, 否则为无效状态。

1000: **可重触发 OPM 模式 1** - 在向上计数模式下, 通道在检测到触发事件 (tim\_trgi 信号) 之前一直处于活动状态。然后, 会像 PWM 模式 1 那样执行比较, 并在下次更新时使通道再次处于活动状态。在向下计数模式下, 通道在检测到触发事件 (tim\_trgi 信号) 之前一直处于非活动状态。然后, 会像 PWM 模式 1 那样执行比较, 并在下次更新时使通道再次处于非活动状态。

1001: **可重触发 OPM 模式 2** - 在向上计数模式下, 通道在检测到触发事件 (tim\_trgi 信号) 之前一直处于非活动状态。然后, 会像 PWM 模式 2 那样执行比较, 并在下次更新时使通道再次处于非活动状态。在向下计数模式下, 通道在检测到触发事件 (tim\_trgi 信号) 之前一直处于活动状态。然后, 会像 PWM 模式 1 那样执行比较, 并在下次更新时使通道再次处于活动状态。

1010: 保留。

1011: 保留。

1100: **组合 PWM 模式 1** - tim\_oc1ref 的行为与 PWM 模式 1 相同。tim\_oc1refc 是 tim\_oc1ref 和 tim\_oc2ref 之间的逻辑 OR。

1101: **组合 PWM 模式 2** - tim\_oc1ref 的行为与 PWM 模式 2 相同。tim\_oc1refc 是 tim\_oc1ref 和 tim\_oc2ref 之间的逻辑 AND。

1110: **非对称 PWM 模式 1** - tim\_oc1ref 的行为与 PWM 模式 1 相同。当计数器向上计数时, tim\_oc1refc 输出 tim\_oc1ref; 当计数器向下计数时, tim\_oc1refc 输出 tim\_oc2ref。

1111: **非对称 PWM 模式 2** - tim\_oc1ref 的行为与 PWM 模式 2 相同。当计数器向上计数时, tim\_oc1refc 输出 tim\_oc1ref; 当计数器向下计数时, tim\_oc1refc 输出 tim\_oc2ref。

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 3 且 CC1S=“00” (通道配置为输出), 这些位即无法修改。

注: 在 PWM 模式 1 或 PWM 模式 2 下, 仅当比较结果发生改变或输出比较模式由“冻

结”模式切换到“PWM”模式时，OCREF 电平才会发生更改。

注：此位域将在具有互补输出的通道上进行预装载。如果TIMx\_CR2 寄存器中的CCPC 位置1，则仅当生成COM 事件时，OC1M 有效位才会从预装载位获取新值。

Bit 3 **OC1PE**：输出比较 1 预装载使能（Output Compare 1 preload enable）

- 0：禁止与 TIMx\_CCR1 相关的预装载寄存器。可随时向 TIMx\_CCR1 写入数据，写入后将立即使用新值。
- 1：使能与 TIMx\_CCR1 相关的预装载寄存器。可读/写访问预装载寄存器。TIMx\_CCR1 预装载值在每次生成更新事件时都会装载到活动寄存器中。

注：只要编程了LOCK（TIMx\_BDTR 寄存器中的LOCK 位）级别3 且CC1S=“00”（通道配置为输出），这些位即无法修改。

Bit 2 **OC1FE**：输出比较 1 快速使能（Output Compare 1 fast enable）

此位减少了触发事件和定时器输出上的转换之间的延迟。它必须在单脉冲模式（在TIMx\_CR1 寄存器中设置 OPM 位）中使用，以便在启动触发后尽快开始输出脉冲。

- 0：即使触发开启，CC1 也将根据计数器和 CCR1 值正常工作。触发输入出现边沿时，激活 CC1 输出的最短延迟时间为 5 个时钟周期。
- 1：触发输入上出现有效边沿相当于 CC1 输出上的比较匹配。随后，无论比较结果如何，OC 都设置为比较电平。采样触发输入和激活 CC1 输出的延迟时间缩短为 3 个时钟周期。仅当通道配置为 PWM1 或 PWM2 模式时，OCFE 才会起作用。

Bit 1:0 **CC1S[1:0]**：捕获/比较 1 选择（Capture/Compare 1 selection）

此位域定义通道方向（输入/输出）以及所使用的输入。

- 00：CC1 通道配置为输出
- 01：CC1 通道配置为输入，tim\_ic1 映射到 tim\_ti1 上
- 10：CC1 通道配置为输入，tim\_ic1 映射到 tim\_ti2 上
- 11：CC1 通道配置为输入，tim\_ic1 映射到 tim\_trc 上。此模式仅在通过 TS 位（TIMx\_SMCR 寄存器）选择内部触发输入时有效

注：仅当通道关闭时（TIMx\_CCER 中的 CC1E = 0），才可向 CC1S 位写入数据

### 23.6.9 TIMx 捕获/比较模式寄存器 2 [复用]（TIMx\_CCMR2）（x=1, 8）

偏移地址：0x01C

复位值：0x0000 0000

相同的寄存器可以用于输入捕获模式（本节）或输出比较模式（下节）。通道方向通过配置相应的CCxS 位进行定义。此寄存器的所有其它位在输入模式和输出模式下的功能均不同。可以独立组合这两种模式（例如，通道 1 用作输入捕获模式而通道 2 用作输出比较模式）

|           |    |    |    |             |    |           |    |           |    |    |    |             |    |           |    |
|-----------|----|----|----|-------------|----|-----------|----|-----------|----|----|----|-------------|----|-----------|----|
| 31        | 30 | 29 | 28 | 27          | 26 | 25        | 24 | 23        | 22 | 21 | 20 | 19          | 18 | 17        | 16 |
| Reserved  |    |    |    |             |    |           |    |           |    |    |    |             |    |           |    |
| 15        | 14 | 13 | 12 | 11          | 10 | 9         | 8  | 7         | 6  | 5  | 4  | 3           | 2  | 1         | 0  |
| IC4F[3:0] |    |    |    | IC4PSC[1:0] |    | CC4S[1:0] |    | IC3F[3:0] |    |    |    | IC3PSC[1:0] |    | CC3S[1:0] |    |
| rw        | rw | rw | rw | rw          | rw | rw        | rw | rw        | rw | rw | rw | rw          | rw | rw        | rw |

输入捕获模式



Bit 31:16 保留，始终读为 0。

Bit 15:12 **IC4F[3:0]**: 输入捕获 4 滤波器 (Input capture 4 filter)

Bit 11:10 **IC4PSC[1:0]**: 输入/捕获 4 预分频器 (input capture 4 prescaler)

Bit 9:8 **CC4S[1:0]**: 捕获/比较 4 选择 (Capture/compare 4 selection)

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00: CC4 通道被配置为输出;

01: CC4 通道被配置为输入, tim\_ic4 映射在 tim\_ti4 上;

10: CC4 通道被配置为输入, tim\_ic4 映射在 tim\_ti3 上;

11: CC4 通道被配置为输入, tim\_ic4 映射在 tim\_trc 上。此模式仅在通过 TS 位 (TIMx\_SMCR 寄存器) 选择内部触发输入时有效

*注: 仅当通道关闭时 (TIMx\_CCER 中的 CC4E = 0), 才可向 CC4S 位写入数据。*

Bit 7:4 **IC3F[3:0]**: 输入捕获 3 滤波器 (Input capture 3 filter)

Bit 3:2 **IC3PSC[1:0]**: 输入/捕获 1 预分频器 (Input capture 3 prescaler)

Bit 1:0 **CC3S[1:0]**: 捕获/比较 3 选择 (Capture/Compare 3 selection)

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00: CC3 通道被配置为输出;

01: CC3 通道被配置为输入, tim\_ic3 映射在 tim\_ti3 上;

10: CC3 通道被配置为输入, tim\_ic3 映射在 tim\_ti4 上;

11: CC3 通道被配置为输入, tim\_ic3 映射在 tim\_trc 上。此模式仅在通过 TS 位 (TIMx\_SMCR 寄存器) 选择内部触发输入时有效

*注: 仅当通道关闭时 (TIMx\_CCER 中的 CC3E = 0), 才可向 CC3S 位写入数据。*

### 23.6.10 TIMx 捕获/比较模式寄存器 2 [复用] (TIMx\_CCMR2) (x=1, 8)

偏移地址: 0x01C

复位值: 0x0000 0000

相同的寄存器可以用于输出比较模式 (本节) 或输入捕获模式 (上节)。通道方向通过配置相应的 CCxS 位进行定义。此寄存器的所有其它位在输入模式和输出模式下的功能均不同。可以独立组合这两种模式 (例如, 通道 1 用作输入捕获模式而通道 2 用作输出比较模式)

|          |           |    |    |       |       |           |         |          |           |    |    |       |       |           |         |
|----------|-----------|----|----|-------|-------|-----------|---------|----------|-----------|----|----|-------|-------|-----------|---------|
| 31       | 30        | 29 | 28 | 27    | 26    | 25        | 24      | 23       | 22        | 21 | 20 | 19    | 18    | 17        | 16      |
| Reserved |           |    |    |       |       |           | OC4M[3] | Reserved |           |    |    |       |       |           | OC3M[3] |
|          |           |    |    |       |       |           | rw      |          |           |    |    |       |       |           | rw      |
| 15       | 14        | 13 | 12 | 11    | 10    | 9         | 8       | 7        | 6         | 5  | 4  | 3     | 2     | 1         | 0       |
| OC4CE    | OC4M[2:0] |    |    | OC4PE | OC4FE | CC4S[1:0] |         | OC3CE    | OC3M[2:0] |    |    | OC3PE | OC3FE | CC3S[1:0] |         |
| rw       | rw        | rw | rw | rw    | rw    | rw        | rw      | rw       | rw        | rw | rw | rw    | rw    | rw        | rw      |

**输出比较模式:**

Bit 31:25 保留, 必须保持复位值。

Bit 23:17 保留, 必须保持复位值。

Bit 15 **OC4CE**: 输出比较 4 清零使能 (Output Compare 4 clear enable)



Bit 24, 14:12 **OC4M[2:0]**: 输出比较 4 模式 (Output Compare 4 mode)

参考 OC3M[3:0]位的描述

Bit 11 **OC4PE**: 输出比较 4 预装载使能 (Output Compare 4 preload enable)

Bit 10 **OC4FE**: 输出比较 4 快速使能 (Output Compare 4 fast enable)

Bit 9:8 **CC4S[1:0]**: 捕获/比较 4 选择 (Capture/Compare 4 selection)

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00: CC4 通道配置为输出

01: CC4 通道配置为输入, tim\_ic4 映射到 tim\_ti4 上

10: CC4 通道配置为输入, tim\_ic4 映射到 tim\_ti3 上

11: CC4 通道配置为输入, tim\_ic4 映射到 tim\_trc 上。此模式仅在通过 TS 位 (TIMx\_SMCR 寄存器) 选择内部触发输入时有效

注: 仅当通道关闭时 (TIMx\_CCER 中的 CC4E = 0), 才可向 CC4S 位写入数据

Bit 7 **OC3CE**: 输出比较 3 清零使能 (Output Compare 3 clear enable)

Bit 16, 6:4 **OC3M[3:0]**: 输出比较 3 模式 (Output Compare 3 mode)

这些位定义提供 tim\_oc3 和 tim\_oc3n 的输出参考信号 tim\_oc3ref 的行为。tim\_oc3ref 为高电平有效, 而 tim\_oc3 和 tim\_oc3n 的有效电平则取决于 CC3P 位和 CC3NP 位。

0000: **冻结** - 输出比较寄存器 TIMx\_CCR3 与计数器 TIMx\_CNT 进行比较不会对输出造成任何影响。(该模式用于生成时基)。

0001: 将通道 3 设置为匹配时输出有效电平。当计数器 TIMx\_CNT 与捕获/比较寄存器 3 (TIMx\_CCR3) 匹配时, tim\_oc3ref 信号强制变为高电平。

0010: 将通道 3 设置为匹配时输出无效电平。当计数器 TIMx\_CNT 与捕获/比较寄存器 3 (TIMx\_CCR3) 匹配时, tim\_oc3ref 信号强制变为低电平。

0011: **翻转** - TIMx\_CNT=TIMx\_CCR3 时, tim\_oc3ref 发生翻转。

0100: **强制变为无效电平** - tim\_oc3ref 强制变为低电平。

0101: **强制变为有效电平** - tim\_oc3ref 强制变为高电平。

0110: **PWM 模式 1** - 在递增计数模式下, 只要 TIMx\_CNT < TIMx\_CCR3, 通道 3 便为有效状态, 否则为无效状态。在递减计数模式下, 只要 TIMx\_CNT > TIMx\_CCR3, 通道 3 便为无效状态 (tim\_oc3ref=0), 否则为有效状态 (tim\_oc3ref=1)。

0111: **PWM 模式 2** - 在递增计数模式下, 只要 TIMx\_CNT < TIMx\_CCR3, 通道 3 便为无效状态, 否则为有效状态。在递减计数模式下, 只要 TIMx\_CNT > TIMx\_CCR3, 通道 3 便为有效状态, 否则为无效状态。

1000: **可重触发 OPM 模式 1** - 在向上计数模式下, 通道在检测到触发事件 (tim\_trgi 信号) 之前一直处于活动状态。然后, 会像 PWM 模式 1 那样执行比较, 并在下次更新时使通道再次处于活动状态。在向下计数模式下, 通道在检测到触发事件 (tim\_trgi 信号) 之前一直处于非活动状态。然后, 会像 PWM 模式 1 那样执行比较, 并在下次更新时使通道再次处于非活动状态。

1001: **可重触发 OPM 模式 2** - 在向上计数模式下, 通道在检测到触发事件 (tim\_trgi 信号) 之前一直处于非活动状态。然后, 会像 PWM 模式 2 那样执行比较, 并在下次更新时使通道再次处于非活动状态。在向下计数模式下, 通道在检测到触发事件 (tim\_trgi 信号) 之前一直处于活动状态。然后, 会像 PWM 模式 1 那样执行比较, 并在下次更新时使通道再次处于活动状态。

1010: **比较脉冲输出**: 当 CCR3 匹配事件发生时, 根据 TIMx\_ECR 中的 PWPRSC[2:0] 和 PW[7:0] 位域编程, 产生 tim\_oc3ref 脉冲。

1011: **方向输出**。tim\_oc3ref 信号被 DIR 位的副本覆盖。

1100: **组合 PWM 模式 1** - tim\_oc3ref 的行为与 PWM 模式 1 相同。tim\_oc3refc 是 tim\_oc3ref 和 tim\_oc4ref 之间的逻辑 OR。

1101: **组合 PWM 模式 2** - tim\_oc3ref 的行为与 PWM 模式 2 相同。tim\_oc3refc 是 tim\_oc3ref 和 tim\_oc4ref 之间的逻辑 AND。

1110: **非对称 PWM 模式 1** - tim\_oc3ref 的行为与 PWM 模式 1 相同。当计数器向上计数时, tim\_oc3refc 输出 tim\_oc3ref; 当计数器向下计数时, tim\_oc3refc 输出 tim\_oc4ref。

1111: **非对称 PWM 模式 2** - tim\_oc3ref 的行为与 PWM 模式 2 相同。当计数器向上计数时, tim\_oc3refc 输出 tim\_oc3ref; 当计数器向下计数时, tim\_oc3refc 输出 tim\_oc4ref。

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 3 且 CC1S = "00"

(通道配置为输出), 这些位即无法修改。

注: 在 PWM 模式1 或PWM 模式2 下, 仅当比较结果发生改变或输出比较模式由“冻结”模式切换到“PWM”模式时, OCREF 电平才会发生更改。

注: 此位域将在具有互补输出的通道上进行预装载。如果TIMx\_CR2 寄存器中的CCPC 位置1, 则仅当生成COM 事件时, OC3M 有效位才会从预装载位获取新值。

Bit 3 **OC3PE**: 输出比较 3 预装载使能 (Output Compare 3 preload enable)

Bit 2 **OC3FE**: 输出比较 3 快速使能 (Output Compare 3 fast enable)

Bit 1:0 **CC3S[1:0]**: 捕获/比较 3 选择 (Capture/Compare 3 selection)

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00: CC3 通道配置为输出

01: CC3 通道配置为输入, tim\_ic3 映射到 tim\_ti3 上

10: CC3 通道配置为输入, tim\_ic3 映射到 tim\_ti4 上

11: CC3 通道配置为输入, tim\_ic3 映射到 tim\_trc 上。此模式仅在通过 TS 位 (TIMx\_SMCR 寄存器) 选择内部触发输入时有效

注: 仅当通道关闭时 (TIMx\_CCER 中的CC3E = 0), 才可向CC3S 位写入数据

### 23.6.11 TIMx 捕获/比较使能寄存器 (TIMx\_CCER) (x = 1, 8)

偏移地址: 0x020

复位值: 0x0000 0000

|          |       |      |      |       |       |      |      |       |       |      |      |          |       |      |      |
|----------|-------|------|------|-------|-------|------|------|-------|-------|------|------|----------|-------|------|------|
| 31       | 30    | 29   | 28   | 27    | 26    | 25   | 24   | 23    | 22    | 21   | 20   | 19       | 18    | 17   | 16   |
| Reserved |       |      |      |       |       |      |      |       |       | CC6P | CC6E | Reserved |       | CC5P | CC5E |
|          |       |      |      |       |       |      |      |       |       | rw   | rw   |          |       | rw   | rw   |
| 15       | 14    | 13   | 12   | 11    | 10    | 9    | 8    | 7     | 6     | 5    | 4    | 3        | 2     | 1    | 0    |
| CC4NP    | CC4NE | CC4P | CC4E | CC3NP | CC3NE | CC3P | CC3E | CC2NP | CC2NE | CC2P | CC2E | CC1NP    | CC1NE | CC1P | CC1E |
| rw       | rw    | rw   | rw   | rw    | rw    | rw   | rw   | rw    | rw    | rw   | rw   | rw       | rw    | rw   | rw   |

Bit 31:22 保留, 必须保持复位值。

Bit 21 **CC6P**: 输入/捕获 6 输出极性

参考 CC1P 的描述。

Bit 20 **CC6E**: 输入/捕获 6 输出使能

参考 CC1E 的描述。

Bit 19:18 保留, 必须保持复位值。

Bit 17 **CC5P**: 输入/捕获 5 输出极性

参考 CC1P 的描述。

Bit 16 **CC5E**: 输入/捕获 5 输出使能

参考 CC1E 的描述。

Bit 15 **CC4NP**: 输入/捕获 4 互补输出极性

参考 CC1NP 的描述。

Bit 14 **CC4NE**: 输入/捕获 4 互补输出使能

参考 CC1NE 的描述。

Bit 13 **CC4P**: 输入/捕获 4 输出极性

参考 CC1P 的描述。

Bit 12 **CC4E**: 输入/捕获 4 输出使能

参考 CC1E 的描述。

Bit 11 **CC3NP**: 输入/捕获 3 互补输出极性

参考 CC1NP 的描述。

Bit 10 **CC3NE**: 输入/捕获 3 互补输出使能

参考 CC1NE 的描述。

Bit 9 **CC3P**: 输入/捕获 3 输出极性

参考 CC1P 的描述。

Bit 8 **CC3E**: 输入/捕获 3 输出使能

参考 CC1E 的描述。

Bit 7 **CC2NP**: 输入/捕获 2 互补输出极性

参考 CC1NP 的描述。

Bit 6 **CC2NE**: 输入/捕获 2 互补输出使能

参考 CC1NE 的描述。

Bit 5 **CC2P**: 输入/捕获 2 输出极性

参考 CC1P 的描述。

Bit 4 **CC2E**: 输入/捕获 2 输出使能

参考 CC1E 的描述。

Bit 3 **CC1NP**: 输入/捕获 1 互补输出极性

**CC1 通道配置为输出:**

0: tim\_oc1n 高电平有效;

1: tim\_oc1n 低电平有效。

**CC1 通道配置为输入:**

此位与 CC1P 配合使用, 用以定义 tim\_ti1fp1 和 tim\_ti2fp1 的极性。请参见 CC1P 说明。

注: 此位将在具有互补输出的通道上进行预装载。如果 TIMx\_CR2 寄存器中的 CCPC 位置 1, 则仅当生成换向事件时, CC1NP 有效位才会从预装载位获取新值。

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 2 或 3 且 CC1S=00 (通道配置为输出), 此位立即变为不可写状态。

Bit 2 **CC1NE**: 输入/捕获 1 互补输出使能

0: 关闭—OC1N 禁止输出, 因此 OC1N 的电平依赖于 MOE、OSSI、OSSR、OIS1、OIS1N 和 CC1E 位的值。

1: 开启—OC1N 信号输出到对应的输出引脚, 其输出电平依赖于 MOE、OSSI、OSSR、OIS1、OIS1N 和 CC1E 位的值。

注: 此位将在具有互补输出的通道上进行预装载。如果 TIMx\_CR2 寄存器中的 CCPC 位置 1, 则仅当生成换向事件时, CC1NE 有效位才会从预装载位获取新值。

Bit 1 **CC1P**: 输入/捕获 1 输出极性

0: OC1 高电平有效 (输出模式) / 边沿灵敏度选择 (输入模式, 参见以下)

1: OC1 低电平有效 (输出模式) / 边沿灵敏度选择 (输入模式, 参见以下)

**CC1 通道配置为输入**, CC1NP/CC1P 位可针对触发或捕获操作选择 TI1FP1 和 TI2FP1 的极性。

CC1NP=0, CC1P=0: 非反相/上升沿触发。电路对 TIxFP1 上升沿敏感 (在复位模式、外部时钟模式或触发模式下执行捕获或触发操作), TIxFP1 未反相 (在门控模式或编码器模式下执行触发操作)。

CC1NP=0, CC1P=1: 反相/下降沿触发。电路对 TIxFP1 下降沿敏感 (在复位模式、外部时钟模式或触发模式下执行捕获或触发操作), TIxFP1 反相 (在门控模式或编码器模式下执行触发操作)。

CC1NP=1, CC1P=1: 非反相/上升沿和下降沿均触发。电路对 TIxFP1 上升沿和下降沿都敏感 (在复位模式、外部时钟模式或触发模式下执行捕获或触发操作), TIxFP1 未反相 (在门控模式下执行触发操作)。编码器模式下不得使用此配置。

CC1NP=1, CC1P=0: 此配置是保留的, 不应该使用此配置。

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 2 或 3, 此位立即变为不可写状态。

注: 此位将在具有互补输出的通道上进行预装载。如果 TIMx\_CR2 寄存器中的 CCPC 位置 1, 则仅当生成换向事件时, CC1P 有效位才会从预装载位获取新值。

Bit 0 **CC1E**: 输入/捕获 1 输出使能

0: 禁止捕获模式/OC1 未激活 (参见以下)

1: 使能捕获模式/ OC1 信号输出到对应的输出引脚

当 **CC1 通道配置为输出**, C1 的输出电平依赖于 MOE、OSSI、OSSR、OIS1、OIS1N 和 CC1NE 位的值, 不论 CC1E 位状态。请参考下表获得更多细节

注: 此位将在具有互补输出的通道上进行预装载。如果 TIMx\_CR2 寄存器中的 CCPC 位置 1, 则仅当生成换向事件时, CC1E 有效位才会从预装载位获取新值。

表 23.22 带刹车功能的互补输出通道 tim\_ocx 和 tim\_ocxn 的控制位

| 控制位      |           |           |           |            | 输出状态 <sup>(1)</sup>                                                                                                                                                                                                                                                                            |                                                   |
|----------|-----------|-----------|-----------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| MOE<br>位 | OSSI<br>位 | OSSR<br>位 | CCxE<br>位 | CCxNE<br>位 | tim_ocx 输出状态                                                                                                                                                                                                                                                                                   | tim_ocxn 输出状态                                     |
| 1        | X         | X         | 0         | 0          | 禁止输出（不由定时器驱动: Hi-Z）<br>tim_ocx=0, tim_ocxn=0                                                                                                                                                                                                                                                   |                                                   |
|          |           | 0         | 0         | 1          | 禁止输出（不由定时器驱动: Hi-Z）<br>tim_ocx=0                                                                                                                                                                                                                                                               | tim_ocxref + 极性 tim_ocxn<br>=tim_ocxref 异或 CCxNP  |
|          |           | 0         | 1         | 0          | tim_ocxref + 极性<br>tim_ocx=tim_ocxref 异或 CCxP                                                                                                                                                                                                                                                  | 禁止输出（不由定时器驱动: Hi-Z）<br>tim_ocxn=0                 |
|          |           | X         | 1         | 1          | OCREF+极性+死区                                                                                                                                                                                                                                                                                    | OCREF 互补项（而非 OCREF）<br>+极性+死区                     |
|          |           | 1         | 0         | 1          | 关闭状态（输出使能为无效状态）<br>tim_ocx=CCxP                                                                                                                                                                                                                                                                | tim_ocxref + 极性 tim_ocxn<br>=tim_ocxref x 或 CCxNP |
|          |           | 1         | 1         | 0          | tim_ocxref + 极性<br>tim_ocx=tim_ocxref 异或 CCxP                                                                                                                                                                                                                                                  | 关闭状态（输出使能为无效状态）<br>tim_ocxn=CCxNP                 |
| 0        | 0         | X         | X         | X          | 禁止输出（不由定时器驱动: Hi-Z）<br><br>关闭状态（输出使能为无效状态）<br>异步：tim_ocx=CCxP, tim_ocxn=CCxNP<br>(如果 tim_brk 或 tim_brk2 被触发)。<br>然后（仅当 tim_brk 被触发时是有效的），如果存在时钟：在死区后<br>tim_ocx=OISx 且 tim_ocxn=OISxN，从而假定 OISx 和 OISxN 在有效状态下与 OCX 和 OCxN 不对应（当在半桥配置时驱动开关，可能会导致短路）。<br><i>注：tim_brk2 只能在 OSSI=OSSR=1 时使用。</i> |                                                   |
|          | 0         |           | 0         |            |                                                                                                                                                                                                                                                                                                |                                                   |
|          | 0         |           | 1         |            |                                                                                                                                                                                                                                                                                                |                                                   |
|          | 1         |           | 0         |            |                                                                                                                                                                                                                                                                                                |                                                   |
|          | 1         |           | 1         |            |                                                                                                                                                                                                                                                                                                |                                                   |

1. 如果一个通道的两个输出均未使用 (CCxE = CCxNE = 0), 则 OISx、OISxN、CCxP 和 CCxNP 位必须保持清零状态。

注: 与互补通道 tim\_ocx 和 tim\_ocxn 相连的外部 I/O 引脚的状态取决于通道 tim\_ocx 和 tim\_ocxn 的状态以及 GPIO 寄存器。



### 23.6.12 TIMx 计数器 (TIMx\_CNT) (x = 1, 8)

偏移地址: 0x024

复位值: 0x0000 0000

|           |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-----------|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31        | 30       | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| UIF       | Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| CPY       |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| r         |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15        | 14       | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CNT[15:0] |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw        | rw       | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31 UIFCPY: UIF 副本 (UIF copy)

该位是 TIMx\_SR 寄存器中 UIF 位的一个只读副本。如果 TIMx\_CR1 上的 UIFREMAP 位被清零, 则位 31 被保留并读为 0。

Bit 30:16 保留, 必须保持复位值。

Bit 15:0 CNT[15:0]: 计数器的值 (Counter value)

在非抖动模式下 (DITHEN = 0)

该寄存器 (CNT[15:0]) 保存的是计数器的值。

在抖动模式下 (DITHEN = 1)

该寄存器 (CNT[15:0]) 只保存计数器的不抖动部分。抖动部分不可用。

### 23.6.13 TIMx 预分频器 (TIMx\_PSC) (x = 1, 8)

偏移地址: 0x028

复位值: 0x0000

|           |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 15        | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| PSC[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw        | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 15:0 PSC[15:0]: 预分频器的值 (Prescaler value)

计数器时钟频率 ( $f_{tim\_cnt\_ck}$ ) 等于  $f_{tim\_psc\_ck} / (PSC[15:0] + 1)$ 。

PSC 包含每次发生更新事件 (包括计数器通过 TIMx\_EGR 寄存器中的 UG 位清零时, 或在配置为“复位模式”时通过触发控制器清零时) 时要装载到活动预分频器寄存器的值。



### 23.6.14 TIMx 自动重载寄存器 (TIMx\_ARR) (x = 1, 8)

偏移地址: 0x02C

复位: 0x0000 FFFF

|           |    |    |    |    |    |    |    |    |    |    |    |            |    |    |    |
|-----------|----|----|----|----|----|----|----|----|----|----|----|------------|----|----|----|
| 31        | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19         | 18 | 17 | 16 |
| Reserved  |    |    |    |    |    |    |    |    |    |    |    | ARR[19:16] |    |    |    |
|           |    |    |    |    |    |    |    |    |    |    |    | rw         | rw | rw | rw |
| 15        | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3          | 2  | 1  | 0  |
| ARR[15:0] |    |    |    |    |    |    |    |    |    |    |    |            |    |    |    |
| rw        | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw         | rw | rw | rw |

Bit 31:20 保留, 必须保持复位值。

Bit 19:0 **ARR[19:0]**: 自动重载的值 (Auto-reload value)

ARR 为要装载到实际自动重载寄存器的值。

有关 ARR 更新和行为的详细信息, 请参考章节 TIMx 功能描述: 时基单元。

当自动重载值为空时, 计数器不工作。

在非抖动模式下 (DITHEN = 0)

该寄存器存储的是自动重新加载的值

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 ARR[19:4]中的整数部分, 而 ARR[3:0]位域存储的是抖动的部分

### 23.6.15 TIMx 重复计数寄存器 (TIMx\_RCR) (x = 1, 8)

偏移地址: 0x030

复位值: 0x0000

|           |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 15        | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| REP[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw        | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 15:0 **REP[15:0]**: 重复计数器的值 (Repetition counter value)

使能预装载寄存器时, 用户可通过这些位设置比较寄存器的更新频率 (即从预装载寄存器向活动寄存器周期性传输数据); 使能更新中断时, 也可设置更新中断的生成速率。

与 REP\_CNT 相关的减计数器每次计数到 0 时, 都将生成一个更新事件并且计数器从 REP 值重新开始计数。由于只有生成重复更新事件 U\_RC 时, REP\_CNT 才会重载 REP 值, 因此在生成下一重复更新事件之前, 无论向 TIMx\_RCR 寄存器写入何值都无影响。

这意味着 PWM 模式下 (REP+1) 相当于:

- 边沿对齐模式下的 PWM 周期数;
- 中心对称模式下的 PWM 半周期数;

### 23.6.16 TIMx 捕获/比较寄存器 1 (TIMx\_CCR1) (x = 1, 8)

偏移地址: 0x034

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|--------------|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19           | 18 | 17 | 16 |
| Reserved    |    |    |    |    |    |    |    |    |    |    |    | CCR1 [19:16] |    |    |    |
|             |    |    |    |    |    |    |    |    |    |    |    | rw           | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3            | 2  | 1  | 0  |
| CCR1 [15:0] |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw           | rw | rw | rw |

Bit 31:20 保留, 必须保持复位值。

Bit 19:0 **CCR1[19:0]**: 捕获/比较 1 值 (Capture/Compare 1 value)

若 **CC1 通道配置为输出**:

CCR1 为要装载到实际捕获/比较 1 寄存器的值 (预装载值)。

如果没有通过 TIMx\_CCMR1 寄存器中的 OC1PE 位来使能预装载功能, 写入的数值会被直接传输至当前寄存器中。否则只有发生更新事件时, 预装载值才会复制到活动捕获/比较 1 寄存器中。实际捕获/比较寄存器中包含要与计数器 TIMx\_CNT 进行比较并在 tim\_oc1 输出上发出信号的值。

在非抖动模式下 (DITHEN = 0)

该寄存器存储的是 CCR1[15:0]中的比较值。CCR1[19:16]位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 CCR1[19:4]中的整数部分。CCR1[3:0]位域存储的是抖动的部分。

若 **CC1 通道配置为输入**:

CCR1 为上一个输入捕获 1 事件 (tim\_ic1) 发生时的计数器值。TIMx\_CCR1 是只读的并且不可编程。

在非抖动模式下 (DITHEN = 0)

CCR1[15:0]位存储了捕获值。CCR1[19:16]位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储捕获在 CCR1[19:4]中。CCR1[3:0]位会被重置。

### 23.6.17 TIMx 捕获/比较寄存器 2 (TIMx\_CCR2) (x = 1, 8)

偏移地址: 0x038

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|--------------|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19           | 18 | 17 | 16 |
| Reserved    |    |    |    |    |    |    |    |    |    |    |    | CCR2 [19:16] |    |    |    |
|             |    |    |    |    |    |    |    |    |    |    |    | rw           | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3            | 2  | 1  | 0  |
| CCR2 [15:0] |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw           | rw | rw | rw |

Bit 31:20 保留，必须保持复位值。

Bit 19:0 **CCR2[19:0]**: 捕获/比较 2 值 (Capture/Compare 2 value)

若 **CC2 通道配置为输出**:

CCR2 为要装载到实际捕获/比较 1 寄存器的值 (预装载值)。

如果没有通过 TIMx\_CCMR1 寄存器中的 OC2PE 位来使能预装载功能，写入的数值会被直接传输至当前寄存器中。否则只有发生更新事件时，预装载值才会复制到活动捕获/比较 2 寄存器中。实际捕获/比较寄存器中包含要与计数器 TIMx\_CNT 进行比较并在 tim\_oc2 输出上发出信号的值。

在非抖动模式下 (DITHEN = 0)

该寄存器存储的是 CCR2[15:0] 中的比较值。CCR2[19:16] 位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 CCR2[19:4] 中的整数部分。CCR2[3:0] 位域存储的是抖动的部分。

若 **CC2 通道配置为输入**:

CCR2 为上一个输入捕获 2 事件 (tim\_ic2) 发生时的计数器值。TIMx\_CCR2 是只读的并且不可编程。

在非抖动模式下 (DITHEN = 0)

CCR2[15:0] 位存储了捕获值。CCR2[19:16] 位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储捕获在 CCR2[19:4] 中。CCR2[3:0] 位会被重置。

### 23.6.18 TIMx 捕获/比较寄存器 3 (TIMx\_CCR3) (x = 1, 8)

偏移地址: 0x03C

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|--------------|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19           | 18 | 17 | 16 |
| Reserved    |    |    |    |    |    |    |    |    |    |    |    | CCR3 [19:16] |    |    |    |
|             |    |    |    |    |    |    |    |    |    |    |    | rw           | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3            | 2  | 1  | 0  |
| CCR3 [15:0] |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw           | rw | rw | rw |

Bit 31:20 保留，必须保持复位值。

Bit 19:0 **CCR3[19:0]**: 捕获/比较 3 值 (Capture/Compare 3 value)

若 **CC3 通道配置为输出**:

CCR3 为要装载到实际捕获/比较 3 寄存器的值 (预装载值)。

如果没有通过 TIMx\_CCMR2 寄存器中的 OC3PE 位来使能预装载功能，写入的数值会被直接传输至当前寄存器中。否则只有发生更新事件时，预装载值才会复制到活动捕获/比较 3 寄存器中。实际捕获/比较寄存器中包含要与计数器 TIMx\_CNT 进行比较并在 tim\_oc3 输出上发出信号的值。

在非抖动模式下 (DITHEN = 0)

该寄存器存储的是 CCR3[15:0] 中的比较值。CCR3[19:16] 位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 CCR3[19:4] 中的整数部分。CCR3[3:0] 位域存储的是抖动的部分。

若 **CC3 通道配置为输入**:

CCR3 为上一个输入捕获 3 事件 (tim\_ic3) 发生时的计数器值。TIMx\_CCR3 是只读的并且不可编程。

在非抖动模式下 (DITHEN = 0)

CCR3[15:0] 位存储了捕获值。CCR3[19:16] 位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储捕获在 CCR3[19:4] 中。CCR3[3:0] 位会被重置。

### 23.6.19 TIMx 捕获/比较寄存器 4 (TIMx\_CCR4) (x = 1, 8)

偏移地址: 0x040

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|--------------|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19           | 18 | 17 | 16 |
| Reserved    |    |    |    |    |    |    |    |    |    |    |    | CCR4 [19:16] |    |    |    |
|             |    |    |    |    |    |    |    |    |    |    |    | rw           | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3            | 2  | 1  | 0  |
| CCR4 [15:0] |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw           | rw | rw | rw |

Bit 31:20 保留，必须保持复位值。

Bit 19:0 **CCR4[19:0]**: 捕获/比较 4 值 (Capture/Compare 4 value)

若 **CC4 通道配置为输出**:

CCR4 为要装载到实际捕获/比较 4 寄存器的值 (预装载值)。

如果没有通过 TIMx\_CCMR2 寄存器中的 OC4PE 位来使能预装载功能，写入的数值会被直接传输至当前寄存器中。否则只有发生更新事件时，预装载值才会复制到活动捕获/比较 4 寄存器中。实际捕获/比较寄存器中包含要与计数器 TIMx\_CNT 进行比较并在 tim\_oc4 输出上发出信号的值。

在非抖动模式下 (DITHEN = 0)

该寄存器存储的是 CCR4[15:0]中的比较值。CCR4[19:16]位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 CCR4[19:4]中的整数部分。CCR4[3:0]位域存储的是抖动的部分。

若 **CC4 通道配置为输入**:

CCR4 为上一个输入捕获 4 事件 (tim\_ic4) 发生时的计数器值。TIMx\_CCR4 是只读的并且不可编程。

在非抖动模式下 (DITHEN = 0)

CCR4[15:0]位存储了捕获值。CCR4[19:16]位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储捕获在 CCR4[19:4]中。CCR4[3:0]位会被重置。

### 23.6.20 TIMx 刹车和死区寄存器 (TIMx\_BDTR) (x = 1, 8)

偏移地址: 0x044

复位值: 0x0000 0000

| 31       | 30     | 29    | 28      | 27     | 26   | 25        | 24        | 23       | 22 | 21 | 20       | 19 | 18 | 17 | 16 |
|----------|--------|-------|---------|--------|------|-----------|-----------|----------|----|----|----------|----|----|----|----|
| Reserved | BK2BID | BKBID | BK2DSRM | BKDSRM | BK2P | BK2E      | BK2F[3:0] |          |    |    | BKF[3:0] |    |    |    |    |
|          |        |       |         |        |      |           |           |          |    |    |          |    |    |    |    |
| 15       | 14     | 13    | 12      | 11     | 10   | 9         | 8         | 7        | 6  | 5  | 4        | 3  | 2  | 1  | 0  |
| MOE      | AOE    | BKP   | BKE     | OSSR   | OSSI | LOCK[1:0] |           | DTG[7:0] |    |    |          |    |    |    |    |
| rw       | rw     | rw    | rw      | rw     | rw   | rw        | rw        | rw       | rw | rw | rw       | rw | rw | rw | rw |

注: 由于可以根据 LOCK 配置锁定位 BKBID/BK2BID/BK2P, BK2E, BK2F[3:0], BKF[3:0], AOE, BKP, BKE, OSSI, OSSR 和 DTG[7:0]的写操作, 因此必须在第一次对 TIMx\_BDTR 寄存器执行写访问时对这些位进行配置。

Bit 31:30 保留，必须保持复位值。

Bit 29 **BK2BID**: Break2 双向 (Break2 bidirectional)

请参阅 BKBID 说明

Bit 28 **BKBID**: 刹车双向 (Break bidirectional)

0: 在输入模式下刹车输入 tim\_brk

1: 在双向模式下刹车输入 tim\_brk

在双向模式 (BKBID 位设置为 1) 中，刹车输入在输入和输出中都配置模式和开漏输出模式。任何活动中断事件都会在刹车输入，向外部设备指示内部中断事件。

注：只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1，此位即无法修改。

注：对此位的任何写入操作都需要延迟 1 个 APB 时钟周期才能生效。

Bit 27 **BK2DSRM**: 刹车 2 解除 (Break2 disarm)

请参阅 BKDSRM 说明

Bit 26 **BKDSRM**: 刹车解除 (Break disarm)

0: 刹车输入 tim\_brk 已装载

1: 刹车输入 tim\_brk 被解除

当没有刹车源处于有效状态时，此位由硬件清除。

BKDSRM 位必须由软件设置，以释放双向输出控制 (高阻状态下的开漏输出)，然后对其进行轮询，直到它被硬件重置，表明故障状态已消失。

注：对此位的任何写入操作都需要延迟 1 个 APB 时钟周期才能生效。

Bit 25 **BK2P**: 刹车 2 极性 (Break 2 polarity)

0: 输入时间\_brk2 为低电平有效

1: 输入时间中断\_brk2 为高电平有效

注：只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1，此位即无法修改。

注：对此位的任何写入操作都需要延迟 1 个 APB 时钟周期才能生效。

Bit 24 **BK2E**: 刹车 2 使能 (Break 2 enable)

此位使能完全刹车 2 保护 (包括所有连接到 bk\_acth 和 BKIN 的源，参考图片：刹车和刹车 2 电路概述)。

0: Break2 功能禁用

1: 使能 Break2 功能

注：BRKIN2 必须仅在 OSSR = OSS1 = 1 的情况下使用。

注：只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1，此位即无法修改。

注：对此位的任何写入操作都需要延迟 1 个 APB 时钟周期才能生效。

Bit 23:20 **BK2F[3:0]**: 刹车 2 滤波器

这几位定义了 tim\_brk2 输入的采样频率及数字滤波器长度。数字滤波器由一个事件计数器组成，它记录到 N 个事件后会产生一个输出的跳变

0000: No filter, BRK acts asynchronously

0001:  $f_{\text{SAMPLING}} = f_{\text{tim\_ker\_ck}}$ ,  $N=2$ 。

0010:  $f_{\text{SAMPLING}} = f_{\text{tim\_ker\_ck}}$ ,  $N=4$ 。

0011:  $f_{\text{SAMPLING}} = f_{\text{tim\_ker\_ck}}$ ,  $N=8$ 。

0100:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/2$ ,  $N=6$ 。

0101:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/2$ ,  $N=8$ 。

0110:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/4$ ,  $N=6$ 。

0111:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/4$ ,  $N=8$ 。

1000:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/8$ ,  $N=6$ 。

1001:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/8$ ,  $N=8$ 。

1010:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$ ,  $N=5$ 。

1011:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$ ,  $N=6$ 。

1100:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$ ,  $N=8$ 。

1101:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$ ,  $N=5$ 。

1110:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$ ,  $N=6$ 。

1111:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$ ,  $N=8$ 。

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 1, 此位即无法修改。

Bit 19:16 **BKF[3:0]**: 刹车滤波器

这几位定义了 tim\_brk 输入的采样频率及数字滤波器长度。数字滤波器由一个事件计数器组成，它记录到 N 个事件后会产生一个输出的跳变

0000: No filter, BRK acts asynchronously

0001:  $f_{\text{SAMPLING}} = f_{\text{tim\_ker\_ck}}$ ,  $N=2$ 。

0010:  $f_{\text{SAMPLING}} = f_{\text{tim\_ker\_ck}}$ ,  $N=4$ 。

0011:  $f_{\text{SAMPLING}} = f_{\text{tim\_ker\_ck}}$ ,  $N=8$ 。

0100:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/2$ ,  $N=6$ 。

0101:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/2$ ,  $N=8$ 。

0110:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/4$ ,  $N=6$ 。

0111:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/4$ ,  $N=8$ 。

1000:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/8$ ,  $N=6$ 。

1001:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/8$ ,  $N=8$ 。

1010:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$ ,  $N=5$ 。

1011:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$ ,  $N=6$ 。

1100:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$ ,  $N=8$ 。

1101:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$ ,  $N=5$ 。

1110:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$ ,  $N=6$ 。

1111:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$ ,  $N=8$ 。

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 1, 此位即无法修改。



**Bit 15 MOE: 主输出使能 (Main output enable)**

只要刹车输入变为有效状态, 此位便由硬件异步清零。此位由软件置 1, 也可根据 AOE 位状态自动置 1。此位仅对配置为输出的通道有效。

0: OC 和 OCN 输出禁止或被强制为空闲状态。

1: 如果 OC 和 OCN 输出的相应使能位 (TIMx\_CCER 寄存器中的 CCxE 和 CCxNE 位) 均置 1, 则使能 OC 和 OCN 输出。

有关详细信息, 请参见 OC/OCN 使能说明 (TIMx 捕获/比较使能寄存器 (TIMx\_CCER) (x=1, 8))。

**Bit 14 AOE: 自动输出使能 (Automatic output enable)**

0: MOE 只能由软件置 1

1: MOE 可由软件置 1, 也可在发生下一更新事件时自动置 1(如果刹车输入无效)

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

**Bit 13 BKP: 刹车极性 (Break polarity)**

0: 刹车输入低电平有效;

1: 刹车输入高电平有效。

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

注: 对该位执行任何写操作后, 都需要经过1 个 APB 时钟周期的延迟才生效。

**Bit 12 BKE: 刹车使能 (Break enable)**

0: 禁止刹车输入 (BRK 及 CCS 时钟失效事件);

1: 开启刹车输入 (BRK 及 CCS 时钟失效事件)。

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

注: 对该位执行任何写操作后, 都需要经过1 个 APB 时钟周期的延迟才生效。

**Bit 11 OSSR: 运行模式下“关闭状态”选择 (Off-state selection for Run mode)**

此位在 MOE=1 时作用于配置为输出模式且具有互补输出的通道。如果定时器中没有互补输出, 则不存在 OSSR。

有关详细信息, 请参见 OC/OCN 使能说明 (TIMx 捕获/比较使能寄存器 (TIMx\_CCER) (x=1, 8))。

0: 处于无效状态时, 禁止 OC/OCN 输出 (OC/OCN 使能输出信号=0)。

1: 处于无效状态时, 一旦 CCxE=1 或 CCxNE=1, 便使能 OC/OCN 输出并将其设为无效电平。然后设置 OC/OCN 使能输出信号=1

注: 编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别2 后, 此位即无法修改。

**Bit 10 OSSI: 空闲模式下的关闭状态选择 (Off-state selection for Idle mode)**

此位在 MOE=0 时作用于配置为输出的通道。

有关详细信息, 请参见 OC/OCN 使能说明 (TIMx 捕获/比较使能寄存器 (TIMx\_CCER) (x=1, 8))。

0: 处于无效状态时, 禁止 OC/OCN 输出 (OC/OCN 使能输出信号=0)。

1: 处于无效状态时, 一旦 CCxE=1 或 CCxNE=1, 便将 OC/OCN 输出首先强制为其空闲电平。然后设置 OC/OCN 使能输出信号=1

注: 编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别2 后, 此位即无法修改。

**Bit 9:8 LOCK[1:0]: 锁定设置 (Lock configuration)**

这些位用于针对软件错误提供写保护。

00: 关闭锁定--不对任何位提供写保护。

01: 锁定级别 1, 此时无法对 TIMx\_BDTR 寄存器中的 DTG 位、TIMx\_CR2 寄存器中的 OISx 和 OISxN 位以及 TIMx\_BDTR 寄存器中的 BKE/BKP/AOE 位执行写操作。

10: 锁定级别 2, 此时无法对锁定级别 1 中适用的各位、CC 极性位 (TIMx\_CCER 寄存器中的 CCxP/CCxNP 位, 只要通过 CCxS 位将相关通道配置为输出) 以及 OSSR 和 OSSR 位执行写操作。

11: 锁定级别 3, 此时无法对锁定级别 2 中适用的各位、CC 控制位 (TIMx\_CCMRx 寄存器中的 OCxM 和 OCxPE 位, 只要通过 CCxS 位将相关通道配置为输出) 执行写操作。

注: 复位后只能对 LOCK 位执行一次写操作。对 TIMx\_BDTR 寄存器执行写操作后其中的内容将冻结, 直到下一次复位。

**Bit 7:0 DTG[7:0]: 配置死区发生器 (Dead-time generator setup)**

此位域定义插入到互补输出之间的死区持续时间。DT 与该持续时间相对应。

DTG[7:5]=0xx =>  $DT = DTG[7:0] \times T_{dtg}$ , 其中  $T_{dtg} = T_{DTS}$ ;

DTG[7:5]=10x =>  $DT = (64 + DTG[5:0]) \times T_{dtg}$ , 其中  $T_{dtg} = 2 \times T_{DTS}$ ;

DTG[7:5]=110 =>  $DT = (32 + DTG[4:0]) \times T_{dtg}$ , 其中  $T_{dtg} = 8 \times T_{DTS}$ ;

DTG[7:5]=111 =>  $DT = (32 + DTG[4:0]) \times T_{dtg}$ , 其中  $T_{dtg} = 16 \times T_{DTS}$ ;

例: 若  $T_{DTS} = 125\text{ns}$  (8MHZ), 可能的死区时间为:

0 到 15875 ns (步长为 125 ns),

16 us 到 31750 ns (步长为 250 ns),

32 us 到 63us (步长为 1 us),

64 us 到 126 us (步长为 2 us)

注: 编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1、2 或3, 此位即无法修改。

### 23.6.21 TIMx 捕获/比较寄存器 5 (TIMx\_CCR5) (x = 1, 8)

偏移地址: 0x048

复位值: 0x0000 0000

|             |       |       |          |    |    |    |    |    |    |    |    |              |    |    |    |
|-------------|-------|-------|----------|----|----|----|----|----|----|----|----|--------------|----|----|----|
| 31          | 30    | 29    | 28       | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19           | 18 | 17 | 16 |
| GC5C3       | GC5C2 | GC5C1 | Reserved |    |    |    |    |    |    |    |    | CCR5 [19:16] |    |    |    |
| rw          | rw    | rw    |          |    |    |    |    |    |    |    |    | rw           | rw | rw | rw |
| 15          | 14    | 13    | 12       | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3            | 2  | 1  | 0  |
| CCR5 [15:0] |       |       |          |    |    |    |    |    |    |    |    |              |    |    |    |
| rw          | rw    | rw    | rw       | rw | rw | rw | rw | rw | rw | rw | rw | rw           | rw | rw | rw |

Bit 31 **GC5C3**: 通道 5 和通道 3 组 (Group channel 5 and channel 3)

通道 3 输出失真:

0: tim\_oc5ref 对 tim\_oc3refc 无影响

1: tim\_oc3refc 是 tim\_oc3ref 和 tim\_oc5ref 的逻辑 AND 操作

此位要么立即生效, 要么被预加载并在更新事件后被考虑 (如果 TIMx\_CCMR2 中选择了预加载功能)。

注: 也可以在组合 PWM 信号上应用这种失真。

Bit 30 **GC5C2**: 通道 5 和通道 2 组 (Group channel 5 and channel 2)

通道 2 输出失真:

0: tim\_oc5ref 对 tim\_oc2refc 无影响

1: tim\_oc2refc 是 tim\_oc2ref 和 tim\_oc5ref 的逻辑 AND 操作

此位要么立即生效, 要么被预加载并在更新事件后被考虑 (如果 TIMx\_CCMR1 中选择了预加载功能)。

注: 也可以在组合 PWM 信号上应用这种失真。

Bit 29 **GC5C1**: 通道 5 和通道 1 组 (Group channel 5 and channel 1)

通道 1 输出失真:

0: tim\_oc5ref 对 tim\_oc1refc 无影响

1: tim\_oc1refc 是 tim\_oc1ref 和 tim\_oc5ref 的逻辑 AND 操作

此位要么立即生效, 要么被预加载并在更新事件后被考虑 (如果 TIMx\_CCMR1 中选择了预加载功能)。

注: 也可以在组合 PWM 信号上应用这种失真。

Bit 28:20 保留, 必须保持复位值。

Bit 19:0 **CCR5[15:0]**: 捕获/比较 5 值 (Capture/Compare 5 value)

CCR5 为要装载到实际捕获/比较 5 寄存器的值 (预装载值)。

如果没有通过 TIMx\_CCMR3 寄存器中的 OC5PE 位来使能预装载功能, 写入的数值会被直接传输至当前寄存器中。否则只有发生更新事件时, 预装载值才会复制到活动捕获/比较 5 寄存器中。实际捕获/比较寄存器中包含要与计数器 TIMx\_CNT 进行比较并在 tim\_oc5 输出上发出信号的值。

在非抖动模式下 (DITHEN = 0)

该寄存器存储的是 CCR5[15:0]中的比较值。CCR5[19:16]位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 CCR5[19:4]中的整数部分。CCR5[3:0]位域存储的是抖动的部分。

### 23.6.22 TIMx 捕获/比较寄存器 6 (TIMx\_CCR6) (x = 1, 8)

偏移地址: 0x04C

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|--------------|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19           | 18 | 17 | 16 |
| Reserved    |    |    |    |    |    |    |    |    |    |    |    | CCR6 [19:16] |    |    |    |
|             |    |    |    |    |    |    |    |    |    |    |    | rw           | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3            | 2  | 1  | 0  |
| CCR6 [15:0] |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw           | rw | rw | rw |

Bit 31:20 保留, 必须保持复位值。

Bit 19:0 **CCR6[15:0]**: 捕获/比较 6 值 (Capture/Compare 6 value)

CCR6 为要装载到实际捕获/比较 6 寄存器的值 (预装载值)。

如果没有通过 TIMx\_CCMR3 寄存器中的 OC6PE 位来使能预装载功能, 写入的数值会被直接传输至当前寄存器中。否则只有发生更新事件时, 预装载值才会复制到活动捕获/比较 6 寄存器中。实际捕获/比较寄存器中包含要与计数器 TIMx\_CNT 进行比较并在 tim\_oc6 输出上发出信号的值。

在非抖动模式下 (DITHEN = 0)

该寄存器存储的是 CCR6[15:0]中的比较值。CCR6[19:16]位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 CCR6[19:4]中的整数部分。CCR6[3:0]位域存储的是抖动的部分。

### 23.6.23 TIMx 捕获/比较模式寄存器 3 (TIMx\_CCMR3) (x = 1, 8)

偏移地址: 0x050

复位值: 0x0000 0000

参看以上 CCMR1 寄存器的描述, 通道 5 和 6 只能配置为输出。

|          |           |    |    |       |       |          |         |          |           |    |    |       |       |          |         |
|----------|-----------|----|----|-------|-------|----------|---------|----------|-----------|----|----|-------|-------|----------|---------|
| 31       | 30        | 29 | 28 | 27    | 26    | 25       | 24      | 23       | 22        | 21 | 20 | 19    | 18    | 17       | 16      |
| Reserved |           |    |    |       |       |          | OC6M[3] | Reserved |           |    |    |       |       |          | OC5M[3] |
|          |           |    |    |       |       |          | rw      |          |           |    |    |       |       |          | rw      |
| 15       | 14        | 13 | 12 | 11    | 10    | 9        | 8       | 7        | 6         | 5  | 4  | 3     | 2     | 1        | 0       |
| OC6CE    | OC6M[2:0] |    |    | OC6PE | OC6FE | Reserved |         | OC5CE    | OC5M[2:0] |    |    | OC5PE | OC5FE | Reserved |         |
| rw       | rw        | rw | rw | rw    | rw    |          |         | rw       | rw        | rw | rw | rw    | rw    |          |         |

Bit 31:25 保留, 必须保持复位值。

Bit 23:17 保留, 必须保持复位值。

Bit 15 **OC6CE**: 输出比较 6 清零使能 (Output Compare 6 clear enable)

Bit 24, 14:12 **OC6M[2:0]**: 输出比较 6 模式 (Output Compare 6 mode)

Bit 11 **OC6PE**: 输出比较 6 预装载使能 (Output Compare 6 preload enable)

Bit 10 **OC6FE**: 输出比较 6 快速使能 (Output Compare 6 fast enable)

Bit 9:8 保留, 必须保持复位值。

Bit 7 **OC5CE**: 输出比较 5 清零使能 (Output Compare 5 clear enable)

Bit 16, 6:4 **OC5M[2:0]**: 输出比较 5 模式 (Output Compare 5 mode)

Bit 3 **OC5PE**: 输出比较 5 预装载使能 (Output Compare 5 preload enable)

Bit 2 **OC5FE**: 输出比较 5 快速使能 (Output Compare 5 fast enable)

Bit 1:0 保留, 必须保持复位值。

### 23.6.24 TIMx 定时器死区寄存器 2 (TIMx\_DTR2) (x = 1, 8)

偏移地址: 0x054

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |           |    |    |    |    |    |      |      |
|----------|----|----|----|----|----|----|----|-----------|----|----|----|----|----|------|------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23        | 22 | 21 | 20 | 19 | 18 | 17   | 16   |
| Reserved |    |    |    |    |    |    |    |           |    |    |    |    |    | DTPE | DTAE |
|          |    |    |    |    |    |    |    |           |    |    |    |    |    | rw   | rw   |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7         | 6  | 5  | 4  | 3  | 2  | 1    | 0    |
| Reserved |    |    |    |    |    |    |    | DTGF[7:0] |    |    |    |    |    |      |      |
|          |    |    |    |    |    |    |    | rw        | rw | rw | rw | rw | rw | rw   | rw   |

Bit 31:18 保留, 必须保持复位值。

Bit 17 **DTPE**: 死区时间预加载使能 (Deadtime preload enable)

0: 死区时间值未预加载

1: 使能死区时间值预加载

注: 编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 1、2 或 3, 此位即无法修改。

Bit 16 **DTAE**: 死区时间非对称使能 (Deadtime asymmetric enable)

0: 上升沿和下降沿的死区时间是相同的, 并且使用 DTG[7:0]寄存器定义

1: 上升沿的死区时间使用 DTG[7:0]寄存器定义, 下降沿的死区时间使用 DTGF[7:0]位定义。

注: 编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 1、2 或 3, 此位即无法修改。

Bit 15:8 保留, 必须保持复位值。

Bit 7:0 **DTGF[7:0]**: 死区下降沿发生器 (Dead-time falling edge generator setup)

这个位域定义了在下落沿时, 互补输出之间插入死区时间的长度。

$DTG[7:5]=0xx \Rightarrow DT = DTG[7:0] \times T_{dtg}$ , 其中  $T_{dtg} = T_{DTS}$ ;

$DTG[7:5]=10x \Rightarrow DT = (64+DTG[5:0]) \times T_{dtg}$ , 其中  $T_{dtg} = 2 \times T_{DTS}$ ;

$DTG[7:5]=110 \Rightarrow DT = (32+DTG[4:0]) \times T_{dtg}$ , 其中  $T_{dtg} = 8 \times T_{DTS}$ ;

$DTG[7:5]=111 \Rightarrow DT = (32+DTG[4:0]) \times T_{dtg}$ , 其中  $T_{dtg} = 16 \times T_{DTS}$ ;

例: 若  $T_{DTS} = 125ns$  (8MHZ), 可能的死区时间为:

0 到 15875 ns (步长为 125 ns),

16 us 到 31750 ns (步长为 250 ns),

32 us 到 63us (步长为 1 us),

64 us 到 126 us (步长为 2 us)

注: 编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 1、2 或 3, 此位即无法修改。

### 23.6.25 TIMx 定时器编码器控制寄存器 (TIMx\_ECR) (x = 1, 8)

偏移地址: 0x058

复位值: 0x0000 0000

|          |    |    |    |    |             |    |    |           |    |      |          |    |           |    |    |
|----------|----|----|----|----|-------------|----|----|-----------|----|------|----------|----|-----------|----|----|
| 31       | 30 | 29 | 28 | 27 | 26          | 25 | 24 | 23        | 22 | 21   | 20       | 19 | 18        | 17 | 16 |
| Reserved |    |    |    |    | PWPRSC[2:0] |    |    | PW[7:0]   |    |      |          |    |           |    |    |
|          |    |    |    |    | rw          | rw | Rw | rw        | rw | rw   | rw       | rw | rw        | rw | rw |
| 15       | 14 | 13 | 12 | 11 | 10          | 9  | 8  | 7         | 6  | 5    | 4        | 3  | 2         | 1  | 0  |
| Reserved |    |    |    |    |             |    |    | IPOS[1:0] |    | FIDX | Reserved |    | IDIR[1:0] |    | IE |
|          |    |    |    |    |             |    |    | rw        | rw | rw   |          |    | rw        | rw | rw |

Bit 31:27 保留, 必须保持复位值。

Bit 26:24 **PWPRSC[2:0]**: 脉冲宽度预分频器 (Pulse width prescaler)

这个位域为脉冲发生器设置时钟预分频器, 具体如下:

$$t_{PWG} = (2^{PWPRSC[2:0]}) \times t_{tim\_ker\_ck}$$

Bit 23:16 **PW[7:0]**: 脉冲宽度 (Pulse width)

这个位域定义脉冲的持续时间, 具体如下:

$$t_{PW} = PW[7:0] \times t_{PWG}$$

Bit 15:8 保留, 必须保持复位值。

Bit 7:6 **IPOS[1:0]**: 索引位置 (Index positioning)

在正交编码器模式 (SMS[3:0] = 0001, 0010, 0011, 1110, 1111) 下, 该位指示了索引事件在哪个 AB 输入配置下会重置计数器。

00: 当 AB=00 时, 索引重置计数器

01: 当 AB=01 时, 索引重置计数器

10: 当 AB=10 时, 索引重置计数器

11: 当 AB=11 时, 索引重置计数器

在方向时钟模式或时钟加方向模式 (SMS[3:0] = 1010, 1011, 1100, 1101) 下, 此位指示了索引事件在哪个时钟水平上重置计数器。在双向时钟模式下, 这适用于两个时钟输入。具体为:

x0: 当时钟=0 时, 索引重置计数器

x1: 当时钟=1 时, 索引重置计数器

注: IPOS[1] 位不重要

Bit 5 **FIDX[1:0]**: 首位索引 (First index)

该位指示是否仅考虑首位索引

0: 索引始终处于活跃状态

1: 仅首个索引会重置计数器

Bit 4:3 保留, 必须保持复位值。

Bit 2:1 **IDIR[1:0]**: 索引方向 (Index direction)

该位指示索引事件在哪个方向上重置计数器。

00: 无论方向如何, 索引都会重置计数器

01: 仅在递增计数时, 索引会重置计数器

10: 仅在递减计数时, 索引会重置计数器

11: 保留

注: 当 IE 位被复位 (禁止索引) 时, 必须写入 IDR[1:0] 位域

Bit 0 **IE**: 索引使能 (Index enable)



该位指示索引事件是否能重置计数器

0: 禁止索引

1: 使能索引

### 23.6.26 TIMx 定时器输入选择寄存器 (TIMx\_TISEL) (x=1, 8)

偏移地址: 0x05C

复位值: 0x0000 0000

|          |    |    |    |             |    |    |    |          |    |    |    |             |    |    |    |
|----------|----|----|----|-------------|----|----|----|----------|----|----|----|-------------|----|----|----|
| 31       | 30 | 29 | 28 | 27          | 26 | 25 | 24 | 23       | 22 | 21 | 20 | 19          | 18 | 17 | 16 |
| Reserved |    |    |    | TI4SEL[3:0] |    |    |    | Reserved |    |    |    | TI3SEL[3:0] |    |    |    |
|          |    |    |    | rw          | rw | rw | rw |          |    |    |    | rw          | rw | rw | rw |
| 15       | 14 | 13 | 12 | 11          | 10 | 9  | 8  | 7        | 6  | 5  | 4  | 3           | 2  | 1  | 0  |
| Reserved |    |    |    | TI2SEL[3:0] |    |    |    | Reserved |    |    |    | TI1SEL[3:0] |    |    |    |
|          |    |    |    | rw          | rw | rw | rw |          |    |    |    | rw          | rw | rw | rw |

Bit 31:28 保留, 必须保持复位值。

Bit 27:24 **TI4SEL[3:0]**: 选择 tim\_ti4[15:0]输入 (Selects tim\_ti4[15:0] input)

0000: tim\_ti4\_in0: TIMx\_CH4

0001: tim\_ti4\_in1

...

1111: tim\_ti4\_in15

参考章节: TIM1/TIM8 引脚和内部信号获得关联列表。

Bit 23:20 保留, 必须保持复位值。

Bit 19:16 **TI3SEL[3:0]**: 选择 tim\_ti3[15:0]输入 (Selects tim\_ti3[15:0] input)

0000: tim\_ti3\_in0: TIMx\_CH3

0001: tim\_ti3\_in1

...

1111: tim\_ti3\_in15

参考章节: TIM1/TIM8 引脚和内部信号获得关联列表。

Bit 15:12 保留, 必须保持复位值。

Bit 11:8 **TI2SEL[3:0]**: 选择 tim\_ti2[15:0]输入 (Selects tim\_ti2[15:0] input)

0000: tim\_ti2\_in0: TIMx\_CH2

0001: tim\_ti2\_in1

...

1111: tim\_ti2\_in15

参考章节: TIM1/TIM8 引脚和内部信号获得关联列表。

Bit 7:4 保留, 必须保持复位值。

Bit 3:0 **TI1SEL[3:0]**: 选择 tim\_ti1[15:0]输入 (Selects tim\_ti1[15:0] input)

0000: tim\_ti1\_in0: TIMx\_CH1

0001: tim\_ti1\_in1

...

1111: tim\_ti1\_in15

参考章节: TIM1/TIM8 引脚和内部信号获得关联列表。



### 23.6.27 TIMx 复用功能选择寄存器 1 (TIMx\_AF1) (x = 1, 8)

偏移地址: 0x060

复位值: 0x0000 0001

| 31          | 30 | 29          | 28          | 27          | 26          | 25    | 24       | 23 | 22 | 21 | 20          | 19          | 18          | 17          | 16    |
|-------------|----|-------------|-------------|-------------|-------------|-------|----------|----|----|----|-------------|-------------|-------------|-------------|-------|
| Reserved    |    |             |             |             |             |       |          |    |    |    |             |             |             | ETRSEL[3:2] |       |
|             |    |             |             |             |             |       |          |    |    |    |             |             |             | rw          | rw    |
| 15          | 14 | 13          | 12          | 11          | 10          | 9     | 8        | 7  | 6  | 5  | 4           | 3           | 2           | 1           | 0     |
| ETRSEL[1:0] |    | BK<br>CMP4P | BK<br>CMP3P | BK<br>CMP2P | BK<br>CMP1P | BKINP | Reserved |    |    |    | BK<br>CMP4E | BK<br>CMP3E | BK<br>CMP2E | BK<br>CMP1E | BKINE |
| rw          | rw | rw          | rw          | rw          | rw          | rw    |          |    |    |    | rw          | rw          | rw          | rw          | rw    |

Bit 31:18 保留, 必须保持复位值。

Bit 17:14 **ETRSEL[3:0]**: etr\_in 源选择 (etr\_in source selection)

这些位选择 etr\_in 输入源。

0000: tim\_etr0: TIMx\_ETR 输入

0001: tim\_etr1

...

1111: tim\_etr15

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 13 **BKCMP4P**: tim\_brk\_cmp4 输入极性 (tim\_brk\_cmp4 input polarity)

此位选择 tim\_brk\_cmp4 输入敏感度。它必须与 BKP 极性位一起编程。

0: tim\_brk\_cmp4 输入极性未反转 (如果 BKP=0, 则低电平有效, 如果 BKP=1, 则高电平有效)

1: tim\_brk\_cmp4 输入极性反转 (如果 BKP=0, 则高电平有效, 如果 BKP=1, 则低电平有效)

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 12 **BKCMP3P**: tim\_brk\_cmp3 输入极性 (tim\_brk\_cmp3 input polarity)

此位选择 tim\_brk\_cmp3 输入敏感度。它必须与 BKP 极性位一起编程。

0: tim\_brk\_cmp3 输入极性未反转 (如果 BKP=0, 则低电平有效, 如果 BKP=1, 则高电平有效)

1: tim\_brk\_cmp3 输入极性反转 (如果 BKP=0, 则高电平有效, 如果 BKP=1, 则低电平有效)

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 11 **BKCMP2P**: tim\_brk\_cmp2 输入极性 (tim\_brk\_cmp2 input polarity)

此位选择 tim\_brk\_cmp2 输入敏感度。它必须与 BKP 极性位一起编程。

0: tim\_brk\_cmp2 输入极性未反转 (如果 BKP=0, 则低电平有效, 如果 BKP=1, 则高电平有效)

1: tim\_brk\_cmp2 输入极性反转 (如果 BKP=0, 则高电平有效, 如果 BKP=1, 则低电平有效)

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 10 **BKCMP1P**: tim\_brk\_cmp1 输入极性 (tim\_brk\_cmp1 input polarity)

此位选择 tim\_brk\_cmp1 输入敏感度。它必须与 BKP 极性位一起编程。

0: tim\_brk\_cmp1 输入极性未反转 (如果 BKP=0, 则低电平有效, 如果 BKP=1, 则高电平有效)

1: tim\_brk\_cmp1 输入极性反转 (如果 BKP=0, 则高电平有效, 如果 BKP=1, 则低电平有效)

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 9 **BKINP**: TIMx\_BKIN 输入极性 (TIMx\_BKIN input polarity)

该位选择 TIMx\_BKIN 备用功能输入敏感度。必须对其和 BKP 极性位进行编程。

0: TIMx\_BKIN 输入极性未反转 (如果 BKP=0, 则低电平有效, 如果 BKP=1, 则高电平有效)

1: TIMx\_BKIN 输入极性反转 (如果 BKP=0, 则高电平有效, 如果 BKP=1, 则低电平有效)

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 8:5 保留, 必须保持复位值。

Bit 4 **BKCMP4E**: tim\_brk\_cmp4 使能 (tim\_brk\_cmp4 enable)

该位用于使能 tim\_brk 输入的 tim\_brk\_cmp4。tim\_brk\_cmp4 输出与其他 tim\_brk 源进行“或”运算。

0: tim\_brk\_cmp4 输入禁用

1: tim\_brk\_cmp4 输入使能

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 3 **BKCMP3E**: tim\_brk\_cmp3 使能 (tim\_brk\_cmp3 enable)

该位用于使能 tim\_brk 输入的 tim\_brk\_cmp3。tim\_brk\_cmp3 输出与其他 tim\_brk 源进行“或”运算。

0: tim\_brk\_cmp3 输入禁用

1: tim\_brk\_cmp3 输入使能

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 2 **BKCMP2E**: tim\_brk\_cmp2 使能 (tim\_brk\_cmp2 enable)

该位用于使能 tim\_brk 输入的 tim\_brk\_cmp2。tim\_brk\_cmp2 输出与其他 tim\_brk 源进行“或”运算。

0: tim\_brk\_cmp2 输入禁用

1: tim\_brk\_cmp2 输入使能

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 1 **BKCMP1E**: tim\_brk\_cmp1 使能 (tim\_brk\_cmp1 enable)

该位用于使能 tim\_brk 输入的 tim\_brk\_cmp1。tim\_brk\_cmp1 输出与其他 tim\_brk 源进行“或”运算。

0: tim\_brk\_cmp1 输入禁用

1: tim\_brk\_cmp1 输入使能

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 0 **BKINE**: TIMx\_BKIN 输入使能 (TIMx\_BKIN input enable)

该位用于使能 TIMx\_BKIN 复用功能输入作为定时器的 tim\_brk 输入。TIMx\_BKIN 输入与其他 tim\_brk 源进行“或”运算。

0: TIMx\_BKIN 输入禁用

1: TIMx\_BKIN 输入使能

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

注意: 有关产品特定实现的信息, 请参考章节: TIM1/TIM8 引脚和内部信号。

### 23.6.28 TIMx 复用功能选择寄存器 2 (TIMx\_AF2) (x = 1, 8)

偏移地址: 0x064

复位值: 0x0000 0001

| 31       | 30           | 29           | 28           | 27           | 26     | 25       | 24 | 23 | 22 | 21 | 20           | 19           | 18           | 17           | 16     |
|----------|--------------|--------------|--------------|--------------|--------|----------|----|----|----|----|--------------|--------------|--------------|--------------|--------|
| Reserved |              |              |              |              |        |          |    |    |    |    |              |              | OCRSEL[2:0]  |              |        |
|          |              |              |              |              |        |          |    |    |    |    |              |              | rw           | rw           | rw     |
| 15       | 14           | 13           | 12           | 11           | 10     | 9        | 8  | 7  | 6  | 5  | 4            | 3            | 2            | 1            | 0      |
| Reserved | BK2<br>CMP4P | BK2<br>CMP3P | BK2<br>CMP2P | BK2<br>CMP1P | BK2INP | Reserved |    |    |    |    | BK2<br>CMP4E | BK2<br>CMP3E | BK2<br>CMP2E | BK2<br>CMP1E | BK2INE |
|          |              |              |              |              |        |          |    |    |    |    |              |              |              |              |        |
|          | rw           | rw           | rw           | rw           | rw     |          |    |    |    |    | rw           | rw           | rw           | rw           | rw     |

Bit 31:19 保留, 必须保持复位值。

Bit 18:16 **OCRSEL[2:0]**: ocref\_clr 源选择 (ocref\_clr source selection)

这些位选择 ocref\_clr 输入源。

000: tim\_ocref\_clr0

001: tim\_ocref\_clr1

...

111: tim\_ocref\_clr7

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 15:14 保留, 必须保持复位值。

Bit 13 **BK2CMP4P**: tim\_brk2\_cmp4 输入极性 (tim\_brk2\_cmp4 input polarity)

此位选择 tim\_brk2\_cmp4 输入敏感度。它必须与 BK2P 极性位一起编程。

0: tim\_brk2\_cmp4 输入极性未反转 (如果 BK2P=0, 则低电平有效, 如果 BK2P=1, 则高电平有效)

1: tim\_brk2\_cmp4 输入极性反转 (如果 BK2P=0, 则高电平有效, 如果 BK2P=1, 则低电平有效)

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 12 **BK2CMP3P**: tim\_brk2\_cmp3 输入极性 (tim\_brk2\_cmp3 input polarity)

此位选择 tim\_brk2\_cmp3 输入敏感度。它必须与 BK2P 极性位一起编程。

0: tim\_brk2\_cmp3 输入极性未反转 (如果 BK2P=0, 则低电平有效, 如果 BK2P=1, 则高电平有效)

1: tim\_brk2\_cmp3 输入极性反转 (如果 BK2P=0, 则高电平有效, 如果 BK2P=1, 则低电平有效)

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 11 **BK2CMP2P**: tim\_brk2\_cmp2 输入极性 (tim\_brk2\_cmp2 input polarity)

此位选择 tim\_brk2\_cmp2 输入敏感度。它必须与 BK2P 极性位一起编程。

0: tim\_brk2\_cmp2 输入极性未反转 (如果 BK2P=0, 则低电平有效, 如果 BK2P=1, 则高电平有效)

1: tim\_brk2\_cmp2 输入极性反转 (如果 BK2P=0, 则高电平有效, 如果 BK2P=1, 则低电平有效)

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 10 **BK2CMP1P**: tim\_brk2\_cmp1 输入极性 (tim\_brk2\_cmp1 input polarity)

此位选择 tim\_brk2\_cmp1 输入敏感度。它必须与 BK2P 极性位一起编程。

0: tim\_brk2\_cmp1 输入极性未反转 (如果 BK2P=0, 则低电平有效, 如果 BK2P=1,

则高电平有效)

- 1: tim\_brk2\_cmp1 输入极性反转 (如果 BK2P=0, 则高电平有效, 如果 BK2P=1, 则低电平有效)

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 9 **BK2INP**: TIMx\_BKIN2 输入极性 (TIMx\_BKIN2 input polarity)

该位选择 TIMx\_BKIN2 复用功能输入敏感度。它必须与 BK2P 极性位一起编程。

- 0: TIMx\_BKIN2 输入极性未反转 (如果 BK2P=0, 则低电平有效, 如果 BK2P=1, 则高电平有效)

- 1: TIMx\_BKIN2 输入极性反转 (如果 BK2P=0, 则高电平有效, 如果 BK2P=1, 则低电平有效)

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 8:5 保留, 必须保持复位值。

Bit 4 **BK2CMP4E**: tim\_brk2\_cmp4 使能 (tim\_brk2\_cmp4 enable)

该位用于使能 tim\_brk2 输入的 tim\_brk2\_cmp4。tim\_brk2\_cmp4 输出与其他 tim\_brk2 源进行“或”运算。

- 0: tim\_brk2\_cmp4 输入禁用

- 1: tim\_brk2\_cmp4 输入使能

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 3 **BK2CMP3E**: tim\_brk2\_cmp3 使能 (tim\_brk2\_cmp3 enable)

该位用于使能 tim\_brk2 输入的 tim\_brk2\_cmp3。tim\_brk2\_cmp3 输出与其他 tim\_brk2 源进行“或”运算。

- 0: tim\_brk2\_cmp3 输入禁用

- 1: tim\_brk2\_cmp3 输入使能

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 2 **BK2CMP2E**: tim\_brk2\_cmp2 使能 (tim\_brk2\_cmp2 enable)

该位用于使能 tim\_brk2 输入的 tim\_brk2\_cmp2。tim\_brk2\_cmp2 输出与其他 tim\_brk2 源进行“或”运算。

- 0: tim\_brk2\_cmp2 输入禁用

- 1: tim\_brk2\_cmp2 输入使能

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 1 **BK2CMP1E**: tim\_brk2\_cmp1 使能 (tim\_brk2\_cmp1 enable)

该位用于使能 tim\_brk2 输入的 tim\_brk2\_cmp1。tim\_brk2\_cmp1 输出与其他 tim\_brk2 源进行“或”运算。

- 0: tim\_brk2\_cmp1 输入禁用

- 1: tim\_brk2\_cmp1 输入使能

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 0 **BK2INE**: TIMx\_BKIN2 输入使能 (TIMx\_BKIN2 input enable)

该位用于使能 TIMx\_BKIN2 复用功能输入作为定时器的 tim\_brk2 输入。TIMx\_BKIN2 输入与其他 tim\_brk2 源进行“或”运算。

- 0: TIMx\_BKIN2 输入禁用

- 1: TIMx\_BKIN2 输入使能

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

注意: 有关产品特定实现的信息, 请参考章节: TIM1/TIM8 引脚和内部信号。

### 23.6.29 TIMx 捕获/比较寄存器 1N (TIMx\_CCR1N) (x = 1, 8)

偏移地址: 0x068

复位值: 0x0000 0000

| 31          | 30       | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|-------------|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| EN          | Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15          | 14       | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CCR1N[15:0] |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw       | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31 EN: 若 CC1 通道配置为输出:

CCR1N\_EN=1: CNT 向下数比较的寄存器为 CCR1N

CCR1N\_EN=0: CNT 向下数比较的寄存器为 CCR1

Bit 30:16 保留, 必须保持复位值。

Bit 15:0 CCR1N[15:0]: 捕获/比较通道 1N 的值 (Capture/Compare 1N value)

若 CC1 通道配置为输出:

CCR1N 包含了装入当前捕获/比较 1 寄存器的值 (预装载值)。

如果在 TIMx\_CCMR1 寄存器 (OC1PE 位) 中未选择预装载功能, 写入的数值会立即传输至当前寄存器中。否则只有当更新事件发生时, 此预装载值才传输至当前捕获/比较 1 寄存器中。CCR1N\_EN 必须为 1, 寄存器才有效果。

### 23.6.30 TIMx 捕获/比较寄存器 2N (TIMx\_CCR2N) (x = 1, 8)

偏移地址: 0x06C

复位值: 0x0000 0000

| 31          | 30       | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|-------------|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| EN          | Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15          | 14       | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CCR2N[15:0] |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw       | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31 EN: 若 CC2 通道配置为输出:

CCR2N\_EN=1: CNT 向下数比较的寄存器为 CCR2N

CCR2N\_EN=0: CNT 向下数比较的寄存器为 CCR2

Bit 30:16 保留, 必须保持复位值。

Bit 15:0 CCR2N[15:0]: 捕获/比较通道 2N 的值 (Capture/Compare 2N value)

若 CC2 通道配置为输出:

CCR2N 包含了装入当前捕获/比较 2 寄存器的值 (预装载值)。

如果在 TIMx\_CCMR1 寄存器 (OC2PE 位) 中未选择预装载功能, 写入的数值会立即传输至当前寄存器中。否则只有当更新事件发生时, 此预装载值才传输至当前捕获/比较 2 寄存器中。CCR2N\_EN 必须为 1, 寄存器才有效果。



### 23.6.31 TIMx 捕获/比较寄存器 3N (TIMx\_CCR3N) (x = 1, 8)

偏移地址: 0x070

复位值: 0x0000 0000

|             |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31          | 30       | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| EN          | Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15          | 14       | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CCR3N[15:0] |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw       | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31 EN: 若 CC3 通道配置为输出:

CCR3N\_EN=1: CNT 向下数比较的寄存器为 CCR3N

CCR3N\_EN=0: CNT 向下数比较的寄存器为 CCR3

Bit 30:16 保留, 必须保持复位值。

Bit 15:0 CCR3N[15:0]: 捕获/比较通道 3N 的值 (Capture/Compare 3N value)

若 CC3 通道配置为输出:

CCR3N 包含了装入当前捕获/比较 3 寄存器的值 (预装载值)。

如果在 TIMx\_CCMR2 寄存器 (OC3PE 位) 中未选择预装载功能, 写入的数值会立即传输至当前寄存器中。否则只有当更新事件发生时, 此预装载值才传输至当前捕获/比较 1 寄存器中。CCR3N\_EN 必须为 1, 寄存器才有效果。

### 23.6.32 TIMx 捕获/比较寄存器 4N (TIMx\_CCR4N) (x = 1, 8)

偏移地址: 0x074

复位值: 0x0000 0000

|             |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31          | 30       | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| EN          | Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15          | 14       | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CCR4N[15:0] |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw       | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31 EN: 若 CC4 通道配置为输出:

CCR4N\_EN=1: CNT 向下数比较的寄存器为 CCR4N

CCR4N\_EN=0: CNT 向下数比较的寄存器为 CCR4

Bit 30:16 保留, 必须保持复位值。

Bit 15:0 CCR4N[15:0]: 捕获/比较通道 4N 的值 (Capture/Compare 4N value)

若 CC4 通道配置为输出:

CCR4N 包含了装入当前捕获/比较 4 寄存器的值 (预装载值)。

如果在 TIMx\_CCMR2 寄存器 (OC4PE 位) 中未选择预装载功能, 写入的数值会立即传输至当前寄存器中。否则只有当更新事件发生时, 此预装载值才传输至当前捕获/比较 1 寄存器中。CCR4N\_EN 必须为 1, 寄存器才有效果。

### 23.6.33 TIMx 刹车输入控制选择 (TIMx\_BICS) (x = 1, 8)

偏移地址: 0x078

复位值: 0x0000

| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6   | 5   | 4   | 3   | 2   | 1   | 0    |
|----------|----|----|----|----|----|---|---|---|-----|-----|-----|-----|-----|-----|------|
| Reserved |    |    |    |    |    |   |   |   | CH6 | CH5 | CH4 | CH3 | CH2 | CH1 | BICE |
|          |    |    |    |    |    |   |   |   | rw  | rw  | rw  | rw  | rw  | rw  | rw   |

Bit 15:7 保留, 必须保持复位值。

Bit 6 **CH6**: 刹车输入控制选择 CH6 (Break Input Control Selection CH6)

当 BICE=1 时:

0: CH6 刹车有效或设置 MOE=0 有效, 通道根据极性配置输出对应电平

1: CH6 刹车无效, 设置 MOE=0 无效, 通道维持原有输出

Bit 5 **CH5**: 刹车输入控制选择 CH5 (Break Input Control Selection CH5)

当 BICE=1 时:

0: CH5 刹车有效或设置 MOE=0 有效, 通道根据极性配置输出对应电平

1: CH5 刹车无效, 设置 MOE=0 无效, 通道维持原有输出

Bit 4 **CH4**: 刹车输入控制选择 CH4 (Break Input Control Selection CH4)

当 BICE=1 时:

0: CH4 刹车有效或设置 MOE=0 有效, 通道根据极性配置输出对应电平

1: CH4 刹车无效, 设置 MOE=0 无效, 通道维持原有输出

Bit 3 **CH3**: 刹车输入控制选择 CH3 (Break Input Control Selection CH3)

当 BICE=1 时:

0: CH3 刹车有效或设置 MOE=0 有效, 通道根据极性配置输出对应电平

1: CH3 刹车无效, 设置 MOE=0 无效, 通道维持原有输出

Bit 2 **CH2**: 刹车输入控制选择 CH2 (Break Input Control Selection CH2)

当 BICE=1 时:

0: CH2 刹车有效或设置 MOE=0 有效, 通道根据极性配置输出对应电平

1: CH2 刹车无效, 设置 MOE=0 无效, 通道维持原有输出

Bit 1 **CH1**: 刹车输入控制选择 CH1 (Break Input Control Selection CH1)

当 BICE=1 时:

0: CH1 刹车有效或设置 MOE=0 有效, 通道根据极性配置输出对应电平

1: CH1 刹车无效, 设置 MOE=0 无效, 通道维持原有输出

Bit 0 **BICE**: 刹车输入控制使能 (Break Input Control Enable)

当刹车输入或设置 MOE=0 时, 此位控制刹车或设置 MOE=0 是否对

CHx/CHyN(x=1,2,3,4,5,6)(y=1,2,3,4)有效:

BICE=0: 刹车有效或设置 MOE=0 有效, 通道根据极性配置输出对应电平

BICE=1: 根据 BICS\_CHx 位可选某些通道忽略刹车输入或忽略设置 MOE=0 操作维持原有输出



### 23.6.34 TIMx DMA 控制寄存器 (TIMx\_DCR) (x = 1, 8)

偏移地址: 0x3DC

复位值: 0x0000 0000

|          |    |    |          |    |    |    |    |          |    |    |          |    |    |    |    |
|----------|----|----|----------|----|----|----|----|----------|----|----|----------|----|----|----|----|
| 31       | 30 | 29 | 28       | 27 | 26 | 25 | 24 | 23       | 22 | 21 | 20       | 19 | 18 | 17 | 16 |
| Reserved |    |    |          |    |    |    |    |          |    |    |          |    |    |    |    |
| 15       | 14 | 13 | 12       | 11 | 10 | 9  | 8  | 7        | 6  | 5  | 4        | 3  | 2  | 1  | 0  |
| Reserved |    |    | DBL[4:0] |    |    |    |    | Reserved |    |    | DBA[4:0] |    |    |    |    |
|          |    |    | rw       | rw | rw | rw | rw |          |    |    | rw       | rw | rw | rw | rw |

Bit 31:13 保留, 必须保持复位值。

Bit 12:8 **DBL[4:0]**: DMA 突发长度 (DMA burst length)

这个 5 位向量定义了 DMA 传输的长度 (当对 TIMx\_DMAR 地址进行读或写访问时, 定时器识别一次突发传输), 即传输次数。传输可以是半字或字节 (见下面的示例)。

00000: 1 次传输

00001: 2 次传输

00010: 3 次传输

...

11010: 26 次传输

**示例:** 让我们考虑以下传输: DBL = 7 字节 & DBA = TIM2\_CR1。如果 DBL = 7 字节, 并且 DBA = TIM2\_CR1 表示要传输的字节的地址, 那么传输的地址应由以下公式给出: (TIMx\_CR1 地址) + DBA + (DMA 索引), 其中 DMA 索引 = DBL

在这个例子中, 将 7 字节添加到(TIMx\_CR1 地址) + DBA, 这给出了数据从/复制到的地址。在这种情况下, 传输是从以下地址开始的 7 个寄存器: (TIMx\_CR1 地址) + DBA

根据 DMA 数据大小的配置, 可能会发生几种情况:

- 如果 DMA 数据大小配置为半字, 则将 16 位数据传输到 7 个寄存器中的每一个。
- 如果 DMA 数据大小配置为字节, 则数据也传输到 7 个寄存器: 第一个寄存器包含第一个 MSB 字节, 第二个寄存器包含第一个 LSB 字节, 依此类推。

因此, 在使用传输定时器时, 还必须指定 DMA 传输的数据大小。

Bit 7:5 保留, 必须保持复位值。

Bit 4:0 **DBA[4:0]**: DMA 基地址 (DMA base address)

这个 5 位向量定义了 DMA 传输的基地址 (当通过 TIMx\_DMAR 地址进行读/写访问时)。DBA 是从 TIMx\_CR1 寄存器的地址开始的偏移量。

00000: TIMx\_CR1,

00001: TIMx\_CR2,

00010: TIMx\_SMCR,

...

### 23.6.35 TIMx 用于完整传输的 DMA 地址 (TIMx\_DMAR) (x = 1, 8)

偏移地址: 0x3E0

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DMAB[31:16] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| DMAB[15:0]  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:0 **DMAB[31:0]**: 用于突发访问的 DMA 寄存器 (DMA register for burst accesses)

对 DMAR 寄存器进行的读或写操作将访问位于以下地址的寄存器:

$$(TIMx\_CR1 \text{ 地址}) + (DBA + \text{DMA 索引}) \times 4$$

其中, TIMx\_CR1 地址是控制寄存器 1 的地址, DBA 是在 TIMx\_DCR 寄存器中配置的 DMA 基地址, DMA 索引由 DMA 传输自动控制, 其范围从 0 到 DBL (在 TIMx\_DCR 中配置的 DBL)。

## 24 通用定时器 (TIM2/TIM3/TIM4/TIM5)

### 24.1 TIM2/TIM3/TIM4/TIM5 简介

通用定时器包含一个 16 位或 32 位自动重载计数器，该计数器由可编程预分频器驱动。

它们可用于多种用途，包括测量输入信号的脉冲宽度（输入捕获）或生成输出波形（输出比较和 PWM）。使用定时器预分频器和 RCC 时钟控制器预分频器，可将脉冲宽度和波形周期从几微秒调制到几毫秒。这些定时器彼此完全独立，不共享任何资源，它们可以同步操作。

### 24.2 TIM2/TIM3/TIM4/TIM5 主要功能

通用 TIMx (TIM2、TIM3) 定时器功能包括：

- 16 位 (TIM3 和 TIM4) 或 32 位 (TIM2 和 TIM5) 向上、向下、向上/向下自动装载计数器
- 16 位可编程（可以实时修改）预分频器，计数器时钟频率的分频系数为 1~65536 之间的任意数值
- 4 个独立通道：
  - 输入捕获
  - 输出比较
  - PWM 生成（边沿或中间对齐模式）
  - 单脉冲模式输出
- 使用外部信号控制定时器和定时器互连的同步电路
- 如下事件发生时产生中断/DMA 请求：
  - 更新：计数器向上溢出/向下溢出，计数器初始化（通过软件或者内部/外部触发）
  - 触发事件（计数器启动、停止、初始化或者由内部/外部触发计数）
  - 输入捕获
  - 输出比较
- 支持针对定位的增量（正交）编码器和霍尔传感器电路
- 触发输入作为外部时钟或者按周期的电流管理

### 24.3 TIM2/TIM3/TIM4/TIM5 实现

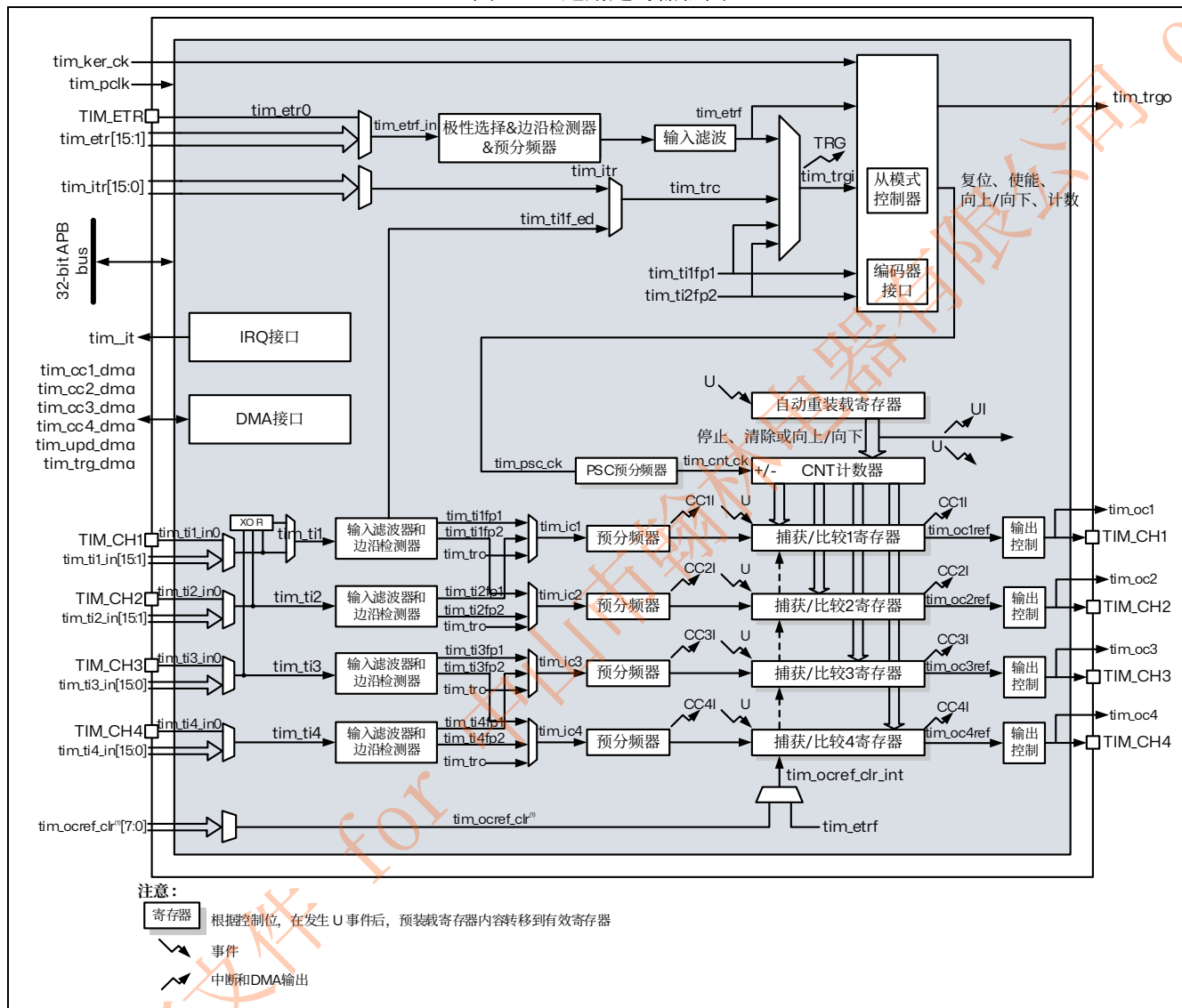
表 24.1 RX32G410 系列通用定时器

| 定时器实例 | TIM2                            | TIM3                            | TIM4           | TIM5           |
|-------|---------------------------------|---------------------------------|----------------|----------------|
| 分辨率   | 32 位                            | 16 位                            | 16 位           | 32 位           |
| OCREF | 是                               | 是                               | 否              | 否              |
| 清除选择源 | tim_etrfr<br>tim_ocref_clr[7:0] | tim_etrfr<br>tim_ocref_clr[7:0] | tim_etrfr<br>- | tim_etrfr<br>- |

## 24.4 TIM2/TIM3/TIM4/TIM5 功能描述

### 24.4.1 框图

图 24.1 通用定时器框图



注 1. 此功能具体请参考章节：TIM2/TIM3/TIM4/TIM5 实现

## 24.4.2 TIM2/TIM3/TIM4/TIM5 引脚和内部信号

本节中的下两张表格总结了 TIM 的输入和输出。

表 24.2 TIM 输入/输出引脚

| 引脚名称                                     | 信号类型  | 描述                                                                                                                                                 |
|------------------------------------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| TIM_CH1<br>TIM_CH2<br>TIM_CH3<br>TIM_CH4 | 输入/输出 | 定时器多功能通道。<br>每个通道用于捕获、比较或 PWM。<br>TIM_CH1 和 TIM_CH2 也可用作外部时钟 (tim_ker_ck 时钟的 1/4 以下)，外部触发器和正交编码器输入。<br>TIM_CH1, TIM_CH2 和 TIM_CH3 可被用作数字霍尔效应传感器接口。 |
| TIM_ETR                                  | 输入    | 外部触发器输入。这个输入可被用作外部触发器或外部时钟源。<br>如果使用 tim_etr_in 预分频器，则该输入可以接收频率高于 tim_ker_ck 的时钟。                                                                  |

表 24.3 TIM 内部输入/输出信号

| 内部信号名称                                                                       | 信号类型 | 描述                                                                                              |
|------------------------------------------------------------------------------|------|-------------------------------------------------------------------------------------------------|
| tim_ti1_in[15:0]<br>tim_ti2_in[15:0]<br>tim_ti3_in[15:0]<br>tim_ti4_in[15:0] | 输入   | 内部定时器输入总线。<br>tim_ti1_in[15:0]和 tim_ti2_in[15:0]输入可被用作捕获或作为外部时钟(tim_ker_ck 时钟的 1/4 以下)和正交编码器信号。 |
| tim_etr[15:0]                                                                | 输入   | 外部触发器内部输入总线。这些输入可被用作触发器、外部时钟或用于硬件的逐周期脉冲宽度控制。如果使用 tim_etr_in 预分频器，这些输入可以接收频率高于 tim_ker_ck 的时钟。   |
| tim_itr[15:0]                                                                | 输入   | 内部触发输入总线。这些输入可用于从模式控制器或作为输入时钟(tim_ker_ck 时钟的 1/4 以下)。                                           |
| tim_trgo                                                                     | 输出   | 内部触发器输出。此触发器可以触发其他片上外设。                                                                         |
| tim_ocref_clr[7:0]                                                           | 输入   | 定时器 tim_ocref_clr 输入总线。这些输入可以用来清除 tim_ocxref 信号，通常用于硬件的逐周期脉冲宽度控制。                               |
| tim_pclk                                                                     | 输入   | 定时器 APB 时钟。                                                                                     |
| tim_ker_ck                                                                   | 输入   | 定时器内核时钟。                                                                                        |
| tim_it                                                                       | 输出   | 全局定时器中断，收集捕获/比较、更新和中断触发器请求。                                                                     |
| tim_cc1_dma<br>tim_cc2_dma<br>tim_cc3_dma<br>tim_cc4_dma                     | 输出   | 定时器捕获/比较 1 至 4 dma 请求。                                                                          |
| tim_upd_dma                                                                  | 输出   | 定时器更新 dma 请求。                                                                                   |
| tim_trg_dma                                                                  | 输出   | 定时器触发 dma 请求。                                                                                   |

如下四张表列出了连接到 tim\_ti[4:1]输入多路复用器的源。

表 24.4 连接到 tim\_ti1 的输入多路复用器

| tim_ti1 输入  | 源         |           |           |           |
|-------------|-----------|-----------|-----------|-----------|
|             | TIM2      | TIM3      | TIM4      | TIM5      |
| tim_ti1_in0 | TIM2_CH1  | TIM3_CH1  | TIM4_CH1  | TIM5_CH1  |
| tim_ti1_in1 | comp1_out | comp1_out | comp1_out | LSI       |
| tim_ti1_in2 | comp2_out | comp2_out | comp2_out | 保留        |
| tim_ti1_in3 | comp3_out | comp3_out | comp3_out | RTC 唤醒    |
| tim_ti1_in4 | comp4_out | comp4_out | comp4_out | comp1_out |
| tim_ti1_in5 | 保留        |           |           | comp2_out |
| tim_ti1_in6 |           |           |           | comp3_out |
| tim_ti1_in7 |           |           |           | comp4_out |

表 24.5 连接到 tim\_ti2 的输入多路复用器

| tim_ti2 输入  | 源         |           |           |           |
|-------------|-----------|-----------|-----------|-----------|
|             | TIM2      | TIM3      | TIM4      | TIM5      |
| tim_ti2_in0 | TIM2_CH2  | TIM3_CH2  | TIM4_CH2  | TIM5_CH2  |
| tim_ti2_in1 | comp1_out | comp1_out | comp1_out | comp1_out |
| tim_ti2_in2 | comp2_out | comp2_out | comp2_out | comp2_out |
| tim_ti2_in3 | comp3_out | comp3_out | comp3_out | comp3_out |
| tim_ti2_in4 | comp4_out | comp4_out | comp4_out | comp4_out |

表 24.6 连接到 tim\_ti3 的输入多路复用器

| tim_ti3 输入        | 源         |           |          |          |
|-------------------|-----------|-----------|----------|----------|
|                   | TIM2      | TIM3      | TIM4     | TIM5     |
| tim_ti3_in0       | TIM2_CH3  | TIM3_CH3  | TIM4_CH3 | TIM5_CH3 |
| tim_ti3_in1       | comp4_out | comp3_out | 保留       | 保留       |
| tim_ti2_in[2..15] | 保留        |           |          |          |

表 24.7 连接到 tim\_ti4 的输入多路复用器

| tim_ti4 输入        | 源         |          |          |          |
|-------------------|-----------|----------|----------|----------|
|                   | TIM2      | TIM3     | TIM4     | TIM5     |
| tim_ti4_in0       | TIM2_CH4  | TIM3_CH4 | TIM4_CH4 | TIM5_CH4 |
| tim_ti4_in1       | comp1_out | 保留       |          |          |
| tim_ti4_in2       | comp2_out |          |          |          |
| tim_ti4_in[3..15] | 保留        |          |          |          |

下表列出连接到 tim\_itr 输入多路复用器的内部源

表 24.8 TIMx 内部触发连接

| TIMx           | TIM2            | TIM3            | TIM4            | TIM5            |
|----------------|-----------------|-----------------|-----------------|-----------------|
| tim_itr0       | tim1_trgo       | tim1_trgo       | tim1_trgo       | tim1_trgo       |
| tim_itr1       | 保留              | tim2_trgo       | tim2_trgo       | tim2_trgo       |
| tim_itr2       | tim3_trgo       | 保留              | tim3_trgo       | tim3_trgo       |
| tim_itr3       | tim4_trgo       | tim4_trgo       | 保留              | tim4_trgo       |
| tim_itr4       | tim5_trgo       | tim5_trgo       | tim5_trgo       | 保留              |
| tim_itr5       | tim8_trgo       | tim8_trgo       | tim8_trgo       | tim8_trgo       |
| tim_itr6       | tim15_trgo      | tim15_trgo      | tim15_trgo      | tim15_trgo      |
| tim_itr7       | tim16_oc1       | tim16_oc1       | tim16_oc1       | tim16_oc1       |
| tim_itr8       | tim17_oc1       | tim17_oc1       | tim17_oc1       | tim17_oc1       |
| tim_itr9       | 保留              |                 |                 |                 |
| tim_itr10      | hrtim_out_sync2 | hrtim_out_sync2 | hrtim_out_sync2 | hrtim_out_sync2 |
| tim_itr11      | USB SOF SYNC    | 保留              |                 |                 |
| tim_itr[15:12] | 保留              |                 |                 |                 |

下表列出连接到 tim\_etr 输入多路复用器的内部源

表 24.9 连接到 tim\_etr 的输入多路复用器

| 定时器外部<br>触发输入信号 | 定时器外部触发信号分配 |           |           |           |
|-----------------|-------------|-----------|-----------|-----------|
|                 | TIM2        | TIM3      | TIM4      | TIM5      |
| tim_etr0        | TIM2_ETR    | TIM3_ETR  | TIM4_ETR  | TIM5_ETR  |
| tim_etr1        | comp1_out   | comp1_out | comp1_out | comp1_out |
| tim_etr2        | comp2_out   | comp2_out | comp2_out | comp2_out |
| tim_etr3        | comp3_out   | comp3_out | comp3_out | comp3_out |
| tim_etr4        | comp4_out   | comp4_out | comp4_out | comp4_out |
| tim_etr5        | 保留          |           |           |           |
| tim_etr6        |             |           |           |           |
| tim_etr7        |             |           |           |           |
| tim_etr8        | TIM3_ETR    | TIM2_ETR  | TIM3_ETR  | TIM2_ETR  |
| tim_etr9        | TIM4_ETR    | TIM4_ETR  | TIM5_ETR  | TIM3_ETR  |
| tim_etr10       | TIM5_ETR    | 保留        | 保留        |           |
| tim_etr11       | 保留          | adc2_awd1 |           |           |
| tim_etr12       |             | 保留        |           |           |
| tim_etr13       |             |           |           |           |
| tim_etr[15:14]  | 保留          |           |           |           |

下表列出连接到 tim\_ocref\_clr 输入多路复用器的内部源



表 24.10 连接到 ocref\_clr 的输入多路复用器

| 定时器信号<br>tim_ocref_clr | 定时器 tim_ocref_clr 信号分配 |           |      |      |
|------------------------|------------------------|-----------|------|------|
|                        | TIM2                   | TIM3      | TIM4 | TIM5 |
| tim_ocref_clr0         | comp1_out              | comp1_out | 保留   |      |
| tim_ocref_clr1         | comp2_out              | comp2_out |      |      |
| tim_ocref_clr2         | comp3_out              | comp3_out |      |      |
| tim_ocref_clr3         | Comp4_out              | Comp4_out |      |      |

### 24.4.3 时基单元

可编程定时器的主要模块由一个 16 位/32 位计数器及其相关的自动重装寄存器组成。此计数器可采用递增方式计数。计数器的时钟可通过预分频器进行分频。

计数器、自动重载寄存器和预分频器寄存器可通过软件进行读写。即使在计数器运行时也可执行读写操作。

时基单元包含：

- 计数器寄存器 (TIMx\_CNT)
- 预分频器寄存器 (TIMx\_PSC)
- 自动装载寄存器 (TIMx\_ARR)

自动重载寄存器是预装载的。对自动重载寄存器执行写入或读取操作时会访问预装载寄存器。预装载寄存器的内容既可以直接传送到影子寄存器，也可以在每次发生更新事件 (UEV) 时传送到影子寄存器，这取决于 TIMx\_CR1 寄存器中的自动重载预装载使能位 (ARPE)。当计数器达到上溢值（或者在递减计数时达到下溢值）并且 TIMx\_CR1 寄存器中的 UDIS 位为 0 时，将发送更新事件。该更新事件也可由软件产生。下文将针对各配置的更新事件的产生 进行详细介绍。

计数器由预分频器输出 CK\_CNT 提供时钟，仅当 TIMx\_CR1 寄存器中的计数器启动位 (CEN) 置 1 时，才会启动计数器（有关计数器使能的更多详细信息，另请参见从模式控制器的相关说明）。

请注意，真正的计数器使能信号 CNT\_EN 在 CEN 置 1 的一个时钟周期后被置 1。

#### 预分频器描述

预分频器可对计数器时钟频率进行分频，分频系数介于 1 到 65536 之间。该预分频器基于 16 位/32 位寄存器 (TIMx\_PSC 寄存器) 所控制的 16 位计数器。由于该控制寄存器具有缓冲功能，因此预分频器可实现实时更改。而新的预分频比将在下一更新事件发生时被采用。

下两张图给出了在预分频器运行时，更改计数器参数的例子。

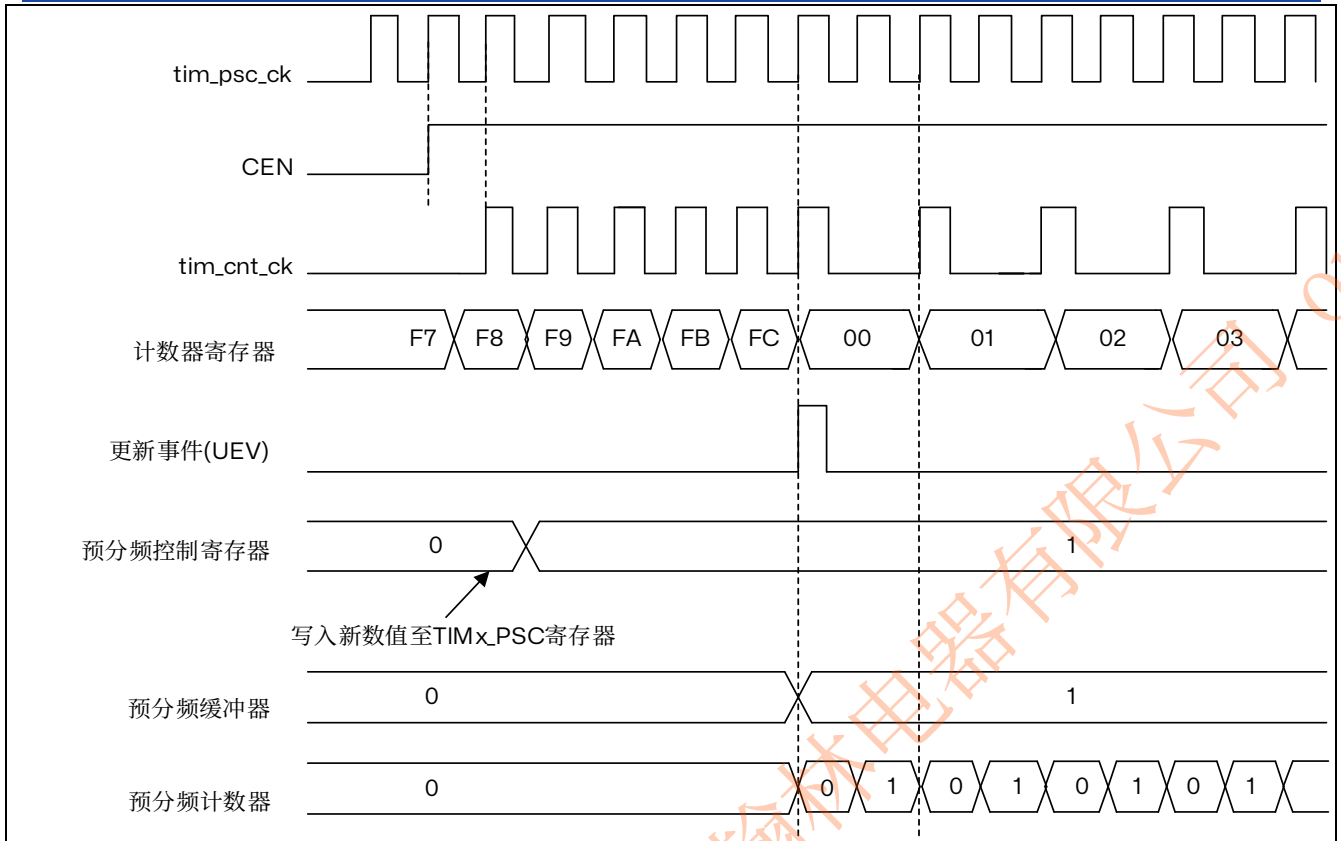


图 24.2 当预分频器的参数从1变到2时，计数器的时序图

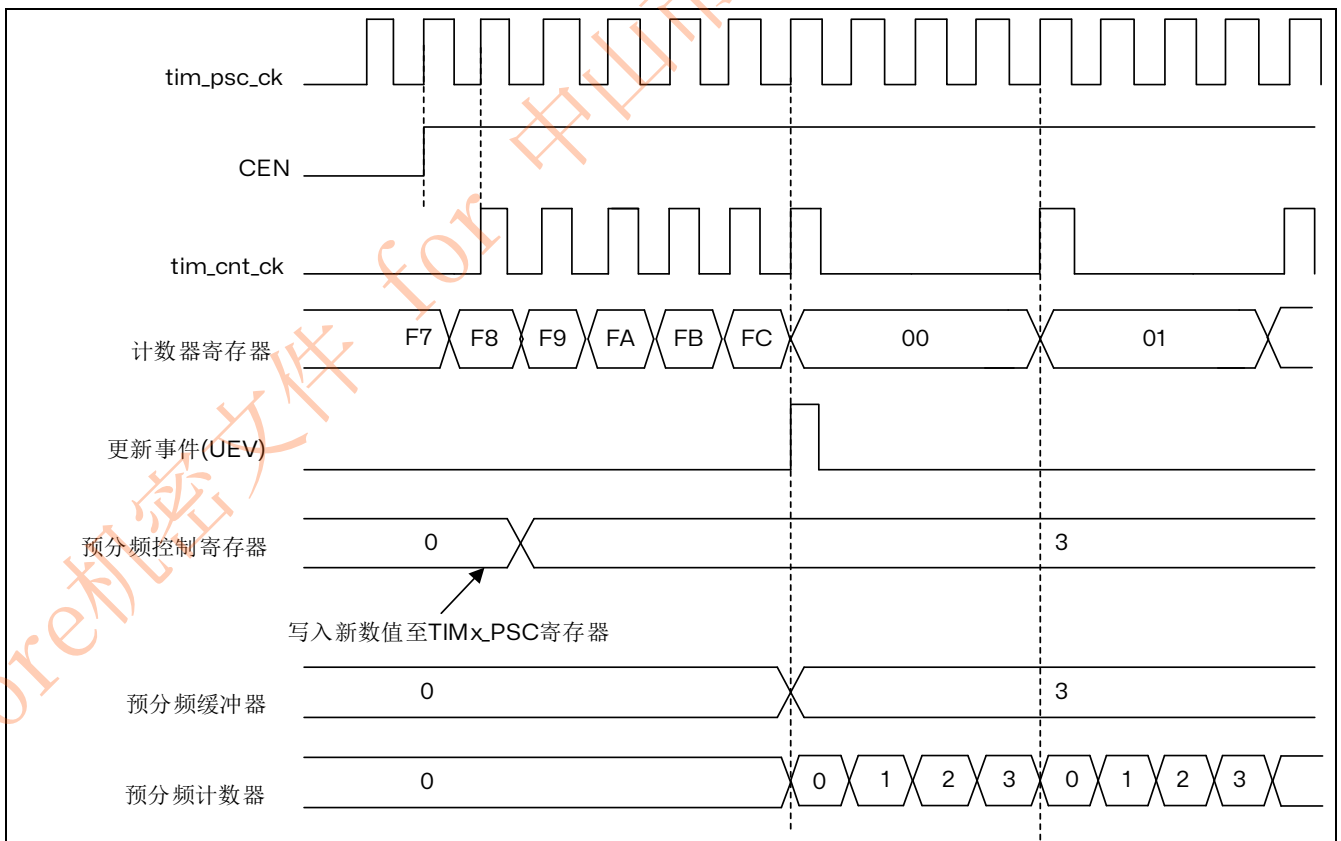


图 24.3 当预分频器的参数从1变到4时，计数器的时序图

## 24.4.4 计数器模式

### 递增计数模式

在递增计数模式下，计数器从 0 计数到自动重载值（TIMx\_ARR 寄存器的内容），然后重新从 0 开始计数并生成计数器上溢事件。

每次发生计数器上溢时会生成更新事件，或将 TIMx\_EGR 寄存器中的 UG 位置 1（通过软件或使用从模式控制器）也可以生成更新事件。

通过软件将 TIMx\_CR1 寄存器中的 UDIS 位置 1 可禁止 UEV 事件。这可避免向预装载寄存器写入新值时更新影子寄存器。在 UDIS 位写入 0 之前不会产生任何更新事件。不过，计数器和预分频器计数器都会重新从 0 开始计数（而预分频比保持不变）。此外，如果 TIMx\_CR1 寄存器中的 URS 位（更新请求选择）已置 1，则将 UG 位置 1 会生成更新事件 UEV，但不会将 UIF 标志置 1（因此，不会发送任何中断或 DMA 请求）。这样一来，如果在发生捕获事件时将计数器清零，将不会同时产生更新中断和捕获中断。

发生更新事件时，将更新所有寄存器且将更新标志（TIMx\_SR 寄存器中的 UIF 位）置 1（取决于 URS 位）：

- 预分频器的缓冲区中将重新装载预装载值（TIMx\_PSC 寄存器的内容）
- 自动重载影子寄存器将以预装载值进行更新

以下各图以一些示例说明当 TIMx\_ARR=0x36 时不同时钟频率下计数器的行为。

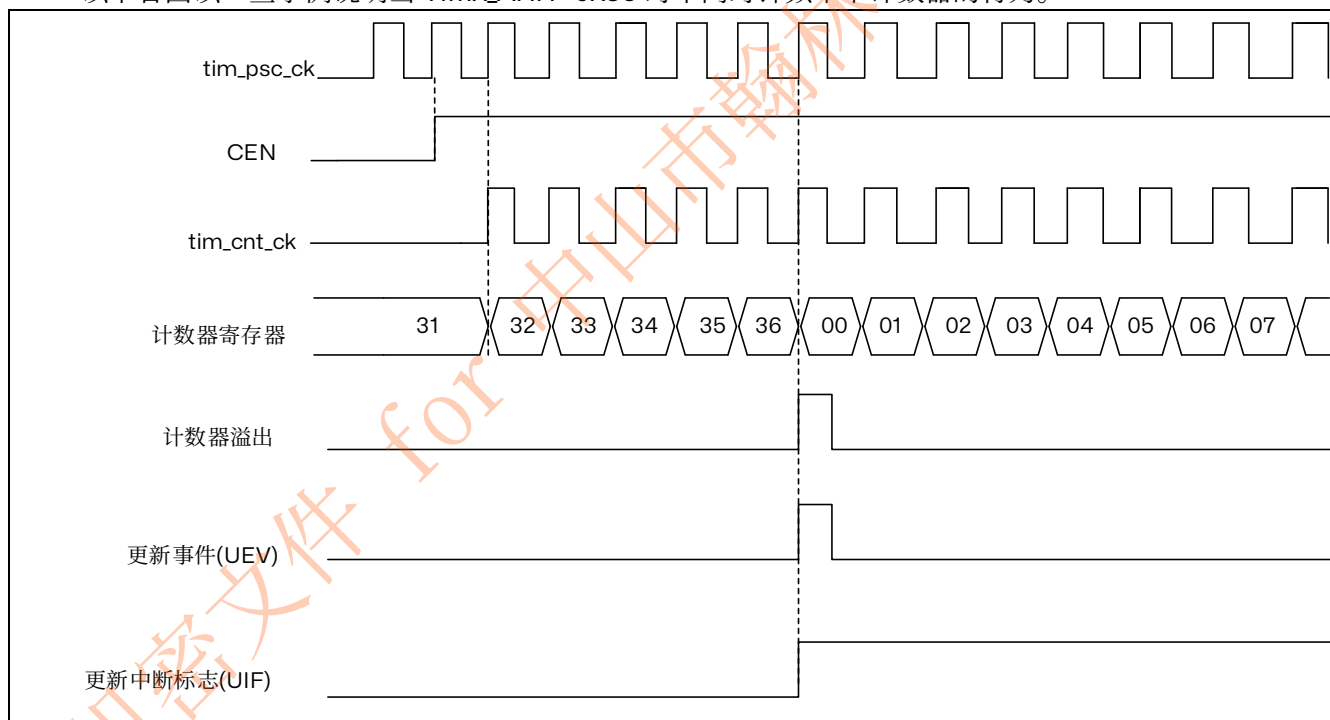


图 24.4 计数器时序图，内部时钟分频系数为 1

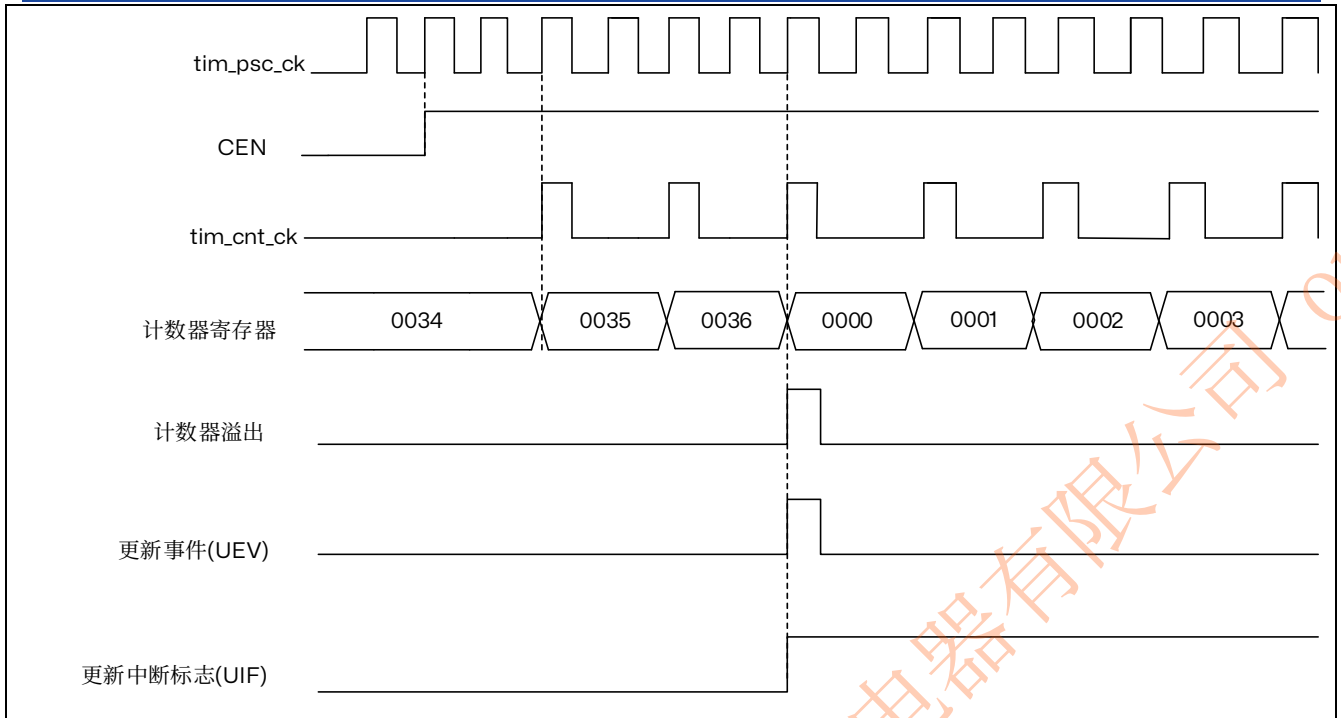


图 24.5 计数器时序图，内部时钟分频系数为 2

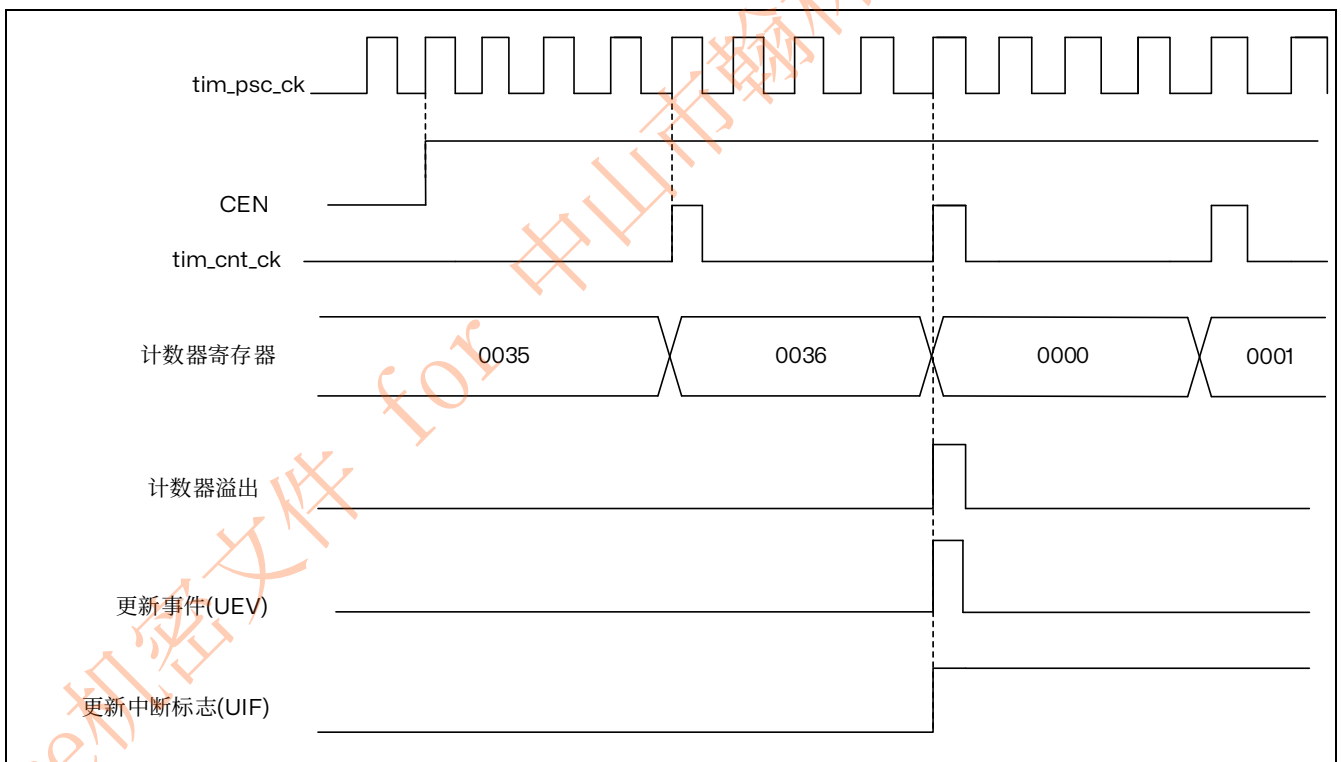


图 24.6 计数器时序图，内部时钟分频系数为 4

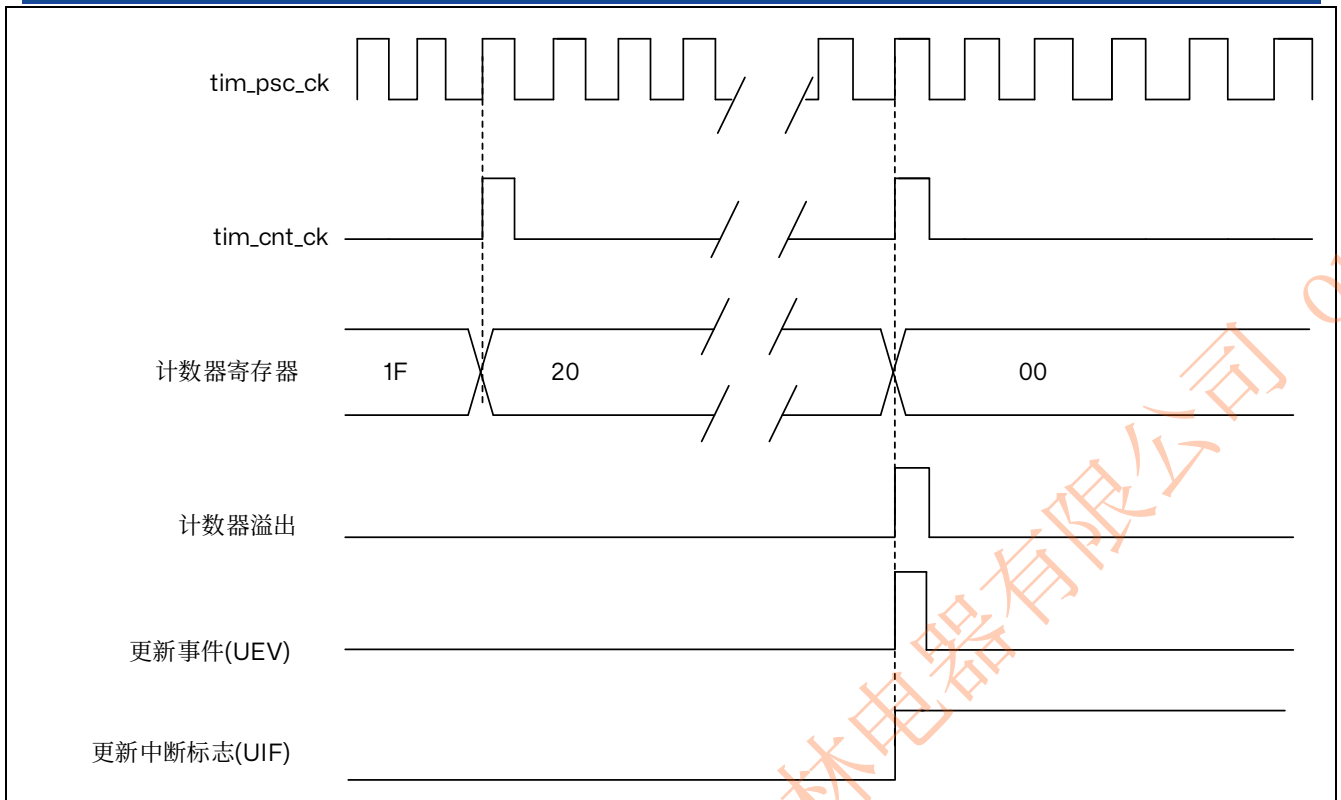


图 24.7 计数器时序图，内部时钟分频系数为 N

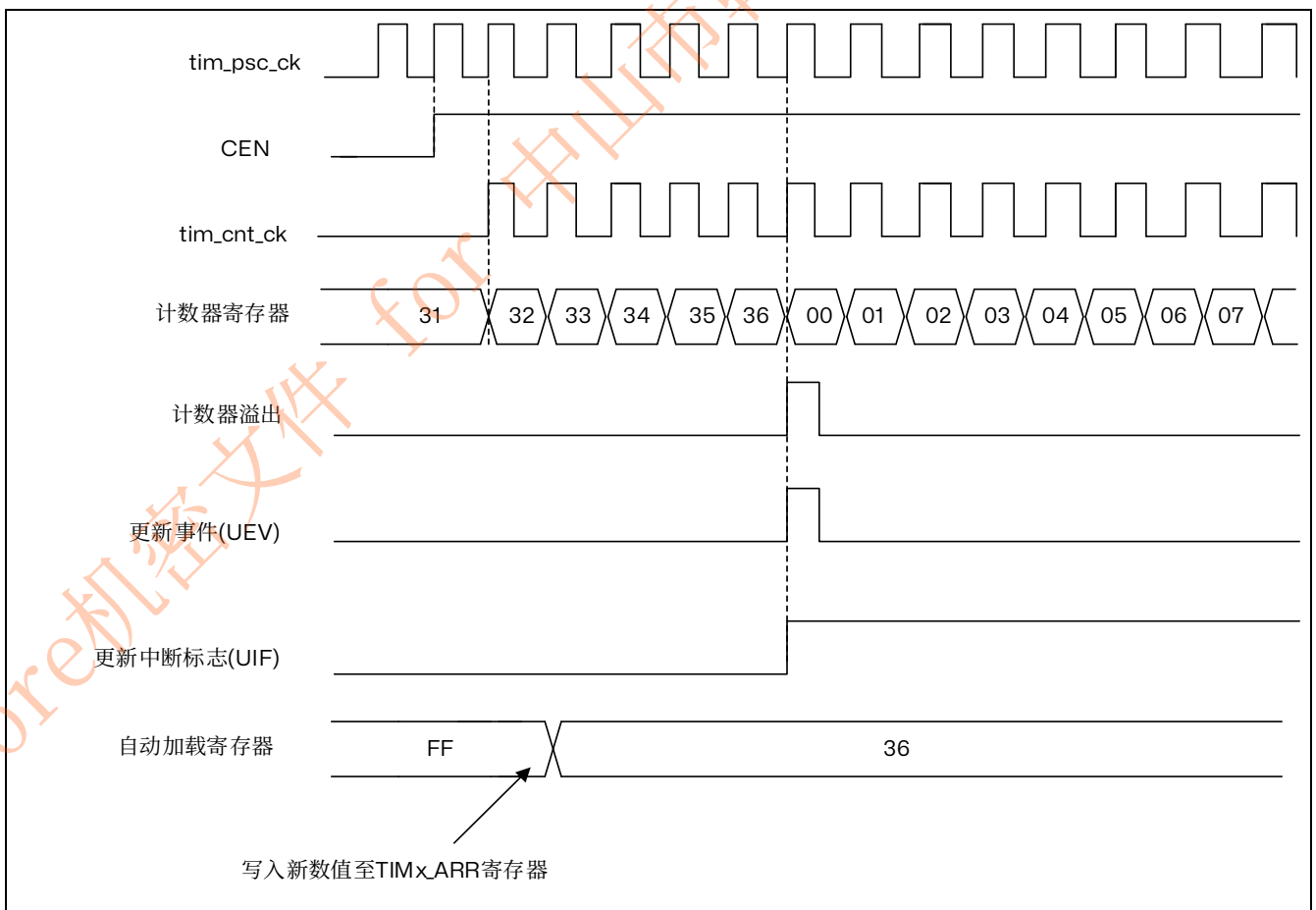


图 24.8 计数器时序图，当 ARPE=0 时的更新事件 (TIMx\_ARR 没有预装入)

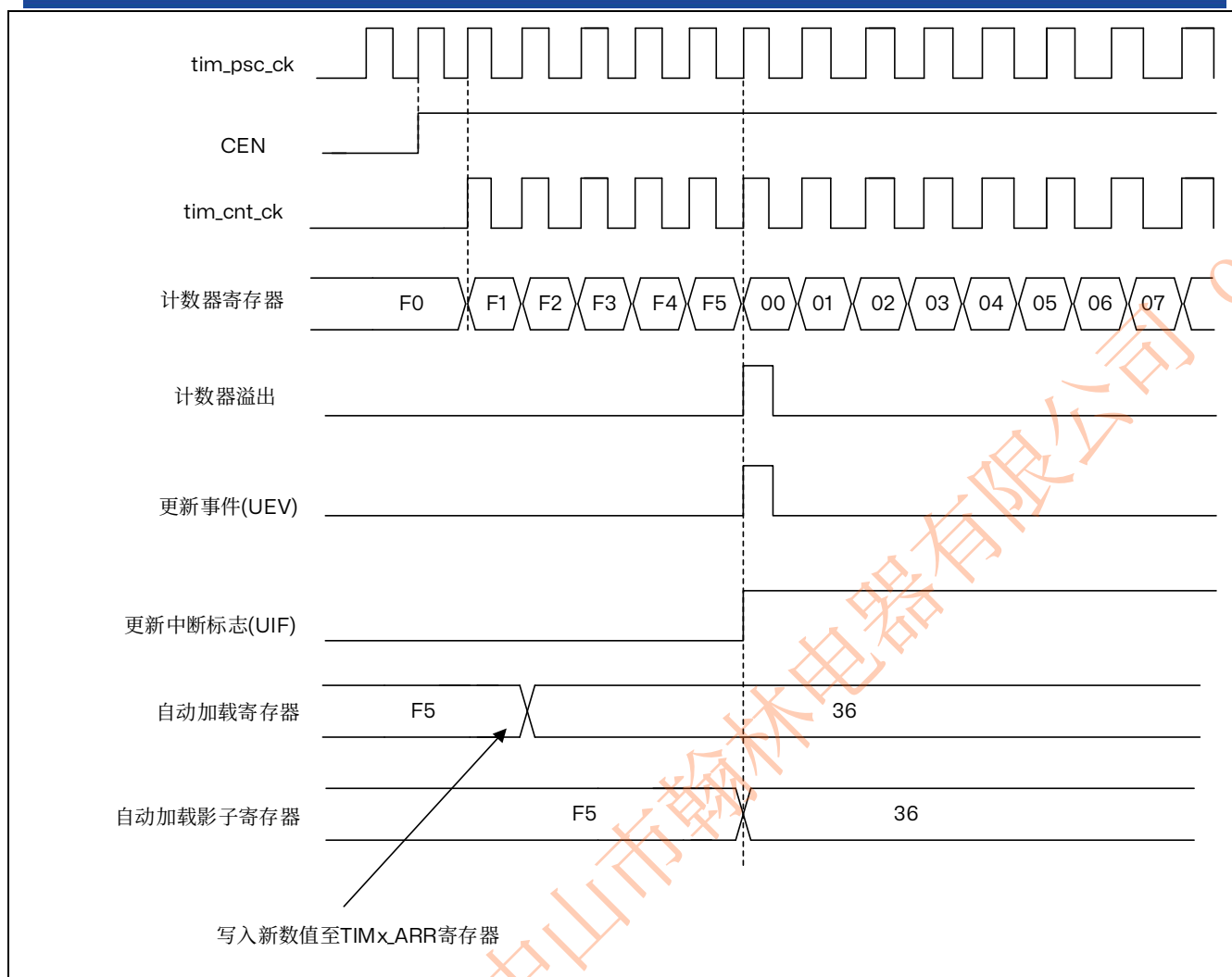


图 24.9 计数器时序图，当 ARPE=1 时的更新事件（预装入了 TIMx\_ARR）

### 递减计数模式

在递减计数模式下，计数器从自动重载值（TIMx\_ARR 寄存器的内容）开始递减计数到 0，然后重新从自动重载值开始计数并生成计数器下溢事件。

每次发生计数器下溢时会生成更新事件，或将 TIMx\_EGR 寄存器中的 UG 位置 1（通过软件或使用从模式控制器）也可以生成更新事件。

通过软件将 TIMx\_CR1 寄存器中的 UDIS 位置 1 可禁止 UEV 更新事件。这可避免向预装载寄存器写入新值时更新影子寄存器。在 UDIS 位写入 0 之前不会产生任何更新事件。不过，计数器会重新从当前自动重载值开始计数，而预分频器计数器则重新从 0 开始计数（但预分频比保持不变）。

此外，如果 TIMx\_CR1 寄存器中的 URS 位（更新请求选择）已置 1，则将 UG 位置 1 会生成更新事件 UEV，但不会将 UIF 标志置 1（因此，不会发送任何中断请求）。这样一来，如果在发生捕获事件时将计数器清零，将不会同时产生更新中断和捕获中断。

发生更新事件时，将更新所有寄存器且将更新标志（TIMx\_SR 寄存器中的 UIF 位）置 1（取决于 URS 位）：

- 预分频器的缓冲区中将重新装载预装载值（TIMx\_PSC 寄存器的内容）。
- 自动重载活动寄存器将以预装载值（TIMx\_ARR 寄存器的内容）进行更新。注意，自动重载寄存

器会在计数器重载之前得到更新，因此，下一个计数周期就是我们所希望的新的周期长度。  
以下各图以一些示例说明当 TIMx\_ARR=0x36 时不同时钟频率下计数器的行为。

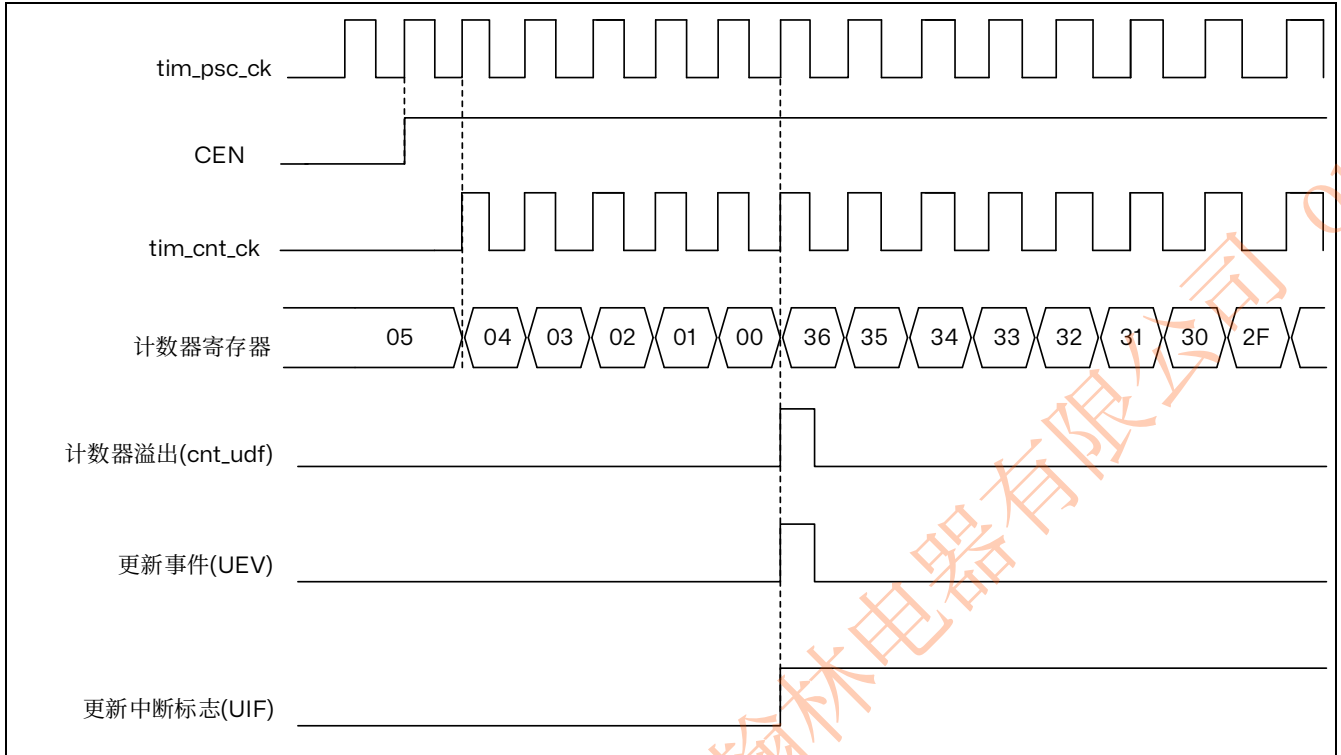


图 24.10 计数器时序图，内部时钟分频系数为 1

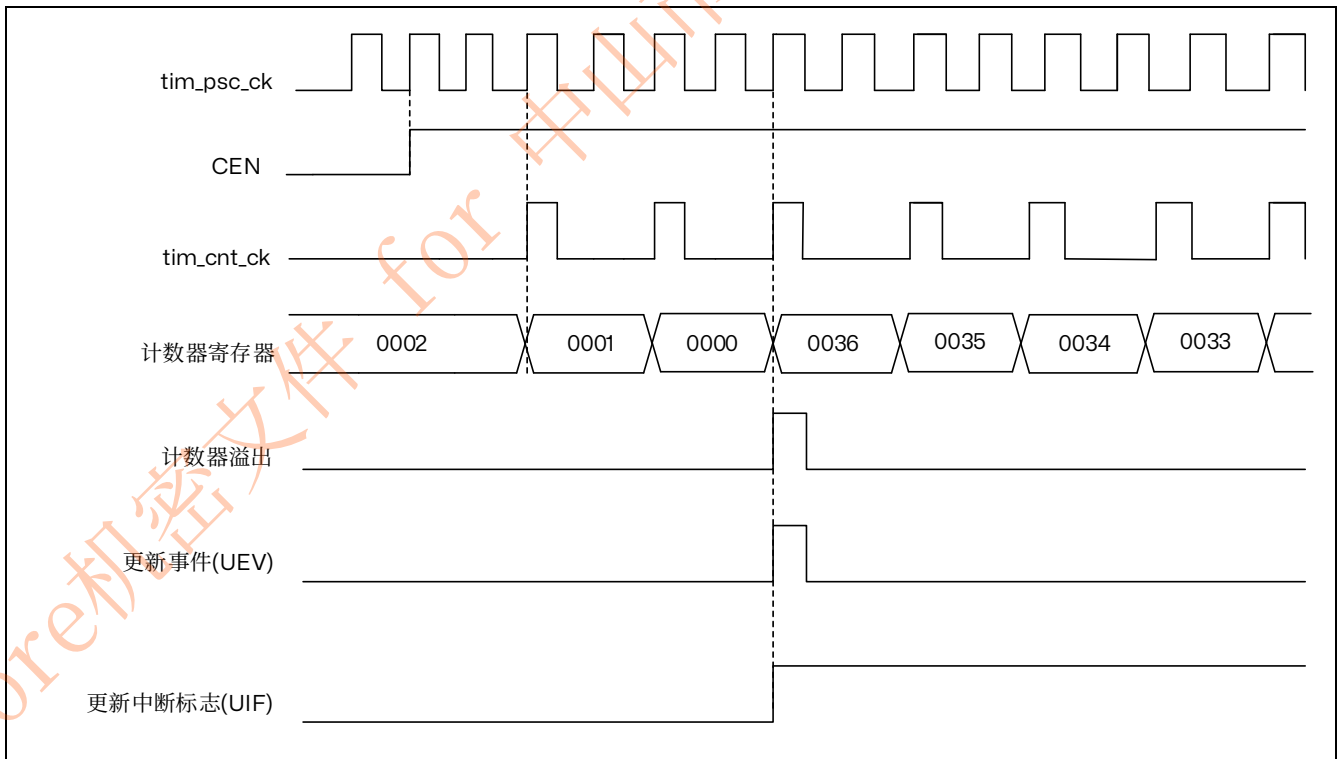


图 24.11 计数器时序图，内部时钟分频系数为 2



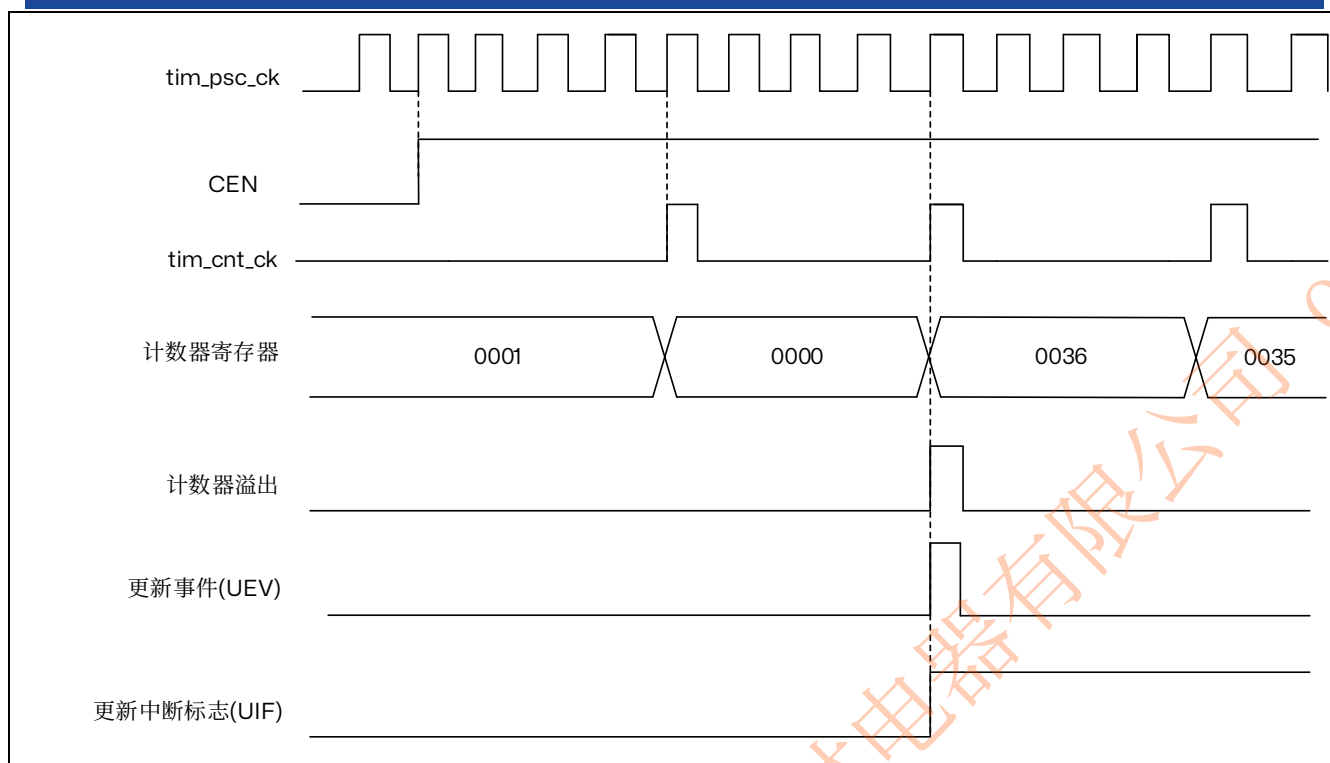


图 24.12 计数器时序图，内部时钟分频系数为 4

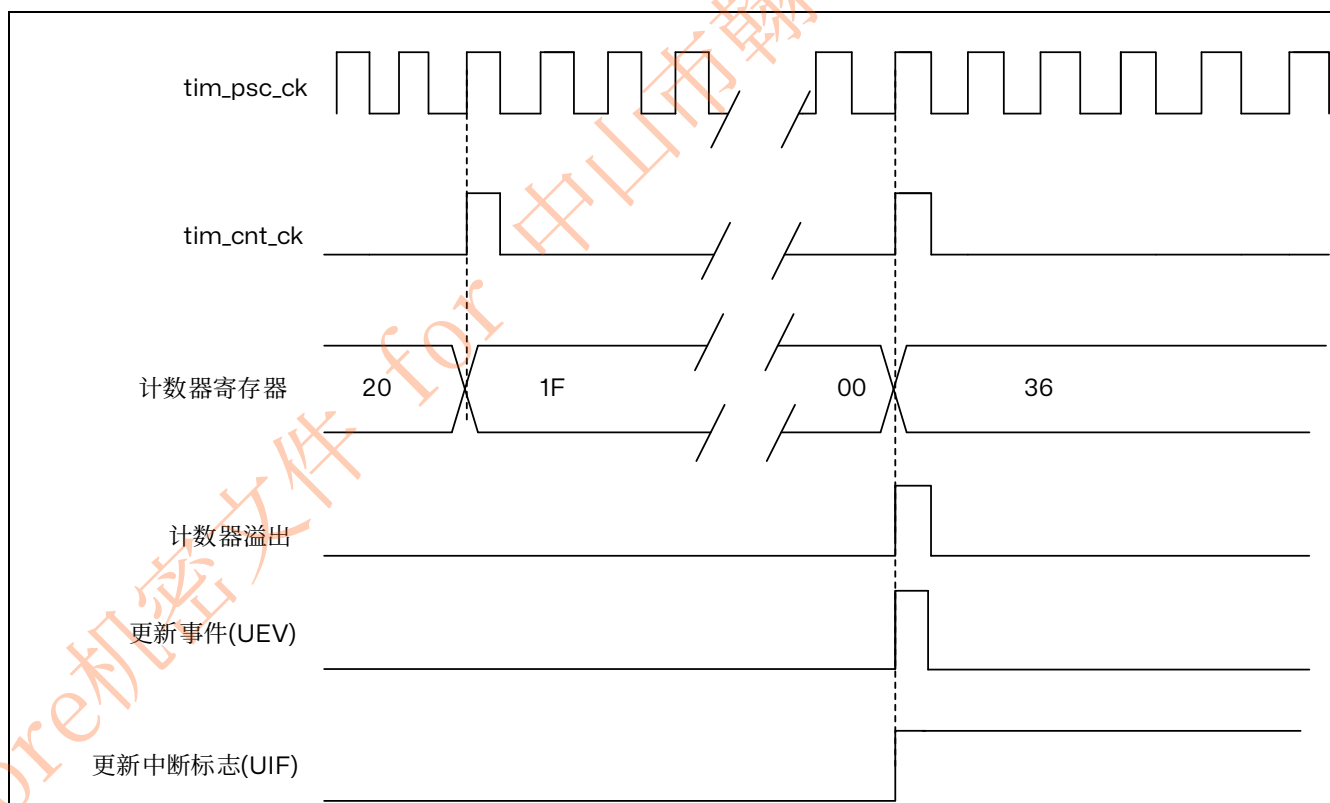


图 24.13 计数器时序图，内部时钟分频系数为 N

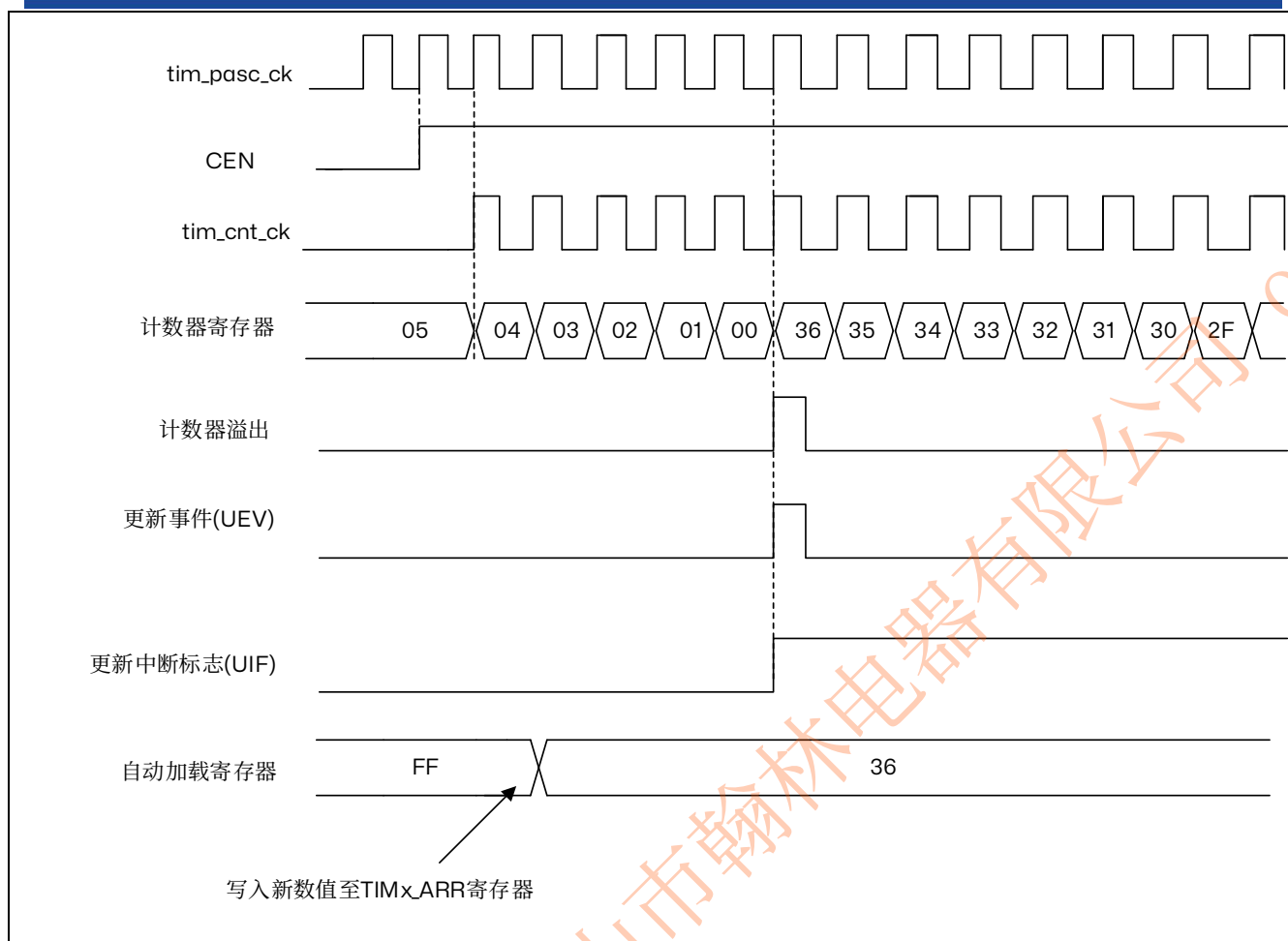


图 24.14 计数器时序图，更新事件

### 中央对齐模式（递增/递减计数）

在中心对齐模式下，计数器从 0 开始计数到自动重载值（TIMx\_ARR 寄存器的内容）- 1，生成计数器上溢事件；然后从自动重载值开始向下计数到 1 并生成计数器下溢事件。之后从 0 开始重新计数。

当 TIMx\_CR1 寄存器中的 CMS 位不为“00”时，中心对齐模式有效。将通道配置为输出模式时，其输出比较中断标志将在以下模式下置 1，即：计数器递减计数（中心对齐模式 1，CMS = “01”）、计数器递增计数（中心对齐模式 2，CMS = “10”）以及计数器递增/递减计数（中心对齐模式 3，CMS = “11”）。

此模式下无法写入方向位（TIMx\_CR1 寄存器中的 DIR 位）。而是由硬件更新并指示当前计数器方向。

每次发生计数器上溢和下溢时都会生成更新事件，或将 TIMx\_EGR 寄存器中的 UG 位置 1（通过软件或使用从模式控制器）也可以生成更新事件。这种情况下，计数器以及预分频器计数器将重新从 0 开始计数。

通过软件将 TIMx\_CR1 寄存器中的 UDIS 位置 1 可禁止 UEV 更新事件。这可避免向预装载寄存器写入新值时更新影子寄存器。在 UDIS 位写入 0 之前不会产生任何更新事件。不过，计数器仍会根据当前自动重载值进行递增和递减计数。

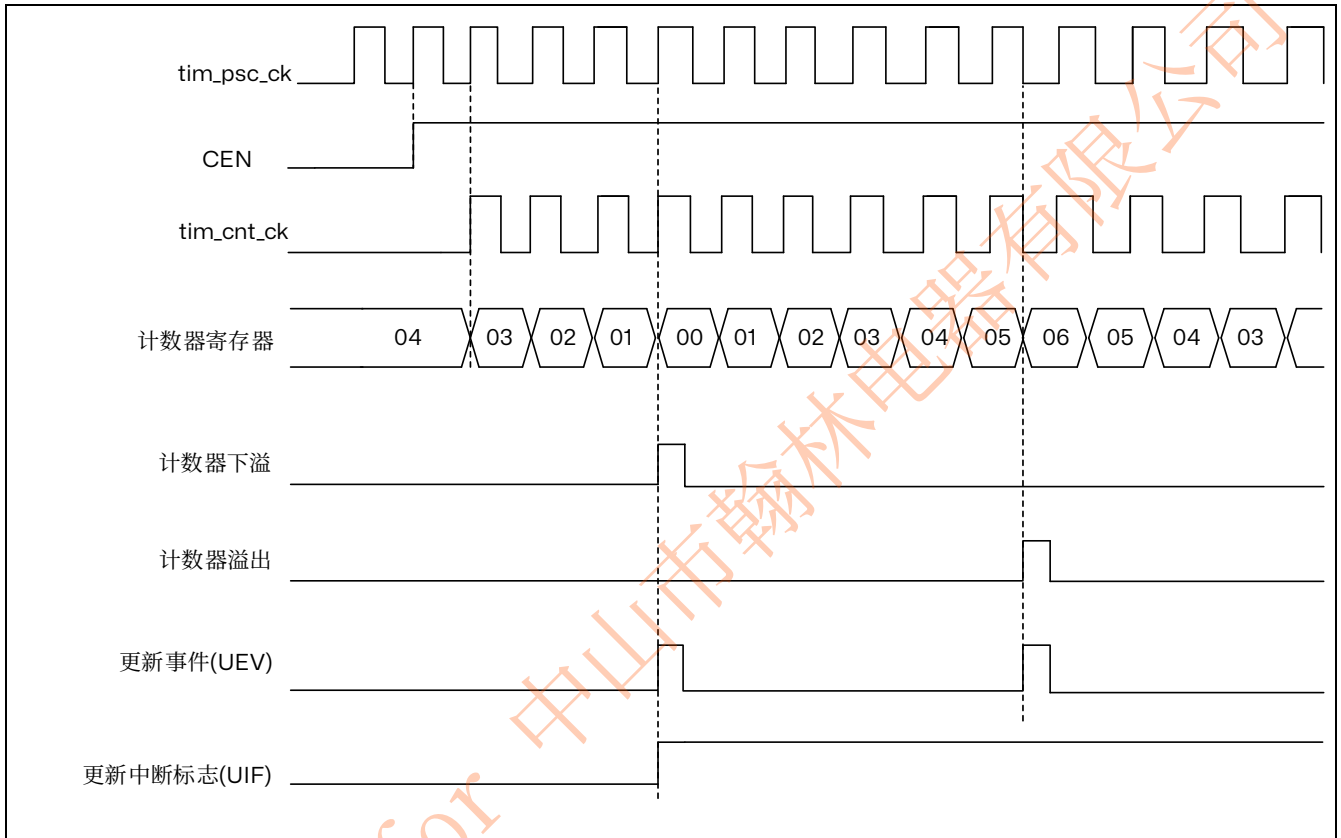
此外，如果 TIMx\_CR1 寄存器中的 URS 位（更新请求选择）已置 1，则将 UG 位置 1 会生成更新事件 UEV，但不会将 UIF 标志置 1（因此，不会发送任何中断请求或 DMA 请求）。这样一来，如果在发生

捕获事件时将计数器清零，将不会同时产生更新中断和捕获中断。

发生更新事件时，将更新所有寄存器且将更新标志（TIMx\_SR 寄存器中的 UIF 位）置 1（取决于 URS 位）：

- 预分频器的缓冲区中将重新装载预装载值（TIMx\_PSC 寄存器的内容）。
- 当自动重载活动寄存器将以预装载值（TIMx\_ARR 寄存器的内容）进行更新。注意，如果更新操作是由计数器上溢触发的，则自动重载寄存器在重载计数器之前更新，因此，下一个计数周期就是我们所希望的新的周期长度（计数器被重载新的值）。

以下各图以一些示例说明不同时钟频率下计数器的行为。



1. 这里使用了中心对齐模式 1

图 24.15 计数器时序图，内部时钟分频系数为 1，TIMx\_ARR=0x6

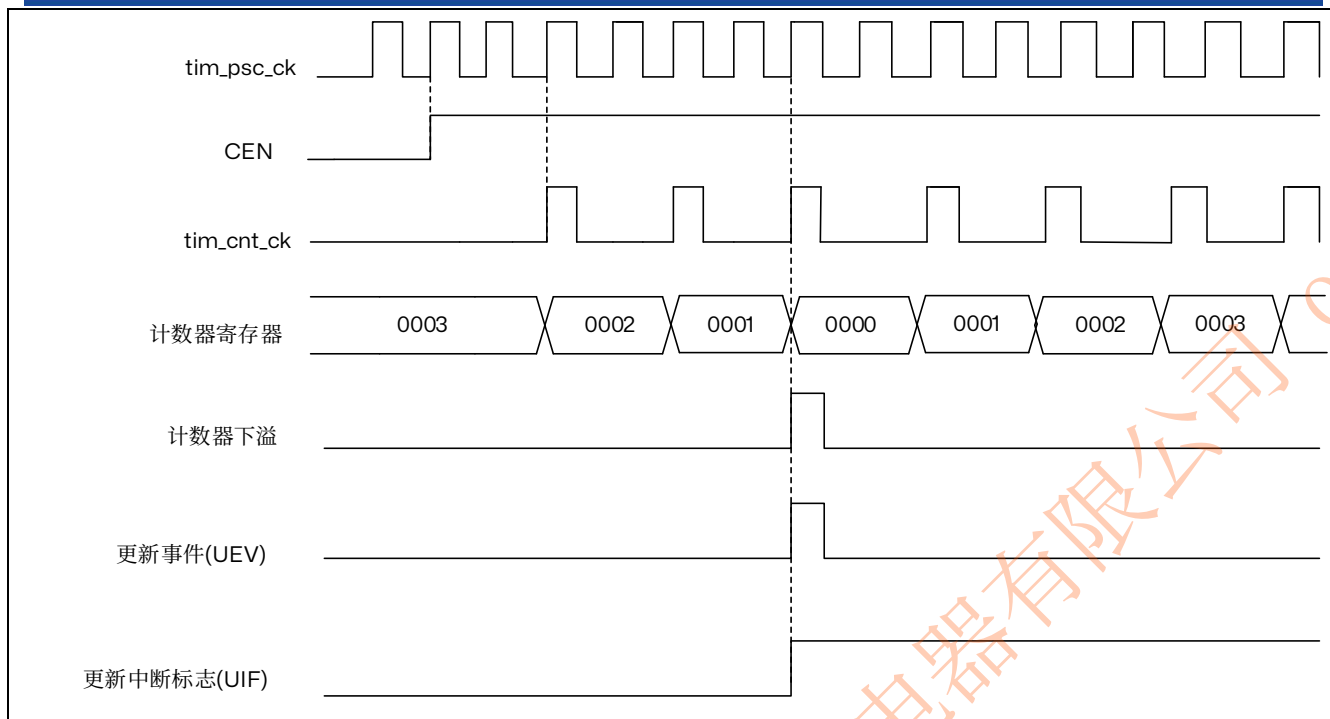


图 24.16 计数器时序图，内部时钟分频系数为 2

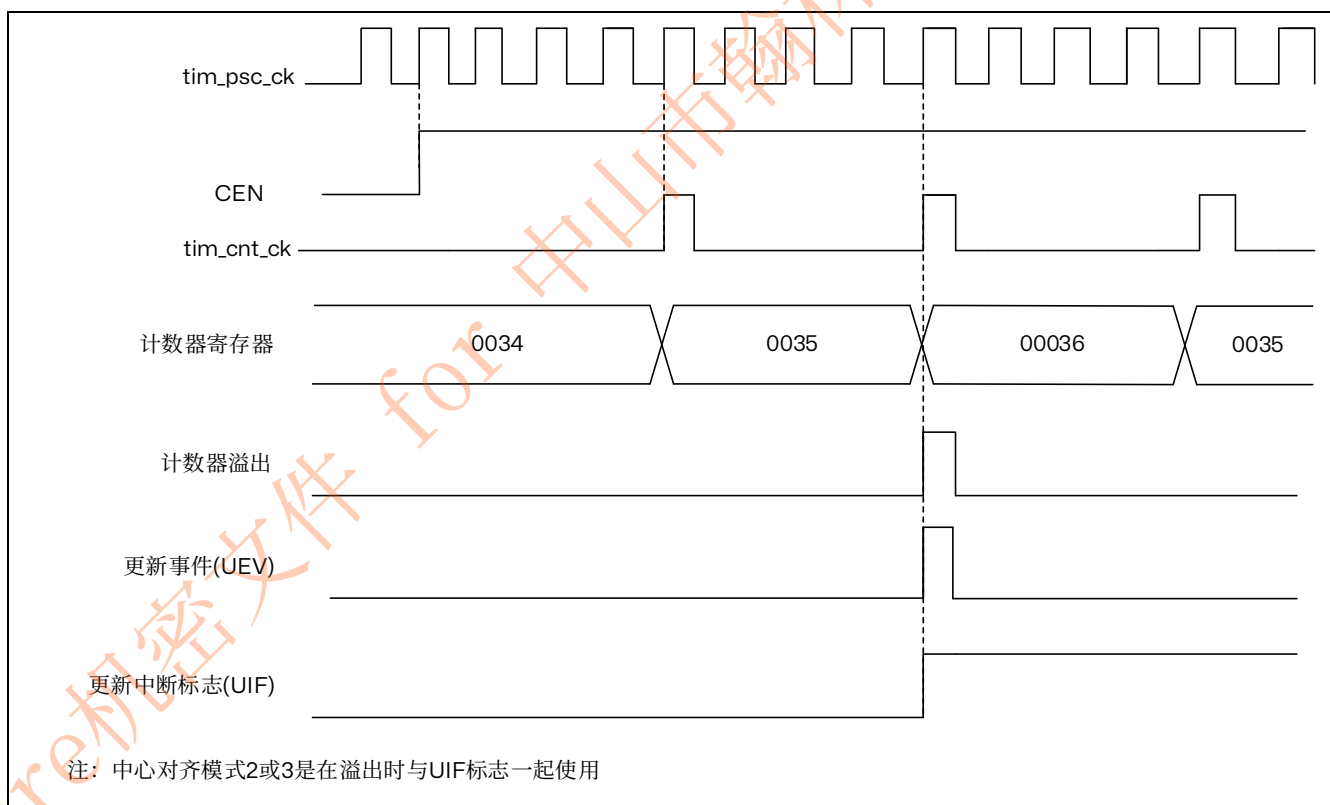


图 24.17 计数器时序图，内部时钟分频系数为 4，TIMx\_ARR=0x36

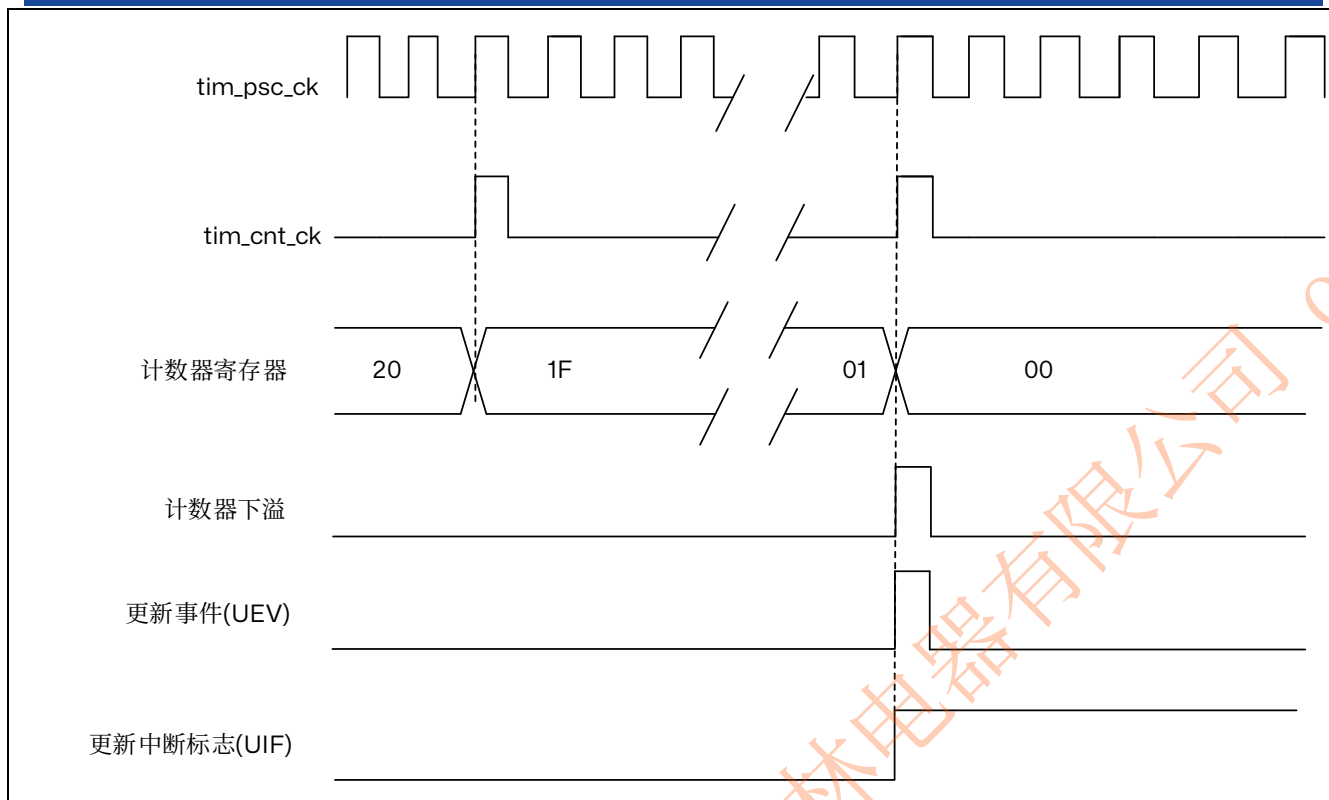


图 24.18 计数器时序图，内部时钟分频系数为 N

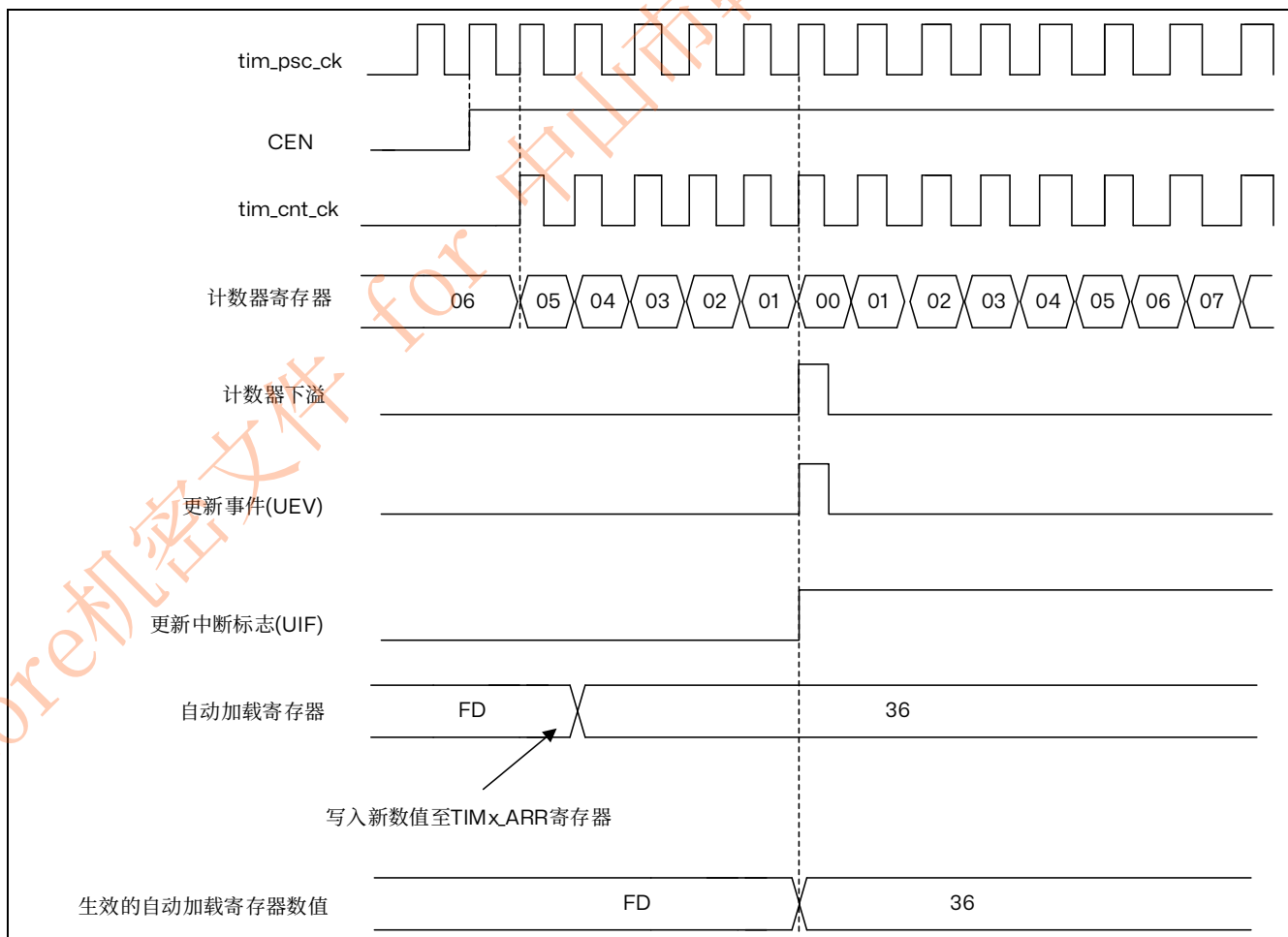


图 24.19 计数器时序图, ARPE=1 时的更新事件 (计数器下溢)

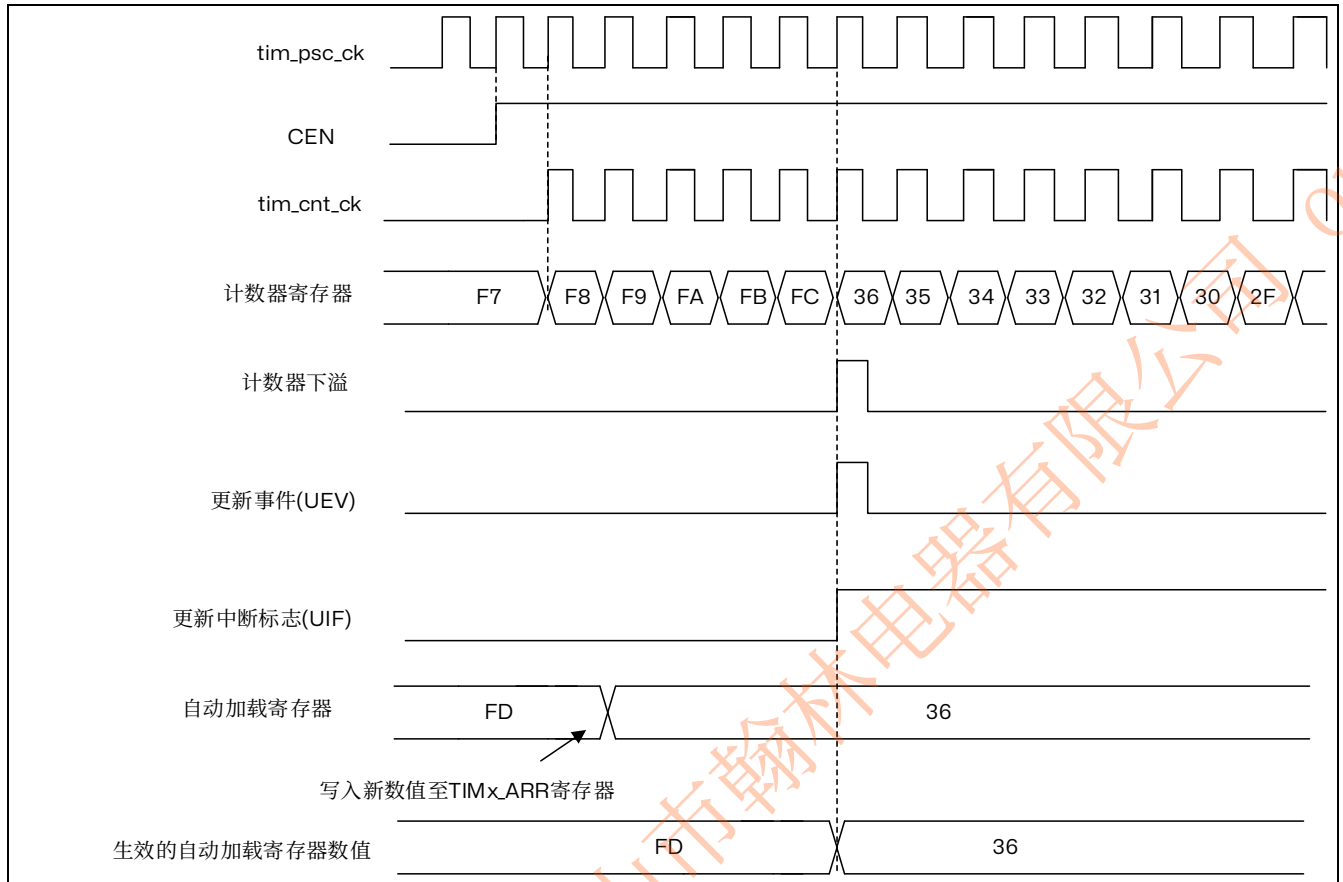


图 24.20 计数器时序图, ARPE=1 时的更新事件 (计数器溢出)

### 24.4.5 时钟选择

计数器时钟可由下列时钟源提供:

- 内部时钟 (tim\_ker\_ck)
- 外部时钟模式 1: 外部输入引脚 (tim\_ti1 or tim\_ti2)
- 外部时钟模式 2: 外部触发输入 ((tim\_etr\_in)
- 内部触发输入 (tim\_itr): 使用一个定时器作为另一个定时器的预分频器, 例如可以将定时器 8 配置为定时器 2 的预分频器。

#### 内部时钟源 (tim\_ker\_ck)

如果禁止从模式控制器 (TIMx\_SMCR 寄存器中 SMS=000), 则 CEN 位、DIR 位 (TIMx\_CR1 寄存器中) 和 UG 位 (TIMx\_EGR 寄存器中) 为实际控制位, 并且只能通过软件进行更改 (UG 除外, 仍自动清零)。当对 CEN 位写入 1 时, 预分频器的时钟就由内部时钟 tim\_ker\_ck 提供。

下图显示了正常模式下控制电路与递增计数器的行为 (没有预分频的情况下)。

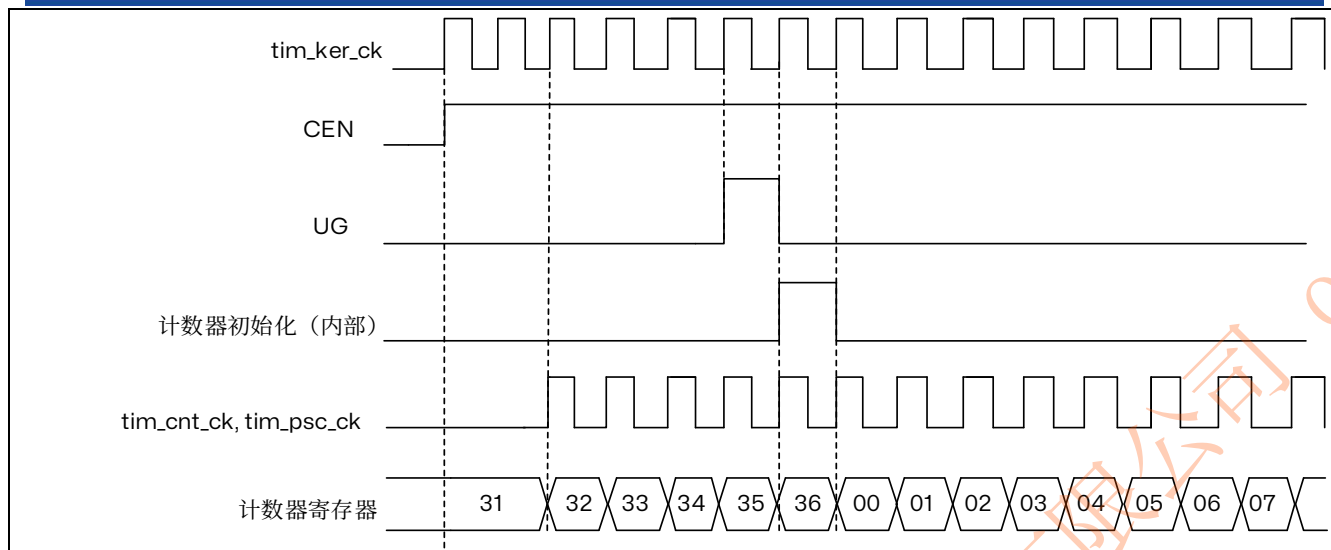


图 24.21 一般模式下的控制电路，内部时钟分频系数为 1

### 外部时钟源模式 1

当 TIMx\_SMCR 寄存器中的 SMS=111 时，可选择此模式。计数器可在选定的输入信号上出现上升沿或下降沿时计数。

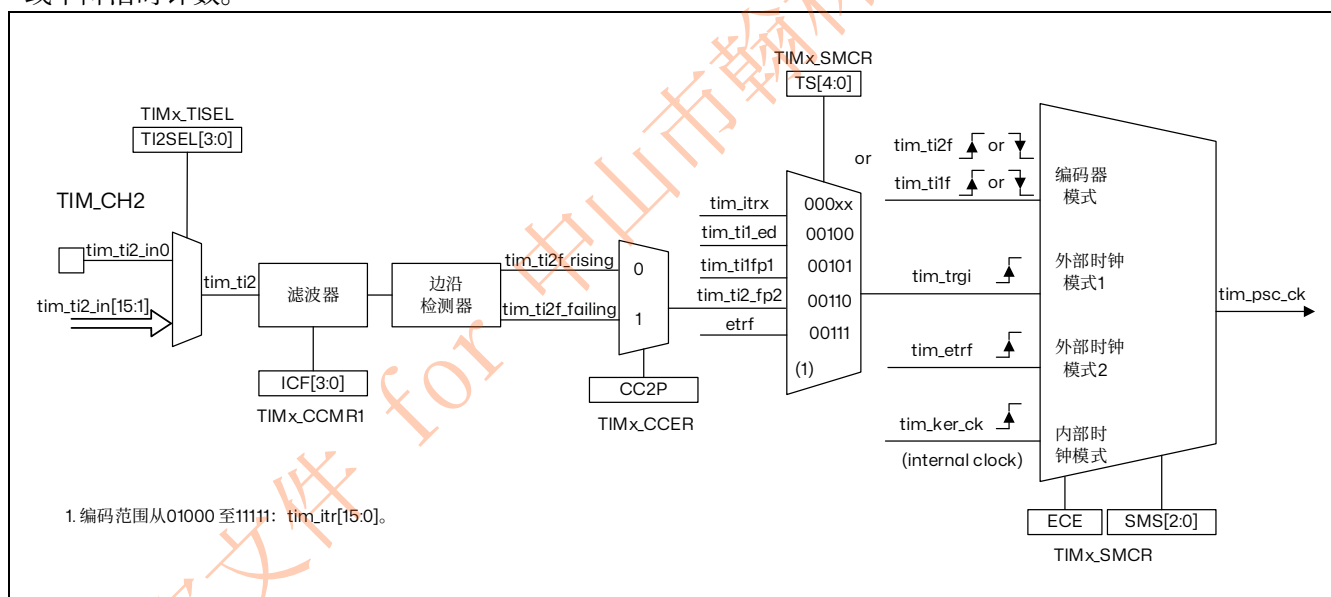


图 24.22 tim\_ti2 外部时钟连接例子

例如，要使递增计数器在 tim\_ti2 输入出现上升沿时计数，请执行以下步骤：

1. 通过在 TIMx\_TISEL 寄存器中使用 TI2SEL[3:0] 位选择合适的 tim\_ti2\_in[15:0] 源（内部或外部）。
2. 通过在 TIMx\_CCMR1 寄存器中写入 CC2S=“01”来配置通道 2，使其能够检测 tim\_ti2 输入的上升沿。
3. 通过在 TIMx\_CCMR1 寄存器中写入 IC2F[3:0] 位来配置输入滤波时间（如果不需要任何滤波，请保持 IC2F=0000）。

注：由于捕获预分频器不用于触发操作，因此无需对其进行配置。

4. 通过在 TIMx\_CCER 寄存器中写入 CC2P=0 和 CC2NP=0 来选择上升沿极性。



5. 通过在 TIMx\_SMCR 寄存器中写入 SMS=111，使定时器在外部时钟模式 1 下工作。
6. 通过在 TIMx\_SMCR 寄存器中写入 TS=110 来选择 tim\_ti2 作为输入源。
7. 通过在 TIMx\_CR1 寄存器中写入 CEN=1 来使能计数器。

当 tim\_ti2 出现上升沿时，计数器便会计数一次并且 TIF 标志置 1。

tim\_ti2 的上升沿与实际计数器时钟之间的延迟是由于 tim\_ti2 输入的重新同步电路引起的。

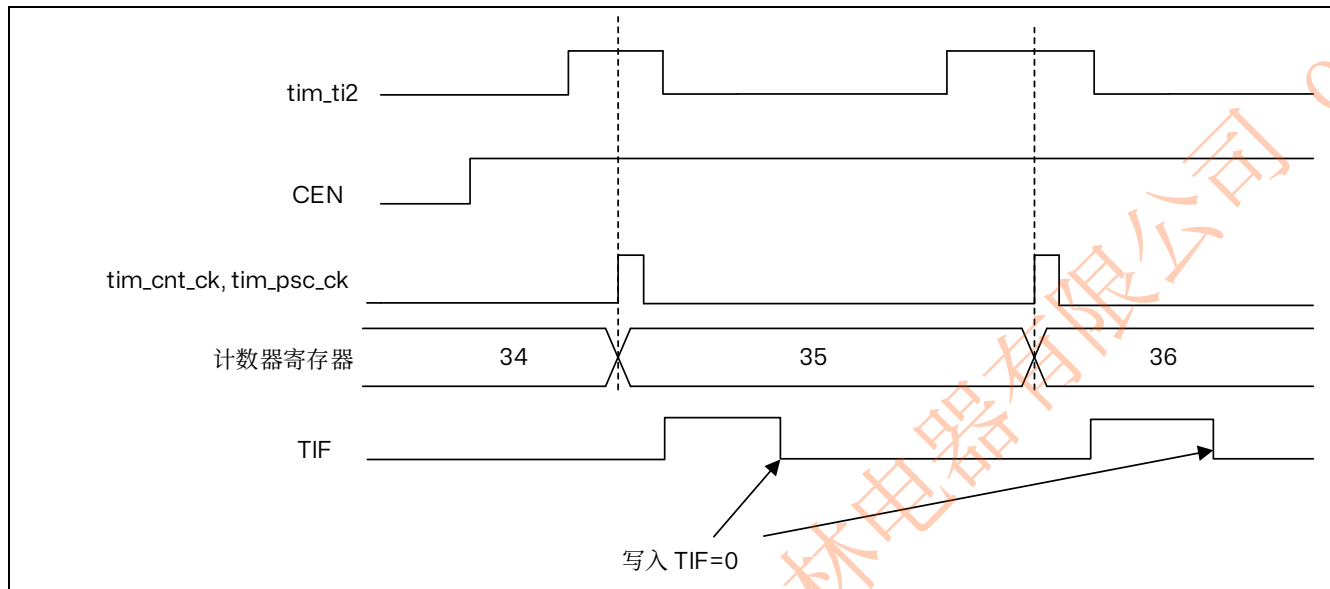


图 24.23 外部时钟模式 1 下的控制电路

## 外部时钟源模式 2

通过在 TIMx\_SMCR 寄存器中写入 ECE=1 选择此模式。

计数器可在外部触发输入 ETR 出现上升沿或下降沿时计数。

下图是外部触发输入的框图

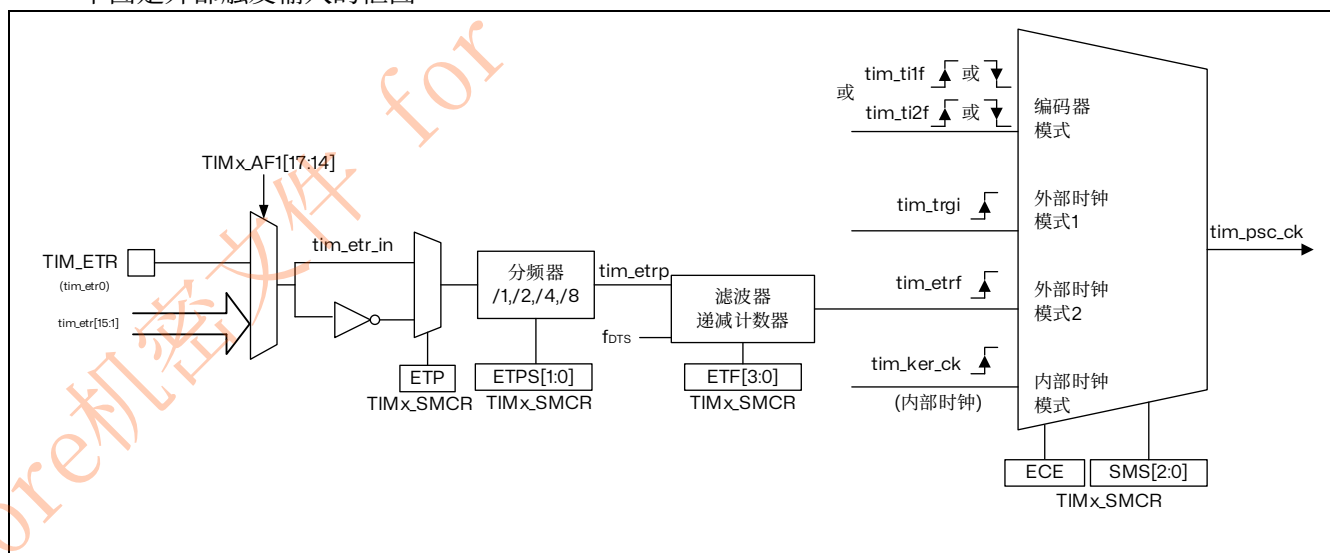


图 24.24 外部触发输入框图

例如，要使递增计数器在 ETR 每出现 2 个上升沿时计数，请执行以下步骤：

1. 通过在 TIMx\_AF1 寄存器中使用 ETRSEL[3:0] 位选择合适的 tim\_etr\_in 源（内部或外部）。

2. 由于此例中不需滤波器，因此在 TIMx\_SMCR 寄存器中写入 ETF[3:0]=0000。
3. 通过在 TIMx\_SMCR 寄存器中写入 ETPS[1:0]=01 来设置预分频器。
4. 通过在 TIMx\_SMCR 寄存器中写入 ETP=0 来选择 tim\_etr\_in 引脚的上升沿检测。
5. 通过在 TIMx\_SMCR 寄存器中写入 ECE=1 来使能外部时钟模式 2。
6. 通过在 TIMx\_CR1 寄存器中写入 CEN=1 来使能计数器。

tim\_etr\_in 每出现 2 个上升沿，计数器计数一次。

在 tim\_etr\_in 的上升沿和计数器实际时钟之间的延时取决于在 tim\_etrp 信号端的重新同步电路。因此，计数器可以正确捕获的最大频率最高为 TIMxCLK 频率的 $\frac{1}{4}$ 。当 ETRP 信号更快时，用户应通过适当的 ETPS 预分频器设置对外部信号进行分频。

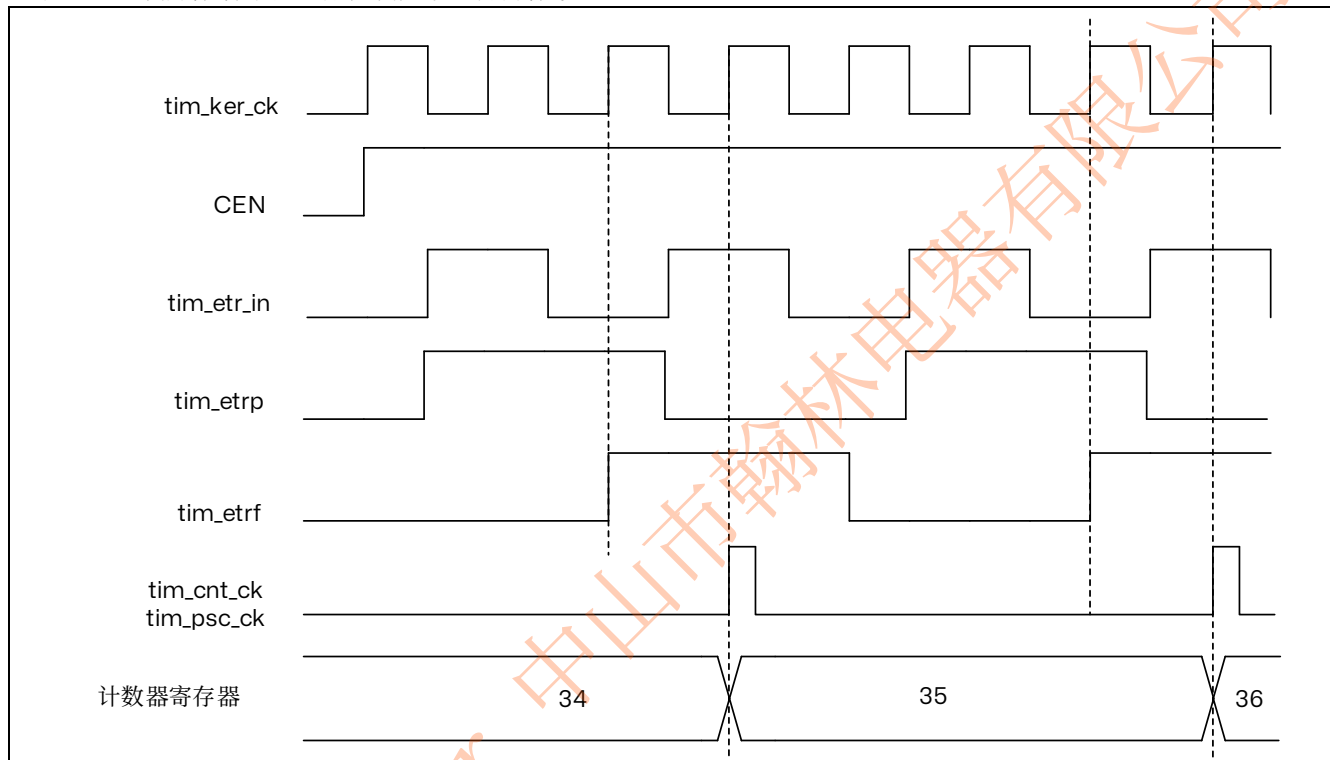


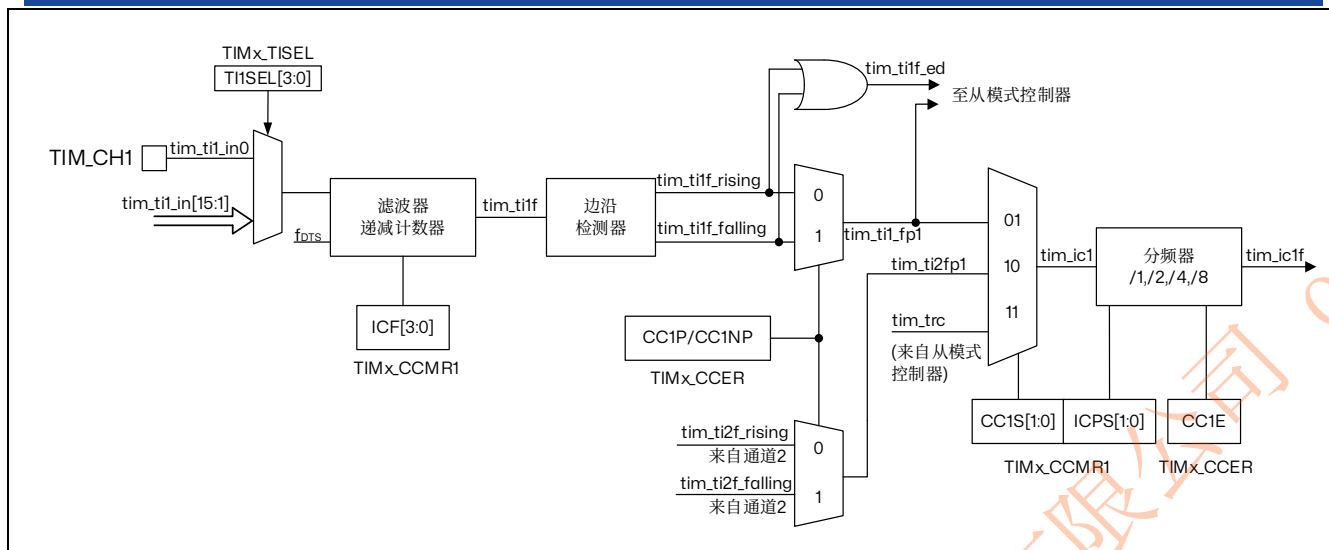
图 24.25 外部时钟模式 2 下的控制电路

#### 24.4.6 捕获/比较通道

每个捕获/比较通道均围绕一个捕获/比较寄存器（包括一个影子寄存器）、一个捕获输入阶段（数字滤波、多路复用和预分频器）和一个输出阶段（比较器和输出控制）构建而成。

下图概括介绍了一个捕获/比较通道。

输入阶段对相应的 tim\_tix 输入进行采样，生成一个滤波后的信号 tim\_tixf。然后，带有极性选择功能的边沿检测器生成一个信号（tim\_tixfpy），该信号可用作从模式控制器的触发输入，也可用作捕获命令。该信号先进行预分频（ICxPS），而后再进入捕获寄存器。



输出阶段生成一个中间波形作为基准：OCxRef（高电平有效）。链的末端决定最终输出信号的极性。

图 24.26 捕获/比较通道（如：通道 1 输入部分）

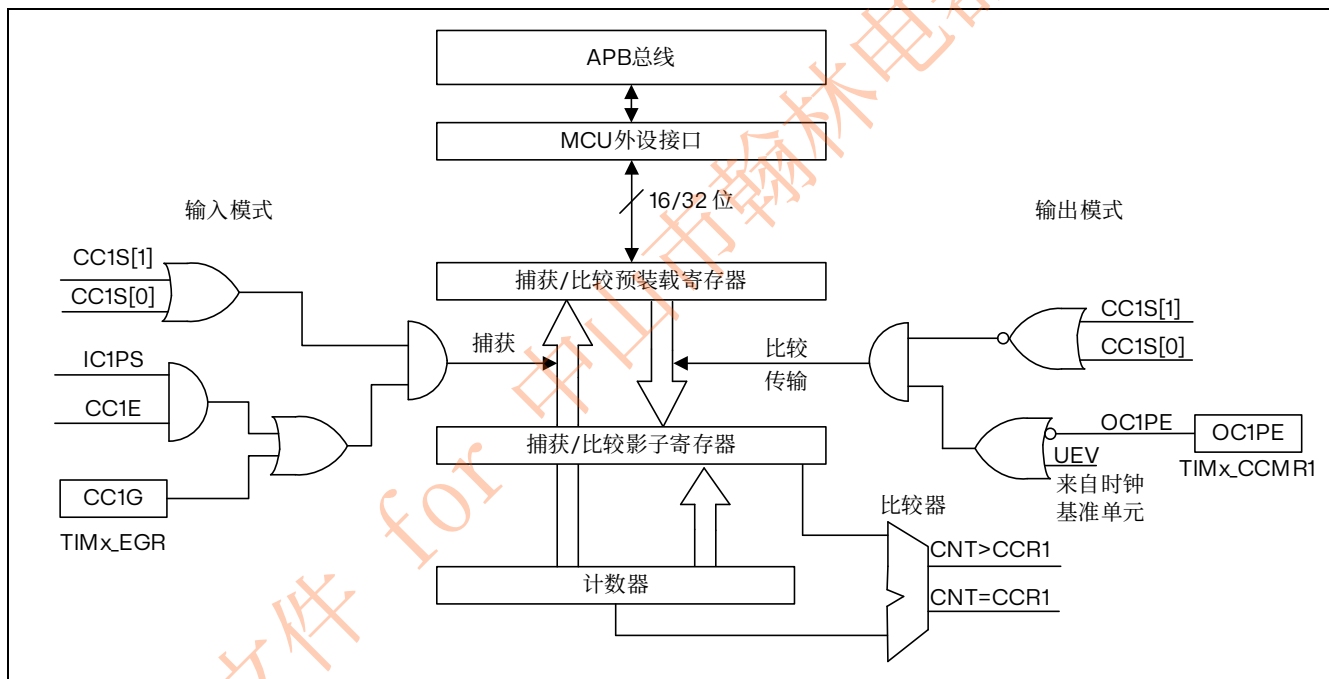


图 24.27 捕获/比较通道 1 的主电路

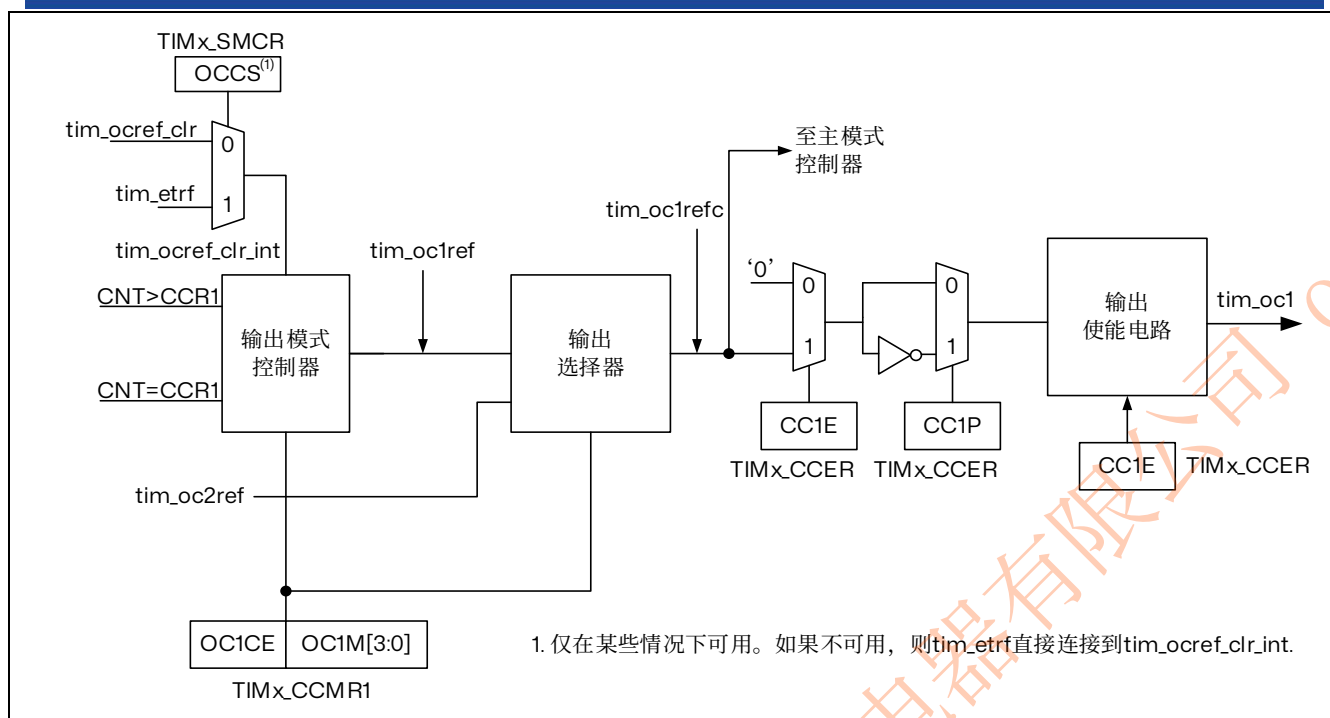


图 24.28 捕获/比较通道的输出阶段（通道 1，通道 2、3 和 4 同上）

捕获/比较模块由一个预装载寄存器和一个影子寄存器组成。始终可通过读写操作访问预装载寄存器。

在捕获模式下，捕获实际发生在影子寄存器中，然后将影子寄存器的内容复制到预装载寄存器中。

在比较模式下，预装载寄存器的内容将复制到影子寄存器中，然后将影子寄存器的内容与计数器进行比较。

### 24.4.7 输入捕获模式

在输入捕获模式下，当相应的 ICx 信号检测到跳变沿后，将使用捕获/比较寄存器(TIMx\_CCRx) 来锁存计数器的值。发生捕获事件时，会将相应的 CCxIF 标志 (TIMx\_SR 寄存器) 置 1，并可发送中断请求（如果已使能）。如果发生捕获事件时 CCxIF 标志已处于高位，则会将重复捕获标志 CCxOF (TIMx\_SR 寄存器) 置 1。可通过软件向 CCxIF 写入 0 来给 CCxIF 清零，或读取存储在 TIMx\_CCRx 寄存器中的已捕获数据。向 CCxOF 写入 0 后会将其清零。

以下示例说明了如何在 tim\_ti1 输入出现上升沿时将计数器的值捕获到 TIMx\_CCR1 中。具体操作步骤如下：

1. 通过在 TIMx\_TISEL 寄存器中使用 TI1SEL[3:0] 位选择合适的 tim\_tix\_in[15:0] 源（内部或外部）。
2. 选择有效输入：TIMx\_CCR1 必须连接到 tim\_ti1 输入，因此向 TIMx\_CCMR1 寄存器中的 CC1S 位写入 01。只要 CC1S 不等于 00，就会将通道配置为输入模式，并且 TIMx\_CCR1 寄存器将处于只读状态。
3. 根据连接到定时器的信号，对所需的输入滤波时间进行编程（如果输入为 tim\_tix 输入之一，则对 TIMx\_CCMRx 寄存器中的 ICxF 位进行编程）。假设信号变化时，输入信号最多在 5 个内部时钟周期内发生抖动。因此，我们必须将滤波时间设置为大于 5 个内部时钟周期。在检测到 8 个具有新电平的连续采样（以 f<sub>DTs</sub> 频率采样）后，可以确认 tim\_ti1 上的跳变沿。然后向 TIMx\_CCMR1 寄存器中的 IC1F 位写入 0011。
4. 通过向 TIMx\_CCER 寄存器中的 CC1P 位和 CC1NP 位写入 0，选择 tim\_ti1 通道的有效转换边沿。

(本例中为上升沿)。

5. 对输入预分频器进行编程。在本例中，我们希望每次有效转换时都执行捕获操作，因此需要禁止预分频器（向 TIMx\_CCMR1 寄存器中的 IC1PS 位写入 00）。
6. 通过将 TIMx\_CCER 寄存器中的 CC1E 位置 1，允许将计数器的值捕获到捕获寄存器中。
7. 如果需要，可通过将 TIMx\_DIER 寄存器中的 CC1IE 位置 1 来使能相关中断请求。

发生输入捕获时：

- 发生有效跳变沿时，TIMx\_CCR1 寄存器会获取计数器的值。
- 将 CC1IF 标志置 1（中断标志）。如果至少发生了两次连续捕获，但 CC1IF 标志未被清零，这样 CC1OF 捕获溢出标志会被置 1。
- 根据 CC1IE 位生成一个中断。
- 根据 CC1DE 位生成一个 DMA 请求。

要处理重复捕获，建议在读出捕获溢出标志之前读取数据。这样可避免丢失在读取捕获溢出标志之后与读取数据之前可能出现的重复捕获信息。

注：通过软件将 TIMx\_EGR 寄存器中的相应 CCxG 位置 1 可生成 IC 中断请求或 DMA 请求。

#### 24.4.8 PWM 输入模式

此模式允许测量连接到单个 tim\_tix 输入的 PWM 信号的周期和占空比：

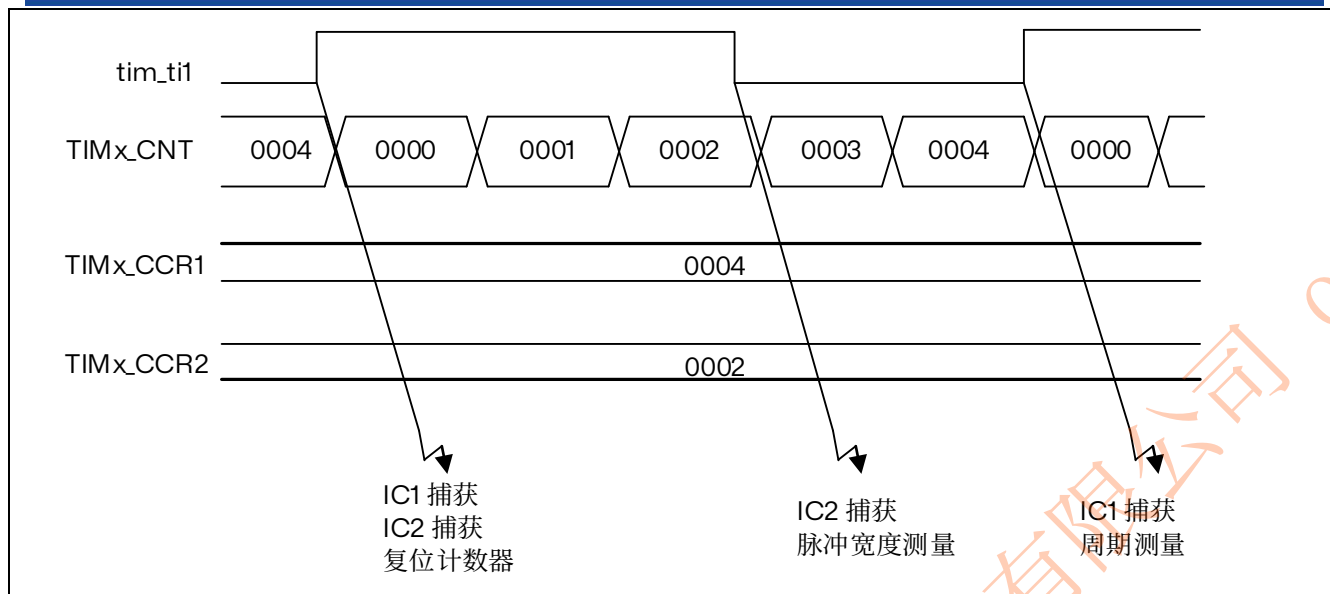
- TIMx\_CCR1 寄存器保持周期值（两个连续上升沿之间的间隔）。
- TIMx\_CCR2 寄存器保持脉冲宽度（两个连续上升沿和下降沿之间的间隔）。

此模式是输入捕获模式的一个特例。其实现步骤与输入捕获模式基本相同，仅存在以下不同之处：

- 两个 ICx 信号被映射至同一个 tim\_tix 输入。
- 这两个 ICx 信号在边沿处有效，但极性相反。
- 选择两个 TIxFP 信号之一作为触发输入，并将从模式控制器配置为复位模式。

可通过以下步骤测量施加在 tim\_ti1 上的 PWM 信号的周期和脉冲宽度：

1. 通过在 TIMx\_TISEL 寄存器中使用 TI1SEL[3:0] 位选择合适的 tim\_tix\_in[15:0] 源（内部或外部）。
2. 选择 TIMx\_CCR1 的有效输入：向 TIMx\_CCMR1 寄存器中的 CC1S 位写入 01（选择 tim\_ti1）。
3. 选择 tim\_ti1fp1 的有效极性（用于 TIMx\_CCR1 中的捕获和计数器清零）：向 CC1P 位和 CC1NP 位写入 “0”（上升沿有效）。
4. 选择 TIMx\_CCR2 的有效输入：向 TIMx\_CCMR1 寄存器中的 CC2S 写入 10（选择 tim\_ti1）。
5. 选择 tim\_ti1fp2 的有效极性（用于 TIMx\_CCR2 中的捕获）：向 CC2P 位写入 “1” 和 CC2NP 位写入 “0”（下降沿有效）。
6. 选择有效触发输入：向 TIMx\_SMCR 寄存器中的 TS 位写入 101（选择 tim\_ti1fp1）。
7. 将从模式控制器配置为复位模式：向 TIMx\_SMCR 寄存器中的 SMS 位写入 100。
8. 使能捕获：向 TIMx\_CCER 寄存器中的 CC1E 位和 CC2E 位写入 “1”。



PWM 输入模式只能与 TIMx\_CH1/TIMx\_CH2 信号一起使用，因为只有 tim\_tilfp1 和 tim\_tilfp2 连接到从模式控制器。

图 24.29 PWM 输入模式时序

#### 24.4.9 强制输出模式

在输出模式（TIMx\_CCMRx 寄存器中的 CCxS 位 = 00）下，可直接由软件将每个输出比较信号（tim\_ocxref 和 tim\_ocx）强制设置为有效电平或无效电平，而无需考虑输出比较寄存器和计数器之间的任何比较结果。

要将输出比较信号（tim\_ocxref/tim\_ocx）强制设置为有效电平，只需向相应 TIMx\_CCMRx 寄存器中的 OCxM 位写入 101。tim\_ocxref 进而强制设置为高电平（tim\_ocxref 始终为高电平有效），同时 tim\_ocx 获取 CCxP 极性位的相反值。

例如：CCxP=0（tim\_ocx 高电平有效）=> tim\_ocx 强制设置为高电平。

通过向 TIMx\_CCMRx 寄存器中的 OCxM 位写入 100，可将 tim\_ocxref 信号强制设置为低电平。

无论如何，TIMx\_CCRx 影子寄存器与计数器之间的比较仍会执行，而且允许将标志置 1。因此可发送相应的中断请求和 DMA 请求。输出比较模式一节对此进行了介绍。

#### 24.4.10 输出比较模式

此功能用于控制输出波形，或指示已经过某一段时间。

当捕获/比较寄存器与计数器之间相匹配时，输出比较功能：

- 将为相应的输出引脚分配一个可编程值，该值由输出比较模式（TIMx\_CCMRx 寄存器中的 OCxM 位）和输出极性（TIMx\_CCER 寄存器中的 CCxP 位）定义。匹配时，输出引脚既可保持其电平（OCxM=000），也可设置为有效电平（OCxM=001）、无效电平（OCxM=010）或进行翻转（OCxM=011）。
- 将中断状态寄存器中的标志置 1（TIMx\_SR 寄存器中的 CCxIF 位）。
- 如果相应中断使能位（TIMx\_DIER 寄存器中的 CCXIE 位）置 1，将生成中断。
- 如果相应 DMA 使能位（TIMx\_DIER 寄存器的 CCxDE 位，TIMx\_CR2 寄存器的 CCDS 位，用来选择 DMA 请求）置 1，将发送 DMA 请求。

使用 TIMx\_CCMRx 寄存器中的 OCxPE 位，可将 TIMx\_CCRx 寄存器配置为带或不带预装载寄存



器。

在输出比较模式下，更新事件 UEV 对 tim\_ocxref 和 tim\_ocx 输出毫无影响。同步的精度可以达到计数器的一个计数周期。输出比较模式也可用于输出单脉冲（在单脉冲模式下）。

步骤：

1. 选择计数器时钟（内部、外部、预分频器）。
2. 在 TIMx\_ARR 和 TIMx\_CCRx 寄存器中写入所需数据。
3. 如果要生成中断请求，将 CCxIE 位和/或 CCxDE 位置 1。
4. 选择输出模式。例如，
  - a) 写入 OCxM=0011 以在 CNT 匹配 CCRx 时翻转 tim\_ocx 输出引脚
  - b) 写入 OCxPE=0 以禁用预装载寄存器
  - c) 写入 CCxP=0 以选择高电平有效
  - d) 写入 CCxE=1 以使能输出
5. 通过将 TIMx\_CR1 寄存器中的 CEN 位置 1 来使能计数器。

可随时通过软件更新 TIMx\_CCRx 寄存器以控制输出波形，前提是未使能预装载寄存器（OCxPE=0，否则仅当发生下一个更新事件 UEV 时，才会更新 TIMx\_CCRx 影子寄存器）。

下图列出了相关示例。

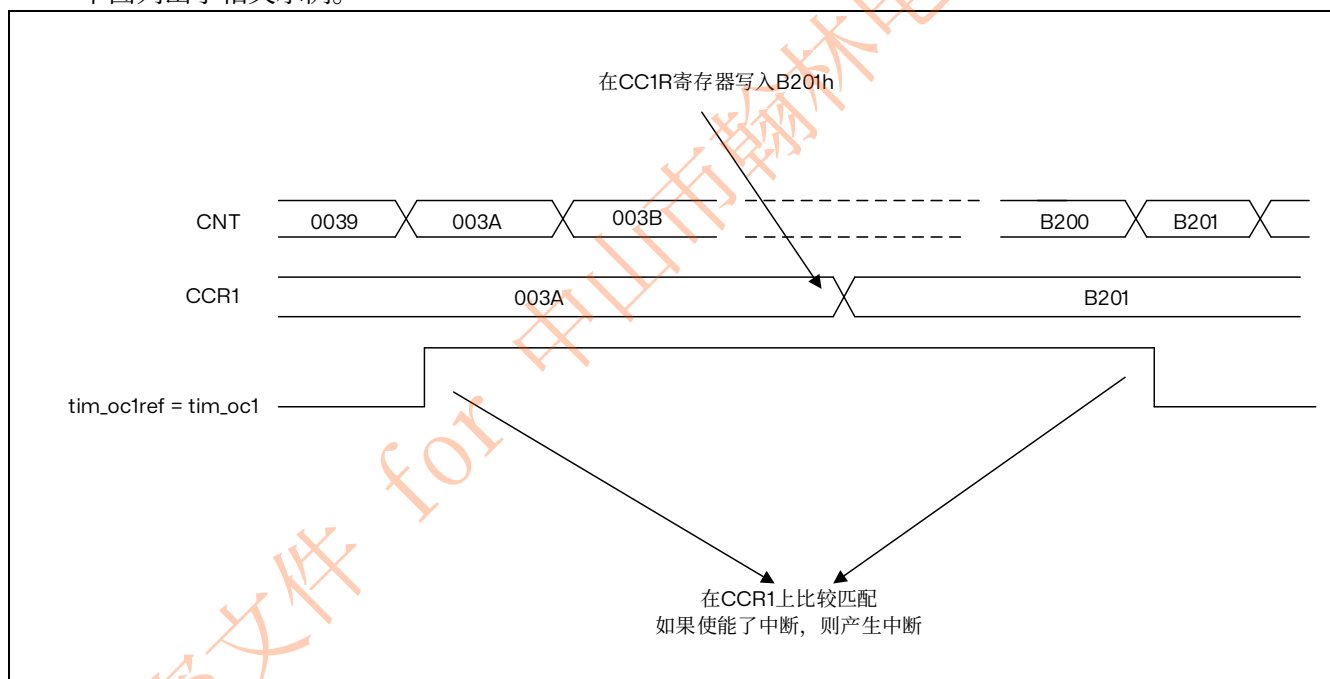


图 24.30 输出比较模式，翻转 tim\_oc1

## 24.4.11 PWM 模式

脉冲宽度调制模式可以生成一个信号，该信号频率由 TIMx\_ARR 寄存器值决定，其占空比则由 TIMx\_CCRx 寄存器值决定。

通过向 TIMx\_CCMRx 寄存器中的 OCxM 位写入 110（PWM 模式 1）或 111（PWM 模式 2），可以独立选择各通道（每个 tim\_ocx 输出对应一个 PWM）的 PWM 模式。必须通过将 TIMx\_CCMRx 寄存器中的 OCxPE 位置 1 使能相应预装载寄存器，最后通过将 TIMx\_CR1 寄存器中的 ARPE 位置 1 使能自动重载预装载寄存器。



由于只有在发生更新事件时预装载寄存器才会传送到影子寄存器，因此启动计数器之前，必须通过将 TIMx\_EGR 寄存器中的 UG 位置 1 来初始化所有寄存器。

tim\_ocx 极性可使用 TIMx\_CCER 寄存器的 CCxP 位来编程。既可以设为高电平有效，也可以设为低电平有效。tim\_ocx 输出通过将 TIMx\_CCER 寄存器中的 CCxE 位置 1 来使能。有关详细信息，请参见 TIMx\_CCERx 寄存器说明。

在 PWM 模式（1 或 2）下，TIMx\_CNT 始终与 TIMx\_CCRx 进行比较，以确定是  $TIMx\_CCRx \leq TIMx\_CNT$  还是  $TIMx\_CNT \leq TIMx\_CCRx$ （取决于计数器计数方向）。外部事件可以通过 tim\_etr\_in 或 tim\_oceref\_clr 信号。在本例中仅声明了 tim\_ocref\_clr 信号

- 比较匹配事件之后
- 输出比较模式（TIMx\_CCMRx 寄存器中的 OCxM 位）从“冻结”配置（不进行比较，OCxM=“000”）切换为任一 PWM 模式（OCxM=“110”或“111”）。定时器运行期间，可以通过软件强制 PWM 输出。

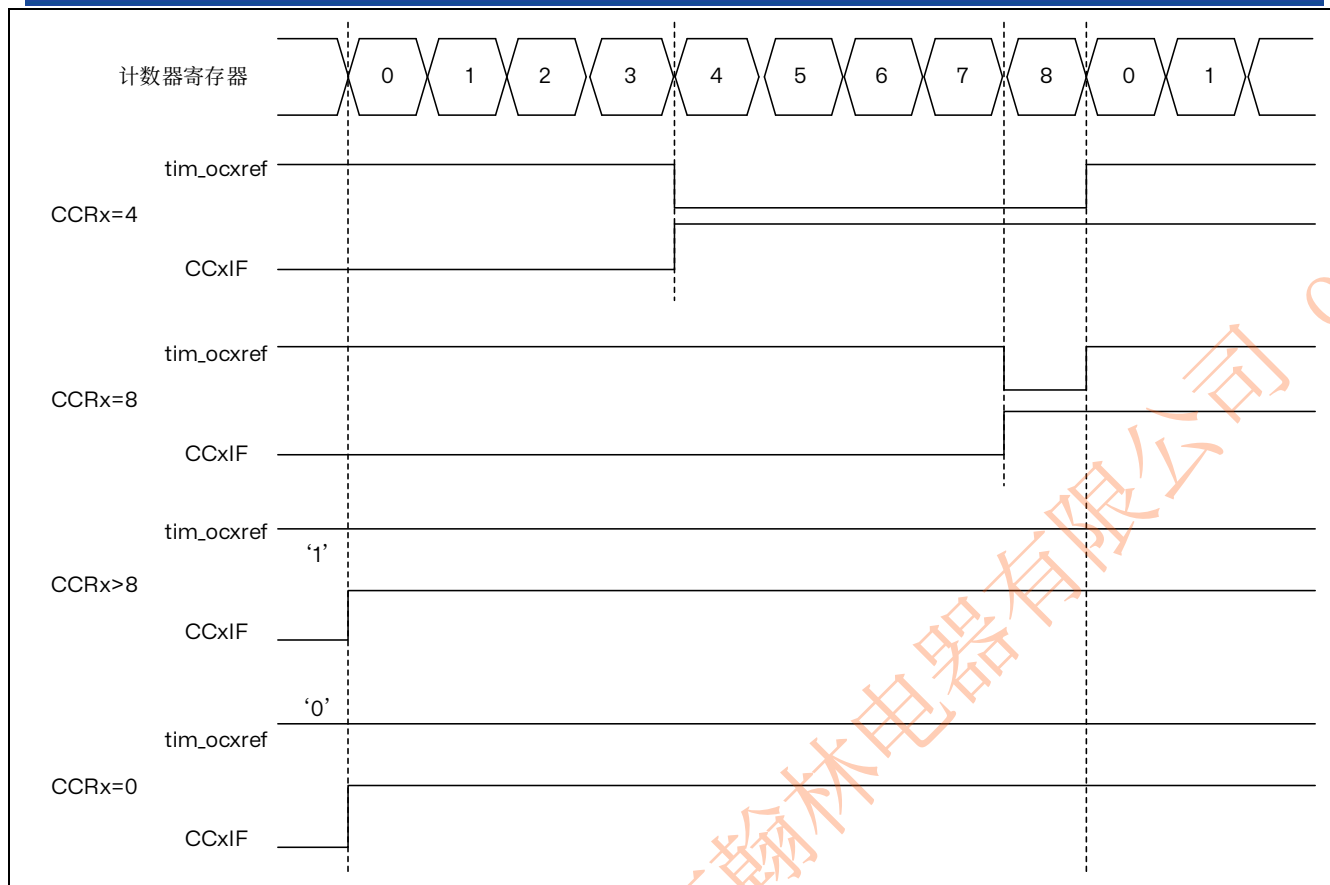
根据 TIMx\_CR1 寄存器中的 CMS 位状态，定时器能够产生边沿对齐模式或中心对齐模式的 PWM 信号。

### PWM 边沿对齐模式

#### 递增计数配置

当 TIMx\_CR1 寄存器中的 DIR 位为低时执行递增计数。

以下以 PWM 模式 1 为例。只要  $TIMx\_CNT < TIMx\_CCRx$ ，PWM 参考信号 tim\_ocxref 便为高电平，否则为低电平。如果 TIMx\_CCRx 中的比较值大于自动重载值（TIMx\_ARR 中），则 tim\_ocxref 保持为“1”。如果比较值为 0，则 tim\_ocxref 保持为“0”。下图举例介绍边沿对齐模式的一些 PWM 波形（TIMx\_ARR=8）。


图 24.31 边沿对齐的 PWM 波形 ( $ARR=8$ )

### 递减计数配置

当  $TIMx\_CR1$  寄存器中的  $DIR$  位为高时执行递减计数。

在 PWM 模式 1 下，只要  $TIMx\_CNT > TIMx\_CCR_x$ ，参考信号  $tim\_ocxref$  便为低电平，否则为高电平。如果  $TIMx\_CCR_x$  中的比较值大于  $TIMx\_ARR$  中的自动重载值，则  $tim\_ocxref$  保持为“1”。此模式下不可能产生 0% 的 PWM 波形。

### PWM 中央对齐模式

当  $TIMx\_CR1$  寄存器中的  $CMS$  位不为“00”时（其余所有配置对  $tim\_ocxref/tim\_ocx$  信号具有相同的作用），中心对齐模式生效。根据  $CMS$  位的配置，可以在计数器递增计数、递减计数或同时递增和递减计数时将比较标志置 1。 $TIMx\_CR1$  寄存器中的方向位 ( $DIR$ ) 由硬件更新，不得通过软件更改。

下图显示了中心对齐模式的 PWM 波形，在此例中：

- $TIMx\_ARR=8$
- PWM 模式为 PWM 模式 1
- 在根据  $TIMx\_CR1$  寄存器中  $CMS=01$  而选择的中心对齐模式 1 下，当计数器递减计数时，比较标志置 1。

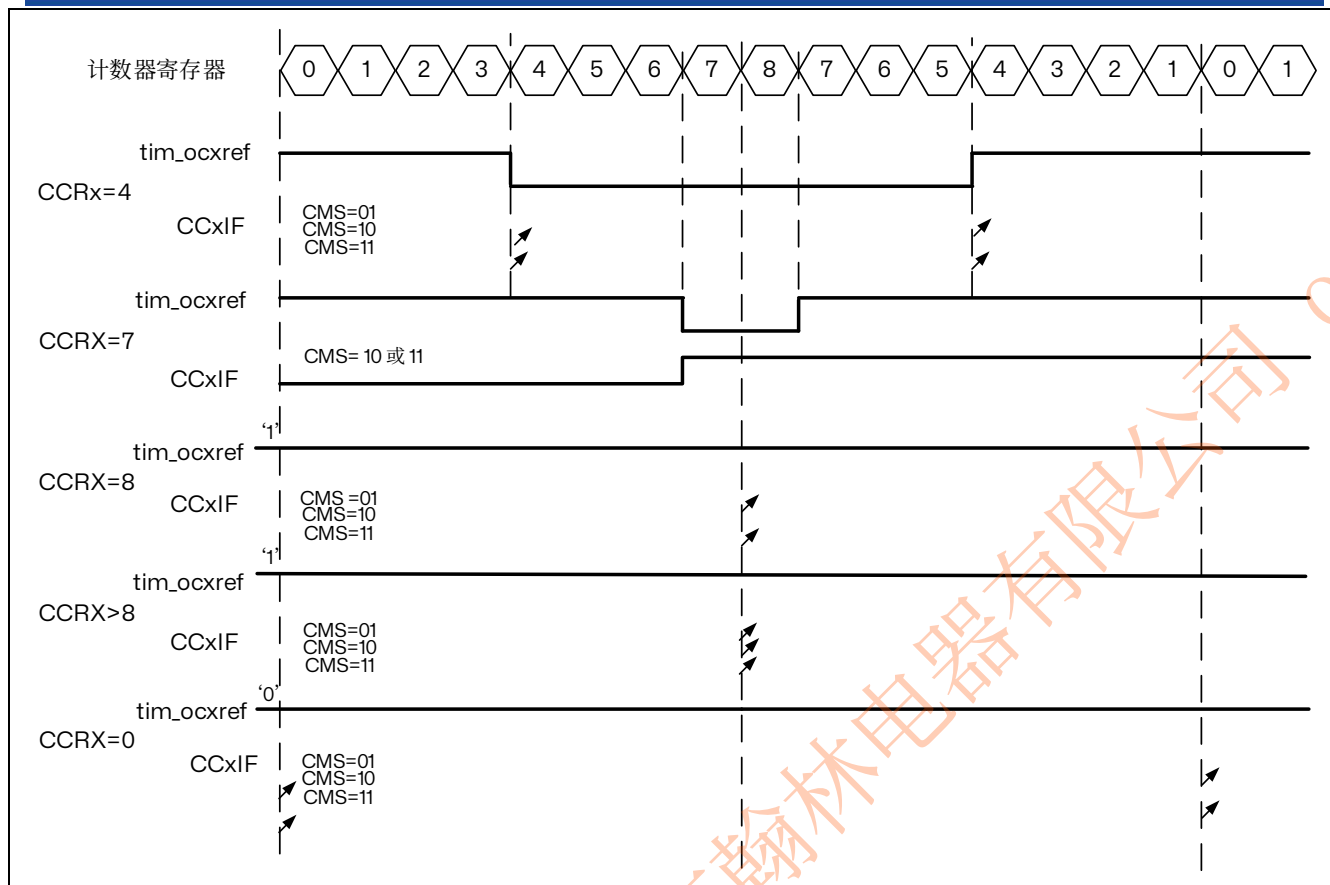


图 24.32 中央对齐的 PWM 波形 (APR=8)

### 中心对齐模式使用建议：

- 启动中心对齐模式时将使用当前的递增/递减计数配置。这意味着计数器将根据写入 TIMx\_CR1 寄存器中 DIR 位的值进行递增或递减计数。此外，不得同时通过软件修改 DIR 和 CMS 位。
- 不建议在运行中心对齐模式时对计数器执行写操作，否则将发生意想不到的结果。尤其是：
  - 如果写入计数器中的值大于自动重载值 (TIMx\_CNT > TIMx\_ARR)，计数方向不会更新。例如，如果计数器之前递增计数，则继续递增计数。
  - 如果向计数器写入 0 或 TIMx\_ARR 的值，计数方向会更新，但不生成更新事件 UEV。
- 使用中心对齐模式最为保险的方法是：在启动计数器前通过软件生成更新（将 TIMx\_EGR 寄存器中的 UG 位置 1），并且不要在计数器运行过程中对其执行写操作。

### 抖动模式

通过使能抖动模式，使用 TIMx\_CR1 寄存器中的 DITHEN 位，可以提高 PWM 模式的有效分辨率。这既适用于 CCR（用于增加占空比分辨率），也适用于 ARR（用于增加 PWM 频率分辨率）。

工作原理是在 16 个连续的 PWM 周期中，以预定义的模式使实际的 CCR（或 ARR）值略有变化（增加或减少一个定时器时钟周期）。这使得占空比或 PWM 期间的平均值提高了 16 倍分辨率。下图展示了应用于 4 个连续 PWM 周期的抖动原理。

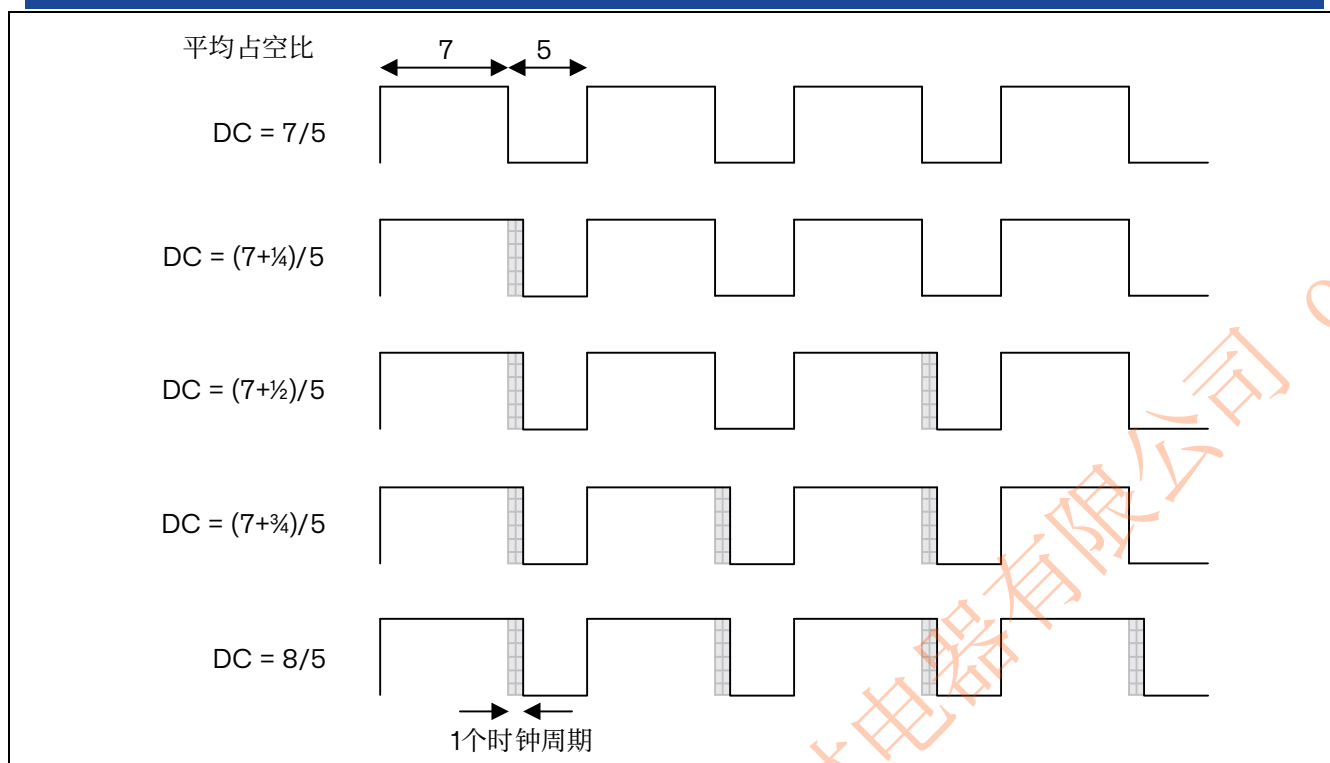


图 24.33 抖动原理

当抖动模式被使能时，寄存器编码将如下所示进行变化（参见下图示例）

- 4个最低有效位（LSB）用于编码增强的分辨率部分（小数部分）
- 最高有效位（MSB）左移至 19:4 位，用于编码基本值

注：如果设置清除 DITHEN 位，ARR 和 CCR 值将自动更新。（例如，如果 ARR=0x05 且 DITHEN=0，它将更新为 ARR=0x50 且 DITHEN=1）。

1. 必须重置 CEN 和 ARPE 位
2. ARR[3:0] 位必须重置
3. DITHEN 位必须重置
4. CCIF 标志必须清除
5. 可以设置 CEN 位（最终设置 ARPE=1）。

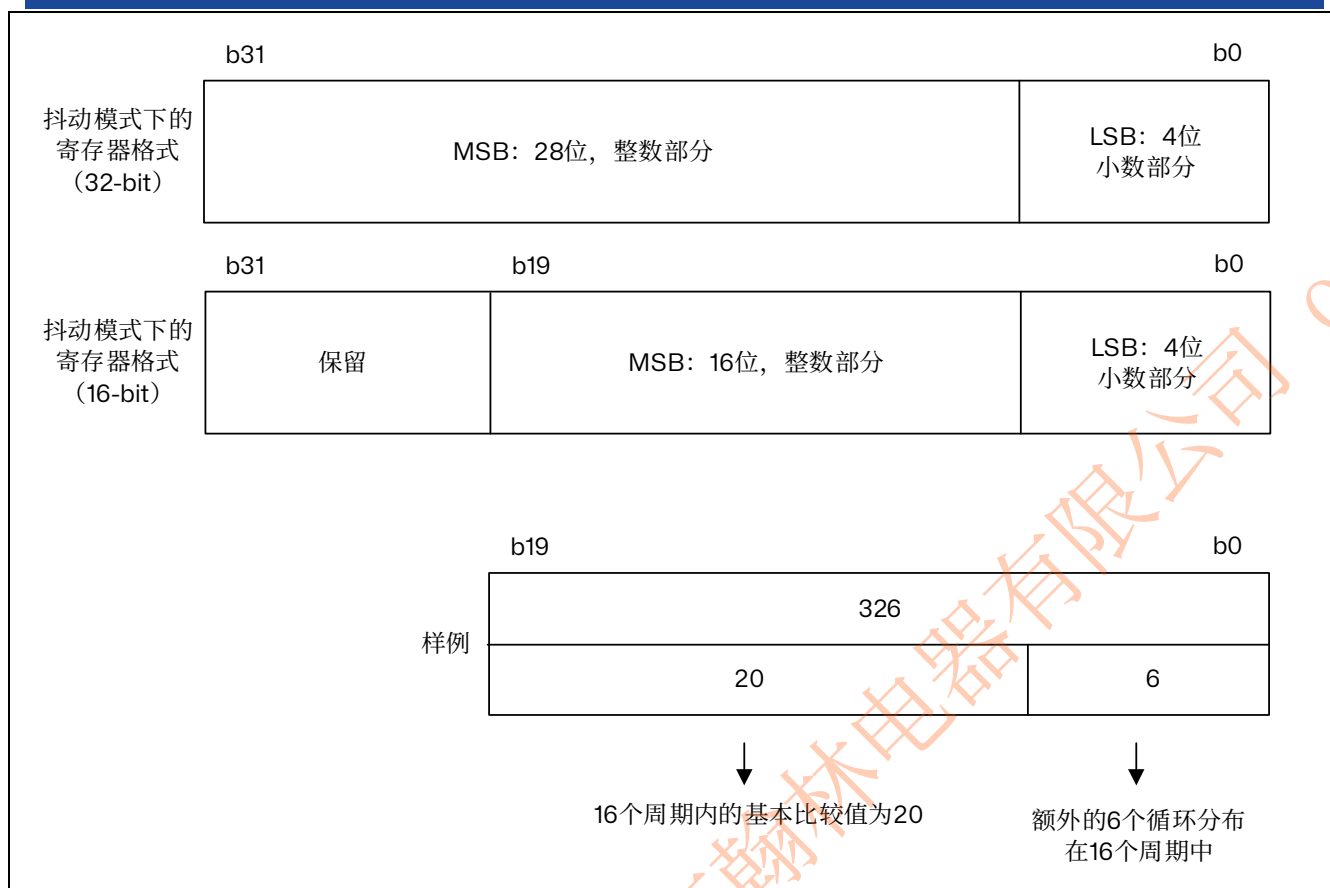


图 24.34 抖动模式下的数据格式和寄存器编码

最小频率由以下公式给出：

$$\text{分辨率} = \frac{FTim}{Fpwm} \Rightarrow FpwmMin = \frac{FTim}{\text{最大分辨率}}$$

$$\text{禁用抖动模式: } FpwmMin = \frac{FTim}{65536}$$

$$\text{使能抖动模式 (16 位定时器): } FpwmMin = \frac{FTim}{65535 + \frac{15}{16}}$$

$$\text{使能抖动模式 (32 位定时器): } FpwmMin = \frac{FTim}{268435454 + \frac{15}{16}}$$

注意：对于16 位定时器，在抖动模式下，TIMx\_ARR 和TIMx\_CCRy 的最大值限制为0xFFFFEF（对应于整数部分的65534 和抖动部分的15）。

对于32 位定时器，在抖动模式下，TIMx\_ARR 和TIMx\_CCRy 的最大值限制为0xFFFFFFFFEF（对应于整数部分的268435454 和抖动部分的15）。

如下两张图所示，抖动模式可以增加 PWM 分辨率。

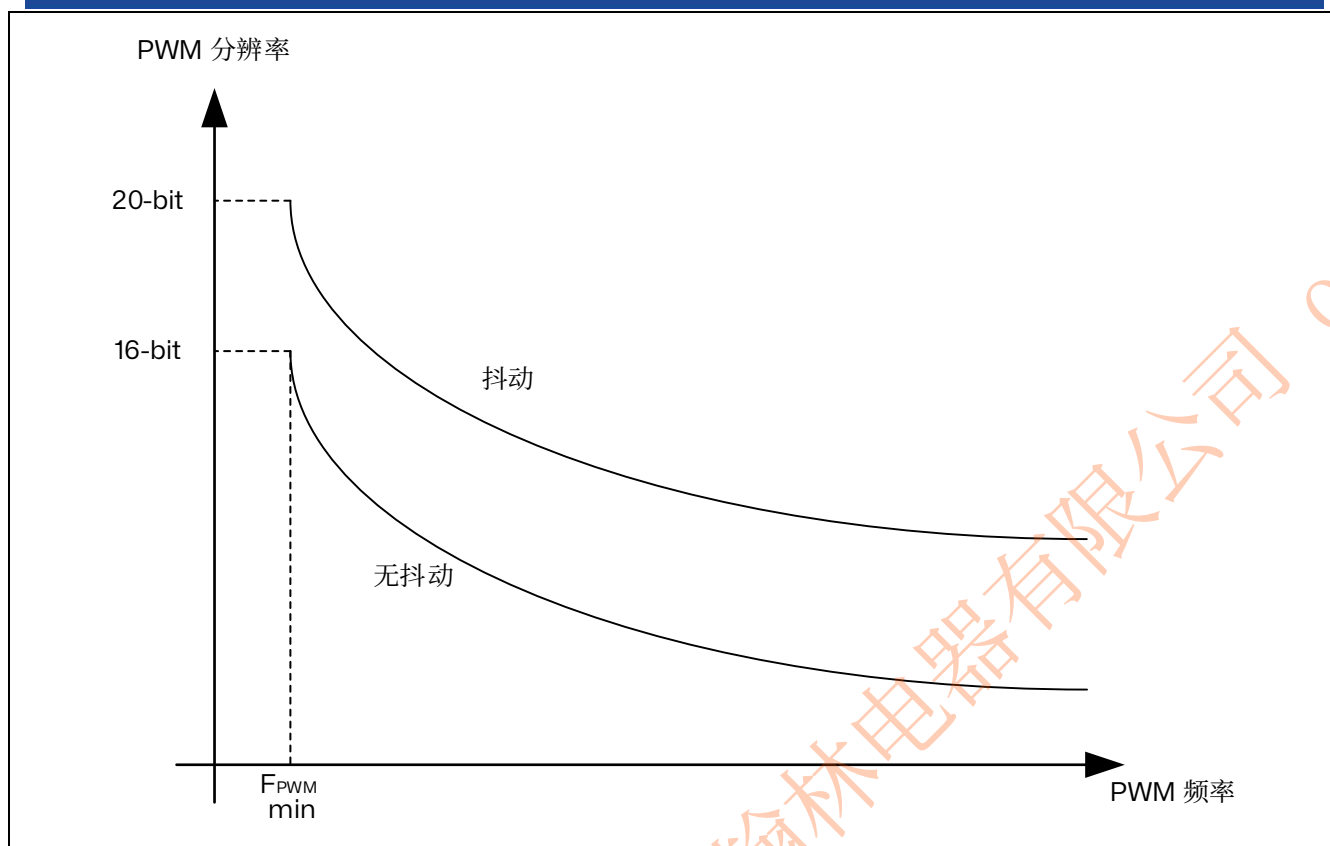


图 24.35 PWM 分辨率 vs 频率 (16 位模式)

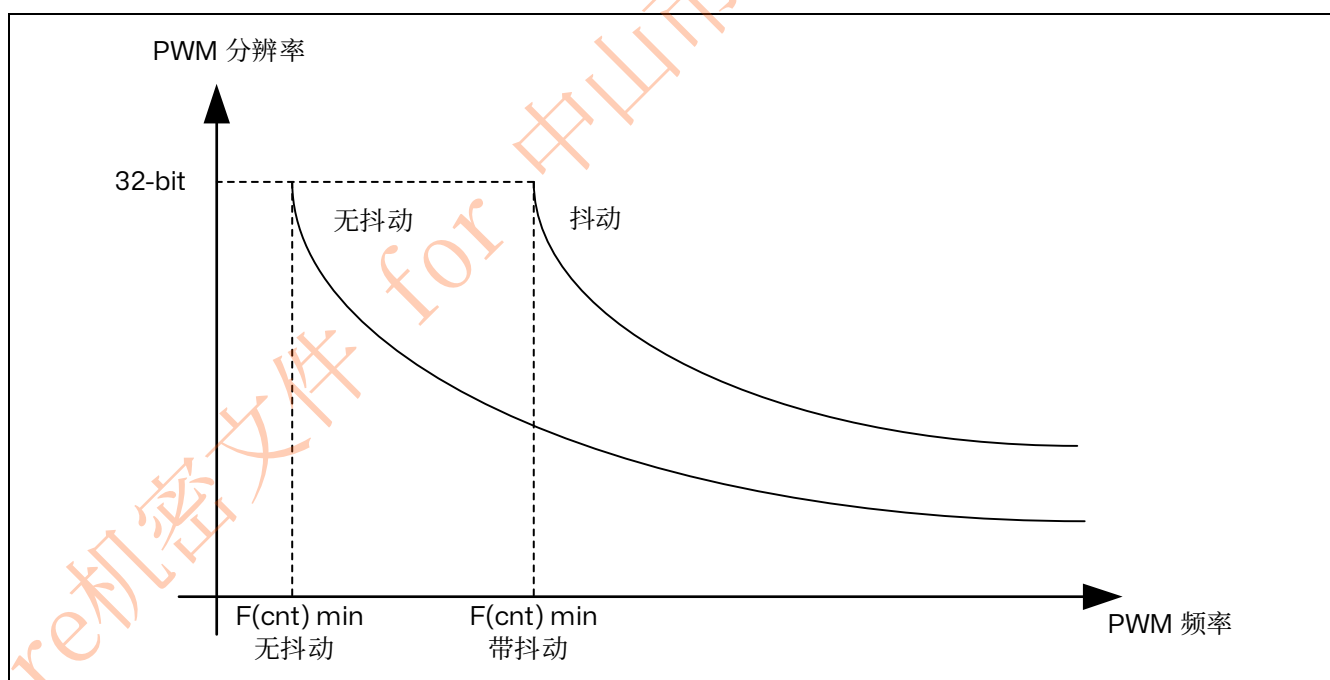


图 24.36 PWM 分辨率 vs 频率 (32 位模式)

如下图所示，占空比和/或周期的变化分散在 16 个连续周期。

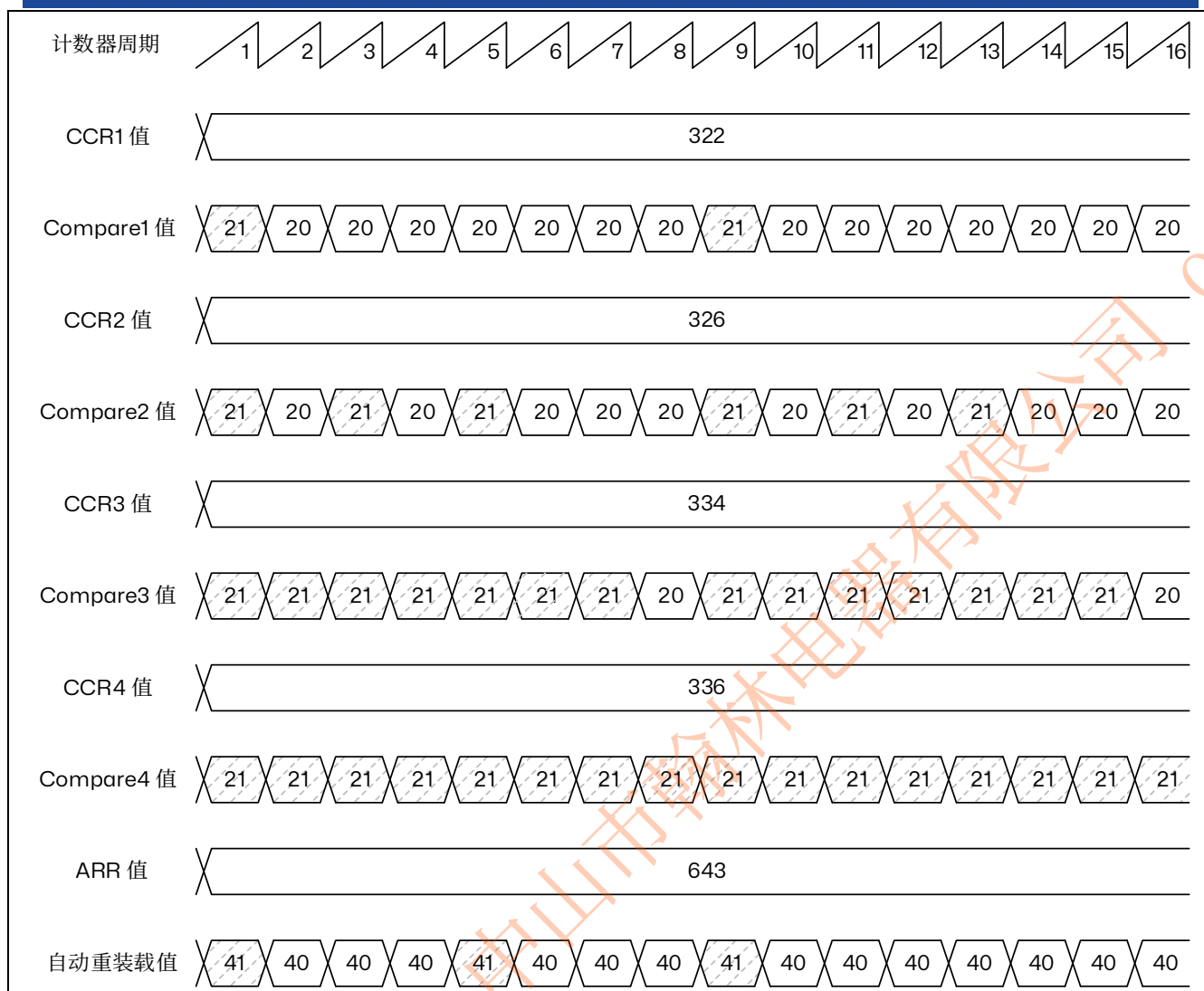


图 24.37 PWM 抖动模式

自动重新加载和比较值的增量按照下表中描述的特定模式分布。抖动序列是为了使增量分布尽可能均匀并最大限度地减少整体波动。

表 24.11 CCR 和 ARR 寄存器的变化抖动模式

| LSB 值 | PWM 周期 |   |    |   |    |   |    |   |    |    |    |    |    |    |    |    |
|-------|--------|---|----|---|----|---|----|---|----|----|----|----|----|----|----|----|
|       | 1      | 2 | 3  | 4 | 5  | 6 | 7  | 8 | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| 0000  | -      | - | -  | - | -  | - | -  | - | -  | -  | -  | -  | -  | -  | -  | -  |
| 0001  | +1     | - | -  | - | -  | - | -  | - | -  | -  | -  | -  | -  | -  | -  | -  |
| 0010  | +1     | - | -  | - | -  | - | -  | - | +1 | -  | -  | -  | -  | -  | -  | -  |
| 0011  | +1     | - | -  | - | +1 | - | -  | - | +1 | -  | -  | -  | -  | -  | -  | -  |
| 0100  | +1     | - | -  | - | +1 | - | -  | - | +1 | -  | -  | -  | +1 | -  | -  | -  |
| 0101  | +1     | - | +1 | - | +1 | - | -  | - | +1 | -  | -  | -  | +1 | -  | -  | -  |
| 0110  | +1     | - | +1 | - | +1 | - | -  | - | +1 | -  | +1 | -  | +1 | -  | -  | -  |
| 0111  | +1     | - | +1 | - | +1 | - | +1 | - | +1 | -  | +1 | -  | +1 | -  | -  | -  |
| 1000  | +1     | - | +1 | - | +1 | - | +1 | - | +1 | -  | +1 | -  | +1 | -  | +1 | -  |



|      |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |
|------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|
| 1001 | +1 | +1 | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | - |
| 1010 | +1 | +1 | +1 | -  | +1 | -  | +1 | -  | +1 | +1 | +1 | -  | +1 | -  | +1 | - |
| 1011 | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | -  | +1 | - |
| 1100 | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | - |
| 1101 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | - |
| 1110 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | -  | +1 | +1 | +1 | +1 | +1 | +1 | +1 | - |
| 1111 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | - |

在中心对齐 PWM 模式下 (TIMx\_CR1 寄存器中的 CMS 位不为“00”), 也提供了抖动模式。在这种模式下, 抖动模式考虑到递增递减计数阶段, 应用于 8 个连续的 PWM 周期, 如下图所示。

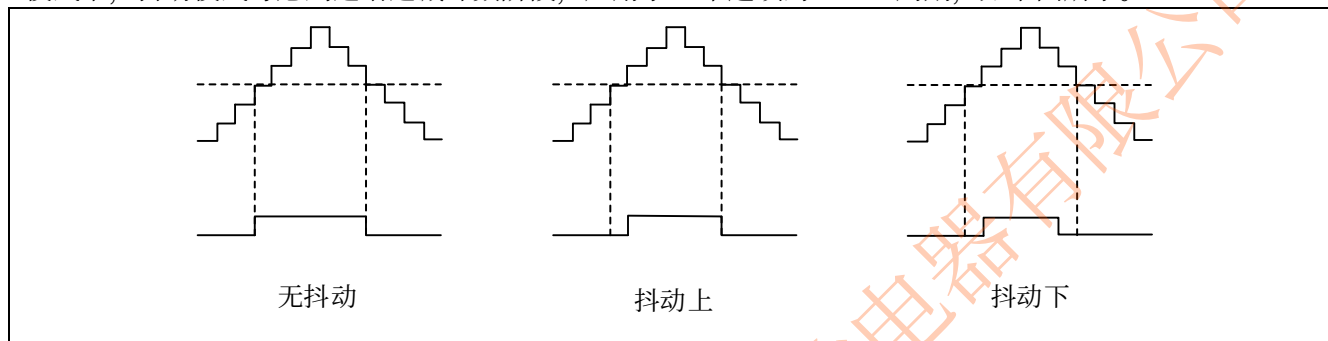


图 24.38 中心对齐 PWM 模式下抖动对占空比的影响

下表显示了如何在中心对齐 PWM 模式下添加抖动模式。

表 24.12 CCR 寄存器在中心对齐 PWM 模式下的变化抖动模式

| LSB 值 | PWM 周期 |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |
|-------|--------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|
|       | 1      |    | 2  |    | 3  |    | 4  |    | 5  |    | 6  |    | 7  |    | 8  |   |
|       | 上      | 下  | 上  | 下  | 上  | 下  | 上  | 下  | 上  | 下  | 上  | 下  | 上  | 下  | 上  | 下 |
| 0000  | -      | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | - |
| 0001  | +1     | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | - |
| 0010  | +1     | -  | -  | -  | -  | -  | -  | -  | +1 | -  | -  | -  | -  | -  | -  | - |
| 0011  | +1     | -  | -  | -  | +1 | -  | -  | -  | +1 | -  | -  | -  | -  | -  | -  | - |
| 0100  | +1     | -  | -  | -  | +1 | -  | -  | -  | +1 | -  | -  | -  | +1 | -  | -  | - |
| 0101  | +1     | -  | +1 | -  | +1 | -  | -  | -  | +1 | -  | -  | -  | +1 | -  | -  | - |
| 0110  | +1     | -  | +1 | -  | +1 | -  | -  | -  | +1 | -  | +1 | -  | +1 | -  | -  | - |
| 0111  | +1     | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | -  | - |
| 1000  | +1     | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | - |
| 1001  | +1     | +1 | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | - |
| 1010  | +1     | +1 | +1 | -  | +1 | -  | +1 | -  | +1 | +1 | +1 | -  | +1 | -  | +1 | - |
| 1011  | +1     | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | -  | +1 | - |
| 1100  | +1     | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | - |
| 1101  | +1     | +1 | +1 | +1 | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | - |
| 1110  | +1     | +1 | +1 | +1 | +1 | +1 | +1 | -  | +1 | +1 | +1 | +1 | +1 | +1 | +1 | - |
| 1111  | +1     | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | - |

### 24.4.12 非对称 PWM 模式

非对称模式允许生成两个中心对齐的 PWM 信号，并具有可编程的相位偏移。虽然频率由 TIMx\_ARR 寄存器的值决定，但占空比和相位偏移由一对 TIMx\_CCRx 寄存器确定。一个寄存器控制 PWM 在计数上升期间，第二个寄存器在计数下降期间，这样 PWM 每半个 PWM 周期调整一次：

- tim\_oc1refc (或 tim\_oc2refc) 由 TIMx\_CCR1 和 TIMx\_CCR2 控制
- tim\_oc3refc (或 tim\_oc4refc) 由 TIMx\_CCR3 和 TIMx\_CCR4 控制

通过将“1110”（非对称 PWM 模式 1）或“1111”（非对称 PWM 模式 2）写入 TIMx\_CCMRx 寄存器的 OCxM 位，可以在两个通道上独立选择非对称 PWM 模式（每个 CCR 寄存器对一个 tim\_ocx 输出）。

*注意：为了兼容性原因，OCxM[3:0] 位域被分为两个部分，最高有效位与最低 3 位不连续。*

当某个通道用作非对称 PWM 通道时，其互补通道也可以使用。例如，如果 tim\_oc1refc 信号在通道 1 上生产（非对称 PWM 模式 1），可以在通道 2 上输出 tim\_oc1refc 信号，或者产生非对称 PWM 模式 1 的结果 tim\_oc2refc 信号。

下图标识使用非对称 PWM 模式生成信号的示例（通道 1 到 4 配置为非对称 PWM 模式 2）。

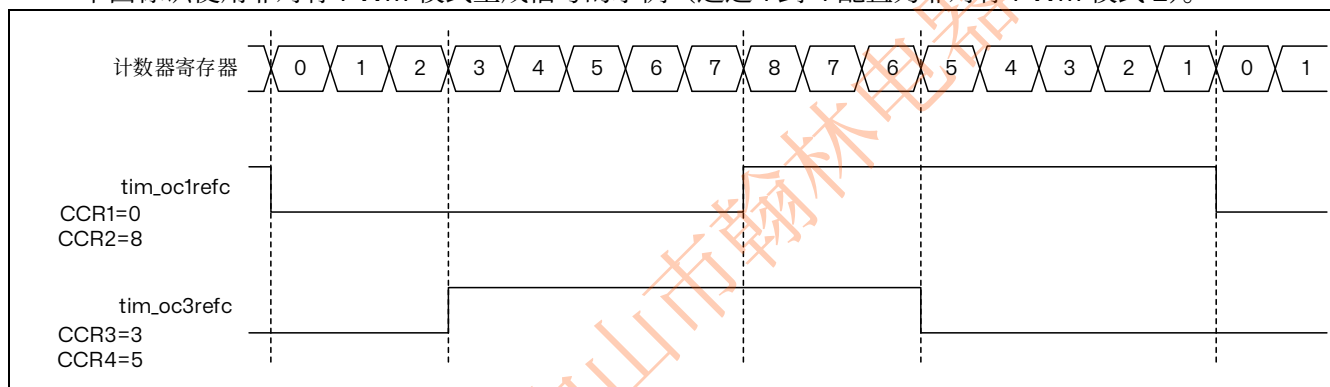


图 24.39 生成两个具有 50% 占空比的相位偏移 PWM 信号

### 24.4.13 组合 PWM 模式

组合 PWM 模式允许生成两个边缘对齐或中心对齐的 PWM 信号，各自脉冲之间的可编程延迟和相位偏移。虽然频率由 TIMx\_ARR 寄存器的值决定，但占空比和延迟由两个 TIMx\_CCRx 寄存器确定。结果信号 tim\_ocxrefc 由两个参考 PWM 的 OR 或 AND 逻辑组合而成：

- tim\_oc1refc (或 tim\_oc2refc) 由 TIMx\_CCR1 和 TIMx\_CCR2 控制
- tim\_oc3refc (或 tim\_oc4refc) 由 TIMx\_CCR3 和 TIMx\_CCR4 控制

通过在 TIMx\_CCMRx 寄存器的 OCxM 位中写入“1100”（组合 PWM 模式 1）或“1101”（组合 PWM 模式 2），可以在两个通道上独立选组合 PWM 模式（每个 CCR 寄存器对一个 tim\_ocx 输出）。

当某个通道用作组合 PWM 通道时，其互补通道必须配置为相反的 PWM 模式（例如，一个通道为组合 PWM 模式 1，另一个通道为组合 PWM 模式 2）。

*注意：为了兼容性原因，OCxM[3:0] 位域被分为两个部分，最高有效位与最低 3 位不连续。*

下图表示使用组合 PWM 模式生成信号的示例，该示例的配置如下：

- 通道 1 配置为组合 PWM 模式 2。
- 通道 2 配置为 PWM 模式 1。
- 通道 3 配置为组合 PWM 模式 2。

— 通道 4 配置为 PWM 模式 1。

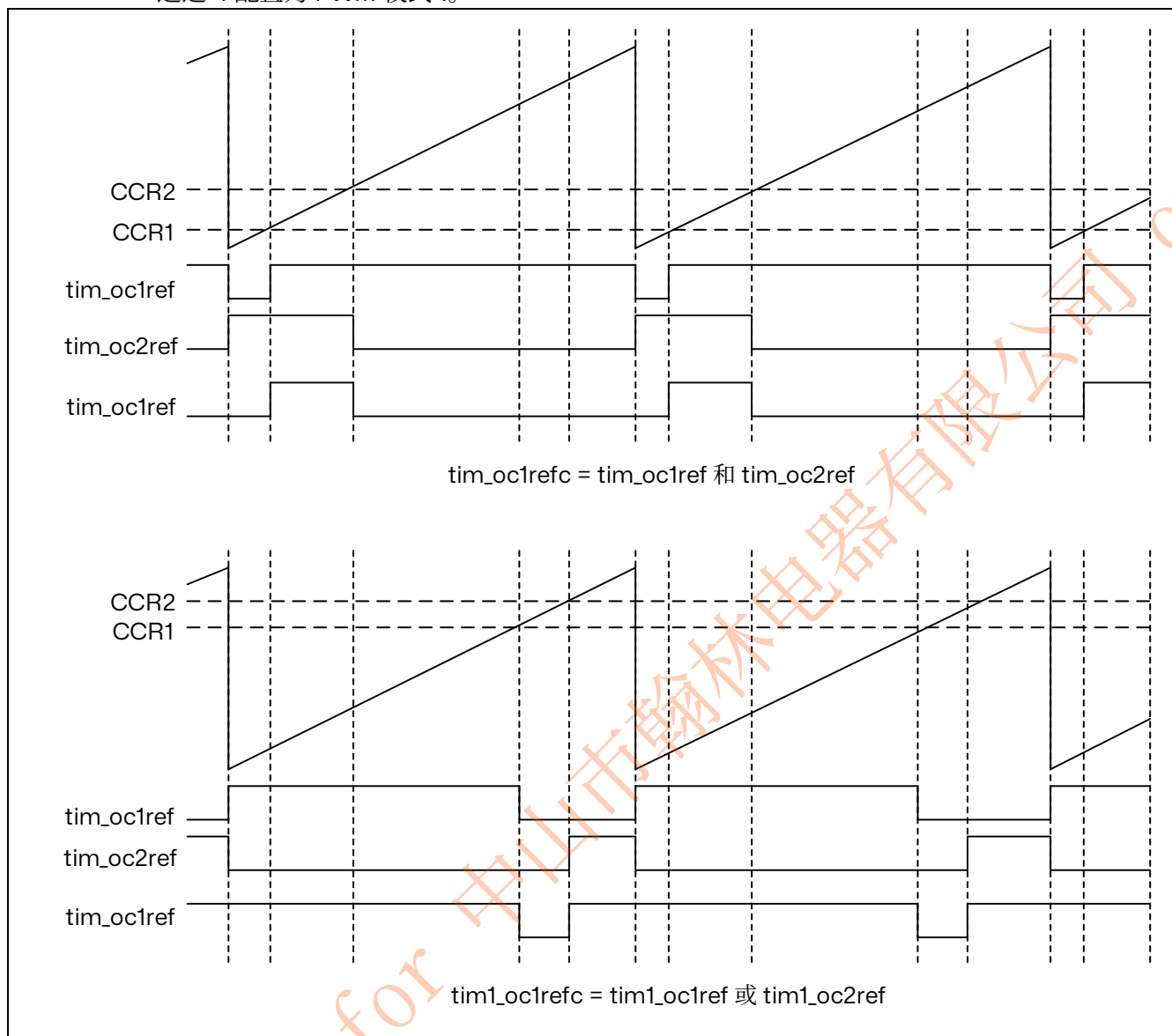


图 24.40 通道 1 和通道 3 的组合 PWM 模式

#### 24.4.14 在外部事件时清除 tim\_ocxref 信号

对于给定通道，在 tim\_ocref\_clr\_int 输入施加高电平（相应 TIMx\_CCMRx 寄存器中的 OCxCE 使能位置“1”），可使 tim\_ocxref 信号变为低电平。tim\_ocxref 信号将保持低电平，直到发生下一更新事件 (UEV)。此功能仅能用于输出比较模式和 PWM 模式，而不适用于强制输出模式。

tim\_ocref\_clr\_int 源取决于 OCREF 清除选择功能实施，请参考前序章节：TIM2/TIM3/TIM4/TIM5 实现。

如果实施了 OCREF 清除选择功能，则可以选择 tim\_ocref\_clr\_int 在 tim\_ocref\_clr 输入和 tim\_etrif 输入（滤波器后的 tim\_etr\_in）之间，通过以下方式进行在 TIMx\_SMCR 寄存器中配置 OCCS 位。tim\_ocref\_clr 输入可以是使用中 OCRSEL[2:0]位域，从多个 tim\_ocref\_clr[7:0]输入中选出 TIMx\_AF2 寄存器，如下图所示。

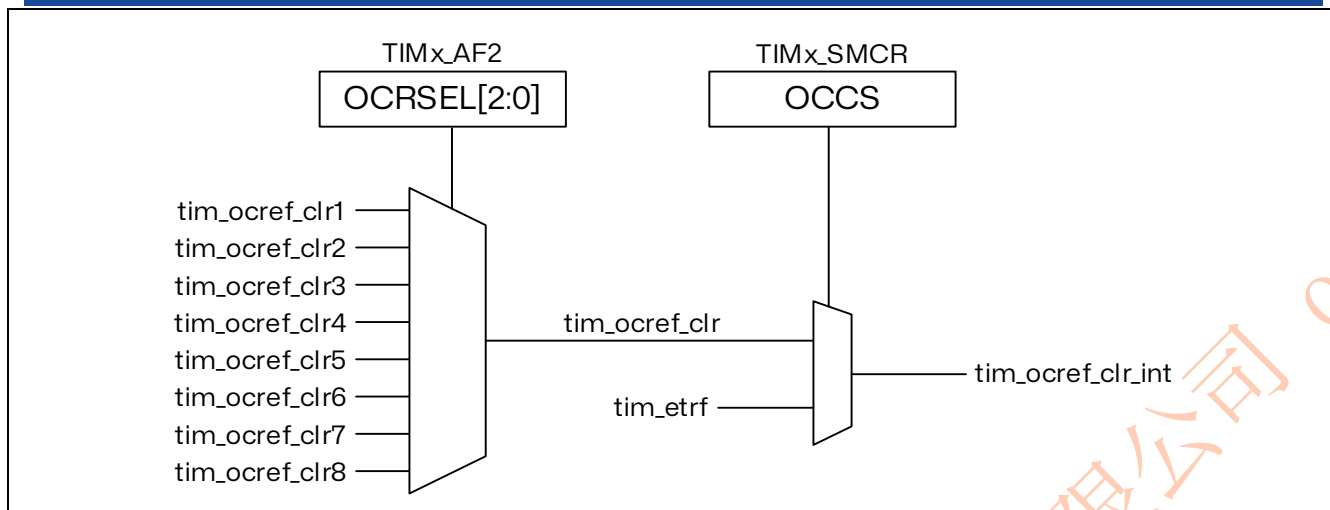


图 24.41 tim\_ocref\_clr 输入选择复用器

如果未实现 OCREF 清除选择功能，则 tim\_ocref\_clr\_int 输入为直接连接到 tim\_etrf 输入。

例如，tim\_ocref\_clr\_int 信号可以连接到比较器的输出，用于控制电流。此时，tim\_etrf\_in 必须如下配置：

1. 必须关闭外部触发预分频器：TIMx\_SMCR 寄存器中的 ETPS[1:0] 位清除为 00。
2. 必须禁止外部时钟模式 2：TIMx\_SMCR 寄存器中的 ECE 位清除为 0。
3. 外部触发极性（ETP）和外部触发滤波器（ETF）可根据应用需要进行配置。

下图对比了使能位 OCxCE 在不同值下的情况，显示了当 tim\_etrf 输入变为高电平时 tim\_ocxref 信号的行为。在本例中，定时器 TIMx 编程为 PWM 模式。

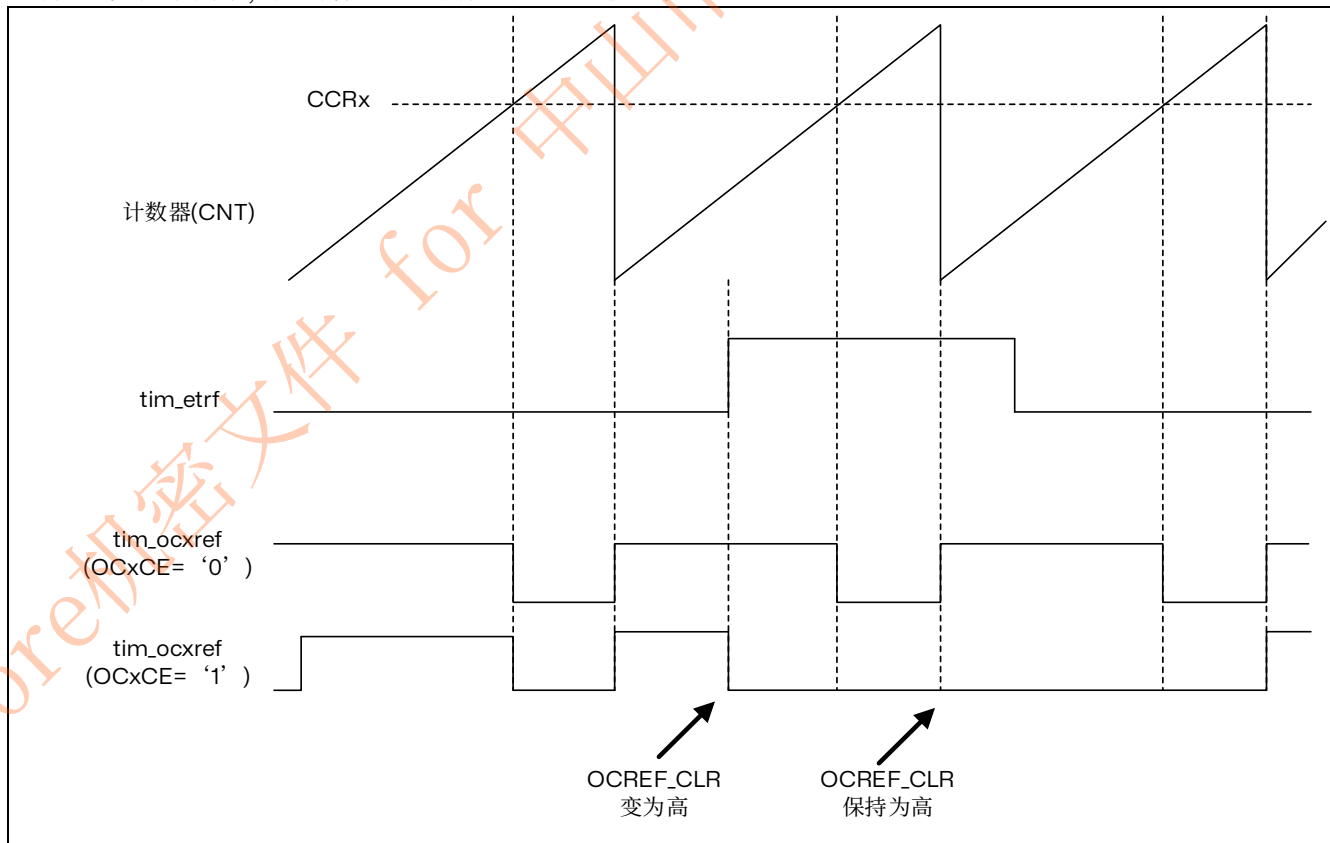


图 24.42 清除 TIMx 的 tim\_ocxref

注：如果 PWM 的占空比为 100% ( $CCR_x > ARR$ )，则下次计数器上溢时会再次使能  $tim\_ocxref$ 。

### 24.4.15 单脉冲模式

单脉冲模式 (OPM) 是上述模式的一个特例。在这种模式下，计数器可以在一个激励信号的触发下启动，并可在一段可编程的延时后产生一个脉宽可编程的脉冲。

可以通过从模式控制器启动计数器。可以在输出比较模式或 PWM 模式下生成波形。将  $TIM_x\_CR1$  寄存器中的 OPM 位置 1，即可选择单脉冲模式。这样，发生下一更新事件 UEV 时，计数器将自动停止。

只有当比较值与计数器初始值不同时，才能正确产生一个脉冲。启动前（定时器等待触发时），必须进行如下配置：

- $CNT < CCR_x \leq ARR$ （特别注意， $0 < CCR_x$ ）

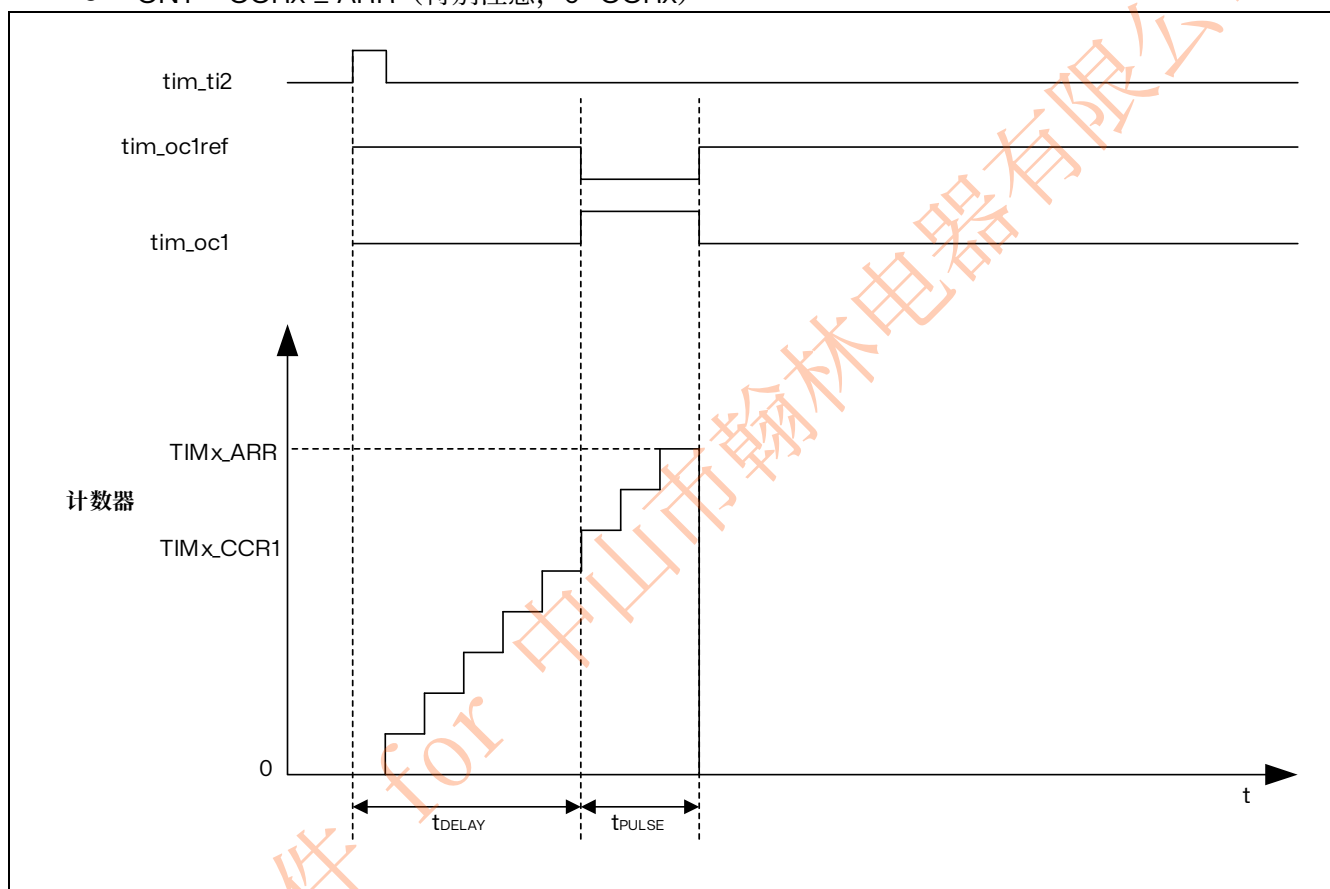


图 24.43 单脉冲模式的例子

例如，用户希望达到这样的效果：在  $tim\_ti2$  输入引脚检测到正沿时，经过  $t_{DELAY}$  的延迟，在  $tim\_oc1$  上产生一个长度为  $t_{PULSE}$  的正脉冲。

使用  $tim\_ti2fp2$  作为触发 1：

1. 通过在  $TIM_x\_TISEL$  寄存器中使用  $TI2SEL[3:0]$  位选择合适的  $tim\_ti2\_in[15:0]$  源（内部或外部）。
  2. 在  $TIM_x\_CCMR1$  寄存器中写入  $CC2S=01$ ，将  $tim\_ti2fp2$  映射到  $tim\_ti2$ 。
  3. 在  $TIM_x\_CCER$  寄存器中写入  $CC2P=0$  和  $CC2NP=“0”$ ，使  $tim\_ti2fp2$  能够检测上升沿。
  4. 在  $TIM_x\_SMCR$  寄存器中写入  $TS=00110$ ，将  $tim\_ti2fp2$  配置为从模式控制器的触发 ( $tim\_trgi$ )。
  5. 在  $TIM_x\_SMCR$  寄存器中写入  $SMS=“110”$ （触发模式），使用  $tim\_ti2fp2$  启动计数器。
- OPM 波形通过对比较寄存器执行写操作来定义（考虑时钟频率和计数器预分频器）。

- $t_{\text{DELAY}}$  由写入 TIMx\_CCR1 寄存器的值定义。
- $t_{\text{PULSE}}$  由自动重载值与比较值 (TIMx\_ARR - TIMx\_CCR1) 之差来定义。
- 假设希望产生这样的波形：信号在发生比较匹配时从“0”变为“1”，在计数器达到自动重载值时由“1”变为“0”。为此，应在 TIMx\_CCMR1 寄存器中写入 OC1M=111，以使能 PWM 模式 2。如果需要，可选择在 TIMx\_CCMR1 寄存器的 OC1PE 和 TIMx\_CR1 寄存器的 ARPE 中写入“1”，以使能预装载寄存器。这种情况下，必须在 TIMx\_CCR1 寄存器中写入比较值并在 TIMx\_ARR 寄存器中写入自动重载值，通过将 UG 位置 1 来产生更新，然后等待 tim\_ti2 上的外部触发事件。本例中，CC1P 的值为“0”。

在本例中，TIMx\_CR1 寄存器中的 DIR 和 CMS 位应为低。

由于仅需要 1 个脉冲（单脉冲模式），因此应向 TIMx\_CR1 寄存器的 OPM 位写入“1”，以便在发生下一更新事件（计数器从自动重载值返回到 0）时使计数器停止计数。TIMx\_CR1 寄存器中的 OPM 位置“0”时，即选择重复模式。

#### 特例：tim\_ocx 快速使能：

在单脉冲模式下，tim\_tix 输入的边沿检测会将 CEN 位置 1，表示使能计数器。然后，在计数器值与比较值之间发生比较时，将切换输出。但是，完成这些操作需要多个时钟周期，这会限制可能的最小延迟 ( $t_{\text{DELAY}}$  最小值)。

如果要输出延迟时间最短的波形，可以将 TIMx\_CCMRx 寄存器中的 OCxFE 位置 1。这样会强制 tim\_ocxref（和 tim\_ocx）对激励信号做出响应，而不再考虑比较的结果。其新电平与发生比较匹配时相同。仅当通道配置为 PWM1 或 PWM2 模式时，OCxFE 才会起作用。

#### 24.4.16 可重触发单脉冲模式

此模式允许计数器在接收到刺激后开始计数，并生成一个具有可编程长度的脉冲，但与上一节中描述的非重触发单脉冲模式存在以下差异：

- 只要触发器发生，脉冲就会立即开始（没有可编程的延迟）
- 如果在先前触发的脉冲完成之前发生新的触发，则脉冲会被延长

定时器必须处于从模式，TIMx\_SMCR 寄存器的 SMS[3:0] 位设置为“1000”（组合重置+触发模式），OCxM[3:0] 位设置为“1000”或“1001”，用于可重触发 OPM 模式 1 或 2。

如果定时器配置为递增计数模式，则相应的 CCRx 必须设置为 0（ARR 寄存器设置脉冲长度）。如果定时器配置为递减计数模式，CCRx 必须大于或等于 ARR。

注：在可重复触发单脉冲模式中，CCxIF 标志不重要。

OCxM[3:0] 和 SMS[3:0] 位域由于兼容性的原因被分成两个部分，最高有效位与最低 3 位不是连续的。

OCxM 这种模式不能与中央对齐 PWM 模式一起使用。TIMx\_CR1 寄存器中的 CMS[1:0] 必须设置为 00。

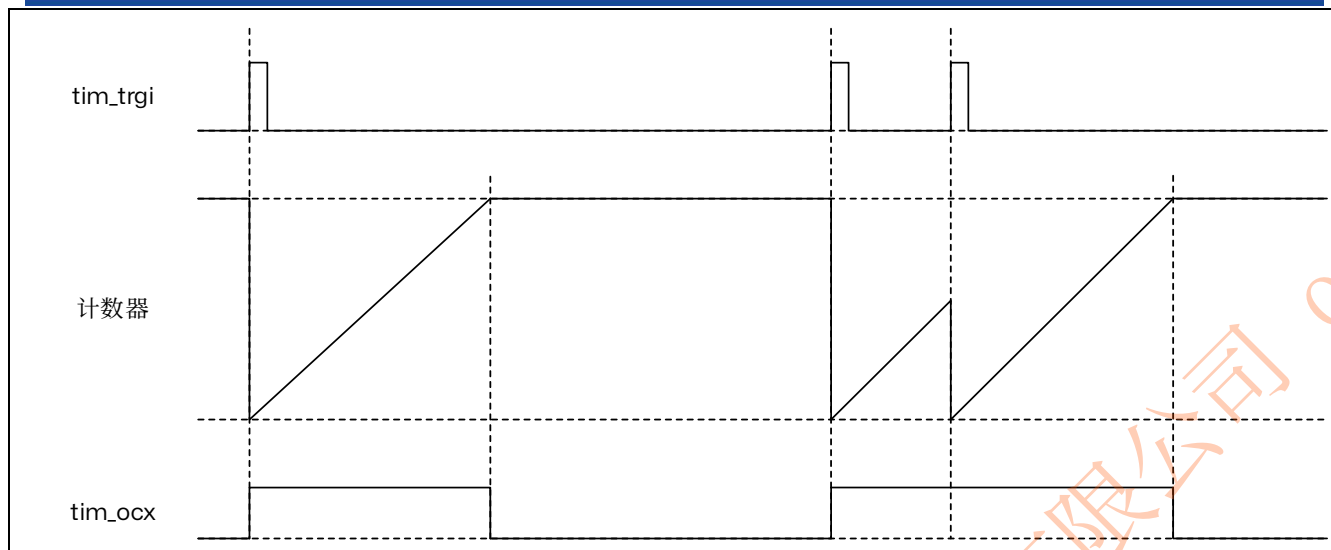


图 24.44 可重触发单脉冲模式

### 24.4.17 在比较模式下的脉冲

在比较匹配事件发生时，可以生成一个脉冲。当计数器的值等于给定的比较值时，可以生成一个具有可编程脉冲宽度的信号，用于调试或同步。

此模式适用于任何从机模式选择，包括编码器模式、边沿对齐和中心对齐计数模式。它仅适用于通道 3 和通道 4。脉冲发生器是唯一的，由两个通道共享，如下图所示。

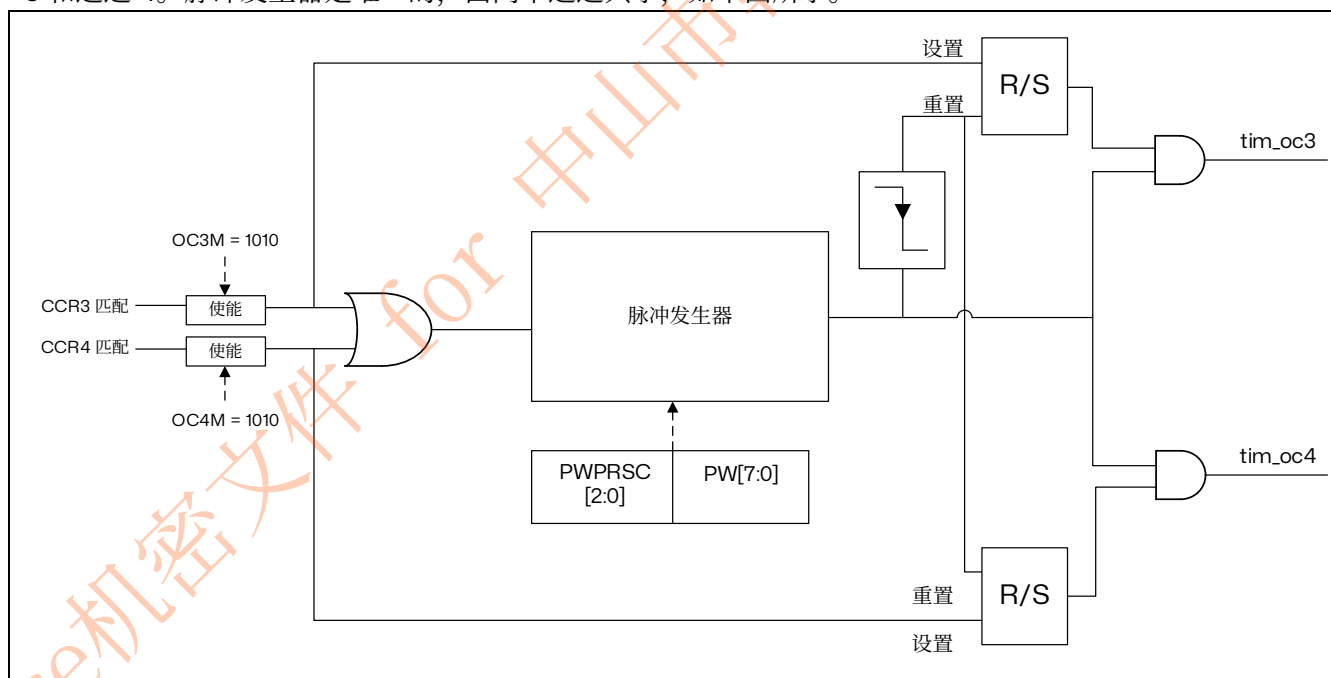


图 24.45 脉冲发生器电路

下图展示了如何在边沿对齐模式和编码器工作模式下生成脉冲。



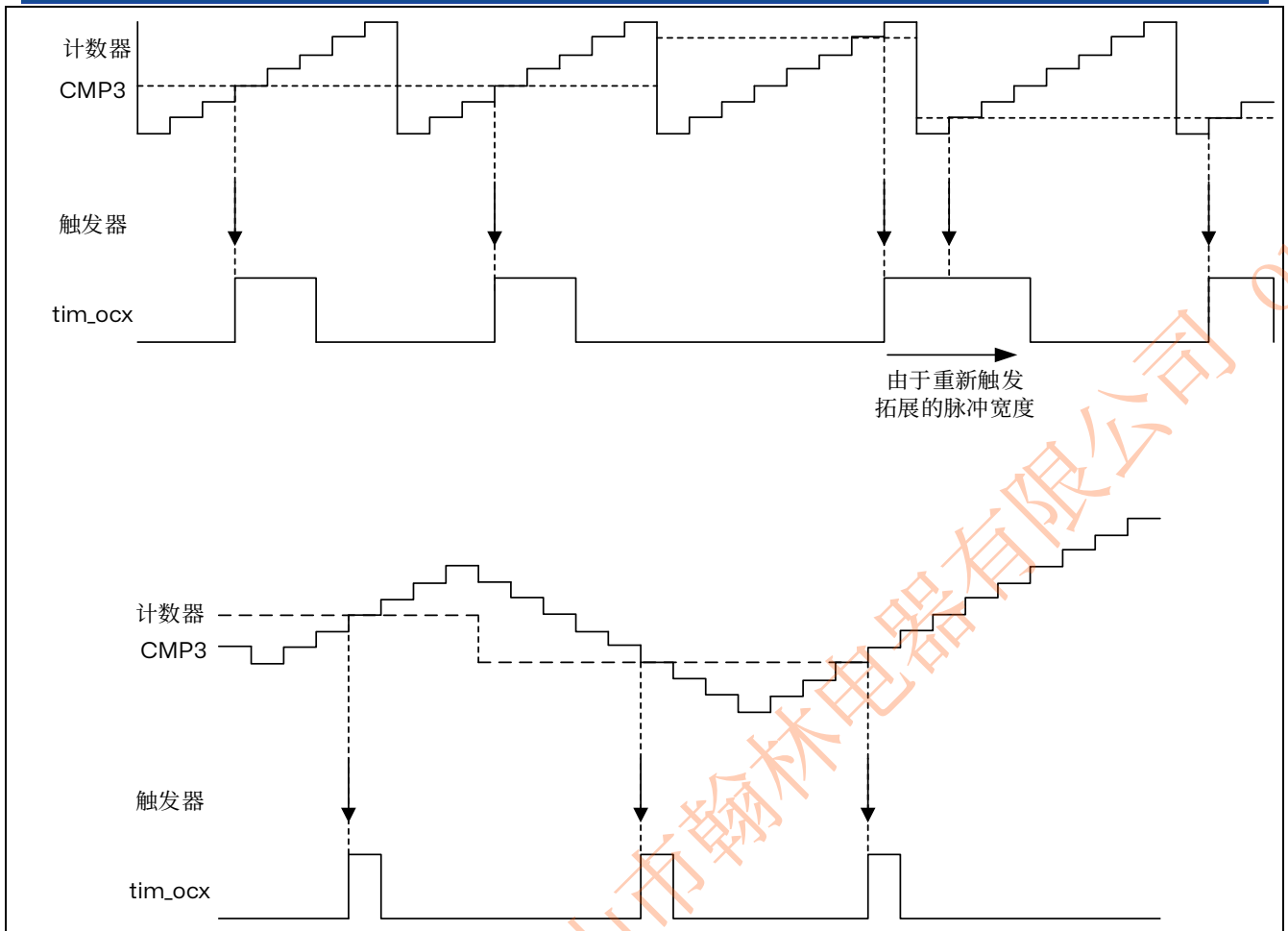


图 24.46 在比较事件上的脉冲生成，适用于边沿对齐模式和编码器模式

此输出比较模式是通过 TIMx\_CCMR2 寄存器中的 OC3M[3:0]和 OC4M[3:0]位域进行选择的。

脉冲宽度通过寄存器中的 PW[7:0]位域进行编程，使用特定的时钟，并根据 PWPRSC[2:0]位的预设进行预分频，具体如下：

$$t_{PW} = PW[7:0] \times t_{PWG}$$

$$\text{其中 } t_{PWG} = (2^{PWPRSC[2:0]}) \times t_{tim\_ker\_ck}$$

给出了分辨率和最大值，具体取决于预分频器的值。

脉冲可以重新触发：在脉冲正在进行时触发新的脉冲，会导致脉冲被延长。

**注意：**如果同时使能两个通道，只要一个通道上的触发器不与并发输出上生成的脉冲重叠，脉冲就会独立发出。相反，如果两个触发器重叠，与第一个到达的触发器相关的脉冲宽度会被延长（由于重新触发），而最后一个到达的触发器的脉冲宽度是正确的（如下图所示）。

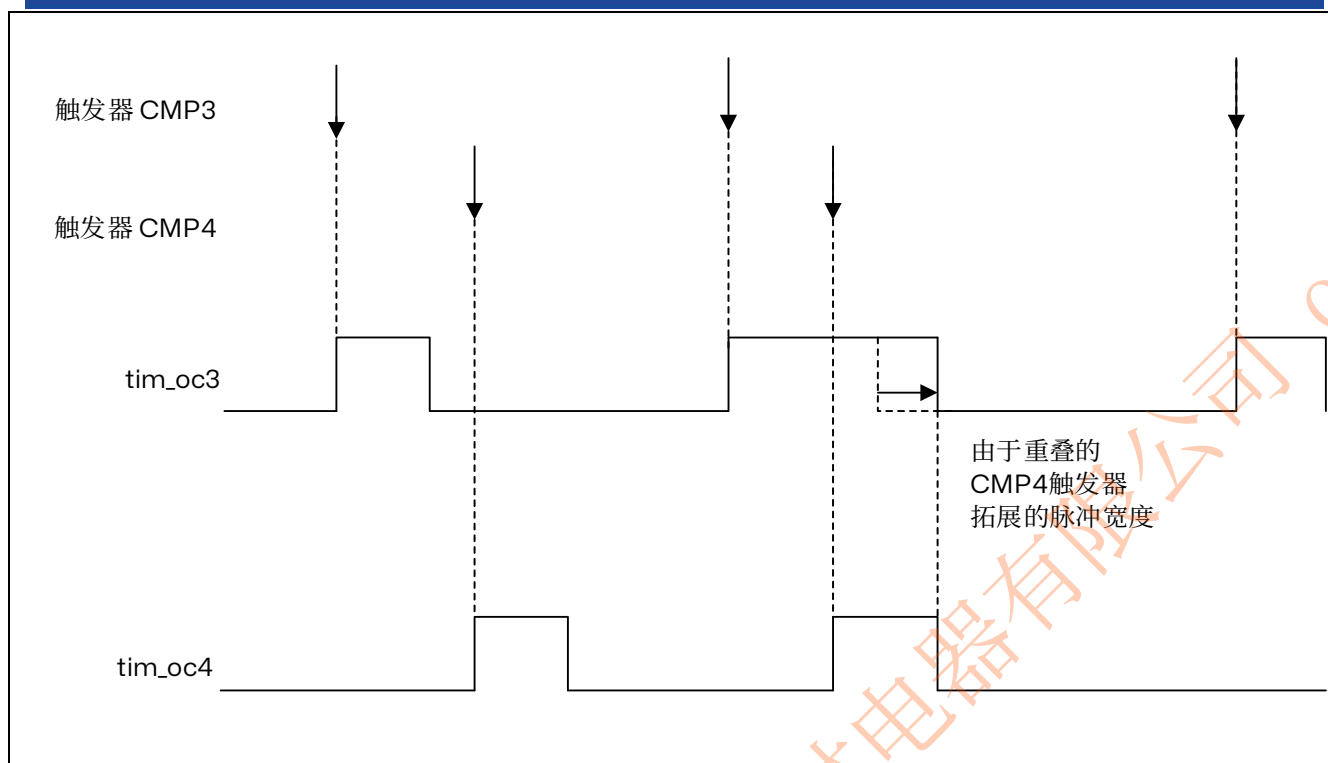


图 24.47 在同时触发的情况下延长脉冲宽度

## 24.4.18 编码器接口模式

### 正交编码器

选择编码器接口模式时，如果计数器仅在 `tim_ti2` 边沿处计数，在 `TIMx_SMCR` 寄存器中写入 `SMS=0001`；如果计数器仅在 `tim_ti1` 边沿处计数，写入 `SMS=0010`；如果计数器在 `tim_ti1` 和 `tim_ti2` 边沿处均计数，则写入 `SMS=0011`。

通过编程 `TIMx_CCER` 寄存器的 `CC1P` 和 `CC2P` 位，选择 `tim_ti1` 和 `tim_ti2` 极性。如果需要，还可对输入滤波器进行编程。

`tim_ti1` 和 `tim_ti2` 两个输入用于连接增量编码器。请参见下表。如果使能计数器（在 `TIMx_CR1` 寄存器的 `CEN` 位中写入“1”），则计数器的时钟由 `tim_ti1fp1` 或 `tim_ti2fp2` 上的每次有效信号转换提供。`tim_ti1fp1` 和 `tim_ti2fp2` 是进行输入滤波器和极性选择后 `tim_ti1` 和 `tim_ti2` 的信号，如果不进行滤波和反相，则 `tim_ti1fp1=tim_ti1`，`tim_ti2fp2=tim_ti2`。将根据两个输入的信号转换序列，产生计数脉冲和方向信号。根据该信号转换序列，计数器相应递增或递减计数，同时硬件对 `TIMx_CR1` 寄存器的 `DIR` 位进行相应修改。任何输入（`tim_ti1` 或 `tim_ti2`）发生信号转换时，都会计算 `DIR` 位，无论计数器是仅在 `tim_ti1` 或 `tim_ti2` 边沿处计数，还是同时在 `tim_ti1` 和 `tim_ti2` 处计数。

编码器接口模式就相当于带有方向选择的外部时钟。这意味着，计数器仅在 0 到 `TIMx_ARR` 寄存器中的自动重载值之间进行连续计数（根据具体方向，从 0 递增计数到 `ARR`，或从 `ARR` 递减计数到 0）。因此，在启动前必须先配置 `TIMx_ARR`。同样，捕获、比较、预分频器、触发输出功能继续正常工作。

在此模式下，计数器会根据增量编码器的速度和方向自动进行修改，因此，其内容始终表示编码器的位置。计数方向对应于所连传感器的旋转方向。下表汇总了可能的组合（假设 `tim_ti1` 和 `tim_ti2` 不同时切换）。

表 24.13 计数方向与编码器信号的关系 (CC1P = CC2P = 0)

| 有效边沿                           | SMS[3:0] | 相对信号的电平<br>(tim_ti1fp1 对应 tim_ti2,<br>tim_ti2fp2 对应 tim_ti1) | tim_ti1fp1 信号 |     | tim_ti2fp2 信号 |     |
|--------------------------------|----------|--------------------------------------------------------------|---------------|-----|---------------|-----|
|                                |          |                                                              | 上升            | 下降  | 上升            | 下降  |
| 仅 x1 模式在<br>tim_ti1 计数         | 1110     | 高                                                            | 递减            | 递增  | 不计数           | 不计数 |
|                                |          | 低                                                            | 不计数           | 不计数 | 不计数           | 不计数 |
| 仅 x1 模式在<br>tim_ti2 计数         | 1111     | 高                                                            | 不计数           | 不计数 | 递增            | 递减  |
|                                |          | 低                                                            | 不计数           | 不计数 | 不计数           | 不计数 |
| 仅 x2 模式在<br>tim_ti1 计数         | 0001     | 高                                                            | 递减            | 递增  | 不计数           | 不计数 |
|                                |          | 低                                                            | 递增            | 递减  | 不计数           | 不计数 |
| 仅 x2 模式在<br>tim_ti2 计数         | 0010     | 高                                                            | 不计数           | 不计数 | 递增            | 递减  |
|                                |          | 低                                                            | 不计数           | 不计数 | 递减            | 递增  |
| x4 模式在 tim_ti1<br>和 tim_ti2 计数 | 0011     | 高                                                            | 递减            | 递增  | 递增            | 递减  |
|                                |          | 低                                                            | 递增            | 递减  | 递减            | 递增  |

外部增量编码器可直接与 MCU 相连，无需外部接口逻辑。不过，通常使用比较器将编码器的差分输出转换为数字信号。这样大幅提高了抗噪声性能。用于指示机械零位的第三个编码器输出可与外部中断输入相连，用以触发计数器复位。

下图以计数器工作为例，说明了计数信号的生成和方向控制。同时也说明了选择双边沿时如何对输入抖动进行补偿。将传感器靠近其中一个切换点放置时可能出现这种情况。本例中假设置如下：

- CC1S= “01” (TIMx\_CCMR1 寄存器，tim\_ti1fp1 映射到 tim\_ti1 上)。
- CC2S= “01” (TIMx\_CCMR2 寄存器，tim\_ti1fp2 映射到 tim\_ti2 上)。
- CC1P= “0”，CC1NP= “0” (TIMx\_CCER 寄存器，tim\_ti1fp1 未反相，tim\_ti1fp1= tim\_ti1)。
- CC2P= “0”，CC2NP= “0” TIMx\_CCER 寄存器，tim\_ti1fp2 未反相，tim\_ti1fp2= tim\_ti2)。
- SMS= “011” (TIMx\_SMCR 寄存器，两个输入在上升沿和下降沿都处于有效状态)。
- CEN= “1” (TIMx\_CR1 寄存器，使能计数器)。

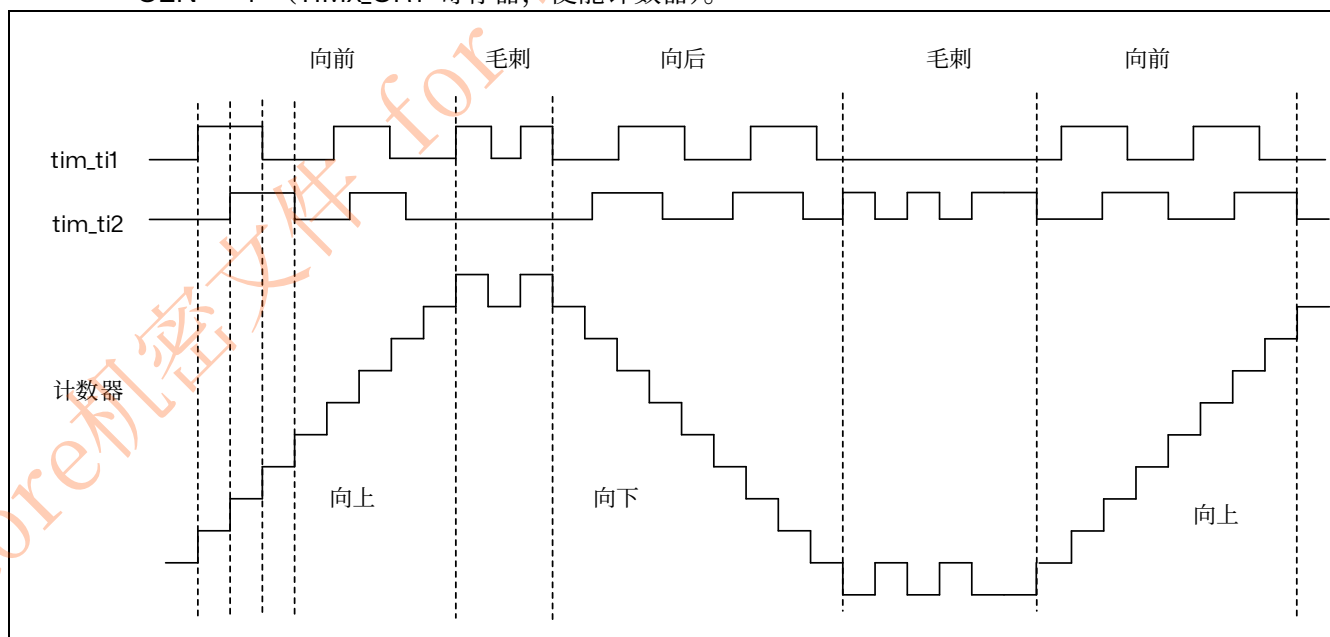


图 24.48 编码器接口模式下的计数器工作示例

下图举例说明 tim\_ti1fp1 极性反相时计数器的行为（除 CC1P=“1”外，其它配置与上例相同）。

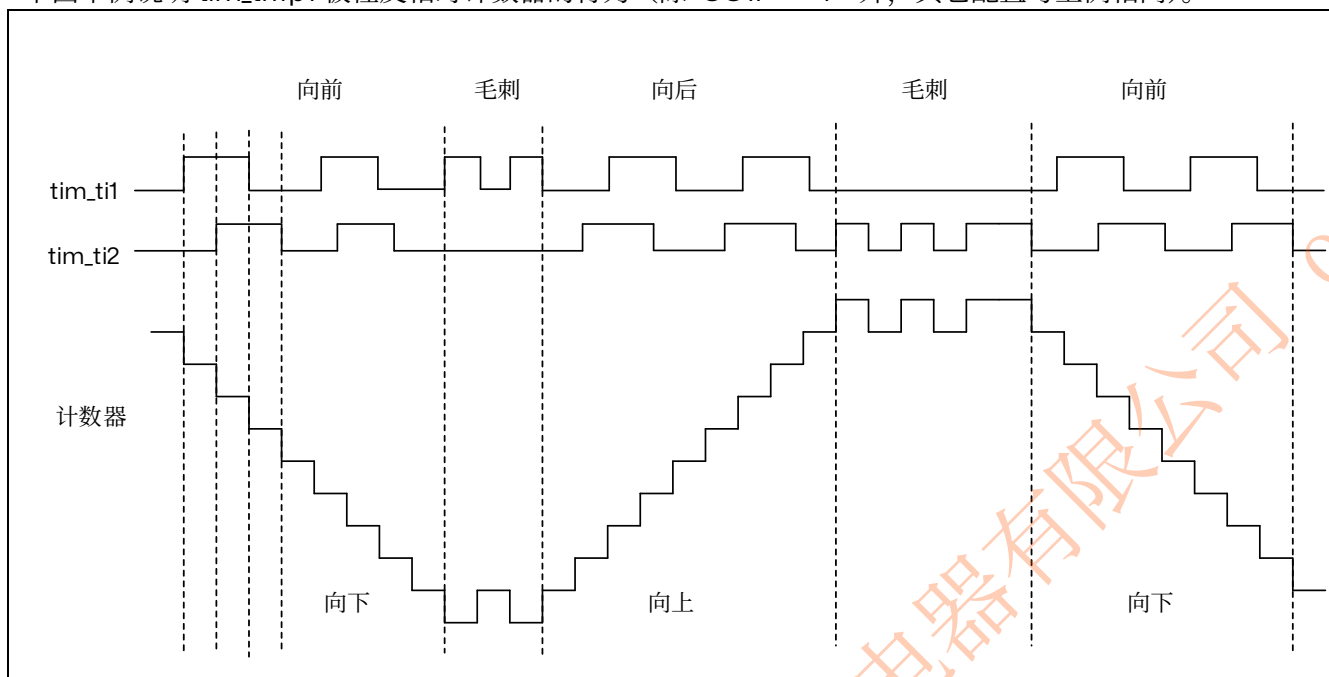


图 24.49 tim\_ti1fp1 极性反相时的编码器接口模式示例

下图显示了各种计数模式下速度反转期间计时器计数器的值。

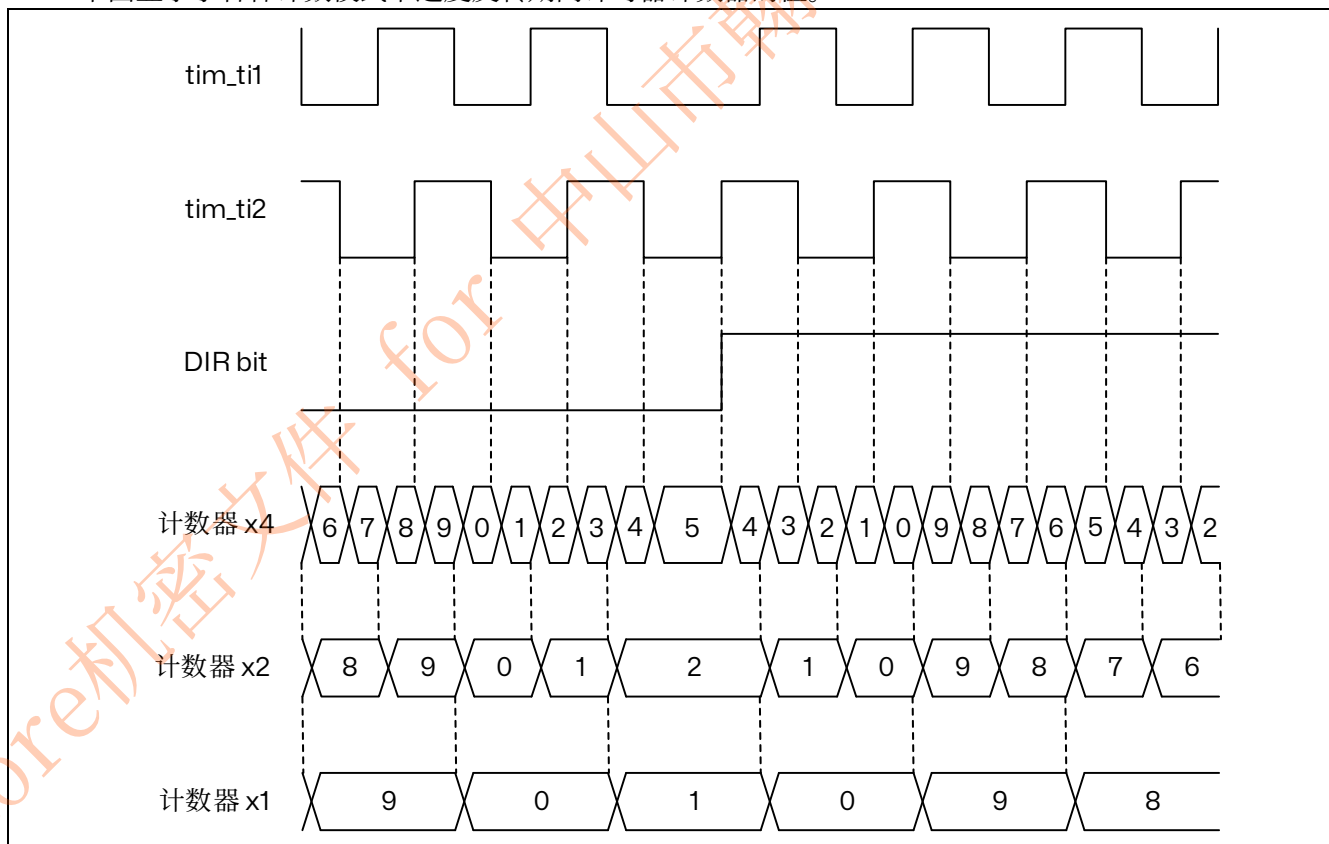


图 24.50 正交编码器的计数模式

当定时器配置为编码器接口模式时，可以提供传感器当前位置的信息。通过测量两个编码器事件之间

的周期，使用配置为捕获模式的第二个定时器，可以获得动态信息（速度、加速度、减速度）。指示机械零的编码器输出可用于此目的。根据两个事件之间的时间，还可以定期读取计数器的值。如果可用，可以通过将计数器的值锁存到第三个输入捕获寄存器来实现（此时捕获信号必须是周期性的，并且可以由另一个定时器生成）。

TIMx\_CR1 寄存器中的 IUFREMAP 位将更新中断标志（UIF）的连续副本强制复制到定时器计数器寄存器的第 31 位（TIMxCNT[31]）。这允许原子地读取计数器的值和由 UIFCPY 标志触发的潜在的翻转条件。这可以通过避免在后台任务（计数器读取）和中断（更新中断）之间共享处理来简化角速度的计算。

UIF 和 UIFCPY 标志之间没有延迟。

在 32 位定时器实现中，当 IUFREMAP 位被设置时，在读取访问时，计数器的第 31 位将被 UIFCPY 标志覆盖（计数器最高有效位仅可在写入模式下访问）。

## 时钟加方向编码器模式

除了正交编码器模式外，定时器还支持其他类型的编码器。

在下图中显示的“时钟加方向”模式下，时钟在 tim\_ti2 上提供单线，而方向使用 tim\_ti1 输入强制。

此模式通过 TIMx\_SMCR 寄存器中的 SMS[3:0] 位域使能，如下所示：

- 1010: x2 模式，计数器在时钟的上升沿和下降沿都更新
- 1011: x1 模式，计数器在单个时钟边沿更新，根据 CC2P 位的值而定：CC2P=0 对应上升沿敏感，CC2P=1 对应下降沿敏感。

tim\_ti1 上的方向信号极性通过 CC1P 位设置：0 对应正极性（当 tim\_ti1 为高电平时向上计数，当 tim\_ti1 为低电平时向下计数），CC1P=1 对应负极性（当 tim\_ti1 为低电平时向上计数）。

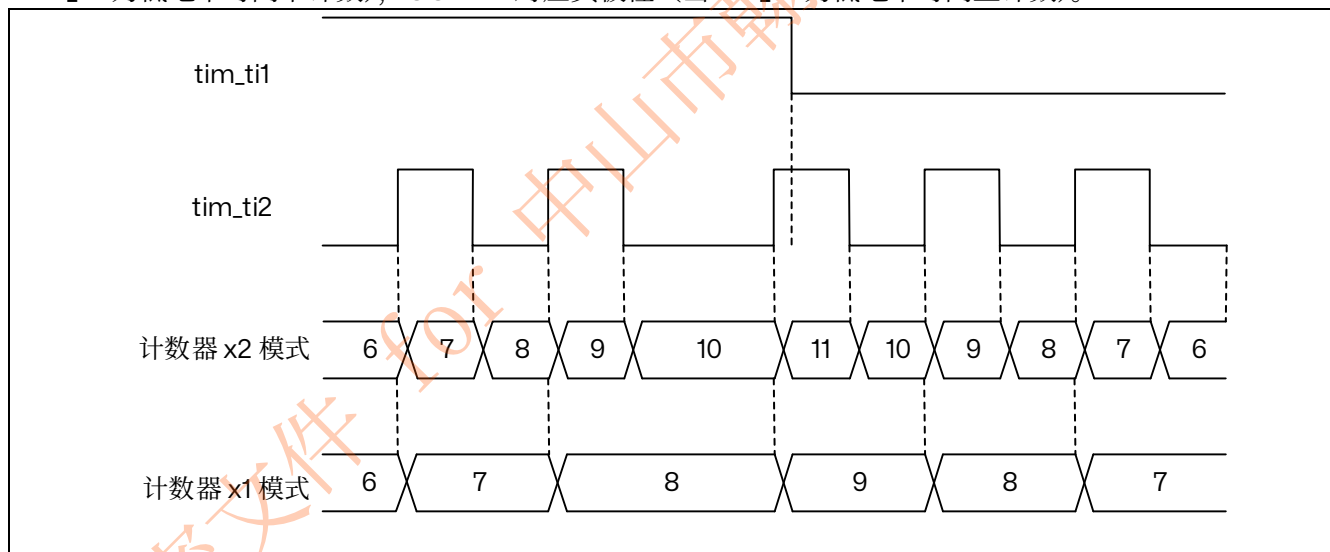


图 24.51 方向加时钟编码器模式

## 方向时钟编码器模式

在下图中的“方向时钟”模式下，时钟在两条线上提供，根据方向一次使用一条，以便有一条向上计数时钟线和一条向下计数时钟线。

此模式通过 TIMx\_SMCR 寄存器中的 SMS[3:0] 位域使能，如下所示：

- 1100: x2 模式，计数器在两条时钟线的上升沿和下降沿都更新。CC1P 和 CC2P 位编码时钟空闲状态。CCxP=0 对应高电平空闲状态（参考下图），CCxP=1 对应低电平空闲状态（参考下图）。
- 1101: x1 模式，计数器在单个时钟边沿更新，根据 CC1P 和 CC2P 位的值而定。CCxP=0 对应下降沿敏感性和高电平空闲状态（参考下图），CCxP=1 对应上升沿敏感性和低电平空闲状态

(参考下图)。

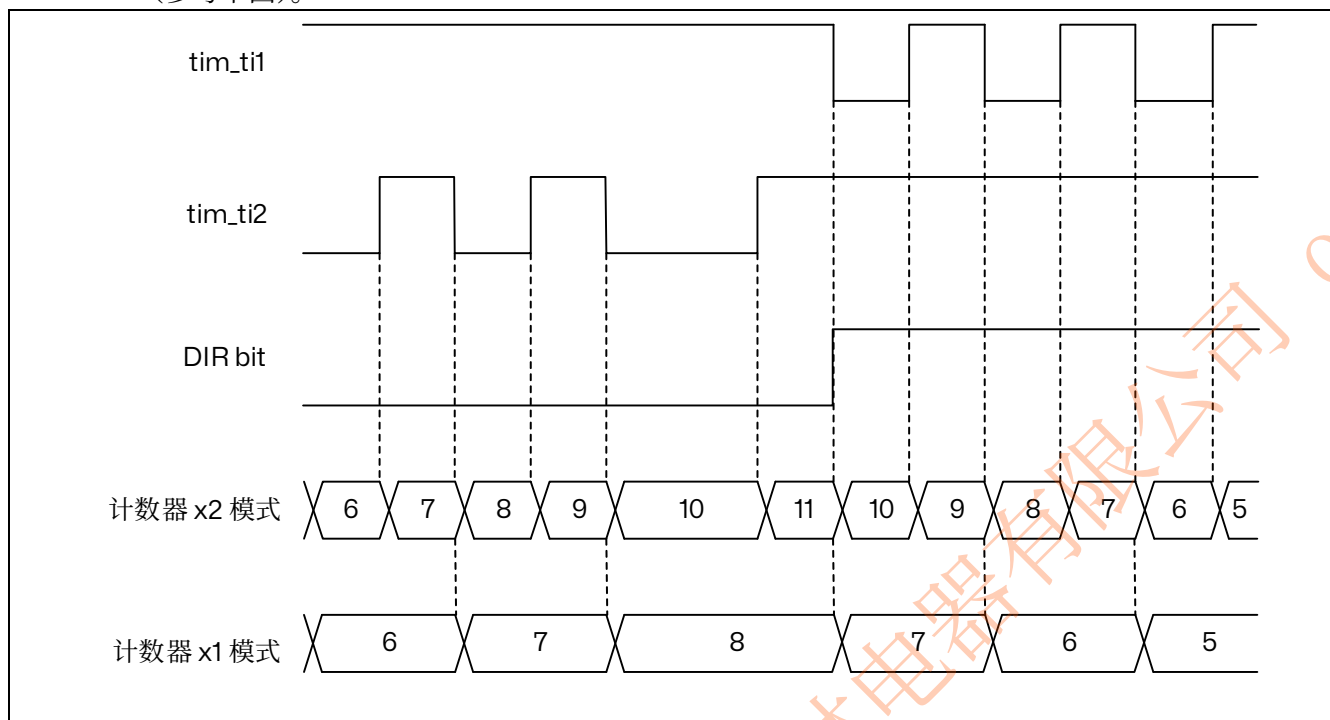


图 24.52 方向时钟编码器模式 (CC1P=CC2P=0)

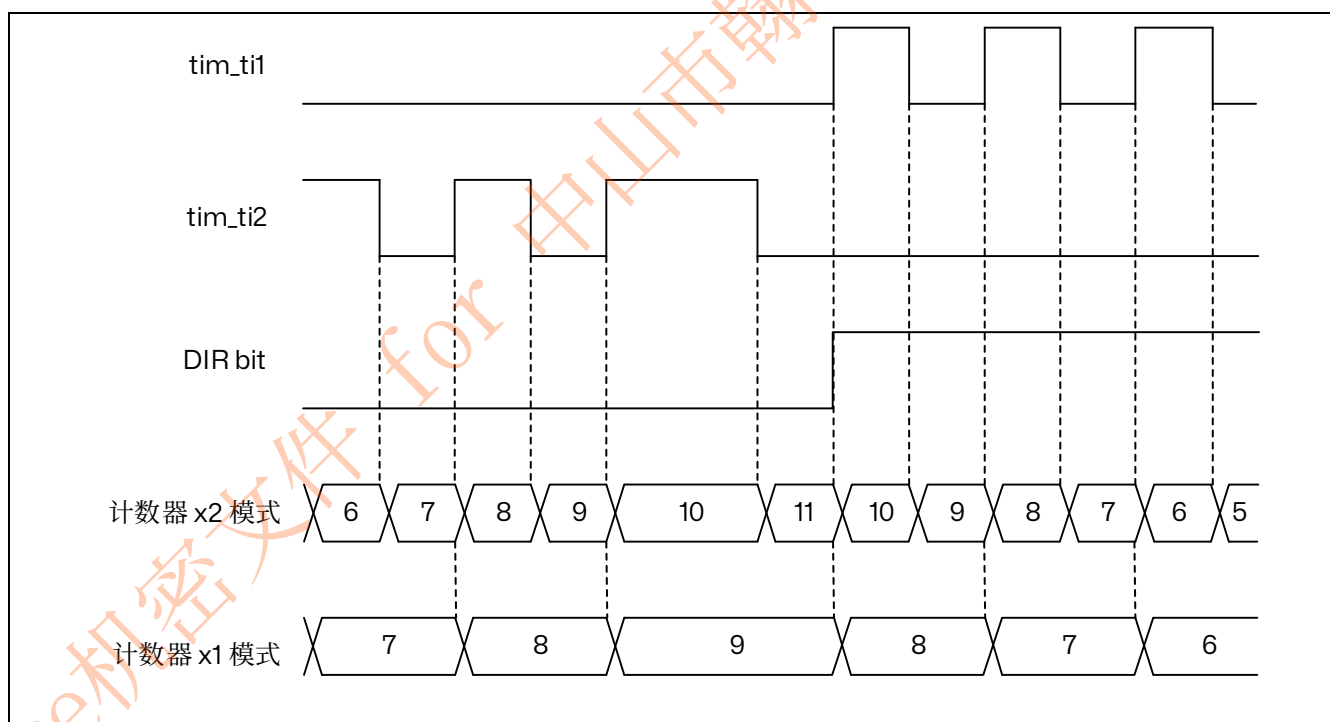


图 24.53 方向时钟编码器模式 (CC1P=CC2P=1)

下表详细说明了方向时钟模式对任何输入转变的操作方式。

表 24.14 计数方向与编码器信号及极性设置的关系

| 方向时钟模式          | SMS[3:0] | 相对信号的电平<br>(tim_ti1fp1 对应 tim_ti2,<br>tim_ti2fp2 对应 tim_ti1) | tim_ti1fp1 信号 |     | tim_ti2fp2 信号 |     |
|-----------------|----------|--------------------------------------------------------------|---------------|-----|---------------|-----|
|                 |          |                                                              | 上升            | 下降  | 上升            | 下降  |
| x2 模式<br>CCxP=0 | 1100     | 高                                                            | 递减            | 递减  | 递增            | 递增  |
|                 |          | 低                                                            | 不计数           | 不计数 | 不计数           | 不计数 |
| x2 模式<br>CCxP=1 | 1100     | 高                                                            | 不计数           | 不计数 | 不计数           | 不计数 |
|                 |          | 低                                                            | 递减            | 递减  | 递增            | 递增  |
| x1 模式<br>CCxP=0 | 1101     | 高                                                            | 不计数           | 递减  | 不计数           | 递增  |
|                 |          | 低                                                            | 不计数           | 不计数 | 不计数           | 不计数 |
| x1 模式<br>CCxP=1 | 1101     | 高                                                            | 不计数           | 不计数 | 不计数           | 不计数 |
|                 |          | 低                                                            | 递减            | 不计数 | 递增            | 不计数 |

## 索引输入

计数器可以通过来自编码器的索引信号进行复位，以指示绝对参考位置。索引信号必须连接到 tim\_etr\_in 输入。它可以使用数字输入滤波器进行滤波。

通过设置 TIMX\_ECR 寄存器中的 IE 位，可以使能索引功能。只有在编码器模式下，当 SMS[3:0] 位域具有以下值时，才可以设置 IE 位：0001、0010、011、1010、1011、1100、1101、1110、1111。

根据下图所示，有几种可设置的编码器选项，用于索引脉冲的调节：

- 与 A 和 B 门控：脉冲宽度为 1/4 的通道周期，与 A 和 B 边缘对齐
- 与 A（或与 B）门控：脉冲宽度为 1/2 的通道周期，与通道 A（或通道 B）的两个边缘对齐
- 非门控：脉冲宽度可达一个通道周期，不与任何边缘对齐

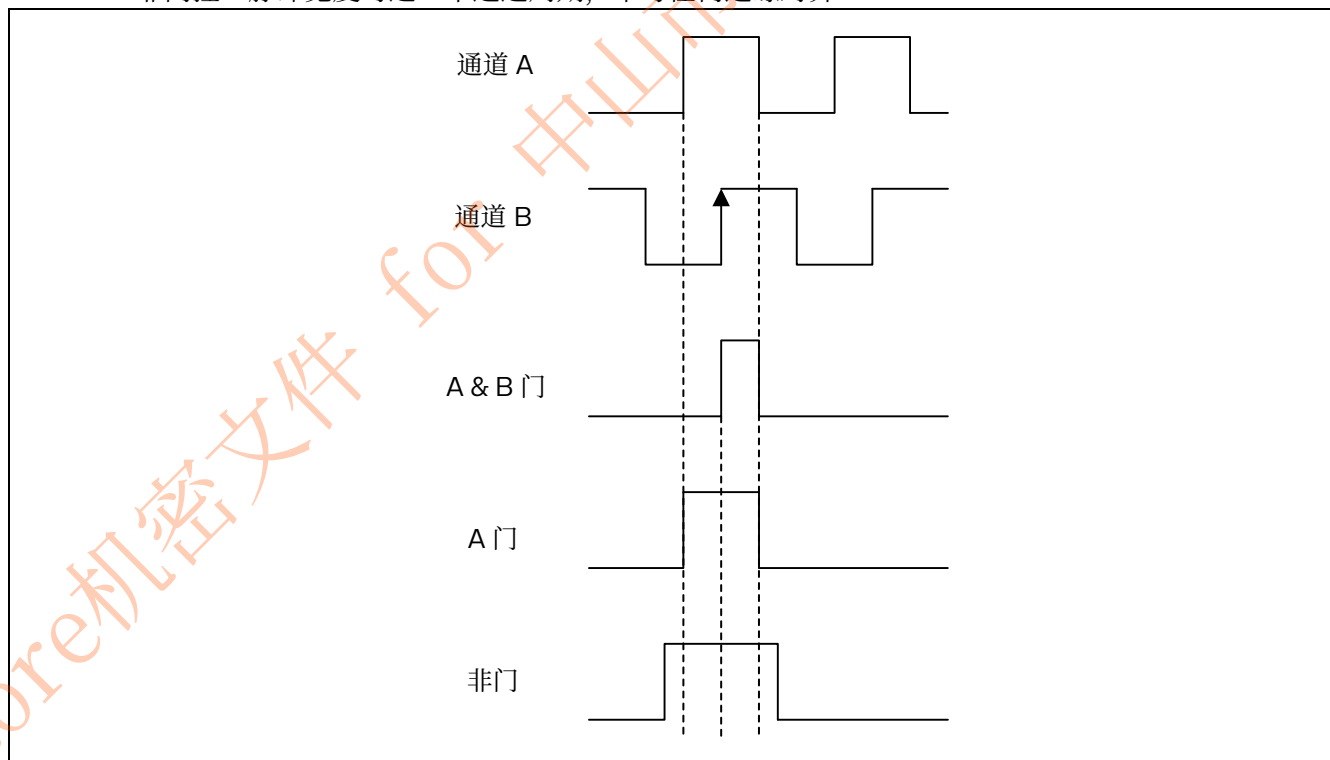


图 24.54 索引门控选项

电路可以容忍索引信号的抖动，无论门控模式如何，如下图所示。

在非门控模式下，信号必须严格低于 2 个编码器周期。如果脉冲宽度大于或等于 2 个编码器周期，则



计数器会多次复位。

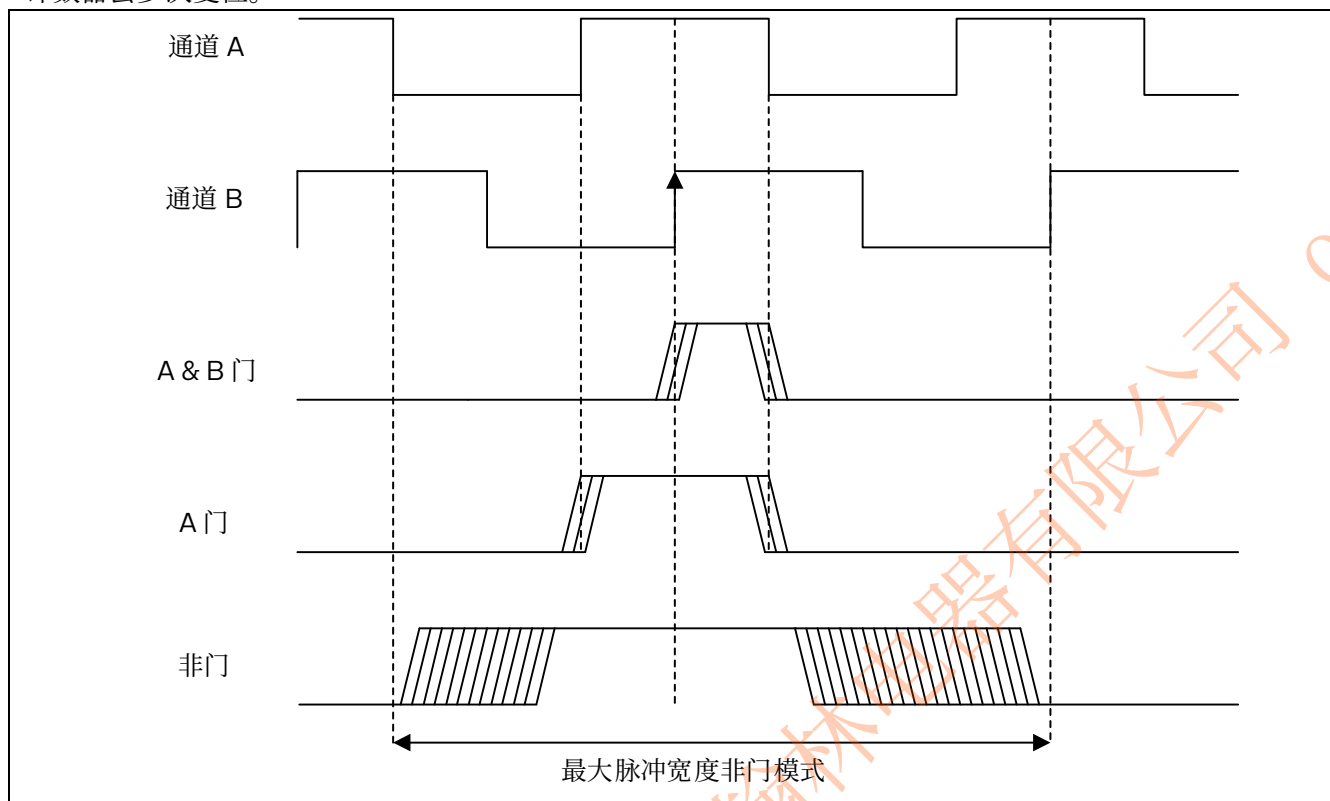


图 24.55 抖动的索引信号

定时器支持 3 种门控选项，不需要任何特定的编程。只需要定义在哪个编码器状态（即通道 A 和通道 B 状态组合）上，索引必须同步，使用 TIMx\_ECR 寄存器中的 IPOS[1:0] 位域。

索引检测事件根据计数方向的不同而有所不同，以确保速度反转时的对称操作：

- 递增计数时（DIR 位=0），计数器被复位。
- 递减计数时，计数器被设置为 TIMx\_ARR。

这使得索引可以在相同的机械角度位置上生成，无论计数方向如何。下图显示了一个简单的例子中（一个编码器每次机械旋转提供 4 个边缘），索引在哪个位置生成。

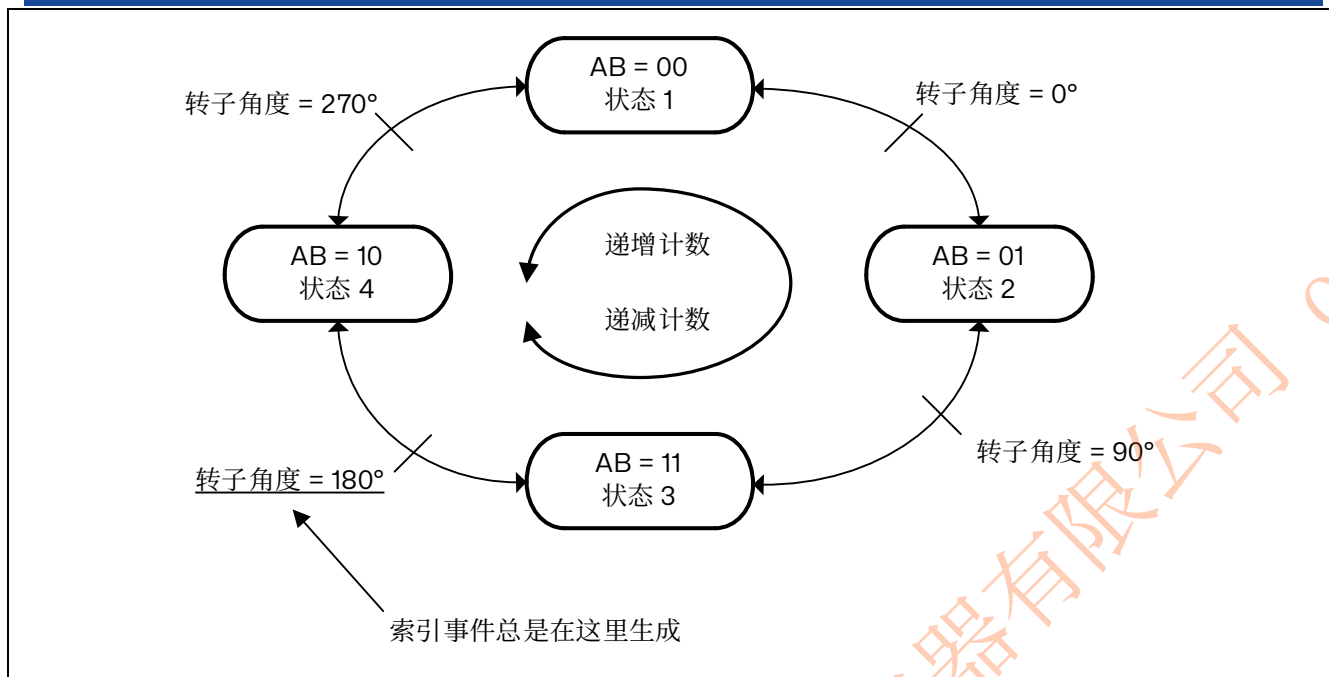


图 24.56 IPOS[1:0] = 11 时的索引生成

下图展示了 IPOS[1:0] = 11 时的波形和相应值。它表明，计数器值被强制的时刻会自动根据计数方向进行调整：

- 递增计数时 (DIR 位=0)，当编码器状态为“11” (ChA=1, ChB=1) 时，计数器设为 0。
- 递减计数时 (DIR 位=1)，当退出“11”状态时，计数器设为 TIMx\_ARR。

可以在索引检测事件发生时发出中断。

箭头指示在哪个转换上生成索引事件中断。

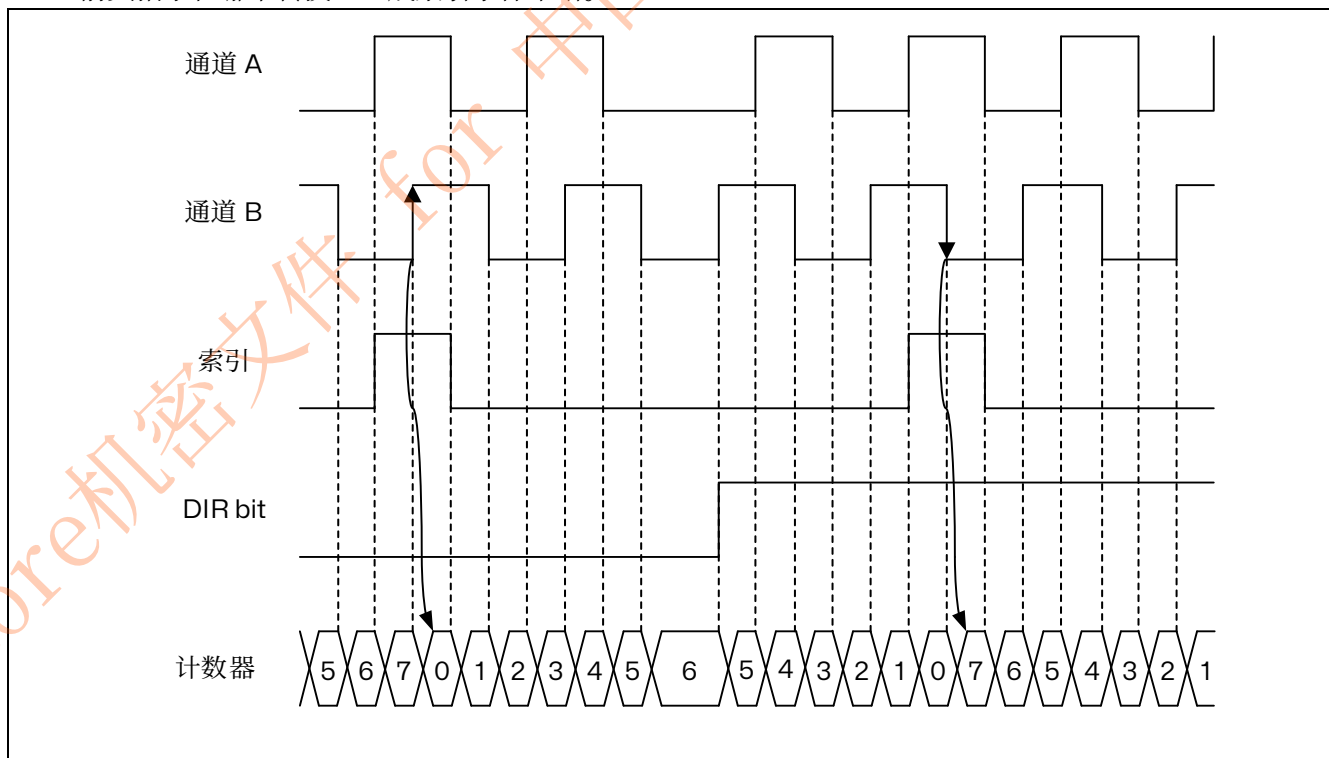


图 24.57 通道 A 上带索引门控的计数器读取 (IPOS[1:0] = 11)

下图展示了非门控模式的波形和相应值。箭头指示在哪个转换上生成索引事件。

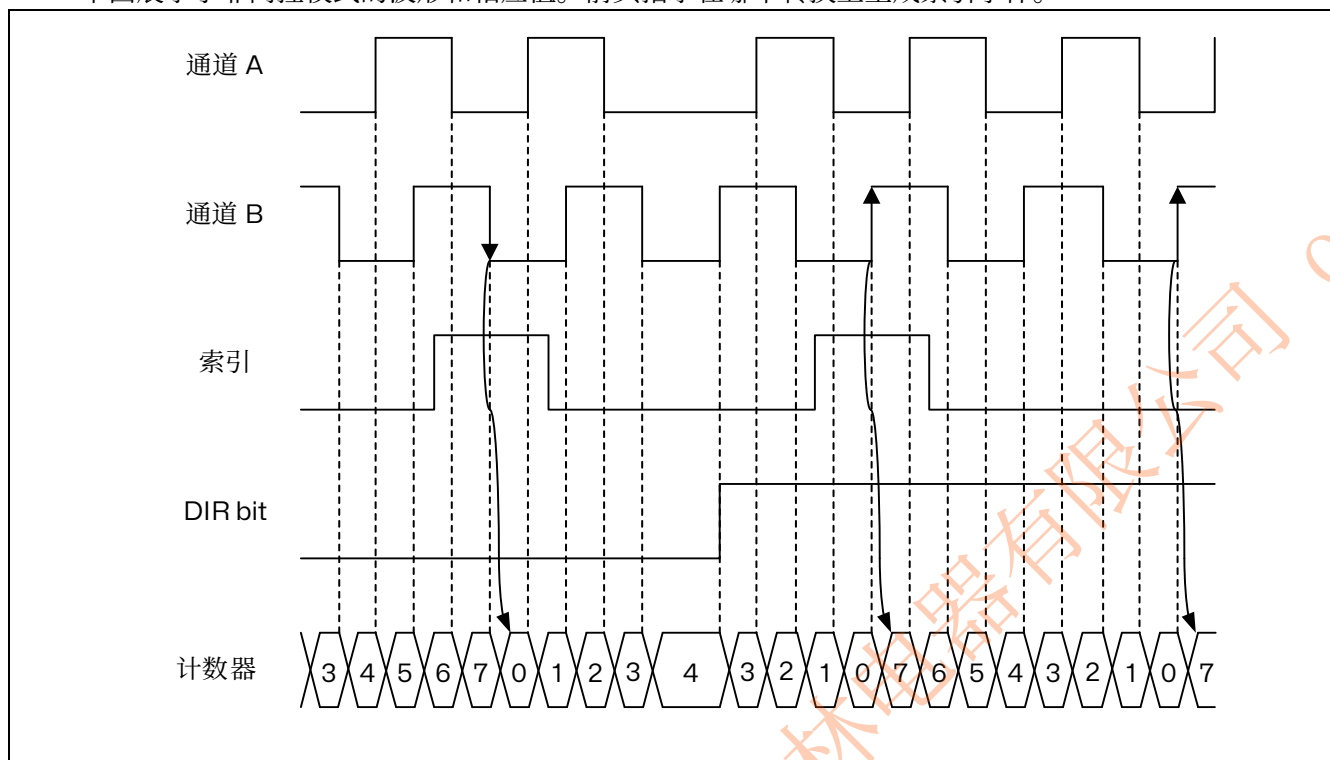


图 24.58 非门控模式下索引的计数器读取 (IPOS[1:0] = 00)

下图展示了针对各种脉冲对齐情况下的“在 A 和 B 门控”模式处理方式。箭头指示在哪个转换上生成索引事件。

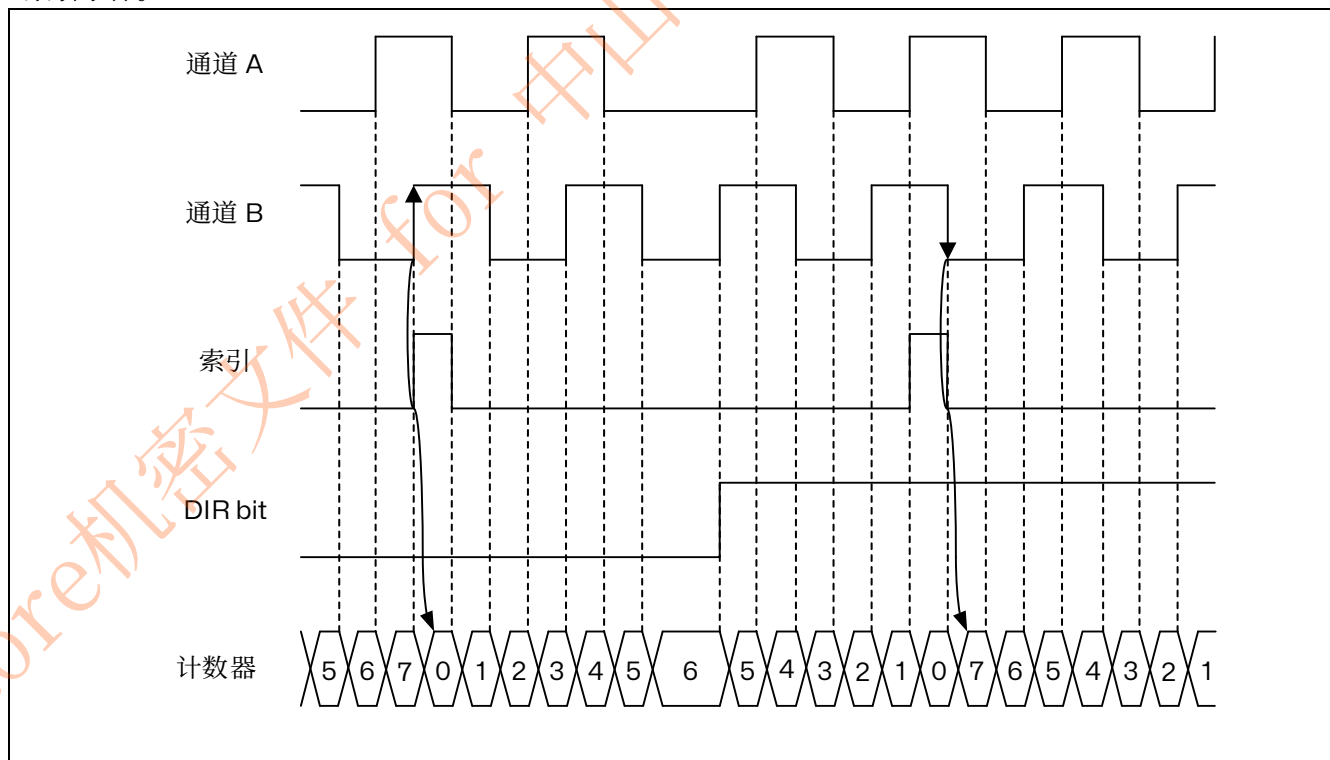


图 24.59 通道 A 和 B 上带索引门控的计数器读取

下两张图详细说明了后续索引脉冲可能比编码器时钟周期的四分之一窄的情况。

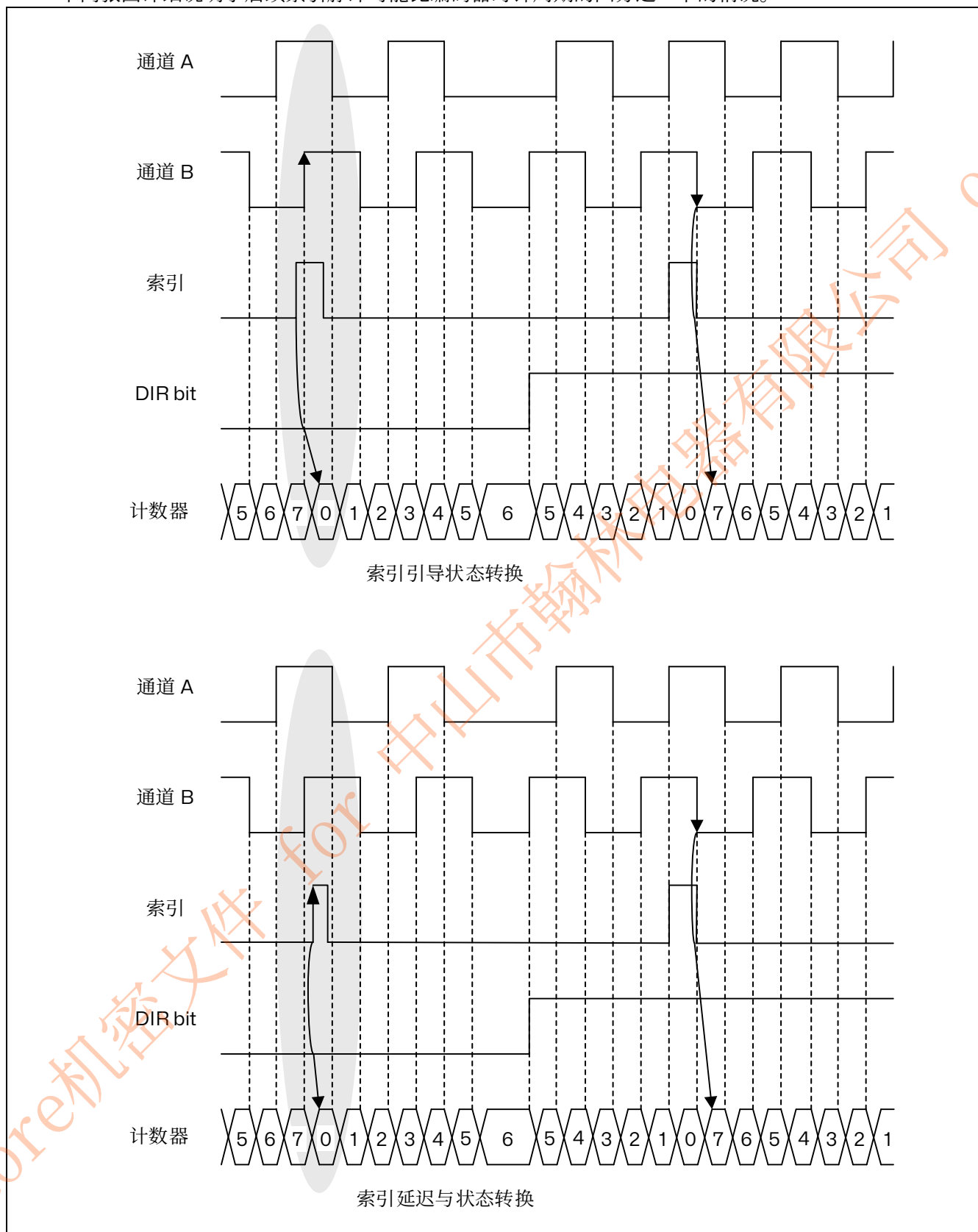


图 24.60 窄索引脉冲情况下的编码器模式行为 (IPOS[1:0] = 11)

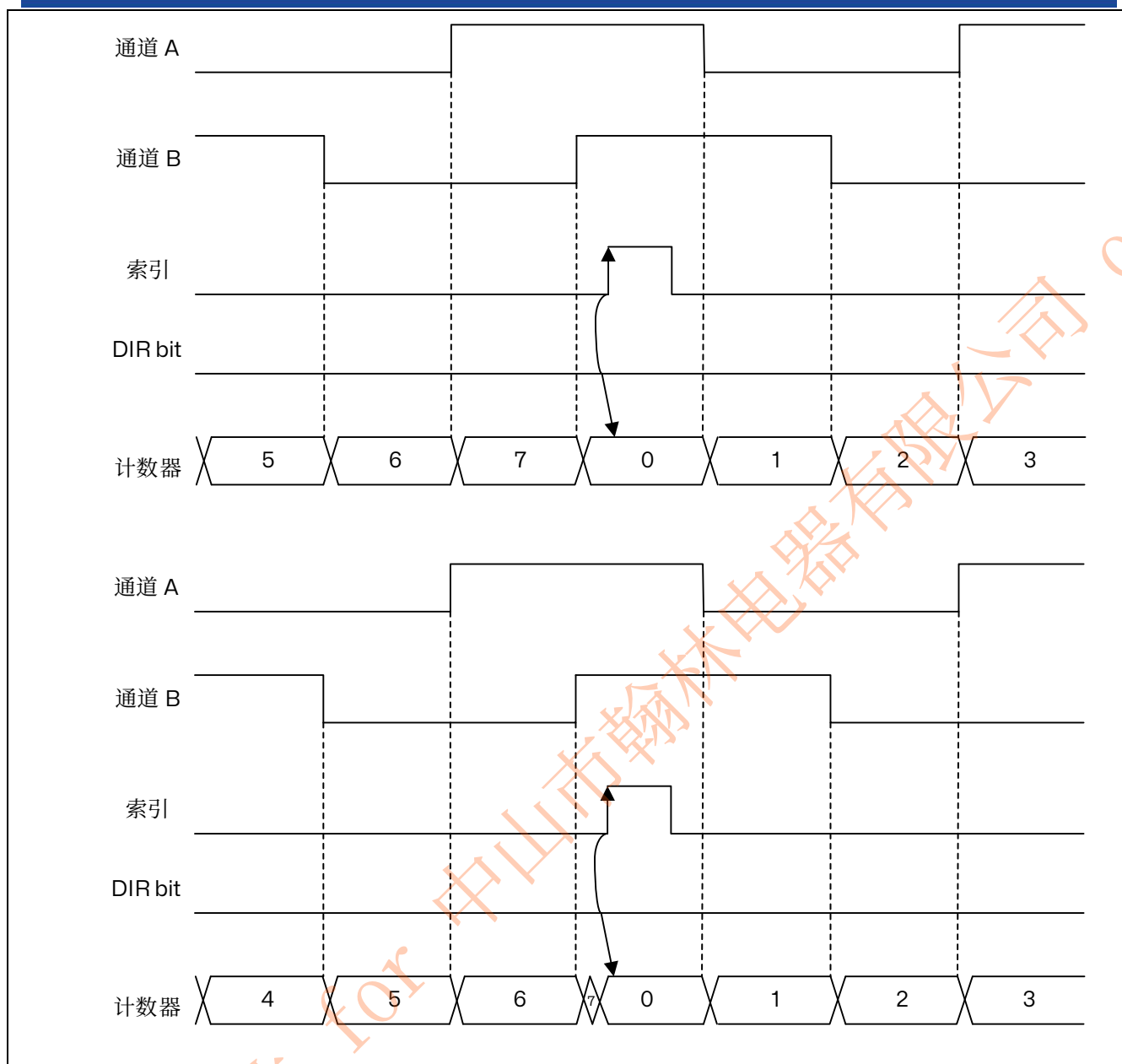


图 24.61 计数器重置窄索引脉冲（更近距离，ARR = 0x07）

下图说明了 x1 和 x2 模式下的索引管理方式。

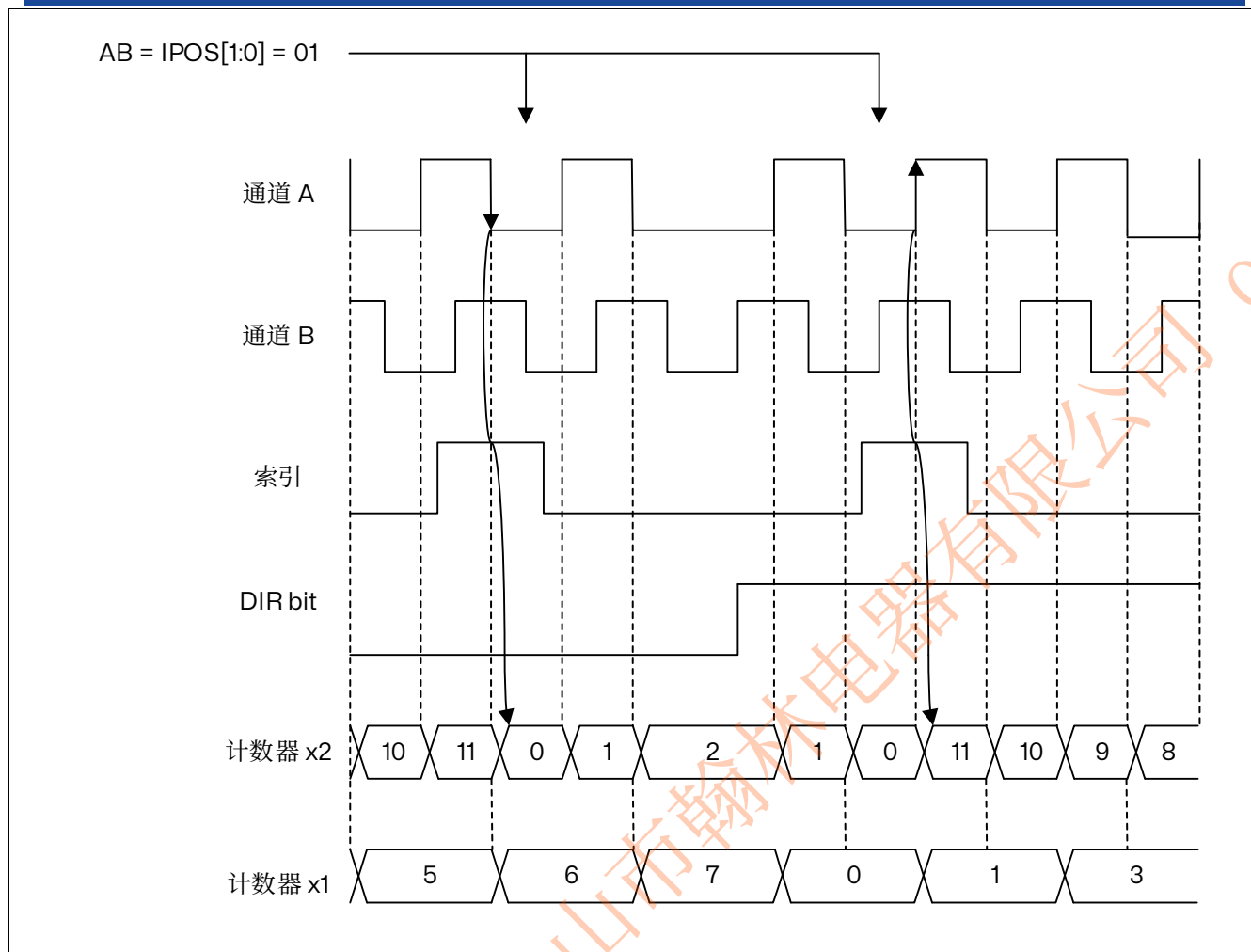


图 24.62 x1 和 x2 模式下的索引行为 (IPOS[1:0] = 01)

#### 方向索引灵敏度

TIMx\_ECR 寄存器中的 IDIR[1:0] 位域允许索引仅在选定计数方向上有效。

下图显示了根据 IDIR[1:0] 值索引和计数器重置事件之间的关系。

注：当 IE 位被复位时（禁用索引模式），必须写入 IDR[1:0] 位域。

注：在时钟+ 方向模式下不支持定向索引灵敏度。当 SMS[3:0] = 1010 或 1011 时，IDIR[1:0] 必须设置为 00。

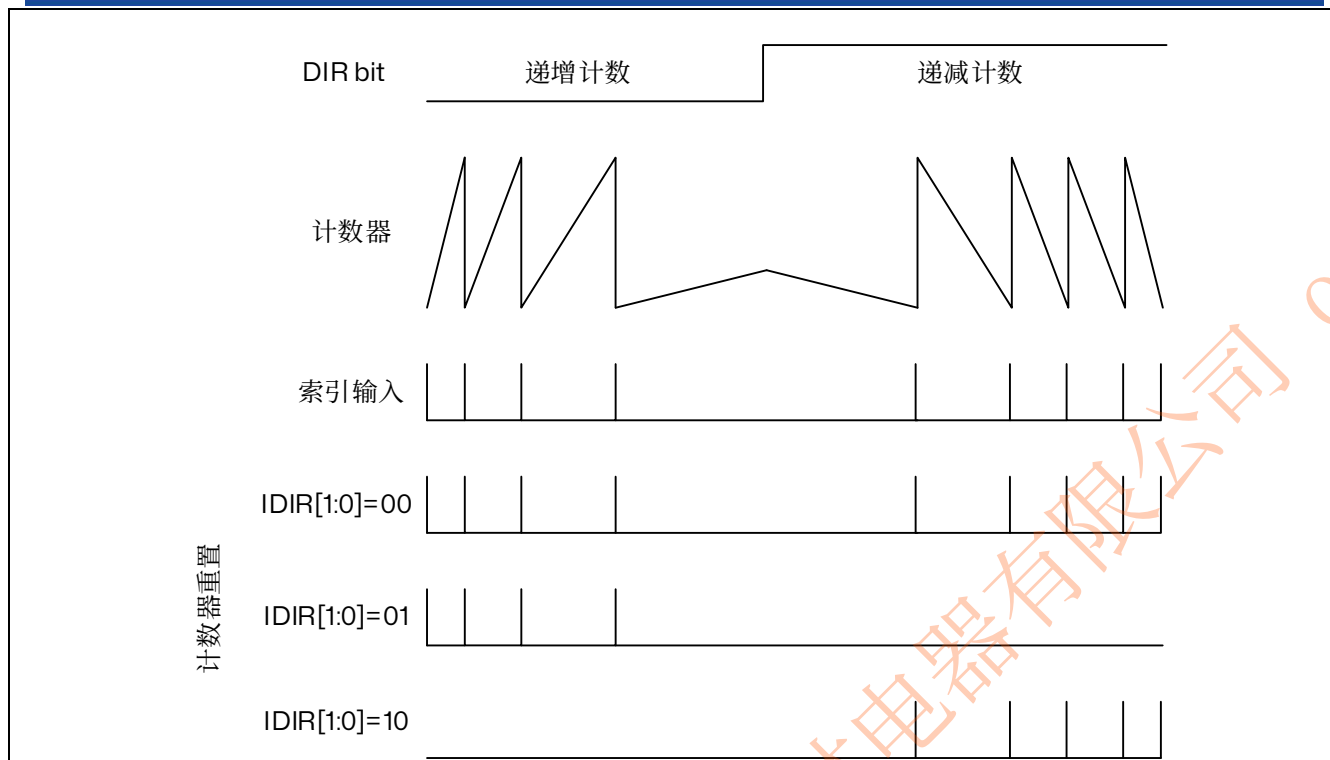


图 24.63 定向索引灵敏度

### 特殊的第一索引事件管理

TIMx\_ECR 寄存器中的 FIDX 位允许仅捕获一次索引，如下图所示。一旦第一个索引到达，任何后续索引都会被忽略。如果需要，可以通过将 FIDX 位写入 0 并再次将其设置为 1 来重新启动电路。

注：当 FIDX = 1 时，如果方向在位置 0 处改变（索引激活），索引可以发布两次（IDXFI 标志设置）。

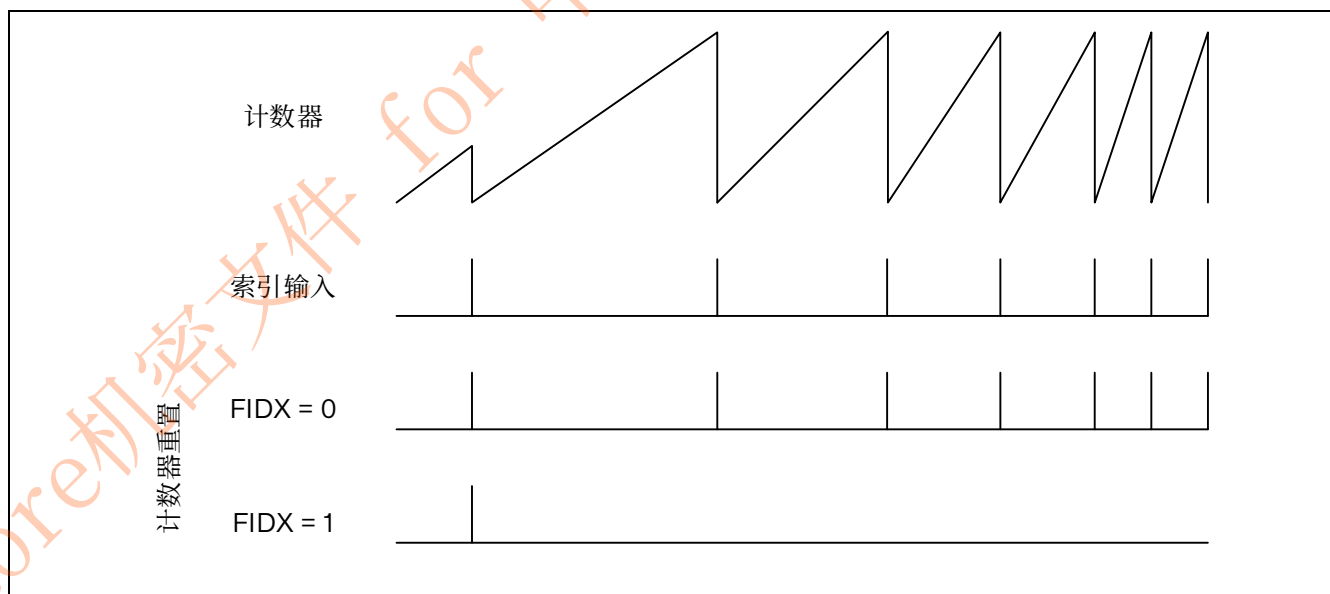


图 24.64 作为 FIDX 位设置的功能的计数器重置

### 非正交模式下的索引管理



下面两张图详细说明了当 SMS[3:0]位域等于 1010、1011、1100、1101 时，如何在方向时钟模式和时钟加方向模式下管理索引。

对于这两种模式，索引灵敏度通过 IPOS[0]位进行设置，具体如下：

- IPOS[0] = 0: 在时钟低电平上检测索引
- IPOS[0] = 1: 在时钟高电平上检测索引

IPOS[1]位不重要。

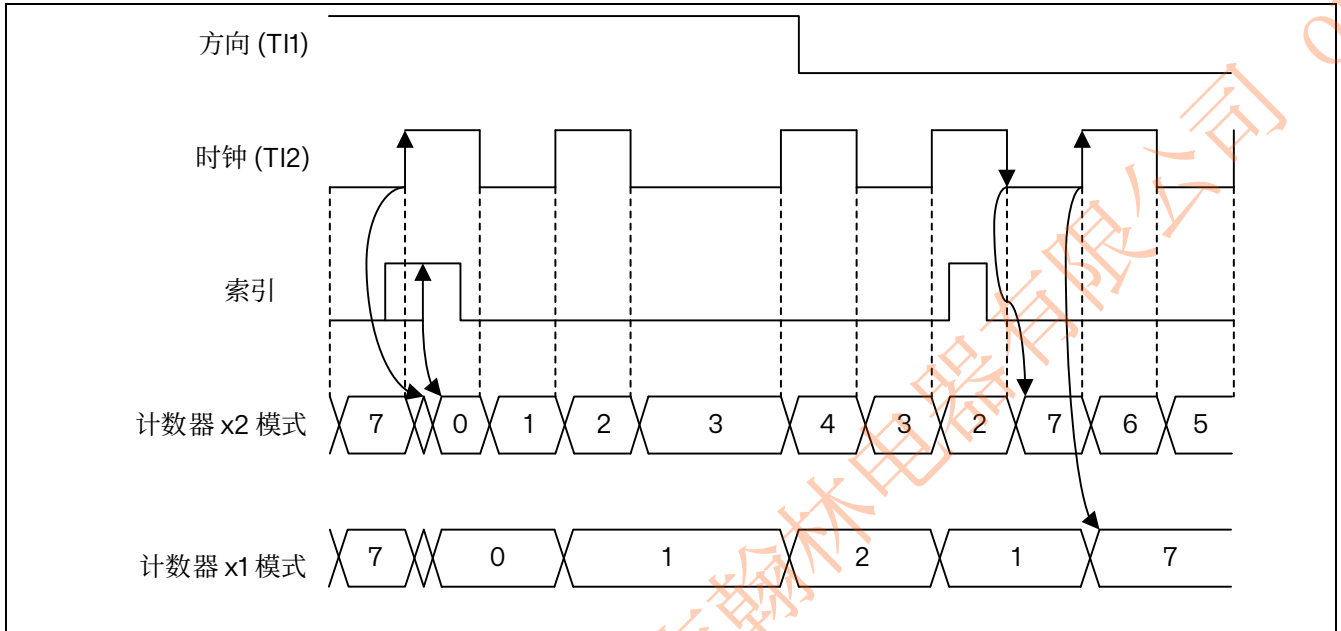


图 24.65 时钟+方向模式下的索引行为, IPOS[0] = 1

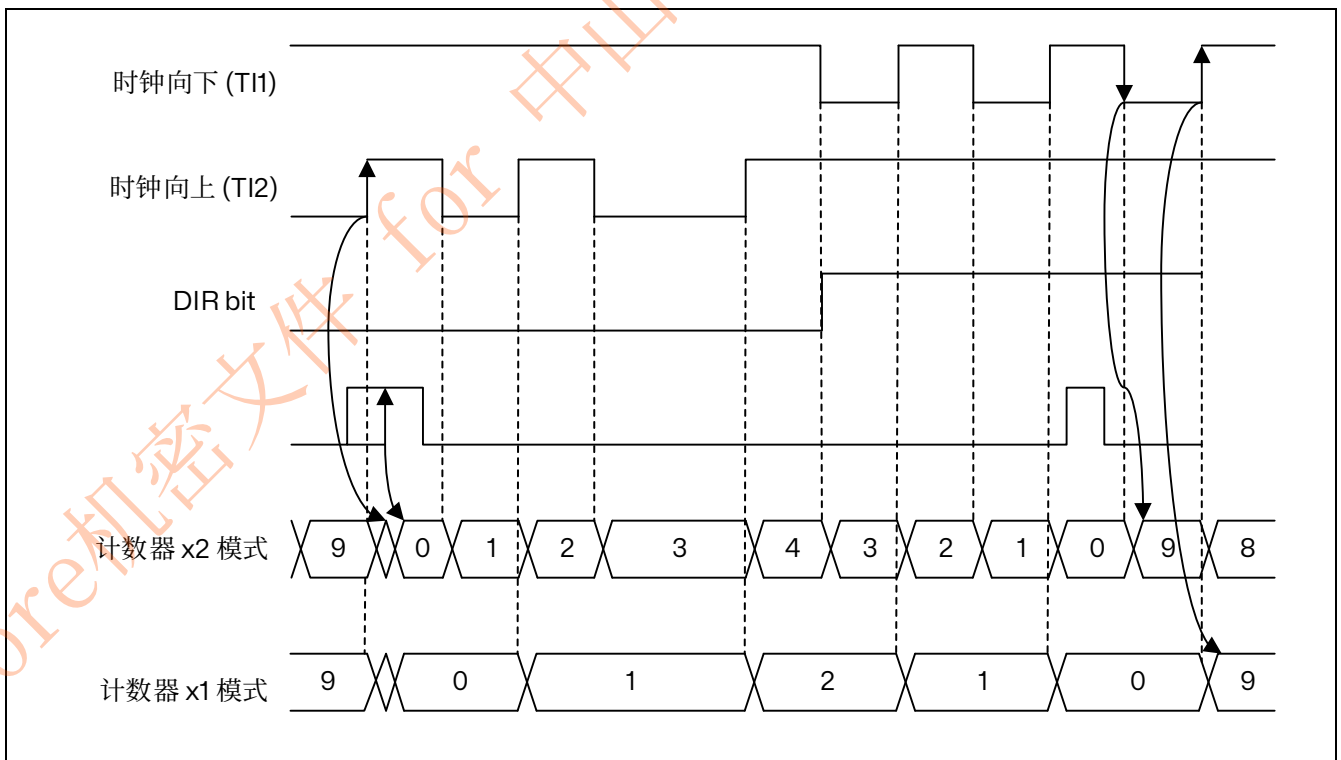


图 24.66 方向时钟模式下的索引行为, IPOS[0] = 1

## 编码器错误管理

对于具有 2 个正交信号的编码器配置，可以检测到转换错误。2 个输入的读数对应于一个 2 位灰度码，该灰度码可以表示为状态图，如以下图所示。期望单个比特立即更改。错误的转换会在 TIMx\_SR 状态寄存器中设置 TERRF 中断标志。如果 TIMx\_DIER 寄存器中的 TERRIE 位被设置，将产生转换错误中断。

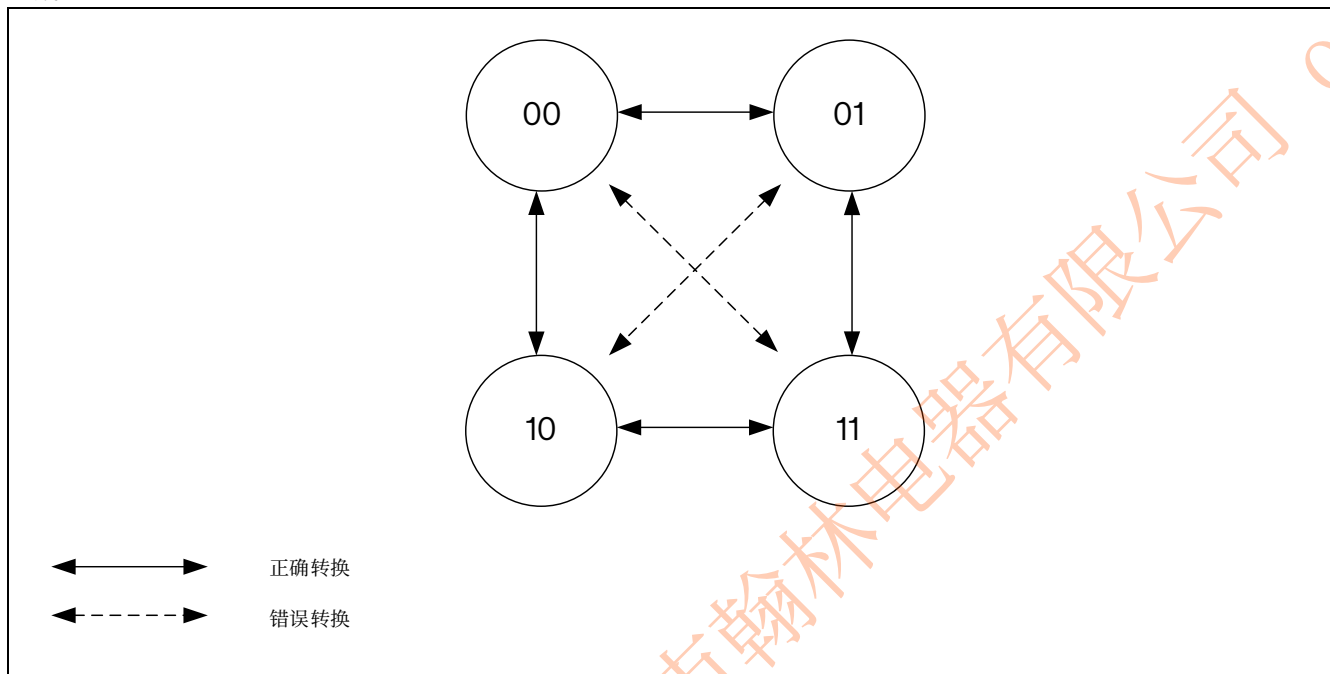


图 24.67 正交编码信号的状态图

对于具有索引信号的编码器，可以检测到导致每转脉冲过多的异常操作。一个编码器每转提供 N 个脉冲，则每转有 4xN 个计数。索引信号每 4xN 个时钟周期重置一次计数器。

如果计数器的值从 TIMx\_ARR 增加到 0 或从 0 减少到 TIMx\_ARR 的值而没有索引事件，则报告为索引位置错误。

可使用 TIMx\_ARR 寄存器来编程溢出阈值。一个 1000 线的编码器导致计数器的值在 0 和 3999 之间（在 4 倍读取模式下）。必须通过设置  $TIMx\_ARR = 3999 + 1 = 4000$  来编程溢出检测阈值。

错误断言被延迟到在递增计数时从 0 到 1 的转换。这是为了处理门控 A 和 B 模式中的窄索引脉冲，如下图所示。

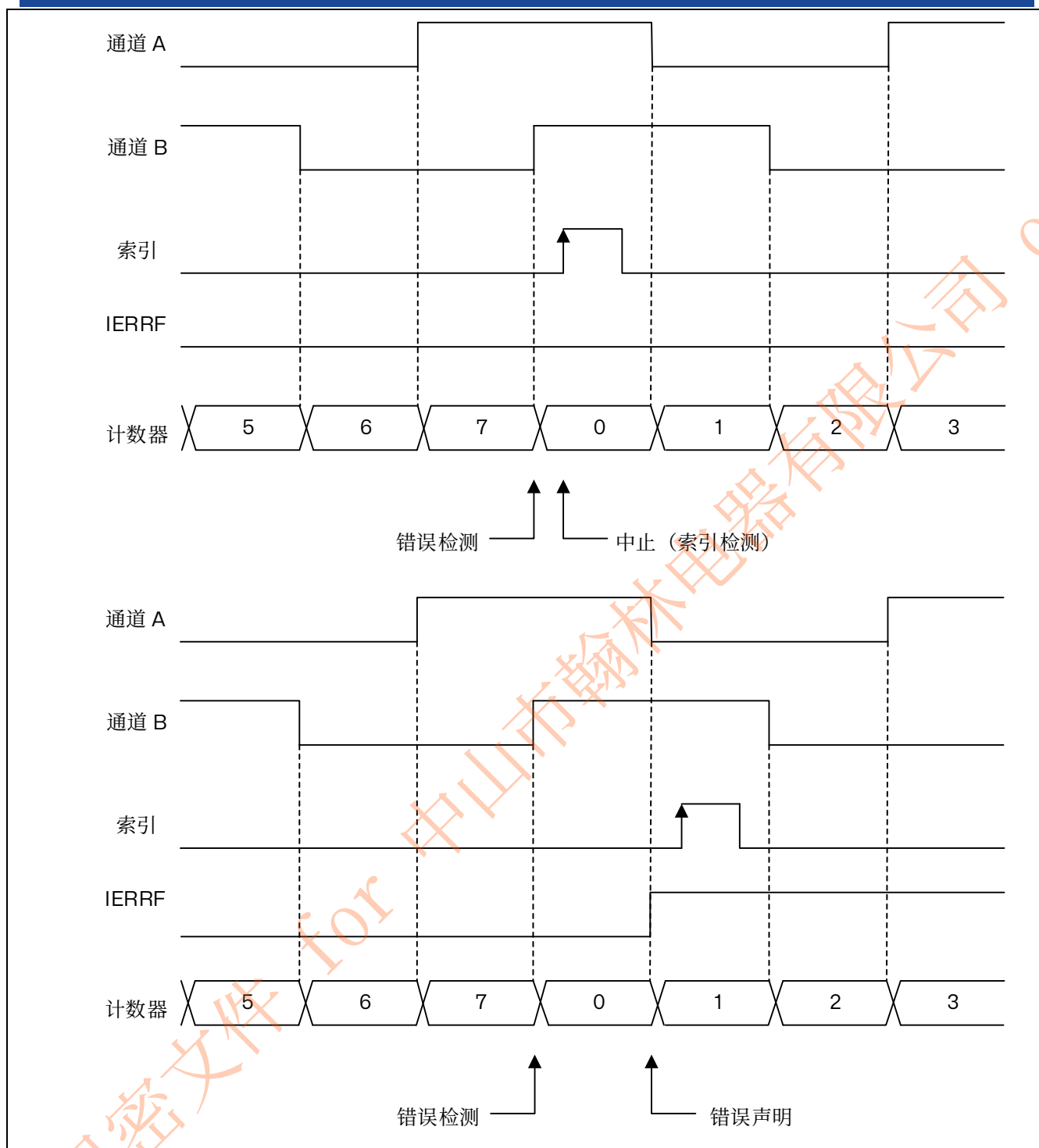


图 24.68 递增计数编码器错误检测

在递减计数模式下，检测是由从 1 到 0 的初步转换引起的。这是为了处理门控 A 和 B 模式中的窄索引脉冲，如下图所示，以避免编码器在索引检测后立即在 TIMx\_ARR 和 0 之间抖动时出现任何错误检测。

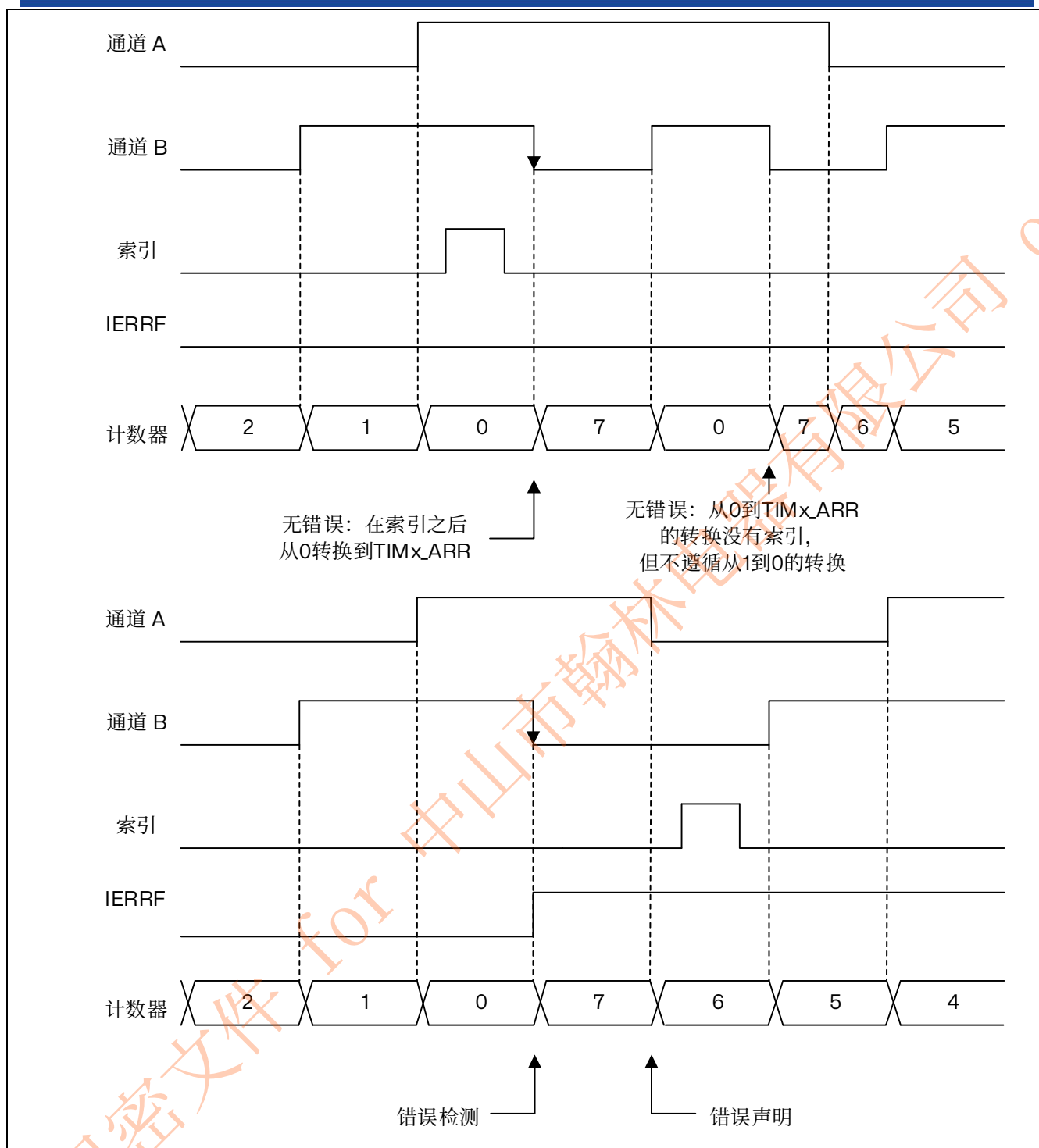


图 24.69 递减计数编码器错误检测

索引错误会在 TIMx\_SR 状态寄存器中设置 IERRF 中断标志。如果 TIMx\_DIER 寄存器中的 IERRIE 位被设置, 将产生索引错误中断。

### 功能编码器中断

编码器模式下也可使用以下中断:

- 方向改变: 在编码器模式下, 任何计数方向的改变都会导致 TIMx\_CR1 寄存器中的 DIR 位发生翻转。方向改变会设置 TIMx\_SR 状态寄存器中的 DIRF 中断标志。如果 DIRIE 位在 TIMx\_DIER 寄

寄存器中被设置，则产生方向改变中断。

- 索引事件：索引事件会设置 TIMx\_SR 状态寄存器中的 IDXF 中断标志。如果 IDXIE 位在 TIMx\_DIER 寄存器中被设置，则产生索引中断。

### 运行时更新编码器模式时从机模式选择预加载

在运行过程中，可能需要从一个编码器模式切换到另一个编码器模式。这通常在高速度下完成，以降低更新中断频率，从 x4 模式切换到 x2 模式再到 x1 模式，如下图所示。

为此，可以预加载 SMS[3:0]位。通过设置 TIMx\_SMCR 寄存器中的 SMSPE 使能位可以使能此功能。从 SMS[3:0]预加载值转换到有效值可以由 TIMx\_SMCR 寄存器中的 SMSPS 位选择触发。

- SMSPS = 0：当计数器递增时溢出并在计数器递减时下溢时，由更新事件(UEV)触发的转换。只有在索引被禁用时（位 IE = 0）才可以使用此模式。
- SMSPS = 1：由索引事件触发的转换。

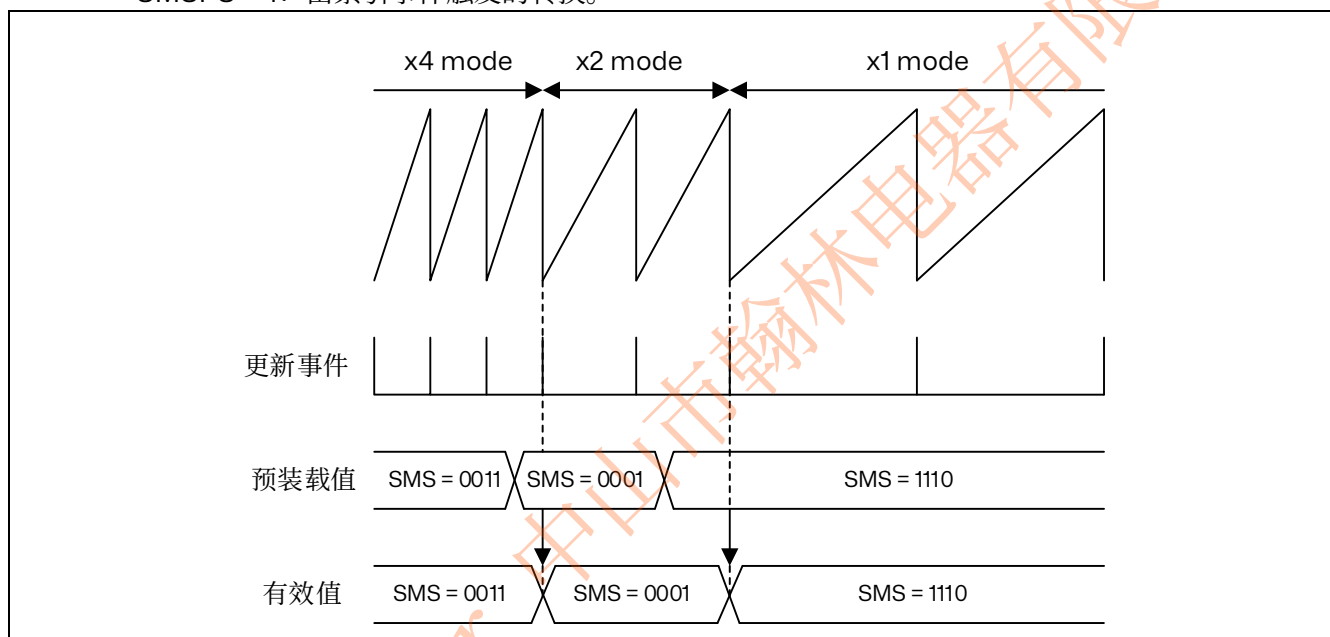


图 24.70 更新时带有预加载转移的编码器模式更改 (SMSPS = 0)

### 编码器时钟输出

编码器模式的操作原理并不完全适合于高分辨率速度测量，在低速时，因为它需要相对较长的积分时间才能获得足够数量的时钟边缘和精确的测量。

在低速时，更好的解决方案是进行边到边的时钟周期测量。这可以通过使用从属定时器来实现。定时器可以在 tim\_trgo 输出上输出编码器时钟信息。然后，从属定时器可以进行周期测量并为每个编码器时钟边缘提供速度信息。

通过将 TIMx\_CR2 寄存器中的 MMS[3:0]位域设置为 1000 可以使能此模式。对于以下 SMS[3:0]值有效：0001、0010、0011、1010、1011、1100、1101、1110、1111。任何可能导致意外行为的其他 SMS[3:0]代码是不被允许的。

### 24.4.19 方向位输出

可以将方向信号从定时器输出到 tim\_oc3n 和 tim\_oc4 输出信号 (TIMx\_CR1 寄存器中的 DIR 位的副本)。这是通过将 TIMx\_CCMR2 寄存器中的 OC3M[3:0]或 OC4M[3:0]位域设置为 1011 来实现的。

此功能可用于监测编码器模式下的计数方向（或旋转方向），或在中心对齐的 PWM 模式下有一个信

号指示上升/下降相位。

#### 24.4.20 UIF 位重映射

TIMx\_CR1 寄存器中的 IUFREMAP 位强制将更新中断标志 UIF 连续复制到定时器计数器寄存器的第 31 位 (TIMxCNT[31])。这允许以原子方式读取计数器的值和一个潜在的由 UIFCPY 标志指示的翻转条件。在特定情况下, 它可以避免由背景任务 (读取计数器) 和中断 (更新中断) 之间的处理引起的竞争条件, 从而简化计算。

UIF 和 UIFCPY 标志之间没有延迟。

在 32 位定时器实现中, 当 IUFREMAP 位被设置时, 定时器的第 31 位为在读取访问时覆盖 UIFCPY 的标志(计数器最重要的部分仅在写入模式中可访问)。

#### 24.4.21 定时器输入异或功能

TIMx\_CR2 寄存器中的 TI1S 位, 可将通道 1 的输入滤波器连接到异或门的输出, 从而将 tim\_ti1, tim\_ti2, tim\_ti3 这三个输入引脚组合在一起。

异或输出可与触发或输入捕获等所有定时器输入功能配合使用。

异或输出能够被用于所有定时器的输入功能, 如触发或输入捕获。

#### 24.4.22 定时器和外部触发的同步

TIMx 定时器以下列模式与外部触发实现同步: 复位模式、门控模式、触发模式、复位+触发和门控+复位模式。

##### 从模式: 复位模式

当触发输入信号发生变化时, 计数器及其预分频器可重新初始化。此外, 如果 TIMx\_CR1 寄存器中的 URS 位处于低电平, 则会生成更新事件 UEV。然后, 所有预装载寄存器 (TIMx\_ARR 和 TIMx\_CCRx) 都将更新。

在以下示例中, tim\_ti1 输入上出现上升沿时, 递增计数器清零:

- 将通道 1 配置为检测 tim\_ti1 的上升沿。配置输入滤波时间 (本例中不需要任何滤波, 因此保持 IC1F=0000)。由于捕获预分频器不用于触发操作, 因此无需对其进行配置。CC1S 位只选择输入捕获源, 即 TIMx\_CCMR1 寄存器中的 CC1S = 01。在 TIMx\_CCER 寄存器中写入 CC1P=0 和 CC1NP=0, 验证极性 (仅检测上升沿)。
- 在 TIMx\_SMCR 寄存器中写入 SMS=100, 将定时器配置为复位模式。在 TIMx\_SMCR 寄存器中写入 TS=101, 选择 tim\_ti1 作为输入源。
- 在 TIMx\_CR1 寄存器中写入 CEN=1, 启动计数器。

计数器使用内部时钟计数, 然后正常运转, 直到出现 tim\_ti1 上升沿。当 tim\_ti1 出现上升沿时, 计数器清零, 然后重新从 0 开始计数。同时, 触发标志 (TIMx\_SR 寄存器中的 TIF 位) 置 1, 使能中断后, 还可发送中断请求 (取决于 TIMx\_DIER 寄存器中的 TIE 和 TDE 位)。

下图显示了自动重载寄存器 TIMx\_ARR=0x36 时的相关行为。tim\_ti1 的上升沿与实际计数器复位之间的延迟是由于 tim\_ti1 输入的重新同步电路引起的。

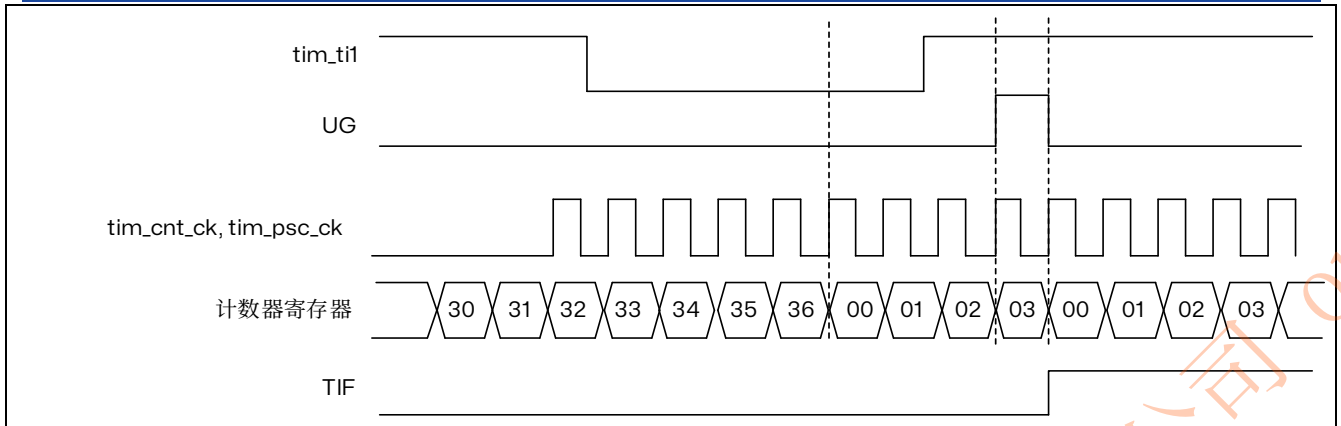


图 24.71 复位模式下的控制电路

### 从模式：门控模式

输入信号的电平可用来使能计数器。

在以下示例中，递增计数器仅在 tim\_ti1 输入为低电平时计数：

- 将通道 1 配置为检测 tim\_ti1 上的低电平。配置输入滤波时间（本例中不需要任何滤波，因此保持 IC1F=0000）。由于捕获预分频器不用于触发操作，因此无需对其进行配置。CC1S 位只选择输入捕获源，即 TIMx\_CCMR1 寄存器中的 CC1S=01。在 TIMx\_CCER 寄存器中写入 CC1P=1 和 CC1NP=0，以确定极性（仅检测低电平）。
- 在 TIMx\_SMCR 寄存器中写入 SMS=101，将定时器配置为门控模式。在 TIMx\_SMCR 寄存器中写入 TS=00101，选择 tim\_ti1 作为输入源。
- 在 TIMx\_CR1 寄存器中写入 CEN=1，使能计数器（在门控模式下，如果 CEN=0，则无论触发输入电平如何，计数器都不启动）。

只要 tim\_ti1 为低电平，计数器就开始根据内部时钟计数，直到 tim\_ti1 变为高电平时停止计数。计数器启动或停止时，TIMx\_SR 寄存器中的 TIF 标志都会置 1。

tim\_ti1 的上升沿与实际计数器停止之间的延迟是由于 tim\_ti1 输入的重新同步电路引起的。

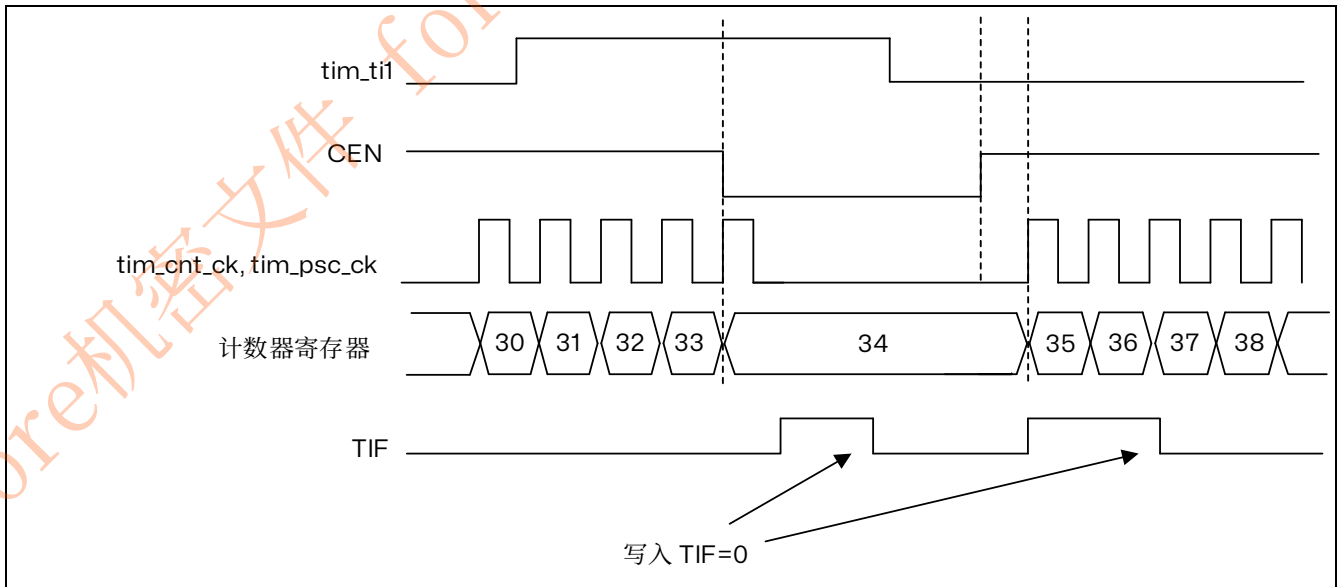


图 24.72 门控模式下的控制电路



注：由于门控模式作用于电平而非边沿，因此在门控模式下，“CCxP=CCxNP=1”（同时检测上升沿和下降沿）不发挥任何作用。

### 从模式：触发模式

所选输入上发生某一事件时可以启动计数器。

在以下示例中，tim\_ti2 输入上出现上升沿时，递增计数器启动：

- 将通道 2 配置为检测 TI2 上的上升沿。配置输入滤波时间（本例中不需要任何滤波，因此保持 IC2F=0000）。由于捕获预分频器不用于触发操作，因此无需对其进行配置。CC2S 位只选择输入捕获源，即 TIMx\_CCMR1 寄存器中的 CC2S=01。在 TIMx\_CCER 寄存器中写入 CC2P=1 和 CC2NP=0，以确定极性（仅检测低电平）。
- 在 TIMx\_SMCR 寄存器中写入 SMS=110，将定时器配置为触发模式。在 TIMx\_SMCR 寄存器中写入 TS=00110，选择 tim\_ti2 作为输入源。

当 tim\_ti2 出现上升沿时，计数器开始根据内部时钟计数，并且 TIF 标志置 1。

tim\_ti2 的上升沿与实际计数器启动之间的延迟是由于 tim\_ti2 输入的重新同步电路引起的。

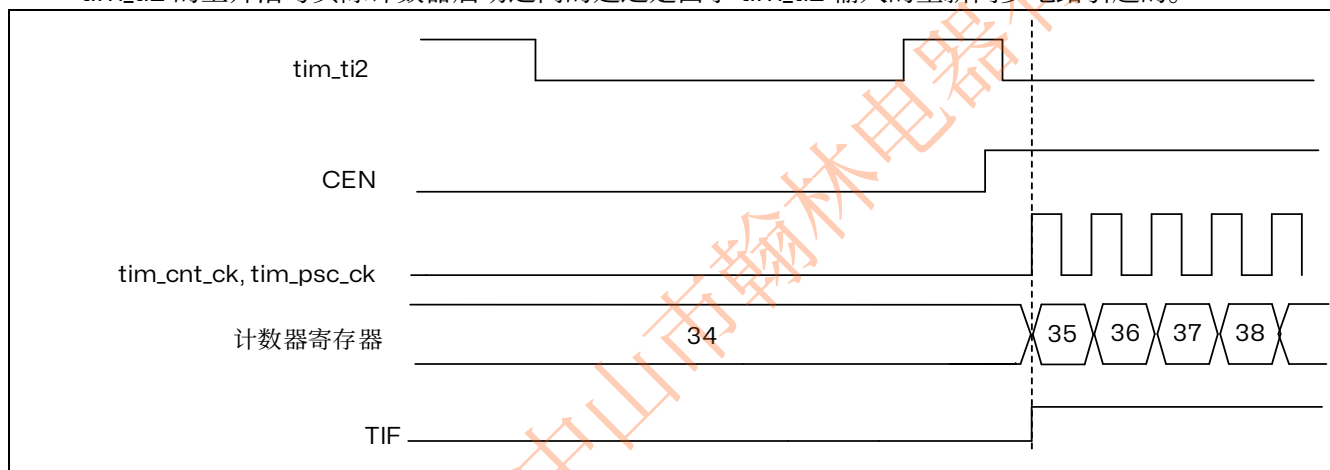


图 24.73 触发器模式下的控制电路

### 对实时更新编码器模式的从模式选择预装载

SMS[3:0]位可以预加载。这是通过在 TIMx\_SMCR 寄存器中设置 SMSPE 使能位来实现的。从 SMS[3:0]预加载到有效值的触发器是计数器溢出时发生的更新事件（UEV）。

### 从模式 - 组合重置 + 触发模式

在这种情况下，所选触发输入（tim\_trgi）的上升沿会重新初始化计数器，此模式用于单脉冲模式。

### 从模式 - 组合重置 + 门控模式

当触发输入（tim\_trgi）为高时，计数器使能。计数器停止并一旦触发器变为低，计数器就会重置。计数器的启动和停止都是受控的。

此模式允许检测超出范围的 PWM 信号（占空比超过最大期望值）。

### 从模式：外部时钟模式 2+触发模式

外部时钟模式 2 可与另一种从模式（外部时钟模式 1 和编码器模式除外）结合使用。这种情况下，

tim\_etr\_in 信号用作外部时钟输入，在复位模式、门控模式或触发模式下工作时，可选择另一个输入作为触发输入。不建议通过 TIMx\_SMCR 寄存器中的 TS 位来选择 tim\_etr\_in 作为 tim\_trgi。

在以下示例中，只要 tim\_ti1 出现上升沿，递增计数器即会在 tim\_etr\_in 信号的每个上升沿处递增：

1. 通过对 TIMx\_SMCR 寄存器进行如下编程，配置外部触发输入电路：

- ETF = 0000：无滤波器
- ETPS = 00：禁止预分频器
- ETP = 0：检测 tim\_etr\_in 的上升沿，并写入 ECE=1，以使能外部时钟模式 2。

2. 如下配置通道 1，以检测 TI 的上升沿：

- IC1F = 0000：无滤波器
- 由于捕获预分频器不用于触发操作，因此无需对其进行配置。
- TIMx\_CCMR1 寄存器中的 CC1S = 01，只选择输入捕获源
- TIMx\_CCER 寄存器中的 CC1P = 0 和 CC1NP=0，以确定极性（仅检测上升沿）。

3. 在 TIMx\_SMCR 寄存器中写入 SMS=110，将定时器配置为触发模式。在 TIMx\_SMCR 寄存器中写入 TS=00101，选择 tim\_ti1 作为输入源。

tim\_ti1 出现上升沿时将使能计数器并且 TIF 标志置 1。然后计数器在 tim\_etr\_in 出现上升沿时计数。

tim\_etr\_in 信号的上升沿与实际计数器复位之间的延迟是由于 tim\_etrp 输入的重新同步电路引起的。

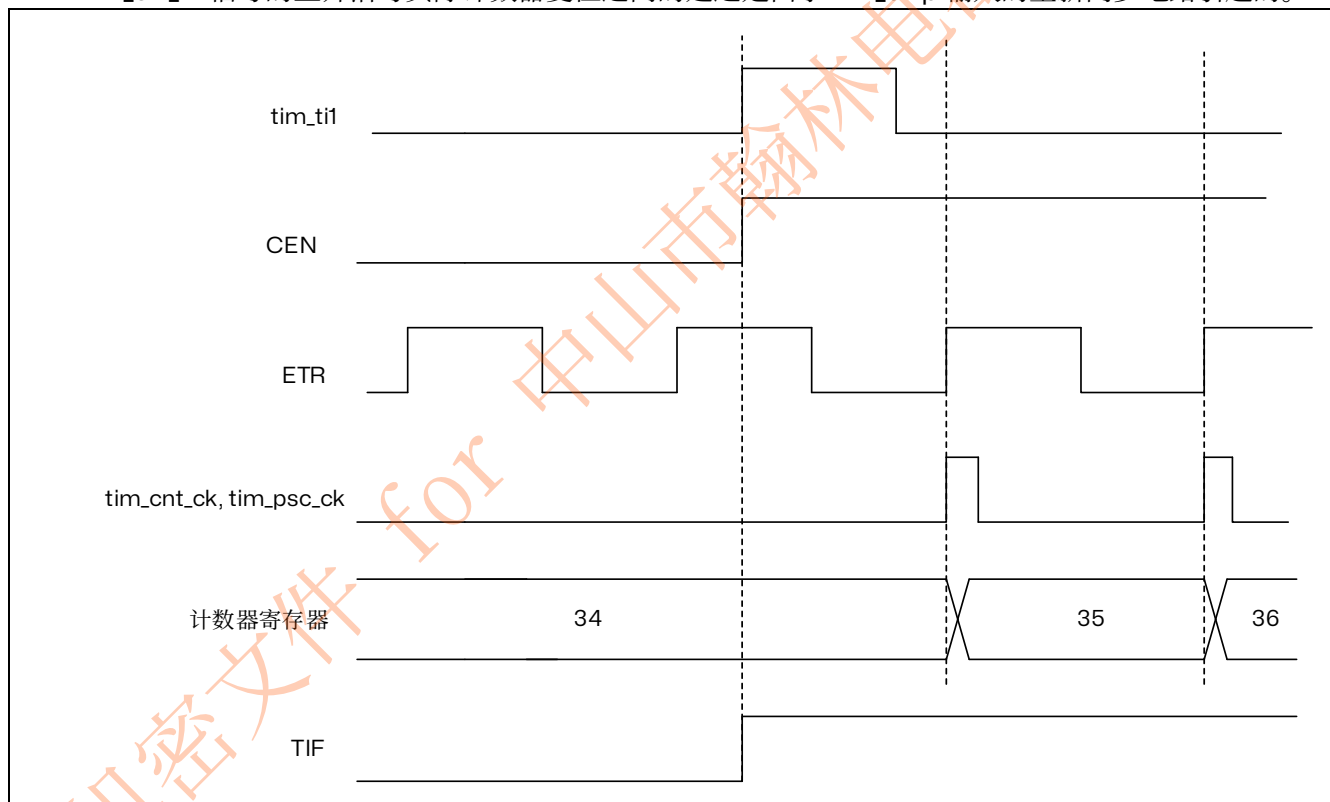


图 24.74 外部时钟模式 2+触发模式下的控制电路

### 24.4.23 定时器同步

TIMx 定时器从内部连接在一起，以实现定时器同步或级联。当某个定时器配置为主模式时，可对另一个配置为从模式的定时器的计数器执行复位、启动、停止操作或为其提供时钟。下图显示了触发选择和主模式选择模块的概况。

下图简要介绍了触发选择和主模式选择框图。

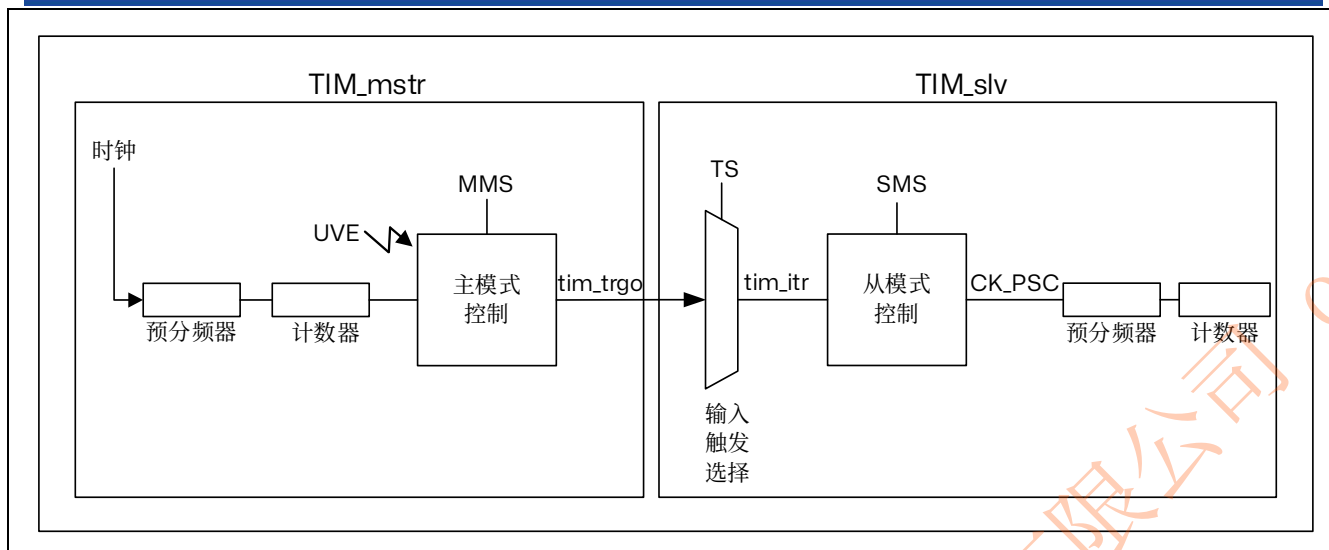


图 24.75 主/从定时器的例子

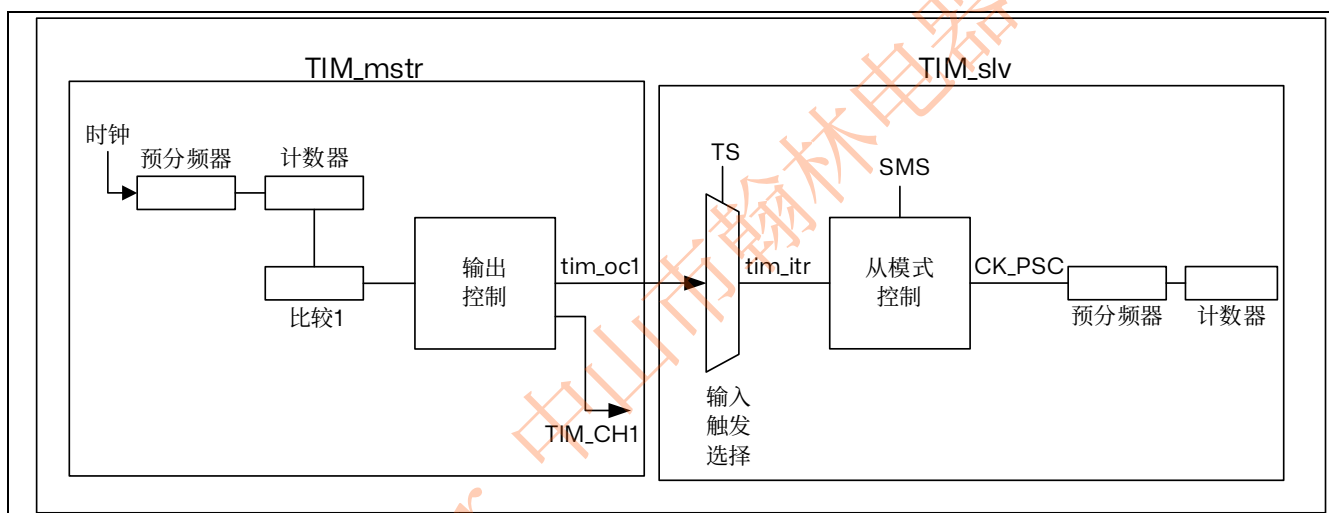


图 24.76 仅定时器通道 1 主/从连接示例

注：只有一个通道的计时器（参见上图）不具有主模式。然而，tim\_oc1 输出信号可以作为从属定时器的触发信号（参见TIMx 内部触发连接表：TIM2/TIM3/TIM4/TIM5 引脚和内部信号）。

必须将tim\_oc1 信号脉冲宽度编程为至少为目的地定时器的 2 个时钟周期，以确保从属定时器检测到触发信号。例如，如果目的地定时器tim\_ker\_ck 时钟比源定时器慢 4 倍，则 OC1 脉冲宽度必须是 8 个时钟周期。

### 使用一个定时器作为另一个定时器的预分频器

例如，可以将 TIM\_mstr 配置为 TIM\_slv 的预分频器。请参见上图。为此：

1. 将 TIM\_mstr 配置为主模式，以便每次发生更新事件 UEV 时都输出一个周期性触发信号。如果在 TIM\_mstr\_CR2 寄存器中写入 MMS=010，则每次生成更新事件时，tim\_trgo 都会输出一个上升沿。
2. 要将 TIM\_mstr 的 tim\_trgo 输出连接到 TIM\_slv，必须将 TIM\_slv 配置为从模式，使用 ITR0 作为内部触发。通过 TIM\_slv\_SMCR 寄存器中的 TS 位（写入 TS=000）可对此进行选择。
3. 然后将从模式控制器设为外部时钟模式 1（在 TIM\_slv\_SMCR 寄存器中写入 SMS=111）。这样一

来，TIM\_slv 的时钟将由 TIM\_mstr 周期性触发信号的上升沿（与 TIM\_mstr 的计数器上溢对应）提供。

- 最后必须通过这两个定时器的相应 CEN 位（TIMx\_CR1 寄存器）置 1 同时使能二者。

注：如果选择 TIM\_mstr 的 tim\_ocx 信号作为触发输出（MMS=1xx），该信号的上升沿将用于驱动 TIM\_slv 的计数器。

### 使用一个定时器使能另一个定时器

本例中通过 TIM\_mstr 的输出比较 1 来使能 TIM\_slv。相关连接图，参考下图。仅当 TIM\_mstr 的 tim\_oc1ref 为高电平时，TIM\_slv 才根据分频后的内部时钟进行计数。两个计数器的时钟频率都基于 tim\_ker\_ck 通过预分频器执行 3 分频 ( $f_{tim\_cnt\_ck} = f_{tim\_ker\_ck} / 3$ )

- 将 TIM\_mstr 配置为主模式，发送其输出比较 1 参考信号（tim\_oc1ref）作为触发输出（TIM\_mstr\_CR2 寄存器中的 MMS=100）。
- 配置 TIM\_mstr 的 tim\_oc1ref 波形（TIM\_mstr\_CCMR1 寄存器）。
- 配置 TIM\_slv 以接收来自 TIM\_mstr 的输入触发（TIM\_slv\_SMCR 寄存器中的 TS=00010）。
- 将 TIM\_slv 配置为门控模式（TIM\_slv\_SMCR 寄存器中的 SMS=101）。
- 通过向 CEN 位（TIM\_slv\_CR1 寄存器）写入“1”使能 TIM\_slv。
- 通过向 CEN 位（TIM\_mstr\_CR1 寄存器）写入“1”启动 TIM\_mstr。

注：计数器 2 的时钟与计数器 1 不同步，此模式仅影响 TIM\_slv 的计数器使能信号。

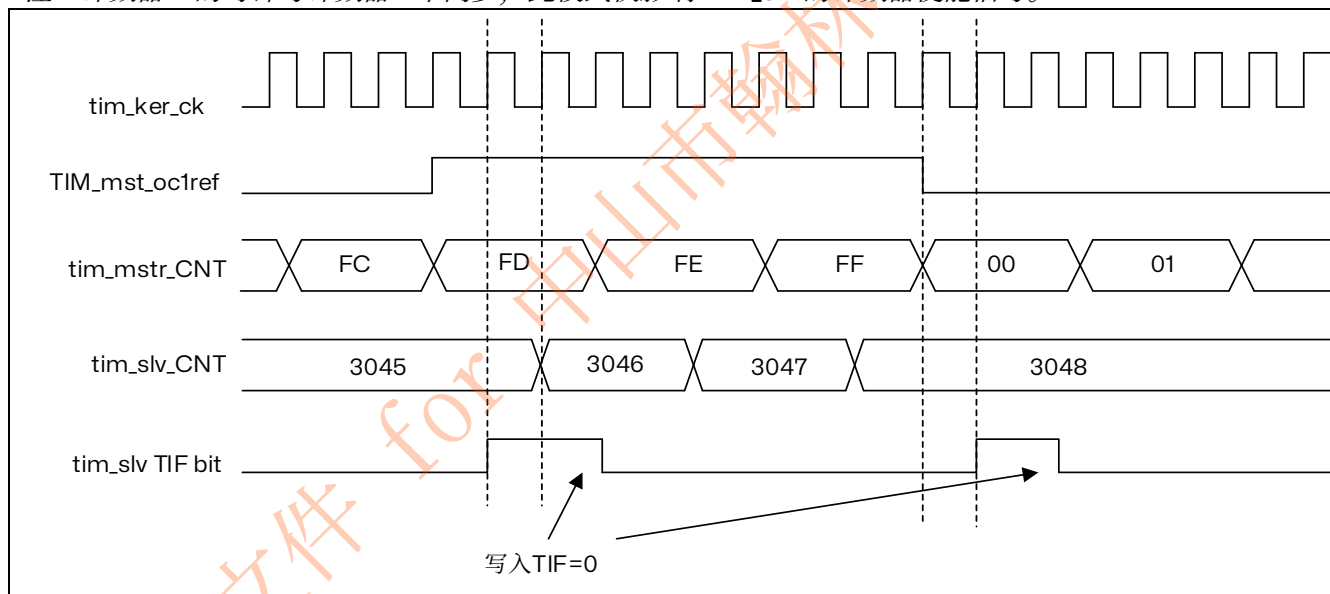


图 24.77 TIM\_mstr 的 tim\_oc1ref 控制 TIM\_slv

在上图示例中，TIM\_slv 的计数器和预分频器在启动前未进行初始化。因此从各自的当前值开始计数。启动 TIM\_mstr 之前，通过复位这两个定时器可以从指定值开始计数。这样便可以在定时器计数器中写入所需的任意值。两个定时器都可通过软件使用 TIMx\_EGR 寄存器中的 UG 位轻松复位。

在下一示例中，TIM\_mstr 与 TIM\_slv 同步。TIM\_mstr 为主模式，从 0 开始计数。TIM\_slv 为从模式，从 0xE7 开始计数。两个定时器的预分频比相同。在 TIM\_mstr\_CR1 寄存器中通过向 CEN 位写入“0”来禁止 TIM\_mstr 时，TIM\_slv 将停止：

- 将 TIM\_mstr 配置为主模式，发送其输出比较 1 参考信号（tim\_oc1ref）作为触发输出（TIM\_mstr\_CR2 寄存器中的 MMS=100）。
- 配置 TIM\_mstr 的 tim\_oc1ref（TIM\_mstr\_CCMR1 寄存器）。

3. 配置 TIM\_slv 以接收来自 TIM\_mstr 的输入触发 (TIM\_slv\_SMCR 寄存器中的 TS=00010)。
4. 将 TIM\_slv 配置为门控模式 (TIM\_slv\_SMCR 寄存器中的 SMS=101)。
5. 通过向 UG 位 (TIM\_mstr\_EGR 寄存器) 写入 “1” 复位 TIM\_mstr。
6. 通过向 UG 位 (TIM\_slv\_EGR 寄存器) 写入 “1” 复位 TIM\_slv。
7. 通过在 TIM\_slv 的计数器 (TIM\_slv\_CNT) 中写入 “0xE7” 使 TIM\_slv 初始化为 0xE7。
8. 通过向 CEN 位 (TIM\_slv\_CR1 寄存器) 写入 “1” 使能 TIM\_slv。
9. 通过向 CEN 位 (TIM\_mstr\_CR1 寄存器) 写入 “1” 启动 TIM\_mstr。
10. 通过向 CEN 位 (TIM\_mstr\_CR1 寄存器) 写入 “0” 停止 TIM\_mstr。

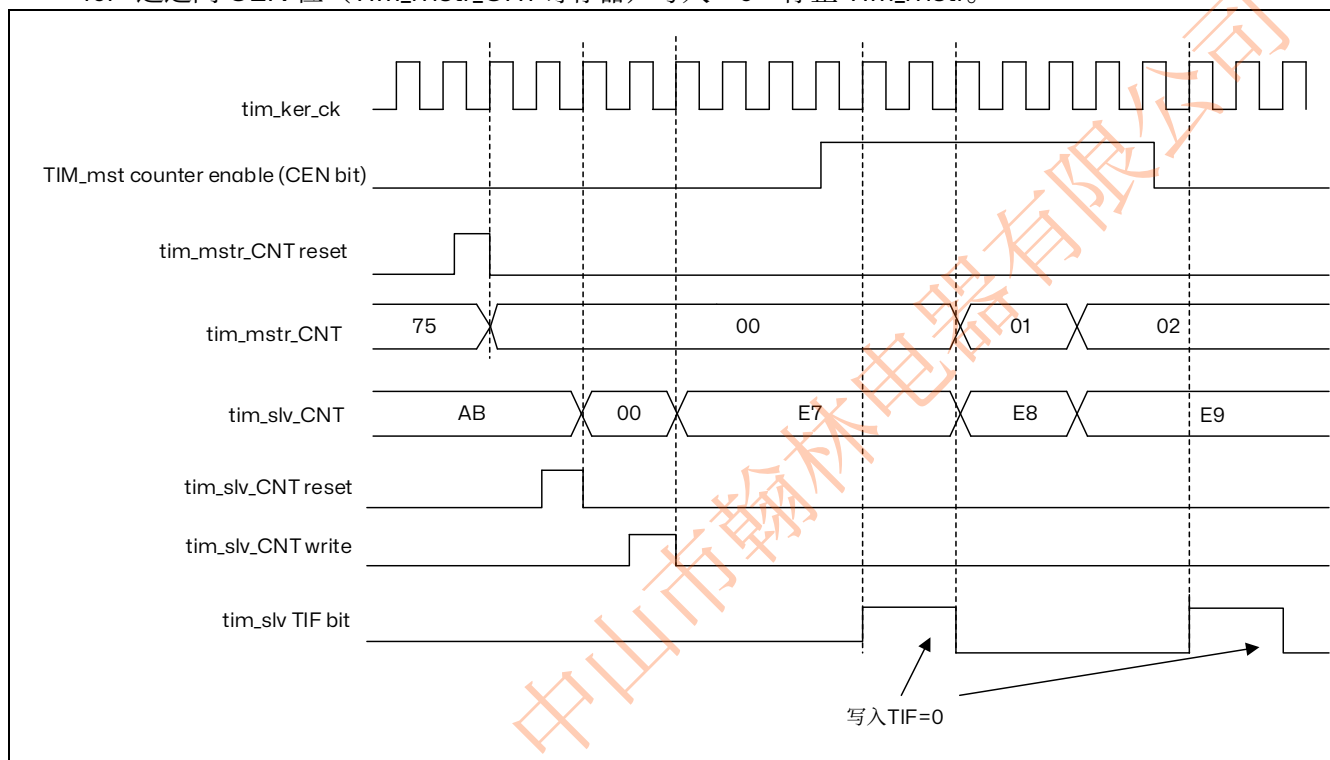


图 24.78 通过使能 TIM\_mstr 可以控制 TIM\_slv

### 使用一个定时器去启动另一个定时器

本例中使用 TIM\_mstr 的更新事件使能 TIM\_slv。相关连接图，请参见下图。只要 TIM\_mstr 生成更新事件，TIM\_slv 便根据分频后的内部时钟从当前值（可以不为 0）开始计数。TIM\_slv 收到触发信号时，其 CEN 位自动置 1，并且计数器开始计数，直到向 TIM\_slv\_CR1 寄存器的 CEN 位写入 “0” 后停止计数。两个计数器的时钟频率都基于 CK\_INT 通过预分频器执行 3 分频 ( $f_{tim\_cnt\_ck} = f_{tim\_ker\_ck}/3$ )。

1. 将 TIM\_mstr 配置为主模式，发送其更新事件 (UEV) 作为触发输出 (TIM\_mstr\_CR2 寄存器中的 MMS=010)。
2. 配置 TIM\_mstr 的周期 (TIM\_mstr\_ARR 寄存器)。
3. 配置 TIM\_slv 以接收来自 TIM\_mstr 的输入触发 (TIM\_slv\_SMCR 寄存器中的 TS=00010)。
4. 将 TIM\_slv 配置为触发模式 (TIM\_slv\_SMCR 寄存器中的 SMS=110)。
5. 通过向 CEN 位 (TIM\_mstr\_CR1 寄存器) 写入 “1” 启动 TIM\_mstr。

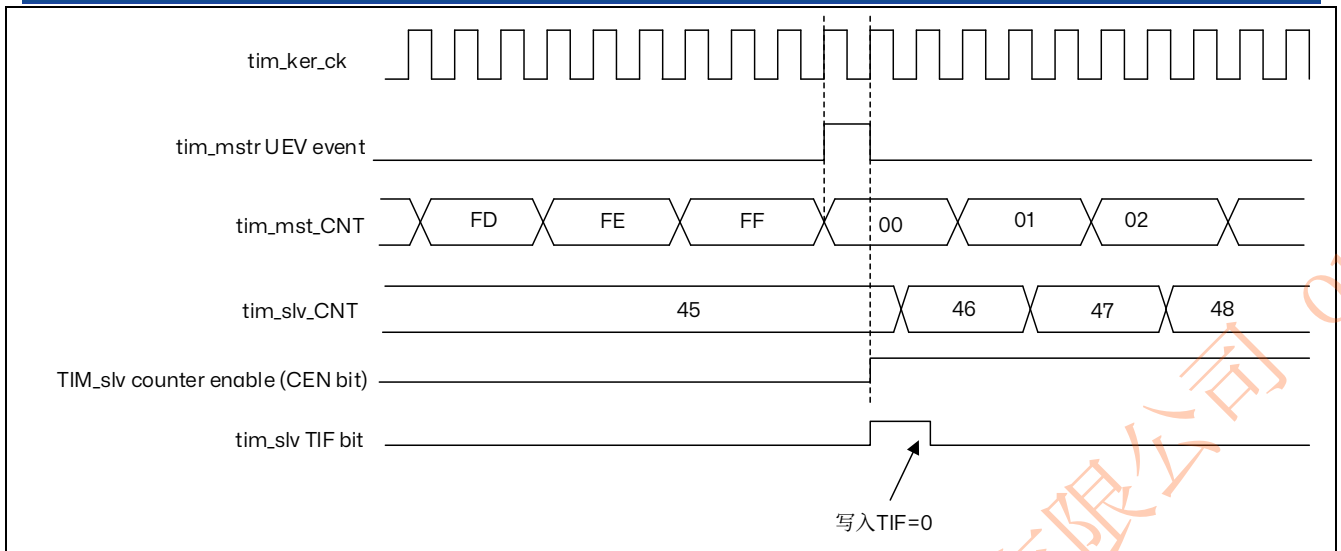


图 24.79 使用 TIM\_mstr 的更新触发 TIM\_slv

如上述示例所示，用户可以在开始计数之前初始化两个计数器。下图显示了与上图具有相同配置，只不过处于触发模式（TIM\_slv\_SMCR 寄存器中的 SMS=110）而非门控模式的计数行为。

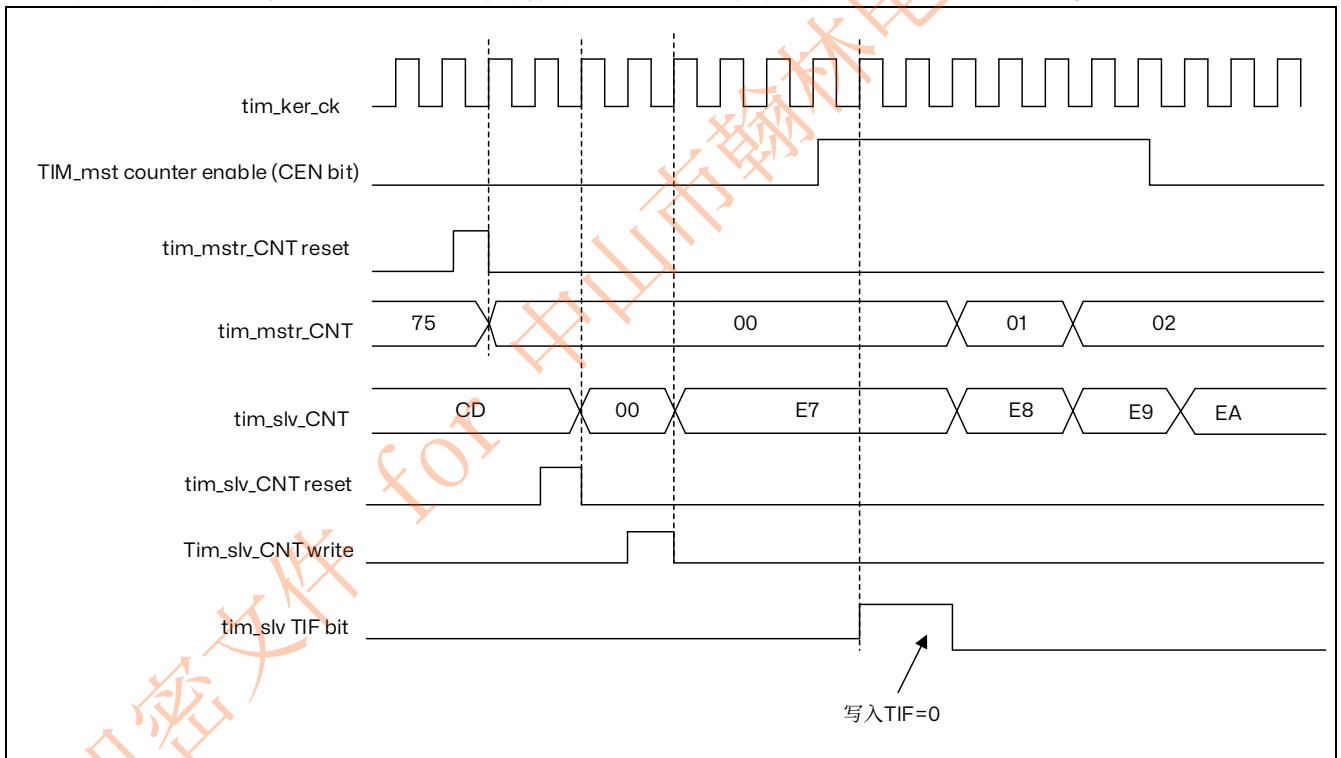


图 24.80 利用 TIM\_mstr 的使能触发 TIM\_slv

### 使用一个外部触发同步地启动 2 个定时器

本例中，TIM\_mstr 的 tim\_ti1 输入出现上升沿时使能 TIM\_mstr，使能 TIM\_mstr 的同时使能 TIM\_slv。要确保两个计数器对齐，TIM\_mstr 必须配置为主/从模式（对应的 tim\_ti1 为从，对应 TIM\_slv 为主）：

1. 将 TIM\_mstr 配置为主模式，发送其使能信号作为触发输出（TIM\_mstr\_CR2 寄存器中



MMS=001)。

2. 将 TIM\_mstr 配置为从模式以接收来自 tim\_ti1 的输入触发 (TIM\_mstr\_SMCR 寄存器中的 TS=00100)。
3. 将 TIM\_mstr 配置为触发模式 (TIM\_mstrSMCR 寄存器中的 SMS=110)。
4. 通过写入 MSM=1 (TIM\_mstr\_SMCR 寄存器) 将 TIM\_mstr 配置为主/从模式。
5. 配置 TIM\_slv 以接收来自 TIM\_mstr 的输入触发 (TIM\_slv\_SMCR 寄存器中的 TS=00000)。
6. 将 TIM\_slv 配置为触发模式 (TIM\_slv\_SMCR 寄存器中的 SMS=110)。

当 tim\_ti1 (TIM\_mstr) 出现上升沿时, 两个计数器开始根据内部时钟同步计数, 并且两个 TIF 标志都置 1。

注: 本例中, 两个定时器都在启动之前进行了初始化 (通过将各自的 UG 位置 1)。两个计数器都从 0 开始计数, 但可以通过对任意一个计数器寄存器 (TIMx\_CNT) 进行写操作, 在二者之间轻松插入一个偏移量。可注意到主/从模式在 TIM\_mstr 的 CNT\_EN 与 CK\_PSC 之间产生了延迟。

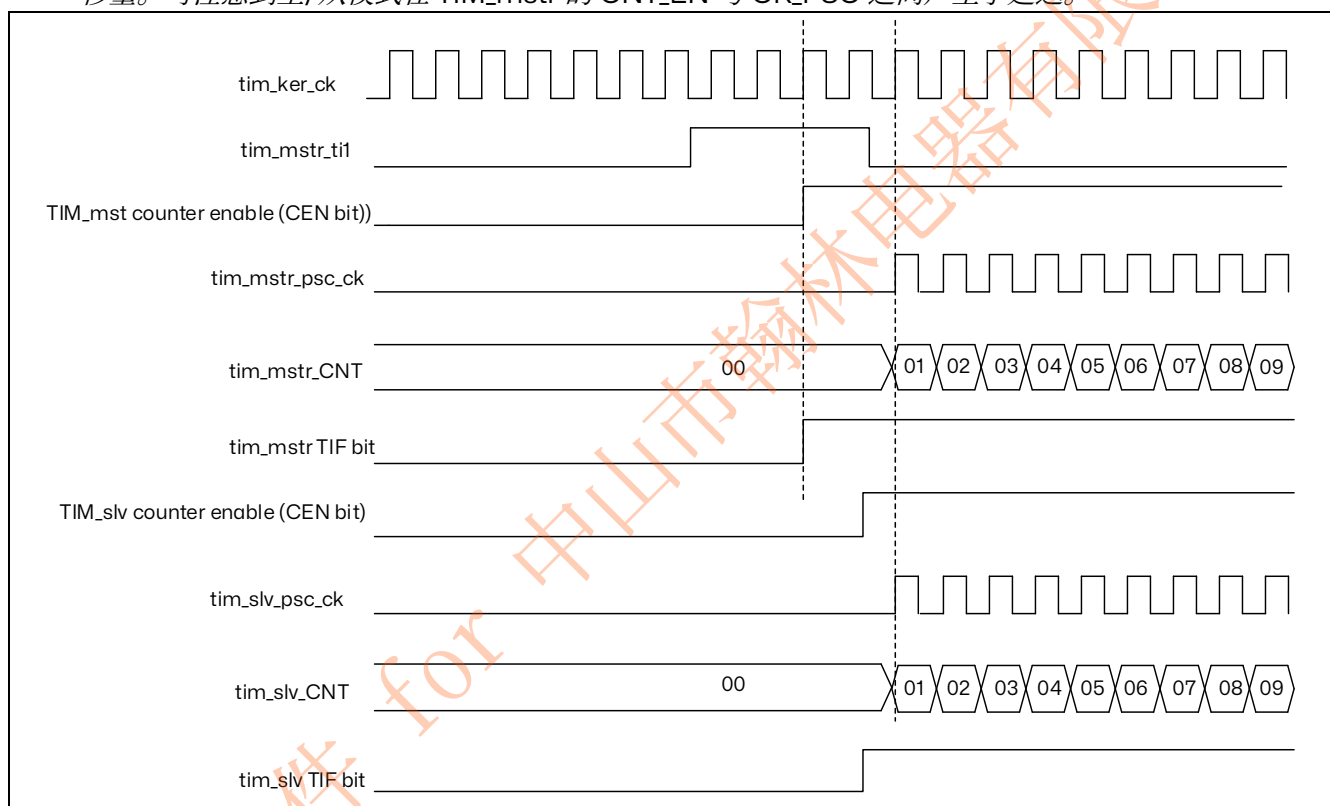


图 24.81 使用 TIM\_mstr 的 tim\_ti1 输入触发 TIM\_mstr 和 TIM\_slv

注: 接收 tim\_trgo 信号的从外设 (定时器, ADC 等) 的时钟必须在从主定时器接收事件之前使能, 并且时钟频率 (预分频器) 在从主定时器接收触发器时不能实时更改。

## 24.4.24 ADC 触发

定时器可以用各种内部信号 (例如重置、使能或比较事件) 生成 ADC 触发事件。

注意: 接收 tim\_trgo 信号的从外设 (例如定时器, ADC) 的时钟必须在从主定时器接收事件之前使能, 并且时钟频率 (预分频器) 在从主定时器接收触发器时不能实时更改。

## 24.4.25 DMA 突发模式

TIMx 定时器具有在单个事件上生成多个 DMA 请求的能力。其主要目的是能够在不增加软件开销的情



况下多次重新编程定时器的一部分，但它也可以用于在固定间隔内连续读取多个寄存器。

DMA 控制器的目标地址是唯一的，必须指向虚拟寄存器 TIMx\_DMAR。在给定的定时器事件上，定时器启动一系列 DMA 请求（突发）。对 TIMx\_DMAR 寄存器的每次写入实际上都被重定向到定时器寄存器之一。

TIMx\_DCR 寄存器中的 DBL[4:0]位设置了 DMA 突发长度。当对 TIMx\_DMAR 地址进行读取或写入访问时，定时器会识别出突发传输，即传输的数量（以半字或字节为单位）。

TIMx\_DCR 寄存器中的 DBA[4:0]位定义了 DMA 传输的 DMA 基地址（当通过 TIMx\_DMAR 地址进行读取/写入访问时）。DBA 是从 TIMx\_CR1 寄存器的地址开始定义的偏移量：

示例：

00000: TIMx\_CR1

00001: TIMx\_CR2

00010: TIMx\_SMCR

作为一个例子，定时器 DMA 突发模式用于在更新事件发生时更新 CCRx 寄存器（x = 2, 3, 4）的内容，其中 DMA 将半字传输到 CCRx 寄存器中。

这是通过以下步骤完成的：

6. 配置相应的 DMA 通道如下：

- 无滤波器 DMA 通道外设地址是 DMAR 寄存器的地址。
- DMA 通道内存地址是 RAM 中缓冲区的地址，其中包含要通过 DMA 传输到 CCRx 寄存器的数据。
- 要传输的数据数量 = 3（见下文注释）。
- 循环模式禁用。

7. 通过配置 DBA 和 DBL 位域来配置 DCR 寄存器，如下所示：

DBL = 3 次传输，DBA = 0xE。

8. 使能 TIMx 更新 DMA 请求（在 DIER 寄存器中设置 UDE 位）。

9. 使能 TIMx

10. 使能 DMA 通道

此示例适用于每个 CCRx 寄存器都需要更新一次的情况。例如，如果要更新每个 CCRx 寄存器两次，则要传输的数据数量应为 6。让我们以一个在 RAM 中包含 data1, data2, data3, data4, data5 和 data6 的缓冲区为例。数据按照以下方式传输到 CCRx 寄存器：在第一个更新 DMA 请求时，data1 传输到 CCR2，data2 传输到 CCR3，data3 传输到 CCR4；在第二个更新 DMA 请求时，data4 传输到 CCR2，data5 传输到 CCR3，data6 传输到 CCR4。

注：可以将空值写入保留的寄存器。

#### 24.4.26 TIM2/TIM3/TIM4/TIM5 DMA 请求

TIM2/TIM3/TIM4/TIM5 可以生成 DMA 请求，如下表所示。

表 24.15 DMA 请求

| DMA 请求信号    | DMA 缩写  | DMA 请求  | 使能控制位 |
|-------------|---------|---------|-------|
| tim_upd_dma | TIM_UP  | 更新      | UDE   |
| tim_cc1_dma | TIM_CH1 | 捕获/比较 1 | CC1DE |
| tim_cc2_dma | TIM_CH2 | 捕获/比较 2 | CC2DE |
| tim_cc3_dma | TIM_CH3 | 捕获/比较 3 | CC3DE |

|             |          |         |       |
|-------------|----------|---------|-------|
| tim_cc4_dma | TIM_CH4  | 捕获/比较 4 | CC4DE |
| tim_trg_dma | TIM_TRIG | 触发      | TDE   |

注：某些定时器的DMA 请求可能未连接到DMA 控制器。请参考DMA 部分以获取更多详细信息。

#### 24.4.27 调试模式

当微控制器进入调试模式（带有 FPU 的 Cortex®-M4 内核停止）时，TIMx 计数器可以继续正常工作或停止。

在调试模式下，每个定时器的行为可以通过 Debug 支持（DBG）模块的专用配置位进行编程。有关更多详细信息，请参阅调试部分。

#### 24.5 TIM2/TIM3/TIM4/TIM5 低功耗模式

表 24.16 低功耗模式对 TIM2/TIM3/TIM4/TIM5 的影响

| 模式 | 描述                        |
|----|---------------------------|
| 睡眠 | 无影响，外设是运行的。中断会导致设备退出睡眠模式。 |
| 停机 | 定时器操作被停止并保留其寄存器内容。无法生成中断。 |
| 待机 | 定时器被断电并且退出待机模式后必须重新初始化。   |

#### 24.6 TIM2/TIM3/TIM4/TIM5 中断

TIM2/TIM3/TIM4/TIM5 可以生成多个中断，如下表所示。

表 24.17 中断请求

| 中断缩写        | 中断事件    | 事件标志  | 使能控制位  | 中断清除方法    | 从睡眠模式退出 | 从停机和待机退出 |
|-------------|---------|-------|--------|-----------|---------|----------|
| TIM_UP      | 更新      | UIF   | UIE    | UIF 写 0   | 是       | 否        |
| TIM_CC      | 捕获/比较 1 | CC1IF | CC1IE  | CC1IF 写 0 | 是       | 否        |
|             | 捕获/比较 2 | CC2IF | CC2IE  | CC2IF 写 0 | 是       | 否        |
|             | 捕获/比较 3 | CC3IF | CC3IE  | CC3IF 写 0 | 是       | 否        |
|             | 捕获/比较 4 | CC4IF | CC4IE  | CC4IF 写 0 | 是       | 否        |
| TIM_TRG     | 触发      | TIF   | TIE    | TIF 写 0   | 是       | 否        |
| TIM_DIR_IDX | 索引      | IDXF  | IDXIE  | IDXF 写 0  | 是       | 否        |
|             | 方向      | DIRF  | DIRIE  | DIRF 写 0  | 是       | 否        |
| TIM_IERR    | 索引错误    | IERRF | IERRIE | IERRF 写 0 | 是       | 否        |
| TIM_TER     | 转换错误    | TERRF | TERRIE | TERRF 写 0 | 是       | 否        |

## 24.7 TIM2/TIM3/TIM4/TIM5 寄存器描述

在寄存器描述中使用的缩写列表请参考章节 2.1。

外设寄存器可以通过半字（16 位）或字（32 位）进行访问。

### 24.7.1 TIMx 控制寄存器 1 (TIMx\_CR1) (x=2~5)

偏移地址：0x000

复位值：0x0000

| 15       | 14 | 13 | 12         | 11           | 10  | 9        | 8  | 7    | 6        | 5  | 4   | 3   | 2   | 1    | 0   |
|----------|----|----|------------|--------------|-----|----------|----|------|----------|----|-----|-----|-----|------|-----|
| Reserved |    |    | DITH<br>EN | UIFRE<br>MAP | Res | CKD[1:0] |    | ARPE | CMS[1:0] |    | DIR | OPM | URS | UDIS | CEN |
|          |    |    | rw         | rw           |     | rw       | rw | rw   | rw       | rw | rw  | rw  | rw  | rw   | rw  |

Bit 15:13 保留，必须保持复位值。

Bit 12 **DITHEN**：使能抖动（Dithering enable）

0：抖动禁止

1：抖动使能

注：DITHEN 位只能在 CEN 位复位时修改

Bit 11 **UIFREMAP**：UIF 状态位重映射（UIF status bit remapping）

0：无重映射。UIF 状态位未复制到 TIMx\_CNT 寄存器位 31

1：重映射使能。UIF 状态位复制到 TIMx\_CNT 寄存器位 31

Bit 10 保留，必须保持复位值。

Bit 9:8 **CKD[1:0]**：时钟分频（Clock division）

此位域指示定时器时钟（tim\_ker\_ck）频率以及数字滤波器（tim\_etr\_in, tim\_tix）所使用的采样时钟（t<sub>DTS</sub>）之间的分频比

00：t<sub>DTS</sub> = t<sub>tim\_ker\_ck</sub>

01：t<sub>DTS</sub> = 2 × t<sub>tim\_ker\_ck</sub>

10：t<sub>DTS</sub> = 4 × t<sub>tim\_ker\_ck</sub>

11：保留，不要设置成此值

Bit 7 **ARPE**：自动重载预装载使能（Auto-reload preload enable）

0：TIMx\_ARR 寄存器不进行缓冲；

1：TIMx\_ARR 寄存器进行缓冲。

Bit 6:5 **CMS[1:0]**：中心对齐模式选择（Center-aligned mode selection）

00：边沿对齐模式。计数器根据方向位（DIR）递增计数或递减计数。

01：中心对齐模式 1。计数器交替进行递增计数和递减计数。仅当计数器递减计数时，配置为输出的通道（TIMx\_CCMRx 寄存器中的 CxS=00）的输出比较中断标志才置 1。

10：中心对齐模式 2。计数器交替进行递增计数和递减计数。仅当计数器递增计数时，配置为输出的通道（TIMx\_CCMRx 寄存器中的 CxS=00）的输出比较中断标志才置 1。

11：中心对齐模式 3。计数器交替进行递增计数和递减计数。当计数器递增计数或递减计数时，配置为输出的通道（TIMx\_CCMRx 寄存器中的 CxS=00）的输出比较中断标志都会置 1。

注：只要计数器处于使能状态（CEN=1），就不得从边沿对齐模式切换为中心对齐模式。

Bit 4 **DIR**: 方向 (Direction)

- 0: 计数器递增计数;
- 1: 计数器递减计数。

注: 当定时器配置为中心对齐模式或编码器模式时, 该位为只读状态。

Bit 3 **OPM**: 单脉冲模式 (One pulse mode)

- 0: 计数器在发生更新事件时不会停止计数;
- 1: 计数器在发生下一更新事件时停止计数 (将 CEN 位清零)

Bit 2 **URS**: 更新请求源 (Update request source)

此位由软件置 1 和清零, 用以选择 UEV 事件源。

- 0: 使能时, 所有以下事件都会生成更新中断。此类事件包括:
  - 计数器上溢/下溢
  - 将 UG 位置 1
  - 通过从模式控制器生成的更新事件
- 1: 使能时, 只有计数器上溢/下溢会生成更新中断。

Bit 1 **UDIS**: 更新禁止 (Update disable)

此位由软件置 1 和清零, 用以使能/禁止 UEV 事件生成。

- 0: 使能 UEV。更新 (UEV) 事件可通过以下事件之一生成:
  - 计数器上出/下溢
  - 将 UG 位置 1
  - 通过从模式控制器生成的更新事件

然后更新影子寄存器的值。

- 1: 禁止 UEV。不会生成更新事件, 各影子寄存器的值 (ARR、PSC 和 CCRx) 保持不变。但如果将 UG 位置 1, 或者从从模式控制器接收到硬件复位, 则会重新初始化计数器和预分频器。

Bit 0 **CEN**: 计数器使能 (Counter enable)

- 0: 禁止计数器;
- 1: 使能计数器。

注: 只有事先通过软件将 CEN 位置 1, 才可以使用外部时钟、门控模式和编码器模式。而触发模式可通过硬件自动将 CEN 位置 1。

## 24.7.2 TIMx 控制寄存器 2 (TIMx\_CR2) (x=2~5)

偏移地址: 0x004

复位值: 0x0000 0000

|          |    |    |    |    |    |        |          |          |    |    |      |          |    |    |    |
|----------|----|----|----|----|----|--------|----------|----------|----|----|------|----------|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25     | 24       | 23       | 22 | 21 | 20   | 19       | 18 | 17 | 16 |
| Reserved |    |    |    |    |    | MMS[3] | Reserved |          |    |    |      |          |    |    |    |
|          |    |    |    |    |    | rw     |          |          |    |    |      |          |    |    |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9      | 8        | 7        | 6  | 5  | 4    | 3        | 2  | 1  | 0  |
| Reserved |    |    |    |    |    |        | TI1S     | MMS[2:0] |    |    | CCDS | Reserved |    |    |    |
|          |    |    |    |    |    |        | rw       | rw       | rw | rw | rw   |          |    |    |    |

Bit 31:26 保留，必须保持复位值。

Bit 24:8 保留，必须保持复位值。

Bit 7 **TI1S**: tim\_ti1 选择 (tim\_ti1 selection)

0: tim\_ti1\_in[15:0]引脚连接到 tim\_ti1 输入

1: tim\_ti1\_in[15:0], tim\_ti2\_in[15:0]和 tim\_ti3\_in[15:0]多路复用器输出被异或并连接至 tim\_ti1 输入

Bit 25, 6:4 **MMS[3:0]**: 主模式选择 (Master mode selection)

这些位可选择主模式下将要发送到从定时器以实现同步的信息(tim\_trgo)。这些位的组合如下:

0000: **复位** - TIMx\_EGR 寄存器中的 UG 位用作触发输出 (tim\_trgo)。如果复位由触发输入生成 (从模式控制器配置为复位模式), 则 TRGO 上的信号相比实际复位会有延迟。

0001: **使能** - 计数器使能信号 CNT\_EN 用作触发输出 (tim\_trgo)。该触发输出可用于同时启动多个定时器, 或者控制在一段时间内使能从定时器。计数器使能信号可由 CEN 控制位产生。当配置为门控模式时, 也可由触发输入产生。当计数器使能信号由触发输入控制时, tim\_trgo 上会存在延迟, 选择主/从模式时除外 (请参见 TIMx\_SMCR 寄存器中 MSM 位的说明)。

0010: **更新** - 选择更新事件作为触发输出 (tim\_trgo)。例如, 主定时器可用作从定时器的预分频器。

0011: **比较脉冲** - 一旦发生输入捕获或比较匹配事件, 当 CC1IF 被置 1 时 (即使已为高电平), 触发输出都会发送一个正脉冲。(tim\_trgo)。

0100: **比较** - OC1REF 信号用作触发输出 (tim\_trgo)

0101: **比较** - OC2REF 信号用作触发输出 (tim\_trgo)

0110: **比较** - OC3REF 信号用作触发输出 (tim\_trgo)

0111: **比较** - OC4REF 信号用作触发输出 (tim\_trgo)

0111: **比较** - OC4REF 信号用作触发输出 (tim\_trgo)

1000: **编码器时钟输出** - 编码器时钟信号用作出发输出 (tim\_trgo)。此编码对以下 SMS[3:0]值有效: 0001, 0010, 0011, 1010, 1011, 1100, 1101, 1110, 1111。任何其他 SMS[3:0]编码都是不允许的, 可能会导致意外行为。

其他编码保留

注: 在从主定时器接收事件之前, 必须使能从机定时器或 ADC 的时钟, 并且在从主定时器接收触发器时不能随意更改。

Bit 3 **CCDS**: 捕获/比较 DMA 选择 (Capture/compare DMA selection)

0: 发生 CCx 事件时发送 CCx DMA 请求

1: 发生更新事件时发送 CCx DMA 请求

Bit 2:0 保留, 必须保持复位值。

### 24.7.3 TIMx 从模式控制寄存器 (TIMx\_SMCR) (x=2~5)

偏移地址: 0x008

复位值: 0x0000 0000

|          |     |           |    |          |    |       |       |          |         |         |    |          |          |    |        |
|----------|-----|-----------|----|----------|----|-------|-------|----------|---------|---------|----|----------|----------|----|--------|
| 31       | 30  | 29        | 28 | 27       | 26 | 25    | 24    | 23       | 22      | 21      | 20 | 19       | 18       | 17 | 16     |
| Reserved |     |           |    |          |    | SMSPS | SMSPE | Reserved |         | TS[4:3] |    | Reserved |          |    | SMS[3] |
|          |     |           |    |          |    | rw    | rw    |          |         | rw      | rw |          |          |    | rw     |
| 15       | 14  | 13        | 12 | 11       | 10 | 9     | 8     | 7        | 6       | 5       | 4  | 3        | 2        | 1  | 0      |
| ETP      | ECE | ETPS[1:0] |    | ETF[3:0] |    |       |       | MSM      | TS[2:0] |         |    | OCCS     | SMS[2:0] |    |        |
| rw       | rw  | rw        | rw | rw       | rw | rw    | rw    | rw       | rw      | rw      | rw | rw       | rw       | rw | rw     |

Bit 31:26 保留, 必须保持复位值。

Bit 25 **SMSPS**: SMS 预加载源 (SMS preload source)

此位选择触发 SMS[3:0]位域从预加载到活动的事件来源:

0: 由定时器的更新事件触发

1: 由索引事件触发

Bit 24 **SMSPE**: SMS 预加载使能 (SMS preload enable)

此位选择是否使能 SMS[3:0]位域的预加载:

0: SMS[3:0]位域不进行预加载

1: 使能 SMS[3:0]预加载

Bit 23:22 保留, 必须保持复位值。

Bit 19:17 保留, 必须保持复位值。

Bit 15 **ETP**: 外部触发极性 (External trigger polarity)

此位可选择将 tim\_etr\_in 还是 tim\_etr\_in 的反相用于触发操作

0: tim\_etr\_in 未反相, 高电平或上升沿有效。

1: tim\_etr\_in 反相, 低电平或下降沿有效。

Bit 14 **ECE**: 外部时钟使能位 (External clock enable)

此位可使能外部时钟模式 2。

0: 禁止外部时钟模式 2

1: 使能外部时钟模式 2。计数器时钟由 tim\_etr\_in 信号的任意有效边沿提供。

注: 将 ECE 位置 1 与选择外部时钟模式 1 并将 tim\_trgi 连接到 tim\_etr\_in (SMS=111 且 TS=00111) 具有相同效果。

外部时钟模式 2 可以和以下从模式同时使用: 复位模式、门控模式和触发模式。不过此类情况下 tim\_trgi 不得连接 tim\_etr\_in (TS 位不得为 00111)。

如果同时使能外部时钟模式 1 和外部时钟模式 2, 则外部时钟输入为 tim\_etr\_in。

Bit 13:12 **ETPS[1:0]**: 外部触发预分频器 (External trigger prescaler)

外部触发信号 tim\_etr\_in 频率不得超过 TIMxCLK 频率的 1/4。可通过使能预分频器来降低 tim\_etr\_in 频率。这种方法在 tim\_etr\_in 输入快速外部时钟时非常有用。

00: 预分频器关闭

01: 2 分频 tim\_etr\_in 频率

10: 4 分频 tim\_etr\_in 频率

11: 8 分频 tim\_etr\_in 频率



Bit 11:8 **ETF[3:0]**: 外部触发滤波器 (External trigger filter)

此位域可定义 `tim_etrp` 信号的采样频率和适用于 `tim_etrp` 的数字滤波时间。数字滤波器由事件计数器组成, 每 `N` 个事件才视为一个有效边沿:

0000: 无滤波器, 按  $f_{DTS}$  频率进行采样

0001:  $f_{SAMPLING}=f_{CK\_INT}$ ,  $N=2$ 。

0010:  $f_{SAMPLING}=f_{CK\_INT}$ ,  $N=4$ 。

0011:  $f_{SAMPLING}=f_{CK\_INT}$ ,  $N=8$ 。

0100:  $f_{SAMPLING}=f_{DTS}/2$ ,  $N=6$ 。

0101:  $f_{SAMPLING}=f_{DTS}/2$ ,  $N=8$ 。

0110:  $f_{SAMPLING}=f_{DTS}/4$ ,  $N=6$ 。

0111:  $f_{SAMPLING}=f_{DTS}/4$ ,  $N=8$ 。

1000:  $f_{SAMPLING}=f_{DTS}/8$ ,  $N=6$ 。

1001:  $f_{SAMPLING}=f_{DTS}/8$ ,  $N=8$ 。

1010:  $f_{SAMPLING}=f_{DTS}/16$ ,  $N=5$ 。

1011:  $f_{SAMPLING}=f_{DTS}/16$ ,  $N=6$ 。

1100:  $f_{SAMPLING}=f_{DTS}/16$ ,  $N=8$ 。

1101:  $f_{SAMPLING}=f_{DTS}/32$ ,  $N=5$ 。

1110:  $f_{SAMPLING}=f_{DTS}/32$ ,  $N=6$ 。

1111:  $f_{SAMPLING}=f_{DTS}/32$ ,  $N=8$ 。

Bit 7 **MSM**: 主/从模式 (Master/slave mode)

0: 不执行任何操作;

1: 当前定时器的触发输入事件(`tim_trgi`)的动作被推迟, 以使当前定时器与其从定时器实现完美同步 (通过 `tim_trgo`)。此设置适用于由单个外部事件对多个定时器进行同步的情况。



**Bit 21,20,6,5,4 TS[4:0]: 触发选择 (Trigger selection)**

此位域可选择将要用于同步计数器的触发输入。

00000: 内部触发 0 (tim\_itr0)

00001: 内部触发 1 (tim\_itr1)

00010: 内部触发 2 (tim\_itr2)

00011: 内部触发 3 (tim\_itr3)

00100: tim\_ti1 边沿检测器 (tim\_ti1f\_ed)

00101: 滤波后的定时器输入 1 (tim\_ti1fp1)

00110: 滤波后的定时器输入 2 (tim\_ti1fp2)

00111: 外部触发输入 (tim\_etr1)

01000: 内部触发器 4 (tim\_itr4)

01001: 内部触发器 5 (tim\_itr5)

01010: 内部触发器 6 (tim\_itr6)

01011: 内部触发器 7 (tim\_itr7)

01100: 内部触发器 8 (tim\_itr8)

01101: 内部触发器 9 (tim\_itr9)

01110: 内部触发器 10 (tim\_itr10)

01111: 内部触发器 11 (tim\_itr11)

10000: 内部触发器 12 (tim\_itr12)

10001: 内部触发器 13 (tim\_itr13)

10010: 内部触发器 14 (tim\_itr14)

10011: 内部触发器 15 (tim\_itr15)

其他: 保留

有关定时器 ITRx 含义的详细信息, 请参见章节: TIM2/TIM3/TIM4/TIM5 引脚和内部信号

*注: 这些位只能在未使用的情况下 (例如, SMS=000 时) 进行更改, 以避免转换时出现错误的边沿检测。*

**Bit 3 OCCS: OCREF 清除源选择 (OCREF clear selection)**

此位用于选择 OCREF 清除源。

0: tim\_ocref\_clr\_int 信号连接到 tim\_ocref\_clr 输入

1: tim\_ocref\_clr\_int 信号连接到 tim\_etr1

*注: 如果不支持 OCREF 清除选择功能, 则此位被保留并由硬件强制设置为 “0”。参考章节: TIM2/TIM3/TIM4/TIM5 实现。*

Bit 16, 2,1,0 **SMS[3:0]**: 从模式选择 (Slave mode selection)

当选择了外部信号, 触发信号 (tim\_trgi) 的有效边沿与选中的外部输入极性相关 (见输入控制寄存器和控制寄存器的说明)

0000: **关闭从模式** - 如果 CEN=1, 则预分频器直接由内部时钟驱动。

0001: **编码器模式 1** - 计数器根据 tim\_ti1fp1 电平在 tim\_ti2fp2 边沿递增/递减计数。

0010: **编码器模式 2** - 计数器根据 tim\_ti2fp2 电平在 tim\_ti1fp1 边沿递增/递减计数。

0011: **编码器模式 3** - 计数器在 tim\_ti1fp1 和 tim\_ti2fp2 的边沿计数, 计数的方向取决于另外一个信号的电平。

0100: **复位模式** - 选中的触发输入 (tim\_trgi) 的上升沿重新初始化计数器, 并且产生一个更新寄存器的信号。

0101: **门控模式** - 当触发输入 (tim\_trgi) 为高时, 计数器的时钟开启。一旦触发输入变为低, 则计数器停止 (但不复位)。计数器的启动和停止都是受控的。

0110: **触发模式** - 计数器在触发输入 tim\_trgi 的上升沿启动 (但不复位), 只有计数器的启动是受控的。

0111: **外部时钟模式 1** - 选中的触发输入 (tim\_trgi) 的上升沿驱动计数器。

1000: **组合重置+触发器模式** - 选定触发器输入 (tim\_trgi) 的上升沿会重新初始化计数器, 生成寄存器的更新并启动计数器。

1001: **组合门控+重置模式** - 当触发器输入 (tim\_trgi) 为高电平时计数器时钟使能。一旦触发器变为低电平计数器将停止并重置。同时控制计数器的启动和停止。

1010: **编码器模式**: 时钟加方向, x2 模式。

1011: **编码器模式**: 时钟加方向, x1 模式, CC2P 设置 tim\_ti2fp2 边沿敏感度。

1100: **编码器模式**: 方向时钟, x2 模式。

1101: **编码器模式**: 方向时钟, x1 模式, CC1P 和 CC2P 设置 tim\_ti1fp1 和 tim\_ti2fp2 边沿敏感度。

1110: **四象限编码器模式**: x1 模式, 仅计算 tim\_ti1fp1 边沿, CC1P 设置边沿敏感度。

1111: **四象限编码器模式**: x1 模式, 仅计算 tim\_ti2fp2 边沿, CC2P 设置边沿敏感度。

注: 如果选择 tim\_ti1f\_ed 作为触发输入 (TS=00100), 则不得使用门控模式。实际上, tim\_ti1f\_ed 为 TI1F 上的每个过渡输出 1 个脉冲, 而门控模式检查触发信号的电平。

注: 接收 tim\_trgo 或 tim\_trgo2 信号的从机外设 (定时器、ADC 等) 的时钟必须在从主定时器接收事件之前使能, 并且时钟频率 (预分频器) 在从主定时器接收触发器时不能随意更改。

## 24.7.4 TIMx DMA/中断使能寄存器 (TIMx\_DIER) (x=2~5)

偏移地址: 0x00C

复位值: 0x0000 0000

|          |     |     |       |       |       |       |     |        |        |       |       |          |       |       |     |
|----------|-----|-----|-------|-------|-------|-------|-----|--------|--------|-------|-------|----------|-------|-------|-----|
| 31       | 30  | 29  | 28    | 27    | 26    | 25    | 24  | 23     | 22     | 21    | 20    | 19       | 18    | 17    | 16  |
| Reserved |     |     |       |       |       |       |     | TERRIE | IERRIE | DIRIE | IDXIE | Reserved |       |       |     |
|          |     |     |       |       |       |       |     | rw     | rw     | rw    | rw    |          |       |       |     |
| 15       | 14  | 13  | 12    | 11    | 10    | 9     | 8   | 7      | 6      | 5     | 4     | 3        | 2     | 1     | 0   |
| Res      | TDE | Res | CC4DE | CC3DE | CC2DE | CC1DE | UDE | Res    | TIE    | Res   | CC4IE | CC3IE    | CC2IE | CC1IE | UIE |
|          | rw  |     | rw    | rw    | rw    | rw    | rw  |        | rw     |       | rw    | rw       | rw    | rw    | rw  |

Bit 31:24 保留, 必须保持复位值。

Bit 23 **TERRIE**: 转换错误中断使能 (Transition error interrupt enable)

0: 禁止转换错误中断;

1: 使能转换错误中断。

Bit 22 **IERRIE**: 索引错误中断使能 (Index error interrupt enable)

0: 禁止索引错误中断;

1: 使能索引错误中断。

Bit 21 **TERRIE**: 方向改变使能 (Direction change interrupt enable)

0: 禁止方向改变中断;

1: 使能方向改变中断。

Bit 20 **IDXIE**: 索引中断使能 (Index interrupt enable)

0: 禁止索引中断;

1: 使能索引中断。

Bit 19:15 保留, 必须保持复位值。

Bit 14 **TDE**: 触发 DMA 请求使能 (Trigger DMA request enable)

0: 禁止触发 DMA 请求;

1: 使能触发 DMA 请求。

Bit 13 保留, 必须保持复位值。

Bit 12 **CC4DE**: 捕获/比较 4 DMA 请求使能 (Capture/Compare 4 DMA request enable)

0: 禁止 CC4 DMA 请求;

1: 使能 CC4 DMA 请求。

Bit 11 **CC3DE**: 捕获/比较 3 DMA 请求使能 (Capture/Compare 3 DMA request enable)

0: 禁止 CC3 DMA 请求;

1: 使能 CC3 DMA 请求。

Bit 10 **CC2DE**: 捕获/比较 2 DMA 请求使能 (Capture/Compare 2 DMA request enable)

0: 禁止 CC2 DMA 请求;

1: 使能 CC2 DMA 请求。

Bit 9 **CC1DE**: 捕获/比较 1 DMA 请求使能 (Capture/Compare 1 DMA request enable)

0: 禁止 CC1 DMA 请求;

1: 使能 CC1 DMA 请求。

Bit 8 **UDE**: 更新 DMA 请求使能 (Capture/Compare 1 DMA request enable)

0: 禁止更新 DMA 请求;

1: 使能更新 DMA 请求。

Bit 7 保留，必须保持复位值。

Bit 6 **TIE**：触发信号 (TGRI) 中断使能 (Trigger interrupt enable)

0：禁止触发信号 (TGRI) 中断；

1：使能触发信号 (TGRI) 中断。

Bit 5 保留，必须保持复位值。

Bit 4 **CC4IE**：捕获/比较 4 中断使能 (Capture/Compare 4 interrupt enable)

0：禁止 CC4 中断；

1：使能 CC4 中断。

Bit 3 **CC3IE**：捕获/比较 3 中断使能 (Capture/Compare 3 interrupt enable)

0：禁止 CC3 中断；

1：使能 CC3 中断。

Bit 2 **CC2IE**：捕获/比较 2 中断使能 (Capture/Compare 2 interrupt enable)

0：禁止 CC2 中断；

1：使能 CC2 中断。

Bit 1 **CC1IE**：捕获/比较 1 中断使能 (Capture/Compare 1 interrupt enable)

0：禁止 CC1 中断；

1：使能 CC1 中断。

Bit 0 **UIE**：更新中断使能 (Update interrupt enable)

0：禁止更新中断；

1：使能更新中断。

## 24.7.5 TIMx 状态寄存器 (TIMx\_SR) (x=2~5)

偏移地址：0x010

复位值：0x0000 0000

|          |    |    |       |       |       |       |          |       |       |       |       |          |       |       |     |
|----------|----|----|-------|-------|-------|-------|----------|-------|-------|-------|-------|----------|-------|-------|-----|
| 31       | 30 | 29 | 28    | 27    | 26    | 25    | 24       | 23    | 22    | 21    | 20    | 19       | 18    | 17    | 16  |
| Reserved |    |    |       |       |       |       |          | TERRF | IERRF | DIRF  | IDXF  | Reserved |       |       |     |
|          |    |    |       |       |       |       |          | rc_w0 | rc_w0 | rc_w0 | rc_w0 |          |       |       |     |
| 15       | 14 | 13 | 12    | 11    | 10    | 9     | 8        | 7     | 6     | 5     | 4     | 3        | 2     | 1     | 0   |
| Reserved |    |    | CC4OF | CC3OF | CC2OF | CC1OF | Reserved |       | TIF   | Res   | CC4IF | CC3IF    | CC2IF | CC1IF | UIF |
|          |    |    | rc_w0 | rc_w0 | rc_w0 | rc_w0 |          |       | rc_w0 |       | rc_w0 | rc_w0    | rc_w0 | rc_w0 |     |

Bit 31:24 保留，必须保持复位值。

Bit 23 **TERRF**：转换错误中断标志 (Transition error interrupt flag)

当在编码器模式下检测到转换错误时，硬件会设置此标志当软件将其写入“0”时，该标志将被清除

0：未检测到编码器转换错误

1：已检测到编码器转换错误

Bit 22 **IERRF**：索引错误中断标志 (Index error interrupt flag)

当检测到索引错误时，硬件会设置此标志当软件将其写入“0”时，该标志将被清除

0：未检测到索引错误

1：已检测到索引错误

Bit 21 **DIRF**: 方向改变中断标志 (Direction change interrupt flag)

当编码器模式中的方向发生变化时 (TIMx\_CR 中的 DIR 位值正在改变), 硬件会设置此标志当软件将其写入 “0” 时, 该标志将被清除

0: 方向未改变

1: 方向已改变

Bit 20 **IDXF**: 索引中断标志 (Index interrupt flag)

当检测到索引事件时, 硬件会设置此标志当软件将其写入 “0” 时, 该标志将被清除

0: 未发生索引事件

1: 已发生索引事件

Bit 19:13 保留, 必须保持复位值。

Bit 12 **CC4OF**: 捕获/比较 4 重复捕获标志

参考 CC1OF 的描述

Bit 11 **CC3OF**: 捕获/比较 3 重复捕获标志

参考 CC1OF 的描述

Bit 10 **CC2OF**: 捕获/比较 2 重复捕获标志

参考 CC1OF 的描述

Bit 9 **CC1OF**: 捕获/比较 1 重复捕获标志

仅当将相应通道配置为输入捕获模式时, 此标志位才会由硬件置 1。通过软件写入 “0” 可将该位清零。

0: 未检测到重复捕获

1: TIMx\_CCR1 寄存器中已捕获到计数器值且 CC1IF 标志已置 1。

Bit 8:7 保留, 必须保持复位值。

Bit 6 **TIF**: 触发中断标志 (Trigger interrupt flag)

在除门控模式以外的所有模式下, 当使能从模式控制器后在 tim\_trgi 输入上检测到有效边沿时, 该标志将由硬件置 1。选择门控模式时, 该标志将在计数器启动或停止时置 1。但需要通过软件清零。

0: 未发生触发事件。

1: 触发中断挂起。

Bit 5 保留, 必须保持复位值。

Bit 4 **CC4IF**: 捕获/比较 4 中断标志 (Capture/Compare 4 interrupt flag)

参考 CC1IF 描述。

Bit 3 **CC3IF**: 捕获/比较 3 中断标志 (Capture/Compare 3 interrupt flag)

参考 CC1IF 描述。

Bit 2 **CC2IF**: 捕获/比较 2 中断标志 (Capture/Compare 2 interrupt flag)

参考 CC1IF 描述。

Bit 1 **CC1IF**: 捕获/比较 1 中断标志 (Capture/Compare 1 interrupt flag)

此标志由硬件设置，它可以通过软件（输入捕获或输出比较模式）或读取 TIMx\_CCR1 寄存器（仅输入捕获模式）来清除。

0: 无比较匹配/无输入捕获发生

1: 发生比较匹配或输入捕获

**如果通道 CC1 配置为输出:** 当计数器 TIMx\_CNT 的内容与 TIMx\_CCR1 寄存器内容匹配时，会设置此标志。当 TIMx\_CCR1 的值大于 TIMx\_ARR 的值时，CC1IF 位将在计数器发生上溢（递增计数模式和增减计数模式下）或下溢（递减计数模式下）时变为高电平。在中心对齐模式下有 3 种可能的标志设置选项，请参考 TIMx\_CR1 寄存器中的 CMS 位以获取完整描述。

**如果通道 CC1 配置为输入:** 当在 TIMx\_CCR1 寄存器中捕获到计数器值时（在 TIMx\_CCER 中定义的 CC1P 和 CC1NP 位设置所确定的 IC1 上检测到边沿时），会设置此位。

Bit 0 **UIF**: 更新中断标志 (Update interrupt flag)

该位在发生更新事件时通过硬件置 1。但需要通过软件清零。

0: 未发生更新。

1: 更新中断挂起。该位在以下情况下更新寄存器时由硬件置 1:

- TIMx\_CR1 寄存器中的 UDIS=0，并且在上溢或下溢时。
- TIMx\_CR1 寄存器中的 URS=0 且 UDIS=0，并且由软件使用 TIMx\_EGR 寄存器中的 UG 位重新初始化 CNT 时。
- TIMx\_CR1 寄存器中的 URS=0 且 UDIS=0，并且 CNT 由触发事件重新初始化时（参考同步控制寄存器描述）。

#### 24.7.6 TIMx 事件产生寄存器 (TIMx\_EGR) (x=2~5)

偏移地址: 0x14

复位: 0x0000

| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6  | 5   | 4    | 3    | 2    | 1    | 0  |
|----------|----|----|----|----|----|---|---|---|----|-----|------|------|------|------|----|
| Reserved |    |    |    |    |    |   |   |   | TG | Res | CC4G | CC3G | CC2G | CC1G | UG |
|          |    |    |    |    |    |   |   |   | w  |     | w    | w    | w    | w    | w  |

Bit 15:7 保留，必须保持复位值。

Bit 6 **TG**: 生成触发信号 (Trigger generation)

此位由软件置 1 以生成事件，并由硬件自动清零。

0: 不执行任何操作;

1: TIMx\_SR 寄存器中的 TIF 标志置 1。使能后可发生相关中断事件。

Bit 5 保留，必须保持复位值。

Bit 4 **CC4G**: 捕获/比较 4 生成 (Capture/Compare 4 generation)

参考 CC1G 描述。

Bit 3 **CC3G**: 捕获/比较 3 生成 (Capture/Compare 3 generation)

参考 CC1G 描述。



Bit 2 **CC2G**: 捕获/比较 2 生成 (Capture/Compare 2 generation)

参考 CC1G 描述。

Bit 1 **CC1G**: 捕获/比较 1 生成 (Capture/Compare 1 generation)

此位由软件置 1 以生成事件，并由硬件自动清零。

0: 不执行任何操作；

1: 通道 1 上生成捕获/比较事件：

**若通道 CC1 配置为输出：**

使能时，CC1IF 标志置 1 并发送相应的中断。

**如果通道 CC1 配置为输入：**

TIMx\_CCR1 寄存器中将捕获到计数器当前值。使能时，CC1IF 标志置 1 并发送相应的中断或 DMA 请求。如果 CC1IF 标志已为高电平，CC1OF 标志将置 1。

Bit 0 **UG**: 更新生成 (Update generation)

该位可通过软件置 1，并由硬件自动清零。

0: 不执行任何操作；

1: 重新初始化计数器并生成一个寄存器更新事件。请注意，预分频器计数器也将清零（但预分频比不受影响）。如果选择中心对齐模式或 DIR=0（递增计数），计数器将清零；如果 DIR=1（递减计数），计数器将使用自动重载值 (TIMx\_ARR)。

#### 24.7.7 TIMx 捕获/比较模式寄存器 1 [复用] (TIMx\_CCMR1) (x=2~5)

偏移地址: 0x018

复位值: 0x0000 0000

相同的寄存器可以用于输入捕获模式（本节）或输出比较模式（下节）。通道方向通过配置相应的 CCxS 位进行定义。此寄存器的所有其它位在输入模式和输出模式下的功能均不同。可以独立组合这两种模式（例如，通道 1 用作输入捕获模式而通道 2 用作输出比较模式）

|           |    |    |    |             |    |           |    |           |    |    |    |             |    |           |    |
|-----------|----|----|----|-------------|----|-----------|----|-----------|----|----|----|-------------|----|-----------|----|
| 31        | 30 | 29 | 28 | 27          | 26 | 25        | 24 | 23        | 22 | 21 | 20 | 19          | 18 | 17        | 16 |
| Reserved  |    |    |    |             |    |           |    |           |    |    |    |             |    |           |    |
| 15        | 14 | 13 | 12 | 11          | 10 | 9         | 8  | 7         | 6  | 5  | 4  | 3           | 2  | 1         | 0  |
| IC2F[3:0] |    |    |    | IC2PSC[1:0] |    | CC2S[1:0] |    | IC1F[3:0] |    |    |    | IC1PSC[1:0] |    | CC1S[1:0] |    |
| rw        | rw | rw | rw | rw          | rw | rw        | rw | rw        | rw | rw | rw | rw          | rw | rw        | rw |

##### 输入捕获模式

Bit 31:16 保留，始终读为 0。

Bit 15:12 **IC2F[3:0]**: 输入捕获 2 滤波器 (Input capture 2 filter)

Bit 11:10 **IC2PSC[1:0]**: 输入/捕获 2 预分频器 (input capture 2 prescaler)



**Bit 9:8 CC2S[1:0]: 捕获/比较 2 选择 (Capture/compare 2 selection)**

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00: CC2 通道被配置为输出;

01: CC2 通道被配置为输入, tim\_ic2 映射在 tim\_ti2 上;

10: CC2 通道被配置为输入, tim\_ic2 映射在 tim\_ti1 上;

11: CC2 通道被配置为输入, tim\_ic2 映射在 tim\_trc 上。此模式仅在通过 TS 位 (TIMx\_SMCR 寄存器) 选择内部触发输入时有效

注: 仅当通道关闭时 (TIMx\_CCER 中的 CC2E = 0), 才可向 CC2S 位写入数据。

**Bit 7:4 IC1F[3:0]: 输入捕获 1 滤波器 (Input capture 1 filter)**

此位域可定义 tim\_ti1 输入的采样频率和适用于 tim\_ti1 的数字滤波器带宽。数字滤波器由事件计数器组成, 每 N 个事件才视为一个有效边沿:

0000: 无滤波器, 按 f<sub>DTS</sub> 频率进行采样

0001: f<sub>SAMPLING</sub> = f<sub>tim\_ker\_ck</sub>, N=2。

0010: f<sub>SAMPLING</sub> = f<sub>tim\_ker\_ck</sub>, N=4。

0011: f<sub>SAMPLING</sub> = f<sub>tim\_ker\_ck</sub>, N=8。

0100: f<sub>SAMPLING</sub> = f<sub>DTS</sub>/2, N=6。

0101: f<sub>SAMPLING</sub> = f<sub>DTS</sub>/2, N=8。

0110: f<sub>SAMPLING</sub> = f<sub>DTS</sub>/4, N=6。

0111: f<sub>SAMPLING</sub> = f<sub>DTS</sub>/4, N=8。

1000: f<sub>SAMPLING</sub> = f<sub>DTS</sub>/8, N=6。

1001: f<sub>SAMPLING</sub> = f<sub>DTS</sub>/8, N=8。

1010: f<sub>SAMPLING</sub> = f<sub>DTS</sub>/16, N=5。

1011: f<sub>SAMPLING</sub> = f<sub>DTS</sub>/16, N=6。

1100: f<sub>SAMPLING</sub> = f<sub>DTS</sub>/16, N=8。

1101: f<sub>SAMPLING</sub> = f<sub>DTS</sub>/32, N=5。

1110: f<sub>SAMPLING</sub> = f<sub>DTS</sub>/32, N=6。

1111: f<sub>SAMPLING</sub> = f<sub>DTS</sub>/32, N=8。

**Bit 3:2 IC1PSC[1:0]: 输入/捕获 1 预分频器 (Input capture 1 prescaler)**

此位域定义 CC1 输入 (tim\_ic1) 的预分频比。

只要 CC1E=0 (TIMx\_CCER 寄存器), 预分频器便立即复位。

00: 无预分频器, 捕获输入上每检测到一个边沿便执行捕获

01: 每发生 2 个事件便执行一次捕获;

10: 每发生 4 个事件便执行一次捕获;

11: 每发生 8 个事件便执行一次捕获。

**Bit 1:0 CC1S[1:0]: 捕获/比较 1 选择 (Capture/Compare 1 selection)**

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00: CC1 通道被配置为输出;

01: CC1 通道被配置为输入, tim\_ic1 映射在 tim\_ti1 上;

10: CC1 通道被配置为输入, tim\_ic1 映射在 tim\_ti2 上;

11: CC1 通道被配置为输入, tim\_ic1 映射在 tim\_trc 上。此模式仅在通过 TS 位 (TIMx\_SMCR 寄存器) 选择内部触发输入时有效

注: 仅当通道关闭时 (TIMx\_CCER 中的 CC1E = 0), 才可向 CC1S 位写入数据。

## 24.7.8 TIMx 捕获/比较模式寄存器 1[复用] (TIMx\_CCMR1) (x=2~5)

偏移地址: 0x018

复位值: 0x0000 0000

相同的寄存器可以用于输出比较模式 (本节) 或输入捕获模式 (上节)。通道方向通过配置相应的 CCxS 位进行定义。此寄存器的所有其它位在输入模式和输出模式下的功能均不同。可以独立组合这两种模式 (例如, 通道 1 用作输入捕获模式而通道 2 用作输出比较模式)

|          |           |    |    |       |       |           |         |          |           |    |    |       |       |           |         |
|----------|-----------|----|----|-------|-------|-----------|---------|----------|-----------|----|----|-------|-------|-----------|---------|
| 31       | 30        | 29 | 28 | 27    | 26    | 25        | 24      | 23       | 22        | 21 | 20 | 19    | 18    | 17        | 16      |
| Reserved |           |    |    |       |       |           | OC2M[3] | Reserved |           |    |    |       |       |           | OC1M[3] |
|          |           |    |    |       |       |           | rw      |          |           |    |    |       |       |           | rw      |
| 15       | 14        | 13 | 12 | 11    | 10    | 9         | 8       | 7        | 6         | 5  | 4  | 3     | 2     | 1         | 0       |
| OC2CE    | OC2M[2:0] |    |    | OC2PE | OC2FE | CC2S[1:0] |         | OC1CE    | OC1M[2:0] |    |    | OC1PE | OC1FE | CC1S[1:0] |         |
| rw       | rw        | rw | rw | rw    | rw    | rw        | rw      | rw       | rw        | rw | rw | rw    | rw    | rw        | rw      |

### 输出比较模式

Bit 31:25 保留, 必须保持复位值。

Bit 23:17 保留, 必须保持复位值。

Bit 15 **OC2CE**: 输出比较 2 清零使能 (Output Compare 2 clear enable)

Bit 24, 14:12 **OC2M[2:0]**: 输出比较 2 模式 (Output Compare 2 mode)

参考 OC1M 描述

Bit 11 **OC2PE**: 输出比较 2 预装载使能 (Output Compare 2 preload enable)

Bit 10 **OC2FE**: 输出比较 2 快速使能 (Output Compare 2 fast enable)

Bit 9:8 **CC2S[1:0]**: 捕获/比较 2 选择 (Capture/Compare 2 selection)

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00: CC2 通道配置为输出

01: CC2 通道配置为输入, tim\_ic2 映射到 tim\_ti2 上

10: CC2 通道配置为输入, tim\_ic2 映射到 tim\_ti1 上

11: CC2 通道配置为输入, tim\_ic2 映射到 tim\_trc 上。此模式仅在通过 TS 位 (TIMx\_SMCR 寄存器) 选择内部触发输入时有效

注: 仅当通道关闭时 (TIMx\_CCER 中的 CC2E = 0), 才可向 CC2S 位写入数据

Bit 7 **OC1CE**: 输出比较 1 清零使能 (Output Compare 1 clear enable)

0: tim\_oc1ref 不受 tim\_ocref\_clr\_int 输入的影响;

1: 一旦在 tim\_ocref\_clr\_int 输入上检测到高电平, tim\_oc1ref 立即清零。

Bit 16, 6:4 OC1M[3:0]: 输出比较 1 模式 (Output Compare 1 mode)

这些位定义提供 tim\_oc1 和 tim\_oc1n 的输出参考信号 tim\_oc1ref 的行为。tim\_oc1ref 为高电平有效, 而 tim\_oc1 和 tim\_oc1n 的有效电平则取决于 CC1P 位和 CC1NP 位。

0000: **冻结** - 输出比较寄存器 TIMx\_CCR1 与计数器 TIMx\_CNT 进行比较不会对输出造成任何影响。(该模式用于生成时基)。

0001: 将通道 1 设置为匹配时输出有效电平。当计数器 TIMx\_CNT 与捕获/比较寄存器 1 (TIMx\_CCR1) 匹配时, tim\_oc1ref 信号强制变为高电平。

0010: 将通道 1 设置为匹配时输出无效电平。当计数器 TIMx\_CNT 与捕获/比较寄存器 1 (TIMx\_CCR1) 匹配时, tim\_oc1ref 信号强制变为低电平。

0011: **翻转** - TIMx\_CNT=TIMx\_CCR1 时, tim\_oc1ref 发生翻转。

0100: **强制变为无效电平** - tim\_oc1ref 强制变为低电平。

0101: **强制变为有效电平** - tim\_oc1ref 强制变为高电平。

0110: **PWM 模式 1** - 在递增计数模式下, 只要 TIMx\_CNT < TIMx\_CCR1, 通道 1 便为有效状态, 否则为无效状态。在递减计数模式下, 只要 TIMx\_CNT > TIMx\_CCR1, 通道 1 便为无效状态 (tim\_oc1ref=0), 否则为有效状态 (tim\_oc1ref=1)。

0111: **PWM 模式 2** - 在递增计数模式下, 只要 TIMx\_CNT < TIMx\_CCR1, 通道 1 便为无效状态, 否则为有效状态。在递减计数模式下, 只要 TIMx\_CNT > TIMx\_CCR1, 通道 1 便为有效状态, 否则为无效状态。

1000: **可重触发 OPM 模式 1** - 在向上计数模式下, 通道在检测到触发事件 (tim\_trgi 信号) 之前一直处于活动状态。然后, 会像 PWM 模式 1 那样执行比较, 并在下次更新时使通道再次处于活动状态。在向下计数模式下, 通道在检测到触发事件 (tim\_trgi 信号) 之前一直处于非活动状态。然后, 会像 PWM 模式 1 那样执行比较, 并在下次更新时使通道再次处于非活动状态。

1001: **可重触发 OPM 模式 2** - 在向上计数模式下, 通道在检测到触发事件 (tim\_trgi 信号) 之前一直处于非活动状态。然后, 会像 PWM 模式 2 那样执行比较, 并在下次更新时使通道再次处于非活动状态。在向下计数模式下, 通道在检测到触发事件 (tim\_trgi 信号) 之前一直处于活动状态。然后, 会像 PWM 模式 1 那样执行比较, 并在下次更新时使通道再次处于活动状态。

1010: 保留。

1011: 保留。

1100: **组合 PWM 模式 1** - tim\_oc1ref 的行为与 PWM 模式 1 相同。tim\_oc1refc 是 tim\_oc1ref 和 tim\_oc2ref 之间的逻辑 OR。

1101: **组合 PWM 模式 2** - tim\_oc1ref 的行为与 PWM 模式 2 相同。tim\_oc1refc 是 tim\_oc1ref 和 tim\_oc2ref 之间的逻辑 AND。

1110: **非对称 PWM 模式 1** - tim\_oc1ref 的行为与 PWM 模式 1 相同。当计数器向上计数时, tim\_oc1refc 输出 tim\_oc1ref; 当计数器向下计数时, tim\_oc1refc 输出 tim\_oc2ref。

1111: **非对称 PWM 模式 2** - tim\_oc1ref 的行为与 PWM 模式 2 相同。当计数器向上计数时, tim\_oc1refc 输出 tim\_oc1ref; 当计数器向下计数时, tim\_oc1refc 输出 tim\_oc2ref。

注: 在 PWM 模式下, 仅当比较结果发生改变或输出比较模式由“冻结”模式切换到“PWM”模式时, tim\_ocref\_clr 电平才会发生改变。

Bit 3 **OC1PE**: 输出比较 1 预装载使能 (Output Compare 1 preload enable)

0: 禁止与 TIMx\_CCR1 相关的预装载寄存器。可随时向 TIMx\_CCR1 写入数据, 写入后将立即使用新值。

1: 使能与 TIMx\_CCR1 相关的预装载寄存器。可读/写访问预装载寄存器。TIMx\_CCR1 预装载值在每次生成更新事件时都会装载到活动寄存器中。

Bit 2 **OC1FE**: 输出比较 1 快速使能 (Output Compare 1 fast enable)

此位减少了触发事件和定时器输出上的转换之间的延迟。它必须在单脉冲模式 (在 TIMx\_CR1 寄存器中设置 OPM 位) 中使用, 以便在启动触发后尽快开始输出脉冲。

0: 即使触发开启, CC1 也将根据计数器和 CCR1 值正常工作。触发输入出现边沿时, 激活 CC1 输出的最短延迟时间为 5 个时钟周期。

1: 触发输入上出现有效边沿相当于 CC1 输出上的比较匹配。随后, 无论比较结果如何, OC 都设置为比较电平。采样触发输入和激活 CC1 输出的延迟时间缩短为 3 个时钟周期。仅当通道配置为 PWM1 或 PWM2 模式时, OCFE 才会起作用。

Bit 1:0 **CC1S[1:0]**: 捕获/比较 1 选择 (Capture/Compare 1 selection)

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00: CC1 通道配置为输出

01: CC1 通道配置为输入, tim\_ic1 映射到 tim\_ti1 上

10: CC1 通道配置为输入, tim\_ic1 映射到 tim\_ti2 上

11: CC1 通道配置为输入, tim\_ic1 映射到 tim\_trc 上。此模式仅在通过 TS 位 (TIMx\_SMCR 寄存器) 选择内部触发输入时有效

注: 仅当通道关闭时 (TIMx\_CCER 中的 CC1E = 0), 才可向 CC1S 位写入数据

## 24.7.9 TIMx 捕获/比较模式寄存器 2 [复用] (TIMx\_CCMR2) (x=2~5)

偏移地址: 0x01C

复位值: 0x0000 0000

相同的寄存器可以用于输入捕获模式 (本节) 或输出比较模式 (下节)。通道方向通过配置相应的 CCxS 位进行定义。此寄存器的所有其它位在输入模式和输出模式下的功能均不同。可以独立组合这两种模式 (例如, 通道 1 用作输入捕获模式而通道 2 用作输出比较模式)

|           |    |    |    |             |    |           |    |           |    |    |    |             |    |           |    |
|-----------|----|----|----|-------------|----|-----------|----|-----------|----|----|----|-------------|----|-----------|----|
| 31        | 30 | 29 | 28 | 27          | 26 | 25        | 24 | 23        | 22 | 21 | 20 | 19          | 18 | 17        | 16 |
| Reserved  |    |    |    |             |    |           |    |           |    |    |    |             |    |           |    |
| 15        | 14 | 13 | 12 | 11          | 10 | 9         | 8  | 7         | 6  | 5  | 4  | 3           | 2  | 1         | 0  |
| IC4F[3:0] |    |    |    | IC4PSC[1:0] |    | CC4S[1:0] |    | IC3F[3:0] |    |    |    | IC3PSC[1:0] |    | CC3S[1:0] |    |
| rw        | rw | rw | rw | rw          | rw | rw        | rw | rw        | rw | rw | rw | rw          | rw | rw        | rw |

### 输入捕获模式

Bit 31:16 保留, 始终读为 0。

Bit 15:12 **IC4F[3:0]**: 输入捕获 4 滤波器 (Input capture 4 filter)

Bit 11:10 **IC4PSC[1:0]**: 输入/捕获 4 预分频器 (input capture 4 prescaler)

Bit 9:8 **CC4S[1:0]**: 捕获/比较 4 选择 (Capture/compare 4 selection)

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00: CC4 通道被配置为输出;

01: CC4 通道被配置为输入, tim\_ic4 映射在 tim\_ti4 上;

10: CC4 通道被配置为输入, tim\_ic4 映射在 tim\_ti3 上;

11: CC4 通道被配置为输入, tim\_ic4 映射在 tim\_trc 上。此模式仅在通过 TS 位 (TIMx\_SMCR 寄存器) 选择内部触发输入时有效

注: 仅当通道关闭时 (TIMx\_CCER 中的 CC4E = 0), 才可向 CC4S 位写入数据。

Bit 7:4 **IC3F[3:0]**: 输入捕获 3 滤波器 (Input capture 3 filter)

Bit 3:2 **IC3PSC[1:0]**: 输入/捕获 1 预分频器 (Input capture 3 prescaler)

Bit 1:0 **CC3S[1:0]**: 捕获/比较 3 选择 (Capture/Compare 3 selection)

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00: CC3 通道被配置为输出;

01: CC3 通道被配置为输入, tim\_ic3 映射在 tim\_ti3 上;

10: CC3 通道被配置为输入, tim\_ic3 映射在 tim\_ti4 上;

11: CC3 通道被配置为输入, tim\_ic3 映射在 tim\_trc 上。此模式仅在通过 TS 位 (TIMx\_SMCR 寄存器) 选择内部触发输入时有效

注: 仅当通道关闭时 (TIMx\_CCER 中的 CC3E = 0), 才可向 CC3S 位写入数据。

## 24.7.10 TIMx 捕获/比较模式寄存器 2 [复用] (TIMx\_CCMR2) (x=2~5)

偏移地址: 0x01C

复位值: 0x0000 0000

相同的寄存器可以用于输出比较模式 (本节) 或输入捕获模式 (上节)。通道方向通过配置相应的 CCxS 位进行定义。此寄存器的所有其它位在输入模式和输出模式下的功能均不同。可以独立组合这两种模式 (例如, 通道 1 用作输入捕获模式而通道 2 用作输出比较模式)

|          |           |    |    |       |       |           |         |          |           |    |    |       |       |           |         |
|----------|-----------|----|----|-------|-------|-----------|---------|----------|-----------|----|----|-------|-------|-----------|---------|
| 31       | 30        | 29 | 28 | 27    | 26    | 25        | 24      | 23       | 22        | 21 | 20 | 19    | 18    | 17        | 16      |
| Reserved |           |    |    |       |       |           | OC4M[3] | Reserved |           |    |    |       |       |           | OC3M[3] |
|          |           |    |    |       |       |           | rw      |          |           |    |    |       |       |           | rw      |
| 15       | 14        | 13 | 12 | 11    | 10    | 9         | 8       | 7        | 6         | 5  | 4  | 3     | 2     | 1         | 0       |
| OC4CE    | OC4M[2:0] |    |    | OC4PE | OC4FE | CC4S[1:0] |         | OC3CE    | OC3M[2:0] |    |    | OC3PE | OC3FE | CC3S[1:0] |         |
| rw       | rw        | rw | rw | rw    | rw    | rw        | rw      | rw       | rw        | rw | rw | rw    | rw    | rw        | rw      |

### 输出比较模式

Bit 31:25 保留, 必须保持复位值。

Bit 23:17 保留, 必须保持复位值。

Bit 15 **OC4CE**: 输出比较 4 清零使能 (Output Compare 4 clear enable)

Bit 24, 14:12 **OC4M[2:0]**: 输出比较 4 模式 (Output Compare 4 mode)

参考 OC1M 描述 (TIMx\_CCMR1 寄存器)

Bit 11 **OC4PE**: 输出比较 4 预装载使能 (Output Compare 4 preload enable)

Bit 10 **OC4FE**: 输出比较 4 快速使能 (Output Compare 4 fast enable)



Bit 9:8 **CC4S[1:0]**: 捕获/比较 4 选择 (Capture/Compare 4 selection)

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00: CC4 通道配置为输出

01: CC4 通道配置为输入, tim\_ic4 映射到 tim\_ti4 上

10: CC4 通道配置为输入, tim\_ic4 映射到 tim\_ti3 上

11: CC4 通道配置为输入, tim\_ic4 映射到 tim\_trc 上。此模式仅在通过 TS 位 (TIMx\_SMCR 寄存器) 选择内部触发输入时有效

注: 仅当通道关闭时 (TIMx\_CCER 中的 CC4E = 0), 才可向 CC4S 位写入数据

Bit 7 **OC3CE**: 输出比较 3 清零使能 (Output Compare 3 clear enable)

Bit 16, 6:4 **OC3M[3:0]**: 输出比较 3 模式 (Output Compare 1 mode)

参考 OC1M 描述 (TIMx\_CCMR1 寄存器)

Bit 3 **OC3PE**: 输出比较 3 预装载使能 (Output Compare 3 preload enable)

Bit 2 **OC3FE**: 输出比较 3 快速使能 (Output Compare 3 fast enable)

Bit 1:0 **CC3S[1:0]**: 捕获/比较 3 选择 (Capture/Compare 3 selection)

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00: CC3 通道配置为输出

01: CC3 通道配置为输入, tim\_ic3 映射到 tim\_ti3 上

10: CC3 通道配置为输入, tim\_ic3 映射到 tim\_ti4 上

11: CC3 通道配置为输入, tim\_ic3 映射到 tim\_trc 上。此模式仅在通过 TS 位 (TIMx\_SMCR 寄存器) 选择内部触发输入时有效

注: 仅当通道关闭时 (TIMx\_CCER 中的 CC3E = 0), 才可向 CC3S 位写入数据

## 24.7.11 TIMx 捕获/比较使能寄存器 (TIMx\_CCER) (x=2~5)

偏移地址: 0x020

复位值: 0x0000

| 15    | 14  | 13   | 12   | 11    | 10  | 9    | 8    | 7     | 6   | 5    | 4    | 3     | 2   | 1    | 0    |
|-------|-----|------|------|-------|-----|------|------|-------|-----|------|------|-------|-----|------|------|
| CC4NP | Res | CC4P | CC4E | CC3NP | Res | CC3P | CC3E | CC2NP | Res | CC2P | CC2E | CC1NP | Res | CC1P | CC1E |
| rw    |     | rw   | rw   | rw    |     | rw   | rw   | rw    |     | rw   | rw   | rw    |     | rw   | rw   |

Bit 15 **CC4NP**: 输入/捕获 4 输出极性 (Capture/Compare 4 output polarity)

参考 CC1NP 的描述。

Bit 14 保留, 必须保持复位值。

Bit 13 **CC4P**: 输入/捕获 4 输出极性 (Capture/Compare 4 output polarity)

参考 CC1P 的描述。

Bit 12 **CC4E**: 输入/捕获 4 输出使能 (Capture/Compare 4 output enable)

参考 CC1E 的描述。

Bit 11 **CC3NP**: 输入/捕获 3 输出极性 (Capture/Compare 3 output polarity)

参考 CC1NP 的描述。

Bit 10 保留, 必须保持复位值。

Bit 9 **CC3P**: 输入/捕获 3 输出极性 (Capture/Compare 3 output polarity)

参考 CC1P 的描述。

Bit 8 **CC3E**: 输入/捕获 3 输出使能 (Capture/Compare 3 output enable)

参考 CC1E 的描述。

Bit 7 **CC2NP**: 输入/捕获 2 输出极性 (Capture/Compare 2 output polarity)

参考 CC1NP 的描述。

Bit 6 保留, 必须保持复位值。

Bit 5 **CC2P**: 输入/捕获 2 输出极性 (Capture/Compare 2 output polarity)

参考 CC1P 的描述。

Bit 4 **CC2E**: 输入/捕获 2 输出使能 (Capture/Compare 2 output enable)

参考 CC1E 的描述。

Bit 3 **CC1NP**: 输入/捕获 1 输出极性 (Capture/Compare 1 output polarity)

**CC1 通道配置为输出:**

在这种情况下, CC1NP 必须保持清零。

**CC1 通道配置为输入:**

此位与 CC1P 配合使用, 用以定义 tim\_ti1fp1/ tim\_ti2fp1 的极性。请参见 CC1P 说明。

Bit 2 保留, 必须保持复位值。

Bit 1 **CC1P**: 输入/捕获 1 输出极性 (Capture/Compare 1 output polarity)

0: OC1 高电平有效 (输出模式) / 边沿灵敏度选择 (输入模式, 参见以下)

1: OC1 低电平有效 (输出模式) / 边沿灵敏度选择 (输入模式, 参见以下)

**CC1 通道配置为输入**, CC1NP/CC1P 位可针对触发或捕获操作选择 TI1FP1 和 TI2FP1 的极性。

CC1NP=0, CC1P=0: 非反相/上升沿触发。电路对 TIxFP1 上升沿敏感 (在复位模式、外部时钟模式或触发模式下执行捕获或触发操作), TIxFP1 未反相 (在门控模式或编码器模式下执行触发操作)。

CC1NP=0, CC1P=1: 反相/下降沿触发。电路对 TIxFP1 下降沿敏感 (在复位模式、外部时钟模式或触发模式下执行捕获或触发操作), TIxFP1 反相 (在门控模式或编码器模式下执行触发操作)。

CC1NP=1, CC1P=1: 非反相/上升沿和下降沿均触发。电路对 TIxFP1 上升沿和下降沿都敏感 (在复位模式、外部时钟模式或触发模式下执行捕获或触发操作), TIxFP1 未反相 (在门控模式下执行触发操作)。编码器模式下不得使用此配置。

CC1NP=1, CC1P=0: 此配置是保留的, 不应该使用此配置。

Bit 0 **CC1E**: 输入/捕获 1 输出使能 (Capture/Compare 1 output enable)

0: 禁止捕获模式/OC1 未激活

1: 使能捕获模式/ OC1 信号输出到对应的输出引脚

表 24.18 标准 tim\_ocx 通道的输出控制位

| CCxE 位 | tim_ocx 输出状态                     |
|--------|----------------------------------|
| 0      | 禁止输出 (不由定时器驱动: Hi-Z)             |
| 1      | 输出使能 (tim_ocx = tim_ocxref + 极性) |

注: 当 CCxE=0 时, 连接到 tim\_ocx 通道的外部 IO 引脚状态仅取决于 GPIO 寄存器



### 24.7.12 TIMx 计数器 (TIMx\_CNT) (x = 3, 4)

偏移地址: 0x024

复位值: 0x0000 0000

|           |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-----------|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31        | 30       | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| UIF       | Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| CPY       |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| r         |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15        | 14       | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CNT[15:0] |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw        | rw       | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31 UIFCPY: 值取决于 TIMx\_CR1 的 UIFREMAP

如果 UIFREMAP = 0

保留

如果 UIFREMAP = 1

UIFCPY: 复制 UIF

该位是 TIMx\_SR 寄存器中 UIF 位的一个只读副本。

Bit 30:16 保留, 必须保持复位值。

Bit 15:0 CNT[15:0]: 计数器的值 (Counter value)

在非抖动模式下 (DITHEN = 0)

该寄存器 (CNT[15:0]) 保存的是计数器的值。

在抖动模式下 (DITHEN = 1)

该寄存器 (CNT[15:0]) 只保存计数器的不抖动部分。抖动部分不可用。

### 24.7.13 TIMx 计数器 (TIMx\_CNT) (x = 2, 5)

偏移地址: 0x024

复位值: 0x0000 0000

|           |            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-----------|------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31        | 30         | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| UIF       | CNT[30:16] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| CPY_      |            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| CNT       |            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| [31]      |            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| r         |            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15        | 14         | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CNT[15:0] |            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw        | rw         | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31 UIFCPY\_CNT[31]: 值取决于 TIMx\_CR1 的 UIFREMAP

如果 UIFREMAP = 0

CNT[31]: 计数器值的最高有效位

如果 UIFREMAP = 1

UIFCPY: 复制 UIF

该位是 TIMx\_SR 寄存器中 UIF 位的一个只读副本。

Bit 30:0 CNT[30:0]: 计数器值的最小有效部分 (Least significant part of counter value)

在非抖动模式下 (DITHEN = 0)

该寄存器 (CNT[15:0]) 保存的是计数器的值。

在抖动模式下 (DITHEN = 1)

该寄存器 (CNT[30:0]) 只保存计数器的不抖动部分。抖动部分不可用。

#### 24.7.14 TIMx 预分频器 (TIMx\_PSC) (x=2~5)

偏移地址: 0x028

复位值: 0x0000

| 15        | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
|-----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| PSC[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw        | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 15:0 PSC[15:0]: 预分频器的值 (Prescaler value)

计数器时钟频率 ( $f_{tim\_cnt\_ck}$ ) 等于  $f_{tim\_psc\_ck} / (PSC[15:0] + 1)$ 。

PSC 包含每次发生更新事件 (包括计数器通过 TIMx\_EGR 寄存器中的 UG 位清零时, 或在配置为“复位模式”时通过触发控制器清零时) 时要装载到活动预分频器寄存器的值。

#### 24.7.15 TIMx 自动重载寄存器 (TIMx\_ARR) (x = 3, 4)

偏移地址: 0x02C

复位: 0x0000 FFFF

| 31        | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19         | 18 | 17 | 16 |
|-----------|----|----|----|----|----|----|----|----|----|----|----|------------|----|----|----|
| Reserved  |    |    |    |    |    |    |    |    |    |    |    | ARR[19:16] |    |    |    |
|           |    |    |    |    |    |    |    |    |    |    |    | rw         | rw | rw | rw |
| 15        | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3          | 2  | 1  | 0  |
| ARR[15:0] |    |    |    |    |    |    |    |    |    |    |    |            |    |    |    |
| rw        | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw         | rw | rw | rw |

Bit 31:20 保留, 必须保持复位值。

Bit 19:0 ARR[19:0]: 自动重载的值 (Auto-reload value)

ARR 为要装载到实际自动重载寄存器的值。

当自动重载值为空时, 计数器不工作。

在非抖动模式下 (DITHEN = 0)

该寄存器存储的是自动重新加载的值

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 ARR[19:4]中的整数部分, 而 ARR[3:0]位域存储的是抖动的部分

#### 24.7.16 TIMx 自动重载寄存器 (TIMx\_ARR) (x = 2, 5)

偏移地址: 0x02C

复位: 0xFFFF FFFF

| 31         | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| ARR[31:16] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw         | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |
| 15         | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| ARR[15:0]  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw         | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:0 ARR[31:0]: 自动重载的值 (Auto-reload value)

ARR 为要装载到实际自动重载寄存器的值。

当自动重载值为空时, 计数器不工作。

在非抖动模式下 (DITHEN = 0)

该寄存器存储的是自动重新加载的值

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 ARR[31:4]中的整数部分, 而 ARR[3:0]位域存储的是抖动的部分

### 24.7.17 TIMx 捕获/比较寄存器 1 (TIMx\_CCR1) (x = 3, 4)

偏移地址: 0x034

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|--------------|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19           | 18 | 17 | 16 |
| Reserved    |    |    |    |    |    |    |    |    |    |    |    | CCR1 [19:16] |    |    |    |
|             |    |    |    |    |    |    |    |    |    |    |    | rw           | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3            | 2  | 1  | 0  |
| CCR1 [15:0] |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw           | rw | rw | rw |

Bit 31:20 保留, 必须保持复位值。

Bit 19:0 CCR1[19:0]: 捕获/比较 1 值 (Capture/Compare 1 value)

若 CC1 通道配置为输出:

CCR1 为要装载到实际捕获/比较 1 寄存器的值 (预装载值)。

如果没有通过 TIMx\_CCMR1 寄存器中的 OC1PE 位来使能预装载功能, 写入的数值会被直接传输至当前寄存器中。否则只有发生更新事件时, 预装载值才会复制到活动捕获/比较 1 寄存器中。实际捕获/比较寄存器中包含要与计数器 TIMx\_CNT 进行比较并在 tim\_oc1 输出上发出信号的值。

在非抖动模式下 (DITHEN = 0)

该寄存器存储的是 CCR1[15:0]中的比较值。CCR1[19:16]位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 CCR1[19:4]中的整数部分。CCR1[3:0]位域存储的是抖动的部分。

若 CC1 通道配置为输入:

CCR1 为上一个输入捕获 1 事件 (tim\_ic1) 发生时的计数器值。TIMx\_CCR1 是只读的并且不可编程。

在非抖动模式下 (DITHEN = 0)

CCR1[15:0]位存储了捕获值。CCR1[19:16]位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储捕获在 CCR1[19:4]中。CCR1[3:0]位会被重置。

### 24.7.18 TIMx 捕获/比较寄存器 1 (TIMx\_CCR1) (x = 2, 5)

偏移地址: 0x034

复位值: 0x0000 0000

|              |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|--------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31           | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| CCR1 [31:16] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw           | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |
| 15           | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CCR1 [15:0]  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw           | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:0 CCR1[31:0]: 捕获/比较 1 值 (Capture/Compare 1 value)

若 CC1 通道配置为输出:

CCR1 为要装载到实际捕获/比较 1 寄存器的值 (预装载值)。

如果没有通过 TIMx\_CCMR1 寄存器中的 OC1PE 位来使能预装载功能, 写入的数值会被直接传输至当前寄存器中。否则只有发生更新事件时, 预装载值才会复制到活动捕获/比较 1 寄存器中。实际捕获/比较寄存器中包含要与计数器 TIMx\_CNT 进行比较并在 tim\_oc1 输出上发出信号的值。

在非抖动模式下 (DITHEN = 0)

该寄存器存储比较值。

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 CCR1[31:4]中的整数部分。CCR1[3:0]位域存储的是抖动的部分。

若 CC1 通道配置为输入:

CCR1 为上一个输入捕获 1 事件 (tim\_ic1) 发生时的计数器值。TIMx\_CCR1 是只读的并且不可编程。

在非抖动模式下 (DITHEN = 0)

该寄存器存储捕获值。

在抖动模式下 (DITHEN = 1)

该寄存器存储捕获在 CCR1[31:4]中。CCR1[3:0]位会被重置。

#### 24.7.19 TIMx 捕获/比较寄存器 2 (TIMx\_CCR2) (x = 3, 4)

偏移地址: 0x038

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|--------------|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19           | 18 | 17 | 16 |
| Reserved    |    |    |    |    |    |    |    |    |    |    |    | CCR2 [19:16] |    |    |    |
|             |    |    |    |    |    |    |    |    |    |    |    | rw           | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3            | 2  | 1  | 0  |
| CCR2 [15:0] |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw           | rw | rw | rw |

Bit 31:20 保留，必须保持复位值。

Bit 19:0 **CCR2[19:0]**: 捕获/比较 2 值 (Capture/Compare 2 value)

若 **CC2 通道配置为输出**:

CCR2 为要装载到实际捕获/比较 2 寄存器的值 (预装载值)。

如果没有通过 TIMx\_CCMR1 寄存器中的 OC2PE 位来使能预装载功能，写入的数值会被直接传输至当前寄存器中。否则只有发生更新事件时，预装载值才会复制到活动捕获/比较 2 寄存器中。实际捕获/比较寄存器中包含要与计数器 TIMx\_CNT 进行比较并在 tim\_oc2 输出上发出信号的值。

在非抖动模式下 (DITHEN = 0)

该寄存器存储的是 CCR2[15:0] 中的比较值。CCR2[19:16] 位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 CCR2[19:4] 中的整数部分。CCR2[3:0] 位域存储的是抖动的部分。

若 **CC2 通道配置为输入**:

CCR2 为上一个输入捕获 2 事件 (tim\_ic2) 发生时的计数器值。TIMx\_CCR2 是只读的并且不可编程。

在非抖动模式下 (DITHEN = 0)

CCR2[15:0] 位存储了捕获值。CCR2[19:16] 位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储捕获在 CCR2[19:4] 中。CCR2[3:0] 位会被重置。

## 24.7.20 TIMx 捕获/比较寄存器 2 (TIMx\_CCR2) (x = 2, 5)

偏移地址: 0x038

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| CCR2[31:16] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CCR2[15:0]  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:0 CCR2[31:0]: 捕获/比较 2 值 (Capture/Compare 2 value)

若 CC2 通道配置为输出:

CCR2 为要装载到实际捕获/比较 2 寄存器的值 (预装载值)。

如果没有通过 TIMx\_CCMR1 寄存器中的 OC2PE 位来使能预装载功能, 写入的数值会被直接传输至当前寄存器中。否则只有发生更新事件时, 预装载值才会复制到活动捕获/比较 2 寄存器中。实际捕获/比较寄存器中包含要与计数器 TIMx\_CNT 进行比较并在 tim\_oc1 输出上发出信号的值。

在非抖动模式下 (DITHEN = 0)

该寄存器存储比较值。

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 CCR2[31:4]中的整数部分。CCR2[3:0]位域存储的是抖动的部分。

若 CC2 通道配置为输入:

CCR2 为上一个输入捕获 2 事件 (tim\_ic2) 发生时的计数器值。TIMx\_CCR2 是只读的并且不可编程。

在非抖动模式下 (DITHEN = 0)

该寄存器存储捕获值。

在抖动模式下 (DITHEN = 1)

该寄存器存储捕获在 CCR2[31:4]中。CCR2[3:0]位会被重置。

#### 24.7.21 TIMx 捕获/比较寄存器 3 (TIMx\_CCR3) (x = 3, 4)

偏移地址: 0x03C

复位值: 0x0000 0000

|            |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
|------------|----|----|----|----|----|----|----|----|----|----|----|--------------|----|----|----|
| 31         | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19           | 18 | 17 | 16 |
| Reserved   |    |    |    |    |    |    |    |    |    |    |    | CCR3 [19:16] |    |    |    |
|            |    |    |    |    |    |    |    |    |    |    |    | rw           | rw | rw | rw |
| 15         | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3            | 2  | 1  | 0  |
| CCR3[15:0] |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
| rw         | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw           | rw | rw | rw |

Bit 31:20 保留，必须保持复位值。

Bit 19:0 **CCR3[19:0]**: 捕获/比较 3 值 (Capture/Compare 3 value)

若 **CC3 通道配置为输出**:

CCR3 为要装载到实际捕获/比较 3 寄存器的值 (预装载值)。

如果没有通过 TIMx\_CCMR2 寄存器中的 OC3PE 位来使能预装载功能，写入的数值会被直接传输至当前寄存器中。否则只有发生更新事件时，预装载值才会复制到活动捕获/比较 3 寄存器中。实际捕获/比较寄存器中包含要与计数器 TIMx\_CNT 进行比较并在 tim\_oc3 输出上发出信号的值。

在非抖动模式下 (DITHEN = 0)

该寄存器存储的是 CCR3[15:0] 中的比较值。CCR3[19:16] 位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 CCR3[19:4] 中的整数部分。CCR3[3:0] 位域存储的是抖动的部分。

若 **CC3 通道配置为输入**:

CCR3 为上一个输入捕获 3 事件 (tim\_ic3) 发生时的计数器值。TIMx\_CCR3 是只读的并且不可编程。

在非抖动模式下 (DITHEN = 0)

CCR3[15:0] 位存储了捕获值。CCR3[19:16] 位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储捕获在 CCR3[19:4] 中。CCR3[3:0] 位会被重置。

## 24.7.22 TIMx 捕获/比较寄存器 3 (TIMx\_CCR3) (x = 2, 5)

偏移地址: 0x03C

复位值: 0x0000 0000

| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| CCR3[31:16] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | Rw | rw | rw | rw | rw | rw | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CCR3[15:0]  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |



Bit 31:0 **CCR3[31:0]**: 捕获/比较 3 值 (Capture/Compare 3 value)

若 **CC3 通道配置为输出**:

CCR3 为要装载到实际捕获/比较 3 寄存器的值 (预装载值)。

如果没有通过 TIMx\_CCMR2 寄存器中的 OC3PE 位来使能预装载功能, 写入的数值会被直接传输至当前寄存器中。否则只有发生更新事件时, 预装载值才会复制到活动捕获/比较 3 寄存器中。实际捕获/比较寄存器中包含要与计数器 TIMx\_CNT 进行比较并在 tim\_oc3 输出上发出信号的值。

在非抖动模式下 (DITHEN = 0)

该寄存器存储比较值。

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 CCR3[31:4]中的整数部分。CCR3[3:0]位域存储的是抖动的部分。

若 **CC3 通道配置为输入**:

CCR3 为上一个输入捕获 3 事件 (tim\_ic3) 发生时的计数器值。TIMx\_CCR3 是只读的并且不可编程。

在非抖动模式下 (DITHEN = 0)

该寄存器存储捕获值。

在抖动模式下 (DITHEN = 1)

该寄存器存储捕获在 CCR3[31:4]中。CCR3[3:0]位会被重置。

### 24.7.23 TIMx 捕获/比较寄存器 4 (TIMx\_CCR4) (x = 3, 4)

偏移地址: 0x040

复位值: 0x0000 0000

|            |    |    |    |    |    |    |    |    |    |    |    |             |    |    |    |
|------------|----|----|----|----|----|----|----|----|----|----|----|-------------|----|----|----|
| 31         | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19          | 18 | 17 | 16 |
| Reserved   |    |    |    |    |    |    |    |    |    |    |    | CCR4[19:16] |    |    |    |
|            |    |    |    |    |    |    |    |    |    |    |    | rw          | rw | rw | rw |
| 15         | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3           | 2  | 1  | 0  |
| CCR4[15:0] |    |    |    |    |    |    |    |    |    |    |    |             |    |    |    |
| rw         | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw          | rw | rw | rw |

Bit 31:20 保留，必须保持复位值。

Bit 19:0 **CCR4[19:0]**: 捕获/比较 4 值 (Capture/Compare 4 value)

若 **CC4 通道配置为输出**:

CCR4 为要装载到实际捕获/比较 4 寄存器的值 (预装载值)。

如果没有通过 TIMx\_CCMR2 寄存器中的 OC4PE 位来使能预装载功能，写入的数值会被直接传输至当前寄存器中。否则只有发生更新事件时，预装载值才会复制到活动捕获/比较 4 寄存器中。实际捕获/比较寄存器中包含要与计数器 TIMx\_CNT 进行比较并在 tim\_oc4 输出上发出信号的值。

在非抖动模式下 (DITHEN = 0)

该寄存器存储的是 CCR4[15:0]中的比较值。CCR4[19:16]位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 CCR4[19:4]中的整数部分。CCR4[3:0]位域存储的是抖动的部分。

若 **CC4 通道配置为输入**:

CCR4 为上一个输入捕获 4 事件 (tim\_ic4) 发生时的计数器值。TIMx\_CCR4 是只读的并且不可编程。

在非抖动模式下 (DITHEN = 0)

CCR4[15:0]位存储了捕获值。CCR4[19:16]位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储捕获在 CCR4[19:4]中。CCR4[3:0]位会被重置。

#### 24.7.24 TIMx 捕获/比较寄存器 4 (TIMx\_CCR4) (x = 2, 5)

偏移地址: 0x040

复位值: 0x0000 0000

| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| CCR4[31:16] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CCR4[15:0]  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:0 **CCR4[31:0]**: 捕获/比较 4 值 (Capture/Compare 4 value)

若 **CC4 通道配置为输出**:

CCR4 为要装载到实际捕获/比较 4 寄存器的值 (预装载值)。

如果没有通过 TIMx\_CCMR2 寄存器中的 OC4PE 位来使能预装载功能, 写入的数值会被直接传输至当前寄存器中。否则只有发生更新事件时, 预装载值才会复制到活动捕获/比较 4 寄存器中。实际捕获/比较寄存器中包含要与计数器 TIMx\_CNT 进行比较并在 tim\_oc4 输出上发出信号的值。

在非抖动模式下 (DITHEN = 0)

该寄存器存储比较值。

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 CCR4[31:4]中的整数部分。CCR4[3:0]位域存储的是抖动的部分。

若 **CC4 通道配置为输入**:

CCR4 为上一个输入捕获 4 事件 (tim\_ic4) 发生时的计数器值。TIMx\_CCR4 是只读的并且不可编程。

在非抖动模式下 (DITHEN = 0)

该寄存器存储捕获值。

在抖动模式下 (DITHEN = 1)

该寄存器存储捕获在 CCR4[31:4]中。CCR4[3:0]位会被重置。

#### 24.7.25 TIMx 编码器控制寄存器 (TIMx\_ECR) (x=2~5)

偏移地址: 0x058

复位值: 0x0000 0000

|          |    |    |    |    |             |    |    |           |    |      |          |    |           |    |    |
|----------|----|----|----|----|-------------|----|----|-----------|----|------|----------|----|-----------|----|----|
| 31       | 30 | 29 | 28 | 27 | 26          | 25 | 24 | 23        | 22 | 21   | 20       | 19 | 18        | 17 | 16 |
| Reserved |    |    |    |    | PWPRSC[2:0] |    |    | PW[7:0]   |    |      |          |    |           |    |    |
|          |    |    |    |    | rw          | rw | rw | rw        | rw | rw   | rw       | rw | rw        | rw | rw |
| 15       | 14 | 13 | 12 | 11 | 10          | 9  | 8  | 7         | 6  | 5    | 4        | 3  | 2         | 1  | 0  |
| Reserved |    |    |    |    |             |    |    | IPOS[1:0] |    | FIDX | Reserved |    | IDIR[1:0] |    | IE |
|          |    |    |    |    |             |    |    | rw        | rw | rw   |          |    | rw        | rw | rw |

Bit 31:27 保留, 必须保持复位值。

Bit 26:24 **PWPRSC[2:0]**: 脉冲宽度预分频器 (Pulse width prescaler)

该位域设置脉冲发生器的时钟预分频, 如下所示:

$$t_{PWG} = (2^{PWPRSC[2:0]}) \times t_{tim\_ker\_ck}$$

Bit 23:16 **PW[7:0]**: 脉冲宽度 (Pulse width)

该位域定义脉冲持续时间, 如下所示:

$$t_{PW} = PW[7:0] \times t_{PWG}$$

Bit 15:8 保留, 必须保持复位值。

### Bit 7:6 IPOS[1:0]: 索引位置 (Index positioning)

在正交编码器模式 (SMS[3:0] = 0001, 0010, 0011, 1110, 1111) 下, 该位指示了索引事件在哪个 AB 输入配置下会重置计数器。

00: 当 AB=00 时, 索引重置计数器

01: 当 AB=01 时, 索引重置计数器

10: 当 AB=10 时, 索引重置计数器

11: 当 AB=11 时, 索引重置计数器

在方向时钟模式或时钟加方向模式 (SMS[3:0] = 1010, 1011, 1100, 1101) 下, 此位指示了索引事件在哪个时钟水平上重置计数器。在双向时钟模式下, 这适用于两个时钟输入。具体为:

x0: 当时钟=0 时, 索引重置计数器

x1: 当时钟=1 时, 索引重置计数器

注: IPOS[1] 位不重要

### Bit 5 FIDX[1:0]: 首位索引 (First index)

该位指示是否仅考虑首位索引

0: 索引始终处于活跃状态

1: 仅首个索引会重置计数器

Bit 4:3 保留, 必须保持复位值。

### Bit 2:1 IDIR[1:0]: 索引方向 (Index direction)

该位指示索引事件在哪个方向上重置计数器。

00: 无论方向如何, 索引都会重置计数器

01: 仅在递增计数时, 索引会重置计数器

10: 仅在递减计数时, 索引会重置计数器

11: 保留

注: 当 IE 位被复位 (禁止索引) 时, 必须写入 IDR[1:0] 位域

### Bit 0 IE: 索引使能 (Index enable)

该位指示索引事件是否能重置计数器

0: 禁止索引

1: 使能索引

## 24.7.26 TIMx 定时器输入选择寄存器 (TIMx\_TISEL) (x=2~5)

偏移地址: 0x05C

复位值: 0x0000 0000

|          |    |    |    |             |    |    |    |          |    |    |    |             |    |    |    |
|----------|----|----|----|-------------|----|----|----|----------|----|----|----|-------------|----|----|----|
| 31       | 30 | 29 | 28 | 27          | 26 | 25 | 24 | 23       | 22 | 21 | 20 | 19          | 18 | 17 | 16 |
| Reserved |    |    |    | TI4SEL[3:0] |    |    |    | Reserved |    |    |    | TI3SEL[3:0] |    |    |    |
|          |    |    |    | rw          | rw | rw | rw |          |    |    |    | rw          | rw | rw | rw |
| 15       | 14 | 13 | 12 | 11          | 10 | 9  | 8  | 7        | 6  | 5  | 4  | 3           | 2  | 1  | 0  |
| Reserved |    |    |    | TI2SEL[3:0] |    |    |    | Reserved |    |    |    | TI1SEL[3:0] |    |    |    |
|          |    |    |    | rw          | rw | rw | rw |          |    |    |    | rw          | rw | rw | rw |

Bit 31:28 保留, 必须保持复位值。

Bit 27:24 **TI4SEL[3:0]**: 选择 tim\_ti4[15:0]输入 (Selects tim\_ti4[15:0] input)

0000: tim\_ti4\_in0: TIMx\_CH4

0001: tim\_ti4\_in1

...

1111: tim\_ti4\_in15

参考章节: TIM2/TIM3/TIM4/TIM5 引脚和内部信号获得关联列表。

Bit 23:20 保留, 必须保持复位值。

Bit 19:16 **TI3SEL[3:0]**: 选择 tim\_ti3[15:0]输入 (Selects tim\_ti3[15:0] input)

0000: tim\_ti3\_in0: TIMx\_CH3

0001: tim\_ti3\_in1

...

1111: tim\_ti3\_in15

参考章节: TIM2/TIM3/TIM4/TIM5 引脚和内部信号获得关联列表。

Bit 15:12 保留, 必须保持复位值。

Bit 11:8 **TI2SEL[3:0]**: 选择 tim\_ti2[15:0]输入 (Selects tim\_ti2[15:0] input)

0000: tim\_ti2\_in0: TIMx\_CH2

0001: tim\_ti2\_in1

...

1111: tim\_ti2\_in15

参考章节: TIM2/TIM3/TIM4/TIM5 引脚和内部信号获得关联列表。

Bit 7:4 保留, 必须保持复位值。

Bit 3:0 **TI1SEL[3:0]**: 选择 tim\_ti1[15:0]输入 (Selects tim\_ti1[15:0] input)

0000: tim\_ti1\_in0: TIMx\_CH1

0001: tim\_ti1\_in1

...

1111: tim\_ti1\_in15

参考章节: TIM2/TIM3/TIM4/TIM5 引脚和内部信号获得关联列表。

## 24.7.27 TIMx 复用功能选择寄存器 1 (TIMx\_AF1) (x=2~5)

偏移地址: 0x060

复位值: 0x0000 0000

|             |    |          |    |    |    |    |    |    |    |    |    |    |    |             |    |
|-------------|----|----------|----|----|----|----|----|----|----|----|----|----|----|-------------|----|
| 31          | 30 | 29       | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17          | 16 |
| Reserved    |    |          |    |    |    |    |    |    |    |    |    |    |    | ETRSEL[3:2] |    |
|             |    |          |    |    |    |    |    |    |    |    |    |    |    | rw          | rw |
| 15          | 14 | 13       | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1           | 0  |
| ETRSEL[1:0] |    | Reserved |    |    |    |    |    |    |    |    |    |    |    |             |    |
| rw          | rw |          |    |    |    |    |    |    |    |    |    |    |    |             |    |

Bit 31:18 保留，必须保持复位值。

Bit 17:14 **ETRSEL[3:0]**: etr\_in 的源选择 (etr\_in source selection)

这些位选择 etr\_in 的输入源

0000: tim\_etr0: TIMx\_ETR 输入

0001: tim\_etr1

...

1111: tim\_etr15

请参考章节: TIM2/TIM3/TIM4/TIM5 引脚和内部信号，了解特定于产品的实现。

Bit 13:0 保留，必须保持复位值。

## 24.7.28 TIMx 复用功能选择寄存器 2 (TIMx\_AF2) (x=2~5)

偏移地址: 0x064

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |    |    |    |    |    |             |    |    |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|-------------|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18          | 17 | 16 |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    | OCRSEL[2:0] |    |    |
|          |    |    |    |    |    |    |    |    |    |    |    |    | rw          | rw | rw |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2           | 1  | 0  |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |             |    |    |

Bit 31:19 保留，必须保持复位值。

Bit 18:16 **OCRSEL[2:0]**: ocref\_clr 的源选择 (ocref\_clr source selection)

这些位选择 ocref\_clr 的输入源

000: tim\_ocref\_clr0

001: tim\_ocref\_clr1

...

111: tim\_ocref\_clr7

请参考章节: TIM2/TIM3/TIM4/TIM5 引脚和内部信号，了解特定于产品的实现。

Bit 15:0 保留，必须保持复位值。

## 24.7.29 TIMx DMA 控制寄存器 (TIMx\_DCR) (x=2~5)

偏移地址: 0x3DC

复位值: 0x0000 0000

|          |    |    |          |    |    |    |    |          |    |    |          |    |    |    |    |
|----------|----|----|----------|----|----|----|----|----------|----|----|----------|----|----|----|----|
| 31       | 30 | 29 | 28       | 27 | 26 | 25 | 24 | 23       | 22 | 21 | 20       | 19 | 18 | 17 | 16 |
| Reserved |    |    |          |    |    |    |    |          |    |    |          |    |    |    |    |
| 15       | 14 | 13 | 12       | 11 | 10 | 9  | 8  | 7        | 6  | 5  | 4        | 3  | 2  | 1  | 0  |
| Reserved |    |    | DBL[4:0] |    |    |    |    | Reserved |    |    | DBA[4:0] |    |    |    |    |
|          |    |    | rw       | rw | rw | rw | rw |          |    |    | rw       | rw | rw | rw | rw |

Bit 31:13 保留, 必须保持复位值。

Bit 12:8 **DBL[4:0]**: DMA 突发长度 (DMA burst length)

这个 5 位向量定义了 DMA 传输的长度 (当对 TIMx\_DMAR 地址进行读或写访问时, 定时器识别一次突发传输), 即传输次数。传输可以是半字或字节 (见下面的示例)。

00000: 1 次传输

00001: 2 次传输

00010: 3 次传输

...

11010: 26 次传输

**示例:** 让我们考虑以下传输: DBL = 7 字节 & DBA = TIM2\_CR1。如果 DBL = 7 字节, 并且 DBA = TIM2\_CR1 表示要传输的字节的地址, 那么传输的地址应由以下公式给出: (TIMx\_CR1 地址) + DBA + (DMA 索引), 其中 DMA 索引 = DBL

在这个例子中, 将 7 字节添加到(TIMx\_CR1 地址) + DBA, 这给出了数据从/复制到的地址。在这种情况下, 传输是从以下地址开始的 7 个寄存器: (TIMx\_CR1 地址) + DBA

根据 DMA 数据大小的配置, 可能会发生几种情况:

- 如果 DMA 数据大小配置为半字, 则将 16 位数据传输到 7 个寄存器中的每一个。
- 如果 DMA 数据大小配置为字节, 则数据也传输到 7 个寄存器: 第一个寄存器包含第一个 MSB 字节, 第二个寄存器包含第一个 LSB 字节, 依此类推。

因此, 在使用传输定时器时, 还必须指定 DMA 传输的数据大小。

Bit 7:5 保留, 必须保持复位值。

Bit 4:0 **DBA[4:0]**: DMA 基地址 (DMA base address)

这个 5 位向量定义了 DMA 传输的基地址 (当通过 TIMx\_DMAR 地址进行读/写访问时)。DBA 是从 TIMx\_CR1 寄存器的地址开始的偏移量。

00000: TIMx\_CR1,

00001: TIMx\_CR2,

00010: TIMx\_SMCR,

...



### 24.7.30 TIMx 用于完整传输的 DMA 地址 (TIMx\_DMAR) (x=2~5)

偏移地址: 0x3E0

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DMAB[31:16] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| DMAB[15:0]  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:0 **DMAB[31:0]**: 用于突发访问的 DMA 寄存器 (DMA register for burst accesses)

对 DMAR 寄存器进行的读或写操作将访问位于以下地址的寄存器:

$$(TIMx\_CR1 \text{ 地址}) + (DBA + \text{DMA 索引}) \times 4$$

其中, TIMx\_CR1 地址是控制寄存器 1 的地址, DBA 是在 TIMx\_DCR 寄存器中配置的 DMA 基地址, DMA 索引由 DMA 传输自动控制, 其范围从 0 到 DBL (在 TIMx\_DCR 中配置的 DBL)。

## 25 通用定时器 (TIM15/TIM16/TIM17)

### 25.1 TIM15/TIM16/TIM17 简介

TIM15/TIM16/TIM17 定时器包含一个 16 位自动重载计数器，该计数器由可编程预分频器驱动。

它可用于多种用途，包括测量输入信号的脉冲宽度（输入捕获）或生成输出波形（输出比较和 PWM 和带死区插入的互补 PWM）。使用定时器预分频器和 RCC 时钟控制器预分频器，可将脉冲宽度和波形周期从几微秒调制到几毫秒。TIM15/TIM16/TIM17 定时器是完全独立的，不与其他定时器共享任何资源。TIM15 可以按照章节：定时器同步（仅 TIM15）的描述进行同步。

### 25.2 TIM15 主要功能

TIM15 定时器功能包括：

- 16 位自动装载计数器
- 16 位可编程（可以实时修改）预分频器，计数器时钟频率的分频系数为 1~65536 之间的任意数值
- 2 个独立通道：
  - 输入捕获
  - 输出比较
  - PWM 生成（边缘模式）
  - 单脉冲模式输出
- 带可编程死区的互补输出（仅限通道 1）
- 使用外部信号控制定时器和定时器互连的同步电路
- 重复计数器，用于仅在给定数目的计数器周期后更新定时器寄存器。
- 用于将定时器的输出信号置于复位状态或已知状态的刹车输入。
- 如下事件发生时产生中断/DMA：
  - 更新：计数器溢出，计数器初始化（通过软件或者内部/外部触发）
  - 触发事件（计数器启动、停止、初始化或者由内部/外部触发计数）
  - 输入捕获
  - 输出比较
  - 刹车输入（中断请求）

### 25.3 TIM16/TIM17 主要功能

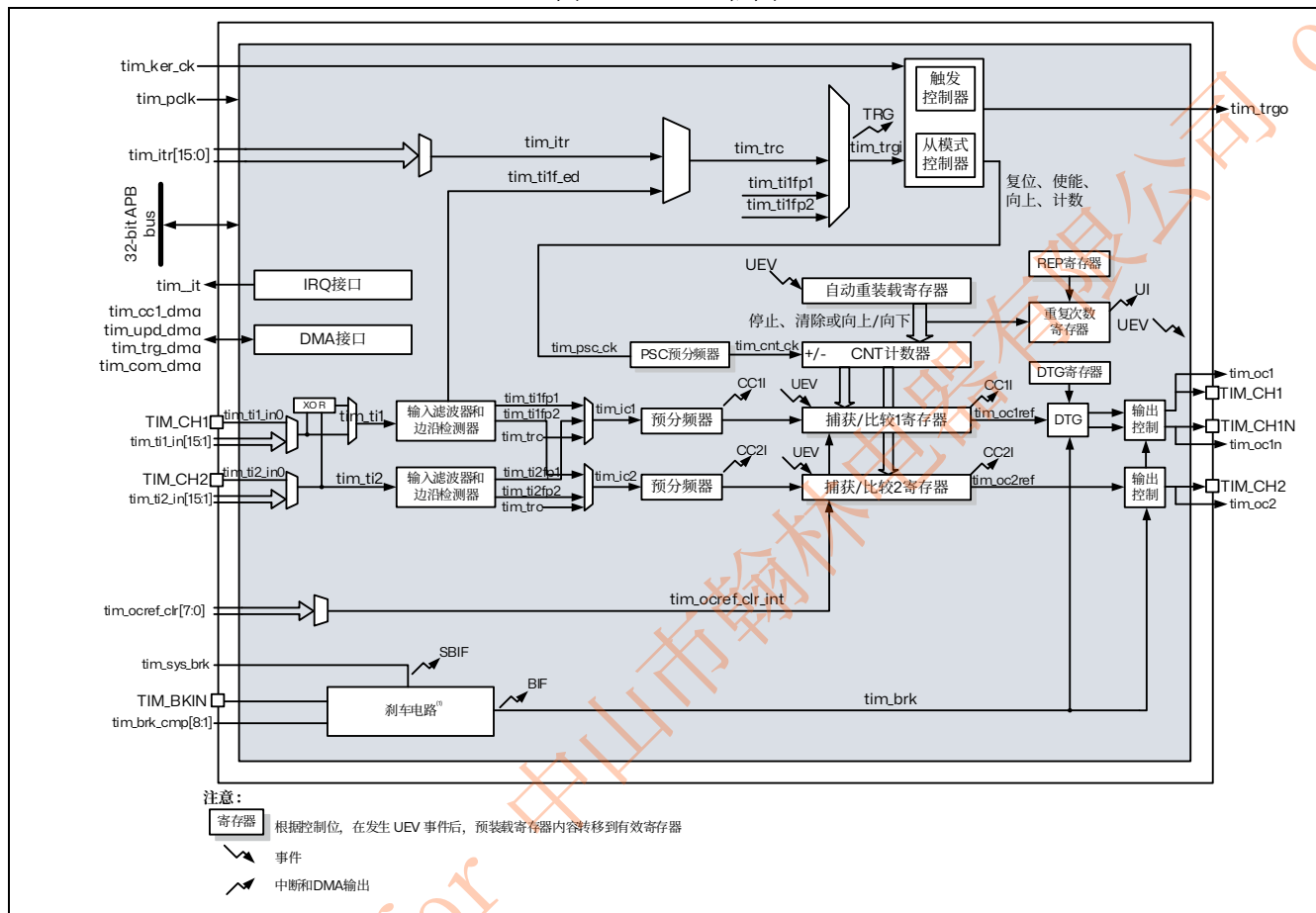
TIM16/TIM17 定时器功能包括：

- 16 位自动装载计数器
- 16 位可编程（可以实时修改）预分频器，计数器时钟频率的分频系数为 1~65536 之间的任意数值
- 1 个通道：
  - 输入捕获
  - 输出比较
  - PWM 生成（边缘模式）
  - 单脉冲模式输出
- 带可编程死区的互补输出
- 重复计数器，用于仅在给定数目的计数器周期后更新定时器寄存器。
- 用于将定时器的输出信号置于复位状态或已知状态的刹车输入。
- 如下事件发生时产生中断/DMA：
  - 更新：计数器溢出
  - 输入捕获
  - 输出比较
  - 刹车输入

## 25.4 TIM15/TIM16/TIM17 功能描述

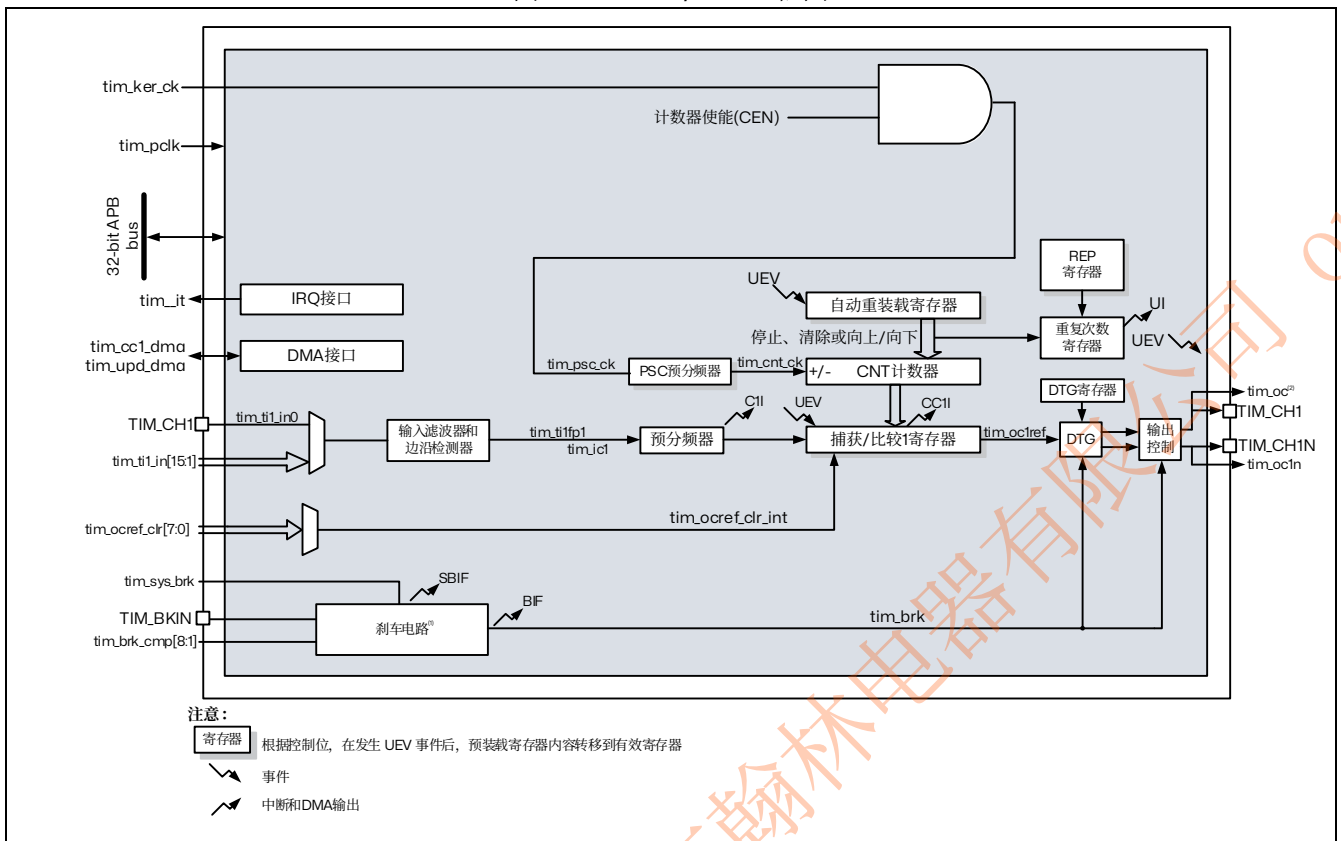
### 25.4.1 框图

图 25.1 TIM15 框图



注 1. 参考章节: 使用刹车功能以获得细节

图 25.2 TIM16/TIM17 框图



注 1. 参考章节: 使用刹车功能以获得细节

注 2. 此信号可作为某些从属定时器的触发信号 (请参阅下一节中的内部触发连接表)。有关详细信息, 请参考章节: 使用定时器输出作为其他定时器的触发信号 (仅 TIM16/TIM17)。

## 25.4.2 TIM15/TIM16/TIM17 引脚和内部信号

本节中的下两张表格总结了 TIM 的输入和输出。

表 25.1 TIM 输入/输出引脚

| 引脚名称                              | 信号类型  | 描述                                                                                             |
|-----------------------------------|-------|------------------------------------------------------------------------------------------------|
| TIM_CH1<br>TIM_CH2 <sup>(1)</sup> | 输入/输出 | 定时器多功能通道。<br>每个通道用于捕获、比较或 PWM。<br>TIM_CH1 和 TIM_CH2 也可用作外部时钟 (tim_ker_ck 时钟的 1/4 以下) 和外部触发器输入。 |
| TIM_CH1N                          | 输出    | 由 TIM_CH1 输出派生的定时器的互补输出，具有插入死区时间的能力。                                                           |
| TIM_BKIN                          | 输入/输出 | 刹车输入。此输入也可以配置为双向模式。                                                                            |

1. 仅供 TIM15 使用。

表 25.2 TIM 内部输入/输出信号

| 内部信号名称                                              | 信号类型 | 描述                                                                                                 |
|-----------------------------------------------------|------|----------------------------------------------------------------------------------------------------|
| tim_ti1_in[15:0]<br>tim_ti2_in[15:0] <sup>(1)</sup> | 输入   | 内部定时器输入总线。<br>这些输入可被用作捕获或作为外部时钟 (tim_ker_ck 时钟的 1/4 以下)。                                           |
| tim_itr[15:0] <sup>(1)</sup>                        | 输入   | 内部触发输入总线。这些输入可用于从模式控制器或作为输入时钟 (tim_ker_ck 时钟的 1/4 以下)。                                             |
| tim_trgo <sup>(1)</sup>                             | 输出   | 内部触发器输出。此触发器可以触发其他片上外设。                                                                            |
| tim_ocref_clr[7:0]                                  | 输入   | 定时器 tim_ocref_clr 输入总线。这些输入可以用来清除 tim_ocxref 信号，通常用于硬件的逐周期脉冲宽度控制。                                  |
| tim_brk_cmp[8:1]                                    | 输入   | 内部信号的刹车输入。                                                                                         |
| tim_sys_brk[n:0]                                    | 输入   | 系统中断输入。该输入收集 MCU 的系统级错误。                                                                           |
| tim_pclk                                            | 输入   | 定时器 APB 时钟。                                                                                        |
| tim_ker_ck                                          | 输入   | 定时器内核时钟。此时钟必须与 tim_pclk 同步 (来自同一源)。<br>时钟比率 tim_ker_ck/ tim_pclk 必须为整数：<br>1, 2, 3, ..., 16 (最大值)。 |
| tim_it                                              | 输出   | 全局定时器中断，收集捕获/比较、更新、中断触发和换向请求。                                                                      |
| tim_cc1_dma                                         | 输出   | 定时器捕获/比较 1 dma 请求。                                                                                 |
| tim_upd_dma                                         | 输出   | 定时器更新 dma 请求。                                                                                      |
| tim_trg_dma                                         | 输出   | 定时器触发 dma 请求。                                                                                      |
| tim_com_dma                                         | 输出   | 定时器换向 dma 请求。                                                                                      |

1. 仅供 TIM15 使用。

如下两张表列出了连接到 tim\_ti[2:1]输入多路复用器的源。

表 25.3 连接到 tim\_ti1 的输入多路复用器

| tim_ti1 输入       | 源         |                         |                         |
|------------------|-----------|-------------------------|-------------------------|
|                  | TIM15     | TIM16                   | TIM17                   |
| tim_ti1_in0      | TIM15_CH1 | TIM16_CH1               | TIM17_CH1               |
| tim_ti1_in1      | 保留        |                         |                         |
| tim_ti1_in2      | comp1_out | MCO                     | MCO                     |
| tim_ti1_in3      | comp2_out | HSE / 32 <sup>(1)</sup> | HSE / 32 <sup>(1)</sup> |
| tim_ti1_in4      | 保留        | RTC Clock               | RTC Clock               |
| tim_ti1_in5      | 保留        |                         |                         |
| tim_ti1_in6      | 保留        | LSI                     | LSI                     |
| tim_ti1_in[15:7] | 保留        |                         |                         |

1. 仅供 TIM15 使用。此信号仅在 TIMx\_OR1 寄存器中设置了 HSE32EN 位时可用，有关详细信息，请参考章节：TIMx 选项寄存器 1 (TIMx\_OR1) (x = 16~17)。

表 25.4 连接到 tim\_ti2 的输入多路复用器

| tim_ti2 输入       | 源         |
|------------------|-----------|
|                  | TIM15     |
| tim_ti2_in0      | TIM15_CH2 |
| tim_ti2_in1      | comp2_out |
| tim_ti2_in2      | comp3_out |
| tim_ti2_in3      | 保留        |
| tim_ti2_in4      |           |
| tim_ti2_in[15:5] |           |

下表列出连接到 tim\_itr 输入多路复用器的内部源

表 25.5 TIMx 内部触发连接

| TIMx           | TIM15           |
|----------------|-----------------|
| tim_itr0       | tim1_trgo       |
| tim_itr1       | tim2_trgo       |
| tim_itr2       | tim3_trgo       |
| tim_itr3       | tim4_trgo       |
| tim_itr4       | tim5_trgo       |
| tim_itr5       | tim8_trgo       |
| tim_itr6       | 保留              |
| tim_itr7       | tim16_oc1       |
| tim_itr8       | tim17_oc1       |
| tim_itr9       | 保留              |
| tim_itr10      | hrtim_out_sync2 |
| tim_itr[15:11] | 保留              |



下两张表列出了连接到 tim\_brk 输入的源

表 25.6 定时器刹车互连

| tim_brk 输入   | TIM15          | TIM16          | TIM17          |
|--------------|----------------|----------------|----------------|
| TIM_BKIN     | TIM15_BKIN pin | TIM16_BKIN pin | TIM17_BKIN pin |
| tim_brk_cmp1 | comp1_out      | comp1_out      | comp1_out      |
| tim_brk_cmp2 | comp2_out      | comp2_out      | comp2_out      |
| tim_brk_cmp3 | comp3_out      | comp3_out      | comp3_out      |
| tim_brk_cmp4 | comp4_out      | comp4_out      | comp4_out      |

表 25.7 系统中断互连

| tim_sys_brk 输入 | TIM15/TIM16/TIM17         | SYSCFG_CFGR2 寄存器的使能位 |
|----------------|---------------------------|----------------------|
| tim_sys_brk0   | 带 FPU 的 Cortex®-M4 LOCKUP | CLL                  |
| tim_sys_brk1   | 可编程电压检测器 (PVD)            | PVDL                 |
| tim_sys_brk2   | SRAM 奇偶校验错误               | SPL                  |
| tim_sys_brk3   | Flash ECC 错误              | ECCL                 |
| tim_sys_brk4   | 时钟安全系统(CSS)               | 无 (始终使能)             |

下表列出连接到 tim\_ocref\_clr 输入多路复用器的内部源

表 25.8 连接到 ocref\_clr 的输入多路复用器

| 定时器 OCREF 清除信号 | 定时器 tim_ocref_clr 信号分配 |           |           |
|----------------|------------------------|-----------|-----------|
|                | TIM15                  | TIM16     | TIM17     |
| tim_ocref_clr0 | comp1_out              | comp1_out | comp1_out |
| tim_ocref_clr1 | comp2_out              | comp2_out | comp2_out |
| tim_ocref_clr2 | comp3_out              | comp3_out | comp3_out |
| tim_ocref_clr3 | comp4_out              | comp4_out | comp4_out |

### 25.4.3 时基单元

可编程定时器的主要模块由一个 16 位递增计数器及其相关的自动重载寄存器组成。计数器的时钟可通过预分频器进行分频。

计数器、自动重载寄存器和预分频器寄存器可通过软件进行读写。即使在计数器运行时也可执行读写操作。

时基单元包含：

- 计数器寄存器 (TIMx\_CNT)
- 预分频器寄存器 (TIMx\_PSC)
- 自动装载寄存器 (TIMx\_ARR)
- 重复计数器寄存器 (TIMx\_RCR)

自动重载寄存器是预装载的。对自动重载寄存器执行写入或读取操作时会访问预装载寄存器。预装载寄存器的内容既可以传送到影子寄存器，也可以在每次发生更新事件(UEV)时传送到影子寄存器，这取决于 TIMx\_CR1 寄存器中的自动重载预装载使能位 (ARPE)。当计数器达到溢出并且 TIMx\_CR1 寄存器中的 UDIS 位为 0 时，将发送更新事件。该更新事件也可由软件产生。下文将针对各配置的更新事件的产生进行详细介绍。

计数器由预分频器输出 CK\_CNT 提供时钟，仅当 TIMx\_CR1 寄存器中的计数器启动位 (CEN) 置 1 时，才会启动计数器（有关计数器使能的更多详细信息，另请参见从模式控制器的相关说明）。

注意，计数器将在 TIMx\_CR1 寄存器的 CEN 位置 1 时刻的一个时钟周期后开始计数。

#### 预分频器描述

预分频器可对计数器时钟频率进行分频，分频系数介于 1 到 65536 之间。该预分频器基于 16 位寄存器 (TIMx\_PSC 寄存器) 所控制的 16 位计数器。由于该控制寄存器具有缓冲功能，因此预分频器可实现实时更改。而新的预分频比将在下一更新事件发生时被采用。

下图给出了在预分频器运行时，更改计数器参数的例子。

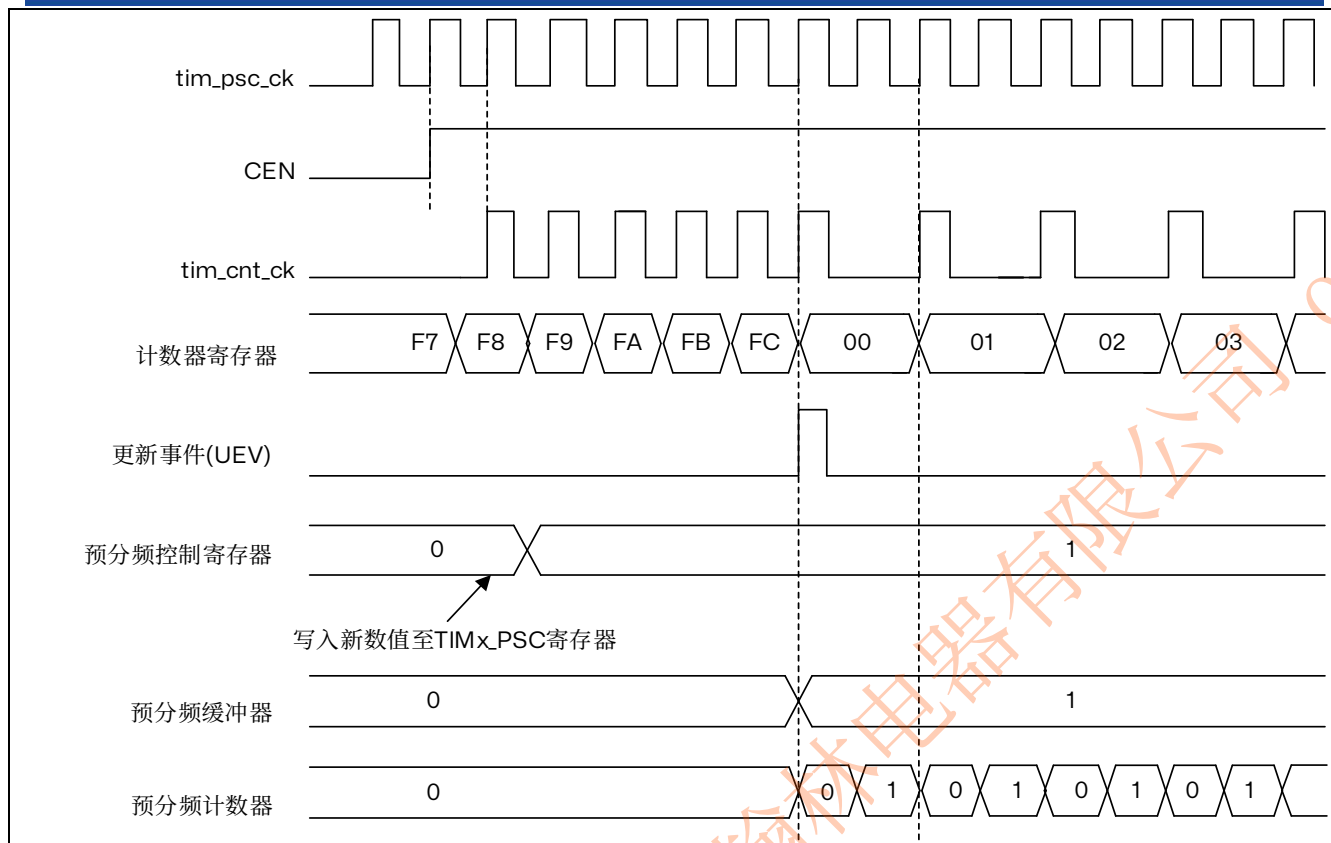


图 25.3 当预分频器的参数从 1 变到 2 时，计数器的时序图

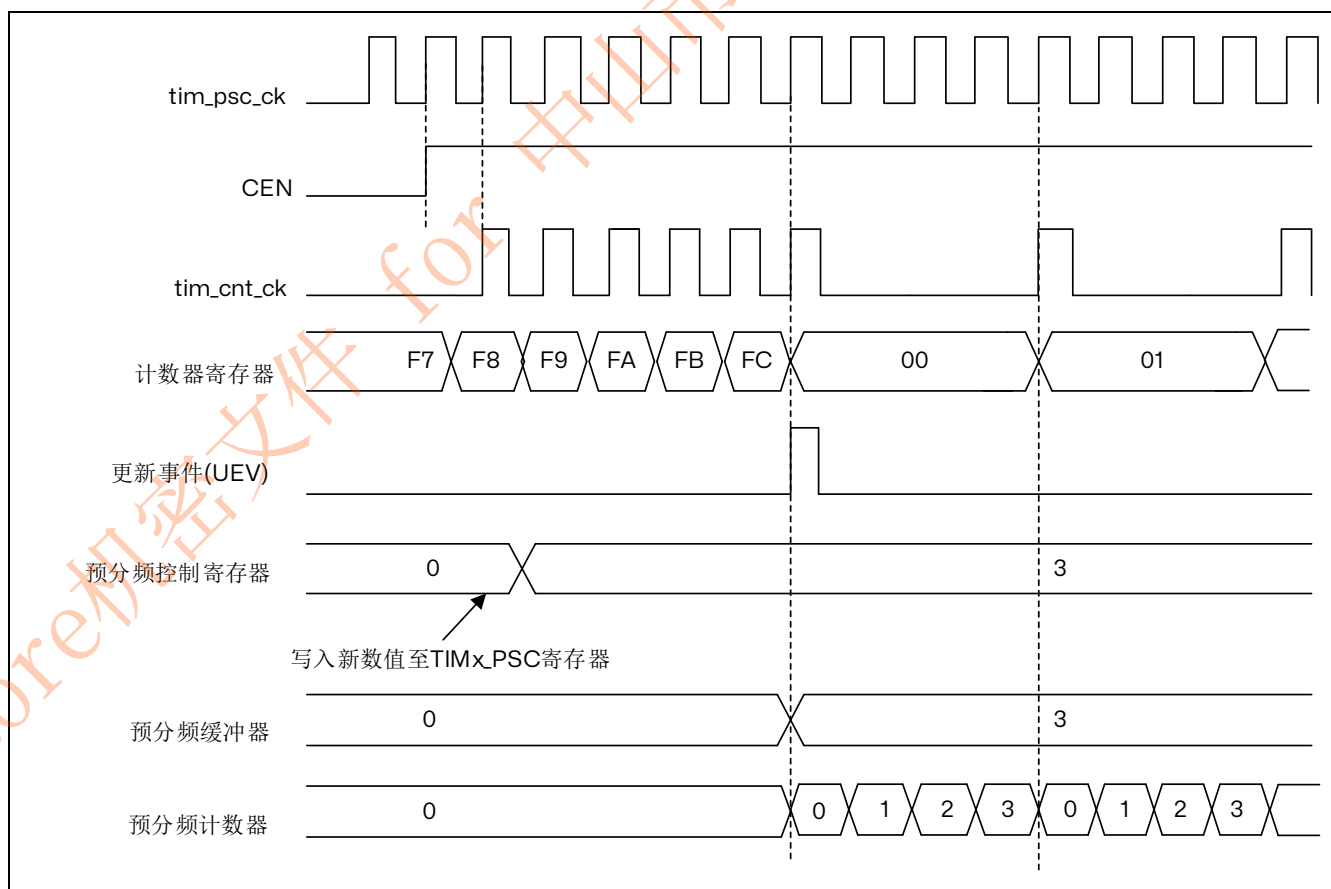


图 25.4 当预分频器的参数从 1 变到 4 时，计数器的时序图

## 25.4.4 计数器模式

### 递增计数模式

在递增计数模式下，计数器从 0 计数到自动重载值（TIMx\_ARR 寄存器的内容），然后重新从 0 开始计数并生成计数器上溢事件。

如果使用重复计数器，则当递增计数的重复次数达到重复计数器寄存器中编程的次数加一次 (TIMx\_RCR+1) 后，将生成更新事件 (UEV)。否则，将在每次计数器上溢时产生更新事件。

每次发生计数器上溢时会生成更新事件，或将 TIMx\_EGR 寄存器中的 UG 位置 1（通过软件或使用从模式控制器）也可以生成更新事件。

通过软件将 TIMx\_CR1 寄存器中的 UDIS 位置 1 可禁止 UEV 事件。这可避免向预装载寄存器写入新值时更新影子寄存器。在 UDIS 位写入 0 之前不会产生任何更新事件。不过，计数器和预分频器计数器都会重新从 0 开始计数（而预分频比保持不变）。此外，如果 TIMx\_CR1 寄存器中的 URS 位（更新请求选择）已置 1，则将 UG 位置 1 会生成更新事件 UEV，但不会将 UIF 标志置 1（因此，不会发送任何中断）。这样一来，如果在发生捕获事件时将计数器清零，将不会同时产生更新中断和捕获中断。

发生更新事件时，将更新所有寄存器且将更新标志（TIMx\_SR 寄存器中的 UIF 位）置 1（取决于 URS 位）：

- 重复计数器中将重新装载 TIMx\_RCR 寄存器的内容
- 自动重载影子寄存器将以预装载值进行更新（TIMx\_ARR）
- 预分频器的缓冲区中将重新装载预装载值（TIMx\_PSC 寄存器的内容）

以下各图以一些示例说明当 TIMx\_ARR=0x36 时不同时钟频率下计数器的行为。

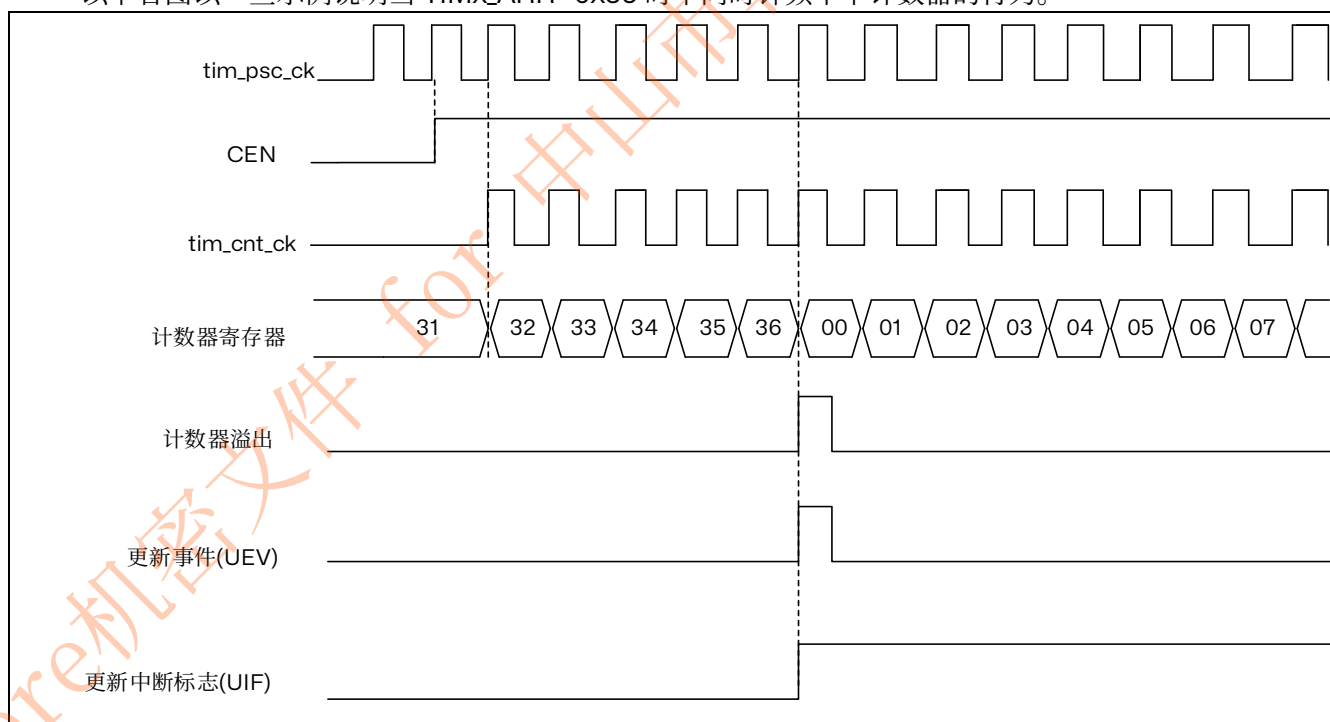


图 25.5 计数器时序图，内部时钟分频系数为 1

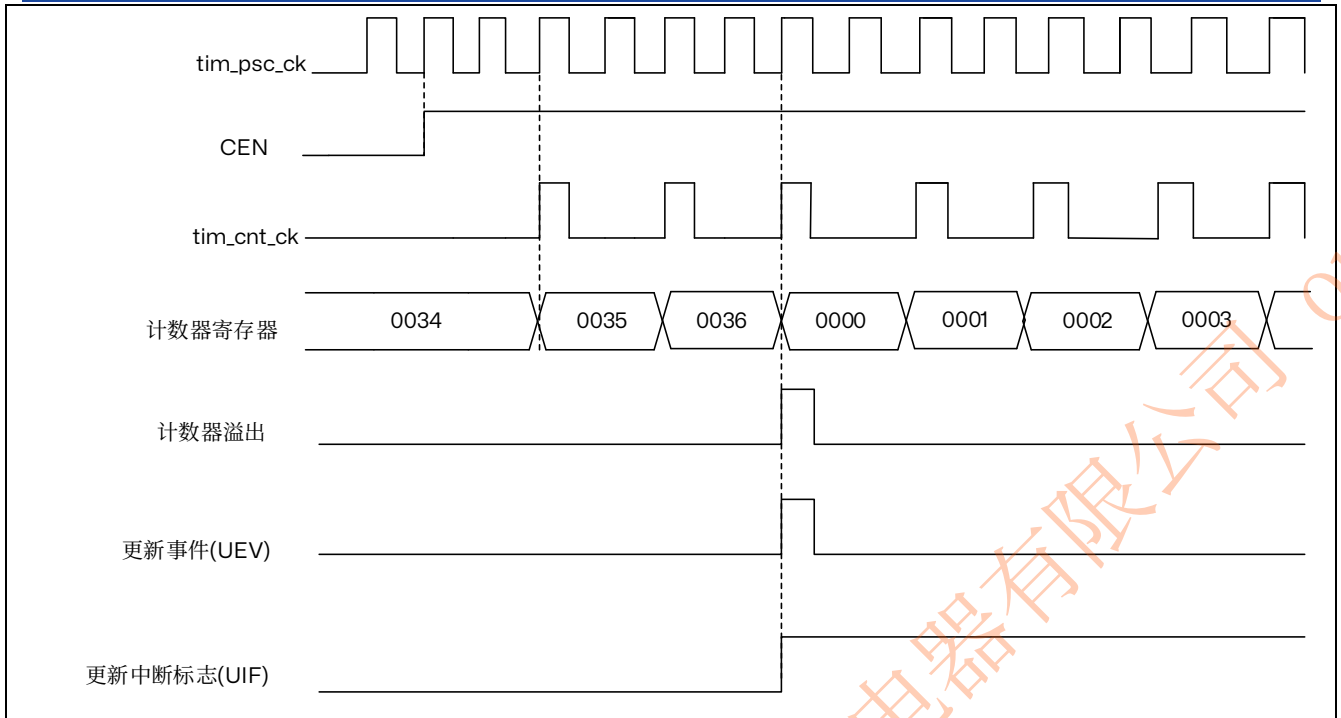


图 25.6 计数器时序图，内部时钟分频系数为 2

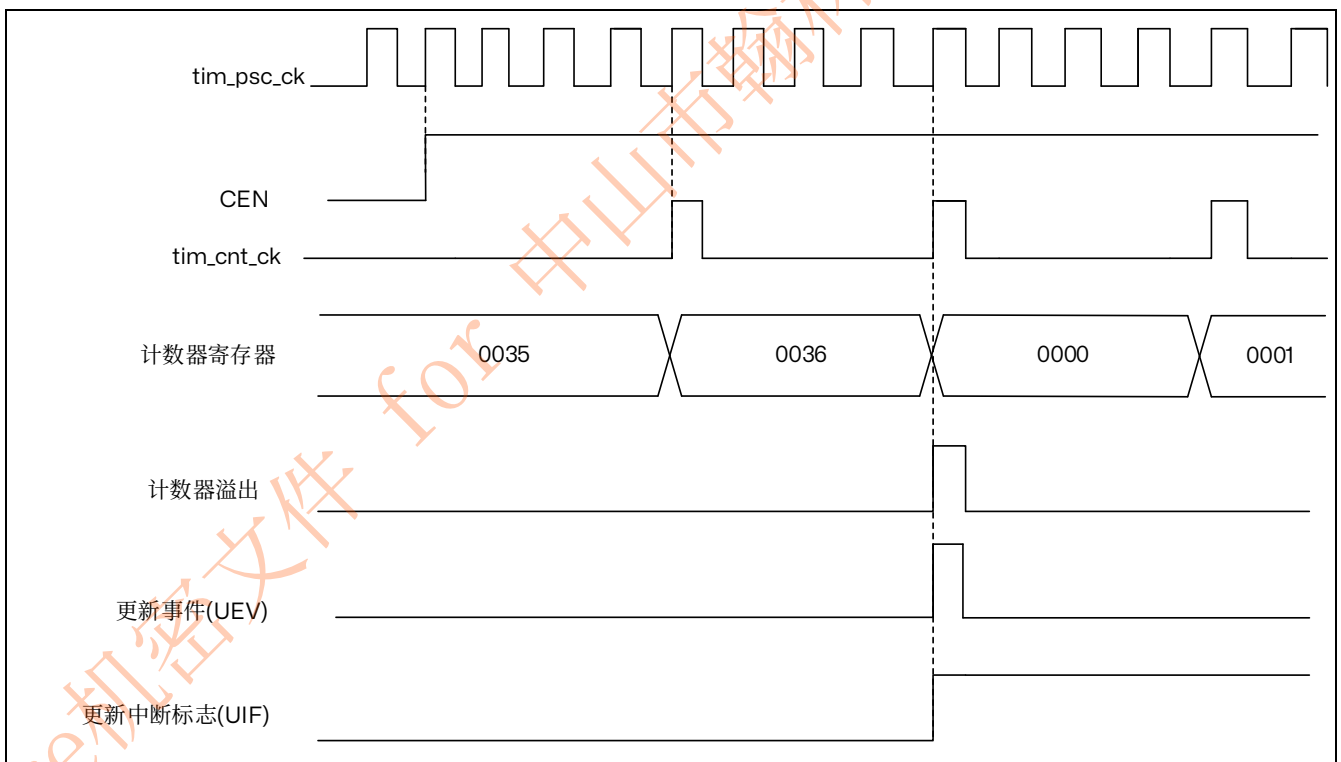


图 25.7 计数器时序图，内部时钟分频系数为 4

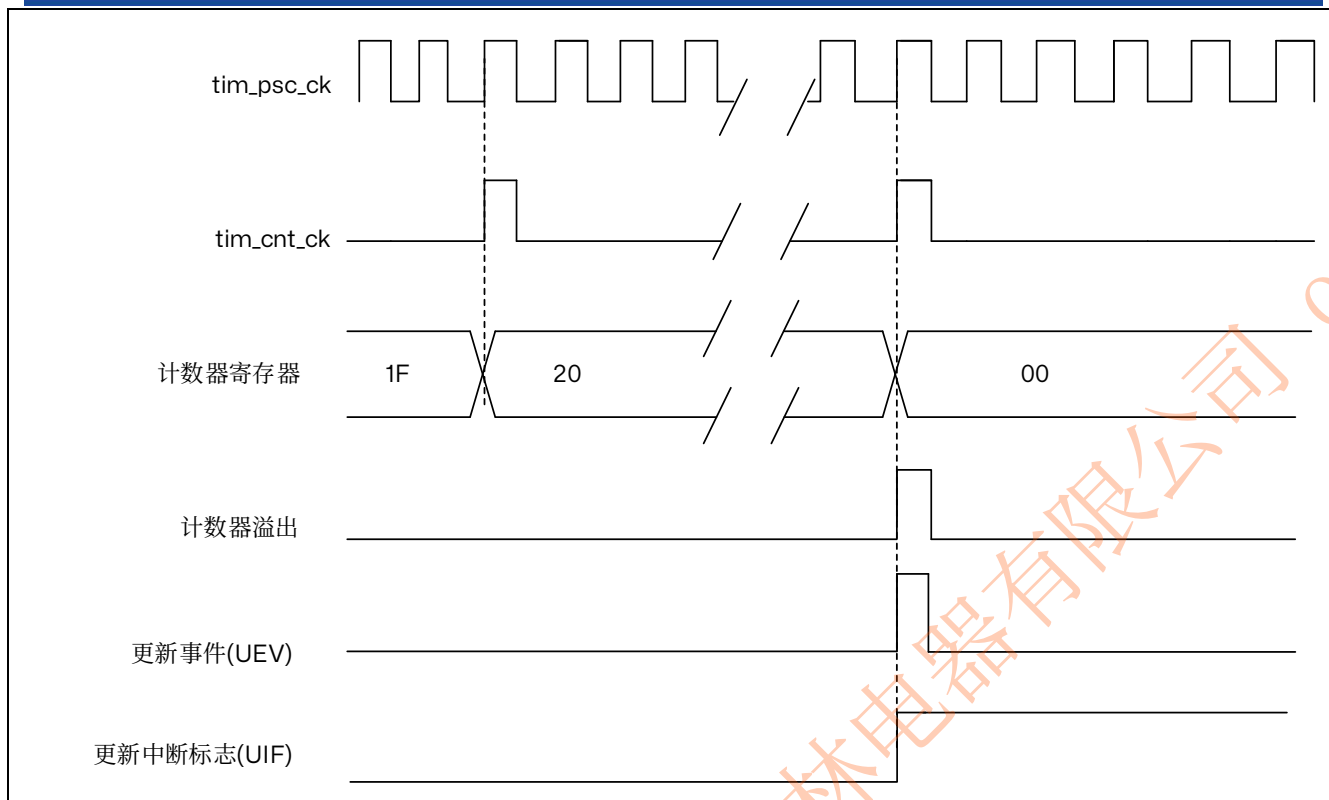


图 25.8 计数器时序图，内部时钟分频系数为 N

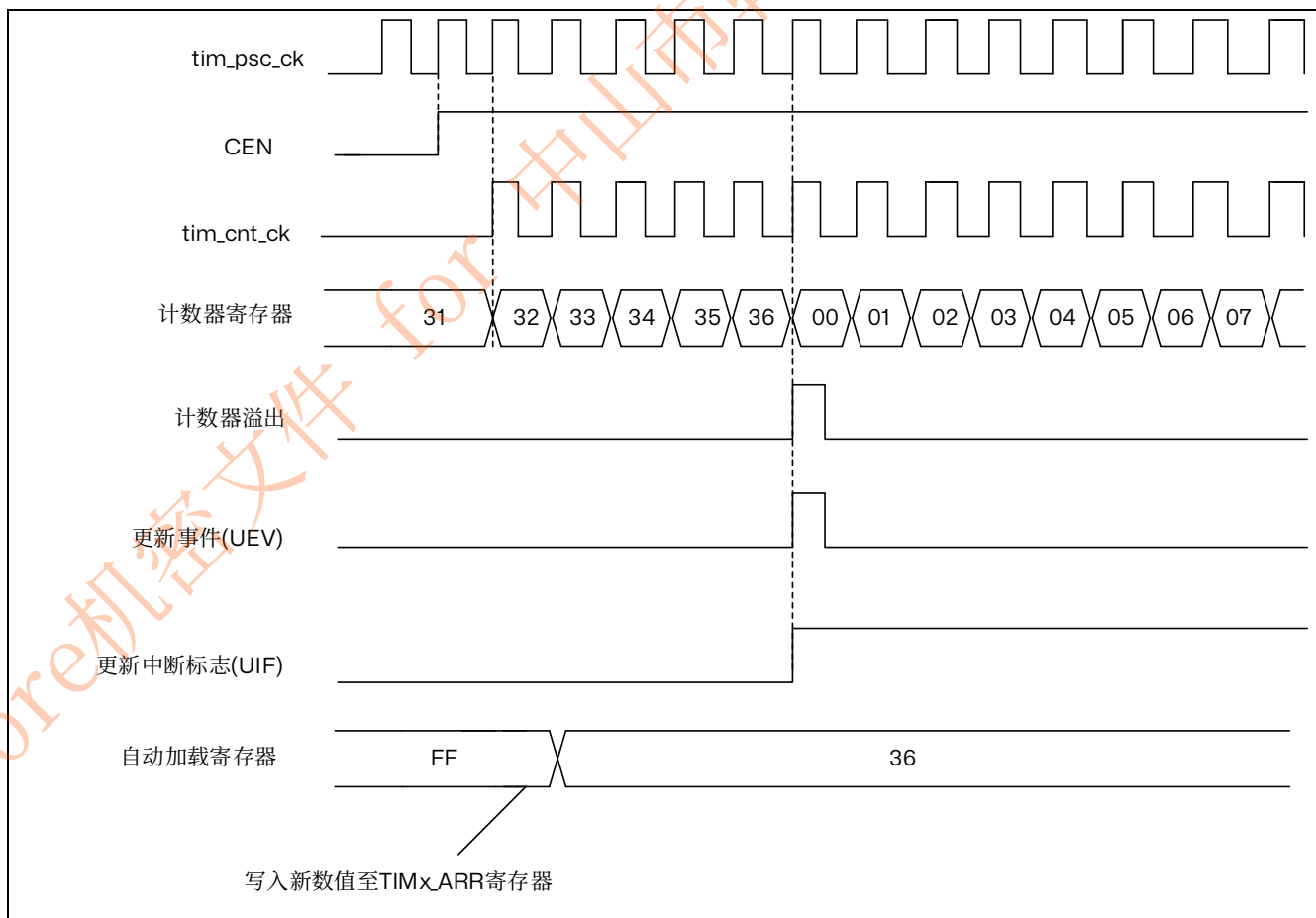


图 25.9 计数器时序图，当 ARPE=0 时的更新事件 (TIMx\_ARR 没有预装入)

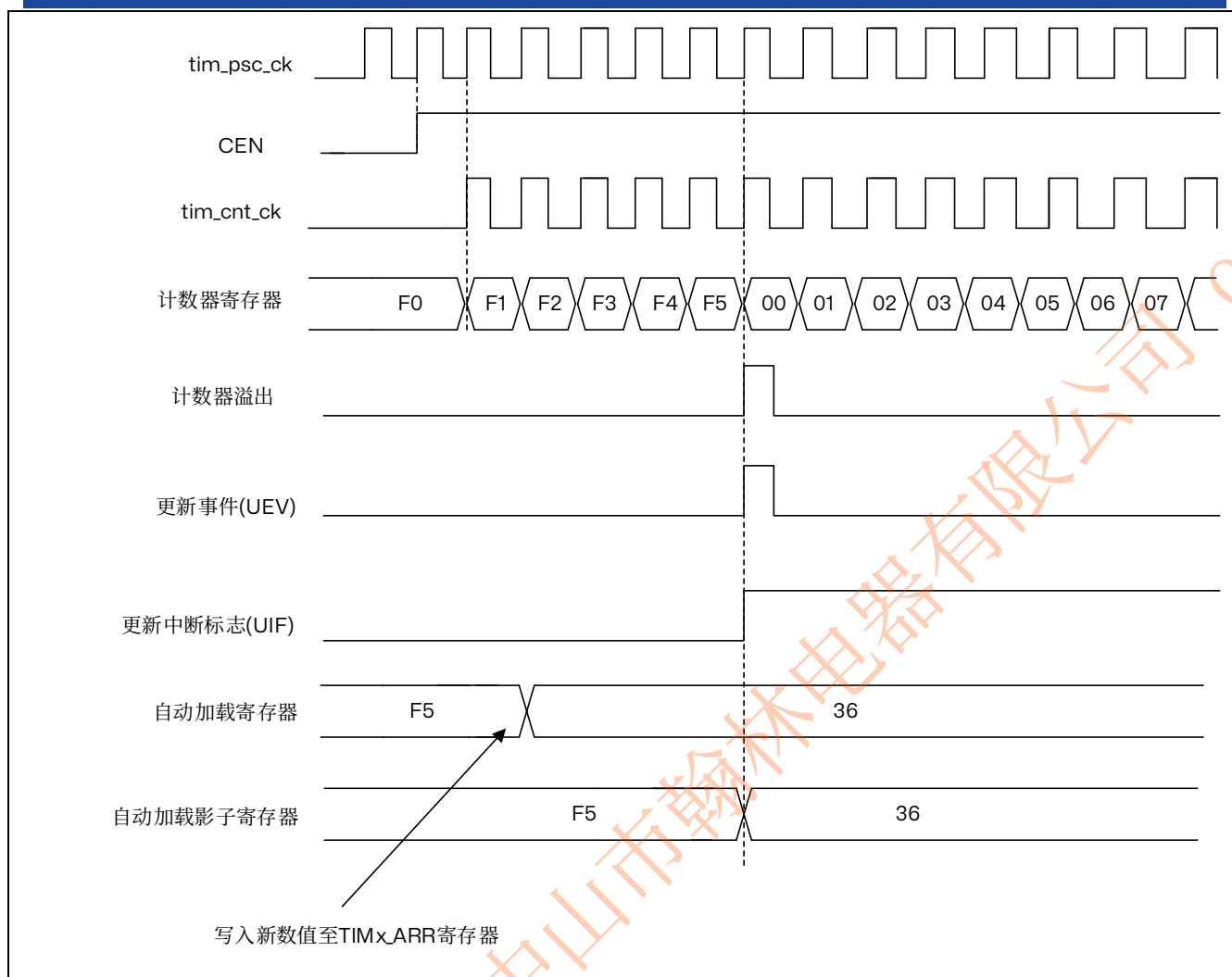


图 25.10 计数器时序图，当 ARPE=1 时的更新事件（预装入了 TIMx\_ARR）

### 25.4.5 重复计数器

时基单元介绍如何因计数器溢出而生成更新事件(UEV)。实际上，只有当重复计数器达到零时，才会生成更新事件。这在生成 PWM 信号时很有用。

这意味着，每当发生 N 个计数器溢出（其中，N 是 TIMx\_RCR 重复计数器寄存器中的值），数据就将从预装载寄存器转移到影子寄存器（TIMx\_ARR 自动重载寄存器、TIMx\_PSC 预分频器寄存器以及比较模式下的 TIMx\_CCRx 捕获/比较寄存器）。

重复计数器在每次计数器溢出时递减：

重复计数器是自动重载类型；其重复率为 TIMx\_RCR 寄存器所定义的值。当更新事件由软件（通过将 TIMx\_EGR 寄存器的 UG 位置 1）或硬件（通过从模式控制器）生成时，无论重复计数器的值为多少，更新事件都将立即发生，并且在重复计数器中重新装载 TIMx\_RCR 寄存器的内容。



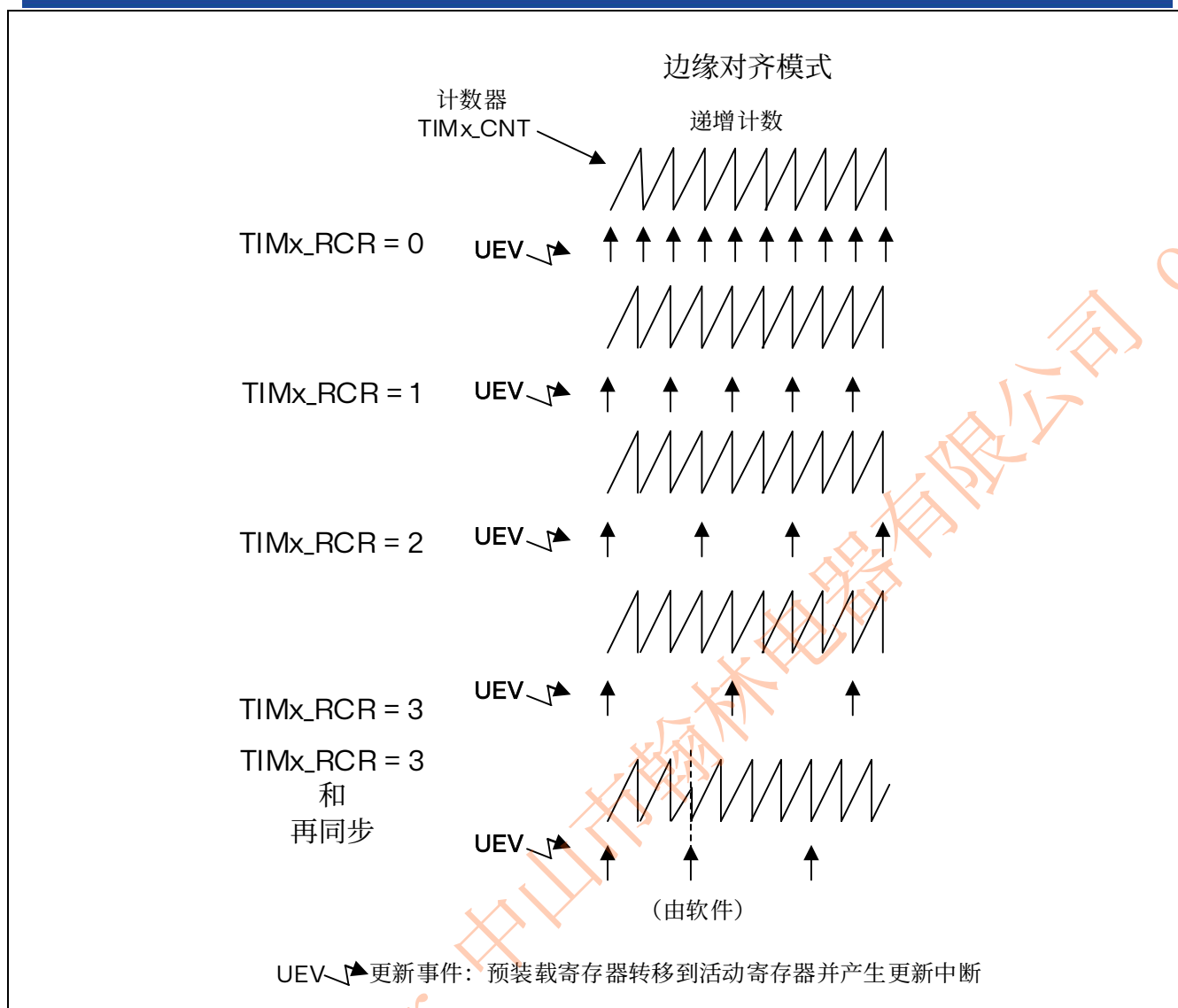


图 25.11 取决于模式和 TIMx\_RCR 寄存器设置的更新频率示例

## 25.4.6 时钟选择

计数器时钟可由下列时钟源提供：

- 内部时钟 (tim\_ker\_ck)
- 外部时钟模式 1：外部输入引脚 (tim\_ti1 or tim\_ti2，如果可用)
- 内部触发输入 (tim\_itrx)：使用一个定时器作为另一个定时器的预分频器，例如可以将 TIM1 配置为 TIM15 的预分频器。

### 内部时钟源 (tim\_ker\_ck)

如果禁止从模式控制器 (TIMx\_SMCR 寄存器中 SMS=000)，则 CEN 位、DIR 位 (TIMx\_CR1 寄存器中) 和 UG 位 (TIMx\_EGR 寄存器中) 为实际控制位，并且只能通过软件进行更改 (UG 除外，仍自动清零)。当对 CEN 位写入 1 时，预分频器的时钟就由内部时钟 tim\_ker\_ck 提供。

下图显示了正常模式下控制电路与递增计数器的行为 (没有预分频的情况下)。

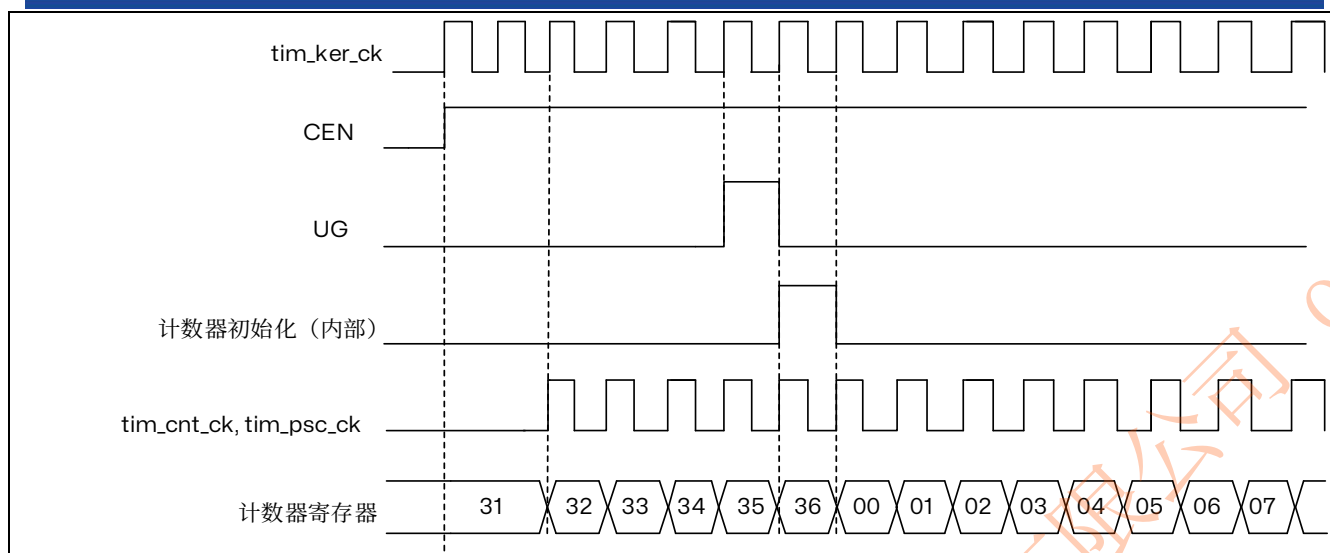


图 25.12 一般模式下的控制电路，内部时钟分频系数为 1

### 外部时钟源模式 1

当 TIMx\_SMCR 寄存器中的 SMS=111 时，可选择此模式。计数器可在选定的输入信号上出现上升沿或下降沿时计数。

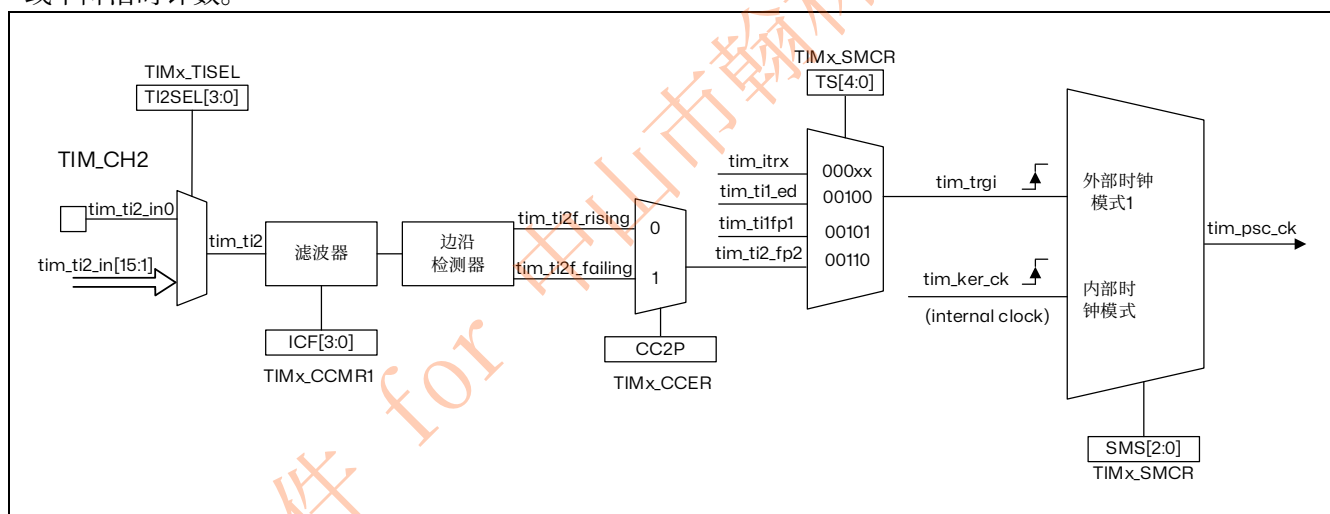


图 25.13 tim\_ti2 外部时钟连接例子

例如，要使递增计数器在 tim\_ti2 输入出现上升沿时计数，请执行以下步骤：

1. 通过在 TIMx\_TISEL 寄存器中使用 TI2SEL[3:0] 位选择合适的 tim\_ti2\_in[15:0] 源（内部或外部）。
2. 通过在 TIMx\_CCMR1 寄存器中写入 CC2S="01" 来配置通道 2，使其能够检测 tim\_ti2 输入的上升沿。
3. 通过在 TIMx\_CCMR1 寄存器中写入 IC2F[3:0] 位来配置输入滤波时间（如果不需要任何滤波，请保持 IC2F=0000）。
4. 通过在 TIMx\_CCER 寄存器中写入 CC2P=0 和 CC2NP=0 来选择上升沿极性。
5. 通过在 TIMx\_SMCR 寄存器中写入 SMS=111，使定时器在外部时钟模式 1 下工作。
6. 通过在 TIMx\_SMCR 寄存器中写入 TS=110 来选择 tim\_ti2 作为输入源。
7. 通过在 TIMx\_CR1 寄存器中写入 CEN=1 来使能计数器。

注：由于捕获预分频器不用于触发操作，因此无需对其进行配置。

当 tim\_ti2 出现上升沿时，计数器便会计数一次并且 TIF 标志置 1。

tim\_ti2 的上升沿与实际计数器时钟之间的延迟是由于 tim\_ti2 输入的重新同步电路引起的。

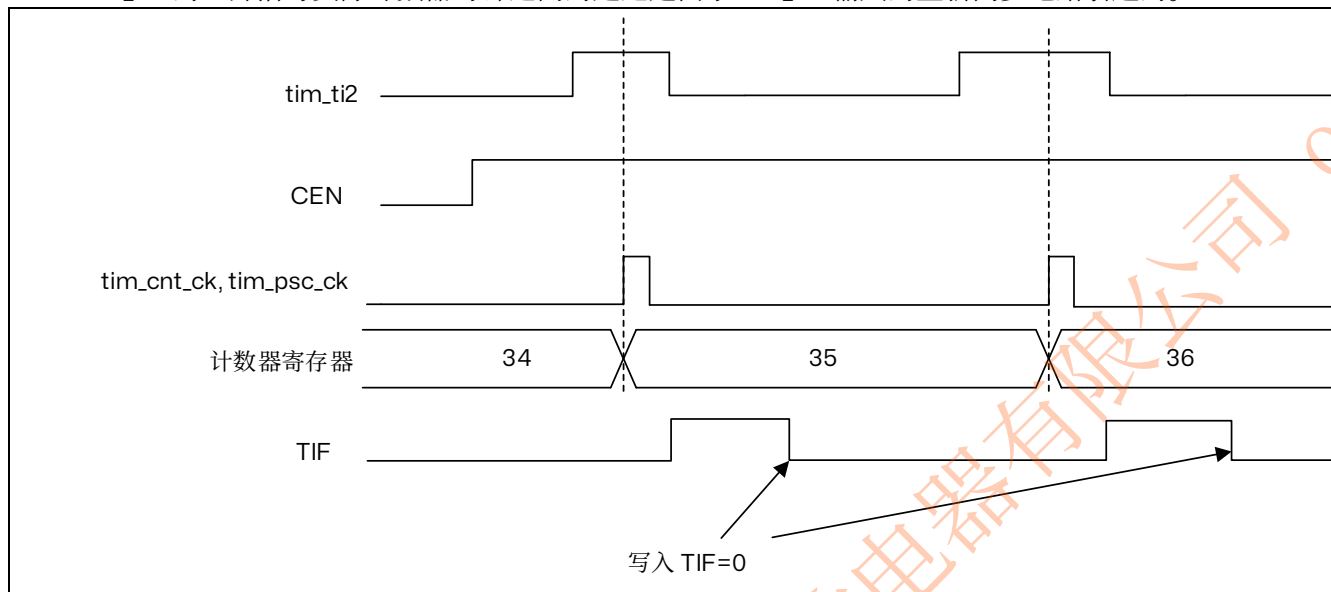


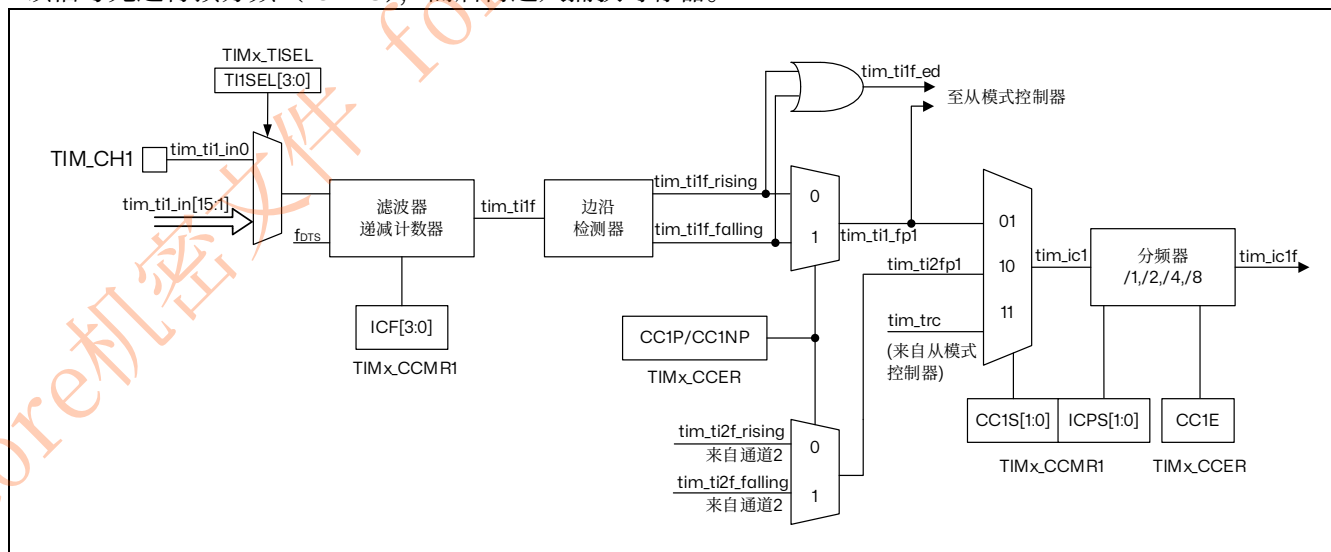
图 25.14 外部时钟模式 1 下的控制电路

### 25.4.7 捕获/比较通道

每个捕获/比较通道均围绕一个捕获/比较寄存器（包括一个影子寄存器）、一个捕获输入阶段（数字滤波、多路复用和预分频器）和一个输出阶段（比较器和输出控制）构建而成。

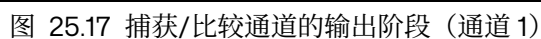
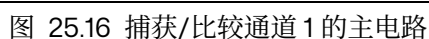
下图概括介绍了一个捕获/比较通道。

输入阶段对相应的 tim\_tix 输入进行采样，生成一个滤波后的信号 tim\_tixf。然后，带有极性选择功能的边沿检测器生成一个信号（tim\_tixfpy），该信号可用作从模式控制器的触发输入，也可用作捕获命令。该信号先进行预分频（ICxPS），而后再进入捕获寄存器。



输出阶段生成一个中间波形作为基准：OCxRef（高电平有效）。链的末端决定最终输出信号的极性。

图 25.15 捕获/比较通道（如：通道 1 输入部分）



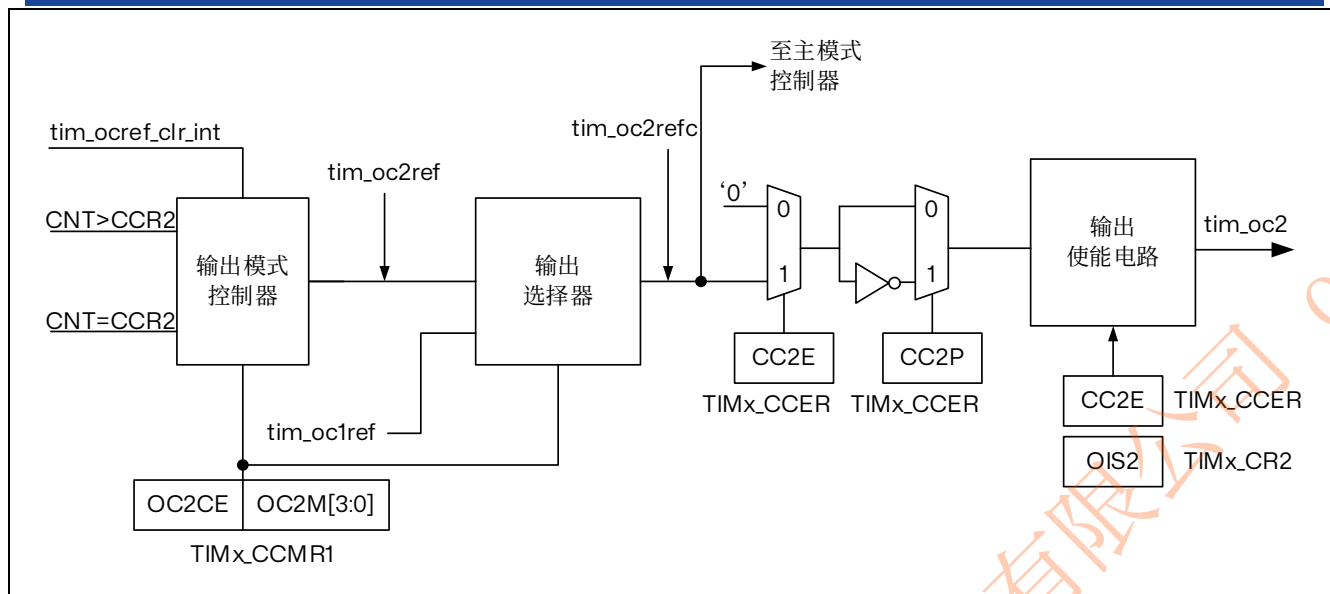


图 25.18 捕获比较通道的输出阶段 (TIM15 的通道 2)

捕获/比较模块由一个预装载寄存器和一个影子寄存器组成。始终可通过读写操作访问预装载寄存器。

在捕获模式下，捕获实际发生在影子寄存器中，然后将影子寄存器的内容复制到预装载寄存器中。

在比较模式下，预装载寄存器的内容将复制到影子寄存器中，然后将影子寄存器的内容与计数器进行比较。

### 25.4.8 输入捕获模式

在输入捕获模式下，当相应的 ICx 信号检测到跳变沿后，将使用捕获/比较寄存器 (TIMx\_CCRx) 来锁存计数器的值。发生捕获事件时，会将相应的 CCXIF 标志 (TIMx\_SR 寄存器) 置 1，并可发送中断或 DMA 请求 (如果已使能)。如果发生捕获事件时 CCxIF 标志已处于高位，则会将重复捕获标志 CCxOF (TIMx\_SR 寄存器) 置 1。可通过软件向 CCxIF 写入 0 来给 CCxIF 清零，或读取存储在 TIMx\_CCRx 寄存器中的已捕获数据。向 CCxOF 写入 0 后会将其清零。

以下示例说明了如何在 tim\_ti1 输入出现上升沿时将计数器的值捕获到 TIMx\_CCR1 中。具体操作步骤如下：

1. 通过在 TIMx\_TISEL 寄存器中使用 TI1SEL[3:0] 位选择合适的 tim\_tix\_in[15:1] 源 (内部或外部)。
2. 选择有效输入：TIMx\_CCR1 必须连接到 tim\_ti1 输入，因此向 TIMx\_CCMR1 寄存器中的 CC1S 位写入 01。只要 CC1S 不等于 00，就会将通道配置为输入模式，并且 TIMx\_CCR1 寄存器将处于只读状态。
3. 根据连接到定时器的信号，对所需的输入滤波时间进行编程 (如果输入为 tim\_tix 输入之一，则对 TIMx\_CCMRx 寄存器中的 ICxF 位进行编程)。假设信号变化时，输入信号最多在 5 个内部时钟周期内发生抖动。因此，我们必须将滤波时间设置为大于 5 个内部时钟周期。在检测到 8 个具有新电平的连续采样 (以 f<sub>OTS</sub> 频率采样) 后，可以确认 tim\_ti1 上的跳变沿。然后向 TIMx\_CCMR1 寄存器中的 IC1F 位写入 0011。
4. 通过向 TIMx\_CCER 寄存器中的 CC1P 位和 CC1NP 位写入 0，选择 tim\_ti1 通道的有效转换边沿 (本例中为上升沿)。
5. 对输入预分频器进行编程。在本例中，我们希望每次有效转换时都执行捕获操作，因此需要禁止

预分频器（向 TIMx\_CCMR1 寄存器中的 IC1PS 位写入 00）。

6. 通过将 TIMx\_CCER 寄存器中的 CC1E 位置 1，允许将计数器的值捕获到捕获寄存器中。
7. 如果需要，可通过将 TIMx\_DIER 寄存器中的 CC1IE 位置 1 来使能相关中断请求。

发生输入捕获时：

- 发生有效跳变沿时，TIMx\_CCR1 寄存器会获取计数器的值。
- 将 CC1IF 标志置 1（中断标志）。如果至少发生了两次连续捕获，但 CC1IF 标志未被清零，这样 CC1OF 捕获溢出标志会被置 1。
- 根据 CC1IE 位生成中断。
- 是否生成 DMA 请求取决于 CC1DE 位。

要处理重复捕获，建议在读出捕获溢出标志之前读取数据。这样可避免丢失在读取捕获溢出标志之后与读取数据之前可能出现的重复捕获信息。

注：通过软件将 TIMx\_EGR 寄存器中的相应 CCxG 位置 1 可生成 IC 中断请求。

#### 25.4.9 PWM 输入模式（仅 TIM15）

此模式允许测量连接到单个 tim\_tix 输入的 PWM 信号的周期和占空比：

- TIMx\_CCR1 寄存器保持周期值（两个连续上升沿之间的间隔）。
- TIMx\_CCR2 寄存器保持脉冲宽度（两个连续上升沿和下降沿之间的间隔）。

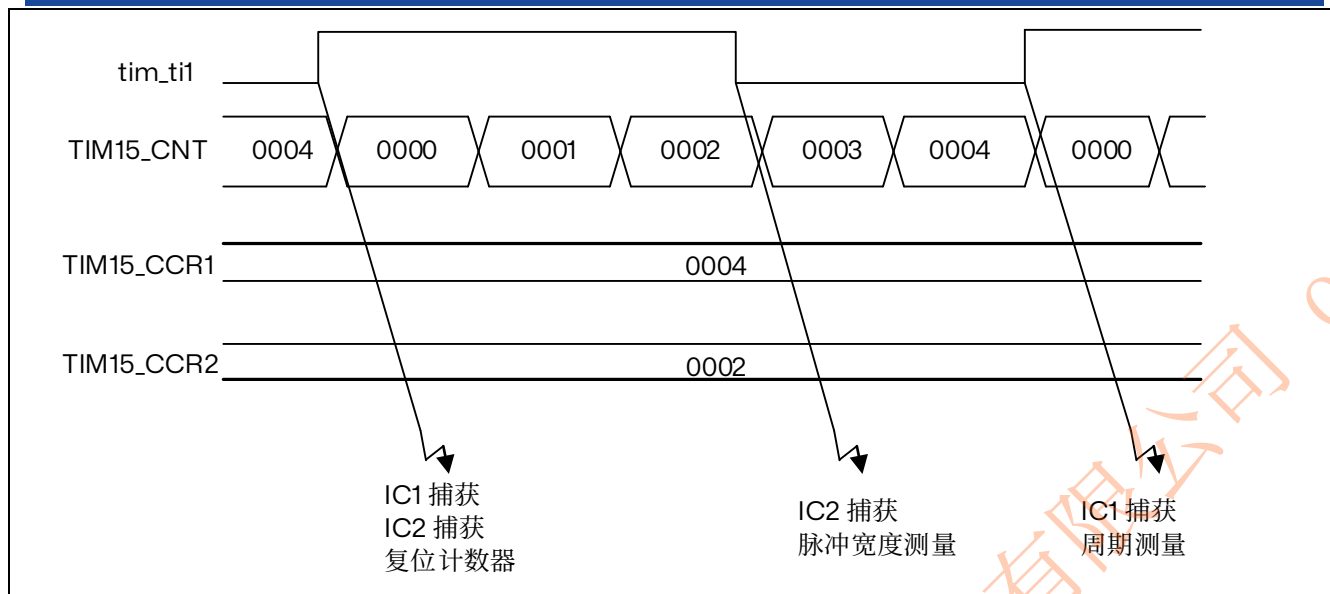
此模式是输入捕获模式的一个特例。其实现步骤与输入捕获模式基本相同，仅存在以下不同之处：

- 两个 tim\_icx 信号被映射至同一个 tim\_tix 输入。
- 这两个 tim\_icx 信号在边沿处有效，但极性相反。
- 选择两个 tim\_tixfp1 信号之一作为触发输入，并将从模式控制器配置为复位模式。

例如，可通过以下步骤测量施加在 tim\_ti1 上的 PWM 的周期（在 TIMx\_CCR1 寄存器中）和占空比（在 TIMx\_CCR2 寄存器中）（取决于 tim\_ker\_ck 频率和预分频器）：

1. 通过在 TIMx\_TISEL 寄存器中使用 TI1SEL[3:0] 位选择合适的 tim\_tix\_in[15:0] 源（内部或外部）。
2. 选择 TIMx\_CCR1 的有效输入：向 TIMx\_CCMR1 寄存器中的 CC1S 位写入 01（选择 tim\_ti1）。
3. 选择 tim\_ti1fp1 的有效极性（用于 TIMx\_CCR1 中的捕获和计数器清零）：向 CC1P 位和 CC1NP 位写入“0”（上升沿有效）。
4. 选择 TIMx\_CCR2 的有效输入：向 TIMx\_CCMR1 寄存器中的 CC2S 写入 10（选择 tim\_ti1）。
5. 选择 tim\_ti1fp2 的有效极性（用于 TIMx\_CCR2 中的捕获）：向 CC2P 位写入 1 和 CC2NP 位写入“0”（下降沿有效）。
6. 选择有效触发输入：向 TIMx\_SMCR 寄存器中的 TS 位写入 101（选择 tim\_ti1fp1）。
7. 将从模式控制器配置为复位模式：向 TIMx\_SMCR 寄存器中的 SMS 位写入 100。
8. 使能捕获：向 TIMx\_CCER 寄存器中的 CC1E 位和 CC2E 位写入“1”。





PWM 输入模式只能与 TIM15\_CH1/TIM15\_CH2 信号一起使用，因为只有 tim\_ti1fp1 和 tim\_ti2fp2 连接到从模式控制器。

图 25.19 PWM 输入模式时序

## 25.4.10 强制输出模式

在输出模式（TIMx\_CCMRx 寄存器中的 CCxS 位 = 00）下，可直接由软件将每个输出比较信号（tim\_ocxref 和 tim\_ocx）强制设置为有效电平或无效电平，而无需考虑输出比较寄存器和计数器之间的任何比较结果。

要将输出比较信号（tim\_ocxref/tim\_ocx）强制设置为有效电平，只需向相应 TIMx\_CCMRx 寄存器中的 OCxM 位写入 101。tim\_ocxref 进而强制设置为高电平（tim\_ocxref 始终为高电平有效），同时 tim\_ocx 获取 CCxP 极性位的相反值。

例如：CCxP=0（tim\_ocx 高电平有效）=> tim\_ocx 强制设置为高电平。

通过向 TIMx\_CCMRx 寄存器中的 OCxM 位写入 100，可将 tim\_ocxref 信号强制设置为低电平。

无论如何，TIMx\_CCRx 影子寄存器与计数器之间的比较仍会执行，而且允许将标志置 1。因此可发送相应的中断请求。输出比较模式一节对此进行了介绍。

## 25.4.11 输出比较模式

此功能用于控制输出波形，或指示已经过某一段时间。

当捕获/比较寄存器与计数器之间相匹配时，输出比较功能：

- 将为相应的输出引脚分配一个可编程值，该值由输出比较模式（TIMx\_CCMRx 寄存器中的 OCxM 位）和输出极性（TIMx\_CCER 寄存器中的 CCxP 位）定义。匹配时，输出引脚既可保持其电平（OCxM=000），也可设置为有效电平（OCxM=001）、无效电平（OCxM=010）或进行翻转（OCxM=011）。
- 将中断状态寄存器中的标志置 1（TIMx\_SR 寄存器中的 CCxIF 位）。
- 如果相应中断使能位（TIMx\_DIER 寄存器中的 CCXIE 位）置 1，将生成中断。

使用 TIMx\_CCMRx 寄存器中的 OCxPE 位，可将 TIMx\_CCRx 寄存器配置为带或不带预装载寄存器。

在输出比较模式下，更新事件 UEV 对 tim\_ocxref 和 tim\_ocx 输出毫无影响。同步的精度可以达到计



数器的一个计数周期。输出比较模式也可用于输出单脉冲（在单脉冲模式下）。

#### 步骤：

1. 选择计数器时钟（内部、外部、预分频器）。
2. 在 TIMx\_ARR 和 TIMx\_CCRx 寄存器中写入所需数据。
3. 如果要生成中断请求，将 CCxIE 位和/或 CCxDE 位置 1。
4. 选择输出模式。例如，
  - a) 写入 OCxM=0011 以在 CNT 匹配 CCRx 时翻转 tim\_ocx 输出引脚
  - b) 写入 OCxPE=0 以禁用预装载寄存器
  - c) 写入 CCxP=0 以选择高电平有效
  - d) 写入 CCxE=1 以使能输出
5. 通过将 TIMx\_CR1 寄存器中的 CEN 位置 1 来使能计数器。

可随时通过软件更新 TIMx\_CCRx 寄存器以控制输出波形，前提是未使能预装载寄存器（OCxPE=0，否则仅当发生下一个更新事件 UEV 时，才会更新 TIMx\_CCRx 影子寄存器）。

下图列出了相关示例。

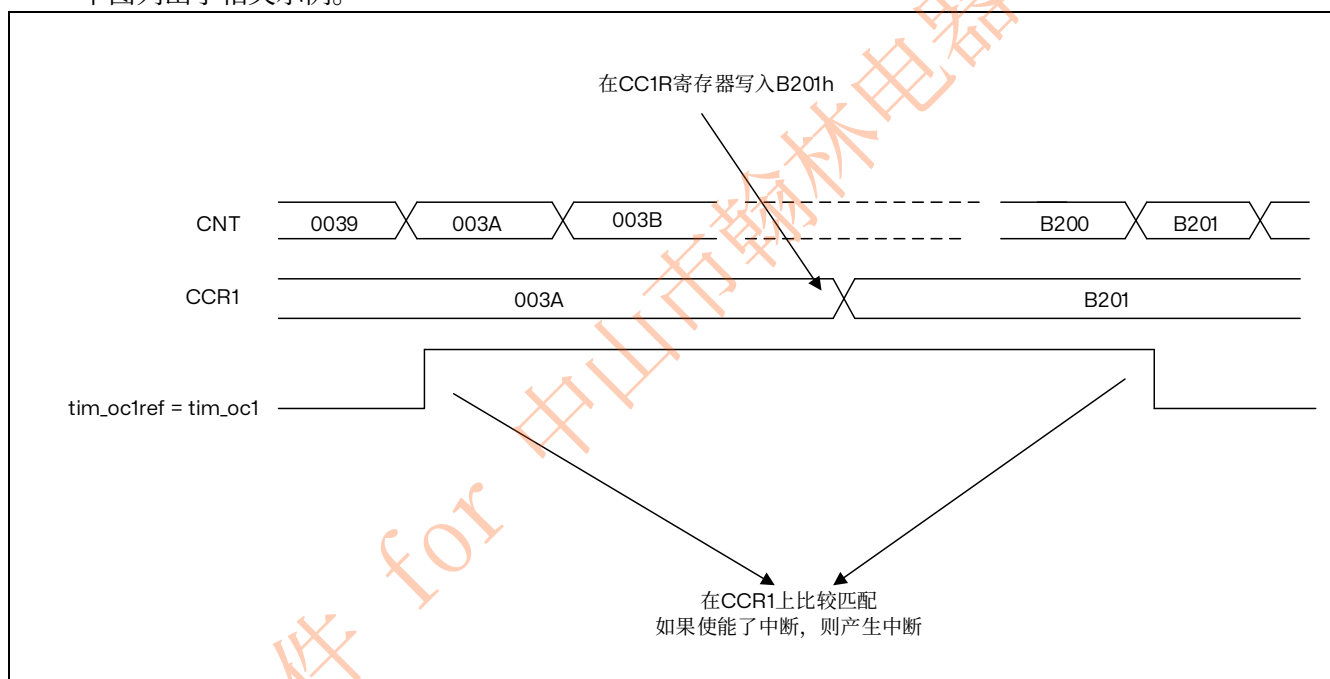


图 25.20 输出比较模式，翻转 tim\_oc1

### 25.4.12 PWM 模式

脉冲宽度调制模式可以生成一个信号，该信号频率由 TIMx\_ARR 寄存器值决定，其占空比则由 TIMx\_CCRx 寄存器值决定。

通过向 TIMx\_CCMRx 寄存器中的 OCxM 位写入 110（PWM 模式 1）或 111（PWM 模式 2），可以独立选择各通道（每个 tim\_ocx 输出对应一个 PWM）的 PWM 模式。必须通过将 TIMx\_CCMRx 寄存器中的 OCxPE 位置 1 使能相应预装载寄存器，最后通过将 TIMx\_CR1 寄存器中的 ARPE 位置 1 使能自动重载预装载寄存器。

由于只有在发生更新事件时预装载寄存器才会传送到影子寄存器，因此启动计数器之前，必须通过将 TIMx\_EGR 寄存器中的 UG 位置 1 来初始化所有寄存器。

tim\_ocx 极性可使用 TIMx\_CCER 寄存器的 CCxP 位来编程。既可以设为高电平有效，也可以设为低电平有效。通过 CCxE、CCxNE、MOE、OSSI 和 OSSR 位（TIMx\_CCER 和 TIMx\_BDTR 寄存器）的组合使能 tim\_ocx 输出。有关详细信息，请参见 TIMx\_CCER 寄存器说明。

在 PWM 模式（1 或 2）下，TIMx\_CNT 始终与 TIMx\_CCRx 进行比较，以确定是  $TIMx\_CCRx \leq TIMx\_CNT$  还是  $TIMx\_CNT \leq TIMx\_CCRx$ （取决于计数器计数方向）。

TIM15/TIM16/TIM17 只能递增计数，具体请参考递增计数模式。

以下以 PWM 模式 1 为例。只要  $TIMx\_CNT < TIMx\_CCRx$ ，PWM 参考信号 tim\_ocxref 便为高电平，否则为低电平。如果 TIMx\_CCRx 中的比较值大于自动重载值（TIMx\_ARR 中），则 tim\_ocxref 保持为“1”。如果比较值为 0，则 tim\_ocxref 保持为“0”。下图举例介绍边沿对齐模式的一些 PWM 波形（TIMx\_ARR=8）。

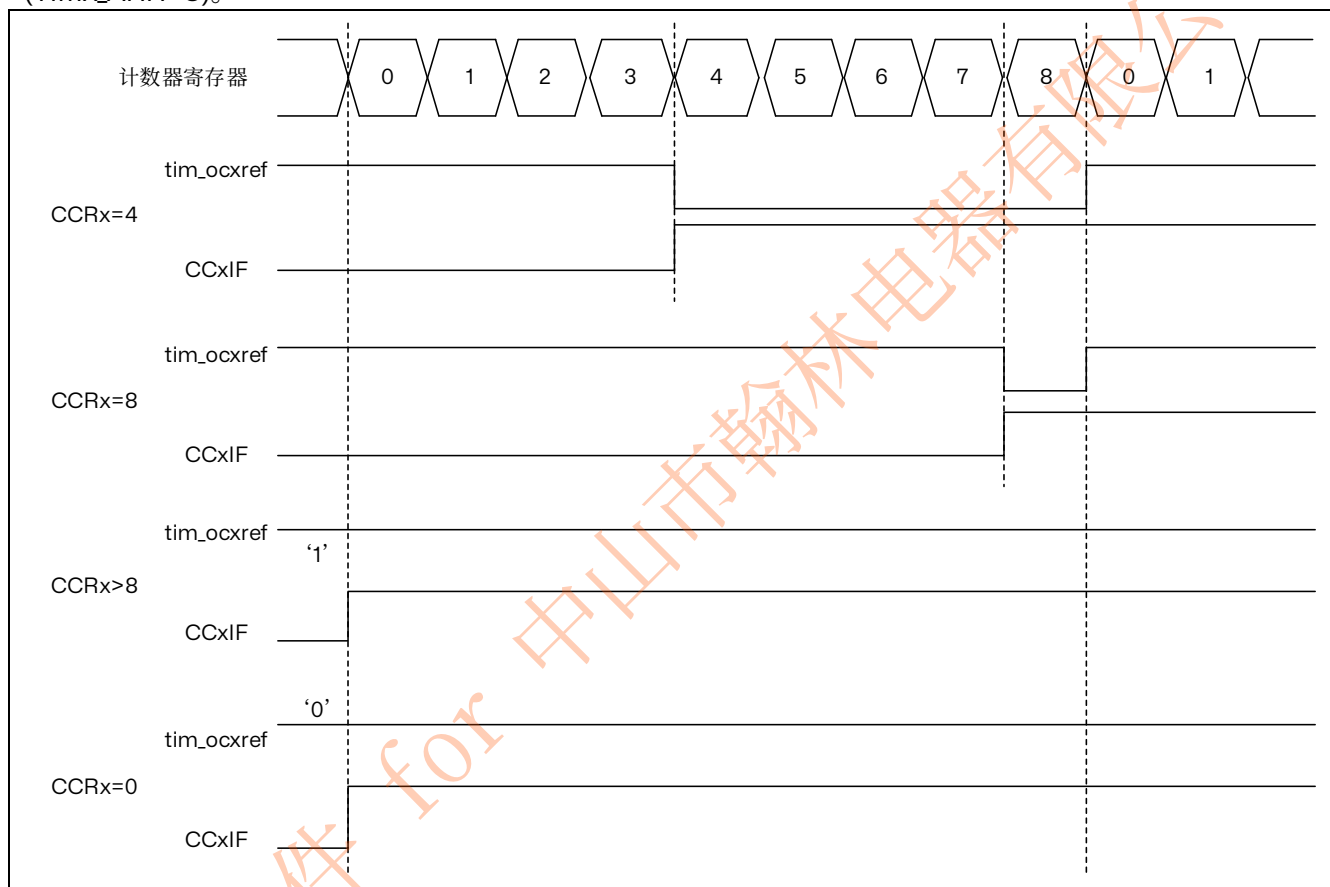


图 25.21 边沿对齐的 PWM 波形（ARR=8）

### 抖动模式

通过使能抖动模式，使用 TIMx\_CR1 寄存器中的 DITHEN 位，可以提高 PWM 模式的有效分辨率。这既适用于 CCR（用于增加占空比分辨率），也适用于 ARR（用于增加 PWM 频率分辨率）。

工作原理是在 16 个连续的 PWM 周期中，以预定义的模式使实际的 CCR（或 ARR）值略有变化（增加或减少一个定时器时钟周期）。这使得占空比或 PWM 期间的平均值提高了 16 倍分辨率。下图展示了应用于 4 个连续 PWM 周期的抖动原理。

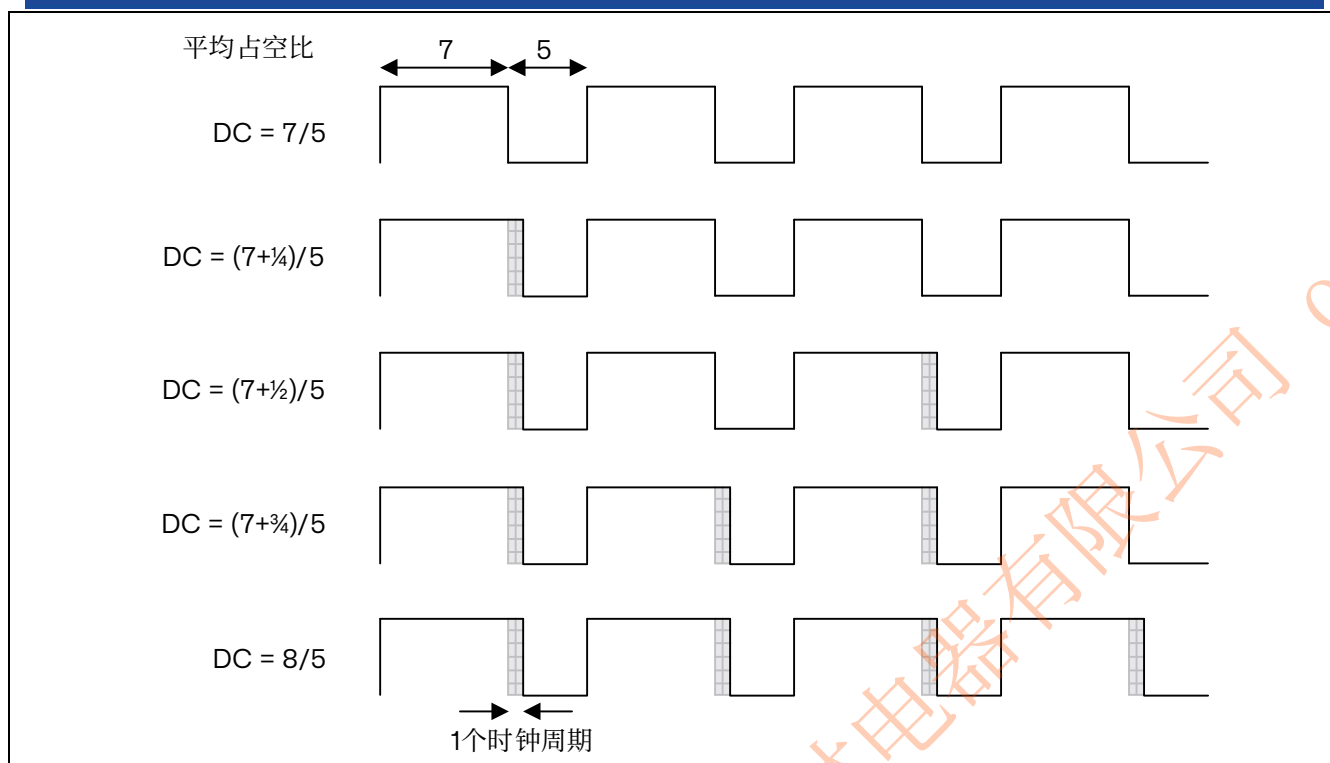


图 25.22 抖动原理

当抖动模式被使能时，寄存器编码将如下所示进行变化（参见下图示例）

- 4个最低有效位（LSB）用于编码增强的分辨率部分（小数部分）
- 最高有效位（MSB）左移至 19:4 位，用于编码基本值

注意：如果设置/清除/DITHEN 位，ARR 和 CCR 值将自动更新。（例如，如果 ARR=0x05 且 DITHEN=0，它将更新为 ARR=0x50 且 DITHEN=1）。

1. 必须重置 CEN 和 ARPE 位
2. ARR[3:0]位必须重置
3. DITHEN 位必须重置
4. CCIF 标志必须清除
5. 可以设置 CEN 位（最终设置 ARPE=1）。

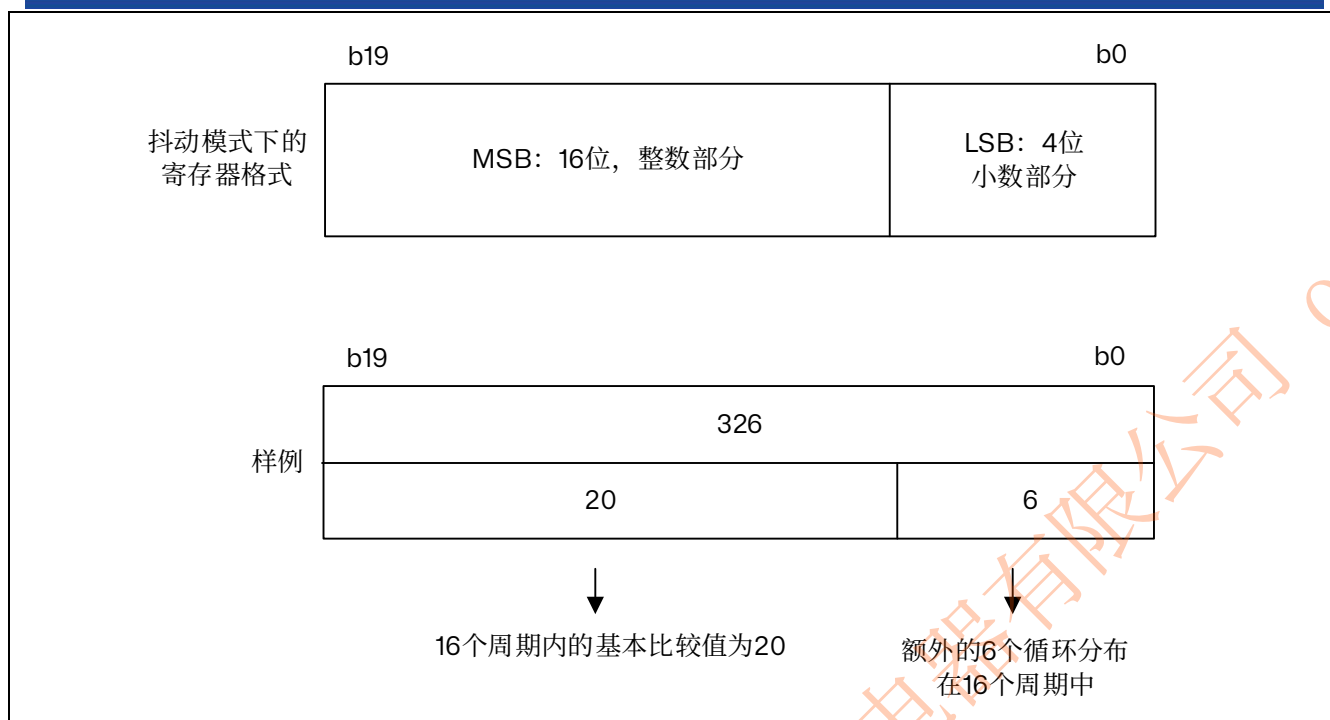


图 25.23 抖动模式下的数据格式和寄存器编码

最小频率由以下公式给出：

$$\text{分辨率} = \frac{F_{Tim}}{F_{pwm}} \Rightarrow F_{pwmMin} = \frac{F_{Tim}}{\text{最大分辨率}}$$

$$\text{禁用抖动模式: } F_{pwmMin} = \frac{F_{Tim}}{65536}$$

$$\text{使能抖动模式: } F_{pwmMin} = \frac{F_{Tim}}{65535 + \frac{15}{16}}$$

注意：在抖动模式下，TIMx\_ARR 和 TIMx\_CCRy 的最大值限制为 0xFFFFEF（对应于整数部分的 65534 和抖动部分的 15）。

如下图所示，无论 PWM 频率如何，抖动模式都可以增加 PWM 分辨率。

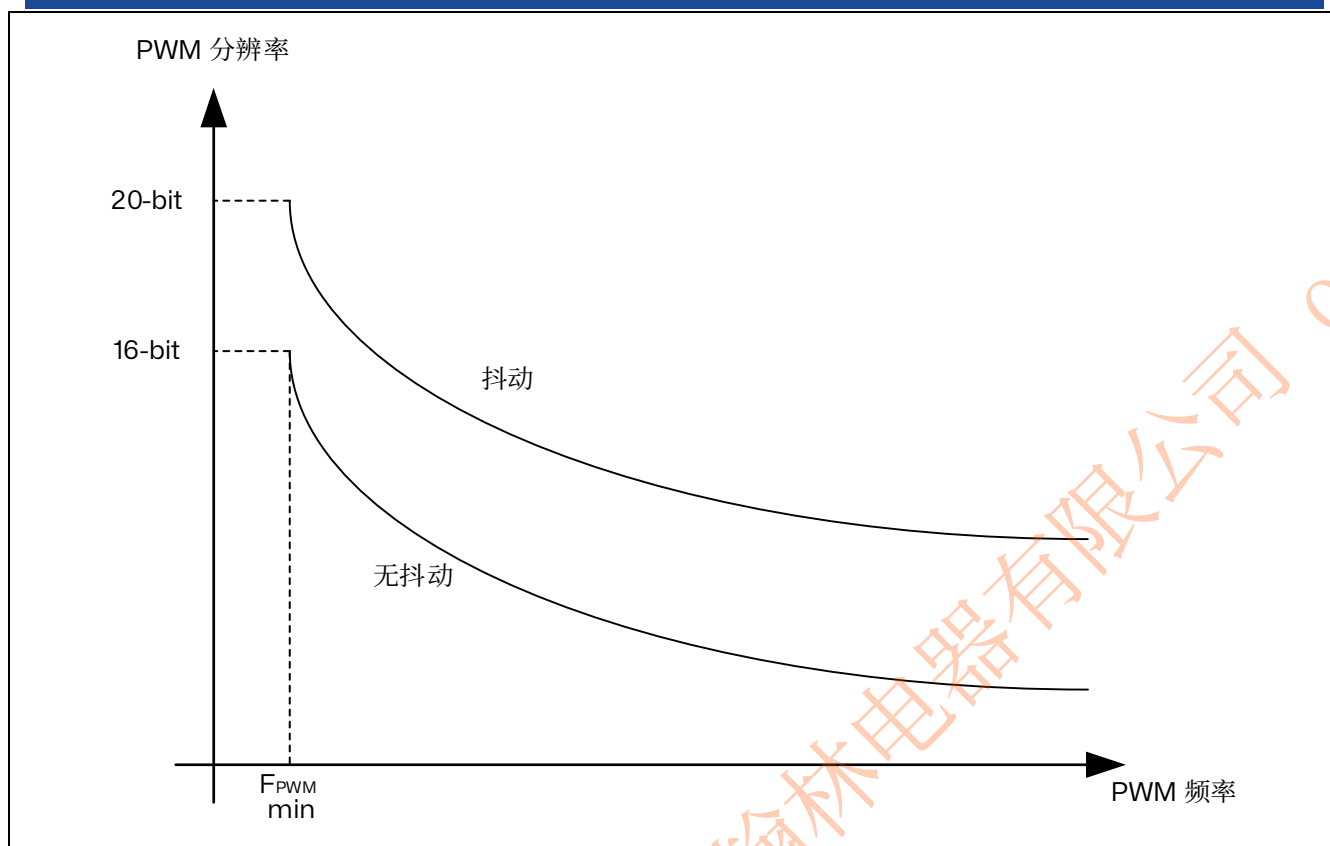


图 25.24 PWM 分辨率 vs 频率

如下图所示，占空比和/或周期的变化分散在 16 个连续周期。

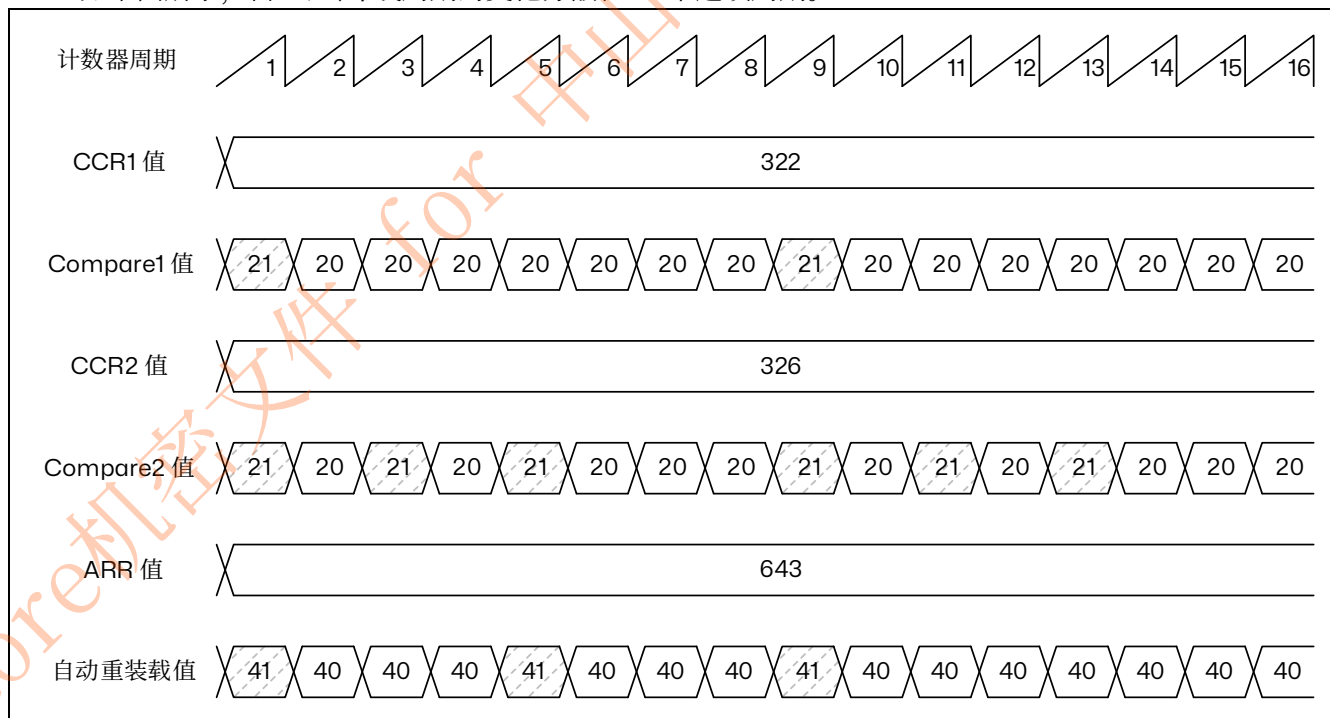


图 25.25 PWM 抖动模式

自动重新加载和比较值的增量按照下表中描述的特定模式分布。抖动序列是为了使增量分布尽可能均

匀并最大限度地减少整体波动。

表 25.9 CCR 和 ARR 寄存器的变化抖动模式

| LSB 值 | PWM 周期 |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------|--------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
|       | 1      | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| 0000  | -      | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  |
| 0001  | +1     | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  |
| 0010  | +1     | -  | -  | -  | -  | -  | -  | -  | +1 | -  | -  | -  | -  | -  | -  | -  |
| 0011  | +1     | -  | -  | -  | +1 | -  | -  | -  | +1 | -  | -  | -  | -  | -  | -  | -  |
| 0100  | +1     | -  | -  | -  | +1 | -  | -  | -  | +1 | -  | -  | -  | +1 | -  | -  | -  |
| 0101  | +1     | -  | +1 | -  | +1 | -  | -  | -  | +1 | -  | -  | -  | +1 | -  | -  | -  |
| 0110  | +1     | -  | +1 | -  | +1 | -  | -  | -  | +1 | -  | +1 | -  | +1 | -  | -  | -  |
| 0111  | +1     | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | -  | -  |
| 1000  | +1     | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  |
| 1001  | +1     | +1 | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  |
| 1010  | +1     | +1 | +1 | -  | +1 | -  | +1 | -  | +1 | +1 | +1 | -  | +1 | -  | +1 | -  |
| 1011  | +1     | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | -  | +1 | -  |
| 1100  | +1     | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  |
| 1101  | +1     | +1 | +1 | +1 | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  |
| 1110  | +1     | +1 | +1 | +1 | +1 | +1 | +1 | -  | +1 | +1 | +1 | +1 | +1 | +1 | +1 | -  |
| 1111  | +1     | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | -  |

### 25.4.13 组合 PWM 模式（仅 TIM15）

组合 PWM 模式允许生成两个边缘对齐或中心对齐的 PWM 信号，各自脉冲之间的可编程延迟和相位偏移。虽然频率由 TIMx\_ARR 寄存器的值决定，但占空比和延迟由两个 TIMx\_CCRx 寄存器确定。结果信号 tim\_ocxrefc 由两个参考 PWM 的 OR 或 AND 逻辑组合而成：

— tim\_oc1refc（或 tim\_oc2refc）由 TIMx\_CCR1 和 TIMx\_CCR2 控制

通过在 TIMx\_CCMRx 寄存器的 OCxM 位中写入“1100”（组合 PWM 模式 1）或“1101”（组合 PWM 模式 2），可以在两个通道上独立选组合 PWM 模式（每个 CCR 寄存器对一个 tim\_ocx 输出）。

当某个通道用作组合 PWM 通道时，其互补通道必须配置为相反的 PWM 模式（例如，一个通道为组合 PWM 模式 1，另一个通道为组合 PWM 模式 2）。

注意：为了兼容性原因，OCxM[3:0] 位域被分为两个部分，最高有效位与最低 3 位不连续。

下图表示使用组合 PWM 模式生成信号的示例，该示例的配置如下：

- 通道 1 配置为组合 PWM 模式 2。
- 通道 2 配置为 PWM 模式 1。

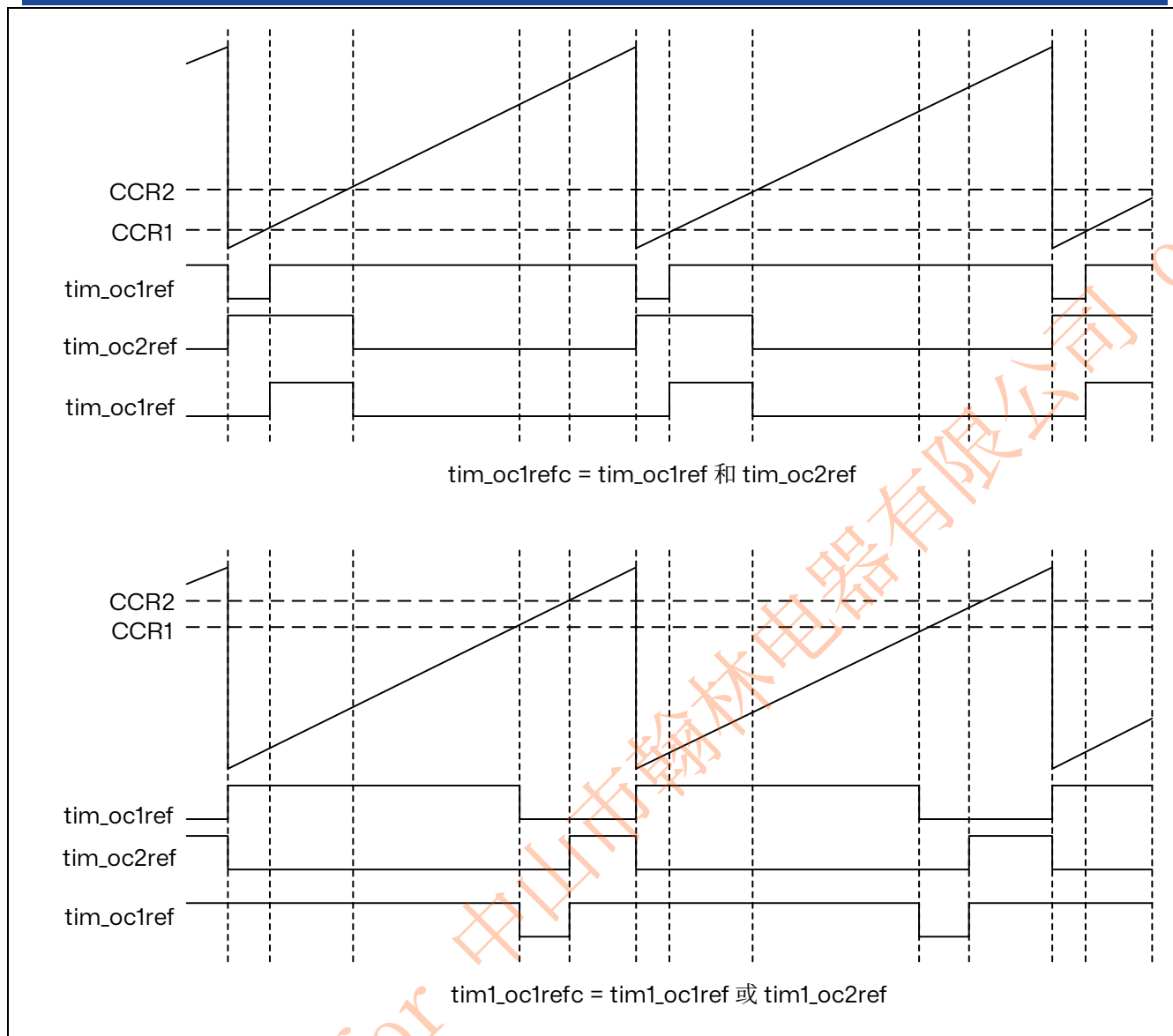


图 25.26 通道 1 和通道 2 的组合 PWM 模式

#### 25.4.14 互补输出和死区插入

通用定时器 (TIM15/TIM16/TIM17) 可以输出一路互补信号，并管理输出的关断与接通瞬间。

这段时间通常称为死区，用户必须根据与输出相连接的器件及其特性（电平转换器的固有延迟、开关器件产生的延迟...）来调整死区时间

每路输出可以独立选择输出极性（主输出 tim\_ocx 或互补输出 tim\_ocxn）。可通过对 TIMx\_CCER 寄存器中的 CCxP 和 CCxNP 位执行写操作来完成极性选择。

互补信号 tim\_ocx 和 tim\_ocxn 通过以下多个控制位的组合进行激活：TIMx\_CCER 寄存器中的 CCxE 和 CCxNE 位以及 TIMx\_BDTR 和 TIMx\_CR2 寄存器中的 MOE、OISx、OISxN、OSSI 和 OSSR 位。更多详细信息，请参考表格：带刹车功能的互补输出通道 tim\_oc1 和 tim\_oc1n 的控制位 (TIM16/TIM17)。应当注意，切换至 idle (MOE 下降到 0) 的时刻，死区仍然有效。

CCxE 和 CCxNE 位同时置 1 并且 MOE 位置 1 (如果存在刹车) 时，将使能死区插入。每个通道都有一个 10 位的死区时间生成器，将基于参考波形 tim\_ocxref 生成 2 个输出 tim\_ocx 和 tim\_ocxn。如果



tim\_ocx 和 tim\_ocxn 为高电平有效：

- 输出信号 OCx 与参考信号相同，只是其上升沿相对参考上升沿存在延迟。
- 输出信号 OCxN 与参考信号相反，并且其上升沿相对参考下降沿存在延迟。

如果延迟时间大于有效输出（OCx 或 OCxN）的宽度，则不会产生相应的脉冲。

下图所示为死区发生器的输出信号与参考信号 OCxREF 之间的关系。（在这些示例中，假定

CCxP=0、CCxNP=0、MOE=1、CCxE=1 并且 CCxNE=1）

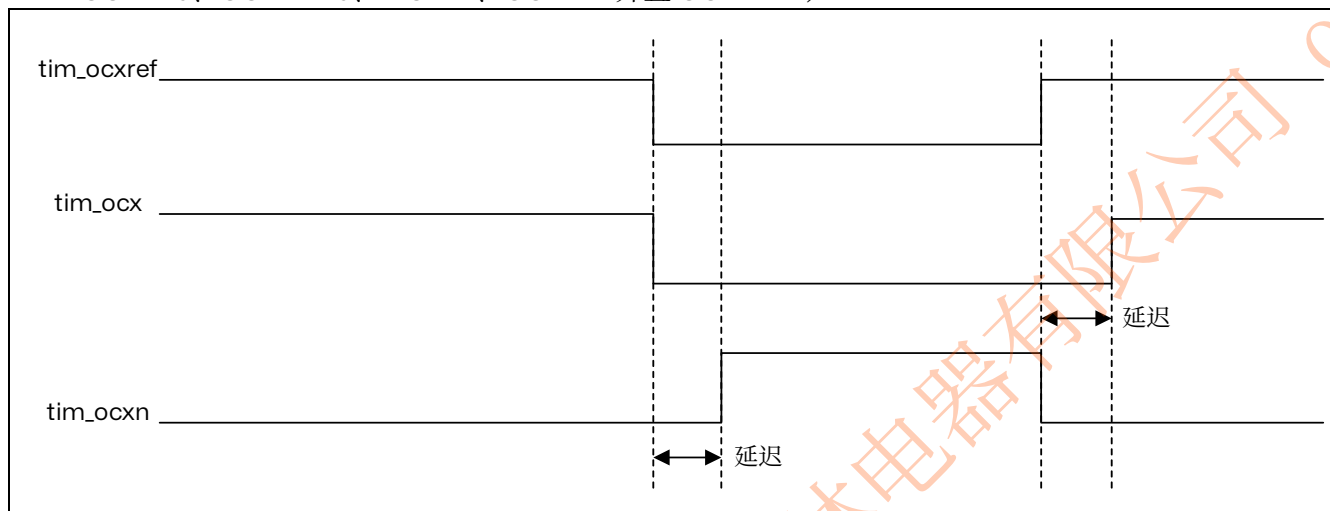


图 25.27 带死区插入的互补输出

TIMx\_DTR2 中的 DTAE 位允许对参考信号上升沿和下降沿的死区值进行区分，如下图所示。

在非对称模式（DTAE=1）下，上升沿参考死区由 TIMx\_BDTR 寄存器中的 DTG[7:0]位域定义，而下降沿参考死区由 TIMx\_DTR2 寄存器中的 DTGF[7:0]位域定义。在使能计数器之前，必须写入 DTAE 位，并且在 CEN=1 时不得修改。

使用预加载机制可以在 PWM 操作期间动态更新死区值。当 TIMx\_DTR2 寄存器中的 DTPE 位被设置时，死区位域 DTG[7:0]和 DTGF[7:0]将被预加载。预加载值将在下一个更新事件上加载到活动寄存器中。

注：如果 DTPE 位在计数器使能时被使能。自上次更新以来写入的新值将被丢弃，并使用先前的值。

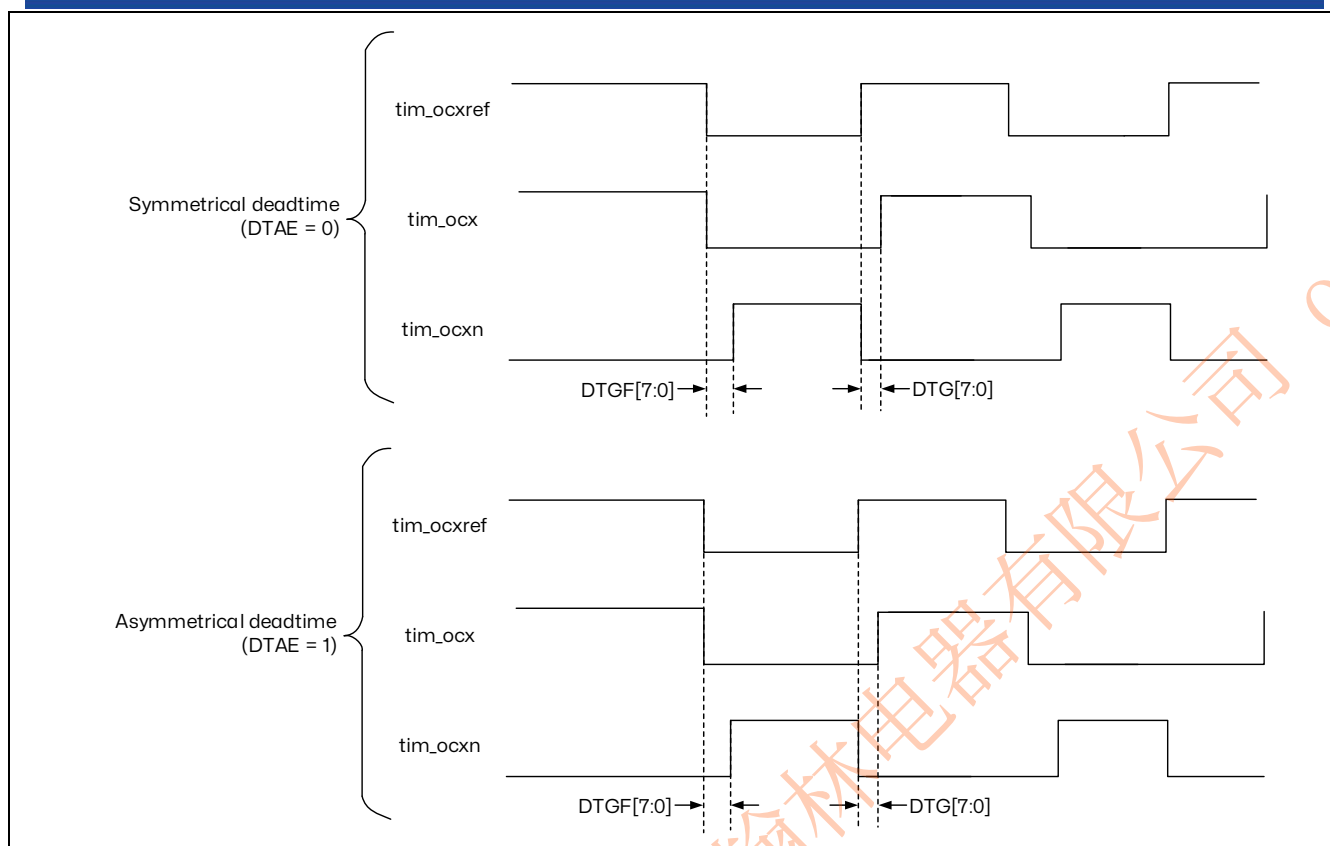


图 25.28 非对称死区时间

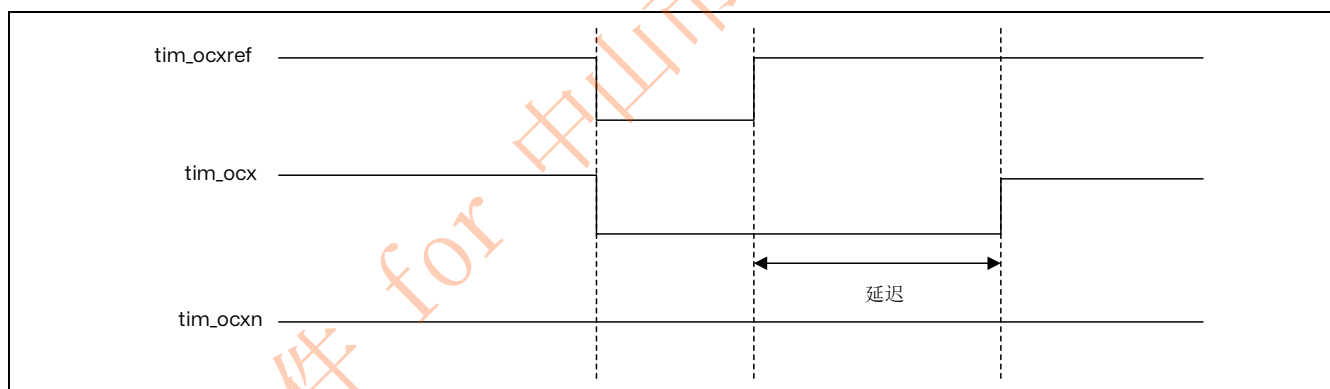


图 25.29 死区波形延迟大于负脉冲

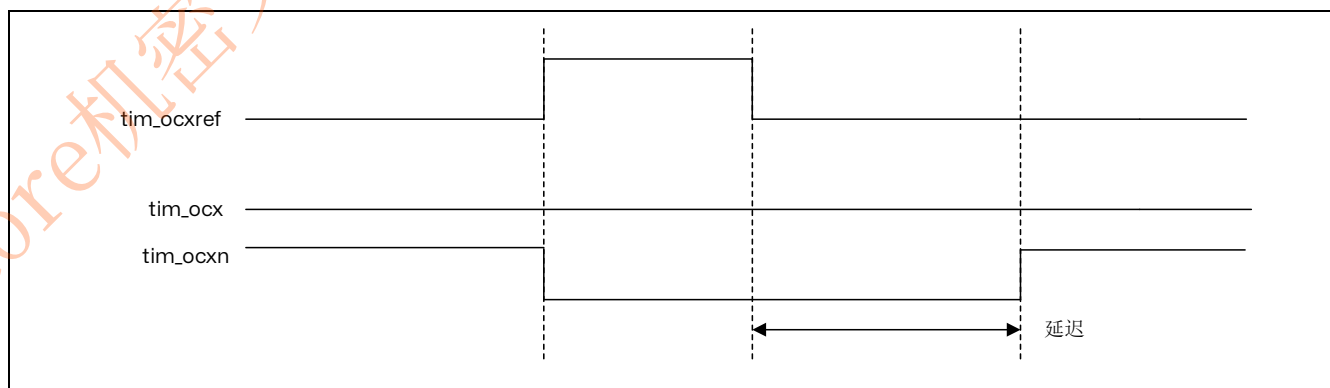


图 25.30 死区波形延迟大于正脉冲

死区延迟对于所有通道均相同，可通过 TIMx\_BDTR 寄存器中的 DTG 位进行编程。有关延迟时间计算的信息，请参见后续章节：TIMx 刹车和死区寄存器(TIMx\_BDTR) (x = 16~17)。

#### 将 OCxREF 重定向到 OCx 或 OCxN

在输出模式（强制输出模式、输出比较模式或 PWM 模式）下，通过配置 TIMx\_CCER 寄存器中的 CCxE 和 CCxNE 位，可将 tim\_ocxref 重定向到 tim\_ocx 输出或 tim\_ocxn 输出。通过此功能，可以在一个输出上发送特定波形（如 PWM 或静态有效电平），而同时使互补输出保持其无效电平。或者，使两个输出同时保持无效电平，或者两个输出同时处于有效电平，两者互补并且带死区。

注：如果仅使能 tim\_ocxn (CCxE=0, CCxNE=1)，两者不互补，一旦 tim\_ocxref 为高电平，tim\_ocxn 即变为有效。例如，如果 CCxNP=0，则 tim\_ocxn = tim\_ocxref。另一方面，如果同时使能 tim\_ocxn 和 tim\_ocxref (CCxE=CCxNE=1)，tim\_ocx 在 tim\_ocxref 为高电平时变为有效，而 tim\_ocxn 则与之互补，在 tim\_ocxref 为低电平时变为有效。

#### 25.4.15 使用刹车功能

刹车功能的目的是保护由定时器产生的 PWM 信号驱动的功率开关。刹车输入通常连接到功率级和三相逆变器的故障输出。当激活时，刹车电路会关闭 PWM 输出并强制它们进入预定义的安全状态。

刹车通道可以收集系统级故障（时钟故障、ECC/奇偶校验错误，.....）和应用故障（来自输入引脚和内置比较器），并可以在死区时间后将输出强制到一个预定义的水平（无论是有效还是无效）。

刹车期间输出使能信号和输出电平取决于几个控制位：

- TIMx\_BDTR 寄存器中的 MOE 位允许通过软件使能/禁用输出，并在发生刹车事件时重置。
- TIMx\_BDTR 寄存器中的 OSSR 位定义了定时器是否在非活动状态下控制输出或将其控制权释放给 GPIO 控制器（通常使其处于高阻模式）。
- TIMx\_CR2 寄存器中的 OISx 和 OISxN 位用于设置输出关闭电平，可以是有效电平或无效电平。tim\_oc1 和 tim\_oc1n 输出不能同时设为有效电平。更多详细信息请参考后续表格：具有刹车功能的互补 tim\_oc1 和 tim\_oc1n 通道的输出控制位。

退出复位时，刹车电路被禁用，MOE 位为低电平。通过设置 TIMx\_BDTR 寄存器中的 BKE 位，可以使能刹车功能。通过配置同一寄存器中的 BKP 位，可以选择刹车输入极性。BKE 和 BKP 可以同时修改。在写入 BKE 和 BKP 位后，需要等待 1 个 APB 时钟周期才能使写入生效。因此，在写入操作后，需要等待 1 个 APB 时钟周期才能正确地读取该位。

因为 MOE 下降沿可以是异步的，所以在实际的信号（作用于输出）和同步控制位（在 TIMx\_BDTR 寄存器中访问）之间插入了一个重新同步电路。这导致异步信号和同步信号之间存在一些延迟。特别是当 MOE 被设置为 1 而之前它是低电平时，在正确读取之前必须插入一个延迟（伪指令）。这是因为写入作用于异步信号，而读取反映的是同步信号。

刹车是由 tim\_brk 输入产生的，它具有：

- 可编程极性（TIMx\_BDTR 寄存器中的 BKP 位）
- 可编程使能位（TIMx\_BDTR 寄存器中的 BKE 位）
- 可编程滤波器（TIMx\_BDTR 寄存器中的 BKF[3:0]位），以避免产生虚假事件。

可以使用 TIMx\_AF1 寄存器，从多个源生成刹车，这些源可以单独使能，并具有可编程的边沿敏感度。

刹车（tim\_brk）通道的源为：

- 连接到 TIMx\_BKIN 引脚的外部源（根据 GPIO 备用功能选择寄存器中的选择而定），具有极性选择和可选的数字滤波

● 内部源:

- 来自 tim\_brk\_cmpx 输入 (请参考章节: TIM15/TIM16/TIM17 引脚和内部信号)
- 来自 tim\_sys\_brk 输入的系统中断请求 (请参考章节: TIM15/TIM16/TIM17 引脚和内部信号)

也可以通过软件使用 TIMx\_EGR 寄存器中的 BG 位生成刹车事件。

所有源在进入定时器的 tim\_brk 输入之前进行 OR 操作, 如下图所示。

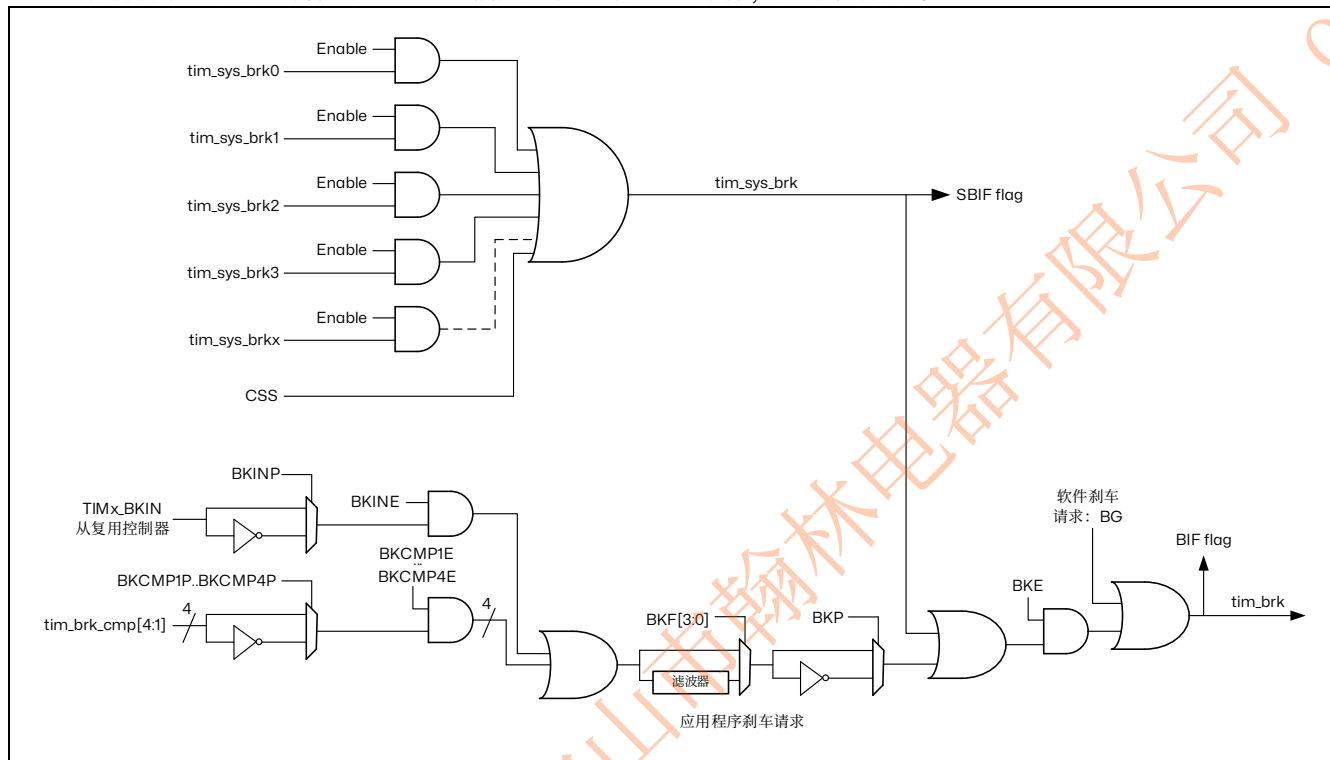


图 25.31 刹车电路概述

注: 只有当可编程滤波器禁用时, 才保证异步 (无时钟) 操作。如果使能, 必须使用故障安全时钟模式 (例如使用内部 PLL 和/或 CSS) 来保证刹车事件得到处理。

当一个刹车发生时 (在刹车输入上选择电平):

- MOE 位被异步清除, 使输出处于无效状态、空闲状态, 甚至释放控制权给 GPIO (由 OSS1 位选择)。即使 MCU 振荡器关闭, 此功能也可工作。
- 每当 MOE=0 时, 每个输出通道都以 TIMx\_CR2 寄存器中的 OISx 位中编程的电平驱动。如果 OSS1=0, 定时器将释放输出控制 (由 GPIO 接管), 否则使能输出保持高电平。
- 当使用互补输出时:
  - 输出首先被置于复位状态 (取决于极性), 这是异步完成的, 因此即使没有向定时器提供时钟也可以工作。
  - 如果定时器的时钟仍然存在, 则重新激活死区时间发生器, 以便在死区时间后驱动输出到编程在 OISx 和 OISxN 位中的电平。即使在这种情况下, tim\_ocx 和 tim\_ocxn 也不能同时驱动到有效电平。请注意, 由于在 MOE 上进行了重新同步, 死区时间持续时间比通常情况稍长 (大约 2 个 tim\_ker\_ck 时钟周期)。
  - 如果 OSS1=0, 则定时器释放使能输出 (由 GPIO 接管, 强制进入高阻态), 否则使能输出在 CCxE 或 CCxNE 位中的任意一位变为高电平时立即变为高电平。
- 刹车状态标志 (TIMx\_SR 寄存器中的 BIF 位) 被设置。如果 TIMx\_DIER 寄存器中的 BIE 位被设

置，则可以生成刹车。

- 如果 TIMx\_BDTR 寄存器中的 AOE 位被设置，则 MOE 位将在下一个更新事件 UEV 处自动重新设置。这可以用于执行调节等操作。否则，MOE 保持低电平，直到再次写入 1。在这种情况下，它可以用于安全，刹车输入可以连接到来自功率驱动器、温度传感器或任何安全组件的警报。

注：如果 MOE 被 CPU 复位，而 AOE 位已设置，则输出处于空闲状态，并强制处于非活动电平或高阻态，具体取决于 OSSR 值。如果 MOE 和 AOE 位都被 CPU 复位，则输出处于禁用状态，并驱动在 TIMx\_CR2 寄存器中 OISx 位编程的电平。

注：刹车输入在电平上有效。因此，当刹车输入处于活动状态时（无论是自动还是通过软件设置），MOE 都无法设置。同时，状态标志 BIF 不能被清除。

除刹车输入和输出管理外，刹车电路内部还实施了写保护，用以保护应用的安全。通过该功能，用户可冻结多个参数配置（死区持续时间、tim\_ocx/tim\_ocxn 极性和禁止时的状态、OCxM 配置、刹车使能和极性）。可以通过 TIMx\_BDTR 寄存器中的 LOCK 位，从 3 种保护级别中进行选择。请参考章节：TIMx 刹车和死区寄存器（TIMx\_BDTR）。MCU 复位后只能对 LOCK 位执行一次写操作。

下图所示为输出对刹车响应行为的示例。

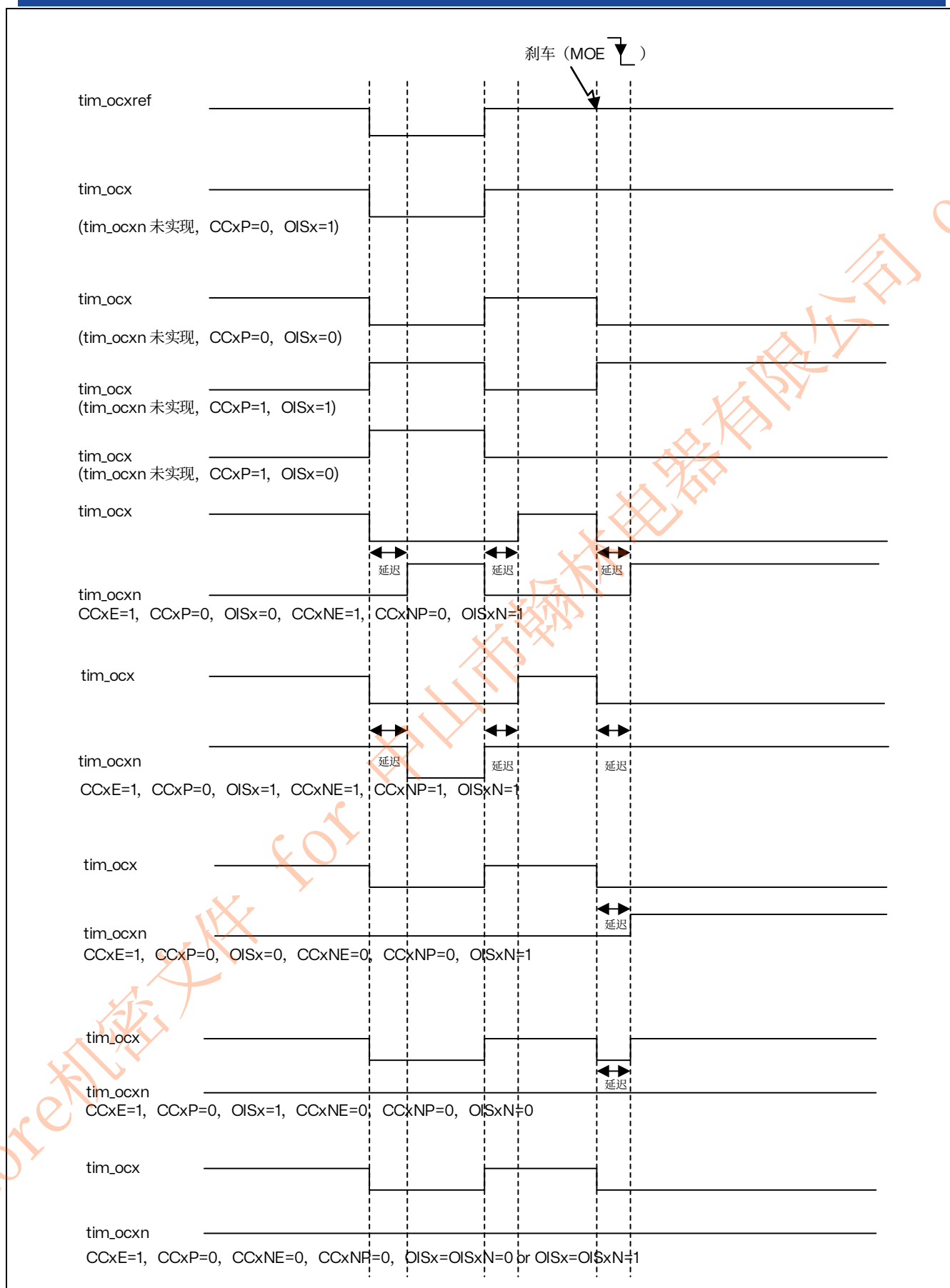


图 25.32 tim\_brk 在发生刹车事件时的各种输出行为 (OSSI=1)

### 25.4.16 双向刹车输入

TIM15/TIM16/TIM17 具有双向刹车 I/O，如下图所示。

这为以下方面提供支持：

- 板级全局刹车信号，用于向外部 MCU 或门驱动器发送故障信号，使用唯一的引脚作为输入和输出状态引脚
- 内部刹车源和多个外部开漏源 OR 连接在一起，以触发唯一的刹车事件，当多个内部和外部刹车源必须合并时

使用 TIMxBDTR 寄存器中的 BKBID 位配置 tim\_brk 输入为双向模式。BKBID 编程位可以使用 TIMxBDTR 寄存器中的 LOCK 位锁定为只读模式（在 LOCK 级别 1 或更高）。

双向模式要求 I/O 配置为开漏模式，使用低电平有效极性（使用 BKINP、BKP 位）。来自系统（例如 CSS）、片上外设或刹车输入的任何刹车请求都会强制将刹车输入置于低电平，以指示故障事件。出于安全原因，如果极性位没有正确设置（高电平有效），则双向模式将被抑制。

刹车软件事件（通过设置 BG 位触发）也会强制将刹车 I/O 置为“0”，以指示外部组件定时器已进入刹车状态。然而，只有使能了刹车（BKE = 1）时，这才有效。当使用 BKE = 0 生成软件刹车事件时，输出将处于安全状态，并且会设置刹车标志，但 TIMx\_BKIN I/O 没有影响。

安全解发机制可防止系统被彻底锁定（刹车输入低电平触发刹车，强制该输入为低电平）。

当 BKDSRM 位设置为 1 时，这将释放刹车输出以清除故障信号并允许重新给系统装载。

在任何时候都不能禁用刹车保护电路：

- 刹车输入路径始终处于活动状态：即使设置了 BKDSRM 位并且开放漏极控制被释放，刹车事件仍然处于活动状态。这可以防止 PWM 输出在刹车条件存在时重新启动。
- 只要输出使能（MOE 位被设置），BKDSRM 位就不能解除刹车保护（参见下表）

表 25.10 刹车保护解除条件

| MOE | BKBID | BKDSRM | 刹车保护状态 |
|-----|-------|--------|--------|
| 0   | 0     | X      | 装载     |
| 0   | 1     | 0      | 装载     |
| 0   | 1     | 1      | 解除     |
| 1   | X     | X      | 装载     |

#### 装载和重装载刹车电路

默认情况下（外设复位配置），刹车电路（输入或双向模式）处于装载状态。

在发生刹车事件后，必须按照以下步骤重新装载保护：

- BKDSRM 位必须设置为释放输出控制
- 软件必须等待系统刹车条件消失，并清除 SBIF 状态标志（或在重新装载之前系统地清除它）
- 软件必须轮询 BKDSRM 位，直到硬件清除该位（当应用程序刹车条件消失时）

从这一点开始，刹车电路处于装载和有效状态，并且可以设置 MOE 位以重新使能 PWM 输出。



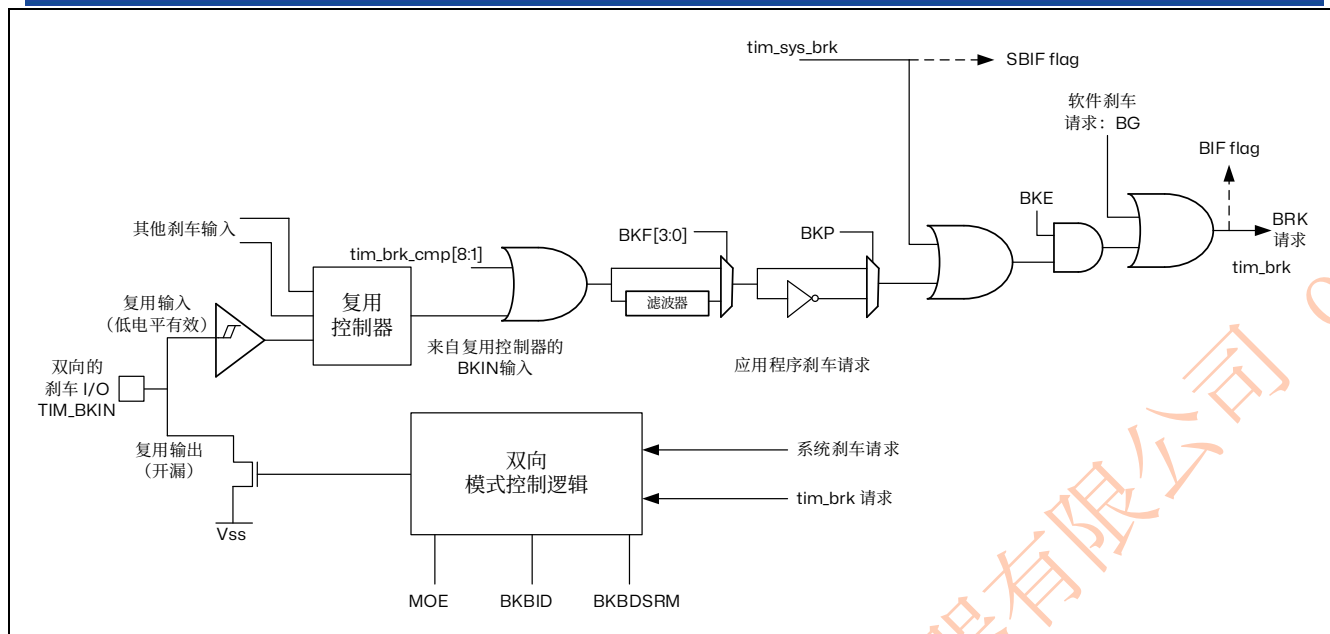


图 25.33 输出重定向

#### 25.4.17 在外部事件时清除 tim\_ocxref 信号

对于给定通道，在 tim\_ocref\_clr\_int 输入施加高电平（相应 TIMx\_CCMRx 寄存器中的 OCxCE 使能位置“1”），可使 tim\_ocxref 信号变为低电平。tim\_ocxref 信号将保持低电平，直到发生下一更新事件（UEV）。此功能仅能用于输出比较模式和 PWM 模式，而不适用于强制输出模式。

tim\_ocref\_clr 输入可以在多个输入中进行选择，如下图所示。

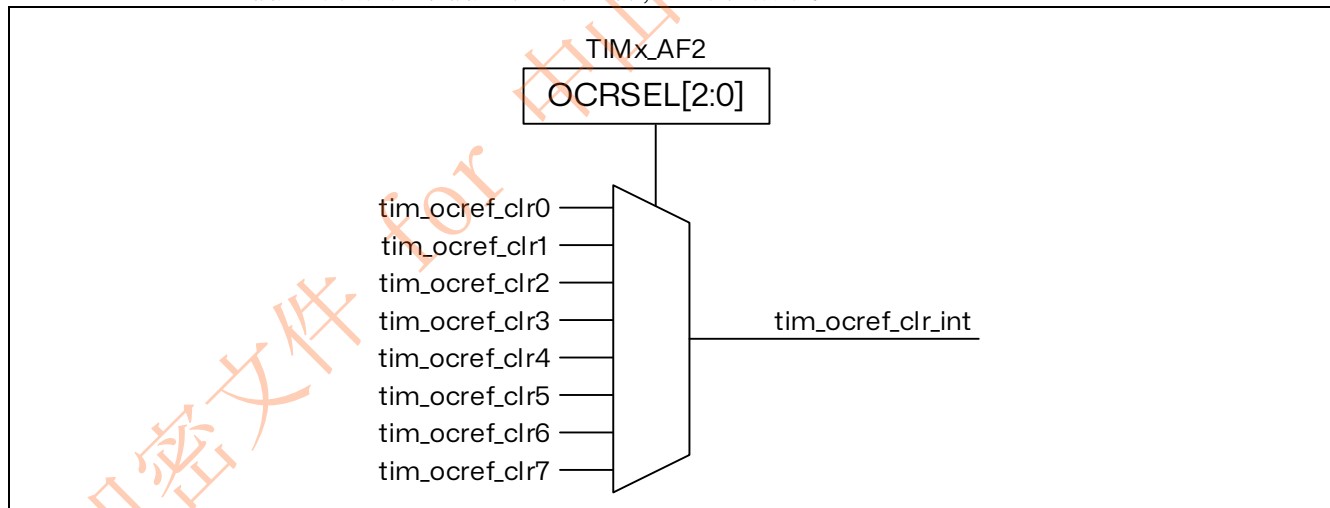


图 25.34 tim\_ocref\_clr 输入选择复用器

#### 25.4.18 单脉冲模式

单脉冲模式（OPM）是上述模式的一个特例。在这种模式下，计数器可以在一个激励信号的触发下启动，并可在一段可编程的延时后产生一个脉宽可编程的脉冲。

可以通过从模式控制器启动计数器。可以在输出比较模式或 PWM 模式下生成波形。将 TIMx\_CR1 寄存器中的 OPM 位置 1，即可选择单脉冲模式。这样，发生下一更新事件 UEV 时，计数器将自动停止。

只有当比较值与计数器初始值不同时，才能正确产生一个脉冲。启动前（定时器等待触发时），必须进行如下配置：

- $CNT < CCRx \leq ARR$ （特别注意， $0 < CCRx$ ）

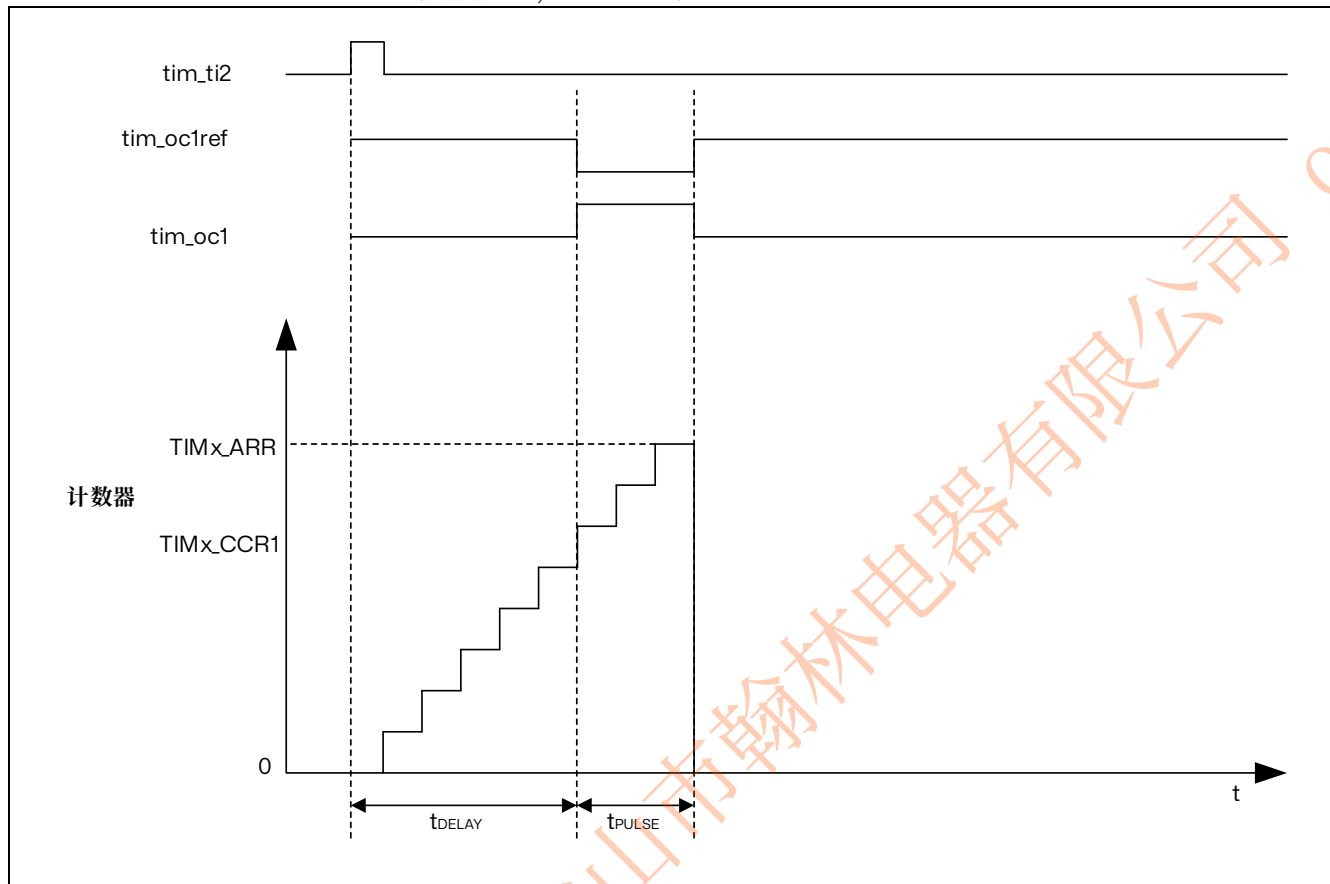


图 25.35 单脉冲模式的例子

例如，用户希望达到这样的效果：在 `tim_ti2` 输入引脚检测到正沿时，经过  $t_{DELAY}$  的延迟，在 `tim_oc1` 上产生一个长度为  $t_{PULSE}$  的正脉冲。

使用 `tim_ti2fp2` 作为触发 1：

1. 通过在 `TIMx_TISEL` 寄存器中使用 `TI2SEL[3:0]` 位选择合适的 `tim_ti2_in[15:0]` 源（内部或外部）。
2. 在 `TIMx_CCMR1` 寄存器中写入 `CC2S=01`，将 `tim_ti2fp2` 映射到 `tim_ti2`。
3. 在 `TIMx_CCER` 寄存器中写入 `CC2P=0` 和 `CC2NP="0"`，使 `tim_ti2fp2` 能够检测上升沿。
4. 在 `TIMx_SMCR` 寄存器中写入 `TS=00110`，将 `tim_ti2fp2` 配置为从模式控制器的触发 (`tim_trgi`)。
5. 在 `TIMx_SMCR` 寄存器中写入 `SMS="110"`（触发模式），使用 `tim_ti2fp2` 启动计数器。

OPM 波形通过对比较寄存器执行写操作来定义（考虑时钟频率和计数器预分频器）。

- $t_{DELAY}$  由写入 `TIMx_CCR1` 寄存器的值定义。
- $t_{PULSE}$  由自动重载值与比较值 (`TIMx_ARR - TIMx_CCR1`) 之差来定义。
- 假设希望产生这样的波形：信号在发生比较匹配时从“0”变为“1”，在计数器达到自动重载值时由“1”变为“0”。为此，应在 `TIMx_CCMR1` 寄存器中写入 `OC1M=111`，以使能 PWM 模式 2。如果需要，可选择在 `TIMx_CCMR1` 寄存器的 `OC1PE` 和 `TIMx_CR1` 寄存器的 `ARPE` 中写入“1”，以使能预装载寄存器。这种情况下，必须在 `TIMx_CCR1` 寄存器中写入比较值并在 `TIMx_ARR` 寄存器中写入自动重载值，通过将 `UG` 位置 1 来产生更新，然后等待 `tim_ti2` 上的外部触发事件。本例中，`CC1P` 的值为“0”。

在本例中，TIMx\_CR1 寄存器中的 DIR 和 CMS 位应为低。

由于仅需要 1 个脉冲（单脉冲模式），因此应向 TIMx\_CR1 寄存器的 OPM 位写入“1”，以便在发生下一更新事件（计数器从自动重载值返回到 0）时使计数器停止计数。TIMx\_CR1 寄存器中的 OPM 位置“0”时，即选择重复模式。

#### 特例：OCx 快速使能：

在单脉冲模式下，Tlx 输入的边沿检测会将 CEN 位置 1，表示使能计数器。然后，在计数器值与比较值之间发生比较时，将切换输出。但是，完成这些操作需要多个时钟周期，这会限制可能的最小延迟（ $t_{\text{DELAY}}$  最小值）。

如果要输出延迟时间最短的波形，可以将 TIMx\_CCMRx 寄存器中的 OCxFE 位置 1。这样会强制 tim\_ocxref（和 tim\_ocx）对激励信号做出响应，而不再考虑比较的结果。其新电平与发生比较匹配时相同。仅当通道配置为 PWM1 或 PWM2 模式时，OCxFE 才会起作用

#### 25.4.19 可重触发单脉冲模式（仅 TIM15）

此模式允许计数器在接收到刺激后开始计数，并生成一个具有可编程长度的脉冲，但与上一节中描述的非重触发单脉冲模式存在以下差异：

- 只要触发器发生，脉冲就会立即开始（没有可编程的延迟）
- 如果在先前触发的脉冲完成之前发生新的触发，则脉冲会被延长

定时器必须处于从模式，TIMx\_SMCR 寄存器的 SMS[3:0] 位设置为“1000”（组合重置+触发模式），OCxM[3:0] 位设置为“1000”或“1001”，用于可重触发 OPM 模式 1 或 2。

如果定时器配置为递增计数模式，则相应的 CCRx 必须设置为 0（ARR 寄存器设置脉冲长度）。如果定时器配置为递减计数模式，CCRx 必须大于或等于 ARR。

注：OCxM[[3:0] 和 SMS[3:0] 位域由于兼容性的原因被分成两个部分，最高有效位与最低 3 位不是连续的。OCxM 这种模式不能与中央对齐 PWM 模式一起使用。TIMx\_CR1 寄存器中的 CMS[1:0] 必须设置为 00。

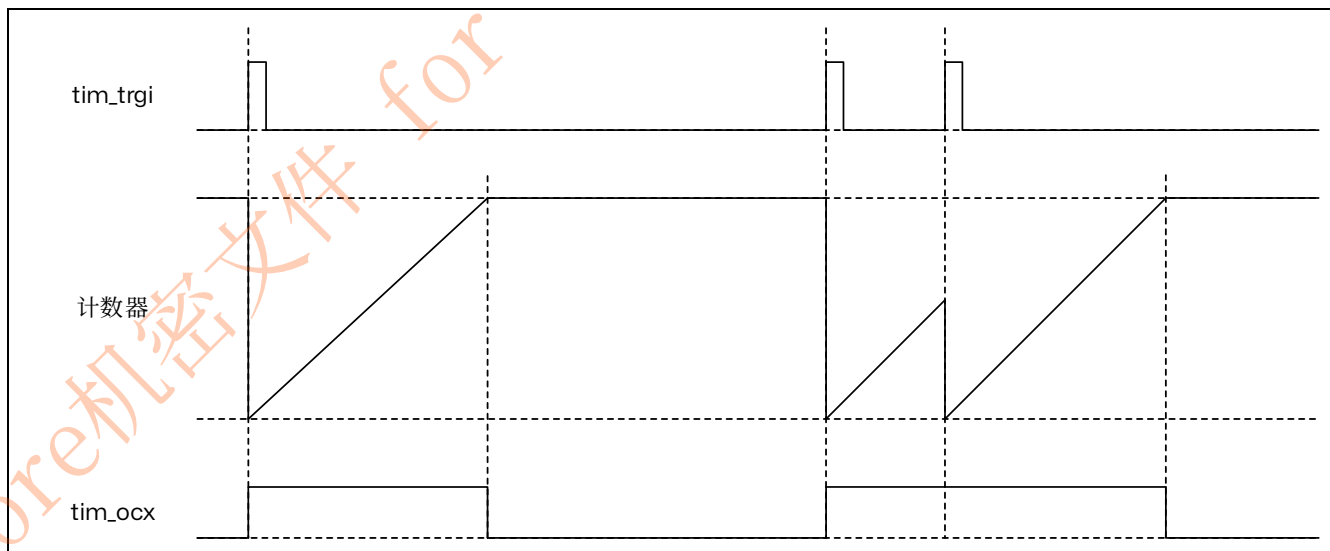


图 25.36 可重触发单脉冲模式

### 25.4.20 UIF 位重映射

TIMx\_CR1 寄存器中的 IUFREMAP 位强制将更新中断标志 UIF 连续复制到定时器计数器寄存器的第 31 位 (TIMxCNT[31])。这允许以原子方式读取计数器的值和一个潜在的由 UIFCPY 标志指示的翻转条件。在特定情况下，它可以避免由背景任务（读取计数器）和中断（更新中断）之间的处理引起的竞争条件，从而简化计算。

UIF 和 UIFCPY 标志之间没有延迟。

### 25.4.21 定时器输入异或功能（仅 TIM15）

TIM15\_CR2 寄存器中的 TI1S 位允许将通道 1 的输入滤波器连接到异或门的输出，将两个输入引脚 tim\_ti1 和 tim\_ti2 组合起来。

异或输出可以与所有定时器输入功能一起使用，例如触发器或输入捕获。它对于测量两个输入信号之间的边沿间隔非常有用，如下图所示。

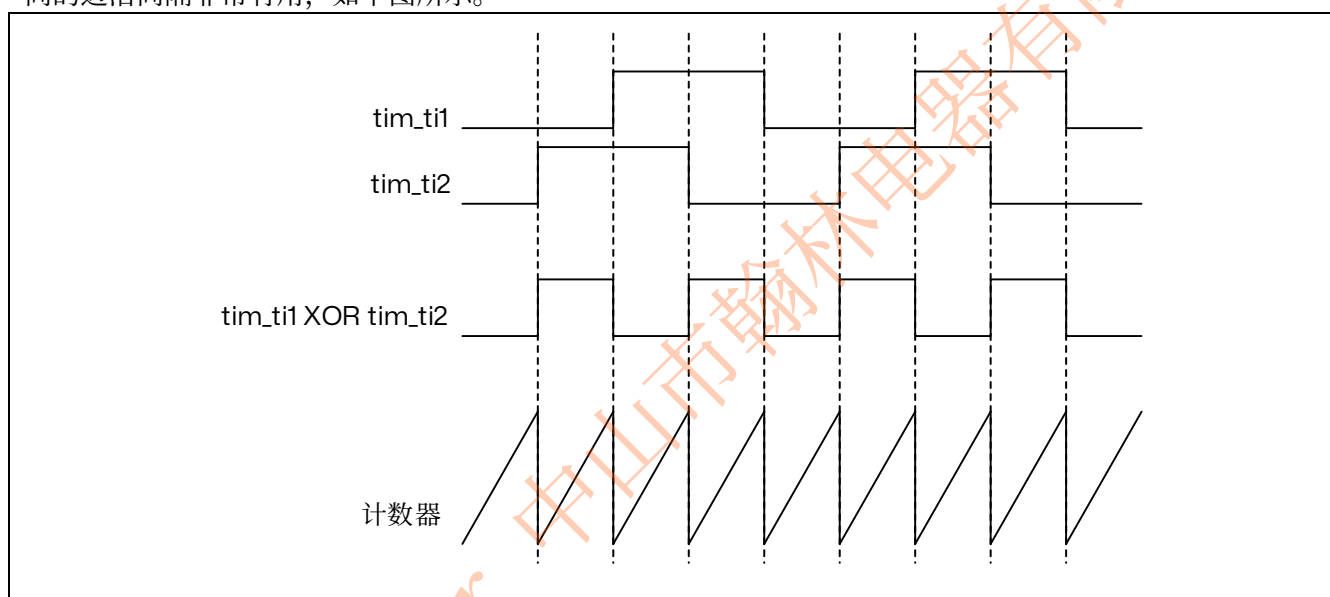


图 25.37 测量两个信号边沿之间的间隔时间

### 25.4.22 定时器和外部触发的同步（仅 TIM15）

TIM 定时器在互相连接用于定时器的同步或链接。

TIM15 定时器以下列模式与外部触发实现同步：复位模式、门控模式、触发模式、复位+触发和门控+复位模式。

#### 从模式：复位模式

当触发输入信号发生变化时，计数器及其预分频器可重新初始化。此外，如果 TIM15\_CR1 寄存器中的 URS 位处于低电平，则会生成更新事件 UEV。然后，所有预装载寄存器 (TIM15\_ARR 和 TIM15\_CCRx) 都将更新。

在以下示例中，tim\_ti1 输入上出现上升沿时，递增计数器清零：

1. 将通道 1 配置为检测 tim\_ti1 的上升沿。配置输入滤波时间（本例中不需要任何滤波，因此保持 IC1F=0000）。由于捕获预分频器不用于触发操作，因此无需对其进行配置。CC1S 位只选择输入捕获源，即 TIMx\_CCMR1 寄存器中的 CC1S = 01。在 TIM15\_CCER 寄存器中写入 CC1P=0

和 CC1NP=0，验证极性（仅检测上升沿）。

- 在 TIM15\_SMCR 寄存器中写入 SMS=100，将定时器配置为复位模式。在 TIM15\_SMCR 寄存器中写入 TS=101，选择 tim\_ti1 作为输入源。
- 在 TIM15\_CR1 寄存器中写入 CEN=1，启动计数器。

计数器使用内部时钟计数，然后正常运转，直到出现 tim\_ti1 上升沿。当 tim\_ti1 出现上升沿时，计数器清零，然后重新从 0 开始计数。同时，触发标志（TIM15\_SR 寄存器中的 TIF 位）置 1，使能中断后，还可发送中断请求（取决于 TIM15\_DIER 寄存器中的 TIE 和 TDE 位）。

下图显示了自动重载寄存器 TIM15\_ARR=0x36 时的相关行为。tim\_ti1 的上升沿与实际计数器复位之间的延迟是由于 tim\_ti1 输入的重新同步电路引起的。

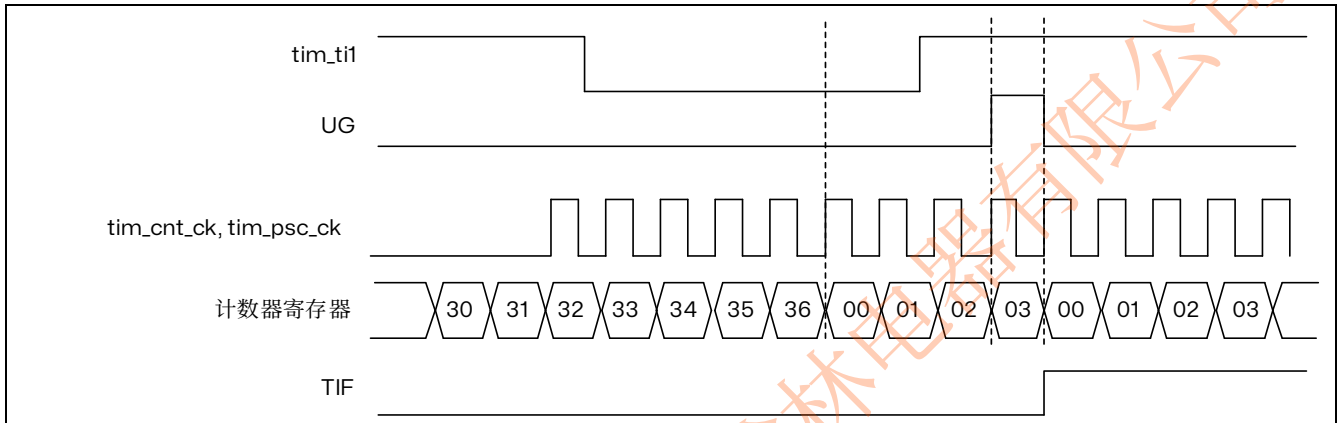


图 25.38 复位模式下的控制电路

### 从模式：门控模式

输入信号的电平可用来使能计数器。

在以下示例中，递增计数器仅在 tim\_ti1 输入为低电平时计数：

- 将通道 1 配置为检测 tim\_ti1 上的低电平。配置输入滤波时间（本例中不需要任何滤波，因此保持 IC1F=0000）。由于捕获预分频器不用于触发操作，因此无需对其进行配置。CC1S 位只选择输入捕获源，即 TIM15\_CCMR1 寄存器中的 CC1S=01。在 TIM15\_CCER 寄存器中写入 CC1P=1 和 CC1NP=0，以确定极性（仅检测低电平）。
- 在 TIM15\_SMCR 寄存器中写入 SMS=101，将定时器配置为门控模式。在 TIM15\_SMCR 寄存器中写入 TS=00101，选择 tim\_ti1 作为输入源。
- 在 TIM15\_CR1 寄存器中写入 CEN=1，使能计数器（在门控模式下，如果 CEN=0，则无论触发输入电平如何，计数器都不启动）。

只要 tim\_ti1 为低电平，计数器就开始根据内部时钟计数，直到 tim\_ti1 变为高电平时停止计数。计数器启动或停止时，TIM15\_SR 寄存器中的 TIF 标志都会置 1。

tim\_ti1 的上升沿与实际计数器停止之间的延迟是由于 tim\_ti1 输入的重新同步电路引起的。

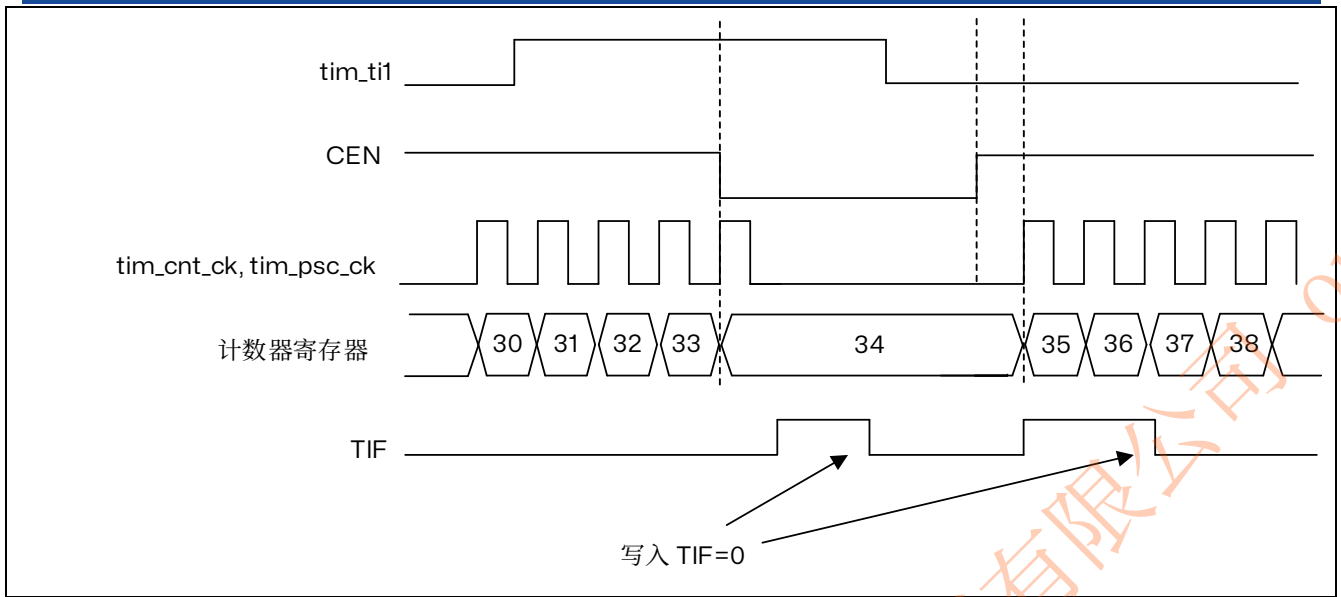


图 25.39 门控模式下的控制电路

### 从模式：触发模式

所选输入上发生某一事件时可以启动计数器。

在以下示例中，tim\_ti2 输入上出现上升沿时，递增计数器启动：

1. 将通道 2 配置为检测 tim\_ti2 上的上升沿。配置输入滤波时间（本例中不需要任何滤波，因此保持 IC2F=0000）。由于捕获预分频器不用于触发操作，因此无需对其进行配置。CC2S 位只选择输入捕获源，即 TIM15\_CCMR1 寄存器中的 CC2S=01。在 TIM15\_CCER 寄存器中写入 CC2P=1 和 CC2NP=0，以确定极性（仅检测低电平）。
2. 在 TIM15\_SMCR 寄存器中写入 SMS=110，将定时器配置为触发模式。在 TIMx\_SMCR 寄存器中写入 TS=00110，选择 tim\_ti2 作为输入源。

当 tim\_ti2 出现上升沿时，计数器开始根据内部时钟计数，并且 TIF 标志置 1。

tim\_ti2 的上升沿与实际计数器启动之间的延迟是由于 tim\_ti2 输入的重新同步电路引起的。

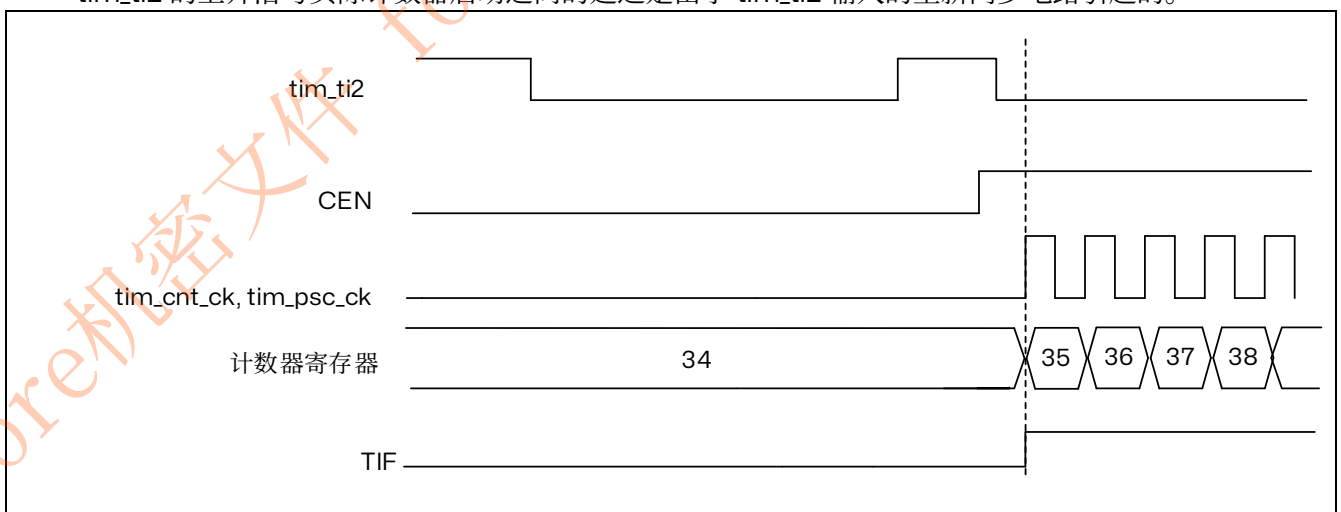


图 25.40 触发器模式下的控制电路

### 对实时更新编码器模式的从模式选择预装载



SMS[3:0]位可以预加载。这是通过在 TIM15\_SMCR 寄存器中设置 SMSPE 使能位来实现的。从 SMS[3:0]预加载到有效值的触发器是计数器溢出时发生的更新事件 (UEV)。

#### 从模式 - 组合重置 + 触发模式

在这种情况下，所选触发输入 (tim\_trgi) 的上升沿会重新初始化计数器，此模式用于单脉冲模式。

#### 从模式 - 组合重置 + 门控模式

当触发输入 (tim\_trgi) 为高时，计数器使能。计数器停止并一旦触发器变为低，计数器就会重置。计数器的启动和停止都是受控的。

此模式允许检测超出范围的 PWM 信号 (占空比超过最大期望值)。

### 25.4.23 定时器同步 (仅 TIM15)

TIM15 定时器从内部连接在一起，以实现定时器同步或级联。

注：接收 tim\_trgo 信号的从外设 (定时器、ADC...) 的时钟必须在主定时器接收事件前使能，并且在主定时器接收触发时，时钟频率 (预分频器) 不得动态更改。

### 25.4.24 使用定时器输出作为其他定时器的触发器 (仅 TIM16/TIM17)

只有一个通道的定时器不具备主模式。但是，OC1 输出信号可用于触发其他定时器 (包括本文档中其他部分描述的定时器)。请检查设备上任何定时器的“TIMx 内部触发器连接”表，以确定哪些定时器可以作为从定时器。

从定时器检测到触发器的 OC1 信号脉冲宽度必须至少为目标定时器的 2 个时钟周期。

例如，如果目标定时器的 CK\_INT 时钟比源定时器慢 4 倍，则 OC1 脉冲宽度必须为 8 个时钟周期。

### 25.4.25 ADC 触发 (仅 TIM15)

定时器可以使用各种内部信号 (例如重置、使能或比较事件) 生成 ADC 触发事件。

注意：接收 tim\_trgo 信号的从属外设 (定时器、ADC 等) 的时钟必须在从主定时器接收事件之前使能，并且时钟频率 (预分频器) 在从主定时器接收触发器时不能更改。

### 25.4.26 DMA 突发模式

TIMx 定时器具有在单个事件上生成多个 DMA 请求的能力。其主要目的是能够在不增加软件开销的情况下多次重新编程定时器的一部分，但它也可以用于在固定间隔内连续读取多个寄存器。

DMA 控制器的目标地址是唯一的，必须指向虚拟寄存器 TIMx\_DMAR。在给定的定时器事件上，定时器启动一系列 DMA 请求 (突发)。对 TIMx\_DMAR 寄存器的每次写入实际上都被重定向到定时器寄存器之一。

TIMx\_DCR 寄存器中的 DBL[4:0]位设置了 DMA 突发长度。当对 TIMx\_DMAR 地址进行读取或写入访问时，定时器会识别出突发传输，即传输的数量 (以半字或字节为单位)。

TIMx\_DCR 寄存器中的 DBA[4:0]位定义了 DMA 传输的 DMA 基地址 (当通过 TIMx\_DMAR 地址进行读取/写入访问时)。DBA 是从 TIMx\_CR1 寄存器的地址开始定义的偏移量：

示例：

00000: TIMx\_CR1

00001: TIMx\_CR2



#### 00010: TIMx\_SMCR

作为一个例子，定时器 DMA 突发模式用于在更新事件发生时更新 CCRx 寄存器 (x = 2, 3, 4) 的内容，其中 DMA 将半字传输到 CCRx 寄存器中。

这是通过以下步骤完成的：

1. 配置相应的 DMA 通道如下：
  - DMA 通道外设地址是 DMAR 寄存器的地址。
  - DMA 通道内存地址是 RAM 中缓冲区的地址，其中包含要通过 DMA 传输到 CCRx 寄存器的数据。
  - 要传输的数据数量 = 3 (见下文注释)。
  - 循环模式禁用。
2. 通过配置 DBA 和 DBL 位域来配置 DCR 寄存器，如下所示：  
DBL = 3 次传输，DBA = 0xE。
3. 使能 TIMx 更新 DMA 请求 (在 DIER 寄存器中设置 UDE 位)。
4. 使能 TIMx
5. 使能 DMA 通道

此示例适用于每个 CCRx 寄存器都需要更新一次的情况。例如，如果要更新每个 CCRx 寄存器两次，则要传输的数据数量应为 6。让我们以一个在 RAM 中包含 data1, data2, data3, data4, data5 和 data6 的缓冲区为例。数据按照以下方式传输到 CCRx 寄存器：在第一个更新 DMA 请求时，data1 传输到 CCR2, data2 传输到 CCR3, data3 传输到 CCR4；在第二个更新 DMA 请求时，data4 传输到 CCR2, data5 传输到 CCR3, data6 传输到 CCR4。

注：可以将空值写入保留的寄存器。

#### 25.4.27 TIM15/TIM16/TIM17 DMA 请求

TIM15/TIM16/TIM17 可以生成 DMA 请求，如下表所示。

表 25.11 DMA 请求

| DMA 请求信号                   | DMA 缩写   | DMA 请求   | 使能控制位 |
|----------------------------|----------|----------|-------|
| tim_upd_dma                | TIM_UP   | 更新       | UDE   |
| tim_cc1_dma                | TIM_CH1  | 捕获/比较 1  | CC1DE |
| tim_com_dma <sup>(1)</sup> | TIM_COM  | 换向 (COM) | COMDE |
| tim_trg_dma <sup>(1)</sup> | TIM_TRIG | 触发       | TDE   |

1. 仅供 TIM15 使用。

#### 25.4.28 调试模式

当微控制器进入调试模式 (带有 FPU 的 Cortex®-M4 内核停止) 时，TIMx 计数器可以继续正常工作或停止。

在调试模式下，每个定时器的行为可以通过 Debug 支持 (DBG) 模块的专用配置位进行编程。

出于安全考虑，当计数器停止时，输出被禁用 (就像将 MOE 位复位一样)。输出可以强制处于非活动状态 (OSSI 位=1)，或者由 GPIO 控制器接管控制 (OSSI 位=0) 以强制其处于高阻态。

有关更多详细信息，请参阅调试部分。

## 25.5 TIM15/TIM16/TIM17 低功耗模式

表 25.12 低功耗模式对 TIM15/TIM16/TIM17 的影响

| 模式 | 描述                        |
|----|---------------------------|
| 睡眠 | 无影响，外设是运行的。中断会导致设备退出睡眠模式。 |
| 停机 | 定时器操作被停止并保留其寄存器内容。无法生成中断。 |
| 待机 | 定时器被断电并且退出待机模式后必须重新初始化。   |

## 25.6 TIM15/TIM16/TIM17 中断

TIM15 可以生成多个中断，如下表所示。

表 25.13 中断请求

| 中断缩写 | 中断事件                   | 事件标志  | 使能控制位 | 中断清除方法    | 从睡眠模式退出 | 从停机和待机退出 |
|------|------------------------|-------|-------|-----------|---------|----------|
| TIM  | 更新                     | UIF   | UIE   | UIF 写 0   | 是       | 否        |
|      | 捕获/比较 1                | CC1IF | CC1IE | CC1IF 写 0 | 是       | 否        |
|      | 捕获/比较 2 <sup>(1)</sup> | CC2IF | CC2IE | CC2IF 写 0 | 是       | 否        |
|      | 换向 (COM)               | COMIF | COMIE | COMIF 写 0 | 是       | 否        |
|      | 触发 <sup>(1)</sup>      | TIF   | TIE   | TIF 写 0   | 是       | 否        |
|      | 刹车                     | BIF   | BIE   | BIF 写 0   | 是       | 否        |

1. 仅供 TIM15 使用。

## 25.7 TIM15 寄存器描述

在寄存器描述中使用的缩写列表请参考章节 2.1。

### 25.7.1 TIM15 控制寄存器 1 (TIM15\_CR1)

偏移地址: 0x000

复位值: 0x0000

| 15       | 14 | 13 | 12         | 11           | 10  | 9        | 8  | 7    | 6        | 5 | 4 | 3   | 2   | 1    | 0   |
|----------|----|----|------------|--------------|-----|----------|----|------|----------|---|---|-----|-----|------|-----|
| Reserved |    |    | DITH<br>EN | UIFRE<br>MAP | Res | CKD[1:0] |    | ARPE | Reserved |   |   | OPM | URS | UDIS | CEN |
|          |    |    | rw         | rw           |     | rw       | rw | rw   |          |   |   | rw  | rw  | rw   | rw  |

Bit 15:13 保留, 必须保持复位值。

Bit 12 **DITHEN**: 使能抖动 (Dithering enable)

0: 抖动禁止

1: 抖动使能

注: DITHEN 位只能在 CEN 位复位时修改

Bit 11 **UIFREMAP**: UIF 状态位重映射 (UIF status bit remapping)

0: 无重映射。UIF 状态位未复制到 TIM15\_CNT 寄存器位 31

1: 重映射使能。UIF 状态位复制到 TIM15\_CNT 寄存器位 31

Bit 10 保留, 必须保持复位值。

Bit 9:8 **CKD[1:0]**: 时钟分频 (Clock division)

此位域指示定时器时钟 (tim\_ker\_ck) 频率以及数字滤波器 (tim\_etr\_in, tim\_tix) 所使用的采样时钟 ( $t_{DTS}$ ) 之间的分频比

00:  $t_{DTS} = t_{tim\_ker\_ck}$

01:  $t_{DTS} = 2 \times t_{tim\_ker\_ck}$

10:  $t_{DTS} = 4 \times t_{tim\_ker\_ck}$

11: 保留, 不要设置成此值

Bit 7 **ARPE**: 自动重载预装载使能 (Auto-reload preload enable)

0: TIM15\_ARR 寄存器不进行缓冲;

1: TIM15\_ARR 寄存器进行缓冲。

Bit 6:4 保留, 必须保持复位值。

Bit 3 **OPM**: 单脉冲模式 (One pulse mode)

0: 计数器在发生更新事件时不会停止计数;

1: 计数器在发生下一更新事件时停止计数 (将 CEN 位清零)

Bit 2 **URS**: 更新请求源 (Update request source)

此位由软件置 1 和清零, 用以选择 UEV 事件源。

0: 使能时, 所有以下事件都会生成更新中断。此类事件包括:

– 计数器上溢/下溢

– 将 UG 位置 1

– 通过从模式控制器生成的更新事件

1: 使能时, 只有计数器上溢/下溢会生成更新中断。

#### Bit 1 UDIS: 更新禁止 (Update disable)

此位由软件置 1 和清零，用以使能/禁止 UEV 事件生成。

0: 使能 UEV。更新 (UEV) 事件可通过以下事件之一生成：

- 计数器上出/下溢
- 将 UG 位置 1
- 通过从模式控制器生成的更新事件

然后更新影子寄存器的值。

1: 禁止 UEV。不会生成更新事件，各影子寄存器的值 (ARR、PSC 和 CCRx) 保持不变。但如果将 UG 位置 1，或者从模式控制器接收到硬件复位，则会重新初始化计数器和预分频器。

#### Bit 0 CEN: 计数器使能 (Counter enable)

0: 禁止计数器；

1: 使能计数器。

注：只有事先通过软件将 CEN 位置 1，才可以使用外部时钟、门控模式。而触发模式可通过硬件自动将 CEN 位置 1。

### 25.7.2 TIM15 控制寄存器 2 (TIM15\_CR2)

偏移地址: 0x004

复位值: 0x0000

| 15       | 14 | 13 | 12 | 11 | 10   | 9     | 8    | 7    | 6        | 5  | 4    | 3    | 2   | 1  | 0    |
|----------|----|----|----|----|------|-------|------|------|----------|----|------|------|-----|----|------|
| Reserved |    |    |    |    | OIS2 | OIS1N | OIS1 | TI1S | MMS[2:0] |    | CCDS | CCUS | Res |    | CCPC |
|          |    |    |    |    | rw   | rw    | rw   | rw   | rw       | rw | rw   | rw   | rw  | rw | rw   |

Bit 15:11 保留，必须保持复位值。

#### Bit 10 OIS2: 输出空闲状态 2 (OC2 输出) (Output idle state 2 (tim\_oc2 output))

0: 当 MOE=0 时, tim\_oc2=0

1: 当 MOE=0 时, tim\_oc2=1

注：只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别 1、2 或 3，此位即无法修改。

#### Bit 9 OIS1N: 输出空闲状态 1 (OC1N 输出) (Output idle state 1 (tim\_oc1n output))

0: 当 MOE=0 时, 经过死区时间后 tim\_oc1n=0

1: 当 MOE=0 时, 经过死区时间后 tim\_oc1n=1

注：只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别 1、2 或 3，此位即无法修改。

#### Bit 8 OIS1: 输出空闲状态 1 (OC1 输出) (Output idle state 1 (tim\_oc1 output))

0: 当 MOE=0 时, (如果 OC1N 有效, 则经过死区时间之后) tim\_oc1=0

1: 当 MOE=0 时, (如果 OC1N 有效, 则经过死区时间之后) tim\_oc1=1

注：只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别 1、2 或 3，此位即无法修改。

Bit 7 **TI1S**: tim\_ti1 选择 (tim\_ti1 selection)

0: tim\_ti1\_in[15:0]多路复用器输出连接到 tim\_ti1\_in 输入

1: tim\_ti1\_in[15:0]和 tim\_ti2\_in[15:0]多路复用器输出连接到 tim\_ti1\_in 输入 (异或组合)

Bit 6:4 **MMS[2:0]**: 主模式选择 (Master mode selection)

这些位可选择主模式下将要发送到从定时器以实现同步的信息 (tim\_trgo)。这些位的组合如下:

000: **复位** - TIM15\_EGR 寄存器中的 UG 位用作触发输出 (tim\_trgo)。如果复位由触发输入生成 (从模式控制器配置为复位模式), 则 tim\_trgo 上的信号相比实际复位会有延迟。

001: **使能** - 计数器使能信号 CNT\_EN 用作触发输出 (tim\_trgo)。该触发输出可用于同时启动多个定时器, 或者控制在一段时间内使能从定时器。计数器使能信号可由 CEN 控制位产生。当配置为门控模式时, 也可由触发输入产生。当计数器使能信号由触发输入控制时, tim\_trgo 上会存在延迟, 选择主/从模式时除外 (请参见 TIM15\_SMCR 寄存器中 MSM 位的说明)。

010: **更新** - 选择更新事件作为触发输出 (tim\_trgo)。例如, 主定时器可用作从定时器的预分频器。

011: **比较脉冲** - 一旦发生输入捕获或比较匹配事件, 当 CC1IF 被置 1 时 (即使已为高电平), 触发输出都会发送一个正脉冲。(tim\_trgo)。

100: **比较** - OC1REF 信号用作触发输出 (tim\_trgo)

101: **比较** - OC2REF 信号用作触发输出 (tim\_trgo)

Bit 3 **CCDS**: 捕获/比较 DMA 选择 (Capture/compare DMA selection)

0: 发生 CCx 事件时发送 CCx DMA 请求

1: 发生更新事件时发送 CCx DMA 请求

Bit 2 **CCUS**: 捕获/比较控制更新选择 (Capture/compare control update selection)

0: 如果捕获/比较控制位(CCPC=1)进行预装载, 仅通过将 COMG 位置 1 来对这些位进行更新;

1: 如果捕获/比较控制位(CCPC=1)进行预装载, 可通过将 COMG 位置 1 或 TRGI 的上升沿对这些位进行更新。

*注: 此位仅对具有互补输出的通道有效。*

Bit 1 保留, 必须保持复位值。

Bit 0 **CCPC**: 捕获/比较预装载控制 (Capture/compare preloaded control)

0: CCxE, CCxNE 和 OCxM 位未进行预装载;

1: CCxE, CCxNE 和 OCxM 位进行了预装载, 写入这些位后, 仅当发生换向事件 (COM) (COMG 位置 1 或在 TRGI 上检测到上升沿, 取决于 CCUS 位) 时才会对这些位进行更新。

*注: 此位仅对具有互补输出的通道有效。*

### 25.7.3 TIM15 从模式控制寄存器 (TIM15\_SMCR)

偏移地址: 0x008

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |     |         |         |    |          |     |          |        |  |
|----------|----|----|----|----|----|----|----|-----|---------|---------|----|----------|-----|----------|--------|--|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23  | 22      | 21      | 20 | 19       | 18  | 17       | 16     |  |
| Reserved |    |    |    |    |    |    |    |     |         | TS[4:3] |    | Reserved |     |          | SMS[3] |  |
|          |    |    |    |    |    |    |    |     |         | rw      | rw |          |     |          | rw     |  |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7   | 6       | 5       | 4  | 3        | 2   | 1        | 0      |  |
| Reserved |    |    |    |    |    |    |    | MSM | TS[2:0] |         |    |          | Res | SMS[2:0] |        |  |
|          |    |    |    |    |    |    |    | rw  | rw      | rw      | rw | rw       |     | rw       |        |  |

Bit 31:22 保留, 必须保持复位值。

Bit 19:17 保留, 必须保持复位值。

Bit 15:8 保留, 必须保持复位值。

Bit 7 **MSM**: 主/从模式 (Master/slave mode)

0: 不执行任何操作;

1: 当前定时器的触发输入事件 (tim\_trgi) 的动作被推迟, 以使当前定时器与其从定时器实现完美同步 (通过 tim\_trgo)。此设置适用于由单个外部事件对多个定时器进行同步的情况。

Bit 21,20,6,5,4 **TS[4:0]**: 触发选择 (Trigger selection)

此位域可选择将要用于同步计数器的触发输入。

00000: 内部触发 0 (tim\_itr0)

00001: 内部触发 1 (tim\_itr1)

00010: 内部触发 2 (tim\_itr2)

00011: 内部触发 3 (tim\_itr3)

00100: tim\_ti1 边沿检测器 (tim\_ti1f\_ed)

00101: 滤波后的定时器输入 1 (tim\_ti1fp1)

00110: 滤波后的定时器输入 2 (tim\_ti1fp2)

00111: 保留

01000: 内部触发器 4 (tim\_itr4)

01001: 内部触发器 5 (tim\_itr5)

01010: 内部触发器 6 (tim\_itr6)

01011: 内部触发器 7 (tim\_itr7)

01100: 内部触发器 8 (tim\_itr8)

01101: 内部触发器 9 (tim\_itr9)

01110: 内部触发器 10 (tim\_itr10)

01111: 内部触发器 11 (tim\_itr11)

10000: 内部触发器 12 (tim\_itr12)

10001: 内部触发器 13 (tim\_itr13)

10010: 内部触发器 14 (tim\_itr14)

10011: 内部触发器 15 (tim\_itr15)

其他: 保留

有关定时器 ITRx 含义的详细信息, 请参见章节: TIM15/TIM16/TIM17 引脚和内部信号。

注: 这些位只能在未使用的情况下 (例如, SMS=000 时) 进行更改, 以避免转换时出现错误的边沿检测。



Bit 3 保留，必须保持复位值。

Bit 16, 2, 1, 0 **SMS[2:0]**: 从模式选择 (Slave mode selection)

当选择了外部信号，触发信号 (tim\_trgi) 的有效边沿与选中的外部输入极性相关 (见输入控制寄存器和控制寄存器的说明)

0000: 关闭从模式 - 如果 CEN=1, 则预分频器直接由内部时钟驱动。

0001: 保留

0010: 保留

0011: 保留

0100: **复位模式** - 选中的触发输入 (tim\_trgi) 的上升沿重新初始化计数器，并且产生一个更新寄存器的信号。

0101: **门控模式** - 当触发输入 (tim\_trgi) 为高时，计数器的时钟开启。一旦触发输入变为低，则计数器停止 (但不复位)。计数器的启动和停止都是受控的。

0110: **触发模式** - 计数器在触发输入 tim\_trgi 的上升沿启动 (但不复位)，只有计数器的启动是受控的。

0111: **外部时钟模式 1** - 选中的触发输入 (TRGI) 的上升沿驱动计数器。

1000: **联合重置+触发模式** - 当选择的触发输入 (tim\_trgi) 上升沿出现时，会重新初始化计数器，生成寄存器的更新并启动计数器。

1001: **联合门控+重置模式** - 当触发输入 (tim\_trgi) 为高电平时，计数器时钟被使能。一旦触发变为低电平，计数器就会停止并重置。这样，计数器的启动和停止都得到了控制。

其他: 保留

注: 如果选择了 tim\_ti1f\_ed 作为触发输入 (TS=00100), 则不能使用门控模式。事实上, tim\_ti1f\_ed 为每个 tim\_ti1f 上的转换输出一个脉冲, 而门控模式检查触发信号的电平。

注: 接收 tim\_trgo 信号的从设备 (定时器、ADC 等) 的时钟必须在从主定时器接收事件之前使能, 并且时钟频率 (预分频器) 在从主定时器接收触发时不能改变。

## 25.7.4 TIM15 DMA/中断使能寄存器 (TIM15\_DIER)

偏移地址: 0x00C

复位值: 0x0000

| 15  | 14  | 13    | 12       | 11 | 10 | 9     | 8   | 7   | 6   | 5     | 4        | 3 | 2 | 1     | 0     |     |
|-----|-----|-------|----------|----|----|-------|-----|-----|-----|-------|----------|---|---|-------|-------|-----|
| Res | TDE | COMDE | Reserved |    |    | CC1DE | UDE | BIE | TIE | COMIE | Reserved |   |   | CC2IE | CC1IE | UIE |
|     | rw  | rw    |          |    |    | rw    | rw  | rw  | rw  | rw    |          |   |   | rw    | rw    | rw  |

Bit 15 保留，必须保持复位值。

Bit 14 **TDE**: 触发 DMA 请求使能 (Trigger DMA request enable)

0: 禁止触发 DMA 请求;

1: 使能触发 DMA 请求。

Bit 13 **COMDE**: COM DMA 请求使能 (COM DMA request enable)

0: 禁止换向 DMA 请求;

1: 使能换向 DMA 请求。

Bit 12:10 保留，必须保持复位值。



Bit 9 **CC1DE**: 捕获/比较 1 DMA 请求使能 (Capture/Compare 1 DMA request enable)

0: 禁止 CC1 DMA 请求;

1: 使能 CC1 DMA 请求。

Bit 8 **UDE**: 更新 DMA 请求使能 (Capture/Compare 1 DMA request enable)

0: 禁止更新 DMA 请求;

1: 使能更新 DMA 请求。

Bit 7 **BIE**: 刹车中断使能 (Break interrupt enable)

0: 禁止刹车中断;

1: 使能刹车中断。

Bit 6 **TIE**: 触发信号 (TGRI) 中断使能 (Trigger interrupt enable)

0: 禁止触发信号 (TGRI) 中断;

1: 使能触发信号 (TGRI) 中断。

Bit 5 **COMIE**: COM 中断使能 (COM interrupt enable)

0: 禁止 COM 中断;

1: 使能 COM 中断。

Bit 4:3 保留, 必须保持复位值。

Bit 2 **CC2IE**: 捕获/比较 2 中断使能 (Capture/Compare 2 interrupt enable)

0: 禁止 CC2 中断;

1: 使能 CC2 中断。

Bit 1 **CC1IE**: 捕获/比较 1 中断使能 (Capture/Compare 1 interrupt enable)

0: 禁止 CC1 中断;

1: 使能 CC1 中断。

Bit 0 **UIE**: 更新中断使能 (Update interrupt enable)

0: 禁止更新中断;

1: 使能更新中断。

## 25.7.5 TIM15 状态寄存器 (TIM15\_SR)

偏移地址: 0x010

复位值: 0x0000

| 15       | 14 | 13 | 12 | 11 | 10    | 9     | 8   | 7     | 6     | 5     | 4        | 3 | 2 | 1     | 0     |     |
|----------|----|----|----|----|-------|-------|-----|-------|-------|-------|----------|---|---|-------|-------|-----|
| Reserved |    |    |    |    | CC2OF | CC1OF | Res | BIF   | TIF   | COMIF | Reserved |   |   | CC2IF | CC1IF | UIF |
|          |    |    |    |    | rc_w0 | rc_w0 |     | rc_w0 | rc_w0 | rc_w0 |          |   |   | rc_w0 |       |     |

Bit 15:11 保留, 必须保持复位值。

Bit 10 **CC2OF**: 捕获/比较 2 重复捕获标志

参考 CC1OF 的描述

Bit 9 **CC1OF**: 捕获/比较 1 重复捕获标志

仅当将相应通道配置为输入捕获模式时, 此标志位才会由硬件置 1。通过软件写入“0”可将该位清零。

0: 未检测到重复捕获

1: TIM15\_CCR1 寄存器中已捕获到计数器值且 CC1IF 标志已置 1。

Bit 8 保留, 必须保持复位值。

**Bit 7 BIF: 刹车中断标志 (Break interrupt flag)**

只要 tim\_brk 输入变为有效状态, 此标志便由硬件置 1。刹车输入无效后可通过软件对其清零。

0: 未发生刹车事件。

1: 在刹车输入上检测到有效电平。

**Bit 6 TIF: 触发中断标志 (Trigger interrupt flag)**

在除门控模式以外的所有模式下, 当使能从模式控制器后在 tim\_trgi 输入上检测到有效边沿时, 该标志将由硬件置 1。选择门控模式时, 该标志将在计数器启动或停止时置 1。但需要通过软件清零。

0: 未发生触发事件。

1: 触发中断挂起。

**Bit 5 COMIF: COM 中断标志 (COM interrupt flag)**

此标志在发生 COM 事件时 (捕获/比较控制位 CCxE、CCxNE 和 OCxM 已更新时) 由硬件置 1。但需要通过软件清零。

0: 未发生 COM 事件。

1: COM 中断挂起。

**Bit 4:3 保留, 必须保持复位值。****Bit 2 CC2IF: 捕获/比较 2 中断标志 (Capture/Compare 2 interrupt flag)**

参考 CC1IF 描述。

**Bit 1 CC1IF: 捕获/比较 1 中断标志 (Capture/Compare 1 interrupt flag)**

此标志由硬件设置, 它可以通过软件 (输入捕获或输出比较模式) 或读取 TIM15\_CCR1 寄存器 (仅输入捕获模式) 来清除。

0: 未比较匹配/无输入捕获发生

1: 发生比较匹配或输入捕获

**如果通道 CC1 配置为输出:** 当计数器与比较值匹配时, 此标志由硬件置 1, 当 TIM15\_CCR1 的值大于 TIM15\_ARR 的值时, CC1IF 位将在计数器发生上溢 (递增计数模式和增减计数模式下) 或下溢 (递减计数模式下) 时变为高电平。在中心对齐模式下有 3 种可能的标志设置选项, 请参考 TIM15\_CR1 寄存器中的 CMS 位以获取完整描述。  
**如果通道 CC1 配置为输入:** 当在 TIM15\_CCR1 寄存器中捕获到计数器值时 (在 IC1 上检测到边沿, 具体取决于在 TIM15\_CCER 中通过 CC1P 和 CC1NP 位设置定义的边沿灵敏度), 会设置此位。

**Bit 0 UIF: 更新中断标志 (Update interrupt flag)**

该位在发生更新事件时通过硬件置 1。但需要通过软件清零。

0: 未发生更新。

1: 更新中断挂起。该位在以下情况下更新寄存器时由硬件置 1:

- TIM15\_CR1 寄存器中的 UDIS=0, 并且重复计数器值上溢或下溢时 (重复计数器=0 时更新)。
- TIM15\_CR1 寄存器中的 URS=0 且 UDIS=0, 并且由软件使用 TIM15\_EGR 寄存器中的 UG 位重新初始化 CNT 时。
- TIM15\_CR1 寄存器中的 URS=0 且 UDIS=0, 并且 CNT 由触发事件重新初始化时 (参考 TIM15 从模式控制寄存器 (TIM15\_SMCR))。

## 25.7.6 TIM15 事件产生寄存器 (TIM15\_EGR)

偏移地址: 0x014

复位: 0x0000

| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7  | 6  | 5    | 4        | 3 | 2    | 1    | 0  |
|----------|----|----|----|----|----|---|---|----|----|------|----------|---|------|------|----|
| Reserved |    |    |    |    |    |   |   | BG | TG | COMG | Reserved |   | CC2G | CC1G | UG |
|          |    |    |    |    |    |   |   | w  | w  | rw   |          |   | w    | w    | w  |

Bit 15:8 保留, 必须保持复位值。

Bit 7 **BG**: 刹车生成 (Break generation)

此位由软件置 1 以生成事件, 并由硬件自动清零。

0: 不执行任何操作;

1: 生成刹车事件。MOE 位清零且 BIF 标志置 1。使能后可发生相关中断或 DMA 传输事件。

Bit 6 **TG**: 生成触发信号 (Trigger generation)

此位由软件置 1 以生成事件, 并由硬件自动清零。

0: 不执行任何操作;

1: TIMx\_SR 寄存器中的 TIF 标志置 1。使能后可发生相关中断事件或 DMA 传输。

Bit 5 **COMG**: 捕获/比较控制更新生成 (Capture/Compare control update generation)

该位可通过软件置 1, 并由硬件自动清零

0: 不执行任何操作;

1: CCPC 位置 1 时, 可更新 CCxE、CCxNE 和 OCxM 位。

注: 此位仅对具有互补输出的通道有效。

Bit 4:3 保留, 必须保持复位值。

Bit 2 **CC2G**: 捕获/比较 2 生成 (Capture/Compare 2 generation)

参考 CC1G 描述。

Bit 1 **CC1G**: 捕获/比较 1 生成 (Capture/Compare 1 generation)

此位由软件置 1 以生成事件, 并由硬件自动清零。

0: 不执行任何操作;

1: 通道 1 上生成捕获/比较事件:

**若通道 CC1 配置为输出:**

使能时, CC1IF 标志置 1 并发送相应的中断或 DMA 请求。

**如果通道 CC1 配置为输入:**

TIMx\_CCR1 寄存器中将捕获到计数器当前值。使能时, CC1IF 标志置 1 并发送相应的中断或 DMA 请求。如果 CC1IF 标志已为高电平, CC1OF 标志将置 1。

Bit 0 **UG**: 更新生成 (Update generation)

该位可通过软件置 1, 并由硬件自动清零。

0: 不执行任何操作;

1: 重新初始化计数器并生成一个寄存器更新事件。请注意, 预分频器计数器也将清零 (但预分频比不受影响)。

### 25.7.7 TIM15 捕获/比较模式寄存器 1 [复用] (TIM15\_CCMR1)

偏移地址：0x018

复位值：0x0000 0000

相同的寄存器可以用于输入捕获模式（本节）或输出比较模式（下节）。通道方向通过配置相应的 CCxS 位进行定义。此寄存器的所有其它位在输入模式和输出模式下的功能均不同。可以独立组合这两种模式（例如，通道 1 用作输入捕获模式而通道 2 用作输出比较模式）

|           |    |    |    |             |    |           |    |           |    |    |    |             |    |           |    |
|-----------|----|----|----|-------------|----|-----------|----|-----------|----|----|----|-------------|----|-----------|----|
| 31        | 30 | 29 | 28 | 27          | 26 | 25        | 24 | 23        | 22 | 21 | 20 | 19          | 18 | 17        | 16 |
| Reserved  |    |    |    |             |    |           |    |           |    |    |    |             |    |           |    |
| 15        | 14 | 13 | 12 | 11          | 10 | 9         | 8  | 7         | 6  | 5  | 4  | 3           | 2  | 1         | 0  |
| IC2F[3:0] |    |    |    | IC2PSC[1:0] |    | CC2S[1:0] |    | IC1F[3:0] |    |    |    | IC1PSC[1:0] |    | CC1S[1:0] |    |
| rw        | rw | rw | rw | rw          | rw | rw        | rw | rw        | rw | rw | rw | rw          | rw | rw        | rw |

#### 输入捕获模式

Bit 31:16 保留，始终读为 0。

Bit 15:12 **IC2F[3:0]**：输入捕获 2 滤波器（Input capture 2 filter）

Bit 11:10 **IC2PSC[1:0]**：输入/捕获 2 预分频器（input capture 2 prescaler）

Bit 9:8 **CC2S[1:0]**：捕获/比较 2 选择（Capture/compare 2 selection）

此位域定义通道方向（输入/输出）以及所使用的输入。

00：CC2 通道被配置为输出；

01：CC2 通道被配置为输入，tim\_ic2 映射在 tim\_ti2 上；

10：CC2 通道被配置为输入，tim\_ic2 映射在 tim\_ti1 上；

11：CC2 通道被配置为输入，tim\_ic2 映射在 tim\_trc 上。此模式仅在通过 TS 位（TIM15\_SMCR 寄存器）选择内部触发输入时有效

注：仅当通道关闭时（TIM15\_CCER 中的 CC2E = 0），才可向 CC2S 位写入数据。

#### Bit 7:4 IC1F[3:0]: 输入捕获 1 滤波器 (Input capture 1 filter)

此位域可定义 tim\_ti1 输入的采样频率和适用于 tim\_ti1 的数字滤波器带宽。数字滤波器由事件计数器组成，每 N 个事件才视为一个有效边沿：

0000: 无滤波器，按  $f_{DTS}$  频率进行采样

0001:  $f_{SAMPLING} = f_{tim\_ker\_ck}$ ,  $N=2$ 。

0010:  $f_{SAMPLING} = f_{tim\_ker\_ck}$ ,  $N=4$ 。

0011:  $f_{SAMPLING} = f_{tim\_ker\_ck}$ ,  $N=8$ 。

0100:  $f_{SAMPLING} = f_{DTS}/2$ ,  $N=6$ 。

0101:  $f_{SAMPLING} = f_{DTS}/2$ ,  $N=8$ 。

0110:  $f_{SAMPLING} = f_{DTS}/4$ ,  $N=6$ 。

0111:  $f_{SAMPLING} = f_{DTS}/4$ ,  $N=8$ 。

1000:  $f_{SAMPLING} = f_{DTS}/8$ ,  $N=6$ 。

1001:  $f_{SAMPLING} = f_{DTS}/8$ ,  $N=8$ 。

1010:  $f_{SAMPLING} = f_{DTS}/16$ ,  $N=5$ 。

1011:  $f_{SAMPLING} = f_{DTS}/16$ ,  $N=6$ 。

1100:  $f_{SAMPLING} = f_{DTS}/16$ ,  $N=8$ 。

1101:  $f_{SAMPLING} = f_{DTS}/32$ ,  $N=5$ 。

1110:  $f_{SAMPLING} = f_{DTS}/32$ ,  $N=6$ 。

1111:  $f_{SAMPLING} = f_{DTS}/32$ ,  $N=8$ 。

#### Bit 3:2 IC1PSC[1:0]: 输入/捕获 1 预分频器 (Input capture 1 prescaler)

此位域定义 CC1 输入 (tim\_ic1) 的预分频比。

只要 CC1E=0 (TIM15\_CCER 寄存器)，预分频器便立即复位。

00: 无预分频器，捕获输入上每检测到一个边沿便执行捕获

01: 每发生 2 个事件便执行一次捕获；

10: 每发生 4 个事件便执行一次捕获；

11: 每发生 8 个事件便执行一次捕获。

#### Bit 1:0 CC1S[1:0]: 捕获/比较 1 选择 (Capture/Compare 1 selection)

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00: CC1 通道被配置为输出；

01: CC1 通道被配置为输入，tim\_ic1 映射在 tim\_ti1 上；

10: CC1 通道被配置为输入，tim\_ic1 映射在 tim\_ti2 上；

11: CC1 通道被配置为输入，tim\_ic1 映射在 tim\_trc 上。此模式仅在通过 TS 位 (TIM15\_SMCR 寄存器) 选择内部触发输入时有效

注：仅当通道关闭时 (TIM15\_CCER 中的 CC1E = 0)，才可向 CC1S 位写入数据。

### 25.7.8 TIM15 捕获/比较模式寄存器 1 [复用] (TIM15\_CCMR1)

偏移地址: 0x018

复位值: 0x0000 0000

相同的寄存器可以用于输出比较模式 (本节) 或输入捕获模式 (上节)。通道方向通过配置相应的 CCxS 位进行定义。此寄存器的所有其它位在输入模式和输出模式下的功能均不同。可以独立组合这两种模式 (例如，通道 1 用作输入捕获模式而通道 2 用作输出比较模式)

| 31       | 30        | 29 | 28 | 27    | 26    | 25        | 24      | 23       | 22        | 21 | 20 | 19    | 18    | 17        | 16      |
|----------|-----------|----|----|-------|-------|-----------|---------|----------|-----------|----|----|-------|-------|-----------|---------|
| Reserved |           |    |    |       |       |           | OC2M[3] | Reserved |           |    |    |       |       |           | OC1M[3] |
|          |           |    |    |       |       |           | rw      |          |           |    |    |       |       |           | rw      |
| 15       | 14        | 13 | 12 | 11    | 10    | 9         | 8       | 7        | 6         | 5  | 4  | 3     | 2     | 1         | 0       |
| OC2CE    | OC2M[2:0] |    |    | OC2PE | OC2FE | CC2S[1:0] |         | OC1CE    | OC1M[2:0] |    |    | OC1PE | OC1FE | CC1S[1:0] |         |
| rw       | rw        | rw | rw | rw    | rw    | rw        | rw      | rw       | rw        | rw | rw | rw    | rw    | rw        | rw      |

## 输出比较模式

Bit 31:25 保留，必须保持复位值。

Bit 23:17 保留，必须保持复位值。

Bit 15 **OC2CE**: 输出比较 2 清零使能 (Output Compare 2 clear enable)

Bit 24, 14:12 **OC2M[2:0]**: 输出比较 2 模式 (Output Compare 2 mode)

Bit 11 **OC2PE**: 输出比较 2 预装载使能 (Output Compare 2 preload enable)

Bit 10 **OC2FE**: 输出比较 2 快速使能 (Output Compare 2 fast enable)

Bit 9:8 **CC2S[1:0]**: 捕获/比较 2 选择 (Capture/Compare 2 selection)

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00: CC2 通道配置为输出

01: CC2 通道配置为输入, tim\_ic2 映射到 tim\_ti2 上

10: CC2 通道配置为输入, tim\_ic2 映射到 tim\_ti1 上

11: CC2 通道配置为输入, tim\_ic2 映射到 tim\_trc 上。此模式仅在通过 TS 位

(TIM15\_SMCR 寄存器) 选择内部触发输入时有效

注: 仅当通道关闭时 (TIM15\_CCER 中的 CC2E = 0), 才可向 CC2S 位写入数据

Bit 7 **OC1CE**: 输出比较 1 清零使能 (Output Compare 1 clear enable)

0: tim\_oc1ref 不受 tim\_ocref\_clr\_int 输入影响;

1: tim\_oc1ref 在 tim\_ocref\_clr\_int 输入中被检测到高电平, tim\_oc1ref 立即清零。



Bit 16, 6:4 **OC1M[3:0]**: 输出比较 1 模式 (Output Compare 1 mode)

这些位定义提供 tim\_oc1 和 tim\_oc1n 的输出参考信号 tim\_oc1ref 的行为。tim\_oc1ref 为高电平有效, 而 tim\_oc1 和 tim\_oc1n 的有效电平则取决于 CC1P 位和 CC1NP 位。

0000: **冻结** — 输出比较寄存器 TIM15\_CCR1 与计数器 TIM15\_CNT 进行比较不会对输出造成任何影响。(该模式用于生成时基)。

0001: 将通道 1 设置为匹配时输出有效电平。当计数器 TIM15\_CNT 与捕获/比较寄存器 1 (TIM15\_CCR1) 匹配时, tim\_oc1ref 信号强制变为高电平。

0010: 将通道 1 设置为匹配时输出无效电平。当计数器 TIM15\_CNT 与捕获/比较寄存器 1 (TIM15\_CCR1) 匹配时, tim\_oc1ref 信号强制变为低电平。

0011: **翻转** - TIM15\_CNT=TIM15\_CCR1 时, tim\_oc1ref 发生翻转。

0100: **强制变为无效电平** - tim\_oc1ref 强制变为低电平。

0101: **强制变为有效电平** - tim\_oc1ref 强制变为高电平。

0110: **PWM 模式 1** - 在递增计数模式下, 只要 TIM15\_CNT<TIM15\_CCR1, 通道 1 便为有效状态, 否则为无效状态。

0111: **PWM 模式 2** - 在递增计数模式下, 只要 TIM15\_CNT<TIM15\_CCR1, 通道 1 便为无效状态, 否则为有效状态。

1000: **可重触发 OPM 模式 1** - 在向上计数模式下, 通道在检测到触发事件 (tim\_trgi 信号) 之前一直处于活动状态。然后, 会像 PWM 模式 1 那样执行比较, 并在下次更新时使通道再次处于活动状态。在向下计数模式下, 通道在检测到触发事件 (tim\_trgi 信号) 之前一直处于非活动状态。然后, 会像 PWM 模式 1 那样执行比较, 并在下次更新时使通道再次处于非活动状态。

1001: **可重触发 OPM 模式 2** - 在向上计数模式下, 通道在检测到触发事件 (tim\_trgi 信号) 之前一直处于非活动状态。然后, 会像 PWM 模式 2 那样执行比较, 并在下次更新时使通道再次处于非活动状态。在向下计数模式下, 通道在检测到触发事件 (tim\_trgi 信号) 之前一直处于活动状态。然后, 会像 PWM 模式 1 那样执行比较, 并在下次更新时使通道再次处于活动状态。

1010: 保留

1011: 保留

1100: **组合 PWM 模式 1** - tim\_oc1ref 的行为与 PWM 模式 1 相同。tim\_oc1refc 是 tim\_oc1ref 和 tim\_oc2ref 之间的逻辑或。

1101: **组合 PWM 模式 2** - tim\_oc1ref 的行为与 PWM 模式 2 相同。tim\_oc1refc 是 tim\_oc1ref 和 tim\_oc2ref 之间的逻辑与。

1110: 保留

1111: 保留

注: 只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别 3 且 CC1S= "00" (通道配置为输出), 这些位即无法修改。

在 PWM 模式下, 仅当比较结果发生改变或输出比较模式由“冻结”模式切换到“PWM”模式时, tim\_ocref\_clr 电平才会发生更改。

对于具有互补输出的通道, 此位域是预加载的。如果 TIM15\_CR2 寄存器中 CCPC 位被设置, 则 OC1M 有效位仅在生成 COM 事件时从预加载位获取新值。



Bit 3 **OC1PE**: 输出比较 1 预装载使能 (Output Compare 1 preload enable)

0: 禁止与 TIM15\_CCR1 相关的预装载寄存器。可随时向 TIM15\_CCR1 写入数据, 写入后将立即使用新值。

1: 使能与 TIM15\_CCR1 相关的预装载寄存器。可读/写访问预装载寄存器。

TIM15\_CCR1 预装载值在每次生成更新事件时都会装载到有效寄存器中。

注: 只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别 3 且 CC1S=00 (通道配置为输出), 这些位即无法修改。

Bit 2 **OC1FE**: 输出比较 1 快速使能 (Output Compare 1 fast enable)

此位减少了触发事件和定时器输出上的转换之间的延迟。它必须在单脉冲模式 (在 TIM15\_CR1 寄存器中设置 OPM 位) 中使用, 以使输出脉冲在启动触发后尽快开始。

0: 即使触发开启, CC1 也将根据计数器和 CCR1 值正常工作。触发输入出现边沿时, 激活 CC1 输出的最短延迟时间为 5 个时钟周期。

1: 触发输入上出现有效边沿相当于 CC1 输出上的比较匹配。随后, 无论比较结果如何, OC 都设置为比较电平。采样触发输入和激活 CC1 输出的延迟时间缩短为 3 个时钟周期。仅当通道配置为 PWM1 或 PWM2 模式时, OCFE 才会起作用。

Bit 1:0 **CC1S[1:0]**: 捕获/比较 1 选择 (Capture/Compare 1 selection)

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00: CC1 通道配置为输出

01: CC1 通道配置为输入, tim\_ic1 映射到 tim\_ti1 上

10: CC1 通道配置为输入, tim\_ic1 映射到 tim\_ti2 上

11: CC1 通道配置为输入, tim\_ic1 映射到 tim\_trc 上。此模式仅在通过 TS 位 (TIM15\_SMCR 寄存器) 选择内部触发输入时有效

注: 仅当通道关闭时 (TIM15\_CCER 中的 CC1E = 0), 才可向 CC1S 位写入数据

## 25.7.9 TIM15 捕获/比较使能寄存器 (TIM15\_CCER)

偏移地址: 0x020

复位值: 0x0000

| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7     | 6   | 5    | 4    | 3     | 2     | 1    | 0    |
|----------|----|----|----|----|----|---|---|-------|-----|------|------|-------|-------|------|------|
| Reserved |    |    |    |    |    |   |   | CC2NP | Res | CC2P | CC2E | CC1NP | CC1NE | CC1P | CC1E |
|          |    |    |    |    |    |   |   | rw    |     | rw   | rw   | rw    | rw    | rw   | rw   |

Bit 15:8 保留, 必须保持复位值。

Bit 7 **CC2NP**: 输入/捕获 2 输出极性 (Capture/Compare 2 output polarity)

参考 CC1NP 的描述。

Bit 6 保留, 必须保持复位值。

Bit 5 **CC2P**: 输入/捕获 2 输出极性 (Capture/Compare 2 output polarity)

参考 CC1P 的描述。

Bit 4 **CC2E**: 输入/捕获 2 输出使能 (Capture/Compare 2 output enable)

参考 CC1E 的描述。

Bit 3 **CC1NP**: 输入/捕获 1 输出极性 (Capture/Compare 1 output polarity)

**CC1 通道配置为输出:**

0: OC1N 高电平有效。

1: OC1N 低电平有效。

**CC1 通道配置为输入:**

此位与 CC1P 配合使用, 用以定义 tim\_ti1fp1/ tim\_ti2fp1 的极性。请参见 CC1P 说明。

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 2 或 3 且 CC1S=00 (通道配置为输出), 此位立即变为不可写状态

注: 此位将在具有互补输出的通道上进行预装载。如果 TIM15\_CR2 寄存器中的 CCPC 位置 1, 则仅当生成换向事件时, CC1NP 有效位才会从预装载位获取新值。

Bit 2 **CC1NE**: 输入/捕获 1 互补输出使能 (Capture/Compare 1 complementary output enable)

0: 关闭 -- OC1N 未激活。OC1N 电平是 MOE、OSSI、OSSR、OIS1、OIS1N 和 CC1E 位的函数。

1: 开启 -- 在相应输出引脚上输出 OC1N 信号, 具体取决于 MOE、OSSI、OSSR、OIS1、OIS1N 和 CC1E 位。

Bit 1 **CC1P**: 输入/捕获 1 输出极性 (Capture/Compare 1 output polarity)

**CC1 通道配置为输出:**

0: OC1 高电平有效 (输出模式) / 边沿灵敏度选择 (输入模式, 参见以下)

1: OC1 低电平有效 (输出模式) / 边沿灵敏度选择 (输入模式, 参见以下)

**CC1 通道配置为输入, CC1NP/CC1P 位可针对触发或捕获操作选择 TI1FP1 和 TI2FP1 的极性。**

CC1NP=0, CC1P=0: 非反相/上升沿触发。电路对 TIxFP1 上升沿敏感 (在复位模式、外部时钟模式或触发模式下执行捕获或触发操作), TIxFP1 未反相 (在门控模式或编码器模式下执行触发操作)。

CC1NP=0, CC1P=1: 反相/下降沿触发。电路对 TIxFP1 下降沿敏感 (在复位模式、外部时钟模式或触发模式下执行捕获或触发操作), TIxFP1 反相 (在门控模式或编码器模式下执行触发操作)。

CC1NP=1, CC1P=1: 非反相/上升沿和下降沿均触发。电路对 TIxFP1 上升沿和下降沿都敏感 (在复位模式、外部时钟模式或触发模式下执行捕获或触发操作), TIxFP1 未反相 (在门控模式下执行触发操作)。编码器模式下不得使用此配置。

CC1NP=1, CC1P=0: 此配置是保留的, 不应该使用此配置。

注: 只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别 2 或 3, 此位立即变为不可写状态。

注: 此位将在具有互补输出的通道上进行预装载。如果 TIM15\_CR2 寄存器中的 CCPC 位置 1, 则仅当生成换向事件时, CC1P 有效位才会从预装载位获取新值。

Bit 0 **CC1E**: 输入/捕获 1 输出使能 (Capture/Compare 1 output enable)

0: 禁止捕获模式/OC1 未激活 (参见以下)

1: 使能捕获模式/ OC1 信号输出到对应的输出引脚

当 **CC1 通道配置为输出**, C1 的输出电平依赖于 MOE、OSSI、OSSR、OIS1、OIS1N 和 CC1NE 位的值, 不论 CC1E 位状态。请参考下表获得更多细节

表 25.14 带刹车功能的互补输出通道 tim\_oc1 和 tim\_oc1n 的控制位 (TIM15)

| 控制位      |           |           |           |            | 输出状态 <sup>(1)</sup>                           |                                                                                                                                  |
|----------|-----------|-----------|-----------|------------|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| MOE<br>位 | OSSI<br>位 | OSSR<br>位 | CCxE<br>位 | CCxNE<br>位 | tim_ocx 输出状态                                  | tim_ocxn 输出状态                                                                                                                    |
| 1        | X         | X         | 0         | 0          | 禁止输出（不由定时器驱动: Hi-Z）<br>tim_ocx=0, tim_ocxn=0  |                                                                                                                                  |
|          |           | 0         | 0         | 1          | 禁止输出（不由定时器驱动: Hi-Z）<br>tim_ocx=0              | tim_ocxref + 极性 tim_ocxn<br>=tim_ocxref 异或 CCxNP                                                                                 |
|          |           | 0         | 1         | 0          | tim_ocxref + 极性<br>tim_ocx=tim_ocxref 异或 CCxP | 禁止输出(不由定时器驱动: Hi-Z)<br>tim_ocxn=0                                                                                                |
|          |           | X         | 1         | 1          | OCREF+极性+死区                                   | OCREF 互补项（而非 OCREF）<br>+极性+死区                                                                                                    |
|          |           | 1         | 0         | 1          | 关闭状态（输出使能为无效状态）<br>tim_ocx=CCxP               | tim_ocxref + 极性 tim_ocxn<br>=tim_ocxref x 或 CCxNP                                                                                |
|          |           | 1         | 1         | 0          | tim_ocxref + 极性<br>tim_ocx=tim_ocxref 异或 CCxP | 关闭状态（输出使能为无效状态）<br>tim_ocxn=CCxNP                                                                                                |
| 0        | 0         | X         | X         | X          | 禁止输出（不由定时器驱动: Hi-Z）                           |                                                                                                                                  |
|          | 0         |           | 0         |            |                                               |                                                                                                                                  |
|          | 1         |           | 0         | 1          | 关闭状态（输出使能为无效状态）                               | 异步: tim_ocx=CCxP, tim_ocxn=CCxNP<br><br>然后如果存在时钟：在死区后 tim_ocx=OISx 且 tim_ocxn=OISxN,<br>从而假定 OISx 和 OISxN 在有效状态下与 OCX 和 OCxN 不对应 |
|          |           |           | 1         | 0          |                                               |                                                                                                                                  |
|          |           |           | 1         | 1          |                                               |                                                                                                                                  |

1. 如果一个通道的两个输出均未使用 (CCxE = CCxNE = 0), 则 OISx、OISxN、CCxP 和 CCxNP 位必须保持清零状态。

注: 与互补通道 tim\_ocx 和 tim\_ocxn 相连的外部 I/O 引脚的状态取决于通道 tim\_ocx 和 tim\_ocxn 的状态以及 GPIO 寄存器。

### 25.7.10 TIM15 计数器 (TIM15\_CNT)

偏移地址: 0x024

复位值: 0x0000 0000

| 31        | 30       | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|-----------|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| UIF       | Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| CPY       |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| r         |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15        | 14       | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CNT[15:0] |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw        | rw       | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31 UIFCPY: 复制 UIF

该位是 TIM15\_SR 寄存器中 UIF 位的一个只读副本。

Bit 30:16 保留, 必须保持复位值。

Bit 15:0 CNT[15:0]: 计数器的值 (Counter value)

在非抖动模式下 (DITHEN = 0)

该寄存器 (CNT[15:0]) 保存的是计数器的值。

在抖动模式下 (DITHEN = 1)

该寄存器 (CNT[15:0]) 只保存计数器的不抖动部分。抖动部分不可用。

### 25.7.11 TIM15 预分频器 (TIM15\_PSC)

偏移地址: 0x028

复位值: 0x0000

| 15        | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
|-----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| PSC[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw        | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 15:0 PSC[15:0]: 预分频器的值 (Prescaler value)

计数器时钟频率 ( $f_{tim\_cnt\_ck}$ ) 等于  $f_{tim\_psc\_ck} / (PSC[15:0] + 1)$ 。

PSC 包含每次发生更新事件 (包括计数器通过 TIM15\_EGR 寄存器中的 UG 位清零时, 或在配置为“复位模式”时通过触发控制器清零时) 时要装载到活动预分频器寄存器的值。

### 25.7.12 TIM15 自动重载寄存器 (TIM15\_ARR)

偏移地址: 0x02C

复位: 0x0000 FFFF

|           |    |    |    |    |    |    |    |    |    |    |    |            |    |    |    |
|-----------|----|----|----|----|----|----|----|----|----|----|----|------------|----|----|----|
| 31        | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19         | 18 | 17 | 16 |
| Reserved  |    |    |    |    |    |    |    |    |    |    |    | ARR[19:16] |    |    |    |
|           |    |    |    |    |    |    |    |    |    |    |    | rw         | rw | rw | rw |
| 15        | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3          | 2  | 1  | 0  |
| ARR[15:0] |    |    |    |    |    |    |    |    |    |    |    |            |    |    |    |
| rw        | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw         | rw | rw | rw |

Bit 31:20 保留, 必须保持复位值。

Bit 19:0 **ARR[19:0]**: 自动重载的值 (Auto-reload value)

ARR 为要装载到实际自动重载寄存器的值。

当自动重载值为空时, 计数器不工作。

在非抖动模式下 ( $DITHEN = 0$ )

该寄存器存储的是自动重新加载的值

在抖动模式下 ( $DITHEN = 1$ )

该寄存器存储的是 ARR[19:4]中的整数部分, 而 ARR[3:0]位域存储的是抖动的部分

### 25.7.13 TIM15 重复计数寄存器 (TIM15\_RCR)

偏移地址: 0x030

复位值: 0x0000

|          |    |    |    |    |    |   |   |          |    |    |    |    |    |    |    |
|----------|----|----|----|----|----|---|---|----------|----|----|----|----|----|----|----|
| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7        | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    |    |    |   |   | REP[7:0] |    |    |    |    |    |    |    |
|          |    |    |    |    |    |   |   | rw       | rw | rw | rw | rw | rw | rw | rw |

Bit 15:8 保留, 必须保持复位值。

Bit 7:0 **REP[7:0]**: 重复计数器的值 (Repetition counter value)

使能预装载寄存器时, 用户可通过这些位设置比较寄存器的更新频率 (即从预装载寄存器向活动寄存器周期性传输数据); 使能更新中断时, 也可设置更新中断的生成速率。

与 REP\_CNT 相关的减计数器每次计数到 0 时, 都将生成一个更新事件并且计数器从 REP 值重新开始计数。由于只有生成重复更新事件 U\_RC 时, REP\_CNT 才会重载 REP 值, 因此在生成下一重复更新事件之前, 无论向 TIMx\_RCR 寄存器写入何值都无影响。

这意味着 PWM 模式下 ( $REP+1$ ) 相当于:

- 边沿对齐模式下的 PWM 周期数;
- 中心对称模式下的 PWM 半周期数;

## 25.7.14 TIM15 捕获/比较寄存器 1 (TIM15\_CCR1)

偏移地址: 0x034

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|--------------|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19           | 18 | 17 | 16 |
| Reserved    |    |    |    |    |    |    |    |    |    |    |    | CCR1 [19:16] |    |    |    |
|             |    |    |    |    |    |    |    |    |    |    |    | rw           | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3            | 2  | 1  | 0  |
| CCR1 [15:0] |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw           | rw | rw | rw |

Bit 31:20 保留, 必须保持复位值。

Bit 19:0 **CCR1[19:0]**: 捕获/比较 1 值 (Capture/Compare 1 value)

**若 CC1 通道配置为输出:**

CCR1 为要装载到实际捕获/比较 1 寄存器的值 (预装载值)。

如果没有通过 TIM15\_CCMR1 寄存器中的 OC1PE 位来使能预装载功能, 写入的数值会被直接传输至当前寄存器中。否则只有发生更新事件时, 预装载值才会复制到活动捕获/比较 1 寄存器中。实际捕获/比较寄存器中包含要与计数器 TIM15\_CNT 进行比较并在 tim\_oc1 输出上发出信号的值。

在非抖动模式下 (DITHEN = 0)

该寄存器存储的是 CCR1[15:0]中的比较值。CCR1[19:16]位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 CCR1[19:4]中的整数部分。CCR1[3:0]位域存储的是抖动的部分。

**若 CC1 通道配置为输入:**

CCR1 为上一个输入捕获 1 事件 (tim\_ic1) 发生时的计数器值。TIM15\_CCR1 是只读的并且不可编程。

在非抖动模式下 (DITHEN = 0)

CCR1[15:0]位存储了捕获值。CCR1[19:16]位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储捕获在 CCR1[19:4]中。CCR1[3:0]位会被重置。



### 25.7.15 TIM15 捕获/比较寄存器 2 (TIM15\_CCR2)

偏移地址: 0x038

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|--------------|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19           | 18 | 17 | 16 |
| Reserved    |    |    |    |    |    |    |    |    |    |    |    | CCR2 [19:16] |    |    |    |
|             |    |    |    |    |    |    |    |    |    |    |    | rw           | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3            | 2  | 1  | 0  |
| CCR2 [15:0] |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw           | rw | rw | rw |

Bit 31:20 保留, 必须保持复位值。

Bit 19:0 CCR2[19:0]: 捕获/比较 2 值 (Capture/Compare 2 value)

若 CC2 通道配置为输出:

CCR2 为要装载到实际捕获/比较 2 寄存器的值 (预装载值)。

如果没有通过 TIM15\_CCMR1 寄存器中的 OC2PE 位来使能预装载功能, 写入的数值会被直接传输至当前寄存器中。否则只有发生更新事件时, 预装载值才会复制到活动捕获/比较 2 寄存器中。实际捕获/比较寄存器中包含要与计数器 TIM15\_CNT 进行比较并在 tim\_oc2 输出上发出信号的值。

在非抖动模式下 (DITHEN = 0)

该寄存器存储的是 CCR2[15:0]中的比较值。CCR2[19:16]位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 CCR2[19:4]中的整数部分。CCR2[3:0]位域存储的是抖动的部分。

若 CC2 通道配置为输入:

CCR2 为上一个输入捕获 2 事件 (tim\_ic2) 发生时的计数器值。TIM15\_CCR2 是只读的并且不可编程。

在非抖动模式下 (DITHEN = 0)

CCR2[15:0]位存储了捕获值。CCR2[19:16]位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储捕获在 CCR2[19:4]中。CCR2[3:0]位会被重置。

### 25.7.16 TIM15 刹车和死区寄存器 (TIM15\_BDTR)

偏移地址: 0x044

复位值: 0x0000 0000

|          |     |     |     |       |      |           |          |          |    |    |    |          |    |    |    |
|----------|-----|-----|-----|-------|------|-----------|----------|----------|----|----|----|----------|----|----|----|
| 31       | 30  | 29  | 28  | 27    | 26   | 25        | 24       | 23       | 22 | 21 | 20 | 19       | 18 | 17 | 16 |
| Reserved |     |     |     | BKBID | Res  | BKDSRM    | Reserved |          |    |    |    | BKF[3:0] |    |    |    |
|          |     |     |     | rw    |      | rw        |          |          |    |    |    | rw       | rw | rw | rw |
| 15       | 14  | 13  | 12  | 11    | 10   | 9         | 8        | 7        | 6  | 5  | 4  | 3        | 2  | 1  | 0  |
| MOE      | AOE | BKP | BKE | OSSR  | OSSI | LOCK[1:0] |          | DTG[7:0] |    |    |    |          |    |    |    |
| rw       | rw  | rw  | rw  | rw    | rw   | rw        | rw       | rw       | rw | rw | rw | rw       | rw | rw | rw |

注: 由于可以根据 LOCK 配置锁定位 BKBID, BKDSRM, BKF[3:0], AOE, BKP, BKE, OSSI, OSSR 和 DTG[7:0]的写操作, 因此必须在第一次对 TIM15\_BDTR 寄存器执行写访问时对这些位进行配置。



Bit 31:29 保留，必须保持复位值。

Bit 28 **BKBID**: 刹车双向 (Break bidirectional)

0: 在输入模式下刹车输入 tim\_brk

1: 在双向模式下刹车输入 tim\_brk

在双向模式 (BKBID 位设置为 1) 中，刹车输入在输入和输出中都配置模式和开漏输出模式。任何活动中断事件都会在刹车输入，向外部设备指示内部中断事件。

注：只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 1，此位即无法修改。

注：对此位的任何写入操作都需要延迟 1 个 APB 时钟周期才能生效。

Bit 27 保留，必须保持复位值。

Bit 26 **BKDSRM**: 刹车解除 (Break disarm)

0: 刹车输入 tim\_brk 已装载

1: 刹车输入 tim\_brk 被解除

当没有刹车源处于有效状态时，此位由硬件清除。

BKDSRM 位必须由软件设置，以释放双向输出控制 (高阻状态下的开漏输出)，然后对其进行轮询，直到它被硬件重置，表明故障状态已消失。

注：对此位的任何写入操作都需要延迟 1 个 APB 时钟周期才能生效。

Bit 25:20 保留，必须保持复位值。

Bit 19:16 **BKF[3:0]**: 刹车滤波器

这几位定义了 tim\_brk 输入的采样频率及数字滤波器长度。数字滤波器由一个事件计数器组成，它记录到 N 个事件后会产生一个输出的跳变

0000: No filter, BRK acts asynchronously

0001:  $f_{\text{SAMPLING}} = f_{\text{tim\_ker\_ck}}$ ,  $N=2$ 。

0010:  $f_{\text{SAMPLING}} = f_{\text{tim\_ker\_ck}}$ ,  $N=4$ 。

0011:  $f_{\text{SAMPLING}} = f_{\text{tim\_ker\_ck}}$ ,  $N=8$ 。

0100:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/2$ ,  $N=6$ 。

0101:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/2$ ,  $N=8$ 。

0110:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/4$ ,  $N=6$ 。

0111:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/4$ ,  $N=8$ 。

1000:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/8$ ,  $N=6$ 。

1001:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/8$ ,  $N=8$ 。

1010:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$ ,  $N=5$ 。

1011:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$ ,  $N=6$ 。

1100:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$ ,  $N=8$ 。

1101:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$ ,  $N=5$ 。

1110:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$ ,  $N=6$ 。

1111:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$ ,  $N=8$ 。

注：只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别 1，此位即无法修改。

**Bit 15 MOE: 主输出使能 (Main output enable)**

只要刹车输入变为有效状态, 此位便由硬件异步清零。此位由软件置 1, 也可根据 AOE 位状态自动置 1。此位仅对配置为输出的通道有效。

0: tim\_ocx 和 tim\_ocxn 输出禁止或被强制为空闲状态。

1: 如果 tim\_ocx 和 tim\_ocxn 输出的相应使能位 (TIM15\_CCER 寄存器中的 CCxE 和 CCxNE 位) 均置 1, 则使能 OC 和 OCN 输出。

有关详细信息, 请参见 tim\_ocx / tim\_ocxn 使能说明 (TIM15 捕获/比较使能寄存器 (TIM15\_CCER))。

**Bit 14 AOE: 自动输出使能 (Automatic output enable)**

0: MOE 只能由软件置 1

1: MOE 可由软件置 1, 也可在发生下一更新事件时自动置 1(如果刹车输入无效)

注: 只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别 1, 此位即无法修改。

**Bit 13 BKP: 刹车极性 (Break polarity)**

0: 刹车输入低电平有效;

1: 刹车输入高电平有效。

注: 只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别 1, 此位即无法修改。

注: 对此位的任何写入操作都需要延迟 1 个 APB 时钟周期才能生效。

**Bit 12 BKE: 刹车使能 (Break enable)**

0: 禁止刹车输入 (BRK 及 CCS 时钟失效事件);

1: 开启刹车输入 (BRK 及 CCS 时钟失效事件)。

注: 只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别 1, 此位即无法修改。

注: 对此位的任何写入操作都需要延迟 1 个 APB 时钟周期才能生效。

**Bit 11 OSSR: 运行模式下“关闭状态”选择 (Off-state selection for Run mode)**

此位在 MOE=1 时作用于配置为输出模式且具有互补输出的通道。如果定时器中没有互补输出, 则不存在 OSSR。

有关详细信息, 请参见 tim\_ocx / tim\_ocxn 使能说明 (TIM15 捕获/比较使能寄存器 (TIM15\_CCER))。

0: 处于无效状态时, 禁止 OC/OCN 输出 (OC/OCN 使能输出信号=0)。

1: 处于无效状态时, 一旦 CCxE=1 或 CCxNE=1, 便使能 OC/OCN 输出并将其设为无效电平。然后设置 OC/OCN 使能输出信号=1

注: 编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别 2 后, 此位即无法修改。

**Bit 10 OSSI: 空闲模式下的关闭状态选择 (Off-state selection for Idle mode)**

此位在 MOE=0 时作用于配置为输出的通道。

有关详细信息, 请参见 tim\_ocx / tim\_ocxn 使能说明 (TIM15 捕获/比较使能寄存器 (TIM15\_CCER))。

0: 处于无效状态时, 禁止 OC/OCN 输出 (OC/OCN 使能输出信号=0)。

1: 处于无效状态时, 一旦 CCxE=1 或 CCxNE=1, 便将 OC/OCN 输出首先强制为其空闲电平。然后设置 OC/OCN 使能输出信号=1

注: 编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别 2 后, 此位即无法修改。

### Bit 9:8 LOCK[1:0]: 锁定设置 (Lock configuration)

这些位用于针对软件错误提供写保护。

00: 关闭锁定--不对任何位提供写保护。

01: 锁定级别 1, 此时无法对 TIMx\_BDTR 寄存器中的 DTG 位、TIMx\_CR2 寄存器中的 OISx 和 OISxN 位以及 TIMx\_BDTR 寄存器中的 BKE/BKP/AOE 位执行写操作。

10: 锁定级别 2, 此时无法对锁定级别 1 中适用的各位、CC 极性位 (TIMx\_CCER 寄存器中的 CCxP/CCxNP 位, 只要通过 CCxS 位将相关通道配置为输出) 以及 OSSR 和 OSSR 位执行写操作。

11: 锁定级别 3, 此时无法对锁定级别 2 中适用的各位、CC 控制位 (TIMx\_CCMRx 寄存器中的 OCxM 和 OCxPE 位, 只要通过 CCxS 位将相关通道配置为输出) 执行写操作。

注: 复位后只能对 LOCK 位执行一次写操作。对 TIM15\_BDTR 寄存器执行写操作后其中的内容将冻结, 直到下一次复位。

### Bit 7:0 DTG[7:0]: 配置死区发生器 (Dead-time generator setup)

此位域定义插入到互补输出之间的死区持续时间。DT 与该持续时间相对应。

DTG[7:5]=0xx => DT= DTG[7:0] × T<sub>dtg</sub>, 其中 T<sub>dtg</sub> = T<sub>DTS</sub>;

DTG[7:5]=10x => DT= (64+DTG[5:0]) × T<sub>dtg</sub>, 其中 T<sub>dtg</sub> = 2 × T<sub>DTS</sub>;

DTG[7:5]=110 => DT= (32+DTG[4:0]) × T<sub>dtg</sub>, 其中 T<sub>dtg</sub> = 8 × T<sub>DTS</sub>;

DTG[7:5]=111 => DT= (32+DTG[4:0]) × T<sub>dtg</sub>, 其中 T<sub>dtg</sub> = 16 × T<sub>DTS</sub>;

例: 若 T<sub>DTS</sub> = 125ns (8MHZ), 可能的死区时间为:

0 到 15875 ns (步长为 125 ns),

16 us 到 31750 ns (步长为 250 ns),

32 us 到 63us (步长为 1 us),

64 us 到 126 us (步长为 2 us)

注: 编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别1、2 或3, 此位即无法修改。

## 25.7.17 TIM15 定时器死区寄存器 2 (TIM15\_DTR2)

偏移地址: 0x054

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |           |    |    |    |    |    |      |      |
|----------|----|----|----|----|----|----|----|-----------|----|----|----|----|----|------|------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23        | 22 | 21 | 20 | 19 | 18 | 17   | 16   |
| Reserved |    |    |    |    |    |    |    |           |    |    |    |    |    | DTPE | DTAE |
|          |    |    |    |    |    |    |    |           |    |    |    |    |    | rw   | rw   |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7         | 6  | 5  | 4  | 3  | 2  | 1    | 0    |
| Reserved |    |    |    |    |    |    |    | DTGF[7:0] |    |    |    |    |    |      |      |
|          |    |    |    |    |    |    |    | rw        | rw | rw | rw | rw | rw | rw   | rw   |

Bit 31:18 保留, 必须保持复位值。

### Bit 17 DTPE: 死区时间预加载使能 (Deadtime preload enable)

0: 死区时间值未预加载

1: 使能死区时间值预加载

注: 编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别1、2 或3, 此位即无法修改。

Bit 16 **DTAE**: 死区时间非对称使能 (Deadtime asymmetric enable)

0: 上升沿和下降沿的死区时间是相同的, 并且使用 DTG[7:0]寄存器定义

1: 上升沿的死区时间使用 DTG[7:0]寄存器定义, 下降沿的死区时间使用 DTGF[7:0]位定义。

注: 编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别1、2 或3, 此位即无法修改。

Bit 15:8 保留, 必须保持复位值。

Bit 7:0 **DTGF[7:0]**: 死区下降沿发生器 (Dead-time falling edge generator setup)

这个位域定义了在下落沿时, 互补输出之间插入死区时间的长度。

DTG[7:5]=0xx =>  $DT = DTG[7:0] \times T_{dtg}$ , 其中  $T_{dtg} = T_{DTS}$ ;

DTG[7:5]=10x =>  $DT = (64 + DTG[5:0]) \times T_{dtg}$ , 其中  $T_{dtg} = 2 \times T_{DTS}$ ;

DTG[7:5]=110 =>  $DT = (32 + DTG[4:0]) \times T_{dtg}$ , 其中  $T_{dtg} = 8 \times T_{DTS}$ ;

DTG[7:5]=111 =>  $DT = (32 + DTG[4:0]) \times T_{dtg}$ , 其中  $T_{dtg} = 16 \times T_{DTS}$ ;

例: 若  $T_{DTS} = 125ns$  (8MHZ), 可能的死区时间为:

0 到 15875 ns (步长为 125 ns),

16 us 到 31750 ns (步长为 250 ns),

32 us 到 63us (步长为 1 us),

64 us 到 126 us (步长为 2 us)

注: 编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别1、2 或3, 此位即无法修改。

## 25.7.18 TIM15 定时器输入选择寄存器 (TIM15\_TISEL)

偏移地址: 0x05C

复位值: 0x0000 0000

|          |    |    |    |             |    |    |    |          |    |    |    |             |    |    |    |
|----------|----|----|----|-------------|----|----|----|----------|----|----|----|-------------|----|----|----|
| 31       | 30 | 29 | 28 | 27          | 26 | 25 | 24 | 23       | 22 | 21 | 20 | 19          | 18 | 17 | 16 |
| Reserved |    |    |    |             |    |    |    |          |    |    |    |             |    |    |    |
| 15       | 14 | 13 | 12 | 11          | 10 | 9  | 8  | 7        | 6  | 5  | 4  | 3           | 2  | 1  | 0  |
| Reserved |    |    |    | TI2SEL[3:0] |    |    |    | Reserved |    |    |    | TI1SEL[3:0] |    |    |    |
|          |    |    |    | rw          | rw | rw | rw |          |    |    |    | rw          | rw | rw | rw |

Bit 31:12 保留, 必须保持复位值。

Bit 11:8 **TI2SEL[3:0]**: 选择 tim\_ti2[15:0]输入 (Selects tim\_ti2[15:0] input)

0000: TIM15\_CH2 输入 (tim\_ti2\_in0:)

0001: tim\_ti2\_in1

...

1111: tim\_ti2\_in15

参考章节: TIM15/TIM16/TIM17 引脚和内部信号获得关联列表。

Bit 7:4 保留, 必须保持复位值。

Bit 3:0 **TI1SEL[3:0]**: 选择 tim\_ti1[15:0]输入 (Selects tim\_ti1[15:0] input)

0000: TIM15\_CH1 输入 (tim\_ti1\_in0:)

0001: tim\_ti1\_in1

...

1111: tim\_ti1\_in15

参考章节: TIM15/TIM16/TIM17 引脚和内部信号获得关联列表。

## 25.7.19 TIM15 复用功能选择寄存器 1 (TIM15\_AF1)

偏移地址: 0x060

复位值: 0x0000 0001

| 31       | 30          | 29          | 28          | 27          | 26    | 25       | 24 | 23 | 22 | 21 | 20          | 19          | 18          | 17          | 16    |
|----------|-------------|-------------|-------------|-------------|-------|----------|----|----|----|----|-------------|-------------|-------------|-------------|-------|
| Reserved |             |             |             |             |       |          |    |    |    |    |             |             |             |             |       |
| 15       | 14          | 13          | 12          | 11          | 10    | 9        | 8  | 7  | 6  | 5  | 4           | 3           | 2           | 1           | 0     |
| Reserved | BK<br>CMP4P | BK<br>CMP3P | BK<br>CMP2P | BK<br>CMP1P | BKINP | Reserved |    |    |    |    | BK<br>CMP4E | BK<br>CMP3E | BK<br>CMP2E | BK<br>CMP1E | BKINE |
|          |             |             |             |             |       |          |    |    |    |    |             |             |             |             |       |
|          | rw          | rw          | rw          | rw          | rw    |          |    |    |    |    | rw          | rw          | rw          | rw          | rw    |

注: 有关产品特定实现的信息, 请参考章节: TIM15/TIM16/TIM17 引脚和内部信号。

Bit 31:14 保留, 必须保持复位值。

Bit 13 **BKCOMP4P**: tim\_brk\_cmp4 输入极性 (tim\_brk\_cmp4 input polarity)

此位选择 tim\_brk\_cmp2 输入敏感度。它必须与 BKP 极性位一起编程。

0: tim\_brk\_cmp4 输入高电平有效

1: tim\_brk\_cmp4 输入低电平有效

注: 只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 12 **BKCOMP3P**: tim\_brk\_cmp3 输入极性 (tim\_brk\_cmp3 input polarity)

此位选择 tim\_brk\_cmp3 输入敏感度。它必须与 BKP 极性位一起编程。

0: tim\_brk\_cmp3 输入高电平有效

1: tim\_brk\_cmp3 输入低电平有效

注: 只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 11 **BKCOMP2P**: tim\_brk\_cmp2 输入极性 (tim\_brk\_cmp2 input polarity)

此位选择 tim\_brk\_cmp2 输入敏感度。它必须与 BKP 极性位一起编程。

0: tim\_brk\_cmp2 输入高电平有效

1: tim\_brk\_cmp2 输入低电平有效

注: 只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 10 **BKCOMP1P**: tim\_brk\_cmp1 输入极性 (tim\_brk\_cmp1 input polarity)

此位选择 tim\_brk\_cmp1 输入敏感度。它必须与 BKP 极性位一起编程。

0: tim\_brk\_cmp1 输入高电平有效

1: tim\_brk\_cmp1 输入低电平有效

注: 只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 9 **BKINP**: TIMx\_BKIN 输入极性 (TIMx\_BKIN input polarity)

该位选择 TIMx\_BKIN 备用功能输入敏感度。必须对其进行编程

0: TIMx\_BKIN 输入电平有效

1: TIMx\_BKIN 输入低电平有效

注: 只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 8:5 保留, 必须保持复位值。

Bit 4 **BKCOMP4E**: tim\_brk\_cmp4 使能 (tim\_brk\_cmp4 enable)

该位用于使能或禁用 tim\_brk\_cmp4 输入。如果该位设置为 1, 则 tim\_brk\_cmp4 输入将被使能, 否则将被禁用。

0: tim\_brk\_cmp4 输入禁用

1: tim\_brk\_cmp4 输入使能

注: 只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 4 **BKCMP3E**: tim\_brk\_cmp3 使能 (tim\_brk\_cmp3 enable)

该位用于使能或禁用 tim\_brk\_cmp3 输入。如果该位设置为 1, 则 tim\_brk\_cmp3 输入将被使能, 否则将被禁用。

0: tim\_brk\_cmp3 输入禁用

1: tim\_brk\_cmp3 输入使能

注: 只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 2 **BKCMP2E**: tim\_brk\_cmp2 使能 (tim\_brk\_cmp2 enable)

该位用于使能或禁用 tim\_brk\_cmp2 输入。如果该位设置为 1, 则 tim\_brk\_cmp2 输入将被使能, 否则将被禁用。

0: tim\_brk\_cmp2 输入禁用

1: tim\_brk\_cmp2 输入使能

注: 只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 1 **BKCMP1E**: tim\_brk\_cmp1 使能 (tim\_brk\_cmp1 enable)

该位用于使能或禁用 tim\_brk\_cmp1 输入。如果该位设置为 1, 则 tim\_brk\_cmp1 输入将被使能, 否则将被禁用。

0: tim\_brk\_cmp1 输入禁用

1: tim\_brk\_cmp1 输入使能

注: 只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 0 **BKINE**: TIMx\_BKIN 输入使能 (TIMx\_BKIN input enable)

该位用于使能或禁用 TIMx\_BKIN alternate function 输入作为定时器的 tim\_brk 输入。TIMx\_BKIN 输入与其他 tim\_brk 源进行“或”操作。

0: TIMx\_BKIN 输入禁用

1: TIMx\_BKIN 输入使能

注: 只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。



## 25.7.20 TIM15 复用功能选择寄存器 2 (TIM15\_AF2)

偏移地址: 0x064

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |    |    |    |    |    |             |    |    |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|-------------|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18          | 17 | 16 |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    | OCRSEL[2:0] |    |    |
|          |    |    |    |    |    |    |    |    |    |    |    |    | rw          | rw | rw |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2           | 1  | 0  |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |             |    |    |

Bit 31:19 保留, 必须保持复位值。

Bit 18:16 **OCRSEL[2:0]**: ocref\_clr 源选择 (ocref\_clr source selection)

这些位选择 ocref\_clr 输入源。

000: tim\_ocref\_clr0

001: tim\_ocref\_clr1

010: tim\_ocref\_clr2

011: tim\_ocref\_clr3

100: tim\_ocref\_clr4

101: tim\_ocref\_clr5

110: tim\_ocref\_clr6

111: tim\_ocref\_clr7

参考章节: TIM15/TIM16/TIM17 引脚和内部信号获取更多特定产品信息。

注: 只要编程了 LOCK (TIM15\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 15:0 保留, 必须保持复位值。

## 25.7.21 TIM15 DMA 控制寄存器 (TIM15\_DCR)

偏移地址: 0x3DC

复位值: 0x0000 0000

|          |    |    |    |          |    |    |    |    |          |    |    |    |          |    |    |
|----------|----|----|----|----------|----|----|----|----|----------|----|----|----|----------|----|----|
| 31       | 30 | 29 | 28 | 27       | 26 | 25 | 24 | 23 | 22       | 21 | 20 | 19 | 18       | 17 | 16 |
| Reserved |    |    |    |          |    |    |    |    |          |    |    |    |          |    |    |
| 15       | 14 | 13 | 12 | 11       | 10 | 9  | 8  | 7  | 6        | 5  | 4  | 3  | 2        | 1  | 0  |
| Reserved |    |    |    | DBL[4:0] |    |    |    |    | Reserved |    |    |    | DBA[4:0] |    |    |
|          |    |    |    | rw       | rw | rw | rw | rw |          |    |    |    | rw       | rw | rw |

Bit 31:13 保留, 必须保持复位值。

Bit 12:8 **DBL[4:0]**: DMA 突发长度 (DMA burst length)

这个 5 位向量定义了 DMA 传输的长度 (当对 TIM15\_DMAR 地址进行读或写访问时, 定时器识别一次突发传输), 即传输次数。传输可以是半字或字节 (见下面的示例)。

00000: 1 次传输

00001: 2 次传输

00010: 3 次传输

...

10001: 18 次传输

Bit 7:5 保留, 必须保持复位值。

Bit 4:0 **DBA[4:0]**: DMA 基地址 (DMA base address)



这个 5 位向量定义了 DMA 传输的基地址（当通过 TIM15\_DMAR 地址进行读/写访问时）。DBA 是从 TIM15\_CR1 寄存器的地址开始的偏移量。

00000: TIM15\_CR1,  
00001: TIM15\_CR2,  
00010: TIM15\_SMCR,  
...

## 25.7.22 TIM15 用于完整传输的 DMA 地址（TIM15\_DMAR）

偏移地址: 0x3E0

复位值: 0x0000 0000

| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| DMAB[31:16] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| DMAB[15:0]  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:0 **DMAB[31:0]**: 用于突发访问的 DMA 寄存器（DMA register for burst accesses）

对 DMAR 寄存器进行的读或写操作将访问位于以下地址的寄存器：

$$(\text{TIM15\_CR1 地址}) + (\text{DBA} + \text{DMA 索引}) \times 4$$

其中，TIM15\_CR1 地址是控制寄存器 1 的地址，DBA 是在 TIM15\_DCR 寄存器中配置的 DMA 基地址，DMA 索引由 DMA 传输自动控制，其范围从 0 到 DBL（在 TIM15\_DCR 中配置的 DBL）。

## 25.8 TIM16/TIM17 寄存器描述

在寄存器描述中使用的缩写列表请参考章节 2.1。

### 25.8.1 TIMx 控制寄存器 1 (TIMx\_CR1) (x=16~17)

偏移地址: 0x000

复位值: 0x0000

| 15       | 14 | 13 | 12         | 11           | 10  | 9        | 8  | 7    | 6        | 5 | 4 | 3   | 2   | 1    | 0   |
|----------|----|----|------------|--------------|-----|----------|----|------|----------|---|---|-----|-----|------|-----|
| Reserved |    |    | DITH<br>EN | UIFRE<br>MAP | Res | CKD[1:0] |    | ARPE | Reserved |   |   | OPM | URS | UDIS | CEN |
|          |    |    | rw         | rw           |     | rw       | rw | rw   |          |   |   | rw  | rw  | rw   | rw  |

Bit 15:13 保留, 必须保持复位值。

Bit 12 **DITHEN**: 使能抖动 (Dithering enable)

0: 抖动禁止

1: 抖动使能

注: DITHEN 位只能在 CEN 位复位时修改

Bit 11 **UIFREMAP**: UIF 状态位重映射 (UIF status bit remapping)

0: 无重映射。UIF 状态位未复制到 TIMx\_CNT 寄存器位 31

1: 重映射使能。UIF 状态位复制到 TIMx\_CNT 寄存器位 31

Bit 10 保留, 必须保持复位值。

Bit 9:8 **CKD[1:0]**: 时钟分频 (Clock division)

此位域指示定时器时钟 (tim\_ker\_ck) 频率以及数字滤波器 (tim\_etr\_in, tim\_tix) 所使用的采样时钟 (t<sub>DTS</sub>) 之间的分频比

00: t<sub>DTS</sub> = t<sub>tim\_ker\_ck</sub>

01: t<sub>DTS</sub> = 2 × t<sub>tim\_ker\_ck</sub>

10: t<sub>DTS</sub> = 4 × t<sub>tim\_ker\_ck</sub>

11: 保留, 不要设置成此值

Bit 7 **ARPE**: 自动重载预装载使能 (Auto-reload preload enable)

0: TIMx\_ARR 寄存器不进行缓冲;

1: TIMx\_ARR 寄存器进行缓冲。

Bit 6:4 保留, 必须保持复位值。

Bit 3 **OPM**: 单脉冲模式 (One pulse mode)

0: 计数器在发生更新事件时不会停止计数;

1: 计数器在发生下一更新事件时停止计数 (将 CEN 位清零)

Bit 2 **URS**: 更新请求源 (Update request source)

此位由软件置 1 和清零, 用以选择 UEV 事件源。

0: 使能时, 所有以下事件都会生成更新中断或 DMA 请求。此类事件包括:

- 计数器上溢/下溢
- 将 UG 位置 1
- 通过从模式控制器生成的更新事件

1: 使能时, 只有计数器上溢/下溢会生成更新中断或 DMA 请求。

### Bit 1 UDIS: 更新禁止 (Update disable)

此位由软件置 1 和清零, 用以使能/禁止 UEV 事件生成。

0: 使能 UEV。更新 (UEV) 事件可通过以下事件之一生成:

- 计数器上出/下溢
- 将 UG 位置 1
- 通过从模式控制器生成的更新事件

然后更新影子寄存器的值。

1: 禁止 UEV。不会生成更新事件, 各影子寄存器的值 (ARR、PSC 和 CCRx) 保持不变。但如果将 UG 位置 1, 或者从模式控制器接收到硬件复位, 则会重新初始化计数器和预分频器。

### Bit 0 CEN: 计数器使能 (Counter enable)

0: 禁止计数器;

1: 使能计数器。

注: 只有事先通过软件将 CEN 位置 1, 才可以使用外部时钟、门控模式。而触发模式可通过硬件自动将 CEN 位置 1。

## 25.8.2 TIMx 控制寄存器 2 (TIMx\_CR2) (x=16~17)

偏移地址: 0x004

复位值: 0x0000

| 15       | 14 | 13 | 12 | 11 | 10 | 9     | 8    | 7        | 6 | 5 | 4 | 3    | 2    | 1   | 0    |
|----------|----|----|----|----|----|-------|------|----------|---|---|---|------|------|-----|------|
| Reserved |    |    |    |    |    | OIS1N | OIS1 | Reserved |   |   |   | CCDS | CCUS | Res | CCPC |
|          |    |    |    |    |    | rw    | rw   |          |   |   |   | rw   | rw   |     | rw   |

Bit 15:10 保留, 必须保持复位值。

### Bit 9 OIS1N: 输出空闲状态 1 (OC1N 输出) (Output idle state 1 (tim\_oc1n output))

0: 当 MOE=0 时, 经过死区时间后 tim\_oc1n=0

1: 当 MOE=0 时, 经过死区时间后 tim\_oc1n=1

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 1、2 或 3, 此位即无法修改。

### Bit 8 OIS1: 输出空闲状态 1 (OC1 输出) (Output idle state 1 (tim\_oc1 output))

0: 当 MOE=0 时, (如果 OC1N 有效, 则经过死区时间之后) tim\_oc1=0

1: 当 MOE=0 时, (如果 OC1N 有效, 则经过死区时间之后) tim\_oc1=1

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 1、2 或 3, 此位即无法修改。

Bit 7:4 保留, 必须保持复位值。

### Bit 3 CCDS: 捕获/比较 DMA 选择 (Capture/compare DMA selection)

0: 发生 CCx 事件时发送 CCx DMA 请求

1: 发生更新事件时发送 CCx DMA 请求

### Bit 2 CCUS: 捕获/比较控制更新选择 (Capture/compare control update selection)

0: 如果捕获/比较控制位(CCPC=1)进行预装载, 仅通过将 COMG 位置 1 来对这些位进行更新;

1: 如果捕获/比较控制位(CCPC=1)进行预装载, 可通过将 COMG 位置 1 或 TRGI 的上升沿对这些位进行更新。

注: 此位仅对具有互补输出的通道有效。。

Bit 1 保留，必须保持复位值。

Bit 0 **CCPC**: 捕获/比较预装载控制 (Capture/compare preloaded control)

0: CCxE, CCxNE 和 OCxM 位未进行预装载;

1: CCxE, CCxNE 和 OCxM 位进行了预装载, 写入这些位后, 仅当发生换向事件 (COM) (COMG 位置 1 或在 TRGI 上检测到上升沿, 取决于 CCUS 位) 时才会对这些位进行更新。

注: 此位仅对具有互补输出的通道有效。

### 25.8.3 TIMx DMA/中断使能寄存器 (TIMx\_DIER) (x=16~17)

偏移地址: 0x00C

复位值: 0x0000

| 15       | 14 | 13 | 12 | 11 | 10 | 9     | 8   | 7   | 6   | 5     | 4        | 3 | 2 | 1     | 0   |
|----------|----|----|----|----|----|-------|-----|-----|-----|-------|----------|---|---|-------|-----|
| Reserved |    |    |    |    |    | CC1DE | UDE | BIE | Res | COMIE | Reserved |   |   | CC1IE | UIE |
|          |    |    |    |    |    | rw    | rw  | rw  |     | rw    |          |   |   | rw    | rw  |

Bit 15:10 保留，必须保持复位值。

Bit 9 **CC1DE**: 捕获/比较 1 DMA 请求使能 (Capture/Compare 1 DMA request enable)

0: 禁止 CC1 DMA 请求;

1: 使能 CC1 DMA 请求。

Bit 8 **UDE**: 更新 DMA 请求使能 (Capture/Compare 1 DMA request enable)

0: 禁止更新 DMA 请求;

1: 使能更新 DMA 请求。

Bit 7 **BIE**: 刹车中断使能 (Break interrupt enable)

0: 禁止刹车中断;

1: 使能刹车中断。

Bit 6 保留，必须保持复位值。

Bit 5 **COMIE**: COM 中断使能 (COM interrupt enable)

0: 禁止 COM 中断;

1: 使能 COM 中断。

Bit 4:2 保留，必须保持复位值。

Bit 1 **CC1IE**: 捕获/比较 1 中断使能 (Capture/Compare 1 interrupt enable)

0: 禁止 CC1 中断;

1: 使能 CC1 中断。

Bit 0 **UIE**: 更新中断使能 (Update interrupt enable)

0: 禁止更新中断;

1: 使能更新中断。

## 25.8.4 TIMx 状态寄存器 (TIMx\_SR) (x=16~17)

偏移地址: 0x010

复位值: 0x0000

| 15       | 14 | 13 | 12 | 11 | 10 | 9     | 8   | 7     | 6   | 5     | 4        | 3 | 2 | 1     | 0     |
|----------|----|----|----|----|----|-------|-----|-------|-----|-------|----------|---|---|-------|-------|
| Reserved |    |    |    |    |    | CC1OF | Res | BIF   | Res | COMIF | Reserved |   |   | CC1IF | UIF   |
|          |    |    |    |    |    | rc_w0 |     | rc_w0 |     | rc_w0 |          |   |   | rc_w0 | rc_w0 |

Bit 15:10 保留, 必须保持复位值。

Bit 9 **CC1OF**: 捕获/比较 1 重复捕获标志

仅当将相应通道配置为输入捕获模式时, 此标志位才会由硬件置 1。通过软件写入“0”可将该位清零。

0: 未检测到重复捕获

1: TIMx\_CCR1 寄存器中已捕获到计数器值且 CC1IF 标志已置 1。

Bit 8 保留, 必须保持复位值。

Bit 7 **BIF**: 刹车中断标志 (Break interrupt flag)

只要 tim\_brk 输入变为有效状态, 此标志便由硬件置 1。刹车输入无效后可通过软件对其清零。

0: 未发生刹车事件。

1: 在刹车输入上检测到有效电平。

Bit 6 保留, 必须保持复位值。

Bit 5 **COMIF**: COM 中断标志 (COM interrupt flag)

此标志在发生 COM 事件时 (捕获/比较控制位 CCxE、CCxNE 和 OCxM 已更新时) 由硬件置 1。但需要通过软件清零。

0: 未发生 COM 事件。

1: COM 中断挂起。

Bit 4:2 保留, 必须保持复位值。

Bit 1 **CC1IF**: 捕获/比较 1 中断标志 (Capture/Compare 1 interrupt flag)

此标志由硬件设置, 它可以通过软件 (输入捕获或输出比较模式) 或读取 TIMx\_CCR1 寄存器 (仅输入捕获模式) 来清除。

0: 未比较匹配/无输入捕获发生

1: 发生比较匹配或输入捕获

**如果通道 CC1 配置为输出:** 当计数器与比较值匹配时, 此标志由硬件置 1, 当

TIMx\_CCR1 的值大于 TIMx\_ARR 的值时, CC1IF 位将在计数器发生上溢 (递增计数模式和增减计数模式下) 或下溢 (递减计数模式下) 时变为高电平。在中心对齐模式下有 3 种可能的标志设置选项, 请参考 TIMx\_CR1 寄存器中的 CMS 位以获取完整描述。

**如果通道 CC1 配置为输入:** 当在 TIMx\_CCR1 寄存器中捕获到计数器值时 (在 IC1 上检测到边沿, 具体取决于在 TIMx\_CCER 中通过 CC1P 和 CC1NP 位设置定义的边沿灵敏度), 会设置此位。

Bit 0 **UIF**: 更新中断标志 (Update interrupt flag)

该位在发生更新事件时通过硬件置 1。但需要通过软件清零。

0: 未发生更新。

1: 更新中断挂起。该位在以下情况下更新寄存器时由硬件置 1:

- TIMx\_CR1 寄存器中的 UDIS=0, 并且重复计数器值上溢或下溢时 (重复计数器=0 时更新)。
- TIMx\_CR1 寄存器中的 URS=0 且 UDIS=0, 并且由软件使用 TIMx\_EGR 寄存器中的 UG 位重新初始化 CNT 时。

### 25.8.5 TIMx 事件产生寄存器 (TIMx\_EGR) (x=16~17)

偏移地址: 0x014

复位: 0x0000

| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7  | 6   | 5    | 4        | 3 | 2 | 1    | 0  |
|----------|----|----|----|----|----|---|---|----|-----|------|----------|---|---|------|----|
| Reserved |    |    |    |    |    |   |   | BG | Res | COMG | Reserved |   |   | CC1G | UG |
|          |    |    |    |    |    |   |   | w  |     | rw   |          |   |   | w    | w  |

Bit 15:8 保留, 必须保持复位值。

Bit 7 **BG**: 刹车生成 (Break generation)

此位由软件置 1 以生成事件, 并由硬件自动清零。

0: 不执行任何操作;

1: 生成刹车事件。MOE 位清零且 BIF 标志置 1。使能后可发生相关中断或 DMA 传输事件。

Bit 6 保留, 必须保持复位值。

Bit 5 **COMG**: 捕获/比较控制更新生成 (Capture/Compare control update generation)

该位可通过软件置 1, 并由硬件自动清零

0: 不执行任何操作;

1: CCPC 位置 1 时, 可更新 CCxE、CCxNE 和 OCxM 位。

注: 此位仅对具有互补输出的通道有效。

Bit 4:2 保留, 必须保持复位值。

Bit 1 **CC1G**: 捕获/比较 1 生成 (Capture/Compare 1 generation)

此位由软件置 1 以生成事件, 并由硬件自动清零。

0: 不执行任何操作;

1: 通道 1 上生成捕获/比较事件:

若通道 CC1 配置为输出:

使能时, CC1IF 标志置 1 并发送相应的中断或 DMA 请求。

如果通道 CC1 配置为输入:

TIMx\_CCR1 寄存器中将捕获到计数器当前值。使能时, CC1IF 标志置 1 并发送相应的中断或 DMA 请求。如果 CC1IF 标志已为高电平, CC1OF 标志将置 1。

Bit 0 **UG**: 更新生成 (Update generation)

该位可通过软件置 1, 并由硬件自动清零。

0: 不执行任何操作;

1: 重新初始化计数器并生成一个寄存器更新事件。请注意, 预分频器计数器也将清零 (但预分频比不受影响)。



## 25.8.6 TIMx 捕获/比较模式寄存器 1 [复用] (TIMx\_CCMR1) (x=16~17)

偏移地址: 0x018

复位值: 0x0000 0000

相同的寄存器可以用于输入捕获模式 (本节) 或输出比较模式 (下节)。通道方向通过配置相应的 CCxS 位进行定义。此寄存器的所有其它位在输入模式和输出模式下的功能均不同。可以独立组合这两种模式 (例如, 通道 1 用作输入捕获模式而通道 2 用作输出比较模式)

|          |    |    |    |    |    |    |    |           |    |    |    |             |    |           |    |
|----------|----|----|----|----|----|----|----|-----------|----|----|----|-------------|----|-----------|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23        | 22 | 21 | 20 | 19          | 18 | 17        | 16 |
| Reserved |    |    |    |    |    |    |    |           |    |    |    |             |    |           |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7         | 6  | 5  | 4  | 3           | 2  | 1         | 0  |
| Reserved |    |    |    |    |    |    |    | IC1F[3:0] |    |    |    | IC1PSC[1:0] |    | CC1S[1:0] |    |
|          |    |    |    |    |    |    |    | rw        | rw | rw | rw | rw          | rw | rw        | rw |

### 输入捕获模式

Bit 31:8 保留, 始终读为 0。

Bit 7:4 **IC1F[3:0]**: 输入捕获 1 滤波器 (Input capture 1 filter)

此位域可定义 tim\_ti1 输入的采样频率和适用于 tim\_ti1 的数字滤波器带宽。数字滤波器由事件计数器组成, 每 N 个事件才视为一个有效边沿:

0000: 无滤波器, 按  $f_{DTS}$  频率进行采样

0001:  $f_{SAMPLING} = f_{tim\_ker\_ck}$ ,  $N=2$ 。

0010:  $f_{SAMPLING} = f_{tim\_ker\_ck}$ ,  $N=4$ 。

0011:  $f_{SAMPLING} = f_{tim\_ker\_ck}$ ,  $N=8$ 。

0100:  $f_{SAMPLING} = f_{DTS}/2$ ,  $N=6$ 。

0101:  $f_{SAMPLING} = f_{DTS}/2$ ,  $N=8$ 。

0110:  $f_{SAMPLING} = f_{DTS}/4$ ,  $N=6$ 。

0111:  $f_{SAMPLING} = f_{DTS}/4$ ,  $N=8$ 。

1000:  $f_{SAMPLING} = f_{DTS}/8$ ,  $N=6$ 。

1001:  $f_{SAMPLING} = f_{DTS}/8$ ,  $N=8$ 。

1010:  $f_{SAMPLING} = f_{DTS}/16$ ,  $N=5$ 。

1011:  $f_{SAMPLING} = f_{DTS}/16$ ,  $N=6$ 。

1100:  $f_{SAMPLING} = f_{DTS}/16$ ,  $N=8$ 。

1101:  $f_{SAMPLING} = f_{DTS}/32$ ,  $N=5$ 。

1110:  $f_{SAMPLING} = f_{DTS}/32$ ,  $N=6$ 。

1111:  $f_{SAMPLING} = f_{DTS}/32$ ,  $N=8$ 。

Bit 3:2 **IC1PSC[1:0]**: 输入/捕获 1 预分频器 (Input capture 1 prescaler)

此位域定义 CC1 输入 (tim\_ic1) 的预分频比。

只要 CC1E=0 (TIMx\_CCER 寄存器), 预分频器便立即复位。

00: 无预分频器, 捕获输入上每检测到一个边沿便执行捕获

01: 每发生 2 个事件便执行一次捕获;

10: 每发生 4 个事件便执行一次捕获;

11: 每发生 8 个事件便执行一次捕获。



Bit 1:0 **CC1S[1:0]**: 捕获/比较 1 选择 (Capture/Compare 1 selection)

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00: CC1 通道被配置为输出;

01: CC1 通道被配置为输入, tim\_ic1 映射在 tim\_ti1 上;

其他: 保留

注: 仅当通道关闭时 (TIMx\_CCER 中的 CC1E = 0), 才可向 CC1S 位写入数据。

## 25.8.7 TIMx 捕获/比较模式寄存器 1 [复用] (TIMx\_CCMR1) (x=16~17)

偏移地址: 0x018

复位值: 0x0000 0000

相同的寄存器可以用于输出比较模式 (本节) 或输入捕获模式 (上节)。通道方向通过配置相应的 CCxS 位进行定义。此寄存器的所有其它位在输入模式和输出模式下的功能均不同。可以独立组合这两种模式 (例如, 通道 1 用作输入捕获模式而通道 2 用作输出比较模式)

|          |    |    |    |    |    |    |    |       |           |    |    |       |       |           |         |
|----------|----|----|----|----|----|----|----|-------|-----------|----|----|-------|-------|-----------|---------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23    | 22        | 21 | 20 | 19    | 18    | 17        | 16      |
| Reserved |    |    |    |    |    |    |    |       |           |    |    |       |       |           | OC1M[3] |
|          |    |    |    |    |    |    |    |       |           |    |    |       |       |           | rw      |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7     | 6         | 5  | 4  | 3     | 2     | 1         | 0       |
| Reserved |    |    |    |    |    |    |    | OC1CE | OC1M[2:0] |    |    | OC1PE | OC1FE | CC1S[1:0] |         |
|          |    |    |    |    |    |    |    | rw    | rw        | rw | rw | rw    | rw    | rw        | rw      |

### 输出比较模式

Bit 31:17 保留, 必须保持复位值。

Bit 15:8 保留, 必须保持复位值。

Bit 7 **OC1CE**: 输出比较 1 清零使能 (Output Compare 1 clear enable)

0: tim\_oc1ref 不受 tim\_ocref\_clr 输入信号影响;

1: tim\_oc1ref 在 tim\_ocref\_clr 输入信号中被检测到高电平, tim\_oc1ref 立即清零。

Bit 16, 6:4 **OC1M[3:0]**: 输出比较 1 模式 (Output Compare 1 mode)

这些位定义提供 tim\_oc1 和 tim\_oc1n 的输出参考信号 tim\_oc1ref 的行为。tim\_oc1ref 为高电平有效, 而 tim\_oc1 和 tim\_oc1n 的有效电平则取决于 CC1P 位和 CC1NP 位。

0000: **冻结** — 输出比较寄存器 TIMx\_CCR1 与计数器 TIMx\_CNT 进行比较不会对输出造成任何影响。(该模式用于生成时基)。

0001: 将通道 1 设置为匹配时输出有效电平。当计数器 TIMx\_CNT 与捕获/比较寄存器 1 (TIMx\_CCR1) 匹配时, tim\_oc1ref 信号强制变为高电平。

0010: 将通道 1 设置为匹配时输出无效电平。当计数器 TIMx\_CNT 与捕获/比较寄存器 1 (TIMx\_CCR1) 匹配时, tim\_oc1ref 信号强制变为低电平。

0011: **翻转** — TIMx\_CNT=TIMx\_CCR1 时, tim\_oc1ref 发生翻转。

0100: **强制变为无效电平** — tim\_oc1ref 强制变为低电平。

0101: **强制变为有效电平** — tim\_oc1ref 强制变为高电平。

0110: **PWM 模式 1** — 在递增计数模式下, 只要 TIMx\_CNT<TIMx\_CCR1, 通道 1 便为有效状态, 否则为无效状态。

0111: **PWM 模式 2** — 在递增计数模式下, 只要 TIMx\_CNT<TIMx\_CCR1, 通道 1 便为无效状态, 否则为有效状态。

其他: 保留

注：只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别3 且 CC1S=00 (通道配置为输出)，这些位即无法修改。

在 PWM 模式1 或2 中，仅当比较结果发生改变或输出比较模式由“冻结”模式切换到“PWM”模式时，tim\_ocref\_clr 电平才会发生更改。

对于具有互补输出的通道，此位域是预加载的。如果 TIMx\_CR2 寄存器中 CCPC 位被设置，则 OC1M 有效位仅在生成 COM 事件时从预加载位获取新值。

Bit 3 **OC1PE**：输出比较 1 预装载使能 (Output Compare 1 preload enable)

0：禁止与 TIMx\_CCR1 相关的预装载寄存器。可随时向 TIMx\_CCR1 写入数据，写入后将立即使用新值。

1：使能与 TIMx\_CCR1 相关的预装载寄存器。可读/写访问预装载寄存器。TIMx\_CCR1 预装载值在每次生成更新事件时都会装载到有效寄存器中。

注：只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别3 且 CC1S=00 (通道配置为输出)，这些位即无法修改。

Bit 2 **OC1FE**：输出比较 1 快速使能 (Output Compare 1 fast enable)

此位减少了触发事件和定时器输出上的转换之间的延迟。它必须在单脉冲模式 (在 TIMx\_CR1 寄存器中设置 OPM 位) 中使用，以使输出脉冲在启动触发后尽快开始。

0：即使触发开启，CC1 也将根据计数器和 CCR1 值正常工作。触发输入出现边沿时，激活 CC1 输出的最短延迟时间为 5 个时钟周期。

1：触发输入上出现有效边沿相当于 CC1 输出上的比较匹配。随后，无论比较结果如何，OC 都设置为比较电平。采样触发输入和激活 CC1 输出的延迟时间缩短为 3 个时钟周期。仅当通道配置为 PWM1 或 PWM2 模式时，OCFE 才会起作用。

Bit 1:0 **CC1S[1:0]**：捕获/比较 1 选择 (Capture/Compare 1 selection)

此位域定义通道方向 (输入/输出) 以及所使用的输入。

00：CC1 通道配置为输出

01：CC1 通道配置为输入，tim\_ic1 映射到 tim\_ti1 上

其他：保留

注：仅当通道关闭时 (TIMx\_CCER 中的 CC1E = 0)，才可向 CC1S 位写入数据

## 25.8.8 TIMx 捕获/比较使能寄存器 (TIMx\_CCER) (x=16~17)

偏移地址: 0x020

复位值: 0x0000

| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3     | 2     | 1    | 0    |
|----------|----|----|----|----|----|---|---|---|---|---|---|-------|-------|------|------|
| Reserved |    |    |    |    |    |   |   |   |   |   |   | CC1NP | CC1NE | CC1P | CC1E |
|          |    |    |    |    |    |   |   |   |   |   |   | rw    | rw    | rw   | rw   |

Bit 15:4 保留, 必须保持复位值。

Bit 3 **CC1NP**: 输入/捕获 1 输出极性 (Capture/Compare 1 output polarity)

**CC1 通道配置为输出:**

0: tim\_oc1n 高电平有效。

1: tim\_oc1n 低电平有效。

**CC1 通道配置为输入:**

此位与 CC1P 配合使用, 用以定义 tim\_tifp1 的极性。请参见 CC1P 说明。

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 2 或 3 且 CC1S=00 (通道配置为输出), 此位立即变为不可写状态

此位将在具有互补输出的通道上进行预装载。如果 TIMx\_CR2 寄存器中的 CCPC 位置 1, 则仅当生成换向事件时, CC1NP 有效位才会从预装载位获取新值。

Bit 2 **CC1NE**: 输入/捕获 1 互补输出使能 (Capture/Compare 1 complementary output enable)

0: 关闭 - OC1N 未激活。OC1N 电平是 MOE、OSSI、OSSR、OIS1、OIS1N 和 CC1E 位的函数。

1: 开启 - 在相应输出引脚上输出 OC1N 信号, 具体取决于 MOE、OSSI、OSSR、OIS1、OIS1N 和 CC1E 位。

Bit 1 **CC1P**: 输入/捕获 1 输出极性 (Capture/Compare 1 output polarity)

0: OC1 高电平有效 (输出模式) / 边沿灵敏度选择 (输入模式, 参见以下)

1: OC1 低电平有效 (输出模式) / 边沿灵敏度选择 (输入模式, 参见以下)

**CC1 通道配置为输入**, CC1NP/CC1P 位可针对触发或捕获操作选择 TI1FP1 和 TI2FP1 的极性。

CC1NP=0, CC1P=0: 非反相/上升沿触发。电路对 TIxFP1 上升沿敏感 (在复位模式、外部时钟模式或触发模式下执行捕获或触发操作), TIxFP1 未反相 (在门控模式或编码器模式下执行触发操作)。

CC1NP=0, CC1P=1: 反相/下降沿触发。电路对 TIxFP1 下降沿敏感 (在复位模式、外部时钟模式或触发模式下执行捕获或触发操作), TIxFP1 反相 (在门控模式或编码器模式下执行触发操作)。

CC1NP=1, CC1P=1: 非反相/上升沿和下降沿均触发。电路对 TIxFP1 上升沿和下降沿都敏感 (在复位模式、外部时钟模式或触发模式下执行捕获或触发操作), TIxFP1 未反相 (在门控模式下执行触发操作)。编码器模式下不得使用此配置。

CC1NP=1, CC1P=0: 此配置是保留的, 不应该使用此配置。

*注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 2 或 3 且 CC1S=00 (通道配置为输出), 此位立即变为不可写状态*

*此位将在具有互补输出的通道上进行预装载。如果 TIMx\_CR2 寄存器中的 CCPC 位置 1, 则仅当生成换向事件时, CC1P 有效位才会从预装载位获取新值。*

Bit 0 **CC1E**: 输入/捕获 1 输出使能 (Capture/Compare 1 output enable)

0: 禁止捕获模式/OC1 未激活 (参见以下)

1: 使能捕获模式/ OC1 信号输出到对应的输出引脚

**当 CC1 通道配置为输出**, C1 的输出电平依赖于 MOE、OSSI、OSSR、OIS1、OIS1N 和 CC1NE 位的值, 不论 CC1E 位状态。请参考下表获得更多细节

表 25.15 带刹车功能的互补输出通道 tim\_oc1 和 tim\_oc1n 的控制位 (TIM16/TIM17)

| 控制位      |           |           |           |            | 输出状态 <sup>(1)</sup>                                                                                                                           |                                                   |
|----------|-----------|-----------|-----------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| MOE<br>位 | OSSI<br>位 | OSSR<br>位 | CC1E<br>位 | CC1NE<br>位 | tim_oc1 输出状态                                                                                                                                  | tim_oc1n 输出状态                                     |
| 1        | X         | X         | 0         | 0          | 禁止输出（不由定时器驱动: Hi-Z）<br>tim_oc1=0, tim_oc1n=0                                                                                                  |                                                   |
|          |           | 0         | 0         | 1          | 禁止输出（不由定时器驱动: Hi-Z）<br>tim_oc1=0                                                                                                              | tim_oc1ref + 极性 tim_oc1n<br>=tim_oc1ref 异或 CC1NP  |
|          |           | 0         | 1         | 0          | tim_oc1ref + 极性<br>tim_oc1=tim_oc1ref 异或 CC1P                                                                                                 | 禁止输出（不由定时器驱动: Hi-Z）<br>tim_oc1n=0                 |
|          |           | X         | 1         | 1          | OCREF+极性+死区                                                                                                                                   | OCREF 互补项（而非 OCREF）<br>+极性+死区                     |
|          |           | 1         | 0         | 1          | 关闭状态（输出使能为无效状态）<br>tim_oc1=CC1P                                                                                                               | tim_oc1ref + 极性 tim_oc1n<br>=tim_oc1ref x 或 CC1NP |
|          |           | 1         | 1         | 0          | tim_oc1ref + 极性<br>tim_oc1=tim_oc1ref 异或 CC1P                                                                                                 | 关闭状态（输出使能为无效状态）<br>tim_oc1n=CC1NP                 |
| 0        | 0         | X         | X         | X          | 禁止输出（不由定时器驱动: Hi-Z）                                                                                                                           |                                                   |
|          | 0         |           | 0         |            |                                                                                                                                               |                                                   |
|          | 1         |           | 0         | 1          | 关闭状态（输出使能为无效状态）<br>异步: tim_oc1=CC1P, tim_oc1n=CC1NP<br>然后如果存在时钟: 在死区后 tim_oc1=OIS1 且 tim_oc1n=OIS1N, 从而假定 OIS1 和 OIS1N 在有效状态下与 OC1 和 OC1N 不对应 |                                                   |
|          |           |           | 1         | 0          |                                                                                                                                               |                                                   |
|          |           |           | 1         | 1          |                                                                                                                                               |                                                   |

1. 如果一个通道的两个输出均未使用 (被 GPIO 控制器掌控), 则 OIS1、OIS1N、CC1P 和 CC1NP 位必须保持清零状态。

注: 与互补通道 tim\_oc1 和 tim\_oc1n 相连的外部 I/O 引脚的状态取决于通道 tim\_oc1 和 tim\_oc1n 的状态以及 GPIO 寄存器。

### 25.8.9 TIMx 计数器 (TIMx\_CNT) (x=16~17)

偏移地址: 0x024

复位值: 0x0000 0000

| 31        | 30       | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|-----------|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| UIF       | Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| CPY       |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| r         |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15        | 14       | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CNT[15:0] |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw        | rw       | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31 UIFCPY: 复制 UIF

该位是 TIMx\_SR 寄存器中 UIF 位的一个只读副本。如果 TIMx\_CR1 上的 UIFREMAP 位被清零, 则位 31 被保留并读为 0。

Bit 30:16 保留, 必须保持复位值。

Bit 15:0 CNT[15:0]: 计数器的值 (Counter value)

在非抖动模式下 (DITHEN = 0)

该寄存器 (CNT[15:0]) 保存的是计数器的值。

在抖动模式下 (DITHEN = 1)

该寄存器 (CNT[15:0]) 只保存计数器的不抖动部分。抖动部分不可用。

### 25.8.10 TIMx 预分频器 (TIMx\_PSC) (x=16~17)

偏移地址: 0x028

复位值: 0x0000

| 15        | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
|-----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| PSC[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw        | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 15:0 PSC[15:0]: 预分频器的值 (Prescaler value)

计数器时钟频率 ( $f_{tim\_cnt\_ck}$ ) 等于  $f_{tim\_psc\_ck} / (PSC[15:0] + 1)$ 。

PSC 包含每次发生更新事件 (包括计数器通过 TIMx\_EGR 寄存器中的 UG 位清零时, 或在配置为“复位模式”时通过触发控制器清零时) 时要装载到活动预分频器寄存器的值。

### 25.8.11 TIMx 自动重装载寄存器 (TIMx\_ARR) (x=16~17)

偏移地址: 0x02C

复位: 0x0000 FFFF

|           |    |    |    |    |    |    |    |    |    |    |    |            |    |    |    |
|-----------|----|----|----|----|----|----|----|----|----|----|----|------------|----|----|----|
| 31        | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19         | 18 | 17 | 16 |
| Reserved  |    |    |    |    |    |    |    |    |    |    |    | ARR[19:16] |    |    |    |
|           |    |    |    |    |    |    |    |    |    |    |    | rw         | rw | rw | rw |
| 15        | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3          | 2  | 1  | 0  |
| ARR[15:0] |    |    |    |    |    |    |    |    |    |    |    |            |    |    |    |
| rw        | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw         | rw | rw | rw |

Bit 31:20 保留, 必须保持复位值。

Bit 19:0 **ARR[19:0]**: 自动重装载的值 (Auto-reload value)

ARR 为要装载到实际自动重载寄存器的值。

当自动重载值为空时, 计数器不工作。

在非抖动模式下 ( $DITHEN = 0$ )

该寄存器存储的是自动重新加载的值

在抖动模式下 ( $DITHEN = 1$ )

该寄存器存储的是 ARR[19:4]中的整数部分, 而 ARR[3:0]位域存储的是抖动的部分

### 25.8.12 TIMx 重复计数寄存器 (TIMx\_RCR) (x=16~17)

偏移地址: 0x030

复位值: 0x0000

|          |    |    |    |    |    |   |   |          |    |    |    |    |    |    |    |
|----------|----|----|----|----|----|---|---|----------|----|----|----|----|----|----|----|
| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7        | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    |    |    |   |   | REP[7:0] |    |    |    |    |    |    |    |
|          |    |    |    |    |    |   |   | rw       | rw | rw | rw | rw | rw | rw | rw |

Bit 15:8 保留, 必须保持复位值。

Bit 7:0 **REP[7:0]**: 重复计数器的值 (Repetition counter value)

使能预装载寄存器时, 用户可通过这些位设置比较寄存器的更新频率 (即从预装载寄存器向活动寄存器周期性传输数据); 使能更新中断时, 也可设置更新中断的生成速率。

与 REP\_CNT 相关的减计数器每次计数到 0 时, 都将生成一个更新事件并且计数器从 REP 值重新开始计数。由于只有生成重复更新事件 U\_RC 时, REP\_CNT 才会重载 REP 值, 因此在生成下一重复更新事件之前, 无论向 TIMx\_RCR 寄存器写入何值都无影响。

这意味着 PWM 模式下 ( $REP+1$ ) 相当于:

- 边沿对齐模式下的 PWM 周期数;
- 中心对称模式下的 PWM 半周期数;



### 25.8.13 TIMx 捕获/比较寄存器 1 (TIMx\_CCR1) (x=16~17)

偏移地址: 0x034

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|--------------|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19           | 18 | 17 | 16 |
| Reserved    |    |    |    |    |    |    |    |    |    |    |    | CCR1 [19:16] |    |    |    |
|             |    |    |    |    |    |    |    |    |    |    |    | rw           | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3            | 2  | 1  | 0  |
| CCR1 [15:0] |    |    |    |    |    |    |    |    |    |    |    |              |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw           | rw | rw | rw |

Bit 31:20 保留, 必须保持复位值。

Bit 19:0 **CCR1[19:0]**: 捕获/比较 1 值 (Capture/Compare 1 value)

**若 CC1 通道配置为输出:**

CCR1 为要装载到实际捕获/比较 1 寄存器的值 (预装载值)。

如果没有通过 TIMx\_CCMR1 寄存器中的 OC1PE 位来使能预装载功能, 写入的数值会被直接传输至当前寄存器中。否则只有发生更新事件时, 预装载值才会复制到活动捕获/比较 1 寄存器中。实际捕获/比较寄存器中包含要与计数器 TIMx\_CNT 进行比较并在 tim\_oc1 输出上发出信号的值。

在非抖动模式下 (DITHEN = 0)

该寄存器存储的是 CCR1[15:0]中的比较值。CCR1[19:16]位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 CCR1[19:4]中的整数部分。CCR1[3:0]位域存储的是抖动的部分。

**若 CC1 通道配置为输入:**

CCR1 为上一个输入捕获 1 事件 (tim\_ic1) 发生时的计数器值。TIMx\_CCR1 是只读的并且不可编程。

在非抖动模式下 (DITHEN = 0)

CCR1[15:0]位存储了捕获值。CCR1[19:16]位会被重置。

在抖动模式下 (DITHEN = 1)

该寄存器存储捕获在 CCR1[19:4]中。CCR1[3:0]位会被重置。

## 25.8.14 TIMx 刹车和死区寄存器 (TIMx\_BDTR) (x=16~17)

偏移地址: 0x044

复位值: 0x0000 0000

| 31       | 30  | 29  | 28    | 27   | 26     | 25        | 24 | 23       | 22 | 21 | 20 | 19       | 18 | 17 | 16 |
|----------|-----|-----|-------|------|--------|-----------|----|----------|----|----|----|----------|----|----|----|
| Reserved |     |     | BKBID | Res  | BKDSRM | Reserved  |    |          |    |    |    | BKF[3:0] |    |    |    |
|          |     |     | rw    |      | rw     |           |    |          |    |    |    | rw       | rw | rw | rw |
| 15       | 14  | 13  | 12    | 11   | 10     | 9         | 8  | 7        | 6  | 5  | 4  | 3        | 2  | 1  | 0  |
| MOE      | AOE | BKP | BKE   | OSSR | OSSI   | LOCK[1:0] |    | DTG[7:0] |    |    |    |          |    |    |    |
| rw       | rw  | rw  | rw    | rw   | rw     | rw        | rw | rw       | rw | rw | rw | rw       | rw | rw | rw |

注: 由于可以根据 LOCK 配置锁定位 BKBID, BKDSRM, BKF[3:0], AOE, BKP, BKE, OSSI, OSSR 和 DTG[7:0] 的写操作, 因此必须在第一次对 TIMx\_BDTR 寄存器执行写访问时对这些位进行配置。

Bit 31:29 保留, 必须保持复位值。

Bit 28 **BKBID**: 刹车双向 (Break bidirectional)

0: 在输入模式下刹车输入 tim\_brk

1: 在双向模式下刹车输入 tim\_brk

在双向模式 (BKBID 位设置为 1) 中, 刹车输入在输入和输出中都配置模式和开漏输出模式。任何活动中断事件都会在刹车输入, 向外部设备指示内部中断事件。

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 1, 此位即无法修改。

注: 对此位的任何写入操作都需要延迟 1 个 APB 时钟周期才能生效。

Bit 27 保留, 必须保持复位值。

Bit 26 **BKDSRM**: 刹车解除 (Break disarm)

0: 刹车输入 tim\_brk 已装载

1: 刹车输入 tim\_brk 被解除

当没有刹车源处于有效状态时, 此位由硬件清除。

BKDSRM 位必须由软件设置, 以释放双向输出控制 (高阻状态下的开漏输出), 然后对其进行轮询, 直到它被硬件重置, 表明故障状态已消失。

注: 对此位的任何写入操作都需要延迟 1 个 APB 时钟周期才能生效。

Bit 25:20 保留, 必须保持复位值。

Bit 19:16 **BKF[3:0]**: 刹车滤波器

这几位定义了 tim\_brk 输入的采样频率及数字滤波器长度。数字滤波器由一个事件计数器组成, 它记录到 N 个事件后会产生一个输出的跳变

0000: No filter, BRK acts asynchronously

0001:  $f_{\text{SAMPLING}} = f_{\text{tim\_ker\_ck}}$ ,  $N=2$ 。

0010:  $f_{\text{SAMPLING}} = f_{\text{tim\_ker\_ck}}$ ,  $N=4$ 。

0011:  $f_{\text{SAMPLING}} = f_{\text{tim\_ker\_ck}}$ ,  $N=8$ 。

0100:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/2$ ,  $N=6$ 。

0101:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/2$ ,  $N=8$ 。

0110:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/4$ ,  $N=6$ 。

0111:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/4$ ,  $N=8$ 。

1000:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/8$ ,  $N=6$ 。

1001:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/8$ ,  $N=8$ 。

1010:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$ ,  $N=5$ 。

1011:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$ ,  $N=6$ 。

1100:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$ ,  $N=8$ 。

1101:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$ ,  $N=5$ 。

1110:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$ ,  $N=6$ 。

1111:  $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$ ,  $N=8$ 。

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 1, 此位即无法修改。

## Bit 15 MOE: 主输出使能 (Main output enable)

只要刹车输入变为有效状态, 此位便由硬件异步清零。此位由软件置 1, 也可根据 AOE 位状态自动置 1。此位仅对配置为输出的通道有效。

0: tim\_oc1 和 tim\_oc1n 输出禁止或被强制为空闲状态。

1: 如果 tim\_oc1 和 tim\_oc1n 输出的相应使能位 (TIMx\_CCER 寄存器中的 CCxE 和 CCxNE 位) 均置 1, 则使能 OC 和 OCN 输出。

有关详细信息, 请参见 tim\_oc1 / tim\_oc1n 使能说明 (TIMx 捕获/比较使能寄存器 (TIMx\_CCER)(x=16~17))。

## Bit 14 AOE: 自动输出使能 (Automatic output enable)

0: MOE 只能由软件置 1

1: MOE 可由软件置 1, 也可在发生下一更新事件时自动置 1(如果 tim\_brk 输入无效)

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 1, 此位即无法修改。

## Bit 13 BKP: 刹车极性 (Break polarity)

0: 刹车输入低电平有效;

1: 刹车输入高电平有效。

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 1, 此位即无法修改。

注: 对此位的任何写入操作都需要延迟 1 个 APB 时钟周期才能生效。

## Bit 12 BKE: 刹车使能 (Break enable)

0: 禁止刹车输入 (tim\_brk 及 tim\_sys\_brk 事件);

1: 开启刹车输入 (tim\_brk 及 tim\_sys\_brk 事件)。

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 1, 此位即无法修改。

注: 对此位的任何写入操作都需要延迟 1 个 APB 时钟周期才能生效。

## Bit 11 OSSR: 运行模式下关闭状态选择 (Off-state selection for Run mode)

此位在 MOE=1 时作用于配置为输出模式且具有互补输出的通道。如果定时器中没有互补输出, 则不存在 OSSR。

有关详细信息, 请参见 tim\_oc1 / tim\_oc1n 使能说明 (TIMx 捕获/比较使能寄存器 (TIMx\_CCER)(x=16~17))。

0: 处于无效状态时, 禁止 tim\_oc1/ tim\_oc1n 输出 (定时器释放被 GPIO 接管的输出控制并强制为高阻态)。

1: 处于无效状态时, 一旦 CC1E=1 或 CC1NE=1, 便使能 tim\_oc1/ tim\_oc1n 输出并将其设为无效电平。(输出仍然由定时器控制)

注: 编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 2 后, 此位即无法修改。

**Bit 10 OSSI: 空闲模式下的关闭状态选择 (Off-state selection for Idle mode)**

此位在 MOE=0 时作用于配置为输出的通道。

有关详细信息, 请参见 tim\_oc1 / tim\_oc1n 使能说明 (TIMx 捕获/比较使能寄存器 (TIMx\_CCER)(x=16~17))。

0: 处于无效状态时, 禁止 tim\_oc1/ tim\_oc1n 输出 (tim\_oc1/ tim\_oc1n 使能输出信号 =0)。

1: 处于无效状态时, 一旦 CCxE=1 或 CCxNE=1, 便将 tim\_oc1/ tim\_oc1n 输出首先强制为其空闲电平。然后设置 tim\_oc1/ tim\_oc1n 使能输出信号=1

注: 编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 2 后, 此位即无法修改。

**Bit 9:8 LOCK[1:0]: 锁定设置 (Lock configuration)**

这些位用于针对软件错误提供写保护。

00: 关闭锁定--不对任何位提供写保护。

01: 锁定级别 1, 此时无法对 TIMx\_BDTR 寄存器中的 DTG 位、TIMx\_CR2 寄存器中的 OISx 和 OISxN 位以及 TIMx\_BDTR 寄存器中的 BKE/BKP/AOE 位执行写操作。

10: 锁定级别 2, 此时无法对锁定级别 1 中适用的各位、CC 极性位 (TIMx\_CCER 寄存器中的 CCxP/CCxNP 位, 只要通过 CCxS 位将相关通道配置为输出) 以及 OSSR 和 OSSI 位执行写操作。

11: 锁定级别 3, 此时无法对锁定级别 2 中适用的各位、CC 控制位 (TIMx\_CCMRx 寄存器中的 OCxM 和 OCxPE 位, 只要通过 CCxS 位将相关通道配置为输出) 执行写操作。

注: 复位后只能对 LOCK 位执行一次写操作。对 TIMx\_BDTR 寄存器执行写操作后其中的内容将冻结, 直到下一次复位。

**Bit 7:0 DTG[7:0]: 配置死区发生器 (Dead-time generator setup)**

此位域定义插入到互补输出之间的死区持续时间。DT 与该持续时间相对应。

$DTG[7:5]=0xx \Rightarrow DT = DTG[7:0] \times T_{dtg}$ , 其中  $T_{dtg} = T_{DTS}$ ;

$DTG[7:5]=10x \Rightarrow DT = (64+DTG[5:0]) \times T_{dtg}$ , 其中  $T_{dtg} = 2 \times T_{DTS}$ ;

$DTG[7:5]=110 \Rightarrow DT = (32+DTG[4:0]) \times T_{dtg}$ , 其中  $T_{dtg} = 8 \times T_{DTS}$ ;

$DTG[7:5]=111 \Rightarrow DT = (32+DTG[4:0]) \times T_{dtg}$ , 其中  $T_{dtg} = 16 \times T_{DTS}$ ;

例: 若  $T_{DTS} = 125ns$  (8MHZ), 可能的死区时间为:

0 到 15875 ns (步长为 125 ns),

16 us 到 31750 ns (步长为 250 ns),

32 us 到 63us (步长为 1 us),

64 us 到 126 us (步长为 2 us)

注: 编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别 1、2 或 3, 此位即无法修改。

## 25.8.15 TIMx 定时器死区寄存器 2 (TIMx\_DTR2) (x=16~17)

偏移地址: 0x054

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |           |    |    |    |    |    |      |      |
|----------|----|----|----|----|----|----|----|-----------|----|----|----|----|----|------|------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23        | 22 | 21 | 20 | 19 | 18 | 17   | 16   |
| Reserved |    |    |    |    |    |    |    |           |    |    |    |    |    | DTPE | DTAE |
|          |    |    |    |    |    |    |    |           |    |    |    |    |    | rw   | rw   |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7         | 6  | 5  | 4  | 3  | 2  | 1    | 0    |
| Reserved |    |    |    |    |    |    |    | DTGF[7:0] |    |    |    |    |    |      |      |
|          |    |    |    |    |    |    |    | rw        | rw | rw | rw | rw | rw | rw   | rw   |

Bit 31:18 保留, 必须保持复位值。

Bit 17 **DTPE**: 死区时间预加载使能 (Deadtime preload enable)

0: 死区时间值未预加载

1: 使能死区时间值预加载

注: 编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1、2 或 3, 此位即无法修改。

Bit 16 **DTAE**: 死区时间非对称使能 (Deadtime asymmetric enable)

0: 上升沿和下降沿的死区时间是相同的, 并且使用 DTG[7:0]寄存器定义

1: 上升沿的死区时间使用 DTG[7:0]寄存器定义, 下降沿的死区时间使用 DTGF[7:0]位定义。

注: 编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1、2 或 3, 此位即无法修改。

Bit 15:8 保留, 必须保持复位值。

Bit 7:0 **DTGF[7:0]**: 死区下降沿发生器 (Dead-time falling edge generator setup)

这个位域定义了在下落沿时, 互补输出之间插入死区时间的长度。

$DTG[7:5]=0xx \Rightarrow DT = DTG[7:0] \times T_{dtg}$ , 其中  $T_{dtg} = T_{DTS}$ ;

$DTG[7:5]=10x \Rightarrow DT = (64+DTG[5:0]) \times T_{dtg}$ , 其中  $T_{dtg} = 2 \times T_{DTS}$ ;

$DTG[7:5]=110 \Rightarrow DT = (32+DTG[4:0]) \times T_{dtg}$ , 其中  $T_{dtg} = 8 \times T_{DTS}$ ;

$DTG[7:5]=111 \Rightarrow DT = (32+DTG[4:0]) \times T_{dtg}$ , 其中  $T_{dtg} = 16 \times T_{DTS}$ ;

例: 若  $T_{DTS} = 125ns$  (8MHZ), 可能的死区时间为:

0 到 15875 ns (步长为 125 ns),

16 us 到 31750 ns (步长为 250 ns),

32 us 到 63us (步长为 1 us),

64 us 到 126 us (步长为 2 us)

注: 编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1、2 或 3, 此位即无法修改。

## 25.8.16 TIMx 定时器输入选择寄存器 (TIMx\_TISEL) (x=16~17)

偏移地址: 0x05C

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |    |    |    |    |            |    |    |    |
|----------|----|----|----|----|----|----|----|----|----|----|----|------------|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19         | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |            |    |    |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3          | 2  | 1  | 0  |
| Reserved |    |    |    |    |    |    |    |    |    |    |    | TISEL[3:0] |    |    |    |
|          |    |    |    |    |    |    |    |    |    |    |    | rw         | rw | rw | rw |

Bit 31:4 保留, 必须保持复位值。

Bit 3:0 **TISEL[3:0]**: 选择 tim\_ti1[15:0]输入 (Selects tim\_ti1[15:0] input)

0000: TIMx\_CH1 输入 (tim\_ti1\_in0)

0001: tim\_ti1\_in1

...

1111: tim\_ti1\_in15

参考章节: TIM15/TIM16/TIM17 引脚和内部信号获得关联列表。

## 25.8.17 TIMx 复用功能选择寄存器 1 (TIMx\_AF1) (x=16~17)

偏移地址: 0x060

复位值: 0x0000 0001

|          |             |             |             |             |       |          |    |    |    |    |             |             |             |             |       |
|----------|-------------|-------------|-------------|-------------|-------|----------|----|----|----|----|-------------|-------------|-------------|-------------|-------|
| 31       | 30          | 29          | 28          | 27          | 26    | 25       | 24 | 23 | 22 | 21 | 20          | 19          | 18          | 17          | 16    |
| Reserved |             |             |             |             |       |          |    |    |    |    |             |             |             |             |       |
| 15       | 14          | 13          | 12          | 11          | 10    | 9        | 8  | 7  | 6  | 5  | 4           | 3           | 2           | 1           | 0     |
| Reserved | BK<br>CMP4P | BK<br>CMP3P | BK<br>CMP2P | BK<br>CMP1P | BKINP | Reserved |    |    |    |    | BK<br>CMP4E | BK<br>CMP3E | BK<br>CMP2E | BK<br>CMP1E | BKINE |
|          | rw          | rw          | rw          | rw          | rw    |          |    |    |    |    | rw          | rw          | rw          | rw          | rw    |

注: 有关产品特定实现的信息, 请参考章节: TIM15/TIM16/TIM17 引脚和内部信号。

Bit 31:14 保留, 必须保持复位值。

Bit 13 **BKCOMP4P**: tim\_brk\_cmp4 输入极性 (tim\_brk\_cmp4 input polarity)

此位选择 tim\_brk\_cmp2 输入敏感度。它必须与 BKP 极性位一起编程。

0: tim\_brk\_cmp4 输入高电平有效

1: tim\_brk\_cmp4 输入低电平有效

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 12 **BKCOMP3P**: tim\_brk\_cmp3 输入极性 (tim\_brk\_cmp3 input polarity)

此位选择 tim\_brk\_cmp3 输入敏感度。它必须与 BKP 极性位一起编程。

0: tim\_brk\_cmp3 输入高电平有效

1: tim\_brk\_cmp3 输入低电平有效

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 11 **BKCOMP2P**: tim\_brk\_cmp2 输入极性 (tim\_brk\_cmp2 input polarity)

此位选择 tim\_brk\_cmp2 输入敏感度。它必须与 BKP 极性位一起编程。

0: tim\_brk\_cmp2 输入高电平有效

1: tim\_brk\_cmp2 输入低电平有效

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。



Bit 10 **BKCOMP1P**: tim\_brk\_cmp1 输入极性 (tim\_brk\_cmp1 input polarity)

该位选择 tim\_brk\_cmp1 输入敏感度。它必须与 BKP 极性位一起编程。

0: tim\_brk\_cmp1 输入高电平有效

1: tim\_brk\_cmp1 输入低电平有效

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 9 **BKINP**: TIMx\_BKIN 输入极性 (TIMx\_BKIN input polarity)

该位选择 TIMx\_BKIN 备用功能输入敏感度。必须对其进行编程

0: TIMx\_BKIN 输入电平有效

1: TIMx\_BKIN 输入低电平有效

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 8:5 保留, 必须保持复位值。

Bit 4 **BKCMP4E**: tim\_brk\_cmp4 使能 (tim\_brk\_cmp4 enable)

该位用于使能或禁用 tim\_brk\_cmp4 输入。如果该位设置为 1, 则 tim\_brk\_cmp4 输入将被使能, 否则将被禁用。

0: tim\_brk\_cmp4 输入禁用

1: tim\_brk\_cmp4 输入使能

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 4 **BKCMP3E**: tim\_brk\_cmp3 使能 (tim\_brk\_cmp3 enable)

该位用于使能或禁用 tim\_brk\_cmp3 输入。如果该位设置为 1, 则 tim\_brk\_cmp3 输入将被使能, 否则将被禁用。

0: tim\_brk\_cmp3 输入禁用

1: tim\_brk\_cmp3 输入使能

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 2 **BKCMP2E**: tim\_brk\_cmp2 使能 (tim\_brk\_cmp2 enable)

该位用于使能或禁用 tim\_brk\_cmp2 输入。如果该位设置为 1, 则 tim\_brk\_cmp2 输入将被使能, 否则将被禁用。

0: tim\_brk\_cmp2 输入禁用

1: tim\_brk\_cmp2 输入使能

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 1 **BKCMP1E**: tim\_brk\_cmp1 使能 (tim\_brk\_cmp1 enable)

该位用于使能或禁用 tim\_brk\_cmp1 输入。如果该位设置为 1, 则 tim\_brk\_cmp1 输入将被使能, 否则将被禁用。

0: tim\_brk\_cmp1 输入禁用

1: tim\_brk\_cmp1 输入使能

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 0 **BKINE**: TIMx\_BKIN 输入使能 (TIMx\_BKIN input enable)

该位用于使能或禁用 TIMx\_BKIN alternate function 输入作为定时器的 tim\_brk 输入。TIMx\_BKIN 输入与其他 tim\_brk 源进行“或”操作。

0: TIMx\_BKIN 输入禁用

1: TIMx\_BKIN 输入使能

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。



## 25.8.18 TIMx 复用功能选择寄存器 2 (TIMx\_AF2) (x=16~17)

偏移地址: 0x064

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |    |    |    |    |    |             |    |    |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|-------------|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18          | 17 | 16 |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    | OCRSEL[2:0] |    |    |
|          |    |    |    |    |    |    |    |    |    |    |    |    | rw          | rw | rw |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2           | 1  | 0  |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |             |    |    |

Bit 31:19 保留, 必须保持复位值。

Bit 18:16 **OCRSEL[2:0]**: tim\_ocref\_clr 源选择 (tim\_ocref\_clr source selection)

这些位选择 tim\_ocref\_clr 输入源。

000: tim\_ocref\_clr0

001: tim\_ocref\_clr1

010: tim\_ocref\_clr2

011: tim\_ocref\_clr3

100: tim\_ocref\_clr4

101: tim\_ocref\_clr5

110: tim\_ocref\_clr6

111: tim\_ocref\_clr7

参考章节: TIM15/TIM16/TIM17 引脚和内部信号获取更多特定产品信息。

注: 只要编程了 LOCK (TIMx\_BDTR 寄存器中的 LOCK 位) 级别1, 此位即无法修改。

Bit 15:0 保留, 必须保持复位值。

## 25.8.19 TIMx 选项寄存器 1 (TIMx\_OR1) (x=16~17)

偏移地址: 0x068

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |    |    |    |    |    |    |          |    |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----------|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17       | 16 |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    |          |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1        | 0  |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    | HSE32 EN |    |
|          |    |    |    |    |    |    |    |    |    |    |    |    |    | rw       |    |

Bit 31:1 保留, 必须保持复位值。

Bit 0 **HSE32EN**: HSE 除 32 使能 (HSE Divided by 32 enable)

该位用于使能 tim\_ti\_in3 的 HSE 除 32 的分频器。有关详细信息, 请参阅表: 与 tim\_ti1 输入多路复用器的互联。

0: 禁用 HSE 除 32

1: 使能 HSE 除 32

### 25.8.20 TIMx DMA 控制寄存器 (TIMx\_DCR) (x=16~17)

偏移地址: 0x3DC

复位值: 0x0000 0000

|          |    |    |          |    |    |    |    |          |    |    |          |    |    |    |    |
|----------|----|----|----------|----|----|----|----|----------|----|----|----------|----|----|----|----|
| 31       | 30 | 29 | 28       | 27 | 26 | 25 | 24 | 23       | 22 | 21 | 20       | 19 | 18 | 17 | 16 |
| Reserved |    |    |          |    |    |    |    |          |    |    |          |    |    |    |    |
| 15       | 14 | 13 | 12       | 11 | 10 | 9  | 8  | 7        | 6  | 5  | 4        | 3  | 2  | 1  | 0  |
| Reserved |    |    | DBL[4:0] |    |    |    |    | Reserved |    |    | DBA[4:0] |    |    |    |    |
|          |    |    | rw       | rw | rw | rw | rw |          |    |    | rw       | rw | rw | rw | rw |

Bit 31:13 保留, 必须保持复位值。

Bit 12:8 **DBL[4:0]**: DMA 突发长度 (DMA burst length)

这个 5 位向量定义了 DMA 传输的长度 (当对 TIMx\_DMAR 地址进行读或写访问时, 定时器识别一次突发传输), 即传输次数。传输可以是半字或字节 (见下面的示例)。

00000: 1 次传输

00001: 2 次传输

00010: 3 次传输

...

10001: 18 次传输

Bit 7:5 保留, 必须保持复位值。

Bit 4:0 **DBA[4:0]**: DMA 基地址 (DMA base address)

这个 5 位向量定义了 DMA 传输的基地址 (当通过 TIMx\_DMAR 地址进行读/写访问时)。DBA 是从 TIMx\_CR1 寄存器的地址开始的偏移量。

00000: TIMx\_CR1,

00001: TIMx\_CR2,

00010: TIMx\_SMCR,

...

**样例:** 让我们考虑以下传输: DBL=7 传输和 DBA=TIMx\_CR1。在这种情况下, 从 TIMx\_CR1 地址开始, 传输是到/从 7 个寄存器完成的

### 25.8.21 TIMx 用于完整传输的 DMA 地址 (TIMx\_DMAR) (x=16~17)

偏移地址: 0x3E0

复位值: 0x0000 0000

|             |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31          | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DMAB[31:16] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |
| 15          | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| DMAB[15:0]  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw          | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:0 **DMAB[31:0]**: 用于突发访问的 DMA 寄存器 (DMA register for burst accesses)

对 DMAR 寄存器进行的读或写操作将访问位于以下地址的寄存器:

$$(\text{TIMx\_CR1 地址}) + (\text{DBA} + \text{DMA 索引}) \times 4$$

其中, TIMx\_CR1 地址是控制寄存器 1 的地址, DBA 是在 TIMx\_DCR 寄存器中配置的 DMA 基地址, DMA 索引由 DMA 传输自动控制, 其范围从 0 到 DBL (在 TIMx\_DCR 中配置的 DBL)。

## 26 基本定时器 (TIM6/TIM7)

### 26.1 TIM6/TIM7 简介

基本定时器 TIM6/TIM7 包含一个 16 位自动重载计数器，该计数器由可编程预分频器驱动。

他们可以用作通用定时器以生成时基。

基本定时器还可以用于触发数模转换器 (DAC)。这是通过定时器的触发输出完成的。

这些定时器彼此完全独立，不共享任何资源。

### 26.2 TIM6/TIM7 主要功能

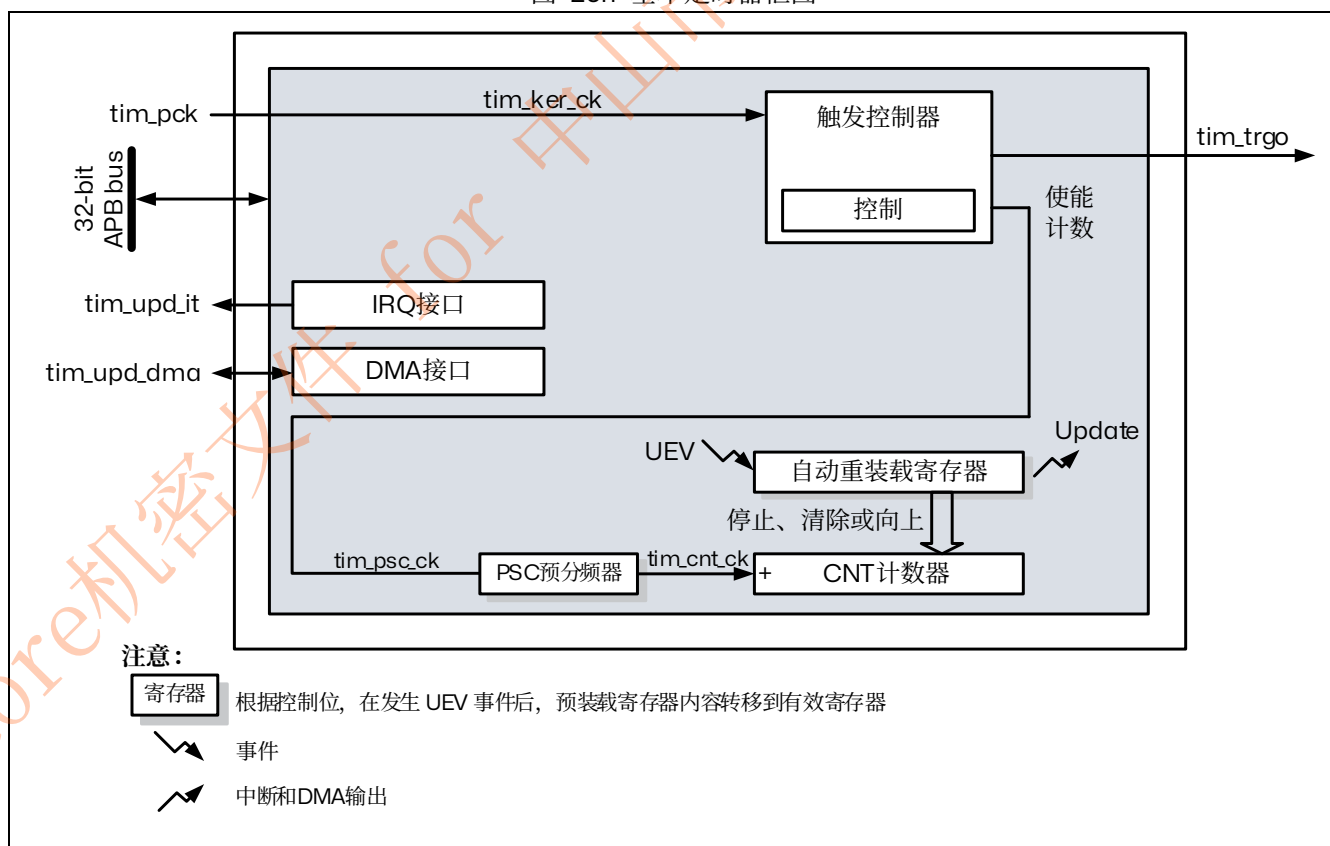
基本定时器 (TIM6/TIM7) 功能包括：

- 16 位自动重载递增计数器
- 16 位可编程预分频器，用于对计数器时钟频率进行分频（即运行时修改），分频系数介于 1 和 65536 之间
- 用于触发 DAC 的同步电路
- 发生如下更新事件时会生成中断/DMA 请求：计数器上溢

### 26.3 TIM6/TIM7 功能描述

#### 26.3.1 框图

图 26.1 基本定时器框图



## 26.3.2 TIM6/TIM7 内部信号

本节中的下表格总结了 TIM 的输入和输出。

表 26.1 TIM 内部输入/输出信号

| 内部信号名称      | 信号类型 | 描述                                                                                       |
|-------------|------|------------------------------------------------------------------------------------------|
| tim_pclk    | 输入   | 定时器 APB 时钟。                                                                              |
| tim_ker_ck  | 输入   | 定时器内核时钟。此时钟必须与 tim_pclk 同步（来自同一源）。时钟比率 tim_ker_ck/ tim_pclk 必须为整数：1, 2, 3, ..., 16（最大值）。 |
| tim_trgo    | 输出   | 内部触发器输出。此触发器可以触发其他片上外设（DAC）。                                                             |
| tim_upd_it  | 输出   | 定时器更新事件中断。                                                                               |
| tim_upd_dma | 输出   | 定时器更新 DMA 请求。                                                                            |

## 26.3.3 TIM6/TIM7 时钟

定时器总线接口由 tim\_pclk APB 时钟驱动。

计数器时钟 tim\_ker\_ck 连接到 tim\_pclk 输入。

CEN 位（TIMx\_CR1 寄存器中）和 UG 位（TIMx\_EGR 寄存器中）为实际控制位，并且只能通过软件进行更改（UG 除外，仍自动清零）。当对 CEN 位写入 1 时，预分频器的时钟就由内部时钟 tim\_ker\_ck 提供。

下图显示了正常模式下控制电路与递增计数器的行为（没有预分频的情况下）。

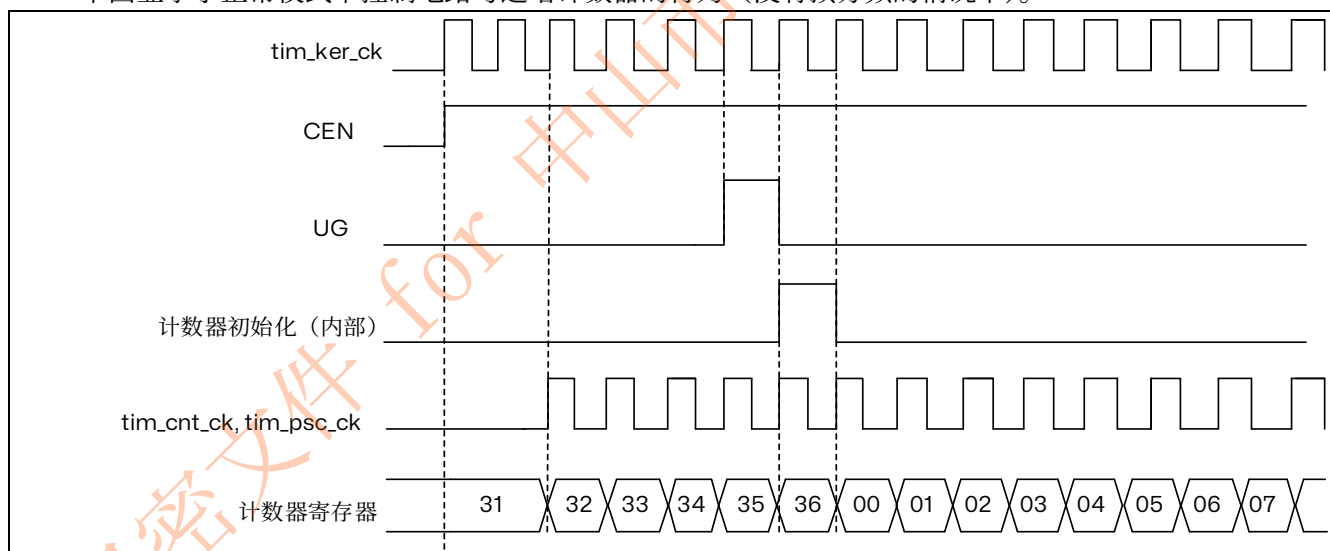


图 26.2 一般模式下的控制电路，内部时钟分频系数为 1

## 26.3.4 时基单元

可编程定时器的主要模块由一个 16 位递增计数器及其相关的自动重载寄存器组成。计数器的时钟可通过预分频器进行分频。

计数器、自动重载寄存器和预分频器寄存器可通过软件进行读写。即使在计数器运行时也可执行读写操作。

时基单元包含：

- 计数器寄存器 (TIMx\_CNT)
- 预分频器寄存器 (TIMx\_PSC)
- 自动重载寄存器 (TIMx\_ARR)

自动重载寄存器是预装载的。每次尝试对自动重载寄存器执行读写操作时，都会访问预装载寄存器。预装载寄存器的内容既可以直接传送到影子寄存器，也可以在每次发生更新事件 UEV 时传送到影子寄存器，这取决于 TIMx\_CR1 寄存器中的自动重载预装载使能位 (ARPE)。当计数器达到上溢值并且 TIMx\_CR1 寄存器中的 UDIS 位为 0 时，将发送更新事件。该更新事件也可由软件产生。下文将针对各配置的更新事件的产生进行详细介绍。

计数器由预分频器输出 CK\_CNT 提供时钟，仅当 TIMx\_CR1 寄存器中的计数器启动位 (CEN) 置 1 时，才会启动计数器。

请注意，实际的计数器使能信号 CNT\_EN 在 CEN 置 1 的一个时钟周期后被置 1。

### 预分频器描述

预分频器可对计数器时钟频率进行分频，分频系数介于 1 和 65536 之间。该预分频器基于 TIMx\_PSC 寄存器中的 16 位寄存器所控制的 16 位计数器。由于 TIMx\_PSC 控制寄存器有缓冲，因此可对预分频器进行实时更改。而新的预分频比将在下一更新事件发生时被采用。

下图给出了在预分频比发生实时变化时一些计数器行为的示例。

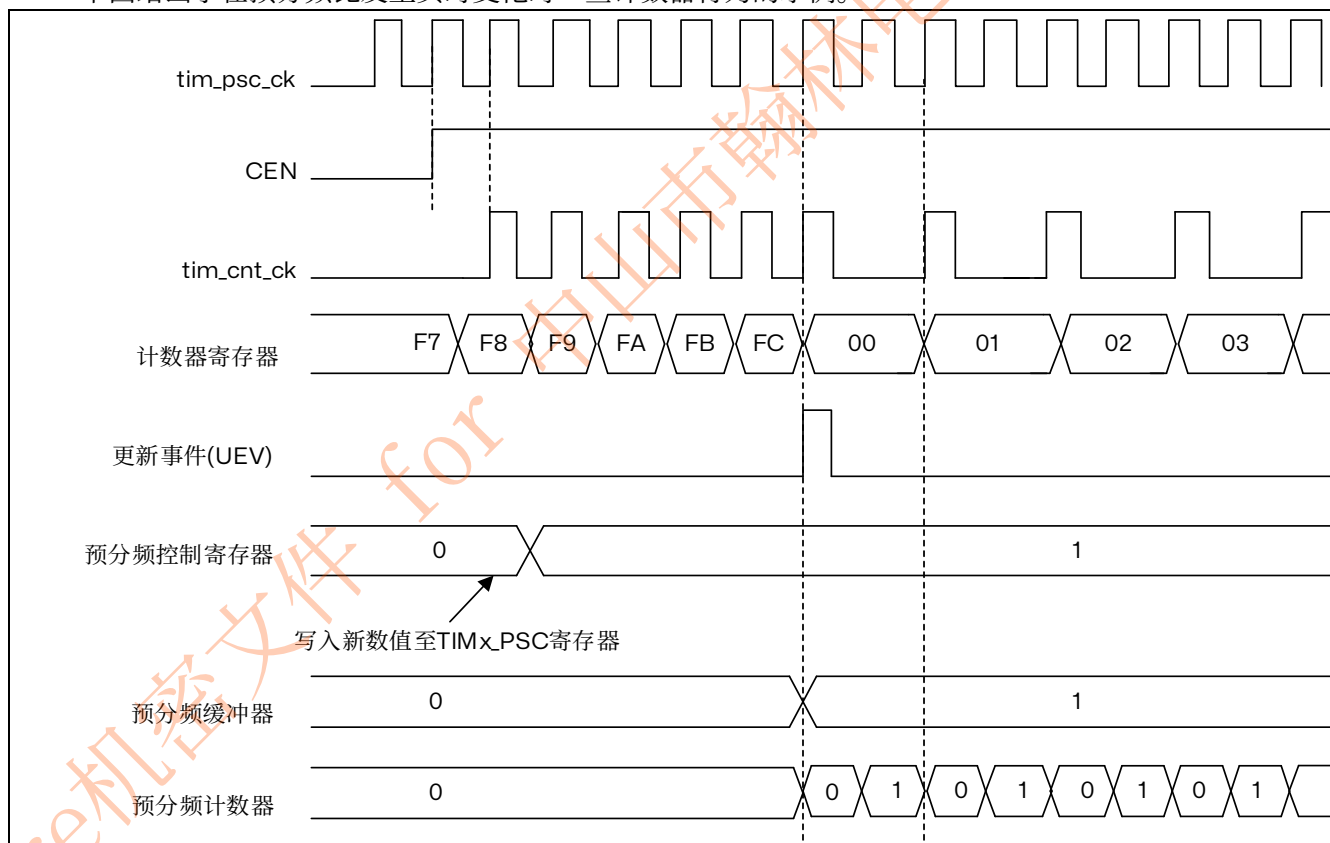


图 26.3 当预分频器的参数从 1 变到 2 时，计数器的时序图

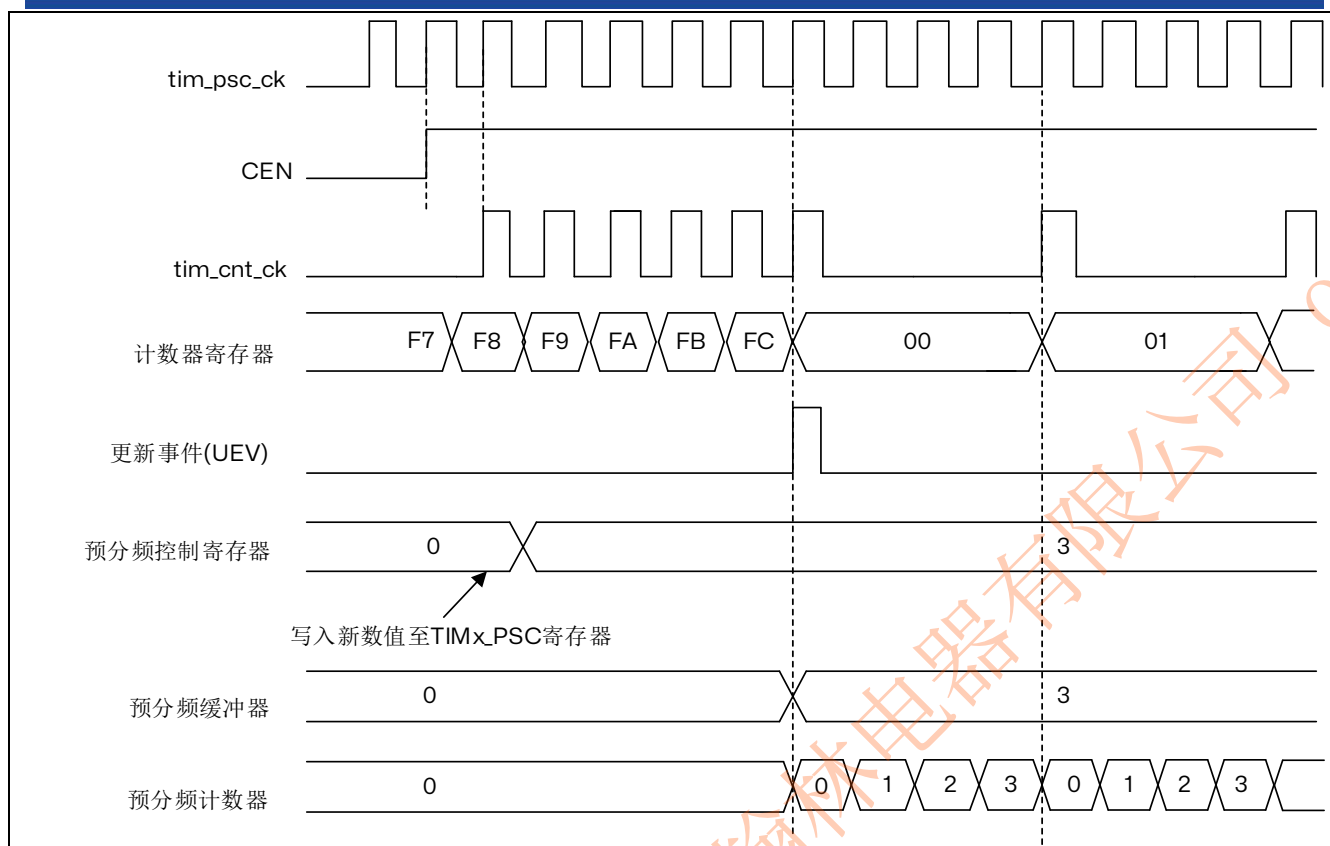


图 26.4 当预分频器的参数从 1 变到 4 时，计数器的时序图

### 26.3.5 计数器模式

计数器从 0 计数到自动重载值 (TIMx\_ARR 寄存器的内容)，然后重新从 0 开始计数并生成计数器上溢事件。

每次发生计数器上溢时会生成更新事件，或将 TIMx\_EGR 寄存器中的 UG 位置 1（通过软件或使用从模式控制器）也可以生成更新事件。

通过软件将 TIMx\_CR1 寄存器中的 UDIS 位置 1 可禁止 UEV 事件。这可避免向预装载寄存器写入新值时更新影子寄存器。这样，直到 UDIS 位中写入 0 前便不会生成任何更新事件，但计数器和预分频器计数器都会重新从 0 开始计数（而预分频比保持不变）。此外，如果 TIMx\_CR1 寄存器中的 URS 位（更新请求选择）已置 1，则将 UG 位置 1 会生成更新事件 UEV，但不会将 UIF 标志置 1（因此，不会发送任何中断请求）。

发生更新事件时，将更新所有寄存器且将更新标志（TIMx\_SR 寄存器中的 UIF 位）置 1（取决于 URS 位）：

- 使用预装载值 (TIMx\_PSC 寄存器的内容) 重新装载预分频器的缓冲区
- 使用预装载值 (TIMx\_ARR) 更新自动重载影子寄存器

以下各图显示了当 TIMx\_ARR=0x36 时不同时钟频率下计数器行为的示例。

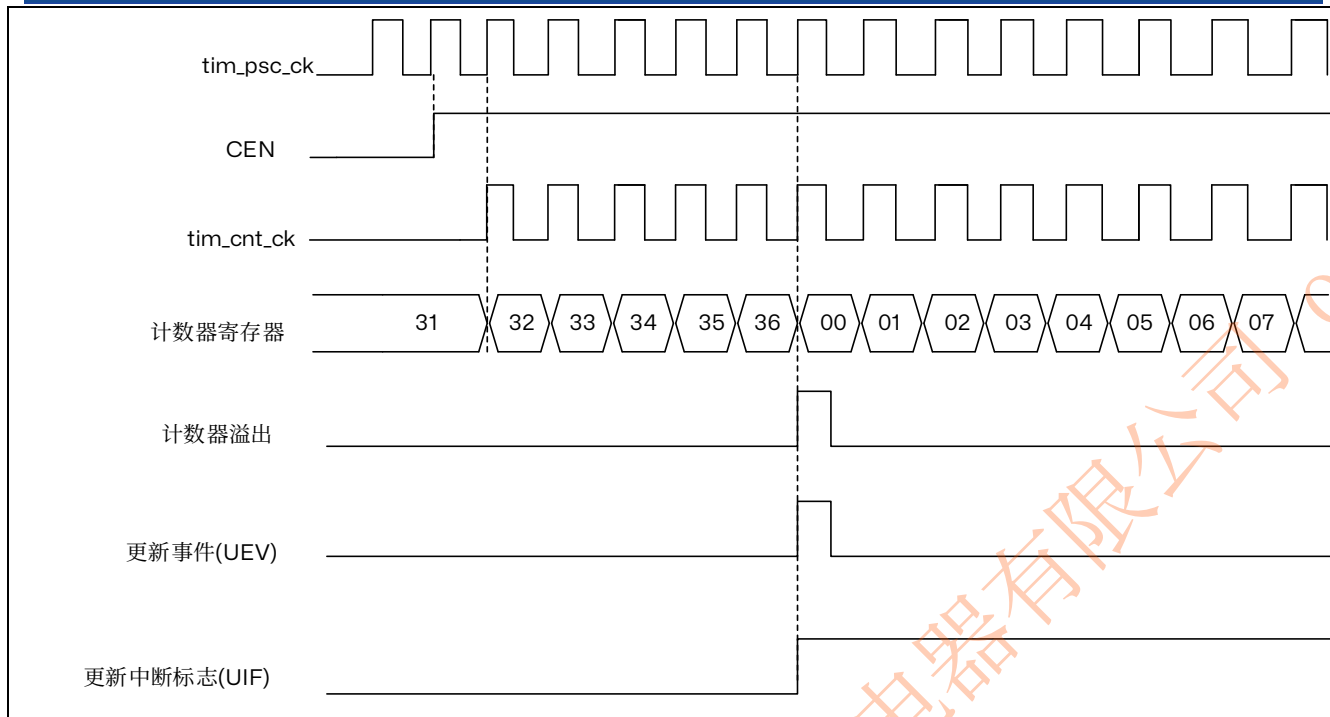


图 26.5 计数器时序图，内部时钟分频系数为1

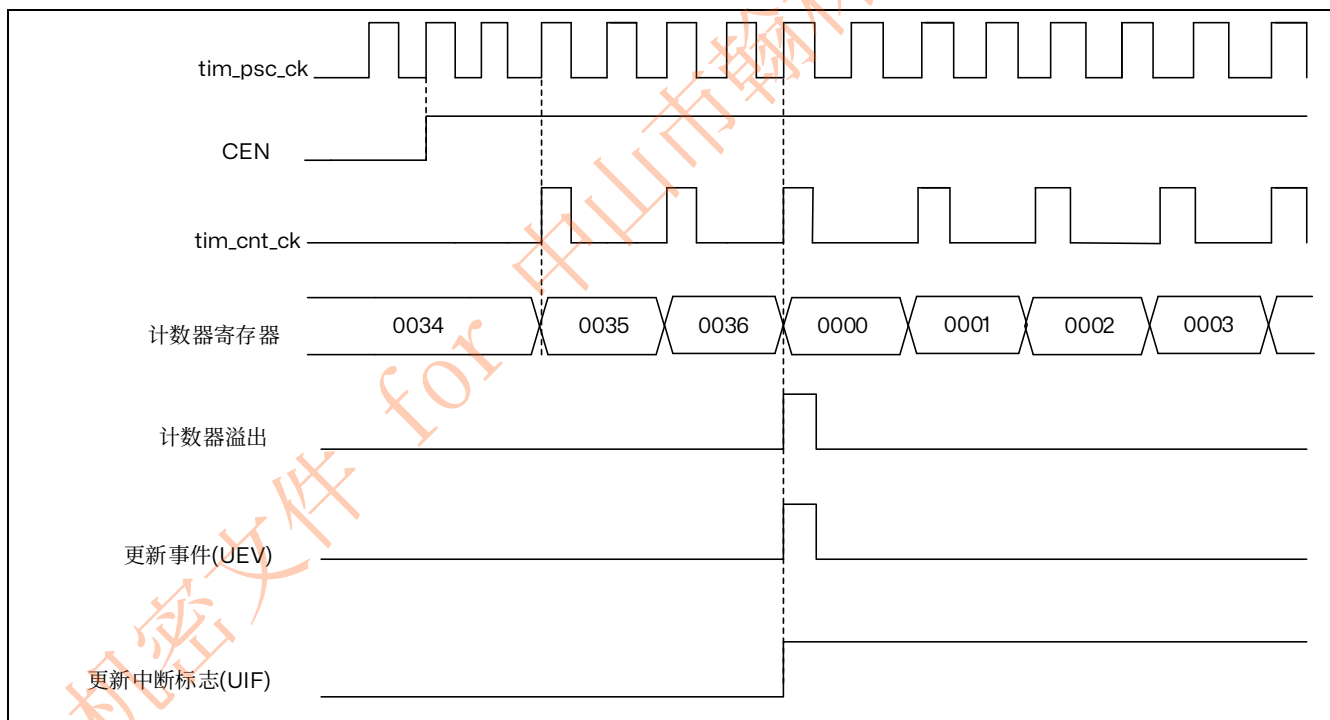


图 26.6 计数器时序图，内部时钟分频系数为2



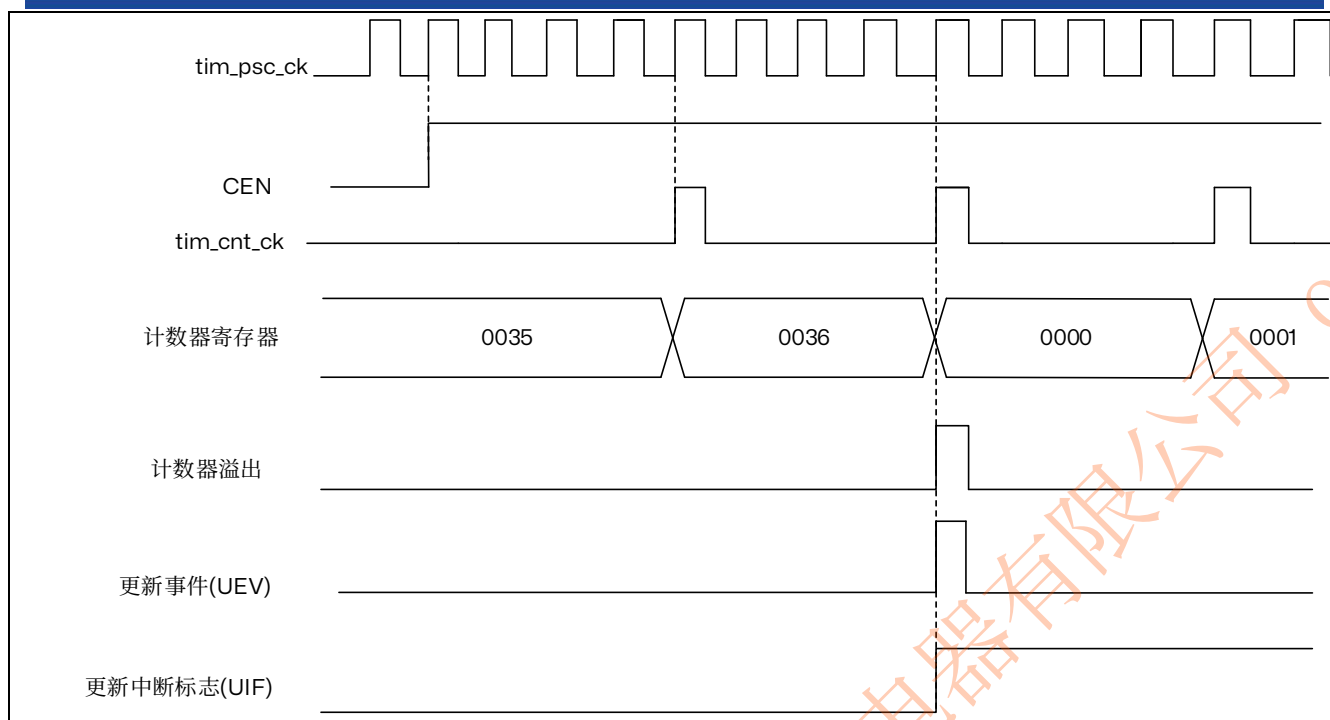


图 26.7 计数器时序图，内部时钟分频系数为 4

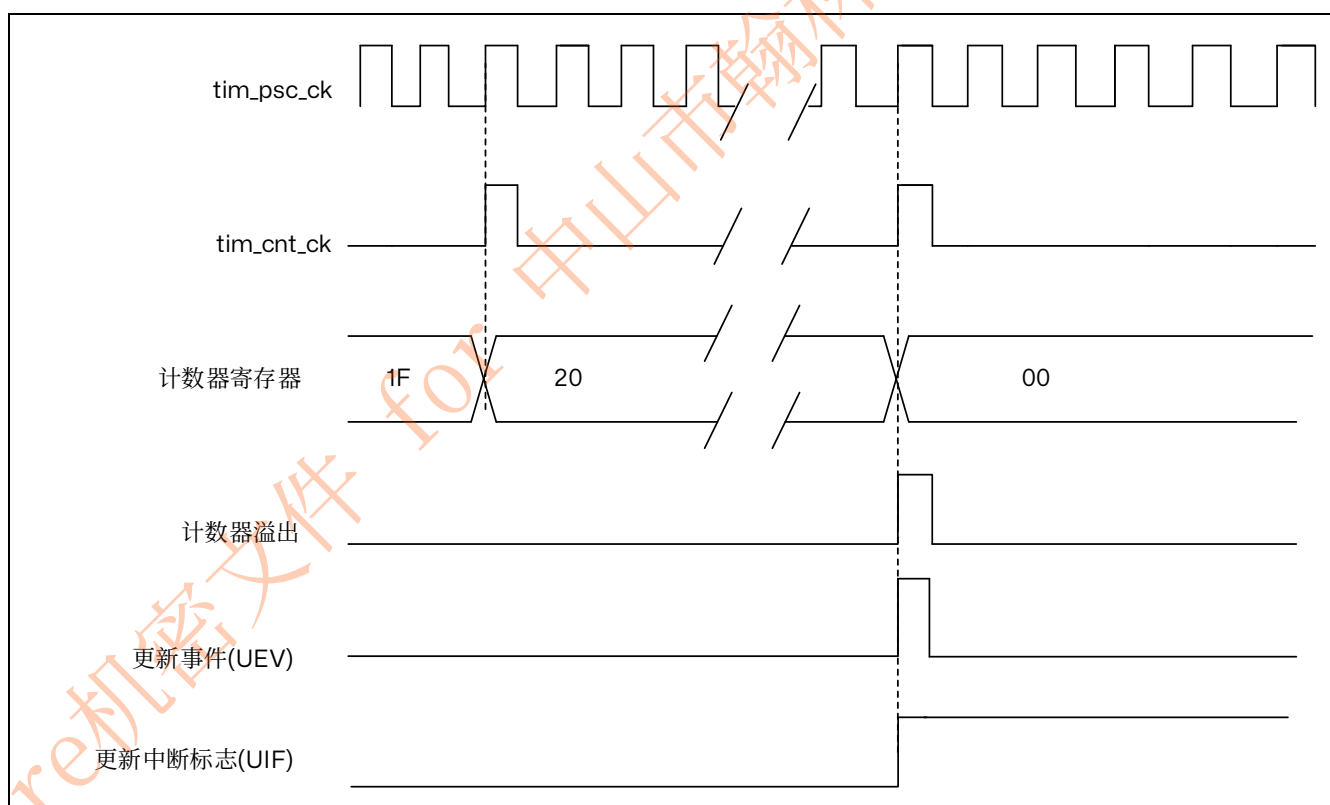


图 26.8 计数器时序图，内部时钟分频系数为 N

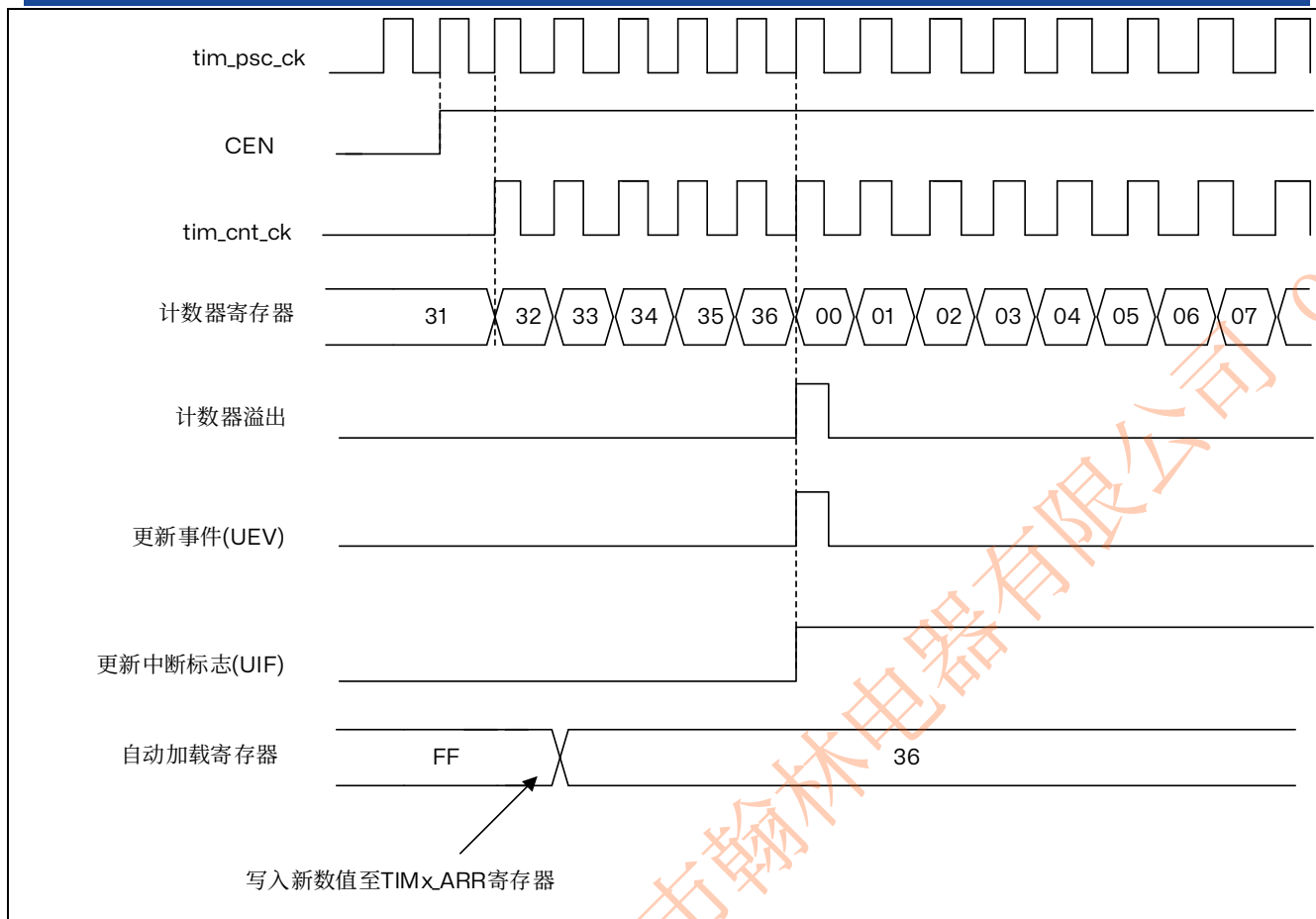


图 26.9 计数器时序图，当  $ARPE=0$  时的更新事件（TIMx\_ARR 没有预装入）

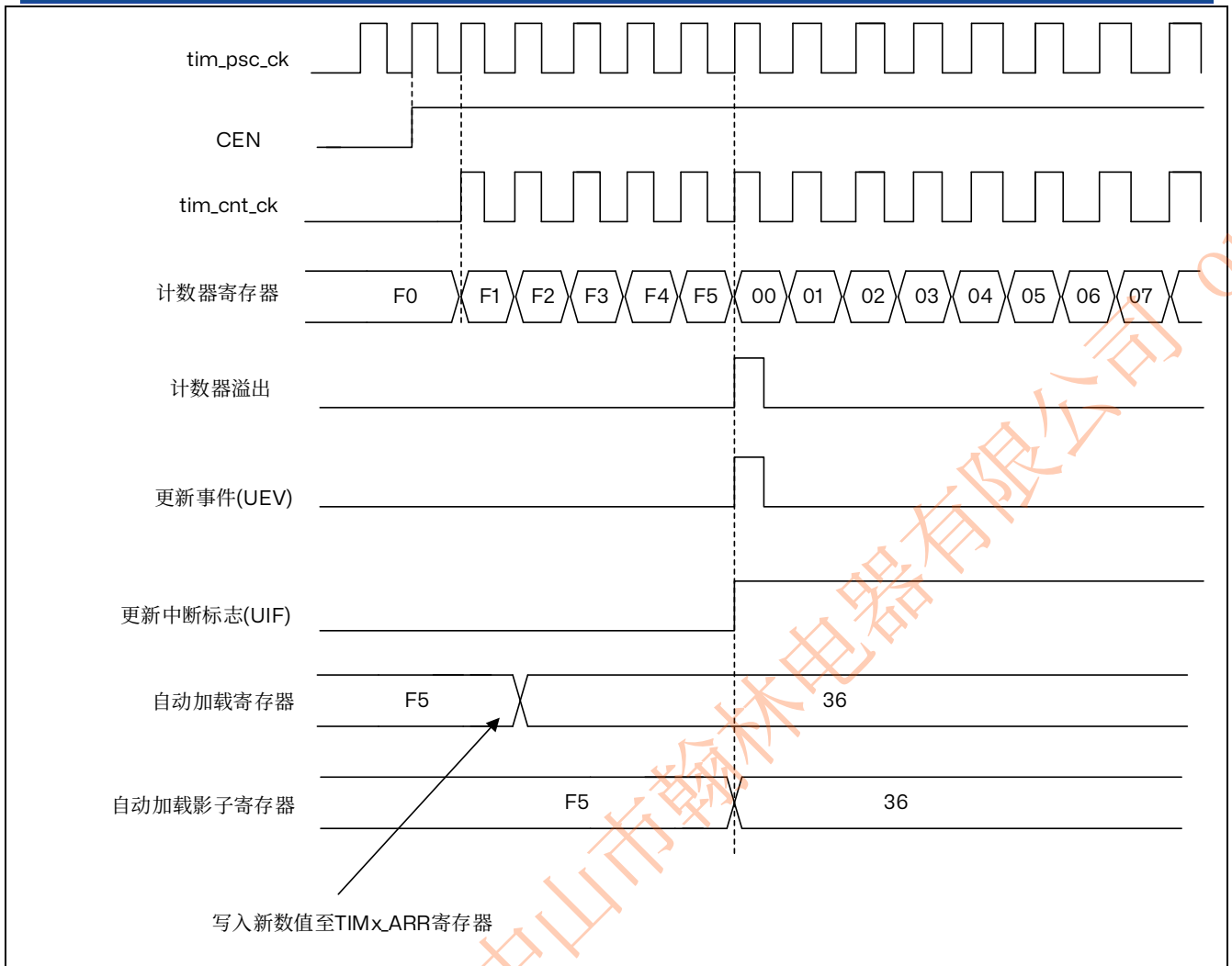


图 26.10 计数器时序图，当 ARPE=1 时的更新事件（预装入了 TIMx\_ARR）

## 抖动模式

通过使用 TIMx\_CR1 寄存器中的 DITHEN 位，可以使能抖动模式，从而提高时间基准的有效分辨率。这会影响 TIMx\_ARR 的行为方式，并可用于调整定时器用作触发器时的平均计数器周期（通常是 DAC）。

工作原理是将实际 ARR 值在 16 个连续计数期间内略有变化（增加或不增加一个定时器时钟周期），并采用预定义的模式。这使得考虑到平均计数周期时，分辨率提高了 16 倍。

下图展示了应用于 4 个连续计数期间的抖动原理。

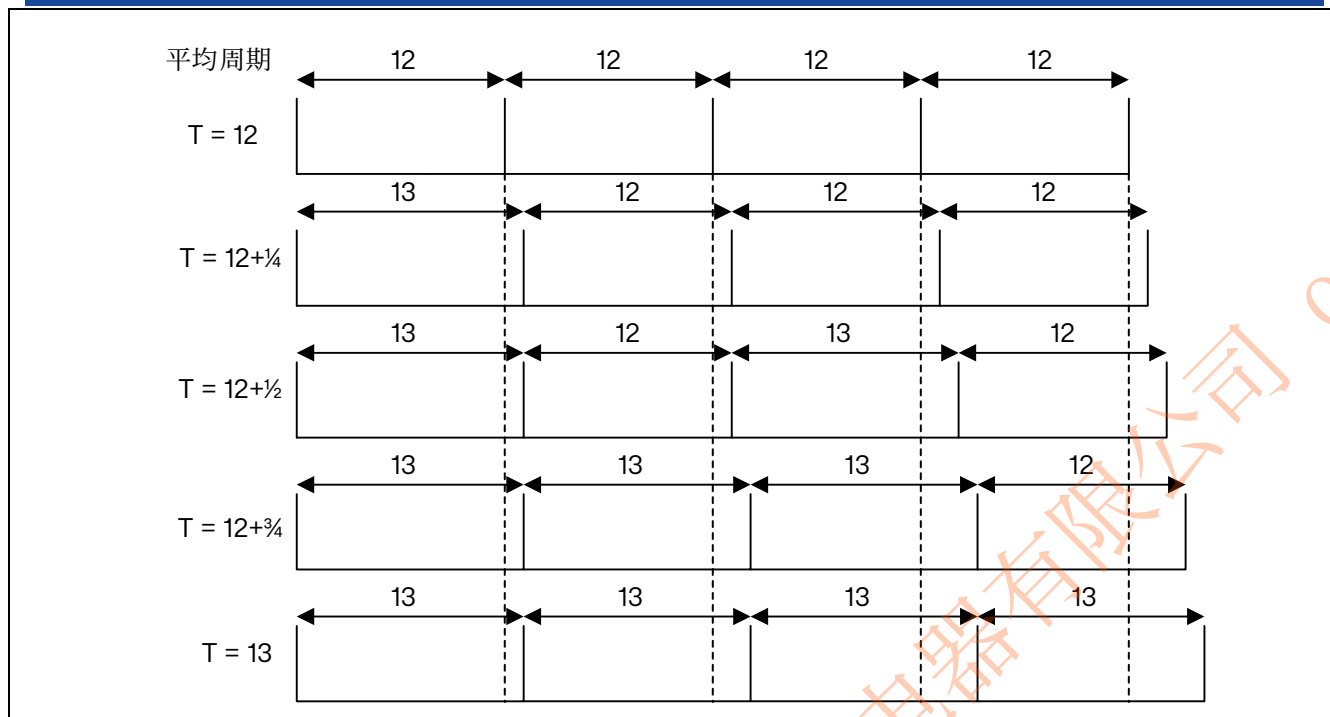


图 26.11 抖动原理

当抖动模式被使能时，寄存器编码将如下所示进行变化（参见下图示例）

- 4 个最低有效位（LSB）用于编码增强的分辨率部分（小数部分）
- 最高有效位（MSB）左移至 19:4 位，用于编码基本值

注：如果设置/清除 DITHEN 位，ARR 值将自动更新。（例如，如果 ARR=0x05 且 DITHEN=0，它将更新为 ARR=0x50 且 DITHEN=1）。

1. 必须重置 CEN 和 ARPE 位
2. ARR[3:0]位必须重置
3. DITHEN 位必须重置
4. 可以设置 CEN 位（最终设置 ARPE=1）。

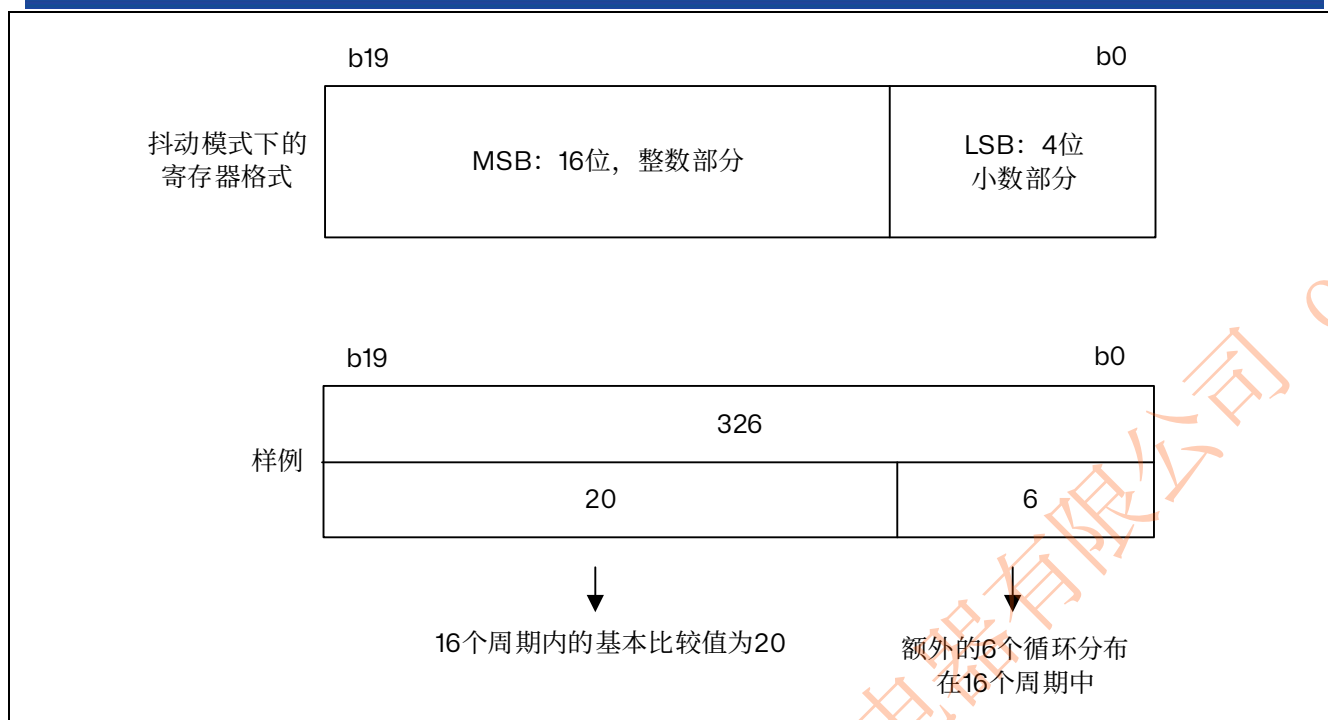


图 26.12 抖动模式下的数据格式和寄存器编码

最小频率由以下公式给出:

$$\text{分辨率} = \frac{FTim}{Fpwm} \Rightarrow FpwmMin = \frac{FTim}{\text{最大分辨率}}$$

$$\text{禁用抖动模式: } FpwmMin = \frac{FTim}{65536}$$

$$\text{使能抖动模式: } FpwmMin = \frac{FTim}{65535 + \frac{15}{16}}$$

注: 在抖动模式下, TIMx\_ARR 的最大值限制为 0xFFFFF (对应于整数部分的 65534 和抖动部分的 15)。

如下图所示, 无论 PWM 频率如何, 抖动模式都可以增加 PWM 分辨率。

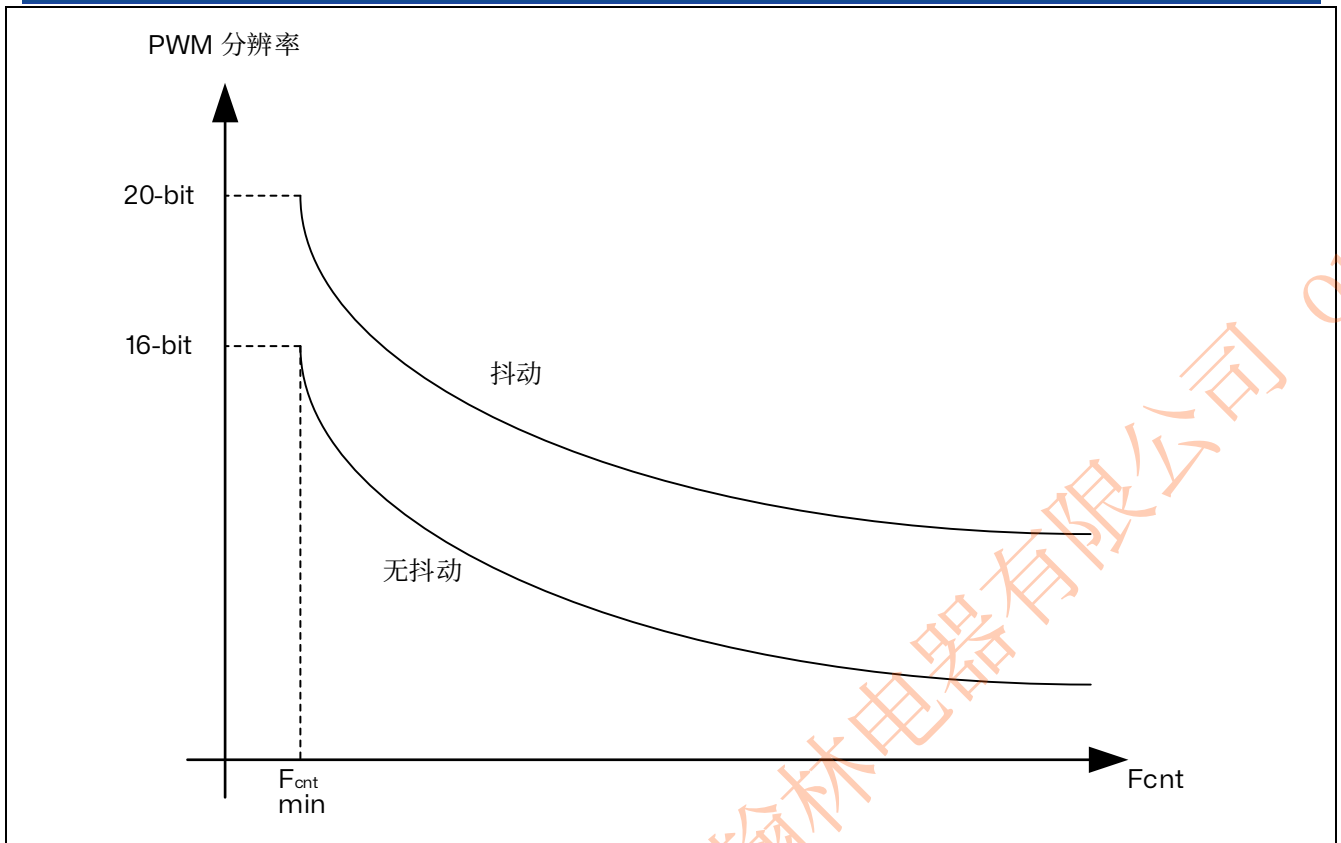


图 26.13 PWM 分辨率 vs 频率

如下图所示，这些周期的变化分散在 16 个连续周期。

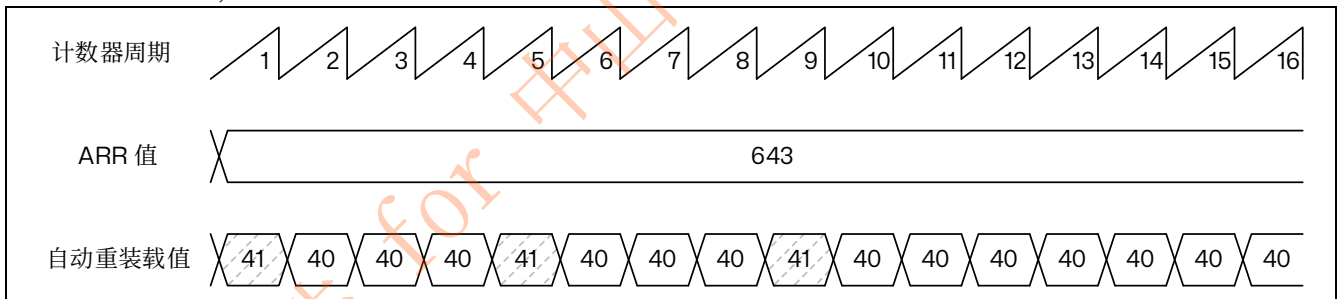


图 26.14 PWM 抖动模式

自动重新加载和比较值的增量按照下表中描述的特定模式分布。抖动序列是为了使增量分布尽可能均匀并最大限度地减少整体波动。

表 26.2 TIMx\_ARR 寄存器的变化抖动模式

| LSB 值 | PWM 周期 |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------|--------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
|       | 1      | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| 0000  | -      | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  |
| 0001  | +1     | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  | -  |
| 0010  | +1     | -  | -  | -  | -  | -  | -  | -  | +1 | -  | -  | -  | -  | -  | -  | -  |
| 0011  | +1     | -  | -  | -  | +1 | -  | -  | -  | +1 | -  | -  | -  | -  | -  | -  | -  |
| 0100  | +1     | -  | -  | -  | +1 | -  | -  | -  | +1 | -  | -  | -  | +1 | -  | -  | -  |
| 0101  | +1     | -  | +1 | -  | +1 | -  | -  | -  | +1 | -  | -  | -  | +1 | -  | -  | -  |
| 0110  | +1     | -  | +1 | -  | +1 | -  | -  | -  | +1 | -  | +1 | -  | +1 | -  | -  | -  |
| 0111  | +1     | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | -  | -  |
| 1000  | +1     | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  |
| 1001  | +1     | +1 | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  | +1 | -  |
| 1010  | +1     | +1 | +1 | -  | +1 | -  | +1 | -  | +1 | +1 | +1 | -  | +1 | -  | +1 | -  |
| 1011  | +1     | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | -  | +1 | -  |
| 1100  | +1     | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  |
| 1101  | +1     | +1 | +1 | +1 | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  | +1 | +1 | +1 | -  |
| 1110  | +1     | +1 | +1 | +1 | +1 | +1 | +1 | -  | +1 | +1 | +1 | +1 | +1 | +1 | +1 | -  |
| 1111  | +1     | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | +1 | -  |

### 26.3.6 UIF 位重映射

TIMx\_CR1 寄存器中的 IUFREMAP 位强制将更新中断标志 UIF 连续复制到定时器计数器寄存器的第 31 位 (TIMxCNT[31])。这允许以原子方式读取计数器的值和一个潜在的由 UIFCPY 标志指示的翻转条件。在特定情况下, 它可以避免由背景任务 (读取计数器) 和中断 (更新中断) 之间的处理引起的竞争条件, 从而简化计算。

UIF 和 UIFCPY 标志之间没有延迟。

### 26.3.7 ADC 触发

定时器可以使用各种内部信号 (例如重置、使能或比较事件) 生成 ADC 触发事件。

**注意:** 接收 tim\_trgo 信号的从属外设 (例如定时器、ADC) 的时钟必须在从主定时器接收事件之前使能, 并且时钟频率 (预分频器) 在从主定时器接收触发器时不能更改。

### 26.3.8 TIM6/TIM7 DMA 请求

TIM6/TIM7 可以生成 DMA 请求, 如下表所示。

表 26.3 DMA 请求

| DMA 请求信号    | DMA 缩写 | DMA 请求 | 使能控制位 |
|-------------|--------|--------|-------|
| tim_upd_dma | TIM_UP | 更新     | UDE   |



### 26.3.9 调试模式

当微控制器进入调试模式（带有 FPU 的 Cortex®-M4 内核停止）时，TIMx 计数器可以继续正常工作或停止。

在调试模式下，每个定时器的行为可以通过 Debug 支持（DBG）模块的专用配置位进行编程。有关更多详细信息，请参阅调试部分。

### 26.4 TIM6/TIM7 低功耗模式

表 26.4 低功耗模式对 TIM6/TIM7 的影响

| 模式 | 描述                        |
|----|---------------------------|
| 睡眠 | 无影响，外设是运行的。中断会导致设备退出睡眠模式。 |
| 停机 | 定时器操作被停止并保留其寄存器内容。无法生成中断。 |
| 待机 | 定时器被断电并且退出待机模式后必须重新初始化。   |

### 26.5 TIM6/TIM7 中断

TIM6/TIM7 可以生成一个单一中断，如下表所示。

表 26.5 中断请求

| 中断缩写         | 中断事件 | 事件标志 | 使能控制位 | 中断清除方法  | 从睡眠模式退出 | 从停机和待机退出 |
|--------------|------|------|-------|---------|---------|----------|
| TIM6<br>TIM7 | 更新   | UIF  | UIE   | UIF 写 0 | 是       | 否        |

## 26.6 TIMx 寄存器描述

在寄存器描述中使用的缩写列表请参考章节 2.1。

外设寄存器可以通过半字（16 位）或字（32 位）进行访问。

### 26.6.1 TIMx 控制寄存器 1 (TIMx\_CR1) (x=6/7)

偏移地址：0x00

复位值：0x0000

| 15       | 14 | 13 | 12         | 11           | 10       | 9 | 8 | 7    | 6        | 5 | 4 | 3   | 2   | 1    | 0   |
|----------|----|----|------------|--------------|----------|---|---|------|----------|---|---|-----|-----|------|-----|
| Reserved |    |    | DITH<br>EN | UIFRE<br>MAP | Reserved |   |   | ARPE | Reserved |   |   | OPM | URS | UDIS | CEN |
|          |    |    | rw         | rw           |          |   |   | rw   |          |   |   | rw  | rw  | rw   | rw  |

Bit 15:13 保留，必须保持复位值。

Bit 12 **DITHEN**：使能抖动（Dithering enable）

0：抖动失能

1：抖动使能

注：DITHEN 位只能在 CEN 位复位时修改

Bit 11 **UIFREMAP**：UIF 状态位重映射（UIF status bit remapping）

0：无重映射。UIF 状态位未复制到 TIMx\_CNT 寄存器位 31

1：重映射使能。UIF 状态位复制到 TIMx\_CNT 寄存器位 31

Bit 10:8 保留，必须保持复位值。

Bit 7 **ARPE**：自动重载预装载使能（Auto-reload preload enable）

0：TIMx\_ARR 寄存器不进行缓冲；

1：TIMx\_ARR 寄存器进行缓冲。

Bit 6:4 保留，必须保持复位值。

Bit 3 **OPM**：单脉冲模式（One pulse mode）

0：计数器在发生更新事件时不会停止计数；

1：计数器在发生下一更新事件时停止计数（将 CEN 位清零）

**Bit 2 URS: 更新请求源 (Update request source)**

此位由软件置 1 和清零，用以选择 UEV 事件源。

0: 使能时，所有以下事件都会生成更新中断。此类事件包括：

- 计数器上溢/下溢
- 将 UG 位置 1
- 通过从模式控制器生成的更新事件

1: 使能时，只有计数器上溢/下溢会生成更新中断。

**Bit 1 UDIS: 更新禁止 (Update disable)**

此位由软件置 1 和清零，用以使能/禁止 UEV 事件生成。

0: 使能 UEV。更新 (UEV) 事件可通过以下事件之一生成：

- 计数器上出/下溢
- 将 UG 位置 1
- 通过从模式控制器生成的更新事件

然后更新影子寄存器的值。

1: 禁止 UEV。不会生成更新事件，各影子寄存器的值 (ARR、PSC) 保持不变。但如果将 UG 位置 1，或者从模式控制器接收到硬件复位，则会重新初始化计数器和预分频器。

**Bit 0 CEN: 计数器使能 (Counter enable)**

0: 禁止计数器；

1: 使能计数器。

单脉冲模式下，当发生更新事件时会自动将 CEN 位清零。

## 26.6.2 TIMx 控制寄存器 2 (TIMx\_CR2) (x=6/7)

偏移地址: 0x04

复位值: 0x0000

|          |    |    |    |    |    |   |   |   |          |    |    |          |   |   |   |
|----------|----|----|----|----|----|---|---|---|----------|----|----|----------|---|---|---|
| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6        | 5  | 4  | 3        | 2 | 1 | 0 |
| Reserved |    |    |    |    |    |   |   |   | MMS[2:0] |    |    | Reserved |   |   |   |
|          |    |    |    |    |    |   |   |   | rw       | rw | rw |          |   |   |   |

Bit 15:7 保留, 必须保持复位值。

Bit 6:4 **MMS[2:0]**: 主模式选择 (Master mode selection)

这些位可选择主模式下将要发送到从定时器以实现同步的信息(tim\_trgo)。这些位的组合如下:

000: **复位** - TIMx\_EGR 寄存器中的 UG 位用作触发输出 (tim\_trgo)。

001: **使能** - 计数器使能信号 CNT\_EN 用作触发输出 (tim\_trgo)。该触发输出可用于同时启动多个定时器, 或者控制在一段时间内使能从定时器。计数器使能信号可由 CEN 控制位产生。

010: **更新** - 选择更新事件作为触发输出 (tim\_trgo)。例如, 主定时器可用作从定时器的预分频器。

注: 从定时器或接收tim\_trgo 的外设时钟必须在从主定时器接收事件之前使能, 在主定时器接收触发期间不能实时更改。

Bit 3:0 保留, 必须保持复位值。

## 26.6.3 TIMx/DMA/中断使能寄存器 (TIMx\_DIER) (x=6/7)

偏移地址: 0x0C

复位值: 0x0000 0000

| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8   | 7        | 6 | 5 | 4 | 3 | 2 | 1 | 0   |
|----------|----|----|----|----|----|---|-----|----------|---|---|---|---|---|---|-----|
| Reserved |    |    |    |    |    |   | UDE | Reserved |   |   |   |   |   |   | UIE |
|          |    |    |    |    |    |   | rw  |          |   |   |   |   |   |   | rw  |

Bit 15:1 保留, 必须保持复位值。

Bit 8 **UDE**: 更新 DMA 请求使能 (Update DMA request enable)

0: 禁止更新 DMA 请求;

1: 使能更新 DMA 请求。

Bit 7:1 保留, 必须保持复位值。

Bit 0 **UIE**: 更新中断使能 (Update interrupt enable)

0: 禁止更新中断;

1: 使能更新中断。

## 26.6.4 TIMx 状态寄存器 (TIMx\_SR) (x=6/7)

偏移地址: 0x10

复位值: 0x0000

| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0     |
|----------|----|----|----|----|----|---|---|---|---|---|---|---|---|---|-------|
| Reserved |    |    |    |    |    |   |   |   |   |   |   |   |   |   | UIF   |
|          |    |    |    |    |    |   |   |   |   |   |   |   |   |   | rc_w0 |

Bit 15:1 保留, 必须保持复位值。

Bit 0 **UIF**: 更新中断标志 (Update interrupt flag)

该位在发生更新事件时通过硬件置 1。但需要通过软件清零。

0: 未发生更新。

1: 更新中断挂起。该位在以下情况下更新寄存器时由硬件置 1:

- TIMx\_CR1 寄存器中的 UDIS=0, 并且重复计数器值上溢或下溢时 (重复计数器=0 时更新)。
- TIMx\_CR1 寄存器中的 URS=0 且 UDIS=0, 并且由软件使用 TIMx\_EGR 寄存器中的 UG 位重新初始化 CNT 时。

## 26.6.5 TIMx 事件产生寄存器 (TIMx\_EGR) (x=6/7)

偏移地址: 0x14

复位值: 0x0000

| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0  |
|----------|----|----|----|----|----|---|---|---|---|---|---|---|---|---|----|
| Reserved |    |    |    |    |    |   |   |   |   |   |   |   |   |   | UG |
|          |    |    |    |    |    |   |   |   |   |   |   |   |   |   | w  |

Bit 15:1 保留, 必须保持复位值。

Bit 0 **UG**: 更新生成 (Update generation)

该位可通过软件置 1, 并由硬件自动清零。

0: 不执行任何操作;

1: 重新初始化计数器并生成一个寄存器更新事件。请注意, 预分频器计数器也将清零 (但预分频比不受影响)。

## 26.6.6 TIMx 计数器 (TIMx\_CNT) (x=6/7)

偏移地址: 0x24

复位值: 0x0000 0000

| 31        | 30       | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|-----------|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| UIF       | Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| CPY       |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| r         |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15        | 14       | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| CNT[15:0] |          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw        | rw       | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31 UIFCPY: 复制 UIF

该位是 TIMx\_SR 寄存器中 UIF 位的一个只读副本。如果 TIMx\_CR1 中的 UIFREMAP 位被复位, 那位 31 被保留且读为 0

Bit 30:16 保留, 必须保持复位值。

Bit 15:0 CNT[15:0]: 计数器的值 (Counter value)

在非抖动模式下 (DITHEN = 0)

该寄存器 (CNT[15:0]) 保存的是计数器的值。

在抖动模式下 (DITHEN = 1)

该寄存器 (CNT[15:0]) 只保存计数器的不抖动部分。抖动部分不可用。

### 26.6.7 TIMx 预分频器 (TIMx\_PSC) (x=6/7)

偏移地址: 0x28

复位值: 0x0000

| 15        | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
|-----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| PSC[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw        | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 15:0 PSC[15:0]: 预分频器的值 (Prescaler value)

计数器时钟频率 ( $f_{tim\_cnt\_ck}$ ) 等于  $f_{tim\_psc\_ck} / (PSC[15:0] + 1)$ 。

PSC 包含每次发生更新事件 (包括计数器通过 TIMx\_EGR 寄存器中的 UG 位清零) 时要装载到活动预分频器寄存器的值。

### 26.6.8 TIMx 自动重载寄存器 (TIMx\_ARR) (x=6/7)

偏移地址: 0x2C

复位: 0x0000 FFFF

| 31        | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19         | 18 | 17 | 16 |
|-----------|----|----|----|----|----|----|----|----|----|----|----|------------|----|----|----|
| Reserved  |    |    |    |    |    |    |    |    |    |    |    | ARR[19:16] |    |    |    |
|           |    |    |    |    |    |    |    |    |    |    |    | rw         | rw | rw | rw |
| 15        | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3          | 2  | 1  | 0  |
| ARR[15:0] |    |    |    |    |    |    |    |    |    |    |    |            |    |    |    |
| rw        | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw         | rw | rw | rw |

Bit 31:20 保留, 必须保持复位值。

Bit 19:0 ARR[19:0]: 自动重载的值 (Auto-reload value)

ARR 为要装载到实际自动重载寄存器的值。

有关 ARR 更新和行为的更多详细信息, 请参阅前序章节: 时基单元。

当自动重载值为空时, 计数器不工作。

在非抖动模式下 (DITHEN = 0)

该寄存器存储的是 ARR[15:0] 中自动重载值。ARR[19:16] 位保留。

在抖动模式下 (DITHEN = 1)

该寄存器存储的是 ARR[19:4] 中的整数部分, 而 ARR[3:0] 位域存储的是抖动的部分

## 27 电压基准缓冲器 (VREFBUF)

### 27.1 简介

该器件嵌入一个电压参考缓冲器，可作为 adc、dac 的电压参考，也可通过 VREF+引脚作为外部元件的电压参考。当封装中的 VREF+引脚与 VDDA 引脚双键合时，电压参考缓冲不可用，必须保持禁用状态（请参阅数据表中的封装引脚描述）。

### 27.2 VREFBUF 功能描述

内部电压参考缓冲器支持三个电压( $\alpha$ )，它们在 VREFBUF\_CSR 寄存器中配置 VRS 位：

- VRS = 00: around 2V
- VRS = 01: around 2.5 V
- VRS = 10: around 2.80 V

根据 ENVR 和 HIZ 位配置，内部基准电压可以配置为四种不同的模式。这些模式如下表所示：

表 VREF 缓冲模式

| ENVR | HIZ | VREF 缓冲器配置                                                                                  |
|------|-----|---------------------------------------------------------------------------------------------|
| 0    | 0   | VREFBUF 缓冲器关闭模式：<br>— VREF+引脚下拉到 vsa                                                        |
| 0    | 1   | 外部电压参考模式(默认值)：<br>— 关闭 VREFBUF 缓冲器<br>— VREF+引脚输入方式                                         |
| 1    | 0   | 内部电压参考模式：<br>— 开启 VREFBUF 缓冲器<br>— VREF+引脚连接到 VREFBUF 缓冲器输出                                 |
| 1    | 0   | 持有模式：<br>— VREF 是使能无输出缓冲，VREF+引脚电压保持与外部电容器<br>— 关闭 VRR 检测，VRR 位保持最后状态—关闭 VRR 检测，VRR 位保持最后状态 |

通过设置 ENVR 位并清除 VREFBUF\_CSR 寄存器中的 HIZ 位使能 VREFBUF 后，用户必须等待 VRR 位设置，即电压参考输出达到预期值。

### 27.3 VREFBUF 校准

VREFBUF 输出电压由 st 进行出厂校准。在复位时，每次 VRS 设置改变时，校准数据自动加载到 TRIM 寄存器。

用户还可以通过直接改变 trim 寄存器位来修整输出电压。在这种情况下，VRS 设置对 TRIM 寄存器没有任何影响，直到设备复位。



## 27.4 VREFBUF 寄存器描述

基地址：0x4001 0030

### 27.4.1 VREFBUF 控制状态寄存器 (VREFBUF\_CSR)

地址偏移：0x00

复位值：0x0000 0000

|          |    |    |    |    |    |    |    |    |    |          |    |     |     |     |      |
|----------|----|----|----|----|----|----|----|----|----|----------|----|-----|-----|-----|------|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21       | 20 | 19  | 18  | 17  | 16   |
| Reserved |    |    |    |    |    |    |    |    |    |          |    |     |     |     |      |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5        | 4  | 3   | 2   | 1   | 0    |
| Reserved |    |    |    |    |    |    |    |    |    | VRS[1:0] |    | VRR | Res | HIZ | ENVR |
|          |    |    |    |    |    |    |    |    |    | rw       | rw | r   |     | rw  | rw   |

Bits 31:3 保留，必须保持为复位值。

Bits 5:4 **VRS[1:0]**：电压基准标度

这些位选择由电压参考缓冲器产生的值

00：电压基准设定为 2V

01：电压基准设定为 2.5 V

10：电压基准设定为 2.80 V

11：保留

注意：只有当 VREFBUF 被禁用(ENVR=0)时，软件才能编程该位域。

Bit 3 **VRR**：电压基准缓冲器就绪

0：电压基准缓冲器输出未准备好。

1：参考电压缓冲输出达到要求的电平。

Bits 2 保留，必须保持为复位值。

Bit 1 **HIZ**：高阻抗模式

这个位控制模拟开关连接或不连接 VREF+引脚。

0：VREF+引脚内部连接到电压参考缓冲输出。

1：VREF+引脚高阻抗。

Bit 0 **ENVR**：使能电压基准缓冲模式

这个位用来使能电压参考缓冲模式。

0：禁用内部参考电压模式（外部参考电压模式）。

1：内部电压参考模式（参考缓冲使能或保持模式）使能。

### 27.4.2 VREFBUF 控制寄存器 (VREFBUF\_CCR)

地址偏移：0x00

复位值：0x0000 0000

|          |    |    |    |    |    |    |    |    |    |    |    |           |    |    |    |
|----------|----|----|----|----|----|----|----|----|----|----|----|-----------|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19        | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |           |    |    |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3         | 2  | 1  | 0  |
| Reserved |    |    |    |    |    |    |    |    |    |    |    | TRIM[3:0] |    |    |    |
|          |    |    |    |    |    |    |    |    |    |    |    | rw        | rw | rw | rw |

Bits 31:4 保留，必须保持为复位值。

Bits 3:0 **TRIM[3:0]**：微调代码

TRIM 代码是一个 6 位无符号数据（最小 00000，最大 11111），根据下面描述的机

制设置和更新。

重置：

TRIM[3:0]在生产测试时自动初始化，VRS = 0 微调值存储在 Flash 内存中。

工具改变：

TRIM[3:0]在生产测试时自动初始化，并使用存储在 Flash 存储器中的微调值（对应于 VRS 设置）。

用 TRIM 写[3:0]：

用户可以用任意值修改 TRIM[3:0]。这将永久禁用 VRS 对微调值的控制（直到设备复位）。

注意：如果用户应用程序执行微调，微调代码必须从 000000 到 111111 依次递增。

## 28 实时时钟 (RTC)

### 28.1 RTC 简介

实时时钟是一个独立的定时器。RTC 模块拥有一组连续计数的计数器，在相应软件配置下，可提供时钟日历的功能。修改计数器的值可以重新设置系统当前的时间和日期。

RTC 模块和时钟配置系统 (RCC\_BDCR 寄存器) 处于后备区域，即在系统复位或从待机模式唤醒后，RTC 的设置和时间维持不变。

系统复位后，对后备寄存器和 RTC 的访问被禁止，这是为了防止对后备区域 (BKP) 的意外写操作。执行以下操作将使能对后备寄存器和 RTC 的访问：

- 设置寄存器 RCC\_APB1ENR 的 PWREN 和 BKPEN 位，使能电源和后备接口时钟
- 设置寄存器 PWR\_CR 的 DBP 位，使能对后备寄存器和 RTC 的访问。

### 28.2 主要特性

- 可编程的预分频系数：分频系数最高为  $2^{20}$ 。
- 32 位的可编程计数器，可用于较长时间段的测量。
- 2 个分离的时钟：用于 APB1 接口的 PCLK1 和 RTC 时钟 (RTC 时钟的频率必须小于 PCLK1 时钟频率的四分之一以上)。
- 可以选择以下两种 RTC 的时钟源：
  - HSE 时钟除以 128；
  - LSI 振荡器时钟
- 2 个独立的复位类型：
  - APB1 接口由系统复位；
  - RTC 核心 (预分频器、闹钟、计数器和分频器) 只能由后备域复位
- 3 个专门的可屏蔽中断：
  - 闹钟中断，用来产生一个软件可编程的闹钟中断。
  - 秒中断，用来产生一个可编程的周期性中断信号 (最长可达 1 秒)。
  - 溢出中断，指示内部可编程计数器溢出并回转为 0 的状态。

### 28.3 功能描述

#### 28.3.1 概述

RTC 由两个主要部分组成 (参见下图)。第一部分 (APB1 接口) 用来和 APB1 总线相连。此单元还包含一组 16 位寄存器，可通过 APB1 总线对其进行读写操作。APB1 接口由 APB1 总线时钟驱动，用来与 APB1 总线接口。

另一部分 (RTC 核心) 由一组可编程计数器组成，分成两个主要模块。第一个模块是 RTC 的预分频模块，它可编程产生最长为 1 秒的 RTC 时间基准 TR\_CLK。RTC 的预分频模块包含了一个 20 位的可编程分频器 (RTC 预分频器)。如果在 RTC\_CR 寄存器中设置了相应的允许位，则在每个 TR\_CLK 周期中 RTC 产生一个中断 (秒中断)。第二个模块是一个 32 位的可编程计数器，可被初始化为当前的系统时间。系统时间按 TR\_CLK 周期累加并与存储在 RTC\_ALR 寄存器中的可编程时间相比较，如果 RTC\_CR 控制寄存器中设置了相应允许位，比较匹配时将产生一个闹钟中断。

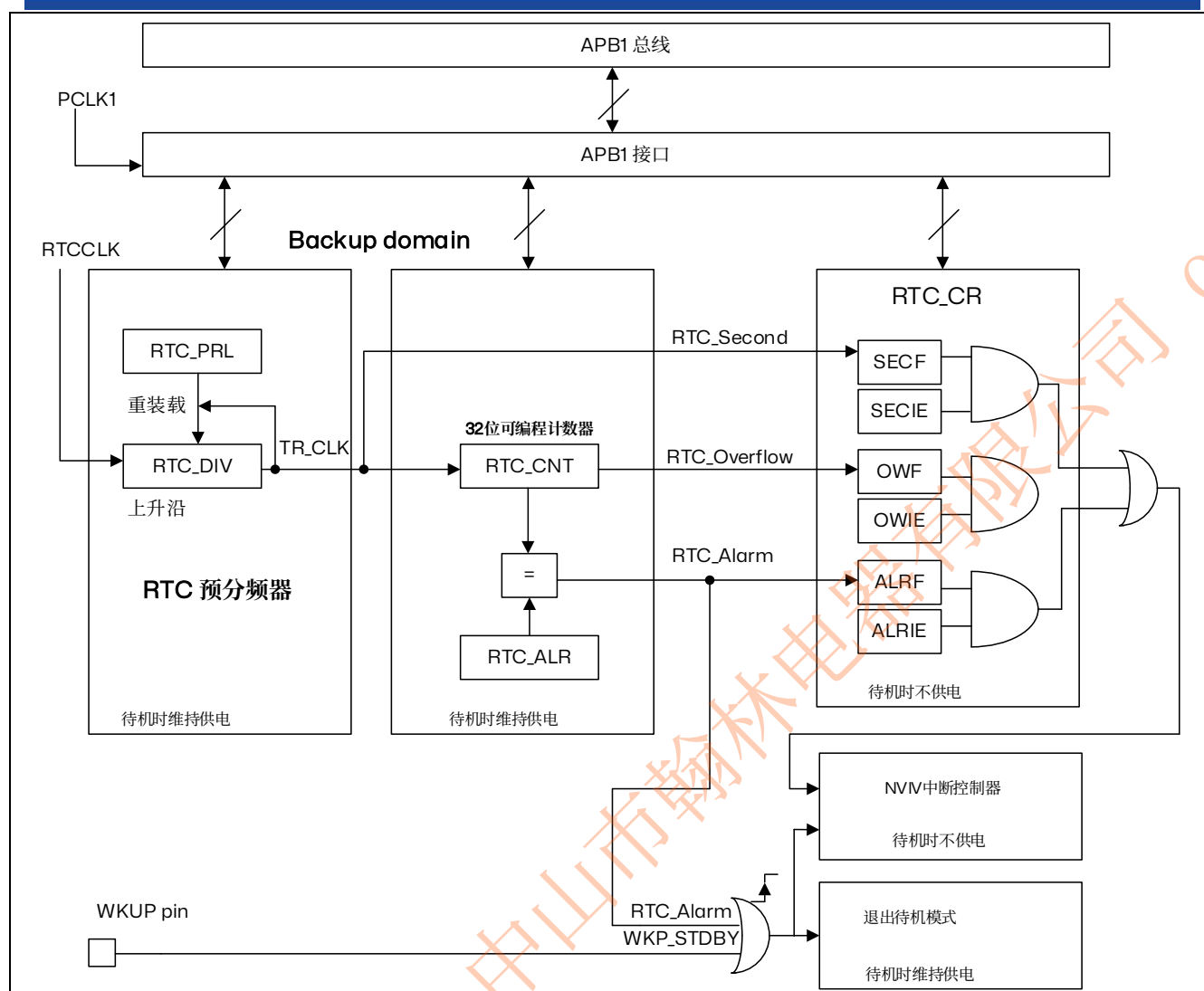


图 28.1 简化的 RTC 框图

### 28.3.2 复位过程

除了 RTC\_PRL、RTC\_ALR、RTC\_CNT 和 RTC\_DIV 寄存器外，所有的系统寄存器都由系统复位或电源复位进行异步复位。

RTC\_PRL、RTC\_ALR、RTC\_CNT 和 RTC\_DIV 寄存器仅能通过备份域复位信号复位，读 RTC 寄存

RTC 核完全独立于 RTC APB1 接口。

软件通过 APB1 接口访问 RTC 的预分频值、计数器值和闹钟值。但是，相关的可读寄存器只在与 RTC APB1 时钟进行重新同步的 RTC 时钟的上升沿被更新。RTC 标志也是如此的。

这意味着，如果 APB1 接口曾经被关闭，而读操作又是在刚刚重新开启 APB1 之后，则在第一次的内部寄存器更新之前，从 APB1 上读出的 RTC 寄存器数值可能被破坏了（通常读到 0）。下述几种情况下能够发生这种情形：

- 发生系统复位或电源复位
- 系统刚从待机模式唤醒。
- 系统刚从停机模式唤醒。

所有以上情况中，APB1 接口被禁止时（复位、无时钟或断电）RTC 核仍保持运行状态。

因此，若在读取 RTC 寄存器时，RTC 的 APB1 接口曾经处于禁止状态，则软件首先必须等待 RTC\_CRL 寄存器中的 RSF 位（寄存器同步标志）被硬件置‘1’。

注：RTC 的 APB1 接口不受 WFI 和 WFE 等低功耗模式的影响。

### 28.3.3 配置 RTC 寄存器

必须设置 RTC\_CRL 寄存器中的 CNF 位，使 RTC 进入配置模式后，才能写入 RTC\_PRL、RTC\_CNT、RTC\_ALR 寄存器。

另外，对 RTC 任何寄存器的写操作，都必须在前一次写操作结束后进行。可以通过查询 RTC\_CR 寄存器中的 RTOFF 状态位，判断 RTC 寄存器是否处于更新中。仅当 RTOFF 状态位是‘1’时，才可以写入 RTC 寄存器。

配置过程：

1. 查询 RTOFF 位，直到 RTOFF 的值变为‘1’
2. 置 CNF 值为 1，进入配置模式
3. 对一个或多个 RTC 寄存器进行写操作
4. 清除 CNF 标志位，退出配置模式
5. 查询 RTOFF，直至 RTOFF 位变为‘1’以确认写操作已经完成。

仅当 CNF 标志位被清除时，写操作才能进行，这个过程至少需要 3 个 RTCCLK 周期。

### 28.3.4 RTC 标志的设置

在每一个 RTC 核心的时钟周期中，更改 RTC 计数器之前设置 RTC 秒标志（SECF）。

在计数器到达 0x0000 之前的最后一个 RTC 时钟周期中，设置 RTC 溢出标志（OWF）。

在计数器的值到达闹钟寄存器的值加 1（RTC\_ALR+1）之前的 RTC 时钟周期中，设置 RTC\_Alarm 和 RTC 闹钟标志（ALRF）。对 RTC 闹钟的写操作必须使用下述过程之一与 RTC 秒标志同步：

- 使用 RTC 闹钟中断，并在中断处理程序中修改 RTC 闹钟和/或 RTC 计数器。
- 等待 RTC 控制寄存器中的 SECF 位被设置，再更改 RTC 闹钟和/或 RTC 计数器。

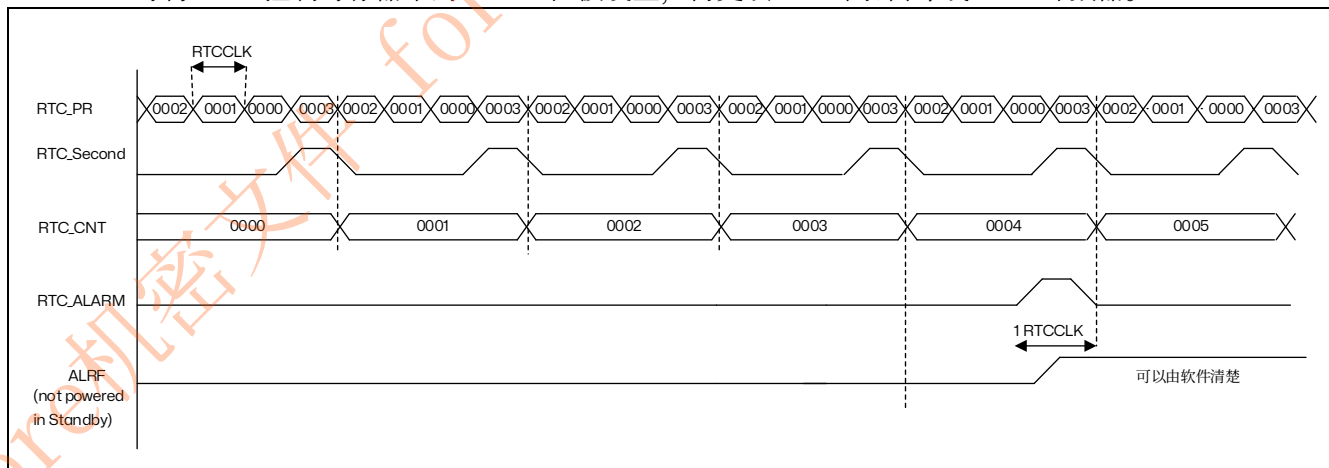


图 28.2 RTC 秒和闹钟波形图示例，PR=0003，ALARM=00004

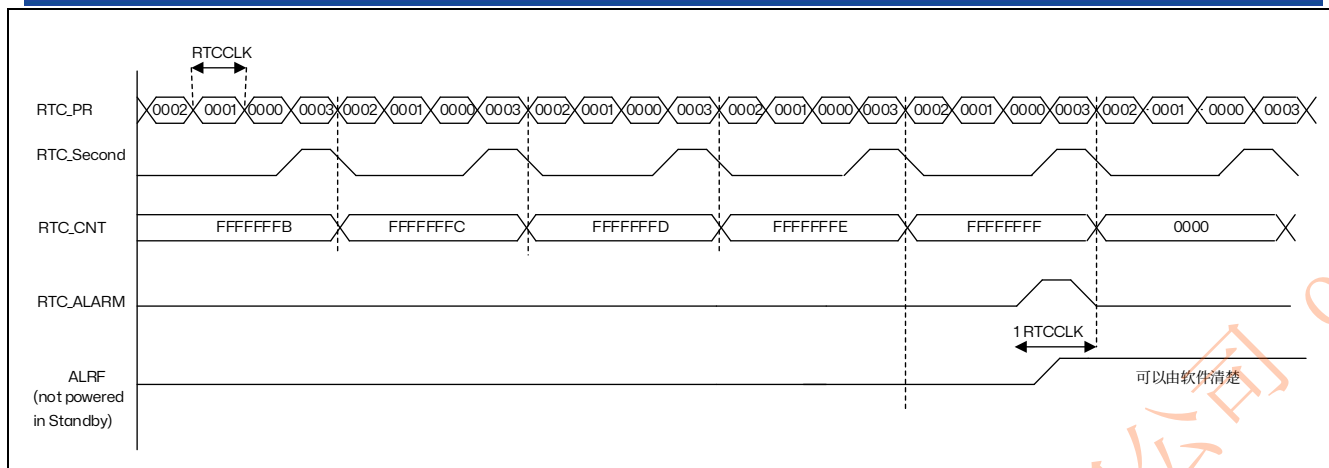


图 28.3 RTC 溢出波形图示例, PR=0003

## 28.4 RTC 寄存器描述

可以用半字（16 位）或字（32 位）的方式操作这些外设寄存器。

### 28.4.1 RTC 控制寄存器高位 (RTC\_CRH)

地址偏移量: 0x00

复位值: 0x0000

| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2    | 1     | 0     |
|----------|----|----|----|----|----|---|---|---|---|---|---|---|------|-------|-------|
| Reserved |    |    |    |    |    |   |   |   |   |   |   |   | OWIE | ALRIE | SECIE |
|          |    |    |    |    |    |   |   |   |   |   |   |   | rw   | rw    | rw    |

Bits 15:3 保留, 必须保持为复位值。

Bit 2 **OWIE**: 允许溢出中断位 (Overflow interrupt enable)

- 0: 屏蔽 (不允许) 溢出中断
- 1: 允许溢出中断

Bit 1 **ALRIE**: 允许闹钟中断 (Alarm interrupt enable)

- 0: 屏蔽 (不允许) 闹钟中断
- 1: 允许闹钟中断

Bit 0 **SECIE**: 允许秒中断 (Second interrupt enable)

- 0: 屏蔽 (不允许) 秒中断
- 1: 允许秒中断

这些位用来屏蔽中断请求。注意: 系统复位后所有的中断被屏蔽, 因此可通过写 RTC 寄存器来确保在初始化后没有挂起的中断请求。当外设正在完成前一次写操作时 (标志位 RTOFF=0), 不能对 RTC\_CRH 寄存器进行写操作。

RTC 功能由这个控制寄存器控制。一些位的写操作必须经过一个特殊的配置过程来完成。

## 28.4.2 RTC 控制寄存器低位 (RTC\_CRL)

偏移地址: 0x04

复位值: 0x0020

| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5     | 4   | 3     | 2     | 1     | 0     |
|----------|----|----|----|----|----|---|---|---|---|-------|-----|-------|-------|-------|-------|
| Reserved |    |    |    |    |    |   |   |   |   | RTOFF | CNF | RSF   | OWF   | ALRF  | SECF  |
|          |    |    |    |    |    |   |   |   |   | r     | rw  | rc_w0 | rc_w0 | rc_w0 | rc_w0 |

Bits 15:6 保留, 必须保持为复位值。

### Bit 5 RTOFF: RTC 操作关闭 (RTC operation OFF)

RTC 模块利用这位来指示对其寄存器进行的最后一次操作的状态, 指示操作是否完成。

若此位为 '0', 则表示无法对任何的 RTC 寄存器进行写操作。此位为只读位。

0: 上一次对 RTC 寄存器的写操作仍在进行;

1: 上一次对 RTC 寄存器的写操作已经完成。

### Bit 4 CNF: 配置标志 (Configuration flag)

此位必须由软件置 '1' 以进入配置模式, 从而允许向 RTC\_CNT、RTC\_ALR 或

RTC\_PRL 寄存器写入数据。只有当此位在设置'1'并重新由软件清 '0' 后, 才会执行写操作。

0: 退出配置模式 (开始更新 RTC 寄存器);

1: 进入配置模式。

### Bit 3 RSF: 寄存器同步标志 (Registers synchronized flag)

每当 RTC\_CNT 寄存器和 RTC\_DIV 寄存器由软件更新或清 '0' 时, 此位由硬件置

'1'。在 APB1 复位后, 或 APB1 时钟停止后, 此位必须由软件清 '0'。要进行任何的读操作之前, 用户程序必须等待这位被硬件置 '1', 以确保 RTC\_CNT、RTC\_ALR 或 RTC\_PRL 已经被同步。

0: 寄存器尚未被同步;

1: 寄存器已经被同步。

### Bit 2 OWF: 溢出标志 (Overflow flag)

当 32 位可编程计数器溢出时, 此位由硬件置'1'。如果 RTC\_CRH 寄存器中 OWIE=1, 则产生中断。此位只能由软件清 '0'。对此位写 '1' 是无效的。

0: 无溢出;

1: 32 位可编程计数器溢出。

### Bit 1 ALRF: 闹钟标志 (Alarm flag)

当 32 位可编程计数器达到 RTC\_ALR 寄存器所设置的预定值, 此位由硬件置 '1'。如果 RTC\_CRH 寄存器中 ALRIE=1, 则产生中断。此位只能由软件清 '0'。对此位写 '1' 是无效的。

0: 无闹钟;

1: 有闹钟。

### Bit 0 SECF: 秒标志 (Second flag)

当 32 位可编程预分频器溢出时, 此位由硬件置 '1' 同时 RTC 计数器加 1。因此, 此标志为分辨率可编程的 RTC 计数器提供一个周期性的信号 (通常为 1 秒)。如果

RTC\_CRH 寄存器中 SECIE=1, 则产生中断。此位只能由软件清除。对此位写 '1' 是无效的。

0: 秒标志条件不成立;

1: 秒标志条件成立。



RTC 的功能由这个控制寄存器控制。当前一个写操作还未完成时，不能写 RTC\_CR 寄存器。

注：1. 任何标志位都将保持挂起状态，直到适当的RTC\_CR 请求位被软件复位，表示所请求的中断已经被接受。

2. 在复位时禁止所有中断，无挂起的中断请求，可以对 RTC 寄存器进行写操作。

3. 当 APB1 时钟不运行时，OWF、ALRF、SECF 和 RSF 位不被更新。

4. OWF、ALRF、SECF 和 RSF 位只能由硬件置位，由软件来清零。

5 若 ALRF=1 且 ALRIE=1，则允许产生 RTC 全局中断。如果在 EXTI 控制器中允许产生 EXTI 线 17 中断，则允许产生 RTC 全局中断和 RTC 闹钟中断。

6 若 ALRF=1，如果在 EXTI 控制器中设置了 EXTI 线 17 的中断模式，则允许产生 RTC 闹钟中断；如果在 EXTI 控制器中设置了 EXTI 线 17 的事件模式，则这条线上会产生一个脉冲（不会产生 RTC 闹钟中断）。

### 28.4.3 RTC 预分频装载寄存器 (RTC\_PRLH/RTC\_PRL)

预分频装载寄存器用来保存 RTC 预分频器的周期计数值。它们受 RTC\_CR 寄存器的 RTOFF 位保护，仅当 RTOFF 值为 '1' 时允许进行写操作。

#### RTC 预分频装载寄存器高位 (RTC\_PRLH)

偏移地址：0x08

只写

复位值：0x0000

|          |    |    |    |    |    |   |   |   |   |   |   |            |   |   |   |
|----------|----|----|----|----|----|---|---|---|---|---|---|------------|---|---|---|
| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3          | 2 | 1 | 0 |
| Reserved |    |    |    |    |    |   |   |   |   |   |   | PRL[19:16] |   |   |   |
|          |    |    |    |    |    |   |   |   |   |   |   | w          | w | w | w |

Bits 15:4 保留，必须保持为复位值。

Bits 3:0 **PRL[19:16]**: RTC 预分频装载值高位 (RTC prescaler reload value high)

根据以下公式，这些位用来定义计数器的时钟频率：

$$f_{TR\_CLK} = f_{RTCCLK} / (PRL[19:0] + 1)$$

注：不推荐使用 0 值，否则无法正确的产生 RTC 中断和标志位。

#### RTC 预分频装载寄存器低位 (RTC\_PRL)

偏移地址：0x0C

只写

复位值：0x0000

|           |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31        | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15        | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| PRL[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| w         | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  |

Bits 31:16 保留，必须保持为复位值。

Bits 15:0 **PRL[15:0]**: RTC 预分频装载值低位

根据以下公式，这些位用来定义计数器的时钟频率：

$$f_{TR\_CLK} = f_{RTCCLK} / (PRL[19:0] + 1)$$

注：如果输入时钟频率是 32.768kHz ( $f_{RTCCLK}$ )，这个寄存器中写入 7FFFh 可获得周期为 1 秒钟的信号。

#### 28.4.4 RTC 预分频器余数寄存器 (RTC\_DIVH / RTC\_DIVL)

在 TR\_CLK 的每个周期里，RTC 预分频器中计数器的值都会被重新设置为 RTC\_PRL 寄存器的值。用户可通过读取 RTC\_DIV 寄存器，以获得预分频计数器的当前值，而不停止分频计数器的工作，从而获得精确的时间测量。此寄存器是只读寄存器，其值在 RTC\_PRL 或 RTC\_CNT 寄存器中的值发生改变后，由硬件重新装载。

##### RTC 预分频器余数寄存器高位 (RTC\_DIVH)

偏移地址：0x10

复位值：0x0000

| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3              | 2 | 1 | 0 |
|----------|----|----|----|----|----|---|---|---|---|---|---|----------------|---|---|---|
| Reserved |    |    |    |    |    |   |   |   |   |   |   | RTC_DIV[19:16] |   |   |   |
|          |    |    |    |    |    |   |   |   |   |   |   | r              | r | r | r |

Bits 15:4 保留，必须保持为复位值。

Bits 3:0 **RTC\_DIV[19:16]**: RTC 时钟分频器余数高位 (RTC clock divider high)

##### RTC 预分频器余数寄存器低位 (RTC\_DIVL)

偏移地址：0x14

复位值：0x8000

| 15            | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---------------|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| RTC_DIV[15:0] |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| r             | r  | r  | r  | r  | r  | r | r | r | r | r | r | r | r | r | r |

Bits 15:0 **RTC\_DIV[15:0]**: RTC 时钟器余数低位 (RTC clock divider low)

#### 28.4.5 RTC 计数器寄存器 (RTC\_CNTH/RTC\_CNTL)

RTC 核有一个 32 位可编程的计数器，可通过两个 16 位的寄存器访问。计数器以预分频器产生的 TR\_CLK 时间基准为参考进行计数。RTC\_CNT 寄存器用来存放计数器的计数值。他们受 RTC\_CR 的位 RTOFF 写保护，仅当 RTOFF 值为 '1' 时，允许写操作。在高或低寄存器 (RTC\_CNTH 或 RTC\_CNTL) 上的写操作，能够直接装载到相应的可编程计数器，并且重新装载 RTC 预分频器。当进行读操作时，直接返回计数器内的计数值 (系统时间)。

##### RTC 计数器寄存器高位 (RTC\_CNTH)

偏移地址：0x18

复位值：0x0000

| 15             | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|----------------|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| RTC_CNT[31:16] |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| r              | r  | r  | r  | r  | r  | r | r | r | r | r | r | r | r | r | r |

Bits 15:0 **RTC\_CNT[31:16]**: RTC 计数器高位 (RTC counter high)

可通过读 RTC\_CNTH 寄存器来获得 RTC 计数器当前值的高位部分。要对此寄存器进行写操作前，必须先进入配置模式。

## RTC 计数器寄存器低位 (RTC\_CNTL)

偏移地址: 0x1C

复位值: 0x0000

| 15            | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
|---------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| RTC_CNT[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw            | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 15:0 **RTC\_CNT[15:0]**: RTC 计数器低位。

可通过读 RTC\_CNTL 寄存器来获得 RTC 计数器当前值的低位部分。要对此寄存器进行写操作，必须先进入配置模式。

## 28.4.6 RTC 闹钟寄存器 (RTC\_ALRH/RTC\_ALRL)

当可编程计数器的值与 RTC\_ALR 中的 32 位值相等时，即触发一个闹钟事件，并且产生 RTC 闹钟中断。此寄存器受 RTC\_CR 寄存器里的 RTOFF 位写保护，仅当 RTOFF 值为 ‘1’ 时，允许写操作。

### RTC 闹钟寄存器高位 (RTC\_ALRH)

偏移地址: 0x20

只写

复位值: 0x0000

| 15             | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|----------------|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| RTC_ALR[31:16] |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| w              | w  | w  | w  | w  | w  | w | w | w | w | w | w | w | w | w | w |

Bits 15:0 **RTC\_ALR[31:16]**: RTC 闹钟值高位 (RTC alarm high)

此寄存器用来保存由软件写入的闹钟时间的高位部分。要对此寄存器进行写操作，必须先进入配置模式。

### RTC 闹钟寄存器低位 (RTC\_ALRL)

偏移地址: 0x24

只写

复位值: 0x0000

| 15            | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---------------|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| RTC_ALR[15:0] |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| w             | w  | w  | w  | w  | w  | w | w | w | w | w | w | w | w | w | w |

Bits 15:0 **RTC\_ALR[15:0]**: RTC 闹钟值低位 (RTC alarm low)

此寄存器用来保存由软件写入的闹钟时间的低位部分。要对此寄存器进行写操作，必须先进入配置模式。

## 29 独立看门狗 (IWDG)

### 29.1 简介

RX32G410 内置两个看门狗，提供了更高的安全性、时间的精确性和使用的灵活性。两个看门狗设备（独立看门狗和窗口看门狗）可用来检测和解决由软件错误引起的故障；当计数器达到给定的超时值时，触发一个中断（仅适用于窗口型看门狗）或产生系统复位。

独立看门狗 (IWDG) 由专用的低速时钟 (LSI) 驱动，即使主时钟发生故障它也仍然有效。窗口看门狗由从 APB1 时钟分频后得到的时钟驱动，通过可配置的时间窗口来检测应用程序非正常的过迟或过早的操作。

IWDG 最适合应用于那些需要看门狗作为一个在主程序之外，能够完全独立工作，并且对时间精度要求较低的场合。WWDG 最适合那些要求看门狗在精确计时窗口起作用的应用程序。

### 29.2 IWDG 主要性能

- 自由运行的递减计数器
- 时钟由独立的 RC 振荡器提供（可在停止和待机模式下工作）
- 看门狗被激活后，则在计数器计数至 0x000 时产生复位

### 29.3 IWDG 功能描述

下图为独立看门狗模块的功能框图。

在键寄存器 (IWDG\_KR) 中写入 0xCCCC，开始使能独立看门狗；此时计数器开始从其复位值 0xFFFF 递减计数。当计数器计数到末尾 0x000 时，会产生一个复位信号 (IWDG\_RESET)。

无论何时，只要在键寄存器 IWDG\_KR 中写入 0xAAAA，IWDG\_RLR 中的值就会被重新加载到计数器，从而避免产生看门狗复位。

#### 29.3.1 硬件看门狗

如果用户在选择字节中使能了“硬件看门狗”功能，在系统上电复位后，看门狗会自动开始运行；如果在计数器计数结束前，若软件没有向键寄存器写入相应的值，则系统会产生复位。

#### 29.3.2 寄存器访问保护

IWDG\_PR 和 IWDG\_RLR 寄存器具有写保护功能。要修改这两个寄存器的值，必须先向 IWDG\_KR 寄存器中写入 0x5555。以不同的值写入这个寄存器将会打乱操作顺序，寄存器将重新被保护。重装载操作（即写入 0xAAAA）也会启动写保护功能。

状态寄存器指示预分频值和递减计数器是否正在被更新。

#### 29.3.3 调试模式

当微控制器进入调试模式时 (Cortex-M4 核心停止)，根据调试模块中的 DBG\_IWDG\_STOP 配置位的状态，IWDG 的计数器能够继续工作或停止。

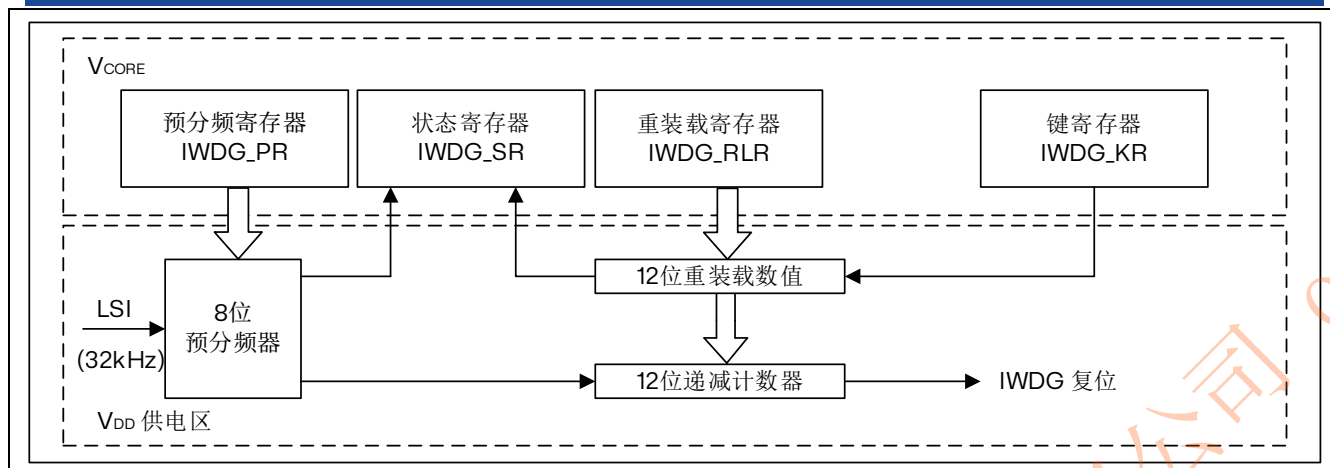


图 29.1 独立看门狗框图

注：看门狗功能处于 VDD 供电区，即在停机和待机模式时仍能正常工作。

表 29.1 看门狗超时时间（32kHz 的输入时钟（LSI））<sup>(1)</sup>

| 预分频系数 | PR[2:0]位 | 最短时间 (ms)<br>RL[11:0] = 0x000 | 最短时间 (ms)<br>RL[11:0] = 0xFFFF |
|-------|----------|-------------------------------|--------------------------------|
| /4    | 0        | 0.125                         | 512                            |
| /8    | 1        | 0.25                          | 1024                           |
| /16   | 2        | 0.5                           | 2048                           |
| /32   | 3        | 1                             | 4096                           |
| /64   | 4        | 2                             | 8192                           |
| /128  | 5        | 4                             | 16384                          |
| /256  | (6 或 7)  | 8                             | 32768                          |

注：这些时间是按照 32kHz 时钟给出。实际上，MCU 内部的 RC 频率会在 30kHz 到 60kHz 之间变化。此外，即使 RC 振荡器的频率是精确的，确切的时序仍然依赖于 APB 接口时钟与 RC 振荡器时钟之间的相位差，因此总会有一个完整的 RC 周期是不确定的。

## 29.4 IWDG 寄存器描述

访问：无等待，支持字，半字访问

### 29.4.1 键寄存器（IWDG\_KR）

地址偏移：0x00

复位值：0x0000 0000（在待机模式复位）

|           |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 31        | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| 15        | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| KEY[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| w         | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  | w  |

Bit 31:16 保留，必须保持为复位值。

Bit 15:0 KEY[15:0]：键值（只写寄存器，读出值为 0x0000）（Key value）

软件必须以一定的间隔写入 0xAAAA，否则，当计数器为 0 时，看门狗会产生复位。

写入 0x5555 表示允许访问 IWDG\_PR 和 IWDG\_RLR 寄存器。

写入 0xCCCC，启动看门狗工作（若选择了硬件看门狗则不受此命令字限制）。

## 29.4.2 预分频寄存器（IWDG\_PR）

地址偏移：0x04

复位值：0x0000 0000

|          |    |    |    |    |    |    |    |    |    |    |    |    |         |    |    |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|---------|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18      | 17 | 16 |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |         |    |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2       | 1  | 0  |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    | PR[2:0] |    |    |
|          |    |    |    |    |    |    |    |    |    |    |    |    | rw      | rw | rw |

Bit 31:3 保留，必须保持为复位值。

Bit 2:0 PR[2:0]：预分频系数（Prescaler divider）

这些位具有写保护设置。通过设置这些位来选择计数器时钟的预分频系数。要改变预分频系数，IWDG\_SR 寄存器的 PVU 位必须为 0。

000：预分频系数=4

100：预分频系数=64

001：预分频系数=8

101：预分频系数=128

010：预分频系数=16

110：预分频系数=256

011：预分频系数=32

111：预分频系数=256

注：对此寄存器进行读操作，将从 VDD 电压域返回预分频值。如果写操作正在进行，则读回的值可能是无效的。因此，只有当 IWDG\_SR 寄存器的 PVU 位为 0 时，读出的值才有效。

## 29.4.3 重装载寄存器（IWDG\_RLR）

地址偏移：0x08

复位值：0x0000 0FFF（待机模式时复位）

|          |    |    |    |          |    |    |    |    |    |    |    |    |    |    |    |
|----------|----|----|----|----------|----|----|----|----|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27       | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |          |    |    |    |    |    |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11       | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    | RL[11:0] |    |    |    |    |    |    |    |    |    |    |    |
|          |    |    |    | rw       | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:12 保留，必须保持为复位值。

Bit 11:0 RL[11:0]：看门狗计数器重装载值（Watchdog counter reload value）

这些位具有写保护功能。用于定义看门狗计数器的重装载值，每当向 IWDG\_KR 寄存器写入 0xAAAA 时，重装载值会被传送到计数器中。随后计数器从这个值开始递减计数。看门狗超时周期可通过此重装载值和时钟预分频值来计算。

只有当 IWDG\_SR 寄存器中的 RVU 位为 0 时，才能对此寄存器进行修改。

注：对此寄存器进行读操作，将从 VDD 电压域返回预分频值。如果写操作正在进行，则读回的值可能是无效的。因此，只有当 IWDG\_SR 寄存器的 RVU 位为 0 时，读出的值才有效。



#### 29.4.4 状态寄存器 (IWDG\_SR)

地址偏移: 0x0C

复位值: 0x0000 0000 (待机模式时不复位)

|          |    |    |    |    |    |    |    |    |    |    |    |    |    |     |     |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|-----|-----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17  | 16  |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    |     |     |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1   | 0   |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    | RVU | PVU |
|          |    |    |    |    |    |    |    |    |    |    |    |    |    | r   | r   |

Bit 31:2 保留, 必须保持为复位值。

Bit 1 RVU: 看门狗计数器重装载值更新 (Watchdog counter reload value update)

此位由硬件置“1”用来指示重装载值的更新正在进行中。当在 VDD 域中的重装载更新结束后, 此位由硬件清“0”(最多需 5 个 32kHz 的 RC 周期)。重装载值只有在 RVU 位被清“0”后才可更新。

Bit 0 PVU: 看门狗预分频值更新 (Watchdog prescaler value update)

此位由硬件置“1”用来指示预分频值的更新正在进行中。当在 VDD 域中的预分频值更新结束后, 此位由硬件清“0”(最多需 5 个 32kHz 的 RC 周期)。预分频值只有在 PVU 位被清“0”后才可更新。

注: 如果在应用程序中使用了多个重装载值或预分频值, 则必须在 RVU 位被清除后才能重新改变预装载值, 在 PVU 位被清除后才能重新改变预分频值。然而, 在预分频和/或重装载值更新后, 不必等待 RVU 或 PVU 复位, 可继续执行下面的代码。(即是在低功耗模式下, 此写操作仍会被继续执行完成。)



## 30 窗口看门狗 (WWDG)

### 30.1 WWDG 简介

窗口看门狗通常被用来监测，由外部干扰或不可预见的逻辑条件造成的应用程序背离正常的运行序列而产生的软件故障。除非递减计数器的值在 T6 位变成 0 前被刷新，看门狗电路在达到预置的时间周期时，会产生一个 MCU 复位。在递减计数器达到窗口寄存器数值之前，如果 7 位的递减计数器数值（在控制寄存器中）被刷新，那么也将产生一个 MCU 复位。这表明递减计数器需要在一个有限的时间窗口中被刷新。

### 30.2 WWDG 主要特性

- 可编程的自由运行递减计数器
- 条件复位
  - 当递减计数器的值小于 0x40，（若看门狗被启动）则产生复位。
  - 当递减计数器在窗口外被重新装载，（若看门狗被启动）则产生复位。
- 如果启动了看门狗并且允许中断，当递减计数器等于 0x40 时产生早期唤醒中断（EWI），它可以被用于重装计数器以避免 WWDG 复位。

### 30.3 WWDG 功能描述

如果看门狗被启动（WWDG\_CR 寄存器中的 WDGA 位被置‘1’），并且当 7 位（T[6:0]）递减计数器从 0x40 翻转到 0x3F（T6 位清零）时，则产生一个复位。如果软件在计数器值大于窗口寄存器中的数值时重新装载计数器，将产生一个复位。

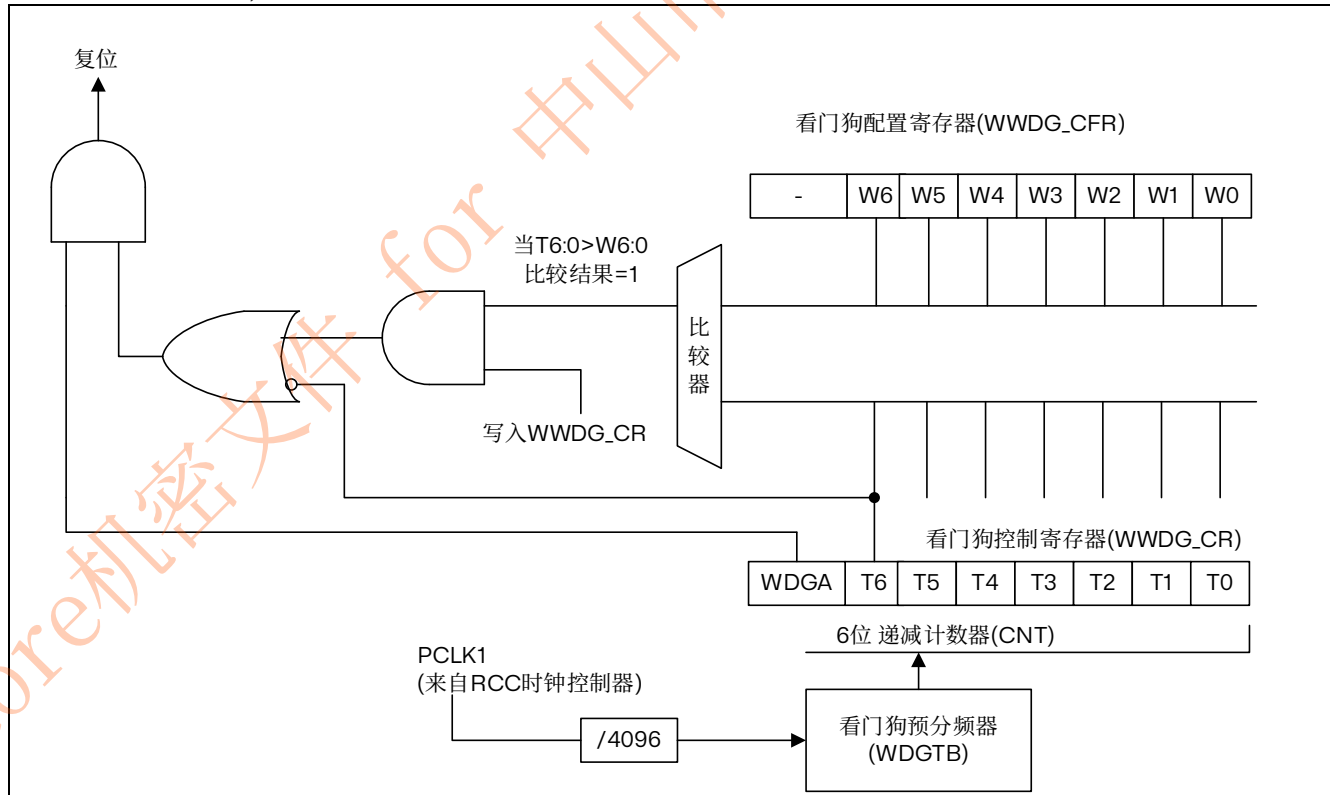


图 30.1 窗口看门狗框图

应用程序在正常运行过程中必须定期地写入 WWDG\_CR 寄存器以防止 MCU 发生复位。只有当计数器值小于窗口寄存器的值时，才能进行写操作。储存在 WWDG\_CR 寄存器中的数值必须在 0xFF 和 0xC0 之间：

- 启动看门狗在系统复位后，看门狗总是处于关闭状态，设置 WWDG\_CR 寄存器的 WDGA 位能够开启看门狗，随后它不能再被关闭，除非发生复位。

- 控制递减计数器

递减计数器处于自由运行状态，即使看门狗被禁止，递减计数器仍继续递减计数。当看门狗被使能时，T6 位必须被设置，以防止立即产生一个复位。

T[5:0]位包含了看门狗产生复位之前的计时数目；复位前的延时时间在一个最小值和一个最大值之间变化，这是因为写入 WWDG\_CR 寄存器时，预分频值是未知的。

配置寄存器（WWDG\_CFR）中包含窗口的上限值：要避免产生复位，递减计数器必须在其值小于窗口寄存器的数值并且大于 0x3F 时被重新装载，0 描述了窗口寄存器的工作过程。

另一个重装载计数器的方法是利用早期唤醒中断（EWI）。设置 WWDG\_CFR 寄存器中的 WEI 位开启该中断。当递减计数器到达 0x40 时，则产生此中断，相应的中断服务程序（ISR）可以用来加载计数器以防止 WWDG 复位。在 WWDG\_SR 寄存器中写 ‘0’ 可以清除该中断。

注：可以用 T6 位产生一个软件复位（设置 WDGA 位为 “1”，T6 位为 “0”）。

### 30.4 如何编写看门狗超时程序

可以使用 0 提供的公式计算窗口看门狗的超时时间。

**警告：**当写入 WWDG\_CR 寄存器时，始终置 T6 位为‘1’以避免立即产生一个复位。

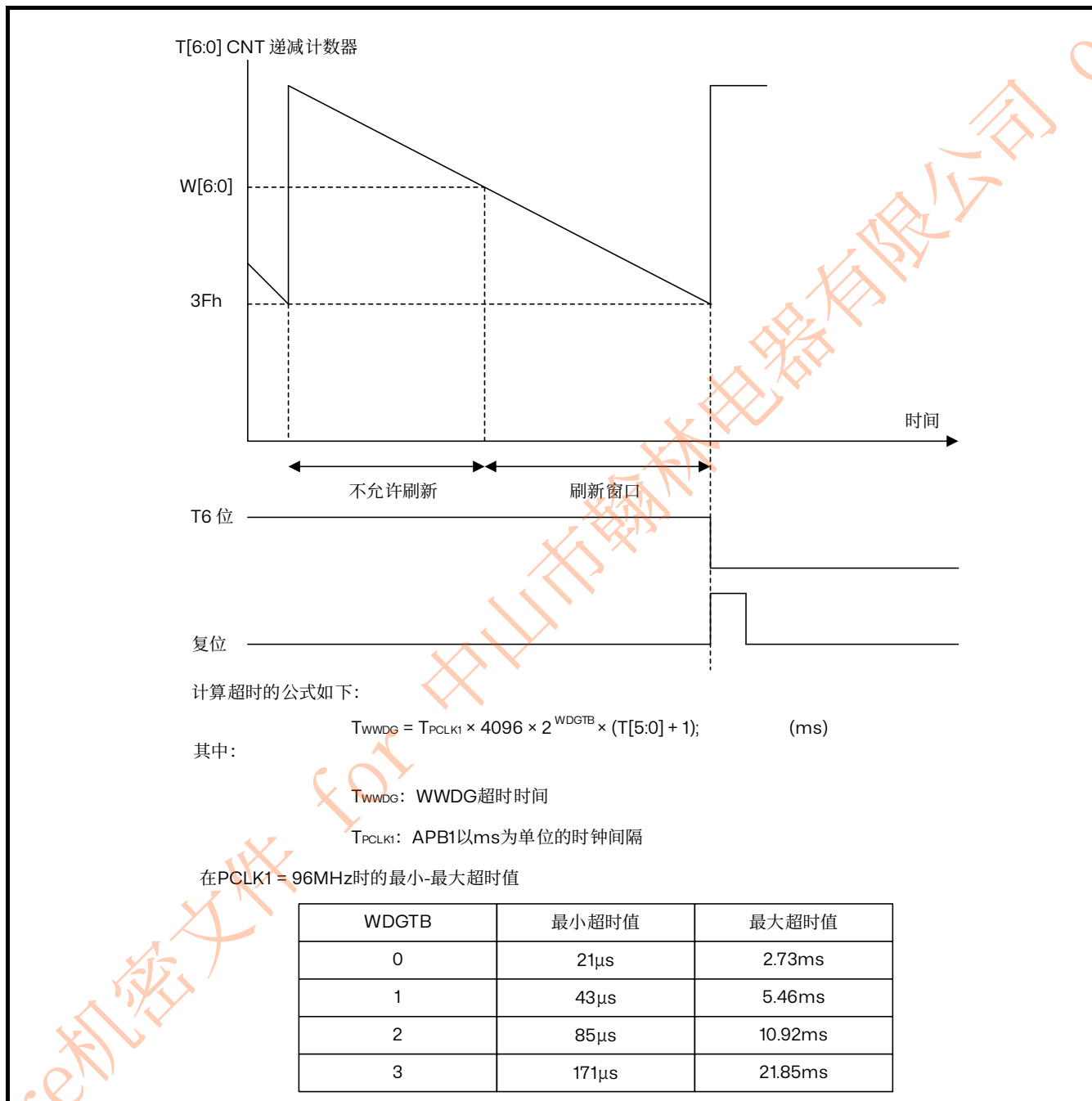


图 30.2 窗口看门狗时序图

### 30.5 调试模式

当微控制器进入调试模式时（Cortex-M4 核心停止），根据调试模块中的 DBG\_WWDG\_STOP 配置位的状态，WWDG 的计数器能够继续工作或停止。

## 30.6 WWDG 寄存器描述

### 30.6.1 控制寄存器 (WWDG\_CR)

地址偏移量: 0x00

复位值: 0x0000 007F

|          |    |    |    |    |    |    |    |      |    |    |    |    |    |    |    |
|----------|----|----|----|----|----|----|----|------|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23   | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |    |    |      |    |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7    | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    |    |    |    |    | WDGA | T6 | T5 | T4 | T3 | T2 | T1 | T0 |
|          |    |    |    |    |    |    |    | rs   | rw | rw | rw | rw | rw | rw | rw |

Bit 31:8 保留, 必须保持为复位值。

Bit 7 WDGA: 激活位 (Activation bit)

此位由软件置 '1', 但仅能由硬件在复位后清 '0'。当 WDGA=1 时, 看门狗可以产生复位。

0: 禁止看门狗

1: 使能看门狗

Bits 6:0 T[6:0]: 7 位计数器 (MSB 至 LSB) (7-bit counter)

这些位用来存储看门狗的计数器值。每  $(4096 \times 2^{WDGTB})$  个 PCLK1 周期减 1。当计数器值从 40h 变为 3Fh 时 (T6 变成 0), 产生看门狗复位。

### 30.6.2 配置寄存器 (WWDG\_CFR)

地址偏移量: 0x04

复位值: 0x0000 007F

|          |    |    |    |    |    |     |    |            |    |    |    |    |    |    |    |    |
|----------|----|----|----|----|----|-----|----|------------|----|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25  | 24 | 23         | 22 | 21 | 20 | 19 | 18 | 17 | 16 |    |
| Reserved |    |    |    |    |    |     |    |            |    |    |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9   | 8  | 7          | 6  | 5  | 4  | 3  | 2  | 1  | 0  |    |
| Reserved |    |    |    |    |    | EWI |    | WDGTB[1:0] |    | W6 | W5 | W4 | W3 | W2 | W1 | W0 |
|          |    |    |    |    |    | rs  | rw | rw         | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 31:10 保留, 必须保持为复位值。

Bit 9 EWI: 提前唤醒中断 (Early wakeup interrupt)

此位若置 '1', 则当计数器值达到 40h, 即产生中断。

此中断只能由硬件在复位后清除。

Bits 8:7 WDG TB[1:0]: 时基 (Timer base)

预分频器的时基可以设置如下:

00: CK 计时器时钟 (PCLK1 除以 4096) 除以 1

01: CK 计时器时钟 (PCLK1 除以 4096) 除以 2

10: CK 计时器时钟 (PCLK1 除以 4096) 除以 4

11: CK 计时器时钟 (PCLK1 除以 4096) 除以 8

Bits 6:0 W[6:0]: 7 位窗口值 (7-bit window value)

这些位包含了用来与递减计数器进行比较用的窗口值。

### 30.6.3 状态寄存器 (WWDG\_SR)

地址偏移量: 0x08

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |    |    |    |    |    |    |       |    |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|-------|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17    | 16 |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    |       |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1     | 0  |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    | EWIF  |    |
|          |    |    |    |    |    |    |    |    |    |    |    |    |    | rc_w0 |    |

Bits 31:1 保留，必须保持为复位值。

Bit 0 **EWIF**: 提前唤醒中断标志 (Early wakeup interrupt flag)

当计数器值达到 40h 时，此位由硬件置 ‘1’。它必须通过软件写 ‘0’ 来清除。对此位写 ‘1’ 无效。若中断未被使能，此位也会被置 ‘1’。

## 31 USB 全速设备接口 (USB)

### 31.1 简介

USB 外设实现了 USB2.0 全速总线和 APB1 总线间的接口。USB 外设支持 USB 挂起/恢复操作，可以停止设备时钟实现低功耗。

### 31.2 主要特征

- 符合 USB2.0 全速设备的技术规范
- 可配置 1 到 8 个 USB 端点
- CRC(循环冗余校验)生成/校验，反向不归零 (NRZI) 编码/解码和位填充
- 支持同步传输
- 支持批量/同步端点的双缓冲区机制
- 支持 USB 挂起/恢复操作
- 帧锁定时钟脉冲生成

注：USB 和 CAN 共用一个专用的 512 字节的 SRAM 存储器用于数据的发送和接收，因此不能同时使用 USB 和 CAN（共享的 SRAM 被 USB 和 CAN 模块互斥地访问）。USB 和 CAN 可以同时用于一个应用中但不能在同一个时间使用。

### 31.3 USB 框图

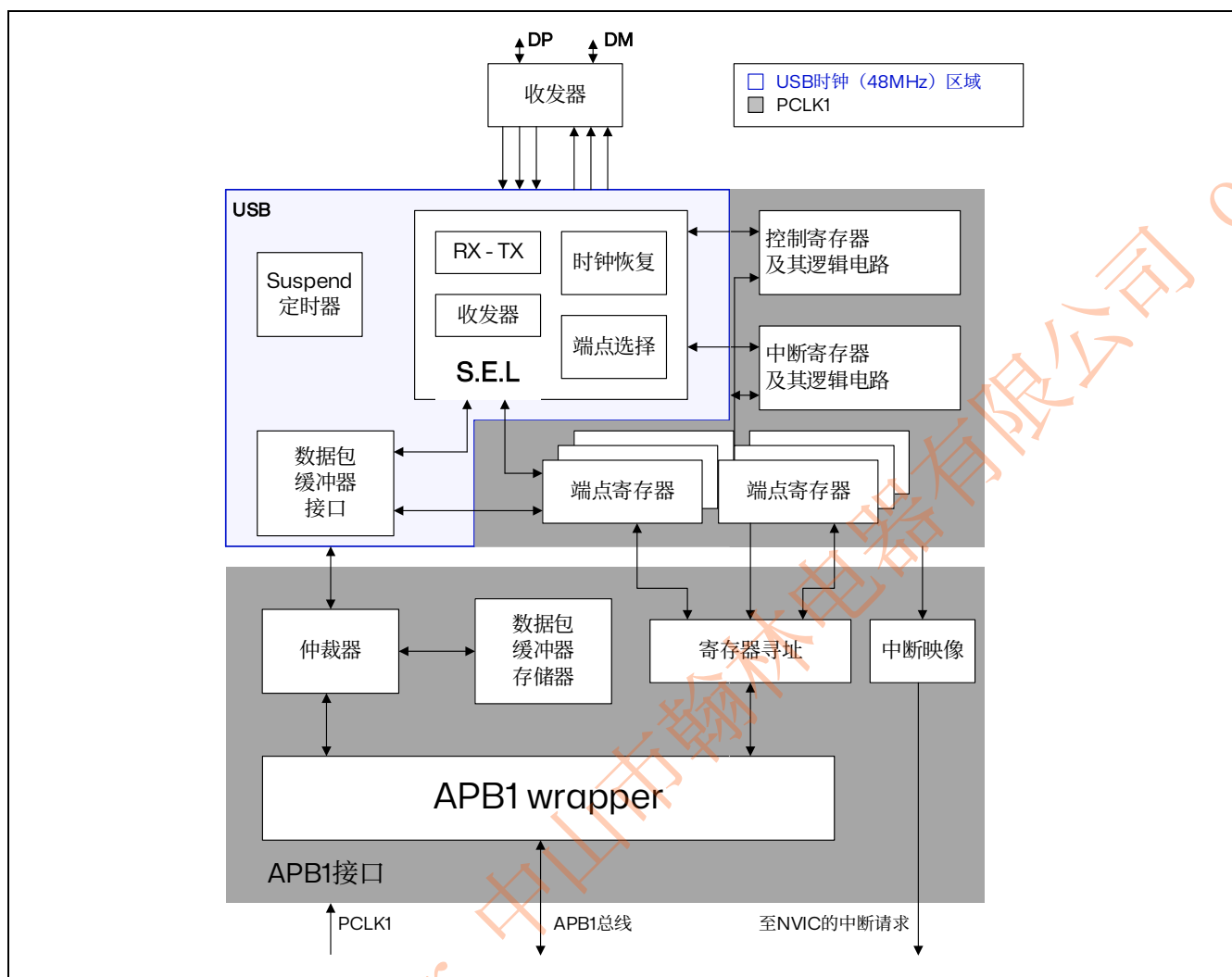


图 31.1 USB 框图

### 31.4 USB 功能描述

USB 模块为 PC 主机和微控制器所实现的功能之间提供了符合 USB 规范的通信连接。PC 主机和微控制器之间的数据传输是通过共享一专用的数据缓冲区来完成的，该数据缓冲区能被 USB 外设直接访问。这块专用数据缓冲区的大小由所使用的端点数目和每个端点最大的数据分组大小 所决定，每个端点最大可使用 512 字节缓冲区，最多可用于 16 个单向或 8 个双向端点。USB 模块同 PC 主机通信，根据 USB 规范实现令牌分组的检测，数据发送/接收的处理，和握手分组的处理。整个传输的格式由硬件完成，其中包括 CRC 的生成和校验。

每个端点都有一个缓冲区描述块，描述该端点使用的缓冲区地址、大小和需要传输的字节数。

当 USB 模块识别出一个有效的功能/端点的令牌分组时，（如果需要传输数据并且端点已配置）随之发生相关的数据传输。USB 模块通过一个内部的 16 位寄存器实现端口与专用缓冲区的数据交换。在所有的数据传输完成后，如果需要，则根据传输的方向，发送或接收适当的握手分组。在数据传输结束时，USB 模块将触发与端点相关的中断，通过读状态寄存器和/或者利用不同的中断处理程序，微控制器可以确定：



- 哪个端点需要得到服务
- 产生如位填充、格式、CRC、协议、缺失 ACK、缓冲区溢出/缓冲区未满足等错误时，正在进行的是哪种类型的传输。

USB 模块对同步传输和高吞吐量的批量传输提供了特殊的双缓冲区机制，在微控制器使用一个缓冲区的时候，该机制保证了 USB 外设总是可以使用另一个缓冲区。

在任何不需要使用 USB 模块的时候，通过写控制寄存器总可以使 USB 模块置于低功耗模式（SUSPEND 模式）。在这种模式下，不产生任何静态电流消耗，同时 USB 时钟也会减慢或停止。通过对 USB 线上数据传输的检测，可以在低功耗模式下唤醒 USB 模块。也可以将一特定的中断输入源直接连接到唤醒引脚上，以使系统能立即恢复正常的时钟系统，并支持直接启动或停止时钟系统。

### 31.4.1 USB 功能模块描述

USB 模块实现了标准 USB 接口的所有特性，它由以下部分组成：

- 串行接口控制器（SIE）：该模块包括的功能有：帧头同步域的识别，位填充，CRC 的产生和校验，PID 的验证/产生，和握手分组处理等。它与 USB 收发器交互，利用分组缓冲接口提供的虚拟缓冲区存储局部数据。它也根据 USB 事件，和类似于传输结束或一个包正确接收等与端点相关事件生成信号，例如帧首（Start of Frame），USB 复位，数据错误等等，这些信号用来产生中断。
- 定时器：本模块的功能是产生一个与帧开始报文同步的时钟脉冲，并在 3ms 内没有数据传输的状态，检测出（主机的）全局挂起条件。
- 分组缓冲器接口：此模块管理那些用于发送和接收的临时本地内存单元。它根据 SIE 的要求分配合适的缓冲区，并定位到端点寄存器所指向的存储区地址。它在每个字节传输后，自动递增地址，直到数据分组传输结束。它记录传输的字节数并防止缓冲区溢出。
- 端点相关寄存器：每个端点都有一个与之相关的寄存器，用于描述端点类型和当前状态。对于单向和单缓冲器端点，一个寄存器就可以用于实现两个不同的端点。一共 8 个寄存器，可以用于实现最多 16 个单向/单缓冲的端点或者 7 个双缓冲的端点或者这些端点的组合。例如，可以同时实现 4 个双缓冲端点和 8 个单缓冲/单向端点。
- 控制寄存器：这些寄存器包含整个 USB 模块的状态信息，用来触发诸如恢复，低功耗等 USB 事件。
- 中断寄存器：这些寄存器包含中断屏蔽信息和中断事件的记录信息。配置和访问这些寄存器可以获取中断源，中断状态等信息，并能清除待处理中断的状态标志。

*注意：端点 0 总是作为单缓冲模式下的控制端点。*

USB 模块通过 APB1 接口部件与 APB1 总线相连，APB1 接口部件包括以下部分：

- 分组缓冲区：数据分组缓存在分组缓冲区中，它由分组缓冲接口控制并创建数据结构。应用软件可以直接访问该缓冲区。它的大小为 512 字节，由 256 个 16 位的字构成。
- 仲裁器：该部件负责处理来自 APB1 总线和 USB 接口的存储器请求。它通过向 APB1 提供较高的访问优先权来解决总线的冲突，并且总是保留一半的存储器带宽供 USB 完成传输。它采用时分复用的策略实现了虚拟的双端口 SRAM，即在 USB 传输的同时，允许应用程序访问存储器。此策略也允许任意长度的多字节 APB1 传输。
- 寄存器映射单元：此部件将 USB 模块的各种字节宽度和位宽度的寄存器映射成能被 APB1 寻址的 16 位宽度的内存集合。
- APB1 封装：此部件为缓冲区和寄存器提供了到 APB1 的接口，并将整个 USB 模块映射到 APB1 地址空间。
- 中断映射单元：将可能产生中断的 USB 事件映射到三个不同的 NVIC 请求线上：

- USB 低优先级中断（通道 20）：可由所有 USB 事件触发（正确传输，USB 复位等）。固件在处理中断前应当首先确定中断源。
- USB 高优先级中断（通道 19）：仅能由同步和双缓冲批量传输的正确传输事件触发，目的是保证最大的传输速率。
- USB 唤醒中断（通道 42）：由 USB 挂起模式的唤醒事件触发。

## 31.5 编程中需要考虑的问题

### 31.5.1 系统复位和上电复位

发生系统复位或者上电复位时，应用程序首先需要做的是提供 USB 模块所需要的时钟信号，然后清除复位信号，使程序可以访问 USB 模块的寄存器。复位之后的初始化流程如下所述：

首先，由应用程序激活寄存器单元的时钟，再配置设备时钟管理逻辑单元的相关控制位，清除复位信号。

其次，必须配置 CNTR 寄存器的 PDWN 位用以开启 USB 收发器相关的模拟部分，这点需要特别的处理。此位能打开为端点收发器供电的内部参照电压。由于打开内部电压需要一段启动时间（数据手册中的  $t_{STARTUP}$ ），在此期间内 USB 收发器处于不确定状态，所以在设置 CNTR 寄存器的 PDWN 后必需等待一段时间之后，才能清除 USB 模块的复位信号（清除 CNTR 寄存器上的 FRES 位），和 ISTR 寄存器的内容，以便在使能其他任何单元的操作之前清除未处理的假中断标志。

最后，应用程序需要通过配置设备时钟管理逻辑的相应控制位来为 USB 模块提供标准所定义的 48MHz 时钟。

当系统复位时，应用程序应该初始化所有需要的寄存器和分组缓冲区描述表，使 USB 模块能够产生正常的中断和完成数据传输。所有与端点无关的寄存器需要根据应用的需求进行初始化（比如中断使能的选择，分组缓冲区地址的选择等）。

### USB 复位（RESET 中断）

发生 USB 复位时，USB 模块进入前面章节中描述过的系统复位状态：所有端点的通信都被禁止（USB 模块不会响应任何分组）。在 USB 复位后，USB 模块被使能，同时地址为 0 的默认控制端点（端点 0）也需要被使能。这可以通过配置 USB\_DADDR 寄存器的 EF 位，EP0R 寄存器和相关的分组缓冲区来实现。在 USB 设备的枚举阶段，主机将分配给设备一个唯一的地址，这个地址必须写入 USB\_DADDR 寄存器的 ADD[6:0]位中，同时配置其他所需的端点。

当复位中断产生时，应用程序必需在中断产生后的 10ms 之内使能端点 0 的传输。

### 分组缓冲区的结构和用途

每个双向端点都可以接收或发送数据。接收到的数据存储在端点指定的专用缓冲区内，而另一个缓冲区则用于存放待发送的数据。对这些缓冲区的访问由分组缓冲区接口模块实现，它提出缓冲区访问请求，并等待确认信息后返回。为防止产生微控制器与 USB 模块对缓冲区的访问冲突，缓冲区接口模块使用仲裁机制，使 APB1 总线的一半周期用于微控制器的访问，另一半保证 USB 模块的访问。这样，微控制器和 USB 模块对分组缓冲区的访问如同对一个双端口 SRAM 的访问，即使微控制器连续访问缓冲区，也不会产生访问冲突。

USB 模块使用固定的时钟，按照 USB 标准，此时钟频率被固定为 48MHz。APB1 总线的时钟可以大于或者小于这个频率。

**注意：**为满足 USB 数据传输率和分组缓冲区接口的系统需求，APB1 总线时钟的频率必须大于 8MHz，以避免数据缓冲区溢出或不满

每个端点对应于两个分组缓冲区（一般一个用于发送，另一个用于接收）。这些缓冲区可以位于整个分组存储区的任意位置，因为它们的地址和长度都定义在缓冲区描述表中，而缓冲区描述表也同样位于分组缓冲区中，其地址由 USB\_BTABLE 寄存器确定。

缓冲区描述表的每个表项都关联到一个端点寄存器，它由 4 个 16 位的字组成，因此缓冲区描述表的起始地址按 8 字节对齐（寄存器的最低 3 位总是 ‘000’）。请参考章节：缓冲区描述表获得详细介绍。

如果是非同步非双缓冲的单向端点，只需要一个分组缓冲区（即发送方向上的分组缓冲区）。

其他未用到的端点或某个未使用的方向上的缓冲区描述表项可以用于其他用途。同步和双缓冲批量端点有特殊的分组缓冲区处理方法（请分别参考章节：同步传输和章节：双缓冲端点）。下图描述了缓冲区描述表项和分组缓冲区区域的关系。

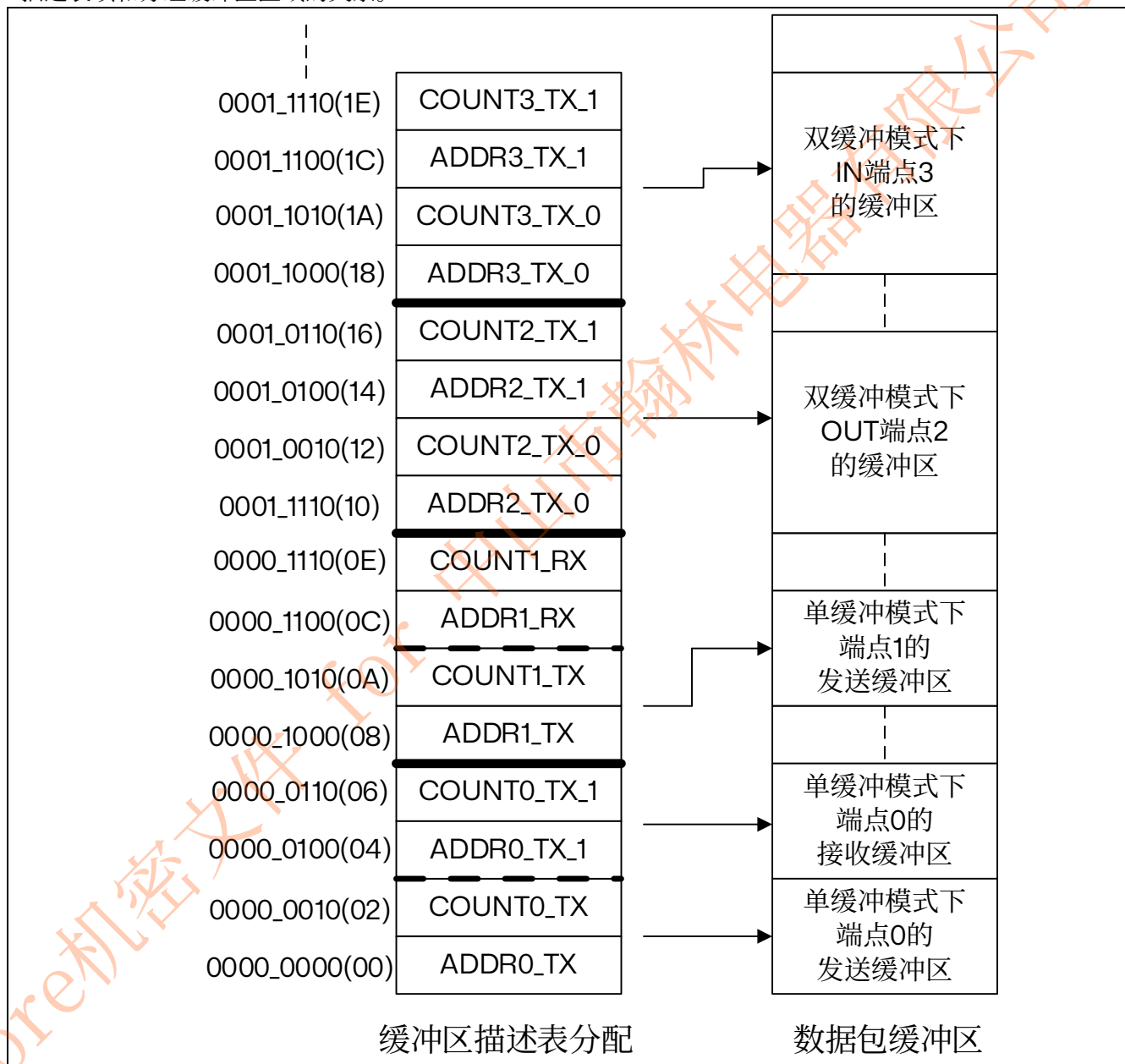


图 31.2 分组缓冲区对应的缓冲区描述表项定位

不管是接收还是发送，分组缓冲区都是从底部开始使用的。USB 模块不会改变超出当前分配到的缓冲区区域以外的其他缓冲区的内容。如果缓冲区收到一个比自己大的数据分组，它只会接收最大为自身大小



的数据，其他的丢掉，即发生了所谓的缓冲区溢出异常。

## 端点初始化

初始化端点的第一步是把适当的值写到 ADDRn\_TX 或 ADDRn\_RX 寄存器中，以便 USB 模块能找到要传输的数据或准备好接收数据的缓冲区。USB\_EPnR 寄存器的 EP\_TYPE 位确定端点的基本类型，EP\_KIND 位确定端点的特殊特性。作为发送方，需要设置 USB\_EPnR 寄存器的 STAT\_TX 位来使能端点，并配置 COUNTn\_TX 位决定发送长度。作为接收方，需要设置 STAT\_RX 位来使能端点，并且设置 BL\_SIZE 和 NUM\_BLOCK 位，确定接收缓冲区的大小，以检测缓冲区溢出的异常。对于非同步非双缓冲批量传输的单向端点，只需要设置一个传输方向上的寄存器。一旦端点被使能，应用程序就不能再修改 USB\_EPnR 寄存器的值和 ADDRn\_TX / ADDRn\_RX，COUNTn\_TX / COUNTn\_RX 所在的位置，因为这些值会被硬件实时修改。当数据传输完成时，CTR 中断会产生，此时上述寄存器可以被访问，并重新使能新的传输。

## IN 分组（用于数据发送）

当接收到一 IN 令牌分组时，如果接收到的地址和一个配置好的端点地址相符合的话，USB 模块将会根据缓冲区描述表的表项，访问相应的 ADDRn\_TX 和 COUNTn\_TX 寄存器，并将这些寄存器中的数值存储到内部的 16 位寄存器 ADDR 和 COUNT(应用程序无法访问)中。此时，USB 模块开始根据 DTOG\_TX 位发送 DATA0 或 DATA1 分组，并访问缓冲区（请参考‘分组缓冲区的结构和用途’段落）。在 IN 分组传输完毕之后，从缓冲区读到的第一个字节将被装载到输出移位寄存器中，并开始发送。最后一个数据字节发送完成之后，计算好的 CRC 将被发送。如果收到的分组所对应的端点是无效的，将根据 USB\_EPnR 寄存器上的 STAT\_TX 位发送 NAK 或 STALL 握手分组而不发送数据。

ADDR 内部寄存器被用作当前缓冲区的指针，COUNT 寄存器用于记录剩下未传输的字节数。USB 总线使用低字节在前的方式传输从缓冲区中读出的数据。数据从 ADDRn\_TX 指向的数据分组缓冲区开始读取，长度为 COUNTn\_TX/2 个字。如果发送的数据分组为奇数个字节，则只使用最后一个字的低 8 位。

在接收到主机响应的 ACK 后，USB\_EPnR 寄存器的值有以下更新：DTOG\_TX 位被翻转，STAT\_TX 位为‘10’，使端点无效，CTR\_TX 位被置位。应用程序需要通过 USB\_ISTR 寄存器的 EP\_ID 和 DIR 位识别产生中断的 USB 端点。CTR\_TX 事件的中断服务程序需要首先清除中断标志位，然后准备好需要发送的数据缓冲区，更新 COUNTn\_TX 为下次需要传输的字节数，最后再设置 STAT\_TX 位为‘11’（端点有效），再次使能数据传输。当 STAT\_TX 位为‘10’时（端点为 NAK 状态），任何发送到该端点的 IN 请求都会被 NAK，USB 主机会重发 IN 请求直到该端点确认请求有效。上述操作过程是必需遵守的，以避免丢失紧随上一次 CTR 中断请求的下一个 IN 传输请求。

## OUT 分组和 SETUP 分组（用于数据接收）

USB 模块对这两种分组的处理方式基本相同；对 SETUP 分组的特殊处理将在下面关于控制传输部分详细说明。当接收到一个 OUT 或 SETUP 分组时，如果地址和某个有效端点的地址相匹配，USB 模块将访问缓冲区描述表，找到与该端点相关的 ADDRn\_RX 和 COUNTn\_RX 寄存器，并将 ADDRn\_RX 寄存器的值保存在内部 ADDR 寄存器中。同时，COUNT 会被复位，从 COUNTn\_RX 中读出的 BL\_SIZE 和 NUM\_BLOCK 的值用于初始化内部 16 位寄存器 BUF\_COUNT，该寄存器用于检测缓冲区溢出（所有的内部寄存器都不能被应用程序访问）。USB 模块将随后收到的数据按字方式组织（先收到的为低字节），并存储到 ADDR 指向的分组缓冲区中。同时，BUF\_COUNT 值自动递减，COUNT 值自动递增。当检测到数据分组的结束信号时，USB 模块校验收到 CRC 的正确性。如果传输中没有任何错误发生，则发送 ACK 握手分组到主机。即使发生 CRC 错误或者其他类型的错误（位填充，帧错误等），数据还是会被保

存到分组缓冲区中，至少会保存到发生错误的点，只是不会发送 ACK 分组，并且 USB\_ISTR 寄存器的 ERR 位将会置位。在这种情况下，应用程序通常不需要干涉处理，USB 模块将从传输错误中自动恢复，并为下一次传输做好准备。如果收到的分组所对应的端点没有准备好，USB 模块将根据 USB\_EPnR 寄存器的 STAT\_RX 位发送 NAK 或 STALL 分组，数据将不会被写入接收缓冲区。

ADDRn\_RX 的值决定接收缓冲区的起始地址，长度由包含 CRC 的数据分组的长度（即有效数据长度 +2）决定，但不能超过 BL\_SIZE 和 NUM\_BLOCK 所定义的缓冲区的长度。如果接收到的数据分组的长度超出了缓冲区的范围，超过范围的数据不会被写入缓冲区，USB 模块将报告缓冲区发生溢出，并向主机发送 STALL 握手分组，通知此次传输失败，也不产生中断。

如果传输正确完成，USB 模块将发送 ACK 握手分组，内部的 COUNT 寄存器的值会被复制到相应的 COUNTn\_RX 寄存器中，BL\_SIZE 和 NUM\_BLOCK 的值保持不变，也不需要重写。USB\_EPnR 寄存器按下列方式更新：DTOG\_RX 位翻转，STAT\_RX=10(NAK)使端点无效，CTR\_RX 位置位（如果 CTR 中断已使能，将触发中断）。如果传输过程中发生了错误或者缓冲区溢出，前面所列出的动作都不会发生。CTR 中断发生时，应用程序需要首先根据 USB\_ISTR 寄存器的 EP\_ID 和 DIR 位识别是哪个端点的中断请求。在处理 CTR\_RX 中断事件时，应用程序首先要确定传输的类型（根据 USB\_EPnR 寄存器的 SETUP 位），同时清除中断标志位，然后读相关的缓冲区描述表项指向的 COUNTn\_RX 寄存器，获得此次传输的总字节数。处理完接收到的数据后，应用程序需要将 USB\_EPnR 中的 STAT\_RX 位置成 '11'，使能下一次的传输。当 STAT\_RX 位为 '10' 时 (NAK)，任何一个发送到端点上的 OUT 请求都会被 NAK，PC 主机将不断重发被 NAK 的分组，直到收到端点的 ACK 握手分组。以上描述的操作次序是必需遵守的，以避免丢失紧随上一个 CTR 中断的另一个 OUT 分组请求。

### 控制传输

控制传输由 3 个阶段组成，首先是主机发送 SETUP 分组的 SETUP 阶段，然后是主机发送零个或多个数据的数据阶段，最后是状态阶段，由与数据阶段方向相反的数据分组构成。SETUP 传输只发生在控制端点，它非常类似于 OUT 分组的传输过程。使能 SETUP 传输除了需要分别初始化 DTOG\_TX 位为 '1'，DTOG\_RX 位为 '0' 外，还需要设置 STAT\_TX 位和 STAT\_RX 位为 10(NAK)，由应用程序根据 SETUP 分组的相应位域决定后面的传输是 IN 还是 OUT。控制端点在每次发生 CTR\_RX 中断时，都必须检查 USB\_EPnR 寄存器的 SETUP 位，以识别是普通的 OUT 分组还是 SETUP 分组。USB 设备应该能够通过 SETUP 分组中的相应数据决定数据阶段传输的字节数和方向，并且能在发生错误的情况下发送 STALL 分组，拒绝数据的传输。因此在数据阶段，未被使用到的方向都应该被设置成 STALL，并且在开始传输数据阶段的最后一个数据分组时，其反方向的传输仍设成 NAK 状态，这样，即使主机立刻改变了传输方向（进入状态阶段），仍然可以保持为等待控制传输结束的状态。在控制传输成功结束后，应用程序可以把 NAK 变为 VALID，如果控制传输出错，就改为 STALL。此时，如果状态分组是由主机发送给设备的，那么 STATUS\_OUT 位（USB\_EPnR 寄存器中的 EP\_KIND）应该被置位，只有这样，在状态传输过程中收到了非零长度的数据分组，才会产生传输错误。在完成状态传输阶段后，应用程序应该清除 STATUS\_OUT 位，并且将 STAT\_RX 设为 VALID 表示已准备好接收一个新的命令请求，STAT\_TX 则设为 NAK，表示在下一个 SETUP 分组传输完成前，不接受数据传输的请求。

USB 规范定义 SETUP 分组不能以非 ACK 握手分组来响应，如果 SETUP 分组传输失败，则会引发下一个 SETUP 分组。因此，以 NAK 或 STALL 分组响应主机的 SETUP 分组是被禁止的。

当 STAT\_RX 位被设置为 '01' (STALL) 或 '10' (NAK) 时，如果收到 SETUP 分组，USB 模块会接收分组，开始分组所要求的数据传输，并回送 ACK 握手分组。如果应用程序在处理前一个 CTR\_RX 事件时 USB 模块又收到了 SETUP 分组（即 CTR\_RX 仍然保持置位），USB 模块会丢掉收到的 SETUP 分组，并且不回答任何握手分组，以此来模拟一个接收错误，迫使主机再次发送 SETUP 分组。这样做是为了避免丢失紧随一次 CTR\_RX 中断之后的又一个 SETUP 分组传输。

### 31.5.2 双缓冲端点

USB 标准不仅为不同的传输模式定义了不同的端点类型，而且对这些数据传输所需要的系统要求做了描述。其中，批量端点适用于在主机 PC 和 USB 设备之间传输大批量的数据，因为主机可以在一帧内利用尽可能多的带宽批量传输数据，使传输效率得到提高。然而，当 USB 设备处理 前一次的数据传输时，又收到新的数据分组，它将回应 NAK 分组，使 PC 主机不断重发同样的数据分组，直到设备在可以处理数据时回应 ACK 分组。这样的重传占用了带宽，影响了批量传输的速率，因此引入了批量端点的双缓冲机制，提高数据传输率。

使用双缓冲机制时，单向端点的数据传输将使用到该端点的接收和发送两块数据缓冲区。数据翻转位用来选择当前使用到两块缓冲区中的哪一块，使应用程序可以在 USB 模块访问其中一块缓冲区的同时，对另一块缓冲区进行操作。例如，对一个双缓冲批量端点进行 OUT 分组传输时，USB 模块将来自 PC 主机的数据保存到一个缓冲区，同时应用程序可以对另一个缓冲区中的数据进行处理（对于 IN 分组来说，情况是一样的）。

因为切换缓冲区的管理机制需要用到所有 4 个缓冲区描述表的表项，分别用来表示每个方向上的两个缓冲区的地址指针和缓冲区大小，因此用来实现双缓冲批量端点的 USB\_EPnR 寄存器必需配置为单向。所以只需要设定 STAT\_RX 位（作为双缓冲批量接收端点）或者 STAT\_TX 位（作为双缓冲批量发送端点）。如果需要一个双向的双缓冲批量端点，则须使用两个 USB\_EPnR 寄存器。

为尽可能利用双缓冲的优势，达到较高的传输速率，双缓冲批量端点的流量控制流程与其他端点的稍有不同。它只在缓冲区发生访问冲突时才会设置端点为 NAK 状态，而不是在每次传输成功后都将端点设为 NAK 状态。

DTOG 位用来标识 USB 模块当前所使用的储存缓冲区。双缓冲批量端点接收方向的缓冲区由 DTOG\_RX (USB\_EPnR 寄存器的第 14 位) 标识，而双缓冲批量端点发送方向的缓冲区由 DTOG\_TX (USB\_EPnR 寄存器的第 6 位) 标识。同时，USB 模块也需要知道当前哪个缓冲区正在被应用程序使用，以避免发生冲突。由于 USB\_EPnR 寄存器中有 2 个 DTOG 位，而 USB 模块只使用其中的一位来标识硬件所使用的缓冲区，因此，应用程序可使用另一位来标识当前正在使用哪个缓冲区，这个新的标识被称为 SW\_BUF 位。下表列出了双缓冲批量端点在实现发送和接收操作时，USB\_EPnR 寄存器的 DTOG 位和 SW\_BUF 位之间的关系。

表 31.1 双缓冲批量端点缓冲区标识定义

| 缓冲区标识位 | 作为发送端点                      | 作为接收端点                       |
|--------|-----------------------------|------------------------------|
| DTOG   | DTOG_TX(USB_EPnR 寄存器的第 6 位) | DTOG_RX(USB_EPnR 寄存器的第 14 位) |
| SW_BUF | USB_EPnR 寄存器的第 14 位         | USB_EPnR 寄存器的第 6 位           |

表 31.2 双缓冲批量端点的缓冲区使用标识

| 端点类型   | DTOG 位 | SW_BUF 位 | USB 模块使用的缓冲区             | 应用程序使用的缓冲区               |
|--------|--------|----------|--------------------------|--------------------------|
| IN 端点  | 0      | 1        | ADDRn_TX_0 / COUNTn_TX_0 | ADDRn_TX_1 / COUNTn_TX_1 |
|        | 1      | 0        | ADDRn_TX_1 / COUNTn_TX_1 | ADDRn_TX_0 / COUNTn_TX_0 |
|        | 0      | 0        | 无 <sup>(1)</sup>         | ADDRn_TX_0 / COUNTn_TX_0 |
|        | 1      | 1        | 无 <sup>(1)</sup>         | ADDRn_TX_0 / COUNTn_TX_0 |
| OUT 端点 | 0      | 1        | ADDRn_RX_0 / COUNTn_RX_0 | ADDRn_RX_1 / COUNTn_RX_1 |
|        | 1      | 0        | ADDRn_RX_1 / COUNTn_RX_1 | ADDRn_RX_0 / COUNTn_RX_0 |
|        | 0      | 0        | 无 <sup>(1)</sup>         | ADDRn_RX_0 / COUNTn_RX_0 |



|  |   |   |                  |                          |
|--|---|---|------------------|--------------------------|
|  | 1 | 1 | 无 <sup>(1)</sup> | ADDRn_RX_0 / COUNTn_RX_0 |
|--|---|---|------------------|--------------------------|

可以通过以下方式设置一个双缓冲批量端点：

- 将 USB\_EPnR 寄存器的 EP\_TYPE 位设为 ‘00’，定义端点为批量端点
- 将 USB\_EPnR 寄存器的 EP\_KIND 位设为 ‘1’，定义端点为双缓冲端点

应用程序根据传输开始时用到的缓冲区来初始化 DTOG 和 SW\_BUF 位；这需要考虑到这两位的数据翻转特性。设置好 DBL\_BUF 位之后，每完成一次传输后，USB 模块将根据双缓冲批量端点的流量控制操作，并且持续到 DBL\_BUF 变为无效为止。每次传输结束，根据端点的传输方向，CTR\_RX 位或 CTR\_TX 位将会置为 ‘1’。与此同时，硬件将设置相应的 DTOG 位，完全独立于软件来实现缓冲区交换机制。DBL\_BUF 位设置后，每次传输结束时，双缓冲批量端点的 STAT 位的取值不会像其他类型端点一样受到传输过程的影响，而是一直保持为 ‘11’（有效）。但是，如果在收到新的数据分组的传输请求时，USB 模块和应用程序发生了缓冲区访问冲突（即 DTOG 和 SW\_BUF 为相同的值，见上表），状态位将会被置为 ‘10’（NAK）。应用程序响应 CTR 中断时，首先要清除中断标志，然后再处理传输完成的数据。应用程序访问缓冲区之后，需要翻转 SW\_BUF 位，以通知 USB 模块该块缓冲区已变为可用状态。由此，双缓冲批量传输的 NAK 分组的数目只由应用程序处理一次数据传输的快慢所决定：如果数据处理的时间小于 USB 总线上完成一次数据传输的时间，则不会发生重传，此时，数据的传输率仅受限于 USB 主机。

应用程序也可以不考虑双缓冲批量端点的特殊控制流程，直接在相应 USB\_EPnR 寄存器的 STAT 位写入非 ‘11’ 的任何状态，在这种情况下，USB 模块将按照写入的状态执行流程而忽略缓冲器实际的使用情况。

### 31.5.3 同步传输

USB 标准定义了一种全速的 need 保持固定和精确的数据传输率的传输方式：同步传输。同步传输一般用于传输音频流、压缩的视频流等对数据传输率有严格要求的数据。一个端点如果在枚举时被定义为“同步端点”，USB 主机则会为每个帧分配固定的带宽，并且保证每个帧正好传送一个 IN 分组或者 OUT 分组（由端点传输方向确定分组类型）。为了满足带宽要求，同步传输中没有出错重传；这也就意味着，同步传输在发送或接收数据分组之后，无握手协议，即不会发送 ACK 分组。同样，同步传输只传送 PID（分组 ID）为 DATA0 的数据包，而不会用到数据翻转机制。

通过设置 USB\_EPnR 寄存器 EP\_TYPE 为 ‘10’，可以使其成为同步端点。同步端点没有握手机制，根据 USB 标准中的说明，USB\_EPnR 寄存器的 STAT\_RX 位和 STAT\_TX 位分别只能设成 ‘00’（禁止）和 ‘11’（有效）。同步传输通过实现双缓冲机制来简化软件应用程序开发，它同样使用两个缓冲区，以确保在 USB 模块使用其中一块缓冲区时，应用程序可以访问另外一块缓冲区。

USB 模块使用的缓冲区根据不同的传输方向，由不同的 DTOG 位来标识。（同一寄存器中的 DTOG\_RX 位用来标识接收同步端点，DTOG\_TX 位用来标识发送同步端点），见下表。

表 31.3 同步端点的缓冲区使用标识

| 端点类型   | DTOG 位值 | USB 模块使用的缓冲区             | 应用程序使用的缓冲区               |
|--------|---------|--------------------------|--------------------------|
| IN 端点  | 0       | ADDRn_TX_0 / COUNTn_TX_0 | ADDRn_TX_1 / COUNTn_TX_1 |
|        | 1       | ADDRn_TX_1 / COUNTn_TX_1 | ADDRn_TX_0 / COUNTn_TX_0 |
| OUT 端点 | 0       | ADDRn_RX_0 / COUNTn_RX_0 | ADDRn_RX_1 / COUNTn_RX_1 |
|        | 1       | ADDRn_RX_1 / COUNTn_RX_1 | ADDRn_RX_0 / COUNTn_RX_0 |

与双缓冲批量端点一样，一个 USB\_EPnR 寄存器只能处理同步端点单方向的数据传输，如果要求同步端点在两个传输方向上都有效，则需要使用两个 USB\_EPnR 寄存器。

应用程序需要根据首次传输的数据分组来初始化 DTOG 位；它的取值还需要考虑到 DTOG\_RX 或



DTOG\_TX 两位的数据翻转特性。每次传输完成时，USB\_EPnR 寄存器的 CTR\_RX 位或 CTR\_TX 位置位。与此同时，相关的 DTOG 位由硬件翻转，从而使得交换缓冲区的操作完全独立于应用程序。传输结束时，STAT\_RX 或 STAT\_TX 位不会发生变化，因为同步传输没有握手机制，所以不需要任何流量控制，而一直设为‘11’（有效）。同步传输中，即使 OUT 分组发生 CRC 错误或者缓冲区溢出，本次传输仍被看作是是正确的，并且可以触发 CTR\_RX 中断事件；但是，发生 CRC 错误时硬件会设置 USB\_ISTR 寄存器的 ERR 位，提醒应用程序数据可能损坏。

### 31.5.4 挂起/恢复事件

USB 标准中定义了一种特殊的设备状态，即挂起状态，在这种状态下 USB 总线上的平均电流消耗不超过 500uA。这种电流限制对于由总线供电的 USB 设备至关重要，而自供电的设备则不需要严格遵守这样的电流消耗限制。USB 主机以 3 毫秒内不发送任何信号标志进入挂起状态。通常情况下 USB 主机每毫秒会发送一个 SOF，当 USB 模块检测到 3 个连续的 SOF 分组丢失事件即可判定主机发出了挂起请求，接着它会置位 SB\_ISTR 寄存器的 SUSP 位，以触发挂起中断。USB 设备进入挂起状态之后，将由“唤醒”序列唤醒。所谓的“唤醒”序列，可以由 USB 主机发起，也可以由 USB 设备本身触发；但是，只有 USB 主机可以结束“唤醒”序列。被挂起的 USB 模块必须至少还具备检测 RESET 信号的功能，它会将其当作一次正常的复位操作来执行。实际的挂起操作过程对于不同的 USB 设备来说是不同的，因为需要不同的操作来降低电源消耗。下面描述了一起典型的挂起操作，重点介绍应用程序如何响应 USB 模块的 SUSP 信号。

1. 将 USB\_CNTR 寄存器的 FSUSP 置为‘1’，这将使 USB 模块进入挂起状态。USB 模块一旦进入挂起状态，对 SOF 的检测立刻停止，以避免在 USB 挂起时又发生新的 SUSP 事件。
2. 消除或减少 USB 模块以外的其他模块的静态电流消耗。
3. 将 USB\_CNTR 寄存器的 LP\_MODE 位置为‘1’，这将消除模拟 USB 收发器的静态电流消耗，但仍能检测到唤醒信号。
4. 可以选择关闭外部振荡器和设备的 PLL，以停止设备内部的任何活动。

当设备处于挂起状态时发生 USB 事件，该设备会被唤醒，并需要调用“唤醒”例程来恢复系统时钟，和 USB 数据传输。如果唤醒设备的是 USB 复位操作，则应该保证唤醒的过程不要超过 10 毫秒（参见“USB 协议规范”）。USB 模块处于挂起状态时，唤醒或复位事件需要清除 USB\_CNTR 寄存器的 LP\_MODE 位。即使唤醒事件可以立刻触发一个 WKUP 中断事件，但由于恢复系统时钟需要比较长的延迟时间，处理 WKUP 中断的中断服务程序必须非常小心；为了减短系统唤醒的时间，建议将唤醒代码直接写在挂起代码后面，这样就可以在系统时钟重启后迅速进入唤醒代码中执行。为防止或减少 ESD 等干扰意外地唤醒系统（从挂起模式退出是一个异步事件），在挂起过程中数据线被过滤，滤波宽度大约为 70nS。

下面是唤醒操作的过程：

启动外部振荡器和设备的 PLL（此项可选）。

1. 清零 USB\_CNTR 寄存器的 FSUSP 位。
2. USB\_FNR 寄存器的 RXDP 和 RXDM 位可以用来判断是什么触发了唤醒事件，如下表所示，它还同时列出了各种情况软件应该采取的操作。如果需要的话，可以通过检测这两位变成‘10’（代表空闲总线状态）的时间来知道唤醒或复位事件的结束。此外，在复位事件结束时，USB\_ISTR 寄存器的 RESET 位被置为‘1’，如果 RESET 中断被使能，就会产生中断。此中断应该按正常的复位操作处理。

表 31.4 唤醒事件检测

| [RXDP, RXDM]的状态 | 唤醒事件        | 应用程序应执行的操作 |
|-----------------|-------------|------------|
| 00              | 复位          | 无          |
| 10              | 无(总线干扰)     | 恢复到挂起状态    |
| 01              | 恢复挂起        | 无          |
| 11              | 未定义的值(总线干扰) | 恢复到挂起状态    |

设备可能不是被与 USB 模块相关的事件唤醒的（例如一个鼠标的移动可唤醒整个系统）。在这种情况下，先将 USB\_CNTR 寄存器的 RESUME 位置为 ‘1’，然后在 1ms—15ms 之间再把它清为 0 可以启动唤醒序列(这个间隔可以用 ESOF 中断来实现，该中断在内核正常运行时每 1ms 发生一次)。RESUME 位被清零后，唤醒过程将由主机 PC 完成，可以利用 USB\_FNR 寄存器的 RXDP 和 RXDM 位来判断唤醒是否完成。

**注意：**只有在 USB 模块被设置为挂起状态时（设置 USB\_CNTR 寄存器的 FSUSP 位为 ‘1’），才可以设置 RESUME 位。

## 31.6 USB 寄存器描述

USB 模块的寄存器有以下三类：

- 通用类寄存器：中断寄存器和控制寄存器
- 端点类寄存器：端点配置寄存器和状态寄存器
- 缓冲区描述表类寄存器：用来确定数据分组存放地址的寄存器

缓冲区描述表类寄存器的基地址由 USB\_BTABLE 寄存器指定，所有其他寄存器的基地址则为 USB 模块的基地址 0x4000 5C00。由于 APB1 总线按 32 位寻址，因此所有的 16 位寄存器的地址都是按 32 位字对齐的。同样的地址对齐方式也用于从 0x4000 6000 开始的分组缓冲存储区。

可以用半字(16 位)或字(32 位)的方式操作这些外设寄存器。

### 31.6.1 通用寄存器

#### USB 控制寄存器(USB\_CNTR)

地址偏移量：0x40

复位值：0x0003

|          |         |      |       |       |        |      |       |          |    |    |        |       |         |      |      |
|----------|---------|------|-------|-------|--------|------|-------|----------|----|----|--------|-------|---------|------|------|
| 31       | 30      | 29   | 28    | 27    | 26     | 25   | 24    | 23       | 22 | 21 | 20     | 19    | 18      | 17   | 16   |
| Reserved |         |      |       |       |        |      |       |          |    |    |        |       |         |      |      |
| 15       | 14      | 13   | 12    | 11    | 10     | 9    | 8     | 7        | 6  | 5  | 4      | 3     | 2       | 1    | 0    |
| CTRM     | PMAOVRM | ERRM | WKUPM | SUSPM | RESETM | SOFM | ESOFM | Reserved |    |    | RESUME | FSUSP | LP_MODE | PDMN | FRES |
| rw       | rw      | rw   | rw    | rw    | rw     | rw   | rw    |          |    |    | rw     | rw    | rw      | rw   | rw   |

Bits 31:16 保留，必须保持为复位值。

Bit 15 **CTRM**：正确传输（CTR）中断屏蔽位（Correct transfer interrupt mask）

0：正确传输(CTR)中断禁止

1：正确传输(CTR)中断使能，在中断寄存器的相应位被置 1 时产生中断。

Bit 14 **PMAOVRM**：分组缓冲区溢出中断屏蔽位（Packet memory area over / underrun interrupt mask）

0：PMAOVR 中断禁止

1：PMAOVR 中断使能，在中断寄存器的相应位被置 1 时产生中断。

Bit 13 **ERRM**: 出错中断屏蔽位 (Error interrupt mask)

0: 出错中断禁止

1: 出错中断使能, 在中断寄存器的相应位被置 1 时产生中断。

Bit 12 **WKUPM**: 唤醒中断屏蔽位 (Wakeup interrupt mask)

0: 唤醒中断禁止

1: 唤醒中断使能, 在中断寄存器的相应位被置 1 时产生中断。

Bit 11 **SUSPM**: 挂起中断屏蔽位 (Suspend mode interrupt mask)

0: 挂起 (SUSP) 中断禁止

1: 挂起 (SUSP) 中断使能, 在中断寄存器的相应位被置 1 时产生中断。

Bit 10 **RESETM**: USB 复位中断屏蔽位 (USB reset interrupt mask)

0: USB RESET 中断禁止

1: USB RESET 中断使能, 在中断寄存器的相应位被置 1 时产生中断。

Bit 9 **SOFM**: 帧首中断屏蔽位 (Start of frame interrupt mask)

0: SOF 中断禁止

1: SOF 中断使能, 在中断寄存器的相应位被置 1 时产生中断。

Bit 8 **ESOFM**: 期望帧首中断屏蔽位 (Expected start of frame interrupt mask)

0: ESOF 中断禁止

1: ESOF 中断使能, 在中断寄存器的相应位被置 1 时产生中断。

Bit 7:5 保留, 必须保持为复位值。

Bit 4 **RESUME**: 唤醒请求 (Resume request)

设置此位将向 PC 主机发送唤醒请求。根据 USB 协议, 如果此位在 1ms 到 15ms 内保持有效, 主机将对 USB 模块实行唤醒操作。

Bit 3 **FSUSP**: 强制挂起 (Force suspend)

当 USB 总线上保持 3ms 没有数据通信时, SUSP 中断会被触发, 此时软件必需设置此位。

0: 无效

1: 进入挂起模式, USB 模拟收发器的时钟和静态功耗仍然保持。如果需要进入低功耗状态(总线供电类的设备), 应用程序需要先置位 FSUSP 再置位 LP\_MODE。

Bit 2 **LP\_MODE**: 低功耗模式 (Low-power mode)

此模式用于在 USB 挂起状态下降低功耗。在此模式下, 除了外接上拉电阻的供电, 其他的静态功耗都被关闭, 系统时钟将会停止或者降低到一定的频率来减少耗电。USB 总线上的活动 (唤醒事件) 将会复位此位 (软件也可以复位此位)。

0: 非低功耗模式

1: 低功耗模式

Bit 1 **PDWN**: 断电模式 (Power down)

此模式用于彻底关闭 USB 模块。当此位被置位时, 不能使用 USB 模块。

0: 退出断电模式

1: 进入断电模式

Bit 0 **FRES**: 强制 USB 复位 (Force USB Reset)

0: 清除 USB 复位信号

1: 对 USB 模块强制复位, 类似于 USB 总线上的复位信号。USB 模块将一直保持在复位状态下直到软件清除此位。如果 USB 复位中断被使能, 将产生一个复位中断。

## USB 中断状态寄存器(USB\_ISTR)

地址偏移量: 0x44

复位值: 0x0000

| 31       | 30      | 29    | 28    | 27    | 26    | 25    | 24    | 23       | 22 | 21 | 20  | 19         | 18 | 17 | 16 |
|----------|---------|-------|-------|-------|-------|-------|-------|----------|----|----|-----|------------|----|----|----|
| Reserved |         |       |       |       |       |       |       |          |    |    |     |            |    |    |    |
| 15       | 14      | 13    | 12    | 11    | 10    | 9     | 8     | 7        | 6  | 5  | 4   | 3          | 2  | 1  | 0  |
| CTR      | PMA OVR | ERR   | WKUP  | SUSP  | RESET | SOF   | ESOF  | Reserved |    |    | DIR | EP_ID[3:0] |    |    |    |
| r        | rc_w0   | rc_w0 | rc_w0 | rc_w0 | rc_w0 | rc_w0 | rc_w0 |          |    |    | r   | r          | r  | r  | r  |

Bits 31:16 保留，必须保持为复位值。

### Bit 15 CTR: 正确的传输 (Correct transfer)

此位在端点正确完成一次数据传输后由硬件置位。应用程序可以通过 DIR 和 EP\_ID 位来识别是哪个端点完成了正确的数据传输。

此位应用程序只读

### Bit 14 PMAOVR: 分组缓冲区溢出 (Packet memory area over / underrun)

此位在微控制器长时间没有响应一个访问 USB 分组缓冲区请求时由硬件置位。USB 模块通常在以下情况时置位该位：在接收过程中一个 ACK 握手分组没有被发送，或者在发送过程中发生了比特填充错误，在以上两种情况下主机都会要求数据重传。在正常的数据传输中不会产生 PMAOVR 中断。由于失败的传输都将由主机发起重传，应用程序就可以在这个中断的服务程序中加速设备的其他操作，并准备重传。但这个中断不会在同步传输中产生（同步传输不支持重传）因此数据可能会丢失。

此位应用程序可读可写，但只有写 0 有效，写 1 无效。

### Bit 13 ERR: 出错 (Error)

在下列错误发生时硬件会置位此位。

NANS: 无应答。主机的应答超时。

CRC: 循环冗余校验码错误。数据或令牌分组中的 CRC 校验出错。

BST: 位填充错误。PID，数据或 CRC 中检测出位填充错误。

FVIO: 帧格式错误。收到非标准帧（如 EOP 出现在错误的时刻，错误的令牌等）。

USB 应用程序通常可以忽略这些错误，因为 USB 模块和主机在发生错误时都会启动重传机制。此位产生的中断可以用于应用程序的开发阶段，可以用来监测 USB 总线的传输质量，标识用户可能发生的错误（连接线松，环境干扰严重，USB 线损坏等）。

此位应用程序可读可写，但只有写 0 有效，写 1 无效。

### Bit 12 WKUP: 唤醒请求 (Wakeup)

当 USB 模块处于挂起状态时，如果检测到唤醒信号，此位将由硬件置位。此时 CTRL 寄存器的 LP\_MODE 位将被清零，同时 USB\_WAKEUP 被激活，通知设备的其他部分（如唤醒单元）将开始唤醒过程。

此位应用程序可读可写，但只有写 0 有效，写 1 无效。

### Bit 11 SUSP: 挂起模块请求 (Suspend mode request)

此位在 USB 线上超过 3ms 没有信号传输时由硬件置位，用以指示一个来自 USB 总线的挂起请求。USB 复位后硬件立即使能对挂起信号的检测，但在挂起模式下（FSUSP = 1）硬件不会再检测挂起信号直到唤醒过程结束。

此位应用程序可读可写，但只有写 0 有效，写 1 无效。



**Bit 10 RESET: USB 复位请求 (USB reset request)**

此位在 USB 模块检测到 USB 复位信号输入时由硬件置位。此时 USB 模块将复位内部协议状态机，并在中断使能的情况下触发复位中断来响应复位信号。USB 模块的发送和接收部分将被禁止，直到此位被清除。所有的配置寄存器不会被复位，除非应用程序对他们清零。这用来保证在复位后 USB 传输还可以立即正确执行。但设备的地址和端点寄存器会被 USB 复位所复位。

此位应用程序可读可写，但只有写 0 有效，写 1 无效。

**Bit 9 SOF: 帧首标志 (Start of frame)**

此位在 USB 模块检测到总线上的 SOF 分组时由硬件置位，标志一个新的 USB 帧的开始。中断服务程序可以通过检测 SOF 事件来完成与主机的 1ms 同步，并正确读出寄存器在收到 SOF 分组时的更新内容（此功能在同步传输时非常有意义）。

此位应用程序可读可写，但只有写 0 有效，写 1 无效。

**Bit 8 ESOF: 期望帧首标识位 (Expected start of frame)**

此位在 USB 模块未收到期望的 SOF 分组时由硬件置位。主机应该每毫秒都发送 SOF 分组，但如果 USB 模块没有收到，挂起定时器将触发此中断。如果连续发生 3 次 ESOF 中断，也就是连续 3 次未收到 SOF 分组，将产生 SUSP 中断。即使在挂起定时器未被锁定时发生 SOF 分组丢失，此位也会被置位。

此位应用程序可读可写，但只有写 0 有效，写 1 无效。

Bit 7:5 保留，必须保持为复位值。

**Bit 4 DIR: 传输方向 (Direction of transaction)**

此位在完成数据传输产生中断后由硬件根据传输方向写入。

如果 DIR=0，相应端点的 CTR\_TX 位被置位，标志一个 IN 分组（数据从 USB 模块传输到 PC 主机）的传输完成。

如果 DIR=1，相应端点的 CTR\_RX 位被置位，标志一个 OUT 分组（数据从 PC 主机传输到 USB 模块）的传输完成。如果 CTR\_TX 位同时也被置位，就标志同时存在挂起的 OUT 分组和 IN 分组。应用程序可以利用该信息访问 USB\_EPnR 位对应的操作，它表示挂起中断传输方向的信息。

该位为只读。

**Bit 3:0 EP\_ID[3:0]: 端点 ID (Endpoint Identifier)**

此位在 USB 模块完成数据传输产生中断后由硬件根据请求中断的端点号写入。如果同时有多个端点的请求中断，硬件写入优先级最高的端点号。端点的优先级按以下方法定义：同步端点和双缓冲批量端点具有高优先级，其他的端点为低优先级。如果多个同优先级的端点请求中断，则根据端点号来确定优先级，即端点 0 具有最高优先级，端点号越小，优先级越高。应用程序可以通过上述的优先级策略顺序处理端点的中断请求。

该位为只读。

**USB 帧编号寄存器 (USB\_FNR)**

地址偏移量: 0x48

复位值: 0x0XXX, X 代表未定义数值

|          |      |     |           |    |          |    |    |    |    |    |    |    |    |    |    |
|----------|------|-----|-----------|----|----------|----|----|----|----|----|----|----|----|----|----|
| 31       | 30   | 29  | 28        | 27 | 26       | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |      |     |           |    |          |    |    |    |    |    |    |    |    |    |    |
| 15       | 14   | 13  | 12        | 11 | 10       | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| RXDP     | RXDM | LCK | LSOF[1:0] |    | FN[10:0] |    |    |    |    |    |    |    |    |    |    |
| r        | r    | r   | r         | r  | r        | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  |

Bits 31:16 保留, 必须保持为复位值。

**Bit 15 RXDP:** D+状态位 (Receive data + line status)

此位用于观察 USB D+数据线的状态, 可在挂起状态下检测唤醒条件的出现。

**Bit 14 RXDM:** D-状态位 (Receive data - line status)

此位用于观察 USB D-数据线的状态, 可在挂起状态下检测唤醒条件的出现。

**Bit 13 LCK:** 锁定位 (Locked)

USB 模块在复位或唤醒序列结束后会检测 SOF 分组, 如果连续检测到至少 2 个 SOF 分组, 则硬件会置位此位。此位一旦锁定, 帧计数器将停止计数, 一直等到 USB 模块复位或总线挂起时再恢复计数。

**Bit 12:11 LSOF[1:0]:** 帧首丢失标志位 (Lost SOF)

当 ESOF 事件发生时, 硬件会将丢失的 SOF 分组的数目写入此位。如果再次收到 SOF 分组, 引脚会清除此位。

**Bit 10:0 FN[10:0]:** 帧编号 (Frame number)

此部分记录了最新收到的 SOF 分组中的 11 位帧编号。主机每发送一个帧, 帧编号都会自加, 这对于同步传输非常有意义。此部分发生 SOF 中断时更新。

**USB 设备地址寄存器 (USB\_DADDR)**

地址偏移量: 0x4C

复位值: 0x0000

|          |    |    |    |    |    |    |    |    |          |    |    |    |    |    |    |
|----------|----|----|----|----|----|----|----|----|----------|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22       | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |    |    |    |          |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6        | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    |    |    |    |    | EF | ADD[6:0] |    |    |    |    |    |    |
|          |    |    |    |    |    |    |    | rw | rw       | rw | rw | rw | rw | rw | rw |

Bit 31:8 保留, 必须保持为复位值。

**Bit 7 EF:** USB 模块使能位 (Enable function)

此位在需要使能 USB 模块时由应用程序置位。如果此位为 0, USB 模块将停止工作, 忽略所有寄存器的设置, 不响应任何 USB 通信。

**Bit 6:0 ADD[6:0]:** 设备地址 (device address)

此位记录了 USB 主机在枚举过程中为 USB 设备分配的地址值。该地址值和端点地址 (EA) 必需和 USB 令牌分组中的地址信息匹配, 才能在指定的端点进行正确的 USB 传输。

**USB 分组缓冲区描述表地址寄存器 (USB\_BTABE)**

地址偏移量: 0x50

复位值: 0x0000

|              |    |    |    |    |    |    |    |    |    |    |    |    |          |    |    |
|--------------|----|----|----|----|----|----|----|----|----|----|----|----|----------|----|----|
| 31           | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18       | 17 | 16 |
| Reserved     |    |    |    |    |    |    |    |    |    |    |    |    |          |    |    |
| 15           | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2        | 1  | 0  |
| BTABLE[12:0] |    |    |    |    |    |    |    |    |    |    |    |    | Reserved |    |    |
| rw           | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |          |    |    |

Bits 31:16 保留, 必须保持为复位值。

Bit 15:3 **BTABLE[15:3]**: 缓冲表 (Buffer table)

此位记录分组缓冲区描述表的起始地址。分组缓冲区描述表用来指示每个端点的分组缓冲区地址和大小, 按 8 字节对齐 (即最低 3 位为 000)。每次传输开始时, USB 模块读取相应端点所对应的分组缓冲区描述表获得缓冲区地址和大小信息。

Bit 2:0 保留, 必须保持为复位值。

### 31.6.2 端点寄存器

**USB 端点 n 寄存器 (USB\_EPnR) (n=[0..7])**

地址偏移量: 0x00 至 0x1C

复位值: 0x0000

|          |         |              |    |       |              |    |         |        |         |              |    |         |    |    |    |
|----------|---------|--------------|----|-------|--------------|----|---------|--------|---------|--------------|----|---------|----|----|----|
| 31       | 30      | 29           | 28 | 27    | 26           | 25 | 24      | 23     | 22      | 21           | 20 | 19      | 18 | 17 | 16 |
| Reserved |         |              |    |       |              |    |         |        |         |              |    |         |    |    |    |
| 15       | 14      | 13           | 12 | 11    | 10           | 9  | 8       | 7      | 6       | 5            | 4  | 3       | 2  | 1  | 0  |
| CTR_RX   | DTOG_RX | STAT_RX[1:0] |    | SETUP | EP_TYPE[1:0] |    | EP_KIND | CTR_TX | DTOG_TX | STAT_TX[1:0] |    | EA[3:0] |    |    |    |
| rc_w0    | t       | t            | t  | r     | rw           | rw | rw      | rc_w0  | t       | t            | t  | rw      | rw | rw | rw |

当 USB 模块收到 USB 总线复位信号, 或 CTRLR 寄存器的 FRES 位置位时, USB 模块将会复位。该寄存器除了 CTR\_RX 和 CTR\_TX 位保持不变以处理紧随的 USB 传输外, 其他位都被复位。每个端点对应一个 USB\_EPnR 寄存器, 其中 n 为端点地址, 即端点 ID 号

对于此类寄存器应避免执行读出一修改一写入操作, 因为在读和写操作之间, 硬件可能会设置某些位, 而这些位又会在写入时被修改, 导致应用程序错过相应的操作。因此, 这些位都有一个写入无效的值, 建议用 Load 指令修改这些寄存器, 以免应用程序修改了不需要修改的位。

Bits 31:16 保留, 必须保持为复位值。

Bit 15 **CTR\_RX**: 正确接收标志位 (Correct Transfer for reception)

此位在正确接收到 OUT 或 SETUP 分组时由硬件置位, 应用程序只能对此位清零。如果 CTRM 位已置位, 相应的中断会产生。收到的是 OUT 分组还是 SETUP 分组可以通过下面描述的 SETUP 位确定。以 NAK 或 STALL 结束的分组和出错的传输不会导致此位置位, 因为没有真正传输数据。

此位应用程序可读可写, 但只有写 0 有效, 写 1 无效。

Bit 14 **DTOG\_RX**: 用于数据接收的数据翻转位 (Data Toggle, for reception transfers)

对于非同步端点, 此位由硬件设置, 用于标记希望接收的下一个数据分组的 Toggle 位 (0=DATA0, 1=DATA1)。在接收到 PID (分组 ID) 正确的数据分组之后, USB 模块发送 ACK 握手分组, 并翻转此位。对于控制端点, 硬件在收到 SETUP 分组后清除此位。



对于双缓冲端点，此位还用于支持双缓冲区的交换（请参考 24.5.3 双缓冲端点）。

对于同步端点，由于仅发送 DATA0，因此此位仅用于支持双缓冲区的交换（请参考 24.5.4 同步传输）而不需进行翻转。同步传输不需要握手分组，因此硬件在收到数据分组后立即设置此位。

应用程序可以对此位进行初始化（对于非控制端点，初始化是必需的），或者翻转此位用于特殊用途。

此位应用程序可读可写，但写 0 无效，写 1 可以翻转此位。

**Bit 13:12 STAT\_RX[1:0]:** 用于数据接收的状态位（Status bits, for reception transfers）

此位用于指示端点当前的状态，表 157 列出了端点的所有状态。当一次正确的 OUT 或 SETUP 数据传输完成后(CTR\_RX=1)，硬件会自动设置此位为 NAK 状态，使应用程序有足够的时间在处理完当前传输的数据后，响应下一个数据分组。

对于双缓冲批量端点，由于使用特殊的传输流量控制策略，因此根据使用的缓冲区状态控制传输状态（请参考 24.5.3 双缓冲端点）。

对于同步端点，由于端点状态只能是有效或禁用，因此硬件不会在正确的传输之后设置此位。

如果应用程序将此位设为 STALL 或者 NAK，USB 模块响应的操作是未定义的。

此位应用程序可读可写，但写 0 无效，写 1 翻转此位。

**Bit 11 SETUP:** SETUP 分组传输完成标志位（Setup transaction completed）

此位在 USB 模块收到一个正确的 SETUP 分组后由硬件置位，只有控制端点才使用此位。在接收完成后(CTR\_RX=1)，应用程序需要检测此位以判断完成的传输是否是 SETUP 分组。为了防止中断服务程序在处理 SETUP 分组时下一个令牌分组修改了此位，只有 CTR\_RX 为 0 时，此位才可以被修改，CTR\_RX 为 1 时不能修改。此位应用程序只读。

**Bit 10:9 EP\_TPYE[1:0]:** 端点类型位（Endpoint type）

此位用于指示端点当前的类型，所有的端点类型都在表 24.6 中列出。所有的 USB 设备都必需包含一个地址为 0 的控制端点，如果需要可以有其他地址的控制端点。只有控制端点才会有 SETUP 传输，其他类型的端点无视此类传输。SETUP 传输不能以 NAK 或 STALL 分组响应，如果控制端点在收到 SETUP 分组时处于 NAK 状态，USB 模块将不响应分组，就会出现接收错误。如果控制端点处于 STALL 状态，SETUP 分组会被正确接收，数据会被正确传输，并产生一个正确传输完成的中断。控制端点的 OUT 分组安装普通端点的方式处理。

批量端点和中断端点的处理方式非常类似，仅在对 EP\_KIND 位的处理上有差别。

24.5.4 同步端点的用法请参考同步传输。

**Bit 8 EP\_KIND:** 端点特殊类型位（Endpoint kind）

此位的需要和 EP\_TYPE 位配合使用，具体的定义请参考表 24.7。

**DBL\_BUF:** 应用程序设置此位能使能批量端点的双缓冲功能。详见 24.5.3 双缓冲端点。

**STATUS\_OUT:** 应用程序设置此位表示 USB 设备期望主机发送一个状态数据分组，此时，设备对于任何长度不为 0 的数据分组都响应 STALL 分组。此功能仅用于控制端点，有利于提供应用程序对于协议层错误的检测。如果 STATUS\_OUT 位被清除，OUT 分组可以包含任意长度的数据。

**Bit 7 CTR\_TX:** 正确发送标志位（Correct transfer for transmission）

此位由硬件在一个正确的 IN 分组传输完成后置位。如果 CTRM 位已被置位，会产生相

应的中断。应用程序需要在处理完该事件后清除此位。在 IN 分组结束时，如果主机响应 NAK 或 STALL 则此位不会被置位，因为数据传输没有成功。

此位应用程序可读可写，但写 0 有效，写 1 无效。

#### Bit 6 DTOG\_RX: 发送数据翻转位 (Data Toggle, for transmission transfers)

对于非同步端点，此位用于指示下一个要传输的数据分组的 Toggle 位(0=DATA0, 1=DATA1)。在一个成功传输的数据分组后，如果 USB 模块接收到主机发送的 ACK 分组，就会翻转此位。对于控制端点，USB 模块会在收到正确的 SETUP PID 后置位此位。

对于双缓冲端点，此位还可用于支持分组缓冲区交换 (请参考 24.5.3 双缓冲端点)。对于同步端点，由于只传送 DATA0，因此该位只用于支持分组缓冲区交换 (请参考 24.5.4 同步传输)。由于同步传输不需要握手分组，因此硬件在接收到数据分组后即设置该位。

应用程序可以初始化该位(对于非控制端点，初始化此位时必需的)，也可以设置该位用于特殊用途。

此位应用程序可读可写，但写 0 无效，写 1 翻转此位。

#### Bit 5:4 STAT\_TX[1:0]: 用于发送数据的状态位 (Status bits, for transmission transfers)

此位用于标识端点的当前状态，表 24.8 列出了所有的状态。应用程序可以翻转这些位来初始化状态信息。在正确完成一次 IN 分组的传输后(CTR\_TX=1)，硬件会自动设置此位为 NAK 状态，保证应用程序有足够的时间准备好数据响应后续的数据传输。

对于双缓冲批量端点，由于使用特殊的传输流量控制策略，是根据缓冲区的状态控制传输的状态的 (请参考 24.5.3 双缓冲端点)。对于同步端点，由于端点的状态只能是有效或禁用，因此硬件不会在数据传输结束时改变端点的状态。如果应用程序将此位设为 STALL 或者 NAK，则 USB 模块后续的操作是未定义的。

此位应用程序可读可写，但写 0 无效，写 1 翻转此位。

#### Bit 3:0 EA[3:0]: 端点地址 (Endpoint address)

应用程序必需设置此 4 位，在使能一个端点前为它定义一个地址。

表 31.5 接收状态编码

| STAT_RX[1:0] | 描述                           |
|--------------|------------------------------|
| 00           | DISABLED: 端点忽略所有的接收请求        |
| 01           | STALL: 端点以 STALL 分组响应所有的接收请求 |
| 10           | NAK: 端点以 NAK 分组响应所有的接收请求     |
| 11           | VALID: 端点可用于接收               |

表 31.6 端点类型编码

| STAT_RX[1:0] | 描述              |
|--------------|-----------------|
| 00           | BULK: 批量端点      |
| 01           | CONTROL: 控制端点   |
| 10           | ISO: 同步端点       |
| 11           | INTERRUPT: 中断端点 |

表 31.7 端点特殊类型定义

| STAT_RX[1:0] |           | 描述                           |
|--------------|-----------|------------------------------|
| 00           | BULK      | DISABLED: 端点忽略所有的接收请求        |
| 01           | CONTROL   | STALL: 端点以 STALL 分组响应所有的接收请求 |
| 10           | ISO       | NAK: 端点以 NAK 分组响应所有的接收请求     |
| 11           | INTERRUPT | VALID: 端点可用于接收               |

表 31.8 发送状态编码

| STAT_RX[1:0] |  | 描述                           |
|--------------|--|------------------------------|
| 00           |  | DISABLED: 端点忽略所有的发送请求        |
| 01           |  | STALL: 端点以 STALL 分组响应所有的发送请求 |
| 10           |  | NAK: 端点以 NAK 分组响应所有的发送请求     |
| 11           |  | VALID: 端点可用于发送               |

### 31.6.3 缓冲区描述表

#### 发送缓冲区地址寄存器 n (USB\_ADDRn\_TX)

地址偏移量: [USB\_BTABLE] + n×16

USB 本地地址: [USB\_BTABLE] + n×8

|                |    |    |    |    |    |    |    |    |    |    |    |    |    |    |     |
|----------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|-----|
| 31             | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16  |
| Reserved       |    |    |    |    |    |    |    |    |    |    |    |    |    |    |     |
| 15             | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0   |
| ADDRn_TX[14:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    | Res |
| rw             | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |     |

Bits 31:16 保留, 必须保持为复位值。

Bit 15:0 ADDRn\_TX[14:0]: 发送缓冲区地址 (Transmission buffer address)

此位记录了收到下一个 IN 分组时, 需要发送的数据所在的缓冲区起始地址。

Bit 0 因为分组缓冲区的地址必须按字对齐, 所以此位必须为 '0'。

#### 发送数据字节数寄存器 n (USB\_COUNTn\_TX)

地址偏移量: [USB\_BTABLE] + n×16 + 4

USB 本地地址: [USB\_BTABLE] + n×8 + 2

|          |    |    |    |    |    |                |    |    |    |    |    |    |    |    |    |
|----------|----|----|----|----|----|----------------|----|----|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25             | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |                |    |    |    |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9              | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    |    |    | COUNTn_TX[9:0] |    |    |    |    |    |    |    |    |    |
|          |    |    |    |    |    | rw             | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:10 由于 USB 模块支持的最大数据分组为 1023 个字节, 所以 USB 模块忽略这些位。

Bit 9:0 COUNTn\_TX[9:0]: 发送数据字节数 (Transmission byte count)

此位记录了收到下一个 IN 分组时要传输的数据字节数。

注: 双缓冲区和同步 IN 端点有两个 USB\_COUNTn\_TX 寄存器: 分别为 USB\_COUNTn\_TX\_1 和 USB\_COUNTn\_TX\_0, 内容如下:

|          |    |    |    |    |    |                  |    |    |    |    |    |    |    |    |    |
|----------|----|----|----|----|----|------------------|----|----|----|----|----|----|----|----|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25               | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |    |    | COUNTn_TX_1[9:0] |    |    |    |    |    |    |    |    |    |

接收缓冲区地址寄存器 n (USB\_ADDRn\_RX)

USB 本地地址:  $[USB\_BTABLE] + n \times 8 + 4$

Bits 31:16 保留，必须保持为复位值。

Bit 15:0 **ADDRn\_RX[15:1]**: 接收缓冲区地址 (Reception buffer address)

此位记录了收到下一个 OUT 或者 SETUP 分组时，用于保存数据的缓冲区起始地址

Bit 0 因为分组缓冲区的地址按字对齐，所以此位必需为‘0’。

## 接收数据字节数寄存器 n (USB\_COUNTn\_RX)

USB 本地地址: [USB\_BTABLE] + n×8 + 6

Bits 31:16 保留，必须保持为复位值。

Bit 15 **BL\_SIZE**: 存储区块的大小 (Block size)

此位用于定义决定缓冲区大小的存储区块的大小。

如果 BL\_SIZE=0，存储区块的大小为 2 字节，因此能分配的分组缓冲区的大小范围为 2—62 个字节。

如果 BL\_SIZE=1, 存储区块的大小为 32 字节, 因此能分配的分组缓冲区的大小范围为 32~512 字节, 符合 USB 协议定义的最大分组长度限制。

Bit 14:10 NUM\_BLOCK[4:0]: 存储区块的数目 (Number of blocks)

此位用以记录分配的存储区块的数目，从而决定最终使用的分组缓冲区的大小。具体请参考表 24.9

Bit 9:0 **COUNTn\_RX[9:0]**: 接收到的字节数 (Reception byte count)

此位由 USB 模块写入，用以记录端点收到的最新的 OUT 或 SETUP 分组的实际字节数。

注：双缓冲区和同步 IN 端点有两个 USB\_COUNTn\_RX 寄存器：分别为 USB\_COUNTn\_RX\_1 和 USB\_COUNTn\_TX\_0。内容如下：

睿兴科技（南京）有限公司

|          |                  |    |    |    |    |                  |    |    |    |    |    |    |    |    |    |
|----------|------------------|----|----|----|----|------------------|----|----|----|----|----|----|----|----|----|
| 15       | 14               | 13 | 12 | 11 | 10 | 9                | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| BLSIZE_0 | NUM_BLOCK_0[4:0] |    |    |    |    | COUNTn_RX_0[9:0] |    |    |    |    |    |    |    |    |    |
| rw       | rw               | rw | rw | rw | rw | rw               | rw | rw | rw | rw | rw | rw | rw | rw | rw |

表 31.9 分组缓冲区大小的定义

| NUM_BLOCK[4:0]的值 | BL_SIZE=0 时的分组缓冲区大小 | 当 BL_SIZE=1 时的分组缓冲区大小 |
|------------------|---------------------|-----------------------|
| 00000            | 不允许使用               | 32 字节                 |
| 00001            | 2 字节                | 64 字节                 |
| 00010            | 4 字节                | 96 字节                 |
| 00011            | 6 字节                | 128 字节                |
| ...              | ...                 | ...                   |
| 01111            | 30 字节               | 512 字节                |
| 10000            | 32 字节               | 保留                    |
| 10001            | 34 字节               | 保留                    |
| 10010            | 36 字节               | 保留                    |
| ...              | ...                 | ...                   |
| 11110            | 62 字节               | 保留                    |
| 11111            | 62 字节               | 保留                    |

## 32 控制器局域网 (bxCAN)

### 32.1 简介

bxCAN 是基本扩展 CAN (Basic Extended CAN) 的缩写, 它支持 CAN 协议 2.0A 和 2.0B。它的设计目标是, 以最小的 CPU 负荷来高效处理大量收到的报文。它也支持报文发送的优先级要求 (优先级特性可软件配置)。

对于安全紧要的应用, bxCAN 提供所有支持时间触发通信模式所需的硬件功能。

### 32.2 bxCAN 主要特点

- 支持 CAN 协议 2.0A 和 2.0B 主动模式
- 波特率最高可达 1 兆位/秒
- 支持时间触发通信功能

#### 发送

- 3 个发送邮箱
- 发送报文的优先级特性可软件配置
- 记录发送 SOF 时刻的时间戳

#### 接收

- 3 级深度的 2 个接收 FIFO
- 可变的过滤器组:
- 标识符列表
- FIFO 溢出处理方式可配置
- 记录接收 SOF 时刻的时间戳

#### 时间触发通信模式

- 禁止自动重传模式
- 16 位自由运行定时器
- 可在最后 2 个数据字节发送时间戳

#### 管理

- 中断可屏蔽
- 邮箱占用单独 1 块地址空间, 便于提高软件效率

### 32.3 bxCAN 总体描述

在当今的 CAN 应用中, CAN 网络的节点在不断增加, 并且多个 CAN 常常通过网关连接起来, 因此整个 CAN 网中的报文数量 (每个节点都需要处理) 急剧增加。除了应用层报文外, 网络管理和诊断报文也被引入。

- 需要一个增强的过滤机制来处理各种类型的报文

此外, 应用层任务需要更多 CPU 时间, 因此报文接收所需的实时响应程度需要减轻。

- 接收 FIFO 的方案允许, CPU 花很长时间处理应用层任务而不会丢失报文。

构筑在底层 CAN 驱动程序上的高层协议软件, 要求跟 CAN 控制器之间有高效的接口。



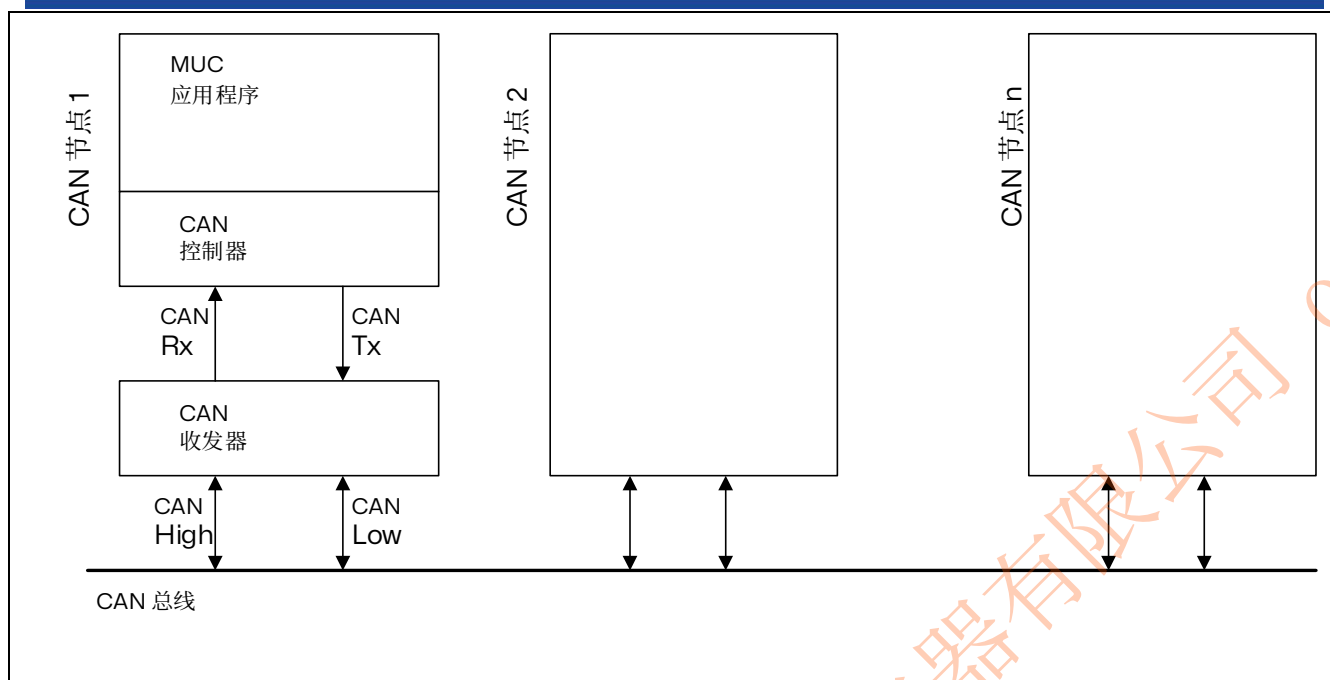


图 32.1 CAN 网拓扑结构

### 32.3.1 CAN 2.0B 主动内核

bxCAN 模块可以完全自动地接收和发送 CAN 报文；且完全支持标准标识符（11 位）和扩展标识符（29 位）。

### 32.3.2 控制、状态和配置寄存器

应用程序通过这些寄存器，可以：

- 配置 CAN 参数，如波特率
- 请求发送报文
- 处理报文接收
- 管理中断
- 获取诊断信息

### 32.3.3 发送邮箱

共有 3 个发送邮箱供软件来发送报文。发送调度器根据优先级决定哪个邮箱的报文先被发送。

### 32.3.4 接收过滤器

#### 接收 FIFO

共有 2 个接收 FIFO，每个 FIFO 都可以存放 3 个完整的报文。它们完全由硬件来管理



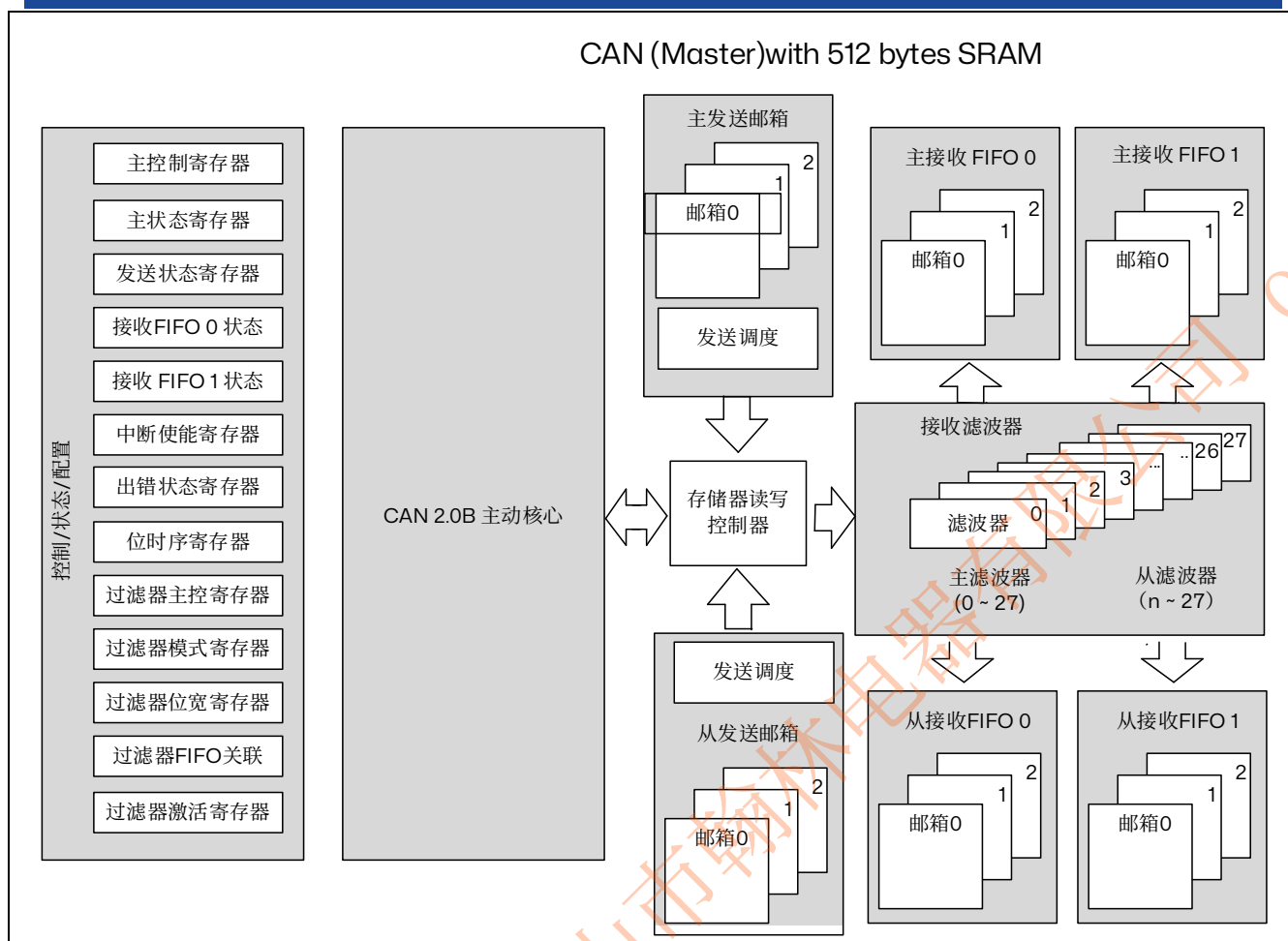


图 32.2 CAN 框图

## 32.4 bxCAN 工作模式

bxCAN 有 3 个主要的工作模式：初始化、正常和睡眠模式。在硬件复位后，bxCAN 工作在睡眠模式以节省电能，同时 CANTX 引脚的内部上拉电阻被激活。软件通过对 CAN\_MCR 寄存器的 INRQ 或 SLEEP 位置 '1'，可以请求 bxCAN 进入初始化或睡眠模式。一旦进入了初始化或睡眠模式，bxCAN 就对 CAN\_MSR 寄存器的 INAK 或 SLAK 位置 '1' 来进行确认，同时内部上拉电阻被禁用。当 INAK 和 SLAK 位都为 '0' 时，bxCAN 就处于正常模式。在进入正常模式前，bxCAN 必须跟 CAN 总线取得同步；为取得同步，bxCAN 要等待 CAN 总线达到空闲状态，即在 CANRX 引脚上监测到 11 个连续的隐性位。

### 32.4.1 初始化模式

软件初始化应该在硬件处于初始化模式时进行。设置 CAN\_MCR 寄存器的 INRQ 位为 '1'，请求 bxCAN 进入初始化模式，然后等待硬件对 CAN\_MSR 寄存器的 INAK 位置 '1' 来进行确认。

清除 CAN\_MCR 寄存器的 INRQ 位为 '0'，请求 bxCAN 退出初始化模式，当硬件对 CAN\_MSR 寄存器的 INAK 位清 '0' 就确认了初始化模式的退出。

当 bxCAN 处于初始化模式时，禁止报文的接收和发送，并且 CANTX 引脚输出隐性位（高电平）。初始化模式的进入，不会改变配置寄存器。

软件对 bxCAN 的初始化，至少包括位时间特性 (CAN\_BTR) 和控制 (CAN\_MCR) 这 2 个寄存器。

在对 bxCAN 的过滤器组（模式、位宽、FIFO 关联、激活和过滤器值）进行初始化前，软件要对 CAN\_FMR 寄存器的 FINIT 位设置 ‘1’。对过滤器的初始化可以在非初始化模式下进行。

*注：当 FINIT=1 时，报文的接收被禁止。*

*可以先对过滤器激活位清 ‘0’（在 CAN\_FA1R 中），然后修改相应过滤器的值。*

*如果过滤器组没有使用，那么就应该让它处于非激活状态（保持其 FACT 位为清 ‘0’ 状态）。*

### 32.4.2 正常模式

在初始化完成后，软件应该让硬件进入正常模式，以便正常接收和发送报文。软件可以通过对 CAN\_MCR 寄存器的 INRQ 位清 ‘0’，来请求从初始化模式进入正常模式，然后要等待硬件对 CAN\_MSR 寄存器的 INAK 位置 ‘0’ 的确认。在跟 CAN 总线取得同步，即在 CANRX 引脚上监测到 11 个连续的隐性位（等效于总线空闲）后，bxCAN 才能正常接收和发送报文。

不需要在初始化模式下进行过滤器初值的设置，但必须在它处在非激活状态下完成（相应的 FACT 位为 0）。而过滤器的位宽和模式的设置，则必须在初始化模式中进入正常模式前完成。

### 32.4.3 睡眠模式(低功耗)

bxCAN 可工作在低功耗的睡眠模式。软件通过对 CAN\_MCR 寄存器的 SLEEP 位置 ‘1’，来请求进入这一模式。在该模式下，bxCAN 的时钟停止了，但软件仍然可以访问邮箱寄存器。当 bxCAN 处于睡眠模式，软件必须对 CAN\_MCR 寄存器的 INRQ 位置 ‘1’ 并且同时对 SLEEP 位清 ‘0’，才能进入初始化模式。

有 2 种方式可以唤醒(退出睡眠模式)bxCAN：通过软件对 SLEEP 位清 ‘0’，或硬件检测到 CAN 总线的活动。

如果 CAN\_MCR 寄存器的 AWUM 位为 ‘1’，一旦检测到 CAN 总线的活动，硬件就自动对 SLEEP 位清 ‘0’ 来唤醒 bxCAN。如果 CAN\_MCR 寄存器的 AWUM 位为 ‘0’，软件必须在唤醒中断里对 SLEEP 位清 ‘0’ 才能退出睡眠状态。

*注：如果唤醒中断被允许（CAN\_IER 寄存器的 WKUIE 位为 ‘1’），那么一旦检测到 CAN 总线活动就会产生唤醒中断，而不管硬件是否会自动唤醒 bxCAN。*

在对 SLEEP 位清 ‘0’ 后，睡眠模式的退出必须与 CAN 总线同步，下图：bxCAN 工作模式。当硬件对 SLAK 位清 ‘0’ 时，就确认了睡眠模式的退出。

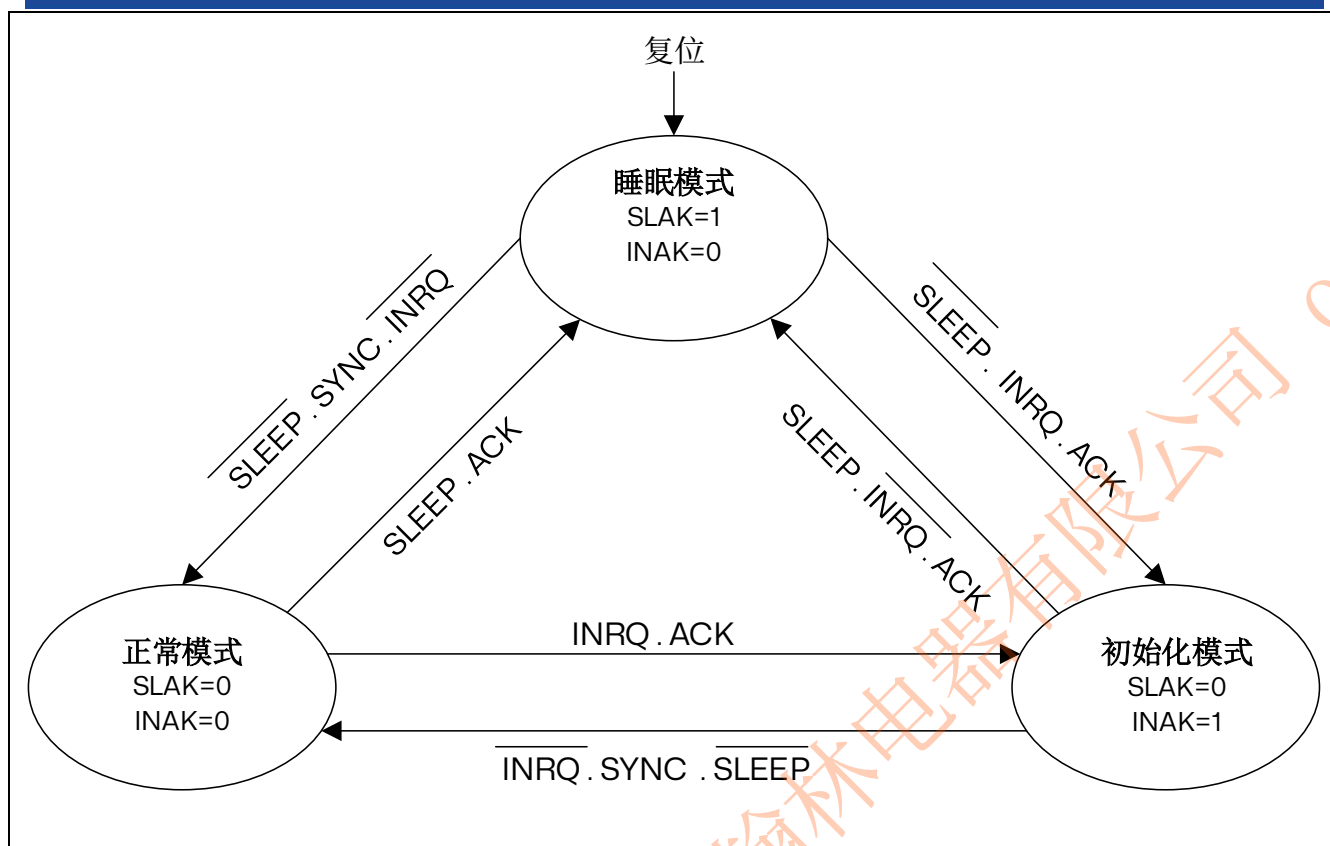


图 32.3 bxCAN 工作模式

- 注: 1 ACK = 硬件响应睡眠或初始化请求,而对 CAN\_MSR 寄存器的 INAK 或 SLAK 位置 1 的状态  
2 SYNC = bxCAN 等待 CAN 总线变为空闲的状态,即在 CANRX 引脚上检测到连续的 11 个隐性位

## 32.5 测试模式

通过对 CAN\_BTR 寄存器的 SILM 和/或 LBKM 位置 ‘1’, 来选择一种测试模式。只能在初始化模式下, 修改这 2 位。在选择了一种测试模式后, 软件需要对 CAN\_MCR 寄存器的 INRQ 位清 ‘0’, 来真正进入测试模式。

### 32.5.1 静默模式

通过对 CAN\_BTR 寄存器的 SILM 位置 ‘1’, 来选择静默模式。

在静默模式下, bxCAN 可以正常地接收数据帧和远程帧, 但只能发出隐性位, 而不能真正发送报文。如果 bxCAN 需要发出显性位 (确认位、过载标志、主动错误标志), 那么这样的显性位在内部被接回来从而可以被 CAN 内核检测到, 同时 CAN 总线不会受到影响而仍然维持在隐性位状态。因此, 静默模式通常用于分析 CAN 总线的活动, 而不会对总线造成影响—显性位 (确认位、错误帧) 不会真正发送到总线上。

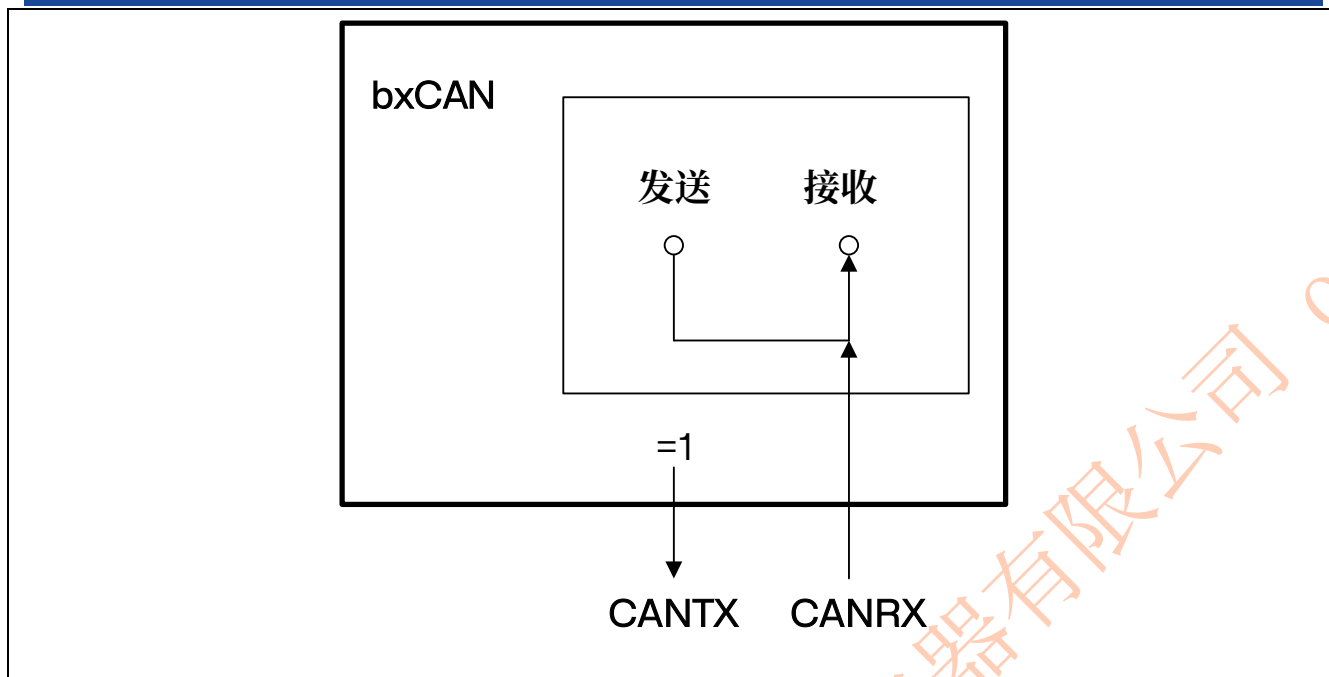


图 32.4 bxCAN 工作在静默模式

### 32.5.2 环回模式

通过对 CAN\_BTR 寄存器的 LBKM 位置 ‘1’，来选择环回模式。在环回模式下，bxCAN 把发送的报文当作接收的报文并保存（如果可以通过接收过滤）在接收邮箱里。

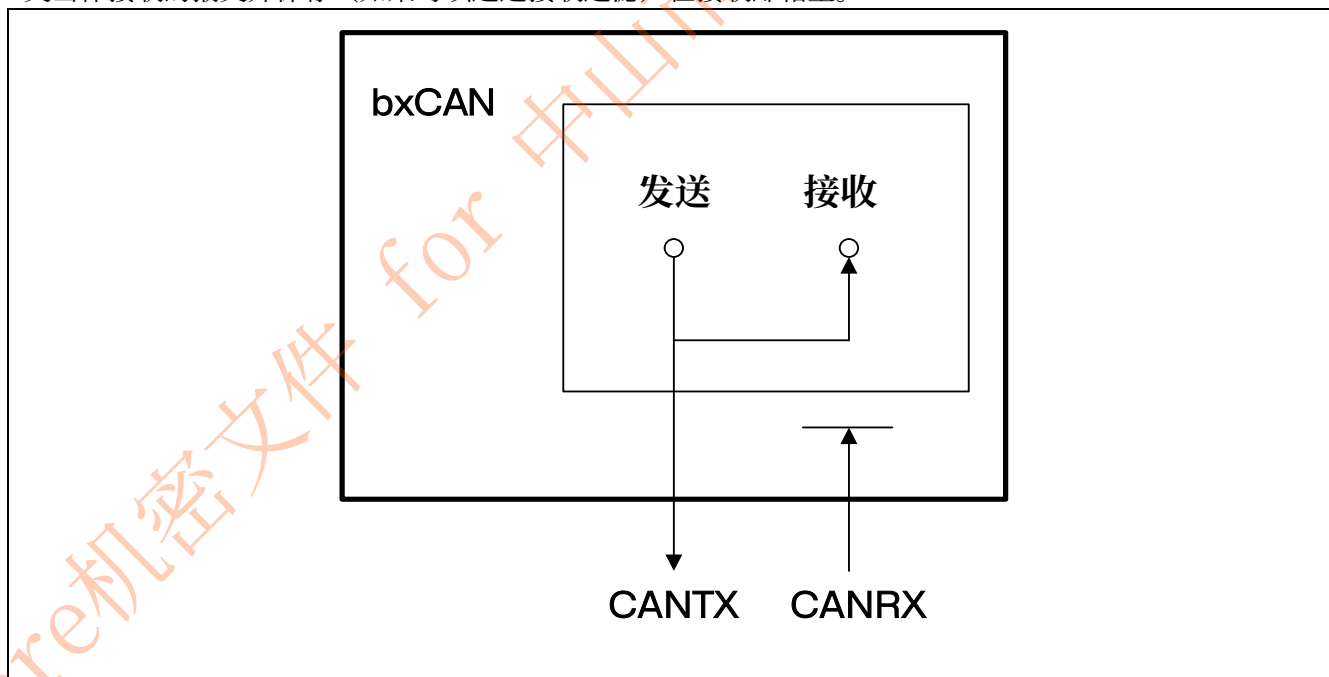


图 32.5 bxCAN 工作在环回模式

环回模式可用于自测试。为了避免外部的影响，在环回模式下 CAN 内核忽略确认错误（在数据/远程帧的确认位时刻，不检测是否有显性位）。在环回模式下，bxCAN 在内部把 Tx 输出回馈到 Rx 输入上，而完全忽略 CANRX 引脚的实际状态。发送的报文可以在 CANTX 引脚上检测到。

### 32.5.3 环回静默模式

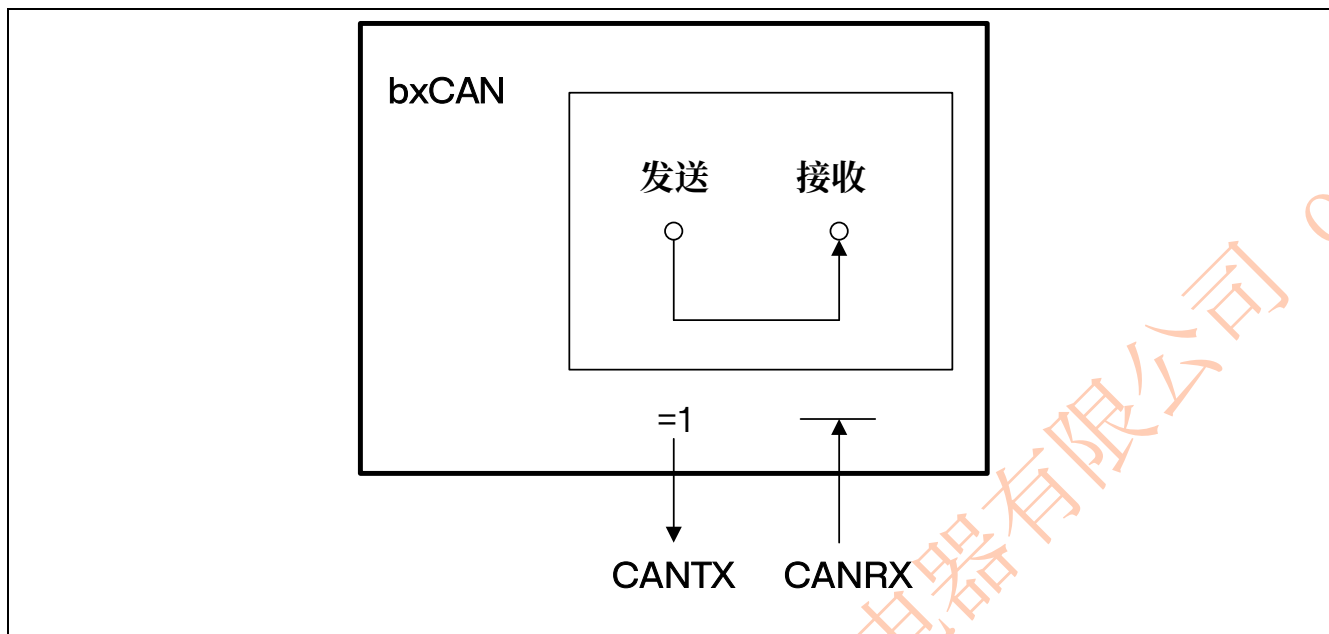


图 32.6 bxCAN 工作在环回静默模式

通过对 CAN\_BTR 寄存器的 LBKM 和 SILM 位同时置 ‘1’，可以选择环回静默模式。该模式可用于“热自测试”，即可以像环回模式那样测试 bxCAN，但却不会影响 CANTX 和 CANRX 所连接的整个 CAN 系统。在环回静默模式下，CANRX 引脚与 CAN 总线断开，同时 CANTX 引脚被驱动到隐性位状态。

### 32.6 处于调试模式时

当微控制器处于调试模式时，Cortex-M4 核心处于暂停状态，依据下述配置位的状态，bxCAN 可以继续正常工作或停止工作：

- 调试(DBG)模块中 CAN1 的 DBG\_CAN1\_STOP 位或 CAN2 的 DBG\_CAN2\_STOP 位。
- CAN\_MCR 中的 DBF 位。

### 32.7 bxCAN 功能描述

#### 32.7.1 发送处理

发送报文的流程为：应用程序选择 1 个空置的发送邮箱；设置标识符，数据长度和待发送数据；然后对 CAN\_TiXR 寄存器的 TXRQ 位置 ‘1’，来请求发送。TXRQ 位置 ‘1’ 后，邮箱就不再是空邮箱；而一旦邮箱不再为空置，软件对邮箱寄存器就不再有写的权限。TXRQ 位置 1 后，邮箱马上进入挂号状态，并等待成为最高优先级的邮箱，参见发送优先级。一旦邮箱成为最高优先级的邮箱，其状态就变为预定发送状态。一旦 CAN 总线进入空闲状态，预定发送邮箱中的报文就马上被发送(进入发送状态)。一旦邮箱中的报文被成功发送后，它马上变为空置邮箱；硬件相应地对 CAN\_TSR 寄存器的 RQCP 和 TXOK 位置 1，来表明一次成功发送。

如果发送失败，由于仲裁引起的就对 CAN\_TSR 寄存器的 ALST 位置 ‘1’，由于发送错误引起的就对 TERR 位置 ‘1’。

#### 发送优先级

### 由标识符决定

当有超过 1 个发送邮箱在挂号时，发送顺序由邮箱中报文的标识符决定。根据 CAN 协议，标识符数值最低的报文具有最高的优先级。如果标识符的值相等，那么邮箱号小的报文先被发送。

### 由发送请求次序决定

通过对 CAN\_MCR 寄存器的 TXFP 位置 ‘1’，可以把发送邮箱配置为发送 FIFO。在该模式下，发送的优先级由发送请求次序决定。

该模式对分段发送很有用。

### 中止

通过对 CAN\_TSR 寄存器的 ABRQ 位置 ‘1’，可以中止发送请求。邮箱如果处于挂号或预定状态，发送请求马上就被中止了。如果邮箱处于发送状态，那么中止请求可能导致 2 种结果。如果邮箱中的报文被成功发送，那么邮箱变为空置邮箱，并且 CAN\_TSR 寄存器的 TXOK 位被硬件置 ‘1’。如果邮箱中的报文发送失败了，那么邮箱变为预定状态，然后发送请求被中止，邮箱变为空置邮箱且 TXOK 位被硬件清 ‘0’。因此如果邮箱处于发送状态，那么在发送操作结束后，邮箱都会变为空置邮箱。

### 禁止自动重传模式

该模式主要用于满足 CAN 标准中，时间触发通信选项的需求。通过对 CAN\_MCR 寄存器的 NART 位置 ‘1’，来让硬件工作在该模式。

在该模式下，发送操作只会执行一次。如果发送操作失败了，不管是由于仲裁丢失或出错，硬件都不会再自动发送该报文。

在一次发送操作结束后，硬件认为发送请求已经完成，从而对 CAN\_TSR 寄存器的 RQCP 位置 ‘1’，同时发送的结果反映在 TXOK、ALST 和 TERR 位上。

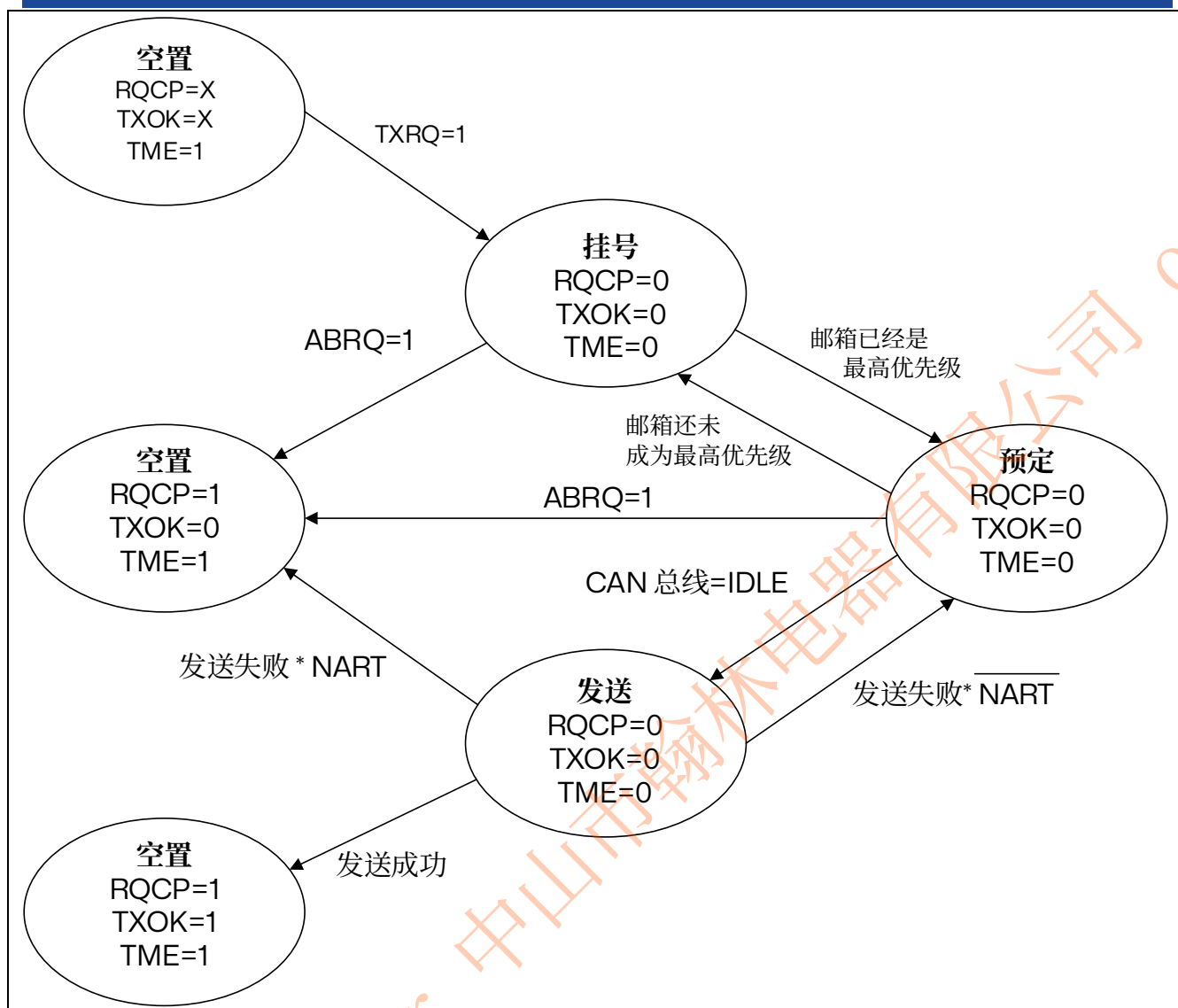


图 32.7 发送邮箱状态

### 32.7.2 时间触发通信模式

在该模式下，CAN 硬件的内部定时器被激活，并且被用于产生（发送与接收邮箱的）时间戳，分别存储在 CAN\_RDTxR/CAN\_TDTxR 寄存器中。内部定时器在每个 CAN 位时间累加。内部定时器在接收和发送的帧起始位的采样点位置被采样，并生成时间戳。

### 32.7.3 接收管理

接收到的报文，被存储在 3 级邮箱深度的 FIFO 中。FIFO 完全由硬件来管理，从而节省了 CPU 的处理负荷，简化了软件并保证了数据的一致性。应用程序只能通过读取 FIFO 输出邮箱，来读取 FIFO 中最先收到的报文。

#### 有效报文

根据 CAN 协议，当报文被正确接收(直到 EOF 域的最后一位都没有错误)，且通过了标识符过滤，那么该报文被认为是有效报文。



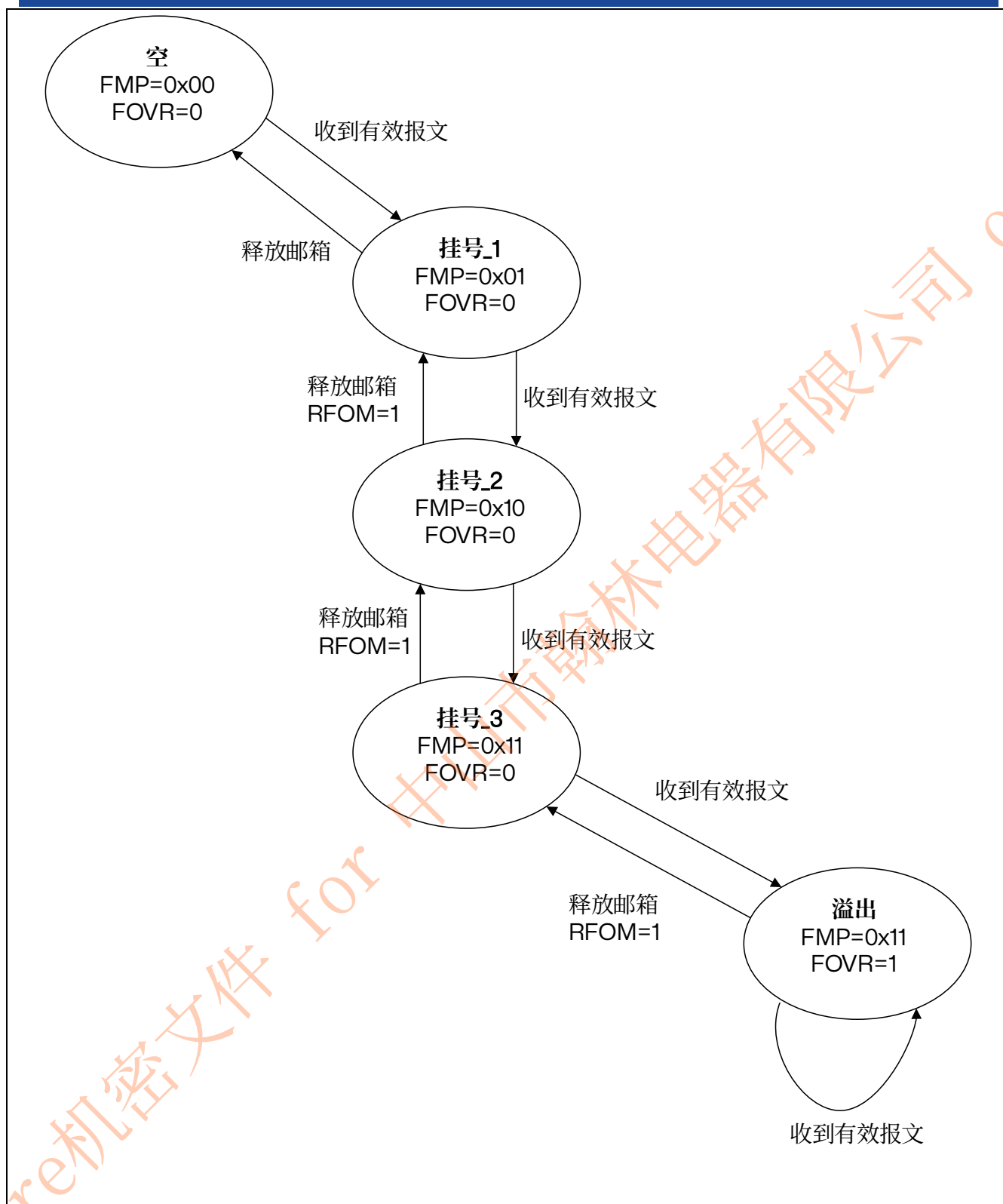


图 32.8 接收 FIFO 状态

### FIFO 管理

FIFO 从空状态开始，在接收到第一个有效的报文后，FIFO 状态变为挂号\_1 (pending\_1)，硬件相应地把 CAN\_RFR 寄存器的 FMP[1:0] 设置为 '01' (二进制 01b)。软件可以读取 FIFO 输出邮箱来读出邮箱

中的报文，然后通过对 CAN\_RFR 寄存器的 RFOM 位设置‘1’来释放邮箱，这样 FIFO 又变为空状态了。如果在释放邮箱的同时，又收到了一个有效的报文，那么 FIFO 仍然保留在挂号 \_1 状态，软件可以读取 FIFO 输出邮箱来读出新收到的报文。

如果应用程序不释放邮箱，在接收到下一个有效的报文后，FIFO 状态变为挂号\_2 (pending\_2)，硬件相应地把 FMP[1:0] 设置为 ‘10’ (二进制 10b)。重复上面的过程，第三个有效的报文把 FIFO 变为挂号 \_3 状态 (FMP[1:0] = 11b)。此时，软件必须对 RFOM 位设置 1 来释放邮箱，以便 FIFO 可以有空间来存放下一个有效的报文；否则，下一个有效的报文到来时就会导致一个报文的丢失

### 溢出

当 FIFO 处于挂号\_3 状态 (即 FIFO 的 3 个邮箱都是满的)，下一个有效的报文就会导致溢出，并且一个报文会丢失。此时，硬件对 CAN\_RFR 寄存器的 FOVR 位进行置‘1’来表明溢出情况。至于哪个报文会被丢弃，取决于对 FIFO 的设置：

- 如果禁用了 FIFO 锁定功能 (CAN\_MCR 寄存器的 RFLM 位被清 ‘0’)，那么 FIFO 中最后收到的报文就被新报文所覆盖。这样，最新收到的报文不会被丢弃。
- 如果使能了 FIFO 锁定功能 (CAN\_MCR 寄存器的 RFLM 位被置 ‘1’)，那么新收到的报文就被丢弃，软件可以读到 FIFO 中最早收到的 3 个报文。

### 接收相关的中断

一旦往 FIFO 存入一个报文，硬件就会更新 FMP[1:0] 位，并且如果 CAN\_IER 寄存器的 FMPIE 位为 ‘1’，那么就会产生一个中断请求。

当 FIFO 变满时 (即第 3 个报文被存入)，CAN\_RFR 寄存器的 FULL 位就被置 ‘1’，并且如果 CAN\_IER 寄存器的 FFIE 位为 ‘1’，那么就会产生一个满中断请求。

在溢出的情况下，FOVR 位被置 ‘1’，并且如果 CAN\_IER 寄存器的 FOVIE 位为 ‘1’，那么就会产生一个溢出中断请求。

## 32.7.4 标识符过滤

在 CAN 协议里，报文的标识符不代表节点的地址，而是跟报文的内容相关的。因此，发送者广播的形式把报文发送给所有的接收者。节点在接收报文时根据标识符的值决定软件是否需要该报文；如果需要，就拷贝到 SRAM 里；如果不需要，报文就被丢弃且无需软件的干预。

为满足这一需求，bxCAN 控制器为应用程序提供了 14 个位宽可变的、可配置的过滤器组 (13~0)，以便只接收那些软件需要的报文。硬件过滤的做法节省了 CPU 开销，否则就必须由软件过滤从而占用一定的 CPU 开销。每个过滤器组 x 由 2 个 32 位寄存器，CAN\_FxR1 和 CAN\_FxR2 组成。

### 可变的位宽

每个过滤器组的位宽都可以独立配置，以满足应用程序的不同需求。根据位宽的不同，每个过滤器组可提供：

- 1 个 32 位过滤器，包括：STID[10:0]、EXID[17:0]、IDE 和 RTR 位
- 2 个 16 位过滤器，包括：STID[10:0]、IDE、RTR 和 EXID[17:15] 位

此外过滤器可配置为，屏蔽位模式和标识符列表模式。

### 屏蔽位模式

在屏蔽位模式下，标识符寄存器和屏蔽寄存器一起，指定报文标识符的任何一位，应该按照“必须匹配”或“不用关心”处理。

### 标识符列表模式

在标识符列表模式下，屏蔽寄存器也被当作标识符寄存器用。因此，不是采用一个标识符加一个屏蔽位的方式，而是使用 2 个标识符寄存器。接收报文标识符的每一位都必须跟过滤器标识符相同。

## 过滤器组位宽和模式的设置

过滤器组可以通过相应的 CAN\_FMR 寄存器配置。在配置一个过滤器组前，必须通过清除 CAN\_FAP 寄存器的 FACT 位，把它设置为禁用状态。通过设置 CAN\_FS1R 的相应 FSCx 位，可以配置一个过滤器组的位宽。通过 CAN\_FMR 的 FBMx 位，可以配置对应的屏蔽/标识符寄存器的标识符列表模式或屏蔽位模式。

为了过滤出一组标识符，应该设置过滤器组工作在屏蔽位模式。

为了过滤出一个标识符，应该设置过滤器组工作在标识符列表模式。

应用程序不用的过滤器组，应该保持在禁用状态。

过滤器组中的每个过滤器，都被编号为（叫做过滤器号）从 0 开始，到某个最大数值—取决于过滤器组的模式和位宽的设置。

关于过滤器配置，参见下图。

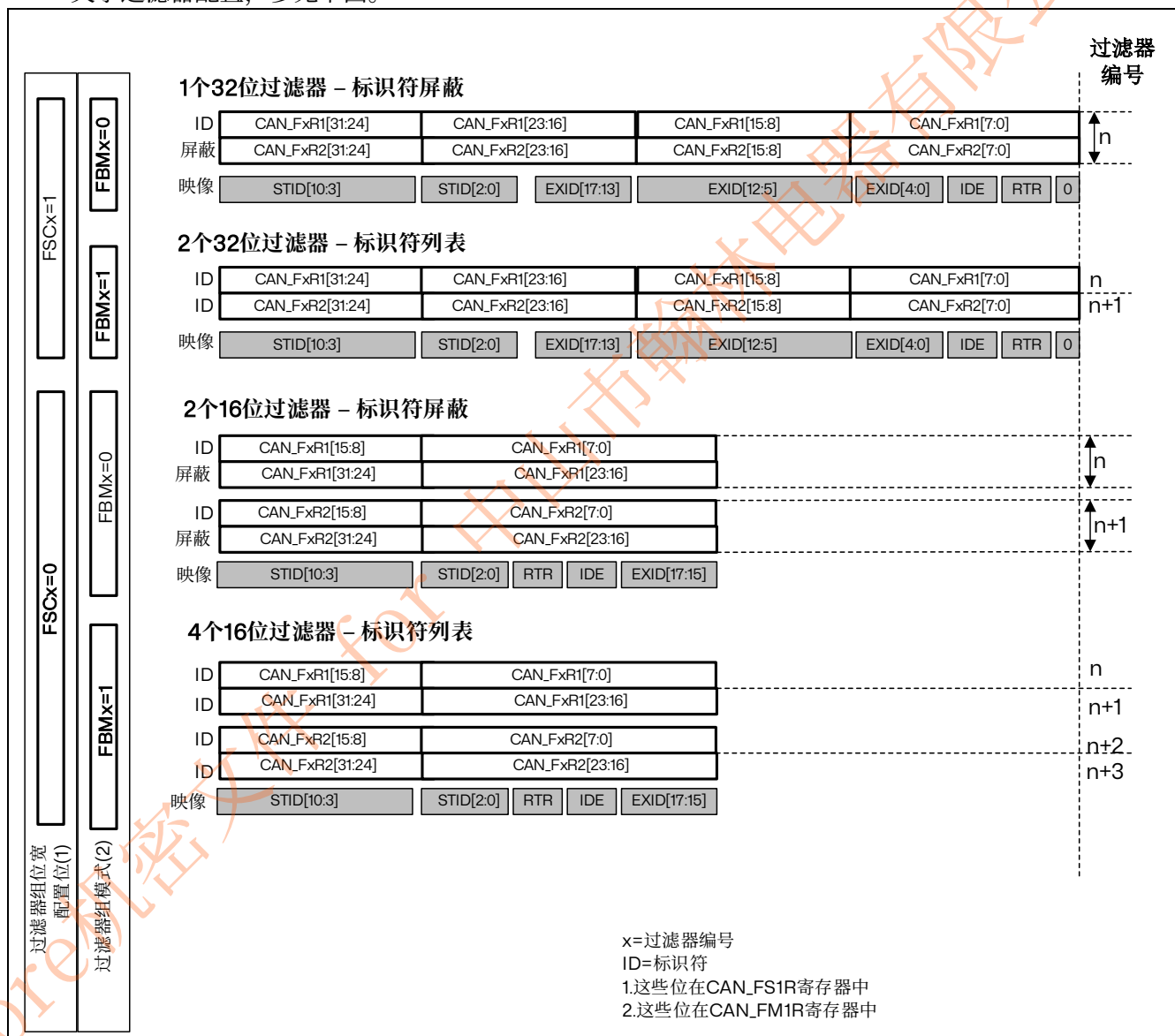


图 32.9 过滤器组位宽设置—寄存器组织

过滤器匹配序号

一旦收到的报文被存入 FIFO，就可被应用程序访问。通常情况下，报文中的数据被拷贝到 SRAM

中；为了把数据拷贝到合适的位置，应用程序需要根据报文的标识符来辨别不同的数据。bxCAN 提供了过滤器匹配序号，以简化这一辨别过程。

根据过滤器优先级规则，过滤器匹配序号和报文一起，被存入邮箱中。因此每个收到的报文，都有与它相关联的过滤器匹配序号。

过滤器匹配序号可以通过下面两种方式来实现：

- 把过滤器匹配序号跟一系列所期望的值进行比较
- 把过滤器匹配序号当作一个索引来访问目标地址

对于标识符列表模式下的过滤器（非屏蔽方式的过滤器），软件不需要直接跟标识符进行比较。

对于屏蔽位模式下的过滤器，软件只须对需要的那些屏蔽位（必须匹配的位）进行比较即可。

在给过滤器编号时，并不考虑过滤器组是否为激活状态。另外，每个 FIFO 各自对其关联的过滤器进行编号。请参考下图的例子。

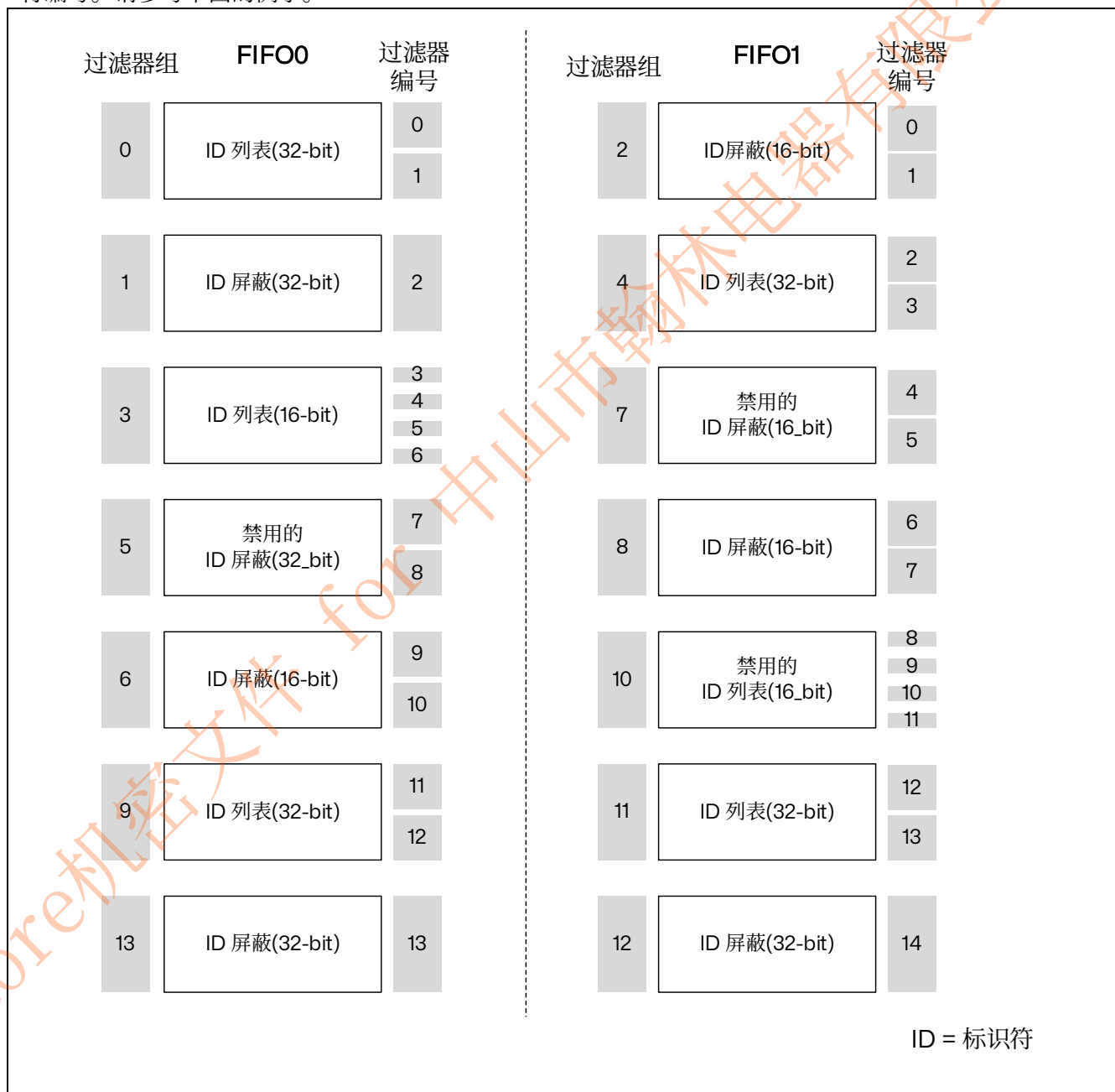


图 32.10 过滤器编号的例子

## 过滤器优先级规则

根据过滤器的不同配置，有可能一个报文标识符能通过多个过滤器的过滤；在这种情况下，存放在接收邮箱中的过滤器匹配序号，根据下列优先级规则来确定：

- 位宽为 32 位的过滤器，优先级高于位宽为 16 位的过滤器
- 对于位宽相同的过滤器，标识符列表模式的优先级高于屏蔽位模式
- 位宽和模式都相同的过滤器，优先级由过滤器号决定，过滤器号小的优先级高

例子：三个过滤器组处于标识符列表模式  
其他的过滤器组处于标识符屏蔽模式

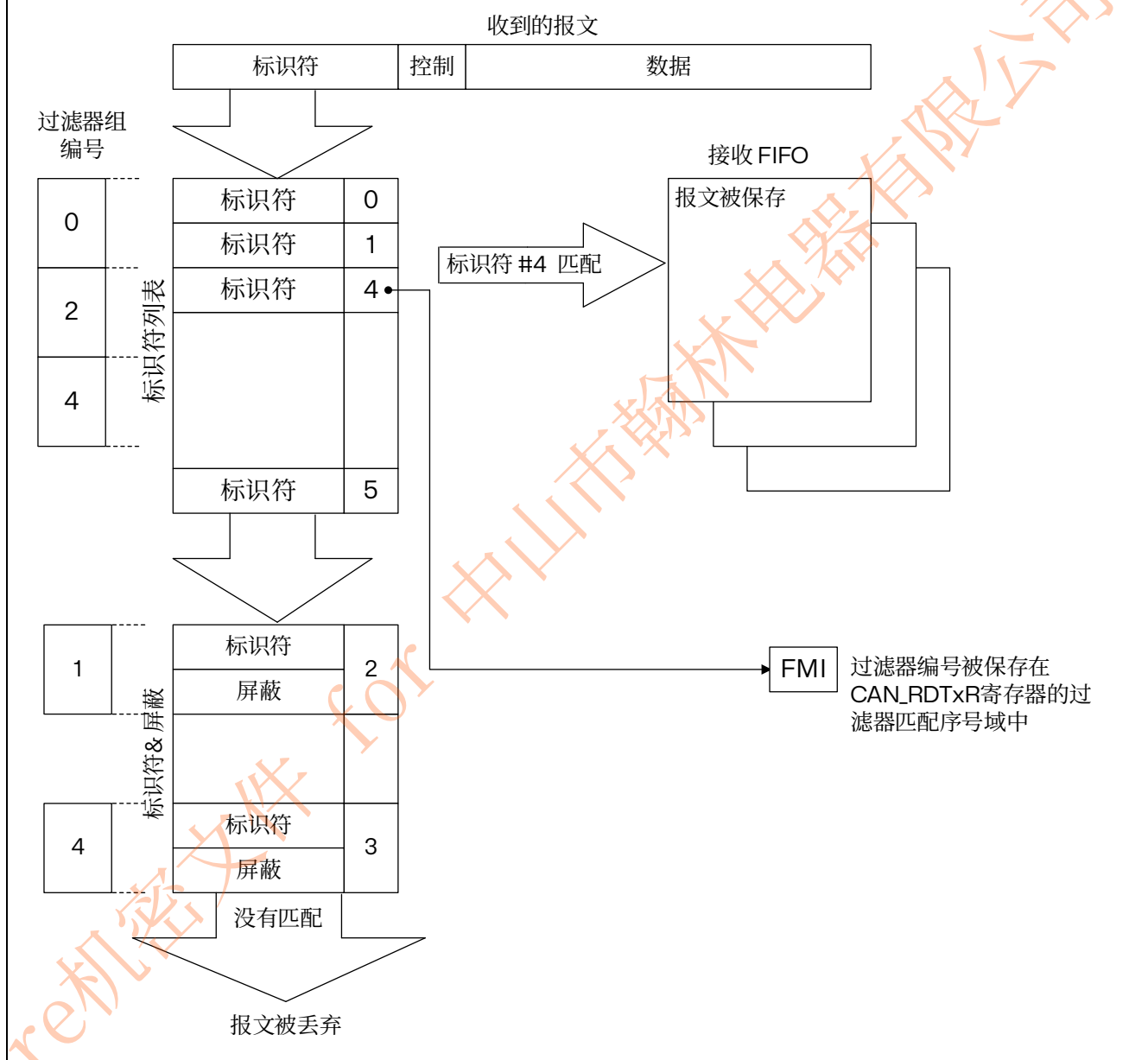


图 32.11 过滤器机制的例子

上面的例子说明了 bxCAN 的过滤器规则：在接收一个报文时，其标识符首先与配置在标识符列表模式下的过滤器相比较；如果匹配上，报文就被存放到相关联的 FIFO 中，并且所匹配的过滤器的序号被存入过滤器匹配序号中。如同例子中所显示，报文标识符跟#4 标识符匹配，因此文内容和 FMI4 被存入 FIFO。

如果没有匹配，报文标识符接着与配置在屏蔽位模式下的过滤器进行比较。

如果报文标识符没有跟过滤器中的任何标识符相匹配，那么硬件就丢弃该报文，且不会对软件有任何干扰。

### 32.7.5 报文存储

邮箱是软件和硬件之间传递报文的接口。邮箱包含了所有跟报文有关的信息：标识符、数据、控制、状态和时间戳信息。

#### 发送邮箱

软件需要在一个空的发送邮箱中，把待发送报文的各种信息设置好（然后再发出发送的请求）。发送的状态可通过查询 CAN\_TSR 寄存器获知。

表 32.1 发送邮箱寄存器列表

| 相对发送邮箱基地址的偏移量 | 寄存器名      |
|---------------|-----------|
| 0             | CAN_TlRxR |
| 4             | CAN_TDTxR |
| 8             | CAN_TDLxR |
| 12            | CAN_TDHxR |

#### 接收邮箱(FIFO)

在接收到一个报文后，软件就可以访问接收 FIFO 的输出邮箱来读取它。一旦软件处理了报文（如把它读出来），软件就应该对 CAN\_RfRxR 寄存器的 RFOM 位进行置 ‘1’，来释放该报文，以便为后面收到的报文留出存储空间。过滤器匹配序号存放在 CAN\_RDTxR 寄存器的 FMI 域中。16 位的时间戳存放在 CAN\_RDTxR 寄存器的 TIME[15:0]域中。

表 32.2 接收邮箱寄存器列表

| 相对接收邮箱基地址的偏移量 | 寄存器名      |
|---------------|-----------|
| 0             | CAN_RlRxR |
| 4             | CAN_RDTxR |
| 8             | CAN_RDLxR |
| 12            | CAN_RDHxR |



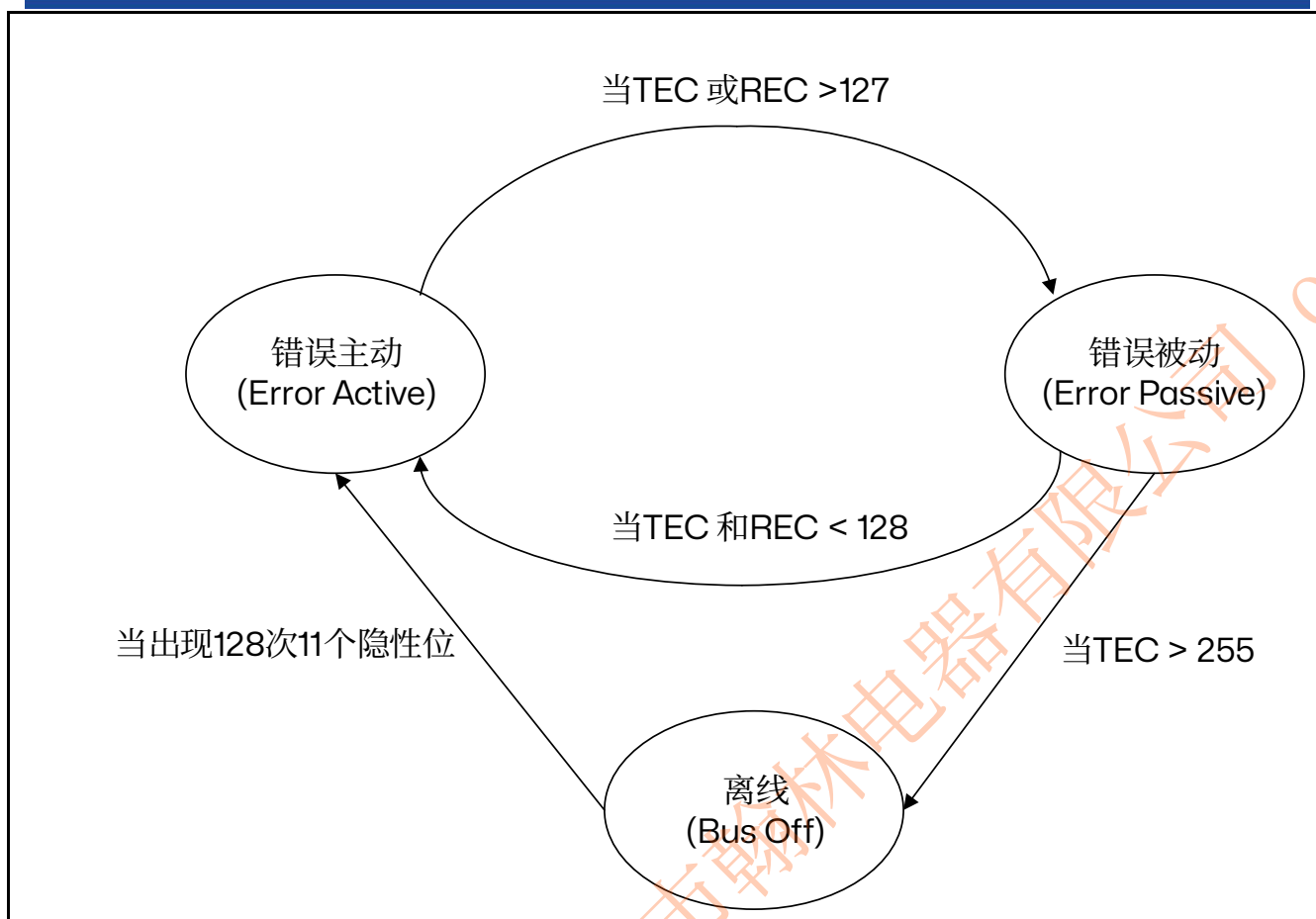


图 32.12 CAN 错误状态图

### 32.7.6 出错管理

CAN 协议描述的出错管理，完全由硬件通过发送错误计数器（CAN\_ESR 寄存器里的 TEC 域），和接收错误计数器（CAN\_ESR 寄存器里的 REC 域）来实现，其值根据错误的情况而增加或减少。关于 TEC 和 REC 管理的详细信息，请参考 CAN 标准。

软件可以读出它们的值来判断 CAN 网络的稳定性。

此外，CAN\_ESR 寄存器提供了当前错误状态的详细信息。通过设置 CAN\_IER 寄存器（比如 ERRIE 位），当检测到出错时软件可以灵活地控制中断的产生。

#### 离线恢复

当 TEC 大于 255 时，bxCAN 就进入离线状态，同时 CAN\_ESR 寄存器的 BOFF 位被置 ‘1’。在离线状态下，bxCAN 无法接收和发送报文。

根据 CAN\_MCR 寄存器中 ABOM 位的设置，bxCAN 可以自动或在软件的请求下，从离线状态恢复（变为错误主动状态）。在这两种情况下，bxCAN 都必须等待一个 CAN 标准所描述的恢复过程（CAN RX 引脚上检测到 128 次 11 个连续的隐性位）。

如果 ABOM 位为 ‘1’，bxCAN 进入离线状态后，就自动开启恢复过程。

如果 ABOM 位为 ‘0’，软件必须先请求 bxCAN 进入然后再退出初始化模式，随后恢复过程才被开启。

注：在初始化模式下，bxCAN 不会监视 CAN RX 引脚的状态，这样就不能完成恢复过程。为了完成恢复过程，bxCAN 必须工作在正常模式。



### 32.7.7 位时间特性

位时间特性逻辑通过采样来监视串行的 CAN 总线，并且通过与帧起始位的边沿进行同步，及通过与后面的边沿进行重新同步，来调整其采样点。

它的操作可以简单解释为，如下所述把名义上的每位时间分为 3 段：

- **同步段 (SYNC\_SEG)：**通常期望位的变化发生在该时间段内。其值固定为 1 个时间单元 ( $1 \times t_{CAN}$ )。
- **时间段 (BS1)：**定义采样点的位置。它包含 CAN 标准里的 PROP\_SEG 和 PHASE\_SEG1。其值可以编程为 1 到 16 个时间单元，但也可以被自动延长，以补偿因为网络中不同节点的频率差异所造成的相位的正向漂移。
- **时间段 2 (BS2)：**定义发送点的位置。它代表 CAN 标准里的 PHASE\_SEG2。其值可以编程为 1 到 8 个时间单元，但也可以被自动缩短以补偿相位的负向漂移。

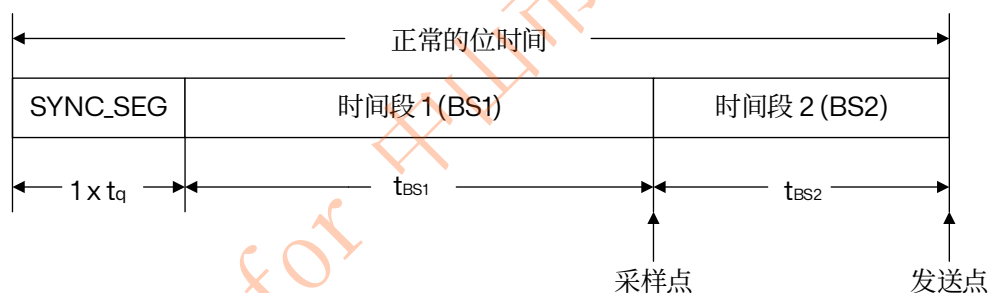
重新同步跳跃宽度 (SJW) 定义了，在每位中可以延长或缩短多少个时间单元的上限。其值可以编程为 1 到 4 个时间单元。

有效跳变被定义为，当 bxCAN 自己没有发送隐性位时，从显性位到隐性位的第 1 次转变。

如果在时间段 1 (BS1) 而不是在同步段 (SYNC\_SEG) 检测到有效跳变，那么 BS1 的时间就被延长最多 SJW 那么长，从而采样点被延迟了。

相反如果在时间段 2 (BS2) 而不是在 SYNC\_SEG 检测到有效跳变，那么 BS2 的时间就被缩短最多 SJW 那么长，从而采样点被提前了。

为了避免软件的编程错误，对位时间特性寄存器 (CAN\_BTR) 的设置，只能在 bxCAN 处于初始化状态下进行。



$$\text{波特率} = \frac{1}{\text{正常的位时间}}$$

$$\text{正常的位时间} = 1 \times t_q + t_{BS1} + t_{BS2}$$

其中：

$$t_{BS1} = t_q \times (TS1[3:0] + 1)$$

$$t_{BS2} = t_q \times (TS2[2:0] + 1)$$

$$t_q = (BRP[9:0] + 1) \times t_{CANCLK}$$

这里  $t_q$  表示 1 个时间单元

$$t_{CANCLK} = \text{CAN 时钟的时间周期}$$

BRP[9:0], TS1[3:0] 和 TS2[2:0] 在 CAN\_BTR 寄存器中定义

图 32.13 位时序

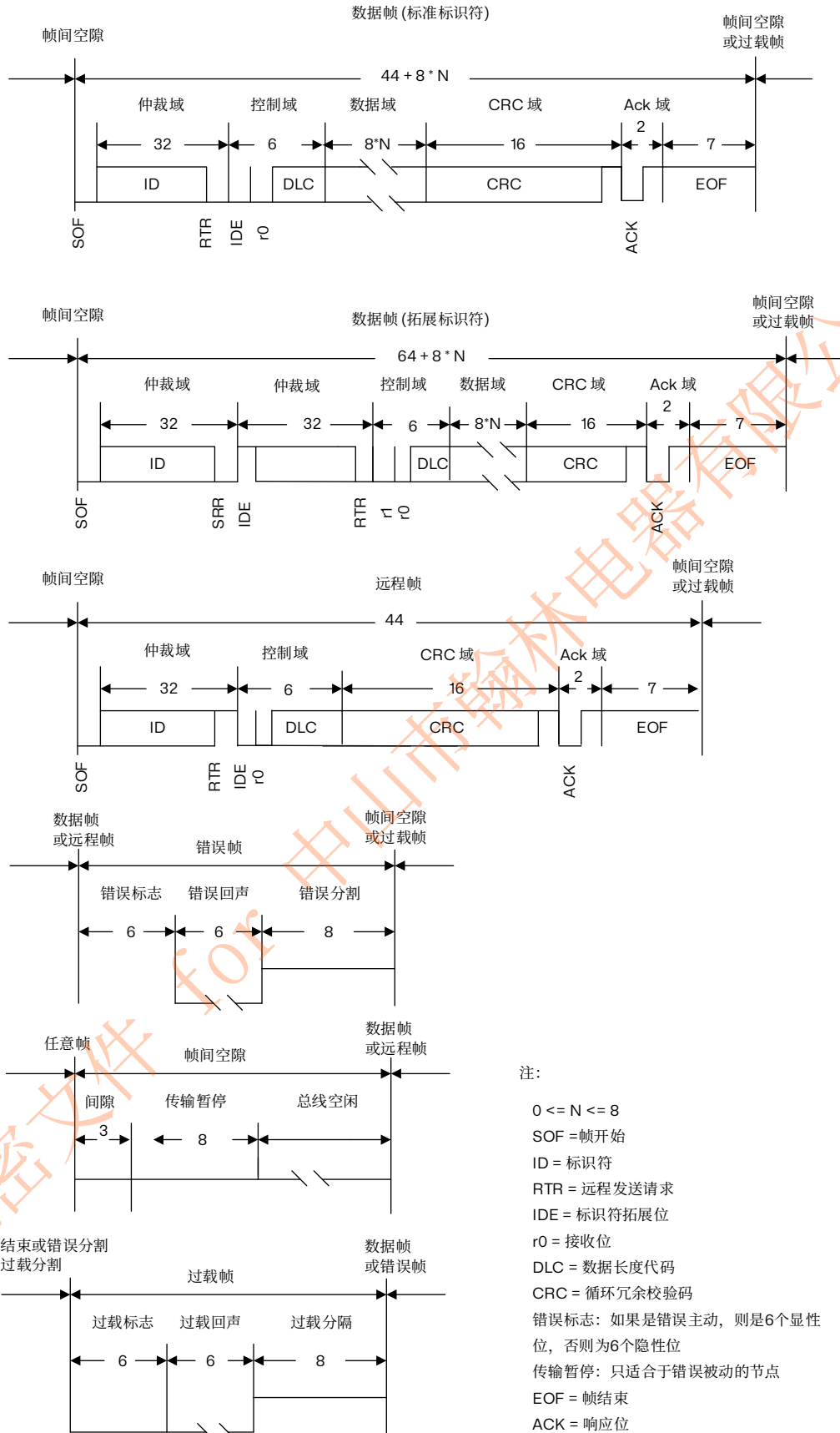


图 32.14 各种 CAN

## 32.8 bxCAN 中断

bxCAN 占用 4 个专用的中断向量。通过设置 CAN 中断允许寄存器 (CAN\_IER)，每个中断源都可以单独允许和禁用。

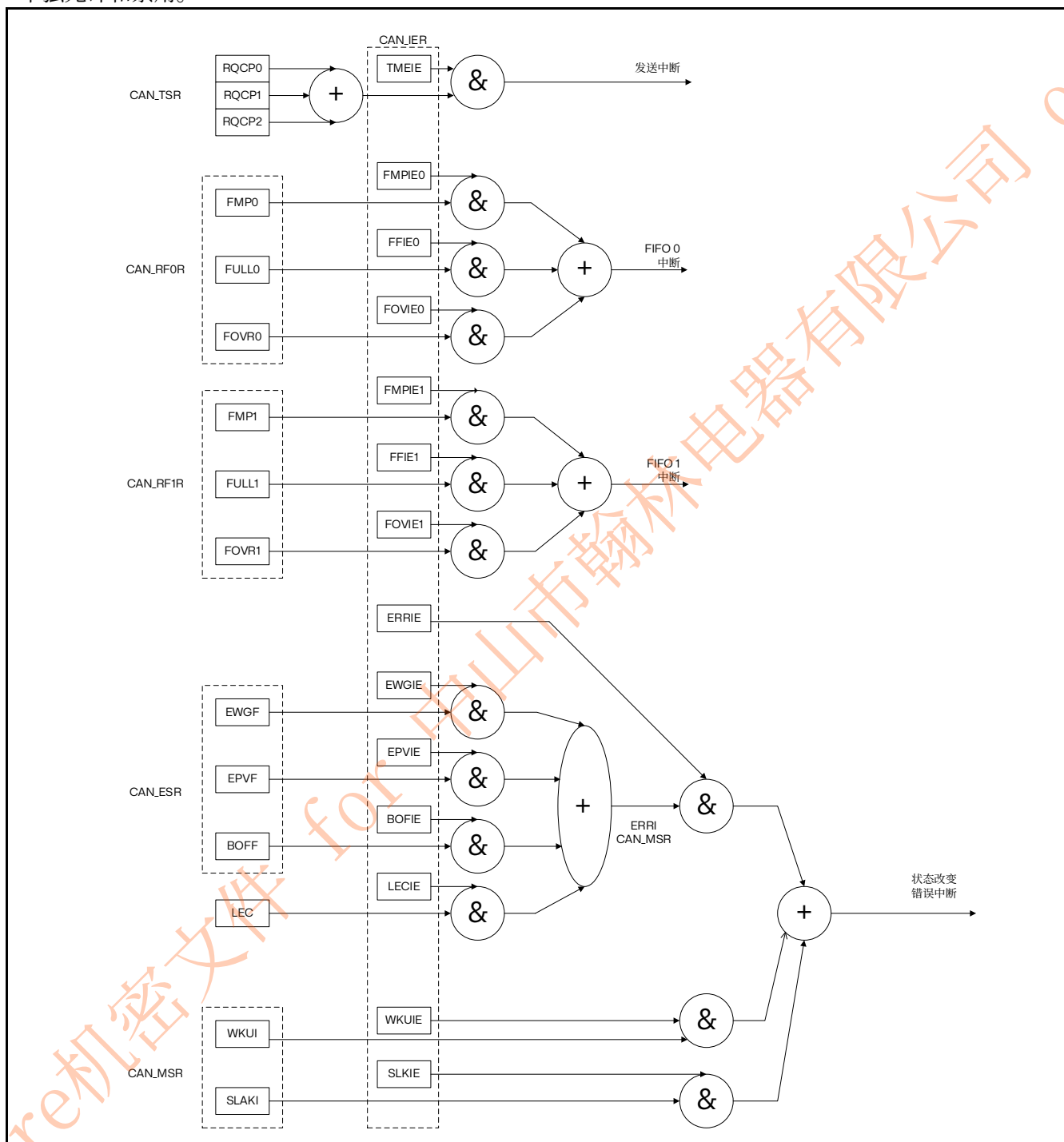


图 32.15 事件标志和中断产生

- 发送中断可由下列事件产生：
  - 发送邮箱 0 变为空，CAN\_TSR 寄存器的 RQCP0 位被置 ‘1’。
  - 发送邮箱 1 变为空，CAN\_TSR 寄存器的 RQCP1 位被置 ‘1’。
  - 发送邮箱 2 变为空，CAN\_TSR 寄存器的 RQCP2 位被置 ‘1’。

- FIFO0 中断可由下列事件产生：
  - FIFO0 接收到一个新报文，CAN\_RF0R 寄存器的 FMP0 位不再是 ‘00’。
  - FIFO0 变为满的情况，CAN\_RF0R 寄存器的 FULL0 位被置 ‘1’。
  - FIFO0 发生溢出的情况，CAN\_RF0R 寄存器的 FOVR0 位被置 ‘1’。
- FIFO1 中断可由下列事件产生：
  - FIFO1 接收到一个新报文，CAN\_RF1R 寄存器的 FMP1 位不再是 ‘00’。
  - FIFO1 变为满的情况，CAN\_RF1R 寄存器的 FULL1 位被置 ‘1’。
  - FIFO1 发生溢出的情况，CAN\_RF1R 寄存器的 FOVR1 位被置 ‘1’。
- 错误和状态变化中断可由下列事件产生：
  - 出错情况，关于出错情况的详细信息请参考 CAN 错误状态寄存器 (CAN\_ESR)。
  - 唤醒情况，在 CAN 接收引脚上监视到帧起始位 (SOF)。
  - CAN 进入睡眠模式。

### 32.9 寄存器访问保护

对某些寄存器的错误访问会导致一个 CAN 节点对整个 CAN 网络的暂时性干扰。因此，软件只能在 CAN 处于初始化模式时修改 CAN\_BTR 寄存器。

虽然错误数据的发送对 CAN 网的网络层不会带来问题，但却会对应用程序造成严重影响。因此，软件只能在发送邮箱为空的状态改变它。

过滤器的数值只能在关闭对应过滤器组的状态下，或设置 FINIT 位为 ‘1’ 后才能修改。此外，只有在设置整个过滤器为初始化模式下（即 FINIT=1），才能修改过滤器的设置，即修改 CAN\_FMR，CAN\_FSxR 和 CAN\_FFAR 寄存器。

## 32.10 CAN 控制和状态寄存器描述

### 32.10.1 CAN 主控制寄存器 (CAN\_MCR)

地址偏移量: 0x00

复位值: 0x0001 0002

|          |          |    |    |    |    |    |    |      |      |      |      |      |      |       |      |
|----------|----------|----|----|----|----|----|----|------|------|------|------|------|------|-------|------|
| 31       | 30       | 29 | 28 | 27 | 26 | 25 | 24 | 23   | 22   | 21   | 20   | 19   | 18   | 17    | 16   |
| Reserved |          |    |    |    |    |    |    |      |      |      |      |      |      |       | DBF  |
|          |          |    |    |    |    |    |    |      |      |      |      |      |      |       | rw   |
| 15       | 14       | 13 | 12 | 11 | 10 | 9  | 8  | 7    | 6    | 5    | 4    | 3    | 2    | 1     | 0    |
| RESET    | Reserved |    |    |    |    |    |    | TTCM | ABOM | AWUM | NART | RFLM | TXFP | SLEEP | INRQ |
| rs       |          |    |    |    |    |    |    | rw   | rw   | rw   | rw   | rw   | rw   | rw    | rw   |

Bits 31:17 保留, 必须保持为复位值。

Bit 16 **DBF**: 调试冻结 (Debug freeze)

0: 在调试时, CAN 照常工作

1: 在调试时, 冻结 CAN 的接收/发送。仍然可以正常地读写和控制接收 FIFO。

Bit 15 **RESET**: bxCAN 软件复位 (bxCAN software master reset)

0: 本外设正常工作;

1: 对 bxCAN 进行强行复位, 复位后 bxCAN 进入睡眠模式 (FMP 位和 CAN\_MCR 寄存器被初始化为其复位值)。此后硬件自动对该位清 '0'。

Bits 14:8 保留, 必须保持为复位值。

Bit 7 **TTCM**: 时间触发通信模式 (Time triggered communication mode)

0: 禁止时间触发通信模式;

1: 允许时间触发通信模式。

Bit 6 **ABOM**: 自动离线 (Bus-Off) 管理 (Automatic bus-off management)

该位决定 CAN 硬件在什么条件下可以退出离线状态。

0: 离线状态的退出过程是, 软件对 CAN\_MCR 寄存器的 INRQ 位进行置 '1' 随后清 '0' 后, 一旦硬件检测到 128 次 11 位连续的隐性位, 则退出离线状态;

1: 一旦硬件检测到 128 次 11 位连续的隐性位, 则自动退出离线状态。

Bit 5 **AWUM**: 自动唤醒模式 (Automatic wakeup mode)

该位决定 CAN 处在睡眠模式时由硬件还是软件唤醒

0: 睡眠模式通过清除 CAN\_MCR 寄存器的 SLEEP 位, 由软件唤醒;

1: 睡眠模式通过检测 CAN 报文, 由硬件自动唤醒。唤醒的同时, 硬件自动对 CAN\_MSR 寄存器的 SLEEP 和 SLAK 位清 '0'。

Bit 4 **NART**: 禁止报文自动重传 (No automatic retransmission)

0: 按照 CAN 标准, CAN 硬件在发送报文失败时会一直自动重传直到发送成功;

1: CAN 报文只被发送 1 次, 不管发送的结果如何 (成功、出错或仲裁丢失)。

Bit 3 **RFLM**: 接收 FIFO 锁定模式 (Receive FIFO locked mode)

0: 在接收溢出时 FIFO 未被锁定, 当接收 FIFO 的报文未被读出, 下一个收到的报文会覆盖原有的报文;

1: 在接收溢出时 FIFO 被锁定, 当接收 FIFO 的报文未被读出, 下一个收到的报文会被丢弃。

Bit 2 **TXFP**: 发送 FIFO 优先级 (Transmit FIFO priority)

当有多个报文同时在等待发送时, 该位决定这些报文的发送顺序

- 0: 优先级由报文的标识符来决定;  
1: 优先级由发送请求的顺序来决定。

#### Bit 1 SLEEP: 睡眠模式请求 (Sleep mode request)

软件对该位置 ‘1’ 可以请求 CAN 进入睡眠模式, 一旦当前的 CAN 活动 (发送或接收报文) 结束, CAN 就进入睡眠。

软件对该位清 ‘0’ 使 CAN 退出睡眠模式。

当设置了 AWUM 位且在 CAN Rx 信号中检测出 SOF 位时, 硬件对该位清 ‘0’。

在复位后该位被置 ‘1’, 即 CAN 在复位后处于睡眠模式。

#### Bit 0 INRQ: 初始化请求 (Initialization request)

软件对该位清 ‘0’ 可使 CAN 从初始化模式进入正常工作模式: 当 CAN 在接收引脚检测到连续的 11 个隐性位后, CAN 就达到同步, 并为接收和发送数据作好准备了。为此, 硬件相应地对 CAN\_MSR 寄存器的 INAK 位清 ‘0’。

软件对该位置 1 可使 CAN 从正常工作模式进入初始化模式: 一旦当前的 CAN 活动 (发送或接收) 结束, CAN 就进入初始化模式。相应地, 硬件对 CAN\_MSR 寄存器的 INAK 位置 ‘1’。

### 32.10.2 CAN 主状态寄存器 (CAN\_MSR)

地址偏移量: 0x04

复位值: 0x0000 0802

|          |    |    |    |    |      |     |     |          |    |    |       |       |       |      |      |
|----------|----|----|----|----|------|-----|-----|----------|----|----|-------|-------|-------|------|------|
| 31       | 30 | 29 | 28 | 27 | 26   | 25  | 24  | 23       | 22 | 21 | 20    | 19    | 18    | 17   | 16   |
| Reserved |    |    |    |    |      |     |     |          |    |    |       |       |       |      |      |
| 15       | 14 | 13 | 12 | 11 | 10   | 9   | 8   | 7        | 6  | 5  | 4     | 3     | 2     | 1    | 0    |
| Reserved |    |    |    | RX | SAMP | RXM | TXM | Reserved |    |    | SLAKI | WKUI  | ERRI  | SLAK | INAK |
|          |    |    |    | r  | r    | r   | r   |          |    |    | rc_w1 | rc_w1 | rc_w1 | r    | r    |

Bits 31:12 保留, 必须保持为复位值。

#### Bit 11 RX: CAN 接收电平 (CAN Rx signal)

该位反映 CAN 接收引脚 (CAN\_RX) 的实际电平。

#### Bit 10 SAMP: 上次采样值 (Last sample point)

CAN 接收引脚的上次采样值 (对应于当前接收位的值)。

#### Bit 9 RXM: 接收模式 (Receive mode)

该位为 ‘1’ 表示 CAN 当前为接收器。

#### Bit 8 TXM: 发送模式 (Transmit mode)

该位为 ‘1’ 表示 CAN 当前为发送器。

Bits 7:5 保留, 必须保持为复位值。

#### Bit 4 SLAKI: 睡眠确认中断 (Sleep acknowledge interrupt)

当 SLKIE=1, 一旦 CAN 进入睡眠模式硬件就对该位置 ‘1’, 紧接着相应的中断被触发。当设置该位为 ‘1’ 时, 如果设置了 CAN\_IER 寄存器中的 SLKIE 位, 将产生一个状态改变中断。

软件可对该位清 ‘0’, 当 SLAK 位被清 ‘0’ 时硬件也对该位清 ‘0’。

注: 当 SLKIE=0, 不应该查询该位, 而应该查询 SLAK 位来获知睡眠状态。

#### Bit 3 WKUI: 唤醒中断挂号 (Wakeup interrupt)

当 CAN 处于睡眠状态, 一旦检测到帧起始位 (SOF), 硬件就置该位为 ‘1’; 并且如果 CAN\_IER 寄存器的 WKUIE 位为 ‘1’, 则产生一个状态改变中断。



该位由软件清 ‘0’。

Bit 2 **ERRI**: 出错中断挂号 (Error interrupt)

当检测到错误时, CAN\_ESR 寄存器的某位被置 ‘1’, 如果 CAN\_IER 寄存器的相应中断使能位也被置 ‘1’ 时, 则硬件对该位置 ‘1’; 如果 CAN\_IER 寄存器的 ERRIE 位为 ‘1’, 则产生状态改变中断。

该位由软件清 ‘0’。

Bit 1 **SLAK**: 睡眠模式确认

该位由硬件置 ‘1’, 指示软件 CAN 模块正处于睡眠模式。该位是对软件请求进入睡眠模式的确认 (对 CAN\_MCR 寄存器的 SLEEP 位置 ‘1’)。

当 CAN 退出睡眠模式时硬件对该位清 ‘0’ (需要跟 CAN 总线同步)。这里跟 CAN 总线同步是指, 硬件需要在 CAN 的 RX 引脚上检测到连续的 11 位隐性位。

注: 通过软件或硬件对 CAN\_MCR 的 SLEEP 位清 ‘0’, 将启动退出睡眠模式的过程。

有关清除 SLEEP 位的详细信息, 参见 CAN\_MCR 寄存器的 AWUM 位的描述。

Bit 0 **INAK**: 初始化确认

该位由硬件置 ‘1’, 指示软件 CAN 模块正处于初始化模式。该位是对软件请求进入初始化模式的确认 (对 CAN\_MCR 寄存器的 INRQ 位置 ‘1’)。

当 CAN 退出初始化模式时硬件对该位清 ‘0’ (需要跟 CAN 总线同步)。这里跟 CAN 总线同步是指, 硬件需要在 CAN 的 RX 引脚上检测到连续的 11 位隐性位。

### 32.10.3 CAN 发送状态寄存器 (CAN\_TSR)

地址偏移量: 0x08

复位值: 0x1C00 0000

| 31    | 30       | 29   | 28   | 27    | 26    | 25        | 24    | 23    | 22       | 21 | 20 | 19    | 18    | 17    | 16    |
|-------|----------|------|------|-------|-------|-----------|-------|-------|----------|----|----|-------|-------|-------|-------|
| LOW2  | LOW1     | LOW0 | TME2 | TME1  | TME0  | CODE[1:0] |       | ABRQ2 | Reserved |    |    | TERR2 | ALST2 | TXOK2 | RQCP2 |
| r     | r        | r    | r    | r     | r     | r         | r     | rs    |          |    |    | rc_w1 | rc_w1 | rc_w1 | rc_w1 |
| 15    | 14       | 13   | 12   | 11    | 10    | 9         | 8     | 7     | 6        | 5  | 4  | 3     | 2     | 1     | 0     |
| ABRQ1 | Reserved |      |      | TERR1 | ALST1 | TXOK1     | RQCP1 | ABRQ0 | Reserved |    |    | TERR0 | ALST0 | TXOK0 | RQCP0 |
| rs    |          |      |      | rc_w1 | rc_w1 | rc_w1     | rc_w1 | rs    |          |    |    | rc_w1 | rc_w1 | rc_w1 | rc_w1 |

Bit 31 **LOW2**: 邮箱 2 最低优先级标志 (Lowest priority flag for mailbox 2)

当多个邮箱在等待发送报文, 且邮箱 2 的优先级最低时, 硬件对该位置 ‘1’。

Bit 30 **LOW1**: 邮箱 1 最低优先级标志 (Lowest priority flag for mailbox 1)

当多个邮箱在等待发送报文, 且邮箱 1 的优先级最低时, 硬件对该位置 ‘1’。

Bit 29 **LOW0**: 邮箱 0 最低优先级标志 (Lowest priority flag for mailbox 0)

当多个邮箱在等待发送报文, 且邮箱 0 的优先级最低时, 硬件对该位置 ‘1’。

注: 如果只有 1 个邮箱在等待, 则 LOW[2:0]被清 ‘0’。

Bit 28 **TME2**: 发送邮箱 2 空 (Transmit mailbox 2 empty)

当邮箱 2 中没有等待发送的报文时, 硬件对该位置 ‘1’。

Bit 27 **TME1**: 发送邮箱 1 空 (Transmit mailbox 1 empty)

当邮箱 1 中没有等待发送的报文时, 硬件对该位置 ‘1’。

Bit 26 **TME0**: 发送邮箱 0 空 (Transmit mailbox 0 empty)

当邮箱 0 中没有等待发送的报文时, 硬件对该位置 ‘1’。

Bits 25:24 **CODE[1:0]**: 邮箱号 (Mailbox code)

当有至少 1 个发送邮箱为空时, 这 2 位表示下一个空的发送邮箱号。

当所有的发送邮箱都为空时, 这 2 位表示优先级最低的那个发送邮箱号。



Bit 23 **ABRQ2**: 邮箱 2 中止发送 (Abort request for mailbox 2)

软件对该位置 ‘1’, 可以中止邮箱 2 的发送请求, 当邮箱 2 的发送报文被清除时硬件对该位清 ‘0’。

如果邮箱 2 中没有等待发送的报文, 则对该位置 ‘1’ 没有任何效果。

Bits 22:20 保留, 必须保持为复位值。

Bit 19 **TERR2**: 邮箱 2 发送失败 (Transmission error of mailbox 2)

当邮箱 2 因为出错而导致发送失败时, 对该位置 ‘1’。

Bit 18 **ALST2**: 邮箱 2 仲裁丢失 (Arbitration lost for mailbox 2)

当邮箱 2 因为仲裁丢失而导致发送失败时, 对该位置 ‘1’。

Bit 17 **TXOK2**: 邮箱 2 发送成功 (Transmission OK of mailbox 2)

每次在邮箱 2 进行发送尝试后, 硬件对该位进行更新:

0: 上次发送尝试失败;

1: 上次发送尝试成功。

当邮箱 2 的发送请求被成功完成后, 硬件对该位置 ‘1’。

Bit 16 **RQCP2**: 邮箱 2 请求完成 (Request completed mailbox 2)

当上次对邮箱 2 的请求 (发送或中止) 完成后, 硬件对该位置 ‘1’。

软件对该位写 ‘1’ 可以对其清 ‘0’; 当硬件接收到发送请求时也对该位清 ‘0’

(CAN\_TI2R 寄存器的 TXRQ 位被置 ‘1’)。

该位被清 ‘0’ 时, 邮箱 2 的其它发送状态位 (TXOK2, ALST2 和 TERR2) 也被清 ‘0’。

Bit 15 **ABRQ1**: 邮箱 1 中止发送 (Abort request for mailbox 1)

软件对该位置 ‘1’, 可以中止邮箱 1 的发送请求, 当邮箱 1 的发送报文被清除时硬件对该位清 ‘0’。

如果邮箱 1 中没有等待发送的报文, 则对该位置 ‘1’ 没有任何效果。

Bits 14:12 保留, 必须保持为复位值。

Bit 11 **TERR1**: 邮箱 1 发送失败 (Transmission error of mailbox 1)

当邮箱 1 因为出错而导致发送失败时, 对该位置 ‘1’。

Bit 10 **ALST1**: 邮箱 1 仲裁丢失 (Arbitration lost for mailbox 1)

当邮箱 1 因为仲裁丢失而导致发送失败时, 对该位置 ‘1’。

Bit 9 **TXOK1**: 邮箱 1 发送成功 (Transmission OK of mailbox 1)

每次在邮箱 1 进行发送尝试后, 硬件对该位进行更新:

0: 上次发送尝试失败;

1: 上次发送尝试成功。

当邮箱 1 的发送请求被成功完成后, 硬件对该位置 ‘1’。

Bit 8 **RQCP1**: 邮箱 1 请求完成 (Request completed mailbox 1)

当上次对邮箱 1 的请求 (发送或中止) 完成后, 硬件对该位置 ‘1’。

软件对该位写 ‘1’ 可以对其清 ‘0’; 当硬件接收到发送请求时也对该位清 ‘0’

(CAN\_TI1R 寄存器的 TXRQ 位被置 ‘1’)。

该位被清 ‘0’ 时, 邮箱 1 的其它发送状态位 (TXOK1, ALST1 和 TERR1) 也被清 ‘0’。

Bit 7 **ABRQ0**: 邮箱 0 中止发送 (Abort request for mailbox 0)

软件对该位置 ‘1’ 可以中止邮箱 0 的发送请求, 当邮箱 0 的发送报文被清除时硬件对该位清 ‘0’。

如果邮箱 0 中没有等待发送的报文，则对该位置 1 没有任何效果。

Bits 6:4 保留，必须保持为复位值。

Bit 3 **TERR0**: 邮箱 0 发送失败 (Transmission error of mailbox 0)

当邮箱 0 因为出错而导致发送失败时，对该位置 ‘1’。

Bit 2 **ALST0**: 邮箱 0 仲裁丢失 (Arbitration lost for mailbox 0)

当邮箱 0 因为仲裁丢失而导致发送失败时，对该位置 ‘1’。

Bit 1 **TXOK0**: 邮箱 0 发送成功 (Transmission OK of mailbox 0)

每次在邮箱 0 进行发送尝试后，硬件对该位进行更新：

0: 上次发送尝试失败；

1: 上次发送尝试成功。

当邮箱 0 的发送请求被成功完成后，硬件对该位置 ‘1’。

Bit 0 **RQCP1**: 邮箱 0 请求完成 (Request completed mailbox 0)

当上次对邮箱 0 的请求（发送或中止）完成后，硬件对该位置 ‘1’。

软件对该位写 ‘1’ 可以对其清 ‘0’；当硬件接收到发送请求时也对该位清 ‘0’

(CAN\_TIDR 寄存器的 TXRQ 位被置 ‘1’)。

该位被清 ‘0’ 时，邮箱 0 的其它发送状态位 (TXOK0, ALST0 和 TERR0) 也被清 ‘0’。

#### 32.10.4 CAN 接收 FIFO 0 寄存器 (CAN\_RF0R)

地址偏移量: 0x0C

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |    |    |       |       |       |     |           |    |
|----------|----|----|----|----|----|----|----|----|----|-------|-------|-------|-----|-----------|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21    | 20    | 19    | 18  | 17        | 16 |
| Reserved |    |    |    |    |    |    |    |    |    |       |       |       |     |           |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5     | 4     | 3     | 2   | 1         | 0  |
| Reserved |    |    |    |    |    |    |    |    |    | RFOM0 | FOVR0 | FULL0 | Res | FMP0[1:0] |    |
|          |    |    |    |    |    |    |    |    |    | rs    | rc_w1 | rc_w1 |     | r         | r  |

Bits 31:6 保留，必须保持为复位值。

Bit 5 **RFOM0**: 释放接收 FIFO 0 输出邮箱 (Release FIFO 0 output mailbox)

软件通过对该位置 ‘1’ 来释放接收 FIFO 的输出邮箱。如果接收 FIFO 为空，那么对该位置 ‘1’ 没有任何效果，即只有当 FIFO 中有报文时对该位置 ‘1’ 才有意义。如果 FIFO 中有 2 个以上的报文，由于 FIFO 的特点，软件需要释放输出邮箱才能访问第 2 个报文。

当输出邮箱被释放时，硬件对该位清 ‘0’。

Bit 4 **FOVR0**: FIFO 0 溢出 (FIFO 0 overrun)

当 FIFO 0 已满，又收到新的报文且报文符合过滤条件，硬件对该位置 ‘1’。

该位由软件清 ‘0’。

Bit 3 **FULL0**: FIFO 0 满 (FIFO 0 full)

当 FIFO 0 中有 3 个报文时，硬件对该位置 ‘1’。

该位由软件清 ‘0’。

Bit 2 保留，必须保持为复位值。

Bits 1:0 **FMP0[1:0]**: FIFO 0 报文数目 (FIFO 0 message pending)

FIFO 0 报文数目这 2 位反映了当前接收 FIFO 0 中存放的报文数目。

每当 1 个新的报文被存入接收 FIFO 0，硬件就对 FMP0 加 1。

每当软件对 RFOM0 位写 ‘1’ 来释放输出邮箱，FMP0 就被减 1，直到其为 0。

### 32.10.5 CAN 接收 FIFO 1 寄存器 (CAN\_RF1R)

地址偏移量：0x10

复位值：0x0000 0000

|          |    |    |    |    |    |    |    |    |    |       |       |       |     |           |    |
|----------|----|----|----|----|----|----|----|----|----|-------|-------|-------|-----|-----------|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21    | 20    | 19    | 18  | 17        | 16 |
| Reserved |    |    |    |    |    |    |    |    |    |       |       |       |     |           |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5     | 4     | 3     | 2   | 1         | 0  |
| Reserved |    |    |    |    |    |    |    |    |    | RFOM1 | FOVR1 | FULL1 | Res | FMP1[1:0] |    |
|          |    |    |    |    |    |    |    |    |    | rs    | rc_w1 | rc_w1 |     | r         | r  |

Bits 31:6 保留，必须保持为复位值。

Bit 5 **RFOM1**: 释放接收 FIFO 1 输出邮箱 (Release FIFO 1 output mailbox)

软件通过对该位置 ‘1’ 来释放接收 FIFO 的输出邮箱。如果接收 FIFO 为空，那么对该位置 ‘1’ 没有任何效果，即只有当 FIFO 中有报文时对该位置 ‘1’ 才有意义。如果 FIFO 中有 2 个以上的报文，由于 FIFO 的特点，软件需要释放输出邮箱才能访问第 2 个报文。

当输出邮箱被释放时，硬件对该位清 ‘0’。

Bit 4 **FOVR1**: FIFO 1 溢出 (FIFO 1 overrun)

当 FIFO 1 已满，又收到新的报文且报文符合过滤条件，硬件对该位置 ‘1’。

该位由软件清 ‘0’。

Bit 3 **FULL1**: FIFO 1 满 (FIFO 1 full)

当 FIFO 1 中有 3 个报文时，硬件对该位置 ‘1’。

该位由软件清 ‘0’。

Bit 2 保留，必须保持为复位值。

Bits 1:0 **FMP1[1:0]**: FIFO 1 报文数目 (FIFO 1 message pending)

FIFO 1 报文数目这 2 位反映了当前接收 FIFO 1 中存放的报文数目。

每当 1 个新的报文被存入接收 FIFO 1，硬件就对 FMP0 加 1。

每当软件对 RFOM1 位写 ‘1’ 来释放输出邮箱，FMP1 就被减 1，直到其为 0。

### 32.10.6 CAN 中断使能寄存器 (CAN\_IER)

地址偏移量：0x14

复位值：0x0000 0000

|          |          |    |    |    |    |       |       |       |       |     |        |       |        |        |       |
|----------|----------|----|----|----|----|-------|-------|-------|-------|-----|--------|-------|--------|--------|-------|
| 31       | 30       | 29 | 28 | 27 | 26 | 25    | 24    | 23    | 22    | 21  | 20     | 19    | 18     | 17     | 16    |
| Reserved |          |    |    |    |    |       |       |       |       |     |        |       |        | SLKIE  | WKUIE |
|          |          |    |    |    |    |       |       |       |       |     |        |       |        | rw     | rw    |
| 15       | 14       | 13 | 12 | 11 | 10 | 9     | 8     | 7     | 6     | 5   | 4      | 3     | 2      | 1      | 0     |
| ERRIE    | Reserved |    |    |    |    | LECIE | BOFIE | EPVIE | EWGIE | Res | FOVIE1 | FFIE1 | FMPIE1 | FOVIE0 | FFIE0 |
| rw       |          |    |    |    |    | rw    | rw    | rw    | rw    |     | rw     | rw    | rw     | rw     | rw    |
|          |          |    |    |    |    |       |       |       |       |     |        |       |        | FMPIE0 | TMEIE |
|          |          |    |    |    |    |       |       |       |       |     |        |       |        | rw     | rw    |

Bits 31:18 保留，必须保持为复位值。

Bit 17 **SLKIE**: 睡眠中断使能 (Sleep interrupt enable)

0: 当 SLAKI 位被置 ‘1’ 时，不产生中断；

1: 当 SLAKI 位被置 ‘1’ 时，产生中断。

Bit 16 **WKUIE**: 唤醒中断使能 (Wakeup interrupt enable)

0: 当 WKUI 位被置 ‘1’ 时，不产生中断；

1: 当 WKUI 位被置 '1' 时, 产生中断。

Bit 15 **ERRIE**: 错误中断使能 (Error interrupt enable)

0: 当 CAN\_ESR 寄存器有错误挂号时, 不产生中断;

1: 当 CAN\_ESR 寄存器有错误挂号时, 产生中断。

Bits 14:12 保留, 必须保持为复位值。

Bit 11 **LECIE**: 上次错误号中断使能 (Last error code interrupt enable)

0: 当检测到错误, 硬件设置 LEC[2:0]时, 不设置 ERRI 位;

1: 当检测到错误, 硬件设置 LEC[2:0]时, 设置 ERRI 位为 '1'。

Bit 10 **BOFIE**: 离线中断使能 (Bus-off interrupt enable)

0: 当 BOFF 位被置 '1' 时, 不设置 ERRI 位;

1: 当 BOFF 位被置 '1' 时, 设置 ERRI 位为 '1'。

Bit 9 **EPVIE**: 错误被动中断使能 (Error Passive Interrupt Enable)

0: 当 EPVF 位被置 '1' 时, 不设置 ERRI 位;

1: 当 EPVF 位被置 '1' 时, 设置 ERRI 位为 '1'。

Bit 8 **EWGIE**: 错误警告中断使能 (Error warning interrupt enable)

0: 当 EWGF 位被置 '1' 时, 不设置 ERRI 位;

1: 当 EWGF 位被置 '1' 时, 设置 ERRI 位为 '1'。

Bit 7 保留, 必须保持为复位值。

Bit 6 **FOVIE1**: FIFO 1 溢出中断使能 (FIFO overrun interrupt enable)

0: 当 FIFO 1 的 FOVR 位被置 '1' 时, 不产生中断;

1: 当 FIFO 1 的 FOVR 位被置 '1' 时, 产生中断。

Bit 5 **FFIE1**: FIFO 1 满中断使能 (FIFO full interrupt enable)

0: 当 FIFO 1 的 FULL 位被置 '1' 时, 不产生中断;

1: 当 FIFO 1 的 FULL 位被置 '1' 时, 产生中断。

Bit 4 **FMPIE1**: FIFO 1 消息挂号中断使能 (FIFO message pending interrupt enable)

0: 当 FIFO 1 的 FMP[1:0]位为非 0 时, 不产生中断;

1: 当 FIFO 1 的 FMP[1:0]位为非 0 时, 产生中断。

Bit 3 **FOVIE0**: FIFO 0 溢出中断使能 (FIFO overrun interrupt enable)

0: 当 FIFO 0 的 FOVR 位被置 '1' 时, 不产生中断;

1: 当 FIFO 0 的 FOVR 位被置 '1' 时, 产生中断。

Bit 2 **FFIE0**: FIFO 0 满中断使能 (FIFO full interrupt enable)

0: 当 FIFO 0 的 FULL 位被置 '1' 时, 不产生中断;

1: 当 FIFO 0 的 FULL 位被置 '1' 时, 产生中断。

Bit 1 **FMPIE0**: FIFO 0 消息挂号中断使能 (FIFO message pending interrupt enable)

0: 当 FIFO 0 的 FMP[1:0]位为非 0 时, 不产生中断;

1: 当 FIFO 0 的 FMP[1:0]位为非 0 时, 产生中断。

Bit 0 **TMEIE**: 发送邮箱空中断使能 (Transmit mailbox empty interrupt enable)

0: 当 RQCPx 位被置 '1' 时, 不产生中断;

1: 当 RQCPx 位被置 '1' 时, 产生中断。

### 32.10.7 CAN 错误状态寄存器 (CAN\_ESR)

地址偏移量: 0x18

复位值: 0x0000 0000

|          |    |    |    |    |    |    |    |          |    |    |     |      |      |      |    |
|----------|----|----|----|----|----|----|----|----------|----|----|-----|------|------|------|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23       | 22 | 21 | 20  | 19   | 18   | 17   | 16 |
| REC[7:0] |    |    |    |    |    |    |    | TEC[7:0] |    |    |     |      |      |      |    |
| r        | r  | r  | r  | r  | r  | r  | r  | r        | r  | r  | r   | r    | r    | r    | r  |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7        | 6  | 5  | 4   | 3    | 2    | 1    | 0  |
| Reserved |    |    |    |    |    |    |    | LEC[2:1] |    |    | Res | BOFF | EPVF | WEGF |    |
|          |    |    |    |    |    |    |    | rw       | rw | rw |     | r    | r    | r    |    |

**Bits 31:24 REC[7:0]:** 接收错误计数器 (Receive error counter)

这个计数器按照 CAN 协议的故障界定机制的接收部分实现。按照 CAN 的标准, 当接收出错时, 根据出错的条件, 该计数器加 1 或加 8; 而在每次接收成功后, 该计数器减 1, 或当该计数器的值大于 127 时, 设置它的值为 120。当该计数器的值超过 127 时, CAN 进入错误被动状态。

**Bits 23:16 TEC[7:0]:** 9 位发送错误计数器的低 8 位 (Least significant byte of the 9-bit transmit error counter)

与上面相似, 这个计数器按照 CAN 协议的故障界定机制的发送部分实现。

**Bits 15:7** 保留, 必须保持为复位值。

**Bits 6:4 LEC[2:0]:** 上次错误代码 (Last error code)

在检测到 CAN 总线上发生错误时, 硬件根据出错情况设置。当报文被正确发送或接收后, 硬件清除其值为 '0'。

硬件没有使用错误代码 7, 软件可以设置该值, 从而可以检测代码的更新。

000: 没有错误;

001: 位填充错;

010: 格式 (Form) 错;

011: 确认 (ACK) 错;

100: 隐性位错;

101: 显性位错;

110: CRC 错;

111: 由软件设置。

**Bit 3** 保留, 必须保持为复位值。

**Bit 2 BOFF:** 离线标志 (Bus-off flag)

当进入离线状态时, 硬件对该位置 '1'。当发送错误计数器 TEC 溢出, 即大于 255 时, CAN 进入离线状态。

**Bit 1 EPVF:** 错误被动标志 (Error passive flag)

当出错次数达到错误被动的阈值时, 硬件对该位置 '1'。

(接收错误计数器或发送错误计数器的值 > 127)。

**Bit 0 EWGF:** 错误警告标志 (Error warning flag)

当出错次数达到警告的阈值时, 硬件对该位置 '1'。

(接收错误计数器或发送错误计数器的值 ≥ 96)。

### 32.10.8 CAN 位时序寄存器 (CAN\_BTR)

地址偏移量: 0x1C

复位值: 0x0123 0000

注: 当 CAN 处于初始化模式时, 该寄存器只能由软件访问。

|      |      |          |    |    |    |          |    |     |          |    |    |          |    |    |    |
|------|------|----------|----|----|----|----------|----|-----|----------|----|----|----------|----|----|----|
| 31   | 30   | 29       | 28 | 27 | 26 | 25       | 24 | 23  | 22       | 21 | 20 | 19       | 18 | 17 | 16 |
| SILM | LBKM | Reserved |    |    |    | SJW[1:0] |    | Res | TS2[2:0] |    |    | TS1[3:0] |    |    |    |
| rw   | rw   |          |    |    |    | rw       | rw |     | rw       | rw | rw | rw       | rw | rw | rw |



| 15       | 14 | 13 | 12 | 11 | 10 | 9        | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
|----------|----|----|----|----|----|----------|----|----|----|----|----|----|----|----|----|
| Reserved |    |    |    |    |    | BRP[9:0] |    |    |    |    |    |    |    |    |    |
|          |    |    |    |    |    | rw       | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31 **SILM**: 静默模式（用于调试）（Silent mode (debug)）

0: 正常状态;

1: 静默模式。

Bit 30 **LBKM**: 环回模式（用于调试）（Loop back mode (debug)）

0: 禁止环回模式;

1: 允许环回模式。

Bits 19:26 保留，必须保持为复位值。

Bits 25:24 **SJW[1:0]**: 重新同步跳跃宽度（Resynchronization jump width）

为了重新同步，该位域定义了 CAN 硬件在每位中可以延长或缩短多少个时间单元的上限。

$$t_{RJW} = t_{CAN} \times (SJW[1:0] + 1)。$$

Bit 23 保留，必须保持为复位值。

Bits 22:20 **TS2[2:0]**: 时间段 2（Time segment 2）

该位域定义了时间段 2 占用了多少个时间单元

$$t_{BS2} = t_{CAN} \times (TS2[2:0] + 1)。$$

Bits 19:16 **TS1[3:0]**: 时间段 1（Time segment 1）

该位域定义了时间段 1 占用了多少个时间单元

$$t_{BS1} = t_{CAN} \times (TS1[3:0] + 1)$$

Bits 15:10 保留，必须保持为复位值。

Bits 9:0 **BRP[9:0]**: 波特率分频器（Baud rate prescaler）

该位域定义了时间单元(tq)的时间长度

$$t_q = (BRP[9:0]+1) \times t_{PCLK}$$

## 32.11 CAN 邮箱寄存器描述

本节描述发送和接收邮箱寄存器。

除了下述例外，发送和接收邮箱几乎一样：

- CAN\_RDTxR 寄存器的 FMI 域;
- 接收邮箱是只读的;
- 发送邮箱只有在它为空时才是可写的，CAN\_TSR 寄存器的相应 TME 位为 ‘1’，表示发送邮箱为空。

共有 3 个发送邮箱和 2 个接收邮箱。每个接收邮箱为 3 级深度的 FIFO，并且只能访问 FIFO 中最先收到的报文。

每个邮箱包含 4 个寄存器。

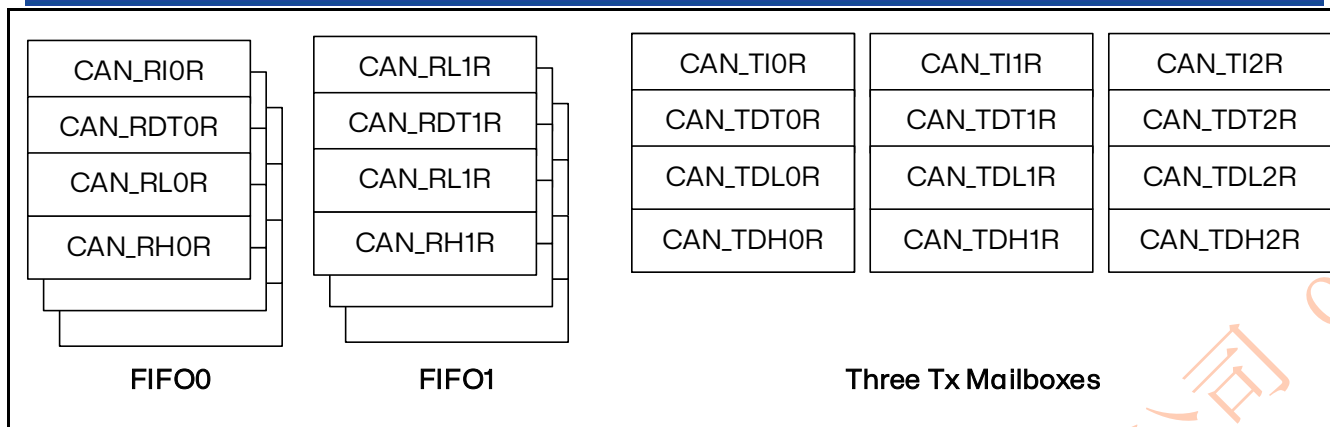


图 32.16 RX 和 TX 邮箱

### 32.11.1 发送邮箱标识符寄存器 (CAN\_TlRxR) (x=0..2)

地址偏移量: 0x180, 0x190, 0x1A0

复位值: 0xFFFF XXXX, X=未定义位 (除了第 0 位, 复位时 TXRQ=0)

注: 1 当其所属的邮箱处在等待发送的状态时, 该寄存器是写保护的

2 该寄存器实现了发送请求控制功能(第 0 位) — 复位值为 0

|                        |    |    |    |    |    |    |    |    |    |    |             |    |     |     |      |
|------------------------|----|----|----|----|----|----|----|----|----|----|-------------|----|-----|-----|------|
| 31                     | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20          | 19 | 18  | 17  | 16   |
| STID[10:0]/EXID[28:18] |    |    |    |    |    |    |    |    |    |    | EXID[17:13] |    |     |     |      |
| rw                     | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw          | rw | rw  | rw  | rw   |
| 15                     | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4           | 3  | 2   | 1   | 0    |
| EXID[12:0]             |    |    |    |    |    |    |    |    |    |    |             |    | IDE | RTR | TXRQ |
| rw                     | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw          | rw | rw  | rw  | rw   |

Bits 31:21 **STID[10:0]/EXID[28:18]**: 标准标识符或扩展标识符 (Standard identifier or extended identifier)

依据 IDE 位的内容, 这些位或是标准标识符, 或是扩展身份标识的高字节。

Bits 20:3 **EXID[17:0]**: 扩展标识符 (Extended identifier)

扩展身份标识的低字节。

Bit 2 **IDE**: 标识符选择 (Identifier extension)

该位决定发送邮箱中报文使用的标识符类型

0: 使用标准标识符;

1: 使用扩展标识符。

Bit 1 **RTR**: 远程发送请求 (Remote transmission request)

0: 数据帧;

1: 远程帧。

Bit 0 **TXRQ**: 发送数据请求 (Transmit mailbox request)

由软件对其置 ‘1’, 来请求发送邮箱的数据。当数据发送完成, 邮箱为空时, 硬件对其清 ‘0’。

### 32.11.2 发送邮箱数据长度和时间戳寄存器 (CAN\_TDTxR) (x=0..2)

当邮箱不在空置状态时, 该寄存器的所有位为写保护。

地址偏移量: 0x184, 0x194, 0x1A4

复位值: 未定义



|            |    |    |    |    |    |    |    |     |          |    |    |    |          |    |    |
|------------|----|----|----|----|----|----|----|-----|----------|----|----|----|----------|----|----|
| 31         | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23  | 22       | 21 | 20 | 19 | 18       | 17 | 16 |
| TIME[15:0] |    |    |    |    |    |    |    |     |          |    |    |    |          |    |    |
| rw         | rw | rw | rw | rw | rw | rw | rw | rw  | rw       | rw | rw | rw | rw       | rw | rw |
| 15         | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7   | 6        | 5  | 4  | 3  | 2        | 1  | 0  |
| Reserved   |    |    |    |    |    |    |    | TGT | Reserved |    |    |    | DLC[3:0] |    |    |
|            |    |    |    |    |    |    |    | rw  |          |    |    |    | rw       | rw | rw |

Bits 31:16 **TIME[15:0]**: 报文时间戳 (Message time stamp)

该域包含了, 在发送该报文 SOF 的时刻, 16 位定时器的值。

Bits 15:9 保留, 必须保持为复位值。

Bit 8 **TGT**: 发送时间戳 (Transmit global time)

只有在 CAN 处于时间触发通信模式, 即 CAN\_MCR 寄存器的 TTCM 位为 ‘1’ 时, 该位才有效。

0: 不发送时间戳 TIME[15:0];

1: 发送时间戳 TIME[15:0]。在长度为 8 的报文中, 时间戳 TIME[15:0] 是最后 2 个发送的字节: TIME[7:0] 作为第 7 个字节, TIME[15:8] 为第 8 个字节, 它们替换了写入 CAN\_TDHxR[31:16] 的数据 (DATA6[7:0] 和 DATA7[7:0])。为了把时间戳的 2 个字节发送出去, DLC 必须编程为 8。

Bits 7:4 保留位。

Bits 3:0 **DLC[15:0]**: 发送数据长度 (Data length code)

该域指定了数据报文的数据长度或者远程帧请求的数据长度。1 个报文包含 0 到 8 个字节数据,

而这由 DLC 决定。

### 32.11.3 发送邮箱低字节数据寄存器 (CAN\_TDLxR) (x=0..2)

当邮箱不在空置状态时, 该寄存器的所有位为写保护。

地址偏移量: 0x188, 0x198, 0x1A8

复位值: 未定义位

|            |    |    |    |    |    |    |    |            |    |    |    |    |    |    |    |
|------------|----|----|----|----|----|----|----|------------|----|----|----|----|----|----|----|
| 31         | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23         | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DATA3[7:0] |    |    |    |    |    |    |    | DATA2[7:0] |    |    |    |    |    |    |    |
| rw         | rw | rw | rw | rw | rw | rw | rw | rw         | rw | rw | rw | rw | rw | rw | rw |
| 15         | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7          | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| DATA1[7:0] |    |    |    |    |    |    |    | DATA0[7:0] |    |    |    |    |    |    |    |
| rw         | rw | rw | rw | rw | rw | rw | rw | rw         | rw | rw | rw | rw | rw | rw | rw |

Bits 31:24 **DATA3[7:0]**: 数据字节 3 (Data byte 3)

报文的数据字节 3。

Bits 23:16 **DATA2[7:0]**: 数据字节 2 (Data byte 2)

报文的数据字节 2。

Bits 15:8 **DATA1[7:0]**: 数据字节 1 (Data byte 1)

报文的数据字节 1。

Bits 7:0 **DATA0[7:0]**: 数据字节 0 (Data byte 0)

报文的数据字节 0。

报文包含 0 到 8 个字节数据, 且从字节 0 开始。

### 32.11.4 发送邮箱高字节数据寄存器 (CAN\_TDHxR) (x=0..2)

当邮箱不在空置状态时, 该寄存器的所有位为写保护。

地址偏移量：0x18C, 0x19C, 0x1AC

复位值：未定义位

|            |    |    |    |    |    |    |    |            |    |    |    |    |    |    |    |
|------------|----|----|----|----|----|----|----|------------|----|----|----|----|----|----|----|
| 31         | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23         | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DATA7[7:0] |    |    |    |    |    |    |    | DATA6[7:0] |    |    |    |    |    |    |    |
| rw         | rw | rw | rw | rw | rw | rw | rw | rw         | rw | rw | rw | rw | rw | rw | rw |
| 15         | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7          | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| DATA5[7:0] |    |    |    |    |    |    |    | DATA4[7:0] |    |    |    |    |    |    |    |
| rw         | rw | rw | rw | rw | rw | rw | rw | rw         | rw | rw | rw | rw | rw | rw | rw |

Bits 31:24 **DATA7[7:0]**：数据字节 7（Data byte 7）

报文的数据字节 7。

注：如果 CAN\_MCR 寄存器的 TTCM 位为 ‘1’，且该邮箱的 TGT 位也为 ‘1’，那么 DATA7 和 DATA6 将被 TIME 时间戳代替。

Bits 23:16 **DATA6[7:0]**：数据字节 6（Data byte 6）

报文的数据字节 6。

Bits 15:8 **DATA5[7:0]**：数据字节 5（Data byte 5）

报文的数据字节 5。

Bits 7:0 **DATA4[7:0]**：数据字节 4（Data byte 4）

报文的数据字节 4。

### 32.11.5 接收 FIFO 邮箱标识符寄存器（CAN\_RlRxR）（x=0..1）

地址偏移量：0x1B0, 0x1C0

复位值：未定义位

注：所有接收邮箱寄存器都是只读的。

|                        |    |    |    |    |    |    |    |    |    |    |             |    |     |     |    |
|------------------------|----|----|----|----|----|----|----|----|----|----|-------------|----|-----|-----|----|
| 31                     | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20          | 19 | 18  | 17  | 16 |
| STID[10:0]/EXID[28:18] |    |    |    |    |    |    |    |    |    |    | EXID[17:13] |    |     |     |    |
| r                      | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r           | r  | r   | r   | r  |
| 15                     | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4           | 3  | 2   | 1   | 0  |
| EXID[12:0]             |    |    |    |    |    |    |    |    |    |    | IDE         |    | RTR | Res |    |
| r                      | r  | r  | r  | r  | r  | r  | r  | r  | r  | r  | r           | r  | r   |     |    |

Bits 31:21 **STID[10:0]/EXID[28:18]**：标准标识符或扩展标识符（Standard identifier or extended identifier）

依据 IDE 位的内容，这些位或是标准标识符，或是扩展身份标识的高字节。

Bits 20:3 **EXID[17:0]**：扩展标识符（Extended identifier）

扩展标识符的低字节。

Bit 2 **IDE**：标识符选择（Identifier extension）

该位决定接收邮箱中报文使用的标识符类型

0：使用标准标识符；

1：使用扩展标识符。

Bit 1 **RTR**：远程发送请求（Remote transmission request）

0：数据帧；

1：远程帧。

Bit 0 保留位。

### 32.11.6 接收 FIFO 邮箱数据长度和时间戳寄存器（CAN\_RDTxR）（x=0..1）

地址偏移量：0x1B4, 0x1C4

复位值：未定义

注：所有接收邮箱寄存器都是只读的。

|            |    |    |    |    |    |    |    |          |    |    |    |          |    |    |    |
|------------|----|----|----|----|----|----|----|----------|----|----|----|----------|----|----|----|
| 31         | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23       | 22 | 21 | 20 | 19       | 18 | 17 | 16 |
| TIME[15:0] |    |    |    |    |    |    |    |          |    |    |    |          |    |    |    |
| r          | r  | r  | r  | r  | r  | r  | r  | r        | r  | r  | r  | r        | r  | r  | r  |
| 15         | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7        | 6  | 5  | 4  | 3        | 2  | 1  | 0  |
| FMI[7:0]   |    |    |    |    |    |    |    | Reserved |    |    |    | DLC[3:0] |    |    |    |
| r          | r  | r  | r  | r  | r  | r  | r  |          |    |    |    | r        | r  | r  | r  |

Bits 31:16 **TIME[15:0]**：报文时间戳（Message time stamp）

该域包含了，在接收该报文 SOF 的时刻，16 位定时器的值。

Bits 15:8 **FMI[15:0]**：过滤器匹配序号（Filter match index）

这里是存在邮箱中的信息传送的过滤器序号。

Bits 7:4 保留，必须保持为复位值。

Bits 3:0 **DLC[15:0]**：接收数据长度（Data length code）

该域表明接收数据帧的数据长度（0~8）。对于远程帧请求，数据长度 DLC 恒为 0。

### 32.11.7 接收 FIFO 邮箱低字节数据寄存器（CAN\_RDLxR）（x=0..1）

地址偏移量：0x1B8, 0x1C8

复位值：未定义位

注：所有接收邮箱寄存器都是只读的。

|            |    |    |    |    |    |    |    |            |    |    |    |    |    |    |    |
|------------|----|----|----|----|----|----|----|------------|----|----|----|----|----|----|----|
| 31         | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23         | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DATA3[7:0] |    |    |    |    |    |    |    | DATA2[7:0] |    |    |    |    |    |    |    |
| r          | r  | r  | r  | r  | r  | r  | r  | r          | r  | r  | r  | r  | r  | r  | r  |
| 15         | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7          | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| DATA1[7:0] |    |    |    |    |    |    |    | DATA0[7:0] |    |    |    |    |    |    |    |
| r          | r  | r  | r  | r  | r  | r  | r  | r          | r  | r  | r  | r  | r  | r  | r  |

Bits 31:24 **DATA3[7:0]**：数据字节 3（Data byte 3）

报文的数据字节 3。

Bits 23:16 **DATA2[7:0]**：数据字节 2（Data byte 2）

报文的数据字节 2。

Bits 15:8 **DATA1[7:0]**：数据字节 1（Data byte 1）

报文的数据字节 1。

Bits 7:0 **DATA0[7:0]**：数据字节 0（Data byte 0）

报文的数据字节 0。

### 32.11.8 接收 FIFO 邮箱高字节数据寄存器（CAN\_RDHxR）（x=0..1）

地址偏移量：0x1BC, 0x1CC

复位值：未定义位

注：所有接收邮箱寄存器都是只读的。

|            |    |    |    |    |    |    |    |            |    |    |    |    |    |    |    |
|------------|----|----|----|----|----|----|----|------------|----|----|----|----|----|----|----|
| 31         | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23         | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DATA7[7:0] |    |    |    |    |    |    |    | DATA6[7:0] |    |    |    |    |    |    |    |
| r          | r  | r  | r  | r  | r  | r  | r  | r          | r  | r  | r  | r  | r  | r  | r  |
| 15         | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7          | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| DATA5[7:0] |    |    |    |    |    |    |    | DATA4[7:0] |    |    |    |    |    |    |    |
| r          | r  | r  | r  | r  | r  | r  | r  | r          | r  | r  | r  | r  | r  | r  | r  |

Bits 31:24 **DATA7[7:0]**：数据字节 7（Data byte 7）

报文的数据字节 7。

Bits 23:16 **DATA6[7:0]**: 数据字节 6 (Data byte 6)

报文的数据字节 6。

Bits 15:8 **DATA5[7:0]**: 数据字节 5 (Data byte 5)

报文的数据字节 5。

Bits 7:0 **DATA4[7:0]**: 数据字节 4 (Data byte 4)

报文的数据字节 4。

## 32.12 CAN 过滤器寄存器描述

### 32.12.1 CAN 过滤器主控寄存器 (CAN\_FMR)

地址偏移量: 0x200

复位值: 0x2A1C 0E01

注: 该寄存器的非保留位完全由软件控制。

|          |    |    |    |    |    |    |    |    |    |    |    |    |    |       |    |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|-------|----|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17    | 16 |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    |       |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1     | 0  |
| Reserved |    |    |    |    |    |    |    |    |    |    |    |    |    | FINIT |    |
|          |    |    |    |    |    |    |    |    |    |    |    |    |    | rw    |    |

Bits 31:1 保留, 必须保持为复位值。

Bit 0 **FINIT**: 过滤器初始化模式 (Filter init mode)

针对所有过滤器组的初始化模式设置。

0: 过滤器组工作在正常模式;

1: 过滤器组工作在初始化模式。

### 32.12.2 CAN 过滤器模式寄存器 (CAN\_FM1R)

地址偏移量: 0x204

复位值: 0x0000 0000

注: 只有在设置 CAN\_FMR (FINIT=1), 使过滤器处于初始化模式下, 才能对该寄存器写入。

|          |    |       |       |       |       |      |      |      |      |      |      |      |      |      |      |
|----------|----|-------|-------|-------|-------|------|------|------|------|------|------|------|------|------|------|
| 31       | 30 | 29    | 28    | 27    | 26    | 25   | 24   | 23   | 22   | 21   | 20   | 19   | 18   | 17   | 16   |
| Reserved |    |       |       |       |       |      |      |      |      |      |      |      |      |      |      |
| 15       | 14 | 13    | 12    | 11    | 10    | 9    | 8    | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| Reserved |    | FBM13 | FBM12 | FBM11 | FBM10 | FBM9 | FBM8 | FBM7 | FBM6 | FBM5 | FBM4 | FBM3 | FBM2 | FBM1 | FBM0 |
|          |    | rw    | rw    | rw    | rw    | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   |

Bits 31:14 保留, 必须保持为复位值。

Bits 13:0 **FBMx**: 过滤器模式 (Filter mode)

过滤器组 x 的工作模式。

0: 过滤器组 x 的 2 个 32 位寄存器工作在标识符屏蔽位模式;

1: 过滤器组 x 的 2 个 32 位寄存器工作在标识符列表模式。

### 32.12.3 CAN 过滤器位宽寄存器 (CAN\_FS1R)

地址偏移量: 0x20C

复位值: 0x0000 0000

注：只有在设置 CAN\_FMR (FINIT=1)，使过滤器处于初始化模式下，才能对该寄存器写入。

|          |       |    |       |       |       |      |      |      |      |      |      |      |      |      |      |
|----------|-------|----|-------|-------|-------|------|------|------|------|------|------|------|------|------|------|
| 31       | 30    | 29 | 28    | 27    | 26    | 25   | 24   | 23   | 22   | 21   | 20   | 19   | 18   | 17   | 16   |
| Reserved |       |    |       |       |       |      |      |      |      |      |      |      |      |      |      |
| 15       | 14    | 13 | 12    | 11    | 10    | 9    | 8    | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| Reserved | FSC13 |    | FSC12 | FSC11 | FSC10 | FSC9 | FSC8 | FSC7 | FSC6 | FSC5 | FSC4 | FSC3 | FSC2 | FSC1 | FSC0 |
|          | rw    |    | rw    | rw    | rw    | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   |

Bits 31:14 保留，必须保持为复位值。

Bits 13:0 **FSCx**: 过滤器位宽设置 (Filter scale configuration)

过滤器组 x (13~0) 的位宽。

0: 过滤器位宽为 2 个 16 位;

1: 过滤器位宽为单个 32 位。

### 32.12.4 CAN 过滤器 FIFO 关联寄存器 (CAN\_FFA1R)

地址偏移量: 0x214

复位值: 0x0000 0000

注：只有在设置 CAN\_FMR (FINIT=1)，使过滤器处于初始化模式下，才能对该寄存器写入

|          |       |    |       |       |       |      |      |      |      |      |      |      |      |      |      |
|----------|-------|----|-------|-------|-------|------|------|------|------|------|------|------|------|------|------|
| 31       | 30    | 29 | 28    | 27    | 26    | 25   | 24   | 23   | 22   | 21   | 20   | 19   | 18   | 17   | 16   |
| Reserved |       |    |       |       |       |      |      |      |      |      |      |      |      |      |      |
| 15       | 14    | 13 | 12    | 11    | 10    | 9    | 8    | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| Reserved | FFA13 |    | FFA12 | FFA11 | FFA10 | FFA9 | FFA8 | FFA7 | FFA6 | FFA5 | FFA4 | FFA3 | FFA2 | FFA1 | FFA0 |
|          | rw    |    | rw    | rw    | rw    | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   |

Bits 31:14 保留，必须保持为复位值。

Bits 13:0 **FFAx**: 过滤器 FIFO 关联设置 (Filter FIFO assignment for filter x)

报文在通过了某过滤器的过滤后，将被存放到其关联的 FIFO 中。

0: 过滤器被关联到 FIFO0;

1: 过滤器被关联到 FIFO1。

### 32.12.5 CAN 过滤器激活寄存器 (CAN\_FACT1R)

地址偏移量: 0x21C

复位值: 0x0000 0000

|          |        |    |        |        |        |       |       |       |       |       |       |       |       |       |       |
|----------|--------|----|--------|--------|--------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 31       | 30     | 29 | 28     | 27     | 26     | 25    | 24    | 23    | 22    | 21    | 20    | 19    | 18    | 17    | 16    |
| Reserved |        |    |        |        |        |       |       |       |       |       |       |       |       |       |       |
| 15       | 14     | 13 | 12     | 11     | 10     | 9     | 8     | 7     | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| Reserved | FACT13 |    | FACT12 | FACT11 | FACT10 | FACT9 | FACT8 | FACT7 | FACT6 | FACT5 | FACT4 | FACT3 | FACT2 | FACT1 | FACT0 |
|          | rw     |    | rw     | rw     | rw     | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    | rw    |

Bits 31:14 保留，必须保持为复位值。

Bits 13:0 **FACTx**: 过滤器激活 (Filter active)

软件对某位设置 ‘1’ 来激活相应的过滤器。只有对 FACTx 位清 ‘0’，或对 CAN\_FMR 寄存器的 FINIT 位设置 ‘1’ 后，才能修改相应的过滤器寄存器 x (CAN\_FxR[0:1])。

0: 过滤器被禁用;

1: 过滤器被激活。

### 32.12.6 CAN 过滤器组 i 的寄存器 x (CAN\_FiRx) ) (i=0..13; x=1..2)

|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 31   | 30   | 29   | 28   | 27   | 26   | 25   | 24   | 23   | 22   | 21   | 20   | 19   | 18   | 17   | 16   |
| FB31 | FB30 | FB29 | FB28 | FB27 | FB26 | FB25 | FB24 | FB23 | FB22 | FB21 | FB20 | FB19 | FB18 | FB17 | FB16 |
| rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   |
| 15   | 14   | 13   | 12   | 11   | 10   | 9    | 8    | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| FB15 | FB14 | FB13 | FB12 | FB11 | FB10 | FB9  | FB8  | FB7  | FB6  | FB5  | FB4  | FB3  | FB2  | FB1  | FB0  |
| rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   |

Bits 31:0 **FB[31:0]**: 过滤器位 (Filter bits)

#### 标识符模式

寄存器的每位对应于所期望的标识符的相应位的电平。

0: 期望相应位为显性位;

1: 期望相应位为隐性位。

#### 屏蔽位模式

寄存器的每位指示是否对应的标识符寄存器位一定要与期望的标识符的相应位一致。

0: 不关心, 该位不用于比较;

1: 必须匹配, 到来的标识符位必须与滤波器对应的标识符寄存器位相一致。

注: 根据过滤器位宽和模式的不同设置, 过滤器组中的两个寄存器的功能也不尽相同。屏蔽位模式下的屏蔽标识符寄存器, 跟标识符列表模式下的寄存器位定义相同。

## 33 串行外设接口 (SPI)

### 33.1 SPI 简介

SPI 接口可以配置为支持 SPI 协议。SPI 接口默认工作在 SPI 方式

串行外设接口 (SPI) 允许芯片与外部设备以半/全双工、同步、串行方式通信。此接口可以被配置成主模式，并为外部从设备提供通信时钟 (SCK)。接口还能以多主配置方式工作。

它可用于多种用途，包括使用一条双向数据线的双线单工同步传输，还可使用 CRC 校验的可靠通信。

### 33.2 SPI 主要特征

#### 33.2.1 SPI 特征

- 3 线全双工同步传输
- 带或不带第三根双向数据线的双线单工同步传输
- 8 或 16 位传输帧格式选择
- 主或从操作
- 支持多主模式
- 8 个主模式波特率预分频系数 (最大为  $f_{PCLK}/2$ )
- 从模式频率 (最大为  $f_{PCLK}/2$ )
- 主模式和从模式的快速通信
- 主模式和从模式下均可以由软件或硬件进行 NSS 管理：主/从操作模式的动态改变
- 可编程的时钟极性和相位
- 可编程的数据顺序，MSB 在前或 LSB 在前
- 可触发中断的专用发送和接收标志
- SPI 总线忙状态标志
- 支持可靠通信的硬件 CRC
  - 在发送模式下，CRC 值可以被作为最后一个字节发送
  - 在全双工模式中对接收到的最后一个字节自动进行 CRC 校验
- 可触发中断的主模式故障、过载以及 CRC 错误标志
- 支持 DMA 功能的 1 字节发送和接收缓冲器：产生发送和接受请求

### 33.3 SPI 功能描述

#### 33.3.1 概述

SPI 的方框图见下图。



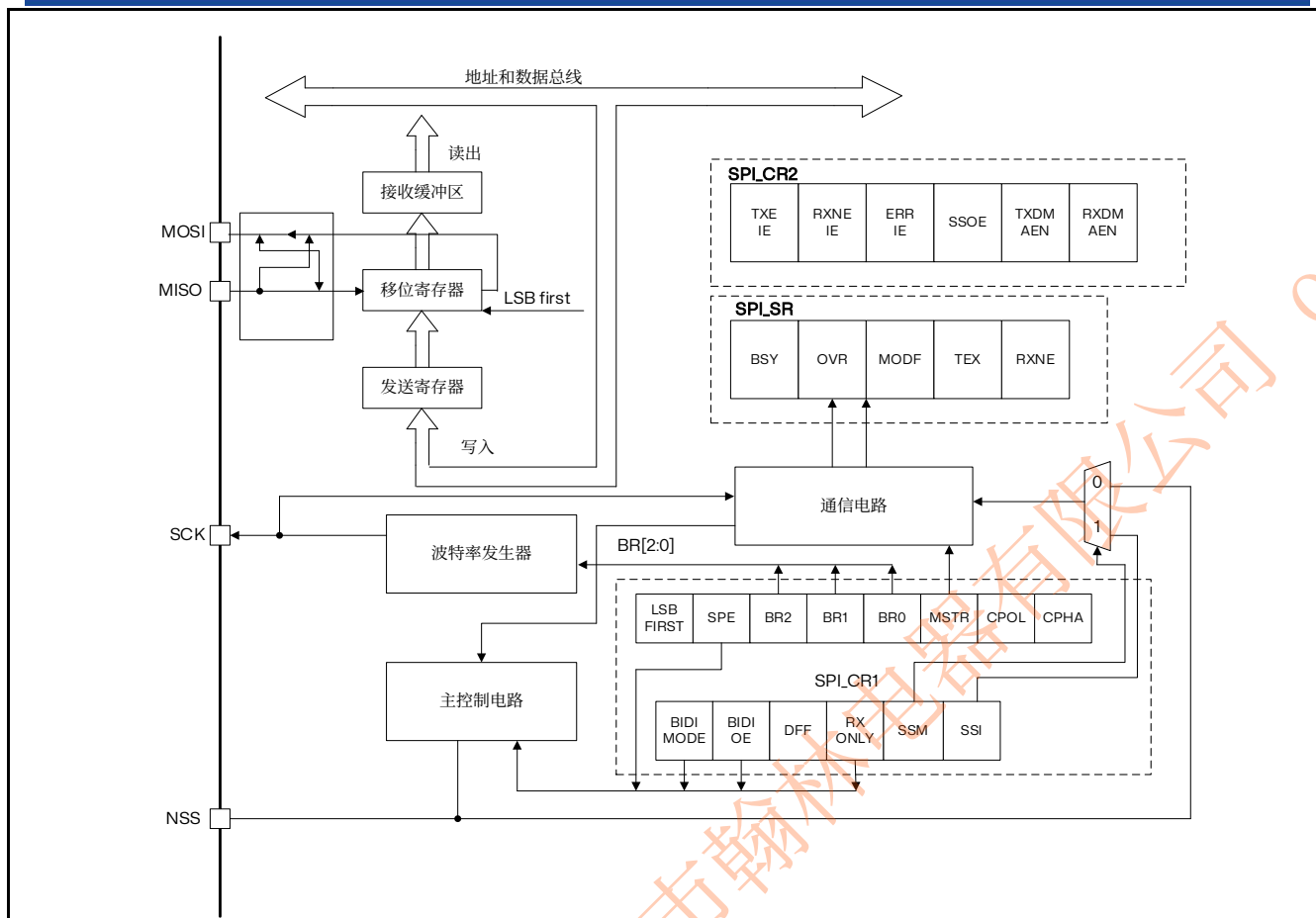
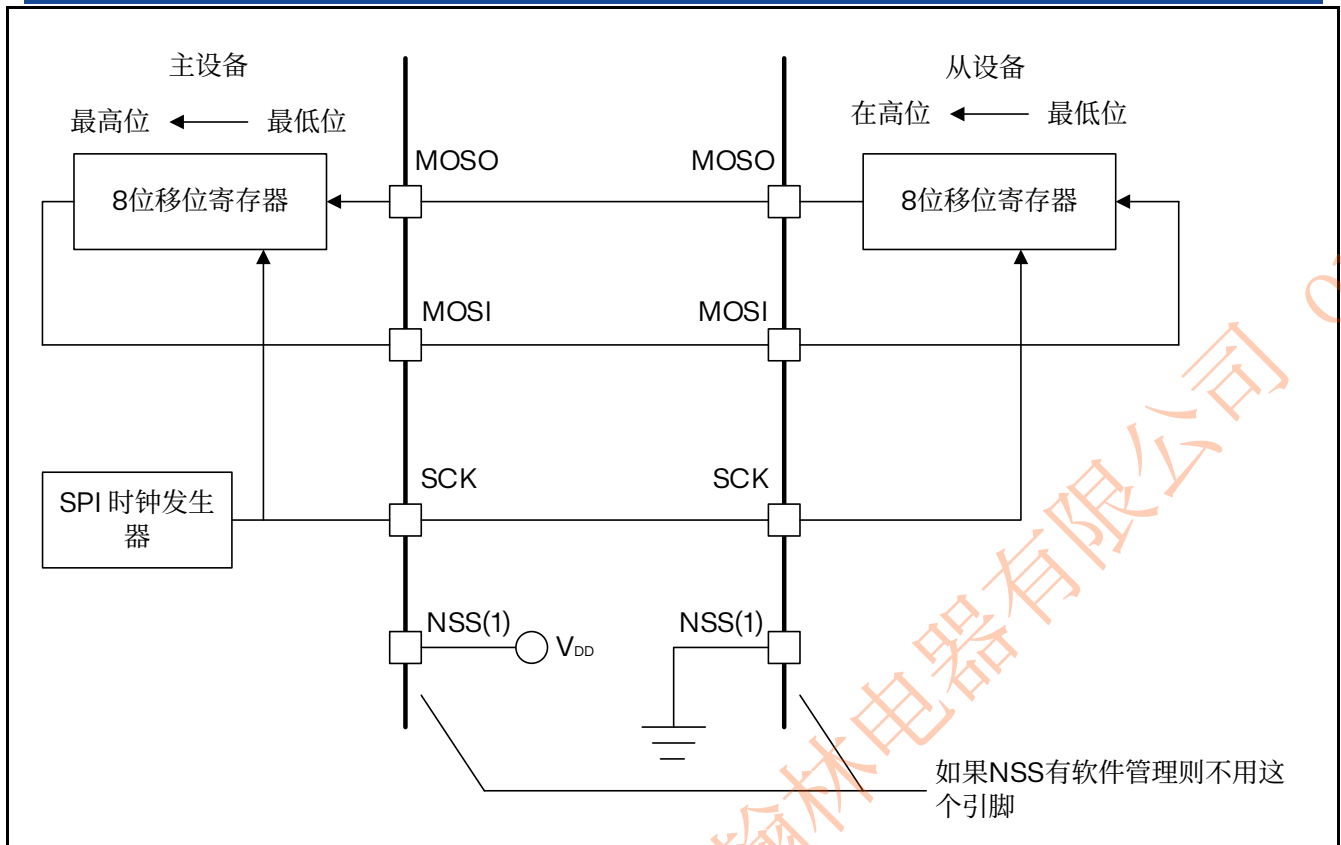


图 33.1 SPI 框图

通常 SPI 通过 4 个引脚与外部器件相连：

- MISO：主设备输入/从设备输出引脚。该引脚在从模式下发送数据，在主模式下接收数据。
- MOSI：主设备输出/从设备输入引脚。该引脚在主模式下发送数据，在从模式下接收数据。
- SCK：串口时钟，作为主设备的输出，从设备的输入
- NSS：从设备选择。这是一个可选的引脚，用来选择主/从设备。它的功能是用来作为“片选引脚”，让主设备可以单独地与特定从设备通讯，避免数据线上的冲突。从设备的 NSS 引脚可以由主设备的一个标准 I/O 引脚来驱动。一旦被使能（SSOE 位），NSS 引脚也可以作为输出引脚，并在 SPI 处于主模式时拉低；此时，所有的 SPI 设备，如果它们的 NSS 引脚连接到主设备的 NSS 引脚，则会检测到低电平，如果它们被设置为 NSS 硬件模式，就会自动进入从设备状态。当配置为主设备、NSS 配置为输入引脚（MSTR=1，SSOE=0）时，如果 NSS 被拉低，则这个 SPI 设备进入主模式失败状态：即 MSTR 位被自动清除，此设备进入从模式。

下图是一个单主和单从设备互连的例子。



1. 这里 NSS 引脚设置为输入

图 33.2 单主和单从应用

MOSI 脚相互连接，MISO 脚相互连接。这样，数据在主和从之间串行地传输（MSB 位在前）。

通信总是由主设备发起。主设备通过 MOSI 脚把数据发送给从设备，从设备通过 MISO 引脚回传数据。这意味全双工通信的数据输出和数据输入是用同一个时钟信号同步的；时钟信号由主设备通过 SCK 脚提供。

#### 从选择（NSS）脚管理

有 2 种 NSS 模式：

- 软件 NSS 模式：可以通过设置 SPI\_CR1 寄存器的 SSM 位来使能这种模式。在这种模式下 NSS 引脚可以用作它用，而内部 NSS 信号电平可以通过写 SPI\_CR1 的 SSI 位来驱动
- 硬件 NSS 模式，分两种情况：
  - NSS 输出被使能：当 RX32G410 工作为主 SPI，并且 NSS 输出已经通过 SPI\_CR2 寄存器的 SSOE 位使能，这时 NSS 引脚被拉低，所有 NSS 引脚与这个主 SPI 的 NSS 引脚相连并配置为硬件 NSS 的 SPI 设备，将自动变成从 SPI 设备。当一个 SPI 设备需要发送广播数据，它必须拉低 NSS 信号，以通知所有其它的设备它是主设备；如果它不能拉低 NSS，这意味着总线上有另外一个主设备在通信，这时将产生一个硬件失败错误（Hard Fault）。
  - NSS 输出被关闭：允许操作于多主环境。

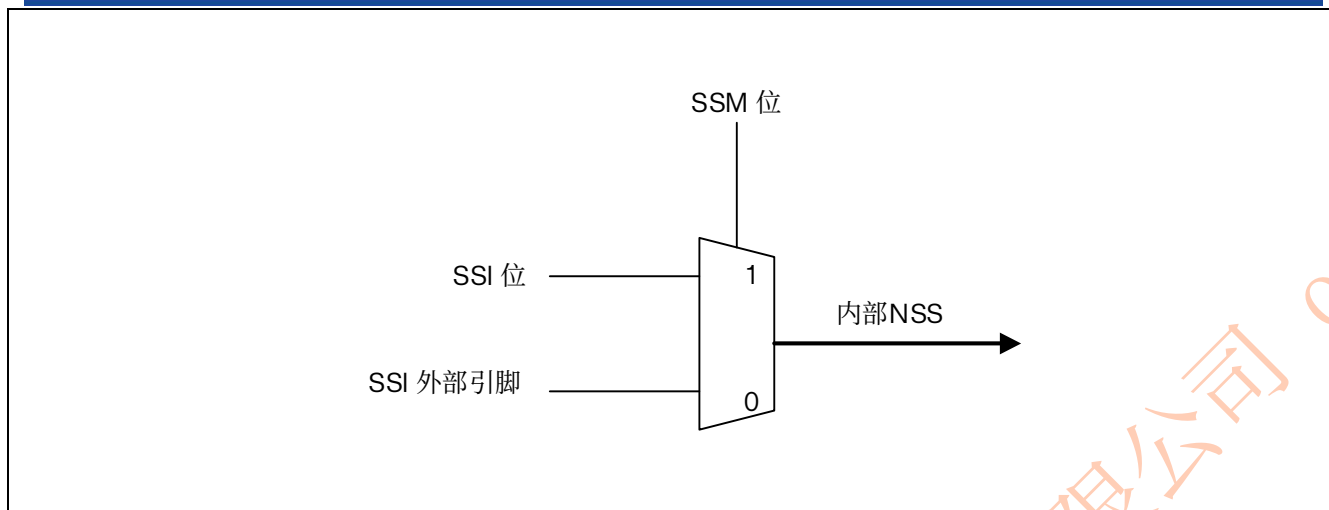


图 33.3 硬件/软件的从选择管理

### 时钟信号的相位和极性

SPI\_CR 寄存器的 CPOL 和 CPHA 位，能够组合成四种可能的时序关系。CPOL（时钟极性）位控制在没有数据传输时时钟的空闲状态电平，此位对主模式和从模式下的设备都有效。如果 CPOL 被清 0，SCK 引脚在空闲状态保持低电平；如果 CPOL 被置 ‘1’，SCK 引脚在空闲状态保持高电平。

如果 CPHA（时钟相位）位被置 ‘1’，SCK 时钟的第二个边沿（CPOL 位为 0 时就是下降沿，CPOL 位为 ‘1’ 时就是上升沿）进行数据位的采样，数据在第二个时钟边沿被锁存。如果 CPHA 位被清 ‘0’，SCK 时钟的第一边沿（CPOL 位为 ‘0’ 时就是上升沿，CPOL 位为 ‘1’ 时就是下降沿）进行数据位采样，数据在第一个时钟边沿被锁存。

CPOL 时钟极性和 CPHA 时钟相位的组合选择数据捕捉的时钟边沿。

下图显示了 SPI 传输的 4 种 CPHA 和 CPOL 位组合。此图可以解释为主设备和从设备的 SCK 脚、MISO 脚、MOSI 脚直接连接的主或从时序图。

注意：1. 在改变 CPOL/CPHA 位之前，必须清除 SPE 位将 SPI 禁止。

2. 主和从必须配置成相同的时序模式。

3. SCK 的空闲状态必须和 SPI\_CR1 寄存器指定的极性一致（CPOL 为 ‘1’ 时，空闲时应上拉 SCK 为高电平；CPOL 为 ‘0’ 时，空闲时应下拉 SCK 为低电平）。

4. 数据帧格式（8 位或 16 位）由 SPI\_CR1 寄存器的 DFF 位选择，并且决定发送/接收的数据长度。

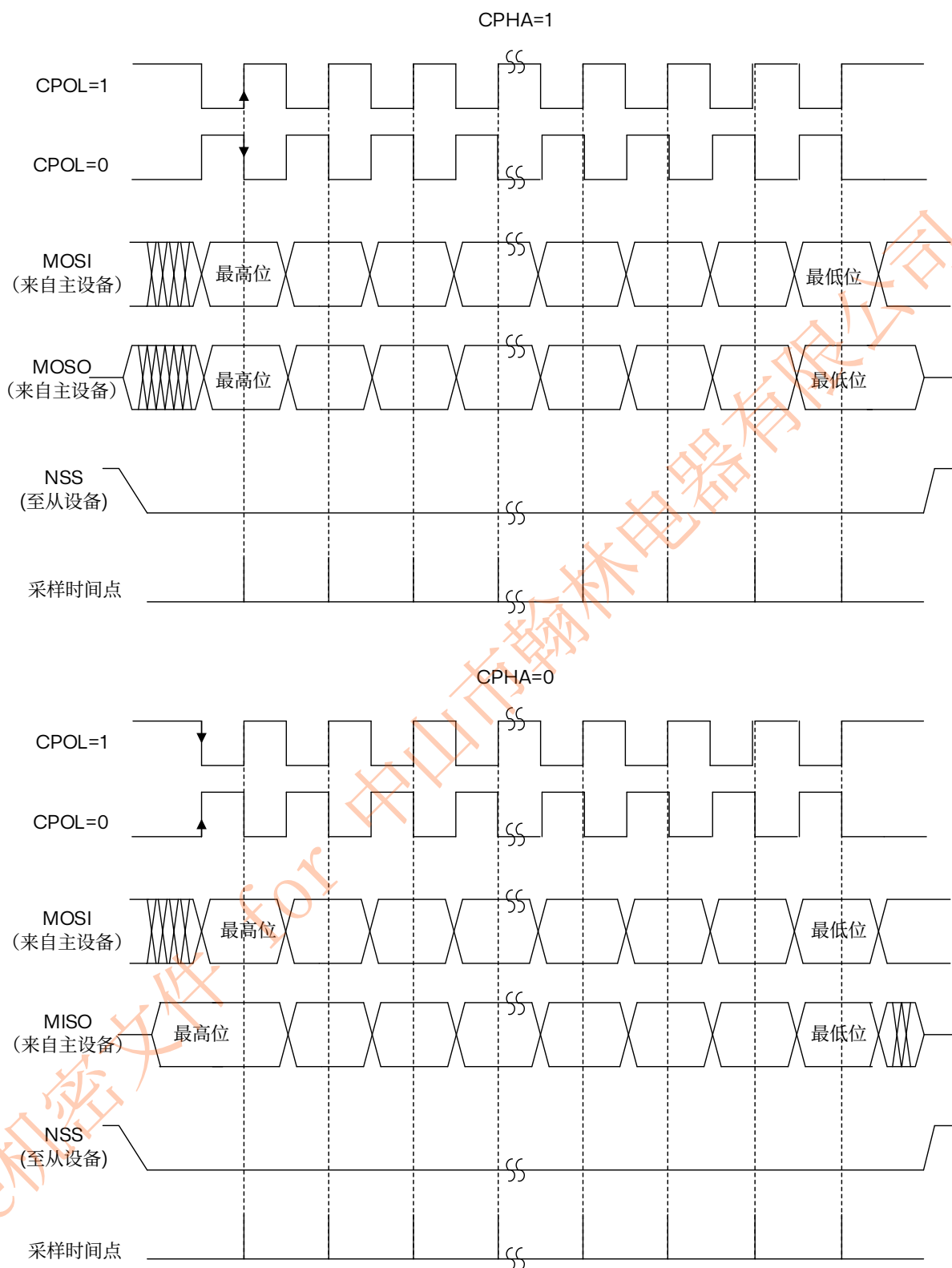


图 33.4 数据时钟时序图

### 数据帧格式

根据 SPL\_CR1 寄存器中的 LSBFIRST 位，输出数据位时可以 MSB 在先也可以 LSB 在先。

根据 SPL\_CR1 寄存器的 DFF 位，每个数据帧可以是 8 位或是 16 位。所选择的数据帧格式对发送和/或接收都有效。

### 33.3.2 配置 SPI 为从模式

在从模式下，SCK 引脚用于接收从主设备来的串行时钟。SPL\_CR1 寄存器中 BR[2:0] 的设置不影响数据传输速率。

注：建议在主设备发送时钟之前使能 SPI 从设备，否则可能会发生意外的数据传输。在通信时钟的第一个边沿到来之前或正在进行的通信结束之前，从设备的数据寄存器必须就绪。在使能从设备和主设备之前，通信时钟的极性必须处于稳定的数值。请按照以下步骤配置 SPI 为从模式：

#### 配置步骤

1. 设置 DFF 位以定义数据帧格式为 8 位或 16 位。
2. 选择 CPOL 和 CPHA 位来定义数据传输和串行时钟之间的相位关系。为保证正确的数据传输，从设备和主设备的 CPOL 和 CPHA 位必须配置成相同的方式。
3. 帧格式（SPL\_CR1 寄存器中的 LSBFIRST 位定义的“MSB 在前”还是“LSB 在前”）必须与主设备相同。
4. 硬件模式下（参考从选择（NSS）脚管理部分），在完整的数据帧（8 位或 16 位）传输过程中，NSS 引脚必须为低电平。在 NSS 软件模式下，设置 SPL\_CR1 寄存器中的 SSM 位并清除 SSI 位。
5. 清除 MSTR 位、设置 SPE 位（SPL\_CR1 寄存器），使相应引脚工作于 SPI 模式下。在这个配置中，MOSI 引脚是数据输入，MISO 引脚是数据输出。

#### 数据发送过程

在写操作中，数据字被并行地写入发送缓冲器。

当从设备收到时钟信号，并且在 MOSI 引脚上出现第一个数据位时，发送过程开始（译注：此时第一个位被发送出去）。余下的位（对于 8 位数据帧格式，还有 7 位；对于 16 位数据帧格式，还有 15 位）被装进移位寄存器。当发送缓冲器中的数据传输到移位寄存器时，SPL\_SP 寄存器的 TXE 标志被设置，如果设置了 SPL\_CR2 寄存器的 TXEIE 位，将会产生中断。

#### 数据接收过程

对于接收器，当数据接收完成时：

- 移位寄存器中的数据传送到接收缓冲器，SPL\_SR 寄存器中的 RXNE 标志被设置。
- 如果设置了 SPL\_CR2 寄存器中的 RXNEIE 位，则产生中断。

在最后一个采样时钟边沿后，RXNE 位被置‘1’，移位寄存器中接收到的数据字节被传送到接收缓冲器。当读 SPL\_DR 寄存器时，SPI 设备返回这个接收缓冲器的数值。

读 SPL\_DR 寄存器时，RXNE 位被清除。

### 33.3.3 配置 SPI 为主模式

在主配置时，在 SCK 脚产生串行时钟。

#### 配置步骤

1. 通过 SPL\_CR1 寄存器的 BR[2:0] 位定义串行时钟波特率。

2. 选择 CPOL 和 CPHA 位，定义数据传输和串行时钟间的相位关系。
3. 设置 DFF 位来定义 8 位或 16 位数据帧格式。
4. 配置 SPL\_CR1 寄存器的 LSBFIRST 位定义帧格式。
5. 如果需要 NSS 引脚工作在输入模式，硬件模式下，在整个数据帧传输期间应把 NSS 脚连接到高电平；在软件模式下，需设置 SPL\_CR1 寄存器的 SSM 位和 SSI 位。如果 NSS 引脚工作在输出模式，则只需设置 SSOE 位。
6. 必须设置 MSTR 位和 SPE 位（只当 NSS 脚被连到高电平，这些位才能保持置位）。在这个配置中，MOSI 引脚是数据输出，而 MISO 引脚是数据输入。

### 数据发送过程

当写入数据至发送缓冲器时，发送过程开始。

在发送第一个数据位时，数据字被并行地（通过内部总线）传入移位寄存器，而后串行地移出到 MOSI 脚上；MSB 在先还是 LSB 在先，取决于 SPL\_CR1 寄存器中的 LSBFIRST 位的设置。数据从发送缓冲器传输到移位寄存器时 TXE 标志将被置位，如果设置了 SPL\_CR1 寄存器中的 TXEIE 位，将产生中断。

### 数据接收过程

对于接收器来说，当数据传输完成时：

- 传送移位寄存器里的数据到接收缓冲器，并且 RXNE 标志被置位。
- 如果设置了 SPL\_CR2 寄存器中的 RXNEIE 位，则产生中断。

在最后采样时钟沿，RXNE 位被设置，在移位寄存器中接收到的数据字被传送到接收缓冲器。读 SPL\_DR 寄存器时，SPI 设备返回接收缓冲器中的数据。

读 SPL\_DR 寄存器将清除 RXNE 位。

一旦传输开始，如果下一个将发送的数据被放进了发送缓冲器，就可以维持一个连续的传输流。在试图写发送缓冲器之前，需确认 TXE 标志应该为 '1'。

注：在 NSS 硬件模式下，从设备的 NSS 输入由 NSS 引脚控制或另一个由软件驱动的 GPIO 引脚控制。

### 33.3.4 配置 SPI 为单工通信

SPI 模块能够以两种配置工作于单工方式：

- 1 条时钟线和 1 条双向数据线；
- 1 条时钟线和 1 条数据线（只接收或只发送）；

#### 1 条时钟线和 1 条双向数据线（BIDIMODE=1）

设置 SPL\_CR1 寄存器中的 BIDIMODE 位而使能此模式。在这个模式下，SCK 引脚作为时钟，主设备使用 MOSI 引脚而从设备使用 MISO 引脚作为数据通信。传输的方向由 SPL\_CR1 寄存器里的 BIDIOE 控制，当这个位是 '1' 的时候，数据线是输出，否则是输入。

#### 1 条时钟和 1 条单向数据线（BIDIMODE=0）

在这个模式下，SPI 模块可以或者作为只发送，或者作为只接收。

- 只发送模式类似于全双工模式（BIDIMODE=0，RXONLY=0）：数据在发送引脚（主模式时是 MOSI、从模式时是 MISO）上传输，而接收引脚（主模式时是 MISO、从模式时是 MOSI）可以作为通用的 I/O 使用。此时，软件不必理会接收缓冲器中的数据（如果读出数据寄存器，它不包含任何接收数据）。
- 在只接收模式，可以通过设置 SPL\_CR2 寄存器的 RXONLY 位而关闭 SPI 的输出功能；此时，发送引脚（主模式时是 MOSI、从模式时是 MISO）被释放，可以作为其它功能使用。配置并使能



SPI 模块为只接收模式的方式是：

- 在主模式时，一旦使能 SPI，通信立即启动，当清除 SPE 位时立即停止当前的接收。在此模式下，不必读取 BSY 标志，在 SPI 通信期间这个标志始终为 ‘1’。
- 在从模式时，只要 NSS 被拉低（或在 NSS 软件模式时，SSI 位为 ‘0’）同时 SCK 有时钟脉冲，SPI 就一直在接收。

### 33.3.5 数据发送与接收过程

#### 接收与发送缓冲器

在接收时，接收到的数据被存放在一个内部的接收缓冲器中；在发送时，在被发送之前，数据将首先被存放在一个内部的发送缓冲器中。

对 SPI\_DR 寄存器的读操作，将返回接收缓冲器的内容；写入 SPI\_DR 寄存器的数据将被写入发送缓冲器中。

#### 主模式下开始传输

- 全双工模式（ $BIDIMODE=0$  并且  $RXONLY=0$ ）
  - 当写入数据到 SPI\_DR 寄存器（发送缓冲器）后，传输开始；
  - 在传送第一位数据的同时，数据被并行地从发送缓冲器传送到 8 位的移位寄存器中，然后按顺序被串行地移位送到 MOSI 引脚上；
  - 与此同时，在 MISO 引脚上接收到的数据，按顺序被串行地移位进入 8 位的移位寄存器中，然后被并行地传送到 SPI\_DR 寄存器（接收缓冲器）中。
- 单向的只接收模式（ $BIDIMODE=0$  并且  $RXONLY=1$ ）
  - SPE=1 时，传输开始；
  - 只有接收器被激活，在 MISO 引脚上接收到的数据，按顺序被串行地移位进入 8 位的移位寄存器中，然后被并行地传送到 SPI\_DR 寄存器（接收缓冲器）中。
- 双向模式，发送时（ $BIDIMODE=1$  并且  $BIDIOE=1$ ）
  - 当写入数据到 SPI\_DR 寄存器（发送缓冲器）后，传输开始；
  - 在传送第一位数据的同时，数据被并行地从发送缓冲器传送到 8 位的移位寄存器中，然后按顺序被串行地移位送到 MOSI 引脚上；
  - 不接收数据。
- 双向模式，接收时（ $BIDIMODE=1$  并且  $BIDIOE=0$ ）
  - SPE=1 并且  $BIDIOE=0$  时，传输开始；
  - 在 MOSI 引脚上接收到的数据，按顺序被串行地移位进入 8 位的移位寄存器中，然后被并行地传送到 SPI\_DR 寄存器（接收缓冲器）中。
  - 不激活发送器，没有数据被串行地送到 MOSI 引脚上。

#### 从模式下开始传输

- 全双工模式（ $BIDIMODE=0$  并且  $RXONLY=0$ ）
  - 当从设备接收到时钟信号并且第一个数据位出现在它的 MOSI 时，数据传输开始，随后的数据位依次移动进入移位寄存器；
  - 与此同时，在传输第一个数据位时，发送缓冲器中的数据被并行地传送到 8 位的移位寄存器，随后被串行地发送到 MISO 引脚上。软件必须保证在 SPI 主设备开始数据传输之前在发送寄存器中写入要发送的数据。



- 单向的只接收模式 (BIDIMODE=0 并且 RXONLY=1)
  - 当从设备接收到时钟信号并且第一个数据位出现在它的 MOSI 时, 数据传输开始, 随后数据位依次移动进入移位寄存器;
  - 不启动发送器, 没有数据被串行地传送到 MISO 引脚上。
- 双向模式, 发送时 (BIDIMODE=1 并且 BIDIOE=1)
  - 当从设备接收到时钟信号并且发送缓冲器中的第一个数据位被传送到 MISO 引脚上的时候, 数据传输开始;
  - 在第一个数据位被传送到 MISO 引脚上的同时, 发送缓冲器中要发送的数据被平行地传送到 8 位的移位寄存器中, 随后被串行地发送到 MISO 引脚上。软件必须保证在 SPI 主设备开始数据传输之前在发送寄存器中写入要发送的数据;
  - 不接收数据。
- 双向模式, 接收时 (BIDIMODE=1 并且 BIDIOE=0)
  - 当从设备接收到时钟信号并且第一个数据位出现在它的 MOSI 时, 数据传输开始;
  - 从 MISO 引脚上接收到的数据被串行地传送到 8 位的移位寄存器中, 然后被平行地传送到 SPI\_DR 寄存器 (接收缓冲器);
  - 不启动发送器, 没有数据被串行地传送到 MISO 引脚上。

#### 处理数据的发送与接收

当数据从发送缓冲器传送到移位寄存器时, 设置 TXE 标志 (发送缓冲器空), 它表示内部的发送缓冲器可以接收下一个数据; 如果在 SPI\_CR2 寄存器中设置了 TXEIE 位, 则此时会产生一个中断; 写入 SPI\_DR 寄存器即可清除 TXE 位。

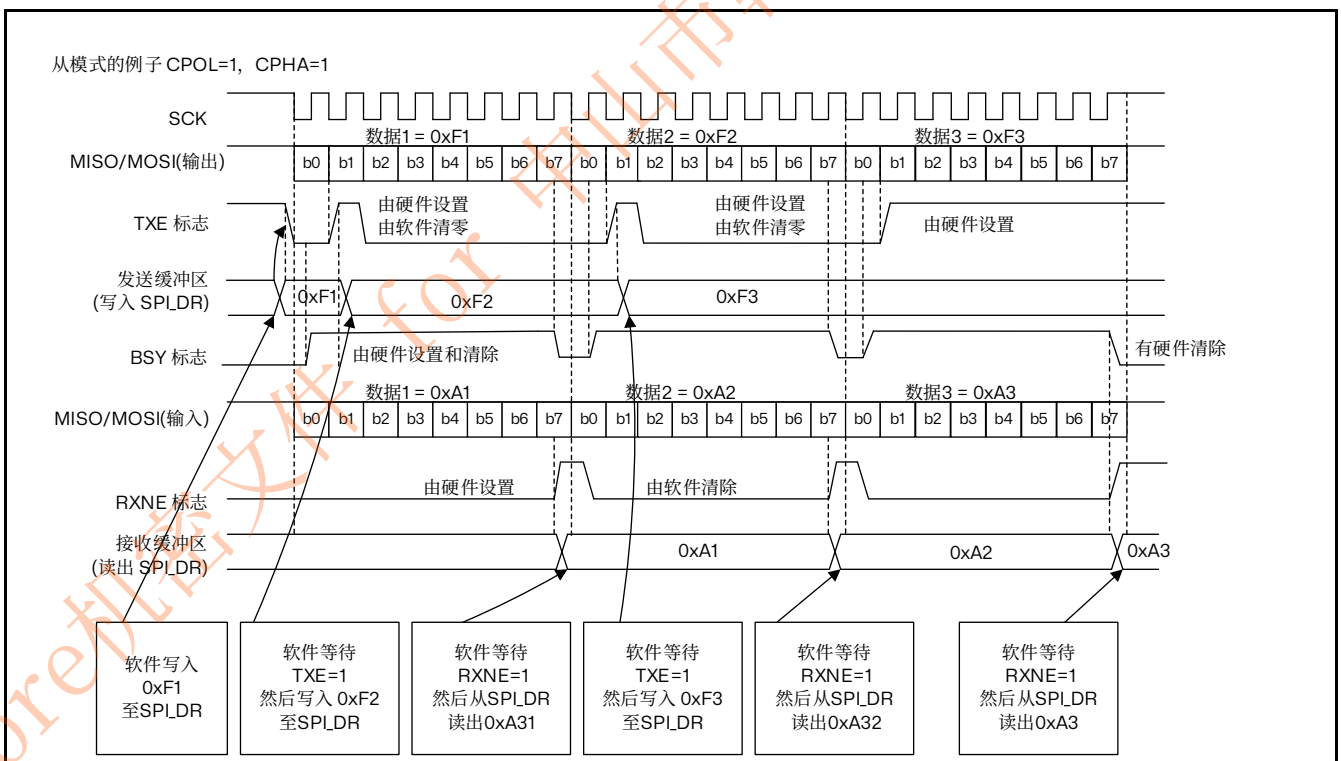
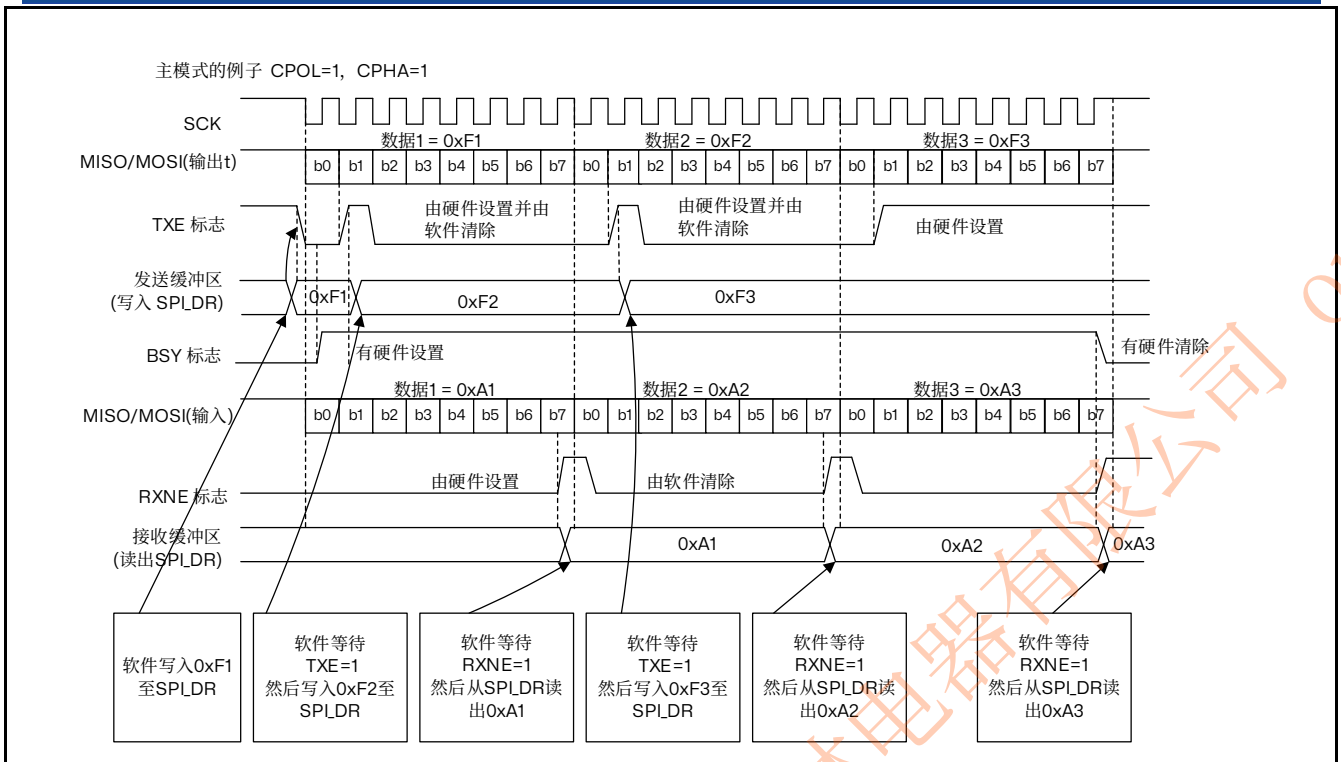
注: 在写入发送缓冲器之前, 软件必须确认 TXE 标志为 '1', 否则新的数据会覆盖已经在发送缓冲器中的数据。

在采样时钟的最后一个边沿, 当数据被从移位寄存器传送到接收缓冲器时, 设置 RXNE 标志 (接收缓冲器非空); 它表示数据已经就绪, 可以从 SPI\_DR 寄存器读出; 如果在 SPI\_CR2 寄存器中设置了 RXNEIE 位, 则此时会产生一个中断; 读出 SPI\_DR 寄存器即可清除 RXNE 标志位。在一些配置中, 传输最后一个数据时, 可以使用 BSY 标志等待数据传输的结束。

#### 主或从模式下 (BIDIMODE=0 并且 RXONLY=0) 全双工发送和接收过程模式

软件必须遵循下述过程, 发送和接收数据:

1. 设置 SPE 位为 '1', 使能 SPI 模块;
2. 在 SPI\_DR 寄存器中写入第一个要发送的数据, 这个操作会清除 TXE 标志;
3. 等待 TXE=1, 然后写入第二个要发送的数据。等待 RXNE=1, 然后读出 SPI\_DR 寄存器并获得第一个接收到的数据, 读 SPI\_DR 的同时清除了 RXNE 位。重复这些操作, 发送后续的数据同时接收 n-1 个数据;
4. 等待 RXNE=1, 然后接收最后一个数据;
5. 等待 TXE=1, 在 BSY=0 之后关闭 SPI 模块。也可以在响应 RXNE 或 TXE 标志的上升沿产生的中断的处理程序中实现这个过程。



只发送过程 (BIDIMODE=0 并且 RXONLY=0)

在此模式下，传输过程可以简要说明如下，使用 BSY 位等待传输的结束：

1. 设置 SPE 位为 ‘1’，使能 SPI 模块；
2. 在 SPI\_DR 寄存器中写入第一个要发送的数据，这个操作会清除 TXE 标志；
3. 等待 TXE=1，然后写入第二个要发送的数据。重复这个操作，发送后续的数据；
4. 写入最后一个数据到 SPI\_DR 寄存器之后，等待 TXE=1；然后等待 BSY=0，这表示最后一个数据的传输已经完成。也可以在响应 TXE 标志的上升沿产生的中断的处理程序中实现这个过程。

注：1. 对于不连续的传输，在写入 SPI\_DR 寄存器的操作与设置 BSY 位之间有 2 个 APB 时钟周期的延迟，因此在只发送模式下，写入最后一个数据后，最好先等待 TXE=1，然后再等待 BSY=0。  
2. 只发送模式下，在传输 2 个数据之后，由于不会读出接收到的数据，SPI\_SR 寄存器中的 OVR 位 ‘1’。（译注：软件不必理会这个 OVR 标志位）

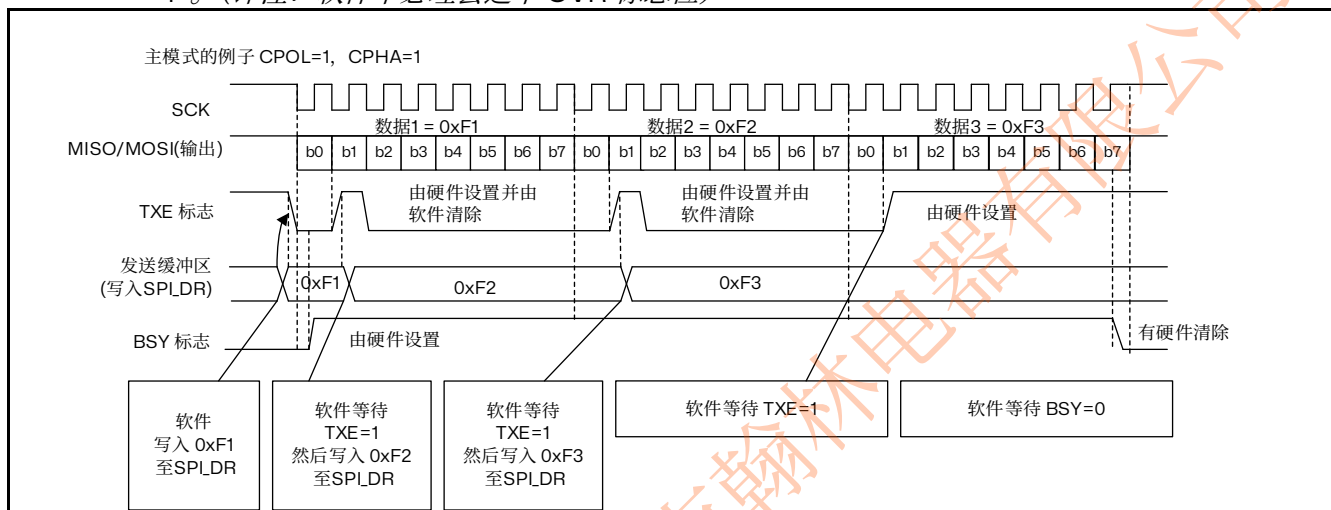


图 33.7 主设备只发送模式 (BIDIMODE=0 并且 RXONLY=0) 下连续传输时，TXE/BSY 变化示意图

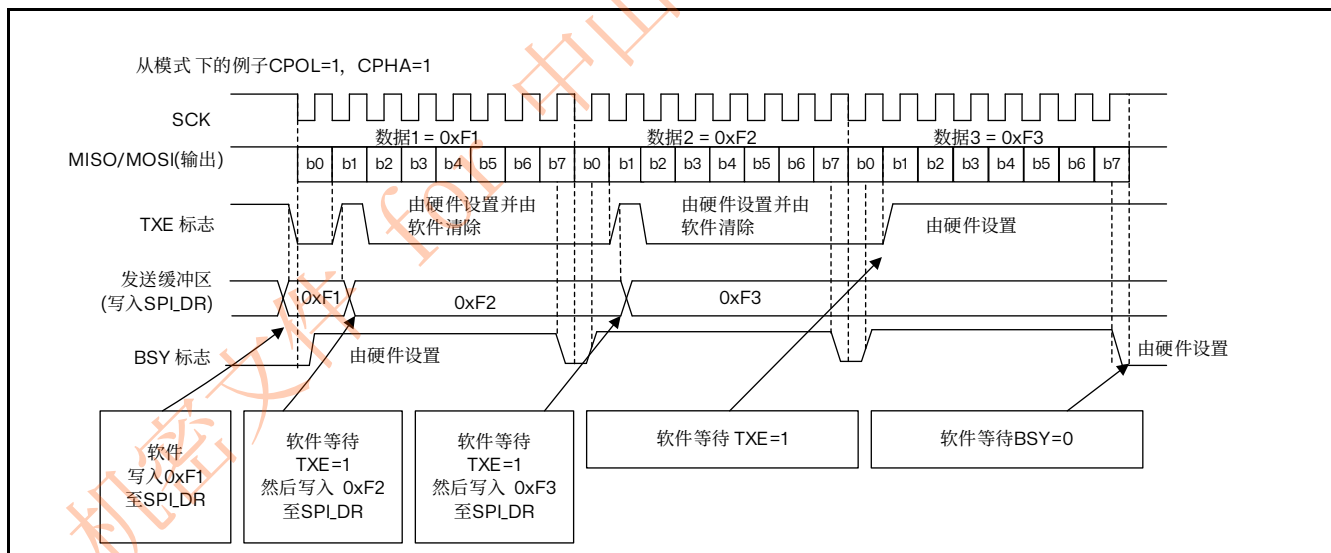


图 33.8 从设备只发送模式 (BIDIMODE=0 并且 RXONLY=0) 下连续传输时，TXE/BSY 变化示意图

### 双向发送过程 (BIDIMODE=1 并且 BIDIOE=1)

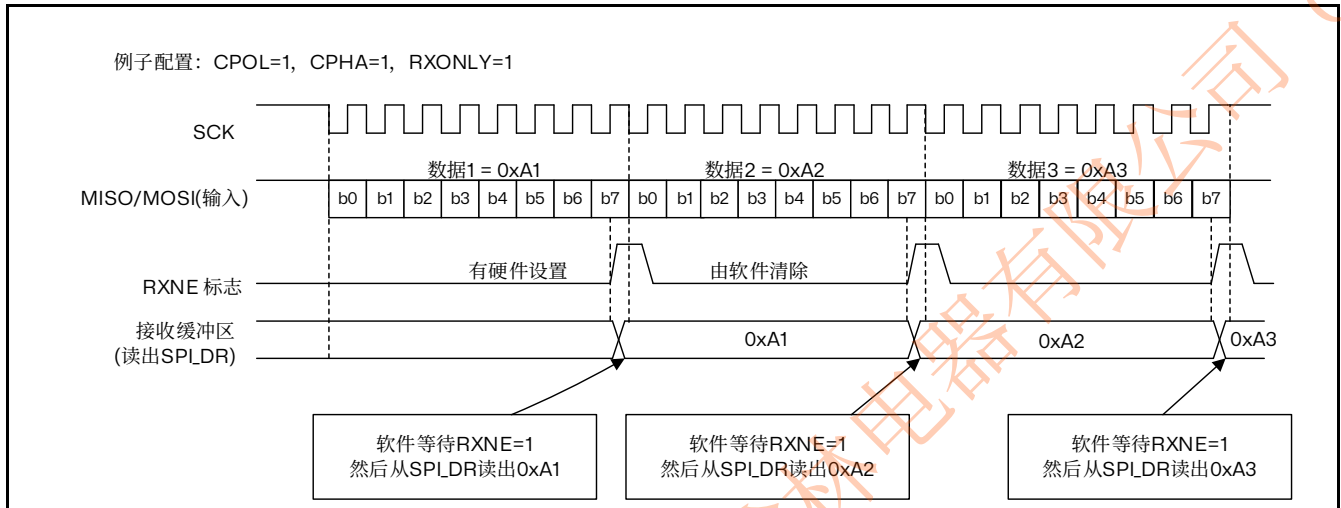
在此模式下，操作过程类似于只发送模式，不同的是：在使能 SPI 模块之前，需要在 SPI\_CR2 寄存器中同时设置 BIDIMODE 和 BIDIOE 位为 ‘1’。

### 单向只接收模式 (BIDIMODE=0 并且 RXONLY=1)

在此模式下，传输过程可以简要说明如下：

1. 在 SPI\_CR2 寄存器中, 设置 RXONLY=1;
2. 设置 SPE=1, 使能 SPI 模块:
  - a) 主模式下, 立刻产生 SCK 时钟信号, 在关闭 SPI (SPE=0) 之前, 不断地接收串行数据;
  - b) 从模式下, 当 SPI 主设备拉低 NSS 信号并产生 SCK 时钟时, 接收串行数据。
3. 等待 RXNE=1, 然后读出 SPI\_DR 寄存器以获得收到的数据 (同时会清除 RXNE 位)。重复这个操作接收所有数据。

也可以在响应 RXNE 标志的上升沿产生的中断的处理程序中实现这个过程。



### 单向接收过程 (BIDIMODE=1 并且 BIDIOE=0)

在此模式下, 操作过程类似于只接收模式, 不同的是: 在使能 SPI 模块之前, 需要在 SPI\_CR2 寄存器中设置 BIDIMODE 为 '1' 并清除 BIDIOE 位为 '0'。

### 连续和非连续传输

当在主模式下发送数据时, 如果软件足够快, 能够在检测到每次 TXE 的上升沿 (或 TXE 中断), 并立即在正在进行的传输结束之前写入 SPI\_DR 寄存器, 则能够实现连续的通信; 此时, 在每个数据项的传输之间的 SPI 时钟保持连续, 同时 BSY 位不会被清除。

如果软件不够快, 则会导致不连续的通信; 这时, 在每个数据传输之间会被清除 (见下图)。

在主模式的只接收模式下 (RXONLY=1), 通信总是连续的, 而且 BSY 标志始终为 '1'。

在从模式下, 通信的连续性由 SPI 主设备决定。不管怎样, 即使通信是连续的, BSY 标志会在每个数据项之间至少有一个 SPI 时钟周期为低。

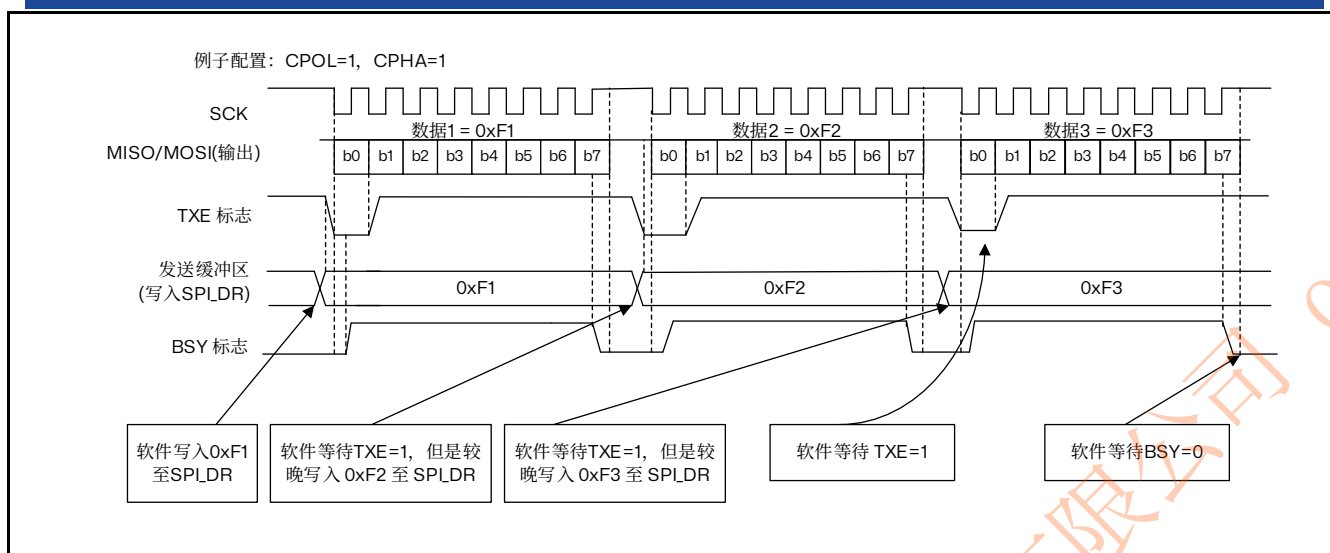


图 33.10 非连续传输发送 (BIDIMODE=0 并且 RXONLY=0) 时, TXE/BSY 变化示意图

### 33.3.6 CRC 计算

CRC 校验用于保证全双工通信的可靠性。数据发送和数据接收分别使用单独的 CRC 计算器。通过对每一个接收位进行可编程的多项式运算来计算 CRC。CRC 的计算是在由 SPL\_CR1 寄存器中 CPHA 和 CPOL 位定义的采样时钟边沿进行的。

**注意:** 该 SPI 接口提供了两种 CRC 计算方法, 取决于所选的发送和/或接收的数据帧格式: 8 位数据帧采用 CR8; 16 位数据帧采用 CRC16。

CRC 计算是通过设置 SPL\_CR1 寄存器中的 CRCEN 位使能的。设置 CRCEN 位时同时复位 CRC 寄存器 (SPL\_RXCRCR 和 SPL\_TXCRCR)。当设置了 SPL\_CR1 的 CRCNEXT 位, SPL\_TXCRCR 的内容将在当前字节发送之后发出。

在传输 SPL\_TXCRCR 的内容时, 如果在移位寄存器中收到的数值与 SPL\_RXCRCR 的内容不匹配, 则 SPL\_SR 寄存器的 CRCERR 标志位被置 1。

如果在 TX 缓冲器中还有数据, CRC 的数值仅在数据字节传输结束后传送。在传输 CRC 期间, CRC 计算器关闭, 寄存器的数值保持不变。

SPI 通信可以通过以下步骤使用 CRC:

- 设置 CPOL、CPHA、LSBFirst、BR、SSM、SSI 和 MSTR 的值;
- 在 SPL\_CRCPR 寄存器输入多项式;
- 通过设置 SPL\_CR1 寄存器 CRCEN 位使能 CRC 计算, 该操作也会清除寄存器 SPL\_RXCRCR 和 SPL\_TXCRCR;
- 设置 SPL\_CR1 寄存器的 SPE 位启动 SPI 功能;
- 启动通信并且维持通信, 直到只剩最后一个字节或者半字;
- 在把最后一个字节或半字写进发送缓冲器时, 设置 SPL\_CR1 的 CRCNext 位, 指示硬件在发送完成最后一个数据之后, 发送 CRC 的数值。在发送 CRC 数值期间, 停止 CRC 计算;
- 当最后一个字节或半字被发送后, SPI 发送 CRC 数值, CRCNext 位被清除。同样, 接收到的 CRC 与 SPL\_RXCRCR 值进行比较, 如果比较不相配, 则设置 SPL\_SR 上的 CRCERR 标志位, 当设置了 SPL\_CR2 寄存器的 ERRRIE 时, 则产生中断。

**注意:** 当 SPI 模块处于从设备模式时, 请注意在时钟稳定之后再使能 CRC 计算, 否则可能会得到错误的 CRC 计算结果。事实上, 只要设置了 CRCEN 位, 只要在 SCK 引脚上有输入时钟, 不管 SPE 位的



状态，都会进行CRC 的计算。

当SPI 时钟频率较高时，用户在发送CRC 时必须小心。在CRC 传输期间，使用CPU 的时间应尽可能少；为了避免在接收最后的数据和CRC 时出错，在发送CRC 过程中应禁止函数调用。必须在发送/接收最后一个数据之前完成设置CRCNEXT 位的操作。

当SPI 时钟频率较高时，因为CPU 的操作会影响SPI 的带宽，建议采用DMA 模式以避免SPI 降低的速度。

当RX32G410 配置为从模式并且使用了NSS 硬件模式，NSS 引脚应该在数据传输和CRC 传输期间保持为低。

当配置SPI 为从模式并且使用CRC 的功能，即使NSS 引脚为高时仍然会执行CRC 的计算（译注：当NSS 信号为高时，如果SCK 引脚上有时钟脉冲，则CRC 计算会继续执行）。例如：当主设备交替地与多个从设备进行通信时，将会出现这种情况（译注：此时要想办法避免CRC 的误操作）。

在不选中一个从设备（NSS 信号为高）转换到选中一个新的从设备（NSS 信号为低）的时候，为了保持主从设备端下次CRC 计算结果的同步，应该清除主从两端的CRC 数值。按照下述步骤清除CRC 数值：

1. 关闭SPI 模块（SPE=0）；
2. 清除CRCEN 位为‘0’；
3. 设置CRCEN 位为‘1’；
4. 使能SPI 模块（SPE=1）

### 33.3.7 状态标志

应用程序通过3 个状态标志可以完全监控SPI 总线的状态。

#### 发送缓冲器空闲标志（TXE）

此标志为‘1’时表明发送缓冲器为空，可以写下一个待发送的数据进入缓冲器中。当写入SPLDR 时，TXE 标志被清除。

#### 接收缓冲器非空（RXNE）

此标志为‘1’时表明在接收缓冲器中包含有效的接收数据。读SPI 数据寄存器可以清除此标志。

#### 忙（Busy）标志

BSY 标志由硬件设置与清除（写入此位无效果），此标志表明SPI 通信层的状态。

当它被设置为‘1’时，表明SPI 正忙于通信，但有一个例外：在主模式的双向接收模式下（MSTR=1、BDM=1 并且BDOE=0），在接收期间BSY 标志保持为低。

在软件要关闭SPI 模块并进入停机模式（或关闭设备时钟）之前，可以使用BSY 标志检测传输是否结束，这样可以避免破坏最后一次传输，因此需要严格按照下述过程执行。

BSY 标志还可以用于在多主系统中避免写冲突。

除了主模式的双向接收模式（MSTR=1、BDM=1 并且BDOE=0），当传输开始时，BSY 标志被置‘1’。

以下情况时此标志将被清除为‘0’：

- 当传输结束（主模式下，如果是连续通信的情况例外）；
- 当关闭SPI 模块；
- 当产生主模式失效（MODF=1）。如果通信不是连续的，则在每个数据项的传输之间，BSY 标志为低。

当通信是连续时：

- 主模式下：在整个传输过程中，BSY 标志保持为高；
- 从模式下：在每个数据项的传输之间，BSY 标志在一个 SPI 时钟周期中为低。

注：不要使用 BSY 标志处理每一个数据项的发送和接收，最好使用 TXE 和 RXNE 标志。

### 33.3.8 关闭 SPI

当通讯结束，可以通过关闭 SPI 模块来终止通讯。清除 SPE 位即可关闭 SPI。

在某些配置下，如果再传输还未完成时，就关闭 SPI 模块并进入停机模式，则可能导致当前的传输被破坏，而且 BSY 标志也变得不可信。

为了避免发生这种情况，关闭 SPI 模块时，建议按照下述步骤操作：

在主或从模式下的全双工模式（BIDIMODE=0，RXONLY=0）

1. 等待 RXNE=1 并接收最后一个数据；
2. 等待 TXE=1；
3. 等待 BSY=0；
4. 关闭 SPI（SPE=0），最后进入停机模式（或关闭该模块的时钟）。

在主或从模式下的单向只发送模式（BIDIMODE=0，RXONLY=0）或双向的发送模式

（BIDIMODE=1，BIDIOE=1）

在 SPI\_DR 寄存器中写入最后一个数据后：

1. 等待 TXE=1；
2. 等待 BSY=0；
3. 关闭 SPI（SPE=0），最后进入停机模式（或关闭该模块的时钟）。

在主或从模式下的单向只接收模式（MSTR=1，BIDIMODE=0，RXONLY=1）或双向的接收模式

（MSTR=1，BIDIMODE=1，BIDIOE=0）

这种情况需要特别地处理，以保证 SPI 不会开始一次新的传输：

1. 等待倒数第二个（第 n-1 个）RXNE=1；
2. 在关闭 SPI（SPE=0）之前等待一个 SPI 时钟周期（使用软件延迟）；
3. 在进入停机模式（或关闭该模块的时钟）之前等待最后一个 RXNE=1。

注：在主模式下的单向只发送模式（MSTR=1，BDM=1，BDOE=0）时，传输过程中 BSY 标志始终为低。

在从模式下的只接收模式（MSTR=0，BIDIMODE=0，RXONLY=1）或双向的接收模式

（MSTR=0，BIDIMODE=1，BIDIOE=0）

1. 可以在任何时候关闭 SPI（SPE=0），SPI 会在当前的传输结束后被关闭；
2. 如果希望进入停机模式，在进入停机模式（或关闭该模块的时钟）之前必须首先等待 BSY=0。

### 33.3.9 利用 DMA 的 SPI 通信

为了达到最大通信速度，需要及时往 SPI 发送缓冲器填数据，同样接收缓冲器中的数据也必须及时读走以防止溢出。为了方便高速率的数据传输，SPI 实现了一种采用简单的请求/应答的 DMA 机制。

当 SPI\_CR2 寄存器上的对应使能位被设置时，SPI 模块可以发出 DMA 传输请求。发送缓冲器和接收缓冲器亦有各自的 DMA 请求。

- 发送时，在每次 TXE 被设置为‘1’时发出 DMA 请求，DMA 控制器则写数据至 SPI\_DR 寄存器，TXE 标志因此而被清除。
- 接收时，在每次 RXNE 被设置为‘1’时发出 DMA 请求，DMA 控制器则从 SPI\_DR 寄存器读出



数据，RXNE 标志因此而被清除。

当只使用 SPI 发送数据时，只需使能 SPI 的发送 DMA 通道。此时，因为没有读取收到的数据，OVR 被置为 ‘1’（译注：软件不必理会这个标志）。

当只使用 SPI 接收数据时，只需使能 SPI 的接收 DMA 通道。

在发送模式下，当 DMA 已经传输了所有要发送的数据（DMA\_ISR 寄存器的 TCIF 标志变为 ‘1’）后，可以通过监视 BSY 标志以确认 SPI 通信结束，这样可以避免在关闭 SPI 或进入停止模式时，破坏最后一个数据的传输。因此软件需要先等待 TXE=1，然后等待 BSY=0。

注：在不连续的通信中，在写数据到 SPI\_DR 的操作与 BSY 位被置为 ‘1’ 之间，有 2 个 APB 时钟周期的延迟，因此，在写完最后一个数据后需要先等待 TXE=1 再等待 BSY=0。

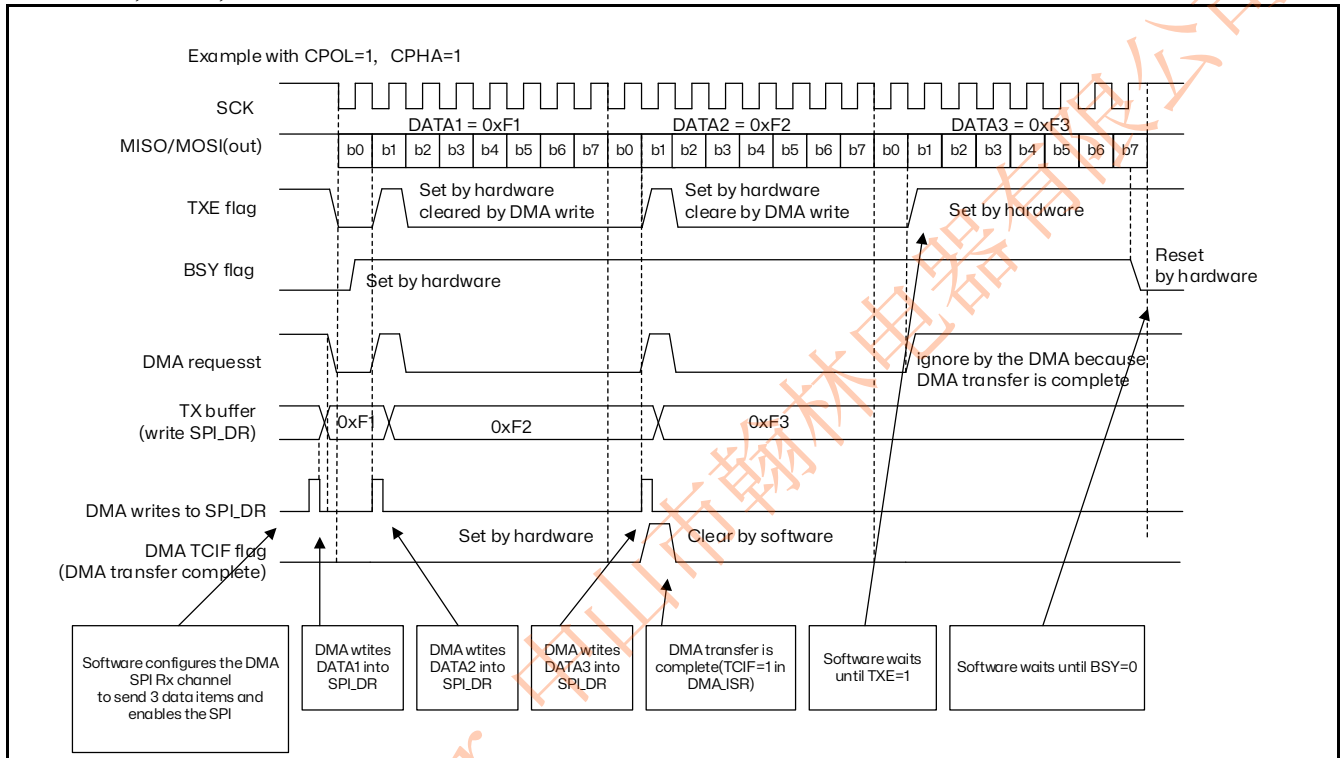


图 33.11 使用 DMA 发送

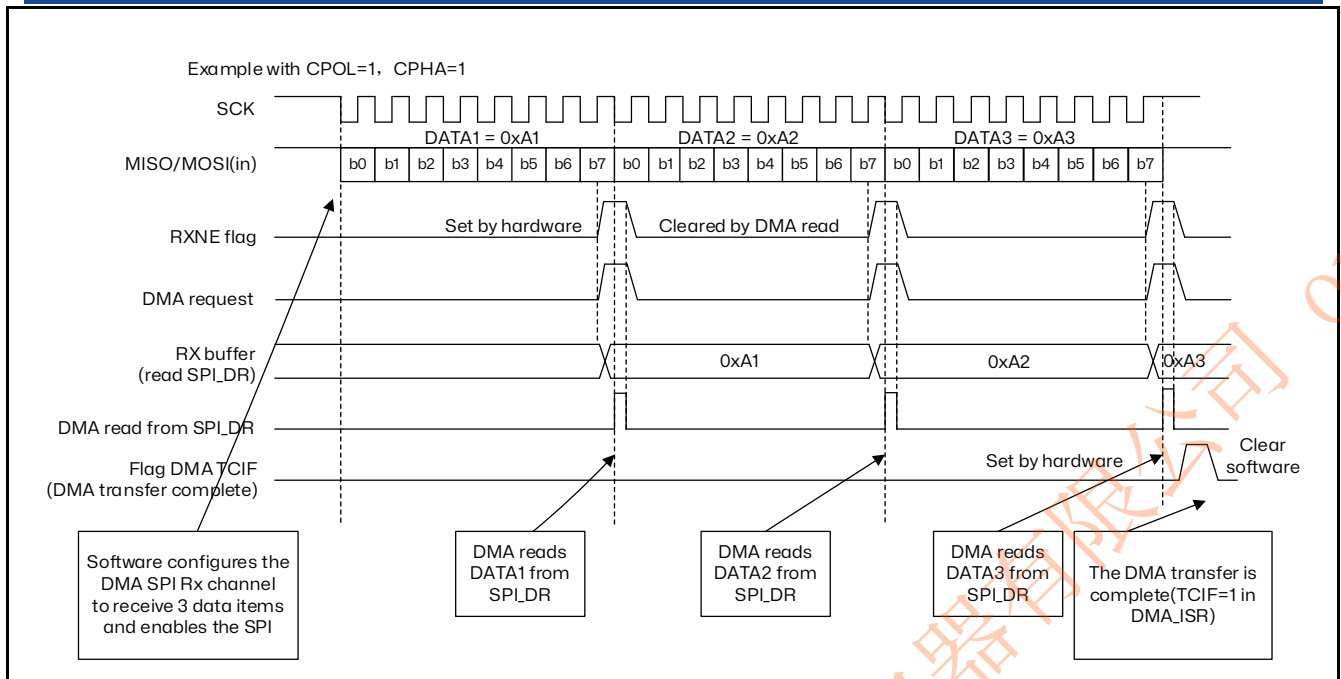


图 33.12 使用 DMA 接收

### 带 CRC 的 DMA 功能

当使能 SPI 使用 CRC 检验并且使能 DMA 模式时，在通信结束时，CRC 字节的发送和接收是自动完成的。

数据和 CRC 传输结束时，SPI\_SR 寄存器的 CRCERR 标志为 ‘1’ 表示在传输期间发生错误。

### 33.3.10 错误标志

#### 主模式失效错误 (MODF)

主模式失效仅发生在：NSS 引脚硬件模式管理下，主设备的 NSS 脚被拉低；或者在 NSS 引脚软件模式管理下，SSI 位被置为 ‘0’ 时；MODF 位被自动置位。主模式失效对 SPI 设备有以下影响：

- MODF 位被置为 ‘1’，如果设置了 ERRIE 位，则产生 SPI 中断；
- SPE 位被清为 ‘0’。这将停止一切输出，并且关闭 SPI 接口；
- MSTR 位被清为 ‘0’，因此强迫此设备进入从模式

下面的步骤用于清除 MODF 位：

1. 当 MODF 位被置为 ‘1’ 时，执行一次对 SPI\_SR 寄存器的读或写操作；
2. 然后写 SPI\_CR1 寄存器。

在有多个 MCU 的系统中，为了避免出现多个从设备的冲突，必须先拉高该主设备的 NSS 脚，再对 MODF 位进行清零。在完成清零之后，SPE 和 MSTR 位可以恢复到它们的原始状态。

出于安全的考虑，当 MODF 位为 ‘1’ 时，硬件不允许设置 SPE 和 MSTR 位。

通常配置下，从设备的 MODF 位不能被置为 ‘1’。然而，在多主配置里，一个设备可以在设置了 MODF 位的情况下，处于从设备模式；此时，MODF 位表示可能出现了多主冲突。中断程序可以执行一个复位或返回到默认状态来从错误状态中恢复。

### 溢出错误

当主设备已经发送了数据字节，而从设备还没有清除前一个数据字节产生的 RXNE 时，即为溢出错

误。当产生溢出错误时：

- OVR 位被置为 ‘1’；当设置了 ERRIE 位时，则产生中断。

此时，接收器缓冲器的数据不是主设备发送的新数据，读 SPI\_DR 寄存器返回的是之前未读的数据，所有随后传送的数据都被丢弃。

依次读出 SPI\_DR 寄存器和 SPI\_SR 寄存器可将 OVR 清除。

### CRC 错误

当设置了 SPI\_CR 寄存器上的 CRCEN 位时，CRC 错误标志用来核对接收数据的有效性。如果移位寄存器中接收到的值（发送方发送的 SPI\_TXCRCR 数值）与接收方 SPI\_RXCRCR 寄存器中的数值不匹配，则 SPI\_SR 寄存器上的 CRCERR 标志被置位为 ‘1’。

### 33.3.11 SPI 中断

表 33.1 SPI 中断请求

| 中断事件      | 事件标志   | 使能控制位  |
|-----------|--------|--------|
| 发送缓冲器空标志  | TXE    | TXEIE  |
| 接收缓冲器非空标志 | RXNE   | RXNEIE |
| 主模式失效事件   | MODF   | ERRIE  |
| 溢出错误      | OVR    |        |
| CRC 标志错误  | CRCERR |        |

### 33.4 SPI 寄存器描述

基地址：

可以用半字（16 位）或字（32 位）的方式操作这些外设寄存器。

#### 33.4.1 SPI 控制寄存器 1 (SPI\_CR1)

地址偏移：0x00

复位值：0x0000 0000

| 15           | 14     | 13    | 12          | 11  | 10         | 9   | 8   | 7            | 6   | 5       | 4  | 3  | 2    | 1    | 0    |
|--------------|--------|-------|-------------|-----|------------|-----|-----|--------------|-----|---------|----|----|------|------|------|
| BIDI<br>MODE | BIDIOE | CRCEN | CRC<br>NEXT | DFF | RX<br>ONLY | SSM | SSI | LSB<br>FIRST | SPE | BR[2:0] |    |    | MSTR | CPOL | CPHA |
| rw           | rw     | rw    | rw          | rw  | rw         | rw  | rw  | rw           | rw  | rw      | rw | rw | rw   | rw   | rw   |

Bit 15 **BIDIMODE**：双向数据模式使能（Bidirectional data mode enable）

0：选择“双线双向”模式；

1：选择“单线双向”模式。

Bit 14 **BIDIOE**：双向模式下的输出使能（Output enable in bidirectional mode）

和 BIDIMODE 位一起决定在“单线双向”模式下数据的输出方向

0：输出禁止（只收模式）；

1：输出使能（只发模式）。

这个“单线”数据线在主设备端为 MOSI 引脚，在从设备端为 MISO 引脚。

Bit 13 **CRCEN**：硬件 CRC 校验使能（Hardware CRC calculation enable）

0：禁止 CRC 计算；

1：启动 CRC 计算。

注：只有在禁止 SPI 时（SPE=0），才能写该位，否则出错。该位只能在全双工模式下

使用。

Bit 12 **CRCNEXT**: 下一个发送 CRC (Transmit CRC next)

0: 下一个发送的值来自发送缓冲区。

1: 下一个发送的值来自发送 CRC 寄存器。

注: 在 SPLDR 寄存器写入最后一个数据后应马上设置该位。

Bit 11 **DFF**: 数据帧格式 (Data frame format)

0: 使用 8 位数据帧格式进行发送/接收;

1: 使用 16 位数据帧格式进行发送/接收。

注: 只有当 SPI 禁止 (SPE=0) 时, 才能写该位, 否则出错。

Bit 10 **RXONLY**: 只接收 (Receive only)

该位和 BIDIMODE 位一起决定在“双线双向”模式下的传输方向。在多个从设备的配置中, 在未被访问的从设备上该位被置 1, 使得只有被访问的从设备有输出, 从而不会造成数据线上数据冲突。

0: 全双工 (发送和接收);

1: 禁止输出 (只接收模式)。

Bit 9 **SSM**: 软件从设备管理 (Software slave management)

当 SSM 被置位时, NSS 引脚上的电平由 SSI 位的值决定。

0: 禁止软件从设备管理;

1: 使能软件从设备管理。

Bit 8 **SSI**: 内部从设备选择 (Internal slave select)

该位只在 SSM 位为 ‘1’ 时有意义。它决定了 NSS 上的电平, 在 NSS 引脚上的 I/O 操作无效。

Bit 7 **LSBFIRST**: 帧格式 (Frame format)

0: 先发送 MSB;

1: 先发送 LSB。

注: 当通信在进行时不能改变该位的值。

Bits 6 **SPE**: SPI 使能 (SPI enable)

0: 禁止 SPI 设备;

1: 开启 SPI 设备。

Bits 5:3 **BR[2:0]**: 波特率控制 (Baud rate control)

000:  $f_{PCLK}/2$

001:  $f_{PCLK}/4$

010:  $f_{PCLK}/8$

011:  $f_{PCLK}/16$

100:  $f_{PCLK}/32$

101:  $f_{PCLK}/64$

110:  $f_{PCLK}/128$

111:  $f_{PCLK}/256$  当通信正在进行的时候, 不能修改这些位。

Bit 2 **MSTR**: 主设备选择 (Master selection)

0: 配置为从设备;

1: 配置为主设备。

注: 当通信正在进行的时候, 不能修改该位。

Bit 1 **CPOL**: 时钟极性 (Clock polarity)

- 0: 空闲状态时, SCK 保持低电平;  
1: 空闲状态时, SCK 保持高电平。  
注: 当通信正在进行的时候, 不能修改该位。

Bit 0 **CPHA**: 时钟相位 (Clock phase)

- 0: 数据采样从第一个时钟边沿开始;  
1: 数据采样从第二个时钟边沿开始。  
注: 当通信正在进行的时候, 不能修改该位。

### 33.4.2 SPI 控制寄存器 2 (SPI\_CR2)

地址偏移: 0x04

复位值: 0x0000 0000

| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7     | 6      | 5     | 4        | 3 | 2 | 1    | 0           |             |
|----------|----|----|----|----|----|---|---|-------|--------|-------|----------|---|---|------|-------------|-------------|
| Reserved |    |    |    |    |    |   |   | TXEIE | RXNEIE | ERRIE | Reserved |   |   | SSOE | TXDMA<br>EN | RXDMA<br>EN |
|          |    |    |    |    |    |   |   | rw    | rw     | rw    |          |   |   | rw   | rw          | rw          |

Bits 15:8 保留, 必须保持为复位值。

Bit 7 **TXEIE**: 发送缓冲区空中断使能 (Tx buffer empty interrupt enable)

- 0: 禁止 TXE 中断;  
1: 允许 TXE 中断, 当 TXE 标志置位为 '1' 时产生中断请求。

Bit 6 **RXNEIE**: 接收缓冲区非空中断使能 (RX buffer not empty interrupt enable)

- 0: 禁止 RXNE 中断;  
1: 允许 RXNE 中断, 当 RXNE 标志置位时产生中断请求。

Bit 5 **ERRIR**: 错误中断使能 (Error interrupt enable)

- 当错误 (CRCERR、OVR、MODF) 产生时, 该位控制是否产生中断  
0: 禁止错误中断;  
1: 允许错误中断。

Bits 4:3 保留, 必须保持为复位值。

Bit 2 **SSOE**: SS 输出使能 (SS output enable)

- 0: 禁止在主模式下 SS 输出, 该设备可以工作在主设备模式;  
1: 设备开启时, 开启主模式下 SS 输出, 该设备不能工作在主设备模式。

Bit 1 **TXDMAEN**: 发送缓冲区 DMA 使能 (Tx buffer DMA enable)

- 当该位被设置时, TXE 标志一旦被置位就发出 DMA 请求  
0: 禁止发送缓冲区 DMA;  
1: 启动发送缓冲区 DMA。

Bit 0 **RXDMAEN**: 接收缓冲区 DMA 使能 (Rx buffer DMA enable)

- 当该位被设置时, RXNE 标志一旦被置位就发出 DMA 请求  
0: 禁止接收缓冲区 DMA;  
1: 启动接收缓冲区 DMA。

### 33.4.3 SPI 状态寄存器 (SPI\_SR)

地址偏移: 0x08

复位值: 0x0000 0002

| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7   | 6   | 5    | 4          | 3   | 2      | 1   | 0    |
|----------|----|----|----|----|----|---|---|-----|-----|------|------------|-----|--------|-----|------|
| Reserved |    |    |    |    |    |   |   | BSY | OVR | MODF | CRC<br>ERR | UDR | CHSIDE | TXE | RXNE |
|          |    |    |    |    |    |   |   |     |     |      |            |     |        |     |      |

|  |   |   |   |       |   |   |   |   |
|--|---|---|---|-------|---|---|---|---|
|  | r | r | r | rc_w0 | r | r | r | r |
|--|---|---|---|-------|---|---|---|---|

Bits 15:8 保留，必须保持为复位值。

Bit 7 **BSY**: 忙标志 (Busy flag)

0: SPI 不忙;

1: SPI 正忙于通信，或者发送缓冲非空。

该位由硬件置位或者复位。

注：使用这个标志时需要特别注意。

Bit 6 **OVR**: 溢出标志 (Overflow flag)

0: 没有出现溢出错误;

1: 出现溢出错误。

该位由硬件置位，由软件序列复位。

Bit 5 **MODF**: 模式错误 (Mode fault)

0: 没有出现模式错误;

1: 出现模式错误。

该位由硬件置位，由软件序列复位。

Bit 4 **CRCERR**: CRC 错误标志 (CRC error flag)

0: 收到的 CRC 值和 SPI\_RXCRCR 寄存器中的值匹配;

1: 收到的 CRC 值和 SPI\_RXCRCR 寄存器中的值不匹配。

该位由硬件置位，由软件写 '0' 而复位。

Bit 3 **UDR**: 下溢标志位 (Underrun flag)

0: 未发生下溢;

1: 发生下溢。

该标志位由硬件置 '1'，由一个软件序列清 '0'。

注：在 SPI 模式下不使用。

Bit 2 **CHSIDE**: 声道 (Channel side)

0: 需要传输或者接收左声道;

1: 需要传输或者接收右声道。

注：在 SPI 模式下不使用。在 PCM 模式下无意义。

Bit 1 **TXE**: 发送缓冲为空 (Transmit buffer empty)

0: 发送缓冲非空;

1: 发送缓冲为空。

Bit 0 **RXNE**: 接收缓冲非空 (Receive buffer not empty)

0: 接收缓冲为空;

1: 接收缓冲非空。

### 33.4.4 SPI 数据寄存器 (SPI\_DR)

地址偏移: 0x0C

复位值: 0x0000

|          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| DR[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw       | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 15:0 **DR[15:0]**: 数据寄存器 (Data register)

待发送或者已经收到的数据

数据寄存器对应两个缓冲区：一个用于写 (发送缓冲); 另外一个用于读 (接收缓冲)。



写操作将数据写到发送缓冲区；读操作将返回接收缓冲区里的数据。

**对 SPI 模式的注释：**根据 SPI\_CR1 的 DFF 位对数据帧格式的选择，数据的发送和接收可以是 8 位或者 16 位的。为保证正确的操作，需要在使能 SPI 之前就确定好数据帧格式。

对于 8 位的数据，缓冲器是 8 位的，发送和接收时只会用到 SPI\_DR[7:0]。在接收时，SPI\_DR[15:8]被强制为 0。

对于 16 位的数据，缓冲器是 16 位的，发送和接收时会用到整个数据寄存器，即 SPI\_DR[15:0]。

### 33.4.5 SPI CRC 多项式寄存器 (SPI\_CRCPR)

地址偏移：0x10

复位值：0x0000 0007

| 15            | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
|---------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| CRCPOLY[15:0] |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| rw            | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

Bits 15:0 **CRCPOLY[15:0]**: CRC 多项式寄存器 (CRC polynomial register)

该寄存器包含了 CRC 计算时用到的多项式。

其复位值为 0x0007，根据应用可以设置其他数值。

### 33.4.6 SPI Rx CRC 寄存器 (SPI\_RXCRCR)

地址偏移：0x14

复位值：0x0000

| 15          | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|-------------|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| RxCRC[15:0] |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| r           | r  | r  | r  | r  | r  | r | r | r | r | r | r | r | r | r | r |

Bits 15:0 **RxCRC[15:0]**: 接收 CRC 寄存器

在使能 CRC 计算时，RxCRC[15:0]中包含了依据收到的字节计算的 CRC 数值。当在 SPI\_CR1 的 CRCEN 位写入 '1' 时，该寄存器被复位。CRC 计算使用 SPI\_CRCPR 中的多项式。

当数据帧格式被设置为 8 位时，仅低 8 位参与计算，并且按照 CRC8 的方法进行；当数据帧格式为 16 位时，寄存器中的所有 16 位都参与计算，并且按照 CRC16 的标准。

注：当 BSY 标志为 '1' 时读该寄存器，将可能读到不正确的数值。

### 33.4.7 SPI Tx CRC 寄存器 (SPI\_TXCRCR)

地址偏移：0x18

复位值：0x0000

| 15          | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|-------------|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| TxCRC[15:0] |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
| r           | r  | r  | r  | r  | r  | r | r | r | r | r | r | r | r | r | r |

Bits 15:0 **TxCRC[15:0]**: 发送 CRC 寄存器

在使能 CRC 计算时，TxCRC[15:0]中包含了依据将要发送的字节计算的 CRC 数值。

当在 SPI\_CR1 中的 CRCEN 位写入 '1' 时，该寄存器被复位。CRC 计算使用 SPI\_CRCPR 中的多项式。

当数据帧格式被设置为 8 位时，仅低 8 位参与计算，并且按照 CRC8 的方法进行；当数据帧格式为 16 位时，寄存器中的所有 16 个位都参与计算，并且按照 CRC16 的标



准。

注：当 BSY 标志为 ‘1’ 时读该寄存器，将可能读到不正确的数值。

iCore机密文件 for 中山市翰林电器有限公司 only

## 34 I<sup>2</sup>C 接口 (I2C)

### 34.1 I<sup>2</sup>C 简介

I<sup>2</sup>C (芯片间) 总线接口连接微控制器和串行 I<sup>2</sup>C 总线。它提供多主机功能, 控制所有 I<sup>2</sup>C 总线特定的时序、协议、仲裁和定时。支持标准和快速两种模式, 同时与 SMBus 2.0 兼容。

I<sup>2</sup>C 模块有多种用途, 包括 CRC 码的生成和校验、SMBus (系统管理总线–System Management Bus) 和 PMBus (电源管理总线–Power Management Bus)。

根据特定设备的需要, 可以使用 DMA 以减轻 CPU 的负担。

### 34.2 I<sup>2</sup>C 主要特点

- 并行总线/I<sup>2</sup>C 总线协议转换器
- 多主机功能: 该模块既可做主设备也可做从设备
- I<sup>2</sup>C 主设备功能
  - 产生时钟
  - 产生起始和停止信号
- I<sup>2</sup>C 从设备功能
  - 可编程的 I<sup>2</sup>C 地址检测
  - 可响应 2 个从地址的双地址能力
  - 停止位检测
- 产生和检测 7 位/10 位地址和广播呼叫
- 支持不同的通讯速度
  - 标准速度 (高达 100 kHz)
  - 快速 (高达 400 kHz)
- 状态标志:
  - 发送器/接收器模式标志
  - 字节发送结束标志
  - I<sup>2</sup>C 总线忙标志
- 错误标志
  - 主模式时的仲裁丢失
  - 地址/数据传输后的应答 (ACK) 错误
  - 检测到错位的起始或停止条件
  - 禁止拉长时钟功能时的上溢或下溢
- 2 个中断向量
  - 1 个中断用于地址/数据通讯成功
  - 1 个中断用于错误
- 可选的拉长时钟功能
- 具单字节缓冲器的 DMA
- 可配置的 PEC (信息包错误检测) 的产生或校验:
  - 发送模式中 PEC 值可以作为最后一个字节传输
  - 用于最后一个接收字节的 PEC 错误校验
- 兼容 SMBus 2.0
  - 25 ms 时钟低超时延时

- 10 ms 主设备累积时钟低扩展时间
- 25 ms 从设备累积时钟低扩展时间
- 带 ACK 控制的硬件 PEC 产生/校验
- 支持地址分辨协议 (ARP)

## ● 兼容 SMBus

注：不是所有产品中都包含上述所有特性。请参考相关的数据手册，确认该产品支持的 I<sup>2</sup>C 功能。

## 34.3 I<sup>2</sup>C 功能描述

I<sup>2</sup>C 模块接收和发送数据，并将数据从串行转换成并行，或并行转换成串行。可以开启或禁止中断。接口通过数据引脚 (SDA) 和时钟引脚 (SCL) 连接到 I<sup>2</sup>C 总线。允许连接到标准 (高达 100kHz) 或快速 (高达 400kHz) 的 I<sup>2</sup>C 总线。

### 34.3.1 模式选择

接口可以下述 4 种模式中的一种运行：

- 从发送器模式
- 从接收器模式
- 主发送器模式
- 主接收器模式

该模块默认地工作于从模式。接口在生成起始条件后自动地从从模式切换到主模式；当仲裁丢失或产生停止信号时，则从主模式切换到从模式。允许多主机功能。

### 通信流

主模式时，I<sup>2</sup>C 接口启动数据传输并产生时钟信号。串行数据传输总是以起始条件开始并以停止条件结束。起始条件和停止条件都是在主模式下由软件控制产生。

从模式时，I<sup>2</sup>C 接口能识别它自己的地址 (7 位或 10 位) 和广播呼叫地址。软件能够控制开启或禁止广播呼叫地址的识别。

数据和地址按 8 位/字节进行传输，高位在前。跟在起始条件后的 1 或 2 个字节是地址 (7 位模式为 1 个字节，10 位模式为 2 个字节)。地址只在主模式发送。

在一个字节传输的 8 个时钟后的第 9 个时钟期间，接收器必须回送一个应答位 (ACK) 给发送器。参考下图。

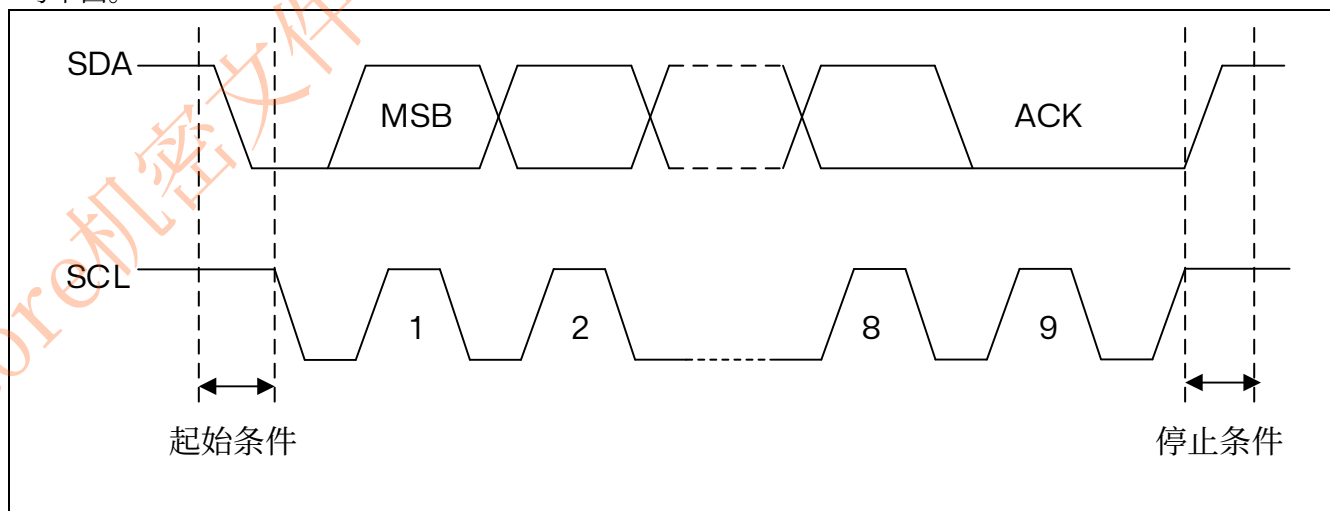


图 34.1 I<sup>2</sup>C 总线协议

软件可以开启或禁止应答 (ACK)，并可以设置 I<sup>2</sup>C 接口的地址 (7 位、10 位地址或广播呼叫地址)。I<sup>2</sup>C 接口的功能框图示于下图。

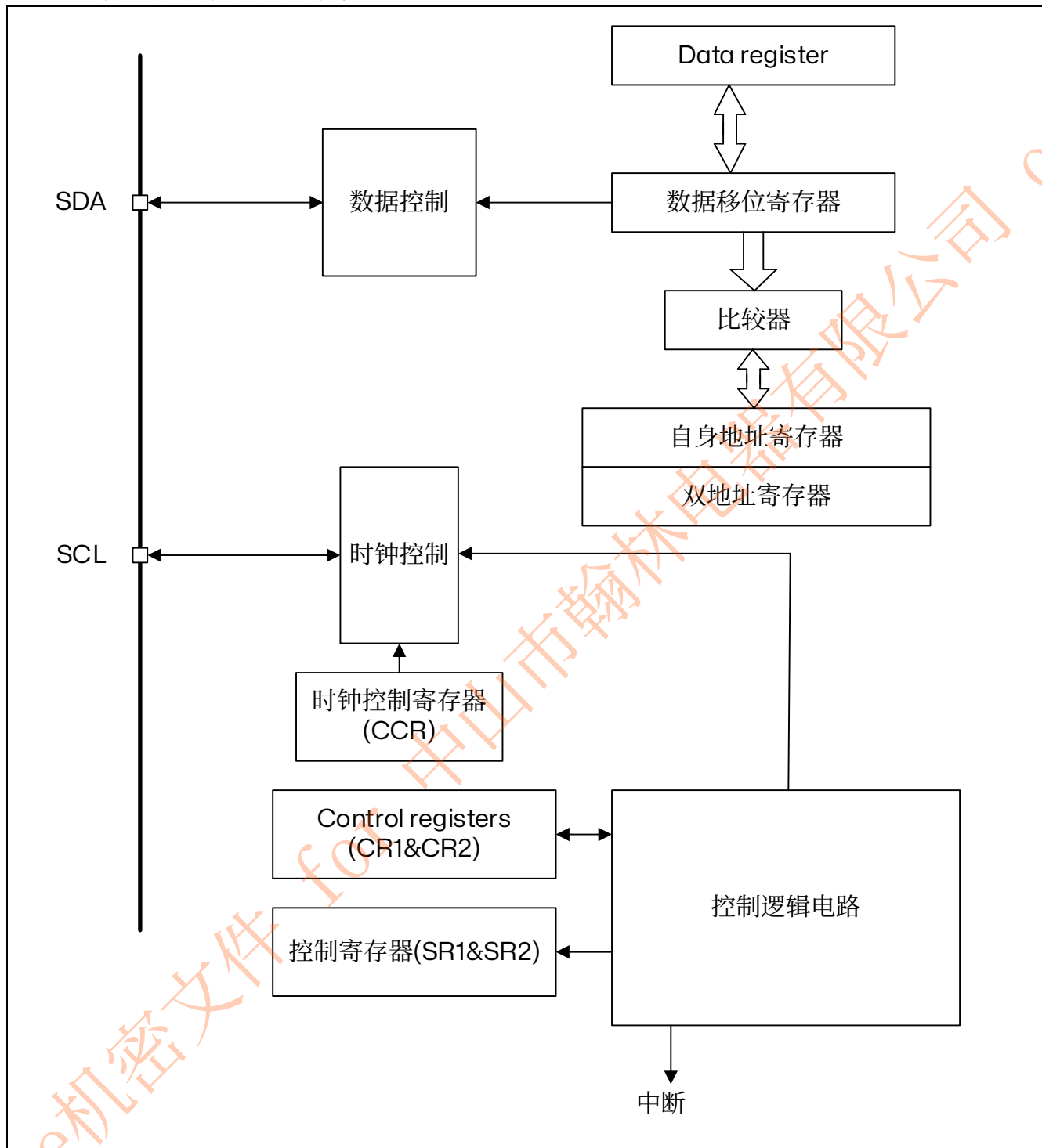


图 34.2 I<sup>2</sup>C 的功能框图

注：在 SMBus 模式下，SMBALERT 是可选信号。如果禁止了 SMBus，则不能使用该信号。

### 34.3.2 I<sup>2</sup>C 从模式

默认情况下，I<sup>2</sup>C 接口总是工作在从模式。从从模式切换到主模式，需要产生一个起始条件。为了产生正确的时序，必须在 I2C\_CR2 寄存器中设定该模块的输入时钟。输入时钟的频率必须

至少是：

- 标准模式下为：2MHz
- 快速模式下为：4MHz

一旦检测到起始条件，在 SDA 线上接收到的地址被送到移位寄存器。然后与芯片自己的地址 OAR1 和 OAR2（当 ENDUAL=1）或者广播呼叫地址（如果 ENGCG=1）相比较。

注：在 10 位地址模式时，比较包括头段序列（11110xx0），其中的 xx 是地址的两个最高有效位。

**头段或地址不匹配：**I<sup>2</sup>C 接口将其忽略并等待另一个起始条件。

**头段匹配（仅 10 位模式）：**如果 ACK 位被置 ‘1’，I<sup>2</sup>C 接口产生一个应答脉冲并等待 8 位从地址。

**地址匹配：**I<sup>2</sup>C 接口产生以下时序：

- 如果 ACK 被置 ‘1’，则产生一个应答脉冲
- 硬件设置 ADDR 位；如果设置了 ITEVFEN 位，则产生一个中断
- 如果 ENDUAL=1，软件必须读 DUALF 位，以确认响应了哪个从地址。

在 10 位模式，接收到地址序列后，从设备总是处于接收器模式。在收到与地址匹配的头序列并且最低位为 ‘1’（即 11110xx1）后，当接收到重复的起始条件时，将进入发送器模式。

在从模式下 TRA 位指示当前是处于接收器模式还是发送器模式。

### 从发送器

在接收到地址和清除 ADDR 位后，从发送器将字节从 DR 寄存器经由内部移位寄存器发送到 SDA 线上。

从设备保持 SCL 为低电平，直到 ADDR 位被清除并且待发送数据已写入 DR 寄存器。（见下图中的 EV1 和 EV3）。

当收到应答脉冲时：

- TxE 位被硬件置位，如果设置了 ITEVFEN 和 ITBUFEN 位，则产生一个中断。

如果 TxE 位被置位，但在下一个数据发送结束之前没有新数据写入到 I2C\_DR 寄存器，则 BTF 位被置位，在清除 BTF 之前 I<sup>2</sup>C 接口将保持 SCL 为低电平；读出 I2C\_SR1 之后再写入 I2C\_DR 寄存器将清除 BTF 位。

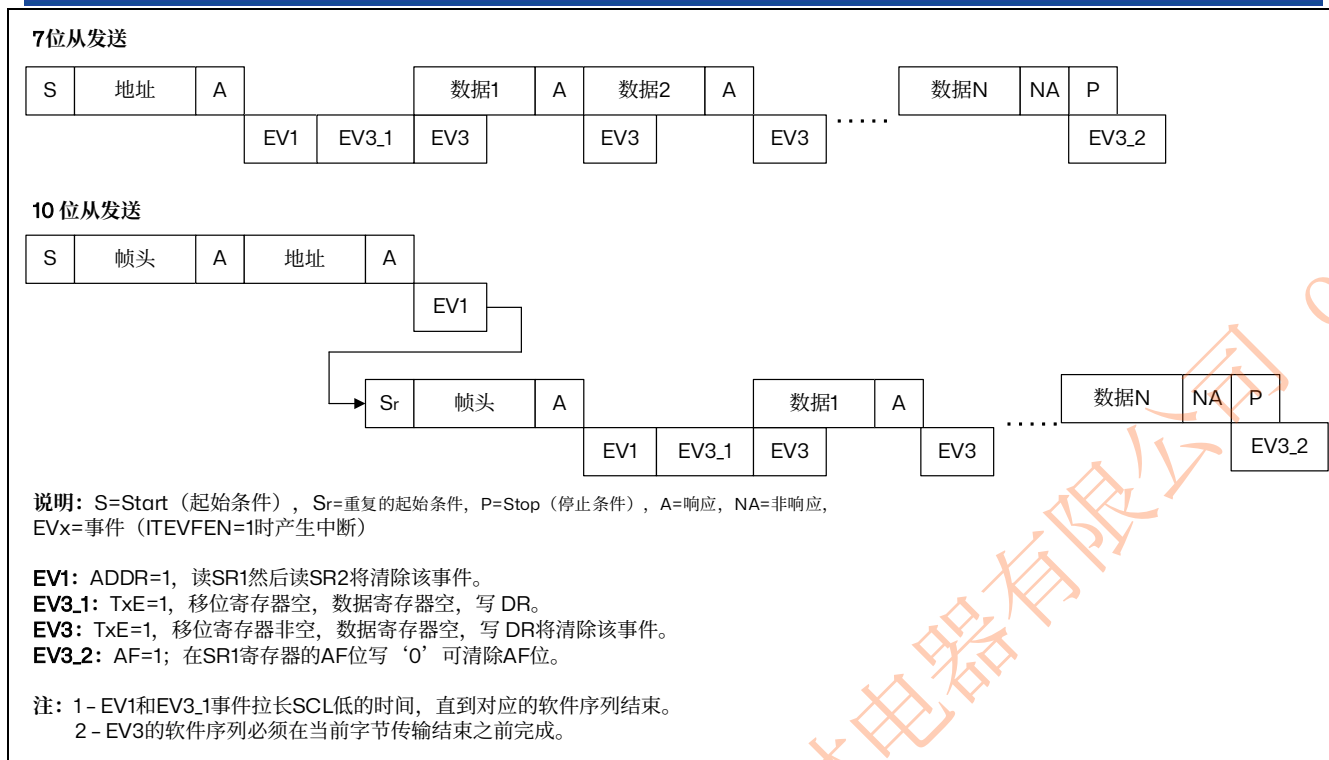


图 34.3 从发送器的传送序列图

### 从接收器

在接收到地址并清除 ADDR 后, 从接收器将通过内部移位寄存器从 SDA 线接收到的字节存进 DR 寄存器。I<sup>2</sup>C 接口在接收到每个字节后都执行下列操作:

- 如果设置了 ACK 位, 则产生一个应答脉冲
- 硬件设置 RxNE=1。如果设置了 ITEVFEN 和 ITBUFEN 位, 则产生一个中断。

如果 RxNE 被置位, 并且在接收新的数据结束之前 DR 寄存器未被读出, BTF 位被置位, 在清除 BTF 之前 I<sup>2</sup>C 接口将保持 SCL 为低电平; 读出 I2C\_SR1 之后再写入 I2C\_DR 寄存器将清除 BTF 位。(见下图)。

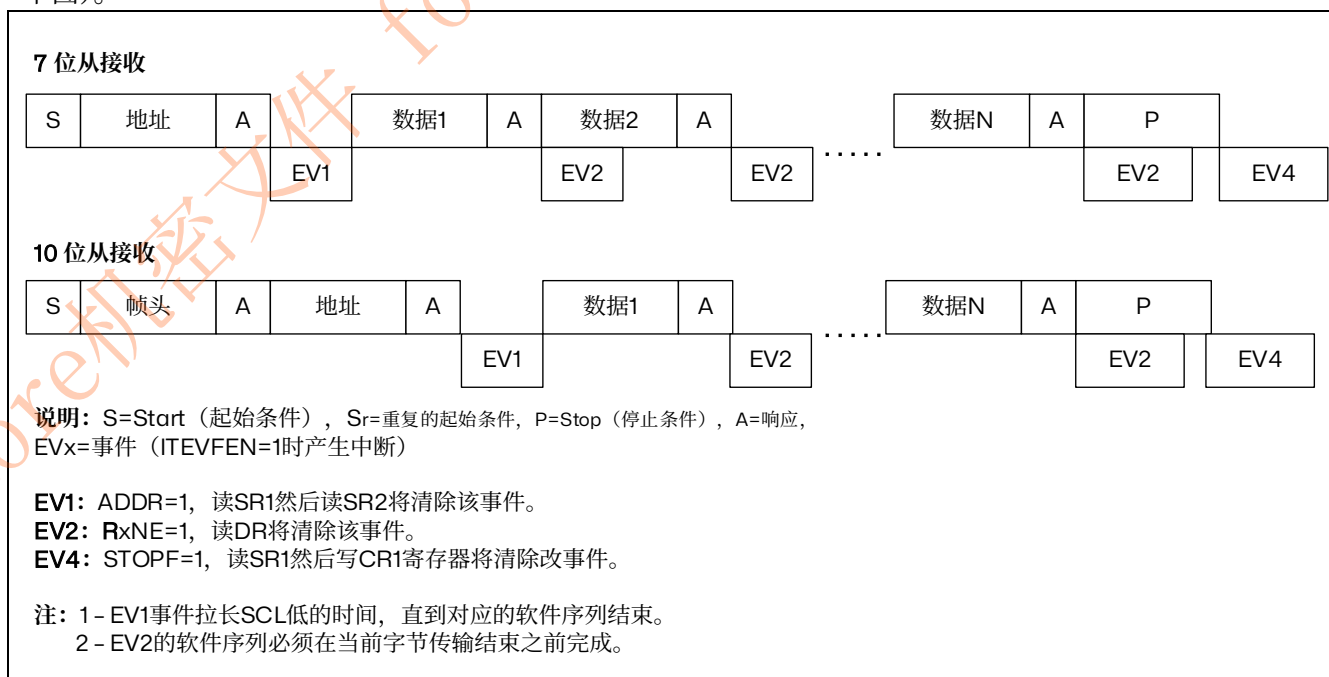


图 34.4 从接收器的传送序列图

### 关闭从通信

在传输完最后一个数据字节后，主设备产生一个停止条件，I<sup>2</sup>C 接口检测到这一条件时：

- 设置 STOPF=1，如果设置了 ITEVFEN 位，则产生一个中断。

然后 I<sup>2</sup>C 接口等待读 SR1 寄存器，再写 CR1 寄存器。（见上图的 EV4）。

### 34.3.3 I<sup>2</sup>C 主模式

在主模式时，I<sup>2</sup>C 接口启动数据传输并产生时钟信号。串行数据传输总是以起始条件开始并以停止条件结束。当通过 START 位在总线上产生了起始条件，设备就进入了主模式。

以下是主模式所要求的操作顺序：

- 在 I2C\_CR2 寄存器中设定该模块的输入时钟以产生正确的时序
- 配置时钟控制寄存器
- 配置上升时间寄存器
- 编程 I2C\_CR1 寄存器启动外设
- 置 I2C\_CR1 寄存器中的 START 位为 1，产生起始条件

I<sup>2</sup>C 模块的输入时钟频率必须至少是：

- 标准模式下为：2MHz
- 快速模式下为：4MHz

### 起始条件

当 BUSY=0 时，设置 START=1，I<sup>2</sup>C 接口将产生一个开始条件并切换至主模式（M/SL 位置位）。

注：在主模式下，设置 START 位将在当前字节传输完后由硬件产生一个重新开始条件。

一旦发出开始条件：

- SB 位被硬件置位，如果设置了 ITEVFEN 位，则会产生一个中断。

然后主设备等待读 SR1 寄存器，紧接着将从地址写入 DR 寄存器。

### 从地址的发送

从地址通过内部移位寄存器被送到 SDA 线上。

- 在 10 位地址模式时，发送一个头段序列产生以下事件：
  - ADD10 位被硬件置位，如果设置了 ITEVFEN 位，则产生一个中断。  
然后主设备等待读 SR1 寄存器，再将第二个地址字节写入 DR 寄存器
  - ADDR 位被硬件置位，如果设置了 ITEVFEN 位，则产生一个中断。  
随后主设备等待一次读 SR1 寄存器，跟着读 SR2 寄存器。
- 在 7 位地址模式时，只需送出一个地址字节。一旦该地址字节被送出，
  - ADDR 位被硬件置位，如果设置了 ITEVFEN 位，则产生一个中断。  
随后主设备等待一次读 SR1 寄存器，跟着读 SR2 寄存器。

根据送出从地址的最低位，主设备决定进入发送器模式还是进入接收器模式。

- 在 7 位地址模式时，
  - 要进入发送器模式，主设备发送从地址时置最低位为 ‘0’。
  - 要进入接收器模式，主设备发送从地址时置最低位为 ‘1’。
- 在 10 位地址模式时



- 要进入发送器模式，主设备先送头字节（11110xx0），然后送最低位为‘0’的从地址。（这里xx代表10位地址中的最高2位。）
- 要进入接收器模式，主设备先送头字节（11110xx0），然后送最低位为‘1’的从地址。然后再重新发送一个开始条件，后面跟着头字节（11110xx1）（这里xx代表10位地址中的最高2位。）

TRA 位指示主设备是在接收器模式还是发送器模式。

## 主发送器

在发送了地址和清除了 ADDR 位后，主设备通过内部移位寄存器将字节从 DR 寄存器发送到 SDA 线上。

主设备等待，直到 TxE 被清除。

当收到应答脉冲时：

- TxE 位被硬件置位，如果设置了 INEVFEN 和 ITBUFEN 位，则产生一个中断。

如果 TxE 被置位并且在上一次数据发送结束之前没有写新的数据字节到 DR 寄存器，则 BTF 被硬件置位，在清除 BTF 之前 I<sup>2</sup>C 接口将保持 SCL 为低电平；读出 I2C\_SR1 之后再写入 I2C\_DR 寄存器将清除 BTF 位。

## 关闭通信

在 DR 寄存器中写入最后一个字节后，通过设置 STOP 位产生一个停止条件，然后 I<sup>2</sup>C 接口将自动回到从模式（M/S 位清除）。

*注：当 TxE 或 BTF 位置位时，停止条件应安排在出现 EV8\_2 事件时。*

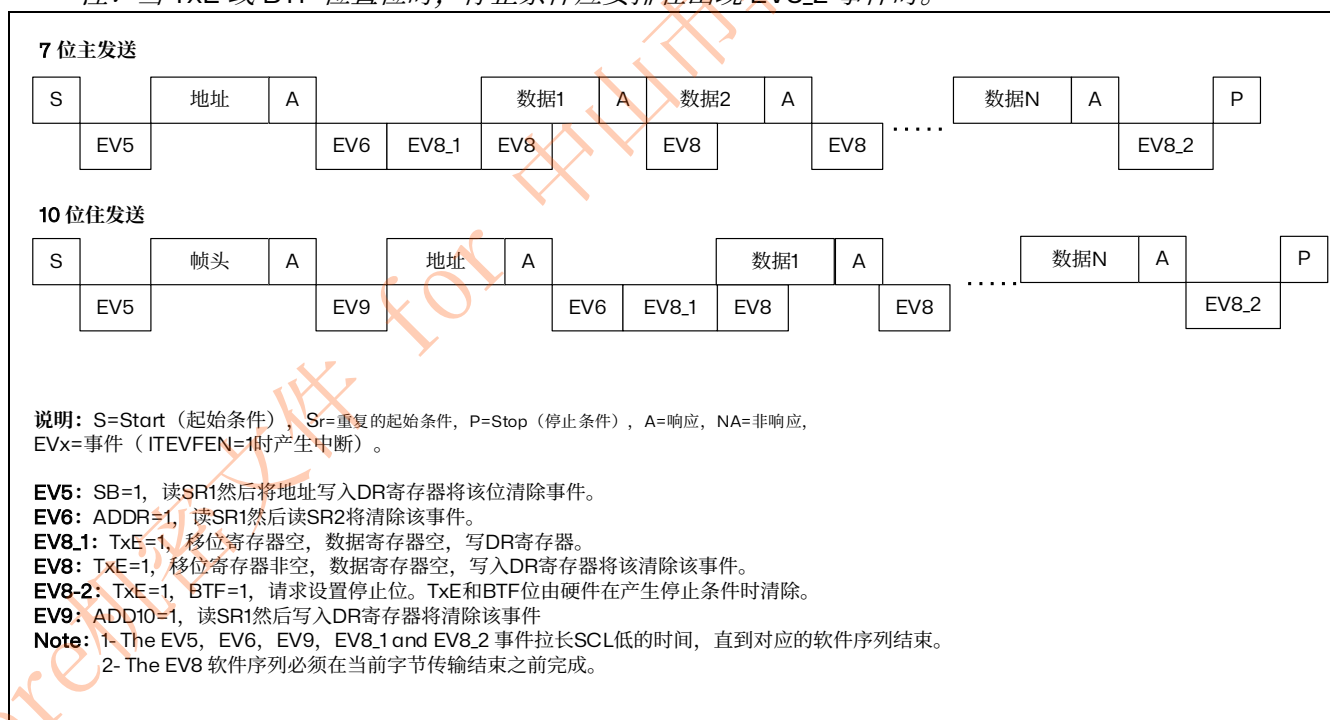


图 34.5 主发送器传送序列图

## 主接收器

在发送地址和清除 ADDR 之后，I<sup>2</sup>C 接口进入主接收器模式。在此模式下 I<sup>2</sup>C 接口从 SDA 线接收数

据字节，并通过内部移位寄存器送至 DR 寄存器。在每个字节后，I<sup>2</sup>C 接口依次执行以下操作：

- 如果 ACK 位被置位，发出一个应答脉冲。
- 硬件设置 RxNE=1，如果设置了 INEVFEN 和 ITBUFEN 位，则会产生一个中断。

如果 RxNE 位被置位，并且在接收新数据结束前，DR 寄存器中的数据没有被读走，硬件将设置 BTF=1，在清除 BTF 之前 I<sup>2</sup>C 接口将保持 SCL 为低电平；读出 I2C\_SR1 之后再读出 I2C\_DR 寄存器将清除 BTF 位。

## 关闭通信

主设备在从从设备接收到最后一个字节后发送一个 NACK。接收到 NACK 后，从设备释放对 SCL 和 SDA 线的控制；主设备就可以发送一个停止/重起始条件。

- 为了在收到最后一个字节后产生一个 NACK 脉冲，在读倒数第二个数据字节之后（在倒数第二个 RxNE 事件之后）必须清除 ACK 位。
- 为了产生一个停止/重起始条件，软件必须在读倒数第二个数据字节之后（在倒数第二个 RxNE 事件之后）设置 STOP/START 位。
- 只接收一个字节时，刚好在 EV6 之后（EV6\_1 时，清除 ADDR 之后）要关闭应答和停止条件的产生位。

在产生了停止条件后，I<sup>2</sup>C 接口自动回到从模式（M/SL 位被清除）。

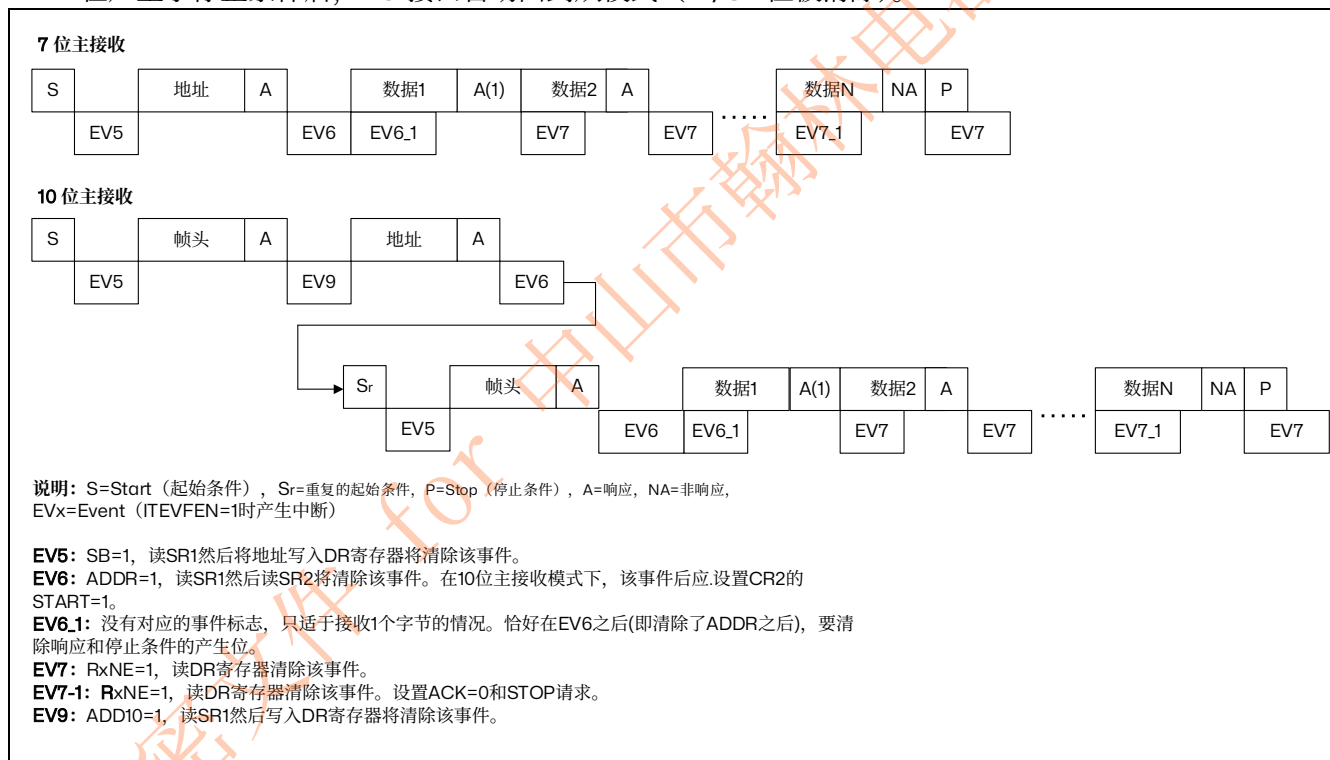


图 34.6 主接收器传送序列图

1. 如果收到一个单独的字节，则是 NA。
2. EV5、EV6 和 EV9 事件拉长 SCL 低电平，直到对应的软件序列结束。
3. EV7 的软件序列必须在当前字节传输结束前完成。
4. EV6\_1 或 EV7\_1 的软件序列 必须在当前传输字节的 ACK 脉冲之前完成。

## 34.3.4 错误条件

以下条件可能造成通讯失败。

### 总线错误 (BERR)

在一个地址或数据字节传输期间，当 I<sup>2</sup>C 接口检测到一个外部的停止或起始条件则产生总线错误。此时：

- BERR 位被置位为 ‘1’；如果设置了 ITERREN 位，则产生一个中断；
- 在从模式情况下，数据被丢弃，硬件释放总线：
  - 如果是错误的开始条件，从设备认为是一个重启动，并等待地址或停止条件。
  - 如果是错误的停止条件，从设备按正常的停止条件操作，同时硬件释放总线。
- 在主模式情况下，硬件不释放总线，同时不影响当前的传输状态。此时由软件决定是否要中止当前的传输。

### 应答错误 (AF)

当接口检测到一个无应答位时，产生应答错误。此时：

- AF 位被置位，如果设置了 ITERREN 位，则产生一个中断；
- 当发送器接收到一个 NACK 时，必须复位通讯：
  - 如果是处于从模式，硬件释放总线。
  - 如果是处于主模式，软件必须生成一个停止条件。

### 仲裁丢失 (ARLO)

当 I<sup>2</sup>C 接口检测到仲裁丢失时产生仲裁丢失错误，此时：

- ARLO 位被硬件置位，如果设置了 ITERREN 位，则产生一个中断；
- I<sup>2</sup>C 接口自动回到从模式 (M/SL 位被清除)。当 I<sup>2</sup>C 接口丢失了仲裁，则它无法在同一个传输中响应它的从地址，但它可以在赢得总线的主设备发送重起始条件之后响应；
- 硬件释放总线。

### 过载/欠载错误 (OVR)

在从模式下，如果禁止时钟延长，I<sup>2</sup>C 接口正在接收数据时，当它已经接收到一个字节 (RxNE=1)，但在 DR 寄存器中前一个字节数据还没有被读出，则发生过载错误。此时：

- 最后接收的数据被丢弃；
- 在过载错误时，软件应清除 RxNE 位，发送器应该重新发送最后一次发送的字节。

在主模式下，如果禁止时钟延长，I<sup>2</sup>C 接口正在发送数据时，在下一个字节的时钟到达之前，新的数据还未写入 DR 寄存器 (TxNE=1)，则发生欠载错误。此时：

- 在 DR 寄存器中的前一个字节将被重复发出；
- 用户应该确定在发生欠载错时，接收端应丢弃重复接收到的数据。发送端应按 I<sup>2</sup>C 总线标准在规定的时间内更新 DR 寄存器。

在发送第一个字节时，必须在清除 ADDR 之后并且第一个 SCL 上升沿之前写入 DR 寄存器；如果不能做到这点，则接收方应该丢弃第一个数据。

### 34.3.5 SDA/SCL 线控制

- 如果允许时钟延长：
  - 发送器模式：如果 TxNE=1 且 BTF=1：I<sup>2</sup>C 接口在传输前保持时钟线为低，以等待软件读取 SR1，然后把数据写进数据寄存器（缓冲器和移位寄存器都是空的）。
  - 接收器模式：如果 RxNE=1 且 BTF=1：I<sup>2</sup>C 接口在接收到数据字节后保持时钟线为低，以等

待软件读 SR1，然后读数据寄存器 DR（缓冲器和移位寄存器都是满的）。

- 如果在从模式中禁止时钟延长：
  - 如果 RxNE=1，在接收到下个字节前 DR 还没有被读出，则发生过载错。接收到的最后一个字节丢失。
  - 如果 TxE=1，在必须发送下个字节之前却没有新数据写进 DR，则发生欠载错。相同的字节将被重复发出。
  - 不控制重复写冲突。

### 34.3.6 SMBus

#### 介绍

系统管理总线（SMBus）是一个双线接口。通过它，各设备之间以及设备与系统的其他部分之间可以互相通信。它基于 I<sup>2</sup>C 操作原理。SMBus 为系统和电源管理相关的任务提供一条控制总线。

一个系统利用 SMBus 可以和多个设备互传信息，而不需使用独立的控制线路。

系统管理总线（SMBus）标准涉及三类设备。从设备：接收或响应命令的设备。主设备：用来发送命令、产生时钟和终止发送的设备。主机：一种专用的主设备，它提供与系统 CPU 的主接口。主机必须具有主-从机功能并且必须支持 SMBus 提醒协议。一个系统里只允许有一个主机。

#### SMBus 和 I<sup>2</sup>C 之间的相似点

- 2 条线的总线协议（1 个时钟，1 个数据）+ 可选的 SMBus 提醒线；
- 主-从通信，主设备提供时钟；
- 多主机功能
- SMBus 数据格式类似于 I<sup>2</sup>C 的 7 位地址格式；

#### SMBus 和 I<sup>2</sup>C 之间的不同点

下表列出了 SMBus 和 I<sup>2</sup>C 的不同点。

表 34.1 SMBus 与 I<sup>2</sup>C 的比较

| SMBus               | I <sup>2</sup> C   |
|---------------------|--------------------|
| 最大传输速度 100kHz       | 最大传输速度 400kHz      |
| 最小传输速度 10kHz        | 无最小传输速度            |
| 35ms 时钟低超时          | 无时钟超时              |
| 固定的逻辑电平             | 逻辑电平由 VDD 决定       |
| 不同的地址类型（保留的、动态的等）   | 7 位、10 位和广播呼叫从地址类型 |
| 不同的总线协议（快速命令、处理呼叫等） | 无总线协议              |

#### SMBus 应用用途

利用系统管理总线，设备可提供制造商信息，告诉系统它的型号/部件号，保存暂停事件的状态，报告不同类型的错误，接收控制参数，和返回它的状态。SMBus 为系统和电源管理相关的任务提供控制总线。

#### 设备标识

在系统管理总线上，任何一个作为从模式的设备都有一个唯一的地址，叫做从地址。

## 总线协议

SMBus 技术规范支持 9 个总线协议。有关这些协议的详细资料和 SMBus 地址类型。

### 地址解析协议 (ARP)

通过给每个从设备动态地分配一个新的唯一地址，可以解决 SMBus 的从地址冲突。地址解析协议 (ARP) 具有以下特性：

- 使用标准 SMBus 物理层仲裁机制分配地址；
- 当设备维持供电期间，分配的地址仍保持不变，也允许设备在断电后保留其地址。
- 在地址分配后，没有额外的 SMBus 的打包开销（也就是说访问分配地址的设备与访问固定地址的设备所用时间是一样的）；
- 任何一个 SMBus 主设备可以遍历总线。

### 唯一的设备标识符 (UDID)

为了分配地址，需要一种区分每个设备的机制，每个设备必须拥有一个唯一的设备标识符。

### SMBus 提醒模式

SMBus 提醒是一个带中断线的可选信号，用于那些希望扩展它们的控制能力而牺牲一个引脚的设备。SMBALERT 和 SCL、SDA 信号一样，是一种线与信号。SMBALERT 通常和 SMBus 广播呼叫地址一起使用。与 SMBus 有关的消息为 2 字节。

一个只具有从功能的设备，可以通过设置 I2C\_CR1 寄存器上的 ALERT 位，使用 SMBALERT 给主机发信号表示它希望进行通信。主机处理该中断并通过提醒响应地址 ARA (Alert Response Address，地址值为 0001100x) 访问所有 SMBALERT 设备。只有那些将 SMBALERT 拉低的设备能应答 ARA。此状态是由 I2C\_SR1 寄存器中的 SMBALERT 状态标记来标识的。主机执行一个修改过的接收字节操作。由从发送设备提供的 7 位设备地址被放在字节的 7 个最高位上，第八个位可以是 '0' 或 '1'。

如果多个设备把 SMBALERT 拉低，最高优先级设备（最小的地址）将在地址传输期间通过标准仲裁赢得通信权。在确认从地址后，此设备不得再拉低它的 SMBALERT，如果当信息传输完成后，主机仍看到 SMBALERT 低，就知道需要再次读 ARA。

没有实现 SMBALERT 信号的主机可以定期访问 ARA。

### 超时错误

在定时规范上 I<sup>2</sup>C 和 SMBus 之间有很多差别。

SMBus 定义了一个时钟低超时，35ms 的超时。SMBus 规定 TLOW:SEXT 为从设备的累积时钟低扩展时间。SMBus 规定 TLOW:MEXT 为主设备的累积时钟低扩展时间。

I2C\_SR1 中的状态标志 Timeout 或 Tlow 错误表明了这个特性的状态。

### 如何使用 SMBus 模式的接口

为了从 I<sup>2</sup>C 模式切换到 SMBus 模式，应该执行下列步骤：

- 设置 I2C\_CR1 寄存器中的 SMBus 位；
- 按应用要求配置 I2C\_CR1 寄存器中的 SMBTYPE 和 ENARP 位。

软件程序必须处理多种 SMBus 协议。

- 如果 ENARP=1 且 SMBTYPE=0，使用 SMB 设备默认地址。
- 如果 ENARP=1 且 SMBTYPE=1，使用 SMB 主设备头位域。



- 如果 SMBALERT=1, 使用 SMB 提醒响应地址。

### 34.3.7 DMA 请求

DMA 请求（当被使能时）仅用于数据传输。发送时数据寄存器变空或接收时数据寄存器变满，则产生 DMA 请求。DMA 请求必须在当前字节传输结束之前被响应。当为相应 DMA 通道设置的数据传输量已完成时，DMA 控制器发送传输结束信号 EOT 到 I<sup>2</sup>C 接口，并且在中断允许时产生一个传输完成中断：

- 主发送器：在 EOT 中断服务程序中，需禁止 DMA 请求，然后在等到 BTF 事件后设置停止条件。
- 主接收器：当要接收的数据数目大于或等于 2 时，DMA 控制器发送一个硬件信号 EOT\_1，它对应 DMA 传输（字节数-1）。如果在 I2C\_CR2 寄存器中设置了 LAST 位，硬件在发送完 EOT\_1 后的下一个字节，将自动发送 NACK。在中断允许的情况下，用户可以在 DMA 传输完成的中断服务程序中产生一个停止条件。

### 利用 DMA 发送

通过设置 I2C\_CR2 寄存器中的 DMAEN 位可以激活 DMA 模式。只要 TxE 位被置位，数据将由 DMA 从预置的存储区装载进 I2C\_DR 寄存器。为 I<sup>2</sup>C 分配一个 DMA 通道，须执行以下步骤（x 是通道号）：

1. 在 DMA\_CPARx 寄存器中设置 I2C\_DR 寄存器地址。数据将在每个 TxE 事件后从存储器传送到这个地址。
2. 在 DMA\_CMARx 寄存器中设置存储器地址。数据在每个 TxE 事件后从这个存储区传送到 I2C\_DR。
3. 在 DMA\_CNDTRx 寄存器中设置所需的传输字节数。在每个 TxE 事件后，此值将被递减。
4. 利用 DMA\_CCRx 寄存器中的 PL[0:1]位配置通道优先级。
5. 设置 DMA\_CCRx 寄存器中的 DIR 位，并根据应用要求可以配置在整个传输完成一半或全部完成时发出中断请求。
6. 通过设置 DMA\_CCTx 寄存器上的 EN 位激活通道。

当 DMA 控制器中设置的数据传输数目已经完成时，DMA 控制器给 I<sup>2</sup>C 接口发送一个传输结束的 EOT/ EOT\_1 信号。在中断允许的情况下，将产生一个 DMA 中断。

注：如果使用 DMA 进行发送时，不要设置 I2C\_CR2 寄存器的 ITBUFEN 位。

### 利用 DMA 接收

通过设置 I2C\_CR2 寄存器中的 DMAEN 位可以激活 DMA 接收模式。每次接收到数据字节时，将由 DMA 把 I2C\_DR 寄存器的数据传送到设置的存储区（参考 DMA 说明）。设置 DMA 通道进行 I<sup>2</sup>C 接收，须执行以下步骤（x 是通道号）：

1. 在 DMA\_CPARx 寄存器中设置 I2C\_DR 寄存器的地址。数据将在每次 RxNE 事件后从此地址传送到存储区。
2. 在 DMA\_CMARx 寄存器中设置存储区地址。数据将在每次 RxNE 事件后从 I2C\_DR 寄存器传送到此存储区。
3. 在 DMA\_CNDTRx 寄存器中设置所需的传输字节数。在每个 RxNE 事件后，此值将被递减。
4. 用 DMA\_CCRx 寄存器中的 PL[0:1]配置通道优先级。
5. 清除 DMA\_CCRx 寄存器中的 DIR 位，根据应用要求可以设置在数据传输完成一半或全部完成时发出中断请求。
6. 设置 DMA\_CCRx 寄存器中的 EN 位激活该通道。

当 DMA 控制器中设置的数据传输数目已经完成时，DMA 控制器给 I<sup>2</sup>C 接口发送一个传输结束的 EOT/ EOT\_1 信号。在中断允许的情况下，将产生一个 DMA 中断。

注：如果使用 DMA 进行接收时，不要设置 I2C\_CR2 寄存器的 ITBUFEN 位。

### 34.3.8 包错误校验 (PEC)

包错误校验 (PEC) 计算器是用于提高通信的可靠性，这个计算器使用下述 CRC-8 多项式对每一位串行数据进行计算：

$$C(x) = x^8 + x^2 + x + 1$$

- PEC 计算由 I2C\_CR1 寄存器的 ENPEC 位激活。PEC 使用 CRC-8 算法对所有信息字节进行计算，包括地址和读/写位在内。
  - 在发送时：在最后一个 TxE 事件时设置 I2C\_CR1 寄存器的 PEC 传输位，PEC 将在最后一个字节后被发送。
  - 在接收时：在最后一个 RxNE 事件之后设置 I2C\_CR1 寄存器的 PEC 位，如果下个接收到的字节不等于内部计算的 PEC，接收器发送一个 NACK。如果是主接收器，不管校对的结果如何，PEC 后都将发送 NACK。PEC 位必须在接收当前字节的 ACK 脉冲之前设置。
- 在 I2C\_SR1 寄存器中可获得 PECERR 错误标记/中断。
- 如果 DMA 和 PEC 计算器都被激活：
  - 在发送时：当 I<sup>2</sup>C 接口从 DMA 控制器处接收到 EOT 信号时，它在最后一个字节后自动发送 PEC。
  - 在接收时：当 I<sup>2</sup>C 接口从 DMA 处接收到一个 EOT\_1 信号时，它将自动把下一个字节作为 PEC，并且将检查它。在接收到 PEC 后产生一个 DMA 请求。
- 为了允许中间 PEC 传输，在 I2C\_CR2 寄存器中有一个控制位 (LAST 位) 用于判别是否真是最后一个 DMA 传输。如果确实是最后一个主接收器的 DMA 请求，在接收到最后一个字节后自动发送 NACK。
- 仲裁丢失时 PEC 计算失效。

### 34.4 I<sup>2</sup>C 中断请求

下表列出了所有的 I<sup>2</sup>C 中断请求

表 34.2 I<sup>2</sup>C 中断请求表：

| 中断事件                | 事件标志  | 开启控制位             |
|---------------------|-------|-------------------|
| 起始位已发送 (主)          | SB    | ITEVFEN           |
| 地址已发送 (主) 或地址匹配 (从) | ADDR  |                   |
| 10 位头段已发送 (主)       | ADD10 |                   |
| 已收到停止 (从)           | STOPF |                   |
| 数据字节传输完成            | BTF   |                   |
| 接收缓冲区非空             | RxNE  | ITEVFEN 和 ITBUFEN |
| 发送缓冲区空              | TxE   |                   |
| 总线错误                | BERR  | ITERREN           |
| 仲裁丢失 (主)            | ARLO  |                   |
| 响应失败                | AF    |                   |



| 中断事件       | 事件标志     | 开启控制位 |
|------------|----------|-------|
| 过载/欠载      | OVR      |       |
| PEC 错误     | PECERR   |       |
| 超时/Tlow 错误 | TIMEOUT  |       |
| SMBus 提醒   | SMBALERT |       |

注: 1. SB、ADDR、ADD10、STOPF、BTF、RxNE 和 TxE 通过逻辑或汇到同一个中断通道中。

2. BERR、ARLO、AF、OVR、PECERR、TIMEOUT 和 SMBALERT 通过逻辑或汇到同一个中断通道中。

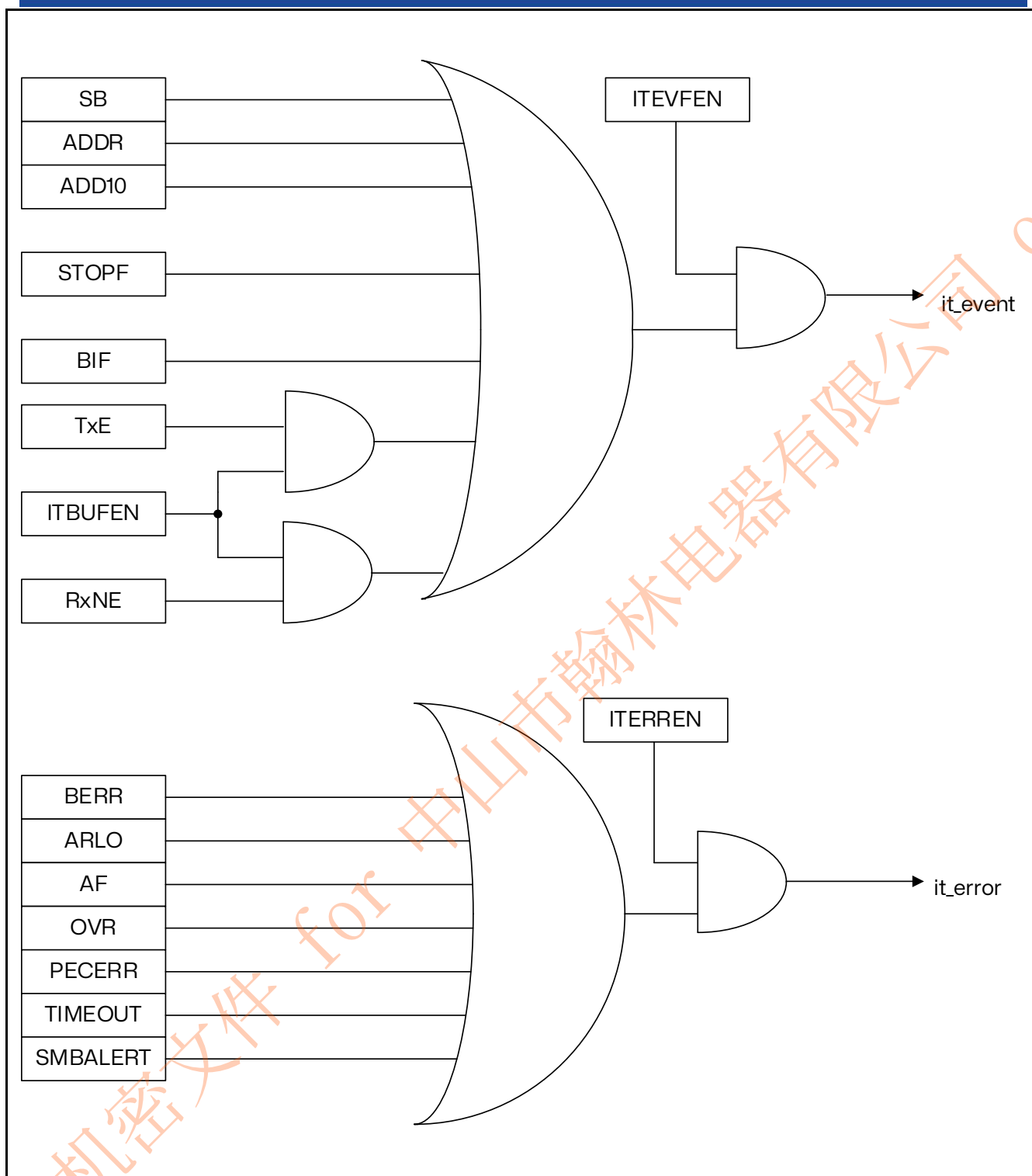


图 34.7 I²C 中断映射图

### 34.5 I²C 调试模式

当微控制器进入调试模式（Cortex-M4 核心处于停止状态）时，根据 DBG 模块中的 DBG\_I2Cx\_SMBUS\_TIMEOUT 配置位，SMBUS 超时控制或者继续正常工作或者可以停止。

## 34.6 I<sup>2</sup>C 寄存器描述

基地址：0x4000 5400

### 34.6.1 控制寄存器 1 (I2C\_CR1)

地址偏移：0x00

复位值：0x0000 0000

| 15    | 14  | 13    | 12  | 11  | 10  | 9    | 8     | 7             | 6     | 5     | 4     | 3           | 2   | 1     | 0  |
|-------|-----|-------|-----|-----|-----|------|-------|---------------|-------|-------|-------|-------------|-----|-------|----|
| SWRST | Res | ALERT | PEC | POS | ACK | STOP | START | NOSTR<br>ETCH | ENGCR | ENPEC | ENARP | SMB<br>TYPE | Res | SMBUS | PE |
| rw    |     | rw    | rw  | rw  | rw  | rw   | rw    | rw            | rw    | rw    | rw    | rw          |     | rw    | rw |

**Bit 15 SWRST:** 软件复位 (Software reset)

当被置位时，I<sup>2</sup>C 处于复位状态。在复位该位前确信 I<sup>2</sup>C 的引脚被释放，总线是空的。

0: I<sup>2</sup>C 模块不处于复位状态；

1: I<sup>2</sup>C 模块处于复位状态。

注：该位可以用于 BUSY 位为 ‘1’，在总线上又没有检测到停止条件时。

**Bit 14 保留，**必须保持为复位值。

**Bit 13 ALERT:** SMBus 提醒 (SMBus alert)

软件可以设置或清除该位；当 PE=0 时，由硬件清除。

0: 释放 SMBAlert 引脚使其变高。提醒响应地址头紧跟在 NACK 信号后面；

1: 驱动 SMBAlert 引脚使其变低。提醒响应地址头紧跟在 ACK 信号后面。

**Bit 12 PEC:** 数据包出错检测 (Packet error checking)

软件可以设置或清除该位；当传送 PEC 后，或起始或停止条件时，或当 PE=0 时硬件将其清除。

0: 无 PEC 传输；

1: PEC 传输 (在发送或接收模式)。

注：仲裁丢失时，PEC 的计算失效。

**Bit 11 POS:** 应答/PEC 位置 (用于数据接收) (Acknowledge/PEC Position (for data reception))

软件可以设置或清除该位，或当 PE=0 时，由硬件清除。

0: ACK 位控制当前移位寄存器内正在接收的字节的 (N) ACK。PEC 位表明当前移位寄存器内的字节是 PEC；

1: ACK 位控制在移位寄存器里接收的下一个字节的 (N) ACK。PEC 位表明在移位寄存器里接收的下一个字节是 PEC。

注：POS 位只能用在 2 字节的接收配置中，必须在接收数据之前配置。

为了 NACK 第 2 个字节，必须在清除 ADDR 为之后清除 ACK 位。

为了检测第 2 个字节的 PEC，必须在配置了 POS 位之后，拉伸 ADDR 事件时设置 PEC 位。

**Bit 10 ACK:** 应答使能 (Acknowledge enable)

软件可以设置或清除该位，或当 PE=0 时，由硬件清除。

0: 无应答返回；

1: 在接收到一个字节后返回一个应答 (匹配的地址或数据)。

**Bit 9 STOP:** 停止条件产生 (Stop generation)

软件可以设置或清除该位；或当检测到停止条件时，由硬件清除；当检测到超时错误

时，硬件将其置位。

在主模式下：

0：无停止条件产生；

1：在当前字节传输或在当前起始条件发出后产生停止条件。

在从模式下：

0：无停止条件产生；

1：在当前字节传输或释放 SCL 和 SDA 线。

注：当设置了 STOP、START 或 PEC 位，在硬件清除这个位之前，软件不要执行任何对 I2C\_CR1 的写操作；否则有可能会第 2 次设置 STOP、START 或 PEC 位。

**Bit 8 START：**起始条件产生（Start generation）

软件可以设置或清除该位，或当起始条件发出后或 PE=0 时，由硬件清除。

在主模式下：

0：无起始条件产生；

1：重复产生起始条件。

在从模式下：

0：无起始条件产生；

1：当总线空闲时，产生起始条件。

**Bit 7 NOSTRETCH：**禁止时钟延长（从模式）（Clock stretching disable （Slave mode））

该位用于当 ADDR 或 BTF 标志被置位，在从模式下禁止时钟延长，直到它被软件复位。

0：允许时钟延长；

1：禁止时钟延长。

**Bit 6 ENGC：**广播呼叫使能（General call enable）

0：禁止广播呼叫。以非应答响应地址 00h；

1：允许广播呼叫。以应答响应地址 00h。

**Bit 5 ENPEC：**PEC 使能（PEC enable）

0：禁止 PEC 计算；

1：开启 PEC 计算。

**Bit 4 ENARP：**ARP 使能（ARP enable）

0：禁止 ARP；

1：使能 ARP。

如果 SMBTYPE=0，使用 SMBus 设备的默认地址。

如果 SMBTYPE=1，使用 SMBus 的主地址。

**Bit 3 SMBTYPE：**SMBus 类型（SMBus type）

0：SMBus 设备；

1：SMBus 主机。

**Bit 2** 保留，必须保持为复位值。

**Bit 1 SMBUS：**SMBus 模式（SMBus mode）

0：I<sup>2</sup>C 模式；

1：SMBus 模式。

**Bit 0 PE：**I<sup>2</sup>C 模块使能（Peripheral enable）

0：禁用 I<sup>2</sup>C 模块；

1：使能 I<sup>2</sup>C 模块：根据 SMBus 位的设置，相应的 I/O 口需配置为复用功能。

注：如果清除该位时通讯正在进行，在当前通讯结束后，I<sup>2</sup>C 模块被禁用并返回空闲状态。

由于在通讯结束后发生 PE=0，所有的位被清除。

在主模式下，通讯结束之前，绝不能清除该位。

### 34.6.2 控制寄存器 2 (I2C\_CR2)

地址偏移：0x04

复位值：0x0000 0000

| 15       | 14 | 13 | 12   | 11    | 10          | 9           | 8           | 7        | 6 | 5 | 4         | 3  | 2  | 1  | 0  |
|----------|----|----|------|-------|-------------|-------------|-------------|----------|---|---|-----------|----|----|----|----|
| Reserved |    |    | LAST | DMAEN | ITBUF<br>EN | ITEVT<br>EN | ITERR<br>EN | Reserved |   |   | FREQ[5:0] |    |    |    |    |
|          |    |    | rw   | rw    | rw          | rw          | rw          |          |   |   | rw        | rw | rw | rw | rw |

Bits 15:13 保留，必须保持为复位值。

Bit 12 **LAST**: DMA 最后一次传输 (DMA last transfer)

0: 下一次 DMA 的 EOT 不是最后的传输;

1: 下一次 DMA 的 EOT 是最后的传输。

注：该位在主接收模式使用，使得在最后一次接收数据时可以产生一个 NACK。

Bit 11 **DMAEN**: DMA 请求使能 (DMA requests enable)

0: 禁止 DMA 请求;

1: 当 TxE=1 或 RxNE=1 时，允许 DMA 请求。

Bit 10 **ITBUFEN**: 缓冲器中断使能 (Buffer interrupt enable)

0: 当 TxE=1 或 RxNE=1 时，不产生任何中断;

1: 当 TxE=1 或 RxNE=1 时，产生事件中断 (不管 DMAEN 是何种状态)。

Bit 9 **ITEVTEN**: 事件中断使能 (Event interrupt enable)

0: 禁止事件中断;

1: 允许事件中断。

在下列条件下，将产生该中断:

- SB = 1 (主模式);
- ADDR = 1 (主/从模式);
- ADD10 = 1 (主模式);
- STOPF = 1 (从模式);
- BTF = 1, 但是没有 TxE 或 RxNE 事件;
- 如果 ITBUFEN = 1, TxE 事件为 1;
- 如果 ITBUFEN = 1, RxNE 事件为 1。

Bit 8 **ITERREN**: 出错中断使能 (Error interrupt enable)

0: 禁止出错中断;

1: 允许出错中断。

在下列条件下，将产生该中断:

- BERR = 1;
- ARLO = 1;
- AF = 1;
- OVR = 1;
- PECERR = 1;
- TIMEOUT = 1;

– SMBAAlert = 1。

Bits 7:6 保留，必须保持为复位值。

Bits 5:0 **FREQ[5:0]**: I<sup>2</sup>C 模块时钟频率 (Peripheral clock frequency)

必须设置正确的输入时钟频率以产生正确的时序，允许的范围在 2~36MHz 之间：

000000: 禁用

000001: 禁用

000010: 2MHz

...

100100: 36MHz

大于 100100: 禁用。

### 34.6.3 自身地址寄存器 1 (I2C\_OAR1)

复位地址偏移: 0x08

复位值: 0x0000 0000

| 15          | 14       | 13 | 12 | 11 | 10 | 9        | 8  | 7        | 6  | 5  | 4  | 3  | 2  | 1  | 0    |
|-------------|----------|----|----|----|----|----------|----|----------|----|----|----|----|----|----|------|
| ADD<br>MODE | Reserved |    |    |    |    | ADD[9:8] |    | ADD[7:1] |    |    |    |    |    |    | ADD0 |
|             |          |    |    |    |    | rw       | rw | rw       | rw | rw | rw | rw | rw | rw | rw   |

Bit 15 **ADDMODE**: 寻址模式 (从模式) (Addressing mode (slave mode))

0: 7 位从地址 (不响应 10 位地址);

1: 10 位从地址 (不响应 7 位地址)。

Bits 14:12 保留，必须保持为复位值。

Bits 9:8 **ADD[9:8]**: 接口地址 (Interface address)

7 位地址模式时不用关心。

10 位地址模式时为地址的 9~8 位。

Bits 7:1 **ADD[7:1]**: 接口地址 (Interface address)

地址的 7~1 位。

Bit 0 **ADD0**: 接口地址 (Interface address)

7 位地址模式时不用关心。

10 位地址模式时为地址第 0 位。

### 34.6.4 自身地址寄存器 2 (I2C\_OAR2)

地址偏移: 0x0C

复位值: 0x0000 0000

| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7         | 6  | 5  | 4  | 3  | 2  | 1  | 0      |
|----------|----|----|----|----|----|---|---|-----------|----|----|----|----|----|----|--------|
| Reserved |    |    |    |    |    |   |   | ADD2[7:1] |    |    |    |    |    |    | ENDUAL |
|          |    |    |    |    |    |   |   | rw        | rw | rw | rw | rw | rw | rw | rw     |

Bits 15:8 保留，必须保持为复位值。

Bits 7:1 **ADD2[7:1]**: 接口地址 (Interface address)

在双地址模式下地址的 7~1 位。

Bit 0 **ENDUAL**: 双地址模式使能位 (Dual addressing mode enable)

0: 在 7 位地址模式下，只有 OAR1 被识别;

1: 在 7 位地址模式下，OAR1 和 OAR2 都被识别。

### 34.6.5 数据寄存器 (I2C\_DR)

地址偏移: 0x10

复位值: 0x0000

| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7       | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
|----------|----|----|----|----|----|---|---|---------|----|----|----|----|----|----|----|
| Reserved |    |    |    |    |    |   |   | DR[7:0] |    |    |    |    |    |    |    |
|          |    |    |    |    |    |   |   | rw      | rw | rw | rw | rw | rw | rw | rw |

Bits 15:8 保留, 必须保持为复位值。

Bits 7:0 **DR[7:0]**: 8 位数据寄存器 (8-bit data register)

用于存放接收到的数据或放置用于发送到总线的的数据

发送器模式: 当写一个字节至 DR 寄存器时, 自动启动数据传输。一旦传输开始

(TxE=1), 如果能及时把下一个需传输的数据写入 DR 寄存器, I<sup>2</sup>C 模块将保持连续的数据流。

接收器模式: 接收到的字节被拷贝到 DR 寄存器 (RxNE=1)。在接收到下一个字节

(RxNE=1) 之前读出数据寄存器, 即可实现连续的数据传送。

注: 在从模式下, 地址不会被拷贝进数据寄存器 DR;

注: 硬件不管理写冲突 (如果 TxE=0, 仍能写入数据寄存器);

注: 如果在处理 ACK 脉冲时发生 ARLO 事件, 接收到的字节不会被拷贝到数据寄存器里, 因此不能读到它。

### 34.6.6 状态寄存器 1 (I2C\_SR1)

地址偏移: 0x14

复位值: 0x0000 0000

| 15           | 14          | 13  | 12         | 11    | 10    | 9     | 8     | 7   | 6    | 5   | 4     | 3     | 2   | 1    | 0  |
|--------------|-------------|-----|------------|-------|-------|-------|-------|-----|------|-----|-------|-------|-----|------|----|
| SMB<br>ALERT | TIME<br>OUT | Res | PEC<br>ERR | OVR   | AF    | ARLO  | BERR  | TxE | RxNE | Res | STOPF | ADD10 | BTF | ADDR | SB |
| rc_w0        | rc_w0       |     | rc_w0      | rc_w0 | rc_w0 | rc_w0 | rc_w0 | r   | r    |     | r     | r     | r   | r    | r  |

Bit 15 **SMBALERT**: SMBus 提醒 (SMBus alert)

在 SMBus 主机模式下:

0: 无 SMBus 提醒;

1: 在引脚上产生 SMBAlert 提醒事件。

在 SMBus 从机模式下:

0: 没有 SMBAlert 响应地址头序列;

1: 收到 SMBAlert 响应地址头序列至 SMBAlert 变低。

— 该位由软件写 '0' 清除, 或在 PE=0 时由硬件清除。

Bit 14 **TIMEOUT**: 超时或 Tlow 错误 (Timeout or Tlow error)

0: 无超时错误;

1: SCL 处于低已达到 25ms (超时); 或者主机低电平累积时钟扩展时间超过 10ms (Tlow:mext); 或从设备低电平累积时钟扩展时间超过 25ms (Tlow:sext)。

— 当在从模式下设置该位: 从设备复位通讯, 硬件释放总线。

— 当在主模式下设置该位: 硬件发出停止条件。

— 该位由软件写 '0' 清除, 或在 PE=0 时由硬件清除。

Bit 13 保留, 必须保持为复位值。

Bit 12 **PECERR**: 在接收时发生 PEC 错误 (PEC Error in reception)

0: 无 PEC 错误: 接收到 PEC 后接收器返回 ACK (如果 ACK=1);



1: 有 PEC 错误: 接收到 PEC 后接收器返回 NACK (不管 ACK 是什么值)。

- 该位由软件写 ‘0’ 清除, 或在 PE=0 时由硬件清除。

Bit 11 **OVR**: 过载/欠载 (Overrun/Underrun)

0: 无过载/欠载;

1: 出现过载/欠载。

- 当 NOSTRETCH=1 时, 在从模式下该位被硬件置位, 同时:
- 在接收模式中当收到一个新的字节时 (包括 ACK 应答脉冲), 数据寄存器里的内容还未被读出, 则新接收的字节将丢失。
- 在发送模式中当要发送一个新的字节时, 却没有新的数据写入数据寄存器, 同样的字节将被发送两次。
- 该位由软件写 ‘0’ 清除, 或在 PE=0 时由硬件清除。

注: 如果数据寄存器的写操作发生时间非常接近 SCL 的上升沿, 发送的数据是不确定的, 并发生保持时间错误。

Bit 10 **AF**: 应答失败 (Acknowledge failure)

0: 没有应答失败;

1: 应答失败。

- 当没有返回应答时, 硬件将置该位为 ‘1’。
- 该位由软件写 ‘0’ 清除, 或在 PE=0 时由硬件清除。

Bit 9 **ARLO**: 仲裁丢失 (主模式) (Arbitration lost (master mode))

0: 没有检测到仲裁丢失;

1: 检测到仲裁丢失。

当接口失去对总线的控制给另一个主机时, 硬件将置该位为 ‘1’。

- 该位由软件写 ‘0’ 清除, 或在 PE=0 时由硬件清除。

在 ARLO 事件之后, I<sup>2</sup>C 接口自动切换回从模式 (M/SL=0)。

注: 在 SMBUS 模式下, 在从模式下对数据的仲裁仅仅发生在数据阶段, 或应答传输区间 (不包括地址的应答)。

Bit 8 **BERR**: 总线出错 (Bus error)

0: 无起始或停止条件出错;

1: 起始或停止条件出错。

- 当接口检测到错误的起始或停止条件, 硬件将该位置 ‘1’。
- 该位由软件写 ‘0’ 清除, 或在 PE=0 时由硬件清除。

Bit 7 **TxE**: 数据寄存器为空 (发送时) (Data register empty (transmitters))

0: 数据寄存器非空;

1: 数据寄存器空。

- 在发送数据时, 数据寄存器为空时该位被置 ‘1’, 在发送地址阶段不设置该位。
- 软件写数据到 DR 寄存器可清除该位; 或在发生一个起始或停止条件后, 或当 PE=0 时由硬件自动清除。

如果收到一个 NACK, 或下一个要发送的字节是 PEC (PEC=1), 该位不被置位。

注: 在写入第 1 个要发送的数据后, 或设置了 BTF 时写入数据, 都不能清除 TxE 位, 这是因为数据寄存器仍然为空。

Bit 6 **RxNE**: 数据寄存器非空 (接收时) (Data register not empty (receivers))

0: 数据寄存器为空;

1: 数据寄存器非空。

- 在接收时，当数据寄存器不为空，该位被置‘1’。在接收地址阶段，该位不被置位。
  - 软件对数据寄存器的读写操作清除该位，或当 PE=0 时由硬件清除。
- 在发生 ARLO 事件时，RxNE 不被置位。

注：当设置了 BTF 时，读取数据不能清除 RxNE 位，因为数据寄存器仍然为满。

Bit 5 保留，必须保持为复位值。

Bit 4 **STOPF**：停止条件检测位（从模式）（Stop detection (slave mode)）

0：没有检测到停止条件；

1：检测到停止条件。

- 在一个应答之后（如果 ACK=1），当从设备在总线上检测到停止条件时，硬件将该位置‘1’。
- 软件读取 SR1 寄存器后，对 CR1 寄存器的写操作将清除该位，或当 PE=0 时，硬件清除该位。

注：在收到 NACK 后，STOPF 位不被置位。

Bit 3 **ADD10**：10 位头序列已发送（主模式）（10-bit header sent (Master mode)）

0：没有 ADD10 事件发生；

1：主设备已经将第一个地址字节发送出去。

- 在 10 位地址模式下，当主设备已经将第一个字节发送出去时，硬件将该位置‘1’。
- 软件读取 SR1 寄存器后，对 CR1 寄存器的写操作将清除该位，或当 PE=0 时，硬件清除该位。

注：收到一个 NACK 后，ADD10 位不被置位。

Bit 2 **BTF**：字节发送结束（Byte transfer finished）

0：字节发送未完成；

1：字节发送结束。

当 NOSTRETCH=0 时，在下列情况下硬件将该位置‘1’：

- 在接收时，当收到一个新字节（包括 ACK 脉冲）且数据寄存器还未被读取（RxNE=1）。
- 在发送时，当一个新数据将被发送且数据寄存器还未被写入新的数据（TxNE=1）。
- 在软件读取 SR1 寄存器后，对数据寄存器的读或写操作将清除该位；或在传输中发送一个起始或停止条件后，或当 PE=0 时，由硬件清除该位。

注：在收到一个 NACK 后，BTF 位不会被置位。

如果下一个要传输的字节是 PEC（I2C\_SR2 寄存器中 TRA 为‘1’，同时 I2C\_CR1 寄存器中 PEC 为‘1’），BTF 位不会被置位。

Bit 1 **ADDR**：地址已被发送（主模式）/地址匹配（从模式）（Address sent (master mode) /matched (slave mode)）

在软件读取 SR1 寄存器后，对 SR2 寄存器的读操作将清除该位，或当 PE=0 时，由硬件清除该位。

**地址匹配（从模式）**

0：地址不匹配或没有收到地址；

1：收到的地址匹配。

- 当收到的从地址与 OAR 寄存器中的内容相匹配、或发生广播呼叫、或 SMBus 设备默认地址或 SMBus 主机识别出 SMBus 提醒时，硬件就将该位置‘1’（当对应的设置被使能时）。

#### 地址已被发送（主模式）

0: 地址发送没有结束;

1: 地址发送结束。

– 10 位地址模式时, 当收到地址的第二个字节的 ACK 后该位被置 ‘1’。

– 7 位地址模式时, 当收到地址的 ACK 后该位被置 ‘1’。

注: 在收到 NACK 后, ADDR 位不会被置位。

Bit 0 **SB**: 起始位（主模式）(Start bit (Master mode))

0: 未发送起始条件;

1: 起始条件已发送。

– 当发送出起始条件时该位被置 ‘1’。

– 软件读取 SR1 寄存器后, 写数据寄存器的操作将清除该位, 或当 PE=0 时, 硬件清除该位。

### 34.6.7 状态寄存器 2 (I2C\_SR2)

地址偏移: 0x18

复位值: 0x0000 0000

| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7     | 6       | 5          | 4       | 3   | 2   | 1    | 0   |
|----------|----|----|----|----|----|---|---|-------|---------|------------|---------|-----|-----|------|-----|
| PEC[7:0] |    |    |    |    |    |   |   | DUALF | SMBHOST | SMBDEFAULT | GENCALL | Res | TRA | BUSY | MSL |
| r        | r  | r  | r  | r  | r  | r | r | r     | r       | r          | r       |     | r   | r    | r   |

Bits 15:8 **PEC[7:0]**: 数据包出错检测 (Packet error checking register)

当 ENPEC=1 时, PEC[7:0] 存放内部的 PEC 的值。

Bit 7 **DUALF**: 双标志 (从模式) (Dual flag (Slave mode))

0: 接收到的地址与 OAR1 内的内容相匹配;

1: 接收到的地址与 OAR2 内的内容相匹配。

– 在产生一个停止条件或一个重复的起始条件时, 或 PE=0 时, 硬件将该位清除。

Bit 6 **SMBHOST**: SMBus 主机头系列 (从模式) (SMBus host header (Slave mode))

0: 未收到 SMBus 主机的地址;

1: 当 SMBTYPE=1 且 ENARP=1 时, 收到 SMBus 主机地址。

– 在产生一个停止条件或一个重复的起始条件时, 或 PE=0 时, 硬件将该位清除。

Bit 5 **SMBDEFAULT**: SMBus 设备默认地址 (从模式) (SMBus device default address (Slave mode))

0: 未收到 SMBus 设备的默认地址;

1: 当 ENARP=1 时, 收到 SMBus 设备的默认地址。

– 在产生一个停止条件或一个重复的起始条件时, 或 PE=0 时, 硬件将该位清除。

Bit 4 **GENCALL**: 广播呼叫地址 (从模式) (General call address (Slave mode))

0: 未收到广播呼叫地址;

1: 当 ENGC=1 时, 收到广播呼叫的地址

– 在产生一个停止条件或一个重复的起始条件时, 或 PE=0 时, 硬件将该位清除。

Bit 3 保留, 必须保持为复位值。

Bit 2 **TRA**: 发送/接收 (Transmitter/receiver)

0: 接收到数据;

1: 数据已发送;

在整个地址传输阶段的结尾, 该位根据地址字节的 R/W 位来设定。

在检测到停止条件（STOPF=1）、重复的起始条件或总线仲裁丢失（ARLO=1）后，或当 PE=0 时，硬件将其清除。

**Bit 1 BUSY:** 总线忙（Bus busy）

0: 在总线上无数据通讯；

1: 在总线上正在进行数据通讯。

– 在检测到 SDA 或 SCI 为低电平时，硬件将该位置 ‘1’；

– 当检测到一个停止条件时，硬件将该位清除。

该位指示当前正在进行的总线通讯，当接口被禁用（PE=0）时该信息仍然被更新。

**Bit 0 MSL:** 主从模式（Master/slave）

0: 从模式；

1: 主模式。

– 当接口处于主模式（SB=1）时，硬件将该位置位；

– 当总线上检测到一个停止条件、仲裁丢失（ARLO=1 时）、或当 PE=0 时，硬件清除该位。

### 34.6.8 时钟控制寄存器（I2C\_CCR）

地址偏移：0x1C

复位值：0x0000 0000

注：1. 要求  $F_{PCLK1}$  应当是 10MHz 的整数倍，这样可以正确地产生 400kHz 的快速时钟。

2. CCR 寄存器只有在关闭 I<sup>2</sup>C 时（PE=0）才能设置

| 15  | 14   | 13       | 12 | 11 | 10        | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
|-----|------|----------|----|----|-----------|----|----|----|----|----|----|----|----|----|----|
| F/S | DUTY | Reserved |    |    | CCR[11:0] |    |    |    |    |    |    |    |    |    |    |
| rw  | rw   |          |    |    | rw        | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

**Bit 15 F/S:** I<sup>2</sup>C 主模式选项（I2C master mode selection）

0: 标准模式的 I<sup>2</sup>C；

1: 快速模式的 I<sup>2</sup>C。

**Bit 14 DUTY:** 快速模式时的占空比（Fast mode duty cycle）

0: 快速模式下： $T_{low}/T_{high} = 2$ ；

1: 快速模式下： $T_{low}/T_{high} = 16/9$ 。

Bits 13:12 保留，必须保持为复位值。

Bits 11:0 **CCR[11:0]:** 快速/标准模式下的时钟控制分频系数（主模式）（Clock control register in Fast/Standard mode（Master mode））

该分频系数用于设置主模式下的 SCL 时钟。

在 I<sup>2</sup>C 标准模式或 SMBus 模式下：

$$T_{high} = CCR \times TPCLK1$$

$$T_{low} = CCR \times TPCLK1$$

在 I<sup>2</sup>C 快速模式下：

如果 DUTY = 0:

$$T_{high} = CCR \times TPCLK1$$

$$T_{low} = 2 \times CCR \times TPCLK1$$

如果 DUTY = 1:（速度达到 400kHz）

$$T_{high} = 9 \times CCR \times TPCLK1$$

$$T_{low} = 16 \times CCR \times TPCLK1$$

例如：在标准模式下，产生 100kHz 的 SCL 的频率：

如果  $FREQR = 08$ ,  $TPCLK1 = 125ns$ , 则 CCR 必须写入  $0x28$  ( $40 \times 125ns = 5000ns$ )。

- 注：1. 允许设定的最小值为  $0x04$ , 在快速 DUTY 模式下允许的最小值为  $0x01$ ;  
 2.  $T_{high} = t_r(SCL) + t_w(SCLH)$ ;  
 3.  $T_{low} = t_f(SCL) + t_w(SCLL)$ ;  
 4. 这些延时没有过滤器;  
 5. 只有在关闭 I<sup>2</sup>C 时 ( $PE = 0$ ) 才能设置 CCR 寄存器;  
 6.  $f_{CK}$  应当是 10MHz 的整数倍, 这样可以正确产生 400kHz 的快速时钟。

### 34.6.9 TRISE 寄存器 (I2C\_TRISE)

地址偏移:  $0x20$

复位值:  $0x0000\ 0002$

| 15       | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5          | 4  | 3  | 2  | 1  | 0  |
|----------|----|----|----|----|----|---|---|---|---|------------|----|----|----|----|----|
| Reserved |    |    |    |    |    |   |   |   |   | TRISE[5:0] |    |    |    |    |    |
|          |    |    |    |    |    |   |   |   |   | rw         | rw | rw | rw | rw | rw |

Bits 15:6 保留, 必须保持为复位值。

Bits 5:0 **TRISE[5:0]**: 在快速/标准模式下的最大上升时间 (主模式) (Maximum rise time in Fast/Standard mode (Master mode))

这些位必须设置为 I<sup>2</sup>C 总线规范里给出的最大的 SCL 上升时间, 增长步幅为 1。

例如: 标准模式中最大允许 SCL 上升时间为 1000ns。如果在 I2C\_CR2 寄存器中  $FREQ[5:0]$  中的值等于  $0x08$  且  $TPCLK1 = 125ns$ , 故  $TRISE[5:0]$  中必须写入  $09h$  ( $1000ns/125ns = 8+1$ )。

滤波器的值也可以加到  $TRISE[5:0]$  内。

如果结果不是一个整数, 则将整数部分写入  $TRISE[5:0]$  以确保  $t_{HIGH}$  参数。

注: 只有当 I<sup>2</sup>C 被禁用 ( $PE=0$ ) 时, 才能设置  $TRISE[5:0]$ 。

## 35 通用异步收发器 (UART)

### 35.1 UART 介绍

通用异步收发器(UART)提供了一种灵活的方法与使用工业标准 NRZ 异步串行数据格式的外部设备之间进行全双工数据交换。UART 利用分数波特率发生器提供宽范围的波特率选择。

它支持同步单向通信和半双工单线通信, 也支持 LIN (局部互连网), 以及调制解调器 (CTS/RTS) 操作。它还允许多处理器通信。

使用多缓冲器配置的 DMA 方式, 可以实现高速数据通信。

### 35.2 UART 主要特性

- 全双工的, 异步通信
- NRZ 标准格式
- 分数波特率发生器系统
- 可编程数据字长度(8 位或 9 位)
- 可配置的停止位-支持 1 或 2 个停止位
- LIN 主发送同步断开符的能力以及 LIN 从检测断开符的能力
  - 当UART硬件配置成LIN时, 生成13位断开符; 检测10/11位断开符
- 单线半双工通信
- 可配置的使用 DMA 的多缓冲器通信
  - 在SRAM里利用集中式DMA缓冲接收/发送字节
- 单独的发送器和接收器使能位
- 检测标志
  - 接收缓冲器满
  - 发送缓冲器空
  - 传输结束标志
- 校验控制
  - 发送校验位
  - 对接收数据进行校验
- 四个错误检测标志
  - 溢出错误
  - 噪音错误
  - 帧错误
  - 校验错误
- 10 个带标志的中断源
  - CTS改变
  - LIN断开符检测
  - 发送数据寄存器空
  - 发送完成
  - 接收数据寄存器满



- 检测到总线为空闲
- 溢出错误
- 帧错误
- 噪音错误
- 校验错误
- 多处理器通信
  - 如果地址不匹配, 则进入静默模式
- 从静默模式中唤醒(通过空闲总线检测或地址标志检测)
- 两种唤醒接收器的方式

### 35.3 UART 功能概述

接口通过三个引脚与其他设备连接在一起。任何 UART 双向通信至少需要两个脚: 接收数据输入(RX)和发送数据输出(TX)。

RX: 接收数据串行输入。通过过采样技术来区别数据和噪音, 从而恢复数据。

TX: 发送数据输出。当发送器被禁止时, 输出引脚恢复到它的 I/O 端口配置。当发送器被激活, 并且不发送数据时, TX 引脚处于高电平。

- 总线在发送或接收前应处于空闲状态
- 一个起始位
- 一个数据字(8 或 9 位), 最低有效位在前
- 使用分数波特率发生器 — 12 位整数和 4 位小数的表示方法。
- 一个状态寄存器(UART\_SR)
- 数据寄存器(UART\_DR)
- 一个波特率寄存器(UART\_BRR), 12 位的整数和 4 位小数

下列引脚在硬件流控模式中需要:

- nCTS: 清除发送, 若是高电平, 在当前数据传输结束时阻断下一次的数据发送。
- nRTS: 发送请求, 若是低电平, 表明 UART 准备好接收数据。



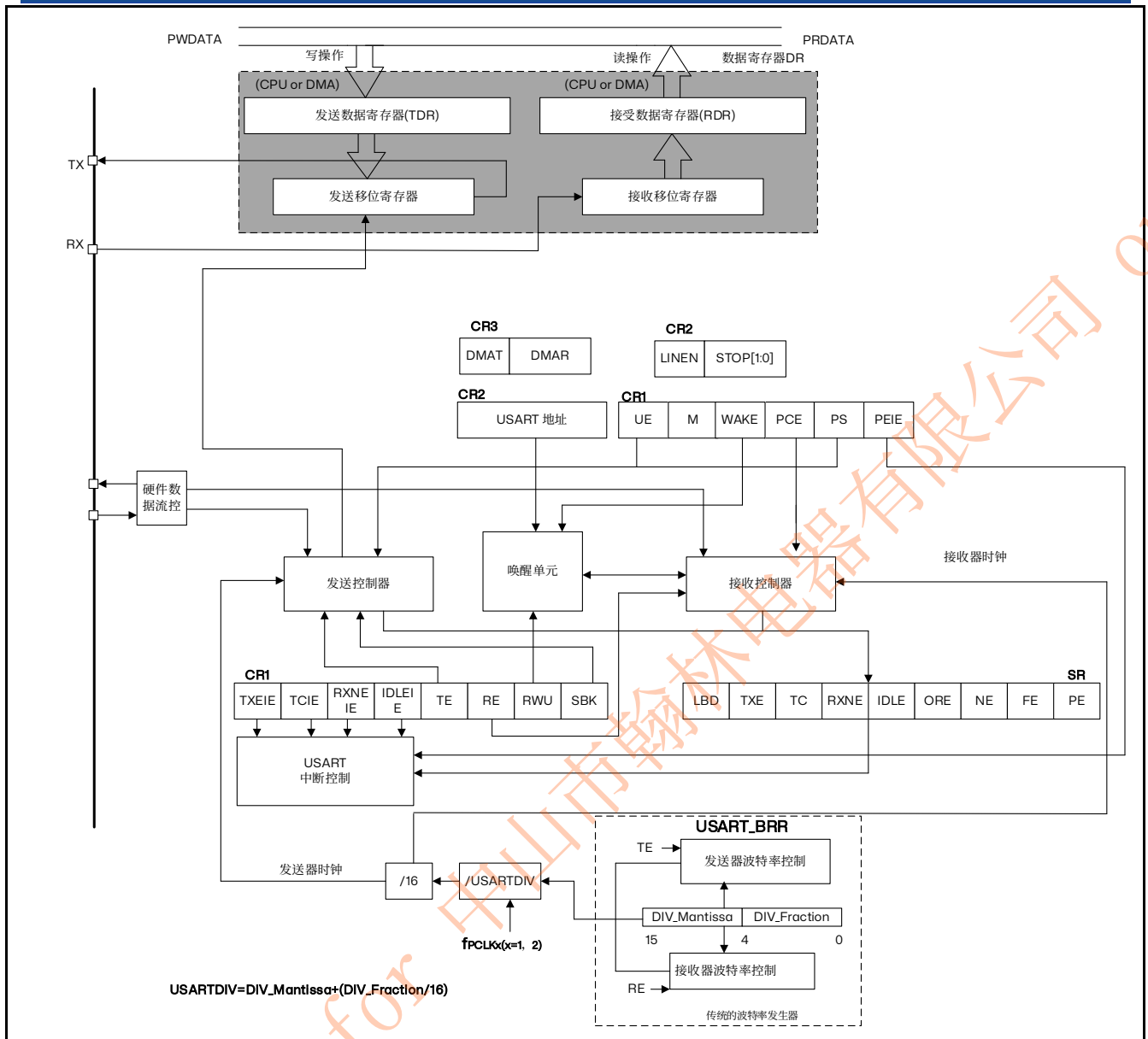


图 35.1 UART 框图

### 35.3.1 UART 特性描述

字长可以通过编程 UART\_CR1 寄存器中的 M 位，选择成 8 或 9 位。在起始位期间，TX 脚处于低电平，在停止位期间处于高电平。

空闲符号被视为完全由 ‘1’ 组成的一个完整的数据帧，后面跟着包含了数据的下一帧的开始位（‘1’ 的位数也包括了停止位的位数）。

断开符号被视为在一个帧周期内全部收到 ‘0’（包括停止位期间，也是 ‘0’）。在断开帧结束时，发送器再插入 1 或 2 个停止位（‘1’）来应答起始位。

发送和接收由一共用的波特率发生器驱动，当发送器和接收器的使能位分别置位时，分别为其产生时钟。

随后将有每个功能块的详细说明。

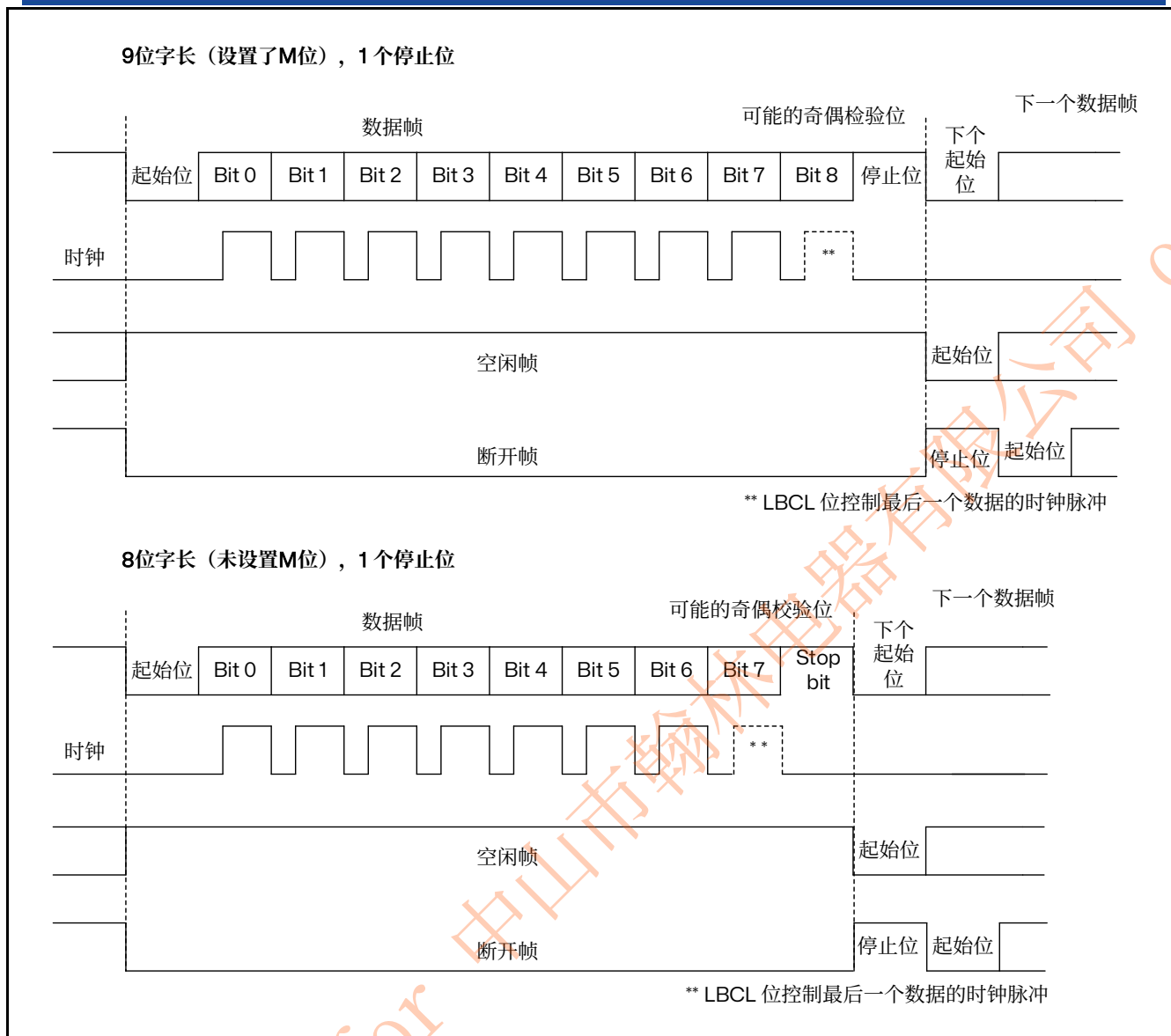


图 35.2 字长设置

## 35.3.2 发送器

发送器根据 M 位的状态发送 8 位或 9 位的数据字。当发送使能位（TE）被设置时，发送移位寄存器中的数据在 TX 脚上输出。

### 字符发送

在 UART 发送期间，在 TX 引脚上首先移出数据的最低有效位。在此模式里，UART\_DR 寄存器包含了一个内部总线和发送移位寄存器之间的缓冲器。

每个字符之前都有一个低电平的起始位；之后跟着的停止位，其数目可配置。

UART 支持多种停止位的配置：1 和 2 个停止位。

注：1. 在数据传输期间不能复位 TE 位，否则将破坏 TX 脚上的数据，因为波特率计数器停止计数。正在传输的当前数据将丢失。

2. TE 位被激活后将发送一个空闲帧。

## 可配置的停止位

随每个字符发送的停止位的位数可以通过控制寄存器 2 的位 13、12 进行编程。

1. 1 个停止位：停止位位数的默认值。
2. 2 个停止位：可用于常规 UART 模式、单线模式以及调制解调器模式。

空闲帧包括了停止位。

断开帧是 10 位低电平，后跟停止位（当  $m=0$  时）；或者 11 位低电平，后跟停止位（ $m=1$  时）。不可能传输更长的断开帧（长度大于 10 或者 11 位）。

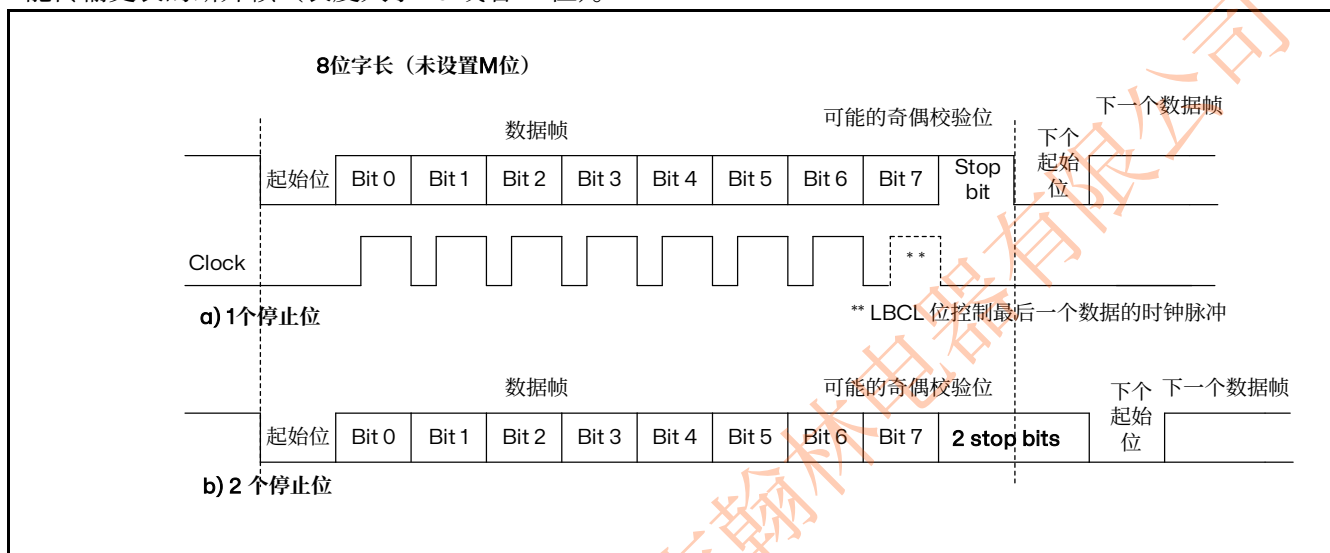


图 35.3 配置停止位

配置步骤：

1. 通过在 UART\_CR1 寄存器上置位 UE 位来激活 UART
2. 编程 UART\_CR1 的 M 位来定义字长。
3. 在 UART\_CR2 中编程停止位的位数。
4. 如果采用多缓冲器通信，配置 UART\_CR3 中的 DMA 使能位(DMAT)。按多缓冲器通信中的描述配置 DMA 寄存器。
5. 利用 UART\_BRR 寄存器选择要求的波特率。
6. 设置 UART\_CR1 中的 TE 位，发送一个空闲帧作为第一次数据发送。
7. 把要发送的数据写进 UART\_DR 寄存器(此动作清除 TXE 位)。在只有一个缓冲器的情况下，对每个待发送的数据重复步骤 7。
8. 在 UART\_DR 寄存器中写入最后一个数据字后，要等待 TC=1，它表示最后一个数据帧的传输结束。当需要关闭 UART 或需要进入停机模式之前，需要确认传输结束，避免破坏最后一次传输。

## 单字节通信

清零 TXE 位总是通过对数据寄存器的写操作来完成的。TXE 位由硬件来设置，它表明：

- 数据已经从 TDR 移送到移位寄存器，数据发送已经开始
- TDR 寄存器被清空
- 下一个数据可以被写进 UART\_DR 寄存器而不会覆盖先前的数据

如果 TXEIE 位被设置，此标志将产生一个中断。

如果此时 UART 正在发送数据，对 UART\_DR 寄存器的写操作把数据存进 TDR 寄存器，并在当前传

输结束时把该数据复制进移位寄存器。

如果此时 UART 没有在发送数据，处于空闲状态，对 UART\_DR 寄存器的写操作直接把数据放进移位寄存器，数据传输开始，TXE 位立即被置起。

当一帧发送完成时（停止位发送后）并且设置了 TXE 位，TC 位被置起，如果 UART\_CR1 寄存器中的 TCIE 位被置起时，则会产生中断。

在 UART\_DR 寄存器中写入了最后一个数据字后，在关闭 UART 模块之前或设置微控制器进入低功耗模式之前，必须先等待 TC=1。

使用下列软件过程清除 TC 位：

1. 读一次 UART\_SR 寄存器；
2. 写一次 UART\_DR 寄存器。

注：TC 位也可以通过软件对它写 '0' 来清除。此清零方式只推荐在多缓冲器通信模式下使用。

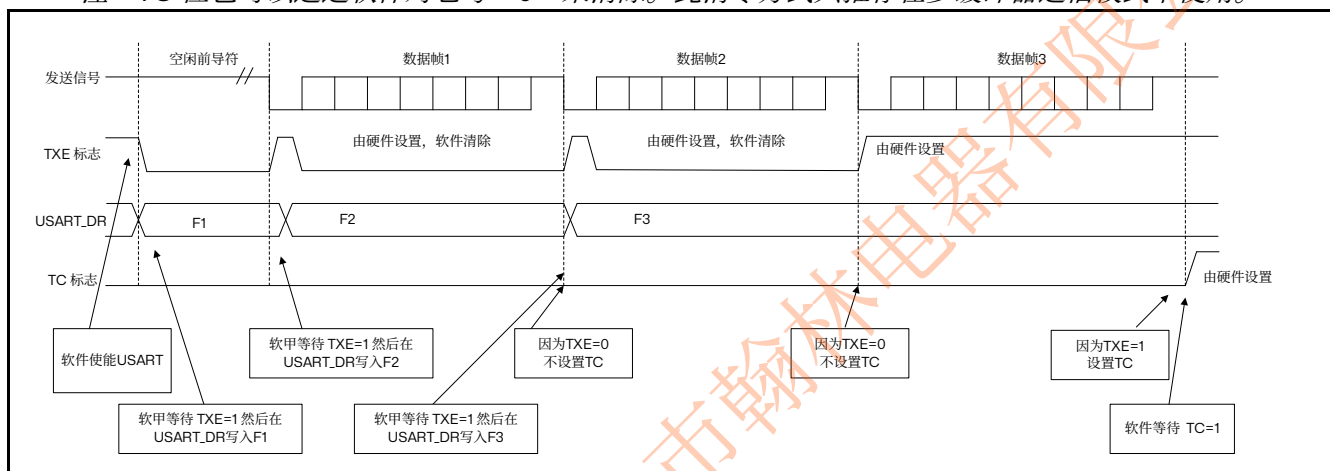


图 35.4 发送时 TC/TXE 的变化情况

## 断开符号

设置 SBK 可发送一个断开符号。断开帧长度取决 M 位。如果设置 SBK=1，在完成当前数据发送后，将在 TX 线上发送一个断开符号。断开字符发送完成时（在断开符号的停止位时）SBK 被硬件复位。UART 在最后一个断开帧的结束处插入一逻辑 '1'，以保证能识别下一帧的起始位。

注意：如果在开始发送断开帧之前，软件又复位了 SBK 位，断开符号将不被发送。如果要发送两个连续的断开帧，SBK 位应该在前一个断开符号的停止位之后置起。

## 空闲符号

置位 TE 将使得 UART 在第一个数据帧前发送一空闲帧。

## 35.3.3 接收器

UART 可以根据 UART\_CR1 的 M 位接收 8 位或 9 位的数据字。

### 起始位侦测

在 UART 中，如果辨认出一个特殊的采样序列，那么就认为侦测到一个起始位。

该序列为：1110X0X0X0000

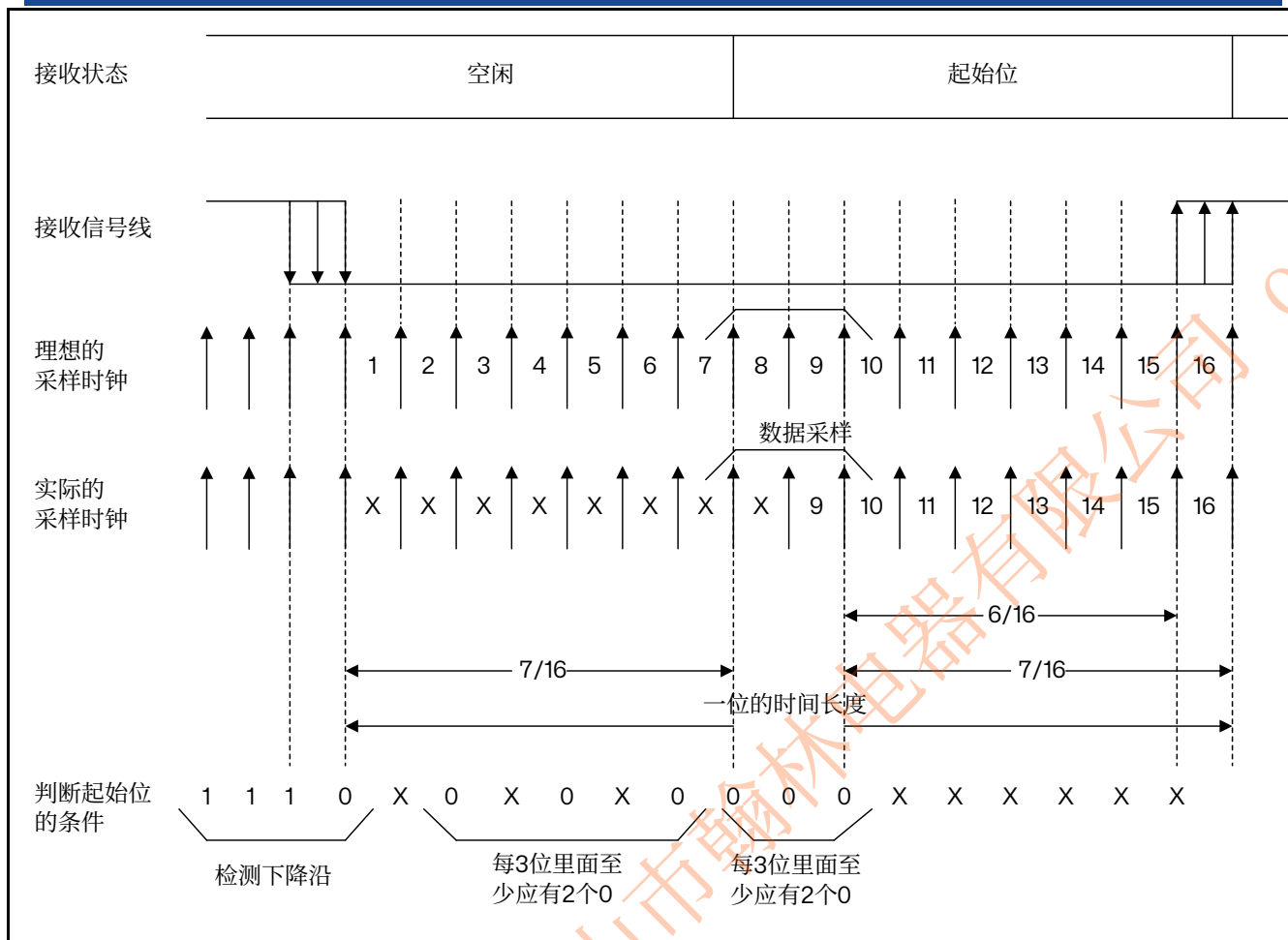


图 35.5 起始位侦测

注意：如果该序列不完整，那么接收端将退出起始位侦测并回到空闲状态（不设置标志位）等待下降沿。

如果3个采样点都为‘0’（在第3、5、7位的第一次采样，和在第8、9、10位的第二次采样都为0），则确认收到起始位，这时设置RXNE标志位，如果RXNEIE=1，则产生中断。

如果两次3个采样点上仅有2个是‘0’（第3、5、7位的采样点和第8、9、10位的采样点），那么起始位仍然是有效的，但是会设置NE噪声标志位。如果不能满足这个条件，则中止起始位的侦测过程，接收器会回到空闲状态（不设置标志位）。

如果有一次3个采样点上仅有2个是‘0’（第3、5、7位的采样点或第8、9、10位的采样点），那么起始位仍然是有效的，但是会设置NE噪声标志位。

### 字符接收

在UART接收期间，数据的最低有效位首先从RX脚移进。在此模式里，UART\_DR寄存器包含的缓冲器位于内部总线和接收移位寄存器之间。

配置步骤：

1. 将UART\_CR1寄存器的UE置1来激活UART。
2. 编程UART\_CR1的M位定义字长
3. 在UART\_CR2中编写停止位的个数
4. 如果需多缓冲器通信，选择UART\_CR3中的DMA使能位(DMAR)。按多缓冲器通信所要求的配

置 DMA 寄存器。

5. 利用波特率寄存器 UART\_BRR 选择希望的波特率。
6. 设置 UART\_CR1 的 RE 位。激活接收器，使它开始寻找起始位。
  - RXNE 位被置位。它表明移位寄存器的内容被转移到 RDR。换句话说，数据已经被接收并且可以被读出（包括与之有关的错误标志）。
  - 如果 RXNEIE 位被设置，产生中断。
  - 在接收期间如果检测到帧错误，噪音或溢出错误，错误标志将被置起，
  - 在单缓冲器模式里，由软件读 UART\_DR 寄存器完成对 RXNE 位清除。RXNE 标志也可以通过对它写 0 来清除。RXNE 位必须在下一字符接收结束前被清零，以避免溢出错误。

*注意：在接收数据时，RE 位不应该被复位。如果 RE 位在接收时被清零，当前字节的接收被丢失*

### 断开符号

当接收到一个断开帧时，UART 像处理帧错误一样处理它。

### 空闲符号

当一空闲帧被检测到时，其处理步骤和接收到普通数据帧一样，但如果 IDLEIE 位被设置将产生一个中断。

### 溢出错误

如果 RXNE 还没有被复位，又接收到一个字符，则发生溢出错误。数据只有当 RXNE 位被清零后才能从移位寄存器转移到 RDR 寄存器。RXNE 标记是接收到每个字节后被置位的。

当溢出错误产生时：

- ORE 位被置位。
- RDR 内容将不会丢失。读 UART\_DR 寄存器仍能得到先前的数据。
- 移位寄存器中以前的内容将被覆盖。随后接收到的数据都将丢失。
- 如果 RXNEIE 位被设置或 EIE 和 DMAR 位都被设置，中断产生。
- 顺序执行对 UART\_SR 和 UART\_DR 寄存器的读操作，可复位 ORE 位

*注意：当 ORE 位置位时，表明至少有 1 个数据已经丢失。有两种可能性：*

- 如果 RXNE=1，上一个有效数据还在接收寄存器 RDR 上，可以被读出。
- 如果 RXNE=0，这意味着上一个有效数据已经被读走，RDR 已经没有东西可读。当上一个有效数据在 RDR 中被读取的同时又接收到新的（也就是丢失的）数据时，此种情况可能发生。在读序列期间（在 UART\_SR 寄存器读访问和 UART\_DR 读访问之间）接收到新的数据，此种情况也可能发生。

### 噪音错误

使用过采样技术（同步模式除外），通过区别有效输入数据和噪音来进行数据恢复。



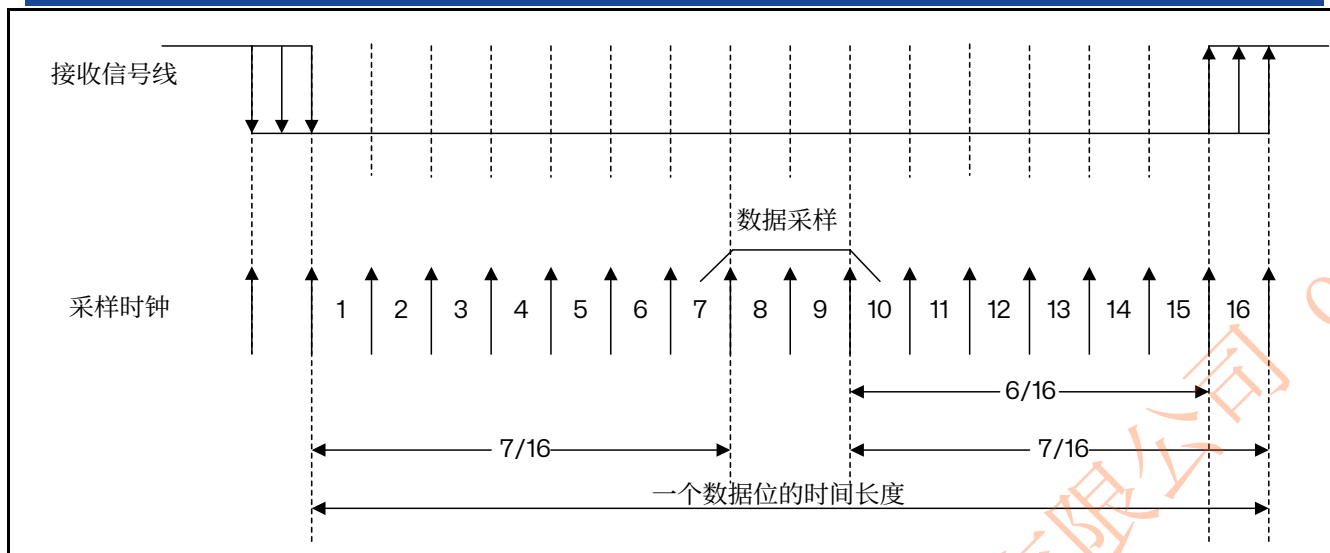


图 35.6 检测噪声的数据采样

表 35.1 检测噪声的数据采样

| 采样值 | NE 状态 | 接收的位 | 数据有效性 |
|-----|-------|------|-------|
| 000 | 0     | 0    | 有效    |
| 001 | 1     | 0    | 无效    |
| 010 | 1     | 0    | 无效    |
| 011 | 1     | 1    | 无效    |
| 100 | 1     | 0    | 无效    |
| 101 | 1     | 1    | 无效    |
| 110 | 1     | 1    | 无效    |
| 111 | 0     | 1    | 有效    |

当在接收帧中检测到噪音时：

- 在 RXNE 位的上升沿设置 NE 标志。
- 无效数据从移位寄存器传送到 UART\_DR 寄存器。
- 在单个字节通信情况下，没有中断产生。然而，因为 NE 标志位和 RXNE 标志位是同时被设置，RXNE 将产生中断。在多缓冲器通信情况下，如果已经设置了 UART\_CR3 寄存器中 EIE 位，将产生一个中断。

先读出 UART\_SR，再读出 UART\_DR 寄存器，将清除 NE 标志位

### 帧错误

当以下情况发生时检测到帧错误：

由于没有同步上或大量噪音的原因，停止位没有在预期的时间上接和收识别出来。

当帧错误被检测到时：

- FE 位被硬件置起
- 无效数据从移位寄存器传送到 UART\_DR 寄存器。
- 在单字节通信时，没有中断产生。然而，这个位和 RXNE 位同时置起，后者将产生中断。在多缓冲器通信情况下，如果 UART\_CR3 寄存器中 EIE 位被置位的话，将产生中断。

顺序执行对 UART\_SR 和 UART\_DR 寄存器的读操作，可复位 FE 位。



### 接收期间的可配置的停止位

被接收的停止位的个数可以通过控制寄存器 2 的控制位来配置，在正常模式时，可以是 1 或 2 个。

- 1 个停止位：对 1 个停止位的采样在第 8，第 9 和第 10 采样点上进行。
- 2 个停止位：对 2 个停止位的采样是在第一停止位的第 8，第 9 和第 10 个采样点完成的。如果第一个停止位期间检测到一个帧错误，帧错误标志将被设置。第二个停止位不再检查帧错误。在第一个停止位结束时 RXNE 标志将被设置。

### 35.3.4 分数波特率的产生

接收器和发送器的波特率在 UARTDIV 的整数和小数寄存器中的值应设置成相同。

$$Tx / Rx \text{ 波特率} = f_{CK} / (16 * UARTDIV)$$

这里的  $f_{CK}$  是给外设的时钟 (PCLK1 用于 UART2, PCLK2 用于 UART1)

UARTDIV 是一个无符号的定点数。这 12 位的值设置在 UART\_BRR 寄存器。

注：在写入 UART\_BRR 之后，波特率计数器会被波特率寄存器的新值替换。因此，不要在通信进行中改变波特率寄存器的数值。

#### 如何从 UART\_BRR 寄存器值得到 UARTDIV

##### 例 1:

如果 DIV\_Mantissa = 27, DIV\_Fraction = 12 (UART\_BRR=0x1BC),

于是

$$\text{Mantissa (UARTDIV)} = 27$$

$$\text{Fraction (UARTDIV)} = 12/16 = 0.75$$

$$\text{所以 UARTDIV} = 27.75$$

##### 例 2:

要求 UARTDIV = 25.62,

$$\text{就有: DIV_Fraction} = 16 * 0.62 = 9.92$$

最接近的整数是: 10 = 0x0A

$$\text{DIV_Mantissa} = \text{mantissa} (25.620) = 25 = 0x19$$

于是, UART\_BRR = 0x19A

##### 例 3:

要求 UARTDIV = 50.99

$$\text{就有: DIV_Fraction} = 16 * 0.99 = 15.84$$

最接近的整数是: 16 = 0x10 => DIV\_frac[3:0]溢出=>进位必须加到小数部分

$$\text{DIV_Mantissa} = \text{mantissa} (50.990 + \text{进位}) = 51 = 0x33$$

于是: UART\_BRR = 0x330, UARTDIV=51

表 35.2 设置波特率时的误差计算

| 波特率 |      | $f_{PCLK} = 36\text{MHz}$ |             |     | $f_{PCLK} = 72\text{MHz}$ |             |     |
|-----|------|---------------------------|-------------|-----|---------------------------|-------------|-----|
| 序号  | Kbps | 实际                        | 置于波特率寄存器中的值 | 误差% | 实际                        | 置于波特率寄存器中的值 | 误差% |
| 1   | 2.4  | 2.400                     | 937.5       | 0%  | 2.4                       | 1875        | 0%  |
| 2   | 9.6  | 9.600                     | 234.375     | 0%  | 9.6x                      | 468.75      | 0%  |

| 波特率 |       | f <sub>PCLK</sub> = 36MHz |          |       | f <sub>PCLK</sub> = 72MHz |         |       |
|-----|-------|---------------------------|----------|-------|---------------------------|---------|-------|
| 3   | 19.2  | 19.2                      | 117.1875 | 0%    | 19.2                      | 234.375 | 0%    |
| 4   | 57.6  | 57.6                      | 39.0625  | 0%    | 57.6                      | 78.125  | 0%    |
| 5   | 115.2 | 115.384                   | 19.5     | 0.15% | 115.2                     | 39.0625 | 0%    |
| 6   | 230.4 | 230.769                   | 9.75     | 0.16% | 230.769                   | 19.5    | 0.16% |
| 7   | 460.8 | 461.538                   | 4.875    | 0.16% | 461.538                   | 9.75    | 0.16% |
| 8   | 921.6 | 923.076                   | 2.4375   | 0.16% | 923.076                   | 4.875   | 0.16% |
| 9   | 2250  | 2250                      | 1        | 0%    | 2250                      | 2       | 0%    |
| 10  | 4500  | 不可能                       | 不可能      | 不可能   | 4500                      | 1       | 0%    |

注:

1. CPU 的时钟频率越低, 则某一特定波特率的误差也越低。可以达到的波特率上限可以由这组数据得到。
2. 只有 UART1 使用 PCLK2 (最高 152MHz)。其它 UART2 使用 PCLK1 (最高 80MHz)。

### 35.3.5 UART 接收器容忍时钟的变化

只有当整体的时钟系统地变化小于 UART 异步接收器能够容忍的范围, UART 异步接收器才能正常工作。影响这些变化的因素有:

- DTRA: 由于发送器误差而产生的变化 (包括发送器端振荡器的变化)
- DQUANT: 接收器端波特率取整所产生的误差
- DREC: 接收器端振荡器的变化
- DTCL: 由于传输线路产生的变化 (通常是由于收发器在由低变高的转换时序, 与由高变低转换时序之间的不一致性所造成)。

需要满足:  $DTRA + DQUANT + DREC + DTCL < \text{UART 接收器的容忍度}$

对于正常接收数据, UART 接收器的容忍度等于最大能容忍的变化, 它依赖于下述选择:

- 由 UART\_CR1 寄存器的 M 位定义的 10 或 11 位字符长度
- 是否使用分数波特率产生

表 35.3 当 DIV\_Fraction=0 时, UART 接收器的容忍度

| M 位 | 认为 NF 是错误 | 不认为 NF 是错误 |
|-----|-----------|------------|
| 0   | 3.75%     | 4.375%     |
| 1   | 3.41%     | 3.97%      |

表 35.4 当 DIV\_Fraction!=0 时, UART 接收器的容忍度

| M 位 | 认为 NF 是错误 | 不认为 NF 是错误 |
|-----|-----------|------------|
| 0   | 3.33%     | 3.88%      |
| 1   | 3.03%     | 3.53%      |

注: 在特殊的情况下, 即当收到的帧包含一些在 M=0 时, 正好是 10 位 (M=1 时是 11 位) 的空闲帧, 上面 2 个表格中的数据可能会有少许出入。

### 35.3.6 校验控制

设置 UART\_CR1 寄存器上的 PCE 位, 可以使能奇偶控制 (发送时生成一个奇偶位, 接收时进行奇偶

校验)。根据 M 位定义的帧长度,可能的 UART 帧格式列在下表中。

表 35.5 帧格式

| M 位 | PCE 位 | UART 帧                    |
|-----|-------|---------------------------|
| 0   | 0     | 起始位   8 位数据   停止位         |
| 0   | 1     | 起始位   7 位数据   奇偶检验位   停止位 |
| 1   | 0     | 起始位   9 位数据   停止位         |
| 1   | 1     | 起始位   8 位数据   奇偶检验位   停止位 |

注意: 在用地址标记唤醒设备时, 地址的匹配只考虑到数据的 MSB 位, 而不用关心校验位。(MSB 是数据位中最后发出的, 后面紧跟校验位或者停止位)

**偶校验:** 校验位使得一帧中的 7 或 8 个 LSB 数据以及校验位中 ‘1’ 的个数为偶数。例如: 数据 =00110101, 有 4 个 ‘1’, 如果选择偶校验 (在 UART\_CR1 中的 PS=0), 校验位将是 ‘0’。

**奇校验:** 此校验位使得一帧中的 7 或 8 个 LSB 数据以及校验位中 ‘1’ 的个数为奇数。例如: 数据 =00110101, 有 4 个 ‘1’, 如果选择奇校验 (在 UART\_CR1 中的 PS=1), 校验位将是 ‘1’。

**传输模式:** 如果 UART\_CR1 的 PCE 位被置位, 写进数据寄存器的数据的 MSB 位被校验位替换后发送出去 (如果选择偶校验偶数个 ‘1’, 如果选择奇校验奇数个 ‘1’)。如果奇偶校验失败, UART\_SR 寄存器中的 PE 标志被置 ‘1’, 并且如果 UART\_CR1 寄存器的 PEIE 在被预先设置的话, 中断产生。

### 35.3.7 LIN (局域互联网) 模式

LIN 模式是通过设置 UART\_CR2 寄存器的 LINEN 位选择。在 LIN 模式下, 下列位必须保持为 0:

- UART\_CR3 寄存器的 STOP[1:0]和 HDSEL

#### LIN 发送

同样步骤适用于 LIN 主发送, 但和正常 UART 发送有以下区别:

- 清零 M 位以配置 8 位字长
- 置位 LINEN 位以进入 LIN 模式。这时, 置位 SBK 将发送 13 位 ‘0’ 作为断开符号。然后发一位 ‘1’, 以允许对下一个开始位的检测。

#### LIN 接收

当 LIN 模式被使能时, 断开符号检测电路被激活。该检测完全独立于 UART 接收器。断开符号只要一出现就能检测到, 不管是在总线空闲时还是在发送某数据帧其间, 数据帧还未完成, 又插入了断开符号的发送。

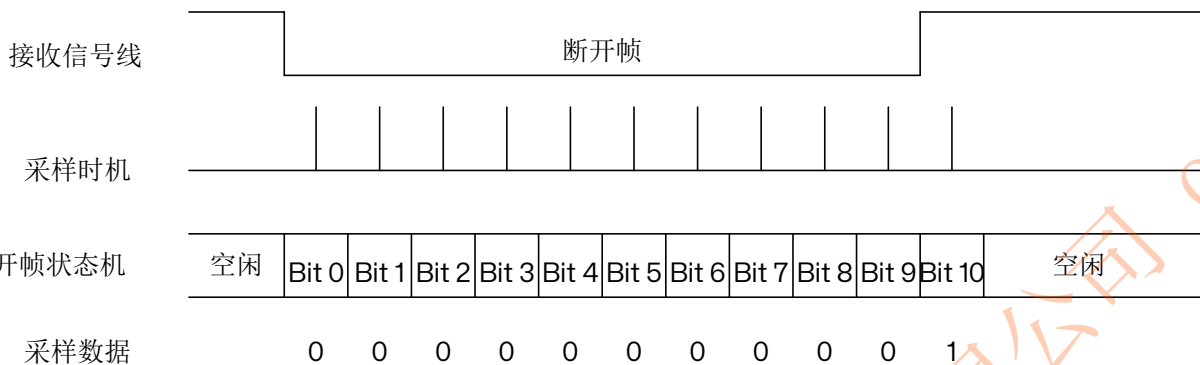
当接收器被激活时 (UART\_CR1 的 RE=1), 电路监测 RX 上的起始信号。监测起始位的方法同检测断开符号或数据是一样的。当起始位被检测到后, 电路对每个接下来的位, 在每个位的第 8, 9, 10 个过采样时钟点上进行采样。如果 10 个 (当 UART\_CR2 的 LBDL = 0) 或 11 个 (当 UART\_CR2 的 LBDL = 1) 连续位都是 ‘0’, 并且又跟着一个定界符, UART\_SR 的 LBD 标志被设置。如果 LBDIE 位=1, 中断产生。在确认断开符号前, 要检查定界符, 因为它意味 RX 线已经回到高电平。

如果在第 10 或 11 个采样点之前采样到了 ‘1’, 检测电路取消当前检测并重新寻找起始位。如果 LIN 模式被禁止, 接收器继续如正常 UART 那样工作, 不需要考虑检测断开符号。

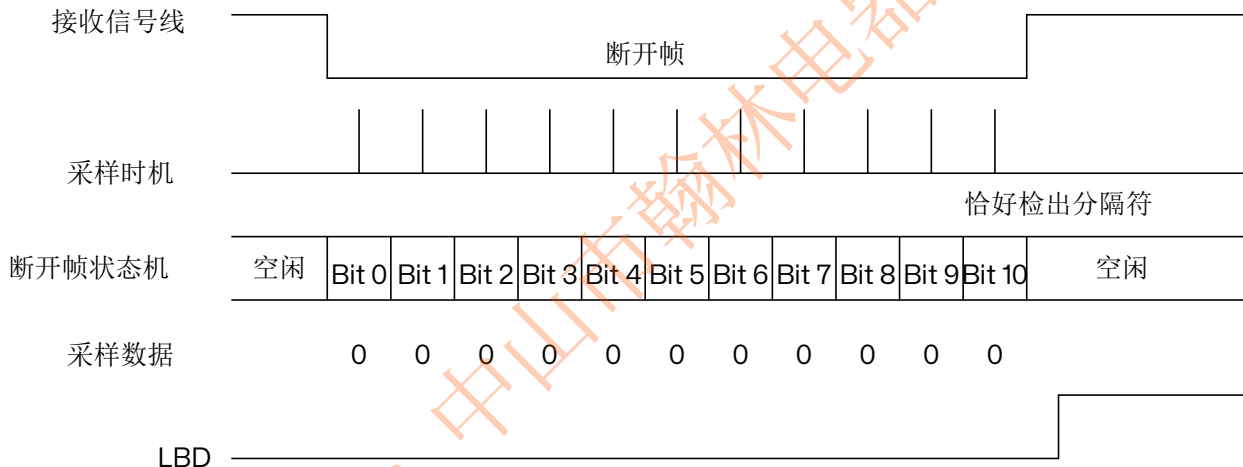
如果 LIN 模式没有被激活 (LINEN=0), 接收器仍然正常工作于 UART 模式, 不会进行断开检测。

如果 LIN 模式被激活 (LINEN=1), 只要一发生帧错误 (也就是停止位检测到 ‘0’, 这种情况出现在断开帧), 接收器就停止, 直到断开符号检测电路接收到一个 ‘1’ (这种情况发生于断开符号没有完整的发出来), 或一个定界符 (这种情况发生于已经检测到一个完整的断开符号)。

情形1：断开信号不够长 => 丢弃这个帧，不设置LBD



情形2：断开信号刚好够长 => 检测到断开，设置LBD



情形3：断开信号足够长 => 检测到断开，设置LBD

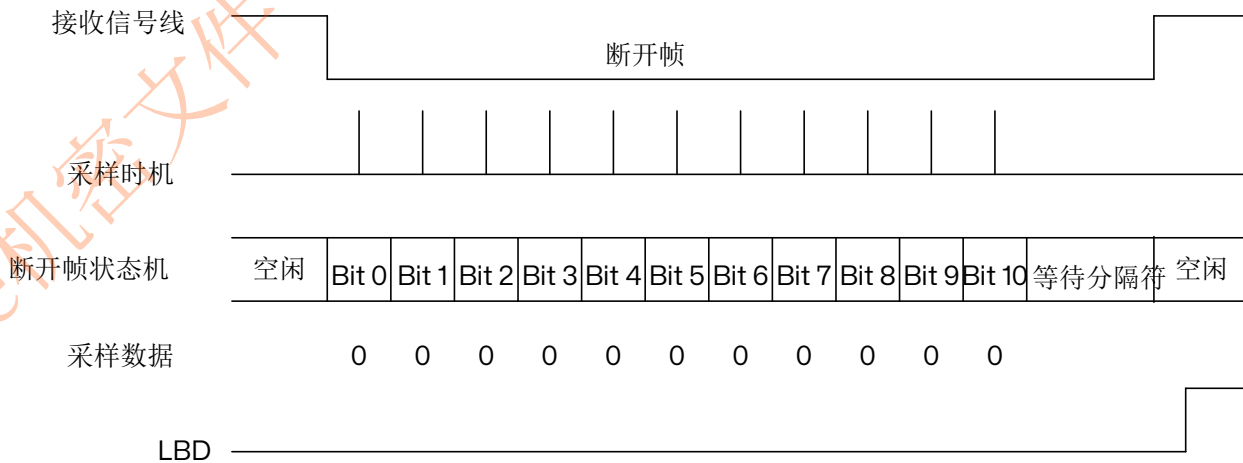
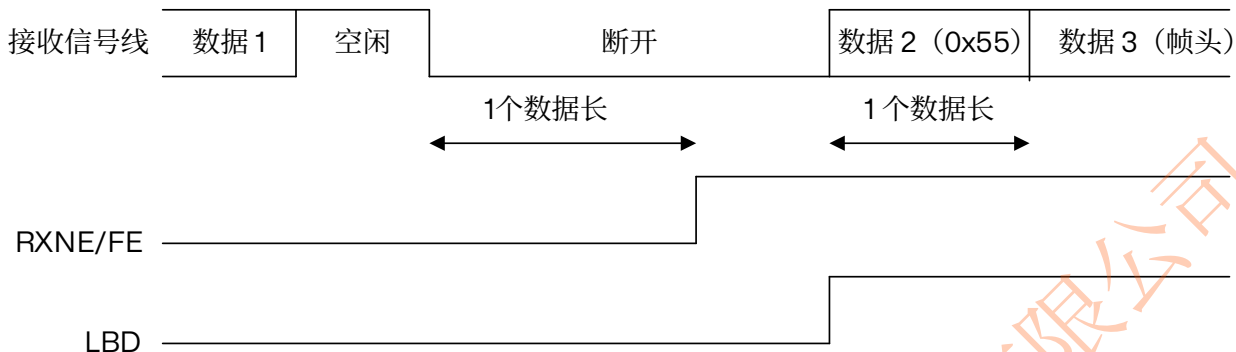


图 35.7 说明了断开符号检测器状态机的行为和断开符号标志的关系。

下面的例子中，假定 LBDL=1（断开帧长度为11位），M=0（8位数据）

#### 情形 1：断开发生在空闲之后



#### 情形 2：断开发生在正在接收数据时

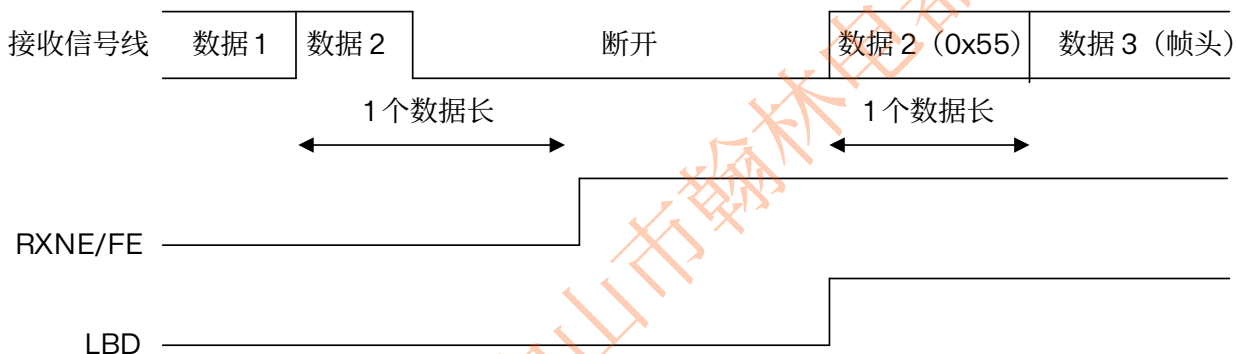


图 35.8 给出了一个断开帧的例子。

### 35.3.8 单线半双工通信

单线半双工模式通过设置 UART\_CR3 寄存器的 HDSEL 位选择。在这个模式里，下面的位必须保持清零状态：

- UART\_CR2 寄存器的 LINEN 位

UART 可以配置成遵循单线半双工协议。在单线半双工模式下，TX 和 RX 引脚在芯片内部互连。使用控制位“HALF DUPLEX SEL”（UART\_CR3 中的 HDSEL 位）选择半双工和全双工通信。当 HDSEL 为 ‘1’ 时

- RX 不再被使用
- 当没有数据传输时，TX 总是被释放。因此，它在空闲状态的或接收状态时表现为一个标准 I/O 口。这就意味该 I/O 在不被 UART 驱动时，必须配置成悬空输入（或开漏的输出高）。

除此以外，通信与正常 UART 模式类似。由软件来管理线上的冲突（例如通过使用一个中央仲裁器）。特别的是，发送从不会被硬件所阻碍。当 TE 位被设置时，只要数据一写到数据寄存器上，发送就继续。

## 35.4 UART 中断请求

表 35.6 UART 中断请求

| 中断事件                  | 事件标志    | 使能位    |
|-----------------------|---------|--------|
| 发送数据寄存器空              | TXE     | TXEIE  |
| 发送完成                  | TC      | TCIE   |
| 接收数据就绪可读              | TXNE    | TXNEIE |
| 检测到数据溢出               | ORE     |        |
| 检测到空闲线路               | IDRE    | IDLEIE |
| 奇偶检验错                 | PE      | PEIE   |
| 断开标志                  | LBD     | LBDIE  |
| 噪声标志, 多缓冲通信中的溢出错误和帧错误 | NE 或 FE | EIE    |

UART 的各种中断事件被连接到同一个中断向量（见下图），有以下各种中断事件：

- 发送期间：发送完成、清除发送。
- 接收期间：空闲总线检测、溢出错误、接收数据寄存器非空、校验错误、LIN 断开符号检测、噪声标志（仅在多缓冲器通信）和帧错误（仅在多缓冲器通信）。

如果设置了对应的使能控制位，这些事件就可以产生各自的中断。

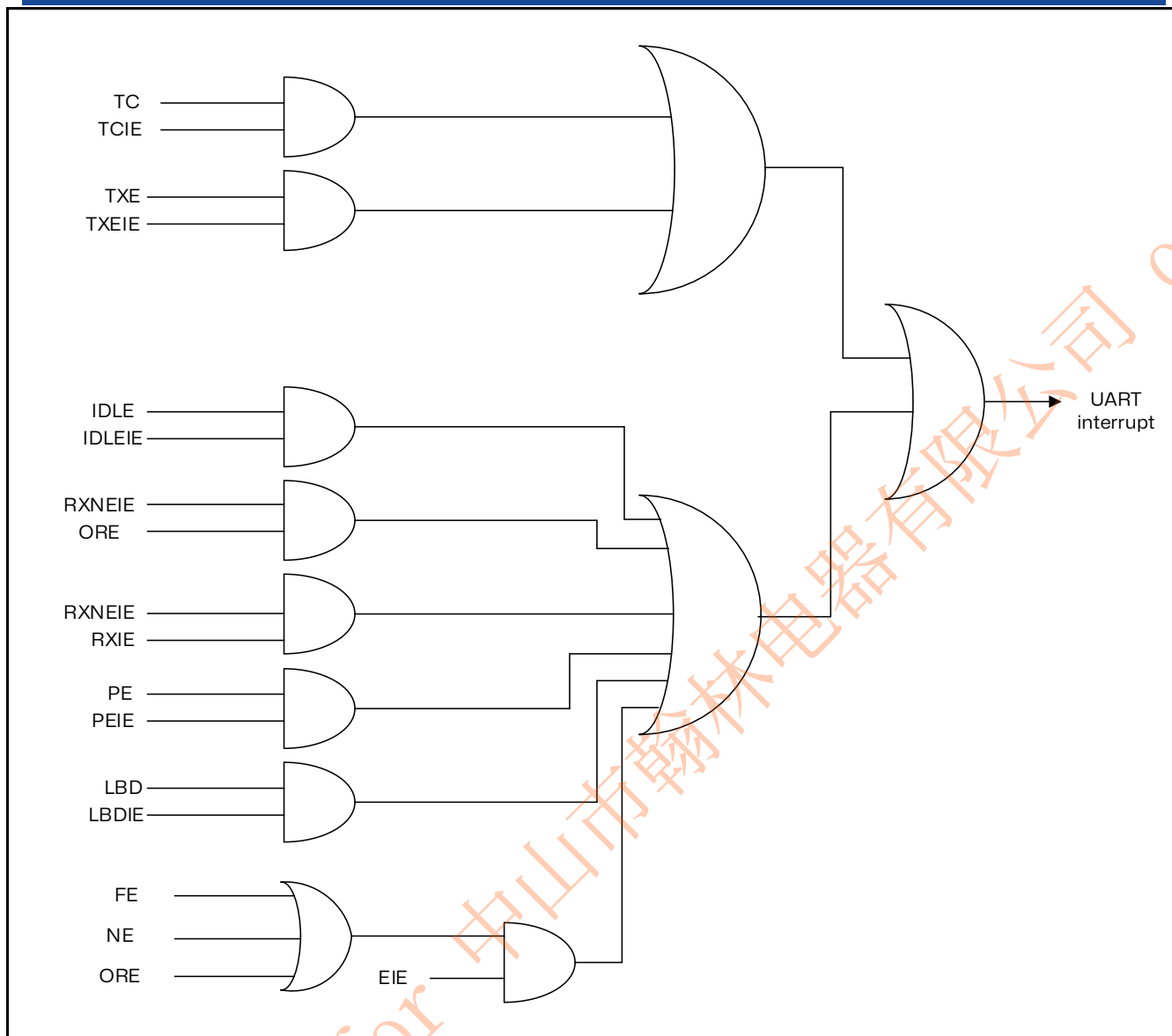


图 35.9 UART 中断映像图

### 35.5 UART 模式配置

表 35.7 UART 模式设置 <sup>(1)</sup>

| UART 模式     | UART1 | UART2 |
|-------------|-------|-------|
| 异步模式        | √     | √     |
| 硬件流控制       | ×     | ×     |
| 多缓存通讯 (DMA) | √     | √     |
| 多处理器通讯      | √     | √     |
| 同步          | ×     | ×     |
| 智能卡         | ×     | ×     |
| 半双工 (单线模式)  | √     | √     |
| IrDA        | ×     | ×     |



| UART 模式 | UART1 | UART2 |
|---------|-------|-------|
| LIN     | √     | √     |

(1) √ = 支持, × = 不支持该应用

## 35.6 UART 寄存器描述

可以用半字（16 位）或字（32 位）的方式操作这些外设寄存器。

### 35.6.1 状态寄存器（UART\_SR）

地址偏移：0x00

复位值：0x0000 00C0

|          |    |    |    |    |    |       |       |     |       |       |      |     |    |    |    |
|----------|----|----|----|----|----|-------|-------|-----|-------|-------|------|-----|----|----|----|
| 31       | 31 | 30 | 29 | 28 | 27 | 26    | 25    | 24  | 23    | 21    | 20   | 19  | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |       |       |     |       |       |      |     |    |    |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9     | 8     | 7   | 6     | 5     | 4    | 3   | 2  | 1  | 0  |
| Reserved |    |    |    |    |    | CTS   | LBD   | TXE | TC    | RXNE  | IDLE | ORE | NE | FE | PE |
|          |    |    |    |    |    | rc_w0 | rc_w0 | r   | rc_w0 | rc_w0 | r    | r   | r  | r  | r  |

Bits 31:10 保留，必须保持为复位值。

Bit 9 **CTS**: CTS 标志（CTS flag）

如果设置了 CTSE 位，当 nCTS 输入变化状态时，该位被硬件置高。由软件将其清零。

如果 UART\_CR3 中的 CTSIE 为 ‘1’，则产生中断。

0: nCTS 状态线上没有变化；

1: nCTS 状态线上发生变化。

注：UART4 和 UART5 上不存在这一位。

Bit 8 **LBD**: LIN 断开检测标志（LIN break detection flag）

当检测到 LIN 断开时，该位由硬件置‘1’，由软件清 ‘0’（向该位写 0）。如果

UART\_CR3 中的 LBDIE = 1，则产生中断。

0: 没有检测到 LIN 断开；

1: 检测到 LIN 断开。

注意：若 LBDIE=1，当 LBD 为 ‘1’ 时要产生中断。

Bit 7 **TXE**: 发送数据寄存器空（Transmit data register empty）

当 TDR 寄存器中的数据被硬件转移到移位寄存器的时候，该位被硬件置位。如果

UART\_CR1 寄存器中的 TXEIE 为 1，则产生中断。对 UART\_DR 的写操作，将该位清零。

0: 数据还没有被转移到移位寄存器；

1: 数据已经被转移到移位寄存器。

注意：单缓冲器传输中使用该位。

Bit 6 **TC**: 发送完成（Transmission complete）

当包含有数据的一帧发送完成后，并且 TXE=1 时，由硬件将该位置 ‘1’。如果

UART\_CR1 中的 TCIE 为 ‘1’，则产生中断。由软件序列清除该位（先读 UART\_SR，然后写入 UART\_DR）。TC 位也可以通过写入 ‘0’ 来清除，只有在多缓存通讯中才推荐这种清除程序。

0: 发送还未完成；

1: 发送完成。

Bit 5 **RXNE**: 读数据寄存器非空（Read data register not empty）

当 RDR 移位寄存器中的数据被转移到 UART\_DR 寄存器中，该位被硬件置位。如果 UART\_CR1 寄存器中的 RXNEIE 为 1，则产生中断。对 UART\_DR 的读操作可以将该位清零。RXNE 位也可以通过写入 0 来清除，只有在多缓存通讯中才推荐这种清除程序。

0: 数据没有收到;

1: 收到数据，可以读出。

**Bit 4 IDLE:** 监测到总线空闲 (IDLE line detected)

当检测到总线空闲时，该位被硬件置位。如果 UART\_CR1 中的 IDLEIE 为 '1'，则产生中断。由软件序列清除该位 (先读 UART\_SR，然后读 UART\_DR)。

0: 没有检测到空闲总线;

1: 检测到空闲总线。

注意: IDLE 位不会再次被置高直到 RXNE 位被置起 (即又检测到一次空闲总线)

**Bit 3 ORE:** 过载错误 (Overrun error)

当 RXNE 仍然是 '1' 的时候，当前被接收在移位寄存器中的数据，需要传送至 RDR 寄存器时，硬件将该位置位。如果 UART\_CR1 中的 RXNEIE 为 '1' 的话，则产生中断。由软件序列将其清零 (先读 UART\_SR，然后读 UART\_CR)。

0: 没有过载错误;

1: 检测到过载错误。

注意: 该位被置位时，RDR 寄存器中的值不会丢失，但是移位寄存器中的数据会被覆盖。如果设置了 EIE 位，在多缓冲器通信模式下，ORE 标志置位会产生中断的。

**Bit 2 NE:** 噪声错误标志 (Noise error flag)

在接收到的帧检测到噪音时，由硬件对该位置位。由软件序列对其清除 (先读 UART\_SR，再读 UART\_DR)。

0: 没有检测到噪声;

1: 检测到噪声。

注意: 该位不会产生中断，因为它和 RXNE 一起出现，硬件会在设置 RXNE 标志时产生中断。在多缓冲区通信模式下，如果设置了 EIE 位，则设置 NE 标志时会产生中断。

**Bit 1 FE:** 帧错误 (Framing error)

当检测到同步错位，过多的噪声或者检测到断开符，该位被硬件置位。由软件序列将其清零 (先读 UART\_SR，再读 UART\_DR)。

0: 没有检测到帧错误;

1: 检测到帧错误或者 break 符。

注意: 该位不会产生中断，因为它和 RXNE 一起出现，硬件会在设置 RXNE 标志时产生中断。如果当前传输的数据既产生了帧错误，又产生了过载错误，硬件还是会继续该数据的传输，并且只设置 ORE 标志位。

在多缓冲区通信模式下，如果设置了 EIE 位，则设置 FE 标志时会产生中断。

**Bit 0 PE:** 校验错误 (Parity error)

在接收模式下，如果出现奇偶校验错误，硬件对该位置位。

由软件序列对其清零 (依次读 UART\_SR 和 UART\_DR)。在清除 PE 位前，软件必须等待 RXNE 标志位被置 '1'。如果 UART\_CR1 中的 PEIE 为 '1'，则产生中断。

0: 没有奇偶校验错误;

1: 奇偶校验错误。

### 35.6.2 数据寄存器 (UART\_DR)

地址偏移: 0x04

复位值: 0x0000

|          |    |    |    |    |    |    |         |    |    |    |    |    |    |    |    |
|----------|----|----|----|----|----|----|---------|----|----|----|----|----|----|----|----|
| 31       | 31 | 30 | 29 | 28 | 27 | 26 | 25      | 24 | 23 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved |    |    |    |    |    |    |         |    |    |    |    |    |    |    |    |
| 15       | 14 | 13 | 12 | 11 | 10 | 9  | 8       | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| Reserved |    |    |    |    |    |    | DR[8:0] |    |    |    |    |    |    |    |    |
|          |    |    |    |    |    |    | rw      | rw | rw | rw | rw | rw | rw | rw | rw |

Bit 31:9 保留, 必须保持为复位值。

Bit 8:0 **DR[8:0]**: 数据值 (Data value)

包含了发送或接收的数据。由于它是由两个寄存器组成的, 一个给发送用 (TDR), 一个给接收用 (RDR), 该寄存器兼具读和写的功能。TDR 寄存器提供了内部总线和输出移位寄存器之间的并行接口。RDR 寄存器提供了输入移位寄存器和内部总线之间的并行接口。

当使能校验位 (UART\_CR1 中 PCE 位被置位) 进行发送时, 写到 MSB 的值 (根据数据的长度不同, MSB 是第 7 位或者第 8 位) 会被后来的校验位取代。

当使能校验位进行接收时, 读到的 MSB 位是接收到的校验位。

### 35.6.3 波特比率寄存器 (UART\_BRR)

注意: 如果 TE 或 RE 被分别禁止, 波特计数器停止计数

地址偏移: 0x08

复位值: 0x0000 0000

|                    |    |    |    |    |    |    |    |    |    |    |    |                   |    |    |    |
|--------------------|----|----|----|----|----|----|----|----|----|----|----|-------------------|----|----|----|
| 31                 | 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 21 | 20 | 19                | 18 | 17 | 16 |
| Reserved           |    |    |    |    |    |    |    |    |    |    |    |                   |    |    |    |
| 15                 | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3                 | 2  | 1  | 0  |
| DIV_Mantissa[11:0] |    |    |    |    |    |    |    |    |    |    |    | DIV_Fraction[3:0] |    |    |    |
| rw                 | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw                | rw | rw | rw |

Bit 31:16 保留, 必须保持为复位值。

Bit 15:4 **DIV\_Mantissa[11:0]**: UARTDIV 的整数部分

这 12 位定义了 UART 分频器除法系数 (UARTDIV) 的整数部分。

Bit 3:0 **DIV\_Fraction[3:0]**: UARTDIV 的小数部分

这 4 位定义了 UART 分频器除法系数 (UARTDIV) 的小数部分。

### 35.6.4 控制寄存器 1 (UART\_CR1)

地址偏移: 0x0C

复位值: 0x0000 0000

|          |    |    |    |      |     |    |      |       |      |        |        |    |    |     |     |
|----------|----|----|----|------|-----|----|------|-------|------|--------|--------|----|----|-----|-----|
| 31       | 31 | 30 | 29 | 28   | 27  | 26 | 25   | 24    | 23   | 21     | 20     | 19 | 18 | 17  | 16  |
| Reserved |    |    |    |      |     |    |      |       |      |        |        |    |    |     |     |
| 15       | 14 | 13 | 12 | 11   | 10  | 9  | 8    | 7     | 6    | 5      | 4      | 3  | 2  | 1   | 0   |
| Reserved | UE |    | M  | WAKE | PCE | PS | PEIE | TXEIE | TCIE | RXNEIE | IDLEIE | TE | RE | RWU | SBK |
|          | rw | rw | rw | rw   | rw  | rw | rw   | rw    | rw   | rw     | rw     | rw | rw | rw  | rw  |

Bit 31:14 保留, 必须保持为复位值。

Bit 13 **UE**: UART 使能 (UART enable)

当该位被清零，在当前字节传输完成后 UART 的分频器和输出停止工作，以减少功耗。  
该位由软件设置和清零。

0: UART 分频器和输出被禁止；

1: UART 模块使能。

**Bit 12 M:** 字长 (Word length)

该位定义了数据字的长度，由软件对其设置和清零

0: 一个起始位，8 个数据位，n 个停止位；

1: 一个起始位，9 个数据位，n 个停止位。

注意：在数据传输过程中（发送或者接收时），不能修改这个位。

**Bit 11 WAKE:** 唤醒的方法 (Wakeup method)

这位决定了把 UART 唤醒的方法，由软件对该位设置和清零。

0: 被空闲总线唤醒；

1: 被地址标记唤醒。

**Bit 10 PCE:** 检验控制使能 (Parity control enable)

用该位选择是否进行硬件校验控制（对于发送来说就是校验位的产生；对于接收来说就是校验位的检测）。当使能了该位，在发送数据的最高位（如果 M=1，最高位就是第 9 位；如果 M=0，最高位就是第 8 位）插入校验位；对接收到的数据检查其校验位。软件对它置 ‘1’ 或清 ‘0’。一旦设置了该位，当前字节传输完成后，校验控制才生效。

0: 禁止校验控制；

1: 使能校验控制。

**Bit 9 PS:** 校验选择 (Parity selection)

当校验控制使能后，该位用来选择是采用偶校验还是奇校验。软件对它置 ‘1’ 或清 ‘0’。当前字节传输完成后，该选择生效。

0: 偶校验；

1: 奇校验。

**Bit 8 PEIE:** PE 中断使能 (PE interrupt enable)

该位由软件设置或清除。

0: 禁止产生中断；

1: 当 UART\_SR 中的 PE 为 ‘1’ 时，产生 UART 中断。

**Bit 7 TXEIE:** 发送缓冲区空中断使能 (TXE interrupt enable)

该位由软件设置或清除。

0: 禁止产生中断；

1: 当 UART\_SR 中的 TXE 为 ‘1’ 时，产生 UART 中断。

**Bit 6 TCIE:** 发送完成中断使能 (Transmission complete interrupt enable)

该位由软件设置或清除。

0: 禁止产生中断；

1: 当 UART\_SR 中的 TC 为 ‘1’ 时，产生 UART 中断。

**Bit 5 RXNEIE:** 接收缓冲区非空中断使能 (RXNE interrupt enable)

该位由软件设置或清除。

0: 禁止产生中断；

1: 当 UART\_SR 中的 ORE 或者 RXNE 为 ‘1’ 时，产生 UART 中断。

**Bit 4 IDLEIE:** IDLE 中断使能 (IDLE interrupt enable)

该位由软件设置或清除。

0: 禁止产生中断;

1: 当 UART\_SR 中的 IDLE 为 ‘1’ 时, 产生 UART 中断。

**Bit 3 TE:** 发送使能 (Transmitter enable)

该位使能发送器。该位由软件设置或清除。

0: 禁止发送;

1: 使能发送。

注意: 1.在数据传输过程中, 除了在智能卡模式下, 如果 TE 位上有个 0 脉冲 (即设置为 ‘0’ 之后再设置为 ‘1’), 会在当前数据字传输完成后, 发送一个“前导符” (空闲总线)。

2.当 TE 被设置后, 在真正发送开始之前, 有一个比特时间的延迟。

**Bit 2 RE:** 接收使能 (Receiver enable)

该位由软件设置或清除。

0: 禁止接收;

1: 使能接收, 并开始搜寻 RX 引脚上的起始位。

**Bit 1 RWU:** 接收唤醒 (Receiver wakeup)

该位用来决定是否把 UART 置于静默模式。该位由软件设置或清除。当唤醒序列到来时, 硬件也会将其清零。

0: 接收器处于正常工作模式;

1: 接收器处于静默模式。

注意: 1.在把 UART 置于静默模式 (设置 RWU 位) 之前, UART 要已经先接收了一个数据字节。否则在静默模式下, 不能被空闲总线检测唤醒。

2.当配置成地址标记检测唤醒 (WAKE 位=1), 在 RXNE 位被置位时, 不能用软件修改 RWU 位。

**Bit 0 SBK:** 发送断开帧 (Send break)

使用该位来发送断开字符。该位可以由软件设置或清除。操作过程应该是软件设置位它, 然后在断开帧的停止位时, 由硬件将该位复位。

0: 没有发送断开字符;

1: 将要发送断开字符。

### 35.6.5 控制寄存器 2 (UART\_CR2)

地址偏移: 0x10

复位值: 0x0000 0000

|          |       |           |    |          |    |    |    |    |       |      |     |          |    |    |    |
|----------|-------|-----------|----|----------|----|----|----|----|-------|------|-----|----------|----|----|----|
| 31       | 31    | 30        | 29 | 28       | 27 | 26 | 25 | 24 | 23    | 21   | 20  | 19       | 18 | 17 | 16 |
| Reserved |       |           |    |          |    |    |    |    |       |      |     |          |    |    |    |
| 15       | 14    | 13        | 12 | 11       | 10 | 9  | 8  | 7  | 6     | 5    | 4   | 3        | 2  | 1  | 0  |
| Res      | LINEN | STOP[1:0] |    | Reserved |    |    |    |    | LBDIE | LBDL | Res | ADD[3:0] |    |    |    |
|          | rw    | rw        | rw |          |    |    |    |    | rw    | rw   |     | rw       | rw | rw | rw |

Bits 31:15 保留, 必须保持为复位值。

**Bit 14 LINEN:** LIN 模式使能 (LIN mode enable)

该位由软件设置或清除。

0: 禁止 LIN 模式;

1: 使能 LIN 模式。

Bits 13:12 **STOP:** 停止位 (STOP bits)

这 2 位用来设置停止位的位数

00: 1 个停止位;

10: 2 个停止位;

其他: reserved

Bit 11:7 保留, 必须保持为复位值。

Bit 6 **LBDIE**: LIN 断开符检测中断使能 (LIN break detection interrupt enable)

断开符中断屏蔽 (使用断开分隔符来检测断开符)

0: 禁止中断;

1: 只要 UART\_SR 寄存器中的 LBD 为 '1' 就产生中断。

Bit 5 **LBDL**: LIN 断开符检测长度 (LIN break detection length)

该位用来选择是 11 位还是 10 位的断开符检测

0: 10 位的断开符检测;

1: 11 位的断开符检测。

Bit 4 保留, 必须保持为复位值。

Bits 3:0 **ADD[3:0]**: 本设备的 UART 节点地址

该位域给出本设备 UART 节点的地址。

这是在多处理器通信下的静默模式中使用的, 使用地址标记来唤醒某个 UART 设备。

## 35.6.6 控制寄存器 3 (UART\_CR3)

地址偏移: 0x14

复位值: 0x0000 0000

|          |    |    |    |    |       |      |      |      |      |          |    |    |       |          |    |     |
|----------|----|----|----|----|-------|------|------|------|------|----------|----|----|-------|----------|----|-----|
| 31       | 31 | 30 | 29 | 28 | 27    | 26   | 25   | 24   | 23   | 21       | 20 | 19 | 18    | 17       | 16 |     |
| Reserved |    |    |    |    |       |      |      |      |      |          |    |    |       |          |    |     |
| 15       | 14 | 13 | 12 | 11 | 10    | 9    | 8    | 7    | 6    | 5        | 4  | 3  | 2     | 1        | 0  |     |
| Reserved |    |    |    |    | CTSIE | CTSE | RTSE | DMAT | DMAR | Reserved |    |    | HDSEL | Reserved |    | EIE |
|          |    |    |    |    | rw    | rw   | rw   | rw   | rw   |          |    |    | rw    |          |    |     |

Bits 31:11 保留, 必须保持为复位值。

Bit 10 **CTSIE**: CTS 中断使能 (CTS interrupt enable)

0: 禁止中断;

1: UART\_SR 寄存器中的 CTS 为 '1' 时产生中断。

Bit 9 **CTSE**: CTS 使能 (CTS enable)

0: 禁止 CTS 硬件流控制;

1: CTS 模式使能, 只有 nCTS 输入信号有效 (拉成低电平) 时才能发送数据。如果在数据传输的过程中, nCTS 信号变成无效, 那么发完这个数据后, 传输就停止下来。如果当 nCTS 为无效时, 往数据寄存器里写数据, 则要等到 nCTS 有效时才会发送这个数据。

Bit 8 **RTSE**: RTS 使能 (RTS enable)

0: 禁止 RTS 硬件流控制;

1: RTS 中断使能, 只有接收缓冲区内有空余的空间时才请求下一个数据。当前数据发送完成后, 发送操作就需要暂停下来。如果可以接收数据了, 将 nRTS 输出置为有效 (拉至低电平)。

Bit 7 **DMAT**: DMA 使能发送 (DMA enable transmitter)

该位由软件设置或清除。



0: 禁止发送时的 DMA 模式。

1: 使能发送时的 DMA 模式；

Bit 6 **DMAR**: DMA 使能接收 (DMA enable receiver)

该位由软件设置或清除。

0: 禁止接收时的 DMA 模式。

1: 使能接收时的 DMA 模式；

Bit 5:4 保留，必须保持为复位值。

Bit 3 **HDSEL**: 半双工选择 (Half-duplex selection)

选择单线半双工模式

0: 不选择半双工模式；

1: 选择半双工模式。

Bit 2:1 保留，必须保持为复位值。

Bit 0 **EIE**: 错误中断使能 (Error interrupt enable)

0: 禁止中断；

1: 只要 UART\_CR3 中的 DMAR=1, 并且 UART\_SR 中的 FE=1, 或者 ORE=1, 或者 NE=1, 则产生中断。



## 36 版本历史

表 36.1 版本历史

| 日期         | 版本   | 更改内容                                                                                                     |
|------------|------|----------------------------------------------------------------------------------------------------------|
| 2024/04/02 | V0.1 | 草稿初版                                                                                                     |
| 2024/05/06 | V0.2 | 审稿修改                                                                                                     |
| 2024/05/31 | V0.3 | 1. 第 5 章节 PWR<br>2. 部分格式细节修缮<br>3. 文档约定寄存器缩写更新<br>4. 第 22 章节 TIM1/8 的 DMA 功能补充, 寄存器核对<br>5. 第 19 章节 COMP |
| 2024/08/05 | V0.4 | 1. 检查修正寄存器和描述<br>2. HRTIM 寄存器更新                                                                          |
| 2024/08/23 | V0.5 | 1. 增加第 5 章节 CRC<br>2. 修正第 6 章节 PWR、第 20 章节 COMP 描述<br>3. 修正部分 HRTIM 描述<br>4. 修正 DAC 寄存器                  |
| 2024/08/27 | V0.6 | 1. 修正部分 HRTIM 描述<br>2. 修正 OPAMP 章节输入信号名称<br>3. 修正了 ADC 引脚/通道<br>4. 修正了 DAC 引脚通道名称                        |
| 2024/09/04 | V0.7 | 1. 修正 HRTIM 章节全部描述<br>2. 修正部分 DAC 描述<br>3. 修正 TIM 章节部分细节<br>4. 修正 ADC 通道描述                               |